The LaTeX 2$_\varepsilon$ Sources

Johannes Braams
David Carlisle
Alan Jeffrey
Leslie Lamport
Frank Mittelbach
Chris Rowley
Rainer Schöpf

2026-11-01

This file is maintained by the LaTeX Project team.
Bug reports can be opened (category `latex`) at
https://latex-project.org/bugs.html.

Contents

01 ltdirchk.dtx **1**

1 LaTeX System Dependent Initializations **1**

2 Initialization **2**
2.1 INITEX . . . 3
2.2 Some bits of 2e . . . 4

3 texsys.cfg **5**
3.1 texsys.cfg . . . 5
3.2 UNIX (web2c) . . . 6
3.3 UNIX (other) . . . 7
3.4 MSDOS (emtex) . . . 7
3.5 MSDOS (other) . . . 7
3.6 VMS (DECUS TeX, PD VMS 3.6) . . . 7
3.7 VMS (???) . . . 7
3.8 MACINTOSH (OzTeX 1.6) . . . 8
3.9 MACINTOSH (other) . . . 8
3.10 FAKE EXAMPLE . . . 8

4 Setting `\@currdir` **9**

5 Setting `\input@path` **10**

6 Filename Parsing **11**

7 TEX Versions **13**

8 ltxcheck.tex **13**

02 ltplain.dtx 14

1 Plain TEX **14**

03 ltvers.dtx 35

1 Version Identification **35**
1.1 Declaring an all-new module 38

04 ltluatex.dtx 40

1 Overview **40**

2 Core TEX functionality **40**

3 Plain TEX interface **41**

4 Lua functionality **41**
4.1 Allocators in Lua 41
4.2 Lua access to TEX register numbers 42
4.3 Module utilities 43
4.4 Callback management 44

5 Implementation **45**
5.1 Minimum LuaTEX version 45
5.2 Older LATEX/Plain TEX setup 45
5.2.1 Fixes to `etex.src/etex.sty` 45
5.2.2 luatex specific settings 46
5.3 Attributes 47
5.4 Category code tables 47
5.5 Named Lua functions 49
5.6 Custom whatsits 50
5.7 Lua bytecode registers 50
5.8 Lua chunk registers 50
5.9 Lua loader 51
5.10 Lua module preliminaries 52
5.11 Lua module utilities 53
5.11.1 Module tracking 53
5.11.2 Module messages 53
5.12 Accessing register numbers from Lua 54
5.13 Attribute allocation 55
5.14 Custom whatsit allocation 56

5.15 Bytecode register allocation . . . 56
5.16 Lua chunk name allocation . . . 57
5.17 Lua function allocation . . . 57
5.18 Lua callback management . . . 58
5.18.1 Housekeeping . . . 58
5.18.2 Handlers . . . 63
5.18.3 Public functions for callback management . . . 65

05 ltexpl.dtx 72

1 expl3-dependent code 72
1.1 Loader . . . 72
1.2 Using expl3 code . . . 75

2 Document-level command names for expl3 functions 76

06 ltdefns.dtx 80

1 Definitions 80
1.1 Initex initializations . . . 80
1.2 Saved versions of TEX primitives . . . 80
1.3 Command definitions . . . 81
1.4 Robust commands and protect . . . 91
1.5 Acting on robust commands . . . 98
1.5.1 Copying robust commands . . . 100
1.5.2 Showing robust commands . . . 103
1.5.3 Commands defined with `\DeclareRobustCommand` . . . 103
1.5.4 Commands defined with `\newcommand` (with optional argument) 105
1.5.5 Showing environments . . . 107
1.6 Internal defining commands . . . 108

2 Discretionary Hyphenation 112

07 ltcmd.dtx 115

1 Creating document commands 115
1.1 Overview . . . 115
1.2 Describing argument types . . . 115
1.3 Modifying argument descriptions . . . 116
1.4 Creating document commands and environments . . . 117
1.5 Optional arguments . . . 118
1.6 Spacing and optional arguments . . . 118
1.7 'Embellishments' . . . 119
1.8 Testing special values . . . 119
1.9 Auto-converting to key–value format . . . 121
1.10 Argument processors . . . 122
1.11 Body of an environment . . . 124
1.12 Fully-expandable document commands . . . 125

1.13 Commands at the start of tabular cells . . . 126
1.14 Using the verbatim argument types . . . 126
1.15 Typesetting verbatim-like material . . . 127
1.16 Verbatim environments . . . 127
1.17 Performance . . . 128
1.18 Details about argument delimiters . . . 128
1.18.1 Character tokens . . . 128
1.18.2 Control sequence tokens . . . 128
1.19 Creating new argument processors . . . 129
1.20 Variables and constants . . . 130
1.21 Declaring commands and environments . . . 133
1.22 Structure of xparse commands . . . 139
1.23 Normalizing the argument specifications . . . 143
1.24 Preparing the signature: general mechanism . . . 152
1.25 Setting up a standard signature . . . 153
1.26 Setting up expandable types . . . 159
1.26.1 Copying a command and its internal structure . . . 162
1.26.2 Showing the definition of a command . . . 168
1.27 Grabbing arguments . . . 173
1.28 Grabbing arguments expandably . . . 191
1.29 Argument processors . . . 196
1.30 Conversion to key–value form . . . 198
1.31 Utilities . . . 201
1.32 Access to the argument specification . . . 205
1.33 Messages . . . 205
1.34 User functions . . . 211

08 lthooks.dtx 217

1 Introduction 217

2 Package writer interface 217
2.1 LaTeX 2ε interfaces . . . 217
2.1.1 Declaring hooks . . . 217
2.1.2 Special declarations for generic hooks . . . 218
2.1.3 Using hooks in code . . . 219
2.1.4 Updating code for hooks . . . 220
2.1.5 Hook names and default labels . . . 223
2.1.6 The top-level label . . . 225
2.1.7 Defining relations between hook code . . . 225
2.1.8 Querying hooks . . . 227
2.1.9 Displaying hook code . . . 228
2.1.10 Debugging hook code . . . 229
2.2 L3 programming layer (expl3) interfaces . . . 229
2.3 On the order of hook code execution . . . 232
2.4 The use of “reversed” hooks . . . 234
2.5 Difference between “normal” and “one-time” hooks . . . 235
2.6 Generic hooks provided by packages . . . 235
2.7 Hooks with arguments . . . 236

2.8 Private LaTeX kernel hooks . . . 238
2.9 Legacy LaTeX 2ε interfaces . . . 238

3 LaTeX 2ε commands and environments augmented by hooks 239
3.1 Generic hooks . . . 239
3.1.1 Generic hooks for all environments . . . 240
3.1.2 Generic hooks for commands . . . 241
3.1.3 Generic hooks provided by file loading operations . . . 241
3.2 Hooks provided by `\begin{document}` . . . 242
3.3 Hooks provided by `\end{document}` . . . 242
3.4 Hooks provided by `\shipout` operations . . . 244
3.5 Hooks provided for paragraphs . . . 244
3.6 Hooks provided in NFSS commands . . . 244
3.7 Hook provided by the mark mechanism . . . 245

4 The Implementation 245
4.1 Debugging . . . 245
4.2 Borrowing from internals of other kernel modules . . . 246
4.3 Declarations . . . 246
4.4 Providing new hooks . . . 248
4.4.1 The data structures of a hook . . . 248
4.4.2 On the existence of hooks . . . 249
4.4.3 Setting hooks up . . . 250
4.4.4 Disabling and providing hooks . . . 256
4.5 Parsing a label . . . 258
4.6 Adding or removing hook code . . . 262
4.7 Setting rules for hooks code . . . 283
4.8 Specifying code for next invocation . . . 305
4.9 Using the hook . . . 307
4.10 Querying a hook . . . 313
4.11 Messages . . . 317
4.12 LaTeX 2ε package interface commands . . . 320
4.13 Deprecated that needs cleanup at some point . . . 323
4.14 Internal commands needed elsewhere . . . 325

09 ltcmdhooks.dtx 327

1 Introduction 327

2 Restrictions and operational details 328
2.1 Patching . . . 329
2.1.1 Timing . . . 329
2.2 Command copies . . . 329
2.3 Grouping . . . 330
2.4 Commands that look ahead . . . 330

3 Package author interface 330
3.1 Arguments and redefining commands . . . 331

4 The Implementation **332**
4.1 Execution plan . . . 332
4.2 Variables . . . 333
4.3 Patching or delaying . . . 334
4.4 Patching commands . . . 336
4.4.1 Patching by expansion and redefinition . . . 337
4.4.2 Patching by retokenization . . . 345
4.5 Messages . . . 351

10 ltsockets.dtx **353**

1 Introduction **353**

2 Configuration of the transformation process **353**
2.1 The template mechanism . . . 353
2.2 The hook mechanism . . . 354
2.3 The socket mechanism . . . 354
2.3.1 Examples . . . 356
2.3.2 Details and semantics . . . 357
2.4 Socket and plug names . . . 359
2.4.1 Command syntax . . . 359
2.4.2 Rationale for error handling . . . 361

3 The Implementation **361**
3.1 Debugging the socket structures . . . 361
3.2 The L3 layer commands . . . 362
3.3 Error messages . . . 366
3.4 The LaTeX 2_ε interface commands . . . 367

11 lttemplates.dtx **369**

1 Introduction **369**

2 What is a document? **369**

3 Types, templates, and instances **370**

4 Template types **370**

5 Templates **370**

6 Multiple choices **374**

7 Instances **375**

8 Document interface **376**

9 Changing existing definitions **377**
9.1 Expanding the values of keys . . . 377

10 Creating equivalent keys **378**
10.1 Modifying existing key values . . . 378

11 Getting information about templates and instances **379**

12 Debugging support **379**

13 The implementation **379**
13.1 Extensions for expl3 . . . 380
13.2 Variables and constants . . . 381
13.3 Debugging support . . . 383
13.4 Testing existence and validity . . . 384
13.5 Saving and recovering property lists . . . 386
13.6 Creating new template types . . . 388
13.7 Design part of template declaration . . . 388
13.8 Equivalent interfaces . . . 392
13.9 Implementation part of template declaration . . . 394
13.10 Editing template defaults . . . 399
13.11 Creating instances of templates . . . 403
13.12 Using templates directly . . . 406
13.13 Assigning values to variables . . . 407
13.14 Using instances . . . 410
13.15 Assignment manipulation . . . 411
13.16 Ad hoc key setting . . . 411
13.17 Showing templates and instances . . . 412
13.18 Messages . . . 413
13.19 User functions . . . 417

12 ltalloc.dtx **421**

1 Counters **421**

13 ltcntrl.dtx **423**

1 Program control structure **423**

14 lterror.dtx **427**

1 Error handling and tracing **427**
1.1 General commands . . . 427
1.2 Specific errors . . . 433
1.3 Tracing . . . 437

15 ltpar.dtx **438**

1 Paragraphs **438**
1.1 Implementation . . . 438

16 ltpara.dtx 440

1 Introduction 440
1.1 The default processing done by the engine . . . 440

2 The new mechanism implemented for LaTeX 442
2.1 The provided hooks . . . 443
2.2 Altered and newly provided commands . . . 444
2.3 Examples . . . 445
2.3.1 Testing the mechanism . . . 445
2.3.2 Mark the first paragraph of each `itemize` . . . 447
2.4 Some technical notes . . . 447
2.4.1 Glue items between paragraphs (found with fancypar) . . . 447

3 The Implementation 448
3.1 Providing hooks for paragraphs . . . 448
3.2 The error messages . . . 456

17 ltmeta.dtx 457

1 Introduction 457
1.1 `\DocumentMetadata` . . . 457
1.2 Storing and retrieving document properties . . . 457

2 The Implementation 458
2.1 `\DocumentMetadata` . . . 458
2.2 Document properties . . . 459
2.3 Targets . . . 460

18 ltspace.dtx 463

1 Spacing 463
1.1 User Commands . . . 463
1.2 Chris' comments . . . 463
1.3 Some immediate actions . . . 465
1.4 The code . . . 466
1.5 Vertical spacing . . . 474
1.6 Horizontal space (and breaks) . . . 479

19 ltlogos.dtx 484

1 Logos 484

20 ltfiles.dtx 485

1 File Handling **485**
1.1 Safe Input Macros . . . 499
1.2 Listing files . . . 506

21 ltoutenc.dtx 509

1 Font encodings **509**
1.1 Removing encoding-specific commands . . . 511
1.2 The order of declarations . . . 512
1.3 Docstrip modules . . . 512
1.4 Definitions for the kernel . . . 513
1.4.1 Declaration commands . . . 513
1.4.2 Hyphenation . . . 521
1.4.3 Miscellania . . . 522
1.4.4 Default encodings . . . 522
1.4.5 Math material . . . 524
1.5 Definitions for the OT1 encoding . . . 525
1.6 Definitions for the T1 encoding . . . 528
1.7 Definitions for the OMS encoding . . . 534
1.8 Definitions for the OML encoding . . . 534
1.9 Definitions for the OT4 encoding . . . 534
1.10 Definitions for the TS1 encoding . . . 537
1.11 Definitions for the TU encoding . . . 541

2 Package files **553**
2.1 The fontenc package . . . 553

22 ltcounts.dtx 556

1 Counters and Lengths **556**
1.1 Document command and environment counter macros . . . 556

23 ltlength.dtx 569

1 Lengths **569**

24 ltfssbas.dtx 571

1 Preliminary macros **571**

2 Macros for setting up the tables **572**

3 Selecting a new font **581**
3.1 Macros for the user . . . 581
3.2 Macros for loading fonts . . . 587

4 Assigning math fonts to *versions* **594**

25 ltfssaxes.dtx 601

1 Changing the font series 601
1.1 The series lookup table . . . 601
1.2 Mapping rules for series changes . . . 602
1.3 Changing to a new series . . . 656

2 Changing the shape 660
2.1 Mapping rules for shape combinations . . . 662
2.2 Changing to a new shape . . . 666

3 Make sure we win . . . 668

26 ltfsstrc.dtx 671

1 Introduction 671

2 A driver for this document 671

3 The Implementation 672

4 Handling Options 672

5 Macros common to `fam.tex` and `tracefnt.sty` 674
5.1 General font loading . . . 674
5.2 Math fonts setup . . . 680
5.2.1 Outline of algorithm for math font sizes . . . 680
5.2.2 Code for math font size setting . . . 681
5.2.3 Other code for math . . . 682

6 Scaled font extraction 685
6.1 Sizefunctions . . . 693

27 ltfsscmp.dtx 697

28 ltfssdcl.dtx 702

1 Interface Commands 702

29 ltfssini.dtx 735

1 NFSS Initialization 735
1.1 Providing math *versions* . . . 735

2 Custom series settings for main document families 736

3 Supporting nested emphasis 755

4 Supporting \strongemph 758
4.1 Legacy . . . 759
4.2 Miscellaneous . . . 760

30 fontdef.dtx 766

1 Introduction 766

2 Customization 766

3 The docstrip modules 767

4 A driver for this document 767

5 The fonttext.ltx file 767
5.1 Encodings . . . 768
5.2 Defaults . . . 770

6 The fontmath.ltx file 772
6.1 The font encodings used . . . 772
6.1.1 Symbolfont and Alphabet declarations . . . 773
6.2 Math font sizes . . . 773
6.3 The math symbol assignments . . . 774
6.3.1 The letters . . . 774
6.3.2 The digits . . . 775
6.3.3 Punctuation, brace, etc. keys . . . 775
6.3.4 Delimitercodes for characters . . . 776
6.4 Symbols accessed via control sequences . . . 776
6.4.1 Greek letters . . . 776
6.4.2 Ordinary symbols . . . 777
6.4.3 Large Operators . . . 778
6.4.4 Binary symbols . . . 778
6.4.5 Relations . . . 779
6.4.6 Arrows . . . 781
6.4.7 Punctuation symbols . . . 781
6.4.8 Math accents . . . 782
6.4.9 Radicals . . . 782
6.4.10 Over and under something, etc . . . 782
6.4.11 Delimiters . . . 783
6.5 Math versions of text commands . . . 784
6.6 Other special functions and parameters . . . 785
6.6.1 Biggggg . . . 785
6.6.2 The log-like functions . . . 785
6.6.3 Parameters . . . 785

7 Default cfg files 785

31 preload.dtx 787

1 Overview 787

2 Customization 787

3 Module switches for the DOCSTRIP program 787

4 A driver for this document 788

5 The code 788

32 ltfntcmd.dtx 790

1 Introduction 790

2 The implementation 792

3 Initialization 798

33 lttextcomp.dtx 799

1 Sub-encodings 801
1.1 Unavailable in sub-encoding 1 and higher (drop symbols not working in Latin Modern) . . . 803
1.2 Unavailable in sub-encoding 2 (majority of new OTF fonts via autoinst) and higher . . . 803
1.3 Unavailable in sub-encoding 3 and higher . . . 806
1.4 Unavailable in sub-encoding 4 and higher . . . 806
1.5 Unavailable in sub-encoding 5 (most older PS fonts) and higher . . . 807
1.6 Unavailable in sub-encoding 6 and higher . . . 807
1.7 Unavailable in sub-encoding 7 and higher . . . 807
1.8 Unavailable in sub-encoding 8 and higher . . . 807
1.9 Unavailable in Sub-encoding 9 (most missing) . . . 808

2 Unicode engine specials 808

3 Font family sub-encodings setup 809

4 Legacy symbol support for lists and footnote symbols 813

5 The textcomp package 818
5.1 The old textcomp package code . . . 819
5.1.1 Supporting oldstyle digits . . . 828
5.1.2 Subset encoding defaults . . . 828

6 The checkencodingsubset.tex file 830

34 ltpageno.dtx 839

1 Page Numbering 839

35 ltxref.dtx 840

1 Cross Referencing 840
1.1 Cross Referencing . . . 840

36 ltproperties.dtx 849

1 Introduction 849

2 Design discussion 849

3 Handling unknown labels and properties 850

4 Rerun messages 850

5 Open points 850

6 Code interfaces 850

7 Auxiliary file interfaces 852

8 LaTeX 2ε interface 853

9 Pre-declared properties 854

10 The Implementation 855
10.1 Reference commands . . . 857
10.2 Tests and warnings . . . 859
10.3 Predeclared properties . . . 861
10.4 Messages . . . 863

37 ltmiscen.dtx 864

1 Miscellaneous Environments 864
1.1 Environments . . . 864
1.2 Center, Flushright, Flushleft . . . 878
1.3 Verbatim . . . 881

38 ltmath.dtx 889

1 Math setup 889
1.1 Math commands based on plain TeX . . . 889
1.1.1 The log-like functions . . . 889
1.1.2 Biggggg . . . 890
1.1.3 The UNSORTED Rest . . . 890
1.2 Math Environments . . . 897
1.3 External options to the standard document classes . . . 904
1.3.1 Left equation numbering . . . 904
1.3.2 Flush left equations . . . 904

39 ltlists.dtx 907

1 List, and related environments 907
1.1 List and Trivlist . . . 908
1.2 Vertical Spacing (skips) . . . 909
1.3 Penalties . . . 909
1.4 Horizontal Spacing (dimens) . . . 909
1.5 Default Values . . . 909
1.6 Itemize and Enumerate . . . 922

40 ltboxes.dtx 925

1 LaTeX Box commands 925
1.1 Some low-level constructs . . . 945

41 lttab.dtx 947

1 Tabbing, Tabular and Array Environments 947
1.1 tabbing . . . 947
1.2 array and tabular environments . . . 956
1.3 Hooks for tabulars . . . 972

42 ltpictur.dtx 974

1 Picture Mode 974
1.1 Curves . . . 1002

43 ltthm.dtx 1007

1 Theorem Environments 1007

44 ltsect.dtx 1012

1 Sectioning Commands 1012
1.1 The Title . . . 1012
1.2 Sectioning . . . 1013
1.2.1 Initializations . . . 1020
1.3 Table of Contents etc. . . . 1020
1.3.1 Convention . . . 1020
1.3.2 Commands . . . 1020

45 ltfloat.dtx 1026

1 Floats **1026**
1.1 Floating Environments 1026
1.2 Footnotes 1040

46 ltidxglo.dtx **1050**

1 Index and Glossary Generation **1050**

47 ltbibl.dtx **1053**

1 Bibliography Generation **1053**
1.1 Default definitions 1057

48 ltmarks.dtx **1058**

1 Introduction **1058**

2 Design-level and code-level interfaces **1059**
2.1 Use cases for conditionals 1061
2.2 Understanding regions 1061
2.3 Debugging mark code 1063

3 Application examples **1063**

4 Legacy LaTeX 2ε interface **1063**
4.1 Legacy design-level and document-level interfaces 1064
4.2 Legacy interface extensions 1064

5 Notes on the mechanism **1065**

6 Public interfaces for packages such as multicol **1066**

7 Internal functions for the standard output routine of LaTeX **1067**

8 The Implementation **1068**
8.1 Allocating new mark classes 1068
8.2 Updating mark structures 1070
8.3 Placing and retrieving marks 1078
8.4 Comparing mark values 1080
8.5 Messages 1081
8.6 Debugging the mark structures 1081
8.7 Designer-level interfaces 1083

9 LaTeX 2ε integration **1084**
9.1 Core LaTeX 2ε integration 1084
9.2 Other LaTeX 2ε output routines 1087
9.3 Rollback information 1088

49 ltpage.dtx 1089

1 Page styles and related commands 1089
1.1 Page Style Commands . . . 1089
1.2 How a page style makes running heads and feet . . . 1089
1.3 marking conventions . . . 1089

50 ltclass.dtx 1094

1 Introduction 1094

2 User interface 1094
2.1 Option processing . . . 1095

3 Class and Package interface 1096
3.1 Class name and version . . . 1096
3.2 Package name and version . . . 1096
3.3 Requiring other packages . . . 1096
3.4 Declaring new options . . . 1098
3.5 Safe Input Macros . . . 1098

4 Implementation 1098
4.1 Hooks . . . 1128
4.2 Providing shipment . . . 1131

5 Package/class rollback mechanism 1139

6 After Preamble 1147

51 ltkeys.dtx 1149

1 Creating and using keyval options 1149
1.1 Implementation of ltkeys . . . 1151
1.2 Key properties . . . 1151
1.3 Main mechanism . . . 1152
1.4 The document interfaces . . . 1156
1.5 Option usage scope . . . 1157
1.6 General key setting . . . 1158

52 ltfilehook.dtx 1160

1 Introduction **1160**
1.1 Provided hooks . . . 1160
1.2 General hooks for file reading . . . 1160
1.3 Hooks for package and class files . . . 1161
1.4 Hooks for `\include` files . . . 1162
1.5 High-level interfaces for LaTeX . . . 1163
1.6 Kernel, class, and package interfaces for LaTeX . . . 1164
1.7 A sample package for structuring the log output . . . 1164

2 The Implementation **1165**
2.1 Document and package-level commands . . . 1165
2.2 expl3 helpers . . . 1166
2.3 Declaring the file-related hooks . . . 1169
2.4 Patching LaTeX's `\InputIfFileExists` command . . . 1169
2.5 Declaring a file substitution . . . 1171
2.6 Selecting a file (`\set@curr@file`) . . . 1173
2.7 Replacing a file and detecting loops . . . 1176
2.7.1 The Tortoise and Hare algorithm . . . 1177
2.8 Preventing a package from loading . . . 1179
2.9 High-level interfaces for LaTeX . . . 1180
2.10 Internal commands needed elsewhere . . . 1180

3 A sample package for structuring the log output **1181**

4 Package emulations **1182**
4.1 Package atveryend emulation . . . 1182

53 ltshipout.dtx **1184**

1 Introduction **1184**
1.1 Overloading the `\shipout` primitive . . . 1184
1.2 Provided hooks . . . 1185
1.3 Legacy LaTeX commands . . . 1187
1.4 Special commands for use inside the hooks . . . 1188
1.5 Provided LuaTeX callbacks . . . 1188
1.6 Information counters . . . 1189
1.7 Debugging shipout code . . . 1189

2 Emulating commands from other packages **1190**
2.1 Emulating atbegshi . . . 1190
2.2 Emulating everyshi . . . 1191
2.3 Emulating atenddvi . . . 1191
2.4 Emulating everypage . . . 1191

3 The Implementation **1192**
3.1 Debugging . . . 1193
3.2 Handling the end of job hook . . . 1205

4 Legacy LaTeX 2ε interfaces **1209**

5 Internal commands needed elsewhere 1209

6 Package emulation for compatibility 1211
6.1 Package atenddvi emulation . . . 1211
6.2 Package atbegshi emulation . . . 1212
6.3 Package everyshi emulation . . . 1213

54 ltoutput.dtx 1214

1 Output Routine and float handling 1214
1.1 Historical notes on the algorithm and commands . . . 1214
1.2 Core definitions . . . 1224
1.2.1 Definition of float boxes . . . 1224
1.2.2 Page layout parameters . . . 1225
1.2.3 Internal registers . . . 1227
1.2.4 Page break commands . . . 1227

2 The LaTeX output routine 1232
2.1 Hooks and replaceable code blocks . . . 1232
2.1.1 Output routine hooks . . . 1232
2.1.2 Replaceable code blocks (sockets) . . . 1232
2.1.3 Tagging sockets . . . 1234
2.1.4 Output routine commands . . . 1235
2.2 The output routine configuration components . . . 1246
2.2.1 Dealing with floats . . . 1260
2.2.2 Kludgeins . . . 1289
2.2.3 Float control . . . 1291
2.2.4 Float placement parameters . . . 1305

55 lttagging.dtx 1309

1 General support for tagged output 1309

2 Implementation 1311
2.1 Math collection . . . 1312
2.2 Tagging sockets . . . 1312
2.3 Generic sockets . . . 1312
2.3.1 Tagging support for paragraph setup . . . 1313
2.3.2 Tagging socket for targets . . . 1317
2.3.3 Tagging Sockets for boxes . . . 1317
2.3.4 Tagging Sockets for lists and blocks . . . 1317
2.3.5 Tagging sockets for headings . . . 1317
2.3.6 Tagging sockets for toc . . . 1318
2.3.7 Tagging support for marginpar . . . 1318
2.3.8 Tagging support for table/tabular packages . . . 1318
2.3.9 Tagging Support for floats . . . 1320
2.4 Tagging support for output routines . . . 1320
2.5 Tagging support for math . . . 1321
2.5.1 General sockets . . . 1321

2.5.2 Sockets specific for luamml . . . 1321
2.6 MathML intent attributes . . . 1323
2.7 Symbolic structure names . . . 1323

3 For lttab.dtx parked here for now 1323
3.1 Variables for row, column and span counting . . . 1323
3.2 Tracing/debugging . . . 1324
3.3 Interface commands . . . 1325

56 lthyphen.dtx 1330

57 ltfinal.dtx 1332

1 Final settings 1332
1.1 Debugging . . . 1332
1.2 Typesetting parameters . . . 1332
1.3 Lccodes for hyphenation . . . 1336
1.4 Hyphenation . . . 1339
1.5 Font loading . . . 1339
1.6 Input encoding . . . 1342
1.7 Lccodes and uccodes . . . 1346
1.8 Case changing . . . 1347
1.9 Automatic insertion of `\par` tokens . . . 1350
1.10 Applying Patch files . . . 1350
1.11 Freeing Memory . . . 1351
1.12 Initialize file list . . . 1352
1.13 Initialize file list . . . 1352
1.14 Preparation for supporting PDF in backends . . . 1352
1.15 Do some temporary work for pre-release . . . 1353
1.16 Some last minute initializations 1353
1.17 Dumping the format . . . 1353

Change History 1357

Index 1445

File 01
ltdirchk.dtx

1 LaTeX System Dependent Initializations

This file implements the semi-automatic determination of various system dependent parts of the initialization. The actual definitions may be placed in a file `texsys.cfg`. Thus for operating systems for which the tests here do not result in acceptable settings, a 'hand written' `texsys.cfg` may be produced.

The macros that must be defined are:

`\@currdir` `\@currdir`⟨*filename*⟩⟨*space*⟩ should expand to a form of the filename that uniquely refers to the 'current directory' if this is possible. (The expansion should also end with a space.) on UNIX, this is `\def\@currdir{./}`. For more exotic operating systems you may want to make `\@currdir` a macro with arguments delimited by `.` and/or ⟨*space*⟩. If the operating system has no concept of directory structure, this macro should be defined to be empty.

`\input@path` If the primitive `\openin` searches the same directories as the primitive `\input`, then it is possible to tell (using `\ifeof`) whether a file exists before trying to input it. For systems like this, `\input@path` should be left undefined.

If `\openin` does not 'follow' `\input` then `\input@path` must be defined to be a list of directories to search for input files. The format for each directory is as for `\@currdir`, normally just a prefix is required, but it may be a macro with space-delimited argument. That is, if ⟨*dir*⟩ is an entry in the input path, TeX will try to load the expansion of ⟨*dir*⟩⟨*filename*⟩⟨*space*⟩

So either ⟨*dir*⟩ should be defined as a macro with argument delimited by space, or it should just expand to a directory name, including the final directory separator, so that it may be concatenated with the ⟨*filename*⟩. This means that for UNIX-like syntax, each ⟨*dir*⟩ should end with a slash, `/`.

`\input@path` should expand to a list of such directories, each in a `{}` group.

`\filename@parse` After a call of the form: `\filename@parse{`⟨*filename*⟩`}`, the three macros `\filename@area`, `\filename@base` and `\filename@ext` should be defined to be the 'area' (or directory), basename and extension respectively. If there was no extension specified in ⟨*filename*⟩, `\filename@ext` should be `\let` to `\relax` (so this case may be tested with `\@ifundefined{filename@ext}` and, perhaps a default extension substituted).

Normally one would not need to define this macro in `texsys.cfg` as the automatic tests can supply parsers that work with UNIX and VMS and Macintosh syntax, as well as a basic parser that will cover many other cases. However some operating systems may need a 'hand produced' parser in which case it should be defined in this file.

The UNIX parser also works for most MSDOS TeX versions. Currently if the UNIX, VMS or Macintosh parser is not used, `\filename@parse` is defined to always return an empty area, and to split the argument into basename and extension at the first '`.`' that occurs in the name. Parsers for other formats may be defined in `texsys.cfg`, in which case they will be used in preference to the default definitions.

`\@TeXversion` `\@TeXversion` is now set automatically by the initialization tests in this file. You should not need to set it in `texsys.cfg`, however the following documentation is left for information. LaTeX does not set this variable exactly, the automatic tests set it to:
2 for any version, v, $v < 3.0$
3 for any version, v, $3.0 \leq v \leq 3.14$

⟨*undefined*⟩ otherwise.
However these values are accurate enough for LaTeX to take appropriate action for these old TeXs.

If your TeX is older than version 3.141, then you should define `\@TeXversion` (using `\def`) to be the version number. If you do not do this[1] , LaTeX will not work around a bug in old TeX versions, and so error messages will appear in a very strange format, with `^^J` appearing instead of line breaks:

```
 LaTeX Error: \rubbish undefined.^^J^^JSee the LaTeX manual or LaTeX=
Companion
for explanation.^^JType  H <return>  for immediate help.
...

.3 \renewcommand{\rubbish}
                           {}
```

However if you put `\def\@TeXversion{3.14}` in `texsys.cfg` the following format will be used:

```
 LaTeX Error: \rubbish undefined.

ee the LaTeX manual or LaTeX Companion for explanation.
ype  H <return>  for immediate help.
 .
...

.3 \renewcommand{\rubbish}
                           {}
```

Note that this has an extra line `! .` which does not appear in error messages that use the default settings with a current version of TeX, but this should not cause any confusion we hope.

2 Initialization

As this file is read at a very early stage, some definitions that are normally considered to be part of the format must be made here.

Most such definitions are repeated later in the "right" place, usually (but not always) with different implementations. To be able to spot this more easily if you look into the file `latex.ltx` (which is stripped of comments) we add some comment lines to that effect that survive the stripping process by `docstrip`.

```
1 ⟨*dircheck⟩
2 %% ---- START temporary definitions for bootstrapping; later overwritten ----
3 ⟨/dircheck⟩
```

[1] Actually if your TeX is really old, version 2, LaTeX can detect this, and sets `\@TeXversion` to 2 if it is not set in the `cfg` file.

2.1 INITEX

```
⟨*dircheck⟩
⟨*initex⟩
⟨initex⟩\ifnum\catcode`\{=1
⟨initex⟩  \errmessage
⟨initex⟩  {LaTeX must be made using an initex with no format preloaded}
⟨initex⟩\fi
\catcode`\{=1
\catcode`\}=2
```

If LuaTEX is in use the extensions and other new primitives have to be activated: this is done as early as possible. Older versions of LuaTEX do not hide the primitives: a version check is not needed as the version itself will be missing in the case where action is needed!

```
\ifx\directlua\undefined
\else
  \ifx\luatexversion\undefined
```

Enable e-TeX/pdfTeX/Umath primitives with their natural names

```
    \directlua{tex.enableprimitives("",%
                  tex.extraprimitives('etex', 'pdftex', 'umath'))}
```

In current formats enable primitives with unprefixed names. the latexrelease guards allow the primitives to be defined with a `\luatex` prefix if older formats are specified.

The unprefixed forms are *not* undefined for improved compatibility with external packages when rolling back the format.

```
⟨/initex⟩
⟨/dircheck⟩
⟨*initex, latexrelease⟩
⟨latexrelease⟩\ifx\directlua\undefined\else
⟨latexrelease⟩\IncludeInRelease{2015/10/01}{\luatexluafunction}
⟨latexrelease⟩                                {LuaTeX (prefixed names)}%
    \directlua{tex.enableprimitives("",%
                  tex.extraprimitives("omega", "aleph", "luatex"))}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\luatexluafunction}
⟨latexrelease⟩                                {LuaTeX (prefixed names)}%
⟨latexrelease⟩\directlua{
⟨latexrelease⟩  tex.enableprimitives(
⟨latexrelease⟩    "luatex",
⟨latexrelease⟩    tex.extraprimitives("core","omega", "aleph", "luatex")
⟨latexrelease⟩  )
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\fi
⟨/initex, latexrelease⟩
⟨*dircheck⟩
⟨*initex⟩
  \fi
\fi
```

A test can now be made for eTEX.

```
⟨initex⟩\ifx\eTeXversion\undefined
⟨initex⟩  \errmessage
⟨initex⟩    {LaTeX requires e-TeX}
⟨initex⟩  \expandafter\endinput
```

```
⟨initex⟩\fi
```

That distraction over, back to the basics of a format.

```
\catcode`\#=6
\catcode`\^=7
\chardef\active=13
\catcode`\@=11
\countdef\count@=255
\let\bgroup={ \let\egroup=}
\ifx\@@input\@undefined\let\@@input\input\fi
\ifx\@@end\@undefined\let\@@end\end\fi
\chardef\@inputcheck0
\chardef\sixt@@n=16
\newlinechar`\^^J
\def\typeout{\immediate\write17}
\def\dospecials{\do\ \do\\\do\{\do\}\do\$\do\&%
  \do\#\do\^\do\_\do\%\do\~\do\^^I}
\def\@makeother#1{\catcode`#1=12\relax}
\def\space{ }
\def\@tempswafalse{\let\if@tempswa\iffalse}
\def\@tempswatrue{\let\if@tempswa\iftrue}
\let\if@tempswa\iffalse
\def\loop#1\repeat{\def\iterate{#1\relax\expandafter\iterate\fi}%
  \iterate \let\iterate\relax}
\let\repeat\fi
⟨/initex⟩
```

2.2 Some bits of 2e

```
⟨*2ekernel⟩
\def\two@digits#1{\ifnum#1<10 0\fi\number#1}
\long\def\@firstoftwo#1#2{#1}
\long\def\@secondoftwo#1#2{#2}
```

This is a special version of `\ProvidesFile` for initex use.

```
\def\ProvidesFile#1{%
  \begingroup
    \catcode`\ 10 %
    \ifnum \endlinechar<256 %
      \ifnum \endlinechar>\m@ne
        \catcode\endlinechar 10 %
      \fi
    \fi
    \@makeother\/%
    \@ifnextchar[{\@providesfile{#1}}{\@providesfile{#1}[]}}
\def\@providesfile#1[#2]{%
    \wlog{File: #1 #2}%
    \@addtofilelist{ #2}%
    \endgroup}
\long\def\@addtofilelist#1{}
\def\@empty{}
\catcode`\%=12
\def\@percentchar{%}
\catcode`\%=14
\let\@currdir\@undefined
\let\input@path\@undefined
```

```
\let\filename@parse\@undefined
```

\strip@prefix

```
\def\strip@prefix#1>{}
⟨/2ekernel⟩
```

(*End of definition for* `\strip@prefix`.)

3 texsys.cfg

As mentioned above, any site specific definitions required to describe the filename handling must be entered into a file `texsys.cfg`. If `texsys.cfg` can not be located by `\openin`, we write a default version out. The default version only contains comments, so we do not actually input the file in that case. The automatic tests later will, hopefully, correctly define the required macros.

The tricky code below checks to see if `texsys.cfg` exists. If it does not, all the text in this file between START and END is copied verbatim to a new file `texsys,cfg`. If `texsys.cfg` is found, then it is simply input. This is only done when this file is being used unstripped.

```
⟨*docstrip⟩
\openin15=texsys.cfg
\ifeof15
\typeout{** Writing a default texsys.cfg}
\immediate\openout15=texsys.cfg
\begingroup
\catcode`\^^M\active%
\let^^M\par%
\def\reserved@a#1^^M{%
 \def\reserved@b{#1}%
 \ifx\reserved@b\reserved@c\endgroup\else%
     \immediate\write15{#1}%
     \expandafter\reserved@a\fi}%
\def\reserved@d#1START^^M{\let\do\@makeother\dospecials\reserved@a}%
\catcode`\%=12
\def\reserved@c{%END}
\reserved@d
```

START

3.1 texsys.cfg

This file contains the site specific definitions of the four macros
`\@currdir`, `\input@path`, `\filename@parse` and `\@TeXversion`.

As distributed it only contains comments, however this 'empty' file will work on many systems because of the automatic tests built into `ltdirchk.dtx`. You *are* allowed to edit this file to add definitions of these macros appropriate to your system.

The macros that must be defined are:

\@currdir `\@currdir`⟨*filename*⟩⟨*space*⟩ should expand to a form of the filename that uniquely refers to the 'current directory' if this is possible. (The expansion should also end with a space.) on UNIX, this is `\def\@currdir{./}`. For more exotic operating systems you may want to make `\@currdir` a macro with arguments delimited by `.` and/or ⟨*space*⟩. If

the operating system has no concept of directory structure, this macro should be defined to be empty.

\input@path If the primitive \openin searches the same directories as the primitive \input, then it is possible to tell (using \ifeof) whether a file exists before trying to input it. For systems like this, \input@path should be left undefined.

If \openin does not 'follow' \input then \input@path must be defined to be a list of directories to search for input files. The format for each directory is as for \@currdir, normally just a prefix is required, but it may be a macro with space-delimited argument. That is, if ⟨*dir*⟩ is an entry in the input path, TEXwill try to load the expansion of

⟨*dir*⟩⟨*filename*⟩⟨*space*⟩

So either ⟨*dir*⟩ should be defined as a macro with argument delimited by space, or it should just expand to a directory name, including the final directory separator, so that it may be concatenated with the ⟨*filename*⟩. This means that for UNIX-like syntax, each ⟨*dir*⟩ should end with a slash, /. One exception to this rule is that the input path should *always* contain the empty directory {} as this will allow 'full pathnames' to be used, and the 'current directory' to be searched.

\input@path should expand to a list of such directories, each in a {} group.

\filename@parse After a call of the form: \filename@parse{⟨*filename*⟩}, the three macros \filename@area, \filename@base, \filename@ext should be defined to be the 'area' (or directory), basename and extension respectively. If there was no extension specified in ⟨*filename*⟩, \filename@ext should be \let to \relax (so this case may be tested with \@ifundefined{filename@ext} and, perhaps a default extension substituted).

Normally one would not need to define this macro in texsys.cfg as the automatic tests can supply parsers that work with UNIX and VMS syntax, as well as a basic parser that will cover many other cases. However some operating systems may need a 'hand produced' parser in which case it should be defined in this file.

The UNIX parser also works for most MSDOS TEX versions. Currently if the UNIX or VMS parser is not used, \filename@parse is defined to always return an empty area, and to split the argument into basename and extension at the first '.' that occurs in the name. Parsers for other formats may be defined in texsys.cfg, in which case they will be used in preference to the default definitions.

\@TeXversion You should not need to set this macro in texsys.cfg. LATEX tests to set this automatically. See the comments in the opening section of ltdirchk.dtx.

The following sections give examples of definitions which might work on various systems. These are currently mainly untested as I only have access to a few systems, all of which do not need this file as the automatic tests work. All the code is commented out.

3.2 UNIX (web2c)

This implementation does make \openin and \input look in the same places. Acceptable settings are made by ltdirchk.dtx, and so this file may be empty. The definitions below are therefore just for information.

```
%\def\@currdir{./}
%\let\input@path\@undefined
```

3.3 UNIX (other)

Apparently some commercial UNIX implementations have different paths for `\openin` and `\input`. For these one could use definitions like the following (with whatever directories are used at your site): note that the directory names should end with `/`.

```
% \def\@currdir{./}
% \def\input@path{%
%   {/usr/local/lib/tex/inputs/distrib/}%
%   {/usr/local/lib/tex/inputs/contrib/}%
%   {/usr/local/lib/tex/inputs/local/}%
% }
```

3.4 MSDOS (emtex)

This implementation does make `\openin` and `\input` look in the same places. Acceptable settings are made by `ltdirchk.dtx`, and so this file may be empty. The definitions below are therefore just for information.

```
% \def\@currdir{./}
% \let\input@path\@undefined
```

3.5 MSDOS (other)

Some PC implementations have different paths for `\openin` and `\input`. For these one could use definitions like the following (with whatever directories are used at your site): note that the directory names should end with `/`. This assumes the implementation uses UNIX style `/` as the directory separator.

```
% \def\@currdir{./}
% \def\input@path{%
%   {c:/tex/inputs/distrib/}%
%   {c:/tex/inputs/contrib/}%
%   {c:/tex/inputs/local/}%
% }
```

3.6 VMS (DECUS TEX, PD VMS 3.6)

This implementation does make `\openin` and `\input` look in the same places. Acceptable settings are made by `ltdirchk.dtx`, and so this file may be empty. The definitions below are therefore just for information.

```
% \def\@currdir{[]}
% \let\input@path\@undefined
```

3.7 VMS (???)

Some VMS implementations have different paths for `\openin` and `\input`. For these one could use definitions like the following:

```
% \def\@currdir{[]}
% \def\input@path{%
%   {tex_inputs:}%
%   {SOMEDISK:[SOME.TEX.DIRECTORY]}%
% }
```

3.8 MACINTOSH (OzTeX 1.6)

This implementation does make `\openin` and `\input` look in the same places. Acceptable settings are made by `ltdirchk.dtx`, and so this file may be empty. The definitions below are therefore just for information.

```
% \def\@currdir{:}
% \let\input@path\@undefined
```

3.9 MACINTOSH (other)

Some Macintosh implementations have different paths for `\openin` and `\input`. For these one could use definitions like the following (with whatever folders are used on your machine): note that the directory names should end with `:`, and they should contain *no* spaces.

```
% \def\@currdir{:}
% \def\input@path{%
%   {Hard-Disk:Applications:TeX:TeX-inputs:}%
%   {Hard-Disk:Applications:TeX:My-inputs:}%
% }
```

3.10 FAKE EXAMPLE

This example is for an operating system that has filenames of the form `<area>name` For maximum compatibility with macro sets, you want `name.ext` to be mapped to `<ext>name`. and `<area>name.ext` to be mapped to `<area.ext>name`. `\input` does this mapping automatically, but `\openin` does not, and does not look in the same places as `\input`. `<>name` is the desired 'current directory' syntax.

the following code would possibly work:

```
% \def\@dir#1#2 {%
%   \@d@r{#1}#2..\@nil}
% \def\@d@r#1#2.#3.#4\@nil{%
%   <\ifx\@dir#1\@dir\else#1\ifx\@dir#3\@dir\else.\fi\fi#3>#2 }
%
% \def\@currdir{\@dir{}}
% \def\input@path{%
%   {\@dir{area.one}}%
%   {\@dir{area.two}}%
% }
```

END

```
\immediate\closeout15
```

If `texsys.cfg` did exist, then input it.

```
\else
\typeout{** Using the existing texsys.cfg}
\closein15
\input texsys.cfg
\fi
```

⟨/docstrip⟩

If the stripped version of this file is being used (in latex2e.ltx) then texsys.cfg should be there, so just input it.

```
⟨dircheck⟩\input texsys.cfg
```

4 Setting \@currdir

\@currdir
\IfFileExists

This is a local definition of \IfFileExists. It tries to relocate texsxys.aux. If it succeeds, then the \@currdir syntax has been determined. If all the tests fail then \@currdir will be set to \@empty, and ltxcheck will warn of this when it checks the format.

```
\begingroup
\count@\time
\divide\count@ 60
\count2=-\count@
\multiply\count2 60
\advance\count2 \time
```

The current date and time stamp.

\today

```
\edef\today{%
  \the\year/\two@digits{\the\month}/\two@digits{\the\day}:%
    \two@digits{\the\count@}:\two@digits{\the\count2}}
```

Create a file texsys.aux (hopefully in the current directory), then try to locate it again.

```
\immediate\openout15=texsys.aux
\immediate\write15{\today^^J}
\immediate\closeout15 %
```

#1 is the file to try, #2 is what to do on success, #3 on failure. Note that this definition is overwritten later on again!

```
\def\IfFileExists#1#2#3{%
  \openin\@inputcheck#1 %
  \ifeof\@inputcheck
     #3\relax
  \else
    \read\@inputcheck to \reserved@a
    \ifx\reserved@a\today
      \typeout{#1 found}#2\relax
    \else
      \typeout{BAD: old file \reserved@a (should be \today)}%
      #3\relax
    \fi
  \fi
  \closein\@inputcheck}
\endlinechar=-1
```

If \@currdir has not been pre-defined in texsys.cfg then test for UNIX, VMS and Oz-TEX-Mac. syntax.

```
\ifx\@currdir\@undefined
  \IfFileExists{./texsys.aux}{\gdef\@currdir{./}}%
    {\IfFileExists{[]texsys.aux}{\gdef\@currdir{[]}}%
      {\IfFileExists{:texsys.aux}{\gdef\@currdir{:}}{}}}
```

If it is still undefined at this point, all the above tests failed. Earlier versions interactively prompted for a definition at this point, but it seems impossible to reliably obtain information from users at this point in the installation. This version of the file produces

a format with no user-interaction. Later if the format is not suitable for the system, `texsys.cfg` may be edited and the format re-made.

```
\ifx\@currdir\@undefined
  \global\let\@currdir\@empty
  \typeout{^^J^^J%
    !! No syntax for the current directory could be found^^J%
    }%
\fi
```

Otherwise `\@currdir` was defined in `texsys.cfg`. In this case check that the syntax specified works on this system. (In case a complete LaTeX system has been copied from one system to another.) If the test fails, give up. The installer should remove or correct the offending `texsys.cfg` and try again.

```
\else
  \IfFileExists{\@currdir texsys.aux}{}{%
    \edef\reserved@a{\errhelp{%
      texsys.cfg specifies the current directory syntax to be^^J%
      \meaning\@currdir^^J%
      but this does not work on this system.^^J%
      Remove texsys.cfg and restart.}}\reserved@a
    \errmessage{Bad texsys.cfg file: \noexpand\@currdir}\@@end}
```

The version of `\@currdir` in `texsys.cfg` looks OK.

```
\fi

\immediate\closeout15 %
\endgroup

\typeout{^^J^^J%
         \noexpand\@currdir set to:
           \expandafter\strip@prefix\meaning\@currdir.^^J%
         }
```

(*End of definition for* `\@currdir`, `\IfFileExists`, *and* `\today`.)

Stop here if the file is being used unstripped.

```
⟨*docstrip⟩
\relax\endinput
⟨/docstrip⟩
```

5 Setting `\input@path`

Earlier versions of this file attempted to automatically test whether `\input@path` was required, and interactively prompt for a path if necessary. This was not found to be very reliable The first-time installer of LaTeX 2ε can not be expected to have enough information to supply the correct information to the prompts. Now the interaction is omitted. After the format is made the installer can attempt to run the test document `ltxcheck.tex` through LaTeX 2ε. This will check, among other things, whether `texsys.cfg` will need to be edited and the format remade.

`\input@path` Now set up the `\input@path`.

`\input@path` should either be undefined, or a list of directories as described in the introduction.

```
\typeout{^^J%
```

```
    Assuming \noexpand\openin and \noexpand\input^^J%
    \ifx\input@path\@undefined
```

`\input@path` has not been pre-defined.

```
      have the same search path.^^J%
    \else
```

`\input@path` has been defined in `texsys.cfg`.

```
      have different  search paths.^^J%
      LaTeX will use the path specified by \noexpand\input@path:^^J%
    \fi
    }
```

(*End of definition for* `\input@path`.)

6 Filename Parsing

`\filename@parse` Split a filename into its components.

```
\ifx\filename@parse\@undefined
  \def\reserved@a{./}\ifx\@currdir\reserved@a
```

`\filename@parse` was not specified in `texsys.cfg`, but `\@currdir` looks like UNIX...

```
    \typeout{^^JDefining UNIX/DOS style filename parser.^^J}
    \def\filename@parse#1{%
      \let\filename@area\@empty
      \expandafter\filename@path#1/\\}
```

Search for the last `/`.

```
    \def\filename@path#1/#2\\{%
      \ifx\\#2\\%
         \def\reserved@a{\filename@simple#1.\\}%
      \else
         \edef\filename@area{\filename@area#1/}%
         \def\reserved@a{\filename@path#2\\}%
      \fi
      \reserved@a}
  \else\def\reserved@a{[]}\ifx\@currdir\reserved@a
```

`\filename@parse` was not specified in `texsys.cfg`, but `\@currdir` looks like VMS...

```
    \typeout{^^JDefining VMS style filename parser.^^J}
    \def\filename@parse#1{%
      \let\filename@area\@empty
      \expandafter\filename@path#1]\\}
```

Search for the last `]`.

```
    \def\filename@path#1]#2\\{%
      \ifx\\#2\\%
         \def\reserved@a{\filename@simple#1.\\}%
      \else
         \edef\filename@area{\filename@area#1]}%
         \def\reserved@a{\filename@path#2\\}%
      \fi
      \reserved@a}
  \else\def\reserved@a{:}\ifx\@currdir\reserved@a
```

`\filename@parse` was not specified in `texsys.cfg`, but `\@currdir` looks like Macintosh...

```
    \typeout{^^JDefining Mac style filename parser.^^J}
    \def\filename@parse#1{%
      \let\filename@area\@empty
      \expandafter\filename@path#1:\\}
```

Search for the last `:`.

```
    \def\filename@path#1:#2\\{%
      \ifx\\#2\\%
         \def\reserved@a{\filename@simple#1.\\}%
      \else
         \edef\filename@area{\filename@area#1:}%
         \def\reserved@a{\filename@path#2\\}%
      \fi
      \reserved@a}
  \else
```

`\filename@parse` was not specified in `texsys.cfg`. So just make a simple parser that always sets `\filename@area` to empty.

```
    \typeout{^^JDefining generic filename parser.^^J}
    \def\filename@parse#1{%
      \let\filename@area\@empty
      \expandafter\filename@simple#1.\\}
  \fi\fi\fi
```

`\filename@simple` is used by all three versions. Finally we can split off the extension.

```
⟨/dircheck⟩
⟨*dircheck, latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2019/10/01}{\filename@simple}
⟨latexrelease⟩                                {Final dot for extension}%
\def\filename@simple#1.#2\\{%
  \ifx\\#2\\%
    \let\filename@ext\relax
    \edef\filename@base{#1}%
  \else
    \filename@dots{#1}#2\\%
  \fi}
\def\filename@dots#1#2.#3\\{%
  \ifx\\#3\\%
    \def\filename@ext{#2}%
    \edef\filename@base{#1}%
  \else
    \filename@dots{#1.#2}#3\\%
  \fi}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\filename@simple}
⟨latexrelease⟩                                {Final dot for extension}%
⟨latexrelease⟩   \def\filename@simple#1.#2\\{%
⟨latexrelease⟩     \ifx\\#2\\%
⟨latexrelease⟩        \let\filename@ext\relax
⟨latexrelease⟩     \else
⟨latexrelease⟩        \edef\filename@ext{\filename@dot#2\\}%
```

```
⟨latexrelease⟩      \fi
⟨latexrelease⟩      \edef\filename@base{#1}}
⟨latexrelease⟩ \EndIncludeInRelease
⟨/dircheck, latexrelease⟩
⟨∗dircheck⟩
```

Remove a final dot, added earlier.

```
  \def\filename@dot#1.\\{#1}
\else
```

Otherwise, `\filename@parse` was specified in `texsys.cfg`.

```
  \typeout{^^J^^J%
    \noexpand\filename@parse was defined in texsys.cfg:^^J%
    \expandafter\strip@prefix\meaning\filename@parse.^^J%
    }
\fi
```

(*End of definition for* `\filename@parse`.)

7 TEX Versions

`\@TeXversion` TEX versions older than 3.141 require `\@TeXversion` to be set. This can be determined automatically due to a trick suggested by Bernd Raichle. Actually this will not always get the correct version number, e.g., TEX3.14 would be detected as TEX3, but LATEX only needs to take account of TEX's older than 3, or between 3 and 3.14.

```
\ifx\@TeXversion\@undefined
  \ifx\@undefined\inputlineno
    \def\@TeXversion{2}
  \else
   {\catcode`\^^J=\active
     \def\reserved@a#1#2\@@{\if#1\string^3\fi}
     \edef\reserved@a{\expandafter\reserved@a\string^^J\@@}
     \ifx\reserved@a\@empty\else\gdef\@TeXversion{3}\fi}
  \fi
\fi
```

(*End of definition for* `\@TeXversion`.)

```
%% ---- END temporary definitions for bootstrapping ----
⟨/dircheck⟩
```

8 ltxcheck.tex

After the format has been made, and article.cls moved with the other files to the 'standard input directory' as specified in `install.txt`, the format may be checked by running the file `ltxcheck.tex`.

File 02
ltplain.dtx

1 Plain TEX

LATEX includes almost all of the functionality of Knuth's original 'Basic Macros' That is, the plain TEX format described in Appendix B of the TEXBook. However, some of the user commands are not much use so, in order to save memory, we may remove them from the kernel into a package. Here is a list of the commands that may be removed (PROBABLY NOT COMPLETE).

```
\magstep    \magstephalf
\mathhexbox
\vglue      \vgl@
\hglue      \hgl@
```

This file is by now very small as most of it has been moved to more appropriate kernel files: it may disappear completely one day.

LATEX font definitions are done using NFSS2 so none of PLAIN's font definitions are in LATEX.

LATEX has its own tabbing environment, so PLAIN's is disabled.

LATEX uses its own output routine, so most of the plain one was removed.

```
⟨*2ekernel⟩
\catcode`\{=1 % left brace is begin-group character
\catcode`\}=2 % right brace is end-group character
\catcode`\$=3 % dollar sign is math shift
\catcode`\&=4 % ampersand is alignment tab
\catcode`\#=6 % hash mark is macro parameter character
\catcode`\^=7 % circumflex and uparrow are for superscripts
\catcode`\_=8 % underline and downarrow are for subscripts
\catcode`\^^I=10 % ascii tab is a blank space
\chardef\active=13 \catcode`\~=\active % tilde is active
\catcode`\^^L=\active \def^^L{\par}% ascii form-feed is \par
\message{catcodes,}
```

We had to define the `\catcodes` right away, before the message line, since `\message` uses the `{` and `}` characters. When INITEX (the TEX initializer) starts up, it has defined the following `\catcode` values:

`\catcode`\^^@=9 %` ascii null is ignored
`\catcode`\^^M=5 %` ascii return is end-line
`\catcode`\\=0 %` backslash is TeX escape character
`\catcode`\%=14 %` percent sign is comment character
`\catcode`\ =10 %` ascii space is blank space
`\catcode`\^^?=15 %` ascii delete is invalid
`\catcode`\A=11 ... \catcode`\Z=11 %` uppercase letters
`\catcode`\a=11 ... \catcode`\z=11 %` lowercase letters
all others are type 12 (other)

Here is a list of the characters that have been specially catcoded:

```
\def\dospecials{\do\ \do\\\do\{\do\}\do\$\do\&%
  \do\#\do\^\do\_\do\%\do\~\do\^^I}
```

(not counting ascii null, linefeed, formfeed, return, delete) Each symbol in the list is preceded by , which can be defined if you want to do something to every item in the list.

We make `@` signs act like letters, temporarily, to avoid conflict between user names and internal control sequences of plain format.

```
\catcode`@=11
```

To make the plain macros more efficient in time and space, several constant values are declared here as control sequences. If they were changed, anything could happen; so they are private symbols.

`\@ne` `\tw@` `\thr@@` `\sixt@@n` `\@cclv` Small constants are defined using `\chardef`.

```
\chardef\@ne=1
\chardef\tw@=2
\chardef\thr@@=3
\chardef\sixt@@n=16
\chardef\@cclv=255
```

(*End of definition for* `\@ne` *and others.*)

`\@cclvi` `\@m` `\@M` `\@MM` Constants above 255 defined using `\mathchardef`.

```
\mathchardef\@cclvi=256
\mathchardef\@m=1000
\mathchardef\@M=10000
\mathchardef\@MM=20000
```

(*End of definition for* `\@cclvi` *and others.*)

Allocation of registers

Here are macros for the automatic allocation of `\count`, `\box`, `\dimen`, `\skip`, `\muskip`, and `\toks` registers, as well as `\read` and `\write` stream numbers, `\fam` codes, `\language` codes, and `\insert` numbers.

```
\message{registers,}
```

When a register is used only temporarily, it need not be allocated; grouping can be used, making the value previously in the register return after the close of the group. The main use of these macros is for registers that are defined by one macro and used by others, possibly at different nesting levels. All such registers should be defined through these macros; otherwise conflicts may occur, especially when two or more macro packages are being used at the same time.

Historical LaTeX 2.09 comments (not necessarily accurate any more):

The following counters are reserved:

0 to 9 page numbering
10 count allocation
11 dimen allocation
12 skip allocation
13 muskip allocation
14 box allocation
15 toks allocation
16 read file allocation
17 write file allocation
18 math family allocation
19 language allocation
20 insert allocation
21 the most recently allocated number

22 constant -1

End of historical LaTeX 2.09 comments.

New counters are allocated starting with 23, 24, etc. Other registers are allocated starting with 10. This leaves 0 through 9 for the user to play with safely, except that counts 0 to 9 are considered to be the page and subpage numbers (since they are displayed during output). In this scheme, `\count` 10 always contains the number of the highest-numbered counter that has been allocated, `\count` 14 the highest-numbered box, etc. Inserts are given numbers 254, 253, etc., since they require a `\count`, `\dimen`, `\skip`, and `\box` all with the same number; `\count` 20 contains the lowest-numbered insert that has been allocated. Of course, `\box`255 is reserved for `\output`; `\count`255, `\dimen`255, and `\skip`255 can be used freely.

It is recommended that macro designers always use `\global` assignments with respect to registers numbered
1, 3, 5, 7, 9,
and always non-`\global` assignments with respect to registers
0, 2, 4, 6, 8, 255.
This will prevent "save stack buildup" that might otherwise occur.

```
\count10=22 % allocates \count registers 23, 24, ...
\count11=9 % allocates \dimen registers 10, 11, ...
\count12=9 % allocates \skip registers 10, 11, ...
\count13=9 % allocates \muskip registers 10, 11, ...
\count14=9 % allocates \box registers 10, 11, ...
\count15=9 % allocates \toks registers 10, 11, ...
\count16=-1 % allocates input streams 0, 1, ...
\count17=-1 % allocates output streams 0, 1, ...
\count18=3 % allocates math families 4, 5, ...
\count19=0 % allocates \language codes 1, 2, ...
\count20=255 % allocates insertions 254, 253, ...
```

`\insc@unt` `\allocationnumber` The insertion counter and most recent allocation.

```
\countdef\insc@unt=20
\countdef\allocationnumber=21
```

(*End of definition for* `\insc@unt` *and* `\allocationnumber`.)

`\m@ne` The constant −1.

```
\countdef\m@ne=22 \m@ne=-1
```

(*End of definition for* `\m@ne`.)

`\wlog` Write on log file (only)

```
\def\wlog{\immediate\write\m@ne}
```

(*End of definition for* `\wlog`.)

`\count@` `\dimen@` `\dimen@i` `\dimen@ii` `\skip@` `\toks@` Here are abbreviations for the names of scratch registers that don't need to be allocated.

```
\countdef\count@=255
\dimendef\dimen@=0
\dimendef\dimen@i=1 % global only
\dimendef\dimen@ii=2
\skipdef\skip@=0
\toksdef\toks@=0
```

(*End of definition for* `\count@` *and others.*)

`\newcount` `\newdimen` `\newskip` `\newmuskip` `\newbox` `\newtoks` `\newread` `\newwrite` `\newfam` `\newlanguage`

Now, we define `\newcount`, `\newbox`, etc. so that you can say `\newcount\foo` and `\foo` will be defined (with `\countdef`) to be the next counter.

To find out which counter `\foo` is, you can look at `\allocationnumber`.

Since there's no `\boxdef` command, `\chardef` is used to define a `\newbox`, `\newinsert`, `\newfam`, and so on.

LaTeX change: remove `\outer` from `\newcount` and `\newdimen` (FMi) This is necessary to use `\newcount` inside `\if...` later on. Also remove from `\newskip`, `\newbox` `\newwrite` and `\newfam` (DPC) to save later redefinition. Rolling back the allocation routine runs us into trouble nowadays as the kernel itself will use up all of the standard pool. So we pretend we've always used the full range: this is needed for roll forward.

```
\def\newcount {\e@alloc\count \countdef {\count10}\insc@unt\float@count}
\def\newdimen {\e@alloc\dimen \dimendef {\count11}\insc@unt\float@count}
\def\newskip  {\e@alloc\skip  \skipdef  {\count12}\insc@unt\float@count}
\def\newmuskip
          {\e@alloc\muskip\muskipdef{\count13}\m@ne\e@alloc@top}
```

For compatibility use `\chardef` in the classical range.

```
\def\newbox   {\e@alloc\box
                  {\ifnum\allocationnumber<\@cclvi
                     \expandafter\chardef
                   \else
                     \expandafter\e@alloc@chardef
                   \fi}
                                        {\count14}\insc@unt\float@count}
\def\newtoks  {\e@alloc\toks \toksdef{\count15}\m@ne\e@alloc@top}
\def\newread  {\e@alloc\read \chardef{\count16}\m@ne\sixt@@n}
```

Skip `\write18` due to its traditional use as a shell-escape.

```
\ifx\directlua\@undefined
  \def\newwrite   {\e@alloc\write \chardef{\count17}\m@ne\sixt@@n}
\else
  \def\newwrite   {\e@alloc\write
                   {\ifnum\allocationnumber=18
                     \advance\count17\@ne
                     \allocationnumber\count17 %
                    \fi
                    \global\chardef}%
                   {\count17}%
                   \m@ne
                   {128}}
\fi

\def\new@mathgroup
  {\e@alloc\mathgroup\chardef{\count18}\m@ne\e@mathgroup@top}
\let\newfam\new@mathgroup

\ifx\directlua\@undefined
  \def\newlanguage  {\e@alloc\language \chardef{\count19}\m@ne\@cclvi}
\else
  \def\newlanguage  {\e@alloc\language \chardef{\count19}\m@ne{16384}}
\fi
```

(*End of definition for* `\newcount` *and others.*)

\e@alloc@chardef
\e@alloc@top

The upper limit of extended registers, which leaves this number (eg \dimen32767) always unallocated by these macros. cf traditional \dimen255.

```
\ifx\directlua\@undefined
  \mathchardef\e@alloc@top=32767
  \let\e@alloc@chardef\mathchardef
\else
```

luatex has 2^{16} registers.

```
  \chardef\e@alloc@top=65535
  \let\e@alloc@chardef\chardef
\fi
```

(*End of definition for* \e@alloc@chardef *and* \e@alloc@top.)

\e@mathgroup@top

The upper limit of extended math groups (\fam) 16 in classic TEX and e-TEX, but 256 in Unicode TeX variants.

```
\ifx\Umathcode\@undefined
```

classic and e tex have 16 fam (0–15).

```
  \chardef\e@mathgroup@top=16
\else
```

xetex and luatex have 256 fam (0–255).

```
  \chardef\e@mathgroup@top=256
\fi
```

(*End of definition for* \e@mathgroup@top.)

\e@alloc

A modified version of \alloc@ that takes the count register rather than just the final digit of its number (assuming \count1x). It also has an extra argument to give the top of the extended range.

```
                #1  #2      #3     #4    #5          #6
  \e@alloc type defcmd current top extended-top newname
```

Note that if just a single allocation range is required (not omitting a range up to 255 for inserts) then −1 should be used for the first upper bound argument, #4.

```
\def\e@alloc#1#2#3#4#5#6{%
  \global\advance#3\@ne
  \e@ch@ck{#3}{#4}{#5}#1%
  \allocationnumber#3\relax
  \global#2#6\allocationnumber
  \wlog{\string#6=\string#1\the\allocationnumber}}%
```

(*End of definition for* \e@alloc.)

\e@ch@ck

Extended check command. If the first range is exceeded, bump to 256 (or 266 for counts) and try again, testing the extended range.

Allocate matching registers from the top of the extended range and add to \@freelist.

\extrafloats

```
\gdef\e@ch@ck#1#2#3#4{%
  \ifnum#1<#2\else
```

If we've reached the classical top limit, bump to 256 or 266 for counts (count 256–265 are reserved by the allocation system).

```
    \ifnum#1=#2\relax
      \global#1\@cclvi
      \ifx\count#4\global\advance#1 10 \fi
    \fi
```

Check we are below the extended limit.

```
    \ifnum#1<#3\relax
    \else
      \errmessage{No room for a new \string#4}%
    \fi
  \fi}%
\let\float@count\e@alloc@top
```

`\extrafloats`

```
\ifx\numexpr\@undefined
```

In classic TeX use `\newinsert` to allocate float boxes.

```
\def\extrafloats#1{%
\count@#1\relax
\ifnum\count@>\z@
\newinsert\reserved@a
\global\expandafter\chardef
            \csname bx@\the\allocationnumber\endcsname\allocationnumber
\@cons\@freelist{\csname bx@\the\allocationnumber\endcsname}%
\advance\count@\m@ne
\expandafter\extrafloats
\expandafter\count@
\fi
}%
\else
```

In e-tex take float boxes from the top of the extended range.

```
\def\extrafloats#1{%
\ifnum#1>\z@
\count@\numexpr\float@count-1\relax
  \ifnum\count@<266 \ch@ck0\m@ne\insert\fi
  \ch@ck0\count@\count
  \ch@ck1\count@\dimen
  \ch@ck2\count@\skip
  \ch@ck4\count@\box
\global\e@alloc@chardef\float@count\count@
\global\expandafter\e@alloc@chardef
            \csname bx@\the\float@count\endcsname\float@count
\@cons\@freelist{\csname bx@\the\float@count\endcsname}%
\expandafter\extrafloats\expandafter{\the\numexpr#1-1\expandafter}%
\fi}%
\fi
```

(*End of definition for* `\e@ch@ck`, `\extrafloats`, *and* `\extrafloats`.)

\alloc@ Since \e@alloc was added in 2015, \@alloc has not been used, but was left as some legacy code calls it. However the original definition gives spurious errors once the "classic" registers run out, so it is now defined to call \e@alloc internally.

```
\def\alloc@#1#2#3#4{\e@alloc#2#3{\count1#1}#4\float@count}
```

(*End of definition for* \alloc@.)

\newinsert The highest register allowed with \insert.

```
\ifx\directlua\@undefined
  \chardef\e@insert@top255
\else
  \chardef\e@insert@top\e@alloc@top
\fi
```

If the classic registers are exhausted, take an insert from the free float list and use \extrafloats to add a new float to that list.

```
\def\newinsert#1{%
\@tempswafalse
\global\advance\insc@unt\m@ne
\ifnum\count10<\insc@unt
\ifnum\count11<\insc@unt
\ifnum\count12<\insc@unt
\ifnum\count14<\insc@unt
  \@tempswatrue
\fi\fi\fi\fi
\if@tempswa
\allocationnumber\insc@unt
\else
\global\advance\insc@unt\@ne
  \extrafloats\@ne
  \@next\@currbox\@freelist
    {\ifnum\@currbox<\e@insert@top
       \allocationnumber\@currbox
     \else
     \ch@ck0\m@ne\insert
     \fi}%
     {\ch@ck0\m@ne\insert}%
\fi
\global\chardef#1\allocationnumber
\wlog{\string#1=\string\insert\the\allocationnumber}%
}
```

(*End of definition for* \newinsert.)

\ch@ck

```
\gdef\ch@ck#1#2#3{%
  \ifnum\count1#1<#2\else
    \errmessage{No room for a new #3}%
  \fi}
```

(*End of definition for* \ch@ck.)

\newhelp

```
\def\newhelp#1#2{\newtoks#1#1\expandafter{\csname#2\endcsname}}
```

(*End of definition for* `\newhelp`.)

`\@inputcheck` `\@unused` Allocate read stream for testing and output stream that is never open an thus writes to the terminal.

```
\newread\@inputcheck
\newwrite\@unused
```

(*End of definition for* `\@inputcheck` *and* `\@unused`.)

`\maxdimen` `\hideskip` Here are some examples of allocation.

```
\newdimen\maxdimen \maxdimen=16383.99999pt % the largest legal <dimen>
\newskip\hideskip \hideskip=-1000pt plus 1fill % negative but can grow
```

(*End of definition for* `\maxdimen` *and* `\hideskip`.)

`\p@` `\z@` `\z@skip` `\voidb@x`

```
\newdimen\p@ \p@=1pt % this saves macro space and time
\newdimen\z@ \z@=0pt % can be used both for 0pt and 0
\newskip\z@skip \z@skip=0pt plus0pt minus0pt
\newbox\voidb@x % permanently void box register
```

(*End of definition for* `\p@` *and others.*)

Assign initial values to TEX's parameters

```
\message{parameters,}
```

All of TEX's numeric parameters are listed here, but the code is commented out if no special value needs to be set. INITEX makes all parameters zero except where noted.

```
\pretolerance=100
\tolerance=200 % INITEX sets this to 10000
\hbadness=1000
\vbadness=1000
\linepenalty=10
\hyphenpenalty=50
\exhyphenpenalty=50
\binoppenalty=700
\relpenalty=500
\clubpenalty=150
\widowpenalty=150
\displaywidowpenalty=50
\brokenpenalty=100
\predisplaypenalty=10000
% \postdisplaypenalty=0
% \interlinepenalty=0
% \floatingpenalty=0, set during \insert
% \outputpenalty=0, set before TeX enters \output
\doublehyphendemerits=10000
\finalhyphendemerits=5000
\adjdemerits=10000
```

```
% \looseness=0, cleared by TeX after each paragraph
% \pausing=0
% \holdinginserts=0
% \tracingonline=0
% \tracingmacros=0
% \tracingstats=0
% \tracingparagraphs=0
% \tracingpages=0
% \tracingoutput=0
```

In the past LaTeX used the default value of `1` for `\tracinglostchars` because this was the best it could do. This way one would at least get a warning in the `.log` file. e-TeX improved on that and supported a value of `2` to show the warning on the terminal, so we could have changed the default when we made the e-TeX extensions required—however, we overlooked that opportunity. In 2021 this parameter was improved on again and now also accepts the value `3` (error on the terminal). This made us realize that we should change the default. Using `3` would really be the best, but for compatibility reasons we only use `2`. For recent LuaTeX releases, we set `4`: this is offset by 2 from the e-TeX value as otherwise LuaTeX only issues a warning for text in a box when the box is used, not saved. (Accepting values of 4 and 5 was updated in the TeX Live 2025 release.)

```
\tracinglostchars=2
\ifdefined\luatexversion
  \ifnum\luatexversion>120
    \tracinglostchars=4
  \fi
\fi
```

```
% \tracingcommands=0
% \tracingrestores=0
```

`\tracingstacklevels` For LuaTeX, the `\tracingstacklevels` functionality was implemented as a callback, so here we just define the count register to hold the value of the parameter.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2021/06/01}{\tracingstacklevels}%
⟨latexrelease⟩                    {tracingstacklevels}%
\ifx\directlua\@undefined
  % \tracingstacklevels=0 % added in 2021
\else
  \newcount\tracingstacklevels
  % Code for \tracingstacklevels defined in ltfinal.dtx
\fi
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\tracingstacklevels}%
⟨latexrelease⟩                    {tracingstacklevels}%
⟨latexrelease⟩\ifx\directlua\@undefined
⟨latexrelease⟩\else
⟨latexrelease⟩   \let\tracingstacklevels\@undefined
⟨latexrelease⟩\fi
⟨latexrelease⟩\EndIncludeInRelease
⟨/2ekernel | latexrelease⟩
⟨*2ekernel⟩
```

(*End of definition for* `\tracingstacklevels`.)

```
\uchyph=1
% \lefthyphenmin=2 \righthyphenmin=3 set below
% \globaldefs=0
% \maxdeadcycles=25 % INITEX does this
% \hangafter=1 % INITEX does this, also TeX after each paragraph
% \fam=0
% \mag=1000 % INITEX does this
% \escapechar=`\\ % INITEX does this
\defaulthyphenchar=`\-
\defaultskewchar=-1
% \endlinechar=`\^^M % INITEX does this
% \newlinechar=-1     \LaTeX\ sets this in ltdefns.dtx.
\delimiterfactor=901
% \time=now % TeX does this at beginning of job
% \day=now % TeX does this at beginning of job
% \month=now % TeX does this at beginning of job
% \year=now % TeX does this at beginning of job
```

In LaTeX we don't want box information in the transcript unless we do a full tracing.

```
\showboxbreadth=-1
\showboxdepth=-1
\errorcontextlines=-1
\hfuzz=0.1pt
\vfuzz=0.1pt
\overfullrule=5pt
\maxdepth=4pt
\splitmaxdepth=\maxdimen
\boxmaxdepth=\maxdimen
% \lineskiplimit=0pt, changed by \normalbaselines
\delimitershortfall=5pt
\nulldelimiterspace=1.2pt
\scriptspace=0.5pt
% \mathsurround=0pt
% \predisplaysize=0pt, set before TeX enters $$
% \displaywidth=0pt, set before TeX enters $$
% \displayindent=0pt, set before TeX enters $$
\parindent=20pt
% \hangindent=0pt, zeroed by TeX after each paragraph
% \hoffset=0pt
% \voffset=0pt
%
% \baselineskip=0pt, changed by \normalbaselines
% \lineskip=0pt, changed by \normalbaselines
\parskip=0pt plus 1pt
\abovedisplayskip=12pt plus 3pt minus 9pt
\abovedisplayshortskip=0pt plus 3pt
\belowdisplayskip=12pt plus 3pt minus 9pt
\belowdisplayshortskip=7pt plus 3pt minus 4pt
```

```
% \leftskip=0pt
% \rightskip=0pt
\topskip=10pt
\splittopskip=10pt
% \tabskip=0pt
% \spaceskip=0pt
% \xspaceskip=0pt
\parfillskip=0pt plus 1fil
```

\normalbaselineskip
\normallineskip
\normallineskiplimit

We also define special registers that function like parameters:

```
\newskip\normalbaselineskip \normalbaselineskip=12pt
\newskip\normallineskip \normallineskip=1pt
\newdimen\normallineskiplimit \normallineskiplimit=0pt
```

(*End of definition for* `\normalbaselineskip`, `\normallineskip`, *and* `\normallineskiplimit`.)

\interfootlinepenalty

```
\newcount\interfootnotelinepenalty \interfootnotelinepenalty=100
```

(*End of definition for* `\interfootlinepenalty`.)

Definitions for preloaded fonts

\magstephalf
\magstep

```
\def\magstephalf{1095 }
\def\magstep#1{\ifcase#1 \@m\or 1200\or 1440\or 1728\or
               2074\or 2488\fi\relax}
```

(*End of definition for* `\magstephalf` *and* `\magstep`.)

Macros for setting ordinary text

\frenchspacing
\nonfrenchspacing

```
\def\frenchspacing{\sfcode`\.\@m \sfcode`\?\@m \sfcode`\!\@m
  \sfcode`\:\@m \sfcode`\;\@m \sfcode`\,\@m}
\def\nonfrenchspacing{\sfcode`\.3000\sfcode`\?3000\sfcode`\!3000%
  \sfcode`\:2000\sfcode`\;1500\sfcode`\,1250 }
```

(*End of definition for* `\frenchspacing` *and* `\nonfrenchspacing`.)

\normalbaselines

```
\def\normalbaselines{\lineskip\normallineskip
  \baselineskip\normalbaselineskip \lineskiplimit\normallineskiplimit}
```

(*End of definition for* `\normalbaselines`.)

\M
\I

Save a bit of space by using `\let` here.

```
\def\^^M{\ } % control <return> = control <space>
\let\^^I\^^M % same for <tab>
```

(*End of definition for* `\M` *and* `\I`.)

\lq
\rq

```
\def\lq{`}
\def\rq{'}
```

(*End of definition for* `\lq` *and* `\rq`.)

`\lbrack`
`\rbrack`

```
\def\lbrack{[}
\def\rbrack{]}
```

(*End of definition for* `\lbrack` *and* `\rbrack`.)

`\aa`
`\AA`

These are not from plain.tex but they are similar to other commands found here and nowhere else, being alternate input forms for characters.

```
\def \aa {\r a}
\def \AA {\r A}
```

(*End of definition for* `\aa` *and* `\AA`.)

`\endgraf`
`\endline`

```
\let\endgraf=\par
\let\endline=\cr
```

(*End of definition for* `\endgraf` *and* `\endline`. *These functions are documented on page* 444.)

`\space`

```
\def\space{ }
```

(*End of definition for* `\space`.)

`\empty`

This probably ought to go altogether, but let it to the LaTeX version to save space.

```
\let\empty\@empty
```

(*End of definition for* `\empty`.)

`\null`

```
\def\null{\hbox{}}
```

(*End of definition for* `\null`.)

`\bgroup`
`\egroup`

```
\let\bgroup={
\let\egroup=}
```

(*End of definition for* `\bgroup` *and* `\egroup`.)

`\obeylines`
`\obeyspaces`

In `\obeylines`, we say `\let^^M=\obeyedline` instead of `\def^^M{\obeyedline}` since this allows, for example, `\let\obeyedline=\cr \obeylines \halign{...`.

This is essentially a plain TeX trick and in its original version where you had to use to use `\let\par=\cr` not really a safe idea in LaTeX. If anybody used this trick this now breaks (and one needs to use `\obeyedline` instead).

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2022/06/01}{\obeylines}%
⟨latexrelease⟩    {Add a redirection to obeylines and obeyspaces}%
```

If the active ^^M escapes, e.g., into a \write (which is effectively in a different context) then we don't want the definition from \obeylines but rather a simple \par (in fact even the primitive one, not the LaTeX version \para_end: which is only defined later).

```
\begingroup
\catcode`\^^M=\active % these lines must end with %
  \gdef\obeylines{\catcode`\^^M\active%
    \let^^M\obeyedline%
```

The next line ending the definition is rather curious and it took me awhile to understand why rollback fails. The problem is the following: if latexrelease is used, then blocks of \IncludeInRelease ... \EndIncludeInRelease are bypassed at high speed by grabbing each as a delimited argument. However, in that case ^^M is seen not as code but as line ending characters and in that mode TeX discards everything from that point onwards to the real end of the line so it works like a comment — pretty strange really (and I think due to the fact that the original pascal compiler could have some garbage showing up after the normal line ending character. Thus we really have to make sure that any closing braces is not one the same line as an ^^M, because otherwise it would get dropped and we end with unbalanced braces and never see the \EndIncludeInRelease — weird. In other places it doesn't matter because we aren't using the incomplete result.

```
  }%
  \global\let^^M\par % this is in case ^^M appears in a \write
\endgroup
```

The \obeyedline expands by default to \par with whatever definition \par has when it is executed. It can, however, be redefined (before calling \obeylines!) to achieve some special effects. If you want to alter this definition when already in the scope of \obeylines, it has no effect (because \let is used above). In that case simply make another call to \obeylines immediately. As you are in a restricted scope all that happens is that your redefinition is applied.

\obeyedline

For the default definition we have to use \def not \let because the meaning of \par can change and we want to use the one that is current when \obeylines act.

There is a small subtlety here: in an \edef the active ^^M stayed put (because it was equal to the primitive \par), now \obeyedline expands and you get what it contains, i.e., in that case \par, into the \edef or \mark unless we use \protected on it.

```
\protected\gdef\obeyedline{\par}
```

The definition of \obeyspaces is changed in the same way and now executes \obeyedspace for each active space.

\obeyedspace

```
\global\let\obeyedspace\space

\begingroup
\catcode`\ =\active%
\gdef\obeyspaces{\catcode`\ \active\let =\obeyedspace}%
```

An active space elsewhere generates \space by default (for example in a \write).

```
\global\let =\space%
\endgroup

⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\obeylines}%
⟨latexrelease⟩    {Add a redirection to obeylines and obeyspaces}%
⟨latexrelease⟩
```

From 2019 onwards the commands are made robust (somewhat later in the kernel sources). So if we roll back they are robust, so when redefining them we have get rid of the robust payload first. Otherwise that is seen by the later rollback below, which then installs a fragile version of the new definition on top of the one we roll back to here, sigh. `\kernel@make@fragile` also changes its definition (later own) so this is done directly.

```
⟨latexrelease⟩\expandafter\let\csname obeylines \endcsname\@undefined
⟨latexrelease⟩\expandafter\let\csname obeyspace \endcsname\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\begingroup
⟨latexrelease⟩\catcode`\^^M=\active % these lines must end with %
⟨latexrelease⟩  \gdef\obeylines{\catcode`\^^M\active \let^^M\par %
```

Closing brace on a separate line (see comment above).

```
⟨latexrelease⟩                    }%
```

Another pitfall: if we do a rollback `\par` is no longer the primitive, so the roll back definition needs `\let` to what is new the primitive.

```
⟨latexrelease⟩  \global\let^^M\RawParEnd % this is in case ^^M appears in a \write
⟨latexrelease⟩\endgroup
⟨latexrelease⟩\def\obeyspaces{\catcode`\ \active}
⟨latexrelease⟩
⟨latexrelease⟩\let\obeyedline\@undefined
⟨latexrelease⟩\let\obeyedspace\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\obeylines` *and others.*)

`\loop` `\iterate` `\repeat`

We use Kabelschacht's method of doing loops, see TUB 8#2 (1987). (unless that breaks something :-). It turned out to need an extra `\relax`: see pr/642 (`\loop` could do one iteration too much in certain cases).

```
\long\def \loop #1\repeat{%
  \def\iterate{#1\relax  % Extra \relax
               \expandafter\iterate\fi
               }%
  \iterate
  \let\iterate\relax
}
```

This setting of `\repeat` is needed to make `\loop...\if...\repeat` skippable within another `\if...`.

```
\let\repeat=\fi
```

(*End of definition for* `\loop`*,* `\iterate`*, and* `\repeat`*.*)

LaTeX defines `\smallskip`, etc. in `ltspace.dtx`.

`\nointerlineskip` `\offinterlineskip`

```
\def\nointerlineskip{\prevdepth-\@m\p@}
\def\offinterlineskip{\baselineskip-\@m\p@
  \lineskip\z@ \lineskiplimit\maxdimen}
```

(*End of definition for* `\nointerlineskip` *and* `\offinterlineskip`*.*)

\vglue
\hglue

```
\def\vglue{\afterassignment\vgl@\skip@=}
\def\vgl@{\par \dimen@\prevdepth \hrule \@height\z@
  \nobreak\vskip\skip@ \prevdepth\dimen@}
\def\hglue{\afterassignment\hgl@\skip@=}
\def\hgl@{\leavevmode \count@\spacefactor \vrule \@width\z@
  \nobreak\hskip\skip@ \spacefactor\count@}
```

(*End of definition for* `\vglue` *and* `\hglue`.)

LaTeX defines `~` in `ltdefns.dtx`.

`\slash` This generates a `/` acting a bit like `-` but still allows hyphenation in the word part preceding it (but not after).

```
\def\slash{/\penalty\exhyphenpenalty}
```

(*End of definition for* `\slash`.)

\break
\nobreak
\allowbreak

```
\def\break{\penalty-\@M}
\def\nobreak{\penalty \@M}
\def\allowbreak{\penalty \z@}
```

(*End of definition for* `\break`, `\nobreak`, *and* `\allowbreak`.)

\filbreak
\goodbreak

```
\def\filbreak{\par\vfil\penalty-200\vfilneg}
\def\goodbreak{\par\penalty-500 }
```

(*End of definition for* `\filbreak` *and* `\goodbreak`.)

`\eject` Define `\eject` as in plain TeX but define `\supereject` only in the compatibility file.

```
\def\eject{\par\break}
```

(*End of definition for* `\eject`.)

\removelastskip

```
\def\removelastskip{\ifdim\lastskip=\z@\else\vskip-\lastskip\fi}
```

(*End of definition for* `\removelastskip`.)

\smallbreak
\medbreak
\bigbreak

```
\def\smallbreak{\par\ifdim\lastskip<\smallskipamount
  \removelastskip\penalty-50\smallskip\fi}
\def\medbreak{\par\ifdim\lastskip<\medskipamount
  \removelastskip\penalty-100\medskip\fi}
\def\bigbreak{\par\ifdim\lastskip<\bigskipamount
  \removelastskip\penalty-200\bigskip\fi}
```

(*End of definition for* `\smallbreak`, `\medbreak`, *and* `\bigbreak`.)

\m@th

```
\def\m@th{\mathsurround\z@}
```

(*End of definition for* `\m@th`.)

\underbar Due to LaTeX's redefinition of \underline plain TeX's \underbar can be done in a simpler fashion (but do we need it at all?).

```
\def\underbar#1{\underline{\sbox\tw@{#1}\dp\tw@\z@\box\tw@}}
```

(*End of definition for* \underbar.)

\strutbox \strut LaTeX sets \strutbox in \set@fontsize.

```
\newbox\strutbox
\def\strut{\relax\ifmmode\copy\strutbox\else\unhcopy\strutbox\fi}
```

(*End of definition for* \strutbox *and* \strut.)

\hidewidth For alignment entries that can stick out.

```
\def\hidewidth{\hskip\hideskip}
```

(*End of definition for* \hidewidth.)

\narrower

```
\def\narrower{%
  \advance\leftskip\parindent
  \advance\rightskip\parindent}
```

(*End of definition for* \narrower.)

LaTeX defines \ae and similar commands elsewhere.

```
\chardef\%=`\%
\chardef\&=`\&
\chardef\#=`\#
```

Most text commands are actually encoding specific and therefore defined later, so commented out or removed from this file.

\leavevmode begins a paragraph, if necessary

```
\def\leavevmode{\unhbox\voidb@x}
```

(*End of definition for* \leavevmode.)

\mathhexbox

```
\def\mathhexbox#1#2#3{\mbox{$\m@th \mathchar"#1#2#3$}}
```

(*End of definition for* \mathhexbox.)

\ialign

```
\def\ialign{\everycr{}\tabskip\z@skip\halign} % initialized \halign
```

(*End of definition for* \ialign.)

\oalign \o@lign \ooalign

```
\def\oalign#1{\leavevmode\vtop{\baselineskip\z@skip \lineskip.25ex%
  \ialign{##\crcr#1\crcr}}}
\def\o@lign{\lineskiplimit\z@ \oalign}
\def\ooalign{\lineskiplimit-\maxdimen \oalign}
```

(*End of definition for* \oalign, \o@lign, *and* \ooalign.)

\sh@ft The definition of this macro in plain.tex was improved in about 1997; but as a result its usage was changed and its new definition is not appropriate for LaTeX.

Since the version given here has been in use by LaTeX for many years it does not seem prudent to remove it now. As far as we can tell it has only been used to define \b and \d but this cannot be certain.

```
\def\sh@ft#1{\dimen@.00#1ex\multiply\dimen@\fontdimen1\font
  \kern-.0156\dimen@} % compensate for slant in lowered accents
```

(*End of definition for* \sh@ft.)

\ltx@sh@ft This is the LaTeX version of the second incarnation of the plain macro \sh@ft, which takes a dimension as its argument. It shifts a pseudo-accent horizontally by an amount proportional to the product of its argument and the slant-per-point (fontdimen 1).

```
\def\ltx@sh@ft #1{%
  \dimen@ #1%
  \kern \strip@pt
    \fontdimen1\font \dimen@
  } % kern by #1 times the current slant
```

(*End of definition for* \ltx@sh@ft.)

LaTeX change: the text commands such as \d, \b, \c, \copyright, \TeX are now defined elsewhere.

LaTeX change: Make \t work in a moving argument. Now defined elsewhere.

\hrulefill \dotfill LaTeX change: \kern\z@ added to end of \hrulefill and \dotfill to make them work in 'tabular' and 'array' environments. (Change made 24 July 1987). LaTeX change: \leavevmode added at beginning of \dotfill and \hrulefill so that they work as expected in vertical mode.

```
\def\hrulefill{\leavevmode\leaders\hrule\hfill\kern\z@}
```

The box in \dotfill originally contained (in plain.tex):
\mkern 1.5mu .\mkern 1.5mu;
the width of .44em differs from this by .04pt which is probably an acceptable difference within leaders.

```
\def\dotfill{%
  \leavevmode
  \cleaders \hb@xt@ .44em{\hss.\hss}\hfill
  \kern\z@}
```

(*End of definition for* \hrulefill *and* \dotfill.)

INITEX sets \sfcode x=1000 for all x, except that \sfcode`X=999 for uppercase letters. The following changes are needed:

```
\sfcode`\)=0 \sfcode`\'=0 \sfcode`\]=0
```

The \nonfrenchspacing macro will make further changes to \sfcode values.

Definitions related to output
\magnification doesn't work in LaTeX.

```
def\magnification{\afterassignment\m@g\count@}
def\m@g{\mag\count@
 \hsize6.5truein\vsize8.9truein\dimen\footins8truein}
```

\showoverfull The following commands are used in debugging:

```
\def\showoverfull{\tracingonline\@ne}
```

(*End of definition for* `\showoverfull`.)

`\showoutput`
`\loggingoutput`

```
\gdef\loggingoutput{\tracingoutput\@ne
    \showboxbreadth\maxdimen\showboxdepth\maxdimen\errorstopmode}
\gdef\showoutput{\loggingoutput\showoverfull}
⟨/2ekernel⟩
```

(*End of definition for* `\showoutput` *and* `\loggingoutput`.)

`\tracingall`
`\loggingall`

```
⟨latexrelease⟩\IncludeInRelease{2021/06/01}{\loggingall}
⟨latexrelease⟩                   {\tracingstacklevels and \tracinglostchars=3}%
⟨*2ekernel | latexrelease⟩
\edef\loggingall{%
  \tracingstats\tw@
  \tracingpages\@ne
  \tracingparagraphs\@ne
  \tracinggroups\@ne
  \tracingifs\@ne
  \tracingscantokens\@ne
  \tracingnesting\@ne
  \errorcontextlines\maxdimen
  \ifdefined\tracingstacklevels \tracingstacklevels\maxdimen \fi
  \noexpand \loggingoutput
  \tracingmacros\tw@
  \tracingcommands\thr@@
  \tracingrestores\@ne
  \tracingassigns\@ne
}%
\def\tracingall{\showoverfull\loggingall}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}{\loggingall}{etex tracing}%
⟨latexrelease⟩\ifx\tracingscantokens\@undefined
⟨latexrelease⟩\gdef\loggingall{%
⟨latexrelease⟩   \tracingstats\tw@
⟨latexrelease⟩   \tracingpages\@ne
⟨latexrelease⟩   \tracinglostchars\@ne
⟨latexrelease⟩   \tracingparagraphs\@ne
⟨latexrelease⟩   \errorcontextlines\maxdimen
⟨latexrelease⟩   \loggingoutput
⟨latexrelease⟩   \tracingmacros\tw@
⟨latexrelease⟩   \tracingcommands\tw@
⟨latexrelease⟩   \tracingrestores\@ne
⟨latexrelease⟩   }%
⟨latexrelease⟩\else
⟨latexrelease⟩\gdef\loggingall{%
⟨latexrelease⟩   \tracingstats\tw@
⟨latexrelease⟩   \tracingpages\@ne
⟨latexrelease⟩   \tracinglostchars\tw@
⟨latexrelease⟩   \tracingparagraphs\@ne
⟨latexrelease⟩   \tracinggroups\@ne
```

```
⟨latexrelease⟩   \tracingifs\@ne
⟨latexrelease⟩   \tracingscantokens\@ne
⟨latexrelease⟩   \tracingnesting\@ne
⟨latexrelease⟩   \errorcontextlines\maxdimen
⟨latexrelease⟩   \loggingoutput
⟨latexrelease⟩   \tracingmacros\tw@
⟨latexrelease⟩   \tracingcommands\thr@@
⟨latexrelease⟩   \tracingrestores\@ne
⟨latexrelease⟩   \tracingassigns\@ne
⟨latexrelease⟩}%
⟨latexrelease⟩\fi
⟨latexrelease⟩\gdef\tracingall{\showoverfull\loggingall}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\loggingall}{etex tracing}%
⟨latexrelease⟩\gdef\loggingall{\tracingcommands\tw@\tracingstats\tw@
⟨latexrelease⟩   \tracingpages\@ne\tracinglostchars\@ne
⟨latexrelease⟩   \tracingmacros\tw@\tracingparagraphs\@ne\tracingrestores\@ne
⟨latexrelease⟩   \errorcontextlines\maxdimen\loggingoutput}
⟨latexrelease⟩   \gdef\tracingall{\loggingall\showoverfull}
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\tracingall` *and* `\loggingall`.)

`\tracingnone`

```
⟨latexrelease⟩\IncludeInRelease{2015/01/01}{\tracingnone}%
⟨latexrelease⟩                                  {turn off etex tracing}%
⟨∗2ekernel | latexrelease⟩
\edef\tracingnone{%
  \tracingassigns\z@
  \tracingrestores\z@
  \tracingonline\z@
  \tracingcommands\z@
  \showboxdepth\m@ne
  \showboxbreadth\m@ne
  \tracingoutput\z@
  \errorcontextlines\m@ne
  \ifdefined\tracingstacklevels \tracingstacklevels\z@ \fi
  \tracingnesting\z@
  \tracingscantokens\z@
  \tracingifs\z@
  \tracinggroups\z@
  \tracingparagraphs\z@
  \tracingmacros\z@
  \tracingpages\z@
  \tracingstats\z@
}%
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}{\tracingnone}%
⟨latexrelease⟩                                  {turn off etex tracing}%
⟨latexrelease⟩\ifx\tracingscantokens\@undefined
⟨latexrelease⟩\def\tracingnone{%
```

```
⟨latexrelease⟩   \tracingonline\z@
⟨latexrelease⟩   \tracingcommands\z@
⟨latexrelease⟩   \showboxdepth\m@ne
⟨latexrelease⟩   \showboxbreadth\m@ne
⟨latexrelease⟩   \tracingoutput\z@
⟨latexrelease⟩   \errorcontextlines\m@ne
⟨latexrelease⟩   \tracingrestores\z@
⟨latexrelease⟩   \tracingparagraphs\z@
⟨latexrelease⟩   \tracingmacros\z@
⟨latexrelease⟩   \tracinglostchars\@ne
⟨latexrelease⟩   \tracingpages\z@
⟨latexrelease⟩   \tracingstats\z@
⟨latexrelease⟩}%
⟨latexrelease⟩\else
⟨latexrelease⟩\def\tracingnone{%
⟨latexrelease⟩   \tracingassigns\z@
⟨latexrelease⟩   \tracingrestores\z@
⟨latexrelease⟩   \tracingonline\z@
⟨latexrelease⟩   \tracingcommands\z@
⟨latexrelease⟩   \showboxdepth\m@ne
⟨latexrelease⟩   \showboxbreadth\m@ne
⟨latexrelease⟩   \tracingoutput\z@
⟨latexrelease⟩   \errorcontextlines\m@ne
⟨latexrelease⟩   \tracingnesting\z@
⟨latexrelease⟩   \tracingscantokens\z@
⟨latexrelease⟩   \tracingifs\z@
⟨latexrelease⟩   \tracinggroups\z@
⟨latexrelease⟩   \tracingparagraphs\z@
⟨latexrelease⟩   \tracingmacros\z@
⟨latexrelease⟩   \tracinglostchars\@ne
⟨latexrelease⟩   \tracingpages\z@
⟨latexrelease⟩   \tracingstats\z@
⟨latexrelease⟩}%
⟨latexrelease⟩\fi
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\tracingnone}%
⟨latexrelease⟩                                {turn off etex tracing}%
⟨latexrelease⟩\let\tracingnone\@undefined
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\tracingnone`.)

`\hideoutput`

```
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}{\hideoutput}%
⟨latexrelease⟩                                {hide output from tracing}%
\def\hideoutput{%
  \tracingoutput\z@
  \showboxbreadth\m@ne
  \showboxdepth\m@ne
  \tracingonline\m@ne
}%
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\hideoutput}%
⟨latexrelease⟩                                  {hide output from tracing}%
⟨latexrelease⟩\let\hideoutput\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨/2ekernel | latexrelease⟩
```

(*End of definition for* `\hideoutput`.)

LaTeX change: `\showhyphens` Defined later.

Punctuation affects the spacing.

```
⟨*2ekernel⟩
\nonfrenchspacing
⟨/2ekernel⟩
```

File 03
ltvers.dtx

1 Version Identification

First we identify the date and version number of this release of LaTeX, and set `\everyjob` so that it is printed at the start of every LaTeX run.

`\fmtname` `\fmtversion` `\latexreleaseversion` `\patch@level`

A `\patch@level` of `0` or higher denotes an official public release. A negative value indicates a candidate release that is not distributed.

If we put code updates into the kernel that are supposed to go into the next release we set the `\patch@level` to `-1` and the `\fmtversion` / `\latexreleaseversion` to the dated of the next release (guessed, the real value is not so important and will get corrected when we make the release official).

If the `\patch@level` is already at `-1` we do nothing here and use the `\fmtversion` date for any new`\IncludeInRelease` line when we add further code.

Finally, if we do make a public release we either just set the `\patch@level` to zero (if our initial guess was good) or we also change the date and then have to additionally change to that date on all the `\IncludeInRelease` statements that used the "guessed" date.

```
⟨*2ekernel⟩
\def\fmtname{LaTeX2e}
\edef\fmtversion
⟨/2ekernel⟩
⟨latexrelease⟩\edef\latexreleaseversion
⟨*2ekernel | latexrelease⟩
   {2026-11-01}
⟨/2ekernel | latexrelease⟩
⟨*2ekernel⟩
\def\patch@level{0}
```

For more fine grain control there is the possibility to name the current development branch. This is only used when the `\patch@level` is negative (i.e., a pre-release format) and is intended to help us internally when we locally install a format out of some development branch.

`\development@branch@name`

```
\edef\development@branch@name{}
```

(*End of definition for* `\fmtname` *and others.*)

Check that the format being made is not too old. The error message complains about 'more than 5 years' but in fact the error is not triggered until 65 months.

This code is currently not activated as we don't know if we already got to the last official 2e version (due to staff shortage or due to a successor (think positive:-)).

```
\iffalse
\def\reserved@a#1/#2/#3\@nil{%
  \count@\year
  \advance\count@-#1\relax
  \multiply\count@ by 12\relax
  \advance\count@\month
  \advance\count@-#2\relax}
\expandafter\reserved@a\fmtversion\@nil
```

`\count@` is now the age of this file in months. Take a generous definition of 'year' so this message is not generated too often.

```
\ifnum\count@>65
  \typeout{^^J%
!!!
```

```
    \fi
  \fi
⟨/2ekernel⟩
```

\IncludeInRelease
\EndIncludeInRelease
\@IncludeInRelease
\@IncludeInRele@se
\@gobble@IncludeInRelease
\@check@IncludeInRelease

```
⟨2ekernel⟩\let\@currname\@empty

⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\newif\if@includeinrelease
⟨latexrelease⟩\@includeinreleasefalse

\def\IncludeInRelease#1{%
 \if@includeinrelease
   \PackageError{latexrelease}{mis-matched IncludeInRelease}%
                {There is an \noexpand\EndIncludeInRelease missing}%
  \@includeinreleasefalse
  \fi
  \ifnum0%
      \ifx\new@moduledate\@empty\else 1\fi
      \ifnum \expandafter\@parse@version#1//00\@nil=0 1\fi
      =11
    \expandafter\@firstoftwo
  \else
    \expandafter\@secondoftwo
  \fi
    {\finish@module@release{#1}}%
    {\kernel@ifnextchar[%
      {\@IncludeInRelease{#1}}
      {\@IncludeInRelease{#1}[#1]}}}
\def\finish@module@release#1#2#3{%
  \toks@{[#1] #3}%
  \begingroup
    \edef\x{\detokenize\expandafter{\new@modulename}}%
    \edef\y{\detokenize{#2}}%
  \expandafter\endgroup
  \ifx\x\y \else
    \@latex@error{\noexpand\IncludeInRelease dated #1 in a module is not
      allowed.\MessageBreak Use a date at least equal to \new@moduledate
      \space for complete rollback}\@ehd
  \fi
  \ifnum\expandafter\@parse@version\new@moduledate//00\@nil
       >\expandafter\@parse@version\fmtversion//00\@nil
    \GenericInfo{}{Applying: \the\toks@}%
  \else
    \GenericInfo{}{Skipping: \the\toks@}%
    \expandafter\gobble@finish@module@release
  \fi}
\long\def\gobble@finish@module@release#1\EndModuleRelease{%
  \EndModuleRelease}
```

If a specific date has not been specified in `latexrelease` use '`#1`'.

```
\def\@IncludeInRelease#1[#2]{\@IncludeInRele@se{#2}}

\def\@IncludeInRele@se#1#2#3{%
  \toks@{[#1] #3}%
  \expandafter\ifx\csname\string#2+\@currname+IIR\endcsname\relax
```

If we roll back and the first patch already match then applying that is actually reapplying what is already in the format, i.e., it is useless and possibly allocating new registers. However, it makes the logic simpler so this is the way it is for now. In theory we could always jump over the first patch because that is only really needed for rolling forward. So maybe one day ...

```
    \ifnum\expandafter\@parse@version#1//00\@nil
         >\expandafter\@parse@version\fmtversion//00\@nil
      \GenericInfo{}{Skipping: \the\toks@}%
      \expandafter\expandafter\expandafter\@gobble@IncludeInRelease
    \else
      \GenericInfo{}{Applying: \the\toks@}%
      \@includeinreleasetrue
      \expandafter\let\csname\string#2+\@currname+IIR\endcsname\@empty
    \fi
  \else
    \GenericInfo{}{Already applied: \the\toks@}%
    \expandafter\@gobble@IncludeInRelease
  \fi
}
\def\EndIncludeInRelease{%
\if@includeinrelease
  \@includeinreleasefalse
\else
  \PackageError{latexrelease}{mis-matched EndIncludeInRelease}{}%
\fi
\if@skipping@module
  \expandafter\new@module@skip
\fi}
\long\def\@gobble@IncludeInRelease#1\EndIncludeInRelease{%
  \@includeinreleasefalse
  \@check@IncludeInRelease#1\IncludeInRelease\@check@IncludeInRelease
  \@end@check@IncludeInRelease}
\long\def\@check@IncludeInRelease#1\IncludeInRelease
                                    #2#3\@end@check@IncludeInRelease{%
  \ifx\@check@IncludeInRelease#2\else
    \PackageError{latexrelease}{skipped IncludeInRelease for tag \string#2}{}%
  \fi
  \if@skipping@module
    \expandafter\new@module@skip
  \fi}
```

(*End of definition for* `\IncludeInRelease` *and others.*)

1.1 Declaring an all-new module

`\if@skipping@module`
`\NewModuleRelease`
`\EndModuleRelease`
`\new@module@skip`
`\new@modulename`
`\new@moduledate`

When we have a whole new module, we can't roll back to a date where such module exists, otherwise hundreds of "command already defined" errors will pop up. But we can't skip it altogether either, because the module might have changes we still want applied, so a more detailed cherry-picking of code chunks have to be done.

```
\let\if@skipping@module\iffalse
\def\@skipping@moduletrue{\let\if@skipping@module\iftrue}
\def\@skipping@modulefalse{\let\if@skipping@module\iffalse}
```

```
\let\new@modulename\@empty
\let\new@moduledate\@empty
\def\NewModuleRelease#1#2#3{%
  \ifx\new@modulename\@empty \else
    \@latex@error{Nested \noexpand\NewModuleRelease forbidden.}\@ehd \fi
  \edef\new@moduledate{#1}%
  \edef\new@modulename{#2}%
  \GenericInfo{}{BEGIN module: \new@modulename\space (\new@moduledate)}%
  \GenericInfo{}{ \@spaces\@spaces\@spaces\space#3\@gobble}%
  \ifnum\sourceLaTeXdate<%
        \expandafter\@parse@version\new@moduledate//00\@nil\relax
    \ifnum\expandafter\@parse@version\fmtversion//00\@nil<%
          \expandafter\@parse@version\new@moduledate//00\@nil\relax
      \GenericInfo{}{Skipping module \new@modulename}%
      \expandafter\expandafter
      \expandafter\gobble@finish@module@release
    \else
      \GenericInfo{}{Applying module \new@modulename}
      \@skipping@modulefalse
    \fi
  \else
    \GenericInfo{}{Skipping module \new@modulename}
    \@skipping@moduletrue
    \expandafter\new@module@skip
  \fi}
\long\def\new@module@skip#1\IncludeInRelease{%
  \long\def\reserved@a##1\EndModuleRelease{}%
  \if\relax\detokenize\expandafter{\reserved@a#1{}{}\EndModuleRelease}\relax
  \else
    \@latex@error{Missing mandatory \string\IncludeInRelease{0000/00/00}}\@ehc
    \expandafter\@secondoftwo
  \fi
  \@gobble
    {\@expandtwoargs\IncludeInRelease
         {0000/00/00}{\new@modulename}%
         {ERROR! Emergency recovery}%
     #1}%
  \IncludeInRelease}
\def\EndModuleRelease{%
  \ifx\new@modulename\@empty
    \@latex@error{Extra \string\EndModuleRelease.}\@eha
  \else
    \GenericInfo{}{END module: \new@modulename\space (\new@moduledate)}%
    \let\new@modulename\@empty
    \let\new@moduledate\@empty
    \@skipping@modulefalse
  \fi}
```

(*End of definition for* `\if@skipping@module` *and others.*)

⟨/2ekernel | latexrelease⟩

File 04
ltluatex.dtx

1 Overview

LuaTEX adds a number of engine-specific functions to TEX. Several of these require set up that is best done in the kernel or need related support functions. This file provides *basic* support for LuaTEX at the LATEX 2_ε kernel level plus as a loadable file which can be used with plain TEX and LATEX.

This file contains code for both TEX (to be stored as part of the format) and Lua (to be loaded at the start of each job). In the Lua code, the kernel uses the namespace `luatexbase`.

The following `\count` registers are used here for register allocation:

`\e@alloc@attribute@count` Attributes (default 258)

`\e@alloc@ccodetable@count` Category code tables (default 259)

`\e@alloc@luafunction@count` Lua functions (default 260)

`\e@alloc@whatsit@count` User whatsits (default 261)

`\e@alloc@bytecode@count` Lua bytecodes (default 262)

`\e@alloc@luachunk@count` Lua chunks (default 263)

(`\count 256` is used for `\newmarks` allocation and `\count 257` is used for `\newXeTeXintercharclass` with XeTEX, with code defined in `ltfinal.dtx`). With any LATEX 2_ε kernel from 2015 onward these registers are part of the block in the extended area reserved by the kernel (prior to 2015 the LATEX 2_ε kernel did not provide any functionality for the extended allocation area).

2 Core TEX functionality

The commands defined here are defined for possible inclusion in a future LATEX format, however also extracted to the file `ltluatex.tex` which may be used with older LATEX formats, and with plain TEX.

`\newattribute` `\newattribute{`⟨*attribute*⟩`}`
Defines a named `\attribute`, indexed from 1 (*i.e.* `\attribute0` is never defined). Attributes initially have the marker value `-"7FFFFFFF` ('unset') set by the engine.

`\newcatcodetable` `\newcatcodetable{`⟨*catcodetable*⟩`}`
Defines a named `\catcodetable`, indexed from 1 (`\catcodetable0` is never assigned). A new catcode table will be populated with exactly those values assigned by IniTEX (as described in the LuaTEX manual).

`\newluafunction` `\newluafunction{`⟨*function*⟩`}`
Defines a named `\luafunction`, indexed from 1. (Lua indexes tables from 1 so `\luafunction0` is not available).

`\newluacmd` `\newluadef{`⟨*function*⟩`}`
Like `\newluafunction`, but defines the command using `\luadef` instead of just assigning an integer.

\newprotectedluacmd `\newluadef{⟨function⟩}`
Like `\newluacmd`, but the defined command is not expandable.

\newwhatsit `\newwhatsit{⟨whatsit⟩}`
Defines a custom `\whatsit`, indexed from 1.

\newluabytecode `\newluabytecode{⟨bytecode⟩}`
Allocates a number for Lua bytecode register, indexed from 1.

\newluachunkname `newluachunkname{⟨chunkname⟩}`
Allocates a number for Lua chunk register, indexed from 1. Also enters the name of the register (without backslash) into the `lua.name` table to be used in stack traces.

\catcodetable@initex
\catcodetable@string
\catcodetable@latex
\catcodetable@atletter
\setattribute
\unsetattribute
Predefined category code tables with the obvious assignments. Note that the `latex` and `atletter` tables set the full Unicode range to the codes predefined by the kernel.

`\setattribute{⟨attribute⟩}{⟨value⟩}`
`\unsetattribute{⟨attribute⟩}`
Set and unset attributes in a manner analogous to `\setlength`. Note that attributes take a marker value when unset so this operation is distinct from setting the value to zero.

3 Plain TEX interface

The ltluatex interface may be used with plain TEX using `\input{ltluatex}`. This inputs `ltluatex.tex` which inputs `etex.src` (or `etex.sty` if used with LATEX) if it is not already input, and then defines some internal commands to allow the ltluatex interface to be defined.

The luatexbase package interface may also be used in plain TEX, as before, by inputting the package `\input luatexbase.sty`. The new version of luatexbase is based on this ltluatex code but implements a compatibility layer providing the interface of the original package.

4 Lua functionality

4.1 Allocators in Lua

new_attribute `luatexbase.new_attribute(⟨attribute⟩)`
Returns an allocation number for the ⟨*attribute*⟩, indexed from 1. The attribute will be initialized with the marker value `-"7FFFFFFF` ('unset'). The attribute allocation sequence is shared with the TEX code but this function does *not* define a token using `\attributedef`. The attribute name is recorded in the `attributes` table. A metatable is provided so that the table syntax can be used consistently for attributes declared in TEX or Lua.

new_whatsit `luatexbase.new_whatsit(⟨whatsit⟩)`
Returns an allocation number for the custom ⟨*whatsit*⟩, indexed from 1.

new_bytecode `luatexbase.new_bytecode(⟨bytecode⟩)`
Returns an allocation number for a bytecode register, indexed from 1. The optional ⟨*name*⟩ argument is just used for logging.

new_chunkname `luatexbase.new_chunkname(⟨chunkname⟩)`
Returns an allocation number for a Lua chunk name for use with `\directlua` and `\latelua`, indexed from 1. The number is returned and also ⟨*name*⟩ argument is added to the `lua.name` array at that index.

new_luafunction `luatexbase.new_luafunction(⟨functionname⟩)`

Returns an allocation number for a lua function for use with `\luafunction`, `\lateluafunction`, and `\luadef`, indexed from 1. The optional ⟨*functionname*⟩ argument is just used for logging.

new_luacommand `luatexbase.new_luacommand(`⟨*csname*⟩, ⟨*fn*⟩, ⟨*...*⟩`)`
Convenience wrapper combining `new_luafunction` with `token.set_lua`. Allocates a new lua function slot for ⟨*csname*⟩ (as `new_luafunction` does), stores the Lua function ⟨*fn*⟩ in it, then calls `token.set_lua(`⟨*csname*⟩, ⟨*fnalloc*⟩, ⟨*...*⟩`)` to bind ⟨*csname*⟩ to that slot. Any further ⟨*...*⟩ arguments are passed on unchanged to `token.set_lua`.

These functions all require access to a named TEX count register to manage their allocations. The standard names are those defined above for access from TEX, *e.g.* \e@alloc@attribute@count, but these can be adjusted by defining the variable ⟨*type*⟩`_count_name` before loading `ltluatex.lua`, for example

```
local attribute_count_name = "attributetracker"
require("ltluatex")
```

would use a TEX `\count` (`\countdef`'d token) called `attributetracker` in place of \e@alloc@attribute@count.

4.2 Lua access to TEX register numbers

registernumber `luatexbase.registernumer(`⟨*name*⟩`)`
Sometimes (notably in the case of Lua attributes) it is necessary to access a register *by number* that has been allocated by TEX. This package provides a function to look up the relevant number using LuaTEX's internal tables. After for example `\newattribute\myattrib`, `\myattrib` would be defined by (say) `\myattrib=\attribute15`. `luatexbase.registernumer("myattrib")` would then return the register number, 15 in this case. If the string passed as argument does not correspond to a token defined by `\attributedef`, `\countdef` or similar commands, the Lua value `false` is returned.

As an example, consider the input:

```
\newcommand\test[1]{%
\typeout{#1: \expandafter\meaning\csname#1\endcsname^^J
\space\space\space\space
\directlua{tex.write(luatexbase.registernumber("#1") or "bad input")}%
}}

\test{undefinedrubbish}

\test{space}

\test{hbox}

\test{@MM}

\test{@tempdima}
\test{@tempdimb}

\test{strutbox}

\test{sixt@@n}
```

```
\attrbutedef\myattr=12
\myattr=200
\test{myattr}
```

If the demonstration code is processed with LuaLaTeX then the following would be produced in the log and terminal output.

```
undefinedrubbish: \relax
     bad input
space: macro:->
     bad input
hbox: \hbox
     bad input
@MM: \mathchar"4E20
     20000
@tempdima: \dimen14
     14
@tempdimb: \dimen15
     15
strutbox: \char"B
     11
sixt@@n: \char"10
     16
myattr: \attribute12
     12
```

Notice how undefined commands, or commands unrelated to registers do not produce an error, just return `false` and so print `bad input` here. Note also that commands defined by `\newbox` work and return the number of the box register even though the actual command holding this number is a `\chardef` defined token (there is no `\boxdef`).

4.3 Module utilities

`provides_module` `luatexbase.provides_module(`⟨*info*⟩`)`
This function is used by modules to identify themselves; the `info` should be a table containing information about the module. The required field `name` must contain the name of the module. It is recommended to provide a field `date` in the usual LaTeX format `yyyy/mm/dd`. Optional fields `version` (a string) and `description` may be used if present. This information will be recorded in the log. Other fields are ignored. If the `version` begins with a digit, a `v` will be added at the start in the log.

`module_info` `luatexbase.module_info(`⟨*module*⟩, ⟨*text*⟩`)`
`module_warning` `luatexbase.module_warning(`⟨*module*⟩, ⟨*text*⟩`)`
`module_error` `luatexbase.module_error(`⟨*module*⟩, ⟨*text*⟩`)`
These functions are similar to LaTeX's `\PackageError`, `\PackageWarning` and `\PackageInfo` in the way they format the output. No automatic line breaking is done, you may still use `\n` as usual for that, and the name of the package will be prepended to each output line.

Note that `luatexbase.module_error` raises an actual Lua error with `error()`, which currently means a call stack will be dumped. While this may not look pretty, at least it provides useful information for tracking the error down.

4.4 Callback management

`add_to_callback` `luatexbase.add_to_callback(`⟨*callback*⟩, ⟨*function*⟩, ⟨*identifier*⟩`)` Registers the ⟨*function*⟩ into the ⟨*callback*⟩ with a textual ⟨*identifier*⟩ of the function. Functions are inserted into the callback in the order loaded. The ⟨*identifier*⟩ should be unique for each callback.

`remove_from_callback` `luatexbase.remove_from_callback(`⟨*callback*⟩, ⟨*identifier*⟩`)` Removes the callback function with ⟨*identifier*⟩ from the ⟨*callback*⟩. The removed function and its identifier are returned as the results of this function.

`in_callback` `luatexbase.in_callback(`⟨*callback*⟩, ⟨*identifier*⟩`)` Checks if the ⟨*identifier*⟩ matches one of the functions added to the list for the ⟨*callback*⟩, returning a boolean value.

`disable_callback` `luatexbase.disable_callback(`⟨*callback*⟩`)` Sets the ⟨*callback*⟩ to `false` as described in the LuaTEX manual for the underlying `callback.register` built-in. Callbacks will only be set to false (and thus be skipped entirely) if there are no functions registered using the callback.

`callback_identifiers` A list of the identifiers of functions registered to the specified callback is returned. `{}` is returned if there are no functions registered.

`callback_descriptions` This is a historical alias for `callback_identifiers`.

`create_callback` `luatexbase.create_callback(`⟨*name*⟩,⟨*type*⟩,⟨*default*⟩`)` Defines a user defined callback. The last argument is a default function or `false`.

`call_callback` `luatexbase.call_callback(`⟨*name*⟩,...`)` Calls a user defined callback with the supplied arguments.

`declare_callback_rule` `luatexbase.declare_callback_rule(`⟨*name*⟩, ⟨*first*⟩, ⟨*relation*⟩, ⟨*second*⟩`)` Adds an ordering constraint between two callback functions for callback ⟨*name*⟩.

The kind of constraint added depends on ⟨*relation*⟩:

before The callback function with identifier ⟨*first*⟩ will be executed before the function with identifier ⟨*second*⟩.

after The callback function with identifier ⟨*first*⟩ will be executed after the function with identifier ⟨*second*⟩.

incompatible-warning When both a callback function with identifier ⟨*first*⟩ and with identifier ⟨*second*⟩ is registered, then a warning is printed when the callback is executed.

incompatible-error When both a callback function with identifier ⟨*first*⟩ and with identifier ⟨*second*⟩ is registered, then an error is printed when the callback is executed.

unrelated Any previously declared callback rule between ⟨*first*⟩ and ⟨*second*⟩ gets disabled.

Every call to `declare_callback_rule` with a specific callback ⟨*name*⟩ and identifier ⟨*first*⟩ and ⟨*second*⟩ overwrites all previous calls with same callback and identifiers.

The callback functions do not have to be registered yet when the functions is called. Only the constraints for which both callback identifiers refer to callbacks registered at the time the callback is called will have an effect.

5 Implementation

```
⟨*2ekernel | tex | latexrelease⟩
⟨2ekernel | latexrelease⟩\ifx\directlua\@undefined\else
```

5.1 Minimum LuaTEX version

LuaTEX has changed a lot over time. In the kernel support for ancient versions is not provided: trying to build a format with a very old binary therefore gives some information in the log and loading stops. The cut-off selected here relates to the tree-searching behaviour of `require()`: from version 0.60, LuaTEX will correctly find Lua files in the `texmf` tree without 'help'.

```
⟨latexrelease⟩\IncludeInRelease{2015/10/01}
⟨latexrelease⟩                   {\newluafunction}{LuaTeX}%
\ifnum\luatexversion<60 %
  \wlog{===============================================}
  \wlog{* LuaTeX version too old for ltluatex support *}
  \wlog{===============================================}
  \expandafter\endinput
\fi
```

Two simple LaTeX macros from `ltdefns.dtx` have to be defined here because ltdefns.dtx is not loaded yet when ltluatex.dtx is executed.

```
\long\def\@gobble#1{}
\long\def\@firstofone#1{#1}
```

5.2 Older LaTeX/Plain TeX setup

```
⟨*tex⟩
```

Older LaTeX formats don't have the primitives with 'native' names: sort that out. If they already exist this will still be safe.

```
\directlua{tex.enableprimitives("",tex.extraprimitives("luatex"))}
\ifx\e@alloc\@undefined
```

In pre-2014 LaTeX, or plain TeX, load `etex.{sty,src}`.

```
  \ifx\documentclass\@undefined
    \ifx\loccount\@undefined
      \input{etex.src}%
    \fi
    \catcode`\@=11 %
    \outer\expandafter\def\csname newfam\endcsname
                           {\alloc@8\fam\chardef\et@xmaxfam}
  \else
    \RequirePackage{etex}
    \expandafter\def\csname newfam\endcsname
                   {\alloc@8\fam\chardef\et@xmaxfam}
    \expandafter\let\expandafter\new@mathgroup\csname newfam\endcsname
  \fi
```

5.2.1 Fixes to `etex.src/etex.sty`

These could and probably should be made directly in an update to `etex.src` which already has some LuaTEX-specific code, but does not define the correct range for LuaTEX.

2015-07-13 higher range in luatex.

```
\edef \et@xmaxregs {\ifx\directlua\@undefined 32768\else 65536\fi}
```

luatex/xetex also allow more math fam.

```
\edef \et@xmaxfam {\ifx\Umathcode\@undefined\sixt@@n\else\@cclvi\fi}

\count 270=\et@xmaxregs % locally allocates \count registers
\count 271=\et@xmaxregs % ditto for \dimen registers
\count 272=\et@xmaxregs % ditto for \skip registers
\count 273=\et@xmaxregs % ditto for \muskip registers
\count 274=\et@xmaxregs % ditto for \box registers
\count 275=\et@xmaxregs % ditto for \toks registers
\count 276=\et@xmaxregs % ditto for \marks classes
```

and 256 or 16 fam. (Done above due to plain/LaTeX differences in ltluatex.)

```
% \outer\def\newfam{\alloc@8\fam\chardef\et@xmaxfam}
```

End of proposed changes to `etex.src`

5.2.2 luatex specific settings

Switch to global cf `luatex.sty` to leave room for inserts not really needed for luatex but possibly most compatible with existing use.

```
\expandafter\let\csname newcount\expandafter\expandafter\endcsname
                \csname globcount\endcsname
\expandafter\let\csname newdimen\expandafter\expandafter\endcsname
                \csname globdimen\endcsname
\expandafter\let\csname newskip\expandafter\expandafter\endcsname
                \csname globskip\endcsname
\expandafter\let\csname newbox\expandafter\expandafter\endcsname
                \csname globbox\endcsname
```

Define`\e@alloc` as in LaTeX (the existing macros in `etex.src` are hard to extend to further register types as they assume specific 26x and 27x count range). For compatibility the existing register allocation is not changed.

```
\chardef\e@alloc@top=65535
\let\e@alloc@chardef\chardef

\def\e@alloc#1#2#3#4#5#6{%
  \global\advance#3\@ne
  \e@ch@ck{#3}{#4}{#5}#1%
  \allocationnumber#3\relax
  \global#2#6\allocationnumber
  \wlog{\string#6=\string#1\the\allocationnumber}}%

\gdef\e@ch@ck#1#2#3#4{%
  \ifnum#1<#2\else
    \ifnum#1=#2\relax
      #1\@cclvi
      \ifx\count#4\advance#1 10 \fi
    \fi
    \ifnum#1<#3\relax
    \else
      \errmessage{No room for a new \string#4}%
    \fi
  \fi}%
```

Fix up allocations not to clash with `etex.src`.

```
\expandafter\csname newcount\endcsname\e@alloc@attribute@count
\expandafter\csname newcount\endcsname\e@alloc@ccodetable@count
\expandafter\csname newcount\endcsname\e@alloc@luafunction@count
\expandafter\csname newcount\endcsname\e@alloc@whatsit@count
\expandafter\csname newcount\endcsname\e@alloc@bytecode@count
\expandafter\csname newcount\endcsname\e@alloc@luachunk@count
```

End of conditional setup for plain TEX / old LATEX.

```
\fi
⟨/tex⟩
```

5.3 Attributes

`\newattribute` As is generally the case for the LuaTEX registers we start here from 1. Notably, some code assumes that `\attribute0` is never used so this is important in this case.

```
\ifx\e@alloc@attribute@count\@undefined
  \countdef\e@alloc@attribute@count=258
  \e@alloc@attribute@count=\z@
\fi
\def\newattribute#1{%
  \e@alloc\attribute\attributedef
    \e@alloc@attribute@count\m@ne\e@alloc@top#1%
}
```

(*End of definition for* `\newattribute`.)

`\setattribute` Handy utilities.
`\unsetattribute`

```
\def\setattribute#1#2{#1=\numexpr#2\relax}
\def\unsetattribute#1{#1=-"7FFFFFFF\relax}
```

(*End of definition for* `\setattribute` *and* `\unsetattribute`.)

5.4 Category code tables

`\newcatcodetable` Category code tables are allocated with a limit half of that used by LuaTEX for everything else. At the end of allocation there needs to be an initialization step. Table 0 is already taken (it's the global one for current use) so the allocation starts at 1.

```
\ifx\e@alloc@ccodetable@count\@undefined
  \countdef\e@alloc@ccodetable@count=259
  \e@alloc@ccodetable@count=\z@
\fi
\def\newcatcodetable#1{%
  \e@alloc\catcodetable\chardef
    \e@alloc@ccodetable@count\m@ne{"8000}#1%
  \initcatcodetable\allocationnumber
}
```

(*End of definition for* `\newcatcodetable`.)

\catcodetable@initex
\catcodetable@string
\catcodetable@latex
\catcodetable@atletter

Save a small set of standard tables. The Unicode data is read here in using a parser simplified from that in `load-unicode-data`: only the nature of letters needs to be detected.

```
\newcatcodetable\catcodetable@initex
\newcatcodetable\catcodetable@string
\begingroup
  \def\setrangecatcode#1#2#3{%
    \ifnum#1>#2 %
      \expandafter\@gobble
    \else
      \expandafter\@firstofone
    \fi
      {%
        \catcode#1=#3 %
        \expandafter\setrangecatcode\expandafter
          {\number\numexpr#1 + 1\relax}{#2}{#3}
      }%
  }
  \@firstofone{%
    \catcodetable\catcodetable@initex
      \catcode0=12 %
      \catcode13=12 %
      \catcode37=12 %
      \setrangecatcode{65}{90}{12}%
      \setrangecatcode{97}{122}{12}%
      \catcode92=12 %
      \catcode127=12 %
      \savecatcodetable\catcodetable@string
    \endgroup
  }%
\newcatcodetable\catcodetable@latex
\newcatcodetable\catcodetable@atletter
\begingroup
  \def\parseunicodedataI#1;#2;#3;#4\relax{%
    \parseunicodedataII#1;#3;#2 First>\relax
  }%
  \def\parseunicodedataII#1;#2;#3 First>#4\relax{%
    \ifx\relax#4\relax
      \expandafter\parseunicodedataIII
    \else
      \expandafter\parseunicodedataIV
    \fi
      {#1}#2\relax%
  }%
  \def\parseunicodedataIII#1#2#3\relax{%
    \ifnum 0%
      \if L#21\fi
      \if M#21\fi
      >0 %
      \catcode"#1=11 %
    \fi
  }%
  \def\parseunicodedataIV#1#2#3\relax{%
    \read\unicoderead to \unicodedataline
    \if L#2%
```

```
      \count0="#1 %
      \expandafter\parseunicodedataV\unicodedataline\relax
    \fi
  }%
  \def\parseunicodedataV#1;#2\relax{%
    \loop
      \unless\ifnum\count0>"#1 %
        \catcode\count0=11 %
        \advance\count0 by 1 %
    \repeat
  }%
  \def\storedpar{\par}%
  \chardef\unicoderead=\numexpr\count16 + 1\relax
  \openin\unicoderead=UnicodeData.txt %
  \loop\unless\ifeof\unicoderead %
    \read\unicoderead to \unicodedataline
    \unless\ifx\unicodedataline\storedpar
      \expandafter\parseunicodedataI\unicodedataline\relax
    \fi
  \repeat
  \closein\unicoderead
  \@firstofone{%
    \catcode64=12 %
    \savecatcodetable\catcodetable@latex
    \catcode64=11 %
    \savecatcodetable\catcodetable@atletter
   }
\endgroup
```

(*End of definition for* `\catcodetable@initex` *and others.*)

5.5 Named Lua functions

`\newluafunction` Much the same story for allocating LuaTEX functions except here they are just numbers so they are allocated in the same way as boxes. Lua indexes from 1 so once again slot 0 is skipped.

```
\ifx\e@alloc@luafunction@count\@undefined
  \countdef\e@alloc@luafunction@count=260
  \e@alloc@luafunction@count=\z@
\fi
\def\newluafunction{%
  \e@alloc\luafunction\e@alloc@chardef
    \e@alloc@luafunction@count\m@ne\e@alloc@top
}
```

(*End of definition for* `\newluafunction`.)

`\newluacmd` `\newprotectedluacmd` Additionally two variants are provided to make the passed control sequence call the function directly.

```
\def\newluacmd{%
  \e@alloc\luafunction\luadef
    \e@alloc@luafunction@count\m@ne\e@alloc@top
}
\def\newprotectedluacmd{%
```

```
  \e@alloc\luafunction{\protected\luadef}
    \e@alloc@luafunction@count\m@ne\e@alloc@top
}
```

(*End of definition for* `\newluacmd` *and* `\newprotectedluacmd`.)

5.6 Custom whatsits

`\newwhatsit` These are only settable from Lua but for consistency are definable here.

```
\ifx\e@alloc@whatsit@count\@undefined
  \countdef\e@alloc@whatsit@count=261
  \e@alloc@whatsit@count=\z@
\fi
\def\newwhatsit#1{%
  \e@alloc\whatsit\e@alloc@chardef
    \e@alloc@whatsit@count\m@ne\e@alloc@top#1%
}
```

(*End of definition for* `\newwhatsit`.)

5.7 Lua bytecode registers

`\newluabytecode` These are only settable from Lua but for consistency are definable here.

```
\ifx\e@alloc@bytecode@count\@undefined
  \countdef\e@alloc@bytecode@count=262
  \e@alloc@bytecode@count=\z@
\fi
\def\newluabytecode#1{%
  \e@alloc\luabytecode\e@alloc@chardef
    \e@alloc@bytecode@count\m@ne\e@alloc@top#1%
}
```

(*End of definition for* `\newluabytecode`.)

5.8 Lua chunk registers

`\newluachunkname` As for bytecode registers, but in addition we need to add a string to the `lua.name` table to use in stack tracing. We use the name of the command passed to the allocator, with no backslash.

```
\ifx\e@alloc@luachunk@count\@undefined
  \countdef\e@alloc@luachunk@count=263
  \e@alloc@luachunk@count=\z@
\fi
\def\newluachunkname#1{%
  \e@alloc\luachunk\e@alloc@chardef
    \e@alloc@luachunk@count\m@ne\e@alloc@top#1%
    \directlua{lua.name[\the\allocationnumber]="\csstring#1"}%
}
```

(*End of definition for* `\newluachunkname`.)

5.9 Lua loader

Lua code loaded in the format often has to be loaded again at the beginning of every job, so we define a helper which allows us to avoid duplicated code:

```
\def\now@and@everyjob#1{%
  \everyjob\expandafter{\the\everyjob
    #1%
  }%
  #1%
}
```

Load the Lua code at the start of every job. For the conversion of TEX into numbers at the Lua side we need some known registers: for convenience we use a set of systematic names, which means using a group around the Lua loader.

```
⟨2ekernel⟩\now@and@everyjob{%
  \begingroup
    \attributedef\attributezero=0 %
    \chardef     \charzero     =0 %
```

Note name change required on older luatex, for hash table access.

```
    \countdef    \CountZero    =0 %
    \dimendef    \dimenzero    =0 %
    \mathchardef \mathcharzero =0 %
    \muskipdef   \muskipzero   =0 %
    \skipdef     \skipzero     =0 %
    \toksdef     \tokszero     =0 %
    \directlua{require("ltluatex")}
  \endgroup
⟨2ekernel⟩}
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}
⟨latexrelease⟩                  {\newluafunction}{LuaTeX}%
⟨latexrelease⟩\let\e@alloc@attribute@count\@undefined
⟨latexrelease⟩\let\newattribute\@undefined
⟨latexrelease⟩\let\setattribute\@undefined
⟨latexrelease⟩\let\unsetattribute\@undefined
⟨latexrelease⟩\let\e@alloc@ccodetable@count\@undefined
⟨latexrelease⟩\let\newcatcodetable\@undefined
⟨latexrelease⟩\let\catcodetable@initex\@undefined
⟨latexrelease⟩\let\catcodetable@string\@undefined
⟨latexrelease⟩\let\catcodetable@latex\@undefined
⟨latexrelease⟩\let\catcodetable@atletter\@undefined
⟨latexrelease⟩\let\e@alloc@luafunction@count\@undefined
⟨latexrelease⟩\let\newluafunction\@undefined
⟨latexrelease⟩\let\e@alloc@luafunction@count\@undefined
⟨latexrelease⟩\let\newwhatsit\@undefined
⟨latexrelease⟩\let\e@alloc@whatsit@count\@undefined
⟨latexrelease⟩\let\newluabytecode\@undefined
⟨latexrelease⟩\let\e@alloc@bytecode@count\@undefined
⟨latexrelease⟩\let\newluachunkname\@undefined
⟨latexrelease⟩\let\e@alloc@luachunk@count\@undefined
⟨latexrelease⟩\directlua{luatexbase.uninstall()}
⟨latexrelease⟩\EndIncludeInRelease
```

In `\everyjob`, if luaotfload is available, load it and switch to TU.

```
⟨latexrelease⟩\IncludeInRelease{2017/01/01}%
⟨latexrelease⟩                  {\fontencoding}{TU in everyjob}%
⟨latexrelease⟩\fontencoding{TU}\let\encodingdefault\f@encoding
⟨latexrelease⟩\ifx\directlua\@undefined\else
⟨2ekernel⟩\everyjob\expandafter{%
⟨2ekernel⟩   \the\everyjob
⟨*2ekernel, latexrelease⟩
  \directlua{%
  if xpcall(function ()%
              require('luaotfload-main')%
            end,texio.write_nl) then %
  local _void = luaotfload.main ()%
  else %
  texio.write_nl('Error in luaotfload: reverting to OT1')%
  tex.print('\string\\def\string\\encodingdefault{OT1}')%
  end %
  }%
  \let\f@encoding\encodingdefault
  \expandafter\let\csname ver@luaotfload.sty\endcsname\fmtversion
⟨/2ekernel, latexrelease⟩
⟨latexrelease⟩\fi
⟨2ekernel⟩  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\fontencoding}{TU in everyjob}%
⟨latexrelease⟩\fontencoding{OT1}\let\encodingdefault\f@encoding
⟨latexrelease⟩\EndIncludeInRelease

⟨2ekernel | latexrelease⟩\fi
⟨/2ekernel | tex | latexrelease⟩
```

5.10 Lua module preliminaries

```
⟨*lua⟩
```

Some set up for the Lua module which is needed for all of the Lua functionality added here.

`luatexbase` Set up the table for the returned functions. This is used to expose all of the public functions.

```
luatexbase       = luatexbase or { }
local luatexbase = luatexbase
```

(*End of definition for* `luatexbase`.)

Some Lua best practice: use local versions of functions where possible.

```
local string_format    = string.format
local string_gsub      = string.gsub
local tex_count        = tex.count
local tex_setcount     = tex.setcount
local texio_write_nl   = texio.write_nl
local flush_list       = node.flush_list
local token_set_lua    = token.set_lua

local luatexbase_warning
local luatexbase_error
```

5.11 Lua module utilities

5.11.1 Module tracking

`modules` To allow tracking of module usage, a structure is provided to store information and to return it.

```
local modules = modules or { }
```

(*End of definition for* `modules`.)

`provides_module` Local function to write to the log.

```
local function luatexbase_log(text)
  texio_write_nl("log", text)
end
```

Modelled on `\ProvidesPackage`, we store much the same information but with a little more structure.

```
local function provides_module(info)
  if not (info and info.name) then
    luatexbase_error("Missing module name for provides_module")
  end
  local function spaced(text)
    return text and (" " .. text) or ""
  end
  luatexbase_log(
    "Lua module: " .. info.name
      .. spaced(info.date)
      .. spaced(info.version and string_gsub(info.version or "","^(%d)","v%1"))
      .. spaced(info.description)
  )
  modules[info.name] = info
end
luatexbase.provides_module = provides_module
```

(*End of definition for* `provides_module`.)

5.11.2 Module messages

There are various warnings and errors that need to be given. For warnings we can get exactly the same formatting as from TEX. For errors we have to make some changes. Here we give the text of the error in the LATEX format then force an error from Lua to halt the run. Splitting the message text is done using `\n` which takes the place of `\MessageBreak`.

First an auxiliary for the formatting: this measures up the message leader so we always get the correct indent.

```
local function msg_format(mod, msg_type, text)
  local leader = ""
  local cont
  local first_head
  if mod == "LaTeX" then
    cont = string_gsub(leader, ".", " ")
    first_head = leader .. "LaTeX: "
  else
    first_head = leader .. "Module "  .. msg_type
```

```
    cont = "(" .. mod .. ")"
      .. string_gsub(first_head, ".", " ")
    first_head =  leader .. "Module "  .. mod .. " " .. msg_type  .. ":"
  end
  if msg_type == "Error" then
    first_head = "\n" .. first_head
  end
  if string.sub(text,-1) ~= "\n" then
    text = text .. " "
  end
  return first_head .. " "
    .. string_gsub(
         text
  .. "on input line "
         .. tex.inputlineno, "\n", "\n" .. cont .. " "
      )
   .. "\n"
end
```

`module_info` Write messages.

`module_warning` `module_error`

```
local function module_info(mod, text)
  texio_write_nl("log", msg_format(mod, "Info", text))
end
luatexbase.module_info = module_info
local function module_warning(mod, text)
  texio_write_nl("term and log",msg_format(mod, "Warning", text))
end
luatexbase.module_warning = module_warning
local function module_error(mod, text)
  error(msg_format(mod, "Error", text))
end
luatexbase.module_error = module_error
```

(*End of definition for* `module_info`*,* `module_warning`*, and* `module_error`*.*)

Dedicated versions for the rest of the code here.

```
function luatexbase_warning(text)
  module_warning("luatexbase", text)
end
function luatexbase_error(text)
  module_error("luatexbase", text)
end
```

5.12 Accessing register numbers from Lua

Collect up the data from the TEX level into a Lua table: from version 0.80, LuaTEX makes that easy.

```
local luaregisterbasetable = { }
local registermap = {
  attributezero = "assign_attr"    ,
  charzero      = "char_given"     ,
  CountZero     = "assign_int"     ,
  dimenzero     = "assign_dimen"   ,
  mathcharzero  = "math_given"     ,
```

```
  muskipzero    = "assign_mu_skip" ,
  skipzero      = "assign_skip"    ,
  tokszero      = "assign_toks"    ,
}
local createtoken
if tex.luatexversion > 81 then
  createtoken = token.create
elseif tex.luatexversion > 79 then
  createtoken = newtoken.create
end
local hashtokens    = tex.hashtokens()
local luatexversion = tex.luatexversion
for i,j in pairs (registermap) do
  if luatexversion < 80 then
    luaregisterbasetable[hashtokens[i][1]] =
      hashtokens[i][2]
  else
    luaregisterbasetable[j] = createtoken(i).mode
  end
end
```

registernumber

Working out the correct return value can be done in two ways. For older LuaTEX releases it has to be extracted from the `hashtokens`. On the other hand, newer LuaTEX's have `newtoken`, and whilst `.mode` isn't currently documented, Hans Hagen pointed to this approach so we should be OK.

```
local registernumber
if luatexversion < 80 then
  function registernumber(name)
    local nt = hashtokens[name]
    if(nt and luaregisterbasetable[nt[1]]) then
      return nt[2] - luaregisterbasetable[nt[1]]
    else
      return false
    end
  end
else
  function registernumber(name)
    local nt = createtoken(name)
    if(luaregisterbasetable[nt.cmdname]) then
      return nt.mode - luaregisterbasetable[nt.cmdname]
    else
      return false
    end
  end
end
luatexbase.registernumber = registernumber
```

(*End of definition for* `registernumber`.)

5.13 Attribute allocation

new_attribute

As attributes are used for Lua manipulations its useful to be able to assign from this end.

```
local attributes=setmetatable(
{},
```

```
{
__index = function(t,key)
return registernumber(key) or nil
end}
)
luatexbase.attributes = attributes
local attribute_count_name =
                     attribute_count_name or "e@alloc@attribute@count"
local function new_attribute(name)
  tex_setcount("global", attribute_count_name,
                         tex_count[attribute_count_name] + 1)
  if tex_count[attribute_count_name] > 65534 then
    luatexbase_error("No room for a new \\attribute")
  end
  attributes[name]= tex_count[attribute_count_name]
  luatexbase_log("Lua-only attribute " .. name .. " = " ..
                 tex_count[attribute_count_name])
  return tex_count[attribute_count_name]
end
luatexbase.new_attribute = new_attribute
```

(*End of definition for* `new_attribute`.)

5.14 Custom whatsit allocation

`new_whatsit` Much the same as for attribute allocation in Lua.

```
local whatsit_count_name = whatsit_count_name or "e@alloc@whatsit@count"
local function new_whatsit(name)
  tex_setcount("global", whatsit_count_name,
                         tex_count[whatsit_count_name] + 1)
  if tex_count[whatsit_count_name] > 65534 then
    luatexbase_error("No room for a new custom whatsit")
  end
  luatexbase_log("Custom whatsit " .. (name or "") .. " = " ..
                 tex_count[whatsit_count_name])
  return tex_count[whatsit_count_name]
end
luatexbase.new_whatsit = new_whatsit
```

(*End of definition for* `new_whatsit`.)

5.15 Bytecode register allocation

`new_bytecode` Much the same as for attribute allocation in Lua. The optional ⟨*name*⟩ argument is used in the log if given.

```
local bytecode_count_name =
                         bytecode_count_name or "e@alloc@bytecode@count"
local function new_bytecode(name)
  tex_setcount("global", bytecode_count_name,
                         tex_count[bytecode_count_name] + 1)
  if tex_count[bytecode_count_name] > 65534 then
    luatexbase_error("No room for a new bytecode register")
  end
```

```
  luatexbase_log("Lua bytecode " .. (name or "") .. " = " ..
                 tex_count[bytecode_count_name])
  return tex_count[bytecode_count_name]
end
luatexbase.new_bytecode = new_bytecode
```

(*End of definition for* `new_bytecode`.)

5.16 Lua chunk name allocation

`new_chunkname` As for bytecode registers but also store the name in the `lua.name` table.

```
local chunkname_count_name =
                         chunkname_count_name or "e@alloc@luachunk@count"
local function new_chunkname(name)
  tex_setcount("global", chunkname_count_name,
                         tex_count[chunkname_count_name] + 1)
  local chunkname_count = tex_count[chunkname_count_name]
  chunkname_count = chunkname_count + 1
  if chunkname_count > 65534 then
    luatexbase_error("No room for a new chunkname")
  end
  lua.name[chunkname_count]=name
  luatexbase_log("Lua chunkname " .. (name or "") .. " = " ..
                 chunkname_count .. "\n")
  return chunkname_count
end
luatexbase.new_chunkname = new_chunkname
```

(*End of definition for* `new_chunkname`.)

5.17 Lua function allocation

`new_luafunction` Much the same as for attribute allocation in Lua. The optional ⟨*name*⟩ argument is used in the log if given.

```
local luafunction_count_name =
                         luafunction_count_name or "e@alloc@luafunction@count"
local function new_luafunction(name)
  tex_setcount("global", luafunction_count_name,
                         math.max(
                           #(lua.get_functions_table()),
                           tex_count[luafunction_count_name])
                          + 1)
  lua.get_functions_table()[tex_count[luafunction_count_name]] = false
  if tex_count[luafunction_count_name] > 65534 then
    luatexbase_error("No room for a new luafunction register")
  end
  luatexbase_log("Lua function " .. (name or "") .. " = " ..
                 tex_count[luafunction_count_name])
  return tex_count[luafunction_count_name]
end
luatexbase.new_luafunction = new_luafunction
```

(*End of definition for* `new_luafunction`.)

new_luacommand A convenience wrapper around `new_luafunction` and `token.set_lua`: it allocates the lua function slot, stores ⟨*fn*⟩ there, and binds ⟨*csname*⟩ to it in one step.

```
local function new_luacommand(csname, fn, ...)
  local fntable = lua.get_functions_table()
  local fnalloc = new_luafunction(csname)
  fntable[fnalloc] = fn
  token_set_lua(csname, fnalloc, ...)
end
luatexbase.new_luacommand = new_luacommand
```

(*End of definition for* `new_luacommand`.)

5.18 Lua callback management

The native mechanism for callbacks in LuaTEX allows only one per function. That is extremely restrictive and so a mechanism is needed to add and remove callbacks from the appropriate hooks.

5.18.1 Housekeeping

The main table: keys are callback names, and values are the associated lists of functions. More precisely, the entries in the list are tables holding the actual function as `func` and the identifier `identifier`. Only callbacks with a non-empty list of functions have an entry in this list.

Actually there are two tables: `realcallbacklist` directly contains the entries as described above while `callbacklist` only directly contains the already sorted entries. Other entries can be queried through `callbacklist` too which triggers a resort.

Additionally `callbackrules` describes the ordering constraints: It contains two element tables with the identifiers of the constrained callback implementations. It can additionally contain a `type` entry indicating the kind of rule. A missing value indicates a normal ordering constraint.

```
local realcallbacklist = {}
local callbackrules = {}
local callbacklist = setmetatable({}, {
  __index = function(t, name)
    local list = realcallbacklist[name]
    local rules = callbackrules[name]
    if list and rules then
      local meta = {}
      for i, entry in ipairs(list) do
        local t = {value = entry, count = 0, pos = i}
        meta[entry.identifier], list[i] = t, t
      end
      local count = #list
      local pos = count
      for i, rule in ipairs(rules) do
        local rule = rules[i]
        local pre, post = meta[rule[1]], meta[rule[2]]
        if pre and post then
          if rule.type then
            if not rule.hidden then
              assert(rule.type == 'incompatible-warning' and luatexbase_warning
```

```
              or rule.type == 'incompatible-error' and luatexbase_error)(
                "Incompatible functions \"" .. rule[1] .. "\" and \"" .. rule[2]
                .. "\" specified for callback \"" .. name .. "\".")
            rule.hidden = true
          end
        else
          local post_count = post.count
          post.count = post_count+1
          if post_count == 0 then
            local post_pos = post.pos
            if post_pos ~= pos then
              local new_post_pos = list[pos]
              new_post_pos.pos = post_pos
              list[post_pos] = new_post_pos
            end
            list[pos] = nil
            pos = pos - 1
          end
          pre[#pre+1] = post
        end
      end
    end
    for i=1, count do -- The actual sort begins
      local current = list[i]
      if current then
        meta[current.value.identifier] = nil
        for j, cur in ipairs(current) do
          local count = cur.count
          if count == 1 then
            pos = pos + 1
            list[pos] = cur
          else
            cur.count = count - 1
          end
        end
        list[i] = current.value
      else
        -- Cycle occurred. TODO: Show cycle for debugging
        -- list[i] = ...
        local remaining = {}
        for name, entry in next, meta do
          local value = entry.value
          list[#list + 1] = entry.value
          remaining[#remaining + 1] = name
        end
        table.sort(remaining)
        local first_name = remaining[1]
        for j, name in ipairs(remaining) do
          local entry = meta[name]
          list[i + j - 1] = entry.value
          for _, post_entry in ipairs(entry) do
            local post_name = post_entry.value.identifier
            if not remaining[post_name] then
              remaining[post_name] = name
```

```
              end
            end
          end
          local cycle = {first_name}
          local index = 1
          local last_name = first_name
          repeat
            cycle[last_name] = index
            last_name = remaining[last_name]
            index = index + 1
            cycle[index] = last_name
          until cycle[last_name]
          local length = index - cycle[last_name] + 1
          table.move(cycle, cycle[last_name], index, 1)
          for i=2, length//2 do
            cycle[i], cycle[length + 1 - i] = cycle[length + 1 - i], cycle[i]
          end
          error('Cycle occurred at ' .. table.concat(cycle, ' -> ', 1, length))
        end
      end
    end
    realcallbacklist[name] = list
    t[name] = list
    return list
  end
})
```

Numerical codes for callback types, and name-to-value association (the table keys are strings, the values are numbers).

```
local list, data, exclusive, simple, reverselist = 1, 2, 3, 4, 5
local types   = {
  list        = list,
  data        = data,
  exclusive   = exclusive,
  simple      = simple,
  reverselist = reverselist,
}
```

Now, list all predefined callbacks with their current type, based on the LuaTEX manual version 1.01. A full list of the currently-available callbacks can be obtained using

```
\directlua{
  for i,_ in pairs(callback.list()) do
    texio.write_nl("- " .. i)
  end
}
\bye
```

in plain LuaTEX. (Some undocumented callbacks are omitted as they are to be removed.)

```
local callbacktypes = callbacktypes or {
```

Section 8.2: file discovery callbacks.

```
  find_read_file     = exclusive,
  find_write_file    = exclusive,
  find_font_file     = data,
```

```
  find_output_file   = data,
  find_format_file   = data,
  find_vf_file       = data,
  find_map_file      = data,
  find_enc_file      = data,
  find_pk_file       = data,
  find_data_file     = data,
  find_opentype_file = data,
  find_truetype_file = data,
  find_type1_file    = data,
  find_image_file    = data,

  open_read_file     = exclusive,
  read_font_file     = exclusive,
  read_vf_file       = exclusive,
  read_map_file      = exclusive,
  read_enc_file      = exclusive,
  read_pk_file       = exclusive,
  read_data_file     = exclusive,
  read_truetype_file = exclusive,
  read_type1_file    = exclusive,
  read_opentype_file = exclusive,
```

Not currently used by luatex but included for completeness. may be used by a font handler.

```
  find_cidmap_file   = data,
  read_cidmap_file   = exclusive,
```

Section 8.3: data processing callbacks.

```
  process_input_buffer  = data,
  process_output_buffer = data,
  process_jobname       = data,
```

Section 8.4: node list processing callbacks.

```
  contribute_filter      = simple,
  buildpage_filter       = simple,
  build_page_insert      = exclusive,
  pre_linebreak_filter   = list,
  linebreak_filter       = exclusive,
  append_to_vlist_filter = exclusive,
  post_linebreak_filter  = reverselist,
  hpack_filter           = list,
  vpack_filter           = list,
  hpack_quality          = exclusive,
  vpack_quality          = exclusive,
  pre_output_filter      = list,
  process_rule           = exclusive,
  hyphenate              = simple,
  ligaturing             = simple,
  kerning                = simple,
  insert_local_par       = simple,
% mlist_to_hlist         = exclusive,
  new_graf               = exclusive,
```

Section 8.5: information reporting callbacks.

```
  pre_dump             = simple,
```

```
  start_run            = simple,
  stop_run             = simple,
  start_page_number    = simple,
  stop_page_number     = simple,
  show_error_hook      = simple,
  show_warning_message = simple,
  show_error_message   = simple,
  show_lua_error_hook  = simple,
  start_file           = simple,
  stop_file            = simple,
  call_edit            = simple,
  finish_synctex       = simple,
  wrapup_run           = simple,
```

Section 8.6: PDF-related callbacks.

```
  finish_pdffile           = data,
  finish_pdfpage           = data,
  page_objnum_provider     = data,
  page_order_index         = data,
  process_pdf_image_content = data,
```

Section 8.7: font-related callbacks.

```
  define_font                    = exclusive,
  glyph_info                     = exclusive,
  glyph_not_found                = exclusive,
  glyph_stream_provider          = exclusive,
  make_extensible                = exclusive,
  font_descriptor_objnum_provider = exclusive,
  input_level_string             = exclusive,
  provide_charproc_data          = exclusive,

}
luatexbase.callbacktypes=callbacktypes
```

Sometimes multiple callbacks correspond to a single underlying engine level callback. Then the engine level callback should be registered as long as at least one of these callbacks is in use. This is implemented though a shared table which counts how many of the involved callbacks are currently in use. The enging level callback is registered iff this count is not 0.

We add `mlist_to_hlist` directly to the list to demonstrate this, but the handler gets added later when it is actually defined.

All callbacks in this list are treated as user defined callbacks.

```
local shared_callbacks = {
  mlist_to_hlist = {
    callback = "mlist_to_hlist",
    count = 0,
    handler = nil,
  },
}
shared_callbacks.pre_mlist_to_hlist_filter = shared_callbacks.mlist_to_hlist
shared_callbacks.post_mlist_to_hlist_filter = shared_callbacks.mlist_to_hlist
```

`callback.register` Save the original function for registering callbacks and prevent the original being used. The original is saved in a place that remains available so other more sophisticated code can override the approach taken by the kernel if desired.

```
local callback_register = callback_register or callback.register
function callback.register()
  luatexbase_error("Attempt to use callback.register() directly\n")
end
```

(*End of definition for* `callback.register`.)

5.18.2 Handlers

The handler function is registered into the callback when the first function is added to this callback's list. Then, when the callback is called, the handler takes care of running all functions in the list. When the last function is removed from the callback's list, the handler is unregistered.

More precisely, the functions below are used to generate a specialized function (closure) for a given callback, which is the actual handler.

The way the functions are combined together depends on the type of the callback. There are currently 4 types of callback, depending on the calling convention of the functions the callback can hold:

simple is for functions that don't return anything: they are called in order, all with the same argument;

data is for functions receiving a piece of data of any type except node list head (and possibly other arguments) and returning it (possibly modified): the functions are called in order, and each is passed the return value of the previous (and the other arguments untouched, if any). The return value is that of the last function;

list is a specialized variant of *data* for functions filtering node lists. Such functions are called with a node list head as the first argument and may return either the head of a modified node list, or the boolean values `true` or `false`. The functions are chained the same way as for *data* except for the following cases. If a function returns `false`, then `false` is immediately returned and the following functions are *not* called. If a function returns `true`, then the same head is passed to the next function. If all functions return `true`, then the original head is returned, otherwise the return value of the last function not returning `true` is used.

reverselist is a specialized variant of *list* which executes functions in inverse order.

exclusive is for functions with more complex signatures; functions in this type of callback are *not* combined: An error is raised if a second callback is registered.

Handler for `data` callbacks.

```
local function data_handler(name)
  return function(data, ...)
    for _,i in ipairs(callbacklist[name]) do
      data = i.func(data,...)
    end
    return data
  end
end
```

Default for user-defined `data` callbacks without explicit default.

```
local function data_handler_default(value)
  return value
end
```

Handler for `exclusive` callbacks. We can assume `callbacklist[name]` is not empty: otherwise, the function wouldn't be registered in the callback any more.

```
local function exclusive_handler(name)
  return function(...)
    return callbacklist[name][1].func(...)
  end
end
```

Handler for `list` callbacks.

```
local function list_handler(name)
  return function(head, ...)
    local ret
    for _,i in ipairs(callbacklist[name]) do
      ret = i.func(head, ...)
      if ret == false then
        luatexbase_warning(
          "Function `" .. i.identifier .. "' returned false\n"
            .. "in callback `" .. name .."'"
         )
        return false
      end
      if ret ~= true then
        head = ret
      end
    end
    return head
  end
end
```

Default for user-defined `list` and `reverselist` callbacks without explicit default.

```
local function list_handler_default(head)
return head
end
```

Handler for `reverselist` callbacks.

```
local function reverselist_handler(name)
  return function(head, ...)
    local ret
    local callbacks = callbacklist[name]
    for i = #callbacks, 1, -1 do
      local cb = callbacks[i]
      ret = cb.func(head, ...)
      if ret == false then
        luatexbase_warning(
          "Function `" .. cb.identifier .. "' returned false\n"
            .. "in callback `" .. name .."'"
         )
        return false
      end
      if ret ~= true then
        head = ret
      end
    end
    return head
  end
```

```
end
```

Handler for `simple` callbacks.

```
local function simple_handler(name)
  return function(...)
    for _,i in ipairs(callbacklist[name]) do
      i.func(...)
    end
  end
end
```

Default for user-defined `simple` callbacks without explicit default.

```
local function simple_handler_default()
end
```

Keep a handlers table for indexed access and a table with the corresponding default functions.

```
local handlers  = {
  [data]        = data_handler,
  [exclusive]   = exclusive_handler,
  [list]        = list_handler,
  [reverselist] = reverselist_handler,
  [simple]      = simple_handler,
}
local defaults = {
  [data]        = data_handler_default,
  [exclusive]   = nil,
  [list]        = list_handler_default,
  [reverselist] = list_handler_default,
  [simple]      = simple_handler_default,
}
```

5.18.3 Public functions for callback management

Defining user callbacks perhaps should be in package code, but impacts on `add_to_callback`. If a default function is not required, it may be declared as `false`. First we need a list of user callbacks.

```
local user_callbacks_defaults = {}
```

`create_callback` The allocator itself.

```
local function create_callback(name, ctype, default)
  local ctype_id = types[ctype]
  if not name  or name  == ""
  or not ctype_id
  then
    luatexbase_error("Unable to create callback:\n" ..
                     "valid callback name and type required")
  end
  if callbacktypes[name] then
    luatexbase_error("Unable to create callback `" .. name ..
                     "':\ncallback is already defined")
  end
  default = default or defaults[ctype_id]
  if not default then
```

```
    luatexbase_error("Unable to create callback `" .. name ..
                     "':\ndefault is required for `" .. ctype ..
                     "' callbacks")
  elseif type (default) ~= "function" then
    luatexbase_error("Unable to create callback `" .. name ..
                     "':\ndefault is not a function")
  end
  user_callbacks_defaults[name] = default
  callbacktypes[name] = ctype_id
end
luatexbase.create_callback = create_callback
```

(*End of definition for* `create_callback`.)

`call_callback` Call a user defined callback. First check arguments.

```
local function call_callback(name,...)
  if not name or name == "" then
    luatexbase_error("Unable to create callback:\n" ..
                     "valid callback name required")
  end
  if user_callbacks_defaults[name] == nil then
    luatexbase_error("Unable to call callback `" .. name
                     .. "':\nunknown or empty")
   end
  local l = callbacklist[name]
  local f
  if not l then
    f = user_callbacks_defaults[name]
  else
    f = handlers[callbacktypes[name]](name)
  end
  return f(...)
end
luatexbase.call_callback=call_callback
```

(*End of definition for* `call_callback`.)

`in_callback` Look for a function identifier in a callback.

```
local function in_callback(name, identifier)
  if not name
    or name == ""
    or not realcallbacklist[name]
    or not callbacktypes[name]
    or not identifier then
      return false
  end
  for _, i in pairs(realcallbacklist[name]) do
    if i.identifier == identifier then
      return true
    end
  end
  return false
end
luatexbase.in_callback = in_callback
```

(*End of definition for* `in_callback`.)

`add_to_callback` Add a function to a callback. First check arguments.

```
local function add_to_callback(name, func, identifier)
  if not name or name == "" then
    luatexbase_error("Unable to register callback:\n" ..
                     "valid callback name required")
  end
  if not callbacktypes[name] or
    type(func) ~= "function" or
    not identifier or
    identifier == "" then
    luatexbase_error(
      "Unable to register callback.\n\n"
        .. "Correct usage:\n"
        .. "add_to_callback(<callback>, <function>, <identifier>)"
    )
  end
```

Check if the specified identifier is already registered. This used to be allowed but will become an error in a future release.

```
  if in_callback(name, identifier) then
  luatexbase_warning(string_format(
      "The identifier %s is already in use in callback %s.\n\z
      Please change the identifier of the newly registered callback \z
      to avoid errors in the next LaTeX release", identifier, name))
  end
```

Then test if this callback is already in use. If not, initialize its list and register the proper handler.

```
  local l = realcallbacklist[name]
  if l == nil then
    l = { }
    realcallbacklist[name] = l
```

Handle count for shared engine callbacks.

```
    local shared = shared_callbacks[name]
    if shared then
      shared.count = shared.count + 1
      if shared.count == 1 then
        callback_register(shared.callback, shared.handler)
      end
```

If it is not a user defined callback use the primitive callback register.

```
    elseif user_callbacks_defaults[name] == nil then
      callback_register(name, handlers[callbacktypes[name]](name))
    end
  end
```

Actually register the function and give an error if more than one `exclusive` one is registered.

```
  local f = {
    func        = func,
    identifier = identifier,
  }
  if callbacktypes[name] == exclusive then
```

```
    if #l == 1 then
      luatexbase_error(
        "Cannot add second callback to exclusive function\n‘" ..
        name .. "’")
    end
  end
  table.insert(l, f)
  callbacklist[name] = nil
```

Keep user informed.

```
  luatexbase_log(
    "Inserting ‘" .. identifier .. "’ in ‘" .. name .. "’."
  )
end
luatexbase.add_to_callback = add_to_callback
```

(*End of definition for* `add_to_callback`.)

`declare_callback_rule` Add an ordering constraint between two callback implementations

```
local function declare_callback_rule(name, desc1, relation, desc2)
  if not callbacktypes[name] or
    not desc1 or not desc2 or
    desc1 == "" or desc2 == "" then
    luatexbase_error(
      "Unable to create ordering constraint. "
        .. "Correct usage:\n"
        .. "declare_callback_rule(<callback>, <identifier_a>, <identifier_b>)"
    )
  end
  if relation == ’before’ then
    relation = nil
  elseif relation == ’after’ then
    desc2, desc1 = desc1, desc2
    relation = nil
  elseif relation == ’incompatible-warning’ or relation == ’incompatible-error’ then
  elseif relation == ’unrelated’ then
  else
    luatexbase_error(
      "Unknown relation type in declare_callback_rule"
    )
  end
  callbacklist[name] = nil
  local rules = callbackrules[name]
  if rules then
    for i, rule in ipairs(rules) do
      if rule[1] == desc1 and rule[2] == desc2 or rule[1] == desc2 and rule[2] == desc1 then
        if relation == ’unrelated’ then
          table.remove(rules, i)
        else
          rule[1], rule[2], rule.type = desc1, desc2, relation
        end
        return
      end
    end
    if relation ~= ’unrelated’ then
```

```
      rules[#rules + 1] = {desc1, desc2, type = relation}
    end
  elseif relation ~= 'unrelated' then
    callbackrules[name] = {{desc1, desc2, type = relation}}
  end
end
luatexbase.declare_callback_rule = declare_callback_rule
```

(*End of definition for* `declare_callback_rule`.)

`remove_from_callback` Remove a function from a callback. First check arguments.

```
local function remove_from_callback(name, identifier)
  if not name or name == "" then
    luatexbase_error("Unable to remove function from callback:\n" ..
                     "valid callback name required")
  end
  if not callbacktypes[name] or
    not identifier or
    identifier == "" then
    luatexbase_error(
      "Unable to remove function from callback.\n\n"
        .. "Correct usage:\n"
        .. "remove_from_callback(<callback>, <identifier>)"
    )
  end
  local l = realcallbacklist[name]
  if not l then
    luatexbase_error(
      "No callback list for `" .. name .. "'\n")
  end
```

Loop over the callback's function list until we find a matching entry. Remove it and check if the list is empty: if so, unregister the callback handler.

```
  local index = false
  for i,j in ipairs(l) do
    if j.identifier == identifier then
      index = i
      break
    end
  end
  if not index then
    luatexbase_error(
      "No callback `" .. identifier .. "' registered for `" ..
      name .. "'\n")
  end
  local cb = l[index]
  table.remove(l, index)
  luatexbase_log(
    "Removing  `" .. identifier .. "' from `" .. name .. "'."
  )
  if #l == 0 then
    realcallbacklist[name] = nil
    callbacklist[name] = nil
    local shared = shared_callbacks[name]
    if shared then
```

```
      shared.count = shared.count - 1
      if shared.count == 0 then
        callback_register(shared.callback, nil)
      end
    elseif user_callbacks_defaults[name] == nil then
      callback_register(name, nil)
    end
  end
  return cb.func,cb.identifier
end
luatexbase.remove_from_callback = remove_from_callback
```

(*End of definition for* `remove_from_callback`.)

`disable_callback` As we subvert the engine interface we need to provide a way to access this functionality.

```
local function disable_callback(name)
  if(realcallbacklist[name] == nil) then
    callback_register(name, false)
  else
    luatexbase_error("Callback list for " .. name .. " not empty")
  end
end
luatexbase.disable_callback = disable_callback
```

(*End of definition for* `disable_callback`.)

`callback_identifiers` List the identifiers of functions registered for the given callback. This will sort the list if necessary.

```
local function callback_identifiers (name)
  local d = {}
  if not name
    or name == ""
    or not realcallbacklist[name]
    or not callbacktypes[name]
    then
    return d
  else
  for k, i in pairs(callbacklist[name]) do
    d[k]= i.identifier
    end
  end
  return d
end
luatexbase.callback_descriptions =callback_identifiers
luatexbase.callback_identifiers = callback_identifiers
```

(*End of definition for* `callback_identifiers`.)

`uninstall` Unlike at the TEX level, we have to provide a back-out mechanism here at the same time as the rest of the code. This is not meant for use by anything other than latexrelease: as such this is *deliberately* not documented for users!

```
local function uninstall()
  module_info(
    "luatexbase",
```

```
    "Uninstalling kernel luatexbase code"
  )
  callback.register = callback_register
  luatexbase = nil
end
luatexbase.uninstall = uninstall
```

(*End of definition for* `uninstall`.)

`mlist_to_hlist` To emulate these callbacks, the "real" `mlist_to_hlist` is replaced by a wrapper calling the wrappers before and after.

```
create_callback('pre_mlist_to_hlist_filter', 'list')
create_callback('mlist_to_hlist', 'exclusive', node.mlist_to_hlist)
create_callback('post_mlist_to_hlist_filter', 'reverselist')
function shared_callbacks.mlist_to_hlist.handler(head, display_type, need_penalties)
  local current = call_callback("pre_mlist_to_hlist_filter", head, display_type, need_penalti
  if current == false then
    flush_list(head)
    return nil
  end
  current = call_callback("mlist_to_hlist", current, display_type, need_penalties)
  local post = call_callback("post_mlist_to_hlist_filter", current, display_type, need_penalt
  if post == false then
    flush_list(current)
    return nil
  end
  return post
end
```

(*End of definition for* `mlist_to_hlist`.)

```
⟨/lua⟩
```

Reset the catcode of `@`.

```
⟨tex⟩\catcode`\@=\etatcatcode\relax
```

File 05
ltexpl.dtx

1 expl3-dependent code

1.1 Loader

\@kernel@after@enddocument
\@kernel@before@enddocument@afterlastpage
\@kernel@after@enddocument@afterlastpage

These three kernel hooks are used by the shipout code. They are defined early (i.e., here) because the lthooks code adds material to them.

```
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩   {kernel@enddocument hooks}{Define several kernel hooks}
```

We initialize these kernel hooks only when they do not already exist. Otherwise they would be set to `\@empty` on rollback, which would be wrong because code that has been added to them may still have to be executed in the rollback situation. Instead code that writes to them needs to handle the rollback as needed. It is likely that we have to change that approach in the future, but for now it should do. (It is enough to test only for the existence of one hook, as all got added at the same time.)

```
\ifx\@kernel@after@enddocument\@undefined
  \let \@kernel@after@enddocument                \@empty
  \let \@kernel@after@enddocument@afterlastpage  \@empty
```

For the similar reasons we also define those that are used in `\document` because they too get material added to in early modules.

```
  \let \@kernel@before@begindocument \@empty
  \let \@kernel@after@begindocument  \@empty
\fi
```

\@kernel@before@begindocument
\@kernel@after@begindocument

This one actually came later so we need to have a separate check, but otherwise we don't bother with a separate rollback.

```
\ifx\@kernel@before@enddocument@afterlastpage\@undefined
  \let \@kernel@before@enddocument@afterlastpage \@empty
\fi
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩   {kernel@enddocument hooks}{Define several kernel hooks}
⟨latexrelease⟩\let\@kernel@after@enddocument\@undefined
⟨latexrelease⟩\let\@kernel@before@enddocument@afterlastpage\@undefined
⟨latexrelease⟩\let\@kernel@after@enddocument@afterlastpage\@undefined
⟨latexrelease⟩\let\@kernel@before@begindocument\@undefined
⟨latexrelease⟩\let\@kernel@after@begindocument\@undefined
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\@kernel@after@enddocument` *and others.*)

First define some blank commands, so that in case something goes wrong while loading expl3, we won't get strange `Undefined control sequence` errors.

```
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩             {\@expl@sys@load@backend@@}{Roll forward support}%
```

```
\def\reserved@a#1{\ifdefined#1\else\def#1{}\fi}
\reserved@a\@expl@sys@load@backend@@
\reserved@a\@expl@push@filename@@
\reserved@a\@expl@push@filename@aux@@
\reserved@a\@expl@pop@filename@@
⟨latexrelease⟩\EndIncludeInRelease
⟨/2ekernel | latexrelease⟩
```

Create a hook for last-minute expl3 material.

```
⟨*2ekernel⟩
\def\@expl@finalise@setup@@{}
\def\@expl@finalize@setup@@{}
⟨/2ekernel⟩
```

Now define some basics to support loading expl3. These macros can be defined here safely, because they are redefined later on by the kernel, so we define simpler versions just to suit our needs.

```
⟨*2ekernel⟩
\long\def\@gobble#1{}
\long\def\@firstofone#1{#1}
\long\def\@firstoftwo#1#2{#1}
\long\def\@secondoftwo#1#2{#2}
\long\def\IfFileExists#1{%
  \openin\@inputcheck"#1" %
  \ifeof\@inputcheck
    \expandafter\@secondoftwo
  \else
    \closein\@inputcheck
    \expandafter\@firstoftwo
  \fi}
\long\def\@ifnextchar#1#2#3{%
  \let\reserved@d=#1%
  \def\reserved@a{#2}%
  \def\reserved@b{#3}%
  \futurelet\@let@token\@ifnch}
\def\@ifnch{%
  \ifx\@let@token\reserved@d
    \expandafter\reserved@a
  \else
    \expandafter\reserved@b
  \fi}
⟨/2ekernel⟩
```

If we are doing a rollback with a format containing expl3 we aren't reloading it as that creates havoc. This may need a refined version!

```
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                    {expl3}{Pre-load expl3}%
\expandafter\ifx\csname tex\string _let:D\endcsname\relax
  \expandafter\@firstofone
\else
  \GenericInfo{}{Skipping: expl3 code already part of the format}%
⟨2ekernel⟩  \expandafter\endinput
⟨latexrelease⟩  \expandafter\@gobble
\fi
```

Check for the required primitive/engine support and the existence of a loader.

```
{%
  \IfFileExists{expl3.ltx}
    {%
      \ifnum0%
        \ifdefined\pdffilesize 1\fi
        \ifdefined\filesize 1\fi
        \ifdefined\luatexversion\ifnum\luatexversion>94 1\fi\fi
        \ifdefined\kanjiskip 1\fi
          >0 %
        \expandafter\@firstofone
      \else
```

In `2ekernel` mode, an error is fatal and building the format is aborted. Use `\batchmode \read -1 to \tokenlist`, which errors with

```
! Emergency stop. (cannot \read from terminal in nonstop modes)
```

and aborts the TEX run. In `latexrelease` mode, raise an error and do nothing. Both ways, the error message shows the minimum expl3 engine requirements.

```
⟨2ekernel⟩          \def~{ }\def\MessageBreak{^^J~~~~~~~~~~~~~~~~}%
⟨2ekernel⟩          \errmessage{LaTeX Error:
⟨latexrelease⟩        \@latex@error{%
            LaTeX requires the e-TeX primitives and additional\MessageBreak
            functionality available in the engines:\MessageBreak
              - pdfTeX v1.40\MessageBreak
              - XeTeX v0.99992\MessageBreak
              - LuaTeX v0.95\MessageBreak
              - e-(u)pTeX mid-2012\MessageBreak
            or later%
⟨latexrelease⟩        }\@ehd \expandafter\@gobble
⟨2ekernel⟩          }\batchmode \read -1 to \reserved@a
        \fi
      }
      {%
⟨*2ekernel⟩
        \errmessage{LaTeX requires expl3}%
        \batchmode \read -1 to \reserved@a
⟨/2ekernel⟩
```

We do not support a roll forward across 2019. You need to start with 2019 if you want to get to 2020 or beyond.

```
⟨*latexrelease⟩
        \@latex@warning@no@line
          {You need a format that already contains a recent\MessageBreak
           expl3 as part of the kernel, e.g. at least a kernel\MessageBreak
           from 2019 to roll forward to that date!\MessageBreak
           --- I'm giving up!\MessageBreak\MessageBreak
           Note that manually loading the expl3 package\MessageBreak
           from your distribution is not enough}%
        \batchmode \read -1 to \reserved@a
⟨/latexrelease⟩
      }%
      {\input expl3.ltx }%
  }
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩
```

To support roll-forward for the case where xparse is fully integrated into the kernel, we do not need to repeat the complex test above as we can simply look for the marker command.

```
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩                  {expl3}{Pre-load expl3}%
⟨latexrelease⟩\IfFileExists{expl3.ltx}
⟨latexrelease⟩  {%
⟨latexrelease⟩    \ifnum0%
⟨latexrelease⟩      \ifdefined\pdffilesize 1\fi
⟨latexrelease⟩      \ifdefined\filesize 1\fi
⟨latexrelease⟩      \ifdefined\luatexversion\ifnum\luatexversion>94 1\fi\fi
⟨latexrelease⟩      >0 %
⟨latexrelease⟩    \else
⟨latexrelease⟩      \message{Skipping expl3-dependent extensions}
⟨latexrelease⟩      \expandafter\@gobbletwo
⟨latexrelease⟩    \fi
⟨latexrelease⟩  }
⟨latexrelease⟩  {%
⟨latexrelease⟩    \message{Skipping expl3-dependent extensions}%
⟨latexrelease⟩    \@gobbletwo
⟨latexrelease⟩  }%
⟨latexrelease⟩\input{expl3.ltx}
⟨latexrelease⟩\EndIncludeInRelease
```

We need to be sure that expl3 is sufficiently new that all of the required functions exist. As the mechanism for date checking isn't in place yet, that is done using a marker function. We use the same abort trick as earlier.

```
\ifcsname\detokenize{tl_if_head_eq_charcode:eNTF}\endcsname
\else
  \def~{ }\def\MessageBreak{^^J~~~~~~~~~~~~~~~}%
  \errmessage{LaTeX Error:
    L3 Programming layer too old.\MessageBreak
    You need to update your installation of 'l3kernel'.\MessageBreak
    LaTeX will abort!}%
  \batchmode \read -1 to \reserved@a
\fi
```

Now in `latexrelease` mode, redefine a few commands to avoid "already defined" errors.

```
⟨latexrelease⟩\@ifundefined{ExplSyntaxOff}{}{\latexrelease@postltexpl}
```

1.2 Using expl3 code

In order to ease the implementation of some new features in LaTeX 2ε we may (temporarily) use some coding based on the expl3-code. Such macros will eventually vanish and may be changed unannounced. They are there for internal use in the LaTeX 2ε kernel and are not meant to be used in third-party packages. These macros will always have the `@expl@` prefix in their name.

The rest of the name matches the expl3 name but with all underscores replaced by `@`s and the `:` replaced by `@@`, e.g.,

```
\cs_new_eq:NN \@expl@tl@trim@spaces@apply@@nN \tl_trim_spaces_apply:nN
```

if that expl3 command is needed in places that are others coded in LaTeX 2_ε conventions.

In this file, each release of LaTeX adds an `\IncludeInRelease` block, in which the macros copied for that release were defined. In case a rollback is requested, the entire block is changed.

Each macro copied has a `\changes` entry to explain when and why it was copied, so that further to that may spot it easily.

Here `\cs_gset_eq:NN` is used, instead of the `new` variant because if different releases use that same name for different purposes, each can copy the macro without worrying about redefinitions.

```
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\@expl@cs@to@str@@N}%
⟨latexrelease⟩        {expl3 macros added for the 2020-10-01 release}%
```

The expl3 activation needs to be inside the release guards as otherwise rolling forward is broken in old kernels that do not have expl3 loaded.

```
\ExplSyntaxOn

\cs_gset_eq:NN \@expl@cs@to@str@@N \cs_to_str:N
\cs_gset_eq:NN \@expl@str@if@eq@@nnTF \str_if_eq:nnTF

\cs_gset_eq:NN \@expl@cs@prefix@spec@@N \cs_prefix_spec:N
\cs_if_exist:NTF \cs_parameter_spec:N
  { \cs_gset_eq:NN \@expl@cs@parameter@spec@@N \cs_parameter_spec:N }
  { \cs_gset_eq:NN \@expl@cs@parameter@spec@@N \cs_argument_spec:N }
\cs_gset_eq:NN \__kernel_cs_parameter_spec:N \@expl@cs@parameter@spec@@N
\cs_gset_eq:NN \@expl@cs@replacement@spec@@N \cs_replacement_spec:N

\cs_gset_eq:NN \@expl@str@map@function@@NN \str_map_function:NN
\cs_gset_eq:NN \@expl@char@generate@@nn \char_generate:nn

\ExplSyntaxOff
```

Here we can't assume that expl3 is available. It will be if we roll back but if this code is executed rolling forward it needs to be pure 2e.

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\@expl@cs@to@str@@N}%
⟨latexrelease⟩        {expl3 macros added for the 2020-10-01 release}%
⟨latexrelease⟩\let \@expl@cs@to@str@@N \@undefined
⟨latexrelease⟩\let \@expl@str@if@eq@@nnTF \@undefined
⟨latexrelease⟩\let \@expl@cs@prefix@spec@@N \@undefined
⟨latexrelease⟩\let \@expl@cs@parameter@spec@@N \@undefined
⟨latexrelease⟩\let \@expl@cs@replacement@spec@@N \@undefined
⟨latexrelease⟩\let \@expl@str@map@function@@NN \@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨/2ekernel | latexrelease⟩
```

2 Document-level command names for expl3 functions

Current home for L3 programming layer functions that we make directly available at the document level. This section may need to be moved later (after `\NewDocumentCommand` is defined in case we want to use that in the setup).

`\fpeval` The expandable command `\fpeval` takes as its argument a floating point expres-

sion and produces a result using the normal rules of mathematics. As this command is expandable it can be used where TEX requires a number and for example within a low-level `\edef` operation to give a purely numerical result. See `usrguide3` for further explanation.

`\inteval` `\dimeval` `\skipeval`

The expandable command `\inteval` takes as its argument an integer expression and produces a result using the normal rules of mathematics. The operations recognized are `+`, `-`, `*` and `/` plus parentheses. Division occurs with *rounding*, and ties are rounded away from zero. As this command is expandable it can be used where TEX requires a number and for example within a low-level `\edef` operation to give a purely numerical result. See `usrguide3` for further explanation. `\dimeval` and `\skipeval` are similar, but generate fixed and rubber length values, respectively.

`\fpeval` `\inteval` `\dimeval` `\skipeval`

A document level wrapper around the code level function for floating point calculations.

```
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2022/06/01}%
⟨latexrelease⟩                    {\fpeval}{fp and int calculations}%
\ExplSyntaxOn
\cs_new_eq:NN \fpeval \fp_eval:n
```

And a few more, this time wrappers around the eTEX primitives.

```
\cs_new_eq:NN \inteval \int_eval:n

\cs_new_eq:NN \dimeval  \dim_eval:n
\cs_new_eq:NN \skipeval \skip_eval:n
\ExplSyntaxOff
```

(*End of definition for* `\fpeval` *and others.*)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\fpeval}{fp and int calculations}%
⟨latexrelease⟩
⟨latexrelease⟩\let\fpeval\@undefined
⟨latexrelease⟩\let\inteval\@undefined
⟨latexrelease⟩\let\dimeval\@undefined
⟨latexrelease⟩\let\skipeval\@undefined
⟨latexrelease⟩\EndIncludeInRelease
```

`\UseName` `\ExpandArgs`

When declaring new commands with `\NewDocumentCommand` or `\NewCommandCopy` or similar, it is sometimes necessary to "construct" the csname. As a general mechanism the L3 programming layer has `\exp_args:N...` for this, but there is no mechanism for it if `\ExplSyntaxOn` is not active. We therefore offer a few of these commands also with CamelCase names.

`\UseName` `\ExpandArgs`

A document wrapper for changing arguments to cs names for use with `\NewDocumentCommand` and similar functions.

```
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2022/06/01}%
⟨latexrelease⟩                    {\ExpandArgs}{Some pre-expansion commands}%
\ExplSyntaxOn
\cs_new_eq:NN \UseName \use:c
```

```
\cs_new:Npn \ExpandArgs #1
  {
    \cs_if_exist_use:cF { exp_args:N #1 }
      { \msg_expandable_error:nnn { kernel } { unknown-arg-expansion } {#1} }
  }
\msg_new:nnn { kernel } { unknown-arg-expansion }
  { Unknown~arg~expansion~"#1" }
\ExplSyntaxOff
```

(*End of definition for* `\UseName` *and* `\ExpandArgs`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\ExpandArgs}{Some pre-expansion commands}%
⟨latexrelease⟩
⟨latexrelease⟩\let\UseName\@undefined
⟨latexrelease⟩\let\ExpandArgs\@undefined
⟨latexrelease⟩\EndIncludeInRelease
```

`\IfExplAtLeastTF` `\IfExplAtLeastT` `\IfExplAtLeastF`

A pretty simple set of wrappers. `\IfExplAtLeastTF` was already introduced in 2023 but since we pretend that these commands were always there, even in rollback, things can be kept simple and we just alter the rollback date.

`\IfExplAtLeastTF`

```
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease⟩                  {\IfExplAtLeastTF}{Test for expl3 date}%
\def\IfExplAtLeastTF{\@ifl@t@r\ExplLoaderFileDate}
\def\IfExplAtLeastT#1#2{\IfExplAtLeastTF{#1}{#2}\@firstofone\@gobble}
\def\IfExplAtLeastF#1{\IfExplAtLeastTF{#1}{}}
```

(*End of definition for* `\IfExplAtLeastTF`.)

We make sure the commands are always available.

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\IfExplAtLeastTF}{Test for expl3 date}%
⟨latexrelease⟩
⟨latexrelease⟩\def\IfExplAtLeastTF{\@ifl@t@r\ExplLoaderFileDate}
⟨latexrelease⟩\def\IfExplAtLeastT#1#2{\IfExplAtLeastTF{#1}{#2}\@firstofone\@gobble}
⟨latexrelease⟩\def\IfExplAtLeastF#1{\IfExplAtLeastTF{#1}{}}
⟨latexrelease⟩\EndIncludeInRelease
```

`\expandableinput`

```
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease⟩                  {\expandableinput}{Expandable input}%
\ExplSyntaxOn
\cs_new_eq:NN \expandableinput \file_input_raw:n
\ExplSyntaxOff
```

(*End of definition for* `\expandableinput`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\expandableinput}{Expandable input}%
⟨latexrelease⟩
⟨latexrelease⟩\let\expandableinput\@undefined
⟨latexrelease⟩\EndIncludeInRelease
```

File 06
ltdefns.dtx

1 Definitions

This section contains commands used in defining other macros.

```
⟨*2ekernel⟩
```

1.1 Initex initializations

`\two@digits` Prefix a number less than 10 with '0'.

```
\def\two@digits#1{\ifnum#1<10 0\fi\number#1}
```

(*End of definition for* `\two@digits`.)

`\typeout` Display something on the terminal.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\typeout}{Allow "par" in \typeout}%
\protected\long\def\typeout#1{\begingroup
  \set@display@protect
  \def\par{^^J^^J}%
  \immediate\write\@unused{#1}\endgroup}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\typeout}{Allow "par" in \typeout}%
⟨latexrelease⟩
⟨latexrelease⟩\def\typeout#1{\begingroup\set@display@protect
⟨latexrelease⟩    \immediate\write\@unused{#1}\endgroup}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\typeout`.)

`\newlinechar` A char to be used as new-line in output to files.

```
\newlinechar`\^^J
```

(*End of definition for* `\newlinechar`.)

1.2 Saved versions of TEX primitives

The TeX primitive `\foo` is saved as `\@@foo`. The following primitives are handled in this way:

`\@@par`

```
\let\@@par=\par
%\let\@@input=\input    %%% moved earlier
%\let\@@end=\end        %%%
```

(*End of definition for* `\@@par`.)

\@@hyph Save original primitive definition.

```
\let\@@hyph=\-
```

(*End of definition for* `\@@hyph`.)

\@@italiccorr Save the original italic correction.

```
\let\@@italiccorr=\/
```

(*End of definition for* `\@@italiccorr`.)

\@height \@depth \@width \@minus \@plus The following definitions save token space. E.g., using `\@height` instead of height saves 5 tokens at the cost in time of one macro expansion.

```
\def\@height{height} \def\@depth{depth} \def\@width{width}
\def\@minus{minus}
\def\@plus{plus}
```

The next one is another 100 tokens worth.

```
\def\hb@xt@{\hbox to}
```

(*End of definition for* `\@height` *and others.*)

```
\message{hacks,}
```

\hb@xt@

1.3 Command definitions

This section defines the following commands:

\@namedef `{`⟨*NAME*⟩`}`
Expands to `\def\`⟨*NAME*⟩, except name can contain any characters.

\@nameuse `{`⟨*NAME*⟩`}`
Expands to `\`⟨*NAME*⟩.

\@ifnextchar X`{`⟨*YES*⟩`}{`⟨*NO*⟩`}`
Expands to ⟨*YES*⟩ if next character is an 'X', and to ⟨*NO*⟩ otherwise. (Uses `\reserved@a`–`\reserved@c`.) NOTE: GOBBLES ANY SPACE FOLLOWING IT.

\@ifstar `{`⟨*YES*⟩`}{`⟨*NO*⟩`}`
Gobbles following spaces and then tests if next the character is a '*'. If it is, then it gobbles the '*' and expands to ⟨*YES*⟩, otherwise it expands to ⟨*NO*⟩.

\@dblarg `{`⟨*CMD*⟩`}{`⟨*ARG*⟩`}`
Expands to `\{`⟨*CMD*⟩`}[`⟨*ARG*⟩`]{`⟨*ARG*⟩`}`. Use `\@dblarg\CS` when `\CS` takes arguments `[ARG1]{ARG2}`, where default is `ARG1` = `ARG2`.

\@ifundefined `{`⟨*NAME*⟩`}{`⟨*YES*⟩`}{`⟨*NO*⟩`}`
: If `\NAME` is undefined then it executes ⟨*YES*⟩, otherwise it executes ⟨*NO*⟩. More precisely, true if `\NAME` either undefined or = `\relax`.

\@ifdefinable `\NAME{`⟨*YES*⟩`}` Executes ⟨*YES*⟩ if the user is allowed to define `\NAME`, otherwise it gives an error. The user can define `\NAME` if `\@ifundefined{NAME}` is true, '`NAME`' ≠ '`relax`' and the first three letters of '`NAME`' are not '`end`', and if `\endNAME` is not defined.

\newcommand `*{\FOO}[`⟨*i*⟩`]{`⟨*TEXT*⟩`}`
User command to define `\FOO` to be a macro with i arguments (i = 0 if missing) having the definition ⟨*TEXT*⟩. Produces an error if `\FOO` already defined.

Normally the command is defined to be `\long` (ie it may take multiple paragraphs in its argument). In the star-form, the command is not defined as `\long` and a blank line in any argument to the command would generate an error.

\renewcommand `*{\FOO}[`⟨*i*⟩`]{`⟨*TEXT*⟩`}`

Same as `\newcommand`, except it checks if `\FOO` already defined.

`\newenvironment` `*{`⟨*FOO*⟩`}[`⟨*i*⟩`]{`⟨*DEF1*⟩`}{`⟨*DEF2*⟩`}`

equivalent to:
`\newcommand{\FOO}[i]{DEF1} \def{\endFOO}{DEF2}`
(or the appropriate star forms).

`\renewenvironment`

Obvious companion to `\newenvironment`.

`\@cons` : See description of `\output` routine.

`\@car` `\@car T1 T2 ... Tn\@nil` == `T1` (unexpanded)

`\@cdr` `\@cdr T1 T2 ... Tn\@nil` == `T2 ... Tn` (unexpanded)

`\typeout` `{`⟨*message*⟩`}`

Produces a warning message on the terminal.

`\typein` `{`⟨*message*⟩`}`

Types message, asks the user to type in a command, then executes it

`\typein` `[`⟨`\CS`⟩`]{`⟨*MSG*⟩`}`

Same as above, except defines `\CS` to be the input instead of executing it.

`\typein`

```
\def\typein{%
  \let\@typein\relax
  \@testopt\@xtypein\@typein}
\ifx\directlua\@undefined
\def\@xtypein[#1]#2{%
 \typeout{#2}%
 \advance\endlinechar\@M
 \read\@inputcheck to#1%
 \advance\endlinechar-\@M
 \@typein}%
\else
\def\@xtypein[#1]#2{%
  \typeout{#2}%
  \begingroup \endlinechar\m@ne
  \read\@inputcheck to#1%
  \expandafter\endgroup
  \expandafter\def\expandafter#1\expandafter{#1}%
  \@typein}%
\fi
```

(*End of definition for* `\typein`.)

`\@namedef`

```
\def\@namedef#1{\expandafter\def\csname #1\endcsname}
```

(*End of definition for* `\@namedef`.)

`\@nameuse`

```
\def\@nameuse#1{\csname #1\endcsname}
```

(*End of definition for* `\@nameuse`.)

\@cons

```
\def\@cons#1#2{\begingroup\let\@elt\relax\xdef#1{#1\@elt #2}\endgroup}
```

(*End of definition for* `\@cons`.)

\@car
\@cdr

```
\def\@car#1#2\@nil{#1}
\def\@cdr#1#2\@nil{#2}
```

(*End of definition for* `\@car` *and* `\@cdr`.)

\@carcube `\@carcube T1 ... Tn\@nil` = `T1 T2 T3` , $n > 3$

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\@carcube}{Make \@carcube long}%
⟨*2ekernel | latexrelease⟩
\long\def\@carcube#1#2#3#4\@nil{#1#2#3}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
%
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\@carcube}{Undo: Make \@carcube long}%
⟨latexrelease⟩\def\@carcube#1#2#3#4\@nil{#1#2#3}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@carcube`.)

\@onlypreamble
\@preamblecmds

This macro adds its argument to the list of commands stored in `\@preamblecmds` to be disabled after `\begin{document}`. These commands are redefined to generate `\@notprerr` at this point.

```
\def\@preamblecmds{}
\def\@onlypreamble#1{%
  \expandafter\gdef\expandafter\@preamblecmds\expandafter{%
       \@preamblecmds\do#1}}
\@onlypreamble\@onlypreamble
\@onlypreamble\@preamblecmds
```

(*End of definition for* `\@onlypreamble` *and* `\@preamblecmds`.)

\@star@or@long

Look ahead for a `*`. If present reset `\l@ngrel@x` so that the next definition, `#1`, will be non-long.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/11/01}%
⟨latexrelease⟩    {\@star@or@long}{Macros without args are short}%
\def\@star@or@long#1{%
```

By default commands defined with `\newcommand` and friends are not `\protected` so `\pr@tectedrel@x` is set to `\relax`.

```
  \let\pr@tectedrel@x\relax
  \@ifstar
   {\let\l@ngrel@x\relax#1}%
   {\let\l@ngrel@x\long#1}}
```

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩    {\@star@or@long}{Macros without args are short}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@star@or@long#1{%
⟨latexrelease⟩   \@ifstar
⟨latexrelease⟩    {\let\l@ngrel@x\relax#1}%
⟨latexrelease⟩    {\let\l@ngrel@x\long#1}}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@star@or@long`.)

`\l@ngrel@x` This is either `\relax` or `\long` depending on whether the `*`-form of a definition command is being executed.

```
\let\l@ngrel@x\relax
```

(*End of definition for* `\l@ngrel@x`.)

`\pr@tectedrel@x` Same for the protection status.

```
\let\pr@tectedrel@x\relax
```

(*End of definition for* `\pr@tectedrel@x`.)

`\newcommand` User level `\newcommand`.

```
\def\newcommand{\@star@or@long\new@command}
```

`\new@command`

```
\def\new@command#1{%
  \@testopt{\@newcommand#1}0}
```

(*End of definition for* `\newcommand` *and* `\new@command`.)

`\@newcommand` `\@argdef` `\@xargdef` Handling arguments for `\newcommand`.

```
\def\@newcommand#1[#2]{%
  \kernel@ifnextchar [{\@xargdef#1[#2]}%
                     {\@argdef#1[#2]}}
```

Define `#1` if it is definable.

Both here and in `\@xargdef` the replacement text is absorbed as an argument because if we are not allowed to make the definition we have to get rid of it completely.

```
\long\def\@argdef#1[#2]#3{%
   \@ifdefinable #1{\@yargdef#1\@ne{#2}{#3}}}
```

Handle the second optional argument.

```
\long\def\@xargdef#1[#2][#3]#4{%
  \@ifdefinable#1{%
```

Define the actual command to be:
`\def\foo{\@protected@testopt\foo\\foo{default}}`
where `\\foo` is a csname generated from applying `\csname` and `\string` to `\foo`, ie the actual name contains a backslash and therefore can't clash easily with existing command names. "Default" is the contents of the second optional argument of `(re)newcommand`.

```
\expandafter\def\expandafter#1\expandafter{%
    \expandafter
    \@protected@testopt
    \expandafter
    #1%
    \csname\string#1\endcsname
    {#3}}%
```

Now we define the internal macro ie `\\foo` which is supposed to pick up all arguments (optional and mandatory).

```
\expandafter\@yargdef
    \csname\string#1\endcsname
     \tw@
     {#2}%
     {#4}}}
```

(*End of definition for* `\@newcommand`, `\@argdef`, *and* `\@xargdef`.)

`\@testopt` This macro encapsulates the most common call to `\@ifnextchar`, saving several tokens each time it is used in the definition of a command with an optional argument. `#1` The code to execute in the case that there is a `[` need not be a single token but can be any sequence of commands that 'expects' to be followed by `[`. If this command were only used in `\newcommand` definitions then `#1` would be a single token and the braces could be omitted from `{#1}` in the definition below, saving a bit of memory.

```
\long\def\@testopt#1#2{%
  \kernel@ifnextchar[{#1}{#1[{#2}]}}
```

(*End of definition for* `\@testopt`.)

`\@protected@testopt` Robust version of `\@testopt`. The extra argument (`#1`) must be a single token. If protection is needed the call expands to `\protect` applied to this token, and the 2nd and 3rd arguments are discarded (by `\@x@protect`). Otherwise `\@testopt` is called on the 2nd and 3rd arguments.

This method of making commands robust avoids the need for using up two csnames per command, the price is the extra expansion time for the `\ifx` test.

```
\def\@protected@testopt#1{%
  \ifx\protect\@typeset@protect
    \expandafter\@testopt
  \else
    \@x@protect#1%
  \fi}
```

(*End of definition for* `\@protected@testopt`.)

`\@yargdef` `\@yargd@f` These generate a primitive argument specification, from a LaTeX `[`⟨*digit*⟩`]` form; in fact ⟨*digit*⟩ can be anything such that `\number` ⟨*digit*⟩ is single digit.

Reorganized slightly so that `\renewcommand{\reserved@a}[1]{foo}` works. I am not sure this is worth it, as a following `\newcommand` would over-write the definition of `\reserved@a`.

Recall that LaTeX2.09 goes into an infinite loop with
`\renewcommand[1]{\@tempa}{foo}`
(DPC 6 October 93).

Reorganized again (DPC 1999). Rather than make a loop to construct the argument spec by counting, just extract the required argument spec by using a delimited argument (delimited by the digit). This is faster and uses less tokens. The coding is slightly odd to preserve the old interface (using `#2 = \tw@` as the flag to surround the first argument with `[]`). But the new method did not allow for the number of arguments `#3` not being given as an explicit digit; hence (further expansion of this argument and use of) `\number` was added later in 1999.

It is not clear why these are still `\long`.

```
\long \def \@yargdef #1#2#3{%
  \ifx#2\tw@
    \def\reserved@b##11{[####1]}%
  \else
    \let\reserved@b\@gobble
  \fi
  \expandafter
    \@yargd@f \expandafter{\number #3}#1%
}
```

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/11/01}%
⟨latexrelease⟩    {\@yargd@f}{Macros without args are short}%
⟨latexrelease⟩
\long \def \@yargd@f#1#2{%
  \def \reserved@a ##1#1##2##{%
    \expandafter\def\expandafter#2\reserved@b ##1#1%
  }%
```

If the command needs to be `\protected` then `\pr@tectedrel@x` has the appropriate value at this point.

```
  \pr@tectedrel@x
```

If the command has no arguments then we don't want to to be `\long` (even if `\l@ngrel@x` has been set to `\long`).

```
  \ifnum#1>\z@ \l@ngrel@x \fi
  \reserved@a 0##1##2##3##4##5##6##7##8##9###1%
}
```

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩    {\@yargd@f}{Macros without args are short}%
⟨latexrelease⟩
⟨latexrelease⟩\long \def \@yargd@f#1#2{%
⟨latexrelease⟩  \def \reserved@a ##1#1##2##{%
⟨latexrelease⟩    \expandafter\def\expandafter#2\reserved@b ##1#1%
⟨latexrelease⟩    }%
⟨latexrelease⟩  \l@ngrel@x \reserved@a 0##1##2##3##4##5##6##7##8##9###1%
⟨latexrelease⟩}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@yargdef` *and* `\@yargd@f`.)

`\@reargdef`

```
\long\def\@reargdef#1[#2]{%
  \@yargdef#1\@ne{#2}}
```

(*End of definition for* `\@reargdef`.)

`\renewcommand` Check the command name is already used. If not give an error message. Then temporarily disable `\@ifdefinable` then call `\newcommand`. (Previous version `\let#1=\relax` but this does not work too well if `#1` is `\@temp`*a–e*.)

```
\def\renewcommand{\@star@or@long\renew@command}
```

`\renew@command`

```
\def\renew@command#1{%
  \begingroup
    \escapechar\m@ne\xdef\@gtempa{{\expandafter\string\@car#1?\@nil}}%
  \endgroup
  \expandafter\@ifundefined\@gtempa
     {\@latex@error{Command \string#1 undefined}\@ehc}%
     \relax
  \let\@ifdefinable\@rc@ifdefinable
  \new@command#1}
```

(*End of definition for* `\renewcommand` *and* `\renew@command`.)

`\@ifdefinable` Test if user is allowed to define a command.

`\@@ifdefinable`

`\@rc@ifdefinable`

```
\long\def\@ifdefinable #1#2{%
      \edef\reserved@a{\expandafter\@gobble\string #1}%
     \@ifundefined\reserved@a
         {\edef\reserved@b{\expandafter\@carcube \reserved@a xxx\@nil}%
          \ifx \reserved@b\@qend \@notdefinable\else
            \ifx \reserved@a\@qrelax \@notdefinable\else
              #2%
            \fi
          \fi}%
         \@notdefinable}
```

Saved definition of `\@ifdefinable`.

```
\let\@@ifdefinable\@ifdefinable
```

Version of `\@ifdefinable` for use with `\renewcommand`. Does not do the check this time, but restores the normal definition.

```
\long\def\@rc@ifdefinable#1#2{%
  \let\@ifdefinable\@@ifdefinable
  #2}
```

(*End of definition for* `\@ifdefinable`, `\@@ifdefinable`, *and* `\@rc@ifdefinable`.)

`\newenvironment` Define a new user environment. `#1` is the environment name. `#2#` Grabs all the tokens up to the first `{`. These will be any optional arguments. They are not parsed at this point, but are just passed to `\@newenv` which will eventually call `\newcommand`. Any optional arguments will then be parsed by `\newcommand` as it defines the command that executes the 'begin code' of the environment.

This #2# trick removed with version 1.2i as it fails if a { occurs in the optional argument. Now use \@ifnextchar directly.

```
\def\newenvironment{\@star@or@long\new@environment}
```

\new@environment

```
\def\new@environment#1{%
  \@testopt{\@newenva#1}0}
```

\@newenva

```
\def\@newenva#1[#2]{%
   \kernel@ifnextchar [{\@newenvb#1[#2]}{\@newenv{#1}{[#2]}}}
```

\@newenvb

```
\def\@newenvb#1[#2][#3]{\@newenv{#1}{[#2][{#3}]}}
```

(*End of definition for* \newenvironment *and others.*)

\renewenvironment Redefine an environment. For \renewenvironment disable \@ifdefinable and then call \newenvironment. It is OK to \let the argument to \relax here as there should not be a @temp... environment.

```
\def\renewenvironment{\@star@or@long\renew@environment}
```

\renew@environment

```
\def\renew@environment#1{%
  \@ifundefined{#1}%
     {\@latex@error{Environment #1 undefined}\@ehc
     }\relax
  \expandafter\let\csname#1\endcsname\relax
  \expandafter\let\csname end#1\endcsname\relax
  \new@environment{#1}}
```

(*End of definition for* \renewenvironment *and* \renew@environment.)

\@newenv The internal version of \newenvironment.

Call \newcommand to define the ⟨*begin-code*⟩ for the environment. \def is used for the ⟨*end-code*⟩ as it does not take arguments. (but may contain \pars)

Make sure that an attempt to define a 'graf' or 'group' environment fails by temporarily letting the undefined \... (begin code) to the definition of \end... and as a result we get an error if that has a definition.

```
\long\def\@newenv#1#2#3#4{%
  \@ifundefined{#1}%
    {\expandafter\let\csname#1\expandafter\endcsname
                          \csname end#1\endcsname}%
    \relax
  \expandafter\new@command
     \csname #1\endcsname#2{#3}%
     \l@ngrel@x\expandafter\def\csname end#1\endcsname{#4}}
```

(*End of definition for* \@newenv.)

\newif And here's a different sort of allocation: For example, \newif\iffoo creates \footrue, \foofalse to go with \iffoo.

```
\def\newif#1{%
  \count@\escapechar \escapechar\m@ne
    \let#1\iffalse
    \@if#1\iftrue
    \@if#1\iffalse
  \escapechar\count@}
```

\@if

```
\def\@if#1#2{%
  \expandafter\def\csname\expandafter\@gobbletwo\string#1%
                      \expandafter\@gobbletwo\string#2\endcsname
                         {\let#1#2}}
```

(*End of definition for* \newif *and* \@if.)

\providecommand \providecommand takes the same arguments as \newcommand, but discards them if #1 is already defined, Otherwise it just acts like \newcommand. This implementation currently leaves any discarded definition in \reserved@a (and possibly \\reserved@a) this wastes a bit of space, but it will be reclaimed as soon as these scratch macros are redefined.

```
\def\providecommand{\@star@or@long\provide@command}
```

\provide@command

```
\def\provide@command#1{%
  \begingroup
    \escapechar\m@ne\xdef\@gtempa{{\expandafter\string\@car#1?\@nil}}%
  \endgroup
  \expandafter\@ifundefined\@gtempa
    {\def\reserved@a{\new@command#1}}%
    {\def\reserved@a{\renew@command\reserved@a}}%
   \reserved@a}%
```

(*End of definition for* \providecommand *and* \provide@command.)

\CheckCommand \CheckCommand takes the same arguments as \newcommand. If the command already exists, with the same definition, then nothing happens, otherwise a warning is issued. Useful for checking the current state before a macro package starts redefining things. Currently two macros are considered to have the same definition if they are the same except for different default arguments. That is, if the old definition was: \newcommand\xxx[2][a]{(#1)(#2)} then \CheckCommand\xxx[2][b]{(#1)(#2)} would *not* generate a warning, but, for instance \CheckCommand\xxx[2]{(#1)(#2)} would.

```
\def\CheckCommand{\@star@or@long\check@command}
```

\CheckCommand is only available in the preamble part of the document.

```
\@onlypreamble\CheckCommand
```

\check@command

```
\def\check@command#1#2#{\@check@c#1{#2}}
\@onlypreamble\check@command
```

(*End of definition for* \CheckCommand *and* \check@command.)

\@check@c

\CheckCommand itself just grabs all the arguments we need, without actually looking for [optional argument forms. Now define \reserved@a. If \\reserved@a is then defined, compare it with the "\#1' otherwise compare \reserved@a with #1.

```
\long\def\@check@c#1#2#3{%
  \expandafter\let\csname\string\reserved@a\endcsname\relax
  \renew@command\reserved@a#2{#3}%
  \@ifundefined{\string\reserved@a}%
   {\@check@eq#1\reserved@a}%
   {\expandafter\@check@eq
           \csname\string#1\expandafter\endcsname
           \csname\string\reserved@a\endcsname}}
\@onlypreamble\@check@c
```

(*End of definition for* \@check@c.)

\@check@eq

Complain if #1 and #2 are not \ifx equal.

```
\def\@check@eq#1#2{%
  \ifx#1#2\else
     \@latex@warning@no@line
               {Command \noexpand#1 has
                changed.\MessageBreak
                Check if current package is valid}%
  \fi}
\@onlypreamble\@check@eq
```

(*End of definition for* \@check@eq.)

\@gobble
\@gobbletwo
\@gobblethree
\@gobblefour

The \@gobble macro is used to get rid of its argument.

```
\long\def \@gobble #1{}
\long\def \@gobbletwo #1#2{}
\long\def \@gobblethree #1#2#3{}
\long\def \@gobblefour #1#2#3#4{}
```

(*End of definition for* \@gobble *and others.*)

There are also \@gobble@om, \@gobble@som, \@gobble@with@sphack@om, and \@gobble@with@sphack@som. They accept an optional and a mandatory argument, possibly preceded by a star. In all cases the expansion is empty or just manipulates the spaces around the command. Used to disable commands such as \index or \label in certain situations. Since they are defined with \DeclareDocumentCommand, which is not yet available at this point, the actual definition happens in ltsect.dtx.

\@firstofone
\@firstoftwo
\@secondoftwo

Some argument-grabbers.

```
\long\def\@firstofone#1{#1}
\long\def\@firstoftwo#1#2{#1}
\long\def\@secondoftwo#1#2{#2}
```

\@iden

\@iden is another name for \@firstofone for compatibility reasons.

```
\let\@iden\@firstofone
```

(*End of definition for* \@firstofone *and others.*)

\@thirdofthree

Another grabber now used in the encoding specific section.

```
\long\def\@thirdofthree#1#2#3{#3}
```

(*End of definition for* `\@thirdofthree`.)

`\@expandtwoargs` A macro to totally expand two arguments to another macro

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2022/11/01}%
⟨latexrelease⟩          {\@expandtwoargs}{protected edef}%
⟨*2ekernel | latexrelease⟩
\def\@expandtwoargs#1#2#3{%
\protected@edef\reserved@a{\noexpand#1{#2}{#3}}\reserved@a}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{00/00/00}%
⟨latexrelease⟩          {\@expandtwoargs}{protected edef}%
⟨latexrelease⟩\def\@expandtwoargs#1#2#3{%
⟨latexrelease⟩\edef\reserved@a{\noexpand#1{#2}{#3}}\reserved@a}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@expandtwoargs`.)

`\@backslashchar` A category code 12 backslash.

```
\edef\@backslashchar{\expandafter\@gobble\string\\}
```

(*End of definition for* `\@backslashchar`.)

1.4 Robust commands and protect

Fragile and robust commands are one of the thornier issues in LaTeX's commands. Whilst typesetting documents, LaTeX makes use of many of TeX's features, such as arithmetic, defining macros, and setting variables. However, there are (at least) three different occasions when these commands are not safe. These are called 'moving arguments' by LaTeX, and consist of:

- writing information to a file, such as indexes or tables of contents.
- writing information to the screen.
- inside an `\edef`, `\message`, `\mark`, or other command which evaluates its argument fully.

The method LaTeX uses for making fragile commands robust is to precede them with `\protect`. This can have one of four possible values:

- `\relax`, for normal typesetting. So `\protect\foo` will execute `\foo`.
- `\string`, for writing to the screen. So `\protect\foo` will write `\foo`.
- `\noexpand`, for writing to a file. So `\protect\foo` will write `\foo` followed by a space.
- `\@unexpandable@protect`, for writing a moving argument to a file. So `\protect\foo` will write `\protect\foo` followed by a space. This value is also used inside `\edef`s, `\mark`s and other commands which evaluate their arguments fully. More precisely, whenever the content of an `\edef` or `\xdef` etc. can contain arbitrary user input not under the direct control of the programmer, one should use `\proetected@edef` instead of `\edef`, etc., so that `\protect` has a suitable definition and the user input will not break if it contains fragile commands.

\@unexpandable@protect

```
\def\@unexpandable@protect{\noexpand\protect\noexpand}
```

(*End of definition for* `\@unexpandable@protect`.)

\DeclareRobustCommand
\declare@robustcommand
\declare@robustcommand@auxi
\declare@robustcommand@auxii
\declare@robustcommand@auxiii

This is a package-writers command, which has the same syntax as `\newcommand`, but which declares a protected command. It does this by having

`\DeclareRobustCommand\foo`

define `\foo` to be `\protect\foo<space>`,

and then use `\newcommand\foo<space>`.

Since the internal command is `\foo<space>`, when it is written to an auxiliary file, it will appear as `\foo`.

We have to be a bit cleverer if we're defining a short command, such as `\_`, in order to make sure that the auxiliary file does not include a space after the command, since `\_ a` and `\_a` aren't the same. In this case we define `\_` to be:

```
\x@protect\_\protect\_<space>
```

which expands to:

```
\ifx\protect\@typeset@protect\else
   \@x@protect@\_
\fi
\protect\_<space>
```

Then if `\protect` is `\@typeset@protect` (normally `\relax`) then we just perform `\_<space>`, and otherwise `\@x@protect@` gobbles everything up and expands to `\protect\_`. None of that works with an active char (as we have only one character token to play with), so we resort to using the engine mechanism there. (The braces in passing `#1` there are defensive in case someone has passed more than one token!)

Note: setting `\protect` to any value other than `\relax` whilst in 'typesetting' mode will cause commands to go into an infinite loop! In particular, setting `\protect` to `\@empty` will cause `\_` to loop forever. It will also break lots of other things, such as protected `\ifmmode`s inside `\halign`s. If you really have to do such a thing, then please set `\@typeset@protect` to be `\@empty` as well. (This is what the code for `\patterns` does, for example.)

More fun with `\expandafter` and `\csname`.

```
\def\DeclareRobustCommand{\@star@or@long\declare@robustcommand}
\def\declare@robustcommand#1{%
   \ifx\@undefined#1\else\ifx\relax#1\else
      \@latex@info{Redefining \string#1}%
   \fi\fi
  \ifcat\noexpand~\noexpand#1%
    \expandafter\declare@robustcommand@auxi
  \else
    \expandafter\declare@robustcommand@auxiii
  \fi
    {#1}%
}
```

For active chars, we cannot use an auxiliary so have to 'pack' everything we need in one definition. To allow for use in file names, etc., we need `\ifincsname`] in the definition, so we use two steps to grab that. We also take advantage of the need for an extra auxiliary to do some expansion to give us the string we will want.

```
\protected\long\def\declare@robustcommand@auxi#1#2#{%
  \expandafter\declare@robustcommand@auxii\expandafter{\string#1}#1{#2}%
}
```

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/11/01}%
⟨latexrelease⟩       {\declare@robustcommand@auxii}{Macros without args are short}%
\protected\long\def\declare@robustcommand@auxii#1#2#3#4{%
```

Here we want a `\protected` command.

```
   \let\pr@tectedrel@x\protected
   \let\@ifdefinable\@rc@ifdefinable
   \new@command#2#3{%
       \ifincsname
         \expandafter\@firstoftwo
       \else
         \expandafter\@secondoftwo
      \fi
      {#1}{#4}%
    }%
}
```

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩    {\declare@robustcommand@auxii}{Macros without args are short}%
⟨latexrelease⟩
⟨latexrelease⟩\protected\long\def\declare@robustcommand@auxii#1#2#3#4{%
⟨latexrelease⟩   \ifx\l@ngrel@x\relax
⟨latexrelease⟩     \let\l@ngrel@x\protected
⟨latexrelease⟩   \else
⟨latexrelease⟩     \def\l@ngrel@x{\protected\long}%
⟨latexrelease⟩   \fi
⟨latexrelease⟩    \let\@ifdefinable\@rc@ifdefinable
⟨latexrelease⟩    \new@command#2#3{%
⟨latexrelease⟩        \ifincsname
⟨latexrelease⟩          \expandafter\@firstoftwo
⟨latexrelease⟩        \else
⟨latexrelease⟩          \expandafter\@secondoftwo
⟨latexrelease⟩       \fi
⟨latexrelease⟩       {#1}{#4}%
⟨latexrelease⟩     }%
⟨latexrelease⟩   }
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

```
\protected\def\declare@robustcommand@auxiii#1{%
   \edef\reserved@a{\string#1}%
   \def\reserved@b{#1}%
   \edef\reserved@b{\expandafter\strip@prefix\meaning\reserved@b}%
```

```
   \edef#1{%
      \ifx\reserved@a\reserved@b
         \noexpand\x@protect
         \noexpand#1%
      \fi
      \noexpand\protect
      \expandafter\noexpand\csname
         \expandafter\@gobble\string#1 \endcsname
   }%
   \let\@ifdefinable\@rc@ifdefinable
   \expandafter\new@command\csname
      \expandafter\@gobble\string#1 \endcsname
}
```

```
\def\x@protect#1{%
   \ifx\protect\@typeset@protect\else
      \@x@protect#1%
   \fi
}
\def\@x@protect#1\fi#2#3{%
   \fi\protect#1%
}
```

\@x@protect
\x@protect

(*End of definition for* `\DeclareRobustCommand` *and others.*)

\@typeset@protect

We set `\@typeset@protect` to `\relax` rather than `\@empty` to make sure that the protection mechanism stops the look-ahead and expansion performed at the start of `\halign` cells.

```
\let\@typeset@protect\relax
```

(*End of definition for* `\@typeset@protect`.)

\set@display@protect
\set@typeset@protect

These macros set `\protect` appropriately for typesetting or displaying.

```
\def\set@display@protect{\let\protect\string}
\def\set@typeset@protect{\let\protect\@typeset@protect}
```

(*End of definition for* `\set@display@protect` *and* `\set@typeset@protect`.)

\protected@edef
\protected@xdef
\unrestored@protected@xdef
\restore@protect

The commands `\protected@edef` and `\protected@xdef` perform 'safe' `\edef`s and `\xdef`s, saving and restoring `\protect` appropriately. For cases where restoring `\protect` doesn't matter, there's an 'unsafe' `\unrestored@protected@xdef`, useful if you know what you're doing!

```
\def\protected@edef{%
   \let\@@protect\protect
   \let\protect\@unexpandable@protect
   \afterassignment\restore@protect
   \edef
}
\def\protected@xdef{%
   \let\@@protect\protect
   \let\protect\@unexpandable@protect
   \afterassignment\restore@protect
   \xdef
```

```
}
\def\unrestored@protected@xdef{%
   \let\protect\@unexpandable@protect
   \xdef
}
\def\restore@protect{\let\protect\@@protect}
```

(*End of definition for* `\protected@edef` *and others.*)

`\protect` The normal meaning of `\protect`

```
\set@typeset@protect
```

(*End of definition for* `\protect`.)

`\MakeRobust` This macro makes an existing fragile macro robust, but only if it hasn't been robust in the past, i.e., it checks for the existence of the macro `\<name>␣` and if that exists it assumes that `\<name>` is already robust. In that case either undefine the inner macro first or use `\DeclareRobustCommand` to define it in a robust way directly. We could probably test the top-level definition to have the right kind of structure, but this is somewhat problematical as we then have to distinguish between `\long` macros and others and also take into account that sometimes the top-level is deliberately done manually (like with `\begin`).

The macro firstly checks if the control sequence in question exists at all.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\MakeRobust}{\MakeRobust}%
⟨*2ekernel | latexrelease⟩
\def\MakeRobust#1{%
  \count@=\escapechar
  \escapechar=`\\
  \@ifundefined{\expandafter\@gobble\string#1}{%
    \@latex@error{Command `\string#1' undefined.%
      \MessageBreak There is nothing here to make robust}%
    \@eha
  }%
```

Then we check if the macro is already robust. We do this by testing if the internal name for a robust macro is defined, namely `\foo␣`. If it is already defined do nothing, otherwise set `\foo␣` equal to `\foo` and redefine `\foo` so that it acts like a macro defined with `\DeclareRobustCommand`. We use `\@kernel@rename@newcommand` to copy `\foo` over to `\foo␣`, including a possible default optional argument.

```
  {%
    \@ifundefined{\expandafter\@gobble\string#1\space}%
    {%
      \expandafter\@kernel@rename@newcommand
        \csname\expandafter\@gobble\string#1\space\endcsname
        #1%
      \edef\reserved@a{\string#1}%
      \def\reserved@b{#1}%
      \edef\reserved@b{\expandafter\strip@prefix\meaning\reserved@b}%
      \xdef#1{%
        \ifx\reserved@a\reserved@b
          \noexpand\x@protect\noexpand#1%
        \fi
        \noexpand\protect\expandafter\noexpand
```

```
      \csname\expandafter\@gobble\string#1\space\endcsname}%
  }%
  {\@latex@info{Command `\string#1' is already robust}}%
 }%
 \escapechar=\count@
}%
```

This macro renames a command, possibly with an optional argument (defined with `\newcommand`) from `#2` to `#1`, by renaming the internal macro `\\#2` to `\\#1` and defining `\#1` appropriately, then undefining `\#2` and `\\#2`. The `\afterassignment` trick is to make both definitions in `\@copy@newcommand` global (which are local by default).

`\@kernel@rename@newcommand`

In case the macro was defined with `\newcommand` and an optional argument, to replicate exactly the behaviour of `\DeclareRobustCommand` we have to move also the internal `\\foo` to `\\foo␣`. In that case, `#1` will be a parameterless macro (`\robust@command@chk@safe` checks that), and `\@if@newcommand` will return true (both defined below in this file). If so, we can use `\@copy@newcommand` rather than plain `\let` to copy the command over. `\@kernel@rename@newcommand` does this test and carries out the renaming.

```
\def\@kernel@rename@newcommand#1#2{%
  \robust@command@chk@safe#2%
    {\@if@newcommand#2%
      {\afterassignment\global
       \global\@copy@newcommand#1#2%
       \global\let#2\@undefined
       \global\expandafter\let\csname\string#2\endcsname\@undefined}%
      {\global\let#1=#2}}%
    {\global\let#1=#2}}
```

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
%
⟨latexrelease⟩\IncludeInRelease{2019/10/01}{\MakeRobust}{\MakeRobust}%
⟨latexrelease⟩\def\MakeRobust#1{%
⟨latexrelease⟩  \@ifundefined{\expandafter\@gobble\string#1}{%
⟨latexrelease⟩    \@latex@error{The control sequence `\string#1' is undefined!%
⟨latexrelease⟩      \MessageBreak There is nothing here to make robust}%
⟨latexrelease⟩    \@eha
⟨latexrelease⟩  }%
⟨latexrelease⟩  {%
⟨latexrelease⟩    \@ifundefined{\expandafter\@gobble\string#1\space}%
⟨latexrelease⟩    {%
⟨latexrelease⟩      \global\expandafter\let\csname
⟨latexrelease⟩      \expandafter\@gobble\string#1\space\endcsname=#1%
⟨latexrelease⟩      \edef\reserved@a{\string#1}%
⟨latexrelease⟩      \def\reserved@b{#1}%
⟨latexrelease⟩      \edef\reserved@b{\expandafter\strip@prefix\meaning\reserved@b}%
⟨latexrelease⟩      \xdef#1{%
⟨latexrelease⟩        \ifx\reserved@a\reserved@b
⟨latexrelease⟩          \noexpand\x@protect\noexpand#1%
⟨latexrelease⟩        \fi
⟨latexrelease⟩        \noexpand\protect\expandafter\noexpand
⟨latexrelease⟩        \csname\expandafter\@gobble\string#1\space\endcsname}%
⟨latexrelease⟩    }%
```

```
⟨latexrelease⟩    {\@latex@info{The control sequence `\string#1' is already robust}}%
⟨latexrelease⟩  }%
⟨latexrelease⟩}%
⟨latexrelease⟩\let\@kernel@rename@newcommand\@undefined
⟨latexrelease⟩\EndIncludeInRelease
%
⟨latexrelease⟩\IncludeInRelease{2015/01/01}{\MakeRobust}{\MakeRobust}%
⟨latexrelease⟩\def\MakeRobust#1{%
⟨latexrelease⟩  \@ifundefined{\expandafter\@gobble\string#1}{%
⟨latexrelease⟩    \@latex@error{The control sequence `\string#1' is undefined!%
⟨latexrelease⟩      \MessageBreak There is nothing here to make robust}%
⟨latexrelease⟩    \@eha
⟨latexrelease⟩  }%
⟨latexrelease⟩  {%
⟨latexrelease⟩    \@ifundefined{\expandafter\@gobble\string#1\space}%
⟨latexrelease⟩    {%
⟨latexrelease⟩      \expandafter\let\csname
⟨latexrelease⟩      \expandafter\@gobble\string#1\space\endcsname=#1%
⟨latexrelease⟩      \edef\reserved@a{\string#1}%
⟨latexrelease⟩      \def\reserved@b{#1}%
⟨latexrelease⟩      \edef\reserved@b{\expandafter\strip@prefix\meaning\reserved@b}%
⟨latexrelease⟩      \edef#1{%
⟨latexrelease⟩        \ifx\reserved@a\reserved@b
⟨latexrelease⟩          \noexpand\x@protect\noexpand#1%
⟨latexrelease⟩        \fi
⟨latexrelease⟩        \noexpand\protect\expandafter\noexpand
⟨latexrelease⟩        \csname\expandafter\@gobble\string#1\space\endcsname}%
⟨latexrelease⟩    }%
⟨latexrelease⟩    {\@latex@info{The control sequence `\string#1' is already robust}}%
⟨latexrelease⟩  }%
⟨latexrelease⟩}%
⟨latexrelease⟩\let\@kernel@rename@newcommand\@undefined
⟨latexrelease⟩\EndIncludeInRelease
%
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\MakeRobust}{\MakeRobust}%
⟨latexrelease⟩\let\MakeRobust\@undefined
⟨latexrelease⟩\let\@kernel@rename@newcommand\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\MakeRobust` *and* `\@kernel@rename@newcommand`.)

`\kernel@make@fragile` The opposite of `\MakeRobust` except that it doesn't do many checks as it is internal to the kernel. Why does one want such a thing? Only for compatibility reasons if `latexrelease` requests a rollback of the kernel. For this reason we pretend that this command existed in all earlier versions of LaTeX i.e., we are not rolling it back since we need it precisely then. But we have to get it into the `latexrelease` file so that a roll forward is possible too.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                    {\kernel@make@fragile}{Undo robustness}%
\def\kernel@make@fragile#1{%
  \@ifundefined{\expandafter\@gobble\string#1\space}%
```

If not robust do nothing.

```
      {}%
```

Otherwise copy `\foo␣` back to `\foo`. Then use `\@kernel@rename@newcommand` to check and copy `\\foo␣` back to `\\foo` in case the command has an optional argument. If so, also undefine `\\foo␣`, and at the end undefine `\foo␣`.

```
      {%
       \global\expandafter\let\expandafter #1\csname
         \expandafter\@gobble\string#1\space\endcsname
       \expandafter\@kernel@rename@newcommand
         \csname\expandafter\@gobble\string#1\expandafter\endcsname
         \csname\expandafter\@gobble\string#1\space\endcsname
       \global\expandafter\let\csname
         \expandafter\@gobble\string#1\space\endcsname\@undefined
      }%
}
⟨latexrelease⟩\EndIncludeInRelease
%
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\kernel@make@fragile}{Undo robustness}%
⟨latexrelease⟩\def\kernel@make@fragile#1{%
⟨latexrelease⟩   \@ifundefined{\expandafter\@gobble\string#1\space}%
⟨latexrelease⟩      {}%
⟨latexrelease⟩      {%
⟨latexrelease⟩       \global\expandafter\let\expandafter #1\csname
⟨latexrelease⟩       \expandafter\@gobble\string#1\space\endcsname
⟨latexrelease⟩       \global\expandafter\let\csname
⟨latexrelease⟩       \expandafter\@gobble\string#1\space\endcsname\@undefined
⟨latexrelease⟩      }%
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨/2ekernel | latexrelease⟩
⟨*2ekernel⟩
```

(*End of definition for* `\kernel@make@fragile`.)

1.5 Acting on robust commands

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2020-10-01}{\robust@command@act}
⟨latexrelease⟩   {Add \robust@command@act}%
⟨*2ekernel | latexrelease⟩
```

With most document level commands being robust now there is more of a requirement to have a standard way of aliasing (or copying) a command to a new name, for example to save an original definition before changing a command. `\DeclareCommandCopy` is analogous to TEX's `\let`, except that it copes with the different types of robust commands defined by LATEX's mechanisms.

A couple of "types of robustness" are defined by the LATEX 2ε kernel, namely robust commands defined with `\DeclareRobustCommand` and commands with optional arguments defined with `\newcommand`. However there are other types of robust commands that are frequently used, which are not defined in the LATEX 2ε kernel, like commands defined with xparse's `\NewDocumentCommand` and etoolbox's `\newrobustcmd`.

In this section we will define a generic extensible machinery to act on robust commands. This code will then be used to test if a command is robust, considered the different types of robustness, and then either copy that definition, if `\DeclareCommandCopy` (or similar) is used, or show the definition of the command, if `\ShowCommand` is used.

`\robust@command@act`

The looping machinery is generic and knows nothing about what is to be done for each case. The syntax of the main macro `\robust@command@act` is:

`\robust@command@act`⟨*action-list*⟩⟨*robust-cmd*⟩
⟨*fallback-action*⟩⟨*act-arg*⟩

⟨*action-list*⟩ is a token list of the form:

{⟨*if-type-1*⟩ ⟨*act-type-1*⟩}
{⟨*if-type-2*⟩ ⟨*act-type-2*⟩}
...

`\robust@command@act` will iterate over the ⟨*action-list*⟩, evaluating each ⟨*if-type-n*⟩ ⟨*robust-cmd*⟩ {⟨*true*⟩}{⟨*false*⟩}. If the ⟨*if-type-n*⟩ conditional returns ⟨*true*⟩, then ⟨*act-type-n*⟩⟨*act-arg*⟩ is executed, and the loop ends. If the conditional returns ⟨*false*⟩, then ⟨*if-type-n + 1*⟩ is executed in the same way, until either one of the conditionals return ⟨*true*⟩, or the end of the ⟨*action-list*⟩ is reached. If the end is reached, then ⟨*fallback-action*⟩⟨*act-arg*⟩ is executed before `\robust@command@act` exits.

`\robust@command@act` will start by using `\robust@command@act@chk@args` to check if the ⟨*robust-cmd*⟩ (`#2`) is a parameterless (possibly `\protected`) macro. If it is not, the command is not a robust command: these always start with a parameterless user-level macro; in that case, `\robust@command@act@end` is used to short-circuit the process and do the ⟨*fallback-action*⟩ (`#3`). This first test is necessary because later on we need to be able to expand the ⟨*robust-cmd*⟩ without the risk of it Breaking Badly, and as a bonus, this speeds up the process in case we used `\NewCommandCopy` in a "normal" macro.

```
\long\def\robust@command@act#1#2#3#4{%
  \robust@command@chk@safe#2%
    {\expandafter\robust@command@act@loop
       \expandafter#2%
         #1{\@nnil\@nnil}%
     \robust@command@act@end}%
    {\robust@command@act@end}%
      {#3}{#4}}%
```

`\robust@command@act@loop`
`\robust@command@act@loop@aux`
`\robust@command@act@do`

If `\robust@command@act@chk@args` branched to false, then `\robust@command@act@loop` will loop over the list of items in the ⟨*action-list*⟩ (`#1`), and process each item as described earlier. If the ⟨*if-type-n*⟩ command expands to ⟨*true*⟩ then `\robust@command@act@do` is used to execute ⟨*act-type-n*⟩ on the ⟨*act-arg*⟩, otherwise the loop resumes with the next item.

```
\long\def\robust@command@act@loop#1#2{\robust@command@act@loop@aux#1#2}
\long\def\robust@command@act@loop@aux#1#2#3{%
  \ifx\@nnil#2%
  \else
    #2{#1}%
      {\robust@command@act@do{#3}}%
      {\expandafter\robust@command@act@loop\expandafter#1}%
  \fi}
```

```
\long\def\robust@command@act@do#1%
  \fi#2%
  \robust@command@act@end#3#4{%
  \fi
  #1#4}
```

If the end is reached and no action was taken, then do ⟨*fallback-action*⟩⟨*act-arg*⟩.

\robust@command@act@end
```
\long\def\robust@command@act@end#1#2{#1#2}
```

\robust@command@chk@safe
\robust@command@act@chk@args
```
\long\def\robust@command@chk@safe#1{%
  \begingroup
    \escapechar=`\\
  \expandafter\endgroup\expandafter
  \robust@command@act@chk@args\meaning#1:->\@nil}
\def\robust@command@act@chk@args#1:->#2\@nil{%
  \@expl@str@if@eq@@nnTF{#1}{macro}%
    {\@firstoftwo}%
    {\@expl@str@if@eq@@nnTF{#1}{\protected macro}%
      {\@firstoftwo}%
      {\@secondoftwo}}}
```

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000-00-00}{\robust@command@act}
⟨latexrelease⟩  {Add \robust@command@act}%
⟨latexrelease⟩\let\robust@command@act\@undefined
⟨latexrelease⟩\let\robust@command@act@loop\@undefined
⟨latexrelease⟩\let\robust@command@act@loop@aux\@undefined
⟨latexrelease⟩\let\robust@command@act@do\@undefined
⟨latexrelease⟩\let\robust@command@act@end\@undefined
⟨latexrelease⟩\let\robust@command@chk@safe\@undefined
⟨latexrelease⟩\let\robust@command@act@chk@args\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\robust@command@act` *and others.*)

1.5.1 Copying robust commands

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2020-10-01}{\DeclareCommandCopy}
⟨latexrelease⟩  {Add \NewCommandCopy, \RenewCommandCopy, and \DeclareCommandCopy}%
⟨*2ekernel | latexrelease⟩
```

\NewCommandCopy
\RenewCommandCopy
\DeclareCommandCopy

`\NewCommandCopy` starts by checking if `#1` is already defined, and raises an error if so, otherwise the definition is carried out. `\RenewCommandCopy` does (almost) the opposite. If the command is *not* defined, then an error is raised. But the definition is carried out anyhow, so the behaviour is consistent with `\renewcommand`.

A `\ProvideCommandCopy` isn't defined because it's not reasonably useful. `\provide...` commands mean "define this if there's no other definition", but copying a command (usually) implies that the command being copied is defined, so `\ProvideCommandCopy` doesn't make a lot of sense. But more importantly, the most common use case of copying a command is to redefine it later, while preserving the old definition, as in:

```
\ProvideCommandCopy \A \B
\renewcommand \B { ... \A ... }
```

then, if `\A` is already defined the first line is skipped, an in this case `\B` won't work as expected.

The three versions call the internal `\declare@commandcopy` with the proper action. `\@firstofone` will carry out the copy. The only case when the copy is not made is the ⟨*false*⟩ case for `\NewCommandCopy`, in which the command already exists and the definition is aborted.

```
\def\NewCommandCopy{%
  \declare@commandcopy
    {\@firstofone}%
    {\@firstoftwo\@notdefinable}}
\def\RenewCommandCopy{%
  \declare@commandcopy
    {\@latex@error{Command \@backslashchar\reserved@a\space undefined}\@ehc
     \@firstofone}%
    {\@firstofone}}
\def\DeclareCommandCopy{%
  \declare@commandcopy
    {\@firstofone}%
    {\@firstofone}}
```

Start by checking if the command is already defined. The proper action is taken by each specific command above. If all's good, then `\robust@command@act` is called with the proper arguments as described earlier, with `\@declarecommandcopylisthook` as the ⟨*action-list*⟩ and `\declare@commandcopy@let` as the ⟨*fallback-action*⟩.

`\declare@commandcopy`
`\declare@commandcopy@do`

```
\long\def\declare@commandcopy#1#2#3#4{%
  \edef\reserved@a{\@expl@cs@to@str@@N#3}%
  \@ifundefined\reserved@a{#1}{#2}%
    {\declare@commandcopy@do{#3}{#4}}}
\long\def\declare@commandcopy@do#1#2{%
  \robust@command@act
    \@declarecommandcopylisthook#2%
    \declare@commandcopy@let{#1#2}}
```

The initial definition of `\@declarecommandcopylisthook` contains the tests for the two types of robust command in the kernel.

`\@declarecommandcopylisthook`

```
\def\@declarecommandcopylisthook{%
  {\@if@DeclareRobustCommand \@copy@DeclareRobustCommand}%
  {\@if@newcommand \@copy@newcommand}}
```

The initial definition of `\@declarecommandcopylisthook` contains the tests for the two types of robust command in the kernel.

`\declare@commandcopy@let`

```
\long\def\declare@commandcopy@let#1#2{\let#1=#2\relax}
```

Now the rollback code.

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000-00-00}{\DeclareCommandCopy}
⟨latexrelease⟩  {Undefine \NewCommandCopy, \RenewCommandCopy, and \DeclareCommandCopy}%
⟨latexrelease⟩\let\NewCommandCopy\@undefined
```

```
⟨latexrelease⟩\let\RenewCommandCopy\@undefined
⟨latexrelease⟩\let\DeclareCommandCopy\@undefined
⟨latexrelease⟩\let\declare@commandcopy\@undefined
⟨latexrelease⟩\let\@declarecommandcopylisthook\@undefined
⟨latexrelease⟩\let\declare@commandcopy@let\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\NewCommandCopy` *and others.*)

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2023-06-01}{\DeclareEnvironmentCopy}
⟨latexrelease⟩  {Add \NewEnvironmentCopy, \RenewEnvironmentCopy, and \DeclareEnvironmentCopy}%
⟨*2ekernel | latexrelease⟩
```

`\NewEnvironmentCopy`
`\RenewEnvironmentCopy`
`\DeclareEnvironmentCopy`

If `\#1` or `\end#1` already exist one gets an error message talking about the problematical command (not the environment). The remainder of the LaTeX run is probably badly broken and it is unlikely that continuing it gives reasonable results.

```
\def\NewEnvironmentCopy{%
  \declare@environmentcopy
    {\@firstofone}%
    {\@firstoftwo\@notdefinable}}
\def\RenewEnvironmentCopy{%
  \declare@environmentcopy
    {\@latex@error{Environment \reserved@a\space undefined}\@ehc
     \@firstofone}%
    {\@firstofone}}
\def\DeclareEnvironmentCopy{%
  \declare@environmentcopy
    {\@firstofone}%
    {\@firstofone}}
\long\def\declare@environmentcopy#1#2#3#4{%
  \edef\reserved@a{\@ifundefined{#3}{end#3}{#3}}%
  \@ifundefined\reserved@a
      {\def\reserved@a{#3}#1}%
      {\def\reserved@a{#3}#2}%
    {\ExpandArgs{cc}\declare@commandcopy@do{#3}{#4}%
     \ExpandArgs{cc}\declare@commandcopy@do{end#3}{end#4}}}
```

Now the rollback code.

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000-00-00}{\DeclareEnvironmentCopy}
⟨latexrelease⟩  {Undefine \NewEnvironmentCopy, \RenewEnvironmentCopy, and \DeclareEnvironmentCop
⟨latexrelease⟩\let\NewEnvironmentCopy\@undefined
⟨latexrelease⟩\let\RenewEnvironmentCopy\@undefined
⟨latexrelease⟩\let\DeclareEnvironmentCopy\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\NewEnvironmentCopy`*,* `\RenewEnvironmentCopy`*, and* `\DeclareEnvironmentCopy`*.*)

1.5.2 Showing robust commands

\ShowCommand

Most of the machinery defined for \NewCommandCopy can be used to show the definition of a robust command, in a similar fashion to texdef. The difference is that after the command is detected to have a given type of robustness, rather than making a copy, we use a separate routine to show its definition.

With all the machinery in place, \ShowCommand itself is quite simple: we use \robust@command@act to iterate through the \@showcommandlisthook list, and if nothing is found, fallback to \show.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2020-10-01}{\ShowCommand}%
⟨latexrelease⟩   {Add \ShowCommand}%
⟨*2ekernel | latexrelease⟩
\long\def\ShowCommand#1{%
  \robust@command@act
    \@showcommandlisthook#1%
    \show#1}
```

\@showcommandlisthook

The initial definition of \@showcommandlisthook contains the same tests as used for copying, but \@show@... commands instead of \@copy@.... Same as before, it is initialized to cope with \DeclareRobustCommand and \newcommand with optional arguments.

```
\def\@showcommandlisthook{%
  {\@if@DeclareRobustCommand \@show@DeclareRobustCommand}%
  {\@if@newcommand \@show@newcommand}}
```

Now the rollback code.

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000-00-00}{\ShowCommand}
⟨latexrelease⟩   {Undefine \ShowCommand}%
⟨latexrelease⟩\let\ShowCommand\@undefined
⟨latexrelease⟩\let\@showcommandlisthook\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* \ShowCommand *and* \@showcommandlisthook.)

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2020-10-01}{\@if@DeclareRobustCommand}
⟨latexrelease⟩   {Add \@if@DeclareRobustCommand, \@if@newcommand,
⟨latexrelease⟩        \@copy@DeclareRobustCommand, \@copy@newcommand,
⟨latexrelease⟩        \@show@DeclareRobustCommand, \@show@newcommand}%
⟨*2ekernel | latexrelease⟩
```

1.5.3 Commands defined with \DeclareRobustCommand

\@if@DeclareRobustCommand

Now that we provided a generic way to copy one macro to another, we need to define a way to check if a command is one of LaTeX 2ε's robust types. These tests are heavily based on Heiko's \LetLtxMacro, but chopped into separate macros.

The command \@if@DeclareRobustCommand checks if a command \cmd was defined by \DeclareRobustCommand. The test returns true if the expansion of \cmd is exactly \protect\cmd␣.

```
\long\def\@if@DeclareRobustCommand#1{%
```

```
  \begingroup
    \escapechar=‘\\
    \edef\reserved@a{\string#1}%
    \edef\reserved@b{\detokenize{#1}}%
    \xdef\@gtempa{%
      \ifx\reserved@a\reserved@b
         \noexpand\x@protect
         \noexpand#1%
      \fi
      \noexpand\protect
      \expandafter\noexpand\csname\@expl@cs@to@str@@N#1 \endcsname}%
  \endgroup
  \ifx\@gtempa#1\relax
    \expandafter\@firstoftwo
  \else
    \expandafter\@secondoftwo
  \fi}
```

\@copy@DeclareRobustCommand
\copy@kernel@robust@command

If a command was defined by `\DeclareRobustCommand` (that is, `\@if@DeclareRobustCommand` returns true), then to make a copy of `\cmd` into `\foo` we define the latter such that it expands to `\protect\foo␣`, then make `\foo␣` equal to `\cmd␣`.

There is one detail we need to take care of: if a command was defined with `\DeclareRobustCommand` it may still have an optional argument, in which case there is one more macro layer before the actual definition of the command. We use `\@if@newcommand` to check that and `\@copy@newcommand` to do the copying.

```
\long\def\@copy@DeclareRobustCommand#1#2{%
  \begingroup
    \escapechar=‘\\
    \edef\reserved@a{\string#1}%
    \edef\reserved@b{\detokenize{#1}}%
    \edef\reserved@a{%
  \endgroup
  \def\noexpand#1{%
    \ifx\reserved@a\reserved@b
       \noexpand\x@protect
       \noexpand#1%
    \fi
    \noexpand\protect
    \expandafter\noexpand\csname\@expl@cs@to@str@@N#1 \endcsname}%
  \noexpand\copy@kernel@robust@command
    \expandafter\noexpand\csname\@expl@cs@to@str@@N#1 \endcsname
    \expandafter\noexpand\csname\@expl@cs@to@str@@N#2 \endcsname}%
  \reserved@a}
\long\def\copy@kernel@robust@command#1#2{%
  \robust@command@chk@safe#2%
    {\@if@newcommand#2%
       {\@copy@newcommand}%
       {\declare@commandcopy@let}}
    {\declare@commandcopy@let}%
  #1#2}
```

\@show@DeclareRobustCommand
\show@kernel@robust@command
\@show@macro

Showing the command is pretty simple. This command prints the top-level expansion as TEX's `\show` would, but with `robust macro:` rather than just `macro:`, then

a blank line and then \show the inner command. For a macro defined with, say, \DeclareRobustCommand\foo[1]{bar}, it will print:

```
> \foo=robust macro:
->\protect \foo  .

> \foo =\long macro:
#1->bar.
```

If the inner command is defined with an optional argument, then \@show@newcommand is also used.

The value of \escapechar is deliberately not enforced, so \ShowCommand behaves more like \show.

```
\long\def\@show@DeclareRobustCommand#1{%
  \typeout{> \string#1=robust macro:}%
  \typeout{->\@expl@cs@replacement@spec@@N#1.^^J}%
  \expandafter\show@kernel@robust@command
    \csname\@expl@cs@to@str@@N#1 \endcsname}
\long\def\show@kernel@robust@command#1{%
  \robust@command@chk@safe#1%
    {\@if@newcommand#1%
       {\@show@newcommand}%
       {\@show@macro}}%
    {\@show@macro}%
  #1}
\let\@show@macro\show
```

(*End of definition for* \@if@DeclareRobustCommand *and others.*)

1.5.4 Commands defined with \newcommand (with optional argument)

\@if@newcommand

A command \cmd (or \cmd␣, if it was defined with \DeclareRobustCommand) with an optional argument will expand to \@protected@testopt\cmd\\cmd{<opt>}. To check that we look at the first three tokens in the expansion of \cmd, and return true or false accordingly.

This test *requires* that the command be a parameterless macro, otherwise it will not work (and probably break). This is ensured with \robust@command@chk@safe before calling \@if@newcommand.

```
\long\def\@if@newcommand#1{%
  \edef\reserved@a{%
    \noexpand\@protected@testopt
    \noexpand#1%
    \expandafter\noexpand\csname\@backslashchar\@expl@cs@to@str@@N#1\endcsname}%
  \edef\reserved@b{%
    \unexpanded\expandafter\expandafter\expandafter
      {\expandafter\@carcube#1{}{}{}\@nil}}%
  \ifx\reserved@a\reserved@b
    \expandafter\@firstoftwo
  \else
    \expandafter\@secondoftwo
  \fi}
```

Then, if a command `\cmd` takes an optional argument, we copy it to `\foo` by defining
`\@copy@newcommand` the latter to expand to `\@protected@testopt\foo\\foo{<opt>}`.

```
\long\def\@copy@newcommand#1#2{%
  \edef#1{\noexpand\@protected@testopt
    \noexpand#1%
    \expandafter\noexpand\csname\@backslashchar\@expl@cs@to@str@@N#1\endcsname
    \unexpanded\expandafter\expandafter\expandafter
      {\expandafter\@gobblethree#2}}%
  \expandafter
  \let\csname\@backslashchar\@expl@cs@to@str@@N#1\expandafter\endcsname
      \csname\@backslashchar\@expl@cs@to@str@@N#2\endcsname}
```

A command being `\shown` here is guaranteed to have an optional argument. Start by
`\@show@newcommand` showing the top-level expansion of the command (using `\typeout` to avoid TeX asking for
`\@show@newcommand@aux` interaction and extra context lines), then call `\@show@newcommand@aux` with the internal command, which contains the actual definition, and with the expansion of the command to extract the default value of the optional argument.

```
\long\def\@show@newcommand#1{%
  \typeout{> \string#1=robust macro:}%
  \typeout{->\@expl@cs@replacement@spec@@N#1.^^J}%
  \expandafter\@show@newcommand@aux
    \csname\@backslashchar\@expl@cs@to@str@@N#1\expandafter\endcsname
    \expandafter{#1}\@show@tokens}
```

For a macro defined with, say, `\newcommand\foo[1][opt]{bar}`, it will print:

```
> \foo=robust macro:
->\@protected@testopt \foo \\foo {opt}.

> \\foo=\long macro:
> default #1=opt.
[#1]->bar.
```

If the command was defined with `\DeclareRobustCommand`, then another pair of lines show the top-level expansion `\protect␣\foo␣␣`.

```
\long\def\@show@newcommand@aux#1#2#3{%
  \typeout{> \string#1=\@expl@cs@prefix@spec@@N#1macro:}%
  #3{default \string##1=\expandafter\detokenize\@gobblethree#2.^^J%
    \@expl@cs@parameter@spec@@N#1->\@expl@cs@replacement@spec@@N#1}}
```

This macro prints the contents of the token list (macro) `#1` using `\showtokens`. The
`\@show@tokens` `\expandafter` gymnastics ensures that `\showtokens` itself, and the internals of this macro aren't showed in the context lines.

```
\long\def\@show@tokens#1{%
  \edef\reserved@a{#1}%
  \showtokens\expandafter
    \expandafter\expandafter{\expandafter\reserved@a}}
```

Now the rollback code.

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000-00-00}{\@if@DeclareRobustCommand}
⟨latexrelease⟩  {Undefine \@if@DeclareRobustCommand, \@if@newcommand,
```

```
⟨latexrelease⟩              \@copy@DeclareRobustCommand, \@copy@newcommand,
⟨latexrelease⟩              \@show@DeclareRobustCommand, \@show@newcommand}%
⟨latexrelease⟩\let\@if@DeclareRobustCommand\@undefined
⟨latexrelease⟩\let\@copy@DeclareRobustCommand\@undefined
⟨latexrelease⟩\let\@show@DeclareRobustCommand\@undefined
⟨latexrelease⟩\let\@if@newcommand\@undefined
⟨latexrelease⟩\let\@copy@newcommand\@undefined
⟨latexrelease⟩\let\@show@newcommand\@undefined
%
⟨latexrelease⟩\let\copy@kernel@robust@command\@undefined
⟨latexrelease⟩\let\show@kernel@robust@command\@undefined
⟨latexrelease⟩\let\@show@newcommand@aux\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@if@newcommand` *and others.*)

1.5.5 Showing environments

`\ShowEnvironment`

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2023-06-01}{\ShowEnvironment}
⟨latexrelease⟩   {Add \ShowEnvironment}%
⟨*2ekernel | latexrelease⟩
```

`\ShowEnvironment` is quite similar to `\ShowCommand`. We will pass the environment ⟨*env*⟩ around as the macro `\env`, because `\robust@command@act` expects a single token.

```
\def\ShowEnvironment#1{%
  \expandafter\@show@environment\csname #1\endcsname}
\long\def\@show@environment#1{%
  \robust@command@act
    \@showenvironmentlisthook#1%
    \@show@normalenv#1}
```

`\@showenvironmentlisthook`

This is similar to `\@showcommandlisthook`, but uses the dedicated versions for environments.

```
\def\@showenvironmentlisthook{%
  {\@if@DeclareRobustCommand \@show@DeclareRobustCommand@env}%
  {\@if@newcommand \@show@newcommand@env}}
```

`\@show@newcommand@env`
`\@show@DeclareRobustCommand@env`

These are similar to the command versions below, except they say "environment" and call `\@show@environment@end` to print the `\end` part.

```
\long\def\@show@newcommand@env#1{%
  \@show@environment@begin#1%
  \expandafter\@show@newcommand@aux
    \csname\@backslashchar\@expl@cs@to@str@@N#1\expandafter\endcsname
    \expandafter{#1}\@show@typeout
  \@show@environment@end#1}
\long\def\@show@DeclareRobustCommand@env#1{%
  \@show@environment@begin#1%
  \begingroup
    \let\@show@tokens\@show@typeout
    \let\@show@macro\@show@nonstop
    \expandafter\show@kernel@robust@command
```

```
      \csname\@expl@cs@to@str@@N#1 \endcsname
  \endgroup
  \@show@environment@end#1}
\long\def\@show@environment@begin#1{%
  \typeout{> \string\begin{\@expl@cs@to@str@@N#1}=environment:}%
  \typeout{\@expl@cs@parameter@spec@@N#1->%
           \@expl@cs@replacement@spec@@N#1.^^J}}
```

A "normal" environment is straightforward. `\@show@environment@end` needs to check if the `\end` part is defined and show it accordingly, otherwise the output would show gibberish.

`\@show@normalenv`
`\@show@environment@end`

```
\long\def\@show@normalenv#1{%
  \@show@environment@begin#1%
  \@show@environment@end#1}
\long\def\@show@environment@end#1{%
  \expandafter\@show@environment@end@aux
    \csname end\@expl@cs@to@str@@N#1\endcsname#1}
\long\def\@show@environment@end@aux#1#2{%
  \@show@tokens{\string\end{\@expl@cs@to@str@@N#2}%
    \ifx\relax#1=undefined%
    \else:^^J\@expl@cs@parameter@spec@@N#1->%
             \@expl@cs@replacement@spec@@N#1%
    \fi}}
```

And here some auxiliaries:

`\@show@nonstop`
`\@show@typeout`

`\@show@nonstop` same output as `\show`, but doesn't stop for interaction;

`\@show@typeout` same output as `\showtokens`, but doesn't stop for interaction.

```
\def\@show@nonstop#1{%
  \typeout{> \string#1=\@expl@cs@prefix@spec@@N#1macro:^^J%
   \@expl@cs@parameter@spec@@N#1->\@expl@cs@replacement@spec@@N#1.}}
\def\@show@typeout#1{\typeout{> #1.^^J}}
```

Now the rollback code.

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000-00-00}{\ShowEnvironment}
⟨latexrelease⟩  {Undefine \ShowEnvironment}%
⟨latexrelease⟩\let\ShowEnvironment\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\ShowEnvironment` *and others.*)

1.6 Internal defining commands

These commands are used internally to define other LaTeX commands.

`\@ifundefined`

Check if first arg is undefined or `\relax` and execute second or third arg depending,

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2018-04-01}{\@ifundefined}
⟨latexrelease⟩{Leave commands undefined in \@ifundefined}%
⟨*2ekernel | latexrelease⟩
```

Version using `\ifcsname` to avoid defining undefined tokens to `\relax`. Defined here to simplify using unmatched `\fi`.

```
\def\@ifundefined#1{%
  \ifcsname#1\endcsname\@ifundefin@d@i\else\@ifundefin@d@ii\fi{#1}}
\long\def\@ifundefin@d@i#1\fi#2{\fi
  \expandafter\ifx\csname #2\endcsname\relax
    \@ifundefin@d@ii
  \fi
  \@secondoftwo}
\long\def\@ifundefin@d@ii\fi#1#2#3{\fi #2}
```

Now test of engine.

```
\ifx\numexpr\@undefined
```

Classic version (should not be needed as etex is assumed).

```
\def\@ifundefined#1{%
  \expandafter\ifx\csname#1\endcsname\relax
    \expandafter\@firstoftwo
  \else
    \expandafter\@secondoftwo
  \fi}
\else\ifx\directlua\@undefined
```

Use the `\ifcsname` defined above.

```
\else
```

Optimised version for LuaTEX, using `\lastnamedcs`

```
\def\@ifundefined#1{%
  \ifcsname#1\endcsname
    \expandafter\ifx\lastnamedcs\relax\else\@ifundefin@d@i\fi
  \fi
  \@firstoftwo}
\long\def\@ifundefin@d@i#1#2#3#4#5{#1#2#5}
\fi
\fi
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000-00-00}{\@ifundefined}
⟨latexrelease⟩{Leave commands undefined in \@ifundefined}%
⟨latexrelease⟩\def\@ifundefined#1{%
⟨latexrelease⟩   \expandafter\ifx\csname#1\endcsname\relax
⟨latexrelease⟩     \expandafter\@firstoftwo
⟨latexrelease⟩   \else
⟨latexrelease⟩     \expandafter\@secondoftwo
⟨latexrelease⟩   \fi}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@ifundefined`.)

`\@qend` `\@qrelax` The following define `\@qend` and `\@qrelax` to be the strings '`end`' and '`relax`' with the characters `\catcode`d 12.

```
\edef\@qend{\expandafter\@cdr\string\end\@nil}
\edef\@qrelax{\expandafter\@cdr\string\relax\@nil}
```

(*End of definition for* `\@qend` *and* `\@qrelax`.)

`\@ifnextchar` `\@ifnextchar` peeks at the following character and compares it with its first argument. If both are the same it executes its second argument, otherwise its third.

```
\long\def\@ifnextchar#1#2#3{%
  \let\reserved@d=#1%
  \def\reserved@a{#2}%
  \def\reserved@b{#3}%
  \futurelet\@let@token\@ifnch}
```

(*End of definition for* `\@ifnextchar`.)

`\kernel@ifnextchar` This macro is the kernel version of `\@ifnextchar` which is used in a couple of places to prevent the AMS variant from being used since in some places this produced chaos. For example, if an `fd` file is loaded in a random place then the optional argument to `\ProvidesFile` could get printed there instead of being written only in the log file. This happened when there was a space or a newline between the mandatory and optional arguments! It should really be fixed in the `amsmath` package one day, but...

Note that there may be other places in the kernel where this version should be used rather than the original, but variable, version.

```
\let\kernel@ifnextchar\@ifnextchar
```

(*End of definition for* `\kernel@ifnextchar`.)

`\@ifnch` `\@ifnch` is a tricky macro to skip any space tokens that may appear before the character in question. If it encounters a space token, it calls xifnch.

```
\def\@ifnch{%
  \ifx\@let@token\@sptoken
    \let\reserved@c\@xifnch
  \else
    \ifx\@let@token\reserved@d
      \let\reserved@c\reserved@a
    \else
      \let\reserved@c\reserved@b
    \fi
  \fi
  \reserved@c}
```

(*End of definition for* `\@ifnch`.)

`\@sptoken` The following code makes `\@sptoken` a space token. It is important here that the control sequence `\:` consists of a non-letter only, so that the following whitespace is significant. Together with the fact that the equal sign in a `\let` may be followed by only one optional space the desired effect is achieved. NOTE: the following hacking must precede the definition of `\:` as math medium space.

```
\def\:{\let\@sptoken= } \:  % this makes \@sptoken a space token
```

(*End of definition for* `\@sptoken`.)

`\@xifnch` In the following definition of `\@xifnch`, `\:` is again used to get a space token as delimiter into the definition.

```
\def\:{\@xifnch} \expandafter\def\: {\futurelet\@let@token\@ifnch}
```

(*End of definition for* `\@xifnch`.)

\@ifstar The new implementation below avoids passing the ⟨*true code*⟩ Through one more \def than the ⟨*false code*⟩, which previously meant that # had to be written as #### in one argument, but ## in the other. The * is gobbled by \@firstoftwo.

```
\def\@ifstar#1{\@ifnextchar *{\@firstoftwo{#1}}}
```

(*End of definition for* \@ifstar.)

\@dblarg
\@xdblarg

```
\long\def\@dblarg#1{\kernel@ifnextchar[{#1}{\@xdblarg{#1}}}
\long\def\@xdblarg#1#2{#1[{#2}]{#2}}
```

(*End of definition for* \@dblarg *and* \@xdblarg.)

\@sanitize The command \@sanitize changes the catcode of all special characters except for braces to 'other'. It can be used for commands like \index that want to write their arguments verbatim. Needless to say, this command should only be executed within a group, or chaos will ensue.

```
\def\@sanitize{\@makeother\ \@makeother\\\@makeother\$\@makeother\&%
\@makeother\#\@makeother\^\@makeother\_\@makeother\%\@makeother\~}
```

(*End of definition for* \@sanitize.)

\@onelevel@sanitize This makes the whole "meaning" of #1 (its one-level expansion) into catcode 12 tokens: it could be used in \DeclareRobustCommand.

If it is to be used on default float specifiers, this should be done when they are defined.

```
\def \@onelevel@sanitize #1{%
  \edef #1{\expandafter\strip@prefix
            \meaning #1}%
}
```

(*End of definition for* \@onelevel@sanitize.)

\string@makeletter
\@string@makeletter
\char@if@alph Iterates through a string, turning each alphabetic character into a catcode-11 token (partly undoes a \detokenize). Useful for \ifx-based string comparisons where \detokenize-ing the other string would break too much code.

The macro uses expl3's \@expl@str@map@function@@NN to iterate on the string (without losing spaces) and applies \@string@makeletter on each character. The latter checks if character is between a–z or A–Z, and uses \@alph or \@Alph to get the corresponding catcode-11 token. Other tokens are passed through unchanged.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\string@makeletter}
⟨latexrelease⟩  {Add \string@makeletter}%
⟨*2ekernel | latexrelease⟩
\def\string@makeletter#1{%
  \@expl@str@map@function@@NN#1\@string@makeletter}
\def\@string@makeletter#1{%
  \char@if@alph{#1}%
    {\@expl@char@generate@@nn{`#1}{11}}%
    {#1}}
\def\char@if@alph#1{%
  \ifnum0\ifnum`#1<`A 1\fi\ifnum`#1>`z 1\fi
      \if\ifnum`#1>`Z @\fi\ifnum`#1<`a @\fi01\fi>0
```

```
    \expandafter\@secondoftwo
  \else
    \expandafter\@firstoftwo
  \fi}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
%
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\string@makeletter}
⟨latexrelease⟩  {Undefine \string@makeletter}%
⟨latexrelease⟩\let\string@makeletter\@undefined
⟨latexrelease⟩\let\@string@makeletter\@undefined
⟨latexrelease⟩\let\char@if@alph\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\string@makeletter`, `\@string@makeletter`, *and* `\char@if@alph`.)

`\makeatletter` `\makeatother` Make internal control sequences accessible or inaccessible.

```
\DeclareRobustCommand\makeatletter{\catcode`\@11\relax}
\DeclareRobustCommand\makeatother{\catcode`\@12\relax}
```

(*End of definition for* `\makeatletter` *and* `\makeatother`.)

2 Discretionary Hyphenation

`\-` `\@dischyph`

Moved here to be after the definition of `\DeclareRobustCommand`.

The primitive `\-` command adds a discretionary hyphen using the current font's `\hyphenchar`. Monospace fonts are usually declared with `\hyphenchar` set to −1 to suppress hyphenation.

LaTeX, from LaTeX2.09 in 1986 defined `\-` by

```
\def\-{\discretionary{-}{}{}}
```

The following comment was added when these commands were first set up, 19 April 1986:

> the `\-` command is redefined to allow it to work in the `\ttfamily` type style, where automatic hyphenation is suppressed by setting `\hyphenchar` to −1. The original primitive TeX definition is saved as `\@@hyph` just in case anyone needs it.

LaTeX 2ε, between 1993 and 2017, had a comment at this point saying that the definition "would probably change" because the definition always uses `-`. The definition used below was given in comments at this point during time.

In 2017 we finally enabled this definition by default, with the older LaTeX definition accessible via latexrelease as usual.

In LuaLaTeX the primitive definition of `\-` is used directly because it's use of extended hyphenation parameters means that `\-` works correctly even with `\hyphenchar` set to −1. This change makes `\-` under LuaLaTeX compatible with language specific hyphenation characters.

Temporary definition of `\@latex@info`, final definition is later.

```
\def\@latex@info#1{}
```

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\-}{Use primitive \- in Lua\LaTeX}%
⟨*2ekernel | latexrelease⟩
\ifx\directlua\@undefined
  \DeclareRobustCommand{\-}{%
    \discretionary{%
      \char \ifnum\hyphenchar\font<\z@
              \defaulthyphenchar
            \else
              \hyphenchar\font
            \fi
                 }{}{}%
  }
\else
  \let\-\@@hyph
\fi
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2017/04/15}{\-}{Use \hyphenchar in \-}%
⟨latexrelease⟩\DeclareRobustCommand{\-}{%
⟨latexrelease⟩   \discretionary{%
⟨latexrelease⟩     \char \ifnum\hyphenchar\font<\z@
⟨latexrelease⟩             \defaulthyphenchar
⟨latexrelease⟩           \else
⟨latexrelease⟩             \hyphenchar\font
⟨latexrelease⟩           \fi
⟨latexrelease⟩                }{}{}%
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\-}{Use \hyphenchar in \-}%
⟨latexrelease⟩\def\-{\discretionary{-}{}{}}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel | latexrelease⟩
\let\@dischyph=\-
⟨/2ekernel | latexrelease⟩
⟨*2ekernel⟩
```

(*End of definition for* `\-` *and* `\@dischyph`.)

Delayed from `ltvers.dtx`

```
\newif\if@includeinrelease
\@includeinreleasefalse
```

Delayed from `ltplain.dtx`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                  {\allowbreak}{Make various commands robust}%
\MakeRobust\allowbreak
\MakeRobust\bigbreak
\MakeRobust\break
\MakeRobust\dotfill
\MakeRobust\frenchspacing
\MakeRobust\goodbreak
\MakeRobust\hrulefill
```

```
\MakeRobust\medbreak
\MakeRobust\nobreak
\MakeRobust\nonfrenchspacing
\MakeRobust\obeylines
\MakeRobust\obeyspaces
\MakeRobust\slash
\MakeRobust\smallbreak
\MakeRobust\strut
\MakeRobust\underbar
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\allowbreak}{Make various commands robust}%
⟨latexrelease⟩
⟨latexrelease⟩\kernel@make@fragile\allowbreak
⟨latexrelease⟩\kernel@make@fragile\bigbreak
⟨latexrelease⟩\kernel@make@fragile\break
⟨latexrelease⟩\kernel@make@fragile\dotfill
⟨latexrelease⟩\kernel@make@fragile\frenchspacing
⟨latexrelease⟩\kernel@make@fragile\goodbreak
⟨latexrelease⟩\kernel@make@fragile\hrulefill
⟨latexrelease⟩\kernel@make@fragile\medbreak
⟨latexrelease⟩\kernel@make@fragile\nobreak
⟨latexrelease⟩\kernel@make@fragile\nonfrenchspacing
⟨latexrelease⟩\kernel@make@fragile\obeylines
⟨latexrelease⟩\kernel@make@fragile\obeyspaces
⟨latexrelease⟩\kernel@make@fragile\slash
⟨latexrelease⟩\kernel@make@fragile\smallbreak
⟨latexrelease⟩\kernel@make@fragile\strut
⟨latexrelease⟩\kernel@make@fragile\underbar
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

`\g@addto@macro` Globally add to the end of a macro. This macro is used by the kernel to add to its internal hooks.

```
\long\def\g@addto@macro#1#2{%
  \begingroup
    \toks@\expandafter{#1#2}%
    \xdef#1{\the\toks@}%
  \endgroup}
```

(*End of definition for* `\g@addto@macro`.)

```
⟨/2ekernel⟩
```

File 07
ltcmd.dtx

1 Creating document commands

1.1 Overview

Creating document commands and environments using the LaTeX3 toolset is based around the idea that a common set of descriptions can be used to cover almost all argument types used in real documents. Thus parsing is reduced to a simple description of which arguments a command takes: this description provides the "glue" between the document syntax and the implementation of the command.

First, we will describe the argument types, then move on to explain how these can be used to create both document commands and environments. Various more specialized features are then described, which allow an even richer application of a simple interface set up.

The details here are intended to help users create document commands in general. More technical detail, suitable for TeX programmers, is included in `interface3`.

1.2 Describing argument types

In order to allow each argument to be defined independently, the parser does not simply need to know the number of arguments for a function, but also the nature of each one. This is done by constructing an *argument specification*, which defines the number of arguments, the type of each argument and any additional information needed for the parser to read the user input and properly pass it through to internal functions.

The basic form of the argument specifier is a list of letters, where each letter defines a type of argument. As will be described below, some of the types need additional information, such as default values. The argument types can be divided into two, those which define arguments that are mandatory (potentially raising an error if not found) and those which define optional arguments. The mandatory types

- `m` A standard mandatory argument, which can either be a single token alone or multiple tokens surrounded by curly braces `{}`. Regardless of the input, the argument will be passed to the internal code without the outer braces. This is the type specifier for a normal TeX argument.

- `r` Given as `r`⟨*token1*⟩⟨*token2*⟩, this denotes a "required" delimited argument, where the delimiters are ⟨*token1*⟩ and ⟨*token2*⟩. If the opening delimiter ⟨*token1*⟩ is missing, the default marker `\NoValue` will be inserted after a suitable error.

- `R` Given as `R`⟨*token1*⟩⟨*token2*⟩`{`⟨*default*⟩`}`, this is a "required" delimited argument as for `r`, but it has a user-definable recovery ⟨*default*⟩ instead of `\NoValue`.

- `v` Reads an argument "verbatim", between the following character and its next occurrence, in a way similar to the argument of the LaTeX 2_ε command `\verb`. Thus a `v`-type argument is read between two identical characters, which cannot be any of `%`, `\`, `#`, `{`, `}` or ␣. The verbatim argument can also be enclosed between braces, `{` and `}`. A command with a verbatim argument will produce an error when it appears within an argument of another command.

`b` Only suitable in the argument specification of an environment, it denotes the body of the environment, between `\begin{`⟨*environment*⟩`}` and `\end{`⟨*environment*⟩`}`. See Section 1.11 for details.

`c` Only suitable in the argument specification of an environment, it denotes collection of the environment verbatim, between `\begin{`⟨*environment*⟩`}` and `\end{`⟨*environment*⟩`}`. See Section 1.16 for details.

The types which define optional arguments are:

`o` A standard LaTeX optional argument, surrounded with square brackets, which will supply the special `\NoValue` marker if not given (as described later).

`d` Given as `d`⟨*token1*⟩⟨*token2*⟩, an optional argument which is delimited by ⟨*token1*⟩ and ⟨*token2*⟩. As with `o`, if no value is given the special marker `\NoValue` is returned.

`O` Given as `O{`⟨*default*⟩`}`, is like `o`, but returns ⟨*default*⟩ if no value is given.

`D` Given as `D`⟨*token1*⟩⟨*token2*⟩`{`⟨*default*⟩`}`, it is as for `d`, but returns ⟨*default*⟩ if no value is given. Internally, the `o`, `d` and `O` types are short-cuts to an appropriated-constructed `D` type argument.

`s` An optional star, which will result in a value `\BooleanTrue` if a star is present and `\BooleanFalse` otherwise (as described later).

`t` An optional ⟨*token*⟩, which will result in a value `\BooleanTrue` if ⟨*token*⟩ is present and `\BooleanFalse` otherwise. Given as `t`⟨*token*⟩.

`e` Given as `e{`⟨*tokens*⟩`}`, a set of optional *embellishments*, each of which requires a *value*. If an embellishment is not present, `\NoValue` is returned. Each embellishment gives one argument, ordered as for the list of ⟨*tokens*⟩ in the argument specification. All ⟨*tokens*⟩ must be distinct.

`E` As for `e` but returns one or more ⟨*defaults*⟩ if values are not given: `E{`⟨*tokens*⟩`}{`⟨*defaults*⟩`}`. See Section 1.7 for more details.

1.3 Modifying argument descriptions

In addition to the argument *types* discussed above, the argument description also gives special meaning to three other characters.

First, `+` is used to make an argument long (to accept paragraph tokens). In contrast to `\newcommand`, this applies on an argument-by-argument basis. So modifying the example to "`s o o +m Odefault`" means that the mandatory argument is now `\long`, whereas the optional arguments are not.

Secondly, `!` is used to control whether spaces are allowed before optional arguments. There are some subtleties to this, as TeX itself has some restrictions on where spaces can be "detected": more detail is given in Section 1.6.

Thirdly, `=` is used to declare that the following argument should be interpreted as a series of keyvals. See Section 1.9 for more details.

Finally, the character `>` is used to declare so-called 'argument processors', which can be used to modify the contents of an argument before it is passed to the macro definition. The use of argument processors is a somewhat advanced topic, (or at least a less commonly used feature) and is covered in Section 1.10.

1.4 Creating document commands and environments

\NewDocumentCommand
\RenewDocumentCommand
\ProvideDocumentCommand
\DeclareDocumentCommand

`\NewDocumentCommand {⟨cmd⟩} {⟨arg spec⟩} {⟨code⟩}`

This family of commands are used to create a ⟨*cmd*⟩. The argument specification for the function is given by ⟨*arg spec*⟩, and the command uses the ⟨*code*⟩ with `#1`, `#2`, etc. replaced by the arguments found by the parser.

An example:

```
\NewDocumentCommand\chapter{s o m}
  {%
    \IfBooleanTF{#1}%
      {\typesetstarchapter{#3}}%
      {\typesetnormalchapter{#2}{#3}}%
  }
```

would be a way to define a `\chapter` command which would essentially behave like the current LaTeX 2_ε command (except that it would accept an optional argument even when a `*` was parsed). The `\typesetnormalchapter` could test its first argument for being `\NoValue` to see if an optional argument was present. (See Section 1.8 for details of `\IfBooleanTF` and testing for `\NoValue`.)

The difference between the `\New...` `\Renew...`, `\Provide...` and `\Declare...` versions is the behavior if ⟨*cmd*⟩ is already defined.

- `\NewDocumentCommand` will issue an error if ⟨*cmd*⟩ has already been defined.
- `\RenewDocumentCommand` will issue an error if ⟨*cmd*⟩ has not previously been defined.
- `\ProvideDocumentCommand` creates a new definition for ⟨*cmd*⟩ only if one has not already been given.
- `\DeclareDocumentCommand` will always create the new definition, irrespective of any existing ⟨*cmd*⟩ with the same name. This should be used sparingly.

If the ⟨*cmd*⟩ can't be provided as a single token but needs "constructing", you can use `\ExpandArgs` as explained in Section **??** which also gives an example in which this is needed.

\NewDocumentEnvironment
\RenewDocumentEnvironment
\ProvideDocumentEnvironment
\DeclareDocumentEnvironment

`\NewDocumentEnvironment {⟨env⟩} {⟨arg spec⟩} {⟨beg-code⟩} {⟨end-code⟩}`

These commands work in the same way as `\NewDocumentCommand`, etc., but create environments (`\begin{`⟨*env*⟩`}` ... `\end{`⟨*env*⟩`}`). Both the ⟨*beg-code*⟩ and ⟨*end-code*⟩ may access the arguments as defined by ⟨*arg spec*⟩. The arguments will be given following `\begin{`⟨*env*⟩`}`. Any spaces at the start and end of the `{`⟨*env*⟩`}` are removed before the definition takes place, thus

```
\NewDocumentEnvironment{foo}
```

and

```
\NewDocumentEnvironment{ foo }
```

both create the same "`foo`" environment.

1.5 Optional arguments

In contrast to commands created using LaTeX 2$_\varepsilon$'s `\newcommand`, optional arguments created using `\NewDocumentCommand` may safely be nested. Thus for example, following

```
\NewDocumentCommand\foo{om}{I grabbed `#1' and `#2'}
\NewDocumentCommand\baz{o}{#1-#1}
```

using the command as

```
\foo[\baz[stuff]]{more stuff}
```

will print

> I grabbed 'stuff-stuff' and 'more stuff'

This is particularly useful when placing a command with an optional argument *inside* the optional argument of a second command.

When an optional argument is followed by a mandatory argument with the same delimiter, the parser issues a warning because the optional argument could not be omitted by the user, thus becoming in effect mandatory. This can apply to `o`, `d`, `O`, `D`, `s`, `t`, `e`, and `E` type arguments followed by `r` or `R`-type required arguments.

The default for `O`, `D` and `E` arguments can be the result of grabbing another argument. Thus for example

```
\NewDocumentCommand\foo{O{#2} m}
```

would use the mandatory argument as the default for the leading optional one.

1.6 Spacing and optional arguments

TeX will find the first argument after a function name irrespective of any intervening spaces. This is true for both mandatory and optional arguments. So `\foo[arg]` and `\foo␣[arg]` are equivalent. Spaces are also ignored when collecting arguments up to the last mandatory argument to be collected (as it must exist). So after

```
\NewDocumentCommand\foo{m o m}{ ... }
```

the user input `\foo{arg1}[arg2]{arg3}` and `\foo{arg1}␣[arg2]␣{arg3}` will both be parsed in the same way.

The behavior of optional arguments *after* any mandatory arguments is selectable. The standard settings will allow spaces here, and thus with

```
\NewDocumentCommand\foobar{m o}{ ... }
```

both `\foobar{arg1}[arg2]` and `\foobar{arg1}␣[arg2]` will find an optional argument. This can be changed by giving the modified `!` in the argument specification:

```
\NewDocumentCommand\foobar{m !o}{ ... }
```

where `\foobar{arg1}␣[arg2]` will not find an optional argument.

There is one subtlety here due to the difference in handling by TEX of 'control symbols', where the command name is made up of a single character, such as '\\'. Spaces are not ignored by TEX here, and thus it is possible to require an optional argument directly follow such a command. The most common example is the use of `\\` in amsmath environments, which in the terms here would be defined as

```
\NewDocumentCommand\\{!s !o}{ ... }
```

Also notable when using optional arguments in the last position is that TEX will necessarily look ahead for the argument opening token. This means that the value of `\inputlineno` will be 'out by one' if such a trailing optional argument is *not* present and the command ends a line; it will be one greater than the line number containing the last mandatory argument.

1.7 'Embellishments'

The `E`-type argument allows one default value per test token. This is achieved by giving a list of defaults for each entry in the list, for example:

```
E{^_}{{UP}{DOWN}}
```

If the list of default values is *shorter* than the list of test tokens, the special `\NoValue` marker will be returned (as for the `e`-type argument). Thus for example

```
E{^_}{{UP}}
```

has default `UP` for the `^` test character, but will return the `\NoValue` marker as a default for `_`. This allows mixing of explicit defaults with testing for missing values.

1.8 Testing special values

Optional arguments make use of dedicated variables to return information about the nature of the argument received.

`\IfNoValueTF`
`\IfNoValueT`
`\IfNoValueF`

`\IfNoValueTF {⟨arg⟩} {⟨true code⟩} {⟨false code⟩}`

The `\IfNoValue(TF)` tests are used to check if ⟨*argument*⟩ (`#1`, `#2`, *etc.*) is the special `\NoValue` marker. For example

```
\NewDocumentCommand\foo{o m}
  {%
    \IfNoValueTF {#1}%
      {\DoSomethingJustWithMandatoryArgument{#2}}%
      {\DoSomethingWithBothArguments{#1}{#2}}%
  }
```

will use a different internal function if the optional argument is given than if it is not present.

Note that three tests are available, depending on which outcome branches are required: `\IfNoValueTF`, `\IfNoValueT` and `\IfNoValueF`.

As the `\IfNoValue(TF)` tests are expandable, it is possible to test these values later, for example at the point of typesetting or in an expansion context.

When two optional arguments follow each other (a syntax we typically discourage), it can make sense to allow users of the command to specify only the second argument by providing an empty first argument. If you wish to test if an argument is blank or not, but are not concerned with distinguishing an entirely absent argument from an empty one, use the `O` type specifier with the conditional `\IfBlankTF` (described below).

`\IfValueTF` `\IfValueTF {⟨arg⟩} {⟨true code⟩} {⟨false code⟩}`
`\IfValueT`
`\IfValueF`

The reverse form of the `\IfNoValue(TF)` tests are also available as `\IfValue(TF)`. The context will determine which logical form makes the most sense for a given code scenario.

`\IfBlankTF` `\IfBlankTF {⟨arg⟩} {⟨true code⟩} {⟨false code⟩}`
`\IfBlankT`
`\IfBlankF`

The `\IfNoValueTF` command chooses the ⟨*true code*⟩ if the optional argument has not been used at all (and it returns the special `\NoValue` marker), but not if it has been given an empty value. In contrast `\IfBlankTF` returns true if its argument is either truly empty or only contains one or more normal blanks. For example

```
\NewDocumentCommand\foo{m!o}{\par #1:
  \IfNoValueTF{#2}
    {No optional}%
    {%
      \IfBlankTF{#2}
        {Blanks in or empty}%
        {Real content in}%
    }%
  \space argument!}
\foo{1}[bar] \foo{2}[  ] \foo{3}[] \foo{4}[\space] \foo{5} [x]
```

results in the following output:

1: Real content in argument!

2: Blanks in or empty argument!

3: Blanks in or empty argument!

4: Real content in argument!

5: No optional argument! [x]

Note that the `\space` in (4) is considered real content—because it is a command and not a "space" character—even though it results in producing a space. You can also

observe in (5) the effect of the `!` specifier, preventing the last `\foo` from interpreting `[x]` as its optional argument.

`\BooleanTrue`
`\BooleanFalse`

The `true` and `false` flags set when searching for an optional character (using `s` or `t`⟨*char*⟩) have names which are accessible outside of code blocks.

`\IfBooleanTF` `\IfBooleanTF {`⟨*arg*⟩`} {`⟨*true code*⟩`} {`⟨*false code*⟩`}`
`\IfBooleanT`
`\IfBooleanF`

Used to test if ⟨*argument*⟩ (`#1`, `#2`, *etc.*) is `\BooleanTrue` or `\BooleanFalse`. For example

```
\NewDocumentCommand\foo{sm}
  {%
    \IfBooleanTF {#1}%
      {\DoSomethingWithStar{#2}}%
      {\DoSomethingWithoutStar{#2}}%
  }
```

checks for a star as the first argument, then chooses the action to take based on this information.

1.9 Auto-converting to key–value format

Some document commands have a long history of accepting a 'free text' optional argument, for example `\caption` and the sectioning commands `\section`, etc. Introducing more sophisticated (keyval) options to these commands therefore needs a method to interpret the optional argument *either* as free text *or* as a series of keyvals. This needs to take place during argument grabbing as there is a need for careful treatment of braces to obtain the correct result.

The `=` modifier is available to allow ltcmd to correctly implement this process. The modifier guarantees that the argument will be passed to further code as a series of keyvals. To do that, the `=` should be followed by an argument containing the default key name. This is used as the key in a key–value pair *if* the "raw" argument does *not* have the correct form to be interpreted as a set of keyvals.

Taking `\caption` as an example, with the demonstration implementation

```
\DeclareDocumentCommand\caption{s ={short-text} +O{#3} +m}
  {%
    \showtokens{Grabbed arguments:^^J(#2)^^Jand^^J(#3)}%
  }
```

the default key name is `short-text`. When the command `\caption` is then used, if the optional argument is free text such as

```
\caption[Some short text]{A much longer and more detailed text for
  demonstration purposes}
```

then the output will be

```
Grabbed arguments:
(short-text={Some short text})
and
(A much longer and more detailed text for demonstration purposes)
```

On the other hand, if the caption is given with a keyval-form argument

```
\caption[label = cap:demo]%
  {A much longer and more detailed text for demonstration purposes}
```

then this will be respected

```
Grabbed arguments:
(label = cap:demo)
and
(A much longer and more detailed text for demonstration purposes)
```

Interpretation as keyval form is determined by the presence of `=` characters within the argument. Those in inline math mode (enclosed within `$...$` or `\(...\)`) are ignored. An argument can be forced to be read as keyvals by including an empty entry at the start

```
\caption[=,This is now a keyval]%
% ...
\caption[This is not $=$ keyval]%
```

This empty entry is *not* passed to the underlying code, so will not lead to issues with keyval parsers that do not allow an empty key name. Any text-mode `=` signs will need to be braced to avoid being misinterpreted: this is likely most conveniently handled by bracing the entire argument

```
\caption[{Not = to a keyval!}]%
```

which will be passed correctly as

```
Grabbed arguments:
(short-text = {Not = to a keyval!})
```

If the argument is completely blank, the conversion results in an empty keyval list, not `short-text␣=␣`. This reflects the fact that with a move toward keyval processing, an empty argument is best modelled as an empty keyval list. If the user does want an empty classical argument, using `[{}]` will work with both the new processor code and older formats.

1.10 Argument processors

Argument processor are applied to an argument *after* it has been grabbed by the underlying system but before it is passed to ⟨*code*⟩. An argument processor can therefore be used to regularize input at an early stage, allowing the internal functions to be completely independent of input form. Processors are applied to user input and to default values for optional arguments, but *not* to the special `\NoValue` marker.

Each argument processor is specified by the syntax `>{`⟨*processor*⟩`}` in the argument specification. Processors are applied from right to left, so that

```
>{\ProcessorB} >{\ProcessorA} m
```

would apply `\ProcessorA` followed by `\ProcessorB` to the tokens grabbed by the `m` argument.

`\SplitArgument` `\SplitArgument {⟨number⟩} {⟨token(s)⟩}`

This processor splits the argument given at each occurrence of the ⟨*tokens*⟩ up to a maximum of ⟨*number*⟩ tokens (thus dividing the input into ⟨*number*⟩+1 parts). An error is given if too many ⟨*tokens*⟩ are present in the input. The processed input is placed inside ⟨*number*⟩ + 1 sets of braces for further use. If there are fewer than `{`⟨*number*⟩`}` of `{`⟨*tokens*⟩`}` in the argument then `\NoValue` markers are added at the end of the processed argument.

```
\NewDocumentCommand\foo{>{\SplitArgument{2}{;}} m}
  {\InternalFunctionOfThreeArguments#1}
```

If only a single character ⟨*token*⟩ is used for the split, any category code 13 (active) character matching the ⟨*token*⟩ will be replaced before the split takes place. Spaces are trimmed at each end of each item parsed.

The `E` argument type is somewhat special, because with a single `E` in the command declaration you may end up with several arguments in a command (one formal argument per embellishment token). Therefore, when an argument processor is applied to an `e`/`E`-type argument, all the arguments pass through that processor before being fed to the ⟨*code*⟩. For example, this command

```
\NewDocumentCommand\foo{ >{\TrimSpaces} e{_^} }
  { [#1](#2) }
```

applies `\TrimSpaces` to both arguments.

`\SplitList` `\SplitList {⟨token(s)⟩}`

This processor splits the argument given at each occurrence of the ⟨*token(s)*⟩ where the number of items is not fixed. Each item is then wrapped in braces within `#1`. The result is that the processed argument can be further processed using a mapping function (see below).

```
\NewDocumentCommand\foo{>{\SplitList{;}} m}
  {\MappingFunction#1}
```

If only a single character ⟨*token*⟩ is used for the split, it will take account of the possibility that the ⟨*token*⟩ has been made active (category code 13) and will split at such tokens. Spaces are trimmed at each end of each item parsed. Exactly one set of braces will be stripped if an entire item is surrounded by them, i.e. the following inputs and outputs result (each separate item as a brace group).

```
a      ==> {a}
{a}    ==> {a}
{a}b   ==> {{a}b}
a,b    ==> {a}{b}
{a},b  ==> {a}{b}
a,{b}  ==> {a}{b}
a,{b}c ==> {a}{{b}c}
```

\ProcessList `\ProcessList {⟨list⟩} {⟨token(s)⟩}`

To support `\SplitList`, the function `\ProcessList` is available to apply ⟨*tokens*⟩ to every entry in a ⟨*list*⟩. The ⟨*tokens*⟩ can be arbitrary contents that should expect one argument after it: the list entry. For example

```
\NewDocumentCommand\foo{>{\SplitList{;}} m}
  {\ProcessList{#1}{\SomeDocumentCommand}}
```

or

```
\NewDocumentCommand\foo{>{\SplitList{;}} m}
  {\ProcessList{#1}{Abc \SomeDocumentCommand}}
```

\ReverseBoolean `\ReverseBoolean`

This processor reverses the logic of `\BooleanTrue` and `\BooleanFalse`, so that the example from earlier would become

```
\NewDocumentCommand\foo{>{\ReverseBoolean} s m}
  {%
    \IfBooleanTF#1%
      {\DoSomethingWithoutStar{#2}}%
      {\DoSomethingWithStar{#2}}%
  }
```

\TrimSpaces `\TrimSpaces`

Removes any leading and trailing spaces (tokens with character code 32 and category code 10) for the ends of the argument. Thus for example declaring a function

```
\NewDocumentCommand\foo{>{\TrimSpaces} m}
  {\showtokens{#1}}
```

and using it in a document as

```
\foo{␣hello␣world␣}
```

will show '`hello␣world`' at the terminal, with the space at each end removed. `\TrimSpaces` will remove multiple spaces from the ends of the input in cases where these have been included such that the standard TEX conversion of multiple spaces to a single space does not apply.

1.11 Body of an environment

While environments `\begin{`⟨*environment*⟩`}` ... `\end{`⟨*environment*⟩`}` are typically used in cases where the code implementing the ⟨*environment*⟩ does not need to access the contents of the environment (its "body"), it is sometimes useful to have the body as a standard argument.

This is achieved by ending the argument specification with `b`, which is a dedicated argument type for this situation. For instance

```
\NewDocumentEnvironment{twice}{O{\ttfamily} +b}
  {#2#1#2} {}
\begin{twice}[\itshape]
  Hello world!
\end{twice}
```

typesets 'Hello world!*Hello world!*'.

The prefix `+` is used to allow multiple paragraphs in the environment's body. Argument processors can also be applied to `b` arguments. By default, spaces are trimmed at both ends of the body: in the example there would otherwise be spaces coming from the ends the lines after `[\itshape]` and `world!`. Putting the prefix `!` before `b` suppresses space-trimming.

When `b` is used in the argument specification, the last argument of the environment declaration (e.g., `\NewDocumentEnvironment`), which consists of an ⟨*end code*⟩ to insert at `\end{`⟨*environment*⟩`}`, is redundant since one can simply put that code at the end of the ⟨*start code*⟩. Nevertheless this (empty) ⟨*end code*⟩ must be provided.

Environments that use this feature can be nested.

1.12 Fully-expandable document commands

Document commands created using `\NewDocumentCommand`, etc., are normally created so that they do not expand unexpectedly. This is done using engine features, so is more powerful than LaTeX 2ε's `\protect` mechanism. There are *very rare* occasion when it may be useful to create functions using a expansion-only grabber. This imposes a number of restrictions on the nature of the arguments accepted by a function, and the code it implements. This facility should only be used when *necessary*.

`\NewExpandableDocumentCommand` `\RenewExpandableDocumentCommand` `\ProvideExpandableDocumentCommand` `\DeclareExpandableDocumentCommand`

`\NewExpandableDocumentCommand {`⟨*cmd*⟩`} {`⟨*arg spec*⟩`} {`⟨*code*⟩`}`

This family of commands is used to create a document-level ⟨*cmd*⟩, which will grab its arguments in a fully-expandable manner. The argument specification for the function is given by ⟨*arg spec*⟩, and the ⟨*cmd*⟩ will execute ⟨*code*⟩. In general, ⟨*code*⟩ will also be fully expandable, although it is possible that this will not be the case (for example, a function for use in a table might expand so that `\omit` is the first non-expandable non-space token).

Parsing arguments by pure expansion imposes a number of restrictions on both the type of arguments that can be read and the error checking available:

- The last argument (if any are present) must be one of the mandatory types `m`, `r` or `R`.
- The "verbatim" argument type `v` is not available.
- Argument processors (using `>`) are not available.
- It is not possible to differentiate between, for example `\foo[` and `\foo{[}`: in both cases the `[` will be interpreted as the start of an optional argument. As a result, checking for optional arguments is less robust than in the standard version.

1.13 Commands at the start of tabular cells

Creating commands that are used at the start of tabular cells imposes some restrictions on the underlying implementation. The standard LaTeX tabular environments (`tabular`, etc.) use a mechanism which requires that any command wrapping `\multicolumn` or similar must be "expandable". This is *not* the case for commands created using `\NewDocumentCommand`, etc., which as detailed in Section 1.12 use an engine feature which prevents such "expansion". Therefore, to create such wrappers for use at the start of tabular cells, you must use `\NewExpandableDocumentCommand`, for example

```
\NewExpandableDocumentCommand\MyMultiCol{m}{\multicolumn{3}{c}{#1}}
\begin{tabular}{lcr}
a & b & c \\
\MyMultiCol{stuff} \\
\end{tabular}
```

1.14 Using the verbatim argument types

As described above, the `v`-type argument may be viewed as similar to `\verb`. Before looking at exactly what that means, it is important to highlight some key differences. Most notably, *grabbing* a verbatim-like argument is separate from *typesetting* it: the latter is covered in the next section.

When grabbing a `v`-type argument, LaTeX first uses the kernel command `\dospecials` to turn off the "special" nature of characters. It then makes both spaces and tabs "active", so that they can be given a custom definition. Any other characters are grabbed as-is: this means that if any characters have been made "special" and are not listed in `\dospecials`, an error will arise (see below).

The characters that are grabbed as the argument are all those between two identical: in contrast to `\verb`, the characters `\`, `{`, `}` and `%` *cannot* be used as the delimiter character. If any of the grabbed tokens have "special" meaning, an error will be issued.

For the `+v`-type argument, which allows line breaks within the argument, newline characters are converted into `\obeyedline` commands. The standard definition of `\obeyedline` is simple `\par`, thus allowing the grabbed tokens to be used directly in typesetting. A local redefinition of `\obeyedline` can be used to achieve other outputs. For example, to retain blank lines whilst typesetting, one could use

```
\renewcommand*\obeyedline{\mbox{}\par}
```

More information about using these arguments in typesetting is in the following subsection.

Some additional details that may be useful for those with more TeX knowledge: do not worry if this does not make sense to you! Spaces and tabs are stored as active characters. In 8-bit engines, non-ASCII characters are "active", whilst other than the letters a–zA–Z, ASCII characters are "other". In Unicode engines, non-ASCII codepoints will be either letters or "other", based on the standard LaTeX settings derived from Unicode data. For token-based comparisons, it is likely that the active spaces and tabs should be replaced: this can be done conveniently by expansion.

1.15 Typesetting verbatim-like material

In contrast to `\verb`, the `(+)v`-type argument is only about *grabbing* the argument, not *typesetting* it. As such, features that users often associate with "verbatim" are not automatically activated, e.g., selecting a monospaced font. Material grabbed by the `v`-type argument does not automatically suppress ligatures: with modern TEX engines, this largely can be done without the token manipulation which `\verb` uses. (In `\verb`, ligatures are suppressed by making characters active and inserting a zero-width kern before the character itself.)

The `\verb` command also selects a monospaced font: this is not intrinsic to verbatim material, so will need to be set up using for example `\ttfamily`. Similarly, the `verbatim` environment sets up the meaning of `\par` suitable for breaking lines.

1.16 Verbatim environments

In some cases, when grabbing the body of an environment you will want the contents to be treated verbatim. This is available using the argument specification `c`. Like the `b` specification, this has to be the last one. Thus for example

```
\NewDocumentEnvironment{MyVerbatim}{!O{\ttfamily} c}
  {\begin{center} #1 #2\end{center}} {}
\begin{MyVerbatim}[\ttfamily\itshape]
  % Some code is shown here
  $y = mx + c$
\end{MyVerbatim}
```

will typeset verbatim the content, thus:

```
%␣␣␣␣%␣Some␣code␣is␣shown␣here
%␣␣␣␣$y␣=␣mx␣+␣c$
%␣
```

Since grabbing the entire contents verbatim will result in there being no `\par` tokens, newlines are always permitted: there is no need for a `+` modifier here. As for the `v` specification, newlines are stored as `\obeyedline`. In a similar fashion to the `b` specification, by default *newlines* are trimmed at both ends of the body. Putting the prefix `!` before `c` suppresses this trimming.

Collection of the body takes place on a line-by-line basis: content is collected up to the end-of-line in the source, then examined before storage. This means that the line ending the environment (containing in the example above `\end{MyVerbatim}`) cannot have any text *after* the end of the environment. Text *before* the end of environment is treated normally, but note that there is no trailing `\obeyedline` added if there is text here. Other than optional arguments, no text is allowed on the opening line of the environment.

Special handling is applied to a `o`, `O`, `d` or `D` specification argument immediately before an `c` specification. This means that when the optional argument is absent, the first character of the next line will be read with the correctly applied verbatim category code. Issues may arise if *multiple* optional arguments are used before a `c` specification: this will only work reliably where the optional tokens are "other" characters.

For technical reasons, we recommend that spaces are *not* ignored when searching for an optional argument before an `c` specification: this can be achieved by adding the `!` modifier as shown in the example. However, this is left as a choice for the user.

1.17 Performance

For document commands where the argument specification is entirely comprised of `m` or `+m` entries (or is entirely empty), the internal structure created by `\NewDocumentCommand` is essentially as efficient as provided by `\newcommand(*)`. As such, document commands may replace constructs arising from `\newcommand`, etc., without a need to be concerned about performance. It should be noted that `\newcommand(*)` produces expandable results, so the direct replacement is `\NewExpandableDocumentCommand`; in most cases, however, it is better to use `\NewDocumentCommand` to give more robust structures.

1.18 Details about argument delimiters

In normal (non-expandable) commands, the delimited types look for the initial delimiter by peeking ahead (using expl3's `\peek_...` functions) looking for the delimiter token. The token has to have the same meaning and "shape" of the token defined as delimiter. There are three possible cases of delimiters: character tokens, control sequence tokens, and active character tokens. For all practical purposes of this description, active character tokens will behave exactly as control sequence tokens.

1.18.1 Character tokens

A character token is characterized by its character code, and its meaning is the category code (`\catcode`). When a command is defined, the meaning of the character token is fixed into the definition of the command and cannot change. A command will correctly see an argument delimiter if the open delimiter has the same character and category codes as at the time of the definition. For example in:

```
\NewDocumentCommand { \foobar } { D<>{default} } {(#1)}
\foobar <hello> \par
\char_set_catcode_letter:N <
\foobar <hello>
```

the output would be:

```
(hello)
(default)<hello>
```

as the open-delimiter `<` changed in meaning between the two calls to `\foobar`, so the second one doesn't see the `<` as a valid delimiter. Commands assume that if a valid open-delimiter was found, a matching close-delimiter will also be there. If it is not (either by being omitted or by changing in meaning), a low-level TEX error is raised and the command call is aborted.

1.18.2 Control sequence tokens

A control sequence (or control character) token is characterized by its name, and its meaning is its definition. A token cannot have two different meanings at the same time. When a control sequence is defined as delimiter in a command, it will be detected as delimiter whenever the control sequence name is found in the document regardless of its current definition. For example in:

```
\cs_set:Npn \x { abc }
\NewDocumentCommand { \foobar } { D\x\y{default} } {(#1)}
\foobar \x hello\y \par
\cs_set:Npn \x { def }
\foobar \x hello\y
```

the output would be:

```
(hello)
(hello)
```

with both calls to the command seeing the delimiter `\x`.

1.19 Creating new argument processors

`\ProcessedArgument`

Argument processors allow manipulation of a grabbed argument before it is passed to the underlying code. New processor implementations may be created as functions which take one trailing argument, and which leave their result in the `\ProcessedArgument` variable. For example, `\ReverseBoolean` is defined as

```
\ExplSyntaxOn
\cs_new_protected:Npn \ReverseBoolean #1
  {
    \bool_if:NTF #1
      { \tl_set:Nn \ProcessedArgument { \c_false_bool } }
      { \tl_set:Nn \ProcessedArgument { \c_true_bool } }
  }
\ExplSyntaxOff
```

[As an aside: the code is written in expl3, so we don't have to worry about spaces creeping into the definition.]

```
⟨@@=cmd⟩
⟨*2ekernel⟩
\message{document commands,}
⟨/2ekernel⟩
```

ltcmd code contains an `^^@` character, which usually has catcode 15, so `\IncludeInRelease` will break when this code is being skipped, so we'll save the catcode of `^^@` to restore later:

```
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\edef\@latexrelease@catcode@null{\the\catcode`\^^@ }
⟨latexrelease⟩\catcode`\^^@=12
\ExplSyntaxOn
⟨latexrelease⟩\NewModuleRelease{2020/10/01}{ltcmd}
⟨latexrelease⟩                    {Document~command~parser}%
\cs_generate_variant:Nn \tl_if_in:nnT { ee }
```

1.20 Variables and constants

`\l__cmd_arg_spec_tl` Holds the argument specification after normalization of shorthands.

```
\tl_new:N \l__cmd_arg_spec_tl
```

`\l__cmd_args_tl` Token list variable for grabbed arguments.

```
\tl_new:N \l__cmd_args_tl
```

`\l__cmd_args_i_tl` `\l__cmd_args_ii_tl` Hold the modified arguments when dealing with default values or processors.

```
\tl_new:N \l__cmd_args_i_tl
\tl_new:N \l__cmd_args_ii_tl
```

`\l__cmd_current_arg_int` The number of the current argument being set up: this is used to make sure there are at most 9 arguments, then for creating the expandable auxiliary functions and knowing how many arguments the code function should take.

```
\int_new:N \l__cmd_current_arg_int
```

`\l__cmd_total_args_int` The total number of arguments found during normalization: this is required where special action is needed for the penultimate argument.

```
\int_new:N \l__cmd_total_args_int
```

`\l__cmd_last_mandatory_arg_int`

Needed to allow handling of verbatim catcodes at the end of the setup.

```
\int_new:N \l__cmd_last_mandatory_arg_int
```

`\l__cmd_defaults_bool` `\l__cmd_defaults_tl` The boolean indicates whether there are any argument with default value other than `\NoValue`; the token list holds the code to determine these default values in terms of other arguments.

```
\bool_new:N \l__cmd_defaults_bool
\tl_new:N \l__cmd_defaults_tl
```

`\l__cmd_environment_bool` Generating environments uses the same mechanism as generating functions. However, full processing of arguments is always needed for environments, and so the function-generating code needs to know this. This variable is also used at run time to give correct error messages.

```
\bool_new:N \l__cmd_environment_bool
```

`\l__cmd_environment_str` Name of the environment, used at definition time and at run time.

```
\str_new:N \l__cmd_environment_str
```

`\l__cmd_expandable_bool` Used to indicate if an expandable command is begin generated, as this affects both the acceptable argument types and how they are implemented.

```
\bool_new:N \l__cmd_expandable_bool
```

`\l__cmd_expandable_aux_name_tl`

Used to create pretty-printing names for the auxiliaries: although the immediate definition does not vary, the full expansion does and so it does not count as a constant.

```
\tl_new:N \l__cmd_expandable_aux_name_tl
\tl_set:Nn \l__cmd_expandable_aux_name_tl
  {
    \l__cmd_function_tl \c_space_tl
    ( arg~ \int_use:N \l__cmd_current_arg_int )
  }
```

`\g__cmd_grabber_int` Used (in exceptional cases) to get unique names for grabbers used by expandable commands.

```
\int_new:N \g__cmd_grabber_int
```

`\l__cmd_fn_tl` For passing the pre-formed name of the auxiliary to be used as the parsing function.

```
\tl_new:N \l__cmd_fn_tl
```

`\l__cmd_fn_code_tl` For passing the pre-formed name of the auxiliary that contains the actual code.

```
\tl_new:N \l__cmd_fn_code_tl
```

`\l__cmd_function_tl` Holds the control sequence name of the function currently being defined: used to avoid passing this as an argument and to avoid repeated use of `\cs_to_str:N`.

```
\tl_new:N \l__cmd_function_tl
```

`\l__cmd_grab_expandably_bool`

When defining a non-expandable command, indicates whether the arguments can all safely be grabbed by expandable grabbers. This is to support abuses of xparse that use protected functions inside csname constructions.

```
\bool_new:N \l__cmd_grab_expandably_bool
```

`\l__cmd_obey_spaces_bool` For trailing optionals.

```
\bool_new:N \l__cmd_obey_spaces_bool
```

`\l__cmd_last_delimiters_tl` Holds the delimiters (first tokens) of all optional arguments since the previous mandatory argument, to warn about cases where it would be impossible to omit optional arguments completely because the following mandatory argument has the same delimiter as one of the optional arguments.

```
\tl_new:N \l__cmd_last_delimiters_tl
```

`\l__cmd_long_bool` Used to indicate that an argument is long, on a per-argument basis.

```
\bool_new:N \l__cmd_long_bool
```

`\l__cmd_suppress_strip_bool`

Used to indicate that an a pair of braces should not be stripped from an optional argument.

```
\bool_new:N \l__cmd_suppress_strip_bool
```

`\l__cmd_m_args_int` The number of `m` arguments: if this is the same as the total number of arguments, then a short-cut can be taken in the creation of the grabber code.

```
\int_new:N \l__cmd_m_args_int
```

`\l__cmd_prefixed_bool` When preparing the signature of non-expandable commands, indicates that the current argument is affected by a processor or by `+` (namely is long).

```
\bool_new:N \l__cmd_prefixed_bool
```

`\l__cmd_process_all_tl`
`\l__cmd_process_one_tl`
`\l__cmd_process_some_bool`

When preparing the signature, the processors that will be applied to a given argument are collected in `\l__cmd_process_one_tl`, while `\l__cmd_process_all_tl` contains processors for all arguments. The boolean indicates whether there are any processors (to bypass the whole endeavour otherwise).

```
\tl_new:N \l__cmd_process_all_tl
\tl_new:N \l__cmd_process_one_tl
\bool_new:N \l__cmd_process_some_bool
```

`\l__cmd_saved_args_tl` Stores `\l__cmd_args_tl` to deal with space-trimming of `b`-type arguments.

```
\tl_new:N \l__cmd_saved_args_tl
```

`\l__cmd_signature_tl` Used when constructing the signature (code for argument grabbing) to hold what will become the implementation of the main function. When arguments are grabbed (at point of use of the command/environment), it also stores the code for grabbing the remaining arguments.

```
\tl_new:N \l__cmd_signature_tl
```

\l__cmd_some_obey_spaces_bool
\l__cmd_some_long_bool
\l__cmd_some_short_bool

These flags are set while normalizing the argument specification. The obey_spaces one is used to detect when ! is used on an argument that is not a trailing optional argument. The other two are used to check whether all short arguments appear before long arguments: this is needed to grab arguments expandably. As soon as the first long argument is seen (other than t-type, whose long status is ignored) the some_long flag is set. The some_short flag is used for expandable commands, to know whether to define a short auxiliary too.

```
\bool_new:N \l__cmd_some_obey_spaces_bool
\bool_new:N \l__cmd_some_long_bool
\bool_new:N \l__cmd_some_short_bool
```

\l__cmd_final_verb_bool

Needed to establish whether optional arguments should be collected "verbatim safe".

```
\bool_new:N \l__cmd_final_verb_bool
```

\q__cmd_recursion_tail
\q__cmd_recursion_stop
__cmd_if_recursion_tail_stop_do:Nn
__cmd_use_i_delimit_by_q_recursion_stop:nw

Quarks and functions for internal processing.

```
\quark_new:N \q__cmd_recursion_tail
\quark_new:N \q__cmd_recursion_stop
\__kernel_quark_new_test:N \__cmd_if_recursion_tail_stop_do:Nn
```

(*End of definition for* __cmd_if_recursion_tail_stop_do:Nn *and* __cmd_use_i_delimit_by_q_recursion_stop:nw.)

\l__cmd_tmp_prop
\l__cmd_tmpa_tl
\l__cmd_tmpb_tl

Scratch space.

```
\prop_new:N \l__cmd_tmp_prop
\tl_new:N \l__cmd_tmpa_tl
\tl_new:N \l__cmd_tmpb_tl
\cs_new_eq:NN \__cmd_tmp:w ?
```

(*End of definition for* __cmd_tmp:w.)

With xparse, information about commands being (re)defined was switched off by default, unless the log-declarations package option was used, so here we'll switch that off as well.

```
\msg_redirect_module:nnn { cmd } { info } { none }
```

Also add cmd to the LaTeX messages.

```
\prop_gput:Nnn \g_msg_module_type_prop { cmd } { LaTeX }
```

1.21 Declaring commands and environments

__cmd_declare_cmd:Nnn
__cmd_declare_expandable_cmd:Nnn
__cmd_declare_cmd_aux:Nnn
__cmd_declare_cmd_internal:Nnnn

The main functions for creating commands set the appropriate flag then use the same internal code to do the definition.

```
\cs_new_protected:Npn \__cmd_declare_cmd:Nnn
  {
```

```
    \bool_set_false:N \l__cmd_expandable_bool
    \__cmd_declare_cmd_aux:Nnn
  }
\cs_new_protected:Npn \__cmd_declare_expandable_cmd:Nnn
  {
    \bool_set_true:N \l__cmd_expandable_bool
    \__cmd_declare_cmd_aux:Nnn
  }
```

The first stage is to log information, both for the user in the log and for programmatic use in a property list of all declared commands.

```
\cs_new_protected:Npn \__cmd_declare_cmd_aux:Nnn #1#2#3
  {
    \cs_if_exist:NTF #1
      {
        \msg_info:nnee { cmd } { redefine }
          { \token_to_str:N #1 } { \tl_to_str:n {#2} }
      }
      {
        \bool_lazy_or:nnT
          { \cs_if_exist_p:c { \cs_to_str:N #1 ~ code } }
          { \cs_if_exist_p:c { \cs_to_str:N #1 ~ defaults } }
          {
            \msg_warning:nne { cmd } { unsupported-let }
              { \token_to_str:N #1 }
          }
        \msg_info:nnee { cmd } { define-command }
          { \token_to_str:N #1 } { \tl_to_str:n {#2} }
      }
    \bool_set_false:N \l__cmd_environment_bool
    \__cmd_declare_cmd_internal:Nnnn #1 {#2} {#3} { }
  }
```

At definition time, the variable `\l__cmd_fn_tl` is only used for error messages. The real business of defining a document command starts with setting up the appropriate name, then normalizing the argument specification to get rid of shorthands.

```
\cs_new_protected:Npn \__cmd_declare_cmd_internal:Nnnn #1#2#3#4
  {
    \tl_set:Ne \l__cmd_function_tl { \cs_to_str:N #1 }
    \__cmd_normalize_arg_spec:n {#2}
    \exp_args:No \__cmd_prepare_signature:n \l__cmd_arg_spec_tl
    \__cmd_declare_cmd_code:Nnn #1 {#2} {#3}
    #4
    \__cmd_break_point:n {#2}
  }
```

(*End of definition for* `\__cmd_declare_cmd:Nnn` *and others.*)

`\__cmd_break_point:n` A marker used to escape from creating a definition if necessary.

```
\cs_new_eq:NN \__cmd_break_point:n \use_none:n
```

(*End of definition for* `\__cmd_break_point:n`.)

`\__cmd_all_m_check:n` A quick loop to check for all (+)m-type arguments.

`\__cmd_all_m_check_aux:n`

```
\cs_new:Npn \__cmd_all_m_check:n #1
```

```
  { \tl_map_function:nN {#1} \__cmd_all_m_check_aux:n }
\cs_new:Npn \__cmd_all_m_check_aux:n #1
  {
    \str_if_eq:nnF {#1} { m }
      {
        \str_if_eq:nnF {#1} { + }
          { X }
      }
  }
```

(*End of definition for* `\__cmd_all_m_check:n` *and* `\__cmd_all_m_check_aux:n`.)

`\__cmd_declare_cmd_code:Nnn`
`\__cmd_declare_cmd_optimized:Nnn`
`\__cmd_declare_cmd_code_aux:Nnn`
`\__cmd_declare_cmd_code_expandable:Nnn`
`\__cmd_start_optimized:`

At this stage we can check for a short-cut possibility: if the argument specification is made up of just `(+)m` tokens, and if all arguments are either short or long, then we can produce an optimized document command. This only applies to document commands, not creation of environments (which are more complex).

```
\cs_new_protected:Npn \__cmd_declare_cmd_code:Nnn #1#2
  {
    \bool_lazy_any:nTF
      {
        { \l__cmd_environment_bool }
        {
          \bool_lazy_and_p:nn
            { \l__cmd_some_short_bool }
            { \l__cmd_some_long_bool }
        }
        { ! \tl_if_blank_p:e { \__cmd_all_m_check:n {#2} } }
      }
      {
        \tl_set:Ne \l__cmd_fn_tl
          { \exp_not:c { \l__cmd_function_tl \c_space_tl } }
        \bool_if:NTF \l__cmd_grab_expandably_bool
          { \__cmd_declare_cmd_code_expandable:Nnn }
          { \__cmd_declare_cmd_code_aux:Nnn }
      }
      { \__cmd_declare_cmd_optimized:Nnn }
        #1 {#2}
   }
```

The optimized version of commands just has to worry about whether to make them protected or long. The commands start with an expandable marker so that other parts of the kernel know these are set up by ltcmd. We need the two layers of redirection so that the `code` internal function has the same form as it would for any other document command. Optimization means that there is no `\group_align_safe_begin:` before grabbing the arguments, so anything involving `&` tokens will not work. However, this is really only intended for making optional argument processing safe anyway, so in practice should not be an issue.

```
\cs_new_protected:Npn \__cmd_declare_cmd_optimized:Nnn #1#2#3
  {
    \bool_if:NTF \l__cmd_expandable_bool
      { \cs_set_nopar:Npe }
      { \cs_set_protected_nopar:Npe }
        #1
        {
```

```
          \exp_not:N \__cmd_start_optimized:
          \exp_not:c { \l__cmd_function_tl \c_space_tl code }
        }
    \exp_args:Ncc \cs_generate_from_arg_count:NNnn
      { \l__cmd_function_tl \c_space_tl code }
      {
        cs_set
        \bool_if:NF \l__cmd_expandable_bool { _protected }
        \bool_if:NF \l__cmd_some_long_bool { _nopar }
        :Npn
      }
      \l__cmd_current_arg_int
      {#3}
  }
\cs_new:Npn \__cmd_start_optimized: { }
```

Standard functions call `\__cmd_start:nNNnnn`, which receives the argument specification, an auxiliary used for grabbing arguments, an auxiliary containing the code, and then the signature, default arguments, and processors.

```
\cs_new_protected:Npn \__cmd_declare_cmd_code_aux:Nnn #1#2#3
  {
    \cs_generate_from_arg_count:cNnn
      { \l__cmd_function_tl \c_space_tl code }
      \cs_set_protected:Npn \l__cmd_current_arg_int {#3}
    \cs_set_protected_nopar:Npe #1
      {
        \bool_if:NTF \l__cmd_environment_bool
          {
            \__cmd_start_env:nnnnn { \exp_not:n {#2} }
              { \l__cmd_environment_str }
          }
          {
            \__cmd_start:nNNnnn { \exp_not:n {#2} }
              \exp_not:c { \l__cmd_function_tl \c_space_tl }
              \exp_not:c { \l__cmd_function_tl \c_space_tl code }
          }
          { \exp_not:o \l__cmd_signature_tl }
          {
            \bool_if:NT \l__cmd_defaults_bool
              { \exp_not:o \l__cmd_defaults_tl }
          }
          {
            \bool_if:NT \l__cmd_process_some_bool
              { \exp_not:o \l__cmd_process_all_tl }
          }
      }
  }
```

Expandable functions and functions whose arguments can be grabbed expandably call `\__cmd_start_expandable:nNNNNn`, which receives the argument specification, four auxiliaries (two for grabbing arguments, one for the code, and one for default arguments), and finally the signature. Non-expandable functions that take this branch should nevertheless be protected, as well as their `code` function. They will only be expanded in contexts such as constructing a csname. The two grabbers (named after the function

with one or two spaces) are needed when there are both short and long arguments; otherwise the same grabber is included twice in the definition. If all arguments are long or all are short the (only) grabber is defined correspondingly to be long/short. Otherwise two grabbers are defined, one long, one short.

```
\cs_new_protected:Npn \__cmd_declare_cmd_code_expandable:Nnn #1#2#3
  {
    \exp_args:Ncc \cs_generate_from_arg_count:NNnn
      { \l__cmd_function_tl \c_space_tl code }
      { cs_set \bool_if:NF \l__cmd_expandable_bool { _protected } :Npn }
      \l__cmd_current_arg_int {#3}
    \bool_if:NT \l__cmd_defaults_bool
      {
        \use:e
          {
            \cs_generate_from_arg_count:cNnn
              { \l__cmd_function_tl \c_space_tl defaults }
              \cs_set:Npn \l__cmd_current_arg_int
              { \exp_not:o \l__cmd_defaults_tl }
          }
      }
    \bool_if:NTF \l__cmd_expandable_bool
      { \cs_set_nopar:Npe } { \cs_set_protected_nopar:Npe } #1
      {
        \exp_not:N \__cmd_start_expandable:nNNNNn
          { \exp_not:n {#2} }
          \exp_not:c { \l__cmd_function_tl \c_space_tl }
          \exp_not:c
            {
              \l__cmd_function_tl \c_space_tl
              \bool_if:NT \l__cmd_some_short_bool
                { \bool_if:NT \l__cmd_some_long_bool { \c_space_tl } }
            }
          \exp_not:c { \l__cmd_function_tl \c_space_tl code }
          \bool_if:NTF \l__cmd_defaults_bool
            { \exp_not:c { \l__cmd_function_tl \c_space_tl defaults } }
            { ? }
          { \exp_not:o \l__cmd_signature_tl }
      }
    \bool_if:NTF \l__cmd_some_long_bool
      {
        \bool_if:NT \l__cmd_some_short_bool
          {
            \cs_set_nopar:cpe { \l__cmd_function_tl \c_space_tl \c_space_tl }
              ##1##2 { ##1 {##2} }
          }
        \cs_set:cpe
      }
      { \cs_set_nopar:cpe }
          { \l__cmd_function_tl \c_space_tl } ##1##2 { ##1 {##2} }
  }
```

(*End of definition for* `\__cmd_declare_cmd_code:Nnn` *and others.*)

__cmd_declare_env:nnnn
__cmd_declare_env:ennn
__cmd_declare_env_internal:nnnn
__cmd_set_environment_end:n

The lead-off to creating an environment is much the same as that for creating a command: issue the appropriate message, store the argument specification then hand off to an internal function.

```
⟨latexrelease⟩\IncludeInRelease{2024/11/01}{\__cmd_declare_env:nnnn}%
⟨latexrelease⟩                  {Use~space-trimmed~envname~directly}
\cs_new_protected:Npn \__cmd_declare_env:nnnn #1#2
  {
    \str_set:Nn \l__cmd_environment_str {#1}
    \cs_if_exist:cTF { #1 }
      { \msg_info:nnnn { cmd } { redefine-env } { #1 } { #2 } }
      { \msg_info:nnnn { cmd } { define-env } { #1 } { #2 } }
    \bool_set_false:N \l__cmd_expandable_bool
    \bool_set_true:N \l__cmd_environment_bool
    \__cmd_declare_env_internal:nnnn {#1} {#2}
  }
\cs_generate_variant:Nn \__cmd_declare_env:nnnn { e }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2024/06/01}{\__cmd_declare_env:nnnn}%
⟨latexrelease⟩                  {Use~space-trimmed~envname~directly}
⟨latexrelease⟩\cs_new_protected:Npn \__cmd_declare_env:nnnn #1#2
⟨latexrelease⟩  {
⟨latexrelease⟩    \str_set:Nx \l__cmd_environment_str {#1}
⟨latexrelease⟩    \str_set:Nx \l__cmd_environment_str
⟨latexrelease⟩      { \tl_trim_spaces:o { \l__cmd_environment_str } }
⟨latexrelease⟩    \cs_if_exist:cTF { \l__cmd_environment_str }
⟨latexrelease⟩      {
⟨latexrelease⟩        \msg_info:nnxx { cmd } { redefine-env }
⟨latexrelease⟩          { \l__cmd_environment_str } { \tl_to_str:n {#2} }
⟨latexrelease⟩      }
⟨latexrelease⟩      {
⟨latexrelease⟩        \msg_info:nnxx { cmd } { define-env }
⟨latexrelease⟩          { \l__cmd_environment_str } { \tl_to_str:n {#2} }
⟨latexrelease⟩      }
⟨latexrelease⟩    \bool_set_false:N \l__cmd_expandable_bool
⟨latexrelease⟩    \bool_set_true:N \l__cmd_environment_bool
⟨latexrelease⟩    \exp_args:NV \__cmd_declare_env_internal:nnnn
⟨latexrelease⟩      \l__cmd_environment_str {#2}
⟨latexrelease⟩  }
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

Creating a document environment requires a few more steps than creating a single command. In order to pass the arguments of the command to the end of the function, it is necessary to store the grabbed arguments. To do that, the function used at the end of the environment has to be redefined to contain the appropriate information. To minimize the amount of expansion at point of use, the code here is expanded now as well as when used. The last argument of `\__cmd_declare_cmd_internal:Nnnn` is only run if the definition succeeded. In package mode this ensures that the original definition of the environment is not changed if the definition fails for any reason. This also avoids an error when defining the `end␣aux␣` function when the user asks for more than 9 arguments.

```
\cs_new_protected:Npn \__cmd_declare_env_internal:nnnn #1#2#3#4
  {
    \exp_args:Nc \__cmd_declare_cmd_internal:Nnnn { environment~ #1 } {#2}
```

```
      {#3}
      {
        \cs_set_nopar:cpe { environment~ #1 ~end }
          { \exp_not:c { environment~ #1 ~end~aux } }
        \cs_generate_from_arg_count:cNnn
          { environment~ #1 ~end~aux~ } \cs_set:Npn
          \l__cmd_current_arg_int {#4}
        \cs_set_eq:cc {#1}       { environment~ #1 }
        \cs_set_eq:cc { end #1 } { environment~ #1 ~end }
      }
  }
\cs_new_protected:Npn \__cmd_set_environment_end:n #1
  {
    \cs_set_nopar:cpe { environment~ #1 ~end~aux }
      {
        \exp_not:c { environment~ #1 ~end~aux~ }
        \exp_not:o \l__cmd_args_tl
      }
  }
```

(*End of definition for* `\__cmd_declare_env:nnnn`, `\__cmd_declare_env_internal:nnnn`, *and* `\__cmd_set_environment_end:n`.)

1.22 Structure of xparse commands

`\__cmd_start_env:nnnnn`
`\__cmd_start:nNNnnn`

For error messages that occur during run-time when getting arguments of environments it is necessary to keep track of the environment name. We begin non-expandable commands with a token equal to `\scan_stop:`, whose name gives a reasonable error message if the command is used inside a csname and protects against `f`-expansion. This is useless for environments since `\begin` is already not expandable. Both the command and environment codes start with `\group_align_safe_begin:`, then `\__cmd_run_code:` (used by both) does `\group_align_safe_end:`, so that delimited arguments may be grabbed in alignments if they contain and alignment tab token (see latex3/latex3/issues/839).

```
\cs_new_protected:Npn \__cmd_start_env:nnnnn #1#2
  {
    \conditionally@traceoff
    \group_align_safe_begin:
    \str_set:Nn \l__cmd_environment_str {#2}
    \bool_set_true:N \l__cmd_environment_bool
    \__cmd_start_aux:ccnnnn
      { environment~ \l__cmd_environment_str \c_space_tl }
      { environment~ \l__cmd_environment_str \c_space_tl code }
      {#1}
  }
\cs_new_protected:Npe \__cmd_start:nNNnnn #1#2#3
  {
    \exp_not:c { xparse~function~is~not~expandable }
    \exp_not:N \conditionally@traceoff
    \exp_not:N \group_align_safe_begin:
    \exp_not:n { \bool_set_false:N \l__cmd_environment_bool }
    \exp_not:N \__cmd_start_aux:NNnnnn
    #2 #3 {#1}
  }
```

(*End of definition for* `\__cmd_start_env:nnnnn` *and* `\__cmd_start:nNNnnn`.)

`\__cmd_start_aux:NNnnnn`
`\__cmd_start_aux:ccnnnn`

This sets up a few variables to minimize the boilerplate code included in all xparse-defined commands. It then runs the grabbers `#4`. Again, the argument specification `#1` is only for diagnostics.

```
\cs_new_protected:Npn \__cmd_start_aux:NNnnnn #1#2#3#4#5#6
  {
    \tl_clear:N \l__cmd_args_tl
    \tl_set:Nn \l__cmd_fn_tl {#1}
    \tl_set:Nn \l__cmd_fn_code_tl {#2}
    \tl_set:Nn \l__cmd_defaults_tl {#5}
    \tl_set:Nn \l__cmd_process_all_tl {#6}
    #4
    \__cmd_run_code:
  }
\cs_generate_variant:Nn \__cmd_start_aux:NNnnnn { cc }
```

(*End of definition for* `\__cmd_start_aux:NNnnnn`.)

`\__cmd_run_code:`

After arguments are grabbed, this function is responsible for inserting default values, running processors, and finally doing `\group_align_safe_end:` as promised, and running the code.

```
\cs_new_protected:Npn \__cmd_run_code:
  {
    \tl_if_empty:NF \l__cmd_defaults_tl { \__cmd_defaults: }
    \tl_if_empty:NF \l__cmd_process_all_tl { \__cmd_args_process: }
    \bool_if:NT \l__cmd_environment_bool
      { \exp_args:No \__cmd_set_environment_end:n \l__cmd_environment_str }
    \group_align_safe_end:
    \conditionally@traceon
    \exp_after:wN \l__cmd_fn_code_tl \l__cmd_args_tl
  }
```

(*End of definition for* `\__cmd_run_code:`.)

`\__cmd_defaults:`
`\__cmd_defaults_def:`
`\__cmd_defaults_def:nn`
`\__cmd_defaults_def:nnn`
`\__cmd_defaults_aux:`
`\__cmd_defaults_error:w`

First construct `\__cmd_tmp:w` (see below) that will receive the arguments found so far and determine default values for any missing argument. Then call it repeatedly until the set of arguments stabilizes. Since that could lead to an infinite loop we only call it up to nine times, the maximal number needed for stabilization if there is a chain of arguments that depend on each other. If that fails to stabilize raise an error.

```
\cs_new_protected:Npn \__cmd_defaults:
  {
    \__cmd_defaults_def:
    \tl_set_eq:NN \l__cmd_args_i_tl \l__cmd_args_tl
    \__cmd_defaults_aux: \__cmd_defaults_aux: \__cmd_defaults_aux:
    \__cmd_defaults_aux: \__cmd_defaults_aux: \__cmd_defaults_aux:
    \__cmd_defaults_aux: \__cmd_defaults_aux: \__cmd_defaults_aux:
    \__cmd_defaults_error:w
    \q_recursion_stop
    \tl_set_eq:NN \l__cmd_args_tl \l__cmd_args_i_tl
  }
\cs_new_protected:Npn \__cmd_defaults_aux:
  {
    \tl_set:Ne \l__cmd_args_ii_tl
```

```
      { \exp_after:wN \__cmd_tmp:w \l__cmd_args_i_tl }
    \tl_if_eq:NNT \l__cmd_args_ii_tl \l__cmd_args_i_tl
      { \use_none_delimit_by_q_recursion_stop:w }
    \tl_set_eq:NN \l__cmd_args_i_tl \l__cmd_args_ii_tl
  }
\cs_new_protected:Npn \__cmd_defaults_error:w \q_recursion_stop
  {
    \msg_error:nne { cmd } { default-loop }
      { \__cmd_environment_or_command: }
  }
```

To construct `\__cmd_tmp:w`, first go through the arguments found and the corresponding defaults, building a token list with `{#⟨arg number⟩}` for arguments found in the input (whose default will not be used) and otherwise `{\exp_not:n{⟨default⟩}}` for arguments whose default will be used.

```
\cs_new_protected:Npn \__cmd_defaults_def:
  {
    \tl_clear:N \l__cmd_tmpa_tl
    \int_zero:N \l__cmd_current_arg_int
    \__cmd_tl_mapthread_function:NNN \l__cmd_args_tl \l__cmd_defaults_tl
      \__cmd_defaults_def:nn
    \cs_generate_from_arg_count:NNno \__cmd_tmp:w \cs_set:Npn
      \l__cmd_current_arg_int \l__cmd_tmpa_tl
  }
\cs_new_protected:Npn \__cmd_defaults_def:nn
  {
    \int_incr:N \l__cmd_current_arg_int
    \exp_args:NV \__cmd_defaults_def:nnn \l__cmd_current_arg_int
  }
\cs_new_protected:Npn \__cmd_defaults_def:nnn #1#2#3
  {
    \tl_put_right:Ne \l__cmd_tmpa_tl
      {
        {
          \exp_not:N \exp_not:n
            {
              \tl_if_novalue:nTF {#2}
                { \exp_not:o {#3} }
                { \exp_not:n { ## #1 } }
            }
        }
      }
  }
```

(*End of definition for* `\__cmd_defaults:` *and others.*)

`\__cmd_args_process:`
`\__cmd_args_process_loop:nn`
`\__cmd_args_process_aux:n`

Loop through arguments (stored in `\l__cmd_args_tl`) and the corresponding processors (in `\l__cmd_process_all_tl`) simultaneously, apply all processors for each argument and store the result back into `\l__cmd_args_tl`. To allow processors to depend on other arguments, for every processor define a temporary auxiliary that receives all arguments `\l__cmd_args_tl`.

```
\cs_new_protected:Npn \__cmd_args_process:
  {
    \tl_clear:N \l__cmd_args_ii_tl
```

```
    \__cmd_tl_mapthread_function:NNN
      \l__cmd_args_tl
      \l__cmd_process_all_tl
      \__cmd_args_process_loop:nn
    \tl_set_eq:NN \l__cmd_args_tl \l__cmd_args_ii_tl
  }
\cs_new_protected:Npn \__cmd_args_process_loop:nn #1#2
  {
    \tl_set:Nn \ProcessedArgument {#1}
    \tl_if_novalue:nF {#1}
      { \tl_map_function:nN {#2} \__cmd_args_process_aux:n }
    \tl_put_right:No \l__cmd_args_ii_tl
      { \exp_after:wN { \ProcessedArgument } }
  }
\cs_new_protected:Npn \__cmd_args_process_aux:n #1
  {
    \cs_generate_from_arg_count:NNnn \__cmd_tmp:w \cs_set:Npn
      { \tl_count:N \l__cmd_args_tl } {#1}
    \exp_args:NNNo \exp_after:wN \__cmd_tmp:w \l__cmd_args_tl
      { \ProcessedArgument }
  }
```

(*End of definition for* `\__cmd_args_process:`, `\__cmd_args_process_loop:nn`, *and* `\__cmd_args_process_aux:n`.)

`\__cmd_start_expandable:nNNNNn`

This is called for all expandable commands. `#6` is the signature, responsible for grabbing arguments. `#5` is used to determine default values (or is `?` if there are none). `#4` is the code to run. `#2` and `#3` are functions (named after the command) that grab a single argument in the input stream (`#3` is short). The argument specification `#1` is only used by diagnostic functions. Same as for the non-expandable version, this starts with `\group_align_safe_begin:`, which expands to nothing, so may be safely used in an expandable context.

```
\cs_new:Npn \__cmd_start_expandable:nNNNNn #1#2#3#4#5#6
  {
    \group_align_safe_begin:
    #6 \__cmd_end_expandable:NNw #5 #4 \q__cmd #2#3
  }
```

(*End of definition for* `\__cmd_start_expandable:nNNNNn`.)

`\__cmd_end_expandable:NNw`
`\__cmd_end_expandable_aux:w`
`\__cmd_end_expandable_aux:nNNNN`
`\__cmd_end_expandable_defaults:nnnNNn`
`\__cmd_end_expandable_defaults:nnw`
`\__cmd_end_expandable_defaults:nw`

Followed by a function `#1` to determine default values (or `?` if there are no defaults), the code `#2`, arguments that have been grabbed, then `\q__cmd` and two generic grabbers. The idea to find default values is similar to the non-expandable case but we cannot define an auxiliary function, so at every step in the loop we need to go through all arguments searching for which ones started out as `\NoValue` and replacing these by the newly computed values. In fact we need to keep track of three versions of all arguments: the original version, the previous version with default values, and the currently built version (first argument of `\__cmd_end_expandable_defaults:nnnNNn`).

```
\cs_new:Npn \__cmd_end_expandable:NNw #1#2
  { \__cmd_end_expandable_aux:w #1#2 \prg_do_nothing: }
\cs_new:Npn \__cmd_end_expandable_aux:w #1#2#3 \q__cmd
  { \exp_args:No \__cmd_end_expandable_aux:nNNNN {#3} #1 #2 }
\cs_new:Npn \__cmd_end_expandable_aux:nNNNN #1#2#3#4#5
```

```
  {
    \token_if_eq_charcode:NNT ? #2 { \exp_after:wN \use_iv:nnnn }
    \__cmd_end_expandable_defaults:nnnNNn {#1} { } {#1} #2#3
      { } { } { } { } { } { } { } { } { } { }
      {
        \msg_expandable_error:nnf { cmd } { default-loop }
          { \exp_args:Nf \tl_trim_spaces:n { \token_to_str:N #4 } }
        \use_iv:nnnn
      }
    \q_stop
  }
\cs_new:Npn \__cmd_end_expandable_defaults:nnnNNn #1#2#3#4#5#6
  {
    #6
    \str_if_eq:nnTF {#1} {#2}
      { \use_i_delimit_by_q_stop:nw { \group_align_safe_end: #5 #1 } }
      {
        \exp_args:No \__cmd_tl_mapthread_function:nnN
          { #4 #1 } {#3}
          \__cmd_end_expandable_defaults:nnw
        \__cmd_end_expandable_defaults:nnnNNn { } {#1} {#3} #4 #5
      }
  }
\cs_new:Npn \__cmd_end_expandable_defaults:nnw #1#2
  {
    \tl_if_novalue:nTF {#2}
      { \exp_args:No \__cmd_end_expandable_defaults:nw {#1} }
      { \__cmd_end_expandable_defaults:nw {#2} }
  }
\cs_new:Npn \__cmd_end_expandable_defaults:nw
    #1#2 \__cmd_end_expandable_defaults:nnnNNn #3
  { #2 \__cmd_end_expandable_defaults:nnnNNn { #3 {#1} } }
```

(*End of definition for* `\__cmd_end_expandable:NNw` *and others.*)

1.23 Normalizing the argument specifications

The goal here is to expand aliases and check that the argument specification is valid before the main parsing run. If it is not valid the entire set up is abandoned to avoid any strange internal errors. A function is provided for each argument type that will grab any extra data items and call the loop function after performing the following checks and tasks.

- Check that each argument has the correct number of data items associated with it, and that where a single character is required, one has actually been supplied.
- Check that processors and the markers `+`, `!` and `=` are followed by an argument for which they make sense, and are not redundant.
- Check the absence of forbidden types for expandable commands, namely `G`/`v` always, and `l`/`u` after optional arguments (xparse may have inserted braces due to a failed search for an optional argument).
- Check that no optional argument is followed by a mandatory argument with the same delimiter, as otherwise the optional argument could never be omitted.

- Keep track in `\l__cmd_some_long_bool` and `\l__cmd_some_short_bool` of whether the command has some long/short arguments.
- Keep track in `\l__cmd_grab_expandably_bool` of whether all arguments are `m`/`l`/`u` type and short arguments appear before long ones, in which case they can be grabbed expandably just as safely as they could be grabbed nonexpandably. Regardless of that, arguments of expandable commands will be grabbed expandably and arguments of environments will not (because the list of arguments built by non-expandable grabbing is used to pass them to the end-environment code).

Further checks happen at the end of the loop:

- that there are at most 9 arguments;
- that an expandable command does not end with an optional argument (this case is detected by using the fact that `\l__cmd_last_delimiters_tl` is cleared by every mandatory argument and filled by every optional argument).

`\__cmd_normalize_arg_spec:n`
`\__cmd_normalize_arg_spec_loop:n`

Loop through the argument specification, calling an auxiliary specific to each argument type. If any argument is unknown stop the definition.

```
\cs_new_protected:Npn \__cmd_normalize_arg_spec:n #1
  {
    \int_zero:N \l__cmd_current_arg_int
    \int_zero:N \l__cmd_last_mandatory_arg_int
    \tl_clear:N \l__cmd_last_delimiters_tl
    \tl_clear:N \l__cmd_arg_spec_tl
    \bool_set_true:N \l__cmd_grab_expandably_bool
    \bool_set_false:N \l__cmd_obey_spaces_bool
    \bool_set_false:N \l__cmd_long_bool
    \bool_set_false:N \l__cmd_suppress_strip_bool
    \bool_set_false:N \l__cmd_some_obey_spaces_bool
    \bool_set_false:N \l__cmd_some_long_bool
    \bool_set_false:N \l__cmd_some_short_bool
    \__cmd_normalize_arg_spec_loop:n #1
      \q_recursion_tail \q_recursion_tail \q_recursion_tail \q_recursion_stop
    \int_compare:nNnT \l__cmd_current_arg_int > 9
      {
        \msg_error:nnee { cmd } { too-many-args }
          { \__cmd_environment_or_command: } { \tl_to_str:n {#1} }
        \__cmd_bad_def:wn
      }
    \bool_if:NT \l__cmd_expandable_bool
      {
        \tl_if_empty:NF \l__cmd_last_delimiters_tl
          {
            \msg_error:nnee { cmd } { expandable-ending-optional }
              { \iow_char:N \\ \l__cmd_function_tl } { \tl_to_str:n {#1} }
            \__cmd_bad_def:wn
          }
      }
    \bool_if:NT \l__cmd_expandable_bool
      { \bool_set_true:N \l__cmd_grab_expandably_bool }
    \bool_if:NT \l__cmd_environment_bool
      { \bool_set_false:N \l__cmd_grab_expandably_bool }
```

```
  }
\cs_new_protected:Npn \__cmd_normalize_arg_spec_loop:n #1
  {
    \quark_if_recursion_tail_stop:n {#1}
    \int_incr:N \l__cmd_current_arg_int
    \cs_if_exist_use:cF { __cmd_normalize_type_ \tl_to_str:n {#1} :w }
      {
        \bool_lazy_any:nTF
          {
            { \str_if_eq_p:nn {#1} { G } }
            { \str_if_eq_p:nn {#1} { g } }
            { \str_if_eq_p:nn {#1} { l } }
            { \str_if_eq_p:nn {#1} { u } }
          }
          {
            \msg_error:nnee { cmd } { xparse-arg-type }
              { \__cmd_environment_or_command: } { \tl_to_str:n {#1} }
          }
          {
            \msg_error:nnee { cmd } { unknown-argument-type }
              { \__cmd_environment_or_command: } { \tl_to_str:n {#1} }
          }
        \__cmd_bad_def:wn
      }
  }
```

(*End of definition for* `\__cmd_normalize_arg_spec:n` *and* `\__cmd_normalize_arg_spec_loop:n`.)

`\__cmd_normalize_type_d:w`
`\__cmd_normalize_type_e:w`
`\__cmd_normalize_type_o:w`
`\__cmd_normalize_type_O:w`
`\__cmd_normalize_type_r:w`
`\__cmd_normalize_type_s:w`

These argument types are aliases of more general ones, for example with the default argument `\NoValue`. For argument types that need additional data, check that the data is present (not `\q_recursion_tail`) before proceeding.

```
\cs_new_protected:Npn \__cmd_normalize_type_d:w #1#2
  {
    \quark_if_recursion_tail_stop_do:nn {#2} { \__cmd_bad_arg_spec:wn }
    \__cmd_normalize_type_D:w {#1} {#2} { \NoValue }
  }
\cs_new_protected:Npn \__cmd_normalize_type_e:w #1
  {
    \quark_if_recursion_tail_stop_do:nn {#1} { \__cmd_bad_arg_spec:wn }
    \__cmd_normalize_type_E:w {#1} { }
  }
\cs_new_protected:Npn \__cmd_normalize_type_o:w
  { \__cmd_normalize_type_D:w [ ] { \NoValue } }
\cs_new_protected:Npn \__cmd_normalize_type_O:w
  { \__cmd_normalize_type_D:w [ ] }
\cs_new_protected:Npn \__cmd_normalize_type_r:w #1#2
  {
    \quark_if_recursion_tail_stop_do:nn {#2} { \__cmd_bad_arg_spec:wn }
    \__cmd_normalize_type_R_aux:w { r #1 #2 } {#1} {#2} { \NoValue }
  }
\cs_new_protected:Npn \__cmd_normalize_type_s:w
  { \__cmd_normalize_type_t:w * }
```

(*End of definition for* `\__cmd_normalize_type_d:w` *and others.*)

__cmd_normalize_type_>:w
__cmd_normalize_type_+:w
__cmd_normalize_type_!:w
__cmd_normalize_type_=:w
__cmd_normalize_type_aux:NnNn

Check that these prefixes have arguments, namely that the next token is not `\q_recursion_tail`, and remember to leave it after the looping macro. Processors are forbidden in expandable commands. If all is good, store the prefix in the cleaned up `\l__cmd_arg_spec_tl`, and decrement the argument number as prefixes do not correspond to arguments.

```
\cs_new_protected:cpn { __cmd_normalize_type_>:w } #1#2
  {
    \quark_if_recursion_tail_stop_do:nn {#2} { \__cmd_bad_arg_spec:wn }
    \bool_if:NT \l__cmd_expandable_bool
      {
        \msg_error:nnee { cmd } { processor-in-expandable }
          { \iow_char:N \\ \l__cmd_function_tl } { \tl_to_str:n {#1} }
        \__cmd_bad_def:wn
      }
    \tl_put_right:Ne \l__cmd_arg_spec_tl { > { \tl_trim_spaces:n {#1} } }
    \int_decr:N \l__cmd_current_arg_int
    \bool_set_false:N \l__cmd_grab_expandably_bool
    \__cmd_normalize_arg_spec_loop:n {#2}
  }
\cs_new_protected:cpn { __cmd_normalize_type_+:w } #1
  {
    \__cmd_normalize_type_aux:NnNn + {#1}
      \l__cmd_long_bool
      { \bool_set_true:N \l__cmd_long_bool }
  }
\cs_new_protected:cpn { __cmd_normalize_type_!:w } #1
  {
    \__cmd_normalize_type_aux:NnNn ! {#1}
      \l__cmd_obey_spaces_bool
      {
        \bool_set_true:N \l__cmd_obey_spaces_bool
        \bool_set_true:N \l__cmd_some_obey_spaces_bool
      }
  }
\cs_new_protected:cpn { __cmd_normalize_type_=:w } #1#2
  {
    \__cmd_normalize_type_aux:NnNn = {#2}
      \l__cmd_suppress_strip_bool
      {
        \bool_if:NT \l__cmd_expandable_bool
          {
            \msg_error:nnee { cmd } { keyval-in-expandable }
              { \iow_char:N \\ \l__cmd_function_tl } { \tl_to_str:n {#1} }
            \__cmd_bad_def:wn
          }
        \bool_set_true:N \l__cmd_suppress_strip_bool
        \bool_set_false:N \l__cmd_grab_expandably_bool
        \tl_put_right:Ne \l__cmd_arg_spec_tl
          { = { \tl_trim_spaces:n {#1} } }
      }
  }
\cs_new_protected:Npn \__cmd_normalize_type_aux:NnNn #1#2#3#4
  {
```

```
    \quark_if_recursion_tail_stop_do:nn {#2} { \__cmd_bad_arg_spec:wn }
    \bool_if:NT #3
      {
        \msg_error:nnee { cmd } { two-markers }
          { \__cmd_environment_or_command: } { #1 }
        \__cmd_bad_def:wn
      }
    #4
    \int_decr:N \l__cmd_current_arg_int
    \__cmd_normalize_arg_spec_loop:n {#2}
  }
```

(*End of definition for* `\__cmd_normalize_type_>:w` *and others.*)

`\__cmd_normalize_type_D:w`
`\__cmd_normalize_type_E:w`
`\__cmd_normalize_type_t:w`
`\__cmd_normalize_E_unique_check:w`

Optional argument types. Check that all required data is present (and consists of single characters if applicable) and check for forbidden types for expandable commands. For E-type require that there is at least one embellishment, that each one is a single character, and that there aren't more optional arguments than embellishments; also remember that each embellishment counts as one argument for `\l__cmd_current_arg_int`. Then in each case store the data in `\l__cmd_arg_spec_tl`, and for later checks store in `\l__cmd_last_delimiters_tl` the tokens whose presence determines whether there is an optional argument (for braces store `{}`, seen later as an empty delimiter).

```
\cs_new_protected:Npn \__cmd_normalize_type_D:w #1#2#3
  {
    \quark_if_recursion_tail_stop_do:nn {#3} { \__cmd_bad_arg_spec:wn }
    \__cmd_single_token_check:n {#1} \__cmd_allowed_token_check:N #1
    \__cmd_single_token_check:n {#2}
    \__cmd_add_arg_spec:n { D #1 #2 {#3} }
    \tl_put_right:Nn \l__cmd_last_delimiters_tl {#1}
    \bool_set_false:N \l__cmd_grab_expandably_bool
    \__cmd_normalize_arg_spec_loop:n
  }
\cs_new_protected:Npn \__cmd_normalize_type_E:w #1#2
  {
    \quark_if_recursion_tail_stop_do:nn {#2} { \__cmd_bad_arg_spec:wn }
    \tl_if_blank:nT {#1} { \__cmd_bad_arg_spec:wn }
    \tl_map_function:nN {#1} \__cmd_single_token_check:n
    \tl_map_function:nN {#1} \__cmd_allowed_token_check:N
    \__cmd_normalize_E_unique_check:w #1 \q_nil \q_stop
    \int_compare:nNnT { \tl_count:n {#2} } > { \tl_count:n {#1} }
      { \__cmd_bad_arg_spec:wn }
    \__cmd_add_arg_spec:n { E {#1} {#2} }
    \tl_put_right:Nn \l__cmd_last_delimiters_tl {#1}
    \bool_set_false:N \l__cmd_grab_expandably_bool
    \int_add:Nn \l__cmd_current_arg_int { \tl_count:n {#1} - 1 }
    \__cmd_normalize_arg_spec_loop:n
  }
\cs_new_protected:Npn \__cmd_normalize_E_unique_check:w #1#2 \q_stop
  {
    \quark_if_nil:NF #1
      {
        \tl_if_in:nnT {#2} {#1} { \__cmd_bad_arg_spec:wn }
        \__cmd_normalize_E_unique_check:w #2 \q_stop
      }
```

```
  }
\cs_new_protected:Npn \__cmd_normalize_type_t:w #1
  {
    \quark_if_recursion_tail_stop_do:Nn #1 { \__cmd_bad_arg_spec:wn }
    \__cmd_single_token_check:n {#1} \__cmd_allowed_token_check:N #1
    \tl_put_right:Ne \l__cmd_arg_spec_tl
      {
        \bool_if:NT \l__cmd_obey_spaces_bool { ! }
        t \exp_not:n {#1}
      }
    \tl_put_right:Nn \l__cmd_last_delimiters_tl {#1}
    \bool_set_false:N \l__cmd_grab_expandably_bool
    \bool_set_false:N \l__cmd_obey_spaces_bool
    \bool_set_false:N \l__cmd_long_bool
    \__cmd_normalize_arg_spec_loop:n
  }
```

(*End of definition for* `\__cmd_normalize_type_D:w` *and others.*)

`\__cmd_normalize_type_m:w`
`\__cmd_normalize_type_R:w`
`\__cmd_normalize_type_R_aux:w`
`\__cmd_normalize_type_v:w`

Mandatory arguments. First check the required data is present, consists of single characters where applicable, and that the argument type is allowed for expandable commands if applicable. For the `m` and `R` argument types check that they do not follow some optional argument with that delimiter as otherwise the optional argument could not be omitted. Then save data in `\l__cmd_arg_spec_tl`, count the mandatory argument, and empty the list of last delimiters.

```
\cs_new_protected:Npn \__cmd_normalize_type_m:w
  {
    \__cmd_delimiter_check:nnn { } { m } { \iow_char:N \{ }
    \__cmd_add_arg_spec_mandatory:n { m }
    \__cmd_normalize_arg_spec_loop:n
  }
\cs_new_protected:Npn \__cmd_normalize_type_R:w #1#2#3
  {
    \quark_if_recursion_tail_stop_do:nn {#3} { \__cmd_bad_arg_spec:wn }
    \__cmd_normalize_type_R_aux:w { R #1 #2 {#3} } {#1} {#2} {#3}
  }
\cs_new_protected:Npn \__cmd_normalize_type_R_aux:w #1#2#3#4
  {
    \__cmd_single_token_check:n {#2} \__cmd_allowed_token_check:N #2
    \__cmd_single_token_check:n {#3}
    \__cmd_delimiter_check:nnn {#2} { R/r } { \tl_to_str:n {#2} }
    \bool_set_false:N \l__cmd_grab_expandably_bool
    \__cmd_add_arg_spec_mandatory:nn { R #2 #3 {#4} } {#1}
    \__cmd_normalize_arg_spec_loop:n
  }
\cs_new_protected:Npn \__cmd_normalize_type_v:w
  {
    \__cmd_normalize_check_gv:N v
    \__cmd_add_arg_spec_mandatory:n { v }
    \__cmd_normalize_arg_spec_loop:n
  }
```

(*End of definition for* `\__cmd_normalize_type_m:w` *and others.*)

__cmd_normalize_type_b:w
__cmd_normalize_type_c:w
__cmd_normalize_type_b_or_c:nn

These argument types are not allowed for commands. They are only allowed at the end of the argument specification, hence we check that `#1` is the end.

```
\cs_new_protected:Npn \__cmd_normalize_type_b:w #1
  { \__cmd_normalize_type_b_or_c:nn {#1} { b } }
\cs_new_protected:Npn \__cmd_normalize_type_c:w #1
  { \__cmd_normalize_type_b_or_c:nn {#1} { c } }
\cs_new_protected:Npn \__cmd_normalize_type_b_or_c:nn #1#2
  {
    \bool_if:NF \l__cmd_environment_bool
      {
        \msg_error:nnee { cmd } { invalid-command-arg }
          { \__cmd_environment_or_command: } {#2}
        \__cmd_bad_def:wn
      }
    \tl_clear:N \l__cmd_last_delimiters_tl
    \__cmd_add_arg_spec:n {#2}
    \quark_if_recursion_tail_stop:n {#1}
    \msg_error:nneee { cmd } { arg-after-body }
      {#2}
      { \__cmd_environment_or_command: }
      { \tl_to_str:n {#1} }
    \__cmd_bad_def:wn
  }
```

(*End of definition for* `\__cmd_normalize_type_b:w`, `\__cmd_normalize_type_c:w`, *and* `\__cmd_normalize_type_b_or_c:nn`.)

__cmd_single_token_check:n

Checks that the argument is a single (non-space) token (possibly surrounded by spaces), and aborts the definition otherwise.

```
\cs_new_protected:Npn \__cmd_single_token_check:n #1
  {
    \tl_trim_spaces_apply:nN {#1} \tl_if_single_token:nF
      {
        \msg_error:nnee { cmd } { not-single-token }
          { \__cmd_environment_or_command: } { \tl_to_str:n {#1} }
        \__cmd_bad_def:wn
      }
  }
```

(*End of definition for* `\__cmd_single_token_check:n`.)

__cmd_allowed_token_check:N

Some tokens are not allowed as delimiters for some argument types, notably implicit begin/end-group tokens (`\bgroup`/`\egroup`). The major problem with these tokens is that for `\peek_...` functions, a literal $\texttt{\{}_1$ is virtually indistinguishable from a `\bgroup` or other token which was `\let` to a $\texttt{\{}_1$, and the same goes for $\texttt{\}}_2$. All other tokens can be easily distinguished from their implicit counterparts by grabbing them and looking at the string length (see `\__cmd_token_if_cs:NTF`), but for begin/end group tokens that is not possible without the risk of mistakenly grabbing the entire brace group (potentially leading to a `! Runaway argument` error) or trying to grab a $\texttt{\}}_2$, leading to an `! Argument of \dots has an extra }` error.

```
\cs_new_protected:Npn \__cmd_allowed_token_check:N #1
  {
    \token_if_eq_meaning:NNTF #1 \c_group_begin_token
        { \use:n }
```

```
        {
          \token_if_eq_meaning:NNTF #1 \c_group_end_token
            { \use:n }
            { \use_none:n }
        }
      {
        \msg_error:nneee { cmd } { forbidden-group-token }
          { \__cmd_environment_or_command: } { \tl_to_str:n {#1} }
          {
            \token_if_eq_meaning:NNTF #1 \c_group_begin_token
              { begin } { end }
          }
        \__cmd_bad_def:wn
      }
  }
```

(*End of definition for* `\__cmd_allowed_token_check:N`.)

`\__cmd_normalize_check_gv:N` `\__cmd_normalize_check_lu:N` Called for arguments that are always forbidden, or forbidden after an optional argument, for expandable commands.

```
\cs_new_protected:Npn \__cmd_normalize_check_gv:N #1
  {
    \bool_if:NT \l__cmd_expandable_bool
      {
        \msg_error:nnee { cmd } { invalid-expandable-arg }
          { \iow_char:N \\ \l__cmd_function_tl } { \tl_to_str:n {#1} }
        \__cmd_bad_def:wn
      }
    \bool_set_false:N \l__cmd_grab_expandably_bool
  }
\cs_new_protected:Npn \__cmd_normalize_check_lu:N #1
  {
    \bool_if:NT \l__cmd_expandable_bool
      {
        \tl_if_empty:NF \l__cmd_last_delimiters_tl
          {
            \msg_error:nnee { cmd } { invalid-after-optional-expandably }
              { \iow_char:N \\ \l__cmd_function_tl } { \tl_to_str:n {#1} }
            \__cmd_bad_def:wn
          }
      }
  }
```

(*End of definition for* `\__cmd_normalize_check_gv:N` *and* `\__cmd_normalize_check_lu:N`.)

`\__cmd_delimiter_check:nnn` Called for `m` and `R` arguments. Checks that the leading token does not coincide with the token denoting the presence of a previous optional argument. Instead of dealing with braces for the `m`-type we use an empty delimiter to denote that case.

```
\cs_new_protected:Npn \__cmd_delimiter_check:nnn #1#2#3
  {
    \tl_map_inline:Nn \l__cmd_last_delimiters_tl
      {
        \tl_if_eq:nnT {##1} {#1}
          {
```

```
            \msg_warning:nnee { cmd } { optional-mandatory }
              {#2} {#3}
          }
      }
  }
```

(*End of definition for* `\__cmd_delimiter_check:nnn`.)

`\__cmd_bad_arg_spec:wn` `\__cmd_bad_def:wn`

If the argument specification is wrong, this provides an escape from the entire definition process.

```
\cs_new_protected:Npn \__cmd_bad_arg_spec:wn #1 \__cmd_break_point:n #2
  {
    \msg_error:nnee { cmd } { bad-arg-spec }
      { \__cmd_environment_or_command: } { \tl_to_str:n {#2} }
  }
\cs_new_protected:Npn \__cmd_bad_def:wn #1 \__cmd_break_point:n #2 { }
```

(*End of definition for* `\__cmd_bad_arg_spec:wn` *and* `\__cmd_bad_def:wn`.)

`\__cmd_add_arg_spec:n` `\__cmd_add_arg_spec_mandatory:n` `\__cmd_add_arg_spec_mandatory:nn`

When adding an argument to the argument specification, set the `some_long` or `some_short` booleans as appropriate and clear the booleans keeping track of `+`, `!` and `=` markers. Before that, test for a short argument following some long arguments: this is forbidden for expandable commands and prevents grabbing arguments expandably.

For mandatory arguments do some more work, in particular complain if they were preceded by `!`.

```
\cs_new_protected:Npn \__cmd_add_arg_spec:n #1
  {
    \bool_lazy_and:nnT
      { ! \l__cmd_long_bool }
      { \l__cmd_some_long_bool }
      {
        \bool_if:NT \l__cmd_expandable_bool
          {
            \msg_error:nne { cmd } { long-short-mix }
              { \iow_char:N \\ \l__cmd_function_tl }
            \__cmd_bad_def:wn
          }
        \bool_set_false:N \l__cmd_grab_expandably_bool
      }
    \bool_if:NTF \l__cmd_long_bool
      { \bool_set_true:N \l__cmd_some_long_bool }
      { \bool_set_true:N \l__cmd_some_short_bool }
    \tl_put_right:Ne \l__cmd_arg_spec_tl
      {
        \bool_lazy_and:nnT
          { \l__cmd_long_bool }
          { ! \str_if_eq_p:nn {#1} { c } }
          { + }
        \bool_if:NT \l__cmd_obey_spaces_bool { ! }
        \exp_not:n {#1}
      }
    \bool_set_false:N \l__cmd_long_bool
    \bool_set_false:N \l__cmd_obey_spaces_bool
  }
```

```
\cs_new_protected:Npn \__cmd_add_arg_spec_mandatory:n #1
  { \__cmd_add_arg_spec_mandatory:nn { #1 } { #1 } }
\cs_new_protected:Npn \__cmd_add_arg_spec_mandatory:nn #1 #2
  {
    \bool_if:NT \l__cmd_some_obey_spaces_bool
      {
        \msg_error:nnee { cmd } { invalid-bang }
          { \__cmd_environment_or_command: }
          {
            \bool_if:NTF \l__cmd_obey_spaces_bool
              { \tl_to_str:n {'#2'} }
              { an~optional~argument~before~mandatory~ \tl_to_str:n {'#2'} }
          }
        \__cmd_bad_def:wn
      }
    \tl_clear:N \l__cmd_last_delimiters_tl
    \int_set_eq:NN \l__cmd_last_mandatory_arg_int \l__cmd_current_arg_int
    \__cmd_add_arg_spec:n {#1}
  }
```

(*End of definition for* `\__cmd_add_arg_spec:n`, `\__cmd_add_arg_spec_mandatory:n`, *and* `\__cmd_add_arg_spec_mandatory:nn`.)

1.24 Preparing the signature: general mechanism

`\__cmd_prepare_signature:n`
`\__cmd_prepare_signature_verb_chk:n`
`\__cmd_prepare_signature:N`
`\__cmd_prepare_signature_bypass:N`

Actually creating the signature uses the same loop approach as normalizing the signature. There are first a number of variables which need to be set to track what is going on. Many of these variables are unused when defining expandable commands.

```
\cs_new_protected:Npn \__cmd_prepare_signature:n #1
  {
    \int_set_eq:NN \l__cmd_total_args_int \l__cmd_current_arg_int
    \int_zero:N \l__cmd_current_arg_int
    \bool_set_false:N \l__cmd_long_bool
    \bool_set_false:N \l__cmd_obey_spaces_bool
    \bool_set_false:N \l__cmd_suppress_strip_bool
    \int_zero:N \l__cmd_m_args_int
    \bool_set_false:N \l__cmd_defaults_bool
    \tl_clear:N \l__cmd_defaults_tl
    \tl_clear:N \l__cmd_process_all_tl
    \tl_clear:N \l__cmd_process_one_tl
    \bool_set_false:N \l__cmd_process_some_bool
    \tl_clear:N \l__cmd_signature_tl
    \bool_set_false:N \l__cmd_final_verb_bool
    \__cmd_prepare_signature_verb_chk:n {#1}
    \__cmd_prepare_signature:N #1 \q_recursion_tail \q_recursion_stop
    \bool_if:NF \l__cmd_expandable_bool { \__cmd_flush_m_args: }
  }
```

A quick check on the final arg. type, done using a string approach (that retains braces) to avoid an issue with `{c}` as the last balanced text.

```
\cs_new_protected:Npn \__cmd_prepare_signature_verb_chk:n #1
  {
    \tl_if_in:eeT
      { \tl_trim_spaces:e { \tl_to_str:n {#1} } \exp_not:N \q_stop }
```

```
      { \tl_to_str:n { c } \exp_not:N \q_stop }
      { \bool_set_true:N \l__cmd_final_verb_bool }
  }
```

The main looping function does not take an argument, but carries out the reset on the processor boolean. This is split off from the rest of the process so that when actually setting up processors the flag-reset can be bypassed.

For each known argument type there is an appropriate function to actually do the addition to the signature. These are separate for expandable and standard functions, as the approaches are different.

```
\cs_new_protected:Npn \__cmd_prepare_signature:N
  {
    \bool_set_false:N \l__cmd_prefixed_bool
    \__cmd_prepare_signature_bypass:N
  }
\cs_new_protected:Npn \__cmd_prepare_signature_bypass:N #1
  {
    \quark_if_recursion_tail_stop:N #1
    \use:c
      {
         __cmd_add
         \bool_if:NT \l__cmd_grab_expandably_bool { _expandable }
         _type_  \token_to_str:N #1 :w
      }
  }
```

(*End of definition for* `\__cmd_prepare_signature:n` *and others.*)

1.25 Setting up a standard signature

Each argument-adding function appends to the signature a grabber (and for some types, the delimiters or default value), except the one for `m` arguments. These are collected and added to the signature all at once by `\__cmd_flush_m_args:`, called for every other argument type. All of the functions then call the loop function `\__cmd_prepare_signature:N`. Default values of arguments are collected by `\__cmd_add_default:n` rather than being stored with the argument; this function and `\__cmd_add_default:` are also responsible for keeping track of `\l__cmd_current_arg_int`.

`\__cmd_add_type_+:w` Making the next argument long means setting the flag. The `m` arguments are recorded here as this has to be done for every case where there is then a long argument.

```
\cs_new_protected:cpn { __cmd_add_type_+:w }
  {
    \__cmd_flush_m_args:
    \bool_set_true:N \l__cmd_long_bool
    \bool_set_true:N \l__cmd_prefixed_bool
    \__cmd_prepare_signature_bypass:N
  }
```

(*End of definition for* `\__cmd_add_type_+:w`.)

`\__cmd_add_type_!:w` Much the same for controlling trailing optional arguments.

```
\cs_new_protected:cpn { __cmd_add_type_!:w }
  {
    \__cmd_flush_m_args:
```

```
    \bool_set_true:N \l__cmd_obey_spaces_bool
    \bool_set_true:N \l__cmd_prefixed_bool
    \__cmd_prepare_signature_bypass:N
  }
```

(*End of definition for* `\__cmd_add_type_!:w`.)

`\__cmd_add_type_>:w` When a processor is found, the processor code is stored. It will be used by `\__cmd_args_process:` once arguments are all found. Here too the loop calls `\__cmd_prepare_signature_bypass:N` rather than `\__cmd_prepare_signature:N` so that the flag is not reset.

```
\cs_new_protected:cpn { __cmd_add_type_>:w } #1
  {
    \__cmd_flush_m_args:
    \bool_set_true:N \l__cmd_prefixed_bool
    \bool_set_true:N \l__cmd_process_some_bool
    \tl_put_left:Nn \l__cmd_process_one_tl { {#1} }
    \__cmd_prepare_signature_bypass:N
  }
```

(*End of definition for* `\__cmd_add_type_>:w`.)

`\__cmd_add_type_=:w` A mix of the ideas from above: set a flag and add a processor.

```
\cs_new_protected:cpn { __cmd_add_type_=:w } #1
  {
    \__cmd_flush_m_args:
    \bool_set_true:N \l__cmd_prefixed_bool
    \bool_set_true:N \l__cmd_suppress_strip_bool
    \bool_set_true:N \l__cmd_process_some_bool
    \tl_put_left:Nn \l__cmd_process_one_tl
      { { \__cmd_arg_to_keyvalue:nn {#1} } }
    \__cmd_prepare_signature_bypass:N
  }
```

(*End of definition for* `\__cmd_add_type_=:w`.)

`\__cmd_add_type_b:w`
`\__cmd_add_type_c:w`
`\__cmd_add_type_b_or_c:N`

```
\cs_new_protected:Npn \__cmd_add_type_b:w
  { \__cmd_add_type_b_or_c:N b }
\cs_new_protected:Npn \__cmd_add_type_c:w
  { \__cmd_add_type_b_or_c:N c }
\cs_new_protected:Npn \__cmd_add_type_b_or_c:N #1
  {
    \__cmd_flush_m_args:
    \__cmd_add_default:
    \__cmd_add_grabber:N #1
    \__cmd_prepare_signature:N
  }
```

(*End of definition for* `\__cmd_add_type_b:w`, `\__cmd_add_type_c:w`, *and* `\__cmd_add_type_b_or_c:N`.)

`\__cmd_add_type_D:w`

```
\cs_new_protected:Npn \__cmd_add_type_D:w #1#2#3
  {
    \__cmd_flush_m_args:
```

```
    \__cmd_add_default:n {#3}
    \__cmd_add_grabber:N D
    \tl_put_right:Nn \l__cmd_signature_tl { #1 #2 }
    \__cmd_prepare_signature:N
  }
```

(*End of definition for* `\__cmd_add_type_D:w`.)

`\__cmd_add_type_E:w` The E-type argument needs a special handling of default values. Since each embellishment is a separate argument, it also needs to replicate the argument processors for each embellishment argument so that the numbers of arguments and processors remain in sync.

```
\cs_new_protected:Npn \__cmd_add_type_E:w #1#2
  {
    \__cmd_flush_m_args:
    \__cmd_add_default_E:nn {#1} {#2}
    \use:e
      {
        \__cmd_replicate_processor:nn { \tl_count:n {#1} }
          { \exp_not:o \l__cmd_process_one_tl }
      }
    \__cmd_add_grabber:N E
    \tl_put_right:Nn \l__cmd_signature_tl { {#1} }
    \__cmd_prepare_signature:N
  }
```

(*End of definition for* `\__cmd_add_type_E:w`.)

`\__cmd_replicate_processor:nn` In the command's argument processor signature (the final argument of `\__cmd_start:nNNnnn`) there is one braced item for each formal argument (up to nine), and in each of these items there is one braced item for each processor (as many as there were processors declared for a given argument). Something like this:

```
{ % argument processors
  { % argument 1
    { processor 1 } { processor 2 } ... { processor n }
  } % end argument 1
  { ... } % argument 2
     ⋮
  { ... } % argument n
} % end argument processors
```

The function `\__cmd_add_grabber:N` adds one single grabber for an argument, and adds the braced item for that one argument. However, in an E-type argument each embellishment requires its own formal argument, so we need to break out of one layer of braces in `\l__cmd_process_one_tl`, add copies of the processor as necessary, and then return the removed brace. The function below does just that: it defines `\l__cmd_process_one_tl` starting with a $\}_2$ and ending with a $\{_1$, so that it adds as many processors as needed when x-expanded.

```
\cs_new_protected:Npn \__cmd_replicate_processor:nn #1 #2
```

```
  {
    \int_compare:nNnF {#1} > { 1 } { \use_none:nnn }
    \tl_set:Ne \l__cmd_process_one_tl
      {
        \exp_not:n { \exp_not:n {#2} \if_false: { \fi: } }
        \prg_replicate:nn { #1 - 2 }
          { \exp_not:n { \exp_not:n { {#2} } } }
        \exp_not:n { { \if_false: } \fi: \exp_not:n {#2} }
      }
  }
```

(*End of definition for* `\__cmd_replicate_processor:nn`.)

`\__cmd_add_type_m:w` The `m` type is special as short arguments which are not post-processed are simply counted at this stage. Thus there is a check to see if either of these cases apply. If so, a one-argument grabber is added to the signature. On the other hand, if a standard short argument is required it is simply counted at this stage, to be added later using `\__cmd_flush_m_args:`.

```
\cs_new_protected:Npn \__cmd_add_type_m:w
  {
    \__cmd_add_default:
    \bool_if:NTF \l__cmd_prefixed_bool
      { \__cmd_add_grabber:N m }
      { \int_incr:N \l__cmd_m_args_int }
    \__cmd_prepare_signature:N
  }
```

(*End of definition for* `\__cmd_add_type_m:w`.)

`\__cmd_add_type_R:w` The `R`-type argument is very similar to the `D`-type.

```
\cs_new_protected:Npn \__cmd_add_type_R:w #1#2#3
  {
    \__cmd_flush_m_args:
    \__cmd_add_default:n {#3}
    \__cmd_add_grabber:N R
    \tl_put_right:Nn \l__cmd_signature_tl { #1 #2 }
    \__cmd_prepare_signature:N
  }
```

(*End of definition for* `\__cmd_add_type_R:w`.)

`\__cmd_add_type_t:w` Setting up a `t` argument means collecting one token for the test, and adding it along with the grabber to the signature.

```
\cs_new_protected:Npn \__cmd_add_type_t:w #1
  {
    \__cmd_flush_m_args:
    \__cmd_add_default:
    \__cmd_add_grabber:N t
    \tl_put_right:Nn \l__cmd_signature_tl {#1}
    \__cmd_prepare_signature:N
  }
```

(*End of definition for* `\__cmd_add_type_t:w`.)

__cmd_add_type_v:w At this stage, the v argument is identical to l except that since the grabber may fail to read a verbatim argument we need a default value.

```
\cs_new_protected:Npn \__cmd_add_type_v:w
  {
    \__cmd_flush_m_args:
    \__cmd_add_default:n { \NoValue }
    \__cmd_add_grabber:N v
    \__cmd_prepare_signature:N
  }
```

(*End of definition for* `\__cmd_add_type_v:w`.)

__cmd_flush_m_args: As m arguments are simply counted, there is a need to add them to the token register in a block. As this function can only be called if something other than m turns up, the flag can be switched here.

```
\cs_new_protected:Npn \__cmd_flush_m_args:
  {
    \int_compare:nNnT \l__cmd_m_args_int > 0
      {
        \tl_put_right:Ne \l__cmd_signature_tl
          { \exp_not:c { __cmd_grab_m_ \int_use:N \l__cmd_m_args_int :w } }
        \tl_put_right:Ne \l__cmd_process_all_tl
          { \prg_replicate:nn { \l__cmd_m_args_int } { { } } }
      }
    \int_zero:N \l__cmd_m_args_int
  }
```

(*End of definition for* `\__cmd_flush_m_args:`.)

__cmd_add_grabber:N To keep the various checks needed in one place, adding the grabber to the signature is done here. The only questions are whether the grabber should be long or not, and whether to obey spaces. The `\l__cmd_obey_spaces_bool` boolean can only be `true` for trailing optional arguments. In that case spaces will not be ignored when looking for that optional argument.

```
\cs_new_protected:Npn \__cmd_add_grabber:N #1
  {
    \tl_put_right:Ne \l__cmd_signature_tl
      {
        \exp_not:c
          {
            __cmd_grab_ #1
            \bool_if:NT \l__cmd_long_bool { _long }
            \bool_if:NT \l__cmd_obey_spaces_bool { _obey_spaces }
            \bool_lazy_and:nnT
              { \l__cmd_suppress_strip_bool }
              { \str_if_eq_p:nn {#1} { D } }
              { _no_strip }
            \bool_lazy_all:nT
              {
                { \l__cmd_final_verb_bool }
                { \str_if_eq_p:nn {#1} { D } }
                {
                  \int_compare_p:nNn \l__cmd_current_arg_int
                    > \l__cmd_last_mandatory_arg_int
```

```
              }
            }
            { _verb_safe }
          :w
        }
    }
    \bool_set_false:N \l__cmd_long_bool
    \bool_set_false:N \l__cmd_obey_spaces_bool
    \bool_set_false:N \l__cmd_suppress_strip_bool
    \bool_set_false:N \l__cmd_verb_safe_bool
    \tl_put_right:Ne \l__cmd_process_all_tl
      {
        {
          \if_charcode:w E #1 \use_i:nn \fi:
          \exp_not:o \l__cmd_process_one_tl
        }
      }
    \tl_clear:N \l__cmd_process_one_tl
  }
```

(*End of definition for* `\__cmd_add_grabber:N`.)

`\__cmd_add_default:n` `\__cmd_add_default:` `\__cmd_add_default_E:nn`

Store the default value of an argument, or rather code that gives that default value (it may involve other arguments). This is (in a brace group) `\prg_do_nothing:` followed by a default value or `\NoValue`. For `E`-type arguments, pad the defaults `#2` with some `\NoValue` until there are as many as embellishments `#1`. These functions are also used when defining expandable commands. The `\prg_do_nothing:` here are removed in `\__cmd_defaults_def:nnn`: for a user-supplied default the token is injected to avoid brace stripping, and for `\NoValue` means we have a common code path later.

```
\cs_new_protected:Npn \__cmd_add_default:n #1
  {
    \tl_if_novalue:nTF {#1}
      { \__cmd_add_default: }
      {
        \int_incr:N \l__cmd_current_arg_int
        \bool_set_true:N \l__cmd_defaults_bool
        \tl_put_right:Nn \l__cmd_defaults_tl { { \prg_do_nothing: #1 } }
      }
  }
\cs_new_protected:Npn \__cmd_add_default:
  {
    \int_incr:N \l__cmd_current_arg_int
    \tl_put_right:Nn \l__cmd_defaults_tl { { \prg_do_nothing: \NoValue } }
  }
\cs_new_protected:Npn \__cmd_add_default_E:nn #1#2
  {
    \tl_map_function:nN {#2} \__cmd_add_default:n
    \prg_replicate:nn
      { \tl_count:n {#1} - \tl_count:n {#2} }
      { \__cmd_add_default: }
  }
```

(*End of definition for* `\__cmd_add_default:n`, `\__cmd_add_default:`, *and* `\__cmd_add_default_E:nn`.)

1.26 Setting up expandable types

The approach here is not dissimilar to that for standard types, but fewer types are supported. There is also a need to define the per-function auxiliaries: this is done here, while the general grabbers are dealt with later.

__cmd_add_expandable_type_+:w

We have already checked that short arguments are before long arguments, so `\l__cmd_long_bool` only changes from `false` to `true` once (and there is no need to reset it after each argument). Continue the loop.

```
\cs_new_protected:cpn { __cmd_add_expandable_type_+:w }
  {
    \bool_set_true:N \l__cmd_long_bool
    \__cmd_prepare_signature:N
  }
```

(*End of definition for* `\__cmd_add_expandable_type_+:w`.)

__cmd_add_expandable_type_D:w
__cmd_add_expandable_type_D_aux:NNNn
__cmd_add_expandable_type_D_aux:NNN
__cmd_add_expandable_type_D_aux:NN

The set up for `D`-type arguments involves constructing a rather complex auxiliary which is used repeatedly when grabbing. There is an auxiliary here so that the `R`-type can share code readily: `#1` is `D` or `R`. The `_aux:NN` auxiliary is needed if the two delimiting tokens are identical: in contrast to the non-expandable route, the grabber here has to act differently for this case.

```
\cs_new_protected:Npn \__cmd_add_expandable_type_D:w
  { \__cmd_add_expandable_type_D_aux:NNNn D }
\cs_new_protected:Npn \__cmd_add_expandable_type_D_aux:NNNn #1#2#3#4
  {
    \__cmd_add_default:n {#4}
    \tl_if_eq:nnTF {#2} {#3}
      { \__cmd_add_expandable_type_D_aux:NN #1 #2 }
      { \__cmd_add_expandable_type_D_aux:NNN #1 #2 #3 }
    \__cmd_prepare_signature:N
  }
\cs_new_protected:Npn \__cmd_add_expandable_type_D_aux:NNN #1#2#3
  {
    \bool_if:NTF \l__cmd_long_bool
      { \cs_set:cpe }
      { \cs_set_nopar:cpe }
      { \l__cmd_expandable_aux_name_tl } ##1 ##2 #2 ##3 \q__cmd ##4 #3
      { ##1 {##2} {##3} {##4} }
    \__cmd_add_expandable_grabber:nn {#1}
      {
        \exp_not:c  { \l__cmd_expandable_aux_name_tl }
        \exp_not:n { #2 #3 }
      }
  }
\cs_new_protected:Npn \__cmd_add_expandable_type_D_aux:NN #1#2
  {
    \bool_if:NTF \l__cmd_long_bool
      { \cs_set:cpe }
      { \cs_set_nopar:cpe }
      { \l__cmd_expandable_aux_name_tl } ##1 #2 ##2 #2
      { ##1 {##2} }
    \__cmd_add_expandable_grabber:nn { #1_alt }
      {
```

```
        \exp_not:c  { \l__cmd_expandable_aux_name_tl }
        \exp_not:n {#2}
      }
  }
```

(*End of definition for* `\__cmd_add_expandable_type_D:w` *and others.*)

`\__cmd_add_expandable_type_E:w`
`\__cmd_add_expandable_type_E_aux:n`

For each embellishment, use `\__cmd_get_grabber:NN` to obtain an auxiliary delimited by that token and store a pair constituted of the auxiliary and the token in `\l__cmd_tmpb_tl`, before appending the whole set of these pairs to the signature, and an equal number of `\NoValue` markers (regardless of the default values of arguments). Set the current argument appropriately.

```
\cs_new_protected:Npn \__cmd_add_expandable_type_E:w #1#2
  {
    \__cmd_add_default_E:nn {#1} {#2}
    \tl_clear:N \l__cmd_tmpb_tl
    \tl_map_function:nN {#1} \__cmd_add_expandable_type_E_aux:n
    \__cmd_add_expandable_grabber:nn
      { E \bool_if:NT \l__cmd_long_bool { _long } }
      {
        { \exp_not:o \l__cmd_tmpb_tl }
        {
          \prg_replicate:nn { \tl_count:n {#1} }
            { { \NoValue } }
        }
      }
    \__cmd_prepare_signature:N
  }
\cs_new_protected:Npn \__cmd_add_expandable_type_E_aux:n #1
  {
    \__cmd_get_grabber:NN #1 \l__cmd_tmpa_tl
    \tl_put_right:Ne \l__cmd_tmpb_tl
      { \exp_not:o \l__cmd_tmpa_tl \exp_not:N #1 }
  }
```

(*End of definition for* `\__cmd_add_expandable_type_E:w` *and* `\__cmd_add_expandable_type_E_aux:n`.)

`\__cmd_add_expandable_type_m:w`

Unlike the standard case, when working expandably each argument is always grabbed separately.

```
\cs_new_protected:Npn \__cmd_add_expandable_type_m:w
  {
    \__cmd_add_default:
    \__cmd_add_expandable_grabber:nn
      { m \bool_if:NT \l__cmd_long_bool { _long } } { }
    \__cmd_prepare_signature:N
  }
```

(*End of definition for* `\__cmd_add_expandable_type_m:w`.)

`\__cmd_add_expandable_type_R:w`

The `R`-type is very similar to the `D`-type argument, and so the same internals are used.

```
\cs_new_protected:Npn \__cmd_add_expandable_type_R:w
  { \__cmd_add_expandable_type_D_aux:NNNn R }
```

(*End of definition for* `\__cmd_add_expandable_type_R:w`.)

__cmd_add_expandable_type_t:w

An auxiliary delimited by #1 is built now. It will be used to test for the presence of that token.

```
\cs_new_protected:Npn \__cmd_add_expandable_type_t:w #1
  {
    \__cmd_add_default:
    \__cmd_get_grabber:NN #1 \l__cmd_tmpa_tl
    \__cmd_add_expandable_grabber:nn { t }
      {
        \exp_not:o \l__cmd_tmpa_tl
        \exp_not:N #1
      }
    \__cmd_prepare_signature:N
  }
```

(*End of definition for* __cmd_add_expandable_type_t:w.)

__cmd_add_expandable_grabber:nn

This is called for all arguments to place the right grabber in the signature.

```
\cs_new_protected:Npn \__cmd_add_expandable_grabber:nn #1#2
  {
    \tl_put_right:Ne \l__cmd_signature_tl
      { \exp_not:c { __cmd_expandable_grab_ #1 :w } #2 }
  }
```

(*End of definition for* __cmd_add_expandable_grabber:nn.)

__cmd_get_grabber:NN
__cmd_get_grabber_auxi:NN
__cmd_get_grabber_auxii:NN

Given a token #1, defines an expandable function delimited by that token and stores it in the token list #2. The function is named after the token, unless that function name is already taken by some other grabber (this can happen in the rare case where delimiters with different category codes are used in the same document): in that case use a global counter to get a unique name. Since the grabbers are not named after xparse commands they should not be used to get material from the input stream.

```
\cs_new_protected:Npn \__cmd_get_grabber:NN #1#2
  {
    \cs_set:Npn \__cmd_tmp:w ##1 #1 {##1}
    \exp_args:Nc \__cmd_get_grabber_auxi:NN
      { __cmd_grabber_ \token_to_str:N #1 :w } #2
  }
\cs_new_protected:Npn \__cmd_get_grabber_auxi:NN #1#2
  {
    \cs_if_eq:NNTF \__cmd_tmp:w #1
      { \tl_set:Nn #2 {#1} }
      {
        \cs_if_exist:NTF #1
          {
            \int_gincr:N \g__cmd_grabber_int
            \exp_args:Nc \__cmd_get_grabber_auxi:NN
              {
                __cmd_grabber_
                - \int_use:N \g__cmd_grabber_int :w
              }
              #2
          }
          { \__cmd_get_grabber_auxii:NN #1 #2 }
      }
```

```
  }
\cs_new_protected:Npn \__cmd_get_grabber_auxii:NN #1#2
  {
    \cs_set_eq:NN #1 \__cmd_tmp:w
    \tl_set:Nn #2 {#1}
  }
```

(*End of definition for* `\__cmd_get_grabber:NN`*,* `\__cmd_get_grabber_auxi:NN`*, and* `\__cmd_get_grabber_auxii:NN`*.*)

1.26.1 Copying a command and its internal structure

```
⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\__cmd_copy:NN}%
⟨latexrelease⟩   {Support~\NewCommandCopy~in~ltcmd}
```

Since the 2020-10-01 LaTeX 2_ε release, support for copying, and showing the definition of, robust commands has been available, but the specifics of each command are implemented separately. Here we'll add support for copying and showing ltcmd definitions.

To fully support copying, we need two commands: a conditional to test if a command is in fact a ltcmd command, and another command to actually copy the command. The conditional is defined later as `\__kernel_cmd_if_xparse:NTF`, so now to the copying:

`\__cmd_copy:NN`
`\__cmd_set_eq_if_exist:NN`
`\__cmd_set_eq_if_exist:cc`

This macro just branches to the proper copying command by using `\__cmd_cmd_type_cases:NnnnnnF`. The copying command takes the names of the commands to be copied to and from, and the actual commands as its four arguments.

```
\cs_new_protected:Npn \__cmd_copy:NN #1 #2
  {
    \use:e
      {
        \int_set:Nn \tex_escapechar:D { 92 }
        \exp_not:N \__cmd_cmd_type_cases:NnnnnnF \exp_not:N #2
          { \__cmd_copy_command:nnNN }
          { \__cmd_copy_expandable:nnNN }
          { \__cmd_copy_optimized:nnNN }
          { \__cmd_copy_environment:nnNN }
          { \__cmd_copy_environment_end:nnNN }
          { \__cmd_cant_copy:nwn { non-ltcmd } }
            { \cs_to_str:N #1 } { \cs_to_str:N #2 }
            \exp_not:N #1 \exp_not:N #2
        \exp_not:N \__cmd_break_point:n { \cs_to_str:N #2 }
        \int_set:Nn \tex_escapechar:D { \int_use:N \tex_escapechar:D }
      }
  }
\cs_new_protected:Npn \__cmd_set_eq_if_exist:NN #1 #2
  { \cs_if_exist:NTF #2 { \cs_set_eq:NN } { \use_none:nn } #1 #2 }
\cs_generate_variant:Nn \__cmd_set_eq_if_exist:NN { cc }
```

An utility macro similar to `\__cmd_bad_def:wn` to abort a command copy. Contrary to `\__cmd_bad_def:wn` though, when this happens the issue is most likely internal, because the command was already (supposedly) correctly defined so it should be copyable. Hopefully this macro will never be used ever, but if it does, apologize and give the reason for the failure so the user can report.

`\__cmd_cant_copy:nwn`

```
\cs_new_protected:Npn \__cmd_cant_copy:nwn #1 #2 \__cmd_break_point:n #3
  { \msg_error:nnnn { cmd } { copy-bug } {#1} {#3} }
```

```
\msg_new:nnn { cmd } { copy-bug }
  {
    Error~while~copying~command~\iow_char:N\\#2:\\
    \str_case:nn {#1}
      {
        { non-ltcmd } { Command~is~not~a~valid~ltcmd~command. }
        { unknown-type } { Found~an~unknown~argument~type. }
        { invalid-end }
          { Target~command~is~not~named~\iow_char:N \\end<name>. }
      }
  }
```

And, of course, add `\__kernel_cmd_if_xparse:NTF` and `\__cmd_copy:NN` to `\@declarecommandcopylisthook`:

```
\tl_gput_right:Nn \@declarecommandcopylisthook
  { { \__kernel_cmd_if_xparse:NTF \__cmd_copy:NN } }
```

(*End of definition for* `\__cmd_copy:NN`*,* `\__cmd_set_eq_if_exist:NN`*, and* `\__cmd_cant_copy:nwn`.)

`\__cmd_copy_command:nnNN`
`\__cmd_copy_command:NnNNnnnn`

A normal (non-expandable) command has a pretty straightforward structure. Its definition is stored in $\langle$*cmd*$\rangle_\sqcup$`code`, its defaults (if any) are stored in $\langle$*cmd*$\rangle_\sqcup$`defaults`, and its top-level definition contains its signature, which can just be copied over. `\__cmd_copy_command:nnNN` copies the command code and defaults, and then defines the top-level command using the auxiliary `\__cmd_copy_command:NnNNnnnn`. This macro takes the signature of the command being copied from its top-level definition, and replaces the named bits with the new name.

```
\cs_new_protected:Npn \__cmd_copy_command:nnNN #1 #2 #3 #4
  {
    \cs_set_eq:cc { #1 ~ code } { #2 ~ code }
    \__cmd_set_eq_if_exist:cc { #1 ~ defaults } { #2 ~ defaults }
    \cs_set_protected_nopar:Npe #3
      { \exp_after:wN \__cmd_copy_command:NnNNnnnn #4 {#1} }
  }
\cs_new:Npn \__cmd_copy_command:NnNNnnnn #1 #2 #3 #4 #5 #6 #7 #8
  {
    #1 \exp_not:n { {#2} }
    \exp_not:c { #8 ~ } \exp_not:c { #8 ~ code }
    \exp_not:n { {#5} {#6} {#7} }
  }
```

(*End of definition for* `\__cmd_copy_command:nnNN` *and* `\__cmd_copy_command:NnNNnnnn`.)

`\__cmd_copy_expandable:nnNN`
`\__cmd_copy_expandable:NnNNNNnnn`

An expandable command is slightly more complicated. Besides the $\langle$*cmd*$\rangle_\sqcup$`code`, and $\langle$*cmd*$\rangle_\sqcup$`defaults`, it also has an auxiliary $\langle$*cmd*$\rangle_\sqcup$ for grabbing delimited arguments, and possibly another auxiliary $\langle$*cmd*$\rangle_{\sqcup\sqcup}$, if the command has both long and short arguments. Then, its signature also has several specific bits that are unique to that command; this is in contrast to non-expandable commands, which use a common set of parsing functions.

We start by copying the basics, then call `\__cmd_copy_expandable_signature:NnNNNNnnn` to parse the signature of the command and build up the modified copy in a temporary token list, then we call `\__cmd_copy_expandable:NnNNNNnnn` that will copy the top-level definition of the command, with the proper internal renames.

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__cmd_copy:NN}%
```

```
⟨latexrelease⟩  {Support~\NewCommandCopy~in~ltcmd}
⟨latexrelease⟩\EndIncludeInRelease
```

There's one variant: a command begins with `\__cmd_start_expandable:nNNNNn` may still be un-expandable/protected if it's defined by `\NewDocumentCommand` and friends, with empty or only m-type arguments.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__cmd_copy_expandable:nnNN}%
⟨latexrelease⟩  {Distinguish~non-expandable~document~commands}
\cs_new_protected:Npn \__cmd_copy_expandable:nnNN #1 #2 #3 #4
  {
    \cs_set_eq:cc { #1 ~ code } { #2 ~ code }
    \__cmd_set_eq_if_exist:cc { #1 ~ } { #2 ~ }
    \__cmd_set_eq_if_exist:cc { #1 ~ \c_space_tl } { #2 ~ \c_space_tl }
    \__cmd_set_eq_if_exist:cc { #1 ~ defaults } { #2 ~ defaults }
    \exp_after:wN \__cmd_copy_expandable_signature:NnNNNNnnn #4 {#1} {#2}
    \token_if_protected_macro:NTF #4
      { \cs_set_protected_nopar:Npe }{ \cs_set_nopar:Npe }
      #3
      { \exp_after:wN \__cmd_copy_expandable:NnNNNNnnn #4 {#1} {#2} }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\__cmd_copy_expandable:nnNN}%
⟨latexrelease⟩  {Support~\NewCommandCopy~in~ltcmd}
⟨latexrelease⟩\cs_new_protected:Npn \__cmd_copy_expandable:nnNN #1 #2 #3 #4
⟨latexrelease⟩  {
⟨latexrelease⟩    \cs_set_eq:cc { #1 ~ code } { #2 ~ code }
⟨latexrelease⟩    \__cmd_set_eq_if_exist:cc { #1 ~ } { #2 ~ }
⟨latexrelease⟩    \__cmd_set_eq_if_exist:cc { #1 ~ \c_space_tl } { #2 ~ \c_space_tl }
⟨latexrelease⟩    \__cmd_set_eq_if_exist:cc { #1 ~ defaults } { #2 ~ defaults }
⟨latexrelease⟩    \exp_after:wN \__cmd_copy_expandable_signature:NnNNNNnnn #4 {#1} {#2}
⟨latexrelease⟩    \cs_set_nopar:Npx #3
⟨latexrelease⟩      { \exp_after:wN \__cmd_copy_expandable:NnNNNNnnn #4 {#1} {#2} }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__cmd_copy_expandable:nnNN}%
⟨latexrelease⟩  {Support~\NewCommandCopy~in~ltcmd}
⟨latexrelease⟩\EndIncludeInRelease
```

Copy the code, simply define the wrapper.

__cmd_copy_optimized:nnNN

```
\cs_new_protected:Npn \__cmd_copy_optimized:nnNN #1#2#3#4
  {
    \cs_set_eq:cc { #1 ~ code } { #2 ~ code }
    \token_if_protected_macro:NTF #4
      { \cs_set_protected_nopar:Npe }
      { \cs_set_nopar:Npe }
        #3
        {
          \exp_not:N \__cmd_start_optimized:
          \exp_not:c { #1 ~ code }
        }
  }
```

```
⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\__cmd_copy:NN (part 2)}%
⟨latexrelease⟩  {Support~\NewCommandCopy~in~ltcmd}
```

```
\cs_new:Npn \__cmd_copy_expandable:NnNNNNnnn #1 #2 #3 #4 #5 #6 #7 #8 #9
  {
    \exp_not:N #1 \exp_not:n { {#2} }
    \exp_not:c { #8 ~ }
    \exp_not:c
      {
        #8 ~
        \str_if_eq:eeT
            { \exp_not:c { #9 ~ \c_space_tl } } { \exp_not:N #4 }
          { \c_space_tl }
      }
    \exp_not:c { #8 ~ code }
    \str_if_eq:eeTF { \exp_not:N #6 } { ? }
      { ? }
      { \exp_not:c { #8 ~ defaults } }
    { \exp_not:V \l__cmd_tmpa_tl }
  }
```

A signature for an expandable command contains as many `\expandable_grab_⟨type⟩:w` as there are arguments, and what follows this macro depends on the ⟨*type*⟩. We'll start a loop through the signature, and at each argument grabber, we'll step the argument count, and look for the ⟨*type*⟩ with `\__cmd_copy_parse_grabber:w` so that we know which `\__cmd_copy_grabber_⟨type⟩:w` to call next.

__cmd_copy_expandable_signature:NnNNNNnnn
__cmd_copy_expandable:nnN
__cmd_copy_parse_grabber:w

```
\cs_new_protected:Npn \__cmd_copy_expandable_signature:NnNNNNnnn
    #1 #2 #3 #4 #5 #6 #7 #8 #9
  {
    \int_zero:N \l__cmd_current_arg_int
    \tl_clear:N \l__cmd_tmpa_tl
    \__cmd_copy_expandable:nnN {#8} {#9} #7
      \q_recursion_tail \q_recursion_stop
  }
\cs_new_protected:Npn \__cmd_copy_expandable:nnN #1 #2 #3
  {
    \quark_if_recursion_tail_stop:n {#3}
    \int_incr:N \l__cmd_current_arg_int
    \exp_after:wN \__cmd_copy_parse_grabber:w \token_to_str:N #3 {#1} {#2}
  }
\use:e
  {
    \cs_new_protected:Npn \exp_not:N \__cmd_copy_parse_grabber:w #1
        \tl_to_str:n { expandable_grab_ } #2 \tl_to_str:n { :w }
      {
        \tl_put_right:Ne \exp_not:N \l__cmd_tmpa_tl
          { \exp_not:N \exp_not:c { __cmd_expandable_grab_#2:w } }
        \exp_not:N \cs_if_exist_use:cF { __cmd_copy_grabber_#2:w }
          { \__cmd_cant_copy:nwn { unknown-type } }
      }
  }
```

The most complicated is the `Delimited` argument: each argument has a dedicated grabbing function named after the command that has to be copied over (of the form `\⟨cmd⟩␣(arg␣⟨num⟩)`).

```
\cs_new_protected:Npn \__cmd_copy_grabber_D:w #1 #2 #3 #4 #5
```

__cmd_copy_grabber_D:w
__cmd_copy_grabber_D_alt:w
__cmd_copy_grabber_R:w
__cmd_copy_grabber_R_alt:w
__cmd_copy_grabber_E:w
__cmd_copy_grabber_E_long:w
__cmd_copy_grabber_t:w
__cmd_copy_grabber_m:w
__cmd_copy_grabber_m_long:w

```
  {
    \tl_put_right:Ne \l__cmd_tmpa_tl
      {
        \exp_not:c { #1 ~ (arg ~ \int_use:N \l__cmd_current_arg_int ) }
        \exp_not:n { #4 #5 }
      }
    \cs_set_eq:cc
      { #1 ~ (arg ~ \int_use:N \l__cmd_current_arg_int ) }
      { #2 ~ (arg ~ \int_use:N \l__cmd_current_arg_int ) }
    \__cmd_copy_expandable:nnN {#1} {#2}
  }
```

`D_alt` is just a special case of `D` that uses a single delimiter (used when both delimiters of the argument are identical):

```
\cs_new_protected:Npn \__cmd_copy_grabber_D_alt:w #1 #2 #3 #4
  { \__cmd_copy_grabber_D:w {#1} {#2} {#3} {#4} { } }
```

As far as copying is concerned, `R` is identical to `D`:

```
\cs_new_eq:NN \__cmd_copy_grabber_R:w \__cmd_copy_grabber_D:w
\cs_new_eq:NN \__cmd_copy_grabber_R_alt:w \__cmd_copy_grabber_D_alt:w
```

`E` is straightforward: we just copy the embellishments over, and increase the current argument number `\l__cmd_current_arg_int` by the number of embellishments (minus one because there is a `\int_incr:N` down the line).

```
\cs_new_protected:Npn \__cmd_copy_grabber_E:w #1 #2 #3 #4
  {
    \tl_put_right:Nn \l__cmd_tmpa_tl { {#3} {#4} }
    \int_add:Nn \l__cmd_current_arg_int { \tl_count:n {#4} - 1 }
    \__cmd_copy_expandable:nnN {#1} {#2}
  }
\cs_new_eq:NN \__cmd_copy_grabber_E_long:w \__cmd_copy_grabber_E:w
```

`t` just needs copying the token to be tested for:

```
\cs_new_protected:Npn \__cmd_copy_grabber_t:w #1 #2 #3 #4
  {
    \tl_put_right:Nn \l__cmd_tmpa_tl { #3 #4 }
    \__cmd_copy_expandable:nnN {#1} {#2}
  }
```

And last but not least, `m` is the simplest; the grabber is just `\__cmd_expandable_grab_m:w`, which is already added to the new command so here we just resume the loop:

```
\cs_new_protected:Npn \__cmd_copy_grabber_m:w { \__cmd_copy_expandable:nnN }
\cs_new_eq:NN \__cmd_copy_grabber_m_long:w \__cmd_copy_grabber_m:w
```

(*End of definition for* `\__cmd_copy_expandable:nnNN` *and others.*)

`\__cmd_copy_environment:nnNN`
`\__cmd_copy_environment:Nnnnnnn`

Copying an environment's `\begin` part is pretty much like copying a command, except it has a longer name, and at the end we have to copy `\environment` ⟨*name*⟩ into `\`⟨*name*⟩.

```
\cs_new_protected:Npn \__cmd_copy_environment:nnNN #1 #2 #3 #4
  {
    \cs_set_eq:cc { environment~ #1 ~ code } { environment~ #2 ~ code }
    \__cmd_set_eq_if_exist:cc
      { environment~ #1 ~ defaults } { environment~ #2 ~ defaults }
    \cs_set_protected_nopar:cpe { environment~ #1 }
      { \exp_after:wN \__cmd_copy_environment:Nnnnnnn #4 {#1} }
    \cs_set_eq:cc {#1} { environment~ #1 }
```

```
  }
\cs_new:Npn \__cmd_copy_environment:Nnnnnnn #1 #2 #3 #4 #5 #6 #7
  { #1 \exp_not:n { {#2} } {#7} \exp_not:n { {#4} {#5} {#6} } }
```

(*End of definition for* `\__cmd_copy_environment:nnNN` *and* `\__cmd_copy_environment:Nnnnnnn`.)

`\__cmd_copy_environment_end:nnNN`
`\__cmd_copy_environment_end_aux:nnNN`

Copying an environment's `\end` part is a bit trickier. We first have to make sure that both parts are named `\end`⟨*name*⟩ (that's actually not a hard requirement, but an environment `\end` command makes no sense without the `end` in its name), and strip the leading `end` from the strings. After that, copying is straightforward.

```
\cs_new_protected:Npn \__cmd_copy_environment_end:nnNN #1 #2
  {
    \__cmd_check_end:Nn \l__cmd_tmpa_tl {#1}
    \__cmd_check_end:Nn \l__cmd_tmpb_tl {#2}
    \exp_args:Noo \__cmd_copy_environment_end_aux:nnNN
      { \l__cmd_tmpa_tl } { \l__cmd_tmpb_tl }
  }
\cs_new_protected:Npn \__cmd_copy_environment_end_aux:nnNN #1 #2 #3 #4
  {
    \cs_set_nopar:cpe { environment~ #1 ~end }
      { \exp_not:c { environment~ #1 ~end~aux } }
    \cs_set_eq:cc
      { environment~ #1 ~end~aux~ } { environment~ #2 ~end~aux~ }
    \cs_set_eq:cc { end #1 } { environment~ #1 ~end }
  }
```

To check whether an `\end` command is valid, we look for the string `end` at the beginning of the command name, and if not found, raise an error:

`\__cmd_check_end:Nn`
`\__cmd_check_end:n`
`\__cmd_check_end:w`

```
\cs_new_protected:Npn \__cmd_check_end:Nn #1 #2
  {
    \tl_set:Ne #1 { \__cmd_check_end:n {#2} }
    \token_if_eq_meaning:NNT #1 \q_nil
      { \__cmd_cant_copy:nwn { invalid-end } }
  }
\cs_set_protected:Npn \__cmd_tmp:w #1
  {
    \cs_new:Npn \__cmd_check_end:n ##1
      {
        \exp_after:wN \__cmd_check_end:w \tl_to_str:n {##1}
          #1 \q_mark #1 \q_stop
      }
    \cs_new:Npn \__cmd_check_end:w ##1 #1 ##2 #1 ##3 \q_stop
      { \if_meaning:w ##2 \q_mark \exp_not:N \q_nil \else: ##2 \fi: }
  }
\exp_args:No \__cmd_tmp:w { \tl_to_str:n { end } }
```

(*End of definition for* `\__cmd_copy_environment_end:nnNN` *and others.*)

Not much to do regarding latexrelease: we could remove the entries from `\@declarecommandcopylist` but it doesn't seem worth it.

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__cmd_copy:NN (part 2)}%
⟨latexrelease⟩   {Support~\NewCommandCopy~in~ltcmd}
⟨latexrelease⟩\EndIncludeInRelease
```

1.26.2 Showing the definition of a command

```
⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\__cmd_show:N}%
⟨latexrelease⟩   {Support~\ShowCommand~in~ltcmd}
```

To show the definition of a command we need more or less the same building blocks as for copying, except that instead of making a copy, we'll just print stuff to the terminal.

__cmd_show:N

This macro just branches to the proper showing command by using `\__cmd_cmd_type_cases:NnnnnnF`. The showing command takes the command to be shown as argument.

```
\cs_new_protected:Npn \__cmd_show:N #1
  {
    \use:e
      {
        \int_set:Nn \tex_escapechar:D { 92 }
        \exp_not:N \__cmd_cmd_type_cases:NnnnnnF \exp_not:N #1
          { \__cmd_show_command:N }
          { \__cmd_show_expandable:N }
          { \__cmd_show_optimized:N }
          { \__cmd_show_environment:N }
          { \__cmd_show_environment_end:N }
          { \__cmd_cant_copy:nwn { non-ltcmd } }
            \exp_not:N #1
        \exp_not:N \__cmd_break_point:n { \cs_to_str:N #1 }
        \int_set:Nn \tex_escapechar:D { \int_use:N \tex_escapechar:D }
      }
  }
```

(*End of definition for* `\__cmd_show:N`.)

__cmd_show_command:N
__cmd_show_command:NnNNwN
__cmd_show_expandable:N
__cmd_show_expandable:NnNNNNnN
__cmd_show_optimized:N
__cmd_show_command_aux:NnNNn
__cmd_show_environment:N
__cmd_show:e

These commands just expand the command once to reveal its innards, then pass the type of command, the control sequence, the signature, and the code macro to `\__cmd_show_command_aux:NnNNn`.

```
\cs_new_protected:Npn \__cmd_show_command:N #1
  { \exp_after:wN \__cmd_show_command:NnNNwN #1 \q__cmd #1 }
\cs_new_protected:Npn \__cmd_show_command:NnNNwN #1 #2 #3 #4 #5 \q__cmd #6
  {
    \__cmd_show_command_aux:NnNNn \tl_show:e
      { document~command } #6 #4 {#2}
  }
\cs_new_protected:Npn \__cmd_show_expandable:N #1
  { \exp_after:wN \__cmd_show_expandable:NnNNNNnN #1 #1 }
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__cmd_show:N}%
⟨latexrelease⟩   {Support~\ShowCommand~in~ltcmd}
⟨latexrelease⟩\EndIncludeInRelease
```

There's one variant: a command begins with `\__cmd_start_expandable:nNNNNn` may still be un-expandable/protected if it's defined by `\NewDocumentCommand` and friends, with empty or only m-type arguments.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__cmd_show_expandable:NnNNNNnN}%
⟨latexrelease⟩   {Distinguish~non-expandable~document~commands}
\cs_new_protected:Npn \__cmd_show_expandable:NnNNNNnN #1 #2 #3 #4 #5 #6 #7 #8
  {
    \exp_args:NNe \__cmd_show_command_aux:NnNNn \tl_show:e
```

```
      { \token_if_protected_macro:NF #8 { expandable~ } document~command }
      #8 #5 {#2}
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\__cmd_show_expandable:NnNNNNnN}%
⟨latexrelease⟩  {Support~\ShowCommand~in~ltcmd}
⟨latexrelease⟩\cs_new_protected:Npn \__cmd_show_expandable:NnNNNNnN #1 #2 #3 #4 #5 #6 #7 #8
⟨latexrelease⟩  {
⟨latexrelease⟩    \__cmd_show_command_aux:NnNNn \tl_show:x
⟨latexrelease⟩      { expandable~document~command } #8 #5 {#2}
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__cmd_show_expandable:NnNNNNnN}%
⟨latexrelease⟩  {Support~\ShowCommand~in~ltcmd}
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\__cmd_show:N (part 2)}%
⟨latexrelease⟩  {Support~\ShowCommand~in~ltcmd}
```

Now just print everything in the required format. The auxiliary `\__cmd_split_signature:n` stores a ready-to-print token list in `\l__cmd_tmpa_tl`, so we ust use that here:

```
\cs_new_protected:Npn \__cmd_show_command_aux:NnNNn #1 #2 #3 #4 #5
  {
    \__cmd_split_signature:n {#5}
    #1
      {
        \token_to_str:N #3 = #2: \iow_newline:
        \tl_use:N \l__cmd_tmpa_tl
        -> \cs_replacement_spec:N #4
      }
  }
```

Optimized functions need things done a bit differently as we need to reconstruct the argument spec.

```
\cs_new_protected:Npn \__cmd_show_optimized:N #1
  {
    \exp_args:Nc \__cmd_show_optimized:NN
      { \cs_to_str:N #1 \c_space_tl code }
      #1
  }
\cs_new_protected:Npn \__cmd_show_optimized:NN #1#2
  {
    \cs_set:Npe \__cmd_show_optimized_aux:N ##1
      {
        \c_space_tl \c_space_tl \c_hash_str ##1 :
        \bool_lazy_or:nnT
          { \token_if_long_macro_p:N #1 }
          { \token_if_protected_long_macro_p:N #1 }
            { + } m
         \iow_newline:
      }
    \tl_show:e
      {
        \token_to_str:N #2 =
```

```
          \bool_lazy_or:nnF
            { \token_if_protected_macro_p:N #1 }
            { \token_if_protected_long_macro_p:N #1 }
              { expandable ~ } document~command:
        \iow_newline:
        \int_step_function:nN
          {
            \int_div_truncate:nn
              { \tl_count:e { \cs_parameter_spec:N #1 } }
              { 2 }
          }
          \__cmd_show_optimized_aux:N
        ->
        \cs_replacement_spec:N #1
      }
  }
```

We can reuse most of the above to show an environment, except that we need to ensure that the proper `\environment ...` are passed to `\__cmd_show_command_aux:NnNNn`. Additionally, when `\ShowCommand\foo` is used (if `foo` is an environment), we show `\endfoo` as well, and when `\ShowCommand\endfoo` is used, change that to `\ShowCommand\foo` and do the same.

```
\cs_new_protected:Npn \__cmd_show_environment:N #1
  {
    \exp_after:wN \__cmd_show_environment:Nnnw #1 \q__cmd
    \tl_show:e
      {
        \token_to_str:N \end { \cs_to_str:N #1 } : \iow_newline:
        -> \exp_args:Nc \cs_replacement_spec:N
          { environment~ \cs_to_str:N #1 ~end~aux~ }
      }
  }
\cs_new_protected:Npn \__cmd_show_environment:Nnnw #1 #2 #3 #4 \q__cmd
  {
    \use:e
      {
        \__cmd_show_command_aux:NnNNn \__cmd_show:e { document~environment }
          { \exp_not:N \begin {#3} }
          \exp_not:c { environment~ #3 ~ code }
          {#2}
      }
  }
\cs_if_exist:NTF \iow_show:e
  {
    \cs_new_protected:Npn \__cmd_show:e #1
      { \iow_show:e { > ~ #1 . \iow_newline: } }
  }
  {
    \cs_new_protected:Npn \__cmd_show:e #1
      { \iow_term:e { > ~ #1 . \iow_newline: } }
  }
\cs_new_protected:Npn \__cmd_show_environment_end:N #1
  {
    \exp_args:NNe \__cmd_check_end:Nn \l__cmd_tmpa_tl { \cs_to_str:N #1 }
```

```
    \exp_args:Nc \__cmd_show_environment:N { \l__cmd_tmpa_tl }
  }
```

And, of course, add `\__kernel_cmd_if_xparse:NTF` and `\__cmd_show:N` to `\@showcommandlisthoo` and to `\@showenvironmentlisthook` (`\__cmd_show:N` takes care of the environment case as well, so both entries are identical):

```
\tl_gput_right:Nn \@showcommandlisthook
  { { \__kernel_cmd_if_xparse:NTF \__cmd_show:N } }
\tl_gput_right:Nn \@showenvironmentlisthook
  { { \__kernel_cmd_if_xparse:NTF \__cmd_show:N } }
```

(*End of definition for* `\__cmd_show_command:N` *and others.*)

`\__cmd_split_signature:n`

Now we'll try a least-effort adventure into splitting the symbolic user-provided signature for a command into individual parameters for pretty-printing. A counter is used to keep track of the current argument number, and two token lists are used: `\l__cmd_tmpa_tl` holds the final token list to be printed, and `\l__cmd_tmpb_tl` holds just the current item, so that we can make changes to an individual item without having to dissect the whole thing (this is used for `e`- and `E`-types).

```
\cs_new_protected:Npn \__cmd_split_signature:n #1
  {
    \int_set:Nn \l__cmd_current_arg_int { 1 }
    \tl_clear:N \l__cmd_tmpa_tl
    \tl_clear:N \l__cmd_tmpb_tl
    \__cmd_split_signature_loop:Nw #1 \q_recursion_tail \q_recursion_stop
  }
```

`\__cmd_split_signature_loop:Nw`

This is the main chunk of the loop: it starts an item with `\__cmd_split_start_item:` (this adds indentation and the argument number to `\l__cmd_tmpb_tl`), then checks if a special token list `\c__cmd_show_type_`⟨*type*⟩`_tl` exists. If it doesn't, the current argument is a "simple" type which needs no extra processing. Otherwise, call a specific function depending on the value of said token list.

```
\cs_new_protected:Npn \__cmd_split_signature_loop:Nw #1
  {
    \quark_if_recursion_tail_stop:N #1
    \tl_if_empty:NT \l__cmd_tmpb_tl { \__cmd_split_start_item: }
    \tl_if_exist:cTF { c__cmd_show_type_#1_tl }
      {
        \use:c
          {
            __cmd_show_
            \if_case:w \tl_use:c { c__cmd_show_type_#1_tl } \exp_stop_f:
            delim \or: delims \or: delims_opt \or: opt \or:
            e \or: E \or: prefix \or: processor \fi: :Nw
          } #1
      }
      { \__cmd_split_end_item:n {#1} \__cmd_split_signature_loop:Nw }
  }
```

The token lists `\c__cmd_show_type_`⟨*type*⟩`_tl` exist for nontrivial (for printing) ⟨*types*⟩ that require special parsing (like delimiters or optional arguments). Values from 0 to 7 are assigned to each type:

`\c__cmd_show_type_t_tl`
`\c__cmd_show_type_r_tl`
`\c__cmd_show_type_d_tl`
`\c__cmd_show_type_R_tl`
`\c__cmd_show_type_D_tl`
`\c__cmd_show_type_O_tl`
`\c__cmd_show_type_e_tl`
`\c__cmd_show_type_E_tl`
`\c__cmd_show_type_+_tl`
`\c__cmd_show_type_!_tl`
`\c__cmd_show_type_>_tl`

1. a single delimiter token;

2. two delimiter tokens;
3. two delimiter tokens plus a default value;
4. a default value;
5. a list of embellishments (exclusive for `e`-type);
6. embellishments plus defaults (exclusive for `E`-type);
7. simple prefixes;
8. prefixes with arguments (argument processors);

```
\cs_set_protected:Npn \__cmd_tmp:w #1 #2
  {
    \quark_if_nil:nF {#1}
      { \tl_const:cn { c__cmd_show_type_#1_tl } {#2} \__cmd_tmp:w }
  }
\__cmd_tmp:w t0 r1 d1 R2 D2 O3 e4 E5 +6 !6 >7 =7 \q_nil \q_nil
```

Now, based on each type we know how to act. In most cases it is just a matter of feeding in the grabbed arguments and resuming the loop. The embellishments require a bit more attention: the `e`-type loops through the list of embellishments and adds each to the token list as a separate argument. The `E`-type does more or less the same, but uses `\__cmd_tl_mapthread_function:nnN` to map over two lists simultaneously, adding each token and default to the token list for printing.

```
\cs_new_protected:Npn \__cmd_show_delim:Nw #1 #2
  { \__cmd_split_end_item:n { #1 #2 } \__cmd_split_signature_loop:Nw }
\cs_new_protected:Npn \__cmd_show_delims:Nw #1 #2 #3
  { \__cmd_split_end_item:n { #1 #2 #3 } \__cmd_split_signature_loop:Nw }
\cs_new_protected:Npn \__cmd_show_delims_opt:Nw #1 #2 #3 #4
  { \__cmd_split_end_item:n { #1 #2 #3 {#4} } \__cmd_split_signature_loop:Nw }
\cs_new_protected:Npn \__cmd_show_opt:Nw #1 #2
  { \__cmd_split_end_item:n { #1 {#2} } \__cmd_split_signature_loop:Nw }
\cs_new_protected:Npn \__cmd_show_e:Nw #1 #2
  {
    \tl_map_inline:nn {#2}
      {
        \__cmd_split_start_item:
        \__cmd_split_end_item:n { #1 ##1 }
      }
    \__cmd_split_signature_loop:Nw
  }
\cs_set_protected:Npn \__cmd_tmp:w #1
  {
    \cs_new_protected:Npn \__cmd_show_E:Nw ##1 ##2 ##3
      {
        \cs_set_protected:Npn \__cmd_tmp:w ####1 ####2
          {
            \__cmd_split_start_item:
            \__cmd_split_end_item:n { ##1 ####1 {####2} }
          }
        \__cmd_tl_mapthread_function:nnN {##2}
          { ##3 {#1} {#1} {#1} {#1} {#1} {#1} {#1} {#1} {#1} } \__cmd_tmp:w
```

__cmd_show_delim:Nw
__cmd_show_delims:Nw
__cmd_show_delims_opt:Nw
__cmd_show_opt:Nw
__cmd_show_e:Nw
__cmd_show_E:Nw
__cmd_show_prefix:Nw
__cmd_show_processor:Nw

```
        \__cmd_split_signature_loop:Nw
      }
  }
\__cmd_tmp:w \NoValue
```

Minor wrinkle with the prefixes: they use `\__cmd_split_add_item:n` instead of `\__cmd_split_end_item:n` (`add` *vs.* `end`) because they are followed by an argument, so they can't end the item.

```
\cs_new_protected:Npn \__cmd_show_prefix:Nw #1
  { \__cmd_split_add_item:n {#1} \__cmd_split_signature_loop:Nw }
\cs_new_protected:Npn \__cmd_show_processor:Nw #1 #2
  { \__cmd_split_add_item:n { #1 {#2} } \__cmd_split_signature_loop:Nw }
```

And now the auxiliaries that store the strings to be printed. `\__cmd_split_start_item:` starts an item from scratch, `\__cmd_split_add_item:n` adds tokens to an item without adding a newline, and `\__cmd_split_end_item:n` adds tokens, terminates the item with a newline, and steps the argument count.

```
\cs_new_protected:Npn \__cmd_split_start_item:
  {
    \tl_set:Ne \l__cmd_tmpb_tl
      { ~ \c_space_tl \c_hash_str \int_use:N \l__cmd_current_arg_int : }
  }
\cs_new_protected:Npn \__cmd_split_add_item:n #1
  { \tl_put_right:Ne \l__cmd_tmpb_tl { \tl_to_str:n {#1} } }
\cs_new_protected:Npn \__cmd_split_end_item:n #1
  {
    \tl_put_right:Ne \l__cmd_tmpa_tl
      { \l__cmd_tmpb_tl \tl_to_str:n {#1} \iow_newline: }
    \tl_clear:N \l__cmd_tmpb_tl
    \int_incr:N \l__cmd_current_arg_int
  }
```

(*End of definition for* `\__cmd_split_signature:n` *and others.*)

Not much to do regarding latexrelease: we could remove the entries from `\@showcommandlisthook`, but it doesn't seem worth it.

`\__cmd_split_start_item:`
`\__cmd_split_add_item:n`
`\__cmd_split_end_item:n`

```
⟨latexrelease⟩\EndIncludeInRelease
%
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__cmd_show:N (part 2)}%
⟨latexrelease⟩   {Support~\ShowCommand~in~ltcmd}
⟨latexrelease⟩\EndIncludeInRelease
```

1.27 Grabbing arguments

All of the grabbers follow the same basic pattern. The initial function stores in `\l__cmd_signature_tl` the code to grab further arguments, defines (the function in) `\l__cmd_fn_tl` that will grab the argument, and calls it.

Defining `\l__cmd_fn_tl` means determining whether to use `\cs_set:Npn` or `\cs_set_nopar:Npn`, and for optional arguments whether to skip spaces. Once the argument is found, `\l__cmd_fn_tl` calls `\__cmd_add_arg:n`, responsible for calling processors and grabbing further arguments.

__cmd_grab_b:w
__cmd_grab_b_long:w
__cmd_grab_b_obey_spaces:w
__cmd_grab_b_long_obey_spaces:w
__cmd_grab_b_aux:NNw
__cmd_grab_b_end:Nw

This uses the well-tested code of D-type arguments, skipping the peeking step because the b-type argument is always present, and adding a cleanup stage at the end by hijacking the signature. The clean-up consists of properly dealing with `\l__cmd_args_tl` and also putting back the `\end` that served as an end-delimiter: this `\end` receives the environment name as its argument and is run normally. The D-type code stores the argument found (body of the environment) as a brace group in `\l__cmd_args_tl` and depending on the presence of a prefix `!` we trim spaces or not before adding this braced argument into the saved `\l__cmd_args_tl`. The strange `\begin␣` control sequence is there for display purposes only: it has to look like `\begin` in the terminal but not to delimited arguments.

```
\cs_new_protected:Npn \__cmd_grab_b:w
  { \__cmd_grab_b_aux:NNw \cs_set_protected_nopar:Npn \tl_trim_spaces:n }
\cs_new_protected:Npn \__cmd_grab_b_long:w
  { \__cmd_grab_b_aux:NNw \cs_set_protected:Npn \tl_trim_spaces:n }
\cs_new_protected:Npn \__cmd_grab_b_obey_spaces:w
  { \__cmd_grab_b_aux:NNw \cs_set_protected_nopar:Npn \exp_not:n }
\cs_new_protected:Npn \__cmd_grab_b_long_obey_spaces:w
  { \__cmd_grab_b_aux:NNw \cs_set_protected:Npn \exp_not:n }
\cs_new_protected:Npn \__cmd_grab_b_aux:NNw #1#2#3 \__cmd_run_code:
  {
    \__cmd_grab_D_aux:NNnNN \begin \end {#3} #1 \use_ii:nn
    \tl_put_left:Nn \l__cmd_signature_tl { \__cmd_grab_b_end:Nw #2 }
    \tl_set_eq:NN \l__cmd_saved_args_tl \l__cmd_args_tl
    \tl_clear:N \l__cmd_args_tl
    \exp_args:Nc \l__cmd_fn_tl { begin ~ }
  }
\cs_new_protected:Npn \__cmd_grab_b_end:Nw #1#2 \__cmd_run_code:
  {
    \tl_set:Ne \l__cmd_args_tl
      {
        \exp_not:V \l__cmd_saved_args_tl
        { \exp_after:wN #1 \l__cmd_args_tl }
      }
    #2
    \__cmd_run_code:
    \end
  }
```

(*End of definition for* `\__cmd_grab_b:w` *and others.*)

__cmd_grab_c:w
__cmd_grab_c_obey_spaces:w
__cmd_grab_c_start:n
__cmd_grab_c_first:w
__cmd_grab_c_loop:w

Collecting an environment body verbatim shares some ideas with the v-type grabber, and others with the standard `filecontents` environment. The start is to set the end-of-line to a predictable value and to deactivate the specials.

```
\cs_new_protected:Npn \__cmd_grab_c:w #1 \__cmd_run_code:
  {
    \bool_set_false:N \l__cmd_obey_spaces_bool
    \__cmd_grab_c_start:n {#1}
  }
\cs_new_protected:Npn \__cmd_grab_c_obey_spaces:w #1 \__cmd_run_code:
  {
    \bool_set_true:N \l__cmd_obey_spaces_bool
    \__cmd_grab_c_start:n {#1}
  }
\cs_new_protected:Npn \__cmd_grab_c_start:n #1
```

```
  {
    \tl_set:Nn \l__cmd_signature_tl {#1}
    \group_begin:
      \tl_clear:N \l__cmd_v_arg_tl
      \tex_escapechar:D = 92 \scan_stop:
      \tex_endlinechar:D = `\^^M \scan_stop:
      \cs_set_eq:NN \do \char_set_catcode_other:N
      \dospecials
      \char_set_catcode_other:n { `\^^M }
      \__cmd_grab_c_first:w
  }
```

Notice here and below that we cannot use `\token_to_str:N \end` as that would have the wrong category codes for the letters.

```
\group_begin:
  \char_set_catcode_other:N \^^M %
  \cs_new_protected:Npn \__cmd_grab_c_first:w #1 ^^M %
    { %
      \tl_if_blank:nTF {#1} %
        { %
          \__cmd_grab_c_loop:w #1 ^^M %
        } %
        { %
          \msg_warning:nnee { cmd } { chars-dropped-first-line }  %
            { \exp_not:n {#1} }  %
            { \exp_not:V \@currenvir } %
           \__cmd_grab_c_loop:w ^^M %
        } %
    } %
  \cs_new_protected:Npe \__cmd_grab_c_loop:w #1 ^^M %
    { %
      \exp_not:N \__cmd_grab_c_auxi:w #1 %
        \c_backslash_str end %
        \scan_stop: %
    } %
\group_end:
```

(*End of definition for* `\__cmd_grab_c:w` *and others.*)

`\__cmd_grab_c_auxi:w`
`\__cmd_grab_c_auxii:w`
`\__cmd_grab_c_auxiii:N`
`\__cmd_grab_c_auxiv:`
`\__cmd_grab_c_auxv:`
`\__cmd_grab_c_auxvi:N`
`\__cmd_grab_c_auxvii:`
`\__cmd_grab_c_auxviii:`

We need to see if the current line contains `\end` followed by the name of the current environment. To do that and allow for spaces, we have to work stepwise. First, establish if there is an `\end` at all: remember that here we are dealing with "other" tokens. Whether these is an `\end` or not, the tokens *before* it form part of the line.

```
\use:e
  {
    \cs_new_protected:Npn \exp_not:N \__cmd_grab_c_auxi:w
      #1 \c_backslash_str end #2 \scan_stop:
  }
  {
    \tl_put_right:Nn \l__cmd_v_arg_tl {#1}
    \tl_if_empty:nTF {#2}
      {
        \tl_put_right:Nn \l__cmd_v_arg_tl { \obeyedline }
        \__cmd_grab_c_loop:w
```

```
      }
      { \__cmd_grab_c_auxii:w #2 \scan_stop: }
  }
```

There is an `\end`, so we now remove the trailing marker we needed to do the test. This is stripped off, then we need to examine the rest of the line one token at a time: see `verbatim.dtx` for the inspiration. Notice that we use `^^M` here as the end marker: this allows looping to look for multiple `\end` entries in the line.

```
\group_begin:
  \char_set_catcode_other:N \^^M %
  \use:e %
    { %
      \cs_new_protected:Npe \exp_not:N \__cmd_grab_c_auxii:w %
        #1 \c_backslash_str end \scan_stop: %
    } %
    { %
      \tl_set:Nn \exp_not:N \l__cmd_tmpa_tl
        { \c_backslash_str end } %
      \exp_not:N \__cmd_grab_c_auxiii:N #1 ^^M %
    } %
```

Within the line, we need to collect up the tokens: if we do not find the end-of-environment argument, we will need those to reinsert. There are three special cases here: `^^M` (end of line: tidy up and back to the main loop), ␣ (possibly skip over) and `{` (start the inner loop). Anything else means we move back to examine the rest of the line for any more `\end` entries.

```
  \cs_new_protected:Npn \__cmd_grab_c_auxiii:N #1 %
    { %
      \token_case_charcode:NnF #1 %
        { %
          ^^M %
            { \__cmd_grab_c_auxiv: } %
          \c_space_token %
            { %
             \tl_put_right:Nn \l__cmd_tmpa_tl {#1} %
             \__cmd_grab_c_auxiii:N %
            } %
          \c_group_begin_token %
            { %
              \tl_set:Nn \l__cmd_tmpb_tl {#1} %
              \__cmd_grab_c_auxvi:N %
            } %
        } %
        { %
          \tl_put_right:Nn \l__cmd_tmpa_tl {#1} %
          \__cmd_grab_c_auxv: %
        } %
    } %
\group_end:
\cs_new_protected:Npn \__cmd_grab_c_auxiv:
  {
    \tl_put_right:Ne \l__cmd_v_arg_tl
      {
        \exp_not:V \l__cmd_tmpa_tl
```

```
        \exp_not:N \obeyedline
      }
    \__cmd_grab_c_loop:w
  }
\cs_new_protected:Npn \__cmd_grab_c_auxv:
  {
    \tl_put_right:NV \l__cmd_v_arg_tl \l__cmd_tmpa_tl
    \__cmd_grab_c_loop:w
  }
```

In the inner loop, we again have only a few special cases. First, we could again have `^^M`, in which case we tidy up using a common auxiliary. Second, we check for the escape char: this cannot happen inside the end of an environment and means we loop, re-inserting the token. Finally, we have `}`, where we need to move on to check what has been collected. Otherwise, collect up and loop. Notice here that the inner loop needs to collect tokens separately: this leaves any spaces after `\end` in `\l__cmd_tmpa_tl`, so we can test `\l__cmd_tmpb_tl` directly.

```
\group_begin:
  \char_set_catcode_other:N \^^M %
  \cs_new_protected:Npe \__cmd_grab_c_auxvi:N #1 %
    { %
      \exp_not:N \token_case_charcode:NnF #1 %
        { %
          ^^M %
            {
              \exp_not:N \__cmd_grab_c_auxvii: %
              \exp_not:N \__cmd_grab_c_auxiv: %
            }%
          \c_backslash_str %
            { %
              \exp_not:N \__cmd_grab_c_auxvii: %
              \exp_not:N \__cmd_grab_c_auxv: #1
            } %
         \c_group_end_token %
           { %
             \tl_put_right:Nn \exp_not:N \l__cmd_tmpb_tl {#1} %
             \exp_not:N \__cmd_grab_c_auxviii: %
           } %
        } %
        { %
          \tl_put_right:Nn \exp_not:N \l__cmd_tmpb_tl {#1} %
          \exp_not:N \__cmd_grab_c_auxvi:N %
        } %
    } %
\group_end: %
\cs_new_protected:Npn \__cmd_grab_c_auxvii:
  { \tl_put_right:NV \l__cmd_tmpa_tl \l__cmd_tmpb_tl }
\cs_new_protected:Npn \__cmd_grab_c_auxviii:
  {
    \str_if_eq:eeTF { \exp_not:V \l__cmd_tmpb_tl } { { \@currenvir } }
      { \__cmd_grab_c_end:w }
      {
        \__cmd_grab_c_auxvii:
        \__cmd_grab_c_auxv:
```

```
    }
  }
```

(*End of definition for* `\__cmd_grab_c_auxi:w` *and others.*)

`\__cmd_grab_c_end:w`
`\__cmd_grab_c_end:n`
`\__cmd_grab_c_end_auxi:w`
`\__cmd_grab_c_end_auxii:w`
`\__cmd_grab_c_end_auxiii:w`

To end the collection, we clean up the last line: once again we need to find `^^M`. Once that is done, we can warn if there is anything left behind.

```
\group_begin:
  \char_set_catcode_other:N \^^M %
  \cs_new_protected:Npn \__cmd_grab_c_end:w #1 ^^M %
    { %
      \tl_if_blank:nF {#1} %
        { %
          \msg_warning:nnee { cmd } { chars-dropped-last-line }  %
            { \exp_not:n {#1} }  %
            { \exp_not:V \@currenvir } %
        } %
      \exp_args:NNNo \group_end: %
      \tl_set:Nn \l__cmd_v_arg_tl { \l__cmd_v_arg_tl } %
      \__cmd_add_arg:e %
        { %
          \bool_if:NTF \l__cmd_obey_spaces_bool %
            { \exp_not:V } %
            { \exp_args:NV \__cmd_grab_c_end:n } %
           \l__cmd_v_arg_tl %
        } %
      \exp_args:NV \end \@currenvir %
    } %
\group_end: %
```

Look for line markers at each end and tidy up if required.

```
\cs_new:Npn \__cmd_grab_c_end:n #1
  {
    \__cmd_grab_c_end_auxi:w \q_nil #1 \q_nil
      \obeyedline \obeyedline \q_nil \q_stop
  }
\cs_new:Npn \__cmd_grab_c_end_auxi:w #1 \q_nil \obeyedline
  { \__cmd_grab_c_end_auxii:w #1 \q_nil }
\cs_new:Npn \__cmd_grab_c_end_auxii:w \q_nil #1 \obeyedline \q_nil
  { \__cmd_grab_c_end_auxiii:w #1 \q_nil }
\cs_new:Npn \__cmd_grab_c_end_auxiii:w #1 \q_nil #2 \q_stop
  { \exp_not:n {#1} }
```

(*End of definition for* `\__cmd_grab_c_end:w` *and others.*)

`\__cmd_grab_D:w`
`\__cmd_grab_D_no_strip:w`
`\__cmd_grab_D_obey_spaces:w`
`\__cmd_grab_D_obey_spaces_no_strip:w`
`\__cmd_grab_D_long:w`
`\__cmd_grab_D_long_no_strip:w`
`\__cmd_grab_D_long_obey_spaces:w`
`\__cmd_grab_D_long_obey_spaces_no_strip:w`

The generic delimited argument grabber. The auxiliary function does a peek test before calling `\__cmd_grab_D_call:Nw`, so that the optional nature of the argument works as expected.

```
\tl_map_inline:nn { { } { _long } }
  {
    \tl_map_inline:nn { { } { _obey_spaces } }
      {
        \tl_map_inline:nn { { } { _no_strip } }
          {
            \tl_map_inline:nn { { } { _verb_safe } }
```

```
            {
              \cs_new_protected:cpe { __cmd_grab_D #1 ##1 ####1 ########1 :w }
                ################1################2################3
                \__cmd_run_code:
                {
                  \exp_not:N \__cmd_grab_D_aux:NNnNNNN
                    ################1 ################2 {################3}
                    \str_if_eq:nnTF {#1} { _long }
                      \cs_set_protected:Npn
                      \cs_set_protected_nopar:Npn
                    \str_if_eq:nnTF {##1} { _obey_spaces }
                      { \exp_not:N \__cmd_peek_meaning_remove:NTF }
                      { \exp_not:N \__cmd_peek_nonspace_remove:NTF }
                    \str_if_eq:nnTF {####1} { _no_strip }
                      { \exp_not:N \use_none:n }
                      { \exp_not:N \use_ii:nn }
                    \str_if_eq:nnTF {########1} { _verb_safe }
                      { \exp_not:N \use:n }
                      { \exp_not:N \use_none:n }
                }
            }
        }
    }
}
```

This is a bit complicated. The idea is that, in order to check for nested optional argument tokens (`[[...]]` and so on) the argument needs to be grabbed without removing any braces at all. If this is not done, then cases like `[{[}]` fail. So after testing for an optional argument, it is collected piece-wise. Inserting a quark prevents loss of braces, and there is then a test to see if there are nested delimiters to handle.

```
\group_begin:
  \char_set_catcode_other:N \^^M
  \cs_new_protected:Npn \__cmd_grab_D_aux:NNnNNNN #1#2#3#4#5#6#7
    {
      \__cmd_grab_D_aux:NNnNN #1#2 {#3} #4 #6
      #7
        {
          \group_begin:
            \__cmd_grab_D_verb_safe:NN #1#5
        }
      #5 #1
        {
          #7 { \group_end: }
          \__cmd_grab_D_call:Nw #1
        }
        {
          #7 { \group_end: }
          \__cmd_add_arg:n { \NoValue }
        }
    }
\group_end:
```

__cmd_grab_D_aux:NNnNNNN
__cmd_grab_D_verb_safe:NN
__cmd_grab_D_aux:NNnNN

The only awkwardness here is the need to preserve the catcode of the search token: this is done low-level for performance reasons. Only values that are realistic are included. As

this does not cover spaces when they are skipped that has to be covered separately.

```
\cs_new_protected:Npn \__cmd_grab_D_verb_safe:NN #1#2
  {
    \cs_set_eq:NN \do \char_set_catcode_other:N
    \dospecials
    \char_set_catcode_other:N \^^M
    \token_if_eq_meaning:NNT #2 \__cmd_peek_nonspace_remove:NTF
      { \char_set_catcode_space:n { `\  } }
    \use:c
      {
        char_set_catcode_
        \if_catcode:w \c_math_toggle_token #1 math_toggle \else:
        \if_catcode:w ^ #1 math_superscript \else:
        \if_catcode:w \c_math_subscript_token #1 math_subscript \else:
        \if_catcode:w A #1 letter \else:
        other \fi: \fi: \fi: \fi:
        :N
      }
        #1
  }
```

Inside the "standard" grabber, there is a test to see if the grabbed argument is entirely enclosed by braces. There are a couple of extra factors to allow for: the argument might be entirely empty, and spaces at the start and end of the input must be retained around a brace group. Also notice that a *blank* argument might still contain spaces. To allow for suppression of brace stripping, the business end is passed here as `#5`.

```
\cs_new_protected:Npn \__cmd_grab_D_aux:NNnNN #1#2#3#4#5
  {
    \tl_set:Nn \l__cmd_signature_tl {#3}
    \exp_after:wN #4 \l__cmd_fn_tl ##1 #2
      {
        \tl_if_in:nnTF {##1} {#1}
          { \__cmd_grab_D_nested:NNnN #1 #2 {##1} #4 }
          {
            \tl_if_blank:oTF { \use_none:n ##1 }
              { \__cmd_add_arg:o { \use_none:n ##1 } }
              {
                \str_if_eq:eeTF
                  { \exp_not:o { \use_none:n ##1 } }
                  { { \exp_not:o { \use_ii:nnn ##1 \q_nil } } }
                  { \__cmd_add_arg:o { #5 ##1 } }
                  { \__cmd_add_arg:o { \use_none:n ##1 } }
              }
          }
      }
  }
```

(*End of definition for* `\__cmd_grab_D:w` *and others.*)

`\__cmd_grab_D_nested:NNnN`
`\__cmd_grab_D_nested:w`
`\l__cmd_nesting_a_tl`
`\l__cmd_nesting_b_tl`
`\q__cmd`

Catching nested optional arguments means more work. The aim here is to collect up each pair of optional tokens without TEX helping out, and without counting anything. The code above will already have removed the leading opening token and a closing token, but the wrong one. The aim is then to work through the material grabbed so far and divide it up on each opening token, grabbing a closing token to match (thus working in

pairs). Once there are no opening tokens, then there is a second check to see if there are any opening tokens in the second part of the argument (for things like `[][]`). Once everything has been found, the entire collected material is added to the output as a single argument. The only tricky part here is ensuring that any grabbing function that might run away is named after the function currently being parsed and not after xparse. That leads to some rather complex nesting! There is also a need to prevent the loss of any braces, hence the insertion and removal of quarks along the way.

```
\tl_new:N \l__cmd_nesting_a_tl
\tl_new:N \l__cmd_nesting_b_tl
\quark_new:N \q__cmd
\cs_new_protected:Npn \__cmd_grab_D_nested:NNnN #1#2#3#4
  {
    \tl_clear:N \l__cmd_nesting_a_tl
    \tl_clear:N \l__cmd_nesting_b_tl
    \exp_after:wN #4 \l__cmd_fn_tl ##1 #1 ##2 \q__cmd ##3 #2
      {
        \tl_put_right:No \l__cmd_nesting_a_tl { \use_none:n ##1 #1 }
        \tl_put_right:No \l__cmd_nesting_b_tl { \use_i:nn #2 ##3 }
        \tl_if_in:nnTF {##2} {#1}
          {
            \l__cmd_fn_tl
              \q_nil ##2 \q__cmd \ERROR
          }
          {
            \tl_put_right:Ne \l__cmd_nesting_a_tl
              { \__cmd_grab_D_nested:w \q_nil ##2 \q_stop }
            \tl_if_in:NnTF \l__cmd_nesting_b_tl {#1}
              {
                \tl_set_eq:NN \l__cmd_tmpa_tl \l__cmd_nesting_b_tl
                \tl_clear:N \l__cmd_nesting_b_tl
                \exp_after:wN \l__cmd_fn_tl \exp_after:wN
                  \q_nil \l__cmd_tmpa_tl \q_nil \q__cmd \ERROR
              }
              {
                \tl_put_right:No \l__cmd_nesting_a_tl
                  \l__cmd_nesting_b_tl
                \__cmd_add_arg:V \l__cmd_nesting_a_tl
              }
          }
      }
    \l__cmd_fn_tl #3 \q_nil \q__cmd \ERROR
  }
\cs_new:Npn \__cmd_grab_D_nested:w #1 \q_nil \q_stop
  { \exp_not:o { \use_none:n #1 } }
```

(*End of definition for* `\__cmd_grab_D_nested:NNnN` *and others.*)

`\__cmd_grab_D_call:Nw` For `D` and `R`-type arguments, to avoid losing any braces, a token needs to be inserted before the argument to be grabbed. If the argument runs away because the closing token is missing then this inserted token shows up in the terminal. Ideally, `#1` would therefore be used directly, but that is no good as it will mess up the rest of the grabber. Instead, a copy of `#1` with an altered category code is used, as this will look right in the terminal but will not mess up the grabber. The only issue then is that the category code of `#1`

is unknown. So there is a quick test to ensure that the inserted token can never be matched by the grabber. (This assumes that the open and close delimiters are not the same character with different category codes, but that really should not happen in any sensible document-level syntax.) An exception is when `#1` is a control sequence token, in which case the character-token treatment is no good because if hit with `\token_to_str:N` it would add sputios tokens to the argument. In this case a different branch is taken. The token inserted is then the same ⟨*csname*⟩ as `#1`, but with a space appended, so that the grabber don't see it as another of the same delimiter.

```
\cs_new_protected_nopar:Npn \__cmd_grab_D_call:Nw #1
  {
    \token_if_eq_catcode:NNTF + #1
      {
        \exp_after:wN \exp_after:wN \exp_after:wN
          \l__cmd_fn_tl \char_generate:nn { `#1 } { 11 }
      }
      {
        \__cmd_token_if_cs:NTF #1
          {
            \exp_after:wN \l__cmd_fn_tl
            \cs:w \cs_to_str:N #1 ~ \cs_end:
          }
          {
            \exp_after:wN \l__cmd_fn_tl
            \token_to_str:N #1
          }
      }
  }
```

(*End of definition for* `\__cmd_grab_D_call:Nw`.)

`\__cmd_grab_E:w`
`\__cmd_grab_E_long:w`
`\__cmd_grab_E_obey_spaces:w`
`\__cmd_grab_E_long_obey_spaces:w`
`\__cmd_grab_E:nnNN`
`\__cmd_grab_E_loop:NnN`
`\__cmd_grab_E_finalize:`

Everything here needs to point to a loop.

```
\cs_new_protected:Npn \__cmd_grab_E:w #1#2 \__cmd_run_code:
  {
    \__cmd_grab_E:nnNN {#1} {#2}
      \cs_set_protected_nopar:Npn
      \__cmd_peek_nonspace_remove:NTF
  }
\cs_new_protected:Npn \__cmd_grab_E_long:w #1#2 \__cmd_run_code:
  {
    \__cmd_grab_E:nnNN {#1} {#2}
      \cs_set_protected:Npn
      \__cmd_peek_nonspace_remove:NTF
  }
\cs_new_protected:Npn \__cmd_grab_E_obey_spaces:w #1#2 \__cmd_run_code:
  {
    \__cmd_grab_E:nnNN {#1} {#2}
      \cs_set_protected_nopar:Npn
      \__cmd_peek_meaning_remove:NTF
  }
\cs_new_protected:Npn \__cmd_grab_E_long_obey_spaces:w #1#2 \__cmd_run_code:
  {
    \__cmd_grab_E:nnNN {#1} {#2}
      \cs_set_protected:Npn
```

```
      \__cmd_peek_meaning_remove:NTF
  }
```

A loop is needed here to allow a random ordering of keys. These are searched for one at a time, with any not found needing to be tracked: they can appear later. The grabbed values are held in a property list which is then turned into an ordered list to be passed back to the user.

```
\cs_new_protected:Npn \__cmd_grab_E:nnNN #1#2#3#4
  {
    \exp_after:wN #3 \l__cmd_fn_tl ##1##2##3
      {
        \prop_put:Nnn \l__cmd_tmp_prop {##1} {##3}
        \__cmd_grab_E_loop:NnN #4 { } ##2 \q_recursion_stop
      }
    \prop_clear:N \l__cmd_tmp_prop
    \tl_set:Nn \l__cmd_signature_tl {#2}
    \cs_set_protected:Npn \__cmd_grab_E_finalize:
      {
        \tl_map_inline:nn {#1}
          {
            \prop_get:NnNF \l__cmd_tmp_prop {####1} \l__cmd_tmpb_tl
              { \tl_set:Nn \l__cmd_tmpb_tl { \NoValue } }
            \tl_put_right:Ne \l__cmd_args_tl
              { { \exp_not:V \l__cmd_tmpb_tl } }
          }
        \l__cmd_signature_tl \__cmd_run_code:
      }
    \__cmd_grab_E_loop:NnN #4 { } #1 \q_recursion_tail \q_recursion_stop
  }
\cs_new_protected:Npn \__cmd_grab_E_loop:NnN #1#2#3#4 \q_recursion_stop
  {
    \cs_if_eq:NNTF #3 \q_recursion_tail
      { \__cmd_grab_E_finalize: }
      {
        #1 #3
          { \l__cmd_fn_tl #3 {#2#4} }
          { \__cmd_grab_E_loop:NnN #1 {#2#3} #4 \q_recursion_stop }
      }
  }
\cs_new_protected:Npn \__cmd_grab_E_finalize: { }
```

(*End of definition for* `\__cmd_grab_E:w` *and others.*)

`\__cmd_grab_m:w`
`\__cmd_grab_m_long:w`

Collecting a single mandatory argument is quite easy.

```
\cs_new_protected:Npn \__cmd_grab_m:w #1 \__cmd_run_code:
  {
    \tl_set:Nn \l__cmd_signature_tl {#1}
    \exp_after:wN \cs_set_protected_nopar:Npn \l__cmd_fn_tl ##1
      { \__cmd_add_arg:n {##1} }
    \l__cmd_fn_tl
  }
\cs_new_protected:Npn \__cmd_grab_m_long:w #1 \__cmd_run_code:
  {
    \tl_set:Nn \l__cmd_signature_tl {#1}
```

```
    \exp_after:wN \cs_set_protected:Npn \l__cmd_fn_tl ##1
      { \__cmd_add_arg:n {##1} }
    \l__cmd_fn_tl
  }
```

(*End of definition for* `\__cmd_grab_m:w` *and* `\__cmd_grab_m_long:w`.)

`\__cmd_grab_m_1:w`
`\__cmd_grab_m_2:w`
`\__cmd_grab_m_3:w`
`\__cmd_grab_m_4:w`
`\__cmd_grab_m_5:w`
`\__cmd_grab_m_6:w`
`\__cmd_grab_m_7:w`
`\__cmd_grab_m_8:w`
`\__cmd_grab_m_9:w`
`\__cmd_grab_m_aux:Nnnnnnnnn`

Grabbing 1–8 mandatory arguments is done by giving 8–1 known arguments to a 9-argument function that stores them in `\l__cmd_args_tl`. For simplicity, grabbing 9 mandatory arguments is done by grabbing 5 then 4 arguments.

```
\cs_new_protected_nopar:Npn \__cmd_grab_m_aux:Nnnnnnnnn #1#2#3#4#5#6#7#8#9
  {
    \tl_put_right:No \l__cmd_args_tl
      { #1 {#2} {#3} {#4} {#5} {#6} {#7} {#8} {#9} }
    \l__cmd_signature_tl \__cmd_run_code:
  }
\cs_new_protected:cpn { __cmd_grab_m_1:w } #1 \__cmd_run_code:
  {
    \tl_set:Nn \l__cmd_signature_tl {#1}
    \exp_after:wN \cs_set_eq:NN \l__cmd_fn_tl \__cmd_grab_m_aux:Nnnnnnnnn
    \l__cmd_fn_tl \use_none:nnnnnnn { } { } { } { } { } { } { }
  }
\cs_new_protected:cpn { __cmd_grab_m_2:w } #1 \__cmd_run_code:
  {
    \tl_set:Nn \l__cmd_signature_tl {#1}
    \exp_after:wN \cs_set_eq:NN \l__cmd_fn_tl \__cmd_grab_m_aux:Nnnnnnnnn
    \l__cmd_fn_tl \use_none:nnnnnn { } { } { } { } { } { }
  }
\cs_new_protected:cpn { __cmd_grab_m_3:w } #1 \__cmd_run_code:
  {
    \tl_set:Nn \l__cmd_signature_tl {#1}
    \exp_after:wN \cs_set_eq:NN \l__cmd_fn_tl \__cmd_grab_m_aux:Nnnnnnnnn
    \l__cmd_fn_tl \use_none:nnnnn { } { } { } { } { }
  }
\cs_new_protected:cpn { __cmd_grab_m_4:w } #1 \__cmd_run_code:
  {
    \tl_set:Nn \l__cmd_signature_tl {#1}
    \exp_after:wN \cs_set_eq:NN \l__cmd_fn_tl \__cmd_grab_m_aux:Nnnnnnnnn
    \l__cmd_fn_tl \use_none:nnnn { } { } { } { }
  }
\cs_new_protected:cpn { __cmd_grab_m_5:w } #1 \__cmd_run_code:
  {
    \tl_set:Nn \l__cmd_signature_tl {#1}
    \exp_after:wN \cs_set_eq:NN \l__cmd_fn_tl \__cmd_grab_m_aux:Nnnnnnnnn
    \l__cmd_fn_tl \use_none:nnn { } { } { }
  }
\cs_new_protected:cpn { __cmd_grab_m_6:w } #1 \__cmd_run_code:
  {
    \tl_set:Nn \l__cmd_signature_tl {#1}
    \exp_after:wN \cs_set_eq:NN \l__cmd_fn_tl \__cmd_grab_m_aux:Nnnnnnnnn
    \l__cmd_fn_tl \use_none:nn { } { }
  }
\cs_new_protected:cpn { __cmd_grab_m_7:w } #1 \__cmd_run_code:
  {
```

```
    \tl_set:Nn \l__cmd_signature_tl {#1}
    \exp_after:wN \cs_set_eq:NN \l__cmd_fn_tl \__cmd_grab_m_aux:Nnnnnnnnn
    \l__cmd_fn_tl \use_none:n { }
  }
\cs_new_protected:cpn { __cmd_grab_m_8:w } #1 \__cmd_run_code:
  {
    \tl_set:Nn \l__cmd_signature_tl {#1}
    \exp_after:wN \cs_set_eq:NN \l__cmd_fn_tl \__cmd_grab_m_aux:Nnnnnnnnn
    \l__cmd_fn_tl \prg_do_nothing:
  }
\cs_new_protected:cpe { __cmd_grab_m_9:w }
  {
    \exp_not:c { __cmd_grab_m_5:w }
    \exp_not:c { __cmd_grab_m_4:w }
  }
```

(*End of definition for* `\__cmd_grab_m_1:w` *and others.*)

`\__cmd_grab_R:w` `\__cmd_grab_R_long:w` `\__cmd_grab_R_aux:NNnN`

The grabber for R-type arguments is basically the same as that for D-type ones, but always skips spaces (as it is mandatory) and has a hard-coded error message.

```
\cs_new_protected:Npn \__cmd_grab_R:w #1#2#3 \__cmd_run_code:
  { \__cmd_grab_R_aux:NNnN #1 #2 {#3} \cs_set_protected_nopar:Npn }
\cs_new_protected:Npn \__cmd_grab_R_long:w #1#2#3 \__cmd_run_code:
  { \__cmd_grab_R_aux:NNnN #1 #2 {#3} \cs_set_protected:Npn }
\cs_new_protected:Npn \__cmd_grab_R_aux:NNnN #1#2#3#4
  {
    \__cmd_grab_D_aux:NNnNN #1 #2 {#3} #4 \use_ii:nn
    \__cmd_peek_nonspace_remove:NTF #1
      { \__cmd_grab_D_call:Nw #1 }
      {
        \msg_error:nnee { cmd } { missing-required }
          { \__cmd_environment_or_command: }
          { \token_to_str:N #1 }
        \__cmd_add_arg:n { \NoValue }
      }
  }
```

(*End of definition for* `\__cmd_grab_R:w`, `\__cmd_grab_R_long:w`, *and* `\__cmd_grab_R_aux:NNnN`.)

`\__cmd_grab_t:w` `\__cmd_grab_t_obey_spaces:w` `\__cmd_grab_t_aux:NNw`

Dealing with a token is quite easy. Check the match, remove the token if needed and add a flag to the output.

```
\cs_new_protected:Npn \__cmd_grab_t:w
  { \__cmd_grab_t_aux:NNw \__cmd_peek_nonspace_remove:NTF }
\cs_new_protected:Npn \__cmd_grab_t_obey_spaces:w
  { \__cmd_grab_t_aux:NNw \__cmd_peek_meaning_remove:NTF }
\cs_new_protected:Npn \__cmd_grab_t_aux:NNw #1#2#3 \__cmd_run_code:
  {
    \tl_set:Nn \l__cmd_signature_tl {#3}
    \exp_after:wN \cs_set_protected:Npn \l__cmd_fn_tl
      {
        #1 #2
          { \__cmd_add_arg:n { \BooleanTrue } }
          { \__cmd_add_arg:n { \BooleanFalse } }
      }
```

```
    \l__cmd_fn_tl
  }
```

(*End of definition for* `\__cmd_grab_t:w`, `\__cmd_grab_t_obey_spaces:w`, *and* `\__cmd_grab_t_aux:NNw`.)

`\l__cmd_v_arg_tl`

```
\tl_new:N \l__cmd_v_arg_tl
```

`\__cmd_grab_v:w`
`\__cmd_grab_v_long:w`
`\__cmd_grab_v_aux:w`
`\__cmd_grab_v_group_end:`

Firstly, it is necessary to change `\tex_endlinechar:D` so that newlines in different catcode regimes (e.g., `\ExplSyntaxOn`) are not misinterpreted as spaces. The opening delimiter is the first non-space token, and is never read verbatim. This is required by consistency with the case where the preceding argument was optional and absent: then TEX has already read and tokenized that token when looking for the optional argument. The first thing is thus to check is that this delimiter is a character, and to distinguish the case of a left brace (in that case, `\group_align_safe_end:` is needed to compensate for the begin-group character that was just seen). Then set verbatim catcodes with `\__cmd_grab_v_aux_catcodes:`.

The group keep catcode changes local, and `\group_align_safe_begin/end:` allow to use a character with category code 4 (normally `&`) as the delimiter (all commands do `\group_align_safe_begin/end:`, so there's no need to do that again here). It is ended by `\__cmd_grab_v_group_end:`, which smuggles the collected argument out of the group.

```
\cs_new_protected:Npn \__cmd_grab_v:w
  {
    \bool_set_false:N \l__cmd_long_bool
    \__cmd_grab_v_aux:w
  }
\cs_new_protected:Npn \__cmd_grab_v_long:w
  {
    \bool_set_true:N \l__cmd_long_bool
    \__cmd_grab_v_aux:w
  }
\cs_new_protected:Npn \__cmd_grab_v_aux:w #1 \__cmd_run_code:
  {
    \tl_set:Nn \l__cmd_signature_tl {#1}
    \group_begin:
      \tex_escapechar:D = 92 \scan_stop:
      \tex_endlinechar:D = `\^^M \scan_stop:
      \tl_clear:N \l__cmd_v_arg_tl
      \peek_remove_spaces:n
        {
          \peek_meaning_remove:NTF \c_group_begin_token
            {
              \group_align_safe_end:
              \__cmd_grab_v_bgroup:
            }
            {
              \peek_N_type:TF
                { \__cmd_grab_v_aux_test:N }
                { \__cmd_grab_v_aux_abort:n { } }
            }
        }
```

```
  }
\cs_new_protected:Npn \__cmd_grab_v_group_end:
  {
      \exp_args:NNNo
    \group_end:
    \tl_set:Nn \l__cmd_v_arg_tl { \l__cmd_v_arg_tl }
  }
```

(*End of definition for* `\__cmd_grab_v:w` *and others.*)

`\__cmd_grab_v_aux_test:N`
`\__cmd_grab_v_aux_loop:N`
`\__cmd_grab_v_aux_loop:NN`
`\__cmd_grab_v_aux_loop_end:`

Check that the opening delimiter is a character, setup category codes, then start reading tokens one by one, keeping the delimiter as an argument. If the verbatim was not nested, we will be grabbing one character at each step. Unfortunately, it can happen that what follows the verbatim argument is already tokenized. Thus, we check at each step that the next token is indeed a "nice" character, *i.e.*, is not a character with category code 1 (begin-group), 2 (end-group) or 6 (macro parameter), nor the space character, with category code 10 and character code 32, nor a control sequence. The partially built argument is stored in `\l__cmd_v_arg_tl`. If we ever meet a token which we cannot grab (non-N-type), or which is not a character according to `\__cmd_grab_v_token_if_char:NTF`, then we bail out with `\__cmd_grab_v_aux_abort:n`. Otherwise, we stop at the first character matching the delimiter.

```
\cs_new_protected:Npn \__cmd_grab_v_aux_test:N #1
  {
    \__cmd_grab_v_token_if_char:NTF #1
      {
        \__cmd_grab_v_aux_put:N #1
        \__cmd_grab_v_aux_catcodes:
        \__cmd_grab_v_aux_loop:N #1
      }
      { \__cmd_grab_v_aux_abort:n {#1} #1 }
  }
\cs_new_protected:Npn \__cmd_grab_v_aux_loop:N #1
  {
    \peek_N_type:TF
      { \__cmd_grab_v_aux_loop:NN #1 }
      { \__cmd_grab_v_aux_abort:n { } }
  }
\cs_new_protected:Npn \__cmd_grab_v_aux_loop:NN #1#2
  {
    \__cmd_grab_v_token_if_char:NTF #2
      {
        \token_if_eq_charcode:NNTF #1 #2
          { \__cmd_grab_v_aux_loop_end: }
          {
            \__cmd_grab_v_aux_put:N #2
            \__cmd_grab_v_aux_loop:N #1
          }
      }
      { \__cmd_grab_v_aux_abort:n {#2} #2 }
  }
\cs_new_protected:Npn \__cmd_grab_v_aux_loop_end:
  {
    \__cmd_grab_v_group_end:
```

```
    \__cmd_add_arg:e { \tl_tail:N \l__cmd_v_arg_tl }
  }
```

(*End of definition for* `\__cmd_grab_v_aux_test:N` *and others.*)

`\l__cmd_v_nesting_int`

```
\int_new:N \l__cmd_v_nesting_int
```

`\__cmd_grab_v_bgroup:`
`\__cmd_grab_v_bgroup_loop:`
`\__cmd_grab_v_bgroup_loop:N`

If the opening delimiter is a left brace, we keep track of how many left and right braces were encountered so far in `\l__cmd_v_nesting_int` (the methods used for optional arguments cannot apply here), and stop as soon as it reaches 0.

Some care was needed when removing the opening delimiter, which has already been assigned category code 1: using `\peek_meaning_remove:NTF` in the `\__cmd_grab_v_aux:w` function would break within alignments. Instead, we first convert that token to a string, and remove the result as a normal undelimited argument.

```
\cs_new_protected:Npe \__cmd_grab_v_bgroup:
  {
    \exp_not:N \__cmd_grab_v_aux_catcodes:
    \exp_not:n { \int_set:Nn \l__cmd_v_nesting_int { 1 } }
    \exp_not:N \__cmd_grab_v_aux_put:N \iow_char:N \{
    \exp_not:N \__cmd_grab_v_bgroup_loop:
  }
\cs_new_protected:Npn \__cmd_grab_v_bgroup_loop:
  {
    \peek_N_type:TF
      { \__cmd_grab_v_bgroup_loop:N }
      { \__cmd_grab_v_aux_abort:n { } }
  }
\cs_new_protected:Npn \__cmd_grab_v_bgroup_loop:N #1
  {
    \__cmd_grab_v_token_if_char:NTF #1
      {
        \token_if_eq_charcode:NNTF \c_group_end_token #1
          {
            \int_decr:N \l__cmd_v_nesting_int
            \int_compare:nNnTF \l__cmd_v_nesting_int > 0
              {
                \__cmd_grab_v_aux_put:N #1
                \__cmd_grab_v_bgroup_loop:
              }
              { \__cmd_grab_v_aux_loop_end: }
          }
          {
            \token_if_eq_charcode:NNT \c_group_begin_token #1
              { \int_incr:N \l__cmd_v_nesting_int }
            \__cmd_grab_v_aux_put:N #1
            \__cmd_grab_v_bgroup_loop:
          }
      }
      { \__cmd_grab_v_aux_abort:n {#1} #1 }
  }
```

(*End of definition for* `\__cmd_grab_v_bgroup:`, `\__cmd_grab_v_bgroup_loop:`, *and* `\__cmd_grab_v_bgroup_loop:N`.)

__cmd_grab_v_aux_catcodes:
__cmd_grab_v_aux_abort:n

The approach for short verbatim arguments is to make the end-line character a macro parameter character: this is forbidden by the rest of the code. Then the error branch can check what caused the bail out and give the appropriate error message.

```
⟨latexrelease⟩\IncludeInRelease{2025/06/01}{\__cmd_grab_v_aux_catcodes:}%
⟨latexrelease⟩  {Active~spaces~and~tabs}
\cs_new_protected:Npn \__cmd_grab_v_aux_catcodes:
  {
    \cs_set_eq:NN \do \char_set_catcode_other:N
    \dospecials
    \char_set_catcode_active:n { `\  }
    \char_set_catcode_active:n { `\^^I }
    \bool_if:NTF \l__cmd_long_bool
      { \char_set_catcode_other:n { \tex_endlinechar:D } }
      { \char_set_catcode_parameter:n { \tex_endlinechar:D } }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__cmd_grab_v_aux_catcodes:}%
⟨latexrelease⟩  {Active~spaces~and~tabs}
⟨latexrelease⟩\cs_new_protected:Npn \__cmd_grab_v_aux_catcodes:
⟨latexrelease⟩  {
⟨latexrelease⟩    \cs_set_eq:NN \do \char_set_catcode_other:N
⟨latexrelease⟩    \dospecials
⟨latexrelease⟩    \bool_if:NTF \l__cmd_long_bool
⟨latexrelease⟩      { \char_set_catcode_other:n { \tex_endlinechar:D } }
⟨latexrelease⟩      { \char_set_catcode_parameter:n { \tex_endlinechar:D } }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
\cs_new_protected:Npn \__cmd_grab_v_aux_abort:n #1
  {
    \__cmd_grab_v_group_end:
    \exp_after:wN \exp_after:wN \exp_after:wN
      \peek_meaning_remove:NTF \char_generate:nn { \tex_endlinechar:D } { 6 }
      {
        \msg_error:nneee { cmd } { verbatim-nl }
          { \__cmd_environment_or_command: }
          { \tl_to_str:N \l__cmd_v_arg_tl }
          { \tl_to_str:n {#1} }
        \__cmd_add_arg:n { \NoValue }
      }
      {
        \msg_error:nneee { cmd } { verbatim-tokenized }
          { \__cmd_environment_or_command: }
          { \tl_to_str:N \l__cmd_v_arg_tl }
          { \tl_to_str:n {#1} }
        \__cmd_add_arg:n { \NoValue }
      }
  }
```

(*End of definition for* `\__cmd_grab_v_aux_catcodes:` *and* `\__cmd_grab_v_aux_abort:n`.)

__cmd_grab_v_aux_put:N

Storing one token in the collected argument: everything as-is except for end-of-lines, with `\exp_not:N` to handle actives.

```
⟨latexrelease⟩\IncludeInRelease{2025-06-01}{\__cmd_grab_v_aux_put:N}%
⟨latexrelease⟩  {Use~more~std~catcodes}
```

```
\cs_new_protected:Npn \__cmd_grab_v_aux_put:N #1
  {
    \tl_put_right:Ne \l__cmd_v_arg_tl
      {
        \int_compare:nNnTF {`#1} = \tex_endlinechar:D
          { \exp_not:N \obeyedline }
          { \exp_not:N #1 }
      }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2024/06/01}{\__cmd_grab_v_aux_put:N}%
⟨latexrelease⟩  {Endlines~as~\obeyedline}
⟨latexrelease⟩\cs_new_protected:Npn \__cmd_grab_v_aux_put:N #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_put_right:Nx \l__cmd_v_arg_tl
⟨latexrelease⟩      {
⟨latexrelease⟩        \token_if_active:NTF #1
⟨latexrelease⟩          { \exp_not:N #1 }
⟨latexrelease⟩          {
⟨latexrelease⟩            \int_compare:nNnTF {`#1} = \tex_endlinechar:D
⟨latexrelease⟩              { \exp_not:N \obeyedline }
⟨latexrelease⟩              { \token_to_str:N #1 }
⟨latexrelease⟩          }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__cmd_grab_v_aux_put:N}%
⟨latexrelease⟩  {Endlines~as~\obeyedline}
⟨latexrelease⟩\cs_new_protected:Npn \__cmd_grab_v_aux_put:N #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_put_right:Nx \l__cmd_v_arg_tl
⟨latexrelease⟩      {
⟨latexrelease⟩        \token_if_active:NTF #1
⟨latexrelease⟩          { \exp_not:N #1 } { \token_to_str:N #1 }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__cmd_grab_v_aux_put:N`.)

`\__cmd_grab_v_token_if_char:NTF` This function assumes that the escape character is printable. Then the string representation of control sequences is at least two characters, and `\str_tail:n` only removes the escape character. Macro parameter characters are doubled by `\tl_to_str:n`, and will also yield a non-empty result, hence are not considered as characters.

```
\cs_new_protected:Npn \__cmd_grab_v_token_if_char:NTF #1
  { \str_if_eq:eeTF { } { \str_tail:n {#1} } }
```

(*End of definition for* `\__cmd_grab_v_token_if_char:NTF`.)

`\__cmd_add_arg:n` `\__cmd_add_arg:V` `\__cmd_add_arg:o` `\__cmd_add_arg:e` When an argument is found it is stored, then further arguments are grabbed by calling `\l__cmd_signature_tl`.

```
\cs_new_protected:Npn \__cmd_add_arg:n #1
  {
    \tl_put_right:Nn \l__cmd_args_tl { {#1} }
```

```
    \l__cmd_signature_tl \__cmd_run_code:
  }
\cs_generate_variant:Nn \__cmd_add_arg:n { V , o , e }
```

(*End of definition for* `\__cmd_add_arg:n`.)

1.28 Grabbing arguments expandably

`\__cmd_expandable_grab_D:w`
`\__cmd_expandable_grab_D:NNNwNNn`
`\__cmd_expandable_grab_D:NNNwNNnnn`
`\__cmd_expandable_grab_D:Nw`
`\__cmd_expandable_grab_D:nnNNNwNN`

The first step is to grab the first token or group. The generic grabbers $\backslash\langle$*function*$\rangle_{\sqcup}$ and $\backslash\langle$*function*$\rangle_{\sqcup}$ are just after `\q__cmd`, we go and find them (and use the long one).

```
\cs_new:Npn \__cmd_expandable_grab_D:w #1 \q__cmd #2#3
  { #2 { \__cmd_expandable_grab_D:NNNwNNn #1 \q__cmd #2 #3 } }
```

We then wish to test whether `#7`, which we just grabbed, is exactly `#2`. A preliminary test is whether their string representations coincide, then expand the only grabber function we have, `#1`, once: the two strings below are equal if and only if `#7` matches `#2` exactly.[2] The preliminary test is needed as `#7` could validly contain `\par` (because a later mandatory argument could be long) and our grabber may be short. If `#7` does not match `#2`, then the optional argument is missing, we use the default `\NoValue`, and put back the argument `#7` in the input stream.

If it does match, then interesting things need to be done. We will grab the argument piece by piece, with the following pattern:

$\langle$*grabber*$\rangle$ {$\langle$*tokens*$\rangle$}
`\q_nil` {$\langle$*piece 1*$\rangle$} $\langle$*piece 2*$\rangle$ `\ERROR` `\q__cmd`
`\q_nil` $\langle$*input stream*$\rangle$

The $\langle$*grabber*$\rangle$ will find an opening delimiter in $\langle$*piece 2*$\rangle$, take the `\q__cmd` as a second delimiter, and find more material delimited by the closing delimiter in the $\langle$*input stream*$\rangle$. We then move the part before the opening delimiter from $\langle$*piece 2*$\rangle$ to $\langle$*piece 1*$\rangle$, and the material taken from the $\langle$*input stream*$\rangle$ to the $\langle$*piece 2*$\rangle$. Thus, the argument moves gradually from the $\langle$*input stream*$\rangle$ to the $\langle$*piece 2*$\rangle$, then to the $\langle$*piece 1*$\rangle$ when we have made sure to find all opening and closing delimiters. This two-step process ensures that nesting works: the number of opening delimiters minus closing delimiters in $\langle$*piece 1*$\rangle$ is always equal to the number of closing delimiters in $\langle$*piece 2*$\rangle$. We stop grabbing arguments once the $\langle$*piece 2*$\rangle$ contains no opening delimiter any more, hence the balance is reached, and the final argument is $\langle$*piece 1*$\rangle$ $\langle$*piece 2*$\rangle$.

```
\cs_new:Npn \__cmd_expandable_grab_D:NNNwNNn #1#2#3#4 \q__cmd #5#6#7
  {
    \str_if_eq:nnTF {#2} {#7}
      {
        \str_if_eq:onTF
          { #1 { } { } #7 #2 \q__cmd #3 }
          { { } {#2} { } }
      }
      { \use_ii:nn }
```

[2] It is obvious that if `#7` matches `#2` then the strings are equal. We must check the converse. The right-hand-side of `\str_if_eq:onTF` does not end with `#3`, implying that the grabber function took everything as its arguments. The first brace group can only be empty if `#7` starts with `#2`, otherwise the brace group preceding `#7` would not vanish. The third brace group is empty, thus the `\q__cmd` that was used by our grabber `#1` must be the one that we inserted (not some token in `#7`), hence the second brace group contains the end of `#7` followed by `#2`. Since this is `#2` on the right-hand-side, and no brace can be lost there, `#7` must contain nothing else than its leading `#2`.

```
      {
        #1
          { \__cmd_expandable_grab_D:NNNwNNnnn #1#2#3#4 \q__cmd #5#6 }
          \q_nil { } #2 \ERROR \q__cmd \ERROR
      }
      { #4 { \NoValue } \q__cmd #5 #6 {#7} }
  }
```

At this stage, `#7` is `\q_nil {`⟨*piece 1*⟩`}` ⟨*more for piece 1*⟩, and we want to concatenate all that, removing `\q_nil`, and keeping the opening delimiter `#2`. Simply use `\use_ii:nn`. Also, `#8` is ⟨*remainder of piece 2*⟩ `\ERROR`, and `#9` is `\ERROR` ⟨*more for piece 2*⟩. We concatenate those, replacing the two `\ERROR` by the closing delimiter `#3`.

```
\cs_new:Npn \__cmd_expandable_grab_D:NNNwNNnnn #1#2#3#4 \q__cmd #5#6#7#8#9
  {
    \exp_args:Nof \__cmd_expandable_grab_D:nnNNNwNN
      { \use_ii:nn #7 #2 }
      { \__cmd_expandable_grab_D:Nw #3 \exp_stop_f: #8 #9 }
    #1#2#3 #4 \q__cmd #5 #6
  }
\cs_new:Npn \__cmd_expandable_grab_D:Nw #1#2 \ERROR \ERROR { #2 #1 }
```

Armed with our two new ⟨*pieces*⟩, we are ready to loop. However, we must first see if ⟨*piece 2*⟩ (here `#2`) contains any opening delimiter `#4`. Again, we expand `#3`, this time removing its whole output with `\use_none:nnn`. The test is similar to `\tl_if_in:nnTF`. The token list is empty if and only if `#2` does not contain the opening delimiter. In that case, we are done, and put the argument (from which we remove a spurious pair of delimiters coming from how we started the loop). Otherwise, we go back to looping with `\__cmd_expandable_grab_D:NNNwNNnnn`. The code to deal with brace stripping is much the same as for the non-expandable case.

```
\cs_new:Npn \__cmd_expandable_grab_D:nnNNNwNN #1#2#3#4#5#6 \q__cmd #7#8
  {
    \exp_args:No \tl_if_empty:oTF
      { #3 { \use_none:nnn } #2 \q__cmd #5 #4 \q__cmd #5 }
      {
        \tl_if_blank:oTF { \use_none:nn #1#2 }
          { \__cmd_put_arg_expandable:ow { \use_none:nn #1#2 } }
          {
            \str_if_eq:eeTF
              { \exp_not:o { \use_none:nn #1#2 } }
              { { \exp_not:o { \use_iii:nnnn #1#2 \q_nil } } }
              { \__cmd_put_arg_expandable:ow { \use_iii:nnn #1#2 } }
              { \__cmd_put_arg_expandable:ow { \use_none:nn #1#2 } }
          }
            #6 \q__cmd #7 #8
      }
      {
        #3
          { \__cmd_expandable_grab_D:NNNwNNnnn #3#4#5#6 \q__cmd #7 #8 }
          \q_nil {#1} #2 \ERROR \q__cmd \ERROR
      }
  }
```

(*End of definition for* `\__cmd_expandable_grab_D:w` *and others.*)

__cmd_expandable_grab_D_alt:w
__cmd_expandable_grab_D_alt:NNwNNn
__cmd_expandable_grab_D_alt:Nwn

When the delimiters are identical, nesting is not possible and a simplified approach is used. The test concept here is the same as for the case where the delimiters are different but there cannot be any nesting.

```
\cs_new:Npn \__cmd_expandable_grab_D_alt:w #1 \q__cmd #2#3
  { #2 { \__cmd_expandable_grab_D_alt:NNwNNn #1 \q__cmd #2 #3 } }
\cs_new:Npn \__cmd_expandable_grab_D_alt:NNwNNn #1#2#3 \q__cmd #4#5#6
  {
    \str_if_eq:nnTF {#6} {#2}
      {
        \str_if_eq:onTF
          { #1 { } #6 #2 #2 }
          { { } #2 }
      }
      { \use_ii:nn }
      {
        #1
          { \__cmd_expandable_grab_D_alt:NNwn #4 #5 #3 \q__cmd }
          #6 \ERROR
      }
      { #3 { \NoValue } \q__cmd #4 #5 {#6} }
  }
\cs_new:Npn \__cmd_expandable_grab_D_alt:NNwn #1#2#3 \q__cmd #4
  {
    \tl_if_blank:oTF { \use_none:n #4 }
      { \__cmd_put_arg_expandable:ow { \use_none:n #4 } }
      {
        \str_if_eq:eeTF
          { \exp_not:o { \use_none:n #4 } }
          { { \exp_not:o { \use_ii:nnn #4 \q_nil } } }
          { \__cmd_put_arg_expandable:ow { \use_ii:nn #4 } }
          { \__cmd_put_arg_expandable:ow { \use_none:n #4 } }
      }
        #3 \q__cmd #1 #2
  }
```

(*End of definition for* `\__cmd_expandable_grab_D_alt:w`, `\__cmd_expandable_grab_D_alt:NNwNNn`, *and* `\__cmd_expandable_grab_D_alt:Nwn`.)

__cmd_expandable_grab_E:w
__cmd_expandable_grab_E_long:w
__cmd_expandable_grab_E_aux:w
__cmd_expandable_grab_E_test:nnw
__cmd_expandable_grab_E_loop:nnnNNw
__cmd_expandable_grab_E_find:w
__cmd_expandable_grab_E_find:nnw
__cmd_expandable_grab_E_end:nnw

We keep track of long/short by placing the appropriate grabber as the third token after `\q__cmd`; it is eventually removed by the `end:nnw` auxiliary. The `aux:w` auxiliary will be called repeatedly with two arguments: the set of pairs ⟨*parser*⟩ ⟨*token*⟩, and the set of arguments found so far (initially all `{\NoValue}`). At each step, grab what follows in the input stream then call the `loop:nnnNNw` auxiliary to compare it with each possible embellishment in turn. This auxiliary's `#1` is what was found in the input, `#2` collects ⟨*parser*⟩ ⟨*token*⟩ pairs that did not match, `#3` collects the corresponding arguments found previously, `#4` and `#5` is the current pair, `#6` is the remaining pairs, `#7` is empty or two `\q_nil`, and `#8` is the current argument. If none of the pairs matched (determined by `\quark_if_nil:NTF`) then call the `end` auxiliary to stop looking for embellishments, remembering to put what was grabbed in the input back where it belongs, and storing the arguments found just before `\q__cmd`. If the current argument `#8` is not `\NoValue` or if the input `#1` does not match `#5` (see `t`-type arguments below for a similar `\str_if_eq:onTF` test) then carry on the loop. Otherwise, we found a new embellishment: grab the corresponding argument in the input using the `find:w` auxiliary. To avoid losing

braces around that auxiliary's argument we include a space, which will be eliminated in the next loop through embellishments.

```
\cs_new:Npn \__cmd_expandable_grab_E:w #1 \q__cmd #2#3
  { \__cmd_expandable_grab_E_aux:w #1 \q__cmd #2 #3 #3 }
\cs_new:Npn \__cmd_expandable_grab_E_long:w #1 \q__cmd #2#3
  { \__cmd_expandable_grab_E_aux:w #1 \q__cmd #2 #3 #2 }
\cs_new:Npn \__cmd_expandable_grab_E_aux:w #1 \q__cmd #2#3#4
  { #2 { \__cmd_expandable_grab_E_test:nnw #1 \q__cmd #2 #3 #4 } }
\cs_new:Npn \__cmd_expandable_grab_E_test:nnw #1#2#3 \q__cmd #4#5#6#7
  {
    \__cmd_expandable_grab_E_loop:nnnNNw {#7} { } { }
      #1 \q_nil \q_nil \q_nil \q_mark #2 \q_nil
    #3 \q__cmd #4 #5 #6
  }
\cs_new:Npn \__cmd_expandable_grab_E_loop:nnnNNw
    #1#2#3#4#5#6 \q_nil #7 \q_mark #8
  {
    \quark_if_nil:NTF #4
      { \__cmd_expandable_grab_E_end:nnw {#1} {#3} }
      {
        \tl_if_novalue:nTF {#8}
          { \str_if_eq:onTF { #4 { } #1 #5 } {#5} }
          { \use_ii:nn }
            { \__cmd_expandable_grab_E_find:w { #2 #4 #5 #6 } {#3} ~ }
            {
              \__cmd_expandable_grab_E_loop:nnnNNw
                {#1} { #2 #4 #5 } { #3 {#8} }
                #6 \q_nil #7 \q_mark
            }
      }
  }
\cs_new:Npn \__cmd_expandable_grab_E_find:w #1 \q__cmd #2#3#4
  { #4 { \__cmd_expandable_grab_E_find:nnw #1 \q__cmd #2 #3 #4 } }
\cs_new:Npn \__cmd_expandable_grab_E_find:nnw #1#2#3 \q_nil #4 \q__cmd #5#6#7#8
  { \__cmd_expandable_grab_E_aux:w {#1} { #2 {#8} #3 } #4 \q__cmd #5 #6 #7 }
\cs_new:Npn \__cmd_expandable_grab_E_end:nnw #1#2#3 \q__cmd #4#5#6
  { #3 #2 \q__cmd #4 #5 {#1} }
```

(*End of definition for* `\__cmd_expandable_grab_E:w` *and others.*)

`\__cmd_expandable_grab_m:w`
`\__cmd_expandable_grab_m_long:w`
`\__cmd_expandable_grab_m_aux:wNn`

The mandatory case is easy: find the auxiliary after the `\q__cmd`, and use it directly to grab the argument, then correctly position the argument before `\q__cmd`.

```
\cs_new:Npn \__cmd_expandable_grab_m:w #1 \q__cmd #2#3
  { #3 { \__cmd_expandable_grab_m_aux:wNn #1 \q__cmd #2 #3 } }
\cs_new:Npn \__cmd_expandable_grab_m_long:w #1 \q__cmd #2#3
  { #2 { \__cmd_expandable_grab_m_aux:wNn #1 \q__cmd #2 #3 } }
\cs_new:Npn \__cmd_expandable_grab_m_aux:wNn #1 \q__cmd #2#3#4
  { #1 {#4} \q__cmd #2 #3 }
```

(*End of definition for* `\__cmd_expandable_grab_m:w`, `\__cmd_expandable_grab_m_long:w`, *and* `\__cmd_expandable_grab_m_aux:wNn`.)

`\__cmd_expandable_grab_R:w`
`\__cmd_expandable_grab_R_aux:NNNwNNn`

Much the same as for the D-type argument, with only the lead-off function varying.

```
\cs_new:Npn \__cmd_expandable_grab_R:w #1 \q__cmd #2#3
```

```
  { #2 { \__cmd_expandable_grab_R_aux:NNNwNNn #1 \q__cmd #2#3 } }
\cs_new:Npn \__cmd_expandable_grab_R_aux:NNNwNNn #1#2#3#4 \q__cmd #5#6#7
  {
    \str_if_eq:nnTF {#7} {#2}
      {
        \str_if_eq:onTF
          { #1 { } { } #7 #2 \q__cmd #3 }
          { { } {#2} { } }
      }
      { \use_ii:nn }
      {
        #1
          { \__cmd_expandable_grab_D:NNNwNNnnn #1#2#3#4 \q__cmd #5#6 }
          \q_nil { } #2 \ERROR \q__cmd \ERROR
      }
      {
        \msg_expandable_error:nnff { cmd } { missing-required }
          { \exp_args:Nf \tl_trim_spaces:n { \token_to_str:N #5 } }
          { \tl_to_str:n {#2} }
        #4 { \NoValue } \q__cmd #5 #6 {#7}
      }
  }
```

(*End of definition for* `\__cmd_expandable_grab_R:w` *and* `\__cmd_expandable_grab_R_aux:NNNwNNn`.)

`\__cmd_expandable_grab_R_alt:w`
`\__cmd_expandable_grab_R_alt_aux:NNwNNn`

When the delimiters are identical, nesting is not possible and a simplified approach is used. The test concept here is the same as for the case where the delimiters are different.

```
\cs_new:Npn \__cmd_expandable_grab_R_alt:w #1 \q__cmd #2#3
  { #2 { \__cmd_expandable_grab_R_alt_aux:NNwNNn #1 \q__cmd #2#3 } }
\cs_new:Npn \__cmd_expandable_grab_R_alt_aux:NNwNNn #1#2#3 \q__cmd #4#5#6
  {
    \str_if_eq:nnTF {#6} {#2}
      {
        \str_if_eq:onTF
          { #1 { } #6 #2 #2 }
          { { } #2 }
      }
      { \use_ii:nn }
      {
        #1
          { \__cmd_expandable_grab_D_alt:NNwn #4 #5 #3 \q__cmd }
          #6 \ERROR
      }
      {
        \msg_expandable_error:nnff { cmd } { missing-required }
          { \exp_args:Nf \tl_trim_spaces:n { \token_to_str:N #4 } }
          { \tl_to_str:n {#2} }
        #3 { \NoValue } \q__cmd #4 #5 {#6}
      }
  }
```

(*End of definition for* `\__cmd_expandable_grab_R_alt:w` *and* `\__cmd_expandable_grab_R_alt_aux:NNwNNn`.)

__cmd_expandable_grab_t:w
__cmd_expandable_grab_t_aux:NNwn

As for a D-type argument, here we compare the grabbed tokens using the only parser we have in order to work out if #2 is exactly equal to the output of the grabber.

```
\cs_new:Npn \__cmd_expandable_grab_t:w #1 \q__cmd #2#3
  { #2 { \__cmd_expandable_grab_t_aux:NNwn #1 \q__cmd #2 #3 } }
\cs_new:Npn \__cmd_expandable_grab_t_aux:NNwn #1#2#3 \q__cmd #4#5#6
  {
    \str_if_eq:onTF { #1 { } #6 #2 } {#2}
      { #3 { \BooleanTrue } \q__cmd #4 #5 }
      { #3 { \BooleanFalse } \q__cmd #4 #5 {#6} }
  }
```

(*End of definition for* `\__cmd_expandable_grab_t:w` *and* `\__cmd_expandable_grab_t_aux:NNwn`.)

__cmd_put_arg_expandable:nw
__cmd_put_arg_expandable:ow

A useful helper, to store arguments when they are ready.

```
\cs_new:Npn \__cmd_put_arg_expandable:nw #1#2 \q__cmd { #2 {#1} \q__cmd }
\cs_generate_variant:Nn \__cmd_put_arg_expandable:nw { o }
```

(*End of definition for* `\__cmd_put_arg_expandable:nw`.)

1.29 Argument processors

__cmd_bool_reverse:N

A simple reversal.

```
\cs_new_protected:Npn \__cmd_bool_reverse:N #1
  {
    \bool_if:NTF #1
      { \tl_set:Nn \ProcessedArgument { \c_false_bool } }
      { \tl_set:Nn \ProcessedArgument { \c_true_bool } }
  }
```

(*End of definition for* `\__cmd_bool_reverse:N`.)

\l__cmd_split_list_seq
\l__cmd_split_list_tl
__cmd_split_list_multi:nn
__cmd_split_list_multi:nV
__cmd_split_list_single:Nn

Splitting can take place either at a single token or at a longer identifier. To deal with single active tokens, a two-part procedure is needed.

```
\seq_new:N \l__cmd_split_list_seq
\tl_new:N \l__cmd_split_list_tl
\cs_new_protected:Npn \__cmd_split_list:nn #1#2
  {
    \tl_if_single:nTF {#1}
      {
        \token_if_cs:NTF #1
          { \__cmd_split_list_multi:nn {#1} {#2} }
          { \__cmd_split_list_single:Nn #1 {#2} }
      }
      { \__cmd_split_list_multi:nn {#1} {#2} }
  }
\cs_new_protected:Npn \__cmd_split_list_multi:nn #1#2
  {
    \seq_set_split:Nnn \l__cmd_split_list_seq {#1} {#2}
    \tl_clear:N \ProcessedArgument
    \seq_map_inline:Nn \l__cmd_split_list_seq
      { \tl_put_right:Nn \ProcessedArgument { {##1} } }
  }
\cs_generate_variant:Nn \__cmd_split_list_multi:nn { nV }
\group_begin:
\char_set_catcode_active:N \^^@
\cs_new_protected:Npn \__cmd_split_list_single:Nn #1#2
  {
    \tl_set:Nn \l__cmd_split_list_tl {#2}
    \group_begin:
    \char_set_lccode:nn { `\^^@ } { `#1 }
    \tex_lowercase:D
      {
        \group_end:
        \tl_replace_all:Nnn \l__cmd_split_list_tl { ^^@ }
      }   {#1}
     \__cmd_split_list_multi:nV {#1} \l__cmd_split_list_tl
   }
\group_end:
```

(*End of definition for* `\__cmd_split_list:nn`*,* `\__cmd_split_list_multi:nn`*, and* `\__cmd_split_list_single:Nn`*.*)

__cmd_split_argument:nnn
__cmd_split_argument_aux:nnnn
__cmd_split_argument_aux:n
__cmd_split_argument_aux:wn

Splitting to a known number of items is a special version of splitting a list, in which the limit is hard-coded and where there will always be exactly the correct number of output items. An auxiliary function is used to save on working out the token list length several times.

```
\cs_new_protected:Npn \__cmd_split_argument:nnn #1#2#3
  {
    \__cmd_split_list:nn {#2} {#3}
    \exp_args:Nf \__cmd_split_argument_aux:nnnn
      { \tl_count:N \ProcessedArgument }
      {#1} {#2} {#3}
  }
```

```
\cs_new_protected:Npn \__cmd_split_argument_aux:nnnn #1#2#3#4
  {
    \int_compare:nNnF {#1} = { #2 + 1 }
      {
        \int_compare:nNnTF {#1} > { #2 + 1 }
          {
            \tl_set:Ne \ProcessedArgument
              {
                \exp_last_unbraced:NnNo
                  \__cmd_split_argument_aux:n
                  { #2 + 1 }
                  \use_none_delimit_by_q_stop:w
                  \ProcessedArgument
                  \q_stop
              }
            \msg_error:nneee { cmd } { arg-split }
              { \tl_to_str:n {#3} } { \int_eval:n { #2 + 1 } }
              { \tl_to_str:n {#4} }
          }
          {
            \tl_put_right:Ne \ProcessedArgument
              {
                \prg_replicate:nn { #2 + 1 - (#1) }
                  { { \NoValue } }
              }
          }
      }
  }
```

Auxiliaries to leave exactly the correct number of arguments in `\ProcessedArgument`.

```
\cs_new:Npn \__cmd_split_argument_aux:n #1
  { \prg_replicate:nn {#1} { \__cmd_split_argument_aux:wn } }
\cs_new:Npn \__cmd_split_argument_aux:wn #1 \use_none_delimit_by_q_stop:w #2
  {
    \exp_not:n { {#2} }
    #1
    \use_none_delimit_by_q_stop:w
  }
```

(*End of definition for* `\__cmd_split_argument:nnn` *and others.*)

`\__cmd_trim_spaces:n` This one is almost trivial.

```
\cs_new_protected:Npn \__cmd_trim_spaces:n #1
  { \tl_set:Ne \ProcessedArgument { \tl_trim_spaces:n {#1} } }
```

(*End of definition for* `\__cmd_trim_spaces:n`.)

1.30 Conversion to key–value form

This is implemented as a process but with no public interfaces, hence is treated separately from the others: it's a feature of ltcmd which just happens to use the same mechanism as a processor.

__cmd_arg_to_keyvalue:nn
__cmd_arg_to_keyvalue_braces:nnn
__cmd_arg_to_keyvalue_auxi:nnn
__cmd_arg_to_keyvalue_auxii:Nnnn
__cmd_arg_to_keyvalue_auxiii:nnn
__cmd_arg_to_keyvalue_auxiv:Nnnn
__cmd_arg_to_keyvalue_auxv:nn
__cmd_arg_to_keyvalue_loop:w
__cmd_arg_to_keyvalue_loop_group:n
__cmd_arg_to_keyvalue_loop_space:w
__cmd_arg_to_keyvalue_loop_N_type:N
__cmd_arg_to_keyvalue_math:w
__cmd_arg_to_keyvalue_math_N_type:N
__cmd_arg_to_keyvalue_math_group:n
__cmd_arg_to_keyvalue_math_space:w
__cmd_arg_to_keyvalue_set_default:nn
__cmd_arg_to_keyvalue_set_keyvalue:nn
__cmd_split_N_head_apply:Nn
__cmd_split_N_head_apply_aux:NNw

If the entire argument is braced, we treat as free text and return as the value for the text key. Alternatively, if the start of the input is `=`, then it is forced to be key–value. To avoid needing to worry about catcodes for this, and to allow spaces around the `=`, we use a series of steps rather than a delimited argument. We also deal with the case where the entire argument is blank (or is an entirely empty brace group).

```
\cs_new_protected:Npn \__cmd_arg_to_keyvalue:nn #1#2
  {
    \tl_trim_spaces_apply:nN {#2} \__cmd_arg_to_keyvalue_braces:nnn
      {#1} {#2}
  }
\cs_new_protected:Npn \__cmd_arg_to_keyvalue_braces:nnn #1#2#3
  {
    \bool_lazy_and:nnTF
      { \tl_if_head_is_group_p:n {#1} }
      { \tl_if_blank_p:o { \use_none:n #1 } }
      { \tl_set:Ne \ProcessedArgument { #2 = { \exp_not:n #1 } } }
      {
        \tl_if_blank:nTF {#1}
          { \tl_clear:N \ProcessedArgument }
          { \__cmd_arg_to_keyvalue_auxi:nnn {#1} {#2} {#3} }
      }
  }
\cs_new:Npn \__cmd_arg_to_keyvalue_auxi:nnn #1
  {
    \tl_if_head_is_N_type:nTF {#1}
      { \__cmd_split_N_head_apply:Nn \__cmd_arg_to_keyvalue_auxii:Nnnn {#1} }
      { \__cmd_arg_to_keyvalue_auxv:nn }
  }
\cs_new:Npn \__cmd_arg_to_keyvalue_auxii:Nnnn #1#2
  {
    \str_if_eq:eeTF { \exp_not:n {#1} } { = }
      { \tl_trim_spaces_apply:nN {#2} \__cmd_arg_to_keyvalue_auxiii:nnn }
      { \__cmd_arg_to_keyvalue_auxv:nn }
  }
\cs_new:Npn \__cmd_arg_to_keyvalue_auxiii:nnn #1
  {
    \tl_if_head_is_N_type:nTF {#1}
      { \__cmd_split_N_head_apply:Nn \__cmd_arg_to_keyvalue_auxiv:Nnnn {#1} }
      { \__cmd_arg_to_keyvalue_auxv:nn }
  }
\cs_new:Npn \__cmd_arg_to_keyvalue_auxiv:Nnnn #1#2
  {
    \str_if_eq:eeTF { \exp_not:n {#1} } { , }
      { \tl_set:Nn \ProcessedArgument {#2} \use_none:nn }
      { \__cmd_arg_to_keyvalue_auxv:nn }
  }
```

The two clear-cut cases have been eliminated, and we therefore have to deal with a search for `=` signs. We need an "action" loop here so we do not get mislead by for example `{=}`. As the code here is for very much predictable types of input, we hard-code what constitutes math mode opening and closing. At the very beginning, the default key (`#1`) and the argument as given by the user (`#2`) are placed right after the `\q__cmd_recursion_stop`, so that when the recursion ends, the macros `\__cmd_arg_to_keyvalue_set_default:nn` or `\__cmd_arg_to_keyvalue_set_keyvalue:nn` can be used to grab these two items and

set the \ProcessedArgument accordingly.

```
\cs_new_protected:Npn \__cmd_arg_to_keyvalue_auxv:nn #1#2
  {
    \__cmd_arg_to_keyvalue_loop:w #2
      \q__cmd_recursion_tail \q__cmd_recursion_stop {#1} {#2}
  }
\cs_new_protected:Npn \__cmd_arg_to_keyvalue_loop:w #1 \q__cmd_recursion_stop
  {
    \tl_if_head_is_N_type:nTF {#1}
      { \__cmd_arg_to_keyvalue_loop_N_type:N }
      {
        \tl_if_head_is_group:nTF {#1}
          { \__cmd_arg_to_keyvalue_loop_group:n }
          { \__cmd_arg_to_keyvalue_loop_space:w }
      }
        #1 \q__cmd_recursion_stop
  }
\cs_new_protected:Npn \__cmd_arg_to_keyvalue_loop_group:n #1
  { \__cmd_arg_to_keyvalue_loop:w }
\use:n { \cs_new_protected:Npn \__cmd_arg_to_keyvalue_loop_space:w } ~
  { \__cmd_arg_to_keyvalue_loop:w }
\cs_new_protected:Npn \__cmd_arg_to_keyvalue_loop_N_type:N #1
  {
    \__cmd_if_recursion_tail_stop_do:Nn #1
      { \__cmd_arg_to_keyvalue_set_default:nn }
    \str_if_eq:nnTF {#1} { = }
      {
        \__cmd_use_i_delimit_by_q_recursion_stop:nw
          { \__cmd_arg_to_keyvalue_set_keyvalue:nn }
      }
      {
        \bool_lazy_or:nnTF
            { \token_if_math_toggle_p:N #1 }
            { \str_if_eq_p:nn {#1} { \( } }
          { \__cmd_arg_to_keyvalue_math:w }
          { \__cmd_arg_to_keyvalue_loop:w }
      }
  }
\cs_new_protected:Npn \__cmd_arg_to_keyvalue_math:w #1 \q__cmd_recursion_stop
  {
    \tl_if_head_is_N_type:nTF {#1}
      { \__cmd_arg_to_keyvalue_math_N_type:N }
      {
        \tl_if_head_is_group:nTF {#1}
          { \__cmd_arg_to_keyvalue_math_group:n }
          { \__cmd_arg_to_keyvalue_math_space:w }
      }
        #1 \q__cmd_recursion_stop
  }
\cs_new_protected:Npn \__cmd_arg_to_keyvalue_math_N_type:N #1
  {
    \__cmd_if_recursion_tail_stop_do:Nn #1
      { \__cmd_arg_to_keyvalue_set_default:nn }
    \bool_lazy_or:nnTF
```

```
        { \token_if_math_toggle_p:N #1 }
        { \str_if_eq_p:nn {#1} { \) } } }
      { \__cmd_arg_to_keyvalue_loop:w }
      { \__cmd_arg_to_keyvalue_math:w }
  }
\cs_new_protected:Npn \__cmd_arg_to_keyvalue_math_group:n #1
  { \__cmd_arg_to_keyvalue_math:w }
\use:n { \cs_new_protected:Npn \__cmd_arg_to_keyvalue_math_space:w } ~
  { \__cmd_arg_to_keyvalue_math:w }
\cs_new_protected:Npn \__cmd_arg_to_keyvalue_set_default:nn #1#2
  { \tl_set:Nn \ProcessedArgument { #1 = {#2} } }
\cs_new_protected:Npn \__cmd_arg_to_keyvalue_set_keyvalue:nn #1#2
  { \tl_set:Nn \ProcessedArgument {#2} }
```

A utility to allow us to grab the first N-type token without risking brace stripping the rest of the input.

```
\cs_new:Npn \__cmd_split_N_head_apply:Nn #1#2
  { \exp:w \if_false: { \fi: \__cmd_split_N_head_apply_aux:NNw #1#2 } }
\cs_new:Npn \__cmd_split_N_head_apply_aux:NNw #1#2
  {
    \exp_after:wN \exp_end:
    \exp_after:wN #1 \exp_after:wN #2 \exp_after:wN { \if_false: } \fi:
  }

```

(*End of definition for* `\__cmd_arg_to_keyvalue:nn` *and others.*)

1.31 Utilities

`\__cmd_check_definable:nNT`
`\__cmd_check_definable_aux:nN`

Check that a token list is appropriate as a first argument of `\NewDocumentCommand` and similar functions and otherwise produce an error. First trim whitespace to allow for spaces around the actual command to be defined. If the result has multiple tokens, it is not a valid argument. The single token is a control sequence exactly if its string representation has more than one character (using `\token_to_str:N` rather than `\tl_to_str:n` to avoid problems with macro parameter characters, and setting `\tex_escapechar:D` to prevent it from being non-printable). Finally, check for an active character: this is done by lowercasing the token to fix its character code (arbitrarily to that of `?`) and comparing the result to an active `?`. Both control sequences and active characters are valid arguments, and non-active character tokens are not. In all cases, the group opened to keep assignments local must be closed.

```
\cs_new_protected:Npn \__cmd_check_definable:nNT #1
  { \tl_trim_spaces_apply:nN {#1} \__cmd_check_definable_aux:nN }
\group_begin:
  \char_set_catcode_active:n { `? }
  \cs_new_protected:Npn \__cmd_check_definable_aux:nN #1#2
    {
      \group_begin:
      \tl_if_single_token:nTF {#1}
        {
          \int_set:Nn \tex_escapechar:D { 92 }
          \exp_args:Ne \tl_if_empty:nTF
            { \exp_args:No \str_tail:n { \token_to_str:N #1 } }
            {
```

```
              \exp_args:Ne \char_set_lccode:nn
                { ‘ \str_head:n {#1} } { ‘? }
              \tex_lowercase:D { \tl_if_eq:nnTF {#1} } { ? }
                { \group_end: \use_iii:nnn }
                { \group_end: \use_i:nnn }
            }
            { \group_end: \use_iii:nnn }
        }
        { \group_end: \use_ii:nnn }
      {
        \msg_error:nnee { cmd } { not-definable }
          { \tl_to_str:n {#1} } { \token_to_str:N #2 }
      }
      {
        \msg_error:nnee { cmd } { not-one-token }
          { \tl_to_str:n {#1} } { \token_to_str:N #2 }
      }
    }
\group_end:
```

(*End of definition for* `\__cmd_check_definable:nNT` *and* `\__cmd_check_definable_aux:nN`.)

`\__cmd_token_if_cs:NTF` Based on the definition of `\__cmd_check_definable_aux:nN` above, but only checks for an actual control sequence (*i.e.*, \⟨*anything*⟩). `\tex_escapechar:D` is temporarily changed to a known value and then it checks if `\string#1` contains more than one character: if it does, it's a control sequence. This test differs from `\token_if_cs:NTF` for example in `\token_if_cs:NTF \c_group_begin_token {T}{F}`, where `\token_if_cs:NTF` returns false.

```
\cs_new_protected:Npn \__cmd_token_if_cs:NTF #1
  {
    \group_begin:
      \int_set:Nn \tex_escapechar:D { 92 }
      \exp_args:Ne \tl_if_empty:nTF
          { \exp_args:No \str_tail:n { \token_to_str:N #1 } }
        { \group_end: \use_ii:nn }
        { \group_end: \use_i:nn }
  }
```

(*End of definition for* `\__cmd_token_if_cs:NTF`.)

`\__cmd_tl_mapthread_function:NNN` Analogue of `\seq_mapthread_function:NNN` for token lists.

`\__cmd_tl_mapthread_function:nnN`

`\__cmd_tl_mapthread_loop:w`

```
\cs_new:Npn \__cmd_tl_mapthread_function:NNN #1#2#3
  {
    \exp_after:wN \exp_after:wN
    \exp_after:wN \__cmd_tl_mapthread_loop:w
    \exp_after:wN \exp_after:wN
    \exp_after:wN #3
    \exp_after:wN #1
    \exp_after:wN \q_recursion_tail
    \exp_after:wN \q_mark
    #2
    \q_recursion_tail
    \q_recursion_stop
  }
```

```
\cs_new:Npn \__cmd_tl_mapthread_function:nnN #1#2#3
  {
    \__cmd_tl_mapthread_loop:w #3
      #1 \q_recursion_tail \q_mark
      #2 \q_recursion_tail \q_recursion_stop
  }
\cs_new:Npn \__cmd_tl_mapthread_loop:w #1#2#3 \q_mark #4
  {
    \quark_if_recursion_tail_stop:n {#2}
    \quark_if_recursion_tail_stop:n {#4}
    #1 {#2} {#4}
    \__cmd_tl_mapthread_loop:w #1#3 \q_mark
  }
```

(*End of definition for* `\__cmd_tl_mapthread_function:NNN`, `\__cmd_tl_mapthread_function:nnN`, *and* `\__cmd_tl_mapthread_loop:w`.)

`\__kernel_cmd_if_xparse:NTF`
`\__cmd_cmd_type_cases:NnnnnnF`
`\__cmd_cmd_if_xparse_aux:N`

To determine whether the command is an xparse command check that its `arg_spec` is empty (this also excludes non-macros) and that its `replacement_spec` starts with either `\__cmd_start:nNNnnn` (non-expandable command) or `\__cmd_start_expandable:nNNNNn` (expandable command) or `\__cmd_start_optimized:` (optimized command) or `\__cmd_start_env:nnnnn` (environment) or `\environment #1 end aux` (environment end).

This conditional is needed in several kernel modules and is therefore has a kernel-internal name.

```
\cs_new:Npn \__cmd_cmd_type_cases:NnnnnnF #1 #2 #3 #4 #5 #6 #7
  {
    \exp_args:Ne \str_case_e:nnF
      {
        \exp_args:Nf \tl_if_empty:nT { \__kernel_cs_parameter_spec:N #1 }
          { \exp_not:N \exp_not:n { \exp_not:e { \tl_head:N #1 } } }
      }
      {
        { \exp_not:N \__cmd_start:nNNnnn } {#2}
        { \exp_not:N \__cmd_start_expandable:nNNNNn } {#3}
        { \exp_not:N \__cmd_start_optimized: } {#4}
        { \exp_not:N \__cmd_start_env:nnnnn } {#5}
        {
          \exp_after:wN \exp_not:N
            \cs:w environment~
              \exp_last_unbraced:Ne \use_none:nnn
                { \cs_to_str:N #1 } ~end~aux \cs_end:
        } {#6}
      }
      {#7}
  }
\cs_new:Npn \__kernel_cmd_if_xparse:NTF #1
  {
    \__cmd_cmd_type_cases:NnnnnnF #1
      { } { } { } { } { } { \use_iii:nnn }
    \use_i:nn
  }
```

(*End of definition for* `\__kernel_cmd_if_xparse:NTF`, `\__cmd_cmd_type_cases:NnnnnnF`, *and* `\__cmd_cmd_if_xparse_aux:N`.)

__cmd_peek_nonspace:NTF
__cmd_peek_nonspace_remove:NTF
__cmd_peek_nonspace_aux:nNNTF

Collect spaces in a loop, and put the collected spaces back in the false branch of a call to \peek_meaning:NTF or \peek_meaning_remove:NTF.

```
\cs_new_protected:Npn \__cmd_peek_nonspace:NTF
  { \__cmd_peek_nonspace_aux:nNNTF { } \__cmd_peek_meaning:NTF }
\cs_new_protected:Npn \__cmd_peek_nonspace_remove:NTF
  { \__cmd_peek_nonspace_aux:nNNTF { } \__cmd_peek_meaning_remove:NTF }
\cs_new_protected:Npn \__cmd_peek_nonspace_aux:nNNTF #1#2#3#4#5
  {
    \peek_meaning_remove:NTF \c_space_token
      { \__cmd_peek_nonspace_aux:nNNTF { #1 ~ } #2 #3 {#4} {#5} }
      { #2 #3 { #4 } { #5 #1 } }
  }
```

(*End of definition for* __cmd_peek_nonspace:NTF, __cmd_peek_nonspace_remove:NTF, *and* __cmd_peek_nonspace_aux:nNNTF.)

__cmd_peek_meaning:NTF
__cmd_peek_meaning_remove:NTF
__cmd_peek_cs_check_equal:NNN
__cmd_peek_meaning_aux:NNTF
__cmd_peek_true_remove:NNw

Peek ahead for a token with a given meaning. In case the search token is a control sequence, also check that the ⟨*csname*⟩ is the same as the control sequence peeked at. This extra verification is necessary when the command is delimited by control sequence tokens (as opposed to character tokens), and we want the exact same control sequence to match.

```
\cs_new_protected:Npn \__cmd_peek_meaning:NTF
  { \__cmd_peek_meaning_aux:NNTF \c_false_bool }
\cs_new_protected:Npn \__cmd_peek_meaning_remove:NTF
  { \__cmd_peek_meaning_aux:NNTF \c_true_bool }
\cs_new_protected:Npn \__cmd_peek_meaning_aux:NNTF #1#2#3#4
  {
    \tl_set:Nn \l__cmd_tmpa_tl {#3}
    \tl_set:Nn \l__cmd_tmpb_tl {#4}
    \peek_meaning:NTF #2
      {
        \token_if_eq_meaning:NNTF #2 \c_group_begin_token
          { \__cmd_peek_true_remove:Nw #1 }
          {
            \__cmd_token_if_cs:NTF #2
              { \__cmd_peek_cs_check_equal:NNN #1 #2 }
              { \__cmd_peek_true_remove:Nw #1 }
          }
      }
      { \l__cmd_tmpb_tl }
  }
\cs_new_protected:Npn \__cmd_peek_cs_check_equal:NNN #1#2#3
  {
    \str_if_eq:nnTF {#2} {#3}
      { \__cmd_peek_true_remove:Nw #1 }
      { \l__cmd_tmpb_tl }
    #3
  }
\cs_new_protected:Npn \__cmd_peek_true_remove:Nw #1
  {
    \bool_if:NTF #1
      {
        \tex_afterassignment:D \l__cmd_tmpa_tl
        \cs_set_eq:NN \__cmd_tmp:w
```

```
      }
      { \l__cmd_tmpa_tl }
  }
```

(*End of definition for* `\__cmd_peek_meaning:NTF` *and others.*)

1.32 Access to the argument specification

`\cmd_arg_spec:N`
`\cmd_arg_spec:c`
`\__cmd_arg_spec_opt:N`

First we check that the command passed does exist, and is one with the correct form. It if does then there are two cases. For optimised commands, we can reconstruct the arg. spec. from the TEX parameter spec, taking care to add `+` if required. For the non-optimized cases, the arg. spec. is stored in the top-level macro. We treat this as a `tl` and extract the appropriate balanced text: the second item.

```
\cs_new:Npn \cmd_arg_spec:N #1
  {
    \cs_if_exist:NTF #1
      {
        \__kernel_cmd_if_xparse:NTF #1
          {
            \exp_args:No \tl_if_head_eq_meaning:nNTF #1
              \__cmd_start_optimized:
              {
                \exp_args:Nc \__cmd_arg_spec_opt:N
                  { \cs_to_str:N #1 \c_space_tl code }
              }
              { \tl_item:Nn #1 { 2 } }
          }
          { X }
      }
      { X }
  }
\cs_generate_variant:Nn \cmd_arg_spec:N { c }
\cs_new:Npn \__cmd_arg_spec_opt:N #1
  {
    \prg_replicate:nn { \str_count:e { \cs_parameter_spec:N #1 } / 2 }
      {
        \bool_lazy_or:nnT
          { \token_if_long_macro_p:N #1 }
          { \token_if_protected_long_macro_p:N #1 }
          { + }
        m
      }
  }
```

(*End of definition for* `\cmd_arg_spec:N` *and* `\__cmd_arg_spec_opt:N`.)

1.33 Messages

`\c__cmd_ignore_def_tl`

```
\tl_const:Nn \c__cmd_ignore_def_tl
  { \\ \\ LaTeX~will~ignore~this~entire~definition. }
```

__cmd_environment_or_command: Two texts used in several messages.

```
\cs_new:Npn \__cmd_environment_or_command:
  {
    \bool_if:NTF \l__cmd_environment_bool
      { environment ~ ' \l__cmd_environment_str ' }
      {
        command ~
          ' \c_backslash_str \tl_to_str:N \l__cmd_function_tl '
      }
  }
```

(*End of definition for* `\__cmd_environment_or_command:`.)

Some messages intended as errors when defining commands/environments.

```
\msg_new:nnnn { cmd } { arg-after-body }
  { Argument~type~'#1'~must~be~last~in~#2. }
  {
    The~'#1'~argument~type~must~come~last~but~it~is~followed~
    by~'#3'~in~the~argument~specification.~This~is~not~allowed.
    \c__cmd_ignore_def_tl
  }
\msg_new:nnnn { cmd } { bad-arg-spec }
  { Bad~argument~specification~'#2'~for~#1. }
  {
    The~argument~specification~provided~is~not~valid:~
    one~or~more~mandatory~parts~are~missing.
    \c__cmd_ignore_def_tl
  }
\msg_new:nnnn { cmd } { already-defined }
  { Command~'#1'~already~defined. }
  {
    You~have~used~#2~
    with~a~command~that~already~has~a~definition. \\ \\
    The~existing~definition~of~'#1'~will~not~be~altered.
  }
\msg_new:nnnn { cmd } { undefined }
  { Command ~'#1'~undefined. }
  {
    You~have~used~#2~
    with~a~command~that~was~never~defined.
    \c__cmd_ignore_def_tl
  }

\msg_new:nnnn { cmd } { chars-dropped-first-line }
  { Characters~'#1'~dropped~on~first~line~of~#2~environment. }
  {
    LaTeX~was~collecting~a~verbatim-like~environment,~and~the~characters~
    '#1'~were~found~after~'\begin{#2}~on~the~first~line:~this~is~not~supported.
  }
\msg_new:nnnn { cmd } { chars-dropped-last-line }
  { Characters~'#1'~dropped~after~end~of~#2~environment. }
  {
    LaTeX~was~collecting~a~verbatim-like~environment,~and~the~characters~
    '#1'~were~found~after~\end{#2}~on~the~last~line:~this~is~not~supported.
  }
```

```
\msg_new:nnnn { cmd } { env-already-defined }
  { Environment~'#1'~already~defined. }
  {
    You~have~used~\NewDocumentEnvironment
    with~an~environment~that~already~has~a~definition. \\ \\
    The~existing~definition~of~'#1'~will~not~be~altered.
  }
\msg_new:nnnn { cmd } { env-end-already-defined }
  { End~of~environment~'#1'~already~defined. }
  {
    You~have~used~\NewDocumentEnvironment
    with~an~environment~that~already~has~a~definition~for~'end#1'. \\ \\
    The~existing~definition~of~'#1'~will~not~be~altered.
  }
\msg_new:nnnn { cmd } { env-undefined }
  { Environment~'#1'~undefined. }
  {
    You~have~used~\RenewDocumentEnvironment
    with~an~environment~that~was~never~defined.
    \c__cmd_ignore_def_tl
  }
\msg_new:nnnn { cmd } { expandable-ending-optional }
  { Bad~argument~specification~'#2'~for~#1. }
  {
    Expandable~commands~must~have~a~final~mandatory~argument~
    (or~no~arguments~at~all).~You~cannot~have~a~terminal~optional~
    argument~with~expandable~commands.
  }
\msg_new:nnnn { cmd } { long-short-mix }
  { Invalid~argument~prefix~'+'~in~command~'#1'. }
  {
    The~arguments~for~an~expandable~command~must~not~involve~short~
    arguments~after~long~arguments.~You~have~tried~to~mix~the~two~types~
    when~defining~'#1'.
  }
\msg_new:nnnn { cmd } { invalid-command-arg }
  { Invalid~argument~type~'#2'~in~#1. }
  {
    The~letter~'#2'~can~only~be~used~in~environment~argument~
    specifications,~but~not~for~commands.
    \\ \\
    LaTeX~will~ignore~the~entire~definition.
  }
\msg_new:nnnn { cmd } { invalid-expandable-arg }
  { Invalid~argument~type~'#2'~in~#1. }
  {
    The~letter~'#2'~specifies~an~argument~type~which~cannot~be~used~
    in~an~expandable~command.
    \c__cmd_ignore_def_tl
  }
\msg_new:nnnn { cmd } { invalid-after-optional-expandably }
  { Argument~'#2'~invalid~after~optional~arg~in~#1. }
  {
    The~letter~'#2'~specifies~an~argument~type~which~cannot~be~used~
```

```
    in~an~expandable~command~after~an~optional~argument.
    \c__cmd_ignore_def_tl
  }
\msg_new:nnnn { cmd } { invalid-bang }
  { Invalid~argument~prefix~'!'~in~#1. }
  {
    The~prefix~'!'~is~only~allowed~for~trailing~optional~arguments.~
    You~tried~to~apply~it~to~#2.
    \c__cmd_ignore_def_tl
  }
\msg_new:nnnn { cmd } { not-definable }
  { First~argument~of~'#2'~must~be~a~command. }
  {
    The~first~argument~of~'#2'~should~be~the~document~command~that~will~
    be~defined.~The~provided~argument~'#1'~is~a~character.~Perhaps~a~
    backslash~is~missing?
    \c__cmd_ignore_def_tl
  }
\msg_new:nnnn { cmd } { not-one-token }
  { First~argument~of~'#2'~must~be~a~command. }
  {
    The~first~argument~of~'#2'~should~be~the~document~command~that~will~
    be~defined.~The~provided~argument~'#1'~contains~more~than~one~
    token.~Perhaps~a~backslash~is~missing?
    \c__cmd_ignore_def_tl
  }
\msg_new:nnnn { cmd } { not-single-token }
  { Argument~delimiter~'#2'~invalid~in~#1. }
  {
    The~argument~specification~contains~
    \tl_if_empty:nTF{#2}{nothing}{'#2'}~
    in~a~place~
    where~a~single~token~is~required.
    \c__cmd_ignore_def_tl
  }
\msg_new:nnnn { cmd } { forbidden-group-token }
  { Argument~delimiter~'#2'~invalid~in~#1. }
  {
    The~argument~specification~contains~the~implicit~
    #3-group~token~'#2'~which~is~not~allowed~as~an~argument~delimiter.
    \c__cmd_ignore_def_tl
  }
\msg_new:nnnn { cmd } { processor-in-expandable }
  { Invalid~argument~prefix~'>'~in~command~'#1'. }
  {
    The~argument~specification~for~'#1'~contains~the~processor~function~'>{#2}'.~
    This~is~only~supported~for~robust~commands,~but~not~for~expandable~ones.
    \c__cmd_ignore_def_tl
  }
\msg_new:nnnn { cmd } { keyval-in-expandable }
  { Invalid~argument~prefix~'='~in~command~'#1'. }
  {
    The~argument~specification~for~'#1'~contains~a~key--value~marker~'={#2}'.~
    This~is~only~supported~for~robust~commands,~but~not~for~expandable~ones.
```

```
    \c__cmd_ignore_def_tl
  }
\msg_new:nnnn { cmd } { too-many-args }
  { Too~many~arguments~for~#1. }
  {
    The~argument~specification~'#2'~asks~for~more~than~9~arguments.~
    This~cannot~be~implemented.
    \c__cmd_ignore_def_tl
  }
\msg_new:nnnn { cmd } { two-markers }
  { Invalid~argument~prefix~'#2'~in~#1. }
  {
    The~argument~specification~provided~for~#1~has~two~'#2'~markers~applied~
    to~the~same~argument;~one~is~redundant.
  }
\msg_new:nnnn { cmd } { unknown-argument-type } % should be unkown-arg-type but dep in xparse
  { Invalid~argument~type~'#2'~in~#1. }
  {
    The~letter~'#2'~does~not~specify~a~known~argument~type.
    \c__cmd_ignore_def_tl
  }
\msg_new:nnnn { cmd } { xparse-arg-type }
  { Invalid~argument~type~'#2'~in~#1~(requires~xparse). }
  {
    The~letter~'#2'~specifies~a~known~but~deprecated~argument~type.~
    If~you~really~need~it~you~have~to~load~the~xparse~package.
    \c__cmd_ignore_def_tl
  }
```

Errors when using commands/environments. The `if-boolean` message is always used in expandable errors. The `default-loop` and `missing-required` messages can be expandable or not expandable.

```
\msg_new:nnn { cmd } { if-boolean }
  { Invalid~argument~{#1}~to~\iow_char:N\\IfBoolean... }
\msg_new:nnnn { cmd } { default-loop }
  { Circular~dependency~in~defaults~of~#1. }
  {
    The~default~values~of~two~or~more~arguments~of~the~#1~
    depend~on~each~other~in~a~way~that~cannot~be~resolved.
  }
\msg_new:nnnn { cmd } { missing-required }
  { Required~argument~missing~for~#1. }
  {
    The~#1~expects~one~of~its~arguments~to~start~with~'#2'.~
    LaTeX~did~not~find~this~argument~and~will~insert~a~default~value~
    for~further~processing.
  }
\msg_new:nnnn { cmd } { arg-split }
  { Too~many~'#1'~separators~in~argument. }
  {
    LaTeX~was~asked~to~split~the~input~'#3'~
    at~each~occurrence~of~the~separator~'#1'~into~#2~parts.~
    Too~many~separators~were~found.
  }
```

```
\msg_new:nnnn { cmd } { verbatim-nl }
  { Verbatim-like~#1~ended~by~end~of~line. }
  {
    The~verbatim~argument~of~the~#1~cannot~contain~more~than~one~line,~
    but~the~end~
    of~the~current~line~has~been~reached.~You~may~have~forgotten~the~
    closing~delimiter.
    \\ \\
    LaTeX~will~ignore~'#2'~and~you~may~get~some~additional~
    (low-level)~errors.
  }
\msg_new:nnnn { cmd } { verbatim-tokenized }
  { Verbatim-like~#1~illegal~in~argument. }
  {
    The~#1~takes~a~verbatim~argument~and~should~therefore~normally~
    not~be~used~in~arguments~of~other~commands~or~environments.~
    LaTeX~found~an~illegal~token~ \tl_if_empty:nF {#3} { (#3)~ }
    after~'#2'~and~will~drop~everything~up~to~this~point.
    \\ \\
    Expect~further~(low-level)~errors.
  }
```

Intended more for information.

```
\msg_new:nnn { cmd } { define-command }    % should be just 'define' but dep in xparse
  {
    Defining~command~#1~
    with~sig.~'#2'~\msg_line_context:.
  }
\msg_new:nnn { cmd } { define-env }
  {
    Defining~environment~'#1'~
    with~sig.~'#2'~\msg_line_context:.
  }
\msg_new:nnn { cmd } { redefine }
  {
    Redefining~command~#1~
    with~sig.~'#2'~\msg_line_context:.
  }
\msg_new:nnn { cmd } { redefine-env }
  {
    Redefining~environment~'#1'~
    with~sig.~'#2'~\msg_line_context:.
  }
\msg_new:nnn { cmd } { optional-mandatory }
  {
    Optional~and~mandatory~argument~with~same~delimiter~'#2'.
    \\ \\
    The~mandatory~argument~specified~with~
    '\str_case:nnF{#1}{ {R/r}{r'~or~'R} }{#1}'~has~the~
    same~delimiter~'#2'~as~an~earlier~optional~argument.~
    It~will~therefore~not~be~possible~to~omit~all~the~earlier~
    optional~arguments~when~calling~this~command.
    \\ \\
    This~may~be~intentional,~but~then~it~might~be~a~mistake.
  }
```

```
\msg_new:nnn { cmd } { unsupported-let }
  {
    The~command~'#1'~was~undefined~but~not~the~associated~commands~
    '#1~code'~and/or~'#1~defaults'.~Maybe~you~tried~using~
    \iow_char:N\\let.~This~may~lead~to~an~infinite~loop.
  }
```

1.34 User functions

The user functions are more or less just the internal functions renamed.

\BooleanFalse
\BooleanTrue

Design-space names for the Boolean values.

```
\cs_new_eq:NN \BooleanFalse \c_false_bool
\cs_new_eq:NN \BooleanTrue  \c_true_bool
```

(*End of definition for* `\BooleanFalse` *and* `\BooleanTrue`*. These functions are documented on page* *121*.)

\NewDocumentCommand
\RenewDocumentCommand
\ProvideDocumentCommand
\DeclareDocumentCommand

The user macros are pretty simple wrappers around the internal ones. There is however a check that the first argument is a single token, possibly surrounded by spaces (hence the strange `\use:nnn`), and is definable.

```
\cs_new_protected:Npn \NewDocumentCommand #1#2#3
  {
    \__cmd_check_definable:nNT {#1} \NewDocumentCommand
      {
        \cs_if_exist:NTF #1
          {
            \msg_error:nnee { cmd } { already-defined }
              { \use:nnn \token_to_str:N #1 { } }
              { \token_to_str:N \NewDocumentCommand }
          }
          { \__cmd_declare_cmd:Nnn #1 {#2} {#3} }
      }
  }
\cs_new_protected:Npn \RenewDocumentCommand #1#2#3
  {
    \__cmd_check_definable:nNT {#1} \RenewDocumentCommand
      {
        \cs_if_exist:NTF #1
          { \__cmd_declare_cmd:Nnn #1 {#2} {#3} }
          {
            \msg_error:nnee { cmd } { undefined }
              { \use:nnn \token_to_str:N #1 { } }
              { \token_to_str:N \RenewDocumentCommand }
          }
      }
  }
\cs_new_protected:Npn \ProvideDocumentCommand #1#2#3
  {
    \__cmd_check_definable:nNT {#1} \ProvideDocumentCommand
      { \cs_if_exist:NF #1 { \__cmd_declare_cmd:Nnn #1 {#2} {#3} } }
  }
\cs_new_protected:Npn \DeclareDocumentCommand #1#2#3
  {
```

```
    \__cmd_check_definable:nNT {#1} \DeclareDocumentCommand
      { \__cmd_declare_cmd:Nnn #1 {#2} {#3} }
  }
```

(*End of definition for* `\NewDocumentCommand` *and others. These functions are documented on page 117.*)

`\NewDocumentEnvironment`
`\RenewDocumentEnvironment`
`\ProvideDocumentEnvironment`
`\DeclareDocumentEnvironment`
`\__cmd_new_env:nnnn`
`\__cmd_renew_env:nnnn`
`\__cmd_provide_env:nnnn`
`\__cmd_new_env:ennn`
`\__cmd_renew_env:ennn`
`\__cmd_provide_env:ennn`

Very similar for environments. Trim spaces from user-specified ⟨*envname*⟩, do existence check then hand off to `\__cmd_declare_env:nnnn`.

```
⟨latexrelease⟩\IncludeInRelease{2024/11/01}{\NewDocumentEnvironment}%
⟨latexrelease⟩                          {Trim~spaces~from~envname~first}
\cs_new_protected:Npn \NewDocumentEnvironment #1#2#3#4
  {
    \__cmd_new_env:ennn { \tl_trim_spaces:e {#1} } {#2} {#3} {#4}
  }
\cs_new_protected:Npn \RenewDocumentEnvironment #1#2#3#4
  {
    \__cmd_renew_env:ennn { \tl_trim_spaces:e {#1} } {#2} {#3} {#4}
  }
\cs_new_protected:Npn \ProvideDocumentEnvironment #1#2#3#4
  {
    \__cmd_provide_env:ennn { \tl_trim_spaces:e {#1} } {#2} {#3} {#4}
  }
\cs_new_protected:Npn \DeclareDocumentEnvironment #1#2#3#4
  {
    \__cmd_declare_env:ennn { \tl_trim_spaces:e {#1} } {#2} {#3} {#4}
  }
```

Each of `\__cmd_(new|renew|provide)_env:nnnn` is curried.

```
\cs_new_protected:Npn \__cmd_new_env:nnnn #1
  {
    \cs_if_exist:cTF {#1}
      {
        \msg_error:nne { cmd } { env-already-defined } {#1}
        \use_none:nnn
      }
      {
        \cs_if_exist:cTF { end #1 }
          {
            \msg_error:nne { cmd } { env-end-already-defined } {#1}
            \use_none:nnn
          }
          { \__cmd_declare_env:nnnn {#1} }
      }
  }
\cs_new_protected:Npn \__cmd_renew_env:nnnn #1
  {
    \cs_if_exist:cTF {#1}
      { \__cmd_declare_env:nnnn {#1} }
      {
        \msg_error:nne { cmd } { env-undefined } {#1}
        \use_none:nnn
      }
  }
\cs_new_protected:Npn \__cmd_provide_env:nnnn #1
  {
```

```
    \cs_if_exist:cTF {#1}
      { \use_none:nnn }
      { \__cmd_declare_env:nnnn {#1} }
  }
\cs_generate_variant:Nn \__cmd_new_env:nnnn { e }
\cs_generate_variant:Nn \__cmd_renew_env:nnnn { e }
\cs_generate_variant:Nn \__cmd_provide_env:nnnn { e }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2024/06/01}{\NewDocumentEnvironment}%
⟨latexrelease⟩                  {Trim~spaces~from~envname~first}
⟨latexrelease⟩\cs_new_protected:Npn \NewDocumentEnvironment #1#2#3#4
⟨latexrelease⟩  {
⟨latexrelease⟩    \cs_if_exist:cTF {#1}
⟨latexrelease⟩      { \msg_error:nnx { cmd } { env-already-defined } {#1} }
⟨latexrelease⟩      {
⟨latexrelease⟩        \cs_if_exist:cTF { end #1 }
⟨latexrelease⟩          { \msg_error:nnx { cmd } { env-end-already-defined } {#1} }
⟨latexrelease⟩          { \__cmd_declare_env:nnnn {#1} {#2} {#3} {#4} }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new_protected:Npn \RenewDocumentEnvironment #1#2#3#4
⟨latexrelease⟩  {
⟨latexrelease⟩    \cs_if_exist:cTF {#1}
⟨latexrelease⟩      { \__cmd_declare_env:nnnn {#1} {#2} {#3} {#4} }
⟨latexrelease⟩      { \msg_error:nnx { cmd } { env-undefined } {#1} }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new_protected:Npn \ProvideDocumentEnvironment #1#2#3#4
⟨latexrelease⟩  { \cs_if_exist:cF {#1} { \__cmd_declare_env:nnnn {#1} {#2} {#3} {#4} } }
⟨latexrelease⟩\cs_new_protected:Npn \DeclareDocumentEnvironment #1#2#3#4
⟨latexrelease⟩  { \__cmd_declare_env:nnnn {#1} {#2} {#3} {#4} }
⟨latexrelease⟩\cs_undefine:N \__cmd_new_env:nnnn
⟨latexrelease⟩\cs_undefine:N \__cmd_new_env:ennn
⟨latexrelease⟩\cs_undefine:N \__cmd_renew_env:nnnn
⟨latexrelease⟩\cs_undefine:N \__cmd_renew_env:ennn
⟨latexrelease⟩\cs_undefine:N \__cmd_provide_env:nnnn
⟨latexrelease⟩\cs_undefine:N \__cmd_provide_env:ennn
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\NewDocumentEnvironment` *and others. These functions are documented on page* 117.)

`\NewExpandableDocumentCommand`
`\RenewExpandableDocumentCommand`
`\ProvideExpandableDocumentCommand`
`\DeclareExpandableDocumentCommand`

The expandable versions are essentially the same as the basic functions. The strange `\use:nnn` is there in case `#1` is surrounded with spaces, as can happen with usual document catcodes in `\RenewExpandableDocumentCommand { \! }` ...

```
\cs_new_protected:Npn \NewExpandableDocumentCommand #1#2#3
  {
    \__cmd_check_definable:nNT {#1} \NewExpandableDocumentCommand
      {
        \cs_if_exist:NTF #1
          {
            \msg_error:nnee { cmd } { already-defined }
              { \use:nnn \token_to_str:N #1 { } }
              { \token_to_str:N \NewExpandableDocumentCommand }
```

```
          }
          { \__cmd_declare_expandable_cmd:Nnn #1 {#2} {#3} }
      }
  }
\cs_new_protected:Npn \RenewExpandableDocumentCommand #1#2#3
  {
    \__cmd_check_definable:nNT {#1} \RenewExpandableDocumentCommand
      {
        \cs_if_exist:NTF #1
          { \__cmd_declare_expandable_cmd:Nnn #1 {#2} {#3} }
          {
            \msg_error:nnee { cmd } { undefined }
              { \use:nnn \token_to_str:N #1 { } }
              { \token_to_str:N \RenewExpandableDocumentCommand }
          }
      }
  }
\cs_new_protected:Npn \ProvideExpandableDocumentCommand #1#2#3
  {
    \__cmd_check_definable:nNT {#1} \ProvideExpandableDocumentCommand
      {
        \cs_if_exist:NF #1
          { \__cmd_declare_expandable_cmd:Nnn #1 {#2} {#3} }
      }
 }
\cs_new_protected:Npn \DeclareExpandableDocumentCommand #1#2#3
  {
    \__cmd_check_definable:nNT {#1} \DeclareExpandableDocumentCommand
      { \__cmd_declare_expandable_cmd:Nnn #1 {#2} {#3} }
  }
```

(*End of definition for* `\NewExpandableDocumentCommand` *and others. These functions are documented on page 125.*)

`\IfBooleanT` `\IfBooleanF` `\IfBooleanTF`

The logical ⟨*true*⟩ and ⟨*false*⟩ statements are just the normal `\c_true_bool` and `\c_false_bool` so `\bool_if:NTF` is almost enough. However, this code-level function blows up badly when passed invalid input. We want `\IfBooleanTF` to accept a single (non-space) token equal to `\c_true_bool` or `\c_false_bool`, possibly surrounded by spaces. If the input is blank or multiple items, jump to the error and pick the false branch. If the input, ignoring spaces (we do this by omitting braces in the `\tl_if_single_token:nF` test), is not a single token then jump to the error as well. It is then safe to compare the token to the two booleans, picking the appropriate branch. If neither matches, we jump to the error as well.

```
\cs_new:Npn \IfBooleanTF #1
  {
    \tl_if_single:nF {#1}
      { \prg_break:n { \use:n } }
    \tl_if_single_token:nF #1
      { \prg_break:n { \use:n } }
    \token_if_eq_meaning:NNT #1 \c_true_bool
      { \prg_break:n { \use_ii:nnn } }
    \token_if_eq_meaning:NNT #1 \c_false_bool
      { \prg_break:n { \use_iii:nnn } }
    \prg_break:n { \use:n }
```

```
    \prg_break_point:
    {
      \msg_expandable_error:nnn { cmd } { if-boolean } {#1}
      \use_ii:nn
    }
  }
\cs_new:Npn \IfBooleanT #1#2 { \IfBooleanTF {#1} {#2} { } }
\cs_new:Npn \IfBooleanF #1 { \IfBooleanTF {#1} { } }
```

(*End of definition for* `\IfBooleanT`, `\IfBooleanF`, *and* `\IfBooleanTF`. *These functions are documented on page 121.*)

We adjust here as currently there is still an older version of `\NoValue` in l3tl.

`\NoValue` A special marker; done using `gset` as there is also a version in expl3, and that has to be in line with what we do here (once everything is synced up, this one will be removed).

```
\cs_gset_protected:Npn \NoValue { -NoValue- }
```

(*End of definition for* `\NoValue`.)

`\c_novalue_tl`

```
\cs_undefine:N \c_novalue_tl
\tl_const:Nn \c_novalue_tl { \NoValue }
```

`\tl_if_novalue_p:n` `\tl_if_novalue:nTF` `\__cmd_if_novalue_aux:w` The first argument of `\__cmd_if_novalue_aux:w` is empty if and only if `#1` starts with `\NoValue`, while the second argument is empty if `#1` is exactly `\NoValue` or if it has a question mark just following the marker. In this second case, however, the material after the first `?!` remains and makes the emptiness test return `false`.

```
⟨@@=tl⟩
\prg_gset_conditional:Npnn \tl_if_novalue:n #1
  { p , T ,  F , TF }
  {
    \__tl_if_empty_if:o
      { \__tl_if_novalue_aux:w { } #1 { } ? ! \NoValue ? ? ! }
      \prg_return_true:
    \else:
      \prg_return_false:
    \fi:
  }
\cs_gset:Npn \__tl_if_novalue_aux:w #1 \NoValue #2 ? #3 ? ! { #1 #2 }
⟨@@=cmd⟩
```

(*End of definition for* `\tl_if_novalue:nTF` *and* `\__cmd_if_novalue_aux:w`.)

`\IfNoValueT` `\IfNoValueF` `\IfNoValueTF` Simple re-naming.

```
\cs_new_eq:NN \IfNoValueF  \tl_if_novalue:nF
\cs_new_eq:NN \IfNoValueT  \tl_if_novalue:nT
\cs_new_eq:NN \IfNoValueTF \tl_if_novalue:nTF
```

(*End of definition for* `\IfNoValueT`, `\IfNoValueF`, *and* `\IfNoValueTF`. *These functions are documented on page 119.*)

`\IfValueT` `\IfValueF` `\IfValueTF` Inverted logic.

```
\cs_new:Npn \IfValueF { \tl_if_novalue:nT }
\cs_new:Npn \IfValueT { \tl_if_novalue:nF }
\cs_new:Npn \IfValueTF #1#2#3 { \tl_if_novalue:nTF {#1} {#3} {#2} }
```

(*End of definition for* `\IfValueT`, `\IfValueF`, *and* `\IfValueTF`. *These functions are documented on page* 120.)

`\IfBlankT` `\IfBlankF` `\IfBlankTF` Another simple re-naming.

```
⟨latexrelease⟩\IncludeInRelease{2022/06/01}%
⟨latexrelease⟩                    {\IfBlankTF}{Testing~for~empty~or~blank}%
\cs_new_eq:NN \IfBlankF  \tl_if_blank:nF
\cs_new_eq:NN \IfBlankT  \tl_if_blank:nT
\cs_new_eq:NN \IfBlankTF \tl_if_blank:nTF
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/11/15}%
⟨latexrelease⟩                    {\IfBlankTF}{Testing~for~empty~or~blank}%
⟨latexrelease⟩\cs_undefine:N \IfBlankF
⟨latexrelease⟩\cs_undefine:N \IfBlankT
⟨latexrelease⟩\cs_undefine:N \IfBlankTF
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\IfBlankT`, `\IfBlankF`, *and* `\IfBlankTF`. *These functions are documented on page* 120.)

`\ProcessedArgument` Processed arguments are returned using this name, which is reserved here although the definition will change.

```
\tl_new:N \ProcessedArgument
```

(*End of definition for* `\ProcessedArgument`. *This function is documented on page* 129.)

`\ReverseBoolean` `\SplitArgument` `\SplitList` `\TrimSpaces` Simple copies.

```
\cs_new_eq:NN \ReverseBoolean \__cmd_bool_reverse:N
\cs_new_eq:NN \SplitArgument  \__cmd_split_argument:nnn
\cs_new_eq:NN \SplitList      \__cmd_split_list:nn
\cs_new_eq:NN \TrimSpaces     \__cmd_trim_spaces:n
```

(*End of definition for* `\ReverseBoolean` *and others. These functions are documented on page* 124.)

`\ProcessList` To support `\SplitList`.

```
\cs_new_eq:NN \ProcessList \tl_map_tokens:nn
```

(*End of definition for* `\ProcessList`. *This function is documented on page* 124.)

Finally as promised, restore `\__kernel_chk_if_free_cs:N`:

```
⟨latexrelease⟩\cs_gset_eq:NN \__kernel_chk_if_free_cs:N \__cmd_chk_if_free_cs:N
⟨latexrelease⟩\cs_undefine:N \__cmd_chk_if_free_cs:N
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{ltcmd}%
⟨latexrelease⟩                    {Document~command~parser}%
⟨latexrelease⟩
⟨latexrelease⟩\EndModuleRelease
\ExplSyntaxOff
```

Now in `latexrelease` mode, redefine `\NewDocumentCommand` to not complain on commands already defined.

```
⟨latexrelease⟩\@ifundefined{ExplSyntaxOff}{}{\latexrelease@postltcmd}
⟨latexrelease⟩\catcode`\^^@=\@latexrelease@catcode@null\relax
⟨/2ekernel | latexrelease⟩
```

We need to stop DocStrip treating `@@` in a special way at this point.

```
⟨@@=⟩
```

File 08

lthooks.dtx

1 Introduction

Hooks are points in the code of commands or environments where it is possible to add processing code into existing commands. This can be done by different packages that do not know about each other, and to allow for hopefully safe processing it is necessary to sort different chunks of code added by different packages into a suitable processing order.

This is done by the packages adding chunks of code (via `\AddToHook`) and labeling their code with some label by default using the package name as a label.

At `\begin{document}` all code for a hook is then sorted according to some rules (given by `\DeclareHookRule`) for fast execution without processing overhead. If the hook code is modified afterwards (or the rules are changed), a new version for fast processing is generated.

Some hooks are used already in the preamble of the document. If that happens then the hook is prepared for execution (and sorted) already at that point.

2 Package writer interface

The hook management system is offered as a set of CamelCase commands for traditional LaTeX 2$_\varepsilon$ packages (and for use in the document preamble if needed) as well as `expl3` commands for modern packages, that use the L3 programming layer of LaTeX. Behind the scenes, a single set of data structures is accessed so that packages from both worlds can coexist and access hooks in other packages.

2.1 LaTeX 2$_\varepsilon$ interfaces

2.1.1 Declaring hooks

With a few exceptions, hooks have to be declared before they can be used. The exceptions are the generic hooks for commands and environments (executed at `\begin` and `\end`), and the generic hooks run when loading files (see section 3.1).

`\NewHook` `\NewHook {`⟨*hook*⟩`}`

Creates a new ⟨*hook*⟩. If this hook is declared within a package it is suggested that its name is always structured as follows: ⟨*package-name*⟩`/`⟨*hook-name*⟩. If necessary you can further subdivide the name by adding more `/` parts. If a hook name is already taken, an error is raised and the hook is not created.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5. The string `??` can't be used as a hook name because it has a special significance as a placeholder in hook rules.

\NewReversedHook `\NewReversedHook {⟨hook⟩}`

Like `\NewHook` declares a new ⟨*hook*⟩. the difference is that the code chunks for this hook are in reverse order by default (those added last are executed first). Any rules for the hook are applied after the default ordering. See sections 2.3 and 2.4 for further details.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

\NewMirroredHookPair `\NewMirroredHookPair {⟨hook-1⟩} {⟨hook-2⟩}`

A shorthand for `\NewHook{`⟨*hook-1*⟩`}\NewReversedHook{`⟨*hook-2*⟩`}`.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

\NewHookWithArguments `\NewHookWithArguments {⟨hook⟩} {⟨number⟩}`

Creates a new ⟨*hook*⟩ whose code takes ⟨*number*⟩ arguments, and otherwise works exactly like `\NewHook`. Section 2.7 explains hooks with arguments.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

\NewReversedHookWithArguments `\NewReversedHookWithArguments {⟨hook⟩} {⟨number⟩}`

Like `\NewReversedHook`, but creates a hook whose code takes ⟨*number*⟩ arguments. Section 2.7 explains hooks with arguments.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

\NewMirroredHookPairWithArguments `\NewMirroredHookPairWithArguments {⟨hook-1⟩} {⟨hook-2⟩} {⟨number⟩}`

A shorthand for `\NewHookWithArguments{`⟨*hook-1*⟩`}{`⟨*number*⟩`}` `\NewReversedHookWithArguments{`⟨*hook-2*⟩`}{`⟨*number*⟩`}`. Section 2.7 explains hooks with arguments.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

2.1.2 Special declarations for generic hooks

The declarations here should normally not be used. They are available to provide support for special use cases mainly involving generic command hooks.

\DisableGenericHook `\DisableGenericHook {⟨hook⟩}`

After this declaration[3] the ⟨*hook*⟩ is no longer usable: Any further attempt to add code to it will result in an error and any use, e.g., via `\UseHook`, will simply do nothing.

This is intended to be used with generic command hooks (see `ltcmdhooks-doc`) as depending on the definition of the command such generic hooks may be unusable. If that is known, a package developer can disable such hooks up front.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

[3] In the 2020/06 release this command was called `\DisableHook`, but that name was misleading as it shouldn't be used to disable non-generic hooks.

\ActivateGenericHook `\ActivateGenericHook {⟨hook⟩}`

This declaration activates a generic hook provided by a package/class (e.g., one used in code with `\UseHook` or `\UseOneTimeHook`) without it being explicitly declared with `\NewHook`). If the hook is already activated, this command does nothing.

Note that this command does not undo the effect of `\DisableGenericHook`. See section 2.6 for a discussion of when this declaration is appropriate.

2.1.3 Using hooks in code

Using a hook that is executing the code that has been associated with it is only allowed if the hook has been previously declared with `\NewHook`. For performance reason there are no runtime checks for this and it is the responsibility of the programmer of a package to ensure that all hooks that are used in a package (with one of the commands in this section) are declared first.

\UseHook `\UseHook {⟨hook⟩}`

Execute the code stored in the ⟨*hook*⟩.

Before `\begin{document}` the fast execution code for a hook is not set up, so in order to use a hook there it is explicitly initialized first. As that involves assignments using a hook at those times is not 100% the same as using it after `\begin{document}`.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally.

\UseHookWithArguments `\UseHookWithArguments {⟨hook⟩} {⟨number⟩} {⟨arg1⟩} ... {⟨argn⟩}`

Execute the code stored in the ⟨*hook*⟩ and pass the arguments {⟨arg_1⟩} through {⟨arg_n⟩} to the ⟨*hook*⟩. Otherwise, it works exactly like `\UseHook`. The ⟨*number*⟩ should be the number of arguments declared for the hook. If the hook is not declared, this command does nothing and it will remove ⟨*number*⟩ items from the input. Section 2.7 explains hooks with arguments.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally.

\UseOneTimeHook `\UseOneTimeHook {⟨hook⟩}`

Some hooks are only used (and can be only used) in one place, for example, those in `\begin{document}` or `\end{document}`. From that point onwards, adding to the hook through a defined `\⟨addto-cmd⟩` command (e.g., `\AddToHook` or `\AtBeginDocument`, etc.) would have no effect (as would the use of such a command inside the hook code itself). It is therefore customary to redefine `\⟨addto-cmd⟩` to simply process its argument, i.e., essentially make it behave like `\@firstofone`.

`\UseOneTimeHook` does that: it records that the hook has been consumed and any further attempt to add to it will result in executing the code to be added immediately.

Using `\UseOneTimeHook` several times with the same {⟨*hook*⟩} means that it only executes the first time it is used. For example, if it is used in a command that can be called several times then the hook executes during only the *first* invocation of that command; this allows its use as an "initialization hook".

Mixing `\UseHook` and `\UseOneTimeHook` for the same {⟨*hook*⟩} should be avoided, but if this is done then neither will execute after the first `\UseOneTimeHook`.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally. See section 2.1.5 for details.

\UseOneTimeHookWithArguments `\UseOneTimeHookWithArguments {⟨hook⟩} {⟨number⟩} {⟨arg$_1$⟩} ... {⟨arg$_n$⟩}`

Works exactly like `\UseOneTimeHook`, but passes arguments {⟨arg$_1$⟩} through {⟨arg$_n$⟩} to the ⟨*hook*⟩. The ⟨*number*⟩ should be the number of arguments declared for the hook. If the hook is not declared, this command does nothing and it will remove ⟨*number*⟩ items from the input.

It should be noted that after a one-time hook is used, it is no longer possible to use `\AddToHookWithArguments` or similar with that hook. `\AddToHook` continues to work as normal. Section 2.7 explains hooks with arguments.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally. See section 2.1.5 for details.

2.1.4 Updating code for hooks

In contrast to the commands from the previous section, declarations such as `\AddToHook` or `\DeclareHookRule` can be used even when the hook is not yet declared. The rationale is that the hook declaration may be in some package that is loaded later, or perhaps not loaded at all.

A side effect of this design is that misspellings do not raise an error but are simply regarded as declarations for hooks with a different name.

\AddToHook `\AddToHook {⟨hook⟩} [⟨label⟩] {⟨code⟩}`

Adds ⟨*code*⟩ to the ⟨*hook*⟩ labeled by ⟨*label*⟩. When the optional argument ⟨*label*⟩ is not provided, the ⟨*default label*⟩ is used (see section 2.1.5). If `\AddToHook` is used in a package/class, the ⟨*default label*⟩ is the package/class name, otherwise it is `top-level` (the `top-level` label is treated differently: see section 2.1.6).

If there already exists code under the ⟨*label*⟩ then the new ⟨*code*⟩ is appended to the existing one (even if this is a reversed hook). If you want to replace existing code under the ⟨*label*⟩, first apply `\RemoveFromHook`.

The hook doesn't have to exist for code to be added to it. However, if it is not declared, then obviously the added ⟨*code*⟩ will never be executed. This allows for hooks to work regardless of package loading order and enables packages to add to hooks from other packages without worrying whether they are actually used in the current document. See section 2.1.8.

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

\AddToHookWithArguments `\AddToHookWithArguments {⟨hook⟩} [⟨label⟩] {⟨code⟩}`

Works exactly like `\AddToHook`, except that the ⟨*code*⟩ can access the arguments passed to the hook using `#1`, `#2`, ..., `#n` (up to the number of arguments declared for the hook). If the ⟨*code*⟩ should contain *parameter tokens* (`#`) that are not supposed to be understood as the arguments of the hook, such tokens should be doubled. For example, with `\AddToHook` one can write:

```
\AddToHook{myhook}{\def\foo#1{Hello, #1!}}
```

but to achieve the same with `\AddToHookWithArguments`, one should write:

```
\AddToHookWithArguments{myhook}{\def\foo##1{Hello, ##1!}}
```

because in the latter case, `#1` refers to the first argument of the hook `myhook`. Section 2.7 explains hooks with arguments.

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

\RemoveFromHook `\RemoveFromHook {⟨hook⟩} [⟨label⟩]`

Removes any code labeled by ⟨*label*⟩ from the ⟨*hook*⟩. When the optional argument ⟨*label*⟩ is not provided, the ⟨*default label*⟩ is used (see section 2.1.5).

If there is no code under the ⟨*label*⟩ in the ⟨*hook*⟩, or if the ⟨*hook*⟩ does not exist, a warning is issued when you attempt to `\RemoveFromHook`, and the command is ignored. `\RemoveFromHook` should be used only when you know exactly what labels are in a hook. Typically this will be when some code gets added to a hook by a package, then later this code is removed by that same package. If you want to prevent the execution of code from another package, use the `voids` rule instead (see section 2.1.7).

Important: *The* `\RemoveFromHook` *command should be only used if one has full control over the code chunk to be removed. In particular it should not be used to remove code chunks from other packages! For this the* `voids` *relation is provided.*

If the optional ⟨*label*⟩ argument is `*`, then all code chunks are removed. This is rather dangerous as it may well drop code from other packages (that one may not know about); it should therefore not be used in packages but only in document preambles!

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

In contrast to the `voids` relationship between two labels in a `\DeclareHookRule` this is a destructive operation as the labeled code is removed from the hook data structure, whereas the relationship setting can be undone by providing a different relationship later.

A useful application for this declaration inside the document body is when one wants to temporarily add code to hooks and later remove it again, e.g.,

```
\AddToHook{env/quote/begin}{\small}
\begin{quote}
  A quote set in a smaller typeface
\end{quote}
...
\RemoveFromHook{env/quote/begin}
... now back to normal for further quotes
```

Note that you can't cancel the setting with

```
\AddToHook{env/quote/begin}{}
```

because that only "adds" a further empty chunk of code to the hook. Adding `\normalsize` would work but that means the hook then contained `\small\normalsize` which means two font size changes for no good reason.

The above is only needed if one wants to typeset several quotes in a smaller typeface. If the hook is only needed once then `\AddToHookNext` is simpler, because it resets itself after one use.

`\AddToHookNext` `\AddToHookNext {⟨hook⟩} {⟨code⟩}`

Adds ⟨*code*⟩ to the next invocation of the ⟨*hook*⟩. The code is executed after the normal hook code has finished and it is executed only once, i.e. it is deleted after it was used.

Using this declaration is a global operation, i.e., the code is not lost even if the declaration is used inside a group and the next invocation of the hook happens after the end of that group. If the declaration is used several times before the hook is executed then all code is executed in the order in which it was declared.[4]

If this declaration is used with a one-time hook then the code is only ever used if the declaration comes before the hook's invocation. This is because, in contrast to `\AddToHook`, the code in this declaration is not executed immediately in the case when the invocation of the hook has already happened—in other words, this code will truly execute only on the next invocation of the hook (and in the case of a one-time hook there is no such "next invocation"). This gives you a choice: should my code execute always, or should it execute only at the point where the one-time hook is used (and not at all if this is impossible)? For both of these possibilities there are use cases.

It is possible to nest this declaration using the same hook (or different hooks): e.g.,

`\AddToHookNext{⟨hook⟩}{⟨code-1⟩\AddToHookNext{⟨hook⟩}{⟨code-2⟩}}`

will execute ⟨*code-1*⟩ next time the ⟨*hook*⟩ is used and at that point puts ⟨*code-2*⟩ into the ⟨*hook*⟩ so that it gets executed on following time the hook is run.

A hook doesn't have to exist for code to be added to it. This allows for hooks to work regardless of package loading order. See section 2.1.8.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

`\AddToHookNextWithArguments` `\AddToHookNextWithArguments {⟨hook⟩} {⟨code⟩}`

Works exactly like `\AddToHookNext`, but the ⟨*code*⟩ can contain references to the arguments of the ⟨*hook*⟩ as described for `\AddToHookWithArguments` above. Section 2.7 explains hooks with arguments.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

`\ClearHookNext` `\ClearHookNext {⟨hook⟩}`

Normally `\AddToHookNext` is only used when you know precisely where it will apply and why you want some extra code at that point. However, there are a few use cases in which such a declaration needs to be canceled, for example, when discarding a page with `\DiscardShipoutBox` (but even then not always), and in such situations `\ClearHookNext` can be used.

[4] There is no mechanism to reorder such code chunks (or delete them).

2.1.5 Hook names and default labels

It is best practice to use `\AddToHook` in packages or classes *without specifying a* ⟨*label*⟩ because then the package or class name is automatically used, which is helpful if rules are needed, and avoids mistyping the ⟨*label*⟩.

Using an explicit ⟨*label*⟩ is only necessary in very specific situations, e.g., if you want to add several chunks of code into a single hook and have them placed in different parts of the hook (by providing some rules).

The other case is when you develop a larger package with several sub-packages. In that case you may want to use the same ⟨*label*⟩ throughout the sub-packages in order to avoid that the labels change if you internally reorganize your code.

Except for `\UseHook`, `\UseOneTimeHook` and `\IfHookEmptyTF` (and their expl3 interfaces `\hook_use:n`, `\hook_use_once:n` and `\hook_if_empty:nTF`), all ⟨*hook*⟩ and ⟨*label*⟩ arguments are processed in the same way: first, spaces are trimmed around the argument, then it is fully expanded until only character tokens remain. If the full expansion of the ⟨*hook*⟩ or ⟨*label*⟩ contains a non-expandable non-character token, a low-level TEX error is raised (namely, the ⟨*hook*⟩ is expanded using TEX's `\csname`...`\endcsname`, as such, Unicode characters are allowed in ⟨*hook*⟩ and ⟨*label*⟩ arguments). The arguments of `\UseHook`, `\UseOneTimeHook`, and `\IfHookEmptyTF` are processed much in the same way except that spaces are not trimmed around the argument, for better performance.

It is not enforced, but highly recommended that the hooks defined by a package, and the ⟨*labels*⟩ used to add code to other hooks contain the package name to easily identify the source of the code chunk and to prevent clashes. This should be the standard practice, so this hook management code provides a shortcut to refer to the current package in the name of a ⟨*hook*⟩ and in a ⟨*label*⟩. If the ⟨*hook*⟩ name or the ⟨*label*⟩ consist just of a single dot (`.`), or starts with a dot followed by a slash (`./`) then the dot denotes the ⟨*default label*⟩ (usually the current package or class name—see `\SetDefaultHookLabel`). A "`.`" or "`./`" anywhere else in a ⟨*hook*⟩ or in ⟨*label*⟩ is treated literally and is not replaced.

For example, inside the package `mypackage.sty`, the default label is `mypackage`, so the instructions:

```
\NewHook   {./hook}
\AddToHook {./hook}[.]{code}     % Same as \AddToHook{./hook}{code}
\AddToHook {./hook}[./sub]{code}
\DeclareHookRule{begindocument}{.}{before}{babel}
\AddToHook {file/foo.tex/after}{code}
```

are equivalent to:

```
\NewHook   {mypackage/hook}
\AddToHook {mypackage/hook}[mypackage]{code}
\AddToHook {mypackage/hook}[mypackage/sub]{code}
\DeclareHookRule{begindocument}{mypackage}{before}{babel}
\AddToHook {file/foo.tex/after}{code}                  % unchanged
```

The ⟨*default label*⟩ is automatically set equal to the name of the current package or class at the time the package is loaded. If the hook command is used outside of a package, or the current file wasn't loaded with `\usepackage` or `\documentclass`, then the `top-level` is used as the ⟨*default label*⟩. This may have exceptions—see `\PushDefaultHookLabel`.

This syntax is available in all ⟨*label*⟩ arguments and most ⟨*hook*⟩ arguments, both in the LaTeX 2_ε interface, and the LaTeX3 interface described in section 2.2.

Important:
The dot-syntax is ***not*** *available with* `\UseHook` *and some other commands that are typically used within code!*

Note, however, that the replacement of `.` by the ⟨*default label*⟩ takes place when the hook command is executed, so actions that are somehow executed after the package ends will have the wrong ⟨*default label*⟩ if the dot-syntax is used. For that reason, this syntax is not available in `\UseHook` (and `\hook_use:n`) because the hook is most of the time used outside of the package file in which it was defined. This syntax is also not available in the hook conditionals `\IfHookEmptyTF` (and `\hook_if_empty:nTF`), because these conditionals are used in some performance-critical parts of the hook management code, and because they are usually used to refer to other package's hooks, so the dot-syntax doesn't make much sense.

In some cases, for example in large packages, one may want to separate the code in logical parts, but still use the main package name as the ⟨*label*⟩, then the ⟨*default label*⟩ can be set using `\PushDefaultHookLabel{...}` ... `\PopDefaultHookLabel` or `\SetDefaultHookLabel{...}`.

`\PushDefaultHookLabel`
`\PopDefaultHookLabel`

```
\PushDefaultHookLabel {⟨default label⟩}
  ⟨code⟩
\PopDefaultHookLabel
```

`\PushDefaultHookLabel` sets the current ⟨*default label*⟩ to be used in ⟨*label*⟩ arguments, or when replacing a leading "`.`" (see above). `\PopDefaultHookLabel` reverts the ⟨*default label*⟩ to its previous value.

Inside a package or class, the ⟨*default label*⟩ is equal to the package or class name, unless explicitly changed. Everywhere else, the ⟨*default label*⟩ is `top-level` (see section 2.1.6) unless explicitly changed.

The effect of `\PushDefaultHookLabel` holds until the next `\PopDefaultHookLabel`. `\usepackage` (and `\RequirePackage` and `\documentclass`) internally use

```
\PushDefaultHookLabel{⟨package name⟩}
  ⟨package code⟩
\PopDefaultHookLabel
```

to set the ⟨*default label*⟩ for the package or class file. Inside the ⟨*package code*⟩ the ⟨*default label*⟩ can also be changed with `\SetDefaultHookLabel`. `\input` and other file input-related commands from the LaTeX kernel do not use `\PushDefaultHookLabel`, so code within files loaded by these commands does *not* get a dedicated ⟨*label*⟩! (that is, the ⟨*default label*⟩ is the current active one when the file was loaded.)

Packages that provide their own package-like interfaces (Ti*k*Z's `\usetikzlibrary`, for example) can use `\PushDefaultHookLabel` and `\PopDefaultHookLabel` to set dedicated labels and to emulate `\usepackage`-like hook behavior within those contexts.

The `top-level` label is treated differently, and is reserved to the user document, so it is not allowed to change the ⟨*default label*⟩ to `top-level`.

\SetDefaultHookLabel \SetDefaultHookLabel {⟨*default label*⟩}

Similarly to \PushDefaultHookLabel, sets the current ⟨*default label*⟩ to be used in ⟨*label*⟩ arguments, or when replacing a leading ".". The effect holds until the label is changed again or until the next \PopDefaultHookLabel. The difference between \PushDefaultHookLabel and \SetDefaultHookLabel is that the latter does not save the current ⟨*default label*⟩.

This command is useful when a large package is composed of several smaller packages, but all should have the same ⟨*label*⟩, so \SetDefaultHookLabel can be used at the beginning of each package file to set the correct label.

\SetDefaultHookLabel is not allowed in the main document, where the ⟨*default label*⟩ is top-level and there is no \PopDefaultHookLabel to end its effect. It is also not allowed to change the ⟨*default label*⟩ to top-level.

2.1.6 The top-level label

The top-level label, assigned to code added from the main document, is different from other labels. Code added to hooks (usually \AtBeginDocument) in the preamble is almost always to change something defined by a package, so it should go at the very end of the hook.

Therefore, code added in the top-level is always executed at the end of the hook, regardless of where it was declared. If the hook is reversed (see \NewReversedHook), the top-level chunk is executed at the very beginning instead.

Rules regarding top-level have no effect: if a user wants to have a specific set of rules for a code chunk, they should use a different label to said code chunk, and provide a rule for that label instead.

The top-level label is exclusive for the user, so trying to add code with that label from a package results in an error.

2.1.7 Defining relations between hook code

The default assumption is that code added to hooks by different packages are independent and the order in which they are executed is irrelevant. While this is true in many cases it is obviously false in others.

Before the hook management system was introduced packages had to take elaborate precautions to determine whether some other package had also been loaded (before or after) and then to find some ways to alter its behavior accordingly. In addition is was often the user's responsibility to load packages in the right order so that alterations made by packages were done in thsat same order; and in some cases even altering the loading order wouldn't resolve the conflicts.

With the new hook management system it is now possible to define rules (i.e., relationships) between code chunks added by different packages and to specify explicitly the order in which they should be processed.

The rules can be declared for hooks before the hook has been declared with \NewHook and they are allowed to refer to code labels that do not yet exist, e.g., because a package defining the code chunk with that label has not yet been loaded. When the hook code is finally sorted for fast execution, all rules that apply are acted on and the others are ignored.

This offers the flexibility needed to handle complicated relationships between code from different packages and to set this up beforehand in a way that is independent of whether or not the packages are actually loaded in a specific document. The downside

of this is that misspellings of hook names or code labels will not raise any error, instead the rule will simply never apply!

`\DeclareHookRule` `\DeclareHookRule {⟨hook⟩} {⟨label1⟩} {⟨relation⟩} {⟨label2⟩}`

Defines a relation between ⟨*label1*⟩ and ⟨*label2*⟩ for a given ⟨*hook*⟩. If ⟨*hook*⟩ is `??` this defines a default relation for all hooks that use the two labels, i.e., that have chunks of code labeled with ⟨*label1*⟩ and ⟨*label2*⟩.

Currently, the supported relations are the following:

`before` or `<` Code for ⟨*label1*⟩ comes before code for ⟨*label2*⟩.

`after` or `>` Code for ⟨*label1*⟩ comes after code for ⟨*label2*⟩.

`incompatible-warning` Only code for either ⟨*label1*⟩ or ⟨*label2*⟩ can appear for that hook (a way to say that two packages—or parts of them—are incompatible). A warning is raised if both labels appear in the same hook.

`incompatible-error` Like `incompatible-error` but instead of a warning a LaTeX error is raised, and the code for both labels are dropped from that hook until the conflict is resolved.

`voids` Code for ⟨*label1*⟩ overwrites code for ⟨*label2*⟩. More precisely, code for ⟨*label2*⟩ is dropped for that hook. This can be used, for example if one package is a superset in functionality of another one and therefore wants to undo code in some hook and replace it with its own version.

`unrelated` The order of code for ⟨*label1*⟩ and ⟨*label2*⟩ is irrelevant. This rule is there to undo an incorrect rule specified earlier.

There can only be a single relation between two labels for a given hook, i.e., a later `\DeclareHookRule` overwrites any previous declaration. In all cases rules specific to a given hook take precedence over default rules that use `??` as the ⟨*hook*⟩.

If a default rule is applied, it is done before reversing the label order in a reversed hook, e.g., `before` in a default rule effectively becomes `after` in such a hook. In contrast, a rule for a specific hook is always applied to the state after any reversal (i.e., the state you see when using `\ShowHook` on that hook).

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

`\ClearHookRule` `\ClearHookRule {⟨hook⟩} {⟨label1⟩} {⟨label2⟩}`

Syntactic sugar for saying that ⟨*label1*⟩ and ⟨*label2*⟩ are unrelated for the given ⟨*hook*⟩.

\DeclareDefaultHookRule `\DeclareDefaultHookRule {⟨label1⟩} {⟨relation⟩} {⟨label2⟩}`

This sets up a relation between ⟨*label1*⟩ and ⟨*label2*⟩ for all hooks unless overwritten by a specific rule for a hook. Useful for cases where one package has a specific relation to some other package, e.g., is `incompatible` or always needs a special ordering `before` or `after`. (Technically it is just a shorthand for using `\DeclareHookRule` with `??` as the hook name.)

If such a rule is applied to a reversed hook it behaves as if the rule is reversed (e.g., `after` becomes `before`) because those rules are applied first and then the order is reversed. The rationale is that in hook pairs (in which the ordering in one is reversed) default rules have to be reversed too in nearly all scenarios. If this is not the case, a default rule can't be used or has to be overwritten with an explicit `\DeclareHookRule` for that specific hook.

Declaring default rules is only supported in the document preamble.[5]

The ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

2.1.8 Querying hooks

Simpler data types, like token lists, have three possible states; they can:

- exist and be empty;
- exist and be non-empty; and
- not exist (in which case emptiness doesn't apply);

Hooks are a bit more complicated: a hook may exist or not, and independently it may or may not be empty. This means that even a hook that doesn't exist may be non-empty and it can also be disabled.

This seemingly strange state may happen when, for example, package *A* defines hook `A/foo`, and package *B* adds some code to that hook. However, a document may load package *B* before package *A*, or may not load package *A* at all. In both cases some code is added to hook `A/foo` without that hook being defined yet, thus that hook is said to be non-empty, whereas it doesn't exist. Therefore, querying the existence of a hook doesn't imply its emptiness, neither does the other way around.

Given that code or rules can be added to a hook even if it doesn't physically exist yet, means that a querying its existence has no real use case (in contrast to other variables that can only be update if they have already been declared). For that reason only the test for emptiness has a public interface.

A hook is said to be empty when no code was added to it, either to its permanent code pool, or to its "next" token list. The hook doesn't need to be declared to have code added to its code pool. A hook is said to exist when it was declared with `\NewHook` or some variant thereof. Generic hooks such as `file` and `env` hooks are automatically declared when code is added to them.

[5] Trying to do so, e.g., via `\DeclareHookRule` with `??` has bad side-effects and is not supported (though not explicitly caught for performance reasons).

\IfHookEmptyTF ⋆ \IfHookEmptyTF {⟨*hook*⟩} {⟨*true code*⟩} {⟨*false code*⟩}
\IfHookEmptyT ⋆
\IfHookEmptyF ⋆

Tests if the ⟨*hook*⟩ is empty (*i.e.*, no code was added to it using either \AddToHook or \AddToHookNext) or such code was removed again (via \RemoveFromHook), and branches to either ⟨*true code*⟩ or ⟨*false code*⟩ depending on the result.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading . is treated literally.

2.1.9 Displaying hook code

If one has to adjust the code execution in a hook using a hook rule it is helpful to get some information about the code associated with a hook, its current order and the existing rules.

\ShowHook \ShowHook {⟨*hook*⟩}
\LogHook \LogHook {⟨*hook*⟩}

Displays information about the ⟨*hook*⟩ such as

- the code chunks (and their labels) added to it,
- any rules set up to order them,
- the computed order in which the chunks are executed,
- any code executed on the next invocation only.

\LogHook prints the information to the .log file, and \ShowHook prints them to the terminal/command window and starts TEX's prompt (only in \errorstopmode) to wait for user action.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

Suppose a hook example-hook whose output of \ShowHook{example-hook} is:

```
1  -> The hook 'example-hook':
2  > Code chunks:
3  >     foo -> [code from package 'foo']
4  >     bar -> [from package 'bar']
5  >     baz -> [package 'baz' is here]
6  > Document-level (top-level) code (executed last):
7  >     -> [code from 'top-level']
8  > Extra code for next invocation:
9  >     -> [one-time code]
10 > Rules:
11 >     foo|baz with relation >
12 >     baz|bar with default relation <
13 > Execution order (after applying rules):
14 >     baz, foo, bar.
```

In the listing above, lines 3 to 5 show the three code chunks added to the hook and their respective labels in the format

⟨*label*⟩ -> ⟨*code*⟩

Line 7 shows the code chunk added by the user in the main document (labeled `top-level`) in the format

```
Document-level (top-level) code (executed ⟨first|last⟩):
  -> ⟨top-level code⟩
```

This code will be either the first or last code executed by the hook (`last` if the hook is normal, `first` if it is reversed). This chunk is not affected by rules and does not take part in sorting.

Line 9 shows the code chunk for the next execution of the hook in the format

```
-> ⟨next-code⟩
```

This code will be used and disappear at the next `\UseHook{example-hook}`, in contrast to the chunks mentioned earlier, which can only be removed from that hook by doing `\RemoveFromHook{⟨label⟩}[example-hook]`.

Lines 11 and 12 show the rules declared that affect this hook in the format

```
⟨label-1⟩|⟨label-2⟩ with ⟨default?⟩ relation ⟨relation⟩
```

which means that the ⟨*relation*⟩ applies to ⟨*label-1*⟩ and ⟨*label-2*⟩, in that order, as detailed in `\DeclareHookRule`. If the relation is `default` it means that this rule applies to ⟨*label-1*⟩ and ⟨*label-2*⟩ in *all* hooks, (unless overridden by a non-default relation).

Finally, line 14 lists the labels in the hook after sorting; that is, in the order they will be executed when the hook is used.

2.1.10 Debugging hook code

`\DebugHooksOn` `\DebugHooksOff`

`\DebugHooksOn ... \DebugHooksOff`

Turn the debugging of hook code on or off. This displays most changes made to the hook data structures. The output is rather coarse and not really intended for normal use, but it can be helpful in case hooks do not work as expected. See also 2.1.9 for commands to inspect individual hooks.

2.2 L3 programming layer (`expl3`) interfaces

This is a quick summary of the LaTeX3 programming interfaces for use with packages written in `expl3`. In contrast to the LaTeX 2ε interfaces they always use mandatory arguments only, e.g., you always have to specify the ⟨*label*⟩ for a code chunk. We therefore suggest to use the declarations discussed in the previous section even in `expl3` packages, but the choice is yours.

`\hook_new:n` `\hook_new_reversed:n` `\hook_new_pair:nn`

`\hook_new:n {⟨hook⟩}`
`\hook_new_reversed:n {⟨hook⟩}`
`\hook_new_pair:nn {⟨hook-1⟩} {⟨hook-2⟩}`

Creates a new ⟨*hook*⟩ with normal or reverse ordering of code chunks. `\hook_new_pair:nn` creates a pair of such hooks with `{`⟨*hook-2*⟩`}` being a reversed hook. If a hook name is already taken, an error is raised and the hook is not created.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

\hook_new_with_args:nn `\hook_new_with_args:nn {⟨hook⟩} {⟨number⟩}`
\hook_new_reversed_with_args:nn `\hook_new_reversed_with_args:nn {⟨hook⟩} {⟨number⟩}`
\hook_new_pair_with_args:nnn `\hook_new_pair_with_args:nnn {⟨hook-1⟩} {⟨hook-2⟩} {⟨number⟩}`

Creates a new ⟨*hook*⟩ with normal or reverse ordering of code chunks, that takes ⟨*number*⟩ arguments from the input stream when it is used. `\hook_new_pair_with_args:nn` creates a pair of such hooks with {⟨*hook-2*⟩} being a reversed hook. If a hook name is already taken, an error is raised and the hook is not created.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

\hook_disable_generic:n `\hook_disable_generic:n {⟨hook⟩}`

Marks {⟨*hook*⟩} as disabled. Any further attempt to add code to it or declare it, will result in an error and any call to `\hook_use:n` will simply do nothing.

This declaration is intended for use with generic hooks that are known not to work (see `ltcmdhooks-doc`) if they receive code.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

\hook_activate_generic:n `\hook_activate_generic:n {⟨hook⟩}`

This is like `\hook_new:n` but it does nothing if the hook was previously declared with `\hook_new:n`. This declaration should be used only in special situations, e.g., when a command from another package needs to be altered and it is not clear whether a generic `cmd` hook (for that command) has been previously explicitly declared.

Normally `\hook_new:n` should be used instead of this.

\hook_use:n `\hook_use:n {⟨hook⟩}`
\hook_use:nnw `\hook_use:nnw {⟨hook⟩} {⟨number⟩} {⟨arg$_1$⟩} ... {⟨arg$_n$⟩}`

Executes the {⟨*hook*⟩} code followed (if set up) by the code for next invocation only, then empties that next invocation code. `\hook_use:nnw` should be used for hooks declared with arguments, and should be followed by as many brace groups as the declared number of arguments. The ⟨*number*⟩ should be the number of arguments declared for the hook. If the hook is not declared, this command does nothing and it will remove ⟨*number*⟩ items from the input.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally.

\hook_use_once:n `\hook_use_once:n {⟨hook⟩}`
\hook_use_once:nnw `\hook_use_once:nnw {⟨hook⟩} {⟨number⟩} {⟨arg$_1$⟩} ... {⟨arg$_n$⟩}`

Changes the {⟨*hook*⟩} status so that from now on any addition to the hook code is executed immediately. Then execute any {⟨*hook*⟩} code already set up. `\hook_use_once:nnw` should be used for hooks declared with arguments, and should be followed by as many brace groups as the declared number of arguments. The ⟨*number*⟩ should be the number of arguments declared for the hook. If the hook is not declared, this command does nothing and it will remove ⟨*number*⟩ items from the input.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally.

\hook_gput_code:nnn `\hook_gput_code:nnn {⟨hook⟩} {⟨label⟩} {⟨code⟩}`
\hook_gput_code_with_args:nnn `\hook_gput_code_with_args:nnn {⟨hook⟩} {⟨label⟩} {⟨code⟩}`

Adds a chunk of ⟨*code*⟩ to the ⟨*hook*⟩ labeled ⟨*label*⟩. If the label already exists the ⟨*code*⟩ is appended to the already existing code.

If `\hook_gput_code_with_args:nnn` is used, the ⟨*code*⟩ can access the arguments passed to `\hook_use:nnw` (or `\hook_use_once:nnw`) with `#1`, `#2`, ..., `#n` (up to the number of arguments declared for the hook). In that case, if an actual parameter token should be added to the code, it should be doubled.

If code is added to an external ⟨*hook*⟩ (of the kernel or another package) then the convention is to use the package name as the ⟨*label*⟩ not some internal module name or some other arbitrary string.

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

\hook_gput_next_code:nn `\hook_gput_next_code:nn {⟨hook⟩} {⟨code⟩}`
\hook_gput_next_code_with_args:nn `\hook_gput_next_code_with_args:nn {⟨hook⟩} {⟨code⟩}`

Adds a chunk of ⟨*code*⟩ for use only in the next invocation of the ⟨*hook*⟩. Once used it is gone.

If `\hook_gput_next_code_with_args:nn` is used, the ⟨*code*⟩ can access the arguments passed to `\hook_use:nnw` (or `\hook_use_once:nnw`) with `#1`, `#2`, ..., `#n` (up to the number of arguments declared for the hook). In that case, if an actual parameter token should be added to the code, it should be doubled.

This is simpler than `\hook_gput_code:nnn`, the code is simply appended to the hook in the order of declaration at the very end, i.e., after all standard code for the hook got executed. Thus if one needs to undo what the standard does one has to do that as part of ⟨*code*⟩.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

\hook_gclear_next_code:n `\hook_gclear_next_code:n {⟨hook⟩}`

Undo any earlier `\hook_gput_next_code:nn`.

\hook_gremove_code:nn `\hook_gremove_code:nn {⟨hook⟩} {⟨label⟩}`

Removes any code for ⟨*hook*⟩ labeled ⟨*label*⟩.

If there is no code under the ⟨*label*⟩ in the ⟨*hook*⟩, or if the ⟨*hook*⟩ does not exist, a warning is issued when you attempt to use `\hook_gremove_code:nn`, and the command is ignored.

If the second argument is `*`, then all code chunks are removed. This is rather dangerous as it drops code from other packages one may not know about, so think twice before using that!

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

`\hook_gset_rule:nnnn` `\hook_gset_rule:nnnn {⟨hook⟩} {⟨label1⟩} {⟨relation⟩} {⟨label2⟩}`

Relate ⟨*label1*⟩ with ⟨*label2*⟩ when used in ⟨*hook*⟩. See `\DeclareHookRule` for the allowed ⟨*relation*⟩s. If ⟨*hook*⟩ is `??` a default rule is specified.

The ⟨*hook*⟩ and ⟨*label*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5. The dot-syntax is parsed in both ⟨*label*⟩ arguments, but it usually makes sense to be used in only one of them.

`\hook_if_empty_p:n` ⋆
`\hook_if_empty:nTF` ⋆ `\hook_if_empty:nTF {⟨hook⟩} {⟨true code⟩} {⟨false code⟩}`

Tests if the ⟨*hook*⟩ is empty (*i.e.*, no code was added to it using either `\AddToHook` or `\AddToHookNext`), and branches to either ⟨*true code*⟩ or ⟨*false code*⟩ depending on the result.

The ⟨*hook*⟩ *cannot* be specified using the dot-syntax. A leading `.` is treated literally.

`\hook_show:n` `\hook_show:n {⟨hook⟩}`
`\hook_log:n` `\hook_log:n {⟨hook⟩}`

Displays information about the ⟨*hook*⟩ such as

- the code chunks (and their labels) added to it,
- any rules set up to order them,
- the computed order in which the chunks are executed,
- any code executed on the next invocation only.

`\hook_log:n` prints the information to the `.log` file, and `\hook_show:n` prints them to the terminal/command window and starts TEX's prompt (only if `\errorstopmode`) to wait for user action.

The ⟨*hook*⟩ can be specified using the dot-syntax to denote the current package name. See section 2.1.5.

`\hook_debug_on:` `\hook_debug_on:`
`\hook_debug_off:`

Turns the debugging of hook code on or off. This displays changes to the hook data.

2.3 On the order of hook code execution

Chunks of code for a ⟨*hook*⟩ under different labels are supposed to be independent if there are no special rules set up that define a relation between the chunks. This means that you can't make assumptions about the order of execution!

Suppose you have the following declarations:

```
\NewHook{myhook}
\AddToHook{myhook}[packageA]{\typeout{A}}
\AddToHook{myhook}[packageB]{\typeout{B}}
\AddToHook{myhook}[packageC]{\typeout{C}}
```

then executing the hook with `\UseHook` will produce the typeout `A B C` in that order. In other words, the execution order is computed to be `packageA`, `packageB`, `packageC` which you can verify with `\ShowHook{myhook}`:

```
-> The hook 'myhook':
> Code chunks:
>     packageA -> \typeout {A}
>     packageB -> \typeout {B}
>     packageC -> \typeout {C}
> Document-level (top-level) code (executed last):
>     ---
> Extra code for next invocation:
>     ---
> Rules:
>     ---
> Execution order:
>     packageA, packageB, packageC.
```

The reason is that the code chunks are internally saved in a property list and the initial order of such a property list is the order in which key-value pairs got added. However, that is only true if nothing other than adding happens!

Suppose, for example, you want to replace the code chunk for `packageA`, e.g.,

```
\RemoveFromHook{myhook}[packageA]
\AddToHook{myhook}[packageA]{\typeout{A alt}}
```

then your order becomes `packageB`, `packageC`, `packageA` because the label got removed from the property list and then re-added (at its end).

While that may not be too surprising, the execution order is also sometimes altered if you add a redundant rule, e.g. if you specify

```
\DeclareHookRule{myhook}{packageA}{before}{packageB}
```

instead of the previous lines we get

```
-> The hook 'myhook':
> Code chunks:
>     packageA -> \typeout {A}
>     packageB -> \typeout {B}
>     packageC -> \typeout {C}
> Document-level (top-level) code (executed last):
>     ---
> Extra code for next invocation:
>     ---
> Rules:
>     packageB|packageA with relation >
> Execution order (after applying rules):
>     packageA, packageC, packageB.
```

As you can see the code chunks are still in the same order, but in the execution order for the labels `packageB` and `packageC` have swapped places. The reason is that, with the rule there are two orders that satisfy it, and the algorithm for sorting happened to pick a different one compared to the case without rules (where it doesn't run at all as there is nothing to resolve). Incidentally, if we had instead specified the redundant rule

```
\DeclareHookRule{myhook}{packageB}{before}{packageC}
```

the execution order would not have changed.

In summary: it is not possible to rely on the order of execution unless there are rules that partially or fully define the order (in which you can rely on them being fulfilled).

2.4 The use of “reversed” hooks

You may have wondered why you can declare a “reversed” hook with `\NewReversedHook` and what that does exactly.

In short: the execution order of a reversed hook (without any rules!) is exactly reversed to the order you would have gotten for a hook declared with `\NewHook`.

This is helpful if you have a pair of hooks where you expect to see code added that involves grouping, e.g., starting an environment in the first and closing that environment in the second hook. To give a somewhat contrived example[6], suppose there is a package adding the following:

```
\AddToHook{env/quote/before}[package-1]{\begin{itshape}}
\AddToHook{env/quote/after} [package-1]{\end{itshape}}
```

As a result, all quotes will be in italics. Now suppose further that another `package-too` makes the quotes also in blue and therefore adds:

```
\usepackage{color}
\AddToHook{env/quote/before}[package-too]{\begin{color}{blue}}
\AddToHook{env/quote/after} [package-too]{\end{color}}
```

Now if the `env/quote/after` hook would be a normal hook we would get the same execution order in both hooks, namely:

```
package-1, package-too
```

(or vice versa) and as a result, would get:

```
\begin{itshape}\begin{color}{blue} ...
\end{itshape}\end{color}
```

and an error message saying that `\begin{color}` was ended by `\end{itshape}`. With `env/quote/after` declared as a reversed hook the execution order is reversed and so all environments are closed in the correct sequence and `\ShowHook` would give us the following output:

```
-> The hook 'env/quote/after':
> Code chunks:
>     package-1 -> \end {itshape}
>     package-too -> \end {color}
> Document-level (top-level) code (executed first):
>     ---
> Extra code for next invocation:
>     ---
> Rules:
>     ---
> Execution order (after reversal):
>     package-too, package-1.
```

If there is a matching default rule (done with `\DeclareDefaultHookRule` or with `??` for the hook name) then this default rule is applied before the reversal so that the order in the reversed hook mirrors the one in the normal hook. However, all rules specific to a hook happen always after the reversal of the execution order, so if you alter the order you will probably have to alter it in both hooks, not just in one, but that depends on the use case.

[6] There are simpler ways to achieve the same effect.

2.5 Difference between “normal” and “one-time” hooks

When executing a hook a developer has the choice of using either `\UseHook` or `\UseOneTimeHook` (or their expl3 equivalents `\hook_use:n` and `\hook_use_once:n`). This choice affects how `\AddToHook` is handled after the hook has been executed for the first time.

With normal hooks adding code via `\AddToHook` means that the code chunk is added to the hook data structure and then used each time `\UseHook` is called.

With one-time hooks it this is handled slightly differently: After `\UseOneTimeHook` has been called, any further attempts to add code to the hook via `\AddToHook` will simply execute the ⟨*code*⟩ immediately.

This has some consequences one needs to be aware of:

- If ⟨*code*⟩ is added to a normal hook after the hook was executed and it is never executed again for one or the other reason, then this new ⟨*code*⟩ will never be executed.
- In contrast if that happens with a one-time hook the ⟨*code*⟩ is executed immediately.

In particular this means that construct such as

```
\AddToHook{myhook}
          { ⟨code-1⟩ \AddToHook{myhook}{⟨code-2⟩} ⟨code-3⟩ }
```

works for one-time hooks[7] (all three code chunks are executed one after another), but it makes little sense with a normal hook, because with a normal hook the first time `\UseHook{myhook}` is executed it would

- execute ⟨*code-1*⟩,
- then execute `\AddToHook{myhook}{code-2}` which adds the code chunk ⟨*code-2*⟩ to the hook for use on the next invocation,
- and finally execute ⟨*code-3*⟩.

The second time `\UseHook` is called it would execute the above and in addition ⟨*code-2*⟩ as that was added as a code chunk to the hook in the meantime. So each time the hook is used another copy of ⟨*code-2*⟩ is added and so that code chunk is executed ⟨*# of invocations*⟩ − 1 times.

2.6 Generic hooks provided by packages

The hook management system also implements a category of hooks that are called “Generic Hooks”. Normally a hook has to be explicitly declared before it can be used in code. This ensures that different packages are not using the same hook name for unrelated purposes—something that would result in absolute chaos. However, there are a number of “standard” hooks where it is unreasonable to declare them beforehand, e.g, each and every command has (in theory) an associated `before` and `after` hook. In such cases, i.e., for command, environment or file hooks, they can be used simply by adding code to them with `\AddToHook`. For more specialized generic hooks, e.g., those provided

[7] This is sometimes used with `\AtBeginDocument` which is why it is supported.

by babel, you have to additionally enable them with \ActivateGenericHook as explained below.

The generic hooks provided by LaTeX are those for cmd, env, file, include, package, and class, and all these are available out of the box: you only have to use \AddToHook to add code to them, but you don't have to add \UseHook or \UseOneTimeHook to your code, because this is already done for you (or, in the case of cmd hooks, the command's code is patched at \begin{document}, if necessary).

However, if you want to provide further generic hooks in your own code, the situation is slightly different. To do this you should use \UseHook or \UseOneTimeHook, but *without declaring the hook* with \NewHook. As mentioned earlier, a call to \UseHook with an undeclared hook name does nothing. So as an additional setup step, you need to explicitly activate your generic hook. Note that a generic hook produced in this way is always a normal hook.

For a truly generic hook, with a variable part in the hook name, such upfront activation would be difficult or impossible, because you typically do not know what kind of variable parts may come up in real documents.

For example, babel provides hooks such as babel/⟨*language*⟩/afterextras. However, language support in babel is often done through external language packages. Thus doing the activation for all languages inside the core babel code is not a viable approach. Instead it needs to be done by each language package (or by the user who wants to use a particular hook).

Because the hooks are not declared with \NewHook their names should be carefully chosen to ensure that they are (likely to be) unique. Best practice is to include the package or command name, as was done in the babel example above.

Generic hooks defined in this way are always normal hooks (i.e., you can't implement reversed hooks this way). This is a deliberate limitation, because it speeds up the processing considerably.

2.7 Hooks with arguments

Sometimes it is necessary to pass contextual information to a hook, and, for one reason or another, it is not feasible to store such information in macros. To serve this purpose, hooks can be declared with arguments, so that the programmer can pass along the data necessary for the code in the hook to function properly.

A hook with arguments works mostly like a regular hook, and most commands that work for regular hooks, also work for hooks that take arguments. The differences are when the hook is declared (\NewHookWithArguments is used instead of \NewHook), then code can be added with both \AddToHook and \AddToHookWithArguments, and when the hook is used (\UseHookWithArguments instead of \UseHook).

A hook with arguments must be declared as such (before it is first used, as all regular hooks) using \NewHookWithArguments{⟨*hook*⟩}{⟨*number*⟩}. All code added to that hook can then use #1 to access the first argument, #2 to access the second, and so forth up to the number of arguments declared. However, it is still possible to add code with references to the arguments of a hook that was not yet declared (we will discuss that later). At their core, hooks are macros, so TeX's limit of 9 arguments applies, and a low-level TeX error is raised if you try to reference an argument number that doesn't exist.

To use a hook with arguments, just write `\UseHookWithArguments{`⟨*hook*⟩`}{`⟨*number*⟩`}` followed by a braced list of the arguments. For example, if the hook `test` takes three arguments, write:

```
\UseHookWithArguments{test}{3}{arg-1}{arg-2}{arg-3}
```

then, in the ⟨*code*⟩ of the hook, all instances of `#1` will be replaced by `arg-1`, `#2` by `arg-2` and so on. If, at the point of usage, the programmer provides more arguments than the hook is declared to take, the excess arguments are simply ignored by the hook. Behavior is unpredictable[8] if too few arguments are provided. If the hook isn't declared, ⟨*number*⟩ arguments are removed from the input stream.

Adding code to a hook with arguments can be done with `\AddToHookWithArguments` as well as with the regular `\AddToHook`, to achieve different outcomes. The main difference when it comes to adding code to a hook, in this case, is firstly the possibility of accessing a hook's arguments, of course, and second, how parameter tokens (`#`$_6$) are treated.

Using `\AddToHook` in a hook that takes arguments will work as it does for all other hooks. This allows a package developer to add arguments to a hook that otherwise had none without having to worry about compatibility. This means that, for example:

```
\AddToHook{test}{\def\foo#1{Hello, #1!}}
```

will define the same macro `\foo` regardless if the hook `test` takes arguments or not.

Using `\AddToHookWithArguments` allows the ⟨*code*⟩ added to access the arguments of the hook with `#1`, `#2`, and so forth, up to the number of the arguments declared in the hook. This means that if one wants to add a `#`$_6$ to the ⟨*code*⟩ that token must be doubled in the input. The same definition from above, using `\AddToHookWithArguments`, needs to be rewritten:

```
\AddToHookWithArguments{test}{\def\foo##1{Hello, ##1!}}
```

Extending the above example to use the hook arguments, we could rewrite something like (now from declaration to usage, to get the whole picture):

```
\NewHookWithArguments{test}{1}
\AddToHookWithArguments{test}{%
  \typeout{Defining foo with "#1"}
  \def\foo##1{Hello, ##1! Some text after: #1}%
}
\UseHook{test}{Howdy!}
\ShowCommand\foo
```

Running the code above prints in the terminal:

```
Defining foo with "Howdy!"
> \foo=macro:
#1->Hello, #1! Some text after: Howdy!.
```

[8] The hook *will* take the declared number of arguments, and what will happen depends on what was grabbed, and what the hook code does with its arguments.

Note how `##1` in the call to `\AddToHookWithArguments` became `#1`, and the `#1` was replaced by the argument passed to the hook. Should the hook be used again, with a different argument, the definition would naturally change.

It is possible to add code referencing a hook's arguments before such hook is declared and the number of hooks is fixed. However, if some code is added to the hook, that references more arguments than will be declared for the hook, there will be a low-level TEX error about an "Illegal parameter number" at the time the hook is declared, which will be hard to track down because at that point TEX can't know whence the offending code came from. Thus it is important that package writers explicitly document how many arguments (if any) each hook can take, so users of those packages know how many arguments can be referenced, and equally important, what each argument means.

2.8 Private LATEX kernel hooks

There are a few places where it is absolutely essential for LATEX to function correctly that code is executed in a precisely defined order. Even that could have been implemented with the hook management (by adding various rules to ensure the appropriate ordering with respect to other code added by packages). However, this makes every document unnecessary slow, because there has to be sorting even though the result is predetermined. Furthermore it forces package writers to unnecessarily add such rules if they add further code to the hook (or break LATEX).

For that reason such code is not using the hook management, but instead private kernel commands directly before or after a public hook with the following naming convention: `\@kernel@before@`⟨*hook*⟩ or `\@kernel@after@`⟨*hook*⟩. For example, in `\enddocument` you find

```
\UseHook{enddocument}%
\@kernel@after@enddocument
```

which means first the user/package-accessible `enddocument` hook is executed and then the internal kernel hook. As their name indicates these kernel commands should not be altered by third-party packages, so please refrain from that in the interest of stability and instead use the public hook next to it.[9]

2.9 Legacy LATEX 2ε interfaces

LATEX 2ε offered a small number of hooks together with commands to add to them. They are listed here and are retained for backwards compatibility.

With the new hook management, several additional hooks have been added to LATEX and more will follow. See the next section for what is already available.

[9] As with everything in TEX there is not enforcement of this rule, and by looking at the code it is easy to find out how the kernel adds to them. The main reason of this section is therefore to say "please don't do that, this is unconfigurable code!"

\AtBeginDocument `\AtBeginDocument [⟨label⟩] {⟨code⟩}`

If used without the optional argument ⟨*label*⟩, it works essentially like before, i.e., it is adding ⟨*code*⟩ to the hook `begindocument` (which is executed inside `\begin{document}`). However, all code added this way is labeled with the label `top-level` (see section 2.1.6) if done outside of a package or class or with the package/class name if called inside such a file (see section 2.1.5).

This way one can add code to the hook using `\AddToHook` or `\AtBeginDocument` using a different label and explicitly order the code chunks as necessary, e.g., run some code before or after another package's code. When using the optional argument the call is equivalent to running `\AddToHook {begindocument} [`⟨*label*⟩`] {`⟨*code*⟩`}`.

`\AtBeginDocument` is a wrapper around the `begindocument` hook (see section 3.2), which is a one-time hook. As such, after the `begindocument` hook is executed at `\begin{document}` any attempt to add ⟨*code*⟩ to this hook with `\AtBeginDocument` or with `\AddToHook` will cause that ⟨*code*⟩ to execute immediately instead. See section 2.5 for more on one-time hooks.

For important packages with known order requirement we may over time add rules to the kernel (or to those packages) so that they work regardless of the loading-order in the document.

\AtEndDocument `\AtEndDocument [⟨label⟩] {⟨code⟩}`

Like `\AtBeginDocument` but for the `enddocument` hook.

The few hooks that existed previously in LaTeX 2ε used internally commands such as `\@begindocumenthook` and packages sometimes augmented them directly rather than working through `\AtBeginDocument`. For that reason there is currently support for this, that is, if the system detects that such an internal legacy hook command contains code it adds it to the new hook system under the label `legacy` so that it doesn't get lost.

However, over time the remaining cases of direct usage need updating because in one of the future release of LaTeX we will turn this legacy support off, as it does unnecessary slow down the processing.

3 LaTeX 2ε commands and environments augmented by hooks

In this section we describe the standard hooks that are now offered by LaTeX, or give pointers to other documents in which they are described. This section will grow over time (and perhaps eventually move to usrguide3).

3.1 Generic hooks

As stated earlier, with the exception of generic hooks, all hooks must be declared with `\NewHook` before they can be used. All generic hooks have names of the form "⟨*type*⟩/⟨*name*⟩/⟨*position*⟩", where ⟨*type*⟩ is from the predefined list shown below, and ⟨*name*⟩ is the variable part whose meaning will depend on the ⟨*type*⟩. The last component, ⟨*position*⟩, has more complex possibilities: it can always be `before` or `after`; for `env` hooks, it can also be `begin` or `end`; and for `include` hooks it can also be `end`. Each specific hook is documented below, or in `ltcmdhooks-doc.pdf` or `ltfilehook-doc.pdf`.

The generic hooks provided by LaTeX belong to one of the six types:

env Hooks executed before and after environments – ⟨*name*⟩ is the name of the environment, and available values for ⟨*position*⟩ are `before`, `begin`, `end`, and `after`;

cmd Hooks added to and executed before and after commands – ⟨*name*⟩ is the name of the command, and available values for ⟨*position*⟩ are `before` and `after`;

file Hooks executed before and after reading a file – ⟨*name*⟩ is the name of the file (with extension), and available values for ⟨*position*⟩ are `before` and `after`;

package Hooks executed before and after loading packages – ⟨*name*⟩ is the name of the package, and available values for ⟨*position*⟩ are `before` and `after`;

class Hooks executed before and after loading classes – ⟨*name*⟩ is the name of the class, and available values for ⟨*position*⟩ are `before` and `after`;

include Hooks executed before and after `\included` files – ⟨*name*⟩ is the name of the included file (without the `.tex` extension), and available values for ⟨*position*⟩ are `before`, `end`, and `after`.

Each of the hooks above are detailed in the following sections and in linked documentation.

3.1.1 Generic hooks for all environments

Every environment ⟨*env*⟩ has now four associated hooks coming with it:

`env/`⟨*env*⟩`/before` This hook is executed as part of `\begin` as the very first action, in particular prior to starting the environment group. Its scope is therefore not restricted by the environment.

`env/`⟨*env*⟩`/begin` This hook is executed as part of `\begin` directly in front of the code specific to the environment start (e.g., the third argument of `\NewDocumentEnvironment` and the second argument of `\newenvironment`). Its scope is the environment body.

`env/`⟨*env*⟩`/end` This hook is executed as part of `\end` directly in front of the code specific to the end of the environment (e.g., the forth argument of `\NewDocumentEnvironment` and the third argument of `\newenvironment`).

`env/`⟨*env*⟩`/after` This hook is executed as part of `\end` after the code specific to the environment end and after the environment group has ended. Its scope is therefore not restricted by the environment.

The hook is implemented as a reversed hook so if two packages add code to `env/`⟨*env*⟩`/before` and to `env/`⟨*env*⟩`/after` they can add surrounding environments and the order of closing them happens in the right sequence.

Given that these generic hook names involve `/` as part of their name they would not work if one tries to define an environment using a name that involves a `/`.[10]

Generic environment hooks are never one-time hooks even with environments that are supposed to appear only once in a document.[11] In contrast to other hooks there is also no need to declare them using `\NewHook`.

[10] Officially, LaTeX names for environments should only consist of a sequence of letters, numbers, and the character `*`, i.e., this is not a new restriction.

[11] Thus if one adds code to such hooks after the environment has been processed, it will only be executed if the environment appears again and if that doesn't happen the code will never get executed.

The hooks are only executed if `\begin{`⟨*env*⟩`}` and `\end{`⟨*env*⟩`}` is used. If the environment code is executed via low-level calls to `\`⟨*env*⟩ and `\end`⟨*env*⟩ (e.g., to avoid the environment grouping) they are not available. If you want them available in code using this method, you would need to add them yourself, i.e., write something like

```
\UseHook{env/quote/before}\quote
    ...
\endquote\UseHook{env/quote/after}
```

to add the outer hooks, etc.

Largely for compatibility with existing packages, the following four commands are also available to set the environment hooks; but for new packages we recommend directly using the hook names and `\AddToHook`.

`\BeforeBeginEnvironment` `\BeforeBeginEnvironment [`⟨*label*⟩`] {`⟨*env*⟩`} {`⟨*code*⟩`}`

This declaration adds to the `env/`⟨*env*⟩`/before` hook using the ⟨*label*⟩. If ⟨*label*⟩ is not given, the ⟨*default label*⟩ is used (see section 2.1.5).

`\AtBeginEnvironment` `\AtBeginEnvironment [`⟨*label*⟩`] {`⟨*env*⟩`} {`⟨*code*⟩`}`

This is like `\BeforeBeginEnvironment` but it adds to the `env/`⟨*env*⟩`/begin` hook.

`\AtEndEnvironment` `\AtEndEnvironment [`⟨*label*⟩`] {`⟨*env*⟩`} {`⟨*code*⟩`}`

This is like `\BeforeBeginEnvironment` but it adds to the `env/`⟨*env*⟩`/end` hook.

`\AfterEndEnvironment` `\AfterEndEnvironment [`⟨*label*⟩`] {`⟨*env*⟩`} {`⟨*code*⟩`}`

This is like `\BeforeBeginEnvironment` but it adds to the `env/`⟨*env*⟩`/after` hook.

3.1.2 Generic hooks for commands

Similar to environments there are now (at least in theory) two generic hooks available for any LaTeX command. These are

`cmd/`⟨*name*⟩`/before` This hook is executed at the very start of the command execution.

`cmd/`⟨*name*⟩`/after` This hook is executed at the very end of the command body. It is implemented as a reversed hook.

In practice there are restrictions and especially the `after` hook works only with a subset of commands. Details about these restrictions are documented in `ltcmdhooks-doc.pdf` or with code in `ltcmdhooks-code.pdf`.

3.1.3 Generic hooks provided by file loading operations

There are several hooks added to LaTeX's process of loading file via its high-level interfaces such as `\input`, `\include`, `\usepackage`, `\RequirePackage`, etc. These are documented in `ltfilehook-doc.pdf` or with code in `ltfilehook-code.pdf`.

3.2 Hooks provided by \begin{document}

Until 2020 \begin{document} offered exactly one hook that one could add to using \AtBeginDocument. Experiences over the years have shown that this single hook in one place was not enough and as part of adding the general hook management system a number of additional hooks have been added at this point. The places for these hooks have been chosen to provide the same support as offered by external packages, such as etoolbox and others that augmented \document to gain better control.

Supported are now the following hooks (all of them one-time hooks):

begindocument/before This hook is executed at the very start of \document, one can think of it as a hook for code at the end of the preamble section and this is how it is used by etoolbox's \AtEndPreamble.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 2.5).

begindocument This hook is added to by using \AddToHook{begindocument} or by using \AtBeginDocument and it is executed after the .aux file has been read and most initialization are done, so they can be altered and inspected by the hook code. It is followed by a small number of further initializations that shouldn't be altered and are therefore coming later.

The hook should not be used to add material for typesetting as we are still in LaTeX's initialization phase and not in the document body. If such material needs to be added to the document body use the next hook instead.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 2.5).

begindocument/end This hook is executed at the end of the \document code in other words at the beginning of the document body. The only command that follows it is \ignorespaces.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 2.5).

The generic hooks executed by \begin also exist, i.e., env/document/before and env/document/begin, but with this special environment it is better use the dedicated one-time hooks above.

3.3 Hooks provided by \end{document}

LaTeX 2ε has always provided \AtEndDocument to add code to the \end{document}, just in front of the code that is normally executed there. While this was a big improvement over the situation in LaTeX 2.09, it was not flexible enough for a number of use cases and so packages, such as etoolbox, atveryend and others patched \enddocument to add additional points where code could be hooked into.

Patching using packages is always problematical as leads to conflicts (code availability, ordering of patches, incompatible patches, etc.). For this reason a number of additional hooks have been added to the \enddocument code to allow packages to add code in various places in a controlled way without the need for overwriting or patching the core code.

Supported are now the following hooks (all of them one-time hooks):

enddocument The hook associated with `\AtEndDocument`. It is immediately called at the beginning of `\enddocument`.

When this hook is executed there may be still unprocessed material (e.g., floats on the deferlist) and the hook may add further material to be typeset. After it, `\clearpage` is called to ensure that all such material gets typeset. If there is nothing waiting the `\clearpage` has no effect.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 2.5).

enddocument/afterlastpage As the name indicates this hook should not receive code that generates material for further pages. It is the right place to do some final housekeeping and possibly write out some information to the `.aux` file (which is still open at this point to receive data). Just before this hook `\write` is redefined and is now always `\immediate\write`. This means that writing e.g. a label or something to the `.toc` works despite the fact that no more pages are output. It is also the correct place to set up any testing code to be run when the `.aux` file is re-read in the next step.

After this hook has been executed the `.aux` file is closed for writing and then read back in to do some tests (e.g., looking for missing references or duplicated labels, etc.).

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 2.5).

enddocument/afteraux At this point, the `.aux` file has been reprocessed and so this is a possible place for final checks and display of information to the user. However, for the latter you might prefer the next hook, so that your information is displayed after the (possibly longish) list of files if that got requested via `\listfiles`.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 2.5).

enddocument/info This hook is meant to receive code that write final information messages to the terminal. It follows immediately after the previous hook (so both could have been combined, but then packages adding further code would always need to also supply an explicit rule to specify where it should go.

This hook already contains some code added by the kernel (under the labels `kernel/filelist` and `kernel/warnings`), namely the list of files when `\listfiles` has been used and the warnings for duplicate labels, missing references, font substitutions etc.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 2.5).

enddocument/end Finally, this hook is executed just in front of the final call to `\@@end`.

This is a one-time hook, so after it is executed, all further attempts to add code to it will execute such code immediately (see section 2.5).is it even possible to add code after this one?

There is also the hook `shipout/lastpage`. This hook is executed as part of the last `\shipout` in the document to allow package to add final `\special`'s to that page. Where this hook is executed in relation to those from the above list can vary from document to

document. Furthermore to determine correctly which of the `\shipout`s is the last one, LaTeX needs to be run several times, so initially it might get executed on the wrong page. See section 3.4 for where to find the details.

It is in also possible to use the generic `env/document/end` hook which is executed by `\end`, i.e., just in front of the first hook above. Note however that the other generic `\end` environment hook, i.e., `env/document/after` will never get executed, because by that time LaTeX has finished the document processing.

3.4 Hooks provided by `\shipout` operations

There are several hooks and mechanisms added to LaTeX's process of generating pages. These are documented in `ltshipout-doc.pdf` or with code in `ltshipout-code.pdf`.

3.5 Hooks provided for paragraphs

The paragraph processing has been augmented to include a number of internal and public hooks. These are documented in `ltpara-doc.pdf` or with code in `ltpara-code.pdf`.

3.6 Hooks provided in NFSS commands

In languages that need to support for more than one script in parallel (and thus several sets of fonts, e.g., supporting both Latin and Japanese fonts), NFSS font commands such as `\sffamily` need to switch both the Latin family to "Sans Serif" and in addition alter a second set of fonts.

To support this, several NFSS commands have hooks to which such support can be added.

`rmfamily` After `\rmfamily` has done its initial checks and prepared a font series update, this hook is executed before `\selectfont`.

`sffamily` This is like the `rmfamily` hook, but for the `\sffamily` command.

`ttfamily` This is like the `rmfamily` hook, but for the `\ttfamily` command.

`normalfont` The `\normalfont` command resets the font encoding, family, series and shape to their document defaults. It then executes this hook and finally calls `\selectfont`.

`expand@font@defaults` The internal `\expand@font@defaults` command expands and saves the current defaults for the metafamilies (rm/sf/tt) and the metaseries (bf/md). If the NFSS machinery has been augmented, e.g., for Chinese or Japanese fonts, then further defaults may need to be set at this point. This can be done in this hook which is executed at the end of this macro.

`bfseries/defaults`, `bfseries` If the `\bfdefault` was explicitly changed by the user, its new value is used to set the bf series defaults for the metafamilies (rm/sf/tt) when `\bfseries` is called. The `bfseries/defaults` hook allows further adjustments to be made in this case. This hook is only executed if such a change is detected. In contrast, the `bfseries` hook is always executed just before `\selectfont` is called to change to the new series.

`mdseries/defaults`, `mdseries` These two hooks are like the previous ones but they are in the `\mdseries` command.

selectfont This hook is executed inside `\selectfont`, after the current values for *encoding*, *family*, *series*, *shape*, and *size* are evaluated and the new font is selected (and if necessary loaded). After the hook has executed, NFSS will still do any updates necessary for a new *size* (such as changing the size of `\strut`) and any updates necessary to a change in *encoding*.

This hook is intended for use cases where, in parallel to a change in the main font, some other fonts need to be altered (e.g., in CJK processing where you may need to deal with several different alphabets).

3.7 Hook provided by the mark mechanism

See `ltmarks-doc.pdf` for details.

insertmark This hook allows for a special setup while `\InsertMark` inserts a mark. It is executed in group so local changes only apply to the mark being inserted.

4 The Implementation

```
⟨@@=hook⟩
⟨*2ekernel | latexrelease⟩
\ExplSyntaxOn
⟨latexrelease⟩\NewModuleRelease{2020/10/01}{lthooks}
⟨latexrelease⟩                    {The~hook~management~system}
```

4.1 Debugging

\g__hook_debug_bool

Holds the current debugging state.

```
\bool_new:N \g__hook_debug_bool
```

(*End of definition for* `\g__hook_debug_bool`.)

\hook_debug_on:
\hook_debug_off:
__hook_debug:n
__hook_debug_gset:

Turns debugging on and off by redefining `\__hook_debug:n`.

```
\cs_new_eq:NN \__hook_debug:n \use_none:n
\cs_new_protected:Npn \hook_debug_on:
  {
    \bool_gset_true:N \g__hook_debug_bool
    \__hook_debug_gset:
  }
\cs_new_protected:Npn \hook_debug_off:
  {
    \bool_gset_false:N \g__hook_debug_bool
    \__hook_debug_gset:
  }
\cs_new_protected:Npn \__hook_debug_gset:
  {
    \cs_gset_protected:Npe \__hook_debug:n ##1
      { \bool_if:NT \g__hook_debug_bool {##1} }
  }
```

(*End of definition for* `\hook_debug_on:` *and others. These functions are documented on page* 232.)

4.2 Borrowing from internals of other kernel modules

__hook_str_compare:nn Private copy of __str_if_eq:nn

```
\cs_new_eq:NN \__hook_str_compare:nn \__str_if_eq:nn
```

(*End of definition for* __hook_str_compare:nn.)

4.3 Declarations

\l__hook_tmpa_bool Scratch boolean used throughout the package.

```
\bool_new:N \l__hook_tmpa_bool
```

(*End of definition for* \l__hook_tmpa_bool.)

\l__hook_return_tl
\l__hook_tmpa_tl
\l__hook_tmpb_tl
Scratch variables used throughout the package.

```
\tl_new:N \l__hook_return_tl
\tl_new:N \l__hook_tmpa_tl
\tl_new:N \l__hook_tmpb_tl
```

(*End of definition for* \l__hook_return_tl, \l__hook_tmpa_tl, *and* \l__hook_tmpb_tl.)

\g__hook_all_seq In a few places we need a list of all hook names ever defined so we keep track if them in this sequence.

```
\seq_new:N \g__hook_all_seq
```

(*End of definition for* \g__hook_all_seq.)

\l__hook_cur_hook_tl Stores the name of the hook currently being sorted.

```
\tl_new:N \l__hook_cur_hook_tl
```

(*End of definition for* \l__hook_cur_hook_tl.)

\l__hook_work_prop A property list holding a copy of the \g__hook_⟨*hook*⟩_code_prop of the hook being sorted to work on, so that changes don't act destructively on the hook data structure.

```
\prop_new:N \l__hook_work_prop
```

(*End of definition for* \l__hook_work_prop.)

\g__hook_used_prop All hooks that receive code (for use in debugging display).

```
\prop_new:N \g__hook_used_prop
```

(*End of definition for* \g__hook_used_prop.)

\g__hook_hook_curr_name_tl
\g__hook_name_stack_seq
Default label used for hook commands, and a stack to keep track of packages within packages.

```
\tl_new:N \g__hook_hook_curr_name_tl
\seq_new:N \g__hook_name_stack_seq
```

(*End of definition for* \g__hook_hook_curr_name_tl *and* \g__hook_name_stack_seq.)

__hook_tmp:w Temporary macro for generic usage.

```
\cs_new_eq:NN \__hook_tmp:w ?
```

(*End of definition for* __hook_tmp:w.)

\c__hook_empty_tl
\c__hook_nine_parameters_tl

An empty token list, and one containing nine parameters.

```
\tl_const:Nn \c__hook_empty_tl { }
\tl_const:Nn \c__hook_nine_parameters_tl { #1#2#3#4#5#6#7#8#9 }
```

(*End of definition for* `\c__hook_empty_tl` *and* `\c__hook_nine_parameters_tl`.)

\str_count:e

Some variants of expl3 functions.

FMi: should probably be moved to expl3

```
\cs_generate_variant:Nn \str_count:n { e }
```

(*End of definition for* `\str_count:e`.)

\s__hook_mark

Scan mark used for delimited arguments.

```
\scan_new:N \s__hook_mark
```

(*End of definition for* `\s__hook_mark`.)

__hook_use_none_delimit_by_s_mark:w
__hook_use_i_delimit_by_s_mark:nw

Removes tokens until the next `\s__hook_mark`.

```
\cs_new:Npn \__hook_use_none_delimit_by_s_mark:w #1 \s__hook_mark { }
\cs_new:Npn \__hook_use_i_delimit_by_s_mark:nw #1 #2 \s__hook_mark {#1}
```

(*End of definition for* `\__hook_use_none_delimit_by_s_mark:w` *and* `\__hook_use_i_delimit_by_s_mark:nw`.)

__hook_tl_set:cn

Private copies of a few expl3 functions. l3debug will only add debugging to the public names, not to these copies, so we don't have to use `\debug_suspend:` and `\debug_resume:` everywhere.

Functions like `\__hook_tl_set:Nn` have to be redefined, rather than copied because in expl3 they use `\__kernel_tl_(g)set:Nx`, which is also patched by l3debug.

```
\cs_new_protected:Npn \__hook_tl_set:cn #1#2
  { \cs_set_nopar:cpe {#1} { \__kernel_exp_not:w {#2} } }
```

(*End of definition for* `\__hook_tl_set:cn`.)

__hook_tl_gset:Nn
__hook_tl_gset:Ne
__hook_tl_gset:cn
__hook_tl_gset:co
__hook_tl_gset:ce

Same as above.

```
\cs_new_protected:Npn \__hook_tl_gset:Nn #1#2
  { \cs_gset_nopar:Npe #1 { \__kernel_exp_not:w {#2} } }
\cs_new_protected:Npn \__hook_tl_gset:Ne #1#2
  { \cs_gset_nopar:Npe #1 {#2} }
\cs_generate_variant:Nn \__hook_tl_gset:Nn { c, co }
\cs_generate_variant:Nn \__hook_tl_gset:Ne { c }
```

(*End of definition for* `\__hook_tl_gset:Nn`.)

__hook_tl_gput_right:Nn
__hook_tl_gput_right:Ne
__hook_tl_gput_right:cn

Same as above.

```
\cs_new_protected:Npn \__hook_tl_gput_right:Nn #1#2
  { \__hook_tl_gset:Ne #1 { \__kernel_exp_not:w \exp_after:wN { #1 #2 } } }
\cs_generate_variant:Nn \__hook_tl_gput_right:Nn { Ne, cn }
```

(*End of definition for* `\__hook_tl_gput_right:Nn`.)

__hook_tl_gput_left:Nn Same as above.

```
\cs_new_protected:Npn \__hook_tl_gput_left:Nn #1#2
  {
    \__hook_tl_gset:Ne #1
      { \__kernel_exp_not:w {#2} \__kernel_exp_not:w \exp_after:wN {#1} }
  }
```

(*End of definition for* `\__hook_tl_gput_left:Nn`.)

__hook_tl_gset_eq:NN Same as above.

```
\cs_new_eq:NN \__hook_tl_gset_eq:NN \tl_gset_eq:NN
```

(*End of definition for* `\__hook_tl_gset_eq:NN`.)

__hook_tl_gclear:N
__hook_tl_gclear:c Same as above.

```
\cs_new_protected:Npn \__hook_tl_gclear:N #1
  { \__hook_tl_gset_eq:NN #1 \c_empty_tl }
\cs_generate_variant:Nn \__hook_tl_gclear:N { c }
```

(*End of definition for* `\__hook_tl_gclear:N`.)

4.4 Providing new hooks

4.4.1 The data structures of a hook

\g__hook_⟨*hook*⟩_code_prop
__hook␣⟨*hook*⟩
\g__hook_⟨*hook*⟩_reversed_tl
\g__hook_⟨*hook*⟩_declared_tl
\g__hook_⟨*hook*⟩_parameter_tl
__hook_next␣⟨*hook*⟩
__hook_toplevel␣⟨*hook*⟩

Hooks have a name (called ⟨*hook*⟩ in the description below) and for each hook we have to provide a number of data structures. These are

`\g__hook_`⟨*hook*⟩`_code_prop` A property list holding the code for the hook in separate chunks. The keys are by default the package names that add code to the hook, but it is possible for packages to define other keys.

`\g__hook_`⟨*hook*⟩`_rule_`⟨*label1*⟩`|`⟨*label2*⟩`_tl` A token list holding the relation between ⟨*label1*⟩ and ⟨*label2*⟩ in the ⟨*hook*⟩. The ⟨*labels*⟩ are lexically (reverse) sorted to ensure that two labels always point to the same token list. For global rules, the ⟨*hook*⟩ name is `??`.

`\__hook␣`⟨*hook*⟩ The code that is actually executed when the hook is called in the document is stored in this token list. It is constructed from the code chunks applying the information. This token list is named like that so that in case of an error inside the hook, the reported token list in the error is shorter, and to make it simpler to normalize hook names in `\__hook_make_name:n`.

`\g__hook_`⟨*hook*⟩`_reversed_tl` Some hooks are "reversed". This token list stores a `-` for such hook so that it can be identified. The `-` character is used because ⟨*reversed*⟩1 is +1 for normal hooks and −1 for reversed ones.

`\g__hook_`⟨*hook*⟩`_declared_tl` This token list serves as a marker for the hook being officially declared. Its existence is tested to raise an error in case another declaration is attempted.

`\c__hook_`⟨*hook*⟩`_parameter_tl` This token list stores the parameter text for a declared hook (its existence almost completely intersects the token list above), which is used for managing hooks with arguments.

`\__hook_toplevel␣`⟨*hook*⟩ This token list stores the code inserted in the hook from the user's document, in the `top-level` label. This label is special, and doesn't participate in sorting. Instead, all code is appended to it and executed after (or before, if the hook is reversed) the normal hook code, but before the `next` code chunk.

`\__hook_next␣`⟨*hook*⟩ Finally there is extra code (normally empty) that is used on the next invocation of the hook (and then deleted). This can be used to define some special behavior for a single occasion from within the document. This token list follows the same naming scheme than the main `\__hook␣`⟨*hook*⟩ token list. It is called `\__hook_next␣`⟨*hook*⟩ rather than `\__hook␣next_`⟨*hook*⟩ because otherwise a hook whose name is `next_`⟨*hook*⟩ would clash with the next code-token list of the hook called ⟨*hook*⟩.

4.4.2 On the existence of hooks

A hook may be in different states of existence. Here we give an overview of the internal commands to set up hooks and explain how the different states are distinguished. The actual implementation then follows in subsequent sections.

One problem we have to solve is that we need to be able to add code to hooks (e.g., with `\AddToHook`) even if that code has not yet been declared. For example, one package needs to write into a hook of another package, but that package may not get loaded, or is loaded only later. Another problem is that most hooks, but not the generic hooks, require a declaration.

We therefore distinguish the following states for a hook, which are managed by four different tests: structure existence (`\__hook_if_structure_exist:nTF`), creation (`\__hook_if_usable:nTF`), declaration (`\__hook_if_declared:nTF`) and disabled or not (`\__hook_if_disabled:nTF`)

not existing Nothing is known about the hook so far. This state can be detected with `\__hook_if_structure_exist:nTF` (which uses the false branch).

In this state the hook can be declared, disabled, rules can be defined or code could be added to it, but it is not possible to use the hook (with `\UseHook`).

basic data structure set up A hook is this state when its basic data structure has been set up (using `\__hook_init_structure:n`). The data structure setup happens automatically when commands such as `\AddToHook` are used and the hook is at that point in state "not existing".

In this state the four tests give the following results:

`\__hook_if_structure_exist:nTF` returns `true`.
`\__hook_if_usable:nTF` returns `false`.
`\__hook_if_declared:nTF` returns `false`.
`\__hook_if_disabled:nTF` returns `false`.

The allowed actions are the same as in the "not existing" state.

declared A hook is in this state it is not disabled and was explicitly declared (e.g., with `\NewHook`). In this case the four tests give the following results:

`\__hook_if_structure_exist:nTF` returns `true`.

`\__hook_if_usable:nTF` returns `true`.

`\__hook_if_declared:nTF` returns `true`.

`\__hook_if_disabled:nTF` returns `false`.

usable A hook is in this state if it is not disabled, was not explicitly declared but nevertheless is allowed to be used (with `\UseHook` or `\hook_use:n`). This state is only possible for generic hooks as they do not need to be declared. Therefore such hooks move directly from state "not existing" to "usable" the moment a declaration such as `\AddToHook` wants to add to the hook data structure. In this state the tests give the following results:

`\__hook_if_structure_exist:nTF` returns `true`.

`\__hook_if_usable:nTF` returns `true`.

`\__hook_if_declared:nTF` returns `false`.

`\__hook_if_disabled:nTF` returns `false`.

disabled A generic hook in any state is moved to this state when `\DisableGenericHook` is used. This changes the tests to give the following results:

`\__hook_if_structure_exist:nTF` *unchanged.*

`\__hook_if_usable:nTF` returns `false`.

`\__hook_if_declared:nTF` returns `true`.

`\__hook_if_disabled:nTF` returns `true`.

The structure test is unchanged (if the hook was unknown before it is `false`, otherwise `true`). The usable test returns `false` so that any `\UseHook` will bypass the hook from now on. The declared test returns true so that any further `\NewHook` generates an error and the disabled test returns true so that `\AddToHook` can return an error.

FMi: maybe it should do this only after begin document?

4.4.3 Setting hooks up

`\hook_new:n`
`\hook_new_with_args:nn`
`\__hook_new:nn`

The `\hook_new:n` declaration declares a new hook and expects the hook ⟨*name*⟩ as its argument, e.g., `begindocument`.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_new_with_args:nn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \hook_new:n #1
  { \__hook_normalize_hook_args:Nn \__hook_new:nn {#1} { 0 } }
\cs_new_protected:Npn \hook_new_with_args:nn #1 #2
  { \__hook_normalize_hook_args:Nn \__hook_new:nn {#1} {#2} }

\cs_new_protected:Npn \__hook_new:nn #1 #2
  {
```

We check if the hook was already *explicitly* declared with `\hook_new:n`, and if it already exists we complain, otherwise set the "created" flag for the hook so that it errors next time `\hook_new:n` is used.

```
    \__hook_if_declared:nTF {#1}
      { \msg_error:nnn { hooks } { exists } {#1} }
```

```
      {
        \tl_new:c { g__hook_#1_declared_tl }
        \cs_undefine:c { __hook~#1 }
        \cs_undefine:c { c__hook_#1_parameter_tl }
        \__hook_make_usable:nn {#1} {#2}
```

In case there is already code in a hook, but it's undeclared, run `\__hook_update_hook_code:n` to make it ready to be executed (see test `lthooks-034`).

```
        \__hook_update_hook_code:n {#1}
      }
  }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_new_with_args:nn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \hook_new:n #1
⟨latexrelease⟩  { \__hook_normalize_hook_args:Nn \__hook_new:n {#1} }
⟨latexrelease⟩\cs_undefine:N \__hook_new:nn
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_new:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_declared:nTF {#1}
⟨latexrelease⟩      { \msg_error:nnn { hooks } { exists } {#1} }
⟨latexrelease⟩      {
⟨latexrelease⟩        \tl_new:c { g__hook_#1_declared_tl }
⟨latexrelease⟩        \__hook_make_usable:n {#1}
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_gset_protected:Npn \hook_new_with_args:nn #1 { }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_new:n`, `\hook_new_with_args:nn`, *and* `\__hook_new:nn`. *These functions are documented on page* 229.)

`\__hook_make_usable:nn`

This initializes all hook data structures for the hook but if used on its own doesn't mark the hook as declared (as `\hook_new:n` does, so a later `\hook_new:n` on that hook will not result in an error. This command is internally used by `\hook_gput_code:nnn` when adding code to a generic hook.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_make_usable:nn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_make_usable:nn #1 #2
  {
```

Now we check if the hook's data structure can be safely created without expl3 raising errors, then we add the hook name to the list of all hooks and allocate the necessary data structures for the new hook, otherwise just do nothing.

```
    \__hook_if_usable:nF {#1}
      {
        \seq_gput_right:Nn \g__hook_all_seq {#1}
```

Here we'll define the `\c__hook_`⟨*hook*⟩`_parameter_tl` to hold a run of parameters up to the number of arguments of the hook (`#2`).

```
        \__kernel_cs_parm_from_arg_count:nnF
          { \tl_const:cn { c__hook_#1_parameter_tl } } {#2}
          {
            \msg_error:nnnn { hooks } { too-many-args } {#1} {#2}
```

```
            \tl_const:ce { c__hook_#1_parameter_tl }
              { \exp_not:V \c__hook_nine_parameters_tl }
          }
```

After that, use `\__hook_normalize_cs_args:nn` to correct the number of parameters of the macros `\__hook_toplevel`␣⟨*hook*⟩ and `\__hook_next`␣⟨*hook*⟩. We need to be able to add code with arguments to a hook without prior knowledge of the number of arguments of that hook, so lthooks assumes 9 until the hook is properly declared and the number of arguments is known. `\__hook_normalize_cs_args:nn` does the normalization by using the `\c__hook_`⟨*hook*⟩`_parameter_tl` defined just above.

```
        \__hook_normalize_cs_args:nn { _toplevel } {#1}
        \__hook_normalize_cs_args:nn { _next } {#1}
```

This is only used by the actual code of the current hook, so declare it normally:

```
        \__hook_code_gset:nn {#1} { }
```

Now ensure that the base data structure for the hook exists:

```
        \__hook_init_structure:n {#1}
```

The call to `\__hook_normalize_code_pool:n` will correct any improper reference to arguments that don't exist in the hook, raising a low-level TEX error and doubling the offending parameter tokens. It has to be done after `\__hook_init_structure:n` because it operates on `\g__hook_`⟨*hook*⟩`_code_prop`.

```
        \__hook_normalize_code_pool:n {#1}
```

The `\g__hook_`⟨*hook*⟩`_labels_clist` holds the sorted list of labels (once it got sorted). This is used only for debugging. These are defined conditionally, in case `\__hook_make_usable:nn` is being used to redefine a hook.

```
        \clist_if_exist:cF { g__hook_#1_labels_clist }
          {
            \clist_new:c { g__hook_#1_labels_clist }
```

Some hooks should reverse the default order of code chunks. To signal this we have a token list which is empty for normal hooks and contains a `-` for reversed hooks.

```
            \tl_new:c { g__hook_#1_reversed_tl }
          }
```

The above is all in L3 convention, but we also provide an interface to legacy LATEX 2_ε hooks of the form `\@...hook`, e.g., `\@begindocumenthook`. there have been a few of them and they have been added to using `\g@addto@macro`. If there exists such a macro matching the name of the new hook, i.e., `\@`⟨*hook-name*⟩`hook` and it is not empty then we add its contents as a code chunk under the label `legacy`.

Warning: this support will vanish in future releases!

```
        \__hook_include_legacy_code_chunk:n {#1}
      }
  }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_make_usable:nn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_make_usable:nn
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_make_usable:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_exist:cF { __hook~#1 }
⟨latexrelease⟩      {
```

```
⟨latexrelease⟩          \seq_gput_right:Nn \g__hook_all_seq {#1}
⟨latexrelease⟩          \tl_new:c { __hook~#1 }
⟨latexrelease⟩          \__hook_init_structure:n {#1}
⟨latexrelease⟩          \clist_new:c { g__hook_#1_labels_clist }
⟨latexrelease⟩          \tl_new:c { g__hook_#1_reversed_tl }
⟨latexrelease⟩          \__hook_include_legacy_code_chunk:n {#1}
⟨latexrelease⟩        }
⟨latexrelease⟩    }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_make_usable:nn`.)

`\__hook_init_structure:n`

This function declares the basic data structures for a hook without explicit declaring the hook itself. This is needed to allow adding to undeclared hooks. Here it is unnecessary to check whether all variables exist, since all three are declared at the same time (either all of them exist, or none).

It creates the hook code pool (`\g__hook_`⟨*hook*⟩`_code_prop`) and the `top-level` and `next` token lists. A hook is initialized with `\__hook_init_structure:n` the first time anything is added to it. Initializing a hook just with `\__hook_init_structure:n` will not make it usable with `\hook_use:n`.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_init_structure:n}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_init_structure:n #1
  {
    \__hook_if_structure_exist:nF {#1}
      {
        \prop_new:c { g__hook_#1_code_prop }
        \__hook_toplevel_gset:nn {#1} { }
        \__hook_next_gset:nn {#1} { }
      }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_init_structure:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_init_structure:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_structure_exist:nF {#1}
⟨latexrelease⟩      {
⟨latexrelease⟩        \prop_new:c { g__hook_#1_code_prop }
⟨latexrelease⟩        \tl_new:c { __hook_toplevel~#1 }
⟨latexrelease⟩        \tl_new:c { __hook_next~#1 }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_init_structure:n`.)

`\hook_new_reversed:n`
`\hook_new_reversed_with_args:nn`
`\__hook_new_reversed:nn`

Declare a new hook. The default ordering of code chunks is reversed, signaled by setting the token list to a minus sign.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_new_reversed_with_args:nn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \hook_new_reversed:n #1
  { \__hook_normalize_hook_args:Nn \__hook_new_reversed:nn {#1} { 0 } }
```

```
\cs_new_protected:Npn \hook_new_reversed_with_args:nn #1 #2
  { \__hook_normalize_hook_args:Nn \__hook_new_reversed:nn {#1} {#2} }
\cs_new_protected:Npn \__hook_new_reversed:nn #1 #2
  {
    \__hook_if_declared:nTF {#1}
      { \msg_error:nnn { hooks } { exists } {#1} }
      {
        \__hook_new:nn {#1} {#2}
        \tl_gset:cn { g__hook_#1_reversed_tl } { - }
      }
  }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_new_reversed_with_args:nn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \hook_new_reversed:n #1
⟨latexrelease⟩  { \__hook_normalize_hook_args:Nn \__hook_new_reversed:n {#1} }
⟨latexrelease⟩\cs_undefine:N \__hook_new_reversed:nn
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_new_reversed:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_new:n {#1}
⟨latexrelease⟩    \tl_gset:cn { g__hook_#1_reversed_tl } { - }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_undefine:N \__hook_new_reversed:nn
⟨latexrelease⟩\cs_gset_protected:Npn \hook_new_reversed_with_args:nn #1 #2 { }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_new_reversed:n`, `\hook_new_reversed_with_args:nn`, *and* `\__hook_new_reversed:nn`. *These functions are documented on page* 229.)

`\hook_new_pair:nn`
`\hook_new_pair_with_args:nnn`

A shorthand for declaring a normal and a (matching) reversed hook in one go.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_new_pair_with_args:nnn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \hook_new_pair:nn #1#2
  { \__hook_normalize_hook_args:Nnn \__hook_new_pair:nnn {#1} {#2} { 0 } }
\cs_new_protected:Npn \hook_new_pair_with_args:nnn #1#2#3
  { \__hook_normalize_hook_args:Nnn \__hook_new_pair:nnn {#1} {#2} {#3} }
\cs_new_protected:Npn \__hook_new_pair:nnn #1 #2 #3
  {
    \__hook_if_declared:nTF {#1}
      { \msg_error:nnn { hooks } { exists } {#1} }
      {
        \__hook_if_declared:nTF {#2}
          { \msg_error:nnn { hooks } { exists } {#2} }
          {
            \__hook_new:nn {#1} {#3}
            \__hook_new_reversed:nn {#2} {#3}
          }
      }
  }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_new_pair_with_args:nnn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \hook_new_pair:nn #1#2
⟨latexrelease⟩  {
```

```
⟨latexrelease⟩    \hook_new:n {#1}
⟨latexrelease⟩    \hook_new_reversed:n {#2}
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_gset_protected:Npn \hook_new_pair_with_args:nnn #1#2#3
⟨latexrelease⟩  { }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_new_pair:nn` *and* `\hook_new_pair_with_args:nnn`. *These functions are documented on page* 229.)

`\__hook_include_legacy_code_chunk:n`

The LaTeX legacy concept for hooks uses with hooks the following naming scheme in the code: `\@...hook`.

If this macro is not empty we add it under the label `legacy` to the current hook and then empty it globally. This way packages or classes directly manipulating commands such as `\@begindocumenthook` still get their hook data added.

Warning: this support will vanish in future releases!

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_include_legacy_code_chunk:n}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_include_legacy_code_chunk:n #1
  {
```

If the macro doesn't exist (which is the usual case) then nothing needs to be done.

```
    \tl_if_exist:cT { @#1hook }
      {
```

Of course if the legacy hook exists but is empty, there is no need to add anything under `legacy` the legacy label.

```
        \tl_if_empty:cF { @#1hook }
          {
```

Here we set `\__hook_replacing_args_false:` because no legacy code will reference hook arguments.

```
            \__hook_replacing_args_false:
            \use:e
              {
                \__hook_hook_gput_code_do:nnn {#1} { legacy }
                  { \exp_not:v { @#1hook } }
              }
            \__hook_replacing_args_reset:
```

Once added to the hook, we need to clear it otherwise it might get added again later if the hook data gets updated.

```
            \__hook_tl_gclear:c { @#1hook }
          }
      }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_include_legacy_code_chunk:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_include_legacy_code_chunk:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_exist:cT { @#1hook }
⟨latexrelease⟩      {
⟨latexrelease⟩        \tl_if_empty:cF { @#1hook }
```

```
⟨latexrelease⟩           {
⟨latexrelease⟩             \exp_args:Nnnv \__hook_hook_gput_code_do:nnn
⟨latexrelease⟩               {#1} { legacy } { @#1hook }
⟨latexrelease⟩             \__hook_tl_gclear:c { @#1hook }
⟨latexrelease⟩           }
⟨latexrelease⟩       }
⟨latexrelease⟩   }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_include_legacy_code_chunk:n`.)

4.4.4 Disabling and providing hooks

`\hook_disable_generic:n`
`\__hook_disable:n`
`\__hook_if_disabled_p:n`
`\__hook_if_disabled:nTF`

Disables a hook by creating its `\g__hook_⟨hook⟩_declared_tl` so that the hook errors when used with `\hook_new:n`, then it undefines `\__hook␣⟨hook⟩` so that it may not be executed.

This does not clear any code that may be already stored in the hook's structure, but doesn't allow adding more code. `\__hook_if_disabled:nTF` uses that specific combination to check if the hook is disabled.

```
⟨latexrelease⟩\IncludeInRelease{2021/06/01}{\hook_disable_generic:n}
⟨latexrelease⟩                  {Disable~hooks}
\cs_new_protected:Npn \hook_disable_generic:n #1
  { \__hook_normalize_hook_args:Nn \__hook_disable:n {#1} }
\cs_new_protected:Npn \__hook_disable:n #1
  {
    \tl_gclear_new:c { g__hook_#1_declared_tl }
    \cs_undefine:c { __hook~#1 }
  }
\prg_new_conditional:Npnn \__hook_if_disabled:n #1 { p, T, F, TF }
  {
    \bool_lazy_and:nnTF
        { \tl_if_exist_p:c { g__hook_#1_declared_tl } }
        { ! \cs_if_exist_p:c { __hook~#1 } }
      { \prg_return_true: }
      { \prg_return_false: }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_disable_generic:n}
⟨latexrelease⟩                  {Disable~hooks}
⟨latexrelease⟩
⟨latexrelease⟩\cs_new_protected:Npn \hook_disable_generic:n #1 {}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_disable_generic:n`, `\__hook_disable:n`, *and* `\__hook_if_disabled:nTF`. *This function is documented on page* 230.)

`\hook_activate_generic:n`
`\__hook_activate_generic:n`

The `\hook_activate_generic:n` declaration declares a new hook if it wasn't declared already, in which case it only checks that the already existing hook is not a reversed hook.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_activate_generic:n}
⟨latexrelease⟩                  {Providing~hooks}
```

```
\cs_new_protected:Npn \hook_activate_generic:n #1
  { \__hook_normalize_hook_args:Nn \__hook_activate_generic:nn {#1} {   } }
\cs_new_protected:Npn \__hook_activate_generic:nn #1 #2
  {
```

If the hook to be activated was disabled we warn (for now — this may change).

```
    \__hook_if_disabled:nTF {#1}
      { \msg_warning:nnn { hooks } { activate-disabled } {#1} }
```

Otherwise we check if the hook is not declared, and if it isn't, figure out if it's reversed or not, then declare it accordingly.

```
      {
        \__hook_if_declared:nF {#1}
          {
            \tl_new:c { g__hook_#1_declared_tl }
            \__hook_make_usable:nn {#1} { 0 }
            \tl_gset:ce { g__hook_#1_reversed_tl }
              { \__hook_if_generic_reversed:nT {#1} { - } }
```

Reflect that we have activated the generic hook and set its execution code.

```
            \__hook_update_hook_code:n {#1}
          }
      }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/06/01}{\hook_activate_generic:n}
⟨latexrelease⟩                  {Providing~hooks}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_activate_generic:nn #1 #2
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_disabled:nTF {#1}
⟨latexrelease⟩      { \msg_warning:nnn { hooks } { activate-disabled } {#1} }
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_if_declared:nF {#1}
⟨latexrelease⟩          {
⟨latexrelease⟩            \tl_new:c { g__hook_#1_declared_tl }
⟨latexrelease⟩            \__hook_make_usable:n {#1}
⟨latexrelease⟩            \tl_gset:cx { g__hook_#1_reversed_tl }
⟨latexrelease⟩              { \__hook_if_generic_reversed:nT {#1} { - } }
⟨latexrelease⟩            \__hook_update_hook_code:n {#1}
⟨latexrelease⟩          }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_activate_generic:n}
⟨latexrelease⟩                  {Providing~hooks}
⟨latexrelease⟩\cs_gset_protected:Npn \hook_activate_generic:n #1 { }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_activate_generic:n` *and* `\__hook_activate_generic:n`. *This function is documented on page* 230.)

4.5 Parsing a label

__hook_parse_label_default:nN

This macro checks if a label was given (not `\c_novalue_tl`), and if so, tries to parse the label looking for a leading `.` to replace by `\__hook_currname_or_default:`. `#2` is a boolean representing if `#1` is a label name.

```
\cs_new:Npn \__hook_parse_label_default:nN #1#2
  {
    \tl_if_novalue:nTF {#1}
      { \__hook_currname_or_default: }
      { \tl_trim_spaces_apply:nN {#1} \__hook_parse_dot_label:nN #2 }
  }
```

(*End of definition for* `\__hook_parse_label_default:nN`.)

__hook_parse_dot_label:nN
__hook_parse_dot_label:w
__hook_parse_dot_label_cleanup:w
__hook_parse_dot_label_aux:w

Start by checking if the label is empty, which raises an error, and uses the fallback value. If not, split the label at a `./`, if any, and check if no tokens are before the `./`, or if the only character is a `.`. If these requirements are fulfilled, the leading `.` is replaced with `\__hook_currname_or_default:`. Otherwise the label is returned unchanged. `#2` is a boolean representing if `#1` is a label name.

```
\cs_new:Npn \__hook_parse_dot_label:nN #1#2
  {
    \tl_if_empty:nTF {#1}
      {
        \bool_if:NTF #2
          { \msg_expandable_error:nn { hooks } { empty-label } }
          { \msg_expandable_error:nn { hooks } { empty-hook } }
        \__hook_currname_or_default:
      }
      {
        \str_if_eq:nnTF {#1} { . }
          { \__hook_currname_or_default: }
          { \__hook_parse_dot_label:w #1 ./ \s__hook_mark }
      }
  }
\cs_new:Npn \__hook_parse_dot_label:w #1 ./ #2 \s__hook_mark
  {
    \tl_if_empty:nTF {#1}
      { \__hook_parse_dot_label_aux:w #2 \s__hook_mark }
      {
        \tl_if_empty:nTF {#2}
          { \__hook_make_name:n {#1} }
          { \__hook_parse_dot_label_cleanup:w #1 ./ #2 \s__hook_mark }
      }
  }
\cs_new:Npn \__hook_parse_dot_label_cleanup:w #1 ./ \s__hook_mark {#1}
\cs_new:Npn \__hook_parse_dot_label_aux:w #1 ./ \s__hook_mark
  { \__hook_currname_or_default: / \__hook_make_name:n {#1} }
```

(*End of definition for* `\__hook_parse_dot_label:nN` *and others.*)

__hook_currname_or_default:

This uses `\g__hook_hook_curr_name_tl` if it is set, otherwise it tries `\@currname`. If neither is set, it raises an error and uses the fallback value `label-missing`.

```
\cs_new:Npn \__hook_currname_or_default:
  {
```

```
    \tl_if_empty:NTF \g__hook_hook_curr_name_tl
      {
        \tl_if_empty:NTF \@currname
          {
            \msg_expandable_error:nnn { latex2e } { should-not-happen }
              { Empty~default~label. }
            \__hook_make_name:n { label-missing }
          }
          { \@currname }
      }
      { \g__hook_hook_curr_name_tl }
  }
```

(*End of definition for* `\__hook_currname_or_default:`.)

`\__hook_make_name:n`
`\__hook_make_name:w`

This provides a standard sanitization of a hook's name. It uses `\cs:w` to build a control sequence out of the hook name, then uses `\cs_to_str:N` to get the string representation of that, without the escape character. `\cs:w`-based expansion is used instead of `e`-based because Unicode characters don't behave well inside `\expanded`. The macro adds the `\__hook␣` prefix to the hook name to reuse the hook's code token list to build the csname and avoid leaving "public" control sequences defined (as `\relax`) in TeX's memory.

```
\cs_new:Npn \__hook_make_name:n #1
  {
    \exp_after:wN \exp_after:wN \exp_after:wN \__hook_make_name:w
    \exp_after:wN \token_to_str:N \cs:w __hook~ #1 \cs_end:
  }
\exp_last_unbraced:NNNNo
\cs_new:Npn \__hook_make_name:w #1 \tl_to_str:n { __hook~ } { }
```

(*End of definition for* `\__hook_make_name:n` *and* `\__hook_make_name:w`.)

`\__hook_normalize_hook_args:Nn`
`\__hook_normalize_hook_args:Nnn`
`\__hook_normalize_hook_rule_args:Nnnnn`
`\__hook_normalize_hook_args_aux:Nn`

This is the standard route for normalizing hook and label arguments. The main macro does the entire operation within a group so that csnames made by `\__hook_make_name:n` are wiped off before continuing. This means that this function cannot be used for `\hook_use:n`!

```
\cs_new_protected:Npn \__hook_normalize_hook_args_aux:Nn #1 #2
  {
    \group_begin:
    \use:e
      {
        \group_end:
        \exp_not:N #1 #2
      }
  }
\cs_new_protected:Npn \__hook_normalize_hook_args:Nn #1 #2
  {
    \__hook_normalize_hook_args_aux:Nn #1
      { { \__hook_parse_label_default:nN {#2} \c_false_bool } }
  }
\cs_new_protected:Npn \__hook_normalize_hook_args:Nnn #1 #2 #3
  {
    \__hook_normalize_hook_args_aux:Nn #1
      {
        { \__hook_parse_label_default:nN {#2} \c_false_bool }
```

```
        { \__hook_parse_label_default:nN {#3} \c_true_bool }
      }
  }
\cs_new_protected:Npn \__hook_normalize_hook_rule_args:Nnnnn #1 #2 #3 #4 #5
  {
    \__hook_normalize_hook_args_aux:Nn #1
      {
        { \__hook_parse_label_default:nN {#2} \c_false_bool }
        { \__hook_parse_label_default:nN {#3} \c_true_bool }
        { \tl_trim_spaces:n {#4} }
        { \__hook_parse_label_default:nN {#5} \c_true_bool }
      }
  }
```

(*End of definition for* `\__hook_normalize_hook_args:Nn` *and others.*)

`\__hook_curr_name_push:n`
`\__hook_curr_name_push_aux:n`
`\__hook_curr_name_pop:`
`\__hook_end_document_label_check:`

The token list `\g__hook_hook_curr_name_tl` stores the name of the current package/file to be used as the default label in hooks. Providing a consistent interface is tricky because packages can be loaded within packages, and some packages may not use `\SetDefaultHookLabel` to change the default label (in which case `\@currname` is used).

To pull that one off, we keep a stack that contains the default label for each level of input. The bottom of the stack contains the default label for the `top-level` (this stack should never go empty). If we're building the format, set the default label to be `top-level`:

```
\tl_gset:Nn \g__hook_hook_curr_name_tl { top-level }
```

Then, in case we're in latexrelease we push something on the stack to support roll forward. But in some rare cases, latexrelease may be loaded inside another package (notably platexrelease), so we'll first push the `top-level` entry:

```
⟨latexrelease⟩\seq_if_empty:NT \g__hook_name_stack_seq
⟨latexrelease⟩  { \seq_gput_right:Nn \g__hook_name_stack_seq { top-level } }
```

then we dissect the `\@currnamestack`, adding `\@currname` to the stack:

```
⟨latexrelease⟩\cs_set_protected:Npn \__hook_tmp:w #1 #2 #3
⟨latexrelease⟩  {
⟨latexrelease⟩    \quark_if_recursion_tail_stop:n {#1}
⟨latexrelease⟩    \seq_gput_right:Nn \g__hook_name_stack_seq {#1}
⟨latexrelease⟩    \__hook_tmp:w
⟨latexrelease⟩  }
⟨latexrelease⟩\exp_after:wN \__hook_tmp:w \@currnamestack
⟨latexrelease⟩  \q_recursion_tail \q_recursion_tail
⟨latexrelease⟩  \q_recursion_tail \q_recursion_stop
```

and finally set the default label to be the `\@currname`:

```
⟨latexrelease⟩\tl_gset:Nx \g__hook_hook_curr_name_tl { \@currname }
⟨latexrelease⟩\seq_gpop_right:NN \g__hook_name_stack_seq \l__hook_tmpa_tl
```

Two commands keep track of the stack: when a file is input, `\__hook_curr_name_push:n` pushes the current default label onto the stack and sets the new default label (all in one go):

```
\cs_new_protected:Npn \__hook_curr_name_push:n #1
  {
      \exp_args:Ne \__hook_curr_name_push_aux:n
        {
          \tl_if_blank:nF {#1}
```

```
            { \__hook_parse_dot_label:nN {#1} \c_false_bool }
        }
  }
\cs_new_protected:Npn \__hook_curr_name_push_aux:n #1
  {
    \tl_if_blank:nTF {#1}
      { \msg_error:nn { hooks } { no-default-label } }
      {
        \str_if_eq:nnTF {#1} { top-level }
          {
            \msg_error:nnnnn { hooks } { set-top-level }
              { to } { PushDefaultHookLabel } {#1}
          }
          {
            \seq_gpush:NV \g__hook_name_stack_seq \g__hook_hook_curr_name_tl
            \tl_gset:Nn \g__hook_hook_curr_name_tl {#1}
          }
      }
  }
```

and when an input is over, the topmost item of the stack is popped, since that label will not be used again, and `\g__hook_hook_curr_name_tl` is updated to equal the now topmost item of the stack:

```
\cs_new_protected:Npn \__hook_curr_name_pop:
  {
    \seq_gpop:NNTF \g__hook_name_stack_seq \l__hook_return_tl
      { \tl_gset_eq:NN \g__hook_hook_curr_name_tl \l__hook_return_tl }
      { \msg_error:nn { hooks } { extra-pop-label } }
  }
```

At the end of the document we want to check if there was no `\__hook_curr_name_push:n` without a matching `\__hook_curr_name_pop:` (not a critical error, but it might indicate that something else is not quite right):

```
\tl_gput_right:Nn \@kernel@after@enddocument@afterlastpage
  { \__hook_end_document_label_check: }
\cs_new_protected:Npn \__hook_end_document_label_check:
  {
    \seq_gpop:NNT \g__hook_name_stack_seq \l__hook_return_tl
      {
        \msg_error:nne { hooks } { missing-pop-label }
          { \g__hook_hook_curr_name_tl }
        \tl_gset_eq:NN \g__hook_hook_curr_name_tl \l__hook_return_tl
        \__hook_end_document_label_check:
      }
  }
```

The token list `\g__hook_hook_curr_name_tl` is but a mirror of the top of the stack.

Now define a wrapper that replaces the top of the stack with the argument, and updates `\g__hook_hook_curr_name_tl` accordingly.

```
\cs_new_protected:Npn \__hook_set_default_hook_label:n #1
  {
    \seq_if_empty:NTF \g__hook_name_stack_seq
      {
        \msg_error:nnnnn { hooks } { set-top-level }
```

`\__hook_set_default_hook_label:n`
`\__hook_set_default_label:n`

```
          { for } { SetDefaultHookLabel } {#1}
      }
      {
          \exp_args:Ne \__hook_set_default_label:n
            {
              \tl_if_blank:nF {#1}
                { \__hook_parse_dot_label:nN {#1} \c_false_bool }
            }
      }
  }
\cs_new_protected:Npn \__hook_set_default_label:n #1
  {
    \str_if_eq:nnTF {#1} { top-level }
      {
        \msg_error:nnnnn { hooks } { set-top-level }
          { to } { SetDefaultHookLabel } {#1}
      }
      { \tl_gset:Nn \g__hook_hook_curr_name_tl {#1} }
  }
```

(*End of definition for* `\__hook_curr_name_push:n` *and others.*)

4.6 Adding or removing hook code

`\hook_gput_code:nnn`
`\hook_gput_code_with_args:nnn`
`\__hook_gput_code:nnn`
`\__hook_gput_code_store:nnn`
`\__hook_hook_gput_code_do:nnn`
`\__hook_prop_gput_labeled_cleanup:nnn`
`\__hook_prop_gput_labeled_do:Nnnn`
`\__hook_hash_check:nTF`
`\__hook_hash_check_aux:w`

With `\hook_gput_code:nnn{`⟨*hook*⟩`}{`⟨*label*⟩`}{`⟨*code*⟩`}` a chunk of ⟨*code*⟩ is added to an existing ⟨*hook*⟩ labeled with ⟨*label*⟩.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_gput_code:nnn}
⟨latexrelease⟩                          {Hooks~with~args}
\cs_new_protected:Npn \hook_gput_code:nnn #1 #2 #3
  {
    \__hook_replacing_args_false:
    \__hook_normalize_hook_args:Nnn \__hook_gput_code:nnn {#1} {#2} {#3}
    \__hook_replacing_args_reset:
  }
\cs_new_protected:Npn \hook_gput_code_with_args:nnn #1 #2 #3
  {
    \__hook_replacing_args_true:
    \__hook_normalize_hook_args:Nnn \__hook_gput_code:nnn {#1} {#2} {#3}
    \__hook_replacing_args_reset:
  }
```

If `\AddToHookWithArguments` was used, do some sanity checking, and if it's not possible to use arguments at this point, fall back to regular `\AddToHook` by using `\__hook_replacing_args_false:`.

```
\cs_new_protected:Npn \__hook_gput_code:nnn #1 #2 #3
  {
    \__hook_chk_args_allowed:nn {#1} { AddToHook }
```

Then check if the code should be executed immediately, rather than stored:

```
    \__hook_if_execute_immediately:nTF {#1}
      {
```

`\AddToHookWithArguments` can't be used on one-time hooks (that were already used).

```
        \__hook_if_replacing_args:TF
          {
```

```
            \msg_error:nnnn { hooks } { one-time-args }
              {#1} { AddToHook }
          }
          { }
        \use:n
      }
      { \__hook_gput_code_store:nnn {#1} {#2} }
        {#3}
  }
\cs_new_protected:Npn \__hook_gput_code_store:nnn #1 #2 #3
  {
```

Then check if the hook is usable.

```
    \__hook_if_usable:nTF {#1}
```

If so we simply add (or append) the new code to the property list holding different chunks for the hook. At `\begin{document}` this is then sorted into a token list for fast execution.

```
      {
        \__hook_hook_gput_code_do:nnn {#1} {#2} {#3}
```

However, if there is an update within the document we need to alter this execution code which is done by `\__hook_update_hook_code:n`. In the preamble this does nothing.

```
        \__hook_update_hook_code:n {#1}
      }
```

If the hook is not usable, before giving up, check if it's not disabled and otherwise try to declare it as a generic hook, if its name matches one of the valid patterns.

```
      {
        \__hook_if_disabled:nTF {#1}
          { \msg_error:nnn { hooks } { hook-disabled } {#1} }
          { \__hook_try_declaring_generic_hook:nnn {#1} {#2} {#3} }
      }
  }
```

This macro will unconditionally add a chunk of code to the given hook.

```
\cs_new_protected:Npn \__hook_hook_gput_code_do:nnn #1 #2 #3
  {
```

However, first some debugging info if debugging is enabled:

```
    \__hook_debug:n{\iow_term:e{[lthooks]~ Add~ to~
                      \__hook_if_usable:nF {#1} { undeclared~ }
                      hook~ '#1'~ (#2) \on@line
                      ^^J [lthooks] \@spaces
                      <-~ \tl_to_str:n{#3}} }
```

Then try to get the code chunk labeled `#2` from the hook. If there's code already there, then append `#3` to that, otherwise just put `#3`. If the current label is `top-level`, the code is added to a dedicated token list `\__hook_toplevel␣`⟨*hook*⟩ that goes at the end of the hook (or at the beginning, for a reversed hook), just before `\__hook_next␣`⟨*hook*⟩.

```
    \str_if_eq:nnTF {#2} { top-level }
      {
        \str_if_eq:eeTF { top-level } { \__hook_currname_or_default: }
          {
```

If the hook's basic structure does not exist, we need to declare it with `\__hook_init_structure:n`.

```
            \__hook_init_structure:n {#1}
```

Then append to the `_toplevel` container for the hook.

```
            \__hook_cs_gput_right:nnn { _toplevel } {#1} {#3}
          }
          { \msg_error:nnn { hooks } { misused-top-level } {#1} }
      }
      {
```

When adding to the code pool, we have to double hashes if `\AddToHook` was used (`replacing_args` is false), so that later it is turned into a single parameter token, rather than a parameter to the hook macro. We skip this step if there are no hashes at all in the argument: the token-by-token approach otherwise becomes a major performance issue when the contents of the hook are long.

```
        \exp_args:Ne \__hook_prop_gput_labeled_cleanup:nnn
          {
            \__hook_if_replacing_args:TF
              { \exp_not:n }
              {
                \__hook_hash_check:nTF {#3}
                  { \exp_not:n }
                  { \__hook_double_hashes:n }
              }
                {#3}
          }
          {#1} {#2}
      }
  }
\cs_new:Npe \__hook_hash_check:nTF #1
  {
    \exp_not:N \exp_after:wN \exp_not:N \__hook_hash_check_aux:w
      \exp_not:N \tl_to_str:n {#1} \c_hash_str \c_hash_str
      \exp_not:N \q_stop
  }
\use:e
  {
    \cs_new:Npn \exp_not:N \__hook_hash_check_aux:w
      #1 \c_hash_str #2 \c_hash_str #3 \exp_not:N \q_stop
  }
  { \tl_if_empty:nTF {#3} }
```

Adds code to a hook's code pool.

```
\cs_new_protected:Npn \__hook_prop_gput_labeled_cleanup:nnn #1 #2 #3
  {
    \tl_set:Nn \l__hook_return_tl {#1}
    \__hook_if_replacing_args:TF
      {
        \__hook_if_usable:nT {#2}
          {
            \__hook_set_normalize_fn:nn {#2}
              { Invalid~code~added~\msg_line_context: }
            \__hook_normalize_fn:nn {#3} {#1}
            \prop_get:NnN \l__hook_work_prop {#3} \l__hook_return_tl
```

```
          }
      }
      { }
    \exp_args:NcV \__hook_prop_gput_labeled_do:Nnn
      { g__hook_#2_code_prop } \l__hook_return_tl {#3}
  }
\cs_new_protected:Npn \__hook_prop_gput_labeled_do:Nnn #1 #2 #3
  {
    \prop_get:NnNTF #1 {#3} \l__hook_return_tl
      { \prop_gput:Nno #1 {#3} { \l__hook_return_tl #2 } }
      { \prop_gput:Nnn #1 {#3} {#2} }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_gput_code:nnn}
⟨latexrelease⟩                  {Providing~hooks}
⟨latexrelease⟩\cs_gset_protected:Npn \hook_gput_code:nnn #1 #2
⟨latexrelease⟩  { \__hook_normalize_hook_args:Nnn
⟨latexrelease⟩        \__hook_gput_code:nnn {#1} {#2} }
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_gput_code:nnn #1 #2 #3
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_execute_immediately:nTF {#1}
⟨latexrelease⟩      {#3}
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_if_usable:nTF {#1}
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_hook_gput_code_do:nnn {#1} {#2} {#3}
⟨latexrelease⟩            \__hook_update_hook_code:n {#1}
⟨latexrelease⟩          }
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_if_disabled:nTF {#1}
⟨latexrelease⟩              { \msg_error:nnn { hooks } { hook-disabled } {#1} }
⟨latexrelease⟩              { \__hook_try_declaring_generic_hook:nnn
⟨latexrelease⟩                  {#1} {#2} {#3} }
⟨latexrelease⟩          }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_hook_gput_code_do:nnn #1 #2 #3
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_debug:n{\iow_term:x{****~ Add~ to~
⟨latexrelease⟩                      \__hook_if_usable:nF {#1} { undeclared~ }
⟨latexrelease⟩                      hook~ #1~ (#2)
⟨latexrelease⟩                      \on@line\space <-~ \tl_to_str:n{#3}} }
⟨latexrelease⟩    \str_if_eq:nnTF {#2} { top-level }
⟨latexrelease⟩      {
⟨latexrelease⟩        \str_if_eq:eeTF { top-level }
⟨latexrelease⟩                        { \__hook_currname_or_default: }
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_init_structure:n {#1}
⟨latexrelease⟩            \__hook_tl_gput_right:cn { __hook_toplevel~#1 } {#3}
⟨latexrelease⟩          }
⟨latexrelease⟩          { \msg_error:nnn { hooks } { misused-top-level } {#1} }
⟨latexrelease⟩      }
⟨latexrelease⟩      {
```

```
⟨latexrelease⟩          \prop_get:cnNTF
⟨latexrelease⟩            { g__hook_#1_code_prop } {#2} \l__hook_return_tl
⟨latexrelease⟩            {
⟨latexrelease⟩              \prop_gput:cno { g__hook_#1_code_prop } {#2}
⟨latexrelease⟩                { \l__hook_return_tl #3 }
⟨latexrelease⟩            }
⟨latexrelease⟩            { \prop_gput:cnn { g__hook_#1_code_prop } {#2} {#3} }
⟨latexrelease⟩        }
⟨latexrelease⟩    }
⟨latexrelease⟩\cs_gset_protected:Npn \hook_gput_code_with_args:nnn #1#2#3 { }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_gput_code:nnn` *and others. These functions are documented on page* 231.)

`\__hook_chk_args_allowed:nn`

This macro checks if it is possible to add code with references to a hook's arguments for hook `#1`. It only does something if the function being run is `replacing_args`. This macro will error if the hook is declared and takes no arguments, then it will set `\__hook_replacing_args_false:` so that the macro which called it will add the code normally.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_chk_args_allowed:nn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_chk_args_allowed:nn #1 #2
  {
    \__hook_if_replacing_args:TF
      {
        \__hook_if_declared:nT {#1}
          { \tl_if_empty:cT { c__hook_#1_parameter_tl } { \use_ii:nn } }
        \use_none:n
          {
            \msg_error:nnnn { hooks } { without-args } {#1} {#2}
            \__hook_replacing_args_false:
          }
      }
      { }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_chk_args_allowed:nn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_chk_args_allowed:nn
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_chk_args_allowed:nn`.)

`\__hook_gput_undeclared_hook:nnn`

Often it may happen that a package A defines a hook `foo`, but package B, that adds code to that hook, is loaded before A. In such case we need to add code to the hook before it is declared. An implicitly declared hook doesn't have arguments (in principle), so use `\c_false_bool` here.

```
\cs_new_protected:Npn \__hook_gput_undeclared_hook:nnn #1 #2 #3
  {
    \__hook_init_structure:n {#1}
    \__hook_hook_gput_code_do:nnn {#1} {#2} {#3}
  }
```

(*End of definition for* `\__hook_gput_undeclared_hook:nnn`.)

__hook_try_declaring_generic_hook:nnn
__hook_try_declaring_generic_next_hook:nn

These entry-level macros just pass the arguments along to the common `\__hook_try_declaring_generic_hook:nNNnn` with the right functions to execute when some action is to be taken.

The wrapper `\__hook_try_declaring_generic_hook:nnn` then defers `\hook_gput_code:nnn` if the generic hook was declared, or to `\__hook_gput_undeclared_hook:nnn` otherwise (the hook was tested for existence before, so at this point if it isn't generic, it doesn't exist).

The wrapper `\__hook_try_declaring_generic_next_hook:nn` for next-execution hooks does the same: it defers the code to `\hook_gput_next_code:nn` if the generic hook was declared, or to `\__hook_gput_next_do:nn` otherwise.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}
⟨latexrelease⟩                  {\__hook_try_declaring_generic_hook:nnn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_try_declaring_generic_hook:nnn #1
  {
    \__hook_try_declaring_generic_hook:wnTF #1 / / / \scan_stop: {#1}
      \__hook_gput_code:nnn
      \__hook_gput_undeclared_hook:nnn
        {#1}
  }
\cs_new_protected:Npn \__hook_try_declaring_generic_next_hook:nn #1
  {
    \__hook_try_declaring_generic_hook:wnTF #1 / / / \scan_stop: {#1}
      \__hook_gput_next_code:nn
      \__hook_gput_next_do:nn
        {#1}
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/11/15}
⟨latexrelease⟩                  {\__hook_try_declaring_generic_hook:nnn}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_try_declaring_generic_hook:nnn #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_try_declaring_generic_hook:wnTF #1 / / / \scan_stop:
⟨latexrelease⟩      {#1}
⟨latexrelease⟩      \hook_gput_code:nnn
⟨latexrelease⟩      \__hook_gput_undeclared_hook:nnn
⟨latexrelease⟩        {#1}
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_gset_protected:Npn
⟨latexrelease⟩  \__hook_try_declaring_generic_next_hook:nn #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_try_declaring_generic_hook:wnTF #1 / / / \scan_stop:
⟨latexrelease⟩      {#1}
⟨latexrelease⟩      \hook_gput_next_code:nn
⟨latexrelease⟩      \__hook_gput_next_do:nn
⟨latexrelease⟩        {#1}
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}
⟨latexrelease⟩                  {\__hook_try_declaring_generic_hook:nnn}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\cs_new_protected:Npn
```

```
⟨latexrelease⟩  \__hook_try_declaring_generic_hook:nnn #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_try_declaring_generic_hook:nNNnn {#1}
⟨latexrelease⟩      \hook_gput_code:nnn \__hook_gput_undeclared_hook:nnn
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new_protected:Npn
⟨latexrelease⟩  \__hook_try_declaring_generic_next_hook:nn #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_try_declaring_generic_hook:nNNnn {#1}
⟨latexrelease⟩      \hook_gput_next_code:nn \__hook_gput_next_do:nn
⟨latexrelease⟩  }
```

(*End of definition for* `\__hook_try_declaring_generic_hook:nnn` *and* `\__hook_try_declaring_generic_next_hook:nn`.)

`\__hook_try_declaring_generic_hook:nNNnn`
`\__hook_try_declaring_generic_hook_split:nNNnn`

`\__hook_try_declaring_generic_hook:nNNnn` now splits the hook name at the first `/` (if any) and first checks if it is a file-specific hook (they require some normalization) using `\__hook_if_file_hook:wTF`. If not then check it is one of a predefined set for generic names. We also split off the second component to see if we have to make a reversed hook. In either case the function returns ⟨*true*⟩ for a generic hook and ⟨*false*⟩ in other cases.

```
⟨latexrelease⟩\cs_new_protected:Npn \__hook_try_declaring_generic_hook:nNNnn #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_file_hook:wTF #1 / / \s__hook_mark
⟨latexrelease⟩      {
⟨latexrelease⟩        \exp_args:Ne
⟨latexrelease⟩        \__hook_try_declaring_generic_hook_split:nNNnn
⟨latexrelease⟩          { \exp_args:Ne \__hook_file_hook_normalize:n {#1} }
⟨latexrelease⟩      }
⟨latexrelease⟩      { \__hook_try_declaring_generic_hook_split:nNNnn {#1} }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new_protected:Npn
⟨latexrelease⟩    \__hook_try_declaring_generic_hook_split:nNNnn #1 #2 #3
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_try_declaring_generic_hook:wnTF #1 / / / \scan_stop:
⟨latexrelease⟩      {#1}
⟨latexrelease⟩      { #2 }
⟨latexrelease⟩      { #3 } {#1}
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_try_declaring_generic_hook:nNNnn` *and* `\__hook_try_declaring_generic_hook_split:nNNnn`.)

`\__hook_try_declaring_generic_hook:wn`*TF*

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}
⟨latexrelease⟩                  {\__hook_try_declaring_generic_hook:wn}
⟨latexrelease⟩                  {Hooks~with~args}
\prg_new_protected_conditional:Npnn
    \__hook_try_declaring_generic_hook:wn
    #1 / #2 / #3 / #4 \scan_stop: #5 { TF }
  {
    \__hook_if_generic:nTF {#5}
      {
        \__hook_if_usable:nF {#5}
          {
```

If the hook doesn't exist yet we check if it is a `cmd` hook and if so we attempt patching the command in addition to declaring the hook.

For some commands this will not be possible, in which case `\__hook_patch_cmd_or_delay:Nnn` (defined in `ltcmdhooks`) will generate an appropriate error message.

```
            \str_if_eq:nnT {#1} { cmd }
              {
                \__hook_try_put_cmd_hook:n {#5}
                \__hook_make_usable:nn {#5} { 9 }
                \use_none:nnn
              }
```

Declare the hook always even if it can't really be used (error message generated elsewhere).

Here we use `\__hook_make_usable:nn`, so that a `\hook_new:n` is still possible later. Generic hooks (except `cmd` hooks) take no arguments, so use zero as the second argument.

```
            \__hook_make_usable:nn {#5} { 0 }
          }
        \__hook_if_generic_reversed:nT {#5}
          { \tl_gset:cn { g__hook_#5_reversed_tl } { - } }
        \prg_return_true:
      }
      {
```

Generic hooks are all named ⟨*type*⟩/⟨*name*⟩/⟨*place*⟩, where ⟨*type*⟩ and ⟨*place*⟩ are predefined (`\c__hook_generic_`⟨*type*⟩`/./`⟨*place*⟩`_tl`), and ⟨*name*⟩ is the variable component. Older releases had some hooks with the ⟨*name*⟩ in the third part, so the code below supports that syntax for a while, with a warning.

The `\exp_after:wN ... \exp:w` trick is there to remove the conditional structure inserted by `\__hook_try_declaring_generic_hook:wnTF` and thus allow access to the tokens that follow it, as is needed to keep things going.

When the deprecation cycle ends, the lines below should all be replaced by `\prg_return_false:`.

```
        \__hook_if_deprecated_generic:nTF {#5}
          {
            \__hook_deprecated_generic_warn:n {#5}
            \exp_after:wN \__hook_declare_deprecated_generic:NNn
            \exp:w % \exp_end:
          }
          { \prg_return_false: }
      }
  }
```

`\__hook_deprecated_generic_warn:n` will issue a deprecation warning for a given hook, and mark that hook such that the warning will not be issued again (multiple warnings can be issued, but only once per hook).

__hook_deprecated_generic_warn:Nn
__hook_deprecated_generic_warn:Nw

```
\cs_new_protected:Npn \__hook_deprecated_generic_warn:n #1
  { \__hook_deprecated_generic_warn:w #1 \s__hook_mark }
\cs_new_protected:Npn \__hook_deprecated_generic_warn:w
    #1 / #2 / #3 \s__hook_mark
  {
    \if_cs_exist:w __hook~#1/#2/#3 \cs_end: \else:
      \msg_warning:nnnnn { hooks } { generic-deprecated } {#1} {#2} {#3}
    \fi:
```

```
    \cs_gset_eq:cN { __hook~#1/#2/#3 } \scan_stop:
  }
```

Now that the user has been told about the deprecation, we proceed by swapping ⟨*name*⟩ and ⟨*place*⟩ and adding the code to the correct hook.

__hook_do_deprecated_generic:Nn
__hook_do_deprecated_generic:Nw
__hook_declare_deprecated_generic:NNw
__hook_declare_deprecated_generic:NNw

```
\cs_new_protected:Npn \__hook_do_deprecated_generic:Nn #1 #2
  { \__hook_do_deprecated_generic:Nw #1 #2 \s__hook_mark }
\cs_new_protected:Npn \__hook_do_deprecated_generic:Nw #1
         #2 / #3 / #4 \s__hook_mark
  { #1 { #2 / #4 / #3 } }
\cs_new_protected:Npn \__hook_declare_deprecated_generic:NNn #1 #2 #3
  { \__hook_declare_deprecated_generic:NNw #1 #2 #3 \s__hook_mark }
\cs_new_protected:Npn \__hook_declare_deprecated_generic:NNw #1 #2
    #3 / #4 / #5 \s__hook_mark
  {
    \__hook_try_declaring_generic_hook:wnTF #3 / #5 / #4 / \scan_stop:
        { #3 / #5 / #4 }
      #1 #2 { #3 / #5 / #4 }
  }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2021/11/15}
⟨latexrelease⟩                  {\__hook_try_declaring_generic_hook:wn}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\prg_new_protected_conditional:Npnn
⟨latexrelease⟩    \__hook_try_declaring_generic_hook:wn
⟨latexrelease⟩    #1 / #2 / #3 / #4 \scan_stop: #5 { TF }
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_generic:nTF {#5}
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_if_usable:nF {#5}
⟨latexrelease⟩          {
⟨latexrelease⟩            \str_if_eq:nnT {#1} { cmd }
⟨latexrelease⟩              { \__hook_try_put_cmd_hook:n {#5} }
⟨latexrelease⟩            \__hook_make_usable:n {#5}
⟨latexrelease⟩          }
⟨latexrelease⟩        \__hook_if_generic_reversed:nT {#5}
⟨latexrelease⟩          { \tl_gset:cn { g__hook_#5_reversed_tl } { - } }
⟨latexrelease⟩        \prg_return_true:
⟨latexrelease⟩      }
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_if_deprecated_generic:nTF {#5}
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_deprecated_generic_warn:n {#5}
⟨latexrelease⟩            \exp_after:wN \__hook_declare_deprecated_generic:NNn
⟨latexrelease⟩            \exp:w % \exp_end:
⟨latexrelease⟩          }
⟨latexrelease⟩          { \prg_return_false: }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2021/06/01}
⟨latexrelease⟩                  {\__hook_try_declaring_generic_hook:wn}
⟨latexrelease⟩                  {Support~cmd~hooks}
```

```
⟨latexrelease⟩\prg_new_protected_conditional:Npnn
⟨latexrelease⟩    \__hook_try_declaring_generic_hook:wn
⟨latexrelease⟩    #1 / #2 / #3 / #4 \scan_stop: #5 { TF }
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_empty:nTF {#2}
⟨latexrelease⟩      { \prg_return_false: }
⟨latexrelease⟩      {
⟨latexrelease⟩        \prop_if_in:NnTF \c__hook_generics_prop {#1}
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_if_usable:nF {#5}
⟨latexrelease⟩              {
⟨latexrelease⟩                \str_if_eq:nnT {#1} { cmd }
⟨latexrelease⟩                  { \__hook_try_put_cmd_hook:n {#5} }
⟨latexrelease⟩                \__hook_make_usable:n {#5}
⟨latexrelease⟩              }
⟨latexrelease⟩            \prop_if_in:NnTF
⟨latexrelease⟩              \c__hook_generics_reversed_ii_prop {#2}
⟨latexrelease⟩              { \tl_gset:cn { g__hook_#5_reversed_tl } { - } }
⟨latexrelease⟩              {
⟨latexrelease⟩                \prop_if_in:NnT
⟨latexrelease⟩                  \c__hook_generics_reversed_iii_prop {#3}
⟨latexrelease⟩                  { \tl_gset:cn { g__hook_#5_reversed_tl } { - } }
⟨latexrelease⟩              }
⟨latexrelease⟩            \prg_return_true:
⟨latexrelease⟩          }
⟨latexrelease⟩          { \prg_return_false: }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/10/01}
⟨latexrelease⟩                 {\__hook_try_declaring_generic_hook:wn}
⟨latexrelease⟩                 {Support~cmd~hooks}
⟨latexrelease⟩\prg_new_protected_conditional:Npnn
⟨latexrelease⟩    \__hook_try_declaring_generic_hook:wn
⟨latexrelease⟩    #1 / #2 / #3 / #4 \scan_stop: #5 { TF }
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_empty:nTF {#2}
⟨latexrelease⟩      { \prg_return_false: }
⟨latexrelease⟩      {
⟨latexrelease⟩        \prop_if_in:NnTF \c__hook_generics_prop {#1}
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_if_declared:nF {#5} { \hook_new:n {#5} }
⟨latexrelease⟩            \prop_if_in:NnTF
⟨latexrelease⟩              \c__hook_generics_reversed_ii_prop {#2}
⟨latexrelease⟩              { \tl_gset:cn { g__hook_#5_reversed_tl } { - } }
⟨latexrelease⟩              {
⟨latexrelease⟩                \prop_if_in:NnT
⟨latexrelease⟩                  \c__hook_generics_reversed_iii_prop {#3}
⟨latexrelease⟩                  { \tl_gset:cn { g__hook_#5_reversed_tl } { - } }
⟨latexrelease⟩              }
⟨latexrelease⟩            \prg_return_true:
⟨latexrelease⟩          }
⟨latexrelease⟩          { \prg_return_false: }
⟨latexrelease⟩      }
```

```
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_try_declaring_generic_hook:wnTF` *and others.*)

`\__hook_if_file_hook_p:w`
`\__hook_if_file_hook:wTF`

`\__hook_if_file_hook:wTF` checks if the argument is a valid file-specific hook (not, for example, `file/before`, but `file/foo.tex/before`). If it is a file-specific hook, then it executes the ⟨*true*⟩ branch, otherwise ⟨*false*⟩.

```
⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\__hook_if_file_hook:w}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_if_file_hook:w}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\prg_new_conditional:Npnn \__hook_if_file_hook:w
⟨latexrelease⟩    #1 / #2 / #3 \s__hook_mark { TF }
⟨latexrelease⟩  {
⟨latexrelease⟩    \str_if_eq:nnTF {#1} { file }
⟨latexrelease⟩      {
⟨latexrelease⟩        \bool_lazy_or:nnTF
⟨latexrelease⟩            { \tl_if_empty_p:n {#3} }
⟨latexrelease⟩            { \str_if_eq_p:nn {#3} { / } }
⟨latexrelease⟩          { \prg_return_false: }
⟨latexrelease⟩          {
⟨latexrelease⟩            \prop_if_in:NnTF \c__hook_generics_file_prop {#2}
⟨latexrelease⟩              { \prg_return_true: }
⟨latexrelease⟩              { \prg_return_false: }
⟨latexrelease⟩          }
⟨latexrelease⟩      }
⟨latexrelease⟩      { \prg_return_false: }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_if_file_hook:wTF`.)

`\__hook_file_hook_normalize:n`
`\__hook_strip_double_slash:n`
`\__hook_strip_double_slash:w`

```
⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\__hook_file_hook_normalize:n}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\EndIncludeInRelease
```

When a file-specific hook is found, before being declared it is lightly normalized by `\__hook_file_hook_normalize:n`. The current implementation just replaces two consecutive slashes (`//`) by a single one, to cope with simple cases where the user did something like `\def\input@path{{./mypath/}}`, in which case a hook would have to be `\AddToHook{file/./mypath//file.tex/after}`.

```
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_file_hook_normalize:n}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\cs_new:Npn \__hook_file_hook_normalize:n #1
⟨latexrelease⟩  { \__hook_strip_double_slash:n {#1} }
⟨latexrelease⟩\cs_new:Npn \__hook_strip_double_slash:n #1
⟨latexrelease⟩  { \__hook_strip_double_slash:w #1 // \s__hook_mark }
```

This function is always called after testing if the argument is a file hook with `\__hook_if_file_hook:wTF`, so we can assume it has three parts (it is either `file/.../before` or `file/.../after`), so we use `#1/#2/#3 //` instead of just `#1 //` to prevent losing a slash if the file name is empty.

```
⟨latexrelease⟩\cs_new:Npn \__hook_strip_double_slash:w #1/#2/#3//#4\s__hook_mark
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_empty:nTF {#4}
⟨latexrelease⟩      { #1/#2/#3 }
⟨latexrelease⟩      { \__hook_strip_double_slash:w #1/#2/#3 /#4\s__hook_mark }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_file_hook_normalize:n`, `\__hook_strip_double_slash:n`, *and* `\__hook_strip_double_slash:w`.)

`\c__hook_generic_cmd/./before_tl` `\c__hook_generic_cmd/./after_tl` `\c__hook_generic_env/./before_tl` `\c__hook_generic_env/./after_tl` `\c__hook_generic_file/./before_tl` `\c__hook_generic_file/./after_tl` `\c__hook_generic_package/./before_tl` `\c__hook_generic_package/./after_tl` `\c__hook_generic_class/./before_tl` `\c__hook_generic_class/./after_tl` `\c__hook_generic_include/./before_tl` `\c__hook_generic_include/./after_tl` `\c__hook_generic_env/./begin_tl` `\c__hook_generic_env/./end_tl` `\c__hook_generic_include/./end_tl`

Token lists defining the possible generic hooks. We don't provide any user interface to this as this is meant to be static.

cmd The generic hooks used for commands.

env The generic hooks used in `\begin` and `\end`.

file, package, class, include The generic hooks used when loading a file

```
⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\c__hook_generics_prop}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
\clist_map_inline:nn { cmd , env , file , package , class , include }
  {
    \tl_const:cn { c__hook_generic_#1/./before_tl } { + }
    \tl_const:cn { c__hook_generic_#1/./after_tl  } { - }
  }
\tl_const:cn { c__hook_generic_env/./begin_tl } { + }
\tl_const:cn { c__hook_generic_env/./end_tl   } { + }

\tl_const:cn { c__hook_generic_include/./end_tl } { - }
\tl_const:cn { c__hook_generic_include/./excluded_tl } { + }
```

Deprecated generic hooks:

```
\clist_map_inline:nn { file , package , class , include }
  {
    \tl_const:cn { c__hook_deprecated_#1/./before_tl } { }
    \tl_const:cn { c__hook_deprecated_#1/./after_tl  } { }
  }
\tl_const:cn { c__hook_deprecated_include/./end_tl } { }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\c__hook_generics_prop}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\prop_const_from_keyval:Nn \c__hook_generics_prop
⟨latexrelease⟩     {cmd=,env=,file=,package=,class=,include=}
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\c__hook_generic_cmd/./before_tl` *and others.*)

`\c__hook_generics_reversed_ii_prop` `\c__hook_generics_reversed_iii_prop` `\c__hook_generics_file_prop`

The following generic hooks are supposed to use reverse ordering (the `ii` and `iii` names are kept for the deprecation cycle):

```
⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\c__hook_generics_reversed_ii_prop}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\c__hook_generics_reversed_ii_prop}
⟨latexrelease⟩                    {Standardize~generic~hook~names}
⟨latexrelease⟩\prop_const_from_keyval:Nn
⟨latexrelease⟩     \c__hook_generics_reversed_ii_prop {after=,end=}
⟨latexrelease⟩\prop_const_from_keyval:Nn
⟨latexrelease⟩     \c__hook_generics_reversed_iii_prop {after=}
⟨latexrelease⟩\prop_const_from_keyval:Nn
⟨latexrelease⟩     \c__hook_generics_file_prop {before=,after=}
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\c__hook_generics_reversed_ii_prop`, `\c__hook_generics_reversed_iii_prop`, *and* `\c__hook_generics_file_prop`.)

`\c__hook_parameter_cmd/./before_tl`
`\c__hook_parameter_cmd/./after_tl`

Token lists defining the number of arguments for a given type of generic hook.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\c__hook_parameter_cmd/./before_tl}
⟨latexrelease⟩                    {Hooks~with~args}
```

`cmd` hooks are declared with 9 arguments because they have a variable number of arguments (depending on the command they are attached to), so we use the maximum here.

```
\tl_const:cn { c__hook_parameter_cmd/./before_tl } { #1#2#3#4#5#6#7#8#9 }
\tl_const:cn { c__hook_parameter_cmd/./after_tl }  { #1#2#3#4#5#6#7#8#9 }

⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\c__hook_parameter_cmd/./before_tl}
⟨latexrelease⟩                    {Hooks~with~args}
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\c__hook_parameter_cmd/./before_tl` *and* `\c__hook_parameter_cmd/./after_tl`.)

`\hook_gremove_code:nn`
`\__hook_gremove_code:nn`

With `\hook_gremove_code:nn{`⟨*hook*⟩`}{`⟨*label*⟩`}` any code for ⟨*hook*⟩ stored under ⟨*label*⟩ is removed.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_gremove_code:nn}
⟨latexrelease⟩                    {Hooks~with~args}
\cs_new_protected:Npn \hook_gremove_code:nn #1 #2
  { \__hook_normalize_hook_args:Nnn \__hook_gremove_code:nn {#1} {#2} }
\cs_new_protected:Npn \__hook_gremove_code:nn #1 #2
  {
```

First check that the hook code pool exists. `\__hook_if_usable:nTF` isn't used here because it should be possible to remove code from a hook before its defined (see section 2.1.8).

```
    \__hook_if_structure_exist:nTF {#1}
      {
```

Then remove the chunk and run `\__hook_update_hook_code:n` so that the execution token list reflects the change if we are after `\begin{document}`.

If all code is to be removed, clear the code pool `\g__hook_`⟨*hook*⟩`_code_prop`, the top-level code `\__hook_toplevel_`⟨*hook*⟩, and the next-execution code `\__hook_next_`⟨*hook*⟩.

```
        \str_if_eq:nnTF {#2} {*}
          {
            \prop_gclear:c { g__hook_#1_code_prop }
            \__hook_toplevel_gset:nn {#1} { }
            \__hook_next_gset:nn {#1} { }
```

```
        }
        {
```

If the label is `top-level` then clear the token list, as all code there is under the same label.

```
          \str_if_eq:nnTF {#2} { top-level }
            { \__hook_toplevel_gset:nn {#1} { } }
            {
              \prop_gpop:cnNF { g__hook_#1_code_prop }
                {#2} \l__hook_return_tl
                { \msg_warning:nnnn { hooks } { cannot-remove } {#1} {#2} }
            }
        }
```

Finally update the code, if the hook exists.

```
      \__hook_if_usable:nT {#1}
        { \__hook_update_hook_code:n {#1} }
    }
```

If the code pool for this hook doesn't exist, show a warning:

```
    {
      \__hook_if_deprecated_generic:nTF {#1}
        {
          \__hook_deprecated_generic_warn:n {#1}
          \__hook_do_deprecated_generic:Nn
              \__hook_gremove_code:nn {#1} {#2}
        }
        { \msg_warning:nnnn { hooks } { cannot-remove } {#1} {#2} }
    }
}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_gremove_code:nn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_new_protected:Npn \__hook_gremove_code:nn #1 #2
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_structure_exist:nTF {#1}
⟨latexrelease⟩      {
⟨latexrelease⟩        \str_if_eq:nnTF {#2} {*}
⟨latexrelease⟩          {
⟨latexrelease⟩            \prop_gclear:c { g__hook_#1_code_prop }
⟨latexrelease⟩            \__hook_tl_gclear:c { __hook_toplevel~#1 }
⟨latexrelease⟩            \__hook_tl_gclear:c { __hook_next~#1 }
⟨latexrelease⟩          }
⟨latexrelease⟩          {
⟨latexrelease⟩            \str_if_eq:nnTF {#2} { top-level }
⟨latexrelease⟩              { \__hook_tl_gclear:c { __hook_toplevel~#1 } }
⟨latexrelease⟩              {
⟨latexrelease⟩                \prop_gpop:cnNF { g__hook_#1_code_prop }
⟨latexrelease⟩                  {#2} \l__hook_return_tl
⟨latexrelease⟩                  { \msg_warning:nnnn { hooks } { cannot-remove }
⟨latexrelease⟩                                      {#1} {#2} }
⟨latexrelease⟩              }
⟨latexrelease⟩          }
⟨latexrelease⟩        \__hook_if_usable:nT {#1}
⟨latexrelease⟩          { \__hook_update_hook_code:n {#1} }
```

```
⟨latexrelease⟩      }
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_if_deprecated_generic:nTF {#1}
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_deprecated_generic_warn:n {#1}
⟨latexrelease⟩            \__hook_do_deprecated_generic:Nn
⟨latexrelease⟩                \__hook_gremove_code:nn {#1} {#2}
⟨latexrelease⟩          }
⟨latexrelease⟩          { \msg_warning:nnnn { hooks } { cannot-remove }
⟨latexrelease⟩                              {#1} {#2} }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_gremove_code:nn` *and* `\__hook_gremove_code:nn`. *This function is documented on page 231.*)

`\__hook_cs_gput_right:nnn`
`\__hook_cs_gput_right_fast:nnn`
`\__hook_cs_gput_right_slow:nnn`
`\__hook_code_gset_auxi:nnnn`
`\__hook_code_gset_auxi:eeen`

This macro is used to append code to the `toplevel` and `next` token lists, treating them correctly depending on their number of arguments, and depending on whether the code being added should have parameter tokens understood as parameters, or doubled to be stored as parameter tokens.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_cs_gput_right:nnn}
⟨latexrelease⟩                  {Hooks~with~args}
```

Check if the current hook is declared and takes no arguments. In this case, we short-circuit and use the simpler and much faster approach that doesn't require hash-doubling.

```
\cs_new_protected:Npn \__hook_cs_gput_right:nnn #1 #2
  {
    \if:w T
        \__hook_if_declared:nF {#2} { F }
        \tl_if_empty:cF { c__hook_#2_parameter_tl } { F }
          T
      \exp_after:wN \__hook_cs_gput_right_fast:nnn
    \else:
      \exp_after:wN \__hook_cs_gput_right_slow:nnn
    \fi:
        {#1} {#2}
  }
\cs_new_protected:Npn \__hook_cs_gput_right_fast:nnn #1 #2 #3
  { \cs_gset:cpe { __hook#1~#2 }
      { \exp_not:v { __hook#1~#2 } \exp_not:n {#3} } }
\cs_new_protected:Npn \__hook_cs_gput_right_slow:nnn #1 #2 #3
  {
```

The auxiliary `\__hook_code_gset_auxi:eeen` just does the assignment at the end. Its first argument is the parameter text of the macro, which is chosen here depending if `\c__hook_`⟨*hook*⟩`_parameter_tl` exists, if the hook is declared, and if it's a generic hook.

```
    \cs_if_exist:cF { __hook#1~#2 }
      { \__hook_code_gset_aux:nnn {#1} {#2} { } }
    \__hook_code_gset_auxi:eeen
      {
        \__hook_if_declared:nTF {#2}
          { \tl_use:c { c__hook_#2_parameter_tl } }
          {
            \__hook_if_generic:nTF {#2}
```

```
            { \__hook_generic_parameter:n {#2} }
            { \c__hook_nine_parameters_tl }
        }
    }
```

Here we take the existing code in the macro, expand it with as many arguments as it takes, then double the hashes so the code can be reused.

PhO: Maybe can be improved. The case of adding to an empty cs can be optimized by quickly checking `\cs_replacement_spec`.

```
    {
      \exp_args:NNo \exp_args:No \__hook_double_hashes:n
        {
          \cs:w __hook#1~#2 \exp_last_unbraced:Ne \cs_end:
            { \__hook_braced_cs_parameter:n { __hook#1~#2 } }
        }
    }
```

Now the new code: if we are replacing arguments, then hashes are left untouched, otherwise they are doubled.

```
    {
      \__hook_if_replacing_args:TF
        { \exp_not:n }
        { \__hook_double_hashes:n }
        {#3}
    }
```

And finally, the csname which we'll define with all the above.

```
    { __hook#1~#2 }
}
```

And as promised, the auxiliary that does the definition.

```
\cs_new_protected:Npn \__hook_code_gset_auxi:nnnn #1 #2 #3 #4
  { \cs_gset:cpn {#4} #1 { #2 #3 } }
\cs_generate_variant:Nn \__hook_code_gset_auxi:nnnn { eeen }

⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_cs_gput_right:nnn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_cs_gput_right:nnn
⟨latexrelease⟩\cs_undefine:N \__hook_cs_gput_right_fast:nnn
⟨latexrelease⟩\cs_undefine:N \__hook_cs_gput_right_slow:nnn
⟨latexrelease⟩\cs_undefine:N \__hook_code_gset_auxi:nnnn
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_cs_gput_right:nnn` *and others.*)

`\__hook_code_gset:nn`
`\__hook_code_gset:ne`
`\__hook_toplevel_gset:nn`
`\__hook_next_gset:nn`
`\__hook_code_gset_aux:nnn`

These macros define `\__hook`⟨*type*⟩`_`⟨*hook*⟩ (with ⟨*type*⟩ being `_next`, `_toplevel`, or empty) with the given code and the parameters stored in `\c__hook_`⟨*hook*⟩`_parameter_tl` (or none, if that doesn't exist).

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_code_gset:nn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_code_gset:nn
  { \__hook_code_gset_aux:nnn { } }
\cs_new_protected:Npn \__hook_toplevel_gset:nn
  { \__hook_code_gset_aux:nnn { _toplevel } }
\cs_new_protected:Npn \__hook_next_gset:nn
  { \__hook_code_gset_aux:nnn { _next } }
\cs_new_protected:Npn \__hook_code_gset_aux:nnn #1 #2 #3
```

```
  {
    \cs_gset:cpn { __hook#1~#2 \exp_last_unbraced:Ne }
      { \__hook_parameter:n {#2} }
      {#3}
  }
\cs_generate_variant:Nn \__hook_code_gset:nn { ne }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_code_gset:nn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_code_gset:nn
⟨latexrelease⟩\cs_undefine:N \__hook_toplevel_gset:nn
⟨latexrelease⟩\cs_undefine:N \__hook_next_gset:nn
⟨latexrelease⟩\cs_undefine:N \__hook_code_gset_aux:nnn
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_code_gset:nn` *and others.*)

`\__hook_normalize_cs_args:nn`

This macro normalizes the parameters of the macros `\__hook`⟨*type*⟩`␣`⟨*hook*⟩ to take the right number of arguments after a hook is declared. At this point we know `\c__hook_`⟨*hook*⟩`_parameter_tl` exists, so use that to count the arguments and use that as ⟨*parameter text*⟩ for the newly (re)defined macro.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_normalize_cs_args:nn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_normalize_cs_args:nn #1 #2
  {
    \cs_if_exist:cT { __hook#1~#2 }
      {
        \__hook_code_gset_auxi:eeen
          { \tl_use:c { c__hook_#2_parameter_tl } }
          {
            \exp_args:NNo \exp_args:No \__hook_double_hashes:n
              {
                \cs:w __hook#1~#2 \exp_last_unbraced:Ne \cs_end:
                  { \__hook_braced_cs_parameter:n { __hook#1~#2 } }
              }
          }
          { }
          { __hook#1~#2 }
      }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_normalize_cs_args:nn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_normalize_cs_args:nn
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_normalize_cs_args:nn`.)

`\__hook_normalize_code_pool:n`
`\__hook_set_normalize_fn:nn`

This one's a bit of a hack. It takes a hook, and iterates over its code pool (`\g__hook_`⟨*hook*⟩`_code_prop`), redefining each code label to use only valid arguments. This is used when, for example, a code is added referencing arguments `#1` and `#2`, but the hook has only `#1`. In this example, every reference to `#2` is changed to `##2`. This is done because

otherwise TEX will throw a low-level error every time some change happens to the hook (code is added, a rule is set, etc), which can get quite repetitive for no good reason.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_normalize_code_pool:n}
⟨latexrelease⟩                        {Hooks~with~args}
\cs_new_protected:Npn \__hook_normalize_code_pool:n #1
  {
```

First, call `\__hook_set_normalize_fn:nn` with the hook name to set everything up, then we'll loop over the hook's code pool applying the normalization above. After that's done, copy the temporary property list back to the hook's.

```
    \__hook_set_normalize_fn:nn {#1} { Offending~label:~'##1' }
    \prop_clear:N \l__hook_work_prop
    \prop_map_function:cN { g__hook_#1_code_prop } \__hook_normalize_fn:nn
    \prop_gset_eq:cN { g__hook_#1_code_prop } \l__hook_work_prop
  }
```

The sole purpose of this function is to define `\__hook_normalize_fn:nn`, which will then do the correcting of the code being added to the hook.

```
\cs_new_protected:Npn \__hook_set_normalize_fn:nn #1 #2
  {
```

To start, we define two auxiliary token lists. `\l__hook_tmpb_tl` contains:

```
{\c__hook_hashes_tl 1}
{\c__hook_hashes_tl 2}
...
{\c__hook_hashes_tl 9}
```

```
    \cs_set:Npn \__hook_tmp:w ##1##2##3##4##5##6##7##8##9 { }
    \tl_set:Ne \l__hook_tmpb_tl
      { \__hook_braced_cs_parameter:n { __hook_tmp:w } }
    \group_begin:
      \__hook_tl_set:cn { c__hook_hash_tl } { \exp_not:N \c__hook_hashes_tl }
      \use:e
        {
    \group_end:
    \tl_set:Nn \exp_not:N \l__hook_tmpb_tl { \l__hook_tmpb_tl }
        }
```

And `\l__hook_tmpa_tl` contains:

```
{\c__hook_hash_tl 1}
{\c__hook_hash_tl 2}
...
{\c__hook_hash_tl <n>}
```

with ⟨*n*⟩ being the number of arguments declared for the hook.

```
    \exp_last_unbraced:NNf
    \cs_set:Npn \__hook_tmp:w { \__hook_parameter:n {#1} } { }
    \tl_set:Ne \l__hook_tmpa_tl
      { \__hook_braced_cs_parameter:n { __hook_tmp:w } }
```

Now this function does the fun part. It is meant to be used with `\prop_map_function:NN`, taking a label name in `##1` and the code stored in that label in `##2`.

```
    \cs_gset_protected:Npe \__hook_normalize_fn:nn ##1 ##2
      {
```

Here we'll define two auxiliary macros: the first one throws an error when it detects an invalid argument reference. It piggybacks on TEX's low-level "Illegal parameter number" error, but it defines a weirdly-named control sequence so that the error comes out nicely formatted. For example, if the label "badpkg" adds some code that references argument `#3` in the hook "foo", which takes only two arguments, the error will be:

```
! Illegal parameter number in definition of hook 'foo'.
(hooks)              Offending label: 'badpkg'.
<to be read again>
                   3
```

At the point of this definition, the error is raised if the code happens to reference an invalid argument. If it was possible to detect that this definition raised no error, the next step would be unnecessary. We'll do all this in a group so this weird definition doesn't leak out, and set `\tex_escapechar:D` to -1 so this hack shows up extra nice in the case of an error.

```
\group_begin:
  \int_set:Nn \tex_escapechar:D { -1 }
  \cs_set:cpn
      {
        hook~'#1'. ^^J
        (hooks) \prg_replicate:nn { 13 } { ~ }
        #2 % more message text
      }
      \exp_not:v { c__hook_#1_parameter_tl }
    {##2}
\group_end:
```

This next macro, with a much less fabulous name, takes always nine arguments, and it just transfers the code `##2` under the label `##1` to the temporary property list. The first $\langle n \rangle$ arguments are taken from `\l__hook_tmpa_tl`, and the other $9-\langle n \rangle$ taken from `\l__hook_tmpb_tl` (which contains twice as many `#` tokens as the former). Then, `\__hook_double_hashes:n` is used to double non-argument hashes, and expand the `\c__hook_hash_tl` and `\c__hook_hashes_tl` to the actual parameter tokens.

```
\cs_set:Npn \exp_not:N \__hook_tmp:w
    \exp_not:V \c__hook_nine_parameters_tl
  {
    \prop_put:Nne \exp_not:N \l__hook_work_prop
      {##1} { \exp_not:N \__hook_double_hashes:n {##2} }
  }
```

This next macro, with a much less fabulous name, takes always nine arguments, and it just transfers the code `##2` under the label `##1` to the temporary property list. The first $\langle n \rangle$ arguments are taken from `\l__hook_tmpa_tl`, and the other $9-\langle n \rangle$ taken from `\l__hook_tmpb_tl` (which contains twice as many `#` tokens as the former). Then, `\__hook_double_hashes:n` is used to double non-argument hashes, and expand the `\c__hook_hash_tl` and `\c__hook_hashes_tl` to the actual parameter tokens.

```
\exp_not:N \__hook_tmp:w
  \exp_not:V \l__hook_tmpa_tl
  \exp_args:No \exp_not:o
    { \exp_after:wN \__hook_tmp:w \l__hook_tmpb_tl }
  }
}
```

```
\cs_new_eq:NN \__hook_normalize_fn:nn ?
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_normalize_code_pool:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_normalize_code_pool:n
⟨latexrelease⟩\EndIncludeInRelease
```

Check if the expansion of a control sequence is empty by looking at its replacement text.

__hook_cs_if_empty_p:c
__hook_cs_if_empty:cTF

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_cs_if_empty:c}
⟨latexrelease⟩                  {Hooks~with~args}
\prg_new_conditional:Npnn \__hook_cs_if_empty:c #1 { p, T, F, TF }
  {
    \if:w \scan_stop: \__hook_replacement_spec:c {#1} \scan_stop:
      \prg_return_true:
    \else:
      \prg_return_false:
    \fi:
  }
\cs_new:Npn \__hook_replacement_spec:c #1
  {
    \exp_args:Nc \token_if_macro:NT {#1}
      { \cs_replacement_spec:c {#1} }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_cs_if_empty:c}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_cs_if_empty:c
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_normalize_code_pool:n`, `\__hook_set_normalize_fn:nn`, *and* `\__hook_cs_if_empty:cTF`.)

__hook_braced_cs_parameter:n
__hook_braced_hidden_loop:w
__hook_cs_parameter_count:N
__hook_cs_parameter_count:w
__hook_cs_end:w

Looks at the ⟨*parameter text*⟩ of a control sequence, and returns a run of "hidden" braced parameters for that macro. This works as long as the macros take a simple run of zero to nine arguments. The parameters are "hidden" because the parameter tokens are returned inside `\c__hook_hash_tl` instead of explicitly, so that `\__hook_double_hashes:n` won't touch these.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_braced_cs_parameter:n}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new:Npn \__hook_braced_cs_parameter:n #1
  {
    \exp_last_unbraced:Ne \__hook_braced_hidden_loop:w
      { \exp_args:Nc \__hook_cs_parameter_count:N {#1} } ? \s__hook_mark
  }
\cs_new:Npn \__hook_braced_hidden_loop:w #1
  {
    \if:w ? #1
      \__hook_use_i_delimit_by_s_mark:nw
    \fi:
    { \exp_not:N \c__hook_hash_tl #1 }
    \__hook_braced_hidden_loop:w
  }
```

```
\cs_new:Npn \__hook_cs_parameter_count:N #1
  {
    \exp_last_unbraced:Nf \__hook_cs_parameter_count:w
      { \token_if_macro:NT #1 { \cs_parameter_spec:N #1 } }
      ? \__hook_cs_end:w ? \__hook_cs_end:w ? \__hook_cs_end:w
      ? \__hook_cs_end:w ? \__hook_cs_end:w ? \__hook_cs_end:w
      ? \__hook_cs_end:w ? \__hook_cs_end:w ? \__hook_cs_end:w
      \s__hook_mark
  }
\cs_new:Npn \__hook_cs_parameter_count:w #1#2 #3#4 #5#6 #7#8
  { #2 #4 #6 #8 \__hook_cs_parameter_count:w }
\cs_new:Npn \__hook_cs_end:w #1 \s__hook_mark { }
⟨latexrelease⟩\EndIncludeInRelease
```

This function can't be undefined when rolling back because it's used at the end of this module to adequate the hook data structures to previous versions.

```
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_braced_cs_parameter:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_braced_cs_parameter:n` *and others.*)

`\__hook_braced_parameter:n`
`\__hook_braced_real_loop:w`

This one is used in simpler cases, where no special handling of hashes is required. This is used only inside `\__hook_initialize_hook_code:n`, so it assumes `\c__hook_`⟨*hook*⟩`_parameter_tl` is defined, but should work otherwise.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_braced_parameter:n}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new:Npn \__hook_braced_parameter:n #1
  {
    \if_case:w
      \int_eval:n
        { \exp_args:Nv \str_count:n { c__hook_#1_parameter_tl } / 3 }
      \exp_stop_f:
    \or: {##1}
    \or: {##1} {##2}
    \or: {##1} {##2} {##3}
    \or: {##1} {##2} {##3} {##4}
    \or: {##1} {##2} {##3} {##4} {##5}
    \or: {##1} {##2} {##3} {##4} {##5} {##6}
    \or: {##1} {##2} {##3} {##4} {##5} {##6} {##7}
    \or: {##1} {##2} {##3} {##4} {##5} {##6} {##7} {##8}
    \or: {##1} {##2} {##3} {##4} {##5} {##6} {##7} {##8} {##9}
    \else:
      \msg_expandable_error:nnn { latex2e } { should-not-happen }
        { Invalid~parameter~spec. }
    \fi:
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_braced_parameter:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_braced_parameter:n
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_braced_parameter:n` *and* `\__hook_braced_real_loop:w`.)

__hook_parameter:n This is just a shortcut to e- or f-expand to the ⟨*parameter text*⟩ of the hook.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_parameter:n}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new:Npn \__hook_parameter:n #1
  {
    \cs:w c__hook_
    \tl_if_exist:cTF { c__hook_#1_parameter_tl }
      { #1_parameter } { empty }
    _tl \cs_end:
  }
\cs_new:Npn \__hook_generic_parameter:n #1
  { \__hook_generic_parameter:w #1 / / / \s__hook_mark }
\cs_new:Npn \__hook_generic_parameter:w #1 / #2 / #3 / #4 \s__hook_mark
  {
    \cs_if_exist_use:cF { c__hook_parameter_#1/./#3_tl }
      { \c__hook_empty_tl }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_parameter:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_parameter:n
⟨latexrelease⟩\cs_undefine:N \__hook_generic_parameter:n
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* __hook_parameter:n.)

4.7 Setting rules for hooks code

\g__hook_??_code_prop
__hook~??
\g__hook_??_reversed_tl
\c__hook_??_parameter_tl

Initially these variables simply used an empty "label" name (not two question marks). This was a bit unfortunate, because then l3doc complains about __ in the middle of a command name when trying to typeset the documentation. However using a "normal" name such as default has the disadvantage of that being not really distinguishable from a real hook name. I now have settled for ?? which needs some gymnastics to get it into the csname, but since this is used a lot, the code should be fast, so this is not done with c expansion in the code later on.

__hook␣?? isn't used, but it has to be defined to trick the code into thinking that ?? is actually a hook.

```
\prop_new:c { g__hook_??_code_prop }
\prop_new:c { __hook~?? }
```

Default rules are always given in normal ordering (never in reversed ordering). If such a rule is applied to a reversed hook it behaves as if the rule is reversed (e.g., after becomes before) because those rules are applied first and then the order is reversed.

```
\tl_new:c { g__hook_??_reversed_tl }
```

The parameter text for the "default" hook is empty.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\c__hook_??_parameter_tl}
⟨latexrelease⟩                  {Hooks~with~args}
\tl_const:cn { c__hook_??_parameter_tl } { }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\c__hook_??_parameter_tl}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:c { c__hook_??_parameter_tl }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\g__hook_??_code_prop` *and others.*)

`\hook_gset_rule:nnnn`
`\__hook_gset_rule:nnnn`

With `\hook_gset_rule:nnnn{`⟨*hook*⟩`}{`⟨*label1*⟩`}{`⟨*relation*⟩`}{`⟨*label2*⟩`}` a relation is defined between the two code labels for the given ⟨***hook***⟩. The special hook `??` stands for *any* hook, which sets a default rule (to be used if no other relation between the two hooks exist).

```
\cs_new_protected:Npn \hook_gset_rule:nnnn #1#2#3#4
  {
    \__hook_normalize_hook_rule_args:Nnnnn \__hook_gset_rule:nnnn
      {#1} {#2} {#3} {#4}
  }
⟨latexrelease⟩\IncludeInRelease{2022/06/01}{\__hook_gset_rule:nnnn}
⟨latexrelease⟩                  {Refuse~setting~rule~for~one-time~hooks}
\cs_new_protected:Npn \__hook_gset_rule:nnnn #1#2#3#4
  {
    \__hook_if_deprecated_generic:nT {#1}
      {
        \__hook_deprecated_generic_warn:n {#1}
        \__hook_do_deprecated_generic:Nn \__hook_gset_rule:nnnn {#1}
          {#2} {#3} {#4}
        \__hook_use_none_delimit_by_s_mark:w
      }
    \__hook_if_execute_immediately:nT {#1}
      {
        \msg_error:nnnnnn { hooks } { rule-too-late }
          {#1} {#2} {#3} {#4}
        \__hook_use_none_delimit_by_s_mark:w
      }
```

First we ensure the basic data structure of the hook exists:

```
    \__hook_init_structure:n {#1}
```

Then we clear any previous relationship between both labels.

```
    \__hook_rule_gclear:nnn {#1} {#2} {#4}
```

Then we call the function to handle the given rule. Throw an error if the rule is invalid.

```
    \cs_if_exist_use:cTF { __hook_rule_#3_gset:nnn }
      {
          {#1} {#2} {#4}
        \__hook_update_hook_code:n {#1}
      }
      {
        \msg_error:nnnnnn { hooks } { unknown-rule }
          {#1} {#2} {#3} {#4}
      }
    \s__hook_mark
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_gset_rule:nnnn}
⟨latexrelease⟩                  {Refuse~setting~rule~for~one-time~hooks}
⟨latexrelease⟩\cs_new_protected:Npn \__hook_gset_rule:nnnn #1#2#3#4
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_deprecated_generic:nT {#1}
```

```
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_deprecated_generic_warn:n {#1}
⟨latexrelease⟩        \__hook_do_deprecated_generic:Nn \__hook_gset_rule:nnnn
⟨latexrelease⟩          {#1} {#2} {#3} {#4}
⟨latexrelease⟩        \exp_after:wN \use_none:nnnnnnnnn \use_none:n
⟨latexrelease⟩      }
⟨latexrelease⟩    \__hook_init_structure:n {#1}
⟨latexrelease⟩    \__hook_rule_gclear:nnn {#1} {#2} {#4}
⟨latexrelease⟩    \cs_if_exist_use:cTF { __hook_rule_#3_gset:nnn }
⟨latexrelease⟩      {
⟨latexrelease⟩          {#1} {#2} {#4}
⟨latexrelease⟩        \__hook_update_hook_code:n {#1}
⟨latexrelease⟩      }
⟨latexrelease⟩      {
⟨latexrelease⟩        \msg_error:nnnnnn { hooks } { unknown-rule }
⟨latexrelease⟩          {#1} {#2} {#3} {#4}
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_gset_rule:nnnn` *and* `\__hook_gset_rule:nnnn`*. This function is documented on page 232.*)

`\__hook_rule_before_gset:nnn`
`\__hook_rule_after_gset:nnn`
`\__hook_rule_<_gset:nnn`
`\__hook_rule_>_gset:nnn`

Then we add the new rule. We need to normalize the rules here to allow for faster processing later. Given a pair of labels l_A and l_B, the rule $l_A > l_B$ is the same as $l_B < l_A$ only presented differently. But by normalizing the forms of the rule to a single representation, say, $l_B < l_A$, reduces the time spent looking for the rules later considerably.

Here we do that normalization by using `\(pdf)strcmp` to lexically sort labels l_A and l_B to a fixed order. This order is then enforced every time these two labels are used together.

Here we use `\__hook_label_pair:nn {`⟨*hook*⟩`} {`⟨l_A⟩`} {`⟨l_B⟩`}` to build a string $l_B \mid l_A$ with a fixed order, and use `\__hook_label_ordered:nnTF` to apply the correct rule to the pair of labels, depending if it was sorted or not.

```
\cs_new_protected:Npn \__hook_rule_before_gset:nnn #1#2#3
  {
    \__hook_tl_gset:ce
      { g__hook_#1_rule_ \__hook_label_pair:nn {#2} {#3} _tl }
      { \__hook_label_ordered:nnTF {#2} {#3} { < } { > } }
  }
\cs_new_eq:cN { __hook_rule_<_gset:nnn } \__hook_rule_before_gset:nnn
\cs_new_protected:Npn \__hook_rule_after_gset:nnn #1#2#3
  {
    \__hook_tl_gset:ce
      { g__hook_#1_rule_ \__hook_label_pair:nn {#3} {#2} _tl }
      { \__hook_label_ordered:nnTF {#3} {#2} { < } { > } }
  }
\cs_new_eq:cN { __hook_rule_>_gset:nnn } \__hook_rule_after_gset:nnn
```

(*End of definition for* `\__hook_rule_before_gset:nnn` *and others.*)

`\__hook_rule_voids_gset:nnn`

This rule removes (clears, actually) the code from label `#3` if label `#2` is in the hook `#1`.

```
\cs_new_protected:Npn \__hook_rule_voids_gset:nnn #1#2#3
  {
```

```
    \__hook_tl_gset:ce
      { g__hook_#1_rule_ \__hook_label_pair:nn {#2} {#3} _tl }
      { \__hook_label_ordered:nnTF {#2} {#3} { -> } { <- } }
  }
```

(*End of definition for* `\__hook_rule_voids_gset:nnn`.)

`\__hook_rule_incompatible-error_gset:nnn`
`\__hook_rule_incompatible-warning_gset:nnn`

These relations make an error/warning if labels `#2` and `#3` appear together in hook `#1`.

```
\cs_new_protected:cpn { __hook_rule_incompatible-error_gset:nnn } #1#2#3
  { \__hook_tl_gset:cn
      { g__hook_#1_rule_ \__hook_label_pair:nn {#2} {#3} _tl }
      { xE }
  }
\cs_new_protected:cpn { __hook_rule_incompatible-warning_gset:nnn } #1#2#3
  { \__hook_tl_gset:cn
      { g__hook_#1_rule_ \__hook_label_pair:nn {#2} {#3} _tl }
      { xW }
  }
```

(*End of definition for* `\__hook_rule_incompatible-error_gset:nnn` *and* `\__hook_rule_incompatible-warning_gset:nnn`.)

`\__hook_rule_unrelated_gset:nnn`
`\__hook_rule_gclear:nnn`

Undo a setting. `\__hook_rule_unrelated_gset:nnn` doesn't need to do anything, since we use `\__hook_rule_gclear:nnn` before setting any rule.

```
\cs_new_protected:Npn \__hook_rule_unrelated_gset:nnn #1#2#3 { }
\cs_new_protected:Npn \__hook_rule_gclear:nnn #1#2#3
  { \cs_undefine:c { g__hook_#1_rule_ \__hook_label_pair:nn {#2} {#3} _tl } }
```

(*End of definition for* `\__hook_rule_unrelated_gset:nnn` *and* `\__hook_rule_gclear:nnn`.)

`\__hook_label_pair:nn`

Ensure that the lexically greater label comes first.

```
\cs_new:Npn \__hook_label_pair:nn #1#2
  {
    \if_case:w \__hook_str_compare:nn {#1} {#2} \exp_stop_f:
          #1 | #1 %  0
    \or:   #1 | #2 % +1
    \else: #2 | #1 % -1
    \fi:
  }
```

(*End of definition for* `\__hook_label_pair:nn`.)

`\__hook_label_ordered_p:nn`
`\__hook_label_ordered:nnTF`

Check that labels `#1` and `#2` are in the correct order (as returned by `\__hook_label_pair:nn`) and if so return true, else return false.

```
\prg_new_conditional:Npnn \__hook_label_ordered:nn #1#2 { TF }
  {
    \if_int_compare:w \__hook_str_compare:nn {#1} {#2} > 0 \exp_stop_f:
      \prg_return_true:
    \else:
      \prg_return_false:
    \fi:
  }
```

(*End of definition for* `\__hook_label_ordered:nnTF`.)

__hook_if_label_case:nnnnn To avoid doing the string comparison twice in __hook_initialize_single:NNn (once with \str_if_eq:nn and again with __hook_label_ordered:nn), we use a three-way branching macro that will compare #1 and #2 and expand to \use_i:nnn if they are equal, \use_ii:nn if #1 is lexically greater, and \use_iii:nn otherwise.

```
\cs_new:Npn \__hook_if_label_case:nnnnn #1#2
  {
    \cs:w use_
      \if_case:w \__hook_str_compare:nn {#1} {#2}
         i \or: ii \else: iii \fi: :nnn
    \cs_end:
  }
```

(*End of definition for* __hook_if_label_case:nnnnn.)

__hook_update_hook_code:n Before \begin{document} this does nothing, in the body it reinitializes the hook code using the altered data.

```
\cs_new_eq:NN \__hook_update_hook_code:n \use_none:n
```

(*End of definition for* __hook_update_hook_code:n.)

__hook_initialize_all: Initialize all known hooks (at \begin{document}), i.e., update the fast execution token lists to hold the necessary code in the right order.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_initialize_all:}
⟨latexrelease⟩                          {Hooks~with~args}
\cs_new_protected:Npn \__hook_initialize_all:
  {
```

First we change __hook_update_hook_code:n which so far was a no-op to now initialize one hook. This way any later updates to the hook will run that code and also update the execution token list.

```
    \cs_gset_eq:NN \__hook_update_hook_code:n \__hook_initialize_hook_code:n
```

Now we loop over all hooks that have been defined and update each of them. Here we have to determine if the hook has arguments so that auxiliaries know what to do with hashes. We look at \c__hook_⟨*hook*⟩_parameter_tl, if it has any parameters, and set replacing_args accordingly.

```
    \__hook_debug:n { \prop_gclear:N \g__hook_used_prop }
    \seq_map_inline:Nn \g__hook_all_seq
      {
        \tl_if_empty:cTF { c__hook_##1_parameter_tl }
          { \__hook_replacing_args_false: }
          { \__hook_replacing_args_true: }
        \__hook_update_hook_code:n {##1}
        \__hook_replacing_args_reset:
      }
```

If we are debugging we show results hook by hook for all hooks that have data.

```
    \__hook_debug:n
      {
        \iow_term:e { ^^J[lthooks]~ All~initialized~(non-empty)~hooks: }
        \prop_map_inline:Nn \g__hook_used_prop
          {
            \iow_term:e
              { ^^J ~ ##1 ~ -> ~ \cs_replacement_spec:c { __hook~##1 } ~ }
          }
      }
```

After all hooks are initialized we change the "use" to just call the hook code and not initialize it (as this was already done in the preamble.

```
    \__hook_post_initialization_defs:
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_initialize_all:}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_initialize_all:
⟨latexrelease⟩  {
⟨latexrelease⟩    \cs_gset_eq:NN \__hook_update_hook_code:n
⟨latexrelease⟩                   \__hook_initialize_hook_code:n
⟨latexrelease⟩    \__hook_debug:n { \prop_gclear:N \g__hook_used_prop }
⟨latexrelease⟩    \seq_map_inline:Nn \g__hook_all_seq
⟨latexrelease⟩      { \__hook_update_hook_code:n {##1} }
⟨latexrelease⟩    \__hook_debug:n
⟨latexrelease⟩      {
⟨latexrelease⟩        \iow_term:x{^^JAll~ initialized~ (non-empty)~ hooks:}
⟨latexrelease⟩        \prop_map_inline:Nn \g__hook_used_prop
⟨latexrelease⟩          {
⟨latexrelease⟩            \iow_term:x
⟨latexrelease⟩              { ^^J ~ ##1 ~ -> ~
⟨latexrelease⟩                \cs_replacement_spec:c { __hook~##1 } ~ }
⟨latexrelease⟩          }
⟨latexrelease⟩      }
⟨latexrelease⟩    \cs_gset_eq:NN \hook_use:n \__hook_use_initialized:n
⟨latexrelease⟩    \cs_gset_eq:NN \__hook_preamble_hook:n \use_none:n
⟨latexrelease⟩  }
⟨@@=⟩
⟨latexrelease⟩\cs_gset_eq:NN \@expl@@@initialize@all@@
⟨latexrelease⟩               \__hook_initialize_all:
⟨@@=hook⟩
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_initialize_all:`.)

`\__hook_initialize_hook_code:n` Initializing or reinitializing the fast execution hook code. In the preamble this is selectively done in case a hook gets used and at `\begin{document}` this is done for all hooks and afterwards only if the hook code changes.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_initialize_hook_code:n}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_initialize_hook_code:n #1
  {
    \__hook_debug:n
      { \iow_term:e { ^^J[lthooks]~ Update~code~for~hook~'#1' \on@line :^^J } }
```

This does the sorting and the updates. First thing we do is to check if a legacy hook macro exists and if so we add it to the hook under the label `legacy`. This might make the hook non-empty so we have to do this before the then following test.

```
    \__hook_include_legacy_code_chunk:n {#1}
```

If there aren't any code chunks for the current hook, there is no point in even starting the sorting routine so we make a quick test for that and in that case just update `\__hook`␣⟨*hook*⟩ to hold the `top-level` and `next` code chunks. If there are code chunks we call `\__hook_initialize_single:NNn` and pass to it ready made csnames as they are

needed several times inside. This way we save a bit on processing time if we do that up front.

```
    \__hook_if_usable:nT {#1}
      {
        \prop_if_empty:cTF { g__hook_#1_code_prop }
          {
            \__hook_code_gset:ne {#1}
              {
```

The hook may take arguments, so we add a run of braced parameters after the `_next` and `_toplevel` macros, so that the arguments passed to the hook are forwarded to them.

```
                \exp_not:c { __hook_toplevel~#1 }
                \__hook_braced_parameter:n {#1}
                \exp_not:c { __hook_next~#1 }
                \__hook_braced_parameter:n {#1}
              }
          }
          {
```

By default the algorithm sorts the code chunks and then saves the result in a token list for fast execution; this is done by adding the code chunks one after another, using `\tl_gput_right:NV`. When we sort code for a reversed hook, all we have to do is to add the code chunks in the opposite order into the token list. So all we have to do in preparation is to change two definitions that are used later on.

```
            \__hook_if_reversed:nTF {#1}
              { \cs_set_eq:NN \__hook_tl_gput:Nn    \__hook_tl_gput_left:Nn
                \cs_set_eq:NN \__hook_clist_gput:NV \clist_gput_left:NV  }
              { \cs_set_eq:NN \__hook_tl_gput:Nn    \__hook_tl_gput_right:Nn
                \cs_set_eq:NN \__hook_clist_gput:NV \clist_gput_right:NV }
```

When sorting, some relations (namely `voids`) need to act destructively on the code property lists to remove code that shouldn't appear in the sorted hook token list, so we make a copy of the code property list that we can safely work on without changing the main one.

```
            \prop_set_eq:Nc \l__hook_work_prop { g__hook_#1_code_prop }
            \__hook_initialize_single:ccn
              { __hook~#1 } { g__hook_#1_labels_clist } {#1}
```

For debug display we want to keep track of those hooks that actually got code added to them, so we record that in plist. We use a plist to ensure that we record each hook name only once, i.e., we are only interested in storing the keys and the value is arbitrary.

```
            \__hook_debug:n
              { \exp_args:NNe \prop_gput:Nnn \g__hook_used_prop {#1} { } }
          }
      }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_initialize_hook_code:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_initialize_hook_code:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_debug:n
⟨latexrelease⟩      { \iow_term:x { ^^J Update~code~for~hook~'#1'
⟨latexrelease⟩                      \on@line :^^J } }
```

```
⟨latexrelease⟩    \__hook_include_legacy_code_chunk:n {#1}
⟨latexrelease⟩    \__hook_if_usable:nT {#1}
⟨latexrelease⟩      {
⟨latexrelease⟩        \prop_if_empty:cTF { g__hook_#1_code_prop }
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_tl_gset:co { __hook~#1 }
⟨latexrelease⟩              {
⟨latexrelease⟩                \cs:w __hook_toplevel~#1 \exp_after:wN \cs_end:
⟨latexrelease⟩                \cs:w __hook_next~#1 \cs_end:
⟨latexrelease⟩              }
⟨latexrelease⟩          }
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_if_reversed:nTF {#1}
⟨latexrelease⟩              { \cs_set_eq:NN \__hook_tl_gput:Nn
⟨latexrelease⟩                              \__hook_tl_gput_left:Nn
⟨latexrelease⟩                \cs_set_eq:NN \__hook_clist_gput:NV
⟨latexrelease⟩                              \clist_gput_left:NV  }
⟨latexrelease⟩              { \cs_set_eq:NN \__hook_tl_gput:Nn
⟨latexrelease⟩                              \__hook_tl_gput_right:Nn
⟨latexrelease⟩                \cs_set_eq:NN \__hook_clist_gput:NV
⟨latexrelease⟩                              \clist_gput_right:NV }
⟨latexrelease⟩            \prop_set_eq:Nc \l__hook_work_prop
⟨latexrelease⟩              { g__hook_#1_code_prop }
⟨latexrelease⟩            \__hook_initialize_single:ccn
⟨latexrelease⟩              { __hook~#1 } { g__hook_#1_labels_clist } {#1}
⟨latexrelease⟩            \__hook_debug:n
⟨latexrelease⟩              { \exp_args:NNx \prop_gput:Nnn \g__hook_used_prop
⟨latexrelease⟩                    {#1} { } }
⟨latexrelease⟩          }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_initialize_hook_code:n`.)

`\__hook_tl_csname:n`
`\__hook_seq_csname:n`

It is faster to pass a single token and expand it when necessary than to pass a bunch of character tokens around.

FMi: note to myself: verify

```
\cs_new:Npn \__hook_tl_csname:n #1 { l__hook_label_#1_tl }
\cs_new:Npn \__hook_seq_csname:n #1 { l__hook_label_#1_seq }
```

(*End of definition for* `\__hook_tl_csname:n` *and* `\__hook_seq_csname:n`.)

`\l__hook_labels_seq`
`\l__hook_labels_int`
`\l__hook_front_tl`
`\l__hook_rear_tl`
`\l__hook_label_0_tl`

For the sorting I am basically implementing Knuth's algorithm for topological sorting as given in TAOCP volume 1 pages 263–266. For this algorithm we need a number of local variables:

- List of labels used in the current hook to label code chunks:

```
\seq_new:N \l__hook_labels_seq
```

- Number of labels used in the current hook. In Knuth's algorithm this is called N:

```
\int_new:N \l__hook_labels_int
```

- The sorted code list to be build is managed using two pointers one to the front of the queue and one to the rear. We model this using token list pointers. Knuth calls them F and R:

```
\tl_new:N \l__hook_front_tl
\tl_new:N \l__hook_rear_tl
```

- The data for the start of the queue is kept in this token list, it corresponds to what Don calls `QLINK[0]` but since we aren't manipulating individual words in memory it is slightly differently done:

```
\tl_new:c { \__hook_tl_csname:n { 0 } }
```

(*End of definition for* `\l__hook_labels_seq` *and others.*)

__hook_initialize_single:NNn
__hook_initialize_single:ccn

`\__hook_initialize_single:NNn` implements the sorting of the code chunks for a hook and saves the result in the token list for fast execution (`#4`). The arguments are ⟨*hook-code-plist*⟩, ⟨*hook-code-tl*⟩, ⟨*hook-top-level-code-tl*⟩, ⟨*hook-next-code-tl*⟩, ⟨*hook-ordered-labels-clist*⟩ and ⟨*hook-name*⟩ (the latter is only used for debugging—the ⟨*hook-rule-plist*⟩ is accessed using the ⟨*hook-name*⟩).

The additional complexity compared to Don's algorithm is that we do not use simple positive integers but have arbitrary alphanumeric labels. As usual Don's data structures are chosen in a way that one can omit a lot of tests and I have mimicked that as far as possible. The result is a restriction I do not test for at the moment: a label can't be equal to the number 0!

FMi: Needs checking for, just in case … maybe

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_initialize_single:NNn}
⟨latexrelease⟩                        {Hooks~with~args}
\cs_new_protected:Npn \__hook_initialize_single:NNn #1#2#3
  {
```

Step T1: Initialize the data structure …

```
    \seq_clear:N \l__hook_labels_seq
    \int_zero:N  \l__hook_labels_int
```

Store the name of the hook:

```
    \tl_set:Nn \l__hook_cur_hook_tl {#3}
```

We loop over the property list holding the code and record all the labels listed there. Only the rules for those labels are of interest to us. While we are at it we count them (which gives us the N in Knuth's algorithm). The prefix `label_` is added to the variables to ensure that labels named `front`, `rear`, `labels`, or `return` don't interact with our code.

```
    \prop_map_inline:Nn \l__hook_work_prop
      {
        \int_incr:N \l__hook_labels_int
        \seq_put_right:Nn \l__hook_labels_seq {##1}
        \__hook_tl_set:cn { \__hook_tl_csname:n {##1} } { 0 }
        \seq_clear_new:c { \__hook_seq_csname:n {##1} }
      }
```

Steps T2 and T3: Here we sort the relevant rules into the data structure...

This loop constitutes a square matrix of the labels in `\l__hook_work_prop` in the vertical and the horizontal directions. However, since the rule $l_A\langle\texttt{rel}\rangle l_B$ is the same as $l_B\langle\texttt{rel}\rangle^{-1} l_A$ we can cut the loop short at the diagonal of the matrix (*i.e.*, when both labels are equal), saving a good amount of time. The way the rules were set up (see the implementation of `\__hook_rule_before_gset:nnn` above) ensures that we have no rule in the ignored side of the matrix, and all rules are seen. The rules are applied in `\__hook_apply_label_pair:nnn`, which takes the properly-ordered pair of labels as argument.

```
\prop_map_inline:Nn \l__hook_work_prop
  {
    \prop_map_inline:Nn \l__hook_work_prop
      {
        \__hook_if_label_case:nnnnn {##1} {####1}
          { \prop_map_break: }
          { \__hook_apply_label_pair:nnn {##1} {####1} }
          { \__hook_apply_label_pair:nnn {####1} {##1} }
              {#3}
      }
  }
```

Now take a breath, and look at the data structures that have been set up:

```
\__hook_debug:n { \__hook_debug_label_data:N \l__hook_work_prop }
```

Step T4:

```
\tl_set:Nn \l__hook_rear_tl { 0 }
\tl_set:cn { \__hook_tl_csname:n { 0 } } { 0 }
\seq_map_inline:Nn \l__hook_labels_seq
  {
    \int_compare:nNnT { \cs:w \__hook_tl_csname:n {##1} \cs_end: } = 0
        {
          \tl_set:cn { \__hook_tl_csname:n { \l__hook_rear_tl } }{##1}
          \tl_set:Nn \l__hook_rear_tl {##1}
        }
  }
\tl_set_eq:Nc \l__hook_front_tl { \__hook_tl_csname:n { 0 } }
\__hook_tl_gclear:N #1
\clist_gclear:N #2
```

The whole loop gets combined in steps T5–T7:

```
\bool_while_do:nn { ! \str_if_eq_p:Vn \l__hook_front_tl { 0 } }
  {
```

This part is step T5:

```
    \int_decr:N \l__hook_labels_int
    \prop_get:NVN \l__hook_work_prop \l__hook_front_tl \l__hook_return_tl
    \exp_args:NNV \__hook_tl_gput:Nn #1 \l__hook_return_tl

    \__hook_clist_gput:NV #2 \l__hook_front_tl
    \__hook_debug:n{ \iow_term:e{[lthooks]~ Handled~ code~ for~ \l__hook_front_tl} }
```

This is step T6, except that we don't use a pointer P to move through the successors, but instead use `##1` of the mapping function.

```
    \seq_map_inline:cn { \__hook_seq_csname:n { \l__hook_front_tl } }
```

```
      {
        \tl_set:ce { \__hook_tl_csname:n {##1} }
                   { \int_eval:n
                       { \cs:w \__hook_tl_csname:n {##1} \cs_end: - 1 }
                   }
        \int_compare:nNnT
            { \cs:w \__hook_tl_csname:n {##1} \cs_end: } = 0
            {
              \tl_set:cn
                { \__hook_tl_csname:n { \l__hook_rear_tl } } {##1}
              \tl_set:Nn \l__hook_rear_tl            {##1}
            }
      }
```

and here is step T7:

```
      \tl_set_eq:Nc \l__hook_front_tl
                    { \__hook_tl_csname:n { \l__hook_front_tl } }
```

This is step T8: If we haven't moved the code for all labels (i.e., if `\l__hook_labels_int` is still greater than zero) we have a loop and our partial order can't be flattened out.

```
    }
  \int_compare:nNnF \l__hook_labels_int = 0
    {
      \iow_term:e{====================}
      \iow_term:e{Error:~ label~ rules~ are~ incompatible:}
```

This is not really the information one needs in the error case but it will do for now ...

FMi: improve output on a rainy day

```
      \__hook_debug_label_data:N \l__hook_work_prop
      \iow_term:e{====================}
    }
```

After we have added all hook code to `#1`, we finish it off by adding extra code for the `top-level` (`#2`) and for one time execution (`#3`). These should normally be empty. The `top-level` code is added with `\__hook_tl_gput:Nn` as that might change for a reversed hook (then `top-level` is the very first code chunk added). The `next` code is always added last (to the right). The hook may take arguments, so we add a run of braced parameters after the `_next` and `_toplevel` macros, so that the arguments passed to the hook are forwarded to them.

```
  \exp_args:NNe \__hook_tl_gput:Nn #1
    { \exp_not:c { __hook_toplevel~#3 } \__hook_braced_parameter:n {#3} }
  \__hook_tl_gput_right:Ne #1
    { \exp_not:c { __hook_next~#3 } \__hook_braced_parameter:n {#3} }
  \use:e
    {
      \cs_gset:cpn { __hook~#3 } \use:c { c__hook_#3_parameter_tl }
        { \exp_not:V #1 }
    }
}
\cs_generate_variant:Nn \__hook_initialize_single:NNn { cc }
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_initialize_single:NNn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_new_protected:Npn \__hook_initialize_single:NNn #1#2#3
⟨latexrelease⟩  {
⟨latexrelease⟩    \seq_clear:N \l__hook_labels_seq
⟨latexrelease⟩    \int_zero:N  \l__hook_labels_int
⟨latexrelease⟩    \tl_set:Nn \l__hook_cur_hook_tl {#3}
⟨latexrelease⟩    \prop_map_inline:Nn \l__hook_work_prop
⟨latexrelease⟩       {
⟨latexrelease⟩         \int_incr:N \l__hook_labels_int
⟨latexrelease⟩         \seq_put_right:Nn \l__hook_labels_seq {##1}
⟨latexrelease⟩         \__hook_tl_set:cn { \__hook_tl_csname:n {##1} } { 0 }
⟨latexrelease⟩         \seq_clear_new:c { \__hook_seq_csname:n {##1} }
⟨latexrelease⟩       }
⟨latexrelease⟩    \prop_map_inline:Nn \l__hook_work_prop
⟨latexrelease⟩      {
⟨latexrelease⟩        \prop_map_inline:Nn \l__hook_work_prop
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_if_label_case:nnnnn {##1} {####1}
⟨latexrelease⟩              { \prop_map_break: }
⟨latexrelease⟩              { \__hook_apply_label_pair:nnn {##1} {####1} }
⟨latexrelease⟩              { \__hook_apply_label_pair:nnn {####1} {##1} }
⟨latexrelease⟩                  {#3}
⟨latexrelease⟩          }
⟨latexrelease⟩      }
⟨latexrelease⟩    \__hook_debug:n
⟨latexrelease⟩      { \__hook_debug_label_data:N \l__hook_work_prop }
⟨latexrelease⟩    \tl_set:Nn \l__hook_rear_tl { 0 }
⟨latexrelease⟩    \tl_set:cn { \__hook_tl_csname:n { 0 } } { 0 }
⟨latexrelease⟩    \seq_map_inline:Nn \l__hook_labels_seq
⟨latexrelease⟩      {
⟨latexrelease⟩        \int_compare:nNnT
⟨latexrelease⟩          { \cs:w \__hook_tl_csname:n {##1} \cs_end: } = 0
⟨latexrelease⟩          {
⟨latexrelease⟩            \tl_set:cn { \__hook_tl_csname:n
⟨latexrelease⟩                         { \l__hook_rear_tl } } {##1}
⟨latexrelease⟩            \tl_set:Nn \l__hook_rear_tl           {##1}
⟨latexrelease⟩          }
⟨latexrelease⟩      }
⟨latexrelease⟩    \tl_set_eq:Nc \l__hook_front_tl { \__hook_tl_csname:n { 0 } }
⟨latexrelease⟩    \__hook_tl_gclear:N #1
⟨latexrelease⟩    \clist_gclear:N #2
⟨latexrelease⟩    \bool_while_do:nn
⟨latexrelease⟩      { ! \str_if_eq_p:Vn \l__hook_front_tl { 0 } }
⟨latexrelease⟩      {
⟨latexrelease⟩        \int_decr:N \l__hook_labels_int
⟨latexrelease⟩        \prop_get:NVN \l__hook_work_prop
⟨latexrelease⟩            \l__hook_front_tl \l__hook_return_tl
⟨latexrelease⟩        \exp_args:NNV \__hook_tl_gput:Nn #1 \l__hook_return_tl
⟨latexrelease⟩        \__hook_clist_gput:NV #2 \l__hook_front_tl
⟨latexrelease⟩        \__hook_debug:n{ \iow_term:x
⟨latexrelease⟩              {Handled~ code~ for~ \l__hook_front_tl} }
⟨latexrelease⟩        \seq_map_inline:cn
⟨latexrelease⟩              { \__hook_seq_csname:n { \l__hook_front_tl } }
```

```
⟨latexrelease⟩          {
⟨latexrelease⟩            \tl_set:cx { \__hook_tl_csname:n {##1} }
⟨latexrelease⟩              { \int_eval:n
⟨latexrelease⟩                { \cs:w \__hook_tl_csname:n {##1} \cs_end: - 1 }
⟨latexrelease⟩              }
⟨latexrelease⟩            \int_compare:nNnT
⟨latexrelease⟩              { \cs:w \__hook_tl_csname:n {##1} \cs_end: } = 0
⟨latexrelease⟩              {
⟨latexrelease⟩                \tl_set:cn { \__hook_tl_csname:n
⟨latexrelease⟩                               { \l__hook_rear_tl } } {##1}
⟨latexrelease⟩                \tl_set:Nn \l__hook_rear_tl           {##1}
⟨latexrelease⟩              }
⟨latexrelease⟩          }
⟨latexrelease⟩        \tl_set_eq:Nc \l__hook_front_tl
⟨latexrelease⟩          { \__hook_tl_csname:n { \l__hook_front_tl } }
⟨latexrelease⟩      }
⟨latexrelease⟩    \int_compare:nNnF \l__hook_labels_int = 0
⟨latexrelease⟩      {
⟨latexrelease⟩        \iow_term:x{====================}
⟨latexrelease⟩        \iow_term:x{Error:~ label~ rules~ are~ incompatible:}
⟨latexrelease⟩        \__hook_debug_label_data:N \l__hook_work_prop
⟨latexrelease⟩        \iow_term:x{====================}
⟨latexrelease⟩      }
⟨latexrelease⟩    \exp_args:NNo \__hook_tl_gput:Nn #1
⟨latexrelease⟩                    { \cs:w __hook_toplevel~#3 \cs_end: }
⟨latexrelease⟩    \__hook_tl_gput_right:No #1 { \cs:w __hook_next~#3 \cs_end: }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_generate_variant:Nn \__hook_tl_gput_right:Nn { No }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_initialize_single:NNn`.)

`\__hook_tl_gput:Nn`
`\__hook_clist_gput:NV`

These append either on the right (normal hook) or on the left (reversed hook). This is setup up in `\__hook_initialize_hook_code:n`, elsewhere their behavior is undefined.

```
\cs_new:Npn \__hook_tl_gput:Nn    { \ERROR }
\cs_new:Npn \__hook_clist_gput:NV { \ERROR }
```

(*End of definition for* `\__hook_tl_gput:Nn` *and* `\__hook_clist_gput:NV`.)

`\__hook_apply_label_pair:nnn`
`\__hook_label_if_exist_apply:nnnF`

This is the payload of steps T2 and T3 executed in the loop described above. This macro assumes `#1` and `#2` are ordered, which means that any rule pertaining the pair `#1` and `#2` is `\g__hook_⟨hook⟩_rule_#1|#2_tl`, and not `\g__hook_⟨hook⟩_rule_#2|#1_tl`. This also saves a great deal of time since we only need to check the order of the labels once.

The arguments here are ⟨*label1*⟩, ⟨*label2*⟩, ⟨*hook*⟩, and ⟨*hook-code-plist*⟩. We are about to apply the next rule and enter it into the data structure. `\__hook_apply_label_pair:nnn` will just call `\__hook_label_if_exist_apply:nnnF` for the ⟨*hook*⟩, and if no rule is found, also try the ⟨*hook*⟩ name `??` denoting a default hook rule.

`\__hook_label_if_exist_apply:nnnF` will check if the rule exists for the given hook, and if so call `\__hook_apply_rule:nnn`.

```
\cs_new_protected:Npn \__hook_apply_label_pair:nnn #1#2#3
  {
```

Extra complication: as we use default rules and local hook specific rules we first have to check if there is a local rule and if that exist use it. Otherwise check if there is a default rule and use that.

```
    \__hook_label_if_exist_apply:nnnF {#1} {#2} {#3}
      {
```

If there is no hook-specific rule we check for a default one and use that if it exists.

```
        \__hook_label_if_exist_apply:nnnF {#1} {#2} { ?? } { }
      }
  }
\cs_new_protected:Npn \__hook_label_if_exist_apply:nnnF #1#2#3
  {
    \if_cs_exist:w g__hook_ #3 _rule_ #1 | #2 _tl \cs_end:
```

What to do precisely depends on the type of rule we have encountered. If it is a `before` rule it will be handled by the algorithm but other types need to be managed differently. All this is done in `\__hook_apply_rule:nnnN`.

```
      \__hook_apply_rule:nnn {#1} {#2} {#3}
      \exp_after:wN \use_none:n
    \else:
      \use:nn
    \fi:
  }
```

(*End of definition for* `\__hook_apply_label_pair:nnn` *and* `\__hook_label_if_exist_apply:nnnF`.)

`\__hook_apply_rule:nnn` This is the code executed in steps T2 and T3 while looping through the matrix This is part of step T3. We are about to apply the next rule and enter it into the data structure. The arguments are ⟨*label1*⟩, ⟨*label2*⟩, ⟨*hook-name*⟩, and ⟨*hook-code-plist*⟩.

```
\cs_new_protected:Npn \__hook_apply_rule:nnn #1#2#3
  {
    \cs:w __hook_apply_
      \cs:w g__hook_#3_reversed_tl \cs_end: rule_
        \cs:w g__hook_ #3 _rule_ #1 | #2 _tl \cs_end: :nnn \cs_end:
      {#1} {#2} {#3}
  }
```

(*End of definition for* `\__hook_apply_rule:nnn`.)

`\__hook_apply_rule_<:nnn` `\__hook_apply_rule_>:nnn` The most common cases are `<` and `>` so we handle that first. They are relations $\prec$ and $\succ$ in TAOCP, and they dictate sorting.

```
\cs_new_protected:cpn { __hook_apply_rule_<:nnn } #1#2#3
  {
    \__hook_debug:n { \__hook_msg_pair_found:nnn {#1} {#2} {#3} }
    \tl_set:ce { \__hook_tl_csname:n {#2} }
       { \int_eval:n{ \cs:w \__hook_tl_csname:n {#2} \cs_end: + 1 } }
    \seq_put_right:cn{ \__hook_seq_csname:n {#1} }{#2}
  }
\cs_new_protected:cpn { __hook_apply_rule_>:nnn } #1#2#3
  {
    \__hook_debug:n { \__hook_msg_pair_found:nnn {#1} {#2} {#3} }
    \tl_set:ce { \__hook_tl_csname:n {#1} }
       { \int_eval:n{ \cs:w \__hook_tl_csname:n {#1} \cs_end: + 1 } }
    \seq_put_right:cn{ \__hook_seq_csname:n {#2} }{#1}
  }
```

(*End of definition for* `\__hook_apply_rule_<:nnn` *and* `\__hook_apply_rule_>:nnn`.)

`\__hook_apply_rule_xE:nnn`
`\__hook_apply_rule_xW:nnn`

These relations make two labels incompatible within a hook. `xE` makes raises an error if the labels are found in the same hook, and `xW` makes it a warning.

```
\cs_new_protected:cpn { __hook_apply_rule_xE:nnn } #1#2#3
  {
    \__hook_debug:n { \__hook_msg_pair_found:nnn {#1} {#2} {#3} }
    \msg_error:nnnnnn { hooks } { labels-incompatible }
      {#1} {#2} {#3} { 1 }
    \use:c { __hook_apply_rule_->:nnn } {#1} {#2} {#3}
    \use:c { __hook_apply_rule_<-:nnn } {#1} {#2} {#3}
  }
\cs_new_protected:cpn { __hook_apply_rule_xW:nnn } #1#2#3
  {
    \__hook_debug:n { \__hook_msg_pair_found:nnn {#1} {#2} {#3} }
    \msg_warning:nnnnnn { hooks } { labels-incompatible }
      {#1} {#2} {#3} { 0 }
  }
```

(*End of definition for* `\__hook_apply_rule_xE:nnn` *and* `\__hook_apply_rule_xW:nnn`.)

`\__hook_apply_rule_->:nnn`
`\__hook_apply_rule_<-:nnn`

If we see `->` we have to drop code for label `#3` and carry on. We could do a little better and drop everything for that label since it doesn't matter where we put such empty code. However that would complicate the algorithm a lot with little gain.[12] So we still unnecessarily try to sort it in and depending on the rules that might result in a loop that is otherwise resolved. If that turns out to be a real issue, we can improve the code.

Here the code is removed from `\l__hook_cur_hook_tl` rather than `#3` because the latter may be `??`, and the default hook doesn't store any code. Removing it instead from `\l__hook_cur_hook_tl` makes the default rules `->` and `<-` work properly.

```
\cs_new_protected:cpn { __hook_apply_rule_->:nnn } #1#2#3
  {
    \__hook_debug:n
      {
        \__hook_msg_pair_found:nnn {#1} {#2} {#3}
        \iow_term:e{--->~ Drop~ '#2'~ code~ from~
          \iow_char:N \\ g__hook_ \l__hook_cur_hook_tl _code_prop ~
          because~ of~ '#1' }
      }
    \prop_put:Nnn \l__hook_work_prop {#2} { }
  }
\cs_new_protected:cpn { __hook_apply_rule_<-:nnn } #1#2#3
  {
    \__hook_debug:n
      {
        \__hook_msg_pair_found:nnn {#1} {#2} {#3}
        \iow_term:e{--->~ Drop~ '#1'~ code~ from~
          \iow_char:N \\ g__hook_ \l__hook_cur_hook_tl _code_prop ~
          because~ of~ '#2' }
      }
    \prop_put:Nnn \l__hook_work_prop {#1} { }
  }
```

[12] This also has the advantage that the result of the sorting doesn't change, as it might otherwise do (for unrelated chunks) if we aren't careful.

(*End of definition for* `\__hook_apply_rule_->:nnn` *and* `\__hook_apply_rule_<-:nnn`.)

`\__hook_apply_-rule_<:nnn`
`\__hook_apply_-rule_>:nnn`
`\__hook_apply_-rule_<-:nnn`
`\__hook_apply_-rule_->:nnn`
`\__hook_apply_-rule_xW:nnn`
`\__hook_apply_-rule_xE:nnn`

Reversed rules.

```
\cs_new_eq:cc { __hook_apply_-rule_<:nnn } { __hook_apply_rule_>:nnn }
\cs_new_eq:cc { __hook_apply_-rule_>:nnn } { __hook_apply_rule_<:nnn }
\cs_new_eq:cc { __hook_apply_-rule_<-:nnn } { __hook_apply_rule_<-:nnn }
\cs_new_eq:cc { __hook_apply_-rule_->:nnn } { __hook_apply_rule_->:nnn }
\cs_new_eq:cc { __hook_apply_-rule_xE:nnn } { __hook_apply_rule_xE:nnn }
\cs_new_eq:cc { __hook_apply_-rule_xW:nnn } { __hook_apply_rule_xW:nnn }
```

(*End of definition for* `\__hook_apply_-rule_<:nnn` *and others.*)

`\__hook_msg_pair_found:nnn`

A macro to avoid moving this many tokens around.

```
\cs_new_protected:Npn \__hook_msg_pair_found:nnn #1#2#3
  {
    \iow_term:e{~ \str_if_eq:nnTF {#3} {??} {default} {~normal} ~
        rule~ \__hook_label_pair:nn {#1} {#2}:~
        \use:c { g__hook_#3_rule_ \__hook_label_pair:nn {#1} {#2} _tl } ~
        found}
  }
```

(*End of definition for* `\__hook_msg_pair_found:nnn`.)

`\__hook_debug_label_data:N`

```
\cs_new_protected:Npn \__hook_debug_label_data:N #1 {
  \iow_term:e{Code~ labels~ for~ sorting:}
  \iow_term:e{~ \seq_use:Nnnn\l__hook_labels_seq {~and~}{,~}{~and~} }
  \iow_term:e{^^J Data~ structure~ for~ label~ rules:}
  \prop_map_inline:Nn #1
       {
         \iow_term:e{~ ##1~ =~ \tl_use:c{ \__hook_tl_csname:n {##1} }~ ->~
           \seq_use:cnnn{ \__hook_seq_csname:n {##1} }{~->~}{~->~}{~->~}
         }
       }
  \iow_term:e{}
}
```

(*End of definition for* `\__hook_debug_label_data:N`.)

`\hook_show:n`
`\hook_log:n`
`\__hook_log_line:e`
`\__hook_log_line_indent:e`
`\__hook_log:nN`

This writes out information about the hook given in its argument onto the `.log` file and the terminal, if `\show_hook:n` is used. Internally both share the same structure, except that at the end, `\hook_show:n` triggers TEX's prompt.

```
\cs_new_protected:Npn \hook_log:n #1
  {
    \cs_set_eq:NN \__hook_log_cmd:e \iow_log:e
    \__hook_normalize_hook_args:Nn \__hook_log:nN {#1} \tl_log:e
  }
\cs_if_exist:NTF \iow_show:e
  {
    \cs_new_protected:Npn \hook_show:n #1
      {
        \cs_set_eq:NN \__hook_log_cmd:e \iow_show:e
        \__hook_normalize_hook_args:Nn \__hook_log:nN {#1} \tl_show:e
      }
```

```
  }
  {
    \cs_new_protected:Npn \hook_show:n #1
      {
        \cs_set_eq:NN \__hook_log_cmd:e \iow_term:e
        \__hook_normalize_hook_args:Nn \__hook_log:nN {#1} \tl_show:e
      }
  }
\cs_new_protected:Npn \__hook_log_line:e #1
  { \__hook_log_cmd:e { >~#1 } }
\cs_new_protected:Npn \__hook_log_line_indent:e #1
  { \__hook_log_cmd:e { >~\@spaces #1 } }

⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_log:nN}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_log:nN #1 #2
  {
    \__hook_if_deprecated_generic:nT {#1}
      {
        \__hook_deprecated_generic_warn:n {#1}
        \__hook_do_deprecated_generic:Nn \__hook_log:nN {#1} #2
        \exp_after:wN \use_none:nnnnnnnnn \use_none:nnnnn
      }
    \__hook_preamble_hook:n {#1}
    \__hook_log_cmd:e
      {
        ^^J ->~The~
        \__hook_if_generic:nT {#1} { generic~ }
        hook~'#1'
        \__hook_if_disabled:nF {#1}
          {
            \exp_args:Nne \__hook_print_args:nn {#1}
              {
                \int_eval:n
                  { \str_count:e { \__hook_parameter:n {#1} } / 3 }
              }
          }
        :
      }

    \__hook_if_usable:nF {#1}
      { \__hook_log_line:e { The~hook~is~not~declared. } }
    \__hook_if_disabled:nT {#1}
      { \__hook_log_line:e { The~hook~is~disabled. } }
    \hook_if_empty:nTF {#1}
      { #2 { The~hook~is~empty } }
      {
        \__hook_log_line:e { Code~chunks: }
        \bool_lazy_or:nnTF
          { ! \prop_if_exist_p:c { g__hook_#1_code_prop } }
          { \prop_if_empty_p:c { g__hook_#1_code_prop } }
          { \__hook_log_line_indent:e { --- } }
          {
            \prop_map_inline:cn { g__hook_#1_code_prop }
              {
```

```
            \exp_after:wN \cs_set:Npn \exp_after:wN \__hook_tmp:w
              \c__hook_nine_parameters_tl {##2}
            \__hook_log_line_indent:e
              { ##1~->~\cs_replacement_spec:N \__hook_tmp:w }
          }
      }
```

If there is code in the `top-level` token list, print it:

```
    \__hook_log_line:e
      {
        Document-level~(top-level)~code
        \__hook_if_usable:nT {#1}
          { ~(executed~\__hook_if_reversed:nTF {#1} {first} {last} ) } :
      }
    \__hook_log_line_indent:e
      {
        \__hook_cs_if_empty:cTF { __hook_toplevel~#1 }
          { --- }
          { -> ~ \cs_replacement_spec:c { __hook_toplevel~#1 } }
      }
    \__hook_log_line:e { Extra~code~for~next~invocation: }
    \__hook_log_line_indent:e
      {
        \__hook_cs_if_empty:cTF { __hook_next~#1 }
          { --- }
```

If the token list is not empty we want to display it but without the first tokens (the code to clear itself) so we call a helper command to get rid of them.

```
          {
            -> ~ \exp_last_unbraced:Nf \__hook_log_next_code:w
              { \cs_replacement_spec:c { __hook_next~#1 } }
          }
      }
```

Loop through the rules in a hook and for every rule found, print it. If no rule is there, print `---`. The boolean `\l__hook_tmpa_bool` here indicates if the hook has no rules.

```
    \__hook_log_line:e { Rules: }
    \bool_set_true:N \l__hook_tmpa_bool
    \__hook_list_rules:nn {#1}
      {
        \bool_set_false:N \l__hook_tmpa_bool
        \__hook_log_line_indent:e
          {
            ##2~ with~
            \str_if_eq:nnT {##3} {??} { default~ }
            relation~ ##1
          }
      }
    \bool_if:NT \l__hook_tmpa_bool
      { \__hook_log_line_indent:e { --- } }
```

When the hook is declared (that is, the sorting algorithm is applied to that hook) and not empty

```
      \bool_lazy_and:nnTF
        { \__hook_if_usable_p:n {#1} }
        { ! \hook_if_empty_p:n {#1} }
      {
        \__hook_log_line:e
          {
            Execution~order
            \bool_if:NTF \l__hook_tmpa_bool
              { \__hook_if_reversed:nT {#1} { ~(after~reversal) } }
              { ~(after~
                \__hook_if_reversed:nT {#1} { reversal~and~ }
                applying~rules)
              } :
          }
        #2 % \tl_show:n
          {
            \@spaces
            \clist_if_empty:cTF { g__hook_#1_labels_clist }
              { --- }
              { \clist_use:cn { g__hook_#1_labels_clist } { ,~ } }
          }
      }
      {
        \__hook_log_line:e { Execution~order: }
        #2
          {
            \@spaces Not~set~because~the~hook~ \__hook_if_usable:nTF {#1}
              { code~pool~is~empty }
              { is~\__hook_if_disabled:nTF {#1} {disabled} {undeclared} }
          }
      }
  }
}
\EndIncludeInRelease
%
\IncludeInRelease{2020/10/01}{\__hook_log:nN}
                 {Hooks~with~args}
\cs_new_protected:Npn \__hook_log:nN #1 #2
  {
    \__hook_if_deprecated_generic:nT {#1}
      {
        \__hook_deprecated_generic_warn:n {#1}
        \__hook_do_deprecated_generic:Nn \__hook_log:nN {#1} #2
        \exp_after:wN \use_none:nnnnnnnnn \use_none:nnnnn
      }
    \__hook_preamble_hook:n {#1}
    \__hook_log_cmd:e
      { ^^J ->~The~ \__hook_if_generic:nT
                      {#1} { generic~ } hook~'#1': }
    \__hook_if_usable:nF {#1}
      { \__hook_log_line:e { The~hook~is~not~declared. } }
    \__hook_if_disabled:nT {#1}
      { \__hook_log_line:e { The~hook~is~disabled. } }
    \hook_if_empty:nTF {#1}
```

```
⟨latexrelease⟩      { #2 { The~hook~is~empty } }
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_log_line:e { Code~chunks: }
⟨latexrelease⟩        \prop_if_empty:cTF { g__hook_#1_code_prop }
⟨latexrelease⟩          { \__hook_log_line_indent:e { --- } }
⟨latexrelease⟩          {
⟨latexrelease⟩            \prop_map_inline:cn { g__hook_#1_code_prop }
⟨latexrelease⟩              { \__hook_log_line_indent:e
⟨latexrelease⟩                  { ##1~->~\tl_to_str:n {##2} } }
⟨latexrelease⟩          }
⟨latexrelease⟩        \__hook_log_line:e
⟨latexrelease⟩          {
⟨latexrelease⟩            Document-level~(top-level)~code
⟨latexrelease⟩            \__hook_if_usable:nT {#1}
⟨latexrelease⟩              { ~(executed~
⟨latexrelease⟩                \__hook_if_reversed:nTF {#1} {first} {last} ) } :
⟨latexrelease⟩          }
⟨latexrelease⟩        \__hook_log_line_indent:e
⟨latexrelease⟩          {
⟨latexrelease⟩            \tl_if_empty:cTF { __hook_toplevel~#1 }
⟨latexrelease⟩              { --- }
⟨latexrelease⟩              { -> ~ \exp_args:Nv \tl_to_str:n
⟨latexrelease⟩                                  { __hook_toplevel~#1 } }
⟨latexrelease⟩          }
⟨latexrelease⟩        \__hook_log_line:e { Extra~code~for~next~invocation: }
⟨latexrelease⟩        \__hook_log_line_indent:e
⟨latexrelease⟩          {
⟨latexrelease⟩            \tl_if_empty:cTF { __hook_next~#1 }
⟨latexrelease⟩              { --- }
⟨latexrelease⟩              { ->~ \exp_args:Nv \__hook_log_next_code:n
⟨latexrelease⟩                                  { __hook_next~#1 } }
⟨latexrelease⟩          }
⟨latexrelease⟩        \__hook_log_line:e { Rules: }
⟨latexrelease⟩        \bool_set_true:N \l__hook_tmpa_bool
⟨latexrelease⟩        \__hook_list_rules:nn {#1}
⟨latexrelease⟩          {
⟨latexrelease⟩            \bool_set_false:N \l__hook_tmpa_bool
⟨latexrelease⟩            \__hook_log_line_indent:e
⟨latexrelease⟩              {
⟨latexrelease⟩                ##2~ with~
⟨latexrelease⟩                \str_if_eq:nnT {##3} {??} { default~ }
⟨latexrelease⟩                relation~ ##1
⟨latexrelease⟩              }
⟨latexrelease⟩          }
⟨latexrelease⟩        \bool_if:NT \l__hook_tmpa_bool
⟨latexrelease⟩          { \__hook_log_line_indent:e { --- } }
⟨latexrelease⟩        \bool_lazy_and:nnTF
⟨latexrelease⟩            { \__hook_if_usable_p:n {#1} }
⟨latexrelease⟩            { ! \hook_if_empty_p:n {#1} }
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_log_line:e
⟨latexrelease⟩              {
⟨latexrelease⟩                Execution~order
⟨latexrelease⟩                \bool_if:NTF \l__hook_tmpa_bool
```

```
⟨latexrelease⟩                  { \__hook_if_reversed:nT
⟨latexrelease⟩                      {#1}{ ~(after~reversal) } }
⟨latexrelease⟩                  { ~(after~
⟨latexrelease⟩                    \__hook_if_reversed:nT {#1} { reversal~and~ }
⟨latexrelease⟩                    applying~rules)
⟨latexrelease⟩                  } :
⟨latexrelease⟩              }
⟨latexrelease⟩            #2 % \tl_show:n
⟨latexrelease⟩              {
⟨latexrelease⟩                \@spaces
⟨latexrelease⟩                \clist_if_empty:cTF { g__hook_#1_labels_clist }
⟨latexrelease⟩                  { --- }
⟨latexrelease⟩                  { \clist_use:cn
⟨latexrelease⟩                      { g__hook_#1_labels_clist } { ,~ } }
⟨latexrelease⟩              }
⟨latexrelease⟩          }
⟨latexrelease⟩          {
⟨latexrelease⟩            \__hook_log_line:e { Execution~order: }
⟨latexrelease⟩            #2
⟨latexrelease⟩              {
⟨latexrelease⟩                \@spaces Not~set~because~the~hook~
⟨latexrelease⟩                    \__hook_if_usable:nTF {#1}
⟨latexrelease⟩                  { code~pool~is~empty }
⟨latexrelease⟩                  { is~\__hook_if_disabled:nTF
⟨latexrelease⟩                      {#1} {disabled} {undeclared} }
⟨latexrelease⟩              }
⟨latexrelease⟩          }
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

To display the code for next invocation only (i.e., from `\AddToHookNext` we have to remove the string `\__hook_clear_next:n{`⟨*hook*⟩`}`, so the simplest is to use a macro delimited by a $\}_1 2$.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_log_next_code:n}
⟨latexrelease⟩                  {Hooks~with~args}
\__hook_log_next_code:n
\exp_last_unbraced:NNNNo
\cs_new:Npn \__hook_log_next_code:w #1 \c_right_brace_str { }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_log_next_code:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset:Npn \__hook_log_next_code:n #1
⟨latexrelease⟩  { \exp_args:No \tl_to_str:n { \use_none:nn #1 } }
⟨latexrelease⟩\EndIncludeInRelease
```

Pretty-prints the number of arguments of a hook.

```
\cs_new:Npn \__hook_print_args:nn #1 #2
  {
    \int_compare:nNnT {#2} > { 0 }
      {
\__hook_print_args:n    \__hook_if_declared:nT {#1} { \use_none:nnn }
        \__hook_if_cmd_hook:nT {#1}
          { \use_i:nnn { ~ (unknown ~ } }
        \use:n { ~ (#2 ~ }
```

```
        argument \int_compare:nNnT {#2} > { 1 } { s } )
    }
}
```

(*End of definition for* `\hook_show:n` *and others. These functions are documented on page* 232.)

`\__hook_list_rules:nn`
`\__hook_list_one_rule:nnn`
`\__hook_list_if_rule_exists:nnnF`

This macro takes a ⟨*hook*⟩ and an ⟨*inline function*⟩ and loops through each pair of ⟨*labels*⟩ in the ⟨*hook*⟩, and if there is a relation between this pair of ⟨*labels*⟩, the ⟨*inline function*⟩ is executed with `#1` = ⟨*relation*⟩, `#2` = ⟨$label_1$⟩|⟨$label_2$⟩, and `#3` = ⟨*hook*⟩ (the latter may be the argument `#1` to `\__hook_list_rules:nn`, or `??` if it is a default rule).

```
\cs_new_protected:Npn \__hook_list_rules:nn #1 #2
  {
    \prop_if_exist:cT { g__hook_#1_code_prop }
      {
        \cs_set_protected:Npn \__hook_tmp:w ##1 ##2 ##3 {#2}
        \prop_map_inline:cn { g__hook_#1_code_prop }
          {
            \prop_map_inline:cn { g__hook_#1_code_prop }
              {
                \__hook_if_label_case:nnnnn {##1} {####1}
                  { \prop_map_break: }
                  { \__hook_list_one_rule:nnn {##1} {####1} }
                  { \__hook_list_one_rule:nnn {####1} {##1} }
                      {#1}
              }
          }
      }
  }
```

These two are quite similar to `\__hook_apply_label_pair:nnn` and `\__hook_label_if_exist_apply:nnnF`, respectively, but rather than applying the rule, they pass it to the ⟨*inline function*⟩.

```
\cs_new_protected:Npn \__hook_list_one_rule:nnn #1#2#3
  {
    \__hook_list_if_rule_exists:nnnF {#1} {#2} {#3}
      { \__hook_list_if_rule_exists:nnnF {#1} {#2} { ?? } { } }
  }
\cs_new_protected:Npn \__hook_list_if_rule_exists:nnnF #1#2#3
  {
    \if_cs_exist:w g__hook_ #3 _rule_ #1 | #2 _tl \cs_end:
      \exp_args:Nv \__hook_tmp:w
        { g__hook_ #3 _rule_ #1 | #2 _tl } { #1 | #2 } {#3}
      \exp_after:wN \use_none:nn
    \fi:
    \use:n
  }
```

(*End of definition for* `\__hook_list_rules:nn`, `\__hook_list_one_rule:nnn`, *and* `\__hook_list_if_rule_exists:nnnF`.)

`\__hook_debug_print_rules:n`

A shorthand for debugging that prints similar to `\prop_show:N`.

```
\cs_new_protected:Npn \__hook_debug_print_rules:n #1
  {
```

```
    \iow_term:n { The~hook~#1~contains~the~rules: }
    \cs_set_protected:Npn \__hook_tmp:w ##1
      {
        \__hook_list_rules:nn {#1}
          {
            \iow_term:e
              {
                > ##1 {####2} ##1 => ##1 {####1}
                \str_if_eq:nnT {####3} {??} { ~(default) }
              }
          }
      }
    \exp_args:No \__hook_tmp:w { \use:nn { ~ } { ~ } }
  }
```

(*End of definition for* `\__hook_debug_print_rules:n`.)

4.8 Specifying code for next invocation

`\hook_gput_next_code:nn`

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_gput_next_code:nn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \hook_gput_next_code:nn #1 #2
  {
    \__hook_replacing_args_false:
    \__hook_normalize_hook_args:Nn \__hook_gput_next_code:nn {#1} {#2}

    \__hook_debug:n{\iow_term:e{[lthooks]~ Add~ to~
                      \__hook_if_usable:nF {#1} { undeclared~ }
                      hook~ '#1'~ ( next~ invocation~ only )
                      \on@line
                      ^^J [lthooks] \@spaces
                      <-~ \tl_to_str:n{#2}
      }
    }

    \__hook_replacing_args_reset:
  }
\cs_new_protected:Npn \hook_gput_next_code_with_args:nn #1 #2
  {
    \__hook_replacing_args_true:
    \__hook_normalize_hook_args:Nn \__hook_gput_next_code:nn {#1} {#2}

    \__hook_debug:n{\iow_term:e{[lthooks]~ Add~ to~
                      \__hook_if_usable:nF {#1} { undeclared~ }
                      hook~ '#1'~ ( next~ invocation~ only )
                      \on@line
                      ^^J [lthooks] \@spaces
                      <-~ \tl_to_str:n{#2}
      }
    }

    \__hook_replacing_args_reset:
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_gput_next_code:nn}
```

```
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \hook_gput_next_code:nn #1
⟨latexrelease⟩  { \__hook_normalize_hook_args:Nn
⟨latexrelease⟩          \__hook_gput_next_code:nn {#1} }
⟨latexrelease⟩\cs_gset_protected:Npn \hook_gput_next_code_with_args:nn #1 #2 { }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_gput_next_code:nn`*. This function is documented on page 231.*)

`\__hook_gput_next_code:nn`

```
\cs_new_protected:Npn \__hook_gput_next_code:nn #1 #2
  {
    \__hook_if_disabled:nTF {#1}
      { \msg_error:nnn { hooks } { hook-disabled } {#1} }
      {
        \__hook_if_structure_exist:nTF {#1}
          { \__hook_gput_next_do:nn }
          { \__hook_try_declaring_generic_next_hook:nn }
              {#1} {#2}
      }
  }
```

(*End of definition for* `\__hook_gput_next_code:nn`*.*)

`\__hook_gput_next_do:nn` Start by sanity-checking with `\__hook_chk_args_allowed:nn`. Then check if the "next code" token list is empty: if so we need to add a `\tl_gclear:c` to clear it, so the code lasts for one usage only. The token list is cleared early so that nested usages don't get lost. `\tl_gclear:c` is used instead of `\tl_gclear:N` in case the hook is used in an expansion-only context, so the token list doesn't expand before `\tl_gclear:N`: that would make an infinite loop. Also in case the main code token list is empty, the hook code has to be updated to add the next execution token list.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_gput_next_do:nn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_gput_next_do:nn #1
  {
    \__hook_init_structure:n {#1}
    \__hook_chk_args_allowed:nn {#1} { AddToHookNext }
    \__hook_cs_if_empty:cT { __hook~#1 }
      { \__hook_update_hook_code:n {#1} }
    \__hook_cs_if_empty:cT { __hook_next~#1 }
      { \__hook_next_gset:nn {#1} { \__hook_clear_next:n {#1} } }
    \__hook_cs_gput_right:nnn { _next } {#1}
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_gput_next_do:nn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_gput_next_do:nn #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \exp_args:Nc \__hook_gput_next_do:Nnn
⟨latexrelease⟩      { __hook_next~#1 } {#1}
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_gput_next_do:Nnn #1 #2
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_empty:cT { __hook~#2 }
```

```
⟨latexrelease⟩       { \__hook_update_hook_code:n {#2} }
⟨latexrelease⟩     \tl_if_empty:NT #1
⟨latexrelease⟩       { \__hook_tl_gset:Nn #1 { \__hook_clear_next:n {#2} } }
⟨latexrelease⟩     \__hook_tl_gput_right:Nn #1
⟨latexrelease⟩   }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_gput_next_do:nn`.)

`\hook_gclear_next_code:n` Discard anything set up for next invocation of the hook.

```
\cs_new_protected:Npn \hook_gclear_next_code:n #1
  { \__hook_normalize_hook_args:Nn \__hook_clear_next:n {#1} }
```

(*End of definition for* `\hook_gclear_next_code:n`. *This function is documented on page* 231.)

`\__hook_clear_next:n`

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_clear_next:n}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_clear_next:n #1
  { \__hook_next_gset:nn {#1} { } }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_clear_next:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_clear_next:n #1
⟨latexrelease⟩  { \cs_gset_eq:cN { __hook_next~#1 } \c_empty_tl }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_clear_next:n`.)

4.9 Using the hook

`\hook_use:n` `\__hook_use_initialized:n` `\__hook_preamble_hook:n`

`\hook_use:n` as defined here is used in the preamble, where hooks aren't initialized by default. `\__hook_use_initialized:n` is also defined, which is the non-`\protected` version for use within the document. Their definition is identical, except for the `\__hook_preamble_hook:n` (which wouldn't hurt in the expandable version, but it would be an unnecessary extra expansion).

`\__hook_use_initialized:n` holds the expandable definition while in the preamble. `\__hook_preamble_hook:n` initializes the hook in the preamble, and is redefined to `\use_none:n` at `\begin{document}`.

Both versions do the same thing internally: they check that the hook exists as given, and if so they use it as quickly as possible.

At `\begin{document}`, all hooks are initialized, and any change in them causes an update, so `\hook_use:n` can be made expandable. This one is better not protected so that it can expand into nothing if containing no code. Also important in case of generic hooks that we do not generate a `\relax` as a side effect of checking for a csname. In contrast to the TEX low-level `\csname ...\endcsname` construct `\tl_if_exist:c` is careful to avoid this.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_use:n}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \hook_use:n #1
  {
    \__hook_preamble_hook:n {#1}
    \__hook_use_initialized:n {#1}
```

```
  }
\cs_new:Npn \__hook_use_initialized:n #1
  {
    \if_cs_exist:w __hook~#1 \cs_end:
      \cs:w __hook~#1 \use_i:nn
    \fi:
    \use_none:n
    \cs_end:
  }
\cs_new_protected:Npn \__hook_preamble_hook:n #1
  {
    \if_cs_exist:w __hook~#1 \cs_end:
      \__hook_initialize_hook_code:n {#1}
    \fi:
  }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\hook_use:n}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\cs_new_protected:Npn \hook_use:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_exist:cT { __hook~#1 }
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_preamble_hook:n {#1}
⟨latexrelease⟩        \cs:w __hook~#1 \cs_end:
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new:Npn \__hook_use_initialized:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \if_cs_exist:w __hook~#1 \cs_end:
⟨latexrelease⟩      \cs:w __hook~#1 \exp_after:wN \cs_end:
⟨latexrelease⟩    \fi:
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new_protected:Npn \__hook_preamble_hook:n #1
⟨latexrelease⟩  { \__hook_initialize_hook_code:n {#1} }
⟨latexrelease⟩\cs_new:Npn \hook_use:nnw #1 { }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_use:n}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\cs_new_protected:Npn \hook_use:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_exist:cTF { __hook~#1 }
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_preamble_hook:n {#1}
⟨latexrelease⟩        \cs:w __hook~#1 \cs_end:
⟨latexrelease⟩      }
⟨latexrelease⟩      { \__hook_use:wn #1 / \s__hook_mark {#1} }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new:Npn \__hook_use_initialized:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \if_cs_exist:w __hook~#1 \cs_end:
⟨latexrelease⟩    \else:
⟨latexrelease⟩      \__hook_use_undefined:w
⟨latexrelease⟩    \fi:
```

```
⟨latexrelease⟩    \cs:w __hook~#1 \__hook_use_end:
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new:Npn \__hook_use_undefined:w
⟨latexrelease⟩    #1 #2 __hook~#3 \__hook_use_end:
⟨latexrelease⟩  {
⟨latexrelease⟩    #1 % fi
⟨latexrelease⟩    \__hook_use:wn #3 / \s__hook_mark {#3}
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new_protected:Npn \__hook_preamble_hook:n #1
⟨latexrelease⟩  { \__hook_initialize_hook_code:n {#1} }
⟨latexrelease⟩\cs_new_eq:NN \__hook_use_end: \cs_end:
⟨latexrelease⟩\cs_new:Npn \hook_use:nnw #1 { }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_use:n`, `\__hook_use_initialized:n`, *and* `\__hook_preamble_hook:n`. *This function is documented on page* 230.)

`\hook_use:nnw`
`\__hook_use_initialized:nnw`

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_use:nnw}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \hook_use:nnw #1
  {
    \__hook_preamble_hook:n {#1}
    \__hook_use_initialized:nnw {#1}
  }
\cs_new:Npn \__hook_use_initialized:nnw #1 #2
  {
    \cs:w
      \if_cs_exist:w __hook~#1 \cs_end:
        __hook~#1
      \else:
        use_none: \prg_replicate:nn {#2} { n }
      \fi:
    \cs_end:
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_use:nnw}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset:Npn \hook_use:nnw #1 #2
⟨latexrelease⟩  { \use:c { use_none: \prg_replicate:nn {#2} { n } } }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_use:nnw` *and* `\__hook_use_initialized:nnw`. *This function is documented on page* 230.)

`\__hook_post_initialization_defs:`

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_post_initialization_defs:}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_post_initialization_defs:
  {
    \cs_gset_eq:NN \hook_use:n \__hook_use_initialized:n
    \cs_gset_eq:NN \hook_use:nnw \__hook_use_initialized:nnw
    \cs_gset_eq:NN \__hook_preamble_hook:n \use_none:n
    \cs_gset_eq:NN \__hook_post_initialization_defs: \prg_do_nothing:
  }
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_post_initialization_defs:}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_post_initialization_defs:
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_post_initialization_defs:`.)

`\__hook_use:wn`
`\__hook_try_file_hook:n`
`\__hook_if_usable_use:n`

`\__hook_use:wn` does a quick check to test if the current hook is a file hook: those need a special treatment. If it is not, the hook does not exist. If it is, then `\__hook_try_file_hook:n` is called, and checks that the current hook is a file-specific hook using `\__hook_if_file_hook:wTF`. If it's not, then it's a generic `file/` hook and is used if it exist.

If it is a file-specific hook, it passes through the same normalization as during declaration, and then it is used if defined. `\__hook_if_usable_use:n` checks if the hook exist, and calls `\__hook_preamble_hook:n` if so, then uses the hook.

```
⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\__hook_use:wn}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_use:wn}
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\cs_new:Npn \__hook_use:wn #1 / #2 \s__hook_mark #3
⟨latexrelease⟩  {
⟨latexrelease⟩    \str_if_eq:nnTF {#1} { file }
⟨latexrelease⟩      { \__hook_try_file_hook:n {#3} }
⟨latexrelease⟩      { } % Hook doesn't exist
⟨latexrelease⟩  }

⟨latexrelease⟩\cs_new_protected:Npn \__hook_try_file_hook:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_file_hook:wTF #1 / / \s__hook_mark
⟨latexrelease⟩      {
⟨latexrelease⟩        \exp_args:Ne \__hook_if_usable_use:n
⟨latexrelease⟩          { \exp_args:Ne \__hook_file_hook_normalize:n {#1} }
⟨latexrelease⟩      }
⟨latexrelease⟩      { \__hook_if_usable_use:n {#1} }
⟨latexrelease⟩           % file/ generic hook (e.g. file/before)
⟨latexrelease⟩  }

⟨latexrelease⟩\cs_new_protected:Npn \__hook_if_usable_use:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_exist:cT { __hook~#1 }
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_preamble_hook:n {#1}
⟨latexrelease⟩        \cs:w __hook~#1 \cs_end:
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_use:wn`, `\__hook_try_file_hook:n`, *and* `\__hook_if_usable_use:n`.)

`\hook_use_once:n`
`\hook_use_once:nnw`

For hooks that can and should be used only once we have a special use command that further inhibits the hook from getting more code added to it. This has the effect that any further code added to the hook is executed immediately rather than stored in the hook.

The code needs some gymnastics to prevent space trimming from the hook name, since `\hook_use:n` and `\hook_use_once:n` are documented to not trim spaces.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_use_once:nnw}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \hook_use_once:n #1
  {
    \__hook_if_execute_immediately:nF {#1}
      { \__hook_normalize_hook_args:Nn \__hook_use_once:nn
          { \use:n {#1} } { 0 } }
  }
\cs_new_protected:Npn \hook_use_once:nnw #1 #2
  {
    \__hook_if_execute_immediately:nF {#1}
      { \__hook_normalize_hook_args:Nn \__hook_use_once:nn
          { \use:n {#1} } {#2} }
  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_use_once:n` *and* `\hook_use_once:nnw`*. These functions are documented on page 230.*)

```
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_use_once:nnw}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \hook_use_once:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_if_execute_immediately:nF {#1}
⟨latexrelease⟩      { \__hook_normalize_hook_args:Nn \__hook_use_once:n
⟨latexrelease⟩          { \use:n {#1} } }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_gset:Npn \hook_use_once:nnw #1 #2
⟨latexrelease⟩  { \use:c { use_none: \prg_replicate:nn {#2} { n } } }
⟨latexrelease⟩\EndIncludeInRelease
```

`\__hook_use_once:nn`

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_use_once:nn}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_use_once:nn #1 #2
  {
    \__hook_preamble_hook:n {#1}
    \__hook_use_once_set:n {#1}
```

When a hook has arguments, the call to `\__hook_use_initialized:n`, should be the very last thing to happen, otherwise the arguments grabbed will be wrong. So, to clean up after the hook we need to cheat a bit and sneak the cleanup code at the end of the hook, along with the next execution code.

```
    \__hook_replacing_args_false:
    \__hook_cs_gput_right:nnn { _next } {#1}
      { \__hook_use_once_clear:n {#1} }
    \__hook_replacing_args_reset:
    \__hook_if_usable:nTF {#1}
      { \__hook_use_initialized:n {#1} }
      {
        \int_compare:nNnT {#2} > { 0 }
          { \use:c { use_none: \prg_replicate:nn {#2} { n } } }
      }
```

```
  }
⟨latexrelease⟩\EndIncludeInRelease
%
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_use_once:nn}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_use_once:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_preamble_hook:n {#1}
⟨latexrelease⟩    \__hook_use_once_set:n {#1}
⟨latexrelease⟩    \__hook_use_initialized:n {#1}
⟨latexrelease⟩    \__hook_use_once_clear:n {#1}
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_undefine:N \__hook_use_once:nn
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_use_once:nn`.)

`\__hook_use_once_set:n`
`\__hook_use_once_clear:n`

`\__hook_use_once_set:n` is used before the actual hook code is executed so that any usage of `\AddToHook` inside the hook causes the code to execute immediately. Setting `\g__hook_⟨hook⟩_reversed_tl` to `I` prevents further code from being added to the hook. `\__hook_use_once_clear:n` then clears the hook so that any further call to `\hook_use:n` or `\hook_use_once:n` will expand to nothing.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_use_once_clear:n}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new_protected:Npn \__hook_use_once_set:n #1
  { \__hook_tl_gset:cn { g__hook_#1_reversed_tl } { I } }
\cs_new_protected:Npn \__hook_use_once_clear:n #1
  {
    \__hook_code_gset:nn {#1} { }
    \__hook_next_gset:nn {#1} { }
    \__hook_toplevel_gset:nn {#1} { }
    \prop_gclear_new:c { g__hook_#1_code_prop }
  }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_use_once_clear:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_new_protected:Npn \__hook_use_once_clear:n #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_tl_gclear:c { __hook~#1 }
⟨latexrelease⟩    \__hook_tl_gclear:c { __hook_next~#1 }
⟨latexrelease⟩    \__hook_tl_gclear:c { __hook_toplevel~#1 }
⟨latexrelease⟩    \prop_gclear_new:c { g__hook_#1_code_prop }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_use_once_set:n` *and* `\__hook_use_once_clear:n`.)

`\__hook_if_execute_immediately_p:n`
`\__hook_if_execute_immediately:nTF`

To check whether the code being added should be executed immediately (that is, if the hook is a one-time hook), we check if `\g__hook_⟨hook⟩_reversed_tl` is `I`. The gymnastics around `\if:w` is there to allow the `reversed` token list to be empty.

```
\prg_new_conditional:Npnn \__hook_if_execute_immediately:n #1 { T, F, TF }
  {
    \exp_after:wN \__hook_use_none_delimit_by_s_mark:w
    \if:w I
```

```
      \if_cs_exist:w g__hook_#1_reversed_tl \cs_end:
        \cs:w g__hook_#1_reversed_tl \exp_after:wN \cs_end:
      \fi:
      X
    \s__hook_mark \prg_return_true:
  \else:
    \s__hook_mark \prg_return_false:
  \fi:
}
```

(*End of definition for* `\__hook_if_execute_immediately:nTF`.)

4.10 Querying a hook

Simpler data types, like token lists, have three possible states; they can exist and be empty, exist and be non-empty, and they may not exist, in which case emptiness doesn't apply (though `\tl_if_empty:N` returns false in this case).

Hooks are a bit more complicated: they have several other states as discussed in 4.4.2. A hook may exist or not, and either way it may or may not be empty (even a hook that doesn't exist may be non-empty) or may be disabled.

A hook is said to be empty when no code was added to it, either to its permanent code pool, or to its "next" token list. The hook doesn't need to be declared to have code added to its code pool (it may happen that a package *A* defines a hook `foo`, but it's loaded after package *B*, which adds some code to that hook. In this case it is important that the code added by package *B* is remembered until package *A* is loaded).

All other states can only be queried with internal tests as the different states are irrelevant for package code.

`\hook_if_empty_p:n`
`\hook_if_empty:nTF`

Test if a hook is empty (that is, no code was added to that hook). A ⟨*hook*⟩ being empty means that all three of its `\g__hook_`⟨*hook*⟩`_code_prop`, its `\__hook_toplevel_`⟨*hook*⟩ and its `\__hook_next_`⟨*hook*⟩ are empty.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\hook_if_empty:n}
⟨latexrelease⟩                  {Hooks~with~args}
\prg_new_conditional:Npnn \hook_if_empty:n #1 { p , T , F , TF }
  {
    \if:w
        T
        \prop_if_exist:cT { g__hook_#1_code_prop }
          { \prop_if_empty:cF { g__hook_#1_code_prop } { F } }
        \__hook_cs_if_empty:cF { __hook_toplevel~#1 } { F }
        \__hook_cs_if_empty:cF { __hook_next~#1 } { F }
        T
      \prg_return_true:
    \else:
      \prg_return_false:
    \fi:
  }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\hook_if_empty:n}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\prg_new_conditional:Npnn \hook_if_empty:n #1 { p , T , F , TF }
⟨latexrelease⟩  {
```

```
⟨latexrelease⟩     \__hook_if_structure_exist:nTF {#1}
⟨latexrelease⟩       {
⟨latexrelease⟩         \bool_lazy_and:nnTF
⟨latexrelease⟩             { \prop_if_empty_p:c { g__hook_#1_code_prop } }
⟨latexrelease⟩             {
⟨latexrelease⟩               \bool_lazy_and_p:nn
⟨latexrelease⟩                 { \tl_if_empty_p:c { __hook_toplevel~#1 } }
⟨latexrelease⟩                 { \tl_if_empty_p:c { __hook_next~#1 } }
⟨latexrelease⟩             }
⟨latexrelease⟩           { \prg_return_true: }
⟨latexrelease⟩           { \prg_return_false: }
⟨latexrelease⟩       }
⟨latexrelease⟩       { \prg_return_true: }
⟨latexrelease⟩   }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\hook_if_empty:nTF`. *This function is documented on page* 232.)

`\__hook_if_usable_p:n`
`\__hook_if_usable:nTF`

A hook is usable if the token list that stores the sorted code for that hook, `\__hook␣⟨hook⟩`, exists. The property list `\g__hook_⟨hook⟩_code_prop` cannot be used here because often it is necessary to add code to a hook without knowing if such hook was already declared, or even if it will ever be (for example, in case the package that defines it isn't loaded).

```
\prg_new_conditional:Npnn \__hook_if_usable:n #1 { p , T , F , TF }
  {
    \cs_if_exist:cTF { __hook~#1 }
      { \prg_return_true: }
      { \prg_return_false: }
  }
```

(*End of definition for* `\__hook_if_usable:nTF`.)

`\__hook_if_structure_exist_p:n`
`\__hook_if_structure_exist:nTF`

An internal check if the hook has already its basic internal structure set up with `\__hook_init_structure:n`. This means that the hook was already used somehow (a code chunk or rule was added to it), but it still wasn't declared with `\hook_new:n`.

```
\prg_new_conditional:Npnn \__hook_if_structure_exist:n #1 { p , T , F , TF }
  {
    \prop_if_exist:cTF { g__hook_#1_code_prop }
      { \prg_return_true: }
      { \prg_return_false: }
  }
```

(*End of definition for* `\__hook_if_structure_exist:nTF`.)

`\__hook_if_declared_p:n`
`\__hook_if_declared:nTF`

Internal test to check if the hook was officially declared with `\hook_new:n` or a variant.

```
\prg_new_conditional:Npnn \__hook_if_declared:n #1 { p, T, F, TF }
  {
    \tl_if_exist:cTF { g__hook_#1_declared_tl }
      { \prg_return_true: }
      { \prg_return_false: }
  }
```

(*End of definition for* `\__hook_if_declared:nTF`.)

__hook_if_reversed_p:n
__hook_if_reversed:nTF

An internal conditional that checks if a hook is reversed.

```
\prg_new_conditional:Npnn \__hook_if_reversed:n #1 { p , T , F , TF }
  {
    \exp_after:wN \__hook_use_none_delimit_by_s_mark:w
    \if:w - \cs:w g__hook_#1_reversed_tl \cs_end:
      \s__hook_mark \prg_return_true:
    \else:
      \s__hook_mark \prg_return_false:
    \fi:
  }
```

(*End of definition for* `\__hook_if_reversed:nTF`.)

__hook_if_generic_p:n
__hook_if_generic:nTF
__hook_if_deprecated_generic_p:n
__hook_if_deprecated_generic:nTF

An internal conditional that checks if a name belongs to a generic hook. The deprecated version needs to check if `#3` is empty to avoid returning true on `file/before`, for example.

```
\prg_new_conditional:Npnn \__hook_if_generic:n #1 { T, TF }
  { \__hook_if_generic:w #1 / / / \s__hook_mark }
\cs_new:Npn \__hook_if_generic:w #1 / #2 / #3 / #4 \s__hook_mark
  {
    \cs_if_exist:cTF { c__hook_generic_#1/./#3_tl }
      { \prg_return_true: }
      { \prg_return_false: }
  }
\prg_new_conditional:Npnn \__hook_if_deprecated_generic:n #1 { T, TF }
  { \__hook_if_deprecated_generic:w #1 / / / \s__hook_mark }
\cs_new:Npn \__hook_if_deprecated_generic:w #1 / #2 / #3 / #4 \s__hook_mark
  {
    \cs_if_exist:cTF { c__hook_deprecated_#1/./#2_tl }
      {
        \tl_if_empty:nTF {#3}
          { \prg_return_false: }
          { \prg_return_true: }
      }
      { \prg_return_false: }
  }
```

(*End of definition for* `\__hook_if_generic:nTF` *and* `\__hook_if_deprecated_generic:nTF`.)

__hook_if_cmd_hook_p:n
__hook_if_cmd_hook:nTF
__hook_if_cmd_hook_p:w
__hook_if_cmd_hook:wTF

An internal conditional that checks if a given hook is a valid generic `cmd` hook.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_if_cmd_hook:n}
⟨latexrelease⟩                          {Hooks~with~args}
\prg_new_conditional:Npnn \__hook_if_cmd_hook:n #1 { T }
  { \__hook_if_cmd_hook:w #1 / / / \s__hook_mark }
\cs_new:Npn \__hook_if_cmd_hook:w #1 / #2 / #3 / #4 \s__hook_mark
  {
    \if:w Y
          \str_if_eq:nnF {#1} { cmd } { N }
          \tl_if_exist:cF { c__hook_generic_#1/./#3_tl } { N }
          Y
      \prg_return_true:
    \else:
      \prg_return_false:
    \fi:
  }
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\__hook_if_cmd_hook:n}
⟨latexrelease⟩                    {Hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_if_cmd_hook:nT
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_if_cmd_hook:nTF` *and* `\__hook_if_cmd_hook:wTF`.)

`\__hook_if_generic_reversed_p:n` `\__hook_if_generic_reversed:n`*TF*

An internal conditional that checks if a name belongs to a generic reversed hook.

```
\prg_new_conditional:Npnn \__hook_if_generic_reversed:n #1 { T }
  { \__hook_if_generic_reversed:w #1 / / / \scan_stop: }
\cs_new:Npn \__hook_if_generic_reversed:w #1 / #2 / #3 / #4 \scan_stop:
  {
    \if_charcode:w - \cs:w c__hook_generic_#1/./#3_tl \cs_end:
      \prg_return_true:
    \else:
      \prg_return_false:
    \fi:
  }
```

(*End of definition for* `\__hook_if_generic_reversed:nTF`.)

`\__hook_if_replacing_args:TF` `\__hook_misused_if_replacing_args:nn` `\__hook_replacing_args_true:` `\__hook_replacing_args_false:` `\__hook_replacing_args_reset:` `\g__hook_replacing_stack_seq`

An internal conditional that checks if the code being added to the hook contains arguments.

```
\seq_new:N \g__hook_replacing_stack_seq
\cs_new:Npn \__hook_misused_if_replacing_args:nn #1 #2
  {
    \msg_expandable_error:nnn { latex2e } { should-not-happen }
      { Misused~\__hook_if_replacing_args:. }
  }
\cs_new:Npn \__hook_if_replacing_args:TF
  { \__hook_misused_if_replacing_args:nn }
\cs_new_protected:Npn \__hook_replacing_args_true:
  {
    \seq_gpush:No \g__hook_replacing_stack_seq
      { \__hook_if_replacing_args:TF }
    \cs_set:Npn \__hook_if_replacing_args:TF { \use_i:nn }
  }
\cs_new_protected:Npn \__hook_replacing_args_false:
  {
    \seq_gpush:No \g__hook_replacing_stack_seq
      { \__hook_if_replacing_args:TF }
    \cs_set:Npn \__hook_if_replacing_args:TF { \use_ii:nn }
  }
\cs_new_protected:Npn \__hook_replacing_args_reset:
  {
    \seq_gpop:NN \g__hook_replacing_stack_seq \l__hook_return_tl
    \cs_gset_eq:NN \__hook_if_replacing_args:TF \l__hook_return_tl
  }
```

(*End of definition for* `\__hook_if_replacing_args:TF` *and others.*)

4.11 Messages

Hook errors are LaTeX kernel errors:

```
\prop_gput:Nnn \g_msg_module_type_prop { hooks } { LaTeX }
```

And so are kernel errors (this should move elsewhere eventually).

```
\prop_gput:Nnn \g_msg_module_type_prop { latex2e } { LaTeX }
\prop_gput:Nnn \g_msg_module_name_prop { latex2e } { kernel }

\msg_new:nnnn { hooks } { labels-incompatible }
  {
    Labels~'#1'~and~'#2'~are~incompatible
    \str_if_eq:nnF {#3} {??} { ~in~hook~'#3' } .~
    \int_compare:nNnTF {#4} = { 1 }
      { The~ code~ for~ both~ labels~ will~ be~ dropped. }
      { You~ may~ see~ errors~ later. }
  }
  { LaTeX~found~two~incompatible~labels~in~the~same~hook.~
    This~indicates~an~incompatibility~between~packages.  }

\msg_new:nnnn { hooks } { exists }
    { Hook~'#1'~ has~ already~ been~ declared. }
    { There~ already~ exists~ a~ hook~ declaration~ with~ this~
      name.\\
      Please~ use~ a~ different~ name~ for~ your~ hook.}

⟨latexrelease⟩\IncludeInRelease{2023/06/01}{too-many-args}
⟨latexrelease⟩                    {Hooks~with~args}

\msg_new:nnnn { hooks } { too-many-args }
  { Too~many~arguments~for~hook~'#1'. }
  {
    You~tried~to~declare~a~hook~with~#2~arguments,~but~a~
    hook~can~only~have~up~to~nine.~LaTeX~will~define~this~
    hook~with~nine~arguments.
  }

\msg_new:nnnn { hooks } { without-args }
  { Hook~'#1'~has~no~arguments. }
  {
    You~tried~to~use~\iow_char:N\\#2WithArguments~
    on~a~hook~that~takes~no~arguments.\\
    Check~the~usage~of~the~hook~or~use~\iow_char:N\\#2~instead.\\
    \\
    LaTeX~will~use~\iow_char:N\\#2.
  }

\msg_new:nnnn { hooks } { one-time-args }
  { You~can't~have~arguments~in~used~one-time~hook~'#1'. }
  {
    You~tried~to~use~\iow_char:N\\#2WithArguments~
    on~a~one-time~hook~that~has~already~been~used.~
    You~have~to~add~the~code~before~the~hook~is~used,~
    or~add~the~code~without~arguments~using~\iow_char:N\\#2~instead.\\
    \\
    LaTeX~will~use~\iow_char:N\\#2.
  }
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{too-many-args}
⟨latexrelease⟩                    {Hooks~with~args}
⟨latexrelease⟩\EndIncludeInRelease

\msg_new:nnnn { hooks } { hook-disabled }
  { Cannot~add~code~to~disabled~hook~'#1'. }
  {
    The~hook~'#1'~you~tried~to~add~code~to~was~previously~disabled~
    with~\iow_char:N\\hook_disable_generic:n~or~
    \iow_char:N\\DisableGenericHook,~so~
    it~cannot~have~code~added~to~it.
  }

\msg_new:nnn { hooks } { empty-label }
  {
    Empty~code~label~\msg_line_context:.~
    Using~'\__hook_currname_or_default:'~instead.
  }

\msg_new:nnn { hooks } { empty-hook }
  {
    Empty~hook~name~\msg_line_context:.
  }

\msg_new:nnn { hooks } { no-default-label }
  {
    Missing~(empty)~default~label~\msg_line_context:. \\
    This~command~was~ignored.
  }

\msg_new:nnnn { hooks } { unknown-rule }
  {
    Unknown~ relationship~ '#3'~
    between~ labels~ '#2'~ and~ '#4'~
    \str_if_eq:nnF {#1} {??} { ~in~hook~'#1' }. ~
    Perhaps~ a~ misspelling?
  }
  {
    The~ relation~ used~ not~ known~ to~ the~ system.~ Allowed~ values~ are~
    'before'~ or~ '<',~
    'after'~ or~ '>',~
    'incompatible-warning',~
    'incompatible-error',~
    'voids'~ or~
    'unrelated'.
  }

\msg_new:nnnn { hooks } { rule-too-late }
  {
    Sorting~rule~for~'#1'~hook~applied~too~late.\\
    Try~setting~this~rule~earlier.
  }
  {
    You~tried~to~set~the~ordering~of~hook~'#1'~using\\
    \ \ \iow_char:N\\DeclareHookRule{#1}{#2}{#3}{#4}\\
    but~hook~'#1'~was~already~used~as~a~one-time~hook,~
```

```
    thus~sorting~is\\
    no~longer~possible.~Declare~the~rule~
    before~the~hook~is~used.
  }
\msg_new:nnnn { hooks } { misused-top-level }
  {
    Illegal~use~of~\iow_char:N \\AddToHook{#1}[top-level]{...}.\\
    'top-level'~is~reserved~for~the~user's~document.
  }
  {
    The~'top-level'~label~is~meant~for~user~code~only,~and~should~only~
    be~used~(sparingly)~in~the~main~document.~Use~the~default~label~
    '\__hook_currname_or_default:'~for~this~\@cls@pkg,~or~another~
    suitable~label.
  }
\msg_new:nnn { hooks } { set-top-level }
  {
    You~cannot~change~the~default~label~#1~'top-level'.~Illegal \\
    \use:nn { ~ } { ~ } \iow_char:N \\#2{#3} \\
    \msg_line_context:.
  }
\msg_new:nnn { hooks } { extra-pop-label }
  {
    Extra~\iow_char:N \\PopDefaultHookLabel. \\
    This~command~will~be~ignored.
  }
\msg_new:nnn { hooks } { missing-pop-label }
  {
    Missing~\iow_char:N \\PopDefaultHookLabel. \\
    The~label~'#1'~was~pushed~but~never~popped.~Something~is~wrong.
  }
\msg_new:nnn { latex2e } { should-not-happen }
  {
    This~should~not~happen.~#1 \\
    Please~report~at~https://github.com/latex3/latex2e.
  }
\msg_new:nnn { hooks } { activate-disabled }
  {
    Cannot~ activate~ hook~ '#1'~ because~ it~ is~ disabled!
  }
\msg_new:nnn { hooks } { cannot-remove }
  {
    Cannot~remove~chunk~'#2'~from~hook~'#1'~because~
    \__hook_if_structure_exist:nTF {#1}
      { it~does~not~exist~in~that~hook. }
      { the~hook~does~not~exist. }
  }
\msg_new:nnn { hooks } { generic-deprecated }
  {
    Generic~hook~'#1/#2/#3'~is~deprecated. \\
    Use~hook~'#1/#3/#2'~instead.
  }
```

4.12 LaTeX 2_ε package interface commands

\NewHook
\NewReversedHook
\NewMirroredHookPair

Declaring new hooks ...

```
\NewDocumentCommand \NewHook             { m }
  { \hook_new:n {#1} }
\NewDocumentCommand \NewReversedHook     { m }
  { \hook_new_reversed:n {#1} }
\NewDocumentCommand \NewMirroredHookPair { mm }
  { \hook_new_pair:nn {#1}{#2} }
```

(*End of definition for* `\NewHook`, `\NewReversedHook`, *and* `\NewMirroredHookPair`. *These functions are documented on page 217.*)

\NewHookWithArguments
\NewReversedHookWithArguments
\NewMirroredHookPairWithArguments

Declaring new hooks with arguments...

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\NewHookWithArguments}
⟨latexrelease⟩                  {Hooks~with~args}
\NewDocumentCommand \NewHookWithArguments             { mm }
  { \hook_new_with_args:nn {#1} {#2} }
\NewDocumentCommand \NewReversedHookWithArguments     { mm }
  { \hook_new_reversed_with_args:nn {#1} {#2} }
\NewDocumentCommand \NewMirroredHookPairWithArguments { mmm }
  { \hook_new_pair_with_args:nnn {#1} {#2} {#3} }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\NewHookWithArguments}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_new_protected:Npn \NewHookWithArguments #1 #2 { }
⟨latexrelease⟩\cs_new_protected:Npn \NewReversedHookWithArguments #1 #2 { }
⟨latexrelease⟩\cs_new_protected:Npn \NewMirroredHookPairWithArguments #1 #2 #3{}
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\NewHookWithArguments`, `\NewReversedHookWithArguments`, *and* `\NewMirroredHookPairWithArguments`. *These functions are documented on page 218.*)

```
⟨latexrelease⟩\IncludeInRelease{2021/06/01}{\ActivateGenericHook}
⟨latexrelease⟩                  {Providing~hooks}
```

\ActivateGenericHook

Providing new hooks ...

```
\NewDocumentCommand \ActivateGenericHook { m }
  { \hook_activate_generic:n {#1} }
```

(*End of definition for* `\ActivateGenericHook`. *This function is documented on page 219.*)

\DisableGenericHook

Disabling a generic hook.

```
\NewDocumentCommand \DisableGenericHook { m }
  { \hook_disable_generic:n {#1} }
```

(*End of definition for* `\DisableGenericHook`. *This function is documented on page 218.*)

```
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\ActivateGenericHook}
⟨latexrelease⟩                  {Providing~hooks}
⟨latexrelease⟩\def \ActivateGenericHook #1 { }
⟨latexrelease⟩\def \DisableGenericHook #1 { }
⟨latexrelease⟩\EndIncludeInRelease
```

\AddToHook
\AddToHookWithArguments

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\AddToHookWithArguments}
⟨latexrelease⟩                  {Hooks~with~args}
\NewDocumentCommand \AddToHook { m o +m }
  { \hook_gput_code:nnn {#1} {#2} {#3} }
\NewDocumentCommand \AddToHookWithArguments { m o +m }
  { \hook_gput_code_with_args:nnn {#1} {#2} {#3} }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\AddToHookWithArguments}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_new_protected:Npn \AddToHookWithArguments #1 #2 #3 { }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\AddToHook` *and* `\AddToHookWithArguments`*. These functions are documented on page 220.*)

\AddToHookNext
\AddToHookNextWithArguments

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\AddToHookNextWithArguments}
⟨latexrelease⟩                  {Hooks~with~args}
\NewDocumentCommand \AddToHookNext { m +m }
  { \hook_gput_next_code:nn {#1} {#2} }
\NewDocumentCommand \AddToHookNextWithArguments { m +m }
  { \hook_gput_next_code_with_args:nn {#1} {#2} }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\AddToHookNextWithArguments}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_new_protected:Npn \AddToHookNextWithArguments #1 #2 { }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\AddToHookNext` *and* `\AddToHookNextWithArguments`*. These functions are documented on page 222.*)

\ClearHookNext

```
\NewDocumentCommand \ClearHookNext { m }
  { \hook_gclear_next_code:n {#1} }
```

(*End of definition for* `\ClearHookNext`*. This function is documented on page 222.*)

\RemoveFromHook

```
\NewDocumentCommand \RemoveFromHook { m o }
  { \hook_gremove_code:nn {#1} {#2} }
```

(*End of definition for* `\RemoveFromHook`*. This function is documented on page 221.*)

\SetDefaultHookLabel
\PushDefaultHookLabel
\PopDefaultHookLabel

Now define a wrapper that replaces the top of the stack with the argument, and updates `\g__hook_hook_curr_name_tl` accordingly.

```
\NewDocumentCommand \SetDefaultHookLabel { m }
  { \__hook_set_default_hook_label:n {#1} }
```

The label is only automatically updated with `\@onefilewithoptions` (`\usepackage` and `\documentclass`), but some packages, like Ti*k*Z, define package-like interfaces, like `\usetikzlibrary` that are wrappers around `\input`, so they inherit the default label currently in force (usually `top-level`, but it may change if loaded in another package). To provide a package-like behavior also for hooks in these files, we provide high-level access to the default label stack.

```
\NewDocumentCommand \PushDefaultHookLabel { m }
  { \__hook_curr_name_push:n {#1} }
\NewDocumentCommand \PopDefaultHookLabel { }
  { \__hook_curr_name_pop: }
```

The current label stack holds the labels for all files but the current one (more or less like `\@currnamestack`), and the current label token list, `\g__hook_hook_curr_name_tl`, holds the label for the current file. However `\@pushfilename` happens before `\@currname` is set, so we need to look ahead to get the `\@currname` for the label. expl3 also requires the current file in `\@pushfilename`, so here we abuse `\@expl@push@filename@aux@@` to do `\__hook_curr_name_push:n`.

```
\cs_gset_protected:Npn \@expl@push@filename@aux@@ #1#2#3
  {
    \__hook_curr_name_push:n {#3}
    \str_gset:Ne \g_file_curr_name_str {#3}
    #1 #2 {#3}
  }
```

(*End of definition for* `\SetDefaultHookLabel`, `\PushDefaultHookLabel`, *and* `\PopDefaultHookLabel`. *These functions are documented on page 225.*)

`\UseHook`
`\UseOneTimeHook`
`\UseHookWithArguments`
`\UseOneTimeHookWithArguments`

Avoid the overhead of xparse and its protection that we don't want here (since the hook should vanish without trace if empty)!

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\UseHookWithArguments}
⟨latexrelease⟩                  {Hooks~with~args}
\cs_new:Npn \UseHook         { \hook_use:n }
\cs_new:Npn \UseOneTimeHook { \hook_use_once:n }
\cs_new:Npn \UseHookWithArguments        { \hook_use:nnw }
\cs_new:Npn \UseOneTimeHookWithArguments { \hook_use_once:nnw }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\UseHookWithArguments}
⟨latexrelease⟩                  {Hooks~with~args}
⟨latexrelease⟩\cs_new:Npn \UseHookWithArguments #1 #2 { }
⟨latexrelease⟩\cs_new:Npn \UseOneTimeHookWithArguments #1 #2 { }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\UseHook` *and others. These functions are documented on page 219.*)

`\ShowHook`
`\LogHook`

```
\cs_new_protected:Npn \ShowHook { \hook_show:n }
\cs_new_protected:Npn \LogHook { \hook_log:n }
```

(*End of definition for* `\ShowHook` *and* `\LogHook`. *These functions are documented on page 228.*)

`\DebugHooksOn`
`\DebugHooksOff`

```
\cs_new_protected:Npn \DebugHooksOn  { \hook_debug_on:  }
\cs_new_protected:Npn \DebugHooksOff { \hook_debug_off: }
```

(*End of definition for* `\DebugHooksOn` *and* `\DebugHooksOff`. *These functions are documented on page 229.*)

`\DeclareHookRule`

```
\NewDocumentCommand \DeclareHookRule { m m m m }
                    { \hook_gset_rule:nnnn {#1}{#2}{#3}{#4} }
```

(*End of definition for* `\DeclareHookRule`. *This function is documented on page* 226.)

`\DeclareDefaultHookRule` This declaration is only supported before `\begin{document}`.

```
\NewDocumentCommand \DeclareDefaultHookRule { m m m }
                    { \hook_gset_rule:nnnn {??}{#1}{#2}{#3} }
\@onlypreamble\DeclareDefaultHookRule
```

(*End of definition for* `\DeclareDefaultHookRule`. *This function is documented on page* 227.)

`\ClearHookRule` A special setup rule that removes an existing relation. Basically `\__hook_rule_gclear:nnn` plus fixing the property list for debugging.

FMi: Needs perhaps an L3 interface, or maybe it should get dropped?

```
\NewDocumentCommand \ClearHookRule { m m m }
{ \hook_gset_rule:nnnn {#1}{#2}{unrelated}{#3} }
```

(*End of definition for* `\ClearHookRule`. *This function is documented on page* 226.)

`\IfHookEmptyTF` `\IfHookEmptyT` `\IfHookEmptyF` Here we avoid the overhead of xparse, since `\IfHookEmptyTF` is used in `\end` (that is, every LaTeX environment). As a further optimization, use `\let` rather than `\def` to avoid one expansion step.

```
\cs_new_eq:NN \IfHookEmptyTF \hook_if_empty:nTF
\cs_new_eq:NN \IfHookEmptyT \hook_if_empty:nT
\cs_new_eq:NN \IfHookEmptyF \hook_if_empty:nF
```

(*End of definition for* `\IfHookEmptyTF`, `\IfHookEmptyT`, *and* `\IfHookEmptyF`. *These functions are documented on page* 228.)

`\IfHookExistsTF` Marked for removal and no longer documented in the doc section!

PhO: `\IfHookExistsTF` *is used in* `jlreq.cls`, `pxatbegshi.sty`, `pxeverysel.sty`, `pxeveryshi.sty`, *so the public name may be an alias of the internal conditional for a while. Regardless, those packages' use for* `\IfHookExistsTF` *is not really correct and can be changed.*

```
\cs_new_eq:NN \IfHookExistsTF \__hook_if_usable:nTF
```

(*End of definition for* `\IfHookExistsTF`.)

4.13 Deprecated that needs cleanup at some point

`\hook_disable:n` `\hook_provide:n` `\hook_provide_reversed:n` `\hook_provide_pair:nn` `\__hook_activate_generic_reversed:n` `\__hook_activate_generic_pair:nn` Deprecated.

```
\cs_new_protected:Npn \hook_disable:n
  {
    \__hook_deprecated_warn:nn
      { hook_disable:n }
      { hook_disable_generic:n }
    \hook_disable_generic:n
  }
\cs_new_protected:Npn \hook_provide:n
  {
    \__hook_deprecated_warn:nn
      { hook_provide:n }
      { hook_activate_generic:n }
    \hook_activate_generic:n
  }
\cs_new_protected:Npn \hook_provide_reversed:n
```

```
  {
    \__hook_deprecated_warn:nn
      { hook_provide_reversed:n }
      { hook_activate_generic:n }
    \__hook_activate_generic_reversed:n
  }
\cs_new_protected:Npn \hook_provide_pair:nn
  {
    \__hook_deprecated_warn:nn
      { hook_provide_pair:nn }
      { hook_activate_generic:n }
    \__hook_activate_generic_pair:nn
  }
\cs_new_protected:Npn \__hook_activate_generic_reversed:n #1
  { \__hook_normalize_hook_args:Nn \__hook_activate_generic:nn {#1} { - } }
\cs_new_protected:Npn \__hook_activate_generic_pair:nn #1#2
  { \hook_activate_generic:n {#1} \__hook_activate_generic_reversed:n {#2} }
```

(*End of definition for* `\hook_disable:n` *and others.*)

`\DisableHook` `\ProvideHook` `\ProvideReversedHook` `\ProvideMirroredHookPair`

Deprecated.

```
\cs_new_protected:Npn \DisableHook
  {
    \__hook_deprecated_warn:nn
      { DisableHook }
      { DisableGenericHook }
    \hook_disable_generic:n
  }
\cs_new_protected:Npn \ProvideHook
  {
    \__hook_deprecated_warn:nn
      { ProvideHook }
      { ActivateGenericHook }
    \hook_activate_generic:n
  }
\cs_new_protected:Npn \ProvideReversedHook
  {
    \__hook_deprecated_warn:nn
      { ProvideReversedHook }
      { ActivateGenericHook }
    \__hook_activate_generic_reversed:n
  }
\cs_new_protected:Npn \ProvideMirroredHookPair
  {
    \__hook_deprecated_warn:nn
      { ProvideMirroredHookPair }
      { ActivateGenericHook }
    \__hook_activate_generic_pair:nn
  }
```

(*End of definition for* `\DisableHook` *and others.*)

`\__hook_deprecated_warn:nn` Warns about a deprecation, telling what should be used instead.

```
\cs_new_protected:Npn \__hook_deprecated_warn:nn #1 #2
  { \msg_warning:nnnn { hooks } { deprecated } {#1} {#2} }
```

```
\msg_new:nnn { hooks } { deprecated }
  {
    Command~\iow_char:N\\#1~is~deprecated~and~will~be~removed~in~a~
    future~release. \\ \\
    Use~\iow_char:N\\#2~instead.
  }
```

(*End of definition for* `\__hook_deprecated_warn:nn`.)

4.14 Internal commands needed elsewhere

Here we set up a few horrible (but consistent) LaTeX 2_ε names to allow for internal commands to be used outside this module. We have to unset the `@@` since we want double "at" sign in place of double underscores.

```
⟨@@=⟩
```

`\@expl@@@initialize@all@@`
`\@expl@@@hook@curr@name@pop@@`

```
\cs_new_eq:NN \@expl@@@initialize@all@@
              \__hook_initialize_all:
\cs_new_eq:NN \@expl@@@hook@curr@name@pop@@
              \__hook_curr_name_pop:
```

(*End of definition for* `\@expl@@@initialize@all@@` *and* `\@expl@@@hook@curr@name@pop@@`.)

Rolling back here doesn't undefine the interface commands as they may be used in packages without rollback functionality. So we just make them do nothing which may or may not work depending on the code usage.

```
%
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{lthooks}
⟨latexrelease⟩                  {The~hook~management}%
⟨latexrelease⟩
⟨latexrelease⟩\def \NewHook#1{}
⟨latexrelease⟩\def \NewReversedHook#1{}
⟨latexrelease⟩\def \NewMirroredHookPair#1#2{}
⟨latexrelease⟩
⟨latexrelease⟩\def \DisableGenericHook #1{}
⟨latexrelease⟩
⟨latexrelease⟩\long\def\AddToHookNext#1#2{}
⟨latexrelease⟩
⟨latexrelease⟩\def\AddToHook#1{\@gobble@AddToHook@args}
⟨latexrelease⟩\providecommand\@gobble@AddToHook@args[2][]{}
⟨latexrelease⟩
⟨latexrelease⟩\def\RemoveFromHook#1{\@gobble@RemoveFromHook@arg}
⟨latexrelease⟩\providecommand\@gobble@RemoveFromHook@arg[1][]{}
⟨latexrelease⟩
⟨latexrelease⟩\def \UseHook        #1{}
⟨latexrelease⟩\def \UseOneTimeHook #1{}
⟨latexrelease⟩\def \ShowHook #1{}
⟨latexrelease⟩\let \DebugHooksOn \@empty
⟨latexrelease⟩\let \DebugHooksOff\@empty
⟨latexrelease⟩
⟨latexrelease⟩\def \DeclareHookRule #1#2#3#4{}
⟨latexrelease⟩\def \DeclareDefaultHookRule #1#2#3{}
⟨latexrelease⟩\def \ClearHookRule #1#2#3{}
```

If the hook management is not provided we make the test for existence false and the test for empty true in the hope that this is most of the time reasonable. If not a package would need to guard against running in an old kernel.

```
⟨latexrelease⟩\long\def \IfHookExistsTF #1#2#3{#3}
⟨latexrelease⟩\long\def \IfHookEmptyTF #1#2#3{#2}
⟨latexrelease⟩
⟨latexrelease⟩\EndModuleRelease
```

⟨@@=hook⟩

```
⟨latexrelease⟩\cs:w __hook_rollback_tidying: \cs_end:
⟨latexrelease⟩\bool_lazy_and:nnT
⟨latexrelease⟩    { \int_compare_p:nNn { \sourceLaTeXdate } > { 20230600 } }
⟨latexrelease⟩    { \int_compare_p:nNn { \requestedLaTeXdate } < { 20230601 } }
⟨latexrelease⟩  {
⟨latexrelease⟩    \cs_gset_protected:Npn \__hook_rollback_tidying:
⟨latexrelease⟩      {
⟨latexrelease⟩        \@latex@error { Rollback~code~executed~twice }
⟨latexrelease⟩          {
⟨latexrelease⟩            Something~went~wrong~(unless~this~was~
⟨latexrelease⟩            done~on~purpose~in~a~testing~environment).
⟨latexrelease⟩          }
⟨latexrelease⟩        \use_none:nnnn
⟨latexrelease⟩      }
⟨latexrelease⟩    \cs_set:Npn \__hook_tmp:w #1 #2
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_tl_gset:ce { __hook#1~#2 }
⟨latexrelease⟩          {
⟨latexrelease⟩            \exp_args:No \exp_not:o
⟨latexrelease⟩              {
⟨latexrelease⟩                \cs:w __hook#1~#2 \exp_last_unbraced:Ne \cs_end:
⟨latexrelease⟩                  { \__hook_braced_cs_parameter:n
⟨latexrelease⟩                      { __hook#1~#2 } }
⟨latexrelease⟩              }
⟨latexrelease⟩          }
⟨latexrelease⟩      }
⟨latexrelease⟩    \seq_map_inline:Nn \g__hook_all_seq
⟨latexrelease⟩      {
⟨latexrelease⟩        \exp_after:wN \cs_gset_nopar:Npn
⟨latexrelease⟩          \cs:w g__hook_#1_code_prop \exp_args:NNo \exp_args:No
⟨latexrelease⟩            \cs_end: { \cs:w g__hook_#1_code_prop \cs_end: }
⟨latexrelease⟩        \__hook_tmp:w { _toplevel } {#1}
⟨latexrelease⟩        \__hook_tmp:w { _next } {#1}
⟨latexrelease⟩      }
⟨latexrelease⟩  }
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
```

⟨@@=⟩

File 09
ltcmdhooks.dtx

1 Introduction

This file implements generic hooks for (arbitrary) commands. In theory every command \⟨*name*⟩ offers now two associated hooks to which code can be added using `\AddToHook`,[13] `\AddToHookNext`, `\AddToHookWithArguments`, and `\AddToHookNextWithArguments`.[14]

However, this is only true "in theory". In practice there are a number of restrictions that makes it impossible to use such generic command hooks in a number of cases, so please read all of section 2 to understand what may prevent you from using them successfully.

The generic command hooks are:

`cmd/`⟨*name*⟩`/before` This hook is executed at the very start of the command, right after its arguments (if any) are parsed. The hook ⟨*code*⟩ runs in the command inside a call to `\UseHookWithArguments`. Any code added to this hook using `\AddToHookWithArguments` or `\AddToHookNextWithArguments` can access the command's arguments using `#1`, `#2`, etc., up to the number of arguments of the command. If `\AddToHook` or `\AddToHookNext` are used, the arguments cannot be accessed (see the lthooks documentation[15] on hooks with arguments).

`cmd/`⟨*name*⟩`/after` This hook is similar to `cmd/`⟨*name*⟩`/before`, but it is executed at the very end of the command body. This hook is implemented as a reversed hook.

The hooks are not physically present before `\begin{document}`[16] (i.e., using a command in the preamble will never execute the hook) and if nobody has declared any code for them, then they are not added to the command code ever. For example, if we have the following definition

```
\newcommand\foo[2]{Code #1 for #2!}
```

then executing `\foo{A}{B}` will simply run `Code␣A␣for␣B!` as it was always the case. However, if somebody, somewhere (e.g., in a package) adds

```
\AddToHook{cmd/foo/before}{<before code>}
```

then, after `\begin{document}` the definition of `\foo` will be:

```
\renewcommand\foo[2]{%
  \UseHookWithArguments{cmd/foo/before}{2}{#1}{#2}%
  Code #1 for #2!}
```

and similarly `\AddToHook{cmd/foo/after}{<after code>}` alters the definition to

[13] In this documentation, when something is being said about `\AddToHook`, the same will be valid for `\AddToHookWithArguments`, unless that particular paragraph is highlighting the differences between both. The same is true for the other hook-related functions and their `...WithArguments` counterparts.

[14] In practice this is not supported for all types of commands, see section 2.4 for the restrictions that apply and what happens if one tries to use this with commands for which this is not supported.

[15] `texdoc lthooks-doc`

[16] More specifically, they are inserted in the commands after the `begindocument` hook, so they are also not present while LaTeX is reading the `.aux` file.

```
\renewcommand\foo[2]{%
  Code #1 for #2!%
  \UseHookWithArguments{cmd/foo/after}{2}{#1}{#2}}
```

In other words, the mechanism is similar to what etoolbox offers with `\pretocmd` and `\apptocmd` with the important differences

- that code can be prepended or appended (i.e., added to the hooks) even if the command itself is not (yet) defined, because the defining package has not been loaded at this point;
- and that by using the hook management interface it is now possible to define how the code chunks added in these places are ordered, if different packages want to add code at these points.

2 Restrictions and operational details

Adding arbitrary material to commands is tricky because most of the time we do not know what the macro expects as arguments when expanding and TEX doesn't have a reliable way to see that, so some guesswork has to be employed.

We can do this in most cases when commands are defined using `\NewDocumentCommand` or `\newcommand` (with a few exceptions). For commands defined with `\def` the situation is less good. Common cases where the command hooks will not work are:

- Commands that use special catcode settings within their definition. In that case it is usually not possible to augment the definition (see 2.1).
- If a command is defined while `\ExplSyntaxOn` is in force **and** the command contains `~` characters to represent spaces, then it can't be patched to include the command hooks. In fact in some very special circumstances you might even get a low-level error rather than the information that the command can't be patched (see, for example, https://github.com/latex3/latex2e/issues/1430.
- Commands that have arguments as far as the user is concerned (e.g., `\section` or `\caption`), but are defined in a way that these arguments are not read by the user level command but only later during the processing. In that case the `after` hook doesn't work at all. The before hook only works with `\AddToHook` but not with `\AddToHookWithArguments` because the arguments haven't been read at that point where the hook is patched in. See section 2.4.
- Adding a specific generic command hook is only attempted once per command, thus after redefining a command such hooks will no longer be there and will also not being re-added, see section 2.1.1.

All this means that you have to have a good understanding of how commands are defined when you attempt to make use of such hooks and something goes wrong. What can help in that case is to turn on `\DebugHooksOn` in which case you get much more (low-level) details on why something fails and what was tried to enable the hooks.

2.1 Patching

The code here tries to find out if a command was defined with `\newcommand` or `\DeclareRobustCommand` or `\NewDocumentCommand`, and if so it *assumes* that the argument specification of the command is as expected (which is not fail-proof, if someone redefines the internals of these commands in devious ways, but is a reasonable assumption).

If the command is one of the defined types, the code here does a sandboxed expansion of the command such that it can be redefined again exactly as before, but with the hook code added.

If however the command is not a known type (it was defined with `\def`, for example), then the code uses an approach similar to etoolbox's `\patchcmd` to retokenize the command with the hook code in place. This procedure, however, is more likely to fail if the catcode settings are not the same as the ones at the time of command's definition, so not always adding a hook to a command will work.

2.1.1 Timing

When `\AddToHook` (or its expl3 equivalent) is called with a generic `cmd` hook, say, `cmd/foo/before`, for the first time (that is, no code was added to that same hook before), in the preamble of a document, it will store a patch instruction for that command until `\begin{document}`, and only then all the commands which had hooks added will be patched in one go. That means that no command in the preamble will have hooks patched into them.

At `\begin{document}` all the delayed patches will be executed, and if the command doesn't exist the code is still added to the hook, but it will not be executed. After `\begin{document}`, when `\AddToHook` is called with a generic `cmd` hook the first time, the command will be immediately patched to include the hook, and if it doesn't exist or if it can't be patched for any reason, an error is thrown; if `\AddToHook` was already used in the preamble no new patching is attempted.

This has the consequence that a command defined or redefined after `\begin{document}` only uses generic `cmd` hook code if `\AddToHook` is called for the first time after the definition is made, or if the command explicitly uses the generic hook in its definition by declaring it with `\NewHookPair` adding `\UseHook` as part of the code.[17]

2.2 Command copies

Once a hook has been added to a command, it will be present in any copies of that command which are made. For example, if in the preamble we have

```
\NewDocumentCommand\foo{}{}
\AddToHook{cmd/foo/after}{1}%
```

then after `\begin{document}`, any use of `\NewCommandCopy` will include the hook in the copy, for example

```
\NewCommandCopy\baz\foo
```

would include the hook in this copied command.

[17] We might change this behavior in the main document slightly after gaining some usage experience.

2.3 Grouping

Command hooks are intended to be added to document commands, which are typically defined globally. As such, adding a hook is a global operation, even if the command was previously only locally-defined. Addition of material to hooks is also global.

2.4 Commands that look ahead

Some commands are defined in different "steps" and they look ahead in the input stream to find more arguments. If you try to add some code to the `cmd/`⟨*name*⟩`/after` hook of such command, it will not work, and it is not possible to detect that programmatically, so the user has to know (or find out) which commands can or cannot have hooks attached to them.

One good example is the `\section` command. You can add something to the `cmd/section/before` hook (but only with `\AddToHook` not `\AddToHookWithArguments`), but if you try to add anything to the `cmd/section/after` hook, `\section` will no longer work at all. That happens because the `\section` macro takes no argument, but instead calls a few internal LaTeX macros to look for the optional and mandatory arguments. By adding code to the `cmd/section/after` hook, you get in the way of that scanning.

In such a case, where it is known that a specific generic command hook does not work if code is added to it, the package author can add a `\DisableGenericHook`[18] declaration to prevent this from happening in user documents and thereby avoiding obscure errors.

3 Package author interface

The `cmd` hooks are, by default, available for all commands that can be patched to add the hooks. For some commands, however, the very beginning or the very end of the code is not the best place to put the hooks, for example, if the command looks ahead for arguments (see section 2.4).

If you are a package author and you want to add the hooks to your own commands in the proper position you can define the command and manually add the `\UseHookWithArguments` calls inside the command in the proper positions, and manually define the hooks with `\NewHookWithArguments` or `\NewReversedHookWithArguments`. When the hooks are explicitly defined, patching is not attempted so you can make sure your command works properly. For example, an (admittedly not really useful) command that typesets its contents in a framed box with width optionally given in parentheses:

```
\newcommand\fancybox{\@ifnextchar({\@fancybox}{\@fancybox(5cm)}}
\def\@fancybox(#1)#2{\fbox{\parbox{#1}{#2}}}
```

If you try that definition, then add some code after it with

```
\AddToHook{cmd/fancybox/after}{<code>}
```

and then use the `\fancybox` command you will see that it will be completely broken, because the hook will get executed in the middle of parsing for optional `(...)` argument.

If, on the other hand, you want to add hooks to your command you can do something like:

[18] Please use `\DisableGenericHook` if at all, only on hooks that you "own", i.e., for commands your package or class defines and not second guess whether or not hooks of other packages should get disabled!

```
\newcommand\fancybox{\@ifnextchar({\@fancybox}{\@fancybox(5cm)}}
\def\@fancybox(#1)#2{\fbox{%
              \UseHookWithArguments{cmd/fancybox/before}{2}{#1}{#2}%
              \parbox{#1}{#2}%
              \UseHookWithArguments{cmd/fancybox/after}{2}{#1}{#2}}}
\NewHookWithArguments{cmd/fancybox/before}{2}
\NewReversedHookWithArguments{cmd/fancybox/after}{2}
```

then the hooks will be executed where they should and no patching will be attempted. It is important that the hooks are declared with `\NewHookWithArguments` or `\NewReversedHookWithArguments`, otherwise the command hook code will try to patch the command. Note also that the call to `\UseHookWithArguments{cmd/fancybox/before}` does not need to be in the definition of `\fancybox`, but anywhere it makes sense to insert it (in this case in the internal `\@fancybox`).

Alternatively, if for whatever reason your command does not support the generic hooks provided here, you can disable a hook with `\DisableGenericHook`[19], so that when someone tries to add code to it they will get an error. Or if you don't want the error, you can simply declare the hook with `\NewHook` and never use it.

The above approach is useful for really complex commands where for one or the other reason the hooks can't be placed at the very beginning and end of the command body and some hand-crafting is needed. However, in the example above the real (and in fact only) issue is the cascading argument parsing in the style developed long ago in LaTeX 2.09. Thus, a much simpler solution for this case is to replace it with the modern `\NewDocumentCommand` syntax and define the command as follows:

```
\DeclareDocumentCommand\fancybox{D(){5cm}m}{\fbox{\parbox{#1}{#2}}}
```

If you do that then both hooks automatically work and are patched into the right places.

3.1 Arguments and redefining commands

The code in ltcmdhooks does its best to find out how many arguments a given command has, and to insert the appropriate call to `\UseHookWithArguments`, so that the arguments seen by the hook are exactly those grabbed by the command (the hook, after all, is a macro call, so the arguments have to be placed in the right order, or they won't match).

When using the package writer interface, as discussed in section 3, to change the position of the hooks in your commands, you are also free to change how the hook code in your command sees its arguments. When a `cmd` hook is declared with `\NewHook` (or `\NewHookWithArguments` or other variations of that), it loses its "generic" nature and works as a regular hook. This means that you may choose to declare it without arguments regardless if the command takes arguments or not, or declare it with arguments, even if the command takes none.

However, this flexibility should not be abused. When using a nonstandard configuration for the hook arguments, think reasonably: a user will expect that the argument `#1` in the hook corresponds to the argument's first argument, and so on. Any other configuration is likely to cause confusion and, if used, will have to be well documented.

This flexibility, however, allows you to "correct" the arguments for the hooks. For example, LaTeX's `\refstepcounter` has a single argument, the name of the counter. The cleveref package adds an optional argument to `\refstepcounter`, making the name of

[19] Please use `\DisableGenericHook` if at all, only on hooks that you "own", i.e., for commands your package or class defines and not second guess whether or not hooks of other packages should get disabled!

the counter argument #2. If the author of cleveref wanted, for whatever reason, to add hooks to \refstepcounter, to preserve compatibility he could write something along the lines of:

```
\NewHookWithArguments{cmd/refstepcounter/before}{1}
\renewcommand\refstepcounter[2][<default>]{%
  \UseHookWithArguments{cmd/refstepcounter/before}{1}{#2}%
  <code for \refstepcounter>}
```

so that the mandatory argument, which is arg #2 in the definition, would still be seen as #1 in the hook code.

Another possibility would be to place the optional argument as the second argument for the hook, so that people looking for it would be able to use it. In either case, it would have to be well documented to cause as little confusion as possible.

4 The Implementation

4.1 Execution plan

To add before and after hooks to a command we will need to peek into the definition of a command, which is always a tricky thing to do. Some cases are easy because we know how the command was defined, so we can assume how its ⟨*parameter text*⟩ looks like (for example a command defined with \newcommand may have an optional argument followed by a run of mandatory arguments), so we can just expand that command and make it grab #1, #2, etc. as arguments and define it all back with the hooks added.

Life's usually not that easy, so with some commands we can't do that (a #1 might as well be $\#_{12}1_{12}$ instead of the expected $\#_6 1_{12}$, for example) so we need to resort to "patching" the command: read its \meaning, and tokenize it again with \scantokens and hope for the best.

So the overall plan is:

1. Check if a command is of a known type (that is, defined with \newcommand[20], \DeclareRobustCommand, or \New(Expandable)DocumentCommand), and if is, take appropriate action.

2. If the command is not a known type, we'll check if the command can be patched. Two things will prevent a command from being patched: if it was defined in a nonstandard catcode setting, or if it is an internal expl3 command with __⟨*module*⟩ in its name, in which case we refuse to patch.

3. If the command was defined in nonstandard catcode settings, we will try a few standard ones to try our best to carry out the pathing. If this doesn't help either, the code will give up and throw an error.

```
⟨@@=hook⟩

⟨*2ekernel | latexrelease⟩
\ExplSyntaxOn
⟨latexrelease⟩\NewModuleRelease{2021/06/01}{ltcmdhooks}
⟨latexrelease⟩                        {The~hook~management~system~for~commands}
```

[20] It's not always possible to reliably detect this case because a command defined with no optional argument is indistinguishable from a \defed command.

4.2 Variables

\g_hook_patch_action_list_tl

Pairs of \if<cmd>..\patch<cmd> to be used with \robust@command@act when looking for a known patching rule. This token list is exposed because we see some future applications (with very specialized packages, such as etoolbox that may want to extend the pairs processed. It is not meant for general use which is why it is not documented in the interface documentation above.

```
\tl_new:N \g_hook_patch_action_list_tl
```

(*End of definition for* \g_hook_patch_action_list_tl.)

\l__hook_patch_num_args_int

The number of arguments in a macro being patched.

```
\int_new:N \l__hook_patch_num_args_int
```

(*End of definition for* \l__hook_patch_num_args_int.)

\l__hook_patch_prefixes_tl
\l__hook_param_text_tl
\l__hook_replace_text_tl

The prefixes and parameters of the definition for the macro being patched.

```
\tl_new:N \l__hook_patch_prefixes_tl
\tl_new:N \l__hook_param_text_tl
\tl_new:N \l__hook_replace_text_tl
```

(*End of definition for* \l__hook_patch_prefixes_tl, \l__hook_param_text_tl, *and* \l__hook_replace_text_tl.)

\c__hook_hash_tl
\c__hook_hashes_tl

Two constant token lists that contain one and two parameter tokens.

```
\tl_const:Nn \c__hook_hash_tl { # }
\tl_const:Nn \c__hook_hashes_tl { # # }
```

(*End of definition for* \c__hook_hash_tl *and* \c__hook_hashes_tl.)

__hook_exp_not:NN
__hook_def_cmd:w

Two temporary macros that change depending on the macro being patched.

```
\cs_new_eq:NN \__hook_exp_not:NN ?
\cs_new_eq:NN \__hook_def_cmd:w ?
```

(*End of definition for* __hook_exp_not:NN *and* __hook_def_cmd:w.)

\q__hook_recursion_tail
\q__hook_recursion_stop

Internal quarks for recursion: they can't appear in any macro being patched.

```
\quark_new:N \q__hook_recursion_tail
\quark_new:N \q__hook_recursion_stop
```

(*End of definition for* \q__hook_recursion_tail *and* \q__hook_recursion_stop.)

\g__hook_delayed_patches_prop

A list containing the patches delayed to \begin{document}, so that patching is not attempted twice.

```
\prop_new:N \g__hook_delayed_patches_prop
```

(*End of definition for* \g__hook_delayed_patches_prop.)

__hook_patch_debug:e

A helper for patching debug info.

```
\cs_new_protected:Npn \__hook_patch_debug:e #1
  { \__hook_debug:n { \iow_term:e { [lthooks]~#1 } } }
```

(*End of definition for* __hook_patch_debug:e.)

4.3 Patching or delaying

Before `\begin{document}` all patching is delayed.

`\__hook_try_put_cmd_hook:n`
`\__hook_try_put_cmd_hook:w`

This function is called from within `\AddToHook`, when code is first added to a generic `cmd` hook. If it is called within in the preamble, it delays the action until `\begin{document}`; otherwise it tries to update the hook.

```
⟨latexrelease⟩\IncludeInRelease{2024/12/22}{\__hook_try_put_cmd_hook:n}%
⟨latexrelease⟩                  {Don't~define~command}
\cs_new_protected:Npn \__hook_try_put_cmd_hook:n #1
  { \__hook_try_put_cmd_hook:w #1 / / / \s__hook_mark {#1} }
\cs_new_protected:Npn \__hook_try_put_cmd_hook:w
    #1 / #2 / #3 / #4 \s__hook_mark #5
  {
    \__hook_debug:n { \iow_term:n { ->~Adding~cmd~hook~to~'#2'~(#3): } }
```

`\__hook_patch_cmd_or_delay:Nnn` expects the command to be patched as its first argument so we need to construct it from its name (`#2`). However, at this moment it may not exist yet, so using `\cs:w` would incorrectly turn it from "undefined" into `\relax`. We therefore use the following curious construction: we start a group and expand out of it to call `\cs:w`. If the command is now changed to `\relax` the `\group_end:` will undo that change, but the token is nevertheless there to be consumed by `\__hook_patch_cmd_or_delay:Nnn`.

```
    \group_begin:
       \exp_after:wN
    \group_end:
       \exp_after:wN
    \__hook_patch_cmd_or_delay:Nnn
       \cs:w #2\cs_end:
       {#2} {#3}
  }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2021/11/15}{\__hook_try_put_cmd_hook:n}%
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\cs_new_protected:Npn \__hook_try_put_cmd_hook:n #1
⟨latexrelease⟩  { \__hook_try_put_cmd_hook:w #1 / / / \s__hook_mark {#1} }
⟨latexrelease⟩\cs_new_protected:Npn \__hook_try_put_cmd_hook:w
⟨latexrelease⟩    #1 / #2 / #3 / #4 \s__hook_mark #5
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_debug:n { \iow_term:n { ->~Adding~cmd~hook~to~'#2'~(#3): } }
⟨latexrelease⟩    \exp_args:Nc \__hook_patch_cmd_or_delay:Nnn {#2} {#2} {#3}
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2021/06/01}{\__hook_try_put_cmd_hook:n}%
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\cs_new_protected:Npn \__hook_try_put_cmd_hook:n #1
⟨latexrelease⟩  { \__hook_try_put_cmd_hook:w #1 / / / \s__hook_mark {#1} }
⟨latexrelease⟩\cs_new_protected:Npn \__hook_try_put_cmd_hook:w
⟨latexrelease⟩    #1 / #2 / #3 / #4 \s__hook_mark #5
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_debug:n { \iow_term:n { ->~Adding~cmd~hook~to~'#2'~(#3): } }
⟨latexrelease⟩    \str_case:nnTF {#3}
⟨latexrelease⟩        { { before } { } { after } { } }
```

```
⟨latexrelease⟩       { \exp_args:Nc \__hook_patch_cmd_or_delay:Nnn {#2} {#2} {#3} }
⟨latexrelease⟩       { \msg_error:nnnn { hooks } { wrong-cmd-hook } {#2} {#3} }
⟨latexrelease⟩   }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_try_put_cmd_hook:n` *and* `\__hook_try_put_cmd_hook:w`.)

`\__hook_patch_cmd_or_delay:Nnn`
`\__hook_cmd_begindocument_code:`

In the preamble, `\__hook_patch_cmd_or_delay:Nnn` just adds the patch instruction to a property list to be executed later.

```
\cs_new_protected:Npn \__hook_patch_cmd_or_delay:Nnn #1 #2 #3
  {
    \__hook_debug:n { \iow_term:n { ->~Add~generic~cmd~hook~for~#2~(#3). } }
    \__hook_debug:n
      { \iow_term:n { !~In~the~preamble:~delaying. } }
    \prop_gput:Nnn \g__hook_delayed_patches_prop { #2 / #3 }
      { \__hook_cmd_try_patch:nn {#2} {#3} }
  }
```

The delayed patches are added to a property list to prevent duplication, and the code stored in the property list for each key is executed. The function `\__hook_patch_cmd_or_delay:Nnn` is also redefined to be `\__hook_patch_command:Nnn` so that no further delaying is attempted.

```
\cs_new_protected:Npn \__hook_cmd_begindocument_code:
  {
    \cs_gset_eq:NN \__hook_patch_cmd_or_delay:Nnn \__hook_patch_command:Nnn
    \prop_map_function:NN \g__hook_delayed_patches_prop { \use_ii:nn }
    \prop_gclear:N \g__hook_delayed_patches_prop
    \cs_undefine:N \__hook_cmd_begindocument_code:
  }
\g@addto@macro \@kernel@after@begindocument
  { \__hook_cmd_begindocument_code: }
```

(*End of definition for* `\__hook_patch_cmd_or_delay:Nnn` *and* `\__hook_cmd_begindocument_code:`.)

`\__hook_cmd_try_patch:nn`

At `\begin{document}` tries patching the command if the hook was not manually created in the meantime. If the document does not exist, no error is raised here as it may hook into a package that wasn't loaded. Hooks added to commands in the document body still raise an error if the command is not defined.

```
\cs_new_protected:Npn \__hook_cmd_try_patch:nn #1 #2
  {
    \__hook_debug:n
      { \iow_term:e { ->~\string\begin{document}~try~cmd / #1 / #2. } }
    \__hook_if_declared:nTF { cmd / #1 / #2 }
      {
        \__hook_debug:n
          { \iow_term:n { .->~Giving~up:~hook~already~created. } }
      }
      {
        \cs_if_exist:cT {#1}
          { \exp_args:Nc \__hook_patch_command:Nnn {#1} {#1} {#2} }
      }
  }
```

(*End of definition for* `\__hook_cmd_try_patch:nn`.)

4.4 Patching commands

__hook_patch_command:Nnn
__hook_patch_check:NNnn
__hook_if_public_command:NTF
__hook_if_public_command:w

`\__hook_patch_command:Nnn` will do some sanity checks on the argument to detect if it is possible to add hooks to the command, and raises an error otherwise. If the command can contain hooks, then it uses `\robust@command@act` to find out what type is the command, and patch it accordingly.

```
\cs_new_protected:Npn \__hook_patch_command:Nnn #1 #2 #3
  {
    \__hook_patch_debug:e { analyzing~'\token_to_str:N #1' }
    \__hook_patch_debug:e { \token_to_str:N #1 = \token_to_meaning:N #1 }
    \__hook_patch_check:NNnn \cs_if_exist:NTF #1 { undef }
      {
        \__hook_patch_debug:e { ++~control~sequence~is~defined }
        \__hook_patch_check:NNnn \token_if_macro:NTF #1 { macro }
          {
            \__hook_patch_debug:e { ++~control~sequence~is~a~macro }
            \__hook_patch_check:NNnn \__hook_if_public_command:NTF #1 { expl3 }
              {
                \__hook_patch_debug:e { ++~macro~is~not~private }
                \robust@command@act
                  \g_hook_patch_action_list_tl #1
                  \__hook_retokenize_patch:Nnn { #1 {#2} {#3} }
              }
          }
      }
  }
```

And here's the auxiliary used above:

```
\cs_new_protected:Npn \__hook_patch_check:NNnn #1 #2 #3 #4
  {
    #1 #2 {#4}
      {
        \msg_error:nnee { hooks } { cant-patch }
          { \token_to_str:N #2 } {#3}
      }
  }
```

and a conditional `\__hook_if_public_command:NTF` to check if a command has `__` in its name (no other checking is performed). Primitives with `:D` in their name could be included here, but they are already discarded in the `\token_if_macro:NTF` test above.

```
\use:e
  {
    \prg_new_protected_conditional:Npnn
        \exp_not:N \__hook_if_public_command:N #1 { TF }
      {
        \exp_not:N \exp_last_unbraced:Nf
          \exp_not:N \__hook_if_public_command:w
            { \exp_not:N \cs_to_str:N #1 }
          \tl_to_str:n { _ _ } \s__hook_mark
      }
  }
\exp_last_unbraced:NNNNo
\cs_new_protected:Npn \__hook_if_public_command:w
    #1 \tl_to_str:n { _ _ } #2 \s__hook_mark
```

```
  {
    \tl_if_empty:nTF {#2}
      { \prg_return_true: }
      { \prg_return_false: }
  }
```

(*End of definition for* `\__hook_patch_command:Nnn` *and others.*)

4.4.1 Patching by expansion and redefinition

`\g_hook_patch_action_list_tl`

This is the list of known command types and the function that patches the command hooks into them. The conditionals are taken from `\ShowCommand`, `\NewCommandCopy` and `\__kernel_cmd_if_xparse:NTF` defined in `ltcmd`.

```
\tl_gset:Nn \g_hook_patch_action_list_tl
  {
    { \@if@DeclareRobustCommand \__hook_patch_DeclareRobustCommand:Nnn }
    { \@if@newcommand \__hook_patch_newcommand:Nnn }
    { \__kernel_cmd_if_xparse:NTF \__hook_cmd_patch_xparse:Nnn }
  }
```

(*End of definition for* `\g_hook_patch_action_list_tl`.)

`\__hook_patch_DeclareRobustCommand:Nnn`

At this point we know that the commands can be patched by expanding then redefining. These are the cases of commands defined with `\newcommand` with an optional argument or with `\DeclareRobustCommand`.

With `\__hook_patch_DeclareRobustCommand:Nnn` we check if the command has an optional argument (with a test counter-intuitively called `\@if@newcommand`; also make sure the command doesn't take args by calling `\robust@command@chk@safe`). If so, we pass the patching action to `\__hook_patch_newcommand:Nnn`, otherwise we call the patching engine `\__hook_patch_expand_redefine:NNnn` with a `\c_false_bool` to indicate that there is no optional argument.

```
\cs_new_protected:Npn \__hook_patch_DeclareRobustCommand:Nnn #1
  {
    \exp_args:Nc \__hook_patch_DeclareRobustCommand_aux:Nnn
      { \cs_to_str:N #1 ~ }
  }
\cs_new_protected:Npn \__hook_patch_DeclareRobustCommand_aux:Nnn #1
  {
    \robust@command@chk@safe #1
      { \@if@newcommand #1 }
      { \use_ii:nn }
        { \__hook_patch_newcommand:Nnn }
        { \__hook_patch_expand_redefine:NNnn \c_false_bool }
          #1
  }
```

(*End of definition for* `\__hook_patch_DeclareRobustCommand:Nnn`.)

`\__hook_patch_newcommand:Nnn`

If the command was defined with `\newcommand` and an optional argument, call the patching engine with a `\c_true_bool` to flag the presence of an optional argument, and with `\\command` to patch the actual code for `\command`.

```
\cs_new_protected:Npn \__hook_patch_newcommand:Nnn #1
  {
```

```
    \exp_args:NNc \__hook_patch_expand_redefine:NNnn \c_true_bool
      { \c_backslash_str \cs_to_str:N #1 }
  }
```

(*End of definition for* `\__hook_patch_newcommand:Nnn`.)

`\__hook_cmd_patch_xparse:Nnn`

And for commands defined by the xparse commands use this for patching:

```
\cs_new_protected:Npn \__hook_cmd_patch_xparse:Nnn #1
  {
    \exp_args:NNc \__hook_patch_expand_redefine:NNnn \c_false_bool
      { \cs_to_str:N #1 ~ code }
  }
```

(*End of definition for* `\__hook_cmd_patch_xparse:Nnn`.)

`\__hook_patch_expand_redefine:NNnn`
`\__hook_redefine_with_hooks:Nnnn`
`\__hook_make_prefixes:w`

Now the real action begins. Here we have in `#1` a boolean indicating if the command has a leading `[...]`-delimited argument, in `#2` the command control sequence, in `#3` the name of the command (note that `#1` $\neq$ `\csname#2\endcsname` at this point!), and in `#4` the hook position, either `before` or `after`.

Patching with expansion+redefinition is trickier than it looks like at first glance. Suppose the simple definition:

```
\def\foo#1{#1##2}
```

When defined, its ⟨*replacement text*⟩ will be a token list containing:

out_param `1`*, mac_param* `#`*, character* `2`

Then, after expanding `\foo{##1}` (here `##` denotes a single $\#_6$) we end up with a token list with *out_param* `1` replaced:

mac_param `#`*, character* `1`*, mac_param* `#`*, character* `2`

that is, the definition would be:

```
\def\foo#1{#1#2}
```

which obviously fails, because the original input in the definition was `##` but TEX reduced that to a single parameter token $\#_6$ when carrying out the definition. That leaves no room for a clever solution with (say) `\unexpanded`, because anything that would double the second $\#_6$, would also (incorrectly) double the first, so there's not much to do other than a manual solution.

There are three cases we can distinguish to make things hopefully faster on simpler cases:

1. a macro with no parameters;
2. a macro with no parameter tokens in its definition;
3. a macro with parameters *and* parameter tokens.

The first case is trivial: if the macro has no parameters, we can just use `\unexpanded` around it, and if there is a parameter token in it, it is handled correctly (the macro can be treated as a `tl` variable).

The second case requires looking at the ⟨*replacement text*⟩ of the macro to see if it has a parameter token in there. If it does not, then there is no worry, and the macro can be redefined normally (without `\unexpanded`).

The third case, as usual, is the devious one. Here we'll have to loop through the definition token by token, and double every parameter token, so that this case can be handled like the previous one.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_patch_expand_redefine:NNnn}
⟨latexrelease⟩                    {cmd~hooks~with~args}
\cs_new_protected:Npn \__hook_patch_expand_redefine:NNnn #1 #2 #3 #4
  {
    \__hook_patch_debug:e { ++~command~can~be~patched~without~rescanning }
```

We'll start by counting the number of arguments in the command by counting the number of characters in the `\cs_parameter_spec:N` of the macro, divided by two, and subtracting one if the command has an optional argument (that is, an extra `[]` in its ⟨*parameter text*⟩).

```
    \int_set:Nn \l__hook_patch_num_args_int
      {
        \exp_args:Nf \str_count:n { \__kernel_cs_parameter_spec:N #2 } / 2
        \bool_if:NT #1 { -1 }
      }
```

Now build two token lists:

`\l__hook_param_text_tl` will contain the ⟨*parameter text*⟩ to be used when redefining the macro. It should be identical to the ⟨*parameter text*⟩ used when originally defining that macro.

`\l__hook_replace_text_tl` will contain braced pairs of `\c__hook_hashes_tl`⟨*num*⟩ to feed to the macro when expanded. This token list as well as the previous will have the first item surrounded by `[...]` in the case of an optional argument.

The use of `\c__hook_hashes_tl` here is to differentiate actual parameters in the macro from parameter tokens in the original definition of the macro. Later on, `\c__hook_hashes_tl` is either replaced by actual parameter tokens, or expanded into them.

```
    \int_compare:nNnTF { \l__hook_patch_num_args_int } > { \c_zero_int }
      {
```

We'll first check if the command has any parameter token in its definition (feeding it empty arguments), and set `\__hook_exp_not:n` accordingly. `\__hook_exp_not:n` will be used later to either leave `\c__hook_hashes_tl` or expand it, and also to remember the result of `\__hook_if_has_hash:nTF` to avoid testing twice (the test can be rather slow).

```
        \tl_set:Ne \l__hook_tmpa_tl { \bool_if:NTF #1 { [ ] } { { } } }
        \int_step_inline:nnn { 2 } { \l__hook_patch_num_args_int }
          { \tl_put_right:Nn \l__hook_tmpa_tl { { } } }
        \exp_args:NNo \exp_args:No \__hook_if_has_hash:nTF
            { \exp_after:wN #2 \l__hook_tmpa_tl }
          { \cs_set_eq:NN \__hook_exp_not:n \exp_not:n }
          { \cs_set_eq:NN \__hook_exp_not:n \use:n }
        \cs_set_protected:Npn \__hook_tmp:w ##1 ##2
```

```
          {
            ##1 \l__hook_param_text_tl   { \use:n ##2 }
            ##1 \l__hook_replace_text_tl { \__hook_exp_not:n {##2} }
          }
```

Here we'll conditionally add `[...]` around the first parameter:

```
        \bool_if:NTF #1
          { \__hook_tmp:w \tl_set:Ne { [ \c__hook_hash_tl 1 ] } }
          { \__hook_tmp:w \tl_set:Ne { { \c__hook_hash_tl 1 } } }
```

Then, for every parameter from the second, just add it normally:

```
        \int_step_inline:nnn { 2 } { \l__hook_patch_num_args_int }
          { \__hook_tmp:w \tl_put_right:Ne { { \c__hook_hash_tl ##1 } } }
```

Now, if the command has any parameter token in its definition (then `\__hook_exp_not:n` is `\exp_not:n`), call `\__hook_double_hashes:n` to double them, and replace every `\c__hook_hashes_tl` by `#`:

```
        \tl_set:Ne \l__hook_replace_text_tl
          { \exp_not:N #2 \exp_not:V \l__hook_replace_text_tl }
        \tl_set:Ne \l__hook_replace_text_tl
          {
            \token_if_eq_meaning:NNTF \__hook_exp_not:n \exp_not:n
              { \exp_args:NNV \exp_args:No \__hook_double_hashes:n }
              { \exp_args:NV \exp_not:o }
                  \l__hook_replace_text_tl
          }
```

And now, set a few auxiliaries for the case that the macro has parameters, so it won't be passed through `\unexpanded` (twice):

```
        \cs_set_eq:NN \__hook_def_cmd:w \tex_gdef:D
        \cs_set_eq:NN \__hook_exp_not:NN \prg_do_nothing:
      }
      {
```

In the case the macro has no parameters, we'll treat it as a token list and things are much simpler (expansion control looks a bit complicated, but it's just a pair of `\exp_not:N` preventing another `\exp_not:n` from expanding):

```
        \tl_clear:N \l__hook_param_text_tl
        \tl_set_eq:NN \l__hook_replace_text_tl #2
        \cs_set_eq:NN \__hook_def_cmd:w \tex_xdef:D
        \cs_set:Npn \__hook_exp_not:NN ##1 { \exp_not:N ##1 \exp_not:N }
      }
```

Before redefining, we need to also get the prefixes used when defining the command. Here we ensure that the `\escapechar` is printable, otherwise a macro defined with prefixes `\protected \long` will have it `\meaning` printed as `protectedlong`, making life unnecessarily complicated. Here the `\escapechar` is changed to `/`, then we loop between pairs of `/.../` extracting the prefixes.

```
    \group_begin:
      \int_set:Nn \tex_escapechar:D { `\/ }
      \use:e
        {
    \group_end:
    \tl_set:Ne \exp_not:N \l__hook_patch_prefixes_tl
      { \exp_not:N \__hook_make_prefixes:w \cs_prefix_spec:N #2 / / }
        }
```

Here we redefine the hook to have the right number of arguments. Disabling the hook, undefining the `parameter` token list then calling `\__hook_make_usable:nn` are enough to redefine the hook to the extent we want. Code stored in the hook and other metadata about it are not lost in the process.

```
    \__hook_disable:n { cmd / #3 / #4 }
    \cs_undefine:c { c__hook_cmd / #3 / #4_parameter_tl }
    \__hook_make_usable:nn { cmd / #3 / #4 } { \l__hook_patch_num_args_int }
```

Now call `\__hook_redefine_with_hooks:Nnnn` with the macro being redefined in `#1`, then `\UseHook{cmd/<name>/before}` in `#2` or `\UseHook{cmd/<name>/after}` in `#3` (one is always empty), and in `#4` the ⟨*replacement text*⟩ of the macro.

```
    \use:e
      {
        \__hook_redefine_with_hooks:Nnnn \exp_not:N #2
        \str_if_eq:nnTF {#4} { after }
          { \use_ii_i:nn }
          { \use:nn }
            { {
                \__hook_exp_not:NN \exp_not:N \UseHookWithArguments
                  { cmd / #3 / #4 } { \int_use:N \l__hook_patch_num_args_int }
                  \__hook_braced_parameter:n { cmd / #3 / #4 }
            } }
            { { } }
            { \__hook_exp_not:NN \exp_not:V \l__hook_replace_text_tl }
      }
```

Finally, update the hook code.

```
    \__hook_update_hook_code:n { cmd / #3 / #4 }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/06/01}{\__hook_patch_expand_redefine:NNnn}
⟨latexrelease⟩                  {cmd~hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_patch_expand_redefine:NNnn #1 #2 #3 #4
⟨latexrelease⟩  {
⟨latexrelease⟩    \__hook_patch_debug:e { ++~command~can~be~patched~without~rescanning }
⟨latexrelease⟩    \int_set:Nn \l__hook_patch_num_args_int
⟨latexrelease⟩      {
⟨latexrelease⟩        \exp_args:Nf \str_count:n { \__kernel_cs_parameter_spec:N #2 } / 2
⟨latexrelease⟩        \bool_if:NT #1 { -1 }
⟨latexrelease⟩      }
⟨latexrelease⟩    \int_compare:nNnTF { \l__hook_patch_num_args_int } > { \c_zero_int }
⟨latexrelease⟩      {
⟨latexrelease⟩        \tl_set:Nx \l__hook_tmpa_tl { \bool_if:NTF #1 { [ ] } { { } } }
⟨latexrelease⟩        \int_step_inline:nnn { 2 } { \l__hook_patch_num_args_int }
⟨latexrelease⟩          { \tl_put_right:Nn \l__hook_tmpa_tl { { } } }
⟨latexrelease⟩        \exp_args:NNo \exp_args:No \__hook_if_has_hash:nTF
⟨latexrelease⟩            { \exp_after:wN #2 \l__hook_tmpa_tl }
⟨latexrelease⟩          { \cs_set_eq:NN \__hook_exp_not:n \exp_not:n }
⟨latexrelease⟩          { \cs_set_eq:NN \__hook_exp_not:n \use:n }
⟨latexrelease⟩        \cs_set_protected:Npn \__hook_tmp:w ##1 ##2
⟨latexrelease⟩          {
⟨latexrelease⟩            ##1 \l__hook_param_text_tl   { \use:n ##2 }
⟨latexrelease⟩            ##1 \l__hook_replace_text_tl { \__hook_exp_not:n {##2} }
⟨latexrelease⟩          }
```

```
⟨latexrelease⟩          \bool_if:NTF #1
⟨latexrelease⟩            { \__hook_tmp:w \tl_set:Nx { [ \c__hook_hash_tl 1 ] } }
⟨latexrelease⟩            { \__hook_tmp:w \tl_set:Nx { { \c__hook_hash_tl 1 } } }
⟨latexrelease⟩          \int_step_inline:nnn { 2 } { \l__hook_patch_num_args_int }
⟨latexrelease⟩            { \__hook_tmp:w \tl_put_right:Nx { { \c__hook_hash_tl ##1 } } }
⟨latexrelease⟩          \tl_set:Nx \l__hook_replace_text_tl
⟨latexrelease⟩            { \exp_not:N #2 \exp_not:V \l__hook_replace_text_tl }
⟨latexrelease⟩          \tl_set:Nx \l__hook_replace_text_tl
⟨latexrelease⟩            {
⟨latexrelease⟩              \token_if_eq_meaning:NNTF \__hook_exp_not:n \exp_not:n
⟨latexrelease⟩                { \exp_args:NNV \exp_args:No \__hook_double_hashes:n }
⟨latexrelease⟩                { \exp_args:NV \exp_not:o }
⟨latexrelease⟩                    \l__hook_replace_text_tl
⟨latexrelease⟩            }
⟨latexrelease⟩          \cs_set_eq:NN \__hook_def_cmd:w \tex_gdef:D
⟨latexrelease⟩          \cs_set_eq:NN \__hook_exp_not:NN \prg_do_nothing:
⟨latexrelease⟩        }
⟨latexrelease⟩        {
⟨latexrelease⟩          \tl_clear:N \l__hook_param_text_tl
⟨latexrelease⟩          \tl_set_eq:NN \l__hook_replace_text_tl #2
⟨latexrelease⟩          \cs_set_eq:NN \__hook_def_cmd:w \tex_xdef:D
⟨latexrelease⟩          \cs_set:Npn \__hook_exp_not:NN ##1 { \exp_not:N ##1 \exp_not:N }
⟨latexrelease⟩        }
⟨latexrelease⟩      \group_begin:
⟨latexrelease⟩        \int_set:Nn \tex_escapechar:D { `\/ }
⟨latexrelease⟩        \use:x
⟨latexrelease⟩          {
⟨latexrelease⟩      \group_end:
⟨latexrelease⟩      \tl_set:Nx \exp_not:N \l__hook_patch_prefixes_tl
⟨latexrelease⟩        { \exp_not:N \__hook_make_prefixes:w \cs_prefix_spec:N #2 / / }
⟨latexrelease⟩          }
⟨latexrelease⟩      \use:x
⟨latexrelease⟩        {
⟨latexrelease⟩          \__hook_redefine_with_hooks:Nnnn \exp_not:N #2
⟨latexrelease⟩          \str_if_eq:nnTF {#4} { after }
⟨latexrelease⟩            { \use_ii_i:nn }
⟨latexrelease⟩            { \use:nn }
⟨latexrelease⟩              { { \__hook_exp_not:NN \exp_not:N \UseHook { cmd / #3 / #4 } } }
⟨latexrelease⟩              { { } }
⟨latexrelease⟩              { \__hook_exp_not:NN \exp_not:V \l__hook_replace_text_tl }
⟨latexrelease⟩        }
⟨latexrelease⟩    }
⟨latexrelease⟩\EndIncludeInRelease
```

Now that all the needed tools are ready, without further ado we'll redefine the command. The definition uses the prefixes gathered in `\l__hook_patch_prefixes_tl`, a primitive `\__hook_def_cmd:w` (which is `\tex_gdef:D` or `\tex_xdef:D`) to avoid adding extra prefixes, and the ⟨*parameter text*⟩ from `\l__hook_param_text_tl`.

Then finally, in the body of the definition, we insert `#2`, which is `cmd/#1/before` or empty, `#4` which is the ⟨*replacement text*⟩, and `#3` which is `cmd/#1/after` or empty.

```
\cs_new_protected:Npn \__hook_redefine_with_hooks:Nnnn #1 #2 #3 #4
  {
    \l__hook_patch_prefixes_tl
      \exp_after:wN \__hook_def_cmd:w
```

```
        \exp_after:wN #1 \l__hook_param_text_tl
      { #2  #4  #3 }
  }
```

Here's the auxiliary that makes the prefix control sequences for the redefinition. Each item has to be `\tl_trim_spaces:n`'d because the last item (and not any other) has a trailing space.

```
\cs_new:Npn \__hook_make_prefixes:w / #1 /
  {
    \tl_if_empty:nF {#1}
      {
        \exp_not:c { tex_ \tl_trim_spaces:n {#1} :D }
        \__hook_make_prefixes:w /
      }
  }
```

(*End of definition for* `\__hook_patch_expand_redefine:NNnn`, `\__hook_redefine_with_hooks:Nnnn`, *and* `\__hook_make_prefixes:w`.)

Here are some auxiliaries for the contraption above.

`\__hook_if_has_hash_p:n` `\__hook_if_has_hash:nTF` `\__hook_if_has_hash:w` `\__hook_if_has_hash_check:w`

`\__hook_if_has_hash:nTF` searches the token list `#1` for a catcode 6 token, and if any is found, it returns `true`, and `false` otherwise. The searching doesn't care about preserving groups or spaces: we can ignore those safely (braces are removed) so that searching is as fast as possible.

```
\prg_new_conditional:Npnn \__hook_if_has_hash:n #1 { TF }
  { \__hook_if_has_hash:w #1 ## \s__hook_mark }
\cs_new:Npn \__hook_if_has_hash:w #1
  {
    \tl_if_single_token:nTF {#1}
      {
        \token_if_eq_catcode:NNTF ## #1
          { \__hook_if_has_hash_check:w }
          { \__hook_if_has_hash:w }
      }
      { \__hook_if_has_hash:w #1 }
  }
\cs_new:Npn \__hook_if_has_hash_check:w #1 \s__hook_mark
  { \tl_if_empty:nTF {#1} { \prg_return_false: } { \prg_return_true: } }
```

(*End of definition for* `\__hook_if_has_hash:nTF`, `\__hook_if_has_hash:w`, *and* `\__hook_if_has_hash_check:w`.)

`\__hook_double_hashes:n` `\__hook_double_hashes:w` `\__hook_double_hashes_output:N` `\__hook_double_hashes_output_aux:N` `\__hook_double_hashes_stop:w` `\__hook_double_hashes_group:n` `\__hook_double_hashes_space:w`

`\__hook_double_hashes:n` loops through the token list `#1` and duplicates any catcode 6 token, and expands tokens `\ifx`-equal to `\c__hook_hashes_tl`, and leaves all other tokens `\notexpanded` with `\exp_not:N`. Unfortunately pairs of explicit catcode 1 and catcode 2 character tokens are normalized to $\{_1$ and $\}_1$ because it's not feasible to expandably detect the character code (*maybe* it could be done using something along the lines of https://tex.stackexchange.com/a/527538, but it's far too much work for close to zero benefit).

`\__hook_double_hashes:w` is the tail-recursive loop macro, that tests which of the three types of item is in the head of the token list.

```
\cs_new:Npn \__hook_double_hashes:n #1
  { \__hook_double_hashes:w #1 \q__hook_recursion_tail \q__hook_recursion_stop }
\cs_new:Npn \__hook_double_hashes:w #1 \q__hook_recursion_stop
```

```
  {
    \tl_if_head_is_N_type:nTF {#1}
      { \__hook_double_hashes_output:N }
      {
        \tl_if_head_is_group:nTF {#1}
          { \__hook_double_hashes_group:n }
          { \__hook_double_hashes_space:w }
      }
    #1 \q__hook_recursion_stop
  }
```

`\__hook_double_hashes_output:N` checks for the end of the token list, then checks if the token is `\c__hook_hashes_tl`, and if so just leaves it.

```
\cs_new:Npn \__hook_double_hashes_output:N #1
  {
    \if_meaning:w \q__hook_recursion_tail #1
      \__hook_double_hashes_stop:w
    \fi:
    \if:w ?
      \if_meaning:w \c__hook_hash_tl #1 ! \fi:
      \if_meaning:w \c__hook_hashes_tl #1 ! \fi:
          ?
    \else:
```

(this `\use_i:nnnn` uses `\fi:` and consumes `\use:n`, the whole `\if_catcode:w` block, and the `\exp_not:N`, leaving just `#1` which is `\c__hook_hashes_tl`.)

```
      \use_i:nnnn
    \fi:
    \use:n
      {
```

If `#1` is not `\c__hook_hashes_tl`, then check if its catcode is 6, and if so, leave it doubled in `\exp_not:n` and consume the following `\exp_not:N #1`.

```
        \__hook_double_hashes_output_aux:N #1
        \use_none:n
          { \exp_not:n { #1 #1 } }
      }
```

If both previous tests returned `false`, then leave the token unexpanded and resume the loop.

```
    \exp_not:N #1
    \__hook_double_hashes:w
  }
```

LuaTEX has a primitive equivalent to `#`, which is replaced in a dedicated path. That code is not needed for other engines, so we avoid the hit on performance by using an auxiliary.

```
\cs_new:Npn \__hook_double_hashes_output_aux:N #1
  {
    \if_catcode:w ## \exp_not:N #1
      \exp_after:wN \use_ii:nnnn
    \fi:
  }
\sys_if_engine_luatex:T
  {
    \cs_gset:Npn \__hook_double_hashes_output_aux:N #1
      {
```

```
        \if_catcode:w ## \exp_not:N #1
          \exp_after:wN \use_ii:nnnn
        \else:
          \if_meaning:w \tex_alignmark:D \tex_alignmark:D #1
            \exp_after:wN \exp_after:wN \exp_after:wN \use_ii:nnnn
          \fi:
        \fi:
      }
  }
\cs_new:Npn \__hook_double_hashes_stop:w #1 \q__hook_recursion_stop { \fi: }
```

Dealing with spaces and grouped tokens is trivial:

```
\cs_new:Npn \__hook_double_hashes_group:n #1
  { { \__hook_double_hashes:n {#1} } \__hook_double_hashes:w }
\exp_last_unbraced:NNo
\cs_new:Npn \__hook_double_hashes_space:w \c_space_tl
  { ~ \__hook_double_hashes:w }
```

(*End of definition for* `\__hook_double_hashes:n` *and others.*)

4.4.2 Patching by retokenization

At this point we've drained the possibilities of patching a command by expansion-and-redefinition, so we have to resort to patching by retokenizing the command. Patching by retokenization is done by getting the `\meaning` of the command, doing the necessary manipulations on the generated string, and the retokenizing that again by using `\scantokens`.

Patching by retokenization is definitely a riskier business, because it relies that the tokens printed by `\meaning` produce the exact same tokens as the ones in the original definition. That is, the catcode régime must be exactly(ish) the same, and there is no way of telling except by trial and error.

`\__hook_retokenize_patch:Nnn` This is the macro that will control the whole process. First we'll try out one final, rather trivial case, of a command with no arguments; that is, a token list. This case can be patched with the expand-and-redefine routine but it has to be the very last case tested for, because most (all?) robust commands start with a top-level macro with no arguments, so testing this first would short-circuit `\robust@command@act` and the top-level macros would be incorrectly patched. In that case, we just check if the `\cs_parameter_spec:N` is empty, and call `\__hook_patch_expand_redefine:NNnn`.

```
\cs_new_protected:Npn \__hook_retokenize_patch:Nnn #1 #2 #3
  {
    \str_if_eq:eeTF { \__kernel_cs_parameter_spec:N #1 } { }
      { \__hook_patch_expand_redefine:NNnn \c_false_bool #1 {#2} {#3} }
      {
        \__hook_patch_debug:e { ..~command~can~only~be~patched~by~rescanning }
```

Otherwise, we start the actual patching by retokenization job. The code calls `\__hook_try_patch_with_catcodes:Nnnnw` with a different catcode setting:

- The current catcode setting;
- Switching the catcode of `@`;
- Switching the expl3 syntax on or off;

- Both of the above.

If patching succeeds, `\__hook_try_patch_with_catcodes:Nnnnw` has the side-effect of patching the macro `#1` (which may be an internal from the command whose name is `#2`).

```
      \tl_set:Ne \l__hook_tmpa_tl
        {
          \int_compare:nNnTF { \char_value_catcode:n {`\@ } } = { 12 }
            { \exp_not:N \makeatletter } { \exp_not:N \makeatother }
        }
      \tl_set:Ne \l__hook_tmpb_tl
        {
          \bool_if:NTF \l__kernel_expl_bool
            { \ExplSyntaxOff }
            { \ExplSyntaxOn \char_set_catcode_space:n { 32 }}
        }
      \use:e
        {
          \exp_not:N \__hook_try_patch_with_catcodes:Nnnnw
              \exp_not:n { #1 {#2} {#3} }
            { \prg_do_nothing: }
            { \exp_not:V \l__hook_tmpa_tl } % @
            { \exp_not:V \l__hook_tmpb_tl } % _:
            {
              \exp_not:V \l__hook_tmpa_tl   % @
              \exp_not:V \l__hook_tmpb_tl   % _:
            }
        }
            \q_recursion_tail \q_recursion_stop
```

If no catcode setting succeeds, give up and raise an error. The command isn't changed in any way in that case.

```
          {
            \msg_error:nnee { hooks } { cant-patch }
              { \c_backslash_str #2 } { retok }
          }
      }
  }
```

(*End of definition for* `\__hook_retokenize_patch:Nnn`.)

`\__hook_try_patch_with_catcodes:Nnnnw` This function is a simple wrapper around `\__hook_cmd_if_scanable:NnTF` and `\__hook_patch_retokenize:Nnnn` if the former returns ⟨*true*⟩, plus some debug messages.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_try_patch_with_catcodes:Nnnnw}
⟨latexrelease⟩                      {cmd~hooks~with~args}
\cs_new_protected:Npn \__hook_try_patch_with_catcodes:Nnnnw #1 #2 #3 #4
  {
    \quark_if_recursion_tail_stop_do:nn {#4} { \use:n }
    \__hook_patch_debug:e { ++~trying~to~patch~by~retokenization }
    \__hook_cmd_if_scanable:NnTF {#1} {#4}
      {
        \__hook_patch_debug:e { ++~macro~can~be~retokenized~cleanly }
        \__hook_patch_debug:e { ==~retokenizing~macro~now }
        \__hook_patch_retokenize:Nnnn #1 { cmd / #2 / #3 } {#3} {#4}
```

```
        \use_i_delimit_by_q_recursion_stop:nw \use_none:n
      }
      {
        \__hook_patch_debug:e { ---macro~cannot~be~retokenized~cleanly }
        \__hook_try_patch_with_catcodes:Nnnnw #1 {#2} {#3}
      }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/06/01}{\__hook_try_patch_with_catcodes:Nnnnw}
⟨latexrelease⟩                  {cmd~hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_try_patch_with_catcodes:Nnnnw #1 #2 #3 #4
⟨latexrelease⟩  {
⟨latexrelease⟩    \quark_if_recursion_tail_stop_do:nn {#4} { \use:n }
⟨latexrelease⟩    \__hook_patch_debug:e { ++~trying~to~patch~by~retokenization }
⟨latexrelease⟩    \__hook_cmd_if_scanable:NnTF {#1} {#4}
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_patch_debug:e { ++~macro~can~be~retokenized~cleanly }
⟨latexrelease⟩        \__hook_patch_debug:e { ==~retokenizing~macro~now }
⟨latexrelease⟩        \__hook_patch_retokenize:Nnnn #1 {#2} {#3} {#4}
⟨latexrelease⟩        \use_i_delimit_by_q_recursion_stop:nw \use_none:n
⟨latexrelease⟩      }
⟨latexrelease⟩      {
⟨latexrelease⟩        \__hook_patch_debug:e { ---macro~cannot~be~retokenized~cleanly }
⟨latexrelease⟩        \__hook_try_patch_with_catcodes:Nnnnw #1 {#2} {#3}
⟨latexrelease⟩      }
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_try_patch_with_catcodes:Nnnnw`.)

`\kerneltmpDoNotUse` This is an oddity required to be safe (as safe as reasonably possible) when patching the command. The entirety of

⟨*prefixes*⟩ `\def` ⟨*cs*⟩ ⟨*parameter text*⟩ {⟨*replacement text*⟩}

will go through `\scantokens`. The ⟨*parameter text*⟩ and ⟨*replacement text*⟩ are what we are trying to retokenize, so not much worry there. The other items, however, should "just work", so some care is needed to not use too fancy catcode settings. Therefore we can't use an expl3-named macro for ⟨*cs*⟩, nor the expl3 versions of `\def` or the ⟨*prefixes*⟩. That is why the definitions that will eventually go into `\scantokens` will use the oddly (but hopefully clearly)-named `\kerneltmpDoNotUse`:

```
\cs_new_eq:NN \kerneltmpDoNotUse !
```

PhO: Maybe this can be avoided by running the ⟨`parameter text`⟩ *and the* ⟨`replacement text`⟩ *separately through* `\scantokens` *and then putting everything together at the end.*

(*End of definition for* `\kerneltmpDoNotUse`.)

`\__hook_patch_required_catcodes:` Here are the catcode settings that are *mandatory* when retokenizing commands. These are the minimum necessary settings to perform the definitions: they identify control sequences, which must be escaped with $\backslash_0$, delimit the definition with $\{_1$ and $\}_2$, and mark parameters with $\#_6$. Everything else may be changed, but not these.

```
\cs_new_protected:Npn \__hook_patch_required_catcodes:
  {
```

```
    \char_set_catcode_escape:N \\
    \char_set_catcode_group_begin:N \{
    \char_set_catcode_group_end:N \}
    \char_set_catcode_parameter:N \#
    % \int_set:Nn \tex_endlinechar:D { -1 }
    % \int_set:Nn \tex_newlinechar:D { -1 }
  }
```

PhO: etoolbox sets the `\endlinechar` *and* `\newlinechar` *when patching, but as far as I tested these didn't make much of a difference, so I left them out for now. Maybe* `\newlinechar=-1` *avoids a space token being added after the definition.*

PhO: If the patching is split by ⟨`parameter text`⟩ *and* ⟨`replacement text`⟩*, then only # will have to stay in that list.*

PhO: Actually now that we patch `\UseHook{cmd/foo/before}`*, all the tokens there need to have the right catcodes, so this list now includes all lowercase letters, U and H, the slash, and whatever characters in the command name. . . sigh. . .*

(*End of definition for* `\__hook_patch_required_catcodes:`.)

`\__hook_cmd_if_scanable:NnTF` Here we'll do a quick test if the command being patched can in fact be retokenized with the specific catcode setting without changing in meaning. The test is straightforward:

1. apply `\meaning` to the command;
2. split the ⟨*prefixes*⟩, ⟨*parameter text*⟩ and ⟨*replacement text*⟩ and arrange them as

   ⟨*prefixes*⟩`\def\kerneltmpDoNotUse`⟨*parameter text*⟩`{`⟨*replacement text*⟩`}`
3. rescan that with the given catcode settings, and do the definition; then finally
4. compare `\kerneltmpDoNotUse` with the original command.

If both are `\ifx`-equal, the command can be safely patched.

```
\prg_new_protected_conditional:Npnn \__hook_cmd_if_scanable:Nn #1 #2 { TF }
  {
    \cs_set_eq:NN \kerneltmpDoNotUse \scan_stop:
    \cs_set_eq:NN \__hook_tmp:w \scan_stop:
    \use:e
      {
        \cs_set:Npn \__hook_tmp:w
            ##1 \tl_to_str:n { macro: } ##2 -> ##3 \s__hook_mark
          { ##1 \def \kerneltmpDoNotUse ##2   {##3} }
        \tl_set:Ne \exp_not:N \l__hook_tmpa_tl
          { \exp_not:N \__hook_tmp:w \token_to_meaning:N #1 \s__hook_mark }
      }
    \tl_rescan:nV { #2 \__hook_patch_required_catcodes: } \l__hook_tmpa_tl
    \token_if_eq_meaning:NNTF #1 \kerneltmpDoNotUse
      { \prg_return_true: }
      { \prg_return_false: }
  }
```

(*End of definition for* `\__hook_cmd_if_scanable:NnTF`.)

__hook_guess_arg_count:NN
__hook_guess_arg_count:wN
__hook_guess_arg_count:nw

Looks at the parameter text of a macro, and counts the parameters by looking at the number after a `#`, and checking if they are sequential. This macro assumes that all parameters are marked with hashes, and not other characters, and that there is no "trick parameter".

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_guess_arg_count:NN}
⟨latexrelease⟩                  {cmd~hooks~with~args}
\cs_new_protected:Npn \__hook_guess_arg_count:NN #1
  {
    \exp_after:wN \__hook_guess_arg_count:wN
      \token_to_meaning:N #1 \s__hook_mark
  }
\exp_last_unbraced:NNNNo
\cs_new_protected:Npe \__hook_guess_arg_count:wN
    #1 { \tl_to_str:n { macro: } } #2 \s__hook_mark #3
  {
    \int_set:Nn #3
      {
        \exp_not:N \__hook_guess_arg_count:nw { 0 } #2
          \c_hash_str 0 \s__hook_mark
      }
  }
\use:e
  { \cs_new:Npn \exp_not:N \__hook_guess_arg_count:nw #1 #2 \c_hash_str #3 }
  {
    \int_compare:nNnTF { #1 + 1 } = {#3}
      { \__hook_guess_arg_count:nw {#3} }
      { #1 \__hook_use_none_delimit_by_s_mark:w }
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/06/01}{\__hook_guess_arg_count:NN}
⟨latexrelease⟩                  {cmd~hooks~with~args}
⟨latexrelease⟩\cs_undefine:N \__hook_guess_arg_count:NN
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_guess_arg_count:NN`*,* `\__hook_guess_arg_count:wN`*, and* `\__hook_guess_arg_count:nw`*.*)

__hook_patch_retokenize:Nnnn

Then, if `\__hook_cmd_if_scanable:NnTF` returned true, we can go on and patch the command.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{\__hook_patch_retokenize:Nnnn}
⟨latexrelease⟩                  {cmd~hooks~with~args}
\cs_new_protected:Npn \__hook_patch_retokenize:Nnnn #1 #2 #3 #4
  {
```

Here, when patching by retokenization, we can only guess the number of arguments of the macro.

```
    \__hook_guess_arg_count:NN #1 \l__hook_patch_num_args_int
```

Then we redefine the hook to have the right number of arguments. Disabling the hook, undefining the `parameter` token list then calling `\__hook_make_usable:nn` are enough to redefine the hook to the extent we want. Code stored in the hook and other metadata about it are not lost in the process.

```
    \__hook_disable:n {#2}
    \cs_undefine:c { c__hook_#2_parameter_tl }
```

```
    \__hook_make_usable:nn {#2} { \l__hook_patch_num_args_int }
    \tl_set:Ne \l__hook_tmpa_tl
      { \exp_args:Ne \tl_to_str:n { \__hook_braced_parameter:n {#2} } }
    \use:e
      {
        \str_replace_all:Nnn \exp_not:N \l__hook_tmpa_tl
          { ## } { \c_hash_str }
      }
```

Then, make some things `\relax` to avoid lots of `\noexpand` below.

```
    \cs_set_eq:NN \kerneltmpDoNotUse \scan_stop:
    \cs_set_eq:NN \__hook_tmp:w \scan_stop:
    \use:e
      {
```

Now we'll define `\__hook_tmp:w` such that it splits the `\meaning` of the macro (`#1`) into its three parts:

`####1.` ⟨*prefixes*⟩

`####2.` ⟨*parameter text*⟩

`####3.` ⟨*replacement text*⟩

and arrange that a complete definition, then place the `before` or `after` hooks around the ⟨*replacement text*⟩: accordingly.

```
        \cs_set:Npn \__hook_tmp:w
            ##1 \tl_to_str:n { macro: } ##2 -> ##3 \s__hook_mark
          {
            ##1 \def \kerneltmpDoNotUse ##2
              {
                \str_if_eq:nnT {#3} { before }
                  {
                    \token_to_str:N \UseHookWithArguments {#2}
                      { \int_use:N \l__hook_patch_num_args_int }
                      \l__hook_tmpa_tl
                  }
                ##3
                \str_if_eq:nnT {#3} { after }
                  {
                    \token_to_str:N \UseHookWithArguments {#2}
                      { \int_use:N \l__hook_patch_num_args_int }
                      \l__hook_tmpa_tl
                  }
              }
          }
```

Now we just have to get the `\meaning` of the command being patched and pass it through the meat grinder above.

```
        \tl_set:Ne \exp_not:N \l__hook_tmpa_tl
          { \exp_not:N \__hook_tmp:w \token_to_meaning:N #1 \s__hook_mark }
      }
```

Now rescan with the given catcode settings (overridden by the `\__hook_patch_required_catcodes:`), and implicitly (by using the rescanned token list) carry out the definition from above.

```
    \tl_rescan:nV { #4 \__hook_patch_required_catcodes: } \l__hook_tmpa_tl
```

And to close, copy the newly-defined command into the old name and the patching is finally completed:

```
    \cs_gset_eq:NN #1 \kerneltmpDoNotUse
```

Finally, update the hook code.

```
    \__hook_update_hook_code:n {#2}
  }
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/06/01}{\__hook_patch_retokenize:Nnnn}
⟨latexrelease⟩                  {cmd~hooks~with~args}
⟨latexrelease⟩\cs_gset_protected:Npn \__hook_patch_retokenize:Nnnn #1 #2 #3 #4
⟨latexrelease⟩  {
⟨latexrelease⟩    \cs_set_eq:NN \kerneltmpDoNotUse \scan_stop:
⟨latexrelease⟩    \cs_set_eq:NN \__hook_tmp:w \scan_stop:
⟨latexrelease⟩    \use:x
⟨latexrelease⟩      {
⟨latexrelease⟩        \cs_set:Npn \__hook_tmp:w
⟨latexrelease⟩            ####1 \tl_to_str:n { macro: } ####2 -> ####3 \s__hook_mark
⟨latexrelease⟩          {
⟨latexrelease⟩            ####1 \def \kerneltmpDoNotUse ####2
⟨latexrelease⟩              {
⟨latexrelease⟩                \str_if_eq:nnT {#3} { before }
⟨latexrelease⟩                  { \token_to_str:N \UseHook { cmd / #2 / #3 } }
⟨latexrelease⟩                ####3
⟨latexrelease⟩                \str_if_eq:nnT {#3} { after }
⟨latexrelease⟩                  { \token_to_str:N \UseHook { cmd / #2 / #3 } }
⟨latexrelease⟩              }
⟨latexrelease⟩          }
⟨latexrelease⟩        \tl_set:Nx \exp_not:N \l__hook_tmpa_tl
⟨latexrelease⟩          { \exp_not:N \__hook_tmp:w \token_to_meaning:N #1 \s__hook_mark }
⟨latexrelease⟩      }
⟨latexrelease⟩    \tl_rescan:nV { #4 \__hook_patch_required_catcodes: } \l__hook_tmpa_tl
⟨latexrelease⟩    \cs_gset_eq:NN #1 \kerneltmpDoNotUse
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\__hook_patch_retokenize:Nnnn`.)

4.5 Messages

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}{wrong-cmd-hook}%
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/06/01}{wrong-cmd-hook}%
⟨latexrelease⟩                  {Standardize~generic~hook~names}
⟨latexrelease⟩\msg_new:nnnn { hooks } { wrong-cmd-hook }
⟨latexrelease⟩  {
⟨latexrelease⟩    Generic~hook~‘cmd/#1/#2’~is~invalid.
⟨latexrelease⟩%    The~hook~should~be~‘cmd/#1/before’~or~‘cmd/#1/after’.
⟨latexrelease⟩  }
⟨latexrelease⟩  {
⟨latexrelease⟩    You~tried~to~add~a~generic~hook~to~command~\iow_char:N \\#1,~but~‘#2’~
⟨latexrelease⟩    is~an~invalid~component.~Only~‘before’~or~‘after’~are~allowed.
⟨latexrelease⟩  }
```

```
⟨latexrelease⟩\EndIncludeInRelease
\msg_new:nnnn { hooks } { cant-patch }
  {
    Generic~hooks~cannot~be~added~to~'#1'.
  }
  {
    You~tried~to~add~a~hook~to~'#1',~but~LaTeX~was~unable~to~
    patch~the~command~because~it~\__hook_unpatchable_cases:n {#2}.
  }
\cs_new:Npn \__hook_unpatchable_cases:n #1
  {
    \str_case:nn {#1}
      {
        { undef } { doesn't~exist }
        { macro } { is~not~a~macro }
        { expl3 } { is~a~private~expl3~macro }
        { retok } { can't~be~retokenized~cleanly }
      }
  }
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{ltcmdhooks}%
⟨latexrelease⟩                  {The~hook~management~system~for~commands}
⟨latexrelease⟩
```

The command `\__hook_cmd_begindocument_code:` is used in an internal hook, so we need to make sure it has a harmless definition after rollback as that will not remove it from the kernel hook.

```
⟨latexrelease⟩\cs_set_eq:NN \__hook_cmd_begindocument_code: \prg_do_nothing:
⟨latexrelease⟩
⟨latexrelease⟩\EndModuleRelease
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
⟨@@=⟩
```

File 10
ltsockets.dtx

Abstract

This code implements sockets which are places in the code into which predeclared chunks of code (plugs) can be placed. Both the sockets and the plugs are "named" and each socket is assigned exactly one plug at any given time.

1 Introduction

A LaTeX source file is transformed into a typeset document by executing code for each command or environment in the document source. Through various steps this code transforms the input and eventually generates typeset output appearing in a "galley" from which individual pages are cut off in an asynchronous way. This page generating process is normally not directly associated with commands in the input[21] but is triggered whenever the galley has received enough material to form another page (giving current settings).

As part of this transformation input data may get stored in some form and later reused, for example, as part of the output routine processing.

2 Configuration of the transformation process

There are three different major methods offered by LaTeX to configure the transformation process:

- through the template mechanism,
- through the hook mechanism, or
- through sockets and plugs.

They offer different possibilities (with different features and limitations) and are intended for specific use cases, though it is possible to combine them.

2.1 The template mechanism

The template mechanism is intended for more complex document-level elements (e.g., headings such as `\section` or environments like `itemize`). The template code implements the overall processing logic for such an element and offers a set of parameters to influence the final result.

The document element is then implemented by (a) selecting a suitable template (there may be more than one available for the kind of document element) and (b) by setting its parameters to desired values. This then forms a so-called instance which is executed when the document element is found in the source.

By altering the parameter values (in a document class or in the document preamble) or, if more drastic layout changes are desired, by selecting a different template and

[21] Excepts for directives such as `\newpage`.

then adjusting its parameters, a wide variety of layouts can be realized through simple configuration setups without the need to develop new code.

The target audience of this method are therefore document class developers or users who wish to alter an existing layout (implemented by a document class) in certain (minor) ways.

The template mechanism is currently documented as part of the `xtemplate` package and one more elaborate implementation can be found as part of the `latex-lab` code for lists (to be documented further).

2.2 The hook mechanism

Hooks are places in the kernel code (or in packages) that offer packages the possibility to inject additional code at specific points in the processing in a controlled way without the need to replace the existing code block (and thereby overwriting modifications/extensions made by other packages). The target audience is therefore mainly package developers, even though some hooks can be useful for document authors.

Obviously, what can reasonably be added into a hook depends on the individual hook (hopefully documented as part of the hook documentation), but in general the idea behind hooks is that more than one package could add code into the hook at the same time. Perhaps the most famous hook (that LaTeX had for a very long time) is `begindocument` into which many packages add code to through `\AtBeginDocument{`⟨*code*⟩`}` (which is nowadays implemented as a shorthand for `\AddToHook{begindocument}{`⟨*code*⟩`}`). To resolve possible conflicts between injections by different packages there is a rule mechanism by which code chunks in a hook can be ordered in a certain way and by which incompatible packages can be detected if a resolution is impossible.

In contrast to template code, there is no standard configuration method through parameters for hooks, i.e., the code added to a hook "is" the configuration. If it wants to provide for configuration through parameters it has to also provide its own method to set such parameters in some way. However, in that case it is likely that using a hook is not the right approach and the developer better calls a template instance instead which then offers configuration through a key/value interface.

In most cases, hooks do not take any arguments as input. Instead, the data that they can (and are allowed to) access depends on the surrounding context.

For example, the various hooks available during the page shipout process in LaTeX's output routine can (and have to) access the accumulated page material stored in a box named `\ShipoutBox`. This way, code added to, say, the `shipout/before` hook could access the page content, alter it, and then write it back into `\ShipoutBox` and any other code added to this hook could then operate on the modified content. Of course, for such a scheme to work the code prior to executing the hook would need to setup up data in appropriate places and the hook documentation would need to document what kind of storage can be accessed (and possibly altered) by the hook.

There are also hooks that take arguments (typically portions of document data) and in that case the hook code can access these arguments through `#1`, `#2`, etc.

The hook mechanism is documented in `lthooks-doc.pdf`.

2.3 The socket mechanism

In some cases there is code that implements a certain programming logic (for example, combining footnotes, floats, and the text for the current page to be shipped out) and

if this logic should change (e.g., footnotes to be placed above bottom floats instead of below) then this whole code block needs to be replaced with different code.

In theory, this could be implemented with templates, i.e., the code simply calls some instance that implements the logic and that instance is altered by selecting a different templates and/or adjusting their parameters. However, in many cases customization through parameters is overkill in such a case (or otherwise awkward, because parameterization is better done on a higher level instead of individually for small blocks of code) and using the template mechanism just to replace one block of code with a different one results in a fairly high performance hit. It is therefore usually not a good choice.

In theory, it would also be possible to use a hook, but again that is basically a misuse of the concept, because in this use case there should never be more than one block of code inside the hook; thus, to alter the processing logic one would need to set up rules that replace code rather than (as intended) execute all code added to the hook.

For this reason LaTeX now offers a third mechanism: "sockets" into which one can place exactly one code block — a "plug".

In a nutshell: instead of having a fixed code block somewhere as part of the code, implementing a certain programming logic there is a reference to a named socket at this point. This is done by first declaring the named socket with:

`\NewSocket{`⟨*socket-name*⟩`}{`⟨*number-of-inputs*⟩`}`

This is then referenced at the point where the replaceable code block should be executed with:

`\UseSocket{`⟨*socket-name*⟩`}`

or, if the socket should take a number of inputs (additional arguments beside the name) with

`\UseSocket{`⟨*socket-name*⟩`}{`⟨arg_1⟩`}...{`⟨$arg_{\langle number\text{-}of\text{-}inputs \rangle}$⟩`}`

In addition, several code blocks (a.k.a. plugs) implementing different logic for this socket are set up, each with a declaration of the form:

`\NewSocketPlug{`⟨*socket-name*⟩`}{`⟨*socket-plug-name*⟩`}{`⟨*code*⟩`}`

Finally, one of them is assigned to the socket:

`\AssignSocketPlug{`⟨*socket-name*⟩`}{`⟨*socket-plug-name*⟩`}`

If the programming logic should change, then all that is necessary is to make a new assignment with `\AssignSocketPlug` to a different `{`⟨*socket-plug-name*⟩`}`. This assignment obeys scope so that an environment can alter a socket without the need to restore the previous setting manually.

If the socket takes inputs, then those need to be provided to `\UseSocket` and in that case they can be referenced in the ⟨*code*⟩ argument of `\NewSocketPlug` with `#1`, `#2`, etc.

In most cases a named socket is used only in a single place, but there is, of course, nothing wrong with using it in several places, as long as the code in all places is supposed to change in the same way.

2.3.1 Examples

We start by declaring a new socket named `foo` that expects two inputs:

```
\NewSocket{foo}{2}
```

Such a declaration has do be unique across the whole LaTeX run. Thus, if another package attempts to use the same name (regardless of the number of inputs) it will generate an error:

```
\NewSocket{foo}{2}
\NewSocket{foo}{1}
```

Both declarations would therefore produce:

```
! LaTeX socket Error: Socket 'foo' already declared!
```

You also get an error if you attempt to declare some socket plug and the socket name is not yet declared, e.g.,

```
\NewSocketPlug{baz}{undeclared}{some code}
```

generates

```
! LaTeX socket Error: Socket 'baz' undeclared!
```

Setting up plugs for the socket is done like this:

```
\NewSocketPlug{foo}{plug-A}
   {\begin{quote}\itshape foo-A: #1!#2\end{quote}}
\NewSocketPlug{foo}{plug-B}
   {\begin{quote}\sffamily foo-B: #2\textsuperscript{2}\end{quote}}
```

This will set up the plugs `plug-A` and `plug-B` for this socket.

We still have to assign one or the other to the socket, thus without doing that the line

```
\UseSocket{foo}{hello}{world}
```

produces nothing because the default plug for sockets with 2 inputs is `noop` (which grabs the additional arguments and throws them away).[22]

So let's do the assignment

```
\AssignSocketPlug{foo}{plug-A}
```

and then

```
\UseSocket{foo}{hello}{world}
```

will properly typeset

foo-A: hello!world

and after

```
\AssignSocketPlug{foo}{plug-B}
```

[22] If socket `foo` would have been a socket with one input, then the default plug would be `identity`, in which case the socket input would remain without braces and gets typeset!

and another call to

```
\UseSocket{foo}{hello}{world}
```

we get

foo-B: world[2]

If we attempt to assign a plug that was not defined, e.g.,

```
\AssignSocketPlug{foo}{plug-C}
```

then we get an error during the assignment

```
! LaTeX socket Error: Plug 'plug-C' for socket 'foo' undeclared!
```

and the previous assignment remains in place.

To see what is known about a socket and its plugs you can use `\ShowSocket` or `\LogSocket` which displays information similar to this on the terminal or in the transcript file:

```
Socket foo:
    number of inputs = 2
    available plugs = noop, plug-A, plug-B
    current plug = plug-B
    definition = \long macro:#1#2->\begin {quote}\sffamily
foo-B: #2\textsuperscript {2}\end {quote}
```

2.3.2 Details and semantics

In this section we collect some normative statements.

- From a functional point of view sockets are like simple TEX macros, i.e., they expect 0 to 9 mandatory arguments (the socket inputs) and get replaced by their "expansion"
- A socket is "named" and the name consists of ASCII letters `[a-z]`, `[A-Z]`, `[0-9]`, `[-/@]` only
- Socket names have to be unique, i.e., there can be only one socket named ⟨*name*⟩. This is ensured by declaring each socket with `\NewSocket`.

  However, there is no requirement that sockets and hook names have to be different. In fact, if a certain action that could otherwise be specified as hook code has to be executed always last (or first) one could ensure this by placing a socket (single action) after a hook (or vice versa) and using the same name to indicate the relationship, e.g.,

  ```
  \UseHook{foo}        % different package can add code here
  \UseSocket{foo}      % only one package can assign a plug
  ```

  This avoids the need to order the hook code to ensure that something is always last.
- Best practice naming conventions are ... *to be documented*

- A socket has documented inputs which are
  - the positional arguments (if any) with a description of what they contain when used
  - implicit data (registers and other 2e/expl3 data stores) that the socket is allowed to make use of, with a documented description of what they contain (if relevant for the task at hand—no need to describe the whole LaTeX universe)
  - information about the state of the TeX engine (again when relevant), e.g. is called in mmode or vmode or in the output routine or ...
  - ... anything missing?
- A socket has documented results/outputs which can be
  - what kind of data it should write to the current list (if that is part of its task)
  - what kind of registers and other 2e/expl3 data stores it should modify and in what way
  - what kind of state changes it should do (if any)
  - *... anything else?*
- At any time a socket has one block of code (a plug :-)) associated with it. Such code is itself named and the association is done by linking the socket name to the code name (putting a plug into the socket).
- The name of a plug consists of ASCII letters `[a-z]`, `[A-Z]`, `[0-9]`, `[-/@]` only.
- Socket plug names have to be unique within on a per socket basis, but it is perfectly allowed (and sensible in some cases) to use the same plug name with different sockets (where based on the sockets' purposes, different actions may be associated with the plug name). For example `noop` is a plug name declared for every socket, yet it action "grab the socket inputs and throw them away" obviously differs depending on how many inputs the socket has.
- When declaring a plug it is stated for which socket it is meant (i.e., its code can only be used with that socket). This means that the same plug name can be used with different sockets referring to different code in each case.
- Configuration of a socket can only be done by linking different code to it. Nevertheless the code linked to it can provide its own means of configuration (but this is outside of the spec).
- Technically execution of a socket (`\UseSocket`) involves
  - doing any house keeping (like writing debugging info, ...);
  - looking up the current code association (what plug is in the socket);
  - executing this code which will pick up the mandatory arguments (happens at this point, not before), i.e., it is like calling a csname defined with

    ```
    \def\foo#1#2...{...#1...#2...}
    ```
  - do some further house keeping (if needed).
- A socket is typically only used in one place in code, but this is not a requirement, i.e., if the same operation with the same inputs need to be carried out in several places the same named socket can be used.

2.4 Socket and plug names

The ⟨*socket-name*⟩ and ⟨*socket-plug-name*⟩ are always expanded using TEX's `\csname...\endcsname`, as such, (expandable) commands and Unicode characters are allowed in ⟨*hook*⟩ and ⟨*label*⟩ arguments.

2.4.1 Command syntax

We give both the LATEX 2ε and the L3 programming layer command names.

`\NewSocket` `\NewSocket {⟨socket-name⟩} {⟨number-of-inputs⟩}`
`\socket_new:nn` `\socket_new:nn {⟨socket-name⟩} {⟨number-of-inputs⟩}`

Declares a new socket with name ⟨*socket-name*⟩ having ⟨*number-of-inputs*⟩ inputs. There is automatically a plug `noop` declared for it, which does nothing, i.e., it gobbles the socket inputs (if any). This is made the default plug except for sockets with one input which additionally define the plug `identity` and assign that as their default.

This `identity` plug simply returns the socket input without its outer braces. The use case for this plug are situations like this:

`\UseSocket{tagsupport/footnote}{`⟨*code*⟩`}`

If tagging is not active and the socket contains the plug `identity` then this returns ⟨*code*⟩ without the outer braces and to activate tagging all that is necessary is to change the plug to say `tagpdf` so that it surrounds ⟨*code*⟩ by some tagging magic. This is the most common use case for sockets with one input, which is why they have this special default.

The socket documentation should describe its purpose, its inputs and the expected results as discussed above.

The declaration is only allowed at top-level, i.e., not inside a group.

`\NewSocketPlug` `\NewSocketPlug {⟨socket-name⟩} {⟨socket-plug-name⟩} {⟨code⟩}`
`\socket_new_plug:nnn` `\socket_new_plug:nnn {⟨socket-name⟩} {⟨socket-plug-name⟩} {⟨code⟩}`
`\socket_set_plug:nnn` `\socket_set_plug:nnn {⟨socket-name⟩} {⟨socket-plug-name⟩} {⟨code⟩}`

Declares a new plug for socket ⟨*socket-name*⟩ that runs ⟨*code*⟩ when executing. It complains if the plug was already declared previously.

The form `\socket_set_plug:nnn` changes an existing plug. As this should normally not be necessary, we currently have only an L3 layer name for the few cases it might be useful.

The declarations can be made inside a group and obey scope, i.e., they vanish if the group ends.

`\AssignSocketPlug` `\AssignSocketPlug {⟨socket-name⟩} {⟨socket-plug-name⟩}`
`\socket_assign_plug:nn` `\socket_assign_plug:nn {⟨socket-name⟩} {⟨socket-plug-name⟩}`

Assigns the plug ⟨*socket-plug-name*⟩ to the socket ⟨*socket-name*⟩. It errors if either socket or plug is not defined.

The assignment is local, i.e., it obeys scope.

\UseSocket
\socket_use:nw
\socket_use:n
\socket_use:nn
\socket_use:nnn
\socket_use:nnnn

```
\UseSocket      {⟨socket-name⟩}
\socket_use:nnn {⟨socket-name⟩} {⟨socket-arg1⟩} {⟨socket-arg2⟩}
```

Executes the socket ⟨*socket-name*⟩ by retrieving the ⟨*code*⟩ of the current plug assigned to the socket. This is the only command that would appear inside macro code in packages.

For performance reasons there is no explicit check that the socket was declared!

The different L3 programming layer commands are really doing the same thing: they grab as many arguments as defined as inputs for the socket and then pass them to the plug. The different names are only there to make the code more readable, i.e., to indicate how many arguments are grabbed in total (note that no runtime check is made to verify that this is actually true). We only provide them for sockets with up to 3 inputs (most likely those with zero or one input would have been sufficient). If you happen to have a socket with more inputs, use `\socket_use:nw`.

\socket_use_expandable:nw ⋆
\socket_use_expandable:n ⋆

```
\socket_use_expandable:n {⟨socket-name⟩}
```

Fully expandable variant of `\socket_use:n`. This can be used in macro code to retrieve code from sockets which need to appear in an expandable context.

This usually requires the plug to only contain expandable code and should therefore only be used for sockets which are clearly documented to be used in an expandable context. This command does not print any debugging info when `\DebugSocketsOn` is active and should therefore be avoided whenever possible.

For performance reasons there is no explicit check that the socket was declared!

\ShowSocket
\LogSocket
\socket_show:n
\socket_log:n

```
\ShowSocket    {⟨socket-name⟩}
\socket_show:n {⟨socket-name⟩}
```

Displays information about the socket ⟨*socket-name*⟩ and its state then stops and waits for further instructions — at the moment some what rudimentary.

`\LogSocket` and `\socket_log:n` only differ in that they don't stop.

It is sometimes necessary/helpful to know if a particular socket or plug exists (or is assigned to a certain socket) and based on that take different actions.

\IfSocketExistsTF ⋆
\socket_if_exist:nTF ⋆

```
\IfSocketExistsTF {⟨socket-name⟩} {⟨true code⟩} {⟨false code⟩}
```

If socket ⟨*socket-name*⟩ exists then execute ⟨*true code*⟩ otherwise ⟨*false code*⟩. Variants with only `T` or `F` are also available.

\IfSocketPlugExistsTF ⋆
\socket_if_plug_exist:nnTF ⋆

```
\IfSocketPlugExistsTF {⟨socket-name⟩} {⟨plug-name⟩}
      {⟨true code⟩} {⟨false code⟩}
```

If plug ⟨*plug-name*⟩ for socket ⟨*socket-name*⟩ exists then execute ⟨*true code*⟩ otherwise ⟨*false code*⟩. Variants with only `T` or `F` are also available.

\IfSocketPlugAssignedTF ⋆
\socket_if_plug_assigned:nnTF ⋆

```
\IfSocketPlugAssignedTF {⟨socket-name⟩} {⟨plug-name⟩}
      {⟨true code⟩} {⟨false code⟩}
```

If plug ⟨*plug-name*⟩ is assigned to socket ⟨*socket-name*⟩ then execute ⟨*true code*⟩ otherwise ⟨*false code*⟩. Variants with only `T` or `F` are also available.

\socket_get_plug:nN `\socket_get_plug:nN {⟨socket-name⟩} {⟨tl var⟩}`

This stores into ⟨*tl var*⟩ the name of the plug currently assigned to the socket ⟨*socket-name*⟩. If the socket is not declared an error is issued.

\DebugSocketsOn
\DebugSocketsOff
\socket_debug_on:
\socket_debug_off:

`\DebugSocketsOn ... \DebugSocketsOff`

Turns debugging of sockets on or off.

2.4.2 Rationale for error handling

The errors during the declarations are produced to help with typos—after all, such declarations might be part of a document preamble (not that likely, but possible). However, `\UseSocket` is not doing much checking, e.g.,

```
\UseSocket{mispelled-socket}{hello}{world}
```

will generate a rather low-level error and then typesets "helloworld" because there is no dedicated runtime check if `mispelled-socket` is a known socket.

The reason is that if the misspelling is in the code, then this is a programming error in the package and for speed reasons LaTeX does not repeately make runtime checks for coding errors unless they can or are likely to be user introduced.

3 The Implementation

The implementation of the socket mechanism should be (partially) redone and we should probably store the different code chunks in a property list so that we can have a decent `\ShowSocket` command that shows the available alternatives.

TODO: *implement?*

```
⟨*2ekernel | latexrelease⟩
\ExplSyntaxOn
⟨@@=socket⟩
⟨latexrelease⟩\NewModuleRelease{2023/11/01}{ltsockets}
⟨latexrelease⟩                    {The~socket~management~system}
```

3.1 Debugging the socket structures

Code and commands in this section are not final, it needs more experimentation to see what kind of tracing information is going to be useful in practice. For now the tracing is mainly meant to be used for code testing and not so much for application testing.

It is quite likely that the commands and the behavior of the tracing might change in the future once we gained some experience with it.

\g__socket_debug_bool Holds the current debugging state.

```
\bool_new:N \g__socket_debug_bool
```

(*End of definition for* `\g__socket_debug_bool`.)

\socket_debug_on:
\socket_debug_off:
__socket_debug:n
__socket_debug_term:n
__socket_debug_gset:

Turns debugging on and off by redefining `\__socket_debug:n` and `\__socket_debug_term:n`. By default they do nothing.

```
\cs_new_eq:NN \__socket_debug:n \use_none:n
\cs_new_eq:NN \__socket_debug_term:n \use_none:n

\cs_new_protected:Npn \socket_debug_on:
  {
    \bool_gset_true:N \g__socket_debug_bool
    \__socket_debug_gset:
  }
\cs_new_protected:Npn \socket_debug_off:
  {
    \bool_gset_false:N \g__socket_debug_bool
    \__socket_debug_gset:
  }
\cs_new_protected:Npn \__socket_debug_gset:
  {
    \cs_gset_protected:Npe \__socket_debug:n ##1
      { \bool_if:NT \g__socket_debug_bool {##1} }
    \cs_gset_protected:Npe \__socket_debug_term:n ##1
      { \bool_if:NT \g__socket_debug_bool

        { \iow_term:e { ^^J [Sockets]~ ==>~ ##1 \exp_not:N\on@line} } }
  }
```

(*End of definition for* `\socket_debug_on:` *and others. These functions are documented on page* *361*.)

3.2 The L3 layer commands

\socket_new:nn

Declaring a socket creates a str to hold the name (a pointer) to the code that should be used when the socket is executed, and an integer to hold the number of inputs of that socket. Initially, an "empty" code chunk is created and assigned so the socket does nothing by default other than swallowing its inputs (if any).

```
\cs_new_protected:Npn \socket_new:nn #1 #2 {
   \socket_if_exist:nTF {#1}
      {
        \msg_error:nnn { socket } { already-declared } {#1}
      }
      {
```

We only support declarations on top-level.

```
        \int_if_zero:nTF \tex_currentgrouplevel:D
          {
            \str_new:c { l__socket_#1_plug_str }
            \seq_new:c { l__socket_#1_plugs_seq }
            \int_const:cn { c__socket_#1_args_int } {#2}
            \socket_new_plug:nnn {#1} { noop } {}
            \int_compare:nNnTF {#2} = 1
              {
                \socket_new_plug:nnn {#1} { identity } {##1}
                \socket_assign_plug:nn {#1} { identity }
              }
              { \socket_assign_plug:nn {#1} { noop } }
            \__socket_debug_term:n
```

```
                { Socket~ '#1'~ declared~ with~ #2~ input(s) }
            }
            {
              \msg_error:nn { socket } { not-top-level }
            }
        }
}
```

(*End of definition for* `\socket_new:nn`. *This function is documented on page 359.*)

`\socket_if_exist_p:n`
`\socket_if_exist:nTF`

Conditional testing the existence of a socket. The argument is fully expanded as part of the csname generation.

```
\prg_new_conditional:Npnn \socket_if_exist:n #1 { p , T , F , TF }
  { \str_if_exist:cTF { l__socket_#1_plug_str }
                      \prg_return_true:
                      \prg_return_false:
  }
```

(*End of definition for* `\socket_if_exist:nTF`. *This function is documented on page 360.*)

`\socket_log:n`
`\socket_show:n`

Show the current state of the socket — for now this is just a quick draft and should be redone and extended.

```
\cs_new_protected:Npn \socket_log:n #1 {
  \typeout{ Socket~ #1:}
   \socket_if_exist:nTF {#1}
      {
        \typeout{ \@spaces number~ of~ inputs~ =~
                  \int_use:c { c__socket_#1_args_int } }
        \typeout{ \@spaces available~plugs~ =~
                  \seq_use:cnnn { l__socket_#1_plugs_seq }{,~}{,~}{,~} }
        \typeout{ \@spaces current~ plug~ =~
                  \str_use:c { l__socket_#1_plug_str } }
        \typeout{ \@spaces definition~ =~
                  \cs_meaning:c
                    { __socket_#1_plug_ \str_use:c { l__socket_#1_plug_str } :w } }
        \typeout{}
      }
      {
```

If we are showing a socket it is not an error if it doesn't exist.

```
        \typeout { Socket~ is~ not~ declared! }
      }
}
```

And here the version that stops:

```
\cs_new_protected:Npn \socket_show:n #1 {\socket_log:n {#1} \errmessage{}}
```

(*End of definition for* `\socket_log:n` *and* `\socket_show:n`. *These functions are documented on page 360.*)

`\socket_new_plug:nnn`
`\socket_set_plug:nnn`

Declaring a code for a socket is just making a definition, taking the number of arguments from the saved int.

```
\cs_new_protected:Npn \socket_new_plug:nnn #1#2#3 {
   \socket_if_exist:nTF {#1}
      {
```

```
        \socket_if_plug_exist:nnTF {#1} {#2}
          {
            \msg_error:nnnn { socket } { plug-already-declared } {#1} {#2}
          }
          {
            \cs_generate_from_arg_count:cNnn
              { __socket_#1_plug_#2:w }
              \cs_new:Npn
              { \int_use:c { c__socket_#1_args_int } }
              {#3}
```

This is a new declaration so we add the name to a seq for the debugging info.

```
            \seq_put_right:cn { l__socket_#1_plugs_seq } {#2}
            \__socket_debug_term:n
              { Plug~ '#2'~ for~ socket~ '#1'~ declared }
          }
      }
      { \msg_error:nnn { socket } { undeclared } {#1} }
}
```

Changing the plug of an existing socket is rather similar, except that we don't have to deal with adding it to the debugging sequence.

```
\cs_new_protected:Npn \socket_set_plug:nnn #1#2#3 {
   \socket_if_exist:nTF {#1}
      {
        \socket_if_plug_exist:nnTF {#1} {#2}
          {
            \cs_generate_from_arg_count:cNnn
              { __socket_#1_plug_#2:w }
              \cs_set:Npn
              { \int_use:c { c__socket_#1_args_int } }
              {#3}
            \__socket_debug_term:n
              {  Plug~ '#2'~ for~ socket~ '#1'~ changed }
          }
          {
            \msg_error:nnnn { socket } { plug-undeclared } {#1} {#2}
          }
      }
      { \msg_error:nnn { socket } { undeclared } {#1} }
}
```

(*End of definition for* `\socket_new_plug:nnn` *and* `\socket_set_plug:nnn`. *These functions are documented on page* 359.)

`\socket_if_plug_exist_p:nn`
`\socket_if_plug_exist:nnTF`

Conditional testing the existence of a plug. Both arguments are fully expanded as part of the csname generation.

```
\prg_new_conditional:Npnn \socket_if_plug_exist:nn #1#2 { p , T , F , TF }
  { \cs_if_exist:cTF { __socket_#1_plug_#2:w }
                     \prg_return_true:
                     \prg_return_false:
  }
```

(*End of definition for* `\socket_if_plug_exist:nnTF`. *This function is documented on page* 360.)

\socket_assign_plug:nn

Assigning a plug to a socket just changes the name in the socket string. The assignment is local to the current group.

```
\cs_new_protected:Npn \socket_assign_plug:nn #1 #2 {
  \socket_if_exist:nTF {#1}
    {
      \socket_if_plug_exist:nnTF {#1} {#2}
        {
          \__socket_debug_term:n
            { Replacing~ plug~ '\str_use:c { l__socket_#1_plug_str }'~
              with~ '#2'~ in~ socket~ '#1' }
          \str_set:cn { l__socket_#1_plug_str } {#2}
        }
        {
          \msg_error:nnnn { socket } { plug-undeclared } {#1} {#2}
        }
    }
    { \msg_error:nnn { socket } { undeclared } {#1} }
}
```

(*End of definition for* `\socket_assign_plug:nn`*. This function is documented on page* 359.)

\socket_if_plug_assigned_p:nn
\socket_if_plug_assigned:nnTF

Conditional testing the assignment of a plug. Both arguments are fully expanded.

```
\prg_new_conditional:Npnn \socket_if_plug_assigned:nn #1#2 { p , T , F , TF }
  { \exp_args:Ne
    \str_if_eq:nvTF {#2} { l__socket_#1_plug_str }
                    \prg_return_true:
                    \prg_return_false:
  }
```

(*End of definition for* `\socket_if_plug_assigned:nnTF`*. This function is documented on page* 360.)

\socket_get_plug:nN

Retrieving the currently assigned plug.

```
\cs_new_protected:Npn \socket_get_plug:nN #1#2
  {
    \socket_if_exist:nTF {#1}
     {
       \tl_set_eq:Nc #2 { l__socket_#1_plug_str }
     }
     {
       \msg_error:nnn { socket } { undeclared } {#1}
     }
  }
```

(*End of definition for* `\socket_get_plug:nN`*. This function is documented on page* 361.)

\socket_use:nw
\socket_use:n
\socket_use:nn
\socket_use:nnn
\socket_use:nnnn

And using it is more or less a `\use:c` so very lightweight. We do not add a runtime check for speed reasons!

This command is named `\socket_use:nw` because we don't know how many inputs the socket has until we have looked at the socket name (in argument `#1`). But, of course, the developer knows so we also offer a few aliases `\socket_use:nn`, etc. so that one can indicate the correct number of arguments (socket inputs plus one) in the L3 layer code.

```
\cs_new_protected:Npn \socket_use:nw #1 {
  \__socket_debug_term:n
    { Socket~ '#1'~ containing~ plug~
```

```
      '\str_use:c { l__socket_#1_plug_str }'~ used }
  \use:c { __socket_#1_plug_ \str_use:c { l__socket_#1_plug_str } :w }
}
```

To make code a bit more readable we also define functions that indicate how many arguments are picked up. However, this is just for code documentation: internally they all do the same and the number of arguments isn't checked by default.

```
\cs_new_eq:NN  \socket_use:n \socket_use:nw     % socket with no inputs
\cs_new_eq:NN  \socket_use:nn \socket_use:nw    % socket with one input
\cs_new_eq:NN  \socket_use:nnn \socket_use:nw   % socket with two inputs
\cs_new_eq:NN  \socket_use:nnnn \socket_use:nw  % socket with three inputs
```

The above commands could be changed to check how many inputs the socket is declared with (for example, when checking is in force).

TODO: *Implement?*

(*End of definition for* `\socket_use:nw` *and others. These functions are documented on page* 360.)

`\socket_use_expandable:nw`
`\socket_use_expandable:n`

The same as the non-expandable code, except for the missing debug output.

```
\cs_new:Npn \socket_use_expandable:nw #1 {
  \use:c { __socket_#1_plug_ \str_use:c { l__socket_#1_plug_str } :w }
}
\cs_new_eq:NN  \socket_use_expandable:n \socket_use_expandable:nw     % socket with no inputs
```

(*End of definition for* `\socket_use_expandable:nw` *and* `\socket_use_expandable:n`. *These functions are documented on page* 360.)

3.3 Error messages

```
\msg_new:nnnn { socket } { already-declared }
    { Socket~ '#1'~ already~ declared! }
    { A~ socket~ can~ only~ be~ declared~ once.~ The~ name~ '#1'~ is~
      already~ taken.~ Use~ \ShowSocket{#1}~ to~ see~ its~ definition. }

\msg_new:nnnn { socket } { undeclared }
    { Socket~ '#1'~ undeclared! }
    { You~ tried~ to~ use~ a~ socket~ that~ was~ not~ declared~ before. }

\msg_new:nnnn { socket } { not-top-level }
    { Sockets~ can~ only~ be~ declared~ at~ top-level! }
    { It~ is~ not~ allowed~ to~ declare~ sockets~ inside~ a~
      group.~ Move~ the~ declaration~ to~ the~ top-level. }

\msg_new:nnnn { socket } { plug-already-declared }
    { Plug~ '#2'~ for~ socket~ '#1'~ already~ declared! }
    { You~ can't~ change~ an~ existing~ plug~ with~ \NewSocketPlug~ and~ it~
      is~ normally~ not~ sensible~ to~ do~ so.~ Use~ the~ L3~ programming~
      layer~ function~ \socket_set_plug:nnn~ if~ you~ really~ have~ to. }

\msg_new:nnnn { socket } { plug-undeclared }
    { Plug~ '#2'~ for~ socket~ '#1'~ undeclared! }
    { The~ plug~ name~ is~ unknown.~ Is~ the~ name~ misspelled~ or~ did~ you~
      intend~ to~ assign~ it~ to~ a~ different~ socket? }

\prop_gput:Nnn \g_msg_module_type_prop { socket } { LaTeX }
```

3.4 The LaTeX 2ε interface commands

\NewSocket
\NewSocketPlug
\ShowSocket
\LogSocket
\AssignSocketPlug
\UseSocket
\DebugSocketsOn
\DebugSocketsOff

As we expect that there are existing LaTeX 2ε packages that may want to make use of the socket mechanism, we provide 2e names for most of the commands.

```
\cs_new_eq:NN \NewSocket          \socket_new:nn
\cs_new_eq:NN \ShowSocket         \socket_show:n
\cs_new_eq:NN \LogSocket          \socket_log:n

\cs_new_eq:NN \NewSocketPlug      \socket_new_plug:nnn
\cs_new_eq:NN \AssignSocketPlug   \socket_assign_plug:nn
\cs_new_eq:NN \UseSocket          \socket_use:nw

\cs_new_eq:NN \DebugSocketsOn     \socket_debug_on:
\cs_new_eq:NN \DebugSocketsOff    \socket_debug_off:
```

(*End of definition for* `\NewSocket` *and others. These functions are documented on page* 359.)

\IfSocketExistsTF
\IfSocketExistsT
\IfSocketExistsF
\IfSocketPlugExistsTF
\IfSocketPlugExistsT
\IfSocketPlugExistsF
\IfSocketPlugAssignedTF
\IfSocketPlugAssignedT
\IfSocketPlugAssignedF

A bunch of conditionals:

```
\cs_new_eq:NN \IfSocketExistsTF \socket_if_exist:nTF
\cs_new_eq:NN \IfSocketExistsT  \socket_if_exist:nT
\cs_new_eq:NN \IfSocketExistsF  \socket_if_exist:nF

\cs_new_eq:NN \IfSocketPlugExistsTF \socket_if_plug_exist:nnTF
\cs_new_eq:NN \IfSocketPlugExistsT  \socket_if_plug_exist:nnT
\cs_new_eq:NN \IfSocketPlugExistsF  \socket_if_plug_exist:nnF

\cs_new_eq:NN \IfSocketPlugAssignedTF \socket_if_plug_assigned:nnTF
\cs_new_eq:NN \IfSocketPlugAssignedT  \socket_if_plug_assigned:nnT
\cs_new_eq:NN \IfSocketPlugAssignedF  \socket_if_plug_assigned:nnF
```

(*End of definition for* `\IfSocketExistsTF` *and others. These functions are documented on page* 360.)

```
%
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{ltsockets}
⟨latexrelease⟩                  {The~socket~management~(undo)}%
⟨latexrelease⟩
⟨latexrelease⟩\let \NewSocket  \@undefined
⟨latexrelease⟩\let \ShowSocket \@undefined
⟨latexrelease⟩\let \LogSocket  \@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let \NewSocketPlug    \@undefined
⟨latexrelease⟩\let \AssignSocketPlug \@undefined
⟨latexrelease⟩\let \UseSocket        \@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let \DebugSocketsOn  \@undefined
⟨latexrelease⟩\let \DebugSocketsOff \@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let \IfSocketExistsTF \@undefined
⟨latexrelease⟩\let \IfSocketExistsT  \@undefined
⟨latexrelease⟩\let \IfSocketExistsF  \@undefined
⟨latexrelease⟩\let \IfSocketPlugExistsTF \@undefined
⟨latexrelease⟩\let \IfSocketPlugExistsT  \@undefined
⟨latexrelease⟩\let \IfSocketPlugExistsF  \@undefined
⟨latexrelease⟩\let \IfSocketPlugAssignedTF \@undefined
⟨latexrelease⟩\let \IfSocketPlugAssignedT  \@undefined
⟨latexrelease⟩\let \IfSocketPlugAssignedF  \@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndModuleRelease
```

```
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
```

Reset module prefix:

```
⟨@@=⟩
```

File 11
lttemplates.dtx

1 Introduction

There are three broad "layers" between putting down ideas into a source file and ending up with a typeset document. These layers of document writing are

1. authoring of the text with mark-up;
2. document layout design;
3. implementation (with TEX programming) of the design.

We write the text as an author, and we see the visual output of the design after the document is generated; the TEX implementation in the middle is the glue between the two.

LATEX's greatest success has been to standardize a system of mark-up that balances the trade-off between ease of reading and ease of writing to suit almost all forms of technical writing. It's other original strength was a good background in typographical design; while the standard LATEX 2_ε classes look somewhat dated now in terms of their visual design, their typography is generally sound (barring the occasional minor faults).

However, LATEX 2_ε has always lacked a standard approach to customising the visual design of a document. Changing the looks of the standard classes involved either:

- Creating a new version of the implementation code of the class and editing it.
- Loading one of the many packages to customise certain elements of the standard classes.
- Loading a completely different document class, such as KOMA-Script or memoir, that allows easy customization.

All three of these approaches have their drawbacks and learning curves.

The idea behind lttemplates is to cleanly separate the three layers introduced at the beginning of this section, so that document authors who are not programmers can easily change the design of their documents. lttemplates also makes it easier for LATEX programmers to provide their own customizations on top of a pre-existing class.

2 What is a document?

Besides the textual content of the words themselves, the source file of a document contains mark-up elements that add structure to the document. These elements include sectional divisions, figure/table captions, lists of various sorts, theorems/proofs, and so on. The list will be different for every document that can be written.

Each element can be represented logically without worrying about the formatting, with mark-up such as `\section`, `\caption`, `\begin{enumerate}` and so on. The output of each one of these document elements will be a typeset representation of the information marked up, and the visual arrangement and design of these elements can vary widely in producing a variety of desired outcomes.

For each type of document element, there may be design variations that contain the same sort of information but present it in slightly different ways. For example, the difference between a numbered and an unnumbered section, `\section` and `\section*`, or the difference between an itemized list or an enumerated list.

There are three distinct layers in the definition of "a document" at this level

1. semantic elements such as the ideas of sections and lists;
2. a set of design solutions for representing these elements visually;
3. specific variations for these designs that represent the elements in the document.

In the parlance of the template system, these are called types, templates, and instances, and they are discussed below in sections 4, 5, and 7, respectively.

3 Types, templates, and instances

By formally declaring documents to be composed of mark-up elements grouped into types, which are interpreted and typeset with a set of templates, each of which has one or more instances with which to compose each and every semantic unit of the text, we can cleanly separate the components of document construction.

All of the structures provided by the template system are global, and do not respect TEX grouping.

4 Template types

An *template type* (sometimes just "type") is an abstract idea of a document element that takes a fixed number of arguments corresponding to the information from the document author that it is representing. A sectioning type, for example, might take three inputs: "title", "short title", and "label".

Any given document class will define which types are to be used in the document, and any template of a given type can be used to generate an instance for the type. (Of course, different templates will produce different typeset representations, but the underlying content will be the same.)

`\NewTemplateType` `\NewTemplateType {⟨template type⟩} {⟨no. of args⟩}`

This function defines an ⟨*template type*⟩ taking ⟨*number of arguments*⟩, where the ⟨*type*⟩ is an abstraction as discussed above. For example,

```
\NewTemplateType{sectioning}{3}
```

creates a type "sectioning", where each use of that type will need three arguments.

5 Templates

A *template* is a generalized design solution for representing the information of a specified type. Templates that do the same thing, but in different ways, are grouped together by their type and given separate names. There are two important parts to a template:

- the parameters it takes to vary the design it is producing;

Key-type	Description of input
`boolean`	`true` or `false`
`choice{`⟨*choices*⟩`}`	A list of pre-defined ⟨*choices*⟩
`commalist`	A comma-separated list
`function{`⟨N⟩`}`	A (protected) function definition with N arguments (N from 0 to 9)
`instance{`⟨*name*⟩`}`	An instance of type ⟨*name*⟩
`integer`	An integer or integer expression
`length`	A fixed length
`muskip`	A math length with shrink and stretch components
`real`	A real (floating point) value
`skip`	A length with shrink and stretch components
`tokenlist`	A token list: any text or commands

Table 1: Key-types for defining template interfaces with `\DeclareTemplateInterface`.

- the implementation of the design.

As a document author or designer does not care about the implementation but rather only the interface to the template, these two aspects of the template definition are split into two independent declarations, `\DeclareTemplateInterface` and `\DeclareTemplateCode`.

`\DeclareTemplateInterface`

```
\DeclareTemplateInterface
  {⟨type⟩} {⟨template⟩} {⟨no. of args⟩}
  {⟨key list⟩}
```

A ⟨*template*⟩ interface is declared for a particular ⟨*type*⟩, where the ⟨*number of arguments*⟩ must agree with the type declaration. The interface itself is defined by the ⟨*key list*⟩, which is itself a key–value list taking a specialized format:

⟨*key1*⟩ ":" ⟨*key type1*⟩ ","
⟨*key2*⟩ ":" ⟨*key type2*⟩ ","
⟨*key3*⟩ ":" ⟨*key type3*⟩ "=" ⟨*default3*⟩ ","
⟨*key4*⟩ ":" ⟨*key type4*⟩ "=" ⟨*default4*⟩ ","
...

Each ⟨*key*⟩ name should consist of ASCII characters, with the exception of `,`, `=` and `:`. The recommended form for key names is to use lower case letters, with dashes to separate out different parts. Spaces are ignored in key names, so they can be included or missed out at will. Each ⟨*key*⟩ must have a ⟨*key type*⟩, which defines the type of input that the ⟨*key*⟩ requires. A full list of key types is given in Table 1. Each key may have a ⟨*default*⟩ value, which will be used in by the template if the ⟨*key*⟩ is not set explicitly. The ⟨*default*⟩ should be of the correct form to be accepted by the ⟨*key type*⟩ of the ⟨*key*⟩: this is not checked by the code. Expressions for numerical values are evaluated when the template is used, thus for example values given in terms of `em` or `ex` will be set respecting the prevailing font.

\KeyValue `\KeyValue {⟨key name⟩}`

There are occasions where the default (or value) for one key should be taken from another. The `\KeyValue` function can be used to transfer this information without needing to know the internal implementation of the key:

```
\DeclareTemplateInterface { type } { template } { no. of args }
  {
    key-name-1 : key-type = value ,
    key-name-2 : key-type = \KeyValue { key-name-1 },
    ...
  }
```

\DeclareTemplateCode `\DeclareTemplateCode {⟨type⟩} {⟨template⟩} {⟨no. of args⟩} {⟨key bindings⟩} {⟨code⟩}`

The relationship between a templates keys and the internal implementation is created using the `\DeclareTemplateCode` function. As with `\DeclareTemplateInterface`, the ⟨*template*⟩ name is given along with the ⟨*type*⟩ and ⟨*number of arguments*⟩ required. The ⟨*key bindings*⟩ argument is a key–value list which specifies the relationship between each ⟨*key*⟩ of the template interface with an underlying ⟨*variable*⟩.

⟨*key1*⟩ "=" ⟨*variable1*⟩,
⟨*key2*⟩ "=" ⟨*variable2*⟩,
⟨*key3*⟩ "=" global ⟨*variable3*⟩,
⟨*key4*⟩ "=" global ⟨*variable4*⟩,
⟨*key5*⟩ "=" name { ⟨*name5*⟩ },
...

As illustrated, the keyword `global` may be included in the listing to indicate that the ⟨*variable*⟩ should be assigned globally. The keyword `name` may also be used to give the variable by (cs)name rather than as a single token. If both keywords are used, `global` should come before `name`.

A full list of variable bindings is given in Table 2.

The ⟨*code*⟩ argument of `\DeclareTemplateCode` is used as the replacement text for the template when it is used, either directly or as an instance. This may therefore accept arguments `#1`, `#2`, *etc.* as detailed by the ⟨*number of arguments*⟩ taken by the type.

\AssignTemplateKeys `\AssignTemplateKeys`

In the final argument of `\DeclareTemplateCode` the assignment of keys defined by the template may be delayed by including the command `\AssignTemplateKeys`. If this is *not* present, keys are assigned immediately before the template code. If an `\AssignTemplateKeys` command is present, assignment is delayed until this point. Note that the command must be *directly* present in the code, not placed within a nested command/macro.

Key-type	Description of binding
`boolean`	Boolean variable, *e.g.* `\l_tmpa_bool`
`choice`	List of choice implementations (see Section 6)
`commalist`	Comma list, *e.g.* `\l_tmpa_clist`
`function`	Function taking N arguments, *e.g.* `\use_i:nn`
`instance`	
`integer`	Integer variable, *e.g.* `\l_tmpa_int`
`length`	Dimension variable, *e.g.* `\l_tmpa_dim`
`muskip`	Muskip variable, *e.g.* `\l_tmpa_muskip`
`real`	Floating-point variable, *e.g.* `\l_tmpa_fp`
`skip`	Skip variable, *e.g.* `\l_tmpa_skip`
`tokenlist`	Token list variable, *e.g.* `\l_tmpa_tl`

Table 2: Bindings required for different key types when defining template implementations with `\DeclareTemplateCode`. Apart from `choice` and `function` all of these accept the keyword `global` to carry out a global assignment. Apart from `choice`, all of these accept the keyword `name` to allow passing command name without the leading backslash: this is useful for auto-generated names.

`\SetKnownTemplateKeys`	`\SetKnownTemplateKeys {`⟨*type*⟩`} {`⟨*template*⟩`} {`⟨*keyvals*⟩`}`
`\SetTemplateKeys`	`\SetTemplateKeys {`⟨*type*⟩`} {`⟨*template*⟩`} {`⟨*keyvals*⟩`}`
`\SetCurrentKnownTemplateKeys`	`\SetCurrentKnownTemplateKeys {`⟨*keyvals*⟩`}`
`\UnusedTemplateKeys`	`\UnusedTemplateKeys % all` ⟨*keyvals*⟩ `unused by previous \SetKnownTemplateKeys`

In the final argument of `\DeclareTemplateCode` one can also overwrite (some of) the current template key value settings by using the command `\SetKnownTemplateKeys` or `\SetTemplateKeys`, i.e., they can overwrite the template default values and the values assigned by the instance.

The `\SetKnownTemplateKeys`, `\SetTemplateKeys` and `\SetCurrentKnownTemplateKeys` commands are only supported within the code of a template; using them elsewhere has unpredictable results. If they are used together with `\AssignTemplateKeys` then the latter command should come first in the template code.

The main use case for these commands is the situation where there is an argument (normally `#1`) to the template in which a key/value list can be specified that overwrites the normal settings. In that case one could use

```
\SetKnownTemplateKeys{⟨type⟩}{⟨template⟩}{#1}
```

to process this key/value list inside the template.

If `\SetKnownTemplateKeys` or `\SetCurrentKnownTemplateKeys` are executed and the ⟨*keyvals*⟩ argument contains keys not known to the ⟨*template*⟩ they are simply ignored and stored in the tokenlist `\UnusedTemplateKeys` without generating an error. This way it is possible to apply the same key/val list specified by the user on a document-level command or environment to several templates, which is useful, if the command or environment is implemented by calling several different template instances.

`\SetCurrentKnownTemplateKeys` works like `\SetKnownTemplateKeys` but obtains the name of the current ⟨*type*⟩/⟨*template*⟩ automatically based on the current context. It should be used before any other template code may be called (and thus change the value of the tracking of the current template).

As a variation of that, you can use this key/val list the first time, and for the next template instance use what remains in `\UnusedTemplateKeys` (i.e., the key/val list with only the keys that have not been processed previously). The final processing step could then be `\SetTemplateKeys`, which unconditionally attempts to set the ⟨*keyvals*⟩ received in its third argument. This command complains if any of them are unknown keys. Alternatively, you could use `\SetKnownTemplateKeys` and afterwards check whether `\UnusedTemplateKeys` is empty.[23]

For example, a list, such as `enumerate`, is made up from a `blockenv`, `block`, `list`, and a `para` template and in the single user-supplied optional argument of `enumerate` key/values for any of these templates might be specified.

In fact, in the particular example of list environments, the supplied key/value list is also saved and then applied to each `\item` which is implemented through an `item` template. This way, one can specify one-off settings for all the items of a single list (on the environment level), as well as to individual items within that list (by specifying them in the optional argument of an `\item`). With `\SetKnownTemplateKeys` and `\SetTemplateKeys` working together, it is possible to provide this flexibility and still alert the user when one of their keys is misspelled.

On the other hand you may want to allow for "misspellings" without generating an error or a warning. For example, if you define a template that accepts only a few keys, you might just want to ignore anything specified in the source when you use this template in place of a different one, without the need to alter the document source. Or you might just generate a warning message, which is easy, given that the unused key/values are available in the `\UnusedTemplateKeys` variable.

`\DeclareTemplateCopy`

```
\DeclareTemplateCopy
  {⟨type⟩} {⟨template2⟩} {⟨template1⟩}
```

Copies ⟨*template1*⟩ of ⟨*type*⟩ to a new name ⟨*template2*⟩: the copy can then be edited independent of the original.

6 Multiple choices

The `choice` key type implements multiple choice input. At the interface level, only the list of valid choices is needed:

```
\DeclareTemplateInterface { foo } { bar } { 0 }
  { key-name : choice { A, B, C } }
```

where the choices are given as a comma-list (which must therefore be wrapped in braces). A default value can also be given:

```
\DeclareTemplateInterface { foo } { bar } { 0 }
  { key-name : choice { A, B, C } = A }
```

At the implementation level, each choice is associated with code, using a nested key–value list.

[23] Using `\SetTemplateKeys` exposes the inner structure of the template keys when generating an error. This is something one may want to avoid as it can be confusing to the user, especially if several templates are involved. In that case use `\SetKnownTemplateKeys` and afterwards check whether `\UnusedTemplateKeys` is empty; if it is not empty then generate your own error message.

```
\DeclareTemplateCode { foo } { bar } { 0 }
  {
    key-name =
      {
        A = Code-A ,
        B = Code-B ,
        C = Code-C
      }
   }
   { ... }
```

The two choice lists should match, but in the implementation a special `unknown` choice is also available. This can be used to ignore values and implement an "else" branch:

```
\DeclareTemplateCode { foo } { bar } { 0 }
  {
    key-name =
      {
        A       = Code-A ,
        B       = Code-B ,
        C       = Code-C ,
        unknown = Else-code
      }
   }
   { ... }
```

The `unknown` entry must be the last one given, and should *not* be listed in the interface part of the template.

For keys which accept the values `true` and `false` both the boolean and choice key types can be used. As template interfaces are intended to prompt clarity at the design level, the boolean key type should be favored, with the choice type reserved for keys which take arbitrary values.

7 Instances

After a template is defined it still needs to be put to use. The parameters that it expects need to be defined before it can be used in a document. Every time a template has parameters given to it, an *instance* is created, and this is the code that ends up in the document to perform the typesetting of whatever pieces of information are input into it.

For example, a template might say "here is a section with or without a number that might be centered or left aligned and print its contents in a certain font of a certain size, with a bit of a gap before and after it" whereas an instance declares "this is a section with a number, which is centered and set in 12 pt italic with a 10 pt skip before and a 12 pt skip after it". Therefore, an instance is just a frozen version of a template with specific settings as chosen by the designer.

\DeclareInstance `\DeclareInstance {⟨type⟩} {⟨instance⟩} {⟨template⟩} {⟨parameters⟩}`

This function uses a ⟨*template*⟩ for an ⟨*type*⟩ to create an ⟨*instance*⟩. The ⟨*instance*⟩ will be set up using the ⟨*parameters*⟩, which will set some of the ⟨*keys*⟩ in the ⟨*template*⟩.

As a practical example, consider a type for document sections (which might include chapters, parts, sections, *etc.*), which is called `sectioning`. One possible template for this type might be called `basic`, and one instance of this template would be a numbered section. The instance declaration might read:

```
\DeclareInstance { sectioning } { section-num } { basic }
  {
    numbered      = true ,
    justification = center ,
    font          =\normalsize\itshape ,
    before-skip   = 10pt ,
    after-skip    = 12pt ,
  }
```

Of course, the key names here are entirely imaginary, but illustrate the general idea of fixing some settings.

\InstanceValue ⋆ `\InstanceValue {⟨type⟩} {⟨instance⟩} {⟨key⟩}`

Expands to the current value for the ⟨*key*⟩ stored in the ⟨*instance*⟩ of ⟨*type*⟩. If the ⟨*instance*⟩ or ⟨*key*⟩ do not exist, the expansion is empty. The result is returned within the `\unexpanded` primitive (`\exp_not:n`).

\IfInstanceExistsT `\IfInstanceExistsTF {⟨type⟩} {⟨instance⟩} {⟨true code⟩} {⟨false code⟩}`
\IfInstanceExistsF
\IfInstanceExistsTF

Tests if the named ⟨*instance*⟩ of a ⟨*type*⟩ exists, and then inserts the appropriate code into the input stream.

\DeclareInstanceCopy `\DeclareInstanceCopy {⟨type⟩} {⟨instance2⟩} {⟨instance1⟩}`

Copies the ⟨*values*⟩ for ⟨*instance1*⟩ for an ⟨*type*⟩ to ⟨*instance2*⟩.

8 Document interface

After the instances have been chosen, document commands must be declared to use those instances in the document. `\UseInstance` calls instances directly, and this command should be used internally in document-level mark-up.

\UseInstance `\UseInstance {⟨type⟩} {⟨instance⟩} ⟨arguments⟩`

Uses an ⟨*instance*⟩ of the ⟨*type*⟩, which will require ⟨*arguments*⟩ as determined by the number specified for the ⟨*type*⟩. The ⟨*instance*⟩ must have been declared before it can be used, otherwise an error is raised.

\UseTemplate `\UseTemplate {⟨type⟩} {⟨template⟩} {⟨settings⟩} ⟨arguments⟩`

Uses the ⟨*template*⟩ of the specified ⟨*type*⟩, applying the ⟨*settings*⟩ and absorbing ⟨*arguments*⟩ as detailed by the ⟨*type*⟩ declaration. This in effect is the same as creating an instance using `\DeclareInstance` and immediately using it with `\UseInstance`, but without the instance having any further existence. This command is therefore useful when a template needs to be used only once.

This function can also be used as the argument to `instance` key types:

```
\DeclareInstance { type } { template } { instance }
  {
    instance-key =
      \UseTemplate { type2 } { template2 } { <settings> }
  }
```

9 Changing existing definitions

Template parameters may be assigned specific defaults for instances to use if the instance declaration doesn't explicit set those parameters. In some cases, the document designer will wish to edit these defaults to allow them to "cascade" to the instances. The alternative would be to set each parameter identically for each instance declaration, a tedious and error-prone process.

\EditTemplateDefaults `\EditTemplateDefaults {⟨type⟩} {⟨template⟩} {⟨new defaults⟩}`

Edits the ⟨*defaults*⟩ for a ⟨*template*⟩ for an ⟨*type*⟩. The ⟨*new defaults*⟩, given as a key–value list, replace the existing defaults for the ⟨*template*⟩. This means that the change will apply to instances declared after the editing, but that instances which have already been created are unaffected.

\EditInstance `\EditInstance {⟨type⟩} {⟨instance⟩} {⟨new values⟩}`

Edits the ⟨*values*⟩ for an ⟨*instance*⟩ for an ⟨*type*⟩. The ⟨*new values*⟩, given as a key–value list, replace the existing values for the ⟨*instance*⟩. This function is complementary to `\EditTemplateDefaults`: `\EditInstance` changes a single instance while leaving the template untouched.

9.1 Expanding the values of keys

To allow the user to apply expansion of values when the key is set, key names can be followed by an expansion specifier. This is given by appending `:` and a single letter specifier to the key name. These letters are the normal argument specifiers for expl3, thus they may be one of `n` (redundant but supported), `o`, `V`, `v`, `e`, `N` (again redundant) or `c`. Expansion of a control sequence name is particularly useful when you need to refer to an internal LaTeX 2_ε or an L3 programming layer variable, e.g.,

```
key-a:c =  @itemdepth , % use \@itemdepth as the value
key-b:v =  @itemdepth   % use the current value of \@itempdepth as the value
```

10 Creating equivalent keys

There are times when it is useful to have different names which map to the same underlying key interfaces, for example where a more complex higher level name is implemented by several simpler underlying keys, or where a key name needs to be changed. To support this, it is possible to define equivalent keys using `\DeclareTemplateInterface`.

`\DeclareTemplateInterfaceEquivalents` `\DeclareTemplateInterfaceEquivalents {⟨type⟩} {⟨template⟩} [⟨deprecation-date⟩] {⟨key list⟩}`

Defines one or more keys to act as equivalents to the base keys of the ⟨*template*⟩. As for `\DeclareTemplateInterface`, these take a specialized format:

⟨*key1*⟩ ":" ⟨*key type1*⟩ "=" ⟨*definition1*⟩ ","
⟨*key2*⟩ ":" ⟨*key type2*⟩ "=" ⟨*definition2*⟩ ","
...

The ⟨*key type*⟩ of the key being created should be one of

- `alias`: A direct equivalence between the key being created and a single parent
- `meta`: An equivalence to one or more parent keys, where additional code may be included as well as the value passed to the key being created; the value (provided in the instance declaration or in the ad hoc setting in the document) is available as `#1`.

for example

```
\DeclareTemplateInterfaceEquivalents { type } { template }
  {
    alias-key-one : alias = base-key-one ,
    alias-key-two : meta =
      { parent-key-two = {#1}, base-key-three = \hspace{#1} }
  }
```

Equivalent key names may be reused to overwrite existing equivalent keys but cannot be the same as base keys of the template.

Optionally, the ⟨*deprecation date*⟩ may be set. Before this date, a warning will be printed concerning the keys; after this date, an error will be issued. In both cases, the keys will still be defined.

10.1 Modifying existing key values

Some changes to key values are relative adjustments to the previous value, for example to add 1 cm to a margin. To support this, the syntax

```
key-a += 1cm
```

can be usd, with a `+` immediately before the `=` sign. This will cause the new value to be *added* to the existing one. This naturally applies to all numeric key types, and for `tokenlist` and `commalist` means adding the new material to the end of the existing value. An error will arise if this is used for other key types, for example `function`.

11 Getting information about templates and instances

\ShowInstanceValues `\ShowInstanceValues {⟨type⟩} {⟨instance⟩}`

Shows the ⟨*values*⟩ for an ⟨*instance*⟩ of the given ⟨*type*⟩ at the terminal.

\ShowTemplateCode `\ShowTemplateCode {⟨type⟩} {⟨template⟩}`

Shows the ⟨*code*⟩ of a ⟨*template*⟩ for an ⟨*type*⟩ in the terminal.

\ShowTemplateDefaults `\ShowTemplateDefaults {⟨type⟩} {⟨template⟩}`

Shows the ⟨*default*⟩ values of a ⟨*template*⟩ for an ⟨*type*⟩ in the terminal.

\ShowTemplateInterface `\ShowTemplateInterface {⟨type⟩} {⟨template⟩}`

Shows the ⟨*keys*⟩ and associated ⟨*key types*⟩ of a ⟨*template*⟩ for an ⟨*type*⟩ in the terminal.

\ShowTemplateVariables `\ShowTemplateVariables {⟨type⟩} {⟨template⟩}`

Shows the ⟨*variables*⟩ and associated ⟨*keys*⟩ of a ⟨*template*⟩ for an ⟨*type*⟩ in the terminal. Note that `code` and `choice` keys do not map directly to variables but to arbitrary code. For `choice` keys, each valid choice is shown as a separate entry in the list, with the key name and choice separated by a space, for example

```
Template 'example' of type 'example' has variable mapping:
>  demo unknown  =>  \def \demo {?}
>  demo c  =>  \def \demo {c}
>  demo b  =>  \def \demo {b}
>  demo a  =>  \def \demo {a}.
```

would be shown for a choice key `demo` with valid choices `a`, `b` and `c`, plus code for an `unknown` branch.

12 Debugging support

\DebugTemplatesOn `\DebugTemplatesOn`
\DebugTemplatesOff

Turn on or off debugging support for use of templates. Debugging information is provided at the point of *use* of templates and instances, i.e. where normal messages should be avoided for performance reasons.

13 The implementation

```
⟨@@=template⟩
⟨*2ekernel⟩
\message{templates,}
⟨/2ekernel⟩
```

```
⟨*2ekernel | latexrelease⟩
\ExplSyntaxOn
⟨latexrelease⟩\NewModuleRelease{2024/06/01}{lttemplates}
⟨latexrelease⟩                    {Prototype~document~commands}%
```

13.1 Extensions for **expl3**

__template_date_compare:nNnTF
__template_date_compare_aux:nNn
__template_date_compare_auxi:nnnNnnn
__template_date_compare_auxii:w
__template_date_compare_auxiii:w

A slight refinement of code from l3doc, likely should move to the core. Note that like other `compare` functions, the arguments are always fully expanded.

```
\prg_new_conditional:Npnn \__template_date_compare:nNn #1#2#3 { TF }
  { \use:e { \exp_not:N \__template_date_compare_aux:nNn {#1} #2 {#3} } }
\cs_new:Npn \__template_date_compare_aux:nNn #1#2#3
  {
    \use:e
      {
        \exp_not:N \__template_date_compare_auxi:nnnNnnn
          \__template_date_compare_auxii:w #1 - - - \q_stop
          #2
          \__template_date_compare_auxii:w #3 - - - \q_stop
      }
  }
\cs_new:Npn \__template_date_compare_auxi:nnnNnnn #1#2#3#4#5#6#7
  {
    \int_compare:nNnTF {#1} = {#5}
      {
        \int_compare:nNnTF {#2} = {#6}
          {
            \int_compare:nNnTF {#3} #4 {#7}
              { \prg_return_true: } { \prg_return_false: }
          }
          {
            \int_compare:nNnTF {#2} #4 {#6}
              { \prg_return_true: } { \prg_return_false: }
          }
      }
      {
        \int_compare:nNnTF {#1} #4 {#5}
          { \prg_return_true: } { \prg_return_false: }
      }
  }
\cs_new:Npn \__template_date_compare_auxii:w #1 - #2 - #3 - #4 \q_stop
  {
    \tl_if_blank:nTF {#2#3}
      { \__template_date_compare_auxiii:w #1 / / / \q_stop }
      { { 0 #1 } { 0 #2 } { 0 #3 } }
  }
\cs_new:Npn \__template_date_compare_auxiii:w #1 / #2 / #3 / #4 \q_stop
  { { 0 #1 } { 0 #2 } { 0 #3 } }
```

(*End of definition for* `\__template_date_compare:nNnTF` *and others.*)

13.2 Variables and constants

```
\c__template_code_root_tl
\c__template_defaults_root_tl
\c__template_instances_root_tl
\c__template_keytypes_root_tl
\c__template_key_order_root_tl
\c__template_restrict_root_tl
\c__template_values_root_tl
\c__template_vars_root_tl
```

`\c__template_msg_root_tl` So that literal values are kept to a minimum.

```
\tl_const:Nn \c__template_code_root_tl      { template~code~>~ }
\tl_const:Nn \c__template_defaults_root_tl  { template~defaults~>~ }
\tl_const:Nn \c__template_instances_root_tl { template~instance~>~  }
\tl_const:Nn \c__template_keytypes_root_tl  { template~key~types~>~ }
\tl_const:Nn \c__template_key_order_root_tl { template~key~order~>~ }
\tl_const:Nn \c__template_msg_root_tl       { template~message~>~ }
\tl_const:Nn \c__template_values_root_tl    { template~values~>~ }
\tl_const:Nn \c__template_vars_root_tl      { template~vars~>~ }
```

`\c__template_keytypes_seq` A list of all keytypes.

```
\exp_args:NNe \seq_const_from_clist:Nn \c__template_keytypes_seq
  {
    \tl_to_str:n
      {
        boolean   ,
        choice    ,
        commalist ,
        function  ,
        instance  ,
        integer   ,
        length    ,
        muskip    ,
        real      ,
        skip      ,
        tokenlist
      }
  }
```

`\c__template_keytypes_arg_seq`

A list of keytypes which also need additional data (an argument), used to parse the keytype correctly.

```
\seq_const_from_clist:Nn \c__template_keytypes_arg_seq
  { choice , function , instance }
```

`\g__template_type_prop` For storing types and the associated number of arguments.

```
\prop_new:N \g__template_type_prop
```

\l__template_assignments_tl

When creating an instance, the assigned values are collected here.

```
\tl_new:N \l__template_assignments_tl
```

\l__template_default_tl The default value for a key is recovered here from the property list in which it is stored.

```
\tl_new:N \l__template_default_tl
```

\l__template_error_bool A flag for errors to be carried forward.

```
\bool_new:N \l__template_error_bool
```

\l__template_global_bool Used to indicate that assignments should be global.

```
\bool_new:N \l__template_global_bool
```

\l__template_key_name_tl
\l__template_keytype_tl
\l__template_keytype_arg_tl
\l__template_value_tl
\l__template_var_tl

When defining each key in a template, the name and type of the key need to be separated and stored. Any argument needed by the keytype is also stored separately.

```
\tl_new:N \l__template_key_name_tl
\tl_new:N \l__template_keytype_tl
\tl_new:N \l__template_keytype_arg_tl
\tl_new:N \l__template_value_tl
\tl_new:N \l__template_var_tl
```

\l__template_value_exp_str

```
\str_new:N \l__template_value_exp_str
```

\l__template_keytypes_prop
\l__template_key_order_seq
\l__template_values_prop
\l__template_value_order_seq
\l__template_vars_prop

To avoid needing too many difficult-to-follow csname assignments, various scratch token registers are used to build up data, which is then transferred

```
\prop_new:N \l__template_keytypes_prop
\seq_new:N \l__template_key_order_seq
\prop_new:N \l__template_values_prop
\seq_new:N \l__template_value_order_seq
\prop_new:N \l__template_vars_prop
```

\l__template_msg_prop

```
\prop_new:N \l__template_msg_prop
```

\l__template_tmp_clist
\l__template_tmp_dim
\l__template_tmp_int
\l__template_tmp_muskip
\l__template_tmp_skip
\l__template_tmp_tl

Scratch space.

```
\clist_new:N \l__template_tmp_clist
\dim_new:N \l__template_tmp_dim
\int_new:N \l__template_tmp_int
\muskip_new:N \l__template_tmp_muskip
\skip_new:N \l__template_tmp_skip
\tl_new:N \l__template_tmp_tl
```

\l__template_current_type_tl
\l__template_current_template_tl

For setting the current template values.

```
\tl_new:N \l__template_current_type_tl
\tl_new:N \l__template_current_template_tl
```

__template_tmp:n

Scratch space.

```
\cs_new:Npn \__template_tmp:n #1 { }
```

(*End of definition for* `\__template_tmp:n`.)

\s__template_mark
\s__template_stop

Internal scan marks.

```
\scan_new:N \s__template_mark
\scan_new:N \s__template_stop
```

\q__template_nil

Internal quarks.

```
\quark_new:N \q__template_nil
```

__template_quark_if_nil_p:n
__template_quark_if_nil:nTF

Branching quark conditional.

```
\__kernel_quark_new_conditional:Nn \__template_quark_if_nil:N { F }
```

(*End of definition for* `\__template_quark_if_nil:nTF`.)

13.3 Debugging support

__template_debug_on:
__template_debug_off:

```
\cs_new_protected:Npn \__template_debug_on:
  {
    \cs_set_protected:Npn \__template_debug:n ##1 {##1}
    \cs_set_protected:Npn \__template_debug_typeout:n ##1
      { \iow_term:e { [ Template ] ~ ==> ~ ##1 } }
  }
\cs_new_protected:Npn \__template_debug_off:
  {
    \cs_set_protected:Npn \__template_debug:n ##1 { }
    \cs_set_protected:Npn \__template_debug_typeout:n ##1 { }
  }
```

(*End of definition for* `\__template_debug_on:` *and* `\__template_debug_off:`.)

__template_debug:n
__template_debug_typeout:n

```
\cs_new_protected:Npn \__template_debug:n #1 { }
\cs_new_protected:Npn \__template_debug_typeout:n #1 { }
```

(*End of definition for* `\__template_debug:n` *and* `\__template_debug_typeout:n`.)

13.4 Testing existence and validity

There are a number of checks needed for either the existence of a type, template or instance. There are also some for the validity of a particular call. All of these are collected up here.

__template_execute_if_arg_agree:nnT

A test agreement between the number of arguments for the template type and that specified when creating a template. This is not done as a separate conditional for efficiency and better error message

```
\cs_new_protected:Npn \__template_execute_if_arg_agree:nnT #1#2#3
  {
    \prop_get:NnN \g__template_type_prop {#1} \l__template_tmp_tl
    \int_compare:nNnTF {#2} = \l__template_tmp_tl
       {#3}
       {
         \msg_error:nneee { template } { argument-number-mismatch }
           {#1} { \l__template_tmp_tl } {#2}
       }
  }
```

(*End of definition for* `\__template_execute_if_arg_agree:nnT`.)

__template_execute_if_code_exist:nnT

A template is only fully declared if the code has been set up, which can be checked by looking for the template function itself.

```
\cs_new_protected:Npn \__template_execute_if_code_exist:nnT #1#2#3
  {
    \cs_if_exist:cTF { \c__template_code_root_tl #1 / #2 }
      {#3}
      { \msg_error:nnnn { template } { no-template-code } {#1} {#2} }
  }
```

(*End of definition for* `\__template_execute_if_code_exist:nnT`.)

__template_execute_if_keytype_exist:nT
__template_execute_if_keytype_exist:VT

The test for valid keytypes looks for a function to set up the key, which is part of the "code" side of the template definition. This avoids having different lists for the two parts of the process.

```
\cs_new_protected:Npn \__template_execute_if_keytype_exist:nT #1#2
  {
    \seq_if_in:NeTF \c__template_keytypes_seq { \tl_to_str:n {#1} }
      {#2}
      { \msg_error:nnn { template } { unknown-keytype } {#1} }
  }
\cs_generate_variant:Nn \__template_execute_if_keytype_exist:nT { V }
```

(*End of definition for* `\__template_execute_if_keytype_exist:nT`.)

__template_execute_if_type_exist:nT To check that a particular type is valid.

```
\cs_new_protected:Npn \__template_execute_if_type_exist:nT #1#2
  {
    \prop_if_in:NnTF \g__template_type_prop {#1}
      {#2}
      { \msg_error:nnn { template } { unknown-type } {#1} }
  }
```

(*End of definition for* `\__template_execute_if_type_exist:nT`.)

__template_execute_if_keyname_free:nT
__template_execute_if_keyname_free:VT

```
\cs_new_protected:Npn \__template_execute_if_keyname_free:nT #1#2
  {
    \seq_if_in:NnTF \l__template_key_order_seq {#1}
      { \msg_error:nnn { template } { duplicate-key-interface } {#1} }
      {#2}
   }
\cs_generate_variant:Nn \__template_execute_if_keyname_free:nT { V }
```

(*End of definition for* `\__template_execute_if_keyname_free:nT`.)

__template_execute_if_keys_exist:nnT To check that the keys for a template have been set up before trying to create any code, a simple check for the correctly-named keytype property list.

```
\cs_new_protected:Npn \__template_if_keys_exist:nnT #1#2#3
  {
    \cs_if_exist:cTF { \c__template_keytypes_root_tl #1 / #2 }
      {#3}
      { \msg_error:nnnn { template } { unknown-template } {#1} {#2} }
   }
```

(*End of definition for* `\__template_execute_if_keys_exist:nnT`.)

__template_if_key_value:n*TF*
__template_if_key_value:V*TF*

Tests for the first token in a string being `\KeyValue`.

```
\prg_new_conditional:Npnn \__template_if_key_value:n #1 { T , F , TF }
  {
    \str_if_eq:noTF { \KeyValue } { \tl_head:w #1 \q_nil \q_stop }
      \prg_return_true:
      \prg_return_false:
  }
\prg_generate_conditional_variant:Nnn \__template_if_key_value:n { V } { T , F , TF }
```

(*End of definition for* `\__template_if_key_value:nTF`.)

__template_if_instance_exist:nn*TF* Testing for an instance

```
\prg_new_conditional:Npnn \__template_if_instance_exist:nn #1#2 { T, F, TF }
  {
    \cs_if_exist:cTF { \c__template_instances_root_tl #1 / #2 }
      \prg_return_true:
      \prg_return_false:
 }
```

(*End of definition for* `\__template_if_instance_exist:nnTF`.)

__template_if_use_template:nTF Tests for the first token in a string being \UseTemplate.

```
\prg_new_conditional:Npnn \__template_if_use_template:n #1 { TF }
  {
    \str_if_eq:noTF { \UseTemplate } { \tl_head:w #1 \q_nil \q_stop }
      \prg_return_true:
      \prg_return_false:
}
```

(*End of definition for* `\__template_if_use_template:nTF`.)

13.5 Saving and recovering property lists

The various property lists for templates have to be shuffled in and out of storage.

__template_store_defaults:nn
__template_store_keytypes:nn
__template_store_values:nn
__template_store_vars:nn

The defaults and keytypes are transferred from the scratch property lists to the "proper" lists for the template being created.

```
\cs_new_protected:Npn \__template_store_defaults:nn #1#2
  {
    \debug_suspend:
    \prop_gclear_new:c { \c__template_defaults_root_tl #1 / #2  }
    \prop_gset_eq:cN { \c__template_defaults_root_tl #1 / #2 }
      \l__template_values_prop
    \debug_resume:
  }
\cs_new_protected:Npn \__template_store_keytypes:nn #1#2
  {
    \debug_suspend:
    \prop_if_exist:cTF { \c__template_keytypes_root_tl #1 / #2 }
      {
        \msg_info:nnnn { template } { declare-template-interface } {#1} {#2}
        \prop_gclear:c { \c__template_keytypes_root_tl #1 / #2 }
      }
      { \prop_new:c { \c__template_keytypes_root_tl #1 / #2 } }
    \prop_gset_eq:cN { \c__template_keytypes_root_tl #1 / #2 }
      \l__template_keytypes_prop
    \seq_gclear_new:c { \c__template_key_order_root_tl #1 / #2 }
    \seq_gset_eq:cN { \c__template_key_order_root_tl #1 / #2 }
      \l__template_key_order_seq
    \debug_resume:
  }
\cs_new_protected:Npn \__template_store_values:nn #1#2
  {
    \debug_suspend:
    \prop_clear_new:c { \c__template_values_root_tl #1 / #2 }
    \prop_set_eq:cN { \c__template_values_root_tl #1 / #2 }
      \l__template_values_prop
    \debug_resume:
  }
\cs_new_protected:Npn \__template_store_vars:nn #1#2
  {
   \debug_suspend:
    \prop_gclear_new:c { \c__template_msg_root_tl #1 / #2 }
    \prop_gset_eq:cN { \c__template_msg_root_tl #1 / #2 }
      \l__template_msg_prop
```

```
    \prop_gclear_new:c { \c__template_vars_root_tl #1 / #2 }
    \prop_gset_eq:cN { \c__template_vars_root_tl #1 / #2 }
      \l__template_vars_prop
    \debug_resume:
  }
```

(*End of definition for* `\__template_store_defaults:nn` *and others.*)

`\__template_recover_defaults:nn`
`\__template_recover_keytypes:nn`
`\__template_recover_values:nn`
`\__template_recover_vars:nn`

Recovering the stored data for a template is rather less complex than storing it. All that happens is the data is transferred from the permanent to the scratch storage. However, we need to check the scratch storage does exist.

```
\cs_new_protected:Npn \__template_recover_defaults:nn #1#2
  {
    \prop_if_exist:cTF
      { \c__template_defaults_root_tl #1 / #2 }
      {
        \prop_set_eq:Nc \l__template_values_prop
          { \c__template_defaults_root_tl #1 / #2 }
      }
      { \prop_clear:N \l__template_values_prop }
  }
\cs_new_protected:Npn \__template_recover_keytypes:nn #1#2
  {
    \prop_if_exist:cTF
      { \c__template_keytypes_root_tl #1 / #2 }
      {
        \prop_set_eq:Nc \l__template_keytypes_prop
          { \c__template_keytypes_root_tl #1 / #2 }
      }
      { \prop_clear:N \l__template_keytypes_prop }
    \seq_if_exist:cTF { \c__template_key_order_root_tl #1 / #2 }
      {
        \seq_set_eq:Nc \l__template_key_order_seq
          { \c__template_key_order_root_tl #1 / #2 }
      }
      { \seq_clear:N \l__template_key_order_seq }
  }
\cs_new_protected:Npn \__template_recover_values:nn #1#2
  {
    \prop_if_exist:cTF
      { \c__template_values_root_tl #1 / #2 }
      {
        \prop_set_eq:Nc \l__template_values_prop
          { \c__template_values_root_tl #1 / #2 }
      }
      { \prop_clear:N \l__template_values_prop }
  }
\cs_new_protected:Npn \__template_recover_vars:nn #1#2
  {
    \prop_if_exist:cTF
      { \c__template_msg_root_tl #1 / #2 }
      {
        \prop_set_eq:Nc \l__template_msg_prop
          { \c__template_msg_root_tl #1 / #2 }
```

```
      }
      { \prop_clear:N \l__template_msg_prop }
    \prop_if_exist:cTF
      { \c__template_vars_root_tl #1 / #2 }
      {
        \prop_set_eq:Nc \l__template_vars_prop
          { \c__template_vars_root_tl #1 / #2 }
      }
      { \prop_clear:N \l__template_vars_prop }
  }
```

(*End of definition for* `\__template_recover_defaults:nn` *and others.*)

13.6 Creating new template types

`\__template_define_type:nn`
`\__template_declare_type:nn`

Although the type is the "top level" of the template system, it is actually very easy to implement. All that happens is that the number of arguments required is recorded, indexed by the name of the type.

```
\cs_new_protected:Npn \__template_define_type:nn #1#2
  {
    \prop_if_in:NnTF \g__template_type_prop {#1}
      { \msg_error:nnn { template } { type-already-defined } {#1} }
      { \__template_declare_type:nn {#1} {#2} }
  }
\cs_new_protected:Npn \__template_declare_type:nn #1#2
  {
    \int_set:Nn \l__template_tmp_int {#2}
    \int_compare:nTF { 0 <= \l__template_tmp_int <= 9 }
      {
        \msg_info:nnnV { template } { declare-type }
          {#1} \l__template_tmp_int
        \prop_gput:NnV \g__template_type_prop {#1}
          \l__template_tmp_int
      }
      {
        \msg_error:nnnV { template } { bad-number-of-arguments }
          {#1} \l__template_tmp_int
      }
  }
```

(*End of definition for* `\__template_define_type:nn` *and* `\__template_declare_type:nn`.)

13.7 Design part of template declaration

The "design" part of a template declaration defines the general behaviour of each key, and possibly a default value. However, it does not include the implementation. This means that what happens here is the two properties are saved to appropriate lists, which can then be used later to recover the information when implementing the keys.

`\__template_declare_template_keys:nnnn`

The main function for the "design" part of creating a template starts by checking that the type exists and that the number of arguments required agree. If that is all fine, then the two storage areas for defaults and keytypes are initialized. The mechanism is then

set up for the l3keys module to actually parse the keys. Finally, the code hands of to the storage routine to save the parsed information properly.

```
\cs_new_protected:Npn \__template_declare_template_keys:nnnn #1#2#3#4
  {
    \__template_execute_if_type_exist:nT {#1}
      {
        \__template_execute_if_arg_agree:nnT {#1} {#3}
          {
            \prop_clear:N \l__template_values_prop
            \prop_clear:N \l__template_keytypes_prop
            \prop_clear:N \l__template_vars_prop
            \seq_clear:N \l__template_key_order_seq
            \keyval_parse:NNn
              \__template_parse_keys_elt:n \__template_parse_keys_elt:nn {#4}
            \__template_store_defaults:nn {#1} {#2}
            \__template_store_keytypes:nn {#1} {#2}
            \__template_store_vars:nn {#1} {#2}
          }
      }
  }
```

(*End of definition for* `\__template_declare_template_keys:nnnn`.)

`\__template_parse_keys_elt:n`
`\__template_parse_keys_elt_aux:n`
`\__template_parse_keys_elt_aux:`

Processing the key part of the key–value pair is always carried out using this function, even if a value was found. First, the key name is separated from the keytype, and if necessary the keytype is separated into two parts. This information is then used to check that the keytype is valid, before storing the keytype (plus argument if necessary) as a property of the key name. The key name is also stored (in braces) in the token list to record the order the keys are defined in.

```
\cs_new_protected:Npn \__template_parse_keys_elt:n #1
  {
    \__template_split_keytype:n {#1}
    \bool_if:NF \l__template_error_bool
      {
        \__template_execute_if_keytype_exist:VT \l__template_keytype_tl
          {
            \seq_map_function:NN \c__template_keytypes_arg_seq
              \__template_parse_keys_elt_aux:n
            \bool_if:NF \l__template_error_bool
              {
                \__template_execute_if_keyname_free:VT
                  \l__template_key_name_tl
                  { \__template_parse_keys_elt_aux: }
              }
          }
      }
  }
\cs_new_protected:Npn \__template_parse_keys_elt_aux:n #1
  {
    \str_if_eq:VnT \l__template_keytype_tl {#1}
      {
        \tl_if_empty:NT \l__template_keytype_arg_tl
          {
            \msg_error:nnn { template } { keytype-requires-argument } {#1}
```

```
            \bool_set_true:N \l__template_error_bool
            \seq_map_break:
          }
      }
  }
\cs_new_protected:Npn \__template_parse_keys_elt_aux:
  {
    \tl_set:Ne \l__template_tmp_tl
      {
        \l__template_keytype_tl
        \tl_if_empty:NF \l__template_keytype_arg_tl
          { { \l__template_keytype_arg_tl } }
      }
    \prop_put:NVV \l__template_keytypes_prop \l__template_key_name_tl
      \l__template_tmp_tl
    \seq_put_right:NV \l__template_key_order_seq \l__template_key_name_tl
    \str_if_eq:VnT \l__template_keytype_tl { choice }
      {
        \clist_if_in:NnT \l__template_keytype_arg_tl { unknown }
          { \msg_error:nn { template } { choice-unknown-reserved } }
      }
  }
```

(*End of definition for* `\__template_parse_keys_elt:n`, `\__template_parse_keys_elt_aux:n`, *and* `\__template_parse_keys_elt_aux:`.)

`\__template_parse_keys_elt:nn` For keys which have a default, the keytype and key name are first separated out by the `\__template_parse_keys_elt:n` routine, before storing the default value in the scratch property list.

```
\cs_new_protected:Npn \__template_parse_keys_elt:nn #1#2
  {
    \__template_parse_keys_elt:n {#1}
    \prop_put:Non \l__template_values_prop \l__template_key_name_tl {#2}
  }
```

(*End of definition for* `\__template_parse_keys_elt:nn`.)

`\__template_split_keytype:n` `\__template_split_keytype_aux:w` The keytype and key name should be separated by `:`. As the definition might be given inside or outside of a code block, the category code of colons is standardized. After that, the standard delimited argument method is used to separate the two parts.

```
\cs_new_protected:Npe \__template_split_keytype:n #1
  {
    \exp_not:N \bool_set_false:N \exp_not:N \l__template_error_bool
    \tl_set:Nn \exp_not:N \l__template_tmp_tl {#1}
    \tl_replace_all:Nnn \exp_not:N \l__template_tmp_tl { : } { \token_to_str:N : }
    \tl_if_in:VnTF \exp_not:N \l__template_tmp_tl { \token_to_str:N : }
      {
        \exp_not:n
          {
            \tl_clear:N \l__template_key_name_tl
            \exp_after:wN \__template_split_keytype_aux:w
              \l__template_tmp_tl \s__template_stop
          }
      }
```

```
      {
        \exp_not:N \bool_set_true:N \exp_not:N \l__template_error_bool
        \msg_error:nnn { template } { missing-keytype } {#1}
      }
  }
\use:e
  {
    \cs_new_protected:Npn \exp_not:N \__template_split_keytype_aux:w
      #1 \token_to_str:N : #2 \s__template_stop
      {
        \tl_put_right:Ne \exp_not:N \l__template_key_name_tl
          {
            \exp_not:N \tl_trim_spaces:e
              { \exp_not:N \tl_to_str:n {#1} }
          }
        \tl_if_in:nnTF {#2} { \token_to_str:N : }
          {
            \tl_put_right:Nn \exp_not:N \l__template_key_name_tl
              { \token_to_str:N : }
            \exp_not:N \__template_split_keytype_aux:w #2 \s__template_stop
          }
          {
            \exp_not:N \tl_if_empty:NTF \exp_not:N \l__template_key_name_tl
              {
                \msg_error:nnn { template } { empty-key-name }
                  { \token_to_str:N : #2 }
              }
              { \exp_not:N \__template_split_keytype_arg:n {#2} }
          }
      }
  }
```

(*End of definition for* `\__template_split_keytype:n` *and* `\__template_split_keytype_aux:w`.)

`\__template_split_keytype_arg:n`
`\__template_split_keytype_arg:V`
`\__template_split_keytype_arg_aux:n`
`\__template_split_keytype_arg_aux:w`

The second stage of sorting out the keytype is to check for an argument. As there is no convenient delimiting token to look for, a check is made instead for each possible text value for the keytype. To keep things faster, this only involves the keytypes that need an argument. If a match is made, then a check is also needed to see that it is at the start of the keytype information. All being well, the split can then be applied. Any non-matching keytypes are assumed to be "correct" as given, and are left alone (this is checked by other code).

```
\cs_new_protected:Npn \__template_split_keytype_arg:n #1
  {
    \tl_set:Ne \l__template_keytype_tl { \tl_trim_spaces:n {#1} }
    \tl_clear:N \l__template_keytype_arg_tl
    \cs_set_protected:Npn \__template_split_keytype_arg_aux:n ##1
      {
        \tl_if_in:nnT {#1} {##1}
          {
            \cs_set:Npn \__template_split_keytype_arg_aux:w
              ####1 ##1 ####2 \s__template_stop
              {
                \tl_if_blank:nT {####1}
                  {
```

```
                    \tl_set:Ne \l__template_keytype_tl
                      { \tl_trim_spaces:n {##1} }
                    \tl_if_blank:nF {####2}
                      {
                        \tl_set:Ne \l__template_keytype_arg_tl
                          { \use:n ####2 }
                      }
                    \seq_map_break:
                  }
              }
            \__template_split_keytype_arg_aux:w #1 \s__template_stop
          }
      }
    \seq_map_function:NN \c__template_keytypes_arg_seq
      \__template_split_keytype_arg_aux:n
  }
\cs_generate_variant:Nn \__template_split_keytype_arg:n { V }
\cs_new:Npn \__template_split_keytype_arg_aux:n #1 { }
\cs_new:Npn \__template_split_keytype_arg_aux:w #1 \s__template_stop { }
```

(*End of definition for* `\__template_split_keytype_arg:n`, `\__template_split_keytype_arg_aux:n`, *and* `\__template_split_keytype_arg_aux:w`.)

13.8 Equivalent interfaces

`\__template_declare_template_equiv:nnnn`

```
\cs_new_protected:Npn \__template_declare_template_equiv:nnnn #1#2#3#4
  {
    \__template_execute_if_type_exist:nT {#1}
      {
        \__template_if_keys_exist:nnT {#1} {#2}
          {
            \__template_recover_keytypes:nn {#1} {#2}
            \__template_recover_vars:nn {#1} {#2}
            \keyval_parse:nnn
              { \__template_parse_equiv_elt:n }
              { \__template_parse_equiv_elt:nnnn { #1 / #2 } {#3} }
              {#4}
            \__template_store_keytypes:nn {#1} {#2}
            \__template_store_vars:nn {#1} {#2}
          }
      }
  }
```

(*End of definition for* `\__template_declare_template_equiv:nnnn`.)

`\__template_parse_equiv_elt:n` We must have key(s) to map to.

```
\cs_new_protected:Npn \__template_parse_equiv_elt:n #1
  { \msg_error:nnn { template } { key-no-equiv } {#1} }
```

(*End of definition for* `\__template_parse_equiv_elt:n`.)

`\__template_parse_equiv_elt:nnnn`
`\__template_parse_equiv_elt_aux:nnn`
`\__template_parse_equiv_alias:nnnn`
`\__template_parse_equiv_meta:nnnn`
`\__template_parse_equiv_auxi:nnnn`
`\__template_parse_equiv_auxii:n`
`\__template_parse_equiv_auxiii:nnnn`

Most of the work is deferred here: the only thing done is to sanity-check the key names to avoid duplication.

```
\cs_new_protected:Npn \__template_parse_equiv_elt:nnnn #1#2#3#4
  {
    \__template_split_keytype:n {#3}
    \bool_if:NF \l__template_error_bool
      {
        \prop_get:NVNTF \l__template_keytypes_prop
          \l__template_key_name_tl \l__template_tmp_tl
          {
            \seq_if_in:NeTF \c__template_keytypes_seq
              { \tl_to_str:N \l__template_tmp_tl }
              {
                \msg_error:nnV { template } { equiv-key-interface }
                  \l__template_key_name_tl
              }
              { \__template_parse_equiv_elt_aux:nnn {#1} {#2} {#4} }
          }
          { \__template_parse_equiv_elt_aux:nnn {#1} {#2} {#4} }
      }
  }
\cs_new_protected:Npn \__template_parse_equiv_elt_aux:nnn #1#2#3
  {
    \cs_if_exist_use:cF
      { __template_parse_equiv_ \l__template_keytype_tl :nnn }
      {
        \msg_error:nnV { template } { unknown-equiv-keytype }
          \l__template_keytype_tl
        \use_none:nnn
      }
        {#1} {#2} {#3}
  }
```

As we only need the key name here, we use an inline mapping and the same code for aliases and meta keys. Note the need to detokenize key names for the sequence check.

```
\cs_new_protected:Npn \__template_parse_equiv_alias:nnn #1#2#3
  { \__template_parse_equiv_auxi:nnnn {#1} {#2} { #3 = {##1} } {#3} }
\cs_new_protected:Npn \__template_parse_equiv_meta:nnn #1#2#3
  { \__template_parse_equiv_auxi:nnnn {#1} {#2} {#3} {#3} }
\cs_new_protected:Npn \__template_parse_equiv_auxi:nnnn #1#2#3#4
  {
    \keyval_map_inline:nnn {#4}
      { \__template_parse_equiv_auxii:n {##1} }
      { \__template_parse_equiv_auxii:n {##1} }
    \__template_parse_equiv_auxiii:nnnn {#1} {#2} {#3} {#4}
  }
\cs_new_protected:Npn \__template_parse_equiv_auxii:n #1
  {
    \seq_if_in:NeF \l__template_key_order_seq { \tl_to_str:n {#1} }
      {
        \msg_error:nnn { template } { unknown-key } {#1}
        \keyval_map_break:n { \use_none:nn }
      }
  }
\cs_new_protected:Npn \__template_parse_equiv_auxiii:nnnn #1#2#3#4
  {
```

```
    \prop_put:NVV \l__template_keytypes_prop \l__template_key_name_tl
      \l__template_keytype_tl
    \seq_put_right:NV \l__template_key_order_seq \l__template_key_name_tl
    \prop_put:NVn \l__template_vars_prop \l__template_key_name_tl {#4}
    \tl_if_empty:nF {#2}
      {
        \prop_put:NVe \l__template_msg_prop \l__template_key_name_tl
          { \exp_not:n {#2} { \exp_not:V \l__template_key_name_tl } }
      }
  }
```

(*End of definition for* `\__template_parse_equiv_elt:nnnn` *and others.*)

13.9 Implementation part of template declaration

`\__template_declare_template_code:nnnnn`
`\__template_declare_template_code:nnnn`

The main function for implementing a template starts with a couple of simple checks to make sure that there are no obvious mistakes: the number of arguments must agree and the template keys must have been declared.

```
\cs_new_protected:Npn \__template_declare_template_code:nnnnn #1#2#3#4#5
  {
    \__template_execute_if_type_exist:nT {#1}
      {
        \__template_execute_if_arg_agree:nnT {#1} {#3}
          {
            \__template_if_keys_exist:nnT {#1} {#2}
              {
                \__template_store_key_implementation:nnn {#1} {#2} {#4}
                \str_if_in:nnTF {#5} { AssignTemplateKeys }
                  {
                    \regex_match:nnTF { \c { AssignTemplateKeys } } {#5}
                      { \__template_declare_template_code:nnnn {#1} {#2} {#3} {#5} }
                      {
                        \__template_declare_template_code:nnnn
                          {#1} {#2} {#3} { \AssignTemplateKeys #5 }
                      }
                  }
                  {
                    \__template_declare_template_code:nnnn
                      {#1} {#2} {#3} { \AssignTemplateKeys #5 }
                  }
              }
          }
      }
  }
\cs_new_protected:Npn \__template_declare_template_code:nnnn #1#2#3#4
  {
    \cs_if_exist:cT { \c__template_code_root_tl #1 / #2 }
      { \msg_info:nnnn { template } { declare-template-code } {#1} {#2} }
    \cs_generate_from_arg_count:cNnn
      { \c__template_code_root_tl #1 / #2 }
      \cs_gset_protected:Npn {#3} {#4}
  }
```

(*End of definition for* `\__template_declare_template_code:nnnnn` *and* `\__template_declare_template_code:nnnn`.)

__template_store_key_implementation:nnn

Actually storing the implementation part of a template is quite easy as it only requires the list of keys given to be turned into a property list. There is also some error-checking to do, hence the need to have the list of defined keytypes available. In certain cases (when choices are involved) parsing the key results in changes to the default values. That is why they are loaded and then saved again.

```
\cs_new_protected:Npn \__template_store_key_implementation:nnn #1#2#3
  {
    \__template_recover_defaults:nn {#1} {#2}
    \__template_recover_keytypes:nn {#1} {#2}
    \__template_recover_vars:nn {#1} {#2}
    \keyval_parse:nnn
      { \__template_parse_vars_elt:n } { \__template_parse_vars_elt:nnn { #1 / #2 } } {#3}
    \__template_store_vars:nn {#1} {#2}
    \prop_map_inline:Nn \l__template_keytypes_prop
      {
        \bool_lazy_or:nnF
          { \str_if_eq_p:nn {##2} { alias } }
          { \str_if_eq_p:nn {##2} { meta } }
          {
             \msg_error:nnnnn { template } { key-not-implemented }
               {##1} {#2} {#1}
          }
      }
  }
```

(*End of definition for* `\__template_store_key_implementation:nnn`.)

__template_parse_vars_elt:n

At the implementation stage, every key must have a value given. So this is an error function.

```
\cs_new_protected:Npn \__template_parse_vars_elt:n #1
  { \msg_error:nnn { template } { key-no-variable } {#1} }
```

(*End of definition for* `\__template_parse_vars_elt:n`.)

__template_parse_vars_elt:nnn
__template_parse_vars_elt_auxi:nn
__template_parse_vars_elt_auxii:nw
__template_parse_vars_elt_auxiii:nnn
__template_parse_vars_elt_auxiii:enn
__template_parse_vars_elt_auxiv:nnw
__template_parse_vars_elt_auxv:nnn
__template_parse_vars_elt_auxv:enn

The actual storage part here is very simple: the storage bin name is placed into the property list. At the same time, a comparison is made with the keytypes defined earlier: if there is a mismatch then an error is raised.

```
\cs_new_protected:Npn \__template_parse_vars_elt:nnn #1#2#3
 {
    \tl_set:Ne \l__template_key_name_tl
      { \tl_trim_spaces:e { \tl_to_str:n {#2} } }
    \prop_get:NVNTF \l__template_keytypes_prop
      \l__template_key_name_tl
      \l__template_keytype_tl
      {
        \__template_split_keytype_arg:V \l__template_keytype_tl
        \__template_parse_vars_elt_auxi:nn {#1} {#3}
        \prop_remove:NV \l__template_keytypes_prop \l__template_key_name_tl
      }
      { \msg_error:nnn { template } { unknown-key } {#2} }
  }
```

Split off any leading `global` and they look for the way to implement.

```
\cs_new_protected:Npn \__template_parse_vars_elt_auxi:nn #1#2
  {
    \__template_parse_vars_elt_auxii:nw {#1} #2 global global \s__template_stop
  }
\cs_new_protected:Npn \__template_parse_vars_elt_auxii:nw
  #1#2 global #3 global #4 \s__template_stop
  {
    \tl_if_blank:nTF {#4}
      { \__template_parse_vars_elt_auxiii:nnn {#2} {#1} { } }
      {
        \tl_if_blank:nTF {#2}
          {
            \__template_parse_vars_elt_auxiii:enn
              { \tl_trim_spaces:n {#3} }
              {#1} { global }
          }
          { \msg_error:nnn { template } { bad-variable } { #2 global #3 } }
      }
  }
\cs_new_protected:Npn \__template_parse_vars_elt_auxiii:nnn #1#2#3
  {
    \cs_if_exist:cTF { __template_parse_vars_ \l__template_keytype_tl :nnn }
      {
        \use:c { __template_parse_vars_ \l__template_keytype_tl :nnn }
          {#1} {#2} {#3}
      }
      {
        \__template_parse_vars_elt_auxiv:nnw
          {#2} {#3} #1 name name \s__template_stop
      }
  }
\cs_generate_variant:Nn \__template_parse_vars_elt_auxiii:nnn { e }
\cs_new_protected:Npn \__template_parse_vars_elt_auxiv:nnw
  #1#2#3 name #4 name #5 \s__template_stop
  {
    \tl_if_blank:nTF {#5}
      { \__template_parse_vars_elt_auxv:nnn {#3} {#1} {#2} }
      {
        \tl_if_blank:nTF {#3}
          {
            \__template_parse_vars_elt_auxv:enn
              { \exp_not:c { \tl_trim_spaces:n {#4} } } {#1} {#2}
          }
          { \msg_error:nnn { template } { bad-variable } { #3 name #4 } }
      }
  }
\cs_new_protected:Npn \__template_parse_vars_elt_auxv:nnn #1#2#3
  {
    \tl_if_single:nTF {#1}
      {
        \cs_if_exist:NF #1
          { \use:c { \__template_map_var_type: _new:N } #1 }
        \prop_put:NVn \l__template_vars_prop
```

```
            \l__template_key_name_tl {#3#1}
        }
        { \msg_error:nnn { template } { bad-variable } {#3#1} }
    }
\cs_generate_variant:Nn \__template_parse_vars_elt_auxv:nnn { e }
```

(*End of definition for* `\__template_parse_vars_elt:nnn` *and others.*)

`\__template_parse_vars_choice:nnn`
`\__template_parse_vars_function:nnn`
`\__template_parse_vars_instance:nnn`

The various special cases.

```
\cs_new_protected:Npn \__template_parse_vars_choice:nnn #1#2#3
  { \__template_implement_choices:nn {#2} {#1} }
\cs_new_protected:Npn \__template_parse_vars_function:nnn #1#2#3
  {
    \cs_if_exist:NF #1
      { \cs_new:Npn #1 { } }
    \prop_put:NVn \l__template_vars_prop
      \l__template_key_name_tl {#3#1}
  }
\cs_new_protected:Npn \__template_parse_vars_instance:nnn #1#2#3
  {
    \prop_put:NVn \l__template_vars_prop
      \l__template_key_name_tl {#3#1}
  }
```

(*End of definition for* `\__template_parse_vars_choice:nnn`*,* `\__template_parse_vars_function:nnn`*, and* `\__template_parse_vars_instance:nnn`*.*)

`\__template_map_var_type:`

Turn a "friendly" variable type into an `expl3` one.

```
\cs_new:Npn \__template_map_var_type:
  {
    \str_case:Vn \l__template_keytype_tl
      {
        { boolean }   { bool }
        { commalist } { clist }
        { integer }   { int }
        { length }    { dim }
        { muskip }    { muskip }
        { real }      { fp }
        { skip }      { skip }
        { tokenlist } { tl }
      }
  }
```

(*End of definition for* `\__template_map_var_type:`*.*)

`\__template_implement_choices:nn`
`\__template_implement_choices_default:`

Implementing choices requires a second key–value loop. So after a little set-up, the standard parser is called.

```
\cs_new_protected:Npn \__template_implement_choices:nn #1#2
  {
    \clist_set:NV \l__template_tmp_clist \l__template_keytype_arg_tl
    \prop_put:NVn \l__template_vars_prop \l__template_key_name_tl { }
    \keyval_parse:nnn
      { \__template_implement_choice_elt:n }
      { \__template_implement_choice_elt:nnn {#1} }
      {#2}
```

```
    \prop_get:NVNT \l__template_values_prop \l__template_key_name_tl
      \l__template_tmp_tl
      { \__template_implement_choices_default: }
    \clist_if_empty:NF \l__template_tmp_clist
      {
        \clist_map_inline:Nn \l__template_tmp_clist
          { \msg_error:nnn { template } { choice-not-implemented } {##1} }
      }
  }
```

A sanity check for the default value, so that an error is raised now and not when converting to assignments.

```
\cs_new_protected:Npn \__template_implement_choices_default:
  {
    \tl_set:Ne \l__template_tmp_tl
      { \l__template_key_name_tl \c_space_tl \l__template_tmp_tl }
    \prop_if_in:NVF \l__template_vars_prop \l__template_tmp_tl
      {
        \tl_set:Ne \l__template_tmp_tl
          { \l__template_key_name_tl \c_space_tl \l__template_tmp_tl }
        \prop_if_in:NVF \l__template_vars_prop \l__template_tmp_tl
          {
            \prop_get:NVN \l__template_keytypes_prop \l__template_key_name_tl
              \l__template_tmp_tl
            \__template_split_keytype_arg:V \l__template_tmp_tl
            \prop_get:NVN \l__template_values_prop \l__template_key_name_tl
              \l__template_tmp_tl
            \msg_error:nnVV { template } { unknown-default-choice }
              \l__template_key_name_tl
              \l__template_key_name_tl
          }
      }
  }
```

(*End of definition for* `\__template_implement_choices:nn` *and* `\__template_implement_choices_default:`.)

`\__template_implement_choice_elt:nnn`
`\__template_implement_choice_elt_aux:nnn`
`\__template_implement_choice_elt_aux:n`
`\__template_implement_choice_elt:n`

The actual storage of the implementation of a choice is mainly about error checking. The code here ensures that all choices have to have been declared, apart from the special `unknown` choice, which must come last. The code for each choice is stored along with the key name in the variables property list.

```
\cs_new_protected:Npn \__template_implement_choice_elt:nnn #1#2#3
  {
    \clist_if_empty:NTF \l__template_tmp_clist
      {
        \str_if_eq:nnTF {#2} { unknown }
          { \__template_implement_choice_elt_aux:nnn {#1} {#2} {#3} }
          { \__template_implement_choice_elt_aux:n {#2} }
      }
      {
        \clist_if_in:NnTF \l__template_tmp_clist {#2}
          {
            \clist_remove_all:Nn \l__template_tmp_clist {#2}
            \__template_implement_choice_elt_aux:nnn {#1} {#2} {#3}
```

```
          }
          { \__template_implement_choice_elt_aux:n {#2} }
      }
  }
\cs_new_protected:Npn \__template_implement_choice_elt_aux:n #1
  {
    \prop_get:NVN \l__template_keytypes_prop \l__template_key_name_tl
      \l__template_tmp_tl
    \__template_split_keytype_arg:V \l__template_tmp_tl
    \msg_error:nnVn { template } { unknown-choice } \l__template_key_name_tl {#1}
  }
\cs_new_protected:Npn \__template_implement_choice_elt_aux:nnn #1#2#3
  {
    \tl_set:Ne \l__template_tmp_tl
      { \l__template_key_name_tl \c_space_tl #2 }
    \prop_put:NVn \l__template_vars_prop \l__template_tmp_tl {#3}
  }
\cs_new_protected:Npn \__template_implement_choice_elt:n #1
  {
    \msg_error:nnVn { template } { choice-requires-code }
      \l__template_key_name_tl {#1}
  }
```

(*End of definition for* `\__template_implement_choice_elt:nnn` *and others.*)

13.10 Editing template defaults

`\__template_edit_defaults:nnn`

Editing the template defaults means getting the values back out of the store, then parsing the list of new values before putting the updated list back into storage.

```
\cs_new_protected:Npn \__template_edit_defaults:nnn #1#2#3
  {
    \__template_if_keys_exist:nnT {#1} {#2}
      {
        \__template_recover_defaults:nn {#1} {#2}
        \__template_parse_values:nnn {#1} {#2} {#3}
        \__template_store_defaults:nn {#1} {#2}
      }
  }
```

(*End of definition for* `\__template_edit_defaults:nnn`.)

`\__template_parse_values:nnn`
`\__template_parse_known_values:nnn`
`\__template_parse_values:NNnnn`

The routine to parse values is the same for both editing a template and setting up an instance. So the code here does only the minimum necessary for reading the values. To allow code sharing, there are two code paths for a missing key: error here or store the information for later processing.

```
\cs_new_protected:Npn \__template_parse_values:nnn
  { \__template_parse_values:NNnnn \use_i:nn \__template_parse_unknown_error:Vn }
\cs_new_protected:Npn \__template_parse_known_values:nnn
  { \__template_parse_values:NNnnn \use_ii:nn \__template_parse_unknown_store:Vn }
\cs_new_protected:Npn \__template_parse_values:NNnnn #1#2#3#4#5
  {
    \__template_recover_keytypes:nn {#3} {#4}
    \seq_clear:N \l__template_value_order_seq
    \keyval_parse:nnn
```

```
      { \__template_parse_values_elt:NNn #1#2 }
      { \__template_parse_values_elt:Nnn #2 }
      {#5}
  }
```

(*End of definition for* `\__template_parse_values:nnn`, `\__template_parse_known_values:nnn`, *and* `\__template_parse_values:NNnnn`.)

`\__template_parse_values_elt:NNn` Almost key needs a value, so this is just an error routine other than for booleans.

```
\cs_new_protected:Npn \__template_parse_values_elt:NNn #1#2#3
  {
    \prop_get:NnN \l__template_keytypes_prop {#3} \l__template_tmp_tl
    \str_if_eq:VnTF \l__template_tmp_tl { boolean }
      { \__template_parse_values_elt:Nnn #2 {#3} { true } }
      {
        #1
          {
            \bool_set_true:N \l__template_error_bool
            \msg_error:nnn { template } { key-no-value } {#3}
          }
          { \clist_put_right:Nn \UnusedTemplateKeys {#3} }
      }
  }
```

(*End of definition for* `\__template_parse_values_elt:NNn`.)

`\__template_parse_values_elt:Nnn` `\__template_parse_values_elt_auxi:w` `\__template_parse_values_elt_auxi:n` `\__template_parse_values_elt_auxiii:Nn` `\__template_parse_values_elt_auxiv:w` `\__template_parse_values_alias:nn` `\__template_parse_values_alias_aux:nn` `\__template_parse_values_meta:nn` `\__template_parse_values_meta_elt:nnnn` `\__template_parse_values_set:n` `\__template_parse_values_set:o` `\__template_parse_values_set:V` `\__template_parse_values_set:v` `\__template_parse_values_set:e` `\__template_parse_values_set:N` `\__template_parse_values_set:c` `\__template_parse_values_add:n` `\__template_parse_values_add:o` `\__template_parse_values_add:V` `\__template_parse_values_add:v` `\__template_parse_values_add:e` `\__template_parse_values_add:N` `\__template_parse_values_add:c` To store the value, find the keytype then call the saving function. These need the current key name, stored in `\l__template_key_name_tl`.

```
\cs_new_protected:Npn \__template_parse_values_elt:Nnn #1#2#3
  {
    \use:e
      {
        \__template_parse_values_elt_auxi:w
          \tl_trim_spaces:e { \tl_to_str:n { #2 : n : } }
          \exp_not:N \q_stop
      }
    \cs_if_exist:cTF { __template_parse_values_set: \l__template_value_exp_str }
      {
        \prop_get:NVNTF \l__template_keytypes_prop \l__template_key_name_tl
          \l__template_tmp_tl
          { \__template_parse_values_elt_auxii:nn { set } }
          { \__template_parse_values_elt_auxiii:Nn #1 }
            {#3}
      }
      {
        \msg_error:nnV { template } { unknown-expansion }
          \l__template_value_exp_str
      }
  }
\use:e
  {
    \cs_new_protected:Npn \exp_not:N \__template_parse_values_elt_auxi:w
      #1 \token_to_str:N : #2 \token_to_str:N : #3 \exp_not:N \q_stop
  }
    {
```

```
      \tl_set:Nn \l__template_key_name_tl {#1}
      \str_set:Nn \l__template_value_exp_str {#2}
    }
\cs_new_protected:Npn \__template_parse_values_elt_auxii:nn #1#2
  {
    \__template_split_keytype_arg:V \l__template_tmp_tl
    \cs_if_exist:cTF
      { __template_parse_values_ \l__template_keytype_tl :nn }
      {
        \prop_get:NVNT \l__template_msg_prop \l__template_key_name_tl
          \l__template_tmp_tl
          { \l__template_tmp_tl }
        \use:c { __template_parse_values_ \l__template_keytype_tl :nn } {#1}
      }
      { \use:c { __template_parse_values_ #1 : \l__template_value_exp_str } }
        {#2}
  }
```

Hunt for a trailing `+`: this allows the `+=` syntax, treating the token as part of the mid-part not the key name.

```
\cs_new_protected:Npn \__template_parse_values_elt_auxiii:Nn #1#2
  {
    \exp_after:wN \__template_parse_values_elt_auxiv:w
      \l__template_key_name_tl \q_nil + \q_nil \q_stop {#2} #1
  }
```

We need the key name after dropping the `+` to see if it exists, but if the key name is not found, the `+` has to be restored.

```
\cs_new_protected:Npn \__template_parse_values_elt_auxiv:w #1 + \q_nil #2 \q_stop #3#4
  {
    \tl_if_empty:nTF {#2}
      { #4 \l__template_key_name_tl }
      {
        \tl_set:Ne \l__template_key_name_tl { \tl_trim_spaces:n {#1} }
        \prop_get:NVNTF \l__template_keytypes_prop \l__template_key_name_tl
          \l__template_tmp_tl
          { \__template_parse_values_elt_auxii:nn { add } }
          {
            \tl_put_right:Nn \l__template_key_name_tl { + }
            #4 \l__template_key_name_tl
          }
      }
        {#3}
  }
```

Equivalent keys need to be “recycled” back around.

```
\cs_new_protected:Npn \__template_parse_values_alias:nn #1#2
  {
    \seq_remove_all:NV \l__template_value_order_seq \l__template_key_name_tl
    \prop_get:NVN \l__template_vars_prop \l__template_key_name_tl
      \l__template_tmp_tl
    \tl_set_eq:NN \l__template_key_name_tl \l__template_tmp_tl
    \__template_parse_values_alias_aux:nn {#2} {#1}
  }
\cs_new_protected:Npn \__template_parse_values_alias_aux:nn #1#2
```

```
  {
    \prop_get:NVN \l__template_keytypes_prop \l__template_key_name_tl
      \l__template_keytype_tl
    \__template_parse_values_elt_auxii:nn {#2} {#1}
  }
\cs_new_protected:Npn \__template_parse_values_meta:nn #1#2
  {
    \seq_remove_all:NV \l__template_value_order_seq \l__template_key_name_tl
    \prop_get:NVN \l__template_vars_prop \l__template_key_name_tl
      \l__template_tmp_tl
    \keyval_parse:nnV
      { \msg_error:nnn { template } { key-no-value } }
      { \__template_parse_values_meta_elt:nnnn {#1} {#2} }
      \l__template_tmp_tl
  }
\cs_new_protected:Npn \__template_parse_values_meta_elt:nnnn #1#2#3#4
  {
    \tl_set:Nn \l__template_key_name_tl {#3}
    \cs_set:Npn \__template_tmp:n ##1 {#4}
    \tl_set:Ne \l__template_tmp_tl { \__template_tmp:n {#2} }
    \exp_args:NV \__template_parse_values_alias_aux:nn
      \l__template_tmp_tl {#1}
  }
```

Setting and adding start quite similar, but diverge later.

```
\cs_new_protected:Npn \__template_parse_values_set:n #1
  {
    \seq_put_right:NV \l__template_value_order_seq \l__template_key_name_tl
    \prop_put:Non \l__template_values_prop \l__template_key_name_tl {#1}
  }
\cs_generate_variant:Nn \__template_parse_values_set:n { o , V , v , e }
\cs_new_eq:NN \__template_parse_values_set:N \__template_parse_values_set:n
\cs_generate_variant:Nn \__template_parse_values_set:N  { c }
\cs_new_protected:Npn \__template_parse_values_add:n #1
  {
    \seq_put_right:NV \l__template_value_order_seq \l__template_key_name_tl
    \prop_get:NVNF \l__template_values_prop \l__template_key_name_tl
      \l__template_tmp_tl
      { \tl_clear:N \l__template_tmp_tl }
    \use:c { __template_parse_values_add_ \l__template_keytype_tl :n } {#1}
  }
\cs_generate_variant:Nn \__template_parse_values_add:n { o , V , v , e }
\cs_new_eq:NN \__template_parse_values_add:N \__template_parse_values_add:n
\cs_generate_variant:Nn \__template_parse_values_add:N  { c }
\clist_map_inline:nn
  { boolean , choice , function , instance }
  {
    \cs_new_protected:cpn { __template_parse_values_add_ #1 :n } ##1
      { \msg_error:nnn { template } { augmented-key-inappropriate } {#1} }
  }
\clist_map_variable:nNn
  { integer , length , muskip , real , skip }
  \l__template_keytype_tl
  {
    \cs_new_protected:cpe
```

```
      { __template_parse_values_add_ \l__template_keytype_tl :n } #1
      {
        \tl_set:Ne \exp_not:N \l__template_tmp_tl
          {
            \exp_not:c { \__template_map_var_type: _eval:n }
              { \exp_not:N \l__template_tmp_tl + #1 }
          }
        \prop_put:NVV \exp_not:N \l__template_values_prop
          \exp_not:N \l__template_key_name_tl \exp_not:N \l__template_tmp_tl
      }
  }
\cs_new_protected:Npn \__template_parse_values_add_commalist:n #1
  {
    \clist_put_right:Nn \l__template_tmp_tl {#1}
    \prop_put:NVV \l__template_values_prop \l__template_key_name_tl \l__template_tmp_tl
  }
\cs_new_protected:Npn \__template_parse_values_add_tokenlist:n #1
  {
    \tl_put_right:Nn \l__template_tmp_tl {#1}
    \prop_put:NVV \l__template_values_prop \l__template_key_name_tl \l__template_tmp_tl
  }
\cs_new_protected:Npn \__template_parse_unknown_error:nn #1#2
  { \msg_error:nnn { template } { unknown-key } {#1} }
\cs_generate_variant:Nn \__template_parse_unknown_error:nn { V }
\cs_new_protected:Npn \__template_parse_unknown_store:nn #1#2
  { \clist_put_right:Nn \UnusedTemplateKeys { #1 = {#2} } }
\cs_generate_variant:Nn \__template_parse_unknown_store:nn { V }
```

(*End of definition for* `\__template_parse_values_elt:Nnn` *and others.*)

`\__template_template_set_eq:nnn` To copy a template, each of the lists plus the code has to be copied across. To keep this independent of the list storage system, it is all done with two-part shuffles.

```
\cs_new_protected:Npn \__template_template_set_eq:nnn #1#2#3
  {
    \__template_recover_defaults:nn {#1} {#3}
    \__template_store_defaults:nn {#1} {#2}
    \__template_recover_keytypes:nn {#1} {#3}
    \__template_store_keytypes:nn {#1} {#2}
    \__template_recover_vars:nn {#1} {#3}
    \__template_store_vars:nn {#1} {#2}
    \cs_if_exist:cT { \c__template_code_root_tl #1 / #2 }
      { \msg_info:nnnn { template } { declare-template-code } {#1} {#2} }
    \cs_gset_eq:cc { \c__template_code_root_tl #1 / #2 }
      { \c__template_code_root_tl #1 / #3 }
  }
```

(*End of definition for* `\__template_template_set_eq:nnn`.)

13.11 Creating instances of templates

`\__template_declare_instance:nnnn` `\__template_declare_instance_aux:nnnn` Making an instance has two distinct parts. First, the keys given are parsed to transfer the values into the structured data format used internally. This allows the default and given values to be combined with no repetition. In the second step, the structured data

is converted to pre-defined variable assignments, and these are stored in the function for the instance.

```
\cs_new_protected:Npn \__template_declare_instance:nnnn #1#2#3#4
  {
    \__template_execute_if_code_exist:nnT {#1} {#2}
      {
        \__template_recover_defaults:nn {#1} {#2}
        \__template_recover_vars:nn {#1} {#2}
        \__template_declare_instance_aux:nnnn {#1} {#2} {#3} {#4}
      }
  }
\cs_new_protected:Npn \__template_declare_instance_aux:nnnn #1#2#3#4
  {
    \bool_set_false:N \l__template_error_bool
    \__template_parse_values:nnn {#1} {#2} {#4}
    \bool_if:NF \l__template_error_bool
      {
        \prop_put:Nnn \l__template_values_prop { from~template } {#2}
        \__template_store_values:nn {#1} {#3}
        \__template_convert_to_assignments:N \l__template_key_order_seq
        \cs_if_exist:cT { \c__template_instances_root_tl #1 / #3 }
          { \msg_info:nnnn { template } { declare-instance } {#3} {#1} }
        \cs_set_protected:cpe { \c__template_instances_root_tl #1 / #3 }
          {
            \tl_set:Nn \exp_not:N \l__template_current_type_tl {#1}
            \tl_set:Nn \exp_not:N \l__template_current_template_tl {#2}
            \exp_not:N \__template_assignments_push:n
              { \exp_not:V \l__template_assignments_tl }
            \exp_not:c { \c__template_code_root_tl #1 / #2 }
          }
      }
  }
```

(*End of definition for* `\__template_declare_instance:nnnn` *and* `\__template_declare_instance_aux:nnnn`.)

`\__template_instance_set_eq:nnn` Copy–paste an instance.

```
\cs_new_protected:Npn \__template_instance_set_eq:nnn #1#2#3
  {
    \__template_if_instance_exist:nnTF {#1} {#3}
      {
        \__template_recover_values:nn {#1} {#3}
        \__template_store_values:nn {#1} {#2}
        \cs_if_exist:cT { \c__template_instances_root_tl #1 / #2 }
          { \msg_info:nnnn { template } { declare-instance } {#2} {#1} }
        \cs_set_eq:cc { \c__template_instances_root_tl #1 / #2 }
          { \c__template_instances_root_tl #1 / #3 }
      }
      { \msg_error:nnnn { template } { unknown-instance } {#1} {#3} }
  }
```

(*End of definition for* `\__template_instance_set_eq:nnn`.)

`\__template_edit_instance:nnn` Editing an instance is almost identical to declaring one. The only variation is the source
`\__template_edit_instance_aux:nnnnn` of the values to use. When editing, they are recovered from the previous instance run.
`\__template_edit_instance_aux:nVnnn`

```
\cs_new_protected:Npn \__template_edit_instance:nnn #1#2#3
  {
    \__template_if_instance_exist:nnTF {#1} {#2}
      {
        \__template_recover_values:nn {#1}  {#2}
        \prop_get:NnN \l__template_values_prop { from~template }
          \l__template_tmp_tl
        \__template_edit_instance_aux:nVnn
          {#1} \l__template_tmp_tl {#2} {#3}
      }
      { \msg_error:nnnn { template } { unknown-instance } {#1} {#2} }
  }
\cs_new_protected:Npn \__template_edit_instance_aux:nnnn #1#2#3#4
  {
    \__template_recover_vars:nn {#1} {#2}
    \__template_declare_instance_aux:nnnn {#1} {#2} {#3} {#4}
  }
\cs_generate_variant:Nn \__template_edit_instance_aux:nnnn { nV }
```

(*End of definition for* `\__template_edit_instance:nnn` *and* `\__template_edit_instance_aux:nnnnn`.)

`\__template_convert_to_assignments:N`
`\__template_convert_to_assignments_aux:n`
`\__template_convert_to_assignments_aux:nn`
`\__template_convert_to_assignments_aux:nV`

The idea on converting to a set of assignments is to loop over each key, so that the loop order follows the declaration order of the keys. This is done using a sequence as property lists are not "ordered".

```
\cs_new_protected:Npn \__template_convert_to_assignments:N #1
  {
    \tl_clear:N \l__template_assignments_tl
    \seq_map_function:NN #1
      \__template_convert_to_assignments_aux:n
  }
\cs_new_protected:Npn \__template_convert_to_assignments_aux:n #1
  {
    \prop_get:NnN \l__template_keytypes_prop {#1} \l__template_tmp_tl
    \__template_convert_to_assignments_aux:nV {#1} \l__template_tmp_tl
  }
```

The second auxiliary function actually does the work. The arguments here are the key name (`#1`) and the keytype (`#2`). From those, the value to assign and the name of the appropriate variable are recovered. A bit of work is then needed to sort out keytypes with arguments (for example instances), and to look for global assignments. Once that is done, a hand-off can be made to the handler for the relevant keytype.

```
\cs_new_protected:Npn \__template_convert_to_assignments_aux:nn #1#2
  {
    \prop_pop:NnNT \l__template_values_prop {#1} \l__template_value_tl
      {
        \prop_get:NnNTF \l__template_vars_prop {#1} \l__template_var_tl
          {
            \tl_set:Nn \l__template_key_name_tl {#1}
            \__template_split_keytype_arg:n {#2}
            \bool_lazy_any:nF
              {
                { \str_if_eq_p:Vn \l__template_keytype_tl { alias } }
                { \str_if_eq_p:Vn \l__template_keytype_tl { choice } }
                { \str_if_eq_p:Vn \l__template_keytype_tl { meta } }
```

```
            }
            { \__template_find_global: }
          \cs_if_exist_use:cF { __template_assign_ \l__template_keytype_tl : }
            { \__template_assign_variable: }
        }
        { \msg_error:nnn { template } { unknown-key } {#1} }
    }
  }
\cs_generate_variant:Nn \__template_convert_to_assignments_aux:nn { nV }
```

(*End of definition for* `\__template_convert_to_assignments:N`, `\__template_convert_to_assignments_aux:n`, *and* `\__template_convert_to_assignments_aux:nn`.)

`\__template_find_global:`
`\__template_find_global_aux:w`

Global assignments should have the phrase `global` at the front. This is pretty easy to find: no other error checking, though.

```
\cs_new_protected:Npn \__template_find_global:
  {
    \bool_set_false:N \l__template_global_bool
    \tl_if_in:onT \l__template_var_tl { global }
      {
        \exp_after:wN \__template_find_global_aux:w \l__template_var_tl \s__template_stop
      }
  }
\cs_new_protected:Npn \__template_find_global_aux:w  #1 global #2 \s__template_stop
  {
    \tl_set:Nn \l__template_var_tl {#2}
    \bool_set_true:N \l__template_global_bool
  }
```

(*End of definition for* `\__template_find_global:` *and* `\__template_find_global_aux:w`.)

`\__template_instance_value:nnn`

For editing templates.

```
\cs_new:Npn \__template_instance_value:nnn #1#2#3
  {
    \__template_if_instance_exist:nnT {#1} {#2}
      { \prop_item:cn { \c__template_values_root_tl #1 / #2 } {#3} }
  }
```

(*End of definition for* `\__template_instance_value:nnn`.)

13.12 Using templates directly

`\__template_use_template:nnn`

Directly use a template with a particular parameter setting. This is also picked up if used in a nested fashion inside a parameter list. The idea is essentially the same as creating an instance, just with no saving of the result.

```
\cs_new_protected:Npn \__template_use_template:nnn #1#2#3
  {
    \__template_execute_if_code_exist:nnT {#1} {#2}
      {
        \__template_recover_defaults:nn {#1} {#2}
        \__template_recover_vars:nn {#1} {#2}
        \__template_parse_values:nnn {#1} {#2} {#3}
        \__template_convert_to_assignments:N \l__template_key_order_seq
        \__template_debug_typeout:n
```

```
          { Using~template~'#2'~of~type~'#1'~directly~ \msg_line_context: }
        \tl_set:Nn \l__template_current_type_tl {#1}
        \tl_set:Nn \l__template_current_template_tl {#2}
        \use:c { \c__template_code_root_tl #1 / #2 }
      }
  }
```

(*End of definition for* `\__template_use_template:nnn`.)

13.13 Assigning values to variables

`\__template_assign_boolean:`
`\__template_assign_boolean_aux:n`

Setting a Boolean value is slightly different to everything else as the value can be used to work out which `set` function to call. As long as there is no need to recover things from another variable, everything is pretty easy. If there is, then we need to allow for the fact that the recovered value here will *not* be expandable, so needs to be converted to something that is.

```
\cs_new_protected:Npn \__template_assign_boolean:
  {
    \bool_if:NTF \l__template_global_bool
      { \__template_assign_boolean_aux:n { bool_gset } }
      { \__template_assign_boolean_aux:n { bool_set } }
  }
\cs_new_protected:Npn \__template_assign_boolean_aux:n #1
  {
    \__template_if_key_value:VTF \l__template_value_tl
      {
        \__template_key_to_value:
        \tl_put_right:Ne \l__template_assignments_tl
          {
            \exp_not:c { #1 _eq:NN }
            \exp_not:V \l__template_var_tl
            \exp_not:V \l__template_value_tl
          }
      }
      {
        \tl_put_right:Ne \l__template_assignments_tl
          {
            \exp_not:c { #1 _ \l__template_value_tl :N }
            \exp_not:V \l__template_var_tl
          }
      }
  }
```

(*End of definition for* `\__template_assign_boolean:` *and* `\__template_assign_boolean_aux:n`.)

`\__template_assign_choice:`
`\__template_assign_choice_aux:nF`
`\__template_assign_choice_aux:eF`

The idea here is to find either the choice as-given or else the special `unknown` choice, and to copy the appropriate code across.

```
\cs_new_protected:Npn \__template_assign_choice:
  {
    \__template_assign_choice_aux:eF
      { \l__template_key_name_tl \c_space_tl \l__template_value_tl }
      {
        \__template_assign_choice_aux:eF
          { \l__template_key_name_tl \c_space_tl unknown }
```

```
          {
            \prop_get:NVN \l__template_keytypes_prop \l__template_key_name_tl
              \l__template_tmp_tl
            \__template_split_keytype_arg:V \l__template_tmp_tl
            \msg_error:nnVV { template } { unknown-choice }
              \l__template_key_name_tl
              \l__template_value_tl
          }
      }
  }
\cs_new_protected:Npn \__template_assign_choice_aux:nF #1
  {
    \prop_get:NnNTF \l__template_vars_prop {#1} \l__template_tmp_tl
      { \tl_put_right:NV \l__template_assignments_tl \l__template_tmp_tl }
  }
\cs_generate_variant:Nn \__template_assign_choice_aux:nF { e }
```

(*End of definition for* `\__template_assign_choice:` *and* `\__template_assign_choice_aux:nF`.)

`\__template_assign_function:`
`\__template_assign_function_aux:N`

This looks a bit messy but is only actually one function.

```
\cs_new_protected:Npn \__template_assign_function:
  {
    \bool_if:NTF \l__template_global_bool
      { \__template_assign_function_aux:N \cs_gset_protected:Npn }
      { \__template_assign_function_aux:N \cs_set_protected:Npn  }
  }
\cs_new_protected:Npn \__template_assign_function_aux:N #1
  {
    \tl_put_right:Ne \l__template_assignments_tl
      {
        \cs_generate_from_arg_count:NNnn
          \exp_not:V \l__template_var_tl
          \exp_not:N #1
          { \exp_not:V \l__template_keytype_arg_tl }
          { \exp_not:V \l__template_value_tl }
      }
  }
```

(*End of definition for* `\__template_assign_function:` *and* `\__template_assign_function_aux:N`.)

`\__template_assign_instance:`
`\__template_assign_instance_aux:N`

Using an instance means adding the appropriate function creation to the tl. No checks are made at this stage, so if the instance is not valid then errors will arise later.

```
\cs_new_protected:Npn \__template_assign_instance:
  {
    \bool_if:NTF \l__template_global_bool
      { \__template_assign_instance_aux:N \cs_gset_protected:Npn }
      { \__template_assign_instance_aux:N \cs_set_protected:Npn  }
  }
\cs_new_protected:Npn \__template_assign_instance_aux:N #1
  {
    \tl_put_right:Ne \l__template_assignments_tl
      {
        \exp_not:N #1 \exp_not:V \l__template_var_tl
          {
```

```
            \__template_use_instance:nn
              { \exp_not:V \l__template_keytype_arg_tl }
              { \exp_not:V \l__template_value_tl }
          }
      }
  }
```

(*End of definition for* `\__template_assign_instance:` *and* `\__template_assign_instance_aux:N`.)

`\__template_assign_variable:`
`\__template_assign_variable:n`

A general-purpose function for all of the other assignments. As long as the value is not coming from another variable, the stored value is simply transferred for output. We use `V`-type expansion for the `\KeyValue` case: for token lists this is essential, whilst for register-based variables, it does no harm and avoids needing a low-level test.

```
\cs_new_protected:Npn \__template_assign_variable:
  {
    \exp_args:Ne \__template_assign_variable:n
      {
        \__template_map_var_type:
        _
        \bool_if:NT \l__template_global_bool { g }
        set:N
      }
  }
```

Notice we need a `V`-type variant for each `(g)set` operation here: these need to be provided by expl3.

```
\cs_new_protected:Npn \__template_assign_variable:n #1
  {
    \__template_if_key_value:VTF \l__template_value_tl
      {
        \__template_key_to_value:
        \tl_put_right:Ne \l__template_assignments_tl
          {
            \exp_not:c { #1 V } \exp_not:V \l__template_var_tl
             \exp_not:V \l__template_value_tl
          }
      }
      {
        \tl_put_right:Ne \l__template_assignments_tl
          {
            \exp_not:c { #1 n } \exp_not:V \l__template_var_tl
             { \exp_not:V \l__template_value_tl }
          }
      }
  }
```

(*End of definition for* `\__template_assign_variable:` *and* `\__template_assign_variable:n`.)

`\__template_key_to_value:`
`\__template_key_to_value_auxi:w`
`\__template_key_to_value_auxii:w`

The idea here is to recover the attribute value of another key. To do that, the marker is removed and a look up takes place. If this is successful, then the name of the variable of the attribute is returned. This assumes that the value will be used in context where it will be converted to a value, for example when setting a number. There is also a need to check in case the copied value happens to be `global`.

```
\cs_new_protected:Npn \__template_key_to_value:
```

```
  { \exp_after:wN \__template_key_to_value_auxi:w \l__template_value_tl }
\cs_new_protected:Npn \__template_key_to_value_auxi:w \KeyValue #1
  {
    \tl_set:Ne \l__template_tmp_tl { \tl_trim_spaces:e { \tl_to_str:n {#1} } }
    \prop_get:NVNTF \l__template_vars_prop \l__template_tmp_tl
      \l__template_value_tl
      {
        \exp_after:wN \__template_key_to_value_auxii:w \l__template_value_tl
          \s__template_mark global \q__template_nil \s__template_stop
      }
      { \msg_error:nnV { template } { unknown-attribute } \l__template_tmp_tl }
  }
\cs_new_protected:Npn \__template_key_to_value_auxii:w #1 global #2#3 \s__template_stop
  {
    \__template_quark_if_nil:NF #2
      { \tl_set:Nn \l__template_value_tl {#2} }
  }
```

(*End of definition for* `\__template_key_to_value:`, `\__template_key_to_value_auxi:w`, *and* `\__template_key_to_value_auxii:w`.)

13.14 Using instances

`\__template_use_instance:nn`
`\__template_use_instance_aux:nNnnn`
`\__template_use_instance_auxi:nn`
`\__template_use_instance_auxii:nn`

Using an instance is just a question of finding the appropriate function. If nothing is found, an error is raised. One complication is that if the first token of argument `#2` is `\UseTemplate` then that is also valid. There is an error-test to make sure that the types agree, and if so the template is used directly.

```
\cs_new_protected:Npn \__template_use_instance:nn #1#2
  {
    \__template_if_use_template:nTF {#2}
      { \__template_use_instance_aux:nNnnn {#1} #2 }
      { \__template_use_instance_auxi:nn {#1} {#2} }
  }
\cs_new_protected:Npn \__template_use_instance_aux:nNnnn #1#2#3#4#5
  {
    \str_if_eq:nnTF {#1} {#3}
      { \__template_use_template:nnn {#3} {#4} {#5} }
      { \msg_error:nnnn { template } { type-mismatch } {#1} {#3} }
  }
\cs_new_protected:Npn \__template_use_instance_auxi:nn #1#2
  {
    \__template_if_instance_exist:nnTF {#1} {#2}
      { \__template_use_instance_auxii:nn {#1} {#2} }
      { \msg_error:nnnn { template } { unknown-instance } {#1} {#2} }
  }
\cs_new_protected:Npn \__template_use_instance_auxii:nn #1#2
  {
    \__template_debug_typeout:n
      { Using~instance~'#2'~of~type~'#1'~ \msg_line_context: }
    \use:c { \c__template_instances_root_tl #1 / #2 }
  }
```

(*End of definition for* `\__template_use_instance:nn` *and others.*)

13.15 Assignment manipulation

A few functions to transfer assignments about, as this is needed by `\AssignTemplateKeys`.

`\__template_assignments_pop:` To actually use the assignments.

```
\cs_new:Npn \__template_assignments_pop: { \l__template_assignments_tl }
```

(*End of definition for* `\__template_assignments_pop:`.)

`\__template_assignments_push:n` Here, the assignments are stored for later use.

```
\cs_new_protected:Npn \__template_assignments_push:n #1
  { \tl_set:Nn \l__template_assignments_tl {#1} }
```

(*End of definition for* `\__template_assignments_push:n`.)

13.16 Ad hoc key setting

For (mainly) user-overrides at point of use. We aim to keep the key structures "inside" lttemplates.

`\__template_keys_set:nnn` Setting keys in the document body requires that we work in a similar way to creating instances. To have keys set in source order, we use a sequence which is set during parsing. This allows us to reuse internals. Note that the key argument is expanded exactly once, which allows for users "recycling" keys as `\UnusedTemplateKeys` directly rather than by value.

```
\cs_new_protected:Npn \__template_keys_set:nnn #1#2#3
  {
    \__template_keys_set_known:nnn {#1} {#2} {#3}
    \tl_if_empty:NF \UnusedTemplateKeys
      {
        \msg_error:nnV { template } { unknown-ad-hoc-keys }
          \UnusedTemplateKeys
      }
  }
\cs_new_protected:Npn \__template_keys_set_known:nnn #1#2#3
  { \exp_args:No \__template_keys_set_known_aux:nnn {#3} {#1} {#2} }
```

Key setting is skipped if the input is entirely blank or if it is exactly `\NoValue` (which could happen if material is passed at a higher level without a branching check). As well as setting native keys, we also check for any in l3keys for compatibility with the previous approach taken.

```
\cs_new_protected:Npn \__template_keys_set_known_aux:nnn #1#2#3
  {
    \tl_clear:N \UnusedTemplateKeys
    \bool_lazy_or:nnF
      { \tl_if_blank_p:n {#1} }
      { \tl_if_novalue_p:n {#1} }
      {
        \keys_set_known:nnN { template / #2  / #3 } {#1} \l__template_tmp_tl
        \__template_recover_vars:nn {#2} {#3}
        \exp_args:NnnV \__template_parse_known_values:nnn {#2} {#3} \l__template_tmp_tl
        \__template_convert_to_assignments:N \l__template_value_order_seq
        \tl_use:N \l__template_assignments_tl
      }
  }
```

(*End of definition for* `\__template_keys_set:nnn`, `\__template_keys_set_known:nnn`, *and* `\__template_keys_set_known_aux:nnn`.)

13.17 Showing templates and instances

`\__template_show_code:nn` Showing the code for a template is just a translation of `\cs_show:c`.

```
\cs_new_protected:Npn \__template_show_code:nn #1#2
  { \cs_show:c { \c__template_code_root_tl #1 / #2 } }
```

(*End of definition for* `\__template_show_code:nn`.)

`\__template_show_defaults:nn` `\__template_show_keytypes:nn` `\__template_show_vars:nn` `\__template_show:Nnnn` A modified version of the property-list printing code, such that the output refers to templates and instances rather than to the underlying structures.

```
\cs_new_protected:Npn \__template_show_defaults:nn #1#2
  {
    \__template_if_keys_exist:nnT {#1} {#2}
      {
        \__template_recover_defaults:nn {#1} {#2}
        \__template_show:Nnnn \l__template_values_prop
          {#1} {#2} { default~values }
      }
  }
\cs_new_protected:Npn \__template_show_keytypes:nn #1#2
  {
    \__template_if_keys_exist:nnT {#1} {#2}
      {
        \__template_recover_keytypes:nn {#1} {#2}
        \__template_show:Nnnn \l__template_keytypes_prop
          {#1} {#2} { interface }
      }
  }
\cs_new_protected:Npn \__template_show_vars:nn #1#2
  {
     \__template_execute_if_code_exist:nnT {#1} {#2}
      {
        \__template_recover_vars:nn {#1} {#2}
        \__template_show:Nnnn \l__template_vars_prop
          {#1} {#2} { variable~mapping }
      }
  }
\cs_new_protected:Npn \__template_show:Nnnn #1#2#3#4
  {
    \msg_show:nneeee { template } { show-attribute }
      { \tl_to_str:n {#2} }
      { \tl_to_str:n {#3} }
      { \tl_to_str:n {#4} }
      { \prop_map_function:NN #1 \msg_show_item_unbraced:nn }
  }
```

(*End of definition for* `\__template_show_defaults:nn` *and others.*)

`\__template_show_values:nn` Instance values are a little more complex, as is the template to consider.

```
\cs_new_protected:Npn \__template_show_values:nn #1#2
  {
```

```
    \__template_if_instance_exist:nnTF {#1} {#2}
      {
        \__template_recover_values:nn {#1} {#2}
        \msg_show:nneee { template } { show-values }
          { \tl_to_str:n {#1} }
          { \tl_to_str:n {#2} }
          {
            \prop_map_function:NN \l__template_values_prop
              \msg_show_item_unbraced:nn
          }
      }
      { \msg_error:nnnn { template } { unknown-instance } {#1} {#2} }
  }
```

(*End of definition for* `\__template_show_values:nn`.)

13.18 Messages

The text for error messages: short and long text for all of them.

```
\msg_new:nnnn { template } { argument-number-mismatch }
  { Template~type~'#1'~takes~#2~argument(s). }
  {
    Templates~of~type~'#1'~require~#2~argument(s).\\
    You~have~tried~to~make~a~template~for~'#1'~
    with~#3~argument(s),~which~is~not~possible:~
    the~number~of~arguments~must~agree.
  }

\msg_new:nnnn { template } { augmented-key-inappropriate }
  { Augmented~key~syntax~"<key>~+=~<value>"~inappropriate. }
  {
    Augmented~key~settings~does~not~apply~to~keys~of~the~type~'#1'.
  }
\msg_new:nnnn { template } { bad-number-of-arguments }
  { Bad~number~of~arguments~for~template~type~'#1'. }
  {
    A~template~may~accept~between~0~and~9~arguments.\\
    You~asked~to~use~#2~arguments:~this~is~not~supported.
  }
\msg_new:nnnn { template } { bad-variable }
  { Incorrect~variable~description~'#1'. }
  {
    The~argument~'#1'~is~not~of~the~form \\
    ~~'<variable>'\\
    ~or~\\
    ~~'global~<variable>'.\\
    It~must~be~given~in~one~of~these~formats~to~be~used~in~a~template.
  }
\msg_new:nnnn { template } { choice-not-implemented }
  { The~choice~'#1'~has~no~implementation. }
  {
    Each~choice~listed~in~the~interface~for~a~template~must~
    have~an~implementation.
  }
```

```
\msg_new:nnnn { template } { choice-no-code }
  { The~choice~'#1'~requires~implementation~details. }
  {
    When~creating~template~code~using~\DeclareTemplateCode,~
    each~choice~name~must~have~an~associated~implementation.\\
    This~should~be~given~after~a~'='~sign:~LaTeX~did~not~find~one.
  }
\msg_new:nnnn { template } { choice-requires-code }
  { The~choice~'#2'~for~key~'#1'~requires~an~implementation. }
  {
    You~should~have~put:\\
    \ \ #1~:~choice~{~#2 = <code> ~} \\
    but~LaTeX~did~not~find~any~<code>.
  }

\msg_new:nnnn { template } { deprecated-equiv }
  { The~key~equivalent~"#3"~ #2 ~deprecated~on~#1~\msg_line_context:. }
  {
    The~key~equivalent~"#3"~ #2 ~deprecated,~and~since~#1~should~
    not~be~used:~you~will~need~to~update~your~source.
  }
\msg_new:nnnn { template } { duplicate-key-interface }
  { Key~'#1'~appears~twice~in~interface~definition~\msg_line_context:. }
  {
    Each~key~can~only~have~one~interface~declared~in~a~template.\\
    LaTeX~found~two~interfaces~for~'#1'.
  }

\msg_new:nnnn { template } { equiv-key-interface }
  {
    Key~'#1'~appears~in~interface~definition~and~this~name~therefore~
    cannot~be~used~as~an~equivalent~\msg_line_context:.
  }
  {
    Each~key~can~only~have~one~interface~declared~in~a~template.\\
    LaTeX~found~two~interfaces~for~'#1':~a~base~one~and~an~equivalent.
  }
\msg_new:nnnn { template } { keytype-requires-argument }
  { The~key~type~'#1'~requires~an~argument~\msg_line_context:. }
  {
    You~should~have~put:\\
    \ \ <key-name>~:~#1~{~<argument>~} \\
    but~LaTeX~did~not~find~an~<argument>.
  }

\msg_new:nnnn { template } { key-no-equiv }
  { The~key~'#1'~has~no~equivalents~\msg_line_context:. }
  {
    When~creating~a~compatibility~key~for~a~template~
    every~key~listed~must~include~an~equivalent:\\
    \ \ <old-key>~:~<type>~=~<new-key(s)>
  }
\msg_new:nnnn { template } { key-no-value }
  { The~key~'#1'~has~no~value~\msg_line_context:. }
  {
```

```
    When~creating~an~instance~of~a~template~
    every~key~listed~must~include~a~value:\\
    \ \ <key>~=~<value>
  }
\msg_new:nnnn { template } { key-no-variable }
  { The~key~'#1'~requires~implementation~details~\msg_line_context:. }
  {
    When~creating~template~code~using~\DeclareTemplateCode,~
    each~key~name~must~have~an~associated~implementation.\\
    This~should~be~given~after~a~'='~sign:~LaTeX~did~not~find~one.
  }
\msg_new:nnnn { template } { key-not-implemented }
  { Key~'#1'~has~no~implementation~\msg_line_context:. }
  {
    The~definition~of~key~implementations~for~template~'#2'~
    of~template~type~'#3'~does~not~include~any~details~for~key~'#1'.\\
    The~key~was~declared~in~the~interface~definition,~
    and~so~an~implementation~is~required.
  }
\msg_new:nnnn { template } { missing-keytype }
  { The~key~'#1'~is~missing~a~key-type~\msg_line_context:. }
  {
    Key~interface~definitions~should~be~of~the~form\\
    \ \ #1~:~<key-type>\\
    but~LaTeX~could~not~find~a~<key-type>.
  }
\msg_new:nnnn { template } { no-template-code }
  {
    The~template~'#2'~of~type~'#1'~is~unknown~
    or~has~no~implementation.
  }
  {
    There~is~no~code~available~for~the~template~name~given.\\
    This~should~be~given~using~\DeclareTemplateCode.
  }
\msg_new:nnnn { template } { type-already-defined }
  { Template~type~'#1'~already~defined. }
  {
    You~have~used~\NewTemplateType~
    with~a~template~type~that~has~already~been~defined.
  }
\msg_new:nnnn { template } { type-mismatch }
  { Template~types~'#1'~and~'#2'~do~not~agree. }
  {
    You~are~trying~to~use~a~template~directly~with~\UseInstance
    (or~a~similar~function),~but~the~template~types~do~not~match.
  }

\msg_new:nnnn { template } { unknown-ad-hoc-keys }
  { Unknown~template~keys~"#1". }
  {
    LaTeX~was~asked~to~set~'ad~hoc'~template~keys,~but~the~keys~
    "#1"~are~not~known~here.
  }
```

```
\msg_new:nnnn { template } { unknown-attribute }
  { The~template~attribute~'#1'~is~unknown. }
  {
    There~is~a~definition~in~the~current~template~reading\\
    \ \ \token_to_str:N \KeyValue {~#1~} \\
    but~there~is~no~key~called~'#1'.
  }
\msg_new:nnnn { template } { unknown-choice }
  { The~choice~'#2'~was~not~declared~for~key~'#1'. }
  {
    The~key~'#1'~takes~a~fixed~list~of~choices~
    and~this~list~does~not~include~'#2'.
  }
\msg_new:nnnn { template } { unknown-default-choice }
  { The~default~choice~'#2'~was~not~declared~for~key~'#1'. }
  {
    The~key~'#1'~takes~a~fixed~list~of~choices~
    and~this~list~does~not~include~'#2'.
  }

\msg_new:nnnn { template } { unknown-equiv-keytype }
  { The~equivalent~key-type~'#1'~is~unknown. }
  {
    Valid~equivalent~key-types~are:\\
    -~alias;\\
    -~meta.\\
  }
\msg_new:nnnn { template } { unknown-expansion }
  { The~expansion~type~'#1'~is~unknown. }
  {
    Key~values~can~only~be~expanded~using~one~of~the~pre-defined~methods:~
    n,~o,~V,~v,~e,~N~or~c.
  }
\msg_new:nnnn { template } { unknown-instance }
  { The~instance~'#2'~of~type~'#1'~is~unknown. }
  {
    You~have~asked~to~use~an~instance~'#2',~
    but~this~has~not~been~created.
  }
\msg_new:nnnn { template } { unknown-key }
  { Unknown~template~key~'#1'. }
  {
    The~key~'#1'~was~not~declared~in~the~interface~
    for~the~current~template.
  }
\msg_new:nnnn { template } { unknown-keytype }
  { The~key-type~'#1'~is~unknown. }
  {
    Valid~key-types~are:\\
    -~boolean;\\
    -~choice;\\
    -~commalist;\\
    -~function;\\
    -~instance;\\
```

```
    ~~integer;\\
    ~~length;\\
    ~~muskip;\\
    ~~real;\\
    ~~skip;\\
    ~~tokenlist.
  }
\msg_new:nnnn { template } { unknown-type }
  { The~template~type~'#1'~is~unknown. }
  {
    A~template~type~needs~to~be~defined~with~\NewTemplateType
    prior~to~using~it.
  }
\msg_new:nnnn { template } { unknown-template }
  { The~template~'#2'~of~type~'#1'~is~unknown. }
  {
    No~interface~has~been~declared~for~a~template~
    '#2'~of~template~type~'#1'.
  }
```

Information messages only have text: more text should not be needed.

```
\msg_new:nnn { template } { declare-instance }
  { Declaring~instance~~'#1'~of~type~#2~\msg_line_context:. }
\msg_new:nnn { template } { declare-template-code }
  { Declaring~code~for~template~'#2'~of~template~type~'#1'~\msg_line_context:. }
\msg_new:nnn { template } { declare-template-interface }
  {
    Declaring~interface~for~template~'#2'~of~template~type~'#1'~
    \msg_line_context:.
  }
\msg_new:nnn { template } { declare-type }
  { Declaring~template~type~'#1'~taking~#2~argument(s)~\msg_line_context:. }
\msg_new:nnn { template } { show-attribute }
  {
    The~template~'#2'~of~type~'#1'~has~
    \tl_if_empty:nTF {#4} { no~#3. } { #3 : #4 }
  }
\msg_new:nnn { template } { show-values }
  {
    The~instance~'#2'~of~type~'#1'~has~
    \tl_if_empty:nTF {#3} { no~values. } { values: #3 }
  }
```

Also add template to the LaTeX messages.

```
\prop_gput:Nnn \g_msg_module_type_prop { template } { LaTeX }
```

13.19 User functions

\NewTemplateType
\DeclareTemplateInterface
\DeclareTemplateCode
\DeclareTemplateCopy
\EditTemplateDefaults
\UseTemplate
\DeclareInstance
\DeclareInstanceCopy
\EditInstance
\UseInstance

All simple translations.

```
\cs_new_protected:Npn \NewTemplateType #1#2
  { \__template_define_type:nn {#1} {#2} }
\cs_new_protected:Npn \DeclareTemplateInterface #1#2#3#4
  { \__template_declare_template_keys:nnnn {#1} {#2} {#3} {#4} }
\cs_new_protected:Npn \DeclareTemplateCode #1#2#3#4#5
```

```
  { \__template_declare_template_code:nnnnn {#1} {#2} {#3} {#4} {#5} }
\cs_new_protected:Npn \DeclareTemplateCopy #1#2#3
  { \__template_template_set_eq:nnn {#1} {#2} {#3} }
\cs_new_protected:Npn \EditTemplateDefaults #1#2#3
  { \__template_edit_defaults:nnn {#1} {#2} {#3} }
\cs_new_protected:Npn \UseTemplate #1#2#3
  { \__template_use_template:nnn {#1} {#2} {#3} }
\cs_new_protected:Npn \DeclareInstance #1#2#3#4
  { \__template_declare_instance:nnnn {#1} {#3} {#2} {#4} }
\cs_new_protected:Npn \DeclareInstanceCopy #1#2#3
  { \__template_instance_set_eq:nnn {#1} {#2} {#3} }
\cs_new_protected:Npn \EditInstance #1#2#3
  { \__template_edit_instance:nnn {#1} {#2} {#3} }
\cs_new_protected:Npn \UseInstance #1#2
  { \__template_use_instance:nn {#1} {#2} }
```

(*End of definition for* `\NewTemplateType` *and others. These functions are documented on page 370.*)

`\DeclareTemplateInterfaceEquivalents`

The optional argument is filtered out directly, with everything else passed on.

```
\NewDocumentCommand \DeclareTemplateInterfaceEquivalents { +m +m o +m }
  {
    \IfNoValueTF {#3}
      { \__template_declare_template_equiv:nnnn {#1} {#2} { } {#4} }
      {
        \__template_date_compare:nNnTF
          {
            \int_use:N \c_sys_year_int -
            \int_use:N \c_sys_month_int -
            \int_use:N \c_sys_day_int
          }
          >
          {#3}
          {
            \__template_declare_template_equiv:nnnn {#1} {#2}
              {
                \msg_error:nnnnn { template } { deprecated-equiv }
                  {#3} { was }
              }
              {#4}
          }
          {
            \__template_declare_template_equiv:nnnn {#1} {#2}
              {
                \msg_warning:nnnnn { template } { deprecated-equiv }
                  {#3} { will~be }
              }
              {#4}
          }
      }
  }
```

(*End of definition for* `\DeclareTemplateInterfaceEquivalents`*. This function is documented on page 378.*)

`\ShowTemplateCode`
`\ShowTemplateDefaults`
`\ShowTemplateInterface`
`\ShowTemplateVariables`
`\ShowInstanceValues`

The show functions are again just translation.

```
\cs_new_protected:Npn \ShowTemplateCode #1#2
  { \__template_show_code:nn {#1} {#2} }
\cs_new_protected:Npn \ShowTemplateDefaults #1#2
  { \__template_show_defaults:nn {#1} {#2} }
\cs_new_protected:Npn \ShowTemplateInterface #1#2
  { \__template_show_keytypes:nn {#1} {#2} }
\cs_new_protected:Npn \ShowTemplateVariables #1#2
  { \__template_show_vars:nn {#1} {#2} }
\cs_new_protected:Npn \ShowInstanceValues #1#2
  { \__template_show_values:nn {#1} {#2} }
```

(*End of definition for* `\ShowTemplateCode` *and others. These functions are documented on page* 379.)

`\IfInstanceExistsT`
`\IfInstanceExistsF`
`\IfInstanceExistsTF`

More direct translation.

```
\cs_new:Npn \IfInstanceExistsTF #1#2
  { \__template_if_instance_exist:nnTF {#1} {#2} }
\cs_new:Npn \IfInstanceExistsT #1#2
  { \__template_if_instance_exist:nnT {#1} {#2} }
\cs_new:Npn \IfInstanceExistsF #1#2
  { \__template_if_instance_exist:nnF {#1} {#2} }
```

(*End of definition for* `\IfInstanceExistsT`, `\IfInstanceExistsF`, *and* `\IfInstanceExistsTF`. *These functions are documented on page* 376.)

`\KeyValue`

Simply dump the argument when executed: this should not happen.

```
\cs_new_protected:Npn \KeyValue #1 {#1}
```

(*End of definition for* `\KeyValue`. *This function is documented on page* 372.)

`\InstanceValue`

```
\cs_new:Npn \InstanceValue #1#2#3
   { \__template_instance_value:nnn {#1} {#2} {#3} }
```

(*End of definition for* `\InstanceValue`. *This function is documented on page* 376.)

`\AssignTemplateKeys`

A short call to use a token register by proxy.

```
\cs_new_protected:Npn \AssignTemplateKeys { \__template_assignments_pop: }
```

(*End of definition for* `\AssignTemplateKeys`. *This function is documented on page* 372.)

`\SetKnownTemplateKeys`
`\SetTemplateKeys`

A friendly wrapper, with some speed up for the common case of the third argument being empty.

```
\cs_new_protected:Npn \SetKnownTemplateKeys #1#2#3
  { \__template_keys_set_known:nnn {#1} {#2} {#3} }
\cs_new_protected:Npn \SetTemplateKeys #1#2#3
  { \__template_keys_set:nnn {#1} {#2} {#3} }
```

(*End of definition for* `\SetKnownTemplateKeys` *and* `\SetTemplateKeys`. *These functions are documented on page* 373.)

\SetCurrentKnownTemplateKeys

```
\cs_new_protected:Npn \SetCurrentKnownTemplateKeys #1
  {
    \exp_args:NVV \__template_keys_set_known:nnn
      \l__template_current_type_tl
      \l__template_current_template_tl
      {#1}
  }
```

(*End of definition for* `\SetCurrentKnownTemplateKeys`. *This function is documented on page* 373.)

\UnusedTemplateKeys

```
\tl_new:N \UnusedTemplateKeys
```

(*End of definition for* `\UnusedTemplateKeys`. *This function is documented on page* 373.)

\DebugTemplatesOn
\DebugTemplatesOff

```
\cs_new_protected:Npn \DebugTemplatesOn  { \__template_debug_on:  }
\cs_new_protected:Npn \DebugTemplatesOff { \__template_debug_off: }
```

(*End of definition for* `\DebugTemplatesOn` *and* `\DebugTemplatesOff`. *These functions are documented on page* 379.)

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{lttemplates}%
⟨latexrelease⟩                  {Prototype~document~commands}%
⟨latexrelease⟩
⟨latexrelease⟩\EndModuleRelease
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
```

We need to stop DocStrip treating `@@` in a special way at this point.

```
⟨@@=⟩
```

File 12
ltalloc.dtx

1 Counters

This section deals with counter and other variable allocation.

```
1 ⟨*2ekernel⟩
```

The following are from plain TEX:

`\z@` A zero dimen or number. It's more efficient to write `\parindent\z@` than `\parindent 0pt`.

`\@ne` The number 1.

`\m@ne` The number −1.

`\tw@` The number 2.

`\sixt@@n` The number 16.

`\@m` The number 1000.

`\@MM` The number 20000.

`\@xxxii` The constant 32.

```
2 \chardef\@xxxii=32
```

(*End of definition for* `\@xxxii`.)

`\@Mi` `\@Mii` `\@Miii` `\@Miv` Constants 10001–10004.

```
3 \mathchardef\@Mi=10001
4 \mathchardef\@Mii=10002
5 \mathchardef\@Miii=10003
6 \mathchardef\@Miv=10004
```

(*End of definition for* `\@Mi` *and others.*)

`\@tempcnta` `\@tempcntb` Scratch count registers used by LATEX kernel commands.

```
7 \newcount\@tempcnta
8 \newcount\@tempcntb
```

(*End of definition for* `\@tempcnta` *and* `\@tempcntb`.)

`\if@tempswa` General boolean switch used by LATEX kernel commands.

```
9 \newif\if@tempswa
```

(*End of definition for* `\if@tempswa`.)

`\@tempdima` `\@tempdimb` `\@tempdimc` Scratch dimen registers used by LATEX kernel commands.

```
10 \newdimen\@tempdima
11 \newdimen\@tempdimb
12 \newdimen\@tempdimc
```

(*End of definition for* `\@tempdima`, `\@tempdimb`, *and* `\@tempdimc`.)

`\@tempboxa` Scratch box register used by LaTeX kernel commands.

```
\newbox\@tempboxa
```

(*End of definition for* `\@tempboxa`.)

`\@tempskipa` `\@tempskipb` Scratch skip registers used by LaTeX kernel commands.

```
\newskip\@tempskipa
\newskip\@tempskipb
```

(*End of definition for* `\@tempskipa` *and* `\@tempskipb`.)

`\@temptokena` Scratch token register used by LaTeX kernel commands.

```
\newtoks\@temptokena
```

(*End of definition for* `\@temptokena`.)

`\@flushglue` Glue used for `\right-` & `\leftskip` = 0pt plus 1fil

```
\newskip\@flushglue \@flushglue = 0pt plus 1fil
```

(*End of definition for* `\@flushglue`.)

⟨/2ekernel⟩

File 13
ltcntrl.dtx

1 Program control structure

This section defines a number of control structure macros, such as while-loops and for-loops.
Historical LaTeX 2.09 comments (not necessarily accurate any more):

```
⟨*2ekernel⟩
\message{control,}
```

```
\@whilenum TEST \do {BODY}
\@whiledim TEST \do {BODY}  : These implement the loop
          while  TEST  do  BODY  od
    where  TEST  is a TeX \ifnum or \ifdim test, respectively.
    They are optimized for the normal case of TEST initially false.

\@whilesw SWITCH \fi {BODY} : Implements the loop
               while SWITCH do BODY od
    Optimized for normal case of SWITCH initially false.

\@for NAME := LIST \do {BODY} : Assumes that LIST expands to A1,A2,
     ... ,An .
     Executes  BODY  n  times, with  NAME = Ai  on the i-th iteration.
     Optimized for the normal case of n = 1.  Works for n=0.

\@tfor NAME := LIST \do {BODY}
     if, before expansion, LIST = T1 ... Tn  where each Ti is a
     token or {...}, then executes  BODY  n  times, with  NAME = Ti
     on the i-th iteration.  Works for n=0.

 NOTES: 1. These macros use no \@temp sequences.
        2. These macros do not work if the body contains anything that
        looks syntactically to TeX like an improperly balanced \if
        \else \fi.

\@whilenum TEST \do {BODY} ==
 BEGIN
   if  TEST
     then  BODY
           \@iwhilenum{TEST \relax BODY}
 END

\@iwhilenum {TEST BODY} ==
 BEGIN
   if  TEST
     then  BODY
```

```
            \@nextwhile = def(\@iwhilenum)
      else  \@nextwhile = def(\@whilenoop)
    fi
    \@nextwhile {TEST BODY}
  END

 \@whilesw SWITCH \fi {BODY} ==
  BEGIN
    if SWITCH
      then BODY
           \@iwhilesw {SWITCH BODY}\fi
    fi
  END

 \@iwhilesw {SWITCH BODY} \fi ==
  BEGIN
    if SWITCH
      then BODY
           \@nextwhile = def(\@iwhilesw)
      else \@nextwhile = def(\@whileswnoop)
    fi
    \@nextwhile {SWITCH BODY} \fi
  END
```

End of historical LaTeX 2.09 comments.

\@whilenoop
\@whilenum
\@iwhilenum

```
\long\def\@whilenum#1\do #2{\ifnum #1\relax #2\relax\@iwhilenum{#1\relax
    #2\relax}\fi}
\long\def\@iwhilenum#1{\ifnum #1\expandafter\@iwhilenum
        \else\expandafter\@gobble\fi{#1}}
```

(*End of definition for* `\@whilenoop`, `\@whilenum`, *and* `\@iwhilenum`.)

\@whiledim
\@iwhiledim

```
\long\def\@whiledim#1\do #2{\ifdim #1\relax#2\@iwhiledim{#1\relax#2}\fi}
\long\def\@iwhiledim#1{\ifdim #1\expandafter\@iwhiledim
       \else\expandafter\@gobble\fi{#1}}
```

(*End of definition for* `\@whiledim` *and* `\@iwhiledim`.)

\@whileswnoop
\@whilesw
\@iwhilesw

```
\long\def\@whilesw#1\fi#2{#1#2\@iwhilesw{#1#2}\fi\fi}
\long\def\@iwhilesw#1\fi{#1\expandafter\@iwhilesw
        \else\@gobbletwo\fi{#1}\fi}
```

(*End of definition for* `\@whileswnoop`, `\@whilesw`, *and* `\@iwhilesw`.)

Historical LaTeX 2.09 comments (not necessarily accurate any more):

```
\@for NAME := LIST \do {BODY} ==
   BEGIN \@forloop expand(LIST),\@nil,\@nil \@@ NAME {BODY} END

\@forloop CAR, CARCDR, CDRCDR \@@ NAME {BODY} ==
  BEGIN
    NAME = CAR
    if def(NAME) = def(\@nnil)
      else BODY;
           NAME = CARCDR
           if def(NAME) = def(\@nnil)
             else BODY
                  \@iforloop CDRCDR \@@ NAME \do {BODY}
           fi
    fi
  END

\@iforloop CAR, CDR \@@ NAME {BODY} =
    NAME = CAR
    if def(NAME) = def(\@nnil)
       then  \@nextwhile = def(\@fornoop)
       else  BODY ;
             \@nextwhile = def(\@iforloop)
    fi
    \@nextwhile name cdr {body}

\@tfor NAME := LIST \do {BODY}
   =  \@tforloop LIST \@nil \@@ NAME {BODY}

\@tforloop car cdr \@@ name {body} =
    name = car
    if def(name) = def(\@nnil)
       then  \@nextwhile == \@fornoop
       else  body ;
             \@nextwhile == \@forloop
    fi
    \@nextwhile name cdr {body}
```

End of historical LaTeX 2.09 comments.

`\@nnil`

```
\def\@nnil{\@nil}
```

(*End of definition for* `\@nnil`.)

`\@empty`

```
\def\@empty{}
```

(*End of definition for* `\@empty`.)

\@fornoop

```
\long\def\@fornoop#1\@@#2#3{}
```

(*End of definition for* `\@fornoop`.)

\@for

```
\long\def\@for#1:=#2\do#3{%
  \expandafter\def\expandafter\@fortmp\expandafter{#2}%
  \ifx\@fortmp\@empty \else
    \expandafter\@forloop#2,\@nil,\@nil\@@#1{#3}\fi}
```

(*End of definition for* `\@for`.)

\@forloop

```
\long\def\@forloop#1,#2,#3\@@#4#5{\def#4{#1}\ifx #4\@nnil \else
       #5\def#4{#2}\ifx #4\@nnil \else#5\@iforloop #3\@@#4{#5}\fi\fi}
```

(*End of definition for* `\@forloop`.)

\@iforloop

```
\long\def\@iforloop#1,#2\@@#3#4{\def#3{#1}\ifx #3\@nnil
       \expandafter\@fornoop \else
      #4\relax\expandafter\@iforloop\fi#2\@@#3{#4}}
```

(*End of definition for* `\@iforloop`.)

\@tfor

```
\def\@tfor#1:={\@tf@r#1 }
\long\def\@tf@r#1#2\do#3{\def\@fortmp{#2}\ifx\@fortmp\space\else
    \@tforloop#2\@nil\@nil\@@#1{#3}\fi}
\long\def\@tforloop#1#2\@@#3#4{\def#3{#1}\ifx #3\@nnil
       \expandafter\@fornoop \else
      #4\relax\expandafter\@tforloop\fi#2\@@#3{#4}}
```

(*End of definition for* `\@tfor`.)

\@break@tfor Break out of a `\@tfor` loop. This should be called *inside* the scope of an `\if`. See `\@iffileonpath` for an example.

```
\long\def\@break@tfor#1\@@#2#3{\fi\fi}
```

(*End of definition for* `\@break@tfor`.)

\@removeelement Removes an element from a comma-separated list and puts it into a control sequence, called as `\@removeelement{`⟨*element*⟩`}{`⟨*list*⟩`}{`⟨*cs*⟩`}`. Due to the implementation method the ⟨*element*⟩ is not allowed to contain braces.

```
\def\@removeelement#1#2#3{%
  \def\reserved@a##1,#1,##2\reserved@a{##1,##2\reserved@b}%
  \def\reserved@b##1,\reserved@b##2\reserved@b{%
    \ifx,##1\@empty\else##1\fi}%
  \edef#3{%
    \expandafter\reserved@b\reserved@a,#2,\reserved@b,#1,\reserved@a}}
```

(*End of definition for* `\@removeelement`.)

⟨/2ekernel⟩

File 14
lterror.dtx

1 Error handling and tracing

This section defines LaTeX's error commands.

```
⟨*2ekernel⟩
```

The '2ekernel' code ensures that a `\usepackage{autoerr}` is essentially ignored if a 'full' format is being used that has the error messages already in the format.

These days we don't support autoloading approach any longer, but this part bit is kept in case it is used in old documents.

```
\expandafter\let\csname ver@autoerr.sty\endcsname\fmtversion
```

1.1 General commands

`\MessageBreak` This command prints a new-line inside a message, followed by a continuation line begun with `\@msg@continuation`. Normally it is defined to be `\relax`, but inside messages, it is let to `\@message@break`.

```
\let\MessageBreak\relax
```

(*End of definition for* `\MessageBreak`.)

`\GenericInfo` This takes two arguments: a continuation and a message, and sends the result to the log file.

```
\DeclareRobustCommand{\GenericInfo}[2]{%
   \begingroup
      \def\MessageBreak{^^J#1}%
      \set@display@protect
      \immediate\write\m@ne{#2\on@line.}%
   \endgroup
}
```

(*End of definition for* `\GenericInfo`.)

`\GenericWarning` This takes two arguments: a continuation and a message, and sends the result to the screen.

```
\DeclareRobustCommand{\GenericWarning}[2]{%
   \begingroup
      \def\MessageBreak{^^J#1}%
      \set@display@protect
      \immediate\write\@unused{^^J#2\on@line.^^J}%
   \endgroup
}
```

(*End of definition for* `\GenericWarning`.)

\GenericError This macro takes four arguments: a continuation, an error message, where to go for further information, and the help information. It displays the error message, and sets the error help (the result of typing `h` to the prompt), and does a horrible hack to turn the last context line (which by default is the only context line) into just three dots. This could be made more efficient.

```
\bgroup
\lccode`\@=`\ %
\lccode`\~=`\ %
\lccode`\}=`\ %
\lccode`\{=`\ %
\lccode`\T=`\T%
\lccode`\H=`\H%
\catcode`\ =11\relax%
\lowercase{%
\egroup%
```

Unfortunately TEX versions older than 3.141 have a bug which means that `^^J` does not force a linebreak in `\message` and `\errmessage` commands. So for these old TEX's we use `\typeout` to produce the message, and then have an empty `\errmessage` command. This causes an extra line of the form

.

To appear on the terminal, but if you do not like it, you can always upgrade your TEX! In order for your format to use this version, you must define the macro `\@TeXversion` to be the version number, e.g., 3.14 of the underlying TEX. See the comments in `ltdircheck.dtx`.

```
\dimen@\ifx\@TeXversion\@undefined4\else\@TeXversion\fi\p@%
\ifdim\dimen@>3.14\p@%
```

First the 'standard case'.

```
\DeclareRobustCommand{\GenericError}[4]{%
\begingroup%
\immediate\write\@unused{}%
\def\MessageBreak{^^J}%
\set@display@protect%
\edef%
%    %<--------------------do not delete this space!-------------------->%
\@err@                                                                   %
{{#4}}%
\errhelp
%    %<--------------------do not delete this space!-------------------->%
\@err@                                                                   %
\let
%    %<--------------------do not delete this space!-------------------->%
\@err@                                                                   %
\@empty
\def\MessageBreak{^^J#1}%
\def~{\errmessage{%
#2.^^J^^J%
#3^^J%
Type  H <return>  for immediate help%
%    %<--------------------do not delete this space!-------------------->%
\@err@                                                                   %
```

```
}}%
~%
\endgroup}%
\else%
```

Secondly the version for old TEX's.

```
\DeclareRobustCommand{\GenericError}[4]{%
\begingroup%
\immediate\write\@unused{}%
\def\MessageBreak{^^J}%
\set@display@protect%
\edef%
%    %<-------------------do not delete this space!------------------->%
\@err@                                                                  %
{{#4}}%
\errhelp
%    %<-------------------do not delete this space!------------------->%
\@err@                                                                  %
\let
%    %<-------------------do not delete this space!------------------->%
\@err@                                                                  %
\errmessage
\def\MessageBreak{^^J#1}%
\def~{\typeout{! %
#2.^^J^^J%
#3^^J%
Type  H <return>  for immediate help.}%
%    %<-------------------do not delete this space!------------------->%
\@err@                                                                  %
{}}%
~%
\endgroup}%
\fi}%
```

(*End of definition for* `\GenericError`.)

`\PackageError`
`\PackageWarning`
`\PackageWarningNoLine`
`\PackageInfo`
`\ClassError`
`\ClassWarning`
`\ClassWarningNoLine`
`\ClassInfo`

These commands are intended for use by package and class writers, to give information to authors. The syntax is:

```
\PackageError{⟨package⟩}{⟨error⟩}{⟨help⟩}
\PackageWarning{⟨package⟩}{⟨warning⟩}
\PackageWarningNoLine{⟨package⟩}{⟨warning⟩}
\PackageInfo{⟨package⟩}{⟨info⟩}
```

and similarly for classes. The `Error` commands print the ⟨*error*⟩ message, and present the interactive prompt; if the author types `h`, then the ⟨*help*⟩ information is displayed. The `Warning` commands produce a warning but do not present the interactive prompt. The `WarningNoLine` commands do the same, but don't print the input line number. The `Info` commands write the message to the `log` file. Within the messages, the command `\MessageBreak` can be used to break a line, `\protect` can be used to protect command names, and `\space` is a space, for example:

```
\newcommand{\foo}{FOO}
\PackageWarning{ethel}{%
   Your hovercraft is full of eels,\MessageBreak
   and \protect\foo\space is \foo}
```

produces:

```
Package ethel warning: Your hovercraft is full of eels,
(ethel)                and \foo is FOO on input line 54.
```

```
\gdef\PackageError#1#2#3{%
   \GenericError{%
      (#1)\@spaces\@spaces\@spaces\@spaces
   }{%
      Package #1 Error: #2%
   }{%
      See the #1 package documentation for explanation.%
   }{#3}%
}
\def\PackageWarning#1#2{%
   \GenericWarning{%
      (#1)\@spaces\@spaces\@spaces\@spaces
   }{%
      Package #1 Warning: #2%
   }%
}
\def\PackageWarningNoLine#1#2{%
   \PackageWarning{#1}{#2\@gobble}%
}
\def\PackageInfo#1#2{%
   \GenericInfo{%
      (#1) \@spaces\@spaces\@spaces
   }{%
      Package #1 Info: #2%
   }%
}
\gdef\ClassError#1#2#3{%
   \GenericError{%
      (#1) \space\@spaces\@spaces\@spaces
   }{%
      Class #1 Error: #2%
   }{%
      See the #1 class documentation for explanation.%
   }{#3}%
}
\def\ClassWarning#1#2{%
   \GenericWarning{%
      (#1) \space\@spaces\@spaces\@spaces
   }{%
      Class #1 Warning: #2%
   }%
}
\def\ClassWarningNoLine#1#2{%
```

```
   \ClassWarning{#1}{#2\@gobble}%
}
\def\ClassInfo#1#2{%
   \GenericInfo{%
      (#1) \space\space\@spaces\@spaces
   }{%
      Class #1 Info: #2%
   }%
}
```

(*End of definition for* `\PackageError` *and others.*)

`\ClassNote`
`\ClassNoteNoLine`
`\PackageNote`
`\PackageNoteNoLine`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2021/11/15}%
⟨latexrelease⟩                    {\ClassNote}{Notes for classes/packages}%
\def\ClassNote#1#2{%
   \GenericWarning{%
      (#1) \space\space\@spaces\@spaces
   }{%
      Class #1 Info: #2%
   }%
}
\def\ClassNoteNoLine#1#2{\ClassNote{#1}{#2\@gobble}}
\def\PackageNote#1#2{%
   \GenericWarning{%
      (#1) \@spaces\@spaces\@spaces
   }{%
      Package #1 Info: #2%
   }%
}
\def\PackageNoteNoLine#1#2{\PackageNote{#1}{#2\@gobble}}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

We don't roll back, because if this code is used by packages then most often they will not have rollback code implemented, so they would immediately break even if they otherwise would work fine.

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\ClassNote}{Notes for classes/packages}%
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\ClassNote` *and others.*)

`\@latex@error`
`\@latex@warning`
`\@latex@warning@no@line`
`\@latex@info`
`\@latex@info@no@line`

Errors and other info, for use in the LaTeX core.

```
\gdef\@latex@error#1#2{%
   \GenericError{%
      \space\space\space\@spaces\@spaces\@spaces
   }{%
      LaTeX Error: #1%
   }{%
```

```
      See the LaTeX manual or LaTeX Companion for explanation.%
   }{#2}%
}
\def\@latex@warning#1{%
   \GenericWarning{%
      \space\space\space\@spaces\@spaces\@spaces
   }{%
      LaTeX Warning: #1%
   }%
}
\def\@latex@warning@no@line#1{%
   \@latex@warning{#1\@gobble}}
\def\@latex@info#1{%
   \GenericInfo{%
      \@spaces\@spaces\@spaces
   }{%
      LaTeX Info: #1%
   }%
}
\def\@latex@info@no@line#1{%
  \@latex@info{#1\@gobble}}
```

`\@font@warning` and `\@font@info` are defined later since they have to be redefined by the `tracefnt` package.

```
def\@font@warning#1{%
  \GenericWarning{%
     {(font)\@spaces\@spaces}%
     {Font Warning: #1}%
}
def\@font@info#1{%
  \GenericInfo{%
     (font)\space\@spaces
  }{%
     Font Info: #1%
  }%
}
```

(*End of definition for* `\@latex@error` *and others.*)

`\@latex@note` `\@latex@note@no@line`

These are "info" messages that display on the terminal not just in the transcript.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2021/11/15}%
⟨latexrelease⟩                    {\@latex@note}{Display notes}%
\def\@latex@note#1{%
   \GenericWarning{%
      \@spaces\@spaces\@spaces
   }{%
      LaTeX Info: #1%
   }%
}
```

```
\def\@latex@note@no@line#1{%
  \@latex@note{#1\@gobble}}
```

We don't make them undefined but rather point to `\@latex@info` because that's what they replace. This way we can change `\@latex@info` elsewhere without the need to further rollback sections.

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\@latex@note}{Display notes}%
⟨latexrelease⟩
⟨latexrelease⟩\let\@latex@note\@latex@info
⟨latexrelease⟩\let\@latex@note@no@line\@latex@info@no@line
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@latex@note` *and* `\@latex@note@no@line`.)

`\c@errorcontextlines` `\errorcontextlines` as a LaTeX counter, so that it may be manipulated with `\setcounter` (once it is defined :-)

```
\let\c@errorcontextlines\errorcontextlines
\c@errorcontextlines=-1
```

(*End of definition for* `\c@errorcontextlines`.)

`\on@line` The message ' on input line n'.

```
\def\on@line{ on input line \the\inputlineno}
```

(*End of definition for* `\on@line`.)

`\@warning` `\@@warning` `\@latexerr` Older LaTeX messages. For the moment, these `\let` to the new message commands. They may be changed later, once only obsolete packages and classes contain them.

```
\let\@warning\@latex@warning
\let\@@warning\@latex@warning@no@line
\global\let\@latexerr\@latex@error
```

(*End of definition for* `\@warning`, `\@@warning`, *and* `\@latexerr`.)

`\@spaces` Four spaces.

```
\def\@spaces{\space\space\space\space}
```

(*End of definition for* `\@spaces`.)

1.2 Specific errors

`\@eha` `\@ehb` `\@ehc` `\@ehd` The more common error help messages.

```
\gdef\@eha{%
  Your command was ignored.\MessageBreak
  Type \space I <command> <return> \space to replace it %
  with another command,\MessageBreak
  or \space <return> \space to continue without it.}
\gdef\@ehb{%
  You've lost some text. \space \@ehc}
\gdef\@ehc{%
  Try typing \space <return> %
```

```
  \space to proceed.\MessageBreak
  If that doesn't work, type \space X <return> \space to quit.}
\gdef\@ehd{%
  You're in trouble here.  \space\@ehc}
```

(*End of definition for* `\@eha` *and others.*)

`\@notdefinable` Error message generated in `\@ifdefinable` from calls to one of the commands `\newcommand`, `\newlength` or `\newtheorem` specifying an already-defined command name or one that begins `\end...`.

```
\gdef\@notdefinable{%
 \@latex@error{%
   Command \@backslashchar\reserved@a\space
   already defined.\MessageBreak
   Or name \@backslashchar\@qend... illegal,
   see p.192 of the manual}\@eha}
```

(*End of definition for* `\@notdefinable`.)

`\@nolnerr` Generated by `\newline` and `\\` when called in vertical mode.

```
\gdef\@nolnerr{%
  \@latex@error{There's no line here to end}\@eha}
```

(*End of definition for* `\@nolnerr`.)

`\@nocounterr` Generated by `\setcounter`, `\addtocounter` or `\newcounter` if applied to an undefined counter ⟨*cnt*⟩.

`\@nocnterr` Obsolete error message generated in LaTeX2.09 by `\setcounter`, `\addtocounter` or `\newcounter` for undefined counter. DO NOT use for LaTeX 2ε it MIGHT vanish! Use `\@nocounterr{`⟨*cnt*⟩`}` instead.

```
\gdef\@nocounterr#1{%
  \@latex@error{No counter '#1' defined}\@eha}
\gdef\@nocnterr{\@nocounterr?}
```

(*End of definition for* `\@nocounterr` *and* `\@nocnterr`.)

`\@ctrerr` Called when trying to print the value of a counter numbered by letters that's greater than 26.

```
\gdef\@ctrerr{%
  \@latex@error{Counter too large}\@ehb}
```

(*End of definition for* `\@ctrerr`.)

`\@nodocument` Error produced if paragraphs are typeset in the preamble.

```
\gdef\@nodocument{%
  \@latex@error{Missing \protect\begin{document}}\@ehd}
```

(*End of definition for* `\@nodocument`.)

`\@badend` Called by `\end` that doesn't match its `\begin`. RmS 1992/08/24: added code to `\@badend` to display position of non-matching `\begin`. FMi 1993/01/14: missing space added.

The environment name has to literally match, i.e., what is stored in `\@currenvir` (after one expansion) must match what is passed to `\end` (without expansion). If not we complain. Not the absolute best solution but at least it avoids getting `\begin{foo} ended by \end{foo}` which was possible in the past.

```
\gdef\@badend#1{%
  \@latex@error{\protect\begin
     {\detokenize\expandafter{\@currenvir}}\@currenvline
     \space ended by \protect\end{\detokenize{#1}}}\@eha}
```

(*End of definition for* `\@badend`.)

`\@badmath` Called by `\[`, `\]`, `\(` or `\)` when used in wrong mode.

```
\gdef\@badmath{%
  \@latex@error{Bad math environment delimiter}\@eha}
```

(*End of definition for* `\@badmath`.)

`\@toodeep` Called by a list environment nested more than six levels deep, or an enumerate or itemize nested more than four levels.

```
\gdef\@toodeep{%
  \@latex@error{Too deeply nested}\@ehd}
```

(*End of definition for* `\@toodeep`.)

`\@badpoptabs` Called by `\endtabbing` when not enough `\poptabs` have occurred, or by `\poptabs` when too many have occurred.

```
\gdef\@badpoptabs{%
  \@latex@error{\protect\pushtabs\space and \protect\poptabs
      \space don't match}\@ehd}
```

(*End of definition for* `\@badpoptabs`.)

`\@badtab` Called by `\>`, `\+` , `\-` or `\<` when stepping to an undefined tab.

```
\gdef\@badtab{%
 \@latex@error{Undefined tab position}\@ehd}
```

(*End of definition for* `\@badtab`.)

`\@preamerr` This error is special: it appears in places where we normally have to `\protect` expansions. However, to prevent a protection of the error message itself (which would result in the message getting printed not issued on the terminal) we need to locally reset `\protect` to `\relax`.

```
\gdef\@preamerr#1{%
  \begingroup
    \let\protect\relax
    \@latex@error{\ifcase #1 Illegal character\or
     Missing @-exp\or Missing p-arg\fi\space
     in array arg}\@ehd
  \endgroup}
```

(*End of definition for* `\@preamerr`.)

\@badlinearg Occurs in \line and \vector command when a bad slope argument is encountered.

```
\gdef\@badlinearg{%
  \@latex@error{%
       Bad \protect\line\space or \protect\vector
       \space argument}\@ehb}
```

(*End of definition for* \@badlinearg.)

\@LRmoderr A command is not allowed in restricted horizontal mode, i.e., in LR-mode in LaTeX terminology.

```
\gdef\@LRmoderr{%
  \@latex@error{Not allowed in LR mode}\@ehb}
```

(*End of definition for* \@LRmoderr.)

\@parmoderr Occurs in a float environment or a \marginpar when encountered in inner vertical mode.

```
\gdef\@parmoderr{%
  \@latex@error{Not in outer par mode}\@ehb}
```

(*End of definition for* \@parmoderr.)

\@fltovf Occurs in float environment or \marginpar when there are no more free boxes for storing floats.

```
\gdef\@fltovf{%
  \@latex@error{Too many unprocessed floats}\@ehb}
```

(*End of definition for* \@fltovf.)

\@latexbug Occurs in output routine. This is bad news.

```
\gdef\@latexbug{%
  \@latex@error{This may be a LaTeX bug}{Call for help}}
```

(*End of definition for* \@latexbug.)

\@badcrerr This error was removed and replaced by \@nolnerr.

```
%\def\@badcrerr {\@latex@error{Bad use of \protect\\}\@ehc}
```

(*End of definition for* \@badcrerr.)

\@noitemerr \addvspace or \addpenalty was called when not in vmode. Probably caused by a missing \item.

```
\gdef\@noitemerr{%
  \@latex@error{Something's wrong--perhaps a missing %
      \protect\item}\@ehc}
```

(*End of definition for* \@noitemerr.)

\@notprerr A command that can be used only in the preamble appears after the command \begin{document}.

```
\gdef\@notprerr{%
  \@latex@error{Can be used only in preamble}\@eha}
```

(*End of definition for* \@notprerr.)

\@inmatherr Issued by commands that don't work correctly within math (like \item). There is no real error recovery happening, e.g., the user might get additional errors afterwards.

```
\gdef\@inmatherr#1{%
  \relax
  \ifmmode
   \@latex@error{Command \protect#1 invalid in math mode}\@ehc
  \fi}
```

(*End of definition for* \@inmatherr.)

\@invalidchar An error for use with invalid characters. This is commented out, since we decided to use catcode 15 instead.

```
%\def\@invalidchar{\@latex@error{Invalid character in input}\@ehc}
```

(*End of definition for* \@invalidchar.)

As well as the above error commands some error messages are directly coded to save space. The messages already present in LaTeX2.09 include:

`Environment --- undefined`

Issued by \begin for undefined environment.

`Tab overflow`

Occurs in \= when maximum number of tabs exceeded.

`\< in mid line`

Occurs in \< when it appears in middle of line.

`Float(s) lost`

In output routine, caused by a float environment or \marginpar occurring in inner vertical mode.

1.3 Tracing

The trace package implements the commands \traceon and \traceoff that work similar to \tracingall but skip certain code blocks that produce a lot of tracing output being of no interest during debugging (for example loading a font). Code blocks that should be hidden during tracing need to be surrounded by the macros \conditionally@traceoff and \contionally@traceon.

For the kernel code the trace package then redefines a number of macros to include this tracing support.

However, in order to allow any macro package to react to \traceon we also provide dummy definitions for the two commands in the kernel so that they can be used by external packages without the need to distinguish between trace being loaded or not.

\conditionally@traceon
\conditionally@traceoff These are only dummy definitions. For details see the trace package.

```
\let\conditionally@traceon\@empty
\let\conditionally@traceoff\@empty
```

(*End of definition for* \conditionally@traceon *and* \conditionally@traceoff.)

⟨/2ekernel⟩

File 15
ltpar.dtx

1 Paragraphs

This section of the kernel declares the commands used to set `\par` and `\everypar` when ever their function needs to be changed for a long time.

This file here describes the interfaces that have been in the kernel forever, used to implement the scenarios described below. They remain valid but are now augmented in the next file (`ltpara.dtx`) to add hooks to paragraphs. At some point we will consolidate the two files further.

There are two situations in which `\par` may be changed:

- Long-term changes, in which the new value is to remain in effect until the current environment is left. The environments that change `\par` in this way are the following:
  - All list environments (itemize, quote, etc.)
  - Environments that turn `\par` into a noop: tabbing, array and tabular.
- Temporary changes, in which `\par` is restored to its previous value the next time it is executed. The following are all such uses.
  - `\end` when preceded by `\@endparenv`, which is called by `\endtrivlist`
  - The mechanism for avoiding page breaks and getting the spacing right after section heads.

1.1 Implementation

`\@setpar` To permit the proper interaction of these two situations, long-term changes are made by the `\@setpar{`⟨*VAL*⟩`}` command. It's function is:

To set `\par`. It `\def`'s `\par` and `\@par` to ⟨*VAL*⟩.

`\@restorepar` Short-term changes are made by the usual `\def\par` commands. The original values are restored after a short-term change by the `\@restorepar` commands.

`\@@par` `\@@par` always is defined to be the original TEX `\par`.

`\everypar` `\everypar` is changed only for the short term. Whenever `\everypar` is set non-null, it should restore itself to null when executed.

The following commands change `\everypar` in this way:

- `\item`
- `\end` when preceded by `\@endparenv`, which is called by `endtrivlist`
- `\minipage`

When dealing with `\par` and `\everypar` remember the following two warnings:

1. Commands that make short-term changes to \par and \everypar must take account of the possibility that the new commands and the ones that do the restoration may be executed inside a group. In particular, \everypar is executed inside a group whenever a new paragraph begins with a left brace. The \everypar command that restores its definition should be local to the current group (in case the command is inside a minipage used inside someplace where \everypar has been redefined). Thus, if \everypar is redefined to do an \everypar{} it could take several executions of \everypar before the restoration "holds". This usually causes no problem. However, to prevent the extra executions from doing harm, use a global switch to keep anything harmful in the new \everypar from being done twice.

2. Commands that change \everypar should remember that \everypar might be supposed to set the following switches false:

   - @nobreak
   - @minipage

   they should do the setting if necessary.

```
⟨*2ekernel⟩
\message{par,}
```

\@setpar
\@par

Initiate a long-term change to \par.

```
\def\@setpar#1{\def\par{#1}\def\@par{#1}}
```

The default definition of \@par will ensure that if \@restorepar defines \par to execute \@par it will redefine itself to the primitive \@@par after one iteration.

```
\def\@par{\let\par\@@par\par}
```

(*End of definition for* \@setpar *and* \@par.)

\@restorepar

Restore from a short-term change to \par.

```
\def\@restorepar{\def\par{\@par}}
⟨/2ekernel⟩
```

(*End of definition for* \@restorepar.)

File 16
ltpara.dtx

Abstract

This code defines four special kernel hooks to support paragraph tagging as well as four public hooks which can be occasionally useful.

1 Introduction

The building of paragraphs in the TEX engine(s) has a number of peculiarities that makes it on one hand fairly flexible but on the other hand somewhat awkward to control or reliably to extend. Thus to better understand the code below we start with a brief introduction of the mechanism; for more details refer to the TEXbook [**?**, chap. 14] (for the full truth you may even have to study the program code).

1.1 The default processing done by the engine

TEX automatically starts building a paragraph when it is currently in vertical mode and encounters anything that can only live in horizontal mode. Most often this is a character, but there are also many commands that can be used only in horizontal mode. If any of them is encountered, TEX will immediately back up (i.e., the character or command is read later again), adds a `\parskip` glue to the current vertical list unless the list is empty, switches to horizontal mode, starts its special "start of paragraph processing" and only then rereads the character or command that caused the mode change.[24]

This "start of paragraph processing" first adds an empty box at the start of the horizontal list of width `\parindent` (which represents the paragraph indentation) unless the paragraph was started with `\noindent` in which case no such box is added[25]. It then reads and processes all tokens stored in the special engine token register `\everypar`. After that it reads and processes whatever has caused the paragraph to start.

Thus out of the box, TEX offers the possibility to put some special code into `\everypar` to gain control at (more or less) the start of the paragraph. For example, in LaTeX and a number of packages, special code like the following is sometimes used:

```
\everypar{{\setbox\z@\lastbox}\everypar{} ...}
```

This removes the paragraph indentation box again (that was already placed by TEX), then resets `\everypar` so that it doesn't do anything on the next paragraph start and then does whatever it wants to do, e.g., in an `\item` of a list it will typeset the label in front of the paragraph text. However, there is only one such `\everypar` token register and if different packages and/or the kernel all attempt to add their own code here, coordination is very difficult if not impossible.

The process when the paragraph ends has different mechanisms and interfaces. A paragraph ends when the engine primitive `\par` is called while TEX is in unrestricted horizontal mode, i.e., is building a paragraph. At other times this primitive does nothing or generates as an error depending on the mode TEX is in, e.g., the `\par` in `\hbox{a\par b}` is ignored, but `$a\par b$` would complain.

[24] Already not quite true: the command `\noindent` starts the paragraph but influences the special processing by suppressing the paragraph indentation box normally inserted by it.

[25] That's a bit different from placing a zero-sized box!

If this primitive ends the paragraph it does some special "end of horizontal list" processing, then calls TEX's paragraph builder; this breaks the horizontal list into lines and then these lines are added as boxes to the enclosing vertical list and TEX returns to vertical mode.

This `\par` command can be given explicitly, but there are also situations in which TEX is generating it on the fly. Most often this happens when TEX encounters a blank line which is automatically changed to a `\par` command which is then executed. The other possibility is that TEX encounters a command which is incompatible with horizontal processing, e.g., `\vskip` (a request for adding vertical space). In such cases it silently backs up, and inserts a `\par` in the hope that this gets it out of horizontal mode and makes the vertical command acceptable.

The important point to note here is that TEX really inserts the command with the name `\par`, which can be redefined. Thus, it may not have its original "primitive" meaning and therefore may not end the horizontal list and call the paragraph builder. This approach offers some flexibility but also allows you to easily produce a TEX document that loops forever, for example, the simple line

```
A \let\par\relax \vskip
```

will start a horizontal list at `A`, redefines `\par`, then sees `\vskip` and inserts `\par` to end the paragraph. But this now only runs `\relax` so nothing changes and `\vskip` is read again, issues a `\par` which .... In short, it only takes a plain TEX document with five tokens to run forever (since no memory is consumed and therefore eventually exhausted).

There is no way other than changing `\par` to gain control at the end of a paragraph, i.e., there is no token list like `\everypar` that is inserted. Hence the only way to change the default behavior is to modify the action that `\par` executes, with similar issues as outlined before: different processes need to ensure that they do not overwrite their modifications or worse, think that the `\par` in front of them is the engine primitive while in fact it has already been changed by other code.

To make matters slightly worse there are a few places where TEX handles the situation differently (most likely for speed reasons back when computers were much slower). If TEX finds itself in unrestricted horizontal mode at the end of building a vertical box (for an `\insert`, `\vadjust` or executing the output routine code), it will finish the horizontal list not by issuing a `\par` command (which would be consistent with all other places) but by simply executing the primitive meaning of `\par`, regardless of the actual definition that `\par` has at the time.

Thus, if you have carefully crafted a redefined `\par` to execute some special actions at the end of a paragraph and you write something like

```
\vbox{Some paragraph ... text.}
```

you will find that your code does not get run for the last paragraph in that box. LATEX avoids this problem, by making sure that its boxes (such as `\parbox` or the `minipage` environment, etc.) all internally add an explicit `\par` at the end so that such code is run and TEX finds itself in vertical mode already without the need to start up the paragraph builder internally. But, of course, this only works for boxes under direct control of the LATEX kernel; if some package uses low-level `\vbox`es without adding this precaution the TEX optimization kicks in and no special `\par` code is executed.

And there is another optimization that is painful: if a paragraph is interrupted by a mathematical display, e.g., `\[...\]` in LATEX or `$$...$$` in plain TEX, then TEX will resume horizontal mode afterward, i.e., it will start to build a new horizontal list

without inserting an indentation box or \everypar at that point. However, if that list immediately ends with an explicit or implicit \par then TEX will simply throw away this "null" paragraph and not do its usual "end of horizontal list" processing, so this special case also needs to be accounted for when introducing any extended processing.

2 The new mechanism implemented for LATEX

To improve the situation (and also to support automatic tagging of PDF documents) we now offer public as well as private hooks at the start and end of the paragraph processing. The public hooks can be used by packages (or by the user in the preamble or within the document) and using the hook mechanisms it is possible to reorder or arrange code from different packages in such a way that these can safely coexist.

To make that happen we have to make use of the basic functionality that is offered by TEX, e.g., we install special code inside \everypar to provide hooks at the beginning and we redefine \par to do some special processing when appropriate to install hooks at the end of the paragraph.

In order to make this work, we have to ensure that package use of \everypar is not overwriting our code. This is done through a trick: we basically hide the real \everypar from the packages and offer them a new token register (with the same name). So if they install their own code it doesn't overwrite ours. Our code then inserts the new \everypar at the right place inside the process so that it looks as if it was the primitive \everypar.[26]

At the end of the paragraph it would be great if we could use a similar trick. However, due to the fact that TEX inserts the token \par (that doesn't have a defined meaning) we can't hide "the real thing™" and offer the package an indistinguishable alternate.

Fortunately, LATEX has already redefined \par for its own purposes. As a result there aren't many packages that attempt to change \par, because without a lot of extra care that would fail miserably. But the bottom line is that, if you load a package that alters \par then the end of paragraph hooks are most likely not executing while that redefinition is active.[27]

[26] Ideally, \everypar wouldn't be used at all by packages and instead they would simply write their code into the hooks now offered by the kernel. However, while this is the longterm goal and clearly an improvement (because then the packages do no longer need to worry about getting their code overwritten or needing to account for already existing code in \everypar), this will not happen overnight. For that reason support for this legacy method is retained.

[27] Similarly to the \everypar situation, the remedy is that such packages stop doing this and instead add their alterations into the paragraph hooks now provided.

2.1 The provided hooks

para/before
para/begin
para/end
para/after

The following four public hooks are defined and executed for each paragraph:

para/before This hook is executed after the kernel hook `\@kernel@before@para@before` (discussed below) in vertical mode immediately after TEX has contributed `\parskip` to the vertical list and before the actual paragraph processing in horizontal mode starts.

This hook should either not produce any typeset material or add only vertical material. If it starts a paragraph an error is generated. The reason is that we are in the starting process of processing a paragraph and so this would lead to endless recursion.[28]

para/begin This hook is executed after the kernel hook `\@kernel@before@para@begin` (discussed below) in horizontal mode immediately before the indentation box is placed (if there is any, i.e., if the paragraph hasn't been started with `\noindent`).

The indentation box to be typeset is available to the hook as `\IndentBox` and its automatic placement (after the hook is executed) can be prevented through `\OmitIndent`. More precisely `\OmitIndent` voids the box.

The indentation box is then typeset directly after the hook execution by something equivalent to `\box\IndentBox` followed by the current content of the token register `\everypar` that it is available to the kernel or to packages (that run some legacy code).

One has to be careful not to add any code to the hook that starts its own paragraph (e.g., by adding a `\parbox` or a `\marginpar` inside) because that would call the hook inside again (as a new paragraph is started there) and thus lead to an endless recursion ending only after exhausting the available memory. This can only be done by making sure that is not executed for the inner paragraphs (or at least not recursively forever).

para/end This hook is executed at the end of a paragraph when TEX is ready to return to vertical mode and after it has removed the last horizontal glue (but not any kerns) placed on the horizontal list. The code is still executed in horizontal mode so it is possible to add further horizontal material at this point, but it should not alter the mode (even a temporary exit from horizontal mode would create chaos—any attempt will cause an error message)! After the hook has ended the kernel hook `\@kernel@after@para@end` is executed and then TEX returns to vertical mode.

The hook is offered as public hook, but because of the requirement to stay within horizontal mode one needs to be careful in what is placed into the hook.[29]

This hook is implemented as a reversed hook.

para/after This hook is executed directly after TEX has returned to vertical mode and after any material that migrated out of the horizontal list (e.g., from a `\vadjust`) has processed.

[28] One could allow it but only if the newly started paragraph is processed without any hooks. Furthermore correct spacing would be a bit of a nightmare so for now this is forbidden.

[29] Maybe we should guard against that, but it would be rather tricky to implement as mode changes can happen across group boundaries so one would need to keep a private stack just for that. Well, something to ponder.

This hook should either not produce any typeset material or add only vertical material. However, for this hook starting a new paragraph is not a disaster so that it isn't prevented.

This hook is implemented as a reversed hook.

Once that hook code has been processed the kernel hook `\@kernel@after@para@after` is executed as the final action of the paragraph processing.

`\@kernel@before@para@before`
`\@kernel@after@para@after`
`\@kernel@before@para@begin`
`\@kernel@after@para@end`

As already mentioned above there are also four kernel hooks that are executed at the start and end of the processing.

`\@kernel@before@para@before` For future extensions, not currently used by the kernel.

`\@kernel@after@para@after` For future extensions, not currently used by the kernel.

`\@kernel@before@para@begin` Used by the kernel to implement tagging. This hook is executed at the very beginning of a paragraph after TEX has switched to horizontal mode but before any indentation box got added or any `\everypar` was run.

It should not generate typeset material that could alter the position. Note that it should never leave hmode, otherwise you will end with a loop! We could guard against this, but since it is an internal kernel hook that shouldn't be touched this isn't checked.

`\@kernel@after@para@end` Used by the kernel to implement tagging. It is executed directly after the public `para/end` hook. After it there is a quick check that we are still in horizontal mode, i.e., that the public hook has not mistakenly ended horizontal mode prematurely (this is an incomplete check just testing the mode and could perhaps be improved (at the cost of speed)).

2.2 Altered and newly provided commands

`\par`
`\endgraf`
`\para_end:`

An explicit request for ending a paragraph is provided in plain TEX under the name `\endgraf`, which simply uses the primitive meaning (regardless of what `\par` may have as its current definition). In LATEX `\endgraf` (with that behavior) was originally also available.

With the new paragraph handling in LATEX, ending a paragraph means a bit more than just calling the engine's paragraph builder: the process also has to add any hook code for the end of a paragraph. Thus `\endgraf` was changed to provide this additional functionality (along with `\par` remaining subject to its current meaning).

The expl3 name for this functionality is `\para_end:`.

Note: *The next two commands are still under discussion and may slightly change their semantics (as described in the document) and/or their names between now and the 2021 Spring release!*

\OmitIndent
\para_omit_indent:

Inside the para/begin hook one can use this command to suppress the indentation box at the start of the paragraph by emptying the \IndentBox.

If the command is used in vertical mode then it simply issues a \noindent, i.e., starts a paragraph without indentation.

The expl3 name for the function is \para_omit_indent:.

\IndentBox
\g_para_indent_box

The box register holding the indentation box for the paragraph is available for inspection (or changes) inside hooks. The \OmitIndent command clears its content.

The expl3 name for the box register is \g_para_indent_box.

\RawIndent
\para_raw_indent:
\RawNoindent
\para_raw_noindent:
\RawParEnd
\para_raw_end:

\RawIndent *hmode material* \RawParEnd
\RawNoindent *hmode material* \RawParEnd

The commands \RawIndent and \RawNoindent are not meant for normal paragraph building (where the result is a textual paragraph in the traditional meaning of the word), but for special cases where TEX's low-level algorithm is used to achieve special effects, but where the result is not a "paragraph".

They are called "raw", because they bypass LATEX's hook mechanism for paragraphs and simply invoke the low-level TEX algorithm. I.e., they are like the original TEX primitives \indent and \noindent (that is they execute no hooks other than \everypar) except that they can only be used in vertical mode and generate an error if found elsewhere.

To avoid issues a paragraph started by them should always be ended by \RawParEnd[30] and not by \par (or a blank line), because the latter will execute hooks which then have no counterpart at the beginning of the paragraph. It is the responsibility of the programmer to make sure that they are properly paired. This also means that one should not put arbitrary user content between these commands if that content could contain stray \pars.

The expl3 names for the functions are \para_raw_indent:, \para_raw_indent: and \para_raw_end:.

2.3 Examples

None of the examples in this section are meant for real use as they are far too simple-minded but they should give some ideas of what could be possible if a bit more care is applied.

2.3.1 Testing the mechanism

The idea is to output for each paragraph encountered some information: a paragraph sequence number, a level number in roman numerals, the environment in which this paragraph appears, and the line number where the start or end of the paragraph is, e.g., something like

```
PARA: 1-i start (document env. on input line 38)
PARA: 1-i end   (document env. on input line 38)
```

[30] Technical note for those who know their *TEXbook*: the \RawParEnd command invokes the original TEX engine definition of \par that (solely) triggers the paragraph builder in TEX when found inside unrestricted horizontal mode and does nothing in other processing modes.

```
PARA: 2-i start (document env. on input line 40)
PARA: 3-ii start (minipage env. on input line 40)
PARA: 3-ii end   (minipage env. on input line 40)
PARA: 2-i end   (document env. on input line 41)
```

As you can see paragraph 2 starts on line 40 and ends on 41 and inside a minipage started paragraph 3 (start and end on line 40). If you run this on some document you will find that LaTeX considers more things "a paragraph" than you have probably thought.

This was generated by the following hook code:

```
\newcounter{paracnt}          % sequence counter
\newcounter{paralevel}        % level counter
```

To support paragraph nesting we need to maintain a stack of the sequence numbers. This is most easily done using expl3 functions, so we switch over. This is not a very general implementation, just enough for what we need and a bit of LaTeX 2_ε thrown in as well. When popping, the result gets stored in `\paracntvalue` and the `\ERROR` should never happen because it means we have tried to pop from an empty stack.

```
\ExplSyntaxOn
\seq_new:N \g_para_seq
\cs_new:Npn \ParaPush
  {\seq_gpush:No  \g_para_seq {\the\value{paracnt}}}
\cs_new:Npn \ParaPop  {\seq_gpop:NNF  \g_para_seq \paracntvalue \ERROR }
\ExplSyntaxOff
```

At the start of the paragraph increment both sequence counter and level and also save the then current sequence number on our stack.

```
\makeatletter % because we use a few internal 2e commands
\AddToHook{para/begin}{%
  \stepcounter{paracnt}\stepcounter{paralevel}%
  \ParaPush
```

To display the sequence number we `\typeout` the current sequence and level number. The command `\@currenvir` gives us the current environment and `\on@line` produces a space and the current input line number.

```
  \typeout{PARA: \arabic{paracnt}-\roman{paralevel} start
    (\@currenvir\space env.\on@line)}%
```

We also typeset the sequence number as a tiny red number in a box that takes up no horizontal space. This helps us seeing where LaTeX sees the start and end of the paragraphs in the document.

```
  \llap{\color{red}\tiny\arabic{paracnt}\ }%
}
```

At the end of the paragraph we display sequence number and level again. The level counter has the correct value but we need to retrieve the right sequence value by popping it off the stack after which it is available in `\paracntvalue` the way we have set this up above.

```
\AddToHook{para/end}{%
  \ParaPop
  \typeout{PARA: \paracntvalue-\roman{paralevel} end \space\space
    (\@currenvir\space env.\on@line)}%
```

We also typeset again a tiny red number with that value, this time sticking out to the right.[31] We also decrement the level counter since our level has finished.

```
  \rlap{\color{red}\tiny\ \paracntvalue}%
  \addtocounter{paralevel}{-1}%
}
\makeatother
```

2.3.2 Mark the first paragraph of each `itemize`

The code for this is rather simple. We supply some code that is executed only once inside a hook at the start of each `itemize`. We explicitly change the color back and forth so that we don't introduce grouping around the paragraph.

```
\AddToHook{env/itemize/begin}{%
  \AddToHookNext{para/begin}{\color{blue}}%
  \AddToHookNext{para/end}{\color{black}}%
}
```

As a result the first paragraph of each `itemize` will appear in blue.

2.4 Some technical notes

The code tries hard to be transparent for package code, but of course any change means that there is a potential for breaking other code. So in section we collect a few cases that may be of importance if low-level code is dealing with paragraphs that are now behaving slightly differently. The notes are from issues we observed and will probably grow over time.

2.4.1 Glue items between paragraphs (found with fancypar)

In the past LaTeX placed two glue items between two consecutive paragraphs, e.g.,

```
text1 \par text2 \par
```

would show something like

```
\glue(\parskip) 0.0 plus 1.0
\glue(\baselineskip) 5.16669
```

but now there is another `\parskip` glue (that is always 0pt):

```
\glue(\parskip) 0.0 plus 1.0
\glue(\parskip) 0.0
\glue(\baselineskip) 5.16669
```

[31] Note that this can alter the document pagination, because a paragraph ending in a display (e.g., an equation) will get an extra line—in that case our tiny number has an effect even though it doesn't take up any space, because it paragraph is no longer empty and thus isn't dropped!

The reason is that we generate a "fake" paragraph to gain control and safely add the early hooks, but this generates an additional glue item. That item doesn't contribute anything vertically but if somebody writes code that unravels a constructed list using `\lastbox`, `\unskip` and `\unpenalty` then the code has to remove one additional glue item or else it will fail.

3 The Implementation

```
⟨@@=para⟩
⟨*2ekernel | latexrelease⟩
\ExplSyntaxOn
⟨latexrelease⟩\NewModuleRelease{2021/06/01}{ltpara}
⟨latexrelease⟩                        {Paragraph~handling~and~hooks}
```

3.1 Providing hooks for paragraphs

para/before
para/after
para/begin
para/end

The public hooks. They are implemented as a paired set of hooks.

```
\hook_new_pair:nn{para/before}{para/after}
\hook_new_pair:nn{para/begin}{para/end}
```

(*End of definition for* `para/before` *and others. These functions are documented on page* 443.)

\@kernel@before@para@before
\@kernel@after@para@after
\@kernel@before@para@begin
\@kernel@after@para@end

The corresponding kernel hooks (for tagging and future extensions).

```
\let \@kernel@before@para@before \@empty
\let \@kernel@before@para@begin  \@empty
\let \@kernel@after@para@end     \@empty
\let \@kernel@after@para@after   \@empty
```

(*End of definition for* `\@kernel@before@para@before` *and others. These functions are documented on page* 444.)

\g__para_standard_everypar_tl

Whenever TEX starts a paragraph it inserts first an indentation box and then executes the tokens stored in `\tex_everypar:D` (known to LATEX as `\everypar`). We alter this behavior slightly here, so that hooks are added into the right place. Otherwise the process change remains transparent to any legacy code for this space.

We keep the standard code to be used by `\tex_everypar:D` in a separate token list because we have to switch back and forth for error recovery and so altering `\tex_everypar:D` all the time should be a tiny bit faster.

```
⟨latexrelease⟩\IncludeInRelease{2023/06/01}
⟨latexrelease⟩        {\g__para_standard_everypar_tl}{minipage~ fix}
\tl_new:N \g__para_standard_everypar_tl
```

Here is now its definition:

```
\tl_gset:Nn \g__para_standard_everypar_tl {
```

First we remove the indentation box and store it in `\g_para_indent_box`. If there was none because the paragraph was started by `\noindent` the box register will be void.

```
  \box_gset_to_last:N \g_para_indent_box
```

This will make the newly started horizontal list empty, so if we stop it now and return to vertical mode it will be dropped by TEX. We do that but inside a group so that any `\parshape` settings will not get lost as we need them for later.

```
  \group_begin:
    \tex_par:D
  \group_end:
```

We then change \tex_everypar:D to generate an error so that we can detect and report if the para/before hook illegally changed out of vmode.

```
\tex_everypar:D { \msg_error:nnnn { hooks }{ para-mode }{before}{vertical} }
\@kernel@before@para@before
\hook_use:n {para/before}
```

Assuming the hooks have been well behaved it is time to return to horizontal mode and start the paragraph in earnest. We already have the indentation box saved away so we now have to restart the paragraph with an empty \tex_everypar:D and with \tex_noindent:D. And we need to make sure not to get another \parskip or rather (since we can't prevent that) that it is of zero size.

```
\group_begin:
  \tex_everypar:D {}
```

There has been a long-standing problem with LaTeX's minipages in that invisible material at the beginning of a minipage (such as a \color setting) would result in \parskip being added in front of the first paragraph—something that is not done by TeX if a vertical list is completely empty. As this is happening on a very low-level in the engine it wasn't really possible to find out if this \parskip was added or if a space we see in front of the current point is legitimate. However, with the new paragraph handling we are in a better position: while we still don't know if there is such a space or not, we do know if we have just created an empty paragraph. Thus, if we now set \parskip to -\parskip the two will cancel each other if present and if the first was ignored because the vertical list was empty, then the second will be ignored too because it is still empty. Of course, we don't want to cancel always but only at the start of a minipage and that is signaled with the @minipage switch.

```
  \skip_set:Nn \tex_parskip:D
       { \if@minipage -\tex_parskip:D \else: \c_zero_skip \fi: }
  \tex_noindent:D
\group_end:
```

That brings us back to the start of the horizontal list but we need to change \tex_everypar:D back to its normal content in case there are nested paragraphs coming up.

```
\tex_everypar:D{\g__para_standard_everypar_tl}
```

This is followed by executing the kernel and the public hook. The kernel hook is there to enable tagging.

```
\@kernel@before@para@begin
\hook_use:n {para/begin}
```

If we aren't in horizontal mode any longer the hooks above misbehaved.

```
\if_mode_horizontal: \else:
  \msg_error:nnnn { hooks }{ para-mode }{begin}{horizontal} \fi:
```

Finally we reinsert the indentation box (unless suppressed) and then call \everypar the way legacy LaTeX code expects it.

However, adding the public \everypar is a bit tricky (see below) so we add that later, and indirectly.

```
\__para_handle_indent:
% \the \everypar           % <--- done differently below
}
```

```
⟨latexrelease⟩\cs_set:Npn \__para_tmp:w #1#2#3#4#5 { }
⟨latexrelease⟩\tl_gput_right:Nx \g__para_standard_everypar_tl {
⟨latexrelease⟩  \exp_not:N \the
⟨latexrelease⟩  \exp_not:N \toks
⟨latexrelease⟩  \exp_after:wN \__para_tmp:w \token_to_meaning:N \everypar
⟨latexrelease⟩  \c_space_tl
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/06/01}
⟨latexrelease⟩       {\g__para_standard_everypar_tl}{minipage~ fix}
⟨latexrelease⟩
⟨latexrelease⟩\tl_gset:Nn \g__para_standard_everypar_tl {
⟨latexrelease⟩  \box_gset_to_last:N \g_para_indent_box
⟨latexrelease⟩  \group_begin:
⟨latexrelease⟩    \tex_par:D
⟨latexrelease⟩  \group_end:
⟨latexrelease⟩  \tex_everypar:D { \msg_error:nnnn { hooks }{ para-mode }{before}{vertical} }
⟨latexrelease⟩  \@kernel@before@para@before
⟨latexrelease⟩  \hook_use:n {para/before}
⟨latexrelease⟩  \group_begin:
⟨latexrelease⟩    \tex_everypar:D {}
⟨latexrelease⟩    \skip_zero:N \tex_parskip:D
⟨latexrelease⟩    \tex_noindent:D
⟨latexrelease⟩  \group_end:
⟨latexrelease⟩  \tex_everypar:D{\g__para_standard_everypar_tl}
⟨latexrelease⟩  \@kernel@before@para@begin
⟨latexrelease⟩  \hook_use:n {para/begin}
⟨latexrelease⟩  \if_mode_horizontal: \else:
⟨latexrelease⟩    \msg_error:nnnn { hooks }{ para-mode }{begin}{horizontal} \fi:
⟨latexrelease⟩  \__para_handle_indent:
⟨latexrelease⟩}
```

We also have to add the `\everypar` toks register at the end. In case of rollback this is already allocated and we have to find out the correct number (hope this is correctly done)

```
⟨latexrelease⟩\cs_set:Npn \__para_tmp:w #1#2#3#4#5 { }
⟨latexrelease⟩\tl_gput_right:Nx \g__para_standard_everypar_tl {
⟨latexrelease⟩    \exp_not:N \the
⟨latexrelease⟩    \exp_not:N \toks
⟨latexrelease⟩    \exp_after:wN \__para_tmp:w \token_to_meaning:N \everypar
⟨latexrelease⟩    \c_space_tl
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\g__para_standard_everypar_tl`.)

`\tex_everypar:D` `\tex_everypar:D` then only has to execute `\g__para_standard_everypar_tl` by default.

```
\tex_everypar:D{\g__para_standard_everypar_tl}
```

(*End of definition for* `\tex_everypar:D`.)

`\everypar` Tokens inserted at the beginning of the paragraph are placed into `\everypar` inside legacy LaTeX code, e.g., by the list environments or by headings to handle `\clubpenalty`, etc.

Now this isn't any longer the primitive but simply a toks register used in the code above but to legacy LaTeX code that is transparent.

There is, however, a problem: a handful packages use exactly the same trick and replace the primitive with a token register and call the token register inside the renamed primitive. That is they assume that `\everypar` is the primitive and that it will still be called at the start of the paragraph even if renamed.

But if we have already replaced it by a token register then all they do is to give that token register a new name. Thus our code in `\tex_everypar:D` would call `\everypar` (which is now their token register) and the code that they added ends up in our token register which is then never used at all. A bit mind boggling I guess.

So what we have to do is not to call the token register `\everypar` by its name inside `\tex_everypar:D` but by using its actual register number.

```
\newtoks \everypar
```

After we have allocated a new toks register with the name `\everypar` the actual register number is available (briefly) inside `\allocationnumber`. So instead of `\the\everypar` we have to put `\the\toks`⟨*allocated number*⟩ at the end of `\tex_everypar:D`.

So what remains doing is to append a few tokens to the token list `\g__para_standard_everypar_tl` which we do now. We use `x` expansion here to get the value of `\allocationnumber` in, all the other tokens should not be expanded at this point.

One important point here is to terminate the register allocation number with a real space. This space will get swallowed up when the number is read. Anything else, such as `\scan_stop:` would remain in the input and that would mean that it would interfere with `\everypar` code that attempts to scan ahead to see how the paragraph text starts.

```
\tl_gput_right:Ne \g__para_standard_everypar_tl {
  \exp_not:N \the
  \exp_not:N \toks
  \the \allocationnumber
  \c_space_tl
}
```

(*End of definition for* `\everypar`.)

`\g_para_indent_box` For managing the indentation we need to provide a public accessible box register

```
\box_new:N \g_para_indent_box
```

(*End of definition for* `\g_para_indent_box`. *This function is documented on page* 445.)

`\__para_handle_indent:` Adding (typesetting) the indent box is straight forward. If it was emptied before it does nothing.

```
\cs_new:Npn \__para_handle_indent: {
  \box_use_drop:N \g_para_indent_box
}
```

The declaration `\para_omit_indent:` (or `\OmitIndent`) changes that to do nothing—the case when used inside the `para/begin` hook. If encountered in vertical mode it starts the paragraph without indentation.

```
\cs_new_protected:Npn \para_omit_indent: {
  \mode_if_vertical:TF
     { \noindent }
     { \box_gclear:N \g_para_indent_box }
}
```

(*End of definition for* `\__para_handle_indent:`.)

`\IndentBox` `\OmitIndent` The LaTeX 2_ε names for the indentation box and for suppressing it for use in the `para/begin` hook.

```
\cs_set_eq:NN \IndentBox \g_para_indent_box
\cs_set_eq:NN \OmitIndent \para_omit_indent:
```

(*End of definition for* `\IndentBox` *and* `\OmitIndent`. *These functions are documented on page* 445.)

`\para_end:` Adding hooks to the end of a paragraph is similar but here we need to alter the command that is used by TeX to end horizontal mode and return to vertical mode, i.e., `\par`.

This is a bit more complicated as this command can appear anywhere either explicitly or implicitly added by TeX in certain situations:

- when using `\par` in the code or the document
- when using a blank line (which is converted to `\par`)
- when TeX finds any commands incompatible with horizontal mode it issues a `\par` and then rereads the command.

Unfortunately, TeX has some (these days) unnecessary optimizations: if a `\vbox` ends and TeX is still in horizontal mode it simply exercises the paragraph builder instead of issuing a `\par`. It is therefore necessary for LaTeX to ensure that this case doesn't happen and all boxes internally have a `\par` command at their end.

This `\par` may or may not run the "par primitive" (which is always available as `\tex_par:D` in expl3); it is permissible to have a changed meaning and it is in fact changed by LaTeX in various ways at various points inside `latex.ltx`. For this LaTeX 2_ε code has the following conventions: `\@@par` and `\endgraf` both refer to the default meaning (in the past this was the initex primitive) while `\par` is the current meaning which maybe does something else.

We are now going to change this default meaning to instead run `\para_end:`, which ultimately executes the initex primitive but additionally adds our hooks when appropriate. This way the change is again transparent to the legacy LaTeX 2_ε code.

In most cases `\para_end:` should behave exactly like the primitive and we achieve this by simply expanding it to the primitive which is available to us as `\tex_par:D`. This way we don't have to care about whether TeX just does nothing (e.g., if in vertical mode already) or generates an error, etc.

```
\cs_new_protected:Npn \para_end: {
```

CCC Maybe needs more explanation. TEMP NOTE: What should happen if in outer hmode with an empty hlist?

The only case we care about is when we are in horizontal mode (i.e., doing typesetting) and not also in inner mode (i.e., making paragraphs and not building an `\hbox`.

```
\bool_lazy_and:nnT
     { \mode_if_horizontal_p: }
     { \bool_not_p:n { \mode_if_inner_p: } }
     { ...
```

Since this is executed for each and every paragraph in a document we try to stay as fast as possible, so we do not use the above construct but two conditionals instead. Using low-level `\if_mode...` conditions would be even faster but has the danger to conflict with conditionals in the user hooks.

If `\para_end:` is executed while TEX is currently doing a low-level assignment the test for horizontal mode may get executed as part of the assignment. That is normally not an issue but we just found one case where it is:

```
\afterassignment\lst@vskip\@tempskipa \z@ \par
```

If TEX is in hmode while that assignment happens then the `\par` is seen in hmode because in the above case the assignment may not be finished (one should have used `\z@skip`) and the `\lst@vskip` will get inserted into the middle of the conditional. The `\lst@vskip` then changes to vmode and you get a surprising error about the `para/end` hook having changed modes even if you don't have any hook code(!): it is the inserted `\lst@vskip` that is actually causing the change of mode.This is what happened when the output routines got started while a `lstlisting` environment (that redefines `\vskip` in this way) was active. This is really faulty coding, but we try to be proactive and guard the conditional so that any scanning is first stopped, thus:

```
\scan_stop:
\mode_if_horizontal:TF {
  \mode_if_inner:F {
```

In that case the action of the primitive would be to remove the last glue (but no kerns) from the horizontal list (constructed to form a paragraph) and then to append a penalty of 10000 and the `\parfillskip`; it then passes the whole list to the paragraph builder, which breaks it into lines and TEX then returns to vertical mode.

What we want to do is to add this hook code at the end of the horizontal list before any of the above happens. If there was a glue item at the end of the list then it should get removed before the hook code gets added so we have to arrange for this removal.

As in other similar cases, it may be best to add here a `\nobreak` in case the hook itself adds glue and thus creates a non-explicit and unwanted potential breakpoont. On the other hand (as has been argued) the code in the hook should perhaps have the responsibility for adding such a guard penalty in this casse. This needs further analysis and decisions (as in emails).

In either case, good documentation of these hooks is essential, covering what the hook may or should provide and all such related considerations convernimg the content.

There is not much point in checking if there was really a glue item at the end of the horizontal list, instead we simply try to remove one using `\tex_unskip:D`: if there wasn't one this will do nothing.

```
        \tex_unskip:D
```

We then execute the public hook (which may add some final typeset material) followed by the kernel hook that we need for adding tagging support. None of this is supposed to change the mode—at the moment we make only a very simple test for this, more devious changes go unnoticed, but too bad as they will then probably backfire badly.

```
        \hook_use:n{para/end}
        \@kernel@after@para@end
        \mode_if_horizontal:TF {
```

The final action (before getting to the point where `\tex_par:D` is called) is to add an extra glue item so that the primitive is prevented from removing intended glue (if there was some). If we don't do this and the horizontal list ends in several glue items we would end up removing two glue items instead of just the last one, which would be wrong. We use glue (rather than a kern) as that will be removed by the primitive.

There is however one other TEX optimization that hurts: in a sequence like this `$$ ... $$ \par` (with `\par` being the primitive) TEX will be in horizontal mode after the display, ready to receive further paragraph text, but since the `\par` follows immediately there is a "null" paragraph at the end and TEX simply throws that away. The space between `$$` and `\par` got already dropped during the display processing so the `\par` is not removing any space and appending `\parfillskip`, instead it simply goes silently to vmode.

Now if we would have added something (to prevent glue removal) that would look to TEX like material after the display and so we would end up with an empty paragraph just containing a penalty and `\parfillskip`.

We therefore check if the current hlist does end in glue (`\tex_lastnodetype:D` has the value `11`) and if so we add a zero-length guard skip which will be removed by the following `\tex_par:D`.

```
            \if_int_compare:w 11 = \tex_lastnodetype:D
              \tex_hskip:D \c_zero_dim
            \fi:
```

To run the `para/after` hook we first end the paragraph. This means that the `\tex_par:D` at the very end is unnecessary but executing it there unnecessarily is better than having code that tests for all the different mode possibilities.

```
            \tex_par:D
            \hook_use:n{para/after}
            \@kernel@after@para@after
          }
```

If we were not horizontal mode (the F case from above) then the earlier hook `para/end` must have been at fault, so we report that.

```
          { \msg_error:nnnn { hooks }{ para-mode }{end}{horizontal} }
```

Finally close out the nested conditionals.

```
    }
  }
```

And then we can use the primitive to truly end the paragraph.

```
  \tex_par:D
}
```

(*End of definition for* `\para_end:`*. This function is documented on page* 444.)

`\para_raw_indent:`
`\para_raw_noindent:`
`\para_raw_end:`

The commands `\para_raw_indent:` and `\para_raw_noindent:` are like the primitives `\indent` and `\noindent` except that they can only be used in vertical mode.

To avoid issues a paragraph started by them should always be ended by `\para_raw_end:` and not by `\para_end:` or `\par` as the latter will execute hooks which then have no counterpart at the beginning of the paragraph. It is the responsibility of the programmer to make sure that they are properly paired.

```
\cs_new:Npn \para_raw_indent: {
  \mode_if_vertical:TF
      {
```

```
          \tex_everypar:D {
            \box_gset_to_last:N \g_para_indent_box
            \tex_everypar:D { \g__para_standard_everypar_tl }
            \__para_handle_indent:
            \the\everypar }
        }
        { \msg_error:nn { latex2e }{ raw-para } }
  \tex_indent:D
}
\cs_new:Npn \para_raw_noindent: {
  \mode_if_vertical:TF
        {
          \tex_everypar:D {
            \tex_everypar:D { \g__para_standard_everypar_tl }
            \the\everypar }
        }
        { \msg_error:nn { latex2e }{ raw-para } }
  \tex_noindent:D
}
\cs_new_eq:NN \para_raw_end: \tex_par:D
```

(*End of definition for* `\para_raw_indent:`, `\para_raw_noindent:`, *and* `\para_raw_end:`. *These functions are documented on page 445.*)

`\RawIndent` `\RawNoIndent` `\RawParEnd`

The LaTeX 2_ε names for starting and ending a paragraph without adding any hooks.

```
\cs_set_eq:NN \RawIndent   \para_raw_indent:
\cs_set_eq:NN \RawNoindent \para_raw_noindent:
\cs_set_eq:NN \RawParEnd   \para_raw_end:
```

(*End of definition for* `\RawIndent`, `\RawNoIndent`, *and* `\RawParEnd`. *These functions are documented on page 445.*)

This ends the `para` module code.

```
⟨@@=⟩
```

`\par` `\endgraf` `\@@par`

Having the new default definition for `\par` we also have to set it up so that it gets used. This involves three commands: `\par`, `\@@par` (to which LaTeX resets `\par` occasionally) and `\endgraf`, which is another name for the "default" action of `\par`.

```
\cs_set_eq:NN \par     \para_end:
\cs_set_eq:NN \@@par   \para_end:
\cs_set_eq:NN \endgraf \para_end:
```

(*End of definition for* `\par`, `\endgraf`, *and* `\@@par`. *These functions are documented on page 444.*)

While this is not integrated properly into the format we have to redo the `\everypar` setting from the kernel, otherwise that gets lost (as it happens before that file is loaded).

```
\everypar{\@nodocument} %% To get an error if text appears before the \document
```

3.2 The error messages

This one is used when we detect that some hook code has changed the mode where it shouldn't, e.g., by starting or ending a paragraph. The first argument is the hook name second the mode it should have stayed in but didn't.

```
\msg_new:nnnn { hooks } { para-mode }
  {
    Illegal~mode~ change~ in~ hook~ 'para/#1'.\\
    Hook~ code~ did~ not~ remain~ in~ #2~ mode.
  }
  {
    Paragraph~ hooks~ cannot~ change~ the~ TeX~ mode~ without~ causing~
    endless~ recursion.~ The~ hook~ code~ in~ 'para/#1'~ needs~ to~ stay~
    in~ #2~ mode,~ but~ it~ didn't.~ Examine~ the~ hook~
    code~ with~ \iow_char:N \\ShowHook~ to~ find~ the~ issue.
  }
```

And here is one used in the "raw" commands when they are used outside of vertical mode.

```
\msg_new:nnnn { latex2e } { raw-para }
  {
     Not~ in~ vertical~ mode.
  }
  {
    Starting~ a~ paragraph~ with~ \iow_char:N \\RawIndent~ or~
    \iow_char:N \\RawNoindent \\
    (or~ \iow_char:N \\para_raw_indent:~ or~
    \iow_char:N \\para_raw_noindent:)~ is~ only~ allowed \\
    if~ LaTeX~ is~ in~ vertical~ mode.
  }
%
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {ltpara}{Undo~hooks~for~paragraphs}
⟨latexrelease⟩
⟨latexrelease⟩\let \OmitIndent  \@undefined
⟨latexrelease⟩\let \IndentBox   \@undefined
⟨latexrelease⟩\let \RawIndent   \@undefined
⟨latexrelease⟩\let \RawNoindent \@undefined
⟨latexrelease⟩\let \RawParEnd   \@undefined
⟨latexrelease⟩
⟨latexrelease⟩\cs_set_eq:NN \par     \tex_par:D
⟨latexrelease⟩\cs_set_eq:NN \@@par   \tex_par:D
⟨latexrelease⟩\cs_set_eq:NN \endgraf \tex_par:D
⟨latexrelease⟩
```

We also need to clean up the primitive "everypar" as that should no longer execute any code by default. And, of course, make `\everypar` become the primitive again.

```
⟨latexrelease⟩\tex_everypar:D {}
⟨latexrelease⟩\cs_set_eq:NN \everypar \tex_everypar:D
⟨latexrelease⟩
⟨latexrelease⟩\EndModuleRelease
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
```

File 17
ltmeta.dtx

Abstract

This code defines the `\DocumentMetadata` interface.

1 Introduction

In the past there was no dedicated location to declare settings concerning a document as a whole. Settings are placed somewhere in the preamble or with the class options or even with some package options. For some settings this can be too late, for example the pdf version can no longer be changed if a package has used code which already opened the PDF.

`\DocumentMetadata` as a new command unifies such settings in one place. It must be used before `\documentclass` but can be issued more than once there.

At the moment most of the code run by `\DocumentMetadata` is external to the format and subject to change. This includes the supported key/values.

For that reason all that happens right now in the format is to look for suitable support files and if found, to redirect the processing to them.

1.1 \DocumentMetadata

`\DocumentMetadata` `\DocumentMetadata{`⟨*key-value list*⟩`}`

The keys defined for `\DocumentMetadata` currently allow to set the PDF version, to set the PDF `/Lang`, to uncompress a PDF, to set the language and to declare a few PDF standards and some color profiles.

`\DocumentMetadata` is also used to activate the new PDF management code and it loads a number of required files for the PDF management code. As this forces the loading of the backend files, a backend which can't be detected automatically like `dvipdfmx`, must be set in the first `\DocumentMetadata` call (if there is more than one).

The full set of keys currently supported is documented in `documentmetadata-support.pdf` for now.

1.2 Storing and retrieving document properties

We provide a storage container where classes, packages and users can store properties and metadata about the current document which may be of interest to other packages or to the user. This data can then be retrieved as necessary by packages or by the document author.

The properties are stored with a key ⟨*label*⟩/⟨*property*⟩. The values can be retrieved expandably.

\AddToDocumentProperties `\AddToDocumentProperties[⟨label⟩]{⟨property⟩}{⟨value⟩}`

This stores ⟨*value*⟩ under the key ⟨*label*⟩/⟨*property*⟩. By default ⟨*label*⟩ is the current package name `\@currname`. If another label is chosen, it should be one which avoids clashes with other packages using the container. The label `document` is used by the PDF management and by the LaTeX kernel and so reserved. If the key ⟨*label*⟩/⟨*property*⟩ already exists in the container, its value is overwritten without warning or error.

\GetDocumentProperty `\GetDocumentProperty{⟨label/property⟩}`

Expands to the ⟨*value*⟩ corresponding to ⟨*label/property*⟩ in the container. If ⟨*label/property*⟩ is missing, this has an empty expansion. In particular, there is no error! The result is returned within `\exp_not:n`, which means that the ⟨*value*⟩ does not expand further when appearing in an e-type or x-type argument expansion.

\ShowDocumentProperties `\ShowDocumentProperties`
\LogDocumentProperties `\LogDocumentProperties`

This shows/logs the current content of the container.

2 The Implementation

2.1 \DocumentMetadata

```
⟨*2ekernel | latexrelease⟩
```

Not needed yet but ...

```
%\ExplSyntaxOn
⟨latexrelease⟩\NewModuleRelease{2022/06/01}{ltmeta}
⟨latexrelease⟩                    {Document Metadata handling}
```

We start by making the conditionals testing for the use of `\DocumentMetadata` execute the false branch. Then, inside `\DocumentMetadata`, we change them to select the true branch.

```
\let \IfDocumentMetadataTF \@secondoftwo
\let \IfDocumentMetadataT \@gobble
\let \IfDocumentMetadataF \@firstofone
\protected\def\DocumentMetadata{%
  \InputIfFileExists{documentmetadata-support.ltx}%
    {%
```

Support for `\DocumentMetadata` is available, so change the conditionals.

```
      \let \IfDocumentMetadataTF \@firstoftwo
      \let \IfDocumentMetadataT  \@firstofone
      \let \IfDocumentMetadataF  \@gobble
    }%
```

The above file is changing `\DocumentMetadata` to a suitable definition (or so we hope) so that it can be safely used several times in the preamble.

If the file can't be found, we say so and carry on without it.

```
    {%
      \@latex@error{No support files for
                    \noexpand\DocumentMetadata found}
        {Is the 'LaTeX-lab' bundle installed?%
        \MessageBreak
```

```
          Without it, the declaration is ignored.}%
```

No point in trying this more than once if there are several calls in the document.

```
      \let\DocumentMetadata\@gobble
    }%
  \DocumentMetadata
}
```

2.2 Document properties

The container for the document properties is a prop:

\g__kernel_documentproperties_prop

```
\ExplSyntaxOn
  \prop_new:N \g__kernel_documentproperties_prop
```

\AddToDocumentProperties

```
\NewDocumentCommand\AddToDocumentProperties{O{\@currname}mm}
  {
    \prop_gput:Nen \g__kernel_documentproperties_prop
      {
        \tl_if_blank:eTF {#1}{top-level/}{#1/} #2
      }
      { #3}
  }
```

(*End of definition for* `\AddToDocumentProperties`. *This function is documented on page* 458.)

\GetDocumentProperty

```
\cs_new:Npn\GetDocumentProperty #1
  {
    \prop_item:Nn \g__kernel_documentproperties_prop {#1}
  }
```

(*End of definition for* `\GetDocumentProperty`. *This function is documented on page* 458.)

\ShowDocumentProperties
\LogDocumentProperties

```
\msg_new:nnn  { latex2e } { show-properties }
  {
    The~following~document~properties~have~been~stored:
    #1
  }
\NewDocumentCommand\ShowDocumentProperties {}
  {
    \msg_show:nne {latex2e}{show-properties}
      {
        \prop_map_function:NN \g__kernel_documentproperties_prop \msg_show_item:nn
      }
  }
\NewDocumentCommand\LogDocumentProperties {}
  {
    \msg_log:nne {latex2e}{show-properties}
      {
        \prop_map_function:NN \g__kernel_documentproperties_prop \msg_show_item:nn
```

```
      }
  }
\ExplSyntaxOff
```

(*End of definition for* `\ShowDocumentProperties` *and* `\LogDocumentProperties`*. These functions are documented on page 458.*)

2.3 Targets

To allow package and class authors to support document links we also provide the new interface commands of the hyperref package for the creation of targets. The commands are not only useful for document links: When tagging is active the commands can also be used to reference structures. This is done in the `recordtarget` socket, which records the target name. Both for links and structure references it can be useful to change the name of the next target. We therefore insert a hook `target/setname/after` after the name has been set. This hook is used in the `\NextLinkTarget` command to adapt the target name `\@currentHref`.

`\MakeLinkTarget`
`\LinkTargetOn`
`\LinkTargetOff`
`\NextLinkTarget`

```
⟨!latexrelease⟩\IncludeInRelease{2026/06/01}%
⟨!latexrelease⟩                    {\MakeLinkTarget}{Record target name for tagging support}%
\ExplSyntaxOn
```

luatex complains about non-PDF specials if `\special` is used, so we use `\latelua` instead to create an empty whatsit node.

```
\ifx\directlua\@undefined
 \def\@kernel@whatsit{\special{}}
\else
 \def\@kernel@whatsit{\latelua{}}
\fi
```

This definition is copied from hyperref, only the command names are changed. It is the code hyperref uses around anchors/destination to avoid affecting the spacing.

```
\let\RestoreLastSkip\relax
\def\SaveLastSkip{%
  \let\RestoreLastSkip\relax
  \ifvmode
    \ifdim\lastskip=\z@
      \ifnum\lastnodetype=11 %
        \let\RestoreLastSkip\nobreak
      \else
        \let\RestoreLastSkip\relax
      \fi
    \else
      \begingroup
        \skip@=-\lastskip
        \edef\x{%
          \endgroup
          \def\noexpand\RestoreLastSkip{%
            \noexpand\ifvmode
              \noexpand\nobreak
              \vskip\the\skip@
              \vskip\the\lastskip\relax
            \noexpand\fi
```

```
          }%
        }%
      \x
    \fi
  \else
    \ifhmode
      \ifdim\lastskip=\z@
        \let\RestoreLastSkip\nobreak
      \else
        \begingroup
          \skip@=-\lastskip
          \edef\x{%
            \endgroup
            \def\noexpand\RestoreLastSkip{%
              \noexpand\ifhmode
                \noexpand\nobreak
                \hskip\the\skip@
                \hskip\the\lastskip\relax
              \noexpand\fi
            }%
          }%
        \x
      \fi
    \fi
  \fi
}
\hook_new:n{target/setname/after}
\int_new:N\g__kernel_target_int
\NewDocumentCommand\MakeLinkTarget{sO{}m}{%
  \ifvmode
    \SaveLastSkip\@kernel@whatsit\RestoreLastSkip
  \else
    \ifhmode
    \@savsf\spacefactor
    \SaveLastSkip\smash{}\RestoreLastSkip
    \spacefactor\@savsf
    \fi
  \fi
  \IfBooleanTF {#1}
   {
     \tl_gset:Ne \@currentHref {#3}
   }
   {
     \int_gincr:N\g__kernel_target_int
     \tl_gset:Ne \@currentHref {target*.\int_use:N\g__kernel_target_int}
   }
  \hook_use:n{target/setname/after}
  \UseTaggingSocket{recordtarget}
   }
\ExplSyntaxOff
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2024/11/01}%
⟨latexrelease⟩                  {\MakeLinkTarget}{Record target name for tagging support}%
⟨latexrelease⟩\ExplSyntaxOn
```

```
⟨latexrelease⟩\NewDocumentCommand\MakeLinkTarget{sO{}m}{%
⟨latexrelease⟩  \ifvmode
⟨latexrelease⟩    \special{}%
⟨latexrelease⟩  \else
⟨latexrelease⟩    \@savsf\spacefactor
⟨latexrelease⟩    \smash{}%
⟨latexrelease⟩    \spacefactor\@savsf
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \IfBooleanTF {#1}
⟨latexrelease⟩   {
⟨latexrelease⟩     \tl_gset:Ne \@currentHref {#3}
⟨latexrelease⟩   }
⟨latexrelease⟩   {
⟨latexrelease⟩     \int_gincr:N\g__kernel_target_int
⟨latexrelease⟩     \tl_gset:Ne \@currentHref {target*.\int_use:N\g__kernel_target_int}
⟨latexrelease⟩   }
⟨latexrelease⟩  \hook_use:n{target/setname/after}
⟨latexrelease⟩  \UseTaggingSocket{recordtarget}
⟨latexrelease⟩   }
⟨latexrelease⟩\ExplSyntaxOff
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2022/06/01}%
⟨latexrelease⟩                  {\MakeLinkTarget}{Record target name for tagging support}%
⟨latexrelease⟩\NewDocumentCommand\MakeLinkTarget{sO{}m}{%
⟨latexrelease⟩  \ifvmode
⟨latexrelease⟩    \special{}%
⟨latexrelease⟩  \else
⟨latexrelease⟩    \@savsf\spacefactor
⟨latexrelease⟩    \smash{}%
⟨latexrelease⟩    \spacefactor\@savsf
⟨latexrelease⟩  \fi}
⟨latexrelease⟩\EndIncludeInRelease
\NewDocumentCommand\LinkTargetOn{}{}
\NewDocumentCommand\LinkTargetOff{}{}
\NewDocumentCommand\NextLinkTarget{m}{%
  \AddToHookNext{target/setname/after}{\xdef\@currentHref {#1}}}
```

(*End of definition for* `\MakeLinkTarget` *and others.*)

We do not undo `\MakeLinkTarget` and friends if we roll back, in case they are used in packages that themselves do not offer rollback. This way a roll forward adds them, but the dummies remain if you roll back and you don't get missing csname errors if they are used.

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{ltmeta}%
⟨latexrelease⟩                  {Undo Document Metadata handling}
⟨latexrelease⟩
⟨latexrelease⟩\let\DocumentMetadata\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndModuleRelease
```

Again for the future ...

```
%\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
```

Restore module prefix (if any):

```
⟨@@=⟩
```

File 18
ltspace.dtx

1 Spacing

This section deals with spacing, and line- and page-breaking.

1.1 User Commands

`\nopagebreak` `[`⟨*i*⟩`]` : ⟨*i*⟩ = 0,...,4.
Default argument = 4. Puts a penalty into the vertical list output as follows:
0 : penalty = 0
1 : penalty = `\@lowpenalty`
2 : penalty = `\@medpenalty`
3 : penalty = `\@highpenalty`
4 : penalty = 10000

`\pagebreak` `[`⟨*i*⟩`]` : same as except negatives of its penalty

`\linebreak` `[`⟨*i*⟩`]` : analog of the above

`\nolinebreak` `[`⟨*i*⟩`]` : analog of the above

`\samepage` : inhibits page breaking most places by setting the following penalties to 10000:

```
\interlinepenalty
\predisplaypenalty
\postdisplaypenalty
\interdisplaylinepenalty
\@beginparpenalty
\@endparpenalty
\@itempenalty
\@secpenalty
\interfootnotelinepenalty
```

`\\` : initially defined to be `\newline`
`\\[`⟨*length*⟩`]` : initially defined to be `\vspace{`⟨*length*⟩`}\newline`
Note: `\\*` adds a `\vadjust{\penalty 10000}`
OBSOLETE COMMANDS (which never made it into the manual):
`\obeycr` : defines <CR> == `\\\relax`
`\restorecr` : restores <CR> to its usual meaning.

1.2 Chris' comments

There are several aspects of the handling of space in horizontal mode that are inconsistent or do not work well in some cases. These are largely concerned with ignoring the effect of space tokens that would otherwise typeset an inter-word space.

Negating the effect of such space tokens is achieved by two mechanisms:

- `\unskip` is used to remove the glue just added by a space that has already had its effect; it is sometimes invoked after an `\ifdim` test on `\lastskip` (see below);
- `\ignorespaces` is used to ignore space-tokens yet to come.

The test done on \lastskip is sometimes for equality with zero and sometimes for being positive. Recall also that the test is only on the natural length of the glue and that no glue cannot be distinguished from glue whose natural length is zero: to summarise, a pretty awful test. It is not clear why these tests are not all the same; I think that they should all be for equality. One place where \unskip is often used is just before a \par (which itself internally does an \unskip) and one bit of code (in \@item) even has two \unskips before a \par. These uses may be fossil code but if they are necessary, maybe \@killglue would be even safer.

Such removal of glue by \unskip may sometimes have the wrong result, removing not the glue from a space-token but other explicit glue; this is sometimes not what is intended.

A common way to prevent such removal is to add an \hskip\z@ after the glue that should not be removed. This protects that glue against one \unskip with no test but not against more than one. It does work for 'tested \unskips'. This is used by \hspace* but not by \hspace; this is inconsistent as the star is supposed to prevent removal only at the beginning of a line, not at the end, or in a tabular, etc.

If this reason for removing glue were the only consideration then a tested-\unskip and protection by \hskip\z@ would suffice but would need to be consistently implemented.

However, the class of invisibles, commands and environments tries to be even cleverer: one of these tries to leave only one inter-word space whenever there is one before it and one after it; and it does this quite well.

But problems can arise when there is not a space-token on both sides of it; in particular, when an invisible appears at the beginning or end of a piece of text the method still leaves one space token whereas usually in these cases it should leave none.

Also, the current rules do not work well when more than one such command appears consecutively, separated by space-tokens; it leaves glue between every other invisible.

There is also a question about what these commands should do when they occur next to spaces that do not come from space tokens but, for example, from \hspace. Should they still produce 'just one space'? If so, which one? It is good to note that the manual is sufficiently cautious about invisibles that we are not obliged to make anything work.

Another interesting side-road to explore is whether the space-tokens either side of an \hspace{...} should be ignored.

One alternative to the current algorithm that is often suggested is that all glue around the invisible should be consolidated into a space after it (usually without stating how much glue should be put there). The command \nolinebreak is implemented this way (and \linebreak should also be). This does not work correctly for the following common case:

```
... some text
\index{some-word}
some-word and more text.
```

This is optimal coding since it is normal to index a word that gets split across a page-break on its starting page. This would, on the other hand, fix another common (and documented) failure of the current system: when the invisible is the last thing in a paragraph the space before it is not removed and, worse, it is also hidden from the paragraph-ending mechanism so that an 'empty' line can be created at the end of the paragraph.

Another deficiency (I think) of the current system is that the following is treated as having the `\index` command between the paragraphs, which is probably not what the author intended (since there is no empty line after it).

```
\index{beginnings}
Beginnings of paragraphs ...
```

I know of no algorithm that will handle satisfactorily even all the most common cases; note that it could be that the best algorithm may be different for different invisibles since, for example, the common uses and expected behaviour of `\index`, `\marginpar`, `\linebreak`, `\pagebreak` and `\vspace` are somewhat different. [For example, is `\vspace` ever used in the middle of a paragraph?]

One method that can (and is) used to make invisible commands produce no space when used at the beginning of text is to put in some glue that is nearly enough the same as no glue or glue of zero length in all respects except for the precise test for not being exactly equal to zero; examples of such glue are `\hskip 1sp` and, possibly better but more complex, `\hskip -1sp \hskip 1sp`. However, this only works when it is known that user-supplied text is about to start.

Some similar concerns apply to the handling of space and penalties in vertical mode; there is an extra hurdle here as `\unskip` does not work on the main vertical list. The complexity of the tests done by `\addvspace` have never been explained.

The implementation of space hacks etc for vertical mode is another major area that needs further attention; my earlier experiments did not produce much improvement over the current unsatisfactory situation.

One particular problem is what happens when the following very natural coding is used (part of the problem here is that this looks like an hmode problem, but it is not):

```
... end of text.

\begin{enumerate}
  \item \label{item:xxx} Item text.
\end{enumerate}
```

1.3 Some immediate actions

- Fix bug in `\linebreak`.
- Fix bug in `\\*`.
- Reimplement `\\`, etc, removing extra `\vadjust`s and getting better error trapping (this seems to involve a lot more tokens).
- Investigate whether `\\`, etc need to be errors in vmode; I think that they could be noops (maybe with a warning).
- Make all(?) `\unskips` include test for zero skip (rather than other tests or no test).
- Consider replacing `\hskip 1sp` by something better (here called an 'infinitesimal' skip).
- Look at all `\hskip\z@` (or similar) to see if they should be changed to an 'infinitesimal' skip.

- Resolve the inconsistency between `\hspace` and `\hspace*`.
- Remove unnecessary `\unskips`.
- Investigate and rationalize the 'newline' code.
- Find better algorithms for all sorts of things or, easier(?), fix TEX itself.

1.4 The code

```
⟨*2ekernel⟩
\message{spacing,}
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                    {\pagebreak}{Make commands robust}%
```

`\pagebreak`
`\nopagebreak`

```
\DeclareRobustCommand\pagebreak{\@testopt{\@no@pgbk-}4}
\DeclareRobustCommand\nopagebreak{\@testopt\@no@pgbk4}
```

(*End of definition for* `\pagebreak` *and* `\nopagebreak`.)

`\linebreak`
`\nolinebreak`

```
\DeclareRobustCommand\linebreak{\@testopt{\@no@lnbk-}4}
\DeclareRobustCommand\nolinebreak{\@testopt\@no@lnbk4}
```

(*End of definition for* `\linebreak` *and* `\nolinebreak`.)

`\samepage`

```
\DeclareRobustCommand\samepage{\interlinepenalty\@M
   \predisplaypenalty\@M
   \postdisplaypenalty\@M
   \interdisplaylinepenalty\@M
   \@beginparpenalty\@M
   \@endparpenalty\@M
   \@itempenalty\@M
   \@secpenalty\@M
   \interfootnotelinepenalty\@M}
```

(*End of definition for* `\samepage`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\pagebreak}{Make commands robust}%
⟨latexrelease⟩
⟨latexrelease⟩\kernel@make@fragile\pagebreak
⟨latexrelease⟩\kernel@make@fragile\nopagebreak
⟨latexrelease⟩\kernel@make@fragile\linebreak
⟨latexrelease⟩\kernel@make@fragile\nolinebreak
⟨latexrelease⟩\kernel@make@fragile\samepage
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

\@no@pgbk

```
\def\@no@pgbk #1[#2]{%
  \ifvmode
    \penalty #1\@getpen{#2}%
  \else
    \@bsphack
    \vadjust{\penalty #1\@getpen{#2}}%
    \@esphack
  \fi}
```

(*End of definition for* `\@no@pgbk`.)

\@no@lnbk

```
\def\@no@lnbk #1[#2]{%
  \ifvmode
    \@nolnerr
  \else
    \@tempskipa\lastskip
    \unskip
    \penalty #1\@getpen{#2}%
    \ifdim\@tempskipa>\z@
      \hskip\@tempskipa
      \ignorespaces
    \fi
  \fi}
```

(*End of definition for* `\@no@lnbk`.)

\\ The purpose of the new code is to fix a few bugs; however, it also attempts to optimize the following, in order of priority:

1. efficient execution of plain `\\`;
2. efficient execution of `\\[...]`;
3. memory use;
4. name-space use.

The changes should make no difference to the typeset output. It appears to be safe to use `\reserved@e` and `\reserved@f` here (other reserved macros are somewhat disastrous).

These changes made `\newline` even less robust than it had been, so now it is explicitly robust, like `\\`.

The internal definition of the 'normal' definition of `\\`.

\@normalcr

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩                  {\@normalcr}{Make robust}%
\protected\def\@normalcr{%
  \let \reserved@e \relax
  \let \reserved@f \relax
  \@ifstar{\let \reserved@e \vadjust \let \reserved@f \nobreak
             \@xnewline}%
  \@xnewline}
```

```
\let\\\@normalcr
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@normalcr}{Make robust}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\\{%
⟨latexrelease⟩   \let \reserved@e \relax
⟨latexrelease⟩   \let \reserved@f \relax
⟨latexrelease⟩   \@ifstar{\let \reserved@e \vadjust \let \reserved@f \nobreak
⟨latexrelease⟩             \@xnewline}%
⟨latexrelease⟩  \@xnewline}
⟨latexrelease⟩\expandafter\let\expandafter\@normalcr
⟨latexrelease⟩     \csname\expandafter\@gobble\string\\ \endcsname
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\\` *and* `\@normalcr`.)

`\@vspace@calcify` Helper command to produce a `\vskip` that is first run through `\setlength`. This way the `calc` package can operate on the argument value.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\@vspace@calcify}{Add calc support}%
\def\@vspace@calcify#1{\setlength\sp@ce@skip{#1}\vskip\sp@ce@skip}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@vspace@calcify}{Add calc support}%
⟨latexrelease⟩
⟨latexrelease⟩\let\@vspace@calcify\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@vspace@calcify`.)

`\newline` A simple form of the 'normal' definition of `\\`.

```
\DeclareRobustCommand\newline{\@normalcr\relax}
```

(*End of definition for* `\newline`.)

`\@xnewline`

```
\def\@xnewline{\@ifnextchar[% ] bracket matching
                   \@newline
                  {\@gnewline\relax}}
```

(*End of definition for* `\@xnewline`.)

`\@newline`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
```

```
⟨latexrelease⟩                  {\@newline}{\newline calc support}%
\def\@newline[#1]{\let \reserved@e \vadjust
                    \@gnewline {\@vspace@calcify{#1}}}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@newline}{\newline calc support}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@newline[#1]{\let \reserved@e \vadjust
⟨latexrelease⟩                    \@gnewline {\vskip #1}}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@newline`.)

`\@gnewline` The `\nobreak` added to prevent null lines when `\\` ends an overfull line. Change made 24 May 89 as suggested by Frank Mittelbach and Rainer Schöpf

```
\def\@gnewline #1{%
  \ifvmode
    \@nolnerr
  \else
    \unskip \reserved@e {\reserved@f#1}\nobreak \hfil \break
  \fi}
```

(*End of definition for* `\@gnewline`.)

`\@getpen`

```
\def\@getpen#1{\ifcase #1 \z@ \or \@lowpenalty\or
         \@medpenalty \or \@highpenalty
         \else \@M \fi}
```

(*End of definition for* `\@getpen`.)

`\if@nobreak` Switch used to avoid page breaks caused by `\label` after a section heading, etc. It should be **GLOBALLY** set true after the `\nobreak` and **globally** set false by the next invocation of `\everypar`.

Commands that reset `\everypar` should globally set it false if appropriate.

```
\def\@nobreakfalse{\global\let\if@nobreak\iffalse}
\def\@nobreaktrue {\global\let\if@nobreak\iftrue}
\@nobreakfalse
```

(*End of definition for* `\if@nobreak`.)

`\@savsk` Registers used to save the space factor and last skip.

`\@savsf`

```
\newdimen\@savsk
\newcount\@savsf
```

(*End of definition for* `\@savsk` *and* `\@savsf`.)

\@bsphack

\@bsphack and \@esphack used by macros such as \index and \begin{@float} ... \end{@float} that want to be invisible — i.e., not leave any extra space when used in the middle of text. Such a macro should begin with \@bsphack and end with \@esphack. The macro in question should not create any text, nor change the mode.

Before giving the current definition we give an extended definition that is currently not used (because it doesn't work as advertised:-)

These are generalized hacks which attempt to do sensible things when 'invisible commands' appear in vmode too.

They need to cope with space in both hmode (plus spacefactor) and vmode, and also cope with breaks etc. In vmode this means ensuring that any following \addvspace, etc sees the correct glue in \lastskip.

In fact, these improved versions should be used for other cases of 'whatsits, thingies etc' which should be invisible. They are only for commands, not environments (see notes on \@Esphack).

BTW, anyone know why the standard hacks are surrounded by \ifmmode\else rather than simply \ifhmode?

And are there any cases where saving the spacefactor is essential? I have some extensions where it is, but it does not appear to be so in the standard uses.

```
def \@bsphack{%
 \relax \ifvmode
   \@savsk \lastskip
   \ifdim \lastskip=\z@
   \else
     \vskip -\lastskip
   \fi
 \else
   \ifhmode
     \@savsk \lastskip
     \@savsf \spacefactor
   \fi
 \fi
```

I think that, in vmode, it is the safest to put in a \nobreak immediately after such things since writes, inserts etc followed by glue give valid breakpoints and, in general, it is possible to create breaks but impossible to destroy them.

```
def \@esphack{%
  \relax \ifvmode
    \nobreak
    \ifdim \@savsk=\z@
    \else
      \vskip\@savsk
    \fi
  \else
    \ifhmode
      \spacefactor \@savsf
      \ifdim \@savsk>\z@
        \ignorespaces
      \fi
```

```
    \fi
  \fi
```

For the moment we are going to ignore the vertical versions until they are correct.

```
\def\@bsphack{%
  \relax
  \ifhmode
    \@savsk\lastskip
    \@savsf\spacefactor
  \fi}
```

(*End of definition for* `\@bsphack`.)

`\@esphack` Companion to `\@bsphack`. If this command is not properly paired with `\@bsphack` one might end up with a low-level TEX error: "BAD spacefactor". One possible cause is calling `\@bsphack` in vertical mode, then doing something that gets you (sometimes) into horizontal mode and finally calling `\@esphack`. Even if no error is generated that is wrong, because `\@esphack` will then use the saved values for `\@savsk` and `\@savsf` from some earlier invocation of `\@bsphack` which will have nothing to do with the current situation.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2026/06/01}%
⟨latexrelease⟩                    {\@esphack}{chained space hacks}%
⟨*2ekernel | latexrelease⟩
\def\@esphack{%
  \relax
  \ifhmode
    \spacefactor\@savsf
    \ifdim\@savsk>\z@
      \ifdim\lastskip=\z@
```

If there is more than one space hack in succession (with a space in between) LATEX still ended up with more than one space because the `\z@skip` prevented the second space hack from recognizing that no space should be added. To avoid this we add a special "zero" space that looks like a positive space to `\lastskip`.

There is a side effect though: if the space hack is the last thing in a paragraph the `1sp` is dropped and we actually allow the last line to be over by 1sp, but this is better than the extra space introduced in the past.

```
        \nobreak
%       \hskip\z@skip
        \hskip-1sp\relax\hskip1sp\relax
      \fi
      \ignorespaces
    \fi
  \else
    \ifvmode
      \if@nobreak\nobreak\else\if@noskipsec\nobreak\fi\fi
    \fi
```

```
  \fi
}%
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2018/10/10}%
⟨latexrelease⟩                  {\@esphack}{hyphenation and nobreak after space hack}%

⟨latexrelease⟩\def\@esphack{%
⟨latexrelease⟩  \relax
⟨latexrelease⟩  \ifhmode
⟨latexrelease⟩    \spacefactor\@savsf
⟨latexrelease⟩    \ifdim\@savsk>\z@
⟨latexrelease⟩      \ifdim\lastskip=\z@
⟨latexrelease⟩        \nobreak \hskip\z@skip
⟨latexrelease⟩      \fi
⟨latexrelease⟩      \ignorespaces
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \else
⟨latexrelease⟩    \ifvmode
⟨latexrelease⟩      \if@nobreak\nobreak\else\if@noskipsec\nobreak\fi\fi
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \fi}%
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2015/10/01}%
⟨latexrelease⟩                  {\@esphack}{hyphenation and nobreak after space hack}%
⟨latexrelease⟩\def\@esphack{%
⟨latexrelease⟩  \relax
⟨latexrelease⟩  \ifhmode
⟨latexrelease⟩    \spacefactor\@savsf
⟨latexrelease⟩    \ifdim\@savsk>\z@
⟨latexrelease⟩      \ifdim\lastskip=\z@
⟨latexrelease⟩        \nobreak \hskip\z@skip
⟨latexrelease⟩      \fi
⟨latexrelease⟩      \ignorespaces
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \fi}%
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩                  {\@esphack}{hyphenation and nobreak after space hack}%
⟨latexrelease⟩\def\@esphack{%
⟨latexrelease⟩  \relax
⟨latexrelease⟩  \ifhmode
⟨latexrelease⟩    \spacefactor\@savsf
⟨latexrelease⟩    \ifdim\@savsk>\z@
⟨latexrelease⟩      \nobreak \hskip\z@skip
⟨latexrelease⟩      \ignorespaces
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \fi}%
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@esphack}{hyphenation and nobreak after space hack}%
⟨latexrelease⟩\def\@esphack{%
⟨latexrelease⟩  \relax
```

```
⟨latexrelease⟩   \ifhmode
⟨latexrelease⟩     \spacefactor\@savsf
⟨latexrelease⟩     \ifdim\@savsk>\z@
⟨latexrelease⟩       \ignorespaces
⟨latexrelease⟩     \fi
⟨latexrelease⟩   \fi}%
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@esphack`.)

`\@Esphack` A variant of `\@esphack` that sets the `@ignore` switch to true (as `\@esphack` used to do previously). This is currently used only for floats and similar environments.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩                  {\@Esphack}{hyphenation after space hack}%
⟨*2ekernel | latexrelease⟩
\def\@Esphack{%
  \relax
  \ifhmode
    \spacefactor\@savsf
    \ifdim\@savsk>\z@
      \nobreak \hskip\z@skip
      \@ignoretrue
      \ignorespaces
    \fi
   \fi}%
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@Esphack}{hyphenation after space hack}%
⟨latexrelease⟩\def\@Esphack{%
⟨latexrelease⟩   \relax
⟨latexrelease⟩   \ifhmode
⟨latexrelease⟩     \spacefactor\@savsf
⟨latexrelease⟩     \ifdim\@savsk>\z@
⟨latexrelease⟩       \@ignoretrue
⟨latexrelease⟩       \ignorespaces
⟨latexrelease⟩     \fi
⟨latexrelease⟩    \fi}%
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@Esphack`.)

`\@vbsphack` Another variant which is useful for invisible things which should not live in vmode (this is how some people feel about marginals).

If it occurs in vmode then it enters hmode and ensures that `\@savsk` is nonzero so that the `\ignorespaces` is put in later. It is not used at present.

```
\def \@vbsphack{ %
   \relax \ifvmode
     \leavevmode
     \@savsk 1sp
```

```
      \@savsf \spacefactor
    \else
      \ifhmode
        \@savsk \lastskip
        \@savsf \spacefactor
      \fi
    \fi
}
```

(*End of definition for* `\@vbsphack`.)

1.5 Vertical spacing

LaTeX supports the plain TeX commands `\smallskip`, `\medskip` and `\bigskip`. However, it redefines them using `\vspace` instead of `\vskip`.

Extra vertical space is added by the command `\addvspace{`⟨*skip*⟩`}`, which adds a vertical skip of ⟨*skip*⟩ to the document. The sequence
`\addvspace{`⟨*s1*⟩`}` `\addvspace{`⟨*s2*⟩`}` is equivalent to
`\addvspace{`⟨*maximum of s1, s2*⟩`}`.

`\addvspace` should be used only in vertical mode, and gives an error if it's not. The `\addvspace` command does *not* add vertical space if `@minipage` is true. The minipage environment uses this to inhibit the addition of extra vertical space at the beginning.

Penalties are put into the vertical list with the `\addpenalty{`⟨*penalty*⟩`}` command. It works properly when `\addpenalty` and `\addvspace` commands are mixed.

The `@nobreak` switch is set true used when in vertical mode and no page break should occur. (Right now, it is used only by the section heading commands to inhibit page breaking after a heading.)

`\@xaddvskip` Internal macro for `\vspace` handling the case that space has previously been added.

```
\def\@xaddvskip{%
  \ifdim\lastskip<\@tempskipb
    \vskip-\lastskip
    \vskip\@tempskipb
  \else
    \ifdim\@tempskipb<\z@
      \ifdim\lastskip<\z@
      \else
        \advance\@tempskipb\lastskip
        \vskip-\lastskip
        \vskip \@tempskipb
      \fi
    \fi
  \fi}
```

(*End of definition for* `\@xaddvskip`.)

`\addvspace` Add vertical space taking into account space already added, as described above.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2024/11/01}%
⟨latexrelease⟩                    {\addvspace}{drop unnecessary no-item error}%
\protected\def\addvspace#1{%
```

When this is encountered in hmode, we check whether we are in an hbox and if so generate a LaTeX error, as otherwise this would cause a bunch of low-level errors. In unrestricted hmode we simply switch to vmode by issuing a `\par`.

```
  \ifhmode \ifinner \@LRmoderr \else \par \fi \fi

  \if@minipage\else
    \ifdim \lastskip =\z@
      \@vspace@calcify{#1}%
    \else
    \setlength\@tempskipb{#1}%
      \@xaddvskip
    \fi
  \fi
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\addvspace}{\addvspace calc support}%
⟨latexrelease⟩\def\addvspace#1{%
⟨latexrelease⟩  \ifvmode
⟨latexrelease⟩     \if@minipage\else
⟨latexrelease⟩       \ifdim \lastskip =\z@
⟨latexrelease⟩         \@vspace@calcify{#1}%
⟨latexrelease⟩       \else
⟨latexrelease⟩       \setlength\@tempskipb{#1}%
⟨latexrelease⟩         \@xaddvskip
⟨latexrelease⟩       \fi
⟨latexrelease⟩     \fi
⟨latexrelease⟩  \else
⟨latexrelease⟩    \@noitemerr
⟨latexrelease⟩  \fi}
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\addvspace}{\addvspace calc support}%
⟨latexrelease⟩
⟨latexrelease⟩\def\addvspace#1{%
⟨latexrelease⟩  \ifvmode
⟨latexrelease⟩     \if@minipage\else
⟨latexrelease⟩       \ifdim \lastskip =\z@
⟨latexrelease⟩         \vskip #1\relax
⟨latexrelease⟩       \else
⟨latexrelease⟩       \@tempskipb#1\relax
⟨latexrelease⟩         \@xaddvskip
⟨latexrelease⟩       \fi
⟨latexrelease⟩     \fi
⟨latexrelease⟩  \else
⟨latexrelease⟩    \@noitemerr
⟨latexrelease⟩  \fi}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\addvspace`.)

\addpenalty

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2024/11/01}%
⟨latexrelease⟩                    {\addpenalty}{\addpenalty drop error}%
⟨*2ekernel | latexrelease⟩
\protected\def\addpenalty#1{%
```

See description of `\addvspace` for documentation of the next line of code.

```
  \ifhmode \ifinner \@LRmoderr \else \par \fi \fi
```

Fix provided by Donald (though the original fix was not good enough). In 2005 Plamen Tanovski discovered that this fix wasn't good enough either as the `\vskip` kept getting bigger if several `\addpenalty` commands followed each other. Donald kindly send a new fix.

```
  \if@minipage
  \else
    \if@nobreak
    \else
      \ifdim\lastskip=\z@
        \penalty#1\relax
      \else
        \@tempskipb\lastskip
```

We have to make sure the final `\vskip` seen by TEX is the correct one, namely `\@tempskipb`. However, we may have to adjust for `\prevdepth` when placing the penalty; that should not affect the skip we pass on to TEX.

```
        \begingroup
          \@tempskipa\@tempskipb
          \advance \@tempskipb
            \ifdim\prevdepth>\maxdepth\maxdepth\else
```

If `\prevdepth` is -1000pt due to `\nointerlineskip` we better not add it!

```
               \ifdim \prevdepth = -\@m\p@ \z@ \else \prevdepth \fi
            \fi
          \vskip -\@tempskipb
          \penalty#1%
          \ifdim\@tempskipa=\@tempskipb
```

Do nothing if the `\prevdepth` check made no adjustment.

```
          \else
```

Combine the prevdepth adjustment into a single skip.

```
            \advance\@tempskipb -\@tempskipa
            \vskip \@tempskipb
          \fi
```

The final skip is always the specified length.

```
          \vskip \@tempskipa
        \endgroup
      \fi
    \fi
  \fi
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩                  {\addpenalty}{\addpenalty}%
⟨latexrelease⟩\def\addpenalty#1{%
⟨latexrelease⟩  \ifvmode
⟨latexrelease⟩    \if@minipage
⟨latexrelease⟩    \else
⟨latexrelease⟩      \if@nobreak
⟨latexrelease⟩      \else
⟨latexrelease⟩        \ifdim\lastskip=\z@
⟨latexrelease⟩          \penalty#1\relax
⟨latexrelease⟩        \else
⟨latexrelease⟩          \@tempskipb\lastskip
⟨latexrelease⟩          \begingroup
⟨latexrelease⟩            \@tempskipa\@tempskipb
⟨latexrelease⟩            \advance \@tempskipb
⟨latexrelease⟩              \ifdim\prevdepth>\maxdepth\maxdepth\else
⟨latexrelease⟩                 \ifdim \prevdepth = -\@m\p@ \z@ \else \prevdepth \fi
⟨latexrelease⟩               \fi
⟨latexrelease⟩             \vskip -\@tempskipb
⟨latexrelease⟩             \penalty#1%
⟨latexrelease⟩             \ifdim\@tempskipa=\@tempskipb
⟨latexrelease⟩             \else
⟨latexrelease⟩               \advance\@tempskipb -\@tempskipa
⟨latexrelease⟩               \vskip \@tempskipb
⟨latexrelease⟩             \fi
⟨latexrelease⟩             \vskip \@tempskipa
⟨latexrelease⟩          \endgroup
⟨latexrelease⟩        \fi
⟨latexrelease⟩      \fi
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \else
⟨latexrelease⟩    \@noitemerr
⟨latexrelease⟩  \fi}%
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\addpenalty}{\addpenalty}%
⟨latexrelease⟩\def\addpenalty#1{%
⟨latexrelease⟩  \ifvmode
⟨latexrelease⟩    \if@minipage
⟨latexrelease⟩    \else
⟨latexrelease⟩      \if@nobreak
⟨latexrelease⟩      \else
⟨latexrelease⟩        \ifdim\lastskip=\z@
⟨latexrelease⟩          \penalty#1\relax
⟨latexrelease⟩        \else
⟨latexrelease⟩          \@tempskipb\lastskip
⟨latexrelease⟩          \vskip -\lastskip
⟨latexrelease⟩          \penalty#1%
⟨latexrelease⟩          \vskip\@tempskipb
⟨latexrelease⟩        \fi
⟨latexrelease⟩      \fi
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \else
⟨latexrelease⟩    \@noitemerr
```

```
⟨latexrelease⟩  \fi}%
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\addpenalty`.)

`\vspace`
`\@vspace`
`\@vspacer`

The new code for these commands depends on the following facts:

- The value of prevdepth is changed only when a box or rule is created and added to a vertical list;
- The value of prevdepth is used only when a box is created and added to a vertical list;
- The value of prevdepth is always local to the building of one vertical list.

```
\DeclareRobustCommand\vspace{\@ifstar\@vspacer\@vspace}
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\@vspace}{Support calc in \vspace}%
```

We support calc syntax in the argument and therefore use `\setlength`.

```
\def\@vspace #1{%
  \ifvmode
    \@vspace@calcify{#1}%
    \vskip\z@skip
   \else
     \@bsphack
     \vadjust{\@restorepar
              \@vspace@calcify{#1}%
              \vskip\z@skip
              }%
     \@esphack
   \fi}
\def\@vspacer#1{%
  \ifvmode
    \dimen@\prevdepth
    \hrule \@height\z@
    \nobreak
    \@vspace@calcify{#1}%
    \vskip\z@skip
    \prevdepth\dimen@
  \else
    \@bsphack
    \vadjust{\@restorepar
             \hrule \@height\z@
             \nobreak
             \@vspace@calcify{#1}%
             \vskip\z@skip}%
    \@esphack
  \fi}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
```

```
⟨latexrelease⟩                {\@vspace}{Support calc in \vspace}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@vspace #1{%
⟨latexrelease⟩  \ifvmode
⟨latexrelease⟩    \vskip #1
⟨latexrelease⟩    \vskip\z@skip
⟨latexrelease⟩   \else
⟨latexrelease⟩     \@bsphack
⟨latexrelease⟩     \vadjust{\@restorepar
⟨latexrelease⟩              \vskip #1
⟨latexrelease⟩              \vskip\z@skip
⟨latexrelease⟩              }%
⟨latexrelease⟩     \@esphack
⟨latexrelease⟩   \fi}
⟨latexrelease⟩\def\@vspacer#1{%
⟨latexrelease⟩  \ifvmode
⟨latexrelease⟩    \dimen@\prevdepth
⟨latexrelease⟩    \hrule \@height\z@
⟨latexrelease⟩    \nobreak
⟨latexrelease⟩    \vskip #1
⟨latexrelease⟩    \vskip\z@skip
⟨latexrelease⟩    \prevdepth\dimen@
⟨latexrelease⟩  \else
⟨latexrelease⟩    \@bsphack
⟨latexrelease⟩    \vadjust{\@restorepar
⟨latexrelease⟩             \hrule \@height\z@
⟨latexrelease⟩             \nobreak
⟨latexrelease⟩             \vskip #1
⟨latexrelease⟩             \vskip\z@skip}%
⟨latexrelease⟩    \@esphack
⟨latexrelease⟩  \fi}
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\vspace`, `\@vspace`, *and* `\@vspacer`.)

`\smallskip` `\medskip` `\bigskip`

```
\def\smallskip{\vspace\smallskipamount}
\def\medskip{\vspace\medskipamount}
\def\bigskip{\vspace\bigskipamount}
```

(*End of definition for* `\smallskip`, `\medskip`, *and* `\bigskip`.)

`\smallskipamount` `\medskipamount` `\bigskipamount`

```
\newskip\smallskipamount \smallskipamount=3pt plus 1pt minus 1pt
\newskip\medskipamount   \medskipamount  =6pt plus 2pt minus 2pt
\newskip\bigskipamount   \bigskipamount =12pt plus 4pt minus 4pt
```

(*End of definition for* `\smallskipamount`, `\medskipamount`, *and* `\bigskipamount`.)

1.6 Horizontal space (and breaks)

`\nobreakdashes` This idea is borrowed from the amsmath package but here we define a robust command.

This command is a low-level command designed for use only before hyphens or dashes (such as `-`, `--`, or `---`).

It could probably be better implemented: it may need its own private token register and temporary command.

Setting the hyphen in a box and then unboxing it means that the normal penalty will not be added after it—and if the penalty is not there a break will not be taken (unless an explicit penalty or glue follows, thus the final `\nobreak`).

Note that even if it is not followed by a '-', it still leaves vmode and sets the space-factor; so use it carefully!

```
\DeclareRobustCommand{\nobreakdashes}{%
  \leavevmode
  \toks@{}%
  \def\reserved@a##1{\toks@\expandafter{\the\toks@-}%
                     \futurelet\@let@token \reserved@b}%
  \def\reserved@b   {\ifx\@let@token -%
                         \expandafter\reserved@a
                     \else
                       \setbox\z@ \hbox{\the\toks@\nobreak}%
                       \unhbox\z@
                       \spacefactor\sfcode`\-
                     \fi}%
  \futurelet\@let@token \reserved@b
}
```

(*End of definition for* `\nobreakdashes`.)

`\nobreakspace` `\@xobeysp`

This is a robust command that produces a horizontal space at which, in paragraph-mode, a line-break is not possible. We then define an active `~` to expand to it since this is the documented behaviour of `~`. One reason for introducing this is that some 8-bit input encodings have a slot for such a space and we do not want to use active characters as the LaTeX internal commands.

The braces in the definition of `~` are needed to ensure that a following space is preserved when reading to/from internal files.

We need to keep `\@xobeysp` as it is widely used; so here it is let to the non-robust command `\nobreakspace` .

The fragile version of `~` needs a brace group after `\nobreakspace` to prevent loss of spaces if it occurs in an expansion context. That's not an issue with the updated `\protected` definition, so we keep the code shorter and avoid that.

```
\DeclareRobustCommand{\nobreakspace}{%
   \leavevmode\nobreak\ }
\catcode `\~=13
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2023/11/01}%
⟨latexrelease⟩                    {\tilde}{Protected tilde}%
⟨*2ekernel | latexrelease⟩
\protected\edef~{%
  \noexpand\ifincsname\noexpand\expandafter\string~%
  \noexpand\else
    \noexpand\expandafter\noexpand\nobreakspace
  \noexpand\fi
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
```

```
⟨latexrelease⟩                  {\tilde}{Protected tilde}%
⟨latexrelease⟩\def~{\nobreakspace{}}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
\expandafter\let\expandafter\@xobeysp\csname nobreakspace \endcsname
```

(*End of definition for* `\nobreakspace` *and* `\@xobeysp`.)

`\@xobeytab` Equivalent to the space case with the default settings.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2023/11/01}%
⟨latexrelease⟩                  {\@xobeytab}{Obeyed tabs}%
⟨*2ekernel | latexrelease⟩
\let\@xobeytab\@xobeysp
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@xobeytab}{Obeyed tabs}%
⟨latexrelease⟩\let\@xobeytab\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@xobeytab`.)

`\@` Placed before a '.', makes it a sentence-ending period. Does the right thing for other punctuation marks as well. Does this by setting spacefactor to 1000.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩                  {\@}{Space after \@}%
⟨*2ekernel | latexrelease⟩
\def\@{\spacefactor\@m{}}%
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@}{Space after \@}%
⟨latexrelease⟩\def\@{\spacefactor\@m}%
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@`.)

`\hspace`

```
\DeclareRobustCommand\hspace{\@ifstar\@hspacer\@hspace}
```

(*End of definition for* `\hspace`.)

`\@hspace`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\@hspace}{Support calc with \hspace}%
```

We use a private register to calculate the space (if calc is used). Previously we used a group but that results in `\everypar` etc. being executed inside the group if the `\hspace` starts a paragraph. This is a bug fix so we do not provide rollback to the incorrect intermediate version.

```
\newskip\sp@ce@skip
\def\@hspace#1{\setlength\sp@ce@skip{#1}\hskip\sp@ce@skip}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\@hspace}{Support calc with \hspace}%

⟨latexrelease⟩
⟨latexrelease⟩\def\@hspace#1{\hskip #1\relax}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@hspace`.)

`\@hspacer` Extra `\hskip 0pt` added 1985/17/12 to guard against a following `\unskip \relax` added 13 Oct 88 for usual TEX lossage replaced both changes by `\hskip\z@skip` 27 Nov 91

```
\def\@hspacer#1{\vrule \@width\z@\nobreak
                \@hspace{#1}\hskip \z@skip}
```

(*End of definition for* `\@hspacer`.)

`\fill`

```
\newskip\fill
\fill = 0pt plus 1fill
```

(*End of definition for* `\fill`.)

`\stretch`

```
\def\stretch#1{\z@ \@plus #1fill\relax}
```

(*End of definition for* `\stretch`.)

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2018/12/01}%
⟨latexrelease⟩                    {\thinspace}{Start LR-mode}%
```

`\enspace`

```
\DeclareRobustCommand\enspace{\leavevmode@ifvmode\kern.5em }
```

(*End of definition for* `\enspace`.)

`\leavevmode@ifvmode` Leave vmode but only if we are really in vmode, otherwise the expansion is empty (which is not the case with the default definition).

```
\protected\def\leavevmode@ifvmode{\ifvmode\expandafter\indent\fi}
```

(*End of definition for* `\leavevmode@ifvmode`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\thinspace}{Start LR-mode}%
⟨latexrelease⟩\def\thinspace{\kern .16667em }
⟨latexrelease⟩\def\negthinspace{\kern-.16667em }
⟨latexrelease⟩\def\enspace{\kern.5em }
⟨latexrelease⟩\let\leavevmode@ifvmode\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

`\enskip` `\quad` `\qquad`

```
\def\enskip{\hskip.5em\relax}
\def\quad{\hskip1em\relax}
\def\qquad{\hskip2em\relax}
```

(*End of definition for* `\enskip`, `\quad`, *and* `\qquad`.)

For Unicode engines, make the Unicode soft hyphen an active character defined as `\-`.

```
\ifx\Umathcode\@undefined\else
  \catcode "AD=13
  \def^^ad{\-}
\fi
```

`\obeycr` `\restorecr`

The following definitions will probably get deleted or moved to compatibility mode soon.

```
{\catcode`\^^M=13 \gdef\obeycr{\catcode`\^^M13 \def^^M{\\\relax}%
    \@gobblecr}%
{\catcode`\^^M=13 \gdef\@gobblecr{\@ifnextchar
\@gobble\ignorespaces}}
\gdef\restorecr{\catcode`\^^M5 }}
```

(*End of definition for* `\obeycr` *and* `\restorecr`.)

```
⟨/2ekernel⟩
```

File 19
ltlogos.dtx

1 Logos

Various logos are defined here.

`\TeX` The TEX logo, adjusted so that a full stop after the logo counts as ending a sentence.

```
⟨*2ekernel⟩
\DeclareRobustCommand\TeX{T\kern-.1667em\lower.5ex\hbox{E}\kern-.125emX\@}
```

(*End of definition for* `\TeX`.)

`\LaTeX` The LATEX logo.

```
\DeclareRobustCommand{\LaTeX}{L\kern-.36em%
        {\sbox\z@ T%
         \vbox to\ht\z@{\hbox{\check@mathfonts
                              \fontsize\sf@size\z@
                              \math@fontsfalse\selectfont
                              A}%
                        \vss}%
        }%
        \kern-.15em%
        \TeX}
```

(*End of definition for* `\LaTeX`.)

`\LaTeXe` The LATEX 2$_\varepsilon$ logo as proposed by A-W designers.

```
\DeclareRobustCommand{\LaTeXe}{\mbox{\m@th
  \if b\expandafter\@car\f@series\@nil\boldmath\fi
  \LaTeX\kern.15em2$_{\textstyle\varepsilon}$}}
⟨/2ekernel⟩
```

(*End of definition for* `\LaTeXe`.)

File 20
ltfiles.dtx

1 File Handling

The following user commands are defined in this part:

`\document` (ie `\begin{document}`)
Reads in the .AUX files and `\catcode`'s `@` to 12.

`\nofiles`
Suppresses all file output by setting `\@filesw` false.

`\includeonly` `{`⟨*NAME1, ... ,NAMEn*⟩`}`
Causes only parts NAME1, ... ,NAMEn to be read by their `\include` commands. Works by setting partsw true and setting `\@partlist` to NAME1, ... ,NAMEn.

`\include` `{`⟨*NAME*⟩`}`
Does an `\input` NAME unless `\@partsw` is true and NAME is not in `\@partlist`. If `\@filesw` is true, then it directs .AUX output to NAME.AUX, including a checkpoint at the end.

`\input` `{`⟨*NAME*⟩`}`
The same as TeX's `\input`, except it allows optional braces around the file name. In LaTeX 2ε, it also avoids the primitive 'missing file' error, if the file can not be found.

`\IfFileExists` `{`⟨*NAME*⟩`}{`⟨*then*⟩`}{`⟨*else*⟩`}`
If the file exists on the system, execute *then* otherwise execute *else*.

`\InputIfFileExists` `{`⟨*NAME*⟩`}{`⟨*then*⟩`}{`⟨*else*⟩`}`
If the file exists on the system, execute *then* and input *NAME* otherwise execute *else*.

Historical LaTeX 2.09 comments (not necessarily accurate any more):

```
⟨*2ekernel⟩
\message{files,}
```

VARIABLES, SWITCHES AND INTERNAL COMMANDS:

`\@mainaux`	: Output file number for main .AUX file.
`\@partaux`	: Output file number for current part's .AUX file.
`\@auxout`	: Either `\@mainout` or `\@partout`, depending on which .AUX file output goes to.
`\@input{foo}`	: If file foo exists, then `\input`'s it, otherwise types a warning message.
@filesw	: Switch – set false if no .AUX, .TOC, .IDX etc files are to be written
@partsw	: Set true by a `\includeonly` command.
`\@partlist`	: Set to the argument of the `\includeonly` command.
`\cp@FOO`	: The checkpoint for `\include`'d file FOO.TEX, written by `\@writeckpt` at the end of file FOO.AUX

`\includeonly{FILELIST}` ==
BEGIN

```
  \@partsw   := T
  \@partlist := FILELIST
 END

\include{FILE} ==
 BEGIN
  \clearpage
  if \@filesw = T
    then  \immediate\write\@mainaux{\string\@input{FILE.AUX}}
  fi
  if  \@partsw = T
    then \@tempswa := F
         \reserved@b == FILE
         for \reserved@a := \@partlist
             do if eval(\reserved@a) = eval(\reserved@b)
                  then \@tempswa := T           fi
             od
  fi

  if \@tempswa = T
     then \@auxout := \@partaux
          if \@filesw = T
            then  \immediate\openout\@partaux{FILE.AUX}
                  \immediate\write\@partaux{\relax}
          fi
          \@input{FILE.TEX}
          \clearpage
          \@writeckpt{FILE}
          if @filesw then \closeout \@partaux fi
          \@auxout := \@mainaux
     else \cp@FILE
  fi
 END

\@writeckpt{FILE} ==
 BEGIN
   if \@filesw = T
       \immediate\write on file \@partaux:
                \@setckpt{FILE}{                    %% }
       for \reserved@a := \cl@@ckpt
          do  \immediate\write on file \@partaux:
                  \global\string\setcounter
                      {eval(\reserved@a)}{eval(\c@eval(\reserved@a))}
          od                                        %% {
       \immediate\write on file \@partaux:  }
   fi
 END

\@setckpt{FILE}{LIST} ==
 BEGIN
```

```
  G \cp@FILE := LIST
END

INITIALIZATION
  \@tempswa := T
```

End of historical LaTeX 2.09 comments.

\@mainaux
\@partaux

```
\newwrite\@mainaux
\newwrite\@partaux
```

(*End of definition for* `\@mainaux` *and* `\@partaux`.)

\if@filesw
\if@partsw

```
\newif\if@filesw \@fileswtrue
\newif\if@partsw \@partswfalse
```

(*End of definition for* `\if@filesw` *and* `\if@partsw`.)

\@clubpenalty This stores the current normal (non-infinite) value of `\clubpenalty`; it should therefore be reset whenever the normal value is changed (as in the bibliography in the standard styles).

```
\newcount\@clubpenalty
\@clubpenalty \clubpenalty
```

(*End of definition for* `\@clubpenalty`.)

\document

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩   {\document}{Added hook to load l3backend code}%
⟨*2ekernel | latexrelease⟩
\def\document{%
```

We do cancel the grouping as part of the `\begin` handling (this is now done inside `\begin` instead) so that the `env/`⟨*env*⟩`/begin` hook is not hidden inside `\begingroup ... \endgroup`.

```
%  \endgroup
  \UseOneTimeHook{begindocument/before}%
  \@kernel@after@begindocument@before
```

Added hook to load l3backend code:

```
  \@expl@sys@load@backend@@
  \ifx\@unusedoptionlist\@empty\else
    \@latex@warning@no@line{Unused global option(s):^^J%
             \@spaces[\@unusedoptionlist]}%
  \fi
  \@colht\textheight
  \@colroom\textheight \vsize\textheight
  \columnwidth\textwidth
  \@clubpenalty\clubpenalty
  \if@twocolumn
    \advance\columnwidth -\columnsep
```

```
    \divide\columnwidth\tw@ \hsize\columnwidth \@firstcolumntrue
  \fi
  \hsize\columnwidth \linewidth\hsize
  \begingroup\@floatplacement\@dblfloatplacement
    \makeatletter\let\@writefile\@gobbletwo

    \global \let \@multiplelabels \relax
    \@input{\jobname.aux}%
  \endgroup
  \if@filesw
    \immediate\openout\@mainaux\jobname.aux
    \immediate\write\@mainaux{\relax}%
  \fi
```

Dateline 1991/03/26: FMi added `\process@table` to support NFSS; This will also work with old lfonts if no other style defines `\process@table`. The following line forces the initialization of the math fonts.

```
  \process@table
  \let\glb@currsize\@empty  % Force math initialization.

  \normalsize
  \everypar{}%
```

So that punctuation in headings is not disturbed by verbatim or other local changes to the space factor codes, save the document default here. This will be locally reset by the output routine. For special cases a class may want to define `\normalsfcodes` directly, in case that definition will be used. (This is an old bug, problem existed in LaTeX2.0x and plain TeX.)

```
  \ifx\normalsfcodes\@empty
    \ifnum\sfcode`\.=\@m
      \let\normalsfcodes\frenchspacing
    \else
      \let\normalsfcodes\nonfrenchspacing
    \fi
  \fi
```

For similar reasons also save the default language, this will be reset locally in the output routine. In particular it allows hyphenation in the page head even if the page break happens in verbatim. If this has already been set by a package, set to the value of `\language` at this point.

```
  \ifx\document@default@language\m@ne
    \chardef\document@default@language\language
  \fi
```

Way back in 1991 (08/26) FMi & RmS set the `@noskipsec` switch to true in the preamble and to false here. This was done to trap lists and related text in the preamble but it does not catch everything; hence Change 1.1g was introduced.

```
  \@noskipsecfalse

  \let \@refundefined \relax
```

Just before disabling the preamble commands we execute the begin document hook which contains any code contributed by `\AtBeginDocument`. Also disable the gathering of the file list, if no `\listfiles` has been issued. `\AtBeginDocument` is redefined at this point so that and such commands that get into the hook do not chase their tail...

```
\@kernel@before@begindocument
\UseOneTimeHook{begindocument}%
\@kernel@after@begindocument
```

Most of the following assignments will be done globally in case the user adds something like `\begin{multicols}` to the document hook, i.e. starts are group in `\begin{document}`.

Since a value of exactly 0pt for `\topskip` causes `\twocolumn[]` to misbehave, we add this check, hoping that it will not cause any problems elsewhere.

```
\ifdim\topskip<1sp\global\topskip 1sp\relax\fi
\global\@maxdepth\maxdepth
\global\let\@begindocumenthook\@undefined
\ifx\@listfiles\@undefined
  \global\let\@filelist\relax
  \global\let\@addtofilelist\@gobble
\fi
```

At the very end we disable all preamble commands. This has to happen after the begin document hooks was executed so that this hook can still use such commands.

```
\gdef\do##1{\global\let ##1\@notprerr}%
\@preamblecmds
```

The next line saves tokens and also allows `\@nodocument` to be used directly to trap preamble errors.

```
\global\let \@nodocument \relax
```

The next line is a pure safety measure in case a do list is ever expanded at the wrong place. In addition it will save a few tokens to get rid of the above definition.

```
\global\let\do\noexpand
\UseOneTimeHook{begindocument/end}%
```

Use of the hook might mean that we are already in horizontal mode, so ignore the space after `\begin{document}`.

```
\ignorespaces}
```

Provide a global definition for `\do` as well, so that it is already defined in the preamble and not late as `\begin{document}` overwriting some definition given by the unsuspecting user in the preamble.

```
\let\do\noexpand
```

The `begindocument` hook already existed in the kernel since 1994 under the name `\atbegindocumenthook` the additional ones are originally from the etoolbox package under the names `\@endpreamblehook` `\afterpreamble`.

```
\NewHook{begindocument}
\NewHook{begindocument/before}
\NewHook{begindocument/end}
```

Above we used two kernel only hooks to be run after the public `begindocument/before` and after `begindocument` hooks.

\@kernel@after@begindocument@before
\@kernel@before@begindocument
\@kernel@after@begindocument

In `\@kernel@after@begindocument@before` we already place one action: drop the fast execution code for the `env/document/begin` hook. That hook marks the end of the preamble and should therefore only be run once. In a normal document that is anyway the case (so the code would just sit there taking up space afterwards, which these days is rather harmless), however, in more complicated scenarios where several full documents are combined to a single document it might get applied several times with harmful effects.

We therefore explicitly drop it at this point. the coding is somewhat obscure due to the name of the macro which requires constructing.

```
\edef \@kernel@after@begindocument@before {%
  \let\expandafter\noexpand\csname
       __hook env/document/begin\endcsname
  \noexpand\@empty}
```

These internal hooks are already declared earlier (in `ltexpl`) so that other modules could write to them.

```
%\let \@kernel@before@begindocument \@empty
%\let \@kernel@after@begindocument  \@empty
```

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨latexrelease⟩\IncludeInRelease{2017/04/15}%
⟨latexrelease⟩  {\document}{Save language for hyphenation}%
⟨latexrelease⟩
⟨latexrelease⟩\def\document{\endgroup
⟨latexrelease⟩  \ifx\@unusedoptionlist\@empty\else
⟨latexrelease⟩    \@latex@warning@no@line{Unused global option(s):^^J%
⟨latexrelease⟩             \@spaces[\@unusedoptionlist]}%
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \@colht\textheight
⟨latexrelease⟩  \@colroom\textheight \vsize\textheight
⟨latexrelease⟩  \columnwidth\textwidth
⟨latexrelease⟩  \@clubpenalty\clubpenalty
⟨latexrelease⟩  \if@twocolumn
⟨latexrelease⟩    \advance\columnwidth -\columnsep
⟨latexrelease⟩    \divide\columnwidth\tw@ \hsize\columnwidth \@firstcolumntrue
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \hsize\columnwidth \linewidth\hsize
⟨latexrelease⟩  \begingroup\@floatplacement\@dblfloatplacement
⟨latexrelease⟩    \makeatletter\let\@writefile\@gobbletwo
⟨latexrelease⟩    \global \let \@multiplelabels \relax
⟨latexrelease⟩    \@input{\jobname.aux}%
⟨latexrelease⟩  \endgroup
⟨latexrelease⟩  \if@filesw
⟨latexrelease⟩    \immediate\openout\@mainaux\jobname.aux
⟨latexrelease⟩    \immediate\write\@mainaux{\relax}%
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \process@table
⟨latexrelease⟩  \let\glb@currsize\@empty  % Force math initialization.
⟨latexrelease⟩  \normalsize
⟨latexrelease⟩  \everypar{}%
⟨latexrelease⟩  \ifx\normalsfcodes\@empty
⟨latexrelease⟩    \ifnum\sfcode`\.=\@m
⟨latexrelease⟩      \let\normalsfcodes\frenchspacing
⟨latexrelease⟩    \else
⟨latexrelease⟩      \let\normalsfcodes\nonfrenchspacing
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \ifx\document@default@language\m@ne
⟨latexrelease⟩    \chardef\document@default@language\language
⟨latexrelease⟩  \fi
```

```
⟨latexrelease⟩  \@noskipsecfalse
⟨latexrelease⟩  \let \@refundefined \relax
⟨latexrelease⟩  \let\AtBeginDocument\@firstofone
⟨latexrelease⟩  \@begindocumenthook
⟨latexrelease⟩  \ifdim\topskip<1sp\global\topskip 1sp\relax\fi
⟨latexrelease⟩  \global\@maxdepth\maxdepth
⟨latexrelease⟩  \global\let\@begindocumenthook\@undefined
⟨latexrelease⟩  \ifx\@listfiles\@undefined
⟨latexrelease⟩    \global\let\@filelist\relax
⟨latexrelease⟩    \global\let\@addtofilelist\@gobble
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \gdef\do##1{\global\let ##1\@notprerr}%
⟨latexrelease⟩  \@preamblecmds
⟨latexrelease⟩  \global\let \@nodocument \relax
⟨latexrelease⟩  \global\let\do\noexpand
⟨latexrelease⟩  \ignorespaces}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩  {\document}{Save language for hyphenation}
⟨latexrelease⟩\def\document{\endgroup
⟨latexrelease⟩  \ifx\@unusedoptionlist\@empty\else
⟨latexrelease⟩    \@latex@warning@no@line{Unused global option(s):^^J%
⟨latexrelease⟩             \@spaces[\@unusedoptionlist]}%
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \@colht\textheight
⟨latexrelease⟩  \@colroom\textheight \vsize\textheight
⟨latexrelease⟩  \columnwidth\textwidth
⟨latexrelease⟩  \@clubpenalty\clubpenalty
⟨latexrelease⟩  \if@twocolumn
⟨latexrelease⟩    \advance\columnwidth -\columnsep
⟨latexrelease⟩    \divide\columnwidth\tw@ \hsize\columnwidth
⟨latexrelease⟩    \@firstcolumntrue
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \hsize\columnwidth \linewidth\hsize
⟨latexrelease⟩  \begingroup\@floatplacement\@dblfloatplacement
⟨latexrelease⟩    \makeatletter\let\@writefile\@gobbletwo
⟨latexrelease⟩    \global \let \@multiplelabels \relax
⟨latexrelease⟩    \@input{\jobname.aux}%
⟨latexrelease⟩  \endgroup
⟨latexrelease⟩  \if@filesw
⟨latexrelease⟩    \immediate\openout\@mainaux\jobname.aux
⟨latexrelease⟩    \immediate\write\@mainaux{\relax}%
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \process@table
⟨latexrelease⟩  \let\glb@currsize\@empty
⟨latexrelease⟩  \normalsize
⟨latexrelease⟩  \everypar{}%
⟨latexrelease⟩  \ifx\normalsfcodes\@empty
⟨latexrelease⟩    \ifnum\sfcode‘\.=\@m
⟨latexrelease⟩      \let\normalsfcodes\frenchspacing
⟨latexrelease⟩    \else
⟨latexrelease⟩      \let\normalsfcodes\nonfrenchspacing
⟨latexrelease⟩    \fi
```

```
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \@noskipsecfalse
⟨latexrelease⟩  \let \@refundefined \relax
⟨latexrelease⟩  \let\AtBeginDocument\@firstofone
⟨latexrelease⟩  \@begindocumenthook
⟨latexrelease⟩  \ifdim\topskip<1sp\global\topskip 1sp\relax\fi
⟨latexrelease⟩  \global\@maxdepth\maxdepth
⟨latexrelease⟩  \global\let\@begindocumenthook\@undefined
⟨latexrelease⟩  \ifx\@listfiles\@undefined
⟨latexrelease⟩    \global\let\@filelist\relax
⟨latexrelease⟩    \global\let\@addtofilelist\@gobble
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \gdef\do##1{\global\let ##1\@notprerr}%
⟨latexrelease⟩  \@preamblecmds
⟨latexrelease⟩  \global\let \@nodocument \relax
⟨latexrelease⟩  \global\let\do\noexpand
⟨latexrelease⟩  \ignorespaces}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
\@onlypreamble\document
```

(*End of definition for* `\document` *and others.*)

`\normalsfcodes` The setting of `\@empty` is just a flag. This command may be defined in a class or package file. If it is still `\@empty` at `\begin{document}` it will be defined to be `\frenchspacing` or `\nonfrenchspacing`, depending on which of those appears to be in effect at that point.

```
\let\normalsfcodes\@empty
```

(*End of definition for* `\normalsfcodes`.)

`\nofiles` Set `\@fileswfalse` which suppresses the places where LaTeX makes `\immediate` writes. The `\makeindex` and `\makeglossary` are disabled. `\protected@write` is redefined not to write to the file specified, but rather to write a blank line to the log file. This ensures that a ⟨*whatsit*⟩ node is still created, and so spacing is not affected by the `\nofiles` command; to ensure this more generally, the `\if@nobreak` test is needed.

```
\def\nofiles{%
  \@fileswfalse
  \typeout{No auxiliary output files.^^J}%
  \long\def\protected@write##1##2##3%
    {\write\m@ne{}\if@nobreak\ifvmode\nobreak\fi\fi}%
  \let\makeindex\relax
  \let\makeglossary\relax}
\@onlypreamble\nofiles
```

(*End of definition for* `\nofiles`.)

`\protected@write` This takes three arguments: an output stream, some initialization code, and some text to write. It then writes this, with appropriate handling of `\protect` and `\thepage`.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease⟩                  {\protected@write}{Allow for immediate write}%
\long\def \protected@write#1#2#3{%
```

```
      \begingroup
       \let\thepage\relax
       #2%
       \let\protect\@unexpandable@protect
       \edef\reserved@a{\endgroup\write#1{#3}}%
       \reserved@a
      \if@nobreak\ifvmode\nobreak\fi\fi
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\protected@write}{Allow for immediate write}%
⟨latexrelease⟩\long\def \protected@write#1#2#3{%
⟨latexrelease⟩      \begingroup
⟨latexrelease⟩       \let\thepage\relax
⟨latexrelease⟩       #2%
⟨latexrelease⟩       \let\protect\@unexpandable@protect
⟨latexrelease⟩       \edef\reserved@a{\write#1{#3}}%
⟨latexrelease⟩       \reserved@a
⟨latexrelease⟩      \endgroup
⟨latexrelease⟩      \if@nobreak\ifvmode\nobreak\fi\fi
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\protected@write`.)

```
\let\@auxout=\@mainaux
```

`\include` `\includeonly`

In the definition of `\include`, `\def\reserved@b` changed to `\edef\reserved@b` to be consistent with the `\edef` in `\includeonly`. (Suggested by Rainer Schöpf & Frank Mittelbach. Change made 20 Jul 88.)

Changed definition of `\include` to allow space at end of file name — otherwise, typing `\include{foo }` would cause LaTeX to overwrite `foo.tex`. Change made 24 May 89, suggested by Rainer Schöpf and Frank Mittelbach

Made `\include` check for being used inside an `\include`'d file, as this will not work and cause surprising results.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\includeonly}{Spaces in file names}%
\def\include#1{\relax
  \ifnum\@auxout=\@partaux
    \@latex@error{\string\include\space cannot be nested}\@eha
  \else
```

Here the normalization will add `.tex` for all files, (it uses the same normalization as the hooks), so we need to remove that manually. `\@strip@tex@ext` does that.

```
    \set@curr@file{#1}%
    \edef\@curr@file{\@strip@tex@ext\@curr@file}%
```

For historical reasons `\@include` expects an argument delimited by a space. This is kept (though unnecessary now) to avoid errors in other packages that use `\@include` directly.

```
    \expandafter\@include\expandafter{\@curr@file} % deliberate space
  \fi}
```

Here in `\includeonly` we also need to strip `.tex` after normalization:

```
\def\includeonly#1{%
  \@partswtrue
```

Because the argument to `\includeonly` is a comma-separated list of filenames where there may be comma's preceding some of the filenames or trailing them. Therefore we need to take the list apart, remove the unwanted spaces while leaving the spaces *in* the filenames intact.

```
  \let\@partlist\@empty
  \@for\reserved@a:=#1 \do
    {%
      \expandafter\set@curr@file\expandafter{\reserved@a}%
      \ifx\@partlist\@empty
        \edef\@partlist{\@strip@tex@ext\@curr@file}%
      \else
        \edef\@partlist{\@partlist,\@strip@tex@ext\@curr@file}%
      \fi
    }%
  }
\@onlypreamble\includeonly
```

(*End of definition for* `\include` *and* `\includeonly`.)

`\@strip@tex@ext`
`\@strip@tex@ext@aux`

These macros take a (`\detokenize`d file name and remove any `.tex` extension). Extra care is taken to not remove the string `.tex` from the middle of a file name: it is only removed if it's the very last thing in the file name.

```
\def\reserved@a#1{%
\def\@strip@tex@ext##1{%
  \expandafter\@strip@tex@ext@aux
    ##1\@nil\@nil
     #1\@nil\relax\@nnil}
\def\@strip@tex@ext@aux##1#1\@nil##2\@nnil{%
  \ifx\relax##2\@empty
    \expandafter\@cdr\expandafter\@empty\@cdr{}##1%
  \else##1\fi}}%
\expandafter\reserved@a
\expandafter{\detokenize{.tex}}
⟨/2ekernel | latexrelease⟩
```

(*End of definition for* `\@strip@tex@ext` *and* `\@strip@tex@ext@aux`.)

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                  {\includeonly}{Spaces in file names}%
⟨latexrelease⟩
⟨latexrelease⟩\def\includeonly#1{%
⟨latexrelease⟩   \@partswtrue
⟨latexrelease⟩   \set@curr@file{\zap@space#1 \@empty}%
⟨latexrelease⟩   \let\@partlist\@curr@file
⟨latexrelease⟩   }
⟨latexrelease⟩
⟨latexrelease⟩\def\include#1{\relax
⟨latexrelease⟩   \ifnum\@auxout=\@partaux
⟨latexrelease⟩     \@latex@error{\string\include\space cannot be nested}\@eha
⟨latexrelease⟩   \else
```

```
⟨latexrelease⟩     \set@curr@file{#1 }%
⟨latexrelease⟩     \expandafter\@include\@curr@file
⟨latexrelease⟩   \fi}
⟨latexrelease⟩
⟨latexrelease⟩\let\@strip@tex@ext\@undefined
⟨latexrelease⟩\let\@strip@tex@ext@aux\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\includeonly}{Spaces in file names}%
⟨latexrelease⟩\def\includeonly#1{%
⟨latexrelease⟩   \@partswtrue
⟨latexrelease⟩   \edef\@partlist{\zap@space#1 \@empty}}
⟨latexrelease⟩
⟨latexrelease⟩\def\include#1{\relax
⟨latexrelease⟩   \ifnum\@auxout=\@partaux
⟨latexrelease⟩     \@latex@error{\string\include\space cannot be nested}\@eha
⟨latexrelease⟩   \else \@include#1 \fi}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

`\@include`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2022/06/01}%
⟨latexrelease⟩                  {\@include}{Spaces in file names and hooks}%
\def\@include#1 {%
\ifx\@nodocument\relax
  \clearpage
  \if@filesw
    \immediate\write\@mainaux{\string\@input{#1.aux}}%
  \fi
  \@tempswatrue
  \if@partsw
    \@tempswafalse
    \edef\reserved@b{#1}%
    \@for\reserved@a:=\@partlist\do
      {\ifx\reserved@a\reserved@b\@tempswatrue\fi}%
  \fi
  \if@tempswa
    \let\@auxout\@partaux
    \if@filesw
      \immediate\openout\@partaux "#1.aux"
      \immediate\write\@partaux{\relax}%
    \fi
```

Now before going to the hooks we need to set `\CurrentFile`:

```
%----------------------------------------
    \@filehook@set@CurrentFile
```

Execute the `before` hooks just after we switched the `.aux` file ...

```
    \UseHook{include/before}%
    \UseOneTimeHook{include/#1/before}%
%----------------------------------------
    \@input@{#1.tex}%
%----------------------------------------
```

... then `end` hooks ...

```
    \UseOneTimeHook{include/#1/end}%
    \UseHook{include/end}%
%----------------------------------------
    \clearpage
%----------------------------------------
```

... and after the `\clearpage` the `after` hooks followed by another `\clearpage` just in case new material got added (after all we need to be in well defined state after the `\include`).

```
    \UseOneTimeHook{include/#1/after}%
    \UseHook{include/after}%
    \clearpage
%----------------------------------------
    \@writeckpt{#1}%
    \if@filesw
      \immediate\closeout\@partaux
    \fi
  \else
```

If the file is not included, reset `\deadcycles`, so that a long list of non-included files does not generate an 'Output loop' error.

```
    \deadcycles\z@
    \@nameuse{cp@#1}%
```

We also execute a hook in this case, first a general used for every include file that is exclude and then a specific one that contains the name of the include file.

```
%----------------------------------------
    \UseHook{include/excluded}%
    \UseOneTimeHook{include/#1/excluded}%
%----------------------------------------
  \fi
  \let\@auxout\@mainaux
\else
\@latex@warning{%
  \noexpand\include should only be used after \string\begin{document}}%
\@input@{#1}%
\fi}
```

Now declare the non-generic `include` hooks used above:

```
\NewHook{include/before}
\NewReversedHook{include/end}
\NewReversedHook{include/after}
\NewHook{include/excluded}
⟨latexrelease⟩ \EndIncludeInRelease
⟨/2ekernel | latexrelease⟩
```

```
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\@include}{Spaces in file names and hooks}%
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\def\@include#1 {%
⟨latexrelease⟩\ifx\@nodocument\relax
⟨latexrelease⟩  \clearpage
⟨latexrelease⟩  \if@filesw
⟨latexrelease⟩    \immediate\write\@mainaux{\string\@input{#1.aux}}%
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \@tempswatrue
⟨latexrelease⟩  \if@partsw
⟨latexrelease⟩    \@tempswafalse
⟨latexrelease⟩    \edef\reserved@b{#1}%
⟨latexrelease⟩    \@for\reserved@a:=\@partlist\do
⟨latexrelease⟩      {\ifx\reserved@a\reserved@b\@tempswatrue\fi}%
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \if@tempswa
⟨latexrelease⟩    \let\@auxout\@partaux
⟨latexrelease⟩    \if@filesw
⟨latexrelease⟩      \immediate\openout\@partaux "#1.aux"
⟨latexrelease⟩      \immediate\write\@partaux{\relax}%
⟨latexrelease⟩    \fi
⟨latexrelease⟩    \@filehook@set@CurrentFile
⟨latexrelease⟩    \UseHook{include/before}%
⟨latexrelease⟩    \UseOneTimeHook{include/#1/before}%
⟨latexrelease⟩    \@input@{#1.tex}%
⟨latexrelease⟩    \UseOneTimeHook{include/#1/end}%
⟨latexrelease⟩    \UseHook{include/end}%
⟨latexrelease⟩    \clearpage
⟨latexrelease⟩    \UseOneTimeHook{include/#1/after}%
⟨latexrelease⟩    \UseHook{include/after}%
⟨latexrelease⟩    \clearpage
⟨latexrelease⟩    \@writeckpt{#1}%
⟨latexrelease⟩    \if@filesw
⟨latexrelease⟩      \immediate\closeout\@partaux
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \else
⟨latexrelease⟩    \deadcycles\z@
⟨latexrelease⟩    \@nameuse{cp@#1}%
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \let\@auxout\@mainaux
⟨latexrelease⟩\else
⟨latexrelease⟩\@latex@warning{%
⟨latexrelease⟩  \noexpand\include should only be used after \string\begin{document}}%
⟨latexrelease⟩\@input@{#1}%
⟨latexrelease⟩\fi}
⟨latexrelease⟩\NewHook{include/before}
⟨latexrelease⟩\NewReversedHook{include/end}
⟨latexrelease⟩\NewReversedHook{include/after}

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@include}{Spaces in file names and hooks}%
⟨latexrelease⟩\def\@include#1 {%
⟨latexrelease⟩  \clearpage
⟨latexrelease⟩  \if@filesw
```

```
⟨latexrelease⟩    \immediate\write\@mainaux{\string\@input{#1.aux}}%
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \@tempswatrue
⟨latexrelease⟩  \if@partsw
⟨latexrelease⟩    \@tempswafalse
⟨latexrelease⟩    \edef\reserved@b{#1}%
⟨latexrelease⟩    \@for\reserved@a:=\@partlist\do
⟨latexrelease⟩      {\ifx\reserved@a\reserved@b\@tempswatrue\fi}%
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \if@tempswa
⟨latexrelease⟩    \let\@auxout\@partaux
⟨latexrelease⟩    \if@filesw
⟨latexrelease⟩      \immediate\openout\@partaux #1.aux
⟨latexrelease⟩      \immediate\write\@partaux{\relax}%
⟨latexrelease⟩    \fi
⟨latexrelease⟩    \@input@{#1.tex}%
⟨latexrelease⟩    \clearpage
⟨latexrelease⟩    \@writeckpt{#1}%
⟨latexrelease⟩    \if@filesw
⟨latexrelease⟩      \immediate\closeout\@partaux
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \else
⟨latexrelease⟩    \deadcycles\z@
⟨latexrelease⟩    \@nameuse{cp@#1}%
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \let\@auxout\@mainaux}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\@include`.)

`\@writeckpt`

```
\def\@writeckpt#1{%
  \if@filesw
    \immediate\write\@partaux{\string\@setckpt{#1}\@charlb}%
    {\let\@elt\@wckptelt \cl@@ckpt}%
    \immediate\write\@partaux{\@charrb}%
  \fi}
```

(*End of definition for* `\@writeckpt`.)

`\@wckptelt`

```
\def\@wckptelt#1{%
  \immediate\write\@partaux{%
    \string\setcounter{#1}{\the\@nameuse{c@#1}}}}
```

(*End of definition for* `\@wckptelt`.)

`\@setckpt` RmS 93/08/31: introduced `\@setckpt`

```
\def\@setckpt#1{\global\@namedef{cp@#1}}
```

(*End of definition for* `\@setckpt`.)

\@charlb
\@charrb

The following defines \@charlb and \@charrb to be { and }, respectively with \catcode 11.

```
{\catcode`[=1 \catcode`]=2
\catcode`{=11 \catcode`}=11
\gdef\@charlb[{]
\gdef\@charrb[}]
]% }brace matching
```

(*End of definition for* \@charlb *and* \@charrb.)

1.1 Safe Input Macros

\@curr@file
\set@curr@file

File name handling is done by generating a csname from the provided file name (which means that UTF-8 octets gets turned into strings as this is what happens if they appear in a csname due to the code in utf8.def). By setting \escapchar to -1 we ensure that we don't get a backslash in front. As a result we end up with all characters as catcode 12 (plus spaces). We then sometimes add quotes around the construct (removing any existing inner quotes). Sometimes we only remove the quotes if they have been supplied by the user. There is clearly some room for improvement.

A side effect of the new code is that we will see quotes around file name displays where there haven't been any before.

For compatibility with existing code using {abc}.tex or {one.two}.png, an initial brace group is discarded before expansion and \string is applied. The content of the brace group is discarded. This means that a leading space will be lost unless protected (by { } or " " or \space) but filenames with a space are hopefully rare.

The definition below is from 2019 and only used during kernel bootstrapping, later on in ltfilehook.dtx it will get overwritten.

```
\def\set@curr@file#1{%
  \begingroup
    \escapechar\m@ne
    \xdef\@curr@file{%
      \expandafter\expandafter\expandafter\unquote@name
      \expandafter\expandafter\expandafter{%
      \expandafter\string
        \csname\@firstofone#1\@empty\endcsname}}%
  \endgroup
}
```

(*End of definition for* \@curr@file *and* \set@curr@file.)

\quote@name
\quote@@name
\unquote@name

Quoting spaces

```
a b c     -> "a b c"
"a b c"   -> "a b c"
a" "b" "c -> "a b c"
          -> ""
```

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                    {\quote@name}{Quote file names}%
\def\quote@name#1{"\quote@@name#1\@gobble""}
\def\quote@@name#1"{#1\quote@@name}
```

and removing quotes ...

```
\def\unquote@name#1{\quote@@name#1\@gobble"}
```

(*End of definition for* `\quote@name`, `\quote@@name`, *and* `\unquote@name`.)

`\IfFileExists`

```
\DeclareRobustCommand\IfFileExists[1]{%
  \set@curr@file{#1}%
  \expandafter\IfFileExists@\expandafter{\@curr@file}}
```

(*End of definition for* `\IfFileExists`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\quote@name}{Quote file names}%
⟨latexrelease⟩
⟨latexrelease⟩\let\quote@name\@undefined
⟨latexrelease⟩\let\quote@@name\@undefined
⟨latexrelease⟩\let\unquote@name\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\long\def \IfFileExists#1#2#3{%
⟨latexrelease⟩  \openin\@inputcheck#1 %
⟨latexrelease⟩  \ifeof\@inputcheck
⟨latexrelease⟩    \ifx\input@path\@undefined
⟨latexrelease⟩      \def\reserved@a{#3}%
⟨latexrelease⟩    \else
⟨latexrelease⟩      \def\reserved@a{\@iffileonpath{#1}{#2}{#3}}%
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \else
⟨latexrelease⟩    \closein\@inputcheck
⟨latexrelease⟩    \edef\@filef@und{#1 }%
⟨latexrelease⟩    \def\reserved@a{#2}%
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \reserved@a}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

`\IfFileExists@` `\IfFileExists@@`

Argument `#1` is `\@curr@file` so catcode 12 string with no quotes.

The original definition picked up arguments `#2` and `#3` in a way that they couldn't contain unbalanced conditionals. A better implementation would have been not to pick up the arguments at all but instead use the usual `\@firstoftwo` and `\secondoftwo`. However, that changes how `#` is interpreted and so we can't do that nowadays without invalidating a lot of code. Therefore the somewhat curious construction near the end.

Earlier versions used `\openin` here, but this led to two code paths, one in expl3 and one here. To avoid that, and as the expl3 approach works by expansion, we use that here. As we need the file name to include the path, the actual expl3 function used is not the file existence test!

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2023/06/01}%
⟨latexrelease⟩                  {\IfFileExists@}{IfFileExists}
\long\def \IfFileExists@#1#2#3{%
  \edef\@filef@und{\IfFileExists@@{#1}}%
```

The expl3 function regards an empty argument as nothing at all, but the LaTeX 2_ε convention is that this is equal to the special `.tex` file.

```
  \ifx\@filef@und\@empty
    \if\relax\detokenize{#1}\relax
      \let\reserved@a\@firstoftwo
      \def\@filef@und{".tex" }%
    \else
      \let\reserved@a\@secondoftwo
    \fi
  \else
    \let\reserved@a\@firstoftwo
    \edef\@filef@und{"\@filef@und" }%
  \fi
```

This is just there so that any `#` inside `#2` or `#3` needs doubling (as that was the case in the past).

```
  \expandafter\def\expandafter\reserved@a
    \expandafter{\reserved@a{#2}{#3}}%
\reserved@a}
```

Pipes are not really files, but at the document level they are supported. To quickly trim of any leading spaces, we use a blank test and `\use:n` rather than `\tl_trim_spaces:n` for speed as we don't care about the end of the input.

```
\ExplSyntaxOn
\cs_new:Npn \IfFileExists@@ #1
  {
    \tl_if_blank:nF {#1}
      {
        \tl_if_head_eq_charcode:oNTF { \use:n #1 } |
          {#1}
          { \file_full_name:n {#1} }
      }
  }
\cs_generate_variant:Nn \tl_if_head_eq_charcode:nNTF { o }
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩                  {\IfFileExists@}{IfFileExists}
⟨latexrelease⟩
⟨latexrelease⟩\long\def \IfFileExists@#1#2#3{%
⟨latexrelease⟩  \openin\@inputcheck"#1" %
⟨latexrelease⟩  \ifeof\@inputcheck
⟨latexrelease⟩    \ifx\input@path\@undefined
⟨latexrelease⟩      \let\reserved@a\@secondoftwo
⟨latexrelease⟩    \else
⟨latexrelease⟩      \def\reserved@a{\@iffileonpath{#1}}%
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \else
⟨latexrelease⟩    \closein\@inputcheck
⟨latexrelease⟩    \edef\@filef@und{"#1" }%
⟨latexrelease⟩    \let\reserved@a\@firstoftwo
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \expandafter\def\expandafter\reserved@a
```

```
⟨latexrelease⟩     \expandafter{\reserved@a{#2}{#3}}%
⟨latexrelease⟩\reserved@a}
⟨latexrelease⟩\let\IfFileExists@@\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                  {\IfFileExists@}{IfFileExists}
⟨latexrelease⟩
⟨latexrelease⟩\long\def \IfFileExists@#1#2#3{%
⟨latexrelease⟩   \openin\@inputcheck"#1" %
⟨latexrelease⟩   \ifeof\@inputcheck
⟨latexrelease⟩     \ifx\input@path\@undefined
⟨latexrelease⟩       \def\reserved@a{#3}%
⟨latexrelease⟩     \else
⟨latexrelease⟩       \def\reserved@a{\@iffileonpath{#1}{#2}{#3}}%
⟨latexrelease⟩     \fi
⟨latexrelease⟩   \else
⟨latexrelease⟩     \closein\@inputcheck
⟨latexrelease⟩     \edef\@filef@und{"#1" }%
⟨latexrelease⟩     \def\reserved@a{#2}%
⟨latexrelease⟩   \fi
⟨latexrelease⟩   \reserved@a}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\IfFileExists@}{IfFileExists}
⟨latexrelease⟩
⟨latexrelease⟩\let\IfFileExists@\@undefined
⟨latexrelease⟩
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\IfFileExists@` *and* `\IfFileExists@@`.)

`\@iffileonpath` If the file is not found by `\openin`, and `\input@path` is defined, look in all the directories specified in `\input@path`.

```
⟨/2ekernel⟩
⟨∗2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                  {\@iffileonpath}{Quote file names}
\long\def\@iffileonpath#1{%
  \let\reserved@a\@secondoftwo
  \expandafter\@tfor\expandafter\reserved@b\expandafter
              :\expandafter=\input@path\do{%
    \openin\@inputcheck\expandafter\quote@name\expandafter{\reserved@b#1} %
    \ifeof\@inputcheck\else
      \edef\@filef@und{\expandafter\quote@name\expandafter{\reserved@b#1} }%
      \let\reserved@a\@firstoftwo%
      \closein\@inputcheck
      \@break@tfor
    \fi}%
  \reserved@a}
```

(*End of definition for* `\@iffileonpath`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\quote@name}{Quote file names}
⟨latexrelease⟩
⟨latexrelease⟩\long\def\@iffileonpath#1{%
⟨latexrelease⟩  \let\reserved@a\@secondoftwo
⟨latexrelease⟩  \expandafter\@tfor\expandafter\reserved@b\expandafter
⟨latexrelease⟩              :\expandafter=\input@path\do{%
⟨latexrelease⟩    \openin\@inputcheck\reserved@b#1 %
⟨latexrelease⟩    \ifeof\@inputcheck\else
⟨latexrelease⟩      \edef\@filef@und{\reserved@b#1 }%
⟨latexrelease⟩      \let\reserved@a\@firstoftwo%
⟨latexrelease⟩      \closein\@inputcheck
⟨latexrelease⟩      \@break@tfor
⟨latexrelease⟩    \fi}%
⟨latexrelease⟩  \reserved@a}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

`\InputIfFileExists` Now define `\InputIfFileExists` to input `#1` if it seems to exist. Immediately prior to the input, `#2` is executed. If the file `#1` does not exist, execute '`#3`'.

This here is a temporary definition for the kernel. The real one comes somewhat later in the file `ltfilehook.dtx`.

```
\DeclareRobustCommand \InputIfFileExists[2]{%
  \IfFileExists{#1}%
    {%
  \expandafter\@swaptwoargs\expandafter
      {\@filef@und}{#2\@addtofilelist{#1}\@@input}}}
```

(*End of definition for* `\InputIfFileExists`.)

`\@swaptwoargs` Swap two arguments and return them unbraced (like `\@firstoftwo` etc).

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩  {\@swaptwoargs}{Don't lose the file name}%
\long\def\@swaptwoargs#1#2{#2#1}

⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩  {\@swaptwoargs}{Don't lose the file name}%
⟨latexrelease⟩\let\@swaptwoargs\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@swaptwoargs`.)

`\input` Input a file: if the argument is given in braces use safe input macros, otherwise use TEX's primitive `\input` command (which is called `\@@input` in LATEX).

```
\def\input{\@ifnextchar\bgroup\@iinput\@@input}
```

(*End of definition for* `\input`.)

`\@iinput` Define `\@iinput` (i.e., `\input`) in terms of `\InputIfIfileExists`.

Changes to `\@iinput`: adapt to the changes to `\@missingfileerror`.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\@iinput}{Change in file error handling}%
\def\@iinput#1{%
  \InputIfFileExists{#1}{}%
  {\filename@parse\@curr@file
   \edef\reserved@a{\noexpand\@missingfileerror
     {\filename@area\filename@base}%
     {\ifx\filename@ext\relax tex\else\filename@ext\fi}}%
```

This line now just sets `\@missingfile@`⟨*part*⟩:

```
   \reserved@a
```

Now here we have to use it. The file here is guaranteed to exist, because `\@missingfileerror` ensures so, but we have to use `\InputIfFileExists` because it executes the file hooks.

```
   \edef\reserved@a{\noexpand\@iinput{%
     \@missingfile@area\@missingfile@base.\@missingfile@ext}}%
   \reserved@a}}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                  {\@iinput}{Quote file names}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@iinput#1{%
⟨latexrelease⟩  \InputIfFileExists{#1}{}%
⟨latexrelease⟩  {\filename@parse\@curr@file
⟨latexrelease⟩   \edef\reserved@a{\noexpand\@missingfileerror
⟨latexrelease⟩     {\filename@area\filename@base}%
⟨latexrelease⟩     {\ifx\filename@ext\relax tex\else\filename@ext\fi}}%
⟨latexrelease⟩   \reserved@a}}
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@iinput}{Quote file names}%
⟨latexrelease⟩\def\@iinput#1{%
⟨latexrelease⟩  \InputIfFileExists{#1}{}%
⟨latexrelease⟩  {\filename@parse{#1}%
⟨latexrelease⟩   \edef\reserved@a{\noexpand\@missingfileerror
⟨latexrelease⟩     {\filename@area\filename@base}%
⟨latexrelease⟩     {\ifx\filename@ext\relax tex\else\filename@ext\fi}}%
⟨latexrelease⟩   \reserved@a}}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@iinput`.)

`\@input` Define `\@input` in terms of `\IfIfileExists`. So this is a 'safe input' command, but the files input are not listed by `\listfiles`.

We don't want `.aux`, `.toc` files etc be listed by `\listfiles`. However, something like `.bbl` probably should be listed and thus should be implemented not by `\@input`.

```
\def\@input#1{%
  \IfFileExists{#1}{\@@input\@filef@und}{\typeout{No file #1.}}}
```

(*End of definition for* `\@input`.)

`\@input@` Version of `\@input` that does add the file to `\@filelist`.

```
\def\@input@#1{\InputIfFileExists{#1}{}{\typeout{No file #1.}}}
```

(*End of definition for* `\@input@`.)

`\@missingfileerror` This 'error' command avoids TEX's primitive missing file loop.

Missing file error. Prompt for a new filename, offering a default extension.

Changes to `\@missingfileerror`: rather than trying to input the file by force, now `\@missingfileerror` just returns three `\@missingfile@`⟨*part*⟩ and the caller macro is responsible for doing the right thing with it.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩         {\@missingfileerror}{Do not load missing file immediately}%
\gdef\@missingfileerror#1#2{%
     \typeout{^^J! LaTeX Error: File `#1.#2' not found.^^J^^J%
      Type X to quit or <RETURN> to proceed,^^J%
      or enter new name. (Default extension: #2)^^J}%
     \message{Enter file name: }%
      {\endlinechar\m@ne
       \global\read\m@ne to\@gtempa}%
    \ifx\@gtempa\@empty
```

If the user answers with ⟨*return*⟩, fallback to the `.tex` file (previously it did nothing).

```
      \let\@missingfile@area\@empty
      \let\@missingfile@base\@empty
      \def\@missingfile@ext{tex}%
    \else
```

Use `\batchmode\read-1 to` ⟨*tl*⟩ to end the TEX run, same as expl3 does (it was `\batchmode\@@end` before).

```
      \def\reserved@b{\batchmode\read-1 to \reserved@a}%
      \def\reserved@a{x}\ifx\reserved@a\@gtempa\reserved@b\fi
      \def\reserved@a{X}\ifx\reserved@a\@gtempa\reserved@b\fi
      \filename@parse\@gtempa
      \edef\filename@ext{%
        \ifx\filename@ext\relax#2\else\filename@ext\fi}%
     \edef\reserved@a{%
```

Only check `\IfFileExists` (it was `\InputIfFileExists`).

```
       \noexpand\IfFileExists
         {\filename@area\filename@base.\filename@ext}%
```

If the file exists, define `\@missingfile@`⟨*part*⟩.

```
         {\def\noexpand\@missingfile@area{\filename@area}%
          \def\noexpand\@missingfile@base{\filename@base}%
          \def\noexpand\@missingfile@ext {\filename@ext}}%
         {\noexpand\@missingfileerror
            {\filename@area\filename@base}{\filename@ext}}}%
      \reserved@a
    \fi
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩        {\@missingfileerror}{Do not load missing file immediately}%
⟨latexrelease⟩
⟨latexrelease⟩\gdef\@missingfileerror#1#2{%
⟨latexrelease⟩     \typeout{^^J! LaTeX Error: File ‘#1.#2’ not found.^^J^^J%
⟨latexrelease⟩      Type X to quit or <RETURN> to proceed,^^J%
⟨latexrelease⟩      or enter new name. (Default extension: #2)^^J}%
⟨latexrelease⟩     \message{Enter file name: }%
⟨latexrelease⟩      {\endlinechar\m@ne
⟨latexrelease⟩       \global\read\m@ne to\@gtempa}%
⟨latexrelease⟩    \ifx\@gtempa\@empty
⟨latexrelease⟩    \else
⟨latexrelease⟩      \def\reserved@a{x}\ifx\reserved@a\@gtempa\batchmode\@@end\fi
⟨latexrelease⟩      \def\reserved@a{X}\ifx\reserved@a\@gtempa\batchmode\@@end\fi
⟨latexrelease⟩      \filename@parse\@gtempa
⟨latexrelease⟩      \edef\filename@ext{%
⟨latexrelease⟩        \ifx\filename@ext\relax#2\else\filename@ext\fi}%
⟨latexrelease⟩     \edef\reserved@a{%
⟨latexrelease⟩       \noexpand\InputIfFileExists
⟨latexrelease⟩         {\filename@area\filename@base.\filename@ext}%
⟨latexrelease⟩         {}%
⟨latexrelease⟩         {\noexpand\@missingfileerror
⟨latexrelease⟩            {\filename@area\filename@base}{\filename@ext}}}%
⟨latexrelease⟩      \reserved@a
⟨latexrelease⟩    \fi}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\@missingfileerror`.)

`\@obsoletefile` For compatibility with LaTeX 2.09 document styles, we distribute files called `article.sty`, `book.sty`, `report.sty`, `slides.sty` and `letter.sty`. These use the command `\@obsoletefile`, which produces a warning message.

```
\def\@obsoletefile#1#2{%
   \@latex@warning@no@line{inputting ‘#1’ instead of obsolete ‘#2’}}
\@onlypreamble\@obsoletefile
```

1.2 Listing files

`\@filelist` A list of files input so far. The initial value of `\@gobble` eats the comma before the first file name.

```
\let\@filelist\@gobble
```

`\@addtofilelist` Add to the list of files input so far. This ‘real’ definition is only used for ‘cfg’ files during initex. An initial definition of `\@gobble` has already been set.

```
%\def\@addtofilelist#1{\xdef\@filelist{\@filelist,#1}}
```

`\if@listfiles@hashes` `\if@listfiles@sizes`

```
\ExplSyntaxOn
\keys_define:nn { __kernel / listfiles }
  {
    hashes .legacy_if_set:n = @listfiles@hashes ,
```

```
    sizes .legacy_if_set:n  = @listfiles@sizes
  }
\ExplSyntaxOff
```

A preamble command to cause `\end{document}` to list files input from the main file.

`\listfiles`

```
\NewDocumentCommand\listfiles{O{}}{%
  \SetKeys[__kernel/listfiles]{#1}%
  \let\listfiles\relax
  \def\@listfiles##1##2##3##4##5##6##7##8##9\@@{%
     \def\reserved@d{\\}%
     \@tfor\reserved@c:=##1##2##3##4##5##6##7##8\do{%
       \ifx\reserved@c\reserved@d
         \edef\filename@area{ \filename@area}%
       \fi}}%

  \def\@dofilelist{%
     \typeout{^^J *File List*}%
     \@for\@currname:=\@filelist\do{%
       \filename@parse\@currname
       \edef\reserved@a{%
          \filename@base.%
          \ifx\filename@ext\relax tex\else\filename@ext\fi}%
       \expandafter\let\expandafter\reserved@b
                               \csname ver@\reserved@a\endcsname
```

Packages that `\relax` their `\ver@...` string to allow for multiple loading (e.g., fontenc) can use `\ver@@...` to store the version information instead.

```
       \ifx\reserved@b\relax
         \expandafter\let\expandafter\reserved@b
                         \csname ver@@\reserved@a\endcsname
       \fi
       \expandafter\expandafter\expandafter\@listfiles\expandafter
            \filename@area\filename@base\\\\\\\\\\\\\\\\\\\@@
       \typeout{%
         \filename@area\reserved@a
         \ifx\reserved@b\relax\else\@spaces\reserved@b\fi
```

Now we add the additional information if requested.

```
         \ifnum0%
           \if@listfiles@hashes1\fi
           \if@listfiles@sizes1\fi
             >0 %
             ^^J\@spaces
             (%
               \if@listfiles@sizes
                 size \@dofilelist@size\@currname
                 \if@listfiles@hashes
                   , %
                 \fi
               \fi
               \if@listfiles@hashes
                 hash \@dofilelist@hash\@currname
               \fi
             )%
         \fi
```

```
      }}%
     \typeout{ ***********^^J}}}
```

```
\ExplSyntaxOn
\@dofilelist@hash \cs_new_eq:NN \@dofilelist@hash \file_mdfive_hash:n
\@dofilelist@size \cs_new_eq:NN \@dofilelist@size \file_size:n
\ExplSyntaxOff
```

The `\@filelist` will be de-activated if `\listfiles` does not appear in the preamble. `\begin{document}` contains code equivalent to the following:

```
\AtBeginDocument{%
  \ifx\@listfiles\@undefined
    \let\@filelist\relax
    \let\@addtofilelist\@gobble
  \fi}
```

```
\@onlypreamble\listfiles
```

`\@dofilelist`

```
\let\@dofilelist\relax
```

⟨/2ekernel⟩

(*End of definition for* `\@obsoletefile` *and others.*)

File 21
ltoutenc.dtx

1 Font encodings

This section of the kernel contains commands for declaring encoding-specific commands, such as accents. It also contains the code for some of the encoding files, including `omlenc.def`, `omsenc.def`, `t1enc.def` and `ot1enc.def` files, which define the `OML`, `OMS`, `T1` and `OT1` encodings, and the `fontenc` package for selecting encodings.

The `fontenc` package has options for encodings, of which the last option is the default encoding. For example, to use the `OT2`, `OT3` and `T1` encodings, with `T1` as the default, you say:

```
\usepackage[OT2,OT3,T1]{fontenc}
```

The standard kernel set-up loads font encoding files and selects an encoding as follows.

```
\input {omlenc.def}
\input  {t1enc.def}
\input {ot1enc.def}
\input {omsenc.def}
\fontencoding{OT1}
```

Note that the files in the standard `inputenc` package depend on this behaviour of the kernel.

The syntax for declaring encoding-specific commands is:

```
\DeclareTextCommand{⟨command⟩}{⟨encoding⟩}
                                   [⟨number⟩][⟨default⟩]{⟨commands⟩}
```

This command is like `\newcommand`, except that it defines a command which is specific to one encoding. The resulting command is always robust, even if its definition is fragile. For example, the definition of `\l` in the `OT1` encoding is:

```
\DeclareTextCommand{\l}{OT1}{{\@xxxii l}}
```

`\DeclareTextCommand` takes the same optional arguments as `\newcommand`.

```
\ProvideTextCommand{⟨command⟩}{⟨encoding⟩}
                                   [⟨number⟩][⟨default⟩]{⟨commands⟩}
```

This acts like `\DeclareTextCommand`, but does nothing if the command is already defined.

```
\DeclareTextSymbol{⟨command⟩}{⟨encoding⟩}{⟨slot⟩}
```

This command defines a text symbol, with a particular slot in that encoding. The commands:

```
\DeclareTextSymbol{\ss}{OT1}{25}
\DeclareTextCommand{\ss}{OT1}{\char25 }
```

have the same effect, but the `\DeclareTextSymbol` is faster.

```
\DeclareTextAccent{⟨command⟩}{⟨encoding⟩}{⟨slot⟩}
```

This command declares a text accent. The commands:

```
\DeclareTextAccent{\"}{OT1}{127}
\DeclareTextCommand{\"}{OT1}{\add@accent {127}}
```

have the same effect.

\DeclareTextComposite{⟨*command*⟩}
{⟨*encoding*⟩}{⟨*argument*⟩}{⟨*slot*⟩}

This command declares a composite letter, for example in the `T1` encoding `\'{a}` is slot 225, which is declared by:

```
\DeclareTextComposite{\'}{T1}{a}{225}
```

The *command* will normally have been declared with `\DeclareTextAccent`, or as a one-argument `\DeclareTextCommand`.

`\DeclareTextComposite` is the most common example of using the more general declaration `\DeclareTextCompositeCommand`, which can define a composite to be an arbitrary piece of text.

\DeclareTextCompositeCommand{⟨*command*⟩}
{⟨*encoding*⟩}{⟨*argument*⟩}{⟨*text*⟩}

For example, in the OT1 encoding Å has a hand-crafted definition this is declared as follows

```
\DeclareTextCompositeCommand{\r}{OT1}{A}
 {\leavevmode\setbox\z@\hbox{!}\dimen@\ht\z@\advance\dimen@-1ex%
  \rlap{\raise.67\dimen@\hbox{\char23}}A}
```

The *command* will normally have been declared with `\DeclareTextAccent`, or as a one-argument `\DeclareTextCommand`.

The commands defined using the above declarations can be used in two ways. Normally they are used by just calling the command in the appropriate encoding, for example `\ss`. However, sometimes you may wish to use a command in an encoding where it is not defined. If the command has no arguments, then you can use it in another encoding by calling `\UseTextSymbol`:

\UseTextSymbol{⟨*encoding*⟩}{⟨*command*⟩}

For example, `\UseTextSymbol{OT1}{\ss}` has the same effect as:

```
{\fontencoding{OT1}\selectfont\ss}
```

If the command has one argument then you can use it in another encoding by calling `\UseTextAccent`:

\UseTextAccent{⟨*encoding*⟩}{⟨*command*⟩}{⟨*text*⟩}

For example, if the current encoding is `OT2` then `\UseTextAccent{OT1}{\'}{a}` has the same effect as:

```
{\fontencoding{OT1}\selectfont\'{\fontencoding{OT2}\selectfont a}}
```

You can also declare a default definition for a text command, which will be used if the current encoding has no appropriate definition. Such use will also set the definition for this command in the current encoding to equal this default definition; this makes subsequent uses of the command much faster.

```
\DeclareTextCommandDefault{⟨command⟩}{⟨definition⟩}
```

For example, the default definition of the command `\textonequarter` (which produces the fraction $\frac{1}{4}$) could be built using math mode:

```
\DeclareTextCommandDefault{\textonequarter}{\ensuremath {\frac14}}
```

There is a matching `\Provide` command which will not override an existing default definition:

```
\ProvideTextCommandDefault{⟨command⟩}{⟨definition⟩}
```

The most common use for these commands is to use symbols from other encodings, so there are some optimizations provided:

```
\DeclareTextSymbolDefault{⟨command⟩}{⟨encoding⟩}
\DeclareTextAccentDefault{⟨command⟩}{⟨encoding⟩}
```

are short for:

```
\DeclareTextCommandDefault{⟨command⟩}
                          {\UseTextSymbol{⟨encoding⟩}{⟨command⟩}}
\DeclareTextCommandDefault[1]{⟨command⟩}
                        {\UseTextAccent{⟨encoding⟩}{⟨command⟩}{#1}}
```

For example, to make `OT1` the default encoding for `\ss` and `\'` you say:

```
\DeclareTextSymbolDefault{\ss}{OT1}
\DeclareTextAccentDefault{\'}{OT1}
```

Note that you can use these commands on any zero- or one-argument commands declared with `\DeclareText*` or `\ProvideText*`, not just those defined using `\DeclareTextSymbol` or `\DeclareTextAccent`.

1.1 Removing encoding-specific commands

In some cases encoding definitions are given to provide some limited support since nothing better is available, for example, the definition for `\textdollar` in `OT1` is a hack since \$ and £ actually share the same slot in this encoding. Thus if such a glyph becomes available in a different encoding (e.g., `TS1`) one would like to get rid of the flaky one and make the default definition point to the new encoding. In such a case defining

```
\DeclareTextSymbol{\textdollar}{TS1}{36}
\DeclareTextSymbolDefault{\textdollar}{TS1}
```

is not enough since if typesetting in `OT1` LaTeX will still find the encoding specific-definition for `OT1` and therefore ignore the new default. Therefore to ensure that in this case the `TS1` version is used we have to remove the `OT1` declaration:

```
\UndeclareTextCommand{\textdollar}{OT1}
```

Since the $ sign is a proper glyph in the T1 encoding there is no point removing its definition and forcing LaTeX to pick up the TS1 version if typesetting in this encoding. However, assume you want to use the variant dollar sign, i.e., $ for your dollars. In that case you have to get rid of the T1 declaration as well, e.g., the following would do that for you:

```
\UndeclareTextCommand{\textdollar}{OT1}
\UndeclareTextCommand{\textdollar} {T1}
\DeclareTextCommandDefault{\textdollar}
                          {\UseTextSymbol{TS1}\textdollaroldstyle}
```

1.2 The order of declarations

If an encoding-specific command is defined for more than one encoding, then it will execute fastest in the encoding in which it was defined last since its top-level definition will be set up to execute in that encoding without any overhead.

For this reason the file fonttext.ltx currently first loads the definitions for the T1 encoding and then those for the OT1 encoding so that typesetting in OT1 is optimized since that is (still) the default. However, when T1 is explicitly requested (via \usepackage[T1]{fontenc}) the top-level definitions are automatically changed to favour T1 since its declarations are reloaded in the process.

For the same reason default declarations should never come last since they are implemented as a special encoding themselves (with the name ?). Specifying them last would simply mean to make those encoding-specific commands equally inefficient in all encodings. Therefore the textcomp package, for example, first sets up all defaults to point to TS1 and then declares the commands in the TS1 encoding.

1.3 Docstrip modules

This .dtx file is be used to generate several related files containing font encoding definitions. The mutually exclusive docstrip options are listed here.

T1	generates t1enc.def for the Cork encoding.
TS1	generates ts1enc.def for the Text Companion encoding.
TS1sty	generates textcomp.sty, package that sets up use of the Text Companion encoding.
OT1	generates ot1enc.def for Knuth's CM encoding.
OMS	generates omsenc.def for Knuth's math symbol encoding.
OML	generates omlenc.def for Knuth's math letters encoding.
OT4	generates ot4enc.def for the Polish extension to the OT1 encoding, created by B. Jackowski and M. Ryćko for use with the Polish version of Computer Modern and Computer Concrete.
TU	generates tuenc.def for Unicode font encoding.
package	generates fontenc.sty for selecting encodings.
2ekernel	for the kernel commands.

1.4 Definitions for the kernel

1.4.1 Declaration commands

This section contains definitions for commands such as accents which depend on the current encoding. These commands will usually be kept in `.def` files, for example `ot1enc.def` contains the definitions for the `OT1` encoding.

```
⟨*2ekernel⟩
\message{font encodings,}
```

Far too many macros in one block here!

\DeclareTextCommand
\ProvideTextCommand
\DeclareTextSymbol
\@dec@text@cmd
\chardef@text@cmd
\@changed@cmd
\@changed@x
\TextSymbolUnavailable
\@inmathwarn

If you say:

```
\DeclareTextCommand{\foo}{T1}...
```

then `\foo` is defined to be `\T1-cmd \foo \T1\foo`, where `\T1\foo` is *one* control sequence, not two! We then call `\newcommand` to define `\T1\foo`.

```
\def\DeclareTextCommand{%
   \@dec@text@cmd\newcommand}
\def\ProvideTextCommand{%
   \@dec@text@cmd\providecommand}

\def\@enc@info#1{%
         \GenericInfo{(Encoding)\@spaces\@spaces\@spaces\space\space}%
                     {LaTeX Encoding Info: \space\space\space#1}}%
\def\@dec@text@cmd#1#2#3{%
```

For logging, there are a number of different cases to cater for.

```
   \ifcsname #3\string#2\endcsname
   \@enc@info{%
       \ifx#1\providecommand
          Ignoring declaration for
        \else
          Redeclaring
        \fi
      text
```

This test distinguishes between commands and symbols:

```
      \ifx#1\chardef@text@cmd
        symbol
      \else
        command
      \fi
      \string#2 (encoding #3)}%
    \fi
   \expandafter\def\expandafter#2%
      \expandafter{%
         \csname#3-cmd\expandafter\endcsname
         \expandafter#2%
         \csname#3\string#2\endcsname
      }%
   \let\@ifdefinable\@rc@ifdefinable
   \expandafter#1\csname#3\string#2\endcsname}
```

This command was introduced to fix a major bug in \@dec@text@cmd without changing that command itself. This was thought to be necessary because it is defined in more than one package. (Perhaps the more serious bug is to put complex low-level commands like this in packages?)

The problem it solves is that whereas both \newcommand and \providecommand (used just above) both handle the resetting of \@ifdefinable (following its disabling in \@dec@text@cmd), the primitive \chardef neither needs the disabling, nor does the resetting.

```
\def\chardef@text@cmd{%
   \let\@ifdefinable\@@ifdefinable
   \chardef
  }
\def\DeclareTextSymbol#1#2#3{%
   \@dec@text@cmd\chardef@text@cmd#1{#2}#3\relax
  }
```

The declarations are only available before \begin{document}.

```
\@onlypreamble\DeclareTextCommand
\@onlypreamble\DeclareTextSymbol
```

The sneaky bit in all this is what \T1-cmd \foo \T1\foo does. There are five possibilities, depending on the current values of \protect, \cf@encoding and \ifmmode:

- If \protect is \@typeset@protect and \cf@encoding is T1, then we execute \T1\foo. This should be the normal behaviour, and is optimized for speed.
- If \protect is \@typeset@protect, \cf@encoding is (say) OT1, and \OT1\foo is defined, then we execute \OT1\foo.
- If \protect is \@typeset@protect, \cf@encoding is (say) OT1, we're in text mode, and \OT1\foo is undefined, then we define \OT1\foo to be the default value of \foo, and execute \OT1\foo.
- If \protect is \@typeset@protect, \cf@encoding is (say) OT1, we're in math mode, and \OT1\foo is undefined, then we execute the default value of \foo. (This is necessary so that things like $X_\copyright$ work properly.)
- If \protect is not \@typeset@protect then we execute \noexpand\foo. For example, if we are writing to a file, then this results in \foo being written. If we are in a \mark, then \foo will be put in the mark—since \foo is robust, it will then survive all the things which may happen to it whilst it's a \mark.

So after all that, we will either execute the appropriate definition of \foo for the current encoding, or we will execute \noexpand\foo.

The default value of \foo is \?\foo if it is defined, and an error message otherwise.

When the encoding is changed from T1 to OT1, \T1-cmd is defined to be \@changed@cmd and \OT1-cmd is defined to be \@current@cmd. This means that the test for what the current encoding is can be performed quickly.

```
\def\@current@cmd#1{%
   \ifx\protect\@typeset@protect
      \@inmathwarn#1%
   \else
      \noexpand#1\expandafter\@gobble
   \fi}
```

```
\def\@changed@cmd#1#2{%
   \ifx\protect\@typeset@protect
      \@inmathwarn#1%
      \expandafter\ifx\csname\cf@encoding\string#1\endcsname\relax
         \expandafter\ifx\csname ?\string#1\endcsname\relax
            \expandafter\def\csname ?\string#1\endcsname{%
               \TextSymbolUnavailable#1%
            }%
         \fi
         \global\expandafter\let
               \csname\cf@encoding \string#1\expandafter\endcsname
               \csname ?\string#1\endcsname
      \fi
      \csname\cf@encoding\string#1%
         \expandafter\endcsname
   \else
      \noexpand#1%
   \fi}

\gdef\TextSymbolUnavailable#1{%
   \@latex@error{%
      Command \protect#1 unavailable in encoding \cf@encoding%
   }\@eha}
```

The command `\@inmathwarn` produces a warning message if we are currently in math mode. Note that since this command is used inside text commands, it can't call `\relax` before the `\ifmmode`. This means that it is possible for the warning to fail to be issued at the beginning of a row of an halign whose template enters math mode. This is probably a bad feature, but there's not much that can be done about it, since adding a `\relax` would break ligatures and kerning between text symbols.

A more efficient solution would be to make `\@inmathwarn` and `\@inmatherr` equal to `\@empty` and `\relax` by default, and to have `\everymath` reset them to their usual definitions. This is left for future investigation (for example it may break some third party code).

```
\def\@inmathwarn#1{%
   \ifmmode
      \@latex@warning{Command \protect#1 invalid in math mode}%
   \fi}
```

(*End of definition for* `\DeclareTextCommand` *and others.*)

`\DeclareTextCommandDefault`
`\ProvideTextCommandDefault`

These define commands with encoding `?`.

Note that `\DeclareTextCommandDefault` can only be used in the preamble, but that the `\Provide` version is allowed in inputenc `.def` files, so is allowed anywhere.

```
\def\DeclareTextCommandDefault#1{%
   \DeclareTextCommand#1?}

\def\ProvideTextCommandDefault#1{%
   \ProvideTextCommand#1?}

\@onlypreamble\DeclareTextCommandDefault
%\@onlypreamble\ProvideTextCommandDefault
```

They require `\?-cmd` to be initialized as `\@changed@cmd`.

```
\expandafter\let\csname?-cmd\endcsname\@changed@cmd
```

(*End of definition for* `\DeclareTextCommandDefault` *and* `\ProvideTextCommandDefault`.)

`\DeclareTextAccent` This is just a disguise for defining a TEX `\accent` command.

```
\def\DeclareTextAccent#1#2#3{%
   \DeclareTextCommand#1{#2}{\add@accent{#3}}}
\@onlypreamble\DeclareTextAccent
```

(*End of definition for* `\DeclareTextAccent`.)

`\add@accent` To save space this code is shared between all text accents that are set using the `\accent` primitive. The argument is pre-set in a box so that any font loading that is needed is already done within the box. This is needed because font-loading involves grouping and that would prevent the accent mechanism from working so that the accent would not be positioned over the argument. Declarations that change the font should be allowed (only low-level ones are at present) inside the argument of an accent command, but not size changes, as they involve `\setbox` operations which also inhibit the mechanism of the `\accent` primitive.

Note that the whole process is within a group. For a detailed discussion of this reimplementation and its deficiencies, see pr/3160.

```
\def\add@accent#1#2{\hmode@bgroup
```

Turn off the group in `\UseTextSymbol` in case this is used inside the argument of `\add@accent`.

```
   \let\hmode@start@before@group\@firstofone
   \setbox\@tempboxa\hbox{#2%
```

When presetting the argument in a box we record its `\spacefactor` for later use after the accent got typeset. This way something like `\‘A` gets the spacefactor of `A` (i.e., 999) rather than the default value of 1000.

```
      \global\mathchardef\accent@spacefactor\spacefactor}%
```

The accent primitive doesn't allow things `\begingroup` to interfere between accent and base character. Therefore we need to avoid that (they are some hidden inside `\maybe@load@fontshape`). As we don't have to load the fontshape in this case (as that already happened in the box above, if necessary), we simply disable that part of the code temporarily. We also ignore `\ignorespaces` which has the same issue and may show up as part of `\normalfont` if that is used.

```
   \let\maybe@load@fontshape\relax
   \let\ignorespaces\relax
   \accent#1 #2\egroup\ifmmode\else\spacefactor\accent@spacefactor\fi}
```

Default definition for `\accent@spacefactor` prevents a horrible death of the above macro inside an unprotected `\edef`.

```
\let\accent@spacefactor\relax
```

(*End of definition for* `\add@accent`.)

`\hmode@bgroup`

```
\def\hmode@bgroup{\leavevmode\bgroup}
```

(*End of definition for* `\hmode@bgroup`.)

\DeclareTextCompositeCommand
\DeclareTextComposite
\@text@composite
\@text@composite@x
\@strip@args

Another amusing game to play with \expandafter, \csname, and \string. When you say \DeclareTextCompositeCommand{\foo}{T1}{a}{bar}, we look to see if the expansion of \T1\foo begins with \@text@composite, and if it doesn't, we redefine \T1\foo to be:

```
#1 -> \@text@composite \T1\foo #1\@empty \@text@composite {...}
```

where ... is the previous definition of \T1\foo. Finally, we define \\T1\foo-a to expand to bar.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2017/04/15}{\DeclareTextCompositeCommand}
⟨latexrelease⟩                                {test for undeclared accent}%
⟨*2ekernel | latexrelease⟩
\def\DeclareTextCompositeCommand#1#2#3#4{%
  \expandafter\let\expandafter\reserved@a\csname#2\string#1\endcsname
  \ifx\reserved@a\relax

   \DeclareTextCommand#1{#2}{%
     \@latex@error{Text composite \string#1 undeclared in encoding #2}\@eha}%
   \@enc@info{Text composite with undeclared \string#1 in encoding #2}%
   \expandafter\let\expandafter\reserved@a\csname#2\string#1\endcsname
  \fi
  \expandafter\expandafter\expandafter\ifx
  \expandafter\@car\reserved@a\relax\relax\@nil \@text@composite \else
      \edef\reserved@b##1{%
         \def\expandafter\noexpand
            \csname#2\string#1\endcsname####1{%
            \noexpand\@text@composite
               \expandafter\noexpand\csname#2\string#1\endcsname
               ####1\noexpand\@empty\noexpand\@text@composite
               {##1}}}%
      \expandafter\reserved@b\expandafter{\reserved@a{##1}}%
   \fi
   \expandafter\def\csname\expandafter\string\csname
      #2\endcsname\string#1-\string#3\@empty\endcsname{#4}%
  }
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\DeclareTextCompositeCommand}
⟨latexrelease⟩                                {test for undeclared accent}%
⟨latexrelease⟩\def\DeclareTextCompositeCommand#1#2#3#4{%
⟨latexrelease⟩  \expandafter\let\expandafter\reserved@a
⟨latexrelease⟩                        \csname#2\string#1\endcsname
⟨latexrelease⟩  \expandafter\expandafter\expandafter\ifx
⟨latexrelease⟩  \expandafter\@car\reserved@a\relax\relax\@nil
⟨latexrelease⟩                                  \@text@composite \else
⟨latexrelease⟩      \edef\reserved@b##1{%
⟨latexrelease⟩         \def\expandafter\noexpand
⟨latexrelease⟩           \csname#2\string#1\endcsname####1{%
⟨latexrelease⟩           \noexpand\@text@composite
⟨latexrelease⟩             \expandafter\noexpand\csname#2\string#1\endcsname
⟨latexrelease⟩             ####1\noexpand\@empty\noexpand\@text@composite
⟨latexrelease⟩             {##1}}}%
⟨latexrelease⟩      \expandafter\reserved@b\expandafter{\reserved@a{##1}}%
```

```
⟨latexrelease⟩    \fi
⟨latexrelease⟩    \expandafter\def\csname\expandafter\string\csname
⟨latexrelease⟩       #2\endcsname\string#1-\string#3\@empty\endcsname{#4}}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩

\@onlypreamble\DeclareTextCompositeCommand
```

This all works because:

```
\@text@composite \T1\foo A\@empty \@text@composite {...}
```

expands to `\\T1\foo-A` if `\\T1\foo-A` has been defined, and `{...}` otherwise.

Note that `\@text@composite` grabs the first token of the argument and puts just that in the csname. This is so that `\'{\textit{e}}` will work—it checks whether `\\T1\'-\textit` is defined (which presumably it isn't) and so expands to `{\accent 1 \textit{e}}`.

This trick won't always work, for example `\'{{\itshape e}}` will expand to (with spaces added for clarity):

```
\csname \string \T1\' - \string {\itshape e} \@empty \endcsname
```

which will die pretty horribly. Unfortunately there's not much can be done about this if we're going to use `\csname` lookups as a fast way of accessing composites.

This has an unfortunate 'misfeature' though, which is that in the T1 encoding, `\'{aa}` produces á. This is not the expected behaviour, and should perhaps be fixed if the fix doesn't affect performance too badly.

Finally, it's worth noting that the `\@empty` is used in `\@text@composite` so that accents will work even when the argument is empty. If you say `\'{}` then this looks up `\\T1\'-\@empty`, which ought to be `\relax`, and so all is well. If we didn't include the `\@empty`, then `\'{}` would expand to:

```
\csname \string \T1\' - \string \endcsname
```

so the `\endcsname` would be `\string`'ed and the whole of the rest of the document would be put inside the `\csname`. This would not be good.

```
\def\@text@composite#1#2#3\@text@composite{%
   \expandafter\@text@composite@x
      \csname\string#1-\string#2\endcsname}
```

Originally the `\@text@composite@x` macro had two arguments and if `#1` was not `\relax` it was executed, otherwise `#2` was executed. All this happened within the `\ifx` code so that neither `#1` nor `#2` could have picked up any additional arguments form the input stream. This has now being changed using the typical `\@firstoftwo` / `\@secondoftwo` coding. This way the final expansion will happen without any `\else` or `\fi` intervening in the case that we need to get a further token from the input stream.

```
\def\@text@composite@x#1{%
   \ifx#1\relax
      \expandafter\@secondoftwo
   \else
      \expandafter\@firstoftwo
   \fi
   #1}
```

The command \DeclareTextComposite uses \DeclareTextCompositeCommand to declare a command which expands out to a single glyph.

```
\catcode\z@=11\relax
\def\DeclareTextComposite#1#2#3#4{%
   \def\reserved@a{\DeclareTextCompositeCommand#1{#2}{#3}}%
   \bgroup
      \lccode\z@#4%
      \lowercase{%
   \egroup
      \reserved@a ^^@}}
\catcode\z@=15\relax
\@onlypreamble\DeclareTextComposite
```

(*End of definition for* `\DeclareTextCompositeCommand` *and others.*)

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                    {\UseTextAccent}{Make commands robust}%
```

\UseTextAccent
\UseTextSymbol
\@use@text@encoding

These fragile commands access glyphs from different encodings. They use grotty low-level calls to the font selection scheme for speed, and in order to make sure that \UseTextSymbol doesn't do anything which you're not allowed to do between an \accent and its glyph.

For a detailed discussion of this reimplementation and its deficiencies, see pr/3160.

```
\DeclareRobustCommand*\UseTextAccent[3]{%
  \hmode@start@before@group
   {%
```

Turn off the group in \UseTextSymbol in case this is used inside the arguments of \UseTextAccent.

```
    \let\hmode@start@before@group\@firstofone
    \let\@curr@enc\cf@encoding
    \@use@text@encoding{#1}%
    #2{\@use@text@encoding\@curr@enc#3}%
   }}
\DeclareRobustCommand*\UseTextSymbol[2]{%
       \hmode@start@before@group
       {%
          \def\@wrong@font@char{\MessageBreak
             for \noexpand\symbol`\string#2'}%
          \@use@text@encoding{#1}%
          #2%
       }%
    }
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\UseTextAccent}{Make commands robust}%
⟨latexrelease⟩
⟨latexrelease⟩\kernel@make@fragile\UseTextAccent
```

```
⟨latexrelease⟩\kernel@make@fragile\UseTextSymbol
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

Switch to a different text encoding without any grouping for use in `\UseTextAccent` or `\UseTextSymbol` (and for `\oldstylenums`).

```
\def\@use@text@encoding#1{%
   \edef\f@encoding{#1}%
   \xdef\font@name{%
      \csname\curr@fontshape/\f@size\endcsname}%
   \pickup@font
   \font@name
   \@@enc@update}
```

(*End of definition for* `\UseTextAccent`, `\UseTextSymbol`, *and* `\@use@text@encoding`.)

`\hmode@start@before@group` The `\hmode@start@before@group` starts hmode and should be immediately followed by an explicit `{...}`. Its purpose is to ensure that hmode is started before this group is opened. Inside `\add@accent` and `\UseTextAccent` it is redefined to remove this group so that it doesn't conflict with the `\accent` primitive.

For a detailed discussion see pr/3160.

```
\let\hmode@start@before@group\leavevmode
```

(*End of definition for* `\hmode@start@before@group`.)

`\DeclareTextSymbolDefault` `\DeclareTextAccentDefault` Some syntactic sugar. Again, these should probably be optimized for speed.

```
\def\DeclareTextSymbolDefault#1#2{%
   \DeclareTextCommandDefault#1{\UseTextSymbol{#2}#1}}
\def\DeclareTextAccentDefault#1#2{%
   \DeclareTextCommandDefault#1{\UseTextAccent{#2}#1}}
\@onlypreamble\DeclareTextSymbolDefault
\@onlypreamble\DeclareTextAccentDefault
```

(*End of definition for* `\DeclareTextSymbolDefault` *and* `\DeclareTextAccentDefault`.)

`\UndeclareTextCommand` This command safely removes an encoding specific declaration for a given encoding. It is helpful if one intends to use the default definition always and therefore wants to get rid of a declaration for some specific encoding.

```
\def\UndeclareTextCommand#1#2{%
```

If there is no declaration for the current encoding do nothing.

```
  \ifcsname #2\string#1\endcsname
```

Else: throw away that declaration.

```
     \@enc@info{Undeclare text command \string#1 (encoding #2)}%
     \global\expandafter\let\csname#2\string#1\endcsname
            \@undefined
```

But this is unfortunately not enough, we have to take a look at the top-level definition of the encoding specific command which for a command `\foo` would look similar to `\T1-cmd \foo \T1\foo` (three tokens).

Of course, instead of `T1` one could see a different encoding name; which one depends the encoding for which `\foo` was declared last.

Now assume we have just removed the declaration for `\foo` in `T1` and the top-level of `\foo` expands to the above. Then we better change that pretty fast otherwise we do get an "undefined csname error" when we try to typeset `\foo` within `T1` instead of getting the default definition for `\foo`. And what is the best way to change that top-level definition? Well, the only "encoding" we know for sure will still be around is the default encoding denoted by `?`.

Thus in case the last token of the top-level expansion is now undefined we change the declaration to look like `\?-cmd \foo \?\foo` which is done by the following (readable?) code:

```
    \expandafter\expandafter\expandafter
    \ifx\expandafter\@thirdofthree#1\@undefined
      \expandafter\gdef\expandafter#1\expandafter
         {\csname ?-cmd\expandafter\endcsname\expandafter
          #1\csname?\string#1\endcsname}%
    \fi
  \else
    \@enc@info{Text command \string#1 (encoding #2) is not declared}%
  \fi
}
\@onlypreamble\UndeclareTextCommand
```

(*End of definition for* `\UndeclareTextCommand`.)

1.4.2 Hyphenation

`\patterns` `\@@patterns` `\hyphenation` `\@@hyphenation`

We redefine `\patterns` and `\hyphenation` to allow the use of commands declared with `\DeclareText*` to be used inside them.

```
%\let\@@patterns\patterns
%\let\@@hyphenation\hyphenation
%\def\patterns{%
%   \bgroup
%      \let\protect\@empty
%      \let\@typeset@protect\@empty
%      \let\@changed@x\@changed@x@mouth
%   \afterassignment\egroup
%   \@@patterns
%}
%\def\hyphenation{%
%   \bgroup
%      \let\protect\@empty
%      \let\@typeset@protect\@empty
%      \let\@changed@x\@changed@x@mouth
%   \afterassignment\egroup
%   \@@hyphenation
%}
```

(*End of definition for* `\patterns` *and others.*)

1.4.3 Miscellania

\a The \a command is used to access the accent commands even when they have been redefined (for example by the tabbing environment). Its internal name is \@tabacckludge.

The \string within the \csname guards against something like ' being active at the point of use.

```
\def\@tabacckludge#1{\expandafter\@changed@cmd
                                  \csname\string#1\endcsname\relax}
\let\a=\@tabacckludge
```

(*End of definition for* \a.)

1.4.4 Default encodings

We define the default encodings for most commands to be either OT1, OML or OMS. These defaults are in the kernel and therefore fonts with these encodings must be available unless these defaults are redefined elsewhere. Recall that the standard kernel loads the encoding files for these encodings, and also that for the T1 encoding.

The naming conventions in the kernel are not what we would use if we were starting from scratch... Those defined by DEK (like \ae and \ss) or by the TEX Users Group Technical Working Group on multi-lingual typesetting (like \th and \ng) have short names. Those which were added to the kernel in 1993 and early 1994 are named after their Adobe glyph names (like \guillemotleft and \quotedblbase). Unfortunately, this naming scheme won't work for all glyphs, since some names (like \space) are already used, and some (like \endash) are very likely to be defined by users. So we're now using the naming scheme of \text followed by the Adobe name, (like \textendash and \textsterling). Except that some glyphs don't have Adobe names, so we're using the names used by fontinst for those (like \textcompwordmark). Sigh.

Some accents from OT1:

```
\DeclareTextAccentDefault{\"}{OT1}
\DeclareTextAccentDefault{\'}{OT1}
\DeclareTextAccentDefault{\.}{OT1}
\DeclareTextAccentDefault{\=}{OT1}
\DeclareTextAccentDefault{\H}{OT1}
\DeclareTextAccentDefault{\^}{OT1}
\DeclareTextAccentDefault{\`}{OT1}
\DeclareTextAccentDefault{\b}{OT1}
\DeclareTextAccentDefault{\c}{OT1}
\DeclareTextAccentDefault{\d}{OT1}
\DeclareTextAccentDefault{\r}{OT1}
\DeclareTextAccentDefault{\u}{OT1}
\DeclareTextAccentDefault{\v}{OT1}
\DeclareTextAccentDefault{\~}{OT1}
```

Some symbols from OT1:

```
%\DeclareTextSymbolDefault{\AA}{OT1}
\DeclareTextSymbolDefault{\AE}{OT1}
\DeclareTextSymbolDefault{\L}{OT1}
\DeclareTextSymbolDefault{\OE}{OT1}
\DeclareTextSymbolDefault{\O}{OT1}
%\DeclareTextSymbolDefault{\aa}{OT1}
\DeclareTextSymbolDefault{\ae}{OT1}
\DeclareTextSymbolDefault{\i}{OT1}
```

```
\DeclareTextSymbolDefault{\j}{OT1}

\DeclareTextSymbolDefault{\ij}{OT1}
\DeclareTextSymbolDefault{\IJ}{OT1}

\DeclareTextSymbolDefault{\l}{OT1}
\DeclareTextSymbolDefault{\oe}{OT1}
\DeclareTextSymbolDefault{\o}{OT1}
\DeclareTextSymbolDefault{\ss}{OT1}
\DeclareTextSymbolDefault{\textdollar}{OT1}
\DeclareTextSymbolDefault{\textemdash}{OT1}
\DeclareTextSymbolDefault{\textendash}{OT1}
\DeclareTextSymbolDefault{\textexclamdown}{OT1}
%\DeclareTextSymbolDefault{\texthyphenchar}{OT1}
%\DeclareTextSymbolDefault{\texthyphen}{OT1}
\DeclareTextSymbolDefault{\textquestiondown}{OT1}
\DeclareTextSymbolDefault{\textquotedblleft}{OT1}
\DeclareTextSymbolDefault{\textquotedblright}{OT1}
\DeclareTextSymbolDefault{\textquoteleft}{OT1}
\DeclareTextSymbolDefault{\textquoteright}{OT1}
\DeclareTextSymbolDefault{\textsterling}{OT1}
```

Some symbols from OMS:

```
\DeclareTextSymbolDefault{\textasteriskcentered}{OMS}
\DeclareTextSymbolDefault{\textbackslash}{OMS}
\DeclareTextSymbolDefault{\textbar}{OMS}
\DeclareTextSymbolDefault{\textbardbl}{OMS}
\DeclareTextSymbolDefault{\textbraceleft}{OMS}
\DeclareTextSymbolDefault{\textbraceright}{OMS}
\DeclareTextSymbolDefault{\textbullet}{OMS}
\DeclareTextSymbolDefault{\textdaggerdbl}{OMS}
\DeclareTextSymbolDefault{\textdagger}{OMS}
\DeclareTextSymbolDefault{\textparagraph}{OMS}
\DeclareTextSymbolDefault{\textperiodcentered}{OMS}
\DeclareTextSymbolDefault{\textsection}{OMS}
\DeclareTextAccentDefault{\textcircled}{OMS}
```

Some symbols from OML:

```
\DeclareTextSymbolDefault{\textless}{OML}
\DeclareTextSymbolDefault{\textgreater}{OML}
\DeclareTextAccentDefault{\t}{OML}
```

Some defaults we can fake.

The interface for defining `\copyright` changed, it used to use `\expandafter` to add braces at the appropriate points.

```
\DeclareTextCommandDefault{\textcopyright}{\textcircled{c}}
% \expandafter\def\expandafter
%                 \copyright\expandafter{\expandafter{\copyright}}

\DeclareTextCommandDefault{\textasciicircum}{\^{}}
\DeclareTextCommandDefault{\textasciitilde}{\~{}}
\DeclareTextCommandDefault{\textunderscore}{%
  \leavevmode \kern.06em\vbox{\hrule\@width.3em}}
```

There is no good reason anymore to fake `\textcompwordmark`.

```
%\DeclareTextCommandDefault{\textcompwordmark}{\leavevmode\kern\z@}
\DeclareTextSymbolDefault{\textcompwordmark}{T1}
```

```
\DeclareTextCommandDefault{\textvisiblespace}{%
   \mbox{\kern.06em\vrule \@height.3ex}%
   \vbox{\hrule \@width.3em}%
   \hbox{\vrule \@height.3ex}}
```

Using `\fontdimen3` in the next definition is some sort of a kludge (since it is the interword stretch) but it makes the ellipsis come out right in mono-spaced fonts too (since there it is zero).

```
\DeclareTextCommandDefault{\textellipsis}{%
   .\kern\fontdimen3\font
   .\kern\fontdimen3\font
   .\kern\fontdimen3\font}

%\DeclareTextCommandDefault{\textregistered}{\textcircled{\scshape r}}
\DeclareTextCommandDefault{\textregistered}{\textcircled{%
     \check@mathfonts\fontsize\sf@size\z@\math@fontsfalse\selectfont R}}
\DeclareTextCommandDefault{\texttrademark}{\textsuperscript{TM}}
\DeclareTextCommandDefault{\SS}{SS}

\DeclareTextCommandDefault{\textordfeminine}{\textsuperscript{a}}
\DeclareTextCommandDefault{\textordmasculine}{\textsuperscript{o}}
```

1.4.5 Math material

Some commands can be used in both text and math mode:

```
\DeclareRobustCommand{\$}{\ifmmode\mathdollar\else\textdollar\fi}
```

We use `\protected` not `\DeclareRobustCommand` so that `\bigl\{` etc. works inside `\protected@edef`.

```
\protected\def\{{\ifmmode\lbrace\else\textbraceleft\fi}
\protected\def\}{\ifmmode\rbrace\else\textbraceright\fi}

\DeclareRobustCommand{\P}{\ifmmode\mathparagraph\else\textparagraph\fi}
\DeclareRobustCommand{\S}{\ifmmode\mathsection\else\textsection\fi}
\DeclareRobustCommand{\dag}{\ifmmode{\dagger}\else\textdagger\fi}
\DeclareRobustCommand{\ddag}{\ifmmode{\ddagger}\else\textdaggerdbl\fi}
```

For historical reasons `\copyright` needs `{}` around the definition in maths.

```
\DeclareRobustCommand{\_}{%
   \ifmmode\nfss@text{\textunderscore}\else\textunderscore\fi}
\DeclareRobustCommand{\copyright}{%
   \ifmmode{\nfss@text{\textcopyright}}\else\textcopyright\fi}
\DeclareRobustCommand{\pounds}{%
   \ifmmode\mathsterling\else\textsterling\fi}

\DeclareRobustCommand{\dots}{%
   \ifmmode\mathellipsis\else\textellipsis\fi}

\let\ldots\dots
```

Default definition of the commabelow accent.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/10/01}{\textcommabelow}{comma accent}%
⟨*2ekernel | latexrelease⟩
\DeclareTextCommandDefault\textcommabelow[1]
  {\hmode@bgroup\ooalign{\null#1\crcr\hidewidth\raise-.31ex
    \hbox{\check@mathfonts\fontsize\ssf@size\z@
```

```
   \math@fontsfalse\selectfont,}\hidewidth}\egroup}
⟨latexrelease⟩\EndIncludeInRelease
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\textcommabelow}{comma accent}%
⟨latexrelease⟩\let\textcommabelow\@undefined
⟨latexrelease⟩\expandafter
⟨latexrelease⟩   \let\csname\string\T1\string\c-G\endcsname\@undefined
⟨latexrelease⟩\expandafter
⟨latexrelease⟩   \let\csname\string\T1\string\c-K\endcsname\@undefined
⟨latexrelease⟩\expandafter
⟨latexrelease⟩   \let\csname\string\T1\string\c-k\endcsname\@undefined
⟨latexrelease⟩\expandafter
⟨latexrelease⟩   \let\csname\string\T1\string\c-L\endcsname\@undefined
⟨latexrelease⟩\expandafter
⟨latexrelease⟩   \let\csname\string\T1\string\c-l\endcsname\@undefined
⟨latexrelease⟩\expandafter
⟨latexrelease⟩   \let\csname\string\T1\string\c-N\endcsname\@undefined
⟨latexrelease⟩\expandafter
⟨latexrelease⟩   \let\csname\string\T1\string\c-n\endcsname\@undefined
⟨latexrelease⟩\expandafter
⟨latexrelease⟩   \let\csname\string\T1\string\c-R\endcsname\@undefined
⟨latexrelease⟩\expandafter
⟨latexrelease⟩   \let\csname\string\T1\string\c-r\endcsname\@undefined
⟨latexrelease⟩\EndIncludeInRelease
```

Default definition of the commaabove accent(E.G.).

```
⟨latexrelease⟩\IncludeInRelease{2016/02/01}{\textcommaabove}{comma above}%
⟨∗2ekernel | latexrelease⟩
\DeclareTextCommandDefault\textcommaabove[1]{%
  \hmode@bgroup
  \ooalign{%
    \hidewidth
    \raise.7ex\hbox{%
      \check@mathfonts\fontsize\ssf@size\z@\math@fontsfalse\selectfont‘%
    }%
   \hidewidth\crcr
   \null#1\crcr
  }%
  \egroup
}
⟨latexrelease⟩\EndIncludeInRelease
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\textcommaabove}{comma above}%
⟨latexrelease⟩\let\textcommaabove\@undefined
⟨latexrelease⟩\expandafter
⟨latexrelease⟩   \let\csname\string\OT1\string\c-g\endcsname\@undefined
⟨latexrelease⟩\expandafter
⟨latexrelease⟩   \let\csname\string\T1\string\c-g\endcsname\@undefined
⟨latexrelease⟩\EndIncludeInRelease
```

1.5 Definitions for the OT1 encoding

The definitions for the ‘TEX text’ (OT1) encoding.

Declare the encoding.

```
⟨*OT1⟩
\DeclareFontEncoding{OT1}{}{}
```

Providing font substitution is essential if accents are missing in a font and substitution tries to find it elsewhere. Without it you might receive a "Corrupted NFSS tables" error.

```
\DeclareFontSubstitution{OT1}{cmr}{m}{n}
```

Declare the accents.

```
\DeclareTextAccent{\"}{OT1}{127}
\DeclareTextAccent{\'}{OT1}{19}
\DeclareTextAccent{\.}{OT1}{95}
\DeclareTextAccent{\=}{OT1}{22}
\DeclareTextAccent{\^}{OT1}{94}
\DeclareTextAccent{\`}{OT1}{18}
\DeclareTextAccent{\~}{OT1}{126}
\DeclareTextAccent{\H}{OT1}{125}
\DeclareTextAccent{\u}{OT1}{21}
\DeclareTextAccent{\v}{OT1}{20}
\DeclareTextAccent{\r}{OT1}{23}
```

Some accents have to be built by hand: Note that `\ooalign` and `\o@lign` must be inside a group. In these definitions we no longer use the helper function `\sh@ft` from plain.tex since that now has two incompatible definitions.

```
\DeclareTextCommand{\b}{OT1}[1]
   {\hmode@bgroup\o@lign{\relax#1\crcr\hidewidth\ltx@sh@ft{-3ex}%
     \vbox to.2ex{\hbox{\char22}\vss}\hidewidth}\egroup}
\DeclareTextCommand{\c}{OT1}[1]
   {\leavevmode\setbox\z@\hbox{#1}\ifdim\ht\z@=1ex\accent24 #1%
    \else{\ooalign{\unhbox\z@\crcr\hidewidth\char24\hidewidth}}\fi}
\DeclareTextCommand{\d}{OT1}[1]
   {\hmode@bgroup
    \o@lign{\relax#1\crcr\hidewidth\ltx@sh@ft{-1ex}.\hidewidth}\egroup}
```

Declare the text symbols.

```
\DeclareTextSymbol{\AE}{OT1}{29}
\DeclareTextSymbol{\OE}{OT1}{30}
\DeclareTextSymbol{\O}{OT1}{31}
\DeclareTextSymbol{\ae}{OT1}{26}
\DeclareTextSymbol{\i}{OT1}{16}
\DeclareTextSymbol{\j}{OT1}{17}
\DeclareTextSymbol{\oe}{OT1}{27}
\DeclareTextSymbol{\o}{OT1}{28}
\DeclareTextSymbol{\ss}{OT1}{25}
\DeclareTextSymbol{\textemdash}{OT1}{124}
\DeclareTextSymbol{\textendash}{OT1}{123}
```

The `\nobreak\hskip\z@` is there to prevent a break after the hyphen but allow later breaks in the remainder of the word.

```
\DeclareTextCommand{\textnonbreakinghyphen}{OT1}{\mbox{-}\nobreak\hskip\z@}
\DeclareTextCommand{\textfiguredash}       {OT1}{\textendash}
\DeclareTextCommand{\texthorizontalbar}    {OT1}{\textemdash}
```

Using the ligatures helps with OT1 fonts that have `\textexclamdown` and `\textquestiondown` in unusual positions.

```
%\DeclareTextSymbol{\textexclamdown}{OT1}{60}
```

```
%\DeclareTextSymbol{\textquestiondown}{OT1}{62}
\DeclareTextCommand{\textexclamdown}{OT1}{!`}
\DeclareTextCommand{\textquestiondown}{OT1}{?`}
%\DeclareTextSymbol{\texthyphenchar}{OT1}{`\-}
%\DeclareTextSymbol{\texthyphen}{OT1}{`\-}
\DeclareTextSymbol{\textquotedblleft}{OT1}{92}
\DeclareTextSymbol{\textquotedblright}{OT1}{`\"}
\DeclareTextSymbol{\textquoteleft}{OT1}{`\`}
\DeclareTextSymbol{\textquoteright}{OT1}{`\'}
```

Some symbols which are faked from others:

```
% \DeclareTextCommand{\aa}{OT1}
%    {{\accent23a}}
\DeclareTextCommand{\L}{OT1}
   {\leavevmode\setbox\z@\hbox{L}\hb@xt@\wd\z@{\hss\@xxxii L}}
\DeclareTextCommand{\l}{OT1}
   {\hmode@bgroup\@xxxii l\egroup}
% \DeclareTextCommand{\AA}{OT1}
%    {\leavevmode\setbox\z@\hbox{h}\dimen@\ht\z@\advance\dimen@-1ex%
%     \rlap{\raise.67\dimen@\hbox{\char23}}A}
```

In the OT1 encoding Å has a hand-crafted definition, so we have here the first recorded explicit use of `\DeclareTextCompositeCommand`.

```
\DeclareTextCompositeCommand{\r}{OT1}{A}
   {\leavevmode\setbox\z@\hbox{!}\dimen@\ht\z@\advance\dimen@-1ex%
    \rlap{\raise.67\dimen@\hbox{\char23}}A}
```

The dutch language uses the letter 'ij'. It is available in `T1` encoded fonts, but not in the `OT1` encoded fonts. Therefore we fake it for the `OT1` encoding.

```
\DeclareTextCommand{\ij}{OT1}{%
  \nobreak\hskip\z@skip i\kern-0.02em\nobreak\hskip\z@skip j}
\DeclareTextCommand{\IJ}{OT1}{%
  \nobreak\hskip\z@skip I\kern-0.02em\nobreak\hskip\z@skip J}
```

In the OT1 encoding, £ and $ share a slot.

```
\DeclareTextCommand{\textdollar}{OT1}{\hmode@bgroup
   \ifdim \fontdimen\@ne\font >\z@
      \slshape
   \else
      \upshape
   \fi
   \char`\$\egroup}
\DeclareTextCommand{\textsterling}{OT1}{\hmode@bgroup
   \ifdim \fontdimen\@ne\font >\z@
      \itshape
   \else
      \fontshape{ui}\selectfont
   \fi
   \char`\$\egroup}
```

Here we are adding some more composite commands to the `OT1` encoding. This makes the use of certain accents with `i` compatible with their use with the `T1` encoding; this enables them to become true LaTeX internal representations. However, it will make these accents work a little less fast since a check will always be made for the existence of a composite.

```
\DeclareTextComposite{\.}{OT1}{i}{‘\i}
\DeclareTextComposite{\.}{OT1}{\i}{‘\i}
\DeclareTextCompositeCommand{\‘}{OT1}{i}{\@tabacckludge‘\i}
\DeclareTextCompositeCommand{\’}{OT1}{i}{\@tabacckludge’\i}
\DeclareTextCompositeCommand{\^}{OT1}{i}{\^\i}
\DeclareTextCompositeCommand{\"}{OT1}{i}{\"\i}
\DeclareTextCompositeCommand{\=}{OT1}{i}{\=\i}
```

T1 encoding is given more extensive set of overloads for `\c` But here we just adjust `\c{g}`.

```
\ifx\textcommaabove\@undefined\else
\DeclareTextCompositeCommand{\c}{OT1}{g}{\textcommaabove{g}}
\fi
```

⟨/OT1⟩

1.6 Definitions for the T1 encoding

The definitions for the ‘Extended TEX text’ (T1) encoding.

Declare the encoding.

```
⟨*T1⟩
\DeclareFontEncoding{T1}{}{}
\DeclareFontSubstitution{T1}{cmr}{m}{n}
```

Declare the accents.

```
\DeclareTextAccent{\‘}{T1}{0}
\DeclareTextAccent{\’}{T1}{1}
\DeclareTextAccent{\^}{T1}{2}
\DeclareTextAccent{\~}{T1}{3}
\DeclareTextAccent{\"}{T1}{4}
\DeclareTextAccent{\H}{T1}{5}
\DeclareTextAccent{\r}{T1}{6}
\DeclareTextAccent{\v}{T1}{7}
\DeclareTextAccent{\u}{T1}{8}
\DeclareTextAccent{\=}{T1}{9}
\DeclareTextAccent{\.}{T1}{10}
```

Some accents have to be built by hand. Note that `\ooalign` and `\o@lign` must be inside a group. In these definitions we no longer use the helper function `\sh@ft` from plain.tex since that now has two incompatible definitions.

```
\DeclareTextCommand{\b}{T1}[1]
   {\hmode@bgroup\o@lign{\relax#1\crcr\hidewidth\ltx@sh@ft{-3ex}%
     \vbox to.2ex{\hbox{\char9}\vss}\hidewidth}\egroup}
\DeclareTextCommand{\c}{T1}[1]
   {\leavevmode\setbox\z@\hbox{#1}\ifdim\ht\z@=1ex\accent11 #1%
     \else{\ooalign{\unhbox\z@\crcr
        \hidewidth\char11\hidewidth}}\fi}
\DeclareTextCommand{\d}{T1}[1]
   {\hmode@bgroup
    \o@lign{\relax#1\crcr\hidewidth\ltx@sh@ft{-1ex}.\hidewidth}\egroup}
\DeclareTextCommand{\k}{T1}[1]
   {\hmode@bgroup\ooalign{\null#1\crcr\hidewidth\char12}\egroup}
\DeclareTextCommand{\textogonekcentered}{T1}[1]
   {\hmode@bgroup\ooalign{%
                \null#1\crcr\hidewidth\char12\hidewidth}\egroup}
```

Some symbols are constructed.
Slot 24 contains a small circle intended for construction of these two glyphs.

```
\DeclareTextCommand{\textperthousand}{T1}
  {\%\char 24 }          % space or ‘relax as delimiter?
\DeclareTextCommand{\textpertenthousand}{T1}
  {\%\char 24\char 24 }  % space or ‘relax as delimiter?
```

For Maltese, `\Hwithstroke` and `\hwithstroke` are needed.

```
\DeclareTextCommand{\Hwithstroke}{T1}
  {%
   \hmode@bgroup
   \vphantom{H}%
   \sbox\z@{H}%
   \ooalign{%
     H\cr
     \hidewidth
     \vrule
       height \dimexpr 0.7\ht\z@+0.1ex\relax
       depth  -0.7\ht\z@
       width  0.8\wd\z@
     \hidewidth\cr
   }%
   \egroup
  }
\DeclareTextCommand{\hwithstroke}{T1}
  {%
   \hmode@bgroup
   \vphantom{h}%
   \sbox\z@{h}%
   \ooalign{%
     h\cr
     \kern0.075\wd\z@
     \vrule
       height \dimexpr 0.7\ht\z@+0.1ex\relax
       depth  -0.7\ht\z@
       width  0.4\wd\z@
     \hidewidth\cr
   }%
   \egroup
  }
```

Declare the text symbols.

```
%\DeclareTextSymbol{\AA}{T1}{197}
\DeclareTextSymbol{\AE}{T1}{198}
\DeclareTextSymbol{\DH}{T1}{208}
\DeclareTextSymbol{\DJ}{T1}{208}
\DeclareTextSymbol{\L}{T1}{138}
\DeclareTextSymbol{\NG}{T1}{141}
\DeclareTextSymbol{\OE}{T1}{215}
\DeclareTextSymbol{\O}{T1}{216}
\DeclareTextSymbol{\SS}{T1}{223}
\DeclareTextSymbol{\TH}{T1}{222}
%\DeclareTextSymbol{\aa}{T1}{229}
\DeclareTextSymbol{\ae}{T1}{230}
```

```
\DeclareTextSymbol{\dh}{T1}{240}
\DeclareTextSymbol{\dj}{T1}{158}

\DeclareTextSymbol{\guillemetleft}{T1}{19}
\DeclareTextSymbol{\guillemetright}{T1}{20}
% old Adobe names
\DeclareTextSymbol{\guillemotleft}{T1}{19}
\DeclareTextSymbol{\guillemotright}{T1}{20}

\DeclareTextSymbol{\guilsinglleft}{T1}{14}
\DeclareTextSymbol{\guilsinglright}{T1}{15}
\DeclareTextSymbol{\i}{T1}{25}
\DeclareTextSymbol{\j}{T1}{26}
\DeclareTextSymbol{\ij}{T1}{188}
\DeclareTextSymbol{\IJ}{T1}{156}
\DeclareTextSymbol{\l}{T1}{170}
\DeclareTextSymbol{\ng}{T1}{173}
\DeclareTextSymbol{\oe}{T1}{247}
\DeclareTextSymbol{\o}{T1}{248}
\DeclareTextSymbol{\quotedblbase}{T1}{18}
\DeclareTextSymbol{\quotesinglbase}{T1}{13}
\DeclareTextSymbol{\ss}{T1}{255}
\DeclareTextSymbol{\textasciicircum}{T1}{`\^}
\DeclareTextSymbol{\textasciitilde}{T1}{`\~}
\DeclareTextSymbol{\textbackslash}{T1}{`\\}
\DeclareTextSymbol{\textbar}{T1}{`\|}
\DeclareTextSymbol{\textbraceleft}{T1}{`\{}
\DeclareTextSymbol{\textbraceright}{T1}{`\}}
\DeclareTextSymbol{\textcompwordmark}{T1}{23}
\DeclareTextSymbol{\textdollar}{T1}{`\$}
\DeclareTextSymbol{\textemdash}{T1}{22}
\DeclareTextSymbol{\textendash}{T1}{21}
```

The `\nobreak\hskip\z@` is there to prevent a break after the hyphen but allow later breaks in the remainder of the word.

```
\DeclareTextCommand{\textnonbreakinghyphen}{T1}{\mbox{-}\nobreak\hskip\z@}
\DeclareTextCommand{\textfiguredash}       {T1}{\textendash}
\DeclareTextCommand{\texthorizontalbar}    {T1}{\textemdash}

\DeclareTextSymbol{\textexclamdown}{T1}{189}
\DeclareTextSymbol{\textgreater}{T1}{`\>}
%\DeclareTextSymbol{\texthyphenchar}{T1}{127}
%\DeclareTextSymbol{\texthyphen}{T1}{`\-}
\DeclareTextSymbol{\textless}{T1}{`\<}
\DeclareTextSymbol{\textquestiondown}{T1}{190}
\DeclareTextSymbol{\textquotedblleft}{T1}{16}
\DeclareTextSymbol{\textquotedblright}{T1}{17}
\DeclareTextSymbol{\textquotedbl}{T1}{`\"}
\DeclareTextSymbol{\textquoteleft}{T1}{`\‘}
\DeclareTextSymbol{\textquoteright}{T1}{`\’}
\DeclareTextSymbol{\textsection}{T1}{159}
\DeclareTextSymbol{\textsterling}{T1}{191}
\DeclareTextSymbol{\textunderscore}{T1}{95}
\DeclareTextSymbol{\textvisiblespace}{T1}{32}
\DeclareTextSymbol{\th}{T1}{254}
```

Declare the composites.

```
\DeclareTextComposite{\.}{T1}{i}{‘\i}
\DeclareTextComposite{\.}{T1}{\i}{‘\i}
```

"80 = 128

```
\DeclareTextComposite{\u}{T1}{A}{128}
\DeclareTextComposite{\k}{T1}{A}{129}
\DeclareTextComposite{\’}{T1}{C}{130}
\DeclareTextComposite{\v}{T1}{C}{131}
\DeclareTextComposite{\v}{T1}{D}{132}
\DeclareTextComposite{\v}{T1}{E}{133}
\DeclareTextComposite{\k}{T1}{E}{134}
\DeclareTextComposite{\u}{T1}{G}{135}
```

"88 = 136

```
\DeclareTextComposite{\’}{T1}{L}{136}
\DeclareTextComposite{\v}{T1}{L}{137}
\DeclareTextComposite{\’}{T1}{N}{139}
\DeclareTextComposite{\v}{T1}{N}{140}
\DeclareTextComposite{\H}{T1}{O}{142}
\DeclareTextComposite{\’}{T1}{R}{143}
```

"90 = 144

```
\DeclareTextComposite{\v}{T1}{R}{144}
\DeclareTextComposite{\’}{T1}{S}{145}
\DeclareTextComposite{\v}{T1}{S}{146}
\DeclareTextComposite{\c}{T1}{S}{147}
\DeclareTextComposite{\v}{T1}{T}{148}
\DeclareTextComposite{\c}{T1}{T}{149}
\DeclareTextComposite{\H}{T1}{U}{150}
\DeclareTextComposite{\r}{T1}{U}{151}
```

"98 = 152

```
\DeclareTextComposite{\"}{T1}{Y}{152}
\DeclareTextComposite{\’}{T1}{Z}{153}
\DeclareTextComposite{\v}{T1}{Z}{154}
\DeclareTextComposite{\.}{T1}{Z}{155}
\DeclareTextComposite{\.}{T1}{I}{157}
```

"A0 = 160

```
\DeclareTextComposite{\u}{T1}{a}{160}
\DeclareTextComposite{\k}{T1}{a}{161}
\DeclareTextComposite{\’}{T1}{c}{162}
\DeclareTextComposite{\v}{T1}{c}{163}
\DeclareTextComposite{\v}{T1}{d}{164}
\DeclareTextComposite{\v}{T1}{e}{165}
\DeclareTextComposite{\k}{T1}{e}{166}
\DeclareTextComposite{\u}{T1}{g}{167}
```

"A8 = 168

```
\DeclareTextComposite{\’}{T1}{l}{168}
\DeclareTextComposite{\v}{T1}{l}{169}
\DeclareTextComposite{\’}{T1}{n}{171}
\DeclareTextComposite{\v}{T1}{n}{172}
\DeclareTextComposite{\H}{T1}{o}{174}
\DeclareTextComposite{\’}{T1}{r}{175}
```

"B0 = 176

```
\DeclareTextComposite{\v}{T1}{r}{176}
\DeclareTextComposite{\'}{T1}{s}{177}
\DeclareTextComposite{\v}{T1}{s}{178}
\DeclareTextComposite{\c}{T1}{s}{179}
\DeclareTextComposite{\v}{T1}{t}{180}
\DeclareTextComposite{\c}{T1}{t}{181}
\DeclareTextComposite{\H}{T1}{u}{182}
\DeclareTextComposite{\r}{T1}{u}{183}
```

"B8 = 184

```
\DeclareTextComposite{\"}{T1}{y}{184}
\DeclareTextComposite{\'}{T1}{z}{185}
\DeclareTextComposite{\v}{T1}{z}{186}
\DeclareTextComposite{\.}{T1}{z}{187}
```

"C0 = 192

```
\DeclareTextComposite{\`}{T1}{A}{192}
\DeclareTextComposite{\'}{T1}{A}{193}
\DeclareTextComposite{\^}{T1}{A}{194}
\DeclareTextComposite{\~}{T1}{A}{195}
\DeclareTextComposite{\"}{T1}{A}{196}
\DeclareTextComposite{\r}{T1}{A}{197}
\DeclareTextComposite{\c}{T1}{C}{199}
```

"C8 = 200

```
\DeclareTextComposite{\`}{T1}{E}{200}
\DeclareTextComposite{\'}{T1}{E}{201}
\DeclareTextComposite{\^}{T1}{E}{202}
\DeclareTextComposite{\"}{T1}{E}{203}
\DeclareTextComposite{\`}{T1}{I}{204}
\DeclareTextComposite{\'}{T1}{I}{205}
\DeclareTextComposite{\^}{T1}{I}{206}
\DeclareTextComposite{\"}{T1}{I}{207}
```

"D0 = 208

```
\DeclareTextComposite{\~}{T1}{N}{209}
\DeclareTextComposite{\`}{T1}{O}{210}
\DeclareTextComposite{\'}{T1}{O}{211}
\DeclareTextComposite{\^}{T1}{O}{212}
\DeclareTextComposite{\~}{T1}{O}{213}
\DeclareTextComposite{\"}{T1}{O}{214}
```

"D8 = 216

```
\DeclareTextComposite{\`}{T1}{U}{217}
\DeclareTextComposite{\'}{T1}{U}{218}
\DeclareTextComposite{\^}{T1}{U}{219}
\DeclareTextComposite{\"}{T1}{U}{220}
\DeclareTextComposite{\'}{T1}{Y}{221}
```

"E0 = 224

```
\DeclareTextComposite{\`}{T1}{a}{224}
\DeclareTextComposite{\'}{T1}{a}{225}
\DeclareTextComposite{\^}{T1}{a}{226}
\DeclareTextComposite{\~}{T1}{a}{227}
\DeclareTextComposite{\"}{T1}{a}{228}
```

```
\DeclareTextComposite{\r}{T1}{a}{229}
\DeclareTextComposite{\c}{T1}{c}{231}
```

"E8 = 232

```
\DeclareTextComposite{\‘}{T1}{e}{232}
\DeclareTextComposite{\’}{T1}{e}{233}
\DeclareTextComposite{\^}{T1}{e}{234}
\DeclareTextComposite{\"}{T1}{e}{235}
\DeclareTextComposite{\‘}{T1}{i}{236}
\DeclareTextComposite{\‘}{T1}{\i}{236}
\DeclareTextComposite{\’}{T1}{i}{237}
\DeclareTextComposite{\’}{T1}{\i}{237}
\DeclareTextComposite{\^}{T1}{i}{238}
\DeclareTextComposite{\^}{T1}{\i}{238}
\DeclareTextComposite{\"}{T1}{i}{239}
\DeclareTextComposite{\"}{T1}{\i}{239}
```

"F0 = 240

```
\DeclareTextComposite{\~}{T1}{n}{241}
\DeclareTextComposite{\‘}{T1}{o}{242}
\DeclareTextComposite{\’}{T1}{o}{243}
\DeclareTextComposite{\^}{T1}{o}{244}
\DeclareTextComposite{\~}{T1}{o}{245}
\DeclareTextComposite{\"}{T1}{o}{246}
```

"F8 = 248

```
\DeclareTextComposite{\‘}{T1}{u}{249}
\DeclareTextComposite{\’}{T1}{u}{250}
\DeclareTextComposite{\^}{T1}{u}{251}
\DeclareTextComposite{\"}{T1}{u}{252}
\DeclareTextComposite{\’}{T1}{y}{253}

\DeclareTextCompositeCommand{\k}{T1}{o}{\textogonekcentered{o}}
\DeclareTextCompositeCommand{\k}{T1}{O}{\textogonekcentered{O}}

\ifx\textcommaabove\@undefined\else
\DeclareTextCompositeCommand{\c}{T1}{g}{\textcommaabove{g}}
\fi
\ifx\textcommabelow\@undefined\else
\DeclareTextCompositeCommand{\c}{T1}{G}{\textcommabelow{G}}
\DeclareTextCompositeCommand{\c}{T1}{K}{\textcommabelow{K}}
\DeclareTextCompositeCommand{\c}{T1}{k}{\textcommabelow{k}}
\DeclareTextCompositeCommand{\c}{T1}{L}{\textcommabelow{L}}
\DeclareTextCompositeCommand{\c}{T1}{l}{\textcommabelow{l}}
\DeclareTextCompositeCommand{\c}{T1}{N}{\textcommabelow{N}}
\DeclareTextCompositeCommand{\c}{T1}{n}{\textcommabelow{n}}
\DeclareTextCompositeCommand{\c}{T1}{R}{\textcommabelow{R}}
\DeclareTextCompositeCommand{\c}{T1}{r}{\textcommabelow{r}}
\fi
```

One oddity.

```
\DeclareTextCompositeCommand{\=}{T1}{i}{\=\i}
```

⟨/T1⟩

1.7 Definitions for the OMS encoding

The definitions for the 'TEX math symbol' (OMS) encoding. Even though this is meant to be a math font, it includes some of the standard LATEX text symbols.

Declare the encoding.

```
⟨*OMS⟩
\DeclareFontEncoding{OMS}{}{}
```

Declare the symbols. Note that slot 13 has in places been named `\Orb`: please root out and destroy this impolity wherever you find it!

```
\DeclareTextSymbol{\textasteriskcentered}{OMS}{3}   % "03
\DeclareTextSymbol{\textbackslash}{OMS}{110}        % "6E
\DeclareTextSymbol{\textbar}{OMS}{106}              % "6A
\DeclareTextSymbol{\textbardbl}{OMS}{107}           % "6B
\DeclareTextSymbol{\textbraceleft}{OMS}{102}        % "66
\DeclareTextSymbol{\textbraceright}{OMS}{103}       % "67
\DeclareTextSymbol{\textbullet}{OMS}{15}            % "0F
\DeclareTextSymbol{\textdaggerdbl}{OMS}{122}        % "7A
\DeclareTextSymbol{\textdagger}{OMS}{121}           % "79
\DeclareTextSymbol{\textparagraph}{OMS}{123}        % "7B
\DeclareTextSymbol{\textperiodcentered}{OMS}{1}     % "01
\DeclareTextSymbol{\textsection}{OMS}{120}          % "78
\DeclareTextSymbol{\textbigcircle}{OMS}{13}         % "0D
\DeclareTextCommand{\textcircled}{OMS}[1]{\hmode@bgroup
   \ooalign{%
      \hfil \raise .07ex\hbox {\upshape#1}\hfil \crcr
      \char 13 % "0D
   }%
 \egroup}
⟨/OMS⟩
```

1.8 Definitions for the OML encoding

The definitions for the 'TEX math italic' (OML) encoding. Even though this is meant to be a math font, it includes some of the standard LATEX text symbols.

Declare the encoding.

```
⟨*OML⟩
\DeclareFontEncoding{OML}{}{}
```

Declare the symbols.

```
\DeclareTextSymbol{\textless}{OML}{'\<}
\DeclareTextSymbol{\textgreater}{OML}{'\>}
\DeclareTextAccent{\t}{OML}{127}  % "7F
⟨/OML⟩
```

1.9 Definitions for the OT4 encoding

These definitions are for the Polish extension to the 'TEX text' (OT1) encoding. This encoding was created by B. Jackowski and M. Ryćko for use with the Polish version of Computer Modern and Computer Concrete. In positions 0–127 it is identical to OT1 but it contains some additional characters in the upper half. The LATEX support was developed by Mariusz Olko.

The PL fonts that use it are available as follows:

Metafont sources `ftp://ftp.gust.org.pl/TeX/language/polish/pl-mf.zip`;

Font files `ftp://ftp.gust.org.pl/TeX/language/polish/pl-tfm.zip`.

Declare the encoding.

```
⟨*OT4⟩
\DeclareFontEncoding{OT4}{}{}
\DeclareFontSubstitution{OT4}{cmr}{m}{n}
```

Declare the accents.

```
\DeclareTextAccent{\"}{OT4}{127}
\DeclareTextAccent{\'}{OT4}{19}
\DeclareTextAccent{\.}{OT4}{95}
\DeclareTextAccent{\=}{OT4}{22}
\DeclareTextAccent{\^}{OT4}{94}
\DeclareTextAccent{\`}{OT4}{18}
\DeclareTextAccent{\~}{OT4}{126}
\DeclareTextAccent{\H}{OT4}{125}
\DeclareTextAccent{\u}{OT4}{21}
\DeclareTextAccent{\v}{OT4}{20}
\DeclareTextAccent{\r}{OT4}{23}
```

The ogonek accent is available only under a e A & E. But we have to provide some definition for `\k`. Some other accents have to be built by hand as in OT1:

```
\DeclareTextCommand{\k}{OT4}[1]{%
    \TextSymbolUnavailable{\k{#1}}#1}
```

In these definitions we no longer use the helper function `\sh@ft` from plain.tex since that now has two incompatible definitions.

```
\DeclareTextCommand{\b}{OT4}[1]
   {\hmode@bgroup\o@lign{\relax#1\crcr\hidewidth\ltx@sh@ft{-3ex}%
     \vbox to.2ex{\hbox{\char22}\vss}\hidewidth}\egroup}
\DeclareTextCommand{\c}{OT4}[1]
   {\leavevmode\setbox\z@\hbox{#1}\ifdim\ht\z@=1ex\accent24 #1%
    \else{\ooalign{\unhbox\z@\crcr\hidewidth\char24\hidewidth}}\fi}
\DeclareTextCommand{\d}{OT4}[1]
   {\hmode@bgroup
    \o@lign{\relax#1\crcr\hidewidth\ltx@sh@ft{-1ex}.\hidewidth}\egroup}
```

Declare the text symbols.

```
\DeclareTextSymbol{\AE}{OT4}{29}
\DeclareTextSymbol{\OE}{OT4}{30}
\DeclareTextSymbol{\O}{OT4}{31}
\DeclareTextSymbol{\L}{OT4}{138}
\DeclareTextSymbol{\ae}{OT4}{26}

\DeclareTextSymbol{\guillemetleft}{OT4}{174}
\DeclareTextSymbol{\guillemetright}{OT4}{175}
% old Adobe names
\DeclareTextSymbol{\guillemotleft}{OT4}{174}
\DeclareTextSymbol{\guillemotright}{OT4}{175}

\DeclareTextSymbol{\i}{OT4}{16}
\DeclareTextSymbol{\j}{OT4}{17}
\DeclareTextSymbol{\l}{OT4}{170}
\DeclareTextSymbol{\o}{OT4}{28}
\DeclareTextSymbol{\oe}{OT4}{27}
```

```
\DeclareTextSymbol{\quotedblbase}{OT4}{255}
\DeclareTextSymbol{\ss}{OT4}{25}
\DeclareTextSymbol{\textemdash}{OT4}{124}
\DeclareTextSymbol{\textendash}{OT4}{123}
\DeclareTextSymbol{\textexclamdown}{OT4}{60}
%\DeclareTextSymbol{\texthyphenchar}{OT4}{`\-}
%\DeclareTextSymbol{\texthyphen}{OT4}{`\-}
\DeclareTextSymbol{\textquestiondown}{OT4}{62}
\DeclareTextSymbol{\textquotedblleft}{OT4}{92}
\DeclareTextSymbol{\textquotedblright}{OT4}{`\"}
\DeclareTextSymbol{\textquoteleft}{OT4}{`\`}
\DeclareTextSymbol{\textquoteright}{OT4}{`\'}
```

Definition for Å as in OT1:

```
\DeclareTextCompositeCommand{\r}{OT4}{A}
   {\leavevmode\setbox\z@\hbox{!}\dimen@\ht\z@\advance\dimen@-1ex%
    \rlap{\raise.67\dimen@\hbox{\char23}}A}
```

In the OT4 encoding, £ and $ share a slot.

```
\DeclareTextCommand{\textdollar}{OT4}{\hmode@bgroup
   \ifdim \fontdimen\@ne\font >\z@
      \slshape
   \else
      \upshape
   \fi
   \char`\$\egroup}
\DeclareTextCommand{\textsterling}{OT4}{\hmode@bgroup
   \ifdim \fontdimen\@ne\font >\z@
      \itshape
   \else
      \fontshape{ui}\selectfont
   \fi
   \char`\$\egroup}
```

Declare the composites.

```
\DeclareTextComposite{\k}{OT4}{A}{129}
\DeclareTextComposite{\'}{OT4}{C}{130}
\DeclareTextComposite{\k}{OT4}{E}{134}
\DeclareTextComposite{\'}{OT4}{N}{139}
\DeclareTextComposite{\'}{OT4}{S}{145}
\DeclareTextComposite{\'}{OT4}{Z}{153}
\DeclareTextComposite{\.}{OT4}{Z}{155}
\DeclareTextComposite{\k}{OT4}{a}{161}
\DeclareTextComposite{\'}{OT4}{c}{162}
\DeclareTextComposite{\k}{OT4}{e}{166}
\DeclareTextComposite{\'}{OT4}{n}{171}
\DeclareTextComposite{\'}{OT4}{s}{177}
\DeclareTextComposite{\'}{OT4}{z}{185}
\DeclareTextComposite{\.}{OT4}{z}{187}
\DeclareTextComposite{\'}{OT4}{O}{211}
\DeclareTextComposite{\'}{OT4}{o}{243}
```

⟨/OT4⟩

1.10 Definitions for the TS1 encoding

```
⟨*TS1⟩
\DeclareFontEncoding{TS1}{}{}
\DeclareFontSubstitution{TS1}{cmr}{m}{n}
```

Some accents have to be built by hand. Note that `\ooalign` and `\o@lign` must be inside a group.

```
\DeclareTextCommand{\capitalcedilla}{TS1}[1]
   {\hmode@bgroup
    \ooalign{\null#1\crcr\hidewidth\char11\hidewidth}\egroup}
\DeclareTextCommand{\capitalogonek}{TS1}[1]
   {\hmode@bgroup
    \ooalign{\null#1\crcr\hidewidth\char12\hidewidth}\egroup}
```

Accents for capital letters.

These commands can be used by the end user either directly or through definitions of the type
`\DeclareTextCompositeCommand{\'}{T1}{X}{\capitalacute X}`

None of the latter definitions are provided by default, since they are probably rarely used.

"00 = 0

```
\DeclareTextAccent{\capitalgrave}{TS1}{0}
\DeclareTextAccent{\capitalacute}{TS1}{1}
\DeclareTextAccent{\capitalcircumflex}{TS1}{2}
\DeclareTextAccent{\capitaltilde}{TS1}{3}
\DeclareTextAccent{\capitaldieresis}{TS1}{4}
\DeclareTextAccent{\capitalhungarumlaut}{TS1}{5}
\DeclareTextAccent{\capitalring}{TS1}{6}
\DeclareTextAccent{\capitalcaron}{TS1}{7}
```

"08 = 8

```
\DeclareTextAccent{\capitalbreve}{TS1}{8}
\DeclareTextAccent{\capitalmacron}{TS1}{9}
\DeclareTextAccent{\capitaldotaccent}{TS1}{10}
```

Tie accents.

The tie accent was borrowed from the `cmmi` font. The tc fonts now provide four tie accents, the first two are done in the classical way with asymmetric glyphs hanging out of their boxes; the new ties are centered in their boxes like all other accents. They need a name: please tell us if you know what to call them.

" =

```
\DeclareTextAccent{\t}{TS1}{26}
\DeclareTextAccent{\capitaltie}{TS1}{27}
\DeclareTextAccent{\newtie}{TS1}{28}
\DeclareTextAccent{\capitalnewtie}{TS1}{29}
```

Compound word marks.

The text companion fonts contain two compound word marks of different heights, one has `cap_height`, the other `asc_height`.

```
\DeclareTextSymbol{\textcapitalcompwordmark}{TS1}{23}
\DeclareTextSymbol{\textascendercompwordmark}{TS1}{31}
```

The text companion symbols.

```
\DeclareTextSymbol{\textquotestraightbase}{TS1}{13}
```

"10 = 16

```
\DeclareTextSymbol{\textquotestraightdblbase}{TS1}{18}
\DeclareTextSymbol{\texttwelveudash}{TS1}{21}
\DeclareTextSymbol{\textthreequartersemdash}{TS1}{22}
```

"18 = 24

```
\DeclareTextSymbol{\textleftarrow}{TS1}{24}
\DeclareTextSymbol{\textrightarrow}{TS1}{25}
```

"20 = 32

```
\DeclareTextSymbol{\textblank}{TS1}{32}
\DeclareTextSymbol{\textdollar}{TS1}{36}
\DeclareTextSymbol{\textquotesingle}{TS1}{39}
```

"28 = 40

The symbol `\textasteriskcentered` "∗" is supposed to be always available in `TS1` and that is important as it is used in footnote symbols. However, in a few fonts it is missing even though they are otherwise fairly complete. We therefore use a rather elaborate method and check if the slot has a glyph and if not produce a poor man's version by using a normal "*" slightly enlarged and somewhat lowered. The main application for this symbol is in footnote symbols and there it should produce a comparable size and show a similar placement.

```
%\DeclareTextSymbol{\textasteriskcentered}{TS1}{42} % that's wanted
\DeclareTextCommand \textasteriskcentered{TS1}{%    % and that's needed
  \iffontchar\font 42 \char42 \else
   \begingroup\fontencoding{T1}%
       \fontsize
       {\the\dimexpr1.3\dimexpr\f@size pt\relax}%
       {\f@baselineskip}%
      \selectfont
      \raisebox{-0.7ex}[\dimexpr\height-0.7ex][0pt]{*}%
   \endgroup
   \fi
}
```

Note that '054 is a comma and '056 is a full stop: these make numbers using oldstyle digits easier to input.

```
\DeclareTextSymbol{\textdblhyphen}{TS1}{45}
\DeclareTextSymbol{\textfractionsolidus}{TS1}{47}
```

Oldstyle digits.

"30 = 48

```
\DeclareTextSymbol{\textzerooldstyle}{TS1}{48}
\DeclareTextSymbol{\textoneoldstyle}{TS1}{49}
\DeclareTextSymbol{\texttwooldstyle}{TS1}{50}
\DeclareTextSymbol{\textthreeoldstyle}{TS1}{51}
\DeclareTextSymbol{\textfouroldstyle}{TS1}{52}
\DeclareTextSymbol{\textfiveoldstyle}{TS1}{53}
\DeclareTextSymbol{\textsixoldstyle}{TS1}{54}
\DeclareTextSymbol{\textsevenoldstyle}{TS1}{55}
```

"38 = 56

```
\DeclareTextSymbol{\texteightoldstyle}{TS1}{56}
\DeclareTextSymbol{\textnineoldstyle}{TS1}{57}
```

More text companion symbols.

```
\DeclareTextSymbol{\textlangle}{TS1}{60}
\DeclareTextSymbol{\textminus}{TS1}{61}
\DeclareTextSymbol{\textrangle}{TS1}{62}
```

"48 = 72

```
\DeclareTextSymbol{\textmho}{TS1}{77}
```

The big circle is here to define the command `\textcircled`. Formerly it was taken from the `cmsy` font.

```
\DeclareTextSymbol{\textbigcircle}{TS1}{79}
\DeclareTextCommand{\textcircled}{TS1}[1]{\hmode@bgroup
   \ooalign{%
      \hfil \raise .07ex\hbox {\upshape#1}\hfil \crcr
      \char 79   % '117 = "4F
   }%
 \egroup}
```

More text companion symbols.

"50 = 80

```
\DeclareTextSymbol{\textohm}{TS1}{87}
```

"58 = 88

```
\DeclareTextSymbol{\textlbrackdbl}{TS1}{91}
\DeclareTextSymbol{\textrbrackdbl}{TS1}{93}
\DeclareTextSymbol{\textuparrow}{TS1}{94}
\DeclareTextSymbol{\textdownarrow}{TS1}{95}
```

"60 = 96

```
\DeclareTextSymbol{\textasciigrave}{TS1}{96}
\DeclareTextSymbol{\textborn}{TS1}{98}
\DeclareTextSymbol{\textdivorced}{TS1}{99}
\DeclareTextSymbol{\textdied}{TS1}{100}
```

"68 = 104

```
\DeclareTextSymbol{\textleaf}{TS1}{108}
\DeclareTextSymbol{\textmarried}{TS1}{109}
\DeclareTextSymbol{\textmusicalnote}{TS1}{110}
```

"78 = 120

```
\DeclareTextSymbol{\texttildelow}{TS1}{126}
```

This glyph, `\textdblhyphenchar` is hanging, like the hyphenchar of the ec fonts.

```
\DeclareTextSymbol{\textdblhyphenchar}{TS1}{127}
```

"80 = 128

```
\DeclareTextSymbol{\textasciibreve}{TS1}{128}
\DeclareTextSymbol{\textasciicaron}{TS1}{129}
```

This next glyph is *not* the same as `\textquotedbl`.

```
\DeclareTextSymbol{\textacutedbl}{TS1}{130}
\DeclareTextSymbol{\textgravedbl}{TS1}{131}
\DeclareTextSymbol{\textdagger}{TS1}{132}
\DeclareTextSymbol{\textdaggerdbl}{TS1}{133}
\DeclareTextSymbol{\textbardbl}{TS1}{134}
\DeclareTextSymbol{\textperthousand}{TS1}{135}
```

"88 = 136

```
\DeclareTextSymbol{\textbullet}{TS1}{136}
\DeclareTextSymbol{\textcelsius}{TS1}{137}
\DeclareTextSymbol{\textdollaroldstyle}{TS1}{138}
\DeclareTextSymbol{\textcentoldstyle}{TS1}{139}
\DeclareTextSymbol{\textflorin}{TS1}{140}
\DeclareTextSymbol{\textcolonmonetary}{TS1}{141}
\DeclareTextSymbol{\textwon}{TS1}{142}
\DeclareTextSymbol{\textnaira}{TS1}{143}
```

"90 = 144

```
\DeclareTextSymbol{\textguarani}{TS1}{144}
\DeclareTextSymbol{\textpeso}{TS1}{145}
\DeclareTextSymbol{\textlira}{TS1}{146}
\DeclareTextSymbol{\textrecipe}{TS1}{147}
\DeclareTextSymbol{\textinterrobang}{TS1}{148}
\DeclareTextSymbol{\textinterrobangdown}{TS1}{149}
\DeclareTextSymbol{\textdong}{TS1}{150}
\DeclareTextSymbol{\texttrademark}{TS1}{151}
```

"98 = 152

```
\DeclareTextSymbol{\textpertenthousand}{TS1}{152}
\DeclareTextSymbol{\textpilcrow}{TS1}{153}
\DeclareTextSymbol{\textbaht}{TS1}{154}
\DeclareTextSymbol{\textnumero}{TS1}{155}
```

This next name may change. For the following sign we know only a german name, which is abzüglich. The meaning is something like "commercial minus". An ASCII ersatz is ./. (dot slash dot). The temporary English name is `\textdiscount`.

```
\DeclareTextSymbol{\textdiscount}{TS1}{156}
\DeclareTextSymbol{\textestimated}{TS1}{157}
\DeclareTextSymbol{\textopenbullet}{TS1}{158}
\DeclareTextSymbol{\textservicemark}{TS1}{159}
```

"A0 = 160

```
\DeclareTextSymbol{\textlquill}{TS1}{160}
\DeclareTextSymbol{\textrquill}{TS1}{161}
\DeclareTextSymbol{\textcent}{TS1}{162}
\DeclareTextSymbol{\textsterling}{TS1}{163}
\DeclareTextSymbol{\textcurrency}{TS1}{164}
\DeclareTextSymbol{\textyen}{TS1}{165}
\DeclareTextSymbol{\textbrokenbar}{TS1}{166}
\DeclareTextSymbol{\textsection}{TS1}{167}
```

"A8 = 168

```
\DeclareTextSymbol{\textasciidieresis}{TS1}{168}
\DeclareTextSymbol{\textcopyright}{TS1}{169}
\DeclareTextSymbol{\textordfeminine}{TS1}{170}
\DeclareTextSymbol{\textcopyleft}{TS1}{171}
\DeclareTextSymbol{\textlnot}{TS1}{172}
```

The meaning of the circled-P is "sound recording copyright".

```
\DeclareTextSymbol{\textcircledP}{TS1}{173}
\DeclareTextSymbol{\textregistered}{TS1}{174}
\DeclareTextSymbol{\textasciimacron}{TS1}{175}
```

"B0 = 176

```
\DeclareTextSymbol{\textdegree}{TS1}{176}
\DeclareTextSymbol{\textpm}{TS1}{177}
\DeclareTextSymbol{\texttwosuperior}{TS1}{178}
\DeclareTextSymbol{\textthreesuperior}{TS1}{179}
\DeclareTextSymbol{\textasciiacute}{TS1}{180}
\DeclareTextSymbol{\textmu}{TS1}{181} % micro sign
\DeclareTextSymbol{\textparagraph}{TS1}{182}
\DeclareTextSymbol{\textperiodcentered}{TS1}{183}
```

"B8 = 184

```
\DeclareTextSymbol{\textreferencemark}{TS1}{184}
\DeclareTextSymbol{\textonesuperior}{TS1}{185}
\DeclareTextSymbol{\textordmasculine}{TS1}{186}
\DeclareTextSymbol{\textsurd}{TS1}{187}
\DeclareTextSymbol{\textonequarter}{TS1}{188}
\DeclareTextSymbol{\textonehalf}{TS1}{189}
\DeclareTextSymbol{\textthreequarters}{TS1}{190}
\DeclareTextSymbol{\texteuro}{TS1}{191}
```

"E0 = 208

```
\DeclareTextSymbol{\texttimes}{TS1}{214}
```

"F0 = 240

```
\DeclareTextSymbol{\textdiv}{TS1}{246}
```

⟨/TS1⟩

1.11 Definitions for the TU encoding

The TU encoding was originally introduced in the contributed package `fontspec` as a Unicode encoding for XeTeX and LuaTeX.

Normally for these engines, the input consists of Unicode characters encoded in UTF-8. There is therefore little need to use the traditional (ASCII) encoding-specific commands

However, sometimes (e.g. for backwards compatibility) it can be useful to access these Unicode characters via such ASCII-based markup. The commands provided here Cover the characters in the T1 and TS1 encodings, but specified in Unicode position. Almost all the command names have been mechanically extracted form the `inputenc` UTF-8 support, which is essentially doing a reverse mapping from UTF-8 data to LaTeX LICR commands.

A few additional names for character which were supported in the original `fontspec` version of this file have also been added, even though they are not currently in the default `inputenc` UTF-8 declarations.

⟨*TU⟩

In the base interface the Unicode encoding is always known as TU But we parameterize the encoding name to allow for modelling differences in Unicode support by different fonts.

```
\providecommand\UnicodeEncodingName{TU}
```

TU is only currently available with XeTeX, HiTeX or LuaTeX, we detect these engines first, and make adjustments for the differing font loading syntax. For other engines, we issue a warning then abort this file, switching back to T1 encoding. Use `\Umathcode` as the marker for TU support (as in `fonttext.ltx`). Here we actually test for `\tex_Umathcode:D` as `\Umathcode` is redefined locally in the LaTeX test suite.

```
\expandafter\ifx\csname tex_Umathcode:D\endcsname\relax
  \expandafter\let\csname tex_Umathcode:D\endcsname\@undefined
```

Not HiTeX, LuaTeX or XeTeX, abort with a warning.

```
  \PackageWarningNoLine{fontenc}
    {\UnicodeEncodingName\space
      encoding is only available with\MessageBreak
      HiTeX, XeTeX and LuaTeX.\MessageBreak
     Defaulting to T1 encoding}
    \def\encodingdefault{T1}
  \expandafter\endinput
\fi
```

LuaTeX.

```
\ifx\directlua\@undefined\else
```

For LuaTeX 1.10+, define a Lua function to disable any handing by the font code. Otherwise we reload the font without TeX ligatures.

```
  \def\UnicodeFontTeXLigatures{+tlig;}
  \ifnum\luatexversion<110
    \def\reserved@a#1{%
      \def\@remove@tlig##1{\@remove@tlig@##1\@nil#1\@nil\relax}
      \def\@remove@tlig@##1#1{\@remove@tlig@@##1}}
    \edef\reserved@b{\detokenize{+tlig;}}
    \expandafter\reserved@a\expandafter{\reserved@b}
    \def\@remove@tlig@@#1\@nil#2\relax{#1}
    \def\remove@tlig#1{%
      \begingroup
      \font\remove@tlig
      \expandafter\@remove@tlig\expandafter{\fontname\font}%
      \remove@tlig
      \char#1\relax
      \endgroup
    }
  \else
    \newprotectedluacmd\@remove@tlig@@@
```

Now we can define the function. Mostly we just have to insert a protected glyph node, which is a glyph node with subtype 256. But we have to keep track of the current mode to avoid inserting the glyph into a vlist.

```
    \now@and@everyjob{\directlua{
      local rawchar_func = token.create'@remove@tlig@@@'.index
      local forcehmode = tex.forcehmode
      local put_next = token.put_next
      local glyph_id = node.id'glyph'
      local rawchar_token = token.new(rawchar_func, token.command_id'lua_call')
      lua.get_functions_table()[rawchar_func] = function()
        local mode = tex.nest.top.mode
```

```
          if mode == 1 or mode == -1 then
            put_next(rawchar_token)
            return forcehmode(true)
          end
          local n = node.new(glyph_id, 256)
          n.font = font.current()
          n.char = token.scan_int()
          return node.write(n)
        end
      }}
```

Now `\remove@tlig` can be implemented almost as in XeTeX.

```
      \def\remove@tlig#1{\@remove@tlig@@@#1\relax}
    \fi
\fi
```

XeTeX

```
\ifx\XeTeXversion\@undefined\else
  \def\UnicodeFontTeXLigatures{mapping=tex-text;}
  \def\remove@tlig#1{\XeTeXglyph\numexpr\XeTeXcharglyph#1\relax}
\fi
```

HiTeX

```
\ifx\HiTeXversion\@undefined\else
  \def\UnicodeFontTeXLigatures{+tlig;}
  \def\remove@tlig#1{\mbox{\Uchar#1}}
\fi
\def\UnicodeFontFile#1#2{"[#1]:#2"}
\def\UnicodeFontName#1#2{"#1:#2"}
```

Declare the encoding

```
\DeclareFontEncoding\UnicodeEncodingName{}{}
```

Declare accent command to use a postpended combining character rather than the TeX `\accent` primitive

```
\def\add@unicode@accent#1#2{%
  \if\relax\detokenize{#2}\relax^^a0\else#2\fi
  \char#1\relax}
```

In its original implementation `\DeclareUnicodeAccent` was given 3 arguments (with second the "Unicode encoding" a.k.a., `\UnicodeEncodingName`) while in other places, e.g., `\DeclareUnicodeComposite`, we always made encoding implicit. So we now change it here to implicit too so that the interfaces become a bit more consistent. To avoid making that a breaking change (even though it only affects two packages on CTAN) we test for `#2` being `\UnicodeEncodingName`. This would not catch if somebody used `\DeclareUnicodeAccent{\=}{TU-sub}{"0304}` but that fortunately hasn't happened. With the implicit argument you would need to change `\UnicodeEncodingName` instead, as you have to do anyway for the other interface commands.

```
\def\DeclareUnicodeAccent#1#2{%
  \edef\reserved@a{#2}%
  \edef\reserved@b{\UnicodeEncodingName}%
  \ifx\reserved@a\reserved@b
    \def\reserved@a{\DeclareUnicodeAccent@{#1}}%
```

```
  \else
    \def\reserved@a{\DeclareUnicodeAccent@{#1}\UnicodeEncodingName}%
  \fi
  \reserved@a{#2}%
}
\def\DeclareUnicodeAccent@#1#2#3{%
  \DeclareTextCommand{#1}{#2}{\add@unicode@accent{#3}}%
}
```

Wrapper around `\DeclareTextCompositeCommand` that uses the declared composite if it exists in the current font or falls back to the default definition for the TU accent if not.

```
{
\catcode\z@=11\relax
\gdef\DeclareUnicodeComposite#1#2#3{%
   \def\reserved@a##1##2{%
     \DeclareTextCompositeCommand#1\UnicodeEncodingName{#2}{%
   \iffontchar\font#3 ##2%
      \else ##1\fi}}%
    \expandafter\expandafter\expandafter\extract@default@composite
    \csname\UnicodeEncodingName\string#1\endcsname{#2}\@nil
   \bgroup
      \lccode\z@#3 %
      \lowercase{\egroup
      \expandafter\reserved@a\expandafter{\reserved@b}{^^@}}}%
}

\def\extract@default@composite#1{%
 \ifx\@text@composite#1%
   \expandafter\extract@default@composite@a
 \else
   \expandafter\extract@default@composite@b\expandafter#1%
 \fi}

\def\extract@default@composite@a#1\@text@composite#2\@nil{%
  \def\reserved@b{#2}}
\def\extract@default@composite@b#1#2\@nil{%
  \def\reserved@b{#1#2}}
```

Next two commands are simply syntactic sugar to go with the other `\DeclareUnicode...` declarations.

```
\def\DeclareUnicodeSymbol#1{\DeclareTextSymbol{#1}{\UnicodeEncodingName}}
\def\DeclareUnicodeCommand#1{\DeclareTextCommand{#1}{\UnicodeEncodingName}}

\DeclareUnicodeCommand\textquotesingle {\remove@tlig{"0027}}
\DeclareUnicodeCommand\textasciigrave  {\remove@tlig{"0060}}
\DeclareUnicodeCommand\textquotedbl    {\remove@tlig{"0022}}

\DeclareUnicodeSymbol{\textdollar}          {"0024}
\DeclareUnicodeSymbol{\textless}            {"003C}
\DeclareUnicodeSymbol{\textgreater}         {"003E}
\DeclareUnicodeSymbol{\textbackslash}       {"005C}
\DeclareUnicodeSymbol{\textasciicircum}     {"005E}
\DeclareUnicodeSymbol{\textunderscore}      {"005F}
\DeclareUnicodeSymbol{\textbraceleft}       {"007B}
\DeclareUnicodeSymbol{\textbar}             {"007C}
\DeclareUnicodeSymbol{\textbraceright}      {"007D}
```

```
\DeclareUnicodeSymbol{\textasciitilde}      {"007E}
\DeclareUnicodeSymbol{\textexclamdown}      {"00A1}
\DeclareUnicodeSymbol{\textcent}            {"00A2}
\DeclareUnicodeSymbol{\textsterling}        {"00A3}
\DeclareUnicodeSymbol{\textcurrency}        {"00A4}
\DeclareUnicodeSymbol{\textyen}             {"00A5}
\DeclareUnicodeSymbol{\textbrokenbar}       {"00A6}
\DeclareUnicodeSymbol{\textsection}         {"00A7}
\DeclareUnicodeSymbol{\textasciidieresis}   {"00A8}
\DeclareUnicodeSymbol{\textcopyright}       {"00A9}
\DeclareUnicodeSymbol{\textordfeminine}     {"00AA}

\DeclareUnicodeSymbol{\guillemetleft}       {"00AB}
% old Adobe name
\DeclareUnicodeSymbol{\guillemotleft}       {"00AB}

\DeclareUnicodeSymbol{\textlnot}            {"00AC}
\DeclareUnicodeSymbol{\textregistered}      {"00AE}
\DeclareUnicodeSymbol{\textasciimacron}     {"00AF}
\DeclareUnicodeSymbol{\textdegree}          {"00B0}
\DeclareUnicodeSymbol{\textpm}              {"00B1}
\DeclareUnicodeSymbol{\texttwosuperior}     {"00B2}
\DeclareUnicodeSymbol{\textthreesuperior}   {"00B3}
\DeclareUnicodeSymbol{\textasciiacute}      {"00B4}
\DeclareUnicodeSymbol{\textmu}              {"00B5}
\DeclareUnicodeSymbol{\textparagraph}       {"00B6}
\DeclareUnicodeSymbol{\textperiodcentered}  {"00B7}
\DeclareUnicodeSymbol{\textonesuperior}     {"00B9}
\DeclareUnicodeSymbol{\textordmasculine}    {"00BA}

\DeclareUnicodeSymbol{\guillemetright}      {"00BB}
% old Adobe name
\DeclareUnicodeSymbol{\guillemotright}      {"00BB}

\DeclareUnicodeSymbol{\textonequarter}      {"00BC}
\DeclareUnicodeSymbol{\textonehalf}         {"00BD}
\DeclareUnicodeSymbol{\textthreequarters}   {"00BE}
\DeclareUnicodeSymbol{\textquestiondown}    {"00BF}
\DeclareUnicodeSymbol{\AE}                  {"00C6}
\DeclareUnicodeSymbol{\DH}                  {"00D0}
\DeclareUnicodeSymbol{\texttimes}           {"00D7}
\DeclareUnicodeSymbol{\O}                   {"00D8}
\DeclareUnicodeSymbol{\TH}                  {"00DE}
\DeclareUnicodeSymbol{\ss}                  {"00DF}
\DeclareUnicodeSymbol{\ae}                  {"00E6}
\DeclareUnicodeSymbol{\dh}                  {"00F0}
\DeclareUnicodeSymbol{\textdiv}             {"00F7}
\DeclareUnicodeSymbol{\o}                   {"00F8}
\DeclareUnicodeSymbol{\th}                  {"00FE}
\DeclareUnicodeSymbol{\DJ}                  {"0110}
\DeclareUnicodeSymbol{\dj}                  {"0111}
\DeclareUnicodeSymbol{\i}                   {"0131}
\DeclareUnicodeSymbol{\IJ}                  {"0132}
\DeclareUnicodeSymbol{\ij}                  {"0133}
\DeclareUnicodeSymbol{\L}                   {"0141}
\DeclareUnicodeSymbol{\l}                   {"0142}
```

```
\DeclareUnicodeSymbol{\NG}                  {"014A}
\DeclareUnicodeSymbol{\ng}                  {"014B}
\DeclareUnicodeSymbol{\OE}                  {"0152}
\DeclareUnicodeSymbol{\oe}                  {"0153}
\DeclareUnicodeSymbol{\textflorin}          {"0192}
\DeclareUnicodeSymbol{\j}                   {"0237}
\DeclareUnicodeSymbol{\textasciicaron}      {"02C7}
\DeclareUnicodeSymbol{\textasciibreve}      {"02D8}
\DeclareUnicodeSymbol{\textacutedbl}        {"02DD}
\DeclareUnicodeSymbol{\textgravedbl}        {"02F5}
\DeclareUnicodeSymbol{\texttildelow}        {"02F7}
\DeclareUnicodeSymbol{\textbaht}            {"0E3F}
\DeclareUnicodeSymbol{\SS}                  {"1E9E}
\DeclareUnicodeSymbol{\textcompwordmark}    {"200C}

%\DeclareUnicodeSymbol{\textnonbreakinghyphen} {"2011}
%\DeclareUnicodeSymbol{\textfiguredash}        {"2012}
\DeclareUnicodeSymbol{\textendash}          {"2013}
\DeclareUnicodeSymbol{\textemdash}          {"2014}
%\DeclareUnicodeSymbol{\texthorizontalbar}   {"2015}
```

Unfortunately some fonts do not implement `"2011`, `"2012` and/or `"2015` (including the LaTeX default fonts for Unicode engines) so we provide some approximations if the glyph is missing, like we do for `OT1` and `T1`.

The `\nobreak\hskip\z@` is there to prevent a break after the hyphen but allow later breaks in the remainder of the word.

```
\DeclareUnicodeCommand{\textnonbreakinghyphen}
     {\iffontchar\font "2011 \char "2011 \else \mbox{-}\nobreak\hskip\z@ \fi}
\DeclareUnicodeCommand{\textfiguredash}
     {\iffontchar\font "2012 \char "2012 \else \char "2013 \fi}
\DeclareUnicodeCommand{\texthorizontalbar}
     {\iffontchar\font "2015 \char "2015 \else \char "2014 \fi}

\DeclareUnicodeSymbol{\textbardbl}          {"2016}
\DeclareUnicodeSymbol{\textquoteleft}       {"2018}
\DeclareUnicodeSymbol{\textquoteright}      {"2019}
\DeclareUnicodeSymbol{\quotesinglbase}      {"201A}
\DeclareUnicodeSymbol{\textquotedblleft}    {"201C}
\DeclareUnicodeSymbol{\textquotedblright}   {"201D}
\DeclareUnicodeSymbol{\quotedblbase}        {"201E}
\DeclareUnicodeSymbol{\textdagger}          {"2020}
\DeclareUnicodeSymbol{\textdaggerdbl}       {"2021}
\DeclareUnicodeSymbol{\textbullet}          {"2022}
\DeclareUnicodeSymbol{\textellipsis}        {"2026}
\DeclareUnicodeSymbol{\textperthousand}     {"2030}
\DeclareUnicodeSymbol{\textpertenthousand}  {"2031}
\DeclareUnicodeSymbol{\guilsinglleft}       {"2039}
\DeclareUnicodeSymbol{\guilsinglright}      {"203A}
\DeclareUnicodeSymbol{\textreferencemark}   {"203B}
\DeclareUnicodeSymbol{\textinterrobang}     {"203D}
\DeclareUnicodeSymbol{\textfractionsolidus} {"2044}
\DeclareUnicodeSymbol{\textlquill}          {"2045}
\DeclareUnicodeSymbol{\textrquill}          {"2046}
\DeclareUnicodeSymbol{\textdiscount}        {"2052}
\DeclareUnicodeSymbol{\textcolonmonetary}   {"20A1}
```

```
\DeclareUnicodeSymbol{\textlira}            {"20A4}
\DeclareUnicodeSymbol{\textnaira}           {"20A6}
\DeclareUnicodeSymbol{\textwon}             {"20A9}
\DeclareUnicodeSymbol{\textdong}            {"20AB}
\DeclareUnicodeSymbol{\texteuro}            {"20AC}
\DeclareUnicodeSymbol{\textpeso}            {"20B1}
\DeclareUnicodeSymbol{\textcelsius}         {"2103}
\DeclareUnicodeSymbol{\textnumero}          {"2116}
\DeclareUnicodeSymbol{\textcircledP}        {"2117}
\DeclareUnicodeSymbol{\textrecipe}          {"211E}
\DeclareUnicodeSymbol{\textservicemark}     {"2120}
\DeclareUnicodeSymbol{\texttrademark}       {"2122}
\DeclareUnicodeSymbol{\textohm}             {"2126}
\DeclareUnicodeSymbol{\textmho}             {"2127}
\DeclareUnicodeSymbol{\textestimated}       {"212E}
\DeclareUnicodeSymbol{\textleftarrow}       {"2190}
\DeclareUnicodeSymbol{\textuparrow}         {"2191}
\DeclareUnicodeSymbol{\textrightarrow}      {"2192}
\DeclareUnicodeSymbol{\textdownarrow}       {"2193}
\DeclareUnicodeSymbol{\textminus}           {"2212}

\DeclareUnicodeSymbol{\Hwithstroke}         {"0126}
\DeclareUnicodeSymbol{\hwithstroke}         {"0127}
```

Not all fonts have U+2217 but using U+002A requires some adjustment.

```
\DeclareUnicodeCommand{\textasteriskcentered}{%
  \iffontchar\font"2217 \char"2217 \else
    \begingroup
      \fontsize
       {\the\dimexpr1.3\dimexpr\f@size pt\relax}%
       {\f@baselineskip}%
      \selectfont
      \raisebox{-0.7ex}[\dimexpr\height-0.7ex][0pt]{*}%
    \endgroup
  \fi
}
\DeclareUnicodeSymbol{\textsurd}            {"221A}
\DeclareUnicodeSymbol{\textlangle}          {"2329}
\DeclareUnicodeSymbol{\textrangle}          {"232A}
\DeclareUnicodeSymbol{\textblank}           {"2422}
\DeclareUnicodeSymbol{\textvisiblespace}    {"2423}
\DeclareUnicodeSymbol{\textopenbullet}      {"25E6}
\DeclareUnicodeSymbol{\textbigcircle}       {"25EF}
\DeclareUnicodeSymbol{\textmusicalnote}     {"266A}
\DeclareUnicodeSymbol{\textmarried}         {"26AD}
\DeclareUnicodeSymbol{\textdivorced}        {"26AE}
\DeclareUnicodeSymbol{\textinterrobangdown} {"2E18}
```

Accents must be declared before the composites that use them.

```
\DeclareUnicodeAccent{\‘}{"0300}
\DeclareUnicodeAccent{\’}{"0301}
\DeclareUnicodeAccent{\^}{"0302}
\DeclareUnicodeAccent{\~}{"0303}
```

```
\DeclareUnicodeAccent{\=}{"0304}
\DeclareUnicodeAccent{\u}{"0306}
\DeclareUnicodeAccent{\.}{"0307}
\DeclareUnicodeAccent{\"}{"0308}
\DeclareUnicodeAccent{\r}{"030A}
\DeclareUnicodeAccent{\H}{"030B}
\DeclareUnicodeAccent{\v}{"030C}
\DeclareUnicodeAccent{\b}{"0332}
\DeclareUnicodeAccent{\d}{"0323}
\DeclareUnicodeAccent{\c}{"0327}
\DeclareUnicodeAccent{\k}{"0328}
```

The odd one out:

```
\DeclareUnicodeCommand\textcommabelow[1]
  {\hmode@bgroup\ooalign{\null#1\crcr\hidewidth\raise-.31ex
   \hbox{\check@mathfonts\fontsize\ssf@size\z@
   \math@fontsfalse\selectfont,}\hidewidth}\egroup}
```

```
\DeclareUnicodeComposite{\^}              {}{"005E}
\DeclareUnicodeComposite{\~}              {}{"007E}
```

```
\DeclareUnicodeComposite{\`}             {A}{"00C0}
\DeclareUnicodeComposite{\'}             {A}{"00C1}
\DeclareUnicodeComposite{\^}             {A}{"00C2}
\DeclareUnicodeComposite{\~}             {A}{"00C3}
\DeclareUnicodeComposite{\"}             {A}{"00C4}
\DeclareUnicodeComposite{\r}             {A}{"00C5}
\DeclareUnicodeComposite{\c}             {C}{"00C7}
\DeclareUnicodeComposite{\`}             {E}{"00C8}
\DeclareUnicodeComposite{\'}             {E}{"00C9}
\DeclareUnicodeComposite{\^}             {E}{"00CA}
\DeclareUnicodeComposite{\"}             {E}{"00CB}
\DeclareUnicodeComposite{\`}             {I}{"00CC}
\DeclareUnicodeComposite{\'}             {I}{"00CD}
\DeclareUnicodeComposite{\^}             {I}{"00CE}
\DeclareUnicodeComposite{\"}             {I}{"00CF}
\DeclareUnicodeComposite{\~}             {N}{"00D1}
\DeclareUnicodeComposite{\`}             {O}{"00D2}
\DeclareUnicodeComposite{\'}             {O}{"00D3}
\DeclareUnicodeComposite{\^}             {O}{"00D4}
\DeclareUnicodeComposite{\~}             {O}{"00D5}
\DeclareUnicodeComposite{\"}             {O}{"00D6}
\DeclareUnicodeComposite{\`}             {U}{"00D9}
\DeclareUnicodeComposite{\'}             {U}{"00DA}
\DeclareUnicodeComposite{\^}             {U}{"00DB}
\DeclareUnicodeComposite{\"}             {U}{"00DC}
\DeclareUnicodeComposite{\'}             {Y}{"00DD}
\DeclareUnicodeComposite{\`}             {a}{"00E0}
\DeclareUnicodeComposite{\'}             {a}{"00E1}
\DeclareUnicodeComposite{\^}             {a}{"00E2}
\DeclareUnicodeComposite{\~}             {a}{"00E3}
\DeclareUnicodeComposite{\"}             {a}{"00E4}
\DeclareUnicodeComposite{\r}             {a}{"00E5}
\DeclareUnicodeComposite{\c}             {c}{"00E7}
\DeclareUnicodeComposite{\`}             {e}{"00E8}
\DeclareUnicodeComposite{\'}             {e}{"00E9}
```

```
\DeclareUnicodeComposite{\^}            {e}{"00EA}
\DeclareUnicodeComposite{\"}            {e}{"00EB}
\DeclareUnicodeComposite{\`}            \i {"00EC}
\DeclareUnicodeComposite{\`}            {i}{"00EC}
\DeclareUnicodeComposite{\'}            \i {"00ED}
\DeclareUnicodeComposite{\'}            {i}{"00ED}
\DeclareUnicodeComposite{\^}            \i {"00EE}
\DeclareUnicodeComposite{\^}            {i}{"00EE}
\DeclareUnicodeComposite{\"}            \i {"00EF}
\DeclareUnicodeComposite{\"}            {i}{"00EF}
\DeclareUnicodeComposite{\~}            {n}{"00F1}
\DeclareUnicodeComposite{\`}            {o}{"00F2}
\DeclareUnicodeComposite{\'}            {o}{"00F3}
\DeclareUnicodeComposite{\^}            {o}{"00F4}
\DeclareUnicodeComposite{\~}            {o}{"00F5}
\DeclareUnicodeComposite{\"}            {o}{"00F6}
\DeclareUnicodeComposite{\`}            {u}{"00F9}
\DeclareUnicodeComposite{\'}            {u}{"00FA}
\DeclareUnicodeComposite{\^}            {u}{"00FB}
\DeclareUnicodeComposite{\"}            {u}{"00FC}
\DeclareUnicodeComposite{\'}            {y}{"00FD}
\DeclareUnicodeComposite{\"}            {y}{"00FF}
\DeclareUnicodeComposite{\=}            {A}{"0100}
\DeclareUnicodeComposite{\=}            {a}{"0101}
\DeclareUnicodeComposite{\u}            {A}{"0102}
\DeclareUnicodeComposite{\u}            {a}{"0103}
\DeclareUnicodeComposite{\k}            {A}{"0104}
\DeclareUnicodeComposite{\k}            {a}{"0105}
\DeclareUnicodeComposite{\'}            {C}{"0106}
\DeclareUnicodeComposite{\'}            {c}{"0107}
\DeclareUnicodeComposite{\^}            {C}{"0108}
\DeclareUnicodeComposite{\^}            {c}{"0109}
\DeclareUnicodeComposite{\.}            {C}{"010A}
\DeclareUnicodeComposite{\.}            {c}{"010B}
\DeclareUnicodeComposite{\v}            {C}{"010C}
\DeclareUnicodeComposite{\v}            {c}{"010D}
\DeclareUnicodeComposite{\v}            {D}{"010E}
\DeclareUnicodeComposite{\v}            {d}{"010F}
\DeclareUnicodeComposite{\=}            {E}{"0112}
\DeclareUnicodeComposite{\=}            {e}{"0113}
\DeclareUnicodeComposite{\u}            {E}{"0114}
\DeclareUnicodeComposite{\u}            {e}{"0115}
\DeclareUnicodeComposite{\.}            {E}{"0116}
\DeclareUnicodeComposite{\.}            {e}{"0117}
\DeclareUnicodeComposite{\k}            {E}{"0118}
\DeclareUnicodeComposite{\k}            {e}{"0119}
\DeclareUnicodeComposite{\v}            {E}{"011A}
\DeclareUnicodeComposite{\v}            {e}{"011B}
\DeclareUnicodeComposite{\^}            {G}{"011C}
\DeclareUnicodeComposite{\^}            {g}{"011D}
\DeclareUnicodeComposite{\u}            {G}{"011E}
\DeclareUnicodeComposite{\u}            {g}{"011F}
\DeclareUnicodeComposite{\.}            {G}{"0120}
\DeclareUnicodeComposite{\.}            {g}{"0121}
```

```
\DeclareUnicodeComposite{\c}            {G}{"0122}
\DeclareUnicodeComposite{\c}            {g}{"0123}
\DeclareUnicodeComposite{\^}            {H}{"0124}
\DeclareUnicodeComposite{\^}            {h}{"0125}
\DeclareUnicodeComposite{\~}            {I}{"0128}
\DeclareUnicodeComposite{\~}            \i {"0129}
\DeclareUnicodeComposite{\~}            {i}{"0129}
\DeclareUnicodeComposite{\=}            {I}{"012A}
\DeclareUnicodeComposite{\=}            \i {"012B}
\DeclareUnicodeComposite{\=}            {i}{"012B}
\DeclareUnicodeComposite{\u}            {I}{"012C}
\DeclareUnicodeComposite{\u}            \i {"012D}
\DeclareUnicodeComposite{\u}            {i}{"012D}
\DeclareUnicodeComposite{\k}            {I}{"012E}
\DeclareUnicodeComposite{\k}            \i {"012F}
\DeclareUnicodeComposite{\k}            {i}{"012F}
\DeclareUnicodeComposite{\.}            {I}{"0130}
\DeclareUnicodeComposite{\^}            {J}{"0134}
\DeclareUnicodeComposite{\^}            \j {"0135}
\DeclareUnicodeComposite{\^}            {j}{"0135}
\DeclareUnicodeComposite{\c}            {K}{"0136}
\DeclareUnicodeComposite{\c}            {k}{"0137}
\DeclareUnicodeComposite{\'}            {L}{"0139}
\DeclareUnicodeComposite{\'}            {l}{"013A}
\DeclareUnicodeComposite{\c}            {L}{"013B}
\DeclareUnicodeComposite{\c}            {l}{"013C}
\DeclareUnicodeComposite{\v}            {L}{"013D}
\DeclareUnicodeComposite{\v}            {l}{"013E}
\DeclareUnicodeComposite{\'}            {N}{"0143}
\DeclareUnicodeComposite{\'}            {n}{"0144}
\DeclareUnicodeComposite{\c}            {N}{"0145}
\DeclareUnicodeComposite{\c}            {n}{"0146}
\DeclareUnicodeComposite{\v}            {N}{"0147}
\DeclareUnicodeComposite{\v}            {n}{"0148}
\DeclareUnicodeComposite{\=}            {O}{"014C}
\DeclareUnicodeComposite{\=}            {o}{"014D}
\DeclareUnicodeComposite{\u}            {O}{"014E}
\DeclareUnicodeComposite{\u}            {o}{"014F}
\DeclareUnicodeComposite{\H}            {O}{"0150}
\DeclareUnicodeComposite{\H}            {o}{"0151}
\DeclareUnicodeComposite{\'}            {R}{"0154}
\DeclareUnicodeComposite{\'}            {r}{"0155}
\DeclareUnicodeComposite{\c}            {R}{"0156}
\DeclareUnicodeComposite{\c}            {r}{"0157}
\DeclareUnicodeComposite{\v}            {R}{"0158}
\DeclareUnicodeComposite{\v}            {r}{"0159}
\DeclareUnicodeComposite{\'}            {S}{"015A}
\DeclareUnicodeComposite{\'}            {s}{"015B}
\DeclareUnicodeComposite{\^}            {S}{"015C}
\DeclareUnicodeComposite{\^}            {s}{"015D}
\DeclareUnicodeComposite{\c}            {S}{"015E}
\DeclareUnicodeComposite{\c}            {s}{"015F}
\DeclareUnicodeComposite{\v}            {S}{"0160}
\DeclareUnicodeComposite{\v}            {s}{"0161}
```

```
\DeclareUnicodeComposite{\c}            {T}{"0162}
\DeclareUnicodeComposite{\c}            {t}{"0163}
\DeclareUnicodeComposite{\v}            {T}{"0164}
\DeclareUnicodeComposite{\v}            {t}{"0165}
\DeclareUnicodeComposite{\~}            {U}{"0168}
\DeclareUnicodeComposite{\~}            {u}{"0169}
\DeclareUnicodeComposite{\=}            {U}{"016A}
\DeclareUnicodeComposite{\=}            {u}{"016B}
\DeclareUnicodeComposite{\u}            {U}{"016C}
\DeclareUnicodeComposite{\u}            {u}{"016D}
\DeclareUnicodeComposite{\r}            {U}{"016E}
\DeclareUnicodeComposite{\r}            {u}{"016F}
\DeclareUnicodeComposite{\H}            {U}{"0170}
\DeclareUnicodeComposite{\H}            {u}{"0171}
\DeclareUnicodeComposite{\k}            {U}{"0172}
\DeclareUnicodeComposite{\k}            {u}{"0173}
\DeclareUnicodeComposite{\^}            {W}{"0174}
\DeclareUnicodeComposite{\^}            {w}{"0175}
\DeclareUnicodeComposite{\^}            {Y}{"0176}
\DeclareUnicodeComposite{\^}            {y}{"0177}
\DeclareUnicodeComposite{\"}            {Y}{"0178}
\DeclareUnicodeComposite{\'}            {Z}{"0179}
\DeclareUnicodeComposite{\'}            {z}{"017A}
\DeclareUnicodeComposite{\.}            {Z}{"017B}
\DeclareUnicodeComposite{\.}            {z}{"017C}
\DeclareUnicodeComposite{\v}            {Z}{"017D}
\DeclareUnicodeComposite{\v}            {z}{"017E}
\DeclareUnicodeComposite{\v}            {A}{"01CD}
\DeclareUnicodeComposite{\v}            {a}{"01CE}
\DeclareUnicodeComposite{\v}            {I}{"01CF}
\DeclareUnicodeComposite{\v}            \i {"01D0}
\DeclareUnicodeComposite{\v}            {i}{"01D0}
\DeclareUnicodeComposite{\v}            {O}{"01D1}
\DeclareUnicodeComposite{\v}            {o}{"01D2}
\DeclareUnicodeComposite{\v}            {U}{"01D3}
\DeclareUnicodeComposite{\v}            {u}{"01D4}

\DeclareUnicodeComposite{\'}            \AE{"01FC}
\DeclareUnicodeComposite{\'}            {Æ}{"01FC}
\DeclareUnicodeComposite{\'}            \ae{"01FD}
\DeclareUnicodeComposite{\'}            {æ}{"01FD}
\DeclareUnicodeComposite{\=}            \AE{"01E2}
\DeclareUnicodeComposite{\=}            {Æ}{"01E2}
\DeclareUnicodeComposite{\=}            \ae{"01E3}
\DeclareUnicodeComposite{\=}            {æ}{"01E3}
\DeclareUnicodeComposite{\v}            {G}{"01E6}
\DeclareUnicodeComposite{\v}            {g}{"01E7}
\DeclareUnicodeComposite{\v}            {K}{"01E8}
\DeclareUnicodeComposite{\v}            {k}{"01E9}
\DeclareUnicodeComposite{\k}            {O}{"01EA}
\DeclareUnicodeComposite{\k}            {o}{"01EB}
\DeclareUnicodeComposite{\v}            \j {"01F0}
\DeclareUnicodeComposite{\v}            {j}{"01F0}
\DeclareUnicodeComposite{\'}            {G}{"01F4}
```

```
\DeclareUnicodeComposite{\'}             {g}{"01F5}
\DeclareUnicodeComposite{\textcommabelow}{S}{"0218}
\DeclareUnicodeComposite{\textcommabelow}{s}{"0219}
\DeclareUnicodeComposite{\textcommabelow}{T}{"021A}
\DeclareUnicodeComposite{\textcommabelow}{t}{"021B}
\DeclareUnicodeComposite{\=}             {Y}{"0232}
\DeclareUnicodeComposite{\=}             {y}{"0233}
\DeclareUnicodeComposite{\.}             {B}{"1E02}
\DeclareUnicodeComposite{\.}             {b}{"1E03}
\DeclareUnicodeComposite{\d}             {B}{"1E04}
\DeclareUnicodeComposite{\d}             {b}{"1E05}
\DeclareUnicodeComposite{\d}             {D}{"1E0C}
\DeclareUnicodeComposite{\d}             {d}{"1E0D}
\DeclareUnicodeComposite{\=}             {G}{"1E20}
\DeclareUnicodeComposite{\=}             {g}{"1E21}
\DeclareUnicodeComposite{\d}             {H}{"1E24}
\DeclareUnicodeComposite{\d}             {h}{"1E25}
\DeclareUnicodeComposite{\d}             {K}{"1E32}
\DeclareUnicodeComposite{\d}             {k}{"1E33}
\DeclareUnicodeComposite{\d}             {L}{"1E36}
\DeclareUnicodeComposite{\d}             {l}{"1E37}
\DeclareUnicodeComposite{\d}             {M}{"1E42}
\DeclareUnicodeComposite{\d}             {m}{"1E43}
\DeclareUnicodeComposite{\d}             {N}{"1E46}
\DeclareUnicodeComposite{\d}             {n}{"1E47}
\DeclareUnicodeComposite{\d}             {R}{"1E5A}
\DeclareUnicodeComposite{\d}             {r}{"1E5B}
\DeclareUnicodeComposite{\d}             {S}{"1E62}
\DeclareUnicodeComposite{\d}             {s}{"1E63}
\DeclareUnicodeComposite{\d}             {T}{"1E6C}
\DeclareUnicodeComposite{\d}             {t}{"1E6D}
\DeclareUnicodeComposite{\d}             {V}{"1E7E}
\DeclareUnicodeComposite{\d}             {v}{"1E7F}
\DeclareUnicodeComposite{\d}             {W}{"1E88}
\DeclareUnicodeComposite{\d}             {w}{"1E89}
\DeclareUnicodeComposite{\d}             {Z}{"1E92}
\DeclareUnicodeComposite{\d}             {z}{"1E93}
\DeclareUnicodeComposite{\d}             {A}{"1EA0}
\DeclareUnicodeComposite{\d}             {a}{"1EA1}
\DeclareUnicodeComposite{\d}             {E}{"1EB8}
\DeclareUnicodeComposite{\d}             {e}{"1EB9}
\DeclareUnicodeComposite{\d}             {I}{"1ECA}
\DeclareUnicodeComposite{\d}             {i}{"1ECB}
\DeclareUnicodeComposite{\d}             {O}{"1ECC}
\DeclareUnicodeComposite{\d}             {o}{"1ECD}
\DeclareUnicodeComposite{\d}             {U}{"1EE4}
\DeclareUnicodeComposite{\d}             {u}{"1EE5}
\DeclareUnicodeComposite{\d}             {Y}{"1EF4}
\DeclareUnicodeComposite{\d}             {y}{"1EF5}
```

⟨/TU⟩

2 Package files

This file now also contains some packages that provide access to the more specialized encodings.

2.1 The fontenc package

This package allows authors to specify which encodings they will use. For each encoding FOO, the package looks to see if the encoding FOO has already been declared. If it has not, the file fooenc.def is loaded. The default encoding is set to be FOO.

In addition the package at the moment contains extra code to extend the \@uclclist (list of upper/lower case pairs) for encodings that involve cyrillic characters. THIS IS A TEMPORARY SOLUTION and will not stay this way forever (or so we hope) but right now we are missing a proper interface for this and didn't wanted to rush it.

1532 ⟨*package⟩

Here we define a macro that extends the \@uclclist if needed and afterwards turns itself in a noop.

```
1533 \def\update@uclc@with@cyrillic{%
1534  \expandafter\def\expandafter\@uclclist\expandafter
1535   {\@uclclist
1536   \cyra\CYRA\cyrabhch\CYRABHCH\cyrabhchdsc\CYRABHCHDSC\cyrabhdze
1537   \CYRABHDZE\cyrabhha\CYRABHHA\cyrae\CYRAE\cyrb\CYRB\cyrbyus
1538   \CYRBYUS\cyrc\CYRC\cyrch\CYRCH\cyrchldsc\CYRCHLDSC\cyrchrdsc
1539   \CYRCHRDSC\cyrchvcrs\CYRCHVCRS\cyrd\CYRD\cyrdelta\CYRDELTA
1540   \cyrdje\CYRDJE\cyrdze\CYRDZE\cyrdzhe\CYRDZHE\cyre\CYRE\cyreps
1541   \CYREPS\cyrerev\CYREREV\cyrery\CYRERY\cyrf\CYRF\cyrfita
1542   \CYRFITA\cyrg\CYRG\cyrgdsc\CYRGDSC\cyrgdschcrs\CYRGDSCHCRS
1543   \cyrghcrs\CYRGHCRS\cyrghk\CYRGHK\cyrgup\CYRGUP\cyrh\CYRH
1544   \cyrhdsc\CYRHDSC\cyrhhcrs\CYRHHCRS\cyrhhk\CYRHHK\cyrhrdsn
1545   \CYRHRDSN\cyri\CYRI\cyrie\CYRIE\cyrii\CYRII\cyrishrt\CYRISHRT
1546   \cyrishrtdsc\CYRISHRTDSC\cyrizh\CYRIZH\cyrje\CYRJE\cyrk\CYRK
1547   \cyrkbeak\CYRKBEAK\cyrkdsc\CYRKDSC\cyrkhcrs\CYRKHCRS\cyrkhk
1548   \CYRKHK\cyrkvcrs\CYRKVCRS\cyrl\CYRL\cyrldsc\CYRLDSC\cyrlhk
1549   \CYRLHK\cyrlje\CYRLJE\cyrm\CYRM\cyrmdsc\CYRMDSC\cyrmhk\CYRMHK
1550   \cyrn\CYRN\cyrndsc\CYRNDSC\cyrng\CYRNG\cyrnhk\CYRNHK\cyrnje
1551   \CYRNJE\cyrnlhk\CYRNLHK\cyro\CYRO\cyrotld\CYROTLD\cyrp\CYRP
1552   \cyrphk\CYRPHK\cyrq\CYRQ\cyrr\CYRR\cyrrdsc\CYRRDSC\cyrrhk
1553   \CYRRHK\cyrrtick\CYRRTICK\cyrs\CYRS\cyrsacrs\CYRSACRS
1554   \cyrschwa\CYRSCHWA\cyrsdsc\CYRSDSC\cyrsemisftsn\CYRSEMISFTSN
1555   \cyrsftsn\CYRSFTSN\cyrsh\CYRSH\cyrshch\CYRSHCH\cyrshha\CYRSHHA
1556   \cyrt\CYRT\cyrtdsc\CYRTDSC\cyrtetse\CYRTETSE\cyrtshe\CYRTSHE
1557   \cyru\CYRU\cyrushrt\CYRUSHRT\cyrv\CYRV\cyrw\CYRW\cyry\CYRY
1558   \cyrya\CYRYA\cyryat\CYRYAT\cyryhcrs\CYRYHCRS\cyryi\CYRYI\cyryo
1559   \CYRYO\cyryu\CYRYU\cyrz\CYRZ\cyrzdsc\CYRZDSC\cyrzh\CYRZH
1560   \cyrzhdsc\CYRZHDSC}%
1561  \let\update@uclc@with@cyrillic\relax
1562 }
```

Here we process each option:

```
1563 \DeclareOption*{%
1564    \let\encodingdefault\CurrentOption
```

From 2020/02/02 release onward we only load the encoding files if they haven't be loaded already. To check this we look at whether `\T@`*encoding* is already defined. If not, we load it later (indicated by setting the switch `@tempswa` to true) and we always load if we are using an older format (or rather in a rollback situation).

```
\@tempswafalse
\@ifl@t@r\fmtversion{2020/02/02}%
    {\expandafter\ifx\csname T@\CurrentOption\endcsname\relax
      \@tempswatrue\fi}%
    {\@tempswatrue}%
```

Load if necessary:

```
\if@tempswa

  \edef\reserved@f{%
    \lowercase{\def\noexpand\reserved@f{\CurrentOption enc.def}}}%
  \reserved@f
  \InputIfFileExists\reserved@f
       {}{\PackageError{fontenc}%
        {Encoding file `\reserved@f' not found.%
         \MessageBreak
         You might have misspelled the name of the encoding
         \MessageBreak
         or a required support package (e.g., cyrillic) is
         \MessageBreak
         missing in your installation}%
        {Necessary code for this encoding was not
         loaded.\MessageBreak
         Thus calling the encoding later on will
         produce further error messages.}}%
  \let\reserved@f\relax
```

In case the current encoding is one of a list of known cyrillic ones we extend the `\@uclclist`:

```
  \expandafter\in@\expandafter{\CurrentOption}%
                              {T2A,T2B,T2C,X2,LCY,OT2}%
  \ifin@
```

But only if it hasn't already been extended. This might happen if there are several calls to fontenc loading one of the above encodings. If we don't do this check the `\@uclclist` gets unnecessarily big, slowing down the processing at runtime.

```
    \expandafter\in@\expandafter\cyra\expandafter
                              {\@uclclist}%
    \ifin@
    \else
      \update@uclc@with@cyrillic
    \fi
  \fi
 \fi
}

\ProcessOptions*
```

We select the new font encoding default (i.e., the last encoding specified in the option list). But this encoding may not work with the current `\f@shape`: e.g., `LY1` is not defined for `cmr` and therefore packages switching to `LY1` usually also change `\rmdefault`. But

that only applies at `\begin{document}` so we get a spurious warning if we use what LaTeX previously used:

```
%\fontencoding\encodingdefault\selectfont
```

So instead we do this here:

```
\usefont\encodingdefault\familydefault\seriesdefault\shapedefault
```

To save some space we get rid of the macro extending the `\@uclclist` (might have happened already).

```
\let\update@uclc@with@cyrillic\relax
```

Finally we pretend that the fontenc package wasn't read in. This allows for using it several times, e.g., in a class file and in the preamble (at the cost of not getting any version info). That kind of hackery shows that using a general purpose package just for loading an encoding is not the right kind of interface for setting up encodings — it will get replaced at some point in the future.

```
\let\@elt\relax
\xdef\@fontenc@load@list{\@fontenc@load@list
  \@elt{\csname opt@fontenc.sty\endcsname}}

\global\expandafter\let\csname ver@@fontenc.sty\expandafter\endcsname
                        \csname ver@fontenc.sty\endcsname
\global\expandafter\let\csname ver@fontenc.sty\endcsname\relax
\global\expandafter\let\csname opt@fontenc.sty\endcsname\relax
\global\let\@ifl@ter@@\@ifl@ter
\def\@ifl@ter#1#2#3#4#5{\global\let\@ifl@ter\@ifl@ter@@}
⟨/package⟩
```

File 22
ltcounts.dtx

1 Counters and Lengths

Commands for defining and using counters. This file defines the following commands. In each case {⟨*counter*⟩} may be * denoting the current counter as set by a previous \refstepcounter.

\newcounter To define a new counter.

\newcounteralias To define an alias name for a counter.

\setcounter To set the value of counters.

\addtocounter Increase the {⟨*counter*⟩} #1 by the number #2.

\stepcounter Increase the {⟨*counter*⟩} by one.

\refstepcounter Increase the {⟨*counter*⟩} by one, also setting the value used by \label.

\value For accessing the value of the counter as a TEX number (as opposed to \the⟨*counter*⟩ which expands to the *printed* representation of ⟨*counter*⟩)

\arabic \arabic{⟨*counter*⟩}: 1, 2, 3, ...

\roman \roman{⟨*counter*⟩}: i, ii, iii, ...

\Roman \Roman{⟨*counter*⟩}: I, II, III, ...

\alph \alph{⟨*counter*⟩}: a, b, c, ...

\Alph \Alph{⟨*counter*⟩}: A, B, C, ...

\fnsymbol \fnsymbol{⟨*counter*⟩}: ∗, †, ‡, ...

\counterwithin \counterwithin[⟨*format*⟩]{⟨*counter*⟩}{⟨*within-counter*⟩}: Resets ⟨*counter*⟩ whenever ⟨*within-counter*⟩ is stepped. Also redefines \the⟨*counter*⟩ command to produce \the⟨*within-counter*⟩.⟨*format*⟩{⟨*counter*⟩} with \arabic as the default for ⟨*format*⟩. Star form omits redefining the print representation.

The * alias for the current counter may not be used in either argument.

\counterwithout \counterwithout[⟨*format*⟩]{⟨*counter*⟩}{⟨*within-counter*⟩}: Removes ⟨*counter*⟩ from the reset list of ⟨*within-counter*⟩. Also redefines \the⟨*counter*⟩ command to produce ⟨*format*⟩{⟨*counter*⟩} with \arabic as the default for ⟨*format*⟩. Star form omits redefining the print representation.

The * alias for the current counter may not be used in either argument.

1 ⟨∗2ekernel⟩

1.1 Document command and environment counter macros

An environment foo has an associated counter defined by the following control sequences:

`\c@foo`	Contains the counter's numerical value. It is defined by `\newcount\foocounter`.

`\thefoo`	Macro that expands to the printed value of `\foocounter`. For example, if sections are numbered within chapters, and section headings look like
	Section II-3. The Nature of Counters
	then `\thesection` might be defined by:
	`\def\thesection`
	`{\@Roman{\c@chapter}-\@arabic{\c@section}}`

`\theHfoo`	Macro that is used to create destination names. It should give a unique value across the document for every `\refstepcounter`.
`\p@foo`	Macro that expands to a printed 'reference prefix' of counter foo. Any `\ref` to a value created by counter foo will produce the expansion of `\p@foo\thefoo` when the `\label` command is executed. See file `ltxref.dtx` for an extension of this mechanism.
`\cl@foo`	List of counters to be reset when foo stepped. Has format `\@elt{countera}\@elt{counterb}\@elt{counterc}`.
`\alias@ctr@foo`	if foo is an alias counter created with `\newcounteralias` this command contains the name of the root counter.

For some environments the counter is named slightly differently, e.g., the counters associated with the `enumerate` environment have the names `enumi`, `enumii`, `enumiii`, and `enumiv` depending on the nesting level.

The same internal macros are defined for counters used by commands such as `\chapter`, `\section`, etc.

NOTE:

`\thefoo` and `\p@foo` *must* be defined in such a way that `\edef\bar{\thefoo}` or `\edef\bar{\p@foo}` defines `\bar` so that it will evaluate to the counter value at the time of the `\edef`, even after `\foocounter` and any other counters have been changed. This will happen if you use the standard commands `\@arabic`, `\@Roman`, etc.

The following commands are used to define and modify counters.

`\refstepcounter{`⟨*foo*⟩`}`

Same as `\stepcounter`, but it also defines `\@currentlabel`, `\@currentHref` and `\@currentcounter` and so that a subsequent `\label{`⟨*bar*⟩`}` command causes `\ref{`⟨*bar*⟩`}` to generate the current value of counter ⟨*foo*⟩.

`\@definecounter{`⟨*foo*⟩`}`

Initializes counter `{`⟨*foo*⟩`}` (with empty reset list), defines `\p@foo` and `\thefoo` to be null and `\theHfoo` to be `\number\value{foo}`. Also adds ⟨*foo*⟩ to `\cl@@ckpt` – the reset list of a dummy counter `@ckpt` used for taking checkpoints for the `\include` system.

`\@addtoreset{`⟨*foo*⟩`}{`⟨*bar*⟩`}` : Adds counter ⟨*foo*⟩ to the list of counters `\cl@bar` to be reset when counter ⟨*bar*⟩ is stepped.

`\@removefromreset{`⟨*foo*⟩`}{`⟨*bar*⟩`}` : Removes counter ⟨*foo*⟩ to the list of counters `\cl@bar` to be reset when counter ⟨*bar*⟩ is stepped.

`\setcounter` `\setcounter{`⟨*foo*⟩`}{`⟨*val*⟩`}` : Globally sets `\foocounter` equal to ⟨*val*⟩.

```
\def\setcounter#1#2{%
  \@ifundefined{c@#1}%
    {\@nocounterr{#1}}%
    {\global\csname c@#1\endcsname#2\relax}}
```

(*End of definition for* `\setcounter`.)

`\addtocounter` `\addtocounter{`⟨*foo*⟩`}{`⟨*val*⟩`}` Globally increments `\foocounter` by ⟨`val`⟩.

```
\def\addtocounter#1#2{%
  \@ifundefined{c@#1}%
    {\@nocounterr{#1}}%
    {\global\advance\csname c@#1\endcsname #2\relax}}
```

(*End of definition for* `\addtocounter`.)

`\newcounter` `\newcounter{`⟨*newctr*⟩`}[`⟨*oldctr*⟩`]` Defines ⟨`newctr`⟩ to be a counter, which is reset when counter ⟨`oldctr`⟩ is stepped. If ⟨`newctr`⟩ already defined produces '`c@newctr  already defined`' error.

```
\def\newcounter#1{%
  \expandafter\@ifdefinable \csname c@#1\endcsname
    {\@definecounter{#1}}%
  \@ifnextchar[{\@newctr{#1}}{}}
```

(*End of definition for* `\newcounter`.)

`\newcounteralias` `\newcounteralias{`⟨*aliasctr*⟩`}{`⟨*rootctr*⟩`}` Defines ⟨`aliasctr`⟩ as an alias for the counter ⟨`rootctr`⟩. The root counter can itself be an alias counter (it is not recommended to chain counter names like this as keeping track of the representations can get tricky, but explicit tests to avoid that seems unnecessary). The various representation commands of the alias counter call the respective representations of the root. This means that if a representation of the root is redefined, e.g. to make `\theH`⟨`counter`⟩ unique then the alias follows. It is possible to redefine an alias representation, but this then breaks the connection to the root representation.

Every alias counter FOO has an associated command `\alias@ctr@FOO` which points to the root. This allows to test if a counter is an alias counter and also allows to find the root.

We do not rollback the definition to avoid errors if packages use it that do not rollback.

```
\newcommand*{\newcounteralias}[2]{%
    \@ifundefined{c@#2}{%
      \@nocounterr{#2}%
    }{%
      \expandafter\@ifdefinable\csname c@#1\endcsname{%
       \global\expandafter\let
        \csname c@#1\expandafter\endcsname\csname c@#2\endcsname
       \expandafter\edef\csname the#1\endcsname{\expandafter\noexpand\csname the#2\endcsname}
       \expandafter\edef\csname theH#1\endcsname{\expandafter\noexpand\csname theH#2\endcsnam
       \expandafter\edef\csname p@#1\endcsname{\expandafter\noexpand\csname p@#2\endcsname}%
       \expandafter\edef\csname cl@#1\endcsname{\expandafter\noexpand\csname cl@#2\endcsname}
       \expandafter\def\csname alias@ctr@#1\endcsname{#2}%
      }%
    }%
}
```

(*End of definition for* `\newcounteralias`.)

`\value` `\value{`⟨*ctr*⟩`}` produces the value of counter ⟨`ctr`⟩, for use with a `\setcounter` or `\addtocounter` command.

```
\def\value#1{\csname c@#1\endcsname}
```

(*End of definition for* `\value`.)

`\c@*` Make the current counter available as a LaTeX counter with name `*`, so `\alph*` returns the current counter as a letter, `\stepcounter{*}` increments the current counter, etc.

```
\protected\expandafter
  \def\csname c@*\endcsname{\value\@currentcounter}
```

(*End of definition for* `\c@*`.)

`\@newctr`

```
\def\@newctr#1[#2]{%
  \@ifundefined{c@#2}{\@nocounterr{#2}}{\@addtoreset{#1}{#2}}}
```

(*End of definition for* `\@newctr`.)

`\stepcounter` `\stepcounter`foo Globally increments counter `\c@FOO` and resets all subsidiary counters.

```
\def\stepcounter#1{%
  \addtocounter{#1}\@ne
  \begingroup
    \let\@elt\@stpelt
    \csname cl@#1\endcsname
  \endgroup}
```

(*End of definition for* `\stepcounter`.)

`\@stpelt` Rather than resetting the "within" counter to zero we set it to -1 and then run `\stepcounter` that moves it to 0 and also initiates resetting the next level down.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}{\@stpelt}
⟨latexrelease⟩                                {Reset nested counters}%
⟨*2ekernel | latexrelease⟩
\def\@stpelt#1{\global\csname c@#1\endcsname \m@ne\stepcounter{#1}}%
⟨latexrelease⟩\EndIncludeInRelease
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\@stpelt}
⟨latexrelease⟩                                {Reset nested counters}%%
⟨latexrelease⟩\def\@stpelt#1{\global\csname c@#1\endcsname \z@}%
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@stpelt`.)

`\cl@@ckpt`

```
\def\cl@@ckpt{\@elt{page}}
```

(*End of definition for* `\cl@@ckpt`.)

`\@definecounter` Older definitions of `\@definecounter` called unconditionally `\newcount`. This had the effect that connection to an alias counter could break if a user did, e.g.,

```
\newcounter{theorem}
\newcounteralias{lemma}{theorem}
\newtheorem{theorem}{Theorem}
\newtheorem{lemma}{Lemma}
```

The command was therefore changed and now tests if the count already exist. If the counter to define is an already existing alias counter then we do not reset the representations and do not add it to `@ckpt`.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2026/06/01}{\@definecounter}
⟨latexrelease⟩                                {add aliascounter}%
⟨∗2ekernel | latexrelease⟩
\def\@definecounter#1{%
     \@ifundefined{c@#1}{\expandafter\newcount\csname c@#1\endcsname}{}%
     \setcounter{#1}\z@
     \@ifundefined{alias@ctr@#1}
      {\global\expandafter\let\csname cl@#1\endcsname\@empty
       \@addtoreset{#1}{@ckpt}%
       \global\expandafter\let\csname p@#1\endcsname\@empty
       \expandafter\xdef\csname theH#1\endcsname{%
         \noexpand\the\noexpand\value{#1}}}{}%
```

If `\the#1` is undefined or `\relax` we define it with the standard definition for counters, otherwise we warn. This will catch, for example, that somebody defines a counter named "index" conflicting with the `theindex` environment.

```
     \expandafter
     \ifx\csname the#1\endcsname\relax
       \expandafter
       \gdef\csname the#1\expandafter\endcsname\expandafter
              {\expandafter\@arabic\csname c@#1\endcsname}%
     \else
       \@latex@warning{Command `\string\the#1' already
                       defined -- not changed}%
     \fi}
⟨latexrelease⟩\EndIncludeInRelease
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2024/11/01}{\@definecounter}
⟨latexrelease⟩                                {provide theHfoo commands}%
⟨latexrelease⟩\def\@definecounter#1{\expandafter\newcount\csname c@#1\endcsname
⟨latexrelease⟩     \setcounter{#1}\z@
⟨latexrelease⟩     \global\expandafter\let\csname cl@#1\endcsname\@empty
⟨latexrelease⟩     \@addtoreset{#1}{@ckpt}%
⟨latexrelease⟩     \global\expandafter\let\csname p@#1\endcsname\@empty
⟨latexrelease⟩     \expandafter\xdef\csname theH#1\endcsname{%
⟨latexrelease⟩       \noexpand\the\noexpand\value{#1}}%
```

If `\the#1` is undefined or `\relax` we define it with the standard definition for counters, otherwise we warn. This will catch, for example, that somebody defines a counter named "index" conflicting with the `theindex` environment.

```
⟨latexrelease⟩     \expandafter
⟨latexrelease⟩     \ifx\csname the#1\endcsname\relax
⟨latexrelease⟩       \expandafter
⟨latexrelease⟩       \gdef\csname the#1\expandafter\endcsname\expandafter
⟨latexrelease⟩              {\expandafter\@arabic\csname c@#1\endcsname}%
⟨latexrelease⟩     \else
⟨latexrelease⟩       \@latex@warning{Command `\string\the#1' already
⟨latexrelease⟩                       defined -- not changed}%
⟨latexrelease⟩     \fi}
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\@definecounter}
⟨latexrelease⟩                              {provide theHfoo commands}%%
⟨latexrelease⟩\def\@definecounter#1{\expandafter\newcount\csname c@#1\endcsname
⟨latexrelease⟩      \setcounter{#1}\z@
⟨latexrelease⟩      \global\expandafter\let\csname cl@#1\endcsname\@empty
⟨latexrelease⟩      \@addtoreset{#1}{@ckpt}%
⟨latexrelease⟩      \global\expandafter\let\csname p@#1\endcsname\@empty
⟨latexrelease⟩      \expandafter
⟨latexrelease⟩      \ifx\csname the#1\endcsname\relax
⟨latexrelease⟩        \expandafter
⟨latexrelease⟩      \gdef\csname the#1\expandafter\endcsname\expandafter
⟨latexrelease⟩           {\expandafter\@arabic\csname c@#1\endcsname}%
⟨latexrelease⟩      \else
⟨latexrelease⟩        \@latex@warning{Command `\string\the#1' already
⟨latexrelease⟩                        defined -- not changed}%
⟨latexrelease⟩      \fi}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@definecounter`.)

`\@kernel@get@ctr@root` A helper command to retrieve the root counter for messages.

```
\def\@kernel@get@ctr@root#1{%
 \@ifundefined{alias@ctr@#1}{#1}
   {\expandafter\@kernel@get@ctr@root\csname alias@ctr@#1\endcsname}}
```

(*End of definition for* `\@kernel@get@ctr@root`.)

`\@addtoreset` If a counter is reset when a parent counter changes it no longer has an unique value across the document. As `\theH<counter>` should be unique this representation is changed to include also the representation of the parent. This is not 100% guaranteed to work but has been used this way by hyperref for many years without causing problems. Changing `\cl@counter` and `\theHcounter` isn't trivial to do if the counter is an alias counter: e.g. only redefining `\theH`⟨*counter*⟩ from the alias counter breaks the connection to the theH-representation of the root counter. Redefining both `\theH`-representations could overwrite changes made by the user. If there is a chain of alias counters it gets even more complicated to keep everything in sync. So unlike the package aliascnt we avoid these complications and do not allow alias counters as arguments to `\@addtoreset` and `\@removefromreset`.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2026/06/01}{\@addtoreset}
⟨latexrelease⟩                              {support alias counter}%
⟨*2ekernel | latexrelease⟩
\def\@addtoreset#1#2{%
  \@ifundefined{alias@ctr@#1}{%
   \@ifundefined{alias@ctr@#2}
     {\expandafter\@cons\csname cl@#2\endcsname {{#1}}%
      \expandafter\xdef\csname theH#1\endcsname{%
       \expandafter\noexpand\csname theH#2\endcsname.%
        \noexpand\the\noexpand\value{#1}}}%
     {\@latex@warning{Alias counters can not be used in a counter reset.\MessageBreak
       Use '\@kernel@get@ctr@root{#2}' instead of '#2'.}}}
   {\@latex@warning{Alias counters can not be used in a counter reset.\MessageBreak
     Use '\@kernel@get@ctr@root{#1}' instead of '#1'.}}}
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2024/11/01}{\@addtoreset}
⟨latexrelease⟩                                {provide theHfoo commands}%
⟨latexrelease⟩\def\@addtoreset#1#2{\expandafter\@cons\csname cl@#2\endcsname {{#1}}%
⟨latexrelease⟩  \expandafter\xdef\csname theH#1\endcsname{%
⟨latexrelease⟩    \expandafter\noexpand\csname theH#2\endcsname.%
⟨latexrelease⟩    \noexpand\the\noexpand\value{#1}}%
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\@addtoreset}
⟨latexrelease⟩                                {provide theHfoo commands}%%
⟨latexrelease⟩\def\@addtoreset#1#2{\expandafter\@cons\csname cl@#2\endcsname {{#1}}}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@addtoreset`.)

```
⟨/2ekernel⟩
```

`\@removefromreset`

```
⟨latexrelease⟩\IncludeInRelease{2026/06/01}{\@addtoreset}
⟨latexrelease⟩                                {support alias counter}%
⟨*2ekernel | latexrelease⟩
\def\@removefromreset#1#2{%
```

Even through this is internal and the programmer should know what he/she is doing we test here if counter `#2` is defined. If not, the execution would run into a tight loop. As in `\@addtoreset` do not allow alias counters as arguments.

```
  \@ifundefined{c@#2}\relax
  {\@ifundefined{alias@ctr@#1}{%
     \@ifundefined{alias@ctr@#2}{%
       \begingroup
         \expandafter\let\csname c@#1\endcsname\@removefromreset
         \def\@elt##1{%
           \expandafter\ifx\csname c@##1\endcsname\@removefromreset
           \else
             \noexpand\@elt{##1}%
           \fi}%
         \expandafter\xdef\csname cl@#2\endcsname
          {\csname cl@#2\endcsname}%
        \endgroup}%
       {\@latex@warning{Alias counters can not be removed from a reset.\MessageBreak
       Use '\@kernel@get@ctr@root{#2}' instead of '#2'.}}}
     {\@latex@warning{Alias counters can not be removed from a reset.\MessageBreak
       Use '\@kernel@get@ctr@root{#1}' instead of '#1'.}}}}
⟨latexrelease⟩\EndIncludeInRelease
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2018-04-01}
⟨latexrelease⟩                  {\@removefromreset}{Add interfaces}%
⟨latexrelease⟩\def\@removefromreset#1#2{%
```

Even through this is internal and the programmer should know what he/she is doing we test here if counter `#2` is defined. If not, the execution would run into a tight loop.

```
⟨latexrelease⟩  \@ifundefined{c@#2}\relax
```

```
⟨latexrelease⟩  {\begingroup
⟨latexrelease⟩     \expandafter\let\csname c@#1\endcsname\@removefromreset
⟨latexrelease⟩     \def\@elt##1{%
⟨latexrelease⟩       \expandafter\ifx\csname c@##1\endcsname\@removefromreset
⟨latexrelease⟩       \else
⟨latexrelease⟩         \noexpand\@elt{##1}%
⟨latexrelease⟩       \fi}%
⟨latexrelease⟩     \expandafter\xdef\csname cl@#2\endcsname
⟨latexrelease⟩      {\csname cl@#2\endcsname}%
⟨latexrelease⟩    \endgroup}}
```

(*End of definition for* `\@removefromreset`.)

`\@ifbothcounters` Test if arg #1 and #2 are counters and if so execute #3.

```
⟨*2ekernel | latexrelease⟩
\begingroup\catcode`\*=11 \lowercase{\endgroup
\def\@ifbothcounters#1#2#3{%
  \begingroup\let\c@*\@undefined
  \@ifundefined{c@#1}{\@nocounterr{#1}}%
    {% else counter is defined
     \@ifundefined{c@#2}{\@nocounterr{#2}}%
       {% else both counter and within  are defined
         #3}}%
   \endgroup}}
```

(*End of definition for* `\@ifbothcounters`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000-00-00}
⟨latexrelease⟩                  {\@removefromreset}{Add interfaces}%
⟨latexrelease⟩\let \@removefromreset \undefined
⟨latexrelease⟩\let \@ifbothcounters  \undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

`\counterwithin` New implementation using xparse and supporting an optional format argument.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease⟩                  {\counterwithin}{counter within}%
\NewDocumentCommand \counterwithin {sO{\arabic}mm}{%
  \@ifbothcounters{#3}{#4}{%
    \@addtoreset{#3}{#4}%
    \IfBooleanF #1%
       {\expandafter
         \xdef\csname the#3\expandafter\endcsname
            {\expandafter\noexpand\csname the#4\endcsname
             .\unexpanded{#2}{#3}}}%
  }%
}
```

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/11/15}%
⟨latexrelease⟩                  {\counterwithin}{counter within}%
⟨latexrelease⟩
⟨latexrelease⟩\NewDocumentCommand \counterwithin {sO{\arabic}mm}{%
⟨latexrelease⟩  \@ifbothcounters{#3}{#4}{%
⟨latexrelease⟩    \@addtoreset{#3}{#4}%
⟨latexrelease⟩    \IfBooleanF #1%
⟨latexrelease⟩       {\expandafter
⟨latexrelease⟩         \gdef\csname the#3\expandafter\endcsname
⟨latexrelease⟩         \expandafter
⟨latexrelease⟩             {\csname the#4\endcsname .#2{#3}}}%
⟨latexrelease⟩  }%
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2018-04-01}
⟨latexrelease⟩                  {\counterwithin}{counter within}%
⟨latexrelease⟩
⟨latexrelease⟩\def\counterwithin{\@ifstar\counterwithin@s\counterwithin@x}
⟨latexrelease⟩\def\counterwithin@s#1#2{%
⟨latexrelease⟩  \@ifbothcounters{#1}{#2}{\@addtoreset{#1}{#2}}}

⟨latexrelease⟩\def\counterwithin@x#1#2{%
⟨latexrelease⟩  \@ifbothcounters{#1}{#2}%
⟨latexrelease⟩      {\@addtoreset{#1}{#2}%
⟨latexrelease⟩       \expandafter
⟨latexrelease⟩       \gdef\csname the#1\expandafter\endcsname\expandafter
⟨latexrelease⟩            {\csname the#2\expandafter\endcsname\expandafter
⟨latexrelease⟩             .\expandafter
⟨latexrelease⟩             \@arabic\csname c@#1\endcsname}}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000-00-00}
⟨latexrelease⟩                  {\counterwithin}{counter within}%
⟨latexrelease⟩\let \counterwithin    \undefined
⟨latexrelease⟩\let \counterwithin@s  \undefined
⟨latexrelease⟩\let \counterwithin@x  \undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\counterwithin`.)

`\counterwithout` New implementation using xparse and supporting an optional format argument.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease⟩                  {\counterwithout}{counter without}%

\NewDocumentCommand \counterwithout {sO{\arabic}mm}{%
  \@ifbothcounters{#3}{#4}{%
    \@removefromreset{#3}{#4}%
    \IfBooleanF #1%
        {\expandafter
         \xdef\csname the#3\endcsname {\unexpanded{#2}{#3}}}%
```

```
  }%
}

⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2021/11/15}%
⟨latexrelease⟩                  {\counterwithout}{counter without}%
⟨latexrelease⟩\NewDocumentCommand \counterwithout {sO{\arabic}mm}{%
⟨latexrelease⟩  \@ifbothcounters{#3}{#4}{%
⟨latexrelease⟩    \@removefromreset{#3}{#4}%
⟨latexrelease⟩    \IfBooleanF #1%
⟨latexrelease⟩        {\expandafter
⟨latexrelease⟩         \gdef\csname the#3\endcsname {#2{#3}}}%
⟨latexrelease⟩  }%
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2018-04-01}
⟨latexrelease⟩                  {\counterwithout}{counter without}%
⟨latexrelease⟩
⟨latexrelease⟩\def\counterwithout {\@ifstar\counterwithout@s\counterwithout@x}
⟨latexrelease⟩\def\counterwithout@s#1#2{%
⟨latexrelease⟩  \@ifbothcounters{#1}{#2}{\@removefromreset{#1}{#2}}}
⟨latexrelease⟩\def\counterwithout@x#1#2{%
⟨latexrelease⟩  \@ifbothcounters{#1}{#2}%
⟨latexrelease⟩      {\@removefromreset{#1}{#2}%
⟨latexrelease⟩       \expandafter
⟨latexrelease⟩       \gdef\csname the#1\expandafter\endcsname\expandafter
⟨latexrelease⟩            {\expandafter
⟨latexrelease⟩             \@arabic\csname c@#1\endcsname}}}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{0000-00-00}
⟨latexrelease⟩                  {\counterwithout}{counter without}%
⟨latexrelease⟩\let \counterwithout   \undefined
⟨latexrelease⟩\let \counterwithout@s \undefined
⟨latexrelease⟩\let \counterwithout@x \undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\counterwithout`.)

Numbering commands for definitions of `\theCOUNTER` and `\list` arguments. All commands can now be used in text and math mode.

`\arabic` Representation of ⟨*counter*⟩ as arabic numerals. Changed 29 Apr 86 to make it print the obvious thing it COUNTER not positive.

```
\def\arabic#1{\expandafter\@arabic\csname c@#1\endcsname}
```

(*End of definition for* `\arabic`.)

`\roman` Representation of ⟨*counter*⟩ as lower-case Roman numerals.

```
\def\roman#1{\expandafter\@roman\csname c@#1\endcsname}
```

(*End of definition for* `\roman`.)

`\Roman` Representation of ⟨*counter*⟩ as upper-case Roman numerals.

```
\def\Roman#1{\expandafter\@Roman\csname c@#1\endcsname}
```

(*End of definition for* `\Roman`.)

`\alph` Representation of ⟨*counter*⟩ as a lower-case letter: 1 = a, 2 = b, etc.

```
\def\alph#1{\expandafter\@alph\csname c@#1\endcsname}
```

(*End of definition for* `\alph`.)

`\Alph` Representation of ⟨*counter*⟩ as an upper-case letter: 1 = A, 2 = B, etc.

```
\def\Alph#1{\expandafter\@Alph\csname c@#1\endcsname}
```

(*End of definition for* `\Alph`.)

`\fnsymbol` Representation of ⟨*COUNTER*⟩ as a footnote symbol: 1 = ∗, 2 = †, etc.

```
\def\fnsymbol#1{\expandafter\@fnsymbol\csname c@#1\endcsname}
```

(*End of definition for* `\fnsymbol`.)

`\@arabic` `\@arabic\FOOcounter` Representation of `\FOOcounter` as arabic numerals.

```
\def\@arabic#1{\number #1}  %% changed 29 Apr 86
```

(*End of definition for* `\@arabic`.)

`\@roman` `\@roman\FOOcounter` Representation of `\FOOcounter` as lower-case Roman numerals.

```
\def\@roman#1{\romannumeral #1}
```

(*End of definition for* `\@roman`.)

`\@Roman` `\@Roman\FOOcounter` Representation of `\FOOcounter` as upper-case Roman numerals.

```
\def\@Roman#1{\expandafter\@slowromancap\romannumeral #1@}
```

(*End of definition for* `\@Roman`.)

`\@slowromancap` Fully expandable macro to change a roman number to uppercase.

```
\def\@slowromancap#1{\ifx @#1% then terminate
     \else
       \if i#1I\else\if v#1V\else\if x#1X\else\if l#1L\else\if
       c#1C\else\if d#1D\else \if m#1M\else#1\fi\fi\fi\fi\fi\fi\fi
       \expandafter\@slowromancap
     \fi
}
```

(*End of definition for* `\@slowromancap`.)

`\@alph` `\@alph\FOOcounter` Representation of `\FOOcounter` as a lower-case letter: 1 = a, 2 = b, etc.

```
\def\@alph#1{%
  \ifcase#1\or a\or b\or c\or d\or e\or f\or g\or h\or i\or j\or
   k\or l\or m\or n\or o\or p\or q\or r\or s\or t\or u\or v\or w\or x\or
    y\or z\else\@ctrerr\fi}
```

(*End of definition for* `\@alph`.)

`\@Alph` `\@Alph\FOOcounter` Representation of `\FOOcounter` as an upper-case letter: 1 = A, 2 = B, etc.

```
\def\@Alph#1{%
  \ifcase#1\or A\or B\or C\or D\or E\or F\or G\or H\or I\or J\or
   K\or L\or M\or N\or O\or P\or Q\or R\or S\or T\or U\or V\or W\or X\or
    Y\or Z\else\@ctrerr\fi}
```

(*End of definition for* `\@Alph`.)

`\@fnsymbol` Typesetting old fashioned footnote symbols. This can be done both in text or math mode now.

This macro is another example of an ever recurring problem in TEX: Determining if something is text-mode or math-mode. It is imperative for the decision between text and math to be delayed until the actual typesetting is done as the code in question may go through an `\edef` or `\write` where an `\ifmmode` test would be executed prematurely. Hence in the implementation below, `\@fnsymbol` is not robust in itself but the parts doing the actual typesetting are.

In the case of `\@fnsymbol` we make use of the robust command `\TextOrMath` which takes two arguments and typesets the first if in text-mode and the second if in math-mode. Note that in order for this command to make the correct decision, it must insert a `\relax` token if run under regular TEX, which ruins any kerning between the preceding characters and whatever awaits typesetting. If you use eTEX as engine for LATEX (as recommended) this unfortunate side effect is not present.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}{\@fnsymbol}{Use \TextOrMath}%
⟨*2ekernel | latexrelease⟩
\def\@fnsymbol#1{%
   \ifcase#1\or \TextOrMath\textasteriskcentered *\or
   \TextOrMath \textdagger \dagger\or
   \TextOrMath \textdaggerdbl \ddagger \or
   \TextOrMath \textsection  \mathsection\or
   \TextOrMath \textparagraph \mathparagraph\or
   \TextOrMath \textbardbl \|\or
   \TextOrMath {\textasteriskcentered\textasteriskcentered}{**}\or
   \TextOrMath {\textdagger\textdagger}{\dagger\dagger}\or
   \TextOrMath {\textdaggerdbl\textdaggerdbl}{\ddagger\ddagger}\else
   \@ctrerr \fi
}%
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\@fnsymbol}{Use \TextOrMath}%
⟨latexrelease⟩\def\@fnsymbol#1{\ensuremath{%
⟨latexrelease⟩    \ifcase#1\or *\or \dagger\or \ddagger\or \mathsection\or
⟨latexrelease⟩      \mathparagraph\or \|\or **\or \dagger\dagger
⟨latexrelease⟩      \or \ddagger\ddagger \else\@ctrerr\fi}}%
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@fnsymbol`.)

`\TextOrMath` When using regular TEX, we make this command robust so that it always selects the correct branch in an `\ifmmode` switch with the usual disadvantage of ruining kerning. For the application we use it for here that shouldn't matter. The alternative would be to

mimic `\IeC` from inputenc but then it will have the disadvantage of choosing the wrong branch if appearing at the beginning of an alignment cell. However, users of eTEX will be pleasantly surprised to get the best of both worlds and no bad side effects.

First some code for checking if we are running eTEX but making sure not to permanently turn `\protected` into `\relax`.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}{\TextOrMath}{\TextOrMath}%
⟨*2ekernel | latexrelease⟩
\begingroup\expandafter\expandafter\expandafter\endgroup
\expandafter\ifx\csname protected\endcsname\relax
```

In case of ordinary TEX we define `\TextOrMath` as a robust command but make sure it always grabs its arguments. If we didn't do this it might very well gobble spaces in the input stream.

```
\DeclareRobustCommand\TextOrMath{%
  \ifmmode  \expandafter\@secondoftwo
  \else     \expandafter\@firstoftwo  \fi}
\protected@edef\TextOrMath#1#2{\TextOrMath{#1}{#2}}
\else
```

For eTEX the situation is similar. The robust macro is a hidden one so that we again avoid problems of gobbling spaces in the input.

```
\protected\expandafter\def\csname TextOrMath\space\endcsname{%
  \ifmmode  \expandafter\@secondoftwo
  \else     \expandafter\@firstoftwo  \fi}
\edef\TextOrMath#1#2{%
  \expandafter\noexpand\csname TextOrMath\space\endcsname
  {#1}{#2}}
\fi
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\TextOrMath}{\TextOrMath}%
⟨latexrelease⟩\let\TextOrMath\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\TextOrMath`.)

```
⟨/2ekernel⟩
```

File 23
ltlength.dtx

1 Lengths

\newlength Declare #1 to be a new length command.
\setlength Set the length command, #1, to the value #2.
\addtolength Increase the value of the length command, #1, by the value #2.
\settowidth Set the length, #1 to the width of a box containing #2.
\settoheight Set the length, #1 to the height of a box containing #2.
\settodepth Set the length, #1 to the depth of a box containing #2.

```
⟨*2ekernel⟩
\message{lengths,}
```

\newlength

```
\def\newlength#1{\@ifdefinable#1{\newskip#1}}
```

(*End of definition for* \newlength.)

\setlength

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩                    {\setlength}{Using \setlength with \dimen0}%
⟨*2ekernel | latexrelease⟩
\def\setlength#1#2{#1 #2\relax}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\setlength}{Using \setlength with \dimen0}%
⟨latexrelease⟩\def\setlength#1#2{#1#2\relax}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* \setlength.)

\addtolength \relax added 24 Mar 86

```
\def\addtolength#1#2{\advance#1 #2\relax}
```

(*End of definition for* \addtolength.)

\settoheight The obvious analogs of \settowidth.
\settodepth
\settowidth
\@settodim

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2024/11/01}%
⟨latexrelease⟩                    {\@settodim}{suspend tagging}%
\def\@settodim#1#2#3{\setbox\@tempboxa\hbox
  {{\SuspendTagging{\@settodim}#3\ResumeTagging{\@settodim}}}#2#1\@tempboxa
```

Clear the memory afterwards (which might be a lot).

```
        \setbox\@tempboxa\box\voidb@x}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\@settodim}{suspend tagging}%
⟨latexrelease⟩\def\@settodim#1#2#3{\setbox\@tempboxa\hbox{{#3}}#2#1\@tempboxa
⟨latexrelease⟩        \setbox\@tempboxa\box\voidb@x}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
\DeclareRobustCommand\settoheight{\@settodim\ht}
\DeclareRobustCommand\settodepth {\@settodim\dp}
\DeclareRobustCommand\settowidth {\@settodim\wd}
```

(*End of definition for* `\settoheight` *and others.*)

`\@settopoint` This macro takes the contents of the skip register that is supplied as its argument and removes the fractional part to make it a whole number of points. This can be used in class files to avoid values like `345.4666666pt` when calculating a dimension.

```
\def\@settopoint#1{\divide#1\p@\multiply#1\p@}
⟨/2ekernel⟩
```

(*End of definition for* `\@settopoint`.)

File 24
ltfssbas.dtx

This file contains the main implementation of the 'low level' font selection commands. See other parts of the LaTeX distribution, or *The LaTeX Companion* for higher level documentation of the LaTeX 'New' Font Selection Scheme.

> **Warning:** The macro documentation is still basically the documentation from the first NFSS release and therefore in some cases probably not completely accurate.

1 Preliminary macros

We define a number of macros that will be used later.

```
⟨*2ekernel⟩
\message{NFSS base,}
```

`\@nomath` `\@nomath` is used by most macros that will have no effect in math mode. It issues a warning message.

```
\def\@nomath#1{\relax\ifmmode
   \@font@warning{Command \noexpand#1invalid in math mode}\fi}
```

(*End of definition for* `\@nomath`.)

`\no@alphabet@error` The macro `\no@alphabet@error` is called whenever the user requests a math *alphabet* that is not available in the current *version*. In math mode an error message is produced otherwise the command keeps silent. The argument is the name of the control sequence that identifies the math *alphabet*. The `\relax` at the beginning is necessary to prevent TeX from scanning too far in certain situations.

```
\gdef\no@alphabet@error#1{\relax \ifmmode
    \@latex@error{Math\space alphabet\space identifier\space
          \noexpand#1is\space undefined\space in\space math\space
           version\space `\math@version'}%
         {Your\space requested\space math\space alphabet\space
          is\space undefined\space in\space the\space current\space
           math\space version.^^JCheck\space the\space spelling\space
           or\space use\space the\space \noexpand\SetMathAlphabet\space
           command.}
     \fi}
```

(*End of definition for* `\no@alphabet@error`.)

`\new@mathgroup` `\mathgroup` We also give a new name to `\newfam` and `\fam` to avoid verbal confusion (see the introduction).[32]

```
%\def\new@mathgroup{\alloc@8\mathgroup\chardef\sixt@@n}
\let\mathgroup\fam
%\let\newfam\new@mathgroup
\@onlypreamble\new@mathgroup
```

(*End of definition for* `\new@mathgroup` *and* `\mathgroup`.)

[32] For the same reason it seems advisable to `\let\fam` and `\newfam` equal to `\relax`, but this is commented out to retain compatibility to existing style files.

2 Macros for setting up the tables

\DeclareFontShape

The macro \DeclareFontShape takes 6 arguments:

```
\def\DeclareFontShape{\begingroup
```

First we restore the catcodes of all characters used in the syntax.

```
    \nfss@catcodes
```

We use \expandafter \endgroup to restore catcode in case something goes wrong with the argument parsing (suggested by Tim Van Zandt)

```
    \expandafter\endgroup
    \DeclareFontShape@}
```

(*End of definition for* \DeclareFontShape.)

\DeclareFontShape@

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩                    {\DeclareFontShape@}{Maybe drop one m}%
\def\DeclareFontShape@#1#2#3#4#5#6{%
   \expandafter\ifx\csname #1+#2\endcsname\relax
     \@latex@error{Font family `#1+#2' unknown}\@eha
   \else
```

If the series value is incorrectly specified with an extra "m", e.g., "mc" instead of just "c", drop the surplus "m" but keep the "m" if it is by its own. In that case also issue a warning that the declaration needs correction.

For this we compare the given value #3 with one where we may have dropped an "m". If nothing has changes, fine. Otherwise there was a wrong value which is now corrected in \reserved@b so we use that and also issue a warning.

```
      \edef\reserved@a{#3}%
      \series@maybe@drop@one@m\reserved@a\reserved@b
      \ifx\reserved@a\reserved@b\else
        \@latex@note{Font shape #1/#2/#3/#4 has incorrect series
          value `#3'.\MessageBreak It should not contain an `m'!
          Please correct it.\MessageBreak Found}%
      \fi
      \expandafter
        \xdef\csname#1/#2/\reserved@b/#4\endcsname
                  {\expandafter\noexpand\csname #5\endcsname}%
%
```

Most of the time #6 is empty so using \let to \@empty saves on space compared to using \def. That's really one of the old space saving techniques and probably not necessary these days.

```
      \def\reserved@a{#6}%
      \global
      \expandafter\let\csname#5\expandafter\endcsname
         \ifx\reserved@a\@empty
           \@empty
         \else
           \reserved@a
         \fi
   \fi
  }
```

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\DeclareFontShape@}{Maybe drop one m}%
⟨latexrelease⟩
⟨latexrelease⟩\def\DeclareFontShape@#1#2#3#4#5#6{%
⟨latexrelease⟩   \expandafter\ifx\csname #1+#2\endcsname\relax
⟨latexrelease⟩     \@latex@error{Font family `#1+#2' unknown}\@eha
⟨latexrelease⟩   \else
⟨latexrelease⟩     \expandafter
⟨latexrelease⟩       \xdef\csname#1/#2/#3/#4\endcsname{\expandafter\noexpand
⟨latexrelease⟩                                       \csname #5\endcsname}%
⟨latexrelease⟩     \def\reserved@a{#6}%
⟨latexrelease⟩     \global
⟨latexrelease⟩     \expandafter\let\csname#5\expandafter\endcsname
⟨latexrelease⟩        \ifx\reserved@a\@empty
⟨latexrelease⟩          \@empty
⟨latexrelease⟩        \else
⟨latexrelease⟩          \reserved@a
⟨latexrelease⟩        \fi
⟨latexrelease⟩   \fi
⟨latexrelease⟩  }
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\DeclareFontShape@`.)

`\DeclareFixedFont` Define a direct font switch that avoids all overhead.

```
\def\DeclareFixedFont#1#2#3#4#5#6{%
   \begingroup
      \math@fontsfalse
      \every@math@size{}%
      \fontsize{#6}\z@
      \usefont{#2}{#3}{#4}{#5}%
      \global\expandafter\let\expandafter#1\the\font
   \endgroup
  }
```

(*End of definition for* `\DeclareFixedFont`.)

`\do@subst@correction`

```
\def\do@subst@correction{%
       \xdef\subst@correction{%
          \font@name
          \global\expandafter\font
            \csname \curr@fontshape/\f@size\endcsname
            \noexpand\fontname\font
           \relax}%
```

Calling `\subst@correction` after the current group means calling it after we have loaded the substitution font which is done inside a group.

```
       \aftergroup\subst@correction
}
```

(*End of definition for* `\do@subst@correction`.)

\DeclareFontFamily

```
\def\DeclareFontFamily#1#2#3{%
```

If we want fast checking for the encoding scheme we can just check for `\T@..` being defined.

```
% \@tempswafalse
% \def\reserved@b{#1}%
% \def\cdp@elt##1##2##3##4{\def\reserved@c{##1}%
%      \ifx\reserved@b\reserved@c \@tempswatrue\fi}%
% \cdp@list
% \if@tempswa
 \@ifundefined{T@#1}%
    {%
     \@latex@error{Encoding scheme  `#1' unknown}\@eha
    }%
    {%
```

Now we have to define the macro \⟨*#1*⟩+⟨*#2*⟩ to contain `#3`. But since most of the time `#3` will be empty we use `\let` in a tricky way rather than a simple `\def` since this will save internal memory. We store the argument `#3` in a temporary macro `\reserved@a`.

```
      \def\reserved@a{#3}%
```

We compare `\reserved@a` with `\@empty` If these two are the same we `\let` the 'extra' macro equal to `\@empty` which is not the same a doing a `\let` to `\reserved@a` — the latter would blow one extra memory location rather then reusing the one from `\@empty`.

```
      \global
      \expandafter\let\csname #1+#2\expandafter\endcsname
            \ifx \reserved@a\@empty
              \@empty
            \else \reserved@a
            \fi
    }%
}
```

(*End of definition for* `\DeclareFontFamily`.)

\cdp@list We initialize the code page list to be empty.

```
\let\cdp@list\@empty
\@onlypreamble\cdp@list
```

(*End of definition for* `\cdp@list`.)

\cdp@elt

```
\let\cdp@elt\relax
\@onlypreamble\cdp@elt
```

(*End of definition for* `\cdp@elt`.)

\DeclareFontEncoding

```
\def\DeclareFontEncoding{%
```

First we start with ignoring all blanks and newlines since every surplus space in the second or third argument will come out in a weird place in the document.

```
  \begingroup
  \nfss@catcodes
  \expandafter\endgroup
  \DeclareFontEncoding@}
\@onlypreamble\DeclareFontEncoding

\def\DeclareFontEncoding@#1#2#3{%
  \expandafter
  \ifx\csname T@#1\endcsname\relax
     \def\cdp@elt{\noexpand\cdp@elt}%
     \xdef\cdp@list{\cdp@list\cdp@elt{#1}%
                    {\default@family}{\default@series}%
                    {\default@shape}}%
```

To support encoding dependent commands (like accents) we initialize the command \⟨*encoding*⟩-cmd to be `\@changed@cmd`. (See `ltoutenc.dtx` for details.)

```
     \expandafter\let\csname#1-cmd\endcsname\@changed@cmd
  \else
     \@font@info{Redeclaring font encoding #1}%
  \fi

  \global\@namedef{T@#1}{#2}%
  \global\@namedef{M@#1}{\default@M#3}%
```

Keep a record of the last encoding being declared:

```
  \xdef\LastDeclaredEncoding{#1}%
  }
\@onlypreamble\DeclareFontEncoding@
```

(*End of definition for* `\DeclareFontEncoding`.)

`\LastDeclaredEncoding` The last encoding being declared by `\DeclareFontEncoding`.

```
\def\LastDeclaredEncoding{}
```

(*End of definition for* `\LastDeclaredEncoding`.)

`\DeclareFontSubstitution`

```
\def\DeclareFontSubstitution#1#2#3#4{%
  \expandafter
  \ifx\csname T@#1\endcsname\relax
    \@latex@error{Encoding scheme  `#1' unknown}\@eha
  \else
    \begingroup
```

We loop through the `\cdp@list` and rebuild it anew in `\toks@` thereby replacing the defaults for the encoding in question with the new defaults. It is important to store the encoding to test against expanded in `\reserved@a` since it might just be `\LastDeclaredEncoding` that is passed as `#1`.

```
       \edef\reserved@a{#1}%
       \toks@{}%
       \def\cdp@elt##1##2##3##4{%
          \def\reserved@b{##1}%
          \ifx\reserved@a\reserved@b
```

Here we use the new defaults but we use `##1` (i.e., the encoding name already stored previously) since we know that it is expanded.

```
              \addto@hook\toks@{\cdp@elt{##1}{#2}{#3}{#4}}%
            \else
```

If `\reserved@a` and `\reserved@b` differ then we simply copy from the old list to the new.

```
              \addto@hook\toks@{\cdp@elt{##1}{##2}{##3}{##4}}%
            \fi}%
          \cdp@list
          \xdef\cdp@list{\the\toks@}%
      \endgroup
      \global
      \@namedef{D@#1}{%
             \def\default@family{#2}%
             \def\default@series{#3}%
             \def\default@shape{#4}%
             }%
    \fi
  }
\@onlypreamble\DeclareFontSubstitution
```

(*End of definition for* `\DeclareFontSubstitution`.)

`\DeclareFontEncodingDefaults`

```
\def\DeclareFontEncodingDefaults#1#2{%
  \ifx\relax#1\else
    \ifx\default@T\@empty\else
      \@font@info{Overwriting encoding scheme text defaults}%
    \fi
    \gdef\default@T{#1}%
  \fi
  \ifx\relax#2\else
    \ifx\default@M\@empty\else
      \@font@info{Overwriting encoding scheme math defaults}%
    \fi
    \gdef\default@M{#2}%
  \fi
}
\@onlypreamble\DeclareFontEncodingDefaults
```

(*End of definition for* `\DeclareFontEncodingDefaults`.)

`\default@T`
`\default@M`

```
\let\default@T\@empty
\let\default@M\@empty
```

(*End of definition for* `\default@T` *and* `\default@M`.)

`\DeclareEncodingSubset` The declaration takes 3 mandatory arguments: an *encoding* for which a subsetting is wanted (currently always `TS1`, and most likely forever), the *font family* for which we declare the subset and finally the *subset* number, with a value between `0` (all of the encoding is supported) and `9` (many glyphs are missing).

For `TS1` the numbers have been chosen in a way that most fonts can be fairly correctly categorized, but the default settings are always conservative, that is they may claim that fewer glyphs are supported than there actually are.

As these days many font families are set up to end in `-LF` (lining figures), `-OsF` (oldstyle figures), etc. the declaration supports a shortcut: if the *font family* name ends in `-*` then the star gets replaced by these common ending, e.g.,

```
\DeclareEncodingSubset{TS1}{Alegreya-*}{2}
```

is the same as writing

```
\DeclareEncodingSubset{TS1}{Alegreya-LF}{2}
\DeclareEncodingSubset{TS1}{Alegreya-OsF}{2}
\DeclareEncodingSubset{TS1}{Alegreya-TLF}{2}
\DeclareEncodingSubset{TS1}{Alegreya-TOsF}{2}
```

If only some are needed then one can define them individually but in many cases all four are wanted, hence the shortcut.

The coding of the declaration has no error checking as it is mostly for internal use.

```
\def\DeclareEncodingSubset#1#2{%
  \DeclareEncodingSubset@aux{#1}#2*\DeclareEncodingSubset@aux
}
\def\DeclareEncodingSubset@aux#1#2*#3\DeclareEncodingSubset@aux#4{%
```

if `#3` is empty then there was no star, otherwise we define all four variants.

```
  \expandafter\ifx\expandafter X\detokenize{#3}X%
    \@DeclareEncodingSubset{#1}{#2}{#4}%
  \else
    \@DeclareEncodingSubset{#1}{#2LF}{#4}%
    \@DeclareEncodingSubset{#1}{#2TLF}{#4}%
    \@DeclareEncodingSubset{#1}{#2OsF}{#4}%
    \@DeclareEncodingSubset{#1}{#2TOsF}{#4}%
  \fi
}
```

The subset info is stored in a command with the name *family*`:`*subset* so if that already exists we change otherwise declare a subset.

```
\def\@DeclareEncodingSubset#1#2#3{%
   \@ifundefined{#1:#2}%
     {\@font@info{Setting #2 sub-encoding to #1/#3}}%
     {\@font@info{Changing #2 sub-encoding to #1/#3}}%
```

This declaration should be usable in `.fd` files and therefore has to make its definition globally, because such files can get loaded in random places.

```
     \global\@namedef{#1:#2}{#3}}
```

(*End of definition for* `\DeclareEncodingSubset`.)

`\CheckEncodingSubset` The command `\CheckEncodingSubset` will check if the current font family has the right encoding subset to typeset a certain command. It takes five arguments as follows: first argument is either `\UseTextSymbol`, `\UseTextAccent` depending on whether or not the symbol is a text symbol or a text accent.

The second argument is the encoding from which this symbol should be fetched.

The third argument is either a fake accessor command or an error message. the code in that argument (if ever executed) receives two arguments: `#2` and `#5` of `\CheckEncodingSubset`.

Argument four is the subset encoding id to test against: if this value is higher than the subset id of the current font family then we typeset the symbol, i.e., execute `#1{#2}#5` otherwise it runs `#3#5`, e.g., to produce an error message or fake the glyph somehow.

Argument five is the symbol or accent command that is being checked.

For usage examples see definitions in `lttextcomp.dtx`.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease⟩                    {\CheckEncodingSubset}{preload .fd file}%
\def\CheckEncodingSubset#1#2#3#4#5{%
```

If the `.fd` is not yet loaded we make an attempt to load it now because it may contain a `\DeclareEncodingSubset` declaration for the font.

```
   \expandafter\ifx\csname #2:\f@family\endcsname\relax
      \LoadFontDefinitionFile{#2}\f@family
   \fi
   \ifnum #4>%
      \expandafter\ifx\csname #2:\f@family\endcsname\relax
        0\csname #2:?\endcsname
      \else
        \csname #2:\f@family\endcsname
      \fi
   \relax
   \expandafter\@firstoftwo
  \else
   \expandafter\@secondoftwo
 \fi
  {#1{#2}}{#3}%
  #5%
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\CheckEncodingSubset}{preload .fd file}%
⟨latexrelease⟩
⟨latexrelease⟩\def\CheckEncodingSubset#1#2#3#4#5{%
⟨latexrelease⟩    \ifnum #4>%
⟨latexrelease⟩      \expandafter\ifx\csname #2:\f@family\endcsname\relax
⟨latexrelease⟩        0\csname #2:?\endcsname
⟨latexrelease⟩      \else
⟨latexrelease⟩        \csname #2:\f@family\endcsname
⟨latexrelease⟩      \fi
⟨latexrelease⟩   \relax
⟨latexrelease⟩   \expandafter\@firstoftwo
⟨latexrelease⟩  \else
⟨latexrelease⟩   \expandafter\@secondoftwo
⟨latexrelease⟩ \fi
⟨latexrelease⟩  {#1{#2}}{#3}%
⟨latexrelease⟩  #5%
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\CheckEncodingSubset`.)

\DeclarePreloadSizes

```
\def\DeclarePreloadSizes#1#2#3#4#5{%
 \@ifundefined{T@#1}%
   {\@latex@error{Encoding scheme  `#1' unknown}\@eha}%
   {%
```

Don't know at the moment what this group here does!

```
    \begingroup
```

We define a macro `\reserved@f`[33] that grabs the next *size* and loads the corresponding font. This is done by delimiting `\reserved@f`'s only argument by the `token ,` (comma).

```
     \def\reserved@f##1,{%
```

The end of the list will be detected when there are no more elements, i.e. when `\reserved@f`'s argument is empty. The trick used here is explained in Appendix D of the TEXbook: if the argument is empty the `\if` will select the first clause and `\let \reserved@f` equal to `\relax`. (We use the `>` character here since it cannot appear in font file names.)

```
        \if>##1>%
          \let\reserved@f\relax
        \else
```

Otherwise, we define `\font@name` appropriately and call `\pickup@font` to do the work. Note that the requested `\curr@fontshape` combination must have been defined, or you will get an error. The definition of `\font@name` is carried out globally to be consistent with the rest of the code in this file.

```
          \xdef\font@name{\csname#1/#2/#3/#4/##1\endcsname}%
          \pickup@font
```

Now we forget the name of the font just loaded. More precisely, we set the corresponding control sequence to `\relax`. This means that later on, when the font is first used, the macro `\define@newfont` is called again to execute the 'extra' macro for this font.

```
          \global\expandafter\let\font@name\relax
        \fi
```

Finally we call `\reserved@f` again to process the next *size*. If `\reserved@f` was `\let` equal to `\relax` this will end the macro.

```
        \reserved@f}%
```

We finish with reinserting the list of sizes after the `\reserved@f` macro and appending an empty element so that the end of the list is recognized properly.

```
     \reserved@f#5,,%
   \endgroup
   }%
}
\@onlypreamble\DeclarePreloadSizes
```

(*End of definition for* `\DeclarePreloadSizes`.)

`\ifmath@fonts` We need a switch to decide if we have to switch math fonts. For this purpose we provide `\ifmath@fonts` that can be set to true or false by the `\S@...` macros depending on if math fonts are provided for this size or not. The default is of course to switch all fonts.

```
\newif\ifmath@fonts \math@fontstrue
```

[33] We cannot use `\@tempa` since it is needed in `\pickup@font`.

(*End of definition for* `\ifmath@fonts`.)

`\DeclareMathSizes` `\DeclareMathSizes*`

`\DeclareMathSizes` takes the text size, math text size, math script size, and math scriptscript size as arguments and defines the right `\S@...` macro.

```
\def\DeclareMathSizes{%
  \@ifstar{\@DeclareMathSizes\math@fontsfalse}%
          {\@DeclareMathSizes{}}}
\@onlypreamble\DeclareMathSizes
```

(*End of definition for* `\DeclareMathSizes` *and* `\DeclareMathSizes*`.)

`\@DeclareMathSizes`

This modification by Michael J. Downes on comp.text.tex on 2002/10/17 allows the user to have settings such as `\DeclareMathSizes{9.5dd}{9.5dd}{7.4dd}{6.6dd}`.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}{\@DeclareMathSizes}%
⟨latexrelease⟩                  {Arbitrary units in \DeclareMathSizes}%
⟨∗2ekernel | latexrelease⟩
\def\@DeclareMathSizes #1#2#3#4#5{%
  \@defaultunits\dimen@ #2pt\relax\@nnil
  \if $#3$%
    \expandafter\let\csname S@\strip@pt\dimen@\endcsname\math@fontsfalse
  \else
    \@defaultunits\dimen@ii #3pt\relax\@nnil
    \@defaultunits\@tempdima #4pt\relax\@nnil
    \@defaultunits\@tempdimb #5pt\relax\@nnil
    \toks@{#1}%
    \expandafter\xdef\csname S@\strip@pt\dimen@\endcsname{%
      \gdef\noexpand\tf@size{\strip@pt\dimen@ii}%
      \gdef\noexpand\sf@size{\strip@pt\@tempdima}%
      \gdef\noexpand\ssf@size{\strip@pt\@tempdimb}%
      \the\toks@
    }%
  \fi
}%
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\@DeclareMathSizes}%
⟨latexrelease⟩                  {Arbitrary units in \DeclareMathSizes}%
⟨latexrelease⟩\def\@DeclareMathSizes#1#2#3#4#5{%
⟨latexrelease⟩    \@defaultunits\dimen@#2pt\relax\@nnil
⟨latexrelease⟩    \if$#3$%
⟨latexrelease⟩      \expandafter \let
⟨latexrelease⟩        \csname S@\strip@pt\dimen@\endcsname
⟨latexrelease⟩        \math@fontsfalse
⟨latexrelease⟩    \else
⟨latexrelease⟩      \expandafter \gdef
⟨latexrelease⟩      \csname S@\strip@pt\dimen@\endcsname
⟨latexrelease⟩            {\gdef\tf@size{#3}\gdef\sf@size{#4}%
⟨latexrelease⟩                              \gdef\ssf@size{#5}%
⟨latexrelease⟩             #1%
⟨latexrelease⟩                              }%
⟨latexrelease⟩    \fi}%
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨*2ekernel⟩
\@onlypreamble\@DeclareMathSizes
```

(*End of definition for* `\@DeclareMathSizes`.)

3 Selecting a new font

3.1 Macros for the user

`\fontencoding` `\f@encoding`

As we said in the introduction a font is described by four parameters. We first define macros to specify the wanted *family*, *series*, or *shape*. These are simply recorded in internal macros `\f@family`, `\f@series`, and `\f@shape`, resp. We use `\edef`'s so that the arguments can also be macros.

```
\DeclareRobustCommand\fontencoding[1]{%
    \expandafter\ifx\csname T@#1\endcsname\relax
      \@latex@error{Encoding scheme `#1' unknown}\@eha
    \else
      \edef\f@encoding{#1}%
      \ifx\cf@encoding\f@encoding
```

If the new encoding is the same as the old encoding we have nothing to do. However, in case we had a sequence of several encoding changes without a `\selectfont` in-between we can save processing by making sure that `\enc@update` is `\relax`.

```
        \let\enc@update\relax
      \else
```

If current and new encoding differ we define the macro `\enc@update` to contain all updates necessary at `\selectfont` time.

```
        \let\enc@update\@@enc@update
      \fi
    \fi
}
```

(*End of definition for* `\fontencoding` *and* `\f@encoding`.)

`\@@enc@update`

```
\def\@@enc@update{%
```

When `\@@enc@update` is executed `\f@encoding` holds the encoding name for the new encoding and `\cf@encoding` the name of the last active encoding.

We start by setting the init command for encoding dependent macros to `\@changed@cmd`.

```
        \expandafter
        \let
          \csname\cf@encoding -cmd\endcsname
          \@changed@cmd
```

Then we turn the one for the new encoding to `\@current@cmd` (see `ltoutenc.dtx` for further explanations).

```
        \expandafter
        \let
          \csname\f@encoding-cmd\endcsname
          \@current@cmd
```

We execute the default settings `\default@T`, followed by the one for the new encoding.

```
        \default@T
        \csname T@\f@encoding\endcsname
```

Finally we change the default substitution values, disable `\enc@update` and make `\f@encoding` officially the current encoding.

```
        \csname D@\f@encoding\endcsname
        \let\enc@update\relax
        \let\cf@encoding\f@encoding
}
```

(*End of definition for* `\@@enc@update`.)

`\enc@update` The default action in `\selectfont` is to do nothing.

```
\let\enc@update\relax
```

(*End of definition for* `\enc@update`.)

`\fontfamily` `\f@family` `\fontseries` `\f@series` `\fontshape` `\f@shape` They are now all defined later (and differently).

```
%\DeclareRobustCommand\fontfamily[1]{\edef\f@family{#1}}
%\DeclareRobustCommand\fontseries[1]{\edef\f@series{#1}}
%\DeclareRobustCommand\fontshape [1]{\edef\f@shape{#1}}
```

(*End of definition for* `\fontfamily` *and others.*)

`\usefont` Some handy abbreviation if you want to get some particular font in the current size. If also the size should change one has to issue a `\fontsize` command first.

`\fontencoding` needs to do some setup work so we call that, but instead of calling `\fontfamily`, `\fontseries` and `\fontshape` it earlier versions of this code did, we now set `\f@family`, etc. directly. If we would call `\fontseries` or `\fontshape` as it was done in the past, they would now interact with the existing series and shape which is not desired if we intend to use an explicit font shape!

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩                    {\usefont}{Force font face}%
\DeclareRobustCommand\usefont[4]{\fontencoding{#1}%
    \edef\f@family{#2}%
    \set@target@series{#3}%
    \edef\f@shape{#4}%
```

Any earlier `\fontseries`, etc. should be canceled and we should switch unconditionally to the requested font face so we drop any code that may have been stored in `\delayed@f@adjustment`.

```
    \let\delayed@f@adjustment\@empty
    \selectfont
    \ignorespaces}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩                    {\usefont}{Drop m in usefont}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\usefont[4]{\fontencoding{#1}%
⟨latexrelease⟩    \edef\f@family{#2}%
```

```
⟨latexrelease⟩    \set@target@series{#3}%
⟨latexrelease⟩    \edef\f@shape{#4}\selectfont
⟨latexrelease⟩    \ignorespaces}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\usefont}{Drop m in usefont}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\usefont[4]{\fontencoding{#1}%
⟨latexrelease⟩    \edef\f@family{#2}%
⟨latexrelease⟩    \edef\f@series{#3}%
⟨latexrelease⟩    \edef\f@shape{#4}\selectfont
⟨latexrelease⟩    \ignorespaces}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\usefont`.)

`\linespread` The command `\linespread` changes the current `\baselinestretch` by calling `\set@fontsize`. The values for `\f@size` and `\f@baselineskip` will be left unchanged.

```
\DeclareRobustCommand\linespread[1]
   {\set@fontsize{#1}\f@size\f@baselineskip}
```

(*End of definition for* `\linespread`.)

`\fontsize` We also define a macro that allows to specify a size. In this case, however, we also need the value of `\baselineskip`. As the first argument to `\set@fontsize` we pass the current value of `\baselinestretch`. This will either match the internal value (in which case nothing changes) or it will be an updated value due to a user change of that macro using `\renewcommand`. If we would pass the internal `\f@linespread` such a change would be effectively overwritten by a size change.

```
\DeclareRobustCommand\fontsize[2]
   {\set@fontsize\baselinestretch{#1}{#2}}
```

(*End of definition for* `\fontsize`.)

`\f@linespread` This macro holds the current internal value for `\baselinestretch`.

```
\let\f@family\@empty
\let\f@series\@empty
\let\f@shape\@empty
\let\f@size\@empty
\let\f@baselineskip\@empty
\let\f@linespread\@empty
```

(*End of definition for* `\f@linespread`.)

`\cf@encoding`

```
\let\f@encoding\@empty
\let\cf@encoding\@empty
```

(*End of definition for* `\cf@encoding`.)

\@defaultunits The function \@defaultunits when wrapped around a dimen or skip assignment supplies default units. Usage:

\@defaultunits\dimen@=#1pt\relax\@nnil

Note: the \relax is *important*. Other units can be substituted for the 'pt' if desired.

We use \remove@to@nnil as an auxiliary macros for \@defaultunits. It just has to gobble the supplied default unit 'pt' or whatever, if it wasn't used in the assignment.

```
\def\@defaultunits{\afterassignment\remove@to@nnil}
```

(*End of definition for* \@defaultunits.)

\strip@pt \rem@pt This macro strips the characters pt produced by using \the on a dimen register.

```
\begingroup
  \catcode`P=12
  \catcode`T=12
  \lowercase{
    \def\x{\def\rem@pt##1.##2PT{##1\ifnum##2>\z@.##2\fi}}}
  \expandafter\endgroup\x
\def\strip@pt{\expandafter\rem@pt\the}
```

(*End of definition for* \strip@pt *and* \rem@pt.)

\mathversion \math@version \mathversion takes the math *version* name as argument, defines \math@version appropriately and switches to the font selected forcing a call to \glb@settings if the *version* is known to the system.

```
\DeclareRobustCommand\mathversion[1]
         {\@nomath\mathversion
          \expandafter\ifx\csname mv@#1\endcsname\relax
          \@latex@error{Math version `#1' is not defined}\@eha\else
```

If there has been a frozen math version reset unconditionally to it if we are at \@math@level one, so that in the typical case of one bold symbol within the normal version this doesn't allocate an additional math alphabet. If the nesting is deeper we do nothing, which means alphabet allocations accumulate until the end of the formula. One could do slightly better but that would mean keeping track of the allocations on all levels severately and this is likely to be overkill in nearly all situations.

```
          \ifcsname mv@#1@frozen\endcsname
            \ifnum \@math@level = \@ne
              \unconditionally@reset@math@version {#1}%
            \fi
          \fi
          \edef\math@version{#1}%
```

We need to force a math font setup both now and at the point where we return to the previous math version. Forcing a math font setup can simply be done by setting \glb@currsize to an invalid value since this will trigger the setup when the formula starts.

```
          \gdef\glb@currsize{}%
```

When the scope of the current `\mathversion` ends we need to restore the old setup. However this time we need to force it directly at least if we are inside math, otherwise we could wait. Another way to enhance this code here is todo the setting only if the version really has changed after all. This might be interesting in case of `amstext` and `boldsymbol`.

```
            \aftergroup\glb@settings
            \fi}
```

Resetting the math version unconditionally means that we have to copy the frozen version to `\mv@#1` and also reset the counter `\c@mv@#1` to the number of math alphabets allocated in the frozen version.

```
\ExplSyntaxOn
\cs_new_protected:Npn \unconditionally@reset@math@version #1 {
   \cs_gset_eq:cc { mv@#1 }{ mv@#1@frozen }
   \int_gset:cn { c@mv@#1 }{ \tl_use:c {g__nfss_frozen_mv_ #1 _tl} }
}
\ExplSyntaxOff
```

(*End of definition for* `\mathversion` *and* `\math@version`.)

If TEX would support a hook just before the end of a formula (opposite of `\everymath` so to speak) the implementation of the algorithm would be much simpler because in that case we would set up the correct math fonts at this point without having to worry about incorrect settings due to nesting. The same would be true if in LATEX the use of `$` (as the primitive TEX command) would be impossible and instead only a higher-level interface would be available. Note that this does not mean that a `$` couldn't be the short-hand for starting and stopping that higher-level interface, it only means that the direct TEX function must be hidden.

Anyway, since we don't have this and won't have it in LATEX 2_ε we need to implement it in a somewhat slower way.

We test for the current math font setup on entry of a formula, i.e., on the hooks `\everymath` and `\everydisplay`. But since these hooks may contain user data we provide ourselves with an internal version of these hooks which stays frozen.

`\frozen@everymath` `\frozen@everydisplay`

New internal names for `\everymath` and `\everydisplay`.

```
\let\frozen@everymath\everymath
\let\frozen@everydisplay\everydisplay
```

(*End of definition for* `\frozen@everymath` *and* `\frozen@everydisplay`.)

`\everymath` `\everydisplay`

Now we provide now user hooks that will be called in the frozen internals.

```
\newtoks\everymath
\newtoks\everydisplay
```

(*End of definition for* `\everymath` *and* `\everydisplay`.)

`\@math@level`

This counter records the nesting level of math within math.

```
\newcount\@math@level
```

(*End of definition for* `\@math@level`.)

\frozen@everydisplay Now we define the behaviour of the frozen hooks: first check the math setup then call the user hook.

The check code may push tokens after the math formula with \aftergroup and they would prevent a $$ from dropping following spaces. We therefore use a switch to be set as the first thing after the group so that following code can determine if there was a display or some inline math (in the latter case we better not drop spaces). After setting the switch we also have to place \ignorespaces because setting the switch may be the only thing that happens after the display. The issue with handling of spaces was found in 2022, but it is really a bug fix for the code added in 2021/11.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2021/11/15}
⟨latexrelease⟩  {\frozen@everydisplay}{Handle spaces after math}%
⟨*2ekernel | latexrelease⟩
\frozen@everydisplay = {%
  \aftergroup\@ignoretrue  \aftergroup\ignorespaces
```

Record that we entered another math level.

```
  \advance\@math@level\@ne
  \check@mathfonts
  \the\everydisplay}
```

(*End of definition for* \frozen@everydisplay.)

\frozen@everymath The frozen code for inline math is similar, except that here we do not want to drop following spaces.

```
\frozen@everymath = {%
  \aftergroup\@ignorefalse
```

Record that we entered another math level.

```
  \advance\@math@level\@ne
  \check@mathfonts
  \the\everymath}
```

(*End of definition for* \frozen@everymath.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}
⟨latexrelease⟩  {\frozen@everydisplay}{Handle spaces after math}%
⟨latexrelease⟩
⟨latexrelease⟩\frozen@everydisplay = {\check@mathfonts
⟨latexrelease⟩                        \the\everydisplay}
⟨latexrelease⟩\frozen@everymath = {\check@mathfonts
⟨latexrelease⟩                      \the\everymath}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

\curr@math@size This holds locally the current math size.

```
\let\curr@math@size\@empty
```

(*End of definition for* \curr@math@size.)

3.2 Macros for loading fonts

\pickup@font The macro \pickup@font which is used in \selectfont is very simple: if the font name is undefined (i.e. not known yet) it calls \define@newfont to load it.

```
\def\pickup@font{%
    \expandafter \ifx \font@name \relax
       \define@newfont
    \fi}
```

(*End of definition for* \pickup@font.)

\split@name \pickup@font assumes that \font@name is set but it is sometimes called when \f@family, \f@series, \f@shape, or \f@size may have the wrong settings (see, e.g., the definition of \getanddefine@fonts). Therefore we need a macro to extract font *family*, *series*, *shape*, and *size* from the font name. To this end we define \split@name which takes the font name as a list of characters of \catcode 12 (without the backslash at the beginning) delimited by the special control sequence \@nil. This is not very complicated: we first ensure that / has the right \catcode

```
{\catcode`\/=12
```

and define \split@name so that it will define our private \f@encoding, \f@family, \f@series, \f@shape, and \f@size macros.

```
\gdef\split@name#1/#2/#3/#4/#5\@nil{\def\f@encoding{#1}%
                                    \def\f@family{#2}%
                                    \def\f@series{#3}%
                                    \def\f@shape{#4}%
                                    \def\f@size{#5}}}
```

(*End of definition for* \split@name.)

\curr@fontshape Abbreviation which may get removed again for speed.

```
\def\curr@fontshape{\f@encoding/\f@family/\f@series/\f@shape}
```

(*End of definition for* \curr@fontshape.)

\define@newfont Now we can tackle the problem of defining a new font.

```
\def\define@newfont{%
```

We have already mentioned that the token list that \split@name will get as argument must not start with a backslash. To reach this goal we will set the \escapechar to -1 so that the \string primitive will not generate an escape character. To keep this change local we open a group. We use \begingroup for this purpose since \define@newfont might be called in math mode, and an empty \bgroup...\egroup would add an empty Ord atom to the math list and thus affect the spacing.

Also locally redefine \typeout so that 'No file ...fd' Warnings become Font Info message just sent to the log file.

```
 \begingroup
   \let\typeout\@font@info
   \escapechar\m@ne
```

Then we extract *encoding scheme*, *family*, *series*, *shape*, and *size* from the font name. Note the four \expandafter's so that \font@name is expanded first, then \string, and finally \split@name.

```
    \expandafter\expandafter\expandafter
       \split@name\expandafter\string\font@name\@nil
```

If the \curr@fontshape combination is not available, (i.e. undefined) we call the macro \wrong@fontshape to take care of this case. Otherwise \extract@font will load the external font for us.

```
%    \expandafter\ifx
%        \csname\curr@fontshape\endcsname \relax
      \try@load@fontshape % try always
%    \fi
    \expandafter\ifx
       \csname\curr@fontshape\endcsname \relax
      \wrong@fontshape\else
```

To allow substitution we call the curr@fontshape macro which usually will expand to \relax but may hold code for substitution (see \subst@fontshape definition).

```
%      \csname\curr@fontshape\endcsname
      \extract@font\fi
```

We are nearly finished and must only restore the \escapechar by closing the group.

```
  \endgroup}

\def\try@load@fontshape{%
   \expandafter
   \ifx\csname \f@encoding+\f@family\endcsname\relax
     \@font@info{Trying to load font information for
                    \f@encoding+\f@family}%
```

We predefine this combination to be \@empty which means that next time we don't try again unnecessary in case we don't find a .fd file. If the file contains a \DeclareFontFamily command than this setting will be overwritten.

```
    \global\expandafter\let
       \csname\f@encoding+\f@family\endcsname\@empty
```

Set the catcodes used in the syntax, but do it only once (this will be restored at the end of the font loading group).

```
      \nfss@catcodes
      \let\nfss@catcodes\relax
```

For increased portability make the external filename monocase, but look for the (old style) mixed case filename if the first attempt fails.

On any monocase system this means that the file is looked for twice which takes up time and string space, but at least for this release Check for both names to give people time to re-install their private fd files with lowercase names.

```
      \edef\reserved@a{%
        \lowercase{%
          \noexpand\InputIfFileExists{\f@encoding\f@family.fd}}}%
      \reserved@a\relax
           {\@input@{\f@encoding\f@family.fd}}%
   \fi}
```

(*End of definition for* \define@newfont.)

\nfss@catcodes This macro should contain the standard \catcode assignments to all characters which are used in the commands found in an .fd file and which might have special \catcodes in the middle of a document. If necessary, this list can be extended in a package file using a suitable number of \expandafter, i.e.,

```
\expandafter\def\expandafter\nfss@catcodes
      \expandafter{\nfss@catcodes <additional settings>}
```

Note, that this macro might get executed several times since it is also called by `\DeclareFontShape`, thus it probably should not be misused as a general purpose hook.

```
\def\nfss@catcodes{%
```

We start by making `@` a letter and ignoring all blanks and newlines.

```
      \makeatletter
      \catcode`\ 9%
      \catcode`\^^I9%
      \catcode`\^^M9%
```

Then we set up `\`, `{`, `}`, `#` and `%` in case an `.fd` file is loaded during a verbatim environment.

```
      \catcode`\\\z@
      \catcode`\{\@ne
      \catcode`\}\tw@
      \catcode`\#6%
      \catcode`\^7%
      \catcode`\%14%
```

The we make sure that the important syntax parts have the right `\catcode`.

```
    \@makeother\<%
    \@makeother\>%
    \@makeother\*%
    \@makeother\.%
    \@makeother\-%
    \@makeother\/%
    \@makeother\[%
    \@makeother\]%
    \@makeother\`%
    \@makeother\'%
    \@makeother\"%
}
```

(*End of definition for* `\nfss@catcodes`.)

`\LoadFontDefinitionFile` Load and `.fd` files for some encoding and family (if it exists).

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩                    {\LoadFontDefinitionFile}{Loading .fd files}%
\def\LoadFontDefinitionFile#1#2{%
  \begingroup
    \edef\f@encoding{#1}%
    \edef\f@family{#2}%
    \try@load@fontshape
  \endgroup
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\LoadFontDefinitionFile}{Loading .fd files}%
⟨latexrelease⟩
```

```
⟨latexrelease⟩\let\LoadFontDefinitionFile\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\LoadFontDefinitionFile`.)

`\DeclareFontFamilySubstitution` The idea for this macro is stolen from the `substitutefont` package by Günter Milde, with some modifications and a new name.

Its purpose is to provide characters in a special encoding that are not available in the current font family to be taken from a different family that is visually compatible (or not if you choose badly). For example, you can match the GFS Didot Greek characters with TEX Gyre Pagella (Palatino) by specifying

```
\DeclareFontFamilySubstitution{LGR}{qpl}{udidot}
```

This way if you ask for the `LGR` encoding in for the `qpl` family you get the characters from the `udidot` family substituted.

We need to ensure that the macro is defined with `\nfss@catcodes` in force (not quite sure why at the moment to be honest).

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩        {\DeclareFontFamilySubstitution}{Provide family substitution}%
\begingroup
\nfss@catcodes
\gdef\DeclareFontFamilySubstitution#1#2#3{%
```

We only provide a set of silent substitutions. The package also (re)declared the family, but this is incorrect in my eyes and it is better to handle that differently.

Of course the families may still need loading at this point and so we arrange for this. Otherwise we might run into trouble because the necessary `\DeclareFontFamily` has not been seen.

```
    \LoadFontDefinitionFile{#1}{#2}%
    \LoadFontDefinitionFile{#1}{#3}%
    \DeclareFontShape{#1}{#2}{m}{it}{<->ssub * #3/m/it}{}%
    \DeclareFontShape{#1}{#2}{m}{n}{<->ssub * #3/m/n}{}%
    \DeclareFontShape{#1}{#2}{m}{sc}{<->ssub * #3/m/sc}{}%
    \DeclareFontShape{#1}{#2}{m}{sl}{<->ssub * #3/m/sl}{}%
```

These days a few more shapes might be around, so we declare those too. If they don't exist then after the first substitution normal fallbacks will happen.

```
    \DeclareFontShape{#1}{#2}{m}{sw}{<->ssub * #3/m/sw}{}%
    \DeclareFontShape{#1}{#2}{m}{scit}{<->ssub * #3/m/scit}{}%
    \DeclareFontShape{#1}{#2}{m}{scsl}{<->ssub * #3/m/scsl}{}%
```

Same game with `b` and `bx`, for other weights you are on your own:

```
    \DeclareFontShape{#1}{#2}{b}{it}{<->ssub * #3/b/it}{}%
    \DeclareFontShape{#1}{#2}{b}{n}{<->ssub * #3/b/n}{}%
    \DeclareFontShape{#1}{#2}{b}{scit}{<->ssub * #3/b/scit}{}%
    \DeclareFontShape{#1}{#2}{b}{scsl}{<->ssub * #3/b/scsl}{}%
    \DeclareFontShape{#1}{#2}{b}{sc}{<->ssub * #3/b/sc}{}%
    \DeclareFontShape{#1}{#2}{b}{sl}{<->ssub * #3/b/sl}{}%
    \DeclareFontShape{#1}{#2}{b}{sw}{<->ssub * #3/b/sw}{}%
    \DeclareFontShape{#1}{#2}{bx}{it}{<->ssub * #3/bx/it}{}%
```

```
   \DeclareFontShape{#1}{#2}{bx}{n}{<->ssub * #3/bx/n}{}%
   \DeclareFontShape{#1}{#2}{bx}{scit}{<->ssub * #3/bx/scit}{}%
   \DeclareFontShape{#1}{#2}{bx}{scsl}{<->ssub * #3/bx/scsl}{}%
   \DeclareFontShape{#1}{#2}{bx}{sc}{<->ssub * #3/bx/sc}{}%
   \DeclareFontShape{#1}{#2}{bx}{sl}{<->ssub * #3/bx/sl}{}%
   \DeclareFontShape{#1}{#2}{bx}{sw}{<->ssub * #3/bx/sw}{}%
}
\endgroup
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩      {\DeclareFontFamilySubstitution}{Provide family substitution}%
⟨latexrelease⟩
⟨latexrelease⟩\let\DeclareFontFamilySubstitution\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\DeclareFontFamilySubstitution`.)

`\DeclareErrorFont` Declare the last resort shape! We assume that in this fontshape there is a 10pt font but it doesn't really matter. We only loose one macro name if the assumption is false. But at least the font should be there!

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                  {\DeclareErrorFont}{No side effects please}%
\def\DeclareErrorFont#1#2#3#4#5{%
      \xdef\error@fontshape{%
          \noexpand\expandafter\noexpand\split@name\noexpand\string
          \expandafter\noexpand\csname#1/#2/#3/#4/#5\endcsname
          \noexpand\@nil}%
```

Initialize all those internal variables which may or may not have values in the first seconds of NFSS' bootstrapping process. Later on such values will be updated when an encoding is selected, etc.

We definitely don't want to set `\f@encoding`; we can set all the others since if they are left "blank" any selection would grab "error default values" as well. However, this probably should go also—and now it did.

```
%       \gdef\f@encoding{#1}%
      \gdef\default@family{#2}%
      \gdef\default@series{#3}%
      \gdef\default@shape{#4}%
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\DeclareErrorFont}{No side effects please}%
⟨latexrelease⟩
⟨latexrelease⟩\def\DeclareErrorFont#1#2#3#4#5{%
⟨latexrelease⟩      \xdef\error@fontshape{%
⟨latexrelease⟩          \noexpand\expandafter\noexpand\split@name\noexpand\string
⟨latexrelease⟩          \expandafter\noexpand\csname#1/#2/#3/#4/#5\endcsname
⟨latexrelease⟩          \noexpand\@nil}%
⟨latexrelease⟩      \gdef\default@family{#2}%
```

```
⟨latexrelease⟩       \gdef\default@series{#3}%
⟨latexrelease⟩       \gdef\default@shape{#4}%
⟨latexrelease⟩       \global\let\f@family\default@family
⟨latexrelease⟩       \global\let\f@series\default@series
⟨latexrelease⟩       \global\let\f@shape\default@shape
⟨latexrelease⟩       \gdef\f@size{#5}%
⟨latexrelease⟩       \gdef\f@baselineskip{#5pt}%
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
\@onlypreamble\DeclareErrorFont
```

(*End of definition for* `\DeclareErrorFont`.)

`\wrong@fontshape` Before we come to the macro `\extract@font` we have to take care of unknown `\curr@fontshape` combinations. The general strategy is to issue a warning and to try a default *shape*, then a default *series*, and finally a default *family*. If this last one also fails TEX will go into an infinite loop. But if the defaults are set incorrectly one deserves nothing else!

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}{\wrong@fontshape}%
⟨latexrelease⟩                   {Font substitution in preamble}%
⟨*2ekernel | latexrelease⟩
\def\wrong@fontshape{%
    \csname D@\f@encoding\endcsname   % install defaults if in math
```

We remember the wanted `\curr@fontshape` combination which we will need in a moment.

```
    \edef\reserved@a{\csname\curr@fontshape\endcsname}%
  \ifx\last@fontshape\reserved@a
     \errmessage{Corrupted NFSS tables}%
     \error@fontshape
  \else
```

Then we warn the user about the mess and set the shape to its default.

```
    \let\f@shape\default@shape
```

If the combination is not known, try the default *series*.

```
    \expandafter\ifx\csname\curr@fontshape\endcsname\relax
       \let\f@series\default@series
```

If this is still undefined, try the default *family*. Otherwise give up. We never try to change the encoding scheme!

```
        \expandafter
          \ifx\csname\curr@fontshape\endcsname\relax
           \let\f@family\default@family
```

If we change the font family and we are in the preamble then the corresponding `.fd` file may not been loaded yet. Therefore we try this now. Otherwise equating the requested font shape with the finally selected fontshape below will fail and can result in "NFSS tables corrupted". After begin document that will not happen as all `.fd` files involved in substitution are loaded at `\begin{document}`.

```
            \begingroup
               \try@load@fontshape
            \endgroup
        \fi \fi
  \fi
```

At this point a valid \curr@fontshape combination must have been found. We inform the user about this fact.

The \expandafter\string here stops TEX adding the space that it usually puts after command names in messages. The similar construction with \@undefined just produces 'undefined', but saves a few tokens.

\@wrong@font@char is locally redefined in \UseTextSymbol from its normal (empty) definition, to report the symbol generating the font switch.

```
\@font@warning{Font shape `\expandafter\string\reserved@a'
                 \expandafter\@gobble\string\@undefined\MessageBreak
               using `\curr@fontshape' instead\@wrong@font@char}%
\global\let\last@fontshape\reserved@a
```

We change \@defaultsubs to produce a warning at the end of the document. The macro \@defaultsubs is initially \relax but gets changed here if some default font substitution happens. It is then executed in \enddocument.

```
\gdef\@defaultsubs{%
  \@font@warning{Some font shapes were not available, defaults
                 substituted.\@gobbletwo}}%
```

If we substitute a \curr@fontshape combination by the default one we don't want the warning to be printed out whenever this (unknown) combination is used. Therefore we globally \let the macro corresponding to the wanted combination equal to its substitution. This requires the use of four \expandafter's since \csname...\endcsname has to be expanded before \reserved@a (i.e. the requested combination), and this must happen before the \let is executed.

```
\global\expandafter\expandafter\expandafter\let
   \expandafter\reserved@a
       \csname\curr@fontshape\endcsname
```

Now we can redefine \font@name accordingly. This *must* be done globally since it might occur in the group opened by \define@newfont. If we would this definition were local the closing \endgroup there would restore the old meaning of \font@name and then switch to the wrong font at the end of \selectfont although the correct font was loaded.

```
\xdef\font@name{%
  \csname\curr@fontshape/\f@size\endcsname}%
```

The last thing this macro does is to call \pickup@font again to load the font if it is not defined yet. At this point this code will loop endlessly if the defaults are not well defined.

```
\pickup@font}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\wrong@fontshape}%
⟨latexrelease⟩                  {Font substitution in preamble}%
⟨latexrelease⟩\def\wrong@fontshape{%
⟨latexrelease⟩    \csname D@\f@encoding\endcsname
⟨latexrelease⟩    \edef\reserved@a{\csname\curr@fontshape\endcsname}%
⟨latexrelease⟩  \ifx\last@fontshape\reserved@a
⟨latexrelease⟩     \errmessage{Corrupted NFSS tables}%
⟨latexrelease⟩     \error@fontshape
⟨latexrelease⟩  \else
⟨latexrelease⟩    \let\f@shape\default@shape
⟨latexrelease⟩    \expandafter\ifx\csname\curr@fontshape\endcsname\relax
⟨latexrelease⟩       \let\f@series\default@series
⟨latexrelease⟩        \expandafter
```

```
⟨latexrelease⟩           \ifx\csname\curr@fontshape\endcsname\relax
⟨latexrelease⟩             \let\f@family\default@family
⟨latexrelease⟩         \fi \fi
⟨latexrelease⟩   \fi
⟨latexrelease⟩     \@font@warning{Font shape
⟨latexrelease⟩              ‘\expandafter\string\reserved@a’
⟨latexrelease⟩              \expandafter\@gobble\string\@undefined
⟨latexrelease⟩              \MessageBreak
⟨latexrelease⟩              using ‘\curr@fontshape’ instead\@wrong@font@char}%
⟨latexrelease⟩     \global\let\last@fontshape\reserved@a
⟨latexrelease⟩     \gdef\@defaultsubs{%
⟨latexrelease⟩       \@font@warning{Some font shapes were not available,
⟨latexrelease⟩                      defaults substituted.\@gobbletwo}}%
⟨latexrelease⟩     \global\expandafter\expandafter\expandafter\let
⟨latexrelease⟩        \expandafter\reserved@a
⟨latexrelease⟩            \csname\curr@fontshape\endcsname
⟨latexrelease⟩     \xdef\font@name{%
⟨latexrelease⟩       \csname\curr@fontshape/\f@size\endcsname}%
⟨latexrelease⟩     \pickup@font}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\wrong@fontshape`.)

`\@wrong@font@char` Normally empty but redefined in `\UseTextSymbol` so that the Font shape undefined message can refer to the symbol causing the problem.

```
\let\@wrong@font@char\@empty
```

(*End of definition for* `\@wrong@font@char`.)

`\@@defaultsubs` See above.

`\@defaultsubs`

```
\let\@defaultsubs\relax
```

(*End of definition for* `\@@defaultsubs` *and* `\@defaultsubs`.)

`\strip@prefix` In `\extract@font` we will need a way to recover the replacement text of a macro. This is done by the primitive `\meaning` together with the macro `\strip@prefix` (for the details see appendix D of the TEXbook, p. 382).

```
\def\strip@prefix#1>{}
```

(*End of definition for* `\strip@prefix`.)

4 Assigning math fonts to *versions*

`\install@mathalphabet` This is just another name for `\gdef` but we can redefine it if necessary later on.

```
\let\install@mathalphabet\gdef
```

(*End of definition for* `\install@mathalphabet`.)

`\math@fonts`

```
\let\math@fonts\@empty
```

(*End of definition for* `\math@fonts`.)

\select@group

\select@group has four arguments: the new ⟨*math alphabet identifier*⟩ (a control sequence), the ⟨*math group number*⟩, the extra macro for math mode and the \curr@fontshape definition macro name. We first check if we are in math mode.

```
%\def\select@group#1#2#3{\relax\ifmmode
```

We do these things locally using \begingroup instead of \bgroup to avoid the appearance of an empty Ord atom on the math list.

```
%  \begingroup
```

We set the math fonts for the *family* in question by calling \getanddefine@fonts in the correct environment.

```
%     \escapechar\m@ne
%    \getanddefine@fonts{\csname c@mv@\math@version\endcsname}#3%
```

We globally select the math fonts...

```
%    \globaldefs\@ne  \math@fonts
```

... and close the group to restore \globaldefs and \escapechar.

```
%  \endgroup
```

As long as no *size* or *version* change occurs the ⟨*math alphabet identifier*⟩ should simply switch to the installed *math group* instead of calling \select@group unnecessarily. So we globally redefine the first argument (the new ⟨*math alphabet identifier*⟩) to expand into a \mathgroup switch and then select this *alphabet*. Note that this redefinition will be overwritten by the next call to a *version* macro. The original code for the end of \select@group was

```
\gdef#1{#3\mathgroup #2}#1\fi}
```

i.e. first redefining the ⟨*math alphabet identifier*⟩ and then calling the new definition to switch to the wanted ⟨*math group*⟩. Now we define the ⟨*math alphabet identifier*⟩ as a call to the \use@mathgroup command.

```
%  \xdef#1{\noexpand\use@mathgroup\noexpand#2%
%          {\number\csname c@mv@\math@version\endcsname}}%
```

But this is not sufficient, as we learned the hard way. The problem here is that the loading of the fonts that comprise the alphabet identifier #1, as well as the necessary math font assignments is deferred until it is used. This is OK so far, but if the fonts are switched within the current formula (which may happen if a sub-formula is a box that contains a math version switch) the font assignments for #1 are not restored unless #1 is used again. This is disastrous since TeX sees the wrong fonts at the end of the math formula, when it converts the math list into a horizontal list.

This is taken into account as follows: When a math alphabet identifier is used for the first time in a certain version it modifies the corresponding macro \mv@⟨*version*⟩ so that it calls \getanddefine@fonts directly in future as well. We use the macro \extract@alph@from@version to do this. It takes the math alphabet identifier #1 and the math version macro as arguments.

```
%     \expandafter\extract@alph@from@version
%        \csname mv@\math@version\expandafter\endcsname
%       \expandafter{\number\csname c@mv@\math@version\endcsname}%
%       #1%
%     \stepcounter{mv@\math@version}%
```

Finally, it is not possible to simply call the new definition since we have an argument (the third argument of `\use@mathgroup` or more exactly the argument of `\math@egroup` if the `margid` option is in force) which would swallow our closing `\fi`. So we use the `\expandafter` technique to remove the `\fi` before the `\use@mathgroup` is expanded.

```
%\expandafter #1\fi}
```

(*End of definition for* `\select@group`.)

`\extract@alph@from@version` We proceed to the definition of the macro `\extract@alph@from@version`. As stated above, it takes a math alphabet identifier and a math version macro (e.g. `\mv@normal`) as its arguments.

```
\def\extract@alph@from@version#1#2#3{%
```

To extract and replace the definition of math alphabet identifier `#3` in macro `#1` we have to recall how this definition looks like: Somewhere in the replacement text of `#1` there is the sequence

`\install@mathalphabet`⟨*math alphabet identifier*⟩ `#3{%`
⟨*Definitions for* ⟩`#3}`

Hence, the first thing we do is to extract the tokens preceding this definitions, the definition itself, and the tokens following it. To this end we define one auxiliary macro `\reserved@a`.

```
    \def\reserved@a##1\install@mathalphabet#3##2##3\@nil{%
```

When `\reserved@a` is expanded, it will have the tokens preceding the definition in question in its first argument (`##1`), the following tokens in its third argument (`##3`), and the replacement text for the math alphabet identifier `#3` in its second argument. (`##2`). This is then recorded for later use in a temporary macro `\reserved@b`.

```
      \def\reserved@b{##2}%
```

Additionally, we define a macro `\reserved@c` to reconstruct the definitions for the math version in question from the tokens that will remain unchanged (`##1` and `##3`) and the yet to build new definitions for the math alphabet identifier `#3`.

```
      \def\reserved@c####1{\gdef#1{##1####1##3}}%
```

Then we execute our auxiliary macro.

```
    \expandafter\reserved@a#1\@nil
```

OK, so now we have to build the new definition for `#3`. To do so, we first extract the interesting parts out of the old one. The old definition looks like:

`\select@group`⟨*math alphabet identifier*⟩
⟨*math group number*⟩⟨*math extra part*⟩
⟨`curr@fontshape` *definition*⟩

So we define a new temporary macro `\reserved@a` that extracts these parts.

```
    \def\reserved@a\select@group#3##1##2\@nil{%
```

This macro can now directly rebuild the math version definition by calling `\reserved@c`:

```
      \reserved@c{%
        \getanddefine@fonts{#2}##2%
        \install@mathalphabet#3{%
          \relax\ifmmode \else \non@alpherr#3\fi
          \use@mathgroup##1{#2}}}%
```

In addition it defines the alphabet the way it should be used from now on.

```
        \gdef#3{\relax\ifmmode \else \non@alpherr#3\fi
                \use@mathgroup##1{#2}}}%
```

Finally, we only have to call this macro `\reserved@a` on the old definitions recorded in `\reserved@b`:

```
      \expandafter\reserved@a\reserved@b\@nil
      }
```

(*End of definition for* `\extract@alph@from@version`.)

`\math@bgroup` `\math@egroup`

Here are the default definitions for `\math@bgroup` and `\math@egroup`. We use `\bgroup` instead of `\begingroup` to avoid 'leaking out' of style changes. This has the side effect of always producing mathord atoms.

```
\let\math@bgroup\bgroup
\def\math@egroup#1{#1\egroup}
```

(*End of definition for* `\math@bgroup` *and* `\math@egroup`.)

`\calculate@math@sizes`

Here is the default definition for `\calculate@math@sizes` a more elaborate interface is under testing in mthscale.sty.

```
\gdef\calculate@math@sizes{%
  \@font@info{Calculating\space math\space sizes\space for\space
              size\space <\f@size>}%
  \dimen@\f@size \p@
  \@tempdimb \defaultscriptratio \dimen@
  \dimen@ \defaultscriptscriptratio \dimen@
  \expandafter\xdef\csname S@\f@size\endcsname{%
    \gdef\noexpand\tf@size{\f@size}%
    \gdef\noexpand\sf@size{\strip@pt\@tempdimb}%
    \gdef\noexpand\ssf@size{\strip@pt\dimen@}%
    \noexpand\math@fontstrue}}
```

(*End of definition for* `\calculate@math@sizes`.)

`\defaultscriptratio` `\defaultscriptscriptratio`

The default ratio for math sizes is:
1 to `\defaultscriptratio` to `\defaultscriptscriptratio`.
By default this is 1 to .7 to .5.

```
\def\defaultscriptratio{.7}
\def\defaultscriptscriptratio{.5}
```

(*End of definition for* `\defaultscriptratio` *and* `\defaultscriptscriptratio`.)

`\noaccents@`

If we don't have a definition for `\noaccents@` we provide a dummy.

```
\ifx\noaccents@\@undefined
  \let\noaccents@\@empty
\fi
```

(*End of definition for* `\noaccents@`.)

\showhyphens The \showhyphens command must be redefined since the version in plain.tex uses \tenrm. We have also made some further adjustments for its use in LaTeX.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2017/01/01}{\showhyphens}%
⟨latexrelease⟩                    {XeTeX support for \showhyphens}%
⟨*2ekernel | latexrelease⟩
\ifx\XeTeXcharclass\@undefined
```

Version for engines other than XeTeX.

```
\DeclareRobustCommand\showhyphens[1]{%
  \setbox0\vbox{%
    \color@begingroup
    \tracinglostchars\z@
    \everypar{}%
    \parfillskip\z@skip\hsize\maxdimen
    \normalfont
    \pretolerance\m@ne\tolerance\m@ne\hbadness\z@\showboxdepth\z@\ #1%
    \color@endgroup}}
\else
```

XeTeX version. When using system fonts XeTeX reports consecutive runs of characters as a single item in box logging, which means the standard \showhyphens does not work. This version typesets the text into a narrow box to force hyphenation and then reconstructs a horizontal list with explicit hyphens to generate the display. Note that the lmr OpenType font is forced, this works even if the characters are not in the font as hyphenation is attempted due to the width of the space and hyphen character. It would generate spurious Missing Character warnings in the log, these are however suppressed from the terminal and log output by ensuring that \tracinglostchars is locally zero.

```
\DeclareRobustCommand\showhyphens[1]{%
  \setbox0\vbox{%
    \usefont{TU}{lmr}{m}{n}%
    \hsize 1sp %
    \hbadness\@M
    \hfuzz\maxdimen
    \tracingonline\z@
    \tracinglostchars\z@
    \everypar={}%
    \leftskip\z@skip
    \rightskip\z@skip
    \parfillskip\z@skip
    \hyphenpenalty=-\@M
    \pretolerance\m@ne
    \interlinepenalty\z@
    \clubpenalty\z@
    \widowpenalty\z@
    \brokenpenalty1127 %
    \setbox\z@\hbox{}%
    \noindent
    \hskip\z@skip
    #1%
    \par
```

Note here we stop the loop if made no progress, non-removable items may mean that we can not process the whole list (which would be testable as \lastnodetype=-1).

```
    \loop
    \@tempswafalse
    \ifnum\lastnodetype=11\unskip\@tempswatrue\fi
    \ifnum\lastnodetype=12\unkern\@tempswatrue\fi
    \ifnum\lastnodetype=13 %
     \count@\lastpenalty
     \unpenalty\@tempswatrue
   \fi
   \ifnum\lastnodetype=\@ne
    \setbox\tw@\lastbox\@tempswatrue
    \setbox0\hbox{\unhbox\tw@\unskip\unskip\unpenalty
                  \ifnum\count@=1127 \else\ \fi
                  \unhbox0}%
    \count@\z@
   \fi
   \if@tempswa
   \repeat
  \hbadness\z@
  \hsize\maxdimen
  \showboxdepth\z@
  \tolerance\m@ne
  \hyphenpenalty\z@
  \noindent\unhbox\z@
}}
\fi
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\showhyphens}%
⟨latexrelease⟩                  {XeTeX support for \showhyphens}%
⟨latexrelease⟩\gdef\showhyphens#1{%
⟨latexrelease⟩  \setbox0\vbox{%
⟨latexrelease⟩    \color@begingroup
⟨latexrelease⟩    \everypar{}%
⟨latexrelease⟩    \parfillskip\z@skip\hsize\maxdimen
⟨latexrelease⟩    \normalfont
⟨latexrelease⟩    \pretolerance\m@ne\tolerance\m@ne
⟨latexrelease⟩    \hbadness\z@\showboxdepth\z@\ #1%
⟨latexrelease⟩    \color@endgroup}}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\showhyphens`.)

`\addto@hook` We need a macro to add tokens to a hook.

```
\long\def\addto@hook#1#2{#1\expandafter{\the#1#2}}
```

(*End of definition for* `\addto@hook`.)

`\@vpt`

```
 \def\@vpt{5}
```

(*End of definition for* `\@vpt`.)

\@vipt

```
\def\@vipt{6}
```

(*End of definition for* `\@vipt`.)

\@viipt

```
\def\@viipt{7}
```

(*End of definition for* `\@viipt`.)

\@viiipt

```
\def\@viiipt{8}
```

(*End of definition for* `\@viiipt`.)

\@ixpt

```
\def\@ixpt{9}
```

(*End of definition for* `\@ixpt`.)

\@xpt

```
\def\@xpt{10}
```

(*End of definition for* `\@xpt`.)

\@xipt

```
\def\@xipt{10.95}
```

(*End of definition for* `\@xipt`.)

\@xiipt

```
\def\@xiipt{12}
```

(*End of definition for* `\@xiipt`.)

\@xivpt

```
\def\@xivpt{14.4}
```

(*End of definition for* `\@xivpt`.)

\@xviipt

```
\def\@xviipt{17.28}
```

(*End of definition for* `\@xviipt`.)

\@xxpt

```
\def\@xxpt{20.74}
```

(*End of definition for* `\@xxpt`.)

\@xxvpt

```
\def\@xxvpt{24.88}
```

(*End of definition for* `\@xxvpt`.)

⟨/2ekernel⟩

File 25
ltfssaxes.dtx

This file contains the implementation for handling extra axes by splitting the series and the shape values into sub-categories. See other parts of the LaTeX distribution, or *The LaTeX Companion* for higher level documentation of the LaTeX Font Selection Scheme.

```
⟨2ekernel⟩\message{NFSS axes,}
```

Everything in the this file got introduced 2020/02/02, so we use large rollback chunks, only interrupted if necessary.

```
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease⟩    {\DeclareFontSeriesChangeRule}{Series change rules}%
```

1 Changing the font series

In the original NFSS implementation the series was a single attribute stored in `\f@series` and so one always had to specify both weight and width together. This meant that it was impossible to typeset a paragraph in a condensed font and inside have a few words in bold weight (but still condensed) without doing this manually by requesting `\fontseries{bc}\selectfont`.

The new implementation now works differently, by looking at both the current value of `\f@series` and the requested new series; these are then used to select a new series value. Thus, if the current series is `c` and we ask for `b` we now get `bc`.

This is done by consulting a simple lookup table. This table is configurable (though most likely that flexibility will seldom if ever be needed). Adding or changing entries in this table is done with `\DeclareFontSeriesChangeRule`.

1.1 The series lookup table

`\DeclareFontSeriesChangeRule`

The macro `\DeclareFontSeriesChangeRule` defines entries in a simple database (implemented as a set of commands) that define mappings from the current series and a requested new series to a result series (and additionally offers an alternative if the desired one is unavailable):

`#1` current `\f@series`

`#2` requested new series

`#3` result (provided this series exists in the given font family)

`#4` alternative result (if `#3` does not exist)

If an `.fd` file has its own substitution rules then `#3` exist and thus `#4` is not applied.

If there is no matching database entry, or if neither the result nor the alternative result exists in this font family, then the requested new series is used (which then may trigger substitutions later on).

```
\def\DeclareFontSeriesChangeRule#1#2#3#4{%
  \@namedef{series@#1@#2}{{#3}{#4}}}
```

(*End of definition for* `\DeclareFontSeriesChangeRule`.)

1.2 Mapping rules for series changes

The rules set up use explicit series values not `\..default` indirections; my current feeling is that this is in fact better.

With 9 weight values (from `ul` to `ub`) and 9 width values (from `uc` to `ux`), this table is now rather large (far more than 1000 entries), but, on the other hand, the table doesn't change and accessing rules is fast when using a table implemented in this way.

We could alternatively split the axis and maintain weight and width separately, but that would take more processing time and would not allow for setting up explicit exceptions nicely (not sure that this would ever get used though).

Design considerations for mapping entries:

- We make `m` reset both weight and width (as this is how it always worked). To reset just the width `?m` is provided and to reset just the weight `m?` is provided.
- We support "`m`⟨*width*⟩" and "⟨*weight*⟩`m`", e.g., `mec` to mean "go to medium weight and extra-condensed width". At the end of the process we automatically drop any leftover `m` in the series name (unless it is just a single `m`).
- If there is no table entry then the requested series is used unconditionally. This means that we usually do not need entries where the second argument (the requested series) and the third argument (the result series) are identical (unless we want to use the fourth argument to specify an alternative result series). In particular, this means:
  - Any request for `m` needs no entry, i.e., there are no entries which have `m` as the second argument.
  - Any request to set both weight and width (e.g., `sbx` or `ulc`) needs no entry. For that reason, there are no entries which have a weight+width as second argument (except for cases involving `bx`, see below). In particular, this is also true for cases involving `m`, e.g., `bm` (bold medium width) which automatically gets reduced to `b`, or `mc` (medium weight condensed) which becomes `c` as a result.

    There is one exception: we do have rules for a request for `bx`; this is because the Computer Modern fonts (or related families like Latin Modern) need `bx` as their bold series default (`\bfdefault`), since they only have a `b` series for a restricted set of font faces. Many other font families have only `b` but no `bx`, so we have explicit rules that fall back from `bx` to `b`.
- For each combination of a non-`m` ⟨*weight*⟩ and a non-`m` ⟨*width*⟩, there are 19 entries which have "⟨*weight*⟩⟨*width*⟩" as first argument: 8 of them have the weight values from `ul` to `ub` except `m` as second argument, another 8 have the width values from `uc` to `ux` except `m` as second argument, and (further down in this file) there is one entry which has `bx` as second argument and two further entries which have `m?` and `?m` as second argument. Rules which have `m` or a weight+width (other than `bx`) as second argument aren't needed (see above).
- For each non-`m` ⟨*weight*⟩, there are at least 11 entries which have "⟨*weight*⟩" as first argument: 8 of them have the width values from `uc` to `ux` except `m` as second argument, and (further down in this file) there is one entry which has `bx` as second argument and two further entries which have `m?` and `?m` as second argument. Rules which have a single weight value as second argument aren't needed because the

second and third arguments would then be identical. In some cases, there are extra rules which make use of the fourth argument to specify an alternative result series.

- Similarly, for each non-`m` ⟨*width*⟩, there are at least 11 entries which have "⟨*width*⟩" as first argument: 8 of them have the weight values from `ul` to `ub` except `m` as second argument, and (further down in this file) there is one entry which has `bx` as second argument and two further entries which have `m?` and `?m` as second argument. Rules which have a single width value as second argument aren't needed because the second and third arguments would then be identical. In some cases, there are extra rules which make use of the fourth argument to specify an alternative result series.

- Only a few entries have "alternative" values.

  The idea is that you don't want the normal substitution to kick in because that would reset the shape first and it may be better to stay with `b` when a change to `c` is requested and `bc` doesn't exist, than to go to first change the shape to `n` and then find that `bc/n` doesn't exist either and thus ending up with `m/n`.

  We do this, for example, when `sb`, `eb`, or `ub` is requested but can't be fulfilled. In that case it is better to try to stay with some sort of bold rather than ending up with `m`. There are some other cases where an "alternative" value is specified; these are explained below in the appropriate places.

```
\DeclareFontSeriesChangeRule {uluc}{ul} {uluc} {}
\DeclareFontSeriesChangeRule {uluc}{el} {eluc} {}
\DeclareFontSeriesChangeRule {uluc}{l}  {luc}  {}
\DeclareFontSeriesChangeRule {uluc}{sl} {sluc} {}
\DeclareFontSeriesChangeRule {uluc}{sb} {sbuc} {buc}
\DeclareFontSeriesChangeRule {uluc}{b}  {buc}  {}
\DeclareFontSeriesChangeRule {uluc}{eb} {ebuc} {buc}
\DeclareFontSeriesChangeRule {uluc}{ub} {ubuc} {buc}
\DeclareFontSeriesChangeRule {uluc}{uc} {uluc} {}
\DeclareFontSeriesChangeRule {uluc}{ec} {ulec} {}
\DeclareFontSeriesChangeRule {uluc}{c}  {ulc}  {}
\DeclareFontSeriesChangeRule {uluc}{sc} {ulsc} {}
\DeclareFontSeriesChangeRule {uluc}{sx} {ulsx} {}
\DeclareFontSeriesChangeRule {uluc}{x}  {ulx}  {}
\DeclareFontSeriesChangeRule {uluc}{ex} {ulex} {}
\DeclareFontSeriesChangeRule {uluc}{ux} {ulux} {}
\DeclareFontSeriesChangeRule {ulec}{ul} {ulec} {}
\DeclareFontSeriesChangeRule {ulec}{el} {elec} {}
\DeclareFontSeriesChangeRule {ulec}{l}  {lec}  {}
\DeclareFontSeriesChangeRule {ulec}{sl} {slec} {}
\DeclareFontSeriesChangeRule {ulec}{sb} {sbec} {bec}
\DeclareFontSeriesChangeRule {ulec}{b}  {bec}  {}
\DeclareFontSeriesChangeRule {ulec}{eb} {ebec} {bec}
\DeclareFontSeriesChangeRule {ulec}{ub} {ubec} {bec}
\DeclareFontSeriesChangeRule {ulec}{uc} {uluc} {}
\DeclareFontSeriesChangeRule {ulec}{ec} {ulec} {}
\DeclareFontSeriesChangeRule {ulec}{c}  {ulc}  {}
\DeclareFontSeriesChangeRule {ulec}{sc} {ulsc} {}
\DeclareFontSeriesChangeRule {ulec}{sx} {ulsx} {}
\DeclareFontSeriesChangeRule {ulec}{x}  {ulx}  {}
\DeclareFontSeriesChangeRule {ulec}{ex} {ulex} {}
\DeclareFontSeriesChangeRule {ulec}{ux} {ulux} {}
```

```
\DeclareFontSeriesChangeRule {ulc}{ul}  {ulc}  {}
\DeclareFontSeriesChangeRule {ulc}{el}  {elc}  {}
\DeclareFontSeriesChangeRule {ulc}{l}   {lc}   {}
\DeclareFontSeriesChangeRule {ulc}{sl}  {slc}  {}
\DeclareFontSeriesChangeRule {ulc}{sb}  {sbc}  {bc}
\DeclareFontSeriesChangeRule {ulc}{b}   {bc}   {}
\DeclareFontSeriesChangeRule {ulc}{eb}  {ebc}  {bc}
\DeclareFontSeriesChangeRule {ulc}{ub}  {ubc}  {bc}
\DeclareFontSeriesChangeRule {ulc}{uc}  {uluc} {}
\DeclareFontSeriesChangeRule {ulc}{ec}  {ulec} {}
\DeclareFontSeriesChangeRule {ulc}{c}   {ulc}  {}
\DeclareFontSeriesChangeRule {ulc}{sc}  {ulsc} {}
\DeclareFontSeriesChangeRule {ulc}{sx}  {ulsx} {}
\DeclareFontSeriesChangeRule {ulc}{x}   {ulx}  {}
\DeclareFontSeriesChangeRule {ulc}{ex}  {ulex} {}
\DeclareFontSeriesChangeRule {ulc}{ux}  {ulux} {}

\DeclareFontSeriesChangeRule {ulsc}{ul} {ulsc} {}
\DeclareFontSeriesChangeRule {ulsc}{el} {elsc} {}
\DeclareFontSeriesChangeRule {ulsc}{l}  {lsc}  {}
\DeclareFontSeriesChangeRule {ulsc}{sl} {slsc} {}
\DeclareFontSeriesChangeRule {ulsc}{sb} {sbsc} {bsc}
\DeclareFontSeriesChangeRule {ulsc}{b}  {bsc}  {}
\DeclareFontSeriesChangeRule {ulsc}{eb} {ebsc} {bsc}
\DeclareFontSeriesChangeRule {ulsc}{ub} {ubsc} {bsc}
\DeclareFontSeriesChangeRule {ulsc}{uc} {uluc} {}
\DeclareFontSeriesChangeRule {ulsc}{ec} {ulec} {}
\DeclareFontSeriesChangeRule {ulsc}{c}  {ulc}  {}
\DeclareFontSeriesChangeRule {ulsc}{sc} {ulsc} {}
\DeclareFontSeriesChangeRule {ulsc}{sx} {ulsx} {}
\DeclareFontSeriesChangeRule {ulsc}{x}  {ulx}  {}
\DeclareFontSeriesChangeRule {ulsc}{ex} {ulex} {}
\DeclareFontSeriesChangeRule {ulsc}{ux} {ulux} {}

\DeclareFontSeriesChangeRule {ul}{uc}   {uluc} {}
\DeclareFontSeriesChangeRule {ul}{ec}   {ulec} {}
\DeclareFontSeriesChangeRule {ul}{c}    {ulc}  {}
\DeclareFontSeriesChangeRule {ul}{sc}   {ulsc} {}
\DeclareFontSeriesChangeRule {ul}{sx}   {ulsx} {}
\DeclareFontSeriesChangeRule {ul}{x}    {ulx}  {}
\DeclareFontSeriesChangeRule {ul}{ex}   {ulex} {}
\DeclareFontSeriesChangeRule {ul}{ux}   {ulux} {}
\DeclareFontSeriesChangeRule {ul}{sb}   {sb}   {b}
\DeclareFontSeriesChangeRule {ul}{eb}   {eb}   {b}
\DeclareFontSeriesChangeRule {ul}{ub}   {ub}   {b}

\DeclareFontSeriesChangeRule {ulsx}{ul} {ulsx} {}
\DeclareFontSeriesChangeRule {ulsx}{el} {elsx} {}
\DeclareFontSeriesChangeRule {ulsx}{l}  {lsx}  {}
\DeclareFontSeriesChangeRule {ulsx}{sl} {slsx} {}
\DeclareFontSeriesChangeRule {ulsx}{sb} {sbsx} {bsx}
\DeclareFontSeriesChangeRule {ulsx}{b}  {bsx}  {}
\DeclareFontSeriesChangeRule {ulsx}{eb} {ebsx} {bsx}
\DeclareFontSeriesChangeRule {ulsx}{ub} {ubsx} {bsx}
\DeclareFontSeriesChangeRule {ulsx}{uc} {uluc} {}
\DeclareFontSeriesChangeRule {ulsx}{ec} {ulec} {}
```

```
\DeclareFontSeriesChangeRule {ulsx}{c}  {ulc}  {}
\DeclareFontSeriesChangeRule {ulsx}{sc} {ulsc} {}
\DeclareFontSeriesChangeRule {ulsx}{sx} {ulsx} {}
\DeclareFontSeriesChangeRule {ulsx}{x}  {ulx}  {}
\DeclareFontSeriesChangeRule {ulsx}{ex} {ulex} {}
\DeclareFontSeriesChangeRule {ulsx}{ux} {ulux} {}

\DeclareFontSeriesChangeRule {ulx}{ul}  {ulx}  {}
\DeclareFontSeriesChangeRule {ulx}{el}  {elx}  {}
\DeclareFontSeriesChangeRule {ulx}{l}   {lx}   {}
\DeclareFontSeriesChangeRule {ulx}{sl}  {slx}  {}
\DeclareFontSeriesChangeRule {ulx}{sb}  {sbx}  {bx}
\DeclareFontSeriesChangeRule {ulx}{b}   {bx}   {}
\DeclareFontSeriesChangeRule {ulx}{eb}  {ebx}  {bx}
\DeclareFontSeriesChangeRule {ulx}{ub}  {ubx}  {bx}
\DeclareFontSeriesChangeRule {ulx}{uc}  {uluc} {}
\DeclareFontSeriesChangeRule {ulx}{ec}  {ulec} {}
\DeclareFontSeriesChangeRule {ulx}{c}   {ulc}  {}
\DeclareFontSeriesChangeRule {ulx}{sc}  {ulsc} {}
\DeclareFontSeriesChangeRule {ulx}{sx}  {ulsx} {}
\DeclareFontSeriesChangeRule {ulx}{x}   {ulx}  {}
\DeclareFontSeriesChangeRule {ulx}{ex}  {ulex} {}
\DeclareFontSeriesChangeRule {ulx}{ux}  {ulux} {}

\DeclareFontSeriesChangeRule {ulex}{ul} {ulex} {}
\DeclareFontSeriesChangeRule {ulex}{el} {elex} {}
\DeclareFontSeriesChangeRule {ulex}{l}  {lex}  {}
\DeclareFontSeriesChangeRule {ulex}{sl} {slex} {}
\DeclareFontSeriesChangeRule {ulex}{sb} {sbex} {bex}
\DeclareFontSeriesChangeRule {ulex}{b}  {bex}  {}
\DeclareFontSeriesChangeRule {ulex}{eb} {ebex} {bex}
\DeclareFontSeriesChangeRule {ulex}{ub} {ubex} {bex}
\DeclareFontSeriesChangeRule {ulex}{uc} {uluc} {}
\DeclareFontSeriesChangeRule {ulex}{ec} {ulec} {}
\DeclareFontSeriesChangeRule {ulex}{c}  {ulc}  {}
\DeclareFontSeriesChangeRule {ulex}{sc} {ulsc} {}
\DeclareFontSeriesChangeRule {ulex}{sx} {ulsx} {}
\DeclareFontSeriesChangeRule {ulex}{x}  {ulx}  {}
\DeclareFontSeriesChangeRule {ulex}{ex} {ulex} {}
\DeclareFontSeriesChangeRule {ulex}{ux} {ulux} {}

\DeclareFontSeriesChangeRule {ulux}{ul} {ulux} {}
\DeclareFontSeriesChangeRule {ulux}{el} {elux} {}
\DeclareFontSeriesChangeRule {ulux}{l}  {lux}  {}
\DeclareFontSeriesChangeRule {ulux}{sl} {slux} {}
\DeclareFontSeriesChangeRule {ulux}{sb} {sbux} {bux}
\DeclareFontSeriesChangeRule {ulux}{b}  {bux}  {}
\DeclareFontSeriesChangeRule {ulux}{eb} {ebux} {bux}
\DeclareFontSeriesChangeRule {ulux}{ub} {ubux} {bux}
\DeclareFontSeriesChangeRule {ulux}{uc} {uluc} {}
\DeclareFontSeriesChangeRule {ulux}{ec} {ulec} {}
\DeclareFontSeriesChangeRule {ulux}{c}  {ulc}  {}
\DeclareFontSeriesChangeRule {ulux}{sc} {ulsc} {}
\DeclareFontSeriesChangeRule {ulux}{sx} {ulsx} {}
\DeclareFontSeriesChangeRule {ulux}{x}  {ulx}  {}
\DeclareFontSeriesChangeRule {ulux}{ex} {ulex} {}
```

```
\DeclareFontSeriesChangeRule {ulux}{ux} {ulux} {}

\DeclareFontSeriesChangeRule {eluc}{ul} {uluc} {}
\DeclareFontSeriesChangeRule {eluc}{el} {eluc} {}
\DeclareFontSeriesChangeRule {eluc}{l}  {luc}  {}
\DeclareFontSeriesChangeRule {eluc}{sl} {sluc} {}
\DeclareFontSeriesChangeRule {eluc}{sb} {sbuc} {buc}
\DeclareFontSeriesChangeRule {eluc}{b}  {buc}  {}
\DeclareFontSeriesChangeRule {eluc}{eb} {ebuc} {buc}
\DeclareFontSeriesChangeRule {eluc}{ub} {ubuc} {buc}
\DeclareFontSeriesChangeRule {eluc}{uc} {eluc} {}
\DeclareFontSeriesChangeRule {eluc}{ec} {elec} {}
\DeclareFontSeriesChangeRule {eluc}{c}  {elc}  {}
\DeclareFontSeriesChangeRule {eluc}{sc} {elsc} {}
\DeclareFontSeriesChangeRule {eluc}{sx} {elsx} {}
\DeclareFontSeriesChangeRule {eluc}{x}  {elx}  {}
\DeclareFontSeriesChangeRule {eluc}{ex} {elex} {}
\DeclareFontSeriesChangeRule {eluc}{ux} {elux} {}

\DeclareFontSeriesChangeRule {elec}{ul} {ulec} {}
\DeclareFontSeriesChangeRule {elec}{el} {elec} {}
\DeclareFontSeriesChangeRule {elec}{l}  {lec}  {}
\DeclareFontSeriesChangeRule {elec}{sl} {slec} {}
\DeclareFontSeriesChangeRule {elec}{sb} {sbec} {bec}
\DeclareFontSeriesChangeRule {elec}{b}  {bec}  {}
\DeclareFontSeriesChangeRule {elec}{eb} {ebec} {bec}
\DeclareFontSeriesChangeRule {elec}{ub} {ubec} {bec}
\DeclareFontSeriesChangeRule {elec}{uc} {eluc} {}
\DeclareFontSeriesChangeRule {elec}{ec} {elec} {}
\DeclareFontSeriesChangeRule {elec}{c}  {elc}  {}
\DeclareFontSeriesChangeRule {elec}{sc} {elsc} {}
\DeclareFontSeriesChangeRule {elec}{sx} {elsx} {}
\DeclareFontSeriesChangeRule {elec}{x}  {elx}  {}
\DeclareFontSeriesChangeRule {elec}{ex} {elex} {}
\DeclareFontSeriesChangeRule {elec}{ux} {elux} {}

\DeclareFontSeriesChangeRule {elc}{ul}  {ulc}  {}
\DeclareFontSeriesChangeRule {elc}{el}  {elc}  {}
\DeclareFontSeriesChangeRule {elc}{l}   {lc}   {}
\DeclareFontSeriesChangeRule {elc}{sl}  {slc}  {}
\DeclareFontSeriesChangeRule {elc}{sb}  {sbc}  {bc}
\DeclareFontSeriesChangeRule {elc}{b}   {bc}   {}
\DeclareFontSeriesChangeRule {elc}{eb}  {ebc}  {bc}
\DeclareFontSeriesChangeRule {elc}{ub}  {ubc}  {bc}
\DeclareFontSeriesChangeRule {elc}{uc}  {eluc} {}
\DeclareFontSeriesChangeRule {elc}{ec}  {elec} {}
\DeclareFontSeriesChangeRule {elc}{c}   {elc}  {}
\DeclareFontSeriesChangeRule {elc}{sc}  {elsc} {}
\DeclareFontSeriesChangeRule {elc}{sx}  {elsx} {}
\DeclareFontSeriesChangeRule {elc}{x}   {elx}  {}
\DeclareFontSeriesChangeRule {elc}{ex}  {elex} {}
\DeclareFontSeriesChangeRule {elc}{ux}  {elux} {}

\DeclareFontSeriesChangeRule {elsc}{ul} {ulsc} {}
\DeclareFontSeriesChangeRule {elsc}{el} {elsc} {}
\DeclareFontSeriesChangeRule {elsc}{l}  {lsc}  {}
\DeclareFontSeriesChangeRule {elsc}{sl} {slsc} {}
```

```
\DeclareFontSeriesChangeRule {elsc}{sb} {sbsc} {bsc}
\DeclareFontSeriesChangeRule {elsc}{b}  {bsc}  {}
\DeclareFontSeriesChangeRule {elsc}{eb} {ebsc} {bsc}
\DeclareFontSeriesChangeRule {elsc}{ub} {ubsc} {bsc}
\DeclareFontSeriesChangeRule {elsc}{uc} {eluc} {}
\DeclareFontSeriesChangeRule {elsc}{ec} {elec} {}
\DeclareFontSeriesChangeRule {elsc}{c}  {elc}  {}
\DeclareFontSeriesChangeRule {elsc}{sc} {elsc} {}
\DeclareFontSeriesChangeRule {elsc}{sx} {elsx} {}
\DeclareFontSeriesChangeRule {elsc}{x}  {elx}  {}
\DeclareFontSeriesChangeRule {elsc}{ex} {elex} {}
\DeclareFontSeriesChangeRule {elsc}{ux} {elux} {}

\DeclareFontSeriesChangeRule {el}{uc}   {eluc} {}
\DeclareFontSeriesChangeRule {el}{ec}   {elec} {}
\DeclareFontSeriesChangeRule {el}{c}    {elc}  {}
\DeclareFontSeriesChangeRule {el}{sc}   {elsc} {}
\DeclareFontSeriesChangeRule {el}{sx}   {elsx} {}
\DeclareFontSeriesChangeRule {el}{x}    {elx}  {}
\DeclareFontSeriesChangeRule {el}{ex}   {elex} {}
\DeclareFontSeriesChangeRule {el}{ux}   {elux} {}
\DeclareFontSeriesChangeRule {el}{sb}   {sb}   {b}
\DeclareFontSeriesChangeRule {el}{eb}   {eb}   {b}
\DeclareFontSeriesChangeRule {el}{ub}   {ub}   {b}

\DeclareFontSeriesChangeRule {elsx}{ul} {ulsx} {}
\DeclareFontSeriesChangeRule {elsx}{el} {elsx} {}
\DeclareFontSeriesChangeRule {elsx}{l}  {lsx}  {}
\DeclareFontSeriesChangeRule {elsx}{sl} {slsx} {}
\DeclareFontSeriesChangeRule {elsx}{sb} {sbsx} {bsx}
\DeclareFontSeriesChangeRule {elsx}{b}  {bsx}  {}
\DeclareFontSeriesChangeRule {elsx}{eb} {ebsx} {bsx}
\DeclareFontSeriesChangeRule {elsx}{ub} {ubsx} {bsx}
\DeclareFontSeriesChangeRule {elsx}{uc} {eluc} {}
\DeclareFontSeriesChangeRule {elsx}{ec} {elec} {}
\DeclareFontSeriesChangeRule {elsx}{c}  {elc}  {}
\DeclareFontSeriesChangeRule {elsx}{sc} {elsc} {}
\DeclareFontSeriesChangeRule {elsx}{sx} {elsx} {}
\DeclareFontSeriesChangeRule {elsx}{x}  {elx}  {}
\DeclareFontSeriesChangeRule {elsx}{ex} {elex} {}
\DeclareFontSeriesChangeRule {elsx}{ux} {elux} {}

\DeclareFontSeriesChangeRule {elx}{ul}  {ulx}  {}
\DeclareFontSeriesChangeRule {elx}{el}  {elx}  {}
\DeclareFontSeriesChangeRule {elx}{l}   {lx}   {}
\DeclareFontSeriesChangeRule {elx}{sl}  {slx}  {}
\DeclareFontSeriesChangeRule {elx}{sb}  {sbx}  {bx}
\DeclareFontSeriesChangeRule {elx}{b}   {bx}   {}
\DeclareFontSeriesChangeRule {elx}{eb}  {ebx}  {bx}
\DeclareFontSeriesChangeRule {elx}{ub}  {ubx}  {bx}
\DeclareFontSeriesChangeRule {elx}{uc}  {eluc} {}
\DeclareFontSeriesChangeRule {elx}{ec}  {elec} {}
\DeclareFontSeriesChangeRule {elx}{c}   {elc}  {}
\DeclareFontSeriesChangeRule {elx}{sc}  {elsc} {}
\DeclareFontSeriesChangeRule {elx}{sx}  {elsx} {}
\DeclareFontSeriesChangeRule {elx}{x}   {elx}  {}
```

```
\DeclareFontSeriesChangeRule {elx}{ex}  {elex} {}
\DeclareFontSeriesChangeRule {elx}{ux}  {elux} {}

\DeclareFontSeriesChangeRule {elex}{ul} {ulex} {}
\DeclareFontSeriesChangeRule {elex}{el} {elex} {}
\DeclareFontSeriesChangeRule {elex}{l}  {lex}  {}
\DeclareFontSeriesChangeRule {elex}{sl} {slex} {}
\DeclareFontSeriesChangeRule {elex}{sb} {sbex} {bex}
\DeclareFontSeriesChangeRule {elex}{b}  {bex}  {}
\DeclareFontSeriesChangeRule {elex}{eb} {ebex} {bex}
\DeclareFontSeriesChangeRule {elex}{ub} {ubex} {bex}
\DeclareFontSeriesChangeRule {elex}{uc} {eluc} {}
\DeclareFontSeriesChangeRule {elex}{ec} {elec} {}
\DeclareFontSeriesChangeRule {elex}{c}  {elc}  {}
\DeclareFontSeriesChangeRule {elex}{sc} {elsc} {}
\DeclareFontSeriesChangeRule {elex}{sx} {elsx} {}
\DeclareFontSeriesChangeRule {elex}{x}  {elx}  {}
\DeclareFontSeriesChangeRule {elex}{ex} {elex} {}
\DeclareFontSeriesChangeRule {elex}{ux} {elux} {}

\DeclareFontSeriesChangeRule {elux}{ul} {ulux} {}
\DeclareFontSeriesChangeRule {elux}{el} {elux} {}
\DeclareFontSeriesChangeRule {elux}{l}  {lux}  {}
\DeclareFontSeriesChangeRule {elux}{sl} {slux} {}
\DeclareFontSeriesChangeRule {elux}{sb} {sbux} {bux}
\DeclareFontSeriesChangeRule {elux}{b}  {bux}  {}
\DeclareFontSeriesChangeRule {elux}{eb} {ebux} {bux}
\DeclareFontSeriesChangeRule {elux}{ub} {ubux} {bux}
\DeclareFontSeriesChangeRule {elux}{uc} {eluc} {}
\DeclareFontSeriesChangeRule {elux}{ec} {elec} {}
\DeclareFontSeriesChangeRule {elux}{c}  {elc}  {}
\DeclareFontSeriesChangeRule {elux}{sc} {elsc} {}
\DeclareFontSeriesChangeRule {elux}{sx} {elsx} {}
\DeclareFontSeriesChangeRule {elux}{x}  {elx}  {}
\DeclareFontSeriesChangeRule {elux}{ex} {elex} {}
\DeclareFontSeriesChangeRule {elux}{ux} {elux} {}

\DeclareFontSeriesChangeRule {luc}{ul}  {uluc} {}
\DeclareFontSeriesChangeRule {luc}{el}  {eluc} {}
\DeclareFontSeriesChangeRule {luc}{l}   {luc}  {}
\DeclareFontSeriesChangeRule {luc}{sl}  {sluc} {}
\DeclareFontSeriesChangeRule {luc}{sb}  {sbuc} {buc}
\DeclareFontSeriesChangeRule {luc}{b}   {buc}  {}
\DeclareFontSeriesChangeRule {luc}{eb}  {ebuc} {buc}
\DeclareFontSeriesChangeRule {luc}{ub}  {ubuc} {buc}
\DeclareFontSeriesChangeRule {luc}{uc}  {luc}  {}
\DeclareFontSeriesChangeRule {luc}{ec}  {lec}  {}
\DeclareFontSeriesChangeRule {luc}{c}   {lc}   {}
\DeclareFontSeriesChangeRule {luc}{sc}  {lsc}  {}
\DeclareFontSeriesChangeRule {luc}{sx}  {lsx}  {}
\DeclareFontSeriesChangeRule {luc}{x}   {lx}   {}
\DeclareFontSeriesChangeRule {luc}{ex}  {lex}  {}
\DeclareFontSeriesChangeRule {luc}{ux}  {lux}  {}

\DeclareFontSeriesChangeRule {lec}{ul}  {ulec} {}
\DeclareFontSeriesChangeRule {lec}{el}  {elec} {}
\DeclareFontSeriesChangeRule {lec}{l}   {lec}  {}
```

```
\DeclareFontSeriesChangeRule {lec}{sl}  {slec} {}
\DeclareFontSeriesChangeRule {lec}{sb}  {sbec} {bec}
\DeclareFontSeriesChangeRule {lec}{b}   {bec}  {}
\DeclareFontSeriesChangeRule {lec}{eb}  {ebec} {bec}
\DeclareFontSeriesChangeRule {lec}{ub}  {ubec} {bec}
\DeclareFontSeriesChangeRule {lec}{uc}  {luc}  {}
\DeclareFontSeriesChangeRule {lec}{ec}  {lec}  {}
\DeclareFontSeriesChangeRule {lec}{c}   {lc}   {}
\DeclareFontSeriesChangeRule {lec}{sc}  {lsc}  {}
\DeclareFontSeriesChangeRule {lec}{sx}  {lsx}  {}
\DeclareFontSeriesChangeRule {lec}{x}   {lx}   {}
\DeclareFontSeriesChangeRule {lec}{ex}  {lex}  {}
\DeclareFontSeriesChangeRule {lec}{ux}  {lux}  {}

\DeclareFontSeriesChangeRule {lc}{ul}   {ulc}  {}
\DeclareFontSeriesChangeRule {lc}{el}   {elc}  {}
\DeclareFontSeriesChangeRule {lc}{l}    {lc}   {}
\DeclareFontSeriesChangeRule {lc}{sl}   {slc}  {}
\DeclareFontSeriesChangeRule {lc}{sb}   {sbc}  {bc}
\DeclareFontSeriesChangeRule {lc}{b}    {bc}   {}
\DeclareFontSeriesChangeRule {lc}{eb}   {ebc}  {bc}
\DeclareFontSeriesChangeRule {lc}{ub}   {ubc}  {bc}
\DeclareFontSeriesChangeRule {lc}{uc}   {luc}  {}
\DeclareFontSeriesChangeRule {lc}{ec}   {lec}  {}
\DeclareFontSeriesChangeRule {lc}{c}    {lc}   {}
\DeclareFontSeriesChangeRule {lc}{sc}   {lsc}  {}
\DeclareFontSeriesChangeRule {lc}{sx}   {lsx}  {}
\DeclareFontSeriesChangeRule {lc}{x}    {lx}   {}
\DeclareFontSeriesChangeRule {lc}{ex}   {lex}  {}
\DeclareFontSeriesChangeRule {lc}{ux}   {lux}  {}

\DeclareFontSeriesChangeRule {lsc}{ul}  {ulsc} {}
\DeclareFontSeriesChangeRule {lsc}{el}  {elsc} {}
\DeclareFontSeriesChangeRule {lsc}{l}   {lsc}  {}
\DeclareFontSeriesChangeRule {lsc}{sl}  {slsc} {}
\DeclareFontSeriesChangeRule {lsc}{sb}  {sbsc} {bsc}
\DeclareFontSeriesChangeRule {lsc}{b}   {bsc}  {}
\DeclareFontSeriesChangeRule {lsc}{eb}  {ebsc} {bsc}
\DeclareFontSeriesChangeRule {lsc}{ub}  {ubsc} {bsc}
\DeclareFontSeriesChangeRule {lsc}{uc}  {luc}  {}
\DeclareFontSeriesChangeRule {lsc}{ec}  {lec}  {}
\DeclareFontSeriesChangeRule {lsc}{c}   {lc}   {}
\DeclareFontSeriesChangeRule {lsc}{sc}  {lsc}  {}
\DeclareFontSeriesChangeRule {lsc}{sx}  {lsx}  {}
\DeclareFontSeriesChangeRule {lsc}{x}   {lx}   {}
\DeclareFontSeriesChangeRule {lsc}{ex}  {lex}  {}
\DeclareFontSeriesChangeRule {lsc}{ux}  {lux}  {}

\DeclareFontSeriesChangeRule {l}{uc}    {luc}  {}
\DeclareFontSeriesChangeRule {l}{ec}    {lec}  {}
\DeclareFontSeriesChangeRule {l}{c}     {lc}   {}
\DeclareFontSeriesChangeRule {l}{sc}    {lsc}  {}
\DeclareFontSeriesChangeRule {l}{sx}    {lsx}  {}
\DeclareFontSeriesChangeRule {l}{x}     {lx}   {}
\DeclareFontSeriesChangeRule {l}{ex}    {lex}  {}
\DeclareFontSeriesChangeRule {l}{ux}    {lux}  {}
```

```
\DeclareFontSeriesChangeRule {l}{sb}    {sb}   {b}
\DeclareFontSeriesChangeRule {l}{eb}    {eb}   {b}
\DeclareFontSeriesChangeRule {l}{ub}    {ub}   {b}

\DeclareFontSeriesChangeRule {lsx}{ul}  {ulsx} {}
\DeclareFontSeriesChangeRule {lsx}{el}  {elsx} {}
\DeclareFontSeriesChangeRule {lsx}{l}   {lsx}  {}
\DeclareFontSeriesChangeRule {lsx}{sl}  {slsx} {}
\DeclareFontSeriesChangeRule {lsx}{sb}  {sbsx} {bsx}
\DeclareFontSeriesChangeRule {lsx}{b}   {bsx}  {}
\DeclareFontSeriesChangeRule {lsx}{eb}  {ebsx} {bsx}
\DeclareFontSeriesChangeRule {lsx}{ub}  {ubsx} {bsx}
\DeclareFontSeriesChangeRule {lsx}{uc}  {luc}  {}
\DeclareFontSeriesChangeRule {lsx}{ec}  {lec}  {}
\DeclareFontSeriesChangeRule {lsx}{c}   {lc}   {}
\DeclareFontSeriesChangeRule {lsx}{sc}  {lsc}  {}
\DeclareFontSeriesChangeRule {lsx}{sx}  {lsx}  {}
\DeclareFontSeriesChangeRule {lsx}{x}   {lx}   {}
\DeclareFontSeriesChangeRule {lsx}{ex}  {lex}  {}
\DeclareFontSeriesChangeRule {lsx}{ux}  {lux}  {}

\DeclareFontSeriesChangeRule {lx}{ul}   {ulx}  {}
\DeclareFontSeriesChangeRule {lx}{el}   {elx}  {}
\DeclareFontSeriesChangeRule {lx}{l}    {lx}   {}
\DeclareFontSeriesChangeRule {lx}{sl}   {slx}  {}
\DeclareFontSeriesChangeRule {lx}{sb}   {sbx}  {bx}
\DeclareFontSeriesChangeRule {lx}{b}    {bx}   {}
\DeclareFontSeriesChangeRule {lx}{eb}   {ebx}  {bx}
\DeclareFontSeriesChangeRule {lx}{ub}   {ubx}  {bx}
\DeclareFontSeriesChangeRule {lx}{uc}   {luc}  {}
\DeclareFontSeriesChangeRule {lx}{ec}   {lec}  {}
\DeclareFontSeriesChangeRule {lx}{c}    {lc}   {}
\DeclareFontSeriesChangeRule {lx}{sc}   {lsc}  {}
\DeclareFontSeriesChangeRule {lx}{sx}   {lsx}  {}
\DeclareFontSeriesChangeRule {lx}{x}    {lx}   {}
\DeclareFontSeriesChangeRule {lx}{ex}   {lex}  {}
\DeclareFontSeriesChangeRule {lx}{ux}   {lux}  {}

\DeclareFontSeriesChangeRule {lex}{ul}  {ulex} {}
\DeclareFontSeriesChangeRule {lex}{el}  {elex} {}
\DeclareFontSeriesChangeRule {lex}{l}   {lex}  {}
\DeclareFontSeriesChangeRule {lex}{sl}  {slex} {}
\DeclareFontSeriesChangeRule {lex}{sb}  {sbex} {bex}
\DeclareFontSeriesChangeRule {lex}{b}   {bex}  {}
\DeclareFontSeriesChangeRule {lex}{eb}  {ebex} {bex}
\DeclareFontSeriesChangeRule {lex}{ub}  {ubex} {bex}
\DeclareFontSeriesChangeRule {lex}{uc}  {luc}  {}
\DeclareFontSeriesChangeRule {lex}{ec}  {lec}  {}
\DeclareFontSeriesChangeRule {lex}{c}   {lc}   {}
\DeclareFontSeriesChangeRule {lex}{sc}  {lsc}  {}
\DeclareFontSeriesChangeRule {lex}{sx}  {lsx}  {}
\DeclareFontSeriesChangeRule {lex}{x}   {lx}   {}
\DeclareFontSeriesChangeRule {lex}{ex}  {lex}  {}
\DeclareFontSeriesChangeRule {lex}{ux}  {lux}  {}

\DeclareFontSeriesChangeRule {lux}{ul}  {ulux} {}
\DeclareFontSeriesChangeRule {lux}{el}  {elux} {}
```

```
\DeclareFontSeriesChangeRule {lux}{l}   {lux}  {}
\DeclareFontSeriesChangeRule {lux}{sl}  {slux} {}
\DeclareFontSeriesChangeRule {lux}{sb}  {sbux} {bux}
\DeclareFontSeriesChangeRule {lux}{b}   {bux}  {}
\DeclareFontSeriesChangeRule {lux}{eb}  {ebux} {bux}
\DeclareFontSeriesChangeRule {lux}{ub}  {ubux} {bux}
\DeclareFontSeriesChangeRule {lux}{uc}  {luc}  {}
\DeclareFontSeriesChangeRule {lux}{ec}  {lec}  {}
\DeclareFontSeriesChangeRule {lux}{c}   {lc}   {}
\DeclareFontSeriesChangeRule {lux}{sc}  {lsc}  {}
\DeclareFontSeriesChangeRule {lux}{sx}  {lsx}  {}
\DeclareFontSeriesChangeRule {lux}{x}   {lx}   {}
\DeclareFontSeriesChangeRule {lux}{ex}  {lex}  {}
\DeclareFontSeriesChangeRule {lux}{ux}  {lux}  {}

\DeclareFontSeriesChangeRule {sluc}{ul} {uluc} {}
\DeclareFontSeriesChangeRule {sluc}{el} {eluc} {}
\DeclareFontSeriesChangeRule {sluc}{l}  {luc}  {}
\DeclareFontSeriesChangeRule {sluc}{sl} {sluc} {}
\DeclareFontSeriesChangeRule {sluc}{sb} {sbuc} {buc}
\DeclareFontSeriesChangeRule {sluc}{b}  {buc}  {}
\DeclareFontSeriesChangeRule {sluc}{eb} {ebuc} {buc}
\DeclareFontSeriesChangeRule {sluc}{ub} {ubuc} {buc}
\DeclareFontSeriesChangeRule {sluc}{uc} {sluc} {}
\DeclareFontSeriesChangeRule {sluc}{ec} {slec} {}
\DeclareFontSeriesChangeRule {sluc}{c}  {slc}  {}
\DeclareFontSeriesChangeRule {sluc}{sc} {slsc} {}
\DeclareFontSeriesChangeRule {sluc}{sx} {slsx} {}
\DeclareFontSeriesChangeRule {sluc}{x}  {slx}  {}
\DeclareFontSeriesChangeRule {sluc}{ex} {slex} {}
\DeclareFontSeriesChangeRule {sluc}{ux} {slux} {}

\DeclareFontSeriesChangeRule {slec}{ul} {ulec} {}
\DeclareFontSeriesChangeRule {slec}{el} {elec} {}
\DeclareFontSeriesChangeRule {slec}{l}  {lec}  {}
\DeclareFontSeriesChangeRule {slec}{sl} {slec} {}
\DeclareFontSeriesChangeRule {slec}{sb} {sbec} {bec}
\DeclareFontSeriesChangeRule {slec}{b}  {bec}  {}
\DeclareFontSeriesChangeRule {slec}{eb} {ebec} {bec}
\DeclareFontSeriesChangeRule {slec}{ub} {ubec} {bec}
\DeclareFontSeriesChangeRule {slec}{uc} {sluc} {}
\DeclareFontSeriesChangeRule {slec}{ec} {slec} {}
\DeclareFontSeriesChangeRule {slec}{c}  {slc}  {}
\DeclareFontSeriesChangeRule {slec}{sc} {slsc} {}
\DeclareFontSeriesChangeRule {slec}{sx} {slsx} {}
\DeclareFontSeriesChangeRule {slec}{x}  {slx}  {}
\DeclareFontSeriesChangeRule {slec}{ex} {slex} {}
\DeclareFontSeriesChangeRule {slec}{ux} {slux} {}

\DeclareFontSeriesChangeRule {slc}{ul}  {ulc}  {}
\DeclareFontSeriesChangeRule {slc}{el}  {elc}  {}
\DeclareFontSeriesChangeRule {slc}{l}   {lc}   {}
\DeclareFontSeriesChangeRule {slc}{sl}  {slc}  {}
\DeclareFontSeriesChangeRule {slc}{sb}  {sbc}  {bc}
\DeclareFontSeriesChangeRule {slc}{b}   {bc}   {}
\DeclareFontSeriesChangeRule {slc}{eb}  {ebc}  {bc}
```

```
\DeclareFontSeriesChangeRule {slc}{ub}  {ubc}  {bc}
\DeclareFontSeriesChangeRule {slc}{uc}  {sluc} {}
\DeclareFontSeriesChangeRule {slc}{ec}  {slec} {}
\DeclareFontSeriesChangeRule {slc}{c}   {slc}  {}
\DeclareFontSeriesChangeRule {slc}{sc}  {slsc} {}
\DeclareFontSeriesChangeRule {slc}{sx}  {slsx} {}
\DeclareFontSeriesChangeRule {slc}{x}   {slx}  {}
\DeclareFontSeriesChangeRule {slc}{ex}  {slex} {}
\DeclareFontSeriesChangeRule {slc}{ux}  {slux} {}

\DeclareFontSeriesChangeRule {slsc}{ul} {ulsc} {}
\DeclareFontSeriesChangeRule {slsc}{el} {elsc} {}
\DeclareFontSeriesChangeRule {slsc}{l}  {lsc}  {}
\DeclareFontSeriesChangeRule {slsc}{sl} {slsc} {}
\DeclareFontSeriesChangeRule {slsc}{sb} {sbsc} {bsc}
\DeclareFontSeriesChangeRule {slsc}{b}  {bsc}  {}
\DeclareFontSeriesChangeRule {slsc}{eb} {ebsc} {bsc}
\DeclareFontSeriesChangeRule {slsc}{ub} {ubsc} {bsc}
\DeclareFontSeriesChangeRule {slsc}{uc} {sluc} {}
\DeclareFontSeriesChangeRule {slsc}{ec} {slec} {}
\DeclareFontSeriesChangeRule {slsc}{c}  {slc}  {}
\DeclareFontSeriesChangeRule {slsc}{sc} {slsc} {}
\DeclareFontSeriesChangeRule {slsc}{sx} {slsx} {}
\DeclareFontSeriesChangeRule {slsc}{x}  {slx}  {}
\DeclareFontSeriesChangeRule {slsc}{ex} {slex} {}
\DeclareFontSeriesChangeRule {slsc}{ux} {slux} {}

\DeclareFontSeriesChangeRule {sl}{uc}   {sluc} {}
\DeclareFontSeriesChangeRule {sl}{ec}   {slec} {}
\DeclareFontSeriesChangeRule {sl}{c}    {slc}  {}
\DeclareFontSeriesChangeRule {sl}{sc}   {slsc} {}
\DeclareFontSeriesChangeRule {sl}{sx}   {slsx} {}
\DeclareFontSeriesChangeRule {sl}{x}    {slx}  {}
\DeclareFontSeriesChangeRule {sl}{ex}   {slex} {}
\DeclareFontSeriesChangeRule {sl}{ux}   {slux} {}
\DeclareFontSeriesChangeRule {sl}{sb}   {sb}   {b}
\DeclareFontSeriesChangeRule {sl}{eb}   {eb}   {b}
\DeclareFontSeriesChangeRule {sl}{ub}   {ub}   {b}

\DeclareFontSeriesChangeRule {slsx}{ul} {ulsx} {}
\DeclareFontSeriesChangeRule {slsx}{el} {elsx} {}
\DeclareFontSeriesChangeRule {slsx}{l}  {lsx}  {}
\DeclareFontSeriesChangeRule {slsx}{sl} {slsx} {}
\DeclareFontSeriesChangeRule {slsx}{sb} {sbsx} {bsx}
\DeclareFontSeriesChangeRule {slsx}{b}  {bsx}  {}
\DeclareFontSeriesChangeRule {slsx}{eb} {ebsx} {bsx}
\DeclareFontSeriesChangeRule {slsx}{ub} {ubsx} {bsx}
\DeclareFontSeriesChangeRule {slsx}{uc} {sluc} {}
\DeclareFontSeriesChangeRule {slsx}{ec} {slec} {}
\DeclareFontSeriesChangeRule {slsx}{c}  {slc}  {}
\DeclareFontSeriesChangeRule {slsx}{sc} {slsc} {}
\DeclareFontSeriesChangeRule {slsx}{sx} {slsx} {}
\DeclareFontSeriesChangeRule {slsx}{x}  {slx}  {}
\DeclareFontSeriesChangeRule {slsx}{ex} {slex} {}
\DeclareFontSeriesChangeRule {slsx}{ux} {slux} {}

\DeclareFontSeriesChangeRule {slx}{ul}  {ulx}  {}
```

```
\DeclareFontSeriesChangeRule {slx}{el}  {elx}  {}
\DeclareFontSeriesChangeRule {slx}{l}   {lx}   {}
\DeclareFontSeriesChangeRule {slx}{sl}  {slx}  {}
\DeclareFontSeriesChangeRule {slx}{sb}  {sbx}  {bx}
\DeclareFontSeriesChangeRule {slx}{b}   {bx}   {}
\DeclareFontSeriesChangeRule {slx}{eb}  {ebx}  {bx}
\DeclareFontSeriesChangeRule {slx}{ub}  {ubx}  {bx}
\DeclareFontSeriesChangeRule {slx}{uc}  {sluc} {}
\DeclareFontSeriesChangeRule {slx}{ec}  {slec} {}
\DeclareFontSeriesChangeRule {slx}{c}   {slc}  {}
\DeclareFontSeriesChangeRule {slx}{sc}  {slsc} {}
\DeclareFontSeriesChangeRule {slx}{sx}  {slsx} {}
\DeclareFontSeriesChangeRule {slx}{x}   {slx}  {}
\DeclareFontSeriesChangeRule {slx}{ex}  {slex} {}
\DeclareFontSeriesChangeRule {slx}{ux}  {slux} {}

\DeclareFontSeriesChangeRule {slex}{ul} {ulex} {}
\DeclareFontSeriesChangeRule {slex}{el} {elex} {}
\DeclareFontSeriesChangeRule {slex}{l}  {lex}  {}
\DeclareFontSeriesChangeRule {slex}{sl} {slex} {}
\DeclareFontSeriesChangeRule {slex}{sb} {sbex} {bex}
\DeclareFontSeriesChangeRule {slex}{b}  {bex}  {}
\DeclareFontSeriesChangeRule {slex}{eb} {ebex} {bex}
\DeclareFontSeriesChangeRule {slex}{ub} {ubex} {bex}
\DeclareFontSeriesChangeRule {slex}{uc} {sluc} {}
\DeclareFontSeriesChangeRule {slex}{ec} {slec} {}
\DeclareFontSeriesChangeRule {slex}{c}  {slc}  {}
\DeclareFontSeriesChangeRule {slex}{sc} {slsc} {}
\DeclareFontSeriesChangeRule {slex}{sx} {slsx} {}
\DeclareFontSeriesChangeRule {slex}{x}  {slx}  {}
\DeclareFontSeriesChangeRule {slex}{ex} {slex} {}
\DeclareFontSeriesChangeRule {slex}{ux} {slux} {}

\DeclareFontSeriesChangeRule {slux}{ul} {ulux} {}
\DeclareFontSeriesChangeRule {slux}{el} {elux} {}
\DeclareFontSeriesChangeRule {slux}{l}  {lux}  {}
\DeclareFontSeriesChangeRule {slux}{sl} {slux} {}
\DeclareFontSeriesChangeRule {slux}{sb} {sbux} {bux}
\DeclareFontSeriesChangeRule {slux}{b}  {bux}  {}
\DeclareFontSeriesChangeRule {slux}{eb} {ebux} {bux}
\DeclareFontSeriesChangeRule {slux}{ub} {ubux} {bux}
\DeclareFontSeriesChangeRule {slux}{uc} {sluc} {}
\DeclareFontSeriesChangeRule {slux}{ec} {slec} {}
\DeclareFontSeriesChangeRule {slux}{c}  {slc}  {}
\DeclareFontSeriesChangeRule {slux}{sc} {slsc} {}
\DeclareFontSeriesChangeRule {slux}{sx} {slsx} {}
\DeclareFontSeriesChangeRule {slux}{x}  {slx}  {}
\DeclareFontSeriesChangeRule {slux}{ex} {slex} {}
\DeclareFontSeriesChangeRule {slux}{ux} {slux} {}

\DeclareFontSeriesChangeRule {uc}{ul}   {uluc} {}
\DeclareFontSeriesChangeRule {uc}{el}   {eluc} {}
\DeclareFontSeriesChangeRule {uc}{l}    {luc}  {}
\DeclareFontSeriesChangeRule {uc}{sl}   {sluc} {}
\DeclareFontSeriesChangeRule {uc}{sb}   {sbuc} {buc}
\DeclareFontSeriesChangeRule {uc}{b}    {buc}  {}
```

```
\DeclareFontSeriesChangeRule {uc}{eb}   {ebuc} {buc}
\DeclareFontSeriesChangeRule {uc}{ub}   {ubuc} {buc}

\DeclareFontSeriesChangeRule {ec}{ul}   {ulec} {}
\DeclareFontSeriesChangeRule {ec}{el}   {elec} {}
\DeclareFontSeriesChangeRule {ec}{l}    {lec}  {}
\DeclareFontSeriesChangeRule {ec}{sl}   {slec} {}
\DeclareFontSeriesChangeRule {ec}{sb}   {sbec} {bec}
\DeclareFontSeriesChangeRule {ec}{b}    {bec}  {}
\DeclareFontSeriesChangeRule {ec}{eb}   {ebec} {bec}
\DeclareFontSeriesChangeRule {ec}{ub}   {ubec} {bec}
```

There are a number of font families that implement condensed series, but often only `c` and `bc`. Therefore, if we see a weight or width change request that can't be fulfilled we try to stay with `c` or `bc`.

```
\DeclareFontSeriesChangeRule {c}{ul}    {ulc}  {c}
\DeclareFontSeriesChangeRule {c}{el}    {elc}  {c}
\DeclareFontSeriesChangeRule {c}{l}     {lc}   {c}
\DeclareFontSeriesChangeRule {c}{sl}    {slc}  {c}
\DeclareFontSeriesChangeRule {c}{sb}    {sbc}  {bc}
\DeclareFontSeriesChangeRule {c}{b}     {bc}   {}
\DeclareFontSeriesChangeRule {c}{eb}    {ebc}  {bc}
\DeclareFontSeriesChangeRule {c}{ub}    {ubc}  {bc}
\DeclareFontSeriesChangeRule {c}{uc}    {uc}   {c}
\DeclareFontSeriesChangeRule {c}{ec}    {ec}   {c}
\DeclareFontSeriesChangeRule {c}{sc}    {sc}   {c}

\DeclareFontSeriesChangeRule {sc}{ul}   {ulsc} {}
\DeclareFontSeriesChangeRule {sc}{el}   {elsc} {}
\DeclareFontSeriesChangeRule {sc}{l}    {lsc}  {}
\DeclareFontSeriesChangeRule {sc}{sl}   {slsc} {}
\DeclareFontSeriesChangeRule {sc}{sb}   {sbsc} {bsc}
\DeclareFontSeriesChangeRule {sc}{b}    {bsc}  {}
\DeclareFontSeriesChangeRule {sc}{eb}   {ebsc} {bsc}
\DeclareFontSeriesChangeRule {sc}{ub}   {ubsc} {bsc}

\DeclareFontSeriesChangeRule {m}{sb}    {sb}   {b}
\DeclareFontSeriesChangeRule {m}{eb}    {eb}   {b}
\DeclareFontSeriesChangeRule {m}{ub}    {ub}   {b}
```

This special rule normally does nothing since nearly every font implements the `b` bold series. The exception are Computer Modern and Latin Modern and fonts based on them. They usually only have `bx`, but then they normally provide an `.fd` file declaration mapping from `b` to `bx` and thus pretend that `b` exists. But in case any of them does not, we offer an alternative result and switch to `bx` if `b` can't be found.

```
\DeclareFontSeriesChangeRule {m}{b}     {b}    {bx}

\DeclareFontSeriesChangeRule {sx}{ul}   {ulsx} {}
\DeclareFontSeriesChangeRule {sx}{el}   {elsx} {}
\DeclareFontSeriesChangeRule {sx}{l}    {lsx}  {}
\DeclareFontSeriesChangeRule {sx}{sl}   {slsx} {}
\DeclareFontSeriesChangeRule {sx}{sb}   {sbsx} {bsx}
\DeclareFontSeriesChangeRule {sx}{b}    {bsx}  {}
\DeclareFontSeriesChangeRule {sx}{eb}   {ebsx} {bsx}
\DeclareFontSeriesChangeRule {sx}{ub}   {ubsx} {bsx}
```

```
\DeclareFontSeriesChangeRule {x}{ul}    {ulx}  {}
\DeclareFontSeriesChangeRule {x}{el}    {elx}  {}
\DeclareFontSeriesChangeRule {x}{l}     {lx}   {}
\DeclareFontSeriesChangeRule {x}{sl}    {slx}  {}
\DeclareFontSeriesChangeRule {x}{sb}    {sbx}  {bx}
\DeclareFontSeriesChangeRule {x}{b}     {bx}   {}
\DeclareFontSeriesChangeRule {x}{eb}    {ebx}  {bx}
\DeclareFontSeriesChangeRule {x}{ub}    {ubx}  {bx}

\DeclareFontSeriesChangeRule {ex}{ul}   {ulex} {}
\DeclareFontSeriesChangeRule {ex}{el}   {elex} {}
\DeclareFontSeriesChangeRule {ex}{l}    {lex}  {}
\DeclareFontSeriesChangeRule {ex}{sl}   {slex} {}
\DeclareFontSeriesChangeRule {ex}{sb}   {sbex} {bex}
\DeclareFontSeriesChangeRule {ex}{b}    {bex}  {}
\DeclareFontSeriesChangeRule {ex}{eb}   {ebex} {bex}
\DeclareFontSeriesChangeRule {ex}{ub}   {ubex} {bex}

\DeclareFontSeriesChangeRule {ux}{ul}   {ulux} {}
\DeclareFontSeriesChangeRule {ux}{el}   {elux} {}
\DeclareFontSeriesChangeRule {ux}{l}    {lux}  {}
\DeclareFontSeriesChangeRule {ux}{sl}   {slux} {}
\DeclareFontSeriesChangeRule {ux}{sb}   {sbux} {bux}
\DeclareFontSeriesChangeRule {ux}{b}    {bux}  {}
\DeclareFontSeriesChangeRule {ux}{eb}   {ebux} {bux}
\DeclareFontSeriesChangeRule {ux}{ub}   {ubux} {bux}

\DeclareFontSeriesChangeRule {sbuc}{ul} {uluc} {}
\DeclareFontSeriesChangeRule {sbuc}{el} {eluc} {}
\DeclareFontSeriesChangeRule {sbuc}{l}  {luc}  {}
\DeclareFontSeriesChangeRule {sbuc}{sl} {sluc} {}
\DeclareFontSeriesChangeRule {sbuc}{sb} {sbuc} {}
\DeclareFontSeriesChangeRule {sbuc}{b}  {buc}  {}
\DeclareFontSeriesChangeRule {sbuc}{eb} {ebuc} {buc}
\DeclareFontSeriesChangeRule {sbuc}{ub} {ubuc} {buc}
\DeclareFontSeriesChangeRule {sbuc}{uc} {sbuc} {}
\DeclareFontSeriesChangeRule {sbuc}{ec} {sbec} {}
\DeclareFontSeriesChangeRule {sbuc}{c}  {sbc}  {}
\DeclareFontSeriesChangeRule {sbuc}{sc} {sbsc} {}
\DeclareFontSeriesChangeRule {sbuc}{sx} {sbsx} {}
\DeclareFontSeriesChangeRule {sbuc}{x}  {sbx}  {}
\DeclareFontSeriesChangeRule {sbuc}{ex} {sbex} {}
\DeclareFontSeriesChangeRule {sbuc}{ux} {sbux} {}

\DeclareFontSeriesChangeRule {sbec}{ul} {ulec} {}
\DeclareFontSeriesChangeRule {sbec}{el} {elec} {}
\DeclareFontSeriesChangeRule {sbec}{l}  {lec}  {}
\DeclareFontSeriesChangeRule {sbec}{sl} {slec} {}
\DeclareFontSeriesChangeRule {sbec}{sb} {sbec} {}
\DeclareFontSeriesChangeRule {sbec}{b}  {bec}  {}
\DeclareFontSeriesChangeRule {sbec}{eb} {ebec} {bec}
\DeclareFontSeriesChangeRule {sbec}{ub} {ubec} {bec}
\DeclareFontSeriesChangeRule {sbec}{uc} {sbuc} {}
\DeclareFontSeriesChangeRule {sbec}{ec} {sbec} {}
\DeclareFontSeriesChangeRule {sbec}{c}  {sbc}  {}
\DeclareFontSeriesChangeRule {sbec}{sc} {sbsc} {}
\DeclareFontSeriesChangeRule {sbec}{sx} {sbsx} {}
```

```
\DeclareFontSeriesChangeRule {sbec}{x}  {sbx}  {}
\DeclareFontSeriesChangeRule {sbec}{ex} {sbex} {}
\DeclareFontSeriesChangeRule {sbec}{ux} {sbux} {}

\DeclareFontSeriesChangeRule {sbc}{ul}  {ulc}  {}
\DeclareFontSeriesChangeRule {sbc}{el}  {elc}  {}
\DeclareFontSeriesChangeRule {sbc}{l}   {lc}   {}
\DeclareFontSeriesChangeRule {sbc}{sl}  {slc}  {}
\DeclareFontSeriesChangeRule {sbc}{sb}  {sbc}  {}
\DeclareFontSeriesChangeRule {sbc}{b}   {bc}   {}
\DeclareFontSeriesChangeRule {sbc}{eb}  {ebc}  {bc}
\DeclareFontSeriesChangeRule {sbc}{ub}  {ubc}  {bc}
\DeclareFontSeriesChangeRule {sbc}{uc}  {sbuc} {}
\DeclareFontSeriesChangeRule {sbc}{ec}  {sbec} {}
\DeclareFontSeriesChangeRule {sbc}{c}   {sbc}  {}
\DeclareFontSeriesChangeRule {sbc}{sc}  {sbsc} {}
\DeclareFontSeriesChangeRule {sbc}{sx}  {sbsx} {}
\DeclareFontSeriesChangeRule {sbc}{x}   {sbx}  {}
\DeclareFontSeriesChangeRule {sbc}{ex}  {sbex} {}
\DeclareFontSeriesChangeRule {sbc}{ux}  {sbux} {}

\DeclareFontSeriesChangeRule {sbsc}{ul} {ulsc} {}
\DeclareFontSeriesChangeRule {sbsc}{el} {elsc} {}
\DeclareFontSeriesChangeRule {sbsc}{l}  {lsc}  {}
\DeclareFontSeriesChangeRule {sbsc}{sl} {slsc} {}
\DeclareFontSeriesChangeRule {sbsc}{sb} {sbsc} {}
\DeclareFontSeriesChangeRule {sbsc}{b}  {bsc}  {}
\DeclareFontSeriesChangeRule {sbsc}{eb} {ebsc} {bsc}
\DeclareFontSeriesChangeRule {sbsc}{ub} {ubsc} {bsc}
\DeclareFontSeriesChangeRule {sbsc}{uc} {sbuc} {}
\DeclareFontSeriesChangeRule {sbsc}{ec} {sbec} {}
\DeclareFontSeriesChangeRule {sbsc}{c}  {sbc}  {}
\DeclareFontSeriesChangeRule {sbsc}{sc} {sbsc} {}
\DeclareFontSeriesChangeRule {sbsc}{sx} {sbsx} {}
\DeclareFontSeriesChangeRule {sbsc}{x}  {sbx}  {}
\DeclareFontSeriesChangeRule {sbsc}{ex} {sbex} {}
\DeclareFontSeriesChangeRule {sbsc}{ux} {sbux} {}

\DeclareFontSeriesChangeRule {sb}{uc}   {sbuc} {}
\DeclareFontSeriesChangeRule {sb}{ec}   {sbec} {}
\DeclareFontSeriesChangeRule {sb}{c}    {sbc}  {}
\DeclareFontSeriesChangeRule {sb}{sc}   {sbsc} {}
\DeclareFontSeriesChangeRule {sb}{sx}   {sbsx} {}
\DeclareFontSeriesChangeRule {sb}{x}    {sbx}  {}
\DeclareFontSeriesChangeRule {sb}{ex}   {sbex} {}
\DeclareFontSeriesChangeRule {sb}{ux}   {sbux} {}
\DeclareFontSeriesChangeRule {sb}{eb}   {eb}   {b}
\DeclareFontSeriesChangeRule {sb}{ub}   {ub}   {b}

\DeclareFontSeriesChangeRule {sbsx}{ul} {ulsx} {}
\DeclareFontSeriesChangeRule {sbsx}{el} {elsx} {}
\DeclareFontSeriesChangeRule {sbsx}{l}  {lsx}  {}
\DeclareFontSeriesChangeRule {sbsx}{sl} {slsx} {}
\DeclareFontSeriesChangeRule {sbsx}{sb} {sbsx} {}
\DeclareFontSeriesChangeRule {sbsx}{b}  {bsx}  {}
\DeclareFontSeriesChangeRule {sbsx}{eb} {ebsx} {bsx}
\DeclareFontSeriesChangeRule {sbsx}{ub} {ubsx} {bsx}
```

```
\DeclareFontSeriesChangeRule {sbsx}{uc} {sbuc} {}
\DeclareFontSeriesChangeRule {sbsx}{ec} {sbec} {}
\DeclareFontSeriesChangeRule {sbsx}{c}  {sbc}  {}
\DeclareFontSeriesChangeRule {sbsx}{sc} {sbsc} {}
\DeclareFontSeriesChangeRule {sbsx}{sx} {sbsx} {}
\DeclareFontSeriesChangeRule {sbsx}{x}  {sbx}  {}
\DeclareFontSeriesChangeRule {sbsx}{ex} {sbex} {}
\DeclareFontSeriesChangeRule {sbsx}{ux} {sbux} {}

\DeclareFontSeriesChangeRule {sbx}{ul}  {ulx}  {}
\DeclareFontSeriesChangeRule {sbx}{el}  {elx}  {}
\DeclareFontSeriesChangeRule {sbx}{l}   {lx}   {}
\DeclareFontSeriesChangeRule {sbx}{sl}  {slx}  {}
\DeclareFontSeriesChangeRule {sbx}{sb}  {sbx}  {}
\DeclareFontSeriesChangeRule {sbx}{b}   {bx}   {}
\DeclareFontSeriesChangeRule {sbx}{eb}  {ebx}  {bx}
\DeclareFontSeriesChangeRule {sbx}{ub}  {ubx}  {bx}
\DeclareFontSeriesChangeRule {sbx}{uc}  {sbuc} {}
\DeclareFontSeriesChangeRule {sbx}{ec}  {sbec} {}
\DeclareFontSeriesChangeRule {sbx}{c}   {sbc}  {}
\DeclareFontSeriesChangeRule {sbx}{sc}  {sbsc} {}
\DeclareFontSeriesChangeRule {sbx}{sx}  {sbsx} {}
\DeclareFontSeriesChangeRule {sbx}{x}   {sbx}  {}
\DeclareFontSeriesChangeRule {sbx}{ex}  {sbex} {}
\DeclareFontSeriesChangeRule {sbx}{ux}  {sbux} {}

\DeclareFontSeriesChangeRule {sbex}{ul} {ulex} {}
\DeclareFontSeriesChangeRule {sbex}{el} {elex} {}
\DeclareFontSeriesChangeRule {sbex}{l}  {lex}  {}
\DeclareFontSeriesChangeRule {sbex}{sl} {slex} {}
\DeclareFontSeriesChangeRule {sbex}{sb} {sbex} {}
\DeclareFontSeriesChangeRule {sbex}{b}  {bex}  {}
\DeclareFontSeriesChangeRule {sbex}{eb} {ebex} {bex}
\DeclareFontSeriesChangeRule {sbex}{ub} {ubex} {bex}
\DeclareFontSeriesChangeRule {sbex}{uc} {sbuc} {}
\DeclareFontSeriesChangeRule {sbex}{ec} {sbec} {}
\DeclareFontSeriesChangeRule {sbex}{c}  {sbc}  {}
\DeclareFontSeriesChangeRule {sbex}{sc} {sbsc} {}
\DeclareFontSeriesChangeRule {sbex}{sx} {sbsx} {}
\DeclareFontSeriesChangeRule {sbex}{x}  {sbx}  {}
\DeclareFontSeriesChangeRule {sbex}{ex} {sbex} {}
\DeclareFontSeriesChangeRule {sbex}{ux} {sbux} {}

\DeclareFontSeriesChangeRule {sbux}{ul} {ulux} {}
\DeclareFontSeriesChangeRule {sbux}{el} {elux} {}
\DeclareFontSeriesChangeRule {sbux}{l}  {lux}  {}
\DeclareFontSeriesChangeRule {sbux}{sl} {slux} {}
\DeclareFontSeriesChangeRule {sbux}{sb} {sbux} {}
\DeclareFontSeriesChangeRule {sbux}{b}  {bux}  {}
\DeclareFontSeriesChangeRule {sbux}{eb} {ebux} {bux}
\DeclareFontSeriesChangeRule {sbux}{ub} {ubux} {bux}
\DeclareFontSeriesChangeRule {sbux}{uc} {sbuc} {}
\DeclareFontSeriesChangeRule {sbux}{ec} {sbec} {}
\DeclareFontSeriesChangeRule {sbux}{c}  {sbc}  {}
\DeclareFontSeriesChangeRule {sbux}{sc} {sbsc} {}
\DeclareFontSeriesChangeRule {sbux}{sx} {sbsx} {}
```

```
\DeclareFontSeriesChangeRule {sbux}{x}  {sbx}  {}
\DeclareFontSeriesChangeRule {sbux}{ex} {sbex} {}
\DeclareFontSeriesChangeRule {sbux}{ux} {sbux} {}

\DeclareFontSeriesChangeRule {buc}{ul}  {uluc} {}
\DeclareFontSeriesChangeRule {buc}{el}  {eluc} {}
\DeclareFontSeriesChangeRule {buc}{l}   {luc}  {}
\DeclareFontSeriesChangeRule {buc}{sl}  {sluc} {}
\DeclareFontSeriesChangeRule {buc}{sb}  {sbuc} {buc}
\DeclareFontSeriesChangeRule {buc}{b}   {buc}  {}
\DeclareFontSeriesChangeRule {buc}{eb}  {ebuc} {buc}
\DeclareFontSeriesChangeRule {buc}{ub}  {ubuc} {buc}
\DeclareFontSeriesChangeRule {buc}{uc}  {buc}  {}
\DeclareFontSeriesChangeRule {buc}{ec}  {bec}  {}
\DeclareFontSeriesChangeRule {buc}{c}   {bc}   {}
\DeclareFontSeriesChangeRule {buc}{sc}  {bsc}  {}
\DeclareFontSeriesChangeRule {buc}{sx}  {bsx}  {}
\DeclareFontSeriesChangeRule {buc}{x}   {bx}   {}
\DeclareFontSeriesChangeRule {buc}{ex}  {bex}  {}
\DeclareFontSeriesChangeRule {buc}{ux}  {bux}  {}

\DeclareFontSeriesChangeRule {bec}{ul}  {ulec} {}
\DeclareFontSeriesChangeRule {bec}{el}  {elec} {}
\DeclareFontSeriesChangeRule {bec}{l}   {lec}  {}
\DeclareFontSeriesChangeRule {bec}{sl}  {slec} {}
\DeclareFontSeriesChangeRule {bec}{sb}  {sbec} {bec}
\DeclareFontSeriesChangeRule {bec}{b}   {bec}  {}
\DeclareFontSeriesChangeRule {bec}{eb}  {ebec} {bec}
\DeclareFontSeriesChangeRule {bec}{ub}  {ubec} {bec}
\DeclareFontSeriesChangeRule {bec}{uc}  {buc}  {}
\DeclareFontSeriesChangeRule {bec}{ec}  {bec}  {}
\DeclareFontSeriesChangeRule {bec}{c}   {bc}   {}
\DeclareFontSeriesChangeRule {bec}{sc}  {bsc}  {}
\DeclareFontSeriesChangeRule {bec}{sx}  {bsx}  {}
\DeclareFontSeriesChangeRule {bec}{x}   {bx}   {}
\DeclareFontSeriesChangeRule {bec}{ex}  {bex}  {}
\DeclareFontSeriesChangeRule {bec}{ux}  {bux}  {}

\DeclareFontSeriesChangeRule {bc}{ul}   {ulc}  {}
\DeclareFontSeriesChangeRule {bc}{el}   {elc}  {}
\DeclareFontSeriesChangeRule {bc}{l}    {lc}   {}
\DeclareFontSeriesChangeRule {bc}{sl}   {slc}  {}
\DeclareFontSeriesChangeRule {bc}{sb}   {sbc}  {bc}
\DeclareFontSeriesChangeRule {bc}{b}    {bc}   {}
\DeclareFontSeriesChangeRule {bc}{eb}   {ebc}  {bc}
\DeclareFontSeriesChangeRule {bc}{ub}   {ubc}  {bc}
\DeclareFontSeriesChangeRule {bc}{uc}   {buc}  {}
\DeclareFontSeriesChangeRule {bc}{ec}   {bec}  {}
\DeclareFontSeriesChangeRule {bc}{c}    {bc}   {}
\DeclareFontSeriesChangeRule {bc}{sc}   {bsc}  {}
\DeclareFontSeriesChangeRule {bc}{sx}   {bsx}  {}
\DeclareFontSeriesChangeRule {bc}{x}    {bx}   {}
\DeclareFontSeriesChangeRule {bc}{ex}   {bex}  {}
\DeclareFontSeriesChangeRule {bc}{ux}   {bux}  {}

\DeclareFontSeriesChangeRule {bsc}{ul}  {ulsc} {}
\DeclareFontSeriesChangeRule {bsc}{el}  {elsc} {}
```

```
\DeclareFontSeriesChangeRule {bsc}{l}   {lsc}  {}
\DeclareFontSeriesChangeRule {bsc}{sl}  {slsc} {}
\DeclareFontSeriesChangeRule {bsc}{sb}  {sbsc} {bsc}
\DeclareFontSeriesChangeRule {bsc}{b}   {bsc}  {}
\DeclareFontSeriesChangeRule {bsc}{eb}  {ebsc} {bsc}
\DeclareFontSeriesChangeRule {bsc}{ub}  {ubsc} {bsc}
\DeclareFontSeriesChangeRule {bsc}{uc}  {buc}  {}
\DeclareFontSeriesChangeRule {bsc}{ec}  {bec}  {}
\DeclareFontSeriesChangeRule {bsc}{c}   {bc}   {}
\DeclareFontSeriesChangeRule {bsc}{sc}  {bsc}  {}
\DeclareFontSeriesChangeRule {bsc}{sx}  {bsx}  {}
\DeclareFontSeriesChangeRule {bsc}{x}   {bx}   {}
\DeclareFontSeriesChangeRule {bsc}{ex}  {bex}  {}
\DeclareFontSeriesChangeRule {bsc}{ux}  {bux}  {}
```

If we are in `b` and a width change is requested that leads to a missing font face we stay in `b` because then the font family probably doesn't implement width changes at all.

```
\DeclareFontSeriesChangeRule {b}{uc}    {buc}  {b}
\DeclareFontSeriesChangeRule {b}{ec}    {bec}  {b}
\DeclareFontSeriesChangeRule {b}{c}     {bc}   {b}
\DeclareFontSeriesChangeRule {b}{sc}    {bsc}  {b}
\DeclareFontSeriesChangeRule {b}{sx}    {bsx}  {b}
\DeclareFontSeriesChangeRule {b}{x}     {bx}   {b}
\DeclareFontSeriesChangeRule {b}{ex}    {bex}  {b}
\DeclareFontSeriesChangeRule {b}{ux}    {bux}  {b}
\DeclareFontSeriesChangeRule {b}{sb}    {sb}   {b}
\DeclareFontSeriesChangeRule {b}{eb}    {eb}   {b}
\DeclareFontSeriesChangeRule {b}{ub}    {ub}   {b}

\DeclareFontSeriesChangeRule {bsx}{ul}  {ulsx} {}
\DeclareFontSeriesChangeRule {bsx}{el}  {elsx} {}
\DeclareFontSeriesChangeRule {bsx}{l}   {lsx}  {}
\DeclareFontSeriesChangeRule {bsx}{sl}  {slsx} {}
\DeclareFontSeriesChangeRule {bsx}{sb}  {sbsx} {bsx}
\DeclareFontSeriesChangeRule {bsx}{b}   {bsx}  {}
\DeclareFontSeriesChangeRule {bsx}{eb}  {ebsx} {bsx}
\DeclareFontSeriesChangeRule {bsx}{ub}  {ubsx} {bsx}
\DeclareFontSeriesChangeRule {bsx}{uc}  {buc}  {}
\DeclareFontSeriesChangeRule {bsx}{ec}  {bec}  {}
\DeclareFontSeriesChangeRule {bsx}{c}   {bc}   {}
\DeclareFontSeriesChangeRule {bsx}{sc}  {bsc}  {}
\DeclareFontSeriesChangeRule {bsx}{sx}  {bsx}  {}
\DeclareFontSeriesChangeRule {bsx}{x}   {bx}   {}
\DeclareFontSeriesChangeRule {bsx}{ex}  {bex}  {}
\DeclareFontSeriesChangeRule {bsx}{ux}  {bux}  {}

\DeclareFontSeriesChangeRule {bx}{ul}   {ulx}  {}
\DeclareFontSeriesChangeRule {bx}{el}   {elx}  {}
\DeclareFontSeriesChangeRule {bx}{l}    {lx}   {}
\DeclareFontSeriesChangeRule {bx}{sl}   {slx}  {}
\DeclareFontSeriesChangeRule {bx}{sb}   {sbx}  {bx}
\DeclareFontSeriesChangeRule {bx}{b}    {bx}   {}
\DeclareFontSeriesChangeRule {bx}{eb}   {ebx}  {bx}
\DeclareFontSeriesChangeRule {bx}{ub}   {ubx}  {bx}
\DeclareFontSeriesChangeRule {bx}{uc}   {buc}  {}
```

```
\DeclareFontSeriesChangeRule {bx}{ec}   {bec}  {}
\DeclareFontSeriesChangeRule {bx}{c}    {bc}   {}
\DeclareFontSeriesChangeRule {bx}{sc}   {bsc}  {}
\DeclareFontSeriesChangeRule {bx}{sx}   {bsx}  {}
\DeclareFontSeriesChangeRule {bx}{x}    {bx}   {}
\DeclareFontSeriesChangeRule {bx}{ex}   {bex}  {}
\DeclareFontSeriesChangeRule {bx}{ux}   {bux}  {}

\DeclareFontSeriesChangeRule {bex}{ul}  {ulex} {}
\DeclareFontSeriesChangeRule {bex}{el}  {elex} {}
\DeclareFontSeriesChangeRule {bex}{l}   {lex}  {}
\DeclareFontSeriesChangeRule {bex}{sl}  {slex} {}
\DeclareFontSeriesChangeRule {bex}{sb}  {sbex} {bex}
\DeclareFontSeriesChangeRule {bex}{b}   {bex}  {}
\DeclareFontSeriesChangeRule {bex}{eb}  {ebex} {bex}
\DeclareFontSeriesChangeRule {bex}{ub}  {ubex} {bex}
\DeclareFontSeriesChangeRule {bex}{uc}  {buc}  {}
\DeclareFontSeriesChangeRule {bex}{ec}  {bec}  {}
\DeclareFontSeriesChangeRule {bex}{c}   {bc}   {}
\DeclareFontSeriesChangeRule {bex}{sc}  {bsc}  {}
\DeclareFontSeriesChangeRule {bex}{sx}  {bsx}  {}
\DeclareFontSeriesChangeRule {bex}{x}   {bx}   {}
\DeclareFontSeriesChangeRule {bex}{ex}  {bex}  {}
\DeclareFontSeriesChangeRule {bex}{ux}  {bux}  {}

\DeclareFontSeriesChangeRule {bux}{ul}  {ulux} {}
\DeclareFontSeriesChangeRule {bux}{el}  {elux} {}
\DeclareFontSeriesChangeRule {bux}{l}   {lux}  {}
\DeclareFontSeriesChangeRule {bux}{sl}  {slux} {}
\DeclareFontSeriesChangeRule {bux}{sb}  {sbux} {bux}
\DeclareFontSeriesChangeRule {bux}{b}   {bux}  {}
\DeclareFontSeriesChangeRule {bux}{eb}  {ebux} {bux}
\DeclareFontSeriesChangeRule {bux}{ub}  {ubux} {bux}
\DeclareFontSeriesChangeRule {bux}{uc}  {buc}  {}
\DeclareFontSeriesChangeRule {bux}{ec}  {bec}  {}
\DeclareFontSeriesChangeRule {bux}{c}   {bc}   {}
\DeclareFontSeriesChangeRule {bux}{sc}  {bsc}  {}
\DeclareFontSeriesChangeRule {bux}{sx}  {bsx}  {}
\DeclareFontSeriesChangeRule {bux}{x}   {bx}   {}
\DeclareFontSeriesChangeRule {bux}{ex}  {bex}  {}
\DeclareFontSeriesChangeRule {bux}{ux}  {bux}  {}

\DeclareFontSeriesChangeRule {ebuc}{ul} {uluc} {}
\DeclareFontSeriesChangeRule {ebuc}{el} {eluc} {}
\DeclareFontSeriesChangeRule {ebuc}{l}  {luc}  {}
\DeclareFontSeriesChangeRule {ebuc}{sl} {sluc} {}
\DeclareFontSeriesChangeRule {ebuc}{sb} {sbuc} {buc}
\DeclareFontSeriesChangeRule {ebuc}{b}  {buc}  {}
\DeclareFontSeriesChangeRule {ebuc}{eb} {ebuc} {}
```

In the following rule, we use `eb` instead of `b` in the fourth argument, since `eb` is a better approximation to `ub` than `b` and `ebuc` is already in the first argument and therefore this font face probably actually exists. A similar consideration also applies to some other rules in the following.

```
\DeclareFontSeriesChangeRule {ebuc}{ub} {ubuc} {ebuc}
\DeclareFontSeriesChangeRule {ebuc}{uc} {ebuc} {}
```

```
\DeclareFontSeriesChangeRule {ebuc}{ec} {ebec} {}
\DeclareFontSeriesChangeRule {ebuc}{c}  {ebc}  {}
\DeclareFontSeriesChangeRule {ebuc}{sc} {ebsc} {}
\DeclareFontSeriesChangeRule {ebuc}{sx} {ebsx} {}
\DeclareFontSeriesChangeRule {ebuc}{x}  {ebx}  {}
\DeclareFontSeriesChangeRule {ebuc}{ex} {ebex} {}
\DeclareFontSeriesChangeRule {ebuc}{ux} {ebux} {}

\DeclareFontSeriesChangeRule {ebec}{ul} {ulec} {}
\DeclareFontSeriesChangeRule {ebec}{el} {elec} {}
\DeclareFontSeriesChangeRule {ebec}{l}  {lec}  {}
\DeclareFontSeriesChangeRule {ebec}{sl} {slec} {}
\DeclareFontSeriesChangeRule {ebec}{sb} {sbec} {bec}
\DeclareFontSeriesChangeRule {ebec}{b}  {bec}  {}
\DeclareFontSeriesChangeRule {ebec}{eb} {ebec} {}
\DeclareFontSeriesChangeRule {ebec}{ub} {ubec} {ebec}
\DeclareFontSeriesChangeRule {ebec}{uc} {ebuc} {}
\DeclareFontSeriesChangeRule {ebec}{ec} {ebec} {}
\DeclareFontSeriesChangeRule {ebec}{c}  {ebc}  {}
\DeclareFontSeriesChangeRule {ebec}{sc} {ebsc} {}
\DeclareFontSeriesChangeRule {ebec}{sx} {ebsx} {}
\DeclareFontSeriesChangeRule {ebec}{x}  {ebx}  {}
\DeclareFontSeriesChangeRule {ebec}{ex} {ebex} {}
\DeclareFontSeriesChangeRule {ebec}{ux} {ebux} {}

\DeclareFontSeriesChangeRule {ebc}{ul}  {ulc}  {}
\DeclareFontSeriesChangeRule {ebc}{el}  {elc}  {}
\DeclareFontSeriesChangeRule {ebc}{l}   {lc}   {}
\DeclareFontSeriesChangeRule {ebc}{sl}  {slc}  {}
\DeclareFontSeriesChangeRule {ebc}{sb}  {sbc}  {bc}
\DeclareFontSeriesChangeRule {ebc}{b}   {bc}   {}
\DeclareFontSeriesChangeRule {ebc}{eb}  {ebc}  {}
\DeclareFontSeriesChangeRule {ebc}{ub}  {ubc}  {ebc}
\DeclareFontSeriesChangeRule {ebc}{uc}  {ebuc} {}
\DeclareFontSeriesChangeRule {ebc}{ec}  {ebec} {}
\DeclareFontSeriesChangeRule {ebc}{c}   {ebc}  {}
\DeclareFontSeriesChangeRule {ebc}{sc}  {ebsc} {}
\DeclareFontSeriesChangeRule {ebc}{sx}  {ebsx} {}
\DeclareFontSeriesChangeRule {ebc}{x}   {ebx}  {}
\DeclareFontSeriesChangeRule {ebc}{ex}  {ebex} {}
\DeclareFontSeriesChangeRule {ebc}{ux}  {ebux} {}

\DeclareFontSeriesChangeRule {ebsc}{ul} {ulsc} {}
\DeclareFontSeriesChangeRule {ebsc}{el} {elsc} {}
\DeclareFontSeriesChangeRule {ebsc}{l}  {lsc}  {}
\DeclareFontSeriesChangeRule {ebsc}{sl} {slsc} {}
\DeclareFontSeriesChangeRule {ebsc}{sb} {sbsc} {bsc}
\DeclareFontSeriesChangeRule {ebsc}{b}  {bsc}  {}
\DeclareFontSeriesChangeRule {ebsc}{eb} {ebsc} {}
\DeclareFontSeriesChangeRule {ebsc}{ub} {ubsc} {ebsc}
\DeclareFontSeriesChangeRule {ebsc}{uc} {ebuc} {}
\DeclareFontSeriesChangeRule {ebsc}{ec} {ebec} {}
\DeclareFontSeriesChangeRule {ebsc}{c}  {ebc}  {}
\DeclareFontSeriesChangeRule {ebsc}{sc} {ebsc} {}
\DeclareFontSeriesChangeRule {ebsc}{sx} {ebsx} {}
\DeclareFontSeriesChangeRule {ebsc}{x}  {ebx}  {}
```

```
\DeclareFontSeriesChangeRule {ebsc}{ex} {ebex} {}
\DeclareFontSeriesChangeRule {ebsc}{ux} {ebux} {}

\DeclareFontSeriesChangeRule {eb}{uc}   {ebuc} {}
\DeclareFontSeriesChangeRule {eb}{ec}   {ebec} {}
\DeclareFontSeriesChangeRule {eb}{c}    {ebc}  {}
\DeclareFontSeriesChangeRule {eb}{sc}   {ebsc} {}
\DeclareFontSeriesChangeRule {eb}{sx}   {ebsx} {}
\DeclareFontSeriesChangeRule {eb}{x}    {ebx}  {}
\DeclareFontSeriesChangeRule {eb}{ex}   {ebex} {}
\DeclareFontSeriesChangeRule {eb}{ux}   {ebux} {}
\DeclareFontSeriesChangeRule {eb}{sb}   {sb}   {b}
\DeclareFontSeriesChangeRule {eb}{ub}   {ub}   {eb}

\DeclareFontSeriesChangeRule {ebsx}{ul} {ulsx} {}
\DeclareFontSeriesChangeRule {ebsx}{el} {elsx} {}
\DeclareFontSeriesChangeRule {ebsx}{l}  {lsx}  {}
\DeclareFontSeriesChangeRule {ebsx}{sl} {slsx} {}
\DeclareFontSeriesChangeRule {ebsx}{sb} {sbsx} {bsx}
\DeclareFontSeriesChangeRule {ebsx}{b}  {bsx}  {}
\DeclareFontSeriesChangeRule {ebsx}{eb} {ebsx} {}
\DeclareFontSeriesChangeRule {ebsx}{ub} {ubsx} {ebsx}
\DeclareFontSeriesChangeRule {ebsx}{uc} {ebuc} {}
\DeclareFontSeriesChangeRule {ebsx}{ec} {ebec} {}
\DeclareFontSeriesChangeRule {ebsx}{c}  {ebc}  {}
\DeclareFontSeriesChangeRule {ebsx}{sc} {ebsc} {}
\DeclareFontSeriesChangeRule {ebsx}{sx} {ebsx} {}
\DeclareFontSeriesChangeRule {ebsx}{x}  {ebx}  {}
\DeclareFontSeriesChangeRule {ebsx}{ex} {ebex} {}
\DeclareFontSeriesChangeRule {ebsx}{ux} {ebux} {}

\DeclareFontSeriesChangeRule {ebx}{ul}  {ulx}  {}
\DeclareFontSeriesChangeRule {ebx}{el}  {elx}  {}
\DeclareFontSeriesChangeRule {ebx}{l}   {lx}   {}
\DeclareFontSeriesChangeRule {ebx}{sl}  {slx}  {}
\DeclareFontSeriesChangeRule {ebx}{sb}  {sbx}  {bx}
\DeclareFontSeriesChangeRule {ebx}{b}   {bx}   {}
\DeclareFontSeriesChangeRule {ebx}{eb}  {ebx}  {}
\DeclareFontSeriesChangeRule {ebx}{ub}  {ubx}  {ebx}
\DeclareFontSeriesChangeRule {ebx}{uc}  {ebuc} {}
\DeclareFontSeriesChangeRule {ebx}{ec}  {ebec} {}
\DeclareFontSeriesChangeRule {ebx}{c}   {ebc}  {}
\DeclareFontSeriesChangeRule {ebx}{sc}  {ebsc} {}
\DeclareFontSeriesChangeRule {ebx}{sx}  {ebsx} {}
\DeclareFontSeriesChangeRule {ebx}{x}   {ebx}  {}
\DeclareFontSeriesChangeRule {ebx}{ex}  {ebex} {}
\DeclareFontSeriesChangeRule {ebx}{ux}  {ebux} {}

\DeclareFontSeriesChangeRule {ebex}{ul} {ulex} {}
\DeclareFontSeriesChangeRule {ebex}{el} {elex} {}
\DeclareFontSeriesChangeRule {ebex}{l}  {lex}  {}
\DeclareFontSeriesChangeRule {ebex}{sl} {slex} {}
\DeclareFontSeriesChangeRule {ebex}{sb} {sbex} {bex}
\DeclareFontSeriesChangeRule {ebex}{b}  {bex}  {}
\DeclareFontSeriesChangeRule {ebex}{eb} {ebex} {}
\DeclareFontSeriesChangeRule {ebex}{ub} {ubex} {ebex}
\DeclareFontSeriesChangeRule {ebex}{uc} {ebuc} {}
```

```
\DeclareFontSeriesChangeRule {ebex}{ec} {ebec} {}
\DeclareFontSeriesChangeRule {ebex}{c}  {ebc}  {}
\DeclareFontSeriesChangeRule {ebex}{sc} {ebsc} {}
\DeclareFontSeriesChangeRule {ebex}{sx} {ebsx} {}
\DeclareFontSeriesChangeRule {ebex}{x}  {ebx}  {}
\DeclareFontSeriesChangeRule {ebex}{ex} {ebex} {}
\DeclareFontSeriesChangeRule {ebex}{ux} {ebux} {}

\DeclareFontSeriesChangeRule {ebux}{ul} {ulux} {}
\DeclareFontSeriesChangeRule {ebux}{el} {elux} {}
\DeclareFontSeriesChangeRule {ebux}{l}  {lux}  {}
\DeclareFontSeriesChangeRule {ebux}{sl} {slux} {}
\DeclareFontSeriesChangeRule {ebux}{sb} {sbux} {bux}
\DeclareFontSeriesChangeRule {ebux}{b}  {bux}  {}
\DeclareFontSeriesChangeRule {ebux}{eb} {ebux} {}
\DeclareFontSeriesChangeRule {ebux}{ub} {ubux} {ebux}
\DeclareFontSeriesChangeRule {ebux}{uc} {ebuc} {}
\DeclareFontSeriesChangeRule {ebux}{ec} {ebec} {}
\DeclareFontSeriesChangeRule {ebux}{c}  {ebc}  {}
\DeclareFontSeriesChangeRule {ebux}{sc} {ebsc} {}
\DeclareFontSeriesChangeRule {ebux}{sx} {ebsx} {}
\DeclareFontSeriesChangeRule {ebux}{x}  {ebx}  {}
\DeclareFontSeriesChangeRule {ebux}{ex} {ebex} {}
\DeclareFontSeriesChangeRule {ebux}{ux} {ebux} {}

\DeclareFontSeriesChangeRule {ubuc}{ul} {uluc} {}
\DeclareFontSeriesChangeRule {ubuc}{el} {eluc} {}
\DeclareFontSeriesChangeRule {ubuc}{l}  {luc}  {}
\DeclareFontSeriesChangeRule {ubuc}{sl} {sluc} {}
\DeclareFontSeriesChangeRule {ubuc}{sb} {sbuc} {buc}
\DeclareFontSeriesChangeRule {ubuc}{b}  {buc}  {}
\DeclareFontSeriesChangeRule {ubuc}{eb} {ebuc} {ubuc}
\DeclareFontSeriesChangeRule {ubuc}{ub} {ubuc} {}
\DeclareFontSeriesChangeRule {ubuc}{uc} {ubuc} {}
\DeclareFontSeriesChangeRule {ubuc}{ec} {ubec} {}
\DeclareFontSeriesChangeRule {ubuc}{c}  {ubc}  {}
\DeclareFontSeriesChangeRule {ubuc}{sc} {ubsc} {}
\DeclareFontSeriesChangeRule {ubuc}{sx} {ubsx} {}
\DeclareFontSeriesChangeRule {ubuc}{x}  {ubx}  {}
\DeclareFontSeriesChangeRule {ubuc}{ex} {ubex} {}
\DeclareFontSeriesChangeRule {ubuc}{ux} {ubux} {}

\DeclareFontSeriesChangeRule {ubec}{ul} {ulec} {}
\DeclareFontSeriesChangeRule {ubec}{el} {elec} {}
\DeclareFontSeriesChangeRule {ubec}{l}  {lec}  {}
\DeclareFontSeriesChangeRule {ubec}{sl} {slec} {}
\DeclareFontSeriesChangeRule {ubec}{sb} {sbec} {bec}
\DeclareFontSeriesChangeRule {ubec}{b}  {bec}  {}
\DeclareFontSeriesChangeRule {ubec}{eb} {ebec} {ubec}
\DeclareFontSeriesChangeRule {ubec}{ub} {ubec} {}
\DeclareFontSeriesChangeRule {ubec}{uc} {ubuc} {}
\DeclareFontSeriesChangeRule {ubec}{ec} {ubec} {}
\DeclareFontSeriesChangeRule {ubec}{c}  {ubc}  {}
\DeclareFontSeriesChangeRule {ubec}{sc} {ubsc} {}
\DeclareFontSeriesChangeRule {ubec}{sx} {ubsx} {}
\DeclareFontSeriesChangeRule {ubec}{x}  {ubx}  {}
```

```
\DeclareFontSeriesChangeRule {ubec}{ex} {ubex} {}
\DeclareFontSeriesChangeRule {ubec}{ux} {ubux} {}

\DeclareFontSeriesChangeRule {ubc}{ul}  {ulc}  {}
\DeclareFontSeriesChangeRule {ubc}{el}  {elc}  {}
\DeclareFontSeriesChangeRule {ubc}{l}   {lc}   {}
\DeclareFontSeriesChangeRule {ubc}{sl}  {slc}  {}
\DeclareFontSeriesChangeRule {ubc}{sb}  {sbc}  {bc}
\DeclareFontSeriesChangeRule {ubc}{b}   {bc}   {}
\DeclareFontSeriesChangeRule {ubc}{eb}  {ebc}  {ubc}
\DeclareFontSeriesChangeRule {ubc}{ub}  {ubc}  {}
\DeclareFontSeriesChangeRule {ubc}{uc}  {ubuc} {}
\DeclareFontSeriesChangeRule {ubc}{ec}  {ubec} {}
\DeclareFontSeriesChangeRule {ubc}{c}   {ubc}  {}
\DeclareFontSeriesChangeRule {ubc}{sc}  {ubsc} {}
\DeclareFontSeriesChangeRule {ubc}{sx}  {ubsx} {}
\DeclareFontSeriesChangeRule {ubc}{x}   {ubx}  {}
\DeclareFontSeriesChangeRule {ubc}{ex}  {ubex} {}
\DeclareFontSeriesChangeRule {ubc}{ux}  {ubux} {}

\DeclareFontSeriesChangeRule {ubsc}{ul} {ulsc} {}
\DeclareFontSeriesChangeRule {ubsc}{el} {elsc} {}
\DeclareFontSeriesChangeRule {ubsc}{l}  {lsc}  {}
\DeclareFontSeriesChangeRule {ubsc}{sl} {slsc} {}
\DeclareFontSeriesChangeRule {ubsc}{sb} {sbsc} {bsc}
\DeclareFontSeriesChangeRule {ubsc}{b}  {bsc}  {}
\DeclareFontSeriesChangeRule {ubsc}{eb} {ebsc} {ubsc}
\DeclareFontSeriesChangeRule {ubsc}{ub} {ubsc} {}
\DeclareFontSeriesChangeRule {ubsc}{uc} {ubuc} {}
\DeclareFontSeriesChangeRule {ubsc}{ec} {ubec} {}
\DeclareFontSeriesChangeRule {ubsc}{c}  {ubc}  {}
\DeclareFontSeriesChangeRule {ubsc}{sc} {ubsc} {}
\DeclareFontSeriesChangeRule {ubsc}{sx} {ubsx} {}
\DeclareFontSeriesChangeRule {ubsc}{x}  {ubx}  {}
\DeclareFontSeriesChangeRule {ubsc}{ex} {ubex} {}
\DeclareFontSeriesChangeRule {ubsc}{ux} {ubux} {}

\DeclareFontSeriesChangeRule {ub}{uc}   {ubuc} {}
\DeclareFontSeriesChangeRule {ub}{ec}   {ubec} {}
\DeclareFontSeriesChangeRule {ub}{c}    {ubc}  {}
\DeclareFontSeriesChangeRule {ub}{sc}   {ubsc} {}
\DeclareFontSeriesChangeRule {ub}{sx}   {ubsx} {}
\DeclareFontSeriesChangeRule {ub}{x}    {ubx}  {}
\DeclareFontSeriesChangeRule {ub}{ex}   {ubex} {}
\DeclareFontSeriesChangeRule {ub}{ux}   {ubux} {}
\DeclareFontSeriesChangeRule {ub}{sb}   {sb}   {b}
\DeclareFontSeriesChangeRule {ub}{eb}   {eb}   {ub}

\DeclareFontSeriesChangeRule {ubsx}{ul} {ulsx} {}
\DeclareFontSeriesChangeRule {ubsx}{el} {elsx} {}
\DeclareFontSeriesChangeRule {ubsx}{l}  {lsx}  {}
\DeclareFontSeriesChangeRule {ubsx}{sl} {slsx} {}
\DeclareFontSeriesChangeRule {ubsx}{sb} {sbsx} {bsx}
\DeclareFontSeriesChangeRule {ubsx}{b}  {bsx}  {}
\DeclareFontSeriesChangeRule {ubsx}{eb} {ebsx} {ubsx}
\DeclareFontSeriesChangeRule {ubsx}{ub} {ubsx} {}
\DeclareFontSeriesChangeRule {ubsx}{uc} {ubuc} {}
```

```
\DeclareFontSeriesChangeRule {ubsx}{ec} {ubec} {}
\DeclareFontSeriesChangeRule {ubsx}{c}  {ubc}  {}
\DeclareFontSeriesChangeRule {ubsx}{sc} {ubsc} {}
\DeclareFontSeriesChangeRule {ubsx}{sx} {ubsx} {}
\DeclareFontSeriesChangeRule {ubsx}{x}  {ubx}  {}
\DeclareFontSeriesChangeRule {ubsx}{ex} {ubex} {}
\DeclareFontSeriesChangeRule {ubsx}{ux} {ubux} {}

\DeclareFontSeriesChangeRule {ubx}{ul}  {ulx}  {}
\DeclareFontSeriesChangeRule {ubx}{el}  {elx}  {}
\DeclareFontSeriesChangeRule {ubx}{l}   {lx}   {}
\DeclareFontSeriesChangeRule {ubx}{sl}  {slx}  {}
\DeclareFontSeriesChangeRule {ubx}{sb}  {sbx}  {bx}
\DeclareFontSeriesChangeRule {ubx}{b}   {bx}   {}
\DeclareFontSeriesChangeRule {ubx}{eb}  {ebx}  {ubx}
\DeclareFontSeriesChangeRule {ubx}{ub}  {ubx}  {}
\DeclareFontSeriesChangeRule {ubx}{uc}  {ubuc} {}
\DeclareFontSeriesChangeRule {ubx}{ec}  {ubec} {}
\DeclareFontSeriesChangeRule {ubx}{c}   {ubc}  {}
\DeclareFontSeriesChangeRule {ubx}{sc}  {ubsc} {}
\DeclareFontSeriesChangeRule {ubx}{sx}  {ubsx} {}
\DeclareFontSeriesChangeRule {ubx}{x}   {ubx}  {}
\DeclareFontSeriesChangeRule {ubx}{ex}  {ubex} {}
\DeclareFontSeriesChangeRule {ubx}{ux}  {ubux} {}

\DeclareFontSeriesChangeRule {ubex}{ul} {ulex} {}
\DeclareFontSeriesChangeRule {ubex}{el} {elex} {}
\DeclareFontSeriesChangeRule {ubex}{l}  {lex}  {}
\DeclareFontSeriesChangeRule {ubex}{sl} {slex} {}
\DeclareFontSeriesChangeRule {ubex}{sb} {sbex} {bex}
\DeclareFontSeriesChangeRule {ubex}{b}  {bex}  {}
\DeclareFontSeriesChangeRule {ubex}{eb} {ebex} {ubex}
\DeclareFontSeriesChangeRule {ubex}{ub} {ubex} {}
\DeclareFontSeriesChangeRule {ubex}{uc} {ubuc} {}
\DeclareFontSeriesChangeRule {ubex}{ec} {ubec} {}
\DeclareFontSeriesChangeRule {ubex}{c}  {ubc}  {}
\DeclareFontSeriesChangeRule {ubex}{sc} {ubsc} {}
\DeclareFontSeriesChangeRule {ubex}{sx} {ubsx} {}
\DeclareFontSeriesChangeRule {ubex}{x}  {ubx}  {}
\DeclareFontSeriesChangeRule {ubex}{ex} {ubex} {}
\DeclareFontSeriesChangeRule {ubex}{ux} {ubux} {}

\DeclareFontSeriesChangeRule {ubux}{ul} {ulux} {}
\DeclareFontSeriesChangeRule {ubux}{el} {elux} {}
\DeclareFontSeriesChangeRule {ubux}{l}  {lux}  {}
\DeclareFontSeriesChangeRule {ubux}{sl} {slux} {}
\DeclareFontSeriesChangeRule {ubux}{sb} {sbux} {bux}
\DeclareFontSeriesChangeRule {ubux}{b}  {bux}  {}
\DeclareFontSeriesChangeRule {ubux}{eb} {ebux} {ubux}
\DeclareFontSeriesChangeRule {ubux}{ub} {ubux} {}
\DeclareFontSeriesChangeRule {ubux}{uc} {ubuc} {}
\DeclareFontSeriesChangeRule {ubux}{ec} {ubec} {}
\DeclareFontSeriesChangeRule {ubux}{c}  {ubc}  {}
\DeclareFontSeriesChangeRule {ubux}{sc} {ubsc} {}
\DeclareFontSeriesChangeRule {ubux}{sx} {ubsx} {}
\DeclareFontSeriesChangeRule {ubux}{x}  {ubx}  {}
```

```
\DeclareFontSeriesChangeRule {ubux}{ex} {ubex} {}
\DeclareFontSeriesChangeRule {ubux}{ux} {ubux} {}
```

Special rules for `lm` etc. aren't needed because if `lm` is requested, it will be used if there is no rule, and that is then reduced to `l` automatically. Same for `mc` and friends.

The following entries handle a request for `bx` and fall back to `b` if that can't be fulfilled.

```
\DeclareFontSeriesChangeRule {uluc}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {ulec}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {ulc}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {ulsc}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {ul}{bx}   {bx}   {b}
\DeclareFontSeriesChangeRule {ulsx}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {ulx}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {ulex}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {ulux}{bx} {bx}   {b}

\DeclareFontSeriesChangeRule {eluc}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {elec}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {elc}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {elsc}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {el}{bx}   {bx}   {b}
\DeclareFontSeriesChangeRule {elsx}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {elx}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {elex}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {elux}{bx} {bx}   {b}

\DeclareFontSeriesChangeRule {luc}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {lec}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {lc}{bx}   {bx}   {b}
\DeclareFontSeriesChangeRule {lsc}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {l}{bx}    {bx}   {b}
\DeclareFontSeriesChangeRule {lsx}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {lx}{bx}   {bx}   {b}
\DeclareFontSeriesChangeRule {lex}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {lux}{bx}  {bx}   {b}

\DeclareFontSeriesChangeRule {sluc}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {slec}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {slc}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {slsc}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {sl}{bx}   {bx}   {b}
\DeclareFontSeriesChangeRule {slsx}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {slx}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {slex}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {slux}{bx} {bx}   {b}

\DeclareFontSeriesChangeRule {uc}{bx}   {bx}   {b}
\DeclareFontSeriesChangeRule {ec}{bx}   {bx}   {b}
\DeclareFontSeriesChangeRule {c}{bx}    {bx}   {b}
\DeclareFontSeriesChangeRule {sc}{bx}   {bx}   {b}
\DeclareFontSeriesChangeRule {m}{bx}    {bx}   {b}
\DeclareFontSeriesChangeRule {sx}{bx}   {bx}   {b}
\DeclareFontSeriesChangeRule {x}{bx}    {bx}   {b}
\DeclareFontSeriesChangeRule {ex}{bx}   {bx}   {b}
\DeclareFontSeriesChangeRule {ux}{bx}   {bx}   {b}
```

```
\DeclareFontSeriesChangeRule {sbuc}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {sbec}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {sbc}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {sbsc}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {sb}{bx}   {bx}   {b}
\DeclareFontSeriesChangeRule {sbsx}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {sbx}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {sbex}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {sbux}{bx} {bx}   {b}

\DeclareFontSeriesChangeRule {buc}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {bec}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {bc}{bx}   {bx}   {b}
\DeclareFontSeriesChangeRule {bsc}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {b}{bx}    {bx}   {b}
\DeclareFontSeriesChangeRule {bsx}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {bx}{bx}   {bx}   {b}
\DeclareFontSeriesChangeRule {bex}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {bux}{bx}  {bx}   {b}

\DeclareFontSeriesChangeRule {ebuc}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {ebec}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {ebc}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {ebsc}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {eb}{bx}   {bx}   {b}
\DeclareFontSeriesChangeRule {ebsx}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {ebx}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {ebex}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {ebux}{bx} {bx}   {b}

\DeclareFontSeriesChangeRule {ubuc}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {ubec}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {ubc}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {ubsc}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {ub}{bx}   {bx}   {b}
\DeclareFontSeriesChangeRule {ubsx}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {ubx}{bx}  {bx}   {b}
\DeclareFontSeriesChangeRule {ubex}{bx} {bx}   {b}
\DeclareFontSeriesChangeRule {ubux}{bx} {bx}   {b}
```

Here are the special rules for `m?`. We offer `m` as alternative result series for all entries where this is not already the result series, because otherwise, if the result series does not exist, `m?` would be tried as the series value, which cannot work. (Of course, this does not help if the `m` series does not exist either, but this should only affect very few fonts.)

```
\DeclareFontSeriesChangeRule {uluc}{m?} {uc}   {m}
\DeclareFontSeriesChangeRule {ulec}{m?} {ec}   {m}
\DeclareFontSeriesChangeRule {ulc}{m?}  {c}    {m}
\DeclareFontSeriesChangeRule {ulsc}{m?} {sc}   {m}
\DeclareFontSeriesChangeRule {ul}{m?}   {m}    {}
\DeclareFontSeriesChangeRule {ulsx}{m?} {sx}   {m}
\DeclareFontSeriesChangeRule {ulx}{m?}  {x}    {m}
\DeclareFontSeriesChangeRule {ulex}{m?} {ex}   {m}
\DeclareFontSeriesChangeRule {ulux}{m?} {ux}   {m}

\DeclareFontSeriesChangeRule {eluc}{m?} {uc}   {m}
\DeclareFontSeriesChangeRule {elec}{m?} {ec}   {m}
```

```
\DeclareFontSeriesChangeRule {elc}{m?}  {c}    {m}
\DeclareFontSeriesChangeRule {elsc}{m?} {sc}   {m}
\DeclareFontSeriesChangeRule {el}{m?}   {m}    {}
\DeclareFontSeriesChangeRule {elsx}{m?} {sx}   {m}
\DeclareFontSeriesChangeRule {elx}{m?}  {x}    {m}
\DeclareFontSeriesChangeRule {elex}{m?} {ex}   {m}
\DeclareFontSeriesChangeRule {elux}{m?} {ux}   {m}

\DeclareFontSeriesChangeRule {luc}{m?}  {uc}   {m}
\DeclareFontSeriesChangeRule {lec}{m?}  {ec}   {m}
\DeclareFontSeriesChangeRule {lc}{m?}   {c}    {m}
\DeclareFontSeriesChangeRule {lsc}{m?}  {sc}   {m}
\DeclareFontSeriesChangeRule {l}{m?}    {m}    {}
\DeclareFontSeriesChangeRule {lsx}{m?}  {sx}   {m}
\DeclareFontSeriesChangeRule {lx}{m?}   {x}    {m}
\DeclareFontSeriesChangeRule {lex}{m?}  {ex}   {m}
\DeclareFontSeriesChangeRule {lux}{m?}  {ux}   {m}

\DeclareFontSeriesChangeRule {sluc}{m?} {uc}   {m}
\DeclareFontSeriesChangeRule {slec}{m?} {ec}   {m}
\DeclareFontSeriesChangeRule {slc}{m?}  {c}    {m}
\DeclareFontSeriesChangeRule {slsc}{m?} {sc}   {m}
\DeclareFontSeriesChangeRule {sl}{m?}   {m}    {}
\DeclareFontSeriesChangeRule {slsx}{m?} {sx}   {m}
\DeclareFontSeriesChangeRule {slx}{m?}  {x}    {m}
\DeclareFontSeriesChangeRule {slex}{m?} {ex}   {m}
\DeclareFontSeriesChangeRule {slux}{m?} {ux}   {m}

\DeclareFontSeriesChangeRule {uc}{m?}   {uc}   {m}
\DeclareFontSeriesChangeRule {ec}{m?}   {ec}   {m}
\DeclareFontSeriesChangeRule {c}{m?}    {c}    {m}
\DeclareFontSeriesChangeRule {sc}{m?}   {sc}   {m}
\DeclareFontSeriesChangeRule {m}{m?}    {m}    {}
\DeclareFontSeriesChangeRule {sx}{m?}   {sx}   {m}
\DeclareFontSeriesChangeRule {x}{m?}    {x}    {m}
\DeclareFontSeriesChangeRule {ex}{m?}   {ex}   {m}
\DeclareFontSeriesChangeRule {ux}{m?}   {ux}   {m}

\DeclareFontSeriesChangeRule {sbuc}{m?} {uc}   {m}
\DeclareFontSeriesChangeRule {sbec}{m?} {ec}   {m}
\DeclareFontSeriesChangeRule {sbc}{m?}  {c}    {m}
\DeclareFontSeriesChangeRule {sbsc}{m?} {sc}   {m}
\DeclareFontSeriesChangeRule {sb}{m?}   {m}    {}
\DeclareFontSeriesChangeRule {sbsx}{m?} {sx}   {m}
\DeclareFontSeriesChangeRule {sbx}{m?}  {x}    {m}
\DeclareFontSeriesChangeRule {sbex}{m?} {ex}   {m}
\DeclareFontSeriesChangeRule {sbux}{m?} {ux}   {m}

\DeclareFontSeriesChangeRule {buc}{m?}  {uc}   {m}
\DeclareFontSeriesChangeRule {bec}{m?}  {ec}   {m}
\DeclareFontSeriesChangeRule {bc}{m?}   {c}    {m}
\DeclareFontSeriesChangeRule {bsc}{m?}  {sc}   {m}
\DeclareFontSeriesChangeRule {b}{m?}    {m}    {}
\DeclareFontSeriesChangeRule {bsx}{m?}  {sx}   {m}
\DeclareFontSeriesChangeRule {bx}{m?}   {x}    {m}
\DeclareFontSeriesChangeRule {bex}{m?}  {ex}   {m}
\DeclareFontSeriesChangeRule {bux}{m?}  {ux}   {m}
```

```
\DeclareFontSeriesChangeRule {ebuc}{m?} {uc}   {m}
\DeclareFontSeriesChangeRule {ebec}{m?} {ec}   {m}
\DeclareFontSeriesChangeRule {ebc}{m?}  {c}    {m}
\DeclareFontSeriesChangeRule {ebsc}{m?} {sc}   {m}
\DeclareFontSeriesChangeRule {eb}{m?}   {m}    {}
\DeclareFontSeriesChangeRule {ebsx}{m?} {sx}   {m}
\DeclareFontSeriesChangeRule {ebx}{m?}  {x}    {m}
\DeclareFontSeriesChangeRule {ebex}{m?} {ex}   {m}
\DeclareFontSeriesChangeRule {ebux}{m?} {ux}   {m}

\DeclareFontSeriesChangeRule {ubuc}{m?} {uc}   {m}
\DeclareFontSeriesChangeRule {ubec}{m?} {ec}   {m}
\DeclareFontSeriesChangeRule {ubc}{m?}  {c}    {m}
\DeclareFontSeriesChangeRule {ubsc}{m?} {sc}   {m}
\DeclareFontSeriesChangeRule {ub}{m?}   {m}    {}
\DeclareFontSeriesChangeRule {ubsx}{m?} {sx}   {m}
\DeclareFontSeriesChangeRule {ubx}{m?}  {x}    {m}
\DeclareFontSeriesChangeRule {ubex}{m?} {ex}   {m}
\DeclareFontSeriesChangeRule {ubux}{m?} {ux}   {m}
```

And here are the special rules for `?m`. Again, we offer `m` as alternative result series for all entries where this is not already the result series.

```
\DeclareFontSeriesChangeRule {uluc}{?m} {ul}   {m}
\DeclareFontSeriesChangeRule {ulec}{?m} {ul}   {m}
\DeclareFontSeriesChangeRule {ulc}{?m}  {ul}   {m}
\DeclareFontSeriesChangeRule {ulsc}{?m} {ul}   {m}
\DeclareFontSeriesChangeRule {ul}{?m}   {ul}   {m}
\DeclareFontSeriesChangeRule {ulsx}{?m} {ul}   {m}
\DeclareFontSeriesChangeRule {ulx}{?m}  {ul}   {m}
\DeclareFontSeriesChangeRule {ulex}{?m} {ul}   {m}
\DeclareFontSeriesChangeRule {ulux}{?m} {ul}   {m}

\DeclareFontSeriesChangeRule {eluc}{?m} {el}   {m}
\DeclareFontSeriesChangeRule {elec}{?m} {el}   {m}
\DeclareFontSeriesChangeRule {elc}{?m}  {el}   {m}
\DeclareFontSeriesChangeRule {elsc}{?m} {el}   {m}
\DeclareFontSeriesChangeRule {el}{?m}   {el}   {m}
\DeclareFontSeriesChangeRule {elsx}{?m} {el}   {m}
\DeclareFontSeriesChangeRule {elx}{?m}  {el}   {m}
\DeclareFontSeriesChangeRule {elex}{?m} {el}   {m}
\DeclareFontSeriesChangeRule {elux}{?m} {el}   {m}

\DeclareFontSeriesChangeRule {luc}{?m}  {l}    {m}
\DeclareFontSeriesChangeRule {lec}{?m}  {l}    {m}
\DeclareFontSeriesChangeRule {lc}{?m}   {l}    {m}
\DeclareFontSeriesChangeRule {lsc}{?m}  {l}    {m}
\DeclareFontSeriesChangeRule {l}{?m}    {l}    {m}
\DeclareFontSeriesChangeRule {lsx}{?m}  {l}    {m}
\DeclareFontSeriesChangeRule {lx}{?m}   {l}    {m}
\DeclareFontSeriesChangeRule {lex}{?m}  {l}    {m}
\DeclareFontSeriesChangeRule {lux}{?m}  {l}    {m}

\DeclareFontSeriesChangeRule {sluc}{?m} {sl}   {m}
\DeclareFontSeriesChangeRule {slec}{?m} {sl}   {m}
\DeclareFontSeriesChangeRule {slc}{?m}  {sl}   {m}
\DeclareFontSeriesChangeRule {slsc}{?m} {sl}   {m}
\DeclareFontSeriesChangeRule {sl}{?m}   {sl}   {m}
```

```
\DeclareFontSeriesChangeRule {slsx}{?m} {sl}   {m}
\DeclareFontSeriesChangeRule {slx}{?m}  {sl}   {m}
\DeclareFontSeriesChangeRule {slex}{?m} {sl}   {m}
\DeclareFontSeriesChangeRule {slux}{?m} {sl}   {m}

\DeclareFontSeriesChangeRule {uc}{?m}   {m}    {}
\DeclareFontSeriesChangeRule {ec}{?m}   {m}    {}
\DeclareFontSeriesChangeRule {c}{?m}    {m}    {}
\DeclareFontSeriesChangeRule {sc}{?m}   {m}    {}
\DeclareFontSeriesChangeRule {m}{?m}    {m}    {}
\DeclareFontSeriesChangeRule {sx}{?m}   {m}    {}
\DeclareFontSeriesChangeRule {x}{?m}    {m}    {}
\DeclareFontSeriesChangeRule {ex}{?m}   {m}    {}
\DeclareFontSeriesChangeRule {ux}{?m}   {m}    {}

\DeclareFontSeriesChangeRule {sbuc}{?m} {sb}   {m}
\DeclareFontSeriesChangeRule {sbec}{?m} {sb}   {m}
\DeclareFontSeriesChangeRule {sbc}{?m}  {sb}   {m}
\DeclareFontSeriesChangeRule {sbsc}{?m} {sb}   {m}
\DeclareFontSeriesChangeRule {sb}{?m}   {sb}   {m}
\DeclareFontSeriesChangeRule {sbsx}{?m} {sb}   {m}
\DeclareFontSeriesChangeRule {sbx}{?m}  {sb}   {m}
\DeclareFontSeriesChangeRule {sbex}{?m} {sb}   {m}
\DeclareFontSeriesChangeRule {sbux}{?m} {sb}   {m}

\DeclareFontSeriesChangeRule {buc}{?m}  {b}    {m}
\DeclareFontSeriesChangeRule {bec}{?m}  {b}    {m}
\DeclareFontSeriesChangeRule {bc}{?m}   {b}    {m}
\DeclareFontSeriesChangeRule {bsc}{?m}  {b}    {m}
\DeclareFontSeriesChangeRule {b}{?m}    {b}    {m}
\DeclareFontSeriesChangeRule {bsx}{?m}  {b}    {m}
\DeclareFontSeriesChangeRule {bx}{?m}   {b}    {m}
\DeclareFontSeriesChangeRule {bex}{?m}  {b}    {m}
\DeclareFontSeriesChangeRule {bux}{?m}  {b}    {m}

\DeclareFontSeriesChangeRule {ebuc}{?m} {eb}   {m}
\DeclareFontSeriesChangeRule {ebec}{?m} {eb}   {m}
\DeclareFontSeriesChangeRule {ebc}{?m}  {eb}   {m}
\DeclareFontSeriesChangeRule {ebsc}{?m} {eb}   {m}
\DeclareFontSeriesChangeRule {eb}{?m}   {eb}   {m}
\DeclareFontSeriesChangeRule {ebsx}{?m} {eb}   {m}
\DeclareFontSeriesChangeRule {ebx}{?m}  {eb}   {m}
\DeclareFontSeriesChangeRule {ebex}{?m} {eb}   {m}
\DeclareFontSeriesChangeRule {ebux}{?m} {eb}   {m}

\DeclareFontSeriesChangeRule {ubuc}{?m} {ub}   {m}
\DeclareFontSeriesChangeRule {ubec}{?m} {ub}   {m}
\DeclareFontSeriesChangeRule {ubc}{?m}  {ub}   {m}
\DeclareFontSeriesChangeRule {ubsc}{?m} {ub}   {m}
\DeclareFontSeriesChangeRule {ub}{?m}   {ub}   {m}
\DeclareFontSeriesChangeRule {ubsx}{?m} {ub}   {m}
\DeclareFontSeriesChangeRule {ubx}{?m}  {ub}   {m}
\DeclareFontSeriesChangeRule {ubex}{?m} {ub}   {m}
\DeclareFontSeriesChangeRule {ubux}{?m} {ub}   {m}

⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

Supporting rollback ...

```
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩    {\DeclareFontSeriesChangeRule}{Series change rules}%
```

The next definition is only needed if somebody rolls forward from a release older than 2020-02-02 but not to the latest version but one before 2025-06-01. Pretty unlikely, but ....

```
⟨latexrelease⟩
⟨latexrelease⟩\def\DeclareFontSeriesChangeRule#1#2#3#4{%
⟨latexrelease⟩   \@namedef{series@#1@#2}{{#3}{#4}}}
⟨latexrelease⟩
```

The huge set of declarations below are those from 2020-02-02 plus all from above that were newly added (but now with empty result and alternative result arguments). To compile this list I sorted both together and then dropped entries appearing twice. This is why the sorting is now different from the one above.

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bc}{bx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bc}{b}{bc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bc}{c}{bc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bc}{eb}{ebc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bc}{ec}{bec} {bc}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bc}{el}{elc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bc}{ex}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bc}{l}{lc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bc}{sb}{sbc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bc}{sc}{bsc} {bc}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bc}{sl}{slc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bc}{sx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bc}{ub}{ubc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bc}{uc}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bc}{ul}{ulc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bc}{ux}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bc}{x}{bx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bec}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bec}{b}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bec}{c}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bec}{eb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bec}{ec}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bec}{el}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bec}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bec}{l}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bec}{sb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bec}{sc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bec}{sl}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bec}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bec}{ub}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bec}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bec}{ul}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bec}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bec}{x}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bex}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bex}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bex}{b}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bex}{c}   {} {}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bex}{eb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bex}{ec}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bex}{el}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bex}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bex}{l}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bex}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bex}{sb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bex}{sc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bex}{sl}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bex}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bex}{ub}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bex}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bex}{ul}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bex}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bex}{x}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsc}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsc}{b}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsc}{c}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsc}{eb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsc}{ec}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsc}{el}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsc}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsc}{l}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsc}{sb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsc}{sc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsc}{sl}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsc}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsc}{ub}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsc}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsc}{ul}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsc}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsc}{x}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsx}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsx}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsx}{b}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsx}{c}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsx}{eb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsx}{ec}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsx}{el}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsx}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsx}{l}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsx}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsx}{sb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsx}{sc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsx}{sl}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsx}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsx}{ub}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsx}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsx}{ul}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsx}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bsx}{x}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {buc}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {buc}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {buc}{b}   {} {}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {buc}{c}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {buc}{eb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {buc}{ec}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {buc}{el}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {buc}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {buc}{l}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {buc}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {buc}{sb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {buc}{sc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {buc}{sl}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {buc}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {buc}{ub}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {buc}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {buc}{ul}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {buc}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {buc}{x}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bux}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bux}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bux}{b}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bux}{c}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bux}{eb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bux}{ec}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bux}{el}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bux}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bux}{l}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bux}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bux}{sb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bux}{sc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bux}{sl}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bux}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bux}{ub}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bux}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bux}{ul}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bux}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bux}{x}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bx}{bx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bx}{b}{bx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bx}{c}  {bc}  {bx}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bx}{eb}{ebx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bx}{ec}  {bec}  {bx}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bx}{el}{elx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bx}{ex}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bx}{l}{lx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bx}{sb} {sbx} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bx}{sc}  {bsc}  {bx}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bx}{sl}{slx}  {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bx}{sx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bx}{ub}{ubx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bx}{uc}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bx}{ul}{ulx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bx}{ux}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {bx}{x}{bx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {b}{bx} {bx}  {b}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {b}{c}  {bc}  {b}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {b}{eb}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {b}{ec}  {bec}  {b}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {b}{ex}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {b}{sb} {sb}  {b}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {b}{sc}  {bsc}  {b}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {b}{sx}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {b}{ub}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {b}{uc}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {b}{ux}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {b}{x}  {bx}  {b}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {c}{bx} {bx}  {b}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {c}{b}{bc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {c}{eb}{ebc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {c}{ec}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {c}{el}{elc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {c}{l}{lc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {c}{sb}{sbc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {c}{sc}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {c}{sl}{slc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {c}{ub}{ubc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {c}{uc}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {c}{x}{x}{m}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebc}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebc}{b}{bc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebc}{c}{ebc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebc}{eb}{ebc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebc}{ec}{ebec}{ebc}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebc}{el}{elc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebc}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebc}{l}{lc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebc}{sb}{sbc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebc}{sc}{ebsc}{ebc}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebc}{sl}{slc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebc}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebc}{ub}{ubc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebc}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebc}{ul}{ulc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebc}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebc}{x}{ebx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebec}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebec}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebec}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebec}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebec}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebec}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebec}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebec}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebec}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebec}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebec}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebec}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebec}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebec}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebec}{ul} {} {}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebec}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebec}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebex}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebex}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebex}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebex}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebex}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebex}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebex}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebex}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebex}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebex}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebex}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebex}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebex}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebex}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebex}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebex}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebex}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebex}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebex}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsc}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsc}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsc}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsc}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsc}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsc}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsc}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsc}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsc}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsc}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsc}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsc}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsc}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsc}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsc}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsc}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsc}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsx}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsx}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsx}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsx}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsx}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsx}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsx}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsx}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsx}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsx}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsx}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsx}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsx}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsx}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsx}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsx}{uc} {} {}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsx}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsx}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebsx}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebuc}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebuc}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebuc}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebuc}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebuc}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebuc}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebuc}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebuc}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebuc}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebuc}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebuc}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebuc}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebuc}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebuc}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebuc}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebuc}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebuc}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebuc}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebuc}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebux}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebux}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebux}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebux}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebux}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebux}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebux}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebux}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebux}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebux}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebux}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebux}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebux}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebux}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebux}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebux}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebux}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebux}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebux}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebx}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebx}{b}{bx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebx}{c}{ebc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebx}{eb}{ebx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebx}{ec}{ebec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebx}{el}{elx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebx}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebx}{l}{lx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebx}{sb}{sbx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebx}{sc}{ebsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebx}{sl}{slx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebx}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebx}{ub}{ubx}{}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebx}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebx}{ul}{ulx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebx}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ebx}{x}{ebx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eb}{bx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eb}{c}{ebc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eb}{ec}{ebec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eb}{ex}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eb}{sb}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eb}{sc}{ebsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eb}{sx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eb}{ub}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eb}{uc}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eb}{ux}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eb}{x}{ebx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ec}{bx} {bx}  {b}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ec}{b}{bec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ec}{eb}{ebec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ec}{el}{elec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ec}{l}{lec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ec}{sb}{sbec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ec}{sl}{slec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ec}{ub}{ubec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ec}{x}{x}{m}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elc}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elc}{b}{bc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elc}{c}{elc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elc}{eb}{ebc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elc}{ec}{elec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elc}{el}{elc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elc}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elc}{l}{lc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elc}{sb}{sbc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elc}{sc}{elsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elc}{sl}{slc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elc}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elc}{ub}{ubc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elc}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elc}{ul}{ulc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elc}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elc}{x}{elx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elec}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elec}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elec}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elec}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elec}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elec}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elec}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elec}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elec}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elec}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elec}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elec}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elec}{ub} {} {}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elec}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elec}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elec}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elec}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elex}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elex}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elex}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elex}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elex}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elex}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elex}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elex}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elex}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elex}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elex}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elex}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elex}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elex}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elex}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elex}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elex}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elex}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elex}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsc}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsc}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsc}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsc}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsc}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsc}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsc}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsc}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsc}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsc}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsc}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsc}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsc}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsc}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsc}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsc}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsc}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsx}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsx}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsx}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsx}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsx}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsx}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsx}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsx}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsx}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsx}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsx}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsx}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsx}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsx}{sx} {} {}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsx}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsx}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsx}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsx}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elsx}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eluc}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eluc}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eluc}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eluc}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eluc}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eluc}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eluc}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eluc}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eluc}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eluc}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eluc}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eluc}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eluc}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eluc}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eluc}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eluc}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eluc}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eluc}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {eluc}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elux}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elux}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elux}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elux}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elux}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elux}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elux}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elux}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elux}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elux}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elux}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elux}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elux}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elux}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elux}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elux}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elux}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elux}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elux}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elx}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elx}{b}{bx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elx}{c}{elc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elx}{eb}{ebx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elx}{ec}{elec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elx}{el}{elx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elx}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elx}{l}{lx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elx}{sb}{sbx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elx}{sc}{elsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elx}{sl}{slx}{}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elx}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elx}{ub}{ubx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elx}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elx}{ul}{ulx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elx}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {elx}{x}{elx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {el}{bx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {el}{c}{elc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {el}{eb}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {el}{ec}{elec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {el}{ex}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {el}{sb}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {el}{sc}{elsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {el}{sx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {el}{ub}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {el}{uc}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {el}{ux}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {el}{x}{elx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ex}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ex}{bx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ex}{b}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ex}{eb}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ex}{el}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ex}{l}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ex}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ex}{sb}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ex}{sl}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ex}{ub}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ex}{ul}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lc}{bx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lc}{b}{bc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lc}{c}{lc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lc}{eb}{ebc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lc}{ec}{lec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lc}{el}{elc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lc}{ex}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lc}{l}{lc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lc}{sb}{sbc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lc}{sc}{lsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lc}{sl}{slc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lc}{sx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lc}{ub}{ubc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lc}{uc}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lc}{ul}{ulc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lc}{ux}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lc}{x}{lx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lec}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lec}{b}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lec}{c}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lec}{eb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lec}{ec}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lec}{el}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lec}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lec}{l}   {} {}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lec}{sb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lec}{sc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lec}{sl}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lec}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lec}{ub}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lec}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lec}{ul}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lec}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lec}{x}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lex}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lex}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lex}{b}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lex}{c}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lex}{eb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lex}{ec}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lex}{el}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lex}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lex}{l}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lex}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lex}{sb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lex}{sc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lex}{sl}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lex}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lex}{ub}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lex}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lex}{ul}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lex}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lex}{x}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsc}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsc}{b}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsc}{c}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsc}{eb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsc}{ec}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsc}{el}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsc}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsc}{l}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsc}{sb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsc}{sc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsc}{sl}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsc}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsc}{ub}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsc}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsc}{ul}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsc}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsc}{x}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsx}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsx}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsx}{b}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsx}{c}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsx}{eb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsx}{ec}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsx}{el}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsx}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsx}{l}   {} {}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsx}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsx}{sb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsx}{sc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsx}{sl}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsx}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsx}{ub}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsx}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsx}{ul}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsx}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lsx}{x}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {luc}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {luc}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {luc}{b}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {luc}{c}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {luc}{eb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {luc}{ec}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {luc}{el}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {luc}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {luc}{l}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {luc}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {luc}{sb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {luc}{sc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {luc}{sl}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {luc}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {luc}{ub}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {luc}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {luc}{ul}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {luc}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {luc}{x}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lux}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lux}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lux}{b}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lux}{c}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lux}{eb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lux}{ec}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lux}{el}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lux}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lux}{l}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lux}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lux}{sb}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lux}{sc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lux}{sl}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lux}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lux}{ub}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lux}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lux}{ul}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lux}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lux}{x}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lx}{bx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lx}{b}{bx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lx}{c}{lc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lx}{eb}{ebx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lx}{ec}{lec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lx}{el}{elx}{}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lx}{ex}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lx}{l}{lx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lx}{sb}{sbx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lx}{sc}{lsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lx}{sl}{slx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lx}{sx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lx}{ub}{ubx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lx}{uc}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lx}{ul}{ulx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lx}{ux}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {lx}{x}{lx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {l}{b}  {b}   {bx}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {l}{c}  {lc}  {l}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {l}{eb}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {l}{ec}  {lec}  {l}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {l}{ex}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {l}{sb} {sb}  {b}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {l}{sc}  {lsc}  {l}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {l}{sx}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {l}{ub}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {l}{uc}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {l}{ux}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {l}{x}  {lx}  {l}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {m}{bx} {bx} {b}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {m}{c}  {c}  {m}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {m}{l}  {l}  {m}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {m}{sc}  {sc}  {m}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {m}{x}  {x}  {m}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbc}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbc}{b}{bc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbc}{c}{sbc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbc}{eb}{ebc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbc}{ec}{sbec}{sbc}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbc}{el}{elc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbc}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbc}{l}{lc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbc}{sb}{sbc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbc}{sc}{sbsc}{sbc}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbc}{sl}{slc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbc}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbc}{ub}{ubc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbc}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbc}{ul}{ulc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbc}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbc}{x}{sbx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbec}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbec}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbec}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbec}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbec}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbec}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbec}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbec}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbec}{sb} {} {}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbec}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbec}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbec}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbec}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbec}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbec}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbec}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbec}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbex}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbex}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbex}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbex}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbex}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbex}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbex}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbex}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbex}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbex}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbex}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbex}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbex}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbex}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbex}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbex}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbex}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbex}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbex}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsc}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsc}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsc}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsc}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsc}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsc}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsc}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsc}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsc}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsc}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsc}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsc}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsc}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsc}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsc}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsc}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsc}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsx}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsx}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsx}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsx}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsx}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsx}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsx}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsx}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsx}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsx}{m?} {} {}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsx}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsx}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsx}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsx}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsx}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsx}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsx}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsx}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbsx}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbuc}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbuc}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbuc}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbuc}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbuc}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbuc}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbuc}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbuc}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbuc}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbuc}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbuc}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbuc}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbuc}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbuc}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbuc}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbuc}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbuc}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbuc}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbuc}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbux}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbux}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbux}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbux}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbux}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbux}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbux}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbux}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbux}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbux}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbux}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbux}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbux}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbux}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbux}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbux}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbux}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbux}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbux}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbx}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbx}{b}{bx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbx}{c}{sbc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbx}{eb}{ebx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbx}{ec}{sbec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbx}{el}{elx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbx}{ex}  {} {}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbx}{l}{lx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbx}{sb}{sbx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbx}{sc}{sbsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbx}{sl}{slx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbx}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbx}{ub}{ubx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbx}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbx}{ul}{ulx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbx}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sbx}{x}{sbx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sb}{bx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sb}{c}  {sbc} {bc}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sb}{eb}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sb}{ec}  {sbec} {sbc}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sb}{ex}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sb}{sc}  {sbsc} {sbc}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sb}{sx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sb}{ub}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sb}{uc}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sb}{ux}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sb}{x}  {sbx} {bx}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sc}{bx} {bx}  {b}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sc}{b}{bsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sc}{eb}{ebsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sc}{el}{elsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sc}{l}{lsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sc}{sb}{sbsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sc}{sl}{slsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sc}{ub}{ubsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sc}{x}{x}{m}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slc}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slc}{b}{bc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slc}{c}{slc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slc}{eb}{ebc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slc}{ec}{slec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slc}{el}{elc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slc}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slc}{l}{lc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slc}{sb}{sbc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slc}{sc}{slsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slc}{sl}{slc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slc}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slc}{ub}{ubc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slc}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slc}{ul}{ulc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slc}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slc}{x}{slx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slec}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slec}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slec}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slec}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slec}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slec}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slec}{ex} {} {}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slec}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slec}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slec}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slec}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slec}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slec}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slec}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slec}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slec}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slec}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slex}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slex}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slex}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slex}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slex}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slex}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slex}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slex}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slex}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slex}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slex}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slex}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slex}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slex}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slex}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slex}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slex}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slex}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slex}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsc}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsc}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsc}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsc}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsc}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsc}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsc}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsc}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsc}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsc}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsc}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsc}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsc}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsc}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsc}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsc}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsc}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsx}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsx}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsx}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsx}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsx}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsx}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsx}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsx}{ex} {} {}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsx}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsx}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsx}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsx}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsx}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsx}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsx}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsx}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsx}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsx}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slsx}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sluc}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sluc}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sluc}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sluc}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sluc}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sluc}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sluc}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sluc}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sluc}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sluc}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sluc}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sluc}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sluc}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sluc}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sluc}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sluc}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sluc}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sluc}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sluc}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slux}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slux}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slux}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slux}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slux}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slux}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slux}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slux}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slux}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slux}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slux}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slux}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slux}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slux}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slux}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slux}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slux}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slux}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slux}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slx}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slx}{b}{bx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slx}{c}{slc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slx}{eb}{ebx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slx}{ec}{slec}{}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slx}{el}{elx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slx}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slx}{l}{lx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slx}{sb}{sbx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slx}{sc}{slsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slx}{sl}{slx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slx}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slx}{ub}{ubx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slx}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slx}{ul}{ulx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slx}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {slx}{x}{slx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sl}{bx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sl}{c}{slc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sl}{eb}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sl}{ec}{slec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sl}{ex}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sl}{sb}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sl}{sc}{slsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sl}{sx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sl}{ub}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sl}{uc}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sl}{ux}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sl}{x}{slx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sx}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sx}{bx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sx}{b}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sx}{eb}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sx}{el}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sx}{l}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sx}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sx}{sb}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sx}{sl}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sx}{ub}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {sx}{ul}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubc}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubc}{b}{bc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubc}{c}{ubc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubc}{eb}{ebc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubc}{ec}{ubec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubc}{el}{elc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubc}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubc}{l}{lc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubc}{sb}{sbc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubc}{sc}{ubsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubc}{sl}{slc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubc}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubc}{ub}{ubc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubc}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubc}{ul}{ulc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubc}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubc}{x}{ubx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubec}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubec}{b}  {} {}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubec}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubec}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubec}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubec}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubec}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubec}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubec}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubec}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubec}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubec}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubec}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubec}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubec}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubec}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubec}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubex}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubex}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubex}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubex}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubex}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubex}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubex}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubex}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubex}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubex}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubex}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubex}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubex}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubex}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubex}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubex}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubex}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubex}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubex}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsc}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsc}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsc}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsc}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsc}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsc}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsc}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsc}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsc}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsc}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsc}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsc}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsc}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsc}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsc}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsc}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsc}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsx}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsx}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsx}{b}  {} {}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsx}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsx}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsx}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsx}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsx}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsx}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsx}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsx}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsx}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsx}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsx}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsx}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsx}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsx}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsx}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubsx}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubuc}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubuc}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubuc}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubuc}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubuc}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubuc}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubuc}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubuc}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubuc}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubuc}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubuc}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubuc}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubuc}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubuc}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubuc}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubuc}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubuc}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubuc}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubuc}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubux}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubux}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubux}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubux}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubux}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubux}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubux}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubux}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubux}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubux}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubux}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubux}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubux}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubux}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubux}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubux}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubux}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubux}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubux}{x}  {} {}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubx}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubx}{b}{bx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubx}{c}{ubc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubx}{eb}{ebx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubx}{ec}{ubec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubx}{el}{elx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubx}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubx}{l}{lx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubx}{sb}{sbx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubx}{sc}{ubsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubx}{sl}{slx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubx}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubx}{ub}{ubx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubx}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubx}{ul}{ulx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubx}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ubx}{x}{ubx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ub}{bx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ub}{c}{ubc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ub}{eb}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ub}{ec}{ubec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ub}{ex}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ub}{sb}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ub}{sc}{ubsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ub}{sx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ub}{uc}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ub}{ux}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ub}{x}{ubx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uc}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uc}{bx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uc}{b}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uc}{eb}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uc}{el}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uc}{l}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uc}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uc}{sb}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uc}{sl}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uc}{ub}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uc}{ul}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulc}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulc}{b}{bc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulc}{c}{ulc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulc}{eb}{ebc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulc}{ec}{ulec}{ulc}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulc}{el}{elc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulc}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulc}{l}{lc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulc}{sb}{sbc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulc}{sc}{ulsc}{ulc}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulc}{sl}{slc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulc}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulc}{ub}{ubc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulc}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulc}{ul}{ulc}{}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulc}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulc}{x}{ulx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulec}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulec}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulec}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulec}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulec}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulec}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulec}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulec}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulec}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulec}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulec}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulec}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulec}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulec}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulec}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulec}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulec}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulex}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulex}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulex}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulex}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulex}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulex}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulex}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulex}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulex}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulex}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulex}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulex}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulex}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulex}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulex}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulex}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulex}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulex}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulex}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsc}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsc}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsc}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsc}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsc}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsc}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsc}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsc}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsc}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsc}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsc}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsc}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsc}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsc}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsc}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsc}{ux} {} {}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsc}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsx}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsx}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsx}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsx}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsx}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsx}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsx}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsx}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsx}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsx}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsx}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsx}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsx}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsx}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsx}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsx}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsx}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsx}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulsx}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uluc}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uluc}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uluc}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uluc}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uluc}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uluc}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uluc}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uluc}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uluc}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uluc}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uluc}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uluc}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uluc}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uluc}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uluc}{ub} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uluc}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uluc}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uluc}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {uluc}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulux}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulux}{bx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulux}{b}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulux}{c}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulux}{eb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulux}{ec} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulux}{el} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulux}{ex} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulux}{l}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulux}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulux}{sb} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulux}{sc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulux}{sl} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulux}{sx} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulux}{ub} {} {}
```

```
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulux}{uc} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulux}{ul} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulux}{ux} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulux}{x}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulx}{bx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulx}{b}{bx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulx}{c}{ulc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulx}{eb}{ebx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulx}{ec}{ulec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulx}{el}{elx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulx}{ex}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulx}{l}{lx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulx}{sb}{sbx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulx}{sc}{ulsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulx}{sl}{slx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulx}{sx}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulx}{ub}{ubx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulx}{uc}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulx}{ul}{ulx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulx}{ux}  {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ulx}{x}{ulx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ul}{bx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ul}{c}{ulc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ul}{eb}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ul}{ec}{ulec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ul}{ex}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ul}{sb}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ul}{sc}{ulsc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ul}{sx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ul}{ub}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ul}{uc}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ul}{ux}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ul}{x}{ulx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ux}{?m} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ux}{bx}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ux}{b}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ux}{eb}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ux}{el}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ux}{l}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ux}{m?} {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ux}{sb}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ux}{sl}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ux}{ub}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {ux}{ul}   {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {x}{bx}    {} {}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {x}{c}{c}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {x}{ec}{ec}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {x}{el}{elx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {x}{l}{lx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {x}{sc}{sc}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {x}{sl}{slx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {x}{ub}{ubx}{}
⟨latexrelease⟩\DeclareFontSeriesChangeRule {x}{ul}{ulx}{}
⟨latexrelease⟩
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩    {\DeclareFontSeriesChangeRule}{Series change rules}%
⟨latexrelease⟩
⟨latexrelease⟩\let\DeclareFontSeriesChangeRule\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

1.3 Changing to a new series

```
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩                    {\fontseries}{delay fontseries update}%
```

`\fontseries` The `\fontseries` command takes one argument which is the requested new font series. In the original implementation it simply saved the expanded value in `\f@series`. Now we do a bit more processing and look up the final value in the font series data base. This is done by `\merge@font@series`. But the lookup should be done within the target family and call to `\fontseries` might be followed by a `\fontfamily` call. So we delay the processing to `\selectfont` and only record the necessary action in `\delayed@f@adjustment`.

```
\DeclareRobustCommand\fontseries[1]{\@forced@seriesfalse
    \expandafter\def\expandafter\delayed@f@adjustment\expandafter
        {\delayed@f@adjustment\delayed@merge@font@series{#1}}}
```

(*End of definition for* `\fontseries`.)

`\delayed@f@adjustment` The macro holding the delayed action(s) for use in `\selectfont`.

```
\let\delayed@f@adjustment\@empty
```

(*End of definition for* `\delayed@f@adjustment`.)

`\fontseriesforce` To change unconditionally to a new series you can use `\fontseriesforce`. Of course, if the series doesn't exist for the current family substitution still happens, but there is not dependency on the current series.

```
\DeclareRobustCommand\fontseriesforce[1]{\@forced@seriestrue
    \expandafter\def\expandafter\delayed@f@adjustment\expandafter
      {\delayed@f@adjustment\edef\f@series{#1}}}
```

(*End of definition for* `\fontseriesforce`.)

`\if@forced@series` If the series gets forced we need to know that fact later on.

```
\newif\if@forced@series
```

(*End of definition for* `\if@forced@series`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩                    {\fontseries}{delay fontseries update}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\fontseries[1]{\@forced@seriesfalse
⟨latexrelease⟩                                        \merge@font@series{#1}}
⟨latexrelease⟩\DeclareRobustCommand\fontseriesforce[1]{\@forced@seriestrue
⟨latexrelease⟩                                          \edef\f@series{#1}}
⟨latexrelease⟩\let\delayed@f@adjustment\@undefined
⟨latexrelease⟩
```

For a roll forward we may have to define `\if@forced@series` but this needs doing in a way that TEX doesn't see it when skipping over conditionals.

```
⟨latexrelease⟩\expandafter\newif\csname if@forced@series\endcsname
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\fontseries}{delay fontseries update}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\fontseries[1]{\edef\f@series{#1}}
⟨latexrelease⟩\let\fontseriesforce\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩    {\merge@font@series}{Merge series values}%
```

`\merge@font@series` We look up the data base value by expanding the right command twice. If no such value exist then the result will be `\relax` otherwise it will be the two brace groups: the desired result and the alternate result. The first case means that the third argument to `\merge@font@series` will be empty.

```
\def\merge@font@series#1{%
  \expandafter\expandafter\expandafter
  \merge@font@series@
    \csname series@\f@series @#1\endcsname
    {#1}%
    \@nil
}
```

(*End of definition for* `\merge@font@series`.)

`\merge@font@series@` This now defines the new `\f@series`:

```
\def\merge@font@series@#1#2#3\@nil{%
```

If the third argument is empty there is no database entry for the combination and the second argument holds the new series so we return that.

Originally the test was simply `\ifx!#3!` but that actually dies if `#3` starts with a conditional and in the definition of `\AmSfont` that is actually the case.

```
%\ifcat\expandafter X\detokenize{#1}X%
  \def\reserved@a{#3}%
  \ifx\reserved@a\@empty
    \set@target@series{#2}%
  \else
```

Otherwise we check if the desired result for the series (`#1`) exists for the font family and the current shape. All this happens inside `\selectfont` which has already taken care to load the `.fd`, file if necessary.

```
    \edef\reserved@a{\f@encoding /\f@family /#1/\f@shape}%
     \ifcsname \reserved@a \endcsname
```

If the desired result is available then we use that. However, we do need some post-processing because we need to drop surplus `ms` due to the way naming convention was designed in the '90s (sigh).

```
       \set@target@series{#1}%
```

If not, then we try the alternate result (#2).

```
    \else
       \ifcsname \f@encoding /\f@family /#2/\f@shape \endcsname
```

If the alternate result exist we use that and also issue a warning (or rather a log entry) that we didn't managed to change to the desired font.

```
         \set@target@series{#2}%
         \@font@shape@subst@warning
```

If that doesn't exist either, then we use the requested series unmodified (again with a warning).

```
       \else
         \set@target@series{#3}%
         \@font@shape@subst@warning
       \fi
    \fi
  \fi
}
```

It is possible that the previous font and the new one are actually identical (and the font was not found because it still needs loading) in which case a warning would look rather odd. So we make a quick check for that (which is the reason why we defined `\@reserveda` above instead of doing inline testing inside `\ifcsname`).

```
\def\@font@shape@subst@warning{%
   \edef\reserved@b{\curr@fontshape}%
   \ifx\reserved@a\reserved@b \else
     \@font@warning{Font shape `\reserved@a' undefined\MessageBreak
                    using `\reserved@b' instead}%
   \fi
}
```

(*End of definition for* `\merge@font@series@`.)

`\merge@font@series@without@substitution`
`\merge@font@series@without@substitution@`
`\delayed@merge@font@series`

`\merge@font@series@without@substitution` works like `\merge@font@series`, i.e., it looks up the combination in the rule base and if there exists an entry it uses it and if not it uses the new series value. However, it doesn't check if there is actually a font face with the new series value as `\merge@font@series` does. This simplified command is used in `\selectfont` at a point where other font attributes are not yet updated so that checking the font face might result incorrect in substitutions.

```
\def\merge@font@series@without@substitution#1{%
  \expandafter\expandafter\expandafter
  \merge@font@series@without@substitution@
    \csname series@\f@series @#1\endcsname
    {#1}%
    \@nil
}
\def\merge@font@series@without@substitution@#1#2#3\@nil{%
  \def\reserved@a{#3}%
  \ifx\reserved@a\@empty
    \set@target@series{#2}%
  \else
    \set@target@series{#1}%
  \fi
}
```

(*End of definition for* `\merge@font@series@without@substitution`, `\merge@font@series@without@substitution@`, *and* `\delayed@merge@font@series`.)

`\delayed@merge@font@series` When we delay the merge action in `\fontseries` we first attempt to use merging without substitution. If that results in a non-existing font face the merge is redone in `\selectfont` using a version with substitution. See `\selectfont` for details.

```
\let\delayed@merge@font@series\merge@font@series@without@substitution
```

(*End of definition for* `\delayed@merge@font@series`.)

`\maybe@load@fontshape` A small helper that we use a couple of times: try loading a fontshape (in a group because `\try@load@fontshape` normalizes catcodes and we also want to change `\typeout` so that it doesn't report missing `.fd` files on the terminal).

```
\def\maybe@load@fontshape{%
  \begingroup
    \let \typeout \@font@info
    \try@load@fontshape
  \endgroup}
```

(*End of definition for* `\maybe@load@fontshape`.)

`\set@target@series` Finally the code for normalizing the `\f@series` value.

The combined series value determined by the mapping may still contain an `m` that we have to remove (as the `.fd` files use `c` not `mc` to denote a medium weight condensed series, etc.). We do this in all branches above because a user might have written

```
\DeclareFontSeriesChangeRule {m}{sc}{msc}{mc}
```

instead of using `sc` and `c` as needed in the `.fd` file.

```
\def\set@target@series#1{%
```

We need to `\edef` the argument first in case it starts with a conditional. Then we check (and perhaps drop) an "m" from the value and assign the result to `\f@series`.

```
    \edef\f@series{#1}%
    \series@maybe@drop@one@m\f@series\f@series
}
```

(*End of definition for* `\set@target@series`.)

`\series@maybe@drop@one@m` If the series value is in NFSS notation then it should not contain any "m" unless it is just an "m" by it own. So we need to drop surplus "m"s. But we better don't do this for full names, such as "`semibold`" as used by `autoinst`, for example. So we test against the possible explicit values that should drop an "m". After that we assign the result to `#2` for further use.

```
\def\series@maybe@drop@one@m#1{%
  \expandafter\series@maybe@drop@one@m@x\expandafter{#1}}

\def\series@maybe@drop@one@m@x#1#2{%
```

The code below is an inline version of the `\in@` macro without the group, so that it works in `\accent`.

```
  \def\in@@##1,#1,{}%
  \series@check@toks\expandafter{\in@@
    ,ulm,elm,lm,slm,mm,sbm,bm,ebm,ubm,muc,mec,mc,msc,msx,mx,mex,mux,{}{},#1,}%
  \edef\in@@{\the\series@check@toks}%
  \ifx\in@@\@empty
```

The default definition for `\bfdefault` etc is actually `b\@empty` so that we can detect if the user has changed the default. However that means a) the above test will definitely fail (maybe something to change) and b) we better use `\edef` on the next line to get rid of it as otherwise the test against `#2` (e.g., `\bfdef@ult`) will fail in other places.

```
    \edef#2{#1}%
  \else
    \edef#2{\expandafter\series@drop@one@m #1m\series@drop@one@m}%
  \fi
}
```

As a precaution we use a private toks register not `\toks@` as that is no longer hidden inside the group.

```
\newtoks\series@check@toks
```

(*End of definition for* `\series@maybe@drop@one@m`.)

`\series@drop@one@m` Drop up to two `m`s but keep one if that makes the series value empty. Actually, with the current implementation we know that there is at least one in the series value itself and we added one after it, so all we have to do is now returning `#1#2` and dropping the rest.

```
\def\series@drop@one@m#1m#2m#3\series@drop@one@m{%
%  \ifx\relax#1#2\relax m\else#1#2\fi
   #1#2%
}
```

(*End of definition for* `\series@drop@one@m`.)

Supporting rollback ...

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩    {\merge@font@series}{Merge series values}%
⟨latexrelease⟩
⟨latexrelease⟩\let\merge@font@series\@undefined
⟨latexrelease⟩\let\merge@font@series@\@undefined
⟨latexrelease⟩\let\@font@shape@subst@warning\@undefined
⟨latexrelease⟩\let\merge@font@series@without@substitution\@undefined
⟨latexrelease⟩\let\merge@font@series@without@substitution@\@undefined
⟨latexrelease⟩\let\delayed@merge@font@series\@undefined
⟨latexrelease⟩\let\maybe@load@fontshape\@undefined
⟨latexrelease⟩\let\set@target@series\@undefined
⟨latexrelease⟩\let\series@maybe@drop@one@m\@undefined
⟨latexrelease⟩\let\series@drop@one@m\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

2 Changing the shape

Shapes are also split in two axes (though it could be more if that is desirable), essentially building in an "sc" axis.

```
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩    {\ulcshape}{Font shape change rules}%
```

\DeclareFontShapeChangeRule

The database for shapes is done in exactly the same way, only that it is much smaller and we usually have no alternative shape (or rather it is empty thus not used).

```
\def\DeclareFontShapeChangeRule #1#2#3#4{%
  \@namedef{shape@#1@#2}{{#3}{#4}}}
```

(*End of definition for* `\DeclareFontShapeChangeRule`.)

There is kind of the same problem with returning back from `sc` to normal. It sort of needs its own letter. In `fontspec` this was solved by the first time `\upshape` changes `it` or `sl` back (so only `sc` remains) and second time it changes then `sc` back to normal. Maybe that's not a bad way to handle it, but decided for a slightly different approach: `n` always returns to "normal", ie resets everything and `up` changes italic or slanted to upright and `ulc` undoes small caps.

So we now offer `\normalshape` (using `\shapedefault`) which is normally the same as calling both `\ulcshape` and `\upshape`, only more efficient.

\ulcshape
\textulc
\ulcdefault

To request going back to upper/lowercase we need a new command. It uses `ulc` as shape name but this shape is virtual, i.e., it doesn't exist as a real shape, it is only used as part of the database table entries and thus only appears in the second argument there (but not in the first).

```
\DeclareRobustCommand\ulcshape
        {\not@math@alphabet\ulcshape\relax
         \fontshape\ulcdefault\selectfont}
\let\ulcdefault\@undefined      % for rollback
\newcommand\ulcdefault{ulc}
```

(*End of definition for* `\ulcshape`, `\textulc`, *and* `\ulcdefault`.)

\swshape
\textsw
\swdefault

New command to select a swash shape. The standard rules put this in the same category as italics or slanted, i.e., if you ask for it then italics are undone. One could provide more complicated rules so that `it` + `sw` becomes `swit` but given that there are only very few fonts that have swash letters that level of flexibility (these days) would be just resulting in a lot of combinations that do not exist.

```
\DeclareRobustCommand\swshape
        {\not@math@alphabet\swshape\relax
         \fontshape\swdefault\selectfont}
\let\swdefault\@undefined      % for rollback
\newcommand\swdefault{sw}
```

(*End of definition for* `\swshape`, `\textsw`, *and* `\swdefault`.)

\sscshape
\textssc
\sscdefault

New commands to select spaced small capitals. There isn't a single free font that supports it. However, some commercial ones do, so we offer it. It is also possible to produce spaced small capitals from normal small capitals in OTF fonts using `otftotfm` with calls such as

```
otftotfm -e TEXMF/fonts/enc/dvips/base/texnansx.enc \
  SourceSerifPro-Regular.otf -fkern -fliga --feature=smcp \
  --letterspacing=80 SourceSerifPro-Regular-ssc-LY1
```

Michael Ummels kindly prepared the necessary rules for `ssc` ages ago, but until recently they managed to hide deep down in my inbox. I finally got around to integrating them (with a few changes) and I also took the opportunity to rationalize (a bit) the rules for the only other uncommon shape, `sw`.

```
\DeclareRobustCommand\sscshape
        {\not@math@alphabet\sscshape\relax
         \fontshape\sscdefault\selectfont}
\let\sscdefault\@undefined      % for rollback
\newcommand\sscdefault{ssc}
```

(*End of definition for* `\sscshape`*,* `\textssc`*, and* `\sscdefault`*.*)

Supporting rollback ...

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩    {\ulcshape}{Font shape change rules}%
⟨latexrelease⟩
⟨latexrelease⟩\let\DeclareFontShapeChangeRule\@undefined
⟨latexrelease⟩\let\ulcshape\@undefined
⟨latexrelease⟩\let\ulcdefault\@undefined
⟨latexrelease⟩\let\swshape\@undefined
⟨latexrelease⟩\let\swdefault\@undefined
⟨latexrelease⟩\let\sscshape\@undefined
⟨latexrelease⟩\let\sscdefault\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

2.1 Mapping rules for shape combinations

Many of the entries are commented out as we will get that result without any entry.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease⟩      {\DeclareFontShapeChangeRule}{Rules for ssc and sw}%
%
% \DeclareFontShapeChangeRule {n}{n}   {n}   {}
\DeclareFontShapeChangeRule {n}{it}  {it}  {sl}
\DeclareFontShapeChangeRule {n}{sl}  {sl}  {it}
```

If `sw` is requested but not available (not many font families offer it) we try to fallback to `it` instead of `sw`. That isn't always perfect, because some swash shapes are actually upright, but it is only a fallback and most of the time it would be better than `n`.

```
\DeclareFontShapeChangeRule {n}{sw}  {sw}  {it}
% \DeclareFontShapeChangeRule {n}{sc}  {sc}  {}
\DeclareFontShapeChangeRule {n}{ulc} {n}   {}
\DeclareFontShapeChangeRule {n}{up}  {n}   {}
```

For the `ssc` shape we make the following general assumptions: if `ssc`⟨*X*⟩ exists then `sc`⟨*X*⟩ and ⟨*X*⟩ also exist in the font. If the `ssc`⟨*X*⟩ shape doesn't exist but the user had asked for `ssc` we try to replace it by `sc` because we then assume that the current font family simply doesn't have any `ssc` shapes. However, if we are already in some `ssc` shape and a shape change is requested we know that at least some `ssc` shapes exist for the current font family, so rather than falling back to some `sc` shape we try to stay within `ssc` shapes in a fallback situation.

```
\DeclareFontShapeChangeRule {n}{ssc} {ssc} {sc}
```

```
% \DeclareFontShapeChangeRule {it}{n}   {n}     {}
% \DeclareFontShapeChangeRule {it}{it}  {it}    {}
\DeclareFontShapeChangeRule {it}{sl}  {sl}    {it}
\DeclareFontShapeChangeRule {it}{sw}  {sw}    {it}
```

If neither `scit` nor `scsl` exist then `sc` will be used as a fallback albeit with a log entry, so except for the latter there will be no change for CM or Latin Modern fonts.

```
\DeclareFontShapeChangeRule {it}{sc}  {scit}  {scsl}
\DeclareFontShapeChangeRule {it}{ulc} {it}    {}
\DeclareFontShapeChangeRule {it}{up}  {n}     {}
\DeclareFontShapeChangeRule {it}{ssc} {sscit} {scit}

% \DeclareFontShapeChangeRule {sl}{n}   {n}     {}
\DeclareFontShapeChangeRule {sl}{it}  {it}    {sl}
% \DeclareFontShapeChangeRule {sl}{sl}  {sl}    {}
\DeclareFontShapeChangeRule {sl}{sw}  {sw}    {it}
\DeclareFontShapeChangeRule {sl}{sc}  {scsl}  {scit}
\DeclareFontShapeChangeRule {sl}{ulc} {sl}    {}
\DeclareFontShapeChangeRule {sl}{up}  {n}     {}
\DeclareFontShapeChangeRule {sl}{ssc} {sscsl} {scsl}

% \DeclareFontShapeChangeRule {sc}{n}   {n}     {}
\DeclareFontShapeChangeRule {sc}{it}  {scit}  {scsl}
\DeclareFontShapeChangeRule {sc}{sl}  {scsl}  {scit}
\DeclareFontShapeChangeRule {sc}{sw}  {scsw}  {scit}
% \DeclareFontShapeChangeRule {sc}{sc}  {sc}    {}
\DeclareFontShapeChangeRule {sc}{ulc}  {n}    {}
```

The next rule might be a bit surprising, and rightly so. It would be more correct if `sc` were not affected by `up`, so that it remains `sc` as shown in the commented out rule. However, for nearly three decades commands such as `\upshape` or `\textup` changed small caps back to the "normal" shape. So for backward compatibility we keep that behavior.

As a result you are currently typesetting in `scit` or `scsl` using `\upshape` twice will return you to the normal shape too, the first will change to `sc` and the second (because of the rule below) change that to `n`. This is the way `fontspec` implemented its version on this interface, so this rule means we are also compatible with the way `fontspec` behaved. Still it remains an oddity which I would rather liked to have avoided.

```
% \DeclareFontShapeChangeRule {sc}{up} {sc}     {}
\DeclareFontShapeChangeRule {sc}{up} {n}     {}
% \DeclareFontShapeChangeRule {sc}{ssc}  {ssc}  {}

% \DeclareFontShapeChangeRule {scit}{n}   {n}    {}
\DeclareFontShapeChangeRule {scit}{it}  {scit}  {}
\DeclareFontShapeChangeRule {scit}{sl}  {scsl}  {scit}
\DeclareFontShapeChangeRule {scit}{sw}  {scsw}  {scit}
\DeclareFontShapeChangeRule {scit}{sc}  {scit}  {}
\DeclareFontShapeChangeRule {scit}{ulc} {it}    {}
```

The next rule assumes that if `scit` exists then `it` exists as well. If not, the mechanism will save `ulc` in `\f@series`, which most certainly doesn't exist. So when a font is later selected that would result in a substitution (so no harm done really). Alternatively, we could in this case use `n` as alternative, which may be a bit faster, but such a setup would be so weird in the first place that this isn't worth the effort.

```
\DeclareFontShapeChangeRule {scit}{up}  {sc}    {}
\DeclareFontShapeChangeRule {scit}{ssc} {sscit} {scit}
```

```
% \DeclareFontShapeChangeRule {scsl}{n}   {n}     {}
\DeclareFontShapeChangeRule {scsl}{it}  {scit}  {scsl}
\DeclareFontShapeChangeRule {scsl}{sl}  {scsl}  {}
\DeclareFontShapeChangeRule {scsl}{sw}  {scsw}  {scsl}
\DeclareFontShapeChangeRule {scsl}{sc}  {scsl}  {}
\DeclareFontShapeChangeRule {scsl}{ulc} {sl}    {}
\DeclareFontShapeChangeRule {scsl}{up}  {sc}    {}
\DeclareFontShapeChangeRule {scsl}{ssc} {sscsl} {scsl}

% \DeclareFontShapeChangeRule {scsw}{n}   {n}     {}
\DeclareFontShapeChangeRule {scsw}{it}  {scit}  {scsw}
\DeclareFontShapeChangeRule {scsw}{sl}  {scsl}  {}
\DeclareFontShapeChangeRule {scsw}{sw}  {scsw}  {}
\DeclareFontShapeChangeRule {scsw}{sc}  {scsw}  {}
\DeclareFontShapeChangeRule {scsw}{ulc} {sw}    {}
\DeclareFontShapeChangeRule {scsw}{up}  {sc}    {}
\DeclareFontShapeChangeRule {scsw}{ssc} {sscsw} {scsw}

% \DeclareFontShapeChangeRule {sw}{n}   {n}     {}
% \DeclareFontShapeChangeRule {sw}{it}  {it}    {}
% \DeclareFontShapeChangeRule {sw}{sl}  {sl}    {}
% \DeclareFontShapeChangeRule {sw}{sw}  {sw}    {}
\DeclareFontShapeChangeRule {sw}{sc}  {scsw}  {scit}
\DeclareFontShapeChangeRule {sw}{ulc} {sw}    {}
\DeclareFontShapeChangeRule {sw}{up}  {n}     {}
\DeclareFontShapeChangeRule {sw}{ssc} {sscsw} {scsw}

% \DeclareFontShapeChangeRule {ssc}{n}   {n}      {}
\DeclareFontShapeChangeRule {ssc}{it}  {sscit}  {sscsl}
\DeclareFontShapeChangeRule {ssc}{sl}  {sscsl}  {sscit}
\DeclareFontShapeChangeRule {ssc}{sw}  {sscsw}  {sscit}
\DeclareFontShapeChangeRule {ssc}{sc}  {sc}     {}
% \DeclareFontShapeChangeRule {ssc}{ssc}  {ssc}   {}
\DeclareFontShapeChangeRule {ssc}{ulc} {n}      {}
```

We implement the same logic as for `sc`, see above. The `ssc` shape doesn't have to care about 30 years of history, but it would be surprising if `\sscshape\upshape` did not work like `\scshape\upshape`.

```
% \DeclareFontShapeChangeRule {ssc}{up} {ssc}     {}
\DeclareFontShapeChangeRule {ssc}{up}  {n}      {}

% \DeclareFontShapeChangeRule {sscit}{n}    {n}     {}
\DeclareFontShapeChangeRule {sscit}{it}   {sscit} {}
\DeclareFontShapeChangeRule {sscit}{sl}   {sscsl} {sscit}
\DeclareFontShapeChangeRule {sscit}{sw}   {sscsw} {sscit}
\DeclareFontShapeChangeRule {sscit}{ssc}  {sscit} {}
\DeclareFontShapeChangeRule {sscit}{sc}   {scit}  {}
\DeclareFontShapeChangeRule {sscit}{ulc}  {it}    {}
\DeclareFontShapeChangeRule {sscit}{up}   {ssc}   {}

% \DeclareFontShapeChangeRule {sscsl}{n}   {n}      {}
\DeclareFontShapeChangeRule {sscsl}{it}  {sscit}  {sscsl}
\DeclareFontShapeChangeRule {sscsl}{sl}  {sscsl}  {}
\DeclareFontShapeChangeRule {sscsl}{sw}  {sscsw}  {sscit}
\DeclareFontShapeChangeRule {sscsl}{sc}  {scsl}   {}
\DeclareFontShapeChangeRule {sscsl}{ulc} {sl}     {}
\DeclareFontShapeChangeRule {sscsl}{up}  {ssc}    {}
```

```
% \DeclareFontShapeChangeRule {sscsw}{n}   {n}      {}
\DeclareFontShapeChangeRule {sscsw}{it}   {sscit} {sscsl}
\DeclareFontShapeChangeRule {sscsw}{sl}   {sscsl} {sscit}
\DeclareFontShapeChangeRule {sscsw}{sw}   {sscsw} {}
\DeclareFontShapeChangeRule {sscsw}{ssc}  {sscsw} {}
\DeclareFontShapeChangeRule {sscsw}{sc}   {scsw}  {scit}
\DeclareFontShapeChangeRule {sscsw}{ulc}  {sw}    {it}
\DeclareFontShapeChangeRule {sscsw}{up}   {ssc}   {}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩       {\DeclareFontShapeChangeRule}{Rules for ssc and sw}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareFontShapeChangeRule {n}{it}  {it}  {sl}
⟨latexrelease⟩\DeclareFontShapeChangeRule {n}{sl}  {sl}  {it}
⟨latexrelease⟩\DeclareFontShapeChangeRule {n}{ulc} {n}   {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {n}{up} {n}    {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {it}{sl}  {sl}    {it}
⟨latexrelease⟩\DeclareFontShapeChangeRule {it}{sc}  {scit}  {scsl}
⟨latexrelease⟩\DeclareFontShapeChangeRule {it}{ulc} {it}    {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {it}{up} {n}      {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sl}{it}  {it}    {sl}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sl}{sc}  {scsl}  {scit}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sl}{ulc} {sl}    {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sl}{up} {n}      {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sc}{it}  {scit}  {scsl}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sc}{sl}  {scsl}  {scit}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sc}{sw}  {scsw}  {sw}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sc}{ulc} {n}     {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sc}{up} {n}     {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {scit}{it}  {scit}  {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {scit}{sl}  {scsl}  {scit}
⟨latexrelease⟩\DeclareFontShapeChangeRule {scit}{sw}  {scsw}  {sc}    % or scit?
⟨latexrelease⟩\DeclareFontShapeChangeRule {scit}{sc}  {scit}  {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {scit}{ulc} {it}    {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {scit}{up} {sc}     {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {scsl}{it}  {scit}  {scsl}
⟨latexrelease⟩\DeclareFontShapeChangeRule {scsl}{sl}  {scsl}  {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {scsl}{sw}  {scsw}  {sc}    % or scsl?
⟨latexrelease⟩\DeclareFontShapeChangeRule {scsl}{sc}  {scsl}  {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {scsl}{ulc} {sl}    {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {scsl}{up}   {sc}   {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {scsw}{it}  {scit}  {scsw}
⟨latexrelease⟩\DeclareFontShapeChangeRule {scsw}{sl}  {scsl}  {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {scsw}{sw}  {scsw}  {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {scsw}{sc}  {scsw}  {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {scsw}{ulc} {sw}    {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {scsw}{up} {sc}     {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sw}{sc}  {scsw}  {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sw}{ulc} {sw}    {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sw}{up} {n}      {}
⟨latexrelease⟩
```

In 2022-02-02 the `ssc` shape had no rules, so that any use of it would have resulted in

just switching to that shape or switching away from it; so we mimic that by providing the corresponding trivial rules to overrule those from above when we roll back.

```
⟨latexrelease⟩\DeclareFontShapeChangeRule {n}{ssc}   {ssc} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {it}{ssc}  {ssc} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sl}{ssc}  {ssc} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sw}{ssc}  {ssc} {}
⟨latexrelease⟩
⟨latexrelease⟩\DeclareFontShapeChangeRule {scit}{ssc}{ssc} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {scsl}{ssc}{ssc} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {scsw}{ssc}{ssc} {}
⟨latexrelease⟩
⟨latexrelease⟩\DeclareFontShapeChangeRule {ssc}{it}  {it} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {ssc}{sl}  {sl} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {ssc}{sw}  {sw} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {ssc}{sc}  {sc} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {ssc}{ulc} {n}  {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {ssc}{up}  {n}  {}
⟨latexrelease⟩
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscit}{it}   {it} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscit}{sl}   {sl} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscit}{sw}   {sw} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscit}{ssc}  {ssc}{}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscit}{sc}   {sc} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscit}{ulc}  {n}  {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscit}{up}   {ssc}{}
⟨latexrelease⟩
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscsl}{it}  {it} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscsl}{sl}  {sl} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscsl}{sw}  {sw} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscsl}{sc}  {sc} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscsl}{ulc} {n}  {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscsl}{up}  {ssc}{}
⟨latexrelease⟩
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscsw}{it}   {it} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscsw}{sl}   {sl} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscsw}{sw}   {sw} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscsw}{ssc}  {ssc}{}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscsw}{sc}   {sc} {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscsw}{ulc}  {n}  {}
⟨latexrelease⟩\DeclareFontShapeChangeRule {sscsw}{up}   {ssc}{}
⟨latexrelease⟩\EndIncludeInRelease
```

Before 2020-02-02 we don't really care about any existing shape change rules since the mechanism isn't used.

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩       {\DeclareFontShapeChangeRule}{Rules for ssc and sw}%
⟨latexrelease⟩\EndIncludeInRelease
```

2.2 Changing to a new shape

```
⟨∗2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩    {\fontshape}{Font shape change}%
```

\fontshape Again the \fontshape now has to do a lookup to get to its new value in \f@shape. The method is exactly the same as in \fontseries.

```
\DeclareRobustCommand\fontshape[1]
   {\expandafter\def\expandafter\delayed@f@adjustment\expandafter
        {\delayed@f@adjustment\delayed@merge@font@shape{#1}}}
```

(*End of definition for* \fontshape.)

\fontshapeforce The unconditional version:

```
\DeclareRobustCommand\fontshapeforce[1]
   {\expandafter\def\expandafter\delayed@f@adjustment\expandafter
        {\delayed@f@adjustment\edef\f@shape{#1}}}
```

(*End of definition for* \fontshapeforce.)

Supporting rollback ...

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩    {\fontshape}{Font shape change}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\fontshape[1]{\merge@font@shape{#1}}
⟨latexrelease⟩\DeclareRobustCommand\fontshapeforce[1]{\edef\f@shape{#1}}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩    {\fontshape}{Font shape change}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\fontshape [1]{\edef\f@shape{#1}}
⟨latexrelease⟩\let\fontshapeforce\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩    {\merge@font@shape}{Font shape change rules}%
```

\merge@font@shape Look up the database entry (if existing) and act accordingly.

```
\def\merge@font@shape#1{%
  \expandafter\expandafter\expandafter
  \merge@font@shape@
    \csname shape@\f@shape @#1\endcsname
    {#1}%
    \@nil
}
```

(*End of definition for* \merge@font@shape.)

\merge@font@shape@ Same game now, except that we look at shapes not series values and we can set the shape without the complication of dropping "m"s from the name as we had to for the series.

```
\def\merge@font@shape@#1#2#3\@nil{%
  \def\reserved@a{#3}%
  \ifx\reserved@a\@empty
    \edef\f@shape{#2}%
  \else
```

\reserved@a is used in \@font@shape@subst@warning so we have to define it in addition to do the \ifcsname test

```
    \edef\reserved@a{\f@encoding /\f@family /\f@series/#1}%
    \ifcsname \reserved@a\endcsname
       \edef\f@shape{#1}%
    \else
       \ifcsname \f@encoding /\f@family /\f@series/#2\endcsname
         \edef\f@shape{#2}%
         \@font@shape@subst@warning
       \else
         \edef\f@shape{#3}%
         \@font@shape@subst@warning
       \fi
    \fi
  \fi
}
```

(*End of definition for* \merge@font@shape@.)

\merge@font@shape@without@substitution
\merge@font@shape@without@substitution@
\delayed@merge@font@shape

See definition of \selectfont for how these macros are used.

```
\def\merge@font@shape@without@substitution#1{%
  \expandafter\expandafter\expandafter
  \merge@font@shape@without@substitution@
    \csname shape@\f@shape @#1\endcsname
    {#1}%
    \@nil
}
\def\merge@font@shape@without@substitution@#1#2#3\@nil{%
  \def\reserved@a{#3}%
  \ifx\reserved@a\@empty
    \edef\f@shape{#2}%
  \else
    \edef\f@shape{#1}%
  \fi
}
\let\delayed@merge@font@shape\merge@font@shape@without@substitution
```

(*End of definition for* \merge@font@shape@without@substitution, \merge@font@shape@without@substitution@, *and* \delayed@merge@font@shape.)

\normalshape

\normalshape resets both sub-axes if the default rules are used.

```
\protected\def\normalshape
    {\not@math@alphabet\normalshape\relax
      \fontshape\shapedefault\selectfont}%
```

(*End of definition for* \normalshape.)

3 Make sure we win ...

This code implements one aspect of what the package fontaxes provide. So its redefinitions for the various shape commands, such as \itshape should no longer happen. We therefore force the standard definitions at \AtBeginDocument (later when this is defined). Once fontaxes is no longer doing such redefinitions that could be taken out again.

We use a separate macro so that we can easily disable this (in case of rollback).

\reinstall@nfss@defs I use \protected here not \DeclareRobustCommand to avoid extra status lines.

```
\def\reinstall@nfss@defs{%
  \protected\def\upshape
          {\not@math@alphabet\upshape\relax
           \fontshape\updefault\selectfont}%
  \protected\def\slshape
          {\not@math@alphabet\slshape\relax
           \fontshape\sldefault\selectfont}%
  \protected\def\scshape
          {\not@math@alphabet\scshape\relax
           \fontshape\scdefault\selectfont}%
  \protected\def\itshape
          {\not@math@alphabet\itshape\mathit
           \fontshape\itdefault\selectfont}%
  \protected\def\ulcshape
          {\not@math@alphabet\ulcshape\relax
           \fontshape{ulc}\selectfont}%
  \protected\def\swshape
          {\not@math@alphabet\swshape\relax
           \fontshape\swdefault\selectfont}%
  \protected\def\sscshape
          {\not@math@alphabet\sscshape\relax
           \fontshape\sscdefault\selectfont}%
}
```

(*End of definition for* \reinstall@nfss@defs.)

Supporting rollback ...

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩    {\merge@font@shape}{Font shape change rules}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\fontshape [1]{\edef\f@shape{#1}}
⟨latexrelease⟩\let\fontshapeforce\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let\merge@font@shape\@undefined
⟨latexrelease⟩\let\merge@font@shape@\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let\merge@font@shape@without@substitution\@undefined
⟨latexrelease⟩\let\merge@font@shape@without@substitution@\@undefined
⟨latexrelease⟩\let\delayed@merge@font@shape\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let\normalshape\@undefined
⟨latexrelease⟩
```

This is always called in \document so don't make it undefined.

```
⟨latexrelease⟩
⟨latexrelease⟩\let\reinstall@nfss@defs\relax
⟨latexrelease⟩\EndIncludeInRelease
```

This initializes the 2020/02/02 extensions to NFSS after any changes in the preamble.

```
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
```

```
⟨latexrelease⟩                  {\reinstall@nfss@defs}{NFSS series init}%
\g@addto@macro\@kernel@after@begindocument@before
                {\reinstall@nfss@defs\init@series@setup}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

The initialization was introduced in 2020/02/02 but

```
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩                  {\reinstall@nfss@defs}{NFSS series init}%
⟨latexrelease⟩\AtBeginDocument{\reinstall@nfss@defs\init@series@setup}
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\reinstall@nfss@defs}{NFSS series init}%
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
⟨/2ekernel⟩
```

File 26
ltfsstrc.dtx

1 Introduction

This package contains the code for tracing font loading and font changes. It basically overlays some of the low-level functions of NFSS with additional code used for tracing.

The package accepts the following options:

errorshow Write all information about font changes etc. only to the transcript file unless an error happens. This means that information about font substitution will not be shown on the terminal.

warningshow Show all NFSS warnings on the terminal. This setting corresponds to the default behaviour of NFSS if the `tracefnt` package is *not* loaded!

infoshow Show all NFSS warning and all NFSS info messages (that are normally only written to the transcript file) also on the terminal. This is the default if the `tracefnt` package is loaded.

debugshow In addition to `infoshow` show also changing of math fonts as far as possible (this option can produce a large amount of output).

loading Show the name of external fonts when they are loaded. This option shows only "newly" loaded fonts not those already preloaded in the format or the class file before the `tracefnt` package became active.

pausing Turn all font warnings into errors so that LaTeX will stop.

2 A driver for this document

The next bit of code contains the documentation driver file for TeX, i.e., the file that will produce the documentation you are currently reading. It will be extracted from this file by the DocStrip program.

When this file is processed directly by LaTeX this will produce the documentation as well.

```
1  ⟨*driver⟩
2  \documentclass{ltxdoc}
3
4
5  %\OnlyDescription  % comment out for implementation details
6
7  \begin{document}
8     \DocInput{ltfsstrc.dtx}
9  \end{document}
10 ⟨/driver⟩
```

3 The Implementation

Warning: Read the macro documentation with a grain of salt. It is still basically the documentation from the first NFSS release and therefore in some cases probably not completely accurate.

If we are making a package file it is a good idea to test whether we are running under 2e. This code is actually placed at the very beginning of this file for easier maintenance, thus commented out here.

```
⟨*package⟩
%\NeedsTeXFormat{LaTeX2e}
%\ProvidesPackage{tracefnt}[??/??/?? v?.??
%                           Standard LaTeX package (font tracing)]
⟨/package⟩
```

The `debug` module makes use of commands contained in a special package file named `trace.sty`.[34]

```
⟨+debug⟩ \input trace.sty
```

4 Handling Options

`\tracingfonts` Here is the definition of the integer register for the font trace. As a default in a package file we use 1 to give error messages if fonts are substituted. If this code is used for debugging or tracing reasons in the format file (i.e. in `fam.dtx`) we use 0 as the default. But if no font trace is used we build a definition that will produce a warning message.

```
⟨*2ekernel⟩
\message{NFSS tracing,}
\def\tracingfonts{%
  \@font@warning{Command \noexpand\tracingfonts
            not provided.\MessageBreak
            Use the 'tracefnt' package.\MessageBreak Command found:}%
        \count@}
⟨/2ekernel⟩
```

The `\count@` in the line above will remove the number after `\tracingfonts`. Note that this definition will be overwritten be the next line if one of these modules are included.

```
⟨*package, trace, debug⟩
\newcount\tracingfonts
\tracingfonts=0
⟨/package, trace, debug⟩
```

(*End of definition for* `\tracingfonts`.)

The option `errorshow` turns off all warnings so that only real errors are shown. `warningshow` corresponds to the NFSS default (when `tracefnt` is not loaded). `infoshow` is the default for this package here; and `debugshow`, `loading`, and `pausing` extend the amount of information even further.

```
⟨*package⟩
\DeclareOption{errorshow}{%
   \def\@font@info#1{%
         \GenericInfo{(Font)\@spaces\@spaces\@spaces\space\space}%
```

[34] This package is not in distribution at the moment (and probably doesn't any longer work). Think of this part of the code as being historical artifacts.

```
                    {LaTeX Font Info: \space\space\space#1}}%
    \def\@font@warning#1{%
         \GenericInfo{(Font)\@spaces\@spaces\@spaces\space\space}%
                        {LaTeX Font Warning: #1}}%
     }
\DeclareOption{warningshow}{%
   \def\@font@info#1{%
         \GenericInfo{(Font)\@spaces\@spaces\@spaces\space\space}%
                    {LaTeX Font Info: \space\space\space#1}}%
    \def\@font@warning#1{%
         \GenericWarning{(Font)\@spaces\@spaces\@spaces\space\space}%
                        {LaTeX Font Warning: #1}}%
     }
\DeclareOption{infoshow}{%
   \def\@font@info#1{%
         \GenericWarning{(Font)\@spaces\@spaces\@spaces\space\space}%
                    {LaTeX Font Info: \space\space\space#1}}%
    \def\@font@warning#1{%
         \GenericWarning{(Font)\@spaces\@spaces\@spaces\space\space}%
                        {LaTeX Font Warning: #1}}%
     }
\DeclareOption{loading}{%
    \tracingfonts\tw@
   }
\DeclareOption{debugshow}{%
    \ExecuteOptions{infoshow}%
    \tracingfonts\thr@@
   }
\DeclareOption{pausing}{%
    \def\@font@warning#1{%
      \GenericError
            {(Font)\@spaces\@spaces\@spaces\space\space}%
            {LaTeX Font Warning: #1}%
            {See the LaTeX Companion for details.}%
            {I'll stop for every LaTeX Font Warning because
             you requested\MessageBreak the 'pausing' option
             to the tracefnt package.}}%
   }
```

We make `infoshow` the default, which in turn defines `\font@warning` and `\font@info`.

```
\ExecuteOptions{infoshow}
\ProcessOptions
⟨/package⟩
```

We also need a default definition inside the kernel:

```
⟨*2ekernel⟩
\def\@font@info#1{%
         \GenericInfo{(Font)\@spaces\@spaces\@spaces\space\space}%
                    {LaTeX Font Info: \space\space\space#1}}%
\def\@font@warning#1{%
         \GenericWarning{(Font)\@spaces\@spaces\@spaces\space\space}%
                        {LaTeX Font Warning: #1}}%
⟨/2ekernel⟩
```

5 Macros common to fam.tex and tracefnt.sty

In the first versions of tracefnt.dtx some macros of fam.dtx[35] were redefined to included the extra tracing information. Now these macros are all defined in this file (i.e. removed from fam.dtx) and different production versions can be obtained simply by specifying a different set of modules to include when generating ltfss.dtx.

5.1 General font loading

\extract@font This macro organizes the font loading. It first calls \get@external@font which will return in \external@font the name of the external font file (the .tfm) as it was determined by the NFSS tables.

```
⟨*2ekernel | package⟩
\def\extract@font{%
   \get@external@font
```

Then the external font is loaded and assigned to the font identifier stored inside \font@name (for this reason we need \expandafter).

```
   \global\expandafter\font\font@name\external@font\relax
```

When tracing we typeout the internal and external font name.

```
⟨*trace⟩
    \ifnum \tracingfonts >\@ne
    \@font@info{External font `\external@font'
              loaded as\MessageBreak \font@name}\fi
⟨/trace⟩
```

Finally we call the corresponding "loading action" macros to finish things. First the font is locally selected to allow the use of \font inside the loading action macros.

```
    \font@name \relax
```

The next two lines execute the "loading actions" for the family and then for the individual font shape.

```
    \csname \f@encoding+\f@family\endcsname
    \csname\curr@fontshape\endcsname
    \relax
       }
⟨/2ekernel | package⟩
```

The \relax at the end needs to be explained. This is inserted to prevent TEX from scanning too far when it is executing the replacement text of the loading code macros.

(*End of definition for* \extract@font.)

\get@external@font This function tries to find an external font name. It will place the name into the macro \external@font. If no font is found it will return the one that was defined via \DeclareErrorFont.

```
⟨*2ekernel⟩
\def\get@external@font{%
```

We don't know the external font name at the beginning.

```
   \let\external@font\@empty
   \edef\font@info{\expandafter\expandafter\expandafter\string
        \csname \curr@fontshape \endcsname}%
   \try@size@range
```

[35] This file is currently not distributed in documented form. Its code is part of ltfss.dtx.

If this failed, we'll try to substitute another size of the same font. This is done by the `\try@size@substitution` macro. It "knows about" `\do@extract@font`, `\font@name`, `\f@size`, and so on.

```
    \ifx\external@font\@empty
       \try@size@substitution
       \ifx\external@font\@empty
          \@latex@error{Font \expandafter \string\font@name\space
                        not found}\@eha
          \error@fontshape
          \get@external@font
    \fi\fi
}
⟨/2ekernel⟩
```

(*End of definition for* `\get@external@font`.)

```
⟨*2ekernel | latexrelease | package⟩
⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩                    {\selectfont}{Add hook to \selectfont}%
```

`\selectfont` The macro `\selectfont` is called whenever a font change must take place.

```
\DeclareRobustCommand\selectfont
        {%
```

When `debug` is specified we actually want something like 'undebug'. The font selection is now stable so that using `\tracingall` on some other macros will show us a lot of unwanted information about font loading. Therefore we disable tracing during font loading as long as `\tracingfonts` is less than 4.

```
⟨+debug⟩  \pushtracing
⟨+debug⟩  \ifnum\tracingfonts<4 \tracingoff
⟨+debug⟩  \else \tracingon\p@selectfont \fi
```

If `\baselinestretch` was redefined by the user it will not longer match its internal counterpart `\f@linespread`. If so we call `\set@fontsize` to prepare `\size@update`.

```
    \ifx\f@linespread\baselinestretch \else
      \set@fontsize\baselinestretch\f@size\f@baselineskip \fi
```

The series and shape updates are only prepared by `\fontseries` and `\fontshape` but not executed until after we are ready to change the font face. This way they happen after a possibly new family is set which is important because they look at the available font faces in that family and alter the selection based on availability. Several calls to `\fontseries` or `\fontshape` are delayed in the order in which they appear, so that by switching them one can work around missing intermediate font faces and avoid substitutions.

We first attempt to do the merge without any substitution. As we might end up with a non-existing font face we may have to restart and therefore save the current values of `\f@series` and `\f@shape` before the merge.

But first we make a quick test to see if there are any delayed actions, because if not it is pointless to make all the assignments and try loading a missing fontshape.

```
    \ifx\delayed@f@adjustment\@empty
    \else
      \let\f@shape@saved\f@shape
      \let\f@series@saved\f@series
```

Then we run the delayed adjustments (which use the `\..@without@substitution` commands):

```
\delayed@f@adjustment
```

We then check if the resulting cominbation is valid but for this we have to make sure that the appropriate `.fd` is loaded if that hasn't happened so far.

```
\maybe@load@fontshape
\ifcsname \f@encoding/\f@family/\f@series/\f@shape \endcsname
```

If this macro is defined then we are good and no further action is necessary.

Otherwise the combination is not valid, so we redo the merge but this time with substitutions.

```
\else
  \let\f@shape\f@shape@saved
  \let\f@series\f@series@saved
  \let\delayed@merge@font@shape\merge@font@shape
  \let\delayed@merge@font@series\merge@font@series
  \delayed@f@adjustment
  \let\delayed@merge@font@shape\merge@font@shape@without@substitution
  \let\delayed@merge@font@series\merge@font@series@without@substitution
\fi
```

Now the series and shape values are updated and we clear `\delayed@f@adjustment`. This is important because on the next execution of `\selectfont` we should not mistakenly redo the delayed actions if there wasn't any series or shape change.

```
  \let\delayed@f@adjustment\@empty
\fi
```

If the series was forced we should now cancel that in case the next series change is done with some low-level setting to `\f@series`.

```
\@forced@seriesfalse
```

Then we generate the internal name of the font by concatenating *family*, *series*, *shape*, and current *size*, with slashes as delimiters between them. This is much more readable than standard LaTeX's `\twfbf`, etc. We define `\font@name` globally, as always. The reason for this is explained later on.

```
\xdef\font@name{%
  \csname\curr@fontshape/\f@size\endcsname}%
```

We call the macro `\pickup@font` which will load the font if necessary.

```
\pickup@font
```

Then we select the font.

```
\font@name
```

After switching fonts we run a hook, so that packages can make last minute alterations based on the new font (originally provided in everysel but using a different interface).

```
\UseHook{selectfont}%
```

Finally we call `\size@update`. This macro is normally empty but will contain actions (like setting the `\baselineskip`) that have to be carried out when the font size, the base `\baselineskip` or the `\baselinestretch` have changed.

```
\size@update
```

A similar function is called to handle anything related to encoding updates. This one is changed from `\relax` by `\fontencoding`.

```
    \enc@update
```

Just before ending this macro we have to pop the tracing stack if it was pushed before.

```
⟨+debug⟩ \poptracing
    }
```

(*End of definition for* `\selectfont`.)

`selectfont` Declare the hook used in selecfont in the kernel, but not inside the tracefnt package.

```
⟨-trace⟩\NewHook{selectfont}
```

(*End of definition for* `selectfont`.)

If `\tracingfonts` is greater than 2 we also show the font switch inside `\selectfont`. We do this by adding this code to the hook in the tracefnt package: macro might redefine `\font@name`.

```
⟨*trace⟩
\AddToHook{selectfont}
   {\ifnum \tracingfonts>\tw@
       \@font@info{Switching to \font@name}\fi}
⟨/trace⟩
⟨/2ekernel | latexrelease | package⟩
⟨latexrelease⟩\EndIncludeInRelease
```

With `\selectfont` having different definitions in different kernels we also have to provide them in the `tracefnt` package to support rollback. In packages that works a bit differently and therefore we have to provide an empty block there.

```
⟨package⟩\IncludeInRelease{2021/06/01}%
⟨package⟩                  {\selectfont}{Add hook to \selectfont}%
⟨package⟩\EndIncludeInRelease

⟨latexrelease | package⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease | package⟩                  {\selectfont}{Add hook to \selectfont}%
⟨latexrelease | package⟩
⟨latexrelease | package⟩\DeclareRobustCommand\selectfont
⟨latexrelease | package⟩   {%
⟨latexrelease | package⟩    \ifx\f@linespread\baselinestretch \else
⟨latexrelease | package⟩      \set@fontsize\baselinestretch\f@size\f@baselineskip \fi
⟨latexrelease | package⟩    \xdef\font@name{%
⟨latexrelease | package⟩      \csname\curr@fontshape/\f@size\endcsname}%
⟨latexrelease | package⟩    \pickup@font
⟨latexrelease | package⟩    \font@name
⟨latexrelease | package⟩    \size@update
⟨latexrelease | package⟩    \enc@update
⟨latexrelease | package⟩   }
⟨latexrelease | package⟩
⟨latexrelease | package⟩\EndIncludeInRelease
```

`\set@fontsize` The macro `\set@fontsize` does the actual work. First it assigns new values to `\f@size`, `\f@baselineskip` and `\f@linespread`.

```
⟨*2ekernel | package⟩
\def\set@fontsize#1#2#3{%
    \@defaultunits\@tempdimb#2pt\relax\@nnil
```

```
    \edef\f@size{\strip@pt\@tempdimb}%
    \@defaultunits\@tempskipa#3pt\relax\@nnil
    \edef\f@baselineskip{\the\@tempskipa}%
    \edef\f@linespread{#1}%
```

For backward compatibility and for later testing within `\selectfont` the internal value of `\f@linespread` is passed back to `\baselinestretch`.

```
    \let\baselinestretch\f@linespread
```

Additional processing will happen within `\selectfont`. For this reason the macro `\size@update` (which will be called in `\selectfont`) will be defined to be:

```
      \def\size@update{%
```

First calculate the new `\baselineskip` and also store it in `normalbaselineskip`

```
        \baselineskip\f@baselineskip\relax
        \baselineskip\f@linespread\baselineskip
        \normalbaselineskip\baselineskip
```

then to set up a new `\strutbox`

```
        \setbox\strutbox\hbox{%
          \vrule\@height.7\baselineskip
                \@depth.3\baselineskip
                \@width\z@}%
```

We end with a bit of tracing information.

```
⟨*trace⟩
   \ifnum \tracingfonts>\tw@
      \ifx\f@linespread\@empty
        \let\reserved@a\@empty
      \else
        \def\reserved@a{\f@linespread x}%
      \fi
      \@font@info{Changing size to \f@size/\reserved@a
                \f@baselineskip}%
      \aftergroup\type@restoreinfo \fi
⟨/trace⟩
```

When all this is processed `\size@update` redefines itself to `\relax` so that in later calls of `\selectfont` no extra code will be executed.

```
        \let\size@update\relax}%
  }
```

Instead of defining this macro internally we might speed things up by placing the code into a separate macro and use `\let`!

(*End of definition for* `\set@fontsize`.)

`\size@update` Normally this macro does nothing; it will be redefined by `\set@fontsize` to initiate an update.

```
\let\size@update\relax
```

(*End of definition for* `\size@update`.)

\type@restoreinfo This macro produces some info when a font size and/or baseline change will get restored.

```
⟨*trace⟩
   \def\type@restoreinfo{%
      \ifx\f@linespread\@empty
        \let\reserved@a\@empty
      \else
        \def\reserved@a{\f@linespread x}%
      \fi
      \@font@info{Restoring size to
                \f@size/\reserved@a\f@baselineskip}}
⟨/trace⟩
```

(*End of definition for* \type@restoreinfo.)

\glb@settings The macro \glb@settings globally selects all math fonts for the current size if necessary.
\glb@currsize

```
\def\glb@settings{%
```

When \glb@settings gains control a size change was requested and all previous font assignments need to be replaced. Therefore the old values of the fonts are no longer needed. For every *math group* the new assignments are appended to \math@fonts. But this happens only if the math@fonts switch is set to true. However, we always set up the correct math sizes for script and scriptscript fonts since they may be needed even if we don't set up the whole math machinery.

Here we set the math size, script size and scriptscript size. If the S@... macro is not defined we have to first calculate the three sizes.

```
      \expandafter\ifx\csname S@\f@size\endcsname\relax
        \calculate@math@sizes
      \fi
```

The effect of this is that \calculate@math@sizes may or may not define the S@... macro. In the first case the next time the same size is requested this macro is used, otherwise \calculate@math@sizes is called again. This also sets the math@fonts switch. If it is true we must switch the math fonts.

```
      \csname S@\f@size\endcsname
      \ifmath@fonts
⟨*trace⟩
        \ifnum \tracingfonts>\tw@
          \@font@info{Setting up math fonts for
                      \f@size/\f@baselineskip}\fi
⟨/trace⟩
```

Inside a group we execute the macro for the current math *version*. This sets \math@fonts to a list of \textfont... assignments. \getanddefine@fonts (which may be called at this point) needs the \escapechar parameter to be set to −1.

```
        \begingroup
          \escapechar\m@ne
          \csname mv@\math@version \endcsname
```

Then we set \globaldefs to 1 so that all following changes are done globally. The math font assignments recorded in \math@fonts are executed and \glb@currsize is set equal to \f@size. This signals that the fonts for math in this size are set up.

```
          \globaldefs\@ne
          \math@fonts
```

```
      \let \glb@currsize \f@size
    \endgroup
```

Finally we execute any code that is supposed to happen whenever the math font setup changes. This register will be executed in local mode which means that everything that is supposed to have any effect should be done globally inside. We can't execute it within `\globaldefs\@ne` as we don't know what ends up inside this register, e.g., it might contain calculations which use some local registers to calculate the final (global) value.

```
     \the\every@math@size
```

Otherwise we announce that the math fonts are not set up for this size.

```
⟨*trace⟩
    \else
      \ifnum \tracingfonts>\tw@
        \@font@info{No math setup for
                    \f@size/\f@baselineskip}\fi
⟨/trace⟩
  \fi
}
⟨/2ekernel | package⟩
```

(*End of definition for* `\glb@settings` *and* `\glb@currsize`.)

`\baselinestretch` In `\selectfont` we used `\baselinestretch` as a factor when assigning a value to `\baselineskip`. We use 1 as a default (i.e. no stretch).

```
⟨*2ekernel⟩
\def\baselinestretch{1}
```

(*End of definition for* `\baselinestretch`.)

`\every@math@size` We must still define the hook `\every@math@size` we used in `\glb@settings`. We initialize it to nothing. It is important to remember that everything that goes into this hook should to global updates, local changes will have weird effects.

```
\newtoks\every@math@size
\every@math@size={}
⟨/2ekernel⟩
```

(*End of definition for* `\every@math@size`.)

5.2 Math fonts setup

5.2.1 Outline of algorithm for math font sizes

TEX uses the math fonts that are current when the end of a formula is reached. If we don't want to keep font setups local to every formula (which would result in an enormous overhead), we have to be careful not to end up with the wrong setup in case formulas are nested, e.g., we need to be able to handle

```
$ a=b+c \mbox{ \small for all $b$ and $c\in Z$}$
```

Here the inner formulae `b` and `c\in Z` are typeset in `\small` but we have to return to `\normalsize` before we reach the closing `$` of the outer formula.

This is handled in the following way:

1. At any point in the document the global variable `\gbl@currsize` contains the point size for which the math fonts currently are set up.

2. Whenever we start a formula we compare its value with the local variable `\f@size` that describes the current text font size.

3. If both are the same we assume that we can use the current math font setup without adjustment.

4. If they differ we call `\gbl@settings` which changes the math font setup and updates `\gbl@currsize`.

   (a) If we are recursively inside another formula (`\if@inmath`) we ensure that `\gbl@settings` is executed again in the outer formula, so that the old setup is automatically restored.

   (b) Otherwise, we set the switch `@inmath` locally to `true` so that all nested formulae will be able to detect that they are nested in some outer formula.

The above algorithm has the following features:

- For sizes which are not containing any formula no math setup is done. Compared to the original algorithm of NFSS this results in the following savings:
  - No unnecessary loading of math fonts for sizes that are not used to typeset any math formulae (explicit or implicit ones).
  - No time overhead due to unnecessary changes of the math font setup on entrance and exit of the text font size.
- Math font setup changes for top-level formulae will survive (there is no restoration after the formula) thus any following formula in the same size will be directly typesetable. Compared to original implementation in NFSS2 the new algorithm has the overhead of one test per formula to see if the current math setup is valid (in the original algorithm the setup was always valid, thus no test was necessary).
- In nested formulae the math font setup is restored in the outer formula by a series of `\aftergroup` commands and checks. Compared to the original algorithm this involves additional checks ($2 \times \langle$non-math levels$\rangle$ per inner formula).

5.2.2 Code for math font size setting

`\check@mathfonts` In the `\check@mathfonts` macros we implement the steps 2 to 4 except that instead of a switch the macro `\init@restore@glb@settings` is used.

```
⟨*2ekernel | package⟩
\def\check@mathfonts{%
  \ifx \glb@currsize \f@size
⟨*trace⟩
    \ifnum \tracingfonts>\thr@@
        \@font@info{*** MATH: no change \f@size\space
         curr/global (\curr@math@size/\glb@currsize)}\fi
⟨/trace⟩
  \else
⟨*trace⟩
    \ifnum \tracingfonts>\thr@@
        \@font@info{*** MATH: setting up \f@size\space
         curr/global (\curr@math@size/\glb@currsize)}\fi
⟨/trace⟩
```

```
    \glb@settings
    \init@restore@glb@settings
  \fi
  \let\curr@math@size\f@size
  \def\init@restore@glb@settings{\aftergroup\restglb@settings}%
}
```

(*End of definition for* `\check@mathfonts`.)

`\init@restore@glb@settings` This macros does by default nothing but get redefined inside `\check@mathfonts` to initiate fontsize restoring in nested formulas.

```
⟨-trace⟩\let\init@restore@glb@settings\relax
⟨*trace⟩
\def\init@restore@glb@settings{%
       \ifnum \tracingfonts>\thr@@
        \@font@info{*** MATH: no resetting (not in
                    nested math)}\fi
}
⟨/trace⟩
```

(*End of definition for* `\init@restore@glb@settings`.)

`\restglb@settings` This macro will be executed the first time after the current formula.

```
\def\restglb@settings{%
⟨*trace⟩
    \ifnum \tracingfonts>\thr@@
        \@font@info{*** MATH: restoring}\fi
⟨/trace⟩
      \begingroup
        \let\f@size\curr@math@size
        \ifx\glb@currsize \f@size
⟨*trace⟩
    \ifnum \tracingfonts>\thr@@
        \@font@info{*** MATH: ... already okay (\f@size)}\fi
⟨/trace⟩
        \else
⟨*trace⟩
    \ifnum \tracingfonts>\thr@@
        \@font@info{*** MATH: ... to \f@size}\fi
⟨/trace⟩
          \glb@settings
        \fi
      \endgroup
}
```

(*End of definition for* `\restglb@settings`.)

5.2.3 Other code for math

`\use@mathgroup` The `\use@mathgroup` macro should be used in user macros to select a math group. Depending on whether or not the `margid` option is in force it has two or three arguments. For this reason it should be called as the last macro.

First we test if we are inside math mode since we don't want to apply a useless definition.

```
\def\use@mathgroup#1#2{\relax\ifmmode
```

```
⟨*trace⟩
  \ifnum \tracingfonts>\tw@
    \count@#2\relax
    \@font@info{Using \noexpand\mathgroup
              (\the\count@) #2}\fi
⟨/trace⟩
```

If so we first call the '=' macro (i.e. argument three) to set up special things for the selected math group. Then we call `\mathgroup` to select the group given by argument two and finally we place `#1` (i.e., the argument of the ⟨*math alphabet identifier*⟩) at the end. This part of the code is surrounded by two commands which behave like `\begingroup` and `\endgroup` if we want ⟨*math alphabet identifier*⟩s but will expand into `\@empty` if we want simply switches to a new math group. Since argument number 2 may be a digit instead of a control sequence we add a `\relax`. Otherwise something like `\mit{1}` would switch to math group 11 (and back) instead of printing an oldstyle 1.

```
    \math@bgroup
        \expandafter\ifx\csname M@\f@encoding\endcsname#1\else
        #1\fi
        \mathgroup#2\relax
```

Before we reinsert the swallowed token (arg. three) into the input stream, in the case that the ⟨*math alphabet identifier*⟩ isn't called in math mode, we remove the `\fi` with the `\expandafter` trick. This is necessary if the token is actually an macro with arguments. In such a case the `\fi` will be misinterpreted as the first argument which would be disastrous.

```
    \expandafter\math@egroup\fi}%
```

The surrounding macros equal `\begingroup` and `\endgroup`. But using internal names makes it possible to overwrite their meaning in certain cases. This is for example used in $\mathcal{AMS}$-TEX macros for placing accents.

(*End of definition for* `\use@mathgroup`.)

`\math@egroup` If the `margid` option is in force (which can be tested by looking at the definition of `\math@bgroup`) we change the `\math@egroup` command a bit to display the current ⟨*math group number*⟩ after it closes the scope of ⟨*math alphabet*⟩ with `\endgroup`.

```
⟨*trace⟩
  \ifx\math@bgroup\bgroup
    \def\math@egroup#1{#1\egroup
      \ifnum \tracingfonts>\tw@
     \@font@info{Restoring \noexpand\mathgroup
        (\ifnum\mathgroup=\m@ne default\else \the\mathgroup \fi)%
        }\fi}
  \fi
⟨/trace⟩
```

(*End of definition for* `\math@egroup`.)

`\getanddefine@fonts` `\getanddefine@fonts` has two arguments: the ⟨*math group number*⟩ and the *family/series/shape* name as a control sequence.

```
\def\getanddefine@fonts#1#2{%
```

First we turn of tracing when `\tracingfonts` is less than 4.

```
⟨+debug⟩  \pushtracing
⟨+debug⟩  \ifnum\tracingfonts<4 \tracingoff
⟨+debug⟩  \else \tracingon\getanddefine@fonts \fi

⟨*trace⟩
  \ifnum \tracingfonts>\tw@
  \count@#1\relax
    \@font@info{\noexpand\mathgroup (\the\count@) #1 :=\MessageBreak
                \string#2 \tf@size/\sf@size/\ssf@size}\fi
⟨/trace⟩
```

We append the current `\tf@size` to `#2` to obtain the font name.[36] Again, `font@name` is defined globally, for the reasons explained in the description of `\wrong@fontshape`.

```
  \xdef\font@name{\csname \string#2/\tf@size\endcsname}%
```

Then we call `\pickup@font` to load it if necessary. We remember the internal name as `\textfont@name`.

```
  \pickup@font \let\textfont@name\font@name
```

Same game for `\scriptfont` and `\scriptscriptfont`, but here we additionally transform the fontname through `\transform@scriptfont` to allow different family names in math scripts.

```
  \xdef\font@name{\csname \transform@scriptfont {sf}{#2}/\sf@size\endcsname}%
  \pickup@font \let\scriptfont@name\font@name
  \xdef\font@name{\csname \transform@scriptfont {ssf}{#2}/\ssf@size\endcsname}%
  \pickup@font
```

Then we append the new `\textfont...` assignments to the `\math@fonts`.

```
  \edef\math@fonts{\math@fonts
                \textfont#1\textfont@name
                \scriptfont#1\scriptfont@name
                \scriptscriptfont#1\font@name}%
```

Just before ending this macro we have to pop the tracing stack if it was pushed before.

```
⟨+debug⟩ \poptracing
    }
```

(*End of definition for* `\getanddefine@fonts`.)

`\transform@scriptfont`

`\transform@scriptfont` has two arguments: the name of the script size we transform the font for (always `sf` or `ssf`) and the *family/series/shape* name as a csname. It expands to a new *family/series/shape* name which will be used for this script. For this `\transform@scriptfont` mostly delegates to `\__nfss_transform_scriptfont:nw`, which searches for a replacement marker `\__nfss_mapped_scriptfont_family_⟨scriptsize⟩_⟨encodi` with a remapped encoding and family.

```
\ExplSyntaxOn
\cs_set:Npn \transform@scriptfont #1 #2 {
  \exp_last_unbraced:Nno
    \__nfss_transform_scriptfont:nw
    {#1}
    {\token_to_str:N #2}
}
```

[36] One might ask why this expansion does not generate a macro name that starts with an additional `\` character. The solution is that `\escapechar` is set to −1 before `\getanddefine@fonts` is called.

```
\cs_set:Npn \__nfss_transform_scriptfont:nw #1 #2 / #3 / {
  \cs_if_exist_use:cF { __nfss_mapped_scriptfont_family_ #1 _ #2 / #3 } { #2 / #3 }
  /
}
```

(*End of definition for* `\transform@scriptfont`.)

`\DeclareMathScriptfontMapping` `\DeclareMathScriptfontMapping` can be used to define replacement font families which will be used for math subscripts or subsubscripts when a specific font family is used as the main math font. For this `\DeclareMathScriptfontMapping` takes three pairs of encoding and family names as arguments: The first pair identifies the base font for which associated script and scriptscript variants should be declared. The second pair describes the family which should be used as scriptfonts (e.g. in subscripts). Finally the third pair describes the family which should be used as scriptscriptfonts (e.g. in nested subscripts).

```
\cs_set:Npn \DeclareMathScriptfontMapping #1 #2 #3 #4 #5 #6 {
  \cs_gset:cpn {  __nfss_mapped_scriptfont_family_sf_ #1 / #2 } { #3 / #4 }
  \cs_gset:cpn {  __nfss_mapped_scriptfont_family_ssf_ #1 / #2 } { #5 / #6 }
}
\ExplSyntaxOff
⟨/2ekernel | package⟩
```

(*End of definition for* `\DeclareMathScriptfontMapping`.)

6 Scaled font extraction

`\ifnot@nil` We begin with a simple auxiliary macro. It checks whether its argument is the token `\@nil`. If so, it expands to `\@gobble` which discards the following argument, otherwise it expands to `\@firstofone` which reproduces it argument.

```
⟨*2ekernel⟩
\def\ifnot@nil#1{\def\reserved@a{#1}%
  \ifx\reserved@a\@nnil \expandafter\@gobble
  \else \expandafter\@firstofone\fi}
```

(*End of definition for* `\ifnot@nil`.)

`\remove@to@nnil` `\remove@angles` `\remove@star` Three other auxiliary macros will be needed in the following: `\remove@to@nnil` gobbles up everything up to, and including, the next `\@nnil` token, and `\remove@angles` and `\remove@star` do the same for the character `>` and `*`, respectively, instead of `\@nnil`.

```
\def\remove@to@nnil#1\@nnil{}
\def\remove@angles#1>{\set@simple@size@args}
\def\remove@star#1*{#1}
```

(*End of definition for* `\remove@to@nnil`, `\remove@angles`, *and* `\remove@star`.)

`\extract@sizefn` This macro takes a size specification and parses it into size function and the optional and mandatory arguments.

```
\def\extract@sizefn#1*#2\@nil{%
  \if>#2>\set@size@funct@args#1\@nil
         \let\sizefn@info\@empty
  \else\expandafter\set@size@funct@args\remove@star#2\@nil
       \def\sizefn@info{#1}\fi
  }
```

(*End of definition for* `\extract@sizefn`.)

`\try@simple@size` This function tries to extract the given size (specified by `\f@size`) for the requested font shape. The font information must already be present in `\font@info`. The central macro that does the real work is `\extract@fontinfo`. We will first give a simple example how this macro works, and describe it in full generality later.

Assume that the requested parameters are: *encoding scheme* 'OT1', *family* 'cm', *series* 'sansserif', *shape* 'normal', and *size* '12'. The corresponding font definitions have already been extracted from the macro `\OT1/cm/sansserif/normal` and stored in `font@info`. (Otherwise `\extract@fontinfo` doesn't get called.) This information consists of a token list made of characters of category code 12 of the form

```
<10*>cmss10<12*>cmss12<17*>cmss17
```

For reasonable packages one usually needs more sizes but this is sufficient to get the flavour. We will define a macro `\extract@fontinfo` to find the external font name ('cmss12') for us:

```
\def\extract@fontinfo#1<12*#2>#3<#4\@nnil{%
    \set@simple@size@args#3<#4\@nnil
    \execute@size@function{#2}}
```

so that when it gets called via

```
\extract@fontinfo<10*>cmss10<12*>cmss12<17*>cmss17\@nnil
```

`#1` will contain all characters before `<12*>`, `#2` will be empty, `#3` will be exactly `cmss12`, and `#3` will be `17>cmss17`. The expansion is therefore

```
\set@simple@size@args cmss12<17*>cmss17\@nnil
\execute@size@function{}
```

This means: the default (empty) size function will be executed, with its optional argument set to empty and its mandatory argument set to `cmss12` by `\set@simple@size@args`. As we discussed earlier, the effect of the default size function is to load the given external font (cmss12) at the specified size (12)—which is exactly what was intended.

But this is only part of the whole story. It may be that the size requested does not occur in the token list `\font@info`. And the simple definition of `\extract@fontinfo` we gave above does not allow to specify give more than one size specification in front of the external font name.

Let's address these two problems separately. The first one is solved with the following trick: We define `\extract@fontinfo` as follows:

```
\def\extract@fontinfo#1<12*#2>#3<#4\@nnil{%
  \ifnot@nil{#3}%
    {\set@simple@size@args#3<#4\@nnil
     \execute@size@function{#2}%
    }}%
```

How does this work? We call `\extract@fontinfo` via

```
\expandafter\extract@fontinfo\font@info<12*>\@nil<\@nnil
```

i.e. by appending `<12*>\@nil<\@nnil`. If the size ('12' in this case) appears in `\font@info` everything works as explained above, the only difference being that argument `#4` of `\extract@fontinfo` additionally gets the tokens `<12*>\@nil<\@nnil`. However, if the size is not found everything up to the final `<12*>` is in argument `#1`, `#3` gets `\@nil`, and `#2` and `#4` are empty. The macro `\ifnot@nil` will discard the calls to `\set@simple@size@args` and `execute@size@function`, and hence `\font@info` will continue to be equal to `\@empty`. This means that no simple size specification matching the requested size could be found.

The second problem (more than one simple size specification for one external font name) will be addressed in `\set@simple@size@args` below.

The macros are hidden inside other control sequences so that we have to build `\extract@fontinfo` in several steps.

So here's the actual definition of `\extract@font` in `\try@simple@size`.

```
% % this could be replaced by \try@size@range making the subst slower!
\def\try@simple@size{%
```

`\reserved@a` is made an abbreviation for the head of the definition of the macro `\extract@fontinfo`.

```
    \def\reserved@a{\def\extract@fontinfo####1}%
```

Now we can define `\extract@fontinfo`. Here we handle a small but convenient variation: in case of the default (empty) size function it is allowed to omit the `*` character.

```
    \expandafter\reserved@a\expandafter<\f@size>##2<##3\@nnil{%
        \ifnot@nil{##2}%
          {\set@simple@size@args##2<##3\@nnil
           \execute@size@function\sizefn@info
          }}%
```

Now we call `\extract@fontinfo`. Note the `<\@nil` tokens at the end.

```
    \expandafter\expandafter
    \expandafter\extract@fontinfo\expandafter\font@info
    \expandafter<\f@size>\@nil<\@nnil
}
```

(*End of definition for* `\try@simple@size`.)

`\set@simple@size@args` As promised above, the macro `\set@simple@size@args` will handle the case of several size specifications in a row. If another size specification follows, the very first token of its argument list is the character `<`. By starting the definition as follows,

```
\def\set@simple@size@args#1<{%
```

parameter `#1` is empty in this case, and contains the size function's arguments otherwise. We distinguish these two cases (Note that the character `<` cannot appear in `#1`) by calling `\remove@angles` for empty `#1` and `\extract@sizefn` otherwise. In the latter case we have to take care of the remaining character tokens and discard them. This is done by `\remove@to@nnil`. Note also the use of Kabelschacht's method.

```
        \if<#1<%
          \expandafter\remove@angles
        \else
          \extract@sizefn#1*\@nil
          \expandafter\remove@to@nnil
        \fi}
```

(*End of definition for* `\set@simple@size@args`.)

Now, we are through with the case of a simple size, except for calling the size function. This will be handled later, as it is the same mechanism for all types of size specification. We will now proceed to macros for extraction of size range specification.

`\extract@rangefontinfo` `\extract@rangefontinfo` goes through a font shape definition in the input until it recognizes the tokens `<\@nil->`. It looks for font ranges with font size functions. It's operation is rather simple: it discards everything up to the next size specification and passes this on to `\is@range` for inspection. The specification (parameter `#2`) is inserted again, in case it is needed later.

```
\def\extract@rangefontinfo#1<#2>{%
      \is@range#2->\@nil#2>}
```

(*End of definition for* `\extract@rangefontinfo`.)

`\is@range` `\is@range` is again a sort of dispatcher macro: if the size specification it is looking at is not a range specification it discards it and calls `\extract@rangefontinfo` to continue the search. Otherwise it calls `\check@range` to check the requested size against the specified range.

From the way `\is@range` is called inside `\extract@rangefontinfo` we see that `#2` is the character `>` if the size specification found is a simple one (that does not contain a `-` character). This is checked easily enough and `\extract@rangefontinfo` called again. Note that the extra tokens inserted after the `\@nil` in the call to `\is@range` appear at the beginning of the first argument to `\extract@rangefontinfo` and are hence ignored.

```
\def\is@range#1-#2\@nil{%
  \if>#2\expandafter\check@single\else
     \expandafter\check@range\fi}
```

(*End of definition for* `\is@range`.)

`\check@range` `\check@range` takes lower bound as parameter `#1`, upper bound as `#2`, size function as `#3` and the size function's arguments as `#4`. If `#3` is the special token `\@nil` `\font@info` is exhausted and we can stop searching.

```
\def\check@range#1-#2>#3<#4\@nnil{%
 \ifnot@nil{#3}{%
```

If `#3` wasn't `\@nil` we have a range. We start by assuming that we have to recurse. Note that we have to reinsert an `<` as it was already removed by scanning.

```
    \def\reserved@f{\extract@rangefontinfo<#4\@nnil}%
```

We have to make sure that both boundaries are present, if not we have to set them. Here we check the upper bound. If `\upper@bound` is zero after the assignment we set it to `\maxdimen` (upper open range). We need to use a ⟨*dimen*⟩ register for the scan since we may have a decimal number as the boundary.

```
    \upper@bound0#2\p@
    \ifdim\upper@bound=\z@ \upper@bound\maxdimen\fi
```

Now we check the upper boundary against `\f@size`. If it is larger or equal than `\f@size` this range is no good and we have to recurse.

```
    \ifdim \f@size \p@<\upper@bound
```

Otherwise we have to check the lower bound. This time it is not necessary to scan the boundary value into a register because if it is empty we get zero as desired. We could even omit the `0` which would result in `1pt` as default lower boundary. If `\f@size` is smaller than the boundary we have to recurse.

```
\lower@bound0#1\p@
\ifdim \f@size \p@<\lower@bound
 \else
```

If both tests are passed we can try executing the size function.

```
  \set@simple@size@args#3<#4\@nnil
  \execute@size@function\sizefn@info
```

If the function was successful it should have left an external font name in `\external@font`. We use this to see if we can stop scanning. Otherwise we recurse.

```
  \ifx\external@font\@empty
  \else
    \let\reserved@f\@empty
  \fi
 \fi
\fi
\reserved@f}}
```

(*End of definition for* `\check@range`.)

`\lower@bound` `\upper@bound` We use two dimen registers `\lower@bound` and `\upper@bound` to store the lower and upper endpoints of the range we found.

```
\newdimen\lower@bound
\newdimen\upper@bound
```

(*End of definition for* `\lower@bound` *and* `\upper@bound`.)

`\check@single` `\check@single` takes the size as parameter `#1`, size function as `#2` and the size function's arguments as `#3`. We can assume that there is always something in the pipeline since the very last entry is a faked range (see above).

```
\def\check@single#1>#2<#3\@nnil{%
```

We start by assuming that we have to recurse. Note that we have to reinsert an `<` as it was already removed by scanning.

```
\def\reserved@f{\extract@rangefontinfo<#3\@nnil}%
```

Now we check the size against `\f@size`. If it is not equal `\f@size` it is no good and we have to recurse.

```
\ifdim \f@size \p@=#1\p@
```

Otherwise if this test is passed we can try executing the size function.

```
  \set@simple@size@args#2<#3\@nnil
  \execute@size@function\sizefn@info
```

If the function was successful it should have left an external font name in `\external@font`. We use this to see if we can stop scanning. Otherwise we recurse.

```
  \ifx\external@font\@empty
  \else
    \let\reserved@f\@empty
  \fi
\fi
\reserved@f}
```

(*End of definition for* `\check@single`.)

`\set@size@funct@args`
`\set@size@funct@args@`

This macro sets the optional and mandatory arguments for a size function. If the optional argument is not present it is set to the empty token list. The mandatory argument is delimited by the token `\@nil`.

```
\def\set@size@funct@args{\@ifnextchar[%
  \set@size@funct@args@{\set@size@funct@args@[]}}
\def\set@size@funct@args@[#1]#2\@nil{%
  \def\mandatory@arg{#2}%
  \def\optional@arg{#1}}
⟨/2ekernel⟩
```

(*End of definition for* `\set@size@funct@args` *and* `\set@size@funct@args@`.)

`\DeclareSizeFunction`

This function defines a new size function hiding the internal from the designer. The body of the size function may use `\optional@arg` and `\mandatory@arg` denoting the optional and mandatory argument that may follow the size specification `<...>`.

```
⟨*2ekernel⟩
\def\DeclareSizeFunction#1#2{\@namedef{s@fct@#1}{#2}}
\@onlypreamble\DeclareSizeFunction
⟨/2ekernel⟩
```

(*End of definition for* `\DeclareSizeFunction`.)

`\execute@size@function`

This macro is very simple. The only point worth noting is that calling an undefined size function will do nothing (actually execute a `\relax`).

```
⟨*2ekernel | package⟩
\def\execute@size@function#1{%
⟨*trace⟩
          \@ifundefined{s@fct@#1}%
            {\errmessage{Undefined font size function #1}%
             \s@fct@}%
            {\csname s@fct@#1\endcsname}%
⟨/trace⟩
⟨-trace⟩       \csname s@fct@#1\endcsname
}
⟨/2ekernel | package⟩
```

(*End of definition for* `\execute@size@function`.)

`\try@size@range`

This macro tries to find a suitable range for requested size (specified by `\f@size`) in `\font@info`. All the relevant action is done in `\extract@rangefontinfo`. All that needs to be done is to stuff in the token list in `\font@info` so that `\extract@rangefontinfo` can inspect it. Note the `<-*\@nil><` token at the end to stop scanning.

```
⟨*2ekernel⟩
\def\try@size@range{%
    \expandafter\extract@rangefontinfo\font@info <-*>\@nil<\@nnil
}
```

(*End of definition for* `\try@size@range`.)

`\try@size@substitution`

This is the last thing that can be tried. If the desired `\f@size` is found neither among the simple size specifications nor in one of the ranges the whole list of size specifications is searched for a nearby simple size.

```
\gdef\try@size@substitution{%
```

First we do some initializations. `\@tempdimb` will hold the difference between the wanted size and the best solution found so far, so we initialize it with `\maxdimen`. The macro `\best@size` will hold the best size found, nothing found is indicated by the empty value.

```
  \@tempdimb \maxdimen
  \let \best@size \@empty
```

Now we loop over the specification

```
  \expandafter \try@simples \font@info <\number\@M>\@nil<\@nnil
}
```

(*End of definition for* `\try@size@substitution`.)

`\font@submax` `\fontsubfuzz` The macro `\font@submax` records the maximal deviation from the desired size encountered so far. Its value is used in a warning message at `\end{document}`. The macro `\fontsubfuzz` contains the amount that will not cause terminal warnings (warnings still go into the transcript file).

```
\def\font@submax{0pt}
\def\fontsubfuzz{.4pt}
⟨/2ekernel⟩
⟨+package⟩\def\fontsubfuzz{0pt}
```

(*End of definition for* `\font@submax` *and* `\fontsubfuzz`.)

`\try@simples` `\try@simples` goes through a font shape definition in the input until it recognizes the tokens `<*\@nil><`. It looks for simple sizes to determine the two closest sizes. It is assumed that simple sizes are in increasing order.

```
⟨*2ekernel⟩
\gdef\try@simples#1<#2>{%
  \tryif@simple#2->\tryif@simple}
```

(*End of definition for* `\try@simples`.)

`\tryis@simple` `\tryis@simple` is similar to `\is@range`. If it sees a simple size, it checks it against the value of `\f@size` and sets `\lower@font@size` or `\higher@font@size`. In the latter case, it stops the iteration. By adding `<\number\@M>` at the end of the line we always have an end point. This is a hack which probably should be corrected.

First it checks whether it is finished already, then whether the size specification in question is a simple one.

```
\gdef\tryif@simple#1-#2\tryif@simple{%
```

Most common case for `\reserved@f` first:

```
  \let \reserved@f \try@simples
  \if>#2%
```

If so, it compares it to the value of `\f@size`. This is done using a dimen register since there may be fractional numbers.

```
    \dimen@ #1\p@
    \ifdim \dimen@<\@M\p@
```

If `\dimen@` is `\@M\p@` we have reached the end of the fontspec (hopefully) otherwise we compare the value with `\f@size` and compute in `\@tempdimc` the absolute value of the difference between the two values.

```
      \ifdim \f@size\p@<\dimen@
        \@tempdimc \dimen@
        \advance\@tempdimc -\f@size\p@
      \else
        \@tempdimc \f@size\p@
        \advance\@tempdimc -\dimen@
      \fi
```

The result is then compared with the smallest difference we have encountered, if the new value (in `\@tempdimc` is smaller) we have found a size which is a better approximation so we make it the `\best@size` and adjust `\@tempdimb`.

```
      \ifdim \@tempdimc<\@tempdimb
        \@tempdimb \@tempdimc
        \def \best@size{#1}%
      \fi
```

When we have reached the end of the fontspec we substitute the best size found (if any). We code this inline to save macro space; in the past this was done by a macro called `\subst@size`.

```
    \else
```

`\subst@size`

This macro substitutes the size recorded in `\best@size` for the unavailable size `\f@size`. `\font@submax` records the maximum difference between desired size and selected size in the whole run.

```
% %\subst@size             %% coded inline
% %\def\subst@size{%
  \ifx \external@font\@empty
    \ifx \best@size\@empty
    \else
      \ifdim \@tempdimb>\font@submax \relax
        \xdef \font@submax {\the\@tempdimb}%
      \fi
      \let \f@user@size \f@size
      \let \f@size \best@size
      \ifdim \@tempdimb>\fontsubfuzz\relax
        \@font@warning{Font\space shape\space
            `\curr@fontshape'\space in\space size\space
             <\f@user@size>\space not\space available\MessageBreak
             size\space <\f@size>\space substituted}%
      \fi
      \try@simple@size
      \do@subst@correction
    \fi
 \fi
% %}
```

This brings us back into the main part of `\tryif@simple`. Finally we get rid of any rubbish left over on the input stack.

```
      \let \reserved@f \remove@to@nnil
    \fi
  \fi
```

If it's a range iterate also.

```
  \reserved@f}
```

(*End of definition for* `\tryis@simple` *and* `\subst@size`.)

6.1 Sizefunctions

In the following we define some useful size functions.

`\s@fct@` This is the default size function. Mandatory argument is an external font name, optional argument a scale factor. The font is scaled to `\f@size` if no optional argument is present, and to `\f@size` multiplied by the optional argument otherwise.

```
\DeclareSizeFunction{}{\empty@sfcnt\@font@warning}
\DeclareSizeFunction{s}{\empty@sfcnt\@font@info}

\def\empty@sfcnt#1{%
      \@tempdimb \f@size\p@
      \ifx\optional@arg\@empty
      \else
        \@tempdimb \optional@arg\@tempdimb
        #1{Font\space shape\space `\curr@fontshape'\space
           will\space be\MessageBreak
           scaled\space to\space size\space \the\@tempdimb}%
      \fi
      \edef\external@font{\mandatory@arg\space at\the\@tempdimb}}
```

(*End of definition for* `\s@fct@`.)

`\s@fct@gen` `\s@fct@sgen` This size function generates the external name from the mandatory argument and the requested user size, and thus can be used for external names where the size is encoded in the font name. The optional argument a scale factor. The font is scaled to `\f@size` if no optional argument is present, and to `\f@size` multiplied by the optional argument otherwise.

```
\DeclareSizeFunction{gen}{\gen@sfcnt\@font@warning}
\DeclareSizeFunction{sgen}{\gen@sfcnt\@font@info}

\def\gen@sfcnt{%
      \edef\mandatory@arg{\mandatory@arg\f@size}%
      \empty@sfcnt}
```

(*End of definition for* `\s@fct@gen` *and* `\s@fct@sgen`.)

`\s@fct@genb` `\s@fct@sgenb` This size function is similar to `gen`, but for fonts where the size is encoded in the font name in centipoints, as in the DC fonts version 1.2. The font is scaled to `\f@size` if no optional argument is present, and to `\f@size` multiplied by the optional argument otherwise.

```
\DeclareSizeFunction{genb}{\genb@sfcnt\@font@warning}
\DeclareSizeFunction{sgenb}{\genb@sfcnt\@font@info}

\def\genb@sfcnt{%
    \edef\mandatory@arg{\mandatory@arg\expandafter\genb@x\f@size..\@@}%
    \empty@sfcnt}
```

(*End of definition for* `\s@fct@genb` *and* `\s@fct@sgenb`.)

\genb@x
\genb@y

The auxiliary macros \genb@x and \genb@y are used to convert the \f@size into centipoints.

```
\def\genb@x#1.#2.#3\@@{\two@digits{#1}\genb@y#200\@@}
\def\genb@y#1#2#3\@@{#1#2}
```

(*End of definition for* \genb@x *and* \genb@y.)

\s@fct@sub

This size function handles font substitution. The mandatory argument is a family/series/shape combination, the optional argument (if present) is ignored. The font encoding scheme cannot be changed. Therefore, the first thing we do is to prepend the encoding scheme.

```
\DeclareSizeFunction{sub}{\sub@sfcnt\@font@warning}
\DeclareSizeFunction{ssub}{\sub@sfcnt\@font@info}

\def\sub@sfcnt#1{%
    \edef\mandatory@arg{\f@encoding/\mandatory@arg}%
```

Next action is split the arg into its individual components and allow for a late font shape load.

```
    \begingroup
     \expandafter\split@name\mandatory@arg/\@nil
     \try@load@fontshape
    \endgroup
```

Then we record the current \f@size since it may get clobbered.

```
    \let\f@user@size\f@size
```

Then we check whether this new combination is defined and give an error message if not. In this case we also switch to \error@fontshape.

```
    \expandafter
    \ifx\csname\mandatory@arg\endcsname\relax
      \errmessage{No\space declaration\space for\space
                  shape\space \mandatory@arg}%
      \error@fontshape
    \else
```

Otherwise we warn the user about the substitution taking place.

```
      #1{Font\space shape\space `\curr@fontshape'\space in\space
         size\space <\f@size>\space not\space available\MessageBreak
         Font\space shape\space `\mandatory@arg'\space tried\space
         instead}%
      \expandafter\split@name\mandatory@arg/\@nil
    \fi
```

Then we restart the font specification scan by calling \get@external@font.

```
    \edef\f@size{\f@user@size}%
    \get@external@font
```

Finally \do@subst@correction is called to get the font name right.

```
    \do@subst@correction
}
```

(*End of definition for* \s@fct@sub.)

\@font@aliasinfo

Sometimes a substitution is only done to map a long font name to a standard shape or series, e.g.,

```
DeclareFontShape{T1}{Roboto-LF}{b}{it}{<-> alias * Roboto-LF/bold/it}{}
```

Using the `ssub` function in that case will give a strange (and incorrect) warning. As an alternative we therefore offer the size function `alias`. It will still add some info into the `.log` file, but no longer complains that the font shape is not available. It is implemented by grabbing the default warning text and replacing it with a new one.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩                    {\@font@aliasinfo}{alias size function}%
\DeclareSizeFunction{alias}{\sub@sfcnt\@font@aliasinfo}
\def\@font@aliasinfo#1{%
  \@font@info{Font\space shape\space `\curr@fontshape'\space
              aliased\space to\MessageBreak `\mandatory@arg'}%
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\@font@aliasinfo}{alias size function}%
⟨latexrelease⟩\let\s@fct@alias\@undefined
⟨latexrelease⟩\let\@font@aliasinfo\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@font@aliasinfo`.)

`\s@fct@subf` The `subf` size function allows substitution of another font. The mandatory argument is the external name of the font to be substituted, the optional argument a size scaling factor like in the default size function. The main difference to the default size function is the warning message.

```
\DeclareSizeFunction{subf}{\subf@sfcnt\@font@warning}
\DeclareSizeFunction{ssubf}{\subf@sfcnt\@font@info}

\def\subf@sfcnt#1{%
      #1{Font\space shape\space `\curr@fontshape'\space in\space
         size\space \f@size\space not\space available\MessageBreak
        external\space font\space `\mandatory@arg'\space used}%
      \empty@sfcnt#1%
      }
```

(*End of definition for* `\s@fct@subf`.)

`\s@fct@fixed` The `fixed` size function is for using a font at a different size than requested. A warning message is printed, and the external font to be used is taken from the mandatory argument. If an optional argument is present it is used as the 'at' size for the font. Otherwise the font is loaded at its design size.

```
\DeclareSizeFunction{fixed}{\fixed@sfcnt\@font@warning}
\DeclareSizeFunction{sfixed}{\fixed@sfcnt\@font@info}

\def\fixed@sfcnt#1{%
  \ifx\optional@arg\@empty
    \let\external@font\mandatory@arg
  \else
    \edef\external@font{\mandatory@arg\space at\optional@arg pt}%
```

```
  \fi
  #1{External\space font\space `\external@font'\space loaded\space
     for\space size\MessageBreak
     <\f@size>}%
}
⟨/2ekernel⟩
```

(*End of definition for* `\s@fct@fixed`.)

File 27
ltfsscmp.dtx

This file contains the implementation of commands giving compatibility with the original 'NFSS1' release of the Font Selection Scheme.

> **Warning:** The macro documentation is still basically the documentation from the first NFSS release and therefore in some cases probably not completely accurate.

Version 1 of NFSS is obsolete now for about 20 years (and was "current" only for a short intermediate time) so with the 2015 release these internal interface commands are removed from the kernel and made available via latexrelease package so that backward compatibility remains ensured for very old documents.

```
⟨*latexrelease⟩
\IncludeInRelease{2015/01/01}{\new@fontshape}%
                              {NFSS version1 commands}%
\let\new@fontshape\@undefined
\let\warn@rel@i\@undefined
\let\scan@fontshape\@undefined
\let\scan@@fontshape\@undefined
\let\subst@fontshape\@undefined
\let\extra@def\@undefined
\let\default@mextra\@undefined
\let\preload@sizes\@undefined
\let\err@rel@i\@undefined
\let\newmathalphabet\@undefined
\let\newmathalphabet@\@undefined
\let\newmathalphabet@@@\@undefined
\let\if@no@font@opt\@undefined
\let\@no@font@optfalse\@undefined
\let\define@mathalphabet\@undefined
\let\define@mathgroup\@undefined
\let\addtoversion\@undefined
\EndIncludeInRelease
```

In older releases we provide the original definitions.

```
\IncludeInRelease{0000/00/00}{\new@fontshape}%
                              {NFSS version1 commands}%
```

`\new@fontshape` The interface is now `\DeclareFontShape`.

```
\gdef\new@fontshape#1#2#3#4{%
     \warn@rel@i\new@fontshape\DeclareFontShape
     \expandafter\scan@fontshape\@gobble#4<\@nil><<%
     \DeclareFontShape U{#1}{#2}{#3}\reserved@f}%
\@onlypreamble\new@fontshape
```

(*End of definition for* `\new@fontshape`.)

`\warn@rel@i` The warning message used above.

```
\gdef\warn@rel@i#1#2{%
 \@font@warning{***  NFSS release 1 command
```

```
              \noexpand#1found\MessageBreak
   ***  Update by using release 2 command
        \string#2.\MessageBreak
   ***  Recovery is probably possible}%
}%
\@onlypreamble\warn@rel@i
```

(*End of definition for* `\warn@rel@i`.)

`\scan@fontshape` This will scan the old font shape definition syntax.

```
\gdef\scan@fontshape{%
  \let\reserved@f\@empty
  \let\reserved@e\@empty %        holds last info
  \scan@@fontshape
}%
\@onlypreamble\scan@fontshape
```

(*End of definition for* `\scan@fontshape`.)

`\scan@@fontshape`

```
\gdef\scan@@fontshape#1>#2#3<{%
  \ifx\@nil#1%
    \edef\reserved@f{\reserved@f\reserved@e}%
  \else
    \def\reserved@b{#1}%       nick names
    \def\reserved@c{#3}%
    \in@{ at}{#3}%
    \ifin@
      \in@{pt}{#3}%  not a proof but a good chance
      \ifin@
```

We grab also everything after pt and discard it if people have forgotten to place a percent sign there.

```
        \def\reserved@a##1 at##2pt##3\@nil{%
           \def\reserved@b{##2}%
           \def\reserved@c{##1}%
           }%
        \reserved@a#3\@nil
      \fi
    \fi
    \ifnum 0<0#2
      \edef\reserved@d{subf*\reserved@c}%
      \ifcase #2\or
      \or
      \else
        \errmessage{*** What's this? NFSS release 0? ***}%
      \fi
    \else
      \edef\reserved@d{#2\reserved@c}%
    \fi
    \ifx\reserved@d\reserved@e
      \edef\reserved@f{\reserved@f<\reserved@b>}%
    \else
      \edef\reserved@f{\reserved@f\reserved@e<\reserved@b>}%add old info
      \let\reserved@e\reserved@d
```

```
    \fi
    \expandafter\scan@@fontshape
  \fi
}%
\@onlypreamble\scan@@fontshape
```

(*End of definition for* `\scan@@fontshape`.)

`\subst@fontshape` This is now also handled by the extend syntax of `\DeclareFontShape`.

```
\gdef\subst@fontshape#1#2#3#4#5#6{%
     \warn@rel@i\subst@fontshape\DeclareFontShape
     \DeclareFontShape{U}{#1}{#2}{#3}{<->sub*#4/#5/#6}{}}%
\@onlypreamble\subst@fontshape
```

(*End of definition for* `\subst@fontshape`.)

`\extra@def` This was replaced by `\DeclareFontFamily`.

```
\gdef\extra@def#1#2#3{%
     \warn@rel@i\extra@def\DeclareFontFamily
     \DeclareFontFamily{U}{#1}{}%
}%
\@onlypreamble\extra@def
```

(*End of definition for* `\extra@def`.)

`\default@mextra` The new name is `\DeclareFontEncodingDefaults` but in this case we don't feel comfortable with this either.

```
\gdef\default@mextra{%
  \warn@rel@i\default@mextra\DeclareFontEncodingDefaults
```

We pick up the argument to `\default@mextra` implicitly as the second argument of `\DeclareFontEncodingDefaults`.

```
  \DeclareFontEncodingDefaults\relax
}%
\@onlypreamble\default@mextra
```

(*End of definition for* `\default@mextra`.)

`\preload@sizes` The new interface is `\DeclarePreloadSizes`.

```
\gdef\preload@sizes{%
     \warn@rel@i\preload@sizes\DeclarePreloadSizes
     \DeclarePreloadSizes U%
}%
\@onlypreamble\preload@sizes
```

(*End of definition for* `\preload@sizes`.)

`\err@rel@i` This macro is used in cases where emulation with NFSS2 features is not really possible.

```
\gdef\err@rel@i#1#2{%
  \@latex@error{***  NFSS release 1 command \noexpand#1found%
          ^^J***  Recovery not possible. Use \string#2}%
       {The new release of NFSS doesn't support the
        \noexpand#1command^^Jany longer.
        Please upgrade your file to the syntax of NFSS
        release 2^^Jusing the \noexpand#2command.}%
```

Let's die.

```
  \batchmode\input.\relax
}%
\@onlypreamble\err@rel@i
```

(*End of definition for* `\err@rel@i`.)

`\newmathalphabet` `\newmathalphabet@@` `\newmathalphabet@@@`

`\newmathalphabet` is the old form.

```
\gdef\newmathalphabet{%
  \if@no@font@opt
    \@latex@error{*** NFSS release 1 command
                    \noexpand\newmathalphabet found%
     ^^J \space*** Automatic recovery not possible.%
     ^^J \space*** TYPE H for Help%
              }%
       {Please look at the file usrguide.tex for hints on
        how to resolve this problem.}%
  \else
     \warn@rel@i\newmathalphabet\DeclareMathAlphabet
  \fi
  \@ifstar\newmathalphabet@@@
          \newmathalphabet@@}%
\gdef\newmathalphabet@@#1{\DeclareMathAlphabet#1{U}{}{}{}}%
\gdef\newmathalphabet@@@#1#2#3#4{%
       \DeclareMathAlphabet{#1}{U}{#2}{#3}{#4}}%
\@onlypreamble\newmathalphabet
\@onlypreamble\newmathalphabet@@
\@onlypreamble\newmathalphabet@@@
```

(*End of definition for* `\newmathalphabet`, `\newmathalphabet@@`, *and* `\newmathalphabet@@@`.)

`\if@no@font@opt` `\@no@font@optfalse`

```
\global\let\if@no@font@opt\iftrue
\gdef\@no@font@optfalse{\let\if@no@font@opt\iffalse}%
```

(*End of definition for* `\if@no@font@opt` *and* `\@no@font@optfalse`.)

`\define@mathalphabet`

This is a case where dying is best.

```
\gdef\define@mathalphabet{%
      \err@rel@i\define@mathalphabet\DeclareMathAlphabet
}%
\@onlypreamble\define@mathalphabet
```

(*End of definition for* `\define@mathalphabet`.)

`\define@mathgroup`

And here is another one

```
\gdef\define@mathgroup{%
      \err@rel@i\define@mathgroup\DeclareSymbolFont
}%
\@onlypreamble\define@mathgroup
```

(*End of definition for* `\define@mathgroup`.)

\addtoversion `\addtoversion` is the old form.

```
\def\addtoversion#1#2{%
  \warn@rel@i\addtoversion\SetMathAlphabet
  \SetMathAlphabet#2{#1}{U}}%
\@onlypreamble\addtoversion
```

(*End of definition for* `\addtoversion`.)

Finishing off this huge `\IncludeInRelease` argument:

```
\EndIncludeInRelease
⟨/latexrelease⟩
```

File 28
ltfssdcl.dtx

This file contains the main implementation of the font selection scheme commands. See other parts of the LaTeX distribution, or *The LaTeX Companion* for higher level documentation of these commands.

> **Warning:** The macro documentation is still basically the documentation from the first NFSS release and therefore in some cases probably not completely accurate.

1 Interface Commands

```
⟨*2ekernel⟩
\message{NFSS declarative interface,}
```

\in@ \ifin@ `\@in` is a utility macro with two arguments. It determines whether its first argument occurs in its second and sets the switch `\ifin@` accordingly. The first argument may not contain braces nor `#` (more precisely, tokens of category code 1, 2, or 6).

```
\def\in@#1#2%
 {%
   \begingroup
     \def\in@@##1#1{}%
     \toks@\expandafter{\in@@#2{}{}#1}%
     \edef\in@@{\the\toks@}%
   \expandafter\endgroup
   \ifx\in@@\@empty
     \in@false
   \else
     \in@true
   \fi
 }
\newif\ifin@
```

(*End of definition for* `\in@` *and* `\ifin@`.)

Before the `\begin{document}` command several ⟨*math versions*⟩ and ⟨*math alphabet identifiers*⟩ may be declared. In principle, there should be exactly one family/series/shape combination be declared for each version/alphabet pair. But we want to allow for defaults as well for automagical filling of holes.

While building the tables for math alphabet identifiers and math versions we keep several lists:

- the list of all math versions, `\version@list`, each entry prefixed by the control sequence `\version@elt`, i.e. this list has the following form

  $$\texttt{\textbackslash version@elt}\langle \textit{version}_1\rangle\texttt{\textbackslash version@elt}\langle \textit{version}_2\rangle\ldots \texttt{\textbackslash version@elt}\langle \textit{version}_n\rangle$$

- the list of all math alphabet identifiers. Here every entry has the form:
  `\group@elt`⟨*math group number*⟩
  `{{`⟨*default family*⟩`}{`⟨*default series*⟩`}{`⟨*default shape*⟩`}}`.

- Each defined math alphabet identifier holds a list containing Information about the *versions* for which it is defined. This list has a more complicated structure: it looks as follows:

```
\set@alpha⟨the alphabet identifier itself⟩
   \reserved@c⟨math version⟩⟨font info⟩
  ...
\@nil
```

  where ⟨*font info*⟩ is either `\reserved@e` (if the combination is not defined yet) or

```
{{⟨family⟩}{⟨series⟩}{⟨shape⟩}}
```

`\version@list` We initialize the version list to be empty.

```
\let\version@list=\@empty
\@onlypreamble\version@list
```

(*End of definition for* `\version@list`.)

`\version@elt`

```
\let\version@elt\relax
\@onlypreamble\version@elt
```

(*End of definition for* `\version@elt`.)

`\new@mathversion` The macro `\new@mathversion` is called with the version control sequence as its argument.

```
%\def\new@mathversion#1{%
```

The first thing this macro does is to check if the version identifier is already present in `\version@list`. We enclose `\version@list` in braces since it might be empty (if no *version* is defined yet). But this means that we need a suitable number of `\expandafter` primitives.

```
%  \expandafter\in@\expandafter#1\expandafter{\version@list}%
%  \ifin@
```

If so it prints an error message. The `\next` macro is used to get rid of the four characters `\mv@` that would otherwise appear at the begin of the version name in the error message.

```
%    \@latex@error{Math version
%                ‘\expandafter\@gobblefour\string#1’
%                already defined}\@eha
```

Otherwise we have a new version, and we can proceed with entering it into the tables. We add it to `\version@list`. This is very easy: we define `\version@elt` (which is the delimiter in `\version@list`) to protect itself and the following token from being expanded and simply redefine `\version@list`.

```
%  \else
%     \global\expandafter\newcount\csname c@\expandafter
%                                  \@gobble\string#1\endcsname
%     \global\csname c@\expandafter
%                                  \@gobble\string#1\endcsname\@ne
%     \def\version@elt{\noexpand\version@elt\noexpand}%
%     \edef\version@list{\version@list\version@elt#1}%
```

Then we prepare to enter the new version into all math alphabet identifier lists. Remember that these lists use `\reserved@c` as delimiter, and that there appears the control sequence `\reserved@e` that must not be expanded. Therefore we take suitable precautions.

```
%     \def\reserved@c{\noexpand\reserved@c\noexpand}%
%     \let\reserved@e\relax
```

We will now go through the `\alpha@list` to process every ⟨*math alphabet identifier*⟩ in turn. Since this list has `\group@elt` as a delimiter we define this control sequence. It has three arguments as every entry consists of three items (as explained above).

```
%     \def\group@elt##1##2##3{%
```

The first of these arguments is the ⟨*math alphabet identifier*⟩. We redefine it by appending the information about the new version at the end of the list contained in it. However, there is one subtlety: the definitions for `\reserved@c` and `\reserved@e` made above prevent the main part of the list from being expanded. But we still have to take care of the header and the trailer. To do this we remove the trailer by means of the macro `\remove@nil` which also protect the header from being expanded. Its definition is given below. Now we can prepare to add the new version.

```
%         \edef##1{\expandafter\remove@nil##1%
%                  \reserved@c
%                  #1%
%                  \reserved@e
%                  \noexpand\@nil}}%
```

Finally we call `\alpha@list` which will now execute the macro `\group@elt` once for every defined ⟨*math alphabet identifier*⟩. And that's all for now.

```
%     \alpha@list
%  \fi}
```

(*End of definition for* `\new@mathversion`.)

`\alpha@list` As we explained above every entry in `\alpha@list` has the form

`\alpha@elt`
⟨*alphabet identifier*⟩⟨*internal group number*⟩⟨*default font assignments*⟩...

We initialize it to `\@empty`.

```
\let\alpha@list\@empty
\@onlypreamble\alpha@list
```

(*End of definition for* `\alpha@list`.)

`\alpha@elt`

```
\let\alpha@elt\relax
\@onlypreamble\alpha@elt
```

(*End of definition for* `\alpha@elt`.)

`\newgroup` Start the group (fam) allocation at 0. (Doesn't belong here.)

```
\count18=-1
```

(*End of definition for* `\newgroup`.)

`\stepcounter`

(*End of definition for* `\stepcounter`.)

`\select@group` We surround `\select@group` with braces so that functions using it can be used directly after `_` or `^`. However, if we use oldstyle syntax where the math alphabet doesn't have arguments (ie if `\math@bgroup` is not `\bgroup`) we need to get rid of the extra group.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}
⟨latexrelease⟩                  {\select@group}{\select@group}%
⟨*2ekernel | latexrelease⟩
\def\select@group#1#2#3#4{%
 \ifx\math@bgroup\bgroup\else\relax\expandafter\@firstofone\fi
 {%
 \ifmmode
  \ifnum\csname c@mv@\math@version\endcsname<\e@mathgroup@top
     \begingroup
       \escapechar\m@ne
       \getanddefine@fonts{\csname c@mv@\math@version\endcsname}#3%
       \globaldefs\@ne  \math@fonts
     \endgroup
     \init@restore@version
     \xdef#1{\noexpand\use@mathgroup\noexpand#2%
             {\number\csname c@mv@\math@version\endcsname}}%
     \global\advance\csname c@mv@\math@version\endcsname\@ne
   \else
     \let#1\relax
     \@latex@error{Too many math alphabets used in
                   version \math@version}%
        \@eha
   \fi
 \else \expandafter\non@alpherr\fi
 #1{#4}%
 }%
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}
⟨latexrelease⟩                  {\select@group}{\select@group}%
⟨latexrelease⟩\def\select@group#1#2#3#4{%
⟨latexrelease⟩ \ifx\math@bgroup\bgroup\else\relax\expandafter\@firstofone\fi
⟨latexrelease⟩ {%
⟨latexrelease⟩ \ifmmode
⟨latexrelease⟩  \ifnum\csname c@mv@\math@version\endcsname<\sixt@@n
⟨latexrelease⟩     \begingroup
⟨latexrelease⟩       \escapechar\m@ne
⟨latexrelease⟩       \getanddefine@fonts
⟨latexrelease⟩         {\csname c@mv@\math@version\endcsname}#3%
⟨latexrelease⟩       \globaldefs\@ne  \math@fonts
⟨latexrelease⟩     \endgroup
⟨latexrelease⟩     \init@restore@version
⟨latexrelease⟩     \xdef#1{\noexpand\use@mathgroup\noexpand#2%
⟨latexrelease⟩             {\number\csname c@mv@\math@version\endcsname}}%
⟨latexrelease⟩     \global\advance\csname c@mv@\math@version\endcsname\@ne
⟨latexrelease⟩   \else
⟨latexrelease⟩     \let#1\relax
⟨latexrelease⟩     \@latex@error{Too many math alphabets used in
⟨latexrelease⟩                   version \math@version}%
```

```
⟨latexrelease⟩          \@eha
⟨latexrelease⟩    \fi
⟨latexrelease⟩ \else \expandafter\non@alpherr\fi
⟨latexrelease⟩ #1{#4}%
⟨latexrelease⟩ }%
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
\@onlypreamble\restore@mathversion
```

(*End of definition for* `\select@group`.)

`\init@restore@version`

```
\def\init@restore@version{%
        \global\let\init@restore@version\relax
        \xdef\restore@mathversion
             {\expandafter\noexpand\csname mv@\math@version\endcsname
              \global\csname c@mv@\math@version\endcsname
              \number\csname c@mv@\math@version\endcsname\relax}%
        \aftergroup\dorestore@version
}
\@onlypreamble\init@restore@version
```

(*End of definition for* `\init@restore@version`.)

`\non@alpherr`

```
\gdef\non@alpherr#1{\@latex@error{%
```

The command here will have a space at the end of its name, so we make sure not to insert an extra one.

```
    \string#1allowed only in math mode}\@ehd}
```

(*End of definition for* `\non@alpherr`.)

`\dorestore@version`

```
\def\dorestore@version
 {\ifmmode
    \aftergroup\dorestore@version
  \else
    \gdef\init@restore@version{%
        \global\let\init@restore@version\relax
        \xdef\restore@mathversion
             {\expandafter\noexpand\csname mv@\math@version\endcsname
              \global\csname c@mv@\math@version\endcsname
              \number\csname c@mv@\math@version\endcsname\relax}%
        \aftergroup\dorestore@version
    }%
    \begingroup
      \let\getanddefine@fonts\@gobbletwo
      \restore@mathversion
    \endgroup
  \fi}%
\@onlypreamble\dorestore@version
```

(*End of definition for* `\dorestore@version`.)

\c@localmathalphabets To avoid hitting the "no more math fams available" limit of 16, we keep a defined number of math alphabets flexible/local. If we have to allocate any of those we roll back the allocation after the formula has ended, so the next formula can use other alphabets in the slot(s). This makes the processing a bit slower if you are working at the limit, but that is better than dying with "out of memory".

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2021/11/15}
⟨latexrelease⟩   {\document@select@group}{\document@select@group}%
⟨*2ekernel | latexrelease⟩
```

We don't really undo the declaration on rollback (as that would be hard to maintain), so rolling forward needs to check if the declaration was already made.

```
\ifx\c@localmathalphabets\@undefined
```

There is no need to have this counter as part of the include checkpoints, given that it makes little sense to alter its settings mid document. All we want is the ability to change it using the `\setcounter` interface.

By default we keep two math fams flexible.

```
 \newcount\c@localmathalphabets
 \setcounter{localmathalphabets}{2}
\fi
```

(*End of definition for* `\c@localmathalphabets`.)

\document@select@group The `\document@select@group` command is the version of `\select@group` (inside math versions) that is used in the document body to set up math alphabets (if used).

```
\def\document@select@group#1#2#3#4{%
 \ifx\math@bgroup\bgroup\else\relax\expandafter\@firstofone\fi
 {%
 \ifmmode
```

We first check if there is still room for allocating another mathgroup. If there is, we check if it can be globally allocated or if we have reached the limit which is given by `\e@mathgroup@top` with `\c@localmathalphabets` subtracted.

```
   \ifnum\csname c@mv@\math@version\endcsname<\e@mathgroup@top
     \ifnum \numexpr\e@mathgroup@top-\c@localmathalphabets
              >\csname c@mv@\math@version\endcsname
     \else
```

If we are past this point we freeze the current state of the math version so that we can return to it after the formula has ended. Of course, that should be done only once, so we check if `\mv@`⟨*version*⟩`@frozen` already exists.

```
       \ifcsname mv@\math@version @frozen\endcsname \else
```

We have to pass the current value of `\math@version` not the macro itself, because some of the processing is delayed to a point where the value may have changed again—not doing this caused a puzzling error in one setup.

```
         \expandafter\freeze@math@version\expandafter{\math@version}%
       \fi
     \fi
     \begingroup
       \escapechar\m@ne
       \getanddefine@fonts{\csname c@mv@\math@version\endcsname}#3%
       \globaldefs\@ne  \math@fonts
```

```
    \endgroup
    \expandafter\extract@alph@from@version
        \csname mv@\math@version\expandafter\endcsname
        \expandafter{\number\csname
                       c@mv@\math@version\endcsname}%
         #1%
    \global\advance\csname c@mv@\math@version\endcsname\@ne
  \else
    \let#1\relax
    \@latex@error{Too many math alphabets used in
                  version \math@version}%
       \@eha
 \fi
```

Extra `\expandafter` to remove the `\expandafter` added below

```
\else \expandafter\expandafter\expandafter\non@alpherr\fi
```

We surround `\select@group` with braces so that functions using it can be used directly after `_` or `^`.

If the legacy interface is used, e.g., `$\sf -1$` the math alphabet `#1` does not take an argument so we better do not surround `#4` with braces, because then we get `{\relax}` into the formula and introduce an extra Ord atom. The two different cases can be distinguished by looking at the current value of `\math@bgroup`.

```
 \expandafter#1\ifx\math@bgroup\bgroup{#4}\else#4\fi
 }%
}
```

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}
⟨latexrelease⟩  {\document@select@group}{\document@select@group}%
⟨latexrelease⟩
⟨latexrelease⟩\def\document@select@group#1#2#3#4{%
⟨latexrelease⟩ \ifx\math@bgroup\bgroup\else\relax\expandafter\@firstofone\fi
⟨latexrelease⟩ {%
⟨latexrelease⟩ \ifmmode
⟨latexrelease⟩   \ifnum\csname c@mv@\math@version\endcsname<\e@mathgroup@top
⟨latexrelease⟩     \begingroup
⟨latexrelease⟩       \escapechar\m@ne
⟨latexrelease⟩       \getanddefine@fonts{\csname c@mv@\math@version\endcsname}#3%
⟨latexrelease⟩       \globaldefs\@ne  \math@fonts
⟨latexrelease⟩     \endgroup
⟨latexrelease⟩     \expandafter\extract@alph@from@version
⟨latexrelease⟩         \csname mv@\math@version\expandafter\endcsname
⟨latexrelease⟩         \expandafter{\number\csname
⟨latexrelease⟩                        c@mv@\math@version\endcsname}%
⟨latexrelease⟩          #1%
⟨latexrelease⟩     \global\advance\csname c@mv@\math@version\endcsname\@ne
⟨latexrelease⟩   \else
⟨latexrelease⟩     \let#1\relax
⟨latexrelease⟩     \@latex@error{Too many math alphabets used
⟨latexrelease⟩                   in version \math@version}%
⟨latexrelease⟩        \@eha
⟨latexrelease⟩  \fi
⟨latexrelease⟩ \else \expandafter\expandafter\expandafter\non@alpherr\fi
```

```
⟨latexrelease⟩ \expandafter#1\ifx\math@bgroup\bgroup{#4}\else#4\fi
⟨latexrelease⟩ }%
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2015/01/01}
⟨latexrelease⟩  {\document@select@group}{\document@select@group}%
⟨latexrelease⟩
⟨latexrelease⟩\def\document@select@group#1#2#3#4{%
⟨latexrelease⟩ \ifx\math@bgroup\bgroup\else\relax\expandafter\@firstofone\fi
⟨latexrelease⟩ {%
⟨latexrelease⟩ \ifmmode
⟨latexrelease⟩   \ifnum\csname c@mv@\math@version\endcsname<\e@mathgroup@top
⟨latexrelease⟩     \begingroup
⟨latexrelease⟩       \escapechar\m@ne
⟨latexrelease⟩       \getanddefine@fonts{\csname c@mv@\math@version\endcsname}#3%
⟨latexrelease⟩       \globaldefs\@ne  \math@fonts
⟨latexrelease⟩     \endgroup
⟨latexrelease⟩     \expandafter\extract@alph@from@version
⟨latexrelease⟩         \csname mv@\math@version\expandafter\endcsname
⟨latexrelease⟩         \expandafter{\number\csname
⟨latexrelease⟩                       c@mv@\math@version\endcsname}%
⟨latexrelease⟩          #1%
⟨latexrelease⟩     \global\advance\csname c@mv@\math@version\endcsname\@ne
⟨latexrelease⟩   \else
⟨latexrelease⟩     \let#1\relax
⟨latexrelease⟩     \@latex@error{Too many math alphabets used
⟨latexrelease⟩                   in version \math@version}%
⟨latexrelease⟩        \@eha
⟨latexrelease⟩  \fi
⟨latexrelease⟩ \else \expandafter\non@alpherr\fi
⟨latexrelease⟩ #1{#4}%
⟨latexrelease⟩ }%
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{0000/00/00}
⟨latexrelease⟩  {\document@select@group}{\document@select@group}%
⟨latexrelease⟩
⟨latexrelease⟩\def\document@select@group#1#2#3#4{%
⟨latexrelease⟩ \ifx\math@bgroup\bgroup\else\relax\expandafter\@firstofone\fi
⟨latexrelease⟩ {%
⟨latexrelease⟩ \ifmmode
⟨latexrelease⟩   \ifnum\csname c@mv@\math@version\endcsname<\sixt@@n
⟨latexrelease⟩     \begingroup
⟨latexrelease⟩       \escapechar\m@ne
⟨latexrelease⟩       \getanddefine@fonts
⟨latexrelease⟩         {\csname c@mv@\math@version\endcsname}#3%
⟨latexrelease⟩       \globaldefs\@ne  \math@fonts
⟨latexrelease⟩     \endgroup
⟨latexrelease⟩     \expandafter\extract@alph@from@version
⟨latexrelease⟩         \csname mv@\math@version\expandafter\endcsname
⟨latexrelease⟩         \expandafter{\number\csname
⟨latexrelease⟩                       c@mv@\math@version\endcsname}%
⟨latexrelease⟩          #1%
```

```
⟨latexrelease⟩      \global\advance\csname c@mv@\math@version\endcsname\@ne
⟨latexrelease⟩    \else
⟨latexrelease⟩      \let#1\relax
⟨latexrelease⟩      \@latex@error{Too many math alphabets used
⟨latexrelease⟩                    in version \math@version}%
⟨latexrelease⟩         \@eha
⟨latexrelease⟩   \fi
⟨latexrelease⟩ \else \expandafter\non@alpherr\fi
⟨latexrelease⟩ #1{#4}%
⟨latexrelease⟩ }%
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\document@select@group`.)

`\freeze@math@version` This command stores the current state of the math version and sets things up to return to it after each formula from now on. We use L3 programming layer code to set it up.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2022/11/01}%
⟨latexrelease⟩                  {\freeze@math@version}{freeze math version}%
\ExplSyntaxOn
\cs_new_protected:Npn\freeze@math@version #1 {
```

Save the current `\mv@⟨version⟩` code and the number of allocated mathgroups inside.

```
  \@font@info{Freeze~ math~ alphabet~ allocation~ in~ version~
              #1.\MessageBreak
              Allocated~math~groups:~\int_use:c{ c@mv@ #1 }~
              (local:~  \int_use:N\c@localmathalphabets)      }
  \cs_gset_eq:cc { mv@#1@frozen }{ mv@#1 }
  \tl_gset:ce { g__nfss_frozen_mv_ #1 _tl }{ \int_use:c { c@mv@#1 } }
```

Here is the definition of `\mv@⟨version⟩@reset`. If there has been no new math alphabet allocation, doing a reset would just cause a lot of unnecessary processing, so we do a quick check upfront for this.

```
  \cs_gset:cpn{mv@#1@reset}
     {
```

If we are back at top-level, or more precisely outside of any (nested) math, we may have to reset the math version back to its frozen version, if that exists and has fewer alphabets allocated.

```
       \int_compare:nNnTF \@math@level = 0
       {
         \int_compare:nNnTF { \int_use:c{c@mv@#1} } >
                            { \tl_use:c{g__nfss_frozen_mv_ #1 _tl} }
             {
               \@font@info{ Undo~ math~ alphabet~ allocation~ in~ version~ #1 }
```

If the undo is necessary, we restore the `\mv@⟨version⟩` code.

```
               \cs_gset_eq:cc { mv@#1 }{ mv@#1@frozen }
               \int_gset:cn { c@mv@#1 }{ \tl_use:c {g__nfss_frozen_mv_ #1 _tl} }
```

But we also should undo changes to the top-level math alphabet definitions. We therefore run this code with a modified definition for `\getanddefine@fonts` because there is no need to do anything to the symbol fonts that are permanently allocated. However, we do this only if the resetting was for the current math version, because otherwise we would give the top-level definitions the values for the last math version being reset (e.g., bold if there are just two).

```
                \tl_if_eq:NnTF \math@version {#1}
                {
                  \group_begin:
                    \cs_set_eq:NN \getanddefine@fonts \use_none:nn
                    \use:c {mv@ \math@version }
                  \group_end:
                }
                {
```

Once we have hit the boundary and have to revert to the frozen version that happens quite often, but typically only for the normal version. Thus, it is better to report only when we do not revert the top-level definition as this is the exceptional case and happens only if several math versions are used in parallel.

```
                  \@font@info{ ...~ but~ do~ not~ reset~ the~
                       top-level~ math~ alphabet~ definitions }
                }
              }
              {
```

If there was no change, we report that in the log (but this branch could go completely).

```
                \@font@info{ No~ math~ alphabet~ change~
                             to~ frozen~ version~ #1 }
              }
        }
        {
          \@font@info{ Nested~ math:~ keeping~ math~ alphabet~
                       allocation~ in~ version~ #1}
        }
```

If this is executed after a math display, we may have to arrange for ignoring spaces, because they are now hidden if the tokens from above intervene. This is signaled by the 2e switch `@ignore` which is set in `\frozen@everymath` and `\frozen@everydisplay`.

This is all 2e code so we use that syntax.

```
        \if@ignore \ignorespaces \fi
    }
}
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/11/15}
⟨latexrelease⟩                  {\freeze@math@version}{freeze math version}%
⟨latexrelease⟩
⟨latexrelease⟩\ExplSyntaxOn
⟨latexrelease⟩\cs_set_protected:Npn\freeze@math@version #1 {
⟨latexrelease⟩  \@font@info{Freeze~ math~ alphabet~ allocation~ in~ version~
⟨latexrelease⟩               #1.\MessageBreak
⟨latexrelease⟩               Allocated~math~groups:~\int_use:c{ c@mv@ #1 }~
⟨latexrelease⟩               (local:~  \int_use:N\c@localmathalphabets)      }
```

```
⟨latexrelease⟩  \cs_gset_eq:cc { mv@#1@frozen }{ mv@#1 }
⟨latexrelease⟩  \tl_gset:cx { g__nfss_frozen_mv_ #1 _tl }{ \int_use:c { c@mv@#1 } }
⟨latexrelease⟩  \group_insert_after:N \__nfss_init_mv_freeze:N
⟨latexrelease⟩  \exp_after:wN \group_insert_after:N \cs:w mv@#1@reset \cs_end:
⟨latexrelease⟩  \tl_gput_right:No \check@mathfonts
⟨latexrelease⟩       {
⟨latexrelease⟩         \exp_after:wN \group_insert_after:N \cs:w mv@#1@reset \cs_end:
⟨latexrelease⟩       }
⟨latexrelease⟩  \cs_gset:cpn{mv@#1@reset}
⟨latexrelease⟩     {
⟨latexrelease⟩         \int_compare:nNnTF { \int_use:c{c@mv@#1} } >
⟨latexrelease⟩                            { \tl_use:c{g__nfss_frozen_mv_ #1 _tl} }
⟨latexrelease⟩             {
⟨latexrelease⟩               \@font@info{Undo~ math~ alphabet~ allocation~ in~ version~ #1}
⟨latexrelease⟩               \cs_gset_eq:cc { mv@#1 }{ mv@#1@frozen }
⟨latexrelease⟩               \int_gset:cn { c@mv@#1 }{ \tl_use:c {g__nfss_frozen_mv_ #1 _tl} }
⟨latexrelease⟩               \group_begin:
⟨latexrelease⟩                 \cs_set_eq:NN \getanddefine@fonts \use_none:nn
⟨latexrelease⟩                 \use:c {mv@#1}
⟨latexrelease⟩               \group_end:
⟨latexrelease⟩             }
⟨latexrelease⟩             {
⟨latexrelease⟩               \@font@info{No~ math~ alphabet~ change~ to~ frozen~ version~ #1}
⟨latexrelease⟩             }
⟨latexrelease⟩         \if@ignore \ignorespaces \fi
⟨latexrelease⟩     }
⟨latexrelease⟩}
⟨latexrelease⟩\cs_set_protected:Npn \__nfss_init_mv_freeze:N #1 {%
⟨latexrelease⟩  \mode_if_math:T { \group_insert_after:N \__nfss_init_mv_freeze:N
⟨latexrelease⟩                    \group_insert_after:N } #1
⟨latexrelease⟩}
⟨latexrelease⟩\ExplSyntaxOff
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\freeze@math@version`.)

`\process@table`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/11/01}%
⟨latexrelease⟩                  {\process@table}{Support metafamily}%
\def\process@table{%
    \def\cdp@elt##1##2##3##4{%
        \@font@info{Checking defaults for
                  ##1/##2/##3/##4}%
        \expandafter
        \ifx\csname##1/##2/##3/##4\endcsname\relax
```

Grouping is important for two reasons, first `\cdp@elt` will get redefined if `\Declare...` functions are executed within the external `.fd` file and secondly `\try@load@fontshape` changes a lot of catcodes without surrounding itself with a group.

```
          \begingroup
           \def\f@encoding{##1}\def\f@family{##2}%
```

```
            \try@load@fontshape
          \endgroup
        \fi
        \expandafter
        \ifx\csname##1/##2/##3/##4\endcsname\relax
             \@latex@error{This NFSS system isn't set up properly}%
                       {For encoding scheme ##1 the defaults
                        ##2/##3/##4 do not form a valid font shape}%
        \else
             \@font@info{... okay}%
        \fi}%
    \cdp@list
```

Now we make sure that `\error@fontshape` is okay.

```
    \begingroup
       \escapechar\m@ne
       \error@fontshape
       \expandafter\ifx\csname \curr@fontshape\endcsname\relax
          \begingroup
            \try@load@fontshape
           \endgroup
       \fi
       \expandafter\ifx\csname \curr@fontshape\endcsname\relax
         \@latex@error{This NFSS system isn't set up properly}%
            {The system maintainer forgot to specify a suitable
             substitution
             font shape using the \noexpand\DeclareErrorFont
             command}%
       \fi
    \endgroup
```

Set `\select@group` to its meaning used within the document body.

```
    \let\select@group\document@select@group
```

Install the default font attributes as they are currently pointing to error font face. We can speed up the process by just using `\edef`, thereby avoiding all kind of extra processing. Don't use `\reset@font` for this since that would trigger `\selectfont`, but run the `normalfont` hook, since this is really what we are doing here (except for the `\selectfont`=.

```
    \fontencoding\encodingdefault
    \edef\f@family{\familydefault}%
    \edef\f@series{\seriesdefault}%
    \edef\f@shape{\shapedefault}%

    \set@current@meta@family
    \UseHook{normalfont}%
```

Drop stuff not longer needed.

```
 \everyjob{}%
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\process@table}{Support metafamily}%
⟨latexrelease⟩
```

```
⟨latexrelease⟩\def\process@table{%
⟨latexrelease⟩    \def\cdp@elt##1##2##3##4{%
⟨latexrelease⟩        \@font@info{Checking defaults for
⟨latexrelease⟩                  ##1/##2/##3/##4}%
⟨latexrelease⟩        \expandafter
⟨latexrelease⟩        \ifx\csname##1/##2/##3/##4\endcsname\relax
⟨latexrelease⟩          \begingroup
⟨latexrelease⟩           \def\f@encoding{##1}\def\f@family{##2}%
⟨latexrelease⟩           \try@load@fontshape
⟨latexrelease⟩          \endgroup
⟨latexrelease⟩        \fi
⟨latexrelease⟩        \expandafter
⟨latexrelease⟩        \ifx\csname##1/##2/##3/##4\endcsname\relax
⟨latexrelease⟩             \@latex@error{This NFSS system isn't set up properly}%
⟨latexrelease⟩                       {For encoding scheme ##1 the defaults
⟨latexrelease⟩                        ##2/##3/##4 do not form a valid font shape}%
⟨latexrelease⟩        \else
⟨latexrelease⟩             \@font@info{... okay}%
⟨latexrelease⟩        \fi}%
⟨latexrelease⟩    \cdp@list
⟨latexrelease⟩    \begingroup
⟨latexrelease⟩       \escapechar\m@ne
⟨latexrelease⟩       \error@fontshape
⟨latexrelease⟩       \expandafter\ifx\csname \curr@fontshape\endcsname\relax
⟨latexrelease⟩          \begingroup
⟨latexrelease⟩            \try@load@fontshape
⟨latexrelease⟩           \endgroup
⟨latexrelease⟩       \fi
⟨latexrelease⟩       \expandafter\ifx\csname \curr@fontshape\endcsname\relax
⟨latexrelease⟩         \@latex@error{This NFSS system isn't set up properly}%
⟨latexrelease⟩            {The system maintainer forgot to specify a suitable
⟨latexrelease⟩             substitution
⟨latexrelease⟩             font shape using the \noexpand\DeclareErrorFont
⟨latexrelease⟩             command}%
⟨latexrelease⟩       \fi
⟨latexrelease⟩    \endgroup
⟨latexrelease⟩    \let\select@group\document@select@group
⟨latexrelease⟩    \fontencoding\encodingdefault
⟨latexrelease⟩    \edef\f@family{\familydefault}%
⟨latexrelease⟩    \edef\f@series{\seriesdefault}%
⟨latexrelease⟩    \edef\f@shape{\shapedefault}%
⟨latexrelease⟩ \everyjob{}%
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
\@onlypreamble\process@table
```

(*End of definition for* `\process@table`.)

```
%\@onlypreamble\set@mathradical
```

`\DeclareMathVersion`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
```

```
⟨latexrelease⟩\IncludeInRelease{2022/11/01}%
⟨latexrelease⟩                  {\DeclareMathVersion}{local alphabets}%
\def\DeclareMathVersion#1{%
```

When declaring a new math version we need to instantiate an L3 variable that is used when we freeze the version, because too many alphabets got allocated. If we don't do this, L3 programming layer complains if it is run in checking mode.

```
  \@namedef{g__nfss_frozen_mv_#1_tl}{}%
```

We also extend `\check@mathfonts` to call a version reset (once frozen) after a formula has finished.

```
  \expandafter\ifx\csname mv@#1\endcsname \relax
    \expandafter \g@addto@macro \expandafter \check@mathfonts
         \expandafter {\expandafter \aftergroup \csname mv@#1@reset\endcsname}%
```

Initially this macro does nothing. It is, however, important that it doesn't stop any `\ignorespaces`, so we make it expandable and not `\relax`.

```
    \@namedef{mv@#1@reset}{}%
  \fi
  \expandafter\new@mathversion\csname mv@#1\endcsname}
\@onlypreamble\DeclareMathVersion
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/11/15}%
⟨latexrelease⟩                  {\DeclareMathVersion}{local alphabets}%
⟨latexrelease⟩\def\DeclareMathVersion#1{%
⟨latexrelease⟩  \@namedef{g__nfss_frozen_mv_#1_tl}{}%
⟨latexrelease⟩  \expandafter\new@mathversion\csname mv@#1\endcsname}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\DeclareMathVersion}{local alphabets}%
⟨latexrelease⟩\def\DeclareMathVersion#1{%
⟨latexrelease⟩  \expandafter\new@mathversion\csname mv@#1\endcsname}
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\DeclareMathVersion`.)

`\new@mathversion`

```
\def\new@mathversion#1{%
  \expandafter\in@\expandafter#1\expandafter{\version@list}%
  \ifin@
    \@font@info{Redeclaring math version
                ‘\expandafter\@gobblefour\string#1’}%
  \else
    \expandafter\newcount\csname c@\expandafter
                                \@gobble\string#1\endcsname
    \def\version@elt{\noexpand\version@elt\noexpand}%
    \edef\version@list{\version@list\version@elt#1}%
  \fi
```

`\toks@` is used to gather all tokens for the math version. `\count@` will be used to count the math groups we add to this version.

```
  \toks@{}%
  \count@\z@
```

Now we loop over `\group@list` to add all math groups defined so far to the version and at the same time to count them.

```
  \def\group@elt##1##2{%
       \advance\count@\@ne
       \addto@hook\toks@{\getanddefine@fonts##1##2}%
       }%
  \group@list
```

We set the counter for this math version to the number of math groups found in `\group@list`.

```
  \global\csname c@\expandafter\@gobble\string#1\endcsname\count@
```

Now we loop over `\alpha@list` to add all math alphabets known so far. We have to distinguish the case that an alphabet by default should produce an error in new versions.

```
  \def\alpha@elt##1##2##3{%
       \ifx##2\no@alphabet@error
         \toks@\expandafter{\the\toks@\install@mathalphabet##1%
             {\no@alphabet@error##1}}%
       \else
         \toks@\expandafter{\the\toks@\install@mathalphabet##1%
             {\select@group##1##2##3}}%
       \fi
          }%
  \alpha@list
```

Finally we define the math version to expand to the contents of `\toks@`.

```
  \xdef#1{\the\toks@}%
}
\@onlypreamble\new@mathversion
```

(*End of definition for* `\new@mathversion`.)

`\DeclareSymbolFont` First drop any surplus `m` from the series argument then do what has been done since 1994.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2022/11/01}%
⟨latexrelease⟩                    {\DeclareSymbolFont}{maybe drop m}%
\def\DeclareSymbolFont #1#2#3#4#5{%
 \def\reserved@a{\DeclareSymbolFont@m@dropped{#1}{#2}{#3}}%
 \edef\reserved@b{#4}%
 \series@maybe@drop@one@m\reserved@b\reserved@b
 \expandafter\reserved@a\expandafter{\reserved@b}{#5}%
}
\def\DeclareSymbolFont@m@dropped #1#2#3#4#5{%
 \@tempswafalse
 \edef\reserved@b{#2}%
 \def\cdp@elt##1##2##3##4{\def\reserved@c{##1}%
      \ifx\reserved@b\reserved@c \@tempswatrue\fi}%
 \cdp@list
 \if@tempswa
   \@ifundefined{sym#1}{%
```

```
      \ifnum\count18<15 %
        \expandafter\new@mathgroup\csname sym#1\endcsname
        \expandafter\new@symbolfont\csname sym#1\endcsname
                        {#2}{#3}{#4}{#5}%
      \else
         \@latex@error{Too many symbol fonts declared}\@eha
      \fi
     }%
     {%
      \@font@info{Redeclaring symbol font `#1'}%
```

Update the group list.

```
      \def\group@elt##1##2{%
           \noexpand\group@elt\noexpand##1%
           \expandafter\ifx\csname sym#1\endcsname##1%
             \expandafter\noexpand\csname#2/#3/#4/#5\endcsname
           \else
               \noexpand##2%
           \fi}%
      \xdef\group@list{\group@list}%
```

Update the version list.

```
      \def\version@elt##1{%
          \expandafter
          \SetSymbolFont@\expandafter##1\csname#2/#3/#4/#5\expandafter
              \endcsname \csname sym#1\endcsname
          }%
      \version@list
     }%
  \else
    \@latex@error{Encoding scheme  `#2' unknown}\@eha
  \fi
  }
\@onlypreamble\DeclareSymbolFont
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\DeclareSymbolFont}{maybe drop m}%
⟨latexrelease⟩
⟨latexrelease⟩\let\DeclareSymbolFont\DeclareSymbolFont@m@dropped
⟨latexrelease⟩\let\DeclareSymbolFont@m@dropped\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\DeclareSymbolFont`.)

`\group@list`

```
\let\group@list\@empty
\@onlypreamble\group@list
```

(*End of definition for* `\group@list`.)

`\group@elt`

```
\let\group@elt\relax
\@onlypreamble\group@elt
```

(*End of definition for* `\group@elt`.)

`\new@symbolfont`

```
\def\new@symbolfont#1#2#3#4#5{%
    \toks@\expandafter{\group@list}%
    \edef\group@list{\the\toks@\noexpand\group@elt\noexpand#1%
                      \expandafter\noexpand\csname#2/#3/#4/#5\endcsname}%
    \def\version@elt##1{\toks@\expandafter{##1}%
                    \edef##1{\the\toks@\noexpand\getanddefine@fonts
                    #1\expandafter\noexpand\csname#2/#3/#4/#5\endcsname}%
                   \global\advance\csname c@\expandafter
                                \@gobble\string##1\endcsname\@ne
                  }%
    \version@list
}
\@onlypreamble\new@symbolfont
```

(*End of definition for* `\new@symbolfont`.)

`\SetSymbolFont` First drop any surplus `m` from the series argument then do what has been done since 1994.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2022/11/01}%
⟨latexrelease⟩                  {\SetSymbolFont}{maybe drop m}%
\def\SetSymbolFont #1#2#3#4#5#6{%
 \def\reserved@a{\SetSymbolFont@m@dropped{#1}{#2}{#3}{#4}}%
 \edef\reserved@b{#5}%
 \series@maybe@drop@one@m\reserved@b\reserved@b
 \expandafter\reserved@a\expandafter{\reserved@b}{#6}%
}
\def\SetSymbolFont@m@dropped#1#2#3#4#5#6{%
 \@tempswafalse
 \edef\reserved@b{#3}%
 \def\cdp@elt##1##2##3##4{\def\reserved@c{##1}%
      \ifx\reserved@b\reserved@c \@tempswatrue\fi}%
 \cdp@list
 \if@tempswa
  \expandafter\SetSymbolFont@
    \csname mv@#2\expandafter\endcsname\csname#3/#4/#5/#6\expandafter
    \endcsname \csname sym#1\endcsname
 \else
  \@latex@error{Encoding scheme  `#3' unknown}\@eha
 \fi
}
\@onlypreamble\SetSymbolFont
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\SetSymbolFont}{maybe drop m}%
⟨latexrelease⟩
⟨latexrelease⟩\let\SetSymbolFont\SetSymbolFont@m@dropped
⟨latexrelease⟩\let\SetSymbolFont@m@dropped\@undefined
⟨latexrelease⟩
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\SetSymbolFont`.)

`\SetSymbolFont@`

```
\def\SetSymbolFont@#1#2#3{%
  \expandafter\in@\expandafter#1\expandafter{\version@list}%
  \ifin@
    \expandafter\in@\expandafter#3\expandafter{\group@list}%
    \ifin@
      \begingroup
        \expandafter\get@cdp\string#2\@nil\reserved@a
        \toks@{}%
        \def\install@mathalphabet##1##2{%
             \addto@hook\toks@{\install@mathalphabet##1{##2}}%
            }%
        \def\getanddefine@fonts##1##2{%
          \ifnum##1=#3%
             \addto@hook\toks@{\getanddefine@fonts#3#2}%
             \expandafter\get@cdp\string##2\@nil\reserved@b
             \ifx\reserved@a\reserved@b\else
                \@font@info{Encoding `\reserved@b' has changed
                    to `\reserved@a' for symbol font\MessageBreak
                   `\expandafter\@gobblefour\string#3' in the
                    math version `\expandafter
                    \@gobblefour\string#1'}%
             \fi
             \@font@info{%
                Overwriting symbol font
                `\expandafter\@gobblefour\string#3' in
                 version `\expandafter
                \@gobblefour\string#1'\MessageBreak
                \@spaces \expandafter\@gobble\string##2 -->
                         \expandafter\@gobble\string#2}%
          \else
             \addto@hook\toks@{\getanddefine@fonts##1##2}%
          \fi}%
         #1%
         \xdef#1{\the\toks@}%
      \endgroup
    \else
       \@latex@error{Symbol font `\expandafter\@gobblefour\string#3'
                  not defined}\@eha
    \fi
  \else
    \@latex@error{Math version `\expandafter\@gobblefour\string#1'
       is not
       defined}{You probably misspelled the name of the math
       version.^^JOr you have to specify an additional package.}%
  \fi
}
\@onlypreamble\SetSymbolFont@
```

(*End of definition for* `\SetSymbolFont@`.)

\get@cdp

```
\def\get@cdp#1#2/#3\@nil#4{\def#4{#2}}
\@onlypreamble\get@cdp
```

(*End of definition for* `\get@cdp`.)

\DeclareMathAlphabet

```
\def\DeclareMathAlphabet#1#2#3#4#5{%
 \@tempswafalse
 \edef\reserved@b{#2}%
 \def\cdp@elt##1##2##3##4{\def\reserved@c{##1}%
      \ifx\reserved@b\reserved@c \@tempswatrue\fi}%
 \cdp@list
 \if@tempswa
   \expandafter\ifx
   \csname\expandafter\@gobble\string#1\endcsname
   \relax
      \new@mathalphabet#1{#2}{#3}{#4}{#5}%
   \else
```

Check if it is already a math alphabet.

```
     \edef\reserved@a{\noexpand\in@{\string\select@group}%
         {\expandafter\meaning\csname \expandafter
          \@gobble\string#1\space\endcsname}}%
     \reserved@a
     \ifin@
       \@font@info{Redeclaring math alphabet \string#1}%
       \def\version@elt##1{%
         \expandafter\SetMathAlphabet@\expandafter
            ##1\csname#2/#3/#4/#5\expandafter\endcsname
            \csname M@#2\expandafter\endcsname
            \csname \expandafter\@gobble\string#1\space\endcsname#1}%
       \version@list
     \else
```

Check if it is a math alphabet defined via `\DeclareSymbolFontAlphabet`.

```
       \edef\reserved@a{\noexpand\in@{\string\use@mathgroup}%
         {\expandafter\meaning\csname \expandafter
          \@gobble\string#1\space\endcsname}}%
       \reserved@a
       \ifin@
```

In that case overwriting is simple since there is nothing inserted in the math version macros.

```
         \@font@info{Redeclaring math alphabet \string#1}%
         \new@mathalphabet#1{#2}{#3}{#4}{#5}%
```

Otherwise panic.

```
       \else
         \@latex@error{Command `\string#1' already defined}\@eha
       \fi
     \fi
  \fi
 \else
  \@latex@error{Encoding scheme  `#2' unknown}\@eha
```

```
 \fi
  }
\@onlypreamble\DeclareMathAlphabet
```

(*End of definition for* `\DeclareMathAlphabet`.)

`\new@mathalphabet`

```
\def\new@mathalphabet#1#2#3#4#5{%
    \toks@\expandafter{\alpha@list}%
    \edef#1{\expandafter\noexpand\csname \expandafter
            \@gobble\string#1\space\endcsname
            \if/#5/%
               \noexpand\no@alphabet@error
               \noexpand\no@alphabet@error
            \else
               \expandafter\noexpand\csname M@#2\endcsname
               \expandafter\noexpand\csname#2/#3/#4/#5\endcsname
            \fi
           }%
    \toks2\expandafter{#1}%
    \edef\alpha@list{\the\toks@\noexpand\alpha@elt\the\toks2}%
    \def\version@elt##1{\toks@\expandafter{##1}%
                   \edef##1{\the\toks@\install@mathalphabet
                            \expandafter\noexpand
                            \csname \expandafter\@gobble
                               \string#1\space\endcsname
                           {\if/#5/%
                             \noexpand\no@alphabet@error
                             \noexpand#1%
                            \else
                             \noexpand\select@group\the\toks2
                            \fi}}%
                  }%
    \version@list
    \expandafter\edef\csname \expandafter\@gobble
                 \string#1\space\endcsname{\if/#5/%
              \noexpand\no@alphabet@error
              \noexpand#1%
            \else
              \noexpand\select@group\the\toks2
            \fi}%
    \edef#1{\noexpand\protect
            \expandafter\noexpand\csname \expandafter
            \@gobble\string#1\space\endcsname}%
}
\@onlypreamble\new@mathalphabet
```

(*End of definition for* `\new@mathalphabet`.)

`\SetMathAlphabet`

```
\def\SetMathAlphabet#1#2#3#4#5#6{%
 \@tempswafalse
 \edef\reserved@b{#3}%
 \def\cdp@elt##1##2##3##4{\def\reserved@c{##1}%
      \ifx\reserved@b\reserved@c \@tempswatrue\fi}%
```

```
 \cdp@list
 \if@tempswa
  \expandafter\SetMathAlphabet@
    \csname mv@#2\expandafter\endcsname\csname#3/#4/#5/#6\expandafter
    \endcsname \csname M@#3\expandafter\endcsname
    \csname \expandafter\@gobble\string#1\space\endcsname#1%
 \else
  \@latex@error{Encoding scheme  `#3' unknown}\@eha
 \fi
}
\@onlypreamble\SetMathAlphabet
```

(*End of definition for* `\SetMathAlphabet`.)

`\SetMathAlphabet@`

```
\def\SetMathAlphabet@#1#2#3#4#5{%
  \expandafter\in@\expandafter#1\expandafter{\version@list}%
  \ifin@
    \expandafter\in@\expandafter#4\expandafter{\alpha@list}%
    \ifin@
      \begingroup
        \toks@{}%
        \def\getanddefine@fonts##1##2{%
             \addto@hook\toks@{\getanddefine@fonts##1##2}%
            }%
        \def\reserved@c##1##2##3##4{%                % for message below
            \expandafter\@gobble\string##4}%
        \def\install@mathalphabet##1##2{%
          \ifx##1#4%
             \addto@hook\toks@
                {\install@mathalphabet#4{\select@group#4#3#2}}%
             \@font@info{Overwriting math alphabet
                `\string#5' in version `\expandafter
                 \@gobblefour\string#1'\MessageBreak
                 \@spaces \reserved@c##2 -->
                        \expandafter\@gobble\string#2}%
          \else
             \addto@hook\toks@{\install@mathalphabet##1{##2}}%
          \fi
          }%
        #1%
        \xdef#1{\the\toks@}%
      \endgroup
    \else
```

If the math alphabet was defined via `\DeclareSymbolFontAlphabet` we have remove its external definition and add it as a normal math alphabet to every version before trying to change it in one version.

```
      \edef\reserved@a{%
        \noexpand\in@{\string\use@mathgroup}{\meaning#4}}%
      \reserved@a
      \ifin@
        \def\reserved@b##1\use@mathgroup##2##3{%
            \def\reserved@b{##3}\def\reserved@c{##2}}%
        \expandafter\reserved@b#4%
```

```
          \begingroup
            \def\install@mathalphabet##1##2{%
                \addto@hook\toks@{\install@mathalphabet##1{##2}}%
                }%
             \def\getanddefine@fonts##1##2{%
               \addto@hook\toks@{\getanddefine@fonts##1##2}%
               \ifnum##1=\reserved@b
                  \expandafter
                  \addto@hook\expandafter\toks@
                  \expandafter{\expandafter\install@mathalphabet
                  \expandafter#4\expandafter
                        {\expandafter\select@group\expandafter
                          #4\reserved@c##2}}%
               \fi
                       }%
            \def\version@elt##1{%
                \toks@{}%
                ##1%
                \xdef##1{\the\toks@}%
               }%
            \version@list
          \endgroup
```

Put it into the `\alpha@list` with default 'error'

```
          \expandafter\gdef\expandafter\alpha@list\expandafter
              {\alpha@list
               \alpha@elt #4\no@alphabet@error \no@alphabet@error}%
          \gdef#4{\no@alphabet@error #5}% fake things :-)
```

Then call the internal setting routine again:

```
          \SetMathAlphabet@{#1}{#2}{#3}#4#5%
        \else
          \@latex@error{Command `\string#5' not defined as a
                        math alphabet}%
             {Use \noexpand\DeclareMathAlphabet to define it.}%
        \fi
      \fi
    \else
      \@latex@error{Math version `\expandafter\@gobblefour\string#1'
         is not
         defined}{You probably misspelled the name of the math
         version.^^JOr you have to specify an additional package.}%
    \fi
}
\@onlypreamble\SetMathAlphabet@
```

(*End of definition for* `\SetMathAlphabet@`.)

`\DeclareMathAccent` Could do with more checks like allowing single number in `#4` lowercase in `#4` etc

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                    {DeclareMathAccent}{Make math accents robust}%
\def\DeclareMathAccent#1#2#3#4{%
  \expandafter\in@\csname sym#3\expandafter\endcsname
```

```
    \expandafter{\group@list}%
  \ifin@
    \begingroup
      \count\z@=#4\relax
      \count\tw@\count\z@
      \divide\count\z@\sixt@@n
      \count@\count\z@
      \multiply\count@\sixt@@n
      \advance\count\tw@-\count@
      \if\relax\noexpand#1% is command?
        \edef\reserved@a{\noexpand\in@
          {\expandafter\@gobble\string\mathaccent}
          {\expandafter\meaning
           \csname\expandafter\@gobble\string#1\space\endcsname}}%
        \reserved@a
        \ifin@
          \expandafter\let
            \csname\expandafter\@gobble\string#1\space\endcsname
            \@undefined
          \expandafter\set@mathaccent
             \csname sym#3\endcsname#1#2%
             {\hexnumber@{\count\z@}\hexnumber@{\count\tw@}}%
          \@font@info{Redeclaring math accent \string#1}%
        \else
          \expandafter\ifx
          \csname\expandafter\@gobble\string#1\endcsname
          \relax
            \expandafter\set@mathaccent
               \csname sym#3\endcsname#1#2%
               {\hexnumber@{\count\z@}\hexnumber@{\count\tw@}}%
          \else
            \@latex@error{Command `\string#1' already defined}\@eha
          \fi
        \fi
      \else
       \@latex@error{Not a command name: `\noexpand#1'}\@eha
      \fi
    \endgroup
  \else
    \@latex@error{Symbol font `#3' is not defined}\@eha
  \fi
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {DeclareMathAccent}{Make math accents robust}%
⟨latexrelease⟩\def\DeclareMathAccent#1#2#3#4{%
⟨latexrelease⟩  \expandafter\in@\csname sym#3\expandafter\endcsname
⟨latexrelease⟩     \expandafter{\group@list}%
⟨latexrelease⟩  \ifin@
⟨latexrelease⟩    \begingroup
⟨latexrelease⟩      \count\z@=#4\relax
⟨latexrelease⟩      \count\tw@\count\z@
⟨latexrelease⟩      \divide\count\z@\sixt@@n
```

```
⟨latexrelease⟩       \count@\count\z@
⟨latexrelease⟩       \multiply\count@\sixt@@n
⟨latexrelease⟩       \advance\count\tw@-\count@
⟨latexrelease⟩       \if\relax\noexpand#1% is command?
⟨latexrelease⟩         \edef\reserved@a{\noexpand\in@
⟨latexrelease⟩            {\expandafter\@gobble\string\mathaccent}{\meaning#1}}%
⟨latexrelease⟩         \reserved@a
⟨latexrelease⟩         \ifin@
⟨latexrelease⟩           \expandafter\set@mathaccent
⟨latexrelease⟩              \csname sym#3\endcsname#1#2%
⟨latexrelease⟩              {\hexnumber@{\count\z@}\hexnumber@{\count\tw@}}%
⟨latexrelease⟩           \@font@info{Redeclaring math accent \string#1}%
⟨latexrelease⟩         \else
⟨latexrelease⟩           \expandafter\ifx
⟨latexrelease⟩           \csname\expandafter\@gobble\string#1\endcsname
⟨latexrelease⟩           \relax
⟨latexrelease⟩             \expandafter\set@mathaccent
⟨latexrelease⟩                \csname sym#3\endcsname#1#2%
⟨latexrelease⟩                {\hexnumber@{\count\z@}\hexnumber@{\count\tw@}}%
⟨latexrelease⟩           \else
⟨latexrelease⟩             \@latex@error{Command ‘\string#1’ already defined}\@eha
⟨latexrelease⟩           \fi
⟨latexrelease⟩         \fi
⟨latexrelease⟩       \else
⟨latexrelease⟩        \@latex@error{Not a command name: ‘\noexpand#1’}\@eha
⟨latexrelease⟩       \fi
⟨latexrelease⟩     \endgroup
⟨latexrelease⟩   \else
⟨latexrelease⟩     \@latex@error{Symbol font ‘#3’ is not defined}\@eha
⟨latexrelease⟩   \fi
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
\@onlypreamble\DeclareMathAccent
```

(*End of definition for* `\DeclareMathAccent`.)

`\set@mathaccent`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                  {\set@mathaccent}{makemath accents robust}%
\def\set@mathaccent#1#2#3#4{%
  \xdef#2{\mathaccent"\mathchar@type#3\hexnumber@#1#4\relax}%
  \MakeRobust#2%
}
\@onlypreamble\set@mathaccent
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\set@mathaccent}{makemath accents robust}%
⟨latexrelease⟩
⟨latexrelease⟩\def\set@mathaccent#1#2#3#4{%
⟨latexrelease⟩  \xdef#2{\mathaccent"\mathchar@type#3\hexnumber@#1#4\relax}}
```

```
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\set@mathaccent`.)

`\DeclareMathSymbol`

```
\def\DeclareMathSymbol#1#2#3#4{%
  \expandafter\in@\csname sym#3\expandafter\endcsname
     \expandafter{\group@list}%
  \ifin@
    \begingroup
      \count\z@=#4\relax
      \count\tw@\count\z@
      \divide\count\z@\sixt@@n
      \count@\count\z@
      \multiply\count@\sixt@@n
      \advance\count\tw@-\count@
      \if\relax\noexpand#1% is command?
```

Store the command name with a space attached inside `\reserved@@b` in case we look at a robust definition.

```
        \edef\reserved@b{\expandafter\noexpand
                          \csname\expandafter\@gobble\string#1\space\endcsname}%
```

Test both `#1` and `#1␣` for containing `mathchar`.

```
        \edef\reserved@a
          {\noexpand\in@{\expandafter\@gobble\string\mathchar}%
                        {\meaning#1\expandafter\meaning\reserved@b}}%
        \reserved@a
```

Drop `#1␣` in case it was defined before.

```
        \global\expandafter\let\reserved@b\@undefined
        \ifin@
          \expandafter\set@mathsymbol
             \csname sym#3\endcsname#1#2%
             {\hexnumber@{\count\z@}\hexnumber@{\count\tw@}}%
          \@font@info{Redeclaring math symbol \string#1}%
        \else
          \expandafter\ifx
            \csname\expandafter\@gobble\string#1\endcsname
            \relax
            \expandafter\set@mathsymbol
               \csname sym#3\endcsname#1#2%
               {\hexnumber@{\count\z@}\hexnumber@{\count\tw@}}%
          \else
            \@latex@error{Command `\string#1' already defined}\@eha
          \fi
        \fi
      \else
        \expandafter\set@mathchar
          \csname sym#3\endcsname#1#2
          {\hexnumber@{\count\z@}\hexnumber@{\count\tw@}}%
      \fi
    \endgroup
```

```
  \else
    \@latex@error{Symbol font `#3' is not defined}\@eha
  \fi
}
\@onlypreamble\DeclareMathSymbol
```

(*End of definition for* `\DeclareMathSymbol`.)

`\set@mathchar`

```
\def\set@mathchar#1#2#3#4{%
  \global\mathcode`#2="\mathchar@type#3\hexnumber@#1#4\relax}
\@onlypreamble\set@mathchar
```

(*End of definition for* `\set@mathchar`.)

`\set@mathsymbol`

```
\def\set@mathsymbol#1#2#3#4{%
  \global\mathchardef#2"\mathchar@type#3\hexnumber@#1#4\relax}
\@onlypreamble\set@mathsymbol
```

(*End of definition for* `\set@mathsymbol`.)

```
%\def\mathsymbol#1#2#3{%
%  \@tempcnta=#3\relax
%  \@tempcntb\@tempcnta
%  \divide\@tempcnta\sixt@@n
%  \count@\@tempcnta
%  \multiply\count@\sixt@@n
%  \advance\@tempcntb-\count@
%  \mathchar"\mathchar@type#1\hexnumber@#2%
%             \hexnumber@\@tempcnta\hexnumber@\@tempcntb\relax}
%
%\def\DeclareMathAlphabetCharacter#1#2#3{%
%  \DeclareMathSymbol{#1}7{#2}{#3}}
```

`\DeclareMathDelimiter`

```
\def\DeclareMathDelimiter#1{%
  \if\relax\noexpand#1%
    \expandafter\@DeclareMathDelimiter
  \else
    \expandafter\@xxDeclareMathDelimiter
  \fi
  #1}
\@onlypreamble\DeclareMathDelimiter
```

(*End of definition for* `\DeclareMathDelimiter`.)

`\@xxDeclareMathDelimiter` This macro checks if the second arg is a "math type" such as `\mathopen`. The undocumented original code didn't use math types when the delimiter was a single letter. For this reason the coding is a bit strange as it tries to support the undocumented syntax for compatibility reasons.

```
\def\@xxDeclareMathDelimiter#1#2#3#4{%
```

7 is the default value returned in the case that `\mathchar@type` is passed something unexpected, like a math symbol font name. We locally move `\mathalpha` out of the way so if you use that the right branch is taken. This will still fail if an explicit number 7 is used!

```
    \begingroup
     \let\mathalpha\mathord
     \ifnum7=\mathchar@type{#2}%
       \endgroup
```

If this branch is taken we have old syntax (5 arguments).

```
       \expandafter\@firstofone
     \else
```

If this branch is taken `\mathchar@type` is different from 7 so we assume new syntax. In this case we also use the arguments to set up the letter as a math symbol for the case where it is not used as a delimiter.

```
       \endgroup
       \DeclareMathSymbol#1{#2}{#3}{#4}%
```

Then we arrange that `\@xDeclareMathDelimiter` only gets `#1`, `#3`, `#4` ... as it does not expect a math type as argument.

```
       \expandafter\@firstoftwo
     \fi
     {\@xDeclareMathDelimiter#1}{#2}{#3}{#4}}
\@onlypreamble\@xxDeclareMathDelimiter
```

(*End of definition for* `\@xxDeclareMathDelimiter`.)

`\@DeclareMathDelimiter`

```
\def\@DeclareMathDelimiter#1#2#3#4#5#6{%
  \expandafter\in@\csname sym#3\expandafter\endcsname
     \expandafter{\group@list}%
  \ifin@
    \expandafter\in@\csname sym#5\expandafter\endcsname
       \expandafter{\group@list}%
    \ifin@
      \begingroup
        \count\z@=#4\relax
        \count\tw@\count\z@
        \divide\count\z@\sixt@@n
        \count@\count\z@
        \multiply\count@\sixt@@n
        \advance\count\tw@-\count@
        \edef\reserved@c{\hexnumber@{\count\z@}\hexnumber@{\count\tw@}}%
      %
        \count\z@=#6\relax
        \count\tw@\count\z@
        \divide\count\z@\sixt@@n
        \count@\count\z@
        \multiply\count@\sixt@@n
        \advance\count\tw@-\count@
        \edef\reserved@d{\hexnumber@{\count\z@}\hexnumber@{\count\tw@}}%
      %
        \edef\reserved@a{\noexpand\in@
            {\expandafter\@gobble\string\delimiter}{\meaning#1}}%
```

```
        \reserved@a
        \ifin@
          \expandafter\set@mathdelimiter
             \csname sym#3\expandafter\endcsname
             \csname sym#5\endcsname#1#2%
             \reserved@c\reserved@d
          \@font@info{Redeclaring math delimiter \string#1}%
        \else
            \expandafter\ifx
            \csname\expandafter\@gobble\string#1\endcsname
            \relax
            \expandafter\set@mathdelimiter
              \csname sym#3\expandafter\endcsname
              \csname sym#5\endcsname#1#2%
              \reserved@c\reserved@d
          \else
            \@latex@error{Command `\string#1' already defined}\@eha
          \fi
        \fi
      \endgroup
    \else
      \@latex@error{Symbol font `#5' is not defined}\@eha
    \fi
  \else
    \@latex@error{Symbol font `#3' is not defined}\@eha
  \fi
}
\@onlypreamble\@DeclareMathDelimiter
```

(*End of definition for* `\@DeclareMathDelimiter`.)

`\@xDeclareMathDelimiter`

```
\def\@xDeclareMathDelimiter#1#2#3#4#5{%
  \expandafter\in@\csname sym#2\expandafter\endcsname
     \expandafter{\group@list}%
  \ifin@
    \expandafter\in@\csname sym#4\expandafter\endcsname
       \expandafter{\group@list}%
    \ifin@
      \begingroup
        \count\z@=#3\relax
        \count\tw@\count\z@
        \divide\count\z@\sixt@@n
        \count@\count\z@
        \multiply\count@\sixt@@n
        \advance\count\tw@-\count@
        \edef\reserved@c{\hexnumber@{\count\z@}\hexnumber@{\count\tw@}}%
      %
        \count\z@=#5\relax
        \count\tw@\count\z@
        \divide\count\z@\sixt@@n
        \count@\count\z@
        \multiply\count@\sixt@@n
        \advance\count\tw@-\count@
```

```
        \edef\reserved@d{\hexnumber@{\count\z@}\hexnumber@{\count\tw@}}%
        \expandafter\set@@mathdelimiter
           \csname sym#2\expandafter\endcsname\csname sym#4\endcsname#1%
           \reserved@c\reserved@d
      \endgroup
    \else
      \@latex@error{Symbol font `#4' is not defined}\@eha
    \fi
  \else
    \@latex@error{Symbol font `#2' is not defined}\@eha
  \fi
}
\@onlypreamble\@xDeclareMathDelimiter
```

(*End of definition for* `\@xDeclareMathDelimiter`.)

`\set@mathdelimiter` We have to end the definition of a math delimiter like `\lfloor` with a space and not with `\relax` as we did before, because otherwise constructs involving `\abovewithdelims` will prematurely end (pr/1329)

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                  {\set@mathdelimiter}{make delimiters robust}%
\def\set@mathdelimiter#1#2#3#4#5#6{%
```

We use `\protected` not `\MakeRobust` so that `\bigl\lfloor` etc. works inside the argument of `\protected@edef`.

```
  \protected
  \xdef#3{\delimiter"\mathchar@type#4\hexnumber@#1#5%
    \hexnumber@#2#6 }%
%  \MakeRobust#3%
}
\@onlypreamble\set@mathdelimiter
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\set@mathdelimiter}{make delimiters robust}%
⟨latexrelease⟩
⟨latexrelease⟩\def\set@mathdelimiter#1#2#3#4#5#6{%
⟨latexrelease⟩  \xdef#3{\delimiter"\mathchar@type#4\hexnumber@#1#5%
⟨latexrelease⟩    \hexnumber@#2#6 }}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\set@mathdelimiter`.)

`\set@@mathdelimiter`

```
\def\set@@mathdelimiter#1#2#3#4#5{%
  \global\delcode`#3="\hexnumber@#1#4\hexnumber@#2#5\relax}
\@onlypreamble\set@@mathdelimiter
```

(*End of definition for* `\set@@mathdelimiter`.)

\DeclareMathRadical

```
\def\DeclareMathRadical#1#2#3#4#5{%
```

Below is a crude fix to make this macro work if `#1` is undefined or `\relax`. Should be improved!

```
  \expandafter\ifx
       \csname\expandafter\@gobble\string#1\endcsname
       \relax
     \let#1\radical
  \fi
  \edef\reserved@a{\noexpand\in@
       {\expandafter\@gobble\string\radical}{\meaning#1}}%
  \reserved@a
  \ifin@
    \expandafter\in@\csname sym#2\expandafter\endcsname
       \expandafter{\group@list}%
    \ifin@
      \expandafter\in@\csname sym#4\expandafter\endcsname
         \expandafter{\group@list}%
      \ifin@
        \begingroup
          \count\z@=#3\relax
          \count\tw@\count\z@
          \divide\count\z@\sixt@@n
          \count@\count\z@
          \multiply\count@\sixt@@n
          \advance\count\tw@-\count@
          \edef\reserved@c{%
            \hexnumber@{\count\z@}\hexnumber@{\count\tw@}}%
          \count\z@=#5\relax
          \count\tw@\count\z@
          \divide\count\z@\sixt@@n
          \count@\count\z@
          \multiply\count@\sixt@@n
          \advance\count\tw@-\count@
          \edef\reserved@d{%
            \hexnumber@{\count\z@}\hexnumber@{\count\tw@}}%
```

Coded inline instead of using `\set@mathradical`

```
%          \expandafter\set@mathradical
%             \csname sym#2\expandafter\endcsname
%             \csname sym#4\endcsname#1%
%             \reserved@c\reserved@d
          \xdef#1{\radical"\expandafter\hexnumber@
                                \csname sym#2\endcsname\reserved@c
                              \expandafter\hexnumber@
                                \csname sym#4\endcsname\reserved@d
                    \relax}%
        \endgroup
      \else
        \@latex@error{Symbol font `#4' is not defined}\@eha
      \fi
    \else
      \@latex@error{Symbol font `#2' is not defined}\@eha
    \fi
```

```
  \else
    \@latex@error{Command `\string#1' already defined}\@eha
  \fi
}
\@onlypreamble\DeclareMathRadical
```

(*End of definition for* `\DeclareMathRadical`.)

Definition below was wrong it contained `\delimiter` !

```
def\set@mathradical#1#2#3#4#5{%
 \xdef#3{\radical"\hexnumber@#1#4\hexnumber@#2#5\relax}}
```

`\mathalpha` just a dummy currently

```
\let\mathalpha\relax
```

(*End of definition for* `\mathalpha`.)

`\mathchar@type`

```
\def\mathchar@type#1{%
  \ifodd 2#11 #1\else            % is this non-negative number?
    \ifx#1\mathord 0\else
     \ifx#1\mathop   1\else
       \ifx#1\mathbin 2\else
         \ifx#1\mathrel 3\else
           \ifx#1\mathopen 4\else
             \ifx#1\mathclose 5\else
               \ifx#1\mathpunct 6\else
                   7%            % anything else is variable ord
               \fi
             \fi
           \fi
         \fi
       \fi
     \fi
    \fi
  \fi}
\@onlypreamble\mathchar@type
```

(*End of definition for* `\mathchar@type`.)

`\DeclareSymbolFontAlphabet`

```
\def\DeclareSymbolFontAlphabet#1#2{%
   \expandafter\DeclareSymbolFontAlphabet@
     \csname \expandafter\@gobble\string#1\space\endcsname{#2}#1}
\@onlypreamble\DeclareSymbolFontAlphabet
```

(*End of definition for* `\DeclareSymbolFontAlphabet`.)

`\DeclareSymbolFontAlphabet@`

```
\def\DeclareSymbolFontAlphabet@#1#2#3{%
```

We use the switch `\if@tempswa` to decide if we can declare this symbol font alphabet.

```
    \@tempswatrue
```

First check if #2 is known to be a symbol font

```
\expandafter\in@\csname sym#2\expandafter\endcsname
   \expandafter{\group@list}%
\ifin@
```

Check if #1 is defined as a math alphabet defined via \DeclareMathAlphabet:

```
  \expandafter\in@\expandafter#1\expandafter{\alpha@list}%
  \ifin@
```

If so remove it from the \alpha@list and from all math version macros.

```
    \@font@info{Redeclaring math alphabet \string#3}%
    \toks@{}%
    \def\alpha@elt##1##2##3{%
        \ifx##1#1\else\addto@hook\toks@{\alpha@elt##1##2##3}\fi}%
    \alpha@list
    \xdef\alpha@list{\the\toks@}%
```

Now we loop over all versions and remove the math alphabet:

```
    \def\version@elt##1{%
        \begingroup
          \toks@{}%
          \def\getanddefine@fonts####1####2{%
             \addto@hook\toks@{\getanddefine@fonts####1####2}}%
          \def\install@mathalphabet####1####2{%
             \ifx####1#1\else
               \addto@hook\toks@{\install@mathalphabet
                                 ####1{####2}}\fi}%
          ##1%
          \xdef##1{\the\toks@}%
        \endgroup
        }%
    \version@list
  \else
```

If #3 is not defined as a math alphabet check if it is defined at all:

```
    \expandafter\ifx
    \csname\expandafter\@gobble\string#1\space\endcsname
    \relax
```

If it is undefined, fine otherwise check if it is a math alphabet defined via \DeclareSymbolFontAlphabet:

```
    \else
      \edef\reserved@a{%
        \noexpand\in@{\string\use@mathgroup}{\meaning#1}}%
      \reserved@a
      \ifin@
        \@font@info{Redeclaring math alphabet \string#3}%
      \else
```

Since the command #3 is defined to be something which is not a math alphabet we have to skip redefining it.

```
        \@tempswafalse
        \@latex@error{Command `\string#3' already defined}\@eha
      \fi
    \fi
  \fi
\else
```

Since the symbol font is not known we better skip defining this alphabet.

```
      \@tempswafalse
      \@latex@error{Unknown symbol font `#2'}\@eha
    \fi
    \if@tempswa
```

When we reach this point we are allowed to define `#1` to be a symbol font math alphabet. This means that we have to set it to

`\use@mathgroup` ⟨*math-settings*⟩ `\sym`⟨*name*⟩

The ⟨*math-settings*⟩ are the one for the encoding that is used in the font shape where `\sym`⟨*name*⟩ is pointing to. This means that we have to get it from the information stored in `\group@list`. Thus we loop through that list after defining `\group@elt` in a suitable way.

```
      \def\group@elt##1##2{%
         \expandafter\ifx\csname sym#2\endcsname##1%
         \expandafter\reserved@a\string##2\@nil
         \fi}%
      \def\reserved@a##1##2/##3\@nil{%
         \def\reserved@a{##2}}%
      \group@list
      \toks@{\relax\ifmmode \else \non@alpherr#1\fi}%
      \edef#1{\the\toks@
              \noexpand\use@mathgroup
              \expandafter\noexpand\csname M@\reserved@a\endcsname
              \csname sym#2\endcsname}%
      \def#3{\protect#1}%
    \fi
}
\@onlypreamble\DeclareSymbolFontAlphabet@
⟨/2ekernel⟩
```

(*End of definition for* `\DeclareSymbolFontAlphabet@`.)

File 29
ltfssini.dtx

This file contains the top level LaTeX interface to the font selection scheme commands. See other parts of the LaTeX distribution, or *The LaTeX Companion* for higher level documentation of these commands.

1 NFSS Initialization

Finally, there are six commands that are to be used in LaTeX and that we will therefore protect against expansion at the wrong point: `\fontfamily`, `\fontseries`, `\fontshape`, `\fontsize`, `\selectfont`, and `\mathversion`.

```
⟨*2ekernel⟩
\message{NFSS initialization,}
```

1.1 Providing math *versions*

LaTeX provides two *versions*. We call them normal and bold, respectively.

```
\DeclareMathVersion{normal}
\DeclareMathVersion{bold}
```

Now we define the standard font change commands. We don't allow the use of `\rmfamily` etc. in math mode.

(Actually most are now defined further down in the file.)

First the changes to another *family*:

```
%\DeclareRobustCommand\rmfamily
%        {\not@math@alphabet\rmfamily\mathrm
%         \fontfamily\rmdefault\selectfont}
%\DeclareRobustCommand\sffamily
%        {\not@math@alphabet\sffamily\mathsf
%         \fontfamily\sfdefault\selectfont}
%\DeclareRobustCommand\ttfamily
%        {\not@math@alphabet\ttfamily\mathtt
%         \fontfamily\ttdefault\selectfont}
```

Then the commands changing the *series*:

```
%\DeclareRobustCommand\bfseries
%        {\not@math@alphabet\bfseries\mathbf
%         \fontseries\bfdefault\selectfont}
%\DeclareRobustCommand\mdseries
%        {\not@math@alphabet\mdseries\relax
%         \fontseries\mddefault\selectfont}
\DeclareRobustCommand\upshape
        {\not@math@alphabet\upshape\relax
         \fontshape\updefault\selectfont}
```

Then the commands changing the *shape*:

```
\DeclareRobustCommand\slshape
        {\not@math@alphabet\slshape\relax
         \fontshape\sldefault\selectfont}
\DeclareRobustCommand\scshape
```

```
        {\not@math@alphabet\scshape\relax
         \fontshape\scdefault\selectfont}
\DeclareRobustCommand\itshape
        {\not@math@alphabet\itshape\mathit
         \fontshape\itdefault\selectfont}
```

2 Custom series settings for main document families

This section was introduced 2020/02/02 and for now we support a full rollback (may need splitting later).

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2021/11/15}%
⟨latexrelease⟩                    {\DeclareFontSeriesDefault}{Custom series}%
```

One problem with the NFSS approach of handling the series axis turned out to be that (especially with respect to "boldness") different font families implemented different strategies. For example, with Computer Modern fonts you normally only have `bx` whereas most PostScript fonts offered only `b` but not `bx`. As a result LaTeX's standard setting for `\bfdefault` didn't work with such fonts, but if it got changed to produce `b`, then that didn't work with Computer Modern if the fonts got combined (e.g., using Computer Modern Typewriter with such fonts).

The solution back then was to provide substitution rules in the font `.fd` such that if a `bx` series got requested the `b` series got used. While this works in that particular case, it isn't a very general solution. For example, if you happen to have a font family that has several weights you may want to typeset the whole document in a somewhat lighter or darker font but if you then modify `\mddefault` to allow for this, then of course your change only works with that particular family but not with the typewriter or sans serif family you also want to use.

A better solution was provided by the `mweights` package by Bob Tennent that offers defaults on the level of the three main font families in the document: for "rm", "sf" and "tt" so that font packages could define defaults for the sans serif document font by providing `\bfseries@sf` which then was used when `\bfseries` got executed and the current family was the `\sffamily`.

`\DeclareFontSeriesDefault`

We now support this concept directly from within LaTeX and for use in font packages (or the document preamble) we offer `\DeclareFontSeriesDefault`. This declaration takes three arguments:

document family interface: Can either be `rm`, `sf` or `tt`. This is optional and if not given the overall default.

document series interface: Can be `md` or `bf`.

series value: This is the value that is going to be used with the combination is requested.

For example, `\DeclareFontSeriesDefault[rm]{bf}{sb}` would use `sb` (semi-bold) when `\rmfamily \bfseries` is asked for.

If used without the optional argument, e.g., `\DeclareFontSeriesDefault{bf}{b}` then this is like redefining `\bfdefault` or `\mddefault`.

If some family specify defaults aren't given, e.g. if there are no declarations for, say, `tt` then the format defaults of `\mddefault` and `\bfdefault` are assumed. If those are later changed this is *not* reflected![37]

`\DeclareFontSeriesDefault` The command to declare font series defaults for the "rm", "sf" or "tt" family.

```
\let\DeclareFontSeriesDefault\@undefined      % for rollback
\newcommand\DeclareFontSeriesDefault[3][]{%

  \expand@font@defaults
  \maybe@update@bfseries@defaults
  \maybe@update@mdseries@defaults

  \def\reserved@a{#1}%
```

No optional argument: set up general default.

```
  \ifx\reserved@a\@empty
    \ifcsname #2series\endcsname           % supported are
                                          % \[md/bf]default
```

Adding `\@empty` allows us to detect if the default gets redefined with `\renewcommand` or `\def` by the user.

```
      \expandafter\def
        \csname #2default\endcsname{#3\@empty}%
      \expandafter\def
        \csname #2default@previous\endcsname{#3\@empty}%
    \else

      \@latex@error{Wrong syntax for \string\DeclareFontSeriesDefault}%
         {Mandatory first argument must be 'md'  or 'bf'.}
    \fi
```

Optional argument given, set up specific default.

```
  \else
    \ifcsname #2series@#1\endcsname          % supported are
                                             % \[md/bf]series@[rm/sf/tt]
      \expandafter\edef
         \csname #2series@#1\endcsname{#3}%
```

If the interface is used we remove the frozen kernel default. This way, we know that something was explicitly set up (even if the setup has the same value as the default).

```
      \expandafter\let
         \csname #2series@#1@kernel\endcsname\@undefined
    \else
       \@latex@error{Wrong syntax for \string\DeclareFontSeriesDefault}%
          {Optional argument must be 'rm', 'sf', or 'tt'. \MessageBreak
           Mandatory first argument must be 'md'  or 'bf'.}
    \fi
  \fi
}
```

[37] I see no easy way to achieve this without compromising compatibility with existing packages that currently use mweights and directly define (some) of the `\mdseries@..` commands but not others.

(*End of definition for* `\DeclareFontSeriesDefault`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩                  {\DeclareFontSeriesDefault}{Custom series}%
⟨latexrelease⟩
⟨latexrelease⟩\let\DeclareFontSeriesDefault\@undefined      % for rollback
⟨latexrelease⟩\newcommand\DeclareFontSeriesDefault[3][]{%
⟨latexrelease⟩  \def\reserved@a{#1}%
⟨latexrelease⟩  \ifx\reserved@a\@empty
⟨latexrelease⟩    \ifcsname #2series\endcsname           % supported are
⟨latexrelease⟩                                          % \[md/bf]default
⟨latexrelease⟩      \expandafter\def
⟨latexrelease⟩        \csname #2default\endcsname{#3\@empty}%
⟨latexrelease⟩      \expandafter\def
⟨latexrelease⟩        \csname #2default@previous\endcsname{#3\@empty}%
⟨latexrelease⟩    \else
⟨latexrelease⟩       \@latex@error{Wrong syntax for \string\DeclareFontSeriesDefault}%
⟨latexrelease⟩          {Mandatory first argument must be 'md'  or 'bf'.}
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \else
⟨latexrelease⟩    \ifcsname #2series@#1\endcsname          % supported are
⟨latexrelease⟩                                          % \[md/bf]series@[rm/sf/tt]
⟨latexrelease⟩      \expandafter\edef
⟨latexrelease⟩         \csname #2series@#1\endcsname{#3}%
⟨latexrelease⟩      \expandafter\let
⟨latexrelease⟩         \csname #2series@#1@kernel\endcsname\@undefined
⟨latexrelease⟩    \else
⟨latexrelease⟩       \@latex@error{Wrong syntax for \string\DeclareFontSeriesDefault}%
⟨latexrelease⟩          {Optional argument must be 'rm', 'sf', or 'tt'. \MessageBreak
⟨latexrelease⟩           Mandatory first argument must be 'md'  or 'bf'.}
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \fi
⟨latexrelease⟩}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\DeclareFontSeriesDefault}{Custom series}%
⟨latexrelease⟩
⟨latexrelease⟩\let\DeclareFontSeriesDefault\@undefined
⟨latexrelease⟩\let\bfseries@rm\@undefined
⟨latexrelease⟩\let\bfseries@sf\@undefined
⟨latexrelease⟩\let\bfseries@tt\@undefined
⟨latexrelease⟩\let\bfseries@rm@kernel\@undefined
⟨latexrelease⟩\let\bfseries@sf@kernel\@undefined
⟨latexrelease⟩\let\bfseries@tt@kernel\@undefined
⟨latexrelease⟩\let\mdseries@rm\@undefined
⟨latexrelease⟩\let\mdseries@sf\@undefined
⟨latexrelease⟩\let\mdseries@tt\@undefined
⟨latexrelease⟩\expandafter\let\csname ver@mweights.sty\endcsname\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let\@meta@family@list\@undefined
⟨latexrelease⟩\let\prepare@family@series@update\@undefined
⟨latexrelease⟩\let\update@series@target@value\@undefined
```

```
⟨latexrelease⟩
```

This is always called in `\document` so don't make it undefined.

```
⟨latexrelease⟩\let\init@series@setup\relax
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩                    {\mdseries@rm}{Custom series}%
```

`\mdseries@rm` `\mdseries@sf` `\mdseries@tt` `\bfseries@rm` `\bfseries@sf` `\bfseries@tt`

We initialize the family specific default at the end of the format generation. Later on they may get overwritten in the preamble or a package via `\DeclareFontSeriesDefault` (or possibly directly).

Conceptual change: The `\bfdefault` will be `b` not `bx` because that is what it should be really for nearly every font except Computer/Latin Modern.

To account for the fact that by default we typeset in CM or LM we set up the `\bfseries@..` defaults to use `bx` instead.

This means that it behaves like before because if the default fonts are used then `\bfseries@rm` etc kick in and make `\textbf` use `bx`. However, if the font gets changed then `\bfdefault` will get used.

```
\def\bfseries@rm{bx}
\def\bfseries@sf{bx}
\def\bfseries@tt{bx}
```

Frozen version of the kernel defaults so we can see if they have changed.

```
\let\bfseries@rm@kernel\bfseries@rm
\let\bfseries@sf@kernel\bfseries@sf
\let\bfseries@tt@kernel\bfseries@tt
```

The default for the medium series is `m` and this will be interpreted as resetting both weight and width. To reset only one of them the virtual value `?m` and `m?` are available.

```
\def\mdseries@rm{m}
\def\mdseries@sf{m}
\def\mdseries@tt{m}
```

(*End of definition for* `\mdseries@rm` *and others.*)

`\series@change@debug`

For debugging, but right now none of this code is extracted. The idea is to have a separate package with debugging code one day.

```
⟨*debug⟩
\let\series@change@debug\typeout
\let\series@change@debug\@gobble
⟨/debug⟩
```

(*End of definition for* `\series@change@debug`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\mdseries@rm}{Custom series}%
⟨latexrelease⟩
⟨latexrelease⟩\let\bfseries@rm\@undefined
```

```
⟨latexrelease⟩\let\bfseries@sf\@undefined
⟨latexrelease⟩\let\bfseries@tt\@undefined
⟨latexrelease⟩\let\bfseries@rm@kernel\@undefined
⟨latexrelease⟩\let\bfseries@sf@kernel\@undefined
⟨latexrelease⟩\let\bfseries@tt@kernel\@undefined
⟨latexrelease⟩\let\mdseries@rm\@undefined
⟨latexrelease⟩\let\mdseries@sf\@undefined
⟨latexrelease⟩\let\mdseries@tt\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/11/01}%
⟨latexrelease⟩      {\prepare@family@series@update}{Support meta family}%
```

\prepare@family@series@update This is core command that prepares for the family update. The big difference to the documented code above is that the nested `\ifx` statements seem to be missing. Instead we loop through an internal list that holds the names of the three meta families. This approach allows us to extend the mechanism at a later stage to allow for additional named meta families.

Here is the current definition of that list:

\@meta@family@list
```
\def\@meta@family@list{\@elt{rm}\@elt{sf}\@elt{tt}}
```

This macro holds the current "meta" family. At the moment it is either `rm`, `sf`, `tt` or `??`.

\@currentmetafamily The latter is used if the current font family does not match any of the document meta families, defined through `\rmdefault`, `\sfdefault` or `\ttdefault`, i.e., in the case that yet another family was selected using `\fontfamily` manually.

The value gets updated by commands like `\rmfamily`, etc.

```
\def\@currentmetafamily{??}

\def\prepare@family@series@update#1#2{%
```

We start by updating the meta family.

```
 \def\@currentmetafamily{#1}%

 \if@forced@series
⟨+debug⟩ \series@change@debug{No series preparation (forced \f@series)\on@line}%
   \fontfamily#2%
 \else
⟨+debug⟩ \series@change@debug{Preparing for switching to #1 (#2)\on@line}%
   \expand@font@defaults
```

We prepare for changing the current series. We have to find it before changing the family as discussed above.

```
   \let\target@series@value\@empty
```

As the very last item in the meta family list we add `\@elt{??}` and define this pseudo meta family to be the current font family. So if none of the real meta families matched then this will match. This will cover the following case:

- `\bfseries` is called for a family using `bx` (e.g., CMR)
- Switch to a font family that is none of the meta families, e.g., via `\fontfamily{ptm}\allowbreak\v`

- Then none of the real meta families, match but the final `\@elt{??}` will.
- Therefore if the current series is `\mddefault` or `\bfdefault` it will be detected and the corresponding target series selected.

```
    \expandafter\edef\csname ??def@ult\endcsname{\f@family}%
```

To find it we loop over the meta family list with a suitable definition of `\@elt`.

```
    \let\@elt\update@series@target@value
       \@meta@family@list
```

Last resort pseudo meta family. Will only be looked at if none of the real ones have matched.

```
       \@elt{??}%
    \let\@elt\relax
```

That will figure out the correct series value to use without updating it. Now we can change the family.

```
    \fontfamily#2%
```

After that we update the series. That code is again like the one above.

```
    \ifx\target@series@value\@empty
⟨+debug⟩ \series@change@debug{Target series still empty ...}%
    \else
      \ifx \f@series\target@series@value
⟨+debug⟩ \series@change@debug{Target series unchanged:
⟨+debug⟩                      \f@series \space = \target@series@value}%
     \else
       \maybe@load@fontshape
⟨+debug⟩ \series@change@debug{Target series:
⟨+debug⟩                      \f@series \space -> \target@series@value}%
```

The `\target@series@value` may contain something like `cm` (coming from a default) and so we can't directly assign it to `\f@series` be have to drop any surplus `m` first.

```
%      \let\f@series\target@series@value
       \series@maybe@drop@one@m\target@series@value\f@series
     \fi
   \fi
 \fi
}
```

(*End of definition for* `\prepare@family@series@update`*,* `\@meta@family@list`*, and* `\@currentmetafamily`*.*)

`\update@series@target@value` In this macro you basically find the nested `\ifx`s from the outline above. The only difference is that is it is parameterized instead of being written out. And we have only for one block of tests because the code is called repeatedly when looping over the meta family list. From the `\@meta@family@list` list we get each meta family name in turn.

```
\def\update@series@target@value#1{%
```

There is one additional test at the beginning, because the list contains all meta families and we need to ignore the case where current one from the list and target one are identical.

```
  \def\reserved@a{#1}%
  \ifx\@currentmetafamily\reserved@a   % rm -> rm do nothing
  \else
```

```
⟨+debug⟩ \series@change@debug{Trying to match #1: \csname#1def@ult\endcsname
⟨+debug⟩                      \space = \f@family\space ?}%
```

We only "do" something if the current font family matches the current meta family.

```
    \expandafter\ifx\csname#1def@ult\endcsname\f@family
```

If that's the case we know that this is the block that applies (only one meta family can match). So to speed things up we change `\@elt` so that the rest of the loop gets gobbled.

```
      \let\@elt\@gobble
```

Then we try to find the right new value for the series (as explained above). The two macros defined first are only there because we now need to use `\csname` and this way the code will be a little faster.

```
      \expandafter\let\expandafter\reserved@b
                      \csname mdseries@\@currentmetafamily\endcsname
      \expandafter\let\expandafter\reserved@c
                      \csname bfseries@\@currentmetafamily\endcsname
⟨+debug⟩\series@change@debug{Targets for mdseries and bfseries:
⟨+debug⟩                     \reserved@b\space and \reserved@c}%
```

This here is now identical to the nested `\ifx` block from the outline, except that it there appeared twice in `\rmfamily`. This is now covered by looping and stopping the loop when a match was found.

We have to sanitize the default value first because it may contain something like `mc` and that would never match `\f@series` because there it would be called `c` with the `m` dropped. It would be probably better to do that differently these days, but it is hard to adjust without causing a lot of issues, so we do the dropping in various places instead.

```
      \expandafter\series@maybe@drop@one@m
          \csname mdseries@#1\endcsname\reserved@d
      \ifx\reserved@d\f@series
⟨+debug⟩   \series@change@debug{mdseries@#1 matched -> \reserved@b}%
                                     \let\target@series@value\reserved@b
      \else
```

Again do some sanitizing.

```
        \expandafter\series@maybe@drop@one@m
           \csname bfseries@#1\endcsname\reserved@d
        \ifx\reserved@d\f@series
⟨+debug⟩  \series@change@debug{bfseries@#1 matched -> \reserved@c}%
                                     \let\target@series@value\reserved@c
      \else\ifx\f@series\mddef@ult    \let\target@series@value\reserved@b
⟨+debug⟩  \series@change@debug{mddef@ult matched -> \reserved@b}%
      \else\ifx\f@series\bfdef@ult    \let\target@series@value\reserved@c
⟨+debug⟩  \series@change@debug{bfdef@ult matched -> \reserved@c}%
      \fi\fi\fi\fi
    \fi
  \fi
}
```

(*End of definition for* `\update@series@target@value`.)

`\@restoremetafamily` In some situations it might be necessary to rerun the command that sets the current meta family, e.g., rerun `\rmfamily` if `rm` is the current meta family (for example, if the hook `rmfamily` contains some conditional code and the condition has changed at that point). The command `\@restoremetafamily` offers a simply way to do this.

```
\def\@restoremetafamily{%
  \csname \@currentmetafamily family\endcsname
}
```

(*End of definition for* `\@restoremetafamily`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩    {\prepare@family@series@update}{Custom series}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@meta@family@list{\@elt{rm}\@elt{sf}\@elt{tt}}
⟨latexrelease⟩\def\prepare@family@series@update#1#2{%
⟨latexrelease⟩ \if@forced@series
⟨latexrelease⟩   \fontfamily#2%
⟨latexrelease⟩ \else
⟨latexrelease⟩   \expand@font@defaults
⟨latexrelease⟩   \let\target@series@value\@empty
⟨latexrelease⟩   \def\target@meta@family@value{#1}%
⟨latexrelease⟩   \expandafter\edef\csname ??def@ult\endcsname{\f@family}%
⟨latexrelease⟩   \let\@elt\update@series@target@value
⟨latexrelease⟩      \@meta@family@list
⟨latexrelease⟩      \@elt{??}%
⟨latexrelease⟩   \let\@elt\relax
⟨latexrelease⟩   \fontfamily#2%
⟨latexrelease⟩   \ifx\target@series@value\@empty
⟨latexrelease⟩   \else
⟨latexrelease⟩     \ifx \f@series\target@series@value
⟨latexrelease⟩     \else
⟨latexrelease⟩       \maybe@load@fontshape
⟨latexrelease⟩       \series@maybe@drop@one@m\target@series@value\f@series
⟨latexrelease⟩     \fi
⟨latexrelease⟩   \fi
⟨latexrelease⟩ \fi
⟨latexrelease⟩}
⟨latexrelease⟩\def\update@series@target@value#1{%
⟨latexrelease⟩  \def\reserved@a{#1}%
⟨latexrelease⟩  \ifx\target@meta@family@value\reserved@a   % rm -> rm do nothing
⟨latexrelease⟩  \else
⟨latexrelease⟩    \expandafter\ifx\csname#1def@ult\endcsname\f@family
⟨latexrelease⟩      \let\@elt\@gobble
⟨latexrelease⟩      \expandafter\let\expandafter\reserved@b
⟨latexrelease⟩                      \csname mdseries@\target@meta@family@value\endcsname
⟨latexrelease⟩      \expandafter\let\expandafter\reserved@c
⟨latexrelease⟩                      \csname bfseries@\target@meta@family@value\endcsname
⟨latexrelease⟩      \expandafter\series@maybe@drop@one@m
⟨latexrelease⟩          \csname mdseries@#1\endcsname\reserved@d
⟨latexrelease⟩      \ifx\reserved@d\f@series
⟨latexrelease⟩                                      \let\target@series@value\reserved@b
⟨latexrelease⟩      \else
⟨latexrelease⟩        \expandafter\series@maybe@drop@one@m
⟨latexrelease⟩           \csname bfseries@#1\endcsname\reserved@d
⟨latexrelease⟩        \ifx\reserved@d\f@series
⟨latexrelease⟩                                      \let\target@series@value\reserved@c
⟨latexrelease⟩      \else\ifx\f@series\mddef@ult    \let\target@series@value\reserved@b
```

```
⟨latexrelease⟩       \else\ifx\f@series\bfdef@ult    \let\target@series@value\reserved@c
⟨latexrelease⟩       \fi\fi\fi\fi
⟨latexrelease⟩     \fi
⟨latexrelease⟩   \fi
⟨latexrelease⟩}

⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩     {\prepare@family@series@update}{Custom series}%
⟨latexrelease⟩
⟨latexrelease⟩\let\@meta@family@list\@undefined
⟨latexrelease⟩\let\prepare@family@series@update\@undefined
⟨latexrelease⟩\let\update@series@target@value\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

`\init@series@setup`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩                    {\init@series@setup}{Custom series}%
```

This is code to be run at begin document ...

```
\def\init@series@setup{%
```

We only want `bx` in `\bfseries@rm` if the roman font is Computer Modern or Latin Modern, otherwise it should be `b`. It was set to `bx` in the kernel so that any font use with the default families in the preamble get this value. Now at the real document start we check if the fonts have been changed. If there was a `\DeclareFontSeriesDefault` declaration or `\bfseries@rm` was directly altered then it differs from `\bfseries@rm@kernel` and we do nothing. Otherwise we check if `\rmdefault` is one of the CM/LM font families and if so we keep `bx` otherwise we change it to `b`.

This approach doesn't cover one case: CM/LM got changed to a different family that supports `bx`, but the support package for that family used `\def\bfseries@rm{bx}` instead of using `\DeclareFontSeriesDefault`. In that case the code here changes it to `b`. Solution: use the `\DeclareFontSeriesDefault` interface.

```
  \ifx\bfseries@rm@kernel\bfseries@rm
     \expandafter\in@\expandafter{\rmdefault}%
                     {cmr,cmss,cmtt,lcmss,lcmtt,lmr,lmss,lmtt}%
    \ifin@ \else \def\bfseries@rm{b}\fi\fi
```

Same approach for `\bfseries@sf` and `\bfseries@tt`:

```
  \ifx\bfseries@sf@kernel\bfseries@sf
    \expandafter\in@\expandafter{\sfdefault}%
                    {cmr,cmss,cmtt,lcmss,lcmtt,lmr,lmss,lmtt}%
    \ifin@ \else \def\bfseries@sf{b}\fi\fi
  \ifx\bfseries@tt@kernel\bfseries@tt
    \expandafter\in@\expandafter{\ttdefault}%
                    {cmr,cmss,cmtt,lcmss,lcmtt,lmr,lmss,lmtt}%
    \ifin@ \else \def\bfseries@tt{b}\fi\fi
```

If the document preamble has changed the `\familydefault` or if the if the `\rmdefault` contains a new font family, we may have to adjust the series defaults accordingly, before starting typesetting.

Similarly, if the user has changed the `\mddefault` or the medium series for the family selected as document font we may also have to adjust the `\seriesdefault`.

On the other hand if the document font is still CM or LM then `\bfdefault` is wrong, because it is now saying `b` and not `bx` as it should for such fonts.

To fix all this we first run `\reset@font` (the internal kernel name for `\normalfont`). This will set up the document encoding, family, series and shape based on the current values of `\encodingdefault`, `\familydefault`, `\seriesdefault` and `\shapedefault`. However, if the family (from `\familydefault`) has special medium default we should switch to that (and not use what is current value from `\seriesdefault`). This can be achieved by afterwards calling `\mediumseries` and then changing `\seriesdefault` to the now current series value (in `\f@series`).

But what should happen if `\seriesdefault` got explicitly changed? In that case the explicit change should survive and we should not alter `\seriesdefault`. This is solved by comparing the current value of `\seriesdefault` with a kernel version saved in the format and if they differ we do not call `\mdseries` or change `\seriesdefault`.

```
  \reset@font
  \ifx\seriesdefault\seriesdefault@kernel
    \mdseries
    \let\seriesdefault\f@series
  \fi
}%
```

(*End of definition for* `\init@series@setup`.)

As the kernel code now implements the same functionality as mweights, albeit internally coded slightly differently, that package shouldn't be loaded any more. We therefore pretend that it already got loaded. Thus, a font package that tries to load it and then sets `\mdseries@..`, etc. will continue to work but will now use the kernel code.

Of course, mid-term such package should probably use `\DeclareFontSeriesDefault` instead of making using low-level definitions.

```
\expandafter\let\csname ver@mweights.sty\endcsname\fmtversion
```

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\init@series@setup}{Custom series}%
```

This is always called in `\document` so don't make it undefined.

```
⟨latexrelease⟩\let\init@series@setup\relax
⟨latexrelease⟩
⟨latexrelease⟩\expandafter\let\csname ver@mweights.sty\endcsname\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2021/11/15}%
⟨latexrelease⟩                    {\bfseries}{Custom series with hooks}%
```

`\bfseries` This document command switches to the bold series.

```
\DeclareRobustCommand\bfseries{%
  \not@math@alphabet\bfseries\mathbf
```

In the original NFSS definition it then called `\fontseries` with the value `\bfdefault`. In the new scheme we have more alternatives and therefore check if the current family (`\f@family`) is the current `\rmdef@ult`, `\sfdef@ult` or `\ttdef@ult` and the select the correct family default in that case.

```
\expand@font@defaults
\maybe@update@bfseries@defaults
\ifx\f@family\rmdef@ult      \fontseries\bfseries@rm
\else\ifx\f@family\sfdef@ult \fontseries\bfseries@sf
\else\ifx\f@family\ttdef@ult \fontseries\bfseries@tt
```

If not `\bfdefault` is used.

```
\else                        \fontseries\bfdefault
\fi\fi\fi
```

This hook in contrast is always executed.

```
\UseHook{bfseries}%
\selectfont
}
```

(*End of definition for* `\bfseries`.)

`\maybe@update@bfseries@defaults` If `\bfdefault` and `\bfdefault@previous` are different then the default got changed directly through the legacy interface (i.e., via `\def` or `\renewcommand`). In that case we reset all meta family defaults so that the document behaves like it was the case before the new mechanism was introduced.

```
\def\maybe@update@bfseries@defaults{%
  \ifx\bfdefault\bfdefault@previous\else
```

We add `\@empty` and then let `\bfdefault@previous` to `\bfdefault` so that we can detect any further change.

```
    \expandafter\def\expandafter\bfdefault
                    \expandafter{\bfdefault\@empty}%
    \let\bfdefault@previous\bfdefault
```

And we reset the meta family defaults (`\bfdef@ult` is an expanded version of `\bfdefault`).

```
    \let\bfseries@rm\bfdef@ult
    \let\bfseries@sf\bfdef@ult
    \let\bfseries@tt\bfdef@ult
```

Formats that set up parallel fonts, e.g., for Japanese, can use this hook to add resets here. Not that this hook is only run when resets are necessary.

```
    \UseHook{bfseries/defaults}%
  \fi
}
```

(*End of definition for* `\maybe@update@bfseries@defaults`.)

`\mdseries` This document command switches to the medium series.

```
\DeclareRobustCommand\mdseries{%
  \not@math@alphabet\mdseries\relax
  \expand@font@defaults
  \maybe@update@mdseries@defaults
  \ifx\f@family\rmdef@ult      \fontseries\mdseries@rm
    \else\ifx\f@family\sfdef@ult \fontseries\mdseries@sf
    \else\ifx\f@family\ttdef@ult \fontseries\mdseries@tt
```

```
    \else                           \fontseries\mddefault
  \fi\fi\fi
  \UseHook{mdseries}%
  \selectfont
}
```

(*End of definition for* `\mdseries`.)

`\maybe@update@mdseries@defaults`

```
\def\maybe@update@mdseries@defaults{%
  \ifx\mddefault\mddefault@previous\else
    \expandafter\def\expandafter\mddefault\expandafter{\mddefault\@empty}%
    \let\mddefault@previous\mddefault
    \let\mdseries@rm\mddef@ult
    \let\mdseries@sf\mddef@ult
    \let\mdseries@tt\mddef@ult
```

Formats that set up parallel fonts, e.g., for Japanese, can use this hook to add resets here.

```
    \UseHook{mdseries/defaults}%
  \fi
}
```

(*End of definition for* `\maybe@update@mdseries@defaults`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\bfseries}{Custom series with hooks}%
⟨latexrelease⟩
⟨latexrelease⟩\let\maybe@update@bfseries@defaults\@undefined
⟨latexrelease⟩\let\maybe@update@mdseries@defaults\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\bfseries{%
⟨latexrelease⟩  \not@math@alphabet\bfseries\mathbf
⟨latexrelease⟩  \expand@font@defaults
⟨latexrelease⟩  \ifx\bfdefault\bfdefault@previous\else
⟨latexrelease⟩    \expandafter\def\expandafter\bfdefault
⟨latexrelease⟩                    \expandafter{\bfdefault\@empty}%
⟨latexrelease⟩    \let\bfdefault@previous\bfdefault
⟨latexrelease⟩    \let\bfseries@rm\bfdef@ult
⟨latexrelease⟩    \let\bfseries@sf\bfdef@ult
⟨latexrelease⟩    \let\bfseries@tt\bfdef@ult
⟨latexrelease⟩    \UseHook{bfseries/defaults}%
⟨latexrelease⟩  \fi
⟨latexrelease⟩    \ifx\f@family\rmdef@ult      \fontseries\bfseries@rm
⟨latexrelease⟩    \else\ifx\f@family\sfdef@ult \fontseries\bfseries@sf
⟨latexrelease⟩    \else\ifx\f@family\ttdef@ult \fontseries\bfseries@tt
⟨latexrelease⟩    \else                        \fontseries\bfdefault
⟨latexrelease⟩    \fi\fi\fi
⟨latexrelease⟩  \UseHook{bfseries}%
⟨latexrelease⟩  \selectfont
⟨latexrelease⟩}
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\mdseries{%
⟨latexrelease⟩  \not@math@alphabet\mdseries\relax
```

```
⟨latexrelease⟩  \expand@font@defaults
⟨latexrelease⟩  \ifx\mddefault\mddefault@previous\else
⟨latexrelease⟩    \expandafter\def\expandafter\mddefault\expandafter{\mddefault\@empty}%
⟨latexrelease⟩    \let\mddefault@previous\mddefault
⟨latexrelease⟩    \let\mdseries@rm\mddef@ult
⟨latexrelease⟩    \let\mdseries@sf\mddef@ult
⟨latexrelease⟩    \let\mdseries@tt\mddef@ult
⟨latexrelease⟩    \UseHook{mdseries/defaults}%
⟨latexrelease⟩  \fi
⟨latexrelease⟩    \ifx\f@family\rmdef@ult      \fontseries\mdseries@rm
⟨latexrelease⟩    \else\ifx\f@family\sfdef@ult \fontseries\mdseries@sf
⟨latexrelease⟩    \else\ifx\f@family\ttdef@ult \fontseries\mdseries@tt
⟨latexrelease⟩    \else                        \fontseries\mddefault
⟨latexrelease⟩    \fi\fi\fi
⟨latexrelease⟩  \UseHook{mdseries}%
⟨latexrelease⟩  \selectfont
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩                 {\bfseries}{Custom series with hooks}%
⟨latexrelease⟩
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\bfseries{%
⟨latexrelease⟩  \not@math@alphabet\bfseries\mathbf
⟨latexrelease⟩  \expand@font@defaults
⟨latexrelease⟩    \ifx\f@family\rmdef@ult      \fontseries\bfseries@rm
⟨latexrelease⟩    \else\ifx\f@family\sfdef@ult \fontseries\bfseries@sf
⟨latexrelease⟩    \else\ifx\f@family\ttdef@ult \fontseries\bfseries@tt
⟨latexrelease⟩    \else                        \fontseries\bfdefault
⟨latexrelease⟩    \fi\fi\fi
⟨latexrelease⟩  \selectfont
⟨latexrelease⟩}
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\mdseries{%
⟨latexrelease⟩  \not@math@alphabet\mdseries\relax
⟨latexrelease⟩  \expand@font@defaults
⟨latexrelease⟩    \ifx\f@family\rmdef@ult      \fontseries\mdseries@rm
⟨latexrelease⟩    \else\ifx\f@family\sfdef@ult \fontseries\mdseries@sf
⟨latexrelease⟩    \else\ifx\f@family\ttdef@ult \fontseries\mdseries@tt
⟨latexrelease⟩    \else                        \fontseries\mddefault
⟨latexrelease⟩    \fi\fi\fi
⟨latexrelease⟩  \selectfont
⟨latexrelease⟩}
⟨latexrelease⟩
⟨latexrelease⟩
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                 {\bfseries}{Custom series with hooks}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\bfseries
⟨latexrelease⟩        {\not@math@alphabet\bfseries\mathbf
⟨latexrelease⟩         \fontseries\bfdefault\selectfont}
⟨latexrelease⟩\DeclareRobustCommand\mdseries
```

```
⟨latexrelease⟩          {\not@math@alphabet\mdseries\relax
⟨latexrelease⟩           \fontseries\mddefault\selectfont}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                    {\expand@font@defaults}{Custom series with hooks}%
```

\expand@font@defaults
\rm@def@ult
\sf@def@ult
\tt@def@ult
\md@def@ult
\bf@def@ult

The family specific defaults are fully expanded, i.e., they are defined via \edef inside \DeclareFontSeriesDefault. However, the overall defaults, e.g., \bfdefault may have been redefined by the user and thus may not be fully expanded. So to enable reliable comparison we make expanded versions of them. That we rerun each time. The alternative would be to only allow for changes before begin document.

```
\def\expand@font@defaults{%
  \edef\rmdef@ult{\rmdefault}%
  \edef\sfdef@ult{\sfdefault}%
  \edef\ttdef@ult{\ttdefault}%
```

The series defaults may contain some surplus m that we need to drop here.

```
  \series@maybe@drop@one@m\bfdefault\bfdef@ult
  \series@maybe@drop@one@m\mddefault\mddef@ult
```

Formats that set up parallel fonts, e.g., for Japanese, can use this hook to add additional code here.

```
  \UseHook{expand@font@defaults}%
}
```

(*End of definition for* `\expand@font@defaults` *and others.*)

\rmfamily

Here are the document level commands for changing the main font families, or rather, here is a documented outline of the code, the actual code is then streamlined and somewhat generalized.

```
DeclareRobustCommand\rmfamily{%
 \not@math@alphabet\rmfamily\mathrm
```

If families are changed then we have to do a bit more work. In the original NFSS implementation a family change kept encoding, series shape and size unchanged but now we can't any longer simply reuse the current series value. Instead we may have to change it from one family default to the next.

```
 \expand@font@defaults
```

We have to do the testing while the current family is still unchanged but we have to do the adjustment of the series after it got changed (because the new family might have different sets of shapes available and we certainly don't want to see substitution going on. So we use \target@series@value to hold the target series (if any).

```
 \let\target@series@value\@empty
```

Thus, if the current family is the sans family

```
\ifx\f@family\sfdef@ult
```

and if we using the medium series of the sans family

```
      \ifx\f@series\mdseries@sf
```

then lets switch to the medium series for the serif family

```
                                          \let\target@series@value\mdseries@rm
```

and if we use the bold series of the sans family switch to the bold default of the serif family:

```
      \else\ifx\f@series\bfseries@sf  \let\target@series@value\bfseries@rm
```

However, the sans family may not have any specific defaults set, so we also compare with the overall defaults.

```
      \else\ifx\f@series\mddef@ult    \let\target@series@value\mdseries@rm
      \else\ifx\f@series\bfdef@ult    \let\target@series@value\bfseries@rm
```

If neither test was true we leave the series alone. This way a special manual setting such as `\fontseries{lc}` is not undone if the family changes (of course there may not be any support for it in the new family but then the NFSS substitution kicks in and sorts it out).

```
      \fi\fi\fi\fi
```

We need to do the same if the current family is the typewriter family:

```
\else\ifx\f@family\ttdef@ult
     \ifx\f@series\mdseries@tt       \let\target@series@value\mdseries@rm
     \else\ifx\f@series\bfseries@tt  \let\target@series@value\bfseries@rm
     \else\ifx\f@series\mddef@ult    \let\target@series@value\mdseries@rm
     \else\ifx\f@series\bfdef@ult    \let\target@series@value\bfseries@rm
     \fi\fi\fi\fi
\fi\fi
```

With these preparations for series out of the way we can now change the font family to `\rmdefault`.

```
\fontfamily\rmdefault
```

If `\target@series@value` is still empty there is nothing more to do other than selecting the new family. However, if not then we should update the font series now as well. But there is one further subtle issue. We may not have loaded an `.fd` file for our target font family yet. In the past that was done in `\selectfont` if necessary but since we are now doing all the comparisons in `\fontseries` we need to make sure that the font family specifications are already loaded prior to calling `\fontseries`.

```
\ifx\target@series@value\@empty \else
  \maybe@load@fontshape
```

Updating the series in this case means directly changing `\f@series` to the target value. We don't want to go through `\fontseries` because that would apply the mappings and then `bx + b` would keep `bx` instead of changing to `b` as desired. as

```
   \let\f@series\target@series@value
 \fi
 \selectfont}
```

So now for the real definition: most of the code above gets delegated to a helper command `\prepare@family@series@update` so that the definition becomes again fairly short. In addition we add a hook, mainly for our Japanese friends so that the code can be extended prior to the call to `\selectfont`.

```
\DeclareRobustCommand\rmfamily{%
   \not@math@alphabet\rmfamily\mathrm
```

This holds all the code discussed above, first argument is the meta family, i.e., `rm` in this case, and second argument is the default family name, e.g., `cmr` indirectly accessed via `\rmdefault`. This is calling `\fontfamily` and if necessary `\fontseries` as outline above.

```
   \prepare@family@series@update{rm}\rmdefault
```

Then comes the hook code (by default a no-op) and finally the call to `\selectfont`.

```
   \UseHook{rmfamily}%
   \selectfont}
```

The definitions for `\sffamily` and `\ttfamily` are similar, the differences are only in what font families get checked.

`\sffamily`
`\ttfamily`

```
\DeclareRobustCommand\sffamily{%
   \not@math@alphabet\sffamily\mathsf
   \prepare@family@series@update{sf}\sfdefault
   \UseHook{sffamily}%
   \selectfont}

\DeclareRobustCommand\ttfamily{%
   \not@math@alphabet\ttfamily\mathtt
   \prepare@family@series@update{tt}\ttdefault
   \UseHook{ttfamily}%
   \selectfont}
```

(*End of definition for* `\rmfamily`*,* `\sffamily`*, and* `\ttfamily`*.*)

Declare the hooks used above.

`rmfamily`
`sffamily`
`ttfamily`
`normalfont`
`expand@font@defaults`
`bfseries`
`bfseries/defaults`
`mdseries`
`mdseries/defaults`

```
\NewHook{rmfamily}
\NewHook{sffamily}
\NewHook{ttfamily}
\NewHook{normalfont}
\NewHook{expand@font@defaults}
\NewHook{bfseries}
\NewHook{bfseries/defaults}
\NewHook{mdseries}
\NewHook{mdseries/defaults}
```

(*End of definition for* `rmfamily` *and others.*)

\@rmfamilyhook
\@sffamilyhook
\@ttfamilyhook
\@defaultfamilyhook

These four hooks have legacy versions used in 2020/02/02 so we should support them until they aren't any longer used.

By default the hooks do nothing and in new code they should not any longer be used. Instead use the standard hook system and the hook names `rmfamily`, `sffamily`, `ttfamily`. Instead of `\@defaultfamilyhook` use `normalfont`.

```
\let\@rmfamilyhook\@empty
\let\@sffamilyhook\@empty
\let\@ttfamilyhook\@empty
\let\@defaultfamilyhook\@empty
```

(*End of definition for* `\@rmfamilyhook` *and others.*)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩                  {\expand@font@defaults}{Custom series with hooks}%
⟨latexrelease⟩
⟨latexrelease⟩\def\expand@font@defaults{%
⟨latexrelease⟩   \edef\rmdef@ult{\rmdefault}%
⟨latexrelease⟩   \edef\sfdef@ult{\sfdefault}%
⟨latexrelease⟩   \edef\ttdef@ult{\ttdefault}%
⟨latexrelease⟩   \edef\bfdef@ult{\bfdefault}%
⟨latexrelease⟩   \edef\mddef@ult{\mddefault}%
⟨latexrelease⟩   \edef\famdef@ult{\familydefault}%
⟨latexrelease⟩}
⟨latexrelease⟩
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\rmfamily{%
⟨latexrelease⟩    \not@math@alphabet\rmfamily\mathrm
⟨latexrelease⟩    \prepare@family@series@update{rm}\rmdefault
⟨latexrelease⟩    \@rmfamilyhook
⟨latexrelease⟩    \selectfont}
⟨latexrelease⟩\DeclareRobustCommand\sffamily{%
⟨latexrelease⟩    \not@math@alphabet\sffamily\mathsf
⟨latexrelease⟩    \prepare@family@series@update{sf}\sfdefault
⟨latexrelease⟩    \@sffamilyhook
⟨latexrelease⟩    \selectfont}
⟨latexrelease⟩\DeclareRobustCommand\ttfamily{%
⟨latexrelease⟩    \not@math@alphabet\ttfamily\mathtt
⟨latexrelease⟩    \prepare@family@series@update{tt}\ttdefault
⟨latexrelease⟩    \@ttfamilyhook
⟨latexrelease⟩    \selectfont}
⟨latexrelease⟩\let\@rmfamilyhook\@empty
⟨latexrelease⟩\let\@sffamilyhook\@empty
⟨latexrelease⟩\let\@ttfamilyhook\@empty
⟨latexrelease⟩
```

```
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\expand@font@defaults}{Custom series with hooks}%
⟨latexrelease⟩
⟨latexrelease⟩\let\expand@font@defaults\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\bfseries
```

```
⟨latexrelease⟩         {\not@math@alphabet\bfseries\mathbf
⟨latexrelease⟩          \fontseries\bfdefault\selectfont}
⟨latexrelease⟩\DeclareRobustCommand\mdseries
⟨latexrelease⟩         {\not@math@alphabet\mdseries\relax
⟨latexrelease⟩          \fontseries\mddefault\selectfont}
⟨latexrelease⟩\DeclareRobustCommand\rmfamily
⟨latexrelease⟩         {\not@math@alphabet\rmfamily\mathrm
⟨latexrelease⟩          \fontfamily\rmdefault\selectfont}
⟨latexrelease⟩\DeclareRobustCommand\sffamily
⟨latexrelease⟩         {\not@math@alphabet\sffamily\mathsf
⟨latexrelease⟩          \fontfamily\sfdefault\selectfont}
⟨latexrelease⟩\DeclareRobustCommand\ttfamily
⟨latexrelease⟩         {\not@math@alphabet\ttfamily\mathtt
⟨latexrelease⟩          \fontfamily\ttdefault\selectfont}
⟨latexrelease⟩
⟨latexrelease⟩\let\@rmfamilyhook\@undefined
⟨latexrelease⟩\let\@sffamilyhook\@undefined
⟨latexrelease⟩\let\@ttfamilyhook\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

`\IfFontSeriesContextTF`

With the ability for `\bfseries` or `\mdseries` to be mapped to different NFSS axis values it becomes important to have the ability to determine the current context as we can no longer look at `\f@series` to answer a question such as "am I currently typesetting in a bold typeface?"

This is provided by the test `\IfFontSeriesContextTF`. It takes three arguments:

- The context we try to check (either `bf` for bold or `md` for medium, i.e., the same that can go into the first mandatory argument of `\DeclareFontSeriesDefault`),
- what to do if we are in this context (true case) and
- what to do if we are not (false case).

This allows you to define commands like `\IfBold`, e.g.,

```
\NewDocumentCommand\IfBold{mm}{\IfFontSeriesContextTF{bf}{#1}{#2}}
```

and then do

```
This is \IfBold{bold}{non-bold} text.
```

and get the appropriate result.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                    {\IfFontSeriesContextTF}{Font series context}%
\DeclareRobustCommand\IfFontSeriesContextTF[1]{%
  \expand@font@defaults
```

In the beginning we haven't found the context we are looking for.

```
  \@font@series@contextfalse
```

We store the requested context away for use in the tests.

```
  \def\requested@test@context{#1}%
```

The next definition is there to ensure that get a final match during testing even if the current family is non of the meta families (`rm`, `sf` or `tt`). This will then basically tests if the current font family matches the overall default.

```
\expandafter\edef\csname ??def@ult\endcsname{\f@family}%
```

Then we run through the meta family list (currently containing just the three values) followed by the artificial meta family `??` and test each of them in turn using `\test@font@series@context` as the testing command.

```
\let\@elt\test@font@series@context
    \@meta@family@list
    \@elt{??}%
\let\@elt\relax
```

Following that we evaluate the status of `\if@font@series@context` to determine which of the remaining arguments (true/false case) we have to execute.

```
\if@font@series@context
\expandafter\@firstoftwo
\else
\expandafter\@secondoftwo
\fi
}
```

The T/F variants were introduced in 2025 but to simplify rollback we pretend that this happened at the same time as `\IfFontSeriesContextTF`.

```
\long\def\IfFontSeriesContextT#1{\IfFontSeriesContextTF{#1}{}}
\def\IfFontSeriesContextF{\IfFontSeriesContextTF{}}
```

(*End of definition for* `\IfFontSeriesContextTF`.)

`\test@font@series@context` This tests the context (stored in `\requested@test@context`) and updates the boolean if the right context is found.

```
\def\test@font@series@context#1{%
```

First task is to figure out whether the current family matches `\rmfamily`, `\sffamily`, etc. so in `\reserved@a` we store the value of `\rmdef@ult` (or whatever the given meta family is) and compare that to `\f@family`.

```
\edef\reserved@a{\csname #1def@ult\endcsname}%
\ifx\f@family\reserved@a
```

If they match we have found the right meta family so we don't need to test any of the remaining meta family and therefore change `\@elt` to `\@gobble`.

```
    \let\@elt\@gobble
```

Now we have to test if `\f@series` matches the requested context (e.g., whether `\bfseries@rm` has that value if the current meta family is `rm` and we are looking for the `bf` context).

```
    \expandafter\ifx
                \csname\requested@test@context series@#1\endcsname\f@series
```

If yes we change the boolean and are done.

```
      \@font@series@contexttrue
```

If not then maybe the reason is that nothing special was set up for that meta family so we also check now check if `\f@series` matches the overall default (e.g., `\bfdef@ult` if we are looking for the bold context). If that matches we change the boolean.

```
    \else
      \expandafter\ifx
                  \csname\requested@test@context def@ult\endcsname\f@series
        \@font@series@contexttrue
  \fi\fi\fi
}
```

(*End of definition for* `\test@font@series@context`.)

`\if@font@series@context` The boolean to signal if we found the requested font series context.

```
\newif\if@font@series@context
```

(*End of definition for* `\if@font@series@context`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\IfFontSeriesContextTF}{Font series context}%
⟨latexrelease⟩
⟨latexrelease⟩\let\IfFontSeriesContextTF\@undefined
⟨latexrelease⟩\let\IfFontSeriesContextT\@undefined
⟨latexrelease⟩\let\IfFontSeriesContextF\@undefined
⟨latexrelease⟩\let\test@font@series@context\@undefined
⟨latexrelease⟩\let\if@font@series@context\@undefined
⟨latexrelease⟩\let\@font@series@contexttrue\@undefined
⟨latexrelease⟩\let\@font@series@contextfalse\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

3 Supporting nested emphasis

By default LaTeX 2$_\varepsilon$ supports two levels of nested emphasis: if the current font has an upright shape then it switches to `\itshape` otherwise to `\eminnershape` (which defaults to `\upshape`). This means nested emphasis will oscillate between italic and upright shapes.

Sometimes it would be nice to allow for a more lengthy sequence, but instead of providing a fixed one LaTeX now offers a general mechanism that allows to define arbitrary sequences.

`\DeclareEmphSequence`

This declaration expects a comma separated list of (font) change declarations corresponding to increasing levels of emphasis. The mechanism tries to be "smart" and verifies that the declarations actually alter the font. If not it will ignore this level and tries the next one—the assumption being that there was a manual font change in the document to the font that is now supposed to be used for emphasis. Of course, this only works if the declarations in the list actually change the font and not, say, just the color. In such a case one has to use `\emforce` to which directs the mechanism to use the level even if the font attributes haven't changed.

`\emforce`

`\emreset` If the nesting is so deep, that the specified levels are exhausted then `\emreset` is

used as a final set of declarations (which by default returns back to the upright shape). Any additional nesting levels will then reuse the list from its beginning.

\strongemph

In addition to support for \emph we now also offer \strongemph (by default boldening the text font. Like with \emph there are declarations that allow you adjust used fonts.

\DeclareStrongEmphSequence

\strongemforce

This declaration expects a comma separated list of (font) change declarations corresponding to increasing levels of "strong emphasis". It works like in a same manner as \DeclareEmphSequence and holds the sequence of font changes for \strongemph. The command \strongemforce forces a given entry to be used even if it doesn't seem to alter the font.

\strongemreset

If the nesting is so deep, that the specified levels are exhausted then \strongemreset is used as a final set of declarations (which by default returns back to bold series and the upright shape). Any additional nesting levels will then reuse the list from its beginning.

\DeclareEmphSequence

\DeclareEmphSequence expects a clist of declaration. Spaces in the argument are dropped to avoid spurious spaces in the output. The declarations are additive. At the very end the shape is reset using \emreset and \emforce so that this case is never skipped.[38] Further nested calls restart at the beginning.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩                    {\DeclareEmphSequence}{Nested emph}%
\def\DeclareEmphSequence#1{%
  \protected@edef\emfontdeclare@clist{\zap@space#1, \@empty\emforce\emreset}%
}
```

By default the it is empty, in which case \eminnershape is used by LaTeX.

```
\let\emfontdeclare@clist\@empty
```

(*End of definition for* \DeclareEmphSequence.)

\emrest

Reset the font to upright and upper/lower case. With the default rules using \shapedefault does that for us but to be on the safe side we do it like this:

```
\DeclareRobustCommand\emreset{\upshape\ulcshape}
```

(*End of definition for* \emrest.)

\em

The new definition for \em (and implicitly \emph) is the same as before as long as \emfontdeclare@clist is empty.

```
\DeclareRobustCommand\em{%
  \@nomath\em
  \ifx\emfontdeclare@clist\@empty
    \ifdim \fontdimen\@ne\font >\z@
      \eminnershape \else \itshape \fi
  \else
```

[38] Maybe we should not add \emforce but allow that case to be skipped as well. Of course, that might result in an endless loop if somebody defines a sequence without any font change and without \emforce but ...

But if not we use the list to decide how to do emphasis.

We use the current font to check if the declarations have any effect, so even a size change is allowed and identified as a modification (but a color change, for example, isn't). So first we save the current status.

```
  \edef\em@currfont{\csname\curr@fontshape/\f@size\endcsname}%
```

Then we grab the next element from the list and check if it can be used.

```
    \expandafter\do@emfont@update\emfontdeclare@clist\do@emfont@update
  \fi
}
\def\eminnershape{\upshape}
```

(*End of definition for* `\em`.)

`\do@emfont@update` We know that the list (if not empty) has at least 2 elements separated by a comma, so we pick up the first in `#1` and the rest in `#2`.

```
\def\do@emfont@update#1,#2\do@emfont@update{%
```

First action is to alter the list and move the first entry to the end

```
  \def\emfontdeclare@clist{#2,#1}%
```

Then we execute the current declaration. Appending `\selectfont` means one can write just `\fontshape{it}` and that works then too.

```
%  \typeout{Use: \detokenize{#1}}%
  #1\selectfont
```

We then compare the current font with our saved version, but with a slight twist: we add `\em@force` at the end of the name. Normally this is empty so has no effect but if there was an `\emforce` as part of `#1` it will append a `/` to the font name (making it invalid) thus this will then always fail the test.

If the test fails we are done and the declarations will be used. Otherwise we will try the next declaration in the sequence.

```
  \expandafter\ifx\csname \curr@fontshape/\f@size\em@force
```

For the comparison with `\ifx` we have to expand `\em@currfont` once as the relevant info is inside.

```
                              \expandafter\endcsname
                    \em@currfont
  \expandafter\do@emfont@update\emfontdeclare@clist\do@emfont@update
```

If `\emforce` was used, we have to undo its effect:

```
  \else
    \let\em@force\@empty
  \fi
}
```

(*End of definition for* `\do@emfont@update`.)

`\emforce` `\em@force` The definition of `\emforce` is simple: change `\em@force` to make the above test always invalid.

```
\protected\def\emforce{\def\em@force{/}}
\let\em@force\@empty
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\emforce` *and* `\em@force`.)

`\em` `\eminnershape` These are the older definitions for `\em`, prior to 2020.

We also have to define the *emphasize* font change command (i.e. `\em`). This command will look is the current font is sloped (i.e. has a positive `\fontdimen1`) and will then select either `\upshape` or `\itshape`.

```
⟨latexrelease⟩\IncludeInRelease{2015/01/01}{\DeclareEmphSequence}{Nested emph}%
⟨latexrelease⟩\let\DeclareEmphSequence\@undefined
⟨latexrelease⟩\let\emfontdeclare@clist\@undefined
⟨latexrelease⟩\let\emreset\@undefined
⟨latexrelease⟩\let\do@emfont@update\@undefined
⟨latexrelease⟩\let\emforce\@undefined
⟨latexrelease⟩\let\em@force\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\em
⟨latexrelease⟩        {\@nomath\em \ifdim \fontdimen\@ne\font >\z@
⟨latexrelease⟩                        \eminnershape \else \itshape \fi}%
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\DeclareEmphSequence}{Nested emph}%
⟨latexrelease⟩\DeclareRobustCommand\em
⟨latexrelease⟩        {\@nomath\em \ifdim \fontdimen\@ne\font >\z@
⟨latexrelease⟩                        \upshape \else \itshape \fi}%
⟨latexrelease⟩\let\eminnershape\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\em` *and* `\eminnershape`.)

4 Supporting `\strongemph`

```
\def\DeclareStrongEmphSequence#1{%
  \protected@edef\strongfontdeclare@clist{\zap@space#1, \@empty\strongemforce\strongemreset}%
}
\let\strongfontdeclare@clist\@empty
\DeclareRobustCommand\strongem{%
  \@nomath\strongem
  \edef\strong@currfont{\csname\curr@fontshape/\f@size\endcsname}%
  \expandafter\do@strongfont@update\strongfontdeclare@clist\do@strongfont@update
}

\def\do@strongfont@update#1,#2\do@strongfont@update{%
  \def\strongfontdeclare@clist{#2,#1}%
  #1\selectfont
  \expandafter\ifx\csname \curr@fontshape/\f@size\strongem@force
                          \expandafter\endcsname
                \strong@currfont
  \expandafter\do@strongfont@update\strongfontdeclare@clist\do@strongfont@update
  \else
    \let\strongem@force\@empty
  \fi
}

```

```
\protected\def\strongemforce{\def\strongem@force{/}}
\let\strongem@force\@empty

```

The command `\strongemph` is defined in `ltfntcmd.dtx`, before then some of the necessary support commands aren't defined.

```
%\DeclareTextFontCommand{\strongemph}{\strongem}
```

Initialization needs delaying too as some of the necessary macros are defined late. This is done in `ltfinal.dtx`.

```
%\DeclareRobustCommand\strongemreset{\bfseries\upshape\ulcshape}
%\DeclareStrongEmphSequence{\bfseries,\itshape}
```

`\not@math@alphabet` This function generates an error message when it is called in math mode. The same function should be defined in `newlfont.sty`.

```
\def\not@math@alphabet#1#2{%
   \relax
   \ifmmode
     \@latex@error{Command \noexpand#1invalid in math mode}%
        {%
         Please
         \ifx#2\relax
            define a new math alphabet^^J%
            if you want to use a special font in math mode%
          \else
```

We have to a `\noexpand` below to prevent expansion of `#2`. In case of `#1` we can omit this (due to the current definition of robust commands since they do come out right there :-).

```
            use the math alphabet \noexpand#2instead of
            the #1command%
         \fi
         .
        }%
   \fi}
```

(*End of definition for* `\not@math@alphabet`.)

Finally we provide two abbreviations to switch to the LaTeX *versions*.

```
\DeclareRobustCommand\boldmath{\@nomath\boldmath
              \mathversion{bold}}
\DeclareRobustCommand\unboldmath{\@nomath\unboldmath
              \mathversion{normal}}
```

Here we switch to the default math version by defining the internal macro `\math@version`. We dare not to call `\mathversion` at this place because this would call `\glb@settings`.

```
\def\math@version{normal}
```

4.1 Legacy

We start by defining a few macros that are part of standard LaTeX's user interface. The use of these functions is not encouraged, but they will allow to process older documents without changes to the source.

`\newfont`

```
\def\newfont#1#2{\@ifdefinable#1{\font#1=#2\relax}}
```

(*End of definition for* `\newfont`.)

\symbol

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                    {\symbol}{XeTeX change for math}%
\ifdefined\XeTeXversion
  \DeclareRobustCommand\symbol[1]{\Ucharcat#1 12\relax}
\else
  \DeclareRobustCommand\symbol[1]{\char#1\relax}
\fi
⟨/2ekernel | latexrelease⟩

⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\symbol}{XeTeX change for math}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\symbol[1]{\char#1\relax}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\symbol`.)

4.2 Miscellaneous

`\@setfontsize` `\@setsize`

This abbreviation is used by LaTeX's user level size changing commands, such as `\large`.

```
\def\@setfontsize#1#2#3{\@nomath#1%
```

For the benefit of people relying on keeping the name of the current font command saved in `\@currsize` we define it. To ensure that `\@setfontsize` keeps being robust we omit this assignment during times where `\protect` differs from `\@typeset@protect`.

```
    \ifx\protect\@typeset@protect
      \let\@currsize#1%
    \fi
    \fontsize{#2}{#3}\selectfont}
```

For compatibility we also define `\@setsize` the 209 command

```
⟨*compat⟩
\def\@setsize#1#2#3#4{\@setfontsize#1{#4}{#2}}
⟨/compat⟩
```

(*End of definition for* `\@setfontsize` *and* `\@setsize`.)

`\hexnumber@`

To set up LaTeX's special math character definitions we first provide a macro to generate hexadecimal numbers. It is a rather simple `\ifcase`.

```
\def\hexnumber@#1{\ifcase\number#1
 0\or 1\or 2\or 3\or 4\or 5\or 6\or 7\or 8\or
 9\or A\or B\or C\or D\or E\or F\fi}
```

(*End of definition for* `\hexnumber@`.)

`\nfss@text`

In its simplest form `\nfss@text` is an `\mbox`. This will produce unbreakable text outside math and inside math you will get text with the same fonts as outside. The only drawback is that such item won't change sizes in subscripts. But this behavior can be easily changed.

With the amstex style option one will get a sub style called amstext which will redefine the \nfss@text macro to produce correct text in all sizes.

We have to use \def instead of the shorter \let since \mbox is undefined when we reach this point.

```
\def\nfss@text#1{{\mbox{#1}}}
```

(*End of definition for* \nfss@text.)

\copyright The definition of \copyright was changed so that it works in other type styles, and to make it robust. We leave the family untouched so that the copyright notice will come out differently if a different font family is in use. This command is commented out, since it is now defined in ltoutenc.dtx.

```
%\DeclareRobustCommand\copyright
%    {{\ooalign{\hfil
%     \raise.07ex\hbox{\mdseries\upshape c}\hfil\crcr
%     \mathhexbox20D}}}
```

(*End of definition for* \copyright.)

\normalfont \reset@font The macro \reset@font is used in LaTeX to switch to a standard font, in order to initialize the current font in situations where typesetting is done in a new visual context (e.g. in a footnote). We define it here to allow the test for the new LaTeX version above but nevertheless are able to run all kind of mixtures.

The user interface name for \reset@font is \normalfont:

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/11/01}%
⟨latexrelease⟩                    {\normalfont}{Support meta family}%
\DeclareRobustCommand\normalfont{%
```

Instead of calling \usefont, as it was done in the past, we inline the code from \usefont as we want to add the hook before \selectfont, but after all the font attributes are set.

```
    \fontencoding\encodingdefault
    \edef\f@family{\familydefault}%
    \edef\f@series{\seriesdefault}%
    \edef\f@shape{\shapedefault}%
```

Now that \f@family is set we can update the meta family macro.

```
    \set@current@meta@family
```

Any earlier \fontseries, etc. should be canceled and we should switch unconditionally to the requested font face so we drop any code that may have been stored in \delayed@f@adjustment.

```
    \let\delayed@f@adjustment\@empty

    \UseHook{normalfont}%
    \selectfont}
\let\reset@font\normalfont
```

(*End of definition for* \normalfont *and* \reset@font.)

\set@current@meta@family This macros determines the current meta family by comparing the current family in \f@family with the various document font defaults, e.g., \rmdefault.

```
\def\set@current@meta@family {%
```

Because these defaults can be set by the user and may contain the family name indirectly, we first expand them to ensure we do not get false negatives.

```
  \edef\rmdef@ult{\rmdefault}%
  \ifx\f@family\rmdef@ult
    \def\@currentmetafamily{rm}%
  \else
    \ifx\f@family\sfdef@ult
      \def\@currentmetafamily{sf}%
    \else
      \ifx\f@family\ttdef@ult
        \def\@currentmetafamily{tt}%
      \else
        \def\@currentmetafamily{??}%
      \fi
    \fi
  \fi
}
```

(*End of definition for* `\set@current@meta@family`.)

```
% \changes{v3.2g}{2021/03/18}
%         {Add missing 2020/02/02 latexrelease entry.}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩                  {\normalfont}{Cancel delayed actions}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\normalfont{%
⟨latexrelease⟩    \fontencoding\encodingdefault
⟨latexrelease⟩    \edef\f@family{\familydefault}%
⟨latexrelease⟩    \edef\f@series{\seriesdefault}%
⟨latexrelease⟩    \edef\f@shape{\shapedefault}%
⟨latexrelease⟩    \let\delayed@f@adjustment\@empty
⟨latexrelease⟩    \UseHook{normalfont}%
⟨latexrelease⟩    \@defaultfamilyhook        % hookname from 2020/02 will vanish
⟨latexrelease⟩    \selectfont}
⟨latexrelease⟩\let\reset@font\normalfont
⟨latexrelease⟩
⟨latexrelease⟩\let\set@current@meta@family\undefined
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\normalfont}{Add hook to \normalfont}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\normalfont{%
⟨latexrelease⟩    \fontencoding\encodingdefault
⟨latexrelease⟩    \edef\f@family{\familydefault}%
⟨latexrelease⟩    \edef\f@series{\seriesdefault}%
⟨latexrelease⟩    \edef\f@shape{\shapedefault}%
⟨latexrelease⟩    \UseHook{normalfont}%
⟨latexrelease⟩    \@defaultfamilyhook        % hookname from 2020/02 will vanish
⟨latexrelease⟩    \selectfont}
⟨latexrelease⟩
⟨latexrelease⟩\let\reset@font\normalfont
⟨latexrelease⟩
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩                  {\normalfont}{Add hook to \normalfont}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\normalfont{%
⟨latexrelease⟩    \fontencoding\encodingdefault
⟨latexrelease⟩    \edef\f@family{\familydefault}%
⟨latexrelease⟩    \edef\f@series{\seriesdefault}%
⟨latexrelease⟩    \edef\f@shape{\shapedefault}%
⟨latexrelease⟩    \@defaultfamilyhook
⟨latexrelease⟩    \selectfont}
⟨latexrelease⟩
⟨latexrelease⟩\let\reset@font\normalfont
⟨latexrelease⟩
⟨latexrelease⟩\let\@defaultfamilyhook\@empty
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\normalfont}{Add hook to \normalfont}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\normalfont
⟨latexrelease⟩          {\usefont\encodingdefault
⟨latexrelease⟩                   \familydefault
⟨latexrelease⟩                   \seriesdefault
⟨latexrelease⟩                   \shapedefault
⟨latexrelease⟩                   \relax}
⟨latexrelease⟩\let\reset@font\normalfont
⟨latexrelease⟩
⟨latexrelease⟩\let\@defaultfamilyhook\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

`\fontfamily` One place where the current meta family should be set is `\fontfamily`, which is why we defined it here after the definition of `\set@current@meta@family` in this file.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/11/01}%
⟨latexrelease⟩                  {\fontfamily}{Set meta family}%
\DeclareRobustCommand\fontfamily[1]
      {\edef\f@family{#1}\set@current@meta@family}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\fontfamily}{Set meta family}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\fontfamily[1]{\edef\f@family{#1}}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\fontfamily`.)

We left out the special LaTeX fonts which are not automatically included in the base version of the font selection since these fonts contain only a few characters which are also included in the AMS fonts so anybody who is using these fonts doesn't need them. But for compatibility reasons we will define these symbols.

```
\def\not@base#1{\@latex@error
  {Command \noexpand#1not provided in base LaTeX2e}%
  {Load the latexsym or the amsfonts package to
   define this symbol}}
\def\mho{\not@base\mho}
\def\Join{\not@base\Join}
\def\Box{\not@base\Box}
\def\Diamond{\not@base\Diamond}
\def\leadsto{\not@base\leadsto}
\def\sqsubset{\not@base\sqsubset}
\def\sqsupset{\not@base\sqsupset}
\def\lhd{\not@base\lhd}
\def\unlhd{\not@base\unlhd}
\def\rhd{\not@base\rhd}
\def\unrhd{\not@base\unrhd}
```

We now initialize all variables set by `\DeclareErrorFont`. These values are not really important since they will be overwritten later on by the definition in `fontdef.ltx`.

However, if `fontdef.cfg` is corrupted then at least a hopefully suitable error font is present.

```
\DeclareErrorFont{OT1}{cmr}{m}{n}{10}  %% don't modify this setting
                                       %% overwrite it in fontdef.cfg
                                       %% if necessary
```

We also set some default values for `\f@family` etc. Note that we don't yet have any encodings that comes later. In the past this was implicitly done by `\DeclareErrorFont`.

```
\def\f@family{cmr}        % can't use \fontfamily at this point as
                          % defaults aren't set up yet
```

Previously the default values for series and shape were set by calling `\fontseries` and `\fontshape`, but their action is now delayed until `\selectfont` which isn't called inside the format (to avoid unnecessarily loading a font that may never get used). We therefore have to set `\f@series` and `\f@shape` directly instead.

```
\def\f@series{m}           % \fontseries{m}
\def\f@shape{n}            % \fontshape{n}

\fontsize{10}{10}
```

The initial `fontenc` package load list. This will get overwritten in `fonttext` and is only provided in case an old `fonttext.cfg` does not define the command:

```
\def\@fontenc@load@list{\@elt{T1,OT1}}
```

We now load the customizable parts of NFSS.

```
\InputIfFileExists{fonttext.cfg}
           {\typeout{=====================================^^J%
                     ^^J%
                      Local config file fonttext.cfg used^^J%
                     ^^J%
                     =====================================}%
```

```
              \def\@addtofilelist##1{\xdef\@filelist{\@filelist,##1}}%
             }
            {\input{fonttext.ltx}}
\let\@addtofilelist\@gobble
```

Ditto for math although I don't think that we will get a lot of customization :-)

```
\InputIfFileExists{fontmath.cfg}
            {\typeout{=====================================^^J%
                      ^^J%
                       Local config file fontmath.cfg used^^J%
                      ^^J%
                      =====================================}%
              \def\@addtofilelist##1{\xdef\@filelist{\@filelist,##1}}%
             }
            {\input{fontmath.ltx}}
\let\@addtofilelist\@gobble
```

Then we preload several fonts. This file might be customized *without* changing the behavior of the format (i.e. necessary font definitions will be loaded at runtime if they are not preloaded). This is done in the file `preload.ltx`.

```
\InputIfFileExists{preload.cfg}
            {\typeout{=====================================^^J%
                      ^^J%
                       Local config file preload.cfg used^^J%
                      ^^J%
                      =====================================}%
              \def\@addtofilelist##1{\xdef\@filelist{\@filelist,##1}}%
             }
            {\input{preload.ltx}}
\let\@addtofilelist\@gobble
```

`\seriesdefault` `\seriesdefault@kernel`

After `\seriesdefault` got defined inside `fonttext.ltx` or a `.cfg` file overwriting it, we alter its value by appending `\@empty` to it. This will vanish if expanded but allows us to check if the default gets altered (even to the same value) in the document preamble. All we have to do is to save the current value somewhere and later compare the two. For this we use `\seriesdefault@kernel`.

```
\expandafter\def\expandafter\seriesdefault\expandafter{\seriesdefault\@empty}
\let\seriesdefault@kernel\seriesdefault
```

(*End of definition for* `\seriesdefault` *and* `\seriesdefault@kernel`.)

`\@acci` `\@accii` `\@acciii`

We also save the values of some accents in `\@acci`, `\@accii` and `\@acciii` so they can be restored by a `minipage` inside a `tabbing` environment.

```
\let\@acci\' \let\@accii\‘ \let\@acciii\=
```

(*End of definition for* `\@acci`, `\@accii`, *and* `\@acciii`.)

`\cal` `\mit`

Here were the two old ⟨*alphabet identifiers*⟩.

(*End of definition for* `\cal` *and* `\mit`.)

```
⟨/2ekernel⟩
```

File 30
fontdef.dtx

1 Introduction

This file is used to generate the files `fonttext.ltx` (text font declarations) and `fontmath.ltx` (math font declarations), which are used during the format generation. It contains the declaration of the standard text encodings used at the site as well as a minimal subset of font shape groups that NFSS will look at to ensure that the specified encodings are valid.

The math part contains the setup for math encodings as well as the default math symbol declarations that belong to the encoding.

It is possible to change this setup (by using other fonts, or defaults) without losing the ability to process documents written at other sites. Portability in this sense means that a document will compile without errors. It does not mean, however, that identical output will be produced. For this it is necessary that the distributed setup is used at both installations.

2 Customization

You are not allowed to change this source file! If you want to change the default encodings and/or the font shape groups preloaded you should create a copy of `fonttext.ltx` under the name `fonttext.cfg` and change this copy. If LaTeX 2$_\varepsilon$ finds a file of this name it will use it, otherwise it uses the standard file which is `fontdef.ltx`.

If you don't plan to use Computer Modern much or at all, it might (!) be a good idea to make your own `fonttext.cfg`. Look at the comments below (docstrip module 'text') to see what should go into such a file.

To change the math font setup use a copy of `fontmath.ltx` under the name `fontmath.cfg` and change this copy. However, dealing with this interface is even more a job for an expert than changing the text font setup — in short, we don't encourage either.

> **Warning:** please note that we don't support customised LaTeX versions. Thus, before sending in a bug report please try your test file with a LaTeX format which is not customised and send in the log from that version (unless the problem goes away).

Please note: the following standard encodings have to be defined in all local variants of `font....cfg` to guarantee that all LaTeX installations behave in the same way.

`T1`	Cork TeX text encoding
`OT1`	old TeX text encoding
`U`	unknown encoding
`OML`	old TeX math letters encoding
`OMS`	old TeX math symbols encoding
`OMX`	old TeX math extension symbols encoding
`TU`	Unicode

Notice that some of these encodings are 'old' in the sense that we hope that they will be superseded soon by encoding standards defined by the TEX user community. Therefore this set of default encodings may change in the future.

The first candidate is `OT1` which will soon be replaced by `T1`, the official TEX text encoding.

> **Warning:** If you add additional encodings to this file there is no guarantee any longer that files processable at your installation will also be processable at other installations. Thus, if you make use of such an encoding in your document, e.g. if you intend to typeset in Cyrillic (`OT2` encoding), you need to specify this encoding in the preamble of your document prior to sending it to another installation. Once the encoding is specified in that place in your document, the document is processable at all LATEX installations (provided they have suitable fonts installed).
>
> For this reason we suggest that you define a short package file that sets up an additional encoding used at your site (rather than putting the encoding into this file) since this package can easily be shipped with your document.

3 The `docstrip` modules

The following modules are used to direct `docstrip` in generating external files:

driver	produce a documentation driver file
text	produce the file `fonttext.ltx`
math	produce the file `fontmath.ltx`
cfgtext	produce a dummy `fonttext.cfg` file
cfgmath	produce a dummy `fontmath.cfg` file

A typical `docstrip` command file would then have entries like:

```
generateFile{fonttext.ltx}{t}{\from{fontdef.dtx}{text}}
```

4 A driver for this document

The next bit of code contains the documentation driver file for TEX, i.e. the file that will produce the documentation you are currently reading. It will be extracted from this file by the DOCSTRIP program.

```
⟨*driver⟩
\documentclass{ltxdoc}
\GetFileInfo{fontdef.dtx}
\begin{document}
   \DocInput{fontdef.dtx}
\end{document}
⟨/driver⟩
```

5 The `fonttext.ltx` file

The identification is done earlier on with a `\ProvidesFile` declaration.

```
⟨*text⟩
\typeout{=== Don't modify this file, use a .cfg file instead ===^^J}
```

5.1 Encodings

This file declares the standard encodings for text and math fonts. All others should be declared in packages or in the documents directly.

For every text encoding there are normally a number of encoding specific commands, e.g. accents, special characters, etc. (The definition for such a command might have to change when the encoding is changed, because the character is in a different position, or not available at all, or the accent is produced in a different way.) This is handled by a general mechanism which is described in `ltoutenc.dtx`.

By convention, text encoding specific declarations, including the `\DeclareFontEncoding` declaration, are kept in separate file of the form ⟨*enc*⟩`enc.def`, e.g. `ot1enc.def`. This allows other applications to make use of the declarations as well.

Similar to the default encoding, the loading of the encoding files for the two major text encodings shouldn't be changed. In particular, the `inputenc` package depends on this.

```
\input {omlenc.def}
\input {omsenc.def}
```

Documents containing a lot of accented characters should really be using T1 fonts. We therefore load this after OT1 so that T1 encoding specific commands are executed as fast as possible (encoding files are no longer reloaded in `fontenc`.

```
\input {ot1enc.def}
\input  {t1enc.def}
\input {ts1enc.def}

\ifx\Umathcode\@undefined
```

We then set the default text font encoding. This will hopefully change some day to `T1`. This setting should *not* be changed to produce a portable format.

```
\fontencoding{OT1}
```

The initial `fontenc` package load list if an 8-bit TEX engine is used:

```
\def\@fontenc@load@list{\@elt{T1,OT1}}

\def\rmsubstdefault{cmr}
\def\sfsubstdefault{cmss}
\def\ttsubstdefault{cmtt}
\LoadFontDefinitionFile{TS1}{cmr}

\else
```

Unicode.

```
\input {tuenc.def}
\fontencoding{TU}
```

The initial `fontenc` package load list if a Unicode engine is used:

```
\def\@fontenc@load@list{\@elt{TU}}

\DeclareFontSubstitution{TU}{lmr}{m}{n}
\LoadFontDefinitionFile{TU}{lmr}
\LoadFontDefinitionFile{TU}{lmss}
\LoadFontDefinitionFile{TU}{lmtt}

\def\rmsubstdefault{lmr}
\def\sfsubstdefault{lmss}
\def\ttsubstdefault{lmtt}
\LoadFontDefinitionFile{TS1}{lmr}
```

```
\DeclareFontSubstitution{TU}{lmr}{m}{n}
```

End of Unicode branch.

```
\fi
```

If different encodings for text fonts are in use one could put the common setup into `\DeclareFontEncodingDefaults`. There is now a better mechanism so using this interface is discouraged!

```
\DeclareFontEncodingDefaults{}{}
```

The default font substitution for an encoding is defined in the corresponding `...enc.def` file so for `OT1`, `T1`, and `TS1` this is already defined.

```
%\DeclareFontSubstitution{T1}{cmr}{m}{n}
%\DeclareFontSubstitution{OT1}{cmr}{m}{n}
%\DeclareFontSubstitution{TS1}{cmr}{m}{n}
```

This release of LaTeX 2_ε assumes that the ec fonts are available. It is possible to change this to point to some other font family (e.g., Times with the appropriate encoding if it is available) without making documents non-portable. However, in such a case documents will produce different page breaks at other sites. The substitution defaults can all be changed without losing portability as long as there are font shape definitions for the selected substitutions.

For every encoding declaration, LaTeX 2_ε will try to verify that the given substitution information makes sense, i.e. that it is impossible to go into an endless loop if font substitution happens. This is done at the moment the `\begin{document}` is encountered. LaTeX 2_ε will then check that for every encoding the substitution defaults form a valid font shape group, which means that it will check if there is a `\DeclareFontShape` declaration for this combination. We will therefore load the corresponding `.fd` files now. If we don't do this they would be loaded at verification time (i.e. at `\begin{document}` which would delay processing unnecessarily.

> **Warning:** Please note that this means that you have to regenerate the format whenever you change any of these `.fd` files since LaTeX 2_ε will not read `.fd` files if it already knows about the encoding/family combination.

The `\nfss@catcodes` ensures that white space is ignored in any definitions made in the fd files.

```
\begingroup
\nfss@catcodes
\input  {t1cmr.fd}
\input  {ot1cmr.fd}
\input  {ts1cmr.fd}
\endgroup
```

We also load some other font definition files which are normally needed in a document. This is only done for processing speed and you can comment the next two lines out to save some memory. If necessary these files are then loaded when your document is processed. (Loading `.fd` files is a less drastic step compared to preloading fonts because the number of fonts is limited 255 at (nearly) every TeX installation, while the amount of main memory is not a limiting factor at most installations.)

```
\begingroup
\nfss@catcodes
\input {t1cmss.fd}
```

```
\input {t1cmtt.fd}
\input {ot1cmss.fd}
\input {ot1cmtt.fd}

\input {ts1cmss.fd}
\input {ts1cmtt.fd}
\endgroup
```

Even though Unicode engines default to `lm` load `ts1cmr` as this may be used for fallback for TS1 encoding.

We now load it in all engines above, so the next lines are no longer necessary. We keep them here if we stop loading other fd files in Unicode engines.

```
%\ifx\Umathcode\@undefined\else
%\begingroup
%\nfss@catcodes
%\input {ts1cmr.fd}
%\endgroup
%\fi
```

Even with all the precautions it is still possible that NFSS will run into problems, for example, when a `.fd` file contains corrupted data. To guard against such cases NFSS has a very low-level fallback font that is installed with the following line.

```
\DeclareErrorFont{OT1}{cmr}{m}{n}{10}
```

This means, "if everything else fails use Computer Modern Roman normal shape at 10pt in the old text encoding". You can change the font used but the encoding should be the same as the one specified with `\fontencoding` above.

5.2 Defaults

To allow the use of `\rmfamily`, `\sffamily`, etc. in documents even if non-standard families are used we provide nine macros which hold the name of the corresponding families, series, and so on. This makes it easy to use other font families (like Times Roman, etc.). One simply has to redefine these defaults.

All these hooks have to be defined in this file but you can change their meaning (except for `\encodingdefault`) without making documents non-portable.

`\encodingdefault` `\rmdefault` `\sfdefault` `\ttdefault`

The following three definitions set up the meaning for `\rmfamily`, `\sffamily`, and `\ttfamily`.

```
\ifx\Umathcode\@undefined
\newcommand\encodingdefault{OT1}
\newcommand\rmdefault{cmr}
\newcommand\sfdefault{cmss}
\newcommand\ttdefault{cmtt}
\else
\newcommand\encodingdefault{TU}
\newcommand\rmdefault{lmr}
\fontfamily{\rmdefault}
\newcommand\sfdefault{lmss}
\newcommand\ttdefault{lmtt}
\fi
⟨/text⟩
⟨latexrelease⟩\IncludeInRelease{2017/01/01}%
⟨latexrelease⟩                  {\encodingdefault}{TU encoding default}%
```

```
⟨latexrelease⟩\ifx\Umathcode\@undefined
⟨latexrelease⟩\renewcommand\encodingdefault{OT1}
⟨latexrelease⟩\fontencoding{\encodingdefault}
⟨latexrelease⟩\renewcommand\rmdefault{cmr}
⟨latexrelease⟩\fontfamily{\rmdefault}
⟨latexrelease⟩\renewcommand\sfdefault{cmss}
⟨latexrelease⟩\renewcommand\ttdefault{cmtt}
⟨latexrelease⟩\else
⟨latexrelease⟩\renewcommand\encodingdefault{TU}
⟨latexrelease⟩%done in everyjob\fontencoding{\encodingdefault}
⟨latexrelease⟩\renewcommand\rmdefault{lmr}
⟨latexrelease⟩\fontfamily{\rmdefault}
⟨latexrelease⟩\renewcommand\sfdefault{lmss}
⟨latexrelease⟩\renewcommand\ttdefault{lmtt}
⟨latexrelease⟩\fi
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                   {\encodingdefault}{TU encoding default}%
⟨latexrelease⟩\fontencoding{OT1}
⟨latexrelease⟩\renewcommand\encodingdefault{OT1}
⟨latexrelease⟩\fontencoding{\encodingdefault}
⟨latexrelease⟩\renewcommand\rmdefault{cmr}
⟨latexrelease⟩\fontfamily{\rmdefault}
⟨latexrelease⟩\renewcommand\sfdefault{cmss}
⟨latexrelease⟩\renewcommand\ttdefault{cmtt}
⟨latexrelease⟩\EndIncludeInRelease
⟨*text⟩
```

(*End of definition for* `\encodingdefault` *and others.*)

`\bfdefault` `\mddefault` Series changing commands are influenced by the following hooks.

```
\newcommand\bfdefault{b}  % overwritten below (for rollback)
\newcommand\mddefault{m}  % overwritten below (for rollback)
```

(*End of definition for* `\bfdefault` *and* `\mddefault`.)

`\itdefault` `\sldefault` `\scdefault` `\updefault` Shape changing commands use the following hooks.

```
\newcommand\itdefault{it}
\newcommand\sldefault{sl}
\newcommand\scdefault{sc}
\newcommand\updefault{up}  % overwritten below (for rollback)
```

(*End of definition for* `\itdefault` *and others.*)

```
⟨/text⟩
⟨*text | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩                   {\updefault}{font defaults change}%
\renewcommand\updefault{up}
```

We append `\@empty` to the series value so that we can detect if it got changed via `\def` or `\renewcommand` later.

```
\renewcommand\bfdefault{b\@empty}
\renewcommand\mddefault{m\@empty}
```

```
\let\bfdefault@previous\bfdefault
\let\mddefault@previous\mddefault
⟨/text | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\updefault}{font defaults change}%
⟨latexrelease⟩
⟨latexrelease⟩\renewcommand\updefault{n}
⟨latexrelease⟩\renewcommand\bfdefault{bx}
⟨latexrelease⟩
⟨latexrelease⟩\let\bfdefault@previous\undefined
⟨latexrelease⟩\let\mddefault@previous\undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*text⟩
```

\familydefault
\seriesdefault
\shapedefault

Finally we have the hooks that describe the behaviour of the `\normalfont` command. To stay portable, the definition of `\encodingdefault` should *not* be changed and should match the setting above for `\fontencoding`. All other values can be set according to your taste.

```
\newcommand\familydefault{\rmdefault}
\newcommand\seriesdefault{\mddefault}
```

In previous releases `\shapedefault` pointed to `\updefault` which resolved to `n`, but these days that is no longer the case (and `up` is wrong when you want to do a reset. So we now use `n` explicitly.

```
\newcommand\shapedefault{n}
```

(*End of definition for* `\familydefault`*,* `\seriesdefault`*, and* `\shapedefault`*.*)

This finishes the low-level setup in `fonttext.ltx`.

```
⟨/text⟩
```

6 The `fontmath.ltx` file

The identification is done earlier on with a `\ProvidesFile` declaration.

```
⟨*math⟩
\typeout{=== Don't modify this file, use a .cfg file instead ===^^J}
```

6.1 The font encodings used

```
\DeclareFontEncoding{OML}{}{}
\DeclareFontEncoding{OMS}{}{}
\DeclareFontEncoding{OMX}{}{}
```

Finally a declaration for `U` encoding which serves for all fonts that do not fit standard encodings. For math this sets up `\noaccents@` providing for AMS-LaTeX. This macro is used therein to handle accented characters if they are not supported by the font. In other words, if fonts with `U` encoding are used in math, all accents (like from `\breve`) are obtained from some other font that has them.

```
\DeclareFontEncoding{U}{}{\noaccents@}
```

The encodings for math are next:

```
\DeclareFontSubstitution{OML}{cmm}{m}{it}
\DeclareFontSubstitution{OMS}{cmsy}{m}{n}
```

```
\DeclareFontSubstitution{OMX}{cmex}{m}{n}
\DeclareFontSubstitution{U}{cmr}{m}{n}
\begingroup
\nfss@catcodes
\input  {omlcmm.fd}
\input  {omscmsy.fd}
\input  {omxcmex.fd}
\input  {ucmr.fd}
\endgroup
```

6.1.1 Symbolfont and Alphabet declarations

We now define the basic symbol fonts used by LaTeX. These four symbol fonts must be defined by this file.

It is possible to make the symbol fonts point to other external fonts without losing the ability to process documents written at other sites, as long as one defines the same symbol font names with the same encodings, e.g. `operators` with `OT1` etc. If other encodings are used documents become non-portable. Such a change should therefore be done in a package file.

```
\DeclareSymbolFont{operators}   {OT1}{cmr} {m}{n}
\DeclareSymbolFont{letters}     {OML}{cmm} {m}{it}
\DeclareSymbolFont{symbols}     {OMS}{cmsy}{m}{n}
\DeclareSymbolFont{largesymbols}{OMX}{cmex}{m}{n}
\SetSymbolFont{operators}{bold}{OT1}{cmr} {bx}{n}
\SetSymbolFont{letters}  {bold}{OML}{cmm} {b}{it}
\SetSymbolFont{symbols}  {bold}{OMS}{cmsy}{b}{n}
```

Below are the seven math alphabets which are defined by NFSS. Again they must be defined by this file. However, as before you can change the fonts used without losing portability, but you should be careful when changing the encoding since that may make documents come out wrong.

```
\DeclareSymbolFontAlphabet{\mathrm}    {operators}
\DeclareSymbolFontAlphabet{\mathnormal}{letters}
\DeclareSymbolFontAlphabet{\mathcal}   {symbols}
\DeclareMathAlphabet      {\mathbf}{OT1}{cmr}{bx}{n}
\DeclareMathAlphabet      {\mathsf}{OT1}{cmss}{m}{n}
\DeclareMathAlphabet      {\mathit}{OT1}{cmr}{m}{it}
\DeclareMathAlphabet      {\mathtt}{OT1}{cmtt}{m}{n}
```

Given the currently available fonts we cannot bold-en `\mathbf` and `\mathtt` but in principle one could use 'ultra bold' or something. The alphabets defined via `\DeclareSymbolFontAlphabet` will change automatically in a new math version if the corresponding symbol font changes.

```
\SetMathAlphabet\mathsf{bold}{OT1}{cmss}{bx}{n}
\SetMathAlphabet\mathit{bold}{OT1}{cmr}{bx}{it}
```

6.2 Math font sizes

The declarations below declare the text, script and scriptscript size to be used for each text font size.

All occurrences of sizes longer than a single character are replaced with the macro name that holds them, saving a number of tokens (but losing a bit of speed, so this may not stay this way).

```
\DeclareMathSizes{5}{5}{5}{5}
\DeclareMathSizes{6}{6}{5}{5}
\DeclareMathSizes{7}{7}{5}{5}
\DeclareMathSizes{8}{8}{6}{5}
\DeclareMathSizes{9}{9}{6}{5}
\DeclareMathSizes{\@xpt}{\@xpt}{7}{5}
\DeclareMathSizes{\@xipt}{\@xipt}{8}{6}
\DeclareMathSizes{\@xiipt}{\@xiipt}{8}{6}
\DeclareMathSizes{\@xivpt}{\@xivpt}{\@xpt}{7}
\DeclareMathSizes{\@xviipt}{\@xviipt}{\@xiipt}{\@xpt}
\DeclareMathSizes{\@xxpt}{\@xxpt}{\@xivpt}{\@xiipt}
\DeclareMathSizes{\@xxvpt}{\@xxvpt}{\@xxpt}{\@xviipt}
```

6.3 The math symbol assignments

We start by setting up math codes for most of the characters typed in directly from the keyboard. Most of them are normally already setup up in the same way by IniTEX. However, we repeat them here to have a complete setup which can be exchanged with another if desired.

6.3.1 The letters

```
\DeclareMathSymbol{a}{\mathalpha}{letters}{`a}
\DeclareMathSymbol{b}{\mathalpha}{letters}{`b}
\DeclareMathSymbol{c}{\mathalpha}{letters}{`c}
\DeclareMathSymbol{d}{\mathalpha}{letters}{`d}
\DeclareMathSymbol{e}{\mathalpha}{letters}{`e}
\DeclareMathSymbol{f}{\mathalpha}{letters}{`f}
\DeclareMathSymbol{g}{\mathalpha}{letters}{`g}
\DeclareMathSymbol{h}{\mathalpha}{letters}{`h}
\DeclareMathSymbol{i}{\mathalpha}{letters}{`i}
\DeclareMathSymbol{j}{\mathalpha}{letters}{`j}
\DeclareMathSymbol{k}{\mathalpha}{letters}{`k}
\DeclareMathSymbol{l}{\mathalpha}{letters}{`l}
\DeclareMathSymbol{m}{\mathalpha}{letters}{`m}
\DeclareMathSymbol{n}{\mathalpha}{letters}{`n}
\DeclareMathSymbol{o}{\mathalpha}{letters}{`o}
\DeclareMathSymbol{p}{\mathalpha}{letters}{`p}
\DeclareMathSymbol{q}{\mathalpha}{letters}{`q}
\DeclareMathSymbol{r}{\mathalpha}{letters}{`r}
\DeclareMathSymbol{s}{\mathalpha}{letters}{`s}
\DeclareMathSymbol{t}{\mathalpha}{letters}{`t}
\DeclareMathSymbol{u}{\mathalpha}{letters}{`u}
\DeclareMathSymbol{v}{\mathalpha}{letters}{`v}
\DeclareMathSymbol{w}{\mathalpha}{letters}{`w}
\DeclareMathSymbol{x}{\mathalpha}{letters}{`x}
\DeclareMathSymbol{y}{\mathalpha}{letters}{`y}
\DeclareMathSymbol{z}{\mathalpha}{letters}{`z}

\DeclareMathSymbol{A}{\mathalpha}{letters}{`A}
\DeclareMathSymbol{B}{\mathalpha}{letters}{`B}
\DeclareMathSymbol{C}{\mathalpha}{letters}{`C}
\DeclareMathSymbol{D}{\mathalpha}{letters}{`D}
\DeclareMathSymbol{E}{\mathalpha}{letters}{`E}
```

```
\DeclareMathSymbol{F}{\mathalpha}{letters}{`F}
\DeclareMathSymbol{G}{\mathalpha}{letters}{`G}
\DeclareMathSymbol{H}{\mathalpha}{letters}{`H}
\DeclareMathSymbol{I}{\mathalpha}{letters}{`I}
\DeclareMathSymbol{J}{\mathalpha}{letters}{`J}
\DeclareMathSymbol{K}{\mathalpha}{letters}{`K}
\DeclareMathSymbol{L}{\mathalpha}{letters}{`L}
\DeclareMathSymbol{M}{\mathalpha}{letters}{`M}
\DeclareMathSymbol{N}{\mathalpha}{letters}{`N}
\DeclareMathSymbol{O}{\mathalpha}{letters}{`O}
\DeclareMathSymbol{P}{\mathalpha}{letters}{`P}
\DeclareMathSymbol{Q}{\mathalpha}{letters}{`Q}
\DeclareMathSymbol{R}{\mathalpha}{letters}{`R}
\DeclareMathSymbol{S}{\mathalpha}{letters}{`S}
\DeclareMathSymbol{T}{\mathalpha}{letters}{`T}
\DeclareMathSymbol{U}{\mathalpha}{letters}{`U}
\DeclareMathSymbol{V}{\mathalpha}{letters}{`V}
\DeclareMathSymbol{W}{\mathalpha}{letters}{`W}
\DeclareMathSymbol{X}{\mathalpha}{letters}{`X}
\DeclareMathSymbol{Y}{\mathalpha}{letters}{`Y}
\DeclareMathSymbol{Z}{\mathalpha}{letters}{`Z}
```

6.3.2 The digits

```
\DeclareMathSymbol{0}{\mathalpha}{operators}{`0}
\DeclareMathSymbol{1}{\mathalpha}{operators}{`1}
\DeclareMathSymbol{2}{\mathalpha}{operators}{`2}
\DeclareMathSymbol{3}{\mathalpha}{operators}{`3}
\DeclareMathSymbol{4}{\mathalpha}{operators}{`4}
\DeclareMathSymbol{5}{\mathalpha}{operators}{`5}
\DeclareMathSymbol{6}{\mathalpha}{operators}{`6}
\DeclareMathSymbol{7}{\mathalpha}{operators}{`7}
\DeclareMathSymbol{8}{\mathalpha}{operators}{`8}
\DeclareMathSymbol{9}{\mathalpha}{operators}{`9}
```

6.3.3 Punctuation, brace, etc. keys

```
\DeclareMathSymbol{!}{\mathclose}{operators}{"21}
\DeclareMathSymbol{*}{\mathbin}{symbols}{"03} % \ast
\DeclareMathSymbol{+}{\mathbin}{operators}{"2B}
\DeclareMathSymbol{,}{\mathpunct}{letters}{"3B}
\DeclareMathSymbol{-}{\mathbin}{symbols}{"00}
\DeclareMathSymbol{.}{\mathord}{letters}{"3A}
\DeclareMathSymbol{:}{\mathrel}{operators}{"3A}
\DeclareMathSymbol{;}{\mathpunct}{operators}{"3B}
\DeclareMathSymbol{=}{\mathrel}{operators}{"3D}
\DeclareMathSymbol{?}{\mathclose}{operators}{"3F}
```

The following symbols are defined as delimiters below which automatically defines them as math symbols.

```
%\DeclareMathSymbol{(}{\mathopen}{operators}{"28}
%\DeclareMathSymbol{)}{\mathclose}{operators}{"29}
%\DeclareMathSymbol{/}{\mathord}{letters}{"3D}
%\DeclareMathSymbol{[}{\mathopen}{operators}{"5B}
%\DeclareMathSymbol{]}{\mathclose}{operators}{"5D}
%\DeclareMathSymbol{|}{\mathord}{symbols}{"6A}
```

```
%\DeclareMathSymbol{<}{\mathrel}{letters}{"3C}
%\DeclareMathSymbol{>}{\mathrel}{letters}{"3E}
```

Should all of the following being activated by default? Probably not.

```
%\DeclareMathSymbol{`\{}{\mathopen}{symbols}{"66}
%\DeclareMathSymbol{`\}}{\mathclose}{symbols}{"67}
%\DeclareMathSymbol{`\\}{\mathord}{symbols}{"6E} % \backslash
\mathcode`\ ="8000 % \space
\mathcode`\'="8000 % ^\prime
\mathcode`\_="8000 % \_
```

6.3.4 Delimitercodes for characters

[to be completed]

Finally, IniTEX sets all `\delcode` values to -1, except `\delcode`.=0

```
\DeclareMathDelimiter{(}{\mathopen} {operators}{"28}{largesymbols}{"00}
\DeclareMathDelimiter{)}{\mathclose}{operators}{"29}{largesymbols}{"01}
\DeclareMathDelimiter{[}{\mathopen} {operators}{"5B}{largesymbols}{"02}
\DeclareMathDelimiter{]}{\mathclose}{operators}{"5D}{largesymbols}{"03}
```

The next two are considered to be relations when not used in the context of a delimiter! And worse, they do even represent different glyphs when being used as delimiter and not as delimiter. This is a user level syntax inherited from plain TEX. Therefore we explicitly redefine the math symbol definitions for these symbols afterwards.

```
\DeclareMathDelimiter{<}{\mathopen}{symbols}{"68}{largesymbols}{"0A}
\DeclareMathDelimiter{>}{\mathclose}{symbols}{"69}{largesymbols}{"0B}
\DeclareMathSymbol{<}{\mathrel}{letters}{"3C}
\DeclareMathSymbol{>}{\mathrel}{letters}{"3E}
```

And here is another case where the non-delimiter version produces a glyph different from the delimiter version.

```
\DeclareMathDelimiter{/}{\mathord}{operators}{"2F}{largesymbols}{"0E}
\DeclareMathSymbol{/}{\mathord}{letters}{"3D}

\DeclareMathDelimiter{|}{\mathord}{symbols}{"6A}{largesymbols}{"0C}

\expandafter\DeclareMathDelimiter\@backslashchar
                          {\mathord}{symbols}{"6E}{largesymbols}{"0F}
```

N.B. `{` and `}` should NOT get delcodes; otherwise parameter grouping fails!

6.4 Symbols accessed via control sequences

6.4.1 Greek letters

```
\DeclareMathSymbol{\alpha}{\mathord}{letters}{"0B}
\DeclareMathSymbol{\beta}{\mathord}{letters}{"0C}
\DeclareMathSymbol{\gamma}{\mathord}{letters}{"0D}
\DeclareMathSymbol{\delta}{\mathord}{letters}{"0E}
\DeclareMathSymbol{\epsilon}{\mathord}{letters}{"0F}
\DeclareMathSymbol{\zeta}{\mathord}{letters}{"10}
\DeclareMathSymbol{\eta}{\mathord}{letters}{"11}
\DeclareMathSymbol{\theta}{\mathord}{letters}{"12}
\DeclareMathSymbol{\iota}{\mathord}{letters}{"13}
\DeclareMathSymbol{\kappa}{\mathord}{letters}{"14}
\DeclareMathSymbol{\lambda}{\mathord}{letters}{"15}
\DeclareMathSymbol{\mu}{\mathord}{letters}{"16}
\DeclareMathSymbol{\nu}{\mathord}{letters}{"17}
```

```
\DeclareMathSymbol{\xi}{\mathord}{letters}{"18}
\DeclareMathSymbol{\pi}{\mathord}{letters}{"19}
\DeclareMathSymbol{\rho}{\mathord}{letters}{"1A}
\DeclareMathSymbol{\sigma}{\mathord}{letters}{"1B}
\DeclareMathSymbol{\tau}{\mathord}{letters}{"1C}
\DeclareMathSymbol{\upsilon}{\mathord}{letters}{"1D}
\DeclareMathSymbol{\phi}{\mathord}{letters}{"1E}
\DeclareMathSymbol{\chi}{\mathord}{letters}{"1F}
\DeclareMathSymbol{\psi}{\mathord}{letters}{"20}
\DeclareMathSymbol{\omega}{\mathord}{letters}{"21}
\DeclareMathSymbol{\varepsilon}{\mathord}{letters}{"22}
\DeclareMathSymbol{\vartheta}{\mathord}{letters}{"23}
\DeclareMathSymbol{\varpi}{\mathord}{letters}{"24}
\DeclareMathSymbol{\varrho}{\mathord}{letters}{"25}
\DeclareMathSymbol{\varsigma}{\mathord}{letters}{"26}
\DeclareMathSymbol{\varphi}{\mathord}{letters}{"27}
\DeclareMathSymbol{\Gamma}{\mathalpha}{operators}{"00}
\DeclareMathSymbol{\Delta}{\mathalpha}{operators}{"01}
\DeclareMathSymbol{\Theta}{\mathalpha}{operators}{"02}
\DeclareMathSymbol{\Lambda}{\mathalpha}{operators}{"03}
\DeclareMathSymbol{\Xi}{\mathalpha}{operators}{"04}
\DeclareMathSymbol{\Pi}{\mathalpha}{operators}{"05}
\DeclareMathSymbol{\Sigma}{\mathalpha}{operators}{"06}
\DeclareMathSymbol{\Upsilon}{\mathalpha}{operators}{"07}
\DeclareMathSymbol{\Phi}{\mathalpha}{operators}{"08}
\DeclareMathSymbol{\Psi}{\mathalpha}{operators}{"09}
\DeclareMathSymbol{\Omega}{\mathalpha}{operators}{"0A}
```

6.4.2 Ordinary symbols

```
\DeclareMathSymbol{\aleph}{\mathord}{symbols}{"40}
\DeclareMathSymbol{\imath}{\mathord}{letters}{"7B}
\DeclareMathSymbol{\jmath}{\mathord}{letters}{"7C}
\DeclareMathSymbol{\ell}{\mathord}{letters}{"60}
\DeclareMathSymbol{\wp}{\mathord}{letters}{"7D}
\DeclareMathSymbol{\Re}{\mathord}{symbols}{"3C}
\DeclareMathSymbol{\Im}{\mathord}{symbols}{"3D}
\DeclareMathSymbol{\partial}{\mathord}{letters}{"40}
\DeclareMathSymbol{\infty}{\mathord}{symbols}{"31}
\DeclareMathSymbol{\prime}{\mathord}{symbols}{"30}
\DeclareMathSymbol{\emptyset}{\mathord}{symbols}{"3B}
\DeclareMathSymbol{\nabla}{\mathord}{symbols}{"72}
\DeclareMathSymbol{\top}{\mathord}{symbols}{"3E}
\DeclareMathSymbol{\bot}{\mathord}{symbols}{"3F}
\DeclareMathSymbol{\triangle}{\mathord}{symbols}{"34}
\DeclareMathSymbol{\forall}{\mathord}{symbols}{"38}
\DeclareMathSymbol{\exists}{\mathord}{symbols}{"39}
\DeclareMathSymbol{\neg}{\mathord}{symbols}{"3A}
```

Alias:

```
%    \let\lnot=\neg
\DeclareMathSymbol{\lnot}{\mathord}{symbols}{"3A}
\DeclareMathSymbol{\flat}{\mathord}{letters}{"5B}
\DeclareMathSymbol{\natural}{\mathord}{letters}{"5C}
\DeclareMathSymbol{\sharp}{\mathord}{letters}{"5D}
```

```
\DeclareMathSymbol{\clubsuit}{\mathord}{symbols}{"7C}
\DeclareMathSymbol{\diamondsuit}{\mathord}{symbols}{"7D}
\DeclareMathSymbol{\heartsuit}{\mathord}{symbols}{"7E}
\DeclareMathSymbol{\spadesuit}{\mathord}{symbols}{"7F}

\DeclareRobustCommand\hbar{{\mathchar'26\mkern-9muh}}
\DeclareRobustCommand\surd{{\mathchar"1270}}
\DeclareRobustCommand\angle{{\vbox{\ialign{$\m@th\scriptstyle##$\crcr
      \not\mathrel{\mkern14mu}\crcr
      \noalign{\nointerlineskip}
      \mkern2.5mu\leaders\hrule \@height.34pt\hfill\mkern2.5mu\crcr}}}}
```

6.4.3 Large Operators

```
\DeclareMathSymbol{\coprod}{\mathop}{largesymbols}{"60}
\DeclareMathSymbol{\bigvee}{\mathop}{largesymbols}{"57}
\DeclareMathSymbol{\bigwedge}{\mathop}{largesymbols}{"56}
\DeclareMathSymbol{\biguplus}{\mathop}{largesymbols}{"55}
\DeclareMathSymbol{\bigcap}{\mathop}{largesymbols}{"54}
\DeclareMathSymbol{\bigcup}{\mathop}{largesymbols}{"53}
\DeclareMathSymbol{\intop}{\mathop}{largesymbols}{"52}
    \DeclareRobustCommand\int{\intop\nolimits}
\DeclareMathSymbol{\prod}{\mathop}{largesymbols}{"51}
\DeclareMathSymbol{\sum}{\mathop}{largesymbols}{"50}
\DeclareMathSymbol{\bigotimes}{\mathop}{largesymbols}{"4E}
\DeclareMathSymbol{\bigoplus}{\mathop}{largesymbols}{"4C}
\DeclareMathSymbol{\bigodot}{\mathop}{largesymbols}{"4A}
\DeclareMathSymbol{\ointop}{\mathop}{largesymbols}{"48}
    \DeclareRobustCommand\oint{\ointop\nolimits}
\DeclareMathSymbol{\bigsqcup}{\mathop}{largesymbols}{"46}
\DeclareMathSymbol{\smallint}{\mathop}{symbols}{"73}
```

6.4.4 Binary symbols

```
\DeclareMathSymbol{\triangleleft}{\mathbin}{letters}{"2F}
\DeclareMathSymbol{\triangleright}{\mathbin}{letters}{"2E}
\DeclareMathSymbol{\bigtriangleup}{\mathbin}{symbols}{"34}
\DeclareMathSymbol{\bigtriangledown}{\mathbin}{symbols}{"35}
```

Alias:

```
%   \let \varbigtriangledown \bigtriangledown
%   \let \varbigtriangleup \bigtriangleup
\DeclareMathSymbol{\varbigtriangleup}{\mathbin}{symbols}{"34}
\DeclareMathSymbol{\varbigtriangledown}{\mathbin}{symbols}{"35}
```

These last two synonyms are needed because the stmaryrd package redefines them as Operators.

```
\DeclareMathSymbol{\wedge}{\mathbin}{symbols}{"5E}
\DeclareMathSymbol{\vee}{\mathbin}{symbols}{"5F}
```

Alias:

```
%   \let\land=\wedge
%   \let\lor=\vee
\DeclareMathSymbol{\land}{\mathbin}{symbols}{"5E}
\DeclareMathSymbol{\lor}{\mathbin}{symbols}{"5F}

\DeclareMathSymbol{\cap}{\mathbin}{symbols}{"5C}
\DeclareMathSymbol{\cup}{\mathbin}{symbols}{"5B}
\DeclareMathSymbol{\ddagger}{\mathbin}{symbols}{"7A}
```

```
\DeclareMathSymbol{\dagger}{\mathbin}{symbols}{"79}
\DeclareMathSymbol{\sqcap}{\mathbin}{symbols}{"75}
\DeclareMathSymbol{\sqcup}{\mathbin}{symbols}{"74}
\DeclareMathSymbol{\uplus}{\mathbin}{symbols}{"5D}
\DeclareMathSymbol{\amalg}{\mathbin}{symbols}{"71}
\DeclareMathSymbol{\diamond}{\mathbin}{symbols}{"05}
\DeclareMathSymbol{\bullet}{\mathbin}{symbols}{"0F}
\DeclareMathSymbol{\wr}{\mathbin}{symbols}{"6F}
\DeclareMathSymbol{\div}{\mathbin}{symbols}{"04}
\DeclareMathSymbol{\odot}{\mathbin}{symbols}{"0C}
\DeclareMathSymbol{\oslash}{\mathbin}{symbols}{"0B}
\DeclareMathSymbol{\otimes}{\mathbin}{symbols}{"0A}
\DeclareMathSymbol{\ominus}{\mathbin}{symbols}{"09}
\DeclareMathSymbol{\oplus}{\mathbin}{symbols}{"08}
\DeclareMathSymbol{\mp}{\mathbin}{symbols}{"07}
\DeclareMathSymbol{\pm}{\mathbin}{symbols}{"06}
\DeclareMathSymbol{\circ}{\mathbin}{symbols}{"0E}
\DeclareMathSymbol{\bigcirc}{\mathbin}{symbols}{"0D}
\DeclareMathSymbol{\setminus}{\mathbin}{symbols}{"6E}
\DeclareMathSymbol{\cdot}{\mathbin}{symbols}{"01}
\DeclareMathSymbol{\ast}{\mathbin}{symbols}{"03}
\DeclareMathSymbol{\times}{\mathbin}{symbols}{"02}
\DeclareMathSymbol{\star}{\mathbin}{letters}{"3F}
```

6.4.5 Relations

```
\DeclareMathSymbol{\propto}{\mathrel}{symbols}{"2F}
\DeclareMathSymbol{\sqsubseteq}{\mathrel}{symbols}{"76}
\DeclareMathSymbol{\sqsupseteq}{\mathrel}{symbols}{"77}
\DeclareMathSymbol{\parallel}{\mathrel}{symbols}{"6B}
\DeclareMathSymbol{\mid}{\mathrel}{symbols}{"6A}
\DeclareMathSymbol{\dashv}{\mathrel}{symbols}{"61}
\DeclareMathSymbol{\vdash}{\mathrel}{symbols}{"60}
\DeclareMathSymbol{\nearrow}{\mathrel}{symbols}{"25}
\DeclareMathSymbol{\searrow}{\mathrel}{symbols}{"26}
\DeclareMathSymbol{\nwarrow}{\mathrel}{symbols}{"2D}
\DeclareMathSymbol{\swarrow}{\mathrel}{symbols}{"2E}
\DeclareMathSymbol{\Leftrightarrow}{\mathrel}{symbols}{"2C}
\DeclareMathSymbol{\Leftarrow}{\mathrel}{symbols}{"28}
\DeclareMathSymbol{\Rightarrow}{\mathrel}{symbols}{"29}
   \DeclareRobustCommand\neq{\not=}
```

As `\neq` is robust we should not use `\let` to define `\ne` as then it would change if `\neq` changes.

```
   \DeclareRobustCommand\ne{\not=}
```

It would ok to use `\let` for those declared by `\DeclareMathSymbol` but for a cleaner interface we avoid it always (just in case the internals change).

```
\DeclareMathSymbol{\leq}{\mathrel}{symbols}{"14}
\DeclareMathSymbol{\geq}{\mathrel}{symbols}{"15}
```

Alias:

```
%   \let\le=\leq
%   \let\ge=\geq
\DeclareMathSymbol{\le}{\mathrel}{symbols}{"14}
\DeclareMathSymbol{\ge}{\mathrel}{symbols}{"15}
```

```
\DeclareMathSymbol{\succ}{\mathrel}{symbols}{"1F}
\DeclareMathSymbol{\prec}{\mathrel}{symbols}{"1E}
\DeclareMathSymbol{\approx}{\mathrel}{symbols}{"19}
\DeclareMathSymbol{\succeq}{\mathrel}{symbols}{"17}
\DeclareMathSymbol{\preceq}{\mathrel}{symbols}{"16}
\DeclareMathSymbol{\supset}{\mathrel}{symbols}{"1B}
\DeclareMathSymbol{\subset}{\mathrel}{symbols}{"1A}
\DeclareMathSymbol{\supseteq}{\mathrel}{symbols}{"13}
\DeclareMathSymbol{\subseteq}{\mathrel}{symbols}{"12}
\DeclareMathSymbol{\in}{\mathrel}{symbols}{"32}
\DeclareMathSymbol{\ni}{\mathrel}{symbols}{"33}
```

Alias:

```
%    \let\owns=\ni
\DeclareMathSymbol{\owns}{\mathrel}{symbols}{"33}
\DeclareMathSymbol{\gg}{\mathrel}{symbols}{"1D}
\DeclareMathSymbol{\ll}{\mathrel}{symbols}{"1C}
\DeclareMathSymbol{\not}{\mathrel}{symbols}{"36}
\DeclareMathSymbol{\leftrightarrow}{\mathrel}{symbols}{"24}
\DeclareMathSymbol{\leftarrow}{\mathrel}{symbols}{"20}
\DeclareMathSymbol{\rightarrow}{\mathrel}{symbols}{"21}
```

Alias:

```
%   \let\gets=\leftarrow
%   \let\to=\rightarrow
\DeclareMathSymbol{\gets}{\mathrel}{symbols}{"20}
\DeclareMathSymbol{\to}{\mathrel}{symbols}{"21}
\DeclareMathSymbol{\mapstochar}{\mathrel}{symbols}{"37}
   \DeclareRobustCommand\mapsto{\mapstochar\rightarrow}
\DeclareMathSymbol{\sim}{\mathrel}{symbols}{"18}
\DeclareMathSymbol{\simeq}{\mathrel}{symbols}{"27}
\DeclareMathSymbol{\perp}{\mathrel}{symbols}{"3F}
\DeclareMathSymbol{\equiv}{\mathrel}{symbols}{"11}
\DeclareMathSymbol{\asymp}{\mathrel}{symbols}{"10}
\DeclareMathSymbol{\smile}{\mathrel}{letters}{"5E}
\DeclareMathSymbol{\frown}{\mathrel}{letters}{"5F}
\DeclareMathSymbol{\leftharpoonup}{\mathrel}{letters}{"28}
\DeclareMathSymbol{\leftharpoondown}{\mathrel}{letters}{"29}
\DeclareMathSymbol{\rightharpoonup}{\mathrel}{letters}{"2A}
\DeclareMathSymbol{\rightharpoondown}{\mathrel}{letters}{"2B}
```

Here cometh much profligate robustification of math constructs. Warning: some of these commands may become non-robust if an AMS package is loaded.

Further potential problems: some math font packages may make unfortunate assumptions about some of these definitions that are not true of the robust versions we need.

```
\DeclareRobustCommand
  \cong{\mathrel{\mathpalette\@vereq\sim}} % congruence sign
\def\@vereq#1#2{\lower.5\p@\vbox{\lineskiplimit\maxdimen\lineskip-.5\p@
    \ialign{$\m@th#1\hfil##\hfil$\crcr#2\crcr=\crcr}}}
\DeclareRobustCommand
  \notin{\mathrel{\m@th\mathpalette\c@ncel\in}}
\def\c@ncel#1#2{\m@th\ooalign{$\hfil#1\mkern1mu/\hfil$\crcr$#1#2$}}
\DeclareRobustCommand
  \rightleftharpoons{\mathrel{\mathpalette\rlh@{}}}
```

```
\def\rlh@#1{\vcenter{\m@th\hbox{\ooalign{\raise2pt
          \hbox{$#1\rightharpoonup$}\crcr
        $#1\leftharpoondown$}}}}
\DeclareRobustCommand
  \doteq{\buildrel\textstyle.\over=}
```

6.4.6 Arrows

```
\DeclareRobustCommand
  \joinrel{\mathrel{\mkern-3mu}}
\DeclareRobustCommand
  \relbar{\mathrel{\smash-}} % \smash, because -
                             % has the same height as +
```

In contrast to `plain.tex` `\Relbar` got braces around the equal sign to guard against it being "math active" expanding to `\futurelet...`. This might be the case when packages are implementing shorthands for math, e.g. `=>` meaning `\Rightarrow` etc. It would actually be better not to use `=` in such definitions but instead define something like `\mathequalsign` and use this. However we can't do this now as it would break other math layouts where characters are in different places (since those wouldn't know about the need for a new command name).

```
\DeclareRobustCommand
  \Relbar{\mathrel{=}}
\DeclareMathSymbol{\lhook}{\mathrel}{letters}{"2C}
   \DeclareRobustCommand\hookrightarrow{\lhook\joinrel\rightarrow}
\DeclareMathSymbol{\rhook}{\mathrel}{letters}{"2D}
   \DeclareRobustCommand\hookleftarrow{\leftarrow\joinrel\rhook}
\DeclareRobustCommand
  \bowtie{\mathrel\triangleright\joinrel\mathrel\triangleleft}
\DeclareRobustCommand
  \models{\mathrel{|}\joinrel\Relbar}
\DeclareRobustCommand
  \Longrightarrow{\Relbar\joinrel\Rightarrow}
```

LaTeX Change: `\longrightarrow` and `\longleftarrow` redefined to make then robust.

```
\DeclareRobustCommand\longrightarrow
     {\relbar\joinrel\rightarrow}
\DeclareRobustCommand\longleftarrow
     {\leftarrow\joinrel\relbar}
\DeclareRobustCommand
  \Longleftarrow{\Leftarrow\joinrel\Relbar}
\DeclareRobustCommand
  \longmapsto{\mapstochar\longrightarrow}
\DeclareRobustCommand
  \longleftrightarrow{\leftarrow\joinrel\rightarrow}
\DeclareRobustCommand
  \Longleftrightarrow{\Leftarrow\joinrel\Rightarrow}
\DeclareRobustCommand
  \iff{\;\Longleftrightarrow\;}
```

6.4.7 Punctuation symbols

```
\DeclareMathSymbol{\ldotp}{\mathpunct}{letters}{"3A}
\DeclareMathSymbol{\cdotp}{\mathpunct}{symbols}{"01}
\DeclareMathSymbol{\colon}{\mathpunct}{operators}{"3A}
```

This is commented out, since \ldots is now defined in ltoutenc.dtx.

```
%\def\@ldots{\mathinner{\ldotp\ldotp\ldotp}}
%\DeclareRobustCommand\ldots
%          {\relax\ifmmode\@ldots\else\mbox{$\m@th\@ldots\,$}\fi}
\DeclareRobustCommand
  \cdots{\mathinner{\cdotp\cdotp\cdotp}}
\DeclareRobustCommand
  \vdots{\vbox{\baselineskip4\p@ \lineskiplimit\z@
    \kern6\p@\hbox{.}\hbox{.}\hbox{.}}}
\DeclareRobustCommand
  \ddots{\mathinner{\mkern1mu\raise7\p@
    \vbox{\kern7\p@\hbox{.}}\mkern2mu
    \raise4\p@\hbox{.}\mkern2mu\raise\p@\hbox{.}\mkern1mu}}
```

6.4.8 Math accents

```
\DeclareMathAccent{\acute}{\mathalpha}{operators}{"13}
\DeclareMathAccent{\grave}{\mathalpha}{operators}{"12}
\DeclareMathAccent{\ddot}{\mathalpha}{operators}{"7F}
\DeclareMathAccent{\tilde}{\mathalpha}{operators}{"7E}
\DeclareMathAccent{\bar}{\mathalpha}{operators}{"16}
\DeclareMathAccent{\breve}{\mathalpha}{operators}{"15}
\DeclareMathAccent{\check}{\mathalpha}{operators}{"14}
\DeclareMathAccent{\hat}{\mathalpha}{operators}{"5E}
\DeclareMathAccent{\vec}{\mathord}{letters}{"7E}
\DeclareMathAccent{\dot}{\mathalpha}{operators}{"5F}
\DeclareMathAccent{\widetilde}{\mathord}{largesymbols}{"65}
\DeclareMathAccent{\widehat}{\mathord}{largesymbols}{"62}
```

For some reason plain TEX never bothered to provide a ring accent in math (although it is available in the fonts), but since we got a request for it here we go:

```
\DeclareMathAccent{\mathring}{\mathalpha}{operators}{"17}
```

6.4.9 Radicals

```
\DeclareMathRadical{\sqrtsign}{symbols}{"70}{largesymbols}{"70}
```

6.4.10 Over and under something, etc

```
\DeclareRobustCommand\overrightarrow[1]{\vbox{\m@th\ialign{##\crcr
      \rightarrowfill\crcr\noalign{\kern-\p@\nointerlineskip}
      $\hfil\displaystyle{#1}\hfil$\crcr}}}
\DeclareRobustCommand\overleftarrow[1]{\vbox{\m@th\ialign{##\crcr
      \leftarrowfill\crcr\noalign{\kern-\p@\nointerlineskip}%
      $\hfil\displaystyle{#1}\hfil$\crcr}}}
\DeclareRobustCommand\overbrace[1]
     {\mathop{\vbox{\m@th\ialign{##\crcr\noalign{\kern3\p@}%
      \downbracefill\crcr\noalign{\kern3\p@\nointerlineskip}%
      $\hfil\displaystyle{#1}\hfil$\crcr}}}\limits}
\DeclareRobustCommand\underbrace[1]{\mathop{\vtop{\m@th\ialign{##\crcr
   $\hfil\displaystyle{#1}\hfil$\crcr
   \noalign{\kern3\p@\nointerlineskip}%
   \upbracefill\crcr\noalign{\kern3\p@}}}}\limits}
```

(quite a waste of tokens, IMHO — Frank)

```
\DeclareRobustCommand\skew[3]
  {{\muskip\z@#1mu\divide\muskip\z@\tw@ \mkern\muskip\z@
    #2{\mkern-\muskip\z@{#3}\mkern\muskip\z@}\mkern-\muskip\z@}{}}
```

```
\DeclareRobustCommand\rightarrowfill{$\m@th\smash-\mkern-7mu%
  \cleaders\hbox{$\mkern-2mu\smash-\mkern-2mu$}\hfill
  \mkern-7mu\mathord\rightarrow$}
\DeclareRobustCommand\leftarrowfill{$\m@th\mathord\leftarrow\mkern-7mu%
  \cleaders\hbox{$\mkern-2mu\smash-\mkern-2mu$}\hfill
  \mkern-7mu\smash-$}
\DeclareMathSymbol{\braceld}{\mathord}{largesymbols}{"7A}
\DeclareMathSymbol{\bracerd}{\mathord}{largesymbols}{"7B}
\DeclareMathSymbol{\braceru}{\mathord}{largesymbols}{"7C}
\DeclareMathSymbol{\bracelu}{\mathord}{largesymbols}{"7D}
\DeclareRobustCommand\downbracefill{$\m@th \setbox\z@\hbox{$\braceld$}%
  \braceld\leaders\vrule \@height\ht\z@ \@depth\z@\hfill\braceru
  \bracelu\leaders\vrule \@height\ht\z@ \@depth\z@\hfill\bracerd$}
\DeclareRobustCommand\upbracefill{$\m@th \setbox\z@\hbox{$\braceld$}%
  \bracelu\leaders\vrule \@height\ht\z@ \@depth\z@\hfill\bracerd
  \braceld\leaders\vrule \@height\ht\z@ \@depth\z@\hfill\braceru$}
```

6.4.11 Delimiters

```
\DeclareMathDelimiter{\lmoustache}   % top from (, bottom from )
   {\mathopen}{largesymbols}{"7A}{largesymbols}{"40}
\DeclareMathDelimiter{\rmoustache}   % top from ), bottom from (
   {\mathclose}{largesymbols}{"7B}{largesymbols}{"41}
\DeclareMathDelimiter{\arrowvert}    % arrow without arrowheads
   {\mathord}{symbols}{"6A}{largesymbols}{"3C}
\DeclareMathDelimiter{\Arrowvert}    % double arrow without arrowheads
   {\mathord}{symbols}{"6B}{largesymbols}{"3D}
\DeclareMathDelimiter{\Vert}
   {\mathord}{symbols}{"6B}{largesymbols}{"0D}
```

`\DeclareMathDelimiter` produces a command that is robust (with an internal macro containing the payload) so we should not use `\let` for making an alias

```
%\let\|=\Vert
\DeclareMathDelimiter{\|}
   {\mathord}{symbols}{"6B}{largesymbols}{"0D}
\DeclareMathDelimiter{\vert}
   {\mathord}{symbols}{"6A}{largesymbols}{"0C}
\DeclareMathDelimiter{\uparrow}
   {\mathrel}{symbols}{"22}{largesymbols}{"78}
\DeclareMathDelimiter{\downarrow}
   {\mathrel}{symbols}{"23}{largesymbols}{"79}
\DeclareMathDelimiter{\updownarrow}
   {\mathrel}{symbols}{"6C}{largesymbols}{"3F}
\DeclareMathDelimiter{\Uparrow}
   {\mathrel}{symbols}{"2A}{largesymbols}{"7E}
\DeclareMathDelimiter{\Downarrow}
   {\mathrel}{symbols}{"2B}{largesymbols}{"7F}
\DeclareMathDelimiter{\Updownarrow}
   {\mathrel}{symbols}{"6D}{largesymbols}{"77}
\DeclareMathDelimiter{\backslash}    % for double coset G\backslash H
   {\mathord}{symbols}{"6E}{largesymbols}{"0F}
\DeclareMathDelimiter{\rangle}
   {\mathclose}{symbols}{"69}{largesymbols}{"0B}
\DeclareMathDelimiter{\langle}
   {\mathopen}{symbols}{"68}{largesymbols}{"0A}
```

```
\DeclareMathDelimiter{\rbrace}
   {\mathclose}{symbols}{"67}{largesymbols}{"09}
\DeclareMathDelimiter{\lbrace}
   {\mathopen}{symbols}{"66}{largesymbols}{"08}
\DeclareMathDelimiter{\rceil}
   {\mathclose}{symbols}{"65}{largesymbols}{"07}
\DeclareMathDelimiter{\lceil}
   {\mathopen}{symbols}{"64}{largesymbols}{"06}
\DeclareMathDelimiter{\rfloor}
   {\mathclose}{symbols}{"63}{largesymbols}{"05}
\DeclareMathDelimiter{\lfloor}
   {\mathopen}{symbols}{"62}{largesymbols}{"04}
```

\lgroup
\rgroup
\bracevert

There are three plain TEX delimiters which are not fully supported by NFSS, since they partly point into a bold cmr font. Allocating a full symbol font, just to have three delimiters seems a bit too much given the limited space available. For this reason only the extensible sizes are supported. If this is not desired one can use, without losing portability, define `\mathbf` and `\mathtt` as font symbol alphabet (setting up `cmr/bx/n` and `cmtt/m/n` as symbol fonts first) and modify the delimiter declarations to point with their small variant to those symbol fonts. (This is done in `oldlfont.dtx` so look there for examples.)

```
\DeclareMathDelimiter{\lgroup} % extensible ( with sharper tips
     {\mathopen}{largesymbols}{"3A}{largesymbols}{"3A}
\DeclareMathDelimiter{\rgroup} % extensible ) with sharper tips
     {\mathclose}{largesymbols}{"3B}{largesymbols}{"3B}
\DeclareMathDelimiter{\bracevert} % the vertical bar that extends braces
     {\mathord}{largesymbols}{"3E}{largesymbols}{"3E}
```

(*End of definition for* `\lgroup`*,* `\rgroup`*, and* `\bracevert`*.*)

6.5 Math versions of text commands

The `\mathunderscore` here is really a text definition, so it has been put back into `ltoutenc.dtx` (by Chris, 30/04/97) and should be removed from here.

These symbols are the math versions of text commands such as `\P`, `\$`, etc.

\mathparagraph
\mathsection
\mathdollar
\mathsterling
\mathunderscore

These math symbols are not in plain TEX.

```
\DeclareMathSymbol{\mathparagraph}{\mathord}{symbols}{"7B}
\DeclareMathSymbol{\mathsection}{\mathord}{symbols}{"78}
\DeclareMathSymbol{\mathdollar}{\mathord}{operators}{"24}
\DeclareRobustCommand\mathsterling{\mathit{\mathchar"7024}}
\DeclareRobustCommand\mathunderscore{\kern.06em\vbox{\hrule\@width.3em}}
```

(*End of definition for* `\mathparagraph` *and others.*)

\mathellipsis

This is plain TEX's `\ldots`.

```
\DeclareRobustCommand\mathellipsis{\mathinner{\ldotp\ldotp\ldotp}}%
```

(*End of definition for* `\mathellipsis`*.*)

6.6 Other special functions and parameters

6.6.1 Biggggg

```
⟨/math⟩
⟨∗math | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2018/12/01}%
⟨latexrelease⟩                    {\Big}{Start LR-mode}%
\DeclareRobustCommand\big[1]{\leavevmode@ifvmode
   {\hbox{$\left#1\vbox to8.5\p@{}\right.\n@space$}}}
\DeclareRobustCommand\Big[1]{\leavevmode@ifvmode
   {\hbox{$\left#1\vbox to11.5\p@{}\right.\n@space$}}}
\DeclareRobustCommand\bigg[1]{\leavevmode@ifvmode
   {\hbox{$\left#1\vbox to14.5\p@{}\right.\n@space$}}}
\DeclareRobustCommand\Bigg[1]{\leavevmode@ifvmode
   {\hbox{$\left#1\vbox to17.5\p@{}\right.\n@space$}}}
⟨/math | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\Big}{Start LR-mode}%
⟨latexrelease⟩\def\big#1{{\hbox{$\left#1\vbox to8.5\p@{}\right.\n@space$}}}
⟨latexrelease⟩\def\Big#1{{\hbox{$\left#1\vbox to11.5\p@{}\right.\n@space$}}}
⟨latexrelease⟩\def\bigg#1{{\hbox{$\left#1\vbox to14.5\p@{}\right.\n@space$}}}
⟨latexrelease⟩\def\Bigg#1{{\hbox{$\left#1\vbox to17.5\p@{}\right.\n@space$}}}
⟨latexrelease⟩\EndIncludeInRelease
⟨∗math⟩
```

```
\def\n@space{\nulldelimiterspace\z@ \m@th}
```

6.6.2 The log-like functions

`\operator@font` The `\operator@font` determines the symbol font used for log-like functions.

```
\def\operator@font{\mathgroup\symoperators}
```

(*End of definition for* `\operator@font`.)

6.6.3 Parameters

```
\thinmuskip=3mu
\medmuskip=4mu plus 2mu minus 4mu
\thickmuskip=5mu plus 5mu
```

This finishes the low-level setup in `fontmath.ltx`.

```
⟨/math⟩
```

7 Default cfg files

We provide default `cfg` files here to ensure that on installations that search large file trees we do not pick up some strange customisation files from somewhere.

```
⟨∗cfgtext | cfgmath | cfgprel⟩
%%
%%
%%
%% Load the standard setup:
%%
⟨+cfgtext⟩\input{fonttext.ltx}
```

```
⟨+cfgmath⟩\input{fontmath.ltx}
⟨+cfgprel⟩\input{preload.ltx}
%%
%% Small changes could go here; see documentation in cfgguide.tex for
%% allowed modifications.
%%
%% In particular it is not allowed to misuse this configuration file
%% to modify internal LaTeX commands!
%%
%% If you use this file as the basis for configuration please change
%% the \ProvidesFile lines to clearly identify your modification, e.g.,
%%
⟨+cfgtext⟩%%  \ProvidesFile{fonttext.cfg}[2001/06/01
⟨+cfgmath⟩%%  \ProvidesFile{fonttext.cfg}[2001/06/01
⟨+cfgprel⟩%%  \ProvidesFile{preload.cfg}[2001/06/01
%%                                Customised local font setup]
%%
%%
⟨/cfgtext | cfgmath | cfgprel⟩
```

File 31
preload.dtx

1 Overview

This file contains an number of possible settings for preloading fonts during installation of NFSS2 (which is used by LaTeX 2_ε). It will be used to generate the following files:

preload.min	minimal subset of fonts necessary to run NFSS2
preload.ori	preload of CM fonts similar to the old `lfonts.tex`
preload.ltx	The standard selection of preloads
cmpreloa.xpt	preload of CM fonts for 10pt document size
cmpreloa.xip	preload of CM fonts for 11pt document size
cmpreloa.xii	preload of CM fonts for 12pt document size
dcpreloa.xpt	preload of DC fonts for 10pt size
dcpreloa.xip	preload of DC fonts for 11pt size
dcpreloa.xii	preload of DC fonts for 12pt size

These files are for installations that make use of Computer Modern fonts either old encoding (OT1) or Cork encoding (T1). The Computer Modern fonts with Cork encoding are known as DC-fonts.

Most important is `preload.ltx` which is used during format generation. You are *not* allowed to change this file.

2 Customization

You can customize the preloaded fonts in your LaTeX 2_ε system by installing a file with the name `preload.cfg`. If this file exists it will be used in place of the system file `preload.ltx`. You can, for example, copy one of the files mentioned above (that can be generated from this source) to `preload.cfg`.

Or you can define completely other preloads. In that case start from `preload.min` since that contains the fonts that have to be preloaded by *all* LaTeX 2_ε systems.

Avoid using `preload.ori`, it will load so many fonts that on most installations it is nearly impossible to load other font families afterwards. This file is only generated to show what fonts have been preloaded by LaTeX 2.09.

If you normally use other fonts than Computer Modern `preload.min` might be best.

> **Warning:** If you preload fonts with encodings other than the normally supported encodings you have to declare that encoding in a `fontdef.cfg` configuration file (see the documentation in the file `fontdef.dtx`). Adding an extra encoding to the format might produce non-portable documents, thus this should be avoided if possible.

3 Module switches for the DOCSTRIP program

The DOCSTRIP will generate the above file from this source using the following module directives:

driver	produce a documentation driver file
preload	produce a preload... file
cm	for OT1 encoded Computer Modern
dc	for T1 encoded Computer Modern
min	produce minimal subset
xpt	produce 10pt preloads
xipt	produce 11pt preloads
xiipt	produce 12pt preloads
ori	produce preloads similar to old `lfonts.tex`
tex	produce preload.ltx

A typical DOCSTRIP command file would then have entries like:

```
generateFile{preload.min}{t}{\from{preload.dtx}{preload,min}}
```

for generating preload files.

4 A driver for this document

The next bit of code contains the documentation driver file for TEX, i.e., the file that will produce the documentation you are currently reading. It will be extracted from this file by the DOCSTRIP program.

```
⟨*driver⟩
\documentclass{ltxdoc}
%\OnlyDescription  % comment out for implementation details
\begin{document}
   \DocInput{preload.dtx}
\end{document}
⟨/driver⟩
```

5 The code

We begin by loading the math extension font (cmex10) and the LATEX line and circle fonts. It is necessary to do this explicitly since these are used by the LATEX format. Since the internal font name contains `/` characters and digits we construct the name via `\csname`. These are the only fonts (!) that must be loaded in this file.

All `\DeclarePreloadSizes` can be removed or others can be added, they only influence the processing speed.

```
\expandafter\font\csname OMX/cmex/m/n/10\endcsname=cmex10\relax
\font\tenln  =line10   \font\tenlnw  =linew10\relax
\font\tencirc=lcircle10 \font\tencircw=lcirclew10\relax
```

The above fonts should not be touched but anything below this point here in the preload suggestions can be modified without any problems.

```
⟨-tex⟩%****************************************
⟨-tex⟩% Start any modification below this point **
⟨-tex⟩%****************************************
⟨-tex⟩
%%
%% Computer Modern Roman:
%%-----------------------
```

```
⟨*ori⟩
\DeclarePreloadSizes{OT1}{cmr}{m}{n}
        {5,6,7,8,9,10,10.95,12,14.4,17.28,20.74,24.88}
\DeclarePreloadSizes{OT1}{cmr}{bx}{n}{9,10,10.95,12,14.4,17.28}
\DeclarePreloadSizes{OT1}{cmr}{m}{sl}{10,10.95,12}
\DeclarePreloadSizes{OT1}{cmr}{m}{it}{7,8,9,10,10.95,12}
⟨/ori⟩
⟨+xpt & cm⟩ \DeclarePreloadSizes{OT1}{cmr}{m}{n}{5,7,10}
⟨+xpt & dc⟩ \DeclarePreloadSizes{T1}{cmr}{m}{n}{5,7,10}
⟨+xipt & cm⟩ \DeclarePreloadSizes{OT1}{cmr}{m}{n}{6,8,10.95}
⟨+xipt & dc⟩ \DeclarePreloadSizes{T1}{cmr}{m}{n}{6,8,10.95}
⟨+xiipt & cm⟩ \DeclarePreloadSizes{OT1}{cmr}{m}{n}{6,8,12}
⟨+xiipt & dc⟩ \DeclarePreloadSizes{T1}{cmr}{m}{n}{6,8,12}
%%
%% Computer Modern Sans:
%%----------------------
⟨+ori⟩ \DeclarePreloadSizes{OT1}{cmss}{m}{n}{10,10.95,12}
%%
%% Computer Modern Typewriter:
%%----------------------------
⟨+ori⟩ \DeclarePreloadSizes{OT1}{cmtt}{m}{n}{9,10,10.95,12}
%%
%% Computer Modern Math:
%%----------------------
⟨*ori⟩
\DeclarePreloadSizes{OML}{cmm}{m}{it}
         {5,6,7,8,9,10,10.95,12,14.4,17.28,20.74}
\DeclarePreloadSizes{OMS}{cmsy}{m}{n}
         {5,6,7,8,9,10,10.95,12,14.4,17.28,20.74}
⟨/ori⟩
```

The math fonts are the same for both DC and CM fonts. So far there isn't an agreed on standard.

```
⟨*xpt⟩
\DeclarePreloadSizes{OML}{cmm}{m}{it}{5,7,10}
\DeclarePreloadSizes{OMS}{cmsy}{m}{n}{5,7,10}
⟨/xpt⟩
⟨*xipt⟩
\DeclarePreloadSizes{OML}{cmm}{m}{it}{6,8,10.95}
\DeclarePreloadSizes{OMS}{cmsy}{m}{n}{6,8,10.95}
⟨/xipt⟩
⟨*xiipt⟩
\DeclarePreloadSizes{OML}{cmm}{m}{it}{6,8,12}
\DeclarePreloadSizes{OMS}{cmsy}{m}{n}{6,8,12}
⟨/xiipt⟩
%%
%% LaTeX symbol fonts:
%%--------------------
⟨*ori⟩
\DeclarePreloadSizes{U}{lasy}{m}{n}
         {5,6,7,8,9,10,10.95,12,14.4,17.28,20.74}
⟨/ori⟩
⟨/preload⟩
```

File 32
ltfntcmd.dtx

Abstract

The commands defined in this file `ltfntcmd` are part of the kernel code for LaTeX 2ε/NFSS2.
It is also meant to serve as documentation for package writers since it demonstrates how to define high-level font changing commands using a small number of creator functions.

1 Introduction

Font changes such as `\bfseries`, `\sffamily`, etc. are declarations; this means that their scope is delimited by the grouping structure, either by the next `\end` of some environment or by explicitly using a group, e.g., writing something like `{\bfseries...}` in the source. If you make the mistake of writing `\bfseries{...}` (thinking of `\bfseries` as a command with one argument) then the result is rather striking.

Font declarations are an artifact of the TeX system and for several reasons it is better to avoid them on the user level whenever possible. In LaTeX3 they will probably all be replaced by environments and by font commands taking one argument.

This file defines a creator function for such declarative font switches. This function creates commands which can be used in both math and text.

This file also defines a number of high-level commands (all starting with `\text..`) that have one argument and typeset this argument in the requested way. Thus these commands are for typesetting short pieces of text in a specific family, series or shape. These are all produced as examples of the use of a creator function which is itself also defined in this file.

Table 3 shows all these high-level commands in action. A further advantage of using these commands is that they automatically take care of any necessary italic correction on either side of their argument.

Thus, when using such commands, one does not have to worry about forgetting the italic correction when changing fonts. Only in very few situations is this additional space wrong but, for example, most typographers recommend omitting the italic correction if a small punctuation character, like a comma, directly follows the font change. Since the amount of correction required is partly a matter of taste, you can define in what situations the italic correction should be suppressed. This is done by putting the characters that should cancel a preceding italic correction in the list `\nocorrlist`.[39] The default definition for this list is produced by the following.

```
\newcommand \nocorrlist {,.}
```

It is best to declare the most often used characters first, because this will make the processing slightly faster. For example,

```
\emph{When using the \NFSS{} high-level commands,
the \emph{proper} use of italic corrections is
automatically taken care of}. Only
```

[39] Any package that changes the `\catcode` of a character inside `\nocorrlist` must then explicitly reset the list. Otherwise the changed character will no longer be recognized by the suppression algorithm.

Command	*Corresponds to*	*Action*
`\textnormal{..}`	`\normalfont`	Typeset argument in normal family
`\textrm{..}`	`\rmfamily`	Typeset argument in roman family
`\textsf{..}`	`\sffamily`	Typeset argument in sans serif family
`\texttt{..}`	`\ttfamily`	Typeset argument in `typewriter` family
`\textmd{..}`	`\mdseries`	Typeset argument in medium series
`\textbf{..}`	`\bfseries`	Typeset argument in **bold** series
`\textup{..}`	`\upshape`	Typeset argument in normal shape
`\textit{..}`	`\itshape`	Typeset argument in *italic* shape
`\textsl{..}`	`\slshape`	Typeset argument in *slanted* shape
`\textsc{..}`	`\scshape`	Typeset argument in SMALL CAPS shape
`\emph{..}`	`\em`	Typeset argument *emphasized*

Table 3: Font-change commands with arguments

The font change commands provided here all start with `\text..` to emphasize that they are for use in normal text and to be easily memorable. They automatically take care of any necessary italic correction on either side of the argument.

```
\emph{sometimes} one has to help \LaTeX{} by
adding a \verb=\nocorr= command.
```

which results in:

> *When using the NFSS high-level commands, the* proper *use of italic corrections is automatically taken care of.* Only *sometimes* one has to help LaTeX by adding a `\nocorr` command.

In contrast, the use of the declaration forms is often more appropriate when you define your own commands or environments.

```
\newenvironment{bfitemize}{\begin{itemize}\normalfont\bfseries}
                          {\end{itemize}}
\begin{bfitemize}
\item This environment produces boldface items.
\item It is defined in terms of \LaTeX's
  \texttt{itemize} environment and NFSS
  declarations.
\end{bfitemize}
```

This gives:

- **This environment produces boldface items.**
- **It is defined in terms of LaTeX's `itemize` environment and NFSS declarations.**

In addition to global customization of when to insert the italic correction, it is of course sometimes necessary to explicitly insert one with `\/`.

It is also possible to suppress the italic correction in individual instances. For this, the command `\nocorr` is provided.

The `\nocorr` must appear as the first or last token inside the braces of the argument of the `\text...` commands, at that end of the text where you wish to suppress the italic correction.

It is worth pointing out here that inserting a `\/` in places where it can have no function (i.e. anywhere except immediately after a slanted letter) is not an error—it will just be silently ignored. Unfortunately this is not true if the redefinition of `\/` in `amstex.sty` is used as this version can cause space to be removed immediately before the `\/`.

2 The implementation

`\DeclareTextFontCommand` This is the creator function for `\text..` commands. It gives a warning if `\foo` or `\fragfoo` is already defined.

In math mode it simply puts the font declaration and text into a box (possibly an automagically sized one).

Otherwise it first scans the text to see where `\nocorr` occurs within it. This sets the `\check@ic` commands to do what is necessary concerning the italic correction at both ends.

The algorithm for deciding whether to put in an italic correction is not very subtle: one is added whenever the newly current font is not itself positively sloped, unless the next token is a character in the 'nocorr' list. At the end of the text this is done after closing the group so as to check the 'outer font'. Note that this will often result in adding an italic correction token after a character in an unsloped font; we believe (in early 2003) that this is perhaps inefficient but not dangerous.

It also now checks for empty contents of the text command and optimizes this case. Some care is also taken to check that doing dangerous things in vertical mode is avoided.

The italic correction token is added to the horizontal list before (in the list) an immediately preceding non-zero glob of glue (skip) and any non-zero penalty preceding that since, in the typical case, this puts it immediately after the last character in the preceding word.

Note that it is necessary to put in the `\aftergroup\maybe@ic` at the end of the group so that it comes after any other aftergroup tokens and immediately before the following tokens. It is also necessary to remove the `\fi` from the token list before the group ends; this is done by adding an `\expandafter` just before the closing brace.

```
⟨*2ekernel⟩
\def \DeclareTextFontCommand #1#2{%
  \DeclareRobustCommand#1[1]{%
    \ifmmode
      \nfss@text{#2##1}%
    \else
      \hmode@bgroup
       \text@command{##1}%
       #2\check@icl ##1\check@icr
       \expandafter
      \egroup
    \fi
                        }%
}
```

(*End of definition for* `\DeclareTextFontCommand`.)

`\textrm` `\textsf` `\texttt` `\textnormal` Now we define the `\text`⟨*family*⟩ commands in terms of the above; `\texttt` does not look very nice!

```
\DeclareTextFontCommand{\textrm}{\rmfamily}
\DeclareTextFontCommand{\textsf}{\sffamily}
\DeclareTextFontCommand{\texttt}{\ttfamily}
\DeclareTextFontCommand{\textnormal}{\normalfont}
```

(*End of definition for* `\textrm` *and others.*)

`\textbf` `\textmd` For the series attribute:

```
\DeclareTextFontCommand{\textbf}{\bfseries}
\DeclareTextFontCommand{\textmd}{\mdseries}
```

(*End of definition for* `\textbf` *and* `\textmd`.)

`\textit` `\textsl` `\textsc` `\textup` And for the shapes:

```
\DeclareTextFontCommand{\textit}{\itshape}
\DeclareTextFontCommand{\textsl}{\slshape}
\DeclareTextFontCommand{\textsc}{\scshape}
\DeclareTextFontCommand{\textup}{\upshape}
```

(*End of definition for* `\textit` *and others.*)

`\textulc` `\textsw` `\textssc`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩                    {\textulc}{Additional text commands}%
\DeclareTextFontCommand{\textulc}{\ulcshape}
\DeclareTextFontCommand{\textsw}{\swshape}
\DeclareTextFontCommand{\textssc}{\sscshape}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\textulc}{Additional text commands}%
⟨latexrelease⟩
⟨latexrelease⟩\let\textulc\@undefined
⟨latexrelease⟩\let\textsw\@undefined
⟨latexrelease⟩\let\textssc\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\textulc`, `\textsw`, *and* `\textssc`.)

`\emph` Finally we have the `\em` font change declaration of LaTeX. The corresponding definition with argument is

```
\DeclareTextFontCommand{\emph}{\em}
```

(*End of definition for* `\emph`.)

`\strongemph`

```
\DeclareTextFontCommand{\strongemph}{\strongem}
```

(*End of definition for* `\strongemph`.)

`\nocorr` This is just a label, so it does nothing; it should also be unexpandable.

```
\let \nocorr \relax
```

(*End of definition for* `\nocorr`.)

`\check@icl` `\check@icr` We define these defaults in case some error causes them to be expanded at the wrong time.

```
\let \check@icl \@empty
\let \check@icr \@empty
```

(*End of definition for* `\check@icl` *and* `\check@icr`.)

`\text@command` `\check@nocorr@` This checks for a `\nocorr` as the first token in its argument and also for one in any other position not protected within braces (the latter is treated as if it were at the end of the argument).

Is this the correct action in the 'empty' case? It is efficient but typographically it is, strictly, incorrect!

```
\def \text@command #1{%
  \edef \reserved@a {\unexpanded{#1}}%
  \ifx \reserved@a \@empty
    \let \check@icl \@empty
    \let \check@icr \@empty
  \else
```

`\space` is a reserved word in LaTeX or actually already in plain TeX. If somebody really redefines it so many things will break that I don't see any reason to make this routine here slower than necessary.

```
%    \def \reserved@b { }%
%    \ifx \reserved@a \reserved@b
    \ifx \reserved@a \space
      \let \check@icl \@empty
      \let \check@icr \@empty
    \else
      \check@nocorr@ #1\nocorr\@nil
    \fi
  \fi
}
\def \check@nocorr@ #1#2\nocorr#3\@nil {%
```

The two checks are initialized here to their values in the normal case.

```
  \let \check@icl \maybe@ic
  \def \check@icr {\ifvmode \else \aftergroup \maybe@ic \fi}%
  \def \reserved@a {\nocorr}%
  \def \reserved@b {#1}%
  \def \reserved@c {#3}%
  \ifx \reserved@a \reserved@b
    \ifx \reserved@c \@empty
```

In this case there is a `\nocorr` at the start but not at the end, so `\check@icl` should be empty.

```
      \let \check@icl \@empty
    \else
```

Otherwise there is a \nocorr both at the start and elsewhere, so no italic corrections should be added.

```
      \let \check@icl \@empty
      \let \check@icr \@empty
    \fi
  \else
    \ifx \reserved@c \@empty
```

In this case there is no \nocorr anywhere, so we need to check for an italic correction at both the beginning and the end. This has been set up as the default so no code is needed here.

```
    \else
```

In this case there is no \nocorr at the start but there is one elsewhere, so no \aftergroup is needed.

```
      \let \check@icr \@empty
    \fi
  \fi
}
```

(*End of definition for* \text@command *and* \check@nocorr@.)

\ifmaybe@ic Switch used solely within \maybe@ic not interfering with other switches.

```
\newif\ifmaybe@ic
```

(*End of definition for* \ifmaybe@ic.)

\maybe@ic
\maybe@ic@ These macros implement the italic correction.

```
\def \maybe@ic {\futurelet\@let@token\maybe@ic@}
\def \maybe@ic@ {%
```

We first check to see if the current font is positively sloped. (But do not forget the message Rainer sent about an upright font with non-zero slope! Or is this an urban myth?) It has been suggested that this should test against a small positive value, but what?

```
  \ifdim  \fontdimen\@ne\font>\z@
  \else
    \maybe@ictrue
```

It would be possible, but probably not worthwhile, to continue the forward scan beyond any closing braces.

```
    \expandafter\@tfor\expandafter\reserved@a\expandafter:\expandafter=%
        \nocorrlist
```

We have to hide the \@let@token in the macro \t@st@ic rather than testing it directly in the loop since it might be \let to a \fi or \else, which would result in chaos.

```
    \do \t@st@ic
```

Frank thinks that the next bit is inefficient if done after the second change. Chris thinks that most all of this is inefficient for the commonest cases: but that is the price of a cleverer algorithm. It is certainly needed to deal with the use of \nolinebreak.

```
    \ifmaybe@ic \sw@slant \fi
  \fi
}
```

(*End of definition for* `\maybe@ic` *and* `\maybe@ic@`.)

`\t@st@ic` The next token in the input stream is stored in `\@let@token` via a `\let`, the current token from `\nocorrlist` is stored via `\def` in `\reserved@a`. To compare them we have to fiddle around a bit.

If the only things to check were characters then this could be done via an `\if` thus their catcodes would not matter; but this will not work whilst `\futurelet` is used above.

```
\def \t@st@ic {%
  \expandafter\let\expandafter\reserved@b\expandafter=\reserved@a\relax
  \ifx\reserved@b\@let@token
```

If they are the same we record the fact and jump out of the loop.

```
    \maybe@icfalse
    \@break@tfor
  \fi
}
```

(*End of definition for* `\t@st@ic`.)

`\sw@slant` The definition of the mysterious `\sw@slant` command is as follows.

`\fix@penalty`

```
\def \sw@slant {%
```

It is surely correct to put in an italic correction when there is no skip. If the last thing on the list is actually a zero skip (including things whose dimension part is zero, such as `\hfill`), or anything other than a character, then the italic correction will have no effect.

In order to work correctly with unbreakable spaces from `~` (and other common forms of line-breaking control) we also move back across a penalty before the glue.

```
  \ifdim \lastskip=\z@
    \fix@penalty
  \else
    \skip@ \lastskip
    \unskip
    \fix@penalty
    \hskip \skip@
  \fi
}
```

The above code means: "If there is a non-zero space just before the current position (`\ifdim...`) save the amount of that space (`\skip@\lastskip`), remove it (`\unskip`), then do a similar thing if there is a penalty just before the skip, and finally put the space back in."

Since zero glue cannot be distinguished in this context from no glue, we dare not put in an `\hskip` in this case as this may produce an unwanted breakpoint. This is not satisfactory.

The penalty before the glue is handled similarly, with the same caveats concerning the zero case. Is this the first recorded use of `\unpenalty` in standard LaTeX code?

```
\def \fix@penalty {%
  \ifnum \lastpenalty=\z@
    \@@italiccorr
  \else
    \count@ \lastpenalty
    \unpenalty
```

```
    \@@italiccorr
    \penalty \count@
  \fi
}
```

(*End of definition for* `\sw@slant` *and* `\fix@penalty`.)

`\nocorrlist` This holds the list of characters that should prevent italic correction. They should be ordered by decreasing frequency of use. If any such character is made active later on one needs to redefine the list so that the active character becomes part of it.

```
\def \nocorrlist {,.}
```

(*End of definition for* `\nocorrlist`.)

`\nfss@text` This command will by default behave like a LaTeX `\mbox` but may be redefined by packages such as `amstext.sty` to be a bit cleverer.

```
\ifx \nfss@text\@undefined
  \def \nfss@text {\leavevmode\hbox}
\fi
```

(*End of definition for* `\nfss@text`.)

`\DeclareOldFontCommand` This is the function used to create declarative font-changing commands that can also be used to change alphabets in math-mode.

Usage: `\DeclareOldFontCommand \fn{`⟨*font-change decls*⟩`} `⟨*math-alphabet*⟩

Here `\fn` is the font-declaration command being defined, ⟨*font-change decls*⟩ is the declaration it will expand to in text-mode, and ⟨*math-alphabet*⟩ is the (single) math alphabet specifier which is to be used in math-mode.

It does not care whether the command being defined already exists but it does give a warning if it redefines anything.

Here are some typical examples of its use in conjunction with more basic NFSS2 font commands.

```
\DeclareOldFontCommand{\rm}{\normalfont\rmfamily}{\mathrm}
\DeclareOldFontCommand{\sf}{\normalfont\sffamily}{\mathsf}
\DeclareOldFontCommand{\tt}{\normalfont\ttfamily}{\mathtt}
```

```
\def \DeclareOldFontCommand #1#2#3{%
  \DeclareRobustCommand #1{\@fontswitch {#2}{#3}}%
}
```

(*End of definition for* `\DeclareOldFontCommand`.)

`\@fontswitch` `\@@math@egroup` `\@@math@egroup` These two commands actually do the necessary tests and declarative font- or alphabet-changing.

```
\def \@fontswitch #1#2{%
  \ifmmode
     \let \math@bgroup \relax
     \def \math@egroup {\let \math@bgroup \@@math@bgroup
                        \let \math@egroup \@@math@egroup}%
```

We need to have a `\relax` in the following line in case the `#2` is something like `\mathsf` grabbing the next token as an argument. For this reason the code also uses explicit arguments again (see pr/1275).

```
      #2\relax
  \else
      #1%
  \fi
}
\let \@@math@bgroup \math@bgroup
\let \@@math@egroup \math@egroup
```

(*End of definition for* `\@fontswitch`, `\@@math@egroup`, *and* `\@@math@egroup`.)

These commands are available only in the preamble.

```
\@onlypreamble \DeclareTextFontCommand
\@onlypreamble \DeclareOldFontCommand
```

3 Initialization

`\normalsize` This is defined to produce an error.

```
\def\normalsize{%
  \@latex@error {The font size command \protect\normalsize\space
              is not defined:\MessageBreak
              there is probably something wrong with
              the class file}\@eha
}
⟨/2ekernel⟩
```

(*End of definition for* `\normalsize`.)

File 33
lttextcomp.dtx

This file contains the implementation for accessing the glyphs provided by the TS1 encoding (Text Companion Encoding). This is now offered as part of the kernel and so the textcomp package which used to provide the definitions is now mainly needed for compatibility reasons (and doesn't do much any more).

```
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\NewModuleRelease{2020/02/02}{lttextcomp}
⟨latexrelease⟩                    {Text Companion symbols}
```

\oldstylenums
\legacyoldstylenums

Preserve the old definition of \oldstylenums under a different name.

This macro implements old style numerals but only works if we assume that the standard math fonts are used. Thus it needs changing in case other math encodings are used.

```
\DeclareRobustCommand\legacyoldstylenums[1]{%
  \begingroup
```

Provide spacing using the interword space of the current font.

```
    \spaceskip\fontdimen\tw@\font
```

Then switch to the math italic font. We don't change the current value of \f@series which means that you can use bold numerals if \bfseries is in force. As family we use \rmdefault which means that this only works if there exist an OML encoded version of that font or rather a corresponding .fd file (which is the case for standard LaTeX fonts even though they only contain substitutions).

```
    \usefont{OML}{\rmdefault}{\f@series}{it}%
    \mathgroup\symletters #1%
  \endgroup
}
```

And here is the improved one that adjusts depending on surroundings.

```
\DeclareRobustCommand\oldstylenums[1]{%
 \begingroup
 \ifmmode
   \mathgroup\symletters #1%
  \else
```

The \CheckEncodingSubset is discussed below.

```
   \CheckEncodingSubset\@use@text@encoding{TS1}\tc@oldstylesubst2{{#1}}%
 \fi
 \endgroup
}
```

The helper to select the substitution if needed.

```
\def\tc@oldstylesubst#1{%
   \tc@errorwarn
          {Oldstyle digits unavailable for
           family \f@family.\MessageBreak
           Default oldstyle digits used instead}\@eha
  \bgroup
      \expand@font@defaults
```

The substitution defaults are provided in the file `fonttext.ltx`.

```
      \ifx\f@family\rmdef@ult
         \fontfamily\rmsubstdefault
      \else\ifx\f@family\sfdef@ult
         \fontfamily\sfsubstdefault
      \else\ifx\f@family\ttdef@ult
         \fontfamily\ttsubstdefault
      \else
        \fontfamily\textcompsubstdefault
      \fi\fi\fi
      \fontencoding{TS1}\selectfont#1%
  \egroup
}
```

(*End of definition for* `\oldstylenums` *and* `\legacyoldstylenums`.)

`\textcompsubstdefault` Here is the default for the "unknown" case:

```
\def\textcompsubstdefault{\rmsubstdefault}
```

(*End of definition for* `\textcompsubstdefault`.)

To set up the glyphs for the subsets we need a number helpers.

`\tc@errorwarn` To we produce errors, warnings, or only info in the transcripts if glyphs require substitutions? By default it is "info" only. With the `textcomp` package that can be changed.

```
\def\tc@errorwarn#1#2{\@latex@info{#1}}
```

(*End of definition for* `\tc@errorwarn`.)

`\tc@subst`

```
\def\tc@subst#1{%
   \tc@errorwarn
    {Symbol \string#1 not provided by\MessageBreak
     font family \f@family\space
     in TS1 encoding.\MessageBreak Default family used instead}\@eha
  \bgroup
      \expand@font@defaults
      \ifx\f@family\rmdef@ult
         \fontfamily\rmsubstdefault
      \else\ifx\f@family\sfdef@ult
         \fontfamily\sfsubstdefault
      \else\ifx\f@family\ttdef@ult
         \fontfamily\ttsubstdefault
      \else
         \fontfamily\textcompsubstdefault
      \fi\fi\fi
```

Whatever default was chosen, we claim now (locally hopefully) that it can handle all slots (even if not true) to avoid looping in certain situations, e.g., when something was set up incorrectly.

```
      \@namedef{TS1:\f@family}{0}%
      \selectfont#1%
  \egroup
}
```

(*End of definition for* `\tc@subst`.)

\tc@fake@euro `\tc@fake@euro` is an example of a "fake" definition to use in arg `#3` of the command `\CheckEncodingSubset` when a symbol is not available in a certain font family. Here we produce a poor man's Euro symbol by combining a "C" with a "=".

```
\def\tc@fake@euro#1{%
   \leavevmode
   \@font@info{Faking \noexpand#1for font family
                          \f@family\MessageBreak in TS1 encoding}%
   \valign{##\cr
      \vfil\hbox to 0.07em{\dimen@\f@size\p@
                           \math@fontsfalse
                           \fontsize{.7\dimen@}\z@\selectfont=\hss}%
      \vfil\cr%
      \hbox{C}\crcr
   }%
}
```

(*End of definition for* `\tc@fake@euro`.)

\tc@check@symbol
\tc@check@accent These are two abbreviations that we use below to check symbols and accents in TS1. Only there to save some space, e.g., we can then write

```
DeclareTextCommandDefault{\textcurrency}{\tc@check@symbol3\textcurrency}
```

to ensure that `\textcurrency` is only typeset if the current font has a `TS1` subset id of less than 3. Otherwise `\tc@error` is called telling the user that for this font family `\textcurrency` is not available.

```
\def\tc@check@symbol{\CheckEncodingSubset\UseTextSymbol{TS1}\tc@subst}
```

Accents have been made an error in the `textcomp` package when not available. Now that we provide the functionality in the kernel we avoid the error by swapping in a `T1` accent if the `TS1` accent is not available.

```
%\def\tc@check@accent{\CheckEncodingSubset\UseTextAccent{TS1}\tc@error}
\def\tc@check@accent#1{\CheckEncodingSubset\UseTextAccent
                                              {TS1}{\tc@swap@accent#1}}
\def\tc@swap@accent#1#2{\UseTextAccent{T1}#1}
```

(*End of definition for* `\tc@check@symbol` *and* `\tc@check@accent`.)

1 Sub-encodings

Here are the default definitions for the `TS1` symbols. First those that we assume are always available if a font implements `TS1`.

```
\DeclareTextSymbolDefault{\textdollar}{TS1}
\UndeclareTextCommand{\textdollar}{OT1}  % don't use the OT1 def any longer
\DeclareTextSymbolDefault{\textsterling}{TS1}
\UndeclareTextCommand{\textsterling}{OT1}% don't use the OT1 def any longer
\DeclareTextSymbolDefault{\textperthousand}{TS1}
\UndeclareTextCommand{\textperthousand}{T1} % don't use the T1 def
```

Using `\UndeclareTextCommand` above is enough only if the encoding definition files are not reloaded afterwards. In the past that happened if `fontenc` was used in the document preamble (not any longer). So in some sense it is better to fully remove them from the encoding files, but for rollbacks it is easier to keep them in for now.

These are the standard `itemize` and footnote symbols originally taken from `OMS` and now from `TS1`:

```
\DeclareTextSymbolDefault{\textasteriskcentered}{TS1}
\DeclareTextSymbolDefault{\textbullet}{TS1}
\DeclareTextSymbolDefault{\textdaggerdbl}{TS1}
\DeclareTextSymbolDefault{\textdagger}{TS1}
\DeclareTextSymbolDefault{\textparagraph}{TS1}
\DeclareTextSymbolDefault{\textperiodcentered}{TS1}
\DeclareTextSymbolDefault{\textsection}{TS1}
```

And here are the other `TS1` glyphs that are implemented by every font (or nearly every)—a few are commented out and moved to sub-encoding 9, because they aren't around in some fonts.

```
%%\DeclareTextSymbolDefault{\textbardbl}{TS1} % subst in sub-enc 9 above
\DeclareTextSymbolDefault{\textbrokenbar}{TS1}
%%\DeclareTextSymbolDefault{\textcelsius}{TS1} % subst in sub-enc 9 above
\DeclareTextSymbolDefault{\textcent}{TS1}
\DeclareTextSymbolDefault{\textcopyright}{TS1}
\DeclareTextSymbolDefault{\textdegree}{TS1}
\DeclareTextSymbolDefault{\textdiv}{TS1}
\DeclareTextSymbolDefault{\textlnot}{TS1}
\DeclareTextSymbolDefault{\textonehalf}{TS1}
\DeclareTextSymbolDefault{\textonequarter}{TS1}
%%\DeclareTextSymbolDefault{\textonesuperior}{TS1} % subst in sub-enc 9 above
\DeclareTextSymbolDefault{\textordfeminine}{TS1}
\DeclareTextSymbolDefault{\textordmasculine}{TS1}
\DeclareTextSymbolDefault{\textpm}{TS1}
\DeclareTextSymbolDefault{\textquotesingle}{TS1}
\DeclareTextSymbolDefault{\textquotestraightbase}{TS1}
\DeclareTextSymbolDefault{\textquotestraightdblbase}{TS1}
\DeclareTextSymbolDefault{\textregistered}{TS1}
%%\DeclareTextSymbolDefault{\textthreequartersemdash}{TS1} % subst in sub-enc 9 above
\DeclareTextSymbolDefault{\textthreequarters}{TS1}
%%\DeclareTextSymbolDefault{\textthreesuperior}{TS1} % subst in sub-enc 9 above
\DeclareTextSymbolDefault{\texttimes}{TS1}
\DeclareTextSymbolDefault{\texttrademark}{TS1}
%%\DeclareTextSymbolDefault{\texttwelveudash}{TS1} % subst in sub-enc 9 above
%%\DeclareTextSymbolDefault{\texttwosuperior}{TS1} % subst in sub-enc 9 above
\DeclareTextSymbolDefault{\textyen}{TS1}

\DeclareTextSymbolDefault{\textcapitalcompwordmark}{TS1}
\DeclareTextSymbolDefault{\textascendercompwordmark}{TS1}
```

In the following sections the remaining default definitions are ordered by sub-encoding in which they are become **unavailable**, i.e., they are not provided in the sub-encoding with that number and all sub-encodings with higher numbers.

Thus the symbols that are available in sub-encoding x are the symbols above (always available) and the symbols listed as becoming unavailable in sub-encodings $x + 1$ and higher.

1.1 Unavailable in sub-encoding 1 and higher (drop symbols not working in Latin Modern)

The \textcircled is available but the glyph is simply too small so we keep using the OMS glyph.

```
\DeclareTextCommandDefault{\textcircled}
  {\CheckEncodingSubset\UseTextAccent{TS1}{\UseTextAccent{OMS}}1\textcircled}
```

1.2 Unavailable in sub-encoding 2 (majority of new OTF fonts via autoinst) and higher

```
\DeclareTextCommandDefault{\t}
  {\CheckEncodingSubset\UseTextAccent{TS1}{\UseTextAccent{OML}}2\t}
```

Capital accents are really only very seldom implemented, so from sub-encoding 2 onwards we use the normal T1 accents if they are asked for in the document.

In Unicode engines we don't implement them at all but always use the basic accents instead. whether that works or not really depends on the font, something like \"X usually comes out wrong in Unicode engines.

```
\ifx\Umathcode\@undefined
  \DeclareTextCommandDefault{\capitalacute}
                            {\tc@check@accent{\'}2\capitalacute}
  \DeclareTextCommandDefault{\capitalbreve}
                            {\tc@check@accent{\u}2\capitalbreve}
  \DeclareTextCommandDefault{\capitalcaron}
                            {\tc@check@accent{\v}2\capitalcaron}
  \DeclareTextCommandDefault{\capitalcedilla}
                            {\tc@check@accent{\c}2\capitalcedilla}
  \DeclareTextCommandDefault{\capitalcircumflex}
                            {\tc@check@accent{\^}2\capitalcircumflex}
  \DeclareTextCommandDefault{\capitaldieresis}
                            {\tc@check@accent{\"}2\capitaldieresis}
  \DeclareTextCommandDefault{\capitaldotaccent}
                            {\tc@check@accent{\.}2\capitaldotaccent}
  \DeclareTextCommandDefault{\capitalgrave}
                            {\tc@check@accent{\`}2\capitalgrave}
  \DeclareTextCommandDefault{\capitalhungarumlaut}
                            {\tc@check@accent{\H}2\capitalhungarumlaut}
  \DeclareTextCommandDefault{\capitalmacron}
                            {\tc@check@accent{\=}2\capitalmacron}
  \DeclareTextCommandDefault{\capitalogonek}
                            {\tc@check@accent{\k}2\capitalogonek}
  \DeclareTextCommandDefault{\capitalring}
                            {\tc@check@accent{\r}2\capitalring}
  \DeclareTextCommandDefault{\capitaltie}
                            {\tc@check@accent{\t}2\capitaltie}
  \DeclareTextCommandDefault{\capitaltilde}
                            {\tc@check@accent{\~}2\capitaltilde}
```

For \newtie and \capitalnewtie this is actually wrong, they should pick up the accent from the substitution font (not done yet).

```
  \DeclareTextCommandDefault{\newtie}
                            {\tc@check@accent{\t}2\newtie}
  \DeclareTextCommandDefault{\capitalnewtie}
```

```
                                {\tc@check@accent{\t}2\capitalnewtie}
```

In Unicode engines we just execute the simple accents:

```
\else
  \DeclareTextCommandDefault\capitalacute{\@tabacckludge'}
  \DeclareTextCommandDefault\capitalbreve{\u}
  \DeclareTextCommandDefault\capitalcaron{\v}
  \DeclareTextCommandDefault\capitalcedilla{\c}
  \DeclareTextCommandDefault\capitalcircumflex{\^}
  \DeclareTextCommandDefault\capitaldieresis{\"}
  \DeclareTextCommandDefault\capitaldotaccent{\.}
  \DeclareTextCommandDefault\capitalgrave{\@tabacckludge`}
  \DeclareTextCommandDefault\capitalhungarumlaut{\H}
  \DeclareTextCommandDefault\capitalmacron{\@tabacckludge=}
  \DeclareTextCommandDefault\capitalnewtie{\t}
  \DeclareTextCommandDefault\capitalogonek{\k}
  \DeclareTextCommandDefault\capitalring{\r}
  \DeclareTextCommandDefault\capitaltie{\t}
  \DeclareTextCommandDefault\capitaltilde{\~}
  \DeclareTextCommandDefault\newtie{\t}
\fi
```

The next two symbols exist in some fonts (faked?), but we ignore that to keep the subsets reasonable compact and most important linear.

```
\DeclareTextCommandDefault{\textlbrackdbl}
                          {\tc@check@symbol2\textlbrackdbl}
\DeclareTextCommandDefault{\textrbrackdbl}
                          {\tc@check@symbol2\textrbrackdbl}
```

Old style numerals are again in some fonts but using `-OsF`, etc. is the better approach to get them, so we claim they aren't in sub-encoding 2 as that's true for most fonts.

```
\DeclareTextCommandDefault{\texteightoldstyle}
                          {\tc@check@symbol2\texteightoldstyle}
\DeclareTextCommandDefault{\textfiveoldstyle}
                          {\tc@check@symbol2\textfiveoldstyle}
\DeclareTextCommandDefault{\textfouroldstyle}
                          {\tc@check@symbol2\textfouroldstyle}
\DeclareTextCommandDefault{\textnineoldstyle}
                          {\tc@check@symbol2\textnineoldstyle}
\DeclareTextCommandDefault{\textoneoldstyle}
                          {\tc@check@symbol2\textoneoldstyle}
\DeclareTextCommandDefault{\textsevenoldstyle}
                          {\tc@check@symbol2\textsevenoldstyle}
\DeclareTextCommandDefault{\textsixoldstyle}
                          {\tc@check@symbol2\textsixoldstyle}
\DeclareTextCommandDefault{\textthreeoldstyle}
                          {\tc@check@symbol2\textthreeoldstyle}
\DeclareTextCommandDefault{\texttwooldstyle}
                          {\tc@check@symbol2\texttwooldstyle}
\DeclareTextCommandDefault{\textzerooldstyle}
                          {\tc@check@symbol2\textzerooldstyle}
```

The next set of glyphs is special to TEX fonts (and available with a few older PS fonts supported through virtual fonts), but not any longer in the majority of fonts provided through autoinst, so we pretend there aren't available in sub-encoding 2 and below.

```
\DeclareTextCommandDefault{\textacutedbl}
```

```
                          {\tc@check@symbol2\textacutedbl}
\DeclareTextCommandDefault{\textasciiacute}
                          {\tc@check@symbol2\textasciiacute}
\DeclareTextCommandDefault{\textasciibreve}
                          {\tc@check@symbol2\textasciibreve}
\DeclareTextCommandDefault{\textasciicaron}
                          {\tc@check@symbol2\textasciicaron}
\DeclareTextCommandDefault{\textasciidieresis}
                          {\tc@check@symbol2\textasciidieresis}
\DeclareTextCommandDefault{\textasciigrave}
                          {\tc@check@symbol2\textasciigrave}
\DeclareTextCommandDefault{\textasciimacron}
                          {\tc@check@symbol2\textasciimacron}
\DeclareTextCommandDefault{\textgravedbl}
                          {\tc@check@symbol2\textgravedbl}
\DeclareTextCommandDefault{\texttildelow}
                          {\tc@check@symbol2\texttildelow}
```

Finally those below are only available in CM-based fonts but in no font that has its origin outside of the TEX world.

```
\DeclareTextCommandDefault{\textbaht}
                          {\tc@check@symbol2\textbaht}
\DeclareTextCommandDefault{\textbigcircle}
                          {\tc@check@symbol2\textbigcircle}
\DeclareTextCommandDefault{\textborn}
                          {\tc@check@symbol2\textborn}
\DeclareTextCommandDefault{\textcentoldstyle}
                          {\tc@check@symbol2\textcentoldstyle}
\DeclareTextCommandDefault{\textcircledP}
                          {\tc@check@symbol2\textcircledP}
\DeclareTextCommandDefault{\textcopyleft}
                          {\tc@check@symbol2\textcopyleft}
\DeclareTextCommandDefault{\textdblhyphenchar}
                          {\tc@check@symbol2\textdblhyphenchar}
\DeclareTextCommandDefault{\textdblhyphen}
                          {\tc@check@symbol2\textdblhyphen}
\DeclareTextCommandDefault{\textdied}
                          {\tc@check@symbol2\textdied}
\DeclareTextCommandDefault{\textdiscount}
                          {\tc@check@symbol2\textdiscount}
\DeclareTextCommandDefault{\textdivorced}
                          {\tc@check@symbol2\textdivorced}
\DeclareTextCommandDefault{\textdollaroldstyle}
                          {\tc@check@symbol2\textdollaroldstyle}
\DeclareTextCommandDefault{\textguarani}
                          {\tc@check@symbol2\textguarani}
\DeclareTextCommandDefault{\textleaf}
                          {\tc@check@symbol2\textleaf}
\DeclareTextCommandDefault{\textlquill}
                          {\tc@check@symbol2\textlquill}
\DeclareTextCommandDefault{\textmarried}
                          {\tc@check@symbol2\textmarried}
\DeclareTextCommandDefault{\textmho}
                          {\tc@check@symbol2\textmho}
\DeclareTextCommandDefault{\textmusicalnote}
```

```
                          {\tc@check@symbol2\textmusicalnote}
\DeclareTextCommandDefault{\textnaira}
                          {\tc@check@symbol2\textnaira}
\DeclareTextCommandDefault{\textopenbullet}
                          {\tc@check@symbol2\textopenbullet}
\DeclareTextCommandDefault{\textpeso}
                          {\tc@check@symbol2\textpeso}
\DeclareTextCommandDefault{\textpilcrow}
                          {\tc@check@symbol2\textpilcrow}
\DeclareTextCommandDefault{\textrecipe}
                          {\tc@check@symbol2\textrecipe}
\DeclareTextCommandDefault{\textreferencemark}
                          {\tc@check@symbol2\textreferencemark}
\DeclareTextCommandDefault{\textrquill}
                          {\tc@check@symbol2\textrquill}
\DeclareTextCommandDefault{\textservicemark}
                          {\tc@check@symbol2\textservicemark}
\DeclareTextCommandDefault{\textsurd}
                          {\tc@check@symbol2\textsurd}
```

The `\textpertenthousand` also belongs in this group but here we have a choice: in T1 there is a definition for `\textpertenthousand` making the symbol up from % and `\char 24` (twice) but in many fonts that char doesn't exist and the slot is reused for random ligatures. So better not use it because often it is wrong. But pointing to TS1 is also not great as only a few fonts have it as a real symbol, so we get a substitution to CM or LM.

Alternatively we could just state that the symbol is unavailable in those fonts. For now I substitute.

```
\DeclareTextCommandDefault{\textpertenthousand}
                          {\tc@check@symbol2\textpertenthousand}
\UndeclareTextCommand{\textpertenthousand}{T1}
```

1.3 Unavailable in sub-encoding 3 and higher

Sub-encoding 2 is the one where we loose many symbols. In the higher-numbered sub-encodings we see only a few dropped additionally.

```
\DeclareTextCommandDefault{\textlangle}
                          {\tc@check@symbol3\textlangle}
\DeclareTextCommandDefault{\textrangle}
                          {\tc@check@symbol3\textrangle}
```

1.4 Unavailable in sub-encoding 4 and higher

```
\DeclareTextCommandDefault{\textcolonmonetary}
                          {\tc@check@symbol4\textcolonmonetary}
\DeclareTextCommandDefault{\textdong}
                          {\tc@check@symbol4\textdong}
\DeclareTextCommandDefault{\textdownarrow}
                          {\tc@check@symbol4\textdownarrow}
\DeclareTextCommandDefault{\textleftarrow}
                          {\tc@check@symbol4\textleftarrow}
\DeclareTextCommandDefault{\textlira}
                          {\tc@check@symbol4\textlira}
\DeclareTextCommandDefault{\textrightarrow}
```

```
                          {\tc@check@symbol4\textrightarrow}
\DeclareTextCommandDefault{\textuparrow}
                          {\tc@check@symbol4\textuparrow}
\DeclareTextCommandDefault{\textwon}
                          {\tc@check@symbol4\textwon}
```

1.5 Unavailable in sub-encoding 5 (most older PS fonts) and higher

Most older PS fonts (supported in TEX since the early nineties when virtual fonts became available) are sorted under this sub-encoding. But in reality, many of them don't have all glyphs that should be available in sub-encoding 5. Instead they show little squares, i.e., they produce "tofu" if you are unlucky.

But the coverage is so random that it is impossible to sort them properly and if we tried to ensure that they only typeset those glyphs that are really always available, we would have to put them all into sub-encoding 9; so putting them into 5 is really a compromise.

Modern fonts usually don't typeset a tofu character if a glyph is missing. They are therefore only classified as sub-encoding 5 if they really support its glyph set completely.

```
\DeclareTextCommandDefault{\textestimated}
                          {\tc@check@symbol5\textestimated}
\DeclareTextCommandDefault{\textnumero}
                          {\tc@check@symbol5\textnumero}
```

1.6 Unavailable in sub-encoding 6 and higher

```
\DeclareTextCommandDefault{\textflorin}
                          {\tc@check@symbol6\textflorin}
\DeclareTextCommandDefault{\textcurrency}
                          {\tc@check@symbol6\textcurrency}
```

1.7 Unavailable in sub-encoding 7 and higher

```
\DeclareTextCommandDefault{\textfractionsolidus}
                          {\tc@check@symbol7\textfractionsolidus}
\DeclareTextCommandDefault{\textohm}
                          {\tc@check@symbol7\textohm}
\DeclareTextCommandDefault{\textmu}
                          {\tc@check@symbol7\textmu}
\DeclareTextCommandDefault{\textminus}
                          {\tc@check@symbol7\textminus}
```

1.8 Unavailable in sub-encoding 8 and higher

```
\DeclareTextCommandDefault{\textblank}
                          {\tc@check@symbol{8}\textblank}
\DeclareTextCommandDefault{\textinterrobangdown}
                          {\tc@check@symbol{8}\textinterrobangdown}
\DeclareTextCommandDefault{\textinterrobang}
                          {\tc@check@symbol{8}\textinterrobang}
```

Fonts with this sub-encoding don't have a Euro symbol, but instead of substituting we fake it.

```
\DeclareTextCommandDefault{\texteuro}
  {\CheckEncodingSubset\UseTextSymbol{TS1}\tc@fake@euro{8}\texteuro}
```

1.9 Unavailable in Sub-encoding 9 (most missing)

```
\DeclareTextCommandDefault{\textcelsius}
                          {\tc@check@symbol{9}\textcelsius}
\DeclareTextCommandDefault{\textonesuperior}
                          {\tc@check@symbol{9}\textonesuperior}
\DeclareTextCommandDefault{\textthreequartersemdash}
                          {\tc@check@symbol{9}\textthreequartersemdash}
\DeclareTextCommandDefault{\textthreesuperior}
                          {\tc@check@symbol{9}\textthreesuperior}
\DeclareTextCommandDefault{\texttwelveudash}
                          {\tc@check@symbol{9}\texttwelveudash}
\DeclareTextCommandDefault{\texttwosuperior}
                          {\tc@check@symbol{9}\texttwosuperior}
\DeclareTextCommandDefault{\textbardbl}
                          {\tc@check@symbol{9}\textbardbl}
```

2 Unicode engine specials

If we are using a unicode engine we handle some glyphs differently, so this here are the definitions for the Unicode encoding (overwriting the defaults above).

```
\ifx \Umathcode\@undefined  \else
```

This set should be taken from TS1 encoding even if it means you get it from the default font for that encoding.

```
%\DeclareTextSymbol{\textcopyleft}{TS1}{171}
%\DeclareTextSymbol{\textdblhyphen}{TS1}{45}
%\DeclareTextSymbol{\textdblhyphenchar}{TS1}{127}
%\DeclareTextSymbol{\textquotestraightbase}{TS1}{13}
%\DeclareTextSymbol{\textquotestraightdblbase}{TS1}{18}
%\DeclareTextSymbol{\textleaf}{TS1}{108}
%\DeclareTextSymbol{\texttwelveudash}{TS1}{21}
%\DeclareTextSymbol{\textthreequartersemdash}{TS1}{22}
```

If oldstyle numerals are asked for we just use \oldstylenums.

```
\DeclareTextCommand{\textzerooldstyle} \UnicodeEncodingName{\oldstylenums{0}}
\DeclareTextCommand{\textoneoldstyle}  \UnicodeEncodingName{\oldstylenums{1}}
\DeclareTextCommand{\texttwooldstyle}  \UnicodeEncodingName{\oldstylenums{2}}
\DeclareTextCommand{\textthreeoldstyle}\UnicodeEncodingName{\oldstylenums{3}}
\DeclareTextCommand{\textfouroldstyle} \UnicodeEncodingName{\oldstylenums{4}}
\DeclareTextCommand{\textfiveoldstyle} \UnicodeEncodingName{\oldstylenums{5}}
\DeclareTextCommand{\textsixoldstyle}  \UnicodeEncodingName{\oldstylenums{6}}
\DeclareTextCommand{\textsevenoldstyle}\UnicodeEncodingName{\oldstylenums{7}}
\DeclareTextCommand{\texteightoldstyle}\UnicodeEncodingName{\oldstylenums{8}}
\DeclareTextCommand{\textnineoldstyle} \UnicodeEncodingName{\oldstylenums{9}}
```

These have Unicode slots so this should be integrated into TU explicitly

```
\DeclareTextSymbol{\textpilcrow}        \UnicodeEncodingName{"00B6}
\DeclareTextSymbol{\textborn}           \UnicodeEncodingName{"002A}
\DeclareTextSymbol{\textdied}           \UnicodeEncodingName{"2020}
\DeclareTextSymbol{\textlbrackdbl}      \UnicodeEncodingName{"27E6}
\DeclareTextSymbol{\textrbrackdbl}      \UnicodeEncodingName{"27E7}
\DeclareTextSymbol{\textguarani}        \UnicodeEncodingName{"20B2}
```

We could make \textcentoldstyle and \textdollaroldstyle point to dollar and cent in the Unicode encoding

```
%\DeclareTextSymbol{\textcentoldstyle}  \UnicodeEncodingName{"00A2}
%\DeclareTextSymbol{\textdollaroldstyle}\UnicodeEncodingName{"0024}
```

but I think it is better to pick them up from TS1 even if that usually means LMR fonts

```
\DeclareTextSymbol{\textdollaroldstyle}{TS1}{138}
\DeclareTextSymbol{\textcentoldstyle}  {TS1}{139}

\fi               % --- END of Unicode engines specials
```

3 Font family sub-encodings setup

We declare the subsets for a good number of fonts in the kernel ...

But first the default for anything that is not declared. We use 9 which is most likely much too conservative, but with the advantage that we aren't getting missing glyphs (or at least that this is very unlikely). For nearly all font in the TEX Live distribution of 2019 "correct" classifications are given below, so that this default is only used for new font families, and over time the right classifications can be added here too.

```
\DeclareEncodingSubset{TS1}{?}{9}
```

This first block contains the fonts that have been already supported by the textcomp package way back, i.e., the font families that have TEX support since the mid-nineties.

```
\DeclareEncodingSubset{TS1}{ccr}     {0}
\DeclareEncodingSubset{TS1}{cmbr}    {0}
```

The following 4 declarations are now part of the corresponding .fd file, hopefully other will follow so that this list of declarations can eventually be removed from the kernel (where it doesn't belong).

```
%\DeclareEncodingSubset{TS1}{cmr}     {0}
%\DeclareEncodingSubset{TS1}{cmss}    {0}
%\DeclareEncodingSubset{TS1}{cmtt}    {0}
%\DeclareEncodingSubset{TS1}{cmvtt}   {0}

\DeclareEncodingSubset{TS1}{cmtl}    {0}
\DeclareEncodingSubset{TS1}{pxr}     {0}
\DeclareEncodingSubset{TS1}{pxss}    {0}
\DeclareEncodingSubset{TS1}{pxtt}    {0}
\DeclareEncodingSubset{TS1}{qag}     {0}
\DeclareEncodingSubset{TS1}{qbk}     {0}
\DeclareEncodingSubset{TS1}{qcr}     {0}
\DeclareEncodingSubset{TS1}{qcs}     {0}
\DeclareEncodingSubset{TS1}{qhvc}    {0}
\DeclareEncodingSubset{TS1}{qhv}     {0}
\DeclareEncodingSubset{TS1}{qpl}     {0}
\DeclareEncodingSubset{TS1}{qtm}     {0}
\DeclareEncodingSubset{TS1}{qzc}     {0}
\DeclareEncodingSubset{TS1}{txr}     {0}
\DeclareEncodingSubset{TS1}{txss}    {0}
\DeclareEncodingSubset{TS1}{txtt}    {0}

\DeclareEncodingSubset{TS1}{lmr}     {1}
\DeclareEncodingSubset{TS1}{lmdh}    {1}
\DeclareEncodingSubset{TS1}{lmss}    {1}
\DeclareEncodingSubset{TS1}{lmssq}   {1}
```

```
\DeclareEncodingSubset{TS1}{lmvtt}   {1}
```

The lmtt family is missing TM, SM, and perthousand for some reason, so the first safe sub-encoding would be 2, but that is then missing out a huge number of glyphs that are available, so we claim it is sub-encoding 1 even if this can lead to missing glyphs.

```
\DeclareEncodingSubset{TS1}{lmtt}    {1} % missing TM, SM and pertenthousand
```

The next three families have been removed from TeX Live, but we keep the definitions

```
\DeclareEncodingSubset{TS1}{ptmx}    {2}
\DeclareEncodingSubset{TS1}{ptmj}    {2}
\DeclareEncodingSubset{TS1}{ul8}     {2}
```

The next set are the early PostScript font implementations, these days there are better alternatives, but .... Note that, their virtual fonts contain a lot of "tofu" in form of black squares, thus they don't even give a missing character warning if you select such a glyph. This is why they are set as sub-encoding 5.

```
\DeclareEncodingSubset{TS1}{bch} {5} % tofu for blank, ohm
\DeclareEncodingSubset{TS1}{futj}{5} % tofu for blank, interrobang/down, ohm
\DeclareEncodingSubset{TS1}{futs}{5} % tofu for blank, ohm
\DeclareEncodingSubset{TS1}{futx}{5} % probably (currently broken distrib)
\DeclareEncodingSubset{TS1}{pag} {5} % tofu for blank, interrobang/down, ohm
\DeclareEncodingSubset{TS1}{pbk} {5} % tofu for blank, interrobang/down, ohm
\DeclareEncodingSubset{TS1}{pcr} {5} % tofu for blank, interrobang/down, ohm
\DeclareEncodingSubset{TS1}{phv} {5} % tofu for blank, interrobang/down, ohm
\DeclareEncodingSubset{TS1}{pnc} {5} % tofu for blank, interrobang/down, ohm
\DeclareEncodingSubset{TS1}{pplj}{5} % tofu for blank
\DeclareEncodingSubset{TS1}{pplx}{5} % tofu for blank
\DeclareEncodingSubset{TS1}{ppl} {5} % tofu for blank interrobang/down
\DeclareEncodingSubset{TS1}{ptm} {5} % tofu for blank, interrobang/down, ohm
\DeclareEncodingSubset{TS1}{pzc} {5} % tofu for blank, interrobang/down, ohm
\DeclareEncodingSubset{TS1}{ul9} {5} % tofu for blank, interrobang/down, ohm
```

The next set suffers from the same problem and they contain even fewer real glyphs.

```
\DeclareEncodingSubset{TS1}{dayroms}{6} % tofu for blank, interrobang/down, ohm
\DeclareEncodingSubset{TS1}{dayrom} {6} % tofu for blank, interrobang/down, ohm
\DeclareEncodingSubset{TS1}{augie}{8} % really only missing euro
\DeclareEncodingSubset{TS1}{put}  {8}
\DeclareEncodingSubset{TS1}{uag}  {8} % probably (currently broken distrib)
\DeclareEncodingSubset{TS1}{ugq}  {8}
\DeclareEncodingSubset{TS1}{zi4}     {9}
```

LucidaBright (sold through TUG) probably not quite correct, I guess as I have the older fonts ...

```
\DeclareEncodingSubset{TS1}{hls}     {5}
\DeclareEncodingSubset{TS1}{hlst}    {5}
\DeclareEncodingSubset{TS1}{hlct}    {5}
\DeclareEncodingSubset{TS1}{hlh}     {5}
\DeclareEncodingSubset{TS1}{hlx}     {8}
\DeclareEncodingSubset{TS1}{hlce}    {8}
\DeclareEncodingSubset{TS1}{hlcn}    {8}
\DeclareEncodingSubset{TS1}{hlcw}    {8}
\DeclareEncodingSubset{TS1}{hlcf}    {8}
```

Below are the newer fonts that have support files for LaTeX. With very few exceptions the classifications are done so that all characters are correctly produced (either being available in the font or substituted).

There are a few fonts that contain "tofu" squares in places (instead of a real glyph) and in a few cases some really seldom needed chars are unavailable, i.e., produce missing glyphs (to avoid that a large number of available chars are unnecessarily substituted).

Encoding declarations for these font families shouldn't really be in the kernel, but part of the `.fd` files for the family. When we introduced the concept in 2021 we had some hope that this would happen over time and that we could take the declarations out—after all it is nearly impossible to maintain it correctly in the kernel, given that fonts may get new glyphs added (happened for several of them in the recent year) which is something we wouldn't notice. However, so far this hasn't happened, so in 2024, I went through the current set and adjusted the declarations in several places.

Next four are wrong and still need adjustment:

```
\DeclareEncodingSubset{TS1}{lato-*}       {0}  % with a bunch of tofu inside
\DeclareEncodingSubset{TS1}{opensans-*}   {0}  % with a bunch of tofu inside
\DeclareEncodingSubset{TS1}{cantarell-*}  {0}  % with a bunch of tofu inside
\DeclareEncodingSubset{TS1}{tli}          {1}  % with lots of tofu inside

\DeclareEncodingSubset{TS1}{fbb-*}        {2}  % missing centoldstyle

\DeclareEncodingSubset{TS1}{Alegreya-*}               {2}
\DeclareEncodingSubset{TS1}{AlegreyaSans-*}           {2}
\DeclareEncodingSubset{TS1}{BaskervilleF-*}           {2}
\DeclareEncodingSubset{TS1}{DejaVuSans-TLF}           {2}
\DeclareEncodingSubset{TS1}{DejaVuSansCondensed-TLF}  {2}
```

Next one is missing `\textfractionsolidus` but is otherwise completely sub-encoding 2 so we use that sub-encoding.

```
\DeclareEncodingSubset{TS1}{DejaVuSansMono-TLF}       {2}

\DeclareEncodingSubset{TS1}{EBGaramond-*}             {2}
\DeclareEncodingSubset{TS1}{Merriwthr-OsF}            {2}
\DeclareEncodingSubset{TS1}{MerriwthrSans-OsF}        {2}
\DeclareEncodingSubset{TS1}{Montserrat-*}             {2}
\DeclareEncodingSubset{TS1}{MontserratAlternates-*}   {2}
\DeclareEncodingSubset{TS1}{NotoSansMono-TLF}         {2}
\DeclareEncodingSubset{TS1}{NotoSansMono-TOsF}        {2}
\DeclareEncodingSubset{TS1}{Tempora-TLF}              {2}
\DeclareEncodingSubset{TS1}{Tempora-TOsF}             {2}
\DeclareEncodingSubset{TS1}{XCharter-TLF}             {2}
\DeclareEncodingSubset{TS1}{XCharter-TOsF}            {2}
\DeclareEncodingSubset{TS1}{erewhon-*}                {2}

\DeclareEncodingSubset{TS1}{Arimo-TLF}                {3}
\DeclareEncodingSubset{TS1}{Crlt-*}                   {3}
\DeclareEncodingSubset{TS1}{IBMPlexMono-TLF}          {3}
\DeclareEncodingSubset{TS1}{IBMPlexSans-TLF}          {3}
\DeclareEncodingSubset{TS1}{IBMPlexSerif-TLF}         {3}
\DeclareEncodingSubset{TS1}{SourceCodePro-TLF}        {3}
\DeclareEncodingSubset{TS1}{SourceCodePro-TOsF}       {3}
\DeclareEncodingSubset{TS1}{SourceSansPro-*}          {3}
\DeclareEncodingSubset{TS1}{SourceSerifPro-*}         {3}
\DeclareEncodingSubset{TS1}{Tinos-TLF}                {3}
```

```
\DeclareEncodingSubset{TS1}{AccanthisADFStdNoThree-LF}{4}
\DeclareEncodingSubset{TS1}{Cabin-TLF}                 {4}
\DeclareEncodingSubset{TS1}{Caladea-TLF}               {4}
\DeclareEncodingSubset{TS1}{Chivo-*}                   {4}
\DeclareEncodingSubset{TS1}{ClearSans-TLF}             {4}
\DeclareEncodingSubset{TS1}{Coelacanth-LF}             {4}
\DeclareEncodingSubset{TS1}{CrimsonPro-*}              {4}
\DeclareEncodingSubset{TS1}{FiraMono-TLF}              {4}
\DeclareEncodingSubset{TS1}{FiraMono-TOsF}             {4}
\DeclareEncodingSubset{TS1}{FiraSans-*}                {4}
\DeclareEncodingSubset{TS1}{Go-TLF}                    {4}
\DeclareEncodingSubset{TS1}{GoMono-TLF}                {4}
\DeclareEncodingSubset{TS1}{InriaSans-*}               {4}
\DeclareEncodingSubset{TS1}{InriaSerif-*}              {4}
\DeclareEncodingSubset{TS1}{LibertinusSans-*}          {4}
\DeclareEncodingSubset{TS1}{LibertinusSerif-*}         {4}
\DeclareEncodingSubset{TS1}{LibreBodoni-TLF}           {4}
\DeclareEncodingSubset{TS1}{LibreFranklin-TLF}         {4}
\DeclareEncodingSubset{TS1}{LinguisticsPro-LF}         {4}
\DeclareEncodingSubset{TS1}{LinguisticsPro-OsF}        {4}
\DeclareEncodingSubset{TS1}{LinuxBiolinumT-*}          {4}
\DeclareEncodingSubset{TS1}{LinuxLibertineT-*}         {4}
\DeclareEncodingSubset{TS1}{MintSpirit-*}              {4}
\DeclareEncodingSubset{TS1}{MintSpiritNoTwo-*}         {4}
\DeclareEncodingSubset{TS1}{PTMono-TLF}                {4}
\DeclareEncodingSubset{TS1}{PTSans-TLF}                {4}
\DeclareEncodingSubset{TS1}{PTSansCaption-TLF}         {4}
\DeclareEncodingSubset{TS1}{PTSansNarrow-TLF}          {4}
\DeclareEncodingSubset{TS1}{PTSerif-TLF}               {4}
\DeclareEncodingSubset{TS1}{PTSerifCaption-TLF}        {4}
\DeclareEncodingSubset{TS1}{Raleway-TLF}               {4}
\DeclareEncodingSubset{TS1}{Raleway-TOsF}              {4}
\DeclareEncodingSubset{TS1}{Roboto-*}                  {4}
\DeclareEncodingSubset{TS1}{RobotoMono-TLF}            {4}
\DeclareEncodingSubset{TS1}{RobotoSlab-TLF}            {4}
\DeclareEncodingSubset{TS1}{Rosario-*}                 {4}
\DeclareEncodingSubset{TS1}{SticksTooText-*}           {4}
\DeclareEncodingSubset{TS1}{UniversalisADFStd-LF}      {4}

\DeclareEncodingSubset{TS1}{Almndr-OsF}                {5}
\DeclareEncodingSubset{TS1}{Baskervaldx-*}             {5}
\DeclareEncodingSubset{TS1}{Bttr-TLF}                  {5}
\DeclareEncodingSubset{TS1}{Cinzel-LF}                 {5}
\DeclareEncodingSubset{TS1}{CinzelDecorative-LF}       {5}
\DeclareEncodingSubset{TS1}{Cochineal-*}               {5}
\DeclareEncodingSubset{TS1}{DejaVuSerif-TLF}           {5}
\DeclareEncodingSubset{TS1}{DejaVuSerifCondensed-TLF}  {5}
\DeclareEncodingSubset{TS1}{GilliusADF-LF}             {5}
\DeclareEncodingSubset{TS1}{GilliusADFCond-LF}         {5}
\DeclareEncodingSubset{TS1}{GilliusADFNoTwo-LF}        {5}
\DeclareEncodingSubset{TS1}{GilliusADFNoTwoCond-LF}    {5}
\DeclareEncodingSubset{TS1}{OldStandard-TLF}           {5}
\DeclareEncodingSubset{TS1}{PlyfrDisplay-TLF}          {5}
\DeclareEncodingSubset{TS1}{PlyfrDisplay-TOsF}         {5}
\DeclareEncodingSubset{TS1}{TheanoDidot-TLF}           {5}
```

```
\DeclareEncodingSubset{TS1}{TheanoDidot-TOsF}          {5}
\DeclareEncodingSubset{TS1}{TheanoModern-TLF}          {5}
\DeclareEncodingSubset{TS1}{TheanoModern-TOsF}         {5}
\DeclareEncodingSubset{TS1}{TheanoOldStyle-TLF}        {5}
\DeclareEncodingSubset{TS1}{TheanoOldStyle-TOsF}       {5}
\DeclareEncodingSubset{TS1}{charssil-TLF}              {5}

\DeclareEncodingSubset{TS1}{Crimson-TLF}               {6}
\DeclareEncodingSubset{TS1}{LibertinusSerifDisplay-LF}{6}
\DeclareEncodingSubset{TS1}{LinuxLibertineDisplayT-*} {6}
\DeclareEncodingSubset{TS1}{LinuxLibertineMonoT-LF}    {6}
\DeclareEncodingSubset{TS1}{LinuxLibertineMonoT-TLF}   {6}
\DeclareEncodingSubset{TS1}{Ovrlck-LF}                 {6}

\DeclareEncodingSubset{TS1}{ComicNeue-TLF}             {7}
\DeclareEncodingSubset{TS1}{ComicNeueAngular-TLF}      {7}
\DeclareEncodingSubset{TS1}{CormorantGaramond-*}       {7}
\DeclareEncodingSubset{TS1}{Heuristica-TLF}            {7}
\DeclareEncodingSubset{TS1}{Heuristica-TOsF}           {7}
\DeclareEncodingSubset{TS1}{IMFELLEnglish-TLF}         {7}
\DeclareEncodingSubset{TS1}{LibreBskrvl-TLF}           {7}
\DeclareEncodingSubset{TS1}{LibreCsln-*}               {7}
\DeclareEncodingSubset{TS1}{Lbstr-LF}                  {7}
\DeclareEncodingSubset{TS1}{Mrcls-LF}                  {7}
```

Strangely enough NotoSerif and NotoSans are sub-encoding 7 as they are missing `\textminus` and several other glyphs. In contrast, the NotoSansMono is far more complete.

```
\DeclareEncodingSubset{TS1}{NotoSans-*}                {7}
\DeclareEncodingSubset{TS1}{NotoSerif-*}               {7}
\DeclareEncodingSubset{TS1}{Quattro-LF}                {7}
\DeclareEncodingSubset{TS1}{QuattroSans-LF}            {7}
\DeclareEncodingSubset{TS1}{Frm-LF}                    {7}  % the superiors are missing

\DeclareEncodingSubset{TS1}{LibertinusMono-TLF}        {8}

\DeclareEncodingSubset{TS1}{AlgolRevived-TLF}          {9}
```

4 Legacy symbol support for lists and footnote symbols

`\UseLegacyTextSymbols`

```
\def\UseLegacyTextSymbols{%
  \DeclareTextSymbolDefault{\textasteriskcentered}{OMS}%
  \DeclareTextSymbolDefault{\textbardbl}{OMS}%
  \DeclareTextSymbolDefault{\textbullet}{OMS}%
  \DeclareTextSymbolDefault{\textdaggerdbl}{OMS}%
  \DeclareTextSymbolDefault{\textdagger}{OMS}%
  \DeclareTextSymbolDefault{\textparagraph}{OMS}%
  \DeclareTextSymbolDefault{\textperiodcentered}{OMS}%
  \DeclareTextSymbolDefault{\textsection}{OMS}%
  \UndeclareTextCommand{\textsection}{T1}%
  \expandafter\let\csname oldstylenums \expandafter\endcsname
                  \csname legacyoldstylenums \endcsname
}
```

(*End of definition for* `\UseLegacyTextSymbols`.)

`\textlegacyasteriskcentered`
`\textlegacybardbl`
`\textlegacybullet`
`\textlegacydaggerdbl`
`\textlegacydagger`
`\textlegacyparagraph`
`\textlegacyperiodcentered`
`\textlegacysection`

Here are new names for the legacy symbols that LaTeX used to pick up from the `OMS` encoded fonts (and used for itemize lists or footnote symbols).

We go the roundabout way via separate OMS declarations so that

```
\renewcommand\textbullet{\textlegacybullet}
```

doesn't produce an endless loop.

```
\DeclareTextSymbol{\textlegacyasteriskcentered}{OMS}{3}   % "03
\DeclareTextSymbol{\textlegacybardbl}{OMS}{107}           % "6B
\DeclareTextSymbol{\textlegacybullet}{OMS}{15}            % "0F
\DeclareTextSymbol{\textlegacydaggerdbl}{OMS}{122}        % "7A
\DeclareTextSymbol{\textlegacydagger}{OMS}{121}           % "79
\DeclareTextSymbol{\textlegacyparagraph}{OMS}{123}        % "7B
\DeclareTextSymbol{\textlegacyperiodcentered}{OMS}{1}     % "01
\DeclareTextSymbol{\textlegacysection}{OMS}{120}          % "78

\DeclareTextSymbolDefault{\textlegacyasteriskcentered}{OMS}
\DeclareTextSymbolDefault{\textlegacybardbl}{OMS}
\DeclareTextSymbolDefault{\textlegacybullet}{OMS}
\DeclareTextSymbolDefault{\textlegacydaggerdbl}{OMS}
\DeclareTextSymbolDefault{\textlegacydagger}{OMS}
\DeclareTextSymbolDefault{\textlegacyparagraph}{OMS}
\DeclareTextSymbolDefault{\textlegacyperiodcentered}{OMS}
\DeclareTextSymbolDefault{\textlegacysection}{OMS}
```

(*End of definition for* `\textlegacyasteriskcentered` *and others.*)

Supporting rollback ...

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩     {lttextcomp}{Undefine text companion symbols}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\oldstylenums[1]{%
⟨latexrelease⟩   \begingroup
⟨latexrelease⟩    \spaceskip\fontdimen\tw@\font
⟨latexrelease⟩    \usefont{OML}{\rmdefault}{\f@series}{it}%
⟨latexrelease⟩    \mathgroup\symletters #1%
⟨latexrelease⟩   \endgroup
⟨latexrelease⟩}
⟨latexrelease⟩\let\legacyoldstylenums\@undefined
⟨latexrelease⟩\def\textcompsubstdefault{cmr}
⟨latexrelease⟩
⟨latexrelease⟩\let\DeclareEncodingSubset\@undefined
⟨latexrelease⟩\let\CheckEncodingSubset\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\DeclareTextSymbolDefault{\textdollar}{OT1}
⟨latexrelease⟩\DeclareTextSymbolDefault{\textsterling}{OT1}
⟨latexrelease⟩\DeclareTextCommand{\textdollar}{OT1}{\hmode@bgroup
⟨latexrelease⟩   \ifdim \fontdimen\@ne\font >\z@
⟨latexrelease⟩      \slshape
⟨latexrelease⟩   \else
⟨latexrelease⟩      \upshape
⟨latexrelease⟩   \fi
```

```
⟨latexrelease⟩    \char`\$\egroup}
⟨latexrelease⟩\DeclareTextCommand{\textsterling}{OT1}{\hmode@bgroup
⟨latexrelease⟩    \ifdim \fontdimen\@ne\font >\z@
⟨latexrelease⟩       \itshape
⟨latexrelease⟩    \else
⟨latexrelease⟩       \fontshape{ui}\selectfont
⟨latexrelease⟩    \fi
⟨latexrelease⟩    \char`\$\egroup}
⟨latexrelease⟩\DeclareTextCommand{\textperthousand}{T1}
⟨latexrelease⟩    {\%\char 24 }
⟨latexrelease⟩
⟨latexrelease⟩\DeclareTextSymbolDefault{\textasteriskcentered}{OMS}
⟨latexrelease⟩\DeclareTextSymbolDefault{\textbullet}{OMS}
⟨latexrelease⟩\DeclareTextSymbolDefault{\textdaggerdbl}{OMS}
⟨latexrelease⟩\DeclareTextSymbolDefault{\textdagger}{OMS}
⟨latexrelease⟩\DeclareTextSymbolDefault{\textparagraph}{OMS}
⟨latexrelease⟩\DeclareTextSymbolDefault{\textperiodcentered}{OMS}
⟨latexrelease⟩\DeclareTextSymbolDefault{\textsection}{OMS}
⟨latexrelease⟩
⟨latexrelease⟩\DeclareTextSymbolDefault{\textbardbl}{OMS}
⟨latexrelease⟩\let\textbrokenbar\@undefined
⟨latexrelease⟩\let\textcelsius\@undefined
⟨latexrelease⟩\let\textcent\@undefined
⟨latexrelease⟩\DeclareTextCommandDefault{\textcopyright}
⟨latexrelease⟩                          {\textcircled{c}}
⟨latexrelease⟩\let\textdegree\@undefined
⟨latexrelease⟩\let\textdiv\@undefined
⟨latexrelease⟩\let\textlnot\@undefined
⟨latexrelease⟩\let\textonehalf\@undefined
⟨latexrelease⟩\let\textonequarter\@undefined
⟨latexrelease⟩\let\textonesuperior\@undefined
⟨latexrelease⟩\DeclareTextCommandDefault{\textordfeminine}
⟨latexrelease⟩                          {\textsuperscript{a}}
⟨latexrelease⟩\DeclareTextCommandDefault{\textordmasculine}
⟨latexrelease⟩                          {\textsuperscript{o}}
⟨latexrelease⟩\let\textpm\@undefined
⟨latexrelease⟩\let\textquotesingle\@undefined
⟨latexrelease⟩\let\textquotestraightbase\@undefined
⟨latexrelease⟩\let\textquotestraightdblbase\@undefined
⟨latexrelease⟩\DeclareTextCommandDefault{\textregistered}
⟨latexrelease⟩    {\textcircled{%
⟨latexrelease⟩        \check@mathfonts\fontsize\sf@size\z@
⟨latexrelease⟩        \math@fontsfalse\selectfont R}}
⟨latexrelease⟩\let\textthreequartersemdash\@undefined
⟨latexrelease⟩\let\textthreequarters\@undefined
⟨latexrelease⟩\let\textthreesuperior\@undefined
⟨latexrelease⟩\let\texttimes\@undefined
⟨latexrelease⟩\DeclareTextCommandDefault{\texttrademark}
⟨latexrelease⟩                          {\textsuperscript{TM}}
⟨latexrelease⟩\let\texttwelveudash\@undefined
⟨latexrelease⟩\let\texttwosuperior\@undefined
⟨latexrelease⟩\let\textyen\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let\textcapitalcompwordmark\@undefined
```

```
⟨latexrelease⟩\let\textascendercompwordmark\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\DeclareTextAccentDefault{\textcircled}{OMS}
⟨latexrelease⟩\DeclareTextAccentDefault{\t}{OML}
⟨latexrelease⟩
⟨latexrelease⟩\let\capitalacute\@undefined
⟨latexrelease⟩\let\capitalbreve\@undefined
⟨latexrelease⟩\let\capitalcaron\@undefined
⟨latexrelease⟩\let\capitalcedilla\@undefined
⟨latexrelease⟩\let\capitalcircumflex\@undefined
⟨latexrelease⟩\let\capitaldieresis\@undefined
⟨latexrelease⟩\let\capitaldotaccent\@undefined
⟨latexrelease⟩\let\capitalgrave\@undefined
⟨latexrelease⟩\let\capitalhungarumlaut\@undefined
⟨latexrelease⟩\let\capitalmacron\@undefined
⟨latexrelease⟩\let\capitalnewtie\@undefined
⟨latexrelease⟩\let\capitalogonek\@undefined
⟨latexrelease⟩\let\capitalring\@undefined
⟨latexrelease⟩\let\capitaltie\@undefined
⟨latexrelease⟩\let\capitaltilde\@undefined
⟨latexrelease⟩\let\newtie\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let\textlbrackdbl\@undefined
⟨latexrelease⟩\let\textrbrackdbl\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let\texteightoldstyle\@undefined
⟨latexrelease⟩\let\textfiveoldstyle\@undefined
⟨latexrelease⟩\let\textfouroldstyle\@undefined
⟨latexrelease⟩\let\textnineoldstyle\@undefined
⟨latexrelease⟩\let\textoneoldstyle\@undefined
⟨latexrelease⟩\let\textsevenoldstyle\@undefined
⟨latexrelease⟩\let\textsixoldstyle\@undefined
⟨latexrelease⟩\let\textthreeoldstyle\@undefined
⟨latexrelease⟩\let\texttwooldstyle\@undefined
⟨latexrelease⟩\let\textzerooldstyle\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let\textacutedbl\@undefined
⟨latexrelease⟩\let\textasciiacute\@undefined
⟨latexrelease⟩\let\textasciibreve\@undefined
⟨latexrelease⟩\let\textasciicaron\@undefined
⟨latexrelease⟩\let\textasciidieresis\@undefined
⟨latexrelease⟩\let\textasciigrave\@undefined
⟨latexrelease⟩\let\textasciimacron\@undefined
⟨latexrelease⟩\let\textgravedbl\@undefined
⟨latexrelease⟩\let\texttildelow\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let\textbaht\@undefined
⟨latexrelease⟩\let\textbigcircle\@undefined
⟨latexrelease⟩\let\textborn\@undefined
⟨latexrelease⟩\let\textcentoldstyle\@undefined
⟨latexrelease⟩\let\textcircledP\@undefined
⟨latexrelease⟩\let\textcopyleft\@undefined
⟨latexrelease⟩\let\textdblhyphenchar\@undefined
⟨latexrelease⟩\let\textdblhyphen\@undefined
```

```
⟨latexrelease⟩\let\textdied\@undefined
⟨latexrelease⟩\let\textdiscount\@undefined
⟨latexrelease⟩\let\textdivorced\@undefined
⟨latexrelease⟩\let\textdollaroldstyle\@undefined
⟨latexrelease⟩\let\textguarani\@undefined
⟨latexrelease⟩\let\textleaf\@undefined
⟨latexrelease⟩\let\textlquill\@undefined
⟨latexrelease⟩\let\textmarried\@undefined
⟨latexrelease⟩\let\textmho\@undefined
⟨latexrelease⟩\let\textmusicalnote\@undefined
⟨latexrelease⟩\let\textnaira\@undefined
⟨latexrelease⟩\let\textopenbullet\@undefined
⟨latexrelease⟩\let\textpeso\@undefined
⟨latexrelease⟩\let\textpilcrow\@undefined
⟨latexrelease⟩\let\textrecipe\@undefined
⟨latexrelease⟩\let\textreferencemark\@undefined
⟨latexrelease⟩\let\textrquill\@undefined
⟨latexrelease⟩\let\textservicemark\@undefined
⟨latexrelease⟩\let\textsurd\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\DeclareTextCommand{\textpertenthousand}{T1}
⟨latexrelease⟩                    {\%\char 24\char 24 }
⟨latexrelease⟩
⟨latexrelease⟩\let\textlangle\@undefined
⟨latexrelease⟩\let\textrangle\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let\textcolonmonetary\@undefined
⟨latexrelease⟩\let\textdong\@undefined
⟨latexrelease⟩\let\textdownarrow\@undefined
⟨latexrelease⟩\let\textleftarrow\@undefined
⟨latexrelease⟩\let\textlira\@undefined
⟨latexrelease⟩\let\textrightarrow\@undefined
⟨latexrelease⟩\let\textuparrow\@undefined
⟨latexrelease⟩\let\textwon\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let\textestimated\@undefined
⟨latexrelease⟩\let\textnumero\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let\textflorin\@undefined
⟨latexrelease⟩\let\textcurrency\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let\textfractionsolidus\@undefined
⟨latexrelease⟩\let\textohm\@undefined
⟨latexrelease⟩\let\textmu\@undefined
⟨latexrelease⟩\let\textminus\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let\textblank\@undefined
⟨latexrelease⟩\let\textinterrobangdown\@undefined
⟨latexrelease⟩\let\textinterrobang\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let\texteuro\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let\textcelsius\@undefined
⟨latexrelease⟩\let\textonesuperior\@undefined
```

```
⟨latexrelease⟩\let\textthreequartersemdash\@undefined
⟨latexrelease⟩\let\textthreesuperior\@undefined
⟨latexrelease⟩\let\texttwelveudash\@undefined
⟨latexrelease⟩\let\texttwosuperior\@undefined
⟨latexrelease⟩\let\textbardbl\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let\UseLegacyTextSymbols\@undefined
⟨latexrelease⟩\let\textlegacyasteriskcentered\@undefined
⟨latexrelease⟩\let\textlegacybardbl\@undefined
⟨latexrelease⟩\let\textlegacybullet\@undefined
⟨latexrelease⟩\let\textlegacydaggerdbl\@undefined
⟨latexrelease⟩\let\textlegacydagger\@undefined
⟨latexrelease⟩\let\textlegacyparagraph\@undefined
⟨latexrelease⟩\let\textlegacyperiodcentered\@undefined
⟨latexrelease⟩\let\textlegacysection\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndModuleRelease
```

5 The `textcomp` package

For any rollback request before 2018-08-11 we make an attempt by loading the 2018 version.

```
⟨*TS1sty⟩
\DeclareRelease{}{1997-12-01}{textcomp-2018-08-11.sty}
\DeclareRelease{}{2018-08-11}{textcomp-2018-08-11.sty}
\DeclareCurrentRelease{}{2020-02-02}

\ProvidesPackage{textcomp}
 [2024/04/24 v2.1b Standard LaTeX package]
```

A precaution in case this is used without rebuilding the format.

```
\NeedsTeXFormat{LaTeX2e}[2020/02/02]
```

This is implemented by defining the default subset:

```
\DeclareOption{full}{\DeclareEncodingSubset{TS1}{?}{0}}
\DeclareOption{almostfull}{\DeclareEncodingSubset{TS1}{?}{1}}
\DeclareOption{euro}{\DeclareEncodingSubset{TS1}{?}{8}}
\DeclareOption{safe}{\DeclareEncodingSubset{TS1}{?}{9}}
```

The default is set up in the kernel is "safe" these days for unknown fonts but LaTeX has definitions for most families so it seldom applies.

If a different default is used then one needs to check the results to ensure that there aren't "missing glyphs".

The next set of options define the warning level (default in the kernel is info only). Using the package options you can change this behavior.

```
\DeclareOption{error}
              {\gdef\tc@errorwarn{\PackageError{textcomp}}}
\DeclareOption{warn}
              {\gdef\tc@errorwarn#1#2{\PackageWarning{textcomp}{#1}}}
\DeclareOption{info}
              {\gdef\tc@errorwarn#1#2{\PackageInfo{textcomp}{#1}}}
\DeclareOption{quiet}{\gdef\tc@errorwarn#1#2{}}
```

The "force" option basically changes the sub-encoding to that of the default (which, unless changes, is 9 these days), i.e., it no longer depends on the font in use. This is mainly there because it might have been used in older documents, but not something that is recommended.

```
\DeclareOption{force}{%
    \def\CheckEncodingSubset#1#2#3#4#5{%
      \ifnum #4>%
           0\csname #2:?\endcsname
           \relax
       \expandafter\@firstoftwo
      \else
       \expandafter\@secondoftwo
    \fi
     {#1{#2}}{#3}%
     #5}%
}

\ProcessOptions\relax
```

There is not much else to do nowadays, because everything is already set up in the LaTeX kernel.

```
\InputIfFileExists{textcomp.cfg}
  {\PackageInfo{textcomp}{Local configuration file used}}{}
```

⟨/TS1sty⟩

5.1 The old textcomp package code

This section contains the old code for the textcomp package and its documentation. It is only used if we roll back prior to 2020. Thus all the rest is mainly for historians. Note that the old code categorized in the sub-encodings only into 6 classes not 10.

⟨∗TS1oldsty⟩

```
\ProvidesPackage{textcomp}
   [2018/08/11 v2.0j Standard LaTeX package]
```

This one is for the TS1 encoding which contains text symbols for use with the T1-encoded text fonts. It therefore first inputs the file TS1enc.def and then sets (or resets) the defaults for the symbols it contains. The result of this is that when one of these symbols is accessed and the current encoding does not provide it, the symbol will be supplied by a silent, local change to this encoding.

Since many PostScript fonts only implement a subset of TS1 many commands only produce black blobs of ink. To resolve the resulting problems a number of options have been introduced and some code has been developed to distinguish sub-encodings.

The sub-encodings have a numerical id and are defined as follows for TS1:

#5 those TS1 symbols that are also in the ISO-Adobe character set; without textcurrency, which is often misused for the Euro. Older Type1 fonts from the non-TeX world provide only this subset.

#4 = #5 + \texteuro. Most newer fonts provide this.

#3 = #4 + \textomega. Can also be described as TS1 ∩ (ISO-Adobe ∪ MacRoman). (Except for the missing "currency".)

#2 = #3 + `\textestimated` + `\textcurrency`. Can also be described as `TS1` ∩ `Adobe-Western-2`. This may be relevant for OpenType fonts, which usually show the Adobe-Western-2 character set.

#1 = `TS1` without `\textcircled` and `\t`. These two glyphs are often not implemented and if their kernel defaults are changed commands like `\copyright` unnecessarily fail.

#0 = full `TS1`

And here a summary to go in the transcript file:

```
\PackageInfo{textcomp}{Sub-encoding information:\MessageBreak
   \space\space 5 = only ISO-Adobe without
                              \string\textcurrency\MessageBreak
   \space\space 4 = 5 + \string\texteuro\MessageBreak
   \space\space 3 = 4 + \string\textohm\MessageBreak
   \space\space 2 = 3 + \noexpand\textestimated+
                                \string\textcurrency\MessageBreak
   \space\space 1 = TS1 - \noexpand\textcircled-
                                           \string\t\MessageBreak
   \space\space 0 = TS1 (full)\MessageBreak
   Font families with sub-encoding setting implement\MessageBreak
   only a restricted character set as indicated.\MessageBreak
   Family '?' is the default used for unknown fonts.\MessageBreak
   See the documentation for details\@gobble}
```

`\DeclareEncodingSubset` An encoding subset to which a font family belongs is declared by the command `\DeclareEncodingSubset` that takes the major encoding as the first argument (e.g., `TS1`), the family name as the second argument (e.g., `cmr`), and the subset encoding id as a third, (e.g., `0` for `cmr`).

The default encoding subset to use when nothing is known about the current font family is named `?`.

```
\def\DeclareEncodingSubset#1#2#3{%
  \@ifundefined{#1:#2}%
    {\PackageInfo{textcomp}{Setting #2 sub-encoding to #1/#3}}%
    {\PackageInfo{textcomp}{Changing #2 sub-encoding to #1/#3}}%
  \@namedef{#1:#2}{#3}}
```

In the original code this was only allowed in the preamble but now `.fd`s might contain it so that even in a rollback situation it is necessary to allow it everywhere in the document.

```
%\@onlypreamble\DeclareEncodingSubset
```

(*End of definition for* `\DeclareEncodingSubset`.)

The options for the package are the following:

safe for unknown font families enables only symbols that are also in the ISO-Adobe character set; without "currency", which is often misused for the Euro. Older Type1 fonts from the non-TeX world provide only this subset.

euro enables the "safe" symbols plus the `\texteuro` command. Most newer fonts provide this.

full enables all `TS1` commands; useful only with fonts like EC or CM bright.

almostfull same as "full", except that `\textcircled` and `\t` are *not* redefined from their defaults to avoid that commands like `\copyright` suddenly no longer work.

force ignore all subset encoding definitions stored in the package itself or in the configuration file and always use the default subset as specified by one of the other options (seldom useful, only dangerous).

`\iftc@forced` Switch used to implement the `force` option

```
\newif\iftc@forced   \tc@forcedfalse
```

(*End of definition for* `\iftc@forced`.)

This is implemented by defining the default subset:

```
\DeclareOption{full}{\DeclareEncodingSubset{TS1}{?}{0}}
\DeclareOption{almostfull}{\DeclareEncodingSubset{TS1}{?}{1}}
\DeclareOption{euro}{\DeclareEncodingSubset{TS1}{?}{4}}
\DeclareOption{safe}{\DeclareEncodingSubset{TS1}{?}{5}}
```

The default is "almostfull" which means that old documents will work except that `\textcircled` and `\t` will use the kernel defaults (with the advantage that this also works if the current font, as often the case, doesn't implement these glyphs).

The "force" option simply sets the switch to true.

```
\DeclareOption{force}{\tc@forcedtrue}
```

The suggestions to user is to use the "safe" option always unless that balks in which case they could switch to "almostfull" but then better check their output manually.

```
\def\tc@errorwarn{\PackageError}
\DeclareOption{warn}{\gdef\tc@errorwarn#1#2#3{\PackageWarning{#1}{#2}}}
\DeclareOption{quiet}{\gdef\tc@errorwarn#1#2#3{}}
\ExecuteOptions{almostfull}
\ProcessOptions\relax
```

`\CheckEncodingSubset` The command `\CheckEncodingSubset` will check if the current font family has the right encoding subset to typeset a certain command. It takes five arguments as follows: first argument is either `\UseTextSymbol`, `\UseTextAccent` depending on whether or not the symbol is a text symbol or a text accent.

The second argument is the encoding from which this symbol should be fetched.

The third argument is either a fake accessor command or an error message. the code in that argument (if ever executed) receives two arguments: `#2` and `#5` of `\CheckEncodingSubset`.

Argument four is the subset encoding id to test against: if this value is higher than the subset id of the current font family then we typeset the symbol, i.e., execute `#1{#2}#5` otherwise it runs `#3#5`, e.g., to produce an error message or fake the glyph somehow.

Argument five is the symbol or accent command that is being checked.

For usage examples see definitions below.

```
\iftc@forced
```

If the "force" option was given we always use the default for testing against.

```
\def\CheckEncodingSubset#1#2#3#4#5{%
    \ifnum #4>%
        0\csname #2:?\endcsname
        \relax
  \expandafter\@firstoftwo
  \else
```

```
    \expandafter\@secondoftwo
  \fi
   {#1{#2}}{#3}%
   #5%
}
```

In normal circumstances the test is a bit more complicated: first check if there exists a macro \⟨*arg2*⟩:⟨*current-family*⟩ and if so use that value to test against, otherwise use the default to test against.

```
\else
\def\CheckEncodingSubset#1#2#3#4#5{%
    \ifnum #4>%
      \expandafter\ifx\csname #2:\f@family\endcsname\relax
        0\csname #2:?\endcsname
      \else
        \csname #2:\f@family\endcsname
      \fi
   \relax
   \expandafter\@firstoftwo
  \else
   \expandafter\@secondoftwo
 \fi
  {#1{#2}}{#3}%
  #5%
}
\fi
```

(*End of definition for* `\CheckEncodingSubset`.)

`\tc@subst`

```
\def\tc@subst#1{%
   \tc@errorwarn{textcomp}%
    {Symbol \string#1 not provided by\MessageBreak
     font family \f@family\space
     in TS1 encoding.\MessageBreak Default family used instead}\@eha
  \bgroup\fontfamily\textcompsubstdefault\selectfont#1\egroup
}
```

(*End of definition for* `\tc@subst`.)

`\tc@error` `\tc@error` is going to be used in arg `#3` of `\CheckEncodingSubset` when a symbol is not available in a certain font family. It gets pass the encoding it normally lives in (arg one) and the name of the symbol or accent that has a problem.

```
% error commands take argument:
% #1 symbol to be used
\def\tc@error#1{%
   \PackageError{textcomp}%  % should be latex error if general
    {Accent \string#1 not provided by\MessageBreak
     font family \f@family\space
     in TS1 encoding}\@eha
}
```

(*End of definition for* `\tc@error`.)

\tc@fake@euro

\tc@fake@euro is an example of a "fake" definition to use in arg #3 of \CheckEncodingSubset when a symbol is not available in a certain font family. Here we produce an Euro symbol by combining a "C" with a "=".

```
\def\tc@fake@euro#1{%
   \leavevmode
   \PackageInfo{textcomp}{Faking \noexpand#1for font family
                          \f@family\MessageBreak in TS1 encoding}%
   \valign{##\cr
      \vfil\hbox to 0.07em{\dimen@\f@size\p@
                           \math@fontsfalse
                           \fontsize{.7\dimen@}\z@\selectfont=\hss}%
      \vfil\cr%
      \hbox{C}\crcr
   }%
}
```

(*End of definition for* \tc@fake@euro.)

\tc@check@symbol
\tc@check@accent

These are two abbreviations that we use below to check symbols and accents in TS1. Only there to save some space, e.g., we can then write

```
DeclareTextCommandDefault{\textcurrency}{\tc@check@symbol3\textcurrency}
```

to ensure that \textcurrency is only typeset if the current font has a TS1 subset id of less than 3. Otherwise \tc@error is called telling the user that for this font family \textcurrency is not available.

```
\def\tc@check@symbol{\CheckEncodingSubset\UseTextSymbol{TS1}\tc@subst}
\def\tc@check@accent{\CheckEncodingSubset\UseTextAccent{TS1}\tc@error}
```

(*End of definition for* \tc@check@symbol *and* \tc@check@accent.)

We start with the commands that are "safe" and which can be unconditionally set up, first the accents...

```
\DeclareTextAccentDefault{\capitalcedilla}{TS1}
\DeclareTextAccentDefault{\capitalogonek}{TS1}
\DeclareTextAccentDefault{\capitalgrave}{TS1}
\DeclareTextAccentDefault{\capitalacute}{TS1}
\DeclareTextAccentDefault{\capitalcircumflex}{TS1}
\DeclareTextAccentDefault{\capitaltilde}{TS1}
\DeclareTextAccentDefault{\capitaldieresis}{TS1}
\DeclareTextAccentDefault{\capitalhungarumlaut}{TS1}
\DeclareTextAccentDefault{\capitalring}{TS1}
\DeclareTextAccentDefault{\capitalcaron}{TS1}
\DeclareTextAccentDefault{\capitalbreve}{TS1}
\DeclareTextAccentDefault{\capitalmacron}{TS1}
\DeclareTextAccentDefault{\capitaldotaccent}{TS1}
```

... and then the other glyphs.

```
\DeclareTextSymbolDefault{\textcapitalcompwordmark}{TS1}
\DeclareTextSymbolDefault{\textascendercompwordmark}{TS1}
\DeclareTextSymbolDefault{\textquotestraightbase}{TS1}
\DeclareTextSymbolDefault{\textquotestraightdblbase}{TS1}
\DeclareTextSymbolDefault{\texttwelveudash}{TS1}
\DeclareTextSymbolDefault{\textthreequartersemdash}{TS1}
\DeclareTextSymbolDefault{\textdollar}{TS1}
```

```
\DeclareTextSymbolDefault{\textquotesingle}{TS1}
\DeclareTextSymbolDefault{\textasteriskcentered}{TS1}
\DeclareTextSymbolDefault{\textfractionsolidus}{TS1}
\DeclareTextSymbolDefault{\textminus}{TS1}
\DeclareTextSymbolDefault{\textlbrackdbl}{TS1}
\DeclareTextSymbolDefault{\textrbrackdbl}{TS1}
\DeclareTextSymbolDefault{\textasciigrave}{TS1}
\DeclareTextSymbolDefault{\texttildelow}{TS1}
\DeclareTextSymbolDefault{\textasciibreve}{TS1}
\DeclareTextSymbolDefault{\textasciicaron}{TS1}
\DeclareTextSymbolDefault{\textgravedbl}{TS1}
\DeclareTextSymbolDefault{\textacutedbl}{TS1}
\DeclareTextSymbolDefault{\textdagger}{TS1}
\DeclareTextSymbolDefault{\textdaggerdbl}{TS1}
\DeclareTextSymbolDefault{\textbardbl}{TS1}
\DeclareTextSymbolDefault{\textperthousand}{TS1}
\DeclareTextSymbolDefault{\textbullet}{TS1}
\DeclareTextSymbolDefault{\textcelsius}{TS1}
\DeclareTextSymbolDefault{\textflorin}{TS1}
\DeclareTextSymbolDefault{\texttrademark}{TS1}
\DeclareTextSymbolDefault{\textcent}{TS1}
\DeclareTextSymbolDefault{\textsterling}{TS1}
\DeclareTextSymbolDefault{\textyen}{TS1}
\DeclareTextSymbolDefault{\textbrokenbar}{TS1}
\DeclareTextSymbolDefault{\textsection}{TS1}
\DeclareTextSymbolDefault{\textasciidieresis}{TS1}
\DeclareTextSymbolDefault{\textcopyright}{TS1}
\DeclareTextSymbolDefault{\textordfeminine}{TS1}
\DeclareTextSymbolDefault{\textlnot}{TS1}
\DeclareTextSymbolDefault{\textregistered}{TS1}
\DeclareTextSymbolDefault{\textasciimacron}{TS1}
\DeclareTextSymbolDefault{\textdegree}{TS1}
\DeclareTextSymbolDefault{\textpm}{TS1}
\DeclareTextSymbolDefault{\texttwosuperior}{TS1}
\DeclareTextSymbolDefault{\textthreesuperior}{TS1}
\DeclareTextSymbolDefault{\textasciiacute}{TS1}
\DeclareTextSymbolDefault{\textmu}{TS1}
\DeclareTextSymbolDefault{\textparagraph}{TS1}
\DeclareTextSymbolDefault{\textperiodcentered}{TS1}
\DeclareTextSymbolDefault{\textonesuperior}{TS1}
\DeclareTextSymbolDefault{\textordmasculine}{TS1}
\DeclareTextSymbolDefault{\textonequarter}{TS1}
\DeclareTextSymbolDefault{\textonehalf}{TS1}
\DeclareTextSymbolDefault{\textthreequarters}{TS1}
\DeclareTextSymbolDefault{\texttimes}{TS1}
\DeclareTextSymbolDefault{\textdiv}{TS1}
```

The `\texteuro` is only available for subsets with id 4 or less. Otherwise we fake the glyph using `\tc@fake@euro`

```
\DeclareTextCommandDefault{\texteuro}
   {\CheckEncodingSubset\UseTextSymbol{TS1}\tc@fake@euro5\texteuro}
```

The `\textohm` is only available for subsets with id 3 or less. Otherwise we produce an error.

```
\DeclareTextCommandDefault{\textohm}{\tc@check@symbol4\textohm}
```

The \textestimated and \textcurrency are only provided for fonts with subset encoding with id 2 or less.

```
\DeclareTextCommandDefault{\textestimated}%
    {\tc@check@symbol3\textestimated}
\DeclareTextCommandDefault{\textcurrency}%
    {\tc@check@symbol3\textcurrency}
```

Nearly all of the remaining glyphs are provided only with fonts with id 1 or 0, i.e., are essentially complete.

```
\DeclareTextCommandDefault{\capitaltie}%
    {\tc@check@accent2\capitaltie}
\DeclareTextCommandDefault{\newtie}%
    {\tc@check@accent2\newtie}
\DeclareTextCommandDefault{\capitalnewtie}%
    {\tc@check@accent2\capitalnewtie}
\DeclareTextCommandDefault{\textleftarrow}%
    {\tc@check@symbol2\textleftarrow}
\DeclareTextCommandDefault{\textrightarrow}%
    {\tc@check@symbol2\textrightarrow}
\DeclareTextCommandDefault{\textblank}%
    {\tc@check@symbol2\textblank}
\DeclareTextCommandDefault{\textdblhyphen}%
    {\tc@check@symbol2\textdblhyphen}
\DeclareTextCommandDefault{\textzerooldstyle}%
    {\tc@check@symbol2\textzerooldstyle}
\DeclareTextCommandDefault{\textoneoldstyle}%
    {\tc@check@symbol2\textoneoldstyle}
\DeclareTextCommandDefault{\texttwooldstyle}%
    {\tc@check@symbol2\texttwooldstyle}
\DeclareTextCommandDefault{\textthreeoldstyle}%
    {\tc@check@symbol2\textthreeoldstyle}
\DeclareTextCommandDefault{\textfouroldstyle}%
    {\tc@check@symbol2\textfouroldstyle}
\DeclareTextCommandDefault{\textfiveoldstyle}%
    {\tc@check@symbol2\textfiveoldstyle}
\DeclareTextCommandDefault{\textsixoldstyle}%
    {\tc@check@symbol2\textsixoldstyle}
\DeclareTextCommandDefault{\textsevenoldstyle}%
    {\tc@check@symbol2\textsevenoldstyle}
\DeclareTextCommandDefault{\texteightoldstyle}%
    {\tc@check@symbol2\texteightoldstyle}
\DeclareTextCommandDefault{\textnineoldstyle}%
    {\tc@check@symbol2\textnineoldstyle}
\DeclareTextCommandDefault{\textlangle}%
    {\tc@check@symbol2\textlangle}
\DeclareTextCommandDefault{\textrangle}%
    {\tc@check@symbol2\textrangle}
\DeclareTextCommandDefault{\textmho}%
    {\tc@check@symbol2\textmho}
\DeclareTextCommandDefault{\textbigcircle}%
    {\tc@check@symbol2\textbigcircle}
\DeclareTextCommandDefault{\textuparrow}%
    {\tc@check@symbol2\textuparrow}
\DeclareTextCommandDefault{\textdownarrow}%
```

```
    {\tc@check@symbol2\textdownarrow}
\DeclareTextCommandDefault{\textborn}%
    {\tc@check@symbol2\textborn}
\DeclareTextCommandDefault{\textdivorced}%
    {\tc@check@symbol2\textdivorced}
\DeclareTextCommandDefault{\textdied}%
    {\tc@check@symbol2\textdied}
\DeclareTextCommandDefault{\textleaf}%
    {\tc@check@symbol2\textleaf}
\DeclareTextCommandDefault{\textmarried}%
    {\tc@check@symbol2\textmarried}
\DeclareTextCommandDefault{\textmusicalnote}%
    {\tc@check@symbol2\textmusicalnote}
\DeclareTextCommandDefault{\textdblhyphenchar}%
    {\tc@check@symbol2\textdblhyphenchar}
\DeclareTextCommandDefault{\textdollaroldstyle}%
    {\tc@check@symbol2\textdollaroldstyle}
\DeclareTextCommandDefault{\textcentoldstyle}%
    {\tc@check@symbol2\textcentoldstyle}
\DeclareTextCommandDefault{\textcolonmonetary}%
    {\tc@check@symbol2\textcolonmonetary}
\DeclareTextCommandDefault{\textwon}%
    {\tc@check@symbol2\textwon}
\DeclareTextCommandDefault{\textnaira}%
    {\tc@check@symbol2\textnaira}
\DeclareTextCommandDefault{\textguarani}%
    {\tc@check@symbol2\textguarani}
\DeclareTextCommandDefault{\textpeso}%
    {\tc@check@symbol2\textpeso}
\DeclareTextCommandDefault{\textlira}%
    {\tc@check@symbol2\textlira}
\DeclareTextCommandDefault{\textrecipe}%
    {\tc@check@symbol2\textrecipe}
\DeclareTextCommandDefault{\textinterrobang}%
    {\tc@check@symbol2\textinterrobang}
\DeclareTextCommandDefault{\textinterrobangdown}%
    {\tc@check@symbol2\textinterrobangdown}
\DeclareTextCommandDefault{\textdong}%
    {\tc@check@symbol2\textdong}
\DeclareTextCommandDefault{\textpertenthousand}%
    {\tc@check@symbol2\textpertenthousand}
\DeclareTextCommandDefault{\textpilcrow}%
    {\tc@check@symbol2\textpilcrow}
\DeclareTextCommandDefault{\textbaht}%
    {\tc@check@symbol2\textbaht}
\DeclareTextCommandDefault{\textnumero}%
    {\tc@check@symbol2\textnumero}
\DeclareTextCommandDefault{\textdiscount}%
    {\tc@check@symbol2\textdiscount}
\DeclareTextCommandDefault{\textopenbullet}%
    {\tc@check@symbol2\textopenbullet}
\DeclareTextCommandDefault{\textservicemark}%
    {\tc@check@symbol2\textservicemark}
\DeclareTextCommandDefault{\textlquill}%
```

```
    {\tc@check@symbol2\textlquill}
\DeclareTextCommandDefault{\textrquill}%
    {\tc@check@symbol2\textrquill}
\DeclareTextCommandDefault{\textcopyleft}%
    {\tc@check@symbol2\textcopyleft}
\DeclareTextCommandDefault{\textcircledP}%
    {\tc@check@symbol2\textcircledP}
\DeclareTextCommandDefault{\textreferencemark}%
    {\tc@check@symbol2\textreferencemark}
\DeclareTextCommandDefault{\textsurd}%
    {\tc@check@symbol2\textsurd}
```

The `\textcircled` and `\t` are handled specially, unless the current font has a subset id of 0 (i.e. full `TS1`) we pick the symbols up from the math font encodings, i.e., the third argument to `\CheckEncodingSubset` uses `\UseTextAccent` to get them from there.

```
\DeclareTextCommandDefault{\textcircled}
  {\CheckEncodingSubset\UseTextAccent{TS1}%
    {\UseTextAccent{OMS}}1\textcircled}
\DeclareTextCommandDefault{\t}
  {\CheckEncodingSubset\UseTextAccent{TS1}%
    {\UseTextAccent{OML}}1\t}
```

Finally input the encoding-specific definitions for `TS1` thus making the top-level definitions optimized for this encoding (and not for the default encoding).

```
\input{ts1enc.def}
```

Now having the new glyphs available we also want to make sure that they are used. For most cases this will automatically happen but for some glyphs there are inferior definitions already known to LaTeX which will prevent the usage of the `TS1` versions. So we better get rid of them:

```
\UndeclareTextCommand{\textsterling}{OT1}
\UndeclareTextCommand{\textdollar}  {OT1}
```

Similar declarations should probably be made for other encodings like `OT4` if they are in use.

```
%\UndeclareTextCommand{\textsterling}{OT4}
%\UndeclareTextCommand{\textdollar}  {OT4}
```

From the `T1` encoding there are two candidates for removal: ‰ and ‱ since these are both constructed from % followed by a tiny ‘o’ rather than being a single glyph. The problem with this approach is that in PostScript fonts this small zero is usually not available resulting in %▪ rather than ‰ while the real glyph (at least for `\textperthousand`) is available in the PostScript version of `TS1`. So for the moment we compromise by removing the `T1` declaration for `\textperthousand` but keeping the one for `\textpertenthousand`. This will have the effect that with Computer Modern fonts everything will come out (although ‰ and ‱ are not taken from the same physical font) and with PostScript fonts ‰ will come out correctly while ‱ will most likely look like %▬ — which is probably an improvement over just getting a single ‘▪’ to indicate a completely missing glyph, which would happen if we also ‘undeclared’ `\textpertenthousand`.

```
\UndeclareTextCommand{\textperthousand}{T1}
%\UndeclareTextCommand{\textpertenthousand}{T1}
```

5.1.1 Supporting oldstyle digits

```
\DeclareRobustCommand\oldstylenums[1]{%
 \begingroup
  \ifmmode
   \mathgroup\symletters #1%
  \else
   \CheckEncodingSubset\@use@text@encoding{TS1}%
       {\PackageWarning{textcomp}%
          {Oldstyle digits unavailable for
           family \f@family.\MessageBreak
           Lining digits used instead}}%
       \tw@{#1}%
   \fi
 \endgroup
}
```

5.1.2 Subset encoding defaults

For many font families commonly used in the TEX world we provide the subset encoding data here. Users can add additional font families in the file `textcomp.cfg` if they own other fonts.

However, if the option "forced" was given then all subset encoding specifications are ignored, so there is no point in setting any of them up:

```
\iftc@forced \else
```

Computer modern based fonts (e.g., CM, CM-Bright, Concrete):

```
\DeclareEncodingSubset{TS1}{cmr}     {0}
\DeclareEncodingSubset{TS1}{cmss}    {0}
\DeclareEncodingSubset{TS1}{cmtt}    {0}
\DeclareEncodingSubset{TS1}{cmvtt}   {0}
\DeclareEncodingSubset{TS1}{cmbr}    {0}
\DeclareEncodingSubset{TS1}{cmtl}    {0}
\DeclareEncodingSubset{TS1}{ccr}     {0}
```

PSNFSS fonts:

```
\DeclareEncodingSubset{TS1}{ptm}     {4}
\DeclareEncodingSubset{TS1}{pcr}     {4}
\DeclareEncodingSubset{TS1}{phv}     {4}
\DeclareEncodingSubset{TS1}{ppl}     {3}
\DeclareEncodingSubset{TS1}{pag}     {4}
\DeclareEncodingSubset{TS1}{pbk}     {4}
\DeclareEncodingSubset{TS1}{pnc}     {4}
\DeclareEncodingSubset{TS1}{pzc}     {4}
\DeclareEncodingSubset{TS1}{bch}     {4}
\DeclareEncodingSubset{TS1}{put}     {5}
```

Other CTAN fonts (probably not complete):

```
\DeclareEncodingSubset{TS1}{uag}     {5}
\DeclareEncodingSubset{TS1}{ugq}     {5}
\DeclareEncodingSubset{TS1}{ul8}     {4}
\DeclareEncodingSubset{TS1}{ul9}     {4}  % (LuxiSans, one day)
\DeclareEncodingSubset{TS1}{augie}   {5}
\DeclareEncodingSubset{TS1}{dayrom}  {3}
\DeclareEncodingSubset{TS1}{dayroms} {3}
\DeclareEncodingSubset{TS1}{pxr}     {0}
```

```
\DeclareEncodingSubset{TS1}{pxss}    {0}
\DeclareEncodingSubset{TS1}{pxtt}    {0}
\DeclareEncodingSubset{TS1}{txr}     {0}
\DeclareEncodingSubset{TS1}{txss}    {0}
\DeclareEncodingSubset{TS1}{txtt}    {0}
```

Latin Modern and TeX Gyre:

```
\DeclareEncodingSubset{TS1}{lmr}     {0}
\DeclareEncodingSubset{TS1}{lmdh}    {0}
\DeclareEncodingSubset{TS1}{lmss}    {0}
\DeclareEncodingSubset{TS1}{lmssq}   {0}
\DeclareEncodingSubset{TS1}{lmvtt}   {0}
\DeclareEncodingSubset{TS1}{lmtt}    {0}
```

```
\DeclareEncodingSubset{TS1}{qhv}     {0}
\DeclareEncodingSubset{TS1}{qag}     {0}
\DeclareEncodingSubset{TS1}{qbk}     {0}
\DeclareEncodingSubset{TS1}{qcr}     {0}
\DeclareEncodingSubset{TS1}{qcs}     {0}
\DeclareEncodingSubset{TS1}{qpl}     {0}
\DeclareEncodingSubset{TS1}{qtm}     {0}
\DeclareEncodingSubset{TS1}{qzc}     {0}
\DeclareEncodingSubset{TS1}{qhvc}    {0}
```

Fourier-GUTenberg:

```
\DeclareEncodingSubset{TS1}{futs}    {4}
\DeclareEncodingSubset{TS1}{futx}    {4}
\DeclareEncodingSubset{TS1}{futj}    {4}
```

Y&Y's Lucida Bright

```
\DeclareEncodingSubset{TS1}{hlh}     {3}
\DeclareEncodingSubset{TS1}{hls}     {3}
\DeclareEncodingSubset{TS1}{hlst}    {3}
```

The remaining settings for Lucida are conservative: the following fonts contain the `\textohm` character but not the `\texteuro`, i.e., belong to neither subset 4 nor subset 3. If you want to use the `\textohm` with these fonts copy these definition to `textcomp.cfg` and change the subset to 3. However in that case make sure that you do not use the `\texteuro`.

```
\DeclareEncodingSubset{TS1}{hlct}    {5}
\DeclareEncodingSubset{TS1}{hlx}     {5}
\DeclareEncodingSubset{TS1}{hlce}    {5}
\DeclareEncodingSubset{TS1}{hlcn}    {5}
\DeclareEncodingSubset{TS1}{hlcw}    {5}
\DeclareEncodingSubset{TS1}{hlcf}    {5}
```

Other commercial families...

```
\DeclareEncodingSubset{TS1}{pplx}    {3}
\DeclareEncodingSubset{TS1}{pplj}    {3}
\DeclareEncodingSubset{TS1}{ptmx}    {4}
\DeclareEncodingSubset{TS1}{ptmj}    {4}
```

If the file `textcomp.cfg` exists it will be loaded at this point. This allows to define further subset encodings for font families not covered by default.

```
\InputIfFileExists{textcomp.cfg}
  {\PackageInfo{textcomp}{Local configuration file used}}{}
```

```
\fi
```
⟨/TS1oldsty⟩

6 The checkencodingsubset.tex file

This is a simple file that asks for a name of a font family and then displays information about the TS1 encoding for this family and recommends the right encoding subset (to be used with `\DeclareEncodingsubset`) for this family.

```
⟨*TS1check⟩
\ProvidesFile{checkencodingsubset.tex}
 [2024/10/18 v0.5b Figure out safe TS1 encoding subsets]
\let\typeoutdetails\typeout
%\def\typeoutdetails#1{}     % alternative definition used below
```

For the purpose of this check a glyph exists if the font slot is occupied—too bad if that contains the wrong glyph or some tofu. If it "exists" we return 0 otherwise 1. This way we can call this macro several times in a row and obtain a number that is 0 if all glyphs are existing or greater than 0 if any of them is missing.

The second argument (holding the command name for a symbol) is not used during these tests.

```
\def\doesglyphexist#1#2{\iffontchar\testFont #1 0\else 1\relax \fi}
```

This macro also tests and outputs some information about the symbol if it is missing. This time we make use of the second argument.

```
\def\glyphmissingdetails#1#2{\iffontchar\testFont #1 \else
  \typeoutdetails{\space\space\space ==> \string#2 (#1) is missing}\fi}
\newif\ifsafesubencodingfound
\newif\ifcoremisses
```

Testing a group of symbols that belong to one sub-encoding. More precisely, the symbols that become unavailable if you change from sub-encoding x (`#2`) to $x+1$ (`#3`). As far as the code is concerned, the symbols that are supposed to be always available (the core) become available if we test the group `-1` and `0`.

The first argument contains the testing code and is supposed to return a single number greater or equal to zero.

```
\def\testgroup#1#2#3{%
  \ifnum 0 = #1%
    \ifnum #2<0
    \typeoutdetails{All glyphs in core exist}%
    \else
    \typeoutdetails{All glyphs between sub-encoding #2 and #3  exist}%
    \fi
  \else
    \ifnum #2<0
      \typeoutdetails{**********************************}%
      \typeoutdetails{Some glyphs are missing from core:}%
      \coremissestrue
      \ifsafesubencodingfound \else
        \def\subencodingresult{#2}%
      \fi
    \else
```

```
      \typeoutdetails{Some glyphs are missing from sub-encoding #2:}%
      \ifsafesubencodingfound \else
        \def\subencodingresult{#3}%
      \fi
    \fi
```

If some glyphs are missing, we rerun the test code but this time using `\glyphmissingdetails`.

```
    {\let\doesglyphexist \glyphmissingdetails #1}%
```

And because we had misses we have definitely found the subset.

```
    \safesubencodingfoundtrue
  \fi
}
```

The currently defined subset for the family is either stored in `\TS1:`⟨*family*⟩ if it was declared, or it is the default subset which is stored in `\TS1:?`.

```
\def\currsubencoding#1{\csname TS1:\ifcsname TS1:#1\endcsname #1\else ?\fi\endcsname}
```

If a font family is not found when declaring it with `\DeclareFixedFont` we end up with the following font. This can then be used as a simple test if we failed loading the TS1 font.

```
\DeclareFixedFont\cmrFont{TS1}{cmr}{m}{n}{10pt}
```

Check for all glyphs in all encoding subsets ...

```
\def\testallgroups#1{%
  \DeclareFixedFont\testFont{TS1}{#1}{m}{n}{10pt}%
  \ifx\testFont\cmrFont
    \typeout{***** Font family #1 not found ****}%
  \else
```

We haven't checked anything yet.

```
    \safesubencodingfoundfalse
    \coremissesfalse
    \typeoutdetails{^^J-----------------------------------------}%
    \typeoutdetails{Testing font family #1^^J(currently TS1-sub-encoding
      \currsubencoding{#1})}%
    \typeout{-----------------------------------------}%
```

Then we start testing the groups beginning with the glyphs between sub-encoding 8 and 9. If any of them is missing (checked with `\doesglyphexist`) then we already know that 9 is the correct answer.

```
    \testgroup{%
       \doesglyphexist{21}{\texttwelveudash}%
       \doesglyphexist{22}{\textthreequartersemdash}%
       \doesglyphexist{134}{\textbardbl}%
       \doesglyphexist{137}{\textcelsius}%
       \doesglyphexist{178}{\texttwosuperior}%
       \doesglyphexist{179}{\textthreesuperior}%
       \doesglyphexist{185}{\textonesuperior}%
    }{8}{9}%
```

Nevertheless we go on with further groups so that the output lists all missing glyphs.

```
    \testgroup{%
       \doesglyphexist{32}{\textblank}%
       \doesglyphexist{148}{\textinterrobang}%
       \doesglyphexist{149}{\textinterrobangdown}%
       \doesglyphexist{191}{\texteuro}%
```

```
}{7}{8}%
\testgroup{%
   \doesglyphexist{47}{\textfractionsolidus}%
   \doesglyphexist{61}{\textminus}%
   \doesglyphexist{87}{\textohm}%
   \doesglyphexist{181}{\textmu}%
}{6}{7}%
\testgroup{%
   \doesglyphexist{140}{\textflorin}%
   \doesglyphexist{164}{\textcurrency}%
}{5}{6}%
\testgroup{%
   \doesglyphexist{155}{\textnumero}%
   \doesglyphexist{157}{\textestimated}%
}{4}{5}%
\testgroup{%
   \doesglyphexist{24}{\textleftarrow}%
   \doesglyphexist{25}{\textrightarrow}%
   \doesglyphexist{94}{\textuparrow}%
   \doesglyphexist{95}{\textdownarrow}%
   \doesglyphexist{141}{\textcolonmonetary}%
   \doesglyphexist{142}{\textwon}%
   \doesglyphexist{146}{\textlira}%
   \doesglyphexist{150}{\textdong}%
}{3}{4}%
\testgroup{%
   \doesglyphexist{60}{\textlangle}%
   \doesglyphexist{62}{\textrangle}%
}{2}{3}%
\testgroup{%
   \doesglyphexist{0}{\capitalgrave}%
   \doesglyphexist{1}{\capitalacute}%
   \doesglyphexist{2}{\capitalcircumflex}%
   \doesglyphexist{3}{\capitaltilde}%
   \doesglyphexist{4}{\capitaldieresis}%
   \doesglyphexist{5}{\capitalhungarumlaut}%
   \doesglyphexist{6}{\capitalring}%
   \doesglyphexist{7}{\capitalcaron}%
   \doesglyphexist{8}{\capitalbreve}%
   \doesglyphexist{9}{\capitalmacron}%
   \doesglyphexist{10}{\capitaldotaccent}%
   \doesglyphexist{11}{\capitalcedilla}%
   \doesglyphexist{12}{\capitalogonek}%
   \doesglyphexist{26}{\t}%
   \doesglyphexist{27}{\capitaltie}%
   \doesglyphexist{28}{\newtie}%
   \doesglyphexist{29}{\capitalnewtie}%
   \doesglyphexist{45}{\textdblhyphen}%
   \doesglyphexist{48}{\textzerooldstyle}%
   \doesglyphexist{49}{\textoneoldstyle}%
   \doesglyphexist{50}{\texttwooldstyle}%
   \doesglyphexist{51}{\textthreeoldstyle}%
   \doesglyphexist{52}{\textfouroldstyle}%
   \doesglyphexist{53}{\textfiveoldstyle}%
```

```
      \doesglyphexist{54}{\textsixoldstyle}%
      \doesglyphexist{55}{\textsevenoldstyle}%
      \doesglyphexist{56}{\texteightoldstyle}%
      \doesglyphexist{57}{\textnineoldstyle}%
      \doesglyphexist{77}{\textmho}%
      \doesglyphexist{79}{\textbigcircle}%
      \doesglyphexist{91}{\textlbrackdbl}%
      \doesglyphexist{93}{\textrbrackdbl}%
      \doesglyphexist{96}{\textasciigrave}%
      \doesglyphexist{98}{\textborn}%
      \doesglyphexist{99}{\textdivorced}%
      \doesglyphexist{100}{\textdied}%
      \doesglyphexist{108}{\textleaf}%
      \doesglyphexist{109}{\textmarried}%
      \doesglyphexist{110}{\textmusicalnote}%
      \doesglyphexist{126}{\texttildelow}%
      \doesglyphexist{127}{\textdblhyphenchar}%
      \doesglyphexist{128}{\textasciibreve}%
      \doesglyphexist{129}{\textasciicaron}%
      \doesglyphexist{175}{\textasciimacron}%
      \doesglyphexist{130}{\textacutedbl}%
      \doesglyphexist{131}{\textgravedbl}%
      \doesglyphexist{138}{\textdollaroldstyle}%
      \doesglyphexist{139}{\textcentoldstyle}%
      \doesglyphexist{143}{\textnaira}%
      \doesglyphexist{144}{\textguarani}%
      \doesglyphexist{145}{\textpeso}%
      \doesglyphexist{147}{\textrecipe}%
      \doesglyphexist{152}{\textpertenthousand}%
      \doesglyphexist{153}{\textpilcrow}%
      \doesglyphexist{154}{\textbaht}%
      \doesglyphexist{156}{\textdiscount}%
      \doesglyphexist{158}{\textopenbullet}%
      \doesglyphexist{159}{\textservicemark}%
      \doesglyphexist{160}{\textlquill}%
      \doesglyphexist{161}{\textrquill}%
      \doesglyphexist{168}{\textasciidieresis}%
      \doesglyphexist{171}{\textcopyleft}%
      \doesglyphexist{173}{\textcircledP}%
      \doesglyphexist{180}{\textasciiacute}%
      \doesglyphexist{184}{\textreferencemark}%
      \doesglyphexist{187}{\textsurd}%
    }{1}{2}%
```

All fonts (up to now) that belong to sub-encoding 1 do have the `\textcircled` glyph, but it is too small to be usable. So this test for this group currently doesn't do much good—but who knows maybe one day a font shows up in which this glyph is actually missing.

```
    \testgroup{%
      \doesglyphexist{79}{\textcircled}%  this is not a proper test because the symbol is
                                          %  usually available but not usable
    }{0}{1}%
    \testgroup{%
      \doesglyphexist{13}{\textquotestraightbase}%
```

```
      \doesglyphexist{18}{\textquotestraightdblbase}%
      \doesglyphexist{23}{\textcapitalcompwordmark}%
      \doesglyphexist{31}{\textascendercompwordmark}%
      \doesglyphexist{36}{\textdollar}%
      \doesglyphexist{39}{\textquotesingle}%
      \doesglyphexist{42}{\textasteriskcentered}%
      \doesglyphexist{132}{\textdagger}%
      \doesglyphexist{133}{\textdaggerdbl}%
      \doesglyphexist{135}{\textperthousand}%
      \doesglyphexist{136}{\textbullet}%
      \doesglyphexist{151}{\texttrademark}%
      \doesglyphexist{162}{\textcent}%
      \doesglyphexist{163}{\textsterling}%
      \doesglyphexist{165}{\textyen}%
      \doesglyphexist{166}{\textbrokenbar}%
      \doesglyphexist{167}{\textsection}%
      \doesglyphexist{169}{\textcopyright}%
      \doesglyphexist{170}{\textordfeminine}%
      \doesglyphexist{172}{\textlnot}%
      \doesglyphexist{174}{\textregistered}%
      \doesglyphexist{176}{\textdegree}%
      \doesglyphexist{177}{\textpm}%
      \doesglyphexist{182}{\textparagraph}%
      \doesglyphexist{183}{\textperiodcentered}%
      \doesglyphexist{186}{\textordmasculine}%
      \doesglyphexist{188}{\textonequarter}%
      \doesglyphexist{189}{\textonehalf}%
      \doesglyphexist{190}{\textthreequarters}%
      \doesglyphexist{214}{\texttimes}%
      \doesglyphexist{246}{\textdiv}%
    }{-1}{0}%
```

If all groups have all glyphs then we have the full encoding (subset 0).

```
    \ifsafesubencodingfound\else
      \def\subencodingresult{0}%
    \fi
```

If the font is missing some of the core glyphs we make a remark about this, because they will never display.

```
    \typeoutdetails{----------------------------------------}%
    \typeout{TS1 encoding subset for #1\ifcoremisses \space(ignoring core misses)\fi
      \space (\ifnum\subencodingresult =
      \currsubencoding{#1} ok\else bad\fi)}%
    \typeout{Use sub-encoding \subencodingresult
      \ifnum\subencodingresult = \currsubencoding{#1}\else
        \space (not \currsubencoding{#1})\fi}
    \typeout{----------------------------------------^^J}%
  \fi
}
```

This tests all declarations (or most of them) that have been added to the kernel. It is called if no family is given interactively.

```
\long\def\testallkerneldefinedfamilies{%
\testallgroups{ccr}%     {0}
\testallgroups{cmbr}%    {0}
```

```
%%\testallgroups{cmr}%   {0}  % don't test this one as it is the fallback
                              % thus reports that the family is not found
\testallgroups{cmss}%    {0}
\testallgroups{cmtl}%    {0}
\testallgroups{cmtt}%    {0}
\testallgroups{cmvtt}%   {0}
\testallgroups{pxr}%     {0}
\testallgroups{pxss}%    {0}
\testallgroups{pxtt}%    {0}
\testallgroups{qag}%     {0}
\testallgroups{qbk}%     {0}
\testallgroups{qcr}%     {0}
\testallgroups{qcs}%     {0}
\testallgroups{qhvc}%    {0}
\testallgroups{qhv}%     {0}
\testallgroups{qpl}%     {0}
\testallgroups{qtm}%     {0}
\testallgroups{qzc}%     {0}
\testallgroups{txr}%     {0}
\testallgroups{txss}%    {0}
\testallgroups{txtt}%    {0}
%
% Next would claim to be 0 (or 2)
%
%\testallgroups{lmr}%     {1}
%\testallgroups{lmdh}%    {1}
%\testallgroups{lmss}%    {1}
%\testallgroups{lmssq}%   {1}
%\testallgroups{lmvtt}%   {1}
%\testallgroups{lmtt}%    {1} % missing TM, SM and pertenthousand so really 2
%
% these are no longer in TeX Live
%
%\testallgroups{ptmx}%    {2} % gone for a long time it seems
%\testallgroups{ptmj}%    {2} % ditto
%\testallgroups{ul8}%     {2} % ditto
%
% next block has tofu chars so results are wrong
%
%\testallgroups{bch}% {5} % tofu for blank, ohm
%\testallgroups{futj}%{5} % tofu for blank, interrobang/down, ohm
%\testallgroups{futs}%{5} % tofu for blank, ohm
%\testallgroups{futx}%{5} % probably (currently broken distrib)
%\testallgroups{pag}% {5} % tofu for blank, interrobang/down, ohm
%\testallgroups{pbk}% {5} % tofu for blank, interrobang/down, ohm
%\testallgroups{pcr}% {5} % tofu for blank, interrobang/down, ohm
%\testallgroups{phv}% {5} % tofu for blank, interrobang/down, ohm
%\testallgroups{pnc}% {5} % tofu for blank, interrobang/down, ohm
%\testallgroups{pplj}%{5} % tofu for blank
%\testallgroups{pplx}%{5} % tofu for blank
%\testallgroups{ppl}% {5} % tofu for blank interrobang/down
%\testallgroups{ptm}% {5} % tofu for blank, interrobang/down, ohm
%\testallgroups{pzc}% {5} % tofu for blank, interrobang/down, ohm
%\testallgroups{ul9}% {5} % tofu for blank, interrobang/down, ohm
```

```
%\testallgroups{dayroms}%{6} % tofu for blank, interrobang/down, ohm
%\testallgroups{dayrom}% {6} % tofu for blank, interrobang/down, ohm
%\testallgroups{augie}%{8} % really only missing euro and full of tofu
%\testallgroups{put}%  {8}
%\testallgroups{uag}%  {8} % probably (currently broken distrib)
%\testallgroups{ugq}%  {8}
%
\testallgroups{zi4}%  {9}
%
%% not installed normally
%
%\testallgroups{hls}%  {5}
%\testallgroups{hlst}% {5}
%\testallgroups{hlct}% {5}
%\testallgroups{hlh}%  {5}
%\testallgroups{hlx}%  {8}
%\testallgroups{hlce}% {8}
%\testallgroups{hlcn}% {8}
%\testallgroups{hlcw}% {8}
%\testallgroups{hlcf}% {8}

\testallgroups{lato-LF}%   {0}  % with a bunch of tofu inside --- should probably be changed
\testallgroups{opensans-TLF}%{0}  % with a bunch of tofu inside --- should probably be change
\testallgroups{cantarell-TLF}%  {0}  % with a bunch of tofu inside --- should probably be cha
\testallgroups{fbb-LF}%     {0}  % missing centoldstyle ---> 2
\testallgroups{tli}%       {1}  % with lots of tofu inside --- should probably be changed
\testallgroups{Alegreya-OsF}%       {2}
\testallgroups{AlegreyaSans-OsF}%   {2}
\testallgroups{DejaVuSans-TLF}%     {2}
\testallgroups{DejaVuSansCondensed-TLF}%  {2}
\testallgroups{DejaVuSansMono-TLF}% {2} this is missing \textfractionsolidus which makes it 7
\testallgroups{EBGaramond-LF}%      {2}
\testallgroups{Tempora-TLF}%        {2}
\testallgroups{Tempora-TOsF}%       {2}
\testallgroups{Arimo-TLF}%          {3}
\testallgroups{Crlt-TLF}%           {3}       changed from Carlito-
\testallgroups{FiraSans-LF}%        {3}     should be 4
\testallgroups{IBMPlexSans-TLF}%    {3}
\testallgroups{Merriwthr-OsF}%   {3}         changed from Merriweather- and should be 2
\testallgroups{Montserrat-LF}%      {3}     now 2
\testallgroups{MontserratAlternates-LF}%{3}     now 2
\testallgroups{SourceCodePro-TLF}%  {3}
\testallgroups{SourceCodePro-TOsF}% {3}
\testallgroups{SourceSansPro-OsF}%  {3}
\testallgroups{SourceSerifPro-LF}%  {3}
\testallgroups{Tinos-TLF}%          {3}
\testallgroups{AccanthisADFStdNoThree-LF}%{4}
\testallgroups{Cabin-TLF}%          {4}
\testallgroups{Caladea-TLF}%        {4}
\testallgroups{Chivo-LF}%           {4}
\testallgroups{ClearSans-TLF}%      {4}
\testallgroups{Coelacanth-LF}%      {4}
\testallgroups{CrimsonPro-LF}%      {4}
\testallgroups{FiraMono-TLF}%       {4}
```

```
\testallgroups{FiraMono-TOsF}%      {4}
\testallgroups{Go-TLF}%             {4}
\testallgroups{GoMono-TLF}%         {4}
\testallgroups{InriaSans-LF}%       {4}
\testallgroups{InriaSerif-LF}%      {4}
\testallgroups{LibertinusSans-LF}%  {4}
\testallgroups{LibertinusSerif-LF}% {4}
\testallgroups{LibreBodoni-TLF}%    {4}
\testallgroups{LibreFranklin-TLF}%  {4}
\testallgroups{LinguisticsPro-LF}%  {4}
\testallgroups{LinguisticsPro-OsF}% {4}
\testallgroups{LinuxBiolinumT-LF}%  {4}
\testallgroups{LinuxLibertineT-LF}% {4}
\testallgroups{MerriwthrSans-OsF}%  {4}          name change and now 2
\testallgroups{MintSpirit-LF}%      {4}
\testallgroups{MintSpiritNoTwo-LF}% {4}
\testallgroups{PTMono-TLF}%         {4}
\testallgroups{PTSans-TLF}%         {4}
\testallgroups{PTSansCaption-TLF}%  {4}
\testallgroups{PTSansNarrow-TLF}%   {4}
\testallgroups{PTSerif-TLF}%        {4}
\testallgroups{PTSerifCaption-TLF}% {4}
\testallgroups{Raleway-TLF}%        {4}
\testallgroups{Raleway-TOsF}%       {4}
\testallgroups{Roboto-LF}%          {4}
\testallgroups{RobotoMono-TLF}%     {4}
\testallgroups{RobotoSlab-TLF}%     {4}
\testallgroups{Rosario-LF}%         {4}
\testallgroups{SticksTooText-LF}%   {4}
\testallgroups{UniversalisADFStd-LF}%{4}
\testallgroups{Almndr-OsF}%       {5}     name change
\testallgroups{Baskervaldx-LF}%   {5}
\testallgroups{BaskervilleF-LF}%  {5}   now 2
\testallgroups{Bttr-TLF}%         {5}   name changed from Bitter-...
\testallgroups{Cinzel-LF}%        {5}
\testallgroups{CinzelDecorative-LF}%{5}
\testallgroups{DejaVuSerif-TLF}%  {5}
\testallgroups{DejaVuSerifCondensed-TLF}% {5}
\testallgroups{GilliusADF-LF}%    {5}
\testallgroups{charssil-TLF} %%  missing should be 5
\testallgroups{GilliusADFCond-LF}%{5}
\testallgroups{GilliusADFNoTwo-LF}%{5}
\testallgroups{GilliusADFNoTwoCond-LF}%{5}
\testallgroups{Lbstr-LF}%      {5}      name change  and should be 7
\testallgroups{OldStandard-TLF}%  {5}
\testallgroups{PlyfrDisplay-LF}%{5}       name change
\testallgroups{PlyfrDisplay-OsF}% {5}       name change
\testallgroups{TheanoDidot-TLF}%  {5}
\testallgroups{TheanoDidot-TOsF}% {5}
\testallgroups{TheanoModern-TLF}% {5}
\testallgroups{TheanoModern-TOsF}%{5}
\testallgroups{TheanoOldStyle-TLF}%{5}
\testallgroups{TheanoOldStyle-TOsF}%{5}
\testallgroups{Crimson-TLF}%      {6}
```

```
\testallgroups{IBMPlexMono-TLF}%  {6}  now 3
\testallgroups{IBMPlexSerif-TLF}% {6}  now 3
\testallgroups{LibertinusMono-TLF}%{6}  should be 8
\testallgroups{LibertinusSerifDisplay-LF}%{6}
\testallgroups{LinuxLibertineDisplayT-LF}%{6}
\testallgroups{LinuxLibertineMonoT-LF}%{6}
\testallgroups{LinuxLibertineMonoT-TLF}%{6}
\testallgroups{Ovrlck-LF}%       {6}     name changed
\testallgroups{CormorantGaramond-LF}%{7}
\testallgroups{Heuristica-TLF}%  {7}
\testallgroups{Heuristica-TOsF}% {7}
\testallgroups{IMFELLEnglish-TLF}%{7}
\testallgroups{LibreBskvl-LF}%   {7}  %%  wrong name LibreBaskerville-TLF
\testallgroups{LibreCsln-LF}%    {7}  changed from LibreCaslon-
\testallgroups{Mrcls-LF}%        {7}       %%  wrong name Marcellus-LF
\testallgroups{NotoSans-LF}%     {7}
\testallgroups{NotoSansMono-TLF}%{7}  now 2
\testallgroups{NotoSansMono-TOsF}%{7}  now 2
\testallgroups{NotoSerif-LF}%    {7}
\testallgroups{Quattro-LF}%      {7}  changed from Quattrocento-
\testallgroups{QuattroSans-LF}%  {7}  changed from QuattrocentoSans-
\testallgroups{XCharter-TLF}%    {7}  now 2
\testallgroups{XCharter-TOsF}%   {7}  now 2
\testallgroups{erewhon-LF}%      {7}  now 2
\testallgroups{ComicNeue-TLF}%   {7}
\testallgroups{ComicNeueAngular-TLF}%{7}
\testallgroups{Frm-LF}%{7}  % the superiors are missing; name changed from Forum-LF
\testallgroups{Cochineal-TLF}%   {8} now 5
\testallgroups{AlgolRevived-TLF}%{9}
}
```

There interaction with the user.

```
\typeout{^^J=========================================================================}
\typeout{| Enter font family to check (or <enter> for kernel defined families)}
\typeout{=========================================================================}
\typein[\FontFamilyToCheck]{}

\if!\FontFamilyToCheck!
  \typeout{=========================================================================}
  \typeout{| Detailed output? (default no)}
  \typeout{=========================================================================}
  \typein[\Details]{}
  \if!\Details!
    \def\typeoutdetails#1{}
  \else
    \let\typeoutdetails\typeout
  \fi
  \testallkerneldefinedfamilies
\else
  \let\typeoutdetails\typeout
  \testallgroups\FontFamilyToCheck
\fi

\stop
```

⟨/TS1check⟩

File 34
ltpageno.dtx

1 Page Numbering

Page numbers are produced by a page counter, used just like any other counter. The only difference is that `\c@page` contains the number of the next page to be output (the one currently being produced), rather than one minus it. Thus, it is normally initialized to 1 rather than 0. `\c@page` is defined to be `\count0`, rather than a count assigned by `\newcount`.

`\pagenumbering` The user sets the page number style with the `\pagenumbering{`⟨*foo*⟩`}` command, which sets the page counter to 1 and defines `\thepage` to be `\foo`. For example, `\pagenumbering{roman}` causes pages to be numbered i, ii, etc.

```
⟨*2ekernel⟩
\message{page nos.,}
\countdef\c@page=0 \c@page=1
\def\cl@page{}
\def\pagenumbering#1{%
  \global\c@page \@ne \gdef\thepage{\csname @#1\endcsname
   \c@page}}
⟨/2ekernel⟩
```

File 35
ltxref.dtx

1 Cross Referencing

The user writes `\label{`⟨*foo*⟩`}` to define the following cross-references:

`\ref*{`⟨*foo*⟩`}`: value of most recently incremented referenceable counter. in the current environment. (Chapter, section, theorem, footnote and enumeration counters and other counters stepped with `\refstepcounter` are referenceable.)

`\pageref*{`⟨*foo*⟩`}`: page number at which `\label{foo}` command appeared. where foo can be any string of characters not containing '`\`', '`{`' or '`}`'.

Note: The scope of the `\label` command is delimited by environments, so
`\begin{theorem} \label{foo} ... \end{theorem} \label{bar}`
defines `\ref{foo}` to be the theorem number and `\ref{bar}` to be the current section number.

Note: `\label` does the right thing in terms of spacing – i.e., leaving a space on both sides of it is equivalent to leaving a space on either side.

Note: the starred versions `\ref*` and `\pageref*` are provided to align with the use of hyperref. Without hyperref (or some other package using the starred form) the star is simply ignored.

Note: starting with 2023-06-01 `\label` stores also the current value of `\@currentlabelname` which should typically contain a (sanitized) title. (A reference command `\nameref` is provided by the nameref package.) `\label` also stores `\@currentHref` which if set should refer to a target name for links. This value is set and used by hyperref. Unlike the other values `\@currentHref` should be set globally. A fifth value `\@kernel@reserved@label@data` is reserved for the kernel to allow future extensions of the cross-reference system.

1.1 Cross Referencing

Historical LaTeX 2.09 comments (not necessarily accurate any more):

```
⟨*2ekernel⟩
\message{x-ref,}
```

This is implemented as follows. A referenceable counter CNT is incremented by the command `\refstepcounter{`CNT`}` , which sets `\@currentlabel` == {CNT}{eval(`\p@cnt\theCNT)}`. The command `\label{`FOO`}` then writes the following on file `\@auxout` :

`\newlabel{`FOO`}{{`eval(`\@currentlabel)}{`eval(`\thepage)}%`

`{`eval(`\@currentlabelname)}{`eval(`\@currentHref)}{`eval(`\@kernel@reserved@label@data)}}`

`\ref{`FOO`}` ==
BEGIN
if `\r@foo` undefined
then @refundefined := G T
??
Warning: 'reference foo on page ... undefined'

```
      else  \@car \eval(\r@FOO)\@nil
    fi
  END

\pageref{foo} =
  BEGIN
    if \r@foo undefined
      then  @refundefined := G T
            ??
            Warning: 'reference foo on page ... undefined'
      else  \@cdr \eval(\r@FOO)\@nil
    fi
  END
```

End of historical LaTeX 2.09 comments.

`\labelformat` A reference via `\ref` produces by default the data associated with the corresponding `\label` command (typically a number); any additional formatting has to be provided by the user. If, for example, references to equations are always to be typeset as "equation (*number*)", one has to code "`equation (\ref{`*key*`})`". With `\labelformat` there is a possibility to generate such frills automatically without resorting to low-level coding. The command takes two arguments: the first is the name of a counter and the second is its representation when referenced. This means that for a successful usage, one has to know the counter name being used for generating the label, though in practice this should not pose a problem. The current counter number is picked up as an argument. Here are two examples:

```
\labelformat{section}{section~#1}
\labelformat{equation}{equation~(#1)}}
```

`\Ref` A side effect of using `\labelformat` is that, depending on the defined formatting, it becomes impossible to use `\ref` at the beginning of a sentence (if its replacement text starts with a lowercase letter). To overcome this problem we introduce the command `\Ref` that behave like `\ref` except that it uppercases the first token of the generated string.

To make `\Ref` work properly the very first token in the second argument of `\labelformat` has to be a simple ASCII or UTF-8 letter, otherwise the capitalization will fail or worse, you will end up with some error messages. If you actually need something more complicated in this place (e.g., an accented letter not written as a UTF-8 character) you have to explicitly surround it with braces, to identify the part that needs to be capitalized. For example, for figure references in the Hungarian language you might want to write `\labelformat{figure}{{\'a}bra~\thefigure}` or use `\labelformat{figure}{ábra~\thefigure}` which avoids the brace problem.

`\G@refundefinedtrue` `\@refundefined` This does not save on name-space (since `\G@refundefinedfalse` was never needed) but it does make the implementation of such one-way switches more consistent. The extra macro to make the change is used since this change appears several times.

Note despite its name, `\G@refundefinedtrue` does *not* correspond to an `\if` command, and there is no matching `...false`. It would be more natural to call the command `\G@refundefined` (as inspection of the change log will reveal) but unfortunately such a change would break any package that had defined a `\ref`-like command that mimicked the definition of `\ref`, calling `\G@refundefinedtrue`. Inspection of the TeX archives

revealed several such packages, and so this command has been named ...`true` so that the definition of `\ref` need not be changed, and the packages will work without change.

```
% \newif\ifG@refundefined
% \def\G@refundefinedtrue{\global\let\ifG@refundefined\iftrue}
% \def\G@refundefinedfalse{\global\let\ifG@refundefined\iffalse}
\def\G@refundefinedtrue{%
  \gdef\@refundefined{%
    \@latex@warning@no@line{There were undefined references}}}
\let\@refundefined\relax
```

(*End of definition for* `\G@refundefinedtrue` *and* `\@refundefined`.)

`\ref` `\pageref` `\@setref`

Referencing a `\label`. RmS 91/10/25: added a few extra `\reset@font`, as suggested by Bernd Raichle

RmS 92/08/14: made `\ref` and `\pageref` robust

RmS 93/09/08: Added setting of refundefined switch.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2023/06/01}%
⟨latexrelease⟩                  {\@kernel@sref}{store five arguments}%
\def\@setref#1#2#3{%
  \ifx#1\relax
   \protect\G@refundefinedtrue
   \nfss@text{\reset@font\bfseries ??}%
   \@latex@warning{Reference `#3' on page \thepage \space
            undefined}%
  \else
   \expandafter#2#1\@empty\@empty\@empty\null
  \fi}
\long\def\@firstoffive#1#2#3#4#5{#1}
\long\def\@secondoffive#1#2#3#4#5{#2}
\def\@kernel@sref#1{\expandafter\@setref\csname r@#1\endcsname\@firstoffive{#1}}
\def\@kernel@spageref#1{\expandafter\@setref\csname r@#1\endcsname
                \@secondoffive{#1}}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2022/06/01}%
⟨latexrelease⟩                  {\@kernel@sref}{store five arguments}%
⟨latexrelease⟩\def\@setref#1#2#3{%
⟨latexrelease⟩  \ifx#1\relax
⟨latexrelease⟩   \protect\G@refundefinedtrue
⟨latexrelease⟩   \nfss@text{\reset@font\bfseries ??}%
⟨latexrelease⟩   \@latex@warning{Reference `#3' on page \thepage \space
⟨latexrelease⟩            undefined}%
⟨latexrelease⟩  \else
⟨latexrelease⟩   \expandafter#2#1\null
⟨latexrelease⟩  \fi}
⟨latexrelease⟩\let\@firstoffive\undefined
⟨latexrelease⟩\let\@secondoffive\undefined
⟨latexrelease⟩\def\@kernel@sref#1{\expandafter\@setref\csname r@#1\endcsname\@firstoftwo{#1}}
⟨latexrelease⟩\def\@kernel@spageref#1{\expandafter\@setref\csname r@#1\endcsname
⟨latexrelease⟩                  \@secondoftwo{#1}}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
```

```
⟨latexrelease⟩                  {\@kernel@sref}{store five arguments}%
⟨latexrelease⟩\def\@setref#1#2#3{%
⟨latexrelease⟩  \ifx#1\relax
⟨latexrelease⟩   \protect\G@refundefinedtrue
⟨latexrelease⟩   \nfss@text{\reset@font\bfseries ??}%
⟨latexrelease⟩   \@latex@warning{Reference `#3' on page \thepage \space
⟨latexrelease⟩              undefined}%
⟨latexrelease⟩  \else
⟨latexrelease⟩   \expandafter#2#1\null
⟨latexrelease⟩  \fi}
⟨latexrelease⟩\let\@firstoffive\undefined
⟨latexrelease⟩\let\@secondoffive\undefined
⟨latexrelease⟩\let\@kernel@sref\undefined
⟨latexrelease⟩\let\@kernel@spageref\undefined
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2022/06/01}%
⟨latexrelease⟩                  {\ref}{Add starred reference commands}%

\let\@kernel@ref\@kernel@sref
\let\@kernel@pageref\@kernel@spageref
\NewDocumentCommand\ref{s}
   {\IfBooleanTF{#1}{\@kernel@sref}{\@kernel@ref}}
\NewDocumentCommand\pageref{s}
   {\IfBooleanTF{#1}{\@kernel@spageref}{\@kernel@pageref}}
```

As the commands are now protected we also need expandable versions for use in `\ifthenelse`:

```
\def\@kernel@pageref@exp#1{\csname cs_if_exist:cTF\endcsname
   {r@#1}{\csname tl_item:cn\endcsname{r@#1}{2}}{0}}
\def\@kernel@ref@exp#1{\csname cs_if_exist:cTF\endcsname
   {r@#1}{\csname tl_item:cn\endcsname{r@#1}{1}}{0}}

⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\ref}{Add starred reference commands}%
⟨latexrelease⟩\def\ref#1{\expandafter\@setref\csname r@#1\endcsname\@firstoftwo{#1}}
⟨latexrelease⟩\def\pageref#1{\expandafter\@setref\csname r@#1\endcsname
⟨latexrelease⟩                                  \@secondoftwo{#1}}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\ref`, `\pageref`, *and* `\@setref`.)

`\newlabel` This command will be written to the `.aux` file to pass label information from one run to another.

`\@newl@bel` The internal form of `\newlabel` and `\bibcite`. Note that this macro does it's work inside a group. That way the local assignments it needs to do don't clutter the save stack. This prevents large documents with many labels to run out of save stack.

```
\def\@newl@bel#1#2#3{{%
  \@ifundefined{#1@#2}%
    \relax
    {\gdef \@multiplelabels {%
```

```
      \@latex@warning@no@line{There were multiply-defined labels}}%
    \@latex@warning@no@line{Label `#2' multiply defined}}%
  \global\@namedef{#1@#2}{#3}}}
\def\newlabel{\@newl@bel r}
\@onlypreamble\@newl@bel
```

(*End of definition for* `\newlabel` *and* `\@newl@bel`.)

`\if@multiplelabels` `\@multiplelabels`

This is redefined to produce a warning if at least one label is defined more than once. It is executed by the `\enddocument` command.

```
\let \@multiplelabels \relax
```

(*End of definition for* `\if@multiplelabels` *and* `\@multiplelabels`.)

`\label`

The commands `\label` and `\refstepcounter` have been changed to allow `\protect`'ed commands to work properly. For example,

```
\def\thechapter{\protect\foo{\arabic{chapter}.\roman{section}}}
```

will cause a `\label{bar}` command to define `\ref{bar}` to expand to something like `\foo{4.d}`. Change made 20 Jul 88.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2026/06/01}%
⟨latexrelease⟩                  {\label}{disable commands while writing}%
\def\label#1{\@bsphack
  \begingroup
  \UseHookWithArguments{label}{1}{#1}%
  \protected@write\@auxout{%
```

`\@currentlabelname` can contain a `\label` command and this can loop (gh1841). We disable the commands that are also disabled when writing `\addtocontents`.

```
    \let\label\@gobble@om
    \let\index\@gobble@som
    \let\glossary\@gobble@om}%
        {\string\newlabel{#1}{{\@currentlabel}{\thepage}%
         {\@currentlabelname}{\@currentHref}{\@kernel@reserved@label@data}}}%
  \endgroup
  \@esphack}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2023/06/01}%
⟨latexrelease⟩                  {\label}{store five label arguments}%
\providecommand\@currentlabelname{}
\providecommand\@currentHref{}
\providecommand\@kernel@reserved@label@data{}
\NewHookWithArguments{label}{1}
⟨latexrelease⟩\def\label#1{\@bsphack
⟨latexrelease⟩  \begingroup
⟨latexrelease⟩  \UseHookWithArguments{label}{1}{#1}%
⟨latexrelease⟩  \protected@write\@auxout{}%
⟨latexrelease⟩          {\string\newlabel{#1}{{\@currentlabel}{\thepage}%
⟨latexrelease⟩           {\@currentlabelname}{\@currentHref}{\@kernel@reserved@label@data}}}%
⟨latexrelease⟩  \endgroup
⟨latexrelease⟩  \@esphack}
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\label}{store five label arguments}%
⟨latexrelease⟩\def\label#1{\@bsphack
⟨latexrelease⟩  \protected@write\@auxout{}%
⟨latexrelease⟩          {\string\newlabel{#1}{{\@currentlabel}{\thepage}}}%
⟨latexrelease⟩  \@esphack}
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\label`. *This function is documented on page* 854.)

`\refstepcounter` Step the counter and allow for labels to point to its current value.

```
⟨latexrelease⟩\IncludeInRelease{2024/11/01}%
⟨latexrelease⟩                  {\refstepcounter}{set theHcounter representation}%
```

`refstepcounter` (*socket*) This socket takes the whole code as argument. The default kernel plug is identity. By changing the plug hyperref can add a conditional and e.g. suppress the processing in a PDF context.

```
\NewSocket{refstepcounter}{1}
```

`refstepcounter/target` (*socket*) This socket takes an argument, the counter name, and should at least set from it the target name `\@currentHref`. With hyperref it sets also the actual target. This is done with a socket so that the target name is not set more than once to (possibly) different names. The socket is not used in `\@kernel@refstepcounter`. The tagging code needs the target name so it is added after this socket.

```
\NewSocket{refstepcounter/target}{1}
```

`(refstepcounter/target)` (*plug*)

```
\NewSocketPlug{refstepcounter/target}{kernel}
 {\xdef\@currentHref {#1.\csname theH#1\endcsname}}%
\AssignSocketPlug{refstepcounter/target}{kernel}
```

A default definition for `\@currentcounter`.

`\@currentcounter`

```
\def\@currentcounter{}
```

Saved `*` for testing the argument of `\refstepcounter`. This was implemented as part of a hotfix which is why we have it in 2024/11 rollback.

`\ltx@star@counter`

```
\def\ltx@star@counter{*}

\def\refstepcounter#1{%
  \UseSocket{refstepcounter}{%
    \stepcounter{#1}%
    \edef\reserved@a{#1}%
    \ifx\reserved@a\ltx@star@counter\else
      \let\@currentcounter\reserved@a
    \fi
    \protected@edef\@currentlabel
```

By generating the second csname first the `\p@...` command can grab it as an argument which can be helpful for more complicated typesetting arrangements.

The trick is to ensure that `\csname the#1\endcsname` is turned into a single token before `\p@...` is expanded further. This way, if the `\p@...` command is a macro with one argument it will receive `\the...`. With the original kernel code (i.e., without the `\expandafter`) it will instead pick up `\csname` which would be disastrous.

Using `\expandafter` instead of braces delimiting the argument is better because, assuming that the `\p@...` command is not defined as a macro with one argument, the braces will stay and prohibit kerning that might otherwise happen between the glyphs generated by `\the...` and surrounding glyphs.

```
      {\csname p@#1\expandafter\endcsname\csname the#1\endcsname}%
    \UseSocket{refstepcounter/target}{#1}%
    \UseTaggingSocket{recordtarget}%
  }%
}
```

This is a version of `\refstepcounter` which does not set and use targets.

`\@kernel@refstepcounter`

```
\def\@kernel@refstepcounter#1{%
  \UseSocket{refstepcounter}{%
    \stepcounter{#1}%
    \edef\reserved@a{#1}%
    \ifx\reserved@a\ltx@star@counter\else
      \let\@currentcounter\reserved@a
    \fi
    \protected@edef\@currentlabel
       {\csname p@#1\expandafter\endcsname\csname the#1\endcsname}}}%

⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2022/06/01}%
⟨latexrelease⟩                  {\refstepcounter}{set theHcounter representation}%
⟨latexrelease⟩\def\refstepcounter#1{\stepcounter{#1}%
⟨latexrelease⟩    \edef\@currentcounter{#1}%
⟨latexrelease⟩    \protected@edef\@currentlabel
⟨latexrelease⟩      {\csname p@#1\expandafter\endcsname\csname the#1\endcsname}%
⟨latexrelease⟩}
⟨latexrelease⟩\let\@kernel@refstepcounter\refstepcounter
⟨latexrelease⟩\let\ltx@star@counter\@undefined
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\refstepcounter}{Add starred version}%
⟨latexrelease⟩\def\@currentcounter{}
⟨latexrelease⟩\def\refstepcounter#1{\stepcounter{#1}%
⟨latexrelease⟩    \edef\@currentcounter{#1}%
⟨latexrelease⟩    \protected@edef\@currentlabel
⟨latexrelease⟩       {\csname p@#1\expandafter\endcsname\csname the#1\endcsname}%
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                  {\refstepcounter}{Support for \labelformat}%
⟨latexrelease⟩\let\@currentcounter\@undefined
⟨latexrelease⟩\def\refstepcounter#1{\stepcounter{#1}%
⟨latexrelease⟩    \protected@edef\@currentlabel
⟨latexrelease⟩      {\csname p@#1\expandafter\endcsname\csname the#1\endcsname}%
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\refstepcounter}{Original from 2.09}%
⟨latexrelease⟩\def\refstepcounter#1{\stepcounter{#1}%
```

```
⟨latexrelease⟩    \protected@edef\@currentlabel
⟨latexrelease⟩       {\csname p@#1\endcsname\csname the#1\endcsname}%
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\refstepcounter` *and others.*)

```
⟨latexrelease⟩\IncludeInRelease{2022/06/01}%
⟨latexrelease⟩                  {\Ref}{Add starred version}%
```

`\labelformat` A shortcut to set the `\p@...` macro for a counter. It will pick up the counter representation as an argument so that it can be specially formatted.

```
\def\labelformat#1{\expandafter\def\csname p@#1\endcsname##1}
```

(*End of definition for* `\labelformat`.)

`\Ref` This macro expands the result of `\ref` and then uppercases the first token. Only useful if the label was generated via `\labelformat` and contains some lower case letter at its start. If the label starts with a complicated construct (e.g., an accented letter that is provided via a command, e.g., `\"a` instead of a UTF-8 character like ä) one has to surround everything that needs uppercasing in a brace group in the definition of `\labelformat`.[40]

```
%  \changes{v1.1s}{2024/12/10}{Replace \cs{@tempa} with \cs{reserved@a} (gh/1579)}
\def\@kernel@Ref#1{\protected@edef\reserved@a{\@kernel@ref{#1}}%
       \expandafter\MakeUppercase\reserved@a}
\def\@kernel@sRef#1{\protected@edef\reserved@a{\@kernel@sref{#1}}%
       \expandafter\MakeUppercase\reserved@a}
\NewDocumentCommand\Ref{s}
   {\IfBooleanTF{#1}{\@kernel@sRef}{\@kernel@Ref}}
```

(*End of definition for* `\Ref`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\Ref}{Add starred version}%
⟨latexrelease⟩\def\labelformat#1{\expandafter\def\csname p@#1\endcsname##1}
⟨latexrelease⟩\DeclareRobustCommand\Ref[1]{\protected@edef\reserved@a{\ref{#1}}%
⟨latexrelease⟩   \expandafter\MakeUppercase\reserved@a}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                  {\Ref}{Add \labelformat and \Ref}%
⟨latexrelease⟩\def\labelformat#1{\expandafter\def\csname p@#1\endcsname##1}
⟨latexrelease⟩\DeclareRobustCommand\Ref[1]{\protected@edef\reserved@a{\ref{#1}}%
⟨latexrelease⟩   \expandafter\MakeUppercase\reserved@a}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\Ref}{Add \labelformat and \Ref}%
⟨latexrelease⟩
⟨latexrelease⟩\let\labelformat\@undefined
⟨latexrelease⟩\let\Ref\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

[40] There is one problem with this approach: the braces are kept in a normal `\ref` which might spoil kerning. Perhaps one day this needs redoing.

`\@currentlabel` Default for `\label` commands that come before any environment.

```
\def\@currentlabel{}
```

(*End of definition for* `\@currentlabel`.)

⟨/2ekernel⟩

File 36
ltproperties.dtx

Abstract

This code implements command to record and (expandably) reference document properties. It extends the standard `\label`/`\ref`/`\pageref` commands.

1 Introduction

The module allows to record the "current state" of various document properties (typically the content of macros and values of counters) and to access them in other places through a label. The list of properties that can be recorded and retrieved are not fix and can be extended by the user. The values of the properties are recorded in the `.aux` file and can be retrieved at the second compilation.

The module uses the ideas of properties and labels. A label is a document reference point: a name for the user. An property is something that LaTeX can track, such as a page number, section number or name. The names of labels and properties may be arbitrary. Note that there is a single namespace for each.

2 Design discussion

The design here largely follows ideas from zref. In particular, there are two independent concepts: properties that can be recorded between runs, and labels which consist of lists of these properties. The reason for the split is that individual labels will want to record some but not all properties. For examples, a label concerned with position would track the x and y coordinates of the current point, but not for example the page number.

In the current implementation, properties share a single namespace. This allows multiple lists to re-use the same properties, for example page number, absolute page number, etc. This does mean that *changing* a standard property is an issue. However, some properties have complex definitions (again, see zref at present): having them in a single shared space avoids the need to copy code.

Labels could be implemented as `prop` data. That is not done at present as there is no obvious need to map to or copy the data. As such, faster performance is available using a hash table approach as in a "classical" set up. Data written to the `.aux` file uses simple paired *balanced text* not keyvals: this avoids any restrictions on names and again offers increased performance.

The expl3 versions of the label command do not use `\@bsphack`/`\@esphack` to avoid double spaces, but the LaTeX 2_ε command does as it lives at the document command level.

The reference commands are expandable.

Currently the code has nearly no impact on the main `\label` and `\ref` commands as too many external packages rely on the concrete implementation. There is one exception: the label names share the same namespace. That means that if both `\label{ABC}` and `\RecordProperties{ABC}{page}` are used there is a warning `Label 'ABC' multiply defined`.

3 Handling unknown labels and properties

With the standard `\label`/`\ref` commands the requested label is either in the `.aux`-file (and so known) or not. In the first case the stored value can be used, in the second case the reference commands print two question marks.

With flexible property lists a reference commands asks for the value of a specific property stored under a label name and we have to consider more variants:

- If the requested property is unknown (not declared) the system is not correctly set up and an error is issued.
- If the label is unknown, the default of the property is used.
- If the label is known, but doesn't provide a value for the property then again the default of the property is used.
- The command `\property_ref:nnn` allows to give a local default which is used instead of the property default in the two cases before.

4 Rerun messages

As the reference commands are expandable they can neither issue a message that the label or the label-property combination is unknown, nor can they trigger the rerun message at the end of the LaTeX run.

Where needed such messages must therefore be triggered manually. For this two commands are provided: `\property_ref_undefined_warn:` and `\property_ref_undefined_warn:nn`. See below for a description.

5 Open points

- The `xpos` and `ypos` properties require that the position is stored first but there is no (public) engine independent interface yet. Code must use `\tex_savepos:D`.

6 Code interfaces

`\property_new:nnnn` `\property_new:nnnn {⟨property⟩} {⟨setpoint⟩} {⟨default⟩} {⟨code⟩}`
`\property_gset:nnnn` `\property_gset:nnnn {⟨property⟩} {⟨setpoint⟩} {⟨default⟩} {⟨code⟩}`

LaTeX 2_ε-interface: see `\NewProperty`, `\SetProperty`.
Sets the ⟨*property*⟩ to have the ⟨*default*⟩ specified, and at the ⟨*setpoint*⟩ (either `now` or `shipout`) to write the result of the ⟨*code*⟩ as part of a label. The ⟨*code*⟩ should be expandable. The expansion of ⟨*code*⟩ (the value of the property) is written to the `.aux` file and read back from there at the next compilation. Values should assume that the standard LaTeX catcode régime with `@` a letter is active then.

If the property is declared within a package it is suggested that its name is build from letters, hyphens and slashes, and is always structured as follows:
⟨*package-name*⟩`/`⟨*property-name*⟩.

\property_record:nN
\property_record:nn
\property_record:(nV|ee)

\property_record:nN {⟨*label*⟩} ⟨*clist var*⟩
\property_record:nn {⟨*label*⟩} {⟨*clist*⟩}

LaTeX 2ε-interface: see \RecordProperties.
Writes the list of properties given by the ⟨*clist*⟩ to the .aux file with the ⟨*label*⟩ specified.

\property_ref:nn ⋆
\property_ref:ee ⋆

\property_ref:nn {⟨*label*⟩} {⟨*property*⟩}

LaTeX 2ε-interface: see \RefProperty.
Expands to the value of the ⟨*property*⟩ for the ⟨*label*⟩, if available, and the default value of the property otherwise. If ⟨*property*⟩ has not been declared with \property_new:nnnn an error is issued. The command raises an internal, expandable, local flag if the reference can not be resolved.

\property_item:nn ⋆
\property_item:ee ⋆

New: 2025-11-20

\property_item:nn {⟨*label*⟩} {⟨*property*⟩}

Retrieves the value of the ⟨*property*⟩ for the ⟨*label*⟩ like \property_ref:nn but the result is returned within the \unexpanded primitive (\exp_not:n), which means that the ⟨*value*⟩ does not expand further when appearing in an e-type or x-type argument expansion. This allows for example to handle values containing user commands which are not safe in an expansion context.

\property_ref:nnn ⋆
\property_ref:een ⋆

\property_ref:nnn {⟨*label*⟩} {⟨*property*⟩} {⟨*local default*⟩}

LaTeX 2ε-interface: see \RefProperty.
Expands to the value of the ⟨*property*⟩ for the ⟨*label*⟩, if available, and to ⟨*local default*⟩ otherwise. If ⟨*property*⟩ has not been declared with \property_new:nnnn an error is issued. The command raises an internal, expandable local flag if the reference can not be resolved.

\property_item:nnn ⋆
\property_item:een ⋆

New: 2026-05-01

\property_item:nnn {⟨*label*⟩} {⟨*property*⟩} {⟨*local default*⟩}

Retrieves the value of the ⟨*property*⟩ for the ⟨*label*⟩, if available, and to ⟨*local default*⟩ otherwise, but the result is returned within the \unexpanded primitive (\exp_not:n), which means that the ⟨*value*⟩ does not expand further when appearing in an e-type or x-type argument expansion. This allows for example to handle values containing user commands which are not safe in an expansion context.

\property_ref_undefined_warn:

\property_ref_undefined_warn:

LaTeX 2ε-interface: not provided.
Triggers the standard warning

```
LaTeX Warning: There were undefined references.
```

at the end of the document if there was a recent \property_ref:nn or \property_ref:nnn which couldn't be resolved and so raised the flag. "Recent" means in the same group or in some outer group!

\property_ref_undefined_warn:n \property_ref_undefined_warn:n {⟨label⟩}
\property_ref_undefined_warn:e

LaTeX 2ε-interface: not provided.
Triggers the standard warning

```
LaTeX Warning: There were undefined references.
```

at the end of the document if ⟨label⟩ is not known. At the point where it is called it also issues the warning

```
Reference '⟨label⟩' on page ⟨page⟩ undefined.
```

\property_ref_undefined_warn:nn \property_ref_undefined_warn:nn {⟨label⟩} {⟨property⟩}
\property_ref_undefined_warn:ee

LaTeX 2ε-interface: see \RefUndefinedWarn.
Triggers the standard warning

```
LaTeX Warning: There were undefined references.
```

at the end of the document if the reference can not be resolved. At the point where it is called it also issues the warning

```
Reference '⟨label⟩' on page ⟨page⟩ undefined
```

if the label is unknown, or the more specific

```
Property '⟨property⟩' undefined for reference '⟨label⟩' on page ⟨page⟩
```

if the label is known but doesn't provide a value for the requested property.

\property_if_exist_p:n ⋆ \property_if_exist_p:n {⟨property⟩}
\property_if_exist_p:e ⋆ \property_if_exist:nTF {⟨property⟩} {⟨true code⟩} {⟨false code⟩}
\property_if_exist:nTF ⋆
\property_if_exist:eTF ⋆

LaTeX 2ε-interface: \IfPropertyExistsTF.
Tests if the ⟨property⟩ has been declared.

\property_if_recorded_p:n ⋆ \property_if_recorded_p:n {⟨label⟩}
\property_if_recorded_p:e ⋆ \property_if_recorded:nTF {⟨label⟩} {⟨true code⟩} {⟨false code⟩}
\property_if_recorded:nTF ⋆
\property_if_recorded:eTF ⋆

LaTeX 2ε-interface: \IfLabelExistsTF
Tests if the ⟨label⟩ is known. This is also true if the label has been set with the standard \label command.

\property_if_recorded_p:nn ⋆ \property_if_recorded_p:nn {⟨label⟩} {⟨property⟩}
\property_if_recorded_p:ee ⋆ \property_if_recorded:nnTF {⟨label⟩} {⟨property⟩} {⟨true code⟩} {⟨false code⟩}
\property_if_recorded:nnTF ⋆
\property_if_recorded:eeTF ⋆

LaTeX 2ε-interface: \IfPropertyRecordedTF.
Tests if the label ⟨label⟩ is known and if it provides a value of the ⟨property⟩.

7 Auxiliary file interfaces

\new@label@record \new@label@record {⟨label⟩} {⟨data⟩}

This is a command only for use in the .aux file. It loads the key–value list of ⟨data⟩ to be available for the ⟨label⟩.

8 LaTeX 2ε interface

The LaTeXe interfaces always expand label and property arguments. This means that one must be careful when using active chars or commands in the names. UTF8-chars are protected and should be safe, similar most babel shorthands.

`\NewProperty` `\NewProperty {⟨property⟩} {⟨setpoint⟩} {⟨default⟩} {⟨code⟩}`
`\SetProperty` `\SetProperty {⟨property⟩} {⟨setpoint⟩} {⟨default⟩} {⟨code⟩}`

Sets the ⟨*property*⟩ to have the ⟨*default*⟩ specified, and at the ⟨*setpoint*⟩ (either `now` or `shipout`) to write the result of the ⟨*code*⟩ as part of a label. The ⟨*code*⟩ should be expandable. The expansion of ⟨*code*⟩ (the value of the property) is written to the `.aux` file and read back from there at the next compilation (at which point normally the standard LaTeX catcode régime with `@` a letter is active).

`\RecordProperties` `\RecordProperties {⟨label⟩} {⟨clist⟩}`

Writes the list of properties given by the ⟨*clist*⟩ to the `.aux` file with the ⟨*label*⟩ specified. Similar to the standard `\label` command the arguments are expanded. So ⟨*clist*⟩ can be a macro containing a list of properties. Also similar to the standard `\label` command, the command is surrounded by an `\@bsphack`/`\@esphack` pair to preserve spacing.

`\RefProperty ⋆` `\RefProperty [⟨local default⟩] {⟨label⟩} {⟨property⟩}`

Expands to the value of the ⟨*property*⟩ for the ⟨*label*⟩, if available, and the default value of the property or – if given – to ⟨*local default*⟩ otherwise. If {⟨*property*⟩} has not been declared an error is issued.

`\IfPropertyExistsTF` `\IfPropertyExistsTF {⟨property⟩} {⟨true code⟩} {⟨false code⟩}`
`\IfPropertyExistsT`
`\IfPropertyExistsF`

Tests if the ⟨*property*⟩ has been declared.

`\IfLabelExistsTF` `\IfLabelExistsTF {⟨label⟩} {⟨true code⟩} {⟨false code⟩}`
`\IfLabelExistsT`
`\IfLabelExistsF`

Tests if the ⟨*label*⟩ has been recorded. This is also true if a label has been set with the standard `\label` command.

`\IfPropertyRecordedTF` `\IfPropertyRecordedTF {⟨label⟩} {⟨property⟩} {⟨true code⟩} {⟨false code⟩}`
`\IfPropertyRecordedT`
`\IfPropertyRecordedF`

Tests if the label and a value of the ⟨*property*⟩ for the ⟨*label*⟩ are both known.

`\RefUndefinedWarn` `\RefUndefinedWarn {⟨label⟩} {⟨property⟩}`

Triggers the standard warning

`LaTeX Warning: There were undefined references.`

at the end of the document if the reference for ⟨*label*⟩ and ⟨*property*⟩ can not be resolved. At the point where it is called it also issues the warning

`Reference ‘⟨label⟩’ on page ⟨page⟩ undefined`

if the label is unknown, or the more specific

`Property ‘⟨property⟩’ undefined for reference ‘⟨label⟩’ on page ⟨page⟩` if the label is known but doesn't provide a value for the requested property.

9 Pre-declared properties

`abspage` (shipout) The absolute value of the current page: starts at 1 and increases monotonically at each shipout.

`page` (shipout) The current page as given by `\thepage`: this may or may not be a numerical value, depending on the current style. Contrast with `\abspage`. You get this value also with the standard `\label`/`\pageref`.

`pagenum` (shipout) The current page as arabic number. This is suitable for integer operations and comparisons.

`label` (now) The content of `\@currentlabel`. This is the value that you get also with the standard `\label`/`\ref`.

`title` (now) The content of `\@currentlabelname`. This command is filled beside others by the nameref package and some classes (e.g. memoir).

`target` (now) The content of `\@currentHref`. This command is normally filled by for example hyperref and gives the name of the last destination it created.

`pagetarget` (shipout) The content of `\@currentHpage`. This command is filled for example by a recent version of hyperref and then gives the name of the last page destination it created.

`counter` (now) The content of `\@currentcounter`. This command contains after a `\refstepcounter` the name of the counter.

`xpos` `ypos` (shipout) This stores the x and y coordinates of a point previously stored with `\pdfsavepos`/`\savepos`. E.g. (if bidi is used it can be necessary to save the position before and after the label):

```
\tex_savepos:D
\property_record:nn{myposition}{xpos,ypos}
\tex_savepos:D
```

10 The Implementation

```
⟨*2ekernel | latexrelease⟩
\ExplSyntaxOn
⟨@@=property⟩
⟨latexrelease⟩\NewModuleRelease{2023/11/01}{ltproperties}
⟨latexrelease⟩                        {Cross-referencing~properties}
```

The approach here is based closely on that from zref; separate out lists of properties and the properties themselves, so the latter can be used multiple times and in varying combinations. However, not everything is a straight copy. Firstly, we treat lists of properties as simple comma lists: that allows us to have either saved or dynamic lists and to avoid another data structure. The cost is that errors are detected at point-of-use, but in any real case that should be true anyway (and is true for `\zref@labelbyprop` already). Secondly, we allow properties to have arbitrary names, as the code does not require them to tokenize as control sequences.

`\property_new:nnnn`
`\property_gset:nnnn`
`\__property_gset:nnnn`

As properties can be reset, they are not constants. But they also have various pieces of required data. So we use the same approach as color and make them declarations. Data-wise, we need the detail of the implementation, the default and a flag to show if the code works now or at shipout. This last entry is done using text so needs a check. We could use a set of `prop` here, but as we never need to map or copy the lists, we can gain performance using the hash table approach.

```
\cs_new_protected:Npn \property_new:nnnn #1#2#3#4
  {
    \cs_if_free:cTF { __property_code_ #1 : }
      {
        \exp_args:Ne \__property_gset:nnnn { \tl_to_str:n {#1} }
         {#2} {#3} {#4}
      }
      {
        \msg_error:nn { property }{ exists }{#1}
      }
  }
\cs_new_protected:Npn \property_gset:nnnn #1#2#3#4
  {
    \__property_gset:ennn { \tl_to_str:n {#1} }
      {#2} {#3} {#4}
  }
\cs_new_protected:Npn \__property_gset:nnnn #1#2#3#4
  {
    \cs_gset:cpn { __property_code_ #1 : } {#4}
    \tl_gclear_new:c { g__property_default_ #1 _tl }
    \tl_gset:cn { g__property_default_ #1 _tl } {#3}
    \bool_if_exist:cF { g__property_shipout_ #1 _bool }
      { \bool_new:c { g__property_shipout_ #1 _bool } }
    \str_case:nnF {#2}
      {
        { now } { { \bool_gset_false:c { g__property_shipout_ #1 _bool } } }
        { shipout }
          { \bool_gset_true:c { g__property_shipout_ #1 _bool } }
      }
      { \msg_error:nnnn { property } { unknown-setpoint } {#1} {#2} }
```

```
  }
\cs_generate_variant:Nn \__property_gset:nnnn {ennn}
```

(*End of definition for* `\property_new:nnnn`, `\property_gset:nnnn`, *and* `\__property_gset:nnnn`. *These functions are documented on page* 850.)

`\NewProperty` `\SetProperty`

For consistency we expand the property name, but this doesn't warant a variant of the L3-commands.

```
\cs_new_protected:Npn \NewProperty #1#2#3#4
  {
    \protected@edef\reserved@a{#1}
    \exp_args:No \property_new:nnnn {\reserved@a} {#2}{#3}{#4}
  }
\cs_new_protected:Npn \SetProperty #1#2#3#4
  {
    \protected@edef\reserved@a{#1}
    \exp_args:No \property_gset:nnnn {\reserved@a} {#2}{#3}{#4}
  }
```

(*End of definition for* `\NewProperty` *and* `\SetProperty`. *These functions are documented on page* 853.)

`\property_record:nN` `\property_record:nn` `\property_record:nV` `\property_record:ee` `\property_record:oo` `\__property_record:nn` `\__property_record:en` `\__property_record_value:n` `\__property_record_value_aux:n` `\__property_record_value_aux:e`

Writing data when it is labelled means expanding at this stage and possibly later too. That is all pretty easy using expl3: we accept a stray comma at the end of the list as that is easier to deal with than trying to tidy up, and there is no real downside.

```
\cs_new_protected:Npn \property_record:nN #1#2
  { \property_record:nV {#1} #2 }
\cs_new_protected:Npn \property_record:nn #1#2
  { \__property_record:en { \tl_to_str:n {#1} } {#2} }
\cs_generate_variant:Nn \property_record:nn { nV , ee, oo }
\cs_new_protected:Npn \__property_record:nn #1#2
  {
    \protected@write \@auxout {}
      {
        \token_to_str:N \new@label@record
          {#1}
          { \clist_map_function:nN {#2} \__property_record_value:n }
      }
  }
\cs_generate_variant:Nn \__property_record:nn { e }
\cs_new:Npn \__property_record_value:n #1
  { \__property_record_value_aux:e { \tl_to_str:n {#1} } }
\cs_new:Npn \__property_record_value_aux:n #1
  {
    \cs_if_exist:cTF { __property_code_ #1 : }
      {
        {#1}
        {
          \bool_if:cTF { g__property_shipout_ #1 _bool }
            { \exp_not:c }
            { \use:c }
              { __property_code_ #1 : }
        }
      }
      { \msg_expandable_error:nnn { property } { not-declared } {#1} }
```

```
  }
\cs_generate_variant:Nn \__property_record_value_aux:n { e }
```

(*End of definition for* `\property_record:nN` *and others. These functions are documented on page 851.*)

`\RecordProperties`

```
\NewDocumentCommand\RecordProperties { m m }
  {
    \@bsphack
    \protected@edef\reserved@a{#1}
    \protected@edef\reserved@b{#2}
    \property_record:oo {\reserved@a}{\reserved@b}
    \@esphack
  }
```

(*End of definition for* `\RecordProperties`*. This function is documented on page 853.*)

10.1 Reference commands

`l__property_ref_flag` A flag that is set if a reference couldn't be resolved.

```
\flag_new:n { l__property_ref_flag }
```

(*End of definition for* `l__property_ref_flag`*.*)

`\property_ref:nn` `\property_ref:ee` Search for the label/property combination, and if not found fall back to the default of the property.

```
\cs_new:Npn \property_ref:nn #1#2
  {
    \__property_ref:een
      { \tl_to_str:n {#1} }
      { \tl_to_str:n {#2} }
      { \tl_use:c { g__property_default_ #2 _tl } }
  }
\cs_generate_variant:Nn \property_ref:nn {ee}
```

(*End of definition for* `\property_ref:nn`*. This function is documented on page 851.*)

`\property_ref:nnn` `\property_ref:een` `\__property_ref:nnn` `\__property_ref:een` This allows to set a local default value which overrides the default value of the property.

```
\cs_new:Npn \property_ref:nnn #1#2#3
  {
    \__property_ref:een
      { \tl_to_str:n {#1} }
      { \tl_to_str:n {#2} }
      {#3}
  }
\cs_new:Npn \__property_ref:nnn #1#2#3
  {
    \tl_if_exist:cTF { g__property_label_ #1 _ #2 _tl }
      { \tl_use:c { g__property_label_ #1 _ #2 _tl } }
      {
        \flag_if_raised:nF
          { l__property_ref_flag } { \flag_raise:n { l__property_ref_flag } }
```

We test for the default of the property only to check if the property has been declared.

```
        \tl_if_exist:cTF { g__property_default_ #2 _tl }
          { #3 }
          { \msg_expandable_error:nnn { property } { not-declared } {#2} }
      }
  }
\cs_generate_variant:Nn \__property_ref:nnn { ee }
\cs_generate_variant:Nn \property_ref:nnn {een}
```

(*End of definition for* `\property_ref:nnn` *and* `\__property_ref:nnn`*. This function is documented on page 851.*)

`\property_item:nn` `\property_item:ee`

Retrieve the value but wrap it into an `\exp_not:n` to avoid further expansion.

```
\cs_new:Npn \property_item:nn #1#2
 {
  \tl_if_exist:cTF { g__property_label_ \tl_to_str:n {#1} _ \tl_to_str:n {#2} _tl }
   {
     \exp_not:v { g__property_label_ \tl_to_str:n {#1} _ \tl_to_str:n {#2} _tl }
   }
   { \tl_use:c { g__property_default_ \tl_to_str:n {#2} _tl } }
 }
\cs_generate_variant:Nn \property_item:nn {ee}
```

(*End of definition for* `\property_item:nn`*. This function is documented on page 851.*)

`\property_item:nnn` `\property_item:een`

Same as `\property_item:nn` but allows setting a local default.

```
\cs_new:Npn \property_item:nnn #1#2#3
 {
  \tl_if_exist:cTF { g__property_label_ \tl_to_str:n {#1} _ \tl_to_str:n {#2} _tl }
   {
     \exp_not:v { g__property_label_ \tl_to_str:n {#1} _ \tl_to_str:n {#2} _tl }
   }
   {
     \tl_if_exist:cTF { g__property_default_ \tl_to_str:n {#2} _tl }
       { #3 }
       { \msg_expandable_error:nnn { property } { not-declared } {#2} }
   }
 }
\cs_generate_variant:Nn \property_item:nnn {ee}
```

(*End of definition for* `\property_item:nnn`*. This function is documented on page 851.*)

`\RefProperty`

Search for the label/property combination, and if not found fall back to the default of the property or the given default.

```
\NewExpandableDocumentCommand \RefProperty { o m m }
  {
    \IfNoValueTF {#1}
      {
        \property_ref:ee {#2}{#3}
      }
      {
        \property_ref:een {#2}{#3}{#1}
      }
  }
```

(*End of definition for* `\RefProperty`. *This function is documented on page 853.*)

`\new@label@record`
`\__property_data:nnn`

A standard recursion loop.

```
\cs_new_protected:Npn \new@label@record #1#2
  {
    \tl_if_exist:cTF { r@#1 }
      {
        \gdef \@multiplelabels
          { \@latex@warning@no@line { There~were~multiply~defined~labels } }
        \@latex@warning@no@line { Label~`#1'~multiply~defined }
      }
      {
        \tl_new:c { r@#1 }
        \tl_gset:cn { r@#1 }{#2}
      }
    \__property_data:nnn {#1} #2 { \q_recursion_tail } { ? } \q_recursion_stop
  }
\cs_new_protected:Npn \__property_data:nnn #1#2#3
  {
    \quark_if_recursion_tail_stop:n {#2}
    \tl_gclear_new:c { g__property_label_ \tl_to_str:n {#1} _ \tl_to_str:n {#2} _tl }
    \tl_gset:cn { g__property_label_ \tl_to_str:n {#1} _ \tl_to_str:n {#2} _tl } {#3}
    \__property_data:nnn {#1}
  }
```

This command is used in `\enddocument` to test if some label values have changed.

```
\cs_new_protected:Npn \@kernel@new@label@record@testdef #1 #2
  {
    \tl_if_eq:cnF { r@#1 } {#2}
      { \@tempswatrue }
  }
```

(*End of definition for* `\new@label@record` *and* `\__property_data:nnn`. *This function is documented on page 852.*)

10.2 Tests and warnings

`\property_if_exist_p:n`
`\property_if_exist:nTF`

Tests if property has been declared.

```
\prg_new_conditional:Npnn \property_if_exist:n #1 { p , T , F,  TF }
  % #1 property
  {
    \cs_if_exist:cTF { __property_code_ #1 : }
      {
        \prg_return_true:
      }
      {
        \prg_return_false:
      }
  }
\prg_generate_conditional_variant:Nnn \property_if_exist:n {e} { p , T , F,  TF }
```

(*End of definition for* `\property_if_exist:nTF`. *This function is documented on page 852.*)

\IfPropertyExistsTF
\IfPropertyExistsT
\IfPropertyExistsF

```
\cs_new_eq:NN \IfPropertyExistsTF \property_if_exist:eTF
\cs_new:Npn   \IfPropertyExistsT #1#2 {\property_if_exist:eTF {#1}{#2}{} }
\cs_new:Npn   \IfPropertyExistsF #1   {\property_if_exist:eTF {#1}{} }
```

(*End of definition for* `\IfPropertyExistsTF`, `\IfPropertyExistsT`, *and* `\IfPropertyExistsF`. *These functions are documented on page 853.*)

\property_if_recorded_p:n
\property_if_recorded:nTF

Tests if the label has been set. This can then be used to setup e.g. rerun messages.

```
\prg_new_conditional:Npnn \property_if_recorded:n #1 { p , T , F,  TF }
  % #1 label
  {
    \tl_if_exist:cTF { r@#1 }
      {
        \prg_return_true:
      }
      {
        \prg_return_false:
      }
  }
\prg_generate_conditional_variant:Nnn \property_if_recorded:n {e} { p , T , F,  TF }
```

(*End of definition for* `\property_if_recorded:nTF`. *This function is documented on page 852.*)

\IfLabelExistsTF
\IfLabelExistsT
\IfLabelExistsF

```
\cs_new_eq:NN \IfLabelExistsTF \property_if_recorded:eTF
\cs_new:Npn   \IfLabelExistsT #1#2 {\property_if_recorded:eTF {#1}{#2}{} }
\cs_new:Npn   \IfLabelExistsF #1   {\property_if_recorded:eTF {#1}{} }
```

(*End of definition for* `\IfLabelExistsTF`, `\IfLabelExistsT`, *and* `\IfLabelExistsF`. *These functions are documented on page 853.*)

\property_if_recorded_p:nn
\property_if_recorded:nnTF

tests if the label/property combination has been set This can then be used to setup e.g. rerun messages.

```
\prg_new_conditional:Npnn \property_if_recorded:nn #1#2 { p , T , F,  TF }
  % #1 label #2 property
  {
    \tl_if_exist:cTF { g__property_label_ \tl_to_str:n {#1} _ \tl_to_str:n {#2} _tl }
      {
        \prg_return_true:
      }
      {
        \prg_return_false:
      }
  }
\prg_generate_conditional_variant:Nnn \property_if_recorded:nn {ee} { p , T , F,  TF }
```

(*End of definition for* `\property_if_recorded:nnTF`. *This function is documented on page 852.*)

\IfPropertyRecordedTF
\IfPropertyRecordedT
\IfPropertyRecordedF

```
\cs_new_eq:NN \IfPropertyRecordedTF \property_if_recorded:eeTF
\cs_new:Npn \IfPropertyRecordedT #1#2#3 { \property_if_recorded:eeTF {#1}{#2}{#3}{} }
\cs_new:Npn \IfPropertyRecordedF #1#2#3 { \property_if_recorded:eeTF {#1}{#2}{}{#3} }
```

(*End of definition for* `\IfPropertyRecordedTF`, `\IfPropertyRecordedT`, *and* `\IfPropertyRecordedF`. *These functions are documented on page 853.*)

\property_ref_undefined_warn: `\G@refundefinedtrue` is defined in `ltxref` and redefines a warning message.

```
\cs_new_protected:Npn \property_ref_undefined_warn:
 {
   \flag_if_raised:nT { l__property_ref_flag }
    {
      \G@refundefinedtrue
    }
 }
```

(*End of definition for* `\property_ref_undefined_warn:`. *This function is documented on page* 851.)

\property_ref_undefined_warn:n
\property_ref_undefined_warn:e

```
\cs_new_protected:Npn \property_ref_undefined_warn:n #1 %#1 label
  {
   \property_if_recorded:nF {#1}
    {
       \G@refundefinedtrue
       \@latex@warning{Reference~‘#1’~on~page~\thepage\space undefined}%
    }
  }
\cs_generate_variant:Nn \property_ref_undefined_warn:n {e}
```

(*End of definition for* `\property_ref_undefined_warn:n`. *This function is documented on page* 852.)

\property_ref_undefined_warn:nn
\property_ref_undefined_warn:ee
\RefUndefinedWarn

```
\cs_new_protected:Npn \property_ref_undefined_warn:nn #1#2 %#1 label, #2 property
 {
   \property_if_recorded:nTF {#1}
    {
      \property_if_recorded:nnF {#1}{#2}
       {
         \G@refundefinedtrue
         \@latex@warning
          { Property~‘#2’~undefined~for~reference~‘#1’~on~page~\thepage }
       }
    }
    {
       \G@refundefinedtrue
       \@latex@warning { Reference~‘#1’~on~page~\thepage\space undefined }%
    }
 }
\cs_generate_variant:Nn \property_ref_undefined_warn:nn {ee}
\cs_set_eq:NN \RefUndefinedWarn \property_ref_undefined_warn:ee
```

(*End of definition for* `\property_ref_undefined_warn:nn` *and* `\RefUndefinedWarn`. *These functions are documented on page* 852.)

10.3 Predeclared properties

abspage

```
\property_new:nnnn { abspage } { shipout }
  { 0 } { \int_use:N \g_shipout_readonly_int }
```

(*End of definition for* `abspage`. *This variable is documented on page* 854.)

page

```
\property_new:nnnn  { page } { shipout } { 0 } { \thepage }
```

(*End of definition for* page. *This variable is documented on page 854.*)

pagenum

```
\property_new:nnnn  { pagenum } { shipout } { 0 } { \the \value { page } }
```

(*End of definition for* pagenum. *This variable is documented on page 854.*)

label

```
\property_new:nnnn  { label } { now } { ?? } { \@currentlabel }
```

(*End of definition for* label. *This variable is documented on page 854.*)

title

```
\property_new:nnnn  { title } { now }
  { \exp_not:n { \textbf { ?? } } } { \@currentlabelname }
```

(*End of definition for* title. *This variable is documented on page 854.*)

target

```
\property_new:nnnn  { target } { now } { } { \@currentHref }
```

(*End of definition for* target. *This variable is documented on page 854.*)

pagetarget

```
\newcommand\@currentHpage{}
\property_new:nnnn  { pagetarget } { shipout } { } { \@currentHpage }
```

(*End of definition for* pagetarget. *This variable is documented on page 854.*)

counter

```
\property_new:nnnn  { counter } { now }  { } { \@currentcounter }
```

(*End of definition for* counter. *This variable is documented on page 854.*)

xpos
ypos

```
\property_new:nnnn  { xpos } { shipout } { 0} { \int_use:N \tex_lastxpos:D }
\property_new:nnnn  { ypos } { shipout } { 0} { \int_use:N \tex_lastypos:D }
```

(*End of definition for* xpos *and* ypos. *These variables are documented on page 854.*)

10.4 Messages

```
\msg_new:nnnn { property } { exists }
    { Property~'#1'~ has~ already~ been~ declared. }
    { There~ already~ exists~ a~ property~ declaration~ with~ this~
      name.\\
      Please~ use~ a~ different~ name~ for~ your~ property.}

\msg_new:nnnn { property } { not-declared }
  { Property~'#1'~not~declared. }
  {
    LaTeX~has~been~asked~to~use~property~'#1',~but~this~
    name~has~not~been~declared.
  }
\msg_new:nnnn { property } { unknown-setpoint }
  { Unknown~keyword~'#2'~for~setting~property~'#1'. }
  {
    LaTeX~has~been~asked~to~set~the~property~'#1',~but~the~keyword~
    '#2'~is~not~one~of~the~two~known~values:~'now'~or~'shipout'.
  }
%
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{ltproperties}
⟨latexrelease⟩                  {cross-referencing~properties~(undo)}%
⟨latexrelease⟩
⟨latexrelease⟩\let \NewProperty  \@undefined
⟨latexrelease⟩\let \SetProperty \@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let \RecordProperties  \@undefined
⟨latexrelease⟩\let \RefProperty \@undefined
⟨latexrelease⟩\let \RefUndefinedWarn  \@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let \IfPropertyExistsTF \@undefined
⟨latexrelease⟩\let \IfLabelExistsTF \@undefined
⟨latexrelease⟩\let \IfPropertyRecordedTF  \@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let\new@label@record \@undefined
⟨latexrelease⟩\let\@kernel@new@label@record@testdef\@undefined
⟨latexrelease⟩\EndModuleRelease

\ExplSyntaxOff

⟨/2ekernel | latexrelease⟩
```

Reset module prefix:

```
⟨@@=⟩
```

File 37
ltmiscen.dtx

1 Miscellaneous Environments

This section implements the basic environment mechanism, and also a few specific environments including `document`, The math environments and related commands, the 'flushing' environments, (`center`, `flushleft`, `flushright`), and `verbatim`.

```
⟨*2ekernel⟩
\message{environments,}
```

1.1 Environments

`\begin{foo}` and `\end{foo}` are used to delimit environment `foo`.

`\begin{foo}` starts a group and calls `\foo` if it is defined, otherwise it does nothing.

`\end{foo}` checks to see that it matches the corresponding `\begin` and if so, it calls `\endfoo` and does an `\endgroup`. Otherwise, `\end{foo}` does nothing.

If `\end{foo}` needs to ignore blanks after it, then `\endfoo` should globally set the `@ignore` switch true with `\@ignoretrue` (this will automatically be global).

NOTE: `\@@end` is defined to be the `\end` command of TEX82.

`\enddocument` is the user's command for ending the manuscript file.

`\stop` is a panic button — to end TEX in the middle.

Historical LATEX 2.09 comments (not necessarily accurate any more):

```
\enddocument ==
  BEGIN
   \@checkend{document}    %% checks for unmatched \begin
   \clearpage
   \begingroup
     if @filesw = true
       then   close file @mainaux
              if G@refundefined = true
               then LaTeX Warning: 'There are undefined references.' fi
              if @multiplelabels = true
                then LaTeX Warning:
                     'One or more label(s) multiply defined.'
                else
                \@setckpt {ARG1}{ARG2} == null
                \newlabel{LABEL}{VAL} ==
                     BEGIN
                       \reserved@a == VAL
                       if def(\reserved@a) = def(\r@LABEL)
                         else @tempswa := true              fi
                     END
                \bibcite{LABEL}{VAL} == null
                     BEGIN
                       \reserved@a == VAL
                       if def(\reserved@a) = def(\g@LABEL)
                         else @tempswa := true              fi
```

```
                    END
                @tempswa := false
                make @ a letter
                \input \jobname.AUX
                if @tempswa = true
                  then LaTeX Warning: 'Label may have changed.
                                      Rerun to get cross-references right.'
      fi      fi      fi
    \endgroup
    finish up
  END

  \@writefile{EXT}{ENTRY} ==
      if tf@EXT undefined
        else \write\tf@EXT{ENTRY}
      fi
```

End of historical LaTeX 2.09 comments.

`\@currenvir` The name of the current environment. Initialized to `document` to so that `\end{document}` works correctly.

```
\def\@currenvir{document}
```

(*End of definition for* `\@currenvir`.)

`\if@ignore`
`\@ignoretrue`
`\@ignorefalse`

```
\def\@ignorefalse{\global\let\if@ignore\iffalse}
\def\@ignoretrue {\global\let\if@ignore\iftrue}
\@ignorefalse
```

(*End of definition for* `\if@ignore`, `\@ignoretrue`, *and* `\@ignorefalse`.)

`\ignorespacesafterend`

```
\let\ignorespacesafterend\@ignoretrue
```

(*End of definition for* `\ignorespacesafterend`.)

`\enddocument` (*End of definition for* `\enddocument`.)

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2023/11/01}%
⟨latexrelease⟩                    {\enddocument}{check property labels}%
\def\enddocument{%
```

The `\end{document}` hook is executed first. If necessary it can contain a `\clearpage` to output dangling floats first. In this position it can also contain something like `\end{foo}` so that the whole document effectively starts and ends with some special environment. However, this must be used with care, eg if two applications would use this without knowledge of each other the order of the environments will be wrong after all. `\AtEndDocument` is redefined at this point so that and such commands that get into the hook do not chase their tail...

```
    \@kernel@before@enddocument
    \UseOneTimeHook{enddocument}%
    \@kernel@after@enddocument
```

```
  \@checkend{document}%
  \clearpage

  \@kernel@before@enddocument@afterlastpage
  \UseOneTimeHook{enddocument/afterlastpage}%
  \@kernel@after@enddocument@afterlastpage
  \begingroup
    \if@filesw
      \immediate\closeout\@mainaux
      \let\@setckpt\@gobbletwo
      \let\@newl@bel\@testdef

      \let\new@label@record\@kernel@new@label@record@testdef
```

The previous line is equiv to setting

```
\def\newlabel{\@testdef r}%
\def\bibcite{\@testdef b}%
```

We use `\@@input` to load the `.aux` file, so that it doesn't show up in the list of files produced by `\listfiles`.

```
      \@tempswafalse
      \global\advance\c@page\@ne
      \makeatletter \@@input\jobname.aux
    \fi
    \UseOneTimeHook{enddocument/afteraux}%
```

Next hook is expect to contain only code for writing info messages on the terminal.

```
    \UseOneTimeHook{enddocument/info}%
  \endgroup
  \UseOneTimeHook{enddocument/end}%
  \deadcycles\z@\@@end}
```

The public hooks used in `\enddocument`:

```
\NewHook{enddocument}
\NewHook{enddocument/afterlastpage}
\NewHook{enddocument/afteraux}
\NewHook{enddocument/info}
\NewHook{enddocument/end}
```

This is one of the few places where we already add data and rules to a hook already in the kernel.

If we roll back we have to drop stuff before adding chunks, otherwise the code will just be appended, and thus doubled. This would result in a harmless warning during the format generation, because in that case the code chunk label doesn't exist, and therefore can't be dropped.

```
⟨latexrelease⟩\RemoveFromHook{enddocument/info}[kernel/filelist]
⟨latexrelease⟩\RemoveFromHook{enddocument/info}[kernel/warnings]
⟨latexrelease⟩\RemoveFromHook{enddocument/info}[kernel/release]

\AddToHook{enddocument/info}[kernel/filelist]{\@dofilelist}
\AddToHook{enddocument/info}[kernel/warnings]{\@enddocument@kernel@warnings}
\AddToHook{enddocument/info}[kernel/release]{%
  \let\show@release@info\wlog
  \show@release@info{ ***********}%
  \the\LaTeXReleaseInfo
```

```
    \show@release@info{ ***********}}

\DeclareHookRule{enddocument/info}{kernel/release}{before}{kernel/filelist}
\DeclareHookRule{enddocument/info}{kernel/filelist}{before}{kernel/warnings}
```

(*End of definition for* `\enddocument`.)

`\@enddocument@kernel@warnings`

```
\def\@enddocument@kernel@warnings{%
```

First we check for font size substitution bigger than `\fontsubfuzz`. The `\relax` is necessary because this is a macro not a register.

```
    \ifdim \font@submax >\fontsubfuzz\relax
```

In case you wonder about the `\@gobbletwo` inside the message below, this is a horrible hack to remove the tokens `\on@line.` that are added by `\@font@warning` at the end.

```
      \@font@warning{Size substitutions with differences\MessageBreak
                     up to \font@submax\space have occurred.\@gobbletwo}%
    \fi
```

The macro `\@defaultsubs` is initially `\relax` but gets redefined to produce a warning if there have been some default font substitutions.

```
    \@defaultsubs
```

The macro `\@refundefined` is initially `\relax` but gets redefined to produce a warning if there are undefined refs.

```
    \@refundefined
```

If a label is defined more than once, `\@tempswa` will always be true and thus produce a "Label(s) may ..." warning. But since a rerun will not solve that problem (unless one uses a package like `varioref` that generates labels on the fly), we suppress this message.

```
    \if@filesw
      \ifx \@multiplelabels \relax
        \if@tempswa
          \@latex@warning@no@line{Label(s) may have changed.
              Rerun to get cross-references right}%
        \fi
      \else
        \@multiplelabels
      \fi
      \ifx \@extra@page@added \relax
          \@latex@warning@no@line{Temporary extra page added at the end.
              Rerun to get it removed}%
      \fi
```

We could think of adding a warning that nothing can be corrected while `\nofiles` is in force. In the past the warnings related to the `aux` file are simply suppressed in this case.

```
    \fi
}
```

(*End of definition for* `\@enddocument@kernel@warnings`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                    {\enddocument}{Use Hooks}%
```

```
⟨latexrelease⟩\def\enddocument{%
⟨latexrelease⟩    \@kernel@before@enddocument
⟨latexrelease⟩    \UseOneTimeHook{enddocument}%
⟨latexrelease⟩    \@kernel@after@enddocument
⟨latexrelease⟩    \@checkend{document}%
⟨latexrelease⟩    \clearpage
⟨latexrelease⟩    \UseOneTimeHook{enddocument/afterlastpage}%
⟨latexrelease⟩    \@kernel@after@enddocument@afterlastpage
⟨latexrelease⟩    \begingroup
⟨latexrelease⟩      \if@filesw
⟨latexrelease⟩        \immediate\closeout\@mainaux
⟨latexrelease⟩        \let\@setckpt\@gobbletwo
⟨latexrelease⟩        \let\@newl@bel\@testdef
⟨latexrelease⟩        \@tempswafalse
⟨latexrelease⟩        \makeatletter \@@input\jobname.aux
⟨latexrelease⟩      \fi
⟨latexrelease⟩      \UseOneTimeHook{enddocument/afteraux}%
⟨latexrelease⟩      \UseOneTimeHook{enddocument/info}%
⟨latexrelease⟩    \endgroup
⟨latexrelease⟩    \UseOneTimeHook{enddocument/end}%
⟨latexrelease⟩    \deadcycles\z@\@@end}
⟨latexrelease⟩\NewHook{enddocument}
⟨latexrelease⟩\NewHook{enddocument/afterlastpage}
⟨latexrelease⟩\NewHook{enddocument/afteraux}
⟨latexrelease⟩\NewHook{enddocument/info}
⟨latexrelease⟩\NewHook{enddocument/end}
```

If we roll back we have to drop stuff before adding chunks, otherwise the code will just be appended, and thus doubled.

```
⟨latexrelease⟩\RemoveFromHook{enddocument/info}[kernel/filelist]
⟨latexrelease⟩\RemoveFromHook{enddocument/info}[kernel/warnings]
⟨latexrelease⟩\RemoveFromHook{enddocument/info}[kernel/release]

⟨latexrelease⟩\AddToHook{enddocument/info}[kernel/filelist]{\@dofilelist}
⟨latexrelease⟩\AddToHook{enddocument/info}[kernel/warnings]{\@enddocument@kernel@warnings}
⟨latexrelease⟩\AddToHook{enddocument/info}[kernel/release]{%
⟨latexrelease⟩    \let\show@release@info\wlog
⟨latexrelease⟩    \show@release@info{ **********}%
⟨latexrelease⟩    \the\LaTeXReleaseInfo
⟨latexrelease⟩    \show@release@info{ **********}}
⟨latexrelease⟩
⟨latexrelease⟩\DeclareHookRule{enddocument/info}{kernel/release}{before}{kernel/filelist}
⟨latexrelease⟩\DeclareHookRule{enddocument/info}{kernel/filelist}{before}{kernel/warnings}
⟨latexrelease⟩\def\@enddocument@kernel@warnings{%
⟨latexrelease⟩    \ifdim \font@submax >\fontsubfuzz\relax
⟨latexrelease⟩      \@font@warning{Size substitutions with differences\MessageBreak
⟨latexrelease⟩                  up to \font@submax\space have occurred.\@gobbletwo}%
⟨latexrelease⟩    \fi
⟨latexrelease⟩    \@defaultsubs
⟨latexrelease⟩    \@refundefined
⟨latexrelease⟩    \if@filesw
⟨latexrelease⟩      \ifx \@multiplelabels \relax
⟨latexrelease⟩        \if@tempswa
⟨latexrelease⟩          \@latex@warning@no@line{Label(s) may have changed.
⟨latexrelease⟩              Rerun to get cross-references right}%
```

```
⟨latexrelease⟩       \fi
⟨latexrelease⟩     \else
⟨latexrelease⟩       \@multiplelabels
⟨latexrelease⟩     \fi
⟨latexrelease⟩     \ifx \@extra@page@added \relax
⟨latexrelease⟩         \@latex@warning@no@line{Temporary extra page added at the end.
⟨latexrelease⟩             Rerun to get it removed}%
⟨latexrelease⟩     \fi
⟨latexrelease⟩   \fi
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                 {\enddocument}{Use Hooks}%
⟨latexrelease⟩
⟨latexrelease⟩\def\enddocument{%
⟨latexrelease⟩   \let\AtEndDocument\@firstofone
⟨latexrelease⟩   \@enddocumenthook
⟨latexrelease⟩   \@checkend{document}%
⟨latexrelease⟩   \clearpage
⟨latexrelease⟩   \begingroup
⟨latexrelease⟩     \if@filesw
⟨latexrelease⟩       \immediate\closeout\@mainaux
⟨latexrelease⟩       \let\@setckpt\@gobbletwo
⟨latexrelease⟩       \let\@newl@bel\@testdef
⟨latexrelease⟩       \@tempswafalse
⟨latexrelease⟩       \makeatletter \@@input\jobname.aux
⟨latexrelease⟩     \fi
⟨latexrelease⟩     \@dofilelist
⟨latexrelease⟩     \ifdim \font@submax >\fontsubfuzz\relax
⟨latexrelease⟩       \@font@warning{Size substitutions with differences\MessageBreak
⟨latexrelease⟩                   up to \font@submax\space have occurred.\@gobbletwo}%
⟨latexrelease⟩     \fi
⟨latexrelease⟩     \@defaultsubs
⟨latexrelease⟩     \@refundefined
⟨latexrelease⟩     \if@filesw
⟨latexrelease⟩       \ifx \@multiplelabels \relax
⟨latexrelease⟩         \if@tempswa
⟨latexrelease⟩           \@latex@warning@no@line{Label(s) may have changed.
⟨latexrelease⟩               Rerun to get cross-references right}%
⟨latexrelease⟩         \fi
⟨latexrelease⟩       \else
⟨latexrelease⟩         \@multiplelabels
⟨latexrelease⟩       \fi
⟨latexrelease⟩     \fi
⟨latexrelease⟩   \endgroup
⟨latexrelease⟩   \deadcycles\z@\@@end}
⟨latexrelease⟩
⟨latexrelease⟩\let\@enddocument@kernel@warnings\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

`\@kernel@before@enddocument` The `\@kernel@before@enddocument` hook is slightly different because we initialize it with `\par` so that `\enddocument` always returns to vertical mode as its first action.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩                    {\@kernel@before@enddocument}{kernel before hook}%
\def\@kernel@before@enddocument{\par}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

The rollback code renders it harmless.

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\@kernel@before@enddocument}{kernel before hook}%
⟨latexrelease⟩
⟨latexrelease⟩\let\@kernel@before@enddocument\@empty
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@kernel@before@enddocument`.)

`\@testdef`

```
\def\@testdef #1#2#3{%
  \def\reserved@a{#3}\expandafter \ifx \csname #1@#2\endcsname
 \reserved@a  \else \@tempswatrue \fi}
```

(*End of definition for* `\@testdef`.)

Reading data from auxiliary files (like `.toc` normally happens in vertical mode and it therefore doesn't matter if line endings are converted to spaces by TEX during that process.

However, especially the `.toc` file might be read in L-R mode (in cases the `\tableofcontents` attempts to put, say, a list of sub-sections as a paragraph). In that case the newlines after a line like

```
\contentsline {subsubsection}{\numberline {1.1.1}A C-head}{2}
```

might result in spurious spaces (e.g., when that level is not included).

That could be fixed by reading in the file using `\endlinechar=-1` but that has the danger that it drops some valid endlines that should be converted to spaces (for example, when the user edited the TOC and then used `\nofiles` to preserve it).

So the approach taken instead is this:

- `\addcontentsline` adds the command `\protected@file@percent` to the end of the second argument of `\@writefile` that is written to the `.aux`. As the name indicates this is a protected macro so it doesn't change if it is written out.
- When the `.aux` is read back in at the end of the run, `\@writefile` is executed and writes its second argument unmodified to the file with the extension given by its first argument. Or rather that was how it was in the past.
- Instead we change `\@writefile` slightly: basically it looks at the second argument and if the last token in there is `\protected@file@percent` then it is replaced by a percent character and that is then written out. If not (for example, if the data came from a user issued `\addtocontents`, or from some package that uses `\@writefile` for writing its own files) then the command behaves exactly as before.

\protected@file@percent Dummy cs to be replaced by a percent sign inside \@writefile. If it survives (when used incorrectly) it will expand to nothing in a typsetting context.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2018/12/01}%
⟨latexrelease⟩                    {\protected@file@percent}{Mask line endings}%
\protected\def\protected@file@percent{}
```

(*End of definition for* \protected@file@percent.)

\add@percent@to@temptokena Helper function which is used to inspect a sequence of tokens (the second argument of \@writefile and it the last token is \protected@file@percent it will replace it by a harmless percent. The result is saved in \@temptokena for later use.

```
\catcode`\^^A=9
\long\gdef\add@percent@to@temptokena
    #1\protected@file@percent#2\add@percent@to@temptokena
```

When we call this macro in \@writefile we stick in \@empty at the beginning, so that in case the tokenlist consists of a single brace group the braces aren't stripped. The \expandafter then expands this extra token away again.

```
    {\expandafter\ifx\expandafter X\detokenize{#2}X\expandafter\dont@add@percent@to@temptoken
             \expandafter\do@add@percent@to@temptokena\fi{#1}}
\long\def\dont@add@percent@to@temptokena#1{%
  \@temptokena\expandafter{#1}}
```

latexrelease will read this code in high-speed mode in certain situations. During that it will only look for \if tests but not actually execute the \catcode change above. As a result it will drop anything after the % character in the definition. Therefore the \fi needs to be on the next line and we need locally another comment character to avoid getting spaces into the definition—a weird problem :-)

```
\begingroup
\catcode`\%=12
\catcode`\^^A=14
\long\gdef\do@add@percent@to@temptokena#1{\@temptokena\expandafter{#1%^^A
```

Can't be on the same line as the % — see above.

```
  }}
\endgroup
```

(*End of definition for* \add@percent@to@temptokena.)

\@writefile

```
\long\def\@writefile#1#2{%
  \@ifundefined{tf@#1}\relax
    {%
```

If we write to the file we first prepare #2 using \add@percent@to@temptokena and then write the token register out.

```
      \add@percent@to@temptokena
        \@empty#2\protected@file@percent
        \add@percent@to@temptokena
     \immediate\write\csname tf@#1\endcsname{\the\@temptokena}%
    }%
}
```

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\protected@file@percent}{Mask line endings}%
⟨latexrelease⟩\let\protected@file@percent\@undefined
⟨latexrelease⟩\let\add@percent@to@temptokena\@undefined
⟨latexrelease⟩\let\do@add@percent@to@temptokena\@undefined
⟨latexrelease⟩\let\dont@add@percent@to@temptokena\@undefined
⟨latexrelease⟩\long\def\@writefile#1#2{%
⟨latexrelease⟩  \@ifundefined{tf@#1}\relax
⟨latexrelease⟩    {\@temptokena{#2}%
⟨latexrelease⟩     \immediate\write\csname tf@#1\endcsname{\the\@temptokena}%
⟨latexrelease⟩    }%
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@writefile`.)

`\stop`

```
\def\stop{\clearpage\deadcycles\z@\let\par\@@par\@@end}
```

(*End of definition for* `\stop`.)

Historical LaTeX 2.09 comments (not necessarily accurate any more):

```
\everypar{\@nodocument} %% To get an error if text appears before the
\nullfont               %% \begin{document}
```

`\begin`, `\end`, and `\@checkend` changed so `\end{`document`}` will catch an unmatched `\begin`. Changed 24 May 89 as suggested by Frank Mittelbach and Rainer Sch\"opf.

```
\begin{NAME} ==
 BEGIN
   IF \NAME undefined  THEN  \reserved@a == BEGIN report error END
                       ELSE  \reserved@a ==
                                  (\@currenvir :=L NAME) \NAME
   FI
   @ignore :=G F       %% Added 30 Nov 88
   \begingroup
   \@endpe := F
   \@currenvir :=L NAME
   \NAME
 END

\end{NAME} ==
 BEGIN
  \endNAME
  \@checkend{NAME}
  \endgroup
  IF @endpe = T              %% @endpe set True by \@endparenv
```

```
    THEN \@doendpe          %% \@doendpe redefines \par and \everypar
                            %% to suppress paragraph indentation in
  FI                        %% immediately following text
  IF @ignore = T
    THEN @ignore :=G F
         \ignorespaces
  FI
 END

\@checkend{NAME} ==
 BEGIN
  IF \@currenvir = NAME
    ELSE \@badend{NAME}
  FI
 END
```

End of historical LaTeX 2.09 comments.

\begin

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                    {\begin}{Use hook system}%
\protected\def\begin#1{%
  \UseHook{env/#1/before}%
  \@ifundefined{#1}%
    {\def\reserved@a{\@latex@error{Environment #1 undefined}\@eha}}%
    {\def\reserved@a{\def\@currenvir{#1}%
        \edef\@currenvline{\on@line}%
        \@execute@begin@hook{#1}%
        \csname #1\endcsname}}%
  \@ignorefalse
  \begingroup\@endpefalse\reserved@a}
```

Before the \document code is executed we have to first undo the \endgroup as there should be none for this environment to avoid that changes on top-level unnecessarily go to TeX's savestack, and we have to initialize all hooks in the hook system. So we need to test for this environment name. But once it has be found all this testing is no longer needed and so we redefine \@execute@begin@hook to simply use the hook.

```
\def\@execute@begin@hook #1{%
  \expandafter\ifx\csname #1\endcsname\document
    \endgroup
    \gdef\@execute@begin@hook##1{\UseHook{env/##1/begin}}%
    \@expl@@@initialize@all@@
  \fi
```

If this is an environment before \begin{document} we just run the hook so this can be outside the test.

```
  \UseHook{env/#1/begin}%
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                  {\begin}{Making \begin/\end robust}%
⟨latexrelease⟩\DeclareRobustCommand\begin[1]{%
⟨latexrelease⟩  \@ifundefined{#1}%
⟨latexrelease⟩    {\def\reserved@a{\@latex@error{Environment #1 undefined}\@eha}}%
⟨latexrelease⟩    {\def\reserved@a{\def\@currenvir{#1}%
⟨latexrelease⟩     \edef\@currenvline{\on@line}%
⟨latexrelease⟩     \csname #1\endcsname}}%
⟨latexrelease⟩  \@ignorefalse
⟨latexrelease⟩  \begingroup\@endpefalse\reserved@a}
⟨latexrelease⟩\EndIncludeInRelease
```

A version that doesn't start out with `\relax` when in typesetting mode would be the following, but since `\begin` issues a `\begingroup` it wouldn't help much with respect to allowing things like `\noalign` or `\multicolumn` inside.

```
%\edef\begin
%  {\unexpanded{%
%       \ifx\protect\@typeset@protect
%         \expandafter\@gobble
%       \fi
%       \protect
%     }%
%   \expandafter\noexpand\csname begin \endcsname
%  }
%\@namedef{begin }#1{%
%  \@ifundefined{#1}%
%    {\def\reserved@a{\@latex@error{Environment #1 undefined}\@eha}}%
%    {\def\reserved@a{\def\@currenvir{#1}%
%     \edef\@currenvline{\on@line}%
%     \csname #1\endcsname}}%
%  \@ignorefalse
%  \begingroup\@endpefalse\reserved@a}
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\begin}{Making \begin/\end robust}%
⟨latexrelease⟩\def\begin#1{%
⟨latexrelease⟩  \@ifundefined{#1}%
⟨latexrelease⟩    {\def\reserved@a{\@latex@error{Environment #1 undefined}\@eha}}%
⟨latexrelease⟩    {\def\reserved@a{\def\@currenvir{#1}%
⟨latexrelease⟩     \edef\@currenvline{\on@line}%
⟨latexrelease⟩     \csname #1\endcsname}}%
⟨latexrelease⟩  \@ignorefalse
⟨latexrelease⟩  \begingroup\@endpefalse\reserved@a}
⟨latexrelease⟩
```

Also undo the internal commands as some packages unfortunately test for their existence instead of using `\IfFormatAtLeastTF`.

```
⟨latexrelease⟩\expandafter\let\csname begin \endcsname\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\begin`.)

`\end` The top level definition for `\end`.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                    {\end}{Making \begin/\end robust}%
```

While `\begin` was made robust simply by using `\DeclareRobustCommand` we need to be a bit more subtle with `\end` as there are packages out there that try to look into the top-level contents of `\end{foo}` (that is at the expansion of `\endfoo`) to see if it contains certain macros. This is done by hitting `\end{foo}` with three `\expandafter`s, the first to get

```
\csname endfoo\endcsname        \@checkend{foo}% etc.
```

the second to expand the `\csname`, i.e., to get to

```
\endfoo                         \@checkend{foo}% etc.
```

and the third to finally get to the top-level content of `\endfoo`, i.e.

```
<top-level content of \endfoo> \@checkend{foo}% etc.
```

Therefore a robust replacement should produce the same results after three expansions (there first is obviously different).

Basically the definition of `\end` should either produce `\protect\end␣` (when not doing typesetting) or it should produce `\end␣` (without the `\protect`) when doing typesetting. Furthermore, it should (when in typesetting mode) show exactly the same result as `\end␣` (which is the original fragile definition of `\end`) when you expand either of them twice, i.e.,

```
\endfoo                         \@checkend{foo}% etc.
```

That is achieved with the code below (which is worth studying carefully).

There is some trickery involved here: in particular we use `\romannumeral` to change a single expansion into three successive expansions in one go. That primitive expands until it has scanned a number (0 in this case, so it doesn't produce any output) and so it allows us to place arbitrary many `\expandafter`s inside that are all going to be executed when `\romannumeral` is hit by a single `\expandafter`.

```
\edef\end
  {\unexpanded{%
     \romannumeral
       \ifx\protect\@typeset@protect
       \expandafter       %1
         \expandafter        %2
       \expandafter       %1
           \expandafter         %3 expands the \csname inside \end<space>
       \expandafter       %1
         \expandafter        %2  expands \end<space>
       \expandafter       %1     expands the \else
           \z@
       \else
         \expandafter\z@\expandafter\protect
       \fi
   }%
   \expandafter\noexpand\csname end \endcsname
  }
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

And here is the original definition of `\end` the way it was in LaTeX for several decades now hidden in `\end␣`.

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\end}{Making \begin/\end robust}%
⟨latexrelease⟩\def\end#1{%
⟨latexrelease⟩   \csname end#1\endcsname\@checkend{#1}%
⟨latexrelease⟩   \expandafter\endgroup\if@endpe\@doendpe\fi
⟨latexrelease⟩   \if@ignore\@ignorefalse\ignorespaces\fi}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\end`.)

`\end\verbvisiblespace` The internal version with a space at the end.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2024/11/01}%
⟨latexrelease⟩                    {\end!space}{New @endpe handling}%
\@namedef{end }#1{%
  \romannumeral
    \IfHookEmptyTF{env/#1/end}%
        {\expandafter\z@}%
        {\z@\UseHook{env/#1/end}}%
    \csname end#1\endcsname\@checkend{#1}%
```

We can now close the environment group and due to the new `\if@endpe` handling we no longer need to `\expandafter` out of the group.

```
%    \expandafter\endgroup\if@endpe\@doendpe\fi
    \endgroup
    \UseHook{env/#1/after}%
    \if@ignore\@ignorefalse\ignorespaces\fi
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

Version that adds hooks (so different from the 2019 version). It fixes tlb3722 but the change should perhaps be made in `tabularx` instead.

```
%    \begin{macrocode}
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                    {\end!space}{Use hook system}%
⟨latexrelease⟩
⟨latexrelease⟩\@namedef{end }#1{%
⟨latexrelease⟩  \romannumeral
⟨latexrelease⟩    \IfHookEmptyTF{env/#1/end}%
⟨latexrelease⟩        {\expandafter\z@}%
⟨latexrelease⟩        {\z@\UseHook{env/#1/end}}%
⟨latexrelease⟩    \csname end#1\endcsname\@checkend{#1}%
⟨latexrelease⟩    \expandafter\endgroup\if@endpe\@doendpe\fi
⟨latexrelease⟩    \UseHook{env/#1/after}%
⟨latexrelease⟩    \if@ignore\@ignorefalse\ignorespaces\fi
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
```

Version without the fix for tlb3722 for the record:

```
@namedef{end }#1{%
 \UseHook{env/#1/end}%
 \csname end#1\endcsname\@checkend{#1}%
 \expandafter\endgroup\if@endpe\@doendpe\fi
 \UseHook{env/#1/after}%
 \if@ignore\@ignorefalse\ignorespaces\fi}%
```

```
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                  {\end!space}{Making \begin/\end robust}%
⟨latexrelease⟩
⟨latexrelease⟩\@namedef{end }#1{%
⟨latexrelease⟩  \csname end#1\endcsname\@checkend{#1}%
⟨latexrelease⟩  \expandafter\endgroup\if@endpe\@doendpe\fi
⟨latexrelease⟩  \if@ignore\@ignorefalse\ignorespaces\fi}
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\end!space}{Making \begin/\end robust}%
```

Undo the internal command as some packages unfortunately test for their existence instead of using `\IfFormatAtLeastTF`.

```
⟨latexrelease⟩\expandafter\let\csname end \endcsname\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\end\verbvisiblespace`.)

`\@checkend`

```
\def\@checkend#1{\def\reserved@a{#1}\ifx
      \reserved@a\@currenvir \else\@badend{#1}\fi}
```

(*End of definition for* `\@checkend`.)

`\@currenvline` We do need a default value for `\@currenvline` on top-level since the document environment cancels the brace group. This means that a mismatch with `\begin{document}` will not produce a line number. Thus the outer default must be `\@empty` or we will end up with two spaces.

```
\let\@currenvline\@empty
```

(*End of definition for* `\@currenvline`.)

`\AtBeginEnvironment` `\AtEndEnvironment` `\BeforeBeginEnvironment` `\AfterEndEnvironment` We provide 4 high-level hook interfaces directly, the others only when etoolbox is loaded

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\AtBeginEnvironment}{Hooks for environments}%

\newcommand\AtBeginEnvironment[2][.]    {\AddToHook{env/#2/begin}[#1]}
\newcommand\AtEndEnvironment[2][.]      {\AddToHook{env/#2/end}[#1]}
\newcommand\BeforeBeginEnvironment[2][.]{\AddToHook{env/#2/before}[#1]}
\newcommand\AfterEndEnvironment[2][.]   {\AddToHook{env/#2/after}[#1]}

⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\AtBeginEnvironment}{Hooks for environments}%
⟨latexrelease⟩
⟨latexrelease⟩\let\AtBeginEnvironment\@undefined
⟨latexrelease⟩\let\AtEndEnvironment\@undefined
⟨latexrelease⟩\let\BeforeBeginEnvironment\@undefined
⟨latexrelease⟩\let\AfterEndEnvironment\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\AtBeginEnvironment` *and others. These functions are documented on page 241.*)

1.2 Center, Flushright, Flushleft

```
\message{center,}
```

Historical LaTeX 2.09 comments (not necessarily accurate any more):

`\center`, `\flushright` and `\flushleft` set
`\rightskip` = 0pt or `\@flushglue` (as appropriate)
`\leftskip` = 0pt or `\@flushglue` (as appropriate)
`\parindent` = 0pt
`\parfillskip` = 0pt. (except `\flushleft`)
`\\` == `\par \vskip -\parskip`
`\\`[LENGTH] == `\\ \vskip` LENGTH
`\\*` == `\par \penalty 10000 \vskip -\parskip`
`\\*`[LEN] == `\\* \vskip` LENGTH

They invoke the trivlist environment to handle vertical spacing before and after them.

`\centering`, `\raggedright` and `\raggedleft` are the declaration analogs of the above.

`\raggedright` has a more universal effect, however. It sets `\@rightskip` := flushglue. Every environment, like the list environments,
that set `\rightskip` to its 'normal' value set it to `\@rightskip`

End of historical LaTeX 2.09 comments.

`\@centercr`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
⟨latexrelease⟩                  {\@centercr}{Make robust}%
\protected\def\@centercr{\ifhmode \unskip\else \@nolnerr\fi
       \par\@ifstar{\nobreak\@xcentercr}\@xcentercr}
⟨/2ekernel | latexrelease⟩
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@centercr}{Make robust}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@centercr{\ifhmode \unskip\else \@nolnerr\fi
⟨latexrelease⟩        \par\@ifstar{\nobreak\@xcentercr}\@xcentercr}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@centercr`.)

`\@xcentercr`

```
\def\@xcentercr{\addvspace{-\parskip}\@ifnextchar
    [\@icentercr\ignorespaces}
```

(*End of definition for* `\@xcentercr`.)

`\@icentercr`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\@icentercr}{centering, etc support calc}%
\def\@icentercr[#1]{\@vspace@calcify{#1}\ignorespaces}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@icentercr}{centering, etc support calc}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@icentercr[#1]{\vskip #1\ignorespaces}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@icentercr`.)

`center` (*env.*) We use `\relax` to prevent `\item` scanning too far.

```
\def\center{\trivlist \centering\item\relax}

\def\endcenter{\endtrivlist}

⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\centering}{Set finaldhypendemerits}%
```

`\centering`

```
\DeclareRobustCommand\centering{%
  \let\\\@centercr
  \rightskip\@flushglue\leftskip\@flushglue
  \finalhyphendemerits=\z@
  \parindent\z@\parfillskip\z@skip}
```

(*End of definition for* `\centering`.)

\raggedright

```
\DeclareRobustCommand\raggedright{%
  \let\\\@centercr\@rightskip\@flushglue \rightskip\@rightskip
  \finalhyphendemerits=\z@
  \leftskip\z@skip
  \parindent\z@}
```

(*End of definition for* `\raggedright`.)

\raggedleft

```
\DeclareRobustCommand\raggedleft{%
  \let\\\@centercr
  \rightskip\z@skip\leftskip\@flushglue
  \finalhyphendemerits=\z@
  \parindent\z@\parfillskip\z@skip}
```

(*End of definition for* `\raggedleft`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                  {\centering}{Make commands robust}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\centering{%
⟨latexrelease⟩  \let\\\@centercr
⟨latexrelease⟩  \rightskip\@flushglue\leftskip\@flushglue
⟨latexrelease⟩  \parindent\z@\parfillskip\z@skip}
⟨latexrelease⟩\DeclareRobustCommand\raggedright{%
⟨latexrelease⟩  \let\\\@centercr\@rightskip\@flushglue \rightskip\@rightskip
⟨latexrelease⟩  \leftskip\z@skip
⟨latexrelease⟩  \parindent\z@}
⟨latexrelease⟩\DeclareRobustCommand\raggedleft{%
⟨latexrelease⟩  \let\\\@centercr
⟨latexrelease⟩  \rightskip\z@skip\leftskip\@flushglue
⟨latexrelease⟩  \parindent\z@\parfillskip\z@skip}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\centering}{Make commands robust}%
⟨latexrelease⟩
⟨latexrelease⟩\kernel@make@fragile\centering
⟨latexrelease⟩\kernel@make@fragile\raggedright
⟨latexrelease⟩\kernel@make@fragile\raggedleft
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

\@rightskip

```
\newskip\@rightskip \@rightskip \z@skip
```

(*End of definition for* `\@rightskip`.)

`flushleft` (*env.*) We use `\relax` to prevent `\item` scanning too far.

```
\def\flushleft{\trivlist \raggedright\item\relax}
\def\endflushleft{\endtrivlist}
```

`flushright` (*env.*) We use `\relax` to prevent `\item` scanning too far.

```
\def\flushright{\trivlist \raggedleft\item\relax}
\def\endflushright{\endtrivlist}
```

1.3 Verbatim

```
\message{verbatim,}
```

The verbatim environment uses the fixed-width `\ttfamily` font, turns blanks into spaces, starts a new line for each carriage return (or sequence of consecutive carriage returns), and interprets *every* character literally. I.e., all special characters `\, {, $`, etc. are `\catcode`'d to 'other'.

The command `\verb` produces in-line verbatim text, where the argument is delimited by any pair of characters. E.g., `\verb #...#` takes '`...`' as its argument, and sets it verbatim in `\ttfamily` font.

The *-variants of these commands are the same, except that spaces print as the TEXbook's space character instead of as blank spaces.

`\@vobeyspaces`
`\@vobeytabs`

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2023/11/01}%
⟨latexrelease⟩                    {\@vobeytabs}{Obeyed tabs}%
⟨*2ekernel | latexrelease⟩
{\catcode`\ =\active%
\gdef\@vobeyspaces{\catcode`\ \active\let \@xobeysp\@vobeytabs}}
{\catcode`\^^I=\active
\gdef\@vobeytabs{\catcode`\^^I\active\let^^I\@xobeytab}%
\global\let^^I=\space
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\@vobeytabs}{Obeyed tabs}%
⟨latexrelease⟩{\catcode`\ =\active%
⟨latexrelease⟩\gdef\@vobeyspaces{\catcode`\ \active\let \@xobeysp}}
⟨latexrelease⟩\let\@vobeytabs\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@vobeyspaces` *and* `\@vobeytabs`.)

`\@xobeysp`

(*End of definition for* `\@xobeysp`.)

`\@xverbatim`
`\@sxverbatim`

```
\begingroup \catcode `|=0 \catcode `[= 1
\catcode`]=2 \catcode `\{=12 \catcode `\}=12
\catcode`\\=12 |gdef|@xverbatim#1\end{verbatim}[#1|end[verbatim]]
|gdef|@sxverbatim#1\end{verbatim*}[#1|end[verbatim*]]
|endgroup
```

(*End of definition for* `\@xverbatim` *and* `\@sxverbatim`.)

`\@verbatim` Real start of verbatim environment We use `\relax` to prevent `\item` scanning too far.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2017-04-15}{\@verbatim}%
⟨latexrelease⟩                  {Disable hyphenation in verbatim}%
\def\@verbatim{\trivlist \item\relax
  \if@minipage\else\vskip\parskip\fi
  \leftskip\@totalleftmargin\rightskip\z@skip
  \parindent\z@\parfillskip\@flushglue\parskip\z@skip
```

Added `\@@par` to clear possible `\parshape` definition from a surrounding list (the verbatim guru says). Switch language when in vertical mode.

```
  \@@par
```

Set `\language` here to suppress hyphenation. Done this way rather than setting `\hyphenchar` as that is a global setting.

```
  \language\l@nohyphenation
  \@tempswafalse
  \def\par{%
    \if@tempswa
```

A `\leavevmode` added: needed if, for example, a blank verbatim line is the first thing in a list item (wow!).

```
      \leavevmode \null \@@par\penalty\interlinepenalty
    \else
      \@tempswatrue
      \ifhmode\@@par\penalty\interlinepenalty\fi
    \fi}%
```

To allow customization we hide the font used in a separate macro.

```
  \let\do\@makeother \dospecials
  \obeylines \verbatim@font \@noligs
```

To avoid a breakpoint after the labels box, we remove the penalty put there by the list macros: another use of `\unpenalty`!

```
  \everypar \expandafter{\the\everypar \unpenalty}%
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000-00-00}{\@verbatim}%
⟨latexrelease⟩                  {Disable hyphenation in verbatim}%
⟨latexrelease⟩\def\@verbatim{\trivlist \item\relax
⟨latexrelease⟩  \if@minipage\else\vskip\parskip\fi
⟨latexrelease⟩  \leftskip\@totalleftmargin\rightskip\z@skip
⟨latexrelease⟩  \parindent\z@\parfillskip\@flushglue\parskip\z@skip
⟨latexrelease⟩  \@@par
⟨latexrelease⟩  \@tempswafalse
⟨latexrelease⟩  \def\par{%
⟨latexrelease⟩    \if@tempswa
⟨latexrelease⟩      \leavevmode \null \@@par\penalty\interlinepenalty
⟨latexrelease⟩    \else
⟨latexrelease⟩      \@tempswatrue
⟨latexrelease⟩      \ifhmode\@@par\penalty\interlinepenalty\fi
⟨latexrelease⟩    \fi}%
⟨latexrelease⟩  \let\do\@makeother \dospecials
```

```
⟨latexrelease⟩  \obeylines \verbatim@font \@noligs
⟨latexrelease⟩  \hyphenchar\font\m@ne
⟨latexrelease⟩  \everypar \expandafter{\the\everypar \unpenalty}%
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@verbatim`.)

`verbatim` (*env.*) `\endverbatim` (RmS 93/09/19) Protected against 'missing item' error message triggered by empty verbatim environment.

```
\def\verbatim{\@verbatim \frenchspacing\@vobeyspaces \@xverbatim}
\def\endverbatim{\if@newlist \leavevmode\fi\endtrivlist}
```

(*End of definition for* `\verbatim` *and* `\endverbatim`.)

`\verbatim@font` Macro to select the font used for verbatim typesetting. It also does other work if necessary for the font used.

```
\def\verbatim@font{\normalfont\ttfamily}
```

(*End of definition for* `\verbatim@font`.)

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2018/12/01}%
⟨latexrelease⟩                    {\verbvisiblespace}{Setup visible space for \verb}%
```

`\asciispace` The character in slot 32, in typewriter fonts (historically) a visible space but in other fonts a real space or something else

```
\DeclareRobustCommand\asciispace{\char 32 }
```

(*End of definition for* `\asciispace`.)

`\verbvisiblespace` This defines how to get a visible space in `\verb*` and friends. In classic TEX this is just the slot 32, but in TU encoded fonts we switch fonts and take the character from cmtt.

```
\ifx\Umathcode\@undefined
  \let\verbvisiblespace\asciispace                              % Pdftex version
\else
  \DeclareRobustCommand\verbvisiblespace
           {\leavevmode{\usefont{OT1}{cmtt}{m}{n}\asciispace}}    % xetex/luatex version
\fi
```

(*End of definition for* `\verbvisiblespace`.)

`\@verbvisiblespacebox` The box to hold the visible space character if it isn't in slot 32 in the current typewriter font.

```
\newbox\@verbvisiblespacebox
```

(*End of definition for* `\@verbvisiblespacebox`.)

`verbatim*` (*env.*) For `verbatim*` we also set up the correct visible space character definition and then run `\@vobeyspaces`. As this code is not called as part of the normal verbatim environment (the method is done the other way around this time) we don't have to check if space is already active—it shouldn't be.

```
\@namedef{verbatim*}{\@verbatim
  \@setupverbvisiblespace
  \frenchspacing\@vobeyspaces\@sxverbatim}
\expandafter\let\csname endverbatim*\endcsname =\endverbatim
```

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\verbvisiblespace}{Setup visible space for \verb}%
⟨latexrelease⟩
⟨latexrelease⟩\@namedef{verbatim*}{\@verbatim\@sxverbatim}
⟨latexrelease⟩
⟨latexrelease⟩\let\asciispace            \@undefined
⟨latexrelease⟩\let\verbvisiblespace      \@undefined
⟨latexrelease⟩\let\@setupverbvisiblespace\@undefined
⟨latexrelease⟩\let\@verbvisiblespacebox  \@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

`\@setupverbvisiblespace` In pdfTEX a catcode 12 space will produce the character in slot 32 which is assumed to be a visible space character (in a typewriter font in OT1 or T1 encoding). In XeTEX or LuaTEX a font in TU encoding is normally used and that has a real space in this slot. So what we do in this case is this: we check the definition of `\verbvisiblespace` and if it is `\asciispace` we assume that the char32 can be used (e.g., in pdfTEX). We then redefine `\@xobeysp` so that after running `\@vobeyspaces` we get characters from slot 32 for each active space.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2023/11/01}%
⟨latexrelease⟩                  {\@setupverbvisiblespace}{Setup visible tab for \verb}%
\def\@setupverbvisiblespace{%
  \ifx\verbvisiblespace\asciispace
    \let\@xobeysp\asciispace
  \else
```

Otherwise we measure the width of a character in the mono-spaced current font and place a `\verbvisiblespace` into a box of the right width which we are then using as the character for a space. By default this will be the space character from OT1 cmtt but by changing `\verbvisiblespace` one could use, for example, the `\textvisiblespace` of the current typewriter font.

```
    \setbox\z@\hbox{x}%
    \setbox\@verbvisiblespacebox\hbox to\wd\z@{\hss\verbvisiblespace\hss}%
    \def\@xobeysp{\leavevmode\copy\@verbvisiblespacebox}%
  \fi
  \@setupverbvisibletab
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2018/12/01}%
⟨latexrelease⟩                  {\@setupverbvisiblespace}{Setup visible space for \verb}%
⟨latexrelease⟩\def\@setupverbvisiblespace{%
⟨latexrelease⟩  \ifx\verbvisiblespace\asciispace
⟨latexrelease⟩    \let\@xobeysp\asciispace
⟨latexrelease⟩  \else
⟨latexrelease⟩    \setbox\z@\hbox{x}%
⟨latexrelease⟩    \setbox\@verbvisiblespacebox\hbox to\wd\z@{\hss\verbvisiblespace\hss}%
⟨latexrelease⟩    \def\@xobeysp{\leavevmode\copy\@verbvisiblespacebox}%
⟨latexrelease⟩  \fi
```

```
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\@setupverbvisiblespace}{Setup visible space for \verb}%
⟨latexrelease⟩\let\@setupverbvisiblespace\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@setupverbvisiblespace`.)

`\@setupverbvisibletab` A redirection: just a simple wrapper.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2023/11/01}%
⟨latexrelease⟩                    {\@setupverbvisibletab}{Setup visible tab for \verb}%
⟨*2ekernel | latexrelease⟩
\def\@setupverbvisibletab{\let\@xobeytab\@xobeysp}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\@setupverbvisibletab}{Setup visible tab for \verb}%
⟨latexrelease⟩\let\@setupverbvisibletab\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@setupverbvisibletab`.)

`\@sverb` `\@@sverb` Definitions of `\@sverb` and `\@verb` changed so `\verb+ foo+` does not lose leading blanks when it comes at the beginning of a line. Change made 24 May 89. Suggested by Frank Mittelbach and Rainer Schöpf.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2023/11/01}%
⟨latexrelease⟩                    {\@sverb}{Support visible tabs}%
```

If the users types `\verb !~! foo` then surprisingly we would get the space as the delimiter and thus "`!~!foo`" in the output. To avoid this scenario we check if `#1` has the character code of a space, if so we recurse otherwise we call `\@@sverb` (which is the original definition of `\@sverb`).

```
\def\@sverb#1{\if\noexpand#1 \expandafter\@sverb\else\@@sverb{#1}\fi}

\def\@@sverb#1{%
  \catcode`#1\active
  \lccode`\~`#1%
  \gdef\verb@balance@group{\verb@egroup
     \@latex@error{\noexpand\verb illegal in argument}\@ehc}%
  \aftergroup\verb@balance@group
  \lowercase{\let~\verb@egroup}%
```

If `\@sverb` is called from `\@verb` then space is already active and supposed to produce a real space. In this case we do nothing. Otherwise we run `\@setupverbvisiblespace` to setup the right visible space char and afterwards `\@vobeyspaces` to make it the definition for the active space character.

```
  \ifnum0%
    \ifnum\catcode`\ =\active\else 1\fi
    \ifnum\catcode`\^^I=\active\else 1\fi
```

```
    =0 %
  \else  \@setupverbvisiblespace \@vobeyspaces \fi
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\@sverb}{Drop spaces before \verb delimiter}%
⟨latexrelease⟩\def\@@sverb#1{%
⟨latexrelease⟩  \catcode`#1\active
⟨latexrelease⟩  \lccode`\~`#1%
⟨latexrelease⟩  \gdef\verb@balance@group{\verb@egroup
⟨latexrelease⟩     \@latex@error{\noexpand\verb illegal in argument}\@ehc}%
⟨latexrelease⟩  \aftergroup\verb@balance@group
⟨latexrelease⟩  \lowercase{\let~\verb@egroup}%
⟨latexrelease⟩  \ifnum\catcode`\ =\active
⟨latexrelease⟩  \else  \@setupverbvisiblespace \@vobeyspaces \fi
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2018/12/01}%
⟨latexrelease⟩                  {\@sverb}{Setup visible space for \verb}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@sverb#1{%
⟨latexrelease⟩  \catcode`#1\active
⟨latexrelease⟩  \lccode`\~`#1%
⟨latexrelease⟩  \gdef\verb@balance@group{\verb@egroup
⟨latexrelease⟩     \@latex@error{\noexpand\verb illegal in command argument}\@ehc}%
⟨latexrelease⟩  \aftergroup\verb@balance@group
⟨latexrelease⟩  \lowercase{\let~\verb@egroup}%
⟨latexrelease⟩  \ifnum\catcode`\ =\active
⟨latexrelease⟩  \else  \@setupverbvisiblespace \@vobeyspaces \fi
⟨latexrelease⟩}
⟨latexrelease⟩\let\@@sverb\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@sverb}{Setup visible space for \verb}%
⟨latexrelease⟩\def\@sverb#1{%
⟨latexrelease⟩  \catcode`#1\active
⟨latexrelease⟩  \lccode`\~`#1%
⟨latexrelease⟩  \gdef\verb@balance@group{\verb@egroup
⟨latexrelease⟩     \@latex@error{\noexpand\verb illegal in command argument}\@ehc}%
⟨latexrelease⟩  \aftergroup\verb@balance@group
⟨latexrelease⟩  \lowercase{\let~\verb@egroup}}%
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@sverb` *and* `\@@sverb`.)

`\@makeother`

```
\def\@makeother#1{\catcode`#112\relax}
```

(*End of definition for* `\@makeother`.)

\verb@balance@group

```
\let\verb@balance@group\@empty
```

(*End of definition for* `\verb@balance@group`.)

\verb@egroup

```
\def\verb@egroup{\global\let\verb@balance@group\@empty\egroup}
```

(*End of definition for* `\verb@egroup`.)

\verb@eol@error

```
\begingroup
  \obeylines%
  \gdef\verb@eol@error{\obeylines%
    \def^^M{\verb@egroup\@latex@error{%
              \noexpand\verb ended by end of line}\@ehc}}%
\endgroup
```

(*End of definition for* `\verb@eol@error`.)

\verb Typesetting a small piece verbatim.

```
⟨/2ekernel⟩
⟨∗2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2017-04-15}{\verb}%
⟨latexrelease⟩                  {Disable hyphenation in verb}%
\def\verb{\relax\ifmmode\hbox\else\leavevmode\null\fi
  \bgroup
    \verb@eol@error \let\do\@makeother \dospecials
    \verb@font\@noligs
```

Set `\language` here to suppress hyphenation. Done this way rather than setting `\hyphenchar` as that is a global setting.

```
    \language\l@nohyphenation
    \@ifstar\@sverb\@verb}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000-00-00}{\verb}%
⟨latexrelease⟩                  {Disable hyphenation in verb}%
⟨latexrelease⟩\def\verb{\relax\ifmmode\hbox\else\leavevmode\null\fi
⟨latexrelease⟩  \bgroup
⟨latexrelease⟩    \verb@eol@error \let\do\@makeother \dospecials
⟨latexrelease⟩    \verb@font\@noligs
⟨latexrelease⟩    \@ifstar\@sverb\@verb}
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\verb`.)

\@verb

```
\def\@verb{\@vobeyspaces \frenchspacing \@sverb}
```

(*End of definition for* `\@verb`.)

\verbatim@nolig@list

```
\def\verbatim@nolig@list{\do\`\do\<\do\>\do\,\do\'\do\-}
```

(*End of definition for* `\verbatim@nolig@list`.)

`\do@noligs`

```
\def\do@noligs#1{%
  \catcode`#1\active
  \begingroup
     \lccode`\~`#1\relax
     \lowercase{\endgroup\def~{\leavevmode\kern\z@\char`#1}}}
```

(*End of definition for* `\do@noligs`.)

`\@noligs` To stay compatible with packages that use `\@noligs` we keep it.

```
\def\@noligs{\let\do\do@noligs \verbatim@nolig@list}
```

(*End of definition for* `\@noligs`.)

```
⟨/2ekernel⟩
```

File 38
ltmath.dtx

1 Math setup

This file contains a lot of the original plain TEX code, as well as the LATEX environments for math. It still needs sorting out.

```
⟨*2ekernel⟩
\message{math definitions,}
```

1.1 Math commands based on plain TEX

1.1.1 The log-like functions

\log The standard operators:

```
\DeclareRobustCommand\log{\mathop{\operator@font log}\nolimits}
\DeclareRobustCommand\lg{\mathop{\operator@font lg}\nolimits}
\DeclareRobustCommand\ln{\mathop{\operator@font ln}\nolimits}
\DeclareRobustCommand\lim{\mathop{\operator@font lim}}
\DeclareRobustCommand\limsup{\mathop{\operator@font lim\,sup}}
\DeclareRobustCommand\liminf{\mathop{\operator@font lim\,inf}}
\DeclareRobustCommand\sin{\mathop{\operator@font sin}\nolimits}
\DeclareRobustCommand\arcsin{\mathop{\operator@font arcsin}\nolimits}
\DeclareRobustCommand\sinh{\mathop{\operator@font sinh}\nolimits}
\DeclareRobustCommand\cos{\mathop{\operator@font cos}\nolimits}
\DeclareRobustCommand\arccos{\mathop{\operator@font arccos}\nolimits}
\DeclareRobustCommand\cosh{\mathop{\operator@font cosh}\nolimits}
\DeclareRobustCommand\tan{\mathop{\operator@font tan}\nolimits}
\DeclareRobustCommand\arctan{\mathop{\operator@font arctan}\nolimits}
\DeclareRobustCommand\tanh{\mathop{\operator@font tanh}\nolimits}
\DeclareRobustCommand\cot{\mathop{\operator@font cot}\nolimits}
\DeclareRobustCommand\coth{\mathop{\operator@font coth}\nolimits}
\DeclareRobustCommand\sec{\mathop{\operator@font sec}\nolimits}
\DeclareRobustCommand\csc{\mathop{\operator@font csc}\nolimits}
\DeclareRobustCommand\max{\mathop{\operator@font max}}
\DeclareRobustCommand\min{\mathop{\operator@font min}}
\DeclareRobustCommand\sup{\mathop{\operator@font sup}}
\DeclareRobustCommand\inf{\mathop{\operator@font inf}}
\DeclareRobustCommand\arg{\mathop{\operator@font arg}\nolimits}
\DeclareRobustCommand\ker{\mathop{\operator@font ker}\nolimits}
\DeclareRobustCommand\dim{\mathop{\operator@font dim}\nolimits}
\DeclareRobustCommand\hom{\mathop{\operator@font hom}\nolimits}
\DeclareRobustCommand\det{\mathop{\operator@font det}}
\DeclareRobustCommand\exp{\mathop{\operator@font exp}\nolimits}
\DeclareRobustCommand\Pr{\mathop{\operator@font Pr}}
\DeclareRobustCommand\gcd{\mathop{\operator@font gcd}}
\DeclareRobustCommand\deg{\mathop{\operator@font deg}\nolimits}
```

(*End of definition for* \log.)

\bmod And some operators have to be done by hand:

```
\DeclareRobustCommand\bmod{%
  \nonscript\mskip-\medmuskip\mkern5mu%
  \mathbin{\operator@font mod}\penalty900\mkern5mu%
  \nonscript\mskip-\medmuskip}
```

(*End of definition for* `\bmod`.)

`\pmod`

```
\DeclareRobustCommand\pmod[1]{%
  \allowbreak\mkern18mu({\operator@font mod}\,\,#1)}
```

(*End of definition for* `\pmod`.)

1.1.2 Biggggg

`\big` Variants on `\big` and friends for use with delimiters:

```
\DeclareRobustCommand\bigl{\mathopen\big}
\DeclareRobustCommand\bigm{\mathrel\big}
\DeclareRobustCommand\bigr{\mathclose\big}
\DeclareRobustCommand\Bigl{\mathopen\Big}
\DeclareRobustCommand\Bigm{\mathrel\Big}
\DeclareRobustCommand\Bigr{\mathclose\Big}
\DeclareRobustCommand\biggl{\mathopen\bigg}
\DeclareRobustCommand\biggm{\mathrel\bigg}
\DeclareRobustCommand\biggr{\mathclose\bigg}
\DeclareRobustCommand\Biggl{\mathopen\Bigg}
\DeclareRobustCommand\Biggm{\mathrel\Bigg}
\DeclareRobustCommand\Biggr{\mathclose\Bigg}
```

(*End of definition for* `\big`.)

1.1.3 The UNSORTED Rest

The other math commands are lifted from plain TEX.

`\jot`

```
\newdimen\jot
\jot=3pt
```

(*End of definition for* `\jot`.)

`\interdisplaylinepenalty`

```
\newcount\interdisplaylinepenalty
\interdisplaylinepenalty=100
```

(*End of definition for* `\interdisplaylinepenalty`.)

`\choose`

```
\def\choose{\atopwithdelims()}
```

(*End of definition for* `\choose`.)

`\brack`

```
\def\brack{\atopwithdelims[]}
```

(*End of definition for* `\brack`.)

\brace

```
\def\brace{\atopwithdelims\{\}}
```

(*End of definition for* \brace.)

\mathpalette

```
\def\mathpalette#1#2{%
  \mathchoice
    {#1\displaystyle{#2}}%
    {#1\textstyle{#2}}%
    {#1\scriptstyle{#2}}%
    {#1\scriptscriptstyle{#2}}}
```

(*End of definition for* \mathpalette.)

\root
\rootbox
\r@@t

```
\newbox\rootbox
\def\root#1\of{%
  \setbox\rootbox\hbox{$\m@th\scriptscriptstyle{#1}$}%
  \mathpalette\r@@t}
\def\r@@t#1#2{%
  \setbox\z@\hbox{$\m@th#1\sqrtsign{#2}$}%
  \dimen@\ht\z@ \advance\dimen@-\dp\z@
  \mkern5mu\raise.6\dimen@\copy\rootbox
  \mkern-10mu\box\z@}
```

(*End of definition for* \root, \rootbox, *and* \r@@t.)

\phantom
\hphantom
\vphantom

```
\newif\ifv@
\newif\ifh@
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                    {\vphantom}{Make commands robust}%
\DeclareRobustCommand\vphantom{\v@true\h@false\ph@nt}
\DeclareRobustCommand\hphantom{\v@false\h@true\ph@nt}
\DeclareRobustCommand\phantom{\v@true\h@true\ph@nt}


\DeclareRobustCommand\mathstrut{\vphantom(}
```

\mathstrut

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\vphantom}{Make commands robust}%
⟨latexrelease⟩
⟨latexrelease⟩\kernel@make@fragile\vphantom
⟨latexrelease⟩\kernel@make@fragile\hphantom
⟨latexrelease⟩\kernel@make@fragile\phantom
⟨latexrelease⟩\kernel@make@fragile\mathstrut
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

```
\def\ph@nt{%
  \ifmmode
    \expandafter\mathpalette\expandafter\mathph@nt
  \else
    \expandafter\makeph@nt
  \fi}
\def\makeph@nt#1{%
  \setbox\z@\hbox{\color@begingroup#1\color@endgroup}\finph@nt}
\def\mathph@nt#1#2{%
  \setbox\z@\hbox{$\m@th#1{#2}$}\finph@nt}
⟨/2ekernel⟩
⟨∗2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2018/12/01}%
⟨latexrelease⟩                  {\finph@nt}{Start LR-mode}%
\def\finph@nt{%
  \setbox\tw@\null
  \ifv@ \ht\tw@\ht\z@ \dp\tw@\dp\z@\fi
  \ifh@ \wd\tw@\wd\z@\fi

  \leavevmode@ifvmode\box\tw@}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\finph@nt}{Start LR-mode}%
⟨latexrelease⟩\def\finph@nt{%
⟨latexrelease⟩  \setbox\tw@\null
⟨latexrelease⟩  \ifv@ \ht\tw@\ht\z@ \dp\tw@\dp\z@\fi
⟨latexrelease⟩  \ifh@ \wd\tw@\wd\z@\fi \box\tw@}
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\phantom` *and others.*)

`\smash`

```
\DeclareRobustCommand\smash{%
  \relax % \relax, in case this comes first in \halign
  \ifmmode
    \expandafter\mathpalette\expandafter\mathsm@sh
  \else
    \expandafter\makesm@sh
  \fi}
\def\makesm@sh#1{%
  \setbox\z@\hbox{\color@begingroup#1\color@endgroup}\finsm@sh}
⟨/2ekernel⟩
⟨∗2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2022/11/01}%
⟨latexrelease⟩                  {\mathsm@sh}{Guard against reboxing}%
\def\mathsm@sh#1#2{%
  \setbox\z@\hbox{$\m@th#1{#2}$}%
```

The empty brace groups in front of the smashed box (which is placed by `\finsm@sh`) ensures that a `\smash` in math is not just producing a single box with its dimensions altered, but a box plus this second ord atom. The reason is that TEX sometimes reboxes

a box if its the only thing in a place like the denominator of a fraction. This would then undo the smashing and the additional ord atom prevents that. Two ord atoms in a row do not alter the horizontal spacing in a formula so this is otherwise transparent.

```
  {}\finsm@sh}

⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\mathsm@sh}{Guard against reboxing}%
⟨latexrelease⟩\def\mathsm@sh#1#2{%
⟨latexrelease⟩   \setbox\z@\hbox{$\m@th#1{#2}$}\finsm@sh}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩

⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2018/12/01}%
⟨latexrelease⟩                    {\finsm@sh}{Start LR-mode}%
\def\finsm@sh{\ht\z@\z@ \dp\z@\z@ \leavevmode@ifvmode\box\z@}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\finsm@sh}{Start LR-mode}%
⟨latexrelease⟩\def\finsm@sh{\ht\z@\z@ \dp\z@\z@ \box\z@}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\smash`.)

`\buildrel`

```
\def\buildrel#1\over#2{\mathrel{\mathop{\kern\z@#2}\limits^{#1}}}
```

(*End of definition for* `\buildrel`.)

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                    {\cases}{Make commands robust}%
```

`\cases`

```
\DeclareRobustCommand*\cases[1]{\left\{\,\vcenter{\normalbaselines\m@th
    \ialign{$##\hfil$&\quad{##}\hfil\crcr#1\crcr}}\right.}
```

(*End of definition for* `\cases`.)

`\matrix`

```
\DeclareRobustCommand*\matrix[1]{\null\,\vcenter{\normalbaselines\m@th
    \ialign{\hfil$##$\hfil&&\quad\hfil$##$\hfil\crcr
      \mathstrut\crcr\noalign{\kern-\baselineskip}
      #1\crcr\mathstrut\crcr\noalign{\kern-\baselineskip}}}\,}
```

(*End of definition for* `\matrix`.)

`\pmatrix`

```
\DeclareRobustCommand*\pmatrix[1]{\left(\matrix{#1}\right)}
```

(*End of definition for* `\pmatrix`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\cases}{Make commands robust}%
⟨latexrelease⟩
⟨latexrelease⟩\kernel@make@fragile\cases
⟨latexrelease⟩\kernel@make@fragile\matrix
⟨latexrelease⟩\kernel@make@fragile\pmatrix
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

`\bordermatrix`

```
\def\bordermatrix#1{\begingroup \m@th
  \@tempdima 8.75\p@
  \setbox\z@\vbox{%
    \def\cr{\crcr\noalign{\kern2\p@\global\let\cr\endline}}%
    \ialign{$##$\hfil\kern2\p@\kern\@tempdima&\thinspace\hfil$##$\hfil
      &&\quad\hfil$##$\hfil\crcr
      \omit\strut\hfil\crcr\noalign{\kern-\baselineskip}%
      #1\crcr\omit\strut\cr}}%
  \setbox\tw@\vbox{\unvcopy\z@\global\setbox\@ne\lastbox}%
  \setbox\tw@\hbox{\unhbox\@ne\unskip\global\setbox\@ne\lastbox}%
  \setbox\tw@\hbox{$\kern\wd\@ne\kern-\@tempdima\left(\kern-\wd\@ne
    \global\setbox\@ne\vbox{\box\@ne\kern2\p@}%
    \vcenter{\kern-\ht\@ne\unvbox\z@\kern-\baselineskip}\,\right)$}%
  \null\;\vbox{\kern\ht\@ne\box\tw@}\endgroup}
```

(*End of definition for* `\bordermatrix`.)

`\openup`

```
\protected\def\openup{\afterassignment\@penup\dimen@}
\def\@penup{\advance\lineskip\dimen@
  \advance\baselineskip\dimen@
  \advance\lineskiplimit\dimen@}
```

(*End of definition for* `\openup`.)

`\displaylines`

```
\newif\ifdt@p
\def\displ@y{\global\dt@ptrue\openup\jot\m@th
  \everycr{\noalign{\ifdt@p \global\dt@pfalse \ifdim\prevdepth>-1000\p@
      \vskip-\lineskiplimit \vskip\normallineskiplimit \fi
      \else \penalty\interdisplaylinepenalty \fi}}
\def\@lign{\tabskip\z@skip\everycr{}} % restore inside \displ@y
\def\displaylines#1{\displ@y \tabskip\z@skip
  \halign{\hb@xt@\displaywidth{$\@lign\hfil\displaystyle##\hfil$}\crcr
    #1\crcr}}
```

(*End of definition for* `\displaylines`.)

\sp
\sb

```
\let\sp=^
\let\sb=_
```

(*End of definition for* `\sp` *and* `\sb`.)

\tmspace
\
\thinspace
\!
\negthinspace
\:
\medspace
\negmedspace
\;
\thickspace
\negthickspace

Originally LaTeX only provided a small set of spacing commands for use in text and math, some of the commands like `\;` were only supported in math mode. `amsmath` normalized and provided all of them in text and math. This code has now been moved to the kernel so that it is generally available.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                    {\tmspace}{amsmath spacing commands}%
```

`\tmspace` is really meant to be an internal command so it doesn't necessarily has to be robust but it was robust in `amsmath` so we leave it like that.

```
\DeclareRobustCommand\tmspace[3]{%
  \ifmmode\mskip#1#2\else\leavevmode@ifvmode\kern#1#3\fi\relax}
```

In `amsmath` the text kern is `.1667em`. For compatibility reasons we keep the longer one.

```
\DeclareRobustCommand\,{\tmspace+\thinmuskip{.16667em}}
\let\thinspace\,

\DeclareRobustCommand\!{\tmspace-\thinmuskip{.16667em}}
\let\negthinspace\!

\DeclareRobustCommand\:{\tmspace+\medmuskip{.2222em}}
\let\medspace\:
```

LaTeX has a second name for this in its manual:

```
\let\>=\:
\DeclareRobustCommand\negmedspace{\tmspace-\medmuskip{.2222em}}

\DeclareRobustCommand\;{\tmspace+\thickmuskip{.2777em}}
\let\thickspace\;
\DeclareRobustCommand\negthickspace{\tmspace-\thickmuskip{.2777em}}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\tmspace}{amsmath spacing commands}%
⟨latexrelease⟩
⟨latexrelease⟩\let\tmspace\@undefined
⟨latexrelease⟩\DeclareRobustCommand{\,}{%
⟨latexrelease⟩    \relax\ifmmode\mskip\thinmuskip\else\thinspace\fi}
⟨latexrelease⟩\DeclareRobustCommand\thinspace{\leavevmode@ifvmode\kern .16667em }
⟨latexrelease⟩\DeclareRobustCommand\negthinspace{\leavevmode@ifvmode\kern-.16667em }
⟨latexrelease⟩\def\>{\mskip\medmuskip}
⟨latexrelease⟩\let\:=\>
⟨latexrelease⟩\def\;{\mskip\thickmuskip}
⟨latexrelease⟩\def\!{\mskip-\thinmuskip}
⟨latexrelease⟩
```

```
⟨latexrelease⟩\let\negmedspace\@undefined
⟨latexrelease⟩\let\negthickspace\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\tmspace` *and others.*)

`\*`

```
\DeclareRobustCommand\*{\discretionary{\thinspace\the\textfont2\char2}{}{}}
```

(*End of definition for* `\*`.)

`\:` Nickname for the medium space since `\>` is not available inside `tabbing`.

```
%\let\:=\>
```

(*End of definition for* `\:`.)

`\active@math@prime` This is the definition of the active math prime.

```
\def\active@math@prime{^\bgroup\prim@s}
```

(*End of definition for* `\active@math@prime`.)

`\prime@s`

```
{\catcode`\'=\active \global\let'\active@math@prime}
\def\prim@s{%
  \prime\futurelet\@let@token\pr@m@s}
\def\pr@m@s{%
  \ifx'\@let@token
    \expandafter\pr@@@s
  \else
    \ifx^\@let@token
      \expandafter\expandafter\expandafter\pr@@@t
    \else
      \egroup
    \fi
  \fi}
\def\pr@@@s#1{\prim@s}
\def\pr@@@t#1#2{#2\egroup}
```

(*End of definition for* `\prime@s`.)

```
{\catcode`\_=\active \gdef_{\_}} % _ in math is
                                 % either subscript or \_
```

1.2 Math Environments

`\(` `\)` Produces `$...$` with checks that `\(` isn't used in math mode, and that `\)` is only used in math mode begun with `\(`.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}{\(}{Make \( robust}%
⟨*2ekernel | latexrelease⟩
\DeclareRobustCommand\({%
  \relax\ifmmode\@badmath\else$\fi}%
\DeclareRobustCommand\){%
  \relax\ifmmode\ifinner$\else\@badmath\fi\else \@badmath\fi}%
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\(}{Make \( robust}%
⟨latexrelease⟩\def\({%
⟨latexrelease⟩  \relax\ifmmode\@badmath\else$\fi}%
⟨latexrelease⟩\expandafter\let\csname\string( \endcsname\@undefined
⟨latexrelease⟩\def\){%
⟨latexrelease⟩  \relax\ifmmode\ifinner$\else\@badmath\fi\else \@badmath\fi}%
⟨latexrelease⟩\expandafter\let\csname\string) \endcsname\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\(` *and* `\)`.)

`\dollardollar@begin` `\dollardollar@end` Two commands for starting and ending display math formulas to be used in the kernel and by packages. They are needed for tagging support and should be used by all display math environments so that these environments can be used when accessible PDFs are produced. We pretend that they have been always available, so that rollback doesn't break packages that have not set up their own rollback.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease⟩  {\dollardollar@begin}{Start and end display math}%
\def\dollardollar@begin{$$}
\def\dollardollar@end{$$}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩  {\dollardollar@begin}{Start and end display math}%
⟨latexrelease⟩
⟨latexrelease⟩\def\dollardollar@begin{$$}
⟨latexrelease⟩\def\dollardollar@end{$$}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩


```

(*End of definition for* `\dollardollar@begin` *and* `\dollardollar@end`.)

`\[` `\]` Produces `$$...$$` with checks that `\[` isn't used in math mode, and that `\]` is only used in display math mode (though there is no real test that this display math started with `\[` and not with `$$`).

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}{\[}{Make \[ robust}%
⟨*2ekernel | latexrelease⟩
\DeclareRobustCommand\[{%
   \relax\ifmmode
      \@badmath
   \else
      \ifvmode
         \nointerlineskip
         \makebox[.6\linewidth]{}%
      \fi
      $$%                    %  amsthm tries to patch this and expects a $
%      \dollardollar@begin   %  in the def so we can't hide it for now
                             %  will be adjusted when amsthm changes
   \fi
}%
\DeclareRobustCommand\]{%
   \relax\ifmmode
      \ifinner
         \@badmath
      \else
         \dollardollar@end
      \fi
   \else
      \@badmath
   \fi
   \ignorespaces
}%
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2015/01/01}{\[}{Make \[ robust}%
⟨latexrelease⟩\DeclareRobustCommand\[{%
⟨latexrelease⟩    \relax\ifmmode
⟨latexrelease⟩       \@badmath
⟨latexrelease⟩    \else
⟨latexrelease⟩       \ifvmode
⟨latexrelease⟩          \nointerlineskip
⟨latexrelease⟩          \makebox[.6\linewidth]{}%
⟨latexrelease⟩       \fi
⟨latexrelease⟩       $$%%$$ BRACE MATCH HACK
⟨latexrelease⟩    \fi
⟨latexrelease⟩}%
⟨latexrelease⟩\DeclareRobustCommand\]{%
⟨latexrelease⟩    \relax\ifmmode
⟨latexrelease⟩       \ifinner
⟨latexrelease⟩          \@badmath
⟨latexrelease⟩       \else
⟨latexrelease⟩          $$%%$$ BRACE MATCH HACK
⟨latexrelease⟩       \fi
⟨latexrelease⟩    \else
⟨latexrelease⟩       \@badmath
⟨latexrelease⟩    \fi
⟨latexrelease⟩    \ignorespaces
```

```
⟨latexrelease⟩}%
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\[}{Make \[ robust}%
⟨latexrelease⟩\def\[{%
⟨latexrelease⟩    \relax\ifmmode
⟨latexrelease⟩       \@badmath
⟨latexrelease⟩    \else
⟨latexrelease⟩       \ifvmode
⟨latexrelease⟩          \nointerlineskip
⟨latexrelease⟩          \makebox[.6\linewidth]{}%
⟨latexrelease⟩       \fi
⟨latexrelease⟩       $$%$$ BRACE MATCH HACK
⟨latexrelease⟩    \fi
⟨latexrelease⟩}%
⟨latexrelease⟩\expandafter\let\csname\string[ \endcsname\@undefined

⟨latexrelease⟩\def\]{%
⟨latexrelease⟩    \relax\ifmmode
⟨latexrelease⟩       \ifinner
⟨latexrelease⟩          \@badmath
⟨latexrelease⟩       \else
⟨latexrelease⟩          $$%$$ BRACE MATCH HACK
⟨latexrelease⟩       \fi
⟨latexrelease⟩    \else
⟨latexrelease⟩       \@badmath
⟨latexrelease⟩    \fi
⟨latexrelease⟩    \ignorespaces
⟨latexrelease⟩}%
⟨latexrelease⟩\expandafter\let\csname\string] \endcsname\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\[` *and* `\]`.)

`math` (*env.*)
`displaymath` (*env.*)

Disguises for `\(...\)` and `\[...\]`.

```
\let\math=\(
\let\endmath=\)

\def\displaymath{\[}
\def\enddisplaymath{\]\@ignoretrue}
```

`equation` (*env.*)

Numbered equations, using the counter `\c@equation`. *Note*: The document style must define `\theequation` etc., and do the appropriate `\@addtoreset`. It should also redefine `\@eqnnum` if another format for the equation number is desired other than the standard (...), or to move the equation numbers to the flushleft. (See comment on the `\def` of `\@eqnnum`.)

```
\@definecounter{equation}

⟨/2ekernel⟩
⟨∗2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease⟩                  {\equation}{Tagging support}%
\def\equation{\dollardollar@begin
              \refstepcounter{equation}}
\def\endequation{\eqno \hbox{\@eqnnum}\dollardollar@end\@ignoretrue}
```

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\equation}{Tagging support}%
⟨latexrelease⟩
⟨latexrelease⟩\def\equation{$$\refstepcounter{equation}}
⟨latexrelease⟩\def\endequation{\eqno \hbox{\@eqnnum}$$\@ignoretrue}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\c@equation`.)

`\@eqnnum` Produces the equation number for equation and eqnarray environments. The following definition is for flushright numbers; for flushleft numbers, see leqno.clo. The equation number is set in black roman type even if an eqnarray environment appears in an italic environment.

```
\def\@eqnnum{{\normalfont \normalcolor (\theequation)}}
```

(*End of definition for* `\@eqnnum`.)

`\stackrel` A disguise for plain TEX's buildrel.

```
\DeclareRobustCommand\stackrel[2]{\mathrel{\mathop{#2}\limits^{#1}}}
```

(*End of definition for* `\stackrel`.)

`\frac` A disguise for plain TEX's `\over`.

```
\DeclareRobustCommand\frac[2]{{\begingroup#1\endgroup\over#2}}
```

(*End of definition for* `\frac`.)

`\sqrt` `\@sqrt` Add an optional argument to plain's `\sqrt` to give the nth root of an expression $\sqrt[n]{e}$.

```
\DeclareRobustCommand\sqrt{\@ifnextchar[\@sqrt\sqrtsign}
\def\@sqrt[#1]{\root #1\of}
```

(*End of definition for* `\sqrt` *and* `\@sqrt`.)

`eqnarray` `\@eqcnt` `\@eqpen` `\if@eqnsw` `\@eqnsel` Here's the eqnarray environment: Default is for left-hand side of equations to be flushright. To make them flushleft, `\let\@eqnsel = \hfil`.

```
\newcount\@eqcnt
\newcount\@eqpen
\newif\if@eqnsw\@eqnswtrue
\newskip\@centering
\@centering = 0pt plus 1000pt
```

To get a proper `\@currentlabel` we have to redefine it for the whole display. Note that we can't use `\refstepcounter` as this results in `\@currentlabel` getting restored at the wrong and thus always writing the first label to the `.aux` file.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease⟩                    {\eqnarray}{Support for tagging}%
```

```
\def\eqnarray{%
   \stepcounter{equation}%
   \def\@currentlabel{\p@equation\theequation}%
   \def\@currentcounter{equation}%
   \global\@eqnswtrue
   \m@th
   \global\@eqcnt\z@
   \tabskip\@centering
   \let\\\@eqncr
   \dollardollar@begin\everycr{}\halign to\displaywidth\bgroup
       \hskip\@centering$\displaystyle\tabskip\z@skip{##}$\@eqnsel
      &\global\@eqcnt\@ne\hskip \tw@\arraycolsep \hfil${##}$\hfil
      &\global\@eqcnt\tw@ \hskip \tw@\arraycolsep
         $\displaystyle{##}$\hfil\tabskip\@centering
      &\global\@eqcnt\thr@@ \hb@xt@\z@\bgroup\hss##\egroup
         \tabskip\z@skip
      \cr
}

\def\endeqnarray{%
      \@@eqncr
      \egroup
      \global\advance\c@equation\m@ne
   \dollardollar@end\@ignoretrue
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\eqnarray}{Support for tagging}%
⟨latexrelease⟩
⟨latexrelease⟩\def\eqnarray{%
⟨latexrelease⟩   \stepcounter{equation}%
⟨latexrelease⟩   \def\@currentlabel{\p@equation\theequation}%
⟨latexrelease⟩   \def\@currentcounter{equation}%
⟨latexrelease⟩   \global\@eqnswtrue
⟨latexrelease⟩   \m@th
⟨latexrelease⟩   \global\@eqcnt\z@
⟨latexrelease⟩   \tabskip\@centering
⟨latexrelease⟩   \let\\\@eqncr
⟨latexrelease⟩   $$\everycr{}\halign to\displaywidth\bgroup
⟨latexrelease⟩       \hskip\@centering$\displaystyle\tabskip\z@skip{##}$\@eqnsel
⟨latexrelease⟩      &\global\@eqcnt\@ne\hskip \tw@\arraycolsep \hfil${##}$\hfil
⟨latexrelease⟩      &\global\@eqcnt\tw@ \hskip \tw@\arraycolsep
⟨latexrelease⟩         $\displaystyle{##}$\hfil\tabskip\@centering
⟨latexrelease⟩      &\global\@eqcnt\thr@@ \hb@xt@\z@\bgroup\hss##\egroup
⟨latexrelease⟩         \tabskip\z@skip
⟨latexrelease⟩      \cr
⟨latexrelease⟩}
⟨latexrelease⟩\def\endeqnarray{%
⟨latexrelease⟩      \@@eqncr
⟨latexrelease⟩      \egroup
⟨latexrelease⟩      \global\advance\c@equation\m@ne
⟨latexrelease⟩   $$\@ignoretrue
⟨latexrelease⟩}
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
\let\@eqnsel=\relax
```

(*End of definition for* `\@eqcnt` *and others.*)

`\nonumber` Switches off equation numbering.

```
\def\nonumber{\global\@eqnswfalse}
```

(*End of definition for* `\nonumber`.)

`\@eqncr`
`\@xeqncr`
`\@yeqncr`

```
\protected\def\@eqncr{%
   {\ifnum0=`}\fi
   \@ifstar{%
      \global\@eqpen\@M\@yeqncr
   }{%
      \global\@eqpen\interdisplaylinepenalty \@yeqncr
   }%
}
\def\@yeqncr{\@testopt\@xeqncr\z@skip}

⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                    {\@xeqncr}{eqnarray support calc syntax}%
\def\@xeqncr[#1]{%
   \ifnum0=`{\fi}%
   \@@eqncr
   \noalign{\penalty\@eqpen\vskip\jot\@vspace@calcify{#1}}%
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\@xeqncr}{eqnarray support calc syntax}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@xeqncr[#1]{%
⟨latexrelease⟩    \ifnum0=`{\fi}%
⟨latexrelease⟩    \@@eqncr
⟨latexrelease⟩    \noalign{\penalty\@eqpen\vskip\jot\vskip #1\relax}%
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@eqncr`, `\@xeqncr`, *and* `\@yeqncr`.)

`\@@eqncr`

```
\def\@@eqncr{\let\reserved@a\relax
    \ifcase\@eqcnt \def\reserved@a{& & &}\or \def\reserved@a{& &}%
     \or \def\reserved@a{&}\else
       \let\reserved@a\@empty
       \@latex@error{Too many columns in eqnarray environment}\@ehc\fi
     \reserved@a \if@eqnsw\@eqnnum\stepcounter{equation}\fi
     \global\@eqnswtrue\global\@eqcnt\z@\cr}
```

(*End of definition for* `\@@eqncr`.)

eqnarray\@seqncr (*continued*) Here's the eqnarray* environment:

```
\let\@seqncr=\@eqncr
\@namedef{eqnarray*}{\protected\def\@eqncr{\nonumber\@seqncr}\eqnarray}
\@namedef{endeqnarray*}{\nonumber\endeqnarray}
```

(*End of definition for* `\@seqncr`.)

`\lefteqn` `\lefteqn{FORMULA}` typesets `FORMULA` in display math style flushleft in a box of width zero.

```
\def\lefteqn#1{\rlap{$\displaystyle #1$}}
```

(*End of definition for* `\lefteqn`.)

`\ensuremath` In math mode, `\ensuremath{text}` is equivalent to text; in LR or paragraph mode, it is equivalent to `$text$`. `\relax` is not needed in front of the `\ifmmode` as `\protect` will be `\let` to `\relax`. This version (due to Donald Arseneau) avoids duplicating its argument in the 'then' and 'else' part of the `\ifmath` which is necessary in nested 'tabular' like environments. See amslatex/2104.

```
\DeclareRobustCommand{\ensuremath}{%
  \ifmmode
    \expandafter\@firstofone
  \else
    \expandafter\@ensuredmath
  \fi}
```

(*End of definition for* `\ensuremath`.)

`\@ensuredmath` The `\relax` stops `\ensuremath{}` starting display math.

```
\long\def\@ensuredmath#1{$\relax#1$}
```

(*End of definition for* `\@ensuredmath`.)

LuaTeX contains new math primitives to place expression over or under horizontally extensible glyphs. Before LuaTeX 1.14 these did not work correctly with the `\mathstyle` primitive and sometimes did not use cramped style in consistent ways. For newer version, we opt into the corrected behavior.

```
\ifx\mathdefaultsmode\@undefined\else
  \mathdefaultsmode=1
\fi
```

`\eqno` `\leqno` Ensure the (deprecated) `$$..\eqno 1 $$` ignores spaces.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2023/06/01}%
⟨latexrelease⟩                    {\eqno}{add ignorespaces to eqno}%
\expandafter\let\expandafter\@kernel@eqno\csname tex_eqno:D\endcsname
\expandafter\let\expandafter\@kernel@leqno\csname tex_leqno:D\endcsname
\protected\def\eqno{\@kernel@eqno\aftergroup\ignorespaces}
\protected\def\leqno{\@kernel@leqno\aftergroup\ignorespaces}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
```

```
⟨latexrelease⟩                  {\eqno}{add ignorespaces to eqno}%
⟨latexrelease⟩\let\eqno\@kernel@eqno
⟨latexrelease⟩\let\leqno\@kernel@leqno
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\eqno` *and* `\leqno`.)

1.3 External options to the standard document classes

1.3.1 Left equation numbering

`\@eqnnum` To put the equation number on the left side of an equation we have to use a little trick. The number is shifted `\displaywidth` to the left inside a box of (approximately) zero width. This fails when the equation is too wide, the equation number than may overprint the equation itself.

```
⟨*leqno⟩
\renewcommand\@eqnnum{\hb@xt@.01\p@{}%
                      \rlap{\normalfont\normalcolor
                        \hskip -\displaywidth(\theequation)}}
⟨/leqno⟩
```

(*End of definition for* `\@eqnnum`.)

1.3.2 Flush left equations

To get the displayed math environments to print the contents flush left (with an indentation) we have to redefine all of LaTeX 2ε's displayed math environments.

`\mathindent` The amount of indentation of the equations is stored in a register.

```
⟨*fleqn⟩
\newskip\mathindent
```

The setting of `\mathindent` has to be deferred until the class file has been processed, because `\leftmargini` is still 0pt wide at the moment `fleqn.clo` is read in.

```
\AtEndOfClass{\mathindent\leftmargini}
```

(*End of definition for* `\mathindent`.)

`\[` Begin display math;

```
\IncludeInRelease{2015/01/01}{\[}{Make \[ robust}%
\DeclareRobustCommand\[{\relax
                  \ifmmode\@badmath
                  \else
                    \begin{trivlist}%
                      \@beginparpenalty\predisplaypenalty
                      \@endparpenalty\postdisplaypenalty
                      \item[]\leavevmode
                      \hb@xt@\linewidth\bgroup $\m@th\displaystyle %$
                        \hskip\mathindent\bgroup
                  \fi}
\EndIncludeInRelease
```

```
\IncludeInRelease{0000/00/00}{\[}{Make \[ robust}%
\renewcommand\[{\relax
                \ifmmode\@badmath
                \else
                  \begin{trivlist}%
                    \@beginparpenalty\predisplaypenalty
                    \@endparpenalty\postdisplaypenalty
                    \item[]\leavevmode
                    \hb@xt@\linewidth\bgroup $\m@th\displaystyle %$
                      \hskip\mathindent\bgroup
                \fi}
\EndIncludeInRelease
```

(*End of definition for* `\[`.)

`\]` end display math;

```
\IncludeInRelease{2015/01/01}{\]}{Make \] robust}%
\DeclareRobustCommand\]{\relax
                \ifmmode
                      \egroup $\hfil% $
                    \egroup
                  \end{trivlist}%
                \else \@badmath
                \fi}
\EndIncludeInRelease
```

```
\IncludeInRelease{0000/00/00}{\]}{Make \] robust}%
\renewcommand\]{\relax
                \ifmmode
                      \egroup $\hfil% $
                    \egroup
                  \end{trivlist}%
                \else \@badmath
                \fi}
\EndIncludeInRelease
```

(*End of definition for* `\]`.)

`equation` (*env.*) The equation environment

```
\renewenvironment{equation}%
    {\@beginparpenalty\predisplaypenalty
     \@endparpenalty\postdisplaypenalty
     \refstepcounter{equation}%
     \trivlist \item[]\leavevmode
       \hb@xt@\linewidth\bgroup $\m@th% $
         \displaystyle
         \hskip\mathindent}%
```

Ensure that there is at least a space between formula and equation number so that they don't bump in each other.

```
        {$\hskip .3em minus.3em\hfil % $
         \displaywidth\linewidth\hbox{\@eqnnum}%
       \egroup
     \endtrivlist}
```

`eqnarray` (*env.*) The eqnarray environment

```
\renewenvironment{eqnarray}{%
    \stepcounter{equation}%
    \def\@currentlabel{\p@equation\theequation}%
    \def\@currentcounter{equation}%
    \global\@eqnswtrue\m@th
    \global\@eqcnt\z@
    \tabskip\mathindent
    \let\\=\@eqncr
    \setlength\abovedisplayskip{\topsep}%
    \ifvmode
      \addtolength\abovedisplayskip{\partopsep}%
    \fi
```

When the documentclass uses a non-zero `\parskip` setting the `\topsep` might have a negative value to compensate for that. Therefore we add `\parskip` to `\abovedisplayskip`.

```
    \addtolength\abovedisplayskip{\parskip}%
    \setlength\belowdisplayskip{\abovedisplayskip}%
    \setlength\belowdisplayshortskip{\abovedisplayskip}%
    \setlength\abovedisplayshortskip{\abovedisplayskip}%
    \dollardollar@begin\everycr{}\halign to\linewidth%
    \bgroup
      \hskip\@centering
      $\displaystyle\tabskip\z@skip{##}$\@eqnsel&%
      \global\@eqcnt\@ne \hskip \tw@\arraycolsep \hfil${##}$\hfil&%
      \global\@eqcnt\tw@ \hskip \tw@\arraycolsep
        $\displaystyle{##}$\hfil \tabskip\@centering&%
      \global\@eqcnt\thr@@
        \hb@xt@\z@\bgroup\hss##\egroup\tabskip\z@skip\cr}%
      {\@@eqncr
    \egroup
    \global\advance\c@equation\m@ne\dollardollar@end
    \@ignoretrue
    }
```

⟨/fleqn⟩

File 39
ltlists.dtx

1 List, and related environments

The generic commands for creating an indented environment – `enumerate`, `itemize`, `quote`, etc – are:

`\list{`⟨*LABEL*⟩`}{`⟨*COMMANDS*⟩`}` ... `\endlist`

which can be invoked by the user as the list environment. The LABEL argument specifies item labeling. COMMANDS contains commands for changing the horizontal and vertical spacing parameters.

Each item of the environment is begun by the command `\item[`ITEMLABEL`]` which produces an item labeled by ITEMLABEL. If the argument is missing, then the LABEL argument of the `\list` command is used as the item label.

The label is formed by putting `\makelabel{`⟨*ITEMLABEL*⟩`}` in an hbox whose width is either its natural width or else `\labelwidth`, whichever is larger. The `\list` command defines `\makelabel` to have the default definition:

`\makelabel{`⟨*ARG*⟩`}` == BEGIN `\hfil` ARG END

which, for a label of width less than `\labelwidth`, puts the label flushright, `\labelsep` to the left of the item's text. However, `\makelabel` can be `\let` to another command by the `\list`'s COMMANDS argument.

A `\usecounter{`⟨*foo*⟩`}` command in the second argument causes the counter *foo* to be initialized to zero, and stepped by every `\item` command without an argument. (`\label` commands within the list refer to this counter.)

When you leave a list environment, returning either to an enclosing list or normal text mode, LaTeX begins a new paragraph if and only if you leave a blank line after the `\end` command. This is accomplished by the `\@endparenv` command.

Blank lines are ignored every other reasonable place–i.e.:

- Between the `\begin{list}` and the first `\item`,
- Between the `\item` and the text of that item.
- Between the end of the last item and the `\end{list}`.

For an environment like quotation, in which items are not labeled, the entire environment is a single item. It is defined by letting `\quotation` == `\list{}{...}\item\relax`. (Note the `\relax`, there in case the first character in the environment is a '['.) The spacing parameters provide a great deal of flexibility in designing the format, including the ability to let the indentation of the first paragraph be different from that of the subsequent ones.

The trivlist environment is equivalent to a list environment whose second argument sets the following parameter values:

`\leftmargin` **= 0:** causes no indentation of left margin

`\labelwidth` **= 0:** see below for precise effect this has.

`\itemindent = 0pt` with a null label, makes first paragraph have no indentation. Succeeding paragraphs have `\parindent` indentation. To give first paragraph same indentation, set `\itemindent = \parindent` before the `\item[]`.

Every `\item` in a trivlist environment must have an argument—in many cases, this will be the null argument (`\item[]`). The trivlist environment is mainly used for paragraphing environments, like verbatim, in which there is no margin change. It provides the same vertical spacing as the list environment, and works reasonably well when it occurs immediately after an `\item` command in an enclosing list.

1.1 List and Trivlist

The following variables are used inside a list environment:

`\@totalleftmargin` The distance that the prevailing left margin is indented from the outermost left margin,

`\linewidth` The width of the current line. Must be initialized to `\hsize`.

`\@listdepth` A count for holding current list nesting depth.

`\makelabel` A macro with a single argument, used to generate the label from the argument (given or implied) of the `\item` command. Initialized to `\@mklab` by the `\list` command. This command must produce some stretch—i.e., an `\hfil`.

`@inlabel` A switch that is false except between the time an `\item` is encountered and the time that TEX actually enters horizontal mode. Should be tested by commands that can be messed up by the list environment's use of `\everypar`.

`\box\@labels` When `@inlabel = true`, it holds the labels to be put out by `\everypar`.

`@noparitem` A switch set by `\list` when `@inlabel = true`. Handles the case of a `\list` being the first thing in an item.

`@noparlist` A switch set true for a list that begins an item. No `\topsep` space is added before or after `\item`'s such a list.

`@newlist` Set true by `\list`, set false by the first text (by `\everypar`).

`@noitemarg` Set true when executing an `\item` with no explicit argument. Used to save space. To save time, make two separate `\@item` commands.

`@nmbrlist` Set true by `\usecounter` command, causes list to be numbered.

`\@listctr` `\def`'ed by `\usecounter` to name of counter.

`@noskipsec` A switch set true by a sectioning command when it is creating an in-text heading with `\everypar`.

Throughout a list environment, `\hsize` is the width of the current line, measured from the outermost left margin to the outermost right margin. Environments like tabbing should use `\linewidth` instead of `\hsize`.

Here are the parameters of a list that can be set by commands in the `\list`'s COMMANDS argument. These parameters are all TEX skips or dimensions (defined by `\newskip` or `\newdimen`), so the usual TEX or LATEX commands can be used to set them. The commands will be executed in vmode if and only if the `\list` was preceded by a `\par` (or something like an `\end{list}`), so the spacing parameters can be set according to whether the list is inside a paragraph or is its own paragraph.

1.2 Vertical Spacing (skips)

\topsep: Space between first item and preceding paragraph.

\partopsep: Extra space added to **\topsep** when environment starts a new paragraph (is called in vmode).

\itemsep: Space between successive items.

\parsep: Space between paragraphs within an item – the **\parskip** for this environment.

1.3 Penalties

\@beginparpenalty: put at the beginning of a list

\@endparpenalty: put at end of list

\@itempenalty: put between items.

1.4 Horizontal Spacing (dimens)

\leftmargin: space between left margin of enclosing environment (or of page if top level list) and left margin of this list. Must be nonnegative.

\rightmargin: analogous.

\listparindent: extra indentation at beginning of every paragraph of a list except the one started by the **\item** command. May be negative! Usually, labeled lists have **\listparindent** equal to zero.

\itemindent: extra indentation added right BEFORE an item label.

\labelwidth: nominal width of box that contains the label. If the natural width of the label $\leq$ **\labelwidth**, then the label is flushed right inside a box of width **\labelwidth** (with an **\hfil**). Otherwise, a box of the natural width is employed, which causes an indentation of the text on that line.

\labelsep: space between end of label box and text of first item.

1.5 Default Values

Defaults for the list environment are set as follows. First, **\rightmargin**, **\listparindent** and **\itemindent** are set to 0pt. Then, one of the commands **\@listi**, **\@listii**, ... , **\@listvi** is called, depending upon the current level of the list. The **\@list** ... commands should be defined by the document style. A convention that the document style should follow is to set **\leftmargin** to **\leftmargini**,..., **\leftmarginvi** for the appropriate level. Items that aren't changed may be left alone, but everything that could possibly be changed must be reset. *Historical LaTeX 2.09 comments (not necessarily accurate any more):*

```
\list{LABEL}{COMMANDS} ==
 BEGIN
   if \@listdepth > 5
     then LaTeX error: 'Too deeply nested'
     else \@listdepth :=G \@listdepth + 1
```

```
    fi
    \rightmargin      := 0pt
    \listparindent    := 0pt
    \itemindent       := 0pt
    \eval(@list \romannumeral\the\@listdepth)  %% Set default values:
    \@itemlabel       :=L LABEL
    \makelabel        == \@mklab
    @nmbrlist         :=L false
    COMMANDS

    \@trivlist                % commands common to \list and \trivlist

    \parskip          :=L \parsep
    \parindent        :=L \listparindent
    \linewidth        :=L \linewidth - \rightmargin -\leftmargin
    \@totalleftmargin :=L \@totalleftmargin + \leftmargin
    \parshape 1 \@totalleftmargin \linewidth
    \ignorespaces                     % gobble space up to \item
   END

\endlist == BEGIN \@listdepth :=G \@listdepth -1
                \endtrivlist
            END

\@trivlist ==
 BEGIN
    if @newlist = T then \@noitemerr fi
                    %% This command removed for some forgotten reason.
    \@topsepadd :=L \topsep
    if @noskipsec then leave vertical mode fi  %% Added 11 Jun 85
    if vertical mode
      then \@topsepadd :=L \@topsepadd + \partopsep
      else \unskip \par             % remove glue from end of last line
    fi
    if @inlabel = true
       then @noparitem :=L true
            @noparlist :=L true
       else @noparlist :=L false
            \@topsep   :=L \@topsepadd
    fi
    \@topsep          :=L \@topsep + \parskip  %% Change 4 Sep 85
    \leftskip         :=L 0pt          % Restore paragraphing parameters
    \rightskip        :=L \@rightskip
    \parfillskip      :=L 0pt + 1fil

  NOTE: \@setpar called on every \list in case \par has been
  temporarily  munged before the \list command.
    \@setpar{if @newlist = false then {\@@par} fi}
    \@newlist         :=G T
    \@outerparskip    :=L \parskip
```

```
END

\trivlist  ==
BEGIN
 \parsep     := \parskip
 @nmbrlist := F
 \@trivlist
 \labelwidth := 0
 \leftmargin := 0
 \itemindent := \parindent
 \@itemlabel :=L "empty"           %% added 93/12/13
 \makelabel{LABEL} == LABEL
END

\endtrivlist ==
  BEGIN
    if @inlabel = T then \indent fi
    if horizontal mode then \unskip \par fi
    if @noparlist = true
      else if \lastskip > 0
               then \@tempskipa := \lastskip
                    \vskip - \lastskip
                    \vskip \@tempskipa -\@outerparskip + \parskip
           fi
           \@endparenv
    fi
  END

\@endparenv ==
  BEGIN
   \addpenalty{@endparpenalty}
   \addvspace{\@topsepadd}
   \endgroup    %% ends the \begin command's \begingroup
   \par  ==  BEGIN
               \@restorepar
               \everypar{}
               \par
             END
   \everypar == BEGIN remove \lastbox \everypar{} END
   \begingroup  %% to match the \end commands \endgroup
  END

\item == BEGIN if math mode then WARNING fi
                 if  next char = [
                 then  \@item
                 else  @noitemarg := true
                       \@item[@itemlabel]
         END

\@item[LAB] ==
```

```
BEGIN
 if @noparitem = true
   then @noparitem := false
            % NOTE: then clause  hardly every taken,
            %  so made a macro \@donoparitem
        \box\@labels :=G \hbox{\hskip -\leftmargin
                              \box\@labels
                              \hskip \leftmargin }
        if @minipage = false then
           \@tempskipa := \lastskip
           \vskip -\lastskip
           \vskip \@tempskipa + \@outerparskip - \parskip
        fi
   else if @inlabel = true
          then \indent \par    % previous item empty.
        fi
        if hmode then 2 \unskip's
                       % To remove any space at end of prev.
                       % paragraph that could cause a blank line.
                 \par
        fi
        if @newlist = T
           then if @nobreak = T    % Kludge if list follows \section
                   then \addvspace{\@outerparskip - \parskip}
                   else \addpenalty{\@beginparpenalty}
                        \addvspace{\@topsep}
                        \addvspace{-\parskip}    %% added 4 Sep 85
                fi
           else \addpenalty{\@itempenalty}
                \addvspace{\itemsep}
        fi
        @inlabel :=G true
 fi

 \everypar{ @minipage :=G F
            @newlist :=G F
            if  @inlabel = true
              then @inlabel :=G false
                   \hskip -\parindent
                   \box\@labels
                   \penalty 0
                         %% 3 Oct 85  – allow line break here
                   \box\@labels :=G null
            fi
            \everypar{} }
 @nobreak :=G false
 if  @noitemarg = true
   then @noitemarg := false
        if @nmbrlist
          then \refstepcounter{\@listctr}
```

```
  fi      fi
  \@tempboxa   :=L \hbox{\makelabel{LAB}}
  \box\@labels :=G \@labels \hskip \itemindent
                   \hskip - (\labelwidth + \labelsep)
                   if \wd \@tempboxa > \labelwidth
                      then \box\@tempboxa
                      else \hbox to \labelwidth {\unhbox\@tempboxa}
                   fi
                   \hskip\labelsep
  \ignorespaces                          %gobble space up to text
END

\makelabel{LABEL} == ERROR    %% default to catch lonely \item


\usecounter{CTR} == BEGIN  @nmbrlist :=L true
                           \@listctr == CTR
                           \setcounter{CTR}{0}
                    END

 DEFINE \dimen's and \count
End of historical LaTeX 2.09 comments.
```

\topsep
\partopsep
\itemsep
\parsep
\@topsep
\@topsepadd
\@outerparskip

```
1  ⟨*2ekernel⟩
2  \newskip\topsep
3  \newskip\partopsep
4  \newskip\itemsep
5  \newskip\parsep
6  \newskip\@topsep
7  \newskip\@topsepadd
8  \newskip\@outerparskip
```

(*End of definition for* `\topsep` *and others.*)

\leftmargin
\rightmargin
\listparindent
\itemindent
\labelwidth
\labelsep
\linewidth
\@totalleftmargin

```
9  \newdimen\leftmargin
10 \newdimen\rightmargin
11 \newdimen\listparindent
12 \newdimen\itemindent
13 \newdimen\labelwidth
14 \newdimen\labelsep
15 \newdimen\linewidth
16 \newdimen\@totalleftmargin \@totalleftmargin=\z@
```

(*End of definition for* `\leftmargin` *and others.*)

```
\leftmargini
\leftmarginii
\leftmarginiii
\leftmarginiv
\leftmarginv
\leftmarginvi
```

```
\newdimen\leftmargini
\newdimen\leftmarginii
\newdimen\leftmarginiii
\newdimen\leftmarginiv
\newdimen\leftmarginv
\newdimen\leftmarginvi
```

(*End of definition for* `\leftmargini` *and others.*)

```
\@listdepth
\@itempenalty
\@beginparpenalty
\@endparpenalty
```

```
\newcount\@listdepth \@listdepth=0
\newcount\@itempenalty
\newcount\@beginparpenalty
\newcount\@endparpenalty
```

(*End of definition for* `\@listdepth` *and others.*)

```
\@labels
```

```
\newbox\@labels
```

(*End of definition for* `\@labels`.)

```
\if@inlabel
\@inlabelfalse
\@inlabeltrue
```

```
\newif\if@inlabel \@inlabelfalse
```

(*End of definition for* `\if@inlabel`, `\@inlabelfalse`, *and* `\@inlabeltrue`.)

```
\if@newlist
\@newlistfalse
\@newlisttrue
```

```
\newif\if@newlist   \@newlistfalse
```

(*End of definition for* `\if@newlist`, `\@newlistfalse`, *and* `\@newlisttrue`.)

```
\if@noparitem
\@noparitemfalse
\@noparitemtrue
```

```
\newif\if@noparitem \@noparitemfalse
```

(*End of definition for* `\if@noparitem`, `\@noparitemfalse`, *and* `\@noparitemtrue`.)

```
\if@noparlist
\@noparlistfalse
\@noparlisttrue
```

```
\newif\if@noparlist \@noparlistfalse
```

(*End of definition for* `\if@noparlist`, `\@noparlistfalse`, *and* `\@noparlisttrue`.)

```
\if@noitemarg
\@noitemargfalse
\@noitemargtrue
```

```
\newif\if@noitemarg \@noitemargfalse
```

(*End of definition for* `\if@noitemarg`, `\@noitemargfalse`, *and* `\@noitemargtrue`.)

```
\if@newlist
\@newlistfalse
\@newlisttrue
```

```
\newif\if@nmbrlist  \@nmbrlistfalse
```

(*End of definition for* `\if@newlist`, `\@newlistfalse`, *and* `\@newlisttrue`.)

list\list

```
\def\list#1#2{%
  \ifnum \@listdepth >5\relax
    \@toodeep
  \else
    \global\advance\@listdepth\@ne
  \fi
  \rightmargin\z@
  \listparindent\z@
  \itemindent\z@
  \csname @list\romannumeral\the\@listdepth\endcsname
  \def\@itemlabel{#1}%
  \let\makelabel\@mklab
  \@nmbrlistfalse
  #2\relax
  \@trivlist
  \parskip\parsep
  \parindent\listparindent
  \advance\linewidth -\rightmargin
  \advance\linewidth -\leftmargin
  \advance\@totalleftmargin \leftmargin
  \parshape \@ne \@totalleftmargin \linewidth
  \ignorespaces}
```

(*End of definition for* `\list`.)

`\par@deathcycles`

```
\newcount\par@deathcycles
```

(*End of definition for* `\par@deathcycles`.)

`\@trivlist` Because `\par` is sometimes made a no-op it is possible for a missing `\item` to produce a loop that does not fill memory and so never gets trapped by TEX. We thus need to trap this here by setting `\par` to count the number of times a paragraph ii is called with no progress being made started.

```
\def\@trivlist{%
  \if@noskipsec \leavevmode \fi
  \@topsepadd \topsep
  \ifvmode
    \advance\@topsepadd \partopsep
  \else
    \unskip \par
  \fi
  \if@inlabel
    \@noparitemtrue
    \@noparlisttrue
  \else
    \if@newlist \@noitemerr \fi
    \@noparlistfalse
    \@topsep \@topsepadd
  \fi
  \advance\@topsep \parskip
  \leftskip \z@skip
  \rightskip \@rightskip
  \parfillskip \@flushglue
```

```
  \par@deathcycles \z@
  \@setpar{\if@newlist
              \advance\par@deathcycles \@ne
              \ifnum \par@deathcycles >\@m
                \@noitemerr
                {\@@par}%
              \fi
            \else
              {\@@par}%
            \fi}%
  \global \@newlisttrue
  \@outerparskip \parskip}
```

(*End of definition for* `\@trivlist`.)

`trivlist` (*End of definition for* `\trivlist`.)

```
\def\trivlist{%
  \parsep\parskip
  \@nmbrlistfalse
  \@trivlist
  \labelwidth\z@
  \leftmargin\z@
  \itemindent\z@
```

We initialize `\@itemlabel` so that a `trivlist` with an `\item` not having an optional argument doesn't produce an error message.

```
  \let\@itemlabel\@empty
  \def\makelabel##1{##1}}
```

(*End of definition for* `\trivlist`.)

`\endlist`

```
\def\endlist{%
  \global\advance\@listdepth\m@ne
  \endtrivlist}
```

(*End of definition for* `\endlist`.)

The definition of `\trivlist` used to be in ltspace.dtx so that other commands could be 'let to it'. They now use `\def`.

`\endtrivlist`

```
\def\endtrivlist{%
  \if@inlabel
    \leavevmode
    \global \@inlabelfalse
  \fi
  \if@newlist
    \@noitemerr
    \global \@newlistfalse
  \fi
  \ifhmode\unskip \par
```

We also check if we are in math mode and issue an error message if so (hoping that `\@currenvir` resolves suitably). Otherwise the usual "perhaps a missing item" error will get triggered later which is confusing.

```
  \else
```

```
    \@inmatherr{\end{\@currenvir}}%
  \fi
  \if@noparlist \else
    \ifdim\lastskip >\z@
      \@tempskipa\lastskip \vskip -\lastskip
      \advance\@tempskipa\parskip \advance\@tempskipa -\@outerparskip
      \vskip\@tempskipa
    \fi
    \@endparenv
  \fi
}
```

(*End of definition for* `\endtrivlist`.)

`\@endparenv` `\@doendpe`

To suppress the paragraph indentation in text immediately following a paragraph-making environment, `\everypar` is changed to remove the space, and `\par` is redefined to restore `\everypar`. Instead of redefining `\par` and `\everypar`, `\@endparenv` was changed to set the `@endpe` switch, letting `\end` redefine `\par` and `\everypar`.

This allows paragraph-making environments to work right when called by other environments. (Changed 27 Oct 86)

In 2024 this logic was partially replaced with a new algorithm:

- `\if@endpe` is now set globally to `true` or `false`.
- In addition `\@endpetrue` initiates an `\aftergroup` call to `\propagate@doendpe` if it is used inside a group.
- `\propagate@doendpe` in turn checks the status of `\if@endpe` and if that is `true` it calls `\@doendpe` otherwise it does nothing.
- `\@doendpe` in turn calls `\@endpetrue` and also makes the necessary changes to `\par` and `\everypar` so that they handle as before any empty line that follows the environment.
- Because of the `\@endpetrue` we get another `\aftergroup`, so the mechanisnm slowly migrates out of several groups if those follow immediately after the end of the environment. If, however, there is a new paragraph started or an explicit `\par` before the next group ends then this will result in a call to `\@endpefalse` and the migration stops (note that `\propagate@doendpe` is still called once after the group but does nothing).
- Using this approach something like

```
{%  some customization here
 \begin{equation}
   x=y
 \end{equation}}
some text
```

  is still correctly identified as a paragraph continuation so that there is no indentation before `some text`.

- We can get away with using global settings of `\if@endpe` even in nested situations (without keeping track of the status in a stack), because the switch change is made only at the very end of such environments, basically directly before the `\endgroup` in `\end`, and it is later set back to `false` by the next `\everypar` or the next `\par`. Even if the environment is called without using `\begin` `\end`, the situation doesn't change (or rather cause a problem).
- However, there is one scenario where the new approach would change the behavior. If a box is being built, e.g., with `\setbox`, we have now the case that a `\@endpetrue` inside would migrate out into a context in which it should not be true (because the box might get used elsewhere, e.g., a float). In the past, due to local switch changes, that didn't happen, i.e., `\if@endpe` would revert to `false` at the end of the box definition.
- Thus, to avoid that one has to explicitly set it back to false at the end of such constructions, just as we also need to prevent colors from migrating out. Thus the correct place to do this is in `\color@endgroup` because that is always called at that point.

```
\def\@endparenv{%
  \addpenalty\@endparpenalty\addvspace\@topsepadd\@endpetrue}
⟨latexrelease⟩\IncludeInRelease{2015/01/01}{\@doendpe}{clubpenalty fix}%
\def\@doendpe{\@endpetrue
    \def\par{\@restorepar
```

If a section heading changes `\clubpenalty` to keep lines after it together then this modification is restored via the `\everypar` mechanism at the start of the next paragraph. As we destroy the contents of this token here we explicitly set `\clubpenalty` back to its default.

```
              \clubpenalty\@clubpenalty
              \@endpefalse
              \everypar{}\par}%
```

Use `\setbox0=\lastbox` instead of `\hskip -\parindent` so that a `\noindent` becomes a no-op when used before a line immediately following a list environment(23 Oct 86).

```
    \everypar
        {{\setbox\z@\lastbox}%
         \everypar{}\@endpefalse}}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\@doendpe}{clubpenalty fix}%
⟨latexrelease⟩\def\@doendpe{\@endpetrue
⟨latexrelease⟩    \def\par{\@restorepar\everypar{}\par\@endpefalse}\everypar
⟨latexrelease⟩                {{\setbox\z@\lastbox}\everypar{}\@endpefalse}}
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\@endparenv` *and* `\@doendpe`.)

`\if@endpe`
`\@endpefalse`
`\@endpetrue`
`\propagate@doendpe`

As outlined above these are no longer simple switches, but we keep the name because it is used all over the place.

```
\newif\if@endpe
```

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2024/11/01}%
⟨latexrelease⟩                    {\@endpetrue}{New @endpe handling}%
\def\@endpefalse{\global\let\if@endpe\iffalse}
\def\@endpetrue {%
  \global\let\if@endpe\iftrue
```

If we are inside a simple or a semi-simple group then propagate to the outside, for all other group types do nothing. Normally, we would start out in the group opened by `\begin` (type 14). When we migrate out of that we are either on top-level (type 0) or in another semi-simple group (type 14) or in some other group. Thus, the best order of tests is to first test for 14, then for 0 and finally for 1 (simple group).

```
  \ifnum\currentgrouptype =14        % semi-simple group
    \aftergroup\propagate@doendpe
  \else
    \ifnum\currentgrouptype =\z@     % no group: top-level
    \else
      \ifnum\currentgrouptype =\@ne  % simple group
        \aftergroup\propagate@doendpe
     \fi
    \fi
  \fi
}
```

If `\if@endpe` is still true after the group ends, we run `\@doendpe` that in turn runs another `\@endpetrue` (besides other things), thus propagating further if necessary. However, if the endpe situation got resolved and `\if@endpe` is `false` then nothing further happens.

```
\def\propagate@doendpe{\if@endpe \@doendpe \fi}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\@endpetrue}{New @endpe handling}%
⟨latexrelease⟩
⟨latexrelease⟩
⟨latexrelease⟩\def\@endpefalse{\let\if@endpe\iffalse}
⟨latexrelease⟩\def\@endpetrue{\let\if@endpe\iftrue}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\if@endpe` *and others.*)

`\@mklab`

```
\def\@mklab#1{\hfil #1}
```

(*End of definition for* `\@mklab`.)

`\item`

```
\def\item{%
  \@inmatherr\item
  \@ifnextchar [\@item{\@noitemargtrue \@item[\@itemlabel]}}
```

(*End of definition for* `\item`.)

\@donoparitem

```
\def\@donoparitem{%
  \@noparitemfalse
  \global\setbox\@labels\hbox{\hskip -\leftmargin
                              \unhbox\@labels
                               \hskip \leftmargin}%
  \if@minipage\else
    \@tempskipa\lastskip
    \vskip -\lastskip
    \advance\@tempskipa\@outerparskip
    \advance\@tempskipa -\parskip
    \vskip\@tempskipa
  \fi}
```

(*End of definition for* `\@donoparitem`.)

\@item

```
\def\@item[#1]{%
  \if@noparitem
    \@donoparitem
  \else
    \if@inlabel
      \indent \par
    \fi
    \ifhmode
      \unskip\unskip \par
    \fi
    \if@newlist
      \if@nobreak
        \@nbitem
      \else
        \addpenalty\@beginparpenalty
        \addvspace\@topsep
        \addvspace{-\parskip}%
      \fi
    \else
      \addpenalty\@itempenalty
      \addvspace\itemsep
    \fi
    \global\@inlabeltrue
  \fi
  \everypar{%
    \@minipagefalse
    \global\@newlistfalse
```

This `\if@inlabel` check is needed in case an item starts of inside a group so that `\everypar` does not become empty outside that group.

```
    \if@inlabel
      \global\@inlabelfalse
```

The paragraph indent is now removed by using `\setbox...` since this makes `\noindent` a no-op here, as it should be. Thus the following comment is redundant but is left here for the sake of future historians: this next command was changed from an hskip to a kern to avoid a break point after the parindent box: the skip could cause a line-break if a very long label occurs in raggedright setting. If `\noindent` was used after `\item` want

to cancel the \itemindent skip. This case can be detected as the indentation box will be void.

```
      {\setbox\z@\lastbox
       \ifvoid\z@
         \kern-\itemindent
       \fi}%
      \box\@labels
      \penalty\z@
    \fi
```

This code is intended to prevent a page break after the first line of an item that comes immediately after a section title. It may be sensible to always forbid a page break after one line of an item? As with all such settings of \clubpenalty it is local so will have no effect if the item starts in a group.

Only resetting @nobreak when it is true is now essential since now it is sometimes set locally.

```
    \if@nobreak
      \@nobreakfalse
      \clubpenalty \@M
    \else
      \clubpenalty \@clubpenalty
      \everypar{}%
    \fi}%
  \if@noitemarg
    \@noitemargfalse
    \if@nmbrlist
      \refstepcounter\@listctr
    \fi
  \fi
```

We use \sbox to support color commands.

```
  \sbox\@tempboxa{\makelabel{#1}}%
  \global\setbox\@labels\hbox{%
    \unhbox\@labels
    \hskip \itemindent
    \hskip -\labelwidth
    \hskip -\labelsep
    \ifdim \wd\@tempboxa >\labelwidth
      \box\@tempboxa
    \else
      \hbox to\labelwidth {\unhbox\@tempboxa}%
    \fi
    \hskip \labelsep}%
  \ignorespaces}
```

(*End of definition for* \@item.)

\makelabel

```
\def\makelabel#1{%
  \@latex@error{Lonely \string\item--perhaps a missing
        list environment}\@ehc}
```

(*End of definition for* `\makelabel`.)

`\@nbitem`

```
\def\@nbitem{%
  \@tempskipa\@outerparskip
  \advance\@tempskipa -\parskip
  \addvspace\@tempskipa}
```

(*End of definition for* `\@nbitem`.)

`\usecounter`

```
\def\usecounter#1{\@nmbrlisttrue\def\@listctr{#1}\setcounter{#1}\z@}
```

(*End of definition for* `\usecounter`.)

1.6 Itemize and Enumerate

Enumeration is done with four counters: `enumi`, `enumii`, `enumiii` and `enumiv`, where `enum`N controls the numbering of the Nth level enumeration. The label is generated by the commands `\labelenumi` ... `\labelenumiv`, which should be defined by the document style. Note that `\p@enum`N`\theenum`N defines the output of a `\ref` command. A typical definition might be:

```
\def\theenumii{\alph{enumii}}
\def\p@enumii{\theenumi}
\def\labelenumii{(\theenumii)}
```

which will print the labels as '(a)', '(b)', ... and print a `\ref` as '3a'.

The item numbers are moved to the right of the label box, so they are always a distance of `\labelsep` from the item.

`\@enumdepth` holds the current enumeration nesting depth.

Itemization is controlled by four commands: `\labelitemi`, `\labelitemii`, `\labelitemiii`, and `\labelitemiv`. To cause the second-level list to be bulleted, you just define `\labelitemii` to be •. `\@itemspacing` and `\@itemdepth` are the analogs of `\@enumspacing` and `\@enumdepth`.

Historical LaTeX 2.09 comments (not necessarily accurate any more):

```
\enumerate ==
  BEGIN
    if \@enumdepth > 3
      then errormessage: "Too deeply nested".
      else \@enumdepth :=L \@enumdepth + 1
           \@enumctr :=L eval(enum@\romannumeral\the\@enumdepth)
           \list{\label(\@enumctr)}
                {\usecounter{\@enumctr}
                 \makelabel{LABEL} ==  \hss \llap{LABEL}}
    fi
  END

\endenumerate == \endlist
```

End of historical LaTeX 2.09 comments.

\@enumdepth

```
\newcount\@enumdepth \@enumdepth = 0
```

(*End of definition for* `\@enumdepth`.)

\c@enumi
\c@enumii
\c@enumii
\c@enumiv

```
\@definecounter{enumi}
\@definecounter{enumii}
\@definecounter{enumiii}
\@definecounter{enumiv}
```

(*End of definition for* `\c@enumi` *and others.*)

`enumerate` (*env.*)

```
\def\enumerate{%
  \ifnum \@enumdepth >\thr@@\@toodeep\else
    \advance\@enumdepth\@ne
    \edef\@enumctr{enum\romannumeral\the\@enumdepth}%

      \expandafter
      \list
        \csname label\@enumctr\endcsname
        {\usecounter\@enumctr\def\makelabel##1{\hss\llap{##1}}}%
  \fi}
\let\endenumerate =\endlist
```

Historical LaTeX 2.09 comments (not necessarily accurate any more):

```
\itemize ==
  BEGIN
    if \@itemdepth > 3
      then  errormessage: 'Too deeply nested'.
      else \@itemdepth :=L \@itemdepth + 1
           \@itemitem  == eval(labelitem\romannumeral\the\@itemdepth)
           \list{\@nameuse{\@itemitem}}
                 {\makelabel{LABEL} ==  \hss \llap{LABEL}}
    fi
  END

\enditemize ==  \endlist
```

End of historical LaTeX 2.09 comments.

\@itemdepth

```
\newcount\@itemdepth \@itemdepth = 0
```

(*End of definition for* `\@itemdepth`.)

`itemize` (*env.*)

```
\def\itemize{%
  \ifnum \@itemdepth >\thr@@\@toodeep\else
    \advance\@itemdepth\@ne
    \edef\@itemitem{labelitem\romannumeral\the\@itemdepth}%
```

```
    \expandafter
    \list
      \csname\@itemitem\endcsname
      {\def\makelabel##1{\hss\llap{##1}}}%
  \fi}
\let\enditemize =\endlist
```
⟨/2ekernel⟩

File 40
ltboxes.dtx

1 LaTeX Box commands

\makebox `\makebox[⟨wid⟩][⟨pos⟩]{⟨obj⟩}`
Puts ⟨*obj*⟩ in an `\hbox` of width ⟨*wid*⟩, positioned by ⟨*pos*⟩.

The possible ⟨*pos*⟩ are:
`s` stretched,
`l` flushleft,
`r` flushright,
`c` (default) centered.

If ⟨*wid*⟩ is missing, then ⟨*pos*⟩ is also missing and ⟨*obj*⟩ is put in an `\hbox` of its natural width.

`\makebox(⟨x⟩,⟨y⟩)[⟨pos⟩]{⟨obj⟩}`
Puts ⟨*obj*⟩ in an `\hbox` of width $x *$ `\unitlength` and height $y *$ `\unitlength`. ⟨*pos*⟩ arguments are `s`, `l`, `r` or `c` (default) for stretched, flushleft, flushright or centered, and `t` or `b` for top, bottom – or combinations like `tr` or `rb`. Default for horizontal and vertical are centered. Note that in this picture mode version of `\makebox` a [b] aligns on the *bottom* of the text as documented. If you want to align on the *baseline* use `\makebox( , )[b]{\raisebox{0pt}[\height][0pt]{xyz}}` or `\makebox( , )[b]{\smash{xyz}}`

\mbox `\mbox{⟨obj⟩}` The same as `\makebox{⟨obj⟩}`, but is more efficient as no checking for optional arguments is done.

\newsavebox `\newsavebox{\cmd}` : If `\cmd` is undefined, then defines it to be a TeX box register.

\savebox `\savebox{\cmd}` ... : `\cmd` is defined to be a TeX box register, and the '...' are any `\makebox` arguments. It is like `\makebox`, except it doesn't produce text but saves the value in `\box \cmd`.

\sbox `\sbox{⟨cmd⟩}{⟨obj⟩}` is an efficient abbreviation for `\savebox{⟨cmd⟩}{⟨obj⟩}`.

lrbox (*env.*) `\begin{lrbox}{⟨cmd⟩}⟨text⟩\end{lrbox}` is equivalent to `\sbox{⟨cmd⟩}{⟨text⟩}`
except that any white space at the beginning and end of ⟨*text*⟩ is ignored.

\framebox `\framebox` ... : like `\makebox`, except it puts a 'frame' around the box. The frame is made of lines of thickness `\fboxrule`, separated by space `\fboxsep` from the text – except for `\framebox(X,Y)` ... , where the thickness of the lines is as for the picture environment, and there is no separation added.

\fbox `\fbox{⟨obj⟩}` is an abbreviation for `\framebox{⟨obj⟩}`.

\parbox `\parbox[⟨pos⟩][⟨height⟩][⟨inner-pos⟩]{⟨width⟩}{⟨text⟩}` : Makes a box with `\hsize` ⟨*width*⟩, positioned by ⟨*pos*⟩ as follows: c : `\vcenter` (placed in `$...$` if not in math mode) b : `\vbox` t : `\vtop` default value is c. Sets `\hsize` := ⟨*width*⟩ and calls `\@parboxrestore`, which does the following: Restores the original definitions of:

```
\par
\\
\-
\'
 \`
\=
```

Resets the following parameters:

`\parindent`	=	0pt	
`\parskip`	=	0pt	added 20 Jan 87
`\linewidth`	=	`\hsize`	
`\@totalleftmargin`	=	0pt	
`\leftskip`	=	0pt	
`\rightskip`	=	0pt	
`\@rightskip`	=	0pt	
`\parfillskip`	=	0pt plus 1fil	
`\lineskip`	=	`\normallineskip`	
`\baselineskip`	=	`\normalbaselineskip`	

Calls `\sloppy`

Note: `\@arrayparboxrestore` same as `\@parboxrestore` but it doesn't restore `\\`.

`minipage` (*env.*) `minipage` : Similar to `\parbox`, except it also makes this look like a page by setting `\textwidth` == `\columnwidth` == box width

changes footnotes by redefining:

`\@mpfn` == mpfootnote

`\thempfn` == `\thempfootnote`

`\@footnotetext` == `\@mpfootnotetext`

resets the following list environment parameters:

`\@listdepth` == `\@mplistdepth`

where `\@mplistdepth` is initialized to zero,

and executes `\@minipagerestore` to allow the document style to reset any other parameters it desires. It sets `@minipage` true, and resets `\everypar` to set it false. This switch keeps `\addvspace` from putting space at the top of a minipage.

Change added 24 May 89: `\minipage` sets `@minipage` globally; `\endminipage` resets it false.

`\rule` `\rule[`⟨*raised*⟩`]{`⟨*width*⟩`}{`⟨*height*⟩`}` : Makes a ⟨*width*⟩ ∗ ⟨*height*⟩ rule, raised ⟨*raised*⟩.

`\underline` `\underline{`⟨*text*⟩`}` : Makes an underlined hbox with ⟨*text*⟩ in it.

`\raisebox` `\raisebox{`⟨*distance*⟩`}[`⟨*height*⟩`][`⟨*depth*⟩`]{`⟨*box*⟩`}` :

Raises ⟨*box*⟩ up by ⟨*distance*⟩ length (down if ⟨*distance*⟩ negative). Makes TEX think that the new box extends ⟨*height*⟩ above the line and ⟨*depth*⟩ below, for a total vertical length of ⟨*height*⟩+⟨*depth*⟩. Default values of ⟨*height*⟩ & ⟨*depth*⟩ = actual height and depth of box in new position.

```
⟨*2ekernel⟩
\message{boxes,}
```

`\makebox` `\makebox` User level command just looks for optional `[` or `(`.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩                    {\makebox}{Make \makebox robust}%
⟨*2ekernel | latexrelease⟩
\DeclareRobustCommand\makebox{%
  \leavevmode
  \@ifnextchar(%)
    \@makepicbox
    {\@ifnextchar[\@makebox\mbox}}%
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\makebox}{Make \makebox robust}%
⟨latexrelease⟩\def\makebox{%
⟨latexrelease⟩  \leavevmode
⟨latexrelease⟩  \@ifnextchar(%)
⟨latexrelease⟩    \@makepicbox
⟨latexrelease⟩    {\@ifnextchar[\@makebox\mbox}}%
⟨latexrelease⟩\expandafter\let\csname makebox \endcsname\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\makebox`.)

`\mbox` The basic horizontal box command for LaTeX.

```
\DeclareRobustCommand\mbox[1]{\leavevmode\hbox{#1}}
```

(*End of definition for* `\mbox`.)

`\@makebox` Look for a possible second optional argument (defaults to `c`).

```
\def\@makebox[#1]{%
  \@ifnextchar [{\@imakebox[#1]}{\@imakebox[#1][c]}}
```

(*End of definition for* `\@makebox`.)

`\@begin@tempboxa` Helper macro for supporting `\height`, `\width` etc. Grab `#1` into `\@tempboxa` and measure it.

```
\long\def\@begin@tempboxa#1#2{%
   \begingroup
     \setbox\@tempboxa#1{\color@begingroup#2\color@endgroup}%
     \def\width{\wd\@tempboxa}%
     \def\height{\ht\@tempboxa}%
     \def\depth{\dp\@tempboxa}%
     \let\totalheight\@ovri
     \totalheight\height
     \advance\totalheight\depth}
```

(*End of definition for* `\@begin@tempboxa`.)

`\@end@tempboxa` End the group started by `\@begin@tempboxa`, so that the scope of `\height` only includes the 'length' argument to the user-command.

```
\let\@end@tempboxa\endgroup
```

(*End of definition for* `\@end@tempboxa`.)

`\bm@c` `\bm@l` `\bm@r` `\bm@s` `\bm@t` `\bm@b` Set up spacing.

```
\def\bm@c{\hss\unhbox\@tempboxa\hss}
\def\bm@l{\unhbox\@tempboxa\hss}\let\bm@t\bm@l
\def\bm@r{\hss\unhbox\@tempboxa}\let\bm@b\bm@r
\def\bm@s{\unhbox\@tempboxa}
```

(*End of definition for* `\bm@c` *and others.*)

`\@imakebox` Internal form of `\makebox`.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2023/11/01}%
⟨latexrelease⟩                  {\@imakebox}{Unknown alignment warning}%
⟨*2ekernel | latexrelease⟩
\long\def\@imakebox[#1][#2]#3{%
  \@begin@tempboxa\hbox{#3}%
    \setlength\@tempdima{#1}%       support calc
    \hb@xt@\@tempdima{%
     \expandafter\ifx\csname bm@#2\endcsname\relax
       \bm@c
       \@latex@warning{Unexpected alignment #2}%
     \else
       \csname bm@#2\endcsname
     \fi}%
  \@end@tempboxa}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@imakebox}{Unknown alignment warning}%
⟨latexrelease⟩\long\def\@imakebox[#1][#2]#3{%
⟨latexrelease⟩  \@begin@tempboxa\hbox{#3}%
⟨latexrelease⟩    \setlength\@tempdima{#1}%       support calc
⟨latexrelease⟩    \hb@xt@\@tempdima{\csname bm@#2\endcsname}%
⟨latexrelease⟩  \@end@tempboxa}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@imakebox`.)

`\@makepicbox` Picture mode form of `\makebox`.

```
\def\@makepicbox(#1,#2){%
  \@ifnextchar[{\@imakepicbox(#1,#2)}{\@imakepicbox(#1,#2)[]}}
```

(*End of definition for* `\@makepicbox`.)

`\@imakepicbox` picture mode version

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\@imakepicbox}{default units}%
\long\def\@imakepicbox(#1,#2)[#3]#4{%
  \@defaultunitsset\@tempdimc{#2}\unitlength
  \vbox to\@tempdimc
   {\let\mb@b\vss \let\mb@l\hss\let\mb@r\hss
    \let\mb@t\vss
    \@tfor\reserved@a :=#3\do{%
      \if s\reserved@a
        \let\mb@l\relax\let\mb@r\relax
      \else
        \expandafter\let\csname mb@\reserved@a\endcsname\relax
      \fi}%
    \mb@t
    \@defaultunitsset\@tempdimc{#1}\unitlength
```

```
    \hb@xt@\@tempdimc{\mb@l #4\mb@r}%
    \mb@b
```

This kern ensures that a `b` option aligns on the bottom of the text rather than the baseline. this is the documented behaviour in the LaTeX Book. The kern is removed in compatibility mode.

```
    \kern\z@}}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@imakepicbox}{default units}%
⟨latexrelease⟩\long\def\@imakepicbox(#1,#2)[#3]#4{%
⟨latexrelease⟩  \vbox to#2\unitlength
⟨latexrelease⟩   {\let\mb@b\vss \let\mb@l\hss\let\mb@r\hss
⟨latexrelease⟩    \let\mb@t\vss
⟨latexrelease⟩    \@tfor\reserved@a :=#3\do{%
⟨latexrelease⟩      \if s\reserved@a
⟨latexrelease⟩        \let\mb@l\relax\let\mb@r\relax
⟨latexrelease⟩      \else
⟨latexrelease⟩        \expandafter\let\csname mb@\reserved@a\endcsname\relax
⟨latexrelease⟩      \fi}%
⟨latexrelease⟩    \mb@t
⟨latexrelease⟩    \hb@xt@ #1\unitlength{\mb@l #4\mb@r}%
⟨latexrelease⟩    \mb@b
⟨latexrelease⟩    \kern\z@}}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@imakepicbox`.)

`\set@color` This macro is initially a no-op, but the color package will redefine it to insert a `\special`.

```
\let\set@color\relax
```

(*End of definition for* `\set@color`.)

`\color@begingroup` `\color@setgroup` `\normalcolor` `\color@hbox` `\color@vbox` `\color@endbox`

In the past these macros were initially no-ops, and the color package redefined redefine them to be `\begingroup`, `\endgroup`, `\begingroup\set@color`, `\hbox\bgroup\color@begingroup`, `\color@endgroup\egroup`. and ⟨*set to main document color*⟩ respectively.

Nowadays we always set the group already in the kernel as this makes the coding simpler.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩                  {\color@begingroup}{color group settings}%
\let\color@begingroup\begingroup
\def\color@setgroup{\color@begingroup}        % changed further in color package
\let\normalcolor\relax                        % remains untouched; only changed in a color pa
\def\color@hbox{\hbox\bgroup\color@begingroup}
\def\color@vbox{\vbox\bgroup\color@begingroup}
\def\color@endbox{\color@endgroup\egroup}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\color@begingroup}{color group settings}%
⟨latexrelease⟩
⟨latexrelease⟩\let\color@begingroup\relax
⟨latexrelease⟩\let\color@setgroup\relax
⟨latexrelease⟩\let\normalcolor\relax
⟨latexrelease⟩\let\color@hbox\relax
⟨latexrelease⟩\let\color@vbox\relax
⟨latexrelease⟩\let\color@endbox\relax
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\color@begingroup` *and others.*)

`\color@endgroup` This macro is separated out because it received an update in 2024, so requires its own rollback.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2026/06/01}%
⟨latexrelease⟩                  {\color@endgroup}{color group settings}%
```

Beside `\endgraf` for handling vertical boxes we also reset `\if@endpe` as we are leaving the context but only if this isn't an `\hbox` we are closing.

```
\def\color@endgroup{\endgraf\ifvmode\@endpefalse\fi\endgroup}
⟨latexrelease⟩
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2024/11/01}%
⟨latexrelease⟩                  {\color@endgroup}{color group settings}%
⟨latexrelease⟩\def\color@endgroup{\endgraf\@endpefalse\endgroup}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩                  {\color@endgroup}{color group settings}%
⟨latexrelease⟩\def\color@endgroup{\endgraf\endgroup}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\color@endgroup}{color group settings}%
⟨latexrelease⟩
⟨latexrelease⟩\let\color@endgroup\relax
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\color@endgroup`.)

`\newsavebox` Allocate a new 'savebox'.

```
\def\newsavebox#1{\@ifdefinable{#1}{\newbox#1}}
```

(*End of definition for* `\newsavebox`.)

\savebox Save #1 in a box register.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩                  {\savebox}{Make \savebox robust}%
⟨*2ekernel | latexrelease⟩
\DeclareRobustCommand\savebox[1]{%
  \@ifnextchar(%)
    {\@savepicbox#1}{\@ifnextchar[{\@savebox#1}{\sbox#1}}}%
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\savebox}{Make \savebox robust}%
⟨latexrelease⟩\def\savebox#1{%
⟨latexrelease⟩  \@ifnextchar(%)
⟨latexrelease⟩    {\@savepicbox#1}{\@ifnextchar[{\@savebox#1}{\sbox#1}}}%
⟨latexrelease⟩\expandafter\let\csname savebox \endcsname\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* \savebox.)

\sbox Save #1 in a box register.

```
\DeclareRobustCommand\sbox[2]{\setbox#1\hbox{%
  \color@setgroup#2\color@endgroup}}
```

(*End of definition for* \sbox.)

\@savebox Look for second optional argument.

```
\def\@savebox#1[#2]{%
  \@ifnextchar [{\@isavebox#1[#2]}{\@isavebox#1[#2][c]}}
```

(*End of definition for* \@savebox.)

\@isavebox

```
\long\def\@isavebox#1[#2][#3]#4{%
  \sbox#1{\@imakebox[#2][#3]{#4}}}
```

(*End of definition for* \@isavebox.)

\@savepicbox Picture mode version of \savebox.

```
\def\@savepicbox#1(#2,#3){%
  \@ifnextchar[%]
    {\@isavepicbox#1(#2,#3)}{\@isavepicbox#1(#2,#3)[]}}
```

(*End of definition for* \@savepicbox.)

\@isavepicbox Picture mode version of \savebox.

```
\long\def\@isavepicbox#1(#2,#3)[#4]#5{%
  \sbox#1{\@imakepicbox(#2,#3)[#4]{#5}}}
```

(*End of definition for* \@isavepicbox.)

`\lrbox` `lrbox`: the new environment form of `\sbox`. Use `\aftergroup` tricks to enable a *local* assignment to be made to the box, in a way that it still has an effect *outside* the `lrbox` environment.

```
\def\lrbox#1{%
  \edef\reserved@a{%
    \endgroup
    \setbox#1\hbox{%
      \begingroup\aftergroup}%
        \def\noexpand\@currenvir{\@currenvir}%
        \def\noexpand\@currenvline{\on@line}}%
  \reserved@a
    \@endpefalse
    \color@setgroup
      \ignorespaces}
```

(*End of definition for* `\lrbox`.)

`\endlrbox` End the `lrbox` environment.

```
\def\endlrbox{\unskip\color@endgroup}
```

(*End of definition for* `\endlrbox`.)

`\usebox` unchanged

```
\DeclareRobustCommand\usebox[1]{\leavevmode\copy #1\relax}
```

(*End of definition for* `\usebox`.)

`\frame` The following definition of `\frame` was written by Pavel Curtis (Extra space removed 14 Jan 88) RmS 92/08/24: Replaced occurrence of `\@halfwidth` by `\@wholewidth`

```
\DeclareRobustCommand\frame[1]{%
  \leavevmode
  \hbox{%
    \hskip-\@wholewidth
    \vbox{%
      \vskip-\@wholewidth
      \hrule \@height\@wholewidth
      \hbox{%
        \vrule\@width\@wholewidth
        #1%
        \vrule\@width\@wholewidth}%
      \hrule\@height\@wholewidth
      \vskip-\@wholewidth}%
    \hskip-\@wholewidth}}
```

(*End of definition for* `\frame`.)

`\fboxrule` `\fboxsep` user level parameters,

```
\newdimen\fboxrule
\newdimen\fboxsep
```

(*End of definition for* `\fboxrule` *and* `\fboxsep`.)

`\fbox` Abbreviated framed box command.

```
\DeclareRobustCommand\fbox[1]{%
  \leavevmode
  \setbox\@tempboxa\hbox{%
    \color@begingroup
      \kern\fboxsep{#1}\kern\fboxsep
    \color@endgroup}%
  \@frameb@x\relax}
```

(*End of definition for* `\fbox`.)

`\framebox` Framed version of `\makebox`.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩                    {\framebox}{Make \framebox robust}%
⟨*2ekernel | latexrelease⟩
\DeclareRobustCommand\framebox{%
  \@ifnextchar(%)
    \@framepicbox{\@ifnextchar[\@framebox\fbox}}%
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\framebox}{Make \framebox robust}%
⟨latexrelease⟩\def\framebox{%
⟨latexrelease⟩   \@ifnextchar(%)
⟨latexrelease⟩     \@framepicbox{\@ifnextchar[\@framebox\fbox}}%
⟨latexrelease⟩\expandafter\let\csname framebox \endcsname\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\framebox`.)

`\@framebox` Deal with optional arguments.

```
\def\@framebox[#1]{%
  \@ifnextchar[%]
    {\@iframebox[#1]}%
    {\@iframebox[#1][c]}}
```

(*End of definition for* `\@framebox`.)

`\@iframebox` The handling the optional arguments. In order to set the whole box, including the frame to the specified dimension, we first determine that dimension from the natural size of the text, `#3`. calculated width.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2023/11/01}%
⟨latexrelease⟩                    {\@iframebox}{Unknown alignment warning}%
⟨*2ekernel | latexrelease⟩
\long\def\@iframebox[#1][#2]#3{%
  \leavevmode
  \@begin@tempboxa\hbox{#3}%
    \setlength\@tempdima{#1}%
    \setbox\@tempboxa\hb@xt@\@tempdima
         {\kern\fboxsep
          \expandafter\ifx\csname bm@#2\endcsname\relax
```

```
            \bm@c
            \@latex@warning{Unexpected alignment #2}%
          \else
            \csname bm@#2\endcsname
          \fi
          \kern\fboxsep}%
    \@frameb@x{\kern-\fboxrule}%
  \@end@tempboxa}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@iframebox}{Unknown alignment warning}%
⟨latexrelease⟩\long\def\@iframebox[#1][#2]#3{%
⟨latexrelease⟩  \leavevmode
⟨latexrelease⟩  \@begin@tempboxa\hbox{#3}%
⟨latexrelease⟩    \setlength\@tempdima{#1}%
⟨latexrelease⟩    \setbox\@tempboxa\hb@xt@\@tempdima
⟨latexrelease⟩         {\kern\fboxsep\csname bm@#2\endcsname\kern\fboxsep}%
⟨latexrelease⟩    \@frameb@x{\kern-\fboxrule}%
⟨latexrelease⟩  \@end@tempboxa}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@iframebox`.)

`\@frameb@x` Common part of `\framebox` and `\fbox`. `#1` is a negative kern in the `\framebox` case so that the vertical rules do not add to the width of the box.

```
\def\@frameb@x#1{%
  \@tempdima\fboxrule
  \advance\@tempdima\fboxsep
  \advance\@tempdima\dp\@tempboxa
  \hbox{%
    \lower\@tempdima\hbox{%
      \vbox{%
        \hrule\@height\fboxrule
        \hbox{%
          \vrule\@width\fboxrule
          #1%
          \vbox{%
            \vskip\fboxsep
            \box\@tempboxa
            \vskip\fboxsep}%
          #1%
          \vrule\@width\fboxrule}%
        \hrule\@height\fboxrule}%
                          }%
        }%
}
```

(*End of definition for* `\@frameb@x`.)

`\@framepicbox` Picture mode version.

```
\def\@framepicbox(#1,#2){%
  \@ifnextchar[{\@iframepicbox(#1,#2)}{\@iframepicbox(#1,#2)[]}}
```

(*End of definition for* `\@framepicbox`.)

`\@iframepicbox` Picture mode version.

```
\long\def\@iframepicbox(#1,#2)[#3]#4{%
  \frame{\@imakepicbox(#1,#2)[#3]{#4}}}
```

(*End of definition for* `\@iframepicbox`.)

`\parbox` The main vertical-box command for LaTeX.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩                  {\parbox}{Make \parbox robust}%
⟨*2ekernel | latexrelease⟩
\DeclareRobustCommand\parbox{%
  \@ifnextchar[%
    \@iparbox
    {\@iiiparbox c\relax[s]}}%
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\parbox}{Make \parbox robust}%
⟨latexrelease⟩\def\parbox{%
⟨latexrelease⟩  \@ifnextchar[%
⟨latexrelease⟩    \@iparbox
⟨latexrelease⟩    {\@iiiparbox c\relax[s]}}%
⟨latexrelease⟩\expandafter\let\csname parbox \endcsname\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\parbox`.)

`\@iparbox` Optional argument handling.

```
\def\@iparbox[#1]{%
  \@ifnextchar[%
    {\@iiparbox{#1}}%
    {\@iiiparbox{#1}\relax[s]}}
```

(*End of definition for* `\@iparbox`.)

`\@iiparbox` Optional argument handling.

```
\def\@iiparbox#1[#2]{%
  \@ifnextchar[%
    {\@iiiparbox{#1}{#2}}%
    {\@iiiparbox{#1}{#2}[#1]}}
```

(*End of definition for* `\@iiparbox`.)

`\vcenter@text` `\vcenter@text@auxi` `\vcenter@text@auxii` `\vcenter@text@axis` We have this idea in the L3 layer, but for use in tabulars, we need close-to primitive syntax: that means some `\afterassignment` and `\aftergroup` fun (remember `\afterassignment` for boxes inserts immediately after the start of the box). To keep the calculation here as close as possible to the one used by `\vcenter`, we implement the algorithm from `tex.web`:

```
delta:=height(v)+depth(v);
height(v):=axis_height(cur_size)+half(delta);
depth(v):=delta-height(v);
```

where

```
@p function half(@!x:integer):integer;
begin if odd(x) then half:=(x+1) div 2
else half:=x @!div 2;
end;
```

As the total height is needed a couple of times, this is worked out in a scratch dimen, before doing everything else using expressions. This way we get pretty much exactly the same as `tex.web`.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2026/06/01}%
⟨latexrelease⟩                    {\vcenter@text}{Use real text vcenter}%
⟨*2ekernel | latexrelease⟩
\protected\def\vcenter@text{%
  \check@mathfonts
  \afterassignment\vcenter@text@auxi
  \setbox0\vbox}
\protected\def\vcenter@text@auxi{%
  \aftergroup\vcenter@text@auxii
}
\protected\def\vcenter@text@auxii{%
  \begingroup
    \dimen0\dimexpr\ht0+\dp0\relax
    \ht0\dimexpr\dimexpr\ifodd\dimen0 1sp + \fi\dimen0\relax/2
      +\vcenter@text@axis\relax
    \dp0\dimexpr\dimen0-\ht0\relax
    \box0 %
  \endgroup
}
\ifdefined\directlua
  \newcommand*\vcenter@text@axis{\Umathaxis\textstyle}
\else
  \newcommand*\vcenter@text@axis{\fontdimen22 \textfont2 }
\fi
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\vcenter@text}{Use real text vcenter}%
⟨latexrelease⟩\let\vcenter@text\@undefined
⟨latexrelease⟩\let\vcenter@text@auxi\@undefined
⟨latexrelease⟩\let\vcenter@text@auxii\@undefined
⟨latexrelease⟩\let\vcenter@text@axis\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\vcenter@text` *and others.*)

`\@iiiparbox` `\@parboxto`

The internal version of `\parbox`.

```
\let\@parboxto\@empty
```

```
366 ⟨/2ekernel⟩
367 ⟨latexrelease⟩\IncludeInRelease{2026/06/01}%
368 ⟨latexrelease⟩                  {\@iiiparbox}{Use real text vcenter}%
369 ⟨*2ekernel | latexrelease⟩
370 \long\def\@iiiparbox#1#2[#3]#4#5{%
371   \leavevmode
372   \setlength\@tempdima{#4}%
373   \@begin@tempboxa\vbox{\hsize\@tempdima\@parboxrestore#5\@@par}%
374     \ifx\relax#2\else
375       \setlength\@tempdimb{#2}%
376       \edef\@parboxto{to\the\@tempdimb}%
377     \fi
378     \if#1b\vbox
379     \else\if #1t\vtop
380     \else\vcenter@text
381     \fi\fi
382     \@parboxto{\let\hss\vss\let\unhbox\unvbox
383      \expandafter\ifx\csname bm@#3\endcsname\relax
384        \bm@c
385        \@latex@warning{Unexpected alignment #3}%
386      \else
387        \csname bm@#3\endcsname
388      \fi}%
389   \@end@tempboxa}
390 ⟨/2ekernel | latexrelease⟩
391 ⟨latexrelease⟩\EndIncludeInRelease
392 ⟨latexrelease⟩\IncludeInRelease{2023/11/01}%
393 ⟨latexrelease⟩                  {\@iiiparbox}{Use real text vcenter}%
394 ⟨latexrelease⟩\long\def\@iiiparbox#1#2[#3]#4#5{%
395 ⟨latexrelease⟩   \leavevmode
396 ⟨latexrelease⟩   \@pboxswfalse
397 ⟨latexrelease⟩   \setlength\@tempdima{#4}%
398 ⟨latexrelease⟩   \@begin@tempboxa\vbox{\hsize\@tempdima\@parboxrestore#5\@@par}%
399 ⟨latexrelease⟩     \ifx\relax#2\else
400 ⟨latexrelease⟩       \setlength\@tempdimb{#2}%
401 ⟨latexrelease⟩       \edef\@parboxto{to\the\@tempdimb}%
402 ⟨latexrelease⟩     \fi
403 ⟨latexrelease⟩     \if#1b\vbox
404 ⟨latexrelease⟩     \else\if #1t\vtop
405 ⟨latexrelease⟩     \else\ifmmode\vcenter
406 ⟨latexrelease⟩     \else\@pboxswtrue $\vcenter
407 ⟨latexrelease⟩     \fi\fi\fi
408 ⟨latexrelease⟩     \@parboxto{\let\hss\vss\let\unhbox\unvbox
409 ⟨latexrelease⟩      \expandafter\ifx\csname bm@#3\endcsname\relax
410 ⟨latexrelease⟩        \bm@c
411 ⟨latexrelease⟩        \@latex@warning{Unexpected alignment #3}%
412 ⟨latexrelease⟩      \else
413 ⟨latexrelease⟩        \csname bm@#3\endcsname
414 ⟨latexrelease⟩      \fi}%
415 ⟨latexrelease⟩     \if@pboxsw \m@th$\fi
416 ⟨latexrelease⟩   \@end@tempboxa}
417 ⟨latexrelease⟩\EndIncludeInRelease
418 ⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
419 ⟨latexrelease⟩                  {\@iiiparbox}{Use real text vcenter}%
```

```
⟨latexrelease⟩\long\def\@iiiparbox#1#2[#3]#4#5{%
⟨latexrelease⟩  \leavevmode
⟨latexrelease⟩  \@pboxswfalse
⟨latexrelease⟩  \setlength\@tempdima{#4}%
⟨latexrelease⟩  \@begin@tempboxa\vbox{\hsize\@tempdima\@parboxrestore#5\@@par}%
⟨latexrelease⟩    \ifx\relax#2\else
⟨latexrelease⟩      \setlength\@tempdimb{#2}%
⟨latexrelease⟩      \edef\@parboxto{to\the\@tempdimb}%
⟨latexrelease⟩    \fi
⟨latexrelease⟩    \if#1b\vbox
⟨latexrelease⟩    \else\if #1t\vtop
⟨latexrelease⟩    \else\ifmmode\vcenter
⟨latexrelease⟩    \else\@pboxswtrue $\vcenter
⟨latexrelease⟩    \fi\fi\fi
⟨latexrelease⟩    \@parboxto{\let\hss\vss\let\unhbox\unvbox
⟨latexrelease⟩       \csname bm@#3\endcsname}%
⟨latexrelease⟩    \if@pboxsw \m@th$\fi
⟨latexrelease⟩  \@end@tempboxa}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@iiiparbox` *and* `\@parboxto`.)

`\@arrayparboxrestore` Restore various paragraph parameters.

The rational for allowing two normally global flags to be set locally here was stated originally by Donald Arseneau and extended by Chris Rowley. It is because these flags are only set globally to true by section commands, and these should never appear within boxes or, indeed, in any group; and they are only ever set globally to false when they are definitely true.

If anyone is unhappy with this argument then both flags should be treated as in `\@setnobreak`; otherwise this command will be redundant.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2017-04-15}%
⟨latexrelease⟩                  {\normallineskiplimit}
⟨latexrelease⟩                  {reset \lineskiplimit}%
⟨*2ekernel | latexrelease⟩
\def\@arrayparboxrestore{%
  \let\if@nobreak\iffalse
  \let\if@noskipsec\iffalse
  \let\par\@@par
  \let\-\@dischyph
```

Redefined accents to allow changes in font encoding

```
  \let\'\@acci\let\`\@accii\let\=\@acciii
  \parindent\z@ \parskip\z@skip
  \everypar{}%
  \linewidth\hsize
  \@totalleftmargin\z@
  \leftskip\z@skip \rightskip\z@skip \@rightskip\z@skip
  \parfillskip\@flushglue
  \lineskip\normallineskip

  \lineskiplimit\normallineskiplimit
```

```
  \baselineskip\normalbaselineskip
  \sloppy}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000-00-00}%
⟨latexrelease⟩                  {\normallineskiplimit}
⟨latexrelease⟩                  {reset \lineskiplimit}%
⟨latexrelease⟩\def\@arrayparboxrestore{%
⟨latexrelease⟩  \let\if@nobreak\iffalse
⟨latexrelease⟩  \let\if@noskipsec\iffalse
⟨latexrelease⟩  \let\par\@@par
⟨latexrelease⟩  \let\-\@dischyph
⟨latexrelease⟩  \let\'\@acci\let\`\@accii\let\=\@acciii
⟨latexrelease⟩  \parindent\z@ \parskip\z@skip
⟨latexrelease⟩  \everypar{}%
⟨latexrelease⟩  \linewidth\hsize
⟨latexrelease⟩  \@totalleftmargin\z@
⟨latexrelease⟩  \leftskip\z@skip \rightskip\z@skip \@rightskip\z@skip
⟨latexrelease⟩  \parfillskip\@flushglue \lineskip\normallineskip
⟨latexrelease⟩  \baselineskip\normalbaselineskip
⟨latexrelease⟩  \sloppy}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@arrayparboxrestore`.)

`\@parboxrestore` Restore various paragraph parameters, and also `\\`.

```
\def\@parboxrestore{\@arrayparboxrestore\let\\\@normalcr}
```

(*End of definition for* `\@parboxrestore`.)

`\if@minipage` Switch that is true at the start of a minipage.

```
\def\@minipagefalse{\global\let\if@minipage\iffalse}
\def\@minipagetrue {\global\let\if@minipage\iftrue}
\@minipagefalse
```

(*End of definition for* `\if@minipage`.)

`\if@in@minipage@env`

```
\newif\if@in@minipage@env
```

(*End of definition for* `\if@in@minipage@env`.)

`\minipage` Essentially an environment form of `\parbox`.

```
\def\minipage{%
  \@ifnextchar[%]
    \@iminipage
    {\@iiiminipage c\relax[s]}}
```

(*End of definition for* `\minipage`.)

`\@iminipage` Optional argument handling.

```
\def\@iminipage[#1]{%
  \@ifnextchar[%]
    {\@iiminipage{#1}}%
    {\@iiiminipage{#1}\relax[s]}}
```

(*End of definition for* `\@iminipage`.)

`\@iiminipage` Optional argument handling.

```
\def\@iiminipage#1[#2]{%
  \@ifnextchar[%]
    {\@iiiminipage{#1}{#2}}%
    {\@iiiminipage{#1}{#2}[#1]}}
```

(*End of definition for* `\@iiminipage`.)

`\@iiiminipage` Internal form of `minipage`.

```
\def\@iiiminipage#1#2[#3]#4{%
  \leavevmode
  \@pboxswfalse
  \setlength\@tempdima{#4}%
  \def\@mpargs{{#1}{#2}[#3]{#4}}%
  \setbox\@tempboxa\vbox\bgroup
    \color@begingroup
      \hsize\@tempdima
      \textwidth\hsize \columnwidth\hsize
```

We check for nested minipages inside the box so that there is always a group resetting the switch even if the code does not use `\begin` to start the minipage.

```
      \if@in@minipage@env
```

We only issue a warning if the outer minipage contained footnotes because that is the problematical case.

```
        \ifvoid\@mpfootins\else
          \@latex@warning{Nested minipage:
            footnotes may be misplaced}%
        \fi
      \else
        \@in@minipage@envtrue
      \fi
      \@parboxrestore
      \def\@mpfn{mpfootnote}\def\thempfn{\thempfootnote}\c@mpfootnote\z@
      \let\@footnotetext\@mpfootnotetext
      \let\@listdepth\@mplistdepth \@mplistdepth\z@
      \@minipagerestore
      \@setminipage}
```

(*End of definition for* `\@iiiminipage`.)

`\@minipagerestore` Hook so that other styles can reset other commands in a minipage.

```
 \let\@minipagerestore=\relax
```

(*End of definition for* `\@minipagerestore`.)

`\endminipage`

```
\def\endminipage{%
    \par
    \unskip
    \ifvoid\@mpfootins\else
      \vskip\skip\@mpfootins
      \normalcolor
```

```
      \footnoterule
      \unvbox\@mpfootins
    \fi
    \@minipagefalse   %% added 24 May 89
  \color@endgroup
  \egroup
  \expandafter\@iiiparbox\@mpargs{\unvbox\@tempboxa}}
```

(*End of definition for* `\endminipage`.)

`\@mplistdepth` `\@mpfootins` Versions of `\@listdepth` and `\footins` local to minipage.

```
\newcount\@mplistdepth
\newinsert\@mpfootins
```

(*End of definition for* `\@mplistdepth` *and* `\@mpfootins`.)

`\@mpfootnotetext` Minipage version of `\@footnotetext`.

Final `\strut` added 27 Mar 89, on suggestion by Don Hosek

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2021/11/15}%
⟨latexrelease⟩                  {\@mpfootnotetext}{footnotetext tagging}%
\long\def\@mpfootnotetext#1{%
  \global\setbox\@mpfootins\vbox{%
    \unvbox\@mpfootins
    \reset@font\footnotesize
    \hsize\columnwidth
    \@parboxrestore
    \def\@currentcounter{mpfootnote}%
    \protected@edef\@currentlabel
         {\csname p@mpfootnote\endcsname\@thefnmark}%
    \color@begingroup
      \@makefntext{%
        \rule\z@\footnotesep\ignorespaces#1\@finalstrut\strutbox}%
    \par
    \color@endgroup}}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩                  {\@mpfootnotetext}{footnotetext tagging}%
⟨latexrelease⟩\long\def\@mpfootnotetext#1{%
⟨latexrelease⟩  \global\setbox\@mpfootins\vbox{%
⟨latexrelease⟩    \unvbox\@mpfootins
⟨latexrelease⟩    \reset@font\footnotesize
⟨latexrelease⟩    \hsize\columnwidth
⟨latexrelease⟩    \@parboxrestore
⟨latexrelease⟩    \protected@edef\@currentlabel
⟨latexrelease⟩         {\csname p@mpfootnote\endcsname\@thefnmark}%
⟨latexrelease⟩    \color@begingroup
⟨latexrelease⟩      \@makefntext{%
⟨latexrelease⟩        \rule\z@\footnotesep\ignorespaces#1\@finalstrut\strutbox}%
⟨latexrelease⟩    \par
⟨latexrelease⟩    \color@endgroup}}
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@mpfootnotetext}{footnotetext tagging}%
⟨latexrelease⟩
⟨latexrelease⟩\long\def\@mpfootnotetext#1{%
⟨latexrelease⟩  \global\setbox\@mpfootins\vbox{%
⟨latexrelease⟩    \unvbox\@mpfootins
⟨latexrelease⟩    \reset@font\footnotesize
⟨latexrelease⟩    \hsize\columnwidth
⟨latexrelease⟩    \@parboxrestore
⟨latexrelease⟩    \protected@edef\@currentlabel
⟨latexrelease⟩        {\csname p@mpfootnote\endcsname\@thefnmark}%
⟨latexrelease⟩    \color@begingroup
⟨latexrelease⟩      \@makefntext{%
⟨latexrelease⟩        \rule\z@\footnotesep\ignorespaces#1\@finalstrut\strutbox}%
⟨latexrelease⟩    \color@endgroup}}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@mpfootnotetext`.)

```
\newif\if@pboxsw
```

`\rule` Draw a rule of the specified size.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩                  {\rule}{Make \rule robust}%
⟨*2ekernel | latexrelease⟩
\DeclareRobustCommand\rule{\@ifnextchar[\@rule{\@rule[\z@]}}%
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\rule}{Make \rule robust}%
⟨latexrelease⟩\def\rule{\@ifnextchar[\@rule{\@rule[\z@]}}%
⟨latexrelease⟩\expandafter\let\csname rule \endcsname\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\rule`.)

`\@rule` Internal form of `\rule`.

```
\def\@rule[#1]#2#3{%
  \leavevmode
  \hbox{%
    \setlength\@tempdima{#1}%
    \setlength\@tempdimb{#2}%
    \setlength\@tempdimc{#3}%
    \advance\@tempdimc\@tempdima
    \vrule\@width\@tempdimb\@height\@tempdimc\@depth-\@tempdima}}
```

(*End of definition for* `\@rule`.)

`\@@underline` Saved primitive `\underline`.

```
\let\@@underline\underline
```

(*End of definition for* `\@@underline`.)

\underline
\underline@text
\underline@text@vgap
\underline@text@thickness
\underline@text@kern

As with other areas where we have removed math mode usage from text mode material, we get as close as possible to primitive behaviors. As in other places, we have to do a low-level box size change rather than add a kern inside the box for the vgap. To preserve glue setting, we need to add an additional box level compared to the TEX primitive: using \unhcopy fails if the content contains any glue components.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2026-11-01}%
⟨latexrelease⟩                    {\underline}{Avoid 'fake math'}%
⟨*2ekernel | latexrelease⟩
\DeclareRobustCommand\underline[1]{%
  \relax
  \ifmmode
    \expandafter\@@underline
  \else
    \expandafter\underline@text
  \fi
    {#1}%
}
\protected\def\underline@text#1{%
  \leavevmode
  \check@mathfonts
  \begingroup
    \setbox0=\hbox{#1}%
    \setbox0=\vtop{%
      \hsize\wd0 %
      \copy0 %
      \kern\dimexpr\underline@text@vgap\relax
      \hrule height\dimexpr\underline@text@thickness\relax
       depth 0pt\relax
    }%
    \dp0\dimexpr\dp0+\underline@text@kern\relax
    \box0 %
  \endgroup
}
\ifdefined\directlua
  \newcommand*\underline@text@vgap{\Umathunderbarvgap\textstyle}
  \newcommand*\underline@text@thickness{\Umathunderbarrule\textstyle}
  \newcommand*\underline@text@kern{\Umathunderbarkern\textstyle}
\else
  \newcommand*\underline@text@vgap{3\fontdimen8\textfont3 }
  \newcommand*\underline@text@thickness{\fontdimen8\textfont3 }
  \newcommand*\underline@text@kern{\fontdimen8\textfont3 }
\fi
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\underline}{Avoid 'fake math'}%
⟨latexrelease⟩\DeclareRobustCommand\underline[1]{%
⟨latexrelease⟩  \relax
⟨latexrelease⟩  \ifmmode\@@underline{#1}%
⟨latexrelease⟩  \else $\@@underline{\hbox{#1}}\m@th$\relax\fi}
⟨latexrelease⟩\let\underline@text\@undefined
⟨latexrelease⟩\let\underline@text@vgap\@undefined
```

```
⟨latexrelease⟩\let\underline@text@thickness\@undefined
⟨latexrelease⟩\let\underline@text@kern\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\underline` *and others.*)

`\raisebox` Raise a box, and change its vertical dimensions.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩                    {\raisebox}{Make \raisebox robust}%
⟨*2ekernel | latexrelease⟩
\DeclareRobustCommand\raisebox[1]{%
  \leavevmode
  \@ifnextchar[{\@rsbox{#1}}{\@irsbox{#1}[]}}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\raisebox}{Make \raisebox robust}%
⟨latexrelease⟩\def\raisebox#1{%
⟨latexrelease⟩   \leavevmode
⟨latexrelease⟩   \@ifnextchar[{\@rsbox{#1}}{\@irsbox{#1}[]}}
⟨latexrelease⟩\expandafter\let\csname raisebox \endcsname\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\raisebox`.)

`\@rsbox` Optional argument handling.

```
\def\@rsbox#1[#2]{%
  \@ifnextchar[{\@iirsbox{#1}[#2]}{\@irsbox{#1}[#2]}}
```

(*End of definition for* `\@rsbox`.)

`\@argrsbox` ...

(*End of definition for* `\@argrsbox`.)

`\@irsbox` Internal version of `\raisebox` (less than two optional args).

```
\long\def\@irsbox#1[#2]#3{%
  \@begin@tempboxa\hbox{#3}%
    \setlength\@tempdima{#1}%
    \ifx\\#2\\\else\setlength\@tempdimb{#2}\fi
    \setbox\@tempboxa\hbox{\raise\@tempdima\box\@tempboxa}%
    \ifx\\#2\\\else\ht\@tempboxa\@tempdimb\fi
    \box\@tempboxa
  \@end@tempboxa}
```

(*End of definition for* `\@irsbox`.)

`\@iirsbox` Internal version of `\raisebox` (two optional args).

```
\long\def\@iirsbox#1[#2][#3]#4{%
  \@begin@tempboxa\hbox{#4}%
    \setlength\@tempdima{#1}%
    \setlength\@tempdimb{#2}%
```

```
    \setlength\dimen@{#3}%
    \setbox\@tempboxa\hbox{\raise\@tempdima\box\@tempboxa}%
    \ht\@tempboxa\@tempdimb
    \dp\@tempboxa\dimen@
    \box\@tempboxa
  \@end@tempboxa}
```

(*End of definition for* `\@iirsbox`.)

`\@finalstrut` This macro adds a special strut the *depth* of the box given as `#1`, and height and width 0pt. It is used for ensuring that the last line of a paragraph has the correct depth in 'p' columns of tables and in footnotes. In vertical mode nothing is done, as adding the strut (as done in 2.09) would start a new paragraph. It would be possible to inspect `\prevdepth` to check the depth of the just-completed paragraph, but we do not do that here. Actually we do even less now, skip the vmode test as it broke tabular 'p' columns.
.

The `\nobreak` was added (1995/10/31) to allow hyphenation of the final word of the paragraph.

In 2024 we changed the macro to account for vertical mode. In that case we use a strut produced with `\hrule` to avoid starting a new paragraph (resulting in spurious extra line) and also account for the `\prevdepth` of the previous line.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2024/06/01}%
⟨latexrelease⟩                    {\@finalstrut}{final strut correction}%
\def\@finalstrut#1{%
  \unskip
  \ifhmode \nobreak
  \else
```

If we are in vmode we now back up by a baseline.

```
    \vskip-\baselineskip
  \fi
```

Finally we unconditionally use `\vrule`.

```
  \vrule\@width\z@\@height\z@\@depth\dp#1}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\@finalstrut}{final strut correction}%
⟨latexrelease⟩\def\@finalstrut#1{%
⟨latexrelease⟩  \unskip\ifhmode\nobreak\fi
⟨latexrelease⟩  \vrule\@width\z@\@height\z@\@depth\dp#1}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@finalstrut`.)

1.1 Some low-level constructs

The following commands are basically inherited from plain TEX.

\leftline
\rightline
\centerline
\@@line

These macros place text on a full line either centered or left or right adjusted.

```
\def\@@line{\hb@xt@\hsize}
\DeclareRobustCommand\leftline[1]{\@@line{#1\hss}}
\DeclareRobustCommand\rightline[1]{\@@line{\hss#1}}
\DeclareRobustCommand\centerline[1]{\@@line{\hss#1\hss}}
```

(*End of definition for* `\leftline` *and others.*)

\rlap
\llap
\clap

These macros place text to the left or right of the current reference point without taking up space.

```
\DeclareRobustCommand\rlap[1]{\hb@xt@\z@{#1\hss}}
\DeclareRobustCommand\llap[1]{\hb@xt@\z@{\hss#1}}
```

And here is the version that centers, it was initially introduced by `mathtools`.

```
\DeclareRobustCommand\clap[1]{\hb@xt@\z@{\hss#1\hss}}
```

(*End of definition for* `\rlap`*,* `\llap`*, and* `\clap`*.*)

```
⟨/2ekernel⟩
```

File 41
lttab.dtx

1 Tabbing, Tabular and Array Environments

This section deals with 'Lining It Up in Columns'. First the `tabbing` environment is defined, and then in second part, `tabular` together with its variants, `tabular*` and `array`.

Note that the `tabular` defined here is essentially the original LaTeX 2.09 version, not the extended version described in *The LaTeX Companion*. Use the `array` package to obtain the extended version.

1.1 tabbing

Historical LaTeX 2.09 comments (not necessarily accurate any more):

```
\dimen(\@firsttab + i) = distance of tab stop i from left margin
       0 <= i <= 15 (?).

\dimen\@firsttab is initialized to \@totalleftmargin, so it starts
    at the prevailing left margin.

\@maxtab           = number of highest defined tab register
                       probably = \@firsttab + 12
\@nxttabmar = tab stop number of next line's left margin
\@curtabmar = tab stop number of current line's left margin
\@curtab    = number of the current tab. At start of line,
                     it equals \@curtabmar
\@hightab   = largest tab number currently defined.
\@tabpush   = depth of \pushtab's

\box\@curline      = contents of current line, excluding left margin
                       skip, and excluding contents of current field
\box\@curfield     = contents of current field

@rjfield           = switch: T iff the last field of the line should
                       be right-justified at the right margin.

\tabbingsep            = distance left by the \' command between the
                       current position and the field that is
                       "left-shifted".

UTILITY MACROS
 \@stopfield  : closes the current field
 \@addfield   : adds the current field to the current line.
 \@contfield  : continues the current field
 \@startfield : begins the next field
 \@stopline   : closes the current line and outputs it
```

```
  \@startline  : starts the next line
  \@ifatmargin : an \if that is true iff the current line.
                 has width zero

\@startline ==
 BEGIN
  \@curtabmar :=G \@nxttabmar
  \@curtab :=G \@curtabmar
  \box\@curline :=G null
  \@startfield
  \strut
 END

\@stopline ==
 BEGIN
  \unskip
  \@stopfield
  if @rjfield = T
    then  @rjfield :=G F
          \@tempdima := \@totalleftmargin + \linewidth
          \hb@xt@ \@tempdima{\@itemfudge
                             \hskip \dimen\@curtabmar
                             \box\@curline
                             \hfil
                             \box\@curfield}
    else \@addfield
         \hbox {\@itemfudge
                \hskip \dimen\@curtabmar
                \box\@curline}
  fi
 END

\@startfield ==
 BEGIN
   \box\@curfield :=G \hbox {
 END

\@stopfield ==
 BEGIN
    }
 END

\@contfield ==
 BEGIN
  \box\@curfield :=G \hbox { \unhbox\@currfield  %%} brace matching
 END
\@addfield ==
 BEGIN
  \box\@curline :=G \unbox\@curline * \unbox\@curfield
 END
```

```
\@ifatmargin ==
 BEGIN
  if  dim of box\@curline = 0pt  then
 END


\tabbing ==
 BEGIN
  \lineskip :=L 0pt
  \> == \@rtab
  \< == \@ltab
  \= == \@settab
  \+ == \@tabplus
  \- == \@tabminus
  \‘ == \@tabrj
  \’ == \@tablab
  \\ == BEGIN \@stopline \@startline END
  \\[DIST] == BEGIN
                \@stopline \vskip DIST \@startline\ignorespaces END
  \\* == BEGIN \@stopline \penalty 10000 \@startline END
  \\*[DIST] == BEGIN \@stopline \penalty 10000 \vskip DIST
                       \@startline\ignorespaces                END
  \@hightab :=  \@nxttabmar :=G \@firsttab
  \@tabpush :=G 0
  \dimen\@firsttab := \@totalleftmargin
  @rjfield :=G F
  \trivlist  \item\relax
  if @minipage = F then \vskip \parskip fi
  \box\@tabfbox = \rlap{\indent\the\everypar}
                          % note: \the\everypar sets @inlabel :=G F
  \@itemfudge == BEGIN \box\@tabfbox END
  \@startline
  \ignorespaces
 END

\@endtabbing ==
 BEGIN
  \@stopline
  if \@tabpush > 0 then error message: "unmatched \poptabs'' fi
  \endtrivlist
 END

\@rtab ==
 BEGIN
  \@stopfield
  \@addfield
  if \@curtab < \@hightab
    then \@curtab :=G \@curtab + 1
    else error message “Undefined Tab”    fi
```

```
  \@tempdima := \dimen\@curtab - \dimen\@curtabmar
                        - width of box \@curline
  \box\@curline :=G \hbox{\unhbox\@curline + \hskip\@tempdima}
  \@startfield
 END

\@settab ==
 BEGIN
  \@stopfield
  \@addfield
  if \@curtab < \@maxtab
    then \@curtab :=G \@curtab+1
    else error message: "Too many tabs"      fi
  if \@curtab > \@hightab
    then \@hightab :=L \@curtab     fi
  \dimen\@curtab :=L \dimen\@curtabmar + width of \box\@curline
  \@startfield
 END

\@ltab ==
 BEGIN
  \@ifatmargin
    then if \@curtabmar > \@firsttab
           then \@curtab :=G \@curtab - 1
                \@curtabmar :=G \@curtabmar - 1
           else error message "Too many untabs"       fi
    else error message "Left tab in middle of line"
  fi
 END

\@tabplus ==
 BEGIN
       if  \@nxttabmar < \@hightab
           then \@nxttabmar :=G \@nxttabmar+1
           else error message "Undefined tab"
       fi
 END

\@tabminus ==
 BEGIN
       if \@nxttabmar > \@firsttab
           then \@nxttabmar :=G \@nxttabmar-1
           else error message "Too many untabs"
       fi
 END

\@tabrj ==
 BEGIN \@stopfield
       \@addfield
       @rjfield :=G T
```

```
      \@startfield
 END

\@tablab ==
 BEGIN \@stopfield
     \box\@curline G:= \hbox{\box\@curline %% 'G' added 17 Jun 86
                           \hskip - width of \box\@curfield
                           \hskip -\tabbingsep
                           \box\@curfield
                           \hskip \tabbingsep }
      \@startfield
 END

\pushtabs ==
  BEGIN
    \@stopfield
    \@tabpush :=G \@tabpush + 1
    \begingroup
    \@contfield
  END

\poptabs ==
 BEGIN
   \@stopfield
   if \@tabpush > 0
     then \endgroup
          \@tabpush :=G \@tabpush - 1
     else error message: "Too many \poptabs''
   fi
   \@contfield
 END
```

End of historical LaTeX 2.09 comments.

\a The accents \‘ , \’ , and \= that have been redefined inside a tabbing environment can be called by typing \a‘ , \a’ , and \a=. The macro \a is defined in `ltoutenc.dtx`.

(*End of definition for* \a.)

The ‘2ekernel’ code ensures that a \usepackage{autotabg} is essentially ignored if a ‘full’ format is being used that has picture mode already in the format.

```
⟨2ekernel⟩\expandafter\let\csname ver@autotabg.sty\endcsname\fmtversion
```

\@firsttab
\@maxtab

```
⟨*2ekernel⟩
\newdimen\@gtempa
\chardef\@firsttab=\the\allocationnumber
\newdimen\@gtempa\newdimen\@gtempa\newdimen\@gtempa\newdimen\@gtempa
\newdimen\@gtempa\newdimen\@gtempa\newdimen\@gtempa\newdimen\@gtempa
\newdimen\@gtempa\newdimen\@gtempa\newdimen\@gtempa\newdimen\@gtempa
\newdimen\@gtempa
\chardef\@maxtab=\the\allocationnumber
\dimen\@firsttab=0pt
```

(*End of definition for* `\@firsttab` *and* `\@maxtab`.)

`\@nxttabmar`
`\@curtabmar`
`\@curtab`
`\@hightab`
`\@tabpush`

```
\newcount\@nxttabmar
\newcount\@curtabmar
\newcount\@curtab
\newcount\@hightab
\newcount\@tabpush
```

(*End of definition for* `\@nxttabmar` *and others.*)

`\@curline`
`\@curfield`
`\@tabfbox`

```
\newbox\@curline
\newbox\@curfield
\newbox\@tabfbox
```

(*End of definition for* `\@curline`, `\@curfield`, *and* `\@tabfbox`.)

`\if@rjfield`

```
\newif\if@rjfield
```

(*End of definition for* `\if@rjfield`.)

`\@startline` It is, in some sense, an error if the current margin tab setting is higher than the value of `\@hightab` (which is a local variable). That this is allowed is a fundamental design flaw which is not going to be corrected now.

```
\def\@startline{%
     \ifnum \@nxttabmar >\@hightab
       \@badtab
       \global\@nxttabmar \@hightab
     \fi
     \global\@curtabmar \@nxttabmar
     \global\@curtab \@curtabmar
     \global\setbox\@curline \hbox {}%
     \@startfield
     \strut}
```

(*End of definition for* `\@startline`.)

`\@stopline`

```
\def\@stopline{%
  \unskip
  \@stopfield
  \if@rjfield
    \global\@rjfieldfalse
    \@tempdima\@totalleftmargin
    \advance\@tempdima\linewidth
    \hb@xt@\@tempdima{%
      \@itemfudge\hskip\dimen\@curtabmar
      \box\@curline
      \hfil
      \box\@curfield}%
  \else
    \@addfield
    \hbox{\@itemfudge\hskip\dimen\@curtabmar\box\@curline}%
  \fi}
```

(*End of definition for* `\@stopline`.)

`\@startfield`

```
\def\@startfield{%
  \global\setbox\@curfield\hbox\bgroup\color@begingroup}
```

(*End of definition for* `\@startfield`.)

`\@stopfield`

```
\def\@stopfield{%
  \color@endgroup\egroup}
```

(*End of definition for* `\@stopfield`.)

`\@contfield`

```
\def\@contfield{%
  \global\setbox\@curfield\hbox\bgroup\color@begingroup
  \unhbox\@curfield}
```

(*End of definition for* `\@contfield`.)

`\@addfield`

```
\def\@addfield{\global\setbox\@curline\hbox{\unhbox
     \@curline\unhbox\@curfield}}
```

(*End of definition for* `\@addfield`.)

`\@ifatmargin`

```
\def\@ifatmargin{\ifdim \wd\@curline =\z@}
```

(*End of definition for* `\@ifatmargin`.)

`\@tabcr`

```
\protected\def\@tabcr{\@stopline \@ifstar{\penalty \@M \@xtabcr}\@xtabcr}
```

(*End of definition for* `\@tabcr`.)

`\@xtabcr`

```
\def\@xtabcr{\@ifnextchar[\@itabcr{\@startline\ignorespaces}}
```

(*End of definition for* `\@xtabcr`.)

`\@itabcr`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                    {\@itabcr}{Tabbing calc syntax}%
\def\@itabcr[#1]{\@vspace@calcify{#1}\@startline\ignorespaces}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\@itabcr}{Tabbing calc syntax}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@itabcr[#1]{\vskip #1\@startline\ignorespaces}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

tabbing (*env.*) We use `\relax` to prevent `\item` from scanning too far.

`\tabbing`

```
\def\tabbing{\lineskip \z@skip\let\>\@rtab\let\<\@ltab\let\=\@settab
     \let\+\@tabplus\let\-\@tabminus\let\‘\@tabrj\let\’\@tablab
     \let\\=\@tabcr
     \@hightab\@firsttab
     \global\@nxttabmar\@firsttab
     \dimen\@firsttab\@totalleftmargin
     \global\@tabpush\z@ \global\@rjfieldfalse
     \trivlist \item\relax
     \if@minipage\else\vskip\parskip\fi

     \setbox\@tabfbox\hbox{%
       \rlap{\hskip\@totalleftmargin\indent\the\everypar}}%
     \def\@itemfudge{\box\@tabfbox}%
     \@startline\ignorespaces}
```

`\endtabbing`

```
\def\endtabbing{%
  \@stopline\ifnum\@tabpush >\z@ \@badpoptabs \fi\endtrivlist}
```

Omitted `\global` added to `\@rtab` 17 Jun 86

`\@rtab`

```
\def\@rtab{\@stopfield\@addfield\ifnum \@curtab<\@hightab
      \global\advance\@curtab \@ne \else\@badtab\fi
      \@tempdima\dimen\@curtab
      \advance\@tempdima -\dimen\@curtabmar
      \advance\@tempdima -\wd\@curline
      \global\setbox\@curline\hbox{\unhbox\@curline\hskip\@tempdima}%
      \@startfield\ignorespaces}
```

`\@settab`

```
\def\@settab{\@stopfield\@addfield
  \ifnum \@curtab <\@maxtab
    \ifnum\@curtab =\@hightab
      \advance\@hightab \@ne
    \fi
    \global\advance\@curtab \@ne
  \else
    \@latex@error{Tab overflow}\@ehd
  \fi
  \dimen\@curtab \dimen\@curtabmar
  \advance\dimen\@curtab \wd\@curline
  \@startfield
  \ignorespaces}
```

`\@ltab`

```
\def\@ltab{\@ifatmargin\ifnum\@curtabmar >\@firsttab
      \global\advance\@curtab \m@ne \global\advance\@curtabmar\m@ne\else
      \@badtab\fi\else
      \@latex@error{\string\<\space in mid line}\@ehd\fi\ignorespaces}
```

`\@tabplus`

```
\def\@tabplus{%
  \ifnum\@nxttabmar<\@hightab
```

```
    \global\advance\@nxttabmar\@ne
  \else
    \@badtab
  \fi
  \ignorespaces}
```

\@tabminus

```
\def\@tabminus{%
  \ifnum\@nxttabmar>\@firsttab
    \global\advance\@nxttabmar\m@ne
  \else
    \@badtab
  \fi
  \ignorespaces}
```

\@tabrj

```
\def\@tabrj{%
  \@stopfield\@addfield\global\@rjfieldtrue\@startfield\ignorespaces}
```

\setbox\@curline made \global in \@tablab. 17 Jun 86

\@tablab

```
\def\@tablab{%
  \@stopfield
  \global\setbox\@curline\hbox{%
    \box\@curline
    \hskip-\wd\@curfield \hskip-\tabbingsep
    \box\@curfield
    \hskip\tabbingsep}%
  \@startfield
  \ignorespaces}
```

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                    {\pushtabs}{Make commands robust}%
```

\pushtabs

```
\DeclareRobustCommand\pushtabs{%
  \@stopfield\@addfield\global\advance\@tabpush \@ne \begingroup
       \@contfield}
```

It is, in some sense, an error if, after the endgroup, the current tab setting is higher than the new value of \@hightab (which is a local variable). That this is allowed is a fundamental design flaw which is not going to be corrected now.

\poptabs

```
\DeclareRobustCommand\poptabs{\@stopfield\@addfield
  \ifnum \@tabpush >\z@
    \endgroup
    \global\advance\@tabpush \m@ne
    \ifnum \@curtab >\@hightab
      \global \@curtab \@hightab
      \@badtab
    \fi
  \else
    \@badpoptabs
  \fi
  \@contfield}
```

```
\DeclareRobustCommand\kill{\@stopfield\@startline\ignorespaces}
```

(*End of definition for* `\@itabcr` *and others.*)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\pushtabs}{Make commands robust}%
⟨latexrelease⟩
⟨latexrelease⟩\kernel@make@fragile\pushtabs
⟨latexrelease⟩\kernel@make@fragile\poptabs
⟨latexrelease⟩\kernel@make@fragile\kill
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

`\tabbingsep`

```
\newdimen\tabbingsep
```

(*End of definition for* `\tabbingsep`.)

1.2 array and tabular environments

Historical LaTeX 2.09 comments (not necessarily accurate any more):

ARRAY PARAMETERS:

`\arraycolsep`
: half the width separating columns in an array environment

`\tabcolsep`
: half the width separating columns in a tabular environment

`\arrayrulewidth`
: width of rules

`\doublerulesep`
: space between adjacent rules in array or tabular

`\arraystretch`
: line spacing in array and tabular environments is done by placing a strut in every row of height and depth `\arraystretch` times the height and depth of the strut produced by an ordinary `\strut` command.

PREAMBLE:

The PREAMBLE argument of an array or tabular environment can contain the following:

- l,r,c : indicate where entry is to be placed.
- | : for vertical rule
- @{EXP} : inserts the text EXP in every column. `\arraycolsep` or `\tabcolsep` spacing is suppressed.
- *{N}{PRE} : equivalent to writing N copies of PRE in the preamble. PRE may contain *{N'}{EXP'} expressions.
- p{LEN} : makes entry in parbox of width LEN.

SPECIAL ARRAY COMMANDS:

\multicolumn{N}{FORMAT}{ITEM} : replaces the next N column items by ITEM, formatted according to FORMAT. FORMAT should contain at most one l,r or c. If it contains none, then ITEM is ignored.

\vline : draws a vertical line the height of the current row. May appear in an array element entry.

\hline : draws a horizontal line between rows. Must appear either before the first entry (to appear above the first row) or right after a \\ command. If followed by another \hline, then adds a \vskip of \doublerulesep.

\cline{i-j} : draws horizontal lines between rows covering columns i through j, inclusive. Multiple commands may follow one another to provide lines covering several disjoint columns

\extracolsep{WIDTH} : for use inside an @ in the preamble. Causes a WIDTH space to be added between columns for the rest of the columns. This is in addition to the ordinary intercolumn space.

```
\array ==
  BEGIN
    \@acol    == \@arrayacol
    \@classz  == \@arrayclassz
    \@classiv == \@arrayclassiv
    \\        == \@arraycr
    \@halignto == NULL
    \@tabarray
  END

\endarray{NAME} ==  BEGIN  \crcr }}  END

\tabular  ==
  BEGIN
    \@halignto == NULL
    \@tabular
  END

\tabular*{WIDTH} ==
  BEGIN
    \@halignto == to WIDTH
    \@tabular
  END

\@tabular ==
  BEGIN
    \leavevmode
    \hbox { $
       \@acol    == \@tabacol
```

```
      \@classz  == \@tabclassz
      \@classiv == \@tabclassiv
      \\        == \@tabularcr
      \@tabarray
  END

\endtabular == BEGIN \crcr}} $} END

\@tabarray == if next char = [ then \@array else \@array[c] fi

\@array[POS]{PREAMBLE} ==
  BEGIN
    define \@arstrutbox to make \@arstrut produce strut of height
      and depth \arraystretch times the height and
      depth of a normal strut.
    \@mkpream{PREAMBLE}
    \@preamble == \halign \@halignto {\tabskip=0pt\@arstrut
                            eval{\@preamble}\tabskip = 0pt\cr %%}
    \@startpbox == \@@startpbox
    \@endpbox == \@@endpbox
    if POS = t then \vtop
               else if POS = b then \vbox
                                    else \vcenter
    fi              fi
   {
    \par          ==L {} % changed 92/09/18
    \@sharp       == #
    \protect      == \relax
    \lineskip     :=L 0pt
    \baselineskip :=L 0pt
    \@preamble
  END


\@arraycr ==
 BEGIN
   $              %% Prevents extra space at end of row's last entry.
   if next char = [
    then  \@argarraycr
    else  $ \cr          %% Needed to balance $
 END

\@argarraycr[LENGTH] ==
 BEGIN
   $                    %% Needed to balance $ of \@arraycr
   if LENGTH > 0
     then  \@tempdima := depth of \@arstrutbox + LENGTH
           \vrule height 0pt width 0pt depth \@tempdima
           \cr
     else  \cr \noalign{\vskip LENGTH}
```

END

\@tabularcr and \@argtabularcr same as \@arraycr and \@argarraycr except without the extra $'s.

End of historical LaTeX 2.09 comments.

\extracolsep This command needs to expand during the tabular preamble construction so can't be robust.

```
\def\extracolsep#1{\tabskip #1\relax}
```

(*End of definition for* \extracolsep.)

array\array (environment)

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2026/06/01}%
⟨latexrelease⟩                  {\array}{Use real text vcenter}%
⟨*2ekernel | latexrelease⟩
\def\array{\let\@acol\@arrayacol \let\@classz\@arrayclassz
 \let\@classiv\@arrayclassiv
 \let\\\@arraycr\let\@halignto\@empty\m@th\@tabarray}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\array}{Use real text vcenter}%
⟨latexrelease⟩\def\array{\let\@acol\@arrayacol \let\@classz\@arrayclassz
⟨latexrelease⟩ \let\@classiv\@arrayclassiv
⟨latexrelease⟩ \let\\\@arraycr\let\@halignto\@empty\@tabarray}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* \array.)

\endarray
\endtabular
\endtabular*

```
\def\endarray{\crcr\egroup\egroup}
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2026/06/01}%
⟨latexrelease⟩                  {\endtabular}{Use real text vcenter}%
⟨*2ekernel | latexrelease⟩
\def\endtabular{\crcr\egroup\egroup\egroup}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\endtabular}{Use real text vcenter}%
⟨latexrelease⟩\def\endtabular{\crcr\egroup\egroup $\egroup}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
\expandafter \let \csname endtabular*\endcsname = \endtabular
```

(*End of definition for* \endarray, \endtabular, *and* \endtabular*.)

tabular\tabular (environment)

```
\def\tabular{\let\@halignto\@empty\@tabular}
```

(*End of definition for* \tabular.)

\tabular* Note that the change to use \setlength slightly alters the timing of the expansion and use of the length in #1 but this is very unlikely to have any practical effect.

```
\@namedef{tabular*}#1{%
 \setlength\dimen@{#1}%
   \edef\@halignto{to\the\dimen@}\@tabular}
```

(*End of definition for* \tabular*.)

\@tabular

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2026/06/01}%
⟨latexrelease⟩                  {\@tabular}{Use real text vcenter}%
⟨*2ekernel | latexrelease⟩
\def\@tabular{\leavevmode \hbox \bgroup \let\@acol\@tabacol
   \let\@classz\@tabclassz
   \let\@classiv\@tabclassiv \let\\\@tabularcr\@tabarray}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@tabular}{Use real text vcenter}%
⟨latexrelease⟩\def\@tabular{\leavevmode \hbox \bgroup $\let\@acol\@tabacol
⟨latexrelease⟩   \let\@classz\@tabclassz
⟨latexrelease⟩   \let\@classiv\@tabclassiv \let\\\@tabularcr\@tabarray}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* \@tabular.)

\@tabarray RmS 91/11/04 added \m@th.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2026/06/01}%
⟨latexrelease⟩                  {\@tabarray}{Use real text vcenter}%
⟨*2ekernel | latexrelease⟩
\def\@tabarray{\@ifnextchar[\@array{\@array[c]}}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@tabarray}{Use real text vcenter}%
⟨latexrelease⟩\def\@tabarray{\m@th\@ifnextchar[\@array{\@array[c]}}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* \@tabarray.)

RmS 1993/11/03 changed \halign to \ialign and removed superfluous \tabskip assignment

\@array

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2026/06/01}%
⟨latexrelease⟩                  {\@array}{Use real text vcenter}%
⟨*2ekernel | latexrelease⟩
\def\@array[#1]#2{%
  \if #1t\vtop \else \if#1b\vbox \else\vcenter@text \fi\fi
  \bgroup
```

This next bit of code sets up the strut and then builds the halign and its preamble according to the specification in the second argument.

This code has been moved inside the box. A side effect of this has been to expose what was a buglet in the previous version: since the `\@arstrut` below is expanded and contains an `\ifmmode` then it could produce an unnecessary extra box in every row, thus wasting 'lots of' main memory.

```
\setbox\@arstrutbox\hbox{%
  \vrule \@height\arraystretch\ht\strutbox
         \@depth\arraystretch \dp\strutbox
         \@width\z@}%
\@mkpream{#2}%
\edef\@preamble{%
  \ialign \noexpand\@halignto
    \bgroup \@arstrut \@preamble \tabskip\z@skip \cr}%
```

That is the end of setting up the preamble; now we reset things before executing the halign built-up in `\@preamble`. The restorations could be done by introducing an extra group, thus saving tokens.

```
\let\@startpbox\@@startpbox \let\@endpbox\@@endpbox
\let\tabularnewline\\%
  \def\par{\ifnum\currentgrouptype=6 \else\@@par\fi}%
  \let\@sharp##%
  \set@typeset@protect
  \lineskip\z@skip\baselineskip\z@skip
```

If the parsing of the preamble goes wrong there my be some characters left which TEX then tries to typeset, i.e., we would be in horizontal mode. That would produce an endless loop because the `\halign` expects vertical mode thus issues a `\par` but that is a no-op at this point. So we better test this case issue some error message and make a crude recovery by ending that horizontal mode with force. A better fix would be to ensure that we never pick up more than a single character token (not done).

```
    \ifhmode \@preamerr\z@ \@@par\fi
    \@preamble}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@array}{Use real text vcenter}%
⟨latexrelease⟩\def\@array[#1]#2{%
⟨latexrelease⟩  \if #1t\vtop \else \if#1b\vbox \else \vcenter \fi\fi
⟨latexrelease⟩  \bgroup
⟨latexrelease⟩  \setbox\@arstrutbox\hbox{%
⟨latexrelease⟩    \vrule \@height\arraystretch\ht\strutbox
⟨latexrelease⟩           \@depth\arraystretch \dp\strutbox
⟨latexrelease⟩           \@width\z@}%
⟨latexrelease⟩  \@mkpream{#2}%
⟨latexrelease⟩  \edef\@preamble{%
⟨latexrelease⟩    \ialign \noexpand\@halignto
⟨latexrelease⟩      \bgroup \@arstrut \@preamble \tabskip\z@skip \cr}%
⟨latexrelease⟩  \let\@startpbox\@@startpbox \let\@endpbox\@@endpbox
⟨latexrelease⟩  \let\tabularnewline\\%
⟨latexrelease⟩    \def\par{\ifnum\currentgrouptype=6 \else\@@par\fi}%
⟨latexrelease⟩    \let\@sharp##%
⟨latexrelease⟩    \set@typeset@protect
```

```
⟨latexrelease⟩     \lineskip\z@skip\baselineskip\z@skip
⟨latexrelease⟩     \ifhmode \@preamerr\z@ \@@par\fi
⟨latexrelease⟩     \@preamble}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@array`.)

`\@arraycr` Array version of `\\`.

```
\protected\def\@arraycr{%
  ${\ifnum0=‘}\fi\@ifstar\@xarraycr\@xarraycr}
```

(*End of definition for* `\@arraycr`.)

`\@arraycr`

```
\def\@xarraycr{\@ifnextchar[\@argarraycr{\ifnum0=‘{\fi}${}\cr}}
```

(*End of definition for* `\@arraycr`.)

`\@argarraycr`

```
\def\@argarraycr[#1]{%
  \ifnum0=‘{\fi}${}\ifdim #1>\z@ \@xargarraycr{#1}\else
   \@yargarraycr{#1}\fi}
```

(*End of definition for* `\@argarraycr`.)

`\tabularnewline` Tabular version of `\\`.

```
\let\tabularnewline\relax
```

(*End of definition for* `\tabularnewline`.)

`\@tabularcr`

```
\protected\def\@tabularcr{%
  {\ifnum0=‘}\fi\@ifstar\@xtabularcr\@xtabularcr}
```

(*End of definition for* `\@tabularcr`.)

`\@xtabularcr`

```
\def\@xtabularcr{\@ifnextchar[\@argtabularcr{\ifnum0=‘{\fi}\cr}}
```

(*End of definition for* `\@xtabularcr`.)

`\@argtabularcr`

```
\def\@argtabularcr[#1]{%
  \ifnum0=‘{\fi}%
    \ifdim #1>\z@
      \unskip\@xargarraycr{#1}%
    \else
      \@yargarraycr{#1}%
    \fi}
```

(*End of definition for* `\@argtabularcr`.)

`\@xargarraycr`

```
\def\@xargarraycr#1{\@tempdima #1\advance\@tempdima \dp \@arstrutbox
   \vrule \@height\z@ \@depth\@tempdima \@width\z@ \cr}
```

(*End of definition for* `\@xargarraycr`.)

`\@yargarraycr`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                    {\@yargarraycr}{tabular support calc syntax}%
\def\@yargarraycr#1{\cr\noalign{\@vspace@calcify{#1}}}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\@yargarraycr}{tabular support calc syntax}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@yargarraycr#1{\cr\noalign{\vskip #1}}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@yargarraycr`.)

`\multicolumn` *Historical LaTeX 2.09 comments (not necessarily accurate any more):*

```
\multicolumn{NUMBER}{FORMAT}{ITEM} ==
  BEGIN
  \multispan{NUMBER}
  \begingroup
  \@addamp == null
  \@mkpream{FORMAT}
  \@sharp == ITEM
  \protect == \relax
  \@startpbox == \@@startpbox
  \@endpbox == \@@endpbox
  \@arstrut
  \@preamble
  \endgroup
  END
```

End of historical LaTeX 2.09 comments.

The command `\def\@addamp{}` was removed from `\multicolumn` on 6 Dec 86 because it caused embedded array environments not to work. I think that it was included originally to prevent an error message if the 2nd argument to the `\multicolumn` command had two column specifiers.

8 Feb 89 — `\hbox{}` added after `\@preamble` to correct bug that occurred if `\multicolumn` preceded `\\[D]` with `D > 0`, caused by `\\[]` command doing an `\unskip`, which removed `\tabcolsep` glue inserted by `\multicolumn`.

This has been made long so that, for example, a `p`-column can contain multiple paragraphs; maybe the arguments of `@`-expressions should also be able to contain multiple paragraphs.

```
\long\def\multicolumn#1#2#3{\multispan{#1}\begingroup
  \@mkpream{#2}%
  \def\@sharp{#3}\set@typeset@protect
  \let\@startpbox\@@startpbox\let\@endpbox\@@endpbox
  \@arstrut \@preamble\hbox{}\endgroup\ignorespaces}
```

(*End of definition for* `\multicolumn`.)

Historical LaTeX 2.09 comments (not necessarily accurate any more):

Codes for classes and character numbers of array, tabular and multicolumn arguments.

Character	Class	Number
c	0	0
l	0	1
r	0	2
|	1	-
@	2	-
p	3	-
{@-exp}	4	-
{p-arg}	5	-

`\@testpach \foo` : expands `\foo`, which should be an array parameter token, and sets `\@chclass` and `\@chnum` to its class and number. Uses `\@lastchclass` to distinguish 4 and 5

Preamble error codes
0: 'illegal character'
1: 'Missing @-exp'
2: 'Missing p-arg'

```
\@addamp ==
  BEGIN if @firstamp = true then @firstamp := false
                                 else &                          fi
  END

\@mkpream TOKENLIST ==
  BEGIN
   @firstamp      := T
   \@lastchclass := 6
   \@preamble     == null
   \@sharp        == \relax
   \protect       == BEGIN \noexpand\protect\noexpand END
   \@startpbox    == \relax
   \@endpbox      == \relax
   \@expast{TOKENLIST}
   for \@nextchar := expand(\reserved@a)
     do  \@testpach{\@nextchar}
         case of \@chclass
           0 -> \@classz
           1 -> \@classi
              ...
           5 -> \@classv
         end case
         \@lastchclass := \@chclass
```

```
    od
    case of \@lastchclass
       0 -> \hskip \arraycolsep              % lrc
       1 ->                                   % |
       2 -> \@preamerr1 % 'Missing @-exp'     % @
       3 -> \@preamerr2 % 'Missing p-arg'     % p
       4 ->                                   % @-exp
       5 -> \hskip \arraycolsep               % p-exp
    end case
 END

\@arrayclassz ==
  BEGIN
    \@preamble := \@preamble *
              case of \@lastchclass
                 0 -> \hskip \arraycolsep \@addamp \hskip \arraycolsep
                 1 -> \@addamp \hskip \arraycolsep
                 2 ->  % impossible
                 3 ->  % impossible
                 4 -> \@addamp
                 5 -> \hskip \arraycolsep \@addamp \hskip \arraycolsep
                 6 -> \@addamp \hskip \arraycolsep
               end case
             * case of \@chnum
                  0 -> \hfil$\relax\@sharp$\hfil
                  1 -> $\relax\@sharp$\hfil
                  2 -> \hfil$\relax\@sharp$
               end case
  END

\@tabclassz == similar to \@arrayclassz

\@classi ==
 BEGIN
   \@preamble := \@preamble *
                 case of \@lastchclass
                    0 -> \hskip \arraycolsep \@arrayrule
                    1 -> \hskip \doublerulesep \@arrayrule
                    2 -> % impossible
                    3 -> % impossible
                    4 -> \@arrayrule
                    5 -> \hskip \arraycolsep \@arrayrule
                    6 -> \@arrayrule
                 end case
 END

\@classii ==
 BEGIN
   \@preamble := \@preamble *
                 case of \@lastchclass
```

```
                    0     ->
                    1     -> \hskip .5\arrayrulewidth
                    2     -> % impossible
                    else ->
                  end case
 END

\@classiii ==
 BEGIN
   \@preamble := \@preamble *
              case of \@lastchclass
                 0 -> \hskip \arraycolsep \@addamp \hskip \arraycolsep
                 1 -> \@addamp \hskip \arraycolsep
                 2 -> % impossible
                 3 -> % impossible
                 4 -> \@addamp
                 5 -> \hskip \arraycolsep \@addamp \hskip \arraycolsep
                 6 -> \@addamp \hskip \arraycolsep
               end case
 END

\@arrayclassiv  ==
     BEGIN  \@preamble := \@preamble * $ \@nextchar$  END

\@tabclassiv   == same as \@arrayclassv except without the $ ... $

\@classv ==
  BEGIN
   \@preamble :=
       \@preamble * \@startpbox{\@nextchar}\ignorespaces\@sharp
                                \@endpbox
  END

\@expast{S}:
 Sets \reserved@a := S with all instances of *{N}{STRING}
 replaced by N copies of STRING, where N > 0.  An *
 appearing inside braces is ignored, but *-expressions
 inside STRING are expanded, so nested *-expressions are
 handled properly.

\@expast{S} == BEGIN  \@xexpast S *0x\@@  END

\@xexpast S1 *{N}{S2} S3 \@@ ==
 BEGIN
   \reserved@a   := S1
   \@tempcnta := N
   if \@tempcnta > 0
     then  while \@tempcnta > 0 do \reserved@a   := \reserved@a S2
                                   \@tempcnta := \@tempcnta - 1 od
           \reserved@b == \@xexpast
```

```
      else   \reserved@b == \@xexnoop
    fi
    \expandafter \reserved@b \reserved@a S3 \@@
  END
```

End of historical LaTeX 2.09 comments.

```
\@xexnoop
```

```
\def\@xexnoop #1\@@{}
```

(*End of definition for* `\@xexnoop`.)

```
\@expast
```

```
\def\@expast#1{\@xexpast #1*0x\@@}
```

(*End of definition for* `\@expast`.)

```
\@xexpast
```

```
\def\@xexpast#1*#2#3#4\@@{%
  \edef\reserved@a{#1}%
  \@tempcnta#2\relax
  \ifnum\@tempcnta>\z@
    \@whilenum\@tempcnta>\z@\do
       {\edef\reserved@a{\reserved@a#3}\advance\@tempcnta \m@ne}%
    \let\reserved@b\@xexpast
  \else
    \let\reserved@b\@xexnoop
  \fi
  \expandafter\reserved@b\reserved@a #4\@@}
```

(*End of definition for* `\@xexpast`.)

```
\if@firstamp
   \@addamp
```

```
\newif\if@firstamp
```

```
\def\@addamp{%
  \if@firstamp
    \@firstampfalse
  \else
    \edef\@preamble{\@preamble &}%
  \fi}
```

(*End of definition for* `\if@firstamp` *and* `\@addamp`.)

```
 \@arrayacol
   \@tabacol
   \@ampacol
\@acolampacol
```

```
\def\@arrayacol{\edef\@preamble{\@preamble \hskip \arraycolsep}}
\def\@tabacol{\edef\@preamble{\@preamble \hskip \tabcolsep}}
\def\@ampacol{\@addamp \@acol}
\def\@acolampacol{\@acol\@addamp\@acol}
```

(*End of definition for* `\@arrayacol` *and others.*)

\@mkpream

```
\def\@mkpream#1{\@firstamptrue\@lastchclass6
  \let\@preamble\@empty
  \let\protect\@unexpandable@protect
  \let\@sharp\relax
  \let\@startpbox\relax\let\@endpbox\relax
  \@expast{#1}%
  \expandafter\@tfor \expandafter
    \@nextchar \expandafter:\expandafter=\reserved@a\do
       {\@testpach\@nextchar
    \ifcase \@chclass \@classz \or \@classi \or \@classii \or \@classiii
      \or \@classiv \or\@classv \fi\@lastchclass\@chclass}%
  \ifcase \@lastchclass \@acol
      \or \or \@preamerr \@ne\or \@preamerr \tw@\or \or \@acol \fi}
```

(*End of definition for* `\@mkpream`.)

\@arrayclassz

```
\def\@arrayclassz{\ifcase \@lastchclass \@acolampacol \or \@ampacol \or
   \or \or \@addamp \or
   \@acolampacol \or \@firstampfalse \@acol \fi
\edef\@preamble{\@preamble
  \ifcase \@chnum
     \hfil$\relax\@sharp$\hfil \or $\relax\@sharp$\hfil
    \or \hfil$\relax\@sharp$\fi}}
```

(*End of definition for* `\@arrayclassz`.)

\@tabclassz RmS 91/08/14 inserted extra braces around entry for NFSS

```
\def\@tabclassz{%
  \ifcase\@lastchclass
    \@acolampacol
  \or
    \@ampacol
  \or
  \or
  \or
    \@addamp
  \or
    \@acolampacol
  \or
    \@firstampfalse\@acol
  \fi
  \edef\@preamble{%
    \@preamble{%
      \ifcase\@chnum
        \hfil

        \hskip1sp%
        \ignorespaces\@sharp\unskip\hfil
      \or

        \hskip1sp\ignorespaces\@sharp\unskip\hfil
      \or
        \hfil\hskip1sp\ignorespaces\@sharp\unskip
      \fi}}}
```

(*End of definition for* `\@tabclassz`.)

`\@classi`

```
\def\@classi{%
  \ifcase\@lastchclass
    \@acol\@arrayrule
  \or
    \@addtopreamble{\hskip \doublerulesep}\@arrayrule
  \or
  \or
  \or
    \@arrayrule
  \or
    \@acol\@arrayrule
  \or
    \@arrayrule
  \fi}
```

(*End of definition for* `\@classi`.)

`\@classii`

```
\def\@classii{%
  \ifcase\@lastchclass
  \or
    \@addtopreamble{\hskip .5\arrayrulewidth}%
  \fi}
```

(*End of definition for* `\@classii`.)

`\@classiii`

```
\def\@classiii{\ifcase \@lastchclass \@acolampacol \or
   \@addamp\@acol \or
   \or \or \@addamp \or
   \@acolampacol \or \@ampacol \fi}
```

(*End of definition for* `\@classiii`.)

`\@tabclassiv`

```
\def\@tabclassiv{\@addtopreamble\@nextchar}
```

(*End of definition for* `\@tabclassiv`.)

`\@arrayclassiv`

```
\def\@arrayclassiv{\@addtopreamble{$\@nextchar$}}
```

(*End of definition for* `\@arrayclassiv`.)

`\@classv`

```
\def\@classv{\@addtopreamble{\@startpbox{\@nextchar}\ignorespaces
\@sharp\@endpbox}}
```

(*End of definition for* `\@classv`.)

`\@addtopreamble`

```
\def\@addtopreamble#1{\edef\@preamble{\@preamble #1}}
```

(*End of definition for* `\@addtopreamble`.)

`\@chclass`
`\@lastchclass`
`\@chnum`

```
\newcount\@chclass
\newcount\@lastchclass
\newcount\@chnum
```

(*End of definition for* `\@chclass`, `\@lastchclass`, *and* `\@chnum`.)

`\arraycolsep`
`\tabcolsep`
`\arrayrulewidth`
`\doublerulesep`

```
\newdimen\arraycolsep
\newdimen\tabcolsep
\newdimen\arrayrulewidth
\newdimen\doublerulesep
```

(*End of definition for* `\arraycolsep` *and others.*)

`\arraystretch`

```
\def\arraystretch{1}    % Default value.
```

(*End of definition for* `\arraystretch`.)

`\@arstrutbox`
`\@arstrut`

```
\newbox\@arstrutbox
\def\@arstrut{%
  \relax\ifmmode\copy\@arstrutbox\else\unhcopy\@arstrutbox\fi}
```

(*End of definition for* `\@arstrutbox` *and* `\@arstrut`.)

`\@arrayrule`

```
\def\@arrayrule{\@addtopreamble{\hskip -.5\arrayrulewidth
   \vrule \@width \arrayrulewidth\hskip -.5\arrayrulewidth}}
```

(*End of definition for* `\@arrayrule`.)

`\@testpach`

```
\def\@testpach#1{\@chclass \ifnum \@lastchclass=\tw@ 4 \else
    \ifnum \@lastchclass=3 5 \else
     \z@ \if #1c\@chnum \z@ \else
                               \if #1l\@chnum \@ne \else
                               \if #1r\@chnum \tw@ \else
          \@chclass \if #1|\@ne \else
                    \if #1@\tw@ \else
                    \if #1p3 \else \z@ \@preamerr 0\fi
  \fi  \fi  \fi  \fi  \fi  \fi
\fi}
```

(*End of definition for* `\@testpach`.)

`\hline`

```
\def\hline{%
  \noalign{\ifnum0=`}\fi\hrule \@height \arrayrulewidth \futurelet
   \reserved@a\@xhline}
```

(*End of definition for* `\hline`.)

\@xhline

```
\def\@xhline{\ifx\reserved@a\hline
               \vskip\doublerulesep
```

Measure from the middle of the rules.

```
               \vskip-\arrayrulewidth
             \fi
      \ifnum0=‘{\fi}}
```

(*End of definition for* `\@xhline`.)

\vline

```
\def\vline{\vrule \@width \arrayrulewidth}
```

(*End of definition for* `\vline`.)

\cline
\@cline

The old LaTeX2.09 implementation of `\cline` used up quite a lot of memory and two precious count registers. This new (1995/09/14) implementation does not use any count registers. It is coded in a way that depends heavily on the definition of `\multispan` so that command has been moved here from the file `ltplain.dtx`.

These counters are no longer declared.

```
\newcount\@cla
\newcount\@clb
```

```
\def\cline#1{\@cline#1\@nil}
```

```
\def\@cline#1-#2\@nil{%
  \omit
```

Use the counter from `\multispan`.

```
  \@multicnt#1%
  \advance\@multispan\m@ne
  \ifnum\@multicnt=\@ne\@firstofone{&\omit}\fi
  \@multicnt#2%
  \advance\@multicnt-#1%
  \advance\@multispan\@ne
```

The original had `\unskip` at this point, but how could a skip get here ???

```
  \leaders\hrule\@height\arrayrulewidth\hfill
  \cr
```

This is back spacing is fairly horrible, but it is what happened in the old version... An alternative would be to make `\cline` look ahead for a following `\cline` as does `\hline`. This would alter the spacing in existing documents so keep the old version in the kernel. Perhaps a package should do this differently.

```
  \noalign{\vskip-\arrayrulewidth}}
```

(*End of definition for* `\cline` *and* `\@cline`.)

\mscount

The `\mscount` counter is no longer declared, saving a csname and a register. It is declared in compatibility mode.

(*End of definition for* `\mscount`.)

\multispan
\@multispan
\sp@n

Modify \multispan slightly from its plain TEX definition to allow more efficient code sharing with \multicolumn. Also share a count register with \multiput.

```
\def\multispan{\omit\@multispan}
\def\@multispan#1{%
  \@multicnt#1\relax
  \loop\ifnum\@multicnt>\@ne \sp@n\repeat}
\def\sp@n{\span\omit\advance\@multicnt\m@ne}
```

(*End of definition for* \multispan, \@multispan, *and* \sp@n.)

\@startpbox
\@endpbox

Helper macros for 'p' columns.

\@startpbox{⟨*width*⟩} *text* \egroup is essentially \parbox{⟨*width*⟩}{⟨*text*⟩}

\@endpbox is essentially \unskip \strut \par \egroup\hfil (Changed 14 Jan 89) (changed again 1994/05/13)

```
\def\@startpbox#1{\vtop\bgroup \setlength\hsize{#1}\@arrayparboxrestore}
\def\@endpbox{\@finalstrut\@arstrutbox\par\egroup\hfil}
```

14 Jan 89: Def of \@endpbox changed from
\def\@endpbox{\par\vskip\dp\@arstrutbox\egroup\hfil}
so vertical spacing works out right if the last line of a 'p' entry has a descender.

(*End of definition for* \@startpbox *and* \@endpbox.)

\@@startpbox
\@@endpbox

```
\let\@@startpbox=\@startpbox
\let\@@endpbox=\@endpbox
```

(*End of definition for* \@@startpbox *and* \@@endpbox.)

1.3 Hooks for tabulars

Here we declare some hooks that for now are only going to be used in array and longtable. But the declarations are done in the kernel so that all packages can expect them to be declared and there is no need to check if they are already declared or not depending on package loading.

tbl/init (*hook*)

Extra initialization at the start of a tabular. It receives the number of columns as its first argument and the full tabular preamble as its second argument so that the code can inspect this data. This hook should not generate any output!

```
\NewHookWithArguments{tbl/init}{2}
```

tbl/row/init (*hook*)

This hook is invoked just before a row is about to start or could start, i.e., just before the first row and then after each row including the last. At this point tagging for the row has not been added and the hook can therefore only contain declarations (which have to to be global as everything is executed within a \noalign) and it can contain special commands such as \hline, \cline, or \rowcolor, that are allowed after \\.

```
\NewHookWithArguments{tbl/row/init}{1}
```

tbl/row/begin (*hook*)
tbl/row/end (*hook*)

Hooks run at the beginning and at the end of each row. The hooks receive the current row number as its argument. This hooks should not generate any output!

In fact, it is not quite clear if they are useful and if we don't find any good use cases they might vanish in the final release.

```
\NewMirroredHookPairWithArguments{tbl/row/begin}{tbl/row/end}{1}
```

`tbl/cell/init` (*hook*) Extra initialization at the start of each cell (horizontal or paragraph) in the tabular, directly in front of the tagging socket setting up cell tagging. It receives the current row and column numbers and the current cell type (`c`, `l`, ...) as arguments. This hook should not generate any output! One important use case for this hook is to manipulate the tagging setup for individual cells.

```
\NewHookWithArguments{tbl/cell/init}{3}
```

`tbl/cell/begin` (*hook*) Hooks run at the beginning and at the end of each cell. The hooks receive the current
`tbl/cell/end` (*hook*) row and column number and the cell type as arguments.

```
\NewMirroredHookPairWithArguments{tbl/cell/begin}{tbl/cell/end}{3}
```

```
⟨/2ekernel⟩
```

File 42
ltpictur.dtx

1 Picture Mode

Picture mode commands. In addition to the commands available in LaTeX2.09, This section adds the new `\qbezier` command for drawing curves.

`\qbezier` `\qbezier[`⟨*N*⟩`](`⟨*AX,AY*⟩`)(`⟨*BX,BY*⟩`)(`⟨*CX,CY*⟩`)` plots a quadratic Bezier curve from `(`⟨*AX,AY*⟩`)` to `(`⟨*CX,CY*⟩`)`, with `(`⟨*BX,BY*⟩`)` as the third Bezier point, using $N+1$ points equally spaced parametrically. If $N = 0$ (the default value), then a sufficient number of points are used to draw a connected curve–except that at most `\qbeziermax`+1 points are drawn. A "point" is a square of side `\@wholewidth`.

`\bezier` In addition, to be compatible with the old `bezier` package, a variant of this command, `\bezier`, is defined, in which the first argument is not optional.

Historical LaTeX 2.09 comments (not necessarily accurate any more):

```
\unitlength     = value of dimension argument
\@wholewidth    = current line width
\@halfwidth     = half of current line width
\@linefnt       = font for drawing lines
\@circlefnt     = font for drawing circles

\linethickness{DIM} : Sets the width of horizontal and vertical lines
    in a picture to DIM.  Does not change width of slanted lines
    or circles.   Width of all lines reset by \thinlines and
    \thicklines

\picture(XSIZE,YSIZE)(XORG,YORG)
  BEGIN
    \@picht :=L YSIZE * \unitlength
    box \@picbox :=
         \hb@xt@ XSIZE * \unitlength
           {\hskip -XORG * \unitlength
            \lower YORG * \unitlength
            \hbox{
            \ignorespaces     %% added 13 June 89
  END

\endpicture ==
  BEGIN
                  } \hss }
                  height of \@picbox := \@picht
                  depth  of \@picbox := 0
                  \mbox{\box\@picbox}   %% change 26 Aug 91
  END

\put(X, Y){OBJ} ==
  BEGIN
```

```
    \@killglue
    \raise Y * \unitlength  \hb@xt@ 0pt { \hskip X * \unitlength
                                          OBJ \hss             }
    \ignorespaces
  END

\multiput(X,Y)(DELX,DELY){N}{OBJ} ==
  BEGIN
   \@killglue
   \@multicnt := N
   \@xdim  := X * \unitlength
   \@ydim  := Y * \unitlength
   while \@multicnt > 0
     do \raise \@ydim \hb@xt@ 0pt { \hskip \@xdim
                                    OBJ \hss   }
        \@multicnt := \@multicnt - 1
        \@xdim     := \@xdim + DELX * \unitlength
        \@ydim     := \@ydim + DELY * \unitlength
     od
   \ignorespaces
  END
```

`\shortstack[POS]{TEXT}` : Makes a `\vbox` containing TEXT stacked as a one-column array, positioned l, r or c as indicated by POS.

End of historical LaTeX 2.09 comments.

The '2ekernel' code ensures that a `\usepackage{autopict}` is essentially ignored if a 'full' format is being used that has picture mode already in the format.

```
⟨2ekernel⟩\expandafter\let\csname ver@autopict.sty\endcsname\fmtversion
```

`\@wholewidth` `\@halfwidth`

```
⟨*2ekernel⟩
\newdimen\@wholewidth
\newdimen\@halfwidth
```

(*End of definition for* `\@wholewidth` *and* `\@halfwidth`.)

`\unitlength`

```
\newdimen\unitlength \unitlength =1pt
```

(*End of definition for* `\unitlength`.)

`\@picbox` `\@picht`

```
\newbox\@picbox
\newdimen\@picht
```

(*End of definition for* `\@picbox` *and* `\@picht`.)

\@defaultunitsset Set a length register, #1, accepting number or an etex length expression, #2, with default unit, #3.

The register name in #1 can be prefixed by \advance so that the register is incremented by the supplied value.

\@defaultunitsset{\advance\@vxx}{\textwidth-15pt}\unitlength

#3 can be a literal unit such as cm or a length register such as \unitlength.

This is used in all picture commands that take picture coordinates. So \put(2,2) as previously but now \put(\textwidth-5cm,0.4\texteight) Note that you can only use expressions with lengths, \put(1+2,0) is not supported.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\@defaultunitsset}{default units}%
\def\@defaultunitsset#1#2#3{%
  \@defaultunits#1\dimexpr#2#3\relax\relax\@nnil}
⟨/2ekernel | latexrelease⟩

⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@defaultunitsset}{default units}%
⟨latexrelease⟩\let\@defaultunitsset\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* \@defaultunitsset.)

\picture #1 should be white space.

#1 should be a ((eating any white space before the bracket),

\pictur@

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2026/06/01}%
⟨latexrelease⟩                  {\pictur@}{optional arg}
\newcommand\picture[2][]{\pictur@#2}
⟨/2ekernel | latexrelease⟩

⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\pictur@}{optional arg}%
⟨latexrelease⟩\long\def\picture#1{\pictur@#1}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
\def\pictur@(#1){%
  \@ifnextchar({\@picture(#1)}{\@picture(#1)(0,0)}}
```

(*End of definition for* \picture *and* \pictur@.)

\@picture

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\@picture}{default units}%
\def\@picture(#1,#2)(#3,#4){%
  \@defaultunitsset\@picht{#2}\unitlength
  \@defaultunitsset\@tempdimc{#1}\unitlength
```

```
  \setbox\@picbox\hb@xt@\@tempdimc\bgroup
    \@defaultunitsset\@tempdimc{#3}\unitlength
    \hskip -\@tempdimc
    \@defaultunitsset\@tempdimc{#4}\unitlength
    \lower\@tempdimc\hbox\bgroup
      \ignorespaces}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@picture}{default units}%
⟨latexrelease⟩\def\@picture(#1,#2)(#3,#4){%
⟨latexrelease⟩  \@picht#2\unitlength
⟨latexrelease⟩  \setbox\@picbox\hb@xt@#1\unitlength\bgroup
⟨latexrelease⟩    \hskip -#3\unitlength
⟨latexrelease⟩    \lower #4\unitlength\hbox\bgroup
⟨latexrelease⟩      \ignorespaces}
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\@picture`.)

`\endpicture`

```
\def\endpicture{%
  \egroup\hss\egroup
    \ht\@picbox\@picht\dp\@picbox\z@
    \mbox{\box\@picbox}}
```

(*End of definition for* `\endpicture`.)

In the definitions of `\put` and `\multiput`, `\hskip` was replaced by `\kern` just in case arg `#3` = "plus". (Bug detected by Don Knuth. changed 20 Jul 87).

```
⟨/2ekernel⟩
⟨∗2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\put}{default units}%
⟨latexrelease⟩\expandafter\let\csname put \endcsname\@undefind
\long\def\put(#1,#2)#3{%
  \@killglue
  \@defaultunitsset\@tempdimc{#2}\unitlength
  \raise\@tempdimc
  \hb@xt@\z@{%
    \@defaultunitsset\@tempdimc{#1}\unitlength
    \kern\@tempdimc
    #3\hss}%
  \ignorespaces}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\put}{default units}%
⟨latexrelease⟩\expandafter\let\csname put \endcsname\@undefind
⟨latexrelease⟩\long\def\put(#1,#2)#3{%
⟨latexrelease⟩  \@killglue\raise#2\unitlength
⟨latexrelease⟩  \hb@xt@\z@{\kern#1\unitlength #3\hss}%
```

```
⟨latexrelease⟩  \ignorespaces}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

`\multiput` `#3` had better be a `(`.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\multiput}{default units}%
⟨latexrelease⟩\expandafter\let\csname multiput \endcsname\@undefind
\def\multiput(#1,#2)#3{%
  \@defaultunitsset\@xdim{#1}\unitlength
  \@defaultunitsset\@ydim{#2}\unitlength
   \@multiput(}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\multiput}{default units}%
⟨latexrelease⟩\expandafter\let\csname multiput \endcsname\@undefind
⟨latexrelease⟩\def\multiput(#1,#2)#3{%
⟨latexrelease⟩  \@xdim #1\unitlength
⟨latexrelease⟩  \@ydim #2\unitlength
⟨latexrelease⟩   \@multiput(}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\multiput`.)

`\@multiput`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\@multiput}{default units}%
\long\def\@multiput(#1,#2)#3#4{%
  \@killglue\@multicnt #3\relax
  \@whilenum \@multicnt >\z@\do
    {\raise\@ydim\hb@xt@\z@{\kern\@xdim #4\hss}%
     \advance\@multicnt\m@ne
     \@defaultunitsset{\advance\@xdim}{#1}\unitlength
     \@defaultunitsset{\advance\@ydim}{#2}\unitlength}%
  \ignorespaces}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@multiput}{default units}%
⟨latexrelease⟩\long\def\@multiput(#1,#2)#3#4{%
⟨latexrelease⟩  \@killglue\@multicnt #3\relax
⟨latexrelease⟩  \@whilenum \@multicnt >\z@\do
⟨latexrelease⟩    {\raise\@ydim\hb@xt@\z@{\kern\@xdim #4\hss}%
⟨latexrelease⟩     \advance\@multicnt\m@ne
⟨latexrelease⟩     \advance\@xdim#1\unitlength\advance\@ydim#2\unitlength}%
⟨latexrelease⟩  \ignorespaces}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@multiput`.)

`\@killglue`

```
\def\@killglue{\unskip\@whiledim \lastskip >\z@\do{\unskip}}
```

(*End of definition for* `\@killglue`.)

`\thinlines`
`\thicklines`

```
\DeclareRobustCommand\thinlines{\let\@linefnt\tenln
  \let\@circlefnt\tencirc
  \@wholewidth\fontdimen8\tenln \@halfwidth .5\@wholewidth}
\DeclareRobustCommand\thicklines{\let\@linefnt\tenlnw
  \let\@circlefnt\tencircw
  \@wholewidth\fontdimen8\tenlnw \@halfwidth .5\@wholewidth}
```

(*End of definition for* `\thinlines` *and* `\thicklines`.)

`\linethickness`

```
\DeclareRobustCommand*\linethickness[1]
   {\@wholewidth #1\relax \@halfwidth .5\@wholewidth \ignorespaces}
```

(*End of definition for* `\linethickness`.)

`\ishortstack`

```
\def\shortstack{\@ifnextchar[\@shortstack{\@shortstack[c]}}
```

(*End of definition for* `\ishortstack`.)

`\@ishortstack`

```
\def\@shortstack[#1]{%
  \leavevmode
  \vbox\bgroup
    \baselineskip-\p@\lineskip 3\p@
    \let\mb@l\hss\let\mb@r\hss
    \expandafter\let\csname mb@#1\endcsname\relax
    \let\\\@stackcr
    \@ishortstack}
```

(*End of definition for* `\@ishortstack`.)

`\@ishortstack`

```
\def\@ishortstack#1{\ialign{\mb@l {##}\unskip\mb@r\cr #1\crcr}\egroup}
```

(*End of definition for* `\@ishortstack`.)

`\@stackcr`
`\@ixstackcr`

```
\protected\def\@stackcr{\@ifstar\@ixstackcr\@ixstackcr}
\def\@ixstackcr{\@ifnextchar[\@istackcr{\cr\ignorespaces}}
```

(*End of definition for* `\@stackcr` *and* `\@ixstackcr`.)

\@istackcr

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                    {\@istackcr}{\shortstack calc support}%
\def\@istackcr[#1]{\cr\noalign{\@vspace@calcify{#1}}\ignorespaces}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\@istackcr}{\shortstack calc support}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@istackcr[#1]{\cr\noalign{\vskip #1}\ignorespaces}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@istackcr`.)

Historical LaTeX 2.09 comments (not necessarily accurate any more):

```
\line(X,Y){LEN} ==
BEGIN
 \@xarg    := X
 \@yarg    := Y
 \@linelen := LEN * \unitlength
 if \@xarg = 0
    then \@vline
    else if \@yarg = 0
            then \@hline
            else \@sline
         if
 if
END

\@sline ==
 BEGIN
   if \@xarg < 0
     then @negarg := T
          \@xarg  := -\@xarg
          \@yyarg := -\@yarg
     else @negarg := F
          \@yyarg := \@yarg
   fi
   \@tempcnta := |\@yyarg|
   if \@tempcnta > 6
     then error: 'LATEX ERROR: Illegal \line or \vector argument.'
          \@tempcnta := 0
   fi
   \box\@linechar := \hbox{\@linefnt \@getlinechar(\@xarg,\@yyarg) }
    if \@yarg > 0 then \@upordown = \raise
                       \@clnht := 0
                  else \@upordown = \lower
                       \@clnht := height of \box\@linechar
```

```
    fi
    \@clnwd  := width of \box\@linechar
    if @negarg
      then \hskip - width of \box\@linechar
           \reserved@a == \hskip - 2* width of box \@linechar
      else \reserved@a == \relax
    fi
 %% Put out integral number of line segments
    while \@clnwd <  \@linelen
      do  \@upordown \@clnht \copy\@linechar
          \reserved@a
          \@clnht := \@clnht + ht of \box\@linechar
          \@clnwd := \@clnwd + width of \box\@linechar
      od

 %% Put out last segment
    \@clnht := \@clnht - height of \box\@linechar
    \@clnwd := \@clnwd - width of \box\@linechar
    \@tempdima   := \@linelen - \@clnwd
    \@tempdimb   := \@tempdima - width of \box\@linechar
    if @negarg  then \hskip -\@tempdimb
                else \hskip  \@tempdimb
    fi
    \@tempdima   := 1000 * \@tempdima
    \@tempcnta   := \@tempdima / width of \box\@linechar
    \@tempdima   := (\@tempcnta * ht of \box\@linechar)/1000
    \@clnht := \@clnht + \@tempdima
    if \@linelen < width of box\@linechar
        then \hskip width of box\@linechar
        else \hbox{\@upordown \@clnht \copy\@linechar}
    fi
END

\@hline ==
  BEGIN
    if \@xarg < 0 then  \hskip -\@linelen \fi
    \vrule height \@halfwidth depth \@halfwidth width \@linelen
    if \@xarg < 0 then  \hskip -\@linelen \fi
 END

\@vline == if \@yarg < 0 \@downline else \@upline  fi


\@getlinechar(X,Y) ==
  BEGIN
    \@tempcnta := 8*X - 9
    if Y > 0
      then \@tempcnta := \@tempcnta + Y
      else \@tempcnta := \@tempcnta - Y + 64
    fi
```

```
    \char\@tempcnta
  END

\vector(X,Y){LEN} ==
BEGIN
 \@xarg    := X
 \@yarg    := Y
 \@linelen := LEN * \unitlength
 if \@xarg = 0
    then \@vvector
    else if \@yarg = 0
           then \@hvector
           else \@svector
         if
 if
END

\@hvector ==
  BEGIN
    \@hline
    {\@linefnt if \@xarg < 0 then  \@getlarrow(1,0)
                             else  \@getrarrow(1,0)
               fi}
  END

\@vvector == if \@yarg < 0 \@downvector else \@upvector  fi

\@svector ==
 BEGIN
  \@sline
  \@tempcnta := |\@yarg|
    if  \@tempcnta < 5
        then  \hskip - width of \box\@linechar
              \@upordown \@clnht \hbox
                        {\@linefnt
                         if @negarg then \@getlarrow(\@xarg,\@yyarg)
                                    else \@getrarrow(\@xarg,\@yyarg)
                         fi }
        else  error: 'LATEX ERROR: Illegal \line or \vector argument.'
    fi
 END

\@getlarrow(X,Y) ==
 BEGIN
  if Y = 0
    then \@tempcnta := '33
    else \@tempcnta := 16 * X  -  9
         \@tempcntb := 2 * Y
         if \@tempcntb > 0
           then  \@tempcnta := \@tempcnta  +  \@tempcntb
```

```
                else  \@tempcnta := \@tempcnta  -  \@tempcntb +  64
              fi
    fi
    \char\@tempcnta
   END

  \@getrarrow(X,Y) ==
   BEGIN
    \@tempcntb := |Y|
    case of \@tempcntb
      0 : \@tempcnta := '55
      1 : if X < 3
            then \@tempcnta :=  24*X - 6
            else if X = 3
                   then \@tempcnta := 49
                   else \@tempcnta := 58  fi
          fi
      2 : if X < 3
            then \@tempcnta :=  24*X - 3
            else \@tempcnta := 51      % X must = 3
          fi
      3 : \@tempcnta := 16*X - 2
      4 : \@tempcnta := 16*X + 7
    endcase
    if Y < 0
      then \@tempcnta := \@tempcnta + 64
    fi
    \char\@tempcnta
   END
```

End of historical LaTeX 2.09 comments.

`\if@negarg`

```
\newif\if@negarg
```

(*End of definition for* `\if@negarg`.)

`\line`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                    {\line}{default units}%
⟨latexrelease⟩\expandafter\let\csname line \endcsname\@undefind
\def\line(#1,#2)#3{\@xarg #1\relax \@yarg #2\relax
  \@defaultunitsset\@linelen{#3}\unitlength
  \ifdim\@linelen<\z@\@badlinearg\else
    \ifnum\@xarg =\z@ \@vline
      \else \ifnum\@yarg =\z@ \@hline \else \@sline\fi
    \fi
  \fi}
⟨/2ekernel | latexrelease⟩
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\line}{default units}%
⟨latexrelease⟩\expandafter\let\csname line \endcsname\@undefind
⟨latexrelease⟩\def\line(#1,#2)#3{\@xarg #1\relax \@yarg #2\relax
⟨latexrelease⟩  \@linelen #3\unitlength
⟨latexrelease⟩  \ifdim\@linelen<\z@\@badlinearg\else
⟨latexrelease⟩    \ifnum\@xarg =\z@ \@vline
⟨latexrelease⟩      \else \ifnum\@yarg =\z@ \@hline \else \@sline\fi
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \fi}
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\line`.)

`\@sline`

```
\def\@sline{%
  \ifnum\@xarg<\z@ \@negargtrue \@xarg -\@xarg \@yyarg -\@yarg
  \else \@negargfalse \@yyarg \@yarg \fi
\ifnum \@yyarg >\z@ \@tempcnta\@yyarg \else \@tempcnta -\@yyarg \fi
\ifnum\@tempcnta>6 \@badlinearg\@tempcnta\z@ \fi
\ifnum\@xarg>6 \@badlinearg\@xarg \@ne \fi
\setbox\@linechar\hbox{\@linefnt\@getlinechar(\@xarg,\@yyarg)}%
```

If we have something like `\line(5,5){30}` the `\@linechar` will not contain a char and later on we will end in an infinite loop. So we check the width of the box and put in something as an emergency fix if necessary.

```
\ifdim\wd\@linechar=\z@
   \setbox\@linechar\hbox{.}%
   \@badlinearg
\fi
\ifnum \@yarg >\z@ \let\@upordown\raise \@clnht\z@
   \else\let\@upordown\lower \@clnht \ht\@linechar\fi
\@clnwd \wd\@linechar
\if@negarg
  \hskip -\wd\@linechar \def\reserved@a{\hskip -2\wd\@linechar}%
\else
     \let\reserved@a\relax
\fi
\@whiledim \@clnwd <\@linelen \do
  {\@upordown\@clnht\copy\@linechar
   \reserved@a
   \advance\@clnht \ht\@linechar
   \advance\@clnwd \wd\@linechar}%
\advance\@clnht -\ht\@linechar
\advance\@clnwd -\wd\@linechar
\@tempdima\@linelen\advance\@tempdima -\@clnwd
\@tempdimb\@tempdima\advance\@tempdimb -\wd\@linechar
\if@negarg \hskip -\@tempdimb \else \hskip \@tempdimb \fi
\multiply\@tempdima \@m
\@tempcnta \@tempdima
\@tempdima \wd\@linechar \divide\@tempcnta \@tempdima
\@tempdima \ht\@linechar \multiply\@tempdima \@tempcnta
\divide\@tempdima \@m
```

```
\advance\@clnht \@tempdima
\ifdim \@linelen <\wd\@linechar
   \hskip \wd\@linechar
```

Warn if line gets so short that it can't be printed. But don't warn if it is exactly zero since that was probably deliberate (e.g., to get a vector head only).

```
   \ifdim \@linelen = \z@
   \else
     \@picture@warn
   \fi
   \else\@upordown\@clnht\copy\@linechar\fi}
```

(*End of definition for* `\@sline`.)

`\@hline`

```
\def\@hline{\ifnum \@xarg <\z@ \hskip -\@linelen \fi
\vrule \@height \@halfwidth \@depth \@halfwidth \@width \@linelen
\ifnum \@xarg <\z@ \hskip -\@linelen \fi}
```

(*End of definition for* `\@hline`.)

`\@getlinechar`

```
\def\@getlinechar(#1,#2){\@tempcnta#1\relax\multiply\@tempcnta 8%
  \advance\@tempcnta -9\ifnum #2>\z@ \advance\@tempcnta #2\relax\else
  \advance\@tempcnta -#2\relax\advance\@tempcnta 64 \fi
  \char\@tempcnta}
```

(*End of definition for* `\@getlinechar`.)

`\vector`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\vector}{default units}%
⟨latexrelease⟩\expandafter\let\csname vector \endcsname\@undefind
\def\vector(#1,#2)#3{\@xarg #1\relax \@yarg #2\relax
  \@tempcnta \ifnum\@xarg<\z@ -\@xarg\else\@xarg\fi
  \ifnum\@tempcnta<5\relax
  \@defaultunitsset\@linelen{#3}\unitlength
  \ifdim\@linelen<\z@\@badlinearg\else
    \ifnum\@xarg =\z@ \@vvector
      \else \ifnum\@yarg =\z@ \@hvector \else \@svector\fi
    \fi
  \fi
  \else\@badlinearg\fi}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\vector}{default units}%
⟨latexrelease⟩\expandafter\let\csname vector \endcsname\@undefind
⟨latexrelease⟩\def\vector(#1,#2)#3{\@xarg #1\relax \@yarg #2\relax
⟨latexrelease⟩  \@tempcnta \ifnum\@xarg<\z@ -\@xarg\else\@xarg\fi
⟨latexrelease⟩  \ifnum\@tempcnta<5\relax
⟨latexrelease⟩  \@linelen #3\unitlength
⟨latexrelease⟩  \ifdim\@linelen<\z@\@badlinearg\else
```

```
⟨latexrelease⟩     \ifnum\@xarg =\z@ \@vvector
⟨latexrelease⟩       \else \ifnum\@yarg =\z@ \@hvector \else \@svector\fi
⟨latexrelease⟩     \fi
⟨latexrelease⟩   \fi
⟨latexrelease⟩   \else\@badlinearg\fi}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\vector`.)

`\@hvector`

```
\def\@hvector{\@hline\hb@xt@\z@{\@linefnt
 \ifnum \@xarg <\z@ \@getlarrow(1,0)\hss\else
    \hss\@getrarrow(1,0)\fi}}
```

(*End of definition for* `\@hvector`.)

`\@vvector`

```
\def\@vvector{\ifnum \@yarg <\z@ \@downvector \else \@upvector \fi}
```

(*End of definition for* `\@vvector`.)

`\@svector`

```
\def\@svector{\@sline
  \@tempcnta\@yarg \ifnum\@tempcnta <\z@ \@tempcnta -\@tempcnta\fi
  \ifnum\@tempcnta <5%
    \hskip -\wd\@linechar
    \@upordown\@clnht \hbox{\@linefnt  \if@negarg
    \@getlarrow(\@xarg,\@yyarg)\else \@getrarrow(\@xarg,\@yyarg)\fi}%
  \else\@badlinearg\fi}
```

(*End of definition for* `\@svector`.)

`\@getlarrow`

```
\def\@getlarrow(#1,#2){\ifnum #2=\z@ \@tempcnta 27 % ’33
  \else
  \@tempcnta #1\relax\multiply\@tempcnta \sixt@@n
  \advance\@tempcnta -9 \@tempcntb #2\relax\multiply\@tempcntb \tw@
  \ifnum \@tempcntb >\z@ \advance\@tempcnta \@tempcntb
  \else\advance\@tempcnta -\@tempcntb\advance\@tempcnta 64
  \fi\fi\char\@tempcnta}
```

(*End of definition for* `\@getlarrow`.)

`\@getrarrow`

```
\def\@getrarrow(#1,#2){\@tempcntb #2\relax
\ifnum\@tempcntb <\z@ \@tempcntb -\@tempcntb\relax\fi
\ifcase \@tempcntb\relax \@tempcnta 45  % ’55
\or
\ifnum #1<\thr@@ \@tempcnta #1\relax\multiply\@tempcnta
24\advance\@tempcnta -6 \else \ifnum #1=\thr@@ \@tempcnta 49
\else\@tempcnta 58 \fi\fi\or
\ifnum #1<\thr@@ \@tempcnta=#1\relax\multiply\@tempcnta
24\advance\@tempcnta -\thr@@ \else \@tempcnta 51 \fi\or
\@tempcnta #1\relax\multiply\@tempcnta
```

```
\sixt@@n \advance\@tempcnta -\tw@ \else
\@tempcnta #1\relax\multiply\@tempcnta
\sixt@@n \advance\@tempcnta 7 \fi\ifnum #2<\z@ \advance\@tempcnta 64 \fi
\char\@tempcnta}
```

(*End of definition for* `\@getrarrow`.)

`\@vline`

```
\def\@vline{\ifnum \@yarg <\z@ \@downline \else \@upline\fi}
```

(*End of definition for* `\@vline`.)

`\@upline`

```
\def\@upline{%
  \hb@xt@\z@{\hskip -\@halfwidth \vrule \@width \@wholewidth
   \@height \@linelen \@depth \z@\hss}}
```

(*End of definition for* `\@upline`.)

`\@downline`

```
\def\@downline{%
  \hb@xt@\z@{\hskip -\@halfwidth \vrule \@width \@wholewidth
   \@height \z@ \@depth \@linelen \hss}}
```

(*End of definition for* `\@downline`.)

`\@upvector`

```
\def\@upvector{\@upline\setbox\@tempboxa\hbox{\@linefnt\char 54}% '66
  \raise \@linelen \hb@xt@\z@{\lower \ht\@tempboxa\box\@tempboxa\hss}}
```

(*End of definition for* `\@upvector`.)

`\@downvector`

```
\def\@downvector{\@downline\lower \@linelen
      \hb@xt@\z@{\@linefnt\char 63  % '77
      \hss}}
```

(*End of definition for* `\@downvector`.)

Historical LaTeX 2.09 comments (not necessarily accurate any more):

```
\dashbox{D}(X,Y) ==
  BEGIN
  leave vertical mode
  \hb@xt@ 0pt {
        \baselineskip := 0pt
        \lineskip     := 0pt
  %% HORIZONTAL DASHES
        \@dashdim := X * \unitlength
        \@dashcnt := \@dashdim + 200 % to prevent roundoff error
        \@dashdim := D * \unitlength
        \@dashcnt := \@dashcnt / \@dashdim
        if \@dashcnt is odd
          then \@dashdim := 0pt
               \@dashcnt := (\@dashcnt + 1) / 2
          else \@dashdim := \@dashdim / 2
```

```
        \@dashcnt := \@dashcnt / 2 - 1
        \box\@dashbox    := \hbox{\vrule height \@halfwidth
                                  depth \@halfwidth width \@dashdim}
        \put(0,0){\copy\@dashbox}
        \put(0,Y){\copy\@dashbox}
        \put(X,0){\hskip -\@dashdim\copy\@dashbox}
        \put(X,Y){\hskip -\@dashdim\box\@dashbox}
        \@dashdim := 3 * \@dashdim
    fi
    \box\@dashbox := \hbox{\vrule height \@halfwidth
                           depth \@halfwidth width D * \unitlength
                           \hskip D * \unitlength}
    \@tempcnta := 0
    \put(0,0){\hskip \@dashdim
              while \@tempcnta < \@dascnt
                do \copy\@dashbox
                   \@tempcnta := \@tempcnta + 1
                od
              }
    \@tempcnta := 0
    put(0,Y){\hskip \@dashdim
              while \@tempcnta < \@dascnt
                do \copy\@dashbox
                   \@tempcnta := \@tempcnta + 1
                od
              }

%% vertical dashes
    \@dashdim := Y * \unitlength
    \@dashcnt := \@dashdim + 200 % to prevent roundoff error
    \@dashdim := D * \unitlength
    \@dashcnt := \@dashcnt / \@dashdim
    if \@dashcnt is odd
      then \@dashdim := 0pt
           \@dashcnt := (\@dashcnt + 1) / 2
      else \@dashdim := \@dashdim / 2
           \@dashcnt := \@dashcnt / 2 - 1
           \box\@dashbox    := \hbox{\hskip -\@halfwidth
                                     \vrule width \@wholewidth
                                              height \@dashdim  }
           \put(0,0){\copy\@dashbox}
           \put(X,0){\copy\@dashbox}
           \put(0,Y){\lower\@dashdim\copy\@dashbox}
           \put(X,Y){\lower\@dashdim\copy\@dashbox}
           \@dashdim := 3 * \@dashdim
    fi
    \box\@dashbox := \hbox{\vrule width \@wholewidth
                              height D * \unitlength       }
    \@tempcnta := 0
    put(0,0){\hskip -\halfwidth
```

```
                \vbox{while \@tempcnta < \@dashcnt
                       do \vskip D*\unitlength
                          \copy\@dashbox
                          \@tempcnta := \@tempcnta + 1
                       od
                      \vskip \@dashdim
                     } }
          \@tempcnta := 0
          put(X,0){\hskip -\halfwidth
                \vbox{while \@tempcnta < \@dashcnt
                       do \vskip D*\unitlength
                          \copy\@dashbox
                          \@tempcnta := \@tempcnta + 1
                       od
                      \vskip \@dashdim
                     }
                }
     }     % END DASHES

   \@imakepicbox(X,Y)
  END
```

End of historical LaTeX 2.09 comments.

\dashbox

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\dashbox}{default units}%
⟨latexrelease⟩\expandafter\let\csname dashbox \endcsname\@undefind
\def\dashbox#1(#2,#3){\leavevmode\hb@xt@\z@{\baselineskip \z@skip
\lineskip \z@skip
\@defaultunitsset\@dashdim{#2}\unitlength
\@dashcnt \@dashdim \advance\@dashcnt 200
\@defaultunitsset\@dashdim{#1}\unitlength
\divide\@dashcnt \@dashdim
\ifodd\@dashcnt\@dashdim \z@
\advance\@dashcnt \@ne \divide\@dashcnt \tw@
\else \divide\@dashdim \tw@ \divide\@dashcnt \tw@
\advance\@dashcnt \m@ne
\setbox\@dashbox \hbox{\vrule \@height \@halfwidth \@depth \@halfwidth
\@width \@dashdim}\put(0,0){\copy\@dashbox}%
\put(0,#3){\copy\@dashbox}%
\put(#2,0){\hskip-\@dashdim\copy\@dashbox}%
\put(#2,#3){\hskip-\@dashdim\box\@dashbox}%
\multiply\@dashdim \thr@@
\fi
\setbox\@dashbox \hbox{%
  \@defaultunitsset\@tempdimc{#1}\unitlength
  \vrule \@height \@halfwidth \@depth \@halfwidth \@width \@tempdimc
  \hskip\@tempdimc}%
\@tempcnta\z@
\put(0,0){\hskip\@dashdim \@whilenum \@tempcnta <\@dashcnt
```

```
\do{\copy\@dashbox\advance\@tempcnta \@ne }}\@tempcnta\z@
\put(0,#3){\hskip\@dashdim \@whilenum \@tempcnta <\@dashcnt
\do{\copy\@dashbox\advance\@tempcnta \@ne }}%
\@defaultunitsset\@dashdim{#3}\unitlength
\@dashcnt \@dashdim \advance\@dashcnt 200
\@defaultunitsset\@dashdim{#1}\unitlength
\divide\@dashcnt \@dashdim
\ifodd\@dashcnt \@dashdim \z@
\advance\@dashcnt \@ne \divide\@dashcnt \tw@
\else
\divide\@dashdim \tw@ \divide\@dashcnt \tw@
\advance\@dashcnt \m@ne
\setbox\@dashbox\hbox{\hskip -\@halfwidth
\vrule \@width \@wholewidth
\@height \@dashdim}\put(0,0){\copy\@dashbox}%
\put(#2,0){\copy\@dashbox}%
\put(0,#3){\lower\@dashdim\copy\@dashbox}%
\put(#2,#3){\lower\@dashdim\copy\@dashbox}%
\multiply\@dashdim \thr@@
\fi
\@defaultunitsset\@tempdimb{#1}\unitlength
\setbox\@dashbox\hbox{%
  \vrule \@width \@wholewidth \@height\@tempdimb}%
\@tempcnta\z@
\put(0,0){\hskip -\@halfwidth \vbox{\@whilenum \@tempcnta <\@dashcnt
\do{\vskip\@tempdimb\copy\@dashbox\advance\@tempcnta \@ne }%
\vskip\@dashdim}}\@tempcnta\z@
\put(#2,0){\hskip -\@halfwidth \vbox{\@whilenum \@tempcnta<\@dashcnt
\do{\vskip\@tempdimb\copy\@dashbox\advance\@tempcnta \@ne }%
\vskip\@dashdim}}\@makepicbox(#2,#3)}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\dashbox}{default units}%
⟨latexrelease⟩\expandafter\let\csname dashbox \endcsname\@undefind
⟨latexrelease⟩\def\dashbox#1(#2,#3){%
⟨latexrelease⟩\leavevmode\hb@xt@\z@{\baselineskip \z@skip
⟨latexrelease⟩\lineskip \z@skip
⟨latexrelease⟩\@dashdim #2\unitlength
⟨latexrelease⟩\@dashcnt \@dashdim \advance\@dashcnt 200
⟨latexrelease⟩\@dashdim #1\unitlength\divide\@dashcnt \@dashdim
⟨latexrelease⟩\ifodd\@dashcnt\@dashdim \z@
⟨latexrelease⟩\advance\@dashcnt \@ne \divide\@dashcnt \tw@
⟨latexrelease⟩\else \divide\@dashdim \tw@ \divide\@dashcnt \tw@
⟨latexrelease⟩\advance\@dashcnt \m@ne
⟨latexrelease⟩\setbox\@dashbox \hbox{%
⟨latexrelease⟩  \vrule \@height \@halfwidth \@depth \@halfwidth
⟨latexrelease⟩  \@width \@dashdim}\put(0,0){\copy\@dashbox}%
⟨latexrelease⟩\put(0,#3){\copy\@dashbox}%
⟨latexrelease⟩\put(#2,0){\hskip-\@dashdim\copy\@dashbox}%
⟨latexrelease⟩\put(#2,#3){\hskip-\@dashdim\box\@dashbox}%
⟨latexrelease⟩\multiply\@dashdim \thr@@
⟨latexrelease⟩\fi
⟨latexrelease⟩\setbox\@dashbox \hbox{%
```

```
⟨latexrelease⟩  \vrule \@height \@halfwidth \@depth \@halfwidth
⟨latexrelease⟩  \@width #1\unitlength\hskip #1\unitlength}\@tempcnta\z@
⟨latexrelease⟩\put(0,0){\hskip\@dashdim \@whilenum \@tempcnta <\@dashcnt
⟨latexrelease⟩\do{\copy\@dashbox\advance\@tempcnta \@ne }}\@tempcnta\z@
⟨latexrelease⟩\put(0,#3){\hskip\@dashdim \@whilenum \@tempcnta <\@dashcnt
⟨latexrelease⟩\do{\copy\@dashbox\advance\@tempcnta \@ne }}%
⟨latexrelease⟩\@dashdim #3\unitlength
⟨latexrelease⟩\@dashcnt \@dashdim \advance\@dashcnt 200
⟨latexrelease⟩\@dashdim #1\unitlength\divide\@dashcnt \@dashdim
⟨latexrelease⟩\ifodd\@dashcnt \@dashdim \z@
⟨latexrelease⟩\advance\@dashcnt \@ne \divide\@dashcnt \tw@
⟨latexrelease⟩\else
⟨latexrelease⟩\divide\@dashdim \tw@ \divide\@dashcnt \tw@
⟨latexrelease⟩\advance\@dashcnt \m@ne
⟨latexrelease⟩\setbox\@dashbox\hbox{\hskip -\@halfwidth
⟨latexrelease⟩\vrule \@width \@wholewidth
⟨latexrelease⟩\@height \@dashdim}\put(0,0){\copy\@dashbox}%
⟨latexrelease⟩\put(#2,0){\copy\@dashbox}%
⟨latexrelease⟩\put(0,#3){\lower\@dashdim\copy\@dashbox}%
⟨latexrelease⟩\put(#2,#3){\lower\@dashdim\copy\@dashbox}%
⟨latexrelease⟩\multiply\@dashdim \thr@@
⟨latexrelease⟩\fi
⟨latexrelease⟩\setbox\@dashbox\hbox{\vrule \@width \@wholewidth
⟨latexrelease⟩\@height #1\unitlength}\@tempcnta\z@
⟨latexrelease⟩\put(0,0){%
⟨latexrelease⟩  \hskip -\@halfwidth \vbox{\@whilenum \@tempcnta <\@dashcnt
⟨latexrelease⟩  \do{\vskip #1\unitlength\copy\@dashbox
⟨latexrelease⟩      \advance\@tempcnta\@ne }%
⟨latexrelease⟩  \vskip\@dashdim}}\@tempcnta\z@
⟨latexrelease⟩\put(#2,0){%
⟨latexrelease⟩  \hskip -\@halfwidth \vbox{\@whilenum \@tempcnta<\@dashcnt
⟨latexrelease⟩  \do{\vskip #1\unitlength\copy\@dashbox
⟨latexrelease⟩      \advance\@tempcnta \@ne }%
⟨latexrelease⟩  \vskip\@dashdim}}\@makepicbox(#2,#3)}
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\dashbox`.)

Historical LaTeX 2.09 comments (not necessarily accurate any more):

CIRCLES AND OVALS

USER COMMANDS:

`\circle{D}` : Produces the circle with the diameter as close as possible to D * `\unitlength`. `\put(X,Y){\circle{D}}` puts the circle with its center at (X,Y).

`\oval(X,Y)` : Makes an oval as round as possible that fits in the rectangle of width X * `\unitlength` and height Y * `\unitlength`. The reference point is the center.

`\oval(X,Y)[POS]` : Save as `\oval(X,Y)` except it draws only the half or quadrant of the oval indicated by POS.

E.G., \oval(X,Y)[t] draws just the top half
and \oval(X,Y)[br] draws just the bottom right
quadrant. In all cases, the reference point is
the same as the unqualified \oval(X,Y) command.

\@ovvert {DELTA1} {DELTA2} : Makes a vbox containing either the left side or the right side of the oval being constructed. The baseline will coincide with the outside bottom edge of the oval; the left side of the box will coincide with the left edge of the vertical rule. The width of the box will be \@tempdima.
DELTA1 and DELTA2 are added to the character number in \@tempcnta to get the characters for the top and bottom quarter circle pieces.

\@ovhorz : Makes an hbox containing the straight rule for either the top or the bottom of the oval being constructed. The baseline will coincide with bottom edge of the rule; the left side of the box will coincide with the left side of the oval.
The width of the box will be \@ovxx.

\@getcirc {DIAM} : Sets \@tempcnta to the character number of the top-right quarter circle with the largest diameter less than or equal to DIAM.
Sets \@tempboxa to an hbox containing that character.
Sets \@tempdima to \wd \@tempboxa, which is the distance from the circle's left outside edge to its right inside edge.
(These characters are like those described in the TeXbook, pp. 389-90.)

```
\@getcirc {DIAM} ==
  BEGIN
    \@tempcnta       := integer coercion of (DIAM + 2pt)
                                          + 2pt added 1 Nov 88
    \@tempcnta       := \@tempcnta / integer coercion of 4pt
    if \@tempcnta > 10
      then \@tempcnta := 10 fi
    if \@tempcnta > 0
      then \@tempcnta := \@tempcnta-1
      else LaTeX Warning: Oval too small.
    fi
    \@tempcnta       := 4 * \@tempcnta
    \@tempboxa       := \hbox{\@circlefnt \char \@tempcnta}
    \@tempdima       := \wd \@tempboxa
  END

\@put{X}{Y}{OBJ} ==
  BEGIN
    \raise Y \hb@xt@ 0pt{\hskip X OBJ \hss}
  END
```

```
\@oval(X,Y)[POS] ==
  BEGIN
    \begingroup
      \boxmaxdepth := \maxdimen
      @ovt := @ovb := @ovl := @ovr := true
      for all E in POS
        do  @ovE := false od
      \@ovxx      := X * \unitlength
      \@ovyy      := Y * \unitlength
      \@tempdimb := min(\@ovxx,\@ovyy)
      \@getcirc{\@tempdimb-2pt}   %% "-2pt" added 7 Dec 89
      \@ovro     := \ht \@tempboxa
      \@ovri     := \dp \@tempboxa
      \@ovdx     := \@ovxx - \@tempdima
      \@ovdx     := \@ovdx/2
      \@ovdy     := \@ovyy - \@tempdima
      \@ovdy     := \@ovyy/2
      \@circlefnt
      \@tempboxa :=
          \hbox{
                if @ovr
                  then \@ovvert{3}{2} \kern -\@tempdima
                fi
                if @ovl
                  then \kern \@ovxx \@ovvert{0}{1} \kern -\@tempdima
                       \kern -\@ovxx
                fi
                if @ovt
                  then \@ovhorz \kern -\@ovxx
                fi
                if @ovb
                  then \raise \@ovyy \@ovhorz
                fi
               }
      \@ovdx    := \@ovdx + \@ovro
      \@ovdy    := \@ovdy + \@ovro
     \ht\@tempboxa := \dp\@tempboxa := 0
      \@put{-\@ovdx}{-\@ovdy}{\box\@tempboxa}
   \endgroup
  END

\@ovvert {DELTA1} {DELTA2} ==
  BEGIN
     \vbox to \@ovyy {
                    if @ovb
                      then \@tempcntb := \@tempcnta + DELTA1
                           \kern -\@ovro
                           \hbox { \char \@tempcntb }
                           \nointerlineskip
                      else \kern \@ovri \kern \@ovdy
```

```
                    fi
                    \leaders \vrule width \@wholewidth \vfil
                    \nointerlineskip
                    if @ovt
                      then \@tempcntb := \@tempcnta + DELTA2
                           \hbox { \char \@tempcntb }
                      else \kern \@ovdy \kern \@ovro
                    fi
                   }
  END

\@ovhorz ==
  BEGIN
   \hb@xt@ \@ovxx{
                  \kern \@ovro
                  if @ovr
                    then
                    else \kern \@ovdx
                  fi
                  \leaders \hrule height \@wholewidth \hfil
                  if @ovl
                    then
                    else \kern \@ovdx
                  fi
                  \kern \@ovri
                 }
  END

\circle{DIAM} ==
  BEGIN
   \begingroup
   \boxmaxdepth := maxdimen
   \@tempdimb := DIAM *\unitlength
   if \@tempdimb > 15.5pt
     then \@getcirc{\@tempdimb}
          \@ovro := \ht \@tempboxa
          \@tempboxa := \hbox{
                  \@circlefnt
                  \@tempcnta := \@tempcnta + 2
                  \char \@tempcnta
                  \@tempcnta := \@tempcnta - 1
                  \char \@tempcnta
                  \kern -2\@tempdima
                  \@tempcnta := \@tempcnta + 2
                  \raise \@tempdima \hbox { \char \@tempcnta }
                  \raise \@tempdima \box\@tempboxa
                 }
          \ht\@tempboxa := \dp\@tempboxa := 0
          \@put{-\@ovro}{-\@ovro}{\@tempboxa}
     else
```

```
            \@circ{\@tempdimb}{96}
     fi
    \endgroup
    END

 \circle*{DIAM}  ==  \@dot{DIAM} == \@circ{DIAM*\unitlength}{112}

 \@circ{DIAM}{CHAR} ==
  BEGIN
   \@tempcnta := integer coercion of (DIAM + .5pt)/1pt.
   if \@tempcnta > 15 then \@tempcnta := 15 fi
   if \@tempcnta > 1  then \@tempcnta := \@tempcnta - 1 fi
   \@tempcnta := \@tempcnta + CHAR
   \@circlefnt
   \char \@tempcnta
  END
```

End of historical LaTeX 2.09 comments.

\if@ovt
\if@ovb
\if@ovl
\if@ovr

If producing the Top Bottom Left or Right of an oval.

```
\newif\if@ovt
\newif\if@ovb
\newif\if@ovl
\newif\if@ovr
```

(*End of definition for* `\if@ovt` *and others.*)

\@ovxx
\@ovyy
\@ovdx
\@ovdy
\@ovro
\@ovri

```
\newdimen\@ovxx
\newdimen\@ovyy
\newdimen\@ovdx
\newdimen\@ovdy
\newdimen\@ovro
\newdimen\@ovri
```

(*End of definition for* `\@ovxx` *and others.*)

`\advance\@tempdima 2pt\relax` added 1 Nov 88 to fix bug in which size of drawn circle not monotonic function of argument of `\circle`, caused by different rounding for dimensions of large and small circles.

\@getcirc

```
\def\@getcirc#1{\@tempdima #1\relax \advance\@tempdima 2\p@
  \@tempcnta\@tempdima
  \@tempdima 4\p@ \divide\@tempcnta\@tempdima
  \ifnum \@tempcnta >10\relax
      \@picture@warn
      \@tempcnta 10\relax
  \fi
  \ifnum \@tempcnta >\z@ \advance\@tempcnta\m@ne
```

Warn if requirements for oval or circle can't be met.

```
    \else \@picture@warn \fi
  \multiply\@tempcnta 4\relax
  \setbox \@tempboxa \hbox{\@circlefnt
  \char \@tempcnta}\@tempdima \wd \@tempboxa}
```

(*End of definition for* `\@getcirc`.)

`\@picture@warn` Generic warning for lines, vectors (used in `\@sline`) and oval or circle (used in `\@getcirc`) are not available at right size.

```
\def\@picture@warn{\@latex@warning{%
     \string\oval, \string\circle, or \string\line\space
     size unavailable}}
```

(*End of definition for* `\@picture@warn`.)

`\@put`

```
\def\@put#1#2#3{\raise #2\hb@xt@\z@{\hskip #1#3\hss}}
```

(*End of definition for* `\@put`.)

`\oval`

```
\def\oval(#1,#2){\@ifnextchar[{\@oval(#1,#2)}{\@oval(#1,#2)[]}}
```

(*End of definition for* `\oval`.)

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2016/03/31}%
⟨latexrelease⟩                  {\@ovhlinetrue}%
⟨latexrelease⟩                  {Avoid almost zero length leaders}%
⟨∗2ekernel | latexrelease⟩
```

`\if@ovvline` `\if@ovhline` Tests whether horizontal or vertical lines are needed.

```
\newif\if@ovvline \@ovvlinetrue
\newif\if@ovhline \@ovhlinetrue
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@ovhlinetrue}%
⟨latexrelease⟩                  {Avoid almost zero length leaders}%
⟨latexrelease⟩\let\if@ovvline\@undefined
⟨latexrelease⟩\let\if@ovhline\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\if@ovvline` *and* `\if@ovhline`.)

`\@oval`

```
⟨/2ekernel⟩
⟨∗2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\@oval}{default units}%
\def\@oval(#1,#2)[#3]{\begingroup\boxmaxdepth \maxdimen
  \@ovttrue \@ovbtrue \@ovltrue \@ovrtrue

  \@ovvlinefalse \@ovhlinefalse

  \@tfor\reserved@a :=#3\do{%
    \csname @ov\reserved@a false\endcsname}%
  \@defaultunitsset\@ovxx{#1}\unitlength
  \@defaultunitsset\@ovyy{#2}\unitlength
```

```
  \@tempdimb \ifdim \@ovyy >\@ovxx \@ovxx \@ovvlinetrue
  \else \@ovyy \ifdim \@ovyy =\@ovxx \else \@ovhlinetrue \fi\fi
  \advance \@tempdimb -2\p@
  \@getcirc \@tempdimb
  \@ovro \ht\@tempboxa \@ovri \dp\@tempboxa
  \@ovdx\@ovxx \advance\@ovdx -\@tempdima \divide\@ovdx \tw@
  \@ovdy\@ovyy \advance\@ovdy -\@tempdima \divide\@ovdy \tw@
  \ifdim \@ovdx >\z@ \@ovhlinetrue \fi
  \ifdim \@ovdy >\z@ \@ovvlinetrue \fi
  \@circlefnt \setbox\@tempboxa
  \hbox{\if@ovr \@ovvert32\kern -\@tempdima \fi
  \if@ovl \kern \@ovxx \@ovvert01\kern -\@tempdima \kern -\@ovxx \fi
  \if@ovt \@ovhorz \kern -\@ovxx \fi
  \if@ovb \raise \@ovyy \@ovhorz \fi}\advance\@ovdx\@ovro
  \advance\@ovdy\@ovro \ht\@tempboxa\z@ \dp\@tempboxa\z@
  \@put{-\@ovdx}{-\@ovdy}{\box\@tempboxa}%
  \endgroup}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2016/03/31}%
⟨latexrelease⟩                  {\@oval}{default units}%
⟨latexrelease⟩\def\@oval(#1,#2)[#3]{\begingroup\boxmaxdepth \maxdimen
⟨latexrelease⟩  \@ovttrue \@ovbtrue \@ovltrue \@ovrtrue
⟨latexrelease⟩  \@ovvlinefalse \@ovhlinefalse
⟨latexrelease⟩  \@tfor\reserved@a :=#3\do{%
⟨latexrelease⟩    \csname @ov\reserved@a false\endcsname}%
⟨latexrelease⟩  \@ovxx #1\unitlength
⟨latexrelease⟩  \@ovyy #2\unitlength
⟨latexrelease⟩  \@tempdimb \ifdim \@ovyy >\@ovxx \@ovxx \@ovvlinetrue
⟨latexrelease⟩  \else \@ovyy \ifdim \@ovyy =\@ovxx \else \@ovhlinetrue
⟨latexrelease⟩   \fi\fi
⟨latexrelease⟩  \advance \@tempdimb -2\p@
⟨latexrelease⟩  \@getcirc \@tempdimb
⟨latexrelease⟩  \@ovro \ht\@tempboxa \@ovri \dp\@tempboxa
⟨latexrelease⟩  \@ovdx\@ovxx \advance\@ovdx -\@tempdima \divide\@ovdx \tw@
⟨latexrelease⟩  \@ovdy\@ovyy \advance\@ovdy -\@tempdima \divide\@ovdy \tw@
⟨latexrelease⟩  \ifdim \@ovdx >\z@ \@ovhlinetrue \fi
⟨latexrelease⟩  \ifdim \@ovdy >\z@ \@ovvlinetrue \fi
⟨latexrelease⟩  \@circlefnt \setbox\@tempboxa
⟨latexrelease⟩  \hbox{\if@ovr \@ovvert32\kern -\@tempdima \fi
⟨latexrelease⟩  \if@ovl
⟨latexrelease⟩   \kern \@ovxx \@ovvert01\kern -\@tempdima \kern -\@ovxx
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \if@ovt \@ovhorz \kern -\@ovxx \fi
⟨latexrelease⟩  \if@ovb \raise \@ovyy \@ovhorz \fi}\advance\@ovdx\@ovro
⟨latexrelease⟩  \advance\@ovdy\@ovro \ht\@tempboxa\z@ \dp\@tempboxa\z@
⟨latexrelease⟩  \@put{-\@ovdx}{-\@ovdy}{\box\@tempboxa}%
⟨latexrelease⟩  \endgroup}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@oval}{default units}%
⟨latexrelease⟩\def\@oval(#1,#2)[#3]{\begingroup\boxmaxdepth \maxdimen
```

```
⟨latexrelease⟩  \@ovttrue \@ovbtrue \@ovltrue \@ovrtrue
⟨latexrelease⟩  \@tfor\reserved@a :=#3\do
⟨latexrelease⟩                {\csname @ov\reserved@a false\endcsname}%
⟨latexrelease⟩  \@ovxx #1\unitlength
⟨latexrelease⟩  \@ovyy #2\unitlength
⟨latexrelease⟩  \@tempdimb \ifdim \@ovyy >\@ovxx \@ovxx\else \@ovyy \fi
⟨latexrelease⟩  \advance \@tempdimb -2\p@
⟨latexrelease⟩  \@getcirc \@tempdimb
⟨latexrelease⟩  \@ovro \ht\@tempboxa \@ovri \dp\@tempboxa
⟨latexrelease⟩  \@ovdx\@ovxx \advance\@ovdx -\@tempdima \divide\@ovdx \tw@
⟨latexrelease⟩  \@ovdy\@ovyy \advance\@ovdy -\@tempdima \divide\@ovdy \tw@
⟨latexrelease⟩  \@circlefnt \setbox\@tempboxa
⟨latexrelease⟩  \hbox{\if@ovr \@ovvert32\kern -\@tempdima \fi
⟨latexrelease⟩  \if@ovl
⟨latexrelease⟩   \kern \@ovxx \@ovvert01\kern -\@tempdima \kern -\@ovxx
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \if@ovt \@ovhorz \kern -\@ovxx \fi
⟨latexrelease⟩  \if@ovb \raise \@ovyy \@ovhorz \fi}\advance\@ovdx\@ovro
⟨latexrelease⟩  \advance\@ovdy\@ovro \ht\@tempboxa\z@ \dp\@tempboxa\z@
⟨latexrelease⟩  \@put{-\@ovdx}{-\@ovdy}{\box\@tempboxa}%
⟨latexrelease⟩  \endgroup}
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\@oval`.)

`\@ovvert`

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2016/03/31}%
⟨latexrelease⟩                  {\@ovvert}{Avoid almost zero length leaders}%
⟨∗2ekernel | latexrelease⟩
\def\@ovvert#1#2{\vbox to\@ovyy{%
    \if@ovb \@tempcntb \@tempcnta \advance \@tempcntb #1\relax
      \kern -\@ovro \hbox{\char \@tempcntb}\nointerlineskip
    \else \kern \@ovri \kern \@ovdy \fi

    \if@ovvline \leaders\vrule \@width \@wholewidth \fi

    \vfil \nointerlineskip
    \if@ovt \@tempcntb \@tempcnta \advance \@tempcntb #2\relax
      \hbox{\char \@tempcntb}%
    \else \kern \@ovdy \kern \@ovro \fi}}
⟨/2ekernel | latexrelease⟩

⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@ovvert}{Avoid almost zero length leaders}%
⟨latexrelease⟩\def\@ovvert#1#2{\vbox to\@ovyy{%
⟨latexrelease⟩    \if@ovb \@tempcntb \@tempcnta \advance \@tempcntb #1\relax
⟨latexrelease⟩      \kern -\@ovro \hbox{\char \@tempcntb}\nointerlineskip
⟨latexrelease⟩    \else \kern \@ovri \kern \@ovdy \fi
⟨latexrelease⟩    \leaders\vrule \@width \@wholewidth\vfil \nointerlineskip
⟨latexrelease⟩    \if@ovt \@tempcntb \@tempcnta \advance \@tempcntb #2\relax
⟨latexrelease⟩      \hbox{\char \@tempcntb}%
⟨latexrelease⟩    \else \kern \@ovdy \kern \@ovro \fi}}
⟨latexrelease⟩\EndIncludeInRelease
```

```
⟨*2ekernel⟩
```

(*End of definition for* `\@ovvert`.)

`\@ovhorz`

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2016/03/31}%
⟨latexrelease⟩                  {\@ovhorz}{Avoid almost zero length leaders}%
⟨*2ekernel | latexrelease⟩
\def\@ovhorz{\hb@xt@\@ovxx{\kern \@ovro
    \if@ovr \else \kern \@ovdx \fi
    \if@ovhline \leaders \hrule \@height \@wholewidth \fi
    \hfil
    \if@ovl \else \kern \@ovdx \fi
    \kern \@ovri}}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@ovhorz}{Avoid almost zero length leaders}%
⟨latexrelease⟩\def\@ovhorz{\hb@xt@\@ovxx{\kern \@ovro
⟨latexrelease⟩    \if@ovr \else \kern \@ovdx \fi
⟨latexrelease⟩    \leaders \hrule \@height \@wholewidth \hfil
⟨latexrelease⟩    \if@ovl \else \kern \@ovdx \fi
⟨latexrelease⟩    \kern \@ovri}}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@ovhorz`.)

`\circle`

```
\def\circle{\@inmatherr\circle\@ifstar\@dot\@circle}
```

(*End of definition for* `\circle`.)

`\@circle`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\@circle}{default units}%
\def\@circle#1{%
  \begingroup \boxmaxdepth \maxdimen
   \@defaultunitsset\@tempdimb{#1}\unitlength
   \ifdim \@tempdimb >15.5\p@ \@getcirc\@tempdimb
      \@ovro\ht\@tempboxa
     \setbox\@tempboxa\hbox{\@circlefnt
      \advance\@tempcnta\tw@ \char \@tempcnta
      \advance\@tempcnta\m@ne \char \@tempcnta \kern -2\@tempdima
      \advance\@tempcnta\tw@
      \raise \@tempdima \hbox{\char\@tempcnta}\raise \@tempdima
        \box\@tempboxa}\ht\@tempboxa\z@ \dp\@tempboxa\z@
      \@put{-\@ovro}{-\@ovro}{\box\@tempboxa}%
  \else  \@circ\@tempdimb{96}\fi\endgroup}
⟨/2ekernel | latexrelease⟩
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@circle}{default units}%
⟨latexrelease⟩\def\@circle#1{%
⟨latexrelease⟩  \begingroup \boxmaxdepth \maxdimen \@tempdimb #1\unitlength
⟨latexrelease⟩   \ifdim \@tempdimb >15.5\p@ \@getcirc\@tempdimb
⟨latexrelease⟩      \@ovro\ht\@tempboxa
⟨latexrelease⟩     \setbox\@tempboxa\hbox{\@circlefnt
⟨latexrelease⟩      \advance\@tempcnta\tw@ \char \@tempcnta
⟨latexrelease⟩      \advance\@tempcnta\m@ne \char \@tempcnta
⟨latexrelease⟩      \kern -2\@tempdima
⟨latexrelease⟩      \advance\@tempcnta\tw@
⟨latexrelease⟩      \raise \@tempdima \hbox{\char\@tempcnta}%
⟨latexrelease⟩      \raise \@tempdima
⟨latexrelease⟩        \box\@tempboxa}\ht\@tempboxa\z@ \dp\@tempboxa\z@
⟨latexrelease⟩      \@put{-\@ovro}{-\@ovro}{\box\@tempboxa}%
⟨latexrelease⟩   \else  \@circ\@tempdimb{96}\fi\endgroup}
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\@circle`.)

`\@dot` Internal form of `\circle*`.

```
⟨/2ekernel⟩
⟨∗2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\@dot}{default units}%
\def\@dot#1{%
  \@defaultunitsset\@tempdimb{#1}\unitlength
  \@circ\@tempdimb{112}}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@dot}{default units}%
⟨latexrelease⟩\def\@dot#1{\@tempdimb #1\unitlength \@circ\@tempdimb{112}}
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\@dot`.)

`\@circ`

```
\def\@circ#1#2{\@tempdima #1\relax \advance\@tempdima .5\p@
   \@tempcnta\@tempdima \@tempdima \p@
   \divide\@tempcnta\@tempdima
   \ifnum\@tempcnta >15\relax \@tempcnta 15\relax \fi
   \ifnum \@tempcnta >\z@ \advance\@tempcnta\m@ne\fi
   \advance\@tempcnta #2\relax
   \@circlefnt \char\@tempcnta}
```

(*End of definition for* `\@circ`.)

`\@xarg` `\@yarg` `\@yyarg` Counters used for manipulating the 'slope' arguments.

```
\newcount\@xarg
\newcount\@yarg
\newcount\@yyarg
```

(*End of definition for* `\@xarg`, `\@yarg`, *and* `\@yyarg`.)

`\@multicnt` Counter used in `\multiput`, and also `\multicolumn`.

```
\newcount\@multicnt
```

(*End of definition for* `\@multicnt`.)

`\@xdim` Length registers.
`\@ydim`

```
\newdimen\@xdim
\newdimen\@ydim
```

(*End of definition for* `\@xdim` *and* `\@ydim`.)

`\@linechar` Box for holding a line segment character, for sloping lines.

```
\newbox\@linechar
```

(*End of definition for* `\@linechar`.)

`\@linelen` Length of the line currently being built.

```
\newdimen\@linelen
```

(*End of definition for* `\@linelen`.)

`\@clnwd` Height and width of current line segment.
`\@clnht`

```
\newdimen\@clnwd
\newdimen\@clnht
```

(*End of definition for* `\@clnwd` *and* `\@clnht`.)

`\@dashdim` `\dashbox` internal registers.
`\@dashbox`
`\@dashcnt`

```
\newdimen\@dashdim
\newbox\@dashbox
\newcount\@dashcnt
```

(*End of definition for* `\@dashdim`, `\@dashbox`, *and* `\@dashcnt`.)

Initialization: "`\thinlines`"

```
\let\@linefnt\tenln
\let\@circlefnt\tencirc
\@wholewidth\fontdimen8\tenln
\@halfwidth .5\@wholewidth
```

1.1 Curves

The new \qbezier command, based on the old \bezier defined in bezier.sty. *Historical LaTeX 2.09 comments (not necessarily accurate any more):*

```
\qbezier[N] == \bezier{N}

\bezier{N}(AX,AY)(BX,BY)(CX,CY) ==
  BEGIN
    IF N = 0
       THEN \@xdima := |BX - AX|
            \@xb := |CX - BX|
            \@xa := Max(\@xa, \@xb)
            \@ya := |BY - AY|
            \@yb := |CY - BY|
            \@ya := Max(\@ya, \@yb)
            @sc := Max(\@xa, \@ya)
            %% The coefficient .5 below is the degree of overlap of
            %% successive points, where 1 is no overlap and 0 is
            %% complete overlap.  A coefficient of C multiplies
            %% the number of points plotted by 1/C.
            %%
            \@xa := .5 * \@halfwidth
            @sc := @sc / \@halfwidth
            @sc := Max(@sc, qbeziermax)
        ELSE @sc := N
    @scp := @sc+1
    \@xb := 2 * (BX - AX) * \unitlength
    \@xa := ((CX-AX)*\unitlength - \@xb)/@sc
    \@yb := 2 * (BY - AY) * \unitlength
    \@ya := ((CY-AY)*\unitlength - \@yb)/@sc
    \@pictdot := square rule of width \@wholewidth
    \count@ := 0
    WHILE \count@ < @scp
      DO  \@xdim := ((\count@*\@xa + @xb) / @sc) * \count@
          \@ydim := ((\count@*\@ya + @yb) / @sc) * \count@
          plot pt with relative coords (\@xdim,\@ydim)
          \count@ := \count@+1
      OD
```

End of historical LaTeX 2.09 comments.

\qbeziermax The maximum number of points to plot.

```
\def\qbeziermax{500}
```

(*End of definition for* `\qbeziermax`.)

In the code below, to save registers `\@a` ... are not used. Instead other registers are reused.

```
\newcounter{@sc} -> \c@multicnt
\newcounter{@scp} -> \@tempcnta
\newdimen\@xa ->  \@ovxx
\newdimen\@xb ->  \@ovdx
\newdimen\@ya ->  \@ovyy
\newdimen\@yb ->  \@ovdy
\newsavebox{\@pictdot} -> \@tempboxa
```

`\qbezier` Main user-level command to plot quadratic bezier curves. `#2` should be `(`.

```
\newcommand\qbezier[2][0]{\bezier{#1}#2}
```

(*End of definition for* `\qbezier`.)

`\bezier` Form of `\bezier` compatible with 2.09 `bezier.sty`, but modified to ignore spaces between its arguments. `#2` should be white space, and `#4` should be `(`.

```
\def\bezier#1)#2(#3)#4({\@bezier#1)(#3)(}
```

`\@bezier`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\@bezier}{default units}%
\def\@bezier#1(#2,#3)(#4,#5)(#6,#7){%
  \ifnum #1=\z@
      \@defaultunitsset\@ovxx{#4}\unitlength
        \@defaultunitsset{\advance\@ovxx}{-#2}\unitlength
        \ifdim \@ovxx<\z@ \@ovxx -\@ovxx \fi
      \@defaultunitsset\@ovdx{#6}\unitlength
        \@defaultunitsset{\advance\@ovdx}{-#4}\unitlength
        \ifdim \@ovdx<\z@ \@ovdx -\@ovdx \fi
        \ifdim \@ovxx<\@ovdx \@ovxx \@ovdx \fi
      \@defaultunitsset\@ovyy{#5}\unitlength
        \@defaultunitsset{\advance\@ovyy}{-#3}\unitlength
        \ifdim \@ovyy<\z@ \@ovyy -\@ovyy \fi
      \@defaultunitsset\@ovdy{#7}\unitlength
        \@defaultunitsset{\advance\@ovdy}{-#5}\unitlength
        \ifdim \@ovdy<\z@  \@ovdy -\@ovdy \fi
        \ifdim \@ovyy<\@ovdy \@ovyy  \@ovdy \fi
      \@multicnt
         \ifdim \@ovxx>\@ovyy \@ovxx \else \@ovyy \fi
      \@ovxx .5\@halfwidth \divide\@multicnt\@ovxx
      \ifnum \qbeziermax<\@multicnt
        \@multicnt\qbeziermax\relax
      \fi
  \else \@multicnt#1\relax \fi
  \@tempcnta\@multicnt \advance\@tempcnta\@ne
  \@defaultunitsset\@ovdx{#4}\unitlength
  \@defaultunitsset{\advance\@ovdx}{-#2}\unitlength
      \multiply\@ovdx \tw@
  \@defaultunitsset\@ovxx{#6}\unitlength
```

```
\@defaultunitsset{\advance\@ovxx}{-#2}\unitlength
    \advance\@ovxx -\@ovdx \divide\@ovxx\@multicnt
\@defaultunitsset\@ovdy{#5}\unitlength
\@defaultunitsset{\advance\@ovdy}{-#3}\unitlength
     \multiply\@ovdy \tw@
\@defaultunitsset\@ovyy{#7}\unitlength
\@defaultunitsset{\advance\@ovyy}{-#3}\unitlength
    \advance\@ovyy -\@ovdy \divide\@ovyy\@multicnt

\setbox\@tempboxa\hbox{%
          \hskip -\@halfwidth
          \vrule \@height\@halfwidth
                 \@depth \@halfwidth
                 \@width \@wholewidth}%
 \put(#2,#3){%
   \count@\z@
   \@whilenum{\count@<\@tempcnta}\do
      {\@xdim\count@\@ovxx
         \advance\@xdim\@ovdx
         \divide\@xdim\@multicnt
         \multiply\@xdim\count@
       \@ydim\count@\@ovyy
         \advance\@ydim\@ovdy
         \divide\@ydim\@multicnt
         \multiply\@ydim\count@
       \raise \@ydim
         \hb@xt@\z@{\kern\@xdim
                    \unhcopy\@tempboxa\hss}%
       \advance\count@\@ne}}}
⟨/2ekernel | latexrelease⟩

⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@bezier}{default units}%
⟨latexrelease⟩\def\@bezier#1(#2,#3)(#4,#5)(#6,#7){%
⟨latexrelease⟩  \ifnum #1=\z@
⟨latexrelease⟩      \@ovxx #4\unitlength
⟨latexrelease⟩        \advance\@ovxx -#2\unitlength
⟨latexrelease⟩        \ifdim \@ovxx<\z@ \@ovxx -\@ovxx \fi
⟨latexrelease⟩      \@ovdx #6\unitlength
⟨latexrelease⟩        \advance\@ovdx -#4\unitlength
⟨latexrelease⟩        \ifdim \@ovdx<\z@ \@ovdx -\@ovdx \fi
⟨latexrelease⟩        \ifdim \@ovxx<\@ovdx \@ovxx \@ovdx \fi
⟨latexrelease⟩      \@ovyy #5\unitlength
⟨latexrelease⟩        \advance\@ovyy -#3\unitlength
⟨latexrelease⟩        \ifdim \@ovyy<\z@ \@ovyy -\@ovyy \fi
⟨latexrelease⟩      \@ovdy #7\unitlength
⟨latexrelease⟩        \advance\@ovdy -#5\unitlength
⟨latexrelease⟩        \ifdim \@ovdy<\z@  \@ovdy -\@ovdy \fi
⟨latexrelease⟩        \ifdim \@ovyy<\@ovdy \@ovyy  \@ovdy \fi
⟨latexrelease⟩      \@multicnt
⟨latexrelease⟩         \ifdim \@ovxx>\@ovyy \@ovxx \else \@ovyy \fi
⟨latexrelease⟩      \@ovxx .5\@halfwidth \divide\@multicnt\@ovxx
⟨latexrelease⟩      \ifnum
⟨latexrelease⟩        \qbeziermax<\@multicnt \@multicnt\qbeziermax\relax
```

```
⟨latexrelease⟩      \fi
⟨latexrelease⟩  \else \@multicnt#1\relax \fi
⟨latexrelease⟩  \@tempcnta\@multicnt \advance\@tempcnta\@ne
⟨latexrelease⟩  \@ovdx #4\unitlength \advance\@ovdx -#2\unitlength
⟨latexrelease⟩      \multiply\@ovdx \tw@
⟨latexrelease⟩  \@ovxx #6\unitlength \advance\@ovxx -#2\unitlength
⟨latexrelease⟩      \advance\@ovxx -\@ovdx \divide\@ovxx\@multicnt
⟨latexrelease⟩  \@ovdy #5\unitlength \advance\@ovdy -#3\unitlength
⟨latexrelease⟩       \multiply\@ovdy \tw@
⟨latexrelease⟩  \@ovyy #7\unitlength \advance\@ovyy -#3\unitlength
⟨latexrelease⟩      \advance\@ovyy -\@ovdy \divide\@ovyy\@multicnt
⟨latexrelease⟩  \setbox\@tempboxa\hbox{%
⟨latexrelease⟩            \hskip -\@halfwidth
⟨latexrelease⟩            \vrule \@height\@halfwidth
⟨latexrelease⟩                   \@depth \@halfwidth
⟨latexrelease⟩                   \@width \@wholewidth}%
⟨latexrelease⟩   \put(#2,#3){%
⟨latexrelease⟩     \count@\z@
⟨latexrelease⟩     \@whilenum{\count@<\@tempcnta}\do
⟨latexrelease⟩        {\@xdim\count@\@ovxx
⟨latexrelease⟩           \advance\@xdim\@ovdx
⟨latexrelease⟩           \divide\@xdim\@multicnt
⟨latexrelease⟩           \multiply\@xdim\count@
⟨latexrelease⟩         \@ydim\count@\@ovyy
⟨latexrelease⟩           \advance\@ydim\@ovdy
⟨latexrelease⟩           \divide\@ydim\@multicnt
⟨latexrelease⟩           \multiply\@ydim\count@
⟨latexrelease⟩         \raise \@ydim
⟨latexrelease⟩           \hb@xt@\z@{\kern\@xdim
⟨latexrelease⟩                          \unhcopy\@tempboxa\hss}%
⟨latexrelease⟩         \advance\count@\@ne}}}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\bezier` *and* `\@bezier`.)

As the commands above all use "picture" interface we couldn't define them with `\DeclareRobustCommand` so we do that now.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                 {\bezier}{Make commands robust}%
\MakeRobust\bezier
\MakeRobust\circle
\MakeRobust\dashbox
\MakeRobust\line
\MakeRobust\linethickness
\MakeRobust\multiput
\MakeRobust\oval
\MakeRobust\put
\MakeRobust\qbezier
\MakeRobust\shortstack
\MakeRobust\thinlines
\MakeRobust\vector
⟨/2ekernel | latexrelease⟩
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\bezier}{Make commands robust}%
⟨latexrelease⟩
⟨latexrelease⟩\kernel@make@fragile\bezier
⟨latexrelease⟩\kernel@make@fragile\circle
⟨latexrelease⟩\kernel@make@fragile\dashbox
⟨latexrelease⟩\kernel@make@fragile\line
⟨latexrelease⟩\kernel@make@fragile\linethickness
⟨latexrelease⟩\kernel@make@fragile\multiput
⟨latexrelease⟩\kernel@make@fragile\oval
⟨latexrelease⟩\kernel@make@fragile\put
⟨latexrelease⟩\kernel@make@fragile\qbezier
⟨latexrelease⟩\kernel@make@fragile\shortstack
⟨latexrelease⟩\kernel@make@fragile\thinlines
⟨latexrelease⟩\kernel@make@fragile\vector
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
⟨/2ekernel⟩
```

File 43
ltthm.dtx

1 Theorem Environments

The user creates his own theorem-like environments with the command
`\newtheorem{`⟨*name*⟩`}{`⟨*text*⟩`}[`⟨*counter*⟩`]` or
`\newtheorem{`⟨*name*⟩`}[`⟨*oldname*⟩`]{`⟨*text*⟩`}`
This defines the environment ⟨*name*⟩ to be just as one would expect a theorem environment to be, except that it prints ⟨*text*⟩ instead of "Theorem".

If ⟨*oldname*⟩ is given, then environments ⟨*name*⟩ and ⟨*oldname*⟩ use the same counter, so using a ⟨*name*⟩ environment advances the number of the next ⟨*name*⟩ environment, and vice-versa.

If ⟨*counter*⟩ is given, then environment ⟨*name*⟩ is numbered within ⟨*counter*⟩.

E.g., if ⟨*counter*⟩ = `subsection`, then the first ⟨*name*⟩ in subsection 7.2 is numbered ⟨*text*⟩ 7.2.1.

The way ⟨*name*⟩ environments are numbered can be changed by redefining `\the`⟨*name*⟩.
Historical LaTeX 2.09 comments (not necessarily accurate any more):

DOCUMENT STYLE PARAMETERS

`\@thmcounter{COUNTER}` : A command such that
`\edef\theCOUNTER{\@thmcounter{COUNTER}}`
defines `\theCOUNTER` to produce a number for a theorem environment. The default is:
BEGIN `\noexpand\arabic{COUNTER}` END

`\@thmcountersep` : A separator placed between a theorem number and the number of the counter within which it is numbered. E.g., to make the third theorem of section 7.2 be numbered 7.2-3, `\@thmcountersep` should be `\def`'ed to '-'. Its default is '.'.

`\@begintheorem{NAME}{NUMBER}` : A command that begins a theorem environment for a 'theorem' named 'NAME NUMBER' – e.g., `\@begintheorem{Lemma}{3.7}` starts Lemma 3.7.

`\@opargbegintheorem{NAME}{NUMBER}{OPARG}` :
A command that begins a theorem environment for a 'theorem' named 'NAME NUMBER' with optional argument OPARG – e.g., `\@begintheorem{Lemma}{3.7}{Jones}` starts 'Lemma 3.7 (Jones):'.

`\@endtheorem` : A command that ends a theorem environment.

`\newtheorem{NAME}{TEXT}[COUNTER]` ==
BEGIN
if `\NAME` is definable

```
        then \@definecounter{NAME}
             if COUNTER present
               then \@newctr{NAME}[COUNTER] fi
                    \theNAME ==  BEGIN \theCOUNTER \@thmcountersep
                                       eval\@thmcounter{NAME}      END
               else \theNAME ==  BEGIN eval\@thmcounter{NAME} END
             \NAME == \@thm{NAME}{TEXT}
             \endNAME == \@endtheorem
        else  error
      fi
    END

  \newtheorem{NAME}[OLDNAME]{TEXT}==
    BEGIN
      if counter OLDNAME nonexistent
        then ERROR
        else
             if \NAME is definable
               then BEGIN
                    \theNAME == \theOLDNAME
                    \NAME == \@thm{OLDNAME}{TEXT}
                    \endNAME == \@endtheorem
                    END
               else  error
             fi
      fi
    END

  \@thm{NAME}{TEXT} ==
    BEGIN
     \refstepcounter{NAME}
     if next char = [
        then \@ythm{NAME}{TEXT}
        else \@xthm{NAME}{TEXT}
     fi
    END

  \@xthm{NAME}{TEXT} ==
    BEGIN
     \@begintheorem{TEXT}{\theNAME}
     \ignorespaces
    END

  \@ythm{NAME}{TEXT}[OPARG] ==
    BEGIN
     \@opargbegintheorem{TEXT}{\theNAME}{OPARG}
     \ignorespaces
    END
```

End of historical LaTeX 2.09 comments.

\newtheorem

\newtheorem ought really be allowed only in the preamble Which would be good document style, and allow some main memory to be saved by declaring these commands to be \@onlypreamble. Unfortunately the LaTeX book indicates that \newtheorem may be used anywhere in the document...

```
⟨*2ekernel⟩
\def\newtheorem#1{%
  \@ifnextchar[{\@othm{#1}}{\@nthm{#1}}}
```

(*End of definition for* \newtheorem.)

\@nthm

```
\def\@nthm#1#2{%
  \@ifnextchar[{\@xnthm{#1}{#2}}{\@ynthm{#1}{#2}}}
```

(*End of definition for* \@nthm.)

\@xnthm

92/09/18 RmS: Changed \@addtoreset to \@newctr to produce error message if counter #3 does not exist (to be consistent with behaviour of \newcounter)

```
\def\@xnthm#1#2[#3]{%
  \expandafter\@ifdefinable\csname #1\endcsname
    {\@definecounter{#1}\@newctr{#1}[#3]%
     \expandafter\xdef\csname the#1\endcsname{%
       \expandafter\noexpand\csname the#3\endcsname \@thmcountersep
          \@thmcounter{#1}}%
     \global\@namedef{#1}{\@thm{#1}{#2}}%
     \global\@namedef{end#1}{\@endtheorem}}}
```

(*End of definition for* \@xnthm.)

\@ynthm

```
\def\@ynthm#1#2{%
  \expandafter\@ifdefinable\csname #1\endcsname
    {\@definecounter{#1}%
     \expandafter\xdef\csname the#1\endcsname{\@thmcounter{#1}}%
     \global\@namedef{#1}{\@thm{#1}{#2}}%
     \global\@namedef{end#1}{\@endtheorem}}}
⟨/2ekernel⟩
```

(*End of definition for* \@ynthm.)

\@othm

```
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2026/06/01}%
⟨latexrelease⟩                    {\@othm}{use alias counter}%
\def\@othm#1[#2]#3{%
  \@ifundefined{c@#2}{\@nocounterr{#2}}%
    {\expandafter\@ifdefinable\csname #1\endcsname
    {\newcounteralias{#1}{#2}%
  \global\@namedef{#1}{\@thm{#1}{#3}}%
  \global\@namedef{end#1}{\@endtheorem}}}}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\@othm}{use alias counter}%
⟨latexrelease⟩\def\@othm#1[#2]#3{%
```

```
⟨latexrelease⟩  \@ifundefined{c@#2}{\@nocounterr{#2}}%
⟨latexrelease⟩    {\expandafter\@ifdefinable\csname #1\endcsname
⟨latexrelease⟩    {\global\@namedef{the#1}{\@nameuse{the#2}}%
⟨latexrelease⟩  \global\@namedef{#1}{\@thm{#2}{#3}}%
⟨latexrelease⟩  \global\@namedef{end#1}{\@endtheorem}}}}
⟨latexrelease⟩\EndIncludeInRelease
⟨/2ekernel | latexrelease⟩
```

(*End of definition for* `\@othm`.)

`\@thm`

```
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2024/03/18}%
⟨latexrelease⟩                  {\@thm}{no link target}%
\def\@thm#1#2{%
  \@kernel@refstepcounter{#1}%
  \@ifnextchar[{\@ythm{#1}{#2}}{\@xthm{#1}{#2}}}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@thm}{no link target}%
⟨latexrelease⟩\def\@thm#1#2{%
⟨latexrelease⟩  \refstepcounter{#1}%
⟨latexrelease⟩  \@ifnextchar[{\@ythm{#1}{#2}}{\@xthm{#1}{#2}}}
⟨latexrelease⟩\EndIncludeInRelease
⟨/2ekernel | latexrelease⟩
```

(*End of definition for* `\@thm`.)

`\@xthm`
`\@ythm`

```
⟨*2ekernel⟩
\def\@xthm#1#2{%
  \@begintheorem{#2}{\csname the#1\endcsname}\ignorespaces}
\def\@ythm#1#2[#3]{%
  \@opargbegintheorem{#2}{\csname the#1\endcsname}{#3}\ignorespaces}
```

(*End of definition for* `\@xthm` *and* `\@ythm`.)

Default values

`\@thmcounter`
`\@thmcountersep`

```
\def\@thmcounter#1{\noexpand\arabic{#1}}
\def\@thmcountersep{.}
```

(*End of definition for* `\@thmcounter` *and* `\@thmcountersep`.)

`\@begintheorem`
`\@opargbegintheorem`
`\@endtheorem`

Providing theorem defaults.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2024/03/18}%
⟨latexrelease⟩                  {\@begintheorem}{add link targets}%
\def\@begintheorem#1#2{\trivlist
   \item[\MakeLinkTarget{\@currentcounter}\hskip \labelsep{\bfseries #1\ #2}]\itshape}
\def\@opargbegintheorem#1#2#3{\trivlist
      \item[\MakeLinkTarget{\@currentcounter}\hskip \labelsep{\bfseries #1\ #2\ (#3)}]\itshap
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
```

```
⟨latexrelease⟩                  {\@begintheorem}{add link targets}%
⟨latexrelease⟩\def\@begintheorem#1#2{\trivlist
⟨latexrelease⟩    \item[\hskip \labelsep{\bfseries #1\ #2}]\itshape}
⟨latexrelease⟩\def\@opargbegintheorem#1#2#3{\trivlist
⟨latexrelease⟩    \item[\hskip \labelsep{\bfseries #1\ #2\ (#3)}]\itshape}
⟨latexrelease⟩\EndIncludeInRelease
⟨/2ekernel | latexrelease⟩
⟨*2ekernel⟩
\def\@endtheorem{\endtrivlist}
⟨/2ekernel⟩
```

(*End of definition for* `\@begintheorem`, `\@opargbegintheorem`, *and* `\@endtheorem`.)

File 44
ltsect.dtx

1 Sectioning Commands

This file defines the declarations such as `\author` which are used by `\maketitle`. `\maketitle` itself is defined by each class, not in the LaTeX kernel.

The second part of the file defines the generic commands used for defining sectioning commands such as `\chapter`. Again the actual document level commands are defined in the class files, in terms of these commands.

```
⟨*2ekernel⟩
\message{title,}
```

1.1 The Title

`\title` `\author` `\date` `\thanks` `\and` `\maketitle`

The user defines the title and author by the declarations `\title{`⟨*name*⟩`}`, `\author{`⟨*name*⟩`}` Similarly the date is declared with `\date{`⟨*date*⟩`}`.

Inside these, the `\thanks{`⟨*footnote text*⟩`}` command may be used to make acknowledgements, notice of address, etc. in a footnote. If there are multiple authors, they have to be separated with the `\and` command.

And finally, the `\maketitle` command produces the actual title, using the information previously saved with the other commands.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                    {\title}{Make commands robust}%
```

`\title` `\title` for use in `\maketitle`. If not given `\maketitle` will produce an error message.

```
\DeclareRobustCommand\title[1]{\gdef\@title{#1}}
```

(*End of definition for* `\title`. *This function is documented on page* 854.)

`\author` `\author` for use in `\maketitle`. If not given `\maketitle` will produce a warning message.

```
\DeclareRobustCommand*\author[1]{\gdef\@author{#1}}
```

(*End of definition for* `\author`.)

`\date` `\date` for use in `\maketitle`. If not given `\maketitle` will produce `\today` as the default.

```
\DeclareRobustCommand*\date[1]{\gdef\@date{#1}}
```

(*End of definition for* `\date`.)

`\thanks`

```
\DeclareRobustCommand\thanks[1]{\footnotemark
    \protected@xdef\@thanks{\@thanks
        \protect\footnotetext[\the\c@footnote]{#1}}%
}
```

(*End of definition for* `\thanks`.)

\and

```
\DeclareRobustCommand\and{%   % \begin{tabular}
  \end{tabular}%
  \hskip 1em \@plus.17fil%
  \begin{tabular}[t]{c}}%     % \end{tabular}
```

(*End of definition for* `\and`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\title}{Make commands robust}%
⟨latexrelease⟩
⟨latexrelease⟩\kernel@make@fragile\title
⟨latexrelease⟩\kernel@make@fragile\author
⟨latexrelease⟩\kernel@make@fragile\date
⟨latexrelease⟩\kernel@make@fragile\thanks
⟨latexrelease⟩\kernel@make@fragile\and
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

\@title

```
\def\@title{\@latex@error{No \noexpand\title given}\@ehc}
```

(*End of definition for* `\@title`.)

\@author

```
\def\@author{\@latex@warning@no@line{No \noexpand\author given}}
```

(*End of definition for* `\@author`.)

\@date

```
\gdef\@date{\today}
```

(*End of definition for* `\@date`.)

\@thanks

```
\let\@thanks\@empty
```

(*End of definition for* `\@thanks`.)

```
\message{sectioning,}
```

1.2 Sectioning

\@secpenalty

```
\newcount\@secpenalty
\@secpenalty = -300
```

(*End of definition for* `\@secpenalty`.)

\if@noskipsec
\@noskipsectrue

Way back in 1991 (08/26) FMi & RmS set the `@noskipsec` switch to true for the preamble and to false in `\document`. This was done to trap lists and related text in the preamble but it does not catch everything.

```
\newif\if@noskipsec \@noskipsectrue
```

(*End of definition for* `\if@noskipsec` *and* `\@noskipsectrue`.)

`\@startsection` The `\@startsection{`⟨*name*⟩`}{`⟨*level*⟩`}{ `⟨*indent*⟩`}{`⟨*beforeskip*⟩`}{`⟨*afterskip*⟩`}{`⟨*style*⟩`}*[`⟨*altheading*⟩` ]{`⟨*heading*⟩`}` command is the mother of all the user level sectioning commands. The part after the `*`, including the `*` is optional.

name: e.g., 'subsection'

level: a number, denoting depth of section – e.g., chapter = 0, section = 1, etc.

indent: Indentation of heading from left margin

beforeskip: Absolute value = skip to leave above the heading. If negative, then paragraph indent of text following heading is suppressed.

afterskip: if positive, then skip to leave below heading, else negative of skip to leave to right of run-in heading.

style: Commands to set style. Since June 1996 release the *last* command in this argument may be a command such as `\MakeUppercase` that takes an argument but doesn't start a paragraph (so `\fbox` or `\textit`, for example, wouldn't work without breaking the formatting). The section heading will be supplied as the argument to this command. So setting `#6` to, say, `\bfseries\MakeUppercase` would produce bold, uppercase headings.

If '`*`' is missing, then increment the counter. If it is present, then there should be no `[`⟨*altheading*⟩`]` argument. The command uses the counter 'secnumdepth'. It contains a pointer to the highest section level that is to be numbered.

Warning: The `\@startsection` command should be at the same or higher grouping level as the text that follows it. For example, you should *not* do something like

```
\def\foo{ \begingroup ...
            \paragraph{...}
          \endgroup}
```

Pseudocode for the `\@startsection` command *Historical LaTeX 2.09 comments (not necessarily accurate any more):*

```
 \@startsection
{NAME}{LEVEL}{INDENT}{BEFORESKIP}{AFTERSKIP}{STYLE} ==
    BEGIN
     IF  @noskipsec = T  THEN  \leavevmode  FI
                               % true if previous section had no body.

     \par
     \@tempskipa  := BEFORESKIP
     @afterindent := T
     IF \@tempskipa < 0  THEN  \@tempskipa  := -\@tempskipa
                               @afterindent := F
     FI
     IF @nobreak = true
       THEN \everypar == null
       ELSE \addpenalty{\@secpenalty}
            \addvspace{\@tempskipa}
```

```
    FI
    IF * next
      THEN \@ssect{INDENT}{BEFORESKIP}{AFTERSKIP}{STYLE}
      ELSE \@dblarg{\@sect
                {NAME}{LEVEL}{INDENT}
                {BEFORESKIP}{AFTERSKIP}{STYLE}}
    FI
 END
```

End of historical LaTeX 2.09 comments.

```
\def\@startsection#1#2#3#4#5#6{%
  \if@noskipsec \leavevmode \fi
  \par
  \@tempskipa #4\relax
  \@afterindenttrue
  \ifdim \@tempskipa <\z@
    \@tempskipa -\@tempskipa \@afterindentfalse
  \fi
  \if@nobreak
    \everypar{}%
  \else
    \addpenalty\@secpenalty\addvspace\@tempskipa
  \fi
  \@ifstar
    {\@ssect{#3}{#4}{#5}{#6}}%
    {\@dblarg{\@sect{#1}{#2}{#3}{#4}{#5}{#6}}}}
```

(*End of definition for* `\@startsection`.)

`\@sect` Pseudocode for the `\@sect` command *Historical LaTeX 2.09 comments (not necessarily accurate any more):*

```
 \@sect{NAME}{LEVEL}
       {INDENT}{BEFORESKIP}{AFTERSKIP}
       {STYLE}[ARG1]{ARG2}
            ==
   BEGIN
    IF LEVEL > \c@secnumdepth
      THEN \@svsec :=L null
      ELSE \refstepcounter{NAME}
           \@svsec :=L BEGIN \@seccntformat{#1}\relax END
    FI
    IF AFTERSKIP > 0
      THEN \begingroup
             STYLE
             \@hangfrom{\hskip INDENT\@svsec}
             {\interlinepenalty 10000 ARG2\par}
           \endgroup
           \NAMEmark{ARG1}
           \addcontentsline{toc}{NAME}
             { IF  LEVEL > \c@secnumdepth
                 ELSE \protect\numberline{\theNAME}  FI
               ARG1 }
```

```
      ELSE \@svsechd == BEGIN   STYLE
                                \hskip INDENT\@svsec
                                ARG2
                                \NAMEmark{ARG1}
                                \addcontentsline{toc}{NAME}
                                   { IF   LEVEL > \c@secnumdepth
                                       ELSE
                                         \protect\numberline{\theNAME}
                                       FI
                                     ARG1 }
                           END
   FI
   \@xsect{AFTERSKIP}
 END
```

End of historical LaTeX 2.09 comments.

```
\def\@sect#1#2#3#4#5#6[#7]#8{%
  \ifnum #2>\c@secnumdepth
    \let\@svsec\@empty
  \else
    \refstepcounter{#1}%
```

Since `\@seccntformat` might end with an improper `\hskip` which is scanning forward for `plus` or `minus` we end the definition of `\@svsec` with `\relax` as a precaution.

```
    \protected@edef\@svsec{\@seccntformat{#1}\relax}%
  \fi
  \@tempskipa #5\relax
  \ifdim \@tempskipa>\z@
    \begingroup
```

This `{` used to be after the argument to `\@hangfrom` but was moved here to allow commands such as `\MakeUppercase` to be used at the end of `#6`.

```
      #6{%
        \@hangfrom{\hskip #3\relax\@svsec}%
          \interlinepenalty \@M #8\@@par}%
    \endgroup
    \csname #1mark\endcsname{#7}%
    \addcontentsline{toc}{#1}{%
      \ifnum #2>\c@secnumdepth \else
        \protect\numberline{\csname the#1\endcsname}%
      \fi
      #7}%
  \else
```

`\relax` added 2 May 90

```
    \def\@svsechd{%
      #6{\hskip #3\relax
      \@svsec #8}%
      \csname #1mark\endcsname{#7}%
      \addcontentsline{toc}{#1}{%
        \ifnum #2>\c@secnumdepth \else
          \protect\numberline{\csname the#1\endcsname}%
        \fi
        #7}}%
```

```
\fi
\@xsect{#5}}
```

(*End of definition for* `\@sect`.)

`\@xsect` Pseudocode for the `\@xsect` command *Historical LaTeX 2.09 comments (not necessarily accurate any more):*

```
\@xsect{AFTERSKIP} ==
 BEGIN
   IF AFTERSKIP > 0
     THEN \par \nobreak
          \vskip AFTERSKIP
          \@afterheading
     ELSE @nobreak :=G F
          @noskipsec :=G T
          \everypar{ IF @noskipsec = T
                       THEN @noskipsec :=G F
                            \clubpenalty := 10000 % local
                            \hskip -\parindent
                            \begingroup
                              \@svsechd
                            \endgroup
                            \unskip
                            \hskip -AFTERSKIP \relax
                                          %% relax added 14 Jan 91
                       ELSE \clubpenalty := \@clubpenalty % local
                            \everypar := NULL
                     FI
                   }
   FI

 END
```

End of historical LaTeX 2.09 comments.

```
\def\@xsect#1{%
  \@tempskipa #1\relax
  \ifdim \@tempskipa>\z@
```

Why not combine `\@sect` and `\@xsect` and save doing the same test twice? It is not possible to change this now as these have become hooks!

This `\par` seems unnecessary.

```
    \par \nobreak
    \vskip \@tempskipa
    \@afterheading
  \else
```

```
    \@nobreakfalse
    \global\@noskipsectrue
    \everypar{%
      \if@noskipsec
        \global\@noskipsecfalse
       {\setbox\z@\lastbox}%
        \clubpenalty\@M
        \begingroup \@svsechd \endgroup
        \unskip
        \@tempskipa #1\relax
        \hskip -\@tempskipa
      \else
        \clubpenalty \@clubpenalty
        \everypar{}%
      \fi}%
  \fi
  \ignorespaces}
```

(*End of definition for* `\@xsect`.)

`\@seccntformat` This command formats the section number including the space following it.

```
\def\@seccntformat#1{\csname the#1\endcsname\quad}
```

(*End of definition for* `\@seccntformat`.)

Pseudocode for the `\@ssect` command *Historical LaTeX 2.09 comments (not necessarily accurate any more):*

```
\@ssect{INDENT}{BEFORESKIP}{AFTERSKIP}{STYLE}{ARG} ==
  BEGIN
   IF AFTERSKIP > 0
     THEN \begingroup
            STYLE
            \@hangfrom{\hskip INDENT}
                      {\interlinepenalty 10000 ARG\par}
          \endgroup
     ELSE \@svsechd == BEGIN STYLE
                           \hskip INDENT
                           ARG
                       END
   FI
   \@xsect{AFTERSKIP}
  END
```

End of historical LaTeX 2.09 comments.

Pseudocode for the `\@afterheading` command *Historical LaTeX 2.09 comments (not necessarily accurate any more):*

```
\@afterheading ==
 BEGIN
   @nobreak :=G true
   \everypar := BEGIN IF @nobreak = T
                      THEN @nobreak  :=G false
                           \clubpenalty := 10000 % local
                           IF @afterindent = F
                             THEN remove \lastbox
```

FI
ELSE \clubpenalty := \@clubpenalty % local
\everypar := NULL
FI
END
END
End of historical LaTeX 2.09 comments.

\@ssect

```
\def\@ssect#1#2#3#4#5{%
  \@tempskipa #3\relax
  \ifdim \@tempskipa>\z@
    \begingroup
```

This { used to be after the argument to \@hangfrom but was moved here to allow commands such as \MakeUppercase to be used at the end of #4.

```
      #4{%
        \@hangfrom{\hskip #1}%
          \interlinepenalty \@M #5\@@par}%
    \endgroup
  \else

    \def\@svsechd{#4{\hskip #1\relax #5}}%
  \fi
  \@xsect{#3}}
```

(*End of definition for* \@ssect.)

\if@afterindent
\@afterindenttrue

```
\newif\if@afterindent \@afterindenttrue
```

(*End of definition for* \if@afterindent *and* \@afterindenttrue.)

\@afterheading This hook is used in setting up custom-built headings in classes.dtx.

```
\def\@afterheading{%
  \@nobreaktrue
  \everypar{%
    \if@nobreak
      \@nobreakfalse
      \clubpenalty \@M
      \if@afterindent \else
        {\setbox\z@\lastbox}%
      \fi
    \else
      \clubpenalty \@clubpenalty
      \everypar{}%
    \fi}}
```

(*End of definition for* \@afterheading.)

\@hangfrom \@hangfrom{⟨*text*⟩} : Puts ⟨*text*⟩ in a box, and makes a hanging indentation of the following material up to the first \par. Should be used in vertical mode.

```
\def\@hangfrom#1{\setbox\@tempboxa\hbox{{#1}}%
      \hangindent \wd\@tempboxa\noindent\box\@tempboxa}
```

(*End of definition for* `\@hangfrom`.)

`\c@secnumdepth`
`\c@tocdepth`

```
\newcount\c@secnumdepth
\newcount\c@tocdepth
```

(*End of definition for* `\c@secnumdepth` *and* `\c@tocdepth`.)

`\secdef` `\secdef{⟨unstarcmds⟩}{⟨unstarcmds⟩}{ ⟨starcmds⟩}`
When defining a `\chapter` or `\section` command without using `\@startsection`, you can use `\secdef` as follows:

1. `\def\chapter{ ...\secdef \⟨starcmd⟩ \⟨unstarcmd⟩ }`
2. `\def\⟨starcmd⟩[#1]#2{ ...}` % Command to define `\chapter[...]{...}`
3. `\def\⟨unstarcmd⟩#1{ ...}` % Command to define `\chapter*{...}`

```
\def\secdef#1#2{\@ifstar{#2}{\@dblarg{#1}}}
```

(*End of definition for* `\secdef`.)

1.2.1 Initializations

`\sectionmark`
`\subsectionmark`
`\subsubsectionmark`
`\paragraphmark`
`\subparagraphmark`

```
\let\sectionmark\@gobble
\let\subsectionmark\@gobble
\let\subsubsectionmark\@gobble
\let\paragraphmark\@gobble
\let\subparagraphmark\@gobble
```

(*End of definition for* `\sectionmark` *and others.*)

```
\message{contents,}
```

1.3 Table of Contents etc.

1.3.1 Convention

`\tf@⟨foo⟩` = file number for output for table foo. The file is opened only if `@filesw` = `true`.

1.3.2 Commands

A `\l@⟨type⟩{⟨entry⟩}{⟨page⟩}` Macro needs to defined by document style for making an entry of type ⟨*type*⟩ in a table of contents, etc. E.g., the document style should define `\l@chapter`, `\l@section`, etc.

Note: When the `\protect` command is used in the ⟨*entry*⟩ or ⟨*text*⟩ of one of the commands below, it causes the following control sequence to be written on the file without being expanded. The sequence will be expanded when the table of contents entry is processed.

Surprise: Inside an `\addcontentsline` or `\addtocontents` command argument, the commands: `\index`, `\glossary`, and `\label` are no-ops . This could cause a problem if the user puts an `\index` or `\label` into one of the commands he writes, or into the optional 'short version' argument of a `\section` or `\caption` command.

`\@starttoc` The `\@starttoc{`⟨*ext*⟩`}` command is used to define the commands: `\tableofcontents`, `\listoffigures`, etc.

For example: `\@starttoc{lof}` is used in `\listoffigures`. This command reads the `.`⟨*ext*⟩ file and sets up to write the new `.`⟨*ext*⟩ file.

Historical LaTeX 2.09 comments (not necessarily accurate any more):

```
\@starttoc{EXT} ==
  BEGIN
    \begingroup
       \makeatletter
       read file \jobname.EXT
       IF @filesw = true
         THEN  open \jobname.EXT as file \tf@EXT
       FI
       @nobreak :=G FALSE  %% added 24 May 89
    \endgroup
  END
```

End of historical LaTeX 2.09 comments.

```
\def\@starttoc#1{%
  \begingroup
    \makeatletter
    \@input{\jobname.#1}%
    \if@filesw
      \expandafter\newwrite\csname tf@#1\endcsname
      \immediate\openout \csname tf@#1\endcsname \jobname.#1\relax
    \fi
    \@nobreakfalse
  \endgroup}
```

(*End of definition for* `\@starttoc`.)

`\addcontentsline` The `\addcontentsline{`⟨*table*⟩`}{`⟨*type*⟩`}{`⟨*entry*⟩`}` command allows the user to add his/her own entry to a table of contents, etc. The command adds the entry `\contentsline{`⟨*type*⟩`}{`⟨*entry*⟩`}{`⟨*page*⟩`}{}` to the `.`⟨*table*⟩ file.

This macro is implemented as an application of `\addtocontents`. Note that `\thepage` is not expandable during `\protected@write` therefore one gets the page number at the time of the `\shipout`.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                    {\addcontentsline}{fourth argument}%
\def\addcontentsline#1#2#3{%
```

We add an empty brace pair at the end of `\contentsline` so that the number of argument is identical in documents with and without hyperref.

```
  \addtocontents{#1}{\protect\contentsline{#2}{#3}{\thepage}{}%
                     \protected@file@percent}}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2018/12/01}%
⟨latexrelease⟩                    {\addcontentsline}{Mask line endings}%
⟨latexrelease⟩ \def\addcontentsline#1#2#3{%
⟨latexrelease⟩   \addtocontents{#1}{\protect\contentsline{#2}{#3}{\thepage}%
```

We add `\protected@file@percent` at the end which is turned inside `\@writefile` into a percent character to mask the newline after the closing argument brace.

```
⟨latexrelease⟩              \protected@file@percent}}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                   {\addcontentsline}{Mask line endings}%
⟨latexrelease⟩\def\addcontentsline#1#2#3{%
⟨latexrelease⟩  \addtocontents{#1}{\protect\contentsline{#2}{#3}{\thepage}}}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\addcontentsline`.)

`\@gobble@om` `\@gobble@som` `\@gobble@with@sphack@om` `\@gobble@with@sphack@som`

Each of these four commands accepts an optional and a mandatory argument, possibly preceded by a star. In all cases the expansion is empty or just manipulates the spaces around the command. Used to disable commands such as `\index` or `\label` in certain situations.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease⟩                   {\@gobble@som}{Extended index/label}%
```

Just getting rid of the input in an expandable way is done by these two.

```
\DeclareExpandableDocumentCommand\@gobble@om{+o+m}{}
\DeclareExpandableDocumentCommand\@gobble@som{s+o+m}{}
```

When something needs to be suppressed during typesetting, one often also wants to ensure that this doesn't end up with two spaces (in case there is one before and one after). This can't be done expandably.

```
\DeclareDocumentCommand\@gobble@with@sphack@om{+o+m}{\@bsphack\@esphack}
\DeclareDocumentCommand\@gobble@with@sphack@som{s+o+m}{\@bsphack\@esphack}
```

(*End of definition for* `\@gobble@om` *and others.*)

`\addtocontents` The `\addtocontents{`⟨*table*⟩`}{`⟨*text*⟩`}` command adds ⟨*text*⟩ to the `.`⟨*table*⟩ file, with no page number.

```
\long\def\addtocontents#1#2{%
  \protected@write\@auxout
      {\let\label\@gobble@om
       \let\index\@gobble@som
       \let\glossary\@gobble@om}%
      {\string\@writefile{#1}{#2}}}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                   {\@gobble@som}{Extended index/label}%
⟨latexrelease⟩\long\def\addtocontents#1#2{%
⟨latexrelease⟩  \protected@write\@auxout
⟨latexrelease⟩      {\let\label\@gobble \let\index\@gobble \let\glossary\@gobble}%
⟨latexrelease⟩      {\string\@writefile{#1}{#2}}}
⟨latexrelease⟩
⟨latexrelease⟩\let\@gobble@om\@undefined
⟨latexrelease⟩\let\@gobble@som\@undefined
⟨latexrelease⟩\let\@gobble@with@sphack@om\@undefined
```

```
⟨latexrelease⟩\let\@gobble@with@sphack@som\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\addtocontents`.)

`\contentsline` The `\contentsline{`⟨*type*⟩`}{`⟨*entry*⟩`}{`⟨*page*⟩`}{}` macro produces a ⟨*type*⟩ entry in a table of contents, etc. It will appear in the `.toc` or other file. For example, the entry for subsection 1.4.3 in the table of contents, might be produced by:

```
\contentsline{subsection}
    {\numberline{1.4.3}Gnats and Gnus}{22}{}
```

The `\protect` command causes command sequences to be written without expanding them.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2021/11/15}%
⟨latexrelease⟩                    {\contentsline}{Four arguments}%
```

In the toc file `\contentsline` is followed by 4 arguments these days, but only the first 3 are used in the old interface. The fourth was by default empty and only used when hyperref was loaded. We now pick up all 4 arguments, save the last one away in `\@contentsline@destination` and then call the old interface. This is done to simplify the interface to hyperref and to prepare for future changes.

```
\def\contentsline#1#2#3#4{\gdef\@contentsline@destination{#4}%
                          \csname l@#1\endcsname{#2}{#3}}
```

Default definition.

```
\let\@contentsline@destination\@empty
```

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\contentsline}{Four arguments}%
⟨latexrelease⟩
⟨latexrelease⟩\def\contentsline#1{\csname l@#1\endcsname}
⟨latexrelease⟩\let\@contentsline@destination\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\contentsline`.)

`\@dottedtocline{`⟨*level*⟩`}{`⟨*indent*⟩`}{`⟨*numwidth*⟩`}{`⟨*title*⟩`}{`⟨*page*⟩`}`: Macro to produce a table of contents line with the following parameters:

level If ⟨*level*⟩ > `\c@tocdepth`, then no line produced.

indent Total indentation from the left margin.

numwidth Width of box for number if the ⟨*title*⟩ has a `\numberline` command. As of 25 Jan 1988, this is also the amount of extra indentation added to second and later lines of a multiple line entry.

title Contents of entry.

page Page number.

Uses the following parameters, which must be set by the document style. They should be defined with `\def`'s.

`pnumwidth` Width of box in which page number is set.

`tocrmarg` Right margin indentation for all but last line of multiple-line entries.

`dotsep` Separation between dots, in mu units. Should be `\def`'d to a number like 2 or 1.7

`\@dottedtocline`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2018/12/01}%
⟨latexrelease⟩                    {\@dottedtocline}{Prevent protrusion}%
\def\@dottedtocline#1#2#3#4#5{%
  \ifnum #1>\c@tocdepth \else
    \vskip \z@ \@plus.2\p@
    {\leftskip #2\relax \rightskip \@tocrmarg \parfillskip -\rightskip
     \parindent #2\relax\@afterindenttrue
     \interlinepenalty\@M
     \leavevmode
     \@tempdima #3\relax

     \advance\leftskip \@tempdima \null\nobreak\hskip -\leftskip
     {#4}\nobreak
     \leaders\hbox{$\m@th
```

If a document uses fonts other than computer modern, the use of a dot from math can be very disturbing despite the fact that this might be the only place in a document that then uses computer modern. Therefore we surround the dot with an `\hbox` to escape to the surrounding text font.

```
        \mkern \@dotsep mu\hbox{.}\mkern \@dotsep
        mu$}\hfill
     \nobreak
     \hb@xt@\@pnumwidth{\hfil\normalfont \normalcolor #5%
```

We finish off by preventing any protrusion if that is enabled. If protrusion happens the number may shift to the right and as a result you may end up with an additional dot in the toc line in some situations.

```
                          \kern-\p@\kern\p@}%
    \par}%
  \fi}
```

(*End of definition for* `\@dottedtocline`.)

`\noprotrusion` This command, if placed directly to the right (or left) of a word, will prevent protrusion of that word into the margin. It is used in the toc entry lines as they shouldn't protrude. It is implemented as to kerns that cancel each other but being there hide the word so that protrusion is not added. Note that a zero kern or an empty box would not work as the protrusion mechanism will skip over those.

```
\DeclareRobustCommand\noprotrusion{\leavevmode\kern-\p@\kern\p@}
```

(*End of definition for* `\noprotrusion`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@dottedtocline}{Prevent protrusion}%
⟨latexrelease⟩\def\@dottedtocline#1#2#3#4#5{%
⟨latexrelease⟩  \ifnum #1>\c@tocdepth \else
⟨latexrelease⟩    \vskip \z@ \@plus.2\p@
⟨latexrelease⟩    {\leftskip #2\relax \rightskip \@tocrmarg \parfillskip -\rightskip
⟨latexrelease⟩     \parindent #2\relax\@afterindenttrue
⟨latexrelease⟩     \interlinepenalty\@M
⟨latexrelease⟩     \leavevmode
⟨latexrelease⟩     \@tempdima #3\relax
⟨latexrelease⟩     \advance\leftskip \@tempdima \null\nobreak\hskip -\leftskip
⟨latexrelease⟩     {#4}\nobreak
⟨latexrelease⟩     \leaders\hbox{$\m@th
⟨latexrelease⟩        \mkern \@dotsep mu\hbox{.}\mkern \@dotsep
⟨latexrelease⟩        mu$}\hfill
⟨latexrelease⟩     \nobreak
⟨latexrelease⟩     \hb@xt@\@pnumwidth{\hfil\normalfont \normalcolor #5}%
⟨latexrelease⟩     \par}%
⟨latexrelease⟩  \fi}
⟨latexrelease⟩
⟨latexrelease⟩\let\noprotrusion\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

Note: `\nobreak`'s added 7 Jan 86 to prevent bad line break that left the page number dangling by itself at left edge of a new line.

Changed 25 Jan 88 to use `\leftskip` instead of `\hangindent` so leaders of multiple-line contents entries would line up properly.

`\numberline` `\numberline{`⟨*number*⟩`}`: For use in a `\contentsline` command. It puts ⟨*number*⟩ flush-left in a box of width `\@tempdima` (Before 25 Jan 88 change, it also added `\@tempdima` to the hanging indentation.)

```
\def\numberline#1{\hb@xt@\@tempdima{#1\hfil}}
⟨/2ekernel⟩
```

(*End of definition for* `\numberline`.)

File 45
ltfloat.dtx

1 Floats

The different types of floats are identified by a ⟨*type*⟩ name, which is the name of the counter for that kind of float. For example, figures are of type 'figure' and tables are of type 'table'. Each ⟨*type*⟩ has associated a positive ⟨*type number*⟩, which is a power of two. E.g.,
figures might be have type number 1, tables type number 2, programs type number 4, etc.

The locations where a float can go are specified by a ⟨*placement specifier*⟩, which is a list of the possible locations, each denoted by a letter as follows:

h : here — at the current location in the text.
t : top — at the top of a text page.
b : bottom — at the bottom of a text page.
p : page — on a separate float page

In addition, in conjunction with these, you can use '!' which means that the current values of the float positioning parameters are ignored for this float. (Has no effect on 'p', float page positioning.) For example, 'pht' specifies that the float can appear in any of three locations: page, here or top.

1.1 Floating Environments

```
⟨*2ekernel⟩
\message{floats,}
```

Historical LaTeX 2.09 comments (not necessarily accurate any more):

Where floats may appear on a page, and how many may appear there are specified by the following float placement parameters. The numbers are named like counters so the user can set them with the ordinary counter-setting commands.

`\c@topnumber` : Number of floats allowed at the top of a column.
`\topfraction` : Fraction of column that can be devoted to floats.
`\c@dbltopnumber, \dbltopfraction`
: Same as above, but for double-column floats.
`\c@bottomnumber, \bottomfraction`
: Same as above for bottom of page.
`\c@totalnumber` : Number of floats allowed in a single column, including in-text floats.
`\textfraction` :Minimum fraction of column that must contain text.
`\floatpagefraction`: Minimum fraction of page that must be taken up by float page.
`\dblfloatpagefraction`
: Same as above, for double-column floats.

The document style must define the following.

`\fps@TYPE` : The default placement specifier for floats of type TYPE.

`\ftype@TYPE` : The type number for floats of type TYPE.

`\ext@TYPE` : The file extension indicating the file on which the contents list for float type TYPE is stored. For example, `\ext@figure` = 'lof'.

`\fnum@TYPE` : A macro to generate the figure number for a caption. For example, `\fnum@TYPE` == Figure `\thefigure.`

`\@makecaption{NUM}{TEXT}` : A macro to make a caption, with NUM the value produced by `\fnum@...` and TEXT the text of the caption. It can assume it's in a `\parbox` of the appropriate width.

`\@float{TYPE}`[PLACEMENT] : This macro begins a float environment for a single-column float of type TYPE with PLACEMENT as the placement specifier. The default value of PLACEMENT is defined by `\fps@TYPE.` The environment is ended by `\end@float.` E.g., `\figure` == `\@float{`figure`}`, `\endfigure` == `\end@float.`

```
\@float{TYPE}[PLACEMENT] ==
 BEGIN
   if hmode then \@bsphack
                 \@floatpenalty := -10002
            else \@floatpenalty := -10003
   fi
   \@captype ==L TYPE
   \@dblflset
   \@fps     ==L PLACEMENT
   \@onelevel@sanitize \@fps
   add default PLACEMENT if at most ! in PLACEMENT ==
\@fpsadddefault
   if inner
     then LaTeX Error: 'Not in outer paragraph mode.'
          \@floatpenalty := 0
     else if \@freelist nonempty
            then \@currbox  :=L head of \@freelist
                 \@freelist :=G tail of \@freelist
                 \count\@currbox :=G 32*\ftype@TYPE +
                                        bits determined by PLACEMENT
            else \@floatpenalty := 0
                 LaTeX Error: 'Too many unprocessed floats'
          fi
   fi
   \@currbox :=G   \color@vbox
```

```
                                \normalcolor
                                  \vbox{
                                   %% 15 Dec 87 –
                                   %% removed \boxmaxdepth :=L 0pt
                                   %% that made box 0 depth because it screwed
                                   %% things up. Instead, added \vskip0pt at end
                                        \hsize = \columnwidth
                                        \@parboxrestore
                                        \@floatboxreset
```

END

```
\caption ==
  BEGIN
   \refstepcounter{\@captype}
   \@dblarg{\@caption{\@captype}}
  END
```

In following definition, `\par` moved from after `\addcontentsline` to before `\addcontentsline` because the `\write` could cause an extra blank line to be added to the paragraph above the caption. (Change made 12 Jun 87)

```
\@caption{TYPE}[STEXT]{TEXT} ==
 BEGIN
   \par
   \addcontentsline{\ext@TYPE}{TYPE}{\numberline{\theTYPE}{STEXT}}
   \begingroup
     \@parboxrestore
     \@normalsize
     \@makecaption{\fnum@TYPE}{TEXT}
     \par
   \endgroup
 END
```

`\@dblfloat{TYPE}`[PLACEMENT] : Macro to begin a float environment for a double-column float of type TYPE with PLACEMENT as the placement specifier. The default value of PLACEMENT is 'tp'
The environment is ended by `\end@dblfloat.`
E.g., `\figure*` == `\@dblfloat{`figure}, `\endfigure*` == `\end@dblfloat.`

`\@dblfloat{TYPE}`[PLACEMENT] ==
Identical to `\@float{TYPE}`[PLACEMENT] except `\hsize` and `\linewidth` are set to `\textwidth.`

End of historical LaTeX 2.09 comments.

`\@floatpenalty`

```
3 \newcount\@floatpenalty
```

(*End of definition for* `\@floatpenalty`.)

\caption This is set to be an error message outside a float since no captype is defined there; this may need to be changed by some classes.

```
\def\caption{%
   \ifx\@captype\@undefined
     \@latex@error{\noexpand\caption outside float}\@ehd
     \expandafter\@gobble
   \else
     \refstepcounter\@captype
     \expandafter\@firstofone
   \fi
   {\@dblarg{\@caption\@captype}}%
}
```

(*End of definition for* `\caption`.)

\@caption

```
\long\def\@caption#1[#2]#3{%
  \par
  \addcontentsline{\csname ext@#1\endcsname}{#1}%
    {\protect\numberline{\csname the#1\endcsname}{\ignorespaces #2}}%
  \begingroup
```

The paragraph setting parameters are normalized at this point, however `\@parboxrestore` resets `\everypar` which is not correct in this context so `\@setminipage` is called if needed.

The float mechanism, like minipage, sets the flag `@minipage` true before executing the user-supplied text. Many LaTeX constructs test for this flag and do not add vertical space when it is true. The intention is that this emulates TeX's 'top of page' behaviour. The flag must be set false at the start of the first paragraph. This is achieved by a redefinition of `\everypar`, but the call to `\@parboxrestore` removes that redefinition, so it is re-inserted if needed. If the flag is already false then the `\caption` was not the first entry in the float, and so some other paragraph has already activated the special `\everypar`. In this case no further action is needed.

```
    \@parboxrestore
    \if@minipage
      \@setminipage
    \fi
    \normalsize
    \@makecaption{\csname fnum@#1\endcsname}{\ignorespaces #3}\par
  \endgroup}
```

(*End of definition for* `\@caption`.)

\@float
\@dblflset

```
\def\@float#1{%
  \@ifnextchar[%
    {\@xfloat{#1}}%
    {\edef\reserved@a{\noexpand\@xfloat{#1}[\csname fps@#1\endcsname]}%
     \reserved@a}}
```

(*End of definition for* `\@float` *and* `\@dblflset`.)

\@dblfloat

```
\def\@dblfloat{%
  \if@twocolumn\let\reserved@a\@dbflt\else\let\reserved@a\@float\fi
  \reserved@a}
```

(*End of definition for* `\@dblfloat`.)

`\fps@dbl` Note that all double floats have default fps 'tp'.

(*End of definition for* `\fps@dbl`.)

`\@setfps` This sets the fps, dealing with error conditions by adding the default.

(*End of definition for* `\@setfps`.)

`\@xfloat` The first part of this sets the count register that stores all the information about the type and fps of the float.

We assume here that the default specifiers already contain no active characters. It may be better to store the defaults as numbers, rather than symbol strings.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩                   {\@xfloat}{Check float options}%
⟨*2ekernel | latexrelease⟩
\def\@xfloat #1[#2]{%
  \@nodocument
  \def \@captype {#1}%
   \def \@fps {#2}%
   \@onelevel@sanitize \@fps
   \def \reserved@b {!}%
   \ifx \reserved@b \@fps
     \@fpsadddefault
   \else
     \ifx \@fps \@empty
       \@fpsadddefault
     \fi
   \fi
   \ifhmode
     \@bsphack
     \@floatpenalty -\@Mii
   \else
     \@floatpenalty-\@Miii
   \fi
  \ifinner
     \@parmoderr\@floatpenalty\z@
  \else
    \@next\@currbox\@freelist
      {%
       \@tempcnta \sixt@@n
       \expandafter \@tfor \expandafter \reserved@a
         \expandafter :\expandafter =\@fps
         \do
```

Start of changes, use a nested if structure, ending in an error.

```
           {%
            \if \reserved@a h%
              \ifodd \@tempcnta
              \else
                \advance \@tempcnta \@ne
              \fi
```

```
            \else\if \reserved@a t%
              \@setfpsbit \tw@
            \else\if \reserved@a b%
              \@setfpsbit 4%
            \else\if \reserved@a p%
              \@setfpsbit 8%
            \else\if \reserved@a !%
              \ifnum \@tempcnta>15
                \advance\@tempcnta -\sixt@@n\relax
              \fi
            \else
              \@latex@error{Unknown float option `\reserved@a'}%
              {Option `\reserved@a' ignored and `p' used.}%
              \@setfpsbit 8%
            \fi\fi\fi\fi\fi
            }%
```

End of changes

```
        \@tempcntb \csname ftype@\@captype \endcsname
        \multiply \@tempcntb \@xxxii
        \advance \@tempcnta \@tempcntb
        \global \count\@currbox \@tempcnta
        }%
    \@fltovf
  \fi
```

The remainder sets up the box in which the float is typeset, and the typesetting environment to be used. It is essential to have the extra box to avoid the unwanted space that would otherwise often be put at the top of the float.

It ends with a hook; not sure how useful this is but it is needed at present to deal with double-column floats.

```
  \global \setbox\@currbox
    \color@vbox
      \normalcolor
      \vbox \bgroup
        \hsize\columnwidth
        \@parboxrestore
        \@floatboxreset
}%
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@xfloat}{Check float options}%
⟨latexrelease⟩\def\@xfloat #1[#2]{%
⟨latexrelease⟩  \@nodocument
⟨latexrelease⟩  \def \@captype {#1}%
⟨latexrelease⟩   \def \@fps {#2}%
⟨latexrelease⟩   \@onelevel@sanitize \@fps
⟨latexrelease⟩   \def \reserved@b {!}%
⟨latexrelease⟩   \ifx \reserved@b \@fps
⟨latexrelease⟩     \@fpsadddefault
⟨latexrelease⟩   \else
⟨latexrelease⟩     \ifx \@fps \@empty
⟨latexrelease⟩       \@fpsadddefault
```

```
⟨latexrelease⟩      \fi
⟨latexrelease⟩    \fi
⟨latexrelease⟩    \ifhmode
⟨latexrelease⟩      \@bsphack
⟨latexrelease⟩      \@floatpenalty -\@Mii
⟨latexrelease⟩    \else
⟨latexrelease⟩      \@floatpenalty-\@Miii
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \ifinner
⟨latexrelease⟩      \@parmoderr\@floatpenalty\z@
⟨latexrelease⟩  \else
⟨latexrelease⟩    \@next\@currbox\@freelist
⟨latexrelease⟩      {%
⟨latexrelease⟩       \@tempcnta \sixt@@n
⟨latexrelease⟩       \expandafter \@tfor \expandafter \reserved@a
⟨latexrelease⟩         \expandafter :\expandafter =\@fps
⟨latexrelease⟩         \do
⟨latexrelease⟩          {%
⟨latexrelease⟩           \if \reserved@a h%
⟨latexrelease⟩             \ifodd \@tempcnta
⟨latexrelease⟩             \else
⟨latexrelease⟩               \advance \@tempcnta \@ne
⟨latexrelease⟩             \fi
⟨latexrelease⟩           \fi
⟨latexrelease⟩           \if \reserved@a t%
⟨latexrelease⟩             \@setfpsbit \tw@
⟨latexrelease⟩           \fi
⟨latexrelease⟩           \if \reserved@a b%
⟨latexrelease⟩             \@setfpsbit 4%
⟨latexrelease⟩           \fi
⟨latexrelease⟩           \if \reserved@a p%
⟨latexrelease⟩             \@setfpsbit 8%
⟨latexrelease⟩           \fi
⟨latexrelease⟩           \if \reserved@a !%
⟨latexrelease⟩             \ifnum \@tempcnta>15
⟨latexrelease⟩               \advance\@tempcnta -\sixt@@n\relax
⟨latexrelease⟩             \fi
⟨latexrelease⟩           \fi
⟨latexrelease⟩           }%
⟨latexrelease⟩       \@tempcntb \csname ftype@\@captype \endcsname
⟨latexrelease⟩       \multiply \@tempcntb \@xxxii
⟨latexrelease⟩       \advance \@tempcnta \@tempcntb
⟨latexrelease⟩       \global \count\@currbox \@tempcnta
⟨latexrelease⟩       }%
⟨latexrelease⟩    \@fltovf
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \global \setbox\@currbox
⟨latexrelease⟩    \color@vbox
⟨latexrelease⟩      \normalcolor
⟨latexrelease⟩      \vbox \bgroup
⟨latexrelease⟩        \hsize\columnwidth
⟨latexrelease⟩        \@parboxrestore
⟨latexrelease⟩        \@floatboxreset
⟨latexrelease⟩}%
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\@xfloat`.)

`\@floatboxreset` The rational for allowing these normally global flags to be set locally here, via `\@parboxrestore`, was stated originally by Donald Arseneau and extended by Chris Rowley. It is because these flags are only set globally to true by section commands, and these should never appear within marginals or floats or, indeed, in any group; and they are only ever set globally to false when they are definitely true.

If anyone is unhappy with this argument then both flags should be treated as in `\@setnobreak`; otherwise this command will be redundant.

```
\def \@floatboxreset {%
        \reset@font
        \normalsize
        \@setminipage
}
```

(*End of definition for* `\@floatboxreset`.)

`\@setnobreak`

```
\def \@setnobreak{%
  \if@nobreak
    \let\outer@nobreak\@nobreaktrue
    \@nobreakfalse
  \fi
}
```

(*End of definition for* `\@setnobreak`.)

`\@setminipage`

```
\def \@setminipage{%
  \@minipagetrue
  \everypar{\@minipagefalse\everypar{}}%
}
```

(*End of definition for* `\@setminipage`.)

`\end@float`

```
\def\end@float{%
  \@endfloatbox
  \ifnum\@floatpenalty <\z@
```

We make sure that we never exceed `\textheight`, otherwise float will never get typeset (91/03/15 FMi).

```
    \@largefloatcheck
    \@cons\@currlist\@currbox
    \ifnum\@floatpenalty <-\@Mii
      \penalty -\@Miv
```

Saving and restoring `\prevdepth` added 26 May 87 to prevent extra vertical space when used in vertical mode.

```
      \@tempdima\prevdepth
      \vbox{}%
      \prevdepth\@tempdima
```

```
      \penalty\@floatpenalty

    \else
      \vadjust{\penalty -\@Miv \vbox{}\penalty\@floatpenalty}\@Esphack
    \fi
  \fi
}
```

(*End of definition for* `\end@float`.)

`\end@dblfloat`

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩                   {\end@dblfloat}{float order in 2-column}%
⟨*2ekernel | latexrelease⟩
\def\end@dblfloat{%
  \if@twocolumn
    \@endfloatbox
    \ifnum\@floatpenalty <\z@
      \@largefloatcheck
```

Force the depth of two column float boxes.

```
      \global\dp\@currbox1sp %
```

What follows is essentially `\end@float` without a starting `\@endfloatbox`.

```
      \@cons\@currlist\@currbox
      \ifnum\@floatpenalty <-\@Mii
        \penalty -\@Miv
        \@tempdima\prevdepth
        \vbox{}%
        \prevdepth\@tempdima
        \penalty\@floatpenalty
      \else
        \vadjust{\penalty -\@Miv \vbox{}\penalty\@floatpenalty}\@Esphack
      \fi

    \fi
  \else
    \end@float
  \fi
}%
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                   {\end@dblfloat}{float order in 2-column}%
⟨latexrelease⟩\def\end@dblfloat{%
⟨latexrelease⟩\if@twocolumn
⟨latexrelease⟩  \@endfloatbox
⟨latexrelease⟩  \ifnum\@floatpenalty <\z@
```

We make sure that we never exceed `\textheight`, otherwise float will never get typeset (91/03/15 FMi).

```
⟨latexrelease⟩    \@largefloatcheck
⟨latexrelease⟩    \@cons\@dbldeferlist\@currbox
⟨latexrelease⟩  \fi
```

RmS 92/03/18 changed `\@esphack` to `\@Esphack`.

```
⟨latexrelease⟩    \ifnum \@floatpenalty =-\@Mii \@Esphack\fi
⟨latexrelease⟩\else
⟨latexrelease⟩  \end@float
⟨latexrelease⟩\fi
⟨latexrelease⟩}%
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\end@dblfloat`.)

`\@endfloatbox` This macro is not intended to be a hook; it is designed to help maintain the integrity of this code, which is used twice and, as can be seen, is subject to frequent changes.

```
\def \@endfloatbox{%
      \par\vskip\z@skip      %% \par\vskip\z@ added 15 Dec 87
      \@minipagefalse
      \outer@nobreak
    \egroup                  %% end of vbox
  \color@endbox
}
```

(*End of definition for* `\@endfloatbox`.)

`\outer@nobreak`

```
\let\outer@nobreak\@empty
```

(*End of definition for* `\outer@nobreak`.)

`\@largefloatcheck` This calculates by how much a float is oversize for the page and prints this in a warning message.

```
\def \@largefloatcheck{%
  \ifdim \ht\@currbox>\textheight
    \@tempdima -\textheight
    \advance \@tempdima \ht\@currbox
    \@latex@warning {Float too large for page by \the\@tempdima}%
    \ht\@currbox \textheight
  \fi
}
```

(*End of definition for* `\@largefloatcheck`.)

`\@dbflt`
`\@xdblfloat`

```
\def\@dbflt#1{\@ifnextchar[{\@xdblfloat{#1}}{\@xdblfloat{#1}[tp]}}
\def\@xdblfloat#1[#2]{%
  \@xfloat{#1}[#2]\hsize\textwidth\linewidth\textwidth}
```

(*End of definition for* `\@dbflt` *and* `\@xdblfloat`.)

Moved to ltoutput 93/12/16

```
%\newcount\c@topnumber
%\newcount\c@dbltopnumber
%\newcount\c@bottomnumber
%\newcount\c@totalnumber
```

`\@floatplacement` An analysis of `\@floatplacement`:

This should be called whenever `\@colht` has been set.

```
\def\@floatplacement{\global\@topnum\c@topnumber
    % Textpage bit, global:
   \global\@toproom \topfraction\@colht
   \global\@botnum  \c@bottomnumber
   \global\@botroom \bottomfraction\@colht
   \global\@colnum  \c@totalnumber
    % Floatpage bit, local:
   \@fpmin   \floatpagefraction\@colht}
⟨/2ekernel⟩
```

(*End of definition for* `\@floatplacement`.)

`\@dblfloatplacement` This should be called only within a group. Now changed to provide extra checks in `\@addtodblcol`, needed when processing a BANG float.

```
⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩           {\@dblfloatplacement}{float order in 2-column}%
⟨*2ekernel | latexrelease⟩
```

When making two column float area, look for floats with 1sp depth.

```
\def\@dblfloatplacement{\global\@dbltopnum\c@dbltopnumber
   \global\@dbltoproom \dbltopfraction\@colht
   \@textmin \@colht
   \advance \@textmin -\@dbltoproom
   \@fpmin \dblfloatpagefraction\textheight
   \@fptop \@dblfptop
   \@fpsep \@dblfpsep
   \@fpbot \@dblfpbot
```

`\f@depth` is used in `\@testwrongwidth` to look for either column or dbl-column floats. A value of `1sp` signals the latter. Because of this setting here, `\@dblfloatplacment` needs to be called inside a group which is a questionable design.

```
    \def\f@depth{1sp}}%
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩           {\@dblfloatplacement}{float order in 2-column}%
⟨latexrelease⟩\def \@dblfloatplacement {%
```

Textpage bit: global, but need not be.

```
⟨latexrelease⟩  \global \@dbltopnum \c@dbltopnumber
⟨latexrelease⟩  \global \@dbltoproom \dbltopfraction\@colht
```

This new bit uses `\@textmin` to locally store the amount of extra room in the column.

```
⟨latexrelease⟩  \@textmin \@colht
⟨latexrelease⟩  \advance \@textmin -\@dbltoproom
```

Floatpage bit: must be local.

```
⟨latexrelease⟩  \@fpmin \dblfloatpagefraction\textheight
⟨latexrelease⟩  \@fptop \@dblfptop
⟨latexrelease⟩  \@fpsep \@dblfpsep
⟨latexrelease⟩  \@fpbot \@dblfpbot
⟨latexrelease⟩}%
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@dblfloatplacement`.)

Historical LaTeX 2.09 comments (not necessarily accurate any more):

MARGINAL NOTES:

Marginal notes use the same mechanism as floats to communicate with the `\output` routine. Marginal notes are distinguished from floats by having a negative placement specification. The command `\marginpar` [LTEXT]{RTEXT} generates a marginal note in a parbox, using LTEXT if it's on the left and RTEXT if it's on the right. (Default is RTEXT = LTEXT.) It uses the following parameters.

`\marginparwidth` : Width of marginal notes.
`\marginparsep` : Distance between marginal note and text.
the page layout to determine how to move the marginal note into the margin. E.g., `\@leftmarginskip` == `\hskip -\marginparwidth \hskip -\marginparsep` .
`\marginparpush` : Minimum vertical separation between `\marginpar`'s

Marginal notes are normally put on the outside of the page if @mparswitch = true, and on the right if @mparswitch = false. The command `\reversemarginpar` reverses the side where they are put. `\normalmarginpar` undoes `\reversemarginpar`. These commands have no effect for two-column output.

SURPRISE: if two marginal notes appear on the same line of text, then the second one could appear on the next page, in a funny position.

```
\marginpar [LTEXT]{RTEXT} ==
 BEGIN
   if hmode then \@bsphack
                 \@floatpenalty := -10002
            else \@floatpenalty := -10003
   fi
   if inner
     then LaTeX Error: 'Not in outer paragraph mode.'
          \@floatpenalty := 0
     else if \@freelist has two elements:
             then get \@marbox, \@currbox  from \@freelist
                  \count\@marbox :=G -1
             else \@floatpenalty := 0
                  LaTeX Error: 'Too many unprocessed floats'
                  \@currbox, \@marbox := \@tempboxa    %%use \def
          fi
   fi
   if optional argument
     then %% \@xmpar ==
          \@savemarbox\@marbox{LTEXT}
          \@savemarbox\@currbox{RTEXT}
```

```
      else %% \@ympar ==
           \@savemarbox\@marbox{RTEXT}
           \box\@currbox :=G \box\@marbox
    fi
    \@xympar
  END

\reversemarginpar == BEGIN \@mparbottom    :=G 0
                           @reversemargin :=G true
                     END

\normalmarginpar  == BEGIN \@mparbottom    :=G 0
                           @reversemargin :=G false
                     END
```

End of historical LaTeX 2.09 comments.

\marginpar

```
\def\marginpar{%
  \ifhmode
    \@bsphack
    \@floatpenalty -\@Mii
  \else
    \@floatpenalty-\@Miii
  \fi
  \ifinner
    \@parmoderr
    \@floatpenalty\z@
  \else
    \@next\@currbox\@freelist{}{}%
    \@next\@marbox\@freelist{\global\count\@marbox\m@ne}%
       {\@floatpenalty\z@
        \@fltovf\def\@currbox{\@tempboxa}\def\@marbox{\@tempboxa}}%
  \fi
  \@ifnextchar [\@xmpar\@ympar}
```

(*End of definition for* \marginpar.)

\@xmpar

```
\long\def\@xmpar[#1]#2{%
  \@savemarbox\@marbox{#1}%
  \@savemarbox\@currbox{#2}%
  \@xympar}
```

(*End of definition for* \@xmpar.)

\@ympar

```
\long\def\@ympar#1{%
  \@savemarbox\@marbox{#1}%
  \global\setbox\@currbox\copy\@marbox
  \@xympar}
```

(*End of definition for* \@ympar.)

\@savemarbox

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩                  {\@savemarbox}{Explicit par for marginpar}%
\long\def \@savemarbox #1#2{%
  \global\setbox #1%
    \color@vbox
      \vtop{%
        \hsize\marginparwidth
        \@parboxrestore
        \@marginparreset
        #2\par
        \@minipagefalse
        \outer@nobreak
        }%
    \color@endbox
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@savemarbox}{Explicit par for marginpar}%
⟨latexrelease⟩
⟨latexrelease⟩\long\def \@savemarbox #1#2{%
⟨latexrelease⟩  \global\setbox #1%
⟨latexrelease⟩    \color@vbox
⟨latexrelease⟩      \vtop{%
⟨latexrelease⟩        \hsize\marginparwidth
⟨latexrelease⟩        \@parboxrestore
⟨latexrelease⟩        \@marginparreset
⟨latexrelease⟩        #2%
⟨latexrelease⟩        \@minipagefalse
⟨latexrelease⟩        \outer@nobreak
⟨latexrelease⟩        }%
⟨latexrelease⟩    \color@endbox
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@savemarbox`.)

`\@marginparreset` The rational for allowing these normally global flags to be set locally here, via `\@parboxrestore` was stated originally by Donald Arseneau and extended by Chris Rowley. It is because these flags are only set globally to true by section commands, and these should never appear within marginals or floats or, indeed, in any group; and they are only ever set globally to false when they are definitely true.

If anyone is unhappy with this argument then both flags should be treated as in `\@setnobreak`; otherwise this command will be redundant.

```
\def \@marginparreset {%
        \reset@font
        \normalsize
%        \let\if@nobreak\iffalse
%        \let\if@noskipsec\iffalse
```

```
%         \@setnobreak
          \@setminipage
}
```

(*End of definition for* `\@marginparreset`.)

`\@xympar`

Setting the box here is done only because the code uses `\end@float`; it will be empty and gets discarded.

```
\def \@xympar{%
  \ifnum\@floatpenalty <\z@\@cons\@currlist\@marbox\fi
  \setbox\@tempboxa
    \color@vbox
      \vbox \bgroup
  \end@float
  \@ignorefalse
  \@esphack
}
```

(*End of definition for* `\@xympar`.)

`\reversemarginpar`
`\normalmarginpar`

```
\def\reversemarginpar{\global\@mparbottom\z@ \@reversemargintrue}
\def\normalmarginpar{\global\@mparbottom\z@ \@reversemarginfalse}
```

(*End of definition for* `\reversemarginpar` *and* `\normalmarginpar`.)

```
\message{footnotes,}
```

1.2 Footnotes

Historical LaTeX 2.09 comments (not necessarily accurate any more):

`\footnote{NOTE}` : User command to insert a footnote.

`\footnote[NUM]{NOTE}`: User command to insert a footnote numbered NUM, where NUM is a number – 1, 2, etc. For example, if footnotes are numbered *, **, etc. within pages, then `\footnote[2]{...}` produces footnote '**'. This command does not step the footnote counter.

`\footnotemark[NUM]` : Command to produce just the footnote mark in the text, but no footnote. With no argument, it steps the footnote counter before generating the mark.

`\footnotetext[NUM]{TEXT}` : Command to produce the footnote but no mark. `\footnote` is equivalent to `\footnotemark \footnotetext` .

As in PLAIN, footnotes use `\insert\footins,` and the following parameters:

`\footnotesize` : Size-changing command for footnotes.

`\footnotesep` : The height of a strut placed at the beginning of every footnote.

`\skip\footins` : Space between main text and footnotes. The rule separating footnotes from text occurs in this space. This space lies above the strut of height `\footnotesep` which is at the beginning of the first footnote.

`\footnoterule` : Macro to draw the rule separating footnotes from text. It is executed right after a `\vspace` of `\skip\footins`. It should take zero vertical space–i.e., it should to a negative skip to compensate for any positive space it occupies. (See PLAIN.TEX.)

`\interfootnotelinepenalty` : Interline penalty for footnotes.

`\thefootnote` : In usual LaTeX style, produces the footnote number. If footnotes are to be numbered within pages, then the document style file must include an `\@addtoreset` command to cause the footnote counter to be reset when the page counter is stepped. This is not a good idea, though, because the counter will not always be reset in time to ensure that the first footnote on a page is footnote number one.

`\@thefnmark` : Holds the current footnote's mark–e.g., `\dag` or '1' or 'a'.

`\@mpfnnumber` : A macro that generates the numbers for `\footnote` and `\footnotemark` commands. It == `\thefootnote` outside a minipage environment, but can be changed inside to generate numbers for `\footnote`'s.

`\@makefnmark` : A macro to generate the footnote marker from `\@thefnmark` The default definition was `\hbox{$^\@thefnmark$}`.

This is now replaced by `\textsuperscript{\@thefnmark}`

`\@makefntext{NOTE}` :

Must produce the actual footnote, using `\@thefnmark` as the mark of the footnote and NOTE as the text. It is called when effectively inside a `\parbox`, with `\hsize` = `\columnwidth`.

For example, it might be as simple as

`$^{\@thefnmark}$` NOTE

In a minipage environment, \footnote and \footnotetext are redefined so that

(a) they use the counter mpfootnote

(b) the footnotes they produce go at the bottom of the minipage.

The switch is accomplished by letting \@mpfn == footnote or mpfootnote and \thempfn == \thefootnote or \thempfootnote, and by redefining \@footnotetext to be \@mpfootnotetext in the minipage.

```
\footnote{NOTE}  ==
 BEGIN
   \stepcounter{\@mpfn}
   begingroup
      \protect == \noexpand
      \@thefnmark :=G eval (\thempfn)
   endgroup
   \@footnotemark
   \@footnotetext{NOTE}
 END

\footnote[NUM]{NOTE} ==
 BEGIN
   begingroup
      \protect == \noexpand
      counter \@mpfn :=L NUM
      \@thefnmark :=G eval (\thempfn)
   endgroup
   \@footnotemark
   \@footnotetext{NOTE}
 END

\footnotemark       ==
 BEGIN \stepcounter{footnote}
       begingroup
          \protect == \noexpand
          \@thefnmark :=G eval(\thefootnote)
       endgroup
       \@footnotemark
 END

\footnotemark[NUM] ==
  BEGIN
      begingroup
        footnote counter :=L NUM
        \protect == \noexpand
       \@thefnmark :=G eval(\thefootnote)
      endgroup
      \@footnotemark
  END
```

```
\@footnotemark ==
  BEGIN
   \leavevmode
   IF hmode THEN \@x@sf := \the\spacefactor FI
   \@makefnmark          % put number in main text
   IF hmode THEN \spacefactor := \@x@sf FI
  END

\footnotetext       ==
   BEGIN begingroup \protect == \noexpand
                    \@thefnmark :=G eval (\thempfn)
         endgroup
         \@footnotetext
   END

\footnotetext[NUM] ==
   BEGIN begingroup  counter \@mpfn :=L NUM
                     \protect == \noexpand
                     \@thefnmark :=G eval (\thempfn)
         endgroup
         \@footnotetext
   END
```

End of historical LaTeX 2.09 comments.

`\footins` LaTeX does use the same insert for footnotes as PLAIN.

```
\newinsert\footins
```

LaTeX leaves these initializations for the `\footins` insert.

```
\skip\footins=\bigskipamount % space added when footnote is present
\count\footins=1000 % footnote magnification factor (1 to 1)
\dimen\footins=8in % maximum footnotes per page
```

(*End of definition for* `\footins`.)

`\footnoterule` LaTeX keeps PLAIN TeX's `\footnoterule` as the default.

```
\def\footnoterule{\kern-3\p@
  \hrule \@width 2in \kern 2.6\p@} % the \hrule is .4pt high
```

(*End of definition for* `\footnoterule`.)

`\thefootnote`

```
\@definecounter{footnote}
\def\thefootnote{\@arabic\c@footnote}
```

(*End of definition for* `\thefootnote`.)

`\thempfootnote` The default display for the footnote counter in minipages is to use italic letters. We use `\itshape` not `\textit` as the latter would add an italic correction.

```
\@definecounter{mpfootnote}
\def\thempfootnote{{\itshape\@alph\c@mpfootnote}}
```

(*End of definition for* `\thempfootnote`.)

\@makefnmark Default definition.

```
%\def\@makefnmark{\hbox{$^{\@thefnmark}\m@th$}}
\def\@makefnmark{\hbox{\@textsuperscript{\normalfont\@thefnmark}}}
```

(*End of definition for* `\@makefnmark`.)

\textsuperscript This command provides superscript characters in the current text font. It's implementation might change!!!

```
\DeclareRobustCommand*\textsuperscript[1]{%
  \@textsuperscript{\selectfont#1}}
```

(*End of definition for* `\textsuperscript`.)

\@textsuperscript
\textsuperscript@offset
\textsuperscript@space

This command should not be used directly, but may be used to define other commands `\textsuperscript`, `\@makefnmark`. `#1` should always start with a font selection command, to activate the font size switch. Doing everything in text mode using the classical dimensions means using the settings from math mode: to allow the user to deviate from those, we use a one-level indirection. Notice that the script space is applied without using a kern to keep the tracing output as similar as possible to the previous math-based approach. The rather involved default for `\textsuperscript@offset` arises so we get the same result as from the math mode predecessor. The one for `\textsuperscript@space` is needed as in math mode, `\scriptspace` is not dropped at a line break (as it's a box adjustment). LuaTeX has rather easier to understand names for the various parameters, and these will be correct whether classical or OpenType fonts are in use for math, so we set up with these if available.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2026/06/01}%
⟨latexrelease⟩                    {\@textsuperscript}{real text superscript}%
⟨*2ekernel | latexrelease⟩
\protected\def\@textsuperscript#1%
  {%
    \leavevmode
    \begingroup
      \check@mathfonts
      \sbox\z@{\fontsize\sf@size\sf@size#1}%
      \raise\dimexpr\textsuperscript@offset\relax\box\z@
      \textsuperscript@space
    \endgroup
  }
\ifdefined\directlua
  \def\textsuperscript@offset{%
    \ifdim\Umathsupshiftup\textstyle<\dimexpr\dp\z@+\Umathsupbottommin\textstyle\relax
      \dp\z@+\Umathsupbottommin\textstyle
    \else
      \Umathsupshiftup\textstyle
    \fi
  }
\else
  \def\textsuperscript@offset{%
    \ifdim\fontdimen14\textfont\tw@<\dimexpr\dp\z@+0.25\fontdimen5\textfont\tw@\relax
      \dp\z@+0.25\fontdimen5\textfont\tw@
    \else
      \fontdimen14\textfont\tw@
```

```
    \fi
  }
\fi
\def\textsuperscript@space{\nobreak\hskip\scriptspace\kern\z@}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\@textsuperscript}{real text superscript}%
⟨latexrelease⟩\def\@textsuperscript#1{%
⟨latexrelease⟩  {\m@th\ensuremath{^{\mbox{\fontsize\sf@size\sf@size#1}}}}}
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@textsuperscript}{real text superscript}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@textsuperscript#1{%
⟨latexrelease⟩  {\m@th\ensuremath{^{\mbox{\fontsize\sf@size\z@#1}}}}}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@textsuperscript`, `\textsuperscript@offset`, *and* `\textsuperscript@space`.)

`\textsubscript`

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩                  {\textsubscript}{\textsubscript}%
⟨*2ekernel | latexrelease⟩

\DeclareRobustCommand*\textsubscript[1]{%
  \@textsubscript{\selectfont#1}}%

⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\textsubscript}{\textsubscript}%
⟨latexrelease⟩\let\textsubscript\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\textsubscript`.)

`\@textsubscript`
`\textsubscript@offset`
`\textsubscript@space`

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2026/06/01}%
⟨latexrelease⟩                  {\@textsubscript}{real text subscript}%
⟨*2ekernel | latexrelease⟩
\protected\def\@textsubscript#1%
  {%
    \leavevmode
    \begingroup
      \check@mathfonts
      \sbox\z@{\fontsize\sf@size\sf@size#1}%
      \lower\dimexpr\textsubscript@offset\relax\box\z@
      \textsubscript@space
    \endgroup
  }
```

```
\ifdefined\directlua
  \def\textsubscript@offset{%
    \ifdim\Umathsubshiftdown\textstyle<\dimexpr\ht\z@-\Umathsubtopmax\textstyle\relax
      \ht\z@-\Umathsubtopmax\textstyle
    \else
      \Umathsubshiftdown\textstyle
    \fi
  }
\else
  \def\textsubscript@offset{%
    \ifdim\fontdimen16\textfont\tw@<\dimexpr\ht\z@-0.8\fontdimen5\textfont\tw@\relax
      \ht\z@-0.8\fontdimen5\textfont\tw@
    \else
      \fontdimen16\textfont\tw@
    \fi
  }
\fi
\def\textsubscript@space{\nobreak\hskip\scriptspace\kern\z@}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\@textsubscript}{real text subscript}%
⟨latexrelease⟩\def\@textsubscript#1{%
⟨latexrelease⟩  {\m@th\ensuremath{_{\mbox{\fontsize\sf@size\sf@size#1}}}}}
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩                  {\@textsubscript}{real text subscript}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@textsubscript#1{%
⟨latexrelease⟩  {\m@th\ensuremath{_{\mbox{\fontsize\sf@size\z@#1}}}}}

⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@textsubscript}{real text subscript}%
⟨latexrelease⟩\let\@textsubscript\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@textsubscript`, `\textsubscript@offset`, *and* `\textsubscript@space`.)

`\footnotesep`

```
\newdimen\footnotesep
```

(*End of definition for* `\footnotesep`.)

`\footnote`

```
\def\footnote{\@ifnextchar[\@xfootnote{\stepcounter\@mpfn
     \protected@xdef\@thefnmark{\thempfn}%
     \@footnotemark\@footnotetext}}
```

(*End of definition for* `\footnote`.)

\@xfootnote

```
\def\@xfootnote[#1]{%
   \begingroup
     \csname c@\@mpfn\endcsname #1\relax
     \unrestored@protected@xdef\@thefnmark{\thempfn}%
   \endgroup
   \@footnotemark\@footnotetext}
```

(*End of definition for* `\@xfootnote`.)

\@footnotetext

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2021/11/15}%
⟨latexrelease⟩                  {\@footnotetext}{footnotetext tagging}%
\long\def\@footnotetext#1{\insert\footins{%
    \reset@font\footnotesize
    \interlinepenalty\interfootnotelinepenalty
    \splittopskip\footnotesep
    \splitmaxdepth \dp\strutbox \floatingpenalty \@MM
    \hsize\columnwidth \@parboxrestore
    \def\@currentcounter{footnote}%
    \protected@edef\@currentlabel{%
       \csname p@footnote\endcsname\@thefnmark
    }%
    \color@begingroup
      \@makefntext{%
        \rule\z@\footnotesep\ignorespaces#1\@finalstrut\strutbox}%
    \par
    \color@endgroup}}%
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩                  {\@footnotetext}{footnotetext tagging}%
⟨latexrelease⟩\long\def\@footnotetext#1{\insert\footins{%
⟨latexrelease⟩    \reset@font\footnotesize
⟨latexrelease⟩    \interlinepenalty\interfootnotelinepenalty
⟨latexrelease⟩    \splittopskip\footnotesep
⟨latexrelease⟩    \splitmaxdepth \dp\strutbox \floatingpenalty \@MM
⟨latexrelease⟩    \hsize\columnwidth \@parboxrestore
⟨latexrelease⟩    \protected@edef\@currentlabel{%
⟨latexrelease⟩       \csname p@footnote\endcsname\@thefnmark
⟨latexrelease⟩    }%
⟨latexrelease⟩    \color@begingroup
⟨latexrelease⟩      \@makefntext{%
⟨latexrelease⟩        \rule\z@\footnotesep\ignorespaces#1\@finalstrut\strutbox}%
⟨latexrelease⟩    \par
⟨latexrelease⟩    \color@endgroup}}%
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@footnotetext}{footnotetext tagging}%
⟨latexrelease⟩
⟨latexrelease⟩\long\def\@footnotetext#1{\insert\footins{%
```

```
⟨latexrelease⟩     \reset@font\footnotesize
⟨latexrelease⟩     \interlinepenalty\interfootnotelinepenalty
⟨latexrelease⟩     \splittopskip\footnotesep
⟨latexrelease⟩     \splitmaxdepth \dp\strutbox \floatingpenalty \@MM
⟨latexrelease⟩     \hsize\columnwidth \@parboxrestore
⟨latexrelease⟩     \protected@edef\@currentlabel{%
⟨latexrelease⟩        \csname p@footnote\endcsname\@thefnmark
⟨latexrelease⟩     }%
⟨latexrelease⟩     \color@begingroup
⟨latexrelease⟩       \@makefntext{%
⟨latexrelease⟩         \rule\z@\footnotesep\ignorespaces#1\@finalstrut\strutbox}%
⟨latexrelease⟩     \color@endgroup}}%
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@footnotetext`.)

`\footnotemark`

```
\def\footnotemark{%
   \@ifnextchar[\@xfootnotemark
     {\stepcounter{footnote}%
      \protected@xdef\@thefnmark{\thefootnote}%
      \@footnotemark}}
```

(*End of definition for* `\footnotemark`.)

`\@xfootnotemark`

```
\def\@xfootnotemark[#1]{%
   \begingroup
      \c@footnote #1\relax
      \unrestored@protected@xdef\@thefnmark{\thefootnote}%
   \endgroup
   \@footnotemark}
```

(*End of definition for* `\@xfootnotemark`.)

`\@footnotemark`

```
\def\@footnotemark{%
  \leavevmode
  \ifhmode\edef\@x@sf{\the\spacefactor}\nobreak\fi
  \@makefnmark
  \ifhmode\spacefactor\@x@sf\fi
  \relax}
```

(*End of definition for* `\@footnotemark`.)

`\footnotetext`

```
\def\footnotetext{%
     \@ifnextchar [\@xfootnotenext
       {\protected@xdef\@thefnmark{\thempfn}%
    \@footnotetext}}
```

(*End of definition for* `\footnotetext`.)

\@xfootnotenext

```
\def\@xfootnotenext[#1]{%
  \begingroup
     \csname c@\@mpfn\endcsname #1\relax
     \unrestored@protected@xdef\@thefnmark{\thempfn}%
  \endgroup
  \@footnotetext}
```

(*End of definition for* `\@xfootnotenext`.)

\thempfn
\@mpfn

```
\def\@mpfn{footnote}
\def\thempfn{\thefootnote}
```

(*End of definition for* `\thempfn` *and* `\@mpfn`.)

\footref This command generates a footnote mark. The value is produced by referencing a `\label` placed into a `\footnote` elsewhere (can be one in the main galley or in a minipage).

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩                    {\footref}{Add footref}%
\def\footref#1{%
  \begingroup
    \unrestored@protected@xdef\@thefnmark{\ref{#1}}%
  \endgroup
  \@footnotemark
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

We don't remove it when rolling back so that packages offered it in the past do not need to alter their behavior in a rollback situation.

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\footref}{Add footref}%
⟨latexrelease⟩
⟨latexrelease⟩  % \let\footref\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\footref`.)

```
⟨/2ekernel⟩
```

File 46
ltidxglo.dtx

1 Index and Glossary Generation

Index and Glossary commands.

`\makeindex` A preamble command to turn on indexing.

`\makeglossary` A preamble command to turn on making glossary entries.

`\index` Make an index entry for `#1`.

`\glossary` Make a glossary entry for `#1`.

Historical LaTeX 2.09 comments (not necessarily accurate any more):

```
\makeindex ==
  BEGIN
            \index ==  BEGIN \@bsphack
                             \begingroup
                                \protect{X} == \string X\space
                                %% added 3 Feb 87 for \index commands
                                %% in \footnotes
                                re-\catcode special characters
                                to 'other'
                                \@wrindex
  END

  \@wrindex{ITEM} ==
    BEGIN
        write of {\indexentry{ITEM}{page number}}
      \endgroup
      \@esphack
    END

  INITIALIZATION:

  \index == BEGIN \@bsphack
                  \begingroup
                     re-\catcode special characters (in case '%' there)
                     \@index
            END

  \@index{ITEM} == BEGIN \endgroup \@esphack END
```

Changes made 14 Apr 89 to write `\glossaryentry`'s instead of `\indexentry`'s on the .glo file.

End of historical LaTeX 2.09 comments.

```
1 ⟨*2ekernel⟩
2 \message{index,}
```

\makeindex

```
\def\makeindex{%
  \newwrite\@indexfile
  \immediate\openout\@indexfile=\jobname.idx
  \def\index{\@bsphack\begingroup
             \@sanitize
             \@wrindex}\typeout
    {Writing index file \jobname.idx}%
```

Opening the write channel should be done only once since on some OS multiple opens are forbidden and in any case it is useless. So we turn this into a no-op after use.

```
  \let\makeindex\@empty
}
\@onlypreamble\makeindex
```

(*End of definition for* `\makeindex`.)

\@wrindex

```
\def\@wrindex#1{%
   \protected@write\@indexfile{}%
      {\string\indexentry{#1}{\thepage}}%
 \endgroup
 \@esphack}
```

(*End of definition for* `\@wrindex`.)

\index

```
\def\index{\@bsphack\begingroup \@sanitize\@index}
```

(*End of definition for* `\index`.)

\@index

```
\def\@index#1{\endgroup\@esphack}
```

(*End of definition for* `\@index`.)

\makeglossary

```
\def\makeglossary{%
  \newwrite\@glossaryfile
  \immediate\openout\@glossaryfile=\jobname.glo
  \def\glossary{\@bsphack\begingroup
                \@sanitize
                \@wrglossary}\typeout
    {Writing glossary file \jobname.glo }%
```

Opening the write channel should be done only once since on some OS multiple opens are forbidden and in any case it is useless. So we turn this into a no-op after use.

```
  \let\makeglossary\@empty
}
\@onlypreamble\makeglossary
```

(*End of definition for* `\makeglossary`.)

\@wrglossary

```
\def\@wrglossary#1{%
   \protected@write\@glossaryfile{}%
      {\string\glossaryentry{#1}{\thepage}}%
 \endgroup
 \@esphack}
```

(*End of definition for* `\@wrglossary`.)

\glossary

```
\def\glossary{\@bsphack\begingroup\@sanitize\@index}
```

(*End of definition for* `\glossary`.)

⟨/2ekernel⟩

File 47
ltbibl.dtx

1 Bibliography Generation

A bibliography is created by the `thebibliography` environment, which generates a title such as "References", and a list of entries. The BIBTEX program will create a file containing such an environment, which will be read in by the `\bibliography` command. With BIBTEX, the following commands will be used.

`\bibliography` `\bibliography{`⟨*file1,file2, . . . ,filen*⟩`}` : specifies the bibdata files. Writes a `\bibdata` entry on the `.aux` file and tries to read in `mainfile.bbl`.

`\bibliographystyle` `\bibliographystyle{`⟨*style*⟩`}` : Writes a `\bibstyle` entry on the `.aux` file.

`thebibliography` (*env.*) The `thebibliography` environment is a list environment. To save the use of an extra counter, it should use `enumiv` as the item counter. Instead of using `\item`, items in the bibliography are produced by the following commands:
`\bibitem{`⟨*name*⟩`}` : Produces a numbered entry cited as ⟨`name`⟩.
`\bibitem[`⟨*label*⟩`]{`⟨*name*⟩`}` : Produces an entry labeled by ⟨`Label`⟩ and cited by ⟨`name`⟩.

The former is used for bibliographies with citations like [1], [2], etc.; the latter is used for citations like [Knuth82].

The document class must define the thebibliography environment. This environment has a single argument, which is the widest bibliography label– e.g., if the [Knuth67] is the widest entry, then this argument will be Knuth67. The `\thebibliography` command must begin a list environment, which the `\endthebibliography` command ends.

`\cite` Entries are cited by the command `\cite{`⟨*name*⟩`}`.

`\nocite` `\nocite{`⟨*citations*⟩`}` puts information on the `.aux` file that causes BIBTEX to include the `{`⟨*citations*⟩`}` list in the bibliography, but puts nothing in the text.

`\nocite{*}` is special: it tells BIBTEX to put the whole of a collection of references into the bibliography.

```
1 ⟨*2ekernel⟩
2 \message{bibliography,}
```

Historical LATEX 2.09 comments (not necessarily accurate any more):
PARAMETERS

`\@cite` : A macro such that `\@cite{`LABEL1,LABEL2`}{`NOTE`}` produces the output for a `\cite[`NOTE`]{`FOO1,FOO2`}` command, where entry FOOi is defined by `\bibitem[`LABELi`]{`FOOi`}`. The switch @tempswa is true if the optional NOTE argument is present.
The default definition is :

```
\@cite{LABELS}{NOTE} ==
    BEGIN [LABELS
           IF @tempswa = T THEN , NOTE FI
           ]
    END
```

`\@biblabel` : A macro to produce the label in the bibliography entry. For `\bibitem[`LABEL`]{`NAME`}`, the label is

generated by `\@biblabel{LABEL}`. It has the default definition `\@biblabel{LABEL}` -> [LABEL].

CONVENTION

`\b@FOO` : The name or number of the reference created by `\cite{FOO}` E.g., if `\cite{FOO}` -> [17] , then `\b@FOO` -> 17.

End of historical LaTeX 2.09 comments.

`\bibitem`

```
\def\bibitem{\@ifnextchar[\@lbibitem\@bibitem}
```

(*End of definition for* `\bibitem`.)

`\@lbibitem`

```
\def\@lbibitem[#1]#2{\item[\@biblabel{#1}\hfill]\if@filesw
      {\let\protect\noexpand
       \immediate
       \write\@auxout{\string\bibcite{#2}{#1}}}\fi\ignorespaces}
```

(*End of definition for* `\@lbibitem`.)

`\@bibitem`

```
\def\@bibitem#1{\item\if@filesw \immediate\write\@auxout
       {\string\bibcite{#1}{\the\value{\@listctr}}}\fi\ignorespaces}
```

(*End of definition for* `\@bibitem`.)

`\bibcite`

```
\def\bibcite{\@newl@bel b}
```

(*End of definition for* `\bibcite`.)

`\citation`

```
\let\citation\@gobble
```

(*End of definition for* `\citation`.)

`\cite`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2022/06/01}%
⟨latexrelease⟩                    {\cite}{check for blank}%
\DeclareRobustCommand\cite{%
  \@ifnextchar [{\@tempswatrue\@citex@checkblank}{\@tempswafalse\@citex@checkblank[]}}
```

Due to the way `\@for` as used in `\@citex` behaves an empty argument to `\cite` did not produce any warning for a missing citation. So we now inject a command before calling `\@citex` that does the checking for us. It is not done in `\@citex` directly, because that command is altered by a number of packages/classes and this way it is more likely that the check survives.

```
\def\@citex@checkblank[#1]#2{%
   \IfBlankTF {#2}%
     {\@citex[#1]{\space}}%
     {\@citex[#1]{#2}}%
}
⟨/2ekernel | latexrelease⟩
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\cite}{check for blank}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\cite{%
⟨latexrelease⟩   \@ifnextchar [{\@tempswatrue\@citex}{\@tempswafalse\@citex[]}}
⟨latexrelease⟩\let\@citex@checkblank\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\cite`.)

`\@citex` `\penalty\@m` added to definition of `\@citex` to allow a line break after the ',' in citations like [Jones80,Smith77] (Added 23 Oct 86)

space added after the ',' (21 Nov 87)

```
\def\@citex[#1]#2{\leavevmode
  \let\@citea\@empty
  \@cite{\@for\@citeb:=#2\do
    {\@citea\def\@citea{,\penalty\@m\ }%
     \edef\@citeb{\expandafter\@firstofone\@citeb\@empty}%
     \if@filesw\immediate\write\@auxout{\string\citation{\@citeb}}\fi
```

Using `\hbox` instead of `\mbox` is fine because of the `\leavevmode` above. In fact the use of a box around the citation contents is more than questionable in my view (FMi), but within 2e I have to keep that for compatibility reasons as it would probably change too many existing documents. Its main reason is to avoid hyphenation of labels such as [FOOB89] into [FOO- B89] so in certain styles it makes sense; but, for example, in author year citations it becomes more than questionable.

So Chris added yet another hook here, as suggested by, at least, Donald Arseneau. Note that this one is inside the first argument of the `\@cite` hook. This decouples the top-level typesetting of the citation from the details of the other business conducted here. All this really needs a complete rethink to get the right modularity.

```
     \@ifundefined{b@\@citeb}{\hbox{\reset@font\bfseries ?}%
       \G@refundefinedtrue
       \@latex@warning
         {Citation `\@citeb' on page \thepage \space undefined}}%
       {\@cite@ofmt{\csname b@\@citeb\endcsname}}}}{#1}}
```

(*End of definition for* `\@citex`.)

`\bibdata` `\bibstyle`

```
\let\bibdata=\@gobble
\let\bibstyle=\@gobble
```

(*End of definition for* `\bibdata` *and* `\bibstyle`.)

`\bibliography`

```
\def\bibliography#1{%
  \if@filesw
    \immediate\write\@auxout{\string\bibdata{\zap@space#1 \@empty}}%
  \fi
  \@input@{\jobname.bbl}}
```

(*End of definition for* `\bibliography`.)

`\bibliographystyle`

```
\def\bibliographystyle#1{%
  \ifx\@begindocumenthook\@undefined\else
    \expandafter\AtBeginDocument
  \fi
    {\if@filesw
       \immediate\write\@auxout{\string\bibstyle{#1}}%
     \fi}}
```

(*End of definition for* `\bibliographystyle`.)

`\nocite` (Added 14 Jun 85)

This puts information on the `.aux` file that causes BibTeX to include the citation list in the bibliography, but puts nothing in the text.

RmS 93/08/06: Made loop for `\nocite` like that for `\@citex`, to get rid of leading spaces.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩                    {\nocite}{Allow nocite in preamble}%
\def\nocite#1{\@bsphack
```

With the implementation designed already in LaTeX 2.09 the `\nocite` command will not work before `\begin{document}` since it tries to write to the `.aux` file which is not open before that point. As a result the "reference" will appear on the terminal and nothing else will happen.

[This would be easy to fix, but then a document using the fix will silently fail on an older release of LaTeX, missing all citations done with `\nocite`. Thus we do only generate an error message and leave the fix for a LaTeX 2_ε successor.]

Given that we are now a quarter century into using LaTeX 2_ε there is no good reason any more do limit ourself to 2.09 considerations. So we now simply delay the `\nocite` if it is issued in the preamble.

```
  \ifx\@onlypreamble\document
```

Since we are after `\begin{document}` we can do the citations:

```
    \@for\@citeb:=#1\do{%
      \edef\@citeb{\expandafter\@firstofone\@citeb}%
      \if@filesw\immediate\write\@auxout{\string\citation{\@citeb}}\fi
      \@ifundefined{b@\@citeb}{\G@refundefinedtrue
          \@latex@warning{Citation `\@citeb' undefined}}{}}%
  \else
```

But before `\begin{document}` we raised an error message in the past but as of 2021/05 not any longer.

```
%    \@latex@error{Cannot be used in preamble}\@eha
```

Instead we delay the declaration to the start of the document. We have to use a late hook for this, so that it comes after the `.aux` file is open for writing and after `\@preamblecmds` was executed to change the above test. Therefore `\AtBeginDocument` would still be too early.

```
    \AddToHook{begindocument/end}[kernel]{\nocite{#1}}%
  \fi
```

```
  \@esphack}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                 {\nocite}{Allow nocite in preamble}%
⟨latexrelease⟩
⟨latexrelease⟩\def\nocite#1{\@bsphack
⟨latexrelease⟩  \ifx\@onlypreamble\document
⟨latexrelease⟩    \@for\@citeb:=#1\do{%
⟨latexrelease⟩      \edef\@citeb{\expandafter\@firstofone\@citeb}%
⟨latexrelease⟩      \if@filesw\immediate\write\@auxout{\string\citation{\@citeb}}\fi
⟨latexrelease⟩      \@ifundefined{b@\@citeb}{\G@refundefinedtrue
⟨latexrelease⟩          \@latex@warning{Citation `\@citeb' undefined}}{}}%
⟨latexrelease⟩  \else
⟨latexrelease⟩    \@latex@error{Cannot be used in preamble}\@eha
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \@esphack}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

Since `\nocite{*}` should not produce a warning about undefined citation keys (see PR 557), we need to set the control sequence '`\b@*`' to something other than `\relax`. As a result `\cite{*}` will not warn either (but that never worked with BibTeX in the first place).

```
\expandafter\let\csname b@*\endcsname\@empty
```

(*End of definition for* `\nocite`.)

1.1 Default definitions

This hook determines the 'relative formatting' of the two logical parts of a citation with comment.

`\@cite`

```
\def\@cite#1#2{[{#1\if@tempswa , #2\fi}]}
```

(*End of definition for* `\@cite`.)

`\@cite@ofmt` This is, in general, a command that appears to have one argument whose value is, in the kernel, a single cs whose name is the expansion of `b@\@citeb`; the expansion of this cs will typically be some hmode material that produces the detailed typeset form of just the citations themselves.

```
\let\@cite@ofmt\hbox
```

(*End of definition for* `\@cite@ofmt`.)

`\@biblabel`

```
\def\@biblabel#1{[#1]}
⟨/2ekernel⟩
```

(*End of definition for* `\@biblabel`.)

File 48
ltmarks.dtx

Abstract

Marks are used to communicate information about the content of a page to the output routine. For example, in order to construct running headers, the output routine needs information about which section names are present on a page, and this information is passed to it through the mark system. However, marks may also be used for other purposes. This module provides a generalized mechanism for marks of independent classes.

1 Introduction

The TEX engines offer a low-level mark mechanism to communicate information about the content of the current page to the asynchronous operating output routine. It works by placing `\mark` commands into the source document. When the material for the current page is assembled in box 255, TEX scans for such marks and sets the commands `\topmark`, `\firstmark` and `\botmark`. The `\firstmark` receives the content of the first `\mark` seen in box 255 and `\botmark` the content of the last mark seen. The `\topmark` holds the content of the last mark seen on the previous page or more exactly the value of `\botmark` from the previous page. If there are no marks on the current page then all three are made equal to the `\botmark` from the previous page.

This mechanism works well for simple formats (such as plain TEX) whose output routines are only called to generate pages. It fails, however, in LATEX (and other more complex formats), because here the output routine is sometimes called without producing a page, e.g., when encountering a float and placing it into one of the float regions. In that case the output routine is called, determines where to place the float, alters the goal for assembling text material (if the float was added to the top or bottom region) and then it resumes collecting textual material.

As a result the `\botmark` gets updated and so `\topmark` no longer reflects the situation at the top of the next page when that page is finally boxed.

Another problem for LATEX was that it wanted to use several "independent" marks and in the early implementations of TEX there was only a single `\mark` command available. For that reason LATEX implemented its own mark mechanism where the marks always contained two parts with their own interfaces: `\markboth` and `\markright` to set marks and `\leftmark` and `\rightmark` to retrieve them.

However, this extended mechanism (while supporting scenarios such as chapter/section marks) was far from general. The mark situation at the top of a page (i.e., `\topmark`) remained unusable and the two marks offered were not really independent of each other because `\markboth` (as the name indicates) was always setting both.

The new mechanism overcomes both issues:

- It provides arbitrarily many, fully independent named marks, that can be allocated and, from that point onwards, used.
- It offers access for each such marks to retrieve its top, first, and bottom values separately.
- Furthermore, the mechanism is augmented to give access to marks in different "regions" which may not be just full pages.

2 Design-level and code-level interfaces

The interfaces are mainly meant for package developers, but they are usable (with appropriate care) also in the document preamble, for example, when setting up special running headers with fancyhdr, etc. They are therefore available both as CamelCase commands as well as commands for use in the L3 programming layer. Both are described together below.

`\NewMarkClass` `\NewMarkClass {⟨class⟩}`
`\mark_new_class:n` `\mark_new_class:n {⟨class⟩}`

Declares a new ⟨*class*⟩ of marks to be tracked by LaTeX. Each ⟨*class*⟩ must be declared before it is used.

Mark classes can only be declared before `\begin{document}`.

`\InsertMark` `\InsertMark {⟨class⟩} {⟨text⟩}`
`\mark_insert:nn` `\mark_insert:nn {⟨class⟩} {⟨text⟩}`

Adds a mark to the current galley for the ⟨*class*⟩, containing the ⟨*text*⟩.

It has no effect in places in which you can't place floats, e.g., a mark inside a box or inside a footnote never shows up anywhere.

If used in vertical mode it obeys LaTeX's internal `@nobreak` switch, i.e., it does not introduce a breakpoint if used after a heading. If used in horizontal mode it doesn't handle spacing (like, for example, `\index` or `\label` does, so it should be attached to material that is typeset.

`insertmark` `\AddToHook {insertmark} {⟨code⟩}`

When marks are inserted, the mark content may need some special treatment, e.g., by default `\label`, `\index`, and `\glossary` do not expand at this time (but only later if and when the mark content is actually used. In order to allow packages to augment or alter this setup there is a public hook `insertmark` that is executed at this point. It runs in a group so local modification to commands are only applied to the ⟨*text*⟩ argument of `\InsertMark` or `\mark_insert:nn`.

\TopMark ⋆ \TopMark [⟨region⟩] {⟨class⟩}
\FirstMark ⋆ \FirstMark [⟨region⟩] {⟨class⟩}
\LastMark ⋆ \LastMark [⟨region⟩] {⟨class⟩}
\mark_use_top:nn ⋆ \mark_use_top:nn {⟨region⟩} {⟨class⟩}
\mark_use_first:nn ⋆ \mark_use_first:nn {⟨region⟩} {⟨class⟩}
\mark_use_last:nn ⋆ \mark_use_last:nn {⟨region⟩} {⟨class⟩}

These functions expand to the appropriate mark ⟨*text*⟩ for the given ⟨*class*⟩ in the specified ⟨*region*⟩. The default ⟨*region*⟩ in the design-level commands is `page`. Note that with the L3 layer commands there are no optional arguments, i.e., both arguments have to be provided.

TEXhackers note: The result is returned within the `\unexpanded` primitive (`\exp_not:n`), which means that the ⟨*text*⟩ does not expand further when appearing in an `x`-type or `e`-type argument expansion.

The "first" and "last" marks are those seen first and last in the current region/page, respectively. The "top" mark is the last mark of the ⟨*class*⟩ seen in an earlier region, i.e., the ⟨*text*⟩ what would be "current" at the very top of the region.

Important!

The commands are only meaningful inside the output routine, in other places their result is (while not random) unpredictable due to the way LaTeX cuts text material into pages. There is, however, one exception: if you produce multiple columns using the multicol package, it is possible to retrieve mark values from the regions `first-column`, `last-column`, `mcol-1`, `mcol-2`,... directly after the environment has ended. This can, for example, be useful if a `multicols` has been be used inside a box.

Currently, ⟨*region*⟩ is one of `page`, `previous-page`, `column`, `previous-column`, `first-column`, `last-column`, and `mcol-1` (first column in a `multicols`), `mcol-2` (second column in a `multicols`), up to `mcol-20` (twentieth column in a `multicols`). See section 2.2 for discussion of how these regions behave and how one can make use of them.

\IfMarksEqualTF ⋆ \IfMarksEqualTF [⟨region⟩] {⟨class⟩} {⟨pos_1⟩} {⟨pos_2⟩} {⟨true⟩} {⟨false⟩}
\IfMarksEqualT ⋆ \mark_if_eq:nnnnTF {⟨region⟩} {⟨class⟩} {⟨pos_1⟩} {⟨pos_2⟩} {⟨true⟩} {⟨false⟩}
\IfMarksEqualF ⋆ \mark_if_eq:nnnnnnTF {⟨$region_1$⟩} {⟨$class_1$⟩} {⟨pos_1⟩}
\mark_if_eq:nnnnTF ⋆ {⟨$region_2$⟩} {⟨$class_2$⟩} {⟨pos_2⟩} {⟨true⟩} {⟨false⟩}
\mark_if_eq:nnnnnnTF ⋆

These conditionals allow you to compare the content of two marks and act based on the result. The commands work in an expansion context, if necessary.

It is quite common when programming with marks to need to interrogate conditions such as whether marks have appeared on a previous page, or if there are multiple marks present on the current page, and so on. The tests above allow for the construction of a variety of typical test scenarios, with three examples presented below.

The first two conditionals cover only the common scenarios. Both marks are picked up from the same ⟨*region*⟩ (by default `page`) and they have to be of the same ⟨*class*⟩.[41] The ⟨pos_i⟩ argument can be either `top`, `first`, or `last`.

Important to note is that the comparison is not with respect to the textual content of the marks but whether or not they originated from the same `\InsertMark` command (or the L3 layer version `\mark_insert:nn`).

If you wish to compare marks across different regions or across different classes, you have to do it using the generic test only available in the L3 programming layer or do it manually, i.e., get the marks and then compare the values yourself.[42]

[41] If an undeclared mark class is used the tests return *true* (not an error).

[42] If two undeclared mark classes are compared the result is always *true*; if a declared and an undeclared

2.1 Use cases for conditionals

However, the basic version is enough for the following typical use cases:

Test for at most one mark of class `myclass` on current page: If the first and last mark in a region are the same then either there was no mark at all, or there was at most one. To test this on the current page:

```
\NewMarkClass{myclass}
\IfMarksEqualTF{myclass}{first}{last}
               { <zero or one mark> }{ <two or more marks> }
```

Test for no mark of class `myclass` in the previous page: If the top mark is the same as the first mark, there is no mark in the region at all. If we wanted to do this test for the previous page:

```
\IfMarksEqualTF[previous-page]{myclass}{top}{first}
               { <no marks> }{ <at least one mark> }
```

Comparing `top` and `last` would give you the same result.

Test for zero, one, or more than one: Combining the two tests from above you can test for zero, one or more than one mark.

```
\IfMarksEqualTF{myclass}{top}{first}
               { <no marks> }
               {\IfMarksEqualTF{myclass}{first}{last}
                 { <exactly one mark> }{ <more than one mark> }}
```

If you need one of such tests more often (or if you want a separate command for it for readability), then consider defining:

```
\providecommand\IfNoMarkTF[2][page]{\IfMarksEqualTF[#1]{#2}{first}{last}}
```

2.2 Understanding regions

If a page has just been finished then the region `page` refers to the current page and `previous-page`, as the name indicates, refers to the page before the current page. This means you are able to access mark information for the current page as well as for the page before (as long as you are inside the output routine) without the need to explicitly save that information beforehand. The `page` region is the region that is most often queried, which is why commands like `\FirstMark` use that region by default.

In single column documents the `column` is the same as the `page` region, but in two-column documents (if not produced by `multicols`), `column` refers to the current column that just got finished and previous-column to the one previously finished. Code for running headers is (in standard LaTeX) evaluated only after both columns have been assembled, which is another way of saying that in that case `previous-column` refers to the left column and `column` to the right column. However, to make these somewhat easier to use, there are also aliased names for these two regions: `first-column` and `last-column`.[43]

mark class is used it is always *false*.

[43] The region is called "last-column" not "second-column" in anticipation of extending the mechanism to multiple columns, where first and last would still make sense. There aren't any `previous-first-column` and `previous-last-column` regions to access the corresponding columns from the previous page.

Note that you can only look backwards at already processed regions, e.g., in a `twoside` document finishing a recto (odd, right-hand) page you can access the data from the facing verso (left-hand) page, but if you are finishing a left-hand page you can't integrate data from the upcoming right-hand page. If such a scenario needs to be realized then it is necessary to save the left-hand page temporarily instead of finalizing it, process material for the right-hand page and once both are ready, attach running headers and footers and shipout out both in one go.[44]

The situation starts getting rather complex if you allow for multiple columns in the way they are supported by the multicol package. In this case you might have a variable number of "columns" on a single page to be shipped out. And even if not, then a `multicols` might start or end in the middle of the page; in either case, the regions `column` and `previous-column` become rather meaningless and you should therefore not use them.[45] Instead, the algorithm offers `mcol-1`, `mcol-2`, `mcol-3`, etc., to represent the columns in the `multicols` on the current page to be shipped out. If there is more than one `multicols` on the current page then in the output routine only the columns of the last one will be accessible.

These provisions cover, out of the box, a number of layouts and use cases, but obviously not all. However, more cases can be supported by storing away mark information during the processing. Here is the full algorithm:

- The `column` region is used by the "current column" that is being built (moving through all columns with `previous-column` trailing behind (to handle top marks properly).
- When the `multicols` starts, the `column` region is cleared, i.e., from that point on it looks as if there have not been any marks so far. This will make sure that the top mark in the first column is always empty.
- If the `multicols` extends beyond the current page, then the material designated for the current page is split into columns. The `column` region is used to represent each column in turn.
  - First we copy the current data from `column` to `previous-column`. Then the mark data from the current column is placed into the `column` region. Then we alias `column` to `mcol-1`.
  - These steps are repeated for all columns of the `multicols` environment.
  - Finally, the first and the last column of that page is also made available as `first-column` and `last-column`, respectively.
- All those marks inside any of the columns are also available in the `page` region. Thus, if you are interested in the top, first, or last mark of a specific class on the whole page you simply need to query for it in the `page` region.
- If the `multicols` continues across several pages then this algorithm above is repeated for each page, except that the `column` region is not cleared again. This means that the top mark of the first column of the next page will be the last mark of the last column from the previous page.

[44] As of now that scenario is not (yet) officially supported but it would be possible to achieve this using the shipout hooks to store the verso page and then on the next shipout use the hook to shipout both with running headers and footers attached.

[45] They return something, because they represent the last two columns of the `multicols` when you are inside the output routine, but that is obviously of little use.

- When the `multicols` finishes the remaining material for the current page is balanced to produce columns of roughly equal height.
- Again `column` and `previous-column` are used while this balancing happens and `mcol-1`, `mcol-2`, etc., are used to represent the column regions and `first-column` and `last-column` are set appropriately.
- Then the balanced set of columns is returned back to the page (since there may be space for further material). In addition, all marks inside that material are reinserted so that they become available in the `page` region.
- As a side effect, it is possible (and useful in certain circumstances) to query for mark classes directly after the `multicols` has ended without the need to be inside the output routine. The regions that can be queried this way are `mcol-1`, `mcol-2`, etc. (up to the number of columns the multicol had) and `first-column` and last-column.

2.3 Debugging mark code

`\DebugMarksOn`
`\DebugMarksOff`
`\mark_debug_on:`
`\mark_debug_off:`

`\DebugMarksOn ... \DebugMarksOff`

Commands to turn the debugging of mark code on or off. The debugging output is rather coarse and not really intended for normal use at this point in time.

3 Application examples

If you want to figure out if a break was taken at a specific point, e.g., whether a heading appears at the top of the page, you can do something like this:

```
\newcounter{breakcounter}
\NewMarkClass{break}
\newcommand\markedbreak[1]{\stepcounter{breakcounter}%
                           \InsertMark{break}{\arabic{breakcounter}%
                           \penalty #1\relax
                           \InsertMark{break}{-\arabic{breakcounter}}
```

To test if the break was taken you can test if `\TopMark{break}` is positive (taken) or negative (not taken) or zero (there was never any marked break so far). The absolute value can be used to keep track of which break it was (with some further coding).

to be extended with additional application examples

4 Legacy LaTeX 2ε interface

Here we describe the interfaces that LaTeX 2ε offered since the early nineties and some minor extensions.

4.1 Legacy design-level and document-level interfaces

\markboth \markboth {⟨*left*⟩} {⟨*right*⟩}
\markright \markright {⟨*right*⟩}

LaTeX 2ε uses two marks which aren't fully independent. A "left" mark generated by the first argument of `\markboth` and a "right" mark generated by the second argument of `\markboth` or by the only argument of `\markright`. The command `\markboth` and `\markright` are in turn called from heading commands such as `\chaptermark` or `\sectionmark` and their behavior is controlled by the document class.

For example, in the article class with `twoside` in force the `\sectionmark` will issue `\markboth` with an empty second argument and `\subsectionmark` will issue `\markright`. As a result the left mark will contain chapter titles and the right mark subsection titles.

Note, however, that in one-sided documents the standard behavior is that only `\markright` is used, i.e., there will only be right-marks but no left marks!

\leftmark ⋆ \leftmark
\rightmark ⋆ \rightmark

These functions return the appropriate mark value from the current page and work as before, that is `\leftmark` will get the last (!) left mark from the page and `\rightmark` the first (!) right mark.

In other words they work reasonably well if you want to show the section title that is current when you are about to turn the page and also show the first subsection title on the current page (or the last from the previous page if there wasn't one). Other combinations can't be shown using this interface.

The commands are fully expandable, because this is how they have been always defined in LaTeX. However, this is of course only true if the content of the mark they return is itself expandable and does not contain any fragile material. Given that this can't be guaranteed for arbitrary content, a programmer using them in this way should use `\protected@edef` and *not* `\edef` to avoid bad surprises as far as this is possible, or use the new interfaces (`\TopMark`, `\FirstMark`, and `\LastMark`) which return the ⟨*text*⟩ in `\exp_not:n` to prevent uncontrolled expansion.

4.2 Legacy interface extensions

The new implementation adds three mark classes: `2e-left`, `2e-right` and `2e-right-nonempty` and patches `\markboth` and `\markright` slightly so that they also update these new mark classes, so that the new classes work with existing document classes.

As a result you can use `\LastMark{2e-left}` and `\FirstMark{2e-right}` instead of `\leftmark` and `\rightmark`. But more importantly, you can use any of the other retrieval commands to get a different status value from those marks, e.g., `\LastMark{2e-right}` would return the last subsection on the page (instead of the first as returned by `\rightmark`).

The difference between `2e-right` and `2e-right-nonempty` is that the latter will only be updated if the material for the mark is not empty. Thus `\markboth{title}{}` as issued by, say, `\sectionmark`, sets a `2e-left` mark with `title` and a `2e-right` mark with the empty string but does not add a `2e-right-nonempty` mark.

Thus, if you have a section at the start of a page and you would ask for `\FirstMark{2e-right}` you would get an empty string even if there are subsections on that page. But `2e-right-nonempty` would then give you the first or last subsection

on that page. Of course, nothing is simple. If there are no subsections it would tell you the last subsection from an earlier page. We therefore need comparison tools, e.g., if top and first are identical you know that the value is bogus, i.e., a suitable implementation would be

```
\IfMarksEqualTF{2e-right-nonempty}{top}{first}
               { <appropriate action if there was no real mark> }
               {\FirstMark{2e-right-nonempty}}
```

5 Notes on the mechanism

In contrast to vanilla TEX, ε-TEX extends the mark system to allow multiple independent marks. However, it does not solve the `\topmark` problem which means that LATEX still needs to manage marks almost independently of TEX. The reason for this is that the more complex output routine used by LATEX to handle floats (and related structures) means that `\topmark(s)` remain unreliable. Each time the output routine is fired up, TEX moves `\botmark` to `\topmark`, and while ε-TEX extends this to multiple registers the fundamental concept remains the same. That means that the state of marks needs to be tracked by LATEX itself. An early implementation of this package used TEX's `\botmark` only to ensure the correct interaction with the output routine (this was before the ε-TEX mechanism was even available). However, other than in a prototype implementation for LATEX3, this package was never made public.

The new implementation now uses ε-TEX's marks as they have some advantages, because with them we can leave the mark text within the galley and only extract the marks during the output routine when we are finally shipping out a page or storing away a column for use in the next page. That means we do not have to maintain a global data structure that we have to keep in sync with informational marks in the galley but can rely on everything being in one place and thus manipulations (e.g. reordering of material) will take the marks with them without a need for updating a fragile linkage.

To allow for completely independent marks we use the following procedure:

- For every type of marks we allocate a mark class so that in the output routine TEX can calculate for each class the current top, first, and bottom mark independently. For this we use `\newmarks`, i.e., one marks register per class.
- As already mentioned firing up an output routine without shipping out a page means that TEX's top marks get wrong so it is impossible to rely on TEX's approach directly. What we do instead is to keep track of the real marks (for the last page or more generally last region) in some global variables.
- These variables are updated in the output routine at defined places, i.e., when we do real output processing but not if we use special output routines to do internal housekeeping.
- The trick we use to get correctly updated variables is the following: the material that contains new marks (for example the page to be shipped out) is stored in a box. We then use TEX primitive box splitting functions by splitting off the largest amount possible (which should be the whole box if nothing goes really wrong). While that seems a rather pointless thing to do, it has one important side effect: TEX sets up first and bottom marks for each mark class from the material it has split off. This way we get the first and last marks (if there have been any) from the material in the box.

- The top marks are simply the last marks from the previous page or region. And if there hasn't been a first or bottom mark in the box then the new top mark also becomes new first and last mark for that class.
- That mark data is then stored in global token lists for use during the output routine and legacy commands such as `\leftmark` or new commands such as `\TopMark` simply access the data stored in these token lists.

That's about it in a nutshell. Of course, there are some details to be taken care of—those are discussed in the implementation sections.

6 Public interfaces for packages such as multicol

The functions in this section are public so that packages can make use of them. However, this must be done with great care, e.g., `\mark_update_structure_from_material:nn` updates the global mark structure and can therefore be used only in places where such an update is meaningful, e.g., in special output routines. Elsewhere, a change to the mark structure would break the whole mechanism and querying the marks would return incorrect data.

`\mark_update_structure_from_material:nn` `\mark_update_structure_from_material:nn {⟨region⟩} {⟨material with marks⟩}`

Helper function that inspects the marks inside the second argument and assigns new mark values based on that to the ⟨*region*⟩ given in the first argument. For this it first copies the mark structure from ⟨*region*⟩ to `previous-`⟨*region*⟩ and then takes all last mark values currently in the region and makes them the new top mark values. Finally it assigns new first and last values for all mark classes based on what was found in the second argument.

As a consequence, the allowed values for ⟨*region*⟩ are `page` and `column` because only they have `previous-...` counterparts.

Another important aspect to keep in mind is that marks are recognized only if they appear on the top level, e.g., if we want to process material stored in boxes we need to put it unboxed (using `\unvcopy` etc.) into the second argument.

`\mark_copy_structure:nn` `\mark_copy_structure:nn {⟨alias⟩} {⟨source⟩}`

Helper function that copies all mark values in the ⟨*source*⟩ region to ⟨*alias*⟩, i.e., make the structures identical. Used to update the `previous-...` structures inside `\mark_update_structure_from_material:nn` and `first-column` and `last-column` structures inside the internal commands `\__mark_update_singlecol_structures:` or `\__mark__update_dblcol_structures:`.

`\mark_set_structure_to_err:n` `\mark_set_structure_to_err:n {⟨region⟩}`

Helper function that sets all mark values in the ⟨*region*⟩ to an error message. This is currently used for `last-column` at times where using marks from it would be questionable/wrong, i.e., when we have just processed the first column in a two-column document.

\mark_clear_structure:n \mark_clear_structure:n {⟨*region*⟩}

Helper function that sets all mark values in the ⟨*region*⟩ to empty. This is currently used for `column` when a multicol environment starts; this is because it wouldn't make sense if the top mark in the first column returned the last mark from a previous multicol (which may have been much earlier, with intermediate material).

\mark_get_marks_for_reinsertion:nNN \mark_get_marks_for_reinsertion:nNN {⟨*source*⟩}
⟨*token-list-var for collecting first marks*⟩
⟨*token-list-var for collecting last marks*⟩

Helper function for extracting marks that would otherwise get lost, for example when they are hidden inside a box. This helper does not update mark structures and can therefore be used outside the output routine as well.

It collects all the top-level marks from inside the ⟨*source*⟩ and then adds suitable `\mark_insert:nn` commands to each of the two token lists. These token lists can then be executed at the right place to reinsert the marks, e.g., directly after the box. This is, for example, going to be used[46] by multicol when a short balanced `multicols` is returned to the galley for typesetting.

If the ⟨*source*⟩ consists of a single vertical box (plus possibly followed by some glue but nothing else) then the box is unpacked and the top-level marks are collected from its content. However, if it is not a vertical box or there are other data then nothing is unpacked and you have to do the unpacking yourself to get at the marks inside.

It is quite likely that one only needs a single token list for returning the `\mark_insert:nn` statements. If that is the case this command may change to take only two arguments.

7 Internal functions for the standard output routine of LaTeX

The functions in this section are tied to the output routine and used in the interface to LaTeX 2ε and perhaps at some later time within a new output routine for LaTeX. They are not (yet) meant for general use and are therefore made internal, even though we already use them in multicol. Internal means that `@@` automatically gets replaced in the code (and in the documentation) so we have to give it a suitable value.

```
1 ⟨@@=mark⟩
```

__mark_update_singlecol_structures: __mark_update_singlecol_structures:

LaTeX 2ε integration function in case we are doing single column layouts. It assumes that the page content is already stored in `\@outputbox` and processes the marks inside that box. It is called as part of `\@opcol`.

__mark_update_dblcol_structures: __mark_update_singlecol_structures:

LaTeX 2ε integration function mark used when we are doing double column documents. It assumes that the page content is already stored in `\@outputbox` and processes the marks inside that box. It then does different post-processing depending on the start of the switch `\if@firstcolumn`. If we are in the second column it also has to update page marks, otherwise it only updates column marks. It too is called as part of `\@opcol`.

[46] Probably not before 2025, though.

8 The Implementation

```
⟨*2ekernel | latexrelease⟩
\ExplSyntaxOn
⟨latexrelease⟩\NewModuleRelease{2022/06/01}{ltmarks}
⟨latexrelease⟩                    {Marks~handling}
```

8.1 Allocating new mark classes

`\g__mark_classes_seq` A list holding all the mark classes that have been declared.

```
\seq_new:N \g__mark_classes_seq
```

`\mark_new_class:n` `\__mark_new_class:nn` A mark class is created by initializing a number of data structures. First, we get a register number to refer to the mark class. The new mark class is then added to the `\g__mark_classes_seq` sequence to be able to easily loop over all classes. Finally a number of top-level global token lists are declared that hold various versions of the mark for access.

```
\cs_new_protected:Npn \mark_new_class:n #1
{
  \seq_if_in:NnTF \g__mark_classes_seq {#1}
      {
        \msg_error:nnn { mark } { class-already-defined }
          {#1}
      }
      { \__mark_new_class:nn {#1} }
}
```

This is only available in the preamble.

```
\@onlypreamble \mark_new_class:n
```

The internal command carries out the necessary allocations.

```
\cs_new_protected:Npn \__mark_new_class:nn #1
{
⟨*trace⟩
  \__mark_debug:n { \iow_term:e { Marks:~new~mark:~#1~\msg_line_context: } }
⟨/trace⟩
```

Use the LaTeX 2_ε interface for now as the L3 programming layer doesn't have one for marks yet.

```
  \exp_args:Nc \newmarks {c__mark_class_ #1 _mark}
```

Remember the new class in the sequence.

```
  \seq_gput_right:Nn \g__mark_classes_seq {#1}
  \__mark_init_region:nn {page}{#1}
```

For the `page` region we also keep track of the `previous-page`.

```
  \__mark_init_region:nn {previous-page}{#1}
```

Same game for `column` and `previous-column`

```
  \__mark_init_region:nn {column}{#1}
  \__mark_init_region:nn {previous-column}{#1}
```

But for columns we also allocate token lists for the alias regions `first-column` and `last-column`.

```
  \__mark_init_region:nn {first-column}{#1}
  \__mark_init_region:nn {last-column}{#1}
```

To support multiple columns produced by the multicol package, we preallocate twenty alias regions (since this is the number of columns that multicol supports as a maximum). They are filled by copying the current `column` into the appropriate `mcol-...`.

```
%fmi  \__mark_init_region:nn {mcol}{#1}
%fmi  \__mark_init_region:nn {previous-mcol}{#1}
  \__mark_init_region:nn {mcol-1}{#1}
  \__mark_init_region:nn {mcol-2}{#1}
  \__mark_init_region:nn {mcol-3}{#1}
  \__mark_init_region:nn {mcol-4}{#1}
  \__mark_init_region:nn {mcol-5}{#1}
  \__mark_init_region:nn {mcol-6}{#1}
  \__mark_init_region:nn {mcol-7}{#1}
  \__mark_init_region:nn {mcol-8}{#1}
  \__mark_init_region:nn {mcol-9}{#1}
  \__mark_init_region:nn {mcol-10}{#1}
  \__mark_init_region:nn {mcol-11}{#1}
  \__mark_init_region:nn {mcol-12}{#1}
  \__mark_init_region:nn {mcol-13}{#1}
  \__mark_init_region:nn {mcol-14}{#1}
  \__mark_init_region:nn {mcol-15}{#1}
  \__mark_init_region:nn {mcol-16}{#1}
  \__mark_init_region:nn {mcol-17}{#1}
  \__mark_init_region:nn {mcol-18}{#1}
  \__mark_init_region:nn {mcol-19}{#1}
  \__mark_init_region:nn {mcol-20}{#1}
```

We also have to initialize the `saved-column` region that is used in multicol. Perhaps we should have a `\NewMarkRegion` so that it would be possible for other packages to add further regions. But let's wait and see if there is a real use case for new regions before making the interface more general.

```
  \__mark_init_region:nn {saved-column}{#1}
}
```

(*End of definition for* `\mark_new_class:n` *and* `\__mark_new_class:nn`*. This function is documented on page* 1059.)

`\__mark_init_region:nn`
`\c__mark_empty_tl`

For each class (`#2`) and region (`#1`), we need three token lists: one for top, first, and last. The default value to be returned is "empty".

```
\cs_new_protected:Npn \__mark_init_region:nn #1 #2 {
  \tl_new:c { g__mark_#1_top_   #2 _tl }
  \tl_new:c { g__mark_#1_first_ #2 _tl }
  \tl_new:c { g__mark_#1_last_  #2 _tl }
  \tl_gset_eq:cN  { g__mark_#1_top_   #2 _tl } \c__mark_empty_tl
  \tl_gset_eq:cN  { g__mark_#1_first_ #2 _tl } \c__mark_empty_tl
  \tl_gset_eq:cN  { g__mark_#1_last_  #2 _tl } \c__mark_empty_tl
}
```

All marks will have an identification in the form of a number[47] that is incremented each time a mark insertion happens; therefore the initial empty values should also have such a number, so that data extraction will be uniform.

```
\tl_const:Nn \c__mark_empty_tl { \__mark_value:nn{0}{} }
```

(*End of definition for* `\__mark_init_region:nn` *and* `\c__mark_empty_tl`.)

8.2 Updating mark structures

`\l__mark_box`
`\l__mark_ii_box`
`\g__mark_tmp_tl`
`\g__mark_new_top_tl`

For some operations we need two temporary private boxes and two private global token lists.

```
\box_new:N \l__mark_box
\box_new:N \l__mark_ii_box
\tl_new:N  \g__mark_tmp_tl
\tl_new:N  \g__mark_new_top_tl
```

(*End of definition for* `\l__mark_box` *and others.*)

`\__mark_extract_and_handle_marks:nn`

This is the main macro to extract and handle marks inside some vertical material. It is used by `\mark_update_structure_from_material:nn` (for updating the mark structure for a region based on the marks found) and by `\mark_get_marks_for_reinsertion:nNN` (for extracting marks from some material and prepare for reinserting them later (e.g., out of a box that is placed as a box into the main galley).

```
\cs_new_protected:Npn \__mark_extract_and_handle_marks:nn #1#2 {
```

This macro expects code to handle extracted marks in its first argument and vertical material (not boxed or just consisting of a single vertical box) as its second. It extracts top-level mark information from `#2`, stores them as split marks and then calls `#1` to make use of this information.

If it finds a forced break in the material it removes it and then restarts the attempt without it.

We start with a group to keep most changes local.

```
  \group_begin:
```

Getting the first and last marks out of the material in `#2` is done by putting the material in a box and then doing a split operation to the maximum size possible (which hopefully gets us all of the content).[48] Because this action is used only to get the mark values, we don't want any underfull box warnings so we (locally) turn those off.

```
    \dim_set_eq:NN \tex_splitmaxdepth:D \c_max_dim
    \int_set_eq:NN \tex_vbadness:D      \c_max_int
    \dim_set_eq:NN \tex_vfuzz:D         \c_max_dim
```

There is a further complication: if the material contains infinite shrinking glue then a `\vsplit` operation will balk with a low-level error. Now pages or columns, which are our main concern here, can't have such infinite shrinkage if they are cut straight from the galley, however the use of `\enlargethispage` actually does add some at the very bottom (and also wraps the whole page into a box by itself, so if we leave it this way then a) we get this error and b) we don't see any marks because they are hidden one level down).

[47] There are a few cases where special identification strings are used, e.g., `2.09-compat`.

[48] With normal column material cut from the main galley we should always get all material in one go, but in certain situations, for example, in a multicols environment that contains some `\columnbreak`s a single split operation will not be enough. Thus, this is something we need to handle.

Another possible issue are packages or user code that place stray \vboxes directly into the main galley (an example is marginnote that attaches its marginals in this way). If such boxes end up as the last item on the page we should not unpack them.

All these issues need to be handled, which is done in __mark_prepare_and_extract:nn.

```
      \__mark_prepare_and_extract:nn {#1} {#2}
```

Once all mark classes have been processed, the data structures are updated and we can close the group, which undoes our local changes and retains only the global ones.

```
    \group_end:
  }
```

(*End of definition for* __mark_extract_and_handle_marks:nn.)

__mark_prepare_and_extract:nn This macro does the dirty work. It is not directly integrated in __mark_extract_and_handle_marks:nn because we may have to call it recursively if we find forced breaks.

```
\cs_new_protected:Npn \__mark_prepare_and_extract:nn #1#2 {
```

To handle the \enlargethispage case we do an \unskip to get rid of any glue that is present at the very end of the material and also check if we have then a \vbox as the last item and if so unpack that too, but only under certain conditions, see below. All this is temporary done in a group, just for getting the marks out, so it doesn't affect the final page production.

```
  \vbox_set:Nn \l__mark_box
    {
      #2
      \tex_unskip:D
      \box_set_to_last:N \l__mark_box
```

After having removed the last box from the current list (if there was one) we check whether the vertical list is now empty. If not, then the last box is definitely not the one from \enlargethispage and so we can, and should, leave it alone. Otherwise we check if this last box is a \vbox.

```
      \int_compare:nNnT \tex_lastnodetype:D < 0
        {
          \box_if_vertical:NT \l__mark_box
```

If it is, we unpack the box.

```
            { \vbox_unpack:N \l__mark_box }
        }
```

If it wasn't a vbox, it was either an hbox or there was no box. Given that we are only interested in the marks we don't need put it back in that case.

```
    }
```

We are now ready to \vsplit the box to get at the marks. If the box contains some infinite negative glue the TEX will produce an error complaining about it but it will correctly find the split marks. Given that we can't prevent that error, we hide it from the user and ensure that TEX doesn't stop. The error message still shows in the log, but even that is mitigated as best as possible—see the definition of __mark_vbox_set_split_to_maxdimen:NN for the tricks employed.

```
    \__mark_vbox_set_split_to_maxdimen:NN \l__mark_ii_box \l__mark_box
```

After splitting we check if there is anything left in `\l__mark_box`. If not then the above split has set some split marks that we can then use to finish the extraction:

```
    \box_if_empty:NTF \l__mark_box
      { #1 }
```

If we have a remainder after the split then this means that there was some forced break in the material. We get rid of that by combining the content of the two boxes and restart.

```
      {
⟨*trace⟩
        \__mark_debug:n { \iow_term:e
          { Marks:~ mark~ extraction~needs~ recursion~
            \msg_line_context: } }
⟨/trace⟩
        \__mark_prepare_and_extract:nn {#1}
           { \vbox_unpack:N \l__mark_ii_box
             \vbox_unpack:N \l__mark_box    }
      }
}
```

(*End of definition for* `\__mark_prepare_and_extract:nn`.)

`\__mark_vbox_set_split_to_maxdimen:NN` Split a box to get at its marks without pausing even if TEX is producing an error message because of infinite negative glue in the box. If there is such an error we ensure that it only shows up in the log but not on the terminal.

With a recent TEX engine that knows the primitive `\ignoreprimitiveerror` we can turn this error into a warning (simply by setting this primitive to the value 1).

```
\if_cs_exist:N \tex_ignoreprimitiveerror:D

\cs_new_protected:Npn \__mark_vbox_set_split_to_maxdimen:NN #1#2 {
  \tl_set:Ne \l__mark_saved_parameters_tl
     { \tex_ignoreprimitiveerror:D
       \int_use:N \tex_ignoreprimitiveerror:D
       \scan_stop:
     }
  \tex_ignoreprimitiveerror:D 1 \scan_stop:
  \vbox_set_split_to_ht:NNn #1 #2 { \c_max_dim }
  \l__mark_saved_parameters_tl
}
```

With older TEX engines we have to make use of David's hack below to render the error (fairly) harmless and prevent TEX from stopping. But, of course, the above solution is better because jumping over the error with a local change to the interaction mode still means that TEX thinks there was an error in the run so the return code is no longer 0 (and that might affect workflows that want to test for this).

```
\else:
```

The nice low-level hack by DPC records in the `.log` that a glue shrinkage error is harmless.

We disguise `\c_max_dim` in an odd looking csname, which then shows up as part of the display of an error message if that error happens. This csname forms part of the error display so what you get is something like

```
! Infinite glue shrinkage found in box being split.
<argument> Infinite shrink error above ignored !
l. ...  }
```

which hopefully makes it clear that the error is harmless and and should be ignored by the reader of the `.log`.

```
\cs_set_eq:cN {Infinite~shrink~error~above~ignored~!}\c_max_dim
```

The whole definition of `\__mark_vbox_set_split_to_maxdimen:NN` below is fully expanded, so we have to use a lot of `\exp_not:N` commands to prevent expansion where necessary.

```
\cs_new_protected:Npe \__mark_vbox_set_split_to_maxdimen:NN #1#2 {
```

We start by saving the current interaction and escape char settings.

```
  \tl_set:Ne \exp_not:N \l__mark_saved_parameters_tl
     {
       \tex_interactionmode:D
          \exp_not:N \int_use:N \tex_interactionmode:D \scan_stop:
       \tex_escapechar:D
          \exp_not:N \int_use:N \tex_escapechar:D \scan_stop:
     }
```

Then we change them so that no escape char is printed in the error message (accounts for the missing backslash in front of `Infinite shrink ...`) and we set the interaction to `\nonstopmode` so that the error (if any) just goes into the `.log` file and TEX doesn't stop at that point.

```
  \tex_escapechar:D     -1 \scan_stop:
  \tex_interactionmode:D 0 \scan_stop:
```

Then we do the splitting of the box to `\c_max_dim` to get at the marks. This may generate the error we are worried about, i.e., if the box contains infinite negative glue. However, TEX makes this glue finite and continues, which means we get our split marks which is really all we care about.

```
  \tex_setbox:D #1 \tex_vsplit:D #2 to
```

The `\use:n` may seem pointless, and it is to some extent, but we need it to get our disguised `\c_max_dim` displayed properly as part of the error message if there is one. Without it, the display would show only part of what we want it to show (try it).

```
      \exp_not:N \use:n {
        \use:c{Infinite~shrink~error~above~ignored~!}
      }
```

Finally, we change the escape char and the interaction mode back to what it was before:

```
  \exp_not:N \l__mark_saved_parameters_tl
}
\fi:
```

(*End of definition for* `\__mark_vbox_set_split_to_maxdimen:NN`.)

`\l__mark_saved_parameters_tl` The temporary variable used for resetting escape char and interaction mode.

```
\tl_new:N \l__mark_saved_parameters_tl
```

(*End of definition for* `\l__mark_saved_parameters_tl`.)

`\mark_update_structure_from_material:nn` This function updates the mark structures of a region. The first argument is the region to update and second argument receives the material that holds the marks. Out of this material we extract the first and last marks for all classes (if there are any) to do the assignments.

```
\cs_new_protected:Npn \mark_update_structure_from_material:nn #1#2 {
  \__mark_extract_and_handle_marks:nn
```

Once the marks can be extracted we update the structure from the split marks (code in `\__mark_update_structure_from_splitmarks:n`).

```
    { \__mark_update_structure_from_splitmarks:n {#1} }
    { #2 }
}
```

(*End of definition for* `\mark_update_structure_from_material:nn`. *This function is documented on page* 1066.)

`\__mark_update_structure_from_splitmarks:n`

This macro is called after we have done a `\tex_vsplit:D` operation and the mark data is in the split marks.

```
\cs_new_protected:Npn \__mark_update_structure_from_splitmarks:n #1 {
```

The first thing we do is to copy the current region structure to `previous-...`; this leaves the current structure untouched so we can update it class by class (as is necessary).

```
  \mark_copy_structure:nn { previous-#1 } {#1}
```

After this action we can get first and last marks of the various classes through `\tex_splitfirstmarks:D` and `\tex_splitbotmarks:D`. So now we loop over all classes stored in `\g__mark_classes_seq`.

```
  \seq_map_inline:Nn \g__mark_classes_seq
            {
```

First action: get the last mark from the previous region, i.e., `previous-#1`. But because it is also still inside `#1`, at the moment we use that to construct the name because this is a tiny bit faster. Given that we need this value in various assignments we store it away which avoids unnecessary further csname generations.

```
              \tl_gset_eq:Nc \g__mark_new_top_tl { g__mark_#1_last_##1_tl }
```

This will first of all become the new top mark for the current class.

```
              \tl_gset_eq:cN { g__mark_#1_top_##1_tl } \g__mark_new_top_tl
```

Next action is to get ourselves the new last mark from the material supplied.

```
              \tl_gset:No \g__mark_tmp_tl
                { \tex_splitbotmarks:D \use:c { c__mark_class_##1_mark } }
```

If this mark doesn't exist then obviously neither does the first mark, so both become the last mark from the previous region. We have to be a little careful here: something like `\mark_insert:nn{foo}{}` adds an "empty" mark that should not be confused with no mark at all. But no mark in our material will result in `\g__mark_tmp_tl` being fully empty. This is why we have to make sure that "empty" from `\mark_insert:nn` only appears to be empty when typeset but fails the next test (see below how this is done).

```
              \tl_if_empty:NTF \g__mark_tmp_tl
                {
                  \tl_gset_eq:cN { g__mark_#1_last_ ##1_tl }
                    \g__mark_new_top_tl
                  \tl_gset_eq:cN { g__mark_#1_first_##1_tl }
                    \g__mark_new_top_tl
                }
```

If it wasn't empty, i.e., if it had a real value then we use this value for our new last mark instead.

```
                {
                  \tl_gset_eq:cN { g__mark_#1_last_##1_tl } \g__mark_tmp_tl
```

Because we had a last mark we also have a first mark (which might be the same, but might be not), so we pick that up and assign it to the appropriate token list. This explains why we first checked for the last mark because that makes the processing faster in case there is none.

```
                \tl_gset:co { g__mark_#1_first_##1_tl }
                  {
                    \tex_splitfirstmarks:D
                      \use:c { c__mark_class_##1_mark }
                  }
              }
          }
}
```

(*End of definition for* `\__mark_update_structure_from_splitmarks:n`.)

`\mark_get_marks_for_reinsertion:nNN` This function extracts the marks from the material in the first argument but it does not update any the mark structures. Instead, it collects the marks in the token lists given as the second and third argument, in such a way that they can be reinserted by just executing the token lists.[49]

```
\cs_new_protected:Npn \mark_get_marks_for_reinsertion:nNN #1#2#3 {
```

First we clear the temporary token lists as we haven't seen any marks yet.

```
  \tl_gclear:N \g__mark_first_marks_tl
  \tl_gclear:N \g__mark_last_marks_tl
```

Then we extract all top-level marks, thereby filling the token lists with suitable `\mark_insert:nn` calls.

```
  \__mark_extract_and_handle_marks:nn
```

The first argument holds the code used for filling the token lists and the second holds the material from which all marks should be extracted.

```
    \__mark_get_from_splitmarks:
    { #1 }
```

Finally, we copy the updated (or not updated) temporary token lists to the two that have been supplied when the function was called. By convention "get" operations return their values in local variables and `\__mark_extract_and_handle_marks:nn` runs in a group, which is why we have to use global temporary variables for collecting.

```
  \tl_set_eq:NN #2 \g__mark_first_marks_tl
  \tl_set_eq:NN #3 \g__mark_last_marks_tl
}
```

(*End of definition for* `\mark_get_marks_for_reinsertion:nNN`. *This function is documented on page* 1067.)

`\__mark_get_from_splitmarks:` This function is called after we have done a `\vsplit` to update the split marks. It loops through all mark classes to find out if there are marks for this class and if so updates the global tls used for collecting.

```
\cs_new_protected:Npn  \__mark_get_from_splitmarks:  {
  \seq_map_inline:Nn \g__mark_classes_seq
             {
```

[49] It is probably enough to collect everything in a single token list as long as we put the first marks first and the last marks last). But for extra flexibility, I currently use 2 token lists. This might change when it is really clear that this is never needed.

First we to get the last mark for the current class from the material supplied.

```
                \tl_gset:No \g__mark_tmp_tl
                  { \tex_splitbotmarks:D \use:c { c__mark_class_##1_mark } }
```

If this mark doesn't exist then obviously first mark doesn't either, so we do nothing (other than issuing some debugging info).

We have to be a little careful here: something like `\mark_insert:nn{foo}{}` adds an "empty" mark that we should not confuse with the case where there is no mark at all.

When there is no mark at all we get a truly empty `\g__mark_tmp_tl` as a result. This is why we have to make sure that an "empty" mark generated with `\mark_insert:nn` only appears to be empty when it is typeset, but fails the next test (see below how this is done).

```
                \tl_if_empty:NTF \g__mark_tmp_tl
                  {
⟨*trace⟩
                    \__mark_debug:n { \iow_term:e { Marks:~no~ marks~
                        for~ class~ '##1'~\msg_line_context: } }
⟨/trace⟩
                  }
```

If it wasn't empty, i.e., if it had a real value then we use this value for our new last mark instead. This means we put an appropriate `\mark_insert:nn` statement into `\g__mark_last_marks_tl`.

```
                  {
⟨*trace⟩
                    \__mark_debug:n { \iow_term:e { Marks:~ extract~ last~
```

The mark content in `\g__mark_tmp_tl` may contain arbitrary code that may react badly if it is expanded in a write. So we better avoid that expansion, otherwise debugging might generate spurious errors when turned on.

```
                         mark~ for~ class~ '##1'~ =~ \exp_not:o \g__mark_tmp_tl } }
⟨/trace⟩
                    \tl_gput_right:Ne \g__mark_last_marks_tl
                       { \mark_insert:nn {##1} { \__mark_drop_id:o { \g__mark_tmp_tl } } }
```

Because we had a last mark we also have a first mark (which might be the same, but might not be), so we pick that up and add it to the `\g__mark_first_marks_tl` token list. This explains why we first checked for the last mark because that makes the processing faster in case there is none.

```
⟨*trace⟩
                    \__mark_debug:n { \iow_term:e {
                        Marks:~ extract~ first~ mark~ for~ class~ '##1'~ =~
```

Again no expansion for the mark content.

```
                        \exp_not:o {
                          \tex_splitfirstmarks:D
                          \use:c { c__mark_class_##1_mark }
                        }
                    } }
⟨/trace⟩
                    \tl_gput_right:Ne \g__mark_first_marks_tl
                       { \mark_insert:nn {##1}
                         {
```

We better drop the id from the returned value otherwise they will accumulate in the marks when reinserted.

```
                            \__mark_drop_id:o {
                              \tex_splitfirstmarks:D
                              \use:c { c__mark_class_##1_mark }
                            }
                        }
                    }
                }
            }
}
```

(*End of definition for* `\__mark_get_from_splitmarks:`.)

`\g__mark_first_marks_tl` `\g__mark_last_marks_tl`

These are two global temporary variables used in the code above.

```
\tl_new:N  \g__mark_first_marks_tl
\tl_new:N  \g__mark_last_marks_tl
```

(*End of definition for* `\g__mark_first_marks_tl` *and* `\g__mark_last_marks_tl`.)

`\mark_copy_structure:nn`

This function copies the structure for one region to another, e.g., from `page` to `previous-page` above, or later from `column` to `first-column`, etc.

```
\cs_new_protected:Npn \mark_copy_structure:nn #1#2 {
```

This requires a simple loop through all mark classes copying the token list from one name to the next.

```
  \seq_map_inline:Nn \g__mark_classes_seq
    {
      \tl_gset_eq:cc { g__mark_ #1 _top_   ##1 _tl }
                     { g__mark_ #2 _top_   ##1 _tl }
      \tl_gset_eq:cc { g__mark_ #1 _first_ ##1 _tl }
                     { g__mark_ #2 _first_ ##1 _tl }
      \tl_gset_eq:cc { g__mark_ #1 _last_  ##1 _tl }
                     { g__mark_ #2 _last_  ##1 _tl }
    }
}
```

(*End of definition for* `\mark_copy_structure:nn`. *This function is documented on page* 1066.)

`\mark_clear_structure:n`

This function sets the structure of one region back to an initial state, so that all classes return an empty value if queried.

```
\cs_new_protected:Npn \mark_clear_structure:n #1 {
```

This requires a simple loop through all mark classes.

```
  \seq_map_inline:Nn \g__mark_classes_seq
    {
      \tl_gset_eq:cN { g__mark_ #1 _top_   ##1 _tl }
                     \c__mark_empty_tl
      \tl_gset_eq:cN { g__mark_ #1 _first_ ##1 _tl }
                     \c__mark_empty_tl
      \tl_gset_eq:cN { g__mark_ #1 _last_  ##1 _tl }
                     \c__mark_empty_tl
    }
}
```

(*End of definition for* `\mark_clear_structure:n`. *This function is documented on page* 1067.)

`\mark_set_structure_to_err:n` `\__mark_error:n`

A slight variation is to install a fixed error message as the value.

```
\cs_new_protected:Npn \mark_set_structure_to_err:n #1 {
  \seq_map_inline:Nn \g__mark_classes_seq
      {
        \tl_gset:ce { g__mark_ #1 _top_   ##1 _tl } { \__mark_value:nn{?}{\__mark_error:nn {#
        \tl_gset:ce { g__mark_ #1 _first_ ##1 _tl } { \__mark_value:nn{?}{\__mark_error:nn {#
        \tl_gset:ce { g__mark_ #1 _last_  ##1 _tl } { \__mark_value:nn{?}{\__mark_error:nn {#
      }
}
```

Given that this is used in only one place, we could hardwire the argument which would be a bit more compact, but who knows, perhaps we end up with another reason to use this error command elsewhere, so for now we keep the argument.

```
\cs_new_protected:Npn \__mark_error:nn #1#2 {
  \msg_error:nnnn { mark } { invalid-use } {#1} {#2}
}
```

(*End of definition for* `\mark_set_structure_to_err:n` *and* `\__mark_error:n`. *This function is documented on page* 1066.)

8.3 Placing and retrieving marks

`\mark_insert:nn`

This function puts a mark for some ⟨*class*⟩ at the current point.

```
\cs_new_protected:Npn \mark_insert:nn #1#2
{
  \seq_if_in:NnTF \g__mark_classes_seq {#1}
      {
```

We need to pass the evaluated argument into the mark but protected commands should not expand including those protected using the `\protect` approach of LaTeX 2_ε. We also disable `\label` and the like.[50]

At this point the code eventually should get a public (and a kernel) hook instead of a set of hardwired settings.

```
        \group_begin:
```

Within the group we alter some comments, e.g, `\label` or `\index`, to do the right at this point. This is done in the kernel hook `\@kernel@before@insertmark` which is followed by the public hook `insertmark` that can be used by packages to augment or alter that setup as necessary.

```
          \@kernel@before@insertmark
          \hook_use:n { insertmark }
          \unrestored@protected@xdef \g__mark_tmp_tl
               {
```

To ensure that marks are unique we insert a hidden sequence marker at the beginning of the content of the mark containing the sequence number of the mark.

```
                 \__mark_value:nn{ \int_use:N\g__mark_int }{#2}
               }
⟨*trace⟩
          \__mark_debug:n{ \iow_term:e { Marks:~ set~#1~<-~
```

[50] Straight copy from `latex.ltx` but is this even correct? At least a label in a running header makes little sense if it get set several times! Maybe that needs looking at in the 2e kernel.

```
              '\tl_to_str:V \g__mark_tmp_tl' ~ \msg_line_context: } }
⟨/trace⟩
          \tex_marks:D \use:c { c__mark_class_ #1 _mark }
            {
```

Here is the trick to avoid truly empty marks: if the result from the above processing is empty we add something which eventually becomes empty, but not immediately; otherwise we just put `\g__mark_tmp_tl` in.

```
% This is no longer needed with 1.0f
%              \tl_if_empty:NTF \g__mark_tmp_tl
%                { \exp_not:n { \prg_do_nothing: } }
%                { \exp_not:o { \g__mark_tmp_tl } }
              \exp_not:o { \g__mark_tmp_tl }
            }
        \group_end:
```

A mark introduces a possible break point and in certain situations that should not happen in vertical mode in LaTeX. This may need some checking and possibly cleanup ....

```
        \if@nobreak\ifvmode\nobreak\fi\fi
      }
```

If the mark class was not known, raise an error.

```
      {
        \msg_error:nne { mark } { unknown-class }
          { \tl_to_str:n {#1} }
      }
}
```

(*End of definition for* `\mark_insert:nn`. *This function is documented on page* 1059.)

`\__mark_value:nn` A hidden marker is placed into every mark added by `\mark_insert:nn`. It will not show up in the output but its argument (a counter value that is incremented) makes all marks unique so the test for "equal" is not fooled by two different marks having the same mark text.

```
\cs_new_protected:Npn \__mark_value:nn #1#2 { #2 }
```

(*End of definition for* `\__mark_value:nn`.)

`\@kernel@before@insertmark` By default `\label`, `\index`, and `\glossary` do nothing when the mark is inserted.

insertmark

```
\int_new:N \g__mark_int
\cs_new:Npn \@kernel@before@insertmark {
          \cs_set_eq:NN \label    \scan_stop:
          \cs_set_eq:NN \index    \scan_stop:
          \cs_set_eq:NN \glossary \scan_stop:
```

We count each mark and use that to place a hidden marker in front of the mark text. To ensure that there is no overflow (very unlikely but you never know) we restart every 100000 marks. Thus, if somebody puts more than that number of marks on a single page you could construct a scenario in which that approach fails.

```
          \int_compare:nNnTF \g__mark_int < {99999}
              { \int_gincr:N \g__mark_int }
              { \int_gzero:N \g__mark_int }

}
```

The public hook to augment the setup.

```
\hook_new:n {insertmark}
```

(*End of definition for* `\@kernel@before@insertmark` *and* `insertmark`.)

`\mark_use_top:nn`
`\mark_use_first:nn`
`\mark_use_last:nn`

To retrieve the first, last or top region mark, we grab the appropriate value stored in the corresponding token list variable and pass its contents back. These functions should be used only in output routines and only after `\mark_update_structure_from_material:nn` has acted, otherwise their value will be wrong.

```
\cs_new:Npn \mark_use_first:nn #1#2 { \__mark_use_check:nnn { g__mark_#1_first_#2_tl } {#1} {
\cs_new:Npn \mark_use_last:nn #1#2 { \__mark_use_check:nnn { g__mark_#1_last_#2_tl } {#1} {#2
\cs_new:Npn \mark_use_top:nn #1#2 { \__mark_use_check:nnn { g__mark_#1_top_#2_tl } {#1} {#2}
```

If used with an unknown class or region these commands will generate an error. If that happens in an expandable context then the error generation is delayed (e.g., if used in a `\section`) and happens when the code is finally used in typesetting, e.g., in the TOC or a running header. If used in a `\typeout` you only see something like `\__mark_error:n{page}`. This is not too good, but probably better than low-level errors, I guess, and I don't want to use an expandable error because of the size restrictions in such error messages.

```
\cs_new:Npn \__mark_use_check:nnn #1#2#3 {
  \tl_if_eq:cNTF {#1} \relax
     { \__mark_error:nn {#2} {#3} }
     { \__mark_drop_id:v {#1} }
}
```

Each mark starts with an id and while the id does not print it is nevertheless better to remove it when returning the mark, so that downstream manipulation of the data doesn't have to deal with it. This is what the `\exp_not:o` accomplishes.

```
\cs_new:Npn \__mark_drop_id:n #1 { \exp_not:o { #1 } }
\cs_generate_variant:Nn \__mark_drop_id:n { o, v }
```

(*End of definition for* `\mark_use_top:nn`, `\mark_use_first:nn`, *and* `\mark_use_last:nn`. *These functions are documented on page* 1060.)

8.4 Comparing mark values

`\mark_if_eq:nnnnTF`
`\mark_if_eq:nnnnnnTF`

Test if in a given region (`#1`) for a given class (`#2`) the marks in position `#3` and `#4` (top, first, or last) are identical

```
\prg_new_conditional:Npnn \mark_if_eq:nnnn #1#2#3#4 { T , F , TF }
{
  \tl_if_eq:ccTF { g__mark_ #1 _#3_ #2 _tl }
                 { g__mark_ #1 _#4_ #2 _tl }
                 \prg_return_true:
                 \prg_return_false:
}
```

The fully general test (with two triplets of the form ⟨*region*⟩, ⟨*class*⟩, and ⟨*position*⟩) is this:

```
\prg_new_conditional:Npnn \mark_if_eq:nnnnnn #1#2#3#4#5#6 { T , F , TF }
{
  \tl_if_eq:ccTF { g__mark_ #1 _#3_ #2 _tl }
                 { g__mark_ #4 _#6_ #5 _tl }
                 \prg_return_true:
```

```
                \prg_return_false:
}
```

(*End of definition for* `\mark_if_eq:nnnnTF` *and* `\mark_if_eq:nnnnnnTF`. *These functions are documented on page 1060.*)

8.5 Messages

Mark errors are LaTeX kernel errors:

```
\prop_gput:Nnn \g_msg_module_type_prop { mark } { LaTeX }
\msg_new:nnnn { mark } { class-already-defined }
  { Mark~class~'#1'~already~defined }
  {
    \c__msg_coding_error_text_tl
    LaTeX~was~asked~to~define~a~new~mark~class~called~'#1':~
    this~mark~class~already~exists.
    \c__msg_return_text_tl
  }
\msg_new:nnnn { mark } { unknown-class }
  { Unknown~mark~class~'#1'. }
  {
    \c__msg_coding_error_text_tl
    LaTeX~was~asked~to~manipulate~a~mark~of~class~'#1',~
    but~this~class~of~marks~does~not~exist.
  }
```

The next error can also happen if the mark class is unknown, so this should perhaps be separated into two different errors.

```
\msg_new:nnnn { mark } { invalid-use }
  { Mark~region~'#1'~not~usable~or~class~'#2'~unknown }
  {
    \c__msg_coding_error_text_tl
    The~region~'#1'~is~either~not~known~or~data~for~it~
    still~needs~to~be~assembled,~e.g.,~last-column~
    while~building~the~first-column.~
    Also~possible:~the~class~name~'#2'~is~misspelled.
    \c__msg_return_text_tl
  }
```

8.6 Debugging the mark structures

Code and commands in this section are not final, it needs more experimentation to see what kind of tracing information is going to be useful in practice. For now the tracing is mainly meant to be used for code testing and not so much for application testing.

It is quite likely that the commands and the behavior of the tracing might change in the future once we gained some experience with it.

`\g__mark_debug_bool` Holds the current debugging state.

```
\bool_new:N \g__mark_debug_bool
```

(*End of definition for* `\g__mark_debug_bool`.)

\mark_debug_on:
\mark_debug_off:
__mark_debug:n
__mark_debug_gset:

Turns debugging on and off by redefining `\__mark_debug:n`.

```
\cs_new_eq:NN \__mark_debug:n \use_none:n
\cs_new_protected:Npn \mark_debug_on:
  {
    \bool_gset_true:N \g__mark_debug_bool
    \__mark_debug_gset:
  }
\cs_new_protected:Npn \mark_debug_off:
  {
    \bool_gset_false:N \g__mark_debug_bool
    \__mark_debug_gset:
  }
\cs_new_protected:Npn \__mark_debug_gset:
  {
    \cs_gset_protected:Npe \__mark_debug:n ##1
      { \bool_if:NT \g__mark_debug_bool {##1} }
  }
```

(*End of definition for* `\mark_debug_on:` *and others. These functions are documented on page* 1063.)

\DebugMarksOn
\DebugMarksOff

CamelCase commands for debugging.

```
\cs_new_eq:NN \DebugMarksOn   \mark_debug_on:
\cs_new_eq:NN \DebugMarksOff  \mark_debug_off:
```

(*End of definition for* `\DebugMarksOn` *and* `\DebugMarksOff`. *These functions are documented on page* 1063.)

__mark_class_status:nnn

Shows the mark values across all regions for one mark class (`#2`).

The first argument gives some ⟨*info*⟩ to help in identifying where the command was called, the second is the class and the third holds the number of `mcol-...` we should display: inside a `multicols` environment this will be `\col@number`, in LaTeX's normal output routines it will be `0`.

```
⟨*trace⟩
\cs_new_protected:Npn \__mark_class_status:nnn #1#2#3 {
   \typeout{ Marks:~#2~ #1:}
   \__mark_region_status:nnn  {#2}{ page~ (previous) }  { previous-page }
   \__mark_region_status:nnn  {#2}{ page~ (current)~ }  { page }
   \__mark_region_status:nnn  {#2}{ column~ (previous) }{ previous-column }
   \__mark_region_status:nnn  {#2}{ column~ (current)~ }{ column }
   \__mark_region_status:nnn  {#2}{ column~ (first) }   { first-column }
   \__mark_region_status:nnn  {#2}{ column~ (last)~ }   { last-column }
```

Then finish by displaying a subset of the `mcol-...` regions: none (`0`) in the standard LaTeX output routine and `\col@number` within a `multicols` environment.

```
   \int_step_inline:nn {#3}
     {
       \__mark_region_status:nnn  {#2}{ column~ (##1)~ }   { mcol-##1 }
     }
}
```

(*End of definition for* `\__mark_class_status:nnn`.)

__mark_region_status:nnn Display the top, first, and last mark of a region unless none of them exist or all of them are empty.

```
\cs_new_protected:Npn \__mark_region_status:nnn #1#2#3 {
  \group_begin:
  \cs_set:Npn \__mark_value:nn ##1##2{ \exp_not:n{ {##1} ~ ##2 } }
  \tl_if_exist:cT  { g__mark_#3_last_ #1 _tl }
     {
       \tl_if_eq:cNF { g__mark_#3_last_ #1 _tl } \c__mark_empty_tl
          {
            \typeout{\@spaces #2 =
               ~|~ \use:c { g__mark_#3_top_ #1 _tl }    ~|~
                   \use:c { g__mark_#3_first_ #1 _tl }  ~|~
                   \use:c { g__mark_#3_last_ #1 _tl }   ~|
            }
          }
     }
  \group_end:
}
```

(*End of definition for* `\__mark_region_status:nnn`.)

__mark_status:nn Show a snapshot of all mark class values across all regions. The first argument is a string to identify the output, the second argument is the number of `mcol-...` regions to show. Outside of a `multicols` environment this is normally set to 0.

```
\cs_new_protected:Npn \__mark_status:nn #1#2
  {
    \seq_map_inline:Nn \g__mark_classes_seq
      { \__mark_class_status:nnn {#1} {##1} {#2} }
  }
⟨/trace⟩
```

(*End of definition for* `\__mark_status:nn`.)

\ShowMarksAt Debugging helper that displays a snapshot of all known mark structures. The first argument is a text string that is displayed to help identifying when the snapshot was made. The optional second one determines how many `mcol-...` regions are displayed (by default 4).

This may not stay like this (or at all), which is why it isn't yet documented as an official command.

```
\NewDocumentCommand \ShowMarksAt {m O{4} } {
⟨*trace⟩
     \__mark_debug:n { \__mark_status:nn {#1}{#2} }
⟨/trace⟩
}
```

(*End of definition for* `\ShowMarksAt`.)

8.7 Designer-level interfaces

\NewMarkClass
\InsertMark These two are identical to the L3 programming layer commands.

```
\cs_new_eq:NN  \NewMarkClass \mark_new_class:n
\@onlypreamble \NewMarkClass

\cs_new_eq:NN \InsertMark  \mark_insert:nn
```

(*End of definition for* `\NewMarkClass` *and* `\InsertMark`. *These functions are documented on page* 1059.)

`\TopMark` `\FirstMark` `\LastMark`

The following commands take an optional argument that defaults to page. There is no checking that the region is actually valid. If not there is simply an empty return.

```
\NewExpandableDocumentCommand \FirstMark { O{page} m }
                     { \mark_use_first:nn {#1}{#2} }
\NewExpandableDocumentCommand \LastMark { O{page} m }
                     { \mark_use_last:nn {#1}{#2} }
\NewExpandableDocumentCommand \TopMark { O{page} m }
                     { \mark_use_top:nn {#1}{#2} }
```

(*End of definition for* `\TopMark`, `\FirstMark`, *and* `\LastMark`. *These functions are documented on page* 1060.)

`\IfMarksEqualTF` `\IfMarksEqualT` `\IfMarksEqualF`

We only provide CamelCase commands for the case with one region (optional) and one class. One could think of also providing a version for the general case with several optional arguments, but use cases for this are most likely rare, so not done yet.

```
\NewExpandableDocumentCommand \IfMarksEqualTF {O{page}mmm} {
  \mark_if_eq:nnnnTF {#1}{#2}{#3}{#4}
}
\NewExpandableDocumentCommand \IfMarksEqualT {O{page}mmm} {
  \mark_if_eq:nnnnT {#1}{#2}{#3}{#4}
}
\NewExpandableDocumentCommand \IfMarksEqualF {O{page}mmm} {
  \mark_if_eq:nnnnF {#1}{#2}{#3}{#4}
}
```

(*End of definition for* `\IfMarksEqualTF`, `\IfMarksEqualT`, *and* `\IfMarksEqualF`. *These functions are documented on page* 1060.)

9 LaTeX 2ε integration

9.1 Core LaTeX 2ε integration

`\__mark_update_singlecol_structures:`

This command updates the mark structures if we are producing a single column document.

```
\cs_new_protected:Npn \__mark_update_singlecol_structures: {
```

First we update the `page` region (which also updates the `previous-page`.

The `\@outputbox` is normally in `\vbox` in LaTeX but we can't take that for granted (an amsmath test document changed it to an `\hbox` just to trip me up) so we are a little careful with unpack now.

```
  \box_if_vertical:NTF \@outputbox
      {
        \mark_update_structure_from_material:nn {page}
           { \vbox_unpack:N  \@outputbox }
      }
      {
        \mark_update_structure_from_material:nn {page}
           { \hbox_unpack:N  \@outputbox }
      }
```

Then we provide the necessary updates for the aliases.

```
  \mark_copy_structure:nn {previous-column}{previous-page}
  \mark_copy_structure:nn {column}{page}
  \mark_copy_structure:nn {first-column}{page}
  \mark_copy_structure:nn {last-column}{page}
⟨*trace⟩
% move this into status itself?
     \__mark_debug:n
         {
           \__mark_status:nn
                { in~ OR~ (
                  \legacy_if:nTF {@twoside}
                         { twoside-
                           \int_if_odd:nTF \c@page
                               { odd }{ even }
                         }
                         { oneside }
                         )
                }
                {0}
         }
⟨/trace⟩
}
```

(*End of definition for* `\__mark_update_singlecol_structures:`.)

`\__mark_update_dblcol_structures:` This commands handles the updates if we are doing two-column pages.

```
\cs_new_protected:Npn \__mark_update_dblcol_structures: {
```

First we update the `column` and `previous-column` regions using the material assembled in `\@outputbox`.

```
  \box_if_vertical:NTF \@outputbox
      {
        \mark_update_structure_from_material:nn {column}
           { \vbox_unpack:N  \@outputbox }
      }
      {
        \mark_update_structure_from_material:nn {column}
           { \hbox_unpack:N  \@outputbox }
      }
```

How we have to update the alias regions depends on whether or not `\@opcol` was called to process the first column or to produce the completed page

```
  \legacy_if:nTF {@firstcolumn}
    {
```

If we are processing the first column then `column` is our `first-column` and there is no `last-column` yet, so we make those an error.

```
      \mark_copy_structure:nn {first-column}{column}
      \mark_set_structure_to_err:n {last-column}
    }
    {
```

If we produce the completed page then the `first-column` is the same as the new `previous-column`. However, the structure should already be correct if you think about it (because is was set to `column` last time which is now the `previous-column`), thus there is no need to make an update.

```
%     \mark_copy_structure:nn {first-column}{previous-column}
```

However, we now have a proper `last-column` so we assign that.

```
      \mark_copy_structure:nn {last-column}{column}
```

What now remains doing is to update the `page` and `previous-page` regions. For this we have to copy the settings in `page` into `previous-page` and then update `page` such that the top and first marks are taken from the `first-column` region and the last marks are taken from the `last-column` region. All this has to be done for all mark classes so we loop over our sequence.

Note that one loop is needed if we arrange the copy statements in a suitable way.

```
      \seq_map_inline:Nn \g__mark_classes_seq
        {
```

The `previous-page` updates need to come before the updates for `page` region because otherwise the values to copy are already overwritten. necessary values.

```
          \tl_gset_eq:cc { g__mark_previous-page_top_   ##1 _tl }
                         { g__mark_page_top_           ##1 _tl }
          \tl_gset_eq:cc { g__mark_previous-page_first_ ##1 _tl }
                         { g__mark_page_first_         ##1 _tl }
          \tl_gset_eq:cc { g__mark_previous-page_last_  ##1 _tl }
                         { g__mark_page_last_          ##1 _tl }
```

To update the `top` we only have to copy what is in `first-column`:

```
          \tl_gset_eq:cc { g__mark_page_top_          ##1 _tl }
                         { g__mark_first-column_top_  ##1 _tl }

```

Updating the `first` mark for the `page` region is more complicated. We first have to find out of there is any mark in the first column (this can be done by comparing the `top` and the `first` mark of that region).

```
          \tl_if_eq:ccTF { g__mark_first-column_top_   ##1 _tl }
                         { g__mark_first-column_first_ ##1 _tl }
            {
```

If there is no mark in the first column we copy the first mark of the last column. If that doesn't contain a mark we still get the right result because the first mark is then equal to the top mark.

```
              \tl_gset_eq:cc { g__mark_page_first_        ##1 _tl }
                             { g__mark_last-column_first_ ##1 _tl }
            }
            {
```

On the other hand, if there is a mark in the first column we copy over the `first` mark from that column.

```
              \tl_gset_eq:cc { g__mark_page_first_         ##1 _tl }
                             { g__mark_first-column_first_ ##1 _tl }
            }
```

The logic for the `last` page mark is again simple, we can just copy the value in the `last` mark of the last column. If that column doesn't contain any marks, then the value in `last` will be automatically the same as the `last` from the first column.

```
          \tl_gset_eq:cc { g__mark_page_last_        ##1 _tl }
                         { g__mark_last-column_last_ ##1 _tl }
        }
    }
⟨*trace⟩
     \__mark_debug:n
         {
           \__mark_status:nn
              { in~ OR~ (
                \legacy_if:nTF {@twoside}
                       { twoside-
                         \int_if_odd:nTF \c@page
                             { odd }{ even }
                       }
                       { oneside }
                       \space
                       \legacy_if:nTF {@firstcolumn}
                              { first~ }{ second~ }
                              column )
              }
              {0}
         }
⟨/trace⟩
}
```

(*End of definition for* `\__mark_update_dblcol_structures:`.)

9.2 Other LaTeX 2_ε output routines

This section will cover support for packages that alter the LaTeX output routine (as necessary). The support for multicol (for now) is handled directly in that package.

```
⟨@@=⟩
```

`\@expl@@@mark@update@singlecol@structures@@`

```
\cs_new_eq:NN  \@expl@@@mark@update@singlecol@structures@@
               \__mark_update_singlecol_structures:
```

(*End of definition for* `\@expl@@@mark@update@singlecol@structures@@`.)

`\@expl@@@mark@update@dblcol@structures@@`

```
\cs_new_eq:NN  \@expl@@@mark@update@dblcol@structures@@
               \__mark_update_dblcol_structures:
```

(*End of definition for* `\@expl@@@mark@update@dblcol@structures@@`.)

9.3 Rollback information

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{ltmarks}%
⟨latexrelease⟩                  {Undo~Marks~handling}
⟨latexrelease⟩
```

We keep the interface commands around even if we roll back in case they are used in packages that don't roll back. Not likely to do a lot of good, but then there is not much we can do, but this at least they won't give unknown csname errors.

```
⟨latexrelease⟩\DeclareRobustCommand \NewMarkClass[1]{}
⟨latexrelease⟩\DeclareRobustCommand \InsertMark[2]{}
⟨latexrelease⟩\RenewExpandableDocumentCommand \FirstMark { O{} m } { }
⟨latexrelease⟩\RenewExpandableDocumentCommand \LastMark { O{} m } { }
⟨latexrelease⟩\RenewExpandableDocumentCommand \TopMark { O{} m } { }
⟨latexrelease⟩\RenewExpandableDocumentCommand \IfMarksEqualTF { O{} mmm }{ }
⟨latexrelease⟩
```

Same here, this avoided extra roll back code in the OR.

```
⟨latexrelease⟩\let \@expl@@@mark@update@singlecol@structures@@ \relax
⟨latexrelease⟩\let \@expl@@@mark@update@dblcol@structures@@ \relax
⟨latexrelease⟩
⟨latexrelease⟩
⟨latexrelease⟩\EndModuleRelease
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
```

Reset module prefix:

```
⟨@@=⟩
```

File 49
ltpage.dtx

1 Page styles and related commands

1.1 Page Style Commands

`\pagestyle{`⟨*style*⟩`}` : sets the page style of the current and succeeding pages to *style*

`\thispagestyle{`⟨*style*⟩`}` : sets the page style of the current page only to *style*.

To define a page style *style*, you must define `\ps@`*style* to set the page style parameters.

1.2 How a page style makes running heads and feet

The `\ps@`... command defines the macros `\@oddhead`, `\@oddfoot`, `\@evenhead`, and `\@evenfoot` to define the running heads and feet. (See output routine.) To make headings determined by the sectioning commands, the page style defines the commands `\chaptermark`, `\sectionmark`, etc., where `\chaptermark{`⟨*text*⟩`}` is called by `\chapter` to set a mark. The `\...mark` commands and the `\...head` macros are defined with the help of the following macros.

(All the `\...mark` commands should be initialized to no-ops.)

1.3 marking conventions

LaTeX extends TeX's `\mark` facility by producing two kinds of marks a 'left' and a 'right' mark, using the following commands:
`\markboth{`⟨*left*⟩`}{`⟨*right*⟩`}` : Adds both marks.
`\markright{`⟨*right*⟩`}` : Adds a 'right' mark.
`\leftmark` : Used in the output routine, gets the current 'left' mark. Works like TeX's `\botmark`.
`\rightmark` : Used in the output routine, gets the current 'right' mark. Works like TeX's `\firstmark`. The marking commands work reasonably well for right marks 'numbered within' left marks—e.g., the left mark is changed by a `\chapter` command and the right mark is changed by a `\section` command. However, it does produce somewhat anomalous results if 2 `\markboth`'s occur on the same page.

Commands like `\tableofcontents` that should set the marks in some page styles use a `\@mkboth` command, which is `\let` by the pagestyle command (`\ps@...`) to `\markboth` for setting the heading or to `\@gobbletwo` to do nothing.

```
1 ⟨*2ekernel⟩
```

`\pagestyle` User command to set the page style for this and following pages.

```
2 \def\pagestyle#1{%
3   \@ifundefined{ps@#1}%
4     \undefinedpagestyle
5     {\@nameuse{ps@#1}}}
```

(*End of definition for* `\pagestyle`.)

\thispagestyle User command to set the page style for this page only.

```
\def\thispagestyle#1{%
  \@ifundefined{ps@#1}%
    \undefinedpagestyle
    {\global\@specialpagetrue\gdef\@specialstyle{#1}}}
```

(*End of definition for* `\thispagestyle`.)

\ps@empty The empty page style: No head or foot line.

```
\def\ps@empty{%
  \let\@mkboth\@gobbletwo\let\@oddhead\@empty\let\@oddfoot\@empty
  \let\@evenhead\@empty\let\@evenfoot\@empty}
```

(*End of definition for* `\ps@empty`.)

\ps@plain The plain page style: No head, centered page number in foot.

```
\def\ps@plain{\let\@mkboth\@gobbletwo
     \let\@oddhead\@empty\def\@oddfoot{\reset@font\hfil\thepage
     \hfil}\let\@evenhead\@empty\let\@evenfoot\@oddfoot}
```

(*End of definition for* `\ps@plain`.)

\@leftmark
\@rightmark We implement `\@leftmark` and `\@rightmark` in terms of already defined commands to save token space. We can't get rid of them since they are sometimes used in applications.

```
\let\@leftmark\@firstoftwo
\let\@rightmark\@secondoftwo
```

(*End of definition for* `\@leftmark` *and* `\@rightmark`.)

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease⟩                    {\markboth}{Drop legacy mark support}%
```

\markboth
\markright User commands for setting LaTeX marks.

Test for `@nobreak` added 15 Apr 86 in `\markboth` and `\markright` letting `\label` and `\index` to `\relax` added 22 Feb 86 so these commands can appear in sectioning command arguments RmS 91/06/21 Same for `\glossary`

```
\ExplSyntaxOn
\DeclareRobustCommand*\markboth[2]{%
    \mark_insert:nn{2e-left}{#1}
    \mark_insert:nn{2e-right}{#2}
    \tl_if_empty:nF{#2}{ \mark_insert:nn{2e-right-nonempty}{#2} }
}

\DeclareRobustCommand*\markright[1]{%
    \mark_insert:nn{2e-right}{#1}
    \tl_if_empty:nF{#1}{ \mark_insert:nn{2e-right-nonempty}{#1} }
}
\ExplSyntaxOff
```

(*End of definition for* `\markboth` *and* `\markright`. *These functions are documented on page 1064.*)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2022/06/01}%
⟨latexrelease⟩                  {\markboth}{New mark support}%
⟨latexrelease⟩\ExplSyntaxOn
⟨latexrelease⟩\DeclareRobustCommand*\markboth[2]{%
⟨latexrelease⟩  \begingroup
⟨latexrelease⟩    \let\label\relax \let\index\relax \let\glossary\relax
⟨latexrelease⟩    \unrestored@protected@xdef\@themark {{#1}{#2}}%
⟨latexrelease⟩    \@temptokena \expandafter{\@themark}%
⟨latexrelease⟩    \mark_insert:nn{2e-left}{#1}
⟨latexrelease⟩    \mark_insert:nn{2e-right}{#2}
⟨latexrelease⟩    \tl_if_empty:nF{#2}{ \mark_insert:nn{2e-right-nonempty}{#2} }
⟨latexrelease⟩    \mark{\the\@temptokena}%
⟨latexrelease⟩  \endgroup
⟨latexrelease⟩  \if@nobreak\ifvmode\nobreak\fi\fi}
⟨latexrelease⟩\DeclareRobustCommand*\markright[1]{%
⟨latexrelease⟩  \begingroup
⟨latexrelease⟩    \let\label\relax \let\index\relax \let\glossary\relax
⟨latexrelease⟩    \expandafter\@markright\@themark {#1}%
⟨latexrelease⟩    \@temptokena \expandafter{\@themark}%
⟨latexrelease⟩    \mark_insert:nn{2e-right}{#1}
⟨latexrelease⟩    \tl_if_empty:nF{#1}{ \mark_insert:nn{2e-right-nonempty}{#1} }
⟨latexrelease⟩    \mark{\the\@temptokena}%
⟨latexrelease⟩  \endgroup
⟨latexrelease⟩  \if@nobreak\ifvmode\nobreak\fi\fi}
⟨latexrelease⟩\ExplSyntaxOff
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩                  {\markboth}{Make commands robust}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand*\markboth[2]{%
⟨latexrelease⟩  \begingroup
⟨latexrelease⟩    \let\label\relax \let\index\relax \let\glossary\relax
⟨latexrelease⟩    \unrestored@protected@xdef\@themark {{#1}{#2}}%
⟨latexrelease⟩    \@temptokena \expandafter{\@themark}%
⟨latexrelease⟩    \mark{\the\@temptokena}%
⟨latexrelease⟩  \endgroup
⟨latexrelease⟩  \if@nobreak\ifvmode\nobreak\fi\fi}
⟨latexrelease⟩\DeclareRobustCommand*\markright[1]{%
⟨latexrelease⟩  \begingroup
⟨latexrelease⟩    \let\label\relax \let\index\relax \let\glossary\relax
⟨latexrelease⟩    \expandafter\@markright\@themark {#1}%
⟨latexrelease⟩    \@temptokena \expandafter{\@themark}%
⟨latexrelease⟩    \mark{\the\@temptokena}%
⟨latexrelease⟩  \endgroup
⟨latexrelease⟩  \if@nobreak\ifvmode\nobreak\fi\fi}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\markboth}{Make commands robust}%
⟨latexrelease⟩
```

Using `\kernel@make@fragile` doesn't really work if there are more redefinitions later on, so we have to repeat the above definition first and then undo it (or use `\def` directly).

```
⟨latexrelease⟩%\kernel@make@fragile\markboth
⟨latexrelease⟩%\kernel@make@fragile\markright
⟨latexrelease⟩\def\markboth#1#2{%
⟨latexrelease⟩  \begingroup
⟨latexrelease⟩    \let\label\relax \let\index\relax \let\glossary\relax
⟨latexrelease⟩    \unrestored@protected@xdef\@themark {{#1}{#2}}%
⟨latexrelease⟩    \@temptokena \expandafter{\@themark}%
⟨latexrelease⟩    \mark{\the\@temptokena}%
⟨latexrelease⟩  \endgroup
⟨latexrelease⟩  \if@nobreak\ifvmode\nobreak\fi\fi}
⟨latexrelease⟩\def\markright#1{%
⟨latexrelease⟩  \begingroup
⟨latexrelease⟩    \let\label\relax \let\index\relax \let\glossary\relax
⟨latexrelease⟩    \expandafter\@markright\@themark {#1}%
⟨latexrelease⟩    \@temptokena \expandafter{\@themark}%
⟨latexrelease⟩    \mark{\the\@temptokena}%
⟨latexrelease⟩  \endgroup
⟨latexrelease⟩  \if@nobreak\ifvmode\nobreak\fi\fi}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

`\leftmark`
`\rightmark`

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease⟩                  {\leftmark}{Use new mark mechanism}%
\ExplSyntaxOn
\cs_new:Npn \leftmark {\mark_use_last:nn{page}{2e-left}}
\cs_new:Npn \rightmark {\mark_use_first:nn{page}{2e-right}}
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\leftmark}{Use new mark mechanism}%
⟨latexrelease⟩
⟨latexrelease⟩\def\leftmark{\expandafter\@leftmark\botmark\@empty\@empty}
⟨latexrelease⟩\def\rightmark{\expandafter\@rightmark\firstmark\@empty\@empty}
```

```
⟨latexrelease⟩\def\@themark{{}{}}
⟨latexrelease⟩\def\@markright#1#2#3{\@temptokena {#1}%
⟨latexrelease⟩\unrestored@protected@xdef\@themark{{\the\@temptokena}{#3}}}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

`\@markright`
`\mark`

(*End of definition for* `\leftmark` *and others. These functions are documented on page* 1064.)

`\raggedbottom` `\raggedbottom` typesets pages with no vertical stretch, so they have their natural height instead of all being exactly the same height. (Uses a space of .0001fil to avoid interfering with the 1fil space of `\newpage`.)

```
\DeclareRobustCommand\raggedbottom{%
  \def\@textbottom{\vskip \z@ \@plus.0001fil}\let\@texttop\relax}
```

(*End of definition for* `\raggedbottom`.)

`\flushbottom` `\flushbottom`: Inverse of `\raggedbottom` — makes all pages the same height.

```
\DeclareRobustCommand\flushbottom{%
  \let\@textbottom\relax \let\@texttop\relax}
```

(*End of definition for* `\flushbottom`.)

`\sloppy` `\sloppy` will never (well, hardly ever) produce overfull boxes, but may produce underfull ones. (14 June 85)

```
\DeclareRobustCommand\sloppy{%
  \tolerance 9999%
  \emergencystretch 3em%
  \hfuzz .5\p@
  \vfuzz\hfuzz}
```

(*End of definition for* `\sloppy`.)

`sloppypar` (*env.*) A sloppypar environment is equivalent to `{\par \sloppy ... \par}`.

```
\def\sloppypar{\par\sloppy}
\def\endsloppypar{\par}
```

`\fussy` Resets TEX's parameters to their normal finicky values.

```
\DeclareRobustCommand\fussy{%
  \emergencystretch\z@
  \tolerance 200%
  \hfuzz .1\p@
  \vfuzz\hfuzz}
```

(*End of definition for* `\fussy`.)

`\overfullrule` LATEX default is no overfull box rule. Changed by document class option.

```
\overfullrule 0pt
```

(*End of definition for* `\overfullrule`.)

```
⟨/2ekernel⟩
```

File 50
ltclass.dtx

1 Introduction

This file implements the following declarations, which replace `\documentstyle` in LaTeX 2ε documents.

Note that old documents containing `\documentstyle` will be run using a compatibility option—thus keeping everyone happy, we hope!

The overall idea is that there are two types of 'style files': 'class files' which define elements and provide a default formatting for them; and 'packages' which provide extra functionality. One difference between LaTeX 2ε and LaTeX2.09 is that LaTeX 2ε packages may have options. Note that options to classes packages may be implemented such that they input files, but these file names are not necessarily directly related to the option name.

2 User interface

`\documentclass[`⟨*main-option-list*⟩`]{`⟨*class*⟩`}[`⟨*version*⟩`]`

There must be exactly one such declaration, and it must come first. The ⟨*main-option-list*⟩ is a list of options which can modify the formatting of elements which are defined in the ⟨*class*⟩ file as well as in all following `\usepackage` declarations (see below). The ⟨*version*⟩ is a version number, beginning with a date in the format `YYYY/MM/DD`. If an older version of the class is found, a warning is issued.

`\documentstyle[`⟨*main-option-list*⟩`]{`⟨*class*⟩`}[`⟨*version*⟩`]`

The `\documentstyle` declaration is kept in order to maintain upward compatibility with LaTeX2.09 documents. It is similar to `\documentclass`, but it causes all options in ⟨*main-option-list*⟩ that the ⟨*class*⟩ does not use to be passed to `\RequirePackage` after the options have been processed. This maintains compatibility with the 2.09 behaviour. Also a flag is set to indicate that the document is to be processed in LaTeX2.09 compatibility mode. As far as most packages are concerned, this only affects the warnings and errors LaTeX generates. This flag does affect the definition of font commands, and `\sloppy`.

`\usepackage[`⟨*package-option-list*⟩`]{`⟨*package-list*⟩`}[`⟨*version*⟩`]`

There can be any number of these declarations. All packages in ⟨*package-list*⟩ are called with the same options.

Each ⟨*package*⟩ file defines new elements (or modifies those defined in the ⟨*class*⟩), and thus extends the range of documents which can be processed. The ⟨*package-option-list*⟩ is a list of options which can modify the formatting of elements defined in the ⟨*package*⟩ file. The ⟨*version*⟩ is a version number, beginning with a date in the format `YYYY/MM/DD`. If an older version of the package is found, a warning is issued.

Each package is loaded only once. If the same package is requested more than once, nothing happens, unless the package has been requested with options that were not given the first time it was loaded, in which case an error is produced.

As well as processing the options given in the ⟨*package-option-list*⟩, each package processes the ⟨*main-option-list*⟩. This means that options that affect all of the packages can be given globally, rather than repeated for every package.

Note that class files have the extension `.cls`, packages have the extension `.sty`.

`filecontents` (*env.*) The environment `filecontents` is intended for passing the contents of packages, options, or other files along with a document in a single file. It has one argument, which is the name of the file to create. If that file already exists (maybe only in the current directory if the OS supports a notion of a 'current directory' or 'default directory') then nothing happens (except for an information message) and the body of the environment is bypassed. Otherwise, the body of the environment is written verbatim to the file name given as the first argument, together with some comments about how it was produced.

The environment can also be called with an optional argument which is used to alter some of its behavior: option `force` or `overwrite` will allow for overwriting existing files, option `nosearch` will only check the current directory when looking if the file exists. This can be useful if you want to generate a local (modified) copy of some file that is already in the search tree of TEX. Finally, you can use `noheader` to prevent it from writing the standard blurb at the top of the file (this is actually the same as using the star form of the environment).

As the environment is intended to be used on document level to bundle different files together, by default it refuses to overwrite files. If the option `force` is used then it always writes the file, but it generates a warning if it overwrites (or might overwrite) an existing file.[51] If necessary, this warning can be suppressed by also using the option `nowarn` but for most use cases we recommend informing the user when an overwrite happens and therefore to not add this option.

The environment is now allowed anywhere in the document, but to ensure that all the necessary packages and options are available when the document is run, it is normally best to place it at the top of your file (before `\documentclass`). A possible use case for using it inside the document body is if you want to reuse some text several times in the document you could then write it and later use `\input` to retrieve it where needed.

The begin and end tags should each be on a line by itself.

2.1 Option processing

When the options are processed, they are divided into two types: *local* and *global*:

- For a class, the options in the `\documentclass` command are local.
- For a package, the options in the `\usepackage` command are local, and the options in the `\documentclass` command are global.

The options for `\documentclass` and `\usepackage` are processed in the following way:

1. The local and global options that have been declared (using `\DeclareOption` as described below) are processed first.

   In the case of `\ProcessOptions`, they are processed in the order that they were declared in the class or package.

   In the case of `\ProcessOptions*`, they are processed in the order that they appear in the option-lists. First the global options, and then the local ones.

[51] TEX might see a file in the `texmf/` tree that is not in current directory, in which case no overwrite happens; but it can't distinguish that case from the one in which an overwrite takes place.

2. Any remaining local options are dealt with using the default option (declared using the `\DeclareOption*` declaration described below). For document classes, this usually does nothing, but records the option on a list of unused options. For packages, this usually produces an error.

Finally, when `\begin{document}` is reached, if there are any global options which have not been used by either the class or any package, the system will produce a warning.

3 Class and Package interface

3.1 Class name and version

`\ProvidesClass` A class can identify itself with the `\ProvidesClass{`⟨*name*⟩`}[`⟨*version*⟩`]` command. The ⟨*version*⟩ should begin with a date in the format `YYYY/MM/DD`.

3.2 Package name and version

`\ProvidesPackage` A package can identify itself with the `\ProvidesPackage{`⟨*name*⟩`}[`⟨*version*⟩`]` command. The ⟨*version*⟩ should begin with a date in the format `YYYY/MM/DD`.

3.3 Requiring other packages

`\RequirePackage` Packages or classes can load other packages using
`\RequirePackage[`⟨*options*⟩`]{`⟨*name*⟩`}[`⟨*version*⟩`]`.
If the package has already been loaded, then nothing happens unless the requested options are not a subset of the options with which it was loaded, in which case an error is called.

`\LoadClass` Similar to `\RequirePackage`, but for classes, may not be used in package files.

`\PassOptionsToPackage` Packages can pass options to other packages using:
`\PassOptionsToPackage{`⟨*options*⟩`}{`⟨*package*⟩`}`.

`\PassOptionsToClass` This adds the ⟨*options*⟩ to the options list of any future `\RequirePackage` or `\usepackage` command. For example:

```
\PassOptionsToPackage{foo,bar}{fred}
```

is the same as:

```
\RequirePackage[foo,bar,baz]{fred}
```

`\LoadClassWithOptions` `\LoadClassWithOptions{`⟨*name*⟩`}[`⟨*version*⟩`]`:
This is similar to `\LoadClass`, but it always calls class ⟨*name*⟩ with exactly the same option list that is being used by the current class, rather than an option explicitly supplied or passed on by `\PassOptionsToClass`. `\RequirePackageWithOptions` is the analogous command for packages.

`\RequirePackageWithOptions`

This is mainly intended to allow one class to simply build on another, for example:

```
\LoadClassWithOptions{article}
```

This should be contrasted with the slightly different construction

```
\DeclareOption*{\PassOptionsToClass{\CurrentOption}{article}}
\ProcessOptions
\LoadClass{article}
```

As used here, the effects are more or less the same, but the version using `\LoadClassWithOptions` is slightly quicker (and less to type). If, however, the class declares options of its own then the two constructions are different; compare, for example:

```
\DeclareOption{landscape}{...}
\ProcessOptions
\LoadClassWithOptions{article}
```

with:

```
\DeclareOption{landscape}{...}
\DeclareOption*{\PassOptionsToClass{\CurrentOption}{article}}
\ProcessOptions
\LoadClass{article}
```

In the first case, the article class will be called with option `landscape` precisely when the current class is called with this option; but in the second example it will not as in that case article is only passed options by the default option handler, which is not used for `landscape` as that option is explicitly declared.

`\IfPackageLoadedTF` `\IfClassLoadedTF` `\@ifpackageloaded` `\@ifclassloaded`

To find out if a package has already been loaded, use

```
\IfPackageLoadedTF{⟨package⟩}{⟨true⟩}{⟨false⟩}
```

or the old name `\@ifpackageloaded`.

`\IfPackageAtLeastTF` `\IfClassAtLeastTF` `\IfFileAtLeastTF` `\@ifpackagelater` `\@ifclasslater`

To find out if a package has already been loaded with a version equal to or more recent than ⟨*date*⟩, use

```
\IfPackageAtLeastTF{⟨package⟩}{⟨date⟩}{⟨true⟩}{⟨false⟩}
```

or the old name `\@ifpackagelater`.

`\IfFormatAtLeastTF`

To test the format date use

```
\IfFormatAtLeastTF{⟨date⟩}{⟨true⟩}{⟨false⟩}
```

`\IfPackageLoadedWithOptionsTF` `\IfClassLoadedWithOptionsTF` `\@ifpackagewith` `\@ifclasswith`

To find out if a package has already been loaded with at least the options ⟨*options*⟩, use

```
\IfPackageLoadedWithOptionsTF{⟨package⟩}{⟨options⟩}{⟨true⟩}{⟨false⟩}
```

or the old name `\@ifpackagewith`.

There exists one package that can't be tested with the above commands: the `fontenc` package pretends that it was never loaded to allow for repeated reloading with different options (see `ltoutenc.dtx` for details).

`\NeedsDocumentMetadata`

To require that a document starts with `\DocumentMetadata` use

```
\NeedsDocumentMetadata
```

3.4 Declaring new options

Options for classes and packages are built using the same macros.

\DeclareOption To define a builtin option, use `\DeclareOption{`⟨*name*⟩`}{`⟨*code*⟩`}`.

\DeclareOption* To define the default action to perform for local options which have not been declared, use `\DeclareOption*{`⟨*code*⟩`}`.

Note: there should be no use of
`\RequirePackage`, `\DeclareOption`, `\DeclareOption*` or `\ProcessOptions`
inside `\DeclareOption` or `\DeclareOption*`.

Possible uses for `\DeclareOption*` include:

```
\DeclareOption*{}
```

Do nothing. Silently accept unknown options. (This suppresses the usual warnings.)

```
\DeclareOption*{\@unkownoptionerror}
```

Complain about unknown local options. (The initial setting for package files.)

```
\DeclareOption*{\PassOptionsToPackage{\CurrentOption}{⟨pkg-name⟩}}
```

Handle the current option by passing it on to the package ⟨*pkg-name*⟩, which will presumably be loaded via `\RequirePackage` later in the file. This is useful for building 'extension' packages, that perhaps handle a couple of new options, but then pass everything else on to an existing package.

```
\DeclareOption*{\InputIfFileExists{xx-\CurrentOption.yyy}%
                {}%
                {\OptionNotUsed}}
```

Handle the option foo by loading the file `xx-foo.yyy` if it exists, otherwise do nothing, but declare that the option was not used. Actually the `\OptionNotUsed` declaration is only needed if this is being used in class files, but does no harm in package files.

3.5 Safe Input Macros

\InputIfFileExists `\InputIfFileExists{`⟨*file*⟩`}{`⟨*then*⟩`}{`⟨*else*⟩`}`

Inputs ⟨*file*⟩ if it exists. Immediately before the input, ⟨*then*⟩ is executed. Otherwise ⟨*else*⟩ is executed.

\IfFileExists As above, but does not input the file.

One thing you might like to put in the ⟨*else*⟩ clause is

\@missingfileerror This starts an interactive request for a filename, supplying default extensions. Just hitting return causes the whole input to be skipped and entering `x` quits the current run,

\input This has been redefined from the LaTeX2.09 definition, in terms of the new commands `\InputIfFileExists` and `\@missingfileerror`.

\listfiles Giving this declaration in the preamble causes a list of all files input via the 'safe input' commands to be listed at the end. Any strings specified in the optional argument to `\ProvidesPackage` are listed alongside the file name. So files in standard (and other non-standard) distributions can put informative strings in this argument.

4 Implementation

```
1 ⟨*2ekernel⟩
```

\if@compatibility The flag for compatibility mode.

```
2 \newif\if@compatibility
```

(*End of definition for* `\if@compatibility`.)

\@documentclasshook This legacy hook is called after the first \documentclass command. It is *not* integrated with the new 2020 hook management system! By default this checks to see if \@normalsize is undefined, and if so, sets it to \normalsize.

```
\def\@documentclasshook{%
   \ifx\@normalsize\@undefined
      \let\@normalsize\normalsize
   \fi
}
```

(*End of definition for* \@documentclasshook.)

\@declaredoptions This list is automatically built by \DeclareOption. It is the list of options (separated by commas) declared in the class or package file and it defines the order in which the corresponding \ds@⟨*option*⟩ commands are executed. All local ⟨*option*⟩s which are not declared will be processed in the order defined by the optional argument of \documentclass or \usepackage.

```
\let\@declaredoptions\@empty
```

(*End of definition for* \@declaredoptions.)

\@classoptionslist List of options of the main class.

```
\let\@classoptionslist\relax
%\@onlypreamble\@classoptionslist
```

(*End of definition for* \@classoptionslist.)

\@raw@classoptionslist List of options of the main class (unprocessed).

```
\let\@raw@classoptionslist\relax
```

(*End of definition for* \@raw@classoptionslist.)

\@unusedoptionlist List of options of the main class that haven't been declared or loaded as class option files.

```
\let\@unusedoptionlist\@empty
%\@onlypreamble\@unusedoptionlist
```

(*End of definition for* \@unusedoptionlist.)

\CurrentOption Name of current package or option.

```
\let\CurrentOption\@empty
```

(*End of definition for* \CurrentOption.)

\@currpath Path to the current file if explicitly given.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\@currpath}%
⟨latexrelease⟩   {Add \@currpath}%
\let\@currpath\@empty
⟨latexrelease⟩\EndIncludeInRelease
%
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\@currpath}%
⟨latexrelease⟩   {Add \@currpath}%
⟨latexrelease⟩\let\@currpath\@undefined
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨/2ekernel | latexrelease⟩
⟨*2ekernel⟩
```

(*End of definition for* `\@currpath`.)

`\@currname` Name of current package or option.

```
\let\@currname\@empty
```

(*End of definition for* `\@currname`.)

`\@currext` The current file extension.

```
\global\let\@currext=\@empty
```

(*End of definition for* `\@currext`.)

`\@clsextension` The two possible values of `\@currext`.
`\@pkgextension`

```
\def\@clsextension{cls}
\def\@pkgextension{sty}
```

(*End of definition for* `\@clsextension` *and* `\@pkgextension`.)

`\@pushfilename` Commands to push and pop the file name and extension.
`\@popfilename` `#1` current name.
`\@currnamestack` `#2` current extension.
`#3` current catcode of `@`.
`#4` Rest of the stack.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2026/11/01}%
⟨latexrelease⟩                    {\@pushfilename}{Manage target date requests}%
\def\@pushfilename{%
```

The push and pop macros are injected in `\@pushfilename` and `\@popfilename` so that they correctly keep track of the hook labels.

This needs cleanup with the expl3 interfaces also playing here, e.g., `\@expl@push@filename@@` needs cleanup and (and should probably not have this name either).

```
  \@expl@push@filename@@
  \xdef\@currnamestack{%
    {\@currname}%
    {\@currext}%
    {\the\catcode`\@}%
    {\pkgcls@outerdate}%
    \@currnamestack}%
```

Temporarily add a stack for `\@currpath` here. This should be integrated in the main file stack eventually, but other packages rely on `\@currnamestack` having three elements per file, so that isn't a trivial change. The prefix `\@kernel@...` hopefully discourages people from using it.

```
  \xdef\@kernel@currpathstack{%
    {\@currpath}%
    \@kernel@currpathstack}%
  \@expl@push@filename@aux@@}
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\@pushfilename}%
⟨latexrelease⟩  {Add \@expl@push@filename@@ and \@expl@push@filename@aux@@}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@pushfilename{%
⟨latexrelease⟩  \@expl@push@filename@@
⟨latexrelease⟩  \xdef\@currnamestack{%
⟨latexrelease⟩    {\@currname}%
⟨latexrelease⟩    {\@currext}%
⟨latexrelease⟩    {\the\catcode`\@}%
⟨latexrelease⟩    \@currnamestack}%
⟨latexrelease⟩  \xdef\@kernel@currpathstack{%
⟨latexrelease⟩    {\@currpath}%
⟨latexrelease⟩    \@kernel@currpathstack}%
⟨latexrelease⟩  \@expl@push@filename@aux@@}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

The following version of `\@pushfilename` didn't formally exist in this file, but in the 2020/02/02 release, expl3 was preloaded and it patched `\@pushfilename` (and `\@popfilename`) by adding some hooks in there. But rolling back to 2020/02/02, expl3 doesn't patch these macros again, so rolling back has to take those hooks into account. Same goes for `\@popfilename`.

```
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/02/02}{\@pushfilename}%
⟨latexrelease⟩  {Add \@expl@push@filename@@}%
⟨latexrelease⟩\def\@pushfilename{%
⟨latexrelease⟩  \@expl@push@filename@@
⟨latexrelease⟩  \xdef\@currnamestack{%
⟨latexrelease⟩    {\@currname}%
⟨latexrelease⟩    {\@currext}%
⟨latexrelease⟩    {\the\catcode`\@}%
⟨latexrelease⟩    \@currnamestack}%
⟨latexrelease⟩    \@expl@push@filename@aux@@}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩
```

When we roll back from a release that has expl3 preloaded, the definitions of `\@pushfilename` and `\@popfilename` can't be completely rolled back otherwise expl3-based packages won't have the automatic `\ExplSyntaxOff` at the end. Here and below for `\@popfilename`, we don't roll back all the way through if coming from $\LaTeX > 2020-02-02$.

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\@pushfilename}%
⟨latexrelease⟩  {Add \@expl@push@filename@@ and \@expl@push@filename@aux@@}%
⟨latexrelease⟩\ifnum\sourceLaTeXdate<20200202\relax
⟨latexrelease⟩  \GenericInfo{}{Defining 00-00-00\string\@pushfilename.}
⟨latexrelease⟩\def\@pushfilename{%
⟨latexrelease⟩  \xdef\@currnamestack{%
⟨latexrelease⟩    {\@currname}%
⟨latexrelease⟩    {\@currext}%
⟨latexrelease⟩    {\the\catcode`\@}%
⟨latexrelease⟩    \@currnamestack}}
⟨latexrelease⟩\else
⟨latexrelease⟩  \GenericInfo{}{Defining 2020-02-02\string\@pushfilename.}
```

```
⟨latexrelease⟩\def\@pushfilename{%
⟨latexrelease⟩   \@expl@push@filename@@
⟨latexrelease⟩   \xdef\@currnamestack{%
⟨latexrelease⟩     {\@currname}%
⟨latexrelease⟩     {\@currext}%
⟨latexrelease⟩     {\the\catcode`\@}%
⟨latexrelease⟩     \@currnamestack}%
⟨latexrelease⟩     \@expl@push@filename@aux@@}
⟨latexrelease⟩\fi
⟨latexrelease⟩\EndIncludeInRelease
\@onlypreamble\@pushfilename

⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\@popfilename}%
⟨latexrelease⟩   {Add \@expl@pop@filename@@}%
\def\@popfilename{\@expl@@@hook@curr@name@pop@@
  \expandafter\@p@pfilename\@currnamestack\@nil
```

Same for popping:

```
  \expandafter\@p@pfilepath\@kernel@currpathstack\@nil
  \@expl@pop@filename@@}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/02/02}{\@popfilename}%
⟨latexrelease⟩   {Add \@expl@push@filename@@}%
⟨latexrelease⟩\def\@popfilename{\expandafter\@p@pfilename\@currnamestack\@nil
⟨latexrelease⟩   \@expl@pop@filename@@}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩

⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\@popfilename}%
⟨latexrelease⟩   {Add \@expl@push@filename@@ and \@expl@push@filename@aux@@}%
⟨latexrelease⟩\ifnum\sourceLaTeXdate<20200202\relax
⟨latexrelease⟩   \GenericInfo{}{Defining 00-00-00\string\@popfilename.}
⟨latexrelease⟩\def\@popfilename{\expandafter\@p@pfilename\@currnamestack\@nil}
⟨latexrelease⟩\else
⟨latexrelease⟩   \GenericInfo{}{Defining 2020-02-02\string\@popfilename.}
⟨latexrelease⟩\def\@popfilename{\expandafter\@p@pfilename\@currnamestack\@nil
⟨latexrelease⟩   \@expl@pop@filename@@}
⟨latexrelease⟩\fi
⟨latexrelease⟩\EndIncludeInRelease
\@onlypreamble\@popfilename

⟨/2ekernel | latexrelease⟩
⟨*2ekernel⟩

⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2026/11/01}%
⟨latexrelease⟩                  {\@p@pfilename}{Manage target date requests}%
\def\@p@pfilename#1#2#3#4#5\@nil{%
  \gdef\@currname{#1}%
  \gdef\@currext{#2}%
  \catcode`\@#3\relax
```

The rollback code uses local variables so we do this here too.

```
  \def\pkgcls@innerdate{#4}%
  \gdef\@currnamestack{#5}}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@p@pfilename}{Manage target date requests}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@p@pfilename#1#2#3#4\@nil{%
⟨latexrelease⟩  \gdef\@currname{#1}%
⟨latexrelease⟩  \gdef\@currext{#2}%
⟨latexrelease⟩  \catcode`\@#3\relax
⟨latexrelease⟩  \gdef\@currnamestack{#4}}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩

\@onlypreamble\@p@pfilename

\gdef\@currnamestack{}
\@onlypreamble\@currnamestack
```

(*End of definition for* `\@pushfilename`*,* `\@popfilename`*, and* `\@currnamestack`*.*)

`\@kernel@currpathstack` Path to the current file if explicitly given. The auxiliary is needed here to insert a `\@empty` to prevent the loss of braces.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\@kernel@currpathstack}%
⟨latexrelease⟩  {Add \@kernel@currpathstack}%
```

If rolling backwards to this release, `\@kernel@currpathstack` will be defined, so the `\gdef` line should not be executed, thus the `\@gobblethree` will take it out, so the satck isn't touched.

```
⟨latexrelease⟩\@ifundefined{@kernel@currpathstack}{}{\@gobblethree}
\gdef\@kernel@currpathstack{}%
```

If rolling forward to this release, then the `\gdef` line above will define the path stack to be empty (which it can't be, inside a file), so the code below will traverse the `\@currnamestack`, and add as many empty items to `\@kernel@currpathstack` as there are items in `\@currnamestack`, so both are back in sync. Most of the time latexrelease is loaded on top-level, so only one item is needed, but platexrelease loads it internally, so the more complicated loop is needed.

```
⟨latexrelease⟩\ifx\@kernel@currpathstack\@empty
⟨latexrelease⟩  \def\reserved@a#1#2#3{%
⟨latexrelease⟩    \ifx\relax#3\else
⟨latexrelease⟩      \g@addto@macro\@kernel@currpathstack{{}}%
⟨latexrelease⟩      \expandafter\reserved@a
⟨latexrelease⟩    \fi}%
⟨latexrelease⟩  \expandafter\reserved@a\@currnamestack{}{}{\relax}%
⟨latexrelease⟩\fi
\def\@p@pfilepath#1{%
  \gdef\@currpath{#1}\@p@pfilepath@aux\@empty}
\def\@p@pfilepath@aux#1\@nil{%
  \xdef\@kernel@currpathstack{#1}}
```

```
⟨latexrelease⟩\EndIncludeInRelease
%
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\@kernel@currpathstack}%
⟨latexrelease⟩  {Add \@kernel@currpathstack}%
⟨latexrelease⟩\let\@kernel@currpathstack\@undefined
⟨latexrelease⟩\let\@p@pfilepath\@undefined
⟨latexrelease⟩\let\@p@pfilepath@aux\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨/2ekernel | latexrelease⟩
⟨*2ekernel⟩
```

(*End of definition for* `\@kernel@currpathstack`.)

`\@ptionlist` Returns the option list of the file.

```
\def\@ptionlist#1{%
  \@ifundefined{opt@#1}\@empty{\csname opt@#1\endcsname}}
%\@onlypreamble\@ptionlist
```

(*End of definition for* `\@ptionlist`.)

`\@ifpackageloaded` `\@ifclassloaded` `\@ifpackageloaded{`⟨*name*⟩`}` Checks to see whether a file has been loaded.

```
\def\@ifpackageloaded{\@ifl@aded\@pkgextension}
\def\@ifclassloaded{\@ifl@aded\@clsextension}

\def\@ifl@aded#1#2{%
  \expandafter\ifx\csname ver@#2.#1\endcsname\relax
    \expandafter\@secondoftwo
  \else
    \expandafter\@firstoftwo
  \fi}
```

(*End of definition for* `\@ifpackageloaded` *and* `\@ifclassloaded`.)

`\@ifpackagelater` `\@ifclasslater` `\@ifpackagelater{`⟨*name*⟩`}{YYYY/MM/DD}{`⟨*true code*⟩`}{`⟨*false code*⟩`}` Checks that the package loaded is more recent or equal to the given date. A better name for it would therefore been `\@ifpackagelaterorequal` but it is in use for more than 30 years, so ...

```
\def\@ifpackagelater{\@ifl@ter\@pkgextension}
\def\@ifclasslater{\@ifl@ter\@clsextension}
```

(*End of definition for* `\@ifpackagelater` *and* `\@ifclasslater`.)

`\IfPackageAtLeastTF` `\IfClassAtLeastTF` `\IfFileAtLeastTF` `\IfFormatAtLeastTF` `\IfFormatAtLeastTF{YYYY/MM/DD}{`⟨*true code*⟩`}{`⟨*false code*⟩`}` Test if the format is later or equal to the given date.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\IfFormatAtLeastTF}{Test format date}%
\def\IfFormatAtLeastTF{\@ifl@t@r\fmtversion}
\let\IfPackageAtLeastTF\@ifpackagelater
\let\IfClassAtLeastTF\@ifclasslater
\def\IfFileAtLeastTF#1{\expandafter\@ifl@t@r\csname ver@#1\endcsname}
```

For rollback pretend it was available since the beginning of dawn.

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\IfFormatAtLeastTF}{Test format date}%
⟨latexrelease⟩\def\IfFormatAtLeastTF{\@ifl@t@r\fmtversion}
⟨latexrelease⟩\let\IfPackageAtLeastTF\@ifpackagelater
⟨latexrelease⟩\let\IfClassAtLeastTF\@ifclasslater
⟨latexrelease⟩\def\IfFileAtLeastTF#1{\expandafter\@ifl@t@r\csname ver@#1\endcsname}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\IfPackageAtLeastTF` *and others.*)

`\@ifl@ter`

```
\def\@ifl@ter#1#2{%
  \expandafter\@ifl@t@r
    \csname ver@#2.#1\endcsname}
⟨/2ekernel⟩
```

This internal macro is also used in `\NeedsTeXFormat`.

```
⟨latexrelease⟩\IncludeInRelease{2018/04/01}%
⟨latexrelease⟩                  {\@ifl@t@r}{Guard against bad input}%
⟨*2ekernel | latexrelease⟩
\def\@ifl@t@r#1#2{%
  \ifnum\expandafter\@parse@version@#1//00\@nil<%
        \expandafter\@parse@version@#2//00\@nil
    \expandafter\@secondoftwo
  \else
    \expandafter\@firstoftwo
  \fi}
\def\@parse@version@#1{\@parse@version0#1}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@ifl@t@r}{Guard against bad input}%
⟨latexrelease⟩\def\@ifl@t@r#1#2{%
⟨latexrelease⟩  \ifnum\expandafter\@parse@version#1//00\@nil<%
⟨latexrelease⟩        \expandafter\@parse@version#2//00\@nil
⟨latexrelease⟩    \expandafter\@secondoftwo
⟨latexrelease⟩  \else
⟨latexrelease⟩    \expandafter\@firstoftwo
⟨latexrelease⟩  \fi}
⟨latexrelease⟩\let\@parse@version@\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@ifl@ter`.)

```
⟨/2ekernel⟩
⟨*2ekernel | latexreleasefirst⟩
\def\@parse@version#1/#2/#3#4#5\@nil{%
\@parse@version@dash#1-#2-#3#4\@nil
}
```

The \if test here ensures that an argument with no / or - produces 0 (actually 00).

```
\def\@parse@version@dash#1-#2-#3#4#5\@nil{%
  \if\relax#2\relax\else#1\fi#2#3#4 }
⟨/2ekernel | latexreleasefirst⟩
⟨*2ekernel⟩
```

\@ifpackagewith
\@ifclasswith

\@ifpackagewith{⟨*name*⟩}{⟨*option-list*⟩} Checks that ⟨*option-list*⟩ is a subset of the options **with** which ⟨*name*⟩ was loaded.

```
\def\@ifpackagewith{\@if@ptions\@pkgextension}
\def\@ifclasswith{\@if@ptions\@clsextension}

\def\@if@ptions#1#2{%
  \@expandtwoargs\@if@pti@ns{\@ptionlist{#2.#1}}}
```

Probably shouldn't use \CurrentOption here... (changed to \reserved@b.)

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2017/01/01}%
⟨latexrelease⟩                  {\@if@pti@ns}{Spaces in option clash check}%
⟨*2ekernel | latexrelease⟩
\def\@if@pti@ns#1#2{%
 \let\reserved@a\@firstoftwo

 \edef\reserved@b{\zap@space#2 \@empty}%
 \@for\reserved@b:=\reserved@b\do{%
   \ifx\reserved@b\@empty
   \else
     \expandafter\in@\expandafter{\expandafter,\reserved@b,}{,#1,}%
     \ifin@
     \else
       \let\reserved@a\@secondoftwo
     \fi
   \fi
 }%
 \reserved@a}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@if@pti@ns}{Spaces in option clash check}%
⟨latexrelease⟩\def\@if@pti@ns#1#2{%
⟨latexrelease⟩ \let\reserved@a\@firstoftwo
⟨latexrelease⟩ \@for\reserved@b:=#2\do{%
⟨latexrelease⟩  \ifx\reserved@b\@empty
⟨latexrelease⟩   \else
⟨latexrelease⟩   \expandafter\in@\expandafter
⟨latexrelease⟩                    {\expandafter,\reserved@b,}{,#1,}%
⟨latexrelease⟩    \ifin@
⟨latexrelease⟩    \else
⟨latexrelease⟩     \let\reserved@a\@secondoftwo
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \fi
⟨latexrelease⟩ }%
⟨latexrelease⟩ \reserved@a}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@ifpackagewith` *and* `\@ifclasswith`.)

`\IfPackageLoadedTF` `\IfPackageLoadedWithOptionsTF` `\IfClassLoadedTF` `\IfClassLoadedWithOptionsTF`

More public names for the commands already available for a long time.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2024/06/01}%
⟨latexrelease⟩                  {\IfPackageLoadedTF}{Test package loading}%
\let \IfPackageLoadedTF            \@ifpackageloaded
\let \IfClassLoadedTF              \@ifclassloaded
\let \IfPackageLoadedWithOptionsTF \@ifpackagewith
\let \IfClassLoadedWithOptionsTF   \@ifclasswith
```

For rollback/rollforward pretend everything was available since the beginning of dawn.

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\IfPackageLoadedTF}{Test package loading}%
⟨latexrelease⟩
⟨latexrelease⟩\let \IfPackageLoadedTF            \@ifpackageloaded
⟨latexrelease⟩\let \IfClassLoadedTF              \@ifclassloaded
⟨latexrelease⟩\let \IfPackageLoadedWithOptionsTF \@ifpackagewith
⟨latexrelease⟩\let \IfClassLoadedWithOptionsTF   \@ifclasswith
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\IfPackageLoadedTF` *and others.*)

`\IfPackageLoadedT` `\IfPackageLoadedF` `\IfPackageAtLeastT` `\IfPackageAtLeastF` `\IfClassAtLeastT` `\IfClassAtLeastF` `\IfFileAtLeastT` `\IfFileAtLeastF` `\IfFormatAtLeastT` `\IfFormatAtLeastF` `\IfPackageLoadedWithOptionsT` `\IfPackageLoadedWithOptionsF` `\IfClassLoadedT` `\IfClassLoadedF` `\IfClassLoadedWithOptionsF` `\IfClassLoadedWithOptionsTF`

A few more conditionals for convenience

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2024/06/01}%
⟨latexrelease⟩                  {\IfPackageLoadedT}{More conditionals}%
\def\IfPackageLoadedT   #1{\IfPackageLoadedTF{#1}\@firstofone\@gobble}
\def\IfPackageLoadedF   #1{\IfPackageLoadedTF{#1}{}}
\def\IfClassLoadedT     #1{\IfClassLoadedTF{#1}\@firstofone\@gobble}
\def\IfClassLoadedF     #1{\IfClassLoadedTF{#1}{}}
\def\IfPackageAtLeastT#1#2{\IfPackageAtLeastTF{#1}{#2}\@firstofone\@gobble}
\def\IfPackageAtLeastF#1#2{\IfPackageAtLeastTF{#1}{#2}{}}
\def\IfClassAtLeastT    #1#2{\IfClassAtLeastTF{#1}{#2}\@firstofone\@gobble}
\def\IfClassAtLeastF    #1#2{\IfClassAtLeastTF{#1}{#2}{}}
\def\IfFileAtLeastT   #1#2{\IfFileAtLeastTF{#1}{#2}\@firstofone\@gobble}
\def\IfFileAtLeastF   #1#2{\IfFileAtLeastTF{#1}{#2}{}}
\def\IfFormatAtLeastT   #1{\IfFormatAtLeastTF{#1}\@firstofone\@gobble}
\def\IfFormatAtLeastF   #1{\IfFormatAtLeastTF{#1}{}}
\def\IfPackageLoadedWithOptionsT #1#2{\IfPackageLoadedWithOptionsTF{#1}{#2}\@firstofone\@gobb
\def\IfPackageLoadedWithOptionsF #1#2{\IfPackageLoadedWithOptionsTF{#1}{#2}{}}
\def\IfClassLoadedWithOptionsT #1#2{\IfClassLoadedWithOptionsTF{#1}{#2}\@firstofone\@gobble}
\def\IfClassLoadedWithOptionsF #1#2{\IfClassLoadedWithOptionsTF{#1}{#2}{}}
```

These three commands haven't been there at all in the past.

```
\def\IfFileLoadedTF#1{%
  \expandafter\ifx\csname ver@#1\endcsname\relax
    \expandafter\@secondoftwo
```

```
  \else
    \expandafter\@firstoftwo
  \fi}
\def\IfFileLoadedT  #1{\IfFileLoadedTF{#1}\@firstofone\@gobble}
\def\IfFileLoadedF  #1{\IfFileLoadedTF{#1}{}}
```

For rollback/rollforward pretend everything was available since the beginning of dawn.

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\IfPackageLoadedT}{More conditionals}%
⟨latexrelease⟩
⟨latexrelease⟩\def\IfPackageLoadedT #1#2{\IfPackageLoadedTF{#1}{#2}{}}
⟨latexrelease⟩\def\IfPackageLoadedF   #1{\IfPackageLoadedTF{#1}{}}
⟨latexrelease⟩\def\IfClassLoadedT   #1#2{\IfClassLoadedTF{#1}{#2}{}}
⟨latexrelease⟩\def\IfClassLoadedF     #1{\IfClassLoadedTF{#1}{}}
⟨latexrelease⟩\def\IfPackageAtLeastT#1#2#3{\IfPackageAtLeastTF{#1}{#2}{#3}{}}
⟨latexrelease⟩\def\IfPackageAtLeastF  #1#2{\IfPackageAtLeastTF{#1}{#2}{}}
⟨latexrelease⟩\def\IfClassAtLeastT  #1#2#3{\IfClassAtLeastTF{#1}{#2}{#3}{}}
⟨latexrelease⟩\def\IfClassAtLeastF    #1#2{\IfClassAtLeastTF{#1}{#2}{}}
⟨latexrelease⟩\def\IfFileAtLeastT   #1#2#3{\IfFileAtLeastTF{#1}{#2}{#3}{}}
⟨latexrelease⟩\def\IfFileAtLeastF     #1#2{\IfFileAtLeastTF{#1}{#2}{}}
⟨latexrelease⟩\def\IfFormatAtLeastT   #1#2{\IfFormatAtLeastTF{#1}{#2}{}}
⟨latexrelease⟩\def\IfFormatAtLeastF     #1{\IfFormatAtLeastTF{#1}{}}
⟨latexrelease⟩\def\IfPackageLoadedWithOptionsT #1#2#3{\IfPackageLoadedWithOptionsTF{#1}{#2}{#3}{
⟨latexrelease⟩\def\IfPackageLoadedWithOptionsF   #1#2{\IfPackageLoadedWithOptionsTF{#1}{#2}{}}
⟨latexrelease⟩\def\IfClassLoadedWithOptionsT #1#2#3{\IfClassLoadedWithOptionsTF{#1}{#2}{#3}{}}
⟨latexrelease⟩\def\IfClassLoadedWithOptionsF   #1#2{\IfClassLoadedWithOptionsTF{#1}{#2}{}}
⟨latexrelease⟩
⟨latexrelease⟩\def\IfFileLoadedTF#1{%
⟨latexrelease⟩  \expandafter\ifx\csname ver@#1\endcsname\relax
⟨latexrelease⟩    \expandafter\@secondoftwo
⟨latexrelease⟩  \else
⟨latexrelease⟩    \expandafter\@firstoftwo
⟨latexrelease⟩  \fi}
⟨latexrelease⟩\def\IfFileLoadedT#1#2{\IfFileLoadedTF{#1}{#2}{}}
⟨latexrelease⟩\def\IfFileLoadedF  #1{\IfFileLoadedTF{#1}{}}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\IfPackageLoadedT` *and others.*)

`\ProvidesPackage` Checks that the current filename is correct, and defines `\ver@filename`.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩  {\ProvidesPackage}{Check name with \strcmp}%
⟨*2ekernel | latexrelease⟩
\def\ProvidesPackage#1{%
  \xdef\@gtempa{#1}%
```

Here `\@currpath` is explicitly added to the file name to report when a package or class is loaded using an explicit path. Loading using a path in the argument is supported but not encouraged.

```
  \@expandtwoargs\@expl@str@if@eq@@nnTF
```

```
      {\@gtempa}{\@currpath\@currname}{}{%
    \@latex@warning@no@line{You have requested
      \@cls@pkg\space`\@currpath\@currname',\MessageBreak
       but the \@cls@pkg\space provides `#1'}%
    }%
  \@ifnextchar[\@pr@videpackage{\@pr@videpackage[]}%]
\@onlypreamble\ProvidesPackage
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
%
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩  {\ProvidesPackage}{Undo: check name with \strcmp}%
⟨latexrelease⟩\def\ProvidesPackage#1{%
⟨latexrelease⟩  \xdef\@gtempa{#1}%
⟨latexrelease⟩  \ifx\@gtempa\@currname\else
⟨latexrelease⟩    \@latex@warning@no@line{You have requested
⟨latexrelease⟩      \@cls@pkg\space`\@currname',\MessageBreak
⟨latexrelease⟩       but the \@cls@pkg\space provides `#1'}%
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \@ifnextchar[\@pr@videpackage{\@pr@videpackage[]}%]
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\ProvidesPackage`.)

`\@pr@videpackage` This is the helper command for `\ProvidesPackage`. It tries to be cautious when handling the identification string in case it contains UTF-8 characters.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\@pr@videpackage}{Allow for package substitution}%
\def\@pr@videpackage[#1]{%
  \expandafter\protected@xdef                %     <-- protected...
     \csname ver@\@currname.\@currext\endcsname{#1}% Loaded package
  \expandafter\let
    \csname ver@\@currpkg@reqd\expandafter\endcsname % Requested package
    \csname ver@\@currname.\@currext\endcsname
  \ifx\@currext\@clsextension
    \typeout{Document Class: \@gtempa\space#1}%
  \else
    \protected@wlog{Package: \@gtempa\space#1}%   <--- protected
  \fi}
```

(*End of definition for* `\@pr@videpackage`.)

`\protected@wlog` This is like plain TEX's `\wlog` but gracefully handles protected commands.

```
\long\def\protected@wlog#1{\begingroup
  \set@display@protect
  \immediate \write \m@ne {#1}\endgroup }
```

(*End of definition for* `\protected@wlog`.)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/02/02}%
```

```
⟨latexrelease⟩                  {\@pr@videpackage}{Protection for package info}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@pr@videpackage[#1]{%
⟨latexrelease⟩  \expandafter\protected@xdef                %     <-- protected...
⟨latexrelease⟩     \csname ver@\@currname.\@currext\endcsname{#1}%
⟨latexrelease⟩\ifx\@currext\@clsextension
⟨latexrelease⟩    \typeout{Document Class: \@gtempa\space#1}%
⟨latexrelease⟩  \else
⟨latexrelease⟩    \protected@wlog{Package: \@gtempa\space#1}%   <--- protected
⟨latexrelease⟩  \fi}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@pr@videpackage}{Protection for package info}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@pr@videpackage[#1]{%
⟨latexrelease⟩  \expandafter\xdef\csname ver@\@currname.\@currext\endcsname{#1}%
⟨latexrelease⟩  \ifx\@currext\@clsextension
⟨latexrelease⟩    \typeout{Document Class: \@gtempa\space#1}%
⟨latexrelease⟩  \else
⟨latexrelease⟩    \wlog{Package: \@gtempa\space#1}%
⟨latexrelease⟩  \fi}
⟨latexrelease⟩\let\protected@wlog\@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩

\@onlypreamble\@pr@videpackage
```

`\ProvidesClass` Like `\ProvidesPackage`, but for classes. This needs a dummy latexrelease block to copy the definition of `\ProvidesPackage` as it changes across releases.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩  {\ProvidesClass}{Track \ProvidesPackage}%
⟨*2ekernel | latexrelease⟩
\let\ProvidesClass\ProvidesPackage
\@onlypreamble\ProvidesClass
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\ProvidesClass`.)

`\ProvidesFile` Like `\ProvidesPackage`, but for arbitrary files. Do not apply `\@onlypreamble` to these, as we may want to label files input during the document.

`\@providesfile`

```
\def\ProvidesFile#1{%
  \begingroup
    \catcode`\ 10 %
    \ifnum \endlinechar<256 %
      \ifnum \endlinechar>\m@ne
        \catcode\endlinechar 10 %
      \fi
    \fi
```

```
    \@makeother\/%
    \@makeother\&%

    \kernel@ifnextchar[{\@providesfile{#1}}{\@providesfile{#1}[]}}
```

During initex a special version of `\@providesfile` is used. The real definition is installed right at the end, in `ltfinal.dtx`.

```
def\@providesfile#1[#2]{%
   \wlog{File: #1 #2}%
   \expandafter\xdef\csname ver@#1\endcsname{#2}%
 \endgroup}
```

(*End of definition for* `\ProvidesFile` *and* `\@providesfile`.)

`\PassOptionsToPackage`
`\PassOptionsToClass`

If the package has been loaded, we check that it was first loaded with the options. Otherwise we add the option list to that of the package.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩                    {\@pass@ptions}{Raw option lists}%
⟨*2ekernel | latexrelease⟩
\def\@pass@ptions#1#2#3{%
  \@expl@@@filehook@set@curr@file@@nNN
    {\@expl@@@filehook@resolve@file@subst@@w #3.#1\@nil}%
      \reserved@a\reserved@b
  \@expl@@@filehook@clear@replacement@flag@@

  \expandafter\protected@xdef\csname opt@\reserved@a\endcsname{%
    \@ifundefined{opt@\reserved@a}\@empty
      {\csname opt@\reserved@a\endcsname,}%
    \zap@space#2 \@empty}%

  \expandafter\let
    \csname opt@#3.#1\expandafter\endcsname
    \csname opt@\reserved@a\endcsname
```

Extend raw option list

```
  \@ifundefined{@raw@opt@#3.#1}%
    {\expandafter\gdef\csname @raw@opt@#3.#1\expandafter\endcsname
              \expandafter{#2}}%
    {\expandafter\g@addto@macro\csname @raw@opt@#3.#1\expandafter\endcsname
              \expandafter{\expandafter,#2}}%
}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2020/10/01}{\@pass@ptions}
⟨latexrelease⟩  {Add file replacement in \@pass@ptions}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@pass@ptions#1#2#3{%
⟨latexrelease⟩  \@expl@@@filehook@set@curr@file@@nNN
⟨latexrelease⟩    {\@expl@@@filehook@resolve@file@subst@@w #3.#1\@nil}%
⟨latexrelease⟩      \reserved@a\reserved@b
⟨latexrelease⟩  \@expl@@@filehook@clear@replacement@flag@@
⟨latexrelease⟩  \expandafter\xdef\csname opt@\reserved@a\endcsname{%
⟨latexrelease⟩    \@ifundefined{opt@\reserved@a}\@empty
```

```
⟨latexrelease⟩       {\csname opt@\reserved@a\endcsname,}%
⟨latexrelease⟩     \zap@space#2 \@empty}%
⟨latexrelease⟩   \expandafter\let
⟨latexrelease⟩     \csname opt@#3.#1\expandafter\endcsname
⟨latexrelease⟩     \csname opt@\reserved@a\endcsname}
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\@pass@ptions}
⟨latexrelease⟩   {\@pass@ptions}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@pass@ptions#1#2#3{%
⟨latexrelease⟩   \expandafter\xdef\csname opt@#3.#1\endcsname{%
⟨latexrelease⟩     \@ifundefined{opt@#3.#1}\@empty
⟨latexrelease⟩       {\csname opt@#3.#1\endcsname,}%
⟨latexrelease⟩     \zap@space#2 \@empty}}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩

\@onlypreamble\@pass@ptions

\def\PassOptionsToPackage{\@pass@ptions\@pkgextension}
\def\PassOptionsToClass{\@pass@ptions\@clsextension}
\@onlypreamble\PassOptionsToPackage
\@onlypreamble\PassOptionsToClass
```

(*End of definition for* `\PassOptionsToPackage` *and* `\PassOptionsToClass`.)

`\DeclareOption`
`\DeclareOption*`

Adds an option as a `\ds@` command, or the default `\default@ds` command.

```
\def\DeclareOption{%
  \let\@fileswith@pti@ns\@badrequireerror
  \@ifstar\@defdefault@ds\@declareoption}
\long\def\@declareoption#1#2{%
   \xdef\@declaredoptions{\@declaredoptions,#1}%
   \toks@{#2}%
   \expandafter\edef\csname ds@#1\endcsname{\the\toks@}}
\long\def\@defdefault@ds#1{%
  \toks@{#1}%
  \edef\default@ds{\the\toks@}}
\@onlypreamble\DeclareOption
\@onlypreamble\@declareoption
\@onlypreamble\@defdefault@ds
```

(*End of definition for* `\DeclareOption` *and* `\DeclareOption*`.)

`\OptionNotUsed`
`\@remove@eq@value`

If we are in a class file, add `\CurrentOption` to the list of unused options. Otherwise, in a package file do nothing.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩                    {\OptionNotUsed}{filter unused option list}%
⟨*2ekernel | latexrelease⟩
\ExplSyntaxOn
\def\@remove@eq@value#1=#2\@nil{\tl_trim_spaces:n{#1}}
\ExplSyntaxOff

\def\OptionNotUsed{%
  \ifx\@currext\@clsextension
    \xdef\@unusedoptionlist{%
```

```
      \ifx\@unusedoptionlist\@empty\else\@unusedoptionlist,\fi
      \expandafter\@remove@eq@value\CurrentOption=\@nil}%
  \fi}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\OptionNotUsed}{filter unused option list}%
⟨latexrelease⟩\let\@remove@eq@value\@undefined
⟨latexrelease⟩\def\OptionNotUsed{%
⟨latexrelease⟩  \ifx\@currext\@clsextension
⟨latexrelease⟩    \xdef\@unusedoptionlist{%
⟨latexrelease⟩      \ifx\@unusedoptionlist\@empty\else\@unusedoptionlist,\fi
⟨latexrelease⟩      \CurrentOption}%
⟨latexrelease⟩  \fi}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
\@onlypreamble\OptionNotUsed
```

(*End of definition for* `\OptionNotUsed` *and* `\@remove@eq@value`.)

`\default@ds` The default option code. Set by `\@onefilewithoptions` to either `\OptionNotUsed` for classes, or `\@unknownoptionerror` for packages. This may be reset in either case with `\DeclareOption*`.

```
% \let\default@ds\OptionNotUsed
```

(*End of definition for* `\default@ds`.)

`\ProcessOptions` `\ProcessOptions*` `\ProcessOptions` calls `\ds@option` for each known package option, then calls `\default@ds` for each option on the local options list. Finally resets all the declared options to `\relax`. The empty option does nothing, this has to be reset on the off chance it's set to `\relax` if an empty element gets into the `\@declaredoptions` list.

The star form is similar but executes options given in the order specified in the document, not the order they are declared in the file. In the case of packages, global options are executed before local ones.

```
\def\ProcessOptions{%
  \let\ds@\@empty
  \protected@edef\@curroptions{\@ptionlist{\@currname.\@currext}}%
  \@ifstar\@xprocess@ptions\@process@ptions}
\@onlypreamble\ProcessOptions

\def\@process@ptions{%
  \@for\CurrentOption:=\@declaredoptions\do{%
    \ifx\CurrentOption\@empty\else
      \@expandtwoargs\in@{,\CurrentOption,}{%
         ,\ifx\@currext\@clsextension\else\@classoptionslist,\fi
         \@curroptions,}%
      \ifin@
        \@use@ption
        \expandafter\let\csname ds@\CurrentOption\endcsname\@empty
      \fi
    \fi}%
  \@process@pti@ns}
\@onlypreamble\@process@ptions
```

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩                  {\@xprocess@ptions}{safer @xprocess@ptions}%
⟨*2ekernel | latexrelease⟩
\def\@xprocess@ptions{%
  \ifx\@currext\@clsextension\else
   \ifx\@classoptionslist\relax\else
    \@for\CurrentOption:=\@classoptionslist\do{%
      \ifx\CurrentOption\@empty\else
        \@ifundefined{ds@\detokenize\expandafter{\CurrentOption}}{}{%
          \@use@ption
          \expandafter\let\csname ds@\CurrentOption\endcsname\@empty
        }%
      \fi}%
    \fi
  \fi
  \@process@pti@ns}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@xprocess@ptions}{safer @xprocess@ptions}%
⟨latexrelease⟩\let\@remove@eq@value\@undefined
⟨latexrelease⟩\def\@xprocess@ptions{%
⟨latexrelease⟩  \ifx\@currext\@clsextension\else
⟨latexrelease⟩    \@for\CurrentOption:=\@classoptionslist\do{%
⟨latexrelease⟩      \ifx\CurrentOption\@empty\else
⟨latexrelease⟩        \@expandtwoargs\in@{,\CurrentOption,}{,\@declaredoptions,}%
⟨latexrelease⟩        \ifin@
⟨latexrelease⟩          \@use@ption
⟨latexrelease⟩          \expandafter\let\csname ds@\CurrentOption\endcsname\@empty
⟨latexrelease⟩        \fi
⟨latexrelease⟩      \fi}%
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \@process@pti@ns}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
\@onlypreamble\@xprocess@ptions
```

The common part of `\ProcessOptions` and `\ProcessOptions*`.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\@process@pti@ns}{Unused options issue}%
\def\@process@pti@ns{%
  \@for\CurrentOption:=\@curroptions\do{%
    \@ifundefined{ds@\detokenize\expandafter{\CurrentOption}}%
      {\@use@ption
       \default@ds}%
```

There should not be any non-empty definition of `\CurrentOption` at this point, as all the declared options were executed earlier. This is for compatibility with 2.09 styles which use `\def\ds@...` directly, and so have options which do not appear in `\@declaredoptions`.

```
      \@use@ption}%
```

Clear all the definitions for option code. First set all the declared options to `\relax`, then reset the 'default' and 'empty' options. and the lst of declared options.

```
\@for\CurrentOption:=\@declaredoptions\do{%
  \expandafter\let\csname ds@\CurrentOption\endcsname\relax}%

\let\CurrentOption\@empty
\let\@fileswith@pti@ns\@@fileswith@pti@ns
\AtEndOfPackage{\expandafter\let
                    \csname unprocessedoptions-\@currname.\@currext\endcsname
                    \relax}}
\@onlypreamble\@process@pti@ns
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                 {\@process@pti@ns}{Unused options issue}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@process@pti@ns{%
⟨latexrelease⟩  \@for\CurrentOption:=\@curroptions\do{%
⟨latexrelease⟩    \@ifundefined{ds@\CurrentOption}%
⟨latexrelease⟩      {\@use@ption
⟨latexrelease⟩       \default@ds}%
⟨latexrelease⟩      \@use@ption}%
⟨latexrelease⟩  \@for\CurrentOption:=\@declaredoptions\do{%
⟨latexrelease⟩    \expandafter\let\csname ds@\CurrentOption\endcsname\relax}%
⟨latexrelease⟩  \let\CurrentOption\@empty
⟨latexrelease⟩  \let\@fileswith@pti@ns\@@fileswith@pti@ns
⟨latexrelease⟩  \AtEndOfPackage{\let\@unprocessedoptions\relax}}
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\ProcessOptions` *and* `\ProcessOptions*`.)

`\@options` `\@options` is a synonym for `\ProcessOptions*` for upward compatibility with LaTeX2.09 style files.

```
\def\@options{\ProcessOptions*}
\@onlypreamble\@options
```

(*End of definition for* `\@options`.)

`\@use@ption` Execute the code for the current option.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩                 {\@use@ption}{filter unused option list}%
⟨∗2ekernel | latexrelease⟩
\def\@use@ption{%
  \@expandtwoargs\@removeelement
     {\expandafter\@remove@eq@value\CurrentOption=\@nil}%
  \@unusedoptionlist\@unusedoptionlist
  \csname ds@\detokenize\expandafter{\CurrentOption}\endcsname}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                 {\@use@ption}{filter unused option list}%
⟨latexrelease⟩\def\@use@ption{%
```

```
⟨latexrelease⟩  \@expandtwoargs\@removeelement\CurrentOption
⟨latexrelease⟩  \@unusedoptionlist\@unusedoptionlist
⟨latexrelease⟩  \csname ds@\CurrentOption\endcsname}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
\@onlypreamble\@use@ption
```

(*End of definition for* `\@use@ption`.)

`\ExecuteOptions` `\ExecuteOptions{`⟨*option-list*⟩`}` executes the code declared for each option.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2017/01/01}%
⟨latexrelease⟩                  {\ExecuteOptions}{Spaces in \ExecuteOptions}%
⟨*2ekernel | latexrelease⟩
\def\ExecuteOptions#1{%
```

Use `\@fortmp` here as it is anyway cleared during `\@for` loop so does not change any existing names.

```
  \edef\@fortmp{\zap@space#1 \@empty}%
  \def\reserved@a##1\@nil{%
    \@for\CurrentOption:=\@fortmp\do
             {\csname ds@\CurrentOption\endcsname}%
    \edef\CurrentOption{##1}}%
  \expandafter\reserved@a\CurrentOption\@nil}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\ExecuteOptions}{Spaces in \ExecuteOptions}%
⟨latexrelease⟩\def\ExecuteOptions#1{%
⟨latexrelease⟩  \def\reserved@a##1\@nil{%
⟨latexrelease⟩    \@for\CurrentOption:=#1\do
⟨latexrelease⟩             {\csname ds@\CurrentOption\endcsname}%
⟨latexrelease⟩    \edef\CurrentOption{##1}}%
⟨latexrelease⟩  \expandafter\reserved@a\CurrentOption\@nil}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
\@onlypreamble\ExecuteOptions
```

(*End of definition for* `\ExecuteOptions`.)

The top-level commands, which just set some parameters then call the internal command, `\@fileswithoptions`.

`\documentclass` The main new-style class declaration.

```
\def\documentclass{%
  \let\documentclass\@twoclasseserror
  \if@compatibility\else\let\usepackage\RequirePackage\fi
  \@fileswithoptions\@clsextension}
\@onlypreamble\documentclass
```

(*End of definition for* `\documentclass`.)

`\documentstyle` 2.09 style class 'style' declaration.

```
\def\documentstyle{%
  \makeatletter\input{latex209.def}\makeatother
  \documentclass}
\@onlypreamble\documentstyle
```

(*End of definition for* `\documentstyle`.)

`\RequirePackage` Load package if not already loaded.

```
\def\RequirePackage{%
  \@fileswithoptions\@pkgextension}
\@onlypreamble\RequirePackage
```

(*End of definition for* `\RequirePackage`.)

`\LoadClass` Load class.

```
\def\LoadClass{%
  \ifx\@currext\@pkgextension
     \@latex@error
      {\noexpand\LoadClass in package file}%
      {You may only use \noexpand\LoadClass in a class file.}%
  \fi
  \@fileswithoptions\@clsextension}
\@onlypreamble\LoadClass
```

(*End of definition for* `\LoadClass`.)

`\@loadwithoptions` Pass the current option list on to a class or package. `#1` is `\@`*cls-or-pkg*`extension`, `#2` is `\RequirePackage` or `\LoadClass`, `#3` is the class or package to be loaded.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩                   {\@loadwithoptions}{Raw option lists load with options}%
⟨*2ekernel | latexrelease⟩
\def\@loadwithoptions#1#2#3{%
  \expandafter\let\csname opt@#3.#1\expandafter\endcsname
       \csname opt@\@currname.\@currext\endcsname
  \expandafter\let\csname @raw@opt@#3.#1\expandafter\endcsname
       \csname @raw@opt@\@currname.\@currext\endcsname
   #2{#3}}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}
⟨latexrelease⟩                   {\@loadwithoptions}{Raw option lists load with options}%
⟨latexrelease⟩\def\@loadwithoptions#1#2#3{%
⟨latexrelease⟩  \expandafter\let\csname opt@#3.#1\expandafter\endcsname
⟨latexrelease⟩       \csname opt@\@currname.\@currext\endcsname
⟨latexrelease⟩   #2{#3}}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩

\@onlypreamble\@loadwithoptions
```

(*End of definition for* `\@loadwithoptions`.)

`\LoadClassWithOptions` Load class '`#1`' with the current option list.

```
\def\LoadClassWithOptions{%
  \@loadwithoptions\@clsextension\LoadClass}
\@onlypreamble\LoadClassWithOptions
```

(*End of definition for* `\LoadClassWithOptions`.)

\RequirePackageWithOptions Load package '#1' with the current option list.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\RequirePackageWithOptions}{Unused options issue}%
\def\RequirePackageWithOptions{%
```

The resetting of the unprocessed options is now done on a par package basis.

```
  \AtEndOfPackage{\expandafter\let
                      \csname unprocessedoptions-\@currname.\@currext\endcsname
                      \relax}%
  \@loadwithoptions\@pkgextension\RequirePackage}
\@onlypreamble\RequirePackageWithOptions
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\RequirePackageWithOptions}{Unused options issue}%
⟨latexrelease⟩
⟨latexrelease⟩\def\RequirePackageWithOptions{%
⟨latexrelease⟩  \AtEndOfPackage{\let\@unprocessedoptions\relax}%
⟨latexrelease⟩  \@loadwithoptions\@pkgextension\RequirePackage}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* \RequirePackageWithOptions.)

\usepackage To begin with, \usepackage produces an error. This is reset by \documentclass.

```
\def\usepackage#1#{%
  \@latex@error
    {\noexpand \usepackage before \string\documentclass}%
    {\noexpand \usepackage may only appear in the document
      preamble, i.e.,\MessageBreak
      between \noexpand\documentclass and
      \string\begin{document}.}%
  \@gobble}
\@onlypreamble\usepackage
```

(*End of definition for* \usepackage.)

\NeedsTeXFormat Check that the document is running on the correct system.

```
\def\NeedsTeXFormat#1{%
  \def\reserved@a{#1}%
  \ifx\reserved@a\fmtname
    \expandafter\@needsformat
  \else
     \@latex@error{This file needs format '\reserved@a'%
       \MessageBreak but this is '\fmtname'}{%
       The current input file will not be processed
       further,\MessageBreak
       because it was written for some other flavor of
       TeX.\MessageBreak\@ehd}%
```

If the file is not meant to be processed by LaTeX 2ε we stop inputting it, but we do not end the run. We just end inputting the current file.

```
     \endinput \fi}
\@onlypreamble\NeedsTeXFormat

\def\@needsformat{%
  \@ifnextchar[%]
    \@needsf@rmat
    {}}
\@onlypreamble\@needsformat

\def\@needsf@rmat[#1]{%
    \@ifl@t@r\fmtversion{#1}{}%
    {\@latex@warning@no@line
        {You have requested release `#1' of LaTeX,\MessageBreak
         but only release `\fmtversion' is available}}}
\@onlypreamble\@needsf@rmat
```

(*End of definition for* `\NeedsTeXFormat`.)

`\NeedsDocumentMetadata` Check that `\DocumentMetadata` has been used.

```
\protected\def\NeedsDocumentMetadata{%
  \IfDocumentMetadataF{%
     \@latex@error{This file needs \string\DocumentMetadata}%
      {The current input file will not be processed
       further,\MessageBreak
       because the document didn't start with \string\DocumentMetadata.%
       \MessageBreak\@ehd}}
  \IfDocumentMetadataF{\endinput}}%
```

(*End of definition for* `\NeedsDocumentMetadata`.)

`\zap@space` `\zap@space foo⟨space⟩\@empty` removes all spaces from `foo` that are not protected by `{ }` groups.

```
\def\zap@space#1 #2{%
  #1%
  \ifx#2\@empty\else\expandafter\zap@space\fi
  #2}
```

(*End of definition for* `\zap@space`.)

`\@fileswithoptions` The common part of `\documentclass` and `\usepackage`.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2024/06/01}%
⟨latexrelease⟩                    {\@fileswithoptions}{Check Group}%
⟨*2ekernel | latexrelease⟩
\def\@fileswithoptions#1{%
    \ifnum\currentgrouplevel>\z@
     \@latex@error
      {Loading a class or package in a group}%
      {Classes and packages should only be loaded at the top level}%
  \fi
  \@ifnextchar[%]
    {\@fileswith@ptions#1}%
    {\@fileswith@ptions#1[]}}
```

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\@fileswithoptions}{Check Group}%
⟨latexrelease⟩\def\@fileswithoptions#1{%
⟨latexrelease⟩   \@ifnextchar[%]
⟨latexrelease⟩     {\@fileswith@ptions#1}%
⟨latexrelease⟩     {\@fileswith@ptions#1[]}}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
\@onlypreamble\@fileswithoptions

\def\@fileswith@ptions#1[#2]#3{%
  \@ifnextchar[%]
  {\@fileswith@pti@ns#1[{#2}]#3}%
  {\@fileswith@pti@ns#1[{#2}]#3[]}}
\@onlypreamble\@fileswith@ptions
```

Then we do some work.

First of all, we define the global variables. Then we look to see if the file has already been loaded. If it has, we check that it was first loaded with at least the current options. If it has not, we add the current options to the package options, set the default version to be `0000/00/00`, and load the file if we can find it. Then we check the version number.

Finally, we restore the old file name, reset the default option, and we set the catcode of `@`.

For classes, we can immediately process the file. For other types, `#2` could be a comma separated list, so loop through, processing each one separately.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩         {\@fileswith@pti@ns}{ifx tests in \@fileswith@pti@ns}%
⟨*2ekernel | latexrelease⟩
\def\@fileswith@pti@ns#1[#2]#3[#4]{%
  \ifx#1\@clsextension
    \ifx\@classoptionslist\relax
      \protected@xdef\@classoptionslist{\zap@space#2 \@empty}%
```

Save raw class list.

```
      \gdef\@raw@classoptionslist{#2}%
      \def\reserved@a{%
        \@onefilewithoptions#3[{#2}][{#4}]#1%
        \@documentclasshook}%
    \else
      \def\reserved@a{%
        \@onefilewithoptions#3[{#2}][{#4}]#1}%
    \fi
  \else
```

build up a list of calls to `\@onefilewithoptions` (one for each package) without thrashing the parameter stack.

```
    \def\reserved@b##1,{%
```

If `#1` is `\@nnil` we have reached the end of the list (older version used `\@nil` here but `\@nil` is undefined so `\ifx` equal to all undefined commands)

```
      \ifx\@nnil##1\relax\else
```

If `\ifx\@nnil##1\@nnil` is true then `#1` is (presumably) empty (Older code used `\relax` which is slightly easier to get into `#1` by mistake, which would spoil this test.)

```
        \ifx\@nnil##1\@nnil\else

         \noexpand\@onefilewithoptions##1[{\unexpanded{#2}}][{#4}]%
         \noexpand\@pkgextension
        \fi
        \expandafter\reserved@b
      \fi}%
      \edef\reserved@a{\zap@space#3 \@empty}%
      \edef\reserved@a{\expandafter\reserved@b\reserved@a,\@nnil,}%
  \fi
  \reserved@a}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2017/01/01}%
⟨latexrelease⟩        {\@fileswith@pti@ns}{ifx tests in \@fileswith@pti@ns}%
⟨latexrelease⟩\def\@fileswith@pti@ns#1[#2]#3[#4]{%
⟨latexrelease⟩  \ifx#1\@clsextension
⟨latexrelease⟩    \ifx\@classoptionslist\relax
⟨latexrelease⟩      \xdef\@classoptionslist{\zap@space#2 \@empty}%
⟨latexrelease⟩      \def\reserved@a{%
⟨latexrelease⟩        \@onefilewithoptions#3[{#2}][{#4}]#1%
⟨latexrelease⟩        \@documentclasshook}%
⟨latexrelease⟩    \else
⟨latexrelease⟩      \def\reserved@a{%
⟨latexrelease⟩        \@onefilewithoptions#3[{#2}][{#4}]#1}%
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \else
⟨latexrelease⟩    \def\reserved@b##1,{%
⟨latexrelease⟩      \ifx\@nnil##1\relax\else
⟨latexrelease⟩        \ifx\@nnil##1\@nnil\else
⟨latexrelease⟩         \noexpand\@onefilewithoptions##1[{#2}][{#4}]%
⟨latexrelease⟩         \noexpand\@pkgextension
⟨latexrelease⟩        \fi
⟨latexrelease⟩        \expandafter\reserved@b
⟨latexrelease⟩      \fi}%
⟨latexrelease⟩      \edef\reserved@a{\zap@space#3 \@empty}%
⟨latexrelease⟩      \edef\reserved@a{\expandafter\reserved@b\reserved@a,\@nnil,}%
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \reserved@a}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩        {\@fileswith@pti@ns}{ifx tests in \@fileswith@pti@ns}%
⟨latexrelease⟩\def\@fileswith@pti@ns#1[#2]#3[#4]{%
⟨latexrelease⟩  \ifx#1\@clsextension
⟨latexrelease⟩    \ifx\@classoptionslist\relax
⟨latexrelease⟩      \xdef\@classoptionslist{\zap@space#2 \@empty}%
⟨latexrelease⟩      \def\reserved@a{%
⟨latexrelease⟩        \@onefilewithoptions#3[{#2}][{#4}]#1%
⟨latexrelease⟩        \@documentclasshook}%
⟨latexrelease⟩    \else
⟨latexrelease⟩      \def\reserved@a{%
⟨latexrelease⟩        \@onefilewithoptions#3[{#2}][{#4}]#1}%
```

```
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \else
⟨latexrelease⟩    \def\reserved@b##1,{%
⟨latexrelease⟩      \ifx\@nil##1\relax\else
⟨latexrelease⟩        \ifx\relax##1\relax\else
⟨latexrelease⟩          \noexpand\@onefilewithoptions##1[{#2}][{#4}]%
⟨latexrelease⟩          \noexpand\@pkgextension
⟨latexrelease⟩        \fi
⟨latexrelease⟩        \expandafter\reserved@b
⟨latexrelease⟩      \fi}%
⟨latexrelease⟩      \edef\reserved@a{\zap@space#3 \@empty}%
⟨latexrelease⟩      \edef\reserved@a{%
⟨latexrelease⟩        \expandafter\reserved@b\reserved@a,\@nil,}%
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \reserved@a}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
\@onlypreamble\@fileswith@pti@ns
```

This macro is used when loading packages or classes.

`\load@onefilewithoptions` Have the main argument as `#1`, so we only need one `\expandafter` above.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩       {\@onefilewithoptions}{Hooks and unused options issue}%
```

Here this macro is called `\@onefilewithoptions`, but further ahead in this file it is renamed to `\load@onefilewithoptions`, and `\@onefilewithoptions` becomes a wrapper around this, used for bookkeeping when rolling back. Therefore, when in latexrelease, we need to define `\load@onefilewithoptions` instead, thus the extra guarded `\def` line below:

```
⟨*2ekernel⟩
\def\@onefilewithoptions#1[#2][#3]#4{%
⟨/2ekernel⟩
⟨latexrelease⟩\def\load@onefilewithoptions#1[#2][#3]#4{%
```

We have to sanitize file names, so that something like

```
\usepackage{some/local/path/array}
\usepackage{array}
```

won't load `array.sty` twice. It is remotely possible that those are two different files, but as a matter of principles, we will consider that the base file name uniquely identifies a package, regardless of where it lives. This assumption already holds for file hooks, for example, which address the hook to a file by its base name only.

We'll use `\@expl@@@filehook@set@curr@file@@nNN` to parse the file name and return the ⟨*path*⟩ and ⟨*base+ext*⟩ in separate token lists. Further ahead, most operations use `\@currname` which doesn't have a path attached to it; only few actions prepend `\@currpath` to `\@currname` (namely loading, as we have to respect the given path).

A file substitution isn't followed just yet because at this point we are parsing user input, so the file is still what the user asked for, and not the file actually loaded.

```
 \@expl@@@filehook@set@curr@file@@nNN{#1.#4}\reserved@a\reserved@b
 \edef\reserved@c{\def\noexpand\reserved@c####1%
```

```
    \detokenize\expandafter{\expanded{.#4}}%
    \noexpand\@nil{\def\noexpand\reserved@a{####1}}}\reserved@c
  \expandafter\reserved@c\reserved@a\@nil
  \@pushfilename
  \xdef\@currname{\string@makeletter\reserved@a}%
  \xdef\@currpath{\ifx\reserved@b\@empty\else\reserved@b/\fi}%
  \global\let\@currext#4%
```

The command `\ver@`⟨*file*⟩`.`⟨*ext*⟩ is used to signal that a package is already loaded, either because it is in fact loaded, or because it's loading was suppressed. In minimal installations, said package may not exist but still have its loading suppressed with `\ver@`⟨*file*⟩`.`⟨*ext*⟩, so before checking if the file exists we have to check that we do need to load it with `\@ifl@aded`. If we don't, then there's no point in checking for a typo or load-disabling.

```
  \@ifl@aded\@currext\@currname
```

In the current preferred approach, a key family name will exist for processing using lt3keys. In that case, we replace the previous package options with the new ones, then call the key handler. Otherwise, we use the more classical clash handler.

```
  {%
    \@ifundefined{opt@handler@\@currname.\@currext}
      {\@onefilewithoptions@clashchk{#2}}
      {%

        \expandafter\protected@edef
          \csname opt@\@currname.\@currext\endcsname
          {\zap@space#2 \@empty}%

        \expandafter\def
          \csname @raw@opt@\@currname.\@currext\expandafter\endcsname
          \expandafter{#2}%
        \@nameuse{opt@handler@\@currname.\@currext}%
      }%
  }%
  {\makeatletter
```

The next line seems to be necessary for 2.09 compatibility (the way the code is written there) This seems questionable and should be look at as in 2e it is definitely unnecessary at this point!

```
    \@reset@ptions
```

First we take the ⟨*name*⟩ and ⟨*ext*⟩ given in the argument and check if the file exists, and issue an error otherwise asking for a correction with `\@missingfileerror`. For checking if the file exists we use `\@currpath` (usually empty) before `\@currname`.

```
    \IfFileExists{\@currpath\@currname.\@currext}{}%
      {\@missing@onefilewithoptions{#2}}%
```

If `\@currname` is empty (the user replied to the "Enter file name" prompt with ⟨*RETURN*⟩), so stop here (do `\@popfilename` to pop the item just added above).

This `\@gobble` omits the date check at the end.

```
    \ifx\@currname\@empty
      \expandafter\@gobble
    \else
```

If the file exists, check if it was load-prevented, and otherwise do the bookkeeping with `\@filehook@file@push` then call `\set@curr@file` to set `\@curr@file` (and do any required substitution), then actually load the class/package with `\load@onefile@withoptions`. `\set@curr@file` also needs the file path.

```
      \@disable@packageload@do{\@currname.\@currext}%
        {\@expl@@@filehook@file@push@@
         \set@curr@file{\@currpath\@currname.\@currext}%
         \@filehook@set@CurrentFile
```

The `\set@curr@file` line above might have replaced the file, so `\@currname` and `\@currext` may no longer hold the actual package being loaded, so in that case we need to update these two token lists (`\@curr@file` holds the file name after replacement, so we parse that).

The requested file is saved in `\@currpkg@reqd` to be used in `\InputIfFileExists` later: if the updated `\@currname` and `\@currext` are used we lose track of the substitution, so `\CurrentFile` and `\CurrentFileUsed` will be (incorrectly) the same.

```
         \expandafter\@swaptwoargs\expandafter
           {\expandafter{\@currpkg@reqd}}%
           {% <
```

`\@currpkg@reqd` doesn't take a path because it is used later to assign `\opt@...` and `\ver@....`

```
         \edef\@currpkg@reqd{\@currname.\@currext}%
         \ifx\CurrentFile\CurrentFileUsed
         \else
           \filename@parse\@curr@file
           \edef\@currpath{\string@makeletter\filename@area}%
           \edef\@currname{\string@makeletter\filename@base}%
           \edef\@currext{\string@makeletter\filename@ext}%
         \fi
         \load@onefile@withoptions{#2}%
         \def\@currpkg@reqd%{\@currpkg@reqd}
           }% >
```

Now just clean up and exit.

```
         \@expl@@@filehook@file@pop@@}%
       \expandafter\@firstofone
     \fi}%
```

Except in the case where `\@currname` is empty, the date is checked against the date marked in the package file:

```
   {\@ifl@ter\@currext{\@currname}{#3}{}%
     {\@latex@warning@no@line
       {You have requested,\on@line,
        version\MessageBreak
          `#3' of \@cls@pkg\space \@currname,\MessageBreak
        but only version\MessageBreak
         `\csname ver@\@currname.\@currext\endcsname'\MessageBreak
        is available}}%

    \ifx\@currext\@clsextension\let\LoadClass\@twoloadclasserror\fi}%
   \@popfilename
   \@reset@ptions}
```

If the package is already loaded, check that there were no option clashes.

\@onefilewithoptions@clashchk
```
\def\@onefilewithoptions@clashchk#1{%
  \@if@ptions\@currext{\@currname}{#1}{}%
     {\@latex@error
       {Option clash for \@cls@pkg\space \@currname}%
       {The package \@currname\space has already been loaded
        with options:\MessageBreak
        \space\space[\@ptionlist{\@currname.\@currext}]\MessageBreak
        There has now been an attempt to load it
         with options\MessageBreak
        \space\space[#1]\MessageBreak
        Adding the global options:\MessageBreak
        \space\space
             \@ptionlist{\@currname.\@currext},#1\MessageBreak
        to your \noexpand\documentclass declaration may fix this.%
        \MessageBreak
        Try typing \space <return> \space to proceed.}}%
    \@firstofone}
\let\@currpkg@reqd\@empty
\@onlypreamble\@onefilewithoptions
```

The kernel no longer uses `\@unprocessedoptions`

```
\let\@unprocessedoptions\@undefined
```

Now the action taken when a file is not found. Path must be included here as it eventually leads to a file lookup.

\@missing@onefilewithoptions
```
\def\@missing@onefilewithoptions#1{%
  \@missingfileerror{\@currpath\@currname}\@currext
  \global\let\@currpath\@missingfile@area
  \global\let\@currname\@missingfile@base
  \global\let\@currext\@missingfile@ext}
```

Now the code that actually does the file loading:

\load@onefile@withoptions
```
\def\load@onefile@withoptions#1{%
  \let\CurrentOption\@empty
  \@reset@ptions
```

Grab everything in a macro, so the parameter stack is popped before any processing begins.

```
  \def\reserved@a{%
    \@pass@ptions\@currext{#1}{\@currname}%
    \expandafter\let
      \csname opt@\@currpkg@reqd\expandafter\endcsname
      \csname opt@\@currname.\@currext\endcsname
    \expandafter\let
      \csname @raw@opt@\@currpkg@reqd\expandafter\endcsname
      \csname @raw@opt@\@currname.\@currext\endcsname
    \global\expandafter
    \let\csname ver@\@currname.\@currext\endcsname\@empty
```

We initialize `\...-h@@k` here and only if we load the file so that it remains undefined otherwise.

```
    \expandafter\let\csname\@currname.\@currext-h@@k\endcsname\@empty
```

When the current extension is `\@pkgextension` we are loading a package otherwise, if it is `\@clsextension`, a class, so depending on that we execute different hooks. If the extension is neither, then it is another type of file without special hooks.

```
%----------------------------------------
    \ifx\@currext\@pkgextension
      \UseHook{package/before}%
      \UseOneTimeHook{package/\@currname/before}%
    \else
      \ifx\@currext\@clsextension
        \UseHook{class/before}%
        \UseOneTimeHook{class/\@currname/before}%
      \fi
    \fi
```

Now actually load the file (at this point we are certain it exists, but use `\InputIfFileExists` so that file hooks are executed). `\@currpath` is needed here too.

```
    \InputIfFileExists{\@currpath\@currpkg@reqd}{}%
      {\@latex@error
        {The \@cls@pkg\space\@currpkg@reqd\space failed to load}\@ehd}%
%----------------------------------------
```

In older versions of the code `\@unprocessedoptions` would generate an error for each specified option in a package unless a `\ProcessOptions` has appeared in the package file.

This has changed in 2020. We now use a separate macro per package to avoid interference in case of nested packages. The whole code for handling this issue (GitHub 22) was provided by Hironobu Yamashita, thanks for that.

```
    \expandafter\let\csname unprocessedoptions-\@currname.\@currext\endcsname
                    \@@unprocessedoptions
    \csname\@currname.\@currext-h@@k\endcsname
    \expandafter\let\csname\@currname.\@currext-h@@k\endcsname
              \@undefined
```

Catch the case where the packages has handled the options and redefined `\@unprocessedoptions` to `\relax` (old interface). In that case no error should be produced.

```
    \ifx\@unprocessedoptions\relax
      \let\@unprocessedoptions\@undefined
```

Otherwise run the per package set of unused options.

```
    \else
      \csname unprocessedoptions-\@currname.\@currext\endcsname
    \fi
```

In either case we drop the macro afterwards as it is no longer needed.

```
    \expandafter\let
        \csname unprocessedoptions-\@currname.\@currext\endcsname
       \@undefined
```

And same procedure, James, when we are finished loading, except that the hook order is now reversed.

```
%----------------------------------------
    \ifx\@currext\@pkgextension
      \UseOneTimeHook{package/\@currname/after}%
      \UseHook{package/after}%
    \else
```

```
      \ifx\@currext\@clsextension
        \UseOneTimeHook{class/\@currname/after}%
        \UseHook{class/after}%
      \fi
    \fi}%
%---------------------------------------
  \@ifl@aded\@currext\@currname{}{\reserved@a}}
```

Now declare the non-generic package and class hooks used above:

```
\NewHook{package/before}
\NewHook{class/before}
\NewReversedHook{package/after}
\NewReversedHook{class/after}
```

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩       {\@onefilewithoptions}{Hooks and unused options issue}%
⟨latexrelease⟩
```

Because of the way `\@onfilewithoptions` is changed for rollback handling below we have to define `\load@onefilewithoptions` when rolling back!

```
⟨latexrelease⟩\def\load@onefilewithoptions#1[#2][#3]#4{%
⟨latexrelease⟩  \@pushfilename
⟨latexrelease⟩  \xdef\@currname{#1}%
⟨latexrelease⟩  \global\let\@currext#4%
⟨latexrelease⟩  \let\CurrentOption\@empty
⟨latexrelease⟩  \@reset@ptions
⟨latexrelease⟩  \makeatletter
⟨latexrelease⟩  \def\reserved@a{%
⟨latexrelease⟩    \@ifl@aded\@currext{#1}%
⟨latexrelease⟩      {\@if@ptions\@currext{#1}{#2}{}%
⟨latexrelease⟩        {\@latex@error
⟨latexrelease⟩            {Option clash for \@cls@pkg\space #1}%
⟨latexrelease⟩            {The package #1 has already been loaded
⟨latexrelease⟩             with options:\MessageBreak
⟨latexrelease⟩             \space\space[\@ptionlist{#1.\@currext}]\MessageBreak
⟨latexrelease⟩             There has now been an attempt to load it
⟨latexrelease⟩              with options\MessageBreak
⟨latexrelease⟩             \space\space[#2]\MessageBreak
⟨latexrelease⟩             Adding the global options:\MessageBreak
⟨latexrelease⟩             \space\space
⟨latexrelease⟩                  \@ptionlist{#1.\@currext},#2\MessageBreak
⟨latexrelease⟩             to your \noexpand\documentclass declaration may fix this.%
⟨latexrelease⟩             \MessageBreak
⟨latexrelease⟩             Try typing \space <return> \space to proceed.}}}%
⟨latexrelease⟩      {\@pass@ptions\@currext{#2}{#1}%
⟨latexrelease⟩       \global\expandafter
⟨latexrelease⟩       \let\csname ver@\@currname.\@currext\endcsname\@empty
⟨latexrelease⟩       \expandafter\let\csname\@currname.\@currext-h@@k\endcsname\@empty
⟨latexrelease⟩       \InputIfFileExists
⟨latexrelease⟩         {\@currname.\@currext}%
⟨latexrelease⟩         {}%
⟨latexrelease⟩         {\@missingfileerror\@currname\@currext}%
⟨latexrelease⟩    \let\@unprocessedoptions\@@unprocessedoptions
```

```
⟨latexrelease⟩    \csname\@currname.\@currext-h@@k\endcsname
⟨latexrelease⟩    \expandafter\let\csname\@currname.\@currext-h@@k\endcsname
⟨latexrelease⟩              \@undefined
⟨latexrelease⟩    \@unprocessedoptions}%
⟨latexrelease⟩    \@ifl@ter\@currext{#1}{#3}{}%
⟨latexrelease⟩      {\@latex@warning@no@line
⟨latexrelease⟩         {You have requested,\on@line,
⟨latexrelease⟩          version\MessageBreak
⟨latexrelease⟩            `#3' of \@cls@pkg\space #1,\MessageBreak
⟨latexrelease⟩          but only version\MessageBreak
⟨latexrelease⟩           `\csname ver@#1.\@currext\endcsname'\MessageBreak
⟨latexrelease⟩          is available}}%
⟨latexrelease⟩    \ifx\@currext\@clsextension\let\LoadClass\@twoloadclasserror\fi
⟨latexrelease⟩    \@popfilename
⟨latexrelease⟩    \@reset@ptions}%
⟨latexrelease⟩  \reserved@a}
⟨latexrelease⟩
⟨latexrelease⟩\let \load@onefile@withoptions    \@undefined
⟨latexrelease⟩\let \@missing@onefilewithoptions \@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@fileswithoptions` *and others.*)

`\@@fileswith@pti@ns` Save the definition (for error checking).

```
\let\@@fileswith@pti@ns\@fileswith@pti@ns
\@onlypreamble\@@fileswith@pti@ns
```

(*End of definition for* `\@@fileswith@pti@ns`.)

`\@reset@ptions` Reset the default option, and clear lists of declared options.

```
\def\@reset@ptions{%
  \global\ifx\@currext\@clsextension
    \let\default@ds\OptionNotUsed
   \else
    \let\default@ds\@unknownoptionerror
  \fi
  \global\let\ds@\@empty
  \global\let\@declaredoptions\@empty}
\@onlypreamble\@reset@ptions
```

(*End of definition for* `\@reset@ptions`.)

4.1 Hooks

Allow code to be saved to be executed at specific later times.

Here we save things in macros. I considered using toks registers (and `\addto@hook` from the NFSS code), but that would require stacking the contents in the case of required packages, so just generate a new macro for each package.

`\@begindocumenthook` `\@enddocumenthook` Stuff to appear at the beginning or end of the document.

```
\ifx\@begindocumenthook\@undefined
  \let\@begindocumenthook\@empty
\fi
\let\@enddocumenthook\@empty
```

(*End of definition for* `\@begindocumenthook` *and* `\@enddocumenthook`.)

`\AtEndOfPackage` `\AtEndOfClass` `\AtBeginDocument` `\AtEndDocument`

The access functions.

```
\def\AtEndOfPackage{%
  \expandafter\g@addto@macro\csname\@currname.\@currext-h@@k\endcsname}
\let\AtEndOfClass\AtEndOfPackage
\@onlypreamble\AtEndOfPackage
\@onlypreamble\AtEndOfClass
```

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                    {\AtBeginDocument}{Use hook system}%
\DeclareRobustCommand\AtBeginDocument{\AddToHook{begindocument}}
\DeclareRobustCommand\AtEndDocument  {\AddToHook{enddocument}}
%\DeclareRobustCommand\AtEndDocument {\AddToHook{env/document/end}} % alternative impl
```

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\AtBeginDocument}{Use hook system}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand\AtBeginDocument{\g@addto@macro\@begindocumenthook}
⟨latexrelease⟩\DeclareRobustCommand\AtEndDocument{\g@addto@macro\@enddocumenthook}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

In its initial implementation (not using the hook system) `\AtBeginDocument` was made `\@onlypreamble` because using it later had no effect whatsoever, thus was most certainly an unintended programming error. With the reimplementation, using the `begindocument` hook internally, this has changed because adding to a onetime hook after it has already been used simply executes the additional code immediately. We therefore no longer generate an error if it is used inside the document so that `\AddToHook{begindocument}` and `\AtBeginDocument` are truly equivalent (as claimed in the hook documentation).

```
%\@onlypreamble\AtBeginDocument
```

(*End of definition for* `\AtEndOfPackage` *and others.*)

`\@cls@pkg` The current file type.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2024/11/01}%
⟨latexrelease⟩                    {\@cls@pkg}{Allow for more extensions}%
\def\@cls@pkg{%
  \ifx\@currext\@clsextension
    document class%
  \else
    \ifx\@currext\@pkgextension
      package%
    \else
      file%
    \fi
  \fi}
```

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@cls@pkg}{Allow for more extensions}%
⟨latexrelease⟩
⟨latexrelease⟩\def\@cls@pkg{%
⟨latexrelease⟩  \ifx\@currext\@clsextension
⟨latexrelease⟩    document class%
⟨latexrelease⟩  \else
⟨latexrelease⟩    package%
⟨latexrelease⟩  \fi}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
\@onlypreamble\@cls@pkg
```

(*End of definition for* `\@cls@pkg`.)

`\@unknownoptionerror` Bad option.

```
\def\@unknownoptionerror{%
  \@latex@error
    {Unknown option `\CurrentOption' for \@cls@pkg\space`\@currname'}%
    {The option `\CurrentOption' was not declared in
     \@cls@pkg\space`\@currname', perhaps you\MessageBreak
      misspelled its name.
     Try typing \space <return>
     \space to proceed.}}
\@onlypreamble\@unknownoptionerror
```

(*End of definition for* `\@unknownoptionerror`.)

`\@@unprocessedoptions` Declare an error for each option, unless a `\ProcessOptions` occurred.

```
\def\@@unprocessedoptions{%
  \ifx\@currext\@pkgextension
    \protected@edef\@curroptions{\@ptionlist{\@currname.\@currext}}%
    \@for\CurrentOption:=\@curroptions\do{%
        \ifx\CurrentOption\@empty\else\@unknownoptionerror\fi}%
  \fi}
\@onlypreamble\@unprocessedoptions
\@onlypreamble\@@unprocessedoptions
```

(*End of definition for* `\@@unprocessedoptions`.)

`\@badrequireerror` `\RequirePackage` or `\LoadClass` occurs in the options section.

```
\def\@badrequireerror#1[#2]#3[#4]{%
  \@latex@error
    {\noexpand\RequirePackage or \noexpand\LoadClass
         in Options Section}%
    {The \@cls@pkg\space `\@currname' is defective.\MessageBreak
     It attempts to load `#3' in the options section, i.e.,\MessageBreak
     between \noexpand\DeclareOption and \string\ProcessOptions.}}
\@onlypreamble\@badrequireerror
```

(*End of definition for* `\@badrequireerror`.)

\@twoloadclasserror Two \LoadClass in a class.

```
\def\@twoloadclasserror{%
  \@latex@error
    {Two \noexpand\LoadClass commands}%
    {You may only use one \noexpand\LoadClass in a class file}}
\@onlypreamble\@twoloadclasserror
```

(*End of definition for* \@twoloadclasserror.)

\@twoclasseserror Two \documentclass or \documentstyle.

```
\def\@twoclasseserror#1#{%
  \@latex@error
    {Two \noexpand\documentclass or \noexpand\documentstyle commands}%
    {The document may only declare one class.}\@gobble}
\@onlypreamble\@twoclasseserror
```

(*End of definition for* \@twoclasseserror.)

4.2 Providing shipment

\two@digits Prefix a number less than 10 with '0'.

```
\def\two@digits#1{\ifnum#1<10 0\fi\number#1}
```

(*End of definition for* \two@digits.)

file\filecontents(arts) \endfilecontents This environment implements inline files. The star-form does not write extra comments into the file.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                    {\filec@ntents}{Define \q@curr@file directly (gh/220)}%
%
```

We use @tempswa to mean no preamble writing and reuse @filesw to indicate no overwriting:

```
\def\filecontents{\@tempswatrue\@fileswtrue
  \@ifnextchar[\filec@ntents@opt\filec@ntents
}
\@namedef{filecontents*}{\@tempswafalse\@fileswtrue
  \@ifnextchar[\filec@ntents@opt\filec@ntents
}
```

To handle the optional argument we execute for each option the command \filec@ntents@OPTION if it exist or complain about unknown option.

```
\def\filec@ntents@opt[#1]{%
  \edef\@fortmp{\zap@space#1 \@empty}%
  \@for\reserved@a:=\@fortmp\do{%
    \ifcsname filec@ntents@\reserved@a\endcsname
      \csname filec@ntents@\reserved@a\endcsname
    \else
    \@latex@error{Unknown filecontents option \reserved@a}%
       {Valid options are force (or overwrite), nosearch, noheader, nowarn}%
    \fi}%
  \filec@ntents
}
```

Option `force` (or `overwrite`) changes the overwriting switch

```
\let\filec@ntents@force\@fileswfalse
\let\filec@ntents@overwrite\@fileswfalse  % alternative name
```

and option `noheader` the preamble switch (which is equivalent to using the star form of the environment).

```
\let\filec@ntents@noheader\@tempswafalse
```

Option `nosearch` only checks the current directory not the whole TEX tree for the existence of the file to write.

```
\def\filec@ntents@nosearch{%
  \let\filec@ntents@checkdir\@currdir
  \def\filec@ntents@where{in current directory}}
```

By default we search the whole tree:

```
\let\filec@ntents@checkdir\@empty
\def\filec@ntents@where{exists on the system}
```

Option `nowarn` does not show any warning on the terminal but still writes it to the `.log`.

```
\def\filec@ntents@nowarn{%
  \let\filec@ntents@warning\@latex@note@no@line
}
```

By default we show terminal warnings.

```
\let\filec@ntents@warning\@latex@warning@no@line
```

```
\begingroup%
\@tempcnta=1
\loop
  \catcode\@tempcnta=12  %
  \advance\@tempcnta\@ne %
\ifnum\@tempcnta<32      %
\repeat                  %
\catcode`\*=11 %
\catcode`\^^M\active%
\catcode`\^^L\active\let^^L\relax%
\catcode`\^^I\active%
```

```
\gdef\filec@ntents#1{%
  \set@curr@file{\filec@ntents@checkdir#1}%
  \edef\q@curr@file{"\@curr@file"}%
```

LuaTEX has more writes (and 18 is safe here).

```
  \chardef\reserved@c\ifx\directlua\@undefined 15 \else 127 \fi%
  \openin\@inputcheck\q@curr@file \space %
  \ifeof\@inputcheck%
    \@latex@note@no@line%
        {Writing file `\@currdir\@curr@file'}%
```

```
    \ch@ck7\reserved@c\write\relax%
    \immediate\openout\reserved@c\q@curr@file\relax%
  \else%
```

```
    \if@filesw%
      \@latex@note@no@line%
          {File `\@curr@file' already \filec@ntents@where.\MessageBreak%
             Not generating it from this source}%
      \let\write\@gobbletwo%
      \let\closeout\@gobble%
    \else%
```

If we are overwriting, we try to make sure that the user is not by mistake overwriting the input file (`\jobname`). Of course, this only works for input files ending in `.tex`. If a different extension is used there is no way to see that we are overwriting ourselves!

```
      \edef\reserved@b{\detokenize\expandafter{\jobname}}%
      \ifx\@curr@file\reserved@b%
        \@fileswtrue%
      \else%
        \edef\reserved@b{\reserved@b\detokenize{.tex}}%
        \ifx\@curr@file\reserved@b
          \@fileswtrue%
        \fi%
      \fi%
```

We allocate a write channel but we open it only if it is (hopefully) safe. If not opened that means we are going to write on the terminal.

```
      \ch@ck7\reserved@c\write\relax%
      \if@filesw%  % Foul ... trying to overwrite \jobname!
      \@latex@error{Trying to overwrite `\jobname.tex'}{You can't %
        write to the file you are reading from!\MessageBreak%
        Data is written to screen instead.}%
      \else%
        \filec@ntents@warning%
           {Writing or overwriting file `\@curr@file'}%
        \immediate\openout\reserved@c\q@curr@file\relax%
      \fi%
    \fi%
  \fi%
```

Closing the `\@inputcheck` is done here to avoid having to do this in each branch.

```
  \closein\@inputcheck%
  \if@tempswa%

    \immediate\write\reserved@c{%
      \@percentchar\@percentchar\space%
          \expandafter\@gobble\string\LaTeX2e file `\@curr@file'^^J%
      \@percentchar\@percentchar\space  generated by the %
        `\@currenvir' \expandafter\@gobblefour\string\newenvironment^^J%
      \@percentchar\@percentchar\space from source `\jobname' on %
         \number\year/\two@digits\month/\two@digits\day.^^J%
      \@percentchar\@percentchar}%
  \fi%
  \let\do\@makeother\dospecials%
```

If there are active characters in the upper half (e.g., from `inputenc`) there would be confusion so we render everything harmless.

```
  \count@ 128\relax%
  \loop%
```

```
    \catcode\count@ 11\relax%
    \advance\count@ \@ne%
    \ifnum\count@<\@cclvi%
  \repeat%

  \edef\E{\@backslashchar end\string{\@currenvir\string}}%
  \edef\reserved@b{%
    \def\noexpand\reserved@b%
         ####1\E####2\E####3\relax}%
  \reserved@b{%
    \ifx\relax##3\relax%
```

There was no `\end{filecontents}`

```
      \immediate\write\reserved@c{##1}%
    \else%
```

There was a `\end{filecontents}`, so stop this time.

```
      \edef^^M{\noexpand\end{\@currenvir}}%
      \ifx\relax##1\relax%
      \else%
```

Text before the `\end`, write it with a warning.

```
          \@latex@warning{Writing text `##1' before %
            \string\end{\@currenvir}\MessageBreak
            as last line of \@curr@file}%
        \immediate\write\reserved@c{##1}%
      \fi%
      \ifx\relax##2\relax%
      \else%
```

Text after the `\end`, ignore it with a warning.

```
          \@latex@warning{%
            Ignoring text `##2' after \string\end{\@currenvir}}%
      \fi%
    \fi%
    ^^M}%

  \catcode`\^^L\active%
  \let\L\@undefined%
  \def^^L{\expandafter\ifx\csname L\endcsname\relax\fi ^^J^^J}%
  \catcode`\^^I\active%
  \let\I\@undefined%
  \def^^I{\expandafter\ifx\csname I\endcsname\relax\fi\space}%
  \catcode`\^^M\active%
  \edef^^M##1^^M{%
    \noexpand\reserved@b##1\E\E\relax}}%
\endgroup%

⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩    {\filec@ntents}{Spaces in file names + optional arg}%
⟨latexrelease⟩
⟨latexrelease⟩\def\filecontents{\@tempswatrue\@fileswtrue
⟨latexrelease⟩  \@ifnextchar[\filec@ntents@opt\filec@ntents
⟨latexrelease⟩}
⟨latexrelease⟩\@namedef{filecontents*}{\@tempswafalse\@fileswtrue
```

```
⟨latexrelease⟩  \@ifnextchar[\filec@ntents@opt\filec@ntents
⟨latexrelease⟩}
⟨latexrelease⟩\def\filec@ntents@opt[#1]{%
⟨latexrelease⟩  \edef\@fortmp{\zap@space#1 \@empty}%
⟨latexrelease⟩  \@for\reserved@a:=\@fortmp\do{%
⟨latexrelease⟩    \ifcsname filec@ntents@\reserved@a\endcsname
⟨latexrelease⟩      \csname filec@ntents@\reserved@a\endcsname
⟨latexrelease⟩    \else
⟨latexrelease⟩    \@latex@error{Unknown filecontents option \reserved@a}%
⟨latexrelease⟩       {Valid options are force (or overwrite), nosearch, noheader}%
⟨latexrelease⟩    \fi}%
⟨latexrelease⟩  \filec@ntents
⟨latexrelease⟩}
⟨latexrelease⟩\let\filec@ntents@force\@fileswfalse
⟨latexrelease⟩\let\filec@ntents@overwrite\@fileswfalse  % alternative name
⟨latexrelease⟩\let\filec@ntents@noheader\@tempswafalse
⟨latexrelease⟩\def\filec@ntents@nosearch{%
⟨latexrelease⟩  \let\filec@ntents@checkdir\@currdir
⟨latexrelease⟩  \def\filec@ntents@where{in current directory}}
⟨latexrelease⟩\let\filec@ntents@checkdir\@empty
⟨latexrelease⟩\def\filec@ntents@where{exists on the system}
⟨latexrelease⟩\begingroup%
⟨latexrelease⟩\@tempcnta=1
⟨latexrelease⟩\loop
⟨latexrelease⟩  \catcode\@tempcnta=12  %
⟨latexrelease⟩  \advance\@tempcnta\@ne %
⟨latexrelease⟩\ifnum\@tempcnta<32      %
⟨latexrelease⟩\repeat                  %
⟨latexrelease⟩\catcode`\*=11 %
⟨latexrelease⟩\catcode`\^^M\active%
⟨latexrelease⟩\catcode`\^^L\active\let^^L\relax%
⟨latexrelease⟩\catcode`\^^I\active%
⟨latexrelease⟩\gdef\filec@ntents#1{%
⟨latexrelease⟩  \set@curr@file{\filec@ntents@checkdir#1}%
⟨latexrelease⟩  \edef\q@curr@file{\expandafter\quote@name\expandafter{\@curr@file}}%
⟨latexrelease⟩  \chardef\reserved@c\ifx\directlua\@undefined 15 \else 127 \fi%
⟨latexrelease⟩  \openin\@inputcheck\q@curr@file \space %
⟨latexrelease⟩  \ifeof\@inputcheck%
⟨latexrelease⟩    \@latex@warning@no@line%
⟨latexrelease⟩        {Writing file `\@currdir\@curr@file'}%
⟨latexrelease⟩    \ch@ck7\reserved@c\write\relax%
⟨latexrelease⟩    \immediate\openout\reserved@c\q@curr@file\relax%
⟨latexrelease⟩  \else%
⟨latexrelease⟩    \if@filesw%
⟨latexrelease⟩      \@latex@warning@no@line%
⟨latexrelease⟩          {File `\@curr@file' already \filec@ntents@where.\MessageBreak%
⟨latexrelease⟩             Not generating it from this source}%
⟨latexrelease⟩      \let\write\@gobbletwo%
⟨latexrelease⟩      \let\closeout\@gobble%
⟨latexrelease⟩    \else%
⟨latexrelease⟩      \edef\reserved@a{#1}%
⟨latexrelease⟩      \edef\reserved@a{\detokenize\expandafter{\reserved@a}}%
⟨latexrelease⟩      \edef\reserved@b{\detokenize\expandafter{\jobname}}%
⟨latexrelease⟩      \ifx\reserved@a\reserved@b%
```

```
⟨latexrelease⟩        \@fileswtrue%
⟨latexrelease⟩      \else%
⟨latexrelease⟩        \edef\reserved@b{\reserved@b\detokenize{.tex}}%
⟨latexrelease⟩        \ifx\reserved@a\reserved@b
⟨latexrelease⟩          \@fileswtrue%
⟨latexrelease⟩        \fi%
⟨latexrelease⟩      \fi%
⟨latexrelease⟩      \ch@ck7\reserved@c\write\relax%
⟨latexrelease⟩      \if@filesw%  % Foul ... trying to overwrite \jobname!
⟨latexrelease⟩      \@latex@error{Trying to overwrite `\jobname.tex'}{You can't %
⟨latexrelease⟩        write to the file you are reading from!\MessageBreak%
⟨latexrelease⟩        Data is written to screen instead.}%
⟨latexrelease⟩      \else%
⟨latexrelease⟩        \@latex@warning@no@line%
⟨latexrelease⟩           {Writing or overwriting file `\@currdir\@curr@file'}%
⟨latexrelease⟩        \immediate\openout\reserved@c\q@curr@file\relax%
⟨latexrelease⟩      \fi%
⟨latexrelease⟩    \fi%
⟨latexrelease⟩  \fi%
⟨latexrelease⟩  \closein\@inputcheck%
⟨latexrelease⟩  \if@tempswa%
⟨latexrelease⟩    \immediate\write\reserved@c{%
⟨latexrelease⟩      \@percentchar\@percentchar\space%
⟨latexrelease⟩          \expandafter\@gobble\string\LaTeX2e file `\@curr@file'^^J%
⟨latexrelease⟩      \@percentchar\@percentchar\space  generated by the %
⟨latexrelease⟩        `\@currenvir' \expandafter\@gobblefour\string\newenvironment^^J%
⟨latexrelease⟩      \@percentchar\@percentchar\space from source `\jobname' on %
⟨latexrelease⟩         \number\year/\two@digits\month/\two@digits\day.^^J%
⟨latexrelease⟩      \@percentchar\@percentchar}%
⟨latexrelease⟩  \fi%
⟨latexrelease⟩  \let\do\@makeother\dospecials%
⟨latexrelease⟩  \count@ 128\relax%
⟨latexrelease⟩  \loop%
⟨latexrelease⟩    \catcode\count@ 11\relax%
⟨latexrelease⟩    \advance\count@ \@ne%
⟨latexrelease⟩    \ifnum\count@<\@cclvi%
⟨latexrelease⟩  \repeat%
⟨latexrelease⟩  \edef\E{\@backslashchar end\string{\@currenvir\string}}%
⟨latexrelease⟩  \edef\reserved@b{%
⟨latexrelease⟩    \def\noexpand\reserved@b%
⟨latexrelease⟩         ####1\E####2\E####3\relax}%
⟨latexrelease⟩  \reserved@b{%
⟨latexrelease⟩    \ifx\relax##3\relax%
⟨latexrelease⟩      \immediate\write\reserved@c{##1}%
⟨latexrelease⟩    \else%
⟨latexrelease⟩      \edef^^M{\noexpand\end{\@currenvir}}%
⟨latexrelease⟩      \ifx\relax##1\relax%
⟨latexrelease⟩      \else%
⟨latexrelease⟩          \@latex@warning{Writing text `##1' before %
⟨latexrelease⟩             \string\end{\@currenvir}\MessageBreak as last line of \@curr@file}%
⟨latexrelease⟩        \immediate\write\reserved@c{##1}%
⟨latexrelease⟩      \fi%
⟨latexrelease⟩      \ifx\relax##2\relax%
⟨latexrelease⟩      \else%
```

```
⟨latexrelease⟩          \@latex@warning{%
⟨latexrelease⟩            Ignoring text `##2' after \string\end{\@currenvir}}%
⟨latexrelease⟩       \fi%
⟨latexrelease⟩     \fi%
⟨latexrelease⟩     ^^M}%
⟨latexrelease⟩   \catcode`\^^L\active%
⟨latexrelease⟩   \let\L\@undefined%
⟨latexrelease⟩   \def^^L{\expandafter\ifx\csname L\endcsname\relax\fi ^^J^^J}%
⟨latexrelease⟩   \catcode`\^^I\active%
⟨latexrelease⟩   \let\I\@undefined%
⟨latexrelease⟩   \def^^I{\expandafter\ifx\csname I\endcsname\relax\fi\space}%
⟨latexrelease⟩   \catcode`\^^M\active%
⟨latexrelease⟩   \edef^^M##1^^M{%
⟨latexrelease⟩     \noexpand\reserved@b##1\E\E\relax}}%
⟨latexrelease⟩\endgroup%
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩     {\filec@ntents}{Spaces in file names + optional arg}%
⟨latexrelease⟩
⟨latexrelease⟩\let\filec@ntents@opt        \@undefined
⟨latexrelease⟩\let\filec@ntents@force      \@undefined
⟨latexrelease⟩\let\filec@ntents@overwrite  \@undefined
⟨latexrelease⟩\let\filec@ntents@noheader   \@undefined
⟨latexrelease⟩\let\filec@ntents@nosearch   \@undefined
⟨latexrelease⟩\let\filec@ntents@checkdir   \@undefined
⟨latexrelease⟩\let\filec@ntents@where      \@undefined
⟨latexrelease⟩
⟨latexrelease⟩\begingroup%
⟨latexrelease⟩\@tempcnta=1
⟨latexrelease⟩\loop
⟨latexrelease⟩   \catcode\@tempcnta=12  %
⟨latexrelease⟩   \advance\@tempcnta\@ne %
⟨latexrelease⟩\ifnum\@tempcnta<32      %
⟨latexrelease⟩\repeat                  %
⟨latexrelease⟩\catcode`\*=11 %
⟨latexrelease⟩\catcode`\^^M\active%
⟨latexrelease⟩\catcode`\^^L\active\let^^L\relax%
⟨latexrelease⟩\catcode`\^^I\active%
⟨latexrelease⟩
⟨latexrelease⟩\gdef\filec@ntents#1{%
⟨latexrelease⟩   \openin\@inputcheck#1 %
⟨latexrelease⟩   \ifeof\@inputcheck%
⟨latexrelease⟩     \@latex@warning@no@line%
⟨latexrelease⟩         {Writing file `\@currdir#1'}%
⟨latexrelease⟩     \chardef\reserved@c15 %
⟨latexrelease⟩     \ch@ck7\reserved@c\write%
⟨latexrelease⟩     \immediate\openout\reserved@c#1\relax%
⟨latexrelease⟩   \else%
⟨latexrelease⟩     \closein\@inputcheck%
⟨latexrelease⟩     \@latex@warning@no@line%
⟨latexrelease⟩             {File `#1' already exists on the system.\MessageBreak%
⟨latexrelease⟩              Not generating it from this source}%
⟨latexrelease⟩     \let\write\@gobbletwo%
⟨latexrelease⟩     \let\closeout\@gobble%
```

```
⟨latexrelease⟩  \fi%
⟨latexrelease⟩  \if@tempswa%
⟨latexrelease⟩    \immediate\write\reserved@c{%
⟨latexrelease⟩      \@percentchar\@percentchar\space%
⟨latexrelease⟩          \expandafter\@gobble\string\LaTeX2e file `#1'^^J%
⟨latexrelease⟩      \@percentchar\@percentchar\space  generated by the %
⟨latexrelease⟩        `\@currenvir' \expandafter\@gobblefour\string\newenvironment^^J%
⟨latexrelease⟩      \@percentchar\@percentchar\space from source `\jobname' on %
⟨latexrelease⟩          \number\year/\two@digits\month/\two@digits\day.^^J%
⟨latexrelease⟩      \@percentchar\@percentchar}%
⟨latexrelease⟩  \fi%
⟨latexrelease⟩  \let\do\@makeother\dospecials%
⟨latexrelease⟩  \count@ 128\relax%
⟨latexrelease⟩  \loop%
⟨latexrelease⟩    \catcode\count@ 11\relax%
⟨latexrelease⟩    \advance\count@ \@ne%
⟨latexrelease⟩    \ifnum\count@<\@cclvi%
⟨latexrelease⟩  \repeat%
⟨latexrelease⟩  \edef\E{\@backslashchar end\string{\@currenvir\string}}%
⟨latexrelease⟩  \edef\reserved@b{%
⟨latexrelease⟩    \def\noexpand\reserved@b%
⟨latexrelease⟩         ####1\E####2\E####3\relax}%
⟨latexrelease⟩  \reserved@b{%
⟨latexrelease⟩    \ifx\relax##3\relax%
⟨latexrelease⟩      \immediate\write\reserved@c{##1}%
⟨latexrelease⟩    \else%
⟨latexrelease⟩      \edef^^M{\noexpand\end{\@currenvir}}%
⟨latexrelease⟩      \ifx\relax##1\relax%
⟨latexrelease⟩      \else%
⟨latexrelease⟩          \@latex@warning{Writing text `##1' before %
⟨latexrelease⟩             \string\end{\@currenvir}\MessageBreak as last line of #1}%
⟨latexrelease⟩        \immediate\write\reserved@c{##1}%
⟨latexrelease⟩      \fi%
⟨latexrelease⟩      \ifx\relax##2\relax%
⟨latexrelease⟩      \else%
⟨latexrelease⟩         \@latex@warning{%
⟨latexrelease⟩           Ignoring text `##2' after \string\end{\@currenvir}}%
⟨latexrelease⟩      \fi%
⟨latexrelease⟩    \fi%
⟨latexrelease⟩    ^^M}%
⟨latexrelease⟩
⟨latexrelease⟩  \catcode`\^^L\active%
⟨latexrelease⟩  \let\L\@undefined%
⟨latexrelease⟩  \def^^L{\expandafter\ifx\csname L\endcsname\relax\fi ^^J^^J}%
⟨latexrelease⟩  \catcode`\^^I\active%
⟨latexrelease⟩  \let\I\@undefined%
⟨latexrelease⟩  \def^^I{\expandafter\ifx\csname I\endcsname\relax\fi\space}%
⟨latexrelease⟩  \catcode`\^^M\active%
⟨latexrelease⟩  \edef^^M##1^^M{%
⟨latexrelease⟩    \noexpand\reserved@b##1\E\E\relax}}%
⟨latexrelease⟩\endgroup%
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
\begingroup
```

```
\catcode`|=\catcode`\%
\catcode`\%=12
\catcode`\*=11
\gdef\@percentchar{%}
\gdef\endfilecontents{|
  \immediate\closeout\reserved@c
  \def\T##1##2##3{|
  \ifx##1\@undefined\else
    \@latex@warning@no@line{##2 has been converted to Blank ##3e}|
  \fi}|
  \T\L{Form Feed}{Lin}|
  \T\I{Tab}{Spac}|
  \immediate\write\@unused{}}
\global\let\endfilecontents*\endfilecontents
```

We no longer prevent the code to be used after begin document (no rollback needed for this change).

```
%\@onlypreamble\filecontents
%\@onlypreamble\endfilecontents
%\@onlypreamble\filecontents*
%\@onlypreamble\endfilecontents*
\endgroup
%\@onlypreamble\filec@ntents
```

(*End of definition for* `\filecontents` *and* `\endfilecontents`.)

5 Package/class rollback mechanism

```
⟨/2ekernel⟩
⟨*2ekernel | latexreleasefirst⟩
```

`\pkgcls@debug` For testing we have a few extra lines of code that by default do nothing but one can set `\pkgcls@debug` to `\typeout` to get extra info. Sometime in the future this will be dropped.

```
⟨*tracerollback⟩
%\let\pkgcls@debug\typeout
\let\pkgcls@debug\@gobble
⟨/tracerollback⟩
```

(*End of definition for* `\pkgcls@debug`.)

`\requestedLaTeXdate` The macro (!) `\requestedLaTeXdate` holds the globally requested rollback date (via `latexrelease`) or zero if no such request was made.

```
\def\requestedLaTeXdate{0}
```

(*End of definition for* `\requestedLaTeXdate`.)

`\pkgcls@targetdate` `\pkgcls@targetlabel` `\pkgcls@innerdate` `\pkgcls@outerdate` If a rollback for a package or class is requested then `\pkgcls@targetdate` holds the requested date as a number YYYYMMDD (if there was one, otherwise the value of `\requestedLaTeXdate`) and `\pkgcls@targetlabel` will be empty. If there was a request for a named version then `\pkgcls@targetlabel` holds the version name and `\pkgcls@targetdate` is set to `1`.

`\pkgcls@targetdate=0` is used to indicate that there was no rollback request. While loading an old release `\pkgcls@targetdate` is also reset to zero so that `\DeclareRelease` declarations are bypassed.

In contrast `\pkgcls@innerdate` will always hold the requested date (in a macro not a counter) if there was one, otherwise, e.g., if there was no request or a request to a version name it will contain TEX largest legal number. While loading a file this can be used to provide conditionals that select code based on the request.

In `\pkgcls@outerdate` we save the inner date before it is changed to a specific target date. That value is then put on the file stack and retrieved when the file has been fully read. It is then used again for `\pkgcls@innerdate` for the remainder of the currently open file.

```
\ifx\pkgcls@targetdate\@undefined
  \newcount\pkgcls@targetdate
\fi
\let\pkgcls@targetlabel\@empty
\def\pkgcls@innerdate{\maxdimen}

\let\pkgcls@outerdate\pkgcls@innerdate
```

(*End of definition for* `\pkgcls@targetdate` *and others.*)

`\pkgcls@candidate` `\pkgcls@releasedate`

When looping through the `\DeclareRelease` declarations we record if the release is the best candidate we have seen so far. This is recorded in `\pkgcls@candidate` and we update it whenever we see a better one.

In `\pkgcls@releasedate` we keep track of the release date of that candidate.

```
\let\pkgcls@candidate\@empty
\let\pkgcls@releasedate\@empty
```

(*End of definition for* `\pkgcls@candidate` *and* `\pkgcls@releasedate`.)

`\load@onefilewithoptions` `\@onefilewithoptions`

the best place to add the rollback code is at the point where `\@onefilewithoptions` is called to load a single class or package.

To make things easy we save the old definition as `\load@onefilewithoptions` and then provide a new interface.

Important: as this code is also unconditionally placed into latexrelease we can only do this name change once otherwise both macros will contain the same code.

```
\ifx\load@onefilewithoptions\@undefined
 \let\load@onefilewithoptions\@onefilewithoptions
 \def\@onefilewithoptions#1[#2][#3]#4{%
```

First a bit of tracing normally disabled.

```
⟨*tracerollback⟩
  \pkgcls@debug{--- File loaded request (\noexpand\usepackage or ...)}%
  \pkgcls@debug{\@spaces 1: #1}%
  \pkgcls@debug{\@spaces 2: #2}%
  \pkgcls@debug{\@spaces 3: #3}%
  \pkgcls@debug{\@spaces 4: #4}%
⟨/tracerollback⟩
```

Three of the arguments are needed later on in error/warning messages so we save them.

```
  \def\pkgcls@name{#1}%                  % for info message
  \def\pkgcls@arg {#3}%                  % for info message
  \edef\pkgcls@ext{%
    \ifx#4\@clsextension document class\else
      \ifx#4\@pkgextension package\else
        file
```

```
      \fi
    \fi
  }%                                        % for info message
```

then we parse the final optional argument to determine if there is a specific rollback request for the current file. This will set `\pkgcls@targetdate`, `\pkgcls@targetlabel` and `\pkgcls@mindate`.

```
  \pkgcls@parse@date@arg{#3}%
```

When determining the correct release to load we keep track of candidates in `\pkgcls@candidate` and initially we don't have any:

```
  \let\pkgcls@candidate\@empty
```

If we had a rollback request then `#3` may contain data but not necessarily a "minimal date" so instead of passing it on we pass on the content of `\pkgcls@mindate`. We need to pass the value not the command, otherwise nested packages may pick up the wrong information.

```
  \begingroup
  \edef\reserved@a{%
    \endgroup
    \unexpanded{\load@onefilewithoptions#1[#2]}%
    [\pkgcls@mindate]%
    \unexpanded{#4}}%
   \reserved@a
 }
\fi
```

(*End of definition for* `\load@onefilewithoptions` *and* `\@onefilewithoptions`.)

`\pkgcls@parse@date@arg` The `\pkgcls@parse@date@arg` command parses the second optional argument of `\usepackage`, `\RequirePackage` or `\documentclass` for a rollback request setting the values of `\pkgcls@targetdate` and `\pkgcls@targetlabel`.

This optional argument has a dual purpose: If it just contains a date string then this means that the package should have at least that date (to ensure that a certain feature is actually available, or a certain bug has been fixed). When the package gets loaded the information in `\Provides...` will then be checked against this request.

But if it starts with an equal sign followed by a date string or followed by a version name then this means that we should roll back to the state of the package at that date or to the version with the requested name.

If there was no optional argument or the optional argument does not start with "`=`" then the `\pkgcls@targetdate` is set to the date of the overall rollback request (via `latexrelease`) or if that was not given it is set to `0`. In either case `\pkgcls@targetlabel` will be made empty.

If the argument doesn't start with "`=`" then it is supposed to be a "minimal date" and we therefore save the value in `\pkgcls@mindate`, otherwise this macro is made empty.

So in summary we have:

Input		`\pkgcls@targetdate`	`\pkgcls@targetlabel`	`\pkgcls@mindate`
⟨*empty*⟩	→	⟨*global-rollbackdate-as-number*⟩	⟨*empty*⟩	⟨*empty*⟩
⟨*date*⟩	→	⟨*global-rollbackdate-as-number*⟩	⟨*empty*⟩	⟨*date*⟩
=⟨*date*⟩	→	⟨*date-as-number*⟩	⟨*empty*⟩	⟨*empty*⟩
=⟨*version*⟩	→	1	⟨*version*⟩	⟨*empty*⟩
⟨*other*⟩	→	⟨*global-rollbackdate-as-number*⟩	⟨*empty*⟩	⟨*other*⟩

where ⟨*global-rollbackdate-as-number*⟩ is a date request given via `latexrelease` or if there wasn't one `0`.

```
\def\pkgcls@parse@date@arg #1{%
```

If the argument is empty we use the rollback date from `latexrelease` which has the value of zero if there was no rollback request. The label and the minimal date is made empty in that case.

```
  \ifx\@nil#1\@nil
    \pkgcls@targetdate\requestedLaTeXdate\relax
    \let\pkgcls@targetlabel\@empty
    \let\pkgcls@mindate\@empty
```

Otherwise we parse the argument further, checking for a `=` as the first character. We append a `=` at the end so that there is at least one such character in the argument.

```
  \else
    \pkgcls@parse@date@arg@#1=\@nil\relax
  \fi
}
```

The actual parsing work then happens in `\pkgcls@parse@date@arg@`:

```
\def\pkgcls@parse@date@arg@#1=#2\@nil{%
```

We set `\pkgcls@targetdate` depending on the parsing result; the code is expandable so we can do the parsing as part of the assignment.

```
  \pkgcls@targetdate
```

If a `=` was in first position then `#1` will be empty. In that case `#2` will be the original argument with a `=` appended.

This can be parsed with `\@parse@version`, the trailing character is simply ignored. This macro returns the parsed date as a number (or zero if it wasn't a date) and accepts both YYYY/MM/DD and YYYY-MM-DD formats.

```
    \ifx\@nil#1\@nil
      \@parse@version0#2//00\@nil\relax
```

Whatever is returned is thus assigned to `\pkgcls@targetdate` and therefore we can now test its value. If the value is zero we assume that the remaining argument string represents a version and change `\pkgcls@targetdate` and set `\pkgcls@targetlabel` to the version name (after stripping off the trailing `=`).

```
      \ifnum \pkgcls@targetdate=\z@
        \pkgcls@targetdate\@ne
        \def\pkgcls@innerdate{\maxdimen}%
        \pkgcls@parse@date@arg@version#2%
      \else
```

Save the `\pkgcls@innerdate` so that it can be placed on the file stack later.

```
        \let\pkgcls@outerdate\pkgcls@innerdate
        \edef\pkgcls@innerdate{\the\pkgcls@targetdate}%
      \fi
      \let\pkgcls@mindate\@empty
    \else
```

If `#1` was not empty then there wasn't a `=` character in first position so we are dealing either with a "minimum date" or with some incorrect data. We assume the former and make the following assignments (the first one finishing the assignment of `\pkgcls@targetdate`):

```
      \requestedLaTeXdate\relax
      \let\pkgcls@targetlabel\@empty
      \def\pkgcls@innerdate{\maxdimen}%
      \def\pkgcls@mindate{#1}%
```

If the min-date is after the requested rollback date (if there is any, i.e., if it is not zero) then we have a conflict and therefore issue a warning.

```
      \ifnum \pkgcls@targetdate > \z@
        \ifnum \@parse@version0#1//00\@nil > \pkgcls@targetdate
          \@latex@warning@no@line{Suspicious rollback/min-date date given\MessageBreak
            A minimal date of #1 has been specified for
             \pkgcls@ext\MessageBreak '\pkgcls@name'.\MessageBreak
             But this is in conflict
             with a rollback request to \requestedpatchdate}
        \fi
      \fi
    \fi
}
```

Strip off the trailing `=` and assign the version name to `\pkgcls@targetlabel`.

```
\def\pkgcls@parse@date@arg@version#1={%
  \def\pkgcls@targetlabel{#1}}
```

(*End of definition for* `\pkgcls@parse@date@arg`.)

`\DeclareRelease` First argument is the "name" of the release and it can be left empty if one doesn't like to give a name to the release. The second argument is that from which on this release was available (or should be used in case of minor updates). The final argument is the external file name of this release, by convention this should be ⟨*pkg/cls-name*⟩-⟨*date*⟩.⟨*extension*⟩ but this is not enforced and through this argument one can overwrite it.

```
\def\DeclareRelease#1#2#3{%
  \ifnum\pkgcls@targetdate>\z@  % some sort of rollback request
⟨*tracerollback⟩
    \pkgcls@debug{---\string\DeclareRelease:}%
    \pkgcls@debug{\@spaces 1: #1}%
    \pkgcls@debug{\@spaces 2: #2}%
    \pkgcls@debug{\@spaces 3: #3}%
⟨/tracerollback⟩
```

If the date argument `#2` is empty we are dealing with a special release that should be only accessible via its name; a typical use case would be a "beta" release. So if we are currently processing a date request we ignore it and otherwise we check if we can match the name and if so load the corresponding release file.

```
    \ifx\@nil#2\@nil
      \ifnum\pkgcls@targetdate=\@ne  % named request
        \def\reserved@a{#1}%
        \ifx\pkgcls@targetlabel\reserved@a
          \pkgcls@use@this@release{#3}{}%
⟨*tracerollback⟩
        \else
          \pkgcls@debug{Label doesn't match}%
⟨/tracerollback⟩
        \fi
⟨*tracerollback⟩
```

```
      \else
        \pkgcls@debug{Date request: ignored}%
⟨/tracerollback⟩
      \fi
    \else
```

If the value of `\pkgcls@targetdate` is greater than 1 (or in reality greater than something like 19930101) we are dealing with a rollback request to a specific date.

```
      \ifnum\pkgcls@targetdate>\@ne  % a real request
```

So we parse the date of this release to check if it is before or after the request date.

```
        \ifnum\@parse@version#2//00\@nil
             >\pkgcls@targetdate
```

If it is after we have to distinguish between two cases: If there was an earlier candidate we use that one because the other is too late, but if there wasn't one (i.e., if current release is the oldest that exists) we use it as the best choice. However in that case something is wrong (as there shouldn't be a rollback to a date when a package used didn't yet exists). So we make a complained to the user.

```
          \ifx\pkgcls@candidate\@empty
            \pkgcls@rollbackdate@error{#2}%
            \pkgcls@use@this@release{#3}{#2}%
          \else
            \pkgcls@use@this@release\pkgcls@candidate
                                    \pkgcls@releasedate
          \fi
        \else
```

Otherwise, if the release date of this version is before the target rollback and we record it as a candidate. But we don't use it yet as there may be another release which is still before the target rollback.

```
          \def\pkgcls@candidate{#3}%
          \def\pkgcls@releasedate{#2}%
⟨*tracerollback⟩
          \pkgcls@debug{New candidate: #3}%
⟨/tracerollback⟩
        \fi
      \else
```

If we end up in this branch we have a named version request. So we check if `\pkgcls@targetlabel` matches the current name and if yes we use this release immediately, otherwise we do nothing as a later declaration may match it.

```
        \def\reserved@a{#1}%
        \ifx\pkgcls@targetlabel\reserved@a
          \pkgcls@use@this@release{#3}{#2}%
⟨*tracerollback⟩
        \else
          \pkgcls@debug{Label doesn't match}%
⟨/tracerollback⟩
        \fi
      \fi
    \fi
  \fi
}
```

(*End of definition for* `\DeclareRelease`.)

\pkgcls@use@this@release If a certain release has been selected (stored in the external file given in #1) we need to input it and afterwards stop reading the current file.

```
\def\pkgcls@use@this@release#1#2{%
```

Before that we record the selection made inside the transcript.

```
  \pkgcls@show@selection{#1}{#2}%
```

We then set the \pkgcls@targetdate to zero so that any \DeclareRelease or \DeclareCurrentRelease in the file we now load are bypassed[52] and then we finally load the correct release.

After loading that file we need to stop reading the current file so we issue \endinput. Note that the \relax before that is essential to ensure that the \endinput is only happening after the file has been fully processed, otherwise it would act after the first line of the \@@input!

```
  \pkgcls@targetdate\z@
  \@addtofilelist{#1}%
  \@@input #1\relax
  \endinput
}
```

(*End of definition for* \pkgcls@use@this@release.)

\pkgcls@show@selection This command records what selection was made. As that is needed in two places (and it is rather lengthy) it was placed in a separate command. The first argument is the name of the external file that is being loaded and is only needed for debugging. The second argument is the date that corresponds to this file and it is used as part of the message.

```
\def\pkgcls@show@selection#1#2{%
⟨*tracerollback⟩
  \pkgcls@debug{Result: use  #1}%
⟨/tracerollback⟩
  \GenericInfo
   {\@spaces\@spaces\space}{Rollback for
    \@cls@pkg\space'\@currname' requested ->
    \ifnum\pkgcls@targetdate>\@ne
       date
       \ifnum\requestedLaTeXdate=\pkgcls@targetdate
          \requestedpatchdate
       \else
          \expandafter\@gobble\pkgcls@arg
       \fi.\MessageBreak
```

Instead of "best approximation" we could say that we have been able to exactly match the date (if it is exact), but that would mean extra tests without much gain, so not done.

```
       Best approximation is
    \else
       version '\pkgcls@targetlabel'.\MessageBreak
       This corresponds to
    \fi
    \ifx\@nil#2\@nil
       a special release%
    \else
```

[52] The older release may also have such declarations inside if it was a simply copy of the .sty or .cls file current at that date. Removing these declarations would make the file load a tiny bit faster, but this way it works in any case.

```
        the release introduced on #2%
    \fi
    \@gobble}%
}
```

(*End of definition for* `\pkgcls@show@selection`.)

`\pkgcls@rollbackdate@error` This is called if the requested rollback date is earlier than the earliest known release of a package or class.

A similar error is given if global rollback date and min-date on a specific package conflict with each other, but that case is happens only once so it is inlined.

```
\def\pkgcls@rollbackdate@error#1{%
  \@latex@error{Suspicious rollback date given}%
     {The \@cls@pkg\space'\@currname'  has no rollback data
      before #1 which\MessageBreak
      is after your requested rollback date --- so
      something may be wrong here.\MessageBreak
      Continue and we use the earliest known release.}}
```

(*End of definition for* `\pkgcls@rollbackdate@error`.)

`\DeclareCurrentRelease` This declares the date (and possible name) of the current version of a package or class.

```
\def\DeclareCurrentRelease#1#2{%
```

First we test if `\pkgcls@targetdate` is greater than zero, otherwise this code is bypassed (as there is no rollback request).

```
  \ifnum\pkgcls@targetdate>\z@  % some sort of rollback request
⟨*tracerollback⟩
    \pkgcls@debug{---DeclareCurrentRelease}%
    \pkgcls@debug{   1: #1}%
    \pkgcls@debug{   2: #2}%
⟨/tracerollback⟩
```

If the value is greater than 1 we have to deal with a date request, so we parse `#2` as a date and compare it with `\pkgcls@targetdate`.

```
    \ifnum\pkgcls@targetdate>\@ne  % a date request
      \ifnum\@parse@version#2//00\@nil
           >\pkgcls@targetdate
```

If it is greater that means the release date if this file is later than the requested rollback date. Again we have two cases: If there was a previous candidate release we use that one as the current release is too young, but if there wasn't we have to use this release nevertheless as there isn't any alternative.

However this case can only happen if there is a `\DeclareCurrentRelease` but no declared older releases (so basically the use of the declaration is a bit dubious).

```
        \ifx\pkgcls@candidate\@empty
          \pkgcls@rollbackdate@error{#2}%
        \else
          \pkgcls@use@this@release\pkgcls@candidate
                                  \pkgcls@releasedate
        \fi
```

Otherwise the current file is the right release, so we record that in the transcript and then carry on.

```
      \else
        \pkgcls@show@selection{current version}{#2}%
      \fi
    \else % a label request
```

Otherwise we have a rollback request to a named version so we check if that fits the current name and if not give an error as this was the last possible opportunity.

```
      \def\reserved@a{#1}%
      \ifx\pkgcls@targetlabel\reserved@a
        \pkgcls@show@selection{current version}{#2}%
      \else
        \@latex@error{Requested version '\pkgcls@targetlabel' for
          \@cls@pkg\space'\@currname' is unknown}\@ehc
      \fi
    \fi
  \fi
}
```

(*End of definition for* `\DeclareCurrentRelease`.)

`\IfTargetDateBefore` This enables a simple form of conditional code inside a class or package file. If there is a date request and the request date is earlier than the first argument the code in the second argument is processed otherwise the code in the third argument is processed. If there was no date request then we also execute the third argument, i.e., we will get the "latest" version of the file.

Most often the second argument (before-date-code) will be empty.

```
\DeclareRobustCommand\IfTargetDateBefore[1]{%
  \ifnum\pkgcls@innerdate <%
        \expandafter\@parse@version\expandafter0#1//00\@nil
    \typeout{Exclude code introduced on #1}%
    \expandafter\@firstoftwo
  \else
    \typeout{Include code introduced on #1}%
    \expandafter\@secondoftwo
  \fi
}
```

(*End of definition for* `\IfTargetDateBefore`.)

```
⟨/2ekernel | latexreleasefirst⟩
```

6 After Preamble

Finally we declare a package that allows all the commands declared above to be `\@onlypreamble` to be used after `\begin{document}`.

```
⟨*afterpreamble⟩
\NeedsTeXFormat{LaTeX2e}
\ProvidesPackage{pkgindoc}
          [2020-08-08 v1.3m Package Interface in Document (DPC)]
\def\reserved@a#1\do\@classoptionslist#2\do\filec@ntents#3\relax{%
  \gdef\@preamblecmds{#1#3}}
```

```
\expandafter\reserved@a\@preamblecmds\relax
⟨/afterpreamble⟩
```

File 51
ltkeys.dtx

1 Creating and using keyval options

As with any key–value input, using key–value pairs as package or class options has two parts: creating the key options and setting (using) them. Options created in this way *may* be used after package loading as general key–value settings: this will depend on the nature of the underlying code.

\DeclareKeys `\DeclareKeys [⟨family⟩] {⟨declarations⟩}`

Creates a series of options from a comma-separated ⟨*declarations*⟩ list. The ⟨*family*⟩ is a namespace for the keys; if not given, the basename of the current class or package is used. Each entry in the ⟨*declarations*⟩ (a commas-separated list) is a key–value pair, with the ⟨*key*⟩ having one or more ⟨*properties*⟩. A small number of "basic" ⟨*properties*⟩ are described below. The full range of properties, provided by l3keys, can also be used for more powerful processing. See `interface3` for the full details.

The basic properties provided here are

- `.code` — execute arbitrary code
- `.if` — sets a TEX `\if...` switch
- `.ifnot` — sets an inverted TEX `\if...` switch
- `.pass-to-packages` — for class options, this specifies whether the option should be treated "global" (read by packages from the global list); for package options this property has no effect
- `.store` — stores a value in a macro
- `.usage` – defines whether the option can be given only when loading (`load`), in the preamble (`preamble`) or has no limitation on scope (`general`)

The part of the ⟨*key*⟩ before the ⟨*property*⟩ is the ⟨*name*⟩, with the ⟨*value*⟩ working with the ⟨*property*⟩ to define the behaviour of the option.

For example, with

```
\DeclareKeys[mypkg]
  {
    draft.if          = @mypkg@draft      ,
    draft.usage       = preamble          ,
    name.store        = \@mypkg@name      ,
    name.usage        = load              ,
    second-name.store = \@mypkg@other@name
  }
```

three options would be create. The option `draft` can be given anywhere in the preamble, and will set a switch called `\if@mypkg@draft`. The option `name` can only be given during package loading, and will save whatever value it is given in `\@mypkg@name`. Finally, the option `second-name` can be given anywhere, and will save its value in `\@mypkg@other@name`.

Keys created *before* the use of `\ProcessKeyOptions` act as package options.

\DeclareUnknownKeyHandler `\DeclareUnknownKeyHandler [⟨family⟩] {⟨code⟩}`

The function `\DeclareUnknownKeyHandler` may be used to define the behavior when an undefined key is encountered. The ⟨*code*⟩ will receive the unknown key name as `#1` and the value as `#2`. These can then be processed as appropriate, e.g. by forwarding to another package.

\ProcessKeyOptions `\ProcessKeyOptions [⟨families⟩]`

The `\ProcessKeyOptions` function is used to check the current option list against the keys defined for ⟨*families*⟩ (a comma-separated list). Global (class) options and local (package) options are checked when this function is called in a package. Where there is more than one ⟨*family*⟩ in the list of ⟨*families*⟩, processing takes place in the order of the ⟨*families*⟩. Every key known in a ⟨*family*⟩ is passed for processing, thus a key defined in more than one ⟨*family*⟩ will be processed several times, and if the code paths are shared, the outcome will be determined by the last entry in the list of ⟨*families*⟩.

\SetKeys `\SetKeys [⟨family⟩] {⟨keyvals⟩}`

Sets (applies) the explicit list of ⟨*keyvals*⟩ for the ⟨*family*⟩: it the latter is not given, the value of `\@currname` used. This command may be used within a package to set options before or after using `\ProcessKeyOptions`.

1.1 Implementation of ltkeys

```
⟨@@=keys⟩
⟨*2ekernel⟩
\ExplSyntaxOn
```

1.2 Key properties

.code
.if
.ifnot
.store
.usage

```
\group_begin:
  \cs_set_protected:Npn \__keys_tmp:nn #1#2
    {
      \quark_if_recursion_tail_stop:n {#1}
      \cs_new_eq:cc
        { \c__keys_props_root_str . #2 }
        { \c__keys_props_root_str . #1 }
      \__keys_tmp:nn
    }
  \__keys_tmp:nn
    { code:n }          { code }
    { legacy_if_set:n } { if }
    { legacy_if_set_inverse:n } { ifnot }
    { tl_set:N }        { store }
    { usage:n }         { usage }
    { \q_recursion_tail } { }
    \q_recursion_stop
\group_end:
```

(*End of definition for* `.code` *and others.*)

.pass-to-packages
__keys_scope:n
__keys_scope:N

Used to force options to be global: as this property (uniquely) has an *optional* value, there is a bit of work to do.

```
\cs_new_protected:cpn { \c__keys_props_root_str .pass-to-packages }
  {
    \bool_if:NTF \l__keys_no_value_bool
      { \__keys_scope:n { true } }
      { \__keys_scope:n }
```

```
  }
\cs_new_protected:Npn \__keys_scope:n #1
  {
    \str_case:nnF {#1}
      {
        { true }
          { \__keys_scope:N \clist_put_right:NV }
        { false }
          { \__keys_scope:N \clist_remove_all:NV }
      }
      {
        \msg_error:nnnn { keys }
          { choice-unknown }
          { .pass-to-packages }
          {#1}
      }
  }
\cs_new_protected:Npn \__keys_scope:N #1
  {
    \exp_after:wN \__keys_find_key_module:wNN
      \l_keys_path_str \s__keys_stop
        \l_keys_key_tl \l_keys_key_str
    #1 \l__keys_forced_global_clist \l_keys_key_str
  }
```

(*End of definition for* `.pass-to-packages`, `\__keys_scope:n`, *and* `\__keys_scope:N`.)

1.3 Main mechanism

```
\cs_generate_variant:Nn \clist_if_in:NnT { Ne }
\cs_generate_variant:Nn \clist_if_in:NnTF { Ne }
```

`\l__keys_class_only_clist` Used to track class-only options.

```
\clist_new:N \l__keys_class_only_clist
```

(*End of definition for* `\l__keys_class_only_clist`.)

`\l__keys_forced_global_clist` Used to force options to be global.

```
\clist_new:N \l__keys_forced_global_clist
```

(*End of definition for* `\l__keys_forced_global_clist`.)

`\l__keys_options_clist` A single list is used for all options, into which they are collected before processing.

```
\clist_new:N \l__keys_options_clist
```

(*End of definition for* `\l__keys_options_clist`.)

`\l__keys_local_clist` Holds the local options when appropriate: otherwise empty. Needed as the LaTeX 2_ε setup here can be equal to `\scan_stop:`, which is not generally supported.

```
\clist_new:N \l__keys_local_clist
```

(*End of definition for* `\l__keys_local_clist`.)

`\l__keys_options_loading_bool`

Used to indicate we are in the loading phase: controls the outcome of warnings.

```
\bool_new:N \l__keys_options_loading_bool
```

__keys_options:n
__keys_options_aux:n

The main function calls functions to collect up the global and local options into `\l__keys_options_clist` before calling the underlying functions to actually do the processing. So that a suitable message is produced if the option is unknown, the special `unknown` key is set if it does not already exist for the current family, and is cleaned up afterwards if required. To allow the LaTeX 2_ε layer to know this mechanism is active, and to deal with the key family not matching the file name, we store the family in all cases. Global options are only considered the first time a package is loaded: this is tracked using `opt@handler@\@currname.\@currext`, as this is defined once keyval processing has been applied for the first time.

```
\cs_new_protected:Npn \__keys_options:n #1
  { \__keys_options_expand_module:Nn \__keys_options_aux:n {#1} }
\cs_new_protected:Npn \__keys_options_aux:n #1
  {
    \__keys_options_local:
    \clist_map_inline:nn {#1}
      {
        \clist_clear:N \l__keys_options_clist
        \cs_if_exist:cF { opt@handler@ \@currname . \@currext }
          { \__keys_options_global:n {##1} }
        \__keys_options_local:n {##1}
        \bool_set_true:N \l__keys_options_loading_bool
        \clist_map_variable:NNn \l__keys_options_clist \CurrentOption
          { \keys_set:nV {##1} \CurrentOption }
        \bool_set_false:N \l__keys_options_loading_bool
        \__keys_options_loaded:n {##1}
      }
    \clist_if_empty:NF \l__keys_unused_clist
      {
        \clist_map_inline:Nn \l__keys_unused_clist
          {
            \msg_error:nnee { keys } { option-unknown }
              {##1} { \@currname }
          }
      }
    \cs_if_exist:cF { opt@handler@ \@currname . \@currext }
      {
        \cs_gset_protected:cpn { opt@handler@ \@currname . \@currext }
          { \ProcessKeyOptions [ {#1} ] }
      }
    \AtEndOfPackage { \cs_set_eq:NN \@unprocessedoptions \scan_stop: }
  }

\msg_new:nnnn { keys } { option-unknown }
  { Unknown~option~'#1'~for~package~#2. }
  {
    LaTeX~has~been~asked~to~set~an~option~called~'#1'~
    but~the~package~"\msg_module_name:n {#2}"~has~not~created~an~option~with~this~name.
  }
```

(*End of definition for* `\__keys_options:n` *and* `\__keys_options_aux:n`.)

__keys_options_global:n

Global (class) options are handled differently for LaTeX 2_ε packages and classes. Hence this function is essentially a check on the current file type. The initial test is needed as

LaTeX 2_ε allows variables to be equal to `\scan_stop:`, which is usually forbidden in expl3 code.

```
\cs_new_protected:Npn \__keys_options_global:n #1
  {
    \cs_if_eq:NNF \@raw@classoptionslist \scan_stop:
      {
        \cs_if_eq:NNTF \@currext \@clsextension
          { \__keys_options_class:n {#1} }
          { \__keys_options_package:n {#1} }
      }
  }
```

(*End of definition for* `\__keys_options_global:n`.)

`\__keys_options_class:n`
`\__keys_options_class:nnn`
`\__keys_options_class:nn`

For classes, each option (stripped of any content after `=`) is checked for existence as a key. If found, the option is added to the combined list for processing. On the other hand, unused options are stored up in `\@unusedoptionlist`. An earlier version of this code checked for the `unknown` key just once and if found short-cutted the loop: that though makes handling more complex situations harder, so we take the performance hit instead. Options used by classes are tracked but the catch-all `unknown` is excluded (hence not using a lazy evaluation for the key testing).

```
\cs_new_protected:Npn \__keys_options_class:n #1
  {
    \cs_if_free:cF { @raw@opt@ \@currname . \@currext }
      {
        \clist_map_inline:cn { @raw@opt@ \@currname . \@currext }
          {
            \exp_args:Ne \__keys_options_class:nnn
              { \tl_trim_spaces:e { \__keys_remove_equals:n {##1} } }
                {##1} {#1}
          }
      }
  }
\cs_new_protected:Npn \__keys_options_class:nnn #1#2#3
  {
    \keys_if_exist:nnTF {#3} {#1}
      {
        \__keys_options_class:nn {#1} {#2}
        \clist_put_right:Ne \l__keys_class_only_clist { \tl_to_str:n {#1} }
      }
      {
        \keys_if_exist:nnTF {#3} { unknown }
          { \__keys_options_class:nn {#1} {#2} }
          {
            \clist_if_in:NnF \@unusedoptionlist {#1}
              { \clist_put_right:Nn \@unusedoptionlist {#1} }
          }
      }
  }
\cs_new_protected:Npn \__keys_options_class:nn #1#2
  {
    \clist_remove_all:Nn \@unusedoptionlist {#1}
    \clist_put_right:Nn \l__keys_options_clist {#2}
  }
```

(*End of definition for* `\__keys_options_class:n`, `\__keys_options_class:nnn`, *and* `\__keys_options_class:nn`.)

`\__keys_options_package:n`
`\__keys_options_package:nnn`
`\__keys_options_package:nn`

For global options when processing a package, the tasks are slightly different from those for a class. The check is the same, but here there is nothing to do if the option is not applicable. Each valid option also needs to be removed from `\@unusedoptionlist`.

```
\cs_new_protected:Npn \__keys_options_package:n #1
  {
    \clist_map_inline:Nn \@raw@classoptionslist
      {
        \exp_args:Ne \__keys_options_package:nnn
          { \tl_trim_spaces:e { \__keys_remove_equals:n {##1} } }
          {##1} {#1}
      }
  }
```

The forced-global test here needs to use `\tl_to_str:n` as the data come from a key name, which is always a string.

```
\cs_new_protected:Npn \__keys_options_package:nnn #1#2#3
  {
    \keys_if_exist:nnT {#3} {#1}
      {
        \clist_if_in:NeTF \l__keys_class_only_clist { \tl_to_str:n {#1} }
          {
            \clist_if_in:NeT \l__keys_forced_global_clist { \tl_to_str:n {#1} }
              { \__keys_options_package:nn {#1} {#2} }
          }
          { \__keys_options_package:nn {#1} {#2} }
      }
   }
\cs_new_protected:Npn \__keys_options_package:nn #1#2
  {
    \clist_put_right:Nn \l__keys_options_clist {#2}
    \clist_remove_all:Nn \@unusedoptionlist {#1}
  }
```

(*End of definition for* `\__keys_options_package:n`, `\__keys_options_package:nnn`, *and* `\__keys_options_package:nn`.)

`\__keys_options_local:`

If local options are found, they are added to the processing list. LaTeX 2_ε stores options for each file in a macro which may or may not exist, hence the need to use `\cs_if_exist:c`. For tracking the unused options, we need to hold only the option name, hence needing a mapping.

```
\cs_new_protected:Npn \__keys_options_local:
  {
    \clist_clear:N \l__keys_local_clist
    \cs_if_eq:NNF \@currext \@clsextension
      {
        \cs_if_exist:cT { @raw@opt@ \@currname . \@currext }
          {
            \clist_set_eq:Nc \l__keys_local_clist
              { @raw@opt@ \@currname . \@currext }
          }
      }
```

```
    \clist_map_inline:Nn \l__keys_local_clist
      {
        \clist_put_right:Ne \l__keys_unused_clist
          { \tl_trim_spaces:e { \__keys_remove_equals:n {##1} } }
      }
  }
```

(*End of definition for* `\__keys_options_local:`.)

`\__keys_options_local:n` `\__keys_options_local:nnn` A simpler version of the code used for package options: we stack up those that are known, unless the family supports unknown keys in which case we hover up everything.

```
\cs_new_protected:Npn \__keys_options_local:n #1
  {
    \keys_if_exist:nnTF {#1} { unknown }
      {
        \clist_put_right:NV \l__keys_options_clist \l__keys_local_clist
        \clist_clear:N \l__keys_unused_clist
      }
      {
        \clist_map_inline:Nn \l__keys_local_clist
          {
            \exp_args:Ne \__keys_options_local:nnn
              { \tl_trim_spaces:e { \__keys_remove_equals:n {##1} } }
              {##1} {#1}
          }
      }
  }
\cs_new_protected:Npn \__keys_options_local:nnn #1#2#3
  {
    \keys_if_exist:nnT {#3} {#1}
      {
        \clist_put_right:Nn \l__keys_options_clist {#2}
        \clist_remove_all:Nn \l__keys_unused_clist {#1}
      }
   }
```

(*End of definition for* `\__keys_options_local:n` *and* `\__keys_options_local:nnn`.)

`\__keys_remove_equals:n` `\__keys_remove_equals:w` As the name suggests, this is a simple function to remove an equals sign from the input. This is all wrapped up in an `n` function so that there will always be a sign available.

```
\cs_new:Npn \__keys_remove_equals:n #1
  { \__keys_remove_equals:w #1 = \s__keys_stop }
\cs_new:Npn \__keys_remove_equals:w #1 = #2 \s__keys_stop { \exp_not:n {#1} }
```

(*End of definition for* `\__keys_remove_equals:n` *and* `\__keys_remove_equals:w`.)

1.4 The document interfaces

`\__keys_options_expand_module:Nn` `\__keys_options_expand_module:nN` To deal with active characters inside the module argument whilst also expanding that argument, we use a combination of `c`- and `f`-type expansion. This works as the definitions for active UTF-8 bytes contain an `\ifincsname` test.

```
\cs_new_protected:Npn \__keys_options_expand_module:Nn #1#2
  {
    \cs:w __keys_options_expand_module:nN \use:e { \cs_end: {#2} } #1
```

```
  }
\cs_new_protected:Npn \__keys_options_expand_module:nN #1#2
  { #2 {#1} }
```

(*End of definition for* `\__keys_options_expand_module:Nn` *and* `\__keys_options_expand_module:nN`.)

`\DeclareKeys` Defining key options is quite straight-forward: we have an intermediate function to allow for potential set-up steps.

```
\NewDocumentCommand \DeclareKeys { O { \@currname } +m }
  { \__keys_options_expand_module:Nn \keys_define:nn {#1} {#2} }
```

(*End of definition for* `\DeclareKeys`. *This function is documented on page* 1150.)

`\DeclareUnknownKeyHandler`

```
\NewDocumentCommand \DeclareUnknownKeyHandler { O { \@currname } +m }
  {
    \cs_set_protected:cpn { __keys_unknown_handler_ #1 :nn } ##1##2 {#2}
    \__keys_options_expand_module:Nn \keys_define:ne {#1}
      {
        unknown .code:n =
          \exp_not:N \exp_args:NV
            \exp_not:c { __keys_unknown_handler_ #1 :nn }
            \exp_not:N \l_keys_key_str {##1}
      }
  }
```

(*End of definition for* `\DeclareUnknownKeyHandler`. *This function is documented on page* 1150.)

`\ProcessKeyOptions` We need to deal with the older interface from l3keys2e here: it had a mandatory argument. We can mop that up using a look-ahead, and then exploit that information to determine whether the package option handling is set up for the new approach for clash handling.

```
\NewDocumentCommand \ProcessKeyOptions { O { \@currname } }
  { \__keys_options:n {#1} }
\@onlypreamble \ProcessKeyOptions
```

(*End of definition for* `\ProcessKeyOptions`. *This function is documented on page* 1151.)

1.5 Option usage scope

`\__keys_options_loaded:n` `\__keys_options_loaded:nn` Indicates that the load-time options for a package have been processed: once this has happened, make them unavailable either with a warning or an error.

```
\cs_new_protected:Npn \__keys_options_loaded:n #1
  {
    \prop_get:NnNT \l_keys_usage_load_prop {#1} \l__keys_tmpa_tl
      {
        \clist_map_inline:Nn \l__keys_tmpa_tl
          {
            \keys_define:nn {#1}
              {
                ##1 .code:n =
                  \__keys_options_loaded:nn {#1} {##1}
              }
          }
      }
```

```
  }
\cs_new_protected:Npn \__keys_options_loaded:nn #1#2
  {
    \bool_if:NTF \l__keys_options_loading_bool
      { \msg_warning:nnnn { keys } { load-option-ignored } }
      { \msg_error:nnnn { keys } { load-only } }
        {#1} {#2}
  }

\msg_new:nnn { keys } { load-option-ignored }
  {
    Package~"\msg_module_name:n {#1}"~has~already~been~loaded:~
    ignoring~load-time~option~"#2".
  }

\msg_new:nnnn { keys } { load-only }
  {
    Key~"#2"~may~only~be~used~during~loading~of~package~
    "\msg_module_name:n {#1}".
  }
  {
    LaTeX~was~asked~to~set~a~key~called~"#2",~but~this~is~only~allowed~
    in~the~optional~argument~when~loading~package~"\msg_module_name:n{#1}".
  }
```

(*End of definition for* `\__keys_options_loaded:n` *and* `\__keys_options_loaded:nn`.)

Disable all preamble options in one shot.

```
\tl_gput_left:Nn \@kernel@after@begindocument
  {
    \prop_map_inline:Nn \l_keys_usage_preamble_prop
      {
        \clist_map_inline:nn {#2}
          {
            \keys_define:nn {#1}
              {
                ##1 .code:n =
                  \msg_error:nnn { keys } { preamble-only } {##1}
              }
          }
      }
  }
\msg_new:nnnn { keys } { preamble-only }
  { Key~"#1"~may~only~be~used~in~the~preamble. }
  {
    LaTeX~was~asked~to~set~a~key~called~"#1",~but~this~is~only~allowed~
    before~\begin{document}.~You~will~need~to~set~the~key~earlier.
  }
```

1.6 General key setting

`\SetKeys` A simple wrapper.

```
\NewDocumentCommand \SetKeys { O { \@currname } +m }
  { \__keys_options_expand_module:Nn \keys_set:nn {#1} {#2} }
```

(*End of definition for* `\SetKeys`. *This function is documented on page 1151.*)

```
\ExplSyntaxOff
⟨/2ekernel⟩
```

File 52
ltfilehook.dtx

1 Introduction

1.1 Provided hooks

The code offers a number of hooks into which packages (or the user) can add code to support different use cases. Many hooks are offered as pairs (i.e., the second hook is reversed. Also important to know is that these pairs are properly nested with respect to other pairs of hooks.

There are hooks that are executed for all files of a certain type (if they contain code), e.g., for all "include files" or all "packages", and there are also hooks that are specific to a single file, e.g., do something after the package `foo.sty` has been loaded.

1.2 General hooks for file reading

There are four hooks that are called for each file that is read using document-level commands such as `\input`, `\include`, `\usepackage`, etc. They are not called for files read using internal low-level methods, such as `\@input` or `\openin`.

`file/before`
`file/.../before`
`file/.../after`
`file/after`

These are:

`file/before`, `file/`⟨*file-name*⟩`/before` These hooks are executed in that order just before the file is loaded for reading. The code of the first hook is used with every file, while the second is executed only for the file with matching ⟨*file-name*⟩ allowing you to specify code that only applies to one file.

`file/`⟨*file-name*⟩`/after`, `file/after` These hooks are executed after the file with name ⟨*file-name*⟩ has been fully consumed. The order is swapped (the specific one comes first) so that the `/before` and `/after` hooks nest properly, which is important if any of them involve grouping (e.g., contain environments, for example). Furthermore both hooks are reversed hooks to support correct nesting of different packages adding code to both `/before` and `/after` hooks.

So the overall sequence of hook processing for any file read through the user interface commands of LaTeX is:

```
\UseHook{file/before}
\UseHook{file/⟨file name⟩/before}
   ⟨file contents⟩
\UseHook{file/⟨file name⟩/after}
\UseHook{file/after}
```

The file hooks only refer to the file by its name and extension, so the ⟨*file name*⟩ should be the file name as it is on the filesystem with extension (if any) and without paths. Different from `\input` and similar commands, the `.tex` extension is not assumed in hook ⟨*file name*⟩, so `.tex` files must be specified with their extension to be recognized. Files within subfolders should also be addressed by their name and extension only.

Extensionless files also work, and should then be given without extension. Note however that TEX prioritizes `.tex` files, so if two files `foo` and `foo.tex` exist in the search path, only the latter will be seen.

When a file is input, the ⟨*file name*⟩ is available in `\CurrentFile`, which is then used when accessing the `file/`⟨*file name*⟩`/before` and `file/`⟨*file name*⟩`/after`.

`\CurrentFile`

The name of the file about to be read (or just finished) is available to the hooks through `\CurrentFile` (there is no `expl3` name for it for now). The file is always provided with its extension, i.e., how it appears on your hard drive, but without any specified path to it. For example, `\input{sample}` and `\input{app/sample.tex}` would both have `\CurrentFile` being `sample.tex`.

`\CurrentFilePath`

The path to the current file (complement to `\CurrentFile`) is available in `\CurrentFilePath` if needed. The paths returned in `\CurrentFilePath` are only user paths, given through `\input@path` (or `expl3`'s equivalent `\l_file_search_path_seq`) or by directly typing in the path in the `\input` command or equivalent. Files located by `kpsewhich` get the path added internally by the TEX implementation, so at the macro level it looks as if the file were in the current folder, so the path in `\CurrentFilePath` is empty in these cases (package and class files, mostly).

`\CurrentFileUsed`
`\CurrentFilePathUsed`

In normal circumstances these are identical to `\CurrentFile` and `\CurrentFilePath`. They will differ when a file substitution has occurred for `\CurrentFile`. In that case, `\CurrentFileUsed` and `\CurrentFilePathUsed` will hold the actual file name and path loaded by LATEX, while `\CurrentFile` and `\CurrentFilePath` will hold the names that were *asked for*. Unless doing very specific work on the file being read, `\CurrentFile` and `\CurrentFilePath` should be enough.

1.3 Hooks for package and class files

Commands to load package and class files (e.g., `\usepackage`, `\RequirePackage`, `\LoadPackageWithOptions`, etc.) offer the hooks from section 1.2 when they are used to load a package or class file, e.g., `file/array.sty/after` would be called after the `array` package got loaded. But as packages and classes form as special group of files, there are some additional hooks available that only apply when a package or class is loaded.

`package/before`
`package/after`
`package/.../before`
`package/.../after`
`class/before`
`class/after`
`class/.../before`
`class/.../after`

These are:

`package/before`, `package/after` These hooks are called for each package being loaded.

`package/`⟨*name*⟩`/before`, `package/`⟨*name*⟩`/after` These hooks are additionally called if the package name is ⟨*name*⟩ (without extension).

`class/before`, `class/after` These hooks are called for each class being loaded.

`class/`⟨*name*⟩`/before`, `class/`⟨*name*⟩`/after` These hooks are additionally called if the class name is ⟨*name*⟩ (without extension).

All `/after` hooks are implemented as reversed hooks.

The overall sequence of execution for `\usepackage` and friends is:

```
\UseHook{package/before}
\UseOneTimeHook{package/⟨package name⟩/before}
    \UseHook{file/before}
    \UseHook{file/⟨package name⟩.sty/before}
       ⟨package contents⟩
    \UseHook{file/⟨package name⟩.sty/after}
    \UseHook{file/after}
    code from \AtEndOfPackage if used inside the package
\UseOneTimeHook{package/⟨package name⟩/after}
\UseHook{package/after}
```

and similar for class file loading, except that `package/` is replaced by `class/` and `\AtEndOfPackage` by `\AtEndOfClass`.

If a package or class is not loaded none of the hooks are executed!

All class or package hooks involving the name of the class or package are implemented as one-time hooks, whereas all other such hooks are normal hooks. This allows for the following use case

```
\AddToHook{package/varioref/after}
      { ... apply my customizations if the package gets
        loaded (or was loaded already) ... }
```

without the need to first test if the package is already loaded.

1.4 Hooks for `\include` files

To manage `\include` files, LaTeX issues a `\clearpage` before and after loading such a file. Depending on the use case one may want to execute code before or after these `\clearpage`s especially for the one that is issued at the end.

Executing code before the final `\clearpage`, means that the code is processed while the last page of the included material is still under construction. Executing code after it means that all floats from inside the include file are placed (which might have added further pages) and the final page has finished.

Because of these different scenarios we offer hooks in three places.[53] None of the hooks are executed when an `\include` file is bypassed because of an `\includeonly` declaration. They are, however, all executed if LaTeX makes an attempt to load the `\include` file (even if it doesn't exist and all that happens is "`No file ⟨filename⟩.tex`").

[53] If you want to execute code before the first `\clearpage` there is no need to use a hook—you can write it directly in front of the `\include`.

include/before
include/.../before
include/end
include/.../end
include/after
include/.../after

These are:

include/before, include/⟨*name*⟩/before These hooks are executed (in that order) after the initial `\clearpage` and after `.aux` file is changed to use ⟨*name*⟩`.aux`, but before the ⟨*name*⟩`.tex` file is loaded. In other words they are executed at the very beginning of the first page of the `\include` file.

include/⟨*name*⟩/end, include/end These hooks are executed (in that order) after LaTeX has stopped reading from the `\include` file, but before it has issued a `\clearpage` to output any deferred floats.

include/⟨*name*⟩/after, include/after These hooks are executed (in that order) after LaTeX has issued the `\clearpage` but before is has switched back writing to the main `.aux` file. Thus technically we are still inside the `\include` and if the hooks generate any further typeset material including anything that writes to the `.aux` file, then it would be considered part of the included material and bypassed if it is not loaded because of some `\includeonly` statement.[54]

include/excluded, include/⟨*name*⟩/excluded The above hooks for `\include` files are only executed when the file is loaded (or more exactly the load is attempted). If, however, the `\include` file is explicitly excluded (through an `\includeonly` statement) the above hooks are bypassed and instead the `include/excluded` hook followed by the `include/`⟨*name*⟩`/excluded` hook are executed. This happens after LaTeX has loaded the `.aux` file for this include file, i.e., after LaTeX has updated its counters to pretend that the file was seen.

All `include` hooks involving the name of the included file are implemented as one-time hooks (whereas all other such hooks are normal hooks).

If you want to execute code that is run for every `\include` regardless of whether or not it is excluded, use the `cmd/include/before` or `cmd/include/after` hooks.

1.5 High-level interfaces for LaTeX

We do not provide any additional wrappers around the hooks (like filehook or scrlfile do) because we believe that for package writers the high-level commands from the hook management, e.g., `\AddToHook`, etc. are sufficient and in fact easier to work with, given that the hooks have consistent naming conventions.

[54] For that reason another `\clearpage` is executed after these hooks which normally does nothing, but starts a new page if further material got added this way.

1.6 Kernel, class, and package interfaces for LaTeX

\declare@file@substitution `\declare@file@substitution {⟨file⟩} {⟨replacement-file⟩}`
\undeclare@file@substitution `\undeclare@file@substitution {⟨file⟩}`

If ⟨*file*⟩ is requested for loading replace it with ⟨*replacement-file*⟩. `\CurrentFile` remains pointing to ⟨*file*⟩ but `\CurrentFileUsed` will show the file actually loaded.

The main use case for this declaration is to provide a corrected version of a package that can't be changed (due to its license) but no longer functions because of LaTeX kernel changes, for example, or to provide a version that makes use of new kernel functionality while the original package remains available for use with older releases. As such it is mainly meant for use in the LaTeX kernel but other use cases are conceivable.

The `\undeclare@file@substitution` declaration undoes a substitution made earlier.

> *Please do not misuse this functionality and replace a file with another unless if really needed and only if the new version is implementing the same functionality as the original one!*

\disable@package@load `\disable@package@load {⟨package⟩} {⟨alternate-code⟩}`
\reenable@package@load `\reenable@package@load {⟨package⟩}`

If ⟨*package*⟩ is requested, do not load it but instead run ⟨*alternate-code*⟩ which could issue a warning, error or any other code.

The main use case is for classes that want to restrict the set of supported packages or contain code that make the use of some packages impossible. So rather than waiting until the document breaks they can set up informative messages why certain packages are not available.

The function is only implemented for packages not for arbitrary files and again it should only be applied if there are good reasons for doing this.[55]

1.7 A sample package for structuring the log output

As an application we provide the package structuredlog that adds lines to the `.log` when a file is opened and closed for reading keeping track of nesting level es well. For example, for the current document it adds the lines

```
= (LEVEL 1 START) t1lmr.fd
= (LEVEL 1 STOP) t1lmr.fd
= (LEVEL 1 START) supp-pdf.mkii
= (LEVEL 1 STOP) supp-pdf.mkii
= (LEVEL 1 START) nameref.sty
== (LEVEL 2 START) refcount.sty
== (LEVEL 2 STOP) refcount.sty
== (LEVEL 2 START) gettitlestring.sty
== (LEVEL 2 STOP) gettitlestring.sty
= (LEVEL 1 STOP) nameref.sty
= (LEVEL 1 START) ltfilehook-doc.out
```

[55] Just to be sure: "I don't like this package by somebody else" is not a good one :-)

```
= (LEVEL 1 STOP) ltfilehook-doc.out
= (LEVEL 1 START) ltfilehook-doc.out
= (LEVEL 1 STOP) ltfilehook-doc.out
= (LEVEL 1 START) ltfilehook-doc.hd
= (LEVEL 1 STOP) ltfilehook-doc.hd
= (LEVEL 1 START) ltfilehook.dtx
== (LEVEL 2 START) ot1lmr.fd
== (LEVEL 2 STOP) ot1lmr.fd
== (LEVEL 2 START) omllmm.fd
== (LEVEL 2 STOP) omllmm.fd
== (LEVEL 2 START) omslmsy.fd
== (LEVEL 2 STOP) omslmsy.fd
== (LEVEL 2 START) omxlmex.fd
== (LEVEL 2 STOP) omxlmex.fd
== (LEVEL 2 START) umsa.fd
== (LEVEL 2 STOP) umsa.fd
== (LEVEL 2 START) umsb.fd
== (LEVEL 2 STOP) umsb.fd
== (LEVEL 2 START) ts1lmr.fd
== (LEVEL 2 STOP) ts1lmr.fd
== (LEVEL 2 START) t1lmss.fd
== (LEVEL 2 STOP) t1lmss.fd
= (LEVEL 1 STOP) ltfilehook.dtx
```

Thus if you inspect an issue in the `.log` it is easy to figure out in which file it occurred, simply by searching back for `LEVEL` and if it is a `STOP` then remove 1 from the level value and search further for `LEVEL` with that value which should then be the `START` level of the file you are in.

2 The Implementation

1 ⟨*2ekernel⟩

2 ⟨@@=filehook⟩

2.1 Document and package-level commands

\CurrentFile
\CurrentFilePath
\CurrentFileUsed
\CurrentFilePathUsed

User-level macros that hold the current file name and file path. These are used internally as well because the code takes care to protect against a possible redefinition of these macros in the loaded file (it's necessary anyway to make hooks work with nested `\input`). The versions `\...Used` hold the *actual* file name and path that is loaded by LaTeX, whereas the other two hold the name as requested. They will differ in case there's a file substitution.

3 ⟨/2ekernel⟩

4 ⟨*2ekernel | latexrelease⟩

5 ⟨latexrelease⟩\IncludeInRelease{2020/10/01}%

6 ⟨latexrelease⟩ {\CurrentFile}{Hook management file}%

```
7  \ExplSyntaxOn
8  \tl_new:N \CurrentFile
9  \tl_new:N \CurrentFilePath
10 \tl_new:N \CurrentFileUsed
11 \tl_new:N \CurrentFilePathUsed
```

```
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\CurrentFile}{Hook management file}%
⟨latexrelease⟩
⟨latexrelease⟩\let \CurrentFile          \@undefined
⟨latexrelease⟩\let \CurrentFilePath      \@undefined
⟨latexrelease⟩\let \CurrentFileUsed      \@undefined
⟨latexrelease⟩\let \CurrentFilePathUsed \@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\CurrentFile` *and others. These functions are documented on page* 1161.)

2.2 expl3 helpers

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩          {\__filehook_file_parse_full_name:nN}{File helpers}%
\ExplSyntaxOn
```

`\__filehook_file_parse_full_name:nN`
`\__filehook_full_name:nn`

A utility macro to trigger expl3's file-parsing and lookup, and return a normalized representation of the file name. If the queried file doesn't exist, no normalization takes place. The output of `\__filehook_file_parse_full_name:nN` is passed on to the `#2`—a 3-argument macro that takes the ⟨*path*⟩, ⟨*base*⟩, and ⟨*ext*⟩ parts of the file name.

```
\cs_new:Npn \__filehook_file_parse_full_name:nN #1
  {
    \exp_args:Nf \file_parse_full_name_apply:nN
      {
        \exp_args:Nf \__filehook_full_name:nn
          { \file_full_name:n {#1} } {#1}
      }
  }
\cs_new:Npn \__filehook_full_name:nn #1 #2
  {
    \tl_if_empty:nTF {#1}
      { \tl_trim_spaces:n {#2} }
      { \tl_trim_spaces:n {#1} }
  }
```

(*End of definition for* `\__filehook_file_parse_full_name:nN` *and* `\__filehook_full_name:nn`.)

`\__filehook_if_no_extension:nTF`
`\__filehook_drop_extension:N`

Some actions depend on whether the file extension was explicitly given, and sometimes the extension has to be removed. The macros below use `\__filehook_file_parse_full_name:nN` to split up the file name and either check if ⟨*ext*⟩ (`#3`) is empty, or discard it.

```
\cs_new:Npn \__filehook_if_no_extension:nTF #1
  {
    \exp_args:Ne \tl_if_empty:nTF
      { \file_parse_full_name_apply:nN {#1} \use_iii:nnn }
  }
```

```
\cs_new_protected:Npn \__filehook_drop_extension:N #1
  {
    \tl_gset:Ne #1
      {
        \exp_args:NV \__filehook_file_parse_full_name:nN #1
          \__filehook_drop_extension_aux:nnn
      }
  }
\cs_new:Npn \__filehook_drop_extension_aux:nnn #1 #2 #3
   { \tl_if_empty:nF {#1} { #1 / } #2 }
```

(*End of definition for* `\__filehook_if_no_extension:nTF` *and* `\__filehook_drop_extension:N`.)

`\g__filehook_input_file_seq`
`\l__filehook_internal_tl`
`\__filehook_file_push:`
`\__filehook_file_pop:`
`\__filehook_file_pop_assign:nnnn`

Yet another stack, to keep track of `\CurrentFile` and `\CurrentFilePath` with nested `\inputs`. At the beginning of `\InputIfFileExists`, the current value of `\CurrentFilePath` and `\CurrentFile` is pushed to `\g__filehook_input_file_seq`, and at the end, it is popped and the value reassigned. Some other places don't use `\InputIfFileExists` directly (`\include`) or need `\CurrentFile` earlier (`\@onefilewithoptions`), so these are manually used elsewhere as well.

```
\tl_new:N \l__filehook_internal_tl
\seq_if_exist:NF \g__filehook_input_file_seq
  { \seq_new:N \g__filehook_input_file_seq }
\cs_new_protected:Npn \__filehook_file_push:
  {
    \seq_gpush:Ne \g__filehook_input_file_seq
      {
        { \CurrentFilePathUsed } { \CurrentFileUsed }
        { \CurrentFilePath     } { \CurrentFile     }
      }
  }
\cs_new_protected:Npn \__filehook_file_pop:
  {
    \seq_gpop:NNTF \g__filehook_input_file_seq \l__filehook_internal_tl
      { \exp_after:wN \__filehook_file_pop_assign:nnnn \l__filehook_internal_tl }
      {
        \msg_error:nnn { latex2e } { should-not-happen }
          { Tried~to~pop~from~an~empty~file~name~stack. }
      }
  }
\cs_new_protected:Npn \__filehook_file_pop_assign:nnnn #1 #2 #3 #4
  {
    \tl_set:Nn \CurrentFilePathUsed {#1}
    \tl_set:Nn \CurrentFileUsed {#2}
    \tl_set:Nn \CurrentFilePath {#3}
    \tl_set:Nn \CurrentFile {#4}
  }
\ExplSyntaxOff
```

(*End of definition for* `\g__filehook_input_file_seq` *and others.*)

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

When rolling forward the following expl3 functions may not be defined. If we roll back the code does nothing.

```
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\file_parse_full_name_apply:nN}{Roll forward help}%
⟨latexrelease⟩
⟨latexrelease⟩\ExplSyntaxOn
⟨latexrelease⟩\cs_if_exist:NF\file_parse_full_name_apply:nN
⟨latexrelease⟩{
⟨latexrelease⟩\cs_new:Npn \file_parse_full_name_apply:nN #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \exp_args:Ne \__file_parse_full_name_auxi:nN
⟨latexrelease⟩      { \__kernel_file_name_sanitize:n {#1} }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new:Npn \__file_parse_full_name_auxi:nN #1
⟨latexrelease⟩  {
⟨latexrelease⟩    \__file_parse_full_name_area:nw { } #1
⟨latexrelease⟩      / \s__file_stop
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new:Npn \__file_parse_full_name_area:nw #1 #2 / #3 \s__file_stop
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_empty:nTF {#3}
⟨latexrelease⟩      { \__file_parse_full_name_base:nw { } #2 . \s__file_stop {#1} }
⟨latexrelease⟩      { \__file_parse_full_name_area:nw { #1 / #2 }
⟨latexrelease⟩                                       #3 \s__file_stop }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new:Npn \__file_parse_full_name_base:nw #1 #2 . #3 \s__file_stop
⟨latexrelease⟩  {
⟨latexrelease⟩    \tl_if_empty:nTF {#3}
⟨latexrelease⟩      {
⟨latexrelease⟩        \tl_if_empty:nTF {#1}
⟨latexrelease⟩          {
⟨latexrelease⟩            \tl_if_empty:nTF {#2}
⟨latexrelease⟩              { \__file_parse_full_name_tidy:nnnN { } { } }
⟨latexrelease⟩              { \__file_parse_full_name_tidy:nnnN { .#2 } { } }
⟨latexrelease⟩          }
⟨latexrelease⟩          { \__file_parse_full_name_tidy:nnnN {#1} { .#2 } }
⟨latexrelease⟩      }
⟨latexrelease⟩      { \__file_parse_full_name_base:nw { #1 . #2 }
⟨latexrelease⟩                                       #3 \s__file_stop }
⟨latexrelease⟩  }
⟨latexrelease⟩\cs_new:Npn \__file_parse_full_name_tidy:nnnN #1 #2 #3 #4
⟨latexrelease⟩  {
⟨latexrelease⟩    \exp_args:Nee #4
⟨latexrelease⟩      {
⟨latexrelease⟩        \str_if_eq:nnF {#3} { / } { \use_none:n }
⟨latexrelease⟩        #3 \prg_do_nothing:
⟨latexrelease⟩      }
⟨latexrelease⟩      { \use_none:n #1 \prg_do_nothing: }
⟨latexrelease⟩      {#2}
⟨latexrelease⟩  }
⟨latexrelease⟩}
⟨latexrelease⟩\ExplSyntaxOff
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
⟨@@=⟩
```

2.3 Declaring the file-related hooks

These hooks have names with three-parts that start with `file/`, `include/`, `class/` or `package/` and end with `/before` or `/after` (or `/end` in the case of `include/`). They are all generic hooks so will be declared only if code is added to them; this declaration is done for you automatically and, indeed, they should not be declared explicitly.

Those named `.../after` and `include/.../end` are, when code is added, declared as reversed hooks.

2.4 Patching LaTeX's `\InputIfFileExists` command

Most of what we have to do is adding `\UseHook` into several LaTeX 2_ε core commands, because of some circular dependencies in the kernel we do this only now and not in `ltfiles`.

`\InputIfFileExists` `\@input@file@exists@with@hooks` `\unqu@tefilef@und`

`\InputIfFileExists` loads any file if it is available so we have to add the hooks `file/before` and `file/after` in the right places. If the file doesn't exist no hooks should be executed.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩           {\InputIfFileExists}{Hook management (files)}%
⟨*2ekernel | latexrelease⟩
\let\InputIfFileExists\@undefined
\DeclareRobustCommand \InputIfFileExists[2]{%
  \IfFileExists{#1}%
    {%
      \@expl@@@filehook@file@push@@
      \@filehook@set@CurrentFile
```

We pre-expand `\@filef@und` so that in case another file is loaded in the true branch of `\InputIfFileExists`, these don't change their value meanwhile. This isn't a worry with `\CurrentFile...` because they are kept in a stack.

```
      \expandafter\@swaptwoargs\expandafter
        {\expandafter\@input@file@exists@with@hooks
          \expandafter{\@filef@und}}%
        {#2}%
      \@expl@@@filehook@file@pop@@
    }%
}
\def\@input@file@exists@with@hooks#1{%
```

If the file exists then `\CurrentFile` holds its name. But we can't rely on that still being true after the file has been processed. Thus for using the name in the file hooks we need to preserve the name and then restore it for the `file/.../after` hook.

The hook always refers to the file requested by the user. The hook is *always* loaded for `\CurrentFile` which usually is the same as `\CurrentFileUsed`. In the case of a file replacement, the `\CurrentFileUsed` holds the actual file loaded. In any case the file names are normalized so that the hooks work on the real file name, rather than what the user typed in.

expl3's `\file_full_name:n` normalizes the file name (to factor out differences in the `.tex` extension), and then does a file lookup to take into account a possible path from `\l_file_search_path_seq` and `\input@path`. However only the file name and extension

are returned so that file hooks can refer to the file by their name only. The path to the file is returned in `\CurrentFilePath`.

```
  \edef\reserved@a{%
    \@expl@@@filehook@file@pop@assign@@nnnn
      {\CurrentFilePathUsed}%
      {\CurrentFileUsed}%
      {\CurrentFilePath}%
      {\CurrentFile}}%
  \expandafter\@swaptwoargs\expandafter{\reserved@a}%
```

Before adding to the file list we need to make all (letter) characters catcode 11, because several packages use constructions like

```
\filename@parse{<filename>}
\ifx\filename@ext\@clsextension
  ...
\fi
```

and that doesn't work if `\filename@ext` is `\detokenize`d. Making `\@clsextension` a string doesn't help much because some packages define their own `\<prefix>@someextension` with normal catcodes. This is not entirely correct because packages loaded (somehow) with catcode 12 alphabetic tokens (say, as the result of a `\string` or `\detokenize` command, or from a TEX string like `\jobname`) will have these character tokens incorrectly turned into letter tokens. This however is rare, so we'll go for the all-letters approach (grepping the packages in TEX Live didn't bring up any obvious candidate for breaking with this catcode change).

```
    {\edef\reserved@a{\unqu@tefilef@und#1\@nil}%
     \@addtofilelist{\string@makeletter\reserved@a}%
     \UseHook{file/before}%
```

The current file name is available in `\CurrentFile` so we use that in the specific hook.

```
     \UseHook{file/\CurrentFile/before}%
     \@@input #1% <- trailing space comes from \@filef@und
    }%
```

And here, `\CurrentFile` is restored (by `\@expl@@@filehook@file@pop@assign@@nnnn`) so we can use it once more.

```
  \UseHook{file/\CurrentFile/after}%
  \UseHook{file/after}}
\def\unqu@tefilef@und"#1" \@nil{#1}
```

Now declare the non-generic file hooks used above:

```
\NewHook{file/before}
\NewReversedHook{file/after}
⟨latexrelease⟩\EndIncludeInRelease
⟨/2ekernel | latexrelease⟩
```

Now define `\InputIfFileExists` to input `#1` if it seems to exist. Immediately prior to the input, `#2` is executed. If the file `#1` does not exist, execute '`#3`'.

```
⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩           {\InputIfFileExists}{Hook management (files)}%
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand \InputIfFileExists[2]{%
⟨latexrelease⟩  \IfFileExists{#1}%
⟨latexrelease⟩    {%
```

```
⟨latexrelease⟩  \expandafter\@swaptwoargs\expandafter
⟨latexrelease⟩      {\@filef@und}{#2\@addtofilelist{#1}\@@input}}}
⟨latexrelease⟩\let\@input@file@exists@with@hooks\@undefined
⟨latexrelease⟩\let\unqu@tefilef@und\@undefined
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩          {\InputIfFileExists}{Hook management (files)}%
⟨latexrelease⟩\long\def \InputIfFileExists#1#2{%
⟨latexrelease⟩  \IfFileExists{#1}%
⟨latexrelease⟩    {#2\@addtofilelist{#1}\@@input \@filef@und}}
```

Also undo the internal command as some packages unfortunately test for their existence instead of using `\IfFormatAtLeastTF`.

```
⟨latexrelease⟩\expandafter\let\csname InputIfFileExists \endcsname\@undefined

⟨latexrelease⟩\let\@input@file@exists@with@hooks\@undefined
⟨latexrelease⟩\let\unqu@tefilef@und\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\InputIfFileExists`*,* `\@input@file@exists@with@hooks`*, and* `\unqu@tefilef@und`*.*)

2.5 Declaring a file substitution

```
⟨@@=filehook⟩

⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩          {\__filehook_subst_add:nn}{Declaring file substitution}%
\ExplSyntaxOn
```

`\__filehook_subst_add:nn`
`\__filehook_subst_remove:n`
`\__filehook_subst_file_normalize:Nn`
`\__filehook_subst_empty_name_chk:NN`

`\__filehook_subst_add:nn` declares a file substitution by doing a (global) definition of the form `\def\@file-subst@`⟨*file*⟩`{`⟨*replacement*⟩`}`. The file names are properly sanitized, and normalized with the same treatment done for the file hooks. That is, a file replacement is declared by using the file name (and extension, if any) only, and the file path should not be given. If a file name is empty it is replaced by `.tex` (the empty csname is used to check that).

```
\cs_new_protected:Npn \__filehook_subst_add:nn #1 #2
  {
    \group_begin:
      \cs_set:cpe { } { \exp_not:o { \cs:w\cs_end: } }
      \int_set:Nn \tex_escapechar:D { -1 }
      \cs_gset:cpe
        {
          @file-subst@
          \__filehook_subst_file_normalize:Nn \use_ii_iii:nnn {#1}
        }
        { \__filehook_subst_file_normalize:Nn \__filehook_file_name_compose:nnn
                                        {#2} }
    \group_end:
  }
\cs_new_protected:Npn \__filehook_subst_remove:n #1
  {
```

```
    \group_begin:
      \cs_set:cpe { } { \exp_not:o { \cs:w\cs_end: } }
      \int_set:Nn \tex_escapechar:D { -1 }
      \cs_undefine:c
        {
          @file-subst@
          \__filehook_subst_file_normalize:Nn \use_ii_iii:nnn {#1}
        }
    \group_end:
  }
\cs_new:Npn \__filehook_subst_file_normalize:Nn #1 #2
  {
    \exp_after:wN \__filehook_subst_empty_name_chk:NN
      \cs:w \exp_after:wN \cs_end:
        \cs:w \__filehook_file_parse_full_name:nN {#2} #1 \cs_end:
  }
\cs_new:Npn \__filehook_subst_empty_name_chk:NN #1 #2
  { \if_meaning:w #1 #2 .tex \else: \token_to_str:N #2 \fi: }
```

(*End of definition for* `\__filehook_subst_add:nn` *and others.*)

`\use_ii_iii:nnn` A variant of `\use_...` to discard the first of three arguments.

Todo: this should move to expl3

```
\cs_gset:Npn \use_ii_iii:nnn #1 #2 #3 {#2 #3}
```

(*End of definition for* `\use_ii_iii:nnn`.)

```
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

`\declare@file@substitution` `\undeclare@file@substitution` For two internals we provide LaTeX 2_ε names so that we can use them elsewhere in the kernel (and so that they can be used in packages if really needed, e.g., scrlfile).

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩           {\declare@file@substitution}{File substitution}%
\ExplSyntaxOn
\cs_new_eq:NN \declare@file@substitution   \__filehook_subst_add:nn
\cs_new_eq:NN \undeclare@file@substitution \__filehook_subst_remove:n
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

We are not fully rolling back the file substitutions in case a rollback encounters a package that contains them, but is itself not setup for rollback. So we just bypass them and hope for the best.

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩           {\declare@file@substitution}{File substitution}%
⟨latexrelease⟩
⟨latexrelease⟩\let \declare@file@substitution   \@gobbletwo
⟨latexrelease⟩\let \undeclare@file@substitution \@gobble
⟨latexrelease⟩
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\declare@file@substitution` *and* `\undeclare@file@substitution`. *These functions are documented on page* 1164.)

```
⟨@@=⟩
```

2.6 Selecting a file (`\set@curr@file`)

`\set@curr@file`
`\set@curr@file@nosearch`
`\@curr@file`
`\@curr@file@reqd`

Now we hook into `\set@curr@file` to resolve a possible file substitution, and add `\@expl@@@filehook@set@curr@file@@nNN` at the end, after `\@curr@file` is set.

A file name is built using `\expandafter\string\csname`⟨*filename*⟩`\endcsname` to avoid expanding utf8 active characters. The `\csname` expands the normalization machinery and the routine to resolve a file substitution, returning a control sequence with the same name as the file.

It happens that when ⟨*filename*⟩ is empty, the generated control sequence is `\csname\endcsname`, and doing `\string` on that results in the file `csnameendcsname.tex`. To guard against that we `\ifx`-compare the generated control sequence with the empty csname. To do so, `\csname\endcsname` has to be defined, otherwise it would be equal to `\relax` and we would have false positives. Here we define `\csname\endcsname` to expand to itself to avoid it matching the definition of some other control sequence.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2022/06/01}%
⟨latexrelease⟩           {\set@curr@file}{Setting current file name}%
\def\set@curr@file{%
  \begingroup
    \set@curr@file@aux}
\edef\set@curr@file@nosearch{%
  \begingroup
    \let\noexpand\input@path\noexpand\@empty
    \csname seq_clear:N\endcsname
      \expandafter\noexpand\csname l_file_search_path_seq\endcsname
    \noexpand\set@curr@file@aux}
\def\set@curr@file@aux#1{%
    \escapechar\m@ne
    \let\protect\string
    \edef~{\string~}%
    \expandafter\def\csname\expandafter\endcsname
      \expandafter{\csname\endcsname}%
```

Two file names are set here: `\@curr@file@reqd` which is the file requested by the user, and `\@curr@file` which should be the same, except when we have a file substitution, in which case it holds the actual loaded file. `\@curr@file` is resolved first, to check if a substitution happens. If it doesn't, `\@expl@@@filehook@if@file@replaced@@TF` short-cuts and just copies `\@curr@file`, otherwise the full normalization procedure is executed.

At this stage the file name is parsed and normalized, but if the input doesn't have an extension, the default `.tex` is *not* added to `\@curr@file` because for applications other than `\input` (graphics, for example) the default extension may not be `.tex`. First check if the input has an extension, then if the input had no extension, call

`\@expl@@@filehook@drop@extension@@N`. In case of a file substitution, `\@curr@file` will have an extension.

```
    \@expl@@@filehook@if@no@extension@@nTF{#1}%
      {\@tempswatrue}{\@tempswafalse}%
    \@kernel@make@file@csname\@curr@file
      \@expl@@@filehook@resolve@file@subst@@w {#1}%
    \@expl@@@filehook@if@file@replaced@@TF
      {\@kernel@make@file@csname\@curr@file@reqd
         \@expl@@@filehook@normalize@file@name@@w{#1}%
       \if@tempswa \@expl@@@filehook@drop@extension@@N\@curr@file@reqd \fi}%
      {\if@tempswa \@expl@@@filehook@drop@extension@@N\@curr@file \fi
       \global\let\@curr@file@reqd\@curr@file}%
    \@expl@@@filehook@clear@replacement@flag@@
  \endgroup}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩           {\set@curr@file}{Setting current file name}%
⟨latexrelease⟩\def\set@curr@file#1{%
⟨latexrelease⟩  \begingroup
⟨latexrelease⟩    \escapechar\m@ne
⟨latexrelease⟩    \let\protect\string
⟨latexrelease⟩    \edef~{\string~}%
⟨latexrelease⟩    \expandafter\def\csname\expandafter\endcsname
⟨latexrelease⟩      \expandafter{\csname\endcsname}%
⟨latexrelease⟩    \@expl@@@filehook@if@no@extension@@nTF{#1}%
⟨latexrelease⟩      {\@tempswatrue}{\@tempswafalse}%
⟨latexrelease⟩    \@kernel@make@file@csname\@curr@file
⟨latexrelease⟩      \@expl@@@filehook@resolve@file@subst@@w {#1}%
⟨latexrelease⟩    \@expl@@@filehook@if@file@replaced@@TF
⟨latexrelease⟩      {\@kernel@make@file@csname\@curr@file@reqd
⟨latexrelease⟩         \@expl@@@filehook@normalize@file@name@@w{#1}%
⟨latexrelease⟩       \if@tempswa \@expl@@@filehook@drop@extension@@N\@curr@file@reqd \fi}%
⟨latexrelease⟩      {\if@tempswa \@expl@@@filehook@drop@extension@@N\@curr@file \fi
⟨latexrelease⟩       \global\let\@curr@file@reqd\@curr@file}%
⟨latexrelease⟩    \@expl@@@filehook@clear@replacement@flag@@
⟨latexrelease⟩  \endgroup}
⟨latexrelease⟩\let\set@curr@file@nosearch\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩           {\set@curr@file}{Setting current file name}%
⟨latexrelease⟩\def\set@curr@file#1{%
⟨latexrelease⟩  \begingroup
⟨latexrelease⟩    \escapechar\m@ne
⟨latexrelease⟩    \expandafter\def\csname\expandafter\endcsname
⟨latexrelease⟩      \expandafter{\csname\endcsname}%
⟨latexrelease⟩    \@expl@@@filehook@if@no@extension@@nTF{#1}%
⟨latexrelease⟩      {\@tempswatrue}{\@tempswafalse}%
⟨latexrelease⟩    \@kernel@make@file@csname\@curr@file
⟨latexrelease⟩      \@expl@@@filehook@resolve@file@subst@@w {#1}%
⟨latexrelease⟩    \@expl@@@filehook@if@file@replaced@@TF
⟨latexrelease⟩      {\@kernel@make@file@csname\@curr@file@reqd
⟨latexrelease⟩         \@expl@@@filehook@normalize@file@name@@w{#1}%
```

```
⟨latexrelease⟩       \if@tempswa \@expl@@@filehook@drop@extension@@N\@curr@file@reqd \fi}%
⟨latexrelease⟩      {\if@tempswa \@expl@@@filehook@drop@extension@@N\@curr@file \fi
⟨latexrelease⟩       \global\let\@curr@file@reqd\@curr@file}%
⟨latexrelease⟩    \@expl@@@filehook@clear@replacement@flag@@
⟨latexrelease⟩  \endgroup}
⟨latexrelease⟩\let\set@curr@file@nosearch\@undefined
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{2019/10/01}%
⟨latexrelease⟩          {\set@curr@file}{Setting current file name}%
⟨latexrelease⟩\def\set@curr@file#1{%
⟨latexrelease⟩  \begingroup
⟨latexrelease⟩    \escapechar\m@ne
⟨latexrelease⟩    \xdef\@curr@file{%
⟨latexrelease⟩      \expandafter\expandafter\expandafter\unquote@name
⟨latexrelease⟩      \expandafter\expandafter\expandafter{%
⟨latexrelease⟩      \expandafter\string
⟨latexrelease⟩        \csname\@firstofone#1\@empty\endcsname}}%
⟨latexrelease⟩  \endgroup
⟨latexrelease⟩}
⟨latexrelease⟩\let\set@curr@file@nosearch\@undefined
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩          {\set@curr@file}{Setting current file name}%
⟨latexrelease⟩\let\set@curr@file\@undefined
⟨latexrelease⟩\let\set@curr@file@nosearch\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

(*End of definition for* `\set@curr@file` *and others.*)

`\@filehook@set@CurrentFile`
`\@kernel@make@file@csname`
`\@set@curr@file@aux`

Todo: This should get internalized using `@expl@` *names*

```
⟨/2ekernel⟩
⟨∗2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩          {\@kernel@make@file@csname}{Make file csname}%

\def\@kernel@make@file@csname#1#2#3{%
  \xdef#1{\expandafter\@set@curr@file@aux
    \csname\expandafter#2\@firstofone#3\@nil\endcsname}}
```

This auxiliary compares \⟨*filename*⟩ with `\csname\endcsname` to check if the empty `.tex` file was requested.

```
\long\def\@set@curr@file@aux#1{%
  \expandafter\ifx\csname\endcsname#1%
    .tex\else\string#1\fi}
```

Then we call `\@expl@@@filehook@set@curr@file@@nNN` once for `\@curr@file` to set `\CurrentFile(Path)Used` and once for `\@curr@file@reqd` to set `\CurrentFile(Path)`. Here too the slower route is only used if a substitution happened, but here `\@expl@@@filehook@if@file@replaced@@TF` can't be used because the flag is reset at the `\endgroup` above, so we check if `\@curr@file` and `\@curr@file@reqd` differ. This macro is issued separate from `\set@curr@file` because it changes `\CurrentFile`, and side-effects would quickly get out of control.

```
\def\@filehook@set@CurrentFile{%
```

```
  \@expl@@@filehook@set@curr@file@@nNN{\@curr@file}%
    \CurrentFileUsed\CurrentFilePathUsed
  \ifx\@curr@file@reqd\@curr@file
    \let\CurrentFile\CurrentFileUsed
    \let\CurrentFilePath\CurrentFilePathUsed
  \else
    \@expl@@@filehook@set@curr@file@@nNN{\@curr@file@reqd}%
      \CurrentFile\CurrentFilePath
  \fi}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@filehook@set@CurrentFile`, `\@kernel@make@file@csname`, *and* `\@set@curr@file@aux`.)

```
⟨@@=filehook⟩
```

`\__filehook_set_curr_file:nNN`
`\__filehook_set_curr_file_assign:nnnNN`

When inputting a file, `\set@curr@file` does a file lookup (in `\input@path` and `\l_file_search_path_seq`) and returns the actual file name (⟨*base*⟩ plus ⟨*ext*⟩) in `\CurrentFileUsed`, and in case there's a file substitution, the requested file in `\CurrentFile` (otherwise both are the same). Only the base and extension are returned, regardless of the input (both `path/to/file.tex` and `file.tex` end up as `file.tex` in `\CurrentFile`). The path is returned in `\CurrentFilePath`, in case it's needed.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩           {\__filehook_set_curr_file:nNN}{Set curr file}%
\ExplSyntaxOn
\cs_new_protected:Npn \__filehook_set_curr_file:nNN #1
  {
    \exp_args:Nf \__filehook_file_parse_full_name:nN {#1}
      \__filehook_set_curr_file_assign:nnnNN
  }
\cs_new_protected:Npn \__filehook_set_curr_file_assign:nnnNN #1 #2 #3 #4 #5
  {
    \str_set:Nn #5 {#1}
    \str_set:Nn #4 {#2#3}
  }
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\__filehook_set_curr_file:nNN` *and* `\__filehook_set_curr_file_assign:nnnNN`.)

2.7 Replacing a file and detecting loops

`\__filehook_resolve_file_subst:w`
`\__filehook_normalize_file_name:w`
`\__filehook_file_name_compose:nnn`

Start by sanitizing the file with `\__filehook_file_parse_full_name:nN` then do `\__filehook_file_subst_begin:nnn{`⟨*path*⟩`}{`⟨*name*⟩`}{`⟨*ext*⟩`}`.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
```

```
⟨latexrelease⟩      {\__filehook_resolve_file_subst:w}{Replace files detect loops}%
\ExplSyntaxOn
\cs_new:Npn \__filehook_resolve_file_subst:w #1 \@nil
  { \__filehook_file_parse_full_name:nN {#1} \__filehook_file_subst_begin:nnn }
\cs_new:Npn \__filehook_normalize_file_name:w #1 \@nil
  { \__filehook_file_parse_full_name:nN {#1} \__filehook_file_name_compose:nnn }
\cs_new:Npn \__filehook_file_name_compose:nnn #1 #2 #3
  { \tl_if_empty:nF {#1} { #1 / } #2#3 }
```

Since the file replacement is done expandably in a `\csname`, use a flag to remember if a substitution happened. We use this in `\set@curr@file` to short-circuit some of it in case no substitution happened (by far the most common case, so it's worth optimizing). The flag raised during the file substitution algorithm must be explicitly cleared after the `\__filehook_if_file_replaced:TF` conditional is no longer needed, otherwise further uses of `\__filehook_if_file_replaced:TF` will wrongly return true.

`__filehook_file_replaced`
`\__filehook_if_file_replaced:TF`
`\__filehook_clear_replacement_flag:`

```
\flag_new:n { __filehook_file_replaced }
\cs_new:Npn \__filehook_if_file_replaced:TF #1 #2
  { \flag_if_raised:nTF { __filehook_file_replaced } {#1} {#2} }
\cs_new_protected:Npn \__filehook_clear_replacement_flag:
  { \flag_clear:n { __filehook_file_replaced } }
```

First off, start by checking if the current file (⟨*name*⟩+⟨*ext*⟩) has a declared substitution. If not, then just put that as the name (including a possible ⟨*path*⟩ in this case): this is the default case with no substitutions, so it's the first to be checked. The auxiliary `\__filehook_file_subst_tortoise_hare:nn` sees that there's no replacement for `#2#3` and does nothing else.

`\__filehook_file_subst_begin:nnn`

```
\cs_new:Npn \__filehook_file_subst_begin:nnn #1 #2 #3
  {
    \__filehook_file_subst_tortoise_hare:nn { #2#3 } { #2#3 }
      { \__filehook_file_name_compose:nnn {#1} {#2} {#3} }
  }
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

2.7.1 The Tortoise and Hare algorithm

If there is a substitution (⟨*true*⟩ in the first `\cs_if_exist:cTF` below), then first check if there is no substitution down the line: this should be the second most common case, of one file replaced by another. In that case just leave the substitution there and the job is done. If any substitution happens, then the `\flag __filehook_file_replaced` is raised (conditionally, because checking if a flag is raised is much faster than raising it over and over again).

`\__filehook_file_subst_tortoise_hare:nn`
`\__filehook_file_subst_loop:NN`
`\__filehook_file_subst_loop:cc`

If, however there are more substitutions, then we need to check for a possible loop in the substitutions, which would otherwise put TEX in an infinite loop if just an exhaustive expansion was used.

To detect a loop, the *Tortoise and Hare* algorithm is used. The name of the algorithm is an analogy to Aesop's fable, in which the Hare outruns a Tortoise. The two pointers here are the csnames which contains each file replacement, both of which start at the position zero, which is the file requested. In the inner part of the macro below, `\__filehook_file_subst_loop:cc` is called with `\@file-subst@`⟨*file*⟩ and

`\@file-subst@\@file-subst@⟨file⟩`; that is, the substitution of ⟨*file*⟩ and the substitution of that substitution: the Tortoise walks one step while the Hare walks two.

Within `\__filehook_file_subst_loop:NN` the two substitutions are compared, and if they lead to the same file it means that there is a loop in the substitutions. If there's no loop, `\__filehook_file_subst_tortoise_hare:nn` is called again with the Tortoise at position 1 and the hare at 2. Again, the substitutions are checked ahead of the Hare pointer to check that it won't run too far; in case there is no loop in the declarations, eventually one of the `\cs_if_exist:cTF` below will go ⟨*false*⟩ and the algorithm will end; otherwise it will run until the Hare reaches the same spot as the tortoise and a loop is detected.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩  {\__filehook_file_subst_tortoise_hare:nn}{Tortoise and Hare}%
\ExplSyntaxOn
\cs_new:Npn \__filehook_file_subst_tortoise_hare:nn #1 #2 #3
  {
    \cs_if_exist:cTF { @file-subst@ #2 }
      {
        \flag_if_raised:nF { __filehook_file_replaced }
          { \flag_raise:n { __filehook_file_replaced } }
        \cs_if_exist:cTF { @file-subst@ \use:c { @file-subst@ #2 } }
          {
            \__filehook_file_subst_loop:cc
              { @file-subst@ #1 }
              { @file-subst@ \use:c { @file-subst@ #2 } }
          }
          { \use:c { @file-subst@ #2 } }
      }
      { #3 }
  }
```

This is just an auxiliary to check if a loop was found, and continue the algorithm otherwise. If a loop is found, the `.tex` file is used as fallback and `\__filehook_file_subst_cycle_error:cN` is called to report the error.

```
\cs_new:Npn \__filehook_file_subst_loop:NN #1 #2
  {
    \token_if_eq_meaning:NNTF #1 #2
      {
        .tex
        \__filehook_file_subst_cycle_error:cN { @file-subst@ #1 } #1
      }
      { \__filehook_file_subst_tortoise_hare:nn {#1} {#2} {#2} }
  }
\cs_generate_variant:Nn \__filehook_file_subst_loop:NN { cc }
```

Showing this type of error expandably is tricky, as we have a very limited amount of characters to show and a potentially large list. As a work around, several errors are printed, each showing one step of the loop, until all the error messages combined show the loop.

__filehook_file_subst_cycle_error:NN
__filehook_file_subst_cycle_error:cN

```
\cs_new:Npn \__filehook_file_subst_cycle_error:NN #1 #2
  {
    \msg_expandable_error:nnff { latex2e } { file-cycle }
```

```
      {#1} { \use:c { @file-subst@ #1 } }
    \token_if_eq_meaning:NNF #1 #2
      { \__filehook_file_subst_cycle_error:cN { @file-subst@ #1 } #2 }
  }
\cs_generate_variant:Nn \__filehook_file_subst_cycle_error:NN { c }
```

And the error message:

```
\msg_new:nnn { latex2e } { file-cycle }
  { File~loop!~#1~replaced~by~#2... }
```

(*End of definition for* `\__filehook_resolve_file_subst:w` *and others.*)

```
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

```
⟨@@=⟩
```

2.8 Preventing a package from loading

We support the use case of preventing a package from loading but not any other type of files (e.g., classes).

\disable@package@load
\reenable@package@load
\@disable@packageload@do

`\disable@package@load` defines `\@pkg-disable@`⟨*package*⟩ to expand to some code `#2` instead of loading the package.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩           {\disable@package@load}{Disable packages}%
\def\disable@package@load#1#2{%
  \global\@namedef{@pkg-disable@#1.\@pkgextension}{#2}}
```

Here we check if a control sequence named `\@pkg-disable@`⟨*name*⟩`.sty` is defined, and if so don't use the package loading code `#2`, but use the replacement code stored in that control sequence, write something to the log, and then prevent `\@onefilewithoptions` from sanity-checking the requested package date (the `\expandafter` here triggers one in `\@onefilewithoptions` that ends a conditional there, and the `\@gobbletwo` removes the date checking code from the input stream).

```
\def\@disable@packageload@do#1#2{%
  \@ifundefined{@pkg-disable@#1}%
     {#2}%
     {\@nameuse{@pkg-disable@#1}%
      \@latex@info{Package '#1' has been disabled.%
       \MessageBreak Load request ignored}%
      \expandafter\@gobbletwo}}
```

`\reenable@package@load` undefines `\@pkg-disable@`⟨*package*⟩ to reallow loading a package.

```
\def\reenable@package@load#1{%
  \global\expandafter\let
  \csname @pkg-disable@#1.\@pkgextension \endcsname \@undefined}
```

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩           {\disable@package@load}{Disable packages}%
⟨latexrelease⟩
⟨latexrelease⟩\let\disable@package@load   \@undefined
⟨latexrelease⟩\let\@disable@packageload@do\@undefined
⟨latexrelease⟩\let\reenable@package@load  \@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\disable@package@load`, `\reenable@package@load`, *and* `\@disable@packageload@do`. *These functions are documented on page* 1164.)

2.9 High-level interfaces for LaTeX

None so far and the general feeling for now is that the hooks are enough. Packages like filehook, etc., may use them to set up their interfaces (samples are given below) but for the now the kernel will not provide any.

2.10 Internal commands needed elsewhere

Here we set up a few horrible (but consistent) LaTeX 2_ε names to allow for internal commands to be used outside this module (and in parts that still use LaTeX 2_ε syntax. We have to unset the `@@` since we want double "at" sign in place of double underscores.

```
⟨@@=⟩
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩    {\@expl@@@filehook@if@no@extension@@nTF}{2e tmp interfaces}%
\ExplSyntaxOn
\cs_new_eq:NN \@expl@@@filehook@if@no@extension@@nTF
              \__filehook_if_no_extension:nTF
\cs_new_eq:NN \@expl@@@filehook@set@curr@file@@nNN
              \__filehook_set_curr_file:nNN
\cs_new_eq:NN \@expl@@@filehook@resolve@file@subst@@w
              \__filehook_resolve_file_subst:w
\cs_new_eq:NN \@expl@@@filehook@normalize@file@name@@w
              \__filehook_normalize_file_name:w
\cs_new_eq:NN \@expl@@@filehook@if@file@replaced@@TF
              \__filehook_if_file_replaced:TF
\cs_new_eq:NN \@expl@@@filehook@clear@replacement@flag@@
              \__filehook_clear_replacement_flag:
\cs_new_eq:NN \@expl@@@filehook@drop@extension@@N
              \__filehook_drop_extension:N
\cs_new_eq:NN \@expl@@@filehook@file@push@@
              \__filehook_file_push:
\cs_new_eq:NN \@expl@@@filehook@file@pop@@
              \__filehook_file_pop:
```

```
\cs_new_eq:NN \@expl@@@filehook@file@pop@assign@@nnnn
              \__filehook_file_pop_assign:nnnn
\ExplSyntaxOff
```

This one specifically has to be undefined because it is left over in the input stream from `\InputIfFileExists` and executed when latexrelease is loaded. It cannot be `\let` to `\@undefined` otherwise it would error as well, so it is `\let` to `\relax` to be silently ignored when loading `\latexrelease`.

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩    {\@expl@@@filehook@if@no@extension@@nTF}{2e tmp interfaces}%
⟨latexrelease⟩\let\@expl@@@filehook@file@pop@@\relax
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel⟩
```

This ends the kernel code in this file.

```
⟨/2ekernel⟩
```

3 A sample package for structuring the log output

```
⟨∗structuredlog⟩
⟨@@=filehook⟩
\ProvidesExplPackage
    {structuredlog}{\ltfilehookdate}{\ltfilehookversion}
    {Structuring the TeX transcript file}
```

\g__filehook_nesting_level_int

Stores the current package nesting level.

```
\int_new:N \g__filehook_nesting_level_int
```

Initialize the counter with the number of files in the `\@currnamestack` (the number of items divided by the number of items we place on the stack for each file (which right now is 4)) minus one, because this package is skipped when printing to the log.

```
\int_gset:Nn \g__filehook_nesting_level_int
  { ( \tl_count:N \@currnamestack ) / 4 - 1 }
```

(*End of definition for* `\g__filehook_nesting_level_int`.)

__filehook_log_file_record:n

This macro is responsible for increasing and decreasing the file nesting level, as well as printing to the log. The argument is either `STOPTART` or `STOP` and the action it takes on the nesting integer depends on that.

```
\cs_new_protected:Npn \__filehook_log_file_record:n #1
  {
    \str_if_eq:nnT {#1} {START} { \int_gincr:N \g__filehook_nesting_level_int }
    \iow_term:e
      {
        \prg_replicate:nn { \g__filehook_nesting_level_int } { = } ~
        ( LEVEL ~ \int_use:N \g__filehook_nesting_level_int \c_space_tl #1 ) ~
        \CurrentFileUsed
```

If there was a file replacement, show that as well:

```
        \str_if_eq:NNF \CurrentFileUsed \CurrentFile
          { ~ ( \CurrentFile \c_space_tl requested ) }
        \iow_newline:
      }
    \str_if_eq:nnT {#1} {STOP} { \int_gdecr:N \g__filehook_nesting_level_int }
  }
```

Now just hook the macro above in the generic `file/before`...

```
\AddToHook{file/before}{ \__filehook_log_file_record:n { START } }
```

...and `file/after` hooks. We don't want to install the `file/after` hook immediately, because that would mean it is the first time executed when the package finishes. We therefore put the declaration inside `\AddToHookNext` so that it gets only installed when we have left this package.

```
\AddToHookNext{file/after}
  { \AddToHook{file/after}{ \__filehook_log_file_record:n { STOP } } }
```

(*End of definition for* `\__filehook_log_file_record:n`.)

```
⟨@@=⟩
⟨/structuredlog⟩
```

4 Package emulations

4.1 Package **atveryend** emulation

With the new hook management and the hooks in `\enddocument` all of atveryend is taken care of. We can make an emulation only here after the substitution functionality is available:

```
⟨*2ekernel⟩
\declare@file@substitution{atveryend.sty}{atveryend-ltx.sty}
⟨/2ekernel⟩
```

Here is the package file we point to:

```
⟨*atveryend-ltx⟩
\ProvidesPackage{atveryend-ltx}
   [2020/08/19 v1.0a
     Emulation of the original atveryend package^^Jwith kernel methods]
```

Here are new definitions for its interfaces now pointing to the hooks in `\enddocument`

```
\newcommand\AfterLastShipout  {\AddToHook{enddocument/afterlastpage}}
\newcommand\AtVeryEndDocument {\AddToHook{enddocument/afteraux}}
```

Next one is a bit of a fake, but the result should normally be as expected. If not, one needs to add a rule to sort the code chunks in `enddocument/info`.

```
\newcommand\AtEndAfterFileList{\AddToHook{enddocument/info}}
\newcommand\AtVeryVeryEnd     {\AddToHook{enddocument/end}}
```

`\BeforeClearDocument` This one is the only one we don't implement or rather don't have a dedicated hook in the code.

```
\ExplSyntaxOn
\newcommand\BeforeClearDocument[1]
  { \AtEndDocument{#1}
    \atveryend@DEPRECATED{BeforeClearDocument \tl_to_str:n{#1}}
  }
```

```
\cs_new:Npn\atveryend@DEPRECATED #1
   {\iow_term:e{======~DEPRECATED~USAGE~#1~==========}}
\ExplSyntaxOff
```

(*End of definition for* `\BeforeClearDocument`.)

⟨/atveryend-ltx⟩

File 53
ltshipout.dtx

1 Introduction

The code provides an interface to the `\shipout` primitive of TEX which is called when a finished pages is finally "shipped out" to the target output file, e.g., the `.dvi` or `.pdf` file. A good portion of the code is based on ideas by Heiko Oberdiek implemented in his packages atbegshi and atenddvi even though the interfaces are somewhat different.[56]

1.1 Overloading the `\shipout` primitive

`\shipout` With this implementation TEX's shipout primitive is no longer available for direct use. Instead `\shipout` is running some (complicated) code that picks up the box to be shipped out regardless of how that is done, i.e., as a constructed `\vbox` or `\hbox` or as a box register.

It then stores it in a named box register. This box can then be manipulated through a set of hooks after which it is shipped out for real.

Each shipout that actually happens (i.e., where the material is not discarded for one or the other reason) is recorded and the total number is available in a readonly variable and in a LATEX counter.

`\RawShipout` This command implements a simplified shipout that bypasses the foreground and background hooks, e.g., only `shipout/firstpage` and `shipout/lastpage` are executed and the total shipout counters are incremented.

The command doesn't use `\ShipoutBox` but its own private box register so that it can be used inside of shipout hooks to do some additional shipouts while already in the output routine with the current page being stored in `\ShipoutBox`. It does have access to `\ShipoutBox` if it is used in `shipout/before` (or `shipout/after`) and can use its content.

It is safe to use it in `shipout/before` or `shipout/after` but not necessarily in the other `shipout/...` hooks as they are intended for special processing.

[56] Heiko's interfaces are emulated by the kernel code, if a document requests his packages, so older documents will continue to work.

\ShipoutBox
\l_shipout_box

This box register is called \ShipoutBox (alternatively available via the L3 name \l_shipout_box).

This box is a "local" box and assignments to it should be done only locally. Global assignments (as done by some packages with older code where this is box is known as 255) may work but they are conceptually wrong and may result in errors under certain circumstances.

During the execution of shipout/before this box contains the accumulated material for the page, but not yet any material added by other shipout hooks. During execution of shipout/after, i.e., after the shipout has happened, the box also contains any background or foreground material.

Material from the hooks shipout/firstpage or shipout/lastpage is not included (but only used during the actual shipout) to facilitate reuse of the box data (e.g., shipout/firstpage material should never be added to a later page of the output).

\l_shipout_box_ht_dim
\l_shipout_box_dp_dim
\l_shipout_box_wd_dim
\l_shipout_box_ht_plus_dp_dim

The shipout box dimensions are available in the L3 registers \l_shipout_box_ht_dim, etc. (there are no LaTeX 2_ε names).[57] These variables can be used inside the hook code for shipout/before, shipout/foreground and shipout/background if needed.

1.2 Provided hooks

shipout/before
shipout/after
shipout/foreground
shipout/background
shipout/firstpage
shipout/lastpage

The code for \shipout offers a number of hooks into which packages (or the user) can add code to support different use cases. These are:

shipout/before This hook is executed after the finished page has been stored in \ShipoutBox / \l_shipout_box). It can be used to alter that box content or to discard it completely (see \DiscardShipoutBox below).

You can use \RawShipout inside this hook for special use cases. It can make use of \ShipoutBox (which doesn't yet include the background and foreground material).

Note: It is not possible (or say advisable) to try and use this hook to typeset material with the intention to return it to main vertical list, it will go wrong and give unexpected results in many cases—for starters it will appear after the current page not before or it will vanish or the vertical spacing will be wrong!

shipout/background This hook adds a picture environment into the background of the page with the (0,0) coordinate in the top-left corner using a \unitlength of 1pt.

It should therefore only receive \put commands or other commands suitable in a picture environment and the vertical coordinate values would normally be negative.

[57] Might need changing, but HO's version as strings is not really helpful I think).

Technically this is implemented by adding a zero-sized `\hbox` as the very first item into the `\ShipoutBox` containing that `picture` environment. Thus the rest of the box content will overprint what ever is typeset by that hook.

`shipout/foreground` This hook adds a picture environment into the foreground of the page with the `(0,0)` coordinate in the top-left corner using a `\unitlength` of `1pt`.

Technically this is implemented by adding a zero-sized `\hbox` as the very last item into the `\ShipoutBox` and raising it up so that it still has its `(0,0)` point in the top-left corner. But being placed after the main box content it will be typeset later and thus overprints it (i.e., is in the foreground).

`shipout` This hook is executed after foreground and/or background material has been added, i.e., just in front of the actual shipout operation. Its purpose is to allow manipulation of the finalized box (stored in `\ShipoutBox`) with the extra material also in place (which is not yet the case in `shipout/before`).

It cannot be used to cancel the shipout operation via `\DiscardShipoutBox` (that has to happen in `shipout/before`, if desired!

`shipout/firstpage` The material from this hook is executed only once at the very beginning of the first output page that is shipped out (i.e., not discarded at the last minute). It should only contain `\special` or similar commands needed to direct post processors handling the `.dvi` or `.pdf` output.[58]

This hook is added to the very first page regardless of how it is shipped out (i.e., with `\shipout` or `\RawShipout`).

`shipout/lastpage` The corresponding hook to add `\special`s at the very end of the output file. It is only executed on the very last page of the output file — or rather on the page that LaTeX believes is the last one. Again it is executed regardless of the shipout method.

It may not be possible for LaTeX to correctly determine which page is the last one without several reruns. If this happens and the hook is non-empty then LaTeX will add an extra page to place the material and also request a rerun to get the correct placement sorted out.

`shipout/after` This hook is executed after a shipout has happened. If the shipout box is discarded this hook is not looked at.

You can use `\RawShipout` inside this hook for special use cases and the main `\ShipoutBox` is still available at this point (but in contrast to `shipout/before` it now includes the background and foreground material).

Note: Just like `shipout/before` this hook is not meant to be used for adding typeset material back to the main vertical list—it might vanish or the vertical spacing will be wrong!

As mentioned above the hook `shipout/before` is executed first and can manipulate the prepared shipout box stored in `\ShipoutBox` or set things up for use in `\write` during the actual shipout. It is even run if there was a `\DiscardShipoutBox` request in the document.

The other hooks (except `shipout` and `shipout/after`) are added inside hboxes to the box being shipped out in the following order:

[58] In LaTeX 2_ε that was already existing, but implemented using a box register with the name `\@begindvibox`.

`shipout/firstpage`	only on the first page
`shipout/background`	
⟨*boxed content of* `\ShipoutBox`⟩	
`shipout/foreground`	
`shipout/lastpage`	only on the last page

If any of the hooks has no code then the corresponding box is added at that point.

Once the (page) box has got the above extra content it can again be manipulated using the `shipout` hook and then is shipped out for real.

Once the (page) box has been shipped out the `shipout/after` hook is called (while you are still inside the output routine). It is not called if the shipout box was discarded.

In a document that doesn't produce pages, e.g., only makes `\typeout`s, none of the hooks are ever executed (as there is no `\shipout`) not even the `shipout/lastpage` hook.

If `\RawShipout` is used instead of `\shipout` then only the hooks `shipout/firstpage` and `shipout/lastpage` are executed (on the first or last page), all others are bypassed.

1.3 Legacy LaTeX commands

`\AtBeginDvi` `\AtBeginDvi {`⟨*code*⟩`}`
`\AtEndDvi`

`\AtBeginDvi` is the existing LaTeX 2_ε interface to fill the `shipout/firstpage` hook. This is not really a good name as it is not just supporting `.dvi` but also `.pdf` output or `.xdv`.

`\AtEndDvi` is the counterpart that was not available in the kernel but only through the package `atenddvi`. It fills the `shipout/lastpage` hook.

Neither interface can set a code label but uses the current default label.

As these two wrappers have been available for a long time we continue offering them (but not enhancing them, e.g., by providing support for code labels).

For new code we strongly suggest using the high-level hook management commands directly instead of "randomly-named" wrappers. This will lead to code that is easier to understand and to maintain and it also allows you to set code labels if needed.

For this reason we do not provide any other "new" wrapper commands for the above hooks in the kernel, but only keep the existing ones for backward compatibility.

1.4 Special commands for use inside the hooks

\DiscardShipoutBox
\shipout_discard:

`\AddToHookNext {shipout/before} {...\DiscardShipoutBox...}`

The `\DiscardShipoutBox` declaration (L3 name `\shipout_discard:`) requests that on the next shipout the page box is thrown away instead of being shipped to the `.dvi` or `.pdf` file.

Typical applications wouldn't do this unconditionally, but have some processing logic that decides to use or not to use the page.

Note that if this declaration is used directly in the document it may depend on the placement to which page it applies, given that LaTeX output routine is called in an asynchronous manner! Thus normally one would use this only as part of the `shipout/before` code.

> *Todo: Once we have a new mark mechanism available we can improve on that and make sure that the declaration applies to the page that contains it — not done (yet)*

`\DiscardShipoutBox` cannot be used in any of the `shipout/...` hooks other than `shipout/before`.

In the atbegshi package there are a number of additional commands for use inside the `shipout/before` hook. They should normally not be needed any more as one can instead simply add code to the hooks `shipout/before`, `shipout`, `shipout/background` or `shipout/foreground`.[59] If atbegshi gets loaded then those commands become available as public functions with their original names as given below.

1.5 Provided LuaTeX callbacks

pre_shipout_filter

Under LuaTeX the `pre_shipout_filter` Lua callback is provided which gets called directly after the `shipout` hook, immediately before the shipout primitive gets invoked. The signature is

```
function(<node> head)
  return true
end
```

The `head` is the list node corresponding to the box to be shipped out. The return value should always be `true`.

[59] If that assumption turns out to be wrong it would be trivial to change them to public functions (right now they are private).

1.6 Information counters

\ReadonlyShipoutCounter `\ifnum\ReadonlyShipoutCounter=...`
\g_shipout_readonly_int `\int_use:N \g_shipout_readonly_int % expl3 usage`

This integer holds the number of pages shipped out up to now (including the one to be shipped out when inside the output routine). More precisely, it is incremented only after it is clear that a page will be shipped out, i.e., after the `shipout/before` hook (because that might discard the page)! In contrast `shipout/after` sees the incremented value.

Just like with the `page` counter its value is only accurate within the output routine. In the body of the document it may be off by one as the output routine is called asynchronously!

Also important: it *must not* be set, only read. There are no provisions to prevent that restriction, but if you manipulate it, chaos will be the result. To emphasize this fact it is not provided as a LaTeX counter but as a TeX counter (i.e., a command), so `\Alph{\ReadonlyShipoutCounter}` etc, would not work.

totalpages `\arabic{totalpages}`
\g_shipout_totalpages_int `\int_use:N \g_shipout_totalpage_int % expl3 usage`

In contrast to `\ReadonlyShipoutCounter`, the `totalpages` counter is a LaTeX counter and incremented for each shipout attempt including those pages that are discarded for one or the other reason. Again `shipout/before` sees the counter before it is incremented. In contrast `shipout/after` sees the incremented value.

Furthermore, while it is incremented for each page, its value is never used by LaTeX. It can therefore be freely reset or changed by user code, for example, to additionally count a number of pages that are not build by LaTeX but are added in a later part of the process, e.g., cover pages or picture pages made externally.

Important: as this is a page-related counter its value is only reliable inside the output routine!

\PreviousTotalPages `\PreviousTotalPages`

Command that expands to the number of total pages from the previous run. If there was no previous run or if used in the preamble it expands to `0`. Note that this is a command and not a counter, so in order to display the number in, say, Roman numerals you have to assign its value to a counter and then use `\Roman` on that counter.

1.7 Debugging shipout code

\DebugShipoutsOn `\DebugShipoutsOn`
\DebugShipoutsOff
\shipout_debug_on:
\shipout_debug_off:

Turn the debugging of shipout code on or off. This displays changes made to the shipout data structures.

Todo: This needs some rationalizing and may not stay this way.

2 Emulating commands from other packages

The packages in this section are no longer necessary, but as they are used by other packages, they are emulated when they are explicitly loaded with `\usepackage` or `\RequirePackage`.

Please note that the emulation only happens if the package is explicitly requested, i.e., the commands documented below are not automatically available in the LaTeX kernel! If you write a new package we suggest to use the appropriate kernel hooks directly instead of loading the emulation.

2.1 Emulating atbegshi

`\AtBeginShipoutUpperLeft`
`\AtBeginShipoutUpperLeftForeground`

`\AddToHook {shipout/before} {...\AtBeginShipoutUpperLeft{`⟨*code*⟩`}...}`

This adds a `picture` environment into the background of the shipout box expecting ⟨*code*⟩ to contain `picture` commands. The same effect can be obtained by simply using kernel features as follows:

```
\AddToHook{shipout/background}{⟨code⟩}
```

There is one technical difference: if `\AtBeginShipoutUpperLeft` is used several times each invocation is put into its own box inside the shipout box whereas all ⟨*code*⟩ going into `shipout/background` ends up all in the same box in the order it is added or sorted based on the rules for the hook chunks.

`\AtBeginShipoutUpperLeftForeground` is similar with the difference that the `picture` environment is placed in the foreground. To model it with the kernel functions use the hook `shipout/foreground` instead.

`\AtBeginShipoutAddToBox`
`\AtBeginShipoutAddToBoxForeground`

`\AddToHook {shipout/before} {...\AtBeginShipoutAddToBox{`⟨*code*⟩`}...}`

These work like `\AtBeginShipoutUpperLeft` and `\AtBeginShipoutUpperLeftForeground` with the difference that ⟨*code*⟩ is directly placed into an `\hbox` inside the shipout box and not surrounded by a `picture` environment.

To emulate them using `shipout/background` or `shipout/foreground` you may have to wrap ⟨*code*⟩ into a `\put` statement but if the code is not doing any typesetting just adding it to the hook should be sufficient.

`\AtBeginShipoutBox` This is the name of the shipout box as atbegshi knows it.

`\AtBeginShipoutOriginalShipout`

This is the name of the `\shipout` primitive as atbegshi knows it. This bypasses all the mechanisms set up by the LaTeX kernel and there are various scenarios in which it can therefore fail. It should only be used to run existing legacy atbegshi code but not in newly developed applications.

The kernel alternative is `\RawShipout` which is integrated with the LaTeX mechanisms and updates, for example, the `\ReadonlyShipoutCounter` counter. Please use `\RawShipout` for new code if you want to bypass the before, foreground and background hooks.

\AtBeginShipoutInit By default atbegshi delayed its action until \begin{document}. This command was forcing it in an earlier place. With the new concept it does nothing.

\AtBeginShipout `\AtBeginShipout{`⟨*code*⟩`}` ≡ `\AddToHook{shipout/before}{`⟨*code*⟩`}`
\AtBeginShipoutNext `\AtBeginShipoutNext{`⟨*code*⟩`}` ≡ `\AddToHookNext{shipout/before}{`⟨*code*⟩`}`

This is equivalent to filling the `shipout/before` hook by either using `\AddToHook` or `\AddToHookNext`, respectively.

\AtBeginShipoutFirst
\AtBeginShipoutDiscard
The atbegshi names for `\AtBeginDvi` and `\DiscardShipoutBox`.

2.2 Emulating everyshi

The everyshi package is providing commands to run arbitrary code just before the shipout starts. One point of difference: in the new shipout hooks the page is available as `\ShipoutBox` for inspection of change, one should not manipulate box 255 directly inside `shipout/before`, so old code doing this would change to use `\ShipoutBox` instead of 255 or `\@cclv`.

\EveryShipout `\EveryShipout{`⟨*code*⟩`}` ≡ `\AddToHook{shipout/before}{`⟨*code*⟩`}`

\AtNextShipout `\AtNextShipout{`⟨*code*⟩`}` ≡ `\AddToHookNext{shipout/before}{`⟨*code*⟩`}`

However, most use cases for everyshi are attempts to put some picture or text into the background or foreground of the page and that can be done today simply by using the `shipout/background` and `shipout/foreground` hooks without any need to coding.

2.3 Emulating atenddvi

The atenddvi package implemented only a single command: `\AtEndDvi` and that is now available out of the box so the emulation makes the package a no-op.

2.4 Emulating everypage

This package patched the original `\@begindvi` hook and replaced it with its own version. Its functionality is now covered by the hooks offered by the kernel so that there is no need for such patching any longer.

\AddEverypageHook `\AddEverypageHook{`⟨*code*⟩`}` ≡
`\AddToHook{shipout/background}{\put(1in,-1in){`⟨*code*⟩`}}`

`\AddEverypageHook` is adding something into the background of every page at a position of 1in to the right and 1in down from the top left corner of the page. By using the kernel hook directly you can put your material directly to the right place, i.e., use other coordinates in the `\put` statement above.

\AddThispageHook `\AddThispageHook{`⟨*code*⟩`}` ≡
`\AddToHookNext{shipout/background}{\put(1in,-1in){`⟨*code*⟩`}}`

The `\AddThispageHook` wrapper is similar but uses `\AddToHookNext`.

3 The Implementation

```
⟨@@=shipout⟩
```

At the moment most the whole module rolls back in one go, but if we make any modifications in later releases this will then need splitting.

```
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2026/11/01}%
⟨latexrelease⟩                    {\__shipout_picture_overlay:n}{Link handling in overlay pictures}
\ExplSyntaxOn
```

`\__shipout_picture_overlay:n` Put the argument into a `picture` environment that doesn't take up any size and uses `1pt` for `\unitlength`.

> *Todo: Could perhaps be generalized as it might be useful elsewhere. For now it is not.*

We want to avoid that links in the main galley which break over a page spills over into the picture (gh/2210). The needed code is different for lualatex and other engines and also must be inserted in different places: in vmode for lualatex and in hmode else. The code only works if the PDF management is loaded.

```
\cs_new:Npn \__shipout_picture_overlay_linkon:    {}
\cs_new:Npn \__shipout_picture_overlay_linkoff:   {}
\cs_new:Npn \__shipout_picture_overlay_linkstate: {}
```

We need to wait until we know if the PDF management has been loaded.

```
\AddToHookNext{class/before}
 {
   \IfPDFManagementActiveT
    {
      \sys_if_engine_luatex:TF
        {
          \cs_set:Npn \__shipout_picture_overlay_linkstate: {\pdfextension linkstate-3}
        }
        {
          \cs_set_protected:Npn \__shipout_picture_overlay_linkon:  {\pdfannot_link_on:}
          \cs_set_protected:Npn \__shipout_picture_overlay_linkoff: {\pdfannot_link_off:}
        }
    }
  }
\cs_new:Npn \__shipout_picture_overlay:n #1 {
```

The very first time this is executed we have to initialize (and freeze) the origins.

```
    \__shipout_init_page_origins:
    \kern -\c__shipout_horigin_tl \scan_stop:
    \vbox_to_zero:n {
      \kern -\c__shipout_vorigin_tl \scan_stop:
      \unitlength 1pt \scan_stop:
      \__shipout_picture_overlay_linkstate:
```

This mimics a simple zero-sized picture environment. The `\hss` is need in case there is horizontal material (without using `\put` with a positive width.

```
      \hbox_set_to_wd:Nnn \l__shipout_tmp_box \c_zero_dim
        {
          \__shipout_picture_overlay_linkoff:
```

```
          \ignorespaces #1 \hss
          \__shipout_picture_overlay_linkon:
        }
      \box_set_ht:Nn \l__shipout_tmp_box \c_zero_dim
      \box_set_dp:Nn \l__shipout_tmp_box \c_zero_dim
      \box_use:N \l__shipout_tmp_box
      \tex_vss:D
    }
}
```

(*End of definition for* `\__shipout_picture_overlay:n`.)

```
\ExplSyntaxOff
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\__shipout_picture_overlay:n}{Link handling in overlay pictures}
⟨latexrelease⟩\ExplSyntaxOn
⟨latexrelease⟩\cs_new:Npn \__shipout_picture_overlay:n #1 {
⟨latexrelease⟩    \__shipout_init_page_origins:
⟨latexrelease⟩    \kern -\c__shipout_horigin_tl \scan_stop:
⟨latexrelease⟩    \vbox_to_zero:n {
⟨latexrelease⟩      \kern -\c__shipout_vorigin_tl \scan_stop:
⟨latexrelease⟩      \unitlength 1pt \scan_stop:
⟨latexrelease⟩      \hbox_set_to_wd:Nnn \l__shipout_tmp_box \c_zero_dim
⟨latexrelease⟩                          { \ignorespaces #1 \hss }
⟨latexrelease⟩      \box_set_ht:Nn \l__shipout_tmp_box \c_zero_dim
⟨latexrelease⟩      \box_set_dp:Nn \l__shipout_tmp_box \c_zero_dim
⟨latexrelease⟩      \box_use:N \l__shipout_tmp_box
⟨latexrelease⟩      \tex_vss:D
⟨latexrelease⟩    }
⟨latexrelease⟩}
⟨latexrelease⟩\ExplSyntaxOff
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\shipout}{Hook management (shipout)}%
\ExplSyntaxOn
```

3.1 Debugging

`\g__shipout_debug_bool` Holds the current debugging state.

```
\bool_new:N \g__shipout_debug_bool
```

(*End of definition for* `\g__shipout_debug_bool`.)

`\shipout_debug_on:` `\shipout_debug_off:` `\__shipout_debug:n` `\__shipout_debug_gset:` Turns debugging on and off by redefining `\__shipout_debug:n`.

```
\cs_new_eq:NN \__shipout_debug:n  \use_none:n
\cs_new_protected:Npn \shipout_debug_on:
  {
    \bool_gset_true:N \g__shipout_debug_bool
    \__shipout_debug_gset:
  }
\cs_new_protected:Npn \shipout_debug_off:
```

```
  {
    \bool_gset_false:N \g__shipout_debug_bool
    \__shipout_debug_gset:
  }
\cs_new_protected:Npn \__shipout_debug_gset:
  {
    \cs_gset_protected:Npe \__shipout_debug:n ##1
      { \bool_if:NT \g__shipout_debug_bool {##1} }
  }
```

(*End of definition for* `\shipout_debug_on:` *and others. These functions are documented on page* 1189.)

`\ShipoutBox` `\l_shipout_box` The box filled with the page to be shipped out (both L3 and LaTeX 2_ε name).

```
\box_new:N  \l_shipout_box
\cs_set_eq:NN \ShipoutBox \l_shipout_box
```

(*End of definition for* `\ShipoutBox` *and* `\l_shipout_box`. *These functions are documented on page* 1185.)

`\l__shipout_raw_box` The `\RawShipout` gets its own box but it is internal as there is no hook manipulation for it.

```
\box_new:N  \l__shipout_raw_box
```

(*End of definition for* `\l__shipout_raw_box`.)

`\__shipout_finalize_box:` For LuaTeX invoke the `pre_shipout_filter` callback.

```
\sys_if_engine_luatex:TF
  {
    \newprotectedluacmd \__shipout_finalize_box:
    \exp_args:Ne \everyjob {
      \exp_not:V \everyjob
      \exp_not:N \lua_now:n {
        luatexbase.create_callback('pre_shipout_filter', 'list')
        local~call, getbox, setbox = luatexbase.call_callback, tex.getbox, tex.setbox~
        lua.get_functions_table()[\the \allocationnumber] = function()
          local~head = getbox(\the \l_shipout_box)
          local~result = call('pre_shipout_filter', head)
          if~not (result == head) then~
            setbox(\the \l_shipout_box, result~or~nil)
          end~
        end
      }
    }
  } {
    \cs_set_eq:NN \__shipout_finalize_box: \scan_stop:
  }
```

(*End of definition for* `\__shipout_finalize_box:`.)

`\__shipout_execute:` This is going to the be the code run by `\shipout`. The code follows closely the ideas from atbegshi, so not documenting that here for now.

```
\cs_set_protected:Npn \__shipout_execute: {
  \tl_set:Ne \l__shipout_group_level_tl
     { \int_value:w \tex_currentgrouplevel:D }
```

```
  \tex_afterassignment:D \__shipout_execute_test_level:
  \tex_setbox:D \l_shipout_box
}
```

(*End of definition for* `\__shipout_execute:`.)

`\shipout` Overloading the `\shipout` primitive:

```
\cs_gset_eq:NN \shipout \__shipout_execute:
```

(*End of definition for* `\shipout`. *This function is documented on page 1184.*)

`\l__shipout_group_level_tl` Helper token list to record the group level at which `\__shipout_execute:` is encountered.

```
\tl_new:N \l__shipout_group_level_tl
```

(*End of definition for* `\l__shipout_group_level_tl`.)

`\__shipout_execute_test_level:` If the group level has changed then we are still constructing `\l_shipout_box` and to continue we need to wait until the current group has finished, hence the `\tex_aftergroup:D`.

```
\cs_new:Npn \__shipout_execute_test_level: {
  \int_compare:nNnT
     \l__shipout_group_level_tl < \tex_currentgrouplevel:D
     \tex_aftergroup:D \__shipout_execute_cont:
}
```

(*End of definition for* `\__shipout_execute_test_level:`.)

`\__shipout_execute_cont:` This does the actual shipout running several hooks as part of it. The code for them is passed as argument `#2` to `#4` to `\__shipout_execute_main_cont:Nnnn`; the first argument is the box to be shipped out.

```
\cs_new:Npn \__shipout_execute_cont: {
  \__shipout_execute_main_cont:Nnnn
     \l_shipout_box
     { \hook_use:n {shipout/before} }
     { \hook_if_empty:nF {shipout/foreground}
         { \__shipout_add_foreground_picture:n
             { \hook_use:n {shipout/foreground} } }
```

If the user hook for the background (`shipout/background`) has no code, there might still code in the kernel hook so we need to test for this too. We only test for the `\@kernel@before@shipout@background` though. If the `\@kernel@after@shipout@background` needs executing even if the user hook is empty then we can add another test (or the kernel could put something into the before hook).

```
       \bool_lazy_and:nnF
         { \hook_if_empty_p:n {shipout/background} }
         { \tl_if_empty_p:N \@kernel@before@shipout@background }
         { \__shipout_add_background_picture:n
           { \@kernel@before@shipout@background
             \hook_use:n {shipout/background}
             \@kernel@after@shipout@background }
         }
     }
     { \hook_use:n {shipout/after} }
}
```

(*End of definition for* `\__shipout_execute_cont:`.)

__shipout_execute_main_cont:Nnnn When we have reached this point the shipout box has been processed and is available in \l_shipout_box and ready for real ship out (unless it gets discarded during the process).

The three arguments hold hook code that is executed just before the actual shipout (#1), within the shipout adding background and foreground material (#2) and after the shipout has happened (#3). These are passed as arguments because the same code without those hooks is also used when doing a "raw" shipout implemented by \RawShipout. The only hook that is always executed is that for the very last page, i.e., shipout/lastpage.

First we quickly check if it is void (can't happen in the standard LaTeX output routine but \shipout might be called from a package that has some special processing logic). If it is void we aren't shipping anything out and processing ends.[60]

```
\cs_new:Npn \__shipout_execute_main_cont:Nnnn #1#2#3#4 {
  \box_if_empty:NTF #1
    { \@latex@warning@no@line{ Ignoring~ void~ shipout~ box } }
    {
```

Otherwise we assume that we will ship something and prepare for final adjustments (in particular setting the state of \protect while we are running the hook code). We also save the current \protect state to restore it later.

```
%      \bool_gset_false:N \g__shipout_discard_bool  % setting this would disable
                                                    % \DiscardShipoutBox on doc-level
      \cs_set_eq:NN \__shipout_saved_protect: \protect
      \set@typeset@protect
```

We also store the current shipout box dimension in registers, so that they can be used in the hook code.[61]

```
      \__shipout_get_box_size:N #1
```

Then we execute the shipout/before hook (or nothing in case of \RawShipout).

```
      #2
```

In \g_shipout_totalpages_int we count all shipout attempts so we increment that counter already here (the other one is incremented later when we know for sure that we do a \shipout.

We increment it after running the above hook so that the values for \g_shipout_totalpages_int and \g_shipout_readonly_int are in sync while the hook is executed (in the case that totalpages isn't manually altered or through discarding pages that is).

```
      \int_gincr:N \g_shipout_totalpages_int
```

The above hook might contain code that requests the page to be discarded so we now test for it.

```
      \bool_if:NTF \g__shipout_discard_bool
        { \@latex@info@no@line{Completed~ page~ discarded}
          \bool_gset_false:N \g__shipout_discard_bool
```

As we are discarding the page box and not shipping anything out, we need to do some house cleaning and reset TeX's deadcycles so that it doesn't complain about too many calls to the OR without any shipout.

```
          \tex_deadcycles:D \c_zero_int
```

[60] In that case we don't reset the deadcycles, that would be up to the OR processing logic to do.

[61] This is not really necessary as the code could access them via \box_ht:N, etc., but it is perhaps convenient.

Todo: In atbegshi the box was dropped but is that actually needed? Or the resetting of `\protect` *to its kernel value?*

```
%           \group_begin:
%             \box_set_eq_drop:NN #1 #1
%           \group_end:
%           \cs_set_eq:NN \protect \exp_not:N
        }
```

Even if there was no explicit request to discard the box it is possible that the code for the hook `shipout/before` has voided the box (by mistake or deliberately). We therefore test once more but this time make it a warning, because the best practice way is to use the request mechanism.

```
        { \box_if_empty:NTF #1
            { \@latex@warning@no@line { Ignoring~ void~ shipout~ box.
                 \MessageBreak The~ shipout~ box~ was~ voided~ by~ hook~ code }
            }
```

Finally, if the box is still non-empty we are nearly ready to ship it out. First we increment the total page counter so that we can later test if we have reached the final page according to our available information.[62]

```
            {
              \int_gincr:N \g_shipout_readonly_int
              \__shipout_debug:n {
                \typeout{Absolute~ page~ =~ \int_use:N \g_shipout_readonly_int
                         \space (target:~ \@abspage@last)}
              }
```

Then we store the box sizes again (as they may have changed) and then look at the hooks `shipout/foreground` and `shipout/background`. If either or both are non-empty we add a `picture` environment to the box (in the foreground and/or in the background) and execute the hook code inside that environment.

```
              \__shipout_get_box_size:N #1
```

The first hook we run is the `shipout/firstpage` hook. This is only done once, then the `\__shipout_run_firstpage_hook:` command redefines itself to do nothing. If the hook contains `\special`s for integration at the top of the page they will be temporarily stored in a safe place and added later with `\__shipout_add_firstpage_specials:`.

```
              \__shipout_run_firstpage_hook:
```

Run the hooks for background and foreground or, if this is called by `\RawShipout`, copy the box `\l__shipout_raw_box` to `\l_shipout_box` so that firstpage and lastpage material gets added if necessary (that is always done to `\l_shipout_box`.

```
              #3
```

We then run `\__shipout_add_firstpage_specials:` that adds the content of the hook `shipout/firstpage` to the start of the first page (if non-empty). It is then redefined to do nothing on later pages.

```
              \__shipout_add_firstpage_specials:
```

[62] Doing that earlier would be wrong because we might end up with the last page counted but discard and then we have no place to add the final objects into the output file.

Then we check if we have to add the `shipout/lastpage` hook or the corresponding kernel hook because we have reached the last page. This test will be false for all but one (and hopefully the correct) page.

```
\int_compare:nNnT \@abspage@last = \g_shipout_readonly_int
  { \bool_lazy_and:nnF
      { \hook_if_empty_p:n {shipout/lastpage} }
      { \tl_if_empty_p:N \@kernel@after@shipout@lastpage }
      { \__shipout_debug:n { \typeout{Executing~ lastpage~ hook~
            on~ page~ \int_use:N \g_shipout_readonly_int } }
        \__shipout_add_foreground_box:n
            { \UseHook{shipout/lastpage}
              \@kernel@after@shipout@lastpage }
```

We record that we have handled the `shipout/lastpage` hook but only if we really did.

```
            \bool_gset_true:N \g__shipout_lastpage_handled_bool
        }
  }

\hook_use:n {shipout}
\__shipout_finalize_box:
```

Finally we run the actual TEX primitive for shipout. As that will expand delayed `\write` statements inside the page in which protected commands should not expand we first change `\protect` to the appropriate definition for that case.

```
\cs_set_eq:NN \protect \exp_not:N
\tex_shipout:D \box_use:N \l_shipout_box
```

The `\l_shipout_box` may contain the firstpage material if this was the very first shipout. That makes it unsuitable for reuse in another shipout, so as a safety measure the next command resets `\l_shipout_box` to its earlier state if that is necessary. On later pages this is then a no-op.

```
\__shipout_drop_firstpage_specials:
```

The `shipout/after` hook (if in `#4`) needs to run with `\protect`ed commands again being executed, because that hook will "typeset" material added at the top of the next page.

```
            \set@typeset@protect
            #4
          }
      }
```

Restore the value of `\protect` in case `\shipout` is called outside of the output routine (where it is automatically restored because of the implicit group).

```
    \cs_set_eq:NN \protect \__shipout_saved_protect:
  }
}
```

(*End of definition for* `\__shipout_execute_main_cont:Nnnn`.)

`\__shipout_execute_raw:`
`\__shipout_execute_test_level_raw:`

This implements the "raw" shipout which bypasses the before, foreground, background and after hooks. It follows the same pattern than `\__shipout_execute_raw:` except that it finally calls `\__shipout_execute_main_cont:Nnnn` with three empty arguments. instead of the hook code.

```
\cs_set_protected:Npn \__shipout_execute_raw: {
  \tl_set:Ne \l__shipout_group_level_tl
     { \int_value:w \tex_currentgrouplevel:D }
```

```
  \tex_afterassignment:D \__shipout_execute_test_level_raw:
  \tex_setbox:D \l__shipout_raw_box
}
\cs_new:Npn \__shipout_execute_test_level_raw: {
  \int_compare:nNnT
     \l__shipout_group_level_tl < \tex_currentgrouplevel:D
     \tex_aftergroup:D \__shipout_execute_nohooks_cont:
}
```

Well, not totally empty arguments, we add some debugging if we are actually doing a shipout.

```
\cs_new:Npn \__shipout_execute_nohooks_cont: {
  \__shipout_execute_main_cont:Nnnn \l__shipout_raw_box
     {} { \__shipout_debug:n{ \typeout{Doing~ raw~ shipout~ ...} }
          \box_set_eq:NN \l_shipout_box \l__shipout_raw_box } {}
}
```

(*End of definition for* `\__shipout_execute_raw:` *and* `\__shipout_execute_test_level_raw:`.)

`\RawShipout` The interface name for raw shipout.

```
\cs_gset_eq:NN \RawShipout \__shipout_execute_raw:
```

(*End of definition for* `\RawShipout`. *This function is documented on page 1184.*)

`\__shipout_saved_protect:` Remember the current `\protect` state.

```
\cs_new_eq:NN  \__shipout_saved_protect: \protect
```

(*End of definition for* `\__shipout_saved_protect:`.)

`shipout/before` `shipout` `shipout/after` `shipout/foreground` `shipout/background` `shipout/firstpage` `shipout/lastpage` Declaring all hooks for the shipout code.

```
\hook_new:n{shipout/before}
\hook_new:n{shipout}
\hook_new:n{shipout/after}
\hook_new:n{shipout/foreground}
\hook_new:n{shipout/background}
\hook_new:n{shipout/firstpage}
\hook_new:n{shipout/lastpage}
```

(*End of definition for* `shipout/before` *and others. These functions are documented on page 1185.*)

`\@kernel@after@shipout@lastpage` `\@kernel@before@shipout@background` `\@kernel@after@shipout@background` And here are the internal kernel hooks going before or after the public ones where needed.

```
\let\@kernel@after@shipout@lastpage\@empty
\let\@kernel@before@shipout@background\@empty
\let\@kernel@after@shipout@background\@empty
```

(*End of definition for* `\@kernel@after@shipout@lastpage`, `\@kernel@before@shipout@background`, *and* `\@kernel@after@shipout@background`.)

__shipout_run_firstpage_hook:

There are three commands to handle the `shipout/firstpage` hook: `\__shipout_run_firstpage_hook:`, `\__shipout_add_firstpage_specials:` and `\__shipout_drop_firstpage_specials:`.

That hook is supposed to contain `\special`s and similar material to be placed at the very beginning of the output page and so it needs careful placing to avoid that anything else gets in front of it. And this means we have to wait with this until other hooks such as `shipout/background` have added their bits. It is also important that such `\special`s show up only on the very first page, so if this page gets saved before `\shipout` for later reuse, we have to make sure that they aren't in the saved version.

In addition the hook may also contain code to be executed "first", e.g., visible from code in `shipout/background` and this conflicts with adding the `\special`s late.

Therefore the processing is split into different parts: `\__shipout_run_firstpage_hook:` is done early and checks if there is any material in the hook.

```
\cs_new:Npn \__shipout_run_firstpage_hook: {
  \hook_if_empty:nTF {shipout/firstpage}
```

If not then we define the other two commands to do nothing.

```
      {
        \cs_gset_eq:NN \__shipout_add_firstpage_specials:  \prg_do_nothing:
        \cs_gset_eq:NN \__shipout_drop_firstpage_specials: \prg_do_nothing:
      }
```

If there is material we execute inside a box, which means any `\special` will end up in that box and any other code is executed and can have side effects (as long as they are global).

```
      {
        \hbox_set:Nn \l__shipout_firstpage_box { \UseHook{shipout/firstpage} }
      }
```

Once we are here we change the definition to do nothing next time and we also change the command used to implement `\AtBeginDvi` to become a warning and not add further material to a hook that is never used again.

```
  \cs_gset_eq:NN \__shipout_run_firstpage_hook: \prg_do_nothing:
  \cs_gset:Npn \__shipout_add_firstpage_material:Nn ##1 ##2 {
    \@latex@warning{ First~ page~ is~ already~ shipped~ out,~ ignoring
                     \MessageBreak \string##1 }
  }
}
```

(*End of definition for* `\__shipout_run_firstpage_hook:`.)

__shipout_add_firstpage_specials:
__shipout_drop_firstpage_specials:

The `\__shipout_add_firstpage_specials:` then adds the `\special`s stored in `\l__shipout_firstpage_box` to the page to be shipped out when the time is ready. Note that if there was no material in the `shipout/firstpage` hook then this command gets redefined to do nothing. But for most documents there is something, e.g., some PostScript header, or some metadata declaration, etc. so by default we assume there is something to do.

```
\cs_new:Npn \__shipout_add_firstpage_specials: {
```

First we make a copy of the `\l_shipout_box` that we can restore it later on.

```
  \box_set_eq:NN \l__shipout_raw_box \l_shipout_box
```

Adding something to the beginning means adding it to the background as that layer is done first in the output.

```
\__shipout_add_background_box:n { \hbox_unpack_drop:N \l__shipout_firstpage_box }
```

After the actual shipout `\__shipout_drop_firstpage_specials:` is run to restore the earlier content of `\l_shipout_box` and then redefines itself again to do nothing.

As a final act we change the definition to do nothing next time.

```
 \cs_gset_eq:NN \__shipout_add_firstpage_specials: \prg_do_nothing:
}
```

The `\__shipout_drop_firstpage_specials:` is run after the shipout has occurred but before the `shipout/afterpage` hook is executed. That is the point where we have to restore the `\ShipoutBox` to its state without the `shipout/firstpage` material.

```
\cs_new:Npn \__shipout_drop_firstpage_specials: {
    \box_set_eq:NN \l_shipout_box \l__shipout_raw_box
```

If there was no such material then `\__shipout_run_firstpage_hook:` will have changed the definition to a no-op already. Otherwise this is what we do here.

```
    \cs_gset_eq:NN \__shipout_drop_firstpage_specials:  \prg_do_nothing:
  }
```

(*End of definition for* `\__shipout_add_firstpage_specials:` *and* `\__shipout_drop_firstpage_specials:`.)

`\l__shipout_firstpage_box` The box to hold any firstpage `\specials`.

```
\box_new:N \l__shipout_firstpage_box
```

(*End of definition for* `\l__shipout_firstpage_box`.)

`\g__shipout_lastpage_handled_bool` A boolean to signal if we have already handled the `shipout/lastpage` hook.

```
\bool_new:N \g__shipout_lastpage_handled_bool
```

(*End of definition for* `\g__shipout_lastpage_handled_bool`.)

`\__shipout_add_firstpage_material:Nn` This command adds material to the `shipout/firstpage` hook. It is used in `\AtBeginDvi`, etc. The first argument is the command through which is it called. Initially this is ignored but once we are passed the first page it can be used to generate a warning message mentioning the right user command.

```
\cs_new:Npn \__shipout_add_firstpage_material:Nn #1#2 {
   \AddToHook{shipout/firstpage}{#2}
}
```

(*End of definition for* `\__shipout_add_firstpage_material:Nn`.)

`\__shipout_get_box_size:N` Store the box dimensions in dimen registers.

Todo: This could/should perhaps be generalized to set height depth and width given an arbitrary box.

```
\cs_new:Npn \__shipout_get_box_size:N #1 {
  \dim_set:Nn \l_shipout_box_ht_dim { \box_ht:N #1 }
  \dim_set:Nn \l_shipout_box_dp_dim { \box_dp:N #1 }
  \dim_set:Nn \l_shipout_box_wd_dim { \box_wd:N #1 }
  \dim_set:Nn \l_shipout_box_ht_plus_dp_dim
      { \l_shipout_box_ht_dim + \l_shipout_box_dp_dim }
}
```

(*End of definition for* `\__shipout_get_box_size:N`.)

`\l_shipout_box_ht_dim`
`\l_shipout_box_dp_dim`
`\l_shipout_box_wd_dim`
`\l_shipout_box_ht_plus_dp_dim`

And here are the variables set by `\__shipout_get_box_size:N`.

```
\dim_new:N \l_shipout_box_ht_dim
\dim_new:N \l_shipout_box_dp_dim
\dim_new:N \l_shipout_box_wd_dim
\dim_new:N \l_shipout_box_ht_plus_dp_dim
```

(*End of definition for* `\l_shipout_box_ht_dim` *and others. These functions are documented on page* *1185*.)

`\g__shipout_discard_bool`

Indicate whether or not the current page box should be discarded

```
\bool_new:N \g__shipout_discard_bool
```

(*End of definition for* `\g__shipout_discard_bool`.)

`\l__shipout_tmp_box`
`\l__shipout_saved_badness_tl`

We need a box for the background and foreground material and a token register to remember badness settings as we disable them during the buildup below.

```
\box_new:N \l__shipout_tmp_box
\tl_new:N  \l__shipout_saved_badness_tl
```

(*End of definition for* `\l__shipout_tmp_box` *and* `\l__shipout_saved_badness_tl`.)

`\__shipout_add_background_box:n`

In standard LaTeX the shipout box is always a `\vbox` but here we are allow for other usage as well, in case some package has its own output routine.

```
\cs_new:Npn \__shipout_add_background_box:n #1
{ \__shipout_get_box_size:N \l_shipout_box
```

But we start testing for a vertical box as that should be the normal case.

```
  \box_if_vertical:NTF \l_shipout_box
      {
```

Save current values of `\vfuzz` and `\vbadness` then change them to allow box manipulations without warnings.

```
        \tl_set:Ne \l__shipout_saved_badness_tl
           { \vfuzz=\the\vfuzz\relax
             \vbadness=\the\vbadness\relax }
        \vfuzz=\c_max_dim
        \vbadness=\c_max_int
```

Then we reconstruct `\l_shipout_box` ...

```
        \vbox_set_to_ht:Nnn \l_shipout_box \l_shipout_box_ht_plus_dp_dim
            {
```

... the material in `#1` is placed into a horizontal box with zero dimensions.

```
              \hbox_set:Nn \l__shipout_tmp_box
                   { \l__shipout_saved_badness_tl #1 }
              \box_set_wd:Nn \l__shipout_tmp_box \c_zero_dim
              \box_set_ht:Nn \l__shipout_tmp_box \c_zero_dim
              \box_set_dp:Nn \l__shipout_tmp_box \c_zero_dim
```

The we typeset that box followed by whatever was in `\l_shipout_box` before (unpacked).

```
              \skip_zero:N \baselineskip
              \skip_zero:N \lineskip
              \skip_zero:N \lineskiplimit
              \box_use:N \l__shipout_tmp_box
              \vbox_unpack:N \l_shipout_box
```

The `\kern` ensures that the box has no depth which is afterwards explicitly corrected.

```
                \kern \c_zero_dim
              }
          \box_set_ht:Nn \l_shipout_box \l_shipout_box_ht_dim
          \box_set_dp:Nn \l_shipout_box \l_shipout_box_dp_dim
```

Todo: The whole boxing maneuver looks a bit like overkill to me, but for the moment I leave.

```
          \l__shipout_saved_badness_tl
        }
        {
```

A horizontal box is handled in a similar way. The last case would be a void box in which case we do nothing hence the missing `F` branch.

```
          \box_if_horizontal:NT \l_shipout_box
              {
                \tl_set:Ne \l__shipout_saved_badness_tl
                   { \hfuzz=\the\hfuzz\relax
                     \hbadness=\the\hbadness\relax }
                \hfuzz=\c_max_dim
                \hbadness=\c_max_int
                \hbox_set_to_wd:Nnn \l_shipout_box \l_shipout_box_wd_dim
                     {
                       \hbox_set:Nn \l__shipout_tmp_box
                            { \l__shipout_saved_badness_tl #1 }
                       \box_set_wd:Nn \l__shipout_tmp_box \c_zero_dim
                       \box_set_ht:Nn \l__shipout_tmp_box \c_zero_dim
                       \box_set_dp:Nn \l__shipout_tmp_box \c_zero_dim
                       \box_move_up:nn
                           \l_shipout_box_ht_dim
                           { \box_use:N \l__shipout_tmp_box }
                       \hbox_unpack:N \l_shipout_box
                     }
                \l__shipout_saved_badness_tl
              }
        }
}
```

(*End of definition for* `\__shipout_add_background_box:n`.)

`\__shipout_add_foreground_box:n` Foreground boxes are done in the same way, only the order and placement of boxes has to be done differently.

```
\cs_new:Npn \__shipout_add_foreground_box:n #1
{
  \box_if_vertical:NTF \l_shipout_box
    {
      \tl_set:Ne \l__shipout_saved_badness_tl
         { \vfuzz=\the\vfuzz\relax
           \vbadness=\the\vbadness\relax }
      \vfuzz=\c_max_dim
      \vbadness=\c_max_int
      \vbox_set_to_ht:Nnn \l_shipout_box \l_shipout_box_ht_plus_dp_dim
           {
```

```
            \hbox_set:Nn \l__shipout_tmp_box
                 { \l__shipout_saved_badness_tl #1 }
            \box_set_wd:Nn \l__shipout_tmp_box \c_zero_dim
            \box_set_ht:Nn \l__shipout_tmp_box \c_zero_dim
            \box_set_dp:Nn \l__shipout_tmp_box \c_zero_dim
            \skip_zero:N \baselineskip
            \skip_zero:N \lineskip
            \skip_zero:N \lineskiplimit
            \vbox_unpack:N \l_shipout_box
            \kern -\l_shipout_box_ht_plus_dp_dim
            \box_use:N \l__shipout_tmp_box
            \kern  \l_shipout_box_ht_plus_dp_dim
          }
      \l__shipout_saved_badness_tl
      \box_set_ht:Nn \l_shipout_box \l_shipout_box_ht_dim
      \box_set_dp:Nn \l_shipout_box \l_shipout_box_dp_dim
    }
    {
      \box_if_horizontal:NT \l_shipout_box
        {
          \tl_set:Ne \l__shipout_saved_badness_tl
            { \hfuzz=\the\hfuzz\relax
              \hbadness=\the\hbadness\relax }
          \hfuzz=\c_max_dim
          \hbadness=\c_max_int
          \hbox_set_to_wd:Nnn \l_shipout_box \l_shipout_box_wd_dim
               {
                 \hbox_unpack:N \l_shipout_box
                 \kern -\box_wd:N \l_shipout_box
                 \hbox_set:Nn \l__shipout_tmp_box
                     { \l__shipout_saved_badness_tl #1 }
                 \box_set_wd:Nn \l__shipout_tmp_box \c_zero_dim
                 \box_set_ht:Nn \l__shipout_tmp_box \c_zero_dim
                 \box_set_dp:Nn \l__shipout_tmp_box \c_zero_dim
                 \box_move_up:nn { \box_ht:N \l_shipout_box }
                                 { \box_use:N \l__shipout_tmp_box }
                 \kern \box_wd:N \l_shipout_box
               }%
               \l__shipout_saved_badness_tl
        }
    }
}
```

(*End of definition for* `\__shipout_add_foreground_box:n`.)

`\__shipout_init_page_origins:`
`\c__shipout_horigin_tl`
`\c__shipout_vorigin_tl`

Two constants holding the offset of the top-left with respect to the media box.

Setting the constants this way is courtesy of Bruno.

We delay setting the constants to the last possible place as there might be updates in the preamble or even in the `begindocument` hook that affects their setup.

```
\cs_new:Npn \__shipout_init_page_origins: {
  \tl_const:Ne \c__shipout_horigin_tl
     {
       \cs_if_exist_use:NTF \pdfvariable { horigin }
          { \cs_if_exist_use:NF \pdfhorigin { 1in } }
```

```
    }
  \tl_const:Ne \c__shipout_vorigin_tl
    {
      \cs_if_exist_use:NTF \pdfvariable { vorigin }
        { \cs_if_exist_use:NF \pdfvorigin { 1in } }
    }
```

After the constants have been set there is no need to execute this command again, in fact it would raise an error, so we redefine it to do nothing.

```
  \cs_gset_eq:NN \__shipout_init_page_origins: \prg_do_nothing:
}
```

(*End of definition for* `\__shipout_init_page_origins:`, `\c__shipout_horigin_tl`, *and* `\c__shipout_vorigin_tl`.)

`\__shipout_add_background_picture:n` Put a `picture` env in the background of the shipout box with its reference point in the top-left corner.

```
\cs_new:Npn \__shipout_add_background_picture:n #1 {
  \__shipout_add_background_box:n { \__shipout_picture_overlay:n {#1} }
}
```

(*End of definition for* `\__shipout_add_background_picture:n`.)

`\__shipout_add_foreground_picture:n` Put a `picture` env in the foreground of the shipout box with its reference point in the top-left corner.

```
\cs_new:Npn \__shipout_add_foreground_picture:n #1 {
  \__shipout_add_foreground_box:n { \__shipout_picture_overlay:n {#1} }
}
```

(*End of definition for* `\__shipout_add_foreground_picture:n`.)

`\shipout_discard:` Request that the next shipout box should be discarded. At the moment this is just setting a boolean, but we may want to augment this behavior that the position of the call is taken into account (in case LaTeX looks ahead and is not using the position for on the next page).

```
\cs_new_protected:Npn \shipout_discard: {
  \bool_gset_true:N \g__shipout_discard_bool
}
```

(*End of definition for* `\shipout_discard:`. *This function is documented on page 1188.*)

3.2 Handling the end of job hook

At the moment this is partly solved by using the existing hooks. But rather than putting the code into these hooks it should be moved to the right place directly as we shouldn't prefill hooks with material unless it needs to interact with other code.

`\g_shipout_readonly_int` `\ReadonlyShipoutCounter` We count every shipout activity that makes a page (but not those that are discarded) in order to know how many pages got produced.

```
\int_new:N \g_shipout_readonly_int
```

For LaTeX 2_ε it is available as a command (i.e., a TeX counter only.

```
\cs_new_eq:NN \ReadonlyShipoutCounter  \g_shipout_readonly_int
```

(*End of definition for* `\g_shipout_readonly_int` *and* `\ReadonlyShipoutCounter`*. These functions are documented on page 1189.*)

`\g_shipout_totalpages_int`
`\c@totalpages`

We count every shipout attempt (even those that are discarded) in this counter. It is not used in the code but may get used in user code.

```
\int_new:N \g_shipout_totalpages_int
```

For LaTeX 2ε this is offered as a LaTeX counter so can be easily typeset inside the output routine to display things like "`\thepage/\thetotalpages`", etc.

```
\cs_new_eq:NN \c@totalpages \g_shipout_totalpages_int
\cs_new:Npn \thetotalpages { \arabic{totalpages} }
```

(*End of definition for* `\g_shipout_totalpages_int` *and* `\c@totalpages`*. These functions are documented on page 1189.*)

`\@abspage@last`

In `\@abspage@last` record the number of pages from the last run. This is written to the `.aux` and this way made available to the next run. In case there is no `.aux` file or the statement is missing from it we initialize it with the largest possible number in TeX. We use this as the default because then we are inserting the `shipout/lastpage` on the last page (or after the last page) but not on page 1 for a multipage document.

```
\xdef\@abspage@last{\number\maxdimen}
```

(*End of definition for* `\@abspage@last`*.*)

`\enddocument`

Instead of using the hooks `enddocument` and `enddocument/afterlastpage` we add this code to private kernel hooks to be 100% sure when it is executed and to avoid cluttering the hooks with data that is always there.

Inside `\enddocument` there is a `\clearpage`. Just before that we execute this code here. There is a good chance that we are on the last page. Therefore, if we don't know the value from the last run, we assume that the current page is the right one. So we set `\@abspage@last` and as a result the next shipout will run the `shipout/lastpage` code. Of course, if there are floats that still need a placement this guess will be wrong but then rerunning the document will give us the correct value next time around.

`\@kernel@after@enddocument`

```
\g@addto@macro \@kernel@after@enddocument {
  \int_compare:nNnT \@abspage@last = \maxdimen
    {
```

We use LaTeX 2ε coding as `\@abspage@last` is not an L3 name.

```
      \xdef\@abspage@last{ \int_eval:n {\g_shipout_readonly_int + 1} }
    }
}
```

`\@kernel@before@enddocument@afterlastpage`

After the last page has been shipped out, we force further `\write` calls to be always `\immediate` because we aren't going to ship out any more pages. This goes before the `enddocument/afterlastpage` hook because that may contain such `\write` commands.

```
\g@addto@macro \@kernel@before@enddocument@afterlastpage {
  \__shipout_force_immediate_writes:
  % This is also the point where \__tag_lastpage_label: could be executed
  % instead of going into enddocument/afterlastpage
}
```

\@kernel@after@enddocument@afterlastpage

Once the `\clearpage` has done its work inside `\enddocument` we know for sure how many pages this document has, so we record that in the `.aux` file for the next run.

```
\g@addto@macro \@kernel@after@enddocument@afterlastpage {
```

There is one special case: If no output is produced then there is no point in a) recording the number as 0 will never match the page number of a real page and b) adding an extra page to ran the `shipout/lastpage` is pointless as well (as it would remain forever). So we test for this and run the code only if there have been pages.

```
  \int_compare:nNnF \g_shipout_readonly_int = 0
    {
```

This ends up in the `.aux` so we use LaTeX 2_ε names here.

Todo: This needs an interface for `\nofiles` *in expl3, doesn't at the moment!*

```
      \if@filesw
         \iow_now:Ne \@auxout {
           \gdef\string\@abspage@last {\int_use:N \g_shipout_readonly_int}}
      \fi
```

But we may have guessed wrongly earlier and have run it too early or we still have to run the `shipout/lastpage` even though there is no page to place it into. If that is the case we make a trivial extra page and put it there. This temporary page will then vanish again on the next run but helps to keep pdf viewers happy. In either case we should put out an appropriate "rerun" warning.

```
       \bool_if:NTF \g__shipout_lastpage_handled_bool
          {
```

If the hook was already executed, we have to test if that total shipouts match the shipouts from last run (because that corresponds to the page it was executed). If not we output a warning.

```
            \int_compare:nNnF \@abspage@last = \g_shipout_readonly_int
             {
               \@latex@warning@no@line{Hook~ 'shipout/lastpage'~ executed~
                 on~ wrong~ page~ (\@abspage@last\space not~
                 \int_use:N\g_shipout_readonly_int).\MessageBreak
                 Rerun~ to~ correct~ this}%
             }
          }
          {
```

If the hook was not run, we need to add an extra page and place it there. However, making this extra page in case the hook is actually empty would be forcing a rerun without any reason, so we check that condition and also check if `\@kernel@after@shipout@lastpage` contains any code. If both are empty we omit the page generation.

```
            \bool_lazy_and:nnF
              { \hook_if_empty_p:n {shipout/lastpage} }
              { \tl_if_empty_p:N \@kernel@after@shipout@lastpage }
              {
                \global\advance\c@page\@ne
                \tex_shipout:D\vbox to\textheight
                  {
                    \hbox:n { \UseHook{shipout/lastpage}
                              \@kernel@after@shipout@lastpage }
```

This extra page could be totally empty except for the hook content, but to help the user understanding why it is there we put some text into it.

```
                    \__shipout_excuse_extra_page:
                    \null
                  }
```

At this point we also signal to LaTeX's endgame that a rerun is necessary so that an appropriate message can be shown on the terminal. We do this by simply defining a command used as a flag and tested in `\enddocument`.

```
                \cs_gset_eq:NN \@extra@page@added \relax
              }
            }
        }
}
```

(*End of definition for* `\enddocument` *and others.*)

`\__shipout_excuse_extra_page:` Say mea culpa ...

```
\cs_new:Npn \__shipout_excuse_extra_page: {
  \vfil
  \begin{center}
    \bfseries Temporary~ page!
  \end{center}
    \LaTeX{}~ was~ unable~ to~ guess~ the~ total~ number~ of~ pages~
    correctly.~ ~ As~ there~ was~ some~ unprocessed~ data~ that~
    should~ have~ been~ added~ to~ the~ final~ page~ this~ extra~
    page~ has~ been~ added~ to~ receive~ it.
    \par
    If~ you~ rerun~ the~ document~ (without~ altering~ it)~ this~
    surplus~ page~ will~ go~ away,~ because~ \LaTeX{}~ now~ knows~
    how~ many~ pages~ to~ expect~ for~ this~ document.
  \vfil
}
```

(*End of definition for* `\__shipout_excuse_extra_page:`.)

`\PreviousTotalPages` In the preamble before the `aux` file was read `\PreviousTotalPages` is always zero.

`\@kernel@before@begindocument`

```
\def\PreviousTotalPages{0}
```

In the `aux` file there should be an update for `\@abspage@last` recording the number of pages from the previous run. If not that macro holds the value of `\maxdimen`. So we test for it and update `\PreviousTotalPages` if there was a real value. This should happen just before the `begindocument` hook is executed so that the value can be used inside that hook.

```
\g@addto@macro\@kernel@before@begindocument
  {\ifnum\@abspage@last<\maxdimen
     \xdef\PreviousTotalPages{\@abspage@last}\fi}
```

(*End of definition for* `\PreviousTotalPages` *and* `\@kernel@before@begindocument`. *These functions are documented on page* 1189.)

4 Legacy LaTeX 2ε interfaces

\DiscardShipoutBox

Request that the next shipout box is to be discarded.

```
\cs_new_eq:NN \DiscardShipoutBox \shipout_discard:
```

(*End of definition for* `\DiscardShipoutBox`. *This function is documented on page 1188.*)

\AtBeginDvi

If we roll forward from an earlier kernel `\AtBeginDvi` is defined so we better not use `\cs_new_protected:Npn` here.

```
\cs_set_protected:Npn \AtBeginDvi
                      {\__shipout_add_firstpage_material:Nn \AtBeginDvi}
```

(*End of definition for* `\AtBeginDvi`. *This function is documented on page 1187.*)

\DebugShipoutsOn
\DebugShipoutsOff

```
\cs_new_eq:NN \DebugShipoutsOn  \shipout_debug_on:
\cs_new_eq:NN \DebugShipoutsOff \shipout_debug_off:
```

(*End of definition for* `\DebugShipoutsOn` *and* `\DebugShipoutsOff`. *These functions are documented on page 1189.*)

5 Internal commands needed elsewhere

These internal commands use double and triple `@` signs so we need to stop getting them translated to the module name.

```
⟨@@=⟩
```

\@expl@@@shipout@add@firstpage@material@@Nn
\@expl@@@shipout@add@background@box@@n
\@expl@@@shipout@add@foreground@box@@n
\@expl@@@shipout@add@background@picture@@n
\@expl@@@shipout@add@foreground@picture@@n

Some internals needed elsewhere.

```
\cs_set_eq:NN \@expl@@@shipout@add@firstpage@material@@Nn
              \__shipout_add_firstpage_material:Nn

\cs_set_eq:NN \@expl@@@shipout@add@background@box@@n
              \__shipout_add_background_box:n

\cs_set_eq:NN \@expl@@@shipout@add@foreground@box@@n
              \__shipout_add_foreground_box:n

\cs_set_eq:NN \@expl@@@shipout@add@background@picture@@n
              \__shipout_add_background_picture:n

\cs_set_eq:NN \@expl@@@shipout@add@foreground@picture@@n
              \__shipout_add_foreground_picture:n
```

(*End of definition for* `\@expl@@@shipout@add@firstpage@material@@Nn` *and others.*)

```
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
```

Rolling back here doesn't undefine the interface commands as they may be used in packages without rollback functionality. So we just make them do nothing which may or may not work depending on the code usage.

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\shipout}{Hook management (shipout)}%
⟨latexrelease⟩
```

If we roll forward then `\tex_shipout:D` may not be defined in which case `\shipout` does have it original definition and so we must not `\let` it to something else which is `\relax`!

```
⟨latexrelease⟩\ifcsname tex_shipout:D\endcsname
⟨latexrelease⟩\expandafter\let\expandafter\shipout
⟨latexrelease⟩                 \csname tex_shipout:D\endcsname
⟨latexrelease⟩\fi
⟨latexrelease⟩
⟨latexrelease⟩\let \RawShipout\@undefined
⟨latexrelease⟩\let \ShipoutBox\@undefined
⟨latexrelease⟩\let \ReadonlyShipoutCounter \@undefined
⟨latexrelease⟩\let \c@totalpages \@undefined
⟨latexrelease⟩\let \thetotalpages \@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let \DiscardShipoutBox \@undefined
⟨latexrelease⟩\let \DebugShipoutsOn \@undefined
⟨latexrelease⟩\let \DebugShipoutsOff \@undefined
⟨latexrelease⟩
⟨latexrelease⟩\DeclareRobustCommand \AtBeginDvi [1]{%
⟨latexrelease⟩  \global \setbox \@begindvibox
⟨latexrelease⟩    \vbox{\unvbox \@begindvibox #1}%
⟨latexrelease⟩}
⟨latexrelease⟩
⟨latexrelease⟩\let \AtBeginShipout \@undefined
⟨latexrelease⟩\let \AtBeginShipoutNext \@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let \AtBeginShipoutFirst \@undefined
⟨latexrelease⟩
⟨latexrelease⟩\let \ShipoutBoxHeight \@undefined
⟨latexrelease⟩\let \ShipoutBoxDepth \@undefined
⟨latexrelease⟩\let \ShipoutBoxWidth \@undefined
⟨latexrelease⟩
```

We do not undo a substitution when rolling back. As the file support gets undone the underlying data is no longer used (and sufficiently obscure that it should not interfere with existing commands) and properly removing it would mean we need to make the `\undeclare@...` and its support macros available in all earlier kernel releases which is pointless (and actually worse).

```
⟨latexrelease⟩
⟨latexrelease⟩\let  \AtEndDvi \@undefined
```

We do not reenable a disabled package load when rolling back. As the file support gets undone the underlying data is no longer checked (and sufficiently obscure that it should not interfere with existing commands) and properly removing it would mean we need to make the `\reenable@package@load` command available in all earlier kernel releases which is pointless (and actually worse).

```
%\reenable@package@load{atenddvi}
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

`\__shipout_force_immediate_writes:` Change all writes to be immediate. To be used after the final page has been shipped out.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease⟩  {\__shipout_force_immediate_writes:}{Force immediate writes}%
\ExplSyntaxOn
\cs_new:Npn \__shipout_force_immediate_writes: {
```

Need to check if any variants are manually defined and if so adjust them too — not done!

```
  \cs_gset_eq:NN \iow_shipout:Nn \iow_now:Nn
  \cs_gset_eq:NN \lua_shipout:n \lua_now:n
  \cs_gset:Npn \write {\tex_immediate:D \tex_write:D}
```

As the writes will happen without a page break, reset the page number so they reference the last page.

```
  \global\advance\c@page\m@ne
}
\ExplSyntaxOff
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩  {\__shipout_force_immediate_writes:}{Force immediate writes}%
```

We want a definition for this even when doing rollback so that it can stay in the kernel hook without forcing a rollback there as well.

```
⟨latexrelease⟩\ExplSyntaxOn
⟨latexrelease⟩ \cs_new_eq:NN \__shipout_force_immediate_writes: \prg_do_nothing:
⟨latexrelease⟩\ExplSyntaxOff
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\__shipout_force_immediate_writes:`.)

6 Package emulation for compatibility

6.1 Package **atenddvi** emulation

`\AtEndDvi` This package has only one public command, so simulating it is easy and actually sensible to provide as part of the kernel.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2020/10/01}%
⟨latexrelease⟩                  {\AtEndDvi}{atenddvi emulation}%
\ExplSyntaxOn
\cs_new_protected:Npn \AtEndDvi #1 {\AddToHook{shipout/lastpage}{#1}}
\ExplSyntaxOff
```

As the package is integrate we prevent loading (no need to roll that back):

```
\disable@package@load{atenddvi}
   {\PackageWarning{atenddvi}
     {Functionality of this package is already\MessageBreak
      provided by LaTeX.\MessageBreak\MessageBreak
      It is there no longer necessary to load it\MessageBreak
```

```
      and you can safely remove it.\MessageBreak
      Found on}}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\AtEndDvi}{atenddvi emulation}%
⟨latexrelease⟩\let \AtEndDvi \@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\AtEndDvi`. *This function is documented on page* 1187.)

```
⟨/2ekernel⟩
```

6.2 Package **atbegshi** emulation

```
⟨*atbegshi-ltx⟩
\ProvidesPackage{atbegshi-ltx}
   [2021/01/10 v1.0c
     Emulation of the original atbegshi^^Jpackage with kernel methods]
```

`\AtBeginShipoutBox`

```
\let \AtBeginShipoutBox \ShipoutBox
```

(*End of definition for* `\AtBeginShipoutBox`. *This function is documented on page* 1190.)

`\AtBeginShipoutInit` Compatibility only, we aren't delaying ...

```
\let \AtBeginShipoutInit \@empty
```

(*End of definition for* `\AtBeginShipoutInit`. *This function is documented on page* 1191.)

`\AtBeginShipout` Filling hooks
`\AtBeginShipoutNext`

```
\protected\long\def\AtBeginShipout     #1{\AddToHook{shipout/before}{#1}}
\protected\long\def\AtBeginShipoutNext #1{\AddToHookNext{shipout/before}{#1}}
```

(*End of definition for* `\AtBeginShipout` *and* `\AtBeginShipoutNext`. *These functions are documented on page* 1191.)

`\AtBeginShipoutFirst` Slightly more complex as we need to know the name of the command under which the `shipout/firstpage` hook is filled.

```
\protected \def \AtBeginShipoutFirst
   {\@expl@@@shipout@add@firstpage@material@@Nn \AtBeginShipoutFirst}
```

(*End of definition for* `\AtBeginShipoutFirst`. *This function is documented on page* 1191.)

`\AtBeginShipoutDiscard` Just a different name.

```
\let \AtBeginShipoutDiscard \DiscardShipoutBox
```

(*End of definition for* `\AtBeginShipoutDiscard`. *This function is documented on page* 1191.)

`\AtBeginShipoutAddToBox` We don't expose them.
`\AtBeginShipoutAddToBoxForeground`
`\AtBeginShipoutUpperLeft`
`\AtBeginShipoutUpperLeftForeground`

```
\let \AtBeginShipoutAddToBox
               \@expl@@@shipout@add@background@box@@n
\let \AtBeginShipoutAddToBoxForeground
               \@expl@@@shipout@add@foreground@box@@n
```

```
\let \AtBeginShipoutUpperLeft
                \@expl@@@shipout@add@background@picture@@n
\let \AtBeginShipoutUpperLeftForeground
                \@expl@@@shipout@add@foreground@picture@@n
```

(*End of definition for* `\AtBeginShipoutAddToBox` *and others. These functions are documented on page* 1190.)

`\AtBeginShipoutOriginalShipout`

This offers the raw `\shipout` primitive of the engine. A page shipped out with this is not counted by `\ReadonlyShipoutCounter` counter and thus the mechanism to place `\special`s at the very end of the output might fail, etc. It should therefore not be used in new applications but is only provided to allow running legacy code. For new code use the commands provided by the kernel instead.

```
\ExplSyntaxOn
\cs_new_eq:NN \AtBeginShipoutOriginalShipout \tex_shipout:D
```

(*End of definition for* `\AtBeginShipoutOriginalShipout`. *This function is documented on page* 1190.)

`\ShipoutBoxHeight`
`\ShipoutBoxWidth`
`\ShipoutBoxDepth`

This is somewhat different from the original in atbegshi where `\ShipoutBoxHeight` etc. only holds the `\the\ht<box>` value. This may has some implications in some use cases and if that is a problem then it might need changing.

```
\cs_new:Npn \ShipoutBoxHeight { \dim_use:N \l_shipout_box_ht_dim }
\cs_new:Npn \ShipoutBoxDepth  { \dim_use:N \l_shipout_box_dp_dim }
\cs_new:Npn \ShipoutBoxWidth  { \dim_use:N \l_shipout_box_wd_dim }
\ExplSyntaxOff
```

(*End of definition for* `\ShipoutBoxHeight`, `\ShipoutBoxWidth`, *and* `\ShipoutBoxDepth`.)

```
⟨/atbegshi-ltx⟩
```

If the package is requested we substitute the one above:

```
⟨*2ekernel⟩
\declare@file@substitution{atbegshi.sty}{atbegshi-ltx.sty}
⟨/2ekernel⟩
```

6.3 Package **everyshi** emulation

This is now directly handled in that package so emulation is not necessary any more. Rather important :-)

```
⟨@@=⟩
```

File 54
ltoutput.dtx

1 Output Routine and float handling

```
⟨*2ekernel⟩
\message{output,}
```

1.1 Historical notes on the algorithm and commands

Historical LaTeX 2.09 comments (not necessarily accurate any more):

```
****************************************
*                OUTPUT                *
****************************************
```

PAGE LAYOUT PARAMETERS

```
\topmargin      : Extra space added to top of page.
@twoside        : boolean.  T if two-sided printing
\oddsidemargin  : IF @twoside = T
                    THEN extra space added to left of odd-numbered
                         pages.
                    ELSE extra space added to left of all pages.
\evensidemargin : IF @twoside = T
                    THEN extra space added to left of even-numbered
                         pages.
\headheight     : height of head
\headsep        : separation between head and text
\footskip       : distance separation between baseline of last
                  line of text and baseline of foot.
                  Note difference between \footSKIP and \headSEP.
\textheight     : height of text on page, excluding head and foot
\textwidth      : width of printing on page
\columnsep      : IF @twocolumn = T
                    THEN width of space between columns
\columnseprule  : IF @twocolumn = T
                    THEN width of rule between columns (0 if none).
\columnwidth    : IF @twocolumn = T
                    THEN (\textwidth - \columnsep)/2
                    ELSE \textwidth
                  It is set by the \twocolumn and
                  \onecolumn commands.
\@textbottom    : Command executed at bottom of vbox holding text of
                  page (including figures).  The \raggedbottom
                  command almost \let's this to \vfil (actually sets
                  it to \vskip \z@ plus.0001fil).
                  Should have depth 0pt.
```

\@texttop : Command executed at top of vbox holding text of page (including figures). Used by letter style; can also be used to produce centered pages. Let to \relax by \raggedbottom and \flushbottom.

Page layout must initialize \@colht and \@colroom to \textheight.

PAGE STYLE PARAMETERS:

\floatsep : Space left between floats.
\textfloatsep : Space between last top float or first bottom float and the text.
\topfigrule : Command to place rule (or whatever) between floats at top of page and text. Executed in inner vertical mode right before the \textfloatsep skip separating the floats from the text. Must occupy zero vertical space. (See \footnoterule.)
\botfigrule : Same as \topfigrule, but put after the \textfloatsep skip separating text from the floats at bottom of page.
\intextsep : Space left on top and bottom of an in-text float.
\dblfloatsep : Space between double-column floats.
\dbltextfloatsep : Space between top double-column floats and text.
\dblfigrule : Similar to \topfigrule, but for double-column floats.
\@fptop : Glue to go at top of float column – must be 0pt + stretch
\@fpsep : Glue to go between floats in a float column.
\@fpbot : Glue to go at bottom of float column – must be 0pt + stretch
\@dblfptop, \@dblfpsep, \@dblfpbot : Analogous for double-column float page in two-column format.

FOOTNOTES: As in PLAIN, footnotes use \insert\footins.

PAGE LAYOUT SWITCHES AND MACROS

@twocolumn : Boolean. T if two columns per page globally.

PAGE STYLE MACROS AND SWITCHES

\@oddhead : IF @twoside = T THEN macro to generate head of odd-numbered pages. ELSE macro to generate head of all pages.
\@evenhead : IF @twoside = T THEN macro to generate head of even-numbered

pages.

`\@oddfoot` : IF @twoside = T
THEN macro to generate foot of odd-numbered pages.
ELSE macro to generate foot of all pages.

`\@evenfoot` : IF @twoside = T
THEN macro to generate foot of even-numbered pages.

@specialpage : boolean. T if current page is to have a special format.

`\@specialstyle` : If its value is foo then
IF @specialpage = T
THEN the command `\ps@foo` is executed to temporarily reset the page style parameters before composing the current page.
This command should execute only `\def`'s and `\edef`'s, making only local definitions.

FLOAT PLACEMENT PARAMETERS

The following parameters are set by the macro `\@floatplacement`.
When `\@floatplacement` is called,
`\@colht` is the height of the page or column being built. I.e.:
* For single-column page it equals `\textheight`.
* For double-column page it equals `\textheight` - height of double-column floats on page.

Note that some are set globally and some locally:

`\@topnum` :=G Maximum number of floats allowed on the top of a column.

`\@toproom` :=G Maximum amount of top of column devoted to floats– excluding `\textfloatsep` separation below the floats and `\floatsep` separation between them. For two-column output, should be computed as a function of `\@colht`.

`\@botnum, \@botroom`
: Analogous to above.

`\@colnum` :=G Maximum number of floats allowed in a column, including in-text floats.

`\@textmin` :=L Minimum amount of text (excluding footnotes) that must appear on a text page.
`%%` 27 Sep 85 : made local to
`%%` `\@addtocurcol` and `\@addtonextcol`
It is now also used locally in processing double floats.

`\@fpmin` :=L Minimum height of floats in a float column.

The macro `\@dblfloatplacement` sets the following parameters.

`\@dbltopnum` :=G Maximum number of double-column floats allowed at the top of a two-column page.

`\@dbltoproom` :=G Maximum height of double-column floats allowed at

top of two-column page.

`\@fpmin` :=L Minimum height of floats in a float column.

It should also perform the following local assignments where necessary – i.e., where the new value differs from the old one:

```
\@fptop     :=L \@dblfptop
\@fpsep     :=L \@dblfpsep
\@fpbot     :=L \@dblfpbot
```

OUTPUT ROUTINE VARIABLES

`\@colht` : The total height of the current column. In single column style, it equals `\textheight`. In two-column style, it is `\textheight` minus the height of the double-column floats on the current page. MUST BE INITIALIZED TO `\textheight`.

`\@colroom` : The height available in the current column for text and footnotes. It equals `\@colht` minus the height of all floats committed to the top and bottom of the current column.

`\@textfloatsheight` : The total height of in-text floats on the current page.

`\footins` : Footnote insertion number.

`\@maxdepth` : Saved value of TeX's `\maxdepth`. Must be set when any routine sets `\maxdepth`.

CALLING THE OUTPUT ROUTINE

The output routine is called either by TeX's normal page-breaking mechanism, or by a macro putting a penalty < or = -10000 in the output list. In the latter case, the penalty indicates why the output routine was called, using the following code.

penalty	reason
-10000	`\pagebreak` `\newpage`
-10001	`\clearpage` (`\penalty` -10000 `\vbox{}` `\penalty` -10001)
-10002	float insertion, called from horizontal mode
-10003	float insertion, called from vertical mode.
-10004	float insertion.

Note: A float or marginpar puts the following sequence in the output list: (i) a penalty of -10004,
(ii) a null `\vbox`
(iii) a penalty of -10002 or -10003.
This solves two special problems:

1. If the float comes right after a `\newpage` or `\clearpage`, then the first penalty is ignored, but the second one invokes the output routine.
2. If there is a split footnote on the page, the second 'page' puts out the rest of the footnote.

THE OUTPUT ROUTINE

FUNCTIONS USED IN THE OUTPUT ROUTINE:

`\@outputpage` : Produces an output page with the contents of box `\@outputbox` as the text part.
Also sets `\@colht` :=G `\textheight`.
The page style is determined as follows.
IF @thispagestyle = true
THEN use `\thispagestyle` style
ELSE use ordinary page style.

`\@tryfcolumn\FLIST` : Tries to form a float column composed of floats from `\FLIST` (if nonempty) with the following parameters:
`\@colht` : height of box
`\@fpmin` : minimum height of floats in the box
`\@fpsep` : interfloat space
`\@fptop` : glue at top of box
`\@fpbot` : glue at bottom of box.
If it succeeds, then it does the following:
* `\@outputbox` :=L the composed float box.
* @fcolmade :=G true
* `\FLIST` :=G `\FLIST` - floats put in box
* `\@freelist` :=G `\@freelist` + floats put in box
If it fails, then:
* @fcolmade :=G false
NOTE: BIT MUST BE A SINGLE TOKEN!

`\@makefcolumn \FLIST` : Same as `\@tryfcolumn` except that it fails to make a float column only if `\FLIST` is empty. Otherwise, it makes a float column containing at least the first box in `\FLIST`, disregarding `\@fpmin`.

`\@startcolumn` :
Calls `\@tryfcolumn\@deferlist`. If `\@tryfcolumn` returns with (globally set) @fcolmade = false, then:
* Globally sets `\@toplist` and `\@botlist` to floats from `\@deferlist` to go at top and bottom of column, deleting them from `\@deferlist`. It does this using `\@colht` as the total height, the page style parameters `\@floatsep` and `\@textfloatsep`, and the float placement parameters `\@topnum`, `\@toproom`, `\@botnum`, `\@botroom`, `\@colnum` and `\textfraction`.

* Globally sets `\@colroom` to `\@colht` minus the height of the added floats.

`\@startdblcolumn` :
Calls `\@tryfcolumn\@dbldeferlist{8}`. If `\@tryfcolumn` returns with (globally set) @fcolmade = false, then:
* Globally sets `\@dbltoplist` to floats from `\@dbldeferlist` to go at top and bottom of column, deleting them from `\@dbldeferlist.` It does this using `\textheight` as the total height, and the parameters `\@dblfloatsep,` etc.
* Globally sets `\@colht` to `\textheight` minus the height of the added floats.

`\@combinefloats` : Combines the text from box `\@outputbox` with the floats from `\@toplist` and `\@botlist,` putting the new box in `\@outputbox.` It uses `\floatsep` and `\textfloatsep` for the appropriate separations. It puts the elements of `\TOPLIST` and `\BOTLIST` onto `\@freelist,` and makes those lists null.

`\@makecol` : Makes the contents of `\box255` plus the accumulated footnotes, plus the floats in `\@toplist` and `\@botlist,` into a single column of height `\@colht` (unless the page height has been locally changed), which it puts into box `\@outputbox.` It puts boxes in `\@midlist` back onto `\@freelist` and restores `\maxdepth.`

`\@opcol` : Outputs a column whose text is in box `\@outputbox`
If @twocolumn = false, then it calls `\@outputpage,` sets `\@colht` :=G `\textheight,` and calls `\@floatplacement.`

If @twocolumn = true, then:
If @firstcolumn = true, then it puts box `\@outputbox` into `\@leftcolumn` and sets @firstcolumn :=G false.

If @firstcolumn = false, then it puts out the current two-column page, any possible two-column float pages, and determines `\@dbltoplist` for the next page.

USER COMMANDS THAT CALL OR AFFECT THE OUTPUT ROUTINE

`\newpage` == BEGIN `\par\vfil\penalty` -10000 END

```
\clearpage == BEGIN \newpage
                    \write -1{}     % Part of hack to make sure no
```

```
                \vbox{}         % \write's get lost.
                \penalty -10001
        END

\cleardoublepage == BEGIN \clearpage
                      if @twoside = true and c@page is even
                        then \hbox{} \newpage fi
                \vbox{}         % \write's get lost.
                \penalty -10001
        END
```

`\twocolumn[BOX]` : starts a new page, changing to twocolumn setting and puts BOX in a parbox of width `\textwidth` across the top. Useful for full-width titles for double-column pages.
SURPRISE: The stretch from `\@dbltextfloatsep` will be inserted between the BOX and the top of the two columns.

FLOAT-HANDLING MECHANISMS

The float environment obtains an insertion number B from the `\@freelist` (see below for a description of list manipulation), puts the float into box B and sets `\count` B to a FLOAT SPECIFIER. For a normal (not double-column) float, it then causes a page break in one of the following two ways:

- In outer hmode: `\vadjust{\penalty -10002}`
- In vmode : `\penalty -10003`.

For a double-column float, it puts B onto the `\@dbldeferlist`.
The float specifier has two components:

* A PLACEMENT SPECIFICATION, describing where the float may be placed.
* A TYPE, which is a power of two–e.g., figures might be type 1 floats, tables type 2 floats, programs type 4 floats, etc.

The float specifier is encoded as follows, where bit 0 is the least significant bit.

Bit	Meaning
0	1 iff the float may go where it appears in the text.
1	1 iff the float may go on the top of a page.
2	1 iff the float may go on the bottom of a page.
3	1 iff the float may go on a float page.
4	1 unless the PLACEMENT includes a !
5	1 iff a type 1 float
6	1 iff a type 2 float
etc.	

A negative float specifier is used to indicate a marginal note.

MACROS AND DATA STRUCTURES FOR PROCESSING FLOATS

A FLOAT LIST consisting of the floats in boxes \boxa ... \boxN has the form:

```
\@elt \boxa ... \@elt \boxN
```

where \boxI is defined by

```
\newinsert\boxI
```

Normally, \@elt is \let to \relax. A test can be performed on the entire float list by locally \def'ing \@elt appropriately and executing the list.
This is a lot more efficient than looping through the list.

The following macros are used for manipulating float lists.

```
\@next \CS \LIST {NONEMPTY}{EMPTY} ==  %% NOTE: ASSUME \@elt
= \relax
  BEGIN  assume that \LIST == \@elt \B1 ... \@elt \Bn
         if n = 0
           then  EMPTY
           else  \CS    :=L \B1
                 \LIST  :=G \@elt \B2 ... \@elt \Bn
                 NONEMPTY
         fi
  END
```

\@bitor\NUM\LIST : Globally sets switch @test to the disjunction for all I of bit log2 \NUM of the float specifiers of all the floats in \LIST.
I.e., @test is set to true iff there is at least one float in \LIST having bit log2 \NUM of its float specifier equal to 1.

Note: log2 [(\count I)/32] is the bit number corresponding to the type of float I. To see if there is any float in \LIST having the same type as float I, you run \@bitor with
\NUM = [(\count I)/32] * 32.

```
\@bitor\NUM\LIST ==
  BEGIN
     @test :=G false
     { \@elt \CTR ==  if \NUM <> 0 then
                        if \count\CTR / \NUM is odd
                           then  @test := true        fi fi
       \LIST
     }
  END
```

\@cons\LIST\NUM : Globally sets \LIST := \LIST * \@elt \NUM

```
\@cons\LIST\NUM ==
  BEGIN {  \@elt == \relax
           \LIST :=G \LIST \@elt \NUM
        }
```

BOX LISTS FOR FLOAT-PLACEMENT ALGORITHMS

```
  \@freelist     : List of empty boxes for placing new floats.
  \@toplist      : List of floats to go at top of current column.
  \@midlist      : List of floats in middle of current column.
  \@botlist      : List of floats to go at bottom of current column.
  \@deferlist    : List of floats to go after current column.
  \@dbltoplist   : List of double-col. floats to go at top of current
                   page.
  \@dbldeferlist : List of double-column floats to go on subsequent
                   pages.
```

FLOAT-PLACEMENT ALGORITHMS

```
\@addtobot : Tries to put insert \@currbox on \@botlist.
             Called only when:
                * \ht BOX < \@colroom
                * type of \@currbox not on \@deferlist
                * \@colnum > 0
                * @insert = false
             If it succeeds, then:
                * sets @insert true
                * decrements \@botroom by \ht BOX
                * decrements \@botnum and \@colnum by 1
                * decrements \@colroom by \ht BOX + either \floatsep
                  or \textfloatsep, as appropriate.
                * sets \maxdepth to 0pt     % <- as of 2025/06/01
                                              %      no longer the case

\@addtotoporbot : Tries to put insert \@currbox on \@toplist or
                  \@botlist.
                  Called only under same conditions as \@addtobot.
                  If it succeeds, then:
                     * sets @insert true
                     * decrements \@toproom or \@botroom by \ht BOX
                     * decrements \@colnum and either \@topnum or
                       \@botnum by 1
                     * decrements \@colroom by \ht BOX + \floatsep
                       or \textfloatsep, as appropriate.

\@addtocurcol : Tries to add \@currbox to current column, setting
                @insert true if it succeeds, false otherwise.
                It will add \@currbox to top only if bit 0 of
```

\count \@currbox is 0, and to the bottom only if bit 0 = 0 or an earlier float of the same type is put on the bottom.
If the float is put in the text, then \penalty\interlinepenalty is put right after the float, before the following \vskip, and \outputpenalty :=L 0.

\@addtonextcol : Tries to add \@currbox to the next column, setting @insert true if it succeeds, false otherwise.

\@addtodblcol : Tries to add \@currbox to the next double-column page, adding it to \@dbltoplist if it succeeds and \@dbldeferlist if it fails.

```
\@addmarginpar ==
 BEGIN
   if \@currlist nonempty
     then remove \@marbox from \@currlist
          add \@marbox and \@currbox to \@freelist
              %% NOTE: \@currbox = left box
     else LaTeX error: ?  %% shouldn't happen
   fi
   \@tempcnta := 1      %% 1 = right, -1 = left
   if @twocolumn = true
     then if @firstcolumn = true
            then \@tempcnta := -1
          fi
     else if @mparswitch = true
            then if count0 odd
                   else \@tempcnta := -1
                 fi
          fi
          if @reversemargin = true
             then \@tempcnta := -\@tempcnta
          fi
   fi
   if \@tempcnta < 0 then \box\@marbox :=G \box\@currbox
   fi
   \@tempdima   :=L maximum(\@mparbottom - \@pageht
                                         + ht of \@marbox, 0)
   if \@tempdima > 0 then LaTeX warning: 'marginpar moved' fi
   \@mparbottom :=G \@pageht + \@tempdima + depth of \@marbox
                    + \marginparpush
   \@tempdima   :=L \@tempdima - ht of \@marbox
   \box\@marbox :=G \box\@currbox
                            \vbox { \vskip \@tempdima
                                    \box\@marbox
                                  }
```

```
    height of \@marbox :=G depth of \@marbox :=G 0
    \kern -\@pagedp
    \nointerlineskip
    \hbox{ if @tempcnta > 0 then \hskip \columnwidth
                                 \hskip \marginparsep
                           else \hskip -\marginparsep
                                 \hskip -\marginparwidth
           fi
           \box\@marbox \hss
         }
    \nobreak
    \nointerlineskip
    \hbox{\vrule height 0 width 0 depth \@pagedp}
  END
```

End of historical LaTeX 2.09 comments.

1.2 Core definitions

Floats and marginpars add a lot of dead cycles.

```
\maxdeadcycles = 100
\let\@elt\relax
\def\@next#1#2#3#4{\ifx#2\@empty #4\else
   \expandafter\@xnext #2\@@#1#2#3\fi}
\def\@xnext \@elt #1#2\@@#3#4{\def#3{#1}\gdef#4{#2}}

\def\@testfalse{\global\let\if@test\iffalse}
\def\@testtrue {\global\let\if@test\iftrue}
\@testfalse

\def\@bitor#1#2{\@testfalse {\let\@elt\@xbitor
   \@tempcnta #1\relax #2}}
```

RmS 91/11/22: Added test for `\count#1 = 0`. Suggested by Chris Rowley.

```
\def\@xbitor #1{\@tempcntb \count#1
   \ifnum \@tempcnta =\z@
   \else
     \divide\@tempcntb\@tempcnta
     \ifodd\@tempcntb \@testtrue\fi
   \fi}
```

1.2.1 Definition of float boxes

```
\let\@elt\newinsert
\def\@freelist{%
  \@elt\bx@A\@elt\bx@B\@elt\bx@C\@elt\bx@D\@elt\bx@E
  \@elt\bx@F\@elt\bx@G\@elt\bx@H\@elt\bx@I\@elt\bx@J
  \@elt\bx@K\@elt\bx@L\@elt\bx@M\@elt\bx@N
  \@elt\bx@O\@elt\bx@P\@elt\bx@Q\@elt\bx@R}
\@freelist
\def\reserved@a{%
  \@elt\bx@S\@elt\bx@T\@elt\bx@U\@elt\bx@V
```

```
  \@elt\bx@W\@elt\bx@X\@elt\bx@Y\@elt\bx@Z
  \@elt\bx@AA\@elt\bx@BB\@elt\bx@CC\@elt\bx@DD\@elt\bx@EE
  \@elt\bx@FF\@elt\bx@GG\@elt\bx@HH\@elt\bx@II\@elt\bx@JJ
  \@elt\bx@KK\@elt\bx@LL\@elt\bx@MM\@elt\bx@NN
  \@elt\bx@OO\@elt\bx@PP\@elt\bx@QQ\@elt\bx@RR
  \@elt\bx@SS\@elt\bx@TT\@elt\bx@UU\@elt\bx@VV
  \@elt\bx@WW\@elt\bx@XX\@elt\bx@YY\@elt\bx@ZZ}
\reserved@a
\def\@elt{\noexpand\@elt\noexpand}
\edef\@freelist{\@freelist\reserved@a}
\let\reserved@a\relax
\let\@elt\relax
\gdef\@toplist{}
\gdef\@botlist{}
\gdef\@midlist{}
\gdef\@currlist{}
\gdef\@deferlist{}
\gdef\@dbltoplist{}
```

The new algorithm stores page wide floats together with column floats in a single `\@deferlist` list. We keep `\@dbldeferlist` initialized as empty so that packages that are testing for deferred floats can use the same code for old or new float handling.

```
\gdef\@dbldeferlist{}
```

1.2.2 Page layout parameters

```
\newdimen\topmargin
\newdimen\oddsidemargin
\newdimen\evensidemargin
\let\@themargin=\oddsidemargin
\newdimen\headheight
\newdimen\headsep
\newdimen\footskip
\newdimen\textheight
\newdimen\textwidth
\newdimen\columnwidth
\newdimen\columnsep
\newdimen\columnseprule
\newdimen\marginparwidth
\newdimen\marginparsep
\newdimen\marginparpush
```

\AtBeginDvi
\@begindvibox

We use a box register in which to put stuff that must appear before anything else in the `.dvi` file.

The stuff in the box should not add any typeset material to the page when it is unboxed.

This interface is no longer used. Instead a new one is inside `ltshipout.dtx`. We only keep the box in case some old code refers to it directly (or we do some rollback).

```
\newbox\@begindvibox
%\DeclareRobustCommand \AtBeginDvi [1]{%
%  \global \setbox \@begindvibox
%    \vbox{\unvbox \@begindvibox #1}%
%}
```

(*End of definition for* `\AtBeginDvi` *and* `\@begindvibox`. *These functions are documented on page* 1187.)

\@maxdepth This is not the right place to set this; it needs to be set in a class/style file when \maxdepth is set.

Also, many settings to \maxdepth should be to \@maxdepth, probably?

```
\newdimen\@maxdepth
\@maxdepth = \maxdepth
```

(*End of definition for* \@maxdepth.)

\paperheight \paperwidth New \paper... registers.

```
\newdimen\paperheight
\newdimen\paperwidth
```

(*End of definition for* \paperheight *and* \paperwidth.)

\stockheight \stockwidth New \stock... registers.

```
\newdimen\stockheight
\newdimen\stockwidth
```

(*End of definition for* \stockheight *and* \stockwidth.)

\if@insert \if@fcolmade \if@specialpage \if@firstcolumn \if@twocolumn \if@twoside \if@reversemargin \if@mparswitch \col@number Local switches first:

```
\newif \if@insert
```

These should definitely be global:

```
\newif \if@fcolmade
\newif \if@specialpage \@specialpagefalse
```

These should be global but are not always set globally in other files.

```
\newif \if@firstcolumn \@firstcolumntrue
\newif \if@twocolumn   \@twocolumnfalse
```

Not sure about these: two questions. Should things which must apply to a whole document be local or global (they probably should be 'preamble only' commands)? Are these three such things?

```
\newif \if@twoside     \@twosidefalse
\newif \if@reversemargin \@reversemarginfalse
\newif \if@mparswitch  \@mparswitchfalse
```

This counter has been imported from 'multicol'.

```
\newcount \col@number
\col@number \@ne
```

(*End of definition for* \if@insert *and others.*)

1.2.3 Internal registers

```
\newcount\@topnum
\newdimen\@toproom
\newcount\@dbltopnum
\newdimen\@dbltoproom
\newcount\@botnum
\newdimen\@botroom
\newcount\@colnum
\newdimen\@textmin
\newdimen\@fpmin
\newdimen\@colht
\newdimen\@colroom
\newdimen\@pageht
\newdimen\@pagedp
\newdimen\@mparbottom \@mparbottom\z@
\newcount\@currtype
\newbox\@outputbox
\newbox\@leftcolumn
\newbox\@holdpg

\def\@thehead{\@oddhead} % initialization
\def\@thefoot{\@oddfoot}
```

1.2.4 Page break commands

`\clearpage` The tests at the beginning are an experimental attempt to avoid a completely empty page after a `\twocolumn[...]`. This prevents the text from the argument vanishing into a float box, never to be seen again. We hope that it does not produce wrong formatting in other cases.

```
\def\clearpage{%
  \ifvmode
    \ifnum \@dbltopnum =\m@ne
      \ifdim \pagetotal <\topskip
        \hbox{}%
      \fi
    \fi
  \fi
  \newpage
```

After the above `\newpage` there may be some still unwritten `\write`'s issued in the output routine and we need to make sure that they are not lost when `\@doclearpage` produces float pages. Therefore the following trick is applied: We add a harmless `\write` followed by an empty `\vbox` and then call the output routine again through a special penalty. That triggers `\@doclearpage` in the OR which splits off a zero-sized chunk from box 255 (which should go up to, but not including, the `\vbox`), then `\unvbox` that (which should contain any `\write`'s, and then throw away the rest of 255 (which should just contain the `\vbox`). That explains the strange `\write-1{}`: that statement introduces a necessary breakpoint so that the split is done before the `\vbox`. If it would not be there and *there are no real* `\write`*'s*, then the split would grab the whole of 255 and so would put the `\vbox` back on the page and we would end with an extra page only containing that empty `\vbox`.

Deep breath—the above is my (Frank's) attempt of a slightly longer version of Leslie's cryptic comment earlier in the file that this `\write` is "part of a hack to make sure that no `\write`'s are lost".

This \write needs to be non-immediate even if we are supposedly already past the last page because it has to end up on the page sent to the output routine if for some reason that happens.

```
  \@@write\m@ne{}%
  \vbox{}%
  \penalty -\@Mi
}

\let\@@write\write
```

(*End of definition for* \clearpage.)

\cleardoublepage

```
\def\cleardoublepage{\clearpage\if@twoside \ifodd\c@page\else
    \hbox{}\newpage\if@twocolumn\hbox{}\newpage\fi\fi\fi}
⟨/2ekernel⟩
```

(*End of definition for* \cleardoublepage.)

\onecolumn

```
⟨*2ekernel | fltrace⟩
\def\onecolumn{%
  \clearpage
  \global\columnwidth\textwidth
  \global\hsize\columnwidth
  \global\linewidth\columnwidth
  \global\@twocolumnfalse
  \col@number \@ne
  \@floatplacement}
```

(*End of definition for* \onecolumn.)

\newpage The two checks at the beginning ensure that an item label or run-in section title immediately before a \newpage get printed on the correct page, the one before the page break.

All three tests are largely to make error processing more robust; that is why they all reset the flags explicitly, even when it would appear that this would be done by a \leavevmode.

```
⟨/2ekernel | fltrace⟩
⟨latexrelease⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease⟩                    {\newpage}{Check depth of page}%
⟨*2ekernel | latexrelease | fltrace⟩
\def \newpage {%
  \if@noskipsec
    \ifx \@nodocument\relax
      \leavevmode
      \global \@noskipsecfalse
    \fi
  \fi
  \if@inlabel
    \leavevmode
    \global \@inlabelfalse
  \fi
  \if@nobreak \@nobreakfalse \everypar{}\fi
```

The \vfil at the end of the macro before the break penalty will normally result in the page being run short, even with \flushbottom in effect (in contrast to the behavior of \pagebreak). However, if there is some explicit stretch on the page, say, a \vfill, it has the undesired side-effect, that the last line will not align at its baseline if it contains characters going below the baseline, as the value of \prevdepth is no longer taken into account by TEX. So we have to back up by that amount (or by \maxdepth if it is really huge), to mimic the normal behavior without the \newpage.

In 2017 this was fixed by doing the backing-up here. However, the problem with that is that if the page contains footnotes the \skip\ý is measured from the baseline of the last line and not (as on all other pages) from the bottom of the last line. One can argue that measuring from the baseline is better, but if that is done it should be done everywhere not just on some pages.

Thus this backing up should only happen when there are no footnotes. The problem is that, at the point when \newpage is called, it is not known whether or not there are footnotes; that is only determined in the output routine. We have therefore moved this code to the newly introduced output routine sockets.

An explicit \par token is required so that after environments like lists \par is reset to its standard meaning and does not suppress indentation on the next page.

```
   \par
%  \ifdim\prevdepth>\z@
%     \vskip -%
%       \ifdim\prevdepth>\maxdepth
%         \maxdepth
%       \else
%         \prevdepth
%       \fi
%  \fi
  \vfil
  \penalty -\@M}
⟨/2ekernel | latexrelease | fltrace⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2017/04/15}%
⟨latexrelease⟩                  {\newpage}{Check depth of page}%
⟨latexrelease⟩\def \newpage {%
⟨latexrelease⟩  \if@noskipsec
⟨latexrelease⟩    \ifx \@nodocument\relax
⟨latexrelease⟩      \leavevmode
⟨latexrelease⟩      \global \@noskipsecfalse
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \if@inlabel
⟨latexrelease⟩    \leavevmode
⟨latexrelease⟩    \global \@inlabelfalse
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \if@nobreak \@nobreakfalse \everypar{}\fi
⟨latexrelease⟩  \par
⟨latexrelease⟩  \ifdim\prevdepth>\z@
⟨latexrelease⟩     \vskip -%
⟨latexrelease⟩       \ifdim\prevdepth>\maxdepth
⟨latexrelease⟩         \maxdepth
⟨latexrelease⟩       \else
⟨latexrelease⟩         \prevdepth
```

```
⟨latexrelease⟩       \fi
⟨latexrelease⟩   \fi
⟨latexrelease⟩   \vfil
⟨latexrelease⟩   \penalty -\@M}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\newpage}{Check depth of page}%
⟨latexrelease⟩\def \newpage {%
⟨latexrelease⟩   \if@noskipsec
⟨latexrelease⟩     \ifx \@nodocument\relax
⟨latexrelease⟩       \leavevmode
⟨latexrelease⟩       \global \@noskipsecfalse
⟨latexrelease⟩     \fi
⟨latexrelease⟩   \fi
⟨latexrelease⟩   \if@inlabel
⟨latexrelease⟩     \leavevmode
⟨latexrelease⟩     \global \@inlabelfalse
⟨latexrelease⟩   \fi
⟨latexrelease⟩   \if@nobreak \@nobreakfalse \everypar{}\fi
⟨latexrelease⟩   \par
⟨latexrelease⟩   \vfil
⟨latexrelease⟩   \penalty -\@M}
⟨latexrelease⟩\EndIncludeInRelease
⟨∗2ekernel | fltrace⟩
```

(*End of definition for* `\newpage`.)

`\@emptycol` It may be better to use an invisible rule rather than an empty box here.

```
\def \@emptycol {\vbox{}\penalty -\@M}
```

(*End of definition for* `\@emptycol`.)

`\twocolumn` `\@topnewpage` There are several bug fixes to the two-column stuff here.

```
\def \twocolumn {%
  \clearpage
  \global\columnwidth\textwidth
  \global\advance\columnwidth-\columnsep
  \global\divide\columnwidth\tw@
  \global\hsize\columnwidth
  \global\linewidth\columnwidth
  \global\@twocolumntrue
  \global\@firstcolumntrue
  \col@number \tw@
```

There is no reason to put a `\@dblfloatplacement` here since `\@topnewpage` ignores these settings. The `\@floatplacement` is needed in case this comes after some changes.

```
  \@ifnextchar [\@topnewpage\@floatplacement
}
```

Note that here, getting a box from the freelist can assume success since this comes just after a `\clearpage`.

```
\long\def \@topnewpage [#1]{%
  \@nodocument
  \@next\@currbox\@freelist{}{}%
```

```
\global \setbox\@currbox
  \color@vbox
    \normalcolor
    \vbox {%
      \hsize\textwidth
      \@parboxrestore
      \col@number \@ne
      #1%
      \vskip -\dbltextfloatsep
           }%
  \color@endbox
```

Added size test and warning message; perhaps we should use an error message.

```
\ifdim \ht\@currbox>\textheight
  \ht\@currbox \textheight
\fi
```

This next line is not essential but it is more robust to make this value non-zero, in case of weird errors.

This next bit is what is needed from `\@addtodblcol`, plus some extra checks for error trapping.

```
\global \count\@currbox \tw@
\@tempdima -\ht\@currbox
\advance \@tempdima -\dbltextfloatsep
\global \advance \@colht \@tempdima
\ifx \@dbltoplist \@empty
\else
  \@latex@error{Float(s) lost}\@ehb
  \let \@dbltoplist \@empty
\fi
\@cons \@dbltoplist \@currbox
```

This setting of `\@dbltopnum` is used only to change the typesetting in `\@combinedblfloats`.

```
  \global \@dbltopnum \m@ne
⟨*trace⟩
    \fl@trace{dbltopnum set to -1 (= \the \@dbltopnum) (topnewpage)}%
⟨/trace⟩
```

At points such as this we need to check that there is still a minimal amount of room left on the page; this uses an arbitrary small value at present; but note that this value is larger than that used when checking that page is too full of normal floats.

If there is little room left we just force a page-break, OK? This involves producing two empty columns. The second empty column may be produced by `\output`, in which case an extra, misleading, warning will be generated, OK? (This happens only when there is too little room left on the page for any float.) Otherwise (i.e. if the size is such that it is allowed as a normal float) the extra `\@emptycol` will be invoked in the second column by the conditional code guarded by the `\if@firstcolumn` test.

I now think that the cut-off point here should be `3\baselineskip`, but we make it a bit less so that 3 lines of text will be allowed, OK?

Since this happens only when there is nothing on the page but the 'top-box', the empty box should not cause any problem other than some overfull box messages, which is not entirely misleading.

Here we need two page-ends since both columns need to be empty.

```
  \ifdim \@colht<2.5\baselineskip
    \@latex@warning@no@line {Optional argument of \noexpand\twocolumn
                  too tall on page \thepage}%
    \@emptycol
    \if@firstcolumn
    \else
      \@emptycol
    \fi
  \else
    \global \vsize \@colht
    \global \@colroom \@colht
    \@floatplacement
  \fi
}
```

(*End of definition for* `\twocolumn` *and* `\@topnewpage`.)

2 The LaTeX output routine

2.1 Hooks and replaceable code blocks

To support packages that want to augment aspects of the output routine we offer a number of hooks (where several packages can add code) as well as some sockets (that can only be changed by one package or through options like those of the footmisc package).

2.1.1 Output routine hooks

`build/page/before`, `build/page/after` These two hooks enable packages to prepend or append code to the page processing in the output routine. They are implemented as mirrored hooks.

Technically, they are executed at the start and the end of the internal LaTeX 2ε `\@outputpage` command, respectively.

`build/page/reset` Packages that set up special conventions for text in the main galley (such as catcode changes, etc.) can use this hook to undo these changes within the output routine, so that they aren't applied to unrelated material, e.g., the text for running header or footers.

`build/column/before`, `build/column/after` These two hooks enable packages to prepend or append code to the column processing in the output routine. They are implemented as mirrored hooks.

Technically, they are executed at the start and the end of the internal LaTeX 2ε `\@makecol` command, respectively.

2.1.2 Replaceable code blocks (sockets)

To cater for different layouts with respect to text, footnotes, and bottom-floats placements there are sockets for now. They are sockets not hooks, because the overall layout can only be controlled by one package, i.e., the last setting wins.

build/column/outputbox (0 arguments) In code for this socket the `\@outputbox` (holding the galley text for the current column or page) is augmented by attaching floats and footnote areas together with appropriate spacing.

Prior to calling the socket the output routine has already decided which floats go into which area and which get deferred. Therefore, the assumption is that the code in the socket attaches all areas that contain floats. If this is not done, then the order of floats is likely to be screwed up unless unused floats are moved to the defer list in an appropriate way (for now we don't offer any interface for that scenario).

Before the code in the socket is run, any existing glue at the bottom of the `\@outputbox` is removed and stored in a safe place. If needed, it can be reinserted with one of the helper commands.

To support setting this up the following helper commands are available:

`\@outputbox@append` **(1 argument)**
: This general purpose command alters the `\@outputbox` box by appending material to it.

`\@outputbox@appendfil` **(0 arguments)** Append a `\vfil` to the `\@outputbox`. Based on the plug in `build/column/baselineattach` either measured from the bottom or from the baseline of the last line in the box.

`\@outputbox@appendfootnotes` **(0 arguments)**
: This command appends the footnotes to the `\@outputbox` (if there are any). If not, then it does nothing.

`\@outputbox@attachfloats` **(0 arguments)**

`\@outputbox@attachtopfloats` **(0 arguments)**

`\@outputbox@attachbottomfloats` **(0 arguments)**
: Attaching top and bottom floats can usually be done in one go, but for special layouts we might want more control so we provide also separate commands.

`\@outputbox@reinsertbskip` **(0 arguments)**
: Reinsert the bottom skip of the `\@outputbox` that was saved before in `\@makecol`.

Testing for existence of material
: There are a number of helpers to run conditional code depending on whether or not there are footnotes or bottom floats. They are `\@if@footnotes@TF` and `\@if@bottomfloats@TF` (names are likely to change).

This socket cannot be empty but needs appropriate code; a set of suitable plugs for it are already given in the kernel. These are

space-footnotes-floats After the galley text there is a vertical `\vfil` followed by any footnotes followed by the bottom floats, if any.

footnotes-space-floats As before but the `\vfil` is between footnotes and floats.

floats-space-footnotes Floats are directly after the text, then a space and then footnotes at the bottom.

space-floats-footnotes Both floats and footnotes are pushed to the bottom with footnotes last.[63]

[63] There are two more permutations, but neither of them has ever been requested so they aren't set up by default — doing that in a class would be trivial though.

`floats-footnotes` All excess space is distributed across the existing glue on the page, e.g., within the text galley, the separation between blocks, etc. The order is text, floats, footnotes.

`footnotes-floats` As the previous one but floats and footnotes are swapped. This is the LaTeX default for newer document, i.e., this plug is assigned to the socket when `\DocumentMetadata` is used.

`footnotes-floats-legacy` As the previous one but LaTeX's bottom skip bug is not corrected, i.e., in ragged bottom designs where footnotes are supposed to be directly attached to the text, they suddenly appear at the bottom of the page when the page is ended with `\newpage` or `\clearpage`. While this is clearly a bug, it was the case since the days of LaTeX 2.09; thus for compatibility we continue to support this behavior.

`build/column/footnotes` (0 arguments) This socket is used to manipulate the footnote material inside `\box\footins`. It if contains code, it is supposed to do some processing of that box and then write the result back into it (and nothing else!). By default it does nothing, i.e., has the `noop` assigned.

If (short) footnotes are run as a paragraph this socket gets the plug `para` assigned which is defined elsewhere.

`build/column/baselineattach` (0 arguments) By default, footnotes are attached after the last line of text by adding a `\vskip\skip\footins`, i.e., the space is measured from the bottom of `\@outputbox` (plug `off`). Bottom floats are attached in a similar way by using `\vskip\textfloatsep`.

The socket can get the plug `on` assigned, in which case the skip starts at the baseline of the last line in `\@outputbox`, if there is one; otherwise it starts still from the bottom of the box.

2.1.3 Tagging sockets

The following sockets are used to implement tagging. They are used via `\UseTaggingSocket` which turns them off when tagging is disabled. For each of them a `default` plug is implemented that holds the tagging code.

`build/column/outputbox` (0 arguments) This socket is used to add any missing tagging structures to the `\@outputbox` box, if necessary.

`build/column/footins` (0 arguments) This socket is used to add any missing tagging structures to `\footins` box, if necessary.

`build/page/header` (2 arguments) This socket receives the content of the formatted page header as its second argument and adds the necessary tagging around it. The first argument is empty as no special setup is necessary.

`build/page/footer` (2 arguments) This socket receives the content of the formatted page footer as its second argument and adds the necessary tagging around it. The first argument is empty as no special setup is necessary.

2.1.4 Output routine commands

\output
\@specialoutput

This needs some small adjustments. We cannot guarantee that the float mechanism will interact correctly with this stuff, but that mechanism does not always work properly with footnotes already.

RmS 91/09/29:

added reset of \par to the output routine. This avoids problems when the output routine is called within a list where \par may be a no-op.

```
\output {%
  \let \par \@@par
  \ifnum \outputpenalty<-\@M
    \@specialoutput
  \else
    \@makecol
    \@opcol
```

Moved to \@opcol: \@floatplacement.

```
    \@startcolumn
```

This loop could be replaced by an \expandafter tail recursion in \@startcolumn.

```
    \@whilesw \if@fcolmade \fi
      {%
⟨*trace⟩
       \fl@trace{PAGE: float \if@twocolumn column \else page \fi
                     completed}%
⟨/trace⟩
       \@opcol\@startcolumn}%
  \fi
  \ifnum \outputpenalty>-\@Miv
```

At points such as this we need to check that there is still a minimal amount of room left on the page; this uses an arbitrary small value at present. If there is little room left we just force a page-break, OK?

This bit is essential only if a float has just been processed so maybe it should be moved; but this is the natural place at which to set the vsize and a test would need to be done anyway. A check has been added to ensure that there really has been a change in the value of \@colroom.

Since this happens only when there is nothing on the page but floats, the empty box should not cause any problem other than some overfull box messages, which is not entirely misleading.

The twocolumn case does not need any extra code here since this is the \output itself; in the second column there will still not be enough room left so \@emptycol will be executed again when the OR is called by the-page builder when it gets to the penalty inserted by the first execution. (The page-builder is never invoked whilst the OR is being executed since it builds a inner vlist; thus any conditional code for the two-column case within \output may not get executed with the correct value of \if@firstcolumn.

```
    \ifdim \@colroom<1.5\baselineskip
      \ifdim \@colroom<\textheight
        \@latex@warning@no@line {Text page \thepage\space
                                 contains only floats}%
        \@emptycol
%         \if@twocolumn
%           \if@firstcolumn
```

```
%           \else
%             \@emptycol
%           \fi
%         \fi
      \else
        \global \vsize \@colroom
      \fi
    \else
      \global \vsize \@colroom
    \fi
  \else
    \global \vsize \maxdimen
  \fi
}
```

Historical LaTeX 2.09 comments (not necessarily accurate any more):
CHANGES TO `\@specialoutput`:
* `\penalty\z@` changed to `\penalty\interlinepenalty` so `\samepage` works properly with figure and table environments. (Changed 23 Oct 86)

* Definition of `\@specialoutput` changed 26 Feb 88 so `\@pageht` and `\@pagedp` aren't changed for a marginal note. (Change suggested by Chris Rowley.)

End of historical LaTeX 2.09 comments.

```
\gdef\@specialoutput{%
   \ifnum \outputpenalty>-\@Mii
     \@doclearpage
   \else
     \ifnum \outputpenalty<-\@Miii
       \ifnum \outputpenalty<-\@MM \deadcycles \z@ \fi
       \global \setbox\@holdpg \vbox {\unvbox\@cclv}%
     \else
```

Note that `\boxmaxdepth` should not be set here since we wish to record the natural depth of the holdpg box.

This is changed so as to not lose anything, such as writes and marks, which may get into box 255 and should be returned to the list. This should only happen when the first penalty in the mechanism is discarded and therefore `\@holdpg` should always be void in this case. This can happen because a penalty is discarded whenever there is no box on the list.

It was just: `\setbox\@tempboxa \box \@cclv`.

The last box which is removed is the box put there by the double-penalty mechanism. The `\unskip` then removes the `\topskip` which is put there since the box is the first on the page.

```
       \global \setbox\@holdpg \vbox{%
                       \unvbox\@holdpg
                       \unvbox\@cclv
```

We must now remove the box added by the float mechanism and the `\topskip` glue therefore added above it by TeX.

```
                       \setbox\@tempboxa \lastbox
                       \unskip
```

```
}%
```

These two are needed as separate dimensions only by `\@addmarginpar`; for other purposes we put the whole size into `\@pageht` (see below).

```
      \@pagedp \dp\@holdpg
      \@pageht \ht\@holdpg
      \unvbox \@holdpg
      \@next\@currbox\@currlist{%
        \ifnum \count\@currbox>\z@
```

Putting the whole size into `\@pageht` (see above).

```
          \advance \@pageht \@pagedp
          \ifvoid\footins \else
            \advance \@pageht \ht\footins
            \advance \@pageht \skip\footins
            \advance \@pageht \dp\footins
          \fi
          \ifvbox \@kludgeins
```

We want to make the adjustment due to this insert only if the non-star form is used. The *-form will probably not work with floats, but maybe it still could make some adjustment here even so?

```
            \ifdim \wd\@kludgeins=\z@
              \advance \@pageht \ht\@kludgeins
⟨*trace⟩
              \fl@trace {Extra size added: \the \ht\@kludgeins}%
⟨/trace⟩
            \fi
          \fi
```

This version puts the inserts back just before the additional material; it could be moved earlier, before unboxing the page-so-far. Neither is guaranteed not to put things on the wrong page. This version is similar to the original version.

```
          \@reinserts
          \@addtocurcol
        \else
          \@reinserts
          \@addmarginpar
        \fi
        }\@latexbug
```

A 2e change: use `\addpenalty` instead of `\penalty` here. Some penalty is needed to create a potential break-point immediately after the reinserts (or the marginal). Otherwise there can be no possibility to break here and this can cause the reinserts or the marginal to appear on the next page (which is often incorrect). However, if the nobreak flag is true, a `\nobreak` must be correct.

```
      \ifnum \outputpenalty<\z@
        \if@nobreak
          \nobreak
        \else
          \addpenalty \interlinepenalty
        \fi
      \fi
    \fi
  \fi
```

```
}
⟨/2ekernel | fltrace⟩
```

(*End of definition for* `\output` *and* `\@specialoutput`.)

`\@testwrongwidth` `\f@depth` Test if the float box has the wrong width when trying to place it into some area. (Actually the test is for a conventional depth setting rather than for the width of the float. For that reason the box depth was explicitly tailored when the float was created).

```
⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩                  {\@testwrongwidth}{float order in 2-column}%
⟨*2ekernel | latexrelease | fltrace⟩
\def\@testwrongwidth #1{%
  \ifdim\dp#1=\f@depth
⟨*trace⟩
    \fl@trace{\string#1
              \ifdim\f@depth=\z@ single \else double \fi
              column float -- ok}%
⟨/trace⟩
  \else
    \global\@testtrue
⟨*trace⟩
    \fl@trace{\string#1
              \ifdim\f@depth=\z@ double \else single \fi
              column float -- wrong}%
⟨/trace⟩
  \fi}%
```

Normally looking for single column floats, which have zero depth.

```
\let\f@depth\z@
⟨/2ekernel | latexrelease | fltrace⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\@testwrongwidth}{float order in 2-column}%
⟨latexrelease⟩\let\@testwrongwidth\@undefined
⟨latexrelease⟩\let\f@depth\@undefined
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\@testwrongwidth` *and* `\f@depth`.)

`\@doclearpage` This is a very much an emergency action, just dumping everything: footnotes first then floats. A more sophisticated version is needed; but even more urgent is a bug-free version (see, for example, pr/3528).

Also, it puts any left-over non-boxes (writes, specials, etc.) back after any float pages created: this is a very bad bug since, for example, a kludge insert will be in quite the wrong place and, worse, be irremovable and uncancelable.

All the remaining changes are replacing the double column defer list or inserting the extra test `\@testwrongwidth{`⟨*box*⟩`}` at suitable places. That is at places where a box is taken off the deferlist.

```
⟨latexrelease⟩\IncludeInRelease{2015/01/01}{\@doclearpage}%
⟨latexrelease⟩                             {float order in 2-column}%
⟨*2ekernel | latexrelease⟩
\def \@doclearpage {%
     \ifvoid\footins
```

```
        \ifvbox\@kludgeins
          {\setbox \@tempboxa \box \@kludgeins}%
⟨*trace⟩
          \fl@trace {kludgeins box made void}%
⟨/trace⟩
        \fi
        \setbox\@tempboxa\vsplit\@cclv to\z@ \unvbox\@tempboxa
        \setbox\@tempboxa\box\@cclv
        \xdef\@deferlist{\@toplist\@botlist\@deferlist}%
        \global \let \@toplist \@empty
        \global \let \@botlist \@empty
        \global \@colroom \@colht
        \ifx \@currlist\@empty
        \else
           \@latex@error{Float(s) lost}\@ehb
           \global \let \@currlist \@empty
        \fi
        \@makefcolumn\@deferlist
        \@whilesw\if@fcolmade \fi{\@opcol\@makefcolumn\@deferlist}%
        \if@twocolumn
          \if@firstcolumn
            \xdef\@deferlist{\@dbltoplist\@deferlist}%
            \global \let \@dbltoplist \@empty
            \global \@colht \textheight
            \begingroup
               \@dblfloatplacement
               \@makefcolumn\@deferlist
               \@whilesw\if@fcolmade \fi{\@outputpage
                                         \@makefcolumn\@deferlist}%
            \endgroup
          \else
            \vbox{}\clearpage
          \fi
        \fi
```

the next line is needed to avoid losing floats in certain circumstances a single call to the original `\doclearpage` will now no longer output all floats.

```
        \ifx\@deferlist\@empty \else\clearpage \fi
      \else
        \setbox\@cclv\vbox{\box\@cclv\vfil}%
        \@makecol\@opcol
        \clearpage
      \fi
}%
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\@doclearpage}%
⟨latexrelease⟩                                {float order in 2-column}%
⟨latexrelease⟩\def \@doclearpage {%
⟨latexrelease⟩      \ifvoid\footins
```

We empty any left over kludge insert box here; this is a temporary fix. It should perhaps be applied to one page of cleared floats, but who cares? The whole of this stuff needs

completely redoing for many such reasons.

```
⟨latexrelease⟩        \ifvbox\@kludgeins
⟨latexrelease⟩          {\setbox \@tempboxa \box \@kludgeins}%
⟨*trace⟩
⟨latexrelease⟩          \fl@trace {kludgeins box made void}%
⟨/trace⟩
⟨latexrelease⟩        \fi
⟨latexrelease⟩        \setbox\@tempboxa\vsplit\@cclv to\z@ \unvbox\@tempboxa
⟨latexrelease⟩        \setbox\@tempboxa\box\@cclv
⟨latexrelease⟩        \xdef\@deferlist{\@toplist\@botlist\@deferlist}%
⟨latexrelease⟩        \global \let \@toplist \@empty
⟨latexrelease⟩        \global \let \@botlist \@empty
⟨latexrelease⟩        \global \@colroom \@colht
⟨latexrelease⟩        \ifx \@currlist\@empty
⟨latexrelease⟩        \else
⟨latexrelease⟩           \@latexerr{Float(s) lost}\@ehb
⟨latexrelease⟩           \global \let \@currlist \@empty
⟨latexrelease⟩        \fi
⟨latexrelease⟩        \@makefcolumn\@deferlist
⟨latexrelease⟩        \@whilesw\if@fcolmade \fi
⟨latexrelease⟩                      {\@opcol\@makefcolumn\@deferlist}%
⟨latexrelease⟩        \if@twocolumn
⟨latexrelease⟩          \if@firstcolumn
⟨latexrelease⟩            \xdef\@dbldeferlist{\@dbltoplist\@dbldeferlist}%
⟨latexrelease⟩            \global \let \@dbltoplist \@empty
⟨latexrelease⟩            \global \@colht \textheight
⟨latexrelease⟩            \begingroup
⟨latexrelease⟩               \@dblfloatplacement
⟨latexrelease⟩               \@makefcolumn\@dbldeferlist
⟨latexrelease⟩               \@whilesw\if@fcolmade \fi
⟨latexrelease⟩                     {\@outputpage\@makefcolumn\@dbldeferlist}%
⟨latexrelease⟩            \endgroup
⟨latexrelease⟩          \else
⟨latexrelease⟩            \vbox{}\clearpage
⟨latexrelease⟩          \fi
⟨latexrelease⟩        \fi
⟨latexrelease⟩      \else
⟨latexrelease⟩        \setbox\@cclv\vbox{\box\@cclv\vfil}%
⟨latexrelease⟩        \@makecol\@opcol
⟨latexrelease⟩        \clearpage
⟨latexrelease⟩      \fi
⟨latexrelease⟩   }%
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\@doclearpage`.)

`\@opcol` Several changes in detail here.

```
⟨*2ekernel | fltrace⟩
\def \@opcol {%
  \if@twocolumn
```

The funny-looking internal commands are interfacing with the new marks mechanism. We make sure (elsewhere) that those are always defined, even when we roll back, so here we add them unconditionally. This still need turning into a hook or config point eventually:

```
    \@expl@@@mark@update@dblcol@structures@@
    \@outputdblcol
  \else
    \@expl@@@mark@update@singlecol@structures@@
    \@outputpage
⟨*trace⟩
    \fl@trace{PAGE: one column (float? see above) page completed}%
⟨/trace⟩
```

Not needed since it comes after `\@outputpage`:

```
%   \global\@colht\textheight
  \fi
```

These do not need to be done every time `\@opcol` is used: they should be grouped together since they all need to be done at the end of the non-special output routine, or at the end of a clearpage one.

```
  \global \@mparbottom \z@ \global \@textfloatsheight \z@
  \@floatplacement
}
⟨/2ekernel | fltrace⟩
```

(*End of definition for* `\@opcol`.)

```
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease⟩    {\@makecol}{\@makecol adding hooks/sockets}%
```

`\@makecol` `\@makecol` is shortened a lot, basically all the hardwired code in the middle has moved into a socket.

```
\def \@makecol {%
```

A number of packages want to prepend code to `\@makecol`; this is the hook for that:

```
  \UseHook {build/column/before}%
```

Save away box 255 as `\@outputbox` to make it available for further adjustments.

```
  \setbox\@outputbox \box\@cclv
```

The only real addition is the next command which either does nothing or removes an infinite glue from the bottom of the `\@outputbox`.

```
  \@outputbox@removebskip
```

Now a kernel hook for tagging that adjusts the content of `\@outputbox`, if necessary. At this point it just contains the material from the galley.

```
  \UseTaggingSocket{build/column/outputbox}%
```

When this code is run any "here" floats in the `\@outputbox` are already handled, so we recycle their registers and put them back to the `\@freelist`.

```
  \let\@elt\relax
  \xdef\@freelist{\@freelist\@midlist}%
  \global \let \@midlist \@empty
```

Here we have the configurable part. This socket is supposed to add floats, footnotes and stretchable vertical space as appropriate to the `\@outputbox`. It is used by packages such as footmisc to implement different layout, e.g., footnotes above or below bottom floats, etc.

```
\UseSocket {build/column/outputbox}%
```

Then we deal with any `\enlargethispage` or run the normal code to build a column.

```
\ifvbox\@kludgeins
   \@make@specialcolbox
\else
   \@make@normalcolbox
\fi
\global \maxdepth \@maxdepth
```

Finally, another hook for external packages or classes that want to augment or alter the output routine by appending to `\@makecol`.

```
\UseHook {build/column/after}%
}
```

(*End of definition for* `\@makecol`.)

`\@outputbox@depth` We need to know the depth of `\@outputbox` once in a while. Rather than using a temp dimen (as it was done in the past), we give it a proper register.

The value of this register is only correct inside of `\outputbox@append`, elsewhere you can't rely on it!

```
\newdimen\@outputbox@depth
```

(*End of definition for* `\@outputbox@depth`.)

`\@make@normalcolbox` Taken out of `\@makecol` for readability.

```
\def \@make@normalcolbox {%
   \setbox\@outputbox \vbox to\@colht {%
       \@texttop
       \@outputbox@depth \dp\@outputbox
       \unvbox \@outputbox
```

The `\vskip -\@outputbox@depth@` ensures that the visible depth of the box does not affect the placement of anything on the page. Thus very deep pages will overprint the footer; but these should have been prevented by suitable settings of the maxdepths at appropriate times.

If `\@textbottom` ends with a box or rule of non-zero depth then this skip adjustment should be done again after it.

```
       \vskip -\@outputbox@depth
       \@textbottom
      }%
}
```

(*End of definition for* `\@make@normalcolbox`.)

`\@make@specialcolbox` Make the colbox when `\enlargethispage` was used.

```
⟨/2ekernel | latexrelease⟩
⟨*2ekernel | latexrelease | fltrace⟩
\def \@make@specialcolbox {%
⟨*trace⟩
```

```
    \fl@trace{Kludgeins ht \the\ht\@kludgeins\space
                           dp \the\dp\@kludgeins\space
                           wd \the\wd\@kludgeins}%
⟨/trace⟩
    \@outputbox@append {\vskip-\@outputbox@depth}%
    \@tempdima \@colht
    \ifdim \wd\@kludgeins>\z@
```

Note that in this case (the *-version), the height of the `\@kludgeins` box is not used since its value is somewhat arbitrary: it need only be big enough to ensure that the page-break is not taken prematurely.

Here we calculate how much vertical space needs to be added in order to enable the column to fit into a box of size `\@colht` using the best information we have about the amount of shrink available (another thing which is known internally about a box, but cannot be accessed at the TEX level!).

This needs TEX3 otherwise `\pageshrink` is zero anyway; it may not be exactly the figure we wish as it is the total available from the all the material collected before the page-break decision is made. It will, we think, always be an overestimate of the actual shrink in the box; therefore this should always force the shortest possible column with the possibility of an overfull box.

This should work for both flush- and ragged-bottom setting since it makes the contents no smaller than the size (`\@colht`) of the box into which they are put.

Their should perhaps be an upper limit, of 0pt?, on the extra space added to force shrinking.

```
      \advance \@tempdima -\ht\@outputbox
      \advance \@tempdima \pageshrink
⟨*trace⟩
      \fl@trace {Natural ht of col: \the \ht\@outputbox}%
      \fl@trace {\string \@colht: \the \@colht}%
      \fl@trace {Pageshrink added: \the \pageshrink}%
      \fl@trace {Hence, space added: \the \@tempdima}%
⟨/trace⟩
      \setbox\@outputbox \vbox to \@colht {%
        \unvbox\@outputbox
        \vskip \@tempdima
        \@textbottom
        }%
```

For the unstarred version, the final size of the page is precisely specified. Therefore, at least for the flush-bottom case, we need to ensure that, visually, it has this size exactly.

Thus we calculate this size and set the material in a box of this size, which is then put into a box of size `\@colht` with `\vss` at the bottom.

```
    \else
      \advance \@tempdima -\ht\@kludgeins
⟨*trace⟩
      \fl@trace {Natural ht of col: \the \ht\@outputbox}%
      \fl@trace {\string \@colht: \the \@colht}%
      \fl@trace {Extra size added: -\the \ht \@kludgeins}%
      \fl@trace {Hence, height of inner box: \the \@tempdima}%
      \fl@trace {Max? pageshrink available: \the \pageshrink}%
⟨/trace⟩
```

This type of final packaging could be done always; this may simplify all of this page-makeup.

It is not necessary to set `\boxmaxdepth` here since the `\@outputbox` ends with glue.

```
      \setbox \@outputbox \vbox to \@colht {%
        \vbox to \@tempdima {%
          \unvbox\@outputbox
          \@textbottom}%
        \vss}%
    \fi
    {\setbox \@tempboxa \box \@kludgeins}%
⟨*trace⟩
      \fl@trace {kludgeins box made void}%
⟨/trace⟩
}
⟨/2ekernel | latexrelease | fltrace⟩
⟨*2ekernel | latexrelease⟩
```

(*End of definition for* `\@make@specialcolbox`.)

`\@outputbox@removebskip`

This is really a bug fix for the kernel (from the 2.09 days) but one we only make by default in new documents that are using `\DocumentMetadata`. If `\raggedbottom` is in force, footnotes get attached to the main galley at a distance of `\footskip` on all pages except on those that are ended by `\newpage` or `\clearpage` where the `\vfil` from `\newpage` pushes the footnotes to the very bottom.

This is kind of a weird difference to a page ending with `\pagebreak`—in that case the page is also run short, but the footnotes are not pushed to the bottom.

In footmisc `\@outputbox@removebskip` is only applied when footmisc is called with an option specifying the footnote placement, i.e., not in the default case. In new documents we apply it always.

```
\def\@outputbox@removebskip{%
```

In some special circumstances the `\@outputbox` might be void. As `\@outputbox@append` below would change this, we handle this case by leaving it unchanged, because otherwise we can no longer detect that situation (needed, for example, in ftnright).

```
    \ifvoid \@outputbox
      \global\let\@outputbox@reinsertbskip\relax
    \else
```

If it is not void then we drop the final skip at the end of `\@outputbox` provided there is one and it has a glue stretch order of 1 or more (i.e., contains a `fil` or `fill` part).

```
      \@outputbox@append{%
        \@tempskipa\lastskip
        \ifnum \gluestretchorder\@tempskipa>\z@
          \unskip
```

`\@outputbox@reinsertbskip`

We also prepare for reinserting the skip we removed elsewhere on the page. As we have to do this globally, we also need to explicitly reset `\@outputbox@reinsertbskip` if we don't find any such glue.

```
          \xdef\@outputbox@reinsertbskip
              {%
```

Everything is done inside `\@outputbox@append` so that a call to `\@outputbox@reinsertbskip` will update the box. If there are no footnotes and no bottom floats that still needs to be attached we first backup by the depth of the `\@outputbox` so that the skip we add starts from the baseline of the last line inside (if there is one). As it is a `fil`, `fill`, or `filll` glue this normally makes no differences, but if the box itself contains a fill skip of higher order our fill is ignored and text is pushed to the bottom of the page. In that case and without backing up first, the last line would not be with its baseline on the bottom, but slightly too high if it has characters with descenders.

```
                \noexpand\@outputbox@append{%
                  \noexpand\@if@footnotes@TF
                      {}%
                      {%
                       \noexpand\@if@bottomfloats@TF
                          {}%
                          {\noexpand\@backup@outputbox@depth}%
                      }%
                  \vskip\the\@tempskipa}}%
      \else
        \global\let\@outputbox@reinsertbskip\relax
      \fi
    }%
  \fi
}
```

We need a trivial top-level definition for `\@outputbox@reinsertbskip` in case the first page has no bottom glue and the command gets called.

```
\let \@outputbox@reinsertbskip \relax
```

(*End of definition for* `\@outputbox@removebskip` *and* `\@outputbox@reinsertbskip`.)

`\@backup@outputbox@depth` In some places we have to make the depth of the `\@outputbox` zero by backing up by its current depth. This is done by `\@backup@outputbox@depth`. The macro can only be used inside of `\@outputbox@append` (or after you have manually set the `\@outputbox@depth` to the right value).

If the depth is zero, there is no point in doing anything, and if the depth is negative it must have been deliberately set to this value, so in this case we also do not back up and change it.

If it is positive we back up by the minimum of `\@outputbox@depth` and `\maxdepth`, i.e., if the depth is unusually large we do not perform a full backup.

```
\def\@backup@outputbox@depth{%
%  \typeout{--> @backup@outputbox@depth : \the \@outputbox@depth}%
  \ifdim\@outputbox@depth>\z@
    \vskip -%
      \ifdim\@outputbox@depth>\maxdepth
        \maxdepth
      \else
        \@outputbox@depth
      \fi
```

After the skip was added the depth of the box will become zero but this is not immediately reflected in `\@outputbox@depth`. Thus, if there is further code in the same `\@outputbox@append` testing the register it would make a wrong conclusion. We therefore set it locally to zero.

```
    \@outputbox@depth\z@
  \fi
}
```

(*End of definition for* `\@backup@outputbox@depth`.)

2.2 The output routine configuration components

Here we provide the commands that are used to define code for the socket `build/column/outputbox`. The all manipulate the `\@outputbox` box in one way or another.

`\@outputbox@append` This general purpose command alters the `\@outputbox` box by appending material to it. As this is a box typesetting operation we make sure that the last line of the box reflects the true depth of the last line (in case that is needed later). We also expose the current depth of `\@outputbox` as `\@outputbox@depth` before unboxing so that its value can be used by `#1` if wanted.

```
\def\@outputbox@append #1{%
  \setbox\@outputbox \vbox {%
```

This `\boxmaxdepth` setting is to ensure that deep footnotes do not overwrite the footer (on account of the negative skip added later): it should use `\@maxdepth` otherwise the change is pointless when there are footnotes.

But see also its use when combining floats.

```
    \boxmaxdepth \@maxdepth
    \@outputbox@depth\dp\@outputbox      % if needed in #1
    \unvbox \@outputbox
    #1%
  }%
}
```

(*End of definition for* `\@outputbox@append`.)

`\@outputbox@appendfil` Append a `\vfil` to the `\@outputbox`. Depending on the plug in the `build/column/baselineattach` either from the bottom of the box or the baseline of the last line.

```
\def\@outputbox@appendfil{\@outputbox@append
 {\UseSocket{build/column/baselineattach}\vfil}}
```

(*End of definition for* `\@outputbox@appendfil`.)

`\@outputbox@appendfootnotes` This command appends the footnotes to the `\@outputbox` (if there are any). If not, then it does nothing.

```
\def\@outputbox@appendfootnotes {%
  \ifvoid\footins \else
```

Handling split footnotes need further work so this is for the moment just a dummy that does nothing. It might vanish and get replaced by a socket eventually.

```
    \@makecol@handlesplitfootnotes
```

The following socket can be used to manipulate the material in `\footins` box, for example, if the footnotes are to be presented all together in a single paragraph. By default it does nothing.

```
    \UseSocket{build/column/footnotes}%
```

Then the footnotes are appended:

```
    \@outputbox@append{%
```

First we call a socket which controls whether or not we back up by the depth of the `\@outputbox` so that the skip that separates the footnotes from the text starts at the baseline of the last text line.

```
        \UseSocket{build/column/baselineattach}%
        \vskip \skip\footins
```

The next socket adjusts the tagging of the `\footins` box, if necessary.

```
        \UseTaggingSocket{build/column/footins}%
        \color@begingroup
          \normalcolor
          \footnoterule
```

Support for pdfcolfoot; eventually this can go once color is properly supported. The csname is constructed in case the command is not defined, i.e., the package not loaded.

```
          \csname pdfcolfoot@current\endcsname
          \unvbox \footins
        \color@endgroup
      }%
  \fi
}
```

(*End of definition for* `\@outputbox@appendfootnotes`.)

`\@makecol@handlesplitfootnotes` For future extensions ...

```
\let \@makecol@handlesplitfootnotes \@empty
```

(*End of definition for* `\@makecol@handlesplitfootnotes`.)

`\@outputbox@attachfloats` `\@outputbox@attachtopfloats` `\@outputbox@attachbottomfloats` Attaching top and bottom floats can usually be done in one go, but for special layouts we might want more control so we provide also separate commands.

The next command was called `\@combinefloats` in the past.

```
\def \@outputbox@attachfloats {%
    \@outputbox@attachtopfloats
    \@outputbox@attachbottomfloats
}
\def \@outputbox@attachtopfloats {%
  \ifx \@toplist\@empty \else \@cflt \fi
}
```

Before we append the bottom floats we run the `build/column/baselineattach` so that the `\textfloatsep` is measured from the baseline if that is desired.

```
\def \@outputbox@attachbottomfloats {%
  \ifx \@botlist\@empty \else
    \@outputbox@append{\UseSocket{build/column/baselineattach}}%
    \@cflb
  \fi
}
```

(*End of definition for* `\@outputbox@attachfloats`, `\@outputbox@attachtopfloats`, *and* `\@outputbox@attachbottomfloats`.)

The next three conditionals might be useful when setting up running headers and footers so perhaps they should be change to CamelCase names. For now they are internal.

`\@if@flushbottom@TF` Test for `\flushbottom` (currently not used).

```
\def\@if@flushbottom@TF{%
  \ifx\@textbottom\relax
    \expandafter\@firstoftwo
  \else
    \expandafter\@secondoftwo
  \fi
}
```

(*End of definition for* `\@if@flushbottom@TF`.)

`\@if@footnotes@TF` Test if footnotes are present on the current page or column and not yet added to the `\@outputbox`.

```
\def\@if@footnotes@TF{%
  \ifvoid\footins
    \expandafter\@secondoftwo
  \else
    \expandafter\@firstoftwo
  \fi
}
```

(*End of definition for* `\@if@footnotes@TF`.)

`\@if@bottomfloats@TF` Test if bottom floats are around and not yet added to the `\@outputbox`.

```
\def\@if@bottomfloats@TF{%
  \ifx \@botlist\@empty
    \expandafter\@secondoftwo
  \else
    \expandafter\@firstoftwo
  \fi
}
```

(*End of definition for* `\@if@bottomfloats@TF`.)

`build/column/outputbox` (*socket*) We have one socket that is supposed to augment the `\@outputbox` by attaching floats and footnotes with appropriate spacing.

```
\NewSocket{build/column/outputbox}{0}
```

The following plugs are available for this socket:

`space-footnotes-floats` (*plug*) After the galley text there is a vertical `\vfil` followed by any footnotes followed by the bottom floats, if any.

```
\NewSocketPlug {build/column/outputbox}{space-footnotes-floats} {%
  \@if@footnotes@TF
      {\@outputbox@appendfil}%
      {\@if@bottomfloats@TF
        {\@outputbox@appendfil}%
        {\@outputbox@reinsertbskip}%
      }%
  \@outputbox@appendfootnotes
  \@outputbox@attachfloats
}
```

`footnotes-space-floats` (*plug*) As before but the `\vfil` is between footnotes and floats.

```
\NewSocketPlug {build/column/outputbox}{footnotes-space-floats} {%
  \@outputbox@appendfootnotes
  \@if@bottomfloats@TF
      {\@outputbox@appendfil}%
      {\@outputbox@reinsertbskip}%
  \@outputbox@attachfloats
}
```

`floats-space-footnotes` (*plug*) Floats immediately after the galley text and footnotes at the bottom.

```
\NewSocketPlug {build/column/outputbox}{floats-space-footnotes} {%
  \@outputbox@attachfloats
  \@if@footnotes@TF
      {\@outputbox@appendfil}%
      {\@outputbox@reinsertbskip}%
  \@outputbox@appendfootnotes
}
```

`space-floats-footnotes` (*plug*) Both floats and footnotes are pushed to the bottom with footnotes last.[64]

```
\NewSocketPlug {build/column/outputbox}{space-floats-footnotes} {%
  \@if@bottomfloats@TF
      {\@outputbox@appendfil}%
      {\@if@footnotes@TF
         {\@outputbox@appendfil}%
         {\@outputbox@reinsertbskip}}%
  \@outputbox@attachfloats
  \@outputbox@appendfootnotes
}
```

`floats-footnotes` (*plug*) All excess space has to be distributed across the existing glue on the page, e.g., within the text galley, the separation between blocks, etc. The order is text, floats, footnotes.

```
\NewSocketPlug {build/column/outputbox}{floats-footnotes} {%
  \@outputbox@attachfloats
  \@outputbox@appendfootnotes
```

We do reinsert the bottom skip from `\newpage` if it was taken out earlier. This is, strictly speaking, not necessary in most cases, but it is a `\vfil` while `\raggedbottom` is only generating `\vspace{0pt plus .0001fil}`, so if you have several `\vfil` on the page before the `\newpage` you would alter the space distribution if one is taken out.

```
  \@outputbox@reinsertbskip
}
```

`footnotes-floats` (*plug*)

```
\NewSocketPlug {build/column/outputbox}{footnotes-floats} {%
  \@outputbox@appendfootnotes
  \@outputbox@attachfloats
  \@outputbox@reinsertbskip
}
```

[64] There are two more permutations, but neither of them has ever been requested so they aren't set up by default — doing that in a class would be trivial though.

The footnote-floats plug implements the layout used by LaTeX but with the bottom skip bug corrected. This will be the default when \DocumentMetadata is used; it can be overwritten either through footmisc or by assigning any of the other plugs (or by coding yet another plug for the socket).

footnotes-floats-legacy (*plug*) This implements the 2.09 layout (including its bottom skip bug).

```
\NewSocketPlug {build/column/outputbox}{footnotes-floats-legacy} {%
```

In the legacy case we don't really want to take out the bottom skip, but rather than altering \@outputbox@removebskip in \makecol to do nothing, which we would then have to undo in every other layout, we immediately reinsert the dropped skip again.

In the legacy case there is, however, one other thing that needs doing: if there was a \vfil at the bottom of the page (e.g., from \newpage) and there are no footnotes, then we need to back up by the depth of the last line in the page box. Otherwise you might end up with incorrect alignment if there is a higher-order fill on the page (e.g., \vfill) that pushes the text material to the bottom, since in such cases the depth will not be taken into account by TeX.

This is done automatically by \@outputbox@reinsertbskip as long as there are no footnotes waiting to be placed. If there are footnotes, the operation is not done because by default the \skip\footins separation is measured from the bottom of the line not from the baseline. But if the user has asked to change this behavior we should honor it and that is done by putting the experimental socket build/column/baselineattach in which performs the backing up if desired. This should happen only if footnotes are actually present, so we test for this. Otherwise, there may be bottom floats and the \textfloatsep should be measured from the bottom of the last line and if not then any necessary back up is done by \@outputbox@reinsertbskip.

Note that after this code has backed up, the depth of \@outputbox will be zero as there is a skip at its bottom. So even if the same code is executed a second time (in \@outputbox@reinsertbskip or in \@outputbox@appendfootnotes) it will not alter the box further.

```
  \@if@footnotes@TF
    {\@outputbox@append{%
        \UseSocket{build/column/baselineattach}}%
    }{%
```

If there are no footnotes but there are bottom floats we execute the socket build/column/baselineattach so that \textfloatsep is measured from the baseline if that is being asked for.

```
      \@if@bottomfloats@TF
        {\@outputbox@append{%
            \UseSocket{build/column/baselineattach}}%
        }{}%
    }%
  \@outputbox@reinsertbskip
  \@outputbox@appendfootnotes
  \@outputbox@attachfloats
}
```

The footnote-floats-legacy plug is the default used by LaTeX when \DocumentMetadata is not used; it can be overwritten either through footmisc or by assigning any of the other plugs (or by coding yet another plug for the socket).

```
\AssignSocketPlug {build/column/outputbox}{footnotes-floats-legacy}
```

`build/column/footnotes` (*socket*) The socket allowing the manipulation of `\footins` box (result needs to be moved back in there). Used when footnotes are reformatted into a single paragraph by the `para` option of footmisc. By default it does nothing.

```
\NewSocket{build/column/footnotes}{0}
```

`l/column/baselineattach` (*socket*) Socket that allows backing up by the depth of the outputbox (but not more than `\maxdepth` before attaching any footnotes or floats.

```
\NewSocket{build/column/baselineattach}{0}
```

`on` (*plug*)

```
\NewSocketPlug {build/column/baselineattach}{on} {%
```

We do nothing if the depth is not positive, and we never back up by more than `\maxdepth`.

```
  \@backup@outputbox@depth
}
```

`off` (*plug*) The `off` plug is actually identical to the `noop` plug which is automatically made available; but offering that it under the name `off` makes the interface a bit nicer.

```
\NewSocketPlug {build/column/baselineattach}{off}{}
```

This feature is not turned on by default to preserve the behavior of old documents. With new documents using `\DocumentMetadata` it may become the default.

```
\AssignSocketPlug {build/column/baselineattach}{off}
```

`build/page/before` (*hook*) Hooks at the start and end of `\@outputpage` for use by packages.
`build/page/after` (*hook*)

```
\NewMirroredHookPair{build/page/before}{build/page/after}
```

`build/page/reset` (*hook*) Hook in `\@outputpage` to reset special galley conventions within the output routine. LaTeX does a lot of resetting in front of this hook, so by default it is empty. They are not made part of the hook (which would have been possible) to ensure that the kernel resets always come first and can be overwritten without the need to apply hook rules unnecessarily.

```
\NewHook {build/page/reset}
```

`build/column/before` (*hook*) Hooks at the start and end of `\@makecol` for use by packages.
`build/column/after` (*hook*)

```
\NewMirroredHookPair{build/column/before}{build/column/after}

⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩    {\@makecol}{\@makecol adding hooks/sockets}%
⟨latexrelease⟩
⟨latexrelease⟩\gdef \@makecol {%
⟨latexrelease⟩    \ifvoid\footins
⟨latexrelease⟩      \setbox\@outputbox \box\@cclv
⟨latexrelease⟩    \else
⟨latexrelease⟩      \setbox\@outputbox \vbox {%
⟨latexrelease⟩        \boxmaxdepth \@maxdepth
⟨latexrelease⟩        \unvbox \@cclv
⟨latexrelease⟩        \vskip \skip\footins
⟨latexrelease⟩        \color@begingroup
⟨latexrelease⟩          \normalcolor
```

```
⟨latexrelease⟩          \footnoterule
⟨latexrelease⟩          \unvbox \footins
⟨latexrelease⟩        \color@endgroup
⟨latexrelease⟩        }%
⟨latexrelease⟩    \fi
⟨latexrelease⟩    \let\@elt\relax
⟨latexrelease⟩    \xdef\@freelist{\@freelist\@midlist}%
⟨latexrelease⟩    \global \let \@midlist \@empty
⟨latexrelease⟩    \@combinefloats
⟨latexrelease⟩    \ifvbox\@kludgeins
⟨latexrelease⟩      \@makespecialcolbox
⟨latexrelease⟩    \else
⟨latexrelease⟩      \setbox\@outputbox \vbox to\@colht {%
⟨latexrelease⟩        \@texttop
⟨latexrelease⟩        \dimen@ \dp\@outputbox
⟨latexrelease⟩        \unvbox \@outputbox
⟨latexrelease⟩        \vskip -\dimen@
⟨latexrelease⟩        \@textbottom
⟨latexrelease⟩        }%
⟨latexrelease⟩    \fi
⟨latexrelease⟩    \global \maxdepth \@maxdepth
⟨latexrelease⟩}
⟨latexrelease⟩
⟨latexrelease⟩\gdef \@makespecialcolbox {%
⟨latexrelease⟩    \setbox\@outputbox \vbox {%
⟨latexrelease⟩      \@texttop
⟨latexrelease⟩      \dimen@ \dp\@outputbox
⟨latexrelease⟩      \unvbox\@outputbox
⟨latexrelease⟩      \vskip-\dimen@
⟨latexrelease⟩      }%
⟨latexrelease⟩    \@tempdima \@colht
⟨latexrelease⟩    \ifdim \wd\@kludgeins>\z@
⟨latexrelease⟩      \advance \@tempdima -\ht\@outputbox
⟨latexrelease⟩      \advance \@tempdima \pageshrink
⟨latexrelease⟩      \setbox\@outputbox \vbox to \@colht {%
⟨latexrelease⟩        \unvbox\@outputbox
⟨latexrelease⟩        \vskip \@tempdima
⟨latexrelease⟩        \@textbottom
⟨latexrelease⟩        }%
⟨latexrelease⟩    \else
⟨latexrelease⟩      \advance \@tempdima -\ht\@kludgeins
⟨latexrelease⟩      \setbox \@outputbox \vbox to \@colht {%
⟨latexrelease⟩        \vbox to \@tempdima {%
⟨latexrelease⟩          \unvbox\@outputbox
⟨latexrelease⟩          \@textbottom}%
⟨latexrelease⟩        \vss}%
⟨latexrelease⟩    \fi
⟨latexrelease⟩    {\setbox \@tempboxa \box \@kludgeins}%
⟨latexrelease⟩}
⟨latexrelease⟩
⟨latexrelease⟩ \let \@make@normalcolbox \@undefined
⟨latexrelease⟩ \let \@make@specialcolbox \@undefined
⟨latexrelease⟩ \let \@outputbox@removebskip \@undefined
⟨latexrelease⟩ \let \@outputbox@reinsertbskip \@undefined
```

```
⟨latexrelease⟩
⟨latexrelease⟩ \let \@outputbox@append \@undefined
⟨latexrelease⟩ \let \@outputbox@appendfootnotes \@undefined
⟨latexrelease⟩ \let \@outputbox@attachfloats \@undefined
⟨latexrelease⟩ \let \@outputbox@attachtopfloats \@undefined
⟨latexrelease⟩ \let \@outputbox@attachbottomfloats \@undefined
⟨latexrelease⟩
```

If we roll back we also need to restore this definition.

```
⟨latexrelease⟩ \def \@combinefloats {%
⟨latexrelease⟩    \ifx \@toplist\@empty \else \@cflt \fi
⟨latexrelease⟩    \ifx \@botlist\@empty \else \@cflb \fi
⟨latexrelease⟩ }
⟨latexrelease⟩
⟨latexrelease⟩ \let \@if@flushbottom@TF \@undefined
⟨latexrelease⟩ \let \@if@footnotes@TF \@undefined
⟨latexrelease⟩ \let \@if@bottomfloats@TF \@undefined
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

`\@reinserts` This is the code which reinserts the inserts. It puts them all in one place; this can make some of them come out on the wrong page. It has been put into a separate macro to expedite experimentation.

```
\gdef \@reinserts{%
  \ifvoid\footins\else\insert\footins{\unvbox\footins}\fi
  \ifvbox\@kludgeins\insert\@kludgeins
                       {\unvbox\@kludgeins}\fi
}
```

(*End of definition for* `\@reinserts`.)

`\@texttop` `\@textbottom` These do nothing as a default.

```
\let \@texttop \relax
\let \@textbottom \relax
```

(*End of definition for* `\@texttop` *and* `\@textbottom`.)

`\@resetactivechars` `\@activechar@info` RmS 93/09/06: added hook to protect against certain active characters in the output routine. Default checks are for active space and end-of-line.

```
\def\@activechar@info #1{%
      \@latex@info@no@line {Active #1 character found while
                            output routine is active
                            \MessageBreak
                            This may be a bug in a package file
                            you are using}%
}
```

Do not put any spaces in this next bit!

```
\begingroup
\obeylines\obeyspaces%
\catcode`\'\active%
\gdef\@resetactivechars{%
\def^^M{\@activechar@info{EOL}\space}%
\def {\@activechar@info{space}\space}%
```

```
\let'\active@math@prime}%
\endgroup
```

(*End of definition for* `\@resetactivechars` *and* `\@activechar@info`.)

`\@outputpage` `\@shipoutsetup` `\@writesetup`

The `\color@hbox` hooks here are used to avoid putting just a color special into an otherwise empty box (in a header or footer). These boxes are often set to be completely empty and so adding a special produces a very underfull box message.

There has been extensive tidying up of the old code here; including the removal of a level of grouping.

The setting of `\protect` immediately before the `\shipout` is needed so that protected commands within `\write`s are handled correctly.

Within shipout's vbox it is reset to its default value, `\relax`.

Resetting it to its default value after the shipout has been completed (and the contents of the writes have been expanded) must be done by use of `\aftergroup`. This is because it must have the value `\relax` before macros coming from other uses of `\aftergroup` within this box are expanded.

Putting this into the `\aftergroup` token list does not affect the definition used in expanding the `\write`s because the aftergroup token list is only constructed when popping the save-stack, it is not expanded until after the shipout is completed.

Question: should things from an `\aftergroup` within the shipped out box be executed in the environment set up for the writes, or after it finishes?

A lot of this code has been in-lined to prevent mis-use of internal commands as hooks.

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease⟩  {\@outputpage}{Use new mark mechanism}%
⟨*2ekernel | latexrelease⟩
```

Temp definition to vanish again when something is offered by the backend:

```
\protected \def \pdfannot@link@on@@  { \csname pdfannot_link_on:\endcsname }
\protected \def \pdfannot@link@off@@ { \csname pdfannot_link_off:\endcsname }
\def \@outputpage {%
```

We start with a hook available to packages that want to prepend code to `\@outputpage`.

```
 \UseHook {build/page/before}%
```

The `\endgroup` is put in by `\aftergroup`.

```
\begingroup
```

Now all the set-up stuff has been in-lined for Frank.

First the stuff for the writes.

From here on was originally in the command `\@writesetup`.

RmS 93/08/19: Redefined accents to allow changes in font encoding; but exactly why was this needed?

Reset `\language` to the value current at `\begin{document}`. In particular this ensures that a pagebreak in `verbatim` does not prevent hyphenation in the page head.

```
 \language\document@default@language
```

Now a few more resets for things that may have changed during galley processing. We start with resetting some active character definitions.

```
 \@resetactivechars
```

Then we deal with catcodes that might got altered by `verbatim` or similar environments and reset them to standard values.

```
\catcode`\\\z@
\catcode`\{\@ne
\catcode`\}\tw@
\catcode`\$\thr@@
\catcode`\&4\relax
\catcode`\^^M5\relax
\catcode`\#6\relax
\catcode`\^7\relax
\catcode`\_8\relax
\catcode`\ 10\relax
\catcode`\^^I10\relax
\@makeother\<%
\@makeother\>%
\@makeother\*%
\@makeother\.%
\@makeother\-%
\@makeother\/%
\@makeother\[%
\@makeother\]%
\@makeother\‘%
\@makeother\’%
\@makeother\"%

\@makeother\,%
\catcode`\~13\relax
\catcode`\%14\relax
```

If a page break happens between the start of a list and its first item then `@newlist` will be true and this will mess up any list that is used in the header or footer of the page. So we have to reset that flag.

```
\global\let\@@if@newlist\if@newlist
\global\@newlistfalse
```

This next hook replaces the following:

```
\let\-\@dischyph
\let\’\@acci\let\‘\@accii\let\=\@acciii
\let\\\@normalcr
\let\par\@@par %% 15 Sep 87 (this was once inside the box)
```

and it does more than they did; in particular it sets:

```
\parindent\z@
\parskip\z@skip
\everypar{}%
\leftskip\z@skip
\rightskip\z@skip
\parfillskip\@flushglue
\lineskip\normallineskip
\baselineskip\normalbaselineskip
\sloppy
```

```
\@parboxrestore
```

... to here was in the command `\@writesetup`.

Hook to allow adding resets local to the output routine processing, e.g., in `\write` or running headers/footers, for packages that set up special conventions in the main galley that should not leak into the page production just because a page break happens in the middle of such an environment.

```
\UseHook {build/page/reset}%
```

The following definition of `\protect` is the one active during the `\write` statement that migrate out of the `\shipout` box, so even though it is immediately changed below (inside `\shipout` it still has to be here!

```
\let \protect \noexpand
\shipout \vbox{%
```

Inside the `\shipout` box we have a different setting for `\protect`.

```
  \set@typeset@protect
  \aftergroup \endgroup
```

Once the `\shipout` box ends `\protect` is again `\noexpand` which is correct for any `\write` statements, but once they are processed we want to get back to the following setting:

```
  \aftergroup \set@typeset@protect
```

Now the setup inside the shipped out box; this should contain all the stuff that could only affect typesetting; other stuff may need to be reset for the writes also.

From here ... was in the command `\@shipoutsetup`.

```
 \if@specialpage
   \global \@specialpagefalse
   \@nameuse {ps@\@specialstyle}%
 \fi
 \if@twoside
   \ifodd\count\z@
     \let \@thehead \@oddhead
     \let \@thefoot \@oddfoot
     \let \@themargin \oddsidemargin
   \else
     \let \@thehead \@evenhead
     \let \@thefoot \@evenfoot
     \let \@themargin \evensidemargin
   \fi
 \fi
```

The rest was always inside the box.

RmS 91/08/15: added this line:

```
 \reset@font
```

RmS 93/08/06 Added `\lineskiplimit=0pt` to guard against it being nonzero: e.g. by `\offinterlineskip` being in effect.

There are probably lots of other things that may need resetting.

```
 \normalsize
```

Reset the space factors.

```
 \normalsfcodes
```

Reset these here (previously reset separately for head and foot)

```
\let \label \@gobble@with@sphack@om
\let \index \@gobble@with@sphack@som
\let \glossary \@gobble@with@sphack@om
\baselineskip \z@skip
\lineskip \z@skip
\lineskiplimit \z@
```

... to here was in the command `\@shipoutsetup`.

```
    \@begindvi
    \vskip \topmargin
    \moveright\@themargin \vbox {%
      \setbox\@tempboxa \vbox to\headheight {%
        \vfil
```

Tagging socket that receives the header in its second argument to surround the header with appropriate tagging structures (first argument is unused). If tagging is disabled it returns the content of the second argument.

```
        \pdfannot@link@off@@
        \UseTaggingSocket{build/page/header}{}%
          {
            \color@hbox
              \normalcolor
              \hb@xt@ \textwidth {\@thehead }%
            \color@endbox
          }
        \pdfannot@link@on@@
      }
      \dp \@tempboxa \z@
      \box \@tempboxa
      \vskip \headsep
      \box \@outputbox
      \baselineskip \footskip
```

Tagging socket that receives the footer in its second argument to surround the footer with appropriate tagging structures (first argument is unused). If tagging is disabled it returns the content of the second argument.

```
      \pdfannot@link@off@@
      \UseTaggingSocket{build/page/footer}{}%
         {
            \color@hbox
               \normalcolor
               \hb@xt@ \textwidth {\@thefoot }%
             \color@endbox
         }
      \pdfannot@link@on@@
      }
    }
```

`\endgroup` now inserted by `\aftergroup`

Restore `\if@newlist`

```
\global \let \if@newlist \@@if@newlist
\global \@colht \textheight
\stepcounter{page}%
```

Another hook for packages that want to append material to `\@outputpage`.

```
    \UseHook {build/page/after}%
}
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2017/04/15}%
⟨latexrelease⟩  {\@outputpage}{Reset language for hyphenation}%
⟨latexrelease⟩\def\@outputpage{%
⟨latexrelease⟩\begingroup
⟨latexrelease⟩  \let \protect \noexpand
⟨latexrelease⟩  \language\document@default@language
⟨latexrelease⟩  \@resetactivechars
⟨latexrelease⟩  \global\let\@@if@newlist\if@newlist
⟨latexrelease⟩  \global\@newlistfalse
⟨latexrelease⟩  \@parboxrestore
⟨latexrelease⟩  \shipout \vbox{%
⟨latexrelease⟩    \set@typeset@protect
⟨latexrelease⟩    \aftergroup \endgroup
⟨latexrelease⟩    \aftergroup \set@typeset@protect
⟨latexrelease⟩  \if@specialpage
⟨latexrelease⟩    \global\@specialpagefalse\@nameuse{ps@\@specialstyle}%
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \if@twoside
⟨latexrelease⟩    \ifodd\count\z@ \let\@thehead\@oddhead \let\@thefoot\@oddfoot
⟨latexrelease⟩         \let\@themargin\oddsidemargin
⟨latexrelease⟩    \else \let\@thehead\@evenhead
⟨latexrelease⟩       \let\@thefoot\@evenfoot \let\@themargin\evensidemargin
⟨latexrelease⟩    \fi
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \reset@font
⟨latexrelease⟩  \normalsize
⟨latexrelease⟩  \normalsfcodes
⟨latexrelease⟩  \let\label\@gobble
⟨latexrelease⟩  \let\index\@gobble
⟨latexrelease⟩  \let\glossary\@gobble
⟨latexrelease⟩  \baselineskip\z@skip \lineskip\z@skip \lineskiplimit\z@
⟨latexrelease⟩    \@begindvi
⟨latexrelease⟩    \vskip \topmargin
⟨latexrelease⟩    \moveright\@themargin \vbox {%
⟨latexrelease⟩      \setbox\@tempboxa \vbox to\headheight{%
⟨latexrelease⟩        \vfil
⟨latexrelease⟩        \color@hbox
⟨latexrelease⟩          \normalcolor
⟨latexrelease⟩          \hb@xt@\textwidth{\@thehead}%
⟨latexrelease⟩        \color@endbox
⟨latexrelease⟩        }%
⟨latexrelease⟩      \dp\@tempboxa \z@
⟨latexrelease⟩      \box\@tempboxa
⟨latexrelease⟩      \vskip \headsep
⟨latexrelease⟩      \box\@outputbox
⟨latexrelease⟩      \baselineskip \footskip
⟨latexrelease⟩      \color@hbox
⟨latexrelease⟩        \normalcolor
⟨latexrelease⟩        \hb@xt@\textwidth{\@thefoot}%
```

```
⟨latexrelease⟩       \color@endbox
⟨latexrelease⟩       }%
⟨latexrelease⟩     }%
⟨latexrelease⟩   \global\let\if@newlist\@@if@newlist
⟨latexrelease⟩   \global \@colht \textheight
⟨latexrelease⟩   \stepcounter{page}%
```

It is now clear that this does something useful (in the old mark mechanism), thanks to Pieter van Oostrum pointing this out. It is needed because a float page is made without using TeX's page-builder; thus the output routine is never called so the marks are not updated.

```
⟨latexrelease⟩   \let\firstmark\botmark
⟨latexrelease⟩}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease

⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩   {\@outputpage}{Reset language for hyphenation}%
⟨latexrelease⟩\def\@outputpage{%
⟨latexrelease⟩\begingroup
⟨latexrelease⟩   \let \protect \noexpand
⟨latexrelease⟩   \@resetactivechars
⟨latexrelease⟩   \global\let\@@if@newlist\if@newlist
⟨latexrelease⟩   \global\@newlistfalse
⟨latexrelease⟩   \@parboxrestore
⟨latexrelease⟩   \shipout \vbox{%
⟨latexrelease⟩     \set@typeset@protect
⟨latexrelease⟩     \aftergroup \endgroup
⟨latexrelease⟩     \aftergroup \set@typeset@protect
⟨latexrelease⟩   \if@specialpage
⟨latexrelease⟩     \global\@specialpagefalse\@nameuse{ps@\@specialstyle}%
⟨latexrelease⟩   \fi
⟨latexrelease⟩   \if@twoside
⟨latexrelease⟩     \ifodd\count\z@
⟨latexrelease⟩          \let\@thehead\@oddhead \let\@thefoot\@oddfoot
⟨latexrelease⟩          \let\@themargin\oddsidemargin
⟨latexrelease⟩     \else \let\@thehead\@evenhead
⟨latexrelease⟩        \let\@thefoot\@evenfoot \let\@themargin\evensidemargin
⟨latexrelease⟩     \fi
⟨latexrelease⟩   \fi
⟨latexrelease⟩   \reset@font
⟨latexrelease⟩   \normalsize
⟨latexrelease⟩   \normalsfcodes
⟨latexrelease⟩   \let\label\@gobble
⟨latexrelease⟩   \let\index\@gobble
⟨latexrelease⟩   \let\glossary\@gobble
⟨latexrelease⟩   \baselineskip\z@skip \lineskip\z@skip \lineskiplimit\z@
⟨latexrelease⟩     \@begindvi
⟨latexrelease⟩     \vskip \topmargin
⟨latexrelease⟩     \moveright\@themargin \vbox {%
⟨latexrelease⟩       \setbox\@tempboxa \vbox to\headheight{%
⟨latexrelease⟩         \vfil
⟨latexrelease⟩         \color@hbox
⟨latexrelease⟩           \normalcolor
⟨latexrelease⟩           \hb@xt@\textwidth{\@thehead}%
```

```
⟨latexrelease⟩          \color@endbox
⟨latexrelease⟩          }%
⟨latexrelease⟩        \dp\@tempboxa \z@
⟨latexrelease⟩        \box\@tempboxa
⟨latexrelease⟩        \vskip \headsep
⟨latexrelease⟩        \box\@outputbox
⟨latexrelease⟩        \baselineskip \footskip
⟨latexrelease⟩        \color@hbox
⟨latexrelease⟩          \normalcolor
⟨latexrelease⟩          \hb@xt@\textwidth{\@thefoot}%
⟨latexrelease⟩        \color@endbox
⟨latexrelease⟩        }%
⟨latexrelease⟩      }%
⟨latexrelease⟩    \global\let\if@newlist\@@if@newlist
⟨latexrelease⟩    \global \@colht \textheight
⟨latexrelease⟩    \stepcounter{page}%
⟨latexrelease⟩    \let\firstmark\botmark
⟨latexrelease⟩}
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@outputpage`, `\@shipoutsetup`, *and* `\@writesetup`.)

`\@begindvi` This unboxes stuff that must appear before anything else in the `.dvi` file, then returns that box register to the free list and cancels itself.

The stuff in the box should not add any typeset material to the page.

```
\def \@begindvi{%
  \unvbox \@begindvibox
  \global\let \@begindvi \@empty
}
```

(*End of definition for* `\@begindvi`.)

2.2.1 Dealing with floats

`\@combinefloats` Old name for what is now called `\@outputbox@attachfloats`; kept to support legacy packages.

```
\let \@combinefloats \@outputbox@attachfloats
```

(*End of definition for* `\@combinefloats`.)

`\@cflt` `\@cflb` The `\boxmaxdepth` setting here was not made local to a box so was dangerous. It is needed only within the box made by `\@cflt` (and not normally even there), so it has been moved there; this also agrees with the original pseudocode.

```
\def \@cflt{%
    \let \@elt \@comflelt
    \setbox\@tempboxa \vbox{}%
    \@toplist
    \setbox\@outputbox \vbox{%
                              \boxmaxdepth \maxdepth
                              \unvbox\@tempboxa
                              \vskip -\floatsep
                              \topfigrule
                              \vskip \textfloatsep
```

```
                                \unvbox\@outputbox
                                }%
    \let\@elt\relax
    \xdef\@freelist{\@freelist\@toplist}%
    \global\let\@toplist\@empty
}
\def \@cflb {%
    \let\@elt\@comflelt
    \setbox\@tempboxa \vbox{}%
    \@botlist
    \setbox\@outputbox \vbox{%
                                \unvbox\@outputbox
                                \vskip \textfloatsep
                                \botfigrule
                                \unvbox\@tempboxa
                                \vskip -\floatsep
                                }%
    \let\@elt\relax
    \xdef\@freelist{\@freelist\@botlist}%
    \global \let \@botlist\@empty
}
```

(*End of definition for* `\@cflt` *and* `\@cflb`.)

`\@comflelt`
`\@comdblflelt`
`\@combinedblfloats`

```
\def\@comflelt#1{\setbox\@tempboxa
      \vbox{\unvbox\@tempboxa\box #1\vskip\floatsep}}
\def\@comdblflelt#1{\setbox\@tempboxa
      \vbox{\unvbox\@tempboxa\box #1\vskip\dblfloatsep}}
\def \@combinedblfloats{%
  \ifx \@dbltoplist \@empty
  \else
    \setbox\@tempboxa \vbox{}%
    \let \@elt \@comdblflelt
    \@dbltoplist
    \let \@elt \relax
    \xdef \@freelist {\@freelist\@dbltoplist}%
    \global\let \@dbltoplist \@empty
    \setbox\@outputbox \vbox to\textheight
```

The setting of `\boxmaxdepth` here has no effect since the `\@outputbox` should already have depth zero. Even so, it would have no effect on the layout of the page.

```
      {%
       \unvbox\@tempboxa\vskip-\dblfloatsep
```

Here we need different typesetting if the top float comes from `\@topnewpage`.

```
       \ifnum \@dbltopnum>\m@ne
         \dblfigrule
       \fi
       \vskip \dbltextfloatsep
```

If pdf links are present in the galley and those links get broken across pages they have to end up being on the same level of boxing (even if not actually in the same structure) due to some engine restrictions in pdfTEX and LuaTEX. We therefore unbox `\@outputbox`

here (which only contains a single \hbox) so that this case has the same boxing level as a normal twocolumn page without top floats.

```
        \unvbox\@outputbox
        }%
  \fi
}
⟨/2ekernel⟩
```

(*End of definition for* `\@comflelt`, `\@comdblflelt`, *and* `\@combinedblfloats`.)

`\@startcolumn`
`\@startdblcolumn`

We could combine (most of) these two into `\@startcol <list>`. Note that `\@xstartcol` was only used once (i.e. in `\@startcolumn`); it has therefore been removed. This is not quite as efficient but it now has the same structure as `\@startdblcolumn`.

The empty-list test has been moved to `\@tryfcolumn`.

```
⟨*2ekernel | fltrace⟩
\def \@startcolumn {%
  \global \@colroom \@colht
  \@tryfcolumn \@deferlist
  \if@fcolmade
⟨*trace⟩
    \fl@trace{PAGE: float \if@twocolumn column \else page \fi
                completed}%
⟨/trace⟩
  \else

    \begingroup
      \let \reserved@b \@deferlist
      \global \let \@deferlist \@empty
      \let \@elt \@scolelt
      \reserved@b
    \endgroup
  \fi
}
```

This one does not need to set `\@colht`.

```
⟨/2ekernel | fltrace⟩
⟨latexrelease | fltrace⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease | fltrace⟩   {\@startdblcolumn}{float order in 2-column}%
⟨*2ekernel | latexrelease | fltrace⟩
\def \@startdblcolumn {%
  \@tryfcolumn \@deferlist
  \if@fcolmade
⟨fltrace⟩    \fl@trace{PAGE: double float page completed}%
  \else
    \begingroup
      \let \reserved@b \@deferlist
      \global \let \@deferlist \@empty
      \let \@elt \@sdblcolelt
      \reserved@b
    \endgroup
  \fi
}%
⟨/2ekernel | latexrelease | fltrace⟩
⟨latexrelease | fltrace⟩\EndIncludeInRelease
```

```
⟨latexrelease | fltrace⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease | fltrace⟩   {\@startdblcolumn}{float order in 2-column}%
⟨latexrelease | fltrace⟩\def \@startdblcolumn {%
```

Not needed since this always comes after `\@outputpage`:

```
⟨latexrelease | fltrace⟩% \global \@colht \textheight
⟨latexrelease | fltrace⟩   \@tryfcolumn \@dbldeferlist
⟨latexrelease | fltrace⟩   \if@fcolmade
⟨*trace⟩
⟨latexrelease | fltrace⟩     \fl@trace{PAGE: double float page completed}%
⟨/trace⟩
⟨latexrelease | fltrace⟩   \else

⟨latexrelease | fltrace⟩     \begingroup
⟨latexrelease | fltrace⟩       \let \reserved@b \@dbldeferlist
⟨latexrelease | fltrace⟩       \global \let \@dbldeferlist \@empty
⟨latexrelease | fltrace⟩       \let \@elt \@sdblcolelt
⟨latexrelease | fltrace⟩       \reserved@b
⟨latexrelease | fltrace⟩     \endgroup
⟨latexrelease | fltrace⟩   \fi
⟨latexrelease | fltrace⟩}%
⟨latexrelease | fltrace⟩\EndIncludeInRelease
⟨*2ekernel | fltrace⟩
```

(*End of definition for* `\@startcolumn` *and* `\@startdblcolumn`.)

`\@tryfcolumn` Now tests if its list is empty before any further exertion.

```
\def \@tryfcolumn #1{%
  \global \@fcolmadefalse
  \ifx #1\@empty
  \else
⟨*trace⟩
     \fl@trace{PAGE: try float \if@twocolumn column/page\else page\fi
                  ---\string #1}%
     \fl@trace{----- \string #1: #1}%
⟨/trace⟩

    \xdef\@trylist{#1}%
    \global \let \@failedlist \@empty
    \begingroup
      \let \@elt \@xtryfc \@trylist
    \endgroup
    \if@fcolmade
      \@vtryfc #1%
    \fi
  \fi
}
⟨/2ekernel | fltrace⟩
```

(*End of definition for* `\@tryfcolumn`.)

```
⟨*2ekernel⟩
```

`\@scolelt`

```
\def\@scolelt#1{\def\@currbox{#1}\@addtonextcol}
```

(*End of definition for* `\@scolelt`.)

`\@sdblcolelt`

```
\def\@sdblcolelt#1{\def\@currbox{#1}\@addtodblcol}
```

(*End of definition for* `\@sdblcolelt`.)

`\@vtryfc`

```
\def\@vtryfc #1{%
  \global\setbox\@outputbox\vbox{}%
  \let\@elt\@wtryfc
  \@flsucceed
  \global\setbox\@outputbox \vbox to\@colht{%
    \vskip \@fptop
    \vskip -\@fpsep
    \unvbox \@outputbox
    \vskip \@fpbot}%
  \let\@elt\relax
  \xdef #1{\@failedlist\@flfail}%
  \xdef\@freelist{\@freelist\@flsucceed}}
```

(*End of definition for* `\@vtryfc`.)

`\@wtryfc`

```
\def\@wtryfc #1{%
  \global\setbox\@outputbox\vbox{%
    \unvbox\@outputbox
    \vskip\@fpsep
    \box #1}}
```

(*End of definition for* `\@wtryfc`.)

`\@xtryfc`

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}{\@xtryfc}%
⟨latexrelease⟩                                    {float order in 2-column}%
⟨*2ekernel | latexrelease⟩
\def\@xtryfc #1{%
  \@next\reserved@a\@trylist{}{}%
  \@currtype \count #1%
  \divide\@currtype\@xxxii
  \multiply\@currtype\@xxxii
  \@bitor \@currtype \@failedlist
  \@testfp #1%
  \@testwrongwidth #1%
  \ifdim \ht #1>\@colht
     \@testtrue
  \fi
  \if@test
    \@cons\@failedlist #1%
  \else
    \@ytryfc #1%
  \fi}%
⟨/2ekernel | latexrelease⟩
```

```
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{\@xtryfc}%
⟨latexrelease⟩                                {float order in 2-column}%
⟨latexrelease⟩\def\@xtryfc #1{%
⟨latexrelease⟩  \@next\reserved@a\@trylist{}{}%
⟨latexrelease⟩  \@currtype \count #1%
⟨latexrelease⟩  \divide\@currtype\@xxxii
⟨latexrelease⟩  \multiply\@currtype\@xxxii
⟨latexrelease⟩  \@bitor \@currtype \@failedlist
⟨latexrelease⟩  \@testfp #1%
⟨latexrelease⟩  \ifdim \ht #1>\@colht
⟨latexrelease⟩    \@testtrue
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \if@test
⟨latexrelease⟩    \@cons\@failedlist #1%
⟨latexrelease⟩  \else
⟨latexrelease⟩    \@ytryfc #1%
⟨latexrelease⟩  \fi}%
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\@xtryfc`.)

`\@ytryfc`

```
\def\@ytryfc #1{%
  \begingroup
    \gdef\@flsucceed{\@elt #1}%
    \global\let\@flfail\@empty
    \@tempdima\ht #1%
    \let\@elt\@ztryfc
    \@trylist
    \ifdim \@tempdima >\@fpmin
      \global\@fcolmadetrue
    \else
      \@cons\@failedlist #1%
    \fi
  \endgroup
  \if@fcolmade
    \let\@elt\@gobble
  \fi}
```

(*End of definition for* `\@ytryfc`.)

`\@ztryfc`

```
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}{@ztryfc}%
⟨latexrelease⟩                                {float order in 2-column}%
⟨*2ekernel | latexrelease⟩
\def\@ztryfc #1{%
  \@tempcnta\count #1%
  \divide\@tempcnta\@xxxii
  \multiply\@tempcnta\@xxxii
  \@bitor \@tempcnta {\@failedlist \@flfail}%
  \@testfp #1%
```

not in fixfloats?

```
  \@testwrongwidth #1%
  \@tempdimb\@tempdima
  \advance\@tempdimb\ht #1%
  \advance\@tempdimb\@fpsep
  \ifdim \@tempdimb >\@colht
    \@testtrue
  \fi
  \if@test
    \@cons\@flfail #1%
  \else
    \@cons\@flsucceed #1%
    \@tempdima\@tempdimb
  \fi}%
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{@ztryfc}%
⟨latexrelease⟩                                {float order in 2-column}%
⟨latexrelease⟩\def\@ztryfc #1{%
⟨latexrelease⟩  \@tempcnta \count#1%
⟨latexrelease⟩  \divide\@tempcnta\@xxxii
⟨latexrelease⟩  \multiply\@tempcnta\@xxxii
⟨latexrelease⟩  \@bitor \@tempcnta {\@failedlist \@flfail}%
⟨latexrelease⟩  \@testfp #1%
⟨latexrelease⟩  \@tempdimb\@tempdima
⟨latexrelease⟩  \advance\@tempdimb \ht#1%
⟨latexrelease⟩  \advance\@tempdimb\@fpsep
⟨latexrelease⟩  \ifdim \@tempdimb >\@colht
⟨latexrelease⟩    \@testtrue
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \if@test
⟨latexrelease⟩    \@cons\@flfail #1%
⟨latexrelease⟩  \else
⟨latexrelease⟩    \@cons\@flsucceed #1%
⟨latexrelease⟩    \@tempdima\@tempdimb
⟨latexrelease⟩  \fi}%
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\@ztryfc`.)

The major changes for float suppression and the changes to the float mechanism to make it conform to the documentation are in these next macros.

`\@addtobot` Lots of changes.

```
⟨*2ekernel | fltrace⟩
\def \@addtobot {%
⟨*trace⟩
   \fl@trace{***Start addtobot}%
⟨/trace⟩
   \@getfpsbit 4\relax
⟨*trace⟩
   \fl@trace{fpstype \ifodd \@tempcnta OK \else not \fi bot:
                                                    \the \@fpstype}%
⟨/trace⟩
```

```
   \ifodd \@tempcnta
     \@flsetnum \@botnum
     \ifnum \@botnum>\z@
       \@tempswafalse
       \@flcheckspace \@botroom \@botlist
       \if@tempswa

%          \global \maxdepth \z@
         \@flupdates \@botnum \@botroom \@botlist
⟨*trace⟩
         \fl@trace{colroom (after-bot) = \the \@colroom}%
         \fl@trace{colnum (after-bot) = \the \@colnum}%
         \fl@trace{botnum (after-bot) = \the \@botnum}%
         \fl@trace{***Success: bot}%
⟨/trace⟩
         \@inserttrue
       \fi
⟨*trace⟩
     \else
       \fl@trace{Fail: botnum = \the \@botnum:
                                  fpstype \the \@fpstype=ORD?}%
       \ifnum \@fpstype<\sixt@@n
         \fl@trace{ERROR: !b float not successful (addtobot)}%
       \fi
⟨/trace⟩
     \fi
   \fi
}
```

(*End of definition for* `\@addtobot`.)

`\@addtotoporbot` Lots of changes.

```
\def \@addtotoporbot {%
⟨*trace⟩
   \fl@trace{***Start addtotoporbot}%
⟨/trace⟩
   \@getfpsbit \tw@
⟨*trace⟩
   \fl@trace{fpstype \ifodd \@tempcnta OK \else not \fi top:
                                                    \the \@fpstype}%
⟨/trace⟩
   \ifodd \@tempcnta
     \@flsetnum \@topnum
     \ifnum \@topnum>\z@
       \@tempswafalse
       \@flcheckspace \@toproom \@toplist
       \if@tempswa
         \@bitor\@currtype{\@midlist\@botlist}%
⟨*trace⟩
           \fl@trace{(mid+bot)list: \@midlist, \@botlist:
                              (addtotoporbot-before)}%
⟨/trace⟩
         \if@test
⟨*trace⟩
```

```
          \fl@trace{type already on list: mid or bot---sent to addtobot}%
⟨/trace⟩
          \else
           \@flupdates \@topnum \@toproom \@toplist
⟨*trace⟩
           \fl@trace{colroom (after-top) = \the \@colroom}%
           \fl@trace{colnum (after-top) = \the \@colnum}%
           \fl@trace{topnum (after-top) = \the \@topnum}%
           \fl@trace{***Success: top}%
⟨/trace⟩
           \@inserttrue
          \fi
        \fi
⟨*trace⟩
      \else
        \fl@trace{Fail: topnum = \the \@topnum: fpstype
                                                \the \@fpstype=ORD?}%
        \ifnum \@fpstype<\sixt@@n
          \fl@trace{ERROR: !t float not successful (addtotoporbot)}%
        \fi
⟨/trace⟩
      \fi
   \fi
   \if@insert
   \else
⟨*trace⟩
      \fl@trace{sent to addtobot (addtotoporbot)}%
⟨/trace⟩
      \@addtobot
   \fi
}
⟨/2ekernel | fltrace⟩
```

(*End of definition for* `\@addtotoporbot`.)

`\@addtocurcol` Lots of changes.

```
⟨latexrelease | fltrace | flafter⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease | fltrace | flafter⟩  {\@addtocurcol}{float order in 2-column}%
⟨*2ekernel | latexrelease | fltrace | flafter⟩
\def \@addtocurcol {%
⟨*trace⟩
  \fl@trace{***Start addtocurcol}%
⟨/trace⟩
   \@insertfalse
   \@setfloattypecounts
   \ifnum \@fpstype=8
⟨*trace⟩
     \fl@trace{fpstype !p only (addtocurcol): \the \@fpstype = 8?}%
⟨/trace⟩
   \else
     \ifnum \@fpstype=24
⟨*trace⟩
       \fl@trace{fpstype p only (addtocurcol): \the \@fpstype = 24?}%
⟨/trace⟩
```

```
      \else
        \@flsettextmin
```

This is a new adjustment which is quite a major change in functionality; but it implements the documentation. Note that `\@reqcolroom` will include the whole of the page-so-far, and hence includes `\@textfloatsheight` of floats, so before comparing it with `\@textmin`, we add this to `\@textmin` also.

```
⟨*trace⟩
        \fl@trace{textfloatsheight (before) = \the \@textfloatsheight}%
⟨/trace⟩
        \advance \@textmin \@textfloatsheight
        \@reqcolroom \@pageht
```

This line must be removed since `\@specialoutput` changed.

```
%       \advance \@reqcolroom \@pagedp
⟨*trace⟩
        \fl@trace{textmin + textfloatsheight: \the \@textmin}%
        \fl@trace{page-so-far: \the \@reqcolroom}%
⟨/trace⟩
        \ifdim \@textmin>\@reqcolroom
          \@reqcolroom \@textmin
⟨*trace⟩
          \fl@trace{ORD? textmin being used}%
⟨/trace⟩
        \fi
        \advance \@reqcolroom \ht\@currbox
```

We save the current value of `\@reqcolroom` so that we can return to this value later, in case a test fails that may have added something to it.

```
        \saved@reqcolroom \@reqcolroom
⟨*trace⟩
        \fl@trace{float size = \the \ht \@currbox (addtocurcol)}%
        \fl@trace{colroom = \the \@colroom (addtocurcol)}%
        \fl@trace{reqcolroom = \the \@reqcolroom (addtocurcol)}%
⟨/trace⟩
        \ifdim \@colroom>\@reqcolroom
          \@flsetnum \@colnum
          \ifnum \@colnum>\z@
            \@bitor\@currtype\@deferlist
```

We need to defer the float also if its width doesn't fit.

```
           \@testwrongwidth\@currbox
⟨*trace⟩
            \fl@trace{deferlist: \@deferlist: (addtocurcol-before)}%
⟨/trace⟩
            \if@test
⟨*trace⟩
              \fl@trace{type already on list: defer (addtocurcol)}%
⟨/trace⟩
            \else
              \@bitor\@currtype\@botlist
⟨*trace⟩
            \fl@trace{botlist: \@botlist: (addtocurcol-before)}%
⟨/trace⟩
              \if@test
```

```
⟨*trace⟩
                \fl@trace{type already on list: bot---sent to addtobot}%
⟨/trace⟩
                \@addtobot
              \else
⟨*trace⟩
                \fl@trace{fpstype \ifodd \@tempcnta OK \else not \fi
                       here: \the \@fpstype}%
⟨/trace⟩
                \ifodd \count\@currbox
                  \advance \@reqcolroom \intextsep
                  \ifdim \@colroom>\@reqcolroom
                    \global \advance \@colnum \m@ne
                    \global \advance \@textfloatsheight \ht\@currbox
```

This may sometimes give an overestimate.

```
                    \global \advance \@textfloatsheight 2\intextsep
                    \@cons \@midlist \@currbox
⟨*trace⟩
                  \fl@trace{***Success: here}%
                  \fl@trace{textfloatsheight (after-here) =
                       \the \@textfloatsheight}%
                  \fl@trace{colnum (after-here) = \the \@colnum}%
⟨/trace⟩
```

CHANGE TO `\@addtocurcol`:

`\penalty\z@` changed to `\penalty\interlinepenalty` so `\samepage` works properly with figure and table environments. (Changed 23 Oct 86)

There is also an `\addpenalty\interlinepenalty` above.

Although it is best to use `\addvspace` in case two h floats come together, this makes other spacing more difficult to adjust; whereas if a user specifies two h floats together then they can more easily get the spacing correct by ad hoc commands.

It is necessary to adjust for the addition of `\parskip` here in case the float is added between paragraphs (i.e. when in vertical mode).

If the nobreak switch is true we need to reset it and clear `\everypar` since the float may not reset the flag and cannot reset the `\everypar` globally.

Typesetting starts here (we are in vertical mode).

```
                  \if@nobreak
                    \nobreak
                    \@nobreakfalse
                    \everypar{}%
                  \else
                    \addpenalty \interlinepenalty
                  \fi
                  \vskip \intextsep
                  \box\@currbox
                  \penalty\interlinepenalty
                  \vskip\intextsep
                  \ifnum\outputpenalty <-\@Mii \vskip -\parskip\fi
```

Typesetting ends here.

```
                  \outputpenalty \z@
                  \@inserttrue
⟨*trace⟩
```

```
                  \else
                    \fl@trace{Fail---no room at 2nd test of colroom
                                  (addtocorcol \string\intextsep)}%
⟨/trace⟩
                    \fi
                  \fi
                  \if@insert
                  \else
```

Next set of docstrip guards are a bit weird, essentially `\@addtotoporbot` ends up inside the kernel and the `fltrace` package and `\@addtobot` shows up in the `flafter` package. Guess that could have been done a bit more obvious :-)

```
⟨*2ekernel | fltrace | latexrelease⟩
⟨*trace⟩
                    \fl@trace{not here: sent to addtotoporbot}%
⟨/trace⟩
                    \@addtotoporbot
⟨/2ekernel | fltrace | latexrelease⟩
⟨*!2ekernel&!fltrace&!latexrelease⟩
⟨*trace⟩
                    \fl@trace{not here: sent to addtobot}%
⟨/trace⟩
                    \@addtobot
⟨/!2ekernel&!fltrace&!latexrelease⟩
                  \fi
                \fi
              \fi
⟨*trace⟩
            \else
              \fl@trace{Fail: colnum = \the \@colnum:
                          fpstype \the \@fpstype=ORD?}%
              \ifnum \@fpstype<\sixt@@n
                \fl@trace{ERROR: BANG float not successful (addtocurcol)}%
              \fi
⟨/trace⟩
            \fi
⟨*trace⟩
          \else
            \fl@trace{Fail---no room: fl box ht: \the \ht \@currbox
                                                    (addtocurcol)}%
⟨/trace⟩
          \fi
        \fi
      \fi
      \if@insert
      \else
        \@resethfps
⟨*trace⟩
        \fl@trace{put on deferlist (addtocurcol)}%
⟨/trace⟩
        \@cons\@deferlist\@currbox
⟨*trace⟩
        \fl@trace{deferlist: \@deferlist: (addtocurcol-after)}%
⟨/trace⟩
```

```
    \fi
}%
⟨/2ekernel | latexrelease | fltrace | flafter⟩
⟨latexrelease | fltrace | flafter⟩\EndIncludeInRelease
```

The rollback code is rather complicated as it has to account for the fact that we roll back not just the kernel but also the packages flafter and fltrace.

```
⟨latexrelease | fltrace | flafter⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease | fltrace | flafter⟩   {\@addtocurcol}{float order in 2-column}%
⟨latexrelease | fltrace | flafter⟩\def \@addtocurcol {%
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩   \fl@trace{***Start addtocurcol}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩    \@insertfalse
⟨latexrelease | fltrace | flafter⟩    \@setfloattypecounts
⟨latexrelease | fltrace | flafter⟩    \ifnum \@fpstype=8
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩      \fl@trace{fpstype !p only (addtocurcol): \the \@fpstype = 8?}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩    \else
⟨latexrelease | fltrace | flafter⟩      \ifnum \@fpstype=24
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩        \fl@trace{fpstype p only (addtocurcol): \the \@fpstype = 24?}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩      \else
⟨latexrelease | fltrace | flafter⟩        \@flsettextmin
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩        \fl@trace{textfloatsheight (before) = \the \@textfloatsheight}
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩        \advance \@textmin \@textfloatsheight
⟨latexrelease | fltrace | flafter⟩        \@reqcolroom \@pageht
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩        \fl@trace{textmin + textfloatsheight: \the \@textmin}%
⟨latexrelease | fltrace | flafter⟩        \fl@trace{page-so-far: \the \@reqcolroom}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩        \ifdim \@textmin>\@reqcolroom
⟨latexrelease | fltrace | flafter⟩          \@reqcolroom \@textmin
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩          \fl@trace{ORD? textmin being used}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩        \fi
⟨latexrelease | fltrace | flafter⟩        \advance \@reqcolroom \ht\@currbox
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩        \fl@trace{float size = \the \ht \@currbox (addtocurcol)}%
⟨latexrelease | fltrace | flafter⟩        \fl@trace{colroom = \the \@colroom (addtocurcol)}%
⟨latexrelease | fltrace | flafter⟩        \fl@trace{reqcolroom = \the \@reqcolroom (addtocurcol)}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩        \ifdim \@colroom>\@reqcolroom
⟨latexrelease | fltrace | flafter⟩          \@flsetnum \@colnum
⟨latexrelease | fltrace | flafter⟩          \ifnum \@colnum>\z@
⟨latexrelease | fltrace | flafter⟩            \@bitor\@currtype\@deferlist
⟨latexrelease | fltrace | flafter⟩           \@testwrongwidth\@currbox
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩             \fl@trace{deferlist: \@deferlist: (addtocurcol-before)}%
```

```
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩            \if@test
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩              \fl@trace{type already on list: defer (addtocurcol)}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩            \else
⟨latexrelease | fltrace | flafter⟩              \@bitor\@currtype\@botlist
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩            \fl@trace{botlist: \@botlist: (addtocurcol-before)}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩              \if@test
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩                \fl@trace{type already on list: bot---sent to addtobot
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩                \@addtobot
⟨latexrelease | fltrace | flafter⟩              \else
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩                \fl@trace{fpstype \ifodd \@tempcnta OK \else not \fi
⟨latexrelease | fltrace | flafter⟩                       here: \the \@fpstype}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩                \ifodd \count\@currbox
⟨latexrelease | fltrace | flafter⟩                  \advance \@reqcolroom \intextsep
⟨latexrelease | fltrace | flafter⟩                  \ifdim \@colroom>\@reqcolroom
⟨latexrelease | fltrace | flafter⟩                    \global \advance \@colnum \m@ne
⟨latexrelease | fltrace | flafter⟩                    \global \advance \@textfloatsheight \ht\@currbox
⟨latexrelease | fltrace | flafter⟩                    \global \advance \@textfloatsheight 2\intextsep
⟨latexrelease | fltrace | flafter⟩                    \@cons \@midlist \@currbox
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩                  \fl@trace{***Success: here}%
⟨latexrelease | fltrace | flafter⟩                  \fl@trace{textfloatsheight (after-here) =
⟨latexrelease | fltrace | flafter⟩                       \the \@textfloatsheight}%
⟨latexrelease | fltrace | flafter⟩                  \fl@trace{colnum (after-here) = \the \@colnum}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩                    \if@nobreak
⟨latexrelease | fltrace | flafter⟩                      \nobreak
⟨latexrelease | fltrace | flafter⟩                      \@nobreakfalse
⟨latexrelease | fltrace | flafter⟩                      \everypar{}%
⟨latexrelease | fltrace | flafter⟩                    \else
⟨latexrelease | fltrace | flafter⟩                      \addpenalty \interlinepenalty
⟨latexrelease | fltrace | flafter⟩                    \fi
⟨latexrelease | fltrace | flafter⟩                    \vskip \intextsep
⟨latexrelease | fltrace | flafter⟩                    \box\@currbox
⟨latexrelease | fltrace | flafter⟩                    \penalty\interlinepenalty
⟨latexrelease | fltrace | flafter⟩                    \vskip\intextsep
⟨latexrelease | fltrace | flafter⟩                    \ifnum\outputpenalty <-\@Mii \vskip -\parskip\fi
⟨latexrelease | fltrace | flafter⟩                    \outputpenalty \z@
⟨latexrelease | fltrace | flafter⟩                    \@inserttrue
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩                  \else
⟨latexrelease | fltrace | flafter⟩                    \fl@trace{Fail---no room at 2nd test of colroom
⟨latexrelease | fltrace | flafter⟩                                 (addtocorcol \string\intextsep)}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩                  \fi
⟨latexrelease | fltrace | flafter⟩                \fi
```

```
⟨latexrelease | fltrace | flafter⟩                \if@insert
⟨latexrelease | fltrace | flafter⟩                \else
⟨*trace⟩
⟨fltrace | latexrelease⟩                    \fl@trace{not here: sent to addtotoporbot}%
⟨/trace⟩
⟨fltrace | latexrelease⟩                    \@addtotoporbot
⟨*trace⟩
⟨flafter⟩                       \fl@trace{not here: sent to addtobot}%
⟨/trace⟩
⟨flafter⟩                       \@addtobot
⟨latexrelease | fltrace | flafter⟩               \fi
⟨latexrelease | fltrace | flafter⟩             \fi
⟨latexrelease | fltrace | flafter⟩           \fi
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩         \else
⟨latexrelease | fltrace | flafter⟩           \fl@trace{Fail: colnum = \the \@colnum:
⟨latexrelease | fltrace | flafter⟩                        fpstype \the \@fpstype=ORD?}%
⟨latexrelease | fltrace | flafter⟩           \ifnum \@fpstype<\sixt@@n
⟨latexrelease | fltrace | flafter⟩             \fl@trace{ERROR: BANG float not successful (addtocurcol)
⟨latexrelease | fltrace | flafter⟩           \fi
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩         \fi
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩       \else
⟨latexrelease | fltrace | flafter⟩         \fl@trace{Fail---no room: fl box ht: \the \ht \@currbox
⟨latexrelease | fltrace | flafter⟩                                                 (addtocurcol)}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩       \fi
⟨latexrelease | fltrace | flafter⟩     \fi
⟨latexrelease | fltrace | flafter⟩   \fi
⟨latexrelease | fltrace | flafter⟩   \if@insert
⟨latexrelease | fltrace | flafter⟩   \else
⟨latexrelease | fltrace | flafter⟩     \@resethfps
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩     \fl@trace{put on deferlist (addtocurcol)}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩     \@cons\@deferlist\@currbox
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩     \fl@trace{deferlist: \@deferlist: (addtocurcol-after)}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩   \fi
⟨latexrelease | fltrace | flafter⟩}%
⟨latexrelease | fltrace | flafter⟩\EndIncludeInRelease
⟨latexrelease | fltrace | flafter⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease | fltrace | flafter⟩  {\@addtocurcol}{float order in 2-column}%
⟨latexrelease | fltrace | flafter⟩\def \@addtocurcol {%
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩  \fl@trace{***Start addtocurcol}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩   \@insertfalse
⟨latexrelease | fltrace | flafter⟩   \@setfloattypecounts
⟨latexrelease | fltrace | flafter⟩   \ifnum \@fpstype=8
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩     \fl@trace{fpstype !p only (addtocurcol):
```

```
⟨latexrelease | fltrace | flafter⟩                              \the \@fpstype = 8?}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩    \else
⟨latexrelease | fltrace | flafter⟩      \ifnum \@fpstype=24
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩       \fl@trace{fpstype p only (addtocurcol):
⟨latexrelease | fltrace | flafter⟩                              \the \@fpstype = 24?}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩      \else
⟨latexrelease | fltrace | flafter⟩        \@flsettextmin
```

This is a new adjustment which is quite a major change in functionality; but it implements the documentation. Note that `\@reqcolroom` will include the whole of the page-so-far, and hence includes `\@textfloatsheight` of floats, so before comparing it with `\@textmin`, we add this to `\@textmin` also.

```
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩        \fl@trace{textfloatsheight (before) =
⟨latexrelease | fltrace | flafter⟩                          \the \@textfloatsheight}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩        \advance \@textmin \@textfloatsheight
⟨latexrelease | fltrace | flafter⟩        \@reqcolroom \@pageht
```

This line must be removed since `\@specialoutput` changed.

```
%       \advance \@reqcolroom \@pagedp
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩        \fl@trace{textmin + textfloatsheight:
⟨latexrelease | fltrace | flafter⟩                                    \the \@textmin}%
⟨latexrelease | fltrace | flafter⟩        \fl@trace{page-so-far: \the \@reqcolroom}%
⟨latexrelease | fltrace | flafter⟩
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩        \ifdim \@textmin>\@reqcolroom
⟨latexrelease | fltrace | flafter⟩          \@reqcolroom \@textmin
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩          \fl@trace{ORD? textmin being used}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩        \fi
⟨latexrelease | fltrace | flafter⟩        \advance \@reqcolroom \ht\@currbox
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩        \fl@trace{float size =
⟨latexrelease | fltrace | flafter⟩                \the \ht \@currbox (addtocurcol)}%
⟨latexrelease | fltrace | flafter⟩        \fl@trace{colroom =
⟨latexrelease | fltrace | flafter⟩                    \the \@colroom (addtocurcol)}%
⟨latexrelease | fltrace | flafter⟩        \fl@trace{reqcolroom =
⟨latexrelease | fltrace | flafter⟩                 \the \@reqcolroom (addtocurcol)}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩        \ifdim \@colroom>\@reqcolroom
⟨latexrelease | fltrace | flafter⟩          \@flsetnum \@colnum
⟨latexrelease | fltrace | flafter⟩          \ifnum \@colnum>\z@
⟨latexrelease | fltrace | flafter⟩            \@bitor\@currtype\@deferlist
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩            \fl@trace{deferlist:
⟨latexrelease | fltrace | flafter⟩               \@deferlist: (addtocurcol-before)}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩            \if@test
```

```
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩          \fl@trace{type already on list:
⟨latexrelease | fltrace | flafter⟩                        defer (addtocurcol)}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩        \else
⟨latexrelease | fltrace | flafter⟩          \@bitor\@currtype\@botlist
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩        \fl@trace{botlist: \@botlist:
⟨latexrelease | fltrace | flafter⟩                       (addtocurcol-before)}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩          \if@test
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩            \fl@trace{type already on list:
⟨latexrelease | fltrace | flafter⟩                      bot---sent to addtobot}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩            \@addtobot
⟨latexrelease | fltrace | flafter⟩          \else
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩            \fl@trace{fpstype
⟨latexrelease | fltrace | flafter⟩            \ifodd \@tempcnta OK \else not \fi
⟨latexrelease | fltrace | flafter⟩            here: \the \@fpstype}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩            \ifodd \count\@currbox
⟨latexrelease | fltrace | flafter⟩              \advance \@reqcolroom \intextsep
⟨latexrelease | fltrace | flafter⟩              \ifdim \@colroom>\@reqcolroom
⟨latexrelease | fltrace | flafter⟩               \global \advance \@colnum \m@ne
⟨latexrelease | fltrace | flafter⟩               \global \advance
⟨latexrelease | fltrace | flafter⟩                \@textfloatsheight\ht\@currbox
```

This may sometimes give an overestimate.

```
⟨latexrelease | fltrace | flafter⟩               \global \advance
⟨latexrelease | fltrace | flafter⟩                \@textfloatsheight 2\intextsep
⟨latexrelease | fltrace | flafter⟩                \@cons \@midlist \@currbox
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩              \fl@trace{***Success: here}%
⟨latexrelease | fltrace | flafter⟩              \fl@trace{textfloatsheight
⟨latexrelease | fltrace | flafter⟩                   (after-here) =
⟨latexrelease | fltrace | flafter⟩                   \the \@textfloatsheight}%
⟨latexrelease | fltrace | flafter⟩              \fl@trace{colnum (after-here) =
⟨latexrelease | fltrace | flafter⟩                        \the \@colnum}%
⟨/trace⟩
```

CHANGE TO `\@addtocurcol`:

`\penalty\z@` changed to `\penalty\interlinepenalty` so `\samepage` works properly with figure and table environments. (Changed 23 Oct 86)

There is also an `\addpenalty\interlinepenalty` above.

Although it is best to use `\addvspace` in case two h floats come together, this makes other spacing more difficult to adjust; whereas if a user specifies two h floats together then they can more easily get the spacing correct by ad hoc commands.

It is necessary to adjust for the addition of `\parskip` here in case the float is added between paragraphs (i.e. when in vertical mode).

If the nobreak switch is true we need to reset it and clear `\everypar` since the float may not reset the flag and cannot reset the `\everypar` globally.

Typesetting starts here (we are in vertical mode).

```
⟨latexrelease | fltrace | flafter⟩                    \if@nobreak
⟨latexrelease | fltrace | flafter⟩                      \nobreak
⟨latexrelease | fltrace | flafter⟩                      \@nobreakfalse
⟨latexrelease | fltrace | flafter⟩                      \everypar{}%
⟨latexrelease | fltrace | flafter⟩                    \else
⟨latexrelease | fltrace | flafter⟩                      \addpenalty\interlinepenalty
⟨latexrelease | fltrace | flafter⟩                    \fi
⟨latexrelease | fltrace | flafter⟩                    \vskip \intextsep
⟨latexrelease | fltrace | flafter⟩                    \box\@currbox
⟨latexrelease | fltrace | flafter⟩                    \penalty\interlinepenalty
⟨latexrelease | fltrace | flafter⟩                    \vskip\intextsep
⟨latexrelease | fltrace | flafter⟩                    \ifnum\outputpenalty
⟨latexrelease | fltrace | flafter⟩                                <-\@Mii \vskip
⟨latexrelease | fltrace | flafter⟩                         -\parskip\fi
```

Typesetting ends here.

```
⟨latexrelease | fltrace | flafter⟩                    \outputpenalty \z@
⟨latexrelease | fltrace | flafter⟩                    \@inserttrue
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩                  \else
⟨latexrelease | fltrace | flafter⟩  \fl@trace{Fail---no room at 2nd test of colroom
⟨latexrelease | fltrace | flafter⟩                (addtocorcol \string\intextsep)}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩                  \fi
⟨latexrelease | fltrace | flafter⟩                \fi
⟨latexrelease | fltrace | flafter⟩                \if@insert
⟨latexrelease | fltrace | flafter⟩                \else
```

Next set of docstrip guards are a bit weird, essentially `\@addtotoporbot` ends up inside the kernel and the `fltrace` package and `\@addtotoporbot` shows up in the `flafter` package. Guess that could have been done a bit more obvious :-)

```
⟨*2ekernel | fltrace⟩
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩      \fl@trace{not here: sent to addtotoporbot}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩                  \@addtotoporbot
⟨/2ekernel | fltrace⟩
⟨*!2ekernel&!autoload&!fltrace⟩
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩       \fl@trace{not here: sent to addtobot}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩                  \@addtobot
⟨/!2ekernel&!autoload&!fltrace⟩
⟨latexrelease | fltrace | flafter⟩                \fi
⟨latexrelease | fltrace | flafter⟩              \fi
⟨latexrelease | fltrace | flafter⟩            \fi
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩          \else
⟨latexrelease | fltrace | flafter⟩          \fl@trace{Fail: colnum = \the \@colnum:
⟨latexrelease | fltrace | flafter⟩                    fpstype \the \@fpstype=ORD?}%
⟨latexrelease | fltrace | flafter⟩          \ifnum \@fpstype<\sixt@@n
⟨latexrelease | fltrace | flafter⟩  \fl@trace{ERROR: BANG float not successful
⟨latexrelease | fltrace | flafter⟩                                  (addtocurcol)}%
⟨latexrelease | fltrace | flafter⟩            \fi
⟨/trace⟩
```

```
⟨latexrelease | fltrace | flafter⟩          \fi
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩        \else
⟨latexrelease | fltrace | flafter⟩          \fl@trace{Fail---no room: fl box ht:
⟨latexrelease | fltrace | flafter⟩                \the \ht \@currbox (addtocurcol)}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩        \fi
⟨latexrelease | fltrace | flafter⟩      \fi
⟨latexrelease | fltrace | flafter⟩    \fi
⟨latexrelease | fltrace | flafter⟩    \if@insert
⟨latexrelease | fltrace | flafter⟩    \else
⟨latexrelease | fltrace | flafter⟩      \@resethfps
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩      \fl@trace{put on deferlist (addtocurcol)}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩      \@cons\@deferlist\@currbox
⟨*trace⟩
⟨latexrelease | fltrace | flafter⟩      \fl@trace{deferlist: \@deferlist:
⟨latexrelease | fltrace | flafter⟩                           (addtocurcol-after)}%
⟨/trace⟩
⟨latexrelease | fltrace | flafter⟩    \fi
⟨latexrelease | fltrace | flafter⟩  }%
⟨latexrelease | fltrace | flafter⟩\EndIncludeInRelease
```

(*End of definition for* `\@addtocurcol`.)

`\@addtonextcol` Lots of changes.

```
⟨latexrelease | fltrace⟩\IncludeInRelease{2025/06/01}
⟨latexrelease | fltrace⟩  {\@addtonextcol}{float order in 2-column}%
⟨*2ekernel | latexrelease | fltrace⟩
\def\@addtonextcol{%
  \begingroup
⟨*trace⟩
   \fl@trace{***Start addtonextcol}%
⟨/trace⟩
   \@insertfalse
   \@setfloattypecounts
   \ifnum \@fpstype=8
⟨*trace⟩
     \fl@trace{fpstype not curcol: \the \@fpstype = 8?}%
⟨/trace⟩
   \else
     \ifnum \@fpstype=24
⟨*trace⟩
       \fl@trace{fpstype not curcol: \the \@fpstype = 24?}%
⟨/trace⟩
     \else
       \@flsettextmin
⟨*trace⟩
       \fl@trace{text-so-far: 0pt (top of col)}%
⟨/trace⟩
       \@reqcolroom \ht\@currbox
⟨*trace⟩
       \fl@trace{float size: \the \@reqcolroom (addtonextcol)}%
```

```
⟨/trace⟩
       \advance \@reqcolroom \@textmin
       \saved@reqcolroom \@reqcolroom
⟨*trace⟩
       \fl@trace{colroom = \the \@colroom (addtonextcol)}%
       \fl@trace{reqcolroom = \the \@reqcolroom (addtonextcol)}%
⟨/trace⟩
       \ifdim \@colroom>\@reqcolroom
         \@flsetnum \@colnum
         \ifnum\@colnum>\z@
            \@bitor\@currtype\@deferlist
⟨*trace⟩
            \fl@trace{deferlist: \@deferlist: (addtonextcol-before)}%
⟨/trace⟩
            \@testwrongwidth\@currbox
            \if@test
⟨*trace⟩
              \fl@trace{type already on list: defer (addtonextcol)}%
⟨/trace⟩
            \else
⟨*trace⟩
              \fl@trace{sent to addtotoporbot (addtonextcol)}%
⟨/trace⟩
              \@addtotoporbot
            \fi
         \fi
⟨*trace⟩
       \else
         \fl@trace{Fail---no room: fl box ht: \the \ht \@currbox
                                                  (addtonextcol)}%
⟨/trace⟩
       \fi
     \fi
   \fi
   \if@insert
   \else
⟨*trace⟩
     \fl@trace{put back on deferlist (addtonextcol)}%
⟨/trace⟩
     \@cons\@deferlist\@currbox
⟨*trace⟩
     \fl@trace{deferlist: \@deferlist: (addtonextcol-after)}%
⟨/trace⟩
   \fi
⟨*trace⟩
  \fl@trace{End of addtonextcol -- locally counts:}%
  \fl@trace{col: \the\@colnum. top: \the \@topnum. bot: \the \@botnum.}%
⟨/trace⟩
  \endgroup
⟨*trace⟩
 \fl@trace{End of addtonextcol -- globally counts:}%
 \fl@trace{col: \the\@colnum. top: \the \@topnum. bot: \the \@botnum.}%
⟨/trace⟩
```

```
}%
⟨/2ekernel | latexrelease | fltrace⟩
⟨latexrelease | fltrace⟩\EndIncludeInRelease
⟨latexrelease | fltrace⟩\IncludeInRelease{2015/01/01}
⟨latexrelease | fltrace⟩  {\@addtonextcol}{float order in 2-column}%
⟨latexrelease | fltrace⟩\def\@addtonextcol{%
⟨latexrelease | fltrace⟩  \begingroup
⟨*trace⟩
⟨latexrelease | fltrace⟩   \fl@trace{***Start addtonextcol}%
⟨/trace⟩
⟨latexrelease | fltrace⟩   \@insertfalse
⟨latexrelease | fltrace⟩   \@setfloattypecounts
⟨latexrelease | fltrace⟩   \ifnum \@fpstype=8
⟨*trace⟩
⟨latexrelease | fltrace⟩     \fl@trace{fpstype not curcol: \the \@fpstype = 8?}%
⟨/trace⟩
⟨latexrelease | fltrace⟩   \else
⟨latexrelease | fltrace⟩     \ifnum \@fpstype=24
⟨*trace⟩
⟨latexrelease | fltrace⟩       \fl@trace{fpstype not curcol: \the \@fpstype = 24?}%
⟨/trace⟩
⟨latexrelease | fltrace⟩     \else
⟨latexrelease | fltrace⟩       \@flsettextmin
⟨*trace⟩
⟨latexrelease | fltrace⟩       \fl@trace{text-so-far: 0pt (top of col)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩       \@reqcolroom \ht\@currbox
⟨*trace⟩
⟨latexrelease | fltrace⟩       \fl@trace{float size: \the \@reqcolroom (addtonextcol)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩       \advance \@reqcolroom \@textmin
⟨*trace⟩
⟨latexrelease | fltrace⟩       \fl@trace{colroom = \the \@colroom (addtonextcol)}%
⟨latexrelease | fltrace⟩       \fl@trace{reqcolroom = \the \@reqcolroom (addtonextcol)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩       \ifdim \@colroom>\@reqcolroom
⟨latexrelease | fltrace⟩         \@flsetnum \@colnum
⟨latexrelease | fltrace⟩         \ifnum\@colnum>\z@
⟨latexrelease | fltrace⟩            \@bitor\@currtype\@deferlist
⟨*trace⟩
⟨latexrelease | fltrace⟩            \fl@trace{deferlist: \@deferlist: (addtonextcol-before)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩            \@testwrongwidth\@currbox
⟨latexrelease | fltrace⟩            \if@test
⟨*trace⟩
⟨latexrelease | fltrace⟩              \fl@trace{type already on list: defer (addtonextcol)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩            \else
⟨*trace⟩
⟨latexrelease | fltrace⟩              \fl@trace{sent to addtotoporbot (addtonextcol)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩              \@addtotoporbot
⟨latexrelease | fltrace⟩            \fi
⟨latexrelease | fltrace⟩         \fi
```

```
⟨*trace⟩
⟨latexrelease | fltrace⟩        \else
⟨latexrelease | fltrace⟩          \fl@trace{Fail---no room: fl box ht: \the \ht \@currbox
⟨latexrelease | fltrace⟩                                                (addtonextcol)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩        \fi
⟨latexrelease | fltrace⟩      \fi
⟨latexrelease | fltrace⟩    \fi
⟨latexrelease | fltrace⟩    \if@insert
⟨latexrelease | fltrace⟩    \else
⟨*trace⟩
⟨latexrelease | fltrace⟩      \fl@trace{put back on deferlist (addtonextcol)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩      \@cons\@deferlist\@currbox
⟨*trace⟩
⟨latexrelease | fltrace⟩      \fl@trace{deferlist: \@deferlist: (addtonextcol-after)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩    \fi
⟨*trace⟩
⟨latexrelease | fltrace⟩   \fl@trace{End of addtonextcol -- locally counts:}%
⟨latexrelease | fltrace⟩   \fl@trace{col: \the\@colnum. top: \the \@topnum. bot: \the \@botnum.}%
⟨/trace⟩
⟨latexrelease | fltrace⟩   \endgroup
⟨*trace⟩
⟨latexrelease | fltrace⟩ \fl@trace{End of addtonextcol -- globally counts:}%
⟨latexrelease | fltrace⟩ \fl@trace{col: \the\@colnum. top: \the \@topnum. bot: \the \@botnum.}%
⟨/trace⟩
⟨latexrelease | fltrace⟩}%
⟨latexrelease | fltrace⟩\EndIncludeInRelease
⟨latexrelease | fltrace⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease | fltrace⟩   {\@addtonextcol}{float order in 2-column}%
⟨latexrelease | fltrace⟩\def\@addtonextcol{%
⟨latexrelease | fltrace⟩   \begingroup
⟨*trace⟩
⟨latexrelease | fltrace⟩    \fl@trace{***Start addtonextcol}%
⟨/trace⟩
⟨latexrelease | fltrace⟩    \@insertfalse
⟨latexrelease | fltrace⟩    \@setfloattypecounts
⟨latexrelease | fltrace⟩    \ifnum \@fpstype=8
⟨*trace⟩
⟨latexrelease | fltrace⟩      \fl@trace{fpstype not curcol:
⟨latexrelease | fltrace⟩                      \the \@fpstype = 8?}%
⟨/trace⟩
⟨latexrelease | fltrace⟩    \else
⟨latexrelease | fltrace⟩      \ifnum \@fpstype=24
⟨*trace⟩
⟨latexrelease | fltrace⟩        \fl@trace{fpstype not curcol:
⟨latexrelease | fltrace⟩                        \the \@fpstype = 24?}%
⟨/trace⟩
⟨latexrelease | fltrace⟩      \else
⟨latexrelease | fltrace⟩        \@flsettextmin
⟨*trace⟩
⟨latexrelease | fltrace⟩        \fl@trace{text-so-far: 0pt (top of col)}%
⟨/trace⟩
```

```
⟨latexrelease | fltrace⟩        \@reqcolroom \ht\@currbox
⟨*trace⟩
⟨latexrelease | fltrace⟩        \fl@trace{float size:
⟨latexrelease | fltrace⟩                \the \@reqcolroom (addtonextcol)}%
⟨latexrelease | fltrace⟩
⟨/trace⟩
⟨latexrelease | fltrace⟩        \advance \@reqcolroom \@textmin
⟨*trace⟩
⟨latexrelease | fltrace⟩        \fl@trace{colroom =
⟨latexrelease | fltrace⟩                  \the \@colroom (addtonextcol)}%
⟨latexrelease | fltrace⟩        \fl@trace{reqcolroom =
⟨latexrelease | fltrace⟩                \the \@reqcolroom (addtonextcol)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩        \ifdim \@colroom>\@reqcolroom
⟨latexrelease | fltrace⟩          \@flsetnum \@colnum
⟨latexrelease | fltrace⟩          \ifnum\@colnum>\z@
⟨latexrelease | fltrace⟩             \@bitor\@currtype\@deferlist
⟨*trace⟩
⟨latexrelease | fltrace⟩             \fl@trace{deferlist: \@deferlist:
⟨latexrelease | fltrace⟩                         (addtonextcol-before)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩             \if@test
⟨*trace⟩
⟨latexrelease | fltrace⟩               \fl@trace{type already on list:
⟨latexrelease | fltrace⟩                           defer (addtonextcol)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩             \else
⟨*trace⟩
⟨latexrelease | fltrace⟩               \fl@trace{sent to addtotoporbot
⟨latexrelease | fltrace⟩                                (addtonextcol)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩               \@addtotoporbot
⟨latexrelease | fltrace⟩             \fi
⟨latexrelease | fltrace⟩          \fi
⟨*trace⟩
⟨latexrelease | fltrace⟩        \else
⟨latexrelease | fltrace⟩          \fl@trace{Fail---no room: fl box ht:
⟨latexrelease | fltrace⟩               \the \ht \@currbox (addtonextcol)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩        \fi
⟨latexrelease | fltrace⟩      \fi
⟨latexrelease | fltrace⟩    \fi
⟨latexrelease | fltrace⟩    \if@insert
⟨latexrelease | fltrace⟩    \else
⟨*trace⟩
⟨latexrelease | fltrace⟩      \fl@trace{put back on deferlist
⟨latexrelease | fltrace⟩                                  (addtonextcol)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩      \@cons\@deferlist\@currbox
⟨*trace⟩
⟨latexrelease | fltrace⟩      \fl@trace{deferlist: \@deferlist:
⟨latexrelease | fltrace⟩                           (addtonextcol-after)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩    \fi
```

```
⟨*trace⟩
⟨latexrelease | fltrace⟩    \fl@trace{End of addtonextcol --
⟨latexrelease | fltrace⟩                                locally counts:}%
⟨latexrelease | fltrace⟩    \fl@trace{col: \the \@colnum.
⟨latexrelease | fltrace⟩       top: \the \@topnum. bot: \the \@botnum.}%
⟨/trace⟩
⟨latexrelease | fltrace⟩   \endgroup
⟨*trace⟩
⟨latexrelease | fltrace⟩   \fl@trace{End of addtonextcol --
⟨latexrelease | fltrace⟩                                globally counts:}%
⟨latexrelease | fltrace⟩   \fl@trace{col: \the \@colnum.
⟨latexrelease | fltrace⟩         top: \the \@topnum. bot: \the \@botnum.}%
⟨/trace⟩
⟨latexrelease | fltrace⟩}%
⟨latexrelease | fltrace⟩\EndIncludeInRelease
```

(*End of definition for* `\@addtonextcol`.)

`\@addtodblcol` Lots of changes.

```
⟨latexrelease | fltrace⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease | fltrace⟩   {\@addtodblcol}{float order in 2-column}%
⟨*2ekernel | latexrelease | fltrace⟩
\def\@addtodblcol{%
  \begingroup
⟨*trace⟩
  \fl@trace{***Start addtodblcol}%
⟨/trace⟩
   \@insertfalse
   \@setfloattypecounts
   \@getfpsbit \tw@
⟨*trace⟩
   \fl@trace{fpstype \ifodd \@tempcnta OK \else not \fi dbltop:
                                                        \the \@fpstype}%
⟨/trace⟩
   \ifodd\@tempcnta
     \@flsetnum \@dbltopnum
     \ifnum \@dbltopnum>\z@
       \@tempswafalse
       \ifdim \@dbltoproom>\ht\@currbox
         \@tempswatrue
⟨*trace⟩
         \fl@trace{Space OK: \@dbltoproom =
                \the \@dbltoproom > \the \ht \@currbox
                                          (dbltoproom)}%
⟨/trace⟩
       \else
⟨*trace⟩
         \fl@trace{fpstype: \the \@fpstype (addtodblcol)}%
⟨/trace⟩
         \ifnum \@fpstype<\sixt@@n
⟨*trace⟩
           \fl@trace{BANG float ignoring \@dbltoproom}%
           \fl@trace{\@spaces \@dbltoproom = \the \@dbltoproom.
                          Ht float: \the \ht \@currbox-BANG}%
```

```
⟨/trace⟩
```

Need to check that there is room on the page, using the local value of `\@textmin` to make the necessary adjustment to `\@dbltoproom`.

```
            \advance \@dbltoproom \@textmin
⟨*trace⟩
            \fl@trace{Local value of texmin: \the\@textmin}%
            \fl@trace{\@spaces space on page = \the \@dbltoproom.
                           Ht float: \the \ht \@currbox-BANG}%
⟨/trace⟩
            \ifdim \@dbltoproom>\ht\@currbox
              \@tempswatrue
⟨*trace⟩
              \fl@trace{Space OK BANG: space on page =
                          \the \@dbltoproom > \the \ht \@currbox}%
            \else
              \fl@trace{fpstype: \the \@fpstype}%
              \fl@trace{Fail---no room dbltoproom-BANG?:}%
              \fl@trace{\@spaces space on page = \the \@dbltoproom.
                           Ht float: \the \ht \@currbox}%
⟨/trace⟩
            \fi
            \advance \@dbltoproom -\@textmin
⟨*trace⟩
          \else
            \fl@trace{fpstype: \the \@fpstype}%
            \fl@trace{Fail---no room dbltoproom-ORD?:}%
            \fl@trace{\@spaces \@dbltoproom = \the \@dbltoproom.
                           Ht float: \the \ht \@currbox}%
⟨/trace⟩
          \fi
        \fi
        \if@tempswa
            \@bitor \@currtype \@deferlist
⟨*trace⟩
            \fl@trace{(dbl)deferlist: \@deferlist: (before)}%
⟨/trace⟩
```

not in fixfloats?

```
          \@testwrongwidth\@currbox
            \if@test
⟨*trace⟩
               \fl@trace{type already on list: (dbl)defer}%
⟨/trace⟩
            \else
               \@tempdima -\ht\@currbox
               \advance\@tempdima
                 -\ifx \@dbltoplist\@empty \dbltextfloatsep \else
                                           \dblfloatsep \fi
               \global \advance \@dbltoproom \@tempdima
               \global \advance \@colht \@tempdima
               \global \advance \@dbltopnum \m@ne
               \@cons \@dbltoplist \@currbox
⟨*trace⟩
```

```
              \fl@trace{dbltopnum (after) = \the \@dbltopnum}%
              \fl@trace{***Success: dbltop}%
⟨/trace⟩
              \@inserttrue
            \fi
        \fi
⟨*trace⟩
      \else
        \fl@trace{Fail: dbltopnum = \the \@dbltopnum: fpstype
                                                \the \@fpstype=ORD?}%
        \ifnum \@fpstype<\sixt@@n
          \fl@trace{ERROR: !t float not successful (addtodblcol)}%
        \fi
⟨/trace⟩
      \fi
    \fi
    \if@insert
    \else
⟨*trace⟩
      \fl@trace{put on deferlist}%
⟨/trace⟩
      \@cons\@deferlist\@currbox
⟨*trace⟩
      \fl@trace{(dbl)deferlist: \@deferlist: (after)}%
⟨/trace⟩
    \fi
⟨*trace⟩
    \fl@trace{End of addtodblcol -- locally count:}%
    \fl@trace{ dbltop: \the \@dbltopnum.}%
⟨/trace⟩
  \endgroup
⟨*trace⟩
  \fl@trace{End of addtodblcol -- globally count:}%
  \fl@trace{dbltop: \the \@dbltopnum.}%
⟨/trace⟩
}%
⟨/2ekernel | latexrelease | fltrace⟩
⟨latexrelease | fltrace⟩\EndIncludeInRelease
⟨latexrelease | fltrace⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease | fltrace⟩   {\@addtodblcol}{float order in 2-column}%
⟨latexrelease | fltrace⟩\def\@addtodblcol{%
⟨latexrelease | fltrace⟩   \begingroup
⟨*trace⟩
⟨latexrelease | fltrace⟩   \fl@trace{***Start addtodblcol}%
⟨/trace⟩
⟨latexrelease | fltrace⟩    \@insertfalse
⟨latexrelease | fltrace⟩    \@setfloattypecounts
⟨latexrelease | fltrace⟩    \@getfpsbit \tw@
⟨*trace⟩
⟨latexrelease | fltrace⟩    \fl@trace{fpstype \ifodd \@tempcnta OK
⟨latexrelease | fltrace⟩                  \else not \fi dbltop: \the \@fpstype}%
⟨/trace⟩
⟨latexrelease | fltrace⟩    \ifodd\@tempcnta
⟨latexrelease | fltrace⟩      \@flsetnum \@dbltopnum
```

```
⟨latexrelease | fltrace⟩      \ifnum \@dbltopnum>\z@
⟨latexrelease | fltrace⟩        \@tempswafalse
⟨latexrelease | fltrace⟩        \ifdim \@dbltoproom>\ht\@currbox
⟨latexrelease | fltrace⟩          \@tempswatrue
⟨∗trace⟩
⟨latexrelease | fltrace⟩          \fl@trace{Space OK: \@dbltoproom =
⟨latexrelease | fltrace⟩                 \the \@dbltoproom > \the \ht \@currbox
⟨latexrelease | fltrace⟩                                      (dbltoproom)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩        \else
⟨∗trace⟩
⟨latexrelease | fltrace⟩       \fl@trace{fpstype: \the \@fpstype (addtodblcol)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩          \ifnum \@fpstype<\sixt@@n
⟨∗trace⟩
⟨latexrelease | fltrace⟩            \fl@trace{BANG float ignoring \@dbltoproom}%
⟨latexrelease | fltrace⟩            \fl@trace{\@spaces \@dbltoproom =
⟨latexrelease | fltrace⟩                     \the \@dbltoproom.
⟨latexrelease | fltrace⟩                     Ht float: \the \ht \@currbox-BANG}%
⟨/trace⟩
```

Need to check that there is room on the page, using the local value of `\@textmin` to make the necessary adjustment to `\@dbltoproom`.

```
⟨latexrelease | fltrace⟩            \advance \@dbltoproom \@textmin
⟨∗trace⟩
⟨latexrelease | fltrace⟩        \fl@trace{Local value of texmin: \the\@textmin}%
⟨latexrelease | fltrace⟩        \fl@trace{\@spaces space on page =
⟨latexrelease | fltrace⟩                    \the \@dbltoproom.
⟨latexrelease | fltrace⟩                     Ht float: \the \ht \@currbox-BANG}%
⟨/trace⟩
⟨latexrelease | fltrace⟩            \ifdim \@dbltoproom>\ht\@currbox
⟨latexrelease | fltrace⟩              \@tempswatrue
⟨∗trace⟩
⟨latexrelease | fltrace⟩          \fl@trace{Space OK BANG: space on page =
⟨latexrelease | fltrace⟩                 \the\@dbltoproom > \the\ht\@currbox}%
⟨latexrelease | fltrace⟩          \else
⟨latexrelease | fltrace⟩          \fl@trace{fpstype: \the \@fpstype}%
⟨latexrelease | fltrace⟩          \fl@trace{Fail---no room dbltoproom-BANG?:}%
⟨latexrelease | fltrace⟩          \fl@trace{\@spaces space on page =
⟨latexrelease | fltrace⟩                      \the \@dbltoproom.
⟨latexrelease | fltrace⟩                       Ht float: \the \ht \@currbox}%
⟨/trace⟩
⟨latexrelease | fltrace⟩            \fi
⟨latexrelease | fltrace⟩            \advance \@dbltoproom -\@textmin
⟨∗trace⟩
⟨latexrelease | fltrace⟩          \else
⟨latexrelease | fltrace⟩            \fl@trace{fpstype: \the \@fpstype}%
⟨latexrelease | fltrace⟩            \fl@trace{Fail---no room dbltoproom-ORD?:}%
⟨latexrelease | fltrace⟩            \fl@trace{\@spaces \@dbltoproom =
⟨latexrelease | fltrace⟩               \the \@dbltoproom.
⟨latexrelease | fltrace⟩               Ht float: \the \ht \@currbox}%
⟨/trace⟩
⟨latexrelease | fltrace⟩          \fi
⟨latexrelease | fltrace⟩        \fi
```

```
⟨latexrelease | fltrace⟩        \if@tempswa
⟨latexrelease | fltrace⟩            \@bitor \@currtype \@dbldeferlist
⟨*trace⟩
⟨latexrelease | fltrace⟩            \fl@trace{dbldeferlist:
⟨latexrelease | fltrace⟩                      \@dbldeferlist: (before)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩            \if@test
⟨*trace⟩
⟨latexrelease | fltrace⟩              \fl@trace{type already on list: dbldefer}%
⟨/trace⟩
⟨latexrelease | fltrace⟩            \else
⟨latexrelease | fltrace⟩               \@tempdima -\ht\@currbox
⟨latexrelease | fltrace⟩               \advance\@tempdima
⟨latexrelease | fltrace⟩                 -\ifx \@dbltoplist\@empty
⟨latexrelease | fltrace⟩                       \dbltextfloatsep
⟨latexrelease | fltrace⟩                  \else \dblfloatsep \fi
⟨latexrelease | fltrace⟩               \global \advance \@dbltoproom \@tempdima
⟨latexrelease | fltrace⟩               \global \advance \@colht \@tempdima
⟨latexrelease | fltrace⟩               \global \advance \@dbltopnum \m@ne
⟨latexrelease | fltrace⟩               \@cons \@dbltoplist \@currbox
⟨*trace⟩
⟨latexrelease | fltrace⟩               \fl@trace{dbltopnum (after) =
⟨latexrelease | fltrace⟩                                  \the \@dbltopnum}%
⟨latexrelease | fltrace⟩               \fl@trace{***Success: dbltop}%
⟨/trace⟩
⟨latexrelease | fltrace⟩               \@inserttrue
⟨latexrelease | fltrace⟩            \fi
⟨latexrelease | fltrace⟩        \fi
⟨*trace⟩
⟨latexrelease | fltrace⟩      \else
⟨latexrelease | fltrace⟩        \fl@trace{Fail: dbltopnum = \the \@dbltopnum:
⟨latexrelease | fltrace⟩                              fpstype \the \@fpstype=ORD?}%
⟨latexrelease | fltrace⟩        \ifnum \@fpstype<\sixt@@n
⟨latexrelease | fltrace⟩          \fl@trace{ERROR: !t float not successful
⟨latexrelease | fltrace⟩                                          (addtodblcol)}%
⟨latexrelease | fltrace⟩        \fi
⟨/trace⟩
⟨latexrelease | fltrace⟩      \fi
⟨latexrelease | fltrace⟩    \fi
⟨latexrelease | fltrace⟩    \if@insert
⟨latexrelease | fltrace⟩    \else
⟨*trace⟩
⟨latexrelease | fltrace⟩      \fl@trace{put on dbldeferlist}%
⟨/trace⟩
⟨latexrelease | fltrace⟩      \@cons\@dbldeferlist\@currbox
⟨*trace⟩
⟨latexrelease | fltrace⟩      \fl@trace{dbldeferlist: \@dbldeferlist: (after)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩    \fi
⟨*trace⟩
⟨latexrelease | fltrace⟩    \fl@trace{End of addtodblcol -- locally count:}%
⟨latexrelease | fltrace⟩    \fl@trace{ dbltop: \the \@dbltopnum.}%
⟨/trace⟩
⟨latexrelease | fltrace⟩  \endgroup
```

```
⟨*trace⟩
⟨latexrelease | fltrace⟩  \fl@trace{End of addtodblcol -- globally count:}%
⟨latexrelease | fltrace⟩  \fl@trace{dbltop: \the \@dbltopnum.}%
⟨/trace⟩
⟨latexrelease | fltrace⟩}%
⟨latexrelease | fltrace⟩\EndIncludeInRelease
```

(*End of definition for* `\@addtodblcol`.)

`\@addmarginpar`

```
⟨*2ekernel⟩
\def\@addmarginpar{\@next\@marbox\@currlist{\@cons\@freelist\@marbox
    \@cons\@freelist\@currbox}\@latexbug\@tempcnta\@ne
    \if@twocolumn
        \if@firstcolumn \@tempcnta\m@ne \fi
    \else
      \if@mparswitch
         \ifodd\c@page \else\@tempcnta\m@ne \fi
      \fi
      \if@reversemargin \@tempcnta -\@tempcnta \fi
    \fi
    \ifnum\@tempcnta <\z@  \global\setbox\@marbox\box\@currbox \fi
    \@tempdima\@mparbottom
    \advance\@tempdima -\@pageht
    \advance\@tempdima\ht\@marbox
    \ifdim\@tempdima >\z@
      \@latex@warning@no@line {Marginpar on page \thepage\space moved}%
    \else
      \@tempdima\z@
    \fi
    \global\@mparbottom\@pageht
    \global\advance\@mparbottom\@tempdima
    \global\advance\@mparbottom\dp\@marbox
    \global\advance\@mparbottom\marginparpush
    \advance\@tempdima -\ht\@marbox
```

Putting box movement inside the 'marbox':

```
    \global\setbox \@marbox
                   \vbox {\vskip \@tempdima
                          \box \@marbox}%
    \global \ht\@marbox \z@
    \global \dp\@marbox \z@
```

Sticking (rather than gluing:-) the 'marbox' to the line above, changed vskip to kern:

```
    \kern -\@pagedp
    \nointerlineskip
    \hb@xt@\columnwidth
      {\ifnum \@tempcnta >\z@
          \hskip\columnwidth \hskip\marginparsep
       \else
          \hskip -\marginparsep \hskip -\marginparwidth
       \fi
       \box\@marbox \hss}%
```

For this reason the following code can vanish:

```
\nobreak             %% No longer needed.  CAR92/12
\vskip -\@tempdima   %% No longer needed.  CAR92/12
    \nointerlineskip
    \hbox{\vrule \@height\z@ \@width\z@ \@depth\@pagedp}}
```

(*End of definition for* `\@addmarginpar`.)

2.2.2 Kludgeins

This part of the file is part of the implementation of the following two new commands for LaTeX2e.

`\enlargethispage{<dim>}`

Adds `<dim>` to the height of the current column only. On the printed page the bottom of this column is extended downwards by exactly `<dim>` without having any effect on the placement of the footer; this may result in an overprinting.

`\enlargethispage*{<dim>}`

Similar to `\enlargethispage` but it tries to squeeze the column to be printed in as small a space as possible, ie it uses any shrinkability in the column. If the column was not explicitly broken (e.g. with `\pagebreak`) this may result in an overfull box message but except for this it will come out as expected (if you know what to expect).

The star form of this command is dedicated to Leslie Lamport, the other we need for ourselves (FMi, CAR).

These commands may well have unwanted effects if used soon before a `\clearpage`: please give keep them clear of such places.

`\@kludgeins` The insert which makes TeX do a lot of the necessary work. All we need to put into it is the amount by which the pagegoal should be changed.

```
\newinsert \@kludgeins
\global\dimen\@kludgeins \maxdimen
\global\count\@kludgeins 1000
```

(*End of definition for* `\@kludgeins`.)

`\enlargethispage` `\enlargethispage*` The user command.

```
\gdef \enlargethispage {%
   \@ifstar
     {%
⟨*trace⟩
      \fl@trace{Enlarging page height * }%
⟨/trace⟩
      \@enlargepage{\hbox{\kern\p@}}}%
     {%
⟨*trace⟩
      \fl@trace{Enlarging page height exactly---}%
⟨/trace⟩
      \@enlargepage\@empty}%
}
```

(*End of definition for* `\enlargethispage` *and* `\enlargethispage*`.)

\@enlargepage This actually inserts the insert, after checking for extreme values of the change.

```
\gdef\@enlargepage#1#2{%
⟨*trace⟩
   \fl@trace{\@spaces\@spaces by #2}%
⟨/trace⟩
   \@tempskipa#2\relax
   \ifdim \@tempskipa>.5\maxdimen
     \@latex@error{Suggested\space extra\space height\space
                   (\the\@tempskipa)\space dangerously\space
                   large}\@eha
   \else
     \ifdim \vsize<.5\maxdimen
⟨*trace⟩
       \fl@trace {Kludgeins added--pagegoal before: \the\pagegoal}%
⟨/trace⟩
       \@bsphack
         \insert\@kludgeins{#1\vskip-\@tempskipa}%
       \@esphack
```

This next bit is for tracing only:

```
⟨*trace⟩
       \ifvmode \par
         \fl@trace {Kludgeins added--pagegoal after: \the \pagegoal}%
       \fi
⟨/trace⟩
     \else
       \@latex@error{Page\space height\space already\space
                     too\space large}\@eha
     \fi
   \fi
}
```

(*End of definition for* `\@enlargepage`.)

\ShowFloat This command provides some information about the contenta of a float register. Float registers have internal names of the form `\bx@`⟨*Uppercase-letter(s)-or numbers*⟩ and you specify just this letter or letters as the argument, e.g., `\ShowFloat{A}`. (There is not much error recovery if you specify something that isn't a float.)

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2021/11/15}%
⟨latexrelease⟩                  {\ShowFloat}{Show float register contents}%
\def\ShowFloat#1{\begingroup
    \let \fl@trace \fl@tracemessage
    \fl@trace{***Float #1 details:}%
    \ifcsname bx@#1\endcsname
      \expandafter\fl@ShowFloat\csname bx@#1\endcsname
    \else
      \fl@trace{Not a float!}%
    \fi
  \endgroup
}
\def\fl@ShowFloat#1{%
  \fl@traceval{\count#1}%  % this here should be interpreted on day
```

```
  \fl@traceval{\ht#1}%
  \fl@traceval{\dp#1}%
  \fl@traceval{\wd#1}%
  {\tracingonline1\showboxbreadth10\showboxdepth3\showbox#1}%
}
```

Here are two definitions from `fltrace` that make the above code work:

```
\def \fl@traceval #1{\fl@trace{\string #1 = \the #1}}
\def \fl@tracemessage #1{{\let\@elt\@empty\typeout{LaTeX2e: #1}}}
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\ShowFloat}{Show float register contents}%
⟨latexrelease⟩
⟨latexrelease⟩\let\ShowFloat\@undefined
⟨latexrelease⟩\let\fl@ShowFloat\@undefined
⟨latexrelease⟩\let\fl@traceval\@undefined
⟨latexrelease⟩\let\fl@tracemessage\@undefined
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\ShowFloat`.)

2.2.3 Float control

This part implements controllable floats and other changes to the float mechanism.

It provides, at the document level, the following command for inclusion in LaTeX2e.

`\suppressfloats`

This suppresses all further floats on the current page.

With an optional argument it suppresses only floats only in certain positions on the current page.

`[t]` suppresses only floats at the top of the page `[b]` suppresses only floats at the bottom of the page

It also enables the use of an extra specifier, `!`, in the location optional argument of a float. If this is present then, just for this particular float, whenever it is processed by the float mechanism the following are ignored:

- all restrictions on the number of floats which can appear;
- all explicit restrictions on the amount of space which should (not) be occupied by floats and/or text.

The mechanism will still attempt to ensure that pages are not overfull.

These specifiers override, for the single float, the suppression commands described above.

In its current form, it also supplies a reasonably exhaustive, and somewhat baroque, means of tracing some aspects of the float mechanism.

More tracing.

\fl@trace
\tracefloatsoff
\tracefloats
\fl@traceval
\tracefloatvals
\fl@tracemessage

Set-up tracing for floats independent of other tracing as it produces mega-output. Default is no tracing.

```
⟨*fltrace⟩
\def \fl@tracemessage #1{{\let\@elt\@empty\typeout{LaTeX2e: #1}}}
\def \tracefloats{\let \fl@trace \fl@tracemessage}
\def \tracefloatsoff {\let \fl@trace \@gobble}
\tracefloatsoff
\def \fl@traceval #1{\fl@trace{\string #1 = \the #1}}
\IncludeInRelease{2015/01/01}{\tracefloatvals}%
                             {trace float vals}%
\def \tracefloatvals{%
```

As `\@dblfloatplacement` sets `\f@depth` it needs to be run inside a group, otherwise the float placement will test for the wrong value.[65]

```
 \begingroup
```

When the user requests `\tracefloatvals` then they should show regardless of the tracing state, so locally we make sure that it is activated.

```
   \tracefloats
   \@dblfloatplacement
   \@floatplacement
   \fl@trace{***Float placement parameters:}%
   \fl@traceval\@colnum
   \fl@traceval\@colroom
   \fl@traceval\@topnum
   \fl@traceval\@toproom
   \fl@traceval\@botnum
   \fl@traceval\@botroom
   \fl@traceval\@fpmin
   \fl@trace{\string\textfraction = \textfraction}%
   \fl@traceval\@dbltopnum
   \fl@traceval\@dbltoproom
   \fl@trace{\string\textfraction = \textfraction}%
   \fl@trace{toplist: \@toplist}%
   \fl@trace{botlist: \@botlist}%
   \fl@trace{midlist: \@midlist}%
   \fl@trace{deferlist: \@deferlist}%
   \fl@trace{dbltoplist: \@dbltoplist}%
%FMi  \fl@trace{dbldeferlist: \@dbldeferlist}%
 \endgroup
}
\EndIncludeInRelease
\IncludeInRelease{0000/00/00}{\tracefloatvals}%
                             {trace float vals}%
\def \tracefloatvals{%
 \begingroup
  \tracefloats
  \@dblfloatplacement
  \@floatplacement
  \fl@trace{***Float placement parameters:}%
  \fl@traceval\@colnum
  \fl@traceval\@colroom
```

[65] This is a somewhat questionable design.

```
  \fl@traceval\@topnum
  \fl@traceval\@toproom
  \fl@traceval\@botnum
  \fl@traceval\@botroom
  \fl@traceval\@fpmin
  \fl@trace{\string\textfraction = \textfraction}%
  \fl@traceval\@dbltopnum
  \fl@traceval\@dbltoproom
  \fl@trace{\string\textfraction = \textfraction}%
  \fl@trace{toplist: \@toplist}%
  \fl@trace{botlist: \@botlist}%
  \fl@trace{midlist: \@midlist}%
  \fl@trace{deferlist: \@deferlist}%
  \fl@trace{dbltoplist: \@dbltoplist}%
% next line only in old releases
  \fl@trace{dbldeferlist: \@dbldeferlist}%
 \endgroup
}
\EndIncludeInRelease
```

We need to make sure that `fltrace` comes before `flafter` to make the tracing work.

```
\@ifpackageloaded{flafter}
 {\PackageWarningNoLine
      {fltrace}{Load 'fltrace' before 'flafter'\MessageBreak
                Attempting to recover by reloading 'flafter'}%
```

Hide the fact that `flafter` was already loaded and then request it anew.

```
    \expandafter\let\csname ver@flafter.sty\endcsname\relax
    \def\reserved@a#1{%
      \expandafter\let\csname\string#1+flafter+IIR\endcsname\relax}%
    \reserved@a\@addtocurcol
    \reserved@a\@addtonextcol
    \RequirePackage{flafter}}{}
```

⟨/fltrace⟩

As the code for `flafter` will contain tracing calls so that it works in conjunction with `fltrace` we need to provide a dummy definition for `\fl@trace` in that package.

```
⟨*flafter⟩
\providecommand\fl@trace[1]{}
⟨/flafter⟩
```

(*End of definition for* `\fl@trace` *and others.*)

`\suppressfloats` `\@flstop`

Float suppression commands: these set the relevant counter globally to zero. Thus they are overridden for a particular float by an ! specifier.

```
⟨*2ekernel⟩
\def \suppressfloats {%
   \@ifnextchar [%
     \@flstop
    {\global \@colnum \z@}%
}
```

Maybe this should be a loop over `#1`?

```
\def \@flstop [#1]{%
   \if t#1%
```

```
    \global \@topnum \z@
  \fi
  \if b#1%
    \global \@botnum \z@
  \fi
}
```

(*End of definition for* `\suppressfloats` *and* `\@flstop`.)

Manipulation of float placement and type; both their strings and the corresponding count registers.

`\@fpstype` `\@reqcolroom` `\@textfloatsheight`

First a new count register to go with `\@currtype`.

Then a new skip register, for information needed to remove the `\@maxsep` conservatism: it is possible that this could use a temporary register.

Finally a dimension register to hold the total height of in-text floats on the current page. This is needed to implement a major change in the functionality of `\@addtocurcol` which is, nevertheless, a bug fix. It is not local and therefore cannot be a temporary register.

```
\newcount \@fpstype
\newdimen \@reqcolroom
\newdimen \@textfloatsheight
```

(*End of definition for* `\@fpstype`, `\@reqcolroom`, *and* `\@textfloatsheight`.)

`\saved@reqcolroom`

Saved value of `\@reqcolroom`; this is needed when making several tests.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease⟩                  {\saved@reqcolroom}{float placement calculation}%
\newdimen \saved@reqcolroom
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\saved@reqcolroom}{float placement calculation}%
⟨latexrelease⟩\let \saved@reqcolroom \@undefined
⟨latexrelease⟩\EndIncludeInRelease
```

(*End of definition for* `\saved@reqcolroom`.)

`\@fpsadddefault`

Adds the default placement to what is already there.

Should not need to change this, but could do it as follows:

```
def \@fpsadddefault {%
  \@temptokena \expandafter\expandafter\expandafter
               {\csname fps@\@captype \endcsname}%
  \edef \reserved@a {\the\@temptokena}%
  \@onelevel@sanitize \reserved@a
  \edef \@fps {\@fps\reserved@a}%
```

```
⟨*2ekernel | fltrace⟩
\def \@fpsadddefault {%
⟨*trace⟩
   \fl@trace{fps changed from: \@fps}%
⟨/trace⟩
   \edef \@fps {\@fps\csname fps@\@captype \endcsname}%
   \@latex@warning {%
     No positions in optional float specifier.\MessageBreak
     Default added (so using `\@fps')}%
}
```

(*End of definition for* `\@fpsadddefault`.)

`\@setfloattypecounts` Sets counters `\@fpstype` and `\@currtype`.
BANG == bit4 of `\count\@currbox` = 0.

```
\def \@setfloattypecounts {%
  \@currtype \count\@currbox
  \@fpstype \count\@currbox
  \divide\@currtype\@xxxii \multiply\@currtype\@xxxii
  \advance \@fpstype -\@currtype
⟨*trace⟩
  \fl@trace{(mod 32) fpstype: \the \@fpstype}%
  \fl@trace{(mult of 32) currtype: \the \@currtype}%
% Tracing only: but some should be changed into real errors/warnings?
  \ifnum \@fpstype<\sixt@@n
    \ifnum \@fpstype=\z@
      \fl@trace{ERROR: no PLACEMENT, fpstype = \the \@fpstype = 0?}%
    \fi
    \ifnum \@fpstype=\@ne
      \fl@trace{WARNING: only h, fpstype = \the \@fpstype = 1?}%
    \fi
    \fl@trace{BANG float}%
  \else
    \ifnum \@fpstype=\sixt@@n
      \fl@trace{ERROR: no PLACEMENT, fpstype = \the \@fpstype = 16?}%
    \fi
    \ifnum \@fpstype=17
      \fl@trace{WARNING: only h, fpstype = \the \@fpstype = 17?}%
    \fi
    \fl@trace{ORD float}%
  \fi
⟨/trace⟩
}
⟨/2ekernel | fltrace⟩
```

(*End of definition for* `\@setfloattypecounts`.)

Macros for getting, testing and setting bits of the fps.

`\@getfpsbit` Sets `\@tempcnta` to required bit of `\count\@currbox`.

```
⟨*2ekernel⟩
\def \@getfpsbit {%
   \@boxfpsbit \@currbox
}
```

(*End of definition for* `\@getfpsbit`.)

`\@boxfpsbit` Used above.

```
\def \@boxfpsbit #1#2{%
   \@tempcnta \count#1%
   \divide \@tempcnta #2\relax
}
```

(*End of definition for* `\@boxfpsbit`.)

`\@testfp` New definition of the float page test.

```
\def \@testfp #1{%
   \@boxfpsbit #18\relax % Really '#1 8' for human readers!
   \ifodd \@tempcnta
   \else
     \@testtrue
   \fi
}
```

(*End of definition for* `\@testfp`.)

`\@setfpsbit` Sets required bit of `\@tempcnta` (to 1).

```
\def \@setfpsbit #1{%
   \@tempcntb \@tempcnta
   \divide \@tempcntb #1\relax
   \ifodd \@tempcntb
   \else
     \advance \@tempcnta #1\relax
   \fi
}
⟨/2ekernel⟩
```

(*End of definition for* `\@setfpsbit`.)

`\@resethfps` Globally adds t as a possible location for an h or !h only placement: this must be done using the count.

Although it will leave `\@fpstype` set to 17 even if it was originally 1, this does not matter since it is the last thing in `\@addtocurcol`.

```
⟨*2ekernel | fltrace⟩
\def \@resethfps {%
   \let\reserved@a\@empty
   \ifnum \@fpstype=\@ne
      \def \reserved@a {!}%
      \@fpstype 17
   \fi
   \ifnum \@fpstype=17
     \global \advance \count\@currbox \tw@
     \@latex@warning@no@line {%
       '\reserved@a h' float specifier changed to '\reserved@a ht'}%
⟨*trace⟩
     \fl@trace{%
        't' added to '\reserved@a h'- new Count: \the \count\@currbox}%
⟨/trace⟩
   \fi
}
```

(*End of definition for* `\@resethfps`.)

Special stuff for BANG floats.

`\@flsetnum` Ignores any zero float counter value in case BANG.

It uses a local assignment to the normally global counter: a bit naughty, perhaps?

These assignments are safe so long as the counter involved is only consulted once (i.e. only for the 'bang float') with the changed value. This is the case within `\@addtocurcol` because it is used only once within a call of the output routine (which forms a group).

For `\@addtonextcol` this is achieved by putting a group around its code; this is needed because it is called (by `\@startcolumn`) for each float which was on the deferlist. Almost identical considerations pertain to `\@addtodblcol`. There may be more efficient ways to handle this, but the group seems to be the simplest.

```
\def \@flsetnum #1{%
⟨*trace⟩
   \fl@trace{fpstype: \the \@fpstype (flsetnum \string#1)}%
⟨/trace⟩
   \ifnum \@fpstype<\sixt@@n
     \ifnum #1=\z@
⟨*trace⟩
       \fl@trace{BANG float resetting \string#1 to 1}%
⟨/trace⟩
       #1\@ne
     \fi
   \fi
⟨*trace⟩
   \fl@trace{#1 (before) = \the #1}%
⟨/trace⟩
}
```

(*End of definition for* `\@flsetnum`.)

`\@flsettextmin` This ignores `\textfraction` space restriction in case BANG.

```
\def \@flsettextmin {%
⟨*trace⟩
   \fl@trace{fpstype: \the \@fpstype (flsettextmin)}%
⟨/trace⟩
   \ifnum \@fpstype<\sixt@@n
⟨*trace⟩
     \fl@trace{BANG ignoring textmin}%
⟨/trace⟩
     \@textmin \z@
   \else
     \@textmin \textfraction\@colht
⟨*trace⟩
     \fl@trace{ORD textmin = \the \@textmin}%
⟨/trace⟩
   \fi
}
```

(*End of definition for* `\@flsettextmin`.)

`\@flcheckspace` This ignores space restriction in case BANG; this is still slightly conservative since it does not allow for the fact that, if there is no text in the column then `\textfloatsep` is not needed. Sets `@tempswa` true if there is room for `\@currbox`.

```
⟨/2ekernel | fltrace⟩
⟨*2ekernel | fltrace | latexrelease⟩
⟨latexrelease | fltrace⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease | fltrace⟩                    {\@flcheckspace}{float placement calculation}%
\def \@flcheckspace #1#2{%
```

When this test is executed, the value of `\@reqcolroom` may no longer be correct due to a previous test that altered it. We therefore reset it to a value saved earlier.

```
   \@reqcolroom \saved@reqcolroom
   \advance \@reqcolroom
     \ifx #2\@empty \textfloatsep \else \floatsep \fi
⟨*trace⟩
   \fl@trace{colroom = \the \@colroom
                                    (flcheckspace \string#1 \string#2)}%
   \fl@trace{reqcolroom = \the \@reqcolroom
                                    (flcheckspace \string#1 \string#2)}%
⟨/trace⟩
   \ifdim \@colroom>\@reqcolroom
     \ifdim #1>\ht\@currbox
       \@tempswatrue
⟨*trace⟩
       \fl@trace{Space OK: #1 = \the #1 > \the \ht \@currbox
                                    (flcheckspace \string#1 \string#2)}%
⟨/trace⟩
     \else
⟨*trace⟩
       \fl@trace{fpstype: \the \@fpstype
                                    (flcheckspace \string#1 \string#2)}%
⟨/trace⟩
       \ifnum \@fpstype<\sixt@@n
⟨*trace⟩
         \fl@trace{BANG float ignoring #1
                                    (flcheckspace \string#1 \string#2):}%
         \fl@trace{\@spaces #1 = \the #1.  Ht float: \the \ht \@currbox
                                                    \space BANG}%
⟨/trace⟩
         \@tempswatrue
⟨*trace⟩
       \else
         \fl@trace{Fail---no room (flcheckspace \string#1 \string#2)
                       (fpstype \the \@fpstype=ORD?):}%
         \fl@trace{\@spaces #1 = \the #1.  Ht float: \the \ht \@currbox
                                                      \space ORD?}%
⟨/trace⟩
       \fi
     \fi
⟨*trace⟩
   \else
     \fl@trace{Fail---no room at 2nd test of colroom
                    (flcheckspace \string#1 \string#2)}%
⟨/trace⟩
```

```
    \fi
}
⟨/2ekernel | fltrace | latexrelease⟩
⟨latexrelease | fltrace⟩\EndIncludeInRelease
⟨latexrelease | fltrace⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease | fltrace⟩                  {\@flcheckspace}{float placement calculation}%
⟨latexrelease | fltrace⟩
⟨latexrelease | fltrace⟩\def \@flcheckspace #1#2{%
⟨latexrelease | fltrace⟩    \advance \@reqcolroom
⟨latexrelease | fltrace⟩      \ifx #2\@empty \textfloatsep \else \floatsep \fi
⟨*trace⟩
⟨latexrelease | fltrace⟩    \fl@trace{colroom = \the \@colroom
⟨latexrelease | fltrace⟩                                    (flcheckspace \string#1 \string#2)}%
⟨latexrelease | fltrace⟩    \fl@trace{reqcolroom = \the \@reqcolroom
⟨latexrelease | fltrace⟩                                    (flcheckspace \string#1 \string#2)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩    \ifdim \@colroom>\@reqcolroom
⟨latexrelease | fltrace⟩      \ifdim #1>\ht\@currbox
⟨latexrelease | fltrace⟩        \@tempswatrue
⟨*trace⟩
⟨latexrelease | fltrace⟩        \fl@trace{Space OK: #1 = \the #1 > \the \ht \@currbox
⟨latexrelease | fltrace⟩                                    (flcheckspace \string#1 \string#2)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩      \else
⟨*trace⟩
⟨latexrelease | fltrace⟩        \fl@trace{fpstype: \the \@fpstype
⟨latexrelease | fltrace⟩                                    (flcheckspace \string#1 \string#2)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩        \ifnum \@fpstype<\sixt@@n
⟨*trace⟩
⟨latexrelease | fltrace⟩          \fl@trace{BANG float ignoring #1
⟨latexrelease | fltrace⟩                                    (flcheckspace \string#1 \string#2):}%
⟨latexrelease | fltrace⟩          \fl@trace{\@spaces #1 = \the #1.  Ht float: \the \ht \@currbox
⟨latexrelease | fltrace⟩                                                  \space BANG}%
⟨/trace⟩
⟨latexrelease | fltrace⟩          \@tempswatrue
⟨*trace⟩
⟨latexrelease | fltrace⟩        \else
⟨latexrelease | fltrace⟩          \fl@trace{Fail---no room (flcheckspace \string#1 \string#2)
⟨latexrelease | fltrace⟩                        (fpstype \the \@fpstype=ORD?):}%
⟨latexrelease | fltrace⟩          \fl@trace{\@spaces #1 = \the #1.  Ht float: \the \ht \@currbox
⟨latexrelease | fltrace⟩                                                    \space ORD?}%
⟨/trace⟩
⟨latexrelease | fltrace⟩        \fi
⟨latexrelease | fltrace⟩      \fi
⟨*trace⟩
⟨latexrelease | fltrace⟩    \else
⟨latexrelease | fltrace⟩      \fl@trace{Fail---no room at 2nd test of colroom
⟨latexrelease | fltrace⟩                    (flcheckspace \string#1 \string#2)}%
⟨/trace⟩
⟨latexrelease | fltrace⟩    \fi
⟨latexrelease | fltrace⟩}
⟨latexrelease | fltrace⟩\EndIncludeInRelease
```

(*End of definition for* `\@flcheckspace`.)

`\@flupdates` This updates everything when a float is placed.

```
⟨*2ekernel⟩
\def \@flupdates #1#2#3{%
   \global \advance #1\m@ne
   \global \advance \@colnum \m@ne
   \@tempdima -\ht\@currbox
   \advance \@tempdima
     -\ifx #3\@empty \textfloatsep \else \floatsep \fi
   \global \advance #2\@tempdima
   \global \advance \@colroom \@tempdima
   \@cons #3\@currbox
}
⟨/2ekernel⟩
```

(*End of definition for* `\@flupdates`.)

Interesting facts about float mechanisms past and present, together with a summary of various features, some unresolved:

1. The value `\textfraction` does not affect the processing of doublecol floats: this seems sensible, but should be documented.
2. `\twocolumn` floatplacement was wrong: dbl not needed, ord needed.
3. `\@floatplacement` was not called after `\@startdblcol` or `\@topnewpage`. This has been changed; it is clearly a bug fix.
4. The use `\@topnewpage` when `\dblfigrule` is non-trivial produced a rule in the wrong place. This has been fixed by not using `\dblfigrule` when processing the 'float' from `\@topnewpage`.
5. If the specifier was just h and the float could not be put here, it went on the deferlist and stayed there until a clearpage. It now gets changed to a 'th': this is only an error-recovery action, putting just h or !h should be deprecated.
6. `\@dblmaxsep` was 'the maximum of `\dblfloatsep` and `\dbltexfloatsep`'. But it was never used! Now gone completely, like `\@maxsep`.
7. After an h float is put on a page, it was counted as text when applying the `\textfraction` test; this is possibly too big a change although it is a bug fix?
8. Two consecutive h floats are separated by twice `\intextsep`: this could be changed to one by use of `\addvspace`, OK? Note that it would also mean that less space is put in if an h float immediately follows other spaces. This is also possibly too big a change, at least for compatibility mode? Or it may be simply wrong! It has not been changed.
9. Now `\@addtocurcol` checks first for just p fps. I think that this is an increase in efficiency, but maybe the coding should be made even more efficient.
10. `\@tryfcolumn` now tests if the list is empty first, otherwise lots of wasted time! Thus this test has been removed from `\@startcolumn`. As Frank pointed out, this makes `\@startcolumn` less efficient. But it is now the same as `\@startdblcolumn`: I can see no reason why they should be different, but which is best?

11. Why is `\@colroom` set in `\@doclearpage`?

12. Footnotes. Check what `\clearpage` does when footnotes are left over. Footnotes are not put on float pages and, also, `\@addtonextcol` ignores the existence of held-over footnotes in deciding what floats can go on the page. Not changed.

13. `\clearpage` can still lose non-boxes, at least when floats are involved. It also moves some to the 'wrong page', but this may be a coding problem.

14. The ! option makes it necessary to check in `\output` that there is enough room left on the page after adding a float. (This would have been necessary anyway if anyone set `\@textmin` too close to zero! A similar danger existed also if the text in a `\twocolumn[text]` entity gets too large.) The current implementation of this also makes the normal case a little less efficient, OK? Not enough room means, at present, less than `\baselineskip`, with a warning: is this OK? Should it be made generic (another parameter)?

15. There are four possibilities for supporting this:

    `\twocolumn[\maketitle more text]`

    One is to change `\maketitle` slightly to allow this. Another is to change `\@topnewpage` so that more than one `\twocolumn[]` command is allowed; in this case `\maketitle\twocolumn[more text]` will work. The former is more robust from the user's viewpoint, but makes the code for `\maketitle` rather ad hoc (maybe it is already?). Another is to misuse the global twocolumn flag locally within `\@topnewpage`. Yet another is to move the column count register from the multicol package into the kernel. This has been done.

16. Where should the reinserts be put to maximise the probability that footnotes come out on the correct page? Or should we go for as much compatibility as possible (but see next item)?

17. Should we continue to support (as much as possible) `\samepage`? Some of its intended functionality is now advertised as being provided by `\enlargethispage`. Use of either is likely to result in wrongly placed footnotes, marginals, etc. Which should have priority: obeying the pagination instructions, or correct placement of notes/marginalia?

18. Is the adjustment of space to cause shrinking in the kludge-* case correct? Should it be limited to 0pt?

19. Is the setting of `\boxmaxdepth` in makecol and friends needed? It only has any effect if `\@textbottom` ends with a box or rule, in which case the vskip to allow for its depth should also be added. If it is kept, it should probably be the last thing in the box. It has now been removed.

    It would perhaps be better to document that `\@textbottom` and `\@texttop` must have natural height 0pt.

20. I cannot see why the vskip adjustment for the depth is needed if boxmaxdepth is used to ensure that there is never a too deep box.

21. The value of `\boxmaxdepth` should be explicitly set whenever necessary: it is too risky to assume that it has any particular value. Care is needed in deciding what to set it to.

It is interesting to note that the value of `\boxmaxdepth` is unique in being read before the local settings for the box group are reset; all other parameter settings which affect the box construction use their values outside the box group.

22. Should `\@maxdepth` store the setting of `\maxdepth` from lplain? Or should we provide a proper interface to class files for setting these?

An analysis of various other macros.

`\@opcol` should do `\@floatplacement`, but where? Right at the end, since it always occurs at the start of a column.

```
\def\@opcol{%
  % Why is this done first?
  \global \@mparbottom \z@
  \if@twocolumn
    \@outputdblcol
  \else
    \@outputpage
    % This is not needed since it is done at the end of
    %   |\@outputpage|:
    \global \@colht \textheight
  \fi}
```

Only tracing has been added to these.

```
⟨latexrelease | fltrace⟩\IncludeInRelease{2017/01/01}%
⟨latexrelease | fltrace⟩  {\@makefcolumn}{negative height floats}%
⟨*2ekernel | fltrace | latexrelease⟩
\def\@makefcolumn #1{%
  \begingroup
    \@fpmin -\maxdimen
    \let \@testfp \@gobble
    \@tryfcolumn #1%
  \endgroup
⟨*trace⟩
  \if@fcolmade
    \fl@trace{PAGE: in \string\clearpage
                                \if@twocolumn ---twocolumn\fi---}%
    \fl@trace{----- float column/page completed from \string#1}%
  \fi
⟨/trace⟩
}
⟨latexrelease | fltrace⟩\EndIncludeInRelease
⟨latexrelease | fltrace⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease | fltrace⟩  {\@makefcolumn}{negative height floats}%
⟨latexrelease | fltrace⟩\def\@makefcolumn #1{%
⟨latexrelease | fltrace⟩  \begingroup
⟨latexrelease | fltrace⟩    \@fpmin \z@
⟨latexrelease | fltrace⟩    \let \@testfp \@gobble
⟨latexrelease | fltrace⟩    \@tryfcolumn #1%
⟨latexrelease | fltrace⟩  \endgroup
⟨*trace⟩
```

```
⟨latexrelease | fltrace⟩   \if@fcolmade
⟨latexrelease | fltrace⟩     \fl@trace{PAGE: in \string\clearpage
⟨latexrelease | fltrace⟩                   \if@twocolumn ---twocolumn\fi---}%
⟨latexrelease | fltrace⟩     \fl@trace{----- float column/page completed
⟨latexrelease | fltrace⟩                   from \string#1}%
⟨latexrelease | fltrace⟩   \fi
⟨/trace⟩
⟨latexrelease | fltrace⟩}
⟨latexrelease | fltrace⟩\EndIncludeInRelease
⟨/2ekernel | fltrace | latexrelease⟩
```

This will line up the last baselines in the two columns provided they are constructed in the normal way: i.e. ending in a skip of minus the original depth, with `\@textbottom` adding nothing.

Thus again it is essential for `\@textbottom` to have depth 0pt.

```
⟨latexrelease | fltrace⟩\IncludeInRelease{2025/06/01}%
⟨latexrelease | fltrace⟩   {\@outputdblcol}{Use new mark mechanism}%
⟨∗2ekernel | fltrace | latexrelease⟩

\def\@outputdblcol{%
  \if@firstcolumn
    \global\@firstcolumnfalse
```

Save the left column

```
    \global\setbox\@leftcolumn\copy\@outputbox
⟨fltrace⟩    \fl@trace{PAGE: first column boxed}%

  \else
    \global\@firstcolumntrue
    \setbox\@outputbox\vbox{%
     \hb@xt@\textwidth{%
        \hb@xt@\columnwidth{\box\@leftcolumn \hss}%
        \hfil
```

The color of the `\vrule` should be `\normalcolor` as to not inherit the color from the column.

```
        {\normalcolor\vrule \@width\columnseprule}%
        \hfil
       \hb@xt@\columnwidth{\box\@outputbox \hss}}}%
⟨fltrace⟩    \fl@trace{PAGE: second column also boxed}%
    \@combinedblfloats
    \@outputpage
⟨fltrace⟩    \fl@trace{PAGE: two column page completed}%
    \begingroup
      \@dblfloatplacement
      \@startdblcolumn
      \@whilesw\if@fcolmade \fi{\@outputpage
⟨fltrace⟩      \fl@trace{PAGE: double float page completed}%
     \@startdblcolumn}%
    \endgroup
  \fi}%

⟨latexrelease | fltrace⟩\EndIncludeInRelease
⟨latexrelease | fltrace⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease | fltrace⟩   {\@outputdblcol}{2 column marks}%
⟨latexrelease | fltrace⟩\def\@outputdblcol{%
```

```
⟨latexrelease | fltrace⟩  \if@firstcolumn
⟨latexrelease | fltrace⟩    \global\@firstcolumnfalse
⟨latexrelease | fltrace⟩    \global\setbox\@leftcolumn\copy\@outputbox
⟨latexrelease | fltrace⟩    \splitmaxdepth\maxdimen
⟨latexrelease | fltrace⟩    \vbadness\maxdimen
⟨latexrelease | fltrace⟩     \setbox\@outputbox\vbox{\unvbox\@outputbox\unskip}%
⟨latexrelease | fltrace⟩     \setbox\@outputbox\vsplit\@outputbox to\maxdimen
⟨latexrelease | fltrace⟩    \toks@\expandafter{\topmark}%
⟨latexrelease | fltrace⟩    \xdef\@firstcoltopmark{\the\toks@}%
⟨latexrelease | fltrace⟩    \toks@\expandafter{\splitfirstmark}%
⟨latexrelease | fltrace⟩    \xdef\@firstcolfirstmark{\the\toks@}%
⟨latexrelease | fltrace⟩    \ifx\@firstcolfirstmark\@empty
⟨latexrelease | fltrace⟩      \global\let\@setmarks\relax
⟨latexrelease | fltrace⟩    \else
⟨latexrelease | fltrace⟩      \gdef\@setmarks{%
⟨latexrelease | fltrace⟩        \let\firstmark\@firstcolfirstmark
⟨latexrelease | fltrace⟩        \let\topmark\@firstcoltopmark}%
⟨latexrelease | fltrace⟩    \fi
⟨latexrelease | fltrace⟩  \else
⟨latexrelease | fltrace⟩    \global\@firstcolumntrue
⟨latexrelease | fltrace⟩    \setbox\@outputbox\vbox{%
⟨latexrelease | fltrace⟩     \hb@xt@\textwidth{%
⟨latexrelease | fltrace⟩        \hb@xt@\columnwidth{\box\@leftcolumn \hss}%
⟨latexrelease | fltrace⟩        \hfil
⟨latexrelease | fltrace⟩        {\normalcolor\vrule \@width\columnseprule}%
⟨latexrelease | fltrace⟩        \hfil
⟨latexrelease | fltrace⟩       \hb@xt@\columnwidth{\box\@outputbox \hss}}}%
⟨latexrelease | fltrace⟩  \@combinedblfloats
⟨latexrelease | fltrace⟩    \@setmarks
⟨latexrelease | fltrace⟩    \@outputpage
⟨latexrelease | fltrace⟩    \begingroup
⟨latexrelease | fltrace⟩      \@dblfloatplacement
⟨latexrelease | fltrace⟩      \@startdblcolumn
⟨latexrelease | fltrace⟩      \@whilesw\if@fcolmade \fi{\@outputpage
⟨latexrelease | fltrace⟩     \@startdblcolumn}%
⟨latexrelease | fltrace⟩    \endgroup
⟨latexrelease | fltrace⟩  \fi}
⟨latexrelease | fltrace⟩\EndIncludeInRelease

⟨latexrelease | fltrace⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease | fltrace⟩  {\@outputdblcol}{2 column marks}%
⟨latexrelease | fltrace⟩\def\@outputdblcol{%
⟨latexrelease | fltrace⟩  \if@firstcolumn
⟨latexrelease | fltrace⟩    \global \@firstcolumnfalse
⟨latexrelease | fltrace⟩    \global \setbox\@leftcolumn \box\@outputbox
⟨*trace⟩
⟨latexrelease | fltrace⟩    \fl@trace{PAGE: first column boxed}%
⟨/trace⟩
⟨latexrelease | fltrace⟩  \else
⟨latexrelease | fltrace⟩    \global \@firstcolumntrue
⟨latexrelease | fltrace⟩    \setbox\@outputbox \vbox {%
⟨latexrelease | fltrace⟩                          \hb@xt@\textwidth {%
⟨latexrelease | fltrace⟩                            \hb@xt@\columnwidth {%
⟨latexrelease | fltrace⟩                              \box\@leftcolumn \hss}%
⟨latexrelease | fltrace⟩                            \hfil
```

```
⟨latexrelease | fltrace⟩                                {\normalcolor\vrule
⟨latexrelease | fltrace⟩                                    \@width\columnseprule}%
⟨latexrelease | fltrace⟩                                \hfil
⟨latexrelease | fltrace⟩                                \hb@xt@\columnwidth {%
⟨latexrelease | fltrace⟩                                  \box\@outputbox \hss}%
⟨latexrelease | fltrace⟩                                                      }%
⟨latexrelease | fltrace⟩                                 }%
⟨*trace⟩
⟨latexrelease | fltrace⟩     \fl@trace{PAGE: second column also boxed}%
⟨/trace⟩
⟨latexrelease | fltrace⟩     \@combinedblfloats
⟨latexrelease | fltrace⟩     \@outputpage
⟨*trace⟩
⟨latexrelease | fltrace⟩     \fl@trace{PAGE: two column page completed}%
⟨/trace⟩
⟨latexrelease | fltrace⟩     \begingroup
⟨latexrelease | fltrace⟩       \@dblfloatplacement
⟨latexrelease | fltrace⟩       \@startdblcolumn
```

This loop could be replaced by an `\expandafter` tail recursion in `\@startdblcolumn`.

```
⟨latexrelease | fltrace⟩       \@whilesw\if@fcolmade \fi
⟨latexrelease | fltrace⟩         {\@outputpage
⟨*trace⟩
⟨latexrelease | fltrace⟩       \fl@trace{PAGE: double float page completed}%
⟨/trace⟩
⟨latexrelease | fltrace⟩          \@startdblcolumn}%
⟨latexrelease | fltrace⟩     \endgroup
⟨latexrelease | fltrace⟩   \fi
⟨latexrelease | fltrace⟩}%
⟨latexrelease | fltrace⟩\EndIncludeInRelease
⟨/2ekernel | fltrace | latexrelease⟩
```

2.2.4 Float placement parameters

The main purpose of this section is to ensure that all the float-placement parameters which need to be set in a class file or package have been declared. It also describes their use and sets values for them which are reasonable for typical documents using US letter or A4 sized paper.

Limits for the placement of floating objects

`\c@topnumber` This counter holds the maximum number of floats that can appear at the top of a text page or column.

```
⟨*2ekernel⟩
\newcount\c@topnumber
\setcounter{topnumber}{2}
```

(*End of definition for* `\c@topnumber`.)

`\topfraction` This macro holds the maximum proportion (as a decimal number) of a text page or column that can be occupied by floats at the top.

```
\newcommand\topfraction{.7}
```

(*End of definition for* `\topfraction`.)

`\c@bottomnumber` This counter holds the maximum number of floats that can appear at the bottom of a text page or column.

```
\newcount\c@bottomnumber
\setcounter{bottomnumber}{1}
```

(*End of definition for* `\c@bottomnumber`.)

`\bottomfraction` This macro holds the maximum proportion (as a decimal number) of a text page or column that can be occupied by floats at the bottom.

```
\newcommand\bottomfraction{.3}
```

(*End of definition for* `\bottomfraction`.)

`\c@totalnumber` This counter holds the maximum number of floats that can appear on any text page or column.

```
\newcount\c@totalnumber
\setcounter{totalnumber}{3}
```

(*End of definition for* `\c@totalnumber`.)

`\textfraction` This macro holds the minimum proportion (as a decimal number) of a text page or column that must be occupied by text.

```
\newcommand\textfraction{.2}
```

(*End of definition for* `\textfraction`.)

`\floatpagefraction` This macro holds the minimum proportion (as a decimal number) of a page or column that must be occupied by floating objects before a 'float page' is produced.

```
\newcommand\floatpagefraction{.5}
```

(*End of definition for* `\floatpagefraction`.)

`\c@dbltopnumber` This counter holds the maximum number of double-column floats that can appear on the top of a two-column text page.

```
\newcount\c@dbltopnumber
\setcounter{dbltopnumber}{2}
```

(*End of definition for* `\c@dbltopnumber`.)

`\dbltopfraction` This macro holds the maximum proportion (as a decimal number) of a two-column text page that can be occupied by double-column floats at the top.

```
\newcommand\dbltopfraction{.7}
```

(*End of definition for* `\dbltopfraction`.)

`\dblfloatpagefraction` This macro holds the minimum proportion (as a decimal number) of a page that must be occupied by double-column floating objects before a 'double-column float page' is produced.

```
\newcommand\dblfloatpagefraction{.5}
```

(*End of definition for* `\dblfloatpagefraction`.)

Floats on a text page

\floatsep
\textfloatsep
\intextsep

When a floating object is placed on a page with text, these parameters control the separation between the float and the other objects on the page. These parameters are used for both one-column mode and single-column floats in two-column mode. They are all rubber lengths.

\floatsep is the space between adjacent floats that are placed at the top or bottom of the text page or column.

\textfloatsep is the space between the main text and floats at the top or bottom of the page or column.

\intextsep is the space between in-text floats and the text.

```
\newskip\floatsep
\newskip\textfloatsep
\newskip\intextsep
\setlength\floatsep    {12\p@ \@plus 2\p@ \@minus 2\p@}
\setlength\textfloatsep{20\p@ \@plus 2\p@ \@minus 4\p@}
\setlength\intextsep   {12\p@ \@plus 2\p@ \@minus 2\p@}
```

(*End of definition for* \floatsep, \textfloatsep, *and* \intextsep.)

\dblfloatsep
\dbltextfloatsep

When double-column floats (floating objects that span the whole \textwidth) are placed at the top of a text page in two-column mode, the separation between the float and the text is controlled by \dblfloatsep and \dbltextfloatsep. They are rubber lengths.

\dblfloatsep is the space between adjacent double-column floats placed at the top of the text page.

\dbltextfloatsep is the space between the main text and double-column floats at the top of the page.

```
\newskip\dblfloatsep
\newskip\dbltextfloatsep
\setlength\dblfloatsep    {12\p@ \@plus 2\p@ \@minus 2\p@}
\setlength\dbltextfloatsep{20\p@ \@plus 2\p@ \@minus 4\p@}
```

(*End of definition for* \dblfloatsep *and* \dbltextfloatsep.)

Floats on their own page or column

\@fptop
\@fpsep
\@fpbot

When floating objects are placed on a separate page or column, called a 'float page', the layout of the page is controlled by these parameters, which are rubber lengths.

At the top of the page \@fptop is inserted; typically this supplies some stretchable whitespace. At the bottom of the page \@fpbot is inserted. Between adjacent floats \@fpsep is inserted.

These parameters are used for all floating objects on a 'float page' in one-column mode, and for single-column floats in two-column mode.

Note that at least one of the two parameters \@fptop and \@fpbot should contain a `plus ...fil` so as to fill the remaining empty space.

```
\newskip\@fptop
\newskip\@fpsep
\newskip\@fpbot
\setlength\@fptop{0\p@ \@plus 1fil}
\setlength\@fpsep{8\p@ \@plus 2fil}
\setlength\@fpbot{0\p@ \@plus 1fil}
```

(*End of definition for* \@fptop, \@fpsep, *and* \@fpbot.)

\@dblfptop
\@dblfpsep
\@dblfpbot

Double-column 'float pages' in two-column mode use similar parameters.

```
\newskip\@dblfptop
\newskip\@dblfpsep
\newskip\@dblfpbot
\setlength\@dblfptop{0\p@ \@plus 1fil}
\setlength\@dblfpsep{8\p@ \@plus 2fil}
\setlength\@dblfpbot{0\p@ \@plus 1fil}
```

(*End of definition for* `\@dblfptop`, `\@dblfpsep`, *and* `\@dblfpbot`.)

\topfigrule
\botfigrule
\dblfigrule

The macros can be used to put in rules between floats and text; whatever they insert should be vertical mode material which takes up zero space.

```
\let\topfigrule=\relax
\let\botfigrule=\relax
\let\dblfigrule=\relax
⟨/2ekernel⟩
```

(*End of definition for* `\topfigrule`, `\botfigrule`, *and* `\dblfigrule`.)

File 55
lttagging.dtx

```
1 ⟨*2ekernel | latexrelease⟩
2 \ExplSyntaxOn
```

1 General support for tagged output

\SuspendTagging
\ResumeTagging
\tag_suspend:n
\tag_resume:n

There are places in code where it is important to stop any tagging activities, e.g., when we are doing trial typesetting that it is done several times. In such a case one must tag only the final version that is actually used, otherwise tagging structures are generated which then do not end up in the PDF and confuse the mechanism. For this we have two commands that can be used in packages: `\SuspendTagging` and `\ResumeTagging` (with corresponding L3 programming layer commands). They are available as part of the LaTeX kernel, so that they can be safely used in packages whether or not tagging is requested. They all take a string argument that is used for debugging to easily identify why tagging was suspended or restarted, for example, in tabularx you find `\SuspendTagging{tabularx}`. By default these four commands do nothing.

The argument is used literally (in `\typeout` messages) without any expansion when debugging is turned on and otherwise it is not used at all. This means it is safe to write something like `\SuspendTagging{\foo}` or even `\SuspendTagging\foo` which means LaTeX has to parse only a single token instead of putting a string of characters into the argument. This means a tiny speed improvement but with many such debugging strings...

\NewTaggingSocket
\NewTaggingSocketPlug
\AssignTaggingSocketPlug

Tagging sockets are implemented as normal sockets but with a name that starts with `tagsupport/` and some special conventions how their arguments (if any) are to be interpreted. This means in principle one can use the standard socket commands, which is what we started out with.

However, providing dedicated declaration commands is more convenient and helps in keeping the interfaces clearer and simpler, e.g., `\NewTaggingSocket` not only declares the socket but also automatically sets up necessary plugs and assigns a suitable default plug when tagging is not enabled.

`\NewTaggingSocketPlug` and `\AssignTaggingSocketPlug` on the other hand are mainly syntactic sugar and do nothing more than adding the `tagsupport/` to the socket name behind the scenes.

\UseTaggingSocket
\tag_socket_use:n
\tag_socket_use:nn
\tag_socket_use:nnn

Given that we sometimes have to suspend tagging, it would be fairly inefficient to put different plugs into these sockets whenever that happens. We therefore offer `\UseTaggingSocket` which is like `\UseSocket` except that is expects a socket starting with `tagsupport/` but the socket name is specified without this prefix, i.e.,

`\UseTaggingSocket{foo}` → `\UseSocket{tagsupport/foo}`

.

Beside being slightly shorter, the big advantage is that this way we can change `\UseTaggingSocket` to do nothing by switching a boolean instead of changing the plugs of the tagging support sockets back and forth.

Usually, these sockets have (beside the default plug defined for every socket) one additional plug defined and directly assigned. This plug is used when tagging is active. There may be more plugs, e.g., tagging with special debugging or special behavior depending on the class or PDF version etc., but right now it is usually just on or off.

When tagging is suspended they all have the same predefined behavior: The sockets with zero arguments do nothing. The sockets with one argument gobble their argument. The sockets with two arguments will drop their first argument and pass the second unchanged.

It is possible to use the tagging support sockets with `\UseSocket` directly, but in this case the socket remains active if `\SuspendTagging` is in force. There may be reasons for doing that but in general we expect to always use `\UseTaggingSocket`.

`\UseExpandableTaggingSocket`
`\tag_socket_use_expandable:n`

For special cases like in some `\halign` contexts we need a fully expandable version of the command. For these cases, `\UseExpandableTaggingSocket` can be used. To allow being expandable, it does not output any debugging information if `\DebugSocketsOn` is in effect and therefore should be avoided whenever possible.

The L3 programming layer versions `\tag_socket_use_expandable:n`, `\tag_socket_use:n`, `\tag_socket_use:nn`, and `\tag_socket_use:nnn` are slightly more efficient than `\UseTaggingSocket` because they do not have to determine how many arguments the socket takes when disabling it.

`\MathCollectTrue`
`\MathCollectFalse`

The tagging of math has to collect/grab the math first. This is not wanted for all uses of `$`. These command allow to control the behavior of the math shift token. Without the math tagging code they do nothing. Their behavior with the math tagging code is documented in latex-lab-math.pdf

`\MathMLintent`
`\MathMLarg`

These two commands take two arguments. The first argument is the value of the `intent` or `arg` attribute to be added to the MathML generated by the term in the second argument. By default the commands are no-op and discard the first argument and expand to the second. If luamml is loaded via the math tagging code, then these commands are redefined. They could also potentially be used by other TEX to MathML converters to control the generated MathML.

`\NewStructureName`
`\UseStructureName`
`\AssignStructureRole`

Structure elements in a document can use as tag a name from the standard PDF namespaces like `Sect`, `H1` or `Figure` but they can also use new names (which then must be rolemapped to a standard name). The second option is useful for three reasons:

- It looks nicer, if, e.g., a bible uses tag names like `Testament` or `Chapter` or `Book` instead of `Sect`.
- It is possible to formulate additional constraint on such structures in a Schema and thus ensure that there is no `Testament` inside a `Book`, something that can not be done if `Sect` is used everywhere.
- We can provide a uniform LaTeX set of names for tags.

To make it possible to adapt the tag names of a structure in document, the tag name should be stored in a command. These three commands offer an interface to declare, use and reassign such symbolic structure names. `\NewStructureName` takes one argument and declares the internal command, initially its value is `NonStruct`. The expandable command `\UseStructureName` takes one argument and allows to use the stored value. `\AssignStructureRole` takes two arguments. The first is a symbolic structure names, the second a role which should be a simply string allowed as a tag name. When the tagging code is loaded `\AssignStructureRole` is redefined to setup the rolemapping, see the description in `latex-lab-namespace.dtx`. There is also a description of the naming scheme and a list of the already predeclared names.

2 Implementation

\tag_suspend:n
\tag_resume:n
\SuspendTagging
\ResumeTagging

In the kernel, these commands get dummy definitions so that they can be used without harm in packages. The real definition is used when tagging gets enabled.

```
\cs_new_eq:NN \tag_suspend:n \use_none:n
\cs_new_eq:NN \tag_resume:n  \use_none:n
\cs_new_protected:Npn \SuspendTagging #1 { \tag_suspend:n {#1} }
\cs_new_protected:Npn \ResumeTagging #1  { \tag_resume:n {#1} }
```

(*End of definition for* `\tag_suspend:n` *and others.*)

\NewTaggingSocket

Initialize a new tagging socket and assign it a suitable plug.

```
\cs_new_protected:Npn \NewTaggingSocket #1 #2 {
  \socket_new:nn { tagsupport / #1 } { #2 }
  \int_case:nnF
      { #2 }
         {
           0 \prg_do_nothing:
           1 { \socket_assign_plug:nn { tagsupport /  #1 } { noop } }
```

Tagging sockets with two arguments use a special `transparent` plug that just passes the second argument. Its already assigned so we only have to alter it.

```
           2 { \socket_new_plug:nnn { tagsupport / #1 } { transparent }
                                    { ##2 }
               \socket_assign_plug:nn { tagsupport /  #1 } { transparent } }
         }
         \ERRORnewtaggingsocket  % that should get a proper error message
}
```

(*End of definition for* `\NewTaggingSocket`.)

\NewTaggingSocketPlug
\AssignTaggingSocketPlug

```
\cs_new_protected:Npn \NewTaggingSocketPlug #1 {
  \NewSocketPlug { tagsupport/ #1 }
}
\cs_new_protected:Npn \AssignTaggingSocketPlug #1 {
  \AssignSocketPlug { tagsupport/ #1 }
}
```

(*End of definition for* `\NewTaggingSocketPlug` *and* `\AssignTaggingSocketPlug`.)

\tag_socket_use:n
\tag_socket_use:nn
\tag_socket_use:nnn
\tag_socket_use_expandable:n
\UseTaggingSocket
\UseExpandableTaggingSocket

Again this is not the final definition for the kernel; it is just a version to get going while some parts of the kernel support are still missing.

```
\prg_new_conditional:Npnn \tag_if_active: { p , T , TF, F }
     { \prg_return_false: }
```

Dummy definitions in the kernel. These definitions will get updated in tagpdf. The default in the kernel is simply to get rid of the first argument, while the second argument is preserved if present:

```
\cs_new:Npn \tag_socket_use_expandable:n #1 { }
\cs_new_protected:Npn \tag_socket_use:n #1 { }
\cs_new_protected:Npn \tag_socket_use:nn #1#2 { }
\cs_new_protected:Npn \tag_socket_use:nnn #1#2#3 { #3 }
\cs_new_protected:Npn \UseTaggingSocket #1 {
```

```
  \int_case:nnF
      { \int_use:c { c__socket_tagsupport/#1_args_int } }
         {
           0 \prg_do_nothing:
           1 \use_none:n
           2 \use_ii:nn
```

We do not expect tagging sockets with more than one or two arguments, so for now we only provide those.

```
         }
         \ERROR usetaggingsocket  % that should get a proper error message
}
```

The same as an expandable command:

```
\cs_new:Npn \UseExpandableTaggingSocket #1 {
  \int_case:nnF
      { \int_use:c { c__socket_tagsupport/#1_args_int } }
         {
           0 \prg_do_nothing:
           1 \use_none:n
           2 \use_ii:nn
         }
         \ERROR usetaggingsocket  % that should get a proper error message
}
```

(*End of definition for* `\tag_socket_use:n` *and others.*)

2.1 Math collection

The documentation is in latex-lab-math.

`\MathCollectTrue`
`\MathCollectFalse`

```
\cs_new_protected:Npn\MathCollectTrue{}
\cs_new_protected:Npn\MathCollectFalse{}
```

(*End of definition for* `\MathCollectTrue` *and* `\MathCollectFalse`.)

2.2 Tagging sockets

This collects tagging sockets that should be generally available so that they can also be used even if the tagging code is not loaded.

2.3 Generic sockets

These sockets are used in various places and should not be reassigned globally.

`mc` (*tag socket*) At first a generic socket to surround content with the mc-commands. The first argument can be used to pass options like `artifact`

```
\NewTaggingSocket{mc}{2}
\NewTaggingSocketPlug{mc}{kernel}
  {
    \tag_mc_begin:n {#1}
      #2
    \tag_mc_end:
  }
\AssignTaggingSocketPlug{mc}{kernel}
```

`struct-mc` (*tag socket*) A socket to surround content with a structure and an mc-command. With the first argument the tag and other options can be set.

```
\NewTaggingSocket{struct-mc}{2}
\NewTaggingSocketPlug{struct-mc}{kernel}
  {
    \tag_struct_begin:n{#1}
      \tag_mc_begin:n {}
          #2
      \tag_mc_end:
    \tag_struct_end:
  }
\AssignTaggingSocketPlug{struct-mc}{kernel}
```

`struct/begin` (*tag socket*) Sockets to start and end a structure. With the first argument the tag and other options
`struct/end` (*tag socket*) can be set.

```
\NewTaggingSocket{struct/begin}{1}
\NewTaggingSocket{struct/end}{0}
\NewTaggingSocketPlug{struct/begin}{kernel}
  {
    \tag_struct_begin:n{#1}
  }
\NewTaggingSocketPlug{struct/end}{kernel}
  {
    \tag_struct_end:
  }
\AssignTaggingSocketPlug{struct/begin}{kernel}
\AssignTaggingSocketPlug{struct/end}{kernel}
```

`inline/begin` (*tag socket*) These sockets can be used to insert small structures (often a Span with some attributes)
`inline/end` (*tag socket*) inside a paragraph: They close and reopen MC's. With the argument structure name and attributes can be passed on.

```
\NewTaggingSocket{inline/begin}{1}
\NewTaggingSocketPlug{inline/begin}{kernel}
  {
    \tag_mc_end_push:
    \tag_struct_begin:n{#1}
    \tag_mc_begin:n{}
  }
\AssignTaggingSocketPlug{inline/begin}{kernel}
\NewTaggingSocket{inline/end}{0}
\NewTaggingSocketPlug{inline/end}{kernel}
  {
    \tag_mc_end:
    \tag_struct_end:
    \tag_mc_begin_pop:n{}
  }
\AssignTaggingSocketPlug{inline/end}{kernel}
```

2.3.1 Tagging support for paragraph setup

Paragraphs are tagged through the code in the para/hooks. This code is sometimes adjusted, e.g. to produce a "flattened" paragraph or to use a different tag. Sockets related to such code parts are collected here.

\l__tag_block_flattened_level_int The block code needs to know if they are nested blockenvs inside a flattened environment. For this it uses a counter. Inside some contexts, e.g. at the begin of a minipage or a footnote this counter must be reset. We therefore define the counter here so that we can use it in the following socket.

```
\int_new:N \l__tag_block_flattened_level_int
```

(*End of definition for* `\l__tag_block_flattened_level_int`.)

`para/on` (*tag socket*) These sockets allow paragraph tagging to be enabled/disabled.
`para/off` (*tag socket*)

```
\NewTaggingSocket{para/on}{0}
\NewTaggingSocketPlug{para/on}{kernel}{\bool_set_true:N \l__tag_para_bool}
\NewTaggingSocket{para/off}{0}
\NewTaggingSocketPlug{para/off}{kernel}{\bool_set_false:N \l__tag_para_bool}
\AssignTaggingSocketPlug{para/on}{kernel}
\AssignTaggingSocketPlug{para/off}{kernel}
```

`para/restore` (*tag socket*) This socket restores the para related settings to their default. It should be used in places where "normal" paragraph tagging must be ensured, for example at the begin of a footnote.

```
\NewTaggingSocket{para/restore}{0}
```

`default` (*tag plug*)

```
\NewTaggingSocketPlug{para/restore}{default}
 {
   \AssignStructureRole{para/semantic}{\c_tag_para_semantic_tl}
   \AssignStructureRole{para/textblock}{\c_tag_para_textblock_tl}
   \bool_set_false:N\l__tag_para_flattened_bool
   \int_zero:N   \l__tag_block_flattened_level_int
   \bool_set_true:N \l__tag_para_bool
 }
\AssignTaggingSocketPlug{para/restore}{default}
```

`para/begin` (*tag socket*) These sockets were previously defined in tagpdf. There are the main sockets to handle
`para/end` (*tag socket*) the paragraph tagging when it is active (i.e., when para/on or para/restore was executed earlier).

```
\NewTaggingSocket{para/begin}{0}
\NewTaggingSocket{para/end}{0}
```

Until tagpdf is updated we need to provide the plain and block plugs

```
\NewTaggingSocketPlug{para/begin}{plain}{}
\NewTaggingSocketPlug{para/end}{plain}{}
\NewTaggingSocketPlug{para/begin}{block}{}
\NewTaggingSocketPlug{para/end}{block}{}

```

`para/semantic/begin` (*tag socket*) These sockets handle the begin and end structure. These are stored in sockets of their
`para/semantic/end` (*tag socket*) own to be able to disable them globally.

```
\NewTaggingSocket{para/semantic/begin}{1}
\NewTaggingSocket{para/semantic/end}{1}
```

(para/semantic/begin) (*tag plug*) This tagging socket adds a structure. The argument allows to add additional commands like the command that puts the structure number of this command on a stack.

```
\tl_new:N \l__tag_para_attr_class_tl
\NewTaggingSocketPlug{para/semantic/begin}{kernel}
 {
    \__tag_gincr_para_main_begin_int:
    \tag_struct_begin:n
      {
        tag=\UseStructureName{para/semantic},
        attribute-class=\l__tag_para_main_attr_class_tl,
      }
    #1
 }
```

l (para/semantic/end) (*tag plug*) This socket should be used if a -structure is closed. The argument allows e.g. to add debug info.

```
\NewTaggingSocketPlug{para/semantic/end}{kernel}
 {
   #1
   \__tag_gincr_para_main_end_int:
   \tag_struct_end:
 }
```

And now we assign these plugs:

```
\AssignTaggingSocketPlug{para/semantic/begin}{kernel}
\AssignTaggingSocketPlug{para/semantic/end}{kernel}
```

ara/textblock/begin (*tag socket*) These sockets handle the begin and end structure. These are stored in sockets of their para/textblock/end (*tag socket*) own to be able to reuse them in places where only a textblock is needed.

```
\NewTaggingSocket{para/textblock/begin}{1}
\NewTaggingSocket{para/textblock/end}{0}
```

para/textblock/begin) (*tag plug*) This tagging socket opens a structure and mc. It is used below in the para/begin tagging socket. The argument is used to adapt the debugging label.

```
\NewTaggingSocketPlug{para/textblock/begin}{kernel}
 {
   \__tag_gincr_para_begin_int:
   \__tag_check_typeout_v:n {==>~increment~ P \on@line }
   \tag_struct_begin:n
     {
       tag=\UseStructureName{para/textblock}
       ,attribute-class=\l__tag_para_attr_class_tl
     }
     \__tag_check_para_begin_show:nn {green}{#1}
     \tag_mc_begin:n {}
 }
```

(para/textblock/end) (*tag plug*) This socket closes a -mc and structure. It is used below in the para/end tagging socket.

TEX automatically removes a final space at the end of the paragraph text but this internalfails if we add the tagging code after that final space, so we have to remove any space manually before adding that code.

```
\NewTaggingSocketPlug{para/textblock/end}{kernel}
```

```
{
  \unskip
   \__tag_gincr_para_end_int:
   \__tag_check_typeout_v:n {==>~increment~ /P \on@line }
   \tag_mc_end:
   \__tag_check_para_end_show:nn {red}{}
   \tag_struct_end:
}
```

And now we assign these plugs:

```
\AssignTaggingSocketPlug{para/textblock/begin}{kernel}
\AssignTaggingSocketPlug{para/textblock/end}{kernel}
```

`kernel (para/begin)` (*tag plug*) This plug is similar to the block socket currently defined in tagpdf and should overwrite it.

```
\NewTaggingSocketPlug{para/begin}{kernel}
  {
    \bool_if:NT \l__tag_para_bool
      {
        \legacy_if:nF { @inlabel }
          {
            \__tag_check_typeout_v:n
              {==>~ @endpe = \legacy_if:nTF { @endpe }{true}{false} \on@line }
            \legacy_if:nF { @endpe }
              {
                \bool_if:NF \l__tag_para_flattened_bool
                   {
                     \UseTaggingSocket{para/semantic/begin}
                      { }
                   }
              }
            \UseTaggingSocket{para/textblock/begin}{NP-}
       }
     }
  }
```

`kernel (para/end)` (*tag plug*) This socket is used at the end of paragraphs.

```
\NewTaggingSocketPlug{para/end}{kernel}
  {
    \bool_if:NT \l__tag_para_bool
      {
        \UseTaggingSocket{para/textblock/end}
        \bool_if:NF \l__tag_para_flattened_bool
         {
           \UseTaggingSocket{para/semantic/end}{}
         }
      }
  }
```

As tagpdf is currently assigning the sockets after the block code we assign if even later (for now):

```
\AddToHook{package/latex-lab-testphase-sec/after}
  {
```

```
    \AssignTaggingSocketPlug{para/begin}{kernel}
    \AssignTaggingSocketPlug{para/end}{kernel}
  }
```

2.3.2 Tagging socket for targets

`refstepcounter` (*tag socket*) When tagging is active we want to track the current structure number when targets are set. This will be mostly used in `\refstepcounter` but also if targets are set manually.

```
\NewTaggingSocket{recordtarget}{0}
```

`kernel (recordtarget)` (*tag plug*)

```
%
\NewTaggingSocketPlug{recordtarget}{kernel}
  {
    \tag_struct_recordtarget:
  }
\AssignTaggingSocketPlug{recordtarget}{kernel}
\ExplSyntaxOff
```

2.3.3 Tagging Sockets for boxes

```
\NewTaggingSocket{minipage/before}{0}
\NewTaggingSocket{minipage/after}{0}
\NewTaggingSocket{parbox/before}{0}
\NewTaggingSocket{parbox/after}{0}
```

2.3.4 Tagging Sockets for lists and blocks

`block/list/label` (*tag socket*) A tagging socket around the label in a list.

```
\NewTaggingSocket{block/list/label}{2}
```

`block/recipe` (*tag socket*) A tagging socket to set the tagging recipe.

```
\NewTaggingSocket{block/recipe}{1}
```

2.3.5 Tagging sockets for headings

Tagging headings is quite tricky. The sockets (and plugs) used by the kernel definitions often expect specially formatted arguments. For this reason some of them may not be suitable in other classes or packages. In that case additional sockets (or plugs) might be required.

`sec/begin` (*tag socket*)
`sec/end` (*tag socket*) These two sockets should surround the whole section by a `Sect` structure. They should be suited also for classes and packages. The begin socket takes two brace groups are argument: the level and key for the structure (typically `tag=somename`), the end socket takes as argument a level.

```
\NewTaggingSocket{sec/begin}{1}
\NewTaggingSocket{sec/end}{1}
```

`sec/title/begin` (*tag socket*)
`sec/title/end` (*tag socket*) These two sockets are used around display heading where the number is on a line of its own. The argument of the begin socket consists of two brace groups containing the level and the title.

```
\NewTaggingSocket{sec/title/begin}{1}
\NewTaggingSocket{sec/title/end}{0}
```

sec/title/hang (*tag socket*) This socket is used in headings with hanging number.

```
\NewTaggingSocket{sec/title/hang}{2}
```

sec/title/init (*tag socket*)
sec/title/split (*tag socket*)
/title/runin/number (*tag socket*)

These sockets are used by run-in heading to initialize tagging and to split heading and following text and tag the title.

```
\NewTaggingSocket{sec/title/init}{1}
\NewTaggingSocket{sec/title/split}{0}
\NewTaggingSocket{sec/title/runin/number}{2}
```

sec/title/number (*tag socket*) This is a socket to tag the number. It takes two arguments: the level and the content.

```
\NewTaggingSocket{sec/title/number}{2}
```

2.3.6 Tagging sockets for toc

contentsline/before (*tag socket*)
/contentsline/after (*tag socket*)

Tagging sockets at the begin and end of contentsline. They receive *all* contentsline arguments as one argument in four brace groups. The socket code should then use the parts it needs.

```
\NewTaggingSocket{toc/contentsline/before}{1}
\NewTaggingSocket{toc/contentsline/after}{1}
```

toc/starttoc/before (*tag socket*)
toc/starttoc/after (*tag socket*)

Tagging sockets for the begin and end of start of `\@starttoc`. They take one argument, the extension.

```
\NewTaggingSocket{toc/starttoc/before}{1}
\NewTaggingSocket{toc/starttoc/after}{1}
```

toc/leaders/before (*tag socket*)
toc/leaders/after (*tag socket*)

Tagging sockets to make the dot leaders an artifact. They do not take an argument.

```
\NewTaggingSocket{toc/leaders/before}{0}
\NewTaggingSocket{toc/leaders/after}{0}
```

2.3.7 Tagging support for marginpar

marginpar/begin (*tag socket*)
marginpar/end (*tag socket*)

```
\NewTaggingSocket{marginpar/begin}{0}
\NewTaggingSocket{marginpar/end}{0}
```

2.3.8 Tagging support for table/tabular packages

The code uses a number of sockets to inject the tagging commands. These can be easily set to a noop-plug in case the automated tagging is not wanted.

tbl/cell/begin (*tag socket*)
tbl/cell/end (*tag socket*)
tbl/pcell/end (*tag socket*)
tbl/pcell/end (*tag socket*)
tbl/row/begin (*tag socket*)
tbl/row/end (*tag socket*)

At first sockets for the begin and end of cells and table rows:

```
\NewTaggingSocket{tbl/cell/begin}{0}
\NewTaggingSocket{tbl/cell/end}{0}
\NewTaggingSocket{tbl/row/begin}{0}
\NewTaggingSocket{tbl/row/end}{0}
```

Multi-line cells have their own sockets (as they start out in vertical mode and need different treatment).

```
\NewTaggingSocket{tbl/pcell/begin}{0}
\NewTaggingSocket{tbl/pcell/end}{0}
```

`tbl/init` (*tag socket*) This socket should be at the begin of the table, inside a group. It is used for settings such as disabling para-tagging inside the table. This socket can perhaps be merged later into the begin-sockets.

```
\NewTaggingSocket{tbl/init}{0}
```

`tbl/init/celldata` (*tag socket*) This socket is used in `\tbl_init_cell_data_for_table`, the command that stores and initialize cell data to handle nested tables. It can be used to restore similar tagging related values

```
\NewTaggingSocket{tbl/init/celldata}{0}
```

`tbl/finalize` (*tag socket*) To fine tune the structure (change cells to header cells, remove unwanted structures, move a foot to the end, etc.). We also need a socket that is executed at the end of the table but *before* all the variables are restored to the outer or default values. The code in the socket can make assignments, but probably shouldn't do typesetting and not write whatsits.

```
\NewTaggingSocket{tbl/finalize}{0}
```

`bl/restore/celldata` (*tag socket*) This socket is used in `\tbl_restore_outer_cell_data:`, the command that restores cell data when quitting a nested table. It can be used to restore similar tagging related values

```
\NewTaggingSocket{tbl/restore/celldata}{0}
```

`tbl/colspan` (*tag socket*) This socket is used to manage spanning cells, e.g., a `\multicolumn`. It expects one argument (the number of cells spanned) and if tagging is enabled set appropriate tag attributes in the background. We probably need a similar socket for row spans eventually.

```
\NewTaggingSocket{tbl/colspan}{1}
```

`tbl/hmode/begin` (*tag socket*)
`tbl/hmode/end` (*tag socket*)
`tbl/vmode/begin` (*tag socket*)
`tbl/vmode/end` (*tag socket*) These sockets are used in the begin and end code of environments, to allow a fast enabling and disabling of the tagging. We distinguish between tables that can be used inside paragraphs and standalone tables such as `longtable` that are always in vertical mode.

```
\NewTaggingSocket{tbl/hmode/begin}{0}
\NewTaggingSocket{tbl/hmode/end}{0}
\NewTaggingSocket{tbl/vmode/begin}{0}
\NewTaggingSocket{tbl/vmode/end}{0}
```

`tbl/longtable/init` (*tag socket*)
`/longtable/finalize` (*tag socket*) `longtable` needs its own sockets to fine tune the structure. Simply switching the plug in the previous socket interferes with enabling/disabling the tagging.

```
\NewTaggingSocket{tbl/longtable/init}{0}
\NewTaggingSocket{tbl/longtable/finalize}{0}
```

`tbl/longtable/head` (*tag socket*)
`tbl/longtable/foot` (*tag socket*) Header and footer boxes need special handling because they are repeatedly used.

```
\NewTaggingSocket{tbl/longtable/head}{0}
\NewTaggingSocket{tbl/longtable/foot}{0}
```

`tbl/leaders/begin` (*tag socket*)
`tbl/leaders/end` (*tag socket*) Sockets around leaders such as rules or dotted lines, that should be tagged as artifacts, used, for example, in `\cline`.

```
\NewTaggingSocket{tbl/leaders/begin}{0}
\NewTaggingSocket{tbl/leaders/end}{0}
```

2.3.9 Tagging Support for floats

`float/hmode/begin` (*tag socket*)
`float/hmode/end` (*tag socket*)

These sockets are used if the float is called in hmode.

```
\NewTaggingSocket{float/hmode/begin}{0}
\NewTaggingSocket{float/hmode/end}{0}
```

`float/begin` (*tag socket*)
`float/end` (*tag socket*)

These sockets start and stop the float structure.

```
\NewTaggingSocket{float/begin}{0}
\NewTaggingSocket{float/end}{0}
```

`caption/begin` (*tag socket*)
`caption/end` (*tag socket*)

These sockets are used in `\@makecaption`. They open and close the `Caption` structure. Their default plugs assume that they are used in vmode. The argument of the begin socket is the structure number of the parent float. If it is empty the current structure number is used.

```
\NewTaggingSocket{caption/begin}{1}
\NewTaggingSocket{caption/end}{0}
```

`caption/label/begin` (*tag socket*)
`caption/label/end` (*tag socket*)

These sockets are used in `\@makecaption` around the label. Their default plugs ensure that the label is outside the paragraph and that the rest of the caption uses flattened para mode. If the caption is not in a hbox, the `para/begin` socket should follow to properly start the paragraph.

```
\NewTaggingSocket{caption/label/begin}{0}
\NewTaggingSocket{caption/label/end}{0}
```

2.4 Tagging support for output routines

`build/page/header` (*tag socket*)
`build/page/footer` (*tag socket*)

These sockets receive the formatted running header/footer in its second argument (the first is not used) following the convention of tagging sockets, i.e., only the second argument is processed if tagging is not active.

```
\NewTaggingSocket{build/page/header}{2}
\NewTaggingSocket{build/page/footer}{2}
```

`ld/column/outputbox` (*tag socket*)

This socket is used to add any missing tagging structures to the `\@outputbox` box, if necessary.

```
\NewTaggingSocket{build/column/outputbox}{0}
```

`uild/column/footins` (*tag socket*)

This socket is used to add any missing tagging structures to the `\footins` box, if necessary.

```
\NewTaggingSocket{build/column/footins}{0}
```

`page@sofar` (*tag socket*)

This socket is declared and used in the multicol output routines. Only listed for reference.

```
%\NewTaggingSocket{page@sofar}{0}
```

2.5 Tagging support for math

2.5.1 General sockets

The following sockets are the main math sockets.

```
\NewTaggingSocket{math/inline/begin}{0}
\NewTaggingSocket{math/inline/end}{0}
\NewTaggingSocket{math/inline/formula/begin}{2}
\NewTaggingSocket{math/inline/formula/end}{0}
\NewTaggingSocket{math/display/begin}{0}
\NewTaggingSocket{math/display/end}{0}
\NewTaggingSocket{math/display/formula/begin}{2}
\NewTaggingSocket{math/display/formula/end}{0}
\NewTaggingSocket{math/display/tag/begin}{0}
\NewTaggingSocket{math/display/tag/end}{0}
```

2.5.2 Sockets specific for luamml

Save sockets These sockets are wrappers around the `\luamml_save:...` commands. They take an argument which should contain the argument of the save command.

`math/luamml/save/nn` (*tag socket*) The argument should contain the two arguments of the command.

```
\NewTaggingSocket{math/luamml/save/nn}{1}
```

`ath/luamml/save/nNn` (*tag socket*) The argument should contain the three arguments of the command.

```
\NewTaggingSocket{math/luamml/save/nNn}{1}
```

Socket to annotate

`amml/annotate/false` (*tag socket*) These socket can be used for content that should be annotated with `core=false`

```
\NewTaggingSocket{math/luamml/annotate/false}{2}
```

Array sockets These sockets will be used in array to add luamml support to the array environment.

`h/luamml/array/save` (*tag socket*) This socket will be used in `\endarray`. The plug is set by luamml.

```
\NewTaggingSocket{math/luamml/array/save}{0}
```

`amml/array/finalize` (*tag socket*) This socket will be used in `\endarray`. The plug is set by luamml.

```
\NewTaggingSocket{math/luamml/array/finalize}{0}
```

`uamml/array/initcol` (*tag socket*) This socket will be used in `\@classz`. The plug is set by luamml.

```
\NewTaggingSocket{math/luamml/array/initcol}{0}
```

`l/array/finalizecol` (*tag socket*) This socket will be used in `\@classz`. The plug is set by luamml. The argument sets the type of the column.

```
\NewTaggingSocket{math/luamml/array/finalizecol}{1}
```

Alignment environments Multiline environments like `align`, `multline` or `gather` are tagged as `mtable`.

`/mtable/finalizecol` (*tag socket*) This sockets is used at the end of alignment cells and adds them to the row. The argument passes a type like `last` or `box`.

```
\NewTaggingSocket{math/luamml/mtable/finalizecol}{1}
```

`mml/mtable/finalize` (*tag socket*) This sockets is used at the end of alignment environment to finalize the mtable code. It should be used normally with `\UseExpandableTaggingSocket`.

```
\NewTaggingSocket{math/luamml/mtable/finalize}{1}
```

`mml/mtable/aligncol` (*tag socket*) This sockets is used in multline to add attributes describing the alignment to the left and right. It takes an argument, the alignment.

```
\NewTaggingSocket{math/luamml/mtable/aligncol}{1}
```

`ble/innertable/save` (*tag socket*) This socket is used in `\endaligned` to save the table. It takes no argument.

```
\NewTaggingSocket{math/luamml/mtable/innertable/save}{0}
```

`le/smallmatrix/save` (*tag socket*) This socket is used in `\endsmallmatrix` to save the table. It takes no argument. TODO: Check if this socket and the innertable socket can/should be merged into a more generic version.

```
\NewTaggingSocket{math/luamml/mtable/smallmatrix/save}{0}
```

`innertable/finalize` (*tag socket*) This socket is used e.g. in `\endsmallmatrix` and `\gathered` to finalize the table. It takes no argument.

```
\NewTaggingSocket{math/luamml/mtable/innertable/finalize}{0}
```

`mml/mtable/tag/save` (*tag socket*) This socket is used to save a tag for later use. has been save before.

```
\NewTaggingSocket{math/luamml/mtable/tag/save}{0}
```

`amml/mtable/tag/set` (*tag socket*) This socket should be used when a tag is placed. It inserts a tag that has been save before.

```
\NewTaggingSocket{math/luamml/mtable/tag/set}{0}
```

mbox socket

`math/luamml/hbox` (*tag socket*) This socket is used around `\hbox` inside an `\mbox`, `\makebox` and `\text` and annotates the content if the `\hbox` is used inside math. The real plug is set by luamml but a default plug is defined here so that the socket can also be used if luamml is not used.

```
\NewTaggingSocket{math/luamml/hbox}{2}
```

math phantom sockets

`ath/luamml/finph@nt` (*tag socket*) This socket handles the annotation of `\finph@nt`

```
\NewTaggingSocket{math/luamml/finph@nt}{2}
```

`ath/luamml/finph@nt` (*tag socket*) This socket handles the annotation of `\finsm@sh`

```
\NewTaggingSocket{math/luamml/finsm@sh}{2}
```

Artifact root sign

ath/luamml/artifact (*tag socket*) Unicode characters like a root sign should be marked as artifacts to avoid duplication, e.g., in derivation if mathml structure elements are used that imply the meaning.

```
\NewTaggingSocket{math/luamml/artifact}{0}
```

2.6 MathML intent attributes

\MathMLintent
\MathMLarg

Stub definitions here to allow these commands to be used in packages whether or not tagging is enabled.

```
\ExplSyntaxOn
\cs_new_protected:Npn\MathMLintent#1#2{#2}
\cs_new_protected:Npn\MathMLarg#1#2{#2}
\ExplSyntaxOff
```

(*End of definition for* `\MathMLintent` *and* `\MathMLarg`.)

2.7 Symbolic structure names

\NewStructureName
\UseStructureName
\AssignStructureRole

Stub definitions here to allow these commands to be used in packages whether or not the tagging support code is loaded.

```
⟨@@=tag⟩
\ExplSyntaxOn
\cs_new_protected:Npn\NewStructureName#1
 {
   \tl_new:c  {l__tag_name_#1_tl}
   \tl_set:cn {l__tag_name_#1_tl}{NonStruct}
 }
\cs_new:Npn\UseStructureName#1
 {
   \cs:w l__tag_name_#1_tl\cs_end:
 }
\cs_new_protected:Npn\AssignStructureRole#1#2
 {
   \tl_set:cn { l__tag_name_#1_tl }{#2}
 }
\ExplSyntaxOff
```

(*End of definition for* `\NewStructureName`, `\UseStructureName`, *and* `\AssignStructureRole`.)

3 For lttab.dtx parked here for now

```
⟨@@=tbl⟩
\ExplSyntaxOn
```

3.1 Variables for row, column and span counting

This part needs a decision on names for various integer registers as well as a decision if those should be also made available for LaTeX 2$_\varepsilon$-style packages in form of 2e names and or as non-internals for the L3 programming layer.

At the moment they are all internal but this probably has to change.

\g__tbl_col_int
\g__tbl_row_int
\g__tbl_span_tl
\g__tbl_table_cols_tl

\g__tbl_row_int holds the current row number in the table. The value 0 means we haven't yet processed the table preamble (or in case of longtable are just in front of the next chunk to be processed). It is incremented by every \cr including the one ending the table preamble.

TODO: due to the gymnastics needed inside the longtable code the row counter is directly exposed there rather than hidden by interfaces. This needs changing when it is decided how to manage these counters.

\g__tbl_col_int holds the current column number. The value 0 means we have not yet started the table or just finished a table row (with \\ typically); any other positive value means we are currently typesetting a cell in that column in some row (denoted by the \g__tbl_row_int).

In a \multicolumn it holds the column number of the first spanned column and \g__tbl_span_tl the info how many cells are spanned.

\g__tbl_span_tl is normally 1 except in a \multicolumn cell.

```
\int_new:N \g__tbl_col_int
\int_new:N \g__tbl_row_int
\tl_new:N  \g__tbl_span_tl
\tl_new:N  \g__tbl_table_cols_tl

\tl_gset:Nn \g__tbl_span_tl {1}
\tl_gset:Nn \g__tbl_table_cols_tl {0}  % indicates outer level
```

(*End of definition for* \g__tbl_col_int *and others.*)

\l__tbl_saved_col_tl
\l__tbl_saved_row_tl
\l__tbl_saved_span_tl
\l__tbl_saved_table_cols_tl

Saving the outer values if we are nesting tables is necessary (as the above variables are globally altered). For this we always use token lists because they don't change and we do not need to blow additional integer registers.

```
\tl_new:N \l__tbl_saved_col_tl
\tl_new:N \l__tbl_saved_row_tl
\tl_new:N \l__tbl_saved_span_tl
\tl_new:N \l__tbl_saved_table_cols_tl

\tl_set:Nn \l__tbl_saved_col_tl{0}
\tl_set:Nn \l__tbl_saved_row_tl{0}
\tl_set:Nn \l__tbl_saved_span_tl{1}
\tl_set:Nn \l__tbl_saved_table_cols_tl{0}  % indicates outer level
```

(*End of definition for* \l__tbl_saved_col_tl *and others.*)

\g__tbl_missingcells_int

This will contain the number of missing cells in a row:

```
\int_new:N \g__tbl_missing_cells_int
```

(*End of definition for* \g__tbl_missingcells_int.)

3.2 Tracing/debugging

\DebugTablesOn
\DebugTablesOff

```
\def\DebugTablesOn{
  \cs_set_eq:NN \__tbl_trace:n \typeout
}
\def\DebugTablesOff{
  \cs_set_eq:NN \__tbl_trace:n \use_none:n
}
```

```
\cs_new_eq:NN \__tbl_trace:n \use_none:n
```

(*End of definition for* `\DebugTablesOn` *and* `\DebugTablesOff`.)

3.3 Interface commands

All interface commands for the cell number determination have to be public on some level because they are needed in other packages as well, e.g., longtable. We may or may not also want to provide 2e style names for them.

`\tbl_update_cell_data:` Updating cell data in columns after the first means we have to increment the `\g__tbl_col_int` by the span count of the previous cell (in case it was a `\multicolumn`) and then reset the `\g__tbl_span_tl` to one (as the default).

```
\cs_new_protected:Npn \tbl_update_cell_data: {
        \int_gadd:Nn \g__tbl_col_int { \g__tbl_span_tl }
        \tl_gset:Nn  \g__tbl_span_tl {1}
}
```

(*End of definition for* `\tbl_update_cell_data:`.)

`\tbl_count_table_cols:` Current implementation of `\@mkpream` uses the scratch counter `\count@` to keep track of the number of toks registers it needs (2 per column), but this can't be used as it counts also insertions made with `!{}` and `@{}`. So similar as does longtable for `\LT@cols` we count the numbers of ampersands instead.

```
\cs_new:Npn \tbl_count_table_cols:  {
  \seq_set_split:NnV\l__tbl_tmpa_seq {&}\@preamble
  \tl_gset:Ne \g__tbl_table_cols_tl { \seq_count:N \l__tbl_tmpa_seq }
  \__tbl_trace:n { ==>~ Table~ has~ \g__tbl_table_cols_tl \space columns }
}
```

(*End of definition for* `\tbl_count_table_cols:`.)

`\l__tbl_tmpa_seq`

```
\seq_new:N \l__tbl_tmpa_seq
```

(*End of definition for* `\l__tbl_tmpa_seq`.)

`\tbl_count_missing_cells:n` We might have the situation that some table package has not implemented the `\tbl_count_table_cols:` in which case `\g__tbl_table_cols_tl` would always be zero and we would get an error below when we try to determine the missing cells, so bypass that calculation if we aren't doing tagging (there the packages should have the proper code added). Recall that this is code, that is called by `\\` and an old table package might rely on whatever the LaTeX kernel offers here.

```
\cs_new:Npn \tbl_count_missing_cells:n #1 {
  \tag_if_active:T {
    \int_compare:nNnT \g__tbl_col_int > 0
      {
        \int_gset:Nn \g__tbl_missing_cells_int
            {
              \g__tbl_table_cols_tl
            - \g__tbl_col_int
            - \g__tbl_span_tl
            + 1
            }
```

```
        \int_compare:nNnT \g__tbl_missing_cells_int < 0 \ERRORmissingcells % should not happe
        \__tbl_trace:n{==>~
          (#1)~
          This~ row~ needs~
          \int_use:N \g__tbl_missing_cells_int \space
          additional~ cell(s)
        }
      }
  }
}
```

(*End of definition for* `\tbl_count_missing_cells:n`.)

`\tbl_init_cell_data_for_table:`

```
\cs_new_protected:Npn \tbl_init_cell_data_for_table: {
  \tl_set:No \l__tbl_saved_col_tl {\int_use:N \g__tbl_col_int }
  \tl_set:No \l__tbl_saved_row_tl {\int_use:N \g__tbl_row_int }
  \tl_set_eq:NN \l__tbl_saved_table_cols_tl  \g__tbl_table_cols_tl
  \tl_set_eq:NN \l__tbl_saved_span_tl  \g__tbl_span_tl
%
  \__tbl_trace:n { ==>~ saved~cell~data:~
                \l__tbl_saved_row_tl,
                \l__tbl_saved_col_tl,
                \l__tbl_saved_span_tl \space
                (
                \int_compare:nNnTF \l__tbl_saved_table_cols_tl = 0
                    { outer~ level }
                    { max:~ \l__tbl_saved_table_cols_tl }
                )
           }
```

Tagging has to initialize cell data too.

```
  \UseTaggingSocket{tbl/init/celldata}
```

These are the initial values when starting a table:

```
  \int_gzero:N \g__tbl_row_int
  \int_gzero:N \g__tbl_col_int
  \tl_gset:Nn  \g__tbl_span_tl {1}
}
```

(*End of definition for* `\tbl_init_cell_data_for_table:`.)

`\tbl_update_cell_data_for_next_row:`

```
\cs_new_protected:Npn \tbl_update_cell_data_for_next_row: {
      \int_gincr:N \g__tbl_row_int          % this row about to start
      \int_gzero:N \g__tbl_col_int          % we are before first col
}
```

(*End of definition for* `\tbl_update_cell_data_for_next_row:`.)

`\tbl_init_cell_data_for_row:` If we start processing a cell in the first column we set `\g__tbl_col_int` to `1` as we are no longer "at" but "in" the first column. We also set `\g__tbl_span_tl` to its default value (not spanning cells).

```
\cs_new_protected:Npn \tbl_init_cell_data_for_row: {
        \int_gset:Nn \g__tbl_col_int {1}
        \tl_gset:Nn  \g__tbl_span_tl {1}
}
```

(*End of definition for* `\tbl_init_cell_data_for_row:`.)

`\tbl_if_row_was_started:T`
`\tbl_if_row_was_started:TF`

We use `\g__tbl_col_int` equal zero to indicate that we are just after a TR (i.e.n between rows or at the very beginning of the table). Using the row count is not so good as longtable may split the table in chunks.

These conditionals have to be expandable (i.e., unprotected) as they are sometimes executed when TEX is scanning inside a table.

```
\cs_new:Npn \tbl_if_row_was_started:T {
      \int_compare:nNnT \g__tbl_col_int > 0
}
\cs_new:Npn \tbl_if_row_was_started:TF {
      \int_compare:nNnTF \g__tbl_col_int > 0
}
```

(*End of definition for* `\tbl_if_row_was_started:T` *and* `\tbl_if_row_was_started:TF`.)

`\tbl_gzero_row_count:`
`\tbl_gincr_row_count:`
`\tbl_gdecr_row_count:`

This here is basically a temporary interface. What it will be in the end depends on what we decide concerning exposing row and column counters, if they stay internal we need something like this here (perhaps using `gincr` etc, or perhaps some other names in the first place).

```
\cs_new_protected:Npn \tbl_gzero_row_count: {
  \int_gzero:N \g__tbl_row_int
}
\cs_new_protected:Npn \tbl_gincr_row_count: {
  \int_gincr:N \g__tbl_row_int
}
\cs_new_protected:Npn \tbl_gdecr_row_count: {
  \int_gdecr:N \g__tbl_row_int
}
```

(*End of definition for* `\tbl_gzero_row_count:` , `\tbl_gincr_row_count:` , *and* `\tbl_gdecr_row_count:`.)

`\tbl_inbetween_rows:`

Again name is not really brilliant so far.

```
\cs_new_protected:Npn \tbl_inbetween_rows: {
       \int_gzero:N  \g__tbl_col_int
}
```

(*End of definition for* `\tbl_inbetween_rows:`.)

`\tbl_restore_outer_cell_data:`

```
\cs_new_protected:Npn \tbl_restore_outer_cell_data: {
  \int_gset:Nn \g__tbl_col_int { \l__tbl_saved_col_tl }
  \int_gset:Nn \g__tbl_row_int { \l__tbl_saved_row_tl }
  \tl_gset_eq:NN \g__tbl_span_tl \l__tbl_saved_span_tl
  \tl_gset_eq:NN \g__tbl_table_cols_tl   \l__tbl_saved_table_cols_tl
  \UseTaggingSocket{tbl/restore/celldata}
  \__tbl_trace:n { ==>~ restored~cell~data:~
                \int_use:N \g__tbl_row_int,
                \int_use:N \g__tbl_col_int,
                \l__tbl_saved_span_tl \space
                (
                \int_compare:nNnTF \g__tbl_table_cols_tl = 0
                    { outer~ level }
                    { max:~ \g__tbl_table_cols_tl }
```

```
                  )
                }
}
```

(*End of definition for* `\tbl_restore_outer_cell_data:`.)

`\tbl_update_multicolumn_cell_data:n` This macro updates `\g__tbl_col_int` and `\g__tbl_span_tl` inside a `\multicolumn` and possibly calls the tagging socket `tbl/row/begin`.

```
\cs_new_protected:Npn \tbl_update_multicolumn_cell_data:n #1 {
```

We execute socket for tagging only if this `\multicolumn` replaces the preamble of the first column. In that case we also have to set `\g__tbl_col_int` to 1 because this is no longer done in the preamble for the cell either.

```
    \int_compare:nNnTF \g__tbl_col_int = 0
        {
          \UseTaggingSocket{tbl/row/begin}
          \int_gset:Nn \g__tbl_col_int {1}
        }
```

If we are in a later column we use `\g__tbl_span_tl` from the previous column to update.

```
        {
          \int_gadd:Nn \g__tbl_col_int { \g__tbl_span_tl }
        }
```

Then we set the span value so that it can be use in the next column.

```
    \tl_gset:Nn \g__tbl_span_tl {#1}
}
```

(*End of definition for* `\tbl_update_multicolumn_cell_data:n`.)

`\tbl_crcr:n` This macro is used instead of the usual `\crcr` at the end of a table. It is deliberately defined without protection because it may get expanded by the scanning mechanism of low-level TEX after a final `\cr` (aka `\\`) in the table. In that case it shouldn't stop the expansion and the conditional inside will be false, thus it just vanishes without doing anything. If there are missing cells (in which case we also haven't see `\cr` yet) the macro `\tbl_count_missing_cells:n` is executed and then the row is finished with a final `\cr`.

```
\cs_new:Npn \tbl_crcr:n #1 {
    \int_compare:nNnT \g__tbl_col_int > 0
        {
          \tbl_count_missing_cells:n {#1}
        }
```

Even if we are at the start of a row we my have to do a `\cr`, so we do a `\crcr` always at the end.

```
    \crcr
}
```

(*End of definition for* `\tbl_crcr:n`.)

```
\ExplSyntaxOff
⟨@@=⟩
```

This is needed for longtable because `\refstepcounter` is setting up a target when hyperref is loaded and we don't want that in longtable. Prevent longtable patching by hyperref until hyperref does so automatically:

```
\def\hyper@nopatch@longtable{}
```

Should there be a module?

```
462 ⟨latexrelease⟩\NewModuleRelease{2024/06/01}{lttagging}
463 ⟨latexrelease⟩                  {Tagging support}
464 ⟨latexrelease⟩\IncludeInRelease{0000/00/00}{lttagging}%
465 ⟨latexrelease⟩                  {Undo tagging support}
466 ⟨latexrelease⟩
467 ⟨latexrelease⟩
468 ⟨latexrelease⟩
469 ⟨latexrelease⟩\EndModuleRelease
470 ⟨/2ekernel | latexrelease⟩
```

File 56
lthyphen.dtx

This file contains the code for loading hyphenation patterns into LaTeX. Most of this will end up in a file called `hyphen.ltx`. If you wish to customize your LaTeX system in respect of hyphenation patterns, write a file `hyphen.cfg`. If this file exists, it will be loaded instead of `hyphen.ltx`. See the comments below for additional information.

To produce the printed version of this file the following code is used. It can be extracted with the DOCSTRIP program, or one can run this file directly through LaTeX 2_ε.

```
⟨*driver⟩
\documentclass{ltxdoc}
\begin{document}
\DocInput{lthyphen.dtx}
\end{document}
⟨/driver⟩
```

The default file `hyphen.ltx` loads hyphenation patterns for US english. If you want to load additional or other hyphenation patterns, you should create a file `hyphen.cfg`. This is best done by starting from `hyphen.ltx`.

For backward compatibility, the default file, `hyphen.ltx`, first tries to load the file `hyphen.tex`. If this file exists, an information message is issued and the appropriate defaults for TeX's internal parameters are set: `\language` is initialized to 0, and `\lefthyphenmin` and `\righthyphenmin` to 2 and 3, respectively, to disallow x- or -xx breaks.

```
⟨*default⟩
\InputIfFileExists{hyphen.tex}%
   {\message{Loading hyphenation patterns for US english.}%
    \language=0
    \lefthyphenmin=2 \righthyphenmin=3 }%
```

Otherwise, since we cannot do anything without any hyphenation patterns, an error message is printed and the IniTeX run is terminated by invoking `\@@end` (which is the LaTeX 2_ε name for TeX's `\end` primitive).

```
   {\errhelp{The configuration for hyphenation is incorrectly
             installed.^^J%
             If you don't understand this error message you need
             to seek^^Jexpert advice.}%
    \errmessage{OOPS! I can't find any hyphenation patterns for
                US english.^^J \space Think of getting some or the
                latex2e setup will never succeed}\@@end}
⟨/default⟩
```

The following example describes the possible contents of a file `hyphen.cfg` that will load both US English and German hyphenation patterns, making the former the default. It sets `\language` to 0 for the US patterns and to 1 for the German patterns. Then `\language` is set to 0 to make this the default and the default values of `\lefthyphenmin` and `\righthyphenmin` are set.

```
language=0
input hyphen % (or \input ushyphen1 if the file has been renamed)
language=1
input ghyph31
```

```
language=0
lefthyphenmin=2
righthyphenmin=3
endinput
```

Another possibility is to use the package `babel`, by Johannes Braams. That package is distributed with a suitable `hyphen.cfg` file.

File 57
ltfinal.dtx

1 Final settings

This section contains the final settings for LaTeX. It initializes some debugging and typesetting parameters, sets the default \catcodes and uc/lc codes, and inputs the hyphenation file.

1.1 Debugging

By default, LaTeX shows statistics:

```
⟨*2ekernel⟩
\tracingstats1
```

1.2 Typesetting parameters

\@lowpenalty
\@medpenalty
\@highpenalty

These are penalties used internally.

```
\newcount\@lowpenalty
\newcount\@medpenalty
\newcount\@highpenalty
```

(*End of definition for* \@lowpenalty*,* \@medpenalty*, and* \@highpenalty*.*)

\newmarks

Allocate extended marks types if etex is active. Placed here at the end of the format to increase compatibility with count allocations in earlier releases.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩                    {\newmarks}{Extended Allocation}%
\ifx\marks\@undefined\else
\def\newmarks{%
  \e@alloc\marks \e@alloc@chardef{\count256}\m@ne\e@alloc@top}
\fi
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\newmarks}{Extended Allocation}%
⟨latexrelease⟩\let\newmarks\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* \newmarks*.*)

Allocate 3 mark classes to be used in \markboth and \markright. Should be done earlier but for that definition of \newmarks needs moving (which it should I guess).

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2022/06/01}%
⟨latexrelease⟩                    {2e-left}{Delayed legacy marks}%
\NewMarkClass {2e-left}
```

```
\NewMarkClass {2e-right}
\NewMarkClass {2e-right-nonempty}
```

No rollback really, the marks will remain.

```
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {2e-left}{Delayed legacy marks}%
⟨latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

\newXeTeXintercharclass
\xe@alloc@intercharclass
\e@alloc@intercharclass@top

Allocate `\XeTeXintercharclass` types if xetex is active. previously defined in `xetex.ini`.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩              {\newXeTeXintercharclass}{Extended Allocation}%
```

Classes allocated 1 to 4094 (or 254 on older xetex) (In earlier XeLaTeX versions 1, 2 and 3 were pre-set for CJK).

```
\ifx\XeTeXcharclass\@undefined
\else

\ifdim\the\XeTeXversion\XeTeXrevision\p@>0.99993\p@
  \chardef\e@alloc@intercharclass@top=4095
\else
  \chardef\e@alloc@intercharclass@top=255
\fi

\def\newXeTeXintercharclass{%
 \e@alloc\XeTeXcharclass
   \chardef\xe@alloc@intercharclass\m@ne\e@alloc@intercharclass@top}
\fi

⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩              {\newXeTeXintercharclass}{Extended Allocation}%
⟨latexrelease⟩ \ifx\XeTeXcharclass\@undefined
⟨latexrelease⟩ \else
⟨latexrelease⟩    \def\xe@alloc@#1#2#3#4#5{\global\advance#1\@ne
⟨latexrelease⟩     \xe@ch@ck#1#4#2%
⟨latexrelease⟩     \allocationnumber#1%
⟨latexrelease⟩     \global#3#5\allocationnumber
⟨latexrelease⟩     \wlog{\string#5=\string#2\the\allocationnumber}}
⟨latexrelease⟩    \def\xe@ch@ck#1#2#3{%
⟨latexrelease⟩     \ifnum#1<#2\else
⟨latexrelease⟩      \errmessage{No room for a new #3}%
⟨latexrelease⟩     \fi}
⟨latexrelease⟩    \def\newXeTeXintercharclass{%
⟨latexrelease⟩     \xe@alloc@\xe@alloc@intercharclass
⟨latexrelease⟩                    \XeTeXcharclass\chardef\@cclv}
⟨latexrelease⟩ \fi
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2016/02/01}%
```

```
⟨latexrelease⟩  {\xe@alloc@intercharclass}{Start of XeTeX class allocator}%
\ifx\XeTeXcharclass\@undefined
\else
  \countdef\xe@alloc@intercharclass=257
  \xe@alloc@intercharclass=\z@
\fi
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2015/01/01}%
⟨latexrelease⟩  {\xe@alloc@intercharclass}{Start of XeTeX class allocator}%
⟨latexrelease⟩ \ifx\XeTeXcharclass\@undefined
⟨latexrelease⟩ \else
⟨latexrelease⟩   \xe@alloc@intercharclass=\thr@@
⟨latexrelease⟩ \fi
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩  {\xe@alloc@intercharclass}{Start of XeTeX class allocator}%
⟨latexrelease⟩ \ifx\XeTeXcharclass\@undefined
⟨latexrelease⟩ \else
⟨latexrelease⟩   \newcount\xe@alloc@intercharclass
⟨latexrelease⟩   \xe@alloc@intercharclass=\thr@@
⟨latexrelease⟩ \fi
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\newXeTeXintercharclass`, `\xe@alloc@intercharclass`, *and* `\e@alloc@intercharclass@top`.)

`trace_stack_levels` Now define the Lua function to emulate `\tracingstacklevels` and install it in the `input_level_string` callback.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
```

In `latexrelease` mode we always remove the function from the callback, then add the correct version later.

```
⟨latexrelease⟩\ifx\directlua\@undefined
⟨latexrelease⟩\else
⟨latexrelease⟩  \directlua{%
⟨latexrelease⟩    if luatexbase.callbacktypes['input_level_string'] and %
⟨latexrelease⟩       luatexbase.in_callback('input_level_string','tracingstacklevels') then
⟨latexrelease⟩        luatexbase.remove_from_callback('input_level_string','tracingstacklevels')
⟨latexrelease⟩    end}%
⟨latexrelease⟩\fi
⟨latexrelease⟩\IncludeInRelease{2021/06/01}{trace_stack_levels}%
⟨latexrelease⟩                  {Lua trace_stack_levels function}%
\ifx\directlua\@undefined
\else
⟨*2ekernel⟩
  \expanded{%
    \everyjob{\the\everyjob
    \noexpand%\directlua
⟨/2ekernel⟩
    \directlua{%
      local function trace_stack_levels (input_ptr)
        local tracingstacklevels = tex.count.tracingstacklevels
```

```
        if tex.tracingmacros > 0 or input_ptr < tracingstacklevels then
          if tracingstacklevels > 0 then
            if input_ptr < tracingstacklevels then
              return "\string\n\string~" .. string.rep(".",input_ptr)
            else
              return "\string~\string~"
            end
          else
            return "\string\n"
          end
        else
          return ""
        end
      end
⟨latexrelease⟩    if luatexbase.callbacktypes['input_level_string'] then
      luatexbase.add_to_callback('input_level_string',
        trace_stack_levels,'tracingstacklevels')
⟨latexrelease⟩    end
    }%
⟨*2ekernel⟩
  }}%
⟨/2ekernel⟩
\fi
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩
```

Then for the full rollback, just do nothing, since the function was already taken out of the rollback above.

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}{trace_stack_levels}%
⟨latexrelease⟩                  {Lua trace_stack_levels function}%
⟨latexrelease⟩% Nothing here
⟨latexrelease⟩\EndIncludeInRelease
⟨/2ekernel | latexrelease⟩
⟨*2ekernel⟩
```

(*End of definition for* `trace_stack_levels`.)

The default values of the picture and `\fbox` parameters:

```
\unitlength = 1pt
\fboxsep = 3pt
\fboxrule = .4pt
```

The saved value of TEX's `\maxdepth`:

```
\@maxdepth       = \maxdepth
```

`\vsize` initialized because a `\clearpage` with `\vsize < \topskip` causes trouble. `\@colroom` and `\@colht` also initialized because `\vsize` may be set to them if a `\clearpage` is done before the `\begin{document}`

```
\vsize = 1000pt
\@colroom = \vsize
\@colht = \vsize
```

Initialize `\textheight` `\textwidth` and page style, to avoid internal errors if they are not set by the class.

```
\textheight=.5\maxdimen
\textwidth=\textheight
\ps@empty
```

1.3 Lccodes for hyphenation

For 7- and 8-bit engines the assumption of T1 encodings is the basis for the hyphenation patterns. That's not the case for the Unicode engines, where the assumption is engine-native working. The common loader system provides access to data from the Unicode Consortium covering not only `\lccode` but also other related data. The `\lccode` part of that at least needs to be loaded before hyphenation is tackled: XeTeX follows the standard TeX route of building patterns into the format. LuaTeX doesn't require this data be loaded *here* but it does need to be loaded somewhere. Rather than test for the Unicode engines by name, the approach here is to look for the extended math mode handling both provide: any other engine developed in this area will presumably also provide `\Umathcode`.

```
\ifnum 0%
  \ifx\Umathcode\@undefined\else 1\fi
  \ifx\XeTeXmathcode\@undefined\else 1\fi
  >\z@
  \message{ Unicode character data,}
  \input{load-unicode-data}
⟨/2ekernel⟩
⟨latexrelease⟩\IncludeInRelease{2016/02/01}%
⟨latexrelease⟩  {\XeTeXintercharclasses}{XeTeX character classes}%
⟨latexrelease⟩  \ifx\XeTeXinterchartoks\undefined
⟨latexrelease⟩  \else
⟨latexrelease⟩    \begingroup
⟨latexrelease⟩      \chardef\XeTeXcharclassID = 0 %
⟨latexrelease⟩      \chardef\XeTeXcharclassOP = 0 %
⟨latexrelease⟩      \chardef\XeTeXcharclassCL = 0 %
⟨latexrelease⟩      \chardef\XeTeXcharclassEX = 0 %
⟨latexrelease⟩      \chardef\XeTeXcharclassIS = 0 %
⟨latexrelease⟩      \chardef\XeTeXcharclassNS = 0 %
⟨latexrelease⟩      \chardef\XeTeXcharclassCM = 0 %
⟨latexrelease⟩      \input{load-unicode-xetex-classes}
⟨latexrelease⟩    \endgroup
⟨latexrelease⟩    \global\let\xtxHanGlue\undefined
⟨latexrelease⟩    \global\let\xtxHanSpace\undefined
⟨latexrelease⟩    \global\XeTeXinterchartoks 0 1 = {}
⟨latexrelease⟩    \global\XeTeXinterchartoks 0 2 = {}
⟨latexrelease⟩    \global\XeTeXinterchartoks 0 3 = {}
⟨latexrelease⟩    \global\XeTeXinterchartoks 1 0 = {}
⟨latexrelease⟩    \global\XeTeXinterchartoks 2 0 = {}
⟨latexrelease⟩    \global\XeTeXinterchartoks 3 0 = {}
⟨latexrelease⟩    \global\XeTeXinterchartoks 1 1 = {}
⟨latexrelease⟩    \global\XeTeXinterchartoks 1 2 = {}
⟨latexrelease⟩    \global\XeTeXinterchartoks 1 3 = {}
⟨latexrelease⟩    \global\XeTeXinterchartoks 2 1 = {}
⟨latexrelease⟩    \global\XeTeXinterchartoks 2 2 = {}
⟨latexrelease⟩    \global\XeTeXinterchartoks 2 3 = {}
⟨latexrelease⟩    \global\XeTeXinterchartoks 3 1 = {}
⟨latexrelease⟩    \global\XeTeXinterchartoks 3 2 = {}
⟨latexrelease⟩    \global\XeTeXinterchartoks 3 3 = {}
⟨latexrelease⟩  \fi
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
```

```
⟨latexrelease⟩  {\XeTeXintercharclasses}{XeTeX character classes}%
⟨latexrelease⟩  \ifx\XeTeXinterchartoks\undefined
⟨latexrelease⟩  \else
⟨latexrelease⟩   \input{load-unicode-xetex-classes}
⟨latexrelease⟩   \gdef\xtxHanGlue{\hskip0pt plus 0.1em\relax}
⟨latexrelease⟩   \gdef\xtxHanSpace{\hskip0.2em plus 0.2em minus 0.1em\relax}
⟨latexrelease⟩   \global\XeTeXinterchartoks 0 1 = {\xtxHanSpace}
⟨latexrelease⟩   \global\XeTeXinterchartoks 0 2 = {\xtxHanSpace}
⟨latexrelease⟩   \global\XeTeXinterchartoks 0 3 = {\nobreak\xtxHanSpace}
⟨latexrelease⟩   \global\XeTeXinterchartoks 1 0 = {\xtxHanSpace}
⟨latexrelease⟩   \global\XeTeXinterchartoks 2 0 = {\nobreak\xtxHanSpace}
⟨latexrelease⟩   \global\XeTeXinterchartoks 3 0 = {\xtxHanSpace}
⟨latexrelease⟩   \global\XeTeXinterchartoks 1 1 = {\xtxHanGlue}
⟨latexrelease⟩   \global\XeTeXinterchartoks 1 2 = {\xtxHanGlue}
⟨latexrelease⟩   \global\XeTeXinterchartoks 1 3 = {\nobreak\xtxHanGlue}
⟨latexrelease⟩   \global\XeTeXinterchartoks 2 1 = {\nobreak\xtxHanGlue}
⟨latexrelease⟩   \global\XeTeXinterchartoks 2 2 = {\nobreak\xtxHanGlue}
⟨latexrelease⟩   \global\XeTeXinterchartoks 2 3 = {\xtxHanGlue}
⟨latexrelease⟩   \global\XeTeXinterchartoks 3 1 = {\xtxHanGlue}
⟨latexrelease⟩   \global\XeTeXinterchartoks 3 2 = {\xtxHanGlue}
⟨latexrelease⟩   \global\XeTeXinterchartoks 3 3 = {\nobreak\xtxHanGlue}
⟨latexrelease⟩  \fi
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

There is one over-ride that makes sense here (see below for the same for 8-bit engines): setting the lccode for `-` to itself.

```
  \lccode`\- =`\- % default hyphen char
```

The alternative is that a "traditional" engine is in use.

```
\else
```

We set things up so that hyphenation files can assume that the default (T1) lccodes are in use (at present this also sets up the uccodes). We temporarily define `\reserved@a` to apply `\reserved@c` to all the numbers in the range of its arguments.

```
\def\reserved@a#1#2{%
   \@tempcnta#1\relax
   \@tempcntb#2\relax
   \reserved@b
}
\def\reserved@b{%
   \ifnum\@tempcnta>\@tempcntb\else
      \reserved@c\@tempcnta
      \advance\@tempcnta\@ne
      \expandafter\reserved@b
   \fi
}
```

Depending on the TEX version, we might not be allowed to do this for non-ASCII characters.

```
\def\reserved@c#1{%
   \count@=#1\advance\count@ by -"20
   \uccode#1=\count@
   \lccode#1=#1
}
```

```
\reserved@a{‘\a}{‘\z}
\reserved@a{"A0}{"BC}
\reserved@a{"E0}{"FF}
```

The upper case characters need their `\uccode` and `\lccode` values set, and their `\sfcode` set to 999.

```
\def\reserved@c#1{%
   \count@=#1\advance\count@ by "20
   \uccode#1=#1
   \lccode#1=\count@
   \sfcode#1=999
}
\reserved@a{‘\A}{‘\Z}
\reserved@a{"80}{"9C}
\reserved@a{"C0}{"DF}
```

Well, it would be nice if that were correct, but unfortunately, the Cork encoding contains some odd slots whose uccode or lccode isn't quite what you'd expect.

```
\uccode‘\^^Y=‘\I     % dotless i
\lccode‘\^^Y=‘\^^Y   % dotless i
\uccode‘\^^Z=‘\J     % dotless j, ae in OT1
\lccode‘\^^Z=‘\^^Z   % dotless j, ae in OT1
\lccode‘\^^9d=‘\i    % dotted I
\uccode‘\^^9d=‘\^^9d % dotted I
\lccode‘\^^9e=‘\^^9e % d-bar
\uccode‘\^^9e=‘\^^d0 % d-bar
```

Finally here is one that helps hyphenation in the OT1 encoding.

```
\lccode‘\^^[=‘\^^[   % oe in OT1
```

And we also set the `\lccode` of `\-` and `\textcompwordmark` so that they do not prevent hyphenation in the remainder of the word (as suggested by Lars Helström).

```
\lccode‘\- =‘\-   % default hyphen char
\lccode 127=127   % alternate hyphen char
\lccode 23 =23    % textcompwordmark in T1
```

End of the conditional to select either Unicode or T1 encoding defaults.

```
\fi
```

At this stage, we can install any last-minute expl3 set-up.

```
\@expl@finalise@setup@@
\@expl@finalize@setup@@
\def\@expl@finalise@setup@@{}
\def\@expl@finalize@setup@@{}
```

This is as good a place as any to active a few XeTEX-specific settings

```
\ifx\XeTeXuseglyphmetrics\@undefined
\else
  \XeTeXuseglyphmetrics=1 %
  \XeTeXdashbreakstate=1 %
\fi
```

1.4 Hyphenation

The following code will be compiled into the format file. It checks for the existence of `hyphen.cfg` in inputs that file if found. Otherwise it inputs `hyphen.ltx`. Note that these are loaded in *before* the `\catcode`s are set, so local hyphenation files can use 8-bit input.

We try to load the customized hyphenation description file.

```
\InputIfFileExists{hyphen.cfg}
           {\typeout{=============================================^^J%
                      Local configuration file hyphen.cfg used^^J%
                     =============================================}%
             \def\@addtofilelist##1{\xdef\@filelist{\@filelist,##1}}%
           }
           {\input{hyphen.ltx}}
\let\@addtofilelist\@gobble
```

`\l@nohyphenation`

```
\ifx\l@nohyphenation \@undefined
  \newlanguage\l@nohyphenation
\fi
```

(*End of definition for* `\l@nohyphenation`.)

`\document@default@language` Default document language. -1 acts as language 0, but used as a flag in `\document` to see if it has been set in the preamble.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2017/04/15}%
⟨latexrelease⟩        {\document@default@language}{Save language for hyphenation}%
\let\document@default@language\m@ne
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩        {\document@default@language}{Save language for hyphenation}%
%
⟨latexrelease⟩\let\document@default@language\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

(*End of definition for* `\document@default@language`.)

1.5 Font loading

Fonts loaded during the formatting process might already have changed the `\font@submax` from `0pt` to something higher. If so, we put out a bold warning.

```
\ifdim \font@submax >\z@
   \@font@warning{Size substitutions with differences\MessageBreak
                  up to \font@submax\space have occurred.\MessageBreak
                 \MessageBreak
                 Please check the transcript file
                 carefully\MessageBreak
                 and redo the format generation if necessary!
                 \@gobbletwo}%
   \errhelp{Only stopped, to give you time to
            read the above message.}
   \errmessage{}
```

We reset the macro. Otherwise every user will get a warning on every job.

```
\def\font@submax{0pt}
\fi
```

For pdfTEX preload and enable automatic glyph to Unicode mapping for more reliable copy and paste support. Since v1.40.22, this data is dumped into the format so is simply loaded directly. For older engines, the same method as LuaTEX is used (see below).

For luaTEX `\pdfextension glyphtounicode` would error in dvi-mode. As the outputmode can be changed in the preamble we have to test in the definition. LuaTEX does not dump this information into the format, so we need to load it in the `\everyjob` hook. We can't use `\input` there, as it would set the `\jobname` in the event of the run being started without loading a file. We also don't want to put all of the tokens from the files *directly* in the hook, so we save to a macro. We need to read with spaces "present" as this is part of the syntax of the primitive, so need `\ExplSyntaxOff` in the setup for `\file_get:nnN`.

For other engines we provide the two commands with noop definitions

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2026/06/01}%
⟨latexrelease⟩                    {\pdfgentounicode}{Preload glyphtounicode}%
\ifdefined\directlua
  \protected\def\pdfglyphtounicode{%
    \ifnum\outputmode=1 %
     \expandafter\@firstofone
    \else
     \expandafter\@gobblethree
    \fi
     {\pdfextension glyphtounicode}%
  }
  \protected\edef\pdfgentounicode{\pdfvariable gentounicode}
\fi
\ifdefined\pdfgentounicode
⟨*2ekernel⟩
  \ifnum 0=0%
    \ifdefined\pdftexversion
% \pdftexversion<140 does not have \pdfgentounicode, so we only check higher values
       \ifnum \pdftexversion=140 \ifnum\pdftexrevision<22 1\fi\fi
    \else 1%
    \fi
    \relax
⟨/2ekernel⟩
    \input glyphtounicode %
    \input glyphtounicode-cmex %
⟨*2ekernel⟩
  \else
    \ExplSyntaxOn
      \file_get:nnN { glyphtounicode } { \ExplSyntaxOff }
        \@@data@glyphtounicode
      \file_get:nnN { glyphtounicode-cmex } { \ExplSyntaxOff }
        \@@data@glyphtounicode@cmex
    \ExplSyntaxOff
    \everyjob\expandafter{%
```

```
      \the\everyjob
      \@@data@glyphtounicode
      \@@data@glyphtounicode@cmex
    }
  \fi
⟨/2ekernel⟩
  \ifdefined\directlua
    \pdfvariable gentounicode=1 %
  \else
    \pdfgentounicode=1 %
  \fi
\fi
```

For other engines we provide the two commands with noop definitions

```
\ifx \pdfgentounicode \@undefined
  \let\pdfglyphtounicode\@gobbletwo
  \newcount\pdfgentounicode
\fi
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{2021/06/01}%
⟨latexrelease⟩                  {\pdfgentounicode}{Preload glyphtounicode}%
⟨latexrelease⟩\ifx \directlua\undefined \else
⟨latexrelease⟩  \pdfvariable gentounicode=0
⟨latexrelease⟩\fi
⟨latexrelease⟩\ifdefined\pdftexversion
⟨latexrelease⟩ \ifx \pdfgentounicode \@undefined \else
⟨latexrelease⟩  \ifnum 0=0%
⟨latexrelease⟩      \ifnum \pdftexversion=140 \ifnum\pdftexrevision<22 1\fi\fi
⟨latexrelease⟩    \relax
⟨latexrelease⟩    \input glyphtounicode
⟨latexrelease⟩  \else
⟨latexrelease⟩    \begingroup
⟨latexrelease⟩      \everyeof{\noexpand}\endlinechar-1
⟨latexrelease⟩      \edef\x{\endgroup
⟨latexrelease⟩        \everyjob{\the\everyjob\@@input glyphtounicode }%
⟨latexrelease⟩    }\x
⟨latexrelease⟩  \fi
⟨latexrelease⟩  \pdfgentounicode=1
⟨latexrelease⟩ \fi
⟨latexrelease⟩\fi
⟨latexrelease⟩\EndIncludeInRelease
```

When rolling back we can't unload the glyphtounicode mappings, but we can reset `\pdfgentounicode` to ensure that they aren't used and then undefine the commands for the engines that do not natively support it.

```
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                  {\pdfgentounicode}{Preload glyphtounicode}%
⟨latexrelease⟩ \pdfgentounicode=0
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

1.6 Input encoding

Starting with the 2018 LaTeX release default the inputencoding to UTF-8. Unless the format is being used with luatex, xetex, enctex or mltex.

This is done in a way largely compatible with older releases: `utf8.def` is input just as if

`\usepackage[utf8]{inputenc}`

had been used, however rather than input the whole package a minimal core part just enough to support loading the UTF-8 encoding files is defined here.

If a document re-specifies UTF-8 this is silently ignored.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
```

Check that a classic 8-bit tex engine is being used (LaTeX or PDFLaTeX).

```
⟨latexrelease⟩\IncludeInRelease{2018/04/01}%
⟨latexrelease⟩                  {\UTFviii@invalid}{UTF-8 default}%
```

Skip this section in Unicode TeX, or if MLTeX and EncTeX are enabled.

```
\ifnum0%
  \ifx\Umathcode\@undefined\else 1\fi
  \ifx\mubyte\@undefined\else 1\fi
  \ifx\charsubdef\@undefined\else 1\fi
  =\z@
\def\saved@space@catcode{10}
\let\@inpenc@test\relax
\def\IeC{%
  \ifx\protect\@typeset@protect
    \expandafter\@firstofone
  \else
    \noexpand\IeC
  \fi
}
```

Make characters active for UTF-8 input formats

```
\@tempcnta=1
\loop
  \catcode\@tempcnta=13  %
  \advance\@tempcnta\@ne %
\ifnum\@tempcnta<32      %
\repeat                  %
\catcode0=15  % null
\catcode9=10  % tab
\catcode10=12 % ctrl J
\catcode12=13 % ctrl L
\catcode13=5  % newline
\@tempcnta=128
\loop
  \catcode\@tempcnta=13
  \advance\@tempcnta\@ne
\ifnum\@tempcnta<256
\repeat
```

`\UseRawInputEncoding` Reset 8 bit characters to catcode 12 so the input encoding matches the "Raw" font encoding. Useful for special behaviours, or for compatibility with older LaTeX formats.

```
\def\UseRawInputEncoding{%
\let\inputencodingname\@undefined                       % revert
\let\DeclareFontEncoding@\DeclareFontEncoding@saved   % revert
\let\DeclareUnicodeCharacter\@undefined                 % revert
\@tempcnta=1
\loop
  \catcode\@tempcnta=15  %
  \advance\@tempcnta\@ne %
\ifnum\@tempcnta<32      %
\repeat                  %
\catcode0=15  % null
\catcode9=10  % tab
\catcode10=12 % ctrl J
\catcode12=13 % ctrl L
\catcode13=5  % newline
\@tempcnta=128
\loop
  \catcode\@tempcnta=12
  \advance\@tempcnta\@ne
\ifnum\@tempcnta<256
\repeat
}
```

(*End of definition for* `\UseRawInputEncoding`.)

`\DeclareFontEncoding@saved` Saved version of `\DeclareFontEncoding@` before `utf8.def` modifies it for use in `\UseRawInputEncoding` above.

```
\let\DeclareFontEncoding@saved\DeclareFontEncoding@
```

(*End of definition for* `\DeclareFontEncoding@saved`.)

```
\edef\inputencodingname{utf8}%
\input{utf8.def}
\let\UTFviii@undefined@err@@\UTFviii@undefined@err
\let\UTFviii@invalid@err@@\UTFviii@invalid@err
\let\UTFviii@two@octets@@\UTFviii@two@octets
\let\UTFviii@three@octets@@\UTFviii@three@octets
\let\UTFviii@four@octets@@\UTFviii@four@octets
⟨2ekernel⟩\def\UTFviii@undefined@err#1{\@gobble#1}%
⟨2ekernel⟩\let\UTFviii@invalid@err\string
⟨2ekernel⟩\let\UTFviii@two@octets\string
⟨2ekernel⟩\let\UTFviii@three@octets\string
⟨2ekernel⟩\let\UTFviii@four@octets\string
⟨2ekernel⟩\everyjob\expandafter{\the\everyjob
⟨2ekernel⟩\let\UTFviii@undefined@err\UTFviii@undefined@err@@
⟨2ekernel⟩\let\UTFviii@invalid@err\UTFviii@invalid@err@@
⟨2ekernel⟩\let\UTFviii@two@octets\UTFviii@two@octets@@
⟨2ekernel⟩\let\UTFviii@three@octets\UTFviii@three@octets@@
⟨2ekernel⟩\let\UTFviii@four@octets\UTFviii@four@octets@@
⟨2ekernel⟩}
\let\@inpenc@test\@undefined
\let\saved@space@catcode\@undefined
```

For formats not set up for UTF-8 default, set the C0 controls to catcode 15.

```
\else
\@tempcnta=0
\loop
  \catcode\@tempcnta=15  %
  \advance\@tempcnta\@ne %
\ifnum\@tempcnta<32      %
\repeat                  %
\catcode0=15  % null
\catcode9=10  % tab
\catcode10=12 % ctrl J
\catcode12=13 % ctrl L
\catcode13=5  % newline

\let\UseRawInputEncoding\relax
```

This ends the skipped code in Unicode engines:

```
\fi
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\UTFviii@invalid}{UTF-8 default}%
```

The first block of commands got only introduced in 2019 but we revert all of Unicode support in one go not jump to the intermediate version.

```
⟨latexrelease⟩  \let\UTFviii@two@octets@combine\@undefined
⟨latexrelease⟩  \let\UTFviii@three@octets@combine\@undefined
⟨latexrelease⟩  \let\UTFviii@four@octets@combine\@undefined
⟨latexrelease⟩  \let\UTFviii@two@octets@string\@undefined
⟨latexrelease⟩  \let\UTFviii@three@octets@string\@undefined
⟨latexrelease⟩  \let\UTFviii@four@octets@string\@undefined
⟨latexrelease⟩  \let\UTFviii@two@octets@noexpand\@undefined
⟨latexrelease⟩  \let\UTFviii@three@octets@noexpand\@undefined
⟨latexrelease⟩  \let\UTFviii@four@octets@noexpand\@undefined

⟨latexrelease⟩\@tempcnta=0
⟨latexrelease⟩\loop
⟨latexrelease⟩  \catcode\@tempcnta=15
⟨latexrelease⟩  \advance\@tempcnta\@ne
⟨latexrelease⟩\ifnum\@tempcnta<32
⟨latexrelease⟩\repeat       %
⟨latexrelease⟩\catcode9=10  % tab
⟨latexrelease⟩\catcode10=12 % ctrl J
⟨latexrelease⟩\catcode12=13 % ctrl L
⟨latexrelease⟩\catcode13=5  % newline
⟨latexrelease⟩\@tempcnta=128
⟨latexrelease⟩\loop
⟨latexrelease⟩\catcode\@tempcnta=12
⟨latexrelease⟩\advance\@tempcnta\@ne
⟨latexrelease⟩\ifnum\@tempcnta<256
⟨latexrelease⟩\repeat
⟨latexrelease⟩\let\IeC\@undefined
⟨latexrelease⟩\def\DeclareFontEncoding@#1#2#3{%
⟨latexrelease⟩  \expandafter
⟨latexrelease⟩  \ifx\csname T@#1\endcsname\relax
```

```
⟨latexrelease⟩      \def\cdp@elt{\noexpand\cdp@elt}%
⟨latexrelease⟩      \xdef\cdp@list{\cdp@list\cdp@elt{#1}%
⟨latexrelease⟩                     {\default@family}{\default@series}%
⟨latexrelease⟩                     {\default@shape}}%
⟨latexrelease⟩      \expandafter\let\csname#1-cmd\endcsname\@changed@cmd
⟨latexrelease⟩   \else
⟨latexrelease⟩      \@font@info{Redeclaring font encoding #1}%
⟨latexrelease⟩   \fi
⟨latexrelease⟩   \global\@namedef{T@#1}{#2}%
⟨latexrelease⟩   \global\@namedef{M@#1}{\default@M#3}%
⟨latexrelease⟩   \xdef\LastDeclaredEncoding{#1}%
⟨latexrelease⟩   }
⟨latexrelease⟩   \let\UseRawInputEncoding\@undefined
⟨latexrelease⟩   \let\DeclareFontEncoding@saved\@undefined
⟨latexrelease⟩   \let\inputencodingname\@undefined
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

We temporarily define `\reserved@a` to apply `\reserved@c` to all the numbers in the range of its arguments.

```
\def\reserved@a#1#2{%
   \@tempcnta#1\relax
   \@tempcntb#2\relax
   \reserved@b
}
\def\reserved@b{%
   \ifnum\@tempcnta>\@tempcntb\else
      \reserved@c\@tempcnta
      \advance\@tempcnta\@ne
      \expandafter\reserved@b
   \fi
}
```

Set the special catcodes (although some of these are useless, since an error will have occurred if the catcodes have changed). Note that `^^J` has catcode 'other' for use in warning messages.

```
\catcode`\ =10
\catcode`\#=6
\catcode`\$=3
\catcode`\%=14
\catcode`\&=4
\catcode`\\=0
\catcode`\^=7
\catcode`\_=8
\catcode`\{=1
\catcode`\}=2
\catcode`\~=13
\catcode`\@=11
\catcode`\^^I=10
\catcode`\^^J=12
\catcode`\^^L=13
\catcode`\^^M=5
```

Set the 'other' catcodes.

```
\def\reserved@c#1{\catcode#1=12\relax}
\reserved@c{`\!}
\reserved@c{`\"}
\reserved@a{`\'}{`\?}
\reserved@c{`\[}
\reserved@c{`\]}
\reserved@c{`\`}
\reserved@c{`\|}
```

Set the 'letter' catcodes.

```
\def\reserved@c#1{\catcode#1=11\relax}
\reserved@a{`\A}{`\Z}
\reserved@a{`\a}{`\z}
```

All the characters in the range 0–31 and 127–255 are illegal, *except* tab (`^^I`), nl (`^^J`), ff (`^^L`) and cr (`^^M`).

1.7 Lccodes and uccodes

We now again set up the default (T1) uc/lccodes. The lower case characters need their `\uccode` and `\lccode` values set. Some of this is a repeat of the set-up before loading hyphenation files. Depending on the TEX version, we might not be allowed to do this for non-ASCII characters. For the Unicode engines (XeTEX and LuaTEX) there is no need to do any of this: they use hyphenation data which does not alter any of the set up and so this entire block is skipped.

```
\ifnum 0%
  \ifx\Umathcode\@undefined\else 1\fi
  \ifx\XeTeXmathcode\@undefined\else 1\fi
  >\z@
\else
\def\reserved@c#1{%
   \count@=#1\advance\count@ by -"20
   \uccode#1=\count@
   \lccode#1=#1
}
\reserved@a{`\a}{`\z}
\reserved@a{"A0}{"BC}
\reserved@a{"E0}{"FF}
```

The upper case characters need their `\uccode` and `\lccode` values set, and their `\sfcode` set to 999.

```
\def\reserved@c#1{%
   \count@=#1\advance\count@ by "20
   \uccode#1=#1
   \lccode#1=\count@
   \sfcode#1=999
}
\reserved@a{`\A}{`\Z}
\reserved@a{"80}{"9C}
\reserved@a{"C0}{"DF}
```

Well, it would be nice if that were correct, but unfortunately, the Cork encoding contains some odd slots whose uccode or lccode isn't quite what you'd expect.

```
\uccode`\^^Y=`\I     % dotless i
\lccode`\^^Y=`\^^Y   % dotless i
```

```
\uccode`\^^Z=`\J     % dotless j, ae in OT1
\lccode`\^^Z=`\^^Z   % dotless j, ae in OT1
\lccode`\^^9d=`\i    % dotted I
\uccode`\^^9d=`\^^9d % dotted I
\lccode`\^^9e=`\^^9e % d-bar
\uccode`\^^9e=`\^^d0 % d-bar
```

Finally here is one that helps hyphenation in the OT1 encoding.

```
\lccode`\^^[=`\^^[   % oe in OT1
\fi % End of reset block for 8-bit engines
```

\BCPdata A stub for use by babel, polyglossia, etc.

```
\ExplSyntaxOn
\newcommand*\BCPdata[1]{
  \str_case:nn {#1}
    {
      { language } { en }
      { region }   { US }
      { script }   { Latn }
      { tag }      { en-US }
    }
}
\ExplSyntaxOff
```

(*End of definition for* \BCPdata.)

1.8 Case changing

\MakeUppercase
\MakeLowercase
\MakeTitlecase
\NoCaseChange
\AddToNoCaseChangeList
\CaseSwitch
\DeclareCaseChangeEquivalent
\DeclareLowercaseMapping
\DeclareTitlecaseMapping
\DeclareUppercaseMapping
\DeclareLowercaseExclusions
\DeclareTitlecaseExclusions
\DeclareUppercaseExclusions
\@uclclist

And whilst we're doing things with uc/lc tables, here are two commands to upper- and lower-case a string.

Wrappers around the L3 case changing functions. \protected to make them mostly safe as replacements for uppercase and \lowercase.

In

```
\markboth{\MakeUppercase\contentsname}
         {\MakeUppercase\contentsname}
```

then the uppercasing is only done to the first letter of the contents name, since the mark expands out to:

```
\mark{\MakeUppercase Table of Contents}
     {\MakeUppercase Table of Contents}
```

In order to get round this, we redefine \MakeUppercase and \MakeLowercase to grab their argument and brace it.

Earlier versions needed to process \@uclclist in an \edef to handle legacy input encodings, but recent (2022) expl3 versions handle non-UTF8 text natively so we simply call the \text_...case:n functions.

```
\ExplSyntaxOn
\keys_define:nn { __kernel }
  {
    lang   .str_set:N = \reserved@a ,
    locale .str_set:N = \reserved@a ,
    words .choices:nn =
```

```
      { all , first }
      { \str_set:Nn \reserved@b {#1} }
  }
\cs_new_protected:Npn \@@text@case@aux #1#2#3
  {
    \cs_set_nopar:Npn \reserved@a { }
    \cs_set_nopar:Npn \reserved@b { first }
    \tl_if_blank:nTF {#2}
      {
        \str_set:Ne \reserved@a
          { \BCPdata { casing } }
        \str_if_empty:NT \reserved@a
          {
            \str_set:Ne \reserved@a
              { \BCPdata { language } }
          }
      }
      { \keys_set:nn { __kernel } {#2} }
    \use:c { text_ #1 :Vn } \reserved@a {#3}
  }
```

The odd use of *three* spaces here is needed as ltcmd uses the name with one and two spaces to give a 'friendly' error message for a runaway argument: that means we can't use it here.

```
\exp_args_generate:n { cnx }
\cs_set_protected:Npn \reserved@a #1
  {
    \cs_generate_variant:cn { text_ \str_lowercase:n {#1} case:nn } { V }
    \ExpandArgs { cnx } \NewExpandableDocumentCommand
       { Make#1case }
       { O{} +m }
       { \exp_not:c { Make#1case \c_space_tl \c_space_tl \c_space_tl } [####1] {####2} }
  }
\reserved@a { Upper }
\reserved@a { Lower }
\reserved@a { Title }
```

These don't get covered above so we do them manually.

```
\cs_generate_variant:Nn \text_titlecase_all:nn { V }
\cs_generate_variant:Nn \text_titlecase_first:nn { V }
```

Currently, babel uses the equivalence of `\oe` and `\OE` to force casing of some material, most notably in `\today`. To enable that to work, we have to set those commands equal even though the current case changing code does not work using this approach.

```
\cs_new_protected:cpn { MakeLowercase \c_space_tl \c_space_tl \c_space_tl } [#1] #2
  {{
    \let \OE \oe
    \@@text@case@aux { lowercase } {#1} {#2}
  }}
\cs_new_protected:cpn { MakeUppercase \c_space_tl \c_space_tl \c_space_tl } [#1] #2
  {{
    \let \oe \OE
    \@@text@case@aux { uppercase } {#1} {#2}
  }}
\cs_new_protected:cpn { MakeTitlecase \c_space_tl \c_space_tl \c_space_tl } [#1] #2
```

```
  {{
    \let \oe \OE
    \@@text@case@aux { titlecase_ \reserved@b } {#1} {#2}
  }}
```

`\NoCaseChange` protects its argument from the case change functions.

`\AddToNoCaseChangeList` Allows new commands to protect their arguments, eg `AddToNoCaseChangeList{\eqref}` would protect the argument of `\eqref` in the same way as the argument of `\ref`.

```
\cs_new_protected_nopar:Npn\AddToNoCaseChangeList
         {\tl_put_right:Nn \l_text_case_exclude_arg_tl}
\AddToNoCaseChangeList{ \NoCaseChange }
\cs_new_protected:Npn \NoCaseChange #1 {#1}
\cs_new_eq:NN \CaseSwitch \text_case_switch:nnnn
\cs_new_eq:NN \DeclareCaseChangeEquivalent
  \text_declare_case_equivalent:Nn
\NewDocumentCommand \DeclareLowercaseMapping { o m m }
  {
    \IfNoValueTF {#1}
      { \text_declare_lowercase_mapping:nn }
      { \text_declare_lowercase_mapping:nnn {#1} }
        {#2} {#3}
  }
\NewDocumentCommand \DeclareTitlecaseMapping { o m m }
  {
    \IfNoValueTF {#1}
      { \text_declare_titlecase_mapping:nn }
      { \text_declare_titlecase_mapping:nnn {#1} }
        {#2} {#3}
  }
\NewDocumentCommand \DeclareUppercaseMapping { o m m }
  {
    \IfNoValueTF {#1}
      { \text_declare_uppercase_mapping:nn }
      { \text_declare_uppercase_mapping:nnn {#1} }
        {#2} {#3}
  }

\cs_new_protected:Npn \DeclareLowercaseExclusions #1
  {
    \clist_map_inline:nn {#1}
      { \text_declare_lowercase_exclusion:n {##1} }
  }
\cs_new_protected:Npn \DeclareTitlecaseExclusions #1
  {
    \clist_map_inline:nn {#1}
      { \text_declare_titlecase_exclusion:n {##1} }
  }
\cs_new_protected:Npn \DeclareUppercaseExclusions #1
  {
    \clist_map_inline:nn {#1}
      { \text_declare_uppercase_exclusion:n {##1} }
  }
\ExplSyntaxOff
```

```
\def\@uclclist{\oe\OE\o\O\ae\AE
      \dh\DH\dj\DJ\l\L\ng\NG\ss\SS\ij\IJ\th\TH}
```

(*End of definition for* `\MakeUppercase` *and others.*)

1.9 Automatic insertion of \par tokens

Since 2022 the major TEX engines have a parameter, `\partokencontext`, that controls whether a `\par` token is added if TEX is in horizontal mode at the end of `\vbox` and similar contexts. This gives more control than the classical behaviour where the internal *end paragraph* routine is invoked with no explicit token being added.

Setting `\partokencontext` to 2 ensures a literal `\par` is used in all contexts, setting to 0 reverts to the classic TEX behavior.

```
⟨/2ekernel⟩
⟨*2ekernel | latexrelease⟩
⟨latexrelease⟩\IncludeInRelease{2025/11/01}%
⟨latexrelease⟩                    {\partokencontext}{automatic par insertion}%
\ifdefined\partokencontext\partokencontext=\tw@\fi
⟨/2ekernel | latexrelease⟩
⟨latexrelease⟩\EndIncludeInRelease
⟨latexrelease⟩\IncludeInRelease{0000/00/00}%
⟨latexrelease⟩                    {\partokencontext}{automatic par insertion}%
⟨latexrelease⟩\ifdefined\partokencontext\partokencontext=\z@\fi
⟨latexrelease⟩\EndIncludeInRelease
⟨*2ekernel⟩
```

1.10 Applying Patch files

Between major releases, small patches will be distributed in files `ltpatch.ltx` which must be added at this point.

Patch file code removed.

```
%\IfFileExists{ltpatch.ltx}
%  {\typeout{=================================^^J%
%              Applying patch file ltpatch.ltx^^J%
%             =================================}
%   \def\fmtversion@topatch{unknown}
%   \input{ltpatch.ltx}
%   \ifx\fmtversion\fmtversion@topatch
%      \ifx\patch@level\@undefined
%        \typeout{^^J^^J^^J%
%         !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!^^J%
%         !! Patch file 'ltpatch.ltx' not suitable for this^^J%
%         !! version of LaTeX.^^J^^J%
%         !! Please check if initex found an old patch file:^^J%
%         !! --- if so, rename it or delete it, and redo the^^J%
%         !! initex run.^^J%
%         !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!^^J}%
%        \batchmode \@@end
%      \else
```

The code below adds the 'patch level' string to the first `\typeout` in the startup banner.

```
%        \def\fmtversion@topatch{0}%
%        \ifx\fmtversion@topatch\patch@level\else
```

```
%          \def\reserved@a\typeout##1##2\reserved@a{%
%                \typeout{##1 patch level \patch@level}##2}
%          \everyjob\expandafter\expandafter\expandafter{%
%             \expandafter\reserved@a\the\everyjob\reserved@a}
%          \let\reserved@a\relax
%          \the\everyjob
%        \fi
%      \fi
%   \else
%      \typeout{^^J^^J^^J%
%    !!!!
```

1.12 Initialize file list

Initialize `\strongemph`. This needs delaying as some of the necessary macros are defined late. By default we alternate between bold and bold italic.

```
\DeclareRobustCommand\strongemreset{\bfseries\upshape\ulcshape}
\DeclareStrongEmphSequence{\bfseries,\itshape}
```

1.13 Initialize file list

`\@providesfile` Initialize for use in the document. During initex a modified version has been used which leaves debugging information for `latexbug.tex`.

```
\def\@providesfile#1[#2]{%
    \wlog{File: #1 #2}%
    \expandafter\xdef\csname ver@#1\endcsname{#2}%
  \endgroup}
```

(*End of definition for* `\@providesfile`.)

`\@filelist` `\@addtofilelist` `\@addtofilelist@aux` Reset `\@filelist` so files input while making the format are not listed. The list built up so far may take up a lot of memory and so it is moved to `\reserved@a` where it will be overwritten as soon as almost any LaTeX command is issued in a class file. However the `latexbug.tex` program will be able to access this information and insert it into a bug report.

```
\let\@filelist\@gobble
\def\@addtofilelist#1{\xdef\@filelist
  {\@filelist,\expanded{\noexpand\@addtofilelist@aux#1},\@nil{#1}}}
\def\@addtofilelist@aux#1,#2\@nil#3{%
  \if\relax\detokenize\expandafter{\@gobble#2?}\relax
    #1%
  \else
    {#3}%
  \fi
}
```

(*End of definition for* `\@filelist`, `\@addtofilelist`, *and* `\@addtofilelist@aux`.)

1.14 Preparation for supporting PDF in backends

At the current point in time, basic support for PDF in backends is not part of LaTeX core; it is provided by external packages. At some time in the future that work will be placed into the kernel but for now it is separate and has to be explicitly loaded in the document.

In that code there is a command `\IfPDFManagementActiveTF` which can be used by packages in order to execute different code depending on the whether this basic backend support is loaded.

To make this also work properly when this external package is not loaded at all, we here add this command already in the kernel (with a trivial definition); thus any package can query this loading state in all circumstances. Once this basic PDF backend support gets moved to the kernel, this definition will vanish again from here or, rather, it will be replaced by a real test.

\IfPDFManagementActiveTF So long as the code for the basic backend support for PDF is not loaded, the test that is implicit here will always return the false branch. Once this code is loaded, this definition will get replaced by a real test (as it is then possible that the management code is either activated or not activated).

```
\let \IfPDFManagementActiveTF \@secondoftwo
```

The T/F variants are defined in a way that they continue to work when the main definition changes.

```
\long\def\IfPDFManagementActiveT#1{\IfPDFManagementActiveTF{#1}{}}
\def\IfPDFManagementActiveF{\IfPDFManagementActiveTF{}}
```

(*End of definition for* `\IfPDFManagementActiveTF`.)

1.15 Do some temporary work for pre-release

This is a good place to load code that hasn't yet been integrated into the other files ...

1.16 Some last minute initializations ...

Load the first aid set of definitions for external packages that await updates.

```
\@input{latex2e-first-aid-for-external-files.ltx}
```

1.17 Dumping the format

Finally we make `@` into a letter, ensure the format will be in the 'normal' error mode, and dump everything into the format file.

```
\makeatother
\errorstopmode
\dump
⟨/2ekernel⟩
```

```
⟨*glyphtounicode⟩
%% A subset of glyhtounicode-cmr.tex
%
%% Copyright (c) 2008,  Han The Thanh <thanh@river-valley.org>
%% Copyright (c) 2014,  Peter Selinger <selinger@mathstat.dal.ca>
%% Copyright (c) 2018,  Ross Moore <ross.moore@mq.edu.au>
%%

%% Glyphs from the cmex fonts:

\pdfglyphtounicode{angbracketleftBig}{27E8 FE02}
\pdfglyphtounicode{angbracketleftBigg}{27E8 FE04}
\pdfglyphtounicode{angbracketleftbig}{27E8 FE01}
\pdfglyphtounicode{angbracketleftbigg}{27E8 FE03}
\pdfglyphtounicode{angbracketrightBig}{27E9 FE02}
\pdfglyphtounicode{angbracketrightBigg}{27E9 FE04}
\pdfglyphtounicode{angbracketrightbig}{27E9 FE01}
\pdfglyphtounicode{angbracketrightbigg}{27E9 FE03}
\pdfglyphtounicode{arrowbt}{2193}
\pdfglyphtounicode{arrowdblbt}{21D3}
\pdfglyphtounicode{arrowdbltp}{21D1}
\pdfglyphtounicode{arrowhookleft}{21AA}
```

```
\pdfglyphtounicode{arrowhookright}{21A9}
\pdfglyphtounicode{arrowtp}{2191}
\pdfglyphtounicode{arrowvertex}{23D0}
\pdfglyphtounicode{arrowvertexdbl}{20E6}%  was {ED12}% PUA
\pdfglyphtounicode{backslashBig}{005C FE02}
\pdfglyphtounicode{backslashBigg}{005C FE04}
\pdfglyphtounicode{backslashbig}{005C FE01}
\pdfglyphtounicode{backslashbigg}{005C FE03}
\pdfglyphtounicode{braceex}{23AA}
\pdfglyphtounicode{bracehtipdownleft}{23DF}% was {ED17}% PUA
\pdfglyphtounicode{bracehtipdownright}{23DF}%  was {ED18}% PUA
\pdfglyphtounicode{bracehtipupleft}{23DE}%  was {ED19}% PUA
\pdfglyphtounicode{bracehtipupright}{23DE}%  was {ED1A}% PUA
\pdfglyphtounicode{braceleftBig}{007B FE02}
\pdfglyphtounicode{braceleftBigg}{007B FE04}
\pdfglyphtounicode{braceleftbig}{007B FE01}
\pdfglyphtounicode{braceleftbigg}{007B FE03}
\pdfglyphtounicode{braceleftbt}{23A9}
\pdfglyphtounicode{braceleftmid}{23A8}
\pdfglyphtounicode{bracelefttp}{23A7}
\pdfglyphtounicode{bracerightBig}{007D FE02}
\pdfglyphtounicode{bracerightBigg}{007D FE04}
\pdfglyphtounicode{bracerightbig}{007D FE01}
\pdfglyphtounicode{bracerightbigg}{007D FE03}
\pdfglyphtounicode{bracerightbt}{23AD}
\pdfglyphtounicode{bracerightmid}{23AC}
\pdfglyphtounicode{bracerighttp}{23AB}
\pdfglyphtounicode{bracketleftBig}{005B FE02}
\pdfglyphtounicode{bracketleftBigg}{005B FE04}
\pdfglyphtounicode{bracketleftbig}{005B FE01}
\pdfglyphtounicode{bracketleftbigg}{005B FE03}
\pdfglyphtounicode{bracketleftbt}{23A3}
\pdfglyphtounicode{bracketleftex}{23A2}
\pdfglyphtounicode{bracketlefttp}{23A1}
\pdfglyphtounicode{bracketrightBig}{005D FE02}
\pdfglyphtounicode{bracketrightBigg}{005D FE04}
\pdfglyphtounicode{bracketrightbig}{005D FE01}
\pdfglyphtounicode{bracketrightbigg}{005D FE03}
\pdfglyphtounicode{bracketrightbt}{23A6}
\pdfglyphtounicode{bracketrightex}{23A5}
\pdfglyphtounicode{bracketrighttp}{23A4}
\pdfglyphtounicode{ceilingleftBig}{2308 FE02}
\pdfglyphtounicode{ceilingleftBigg}{2308 FE04}
\pdfglyphtounicode{ceilingleftbig}{2308 FE01}
\pdfglyphtounicode{ceilingleftbigg}{2308 FE03}
\pdfglyphtounicode{ceilingrightBig}{2309 FE02}
\pdfglyphtounicode{ceilingrightBigg}{2309 FE04}
\pdfglyphtounicode{ceilingrightbig}{2309 FE01}
\pdfglyphtounicode{ceilingrightbigg}{2309 FE03}
\pdfglyphtounicode{circledotdisplay}{2A00 FE02}
\pdfglyphtounicode{circledottext}{2A00 FE01}
\pdfglyphtounicode{circlemultiplydisplay}{2A02 FE02}
\pdfglyphtounicode{circlemultiplytext}{2A02 FE01}
\pdfglyphtounicode{circleplusdisplay}{2A01 FE02}
```

```
\pdfglyphtounicode{circleplustext}{2A01 FE01}
\pdfglyphtounicode{contintegraldisplay}{222E FE02}
\pdfglyphtounicode{contintegraltext}{222E FE01}
\pdfglyphtounicode{coproductdisplay}{2210 FE02}
\pdfglyphtounicode{coproducttext}{2210 FE01}
\pdfglyphtounicode{floorleftBig}{230A FE02}
\pdfglyphtounicode{floorleftBigg}{230A FE04}
\pdfglyphtounicode{floorleftbig}{230A FE01}
\pdfglyphtounicode{floorleftbigg}{230A FE03}
\pdfglyphtounicode{floorrightBig}{230B FE02}
\pdfglyphtounicode{floorrightBigg}{230B FE04}
\pdfglyphtounicode{floorrightbig}{230B FE01}
\pdfglyphtounicode{floorrightbigg}{230B FE03}
\pdfglyphtounicode{hatwide}{02C6 FE01}
\pdfglyphtounicode{hatwider}{02C6 FE02}
\pdfglyphtounicode{hatwidest}{02C6 FE03}
\pdfglyphtounicode{integraldisplay}{222B FE02}
\pdfglyphtounicode{integraltext}{222B FE01}
\pdfglyphtounicode{intersectiondisplay}{22C2 FE02}
\pdfglyphtounicode{intersectiontext}{22C2 FE01}
\pdfglyphtounicode{logicalanddisplay}{22C0 FE02}
\pdfglyphtounicode{logicalandtext}{22C0 FE01}
\pdfglyphtounicode{logicalordisplay}{22C1 FE02}
\pdfglyphtounicode{logicalortext}{22C1 FE01}
\pdfglyphtounicode{mapsto}{21A6}
\pdfglyphtounicode{parenleftBig}{0028 FE02}
\pdfglyphtounicode{parenleftBigg}{0028 FE04}
\pdfglyphtounicode{parenleftbig}{0028 FE01}
\pdfglyphtounicode{parenleftbigg}{0028 FE03}
\pdfglyphtounicode{parenleftbt}{239D}
\pdfglyphtounicode{parenleftex}{239C}
\pdfglyphtounicode{parenlefttp}{239B}
\pdfglyphtounicode{parenrightBig}{0029 FE02}
\pdfglyphtounicode{parenrightBigg}{0029 FE04}
\pdfglyphtounicode{parenrightbig}{0029 FE01}
\pdfglyphtounicode{parenrightbigg}{0029 FE03}
\pdfglyphtounicode{parenrightbt}{23A0}
\pdfglyphtounicode{parenrightex}{239F}
\pdfglyphtounicode{parenrighttp}{239E}
\pdfglyphtounicode{productdisplay}{220F FE02}
\pdfglyphtounicode{producttext}{220F FE01}
\pdfglyphtounicode{radicalBig}{221A FE02}
\pdfglyphtounicode{radicalBigg}{221A FE04}
\pdfglyphtounicode{radicalbig}{221A FE01}
\pdfglyphtounicode{radicalbigg}{221A FE03}
\pdfglyphtounicode{radicalbt}{23B7}%  was {221A}
\pdfglyphtounicode{radicaltp}{231C}%  was {ED6A}% PUA
\pdfglyphtounicode{radicalvertex}{20D3}%  was {ED6B}% PUA
\pdfglyphtounicode{slashBig}{002F FE02}
\pdfglyphtounicode{slashBigg}{002F FE04}
\pdfglyphtounicode{slashbig}{002F FE01}
\pdfglyphtounicode{slashbigg}{002F FE03}
\pdfglyphtounicode{summationdisplay}{2211 FE02}
\pdfglyphtounicode{summationtext}{2211 FE01}
```

```
\pdfglyphtounicode{tie}{2040}
\pdfglyphtounicode{tildewide}{02DC FE01}
\pdfglyphtounicode{tildewider}{02DC FE02}
\pdfglyphtounicode{tildewidest}{02DC FE03}
\pdfglyphtounicode{uniondisplay}{22C3 FE02}
\pdfglyphtounicode{unionmultidisplay}{2A04 FE02}
\pdfglyphtounicode{unionmultitext}{2A04 FE01}
\pdfglyphtounicode{unionsqdisplay}{2A06 FE02}
\pdfglyphtounicode{unionsqtext}{2A06 FE01}
\pdfglyphtounicode{uniontext}{22C3 FE01}
\pdfglyphtounicode{vextenddouble}{20E6}%  was {ED79}% PUA
\pdfglyphtounicode{vextendsingle}{20D3}%%% was {23D0}
⟨/glyphtounicode⟩
```

Change History

1985-11-04 ltmath.dtx LaTeX2.09
General: produce warning message if line extends into margin. Doesn't warn about formula overprinting equation number. 900
1989-04-10 ltfssbas.dtx v1.0a
General: Starting with version numbers! `\ifmmode` added in `\math@group` 571
1989-04-10 ltfssbas.dtx v1.0b
General: `\preload@sizes` added. . . 571
`\wrong@fontshape` changed to define substitution font/shape macro. 571
1989-04-10 ltfssini.dtx v1.0a
General: Starting with version numbers `\newif` for `\@tempswa` added since this switch is unknown at the time when this file is read in. (latex.tex is loaded later.) `\math@famname` changed to `\math@version`. 735
1989-04-14 ltfssbas.dtx v1.0c
General: More documentation added. 571
1989-04-15 ltfssini.dtx v1.0b
General: `\mathfontset` renamed to `\mathversion`. 735
1989-04-19 ltfssbas.dtx v1.0d
General: Even more doc. 571
1989-04-21 ltfssbas.dtx v1.0e
General: Documentation is fun! Parameters of `\define@mathalphabet` changed. 571
1989-04-21 ltfssini.dtx v1.0c
General: Changed to conform to fam.tex. 735
1989-04-23 ltfssbas.dtx v1.0f
General: % in `\getanddefinefonts` added. 571
1989-04-26 ltfssini.dtx v1.0d
General: `\xpt` added. 735
1989-04-27 ltfssbas.dtx v1.0g
General: Documentation revised. . . 571
1989-04-27 ltfssini.dtx v1.0e
General: Definitions of LaTeX symbols corrected. 735
1989-04-29 ltfssbas.dtx v1.0h
General: Documented problem with `\halign`, and `\noalign` 571
`\mathversion`: Test if version defined added. 584
1989-04-29 ltfssbas.dtx v1.0i
General: Removed the `\halign` `\noalign` correction (wasn't bugfree) 571
1989-04-29 ltfssini.dtx v1.0f
General: Corrections to LaTeX tabular env. added. 735
1989-05-01 ltfssbas.dtx v1.0j
General: Default for `\baselinestretch` added. 571
1989-05-22 ltfssbas.dtx v1.0k
General: Lines longer than 72 characters folded. 571
1989-05-22 ltfssini.dtx v1.0g
General: Lines shortened to 72 characters 735
1989-09-14 ltfssbas.dtx v1.0m
General: Global replacement: `\group` to `\mathgroup` 571
`\mathversion`: Corrected typo: `\endscname` to `\endcsname`. . . . 584
1989-11-07 ltfssini.dtx v1.0i
General: All family, series, and shape names abbreviated. 735
1989-11-08 ltfssbas.dtx v1.0o
General: First parameter of `\define@mathalphabet` and `\define@mathgroup` changed from string to control sequence. 571
1989-11-14 ltfssbas.dtx v1.0p
`\math@version`: Math version prefix 'mv@' added. 584
1989-11-19 ltfssbas.dtx v1.0q
`\define@newfont`: Group added. . . 587
`\wrong@fontshape`: Instead of calling `\family\default@family`, etc. we directly set `\f@family`, etc. . . . 592
1989-11-22 ltfssbas.dtx v1.0r
`\math@version`: `\def` → `\edef` for `\math@version`. 584
1989-11-25 ltfssbas.dtx v1.0s
General: All `\edef\font@name` changed to `\xdef\font@name`. Necessary after introduction of `\begingroup`/`\endgroup` in v1.0q. 571
extra// → + in `\extra@def`. . . . 571

1989-11-26 ltfssbas.dtx v1.0t
\select@group: \bgroup/\egroup changed to \begingroup/\endgroup to avoid empty Ord atom on math list. . . 595
1989-12-02 ltfssini.dtx v1.1b
General: \rmmath renamed to \mathrm 735
1989-12-03 ltfssini.dtx v1.1c
General: Some internal macros renamed to make them inaccessible. 735
1989-12-05 ltfssbas.dtx v1.0u
\addto@hook: \addto@hook added. . 599
1989-12-05 ltfsstrc.dtx v1.0u fam.dtx
\every@math@size: Hook \every@size added. 680
1989-12-13 ltfsstrc.dtx v1.0f
\use@mathgroup: \expandafter added before final \fi. 683
1989-12-16 ltfssbas.dtx v1.1a
\select@group: \relax in front added. 595
Now four arguments. 595
Redefinition of alphabet now simpler. 595
Usage of '=' macro added. 595
1989-12-16 ltfsstrc.dtx v1.1a
\selectfont: Changed order of calls. 675
\use@mathgroup: Redefinition of alphabet now simpler. 682
Usage of '=' macro added. 682
1990-01-18 ltfsstrc.dtx v1.0h
General: \tracingfonts meaning changed. 671
1990-01-20 ltfssbas.dtx v1.2a
\math@bgroup: Def. placed in this file. 597
\math@egroup: Def. placed in this file. 597
\select@group: Def for alph id changed. 595
1990-01-21 ltfssbas.dtx v1.2b
\select@group: Code moved to \use@mathgroup. 595
1990-01-21 ltfsstrc.dtx v1.2b
\use@mathgroup: Macro added to allow cleaner interface. 682
1990-01-23 ltfssbas.dtx v1.2c
General: \no@version@warning renamed to \no@alphabet@error. 571
Macro \no@alphabet@help added 571
\no@alphabet@error: Changed to error call 571
1990-01-25 ltfssini.dtx v1.1e
\nfss@text: Macro added. 760
1990-01-27 ltfssbas.dtx v1.2d
\DeclarePreloadSizes: Font identifier set to \relax. 579
1990-01-28 ltfssbas.dtx v1.2e
\mathgroup: \newfam let to \new@mathgroup. 571
1990-01-28 ltfssbas.dtx v1.2f
\define@newfont: Added call to \curr@fontshape macro to allow substitution. 588
\wrong@fontshape: Warning message slightly changed. 592
1990-01-28 ltfssini.dtx v1.2b
\em: Call to \@nomath added. 758
1990-02-08 ltfssini.dtx v1.1g
General: Protected the commands \family, \series, \shape, \size, \selectfont, and \mathversion. 735
1990-02-16 ltfssbas.dtx v1.2g
General: Support for changes of \baselineskip without changing the size. 571
\math@version: \@nomath added. . . 584
1990-02-18 ltfsstrc.dtx v1.0j
\selectfont: Redefine unprotected version \p@selectfont instead of \selectfont. 675
1990-03-14 ltfsstrc.dtx v1.0k
General: Added code for TeX3. . . . 671
\extract@font: Added code for TeX3. 674
1990-03-30 ltfssbas.dtx v1.2h
\math@egroup: Changed to have one arg. 597
1990-03-30 ltfsstrc.dtx v1.2h
\use@mathgroup: Third argument removed (see \math@egroup). . . . 682
1990-04-01 ltfssbas.dtx v1.2i
General: Code added from tracefnt.dtx. 571
Support for TeX3. 571
1990-04-01 ltfsstrc.dtx v1.0l
General: Part of code moved to fam.dtx. 671
\tracingfonts: Check if \tracingfonts already defined. . 672
1990-04-01 ltfsstrc.dtx v1.0o
\tracingfonts: Check if \tracingfonts defined removed again. 672

1990-04-02 ltfssini.dtx v1.1i
General: \input of files now handled by docstrip. 735
1990-04-05 ltfsstrc.dtx v1.0m
\selectfont: Call \tracingon only if \tracingfonts greater than 3. . 675
1990-05-05 ltfsstrc.dtx v1.0n
\selectfont: \tracingon with new syntax. 675
1990-06-23 ltfssini.dtx v1.1k
\nfss@text: Changed to \mbox. . . . 761
1990-06-24 ltfssbas.dtx v1.2j
\DeclarePreloadSizes: Missing percent added. 579
1990-06-24 ltfsstrc.dtx v1.0o
\baselinestretch: Moved to tracefnt.dtx. 680
\getanddefine@fonts: \Adding tracing code. 684
\Macro moved from fam.dtx. . . . 683
Adding debug code. 684
\use@mathgroup: Tracing code added. 683
1990-06-30 ltfssbas.dtx v1.2l
\showhyphens: Macro added. 598
1990-06-30 ltfsstrc.dtx v1.0p
\use@mathgroup: Added \relax after math group number. 683
1990-07-07 ltfsstrc.dtx v1.0q
\getanddefine@fonts: Group number added to tracing. 684
\math@egroup: Tracing code added. 683
\use@mathgroup: Group number added to tracing. 683
1990-08-27 ltfsstrc.dtx 1.0r
\type@restoreinfo: Some extra tracing info. 679
1990-08-27 ltfsstrc.dtx v1.0r
\getanddefine@fonts: Correcting missing name after \tracingon. . 684
1991-03-28 ltfssini.dtx v1.1m
\copyright: Extra braces added. . . 761
1991-03-30 ltfssini.dtx v1.2g
\newfont: Definition added. 759
\symbol: Definition added. 760
1991-07-24 ltmiscen.dtx LaTeX2.09
\@verbatim: Added \penalty\interlinepenalty to definition of \par so that \samepage works 882
1991-08-14 ltmath.dtx LaTeX2.09
\cases: (RmS) inserted extra braces around entry for NFSS 893
1991-08-14 ltpictur.dtx LaTeX2.09
General: (RmS) inserted extra braces around entry for NFSS 979
1991-08-14 ltthm.dtx LaTeX2.09
\@endtheorem: Moved \itshape after \item to make it work with NFSS 1010
1991-08-26 ltfssini.dtx v1.1n
\reset@font: Macro introduced . . . 761
1991-08-26 ltmiscen.dtx LaTeX2.09
\@verbatim: \@@par added 882
1991-08-26 ltpictur.dtx LaTeX2.09
\endpicture: (RmS & FMi) extra boxing level around \@picbox to guard against unboxing in math mode (proposed by John Hobby) 977
1991-08-26 ltplain.dtx LaTeX209
\tracingall: Added \errorcontextlines=\maxdimen, suggested by J. Schrod 31
1991-09-29 ltboxes.dtx LaTeX2.09
\@mpfootnotetext: (RmS) added \reset@font 941
1991-09-29 ltfloat.dtx LaTeX2.09
\@footnotetext: (RmS) added \reset@font 1047
1991-09-29 ltmath.dtx LaTeX2.09
\@eqnnum: RmS: \reset@font added. 900
1991-09-29 ltsect.dtx LaTeX2.09
\@dottedtocline: (RmS) added \reset@font for page number . 1024
1991-10-17 ltcntrl.dtx LaTeX209
\@tfor: (Rms) \xdef replaced by \def (See FMi's array.doc) 426
1991-10-25 ltbibl.dtx LaTeX2.09
\@citex: added \reset@font, suggested by Bernd Raichle. . . 1055
1991-11-01 ltfloat.dtx LaTeX2.09
\footnote: (RmS) Added \let\protect\noexpand in \footnote, \footnotemark, and \footnotetext, since \xdef is used 1046
1991-11-04 ltlists.dtx LaTeX2.09
\makelabel: (RmS) added default definition for \makelabel, to produce an error message. 921
1991-11-04 ltplain.dtx RmS
General: Removed \itemitem since never needed/useful with LaTeX. . 29
1991-11-06 ltbibl.dtx LaTeX2.09
\@citex: added code to remove a leading blank 1055

1991-11-13 ltbibl.dtx LaTeX2.09
\@bibitem: Changed counter enumi to enumiv, as it says in the comment above 1054
1991-11-21 ltfssini.dtx v1.1o
\reset@font: Added extra braces for robustness. 761
Changed to protected version of macro. 761
1991-11-22 ltfloat.dtx LaTeX2.09
\footnote: (RmS) Added \let\protect\noexpand in \@xfootnote, \@xfootnotemark, and \@xfootnotetext 1046
1991-11-22 ltlists.dtx LaTeX2.09
\@item: (RmS) Changed second call to \makelabel to \unhbox\@tempboxa. Avoids problems with side effects in \makelabel and is more efficient. 921
1991-11-27 ltfssbas.dtx v1.3a
General: All \family, \shape etc. renamed to \fontfamily etc. . . . 571
1991-11-27 ltfssini.dtx v1.2a
General: All \family, \shape etc. renamed to \fontfamily etc. . . . 735
1992-01-06 ltfssini.dtx v1.2c
General: added slitex code 735
1992-01-10 ltbibl.dtx LaTeX2.09
\@bibitem: Changed \c@enumiv to \value of \@listctr 1054
1992-01-10 ltmath.dtx LaTeX2.09
equation: RmS: put \hbox around \@eqnnum to typeset the equation number in text mode (as in the eqnarray env.) 899
1992-01-10 ltthm.dtx LaTeX2.09
\@othm: (RmS) Check for existence of theorem environment 1009
1992-01-14 ltbibl.dtx LaTeX2.09
\@biblabel: removed \hfill 1057
1992-01-14 ltsect.dtx 0.0
\@starttoc: (RmS) added \immediate to \openout as all \write commands are also executed \immediate 1021
1992-02-26 ltbibl.dtx LaTeX2.09
\@lbibitem: Added \hfill to restore left-alignment of bibliography labels in alpha style 1054
1992-03-18 ltdefns.dtx LaTeX209
General: (RMS) changed input channel from 0 to \@inputcheck to avoid conflicts with other channels allocated by \newread 82
1992-03-18 ltfloat.dtx LaTeX2.09
\@xympar: (RmS) added \global\@ignorefalse 1040
\end@float: (RmS) changed \@esphack to \@Esphack 1034
1992-03-18 ltlists.dtx 0.0
\trivlist: RmS: added \@nmbrlistfalse 916
1992-03-18 ltmiscen.dtx LaTeX2.09
\begin: Changed \@ignoretrue to \@ignorefalse (as documented) 873
1992-03-21 ltfssini.dtx v1.2d
General: Renamed \text to \nfss@text to make it internal. . 735
1992-05-12 ltfssbas.dtx v1.3c
\extract@alph@from@version: Macro added. 596
\select@group: Added call to \extract@alph@from@version. . 595
1992-07-26 ltfssbas.dtx v1.9a
\curr@fontshape: 587
\DeclareFontShape: Introduced \DeclareFontShape 572
\define@newfont: 587
\math@fonts: 594
\select@group: 595
\split@name: Added splitting into \f@encoding. 587
\wrong@fontshape: 592
1992-07-26 ltfsstrc.dtx v2.0b
\s@fct@: 693
\s@fct@sub: documentation fixes . . 694
\selectfont: 676
\try@simple@size: 687
\try@size@range: 690
\use@mathgroup: 683
1992-08-14 ltbibl.dtx LaTeX2.09
\@citex: added missing argument braces around \hbox, found by Ed Sznyter 1055
1992-08-14 ltboxes.dtx LaTeX209
\endminipage: (RmS) replaced \vskip-\lastskip by \unskip (proposed by FMi) 940
1992-08-17 ltbibl.dtx LaTeX2.09
\@citex: simplified code for removing leading blanks in citation key (proposed by Frank Jensen and Kresten Krab Thorup) 1055
1992-08-19 ltsect.dtx 0.0
\@xsect: (RmS) corrected bug: stretch and shrink in argument to \hskip

previously not negated 1017
1992-08-19 ltthm.dtx LaTeX2.09
`\@othm`: (RmS) Changed error message to complain about undefined counter 1009
1992-08-20 ltfssini.dtx v1.4b
`\@setsize`: Added `\@currsize`. . . . 760
1992-08-24 ltdefns.dtx LaTeX209
`\@ifnextchar`: (Rms) `\@ifnextchar` didn't work if its first argument was an equal sign. 110
1992-08-24 ltmiscen.dtx LaTeX2.09
`\begin`: Added code to `\begin` to remember line number. Used by `\@badend` to display position of non-matching `\begin`. 873
`\verb`: Changed `\verb` and `\@sverb` to work correctly in math mode . . . 887
1992-08-25 ltsect.dtx LaTeX2.09
`\@sect`: (FMi) replaced explicit setting of `\@svsec` by call to `\@seccntformat` 1016
1992-09-18 ltlists.dtx LaTeX2.09
`\item`: (RmS) Added warning if `\item` is used in math mode 919
1992-09-18 lttab.dtx LaTeX2.09
`\@array`: Changed `\par` to `\@empty` to avoid starting new row e.g. after `\hline` 960
1992-09-19 ltfsstrc.dtx v2.0c
`\try@simple@size`: 687
1992-09-21 ltfssini.dtx v1.4d
`\not@math@alphabet`: Macro defined. 759
1992-09-22 ltfssbas.dtx v1.91a
General: Introduced `\tf@size` for math size. 571
1992-09-22 ltfsstrc.dtx v2.1a
`\getanddefine@fonts`: Introduced `\tf@size` for math size. 684
1992-11-13 ltfssini.dtx v?
`\hexnumber@`: Made expandable. . . . 760
1992-11-23 ltcounts.dtx LaTeX209
`\stepcounter`: Replaced {} in `\stepcounter` by `\begingroup` `\endgroup` to avoid adding an empty ord in math mode 559
1992-11-26 ltboxes.dtx LaTeX2.09
`\@mpfootnotetext`: (RmS) added protection for `\edef` 941
1992-11-26 ltfloat.dtx LaTeX2.09
`\@footnotetext`: (RmS) added protection for `\edef` 1047
`\footnote`: (RmS) Changed all to 'def'protect'noexpand'protect'noexpand 1046
1992-12-03 ltfssini.dtx v?
`\hexnumber@`: Make it accept counters. 760
1993-03-08 preload.dtx v2.0b
General: Added 12pt preloads 787
1993-03-18 ltfssbas.dtx v2.0c
General: Changed all `\@tempdima` in `\@tempdimb` to avoid killing `\numberline` 571
1993-03-18 ltfsstrc.dtx v2.1b
General: Changed all `\@tempdima` in `\@tempdimb` to avoid killing `\numberline` 671
Changed all `\@tempdimb` in `\@tempdimx` to avoid killing `\numberline` 671
1993-03-18 ltfsstrc.dtx v2.1c
`\DeclareSizeFunction`: Added all args to avoid blanks problems . . 690
1993-04-09 lterror.dtx v1.0e
`\@latexerr`: Mention The Companion 433
1993-04-11 lterror.dtx v1.0f
`\@latexerr`: Remove setting of errorcontextlines 433
1993-05-05 ltfntcmd.dtx v2.0b
General: Removed all LaTeX related cmds 790
1993-05-16 ltfssbas.dtx v2.0e
`\showhyphens`: Use `\reset@font` . . 598
1993-07-16 ltfsstrc.dtx v2.1h
General: Changed layout of info messages 671
1993-07-17 ltoutenc.dtx 1.0d
General: changed `\catcoding` @ . . . 512
1993-08-03 ltmiscen.dtx LaTeX2.09
`\enddocument`: Changed redefinition of `\global` to redefinition of `\@setckpt`. 865
1993-08-05 ltpictur.dtx LaTeX2.09
`\circle`: (RMS) Added error message if `\circle` is used in math mode. 999
1993-08-05 ltsect.dtx LaTeX2.09
`\@sect`: (RmS) Made sure that `\protect` works correctly in expansion of `\the` counter 1016
1993-08-05 ltspace.dtx LaTeX2e
`\@hspace`: (RmS) Removed superfluous `\leavevmode` in `\@hspace` and `\@hspacer`, as suggested by CAR. 481

1993-08-05 lttab.dtx latex2e
\tabular*: Replaced \expandafter\def by \@namedef. 960
1993-08-06 ltbibl.dtx LaTeX2.09
\@citex: Moved writing to .aux file in loop over citation keys so that leading blanks are removed there as well. 1055
1993-08-13 ltoutenc.dtx 1.0f
General: Protected against active @ sign. 512
1993-08-13 preload.dtx v2.0c
General: Added \relax at end of font names. 788
1993-08-16 ltoutenc.dtx 1.0g
General: Needs space after \string 512
1993-08-18 ltfssdcl.dtx v2.0e
\new@mathversion: Exchanged names of encodings in warning message of \SetSymbolFont. 716
1993-09-02 ltfsstrc.dtx v2.1i
General: Corrected name of sgen size function. 671
1993-09-03 ltmiscen.dtx LaTeX2.09
\verbatim@nolig@list: Replaced \@noligs by extensible list 887
1993-09-07 ltmiscen.dtx LaTeX2.09
\verb@balance@group: (RmS) Changed definition of \verb so that it detects a missing second delimiter. 887
1993-09-08 ltmiscen.dtx LaTeX2.09
\enddocument: Added warning in case of undefined references. 865
1993-09-15 ltfssbas.dtx v2.0g
\DeclareFontEncoding: Corrected: \default@T to \default@M. 575
1993-09-15 ltfsstrc.dtx v2.1j
General: Corrected spelling of \noxpand. 671
1993-09-19 lterror.dtx LaTeX2.09
\@invalidchar: (RmS) Error message for invalid input characters. 437
1993-11-02 ltmath.dtx LaTeX2.09
General: RmS: Corrected description of \@eqnsel, moved \@eqnsel accordingly and removed extra \tabskip assignment. 900
1993-11-03 ltmath.dtx LaTeX2e
General: RmS: Initialized \everycr to empty 900
1993-11-03 ltpictur.dtx LaTeX2.09
General: (RmS) changed \halign to \ialign to initialize \tabskip and \everycr 979
1993-11-11 ltfssini.dtx v2.1a
\normalfont: Macro added 761
1993-11-11 ltfsstrc.dtx v2.2a
General: Option concept added for LaTeX2e 671
1993-11-14 ltclass.dtx v0.2a
\@currext: Name changed from \@currextension 1100
\@reset@ptions: macro added . . . 1128
\AtEndDocument: Included extension in the generated macro name for package and class hooks. 1129
\documentstyle: Added \RequirePackage \@unusedoptionlist stuff. . . . 1116
\load@onefilewithoptions: Moved resetting of \default@ds, \ds@ and \@declaredoptions here, from the end of \ProcessOptions. 1122
\NeedsTeXFormat: made more robust for alternative syntax for other formats. 1118
\ProcessOptions*: Optimize 'empty option' code. 1113
Stop adding the global option list inside class files. 1113
1993-11-14 ltdefns.dtx v0.2a
\g@addto@macro: Made global 114
1993-11-15 ltclass.dtx v0.2b
\documentstyle: Modified to match \ProcessOption* 1116
\ProcessOptions*: Star form added. 1113
1993-11-17 ltclass.dtx v0.2c
\@@fileswith@pti@ns: Macro added 1128
\@badrequireerror: Macro added 1130
\@twoloadclasserror: Macro added 1131
\CurrentOption: Name changed from \@curroption 1099
\DeclareOption*: Error checking added 1112
\load@onefilewithoptions: Added trap for two \LoadClass commands. 1124
\NeedsTeXFormat: Name changed from \NeedsFormat 1118
\ProcessOptions*: restoring \@fileswith@pti@ns added. . . 1113
1993-11-18 ltclass.dtx v0.2d
\documentstyle: Modified \RequirePackage stuff. 1116

`\ExecuteOptions`: Use `\CurrentOption` not `\reserved@a` 1116
`\NeedsTeXFormat`: `\fmtname` `\fmtversion` not `\@`... 1118
1993-11-21 ltfiles.dtx LaTeX2e
`\@missingfileerror`: Stop infinite looping on `\@er@ext` 505
1993-11-21 ltmiscen.dtx v0.9a
`\@verbatim`: use `\verbatim@font` instead of `\tt` 882
`\verb`: Use `\verbatim@font` instead of `\tt`. 887
`\verbatim@font`: Macro added 883
1993-11-22 ltclass.dtx v0.2f
`\@fileswithoptions`: Made the default [] not [`\@unknownversion`] 1120
`\@ifl@ter`: Added //00 so parsing never produces a runaway argument. 1105
General: `\@unknownversion` removed 1094
`\load@onefilewithoptions`: Made the initial version [] not [`\@unknownversion`] 1122
1993-11-22 ltdefns.dtx LaTeX2e
`\@minus`: Macro added 81
`\@plus`: Macro added 81
`\CheckCommand`: Macro added 89
`\providecommand`: Macro added 89
1993-11-22 lterror.dtx LaTeX2e
`\c@errorcontextlines`: Macro added 433
1993-11-22 ltfiles.dtx LaTeX2e
`\listfiles`: Removed checking for `\@unknownversion` 507
1993-11-22 ltlength.dtx LaTeX2e
`\@settodim`: Macro added 569
`\@settopoint`: Macro added 570
`\settodepth`: Macro added 569
`\settoheight`: Macro added 569
1993-11-22 ltlogos.dtx LaTeX2e
`\LaTeXe`: Macro added 484
1993-11-23 ltclass.dtx v0.2g
`\@use@ption`: Name changed from `\@executeoption` 1115
General: Various macros now moved to latex.tex. 1098
Warnings and errors now directly coded. 1098
1993-11-23 ltdefns.dtx LaTeX2e
`\@argdef`: Macro added 84
`\@ifundefined`: Redefined to remove a trailing `\fi` 108
`\@newcommand`: Macro added 84
`\@newenv`: Macro interface changed . . 88
`\@xargdef`: Macro interface changed . 84
`\@yargd@f`: Avoid `\@?@?` token 85
Macro interface changed 85
`\newcommand`: Macro reimplemented and extended 84
`\renewcommand`: Macro reimplemented and extended 87
`\renewenvironment`: Macro reimplemented and extended 88
`\two@digits`: Macro added 80
1993-11-23 ltoutput.dtx v0.1a
`\paperheight`: Register added . . . 1226
`\paperwidth`: Register added 1226
1993-11-23 ltoutput.dtx v0.1c
`\@enlargepage`: Command added . 1290
`\@kludgeins`: Insert added 1289
`\@makecol`: Command changed . . . 1241
`\@specialoutput`: Command changed 1235
`\enlargethispage*`: Commands added 1289
1993-11-24 ltfntcmd.dtx v2.1a
`\maybe@ic@`: Use `\t@st@ic` 795
`\t@st@ic`: Macro added 796
1993-11-24 ltfssini.dtx v2.1a
General: Removed `\xpt` stuff 761
1993-11-24 ltlogos.dtx LaTeX2e
`\LaTeX`: Macro changed 484
1993-11-28 ltclass.dtx v0.2h
`\@twoclasseserror`: Macro added 1131
General: Assorted commands now in the kernel removed. 1098
Directory syntax checking moved to dircheck.dtx 1098
Primitive filenames now terminated by space not `\relax`. 1098
`\endfilecontents`: Don't globally allocate a write stream (always use 15) 1131
1993-11-28 ltfiles.dtx LaTeX2e
`\@missingfileerror`: Use filename parser from dircheck 505
1993-11-29 ltoutput.dtx v1.0b
`\@makecol`: `\@makespecialcolbox` added 1241
1993-11-29 ltplain.dtx LaTeX2e
General: All accents in decimals; suggested by Paul Taylor 30
1993-11-30 ltoutput.dtx v1.0c
`\fl@tracemessage`: Commands added 1292

1993-12-01 fontdef.dtx v2.1a
General: Update for LaTeX2e 766
1993-12-01 ltoutput.dtx v1.0e
\@reinserts: Command added . . 1253
1993-12-03 ltboxes.dtx v0.1a
\@argrsbox: macro removed 944
\@begin@tempboxa: macro added . . 927
\@end@tempboxa: macro added 927
\@iirsbox: redefined to support \height 944
\@imakebox: macro modified 928
\@irsbox: redefined to support \height 944
\@isavebox: color support 931
extra group 931
\@isavepicbox: extra group 931
\@makebox: default changed from x to c 927
\@makepicbox: macro modified 928
\@savebox: default c not x 931
\bm@b: macros added 927
\endlrbox: macro added 932
\fbox: extra group 933
\lrbox: color support 932
macro added 932
\makebox: modified 926
\mbox: extra group 927
\minipage: Redefined to support extra optional arguments 939
\newsavebox: Pass the whole of arg 1 to \@ifdefinable 930
\parbox: Redefined to support extra optional arguments 935
\raisebox: redefined to support \height 944
\sbox: color support 931
extra group 931
\set@color: color support 929
macro added 929
1993-12-03 ltclass.dtx v0.2i
\@cls@pkg: Name changed to avoid clash with output routine. 1129
General: \@onlypreamble: Many commands declared. 1098
Removed obsolete \@documentclass 1098
1993-12-03 lterror.dtx v1.0b
\@latexerr: Set \c@errorcontextlines to -1 . . . 433
1993-12-03 ltfssini.dtx v2.1a
General: update for LaTeX2e 735
1993-12-04 ltfilehook.dtx v0.9b
\unqu@tefilef@und: Macro added 1170
1993-12-04 ltfiles.dtx v0.9b
\@iinput: Macro reimplemented . . . 504
\@input: Macro reimplemented . . . 504
\IfFileExists@: Macro added 500
\input: Macro reimplemented 503
1993-12-05 ltfloat.dtx LaTeX2e
\@dblfloatplacement: Command changed 1036
\@xfloat: Command changed . . . 1030
1993-12-05 ltoutput.dtx v1.0f
\@addtobot: Command changed . . 1266
\@addtocurcol: Command changed 1268
\@addtodblcol: Command changed 1283
\@addtonextcol: Command changed 1278
\@addtotoporbot: Command changed 1267
\@boxfpsbit: Command added . . 1296
\@flcheckspace: Command added 1298
\@flsetnum: Command added . . . 1297
\@flsettextmin: Command added 1297
\@flstop: Commands added 1293
\@flupdates: Command added . . 1300
\@fpsadddefault: Command added 1294
\@getfpsbit: Command added . . 1295
\@opcol: Command changed 1240
Hook added 1240
\@outputpage: Command changed 1254
\@resethfps: Command added . . 1296
\@setfloattypecounts: Command added 1295
\@setfpsbit: Command added . . 1296
\@shipoutsetup: Command added 1254
\@startcolumn: Command changed 1262
\@startdblcolumn: Command changed 1262
\@testfp: Command added 1296
\@textfloatsheight: Commands added 1294
\@topnewpage: Commands changed 1230
\@tryfcolumn: Command changed 1263
\@writesetup: \@startpagehook added 1254
\output: Command changed 1235
1993-12-06 ltclass.dtx v0.2k
\ExecuteOptions: Preserve \CurrentOption. 1116
1993-12-06 ltoutput.dtx v1.0f
\@specialoutput: Unboxing of 255 added to rescue writes 1235
1993-12-06 ltoutput.dtx v1.0g
\@topnewpage: \@floatplacement placement bug fixed 1230
1993-12-07 ltclass.dtx v0.2l
\ProvidesFile: Macro added 1110

1993-12-07 ltclass.dtx v0.2m
\load@onefilewithoptions: Reset \CurrentOption 1122
1993-12-07 ltoutenc.dtx 1.1
General: Protected all special characters with \string. 512
1993-12-07 ltoutenc.dtx v1.1
General: Made all character numbers decimal. 509
Removed a lot of equal signs and the like. 509
1993-12-08 ltboxes.dtx v0.1b
\@begin@tempboxa: Extra braces for color support (braces removed from other macros) 927
\@irsbox: fix typo 944
\@parboxto: \endgraf added due to extra group in \@begin@tempboxa 936
\lrbox: move \@endpefalse out of the inner group 932
1993-12-08 ltfntcmd.dtx v2.1b
General: Macros \rm, \bf and \sf moved to classes.dtx 798
1993-12-08 ltlists.dtx LaTeX2e
\@item: use \sbox to support color . 921
1993-12-08 ltspace.dtx LaTeX2e
\@bsphack: Command reimplemented 470
Command reimplemented; late birthday present for Chris 470
\@vbsphack: Command added 473
1993-12-09 ltboxes.dtx v0.1c
\@irsbox: fix another typo 944
1993-12-09 ltclass.dtx v0.2n
\documentstyle: input 209 compatibility file. 1116
1993-12-09 ltfiles.dtx v0.9e
\document: Hook added 487
1993-12-09 ltmiscen.dtx v0.9e
\enddocument: Hook added 865
1993-12-10 ltoutenc.dtx v1.2
General: Added source code for t1enc.sty. 509
1993-12-11 ltfntcmd.dtx v3.0a
General: Complete reworking of all text commands, using just one creator function 790
italic correction now put in front of penalty before glue 790
newcommands replaced by defs . . 790
newfontswitch command corrected and changed 790
\DeclareTextFontCommand: Macro changed 792
\emph: Macro changed 793
\fix@penalty: Macro added 796
\maybe@ic: Macro name changed . . 795
\maybe@ic@: Macro and name changed 795
\sw@slant: Macro changed 796
\textup: Macros changed 793
1993-12-11 ltmath.dtx v0.9g
General: Added a group around the first argument of \frac to prevent changes (for example font changes) from modifying the contents of the second argument. 900
1993-12-11 ltoutenc.dtx v1.2a
General: Corrected for t1enc, math. 509
1993-12-11 ltsect.dtx LaTeX2e
\author: Added default 1012
\title: Added default 1012
1993-12-11 ltxref.dtx LaTeX2e
\@setref: Macro added 842
\pageref: Macro reimplemented . . . 842
\ref: Macro reimplemented 842
1993-12-12 ltoutput.dtx v1.0h
\@cflb: boxmaxdepth setting moved 1260
defs changed to lets 1260
\@cflt: name changed 1260
\@doclearpage: defs changed to lets 1240
\@makecol: defs changed to lets . . 1241
\@resethfps: Warnings added: minimal 1296
\@startdblcolumn: defs changed to lets 1262, 1263
\@topnewpage: braces removed . . . 1230
\@tryfcolumn: defs changed to lets 1263
\fl@tracemessage: Commands changed 1292
1993-12-13 ltclass.dtx v0.2o
General: Removed setting \errorcontextlines (now in latex.tex) 1098
\documentstyle: compatibility file now latex209.sty. 1116
\usepackage: Fixed error handling 1118
1993-12-13 ltdirchk.dtx v0.2a
General: on the 'docstrip' pass, do not check openin path 10
\IfFileExists: Removed interactive prompting for current directory syntax 10
\strip@prefix: modified, name changed from \stripmeaning. . . . 5
1993-12-13 ltlists.dtx latex2e
\trivlist: Initialized \@itemlabel 916
1993-12-13 ltmiscen.dtx v0.9h
\@noligs: Readded \@noligs 888

`\@verbatim`: Readded `\@noligs` . . . 882
Removed optional argument of `\item` 882
`center`: Removed optional argument of `\item` 879
`flushleft`: Removed optional argument of `\item` 880
`flushright`: Removed optional argument of `\item` 881
1993-12-13 ltoutenc.dtx v1.2b
General: Corrected file name in driver code. 509
1993-12-13 lttab.dtx latex2e
`\tabbing`: Removed optional argument of `\item` 954
1993-12-14 ltoutput.dtx v1.0i
General: Section added to declare all parameters 1305
1993-12-15 ltboxes.dtx v0.1d
`\@iminipage`: Changed default from 'c' to 's' 939
`\@iparbox`: Changed default from 'c' to 's' 935
`\minipage`: Changed default from 'c' to 's' 939
extra space removed. 939
`\parbox`: Changed default from 'c' to 's' 935
1993-12-15 ltclass.dtx v0.2p
General: Removed extra ':'s from `\@@warnings` 1098
1993-12-16 ltlogos.dtx LaTeX2e
`\LaTeXe`: Extended logo by DPC . . 484
1993-12-16 ltmath.dtx v0.9i
`\@@eqncr`: use `\refstepcounter` instead of shortcut 902
General: use `\refstepcounter` instead of shortcut 900
1993-12-16 ltmiscen.dtx v0.9i
General: `\literal` added 888
1993-12-16 ltpage.dtx LaTeX2e
`\mark`: Init `\mark` at begin document 1092
1993-12-16 ltspace.dtx LaTeX2e
`\@bsphack`: Corrected optimisation . 470
1993-12-16 lttab.dtx latex2e
`\@xhline`: Measure from middle of vertical rules 971
1993-12-17 ltclass.dtx v0.2q
`\@documentclasshook`: Macro added 1099
`\@fileswithoptions`: Add `\@compatibility` hook 1120
`\documentstyle`: Match Alan's new code. 1116
1993-12-17 ltoutenc.dtx 1.3
General: Added this section 513
Removed all the hackery for use in `\DeclareFontEncoding`, and redid everything using `\DeclareTextFoo`. 525, 528
Removed the catcode hackery, since the file is only read as a package in the preamble, and removed all the messages on the screen, which just confuse users. Replaced them by the appropriate `\ProvidesPackage` commands. Added XXXenc. . . . 512
1993-12-17 ltoutenc.dtx v1.3
General: Added `\EncodingSpecificAccent`, `\EncodingSpecificAccentedLetter` and `\EncodingSpecificCommand`. 509
Made Rokicki's encoding a proper encoding scheme rather than a variant of OT1. 509
1993-12-17 ltoutput.dtx v1.0j
`\@opcol`: Hook removed 1240
`\@specialoutput`: Page room test added 1235
`\@topnewpage`: check for vsize too small added 1230
Page room test added 1231
`\@writesetup`: —and then removed 1254
`\fl@tracemessage`: tracefloatvals made a document command . . 1292
1993-12-17 ltpage.dtx LaTeX2e
`\mark`: Removed init `\mark` at begin document, since it doesn't work. 1092
`\rightmark`: Stopgap solution to mark `\leftmark` and `\rightmark` work without initializing mark until the problem is solved. 1092
1993-12-18 ltoutenc.dtx 1.3b
General: Fixed typos with `\ProvidesPackage` lines. Added the `\NeedsTeXFormat` line. Added the last argument to `\DeclareEncoding`. Moved the use of the encodings to after their declaration. 512
Replaced the missing last argument to `\DeclareFontEncoding`. 525, 528
1993-12-18 ltoutenc.dtx 1.3c
General: Rewrote for the new syntax of `\EncodingSpecific`. . . . 525, 528
Split `\EncodingSpecificAccent` up into `\EncodingSpecific` and `\DeclareAccent`. 513

1993-12-18 ltoutenc.dtx v1.3a
General: Replaced OT3 by XXX . . 509
1993-12-18 ltoutenc.dtx v1.3b
General: Corrected typos. 509
Replaced the missing last argument to `\DeclareFontEncoding`. . . . 509
1993-12-18 ltoutenc.dtx v1.3c
General: A new syntax, separating accent-definitions from encoding-specific definitions, and allowing encoding-specific `\chardef`, `\let`, etc. 509
Rewrote for the new syntax of `\EncodingSpecific`. 509
1993-12-18 ltoutenc.dtx v1.3d
General: Some T1 stuff had drifted into the OT1 file. 509
1993-12-18 ltpage.dtx LaTeX2e
`\sloppy`: Added `\emergencystretch` 1093
1993-12-19 ltclass.dtx v0.2r
`\endfilecontents`: Different message when ignoring a file 1131
1993-12-19 ltfntcmd.dtx v3.0b
General: `\@pdef` command added . . 790
Added by ASAJ. 798
Made `\@newfontswitch` produce an error if command already exists, and added `\@renewfontswitch`, ASAJ 790
Other tidying 790
Some more tidying done 790
Untidying added, so this is now a TEMPORARY version. 790
Wording changes by CAR. 798
`\DeclareOldFontCommand`: Corrected and tidied 797
`\DeclareTextFontCommand`: Corrected and tidied 792
1993-12-19 ltspace.dtx LaTeX2e
`\@bsphack`: There seem to be problems with selfmade birthday presents . 471
1993-12-20 ltdefns.dtx LaTeX2e
`\@reargdef`: Kept old version of `\@reargdef`, for array.sty 87
1993-12-20 ltfiles.dtx v0.9m
`\@obsoletefile`: Added this command, removed @oldfilewarning 506
1994-01-05 fontdef.dtx v2.1d
General: Removed nf prefix from file names. 768
1994-01-13 ltmath.dtx v0.9o
`\@@eqncr`: correcting 0.9i 902
General: correcting 0.9i 900
1994-01-14 ltdirchk.dtx v0.2d
`\IfFileExists`: Close the texsys.aux output stream 10
1994-01-15 ltfiles.dtx v0.9o
`\document`: move `\@preamblecmds` after document hook 489
1994-01-17 ltclass.dtx v0.2s
`\@fileswithoptions`: Modify to reduce parameter stack usage . 1120
General: Added many more `\@onlypreamble` commands . . . 1098
Wrapped long lines to column 72 1098
`\load@onefile@withoptions`: Modify to reduce parameter stack usage 1125
1994-01-17 ltfiles.dtx LaTeX2e
`\listfiles`: New Version, adds '.tex' if needed, and lines up columns . 507
1994-01-17 ltfssbas.dtx v2.1a
General: New math font setup 571
`\curr@math@size`: New math font setup 586
`\everydisplay`: New math font setup 585
`\everymath`: New math font setup . 585
`\frozen@everydisplay`: New math font setup 586
`\frozen@everymath`: New math font setup 585, 586
`\math@version`: New math font setup 584
1994-01-17 ltfssini.dtx v2.1e
`\not@math@alphabet`: Message changed 759
1994-01-17 ltfsstrc.dtx v2.3a
General: New math font setup 671
`\check@mathfonts`: New math font setup 681
`\glb@currsize`: New math font setup 679
`\restglb@settings`: New math font setup 682
1994-01-18 ltbibl.dtx LaTeX2e
`\bibliography`: Use `\@input@` so include files are listed. 1055
1994-01-18 ltclass.dtx v0.2t
`\@ifclassloaded`: Fix typo `\@pkgetension` 1104
1994-01-18 ltfilehook.dtx v0.9p
`\unqu@tefilef@und`: New Definition 1170
1994-01-18 ltfiles.dtx v0.9p
`\@iffileonpath`: Macro added 502
`\@input`: do not use a different definition for `\input@path` 504
`\@input@`: Macro added 505
`\IfFileExists@`: New Definition . . 500
`\includeonly`: Use `\@input@` so include files are listed. 493

1994-01-18 ltfssini.dtx v2.1f
\not@math@alphabet: Message corrected 759
1994-01-18 ltmiscen.dtx v0.9p
\@verbatim: Add \global\@inlabelfalse 882
Only add \penalty if in hmode . 882
1994-01-19 fontdef.dtx v2.1e
General: Added missing setting for symbols in bold version. 773
1994-01-19 ltdirchk.dtx v0.2e
\IfFileExists: name changed from \test 9
\input@path: No longer check that an empty group is in the path 10
\strip@prefix: name changed from \strip@meaning, to match NFSS. 5
1994-01-19 ltmath.dtx v1.0n classes
\mathindent: Deferred setting of \mathindent 904
1994-01-20 ltdirchk.dtx v0.2f
General: \@copytexsys and the texsys.new file removed 8
Modify all of ltxcheck 13
\IfFileExists: \@copytexsys removed 10
1994-01-21 ltclass.dtx v0.2u
\documentstyle: compatibility file now latex209.def. 1116
1994-01-21 ltdirchk.dtx v0.2g
General: Improve documentation, reorganize docstrip module 1
\filename@parse: Minor changes, and add Mac version (:) 11
\today: Name changed from \stamp, to save memory 9
1994-01-21 ltfloat.dtx LaTeX2e
\@xfloat: Added missing percent characters. 1030
1994-01-21 ltmiscen.dtx v0.9s
\verbatim@font: Removed unnecessary category code hackery. 883
1994-01-24 ltdirchk.dtx v0.2h
\IfFileExists: Stop testing once texsys.aux has been found 9
1994-01-24 ltpage.dtx LaTeX2e
\pagestyle: (DPC) Complain if pagestyle is undefined. 1089
1994-01-25 ltdirchk.dtx v0.2i
General: Protect against looping on \@@input and \@@end. 3
1994-01-25 ltfssbas.dtx v2.1b
\math@version: Corrections for math setup 585
1994-01-25 ltmath.dtx LaTeX2e
\bordermatrix: Removed \p@renwd. 894
1994-01-26 ltfsstrc.dtx v2.3c
\check@mathfonts: Correct trace info placement 681
\restglb@settings: Correct trace info placement 682
1994-01-27 ltfntcmd.dtx v3.1a
\nocorrlist: Only ., used as default for cm fonts 797
1994-01-29 ltclass.dtx v0.2v
\@@unprocessedoptions: Macro added. 1130
\load@onefile@withoptions: All options raise error if no \ProcessOptions appears 1126
1994-01-31 ltdefns.dtx v0.2w
\g@addto@macro: Use toks register to avoid 'hash' problems 114
1994-01-31 ltfiles.dtx v0.9t
\document: set \@normalsize or \normalsize if necessary 488
1994-01-31 ltfntcmd.dtx v3.1b
General: \@normalsize no longer defined 790
1994-02-01 ltpage.dtx LaTeX2e
\pagestyle: (DPC) Modify to get nicer error message 1089
\thispagestyle: (DPC) Modify to get nicer error message 1090
1994-02-02 ltclass.dtx v0.2x
\load@onefile@withoptions: Only run the hook and options check if the file was loaded. 1126
1994-02-03 ltoutput.dtx v1.0k
\@make@specialcolbox: correct mistakes in the documentation . 1244
1994-02-07 ltclass.dtx v0.2y
\@fileswithoptions: Run \@compatibility on the first class to start (not the first to finish) 1120
\@ifclasswith: Add extra ,s so 'two' is not matched with 'twocolumn' 1106
\ProcessOptions*: Add extra ,s so 'two' is not matched with 'twocolumn' 1113, 1114
1994-02-07 ltfssbas.dtx v2.1c
\DeclareFontEncoding: revert catcode settings earlier 575
\DeclareFontShape@: revert catcode settings earlier 572

1994-02-08 ltoutput.dtx v1.0k
General: Documentation and tasks tidied. 1214
1994-02-10 ltclass.dtx v0.2z
`\@documentclasshook`: Changed the name from `\@compatibility` to `\@documentclasshook`, and added the check for whether `\@normalsize` has been defined. ASAJ. 1099
`\@fileswithoptions`: Renamed `\@compatibility` to `\@documentclasshook`. ASAJ. . 1120
1994-02-10 ltfssbas.dtx v2.1d
`\addto@hook`: Made `\addto@hook` long. 599
1994-02-10 ltfsscmp.dtx v2.1d
`\scan@@fontshape`: scan away stuff after pt 698
1994-02-22 ltfssini.dtx v2.1g
General: Correct error message 764
1994-02-24 ltfssbas.dtx v2.1e
`\DeclareFontShape`: Separate restoration of catcodes for fd cmds 572
`\define@newfont`: Separate restoration of catcodes for fd cmds 588
`\nfss@catcodes`: Separate restoration of catcodes for fd cmds 588
1994-02-25 ltdirchk.dtx v0.2j
General: Remove need for drv file 1
1994-03-01 ltdirchk.dtx v0.2k
General: Add unstripped module, so that dircheck.dtx may be used with initex 1
1994-03-02 ltboxes.dtx v0.1e
General: Add 2ekernel module 926
Remove need for drv file 926
1994-03-02 ltclass.dtx v0.3a
General: Remove need for driver file 1098
1994-03-03 ltboxes.dtx v0.1f
`\@irsbox`: Replaced a missing `\else` 944
1994-03-04 ltfloat.dtx v1.0a
General: Initial version, split from latex.dtx 1026
1994-03-04 ltsect.dtx v1.0a
General: Initial version, split from latex.dtx 1012
1994-03-04 lttab.dtx v1.0a
General: Initial version, split from latex.dtx 947
1994-03-04 ltvers.dtx v1.0a
General: Initial version, split from latex.dtx 35
1994-03-07 ltboxes.dtx v0.1a
`\@mpfootnotetext`: Extra group for color 941
1994-03-07 ltboxes.dtx v1.0a
General: Unify format with other Kernel files 926
1994-03-07 ltdefns.dtx v1.0a
`\@@italiccorr`: Macro added 81
1994-03-07 ltfiles.dtx v1.0a
General: Initial version, split from latex.dtx 485
Long lines wrapped to 72 columns 485
1994-03-07 ltfinal.dtx v0.1a
General: Add code from the old dump.dtx 1350
Initial version, split from latex.dtx 1332
move code here from lhyphen.dtx 1339
Remove oldcomments environment 1332
use `\InputIfFileExists` not `\IfFileExists` 1339
1994-03-07 ltfloat.dtx v1.0a
`\@endfloatbox`: (DPC) Extra group for color 1035
`\@footnotetext`: (DPC) Extra group for color 1047
`\@xfloat`: (DPC) Extra group for color 1031
1994-03-07 lthyphen.dtx v0.1c
General: move the 2ekernel code to ltfinal.dtx 1330
1994-03-07 ltlength.dtx v1.0a
`\@settodim`: (DPC) Extra group for color 569
1994-03-07 ltlists.dtx v1.0a
General: Initial version, split from latex.dtx 908
Long lines wrapped to 72 columns 908
1994-03-07 ltpage.dtx v1.0a
General: Initial version, split from ltherest.dtx 1089
1994-03-07 ltpictur.dtx v0.1a
General: Initial version, split from latex.dtx 974
Long lines wrapped to 72 columns 974
1994-03-07 ltsect.dtx v1.0a
`\@hangfrom`: (DPC)Extra groups for color 1019
1994-03-07 lttab.dtx v1.0a
General: Long lines wrapped to 72 columns 947

1994-03-08 ltclass.dtx v0.3b
General: Modify driver code into 'new style' 1098
1994-03-08 ltdirchk.dtx v1.0a
General: Reorganize driver module into 'new style' 1
1994-03-08 ltplain.dtx v1.0a
General: Remove need for a driver file. 14
1994-03-10 ltfssbas.dtx v2.2f
\math@egroup: Changed \begingroup/\endgroup to \bgroup/\egroup. 597
1994-03-11 ltfssdcl.dtx v2.1b
\DeclareSymbolFontAlphabet@: Added check against use of alphabet switch outside of math mode. 734
\SetMathAlphabet@: Changed parameter template in temporary macro to catch check add below. 722
1994-03-12 ltclass.dtx v0.3c
General: Change name from docclass to ltclass 1098
\ProvidesFile: Add \wlog 1110
\ProvidesPackage: Add \wlog . . . 1108
use \@gtempa 1108
1994-03-12 ltdefns.dtx v1.0b
\@reargdef: New defn, in terms of \@yargdef 87
\@yargd@f: Name changed from \XXX@argdef 85
1994-03-12 ltdirchk.dtx v1.0b
General: Change name from dircheck.dtx 1
Minor edits to the typeouts in ltxcheck 1
1994-03-12 ltfloat.dtx v1.0b
\@savemarbox: (DPC) Extra group for color 1039
\@xympar: (DPC) Extra bgroup for color 1040
1994-03-12 ltplain.dtx v1.0b
General: Name changed from lplain. The end of an era 14
1994-03-12 ltplain.dtx v1.0e
General: Replaced remaining width, height, depth by LaTeX macro names to save tokens. 14
1994-03-13 ltcntrl.dtx v1.0c
\@tfor: (DPC) Add \@tf@r so a single group is correctly treated. 426
1994-03-13 ltfilehook.dtx v0.3b
\unqu@tefilef@und: Use new cmd \@addtofilelist 1170
1994-03-13 ltfiles.dtx LaTeX2e
\@addtofilelist: Macro added . . . 506
\listfiles: Reset \@addtofilelist at begin document 508
1994-03-13 ltfssbas.dtx v2.1g
General: add 2ekernel module to omit repeated code 571
1994-03-13 ltfssdcl.dtx v2.1c
General: add 2ekernel module to omit repeated code 702
1994-03-14 ltboxes.dtx v1.0b
\@isavebox: Use \color@setgroup . 931
\@isavepicbox: Use \color@setgroup 931
\color@begingroup: macro added for color support 929
\color@endgroup: macro added for color support 930
\lrbox: Use \color@setgroup 932
\sbox: Use \color@setgroup 931
1994-03-14 ltfloat.dtx 1.0c
\@xympar: (DPC) Use \color@begingroup 1040
1994-03-14 ltfloat.dtx v1.0c
\@endfloatbox: (DPC) Use \color@endgroup 1035
\@footnotetext: (DPC) Use \color@begingroup, add \endgraf 1047
\@savemarbox: (DPC) Use \color@begingroup 1039
\@xfloat: (DPC) Use \color@begingroup 1031
1994-03-15 ltfiles.dtx LaTeX2e
\@missingfileerror: Quit on x or X just like a real error 505
1994-03-15 ltfntcmd.dtx v3.2a
General: Adapted to mass formatting 790
Changed \/ to \@@italiccorr . . 790
Removed \@renewfontswitch . . . 790
Removed defs of short-forms and all sizes except \normalize 790
1994-03-15 ltoutput.dtx v1.0l
\@addtocurcol: Changed \addvspace to \vskip 1270, 1276
\@combinedblfloats: Removed boxmaxdepth setting. 1261
\@make@normalcolbox: Removed boxmaxdepth setting. 1242
\@outputbox@append: \maxdepth changed to \@maxdepth 1246
\@topnewpage: Corrected and amended warning message 1231

Warning added: it should be improved 1231
General: Added some warnings when page gets full of top floats. . . . 1214
Driver added and further tidying. 1214
Removed duplicated code and corrected docstrip options. 1214
Some boxmaxdepth settings removed. 1214
1994-03-16 ltclass.dtx v0.3f
General: Add pkgindoc package . . 1147
1994-03-16 ltfiles.dtx LaTeX2e
`\listfiles`: Move this code directly into `\document` 508
1994-03-16 ltfiles.dtx v1.0c
`\document`: (DPC) directly add file list settings 489
1994-03-16 ltmiscen.dtx v1.0b
`\@verbatim`: Remove `\global\@inlabelfalse` again. . 882
1994-03-28 ltalloc.dtx v1.0d
General: Redefinition of 'new' allocations removed. 421
1994-03-28 ltdirchk.dtx v1.0d
General: Improve documentation 1
1994-03-28 lterror.dtx v1.0d
`\@invalidchar`: (DPC) Comment out (use catcode15 instead) 437
General: Remove test for `\inputlineno` undefined. 433
1994-03-28 ltfiles.dtx v1.0d
`\document`: (DPC) Use `\normalsize` not `\@normalsize` 488
(DPC) remove `\@normalsize check` 488
1994-03-28 ltfloat.dtx v1.0b
`\@caption`: Use `\normalsize` not `\@normalsize` 1029
General: Split further from ltherest.dtx 1026
1994-03-28 ltlists.dtx v1.0b
General: Improve documentation . . 907
1994-03-28 ltmiscen.dtx v1.0c
General: Improve Documentation . . 864
1994-03-28 ltplain.dtx v1.0c
`\newlanguage`: Remove some `\outer` declarations. 17
1994-03-28 ltsect.dtx v1.0b
General: Split further from ltherest.dtx 1012
1994-03-28 lttab.dtx v1.0b
General: Improve documentation . . 947
1994-03-28 ltthm.dtx v1.0a
General: Initial version, split from latex.dtx 1007
1994-03-29 ltcounts.dtx v1.0c
General: Create file from parts of ltmiscen and ltherest. 556
1994-03-29 ltlength.dtx v1.0c
General: Create file ltcntlen from parts of ltmiscen and ltherest. 569
1994-03-29 ltmiscen.dtx v1.0d
General: Remove counter macros to ltcntlen 864
1994-03-29 ltpageno.dtx v1.0c
General: Create file ltcntlen from parts of ltmiscen and ltherest. 839
1994-03-29 ltxref.dtx v1.0c
General: Create file ltcntlen from parts of ltmiscen and ltherest. 840
1994-03-31 ltbibl.dtx v1.0a
General: Initial version of ltidxbib.dtx, split from ltherest.dtx 1053
1994-03-31 ltidxglo.dtx v1.0a
General: Initial version of ltidxbib.dtx, split from ltherest.dtx 1050
1994-04-09 ltcounts.dtx v1.0d
`\@newctr`: `\@nocnterr` now has counter name argument 559
`\addtocounter`: `\@nocnterr` now has counter name argument 558
`\setcounter`: `\@nocnterr` now has counter name argument 557
`\stepcounter`: Use `\addtocounter` to have name checked 559
1994-04-09 ltthm.dtx v1.0b
`\@othm`: Use standard counter error message (FMi) 1009
1994-04-11 ltclass.dtx v0.3g
`\endfilecontents`: Add star form, don't write `\endinput` at the end of the file. 1131
`\ProvidesFile`: Protect against weird catcodes. 1110
1994-04-11 ltfssbas.dtx v2.1h
General: Added `\defaultscriptratio` and `\defaultscriptscriptratio`. ASAJ. 571
`\defaultscriptratio`: Macro added 597
`\defaultscriptscriptratio`: Macro added 597
1994-04-12 ltboxes.dtx v1.0c
General: Remove `\@acci`, now defined in ltplain.dtx 938
Remove `\@dischyph`, now defined in ltinit.dtx 938

1994-04-12 ltdefns.dtx v1.0g
\@dischyph: Define \@dischyph, was previously in ltboxes.dtx 112
1994-04-12 ltplain.dtx v1.0d
General: Define \@acci 30
1994-04-12 ltvers.dtx v1.0b
General: Have version info generated automatically. 35
1994-04-14 ltfntcmd.dtx v3.2b
General: Macros renamed to non-private forms, JB 790
\DeclareOldFontCommand: Renamed from \@newfontswitch 797
1994-04-15 ltboxes.dtx v1.0d
\@isavebox: Added missing percent character. 931
1994-04-17 ltcounts.dtx v1.0e
\@newctr: Use \@nocounterr instead of \@nocnterr 559
\addtocounter: Use \@nocounterr instead of \@nocnterr 558
\setcounter: Use \@nocounterr instead of \@nocnterr 557
1994-04-17 lterror.dtx v1.0h
\@nocounterr: New name for error message, old error message (without arg) kept 434
1994-04-17 ltthm.dtx v1.0c
\@othm: Use new std counter error message (FMi) 1009
\@thm: Use new std counter error message (FMi) 1010
1994-04-18 ltfinal.dtx v0.1b
General: Initialize \textheight, \textwidth and page style . . . 1335
1994-04-18 ltfloat.dtx v1.0d
\@footnotetext: (DPC) Remove Color support 1047
\@savemarbox: (DPC) Remove Color support 1039
1994-04-18 ltfssbas.dtx v2.1i
General: Macro \no@alphabet@help removed again 571
\calculate@math@sizes: Changed message to log only 597
\no@alphabet@error: Use std LaTeX error macro 571
1994-04-18 ltfssdcl.dtx ???
\DeclareMathAlphabet: Pass correct arg (2 not 3) 720
1994-04-18 ltfssdcl.dtx v2.1d
General: Removed surplus \no@alphabet@error (see fam.dtx) 702
1994-04-18 ltfsstrc.dtx v2.3d
General: Changed to new error/warning scheme 671
\font@submax: Changed dimen to macro 691
\fontsubfuzz: Changed dimen to macro 691
\subst@size: \font@submax and \fontsubfuzz now macros 692
1994-04-19 ltpage.dtx v1.0b
General: Improve documentation . 1089
1994-04-20 ltfntcmd.dtx v3.3a
General: Documentation up-dated . 790
New implementation of \nocorr . 790
\check@nocorr@: Macros added . . . 794
\maybe@ic@: \nocorr etc removed from list of tokens to check, leaving only punctuation characters 795
1994-04-20 ltmiscen.dtx v1.0e
\@enddocument@kernel@warnings: Changed logic for producing warning messages 867
1994-04-21 ltboxes.dtx v1.0e
\@iiiminipage: Extra \bgroup for color 940
\@mpfootnotetext: Extra \endgraf for color 941
\endminipage: Extra \egroup for color 940
1994-04-21 ltfinal.dtx v0.1c
General: Added comments, set the catcodes of 128–255. 1332
1994-04-22 ltfssini.dtx v2.1g
\not@math@alphabet: Message changed again 759
1994-04-23 ltfinal.dtx v0.1d
General: Check that \font@submax is still zero 1332
1994-04-24 ltoutput.dtx v1.0m
\@resethfps: Number 2 changed to \tw@ 1296
Warning changed 1296
\@specialoutput: Message changed to give more info and ‘top’ removed 1235
\@topnewpage: Message changed to give more info 1231
Warning message removed as it will be generated later 1231
General: Changed \@normalsize to \normalsize. 1214
Corrected unverbed commands in documentation. 1214
Removed some long lines and other aesthetic changes. 1214

Warning messages changed/corrected. 1214
1994-04-24 ltpictur.dtx v0.1b
General: Removed surplus spaces after `\hbox to` in several cases 974
1994-04-25 ltclass.dtx v0.3h
General: Removed spurious extra ‘:’s at the end of error messages 1098
1994-04-25 ltfloat.dtx v1.0e
`\@largefloatcheck`: Changed warning message to give more info 1035
Command added 1035
General: Changed warning messages 1026
Removed obsolete tracing code . 1026
1994-04-27 ltfsstrc.dtx v2.3e
General: Corrected item that was forgotten in last change. 671
1994-04-28 lterror.dtx v1.0j
`\@inmatherr`: Macro added 437
1994-04-28 lterror.dtx v1.1c
`\@inmatherr`: Replaced `\noexpand` with `\protect`. 437
1994-04-28 ltfssdcl.dtx v2.1e
General: Removed all `\uppercase` in hex num parsing macros 702
1994-04-28 ltlists.dtx v1.0c
`\item`: Replaced `\@ltxnomath` by `\@inmatherr` 919
1994-04-28 ltpictur.dtx v0.1c
`\@multiput`: (DPC) Macro added . . 978
General: bezier curves added 1002
`\multiput`: (DPC) Ignore spaces between)(. 978
`\picture`: (DPC) Ignore spaces before (. 976
1994-04-28 ltplain.dtx v1.0g
General: Turn off overfull box tracing in log 23
1994-04-29 ltclass.dtx v1.0a
General: Change version number to 1 (no other change) 1098
1994-04-29 ltmiscen.dtx v1.0f
`\@verbatim`: `\leavevmode` added . . 882
Change to `\everypar` added 882
1994-04-29 ltoutenc.dtx 1.4a
General: Removed `\EncodingSpecific`. Renamed all the commands. Added `\DeclareTextGlyph` and `\UndeclareTextCommand`. 513
Removed Rokicki’s OT1 variant encoding. Moved the driver to the top. 512
1994-04-30 ltfntcmd.dtx v3.3b
General: Documentation up-dated and tidied 790
Prefix frag@ changed to frag in `\@protecteddef` 790
Title changed 790
Warning changed to info message in `\@protecteddef` 790
1994-04-30 ltoutput.dtx v1.0n
`\@activechar@info`: `\@activechar@warning` changed to `\@activechar@info` 1253
`\@combinedblfloats`: Removed rule in topnewpage case 1261
`\@emptycol`: Empty column action added: `\@emptycol` 1230
`\@flsetnum`: Rogue space removed 1297
`\@specialoutput`: Cut-off point changed to 2`\baselineskip` . . . 1235
Empty column action added: `\@emptycol` 1235
Extra empty column added for twocolumn case 1235
Extra empty column added for twocolumn case (wrong, see below) 1235
`\@topnewpage`: Added setting of `\col@number` 1230
Cut-off point changed to 3`\baselineskip` 1231
Empty column action added: `\@emptycol` 1231
Message changed for Frank 1231
General: `\@activechar@warning` changed to an info message. . . . 1214
Added `\col@number`. 1214
Documentation tidied. 1214
Empty column action added. . . 1214
Fixed bug from `\dblfigrule` with `\@topnewpage`. 1214
Full of floats action improved. . . 1214
`\col@number`: Added `\col@number` 1226
`\onecolumn`: Added setting of `\col@number` 1228
1994-05-01 lterror.dtx v1.0k
`\@latexerr`: (CAR) Added draft `\@latexinfo`. 433
1994-05-01 ltoutenc.dtx 1.4a
General: Added the `\a` command. . . 522
Added the `\SaveAtCatcode` and `\RestoreAtCatcode` commands. . 525
Removed the uc/lc table settings, since the T1 uc/lc table is now the default. 533

Rewrote for the new syntax. . 525, 528
1994-05-01 ltoutenc.dtx v1.4a
General: Removed Rokicki's encoding. 509
Renamed the commands, removed the `\EncodingSpecific` command. Turned all slots into decimal. Added `\a`. 509
1994-05-02 ltcntrl.dtx v1.0l
`\@break@tfor`: Macro added (from ltfiles.dtx) 426
1994-05-02 ltdefns.dtx v1.1f
`\renewcommand`: Removed surplus `\space` in error 87
`\renewenvironment`: Removed surplus `\space` in error 88
1994-05-02 ltfiles.dtx v1.0f
`\@iffileonpath`: `\@break@loop` renamed to `\@break@tfor` 502
`\@obsoletefile`: Make `\@onlypreamble` 506
1994-05-02 ltfinal.dtx v0.1e
General: Added setting the 'letter' catcodes. 1346
Added setting the 'other' catcodes. 1345
Added setting the special catcodes. 1345
Made slot 127 illegal 1346
Set all the catcodes 1332
1994-05-02 ltfinal.dtx v0.1f
General: Set the catcode of control-J. 1345
1994-05-02 ltmiscen.dtx v1.0g
General: Changed 91 to 1991 and moved some bits 864
1994-05-02 ltoutput.dtx v1.0o
`\@resethfps`: Code shortened 1296
General: Code of `\@resethfps` shortened. 1214
1994-05-03 ltbibl.dtx v1.0b
`\nocite`: Make `\nocite` issue a warning for an undefined citation key. 1056
1994-05-03 ltfinal.dtx v0.1f
General: Set the catcode of control-J to be 'other', for use in messages. 1332
1994-05-03 ltfloat.dtx v1.0f
General: (CAR) Added `\@largefloatcheck` 1026
Removed unnecessary braces from arguments of `\@ifnextchar` . . . 1026
`\end@dblfloat`: `\@largefloatcheck` added 1034
`\end@float`: (CAR) Added `\@largefloatcheck` 1033
1994-05-03 ltfssdcl.dtx v2.1f
General: Renamed `\@@DeclareMathDelimiter` to `\@DeclareMathDelimiter` 702
1994-05-03 ltlists.dtx v1.0d
`\@item`: `\hskip` changed to `\kern` . . 920
`\item`: Removed superfluous braces 919
1994-05-03 ltmiscen.dtx v1.0h
`\@centercr`: `\@badcrerr` replaced by `\@nolnerr` 878
1994-05-03 lttab.dtx v1.0d
`\@endpbox`: Use `\@finalstrut` based on depth of `\@arstrutbox` 972
1994-05-04 ltclass.dtx v1.0b
`\NeedsTeXFormat`: Changed wording of the warning 1119
1994-05-04 lterror.dtx v1.0m
`\@badcrerr`: Error message removed 436
1994-05-05 ltbibl.dtx v1.0c
`\@citex`: Set switch for warning and end of run. 1055
`\nocite`: Do not write page number in `\nocite` warning message. . . . 1056
Set switch for warning and end of run. 1056
1994-05-05 ltfinal.dtx v0.1g
General: Added empty errhelp. . . . 1332
`\errhelp`: Set error help empty. . . 1351
1994-05-05 ltfntcmd.dtx v3.3c
`\@@math@egroup`: Corrected `\@fontswitch` and added saved versions 797
General: Corrected `\@fontswitch` . . 790
1994-05-05 ltmiscen.dtx v1.0i
General: Removed braces from ifnextchar and ifstar arguments . 864
1994-05-07 lttab.dtx v1.0c
`\@maxtab`: Changed `\@firsttab` to `\chardef` 951
Changed `\@maxtab` to `\chardef` . 951
General: Removed definition of `\+` . 947
Removed surplus braces from `\@ifnextchar` constructs 947
1994-05-08 ltfntcmd.dtx v3.3d
General: Removed `\@undefinedfonterror` 790
`\normalsize`: Removed `\@undefinedfonterror` 798
1994-05-09 ltfntcmd.dtx v3.3f
General: Replaced all `\next` by `\@let@token` and undo change 3.3e, whatever that was. 790

1994-05-10 ltdefns.dtx v1.0n
General: (ASAJ) Added `\DeclareProtectedCommand`. 80
Added `\DeclareProtectedCommand` 91
Removed braces around `\@ifundefined` argument. ASAJ. 87
`\makeatother`: Added `\makeatletter` and `\makeatother` ASAJ. 112
1994-05-10 lterror.dtx v1.0n
`\@latexerr`: (ASAJ) Added extra blank lines to `\@latexerr`. 433
1994-05-10 ltmiscen.dtx v1.0j
`\@@sverb`: Slight change in error message text. 885
1994-05-11 ltboxes.dtx v1.0f
`\@begin@tempboxa`: Use new `\color@setgroup` concept. 927
`\@iiiminipage`: Use new `\color@setgroup` concept. 940
`\@mpfootnotetext`: Use new `\color@setgroup` concept. 941
Use new `\normalcolor` and `\@finalstrut`. 941
General: Superfluous braces removed from several commands 926
`\color@setgroup`: macro added for color support 929
`\endminipage`: Use new `\color@setgroup` concept. 940
1994-05-11 ltclass.dtx v1.0c
`\endfilecontents`: Add checks for form feed and tab 1131
1994-05-11 ltdirchk.dtx v1.0e
General: Add `\ProvidesFile` as used in fd files. 4
1994-05-11 lterror.dtx v1.0o
`\@latexerr`: (ASAJ) Removed one of the extra blank lines to `\@latexerr`. 433
1994-05-11 ltlogos.dtx v1.0o
`\LaTeX`: Use `\DeclareProtectedCommand`. ASAJ. 484
`\LaTeXe`: Use `\DeclareProtectedCommand`. ASAJ. 484
1994-05-11 ltoutenc.dtx 1.5a
General: Made T1 and OT1 generate packages rather than def files. Renamed the 'package' module to 'teststy'. 512
1994-05-11 ltoutenc.dtx v1.5a
General: Reimplemented `\DeclareTextCommand` using `\@changed@cmd` and `\DeclareProtectedCommand`. . . . 513
Renamed the commands again. Made the encoding part of the command syntax. Added the `\DeclareTextCommand` interface. Used `\DeclareProtectedCommand`. . . . 509
`\DeclareTextAccent`: Reimplemented using `\DeclareTextCommand`. . . . 516
1994-05-11 ltspace.dtx v1.0o
`\hspace`: Use `\DeclareRobustCommand`. ASAJ. 481
1994-05-12 ltboxes.dtx v1.0g
`\@finalstrut`: macro added 945
`\fbox`: New definition, merged with `\framebox` 933
`\framebox`: Merged `\fbox` and `\framebox` 933
`\normalcolor`: macro added for color support 929
1994-05-12 ltdefns.dtx v1.0p
General: (ASAJ) Fixed a bug with `\relax` which was using `\@gobble` before defining it. 80
Fixed a bug with `\relax` which was using `\@gobble` before defining it. 91
1994-05-12 ltfssbas.dtx v2.1j
General: New baselinestretch concept 571
Replaced hand-protected commands by `\DeclareRobustCommand` defs 571
`\f@linespread`: New macro 583
`\fontencoding`: Use `\DeclareRobustCommand`. 581
`\fontfamily`: Use `\DeclareRobustCommand`. 582
`\fontseries`: Use `\DeclareRobustCommand`. 582
`\fontshape`: Use `\DeclareRobustCommand`. 582
`\fontsize`: Redefined to use `\set@fontsize` 583
`\linespread`: New macro 583
`\mathversion`: Use `\DeclareRobustCommand`. 584
1994-05-12 ltfssdcl.dtx v2.1g
General: Allow `\relax` as undefined command 702
Allow `\relax`'ed cmds to be declared 702
1994-05-12 ltfssini.dtx v2.1i
General: Moved `\fontencoding` to fam.dtx 735

Moved \fontfamily to fam.dtx . 735
Moved \fontseries to fam.dtx . 735
Moved \fontshape to fam.dtx . . 735
Moved \fontsize to fam.dtx . . . 735
Moved \mathversion to fam.dtx . 735
Moved \selectfont to tracefnt.dtx 735
1994-05-12 ltfsstrc.dtx v2.3f
\selectfont: Use \DeclareRobustCommand 675
1994-05-12 ltoutenc.dtx 1.5a
General: Removed the \SaveAtCatcode and \RestoreAtCatcode commands. . 525
Rewrote for the new syntax. . 525, 528
1994-05-12 ltoutput.dtx v1.0p
\@writesetup: \normalcoloradded 1254
General: \normalcoloradded in various places (DPC). 1214
1994-05-13 ltboxes.dtx v1.0h
\@arrayparboxrestore: New accent system, use \let not \def 938
1994-05-13 ltcounts.dtx v1.0f
General: Removed \@Ialph 566
Removed \@ialph 566
1994-05-13 ltdefns.dtx v1.0q
General: (ASAJ) Renamed \DeclareProtectedCommand to \DeclareRobustCommand. Removed \@if@short@command. . . 80
(ASAJ) Replaces \space by ‘ ’ in \csname. 80
Renamed \DeclareProtectedCommand to \DeclareRobustCommand. Removed \@if@short@command. Moved to after the definition of \@gobble. 91
1994-05-13 ltdefns.dtx v1.0r
General: (ASAJ) Added logging message to \DeclareProtectedCommand. . . . 80
Added logging message to \DeclareProtectedCommand. . . . 91
1994-05-13 ltdefns.dtx v1.0s
General: (ASAJ) Added \@backslashchar. 80
(ASAJ) Coded \@ifdefinable more efficiently. 80
Coded more efficiently, thanks to FMi. 87
1994-05-13 ltfiles.dtx LaTeX2e
\listfiles: Stop \listfiles being run twice 507
1994-05-13 ltfiles.dtx v1.0g
\document: Added execution of \every@size 488
1994-05-13 ltfinal.dtx v0.1h
General: Added package ot1enc, and defined \@acci, \@accii and \@acciii. 1332
1994-05-13 ltfinal.dtx v1.0h
General: Added output enc stuff . . 1350
1994-05-13 ltfloat.dtx v1.0g
\@footnotetext: (DPC) Add new style color support: \normalcolor 1047
(DPC) Use \@finalstrut 1047
\@xfloat: (DPC) Use \normalcolor 1031
1994-05-13 ltfntcmd.dtx v3.3g
General: Replaced \@protecteddef by \DeclareRobustCommand 790
1994-05-13 ltfssbas.dtx v2.1k
General: Remove File identification ‘typeout’ 571
1994-05-13 ltfssbas.dtx v2.1l
\DeclareFontEncoding: Init encoding change command 575
\define@newfont: Use \@input@ for fd files 588
1994-05-13 ltfssdcl.dtx v2.1h
General: Removed file identification typeout 702
1994-05-13 ltfssini.dtx v2.1j
General: Removed file identification typeout 735
1994-05-13 ltfsstrc.dtx v2.3g
General: Removed typeouts as \ProvidesPackage writes to log. 671
1994-05-13 ltoutenc.dtx v1.5b
General: Added \{, \} and \$. . . . 509
Renamed \DeclareProtectedCommand to \DeclareRobustCommand. 509
Replaces \space by ‘ ’ in \csname. 509
1994-05-13 ltpictur.dtx v0.1d
General: Removed surplus braces from \@if.. constructions 974
1994-05-13 lttab.dtx v1.0d
\@contfield: Color support 953
\@startfield: Color support 953
\@stopfield: Color support 953
\a: moved to ltoutenc 951
1994-05-14 fontdef.dtx v2.1f
General: Removed .def files. 768
1994-05-14 ltfssbas.dtx v2.1m
\enc@update: Macro added 582

1994-05-14 ltfssbas.dtx v2.1n
General: Set defaults for all \f@... 583
\DeclareErrorFont: Don't set \f@encoding 591
\DeclareFontEncoding: Log if encoding is redeclared 575
Only init enc change cmd when new encoding 575
1994-05-14 ltfssini.dtx v2.1k
General: Init error font just before checking for fontdef.cfg 764
\reset@font: Remove surplus braces 761
1994-05-14 ltfsstrc.dtx v2.3h
\selectfont: Added \enc@update . 677
1994-05-14 ltoutenc.dtx 1.5d
General: Moved the driver to the top. 512
1994-05-14 ltoutenc.dtx v1.5c
General: Added the fontenc package 553
Added the fontenc package. 509
Fixed a bug which caused an infinite loop if \f@encoding was incorrectly set. 509, 513
Moved fontsmpl to its own dtx file. 509
1994-05-14 ltoutenc.dtx v1.5d
General: Rewrote \DeclareTextCommand to define its argument to use the current encoding by default, rather than the encoding provided to \DeclareTextCommand. . . . 509, 513
Tidied up the documentation. . . . 509
1994-05-14 ltoutenc.dtx v1.5e
General: Replaced \ENC@cmd by \ENC-cmd. 509
1994-05-15 ltfssbas.dtx v2.1o
General: encoding cmds changed to enc-cmd 571
1994-05-16 fontdef.dtx v2.1g
General: Removed \DeclareFontEncoding for ot1 and t1 and input .def files instead . . 768
1994-05-16 ltalloc.dtx v1.1a
General: (ASAJ) Split from ltinit.dtx. 421
1994-05-16 ltcntrl.dtx v1.0a
General: (ASAJ) Split from ltinit.dtx. 423
1994-05-16 ltdefns.dtx v1.1a
General: (ASAJ) Split from ltinit.dtx. 80
1994-05-16 lterror.dtx v1.1a
General: (ASAJ) Completely new error interface. 427
(ASAJ) Split from ltinit.dtx. . . . 427
1994-05-16 ltfinal.dtx v1.0i
General: moved output enc stuff to lfonts 1350
1994-05-16 ltfssbas.dtx v2.1p
\fontsize: Pass \baselinstretch not \f@linespread 583
\linespread: Remove surplus braces 583
1994-05-16 ltfssini.dtx v2.1m
\@acciii: Define saved versions of accents 765
1994-05-16 ltlogos.dtx v1.1a
General: (ASAJ) Split from ltinit.dtx. 484
1994-05-16 ltmath.dtx v1.0k
\ensuremath: Use \DeclareRobustCommand and add extra braces in math mode 903
1994-05-16 ltoutenc.dtx 1.5h
General: \pounds was still using u rather than ui shape. 525
1994-05-16 ltoutenc.dtx v1.5f
General: enc files now have uc encoding name parts (FMi) 509
Revert code so that the encoding given is used in \DeclareTextCommand (FMi) . . . 509
1994-05-16 ltoutenc.dtx v1.5g
General: Made fontenc.sty use the new mixed-case encoding files. 509
Removed the lowercasing of the filename. 553
1994-05-16 ltoutenc.dtx v1.5h
General: Added \NG, \ng, \TH, \th, \DH, \dh, \DJ and \dj. 509
Added \r (ring accent) and \k (ogonek) accents. 509
Fixed a bug with \pounds. 509
Removed \P from the OT1 definitions file. 509
1994-05-16 ltoutenc.dtx v1.5i
General: Fixed a bug with \d. 509
1994-05-16 ltoutput.dtx v1.0q
\@writesetup: Changed setting of accents (FMi): with the new encoding setup they can use \let. It could also use the new internal commands? 1255
General: Changed setting of accents (FMi). 1214
1994-05-16 ltpar.dtx v1.1a
General: (ASAJ) Split from ltinit.dtx. 438
1994-05-16 ltplain.dtx v1.0h
General: Comment out encoding specific commands 29
Remove \@acci and friends again . 30
Remove unnecessary def for \item 29
\loop: Use Kabelschacht method . . . 27
\m@th: Remove unnecessary space . . . 28

1994-05-16 ltspace.dtx v1.1a
General: (ASAJ) Split from ltinit.dtx. 463
1994-05-17 ltclass.dtx v1.0e
`\@use@ption`: Execute option after removing from list, not before . 1115
1994-05-17 ltdefns.dtx 1.1b
General: (ASAJ) Added the `\@protect@...` commands. 92
1994-05-17 ltdefns.dtx v1.1b
General: (ASAJ) Added definitions for protect. 80
(ASAJ) Removed warnings and logging to lterror.dtx. 80
Added the discussion of protected commands, defined the values that `\protect` should have. 91
1994-05-17 ltdefns.dtx v1.1c
General: (ASAJ) Redid definitions for protect. 80
1994-05-17 lterror.dtx v1.1b
General: (ASAJ) Moved error stuff from ltdefns.dtx. 427
1994-05-17 ltfssini.dtx v2.1n
`\copyright`: Really add extra braces 761
`\nfss@text`: Added braces to allow use in subscripts 761
1994-05-17 ltmath.dtx v1.0i
General: Replaced `\let` by `\gdef`, for indirect definition. 896
1994-05-17 ltoutenc.dtx v1.5j
General: Added braces to `\pounds` so it works as a subscript. 509
1994-05-18 ltdefns.dtx 1.1c
General: (ASAJ) Renamed the commands, and removed one which is no longer needed. 92
1994-05-18 ltdefns.dtx v1.1c
General: Redid the discussion and definitions, in line with the proposed new setting of `\protect` in the output routine. 91
1994-05-18 ltfinal.dtx v0.1j
General: Corrected the lccode for d-bar. 1332
1994-05-18 ltlogos.dtx v1.1b
General: (ASAJ) Added the TEX logo. 484
(ASAJ) Made the LATEX 2ε logo use the text font '2' rather than the math font '2'. 484
1994-05-18 ltoutenc.dtx v1.5k
General: Made dotted-i produce 'i'. . 509
Removed braces from `\pounds` and `\dollar`. 509
Replaced `\defaultencoding` with `\encodingdefault`. 509
1994-05-19 ltbibl.dtx v1.1a
General: Initial version of ltbibl.dtx, split from ltidxbib.dtx 1053
1994-05-19 ltcounts.dtx v1.1a
General: Extracted file from ltcntlen. 556
1994-05-19 ltdefns.dtx v1.1d
General: (RmS) Added definitions for `\@namedef` and `\@nameuse` again. . 80
1994-05-19 ltfinal.dtx v0.1k
General: Removed `\makeat...` . . . 1332
1994-05-19 ltidxglo.dtx v1.1a
General: Initial version of ltidxglo.dtx, split from ltidxbib.dtx 1050
1994-05-19 ltlength.dtx v1.1a
General: Extract file ltlength from ltcntlen. 569
1994-05-19 ltpageno.dtx v1.1a
General: Extract file ltpageno from ltcntlen. 839
1994-05-19 ltplain.dtx v0.1k ltfinal
`\showoutput`: used `\maxdimen` not 99999 31
`\showoverfull`: used `\@ne` not 1 30
1994-05-19 ltxref.dtx v1.1a
General: Extract file ltxref from ltcntlen. 840
1994-05-20 ltdefns.dtx v1.1e
General: Changed command name from `\@checkcommand` to `\CheckCommand`. 80
`\CheckCommand`: Changed name from `\@checkcommand` to `\CheckCommand`. 89
1994-05-20 lterror.dtx v1.1c
General: (ASAJ) Added `\@latex@info@no@line`. 427
(ASAJ) Added missing full stops. 427
(ASAJ) Fixed a bug with `\@inmatherr`. 427
1994-05-20 ltfinal.dtx v0.1l
General: Use new font warning commands 1339
1994-05-20 ltfloat.dtx v1.0h
`\@endfloatbox`: Restore outer value of @nobreak switch. 1035
`\outer@nobreak`: Macro added: default is to do nothing. 1035
1994-05-20 ltfntcmd.dtx v3.3h
General: Use new error commands . 790
1994-05-20 ltfssbas.dtx v2.1q
General: Use new error commands . 571

1994-05-20 ltfsstrc.dtx v2.3i
General: Use new error command names 671
1994-05-20 ltmiscen.dtx v1.0l
`\@writefile`: Added correct setting of `\protect`. 871
1994-05-20 ltmiscen.dtx v1.0m
General: Use new warning commands 864
1994-05-20 ltoutput.dtx v1.0s
`\@writesetup`: Added setting of `\protect` during `\shipout`. . . . 1254
General: Added setting of `\protect` during `\shipout`. 1214
1994-05-20 ltpage.dtx v1.0d
`\markright`: Changed setting for `\protect`. 1090
1994-05-20 ltsect.dtx v1.0c
`\addcontentsline`: Correct setting of `\protect`. 1021
`\addtocontents`: Correct setting of `\protect`. 1022
1994-05-21 ltbibl.dtx v1.1b
General: Use new warning commands 1053
1994-05-21 lterror.dtx v1.1d
General: (ASAJ) Made the error commands robust. 427
1994-05-21 ltfiles.dtx v1.0h
General: Use new error commands . 485
1994-05-21 ltlists.dtx v1.0f
General: Use new error commands . 907
1994-05-21 ltmiscen.dtx v1.0n
General: Use new error commands . 864
1994-05-21 ltsect.dtx v1.0d
General: Use new error commands 1012
1994-05-21 lttab.dtx v1.0f
General: Use new error commands . 947
1994-05-21 ltxref.dtx v1.1b
General: Use new warning commands 840
`\newlabel`: Use new warning commands 843
1994-05-22 ltclass.dtx v1.0f
General: Use new warning and error commands 1094
1994-05-22 ltdefns.dtx v1.1f
General: Use new warning and error cmds 80
1994-05-22 lterror.dtx v1.1e
General: (ASAJ) Replaced bgroup by begingroup in error messages, to stop extra mathords creeping into math mode. 427
1994-05-22 lterror.dtx v1.2a
General: (ASAJ) Made `\GenericError`, `\GenericWarning` and `\GenericInfo` robust. 427
(ASAJ) Replaced \\ and tilde by `\MessageBreak` and `\space`. . . . 427
(ASAJ) Replaced `\@generic@message` and `\@generic@error` by `\GenericError`, `\GenericWarning` and `\GenericInfo`. 427
(ASAJ) Replaces `\string` by `\protect` in some messages. . . . 427
1994-05-22 lterror.dtx v1.2d
`\GenericError`: (DPC) Alternative version added for old TeXs 428
(DPC) New version using long command name. 428
1994-05-22 ltfloat.dtx v1.0i
General: Use new warning commands 1026
1994-05-22 ltoutput.dtx v1.0t
General: Changed warnings and infos to new commands. 1214
1994-05-22 ltpictur.dtx v0.1e
General: Use new warning cmds . . . 974
1994-05-23 ltclass.dtx v1.0h
`\NeedsTeXFormat`: Don't stop completely when format is wrong 1119
`\usepackage`: Remove argument if possible 1118
1994-05-23 ltdirchk.dtx v1.0f
General: Document `\@TeXversion` . . . 1
1994-05-23 ltfsstrc.dtx v2.3j
General: Removed def of `\f@warn@break` 690
1994-05-23 ltoutput.dtx v1.0u
`\@activechar@info`: Added `\MessageBreak` 1253
`\@writesetup`: Changed resetting of `\protect` after shipout to use `\aftergroup` 1254
General: Added `\MessageBreak`. . . 1214
Changed resetting of `\protect` after shipout. 1214
1994-05-24 lterror.dtx v1.2e
`\@latex@info@no@line`: Macro added 431
1994-05-24 lterror.dtx v1.2f
General: (DPC) wrap long lines . . . 427
1994-05-24 ltfntcmd.dtx v3.3i
General: Tidying and typos fixed . . 790
1994-05-24 ltmiscen.dtx v1.0q
`\@currenvline`: Use `\@empty` as outer default 877

1994-05-25 ltdirchk.dtx v1.0g
\filename@parse: Mac parser had " typo for : 12
1994-05-25 ltfntcmd.dtx v3.3j
General: Insertion of \aftergroups to implement \nocorr moved to the end of the group 790
\check@icr: Macros added 794
\check@nocorr@: Insertion of \aftergroups moved and defaults set up for efficiency 794
\DeclareTextFontCommand: \expandafter inserted 792
Insertion of \aftergroups moved 792
1994-05-25 ltoutput.dtx v1.0v
General: Extra documentation. . . . 1214
1994-05-25 ltsect.dtx v1.0e
\@dottedtocline: Put braces around argument 4 (the actual toc entry) to avoid font (and possibly other) changes leaking out to the leaders. 1024
1994-05-25 ltthm.dtx v1.0c
General: Modify documentation . . 1007
1994-05-25 ltvers.dtx v1.0d
General: Remove PRELIMINARY TEST RELEASE from startup banner (spring is here) 35
1994-05-25 ltxref.dtx v1.1c
General: Modify documentation . . . 840
1994-05-26 ltfiles.dtx LaTeX2e
\@missingfileerror: Modify message format 505
1994-05-26 ltlogos.dtx v1.1c
General: Remove \SLiTeX logo 484
1994-05-26 ltmiscen.dtx v1.0r
General: \literal removed 888
1994-05-26 ltplain.dtx v1.1m
\iterate: (CAR) added \long 27
\underbar: (CAR/FMi) changed to use box \tw@ 29
1994-05-26 ltplain.dtx v1.1p
\underbar: (DPC) changed to use \sbox 29
1994-05-29 ltfssdcl.dtx v2.1j
General: Use new error commands . 702
1994-05-31 ltfinal.dtx v1.0n
General: Renamed lthyphen.* to lthyphen.*. 1332
1994-06-01 ltboxes.dtx v1.0i
\@frameb@x: Macro added. 934
\@iframebox: New version, so \width is correct in \framebox 933
\fbox: New version, using \@frameb@x 933
\framebox: New version, so \width is correct in \framebox 933
1994-06-01 ltlogos.dtx v1.1d
\LaTeX: Add \m@th to force math size calculations 484
1994-06-01 ltoutput.dtx v1.0w
General: Tidied up typesetting. . . 1214
1994-06-08 ltfinal.dtx v1.0m
General: Add patch file system . . . 1350
1994-06-09 ltfinal.dtx v1.0n
General: For TEX2, do not set codes for higher half of character table. 1337, 1346
1994-06-09 ltfntcmd.dtx v3.3k
General: Tidying and typos fixed in documentation 790
1994-06-18 ltfntcmd.dtx v3.3l
General: Added check for empty text 790
\check@nocorr@: Added check for empty text 794
1994-06-22 ltfntcmd.dtx v3.3m
General: Removed space from \nfss@text 790
Renamed \check@nocorr 790
\check@nocorr@: Renamed \check@nocorr to \text@command to improve \long error message . 794
\DeclareTextFontCommand: Removed space from \nfss@text 792
1994-06-22 ltmath.dtx v1.2t classes
\mathindent: Set \mathindent at the end of the class instead of at begin document 904
1994-07-20 ltlogos.dtx v1.1e
\LaTeX: Save a few tokens 484
\LaTeXe: Save a few tokens 484
1994-07-20 ltpage.dtx v1.0h
\sloppy: Save a few tokens 1093
1994-09-16 ltfssbas.dtx v2.1s
\nfss@catcodes: Reset [and] as well, just in case 589
1994-10-07 ltoutenc.dtx v1.5l
General: Moved the ogonek accent. . 509
1994-10-11 ltdirchk.dtx v1.0h
\@TeXversion: Check for TeX3.14 . . 13
General: Modify all of ltxcheck again 13
1994-10-12 ltsect.dtx v1.0f
General: Doc. typos 1012
1994-10-14 fontdef.dtx v2.2a
General: New coding 766
1994-10-14 ltfssini.dtx v2.2a
General: New coding for cfg files . . . 735

1994-10-14 ltmiscen.dtx v1.0s
General: Move math to other file . . 864
1994-10-14 ltplain.dtx v1.1a
General: Moved code to other files. . . 14
1994-10-15 ltfssbas.dtx v2.1t
`\extract@alph@from@version`: Warn if math alpha is used outside math 597
1994-10-18 ltboxes.dtx v1.0j
`\@frameb@x`: `\leavevmode` added . . 934
`\@iframebox`: `\leavevmode` moved to `\@frameb@x` 933
`\@parboxto`: Macro added to remove misuse of `\@empty` 936
General: stuff from ltpatch done . . . 926
`\fbox`: `\long` added 933
`\mbox`: `\long` added 927
`\sbox`: `\long` added 931
1994-10-18 ltclass.dtx v1.0j
General: Move `\listfiles` to ltfiles.dtx 1094
1994-10-18 ltdefns.dtx v1.2a
`\@star@or@long`: macro added 83
General: Add extra test for `\endgraf` 80
Add star-forms for all commands . 80
`\renew@environment`: reset end command 88
1994-10-18 ltfiles.dtx v1.0i
`\listfiles`: code moved here from ltclass 507
1994-10-18 ltoutenc.dtx v1.5l
General: Added new definitions of `\patterns` and `\hyphenation`. . . 521
1994-10-18 ltoutenc.dtx v1.5m
General: Added new definitions of `\patterns` and `\hyphenation`. . . 509
1994-10-18 ltsect.dtx v1.0g
`\@dottedtocline`: Added `\normalcolor` for page number 1024
General: Added `\normalcolor` . . . 1012
1994-10-19 ltfssbas.dtx v2.1t
`\DeclareFontEncoding`: Add missing `\relax`. 575
1994-10-23 ltfsstrc.dtx v23.k
`\every@math@size`: Renamed to `\every@math@size` 680
1994-10-23 ltmath.dtx v1.0l
`\@eqnnum`: Added `\normalcolor` since `\eqno` introduces a subgroup of the displayed math group 900
`\ensuremath`: Remove extra braces: but see p 168 of Leslie's book . . 903
1994-10-24 ltboxes.dtx v1.0k
`\fbox`: Inner braces added (to fix latex/1061) 933
1994-10-25 fontdef.dtx v2.2c
General: Added OMSenc.def 768
1994-10-25 ltboxes.dtx v1.0l
`\@isavepicbox`: missing percent (moved from ltpatch) 931
1994-10-25 ltdefns.dtx v1.2b
General: Documentation improvements 80
1994-10-25 ltoutenc.dtx 1.6a
General: Added `\textdollar`, `\textlbrace`, `\textrbrace`, `\textsterling`, `\textunderline`. 528
Removed `\textlbrace`, `\textrbrace`, `\textunderline` to give them their proper names. . . 528
1994-10-25 ltoutenc.dtx v1.6a
General: Added `\ProvideTextCommand`, `\UseTextSymbol`, `\UseTextAccent`, `\DeclareTextSymbolDefault`, `\DeclareTextAccentDefault`, `\DeclareTextCommandDefault`, and `\ProvideTextCommandDefault`. . 509
Added the `\Provide` commands, and the default definitions. . . . 513
Added the defaults. 522
Added the files OT1enc.def, T1enc.def and OMSenc.def. . . . 522
Added the OMS encoding. 534
1994-10-27 ltoutenc.dtx 1.6b
General: Added `\textasciicircum` `\textasciitilde` `\textbackslash` `\textbar` `\textbraceleft` `\textbraceright` `\textcompwordmark` `\textemdash` `\textendash` `\textexclamdown` `\textgreater` `\texthyphenchar` `\texthyphen` `\textless` `\textquestiondown` `\textquotedblleft` `\textquotedblright` `\textquotedbl` `\textquoteleft` `\textquoteright` `\textunderscore` `\textvisiblespace` 528
Added: `\textemdash` `\textendash` `\textexclamdown` `\texthyphenchar` `\texthyphen` `\textquestiondown` `\textquotedblleft`

\textquotedblright
\textquoteleft
\textquoteright 525
1994-10-27 ltoutenc.dtx v1.5d
General: Rewrote \DeclareTextSymbol to define its argument to use the current encoding by default, to fit with \DeclareTextCommand. 513
1994-10-27 ltoutenc.dtx v1.6b
General: Added \textbackslash. . . 534
Added more defaults for OT1. . . 522
Removed the enc.def files 509
Removed the files OT1enc.def, T1enc.def and OMSenc.def. 522
Renamed \textlbrace to \textbraceleft and \textrbrace to \textbraceright. 534
1994-10-29 ltmath.dtx 1.0m
General: ASAJ: Added \DeclareMathOperator. 889
ASAJ: Tidied up documentation. 897
1994-10-29 ltmath.dtx v1.0m
General: ASAJ: Added \mathellipsis, \mathdollar and \mathsterling. 896
ASAJ: Removed \dag, \ddag. . . . 896
ASAJ: Renamed \S and \P to \mathsection and \mathparagraph and made them \mathchardefs. 896
1994-10-29 ltoutenc.dtx v1.6c
General: Added commands like \dots for use in text and math. 522
Renamed \P, \S, \dag and \ddag to \textparagraph, \textsection, \textdagger and \textdaggerdbl. 509
1994-10-30 ltdefns.dtx v1.2c
\@onelevel@sanitize: Macro added 111
General: (CAR)\@onelevel@sanitize added 80
1994-10-30 ltdefns.dtx v1.2f
General: (DPC)\newwrite's moved to ltfiles 80
1994-10-30 ltmath.dtx v1.0n
General: ASAJ: Moved the new commands to ltoutenc. 896
1994-10-30 ltoutenc.dtx v1.6d
General: Added \DeclareTextCompositeCommand. 509
Added \textcircled. . 509, 523, 534
Added \t. 523
Added math commands. 509
Added OML encoding. 509, 523
Added the OML encoding. 534
Made \textless and \textgreater come from OML. 523
Moved math commands here from ltmath. 525
Removed \textregistered. . . . 523
Rewrote \copyright to use \textcircled. 523
1994-10-31 fontdef.dtx v2.2d
General: Added OMLenc.def 768
1994-10-31 fontdef.dtx v2.2e
General: ... and moved further down 768
1994-10-31 ltfloat.dtx v1.1a
\@dblfloat: Major changes since two-column and one-column cases merged 1029
\@dblflset: Macro added 1029
Major changes to parameter parsing, setting of local variables, etc; two-column and one-column cases merged; space hacks moved 1029
\@endfloatbox: (DPC/CAR) Extra box added to remove color resetting from vmode 1035
\@floatboxreset: Macro added . . 1033
\@footnotetext: (DPC/CAR) Move color setting to output routine . 1047
\@savemarbox: (DPC/CAR) Extra box added for color 1039
\@setfps: Macro added 1030
\@xdblfloat: Macros removed: \@dbflt, \@xdblfloat 1035
\@xfloat: (DPC/CAR) Extra box added to remove color resetting from vmode 1031
Major changes, removing setting of local variables, space hacks etc; two-column and one-column cases merged 1030
Reset hook added 1031
\@xympar: (DPC/CAR) Extra box added since needed for floats . . 1040
\fps@dbl: Macro added 1030
1994-10-31 ltoutput.dtx v1.1a
\@topnewpage: (DPC/CAR) Extra box added to remove color resetting from vmode 1230
(DPC/CAR) Use \color@begingroup for color . . 1230
(DPC/CAR) Use \normalcolor 1230
1994-11-02 ltoutenc.dtx v1.6d
General: Wrapped lines longer than 70 characters. 509

1994-11-03 ltclass.dtx v1.0k
General: Move \@missingfileerror to ltfiles 1098
1994-11-03 ltdirchk.dtx v1.0i
General: Generate an error if latex.ltx not used with clean initex 1
1994-11-03 ltfiles.dtx v1.0j
\@missingfileerror: Move here from ltclass 505
1994-11-04 ltboxes.dtx v1.0m
\@mpfootnotetext: Added \protected@edef. ASAJ. 941
1994-11-04 ltdefns.dtx v1.2e
General: Added \set@display@protect to \typeout. ASAJ. 80
Added commands for setting and restoring \protect. ASAJ. 94
Rewrote protected short commands using \x@protect. ASAJ. 92
1994-11-04 lterror.dtx v1.2g
General: Added \set@display@protect to \Generic* commands. ASAJ. . . 427
1994-11-04 ltfiles.dtx v1.0k
\nofiles: Added setting of \protected@write, \makeindex and \makeglossary to \nofiles. ASAJ. 492
\protected@write: Macro added ASAJ. 492
1994-11-04 ltfloat.dtx v1.1b
\@footnotetext: (ASAJ) Added \protected@edef. 1047
\footnotemark: Added \protected@xdef to \footnotemark. 1048
1994-11-04 ltidxglo.dtx v1.1b
\@wrglossary: Added \protected@write to \@wrglossary. 1052
\@wrindex: Added \protected@write to \@wrindex. 1051
General: Removed \if@filesw from \makeindex. 1050
\makeglossary: Removed \if@filesw from \makeglossary. 1051
1994-11-04 ltmiscen.dtx v1.0t
\@writefile: Removed setting of \protect. ASAJ. 871
1994-11-04 ltoutenc.dtx v1.6f
General: Added _. 524
Added \mathunderscore. 525
1994-11-04 ltpage.dtx v1.0e
\markright: Added \@unexpandable@protect. ASAJ. 1090
1994-11-04 ltsect.dtx 1.0h
\@sect: (ASAJ) Added \protected@edef. 1016
General: (ASAJ) Added \protected@xdef to \thanks. . 1012
1994-11-04 ltsect.dtx v1.0h
\addcontentsline: Added \protected@write to \addcontentsline. ASAJ. . . 1021
\addtocontents: Added \protected@write to \addtocontents. ASAJ. 1022
1994-11-04 lttab.dtx v1.0h
\@mkpream: (ASAJ) Added \@unexpandable@protect to \@mkpream. 968
\multicolumn: (ASAJ) added \set@typeset@protect. 963
1994-11-04 ltxref.dtx v1.1d
\label: (ASAJ)Added \protected@edef 844
(ASAJ)Added \protected@write 844
1994-11-05 ltboxes.dtx v1.0n
\@mpfootnotetext: Color resetting for footnotes moved to endminipage: as for main page. 941
\color@endbox: macro added for color support 929
\color@hbox: macro added for color support 929
\endminipage: Color resetting for footnotes moved to here: as for main page. 940
1994-11-05 ltboxes.dtx v1.0o
\@mpfootnotetext: Color groups restored here. 941
1994-11-05 ltfloat.dtx v1.1c
\@dblflset: Add compatibility with old version of \@xfloat. 1029
\@endfloatbox: Use new \color@hbox concept. 1035
\@footnotetext: Removed \normalcolor (again) 1047
\@savemarbox: Use new \color@hbox concept. 1039
\@setfps: Add compatibility with old version of \@xfloat. 1030
\@xfloat: Add compatibility with old version of \@xfloat: but the

arguments, provided at exorbitant cost, are now completely ignored 1030
Use new `\color@hbox` concept. . 1031
`\@xympar`: Use new `\color@hbox` concept. 1040
1994-11-05 ltoutenc.dtx v1.6g
General: Added setting of `\@typeset@protect` to `\patterns` and `\hyphenation`. 521
1994-11-05 ltoutput.dtx v1.1b
`\@topnewpage`: Use new `\color@hbox` concept. 1230
`\@writesetup`: Change protect settings for new-style, protect-free aux-files. 1254
Use new `\color@hbox` concept. . 1254
1994-11-05 ltoutput.dtx v1.1c
`\@begindvi`: Added macro 1260
`\@begindvibox`: Added macro . . . 1225
`\@writesetup`: Add new `\AtBeginDvi` concept 1254
`\AtBeginDvi`: Added macro 1225
1994-11-06 ltfssbas.dtx v2.1u
`\cf@encoding`: New macro 583
`\DeclareFixedFont`: Renamed `\every@size` to `\every@math@size`. 573
1994-11-06 ltfssini.dtx v2.2b
`\@setsize`: Use `\@typeset@protect` 760
1994-11-06 ltfsstrc.dtx v2.3k
`\glb@currsize`: New implementation 679
`\try@simples`: New implementation 691
`\try@size@substitution`: New implementation 690
`\tryis@simple`: New implementation 691
1994-11-07 fontdef.dtx v2.2f
General: (DPC) Add `\DeclareMathSizes` declarations 773
(DPC) Updated to use `\ProvidesFile` 768
1994-11-07 ltfiles.dtx v1.0m
`\document`: Renamed `\every@size` to `\every@math@size`. 488
1994-11-07 ltplain.dtx v1.0l
`\@unused`: move here from ltdefns, remove duplicate `\@mainaux` 21
1994-11-07 preload.dtx v2.1e
General: (DPC) Updated to use `\ProvidesFile` 787
1994-11-09 ltboxes.dtx v1.0p
`\@finalstrut`: Revert `\finalstrut` to 2.09 equivalent (from ltpatch) . . 945
General: more color changes. 926
1994-11-09 ltfssbas.dtx v2.1v
`\@vpt`: (DPC) macros added, from setsize.dtx 599
(DPC) reduce save stack usage latex/1742 599
1994-11-10 ltbibl.dtx v1.1c
General: Fix `\nocite{*}` 1053
`\nocite`: Fix `\nocite{*}` 1056
1994-11-10 ltmath.dtx v1.2v classes
`eqnarray`: Added value of `\parskip` to `\abovedisplayskip` to compensate for negative `\topsep` 906
1994-11-10 ltoutput.dtx v1.1e
`\@writesetup`: Modify `\protect` setting 1254
1994-11-10 ltplain.dtx v1.1b
General: (CAR) added patch to `\loop`. 14
`\iterate`: (CAR) added extra `\relax` 27
1994-11-11 ltspace.dtx v1.2a
`\\`: (DPC) Make robust 467
1994-11-12 ltfntcmd.dtx v3.3o
`\normalsize`: Added `\MessageBreak` 798
1994-11-12 ltlists.dtx v1.2b ltspace
`\endtrivlist`: Changed order of tests to make `\@noitemerror` correct: end of an era. 916
1994-11-12 ltmiscen.dtx v1.0u
`center`: Changed end macro to `\def`: safer and consistent 879
`flushleft`: Changed end macro to `\def`: safer and consistent 880
`flushright`: Changed end macro to `\def`: safer and consistent 881
1994-11-12 ltplain.dtx v1.1c
General: Comment out more encoding specific commands 29
1994-11-12 ltspace.dtx v1.2b
`\addpenalty`: Corrected error message 475
`\addvspace`: Corrected error message 474
1994-11-13 ltspace.dtx v1.2c
`\addpenalty`: Recorrected error message 475
`\addvspace`: Recorrected error message 474
1994-11-14 ltoutput.dtx v1.1f
`\@begindvi`: Use normal box register: why a box? 1260
`\@begindvibox`: Use normal box register: why a box? 1225
`\@writesetup`: Modify new `\AtBeginDvi` concept 1254
General: Removed old definition of `\@testfp`. 1214

1994-11-14 ltspace.dtx v1.2d
\\: (DPC) Macro modified 467
1994-11-14 lttab.dtx v1.0i
\tabularnewline: (DPC) Macro added 962
1994-11-16 fontdef.dtx v2.2h
General: (DPC) Removed \{ and \} 768
1994-11-17 ltboxes.dtx v1.0q
General: \@tempa to \reserved@a . . 926
1994-11-17 ltclass.dtx v1.0l
General: \@tempa to \reserved@a . 1094
1994-11-17 ltcntrl.dtx v1.0b
General: \@tempa to \reserved@a . . 423
1994-11-17 ltdefns.dtx v1.0g
General: \@tempa to \reserved@a . . . 80
1994-11-17 ltdirchk.dtx v1.0j
General: \@tempa to \reserved@a 1
1994-11-17 lterror.dtx v1.2h
General: \@tempa to \reserved@a . . 427
1994-11-17 ltfiles.dtx v1.0n
General: \@tempa to \reserved@a . . 485
1994-11-17 ltfinal.dtx v1.0o
General: \@tempa to \reserved@a . 1332
1994-11-17 ltfloat.dtx v1.1e
General: \@tempa to \reserved@a . 1026
1994-11-17 ltfntcmd.dtx v3.3p
General: \@tempa to \reserved@a . . 790
1994-11-17 ltfssbas.dtx v2.1w
General: \@tempa to \reserved@a . . 571
1994-11-17 ltfssdcl.dtx v2.1m
General: \@tempa to \reserved@a . . 702
1994-11-17 ltfsstrc.dtx v2.3l
General: \@tempa to \reserved@a . . 671
1994-11-17 ltmath.dtx v1.0o
General: \@tempa to \reserved@a . . 889
1994-11-17 ltmiscen.dtx v1.0v
General: \@tempa to \reserved@a . . 864
1994-11-17 ltoutenc.dtx v1.6h
General: (DPC) \@tempa to \reserved@a 509
1994-11-17 ltoutput.dtx v1.1h
General: \@tempa to \reserved@a. 1214
1994-11-17 ltpictur.dtx v1.0f
General: \@tempa to \reserved@a . . 974
1994-11-17 ltsect.dtx v1.0i
General: \@tempa to \reserved@a . 1012
1994-11-17 lttab.dtx v1.0j
General: \@tempa to \reserved@a . . 947
1994-11-18 ltboxes.dtx v1.0r
\color@vbox: macro added for color support 929
1994-11-18 ltfinal.dtx v1.0n
General: re-allow slots 127–255 . . . 1346
1994-11-18 ltfssbas.dtx v2.1x
General: (DPC) use \reserved@f not \next 571
1994-11-18 ltfssdcl.dtx v2.1m
\DeclareMathDelimiter: (DPC) \expandafter instead of \next . 727
1994-11-18 ltfsstrc.dtx v2.3m
General: \next to \reserved@f . . . 671
1994-11-18 ltmath.dtx v1.0p
\phantom: (DPC) color support . . . 891
(DPC) use \expandafter instead of \next 891
\prime@s: (DPC) use \@let@token instead of \next and \expandafter instead of \nxt . . 896
\smash: (DPC) color support 892
(DPC) use \expandafter instead of \next 892
1994-11-21 ltfloat.dtx v1.1f
\@endfloatbox: Added reset of minipage flag 1035
Corrected position of \outer@nobreak 1035
\@marginparreset: Macro added . 1039
\@savemarbox: Added \@setminipage etc 1039
Added resetting of size and font 1039
Changed to \color@vbox 1039
Use \@setnobreak etc 1039
\@setminipage: Macro added 1033
\@setnobreak: Macro added 1033
\@xfloat: Added \@setminipage . 1031
Added resetting of size and font 1031
Changed to \color@vbox so that large floats overflow at the bottom 1031
Missing percents reinserted after 4, 8: these are not numbers. 1030
Use \@setnobreak 1031
\@xympar: Changed to \color@vbox 1040
1994-11-21 ltoutput.dtx v1.1i
\@addtocurcol: Added \if@nobreak test before float box 1270, 1276
\@specialoutput: Added \if@nobreak test 1237
\@topnewpage: Changed to \color@vbox 1230
1994-11-22 ltfssdcl.dtx v2.1o
General: wrap long lines 702
1994-11-22 ltoutenc.dtx v1.6i
General: Corrected \dots so that there's no kerning in monowidth fonts. 509

Corrected typo with `\mathunderscore`. 509
Fixed empty accents. Again. . . . 509
1994-11-24 ltdefns.dtx v1.2h
`\@newenv`: Added test for `\endgraf` . . 88
1994-11-25 ltplain.dtx v1.1f
General: (DPC) Comment out lots of obsolete code 14
1994-11-26 ltfloat.dtx v1.1b
`\footnote`: (ASAJ) Added `\protected@xdef`. 1046
1994-11-28 ltcntrl.dtx v1.0c
General: Documentation improvements 423
1994-11-30 ltfiles.dtx v1.0o
`\@dofilelist`: Macro added 508
`\listfiles`: Use `\@dofilelist` . . . 507
`\nofiles`: There is no `\@gobblethree`... 492
1994-11-30 ltfssbas.dtx v2.1y
`\fontshape`: Use `\@current@cmd` in `\@@enc@update`. ASAJ. 582
1994-11-30 ltmath.dtx 1.0q
General: ASAJ: `\DeclareMathOperator` moved to AMSLaTeX. 889
1994-11-30 ltmiscen.dtx v1.0w
`\@enddocument@kernel@warnings`: (DPC) Do warnings even for `\nofiles` 867
`\enddocument`: (DPC) Use `\@dofilelist` 866
1994-11-30 ltoutenc.dtx 1.7a
General: Redefined `\a` for the new scheme. 522
1994-11-30 ltoutenc.dtx v1.6g
General: Removed new definitions of `\patterns` and `\hyphenation`, since encoding-specific commands now expand in the mouth. 521
1994-11-30 ltoutenc.dtx v1.7a
General: Added new code for encoding-specific commands. These now expand in the mouth, which means that ligaturing and kerning can happen. 509
Always load the enc.def file, so that the default encoding for the commands will change. 553
Redefined `\@changed@cmd` to expand in the mouth. 513
Removed `\@changed@x@mouth` since `\@changed@x` now expands in the mouth. 513
Rewrote `\@text@composite` so it allows an empty argument, or an argument containing lots of commands. 516
1994-12-01 ltfinal.dtx v1.0p
General: Renamed lthyphen.* to hyphen.*. 1332
1994-12-01 lthyphen.dtx v1.0g
General: Rename lthyphen.ltx/cfg to hyphen.ltx/cfg 1330
1994-12-01 ltplain.dtx v1.1g
General: (DPC) More doc changes . . 14
1994-12-02 fontdef.dtx v2.2i
General: Commented out `\ldots`. ASAJ. 766
1994-12-02 ltfssini.dtx v2.2c
`\copyright`: `\copyright` is now in ltoutenc. ASAJ 761
1994-12-02 ltlists.dtx v1.0e
`\@trivlist`: RmS: Added check for looping 915
1994-12-02 ltoutenc.dtx 1.7b
General: Redefined `\a` properly. . . . 522
1994-12-02 ltoutenc.dtx v1.7b
General: Fixed a bug with `\a`. 509
1994-12-04 lthyphen.dtx v1.0h
General: Documentation edits for /1989 1330
1994-12-05 ltoutenc.dtx v1.7c
General: Added braces to `\textcircled`. 509
1994-12-06 ltfssbas.dtx v2.1z
`\DeclareFontEncoding`: use `\nfss@catcodes` 575
`\nfss@catcodes`: Added tab char as well 589
1994-12-08 ltoutenc.dtx v1.7d
General: Added `\null` and `\sh@ft` to `\b` and `\d`. 509
1994-12-08 lttab.dtx v1.0k
`\@array`: Add `\tabularnewline` . . . 961
`\tabularnewline`: (DPC) Made it `\relax` 962
1994-12-09 ltbibl.dtx v1.1d
`\bibliographystyle`: (DPC) Allow use in preamble. 1056
1994-12-10 ltfloat.dtx v1.1g
`\@dblfloat`: Old version reinstated temporarily 1029
`\@dblflset`: Macro removed temporarily 1029
Old version reinstated temporarily 1029

\@setfps: Macro removed temporarily 1030
\@xdblfloat: Macros reinserted temporarily 1035
\@xfloat: Old version reinstated temporarily 1030
Sanitization added temporarily . 1030
General: Some temps reinserted temporarily 1026
\fps@dbl: Macro removed temporarily 1030
1994-12-10 ltfntcmd.dtx v3.3q
\@@math@egroup: Don't read arguments 797
\check@nocorr@: Use \space command for comparison 794
1994-12-10 ltfssdcl.dtx v2.1p
\document@select@group: Surround with braces (add fourth arg) . . . 708
\select@group: Surround with braces (add fourth arg) 705
1994-12-10 ltoutenc.dtx v1.7e
General: Added documentation for the OML encoding. 509
Replaced width with \@width and ditto height in vrules. 509
1994-12-14 ltoutenc.dtx v1.7f
General: Added braces to \copyright so it works unbraced in subscripts. 509
Added check for math mode in \@changed@cmd. 509
Commented out \textasciicircum, \textasciitilde, \textbackslash, \textbar, \textgreater, \texthyphenchar, \texthyphen and \textless to save memory. 509
1995-01-12 ltmath.dtx v1.2y classes
\@eqnnum: Added \normalcolor . . . 904
1995-03-03 ltoutenc.dtx 1.7g
General: Corrected an error in documentation referring to the tabular rather than the tabbing environment. 522
1995-04-02 ltfntcmd.dtx v3.3r
\@@math@egroup: Read them again to be able to add \relax. 797
1995-04-02 ltfssdcl.dtx v2.1q
\document@select@group: fix problem for pr/1275 708
\select@group: fix problem for pr/1275 705
\set@mathdelimiter: fix pr/1329 . . 730
1995-04-02 ltfssini.dtx v2.2d
\not@math@alphabet: add \noexpand to second part of message 759
1995-04-21 ltclass.dtx v1.0m
\DeclareOption*: Made long /1498 1112
\endfilecontents: Close input check stream: latex/1487 1131
1995-04-21 ltfinal.dtx v1.0q
General: Allow initial patch level 0 1350
1995-04-21 ltoutenc.dtx v1.7h
General: Added \null \k latex/1274 509
1995-04-22 ltfiles.dtx v1.0p
\includeonly: Allow blanks in argument 493
1995-04-22 ltmiscen.dtx v1.0x
General: Removed extra def of \@gobble 864
1995-04-23 ltsect.dtx v1.0j
\addcontentsline: Use \contentsline internally. 1021
1995-04-24 ltbibl.dtx v1.1e
\@citex: Add \mbox to undefined case: latex/1239. 1055
1995-04-24 ltbibl.dtx v1.1f
\bibcite: Make \@onlypreamble /1388. 1054
1995-04-24 ltcntrl.dtx v1.0d
\@for: Don't expand second argument with \edef: /1317 (DPC) 426
1995-04-24 ltoutput.dtx v1.1j
\fl@tracemessage: Do not add to kernel unless 'trace' specified . . 1292
1995-04-24 ltoutput.dtx v1.1l
\@begindvibox: Add \vbox latex/1392 1225
\@writesetup: Reset \\ latex/1451 (DPC) 1255
1995-04-24 ltpage.dtx v1.0f
\fussy: reset \emergencystretch latex/1344 1093
1995-04-24 ltplain.dtx v1.1h
\newlanguage: Remove remaining \outer declarations. 17
1995-04-24 ltxref.dtx v1.1e
\newlabel: Make \@onlypreamble for /1388. 843
1995-04-25 ltdefns.dtx v1.2i
\@check@c: Make \long for latex/1346 90
\new@environment: Parse arguments slowly but safely /1507 88
1995-04-25 ltfiles.dtx v1.0q
\document: Removed execution of \every@size latex/1407 488

1995-04-25 ltsect.dtx v1.0k
\@dottedtocline: Added \hbox around dots. 1024
1995-04-27 ltboxes.dtx v1.0s
\@frameb@x: Move \leavevmode for graphics/1512 934
\@iframebox: Move \leavevmode for graphics/1512 933
\@iirsbox: Move \leavevmode for graphics/1512 944
\@irsbox: Move \leavevmode for graphics/1512 944
\fbox: Move \leavevmode for graphics/1512 933
\raisebox: Move \leavevmode for graphics/1512 944
1995-04-27 ltfiles.dtx v1.0r
\document: Added \global to support groups in hook 489
1995-04-27 ltmiscen.dtx v1.0y
\enddocument: \@checkend moved after hook 865
1995-04-27 ltplain.dtx v1.1i
General: Move \hang and \textindent to latex209.def 29
1995-04-29 ltcntrl.dtx v1.0e
General: Moved init of \protect to ltdefns.dtx 426
Removed unused defs for \@setprotect and \@resetprotect 426
1995-04-29 ltdefns.dtx v1.2j
\protect: Init \protect here 95
1995-04-29 ltpar.dtx v1.1b
General: (TO) Comments clean-up. 438
1995-05-02 ltsect.dtx v1.0l
\@dottedtocline: Don't reset to \rmfamily 1024
1995-05-03 ltsect.dtx v1.0m
General: TO: Promoted documentation to doc.sty standard 1012
1995-05-06 ltsect.dtx 1.0n
\@seccntformat: Use \quad instead of \hskip 1018
\@sect: Added \relax after \@seccntformat just in case . . 1016
1995-05-07 ltboxes.dtx v1.0t
General: Use \hb@xt@ 926
1995-05-07 ltdefns.dtx v1.2k
\hb@xt@: Macro added 81
1995-05-07 ltmath.dtx v1.0r
General: Use \hb@xt@ 889
1995-05-07 ltoutput.dtx v1.1m
General: Use \hb@xt@. 1214
1995-05-07 ltpictur.dtx v1.0g
General: Use \hb@xt@ 974
1995-05-07 ltplain.dtx v1.1j
General: Use \hb@xt@ 14
1995-05-07 ltsect.dtx v1.0o
General: Use \hb@xt@ 1012
1995-05-07 lttab.dtx v1.0l
General: Use \hb@xt@ 947
1995-05-08 ltbibl.dtx v1.1g
\@citex: Use \@firstofone 1055
\bibitem: Removed unnecessary braces 1054
\nocite: Use \@firstofone 1056
1995-05-08 ltdefns.dtx v1.2k
\typein: Use \@firstofone 82
1995-05-08 ltdefns.dtx v1.2l
\typein: Remove unnecessary braces 82
Replace \def by \let 82
1995-05-08 ltfsstrc.dtx v2.3n
\ifnot@nil: Use \@firstofone . . . 685
1995-05-11 fontdef.dtx v2.2j
General: Updates to some plain macros 766
1995-05-12 ltclass.dtx v1.0n
\DeclareOption*: Use \toks@ to remove need to double hash /1557 1112
1995-05-12 ltfloat.dtx v1.1h
\@footnotemark: Add \nobreak to allow hyphenation. latex/1605 . 1048
1995-05-12 ltpictur.dtx v1.0h
\pictur@: Macro added for latex/1355 976
1995-05-12 ltvers.dtx v1.0e
General: Add autoload docstrip guards 35
Check for format older than 1 year 35
1995-05-13 ltfsstrc.dtx v2.3o
General: Use single hash mark in \DeclareOption 672
1995-05-16 ltfloat.dtx v1.1i
\@makefnmark: Now use \textsuperscript. 1044
\textsuperscript: Command added./pr1503 1044
\thefootnote: Streamlined parts of code. 1043
1995-05-17 ltboxes.dtx v1.0u
\@irsbox: Removed surplus braces . 944
1995-05-17 ltdefns.dtx v1.0o
\g@addto@macro: Make long for latex/1522 114

1995-05-17 ltlists.dtx v1.0g
\@item: Removed surplus braces . . . 921
\@nbitem: Removed surplus braces . 922
enumerate: Use \thr@@ and remove surplus braces 923
itemize: Use \thr@@ 923
1995-05-18 ltfloat.dtx v1.1j
\@makefnmark: Added \normalfont. 1044
\thempfootnote: Added \itshape. 1043
1995-05-19 ltpictur.dtx v1.1a
General: Support autoloading feature 974
1995-05-20 ltcounts.dtx v1.1b
\@definecounter: Streamlined code 559
\@fnsymbol: Allowing both text and math 567
\fnsymbol: Streamlined code 566
1995-05-20 ltcounts.dtx v1.1c
\@definecounter: And do it right . 559
1995-05-20 ltfloat.dtx v1.1k
\@makefnmark: Moved \normalfont back and use \@textsuperscript 1044
Moved \normalfont to \textsuperscript 1044
\textsuperscript: Use \normalfont. 1044
1995-05-21 ltfssdcl.dtx v2.1t
\DeclareMathRadical: Allow for undefined cs names 731
1995-05-21 ltlists.dtx v1.0f
General: Moved to doc.sty standard 907
1995-05-21 ltmath.dtx v1.0r
\@sqrt: Use \sqrtsign 900
General: Remove \mathhexbox from this file 894
Update some plain macros 889
\lefteqn: Use \rlap 903
\r@@t: Use \sqrtsign instead of \sqrt 891
1995-05-21 ltoutenc.dtx v1.7h
\@inmathwarn: Added several \@onlypreamble 514
1995-05-21 ltoutenc.dtx v1.7j
General: Updated some plain macros 526
1995-05-21 ltplain.dtx v1.1j
General: Moved some code to other files 14
1995-05-22 ltplain.dtx v1.1k
General: Definitions of \footins and \footnoterule moved to ltfloat. . 30
1995-05-22 lttab.dtx v1.1a
General: Support autoloading feature 947
1995-05-23 ltfssini.dtx v2.2e
\newfont: Font assignment made local again. 759
1995-05-24 ltdefns.dtx v1.1l
\newif: (DPC) New implementation . 89
1995-05-24 ltdefns.dtx v1.2m
\typein: (DPC) New implementation 82
1995-05-24 ltfloat.dtx v1.1l
\@textsuperscript: Command added. 1044
General: Moved definition of \footins and \footnoterule from ltplain. 1043
\textsuperscript: Use \@textsuperscript 1044
1995-05-24 ltfssbas.dtx v3.0a
General: (DPC) Make file from previous file, fam.dtx 1995/05/20 v2.2d 571
\mathgroup: (DPC) No need to redefine \newfam as not outer . . 571
1995-05-24 ltfsscmp.dtx v3.0a
General: (DPC) Make file from previous file, fam.dtx 1995/05/20 v2.2d 697
1995-05-24 ltfssdcl.dtx v3.0a
General: (DPC) Make file from previous file, latint.dtx 1995/05/21 v2.1t 702
1995-05-24 ltfssini.dtx v3.0a
General: (DPC) Make file from previous file, lfonts.dtx 1995/05/23 v2.2e 735
\cal: (DPC) Remove definition . . . 765
\mit: (DPC) Remove definition . . . 765
1995-05-24 ltfsstrc.dtx v3.0a
General: (DPC) Make file from previous file, tracefnt 1995/05/16 v2.3o 671
1995-05-24 ltfsstrc.dtx v3.0b
General: (DPC) Fix \ProvidesFile usage 671
1995-05-25 ltclass.dtx v1.0p
\endfilecontents: Delete \filec@ntents after preamble . 1131
1995-05-25 ltfilehook.dtx v1.0t
\unqu@tefilef@und: (CAR) added \long 1170
1995-05-25 ltfiles.dtx v1.0s
\document: Added check for \topskip zero 489
1995-05-25 ltfiles.dtx v1.0t
\@iffileonpath: (CAR) added \long 502
\document: Corrected typo 489
\IfFileExists@: (CAR) added \long 500

`\nofiles`: (CAR) added `\long` 492
`\protected@write`: (CAR) added `\long` 492
1995-05-25 ltfloat.dtx v1.1m
`\@savemarbox`: (CAR) Resettings moved to hook 1039
`\@xfloat`: (CAR) Resettings moved to hook 1031
1995-05-25 ltlists.dtx v1.0i
`\endtrivlist`: Macros moved from ltspace.dtx 916
1995-05-25 ltmath.dtx v1.3c classes
`\@eqnnum`: replace `\reset@font\rmfamily` with `\normalfont` (PR 1578) 904
1995-05-25 ltspace.dtx v1.2f
`\@vbsphack`: (CAR) not used so 'removed'. 473
`\@vspacer`: (CAR) `\@restorepar` added to avoid possible infinite tail recursion caused by a typo in the argument. 478
(CAR) macros modified to be more efficient 478
General: Macros moved to ltlists.dtx 463
1995-05-26 ltdefns.dtx v1.2n
`\@gobblefour`: (CAR) Added `\longs` 90
1995-05-26 ltmath.dtx v1.0s
`\@eqnnum`: Removed `\rmfamily` (PR 1578), replaced `\reset@font` with `\normalfont` 900
1995-05-26 ltpage.dtx v1.0g
`\ps@plain`: removed `\rmfamily` (PR 1578) 1090
1995-05-27 ltfssbas.dtx v3.0b
`\mathgroup`: (FMi) But a need to define `\new@mathgroup` 571
1995-06-05 fontdef.dtx v2.2k
General: Moved math commands from ltoutenc.dtx. 784
1995-06-05 ltfinal.dtx v1.0r
General: Added `\MakeUppercase` and `\MakeLowercase`. 1332
1995-06-05 ltoutenc.dtx v1.7k
`\@inmathwarn`: Removed `\protected@cmd` and replaced with explicit `\noexpand`. 514
General: Allowed `\ProvideTextCommandDefault` after the preamble. 515
Commented out `\textless` and `\textgreater`. 523
Moved math commands to fontdef.dtx. 525
Save some tokens in `\textvisiblespace` and `\textunderscore`. 523
1995-06-06 ltfinal.dtx v1.0s
General: Made `\MakeUppercase` and `\MakeLowercase` brace their argument. 1332
1995-06-09 ltoutenc.dtx v1.7l
`\DeclareTextComposite`: Rewrote `\DeclareTextComposite` to define the composite as a no-argument command rather than a two-argument command. 517
1995-06-11 ltspace.dtx v1.2g
`\restorecr`: (CAR) `\relax` added to stop silent eating of *. 483
1995-06-13 ltfinal.dtx v1.0t
General: Add patch level string more carefully 1350
Call `\errorstopmode` 1353
1995-06-13 ltpictur.dtx v1.1b
General: Use `\ProvidesFile` in autoload 974
1995-06-14 lttab.dtx v1.1b
General: Use `\ProvidesFile` in autoload 947
1995-06-15 ltfssbas.dtx v3.0c
General: (DPC) minor documentation changes 571
1995-06-15 ltfsscmp.dtx v3.0b
General: (DPC) minor documentation edits 697
1995-06-15 ltfssdcl.dtx v3.0b
General: (DPC) minor documentation changes 702
1995-06-19 ltbibl.dtx v1.1h
`\bibcite`: Call `\@newl@bel` so repeated keys produce better warning. 1054
1995-06-19 ltclass.dtx v1.0q
`\documentclass`: Don't redefine `\usepackage` in compat mode for /1634 1116
1995-06-19 ltxref.dtx v1.1e
`\newlabel`: Use `\@newl@bel` to share code with `\bibcite` 843
1995-06-28 ltfssini.dtx v3.0b
General: (DPC) Fix documentation typos 735
1995-06-28 ltmath.dtx v1.0t
General: minor doc edits 889
1995-07-02 ltplain.dtx v1.1n
General: Removed surplus 'by' and '=' in various places 14

\offinterlineskip: Replaced 1000 by \@m 27
\showoutput: Use \showoverfull to save space 31
\tracingall: Use \showoutput to save space 31
1995-07-03 ltdefns.dtx v1.2o
\set@typeset@protect: Use \@typeset@protect for init 94
1995-07-03 ltfntcmd.dtx v3.3s
\t@st@ic: Use clean interface for jump 796
1995-07-05 ltfntcmd.dtx v3.3s
\t@st@ic: Renamed from \test@next 796
1995-07-05 ltspace.dtx v1.2h
\@gnewline: Use \break 469
\@no@pgbk: Macro replaces \@pgbk and \@nopgbk 467
\nopagebreak: Reimplemented both using \@no@pgbk 466
1995-07-09 ltcntrl.dtx v1.0f
\@iforloop: Reimplemented using Kabelschacht method 426
\@iwhiledim: Reimplemented using Kabelschacht method 424
\@iwhilenum: Reimplemented using Kabelschacht method 424
\@iwhilesw: Reimplemented using Kabelschacht method 424
\@tfor: Reimplemented using Kabelschacht method 426
1995-07-09 ltlists.dtx v1.0j
enumerate: Use \expandafter 923
itemize: Use \expandafter 924
1995-07-12 ltpictur.dtx v1.1d
General: allow 2e commands in 209 mode. latex/1737 974
1995-07-13 ltdefns.dtx v1.0p
General: Updates to documentation . 80
1995-07-13 ltfiles.dtx v1.0u
General: Updates to docu 485
1995-07-13 ltfssbas.dtx v3.0d
\@@defaultsubs: macro added 594
\@defaultsubs: macro added 594
General: minor documentation changes 571
\wrong@fontshape: Change a macro not a switch to flag default font substitutions 593
1995-07-13 ltmiscen.dtx v1.0z
\@centercr: Use \nobreak 878
\@enddocument@kernel@warnings: Use \@defaultsubs instead of switch 867
\@writefile: Added missing percent and use \relax in the THEN case 871
\@xobeysp: Use \nobreak 881
General: Improve Documentation . . 864
\enddocument: Set \@setckpt to \@gobbletwo instead of defining it by hand 865
Shorten redefinition of \bibcite and \newlabel 866
1995-07-14 ltbibl.dtx v1.1i
\bibcite: Remove \@onlypreamble so still defined in new \enddocument 1054
1995-07-14 ltxref.dtx v1.1g
\newlabel: Remove \@onlypreamble so still defined in new \enddocument 843
1995-07-19 ltfssini.dtx v3.0d
General: (DPC) TeX2 support 764
1995-07-20 ltboxes.dtx v1.0v
\@isavebox: Use \sbox 931
\@isavepicbox: Use \sbox 931
1995-07-21 ltoutput.dtx v1.1o
\@writesetup: Command added . . 1254
New, experimental, versions: need in-lining 1254
1995-08-09 ltmath.dtx v1.0u
General: Added code for class options leqno and fleqn 904
1995-08-11 ltlength.dtx v1.1b
General: Doc typos fixed for latex/753 569
1995-08-16 ltcntrl.dtx v1.0g
\@break@tfor: Made long 426
\@forloop: Made defs long 426
\@fornoop: Made defs long 426
\@iforloop: Made defs long 426
\@iwhiledim: Made defs long 424
Removed \@whilenoop 424
\@iwhilenum: Made defs long 424
Removed \@whilenoop 424
\@iwhilesw: Removed \@whileswnoop 424
\@tfor: Made defs long 426
1995-08-16 ltfiles.dtx v1.0v
\document: set \@maxdepth 489
set \do globally 489
set \topskip globally 489
1995-08-24 ltfssbas.dtx v3.0f
General: Added autoload code 571
1995-08-24 ltfsstrc.dtx v3.0c
General: Macro \gobble@font@spec removed 685
\tryis@simple: 692

1995-08-25 ltoutput.dtx v1.1p
General: Support autoloading feature (FMi). 1214
1995-09-01 lterror.dtx v1.2i
General: Add autoload support . . . 427
1995-09-01 ltplain.dtx v1.1m
`\empty`: Use `\let` to save space 25
`\I`: Use `\let` to save space 24
1995-09-14 ltplain.dtx v1.1o
General: Moved `\multispan` to lttab.dtx 14
1995-09-14 lttab.dtx v1.1c
`\cline`: (DPC) New implementation 971
1995-09-15 ltfssini.dtx v3.0e
General: (DPC) Modify TeX2 message 764
1995-09-19 ltmiscen.dtx v1.1a
`\verb`: Put `\@noligs` after `\verbatim@font` where it belongs. 887
1995-10-01 ltfiles.dtx LaTeX2e
`\@addtofilelist`: Macro added . . . 506
1995-10-02 ltdefns.dtx v1.2q
`\@ifnch`: Use `\@let@token` for internal/924, save `\reserved@e` . 110
`\@ifnextchar`: Use `\@let@token` . . 110
`\@protected@testopt`: Macro added . 85
`\@testopt`: Macro added 85
`\@xargdef`: New implementation, using `\@test@opt` 84
1995-10-03 fontdef.dtx v2.2l
General: `\@@sqrt` from patch file for /1701 766
1995-10-03 ltdefns.dtx v1.2r
`\typein`: Add missing `\@typein` for /1710 (from patch file) 82
1995-10-03 ltpictur.dtx v1.1e
General: New autoload code 974
1995-10-04 ltfssbas.dtx v3.0g
General: Modify autoload code 571
1995-10-04 ltfsstrc.dtx v3.0d
General: (DPC) Modify autoload code 671
1995-10-04 lttab.dtx v1.1d
General: Modify autoload support . 947
1995-10-06 ltfiles.dtx v1.0w
`\@missingfileerror`: Autoload error 505
1995-10-09 lterror.dtx v1.2j
General: Modify autoload support . 427
1995-10-09 ltoutenc.dtx v1.7m
`\@inmathwarn`: Autoload error 515
1995-10-10 ltfssbas.dtx v3.0h
`\showhyphens`: Use `\normalfont` and make color safe, and autoloadable 598
1995-10-10 ltfssdcl.dtx v3.0c
`\non@alpherr`: (DPC) autoload error message 706
1995-10-10 ltplain.dtx v1.1r
General: Autoload tracing code 14
1995-10-10 ltthm.dtx v1.0f
General: Make `\newtheorem` 'only preamble' 1007
1995-10-11 ltoutput.dtx v1.1r
`\clearpage`: Added a check so that it does not lose the argument of `\twocolumn[...]` 1227
1995-10-16 ltbibl.dtx v1.1j
`\cite`: (DPC) Make robust 1054
1995-10-16 ltboxes.dtx v1.0w
General: Clarify makebox description 926
1995-10-16 ltdefns.dtx v1.2u
`\@ifstar`: (DPC) New implementation, for /1910 111
`\new@command`: (DPC) Use `\@testopt` /1911 84
`\new@environment`: (DPC) Use `\@testopt` /1911 88
`\typein`: (DPC) Use `\@testopt` /1911 82
1995-10-16 ltfssini.dtx v3.0f
`\reset@font`: Added `\relax` after `\usefont`, as the latter eats up spaces. 761
1995-10-16 ltmath.dtx v1.0y
`\@yeqncr`: (DPC) Use `\@testopt` /1911 902
`\sqrt`: (DPC) Make robust /1808 . . 900
1995-10-16 ltspace.dtx v1.2j
`\nolinebreak`: (DPC) Use `\@testopt` /1911 466
`\nopagebreak`: (DPC) Use `\@testopt` /1911 466
1995-10-16 ltthm.dtx v1.0g
General: Revert to previous `\newtheorem` behaviour 1007
1995-10-17 ltclass.dtx v1.0r
`\@providesfile`: Delay definition of `\ProvidesFile` till ltfinal 1110
`\ProcessOptions*`: Reset `\CurrentOption` for graphics/1873 1115
1995-10-17 ltdirchk.dtx v1.0l
General: Modify initex version of `\ProvidesFile` 4
1995-10-17 ltfinal.dtx v1.0v
`\@providesfile`: reset macro 1352
`\reserved@b`: reset here after the `\input` above 1351

1995-10-17 ltplain.dtx v1.1s
\eject: Move \supereject to compat file 28
1995-10-17 lttab.dtx v1.1e
\@cline: (DPC) Use \@multicnt . . 971
\@multispan: (DPC) Macro added. 972
1995-10-19 ltfinal.dtx v1.0w
\@filelist: Move after \reserved@a setting 1352
1995-10-20 ltbibl.dtx v1.1k
\@citex: Removed refundefined flag 1055
\nocite: Removed refundefined flag 1056
1995-10-20 ltclass.dtx v1.0s
\@begindocumenthook: Make setting conditional, for autoload version 1128
1995-10-20 ltfssbas.dtx v3.0i
General: (DPC) Modify autoload code, change \undefined 571
1995-10-20 ltfsstrc.dtx v3.0e
General: (DPC) Modify autoload code 671
1995-10-22 ltfssbas.dtx v3.0j
General: (RmS) New size function macro \genb@sfcnt needs to be disabled at \document. 571
1995-10-22 ltfsstrc.dtx v3.0f
General: Added 'genb' and 'sgenb' size functions to support new DC font naming scheme. 671
1995-10-23 lttab.dtx v1.1f
\@settab: (CAR)Ensure that \@hightab increases by at most one 954
\@startline: (CAR)Ensure that \@nxttabmar is never larger than \@hightab 952
\poptabs: (CAR)Ensure that \@curtab is never larger than \@hightab 955
\tabbing: (CAR)Make \@hightab consistently a local variable 954
1995-10-24 ltfiles.dtx v1.1a
\document: Removed multiplelabels switch 488
Removed refundefined switch . . . 488
1995-10-24 ltfssbas.dtx v3.0k
\@@defaultsubs: macro removed . . 594
\wrong@fontshape: Make this code inline since it happens only here 593
1995-10-24 ltmiscen.dtx v1.1b
\@enddocument@kernel@warnings: Changed logic for producing warning messages and removed switch 867
Use \@refundefined instead of switch 867
1995-10-24 ltxref.dtx v1.1h
\@multiplelabels: Switch for multiplelabels removed 844
\@newl@bel: Switch for multiplelabels replaced by inline code 843
\@refundefined: Switch for refundefined replaced 841
\@setref: Switch for refundefined renamed 842
\if@multiplelabels: Macro removed 844
1995-10-25 ltalloc.dtx v1.1b
General: General doc improvements 421
1995-10-25 ltfloat.dtx v1.1n
\@endfloatbox: (CAR) macro added: to unify code for double and single versions 1035
\end@dblfloat: (CAR) unify code for double and single versions 1034
\end@float: (CAR) unify code for double and single versions 1033
1995-10-25 ltidxglo.dtx v1.1d
General: Doc cleanup 1050
1995-10-25 ltsect.dtx v1.0q
\subparagraphmark: Use \let not \def to save space. 1020
1995-10-27 ltpictur.dtx v1.1f
General: Move initialization to kernel from autoload file 1001
1995-10-31 ltboxes.dtx v1.0x
\@finalstrut: Add \nobreak in horiz mode to allow hyphenation. internal/1931 945
1995-11-01 fontdef.dtx v2.2m
General: add \nfss@catcodes for internal/1932 769
1995-11-01 ltdirchk.dtx v1.0n
General: Initialize \@addtofilelist to \@gobble 4
1995-11-01 ltfinal.dtx v1.0x
General: (DPC) Switch meaning of \@addtofilelist for cfg files . . 1339
1995-11-01 ltfssbas.dtx v3.0m
\DeclareFontShape@: (DPC) Test for \relax not \undefined, internal/1933 572
1995-11-01 ltfssini.dtx v3.0g
General: (DPC) Switch meaning of \@addtofilelist for cfg files . . . 764
1995-11-02 ltfssbas.dtx v3.0n
\wrong@fontshape: (DPC) Remove extra space with \string for latex/1676 593

1995-11-02 ltoutenc.dtx v1.7n
General: Changed internal name \a to \@tabacckludge to protect against redefinition by malicious users. . 522
1995-11-07 ltlists.dtx v1.0k
\@doendpe: Enclosed \setbox0 assignment by a group so that it leaves the contents of box 0 intact. 918
1995-11-07 ltoutenc.dtx v1.7o
General: Added \leavevmode at start of \c, otherwise the output routine might be invoked within the macro. 526
Changed \char32 to \@xxxii (two tokens less). 527
Replaced octal number 27 by decimal number 23 to protect against the quote character being active. 527
Replaced some 0's by \z@ (faster). 527
1995-11-10 ltoutput.dtx v1.1s
\@shipoutsetup: Command removed 1254
\@writesetup: Command removed 1254
In-lined 1254
1995-11-14 ltclass.dtx v1.0t
\@@unprocessedoptions: Allow empty option 1130
\@loadwithoptions: macro added 1117
\LoadClassWithOptions: macro added 1117
\RequirePackageWithOptions: macro added 1118
1995-11-17 ltfssbas.dtx v3.0m
\@wrong@font@char: (DPC) Macro added. latex/1676 594
\define@newfont: Redefine \typeout latex/1676 587
\wrong@fontshape: Support \@wrong@font@char latex/1676 . 593
1995-11-17 ltoutenc.dtx v1.7p
\UseTextSymbol: Support \@wrong@font@char latex/1676 . 519
1995-11-18 ltoutenc.dtx v1.7q
\UseTextSymbol: Modify message slightly 519
1995-11-21 fontdef.dtx v2.2n
General: Incorporate changed figures, as in plain.tex 783
1995-11-27 ltfssbas.dtx v3.0n
\nfss@catcodes: Reset hash, for definitions in fd files 589
1995-11-28 ltfloat.dtx v1.1n
General: documentation fixes 1026
1995-11-28 ltfsstrc.dtx v3.0g
General: documentation fixes 671
1995-11-28 ltoutenc.dtx v1.7r
General: Added math mode checks to text commands. 513
doc fixes 509
Renamed \@changed@x@err to \TextSymbolUnavailable. 513
1995-11-29 ltoutenc.dtx v1.7t
General: Added \textasciicircum, \textasciitilde, \textbackslash, \textbar, \textgreater and \textless. . . 529
Added \textasciicircum, \textasciitilde, \textregistered and \texttrademark. 523
Added \textbackslash and \textbar. 523, 534
Added \textless and \textgreater. 523, 534
1995-12-01 ltoutenc.dtx v1.7u
General: Made \SS a Default, rather than having the default point to the OT1 definition. 523
1995-12-04 ltspace.dtx v1.2k
\nobreakspace: Macro added 480
1995-12-04 ltspace.dtx v1.2l
\@xobeysp: braces added to definition of tilde 480
1995-12-04 preload.dtx v2.4e
General: Ulrik Vieth. added 12pt OMS and OML preloads /1989 . 789
1995-12-05 ltdefns.dtx 1.2w
\@unexpandable@protect: Removed \unexpandable@noexpand as never used. internal/1733 92
1995-12-05 ltfiles.dtx v1.1c
\document: \ignorespaces added for latex/1933 489
1995-12-05 ltfloat.dtx v1.1n
\@textsuperscript: Use \ensuremath for latex/1984. 1044
1995-12-05 ltoutenc.dtx v1.7v
\@inmathwarn: Changed \TextSymbolUnavailable text . . 515
1995-12-06 ltfssbas.dtx v3.00
\nfss@catcodes: Reset hat, for typeouts etc in fd files 589
1995-12-07 ltbibl.dtx v1.1l
\@citex: Restored name of \G@refundefinedtrue 1055

1995-12-07 ltfloat.dtx v1.1m
\@textsuperscript: Move \m@th out of the \ensuremath for latex/1984. 1044
1995-12-07 ltxref.dtx v1.1i
\@setref: Switch for refundefined restored 842
\G@refundefinedtrue: Renamed (back) from \G@refundefined . . 841
1995-12-11 ltoutenc.dtx v1.7w
General: Modified \copyright 523
1995-12-13 ltdefns.dtx 1.2x
\-: Documentation changed. 112
1996-01-10 ltfiles.dtx v1.1d
\@iffileonpath: Change argument handling to not require doubled hash. latex/2024 502
1996-01-20 ltidxglo.dtx v1.1e
\makeglossary: Make no-op after use pr/2048 1051
\makeindex: Make no-op after use pr/2048 1051
1996-01-20 ltspace.dtx v1.2m
\vspace: Made robust 478
1996-03-25 ltmath.dtx v1.1a
\@ensuredmath: Macro added for amslatex/2104 903
\ensuremath: Reimplement for amslatex/2104 903
1996-04-18 ltpage.dtx v1.0i
General: Improve documentation . 1089
1996-04-22 ltmiscen.dtx v1.1c
General: Improve Documentation . . 864
1996-04-22 ltspace.dtx v1.2n
General: Documentation Improvements 463
1996-04-22 lttab.dtx v1.1g
\@tabclassz: (DPC) Extra \hskip keeps tabcolsep in empty columns internal/2122 968
1996-04-23 ltcounts.dtx v1.1d
General: Documentation improvements 556
1996-04-24 ltfiles.dtx v1.1e
\document: (DPC) Reset \AtBeginDocument eg for latex/1297 488
1996-05-08 ltfsstrc.dtx v3.0h
\math@egroup: Use \bgroup instead of \begingroup to match a kernel change made in 1994!! 683
1996-05-09 ltfntcmd.dtx v3.3t
\check@icr: Default definitions added 794
1996-05-17 fontdef.dtx v2.2o
General: \@@sqrt removed, at last 766, 782
1996-05-17 ltfiles.dtx v1.1f
\nofiles: added \write to \protected@write for latex/2146 492
1996-05-18 ltoutenc.dtx v1.7x
General: Produce error if encoding not found. pr/2054 553
1996-05-21 ltoutenc.dtx v1.7y
General: Corrected error message (CAR) 553
1996-05-21 ltsect.dtx v1.0s
\@sect: (DPC) Added extra braces for internal/2148 1016
(DPC) Moved brace to allow commands like \MakeUppercase in 6th argument. Changed \par to \endgraf to allow non-long commands. internal/2148 1016
\@ssect: (DPC) Added extra braces for internal/2148 1019
(DPC) Moved brace to allow commands like \MakeUppercase in 4th argument. Changed \par to \endgraf to allow non-long commands. internal/2148 1019
1996-05-23 ltoutenc.dtx v1.7z
\@strip@args: \expandafter added to match other changes for latex/2133 518
\add@accent: macro added. latex/2133 516
\DeclareTextAccent: Reimplemented using \add@accent to save space latex/2133 516
\DeclareTextCompositeCommand: Modified to cope with new \add@accent command: required removal of check for one argument-command 517
1996-05-24 ltoutput.dtx v1.1t
\@specialoutput: Check that \@colroom is less than \vsize, indicating that a float has been added 1235
Cut-off point changed to 1.5\baselineskip 1235
\@topnewpage: Cut-off point changed to 2.5\baselineskip 1231
1996-05-25 ltoutput.dtx v1.1u
\@specialoutput: Correct the above check 1235

1996-06-03 ltmiscen.dtx v1.1d
\@verbatim: Exchanged the following two code lines so that \dospecials cannot reset the category code of characters handled by \@noligs. 882
General: Move setting of verbatim font and \@noligs. 864
\verb: Put setting of verbatim font after \dospecials so that \dospecials cannot reset the category code of characters handled by \@noligs. 887
1996-06-10 ltboxes.dtx v1.0y
\@parboxto: (DPC) Changed \endgraf to \@@par 936
1996-06-10 ltsect.dtx v1.0t
\@sect: (DPC) Changed \endgraf to \@@par 1016
\@ssect: (DPC) Changed \endgraf to \@@par 1019
1996-06-13 ltdirchk.dtx v1.0r
General: documentation improvements mainly from internal/2174 1
1996-06-14 lttab.dtx v1.1h
\@tabclassz: (DPC) Change both\z@skip to 1sp for latex/2160 968
1996-06-22 ltspace.dtx v1.2o
General: Documentation of problems added 463
1996-07-10 ltfinal.dtx v1.0y
\toks: Free up memory from scratch registers /2213 1351
1996-07-19 ltoutenc.dtx v1.8a
\@strip@args: Use char 0 not @ as carrier for \lowercase /2197 . . . 519
1996-07-26 ltboxes.dtx v1.0z
\if@minipage: put \global into definition 939
1996-07-26 ltclass.dtx v1.0u
\@classoptionslist: made only preamble 1099
\@unusedoptionlist: made only preamble 1099
1996-07-26 ltdefns.dtx v1.2y
\@reargdef: third arg picked up by \@yargdef 87
\renew@command: use \noexpand instead of \string 87
use \relax in place of empty arg . 87
\renew@environment: use \relax in place of empty arg 88
1996-07-26 ltfloat.dtx v1.1n
\@endfloatbox: remove unnecessary \global before \@minipage... 1035
\@savemarbox: remove unnecessary \global before \@minipage... 1039
\@setminipage: remove unnecessary \global before \@minipage... 1033
\@setnobreak: remove unnecessary \global before \@nobreak... . 1033
1996-07-26 ltfssbas.dtx v3.0p
\@DeclareMathSizes: use faster \if test 580
\nfss@catcodes: omit \relax as not needed 589
1996-07-26 ltfssdcl.dtx v3.0e
\init@restore@version: Removed \ifrestore@version switch and replaced by \init@restore@version 706
1996-07-26 ltfsstrc.dtx v3.0i
\init@restore@glb@settings: macro added replacing \if@inmath switch 682
1996-07-26 ltlists.dtx v1.0l
\@item: Remove unnecessary \global before \@minipage... 920
Remove unnecessary \global before \@nobreak... 921
1996-07-26 ltmath.dtx v1.1b
General: Removed \global before \@ignoretrue in various places. . 889
1996-07-26 ltmiscen.dtx v1.1e
\@ignorefalse: put \global into definition 865
\begin: remove \global before \@ignore... 873
\end: remove \global before \@ignore... 875
\ignorespacesafterend: user level macro added 865
1996-07-26 ltoutput.dtx v1.1v
\@testfp: remove \global before \@test... 1296
\@xtryfc: remove \global before \@test... 1264
\@ztryfc: remove \global before \@test... 1265
General: put \global into definition 1224
remove \global before \@test... 1224
\clearpage: add number of missing percents 1227
1996-07-26 ltplain.dtx v1.1t
\sh@ft: replace \dimen\z@ by \dimen@ 30

1996-07-26 ltsect.dtx v1.0u
\@starttoc: removed \global before \@nobreak... 1021
\@xsect: Removed \global before \@nobreak... 1018
1996-07-26 ltspace.dtx v1.2p
\if@nobreak: put \global inside definition 469
1996-07-27 ltfssbas.dtx v3.0q
General: \if@inmath switch removed 586
1996-07-27 ltspace.dtx v1.2q
General: Further documentation of problems 463
1996-07-27 ltspace.dtx v1.2r
General: Correct documentation of problems 463
1996-08-02 ltfloat.dtx v1.1o
\@xympar: Remove \global before \@ignore... 1040
1996-08-02 ltsect.dtx v1.0v
\@afterheading: Removed \global before \@nobreak... 1019
1996-08-02 ltspace.dtx v1.2s
\@Esphack: Remove \global before \@ignore... 473
1996-08-25 ltfssbas.dtx v3.0r
\nfss@catcodes: Reset the acute, grave and double quote chars as well 589
1996-09-21 ltoutput.dtx v1.1w
\@writesetup: Added \@parboxrestore and made consequent deletions: wait for the howls of protest 1254
1996-09-25 ltdirchk.dtx v1.0t
General: Move ltxcheck to separate file 13
1996-09-28 ltmiscen.dtx v1.1f
\@xobeysp: Moved to ltspace.dtx 881
1996-09-28 ltspace.dtx v1.2t
\@xobeysp: Moved from ltmiscen.dtx and redefined to use \nobreakspace 480
1996-09-29 ltfiles.dtx v1.1g
\document: Added disabling of \@nodocument 489
1996-09-29 ltoutput.dtx v1.1x
\newpage: Checks for noskipsec and inlabel added 1228
1996-09-29 ltsect.dtx 1.0w
\@noskipsectrue: Added documentation 1013
1996-09-30 ltoutput.dtx v1.1y
\newpage: Checks for noskipsec and inlabel removed pending further tests 1228
1996-10-04 ltclass.dtx v1.0v
\RequirePackageWithOptions: Reset \@unprocessedoptions for /2269 1118
1996-10-05 ltfiles.dtx v1.1h
\@clubpenalty: Added setting its value 487
1996-10-08 ltfntcmd.dtx v3.3u
\DeclareTextFontCommand: Removed \check@icr when in vmode since it causes various errors (see pr/2157) 792
1996-10-21 lttab.dtx v1.1i
\@array: Use \set@typeset@protect 961
General: Moved the code associated with \@mkpream into the group provided by the box, for robustness (latex/2183) 960
\multicolumn: Make \multicolumn long (latex/2180) 963
\tabbing: Moved the \indent so that the \everypar can remove it when necessary; this is needed because the code for items in lists has changed (see pr/22111) 954
1996-10-23 ltlists.dtx v1.0m
\@item: \@nobreak... moved into the \everypar and not executed unconditionally, see above 921
\kern... changed to \setbox... 920
Added setting of \clubpenalty and set \@nobreakfalse only when necessary 921
1996-10-23 ltsect.dtx v1.0x
\@xsect: Replaced \hskip... with \setbox... as used in \@afterheading 1018
1996-10-24 ltboxes.dtx v1.1a
\@arrayparboxrestore: Added local settings of flags: dangerous! 938
\@iiiminipage: Use it or lose it (@setminpage): Frank will want to lose it 940
1996-10-24 ltfloat.dtx v1.1p
\@floatboxreset: Added local settings of flags: dangerous! 1033
\@marginparreset: Added local settings of flags: dangerous! 1039
\@xfloat: Added \@nodocument to trap floats in the preamble 1030
1996-10-24 ltoutput.dtx v1.1z
\@addtocurcol: Added \nobreak, etc as appropriate 1270, 1276

\@specialoutput: Added \nobreak as appropriate 1237
\@topnewpage: Added \@nodocument to trap \twocolumn in the preamble 1230
\newpage: Better checks for noskipsec and inlabel added, plus nobreak 1228
1996-10-25 ltlists.dtx v1.0n
\endtrivlist: Change \indent to \leavevmode 916
Reset flags explicitly 916
1996-10-25 ltoutput.dtx v1.2a
\newpage: Reset all flags explicitly 1228
1996-10-26 ltlists.dtx v1.0o
\endtrivlist: Correct typo 916
1996-10-27 ltoutenc.dtx v1.8c
\@strip@args: Removed macro . . . 517
General: Added \r A 527
Added \textasteriskcentered . . 523, 534
Corrected syntax descriptions . . . 510
Removed \aa and \AA . 522, 527, 529
1996-10-28 ltplain.dtx v1.1u
General: (CAR) More doc changes . . 14
\dotfill: Removed math mode 30
1996-10-29 ltplain.dtx v1.1v
\dotfill: Got arithmetic correct (CAR) 30
1996-10-29 ltspace.dtx v1.2u
\@gnewline: Added macro 469
\@no@lnbk: Macro replaces \@lnbk and \@nolnbk 467
\\: Corrected and rationalized code 467
\nolinebreak: Reimplemented both using \@no@lnbk 466
1996-10-31 ltfinal.dtx v1.0z
General: Added extra \lcode, hoping it does no harm in T1 (pr/1969) 1338, 1347
1996-10-31 ltlists.dtx v1.0p
\@trivlist: Added check for missing item in outer list 915
1996-10-31 ltsect.dtx v1.0y
General: Corrected and tidied documentation; removed long lines 1012
1996-11-03 ltplain.dtx v1.1w
\dotfill: Saved tokens by using \hb@xt@ 30
1996-11-04 lterror.dtx v1.2m
\@nodocument: Always define \@nodocument in kernel, so that it can be cleared by \document. . . . 434
1996-11-04 ltlists.dtx v1.0q
\@trivlist: Moved check for missing item: only checked when not inlabel flag is false 915
1996-11-05 ltfiles.dtx v1.1i
\nofiles: Standard \if@nobreak test added 492
1996-11-09 ltmath.dtx v1.1c
\@ensuredmath: Made long, as it was before. /2104 903
1996-11-18 ltfssbas.dtx v3.0s
\define@newfont: (DPC) lowercase fd file names. internal/1044 588
1996-11-18 ltoutenc.dtx v1.8d
General: (DPC) lowercase external file names. internal/1044 553
1996-11-20 fontdef.dtx v2.2p
General: lowercase fd and enc.def file names /1044 766
1996-11-20 ltvers.dtx v1.0f
General: Check for old format modified /2319 35
1996-11-23 ltoutenc.dtx v1.8e
General: Corrected description 510
Extended description 511
1996-11-28 ltvers.dtx v1.0g
General: Check for old format modified /2319 35
1996-12-06 ltdirchk.dtx v1.0u
\IfFileExists: *** removed from various messages for GNU Make. internal/2338 10
1996-12-06 ltfloat.dtx v1.1r
\@caption: Call \@setminpage if needed. latex/2318 1029
1996-12-06 ltfssini.dtx v3.0h
General: (DPC) Remove *** from messages internal/2338 764
1996-12-17 ltdefns.dtx v1.0w
\g@addto@macro: Use \begingroup to save making a mathord 114
1996-12-20 ltsect.dtx v1.0z
\@dottedtocline: Added \nobreak for latex/2343 1024
1997-01-08 fontdef.dtx v2.2q
General: Use \DeclareMathDelimiter to set delimiter codes 776
\mathparagraph: Define using \DeclareMathSymbol 784
1997-01-08 ltfiles.dtx v1.1j
\@include: reset \deadcycles latex/2365 496

1997-01-08 ltmath.dtx v1.1d
\root: (DPC) Remove spurious space tokens from plain TEX definition /2359 891
1997-02-05 ltdefns.dtx v1.0x
\g@addto@macro: missing percent /2402 114
1997-02-21 ltlists.dtx v1.0r
\@item: \ifvoid check added for \noindent. latex/2414 920
1997-03-21 ltcounts.dtx v1.1e
\fnsymbol: Use \mathsection and \mathparagraph. latex/2445 . . . 566
1997-04-14 ltfiles.dtx v1.1k
\document: Set the document space factor defaults. latex/2404 488
\normalsfcodes: Macro added (from patch file) latex/2404 492
1997-04-14 ltoutput.dtx v1.2b
\@writesetup: Call \normalsfcodes (from patch file) latex/2404 . . . 1256
Move \label and \index (from patch file) 1257
1997-04-24 ltbibl.dtx v1.1m
\@citex: \@empty to avoid primitive error on empty cite keys. latex/2432 1055
1997-04-30 ltoutenc.dtx v1.9a
General: Changed \textsc to \scshape 524
Introduced \textcopyright and modified \copyright 523
Introduced \textcopyright and modify \copyright 524
Modified \textunderscore, removing \mathunderscore 523
Modified \underscore, removing \mathunderscore 524
1997-04-30 ltoutenc.dtx v1.9b
General: Added \leavevmode to \textunderscore 523
1997-05-04 ltoutenc.dtx v1.9c
General: Added 'hex index tabs' . . . 531
Added TS1 encoding v2.2.beta . . 537
1997-05-07 ltoutenc.dtx v1.9d
General: Added \leavevmode to \textcompwordmark 523
1997-05-07 ltspace.dtx v1.2v
\newline: Made completely robust. 468
1997-05-29 ltfsstrc.dtx v3.0j
General: Replaced \\ by \MessageBreak, as suggested by Donald Arseneau. 673
1997-05-29 ltlogos.dtx v1.1f
\LaTeXe: Added \m@th so that the LATEX 2ε logo works with non-zero values of \mathsurround. 484
1997-06-16 ltdirchk.dtx v1.0v
General: documentation improvements mainly from internal/2520 1
1997-06-16 ltfloat.dtx v1.1s
General: documentation fixes 1026
1997-06-16 ltfntcmd.dtx v3.3v
General: Fix typo in documentation. 790
1997-08-05 ltoutenc.dtx v1.9e
General: Corrected order of arguments in \UseTextSymbol example. . . . 510
1997-08-29 ltoutenc.dtx v1.9f
General: Added OT4 encoding, provided by Marcin Woliński. . . 509
1997-09-09 ltdefns.dtx v1.2z
\provide@command: Use \begingroup to avoid generating math ords if used in math mode. pr/2573 89
1997-09-15 ltpictur.dtx v1.1g
\@getcirc: Warn if lines become invisible pr/2524 995
\@picture@warn: Macro added pr/2524 996
\@sline: Warn if lines become invisible pr/2524 985
1997-10-06 ltcounts.dtx v1.1f
\@Roman: Change \@Roman to be fully expandable, so that the result is written properly to files. 566
\@slowromancap: Macro added. . . . 566
1997-10-08 ltlogos.dtx v1.1h
\LaTeX: Simplify macro (force loading of suitable math fonts once). . . . 484
1997-10-10 ltclass.dtx v1.0y
\endfilecontents: \@currenvir in banner 1133
\reserved@c not \verbatim@out to save a csname 1132
Check for text before or after \end environment. latex/2636 1134
Use \@gobbletwo 1133
1997-10-17 ltfntcmd.dtx v3.3w
\check@nocorr@: Check for vertical mode moved here, from \DeclareTextFontCommand (see PR/2646). 794
\DeclareTextFontCommand: Reinstalled \check@icr as check is now done in \check@nocorr@ (see PR/2646). 792

1997-10-20 ltfinal.dtx v1.1a
\@uclclist: Removed \aa and \AA from \@uclclist as these are macros. 1347
1997-10-21 ltdefns.dtx v1.2z1
\renew@command: Use \begingroup/\endgroup rather than braces for grouping, to avoid generating empty math atom. . . . 87
1997-10-21 ltfssbas.dtx v3.0t
\define@newfont: Move \makeatletter to \nfss@catcodes. 588
\nfss@catcodes: Moved \makeatletter from \try@load@font@shape. 589
1997-11-09 ltoutput.dtx v1.2c
\@specialoutput: Remove incorrect code: only one \@emptycol is needed here 1235
\@topnewpage: Documentation of vsize check enhanced 1230
1997-11-13 ltfssdcl.dtx v3.0f
\DeclareSymbolFont: (DPC) Really update \group@list don't leave new version in \toks@. latex/2661 717
\stepcounter: (DPC) Remove as never used. (Re)defined in ltcounts 704
1997-11-19 ltfloat.dtx v1.1t
\@footnotetext: Missing percent, again 1047
1997-11-19 ltoutput.dtx v1.2d
\@vtryfc: Reindent code, to be understandable(DPC). 1264
1997-11-20 ltfssdcl.dtx v3.0g
\document@select@group: (DPC) inline use of \stepcounter (faster, and saves a csname per math version as no reset list) 707
\select@group: (DPC) inline use of \stepcounter (faster, and saves a csname per math version as no reset list) 705
1997-11-23 ltoutenc.dtx v1.9g
General: Use \textperthousand, \textpertenthousand and \textfractionsolidus not \textpermill, \textpertenmill and \textfraction. /2673 537
1997-12-17 ltoutenc.dtx v1.9h
General: Added \textperthousand and \textpertenthousand . 528, 529
Added code for textcomp.sty. . . . 553
Added section. 553
Added textcomp.sty. 509
As in OT1, Added \leavevmode at start of \c, otherwise the output routine might be invoked within the macro. 528
Changed to decimal codes in \ooalign. 539
Changed to decimal codes. 534
Documentation changes and additions. 509
Example corrected, braces removed. 509
Removed default settings, see next section. 537
1997-12-19 ltoutenc.dtx v1.9i
General: Documentation corrections. 509
1997-12-20 fontdef.dtx v2.2s
General: Added documentation . . . 768
1997-12-31 ltoutenc.dtx v1.9k
General: Further correction 510
1998-01-12 ltoutenc.dtx v1.9k
General: Added \ProvidesPackage for textcomp.sty 509
Adding missing braces and \ushape. 539
1998-01-16 ltoutenc.dtx v1.9m
General: fixed decimal codes. latex/2734 534
1998-03-04 ltdefns.dtx v1.2z2
\@xargdef: Unnecessary \expandafter removed: pr/2758 . 85
1998-03-05 ltoutenc.dtx v1.9n
General: Added masc/fem ords as in pr/2579 524
1998-03-20 ltdefns.dtx v1.2z3
\@thirdofthree: Macro added 90
1998-03-20 ltoutenc.dtx v1.9o
General: Documentation added about order of decls 512
Documentation added for pr/2783 511
\UndeclareTextCommand: Macro added for pr/2783 520
1998-03-20 lttextcomp.dtx v1.9o
General: Added various \UndeclareTextCommand declarations for pr/2783 827
Load decls after defaults for speed. 827
1998-03-21 ltclass.dtx v1.0z
General: Added to documentation of filecontents 1094
1998-03-21 ltclass.dtx v1.1a
\@providesfile: Allow &. Internal/2702 1110

General: Correct to new onlypreamble command list 1147
1998-03-25 ltfssbas.dtx v3.0u
`\showhyphens`: Suppress unnecessary error when used in preamble . . . 598
1998-04-11 fontdef.dtx v2.2t
General: Added `\mathring` accent (pr2785) 782
1998-04-15 fontdef.dtx v2.2u
General: Use new syntax for `\DeclareMathDelimiter` 776
1998-04-15 ltfssdcl.dtx v3.0h
`\@xxDeclareMathDelimiter`: Macro added (pr/2662) 727
1998-04-17 fontdef.dtx v2.2v
General: Reinsert symbol defs for < and > chars. 776
1998-04-18 fontdef.dtx v2.2w
General: Reinsert symbol def for / char. 776
1998-05-07 ltclass.dtx v1.1b
`\@onefilewithoptions@clashchk`: Modify help message for latex/2805 1125
1998-05-18 lttab.dtx v1.1j
`\@endpbox`: Use `\setlength` to set `\hsize`, so that the changes in the calc package apply here. 972
`\tabular*`: Use `\setlength`, so that calc extensions apply. 960
1998-05-20 ltfinal.dtx v1.1b
General: Set up lccodes before loading hyphenation files: pr/2639 1337
Set up uc/lccodes after loading hyphenation files: pr/2639 1346
1998-05-28 lterror.dtx v1.2n
`\@notdefinable`: Added message re 'end...' pr/1555 434
1998-06-04 ltboxes.dtx v1.1c
`\@rule`: Support calc-expressions . . 942
1998-06-12 ltoutenc.dtx v1.9p
General: Corrected 130 and 131, see pr/2834 539
Renamed `\textmacron` pr/2840 . 540
1998-06-12 ltoutenc.dtx v1.9q
`\add@accent`: Explicitly set `\spacefactor` after `\accent` (pr/2877) 516
1998-06-12 lttextcomp.dtx v1.9p
General: Renamed `\textmacron` pr/2840 823
1998-06-18 lttab.dtx v1.1k
General: Small addition to documentation 947
1998-07-06 lttab.dtx v1.1l
General: Small correction to documentation 947
1998-08-17 ltboxes.dtx v1.1e
General: (RmS) Minor Documentation fixes. 925
1998-08-17 ltclass.dtx v1.1c
General: (RmS) Minor documentation fixes. 1094
1998-08-17 ltdirchk.dtx v1.0w
General: (RmS) Documentation improvements. 1
1998-08-17 ltfntcmd.dtx v3.3x
General: (RmS) Minor documentation fixes. 790
1998-08-17 ltfssbas.dtx v3.0v
General: (RmS) Documentation fixes. 571
1998-08-17 ltfssdcl.dtx v3.0i
General: (RmS) Corrected minor glitches in changes entries. 702
1998-08-17 ltfssini.dtx v3.0i
General: (RmS) Minor documentation fixes. 735
1998-08-17 ltlogos.dtx v1.1i
General: (RmS) Minor documentation fixes. 484
1998-08-17 ltmath.dtx v1.1c
General: (RmS) Minor documentation fixes. 889
1998-08-17 ltmiscen.dtx v1.1g
General: (RmS) Minor documentation fixes. 864
1998-08-17 ltspace.dtx v1.2w
General: Documentation fixes. 463
1998-08-17 preload.dtx v2.1g
General: (RmS) Minor documentation fixes. 787
1998-09-19 ltoutenc.dtx v1.9r
`\a`: Added `\string` (pr/2878) 522
1998-11-13 lttab.dtx v1.1m
`\@array`: Check for hmode to see if something went wrong during parsing (pr/2884) 961
1999-01-05 fontdef.dtx v2.2x
General: Need special protection for character > in `\changes` entry. . . 766
1999-01-06 ltfssbas.dtx v3.0w
`\DeclareFontEncoding`: Added `\LastDeclaredEncoding` to support cyrillic integration (pr/2988) 575
`\LastDeclaredEncoding`: Added `\LastDeclaredEncoding` to

support cyrillic integration (pr/2988) 575
1999-01-06 ltoutenc.dtx v1.9r
\@strip@args: New impl for latex/2930 518
General: Minor documentation fix. . 539
1999-01-07 ltdefns.dtx v1.3a
\@ifnextchar: made long 110
\@newenvb: made long and brace optional arg. latex/2896 88
\@testopt: made long and brace optional arg. latex/2896 85
1999-01-07 ltdefns.dtx v1.3b
\@ifnextchar: extra \long. latex/2902 110
1999-01-07 ltoutenc.dtx v1.9r
General: Hackery to allow using fontenc several times 555
Hackery to temp support cyrillic uc/lc 553
1999-01-13 ltoutenc.dtx v1.9s
\@strip@args: Simplified solution for latex/2930 518
1999-01-18 ltdefns.dtx v1.3c
\@yargd@f: New implementation DPC /2942 85
1999-02-09 ltdefns.dtx v1.3d
\@yargd@f: catch bad argument forms by re-inserting #3 85
1999-02-12 lttextcomp.dtx v3.0j
\legacyoldstylenums: Use \rmdefault instead of cmm (pr/2954) 799
1999-02-24 ltoutenc.dtx v1.9t
General: Corrected hackery cyrillic uc/lc list 553
1999-03-01 ltdefns.dtx v1.3e
\@ifnextchar: remove extra \long. internal/2967 110
1999-04-15 ltpictur.dtx v1.1h
\@getlarrow: Replaced octal number, CAR 986
\@upvector: Replaced octal number, CAR 987
General: Replaced octal number, CAR 986, 987
Replaced octal numbers, CAR . . 974
1999-04-19 ltfloat.dtx v1.1u
\caption: Made caption an error outside a float: latex/2815 1029
1999-04-27 ltboxes.dtx v1.1f
\@parboxto: (CAR) Changed \@empty to \relax as flag for natural width: pr/2975 936
1999-04-29 ltdefns.dtx v1.3f
\@yargd@f: Full expansion and conversion needed for digit in new version, see pr/3013 85
New macro added 85
1999-06-10 ltoutenc.dtx v1.9u
General: Ensure that we also forget old options (pr/2888) 555
1999-06-12 ltoutenc.dtx v1.9v
General: Extend \@uclclist only once 554
1999-10-09 ltmath.dtx v1.1e
\active@math@prime: Macro added, see PR 3104. 896
\prime@s: Introduce \active@math@prime. 896
1999-10-09 ltoutput.dtx 1.2f
\@activechar@info: Reset definition of active prime character (used in math mode) 1253
1999-10-28 ltoutenc.dtx v1.9w
\add@accent: Give \accent@spacefactor a default definition (pr/3084) 516
1999-12-08 ltoutenc.dtx v1.9x
General: Changed \CYRRHOOK and \cyrrhook to\CYRRHK and \cyrrhk as name changed in the cyrillic bundle for naming consistency with other “hook” glyphs. 553
2000-01-07 ltmiscen.dtx v1.1h
\@verbatim: Disable hyphenation even if the font allows it. 882
2000-01-15 ltpictur.dtx v1.1i
\@upvector: Removed space at end-of-line, CAR 987
2000-01-30 ltfntcmd.dtx v3.3y
\DeclareTextFontCommand: Use \hmode@bgroup now (pr/3160) . . 792
2000-01-30 ltoutenc.dtx v1.9y
General: Use \hmode@bgroup where applicable (pr/3160) . . . 526–528, 534–537, 539
\add@accent: Use \hmode@bgroup where applicable (pr/3160) 516
\hmode@bgroup: Macro added 516
2000-01-30 ltoutenc.dtx v1.9z
\@use@text@encoding: Macro reimplemented (pr/3160) 519
\add@accent: Macro reimplemented (pr/3160) 516
\hmode@start@before@group: Macro added (pr/3160) 520

2000-05-19 ltmiscen.dtx v1.1i
\enddocument: Reset \AtEndDocument for latex/3060 865
2000-05-26 ltpage.dtx v1.0j
\@markright: Reimplementation to fix expansion error (pr/3203). 1092
\leftmark: Use \@empty instead of brace group (pr/3203). 1092
\markright: Reimplementation to fix expansion error (pr/3203). 1090
\rightmark: Use \@empty instead of brace group (pr/3203). 1092
2000-06-02 ltpage.dtx v1.0k
\@markright: Small adjustment to give slightly less expansion, CAR 1092
\markright: Small adjustment to give slightly less expansion, CAR . . 1090
Tidied 1.0j reimplementation, CAR 1090
2000-07-11 ltmiscen.dtx v1.1j
\@enddocument@kernel@warnings: Fix typo in warning 867
2000-07-12 ltoutput.dtx 1.2g
General: Ensure that rule is in \normalcolor 1303
2000-07-19 ltoutput.dtx v1.2h
\@writesetup: Reset and restore \@if@newlist for internal/3231 1255
2000-08-23 ltfinal.dtx v1.1c
General: Fix typo in warning 1339
2000-08-30 ltoutenc.dtx v1.91
\@use@text@encoding: Rearranged but no change to final code, CAR (pr/3160) 519
\add@accent: Rearranged but no change to final code, CAR (pr/3160) 516
2000-09-01 ltfinal.dtx v1.1d
\errhelp: Set error help empty at very end (pr/449 done correctly). . . 1351
2000-09-24 ltfloat.dtx v1.2b
\end@dblfloat: FMi: use output routine to defer float 1034
2000-09-24 ltoutput.dtx v1.2b
\@doclearpage: FMi: ensure \doclearpage is called again until all floats are output. 1239
2000-09-24 ltoutput.dtx v1.2n
\@addtocurcol: FMi: test for wide float was in wrong place 1269
2001-01-07 ltoutput.dtx v1.2j
\@writesetup: And do it in the right macro (pr/3286) 1255
2001-02-16 ltxref.dtx v1.1k
\@newl@bel: Added an extra grouplevel (PR3250), jlb 843
2001-05-25 ltclass.dtx v1.1d
\@providesfile: Explicitly set catcode of \endlinechar to 10 (pr/3334) 1110
2001-05-25 ltdirchk.dtx v1.0x
General: Explicitly set catcode of \endlinechar to 10 (pr/3334) . . . 4
2001-05-28 ltoutenc.dtx v1.93
General: Added composites for compatibility with T1, pr/3295 . 527
Changed the effect of \.\i, pr/3295 531
2001-06-02 fontdef.dtx v2.2y
General: Provide default cfg files (pr/3264) 785
2001-06-04 fontdef.dtx v2.2z
General: Guard against math active equal and pipe sign in \models (pr/3333) 781
Guard against math active equal sign in \Relbar (pr/3333) 781
2001-06-04 ltclass.dtx v1.1e
\@providesfile: But only if it is a char (pr/3334) 1110
2001-06-04 ltdirchk.dtx v1.0y
General: But only if it is a char (pr/3334) 4
2001-06-04 ltpictur.dtx v1.1j
\@sline: Don't warn for exactly zero pr/3318 985
2001-06-04 ltvers.dtx v1.0i
General: Check for old format disabled 35
2001-06-05 ltoutenc.dtx v1.94
General: Text composite Commands need kludges for ',' – see tlb1903.lvt 527
2001-08-26 ltclass.dtx v1.1f
\@providesfile: Readded setting of space char (pr/3353) 1110
2002-02-24 ltplain.dtx v1.1x
\loggingall: Macro added 31
\loggingoutput: Macro added 31
\showoutput: Use newly added \loggingoutput 31
\tracingall: Use newly added \loggingoutput 31
2002-06-16 ltoutenc.dtx v1.95
General: Added \textbardbl (pr/3400) 534
Added default for \textbardbl (pr/3400) 523

2002-06-17 ltoutenc.dtx v1.95
General: Corrected \c for T1 (pr/3442) 528
Definition of \textexclamdown changed (pr/3368) 526
Definition of \textquestiondown changed (pr/3368) 526
2002-06-18 ltoutenc.dtx v1.95
General: Changed def for \textregistered to avoid small caps (pr/3420) 524
2002-10-01 ltfloat.dtx v1.1v
\thempfootnote: Use braces around \itshape to keep font change local (pr/3460). 1043
2002-10-02 ltfssbas.dtx v3.0x
\DeclareFontSubstitution: Adding \LastDeclaredEncoding introduced a bug as on some occasions that macro name was stored in the internal lists instead of the actual encoding. (pr/3459) 575
2002-10-28 ltlists.dtx v1.0s
\endtrivlist: Check for math mode (pr/3437) 916
2002-10-28 ltoutenc.dtx v1.96
General: coding change, to follow bug fix by DEK in plain.tex (pr/3469) 527, 536
2002-12-13 ltbibl.dtx v1.1n
\@citex: Added \leavevmode in case citation is at start of paragraph (pr/3486) 1055
2003-01-01 ltfntcmd.dtx v3.3z
General: Code checked and documentation extended by Chris 792
2003-05-18 ltbibl.dtx v1.1o
\nocite: Check if we are after \document 1056
2003-08-27 ltpictur.dtx v1.1k
\@bezier: added missing displacement pr/3566 1004
\@sline: check for \@linechar being empty pr/3570 984
2003-10-13 ltfinal.dtx v1.1e
General: Added extra \lccode for \- and \textcompwordmark 1338
2003-12-16 ltoutput.dtx v1.2k
\@makecol: Ensure that \@elt has a defined state (pr/3586) 1241
2003-12-30 ltpictur.dtx v1.1j
\@getcirc: issue warning if circle size can't be met pr/3473 995
2004-01-03 ltoutenc.dtx v1.99b
General: Added \textogonekcentered (pr/3532) 528
Added composites for \k (pr/3532) 533
Use \ooalign for \k (pr/3532) . . 528
2004-01-04 ltbibl.dtx v1.1p
\nocite: Changed error message . 1056
2004-01-04 ltoutenc.dtx v1.99c
General: More adjustments for ogonek (pr/3532) 528
2004-01-23 ltdefns.dtx v1.1g
\@newenva: Use kernel version of \@ifnextchar (pr/3501) 88
\@testopt: Use kernel version of \@ifnextchar (pr/3501) 85
\@xargdef: Use kernel version of \@ifnextchar (pr/3501) 84
\@xdblarg: Use kernel version of \@ifnextchar (pr/3501) 111
2004-01-23 ltdefns.dtx v1.3g
\kernel@ifnextchar: Added macro (pr/3501) 110
2004-01-28 ltclass.dtx v1.1g
\@providesfile: Use kernel version of \@ifnextchar (pr/3501) 1111
2004-01-28 ltvers.dtx v1.0k
General: Check for old format made 5 years (pr/3601) 35
2004-02-02 fontdef.dtx v2.3
General: Many things from here on made robust 780
2004-02-02 ltoutenc.dtx v1.99
General: Added \textbigcircle . . 534
2004-02-04 fontdef.dtx v2.3a
General: Added bigtriangle synonyms for stmaryrd 778
2004-02-04 ltspace.dtx v1.3
\nobreakdashes: Macro added 479
2004-02-06 ltoutenc.dtx v1.99d
\@inmathwarn: New command added to fix severe bug: pr/3563 514
2004-02-07 ltoutput.dtx v1.2l
\@doclearpage: Empty kludgeins box if necessary, pr/3528 1239
2004-02-13 ltoutenc.dtx v1.99e
General: Documentation fixes: typos 509
2004-02-15 ltbibl.dtx v1.1q
\@cite@ofmt: Added hook with default value \hbox 1057
\@citex: Changed to use a hook with default value \hbox 1055
2004-02-15 ltspace.dtx v1.3a
\nobreakdashes: Added spacefactor setting 479

2005-07-27 ltfssdcl.dtx v3.0j
\DeclareMathAlphabet: (MH) Make document commands robust . . . 720
\DeclareSymbolFontAlphabet: (MH) Make document commands robust 732
\new@mathalphabet: (MH) Make document commands robust . . . 721
\non@alpherr: (MH) Change because command is now properly robust 706
\SetMathAlphabet: (MH) Make document commands robust . . . 721
2005-09-27 ltoutenc.dtx v1.99g
General: Replace \sh@ft by \ltx@sh@ft 526, 528, 535
2005-09-27 ltplain.dtx v1.1y
\ltx@sh@ft: New macro 30
\sh@ft: Macro no longer used but left for compatibility 30
2005-11-08 ltoutenc.dtx v1.99h
General: Added \ij and \IJ from babel. (pr/3771) 523, 527, 529
2005-11-10 ltmath.dtx v1.1g
\[: (MH) Fixed potential problem in \[(pr/3399). 897
General: (MH) Minor documentation fixes. 889
2006-05-18 ltboxes.dtx v1.1g
\@parboxto: Ensure \@parboxto holds the value of \@tempdimb not the register itself (pr/3867) 936
2006-09-13 ltoutput.dtx v1.1m
General: Ensure that rule is in \normalcolor 1303
2007-08-05 ltclass.dtx v1.1h
\@fileswithoptions: Prevent loss of brackets PR/3965 1120
2007-08-06 ltcntrl.dtx v1.0h
\@fornoop: Really make defs long . . 426
2007-08-31 ltfssdcl.dtx v3.0l
\SetSymbolFont@: Font warning changed to info for encoding change (pr/3975) 719
2009-09-24 ltvers.dtx v1.0l
General: Stop checking for old format 35
2009-10-20 ltfssdcl.dtx v3.0m
\in@: More robust thanks to Heiko. 702
2009-10-28 lttextcomp.dtx v1.99k
General: Added Latin Modern and TeX Gyre subsets 829
2009-11-04 lttextcomp.dtx v1.99l
General: Added more Latin Modern and TeX Gyre subsets 829
2009-12-14 ltfntcmd.dtx v3.4a
\ifmaybe@ic: Macro added 795
\maybe@ic@: Use switch \ifmaybe@ic instead of \if@tempswa 795
\t@st@ic: Use switch \ifmaybe@ic instead of \if@tempswa 796
2010-08-17 ltmiscen.dtx v1.1k
\enddocument: Use braces around \input arg (pr/4124) 866
2010-08-17 ltmiscen.dtx v1.1l
\enddocument: Change of plan: use \@@input instead (pr/4124) 866
2011-05-08 ltfssdcl.dtx v3.0n
\in@: Simplified thanks to Bruno. . . 702
2011-08-19 ltclass.dtx v1.1i
\@ifclasswith: Re-jig definition after more stringent \in@ test. 1106
2011-09-03 ltfssdcl.dtx v3.0o
\new@mathversion: (Will) Remove \global before \newcount (unnecessary and caused etex bug). 715
2012-01-20 ltplain.dtx v2.0b
\loggingall: etex tracing if available 31
2013-07-07 ltclass.dtx v1.1i
General: Correctly describe how the date in \@ifpackagelater is used 1097
2014-04-24 ltoutput.dtx v1.2n
\fl@tracemessage: Renamed internal trace commands; provide as package 1292
2014-04-27 ltfloat.dtx v1.2b
\end@dblfloat: Inline the code to allow some coexistence with packages that hook into \end@float and do not know about the algorithm change . . . 1034
2014-06-10 ltfloat.dtx v1.2b
\end@dblfloat: missing \fi added 1034
2014-12-30 ltfinal.dtx v2.0a
\newmarks: macro added 1332
\newXeTeXintercharclass: macro added 1333
2014-12-30 ltfloat.dtx v1.2a
\@textsubscript: Command added (latexrelease) 1045
\textsubscript: Command added (latexrelease) 1045
2014-12-30 ltfssbas.dtx v3.0y
\mathgroup: move allocation to ltplain. 571
2014-12-30 ltoutput.dtx v1.2m
General: Command updated (latexrelease) 1303

2014-12-30 ltplain.dtx v2.0a
\e@alloc: macro added 18
\e@alloc@chardef: macro added . . . 18
\e@alloc@top: macro added 18
\e@ch@ck: macro added 18
\extrafloats: macro added 18
\newlanguage: New engine-specific allocation scheme (latexrelease) . . 17
2014-12-30 ltspace.dtx v1.3b
\@: \@ discards spaces when moving (pr3039)(latexrelease) 481
2015-01-03 ltdefns.dtx v1.4a
\typein: use modified definition in luatex 82
2015-01-03 ltdirchk.dtx v1.1
General: Enable extra primitives when LuaTeX is used 3
2015-01-03 ltfinal.dtx v2.0a
General: Skip resetting codes with Unicode engines 1346
Unicode data loading added . . . 1336
2015-01-07 ltvers.dtx v1.0n
\@check@IncludeInRelease: macro added 37
2015-01-08 ltboxes.dtx v1.1h
\framebox: Make Robust (latexrelease) 933
\makebox: Make Robust (latexrelease) 926
\parbox: Make Robust (latexrelease) 935
\raisebox: Make Robust (latexrelease) 944
\rule: Make Robust (latexrelease) . 942
\savebox: Make Robust (latexrelease) 931
2015-01-08 ltdefns.dtx v1.4a
\MakeRobust: Added macro 95
2015-01-08 ltlength.dtx v1.1c
\setlength: to ensure first length argument is terminated. (latexrelease) 569
2015-01-08 ltmath.dtx v1.1h
\): Make Robust (latexrelease) 897
\]: Make Robust (latexrelease) 897
2015-01-09 ltfssini.dtx v3.1a
\em: Allow \emph to produce small caps (latexrelease) 758
\eminnershape: macro added (latexrelease) 758
2015-01-09 ltspace.dtx v1.1h
\addpenalty: Donald Arseneau's fix from PR/377703 (latexrelease) . . 475
2015-01-10 ltcounts.dtx v1.1h
\@fnsymbol: Unse \TextOrMath (latexrelease) 567
\@stpelt: Reset all within counters in one go (latexrelease) 559
2015-01-11 ltcounts.dtx v1.1h
\TextOrMath: Add command to solve robustness issues (pr/3752) (latexrelease) 567
2015-01-11 ltfloat.dtx v1.2b
\@dblfloatplacement: float order in 2-column (latexrelease) 1036
\@xfloat: Check for valid option (latexrelease) 1030
\end@dblfloat: float order in 2-column (latexrelease) 1034
2015-01-11 ltfssbas.dtx v3.0y
\@DeclareMathSizes: Allow arbitrary units (latexrelease) 580
2015-01-11 ltspace.dtx v1.3d
\@Esphack: Allow hyphenation (Donald Arseneau pr/3498) (latexrelease) 473
\@esphack: Allow hyphenation (Donald Arseneau pr/3498) (latexrelease) 471
2015-01-14 ltoutput.dtx v1.2n
\@addtocurcol: float order in 2-column (latexrelease) 1268
\@addtodblcol: float order in 2-column (latexrelease) 1283
\@addtonextcol: float order in 2-column (latexrelease) 1278
\@doclearpage: Empty kludgeins box if necessary, pr/3528 1239
float order in 2-column (latexrelease) 1238
\@startdblcolumn: float order in 2-column (latexrelease) 1262
\@xtryfc: float order in 2-column (latexrelease) 1264
\@ztryfc: float order in 2-column (latexrelease) 1265
2015-01-14 ltspace.dtx v1.3e
\addpenalty: Avoid adding redundant skips (DPC) 476
2015-01-17 ltvers.dtx v1.0m
\@check@IncludeInRelease: modified with \@currname 37
2015-01-19 ltvers.dtx v1.0o
\@check@IncludeInRelease: Optional argument 37
2015-01-20 ltoutput.dtx v1.2m
\fl@tracemessage: Reset \IncludeInRelease flags 1293

2015-01-22 ltvers.dtx v1.0p
General: Preserve any `\everyjob` material inserted by a loader (`.ini` file) 36
2015-01-23 ltfinal.dtx v2.0b
`\newmarks`: use reserved count 256 1332
`\newXeTeXintercharclass`: use reserved count 257 1333
2015-01-23 ltplain.dtx v2.0c
`\extrafloats`: reserve counts 256–265 18
2015-01-24 ltfinal.dtx v2.0c
General: Skip T1-code entirely with Unicode engines 1336
2015-02-03 ltfinal.dtx v2.0d
General: Set `\lccode` for `-` with Unicode engines 1337
2015-02-16 ltoutenc.dtx v1.99m
General: Added `\textcommabelow` latex/4414 524
2015-02-16 ltoutenc.dtx v1.99n
General: Added `\textcommaabove` . . 525
Added composites for ç 533
Added composites for `\c` 528
2015-02-16 lttextcomp.dtx v1.99m
General: Added lmtt (Heiko Oberdiek) latex/4415 829
2015-02-19 ltvers.dtx v1.0q
`\@check@IncludeInRelease`: Swap argument order 37
2015-02-20 ltplain.dtx v2.0d
`\loggingall`: Spell commands correctly :-) 31
2015-02-21 ltdefns.dtx v1.4b
General: Removed autoload support . 80
2015-02-21 lterror.dtx v1.2o
General: Removed autoload support 427
2015-02-21 ltfiles.dtx v1.1m
General: Removed autoload support 485
2015-02-21 ltfssbas.dtx v3.0z
General: Removed autoload code . . 571
2015-02-21 ltfsscmp.dtx v3.0d
General: Removed autoload code . . 697
2015-02-21 ltfssdcl.dtx v3.0p
General: Removed autoload code . . 702
2015-02-21 ltfsstrc.dtx v3.0k
General: Removed autoload code . . 671
2015-02-21 ltoutenc.dtx v1.99m
General: Removed autoload code . . 509
2015-02-21 ltoutput.dtx v1.2n
General: Removed autoload code . 1214
`\f@depth`: macro added(latexrelease) 1238
2015-02-21 ltpictur.dtx v1.1k
General: Removed autoload code . . 974
2015-02-21 ltplain.dtx v2.0e
General: Removed autoload code . . . 14
2015-02-21 lttab.dtx v1.1n
General: Removed autoload code . . 947
2015-02-21 ltvers.dtx v1.0r
General: Removed autoload code . . . 35
2015-02-21 ltvers.dtx v1.0w
`\@check@IncludeInRelease`: set `\@currname` empty here (in case `\IncludeInRelease` input early) . 37
2015-02-22 ltfsscmp.dtx v3.0e
General: Moved all code into latexrelease - obsolete commands are no longer automatically part of the kernel 697
2015-03-02 ltplain.dtx v2.0f
`\e@mathgroup@top`: macro added . . . 18
`\newlanguage`: allow 255 math groups in Unicode engines 17
2015-03-10 ltplain.dtx v2.0g
`\hideoutput`: macro added 33
`\loggingall`: Reorganize to be less noisy 31
`\tracingnone`: macro added 32
2015-03-12 ltoutput.dtx v1.2m
General: initialize `\@dbldeferlist` again 1225
2015-03-18 ltfssdcl.dtx v3.0q
`\DeclareSymbolFont`: Restrict Symbol fonts to 0-15 717
`\document@select@group`: Introduce `\e@mathgroup@top` 707
`\select@group`: Introduce `\e@mathgroup@top` 705
2015-03-26 ltfinal.dtx v2.0d
General: Use renamed `unicode-letters.def` 1336
2015-04-07 ltfssbas.dtx v3.1a
`\wrong@fontshape`: Try loading fd file if family has changed 592
2015-04-28 ltfinal.dtx v2.0f
`\newXeTeXintercharclass`: define `\xe@alloc@intercharclass` for compatibility with older xelatex initialization 1333
2015-05-10 ltlists.dtx v1.0t
`\@doendpe`: Explicitly reset `\clubpenalty` before clearing `\everypar`; see also pr/0462 and pr/4065 918
2015-06-19 ltfinal.dtx v2.0g
`\e@alloc@intercharclass@top`: Use −1 for first range to get contiguous allocation 1333

`\newmarks`: Use −1 for first range to get contiguous allocation 1332
2015-06-19 ltplain.dtx v2.0h
General: delete spurious old definition of `\newtoks` 20
`\e@alloc`: extra braces in case arguments not single token 18
`\newlanguage`: Use −1 for first range to get contiguous allocation 17
2015-06-23 ltfinal.dtx v2.0h
General: set `\patch@level` in ltvers rather than in ltfinal/ltpatch . . 1350
2015-06-23 ltvers.dtx v1.0t
General: set `\patch@level` in ltvers rather than in ltfinal/ltpatch 35
2015-08-06 ltplain.dtx v2.0i
`\extrafloats`: Add `\string` in case argument is not an unexpandable primitive 19
2015-08-23 ltdirchk.dtx v1.2
General: Do not use luatex prefix 3
2015-08-23 ltvers.dtx v1.0v
General: Allow negative patchlevel for pre-release 36
2015-08-30 ltplain.dtx v2.1a
`\newinsert`: new `\newinsert` implementation 20
2015-09-205 ltoutput.dtx v1.3a
General: extended `\@freelist` . . . 1224
2015-09-24 ltluatex.dtx v1.0a
`call_callback`: Function added 66
`callback.register`: Function modified 62
`callback_identifiers`: Function added 70
`\catcodetable@atletter`: Macro added 48
`\catcodetable@initex`: Macro added 48
`\catcodetable@latex`: Macro added . 48
`\catcodetable@string`: Macro added 48
`add_to_callback`: Function added . . 67
`remove_from_callback`: Function added 69
`new_attribute`: Function added 55
`disable_callback`: Function added . 70
`in_callback`: Function added 66
`\newattribute`: Macro added 47
`\newcatcodetable`: Macro added . . . 47
`\newluabytecode`: Macro added 50
`\newluachunkname`: Macro added . . . 50
`\newluafunction`: Macro added 49
`\newwhatsit`: Macro added 50
`module_error`: Function added 54
`module_info`: Function added 54
`module_warning`: Function added . . . 54
`modules`: Function modified 53
`create_callback`: Function added . . 65
`provides_module`: Function added . . 53
`luatexbase`: Table added 52
2015-10-02 ltdirchk.dtx v1.2a
General: Allow backing out of unprefixed names 3
2015-10-02 ltluatex.dtx v1.0b
General: Fix backing out of TEX code 51
2015-10-02 ltluatex.dtx v1.0c
General: Allow backing out of Lua code 51
2015-10-02 ltluatex.dtx v1.0e
`uninstall`: Function added 70
2015-10-03 ltluatex.dtx v1.0f
`provides_module`: use luatexbase_log 53
2015-10-27 ltplain.dtx v2.1b
`\extrafloats`: Use global assignment when switching to extended range 19
2015-11-07 ltspace.dtx v1.3f
`\@esphack`: Only space if there is no space at the end of the hlist latex/4443 471
2015-11-14 ltluatex.dtx v1.0g
General: Track LuaTEX changes for `(new)token.create` 54
2015-11-18 ltplain.dtx v2.2a
`\newlanguage`: Extended stream allocation in luatex (0.85) 17
2015-11-19 ltplain.dtx v2.2b
`\newlanguage`: Only extend allocation of write streams (see luatex list) . 17
2015-11-27 ltluatex.dtx v1.0h
`callback_identifiers`: Match test in in-callback latex/4445 70
`in_callback`: Guard against undefined list latex/4445 66
2015-11-29 ltluatex.dtx v1.0i
General: Declare this as local before used in the module error definitions (PHG) 52
`call_callback`: Check name is not nil in error message (PHG) 66
`create_callback`: Check name is not nil in error message (PHG) 65
2015-12-02 ltluatex.dtx v1.0j
General: Adjust hashtokens to store the result of tex.hashtokens()), not the function (PHG) 54
Assorted typos fixed (PHG) 45
Declaration/use of first_head fixed (PHG) 53

Remove nonlocal iteration variables (PHG) 45
Remove unreachable code after calls to error() (PHG) 45
2015-12-02 ltluatex.dtx v1.0k
General: resolve name and i.description (PHG) 64
`call_callback`: Give more specific error messages (PHG) 66
`add_to_callback`: Give more specific error messages (PHG) 67
`remove_from_callback`: adjust initialization of cb local (PHG) . . 69
Give more specific error messages (PHG) 69
`create_callback`: Give more specific error messages (PHG) 65
2015-12-10 ltfinal.dtx v2.0i
General: Use new common Unicode data loaders 1336
2015-12-18 ltluatex.dtx v1.0l
General: Load Unicode data from source 47
2016-01-04 ltfinal.dtx v2.0j
General: Do not set up inter character classes for XeTeX 1336
`\e@alloc@intercharclass@top`: Start allocation at one not three 1333
2016-01-05 ltfinal.dtx v2.0k
`\e@alloc@intercharclass@top`: Remove duplicated code 1333
2016-01-05 ltfinal.dtx v2.0l
General: Correct latexrelease guards 1336
Ensure old definitions for inter-character class toks are available using latexrelease 1336
Missing brace 1336
2016-01-05 ltfinal.dtx v2.0m
General: Undefine XeTeX classes when using patching an older kernel . 1336
2016-01-05 ltfinal.dtx v2.0p
General: Only apply XeTeX change if XeTeX is in use 1336
2016-02-11 ltluatex.dtx v1.0m
General: pdf_stream_filter_callback removed 62
process_rule, [hv]pack_quality append_to_vlist_filter added . . . 61
read_cidmap_file added 61
show_warning_message added . . . 61
token_filter removed 61
2016-02-18 ltfssdcl.dtx v3.0r
`\@DeclareMathDelimiter`: Check for delimiter not `\delimiter` 728
`\DeclareMathAccent`: Check for mathaccent not `\mathaccemt` . . . 723
`\DeclareMathRadical`: Check for radical not `\radical` 731
`\DeclareMathSymbol`: Check for mathchar not `\mathchar` 726
2016-03-13 ltluatex.dtx v1.0n
General: contribute_filter added 61
insert_local_par added 61
2016-03-29 ltpictur.dtx v1.1l
`\@oval`: add setting of line tests . . . 997
initialize tests 996
`\@ovhorz`: use glue not leaders if horizontal line not required 999
`\@ovvert`: use glue not leaders if vertical line not required 998
`\if@ovhline`: macro added (latex/4452) 996
`\if@ovvline`: macro added (latex/4452) 996
2016-04-22 ltfinal.dtx v2.0q
`\e@alloc@intercharclass@top`: XeTeX 0.99996 has 4096 char classes not 256 1333
2016-06-19 ltoutenc.dtx v1.99m
General: OT1 definition (was duplicate T1 definition) 528
2016-06-20 ltclass.dtx v1.1j
General: don't declare as `\@onlypreamble` 1105
2016-07-29 ltplain.dtx v2.2c
`\extrafloats`: use `\global \chardef` 19
`\newinsert`: fix for tlb-newinsert-001 20
2016-10-02 ltclass.dtx v1.2a
`\@ifclasswith`: Ignore spaces while checking for option clash 1106
`\ExecuteOptions`: Ignore spaces in argument 1116
2016-10-15 ltdirchk.dtx v1.2b
General: Require eTEX 3
2016-10-15 lterror.dtx v1.2p
General: Require eTEX 427
2016-10-15 ltfinal.dtx v2.0r
General: Require eTEX 1332
2016-10-15 ltfinal.dtx v2.0s
General: Tidy up status of char 127 1332
2016-10-15 ltfssini.dtx v3.1b
General: Require eTEX 735
2016-10-15 ltplain.dtx v2.2d
General: Require eTEX 14
2016-10-16 ltplain.dtx v2.3a
`\newlanguage`: Allow languages up to 16383 in luatex 17

2016-10-19 ltcounts.dtx v1.1j
\TextOrMath: Test directly for \protected 567
2016-11-06 ltplain.dtx v2.3b
General: Drop \outer entirely 14
2016-11-09 ltclass.dtx v2.1b
\@fileswithoptions: Improve \ifx tests PR/4497 1120
2016-11-17 ltluatex.dtx v1.0p
General: call_edit added 61
2016-12-03 fontdef.dtx v3.0a
General: (DPC) Default to TU encoding for Unicode TeX engines 768
\shapedefault: (DPC) Default to TU encoding for Unicode TeX engines 772
2016-12-04 ltoutenc.dtx v2.0a
General: Added TU encoding 541
2017-01-01 ltoutput.dtx v1.3b
General: make fpmin negative so ignored even if float height is negative 1302
2017-01-10 ltfssbas.dtx v3.2a
\showhyphens: Add version of \showhyphens that works with XeTeX. 598
2017-01-23 ltoutenc.dtx v2.0b
General: Added TU specific commands in ASCII range pr/4500 541
2017-01-24 ltoutenc.dtx v2.0c
General: Declare TU composites for i and j 541
Make \textasteriskcentered U+2217 not U+204E 541
TeX ligature syntax for xetex and luatex reversed 541
2017-01-24 ltoutenc.dtx v2.0d
General: Declare macron composites for YyGg 541
2017-02-12 ltoutenc.dtx v2.0e
General: Declare fallback code for \textasteriskcentered 541
2017-02-18 ltluatex.dtx v1.1c
new_attribute: Parameterize count used in tracking 55
new_bytecode: Parameterize count used in tracking 56
new_chunkname: Parameterize count used in tracking 57
new_whatsit: Parameterize count used in tracking 56
2017-02-19 ltoutenc.dtx v2.0f
General: add \@empty to guard against 3rd argument being empty 527
declare composites with empty base for hat and tilde, use same slots for \textasciicircum ans \textasciitilde 541
declare straight quotes using new \remove@tlig command 541
2017-02-22 ltoutenc.dtx v2.0g
General: Fix typo introduced at 2.0f 541
2017-02-24 ltoutenc.dtx v2.0h
General: introduce \DeclareUnicodeAccent 541
\DeclareTextCompositeCommand: add check whether the accent command is defined for this encoding 517
2017-03-08 ltclass.dtx v1.2c
General: add \@parse@version@dash to support yyyy-mm-dd as well as yyyy/mm/dd 1105
2017-03-09 ltfinal.dtx v2.0t
\l@nohyphenation: ensure \l@nohyphenation is defined. . 1339
2017-03-09 ltmiscen.dtx v1.1m
\@verbatim: Use \language not \hyphenchar 882
\verb: Use \language to stop hyphenation 887
2017-03-10 ltfiles.dtx v1.1n
\document: Save language default . . 488
2017-03-10 ltoutput.dtx v1.3c
\@writesetup: Reset \language . . 1254
2017-03-13 ltdefns.dtx v1.5a
\-: Define \- in terms of \hyphenchar 112
2017-03-27 ltdefns.dtx v1.5b
\@dischyph: Define \@dischyph after \- 112
2017-03-28 ltluatex.dtx v1.1e
General: glyph_stream_provider added 62
2017-03-29 ltboxes.dtx v1.3a
\@arrayparboxrestore: Reset \lineskiplimit 938
2017-04-05 ltoutenc.dtx v2.0i
\DeclareTextCompositeCommand: Declare accent command if not already declared when declaring a composite. 517
2017-04-10 ltplain.dtx v2.3c
\newlanguage: Correction to code to skip write18 in luatex 17
2017-04-11 ltoutput.dtx v2.4a
\newpage: account for the depth of the last row of the page 1228
2017-12-17 ltoutput.dtx v1.4b
\@addtonextcol: fix doc guards . . 1278

2018-01-06 ltdefns.dtx 1.5c
\@ifundefined: Avoid defining undefined commands to \relax . 108
2018-02-18 ltclass.dtx v1.2d
\@ifl@ter: Added 0 up front to make bad data come out as 0. 1105
General: Introduce rollback concept 1139
2018-03-08 ltcounts.dtx v1.1k
\@ifbothcounters: Interface added 563
\@removefromreset: Interface added 562
\counterwithin: Interface added . . 564
\counterwithout: Interface added . 565
2018-03-24 ltclass.dtx v1.2e
\pkgcls@use@this@release: Use full file name for old release 1145
2018-03-25 ltfinal.dtx v2.1a
General: default to UTF-8 1342
\UseRawInputEncoding: Macro added 1343
2018-03-27 ltclass.dtx v1.2f
\endfilecontents: Use full file name for old release 1133
2018-04-06 ltfinal.dtx v2.1b
\UseRawInputEncoding: Undo changes to \DeclareFontEncoding@ and definition of \DeclareUnicodeCharacter . . . 1343
2018-04-07 ltfinal.dtx v2.1c
\UseRawInputEncoding: Undefine \inputencodingname 1343
2018-04-08 ltclass.dtx v1.2g
\@ifl@ter: Strip leading spaces from dates. 1105
2018-04-08 ltclass.dtx v1.2h
\@onefilewithoptions: Pass expanded date 1141
2018-04-08 ltfinal.dtx v2.1d
General: Delay full UTF-8 handling to \everyjob 1343
2018-04-11 ltcounts.dtx v1.1l
\counterwithin: Correct default (issue/38) 564
2018-05-02 ltluatex.dtx v1.1g
General: find_sfd_file removed 60
finish_synctex_callback added . . . 61
glyph_not_found added 62
read_sfd_file removed 61
2018-05-08 ltclass.dtx v1.2i
\pkgcls@parse@date@arg: Make suspicious rollback a warning not error: github issue 43 1143
2018-05-11 ltfinal.dtx v2.18
General: Make invalid UTF-8 also safe, for legacy filesystem encodings . 1343
2018-05-29 ltclass.dtx v1.2j
\endfilecontents: use \csname not \@undefined 1134
2018-08-11 ltoutenc.dtx v2.0j
General: Provide \guillemetleft and \guillemetright 530, 535, 545
2018-08-18 ltluatex.dtx v1.1h
General: append_to_vlist_filter is exclusive 61
2018-08-24 ltfinal.dtx v2.1f
\document@default@language: Add to latexrelease (github/68) 1339
2018-09-02 ltsect.dtx v1.1b
\@dottedtocline: Prevent protrusion (https://tex.stackexchange.com/q/172785/10109) 1024
2018-09-24 fontdef.dtx v3.0b
General: Start LR-mode if necessary (gh/49) 785
2018-09-24 ltmath.dtx v1.2b
\smash: Start LR-mode if necessary (gh/49) 893
\vphantom: Start LR-mode if necessary (gh/49) 892
2018-09-24 ltspace.dtx v1.3h
\enspace: Start LR-mode if necessary (gh/49) 482
\leavevmode@ifvmode: Macro added (gh/49) 482
2018-09-26 ltdefns.dtx v1.5e
\renew@command: Always explicitly generate a space after the csname and not rely on \noexpand to save tokens (gh/41) 87
2018-09-26 ltmiscen.dtx v1.1n
\@writefile: Sometimes mask the endline char when writing to files (github/73) 871
\add@percent@to@temptokena: Sometimes mask the endline char when writing to files (github/73) 871
\protected@file@percent: Sometimes mask the endline char when writing to files (github/73) 871
2018-09-26 ltsect.dtx v1.1c
\addcontentsline: Sometimes mask the endline char when writing to files (github/73) 1021
2018-10-10 ltspace.dtx v1.3i
\@esphack: Don't introduce breakpoints if @nobreak is true and after sections 471

2018-10-11 ltmiscen.dtx v1.1o
\@@sverb: Provide visible space in \verb* also for XeTeX and LuaTeX (github/69) 885
\@setupverbvisiblespace: Provide visible space in \verb* also for XeTeX and LuaTeX (github/69) 884
\@verbvisiblespacebox: Provide visible space in \verb* also for XeTeX and LuaTeX (github/69) 883
\asciispace: Provide visible space in \verb* also for XeTeX and LuaTeX (github/69) 883
verbatim*: Provide visible space in \verb* also for XeTeX and LuaTeX (github/69) 883
\verbvisiblespace: Provide \verbvisiblespace such that it is usable in normal text (github/70) 883
Provide visible space in \verb* also for XeTeX and LuaTeX (github/69) 883
2018-10-21 ltluatex.dtx v1.1i
new_luafunction: Function added . . 57
2018-11-09 ltbibl.dtx LaTeX2e
\bibliography: Zap spaces in the argument as BibTeX doesn't support them (github/88) 1055
2018-11-18 ltoutenc.dtx v2.0k
General: Provide \Hwithstroke and \hwithstroke 547
2018-11-19 ltoutenc.dtx v2.0k
General: Added \Hwithstroke and \hwithstroke 529
2018-11-28 ltoutput.dtx v1.4d
\@combinedblfloats: Unbox \@outputbox to preserve boxing level (github/94) 1262
2018-12-30 lttab.dtx v1.1p
\@tabclassz: Add extra \hskip to guard against an \unskip at the start of a c-column cell (gh/102) 968
2019-02-07 ltfilehook.dtx v1.1o
\unqu@tefilef@und: Expand \@filef@und before executing second argument (github/109) . 1170
2019-02-07 ltfiles.dtx v1.1o
\@swaptwoargs: Helper macro added 503
2019-02-07 ltfssbas.dtx v3.2b
\define@newfont: Changed wording of warning (github/107) 588
2019-06-18 ltluatex.dtx v1.1j
General: finish_synctex_callback renamed finish_synctex 61
font_descriptor_objnum_provider added 62
make_extensible added 62
new_graf added 61
page_objnum_provider added . . . 62
process_pdf_image_content added 62
wrapup_run added 61
2019-07-01 ltclass.dtx v1.3a
\endfilecontents: Support UTF8 and spaces in filecontents environment file name 1131
2019-07-01 ltfiles.dtx v1.2a
\IfFileExists: Support UTF-8 . . . 500
\includeonly: Support UTF-8 493
\set@curr@file: Support UTF-8 . . 499
2019-07-09 ltfssbas.dtx v3.2c
\DeclareErrorFont: Don't set any \f@... macros 591
2019-07-09 ltfssini.dtx v3.1c
General: Explicitly set some defaults 764
2019-08-22 ltxref.dtx v1.1l
\labelformat: Commanded moved from varioref.sty 847
\Ref: Commanded moved from varioref.sty 847
\refstepcounter: Allow \p@... to have an argument 846
2019-08-27 fontdef.dtx v3.0c
General: Various commands made robust throughout the file 778
2019-08-27 ltboxes.dtx v1.3b
General: Various commands made robust 925
2019-08-27 ltclass.dtx v1.3b
\endfilecontents: Make various commands robust 1131
2019-08-27 ltdefns.dtx v1.5f
General: Make various commands robust 113
\MakeRobust: Make the assignments global as we may need to apply them inside a group 95
2019-08-27 ltfilehook.dtx v1.2b
\unqu@tefilef@und: Make command robust 1170
2019-08-27 ltfiles.dtx v1.2b
\IfFileExists: Make command robust 500
2019-08-27 ltfssbas.dtx v3.2d
General: Make various commands robust 571
2019-08-27 ltfssdcl.dtx v3.0s
\DeclareMathAccent: Make math accents robust 723

`\set@mathdelimiter`: Make math delimiters robust 730
2019-08-27 ltfssini.dtx v3.1d
General: Make various commands robust 735
2019-08-27 ltidxglo.dtx v1.1f
General: Make `\index` and `\glossary` robust 1050
2019-08-27 ltlength.dtx v1.1d
General: Make various command robust 569
2019-08-27 ltlogos.dtx v1.1j
`\TeX`: Make `\TeX` command robust . 484
2019-08-27 ltmath.dtx v1.2c
General: Make various commands robust 889
2019-08-27 ltmiscen.dtx v1.1p
General: Make various commands robust 864
`\begin`: Make command robust . . . 873
`\end`: Make command robust 875
2019-08-27 ltoutput.dtx v1.4e
`\@begindvibox`: Make `\AtBeginDvi` robust 1225
2019-08-27 ltpage.dtx v1.0l
General: Make various commands robust 1089
2019-08-27 ltpictur.dtx v1.1m
General: Make various commands robust 974
Remove several unnecessary `\gdef` definitions 974
2019-08-27 ltsect.dtx v1.1d
General: Make various commands robust 1012
2019-08-27 ltspace.dtx v1.3j
General: Make various commands robust 463
2019-08-27 lttab.dtx v1.1q
General: Make various commands robust 947
Remove several unnecessary `\gdef` definitions 947
2019-08-30 lterror.dtx v1.2q
`\conditionally@traceoff`: Macro added 437
`\conditionally@traceon`: Macro added 437
2019-09-09 ltfssdcl.dtx v3.0s
`\DeclareMathSymbol`: Allow definition if the math symbol was a command already robust 726
2019-09-11 ltclass.dtx v1.3c
`\endfilecontents`: Support optional argument for filecontents 1131
2019-09-14 ltfinal.dtx v2.1h
`\@uclclist`: Expand UTF8 chars when case changing (github/177) 1347
2019-09-16 ltxref.dtx v1.1m
General: Correctly revert the `\p@...` change 847
2019-09-21 fontdef.dtx v3.0d
General: Distangle alias (gh/184) 779, 783
2019-10-02 ltexpl.dtx v0.0
General: Initial version 72
2019-10-02 ltfinal.dtx v2.2
General: Load `ltexpl` 1351
2019-10-02 ltluatex.dtx v1.1k
General: linebreak_filter is `exclusive` 61
mlist_to_hlist is `exclusive` 61
process_rule is `exclusive` 61
2019-10-07 lttab.dtx v1.1q
`\extracolsep`: This needs to expand 959
2019-10-11 ltfiles.dtx v1.2c
`\set@curr@file`: Remove one brace group 499
2019-10-11 ltfsstrc.dtx v3.0l
`\@font@aliasinfo`: Added 'alias' size function 694
2019-10-18 ltclass.dtx v1.3d
`\load@onefilewithoptions`: Initialize `\...-h@@k` only when loading the package or class (gh/198) 1122
2019-10-22 ltluatex.dtx v1.1j
General: page_objnum_provider and process_pdf_image_content classified data 62
2019-10-25 ltmiscen.dtx v1.1q
`\add@percent@to@temptokena`: Allow unbalanced conditionals in `#1` (gh/202) 871
2019-10-26 ltfiles.dtx v1.2d
`\@iffileonpath`: quote on openin . . 502
`\IfFileExists`: don't quote name . 500
`\IfFileExists@`: quote on openin . . 500
`\set@curr@file`: remove quotes . . . 499
2019-11-01 ltdirchk.dtx v1.3a
`\filename@parse`: take last . not first 12
2019-11-02 ltmiscen.dtx v1.1s
`\@centercr`: Make `\@centercr` robust (gh/203) 878
2019-11-02 ltspace.dtx v1.3k
`\@normalcr`: Make also `\@normalcr` robust 467

2019-11-09 ltfiles.dtx v1.2e
`\set@curr@file`: expand and `\string` before removing quotes 499
2019-11-10 ltmiscen.dtx v1.1r
`\add@percent@to@temptokena`: fix to special comment catcodes (gh/202) 871
2019-11-11 ltfiles.dtx v1.2f
`\@iffileonpath`: make `\@filef@und` match quoting used on `\openin` . 502
2019-11-22 ltoutenc.dtx v2.0l
General: Avoid spurious if fontenc selects LY1 as default encoding (gh/199) 555
2019-11-29 ltclass.dtx v1.3e
`\@pr@videpackage`: Protect package info text (gh/52) 1109
2019-12-17 fontdef.dtx v3.0e
`\mddefault`: Set `\bfdefault` to "b" 771
`\shapedefault`: Set `\shapedefault` explicitly to "n" 772
`\updefault`: Set `\updefault` to "up" 771
2019-12-17 ltfntcmd.dtx v3.4c
`\textssc`: Macro added 793
2019-12-17 ltfssbas.dtx v3.2e
`\usefont`: Don't call `\fontseries` or `\fontshape` 582
2019-12-17 ltfssini.dtx v3.1e
General: Provide custom series settings a la mweights 736
`\DeclareEmphSequence`: Provide `\emph` sequences 756
2019-12-18 ltoutenc.dtx v2.0m
General: Don't fake `\textcompwordmark`; take default from T1 instead 523
`\add@accent`: Avoid code that breaks `\accent` 516
2019-12-21 fontdef.dtx v3.0e
General: Distangle alias (gh/184) 777–780
2020-01-05 ltclass.dtx v1.3f
`\endfilecontents`: Support more write streams in LuaTeX gh/238 1131
2020-01-11 ltfssini.dtx v3.1f
`\rmfamily`: Streamlined implementation with hook 751
`\ttfamily`: Streamlined implementation with hook 751
2020-01-20 ltfssdcl.dtx v3.0t
`\set@mathdelimiter`: fix for gh/251 730
2020-01-20 ltoutenc.dtx v2.0n
General: fix for gh/251 524
2020-01-22 lttextcomp.dtx v1.0b
`\tc@subst`: The overall default is `\textcompsubstdefault` not `\substdefault` 800
2020-01-25 fontdef.dtx v3.0f
General: Load t1enc.def last (gh/255) 768
2020-01-25 ltoutenc.dtx v2.0m
General: Load each encoding file only once (gh/255) 554
2020-01-27 ltclass.dtx v1.3g
`\endfilecontents`: Fix typo in error message 1133
2020-01-28 ltclass.dtx v1.3h
`\endfilecontents`: Allow spaces in option string and display only unknown options not the whole option list (gh/256) 1131
2020-01-31 ltvers.dtx v1.1e
General: Allow for upcoming format as pre-release 0 36
2020-02-02 ltluatex.dtx v1.1l
General: Add reverselist callback type 64
glyph_info added 62
page_order_index added 62
post_linebreak_filter is `reverselist` 61
`create_callback`: Provide proper fallbacks for user-defined callbacks without user-provided default handler 65
2020-02-05 ltfssini.dtx v3.1g
`\DeclareFontSeriesDefault`: Clarified error text 737
Corrected misspelled csname (gh/264) 737
2020-02-05 lttextcomp.dtx v2.0n
General: Changed the package default to info (gh/262) 818
Ensure we are on a new format (gh/260) 818
2020-02-07 ltfssini.dtx v3.1h
`\symbol`: XeTeX-specific version to avoid bug in maths mode. 760
2020-02-10 ltfssaxes.dtx v1.0c
`\fontseries`: Switch `\if@forced@series` added 656
`\fontseriesforce`: Switch `\if@forced@series` added 656
`\if@forced@series`: Switch `\if@forced@series` added 656
2020-02-10 ltfssini.dtx v3.1h
`\@defaultfamilyhook`: Add `\@defaultfamilyhook` to `\normalfont` (gh/269) 752

`\reset@font`: Add `\@defaultfamilyhook` to `\normalfont` (gh/269) 761
2020-02-10 lttextcomp.dtx v1.0c
General: Use `\@tabaccludge` for tabbing where necessary (gh/271) 804
2020-02-11 fontdef.dtx v3.0g
General: Provide value for `\@fontenc@load@list` (gh/273) . 768
2020-02-11 ltfssini.dtx v3.1h
General: Provide default value for `\@fontenc@load@list` (gh/273) . 764
2020-02-11 ltoutenc.dtx v2.0o
General: Update `\@fontenc@load@list` with option list (gh/273) 555
2020-02-14 ltpictur.dtx v1.1n
`\linethickness`: Suppress spaces following the declaration (gh/274) 979
2020-02-18 ltfssini.dtx v3.1i
`\bfseries`: Make the `\ifx` selection outside of `\fontseries` argument so that it is not done several times 746
`\mdseries`: Make the `\ifx` selection outside of `\fontseries` argument so that it is not done several times 746
`\prepare@family@series@update`: No series auto-update when forced (gh/277) 740
Recognize current family if it is not a "meta" family and auto-update series using `\bfdefault` (gh/277) 741
2020-02-18 ltmath.dtx v1.2d
`\mathindent`: Make `\mathindent` a skip register to match amsmath (gh/252) 904
`equation`: Separate formula and eqn number by at least a space in fleqn option 905
2020-02-20 ltclass.dtx v1.3j
`\endfilecontents`: Fix missing quotes around file name (gh/284) 1133
2020-02-24 ltfssbas.dtx v3.2f
`\DeclareFontShape@`: Drop surplus "m" in series when defining fontshape (gh/289) 572
2020-02-25 ltfssini.dtx v3.1j
`\bf@def@ult`: Drop surplus "m" from `\bfdef@ult` and `\mddef@ult` (gh/291) 749
`\prepare@family@series@update`: Drop surplus "m" from `\target@series@value` (gh/291) 741
`\update@series@target@value`: Drop surplus "m" from `\reserved@d` (gh/291) 742
2020-02-27 ltdefns.dtx v1.5g
`\@gobblethree`: Macro added 90
2020-02-27 ltfssaxes.dtx v1.0d
`\series@maybe@drop@one@m`: Drop "m" in certain values from a fixed list (gh/293) 659
`\set@target@series`: Drop "m" only in a specific set of values (gh/293) 659
2020-02-27 ltfssbas.dtx v3.2g
`\DeclareFontShape@`: Only "m" if the series value is a member of a fixed list and issue warning if doing it (gh/293) 572
2020-03-02 ltexpl.dtx v1.0a
General: Don't load expl3 if already in the format (gh/295) 72
2020-03-05 ltexpl.dtx v1.1
General: Load xparse.ltx if `\NewDocumentCommand` is not defined by expl3.ltx 72
2020-03-06 ltboxes.dtx v1.3c
`\clap`: Macro `\clap` added 946
2020-03-07 ltluatex.dtx v1.1m
`remove_from_callback`: Do not call callback.register for user-defined callbacks 69
2020-03-07 ltmath.dtx v1.2e
`\negthickspace`: Add `amsmath` math/text spacing commands to the kernel (gh/303) 895
2020-03-07 ltspace.dtx v1.3l
General: Moved `\thinspace`, `\negthinspace` and `\,` to ltmath.dtx (gh/303) 463
2020-03-19 fontdef.dtx v3.0h
General: Support legacy use of `\bfdefault` and `\mddefault` (gh/306) 771
2020-03-19 ltfssdcl.dtx v3.0u
`\document@select@group`: fix for (gnats/3357) 708
2020-03-19 ltfssini.dtx v3.1k
`\DeclareFontSeriesDefault`: Support legacy use of `\bfdefault` and `\mddefault` (gh/306) 737
`\maybe@update@bfseries@defaults`: Support legacy use of `\bfdefault` and `\mddefault` (gh/306) 746
`\mdseries`: Support legacy use of `\bfdefault` and `\mddefault` (gh/306) 746

2020-04-06 ltfssini.dtx v3.1m
\bf@def@ult: Hook added (gh/306) 749
\maybe@update@bfseries@defaults: Hook added (gh/306) 746
\maybe@update@mdseries@defaults: Hook added (gh/306) 747
2020-04-07 ltclass.dtx v1.3k
\IfFormatAtLeastTF: Macro added; also in rollback (gh/168) 1104
\load@onefile@withoptions: Use different method to ignore unprocessed options (gh/22) . . 1126
\ProcessOptions*: Use different method to ignore unprocessed options (gh/22) 1115
\RequirePackageWithOptions: Use different method to ignore unprocessed options (gh/22) . . 1118
2020-04-09 ltfloat.dtx v1.2d
\@textsubscript: Set non-zero baseline (gh/249) 1045
\textsubscript: Set non-zero baseline (gh/249) 1045
2020-04-13 ltfssdcl.dtx v3.0v
\process@table: Small update for speed. 713
2020-04-13 ltfssini.dtx v3.1n
\init@series@setup: Handling \seriesdefault changes (gh/315) 745
\seriesdefault@kernel: Handling \seriesdefault changes (gh/315) 765
2020-04-21 ltmath.dtx v1.2f
\@yeqncr: Support calc syntax (gh/152) 902
2020-04-21 ltmiscen.dtx v1.1t
\@icentercr: Support calc syntax (gh/152) 879
2020-04-21 ltpictur.dtx v1.1o
\@istackcr: Support calc syntax (gh/152) 980
2020-04-21 ltspace.dtx v1.3m
\@hspace: Support calc syntax (gh/152) 481
\@newline: Support calc syntax (gh/152) 468
\@vspace@calcify: Support calc syntax (gh/152) 468
\@vspacer: Support calc syntax (gh/152) 478
\addvspace: Support calc syntax (gh/152) 474
2020-04-21 lttab.dtx v1.1r
\@itabcr: Support calc syntax (gh/152) 953
\@yargarraycr: Support calc syntax (gh/152) 963
2020-04-22 ltmiscen.dtx v1.1u
\@@sverb: Drop spaces before \verb delimiter (gh/327) 885
2020-04-22 ltoutenc.dtx v2.0p
General: y unicode value in tuenc.def 509
2020-04-29 lttextcomp.dtx v1.0d
General: Make all capital accents text commands for hyperref (gh/332) 804
2020-05-02 ltfiles.dtx v1.2g
\@include: Support spaces in filenames by enclosing the names of .aux-files in quotes (gh/217) . 495
\includeonly: Get rid of leading and trailing spaces from the filename (gh/217) 493
Improved support for spaces in filenames (gh/217) 493
Pass the filename to \@include by value instead of by reference (gh/217) 493
2020-05-05 ltxref.dtx v1.1n
\refstepcounter: record the counter name in \@currentcounter 845
2020-05-06 ltspace.dtx v1.3n
General: Made softhyphen active in TU engines 483
2020-05-09 ltdefns.dtx v1.5j
\@if@DeclareRobustCommand: Added \DeclareCommandCopy (gh/239) . 103
\DeclareCommandCopy: Added \DeclareCommandCopy (gh/239) . 100
2020-05-11 ltdefns.dtx v1.5j
\@dischyph: Do not overwrite \- under LuaTeX 112
2020-05-15 ltdefns.dtx v1.5g
\typeout: Allow \par in the argument (gh/335) 80
2020-05-19 ltfssaxes.dtx v1.0e
\series@maybe@drop@one@m: Need to use \edef (gh/336) 660
2020-05-19 ltfssini.dtx v3.2a
\IfFontSeriesContextTF: Macros added (gh/335) 753
2020-05-31 ltmiscen.dtx v1.1u
\centering: Added \finalhyphendemerits setting (gh/247) 879
\raggedleft: Added \finalhyphendemerits setting (gh/247) 880

`\raggedright`: Added `\finalhyphendemerits` setting (gh/247) 880
2020-06-04 ltexpl.dtx v1.2c
General: Define a local version of some LaTeX 2ε basic macros to support package loading 72
2020-06-04 ltfinal.dtx v2.2a
General: Load l3expl in `ltdefns` . . 1351
2020-06-05 ltclass.dtx v1.3l
`\@currnamestack`: Added `\@expl@pop@filename@@` 1102
Added `\@expl@push@filename@@` and `\@expl@push@filename@aux@@` . 1100
2020-06-05 ltfiles.dtx v1.2h
`\document`: Added hook to load l3backend code 487
2020-06-10 ltluatex.dtx v1.1n
General: Define `\@gobble`/`\@firstofone` even for LaTeX to allow early loading. 45
2020-07-04 ltoutenc.dtx v2.0q
General: Implement `\remove@tlig` in LuaTeXwithout font reloading . . 542
2020-07-08 ltexpl.dtx v1.2d
General: Add a last-minute hook for expl3 73
2020-07-08 ltfinal.dtx v2.2b
General: Add a last-minute hook for expl3 1338
2020-07-27 ltmath.dtx v1.2g
`\cases`: Don't make the command `\long` (gh/354) 893
`\matrix`: Don't make the command `\long` (gh/354) 893
`\pmatrix`: Don't make the command `\long` (gh/354) 893
2020-07-27 ltoutenc.dtx v2.0r
`\@use@text@encoding`: Don't make the command `\long` (gh/354) . . 519
2020-07-27 ltpage.dtx v1.0m
`\markright`: Don't make the command `\long` (gh/354) 1090
2020-07-27 ltpictur.dtx v1.1p
`\linethickness`: Don't make the command `\long` (gh/354) 979
2020-07-27 ltsect.dtx v1.1e
`\author`: Don't make the command `\long` (gh/354) 1012
`\date`: Don't make the command `\long` (gh/354) 1012
2020-08-01 ltluatex.dtx v1.1p
General: new_graf is `exclusive` 61
2020-08-02 ltluatex.dtx v1.1q
`\newattribute`: Move reset to 0 inside conditional 47
`\newluabytecode`: Move reset to 0 inside conditional 50
`\newluachunkname`: Move reset to 0 inside conditional 50
`\newluafunction`: Move reset to 0 inside conditional 49
`\newwhatsit`: Move reset to 0 inside conditional 50
2020-08-08 ltclass.dtx v1.3m
`\endfilecontents`: define `\q@curr@file` directly as the quotes have already been removed (gh/220) 1132
2020-08-10 ltluatex.dtx v1.1r
General: Load ltluatex Lua module during format building 51
2020-08-15 ltpictur.dtx v1.2a
`\@defaultunitsset`: Macro added . 976
2020-08-19 ltdefns.dtx v1.5k
`\@carcube`: Made `\long` for `\NewCommandCopy` 83
`\robust@command@act`: Made `\robust@command@act` (was `\declare@command@copy`) more generic 99
`\ShowCommand`: Added `\ShowCommand` (gh/373) 103
2020-08-19 ltexpl.dtx v1.2e
General: Add `\@expl@cs@`⟨*thing*⟩`@spec@@N` for `\ShowCommand` (gh/373) 76
Add `\@expl@cs@to@str@@N` and `\@expl@str@if@eq@@nnTF` for `\NewCommandCopy` (gh/239) 76
2020-08-20 ltplain.dtx v2.3d
`\alloc@`: Define `\alloc@` in terms of `\e@alloc` 20
2020-08-21 ltclass.dtx v1.3o
General: Integration of new hook management interface 1094
2020-08-21 ltdefns.dtx v1.5l
General: Integration of new hook management interface 80
2020-08-21 ltdefns.dtx v1.5m
`\MakeRobust`: Make `\MakeRobust` produce the same command structure as `\DeclareRobustCommand` 95
2020-08-21 ltexpl.dtx v1.2d
General: Dropped unused command . 72

2020-08-21 ltfiles.dtx v1.2i
General: Integration of new hook management interface 485
2020-08-21 ltfinal.dtx v2.2i
General: Integration of new hook management interface 1332
2020-08-21 ltfssaxes.dtx v1.0g
General: Integration of new hook management interface 669
2020-08-21 ltfssini.dtx v3.2b
`\bf@def@ult`: Integration of new hook management interface 749
`mdseries/defaults`: Integration of new hook management interface 751
`\maybe@update@bfseries@defaults`: Integration of new hook management interface 746
`\maybe@update@mdseries@defaults`: Integration of new hook management interface 747
`\reset@font`: Integration of new hook management interface 761
`\rmfamily`: Integration of new hook management interface 751
`\ttfamily`: Integration of new hook management interface 751
2020-08-21 ltmiscen.dtx v1.1v
General: Integration of new hook management interface 864
2020-08-21 ltoutput.dtx v1.4f
`\@begindvibox`: Integration of new hook management interface . . . 1225
2020-08-23 ltxref.dtx v1.1o
`\refstepcounter`: add default definition of `\@currentcounter` . 845
2020-09-06 ltclass.dtx v1.3q
`\load@onefilewithoptions`: Save `\@currpkg@reqd` so that we don't lose track of package substitutions. 1124
2020-09-06 ltdefns.dtx v1.5n
`\char@if@alph`: Macro added 111
2020-09-06 ltexpl.dtx v1.2f
General: Add `\@expl@str@map@function@@NN` and `\@expl@char@generate@@nn` for `\string@makeletter` (gh/386) 76
2020-09-09 ltshipout.dtx v1.0b
`\__shipout_picture_overlay:n`: Prevent overfull box warnings (gh/387) 1192
2020-09-26 ltfinal.dtx v2.2j
General: Load first aid file if existing 1353
2020-09-30 ltfssini.dtx v3.2d
`\maybe@update@bfseries@defaults`: `\bfdefault@previous` not `\bfseries@previous` (gh/395) . . 746
`\mdseries`: `\mddefault@previous` not `\mdseries@previous` (gh/395) . . 746
2020-10-01 ltclass.dtx v1.3r
`\@pr@videpackage`: Allow for package substitution 1109
2020-10-01 ltsect.dtx v1.1e
`\addcontentsline`: add a fourth argument for better hyperref compatibility 1021
2020-10-04 ltfiles.dtx v1.2j
`\@include`: Quotes around the aux file name removed, they are not needed and upset BibTeX (gh/400) 495
2020-10-04 lthooks.dtx v1.0d
General: Definition `\AddToHookNext` was supposed to be for `\AddToHook` vice versa (gh/401) 325
2020-10-08 ltclass.dtx v1.3s
`\@currnamestack`: Added missing 2020/02/02 `\IncludeInRelease` 1100
2020-10-11 ltclass.dtx v1.3t
`\load@onefilewithoptions`: Restore `\@currpkg@reqd` after finished loading a package file (gh/408). 1124
2020-10-18 ltclass.dtx v1.3t
`\PassOptionsToClass`: Drop path from `\input@path` (gh/414). . . 1111
2020-10-23 ltmiscen.dtx v1.1w
`\enddocument`: Make `enddocument/afteraux` one-time 866
2020-10-26 ltmiscen.dtx v1.1x
`\@kernel@before@enddocument`: `\enddocument` should always start out in vmode (gh/385) 869
2020-11-09 ltclass.dtx v1.3u
`\pkgcls@rollbackdate@error`: Change help text because the package may have existed then — there is just no rollback data (gh/423). 1146
2020-11-09 ltmath.dtx v1.2h
`\negthickspace`: `\negmedspace` and `\negthickspace` have been only in amsmath, so we need to undefine for rollback (gh/423) 896
2020-11-20 ltclass.dtx v1.3u
`\@currpath`: Macro added 1099
`\@kernel@currpathstack`: Macro added 1103

`\load@onefile@withoptions`: Copy option list to the requested package. 1125
`\PassOptionsToClass`: Copy option list to the requested package. . . 1111
`\ProvidesPackage`: Use string comparison instead of `\ifx` . . . 1108
2020-11-20 ltcmd.dtx v1.0a
General: Initial version derived from `xparse.dtx` 115
2020-11-20 ltfilehook.dtx v1.0d
`\unqu@tefilef@und`: Move loading to `\@input@file@exists@with@hooks` and expand `\@filef@und` to avoid getting the wrong file name in the case of a substitution. 1169
2020-11-23 ltshipout.dtx v1.0d
`\__shipout_execute_cont:`: Check for both kernel and user hook (gh/431) 1195
`\__shipout_execute_main_cont:Nnnn`: Check for both kernel and user hook (gh/431) 1198
2020-11-24 ltexpl.dtx v1.2g
General: Support for roll forward (gh/434) 74, 76
2020-11-24 ltfilehook.dtx v1.0d
General: Support for roll forward (gh/434) 1167
2020-11-24 lthooks.dtx v1.0f
`\__hook_end_document_label_check:`: Support for roll forward (gh/434) 260
2020-11-24 ltshipout.dtx v1.0d
General: Support for roll forward (gh/434) 1210
`\AtBeginDvi`: Support for roll forward (gh/434) 1209
2020-11-25 ltdefns.dtx v1.5o
`\@carcube`: Added missing latexrelease entry 83
2020-12-02 ltluatex.dtx v1.1s
General: Fix return value of list callbacks 64
2020-12-03 ltfsstrc.dtx v3.0m
`\selectfont`: Install a hook in `\selectfont` (gh/444) 676
2020-12-04 ltfilehook.dtx v1.0d
`\undeclare@file@substitution`: Don't drop file substitution commands on rollback 1172
2020-12-04 ltfssaxes.dtx v1.0h
General: Reorganized the rollback data 601
`\fontseries`: Distangle series and shape update (gh/444) 656
`\fontshape`: Distangle series and shape update (gh/444) 667
`\fontshapeforce`: Distangle series and shape update (gh/444) 667
2020-12-04 ltfssini.dtx v3.2f
General: Adjust start values for series and shape (gh/444) 764
2020-12-10 ltbibl.dtx v1.1s
`\nocite`: Delay any `\nocite` in the preamble instead of raising an error 1056
2020-12-10 ltfssbas.dtx v3.2h
`\usefont`: Drop "m" if the series value is a member of a fixed list and issue warning if doing it (gh/453) 582
2020-12-14 ltclass.dtx v1.3v
`\@currnamestack`: Removed `\@expl@@@hook@curr@name@push@@n` 1100
2020-12-18 ltexpl.dtx v1.2h
`\@kernel@after@enddocument@afterlastpage`: Define kernel `\enddocument` hooks early 72
2020-12-22 ltfssaxes.dtx v1.0h
`\delayed@merge@font@series`: Distangle series and shape update (gh/444) 658, 659
`\delayed@merge@font@shape`: Distangle series and shape update (gh/444) 668
2020-12-22 ltfsstrc.dtx v3.0n
`\selectfont`: Execute delayed series and shape updates (gh/444) . . . 675
2021-01-07 ltfilehook.dtx v1.0e
General: Added rollback for this case to avoid spurious errors (part of gh/463) 1181
`\unqu@tefilef@und`: Restore `\CurrentFile(Path)(Used)` after the input (gh/464) 1170
2021-01-07 lthooks.dtx v1.0h
`\__hook_strip_double_slash:w`: Assume hook name has at least three nonempty parts (gh/464) . 272
`\__hook_tl_set:cn`: Manually define some l3tl commands to work around expl3 changes 247
2021-01-08 ltshipout.dtx v1.0f
`\__shipout_execute_cont:`: Added another kernel hook for more flexibility (cf.

`https://github.com/pgf-tikz/pgf/issues/1002` 1195
2021-01-10 ltshipout.dtx v1.0g
`\@kernel@after@shipout@background`: Internal hook `\@kernel@after@shipout@background` added 1199
`\RawShipout`: Macro added 1199
2021-01-19 ltshipout.dtx v1.0h
`\__shipout_run_firstpage_hook:`: Handling of firstpage hook altered 1200
2021-01-21 ltclass.dtx v1.3w
`\@kernel@currpathstack`: Add empty entry for latexrelease 1103
2021-01-21 ltexpl.dtx v1.3a
General: Move xparse rollback code to `ltcmd.dtx` 75
2021-01-21 ltfinal.dtx v2.2l
General: Load glyphtounicode.tex for pdfTeX 1340
2021-01-22 ltshipout.dtx v1.0i
`\__shipout_finalize_box:`: Add `pre_shipout_filter` Lua callback 1194
2021-01-24 ltexpl.dtx v1.3a
General: Define expl3 hooks conditionally 72
2021-01-31 ltfilehook.dtx v1.0f
`\@curr@file@reqd`: set `\protect` to `\string` gh/481 1173
2021-02-03 ltfloat.dtx v1.2e
`\@savemarbox`: Explicitly end with `\par` (gh/489) 1039
2021-02-04 ltboxes.dtx v1.4b
`\color@endbox`: Always add the color groups (gh/488) 929
2021-02-08 ltfilehook.dtx v1.0g
`\unqu@tefilef@und`: Undo the internal for robust `\InputIfFileExists` in rollback (gh/494) 1171
2021-02-08 ltmiscen.dtx v1.1y
`\begin`: Undo the internals for robust `\begin` and `\end` in rollback (gh/494) 874
`\end\verbvisiblespace`: Undo the internals for robust `\begin` and `\end` in rollback (gh/494) 877
2021-02-10 ltboxes.dtx v1.4b
`\@mpfootnotetext`: Explicitly run `\par` in support for paragraph tagging 941
2021-02-10 ltfloat.dtx v1.2e
`\@footnotetext`: Explicitly run `\par` at the end of footnote text in preparation for paragraph hooks 1047
2021-02-15 ltfssdcl.dtx v3.0w
`\document@select@group`: fix for (gh/501) 708
2021-02-16 ltfloat.dtx v1.2f
`\footref`: `\footref` added 1049
2021-02-17 ltoutenc.dtx v2.0t
General: Adjust values for `\textasteriskcentered` To match TS1 definition (gh/502) 547
Special definition for `\textasteriskcentered` when missing in TS1 (gh/502) 538
2021-02-18 ltclass.dtx v1.3x
`\@fileswithoptions`: save raw class option list (gh/85) 1120
`\@remove@eq@value`: macro added (gh/85) 1112
`\@use@ption`: value from unused option list (gh/85) 1115
`\OptionNotUsed`: value from unused option list (gh/85) 1112
`\PassOptionsToClass`: save raw option lists (gh/85) 1111
2021-02-19 ltoutenc.dtx v2.0u
General: Add `\textnonbreakinghyphen`, `\textfiguredash` and `\texthorizontalbar` (gh/404) 526, 530, 546
2021-02-25 ltfinal.dtx v2.2m
General: Improve speed of ToUnicode everyjob loading code 1340
2021-03-03 ltclass.dtx v1.3y
`\endfilecontents`: Fix overwrite check for files with UTF-8 (gh/415) 1133
2021-03-05 ltclass.dtx v1.3z
`\ProcessOptions*`: modify so braces to not give errors (gh/513) . . . 1114
2021-03-12 ltfiles.dtx v1.2k
`\IfFileExists@`: Allow unbalanced conditionals (gh/530) 500
2021-03-18 ltcmd.dtx v1.0b
General: Use `\NewModuleRelease`. . 129
2021-03-18 ltfilehook.dtx v1.0h
`\__filehook_file_pop_assign:nnnn`: Define `\g__filehook_input_file_seq` to avoid losing data when rolling back. 1167

2021-03-18 ltfssaxes.dtx v1.0i
General: Fix rollforward definition. . 657
2021-03-18 ltfssini.dtx v3.2g
General: Add legacy hook definitions for rollback. 752
2021-03-18 lthooks.dtx v1.0i
`\__hook_end_document_label_check:`: Only add `top-level` if not already there. 260
Remove the (empty) "top-level" from `\@currnamestack`. 260
General: Use `\NewModuleRelease`. . 245
2021-03-18 ltvers.dtx v1.1f
`\@check@IncludeInRelease`: Add support for usage in `\NewModuleRelease` 37
`\new@moduledate`: Added `\NewModuleRelease`. 38
2021-03-19 lttextcomp.dtx v1.0e
General: Use `\NewModuleRelease` . . 799
2021-03-26 ltplain.dtx v2.3e
`\@unused`: Allocate `\@inputcheck` and `\@unused` early so that they are before expl3 allocates more streams (gh/538) 21
2021-03-27 ltclass.dtx v1.4a
`\@currnamestack`: Do not completely roll back if expl3 is loaded. 1101, 1102
2021-04-16 ltvers.dtx v1.1g
`\new@moduledate`: `\NewModuleRelease` with the same arguments as `\IncludeInRelease`. 38
2021-04-17 ltfiles.dtx v1.2m
`\@kernel@after@begindocument`: Move `\@kernel@before@begindocument` and `\@kernel@after@begindocument` init earlier so that other modules can write to the hooks 490
2021-04-18 ltluatex.dtx v1.1t
General: input_level_string added . . 62
2021-04-18 ltplain.dtx v2.3f
`\loggingall`: 3 31
Drop pre-ε-TEX support 31
`\tracingnone`: 3 32
Drop pre-ε-TEX support 32
2021-04-19 ltcmd.dtx v1.0d
`\__cmd_cmd_type_cases:NnnnnnF`: Renamed `\__cmd_cmd_if_xparse:NTF` to `\__kernel_cmd_if_xparse:NTF` for cross-module usage 203
2021-04-20 ltexpl.dtx v1.3c
`\@kernel@after@enddocument@afterlastpage`: Don't empty kernel hooks on rollback 72
2021-04-20 ltfilehook.dtx v1.0i
`\@curr@file@reqd`: Make expand to a string (tracks change in l3kernel) 1173
2021-04-26 ltfssbas.dtx v3.2i
`\usefont`: Unconditionally switch to the requested font face (gh/444) 582
2021-04-26 ltfssini.dtx v3.2h
`\reset@font`: Unconditionally switch to the requested font face (gh/444) 761
2021-04-26 ltfsstrc.dtx v3.0o
`\selectfont`: Unset the forced series boolean when reaching `\selectfont` (gh/444) 676
2021-04-29 lthooks.dtx v1.0m
`\ActivateGenericHook`: Add `\ProvideHook` etc. 320
2021-04-29 ltoutenc.dtx v2.0v
General: Add composites for `\ae/\AE/æ/Æ` (gh/552) 551
2021-05-18 ltclass.dtx v1.4b
`\@raw@classoptionslist`: Initialize to `\relax` to match `\@classoptionslist` 1099
2021-05-24 ltcmd.dtx v1.0e
General: Use `\msg_...` instead of `\__kernel_msg...` 129
2021-05-24 ltcmdhooks.dtx v1.0b
General: Use `\msg_...` instead of `\__kernel_msg...` 332
2021-05-24 ltfilehook.dtx v1.0k
General: Use `\msg_...` instead of `\__kernel_msg...` 1165
2021-05-24 lthooks.dtx v1.0n
General: Use `\msg_...` instead of `\__kernel_msg...` 245
2021-05-24 ltpara.dtx v1.0g
General: Use `\msg_...` instead of `\__kernel_msg...` 448
2021-05-26 ltdefns.dtx v1.5p
`\MakeRobust`: Normalize error message in `\MakeRobust` 95
2021-06-03 ltclass.dtx v1.4c
`\@kernel@currpathstack`: Take care of `\@kernel@currpathstack` when rolling back/forward. 1103
2021-06-04 ltcmd.dtx v1.0f
General: Normalize various error messages 206

2021-06-05 ltmiscen.dtx v1.1z
\@@sverb: Normalize error message . 885
2021-06-06 ltclass.dtx v1.4c
\@loadwithoptions: handle raw options for gh/580 1117
\load@onefile@withoptions: Copy raw options for gh/580 1125
\PassOptionsToClass: apply \expandafter to raw options for gh/580 1111
2021-06-09 ltclass.dtx v1.4b
\endfilecontents: Use \@latex@note@no@line to display the information 1132, 1133
2021-06-09 lterror.dtx v1.2r
\@latex@note@no@line: Macros added 432
2021-06-09 ltfssbas.dtx v3.2j
\DeclareFontShape@: Improve information message 572
2021-07-08 ltcounts.dtx v1.1m
\counterwithin: New implementation for \counterwithin 563
\counterwithout: New implementation for \counterwithout 564
2021-07-11 lterror.dtx v1.2s
\PackageNoteNoLine: Provide \ClassNote and \PackageNote . 431
2021-07-12 ltclass.dtx v1.4d
\@fileswithoptions: add \unexpanded 1121
2021-07-16 ltplain.dtx v2.3g
General: Use 2 as default value for \tracinglostchars 22
2021-07-19 ltclass.dtx v1.4e
\@classoptionslist: Drop \@onlypreamble 1099
\@ifclasslater: Drop \@onlypreamble 1104
\@ifclassloaded: Drop \@onlypreamble 1104
\@ifclasswith: Drop \@onlypreamble 1106
\@ifl@ter: Drop \@onlypreamble 1105
\@pkgextension: Drop \@onlypreamble 1100
\@ptionlist: Drop \@onlypreamble 1104
\@unusedoptionlist: Drop \@onlypreamble 1099
\IfFormatAtLeastTF: Drop \@onlypreamble 1104
2021-07-20 ltcmdhooks.dtx v1.0c
__hook_patch_DeclareRobustCommand:Nnn: Use \robust@command@chk@safe before \@if@newcommand. 337
2021-07-22 lthooks.dtx v1.0o
__hook_gremove_code:nn: Do not queue removals (gh/625) 274
__hook_hash_check_aux:w: Do not queue removals (gh/625) 262
2021-07-23 ltclass.dtx v1.4e
\load@onefile@withoptions: Make class/name/after a one-time hook 1126
Make class/name/before a one-time hook 1126
Make package/name/after a one-time hook 1126
Make package/name/before a one-time hook 1126
2021-07-23 ltfiles.dtx v1.2n
\@include: Make include/name/after a one-time hook 496
Make include/name/before a one-time hook 496
Make include/name/end a one-time hook 496
2021-07-27 lthooks.dtx v1.0o
\ClearHookNext: Macro added 321
\hook_gclear_next_code:n: Macro made public 307
2021-07-28 ltsect.dtx v1.1f
\contentsline: Pick up four arguments (gh/633) 1023
2021-07-30 ltcmd.dtx v1.0d
__cmd_cmd_type_cases:NnnnnnF: Added __cmd_cmd_type_cases:NnnnnF for \NewCommandCopy and \ShowCommand support 203
2021-07-31 ltoutput.dtx v1.4e
\fl@tracemessage: Enable display when doing \tracefloatvals . 1292
2021-07-31 ltoutput.dtx v1.4g
\ShowFloat: Macro added 1290
2021-08-02 lthooks.dtx v1.0o
\ActivateGenericHook: Change name 320
\DisableGenericHook: Change name 320
2021-08-07 ltcmd.dtx v1.0g
__cmd_add_grabber:N: Replicate argument processors for all embellishments (gh/639) 157

`\__cmd_add_type_E:w`: Replicate argument processors for all embellishments (gh/639) 155
2021-08-08 ltfinal.dtx v2.2p
`\IfPDFManagementActiveTF`: Default definition added (gh/640) 1353
2021-08-10 ltvers.dtx v1.1h
`\@check@IncludeInRelease`: Add error to aid debugging 37
2021-08-11 ltluatex.dtx v1.1u
General: Define missing local function 52
2021-08-20 lterror.dtx v1.2t
`\@badend`: Improve `\@badend` error message (gh/587) 435
2021-08-20 lthooks.dtx v1.0p
General: Added deprecation warnings for old generic hook commands (gh/638) 323
Documentation updates for generic hook commands (gh/638) 218
Renames of generic hook commands (gh/638) 256
Section on generic hooks added (gh/638) 235
2021-08-25 ltclass.dtx v1.4f
General: Standardize generic hook names (gh/648) 1094
`\load@onefile@withoptions`: Declare non-generic package and class hooks 1127
2021-08-25 ltcmdhooks.dtx v1.0d
`\__hook_try_put_cmd_hook:w`: Simplify generic hook detection . 334
2021-08-25 ltfilehook.dtx v1.0l
`\unqu@tefilef@und`: Declare non-generic file hooks 1170
2021-08-25 ltfiles.dtx v1.2o
`\@include`: Declare non-generic include hooks 496
Standardize generic hook names (gh/648) 496
2021-08-25 lthooks.dtx v1.0p
`\__hook_try_declaring_generic_next_hook:nn`: Standardize generic hook names (gh/648) 267
2021-08-27 ltcmd.dtx v1.0h
`\NewDocumentEnvironment`: Check for end-of-environment command . . 212
2021-08-27 ltfilehook.dtx v1.0l
`\__filehook_file_pop_assign:nnnn`: Internal message name changes 1167
`\__filehook_file_subst_cycle_error:cN`: Use `\msg_...` not `\__kernel_msg_...` 1178
2021-08-27 lthooks.dtx v1.0q
General: Internal message name changes 317
2021-08-27 ltpara.dtx v1.0i
General: Internal message name changes 456
2021-08-30 ltcmd.dtx v1.0h
General: Added support for `\NewCommandCopy` 162
Added support for `\ShowCommand` 168
2021-09-03 ltoutput.dtx v1.4h
`\ShowFloat`: Renamed, original name never distributed 1290
2021-09-06 ltfinal.dtx v2.2q
`\@uclclist`: Correctly upper and lowercase `\ij` and `\IJ` (gh/658) 1350
2021-09-06 lthooks.dtx v1.0r
`\__hook_hash_check_aux:w`: Use dedicated conditional (gh/606) . 262
`\__hook_if_execute_immediately:n`: Macro added (gh/606) 312
`\__hook_use_once_clear:n`: Clean up after `\UseOneTimeHook` (gh/606) 312
`\hook_use_once:nnw`: Clean up after `\UseOneTimeHook` (gh/606) 311
2021-09-10 ltfssini.dtx v3.2i
`\bfseries`: Do delayed changes to `\bfdefault` in a separate macro for better reuse (gh/663) 746
`\DeclareFontSeriesDefault`: Do delayed changes to `\bfdefault` or `\mddefault` first (gh/663) 737
`\maybe@update@bfseries@defaults`: Do delayed changes to `\bfdefault` in a separate macro for better reuse (gh/663) 746
`\maybe@update@mdseries@defaults`: Do delayed changes to `\mddefault` in a separate macro for better reuse (gh/663) 747
`\mdseries`: Do delayed changes to `\mddefault` in a separate macro for better reuse (gh/663) 746
2021-09-12 ltfntcmd.dtx v3.5a
`\check@nocorr@`: use `\unexpanded` to make # safe 794
2021-09-12 ltoutenc.dtx v2.0w
General: Move zero skip between i and j for hyphenation (gh/658) 527
2021-09-18 ltpara.dtx v1.0i
`\para_end:`: Use skip rather than kern as guard. 454

2021-09-26 ltfssdcl.dtx v3.0x
`\c@localmathalphabets`: Counter added for (gh/676) 707
`\document@select@group`: Test if we should freeze the version (gh/676) 707
`\freeze@math@version`: Macro added for (gh/676) 710
2021-09-28 ltcmdhooks.dtx v1.0e
`\__hook_make_prefixes:w`: Make patching of commands a global operation (gh/674) 342
`\__hook_patch_retokenize:Nnnn`: Make patching of commands a global operation (gh/674) 351
2021-09-28 lthooks.dtx v1.0s
`\__hook_if_usable_use:n`: Correct usage of older `\__hook_if_file_hook:wTF` (gh/675) 310
`\__hook_try_declaring_generic_hook_split:nNNnn`: Correct usage of older `\__hook_if_file_hook:wTF` (gh/675) 268
2021-10-14 ltboxes.dtx v1.4c
`\@mpfootnotetext`: Explicitly set `\@currentcounter` (gh/687) . . . 941
2021-10-14 ltfiles.dtx v1.2p
`\@include`: Warn about use in preamble 495
2021-10-14 ltfloat.dtx v1.2g
`\@footnotetext`: Explicitly set `\@currentcounter` (gh/687) . . 1047
2021-10-14 ltmath.dtx v1.2j
`\@eqnsel`: Explicitly set `\@currentcounter` (gh/687) . . . 900
`eqnarray`: Explicitly set `\@currentcounter` (gh/687) . . . 906
2021-10-15 ltfssdcl.dtx v3.0y
`\DeclareMathVersion`: Initialize variable for freezing math version (gh/676) 715
2021-10-15 ltluatex.dtx v1.1v
General: provide_charproc_data added 62
2021-10-16 ltoutenc.dtx v2.0x
`\add@accent`: Dont set `\spacefactor in math mode gh/643` 516
2021-10-19 ltpara.dtx v1.0k
General: Remove content from `\tex_everypar:D` on rollback . . . 456
2021-10-20 ltcmdhooks.dtx v1.0f
`\__hook_make_prefixes:w`: Correct patching by expansion+redefinition when the macro contains a parameter token (gh/697) 338
2021-11-17 ltluatex.dtx v1.1w
General: hpack_quality is `exclusive` 61
Never pass on `true` return values for list callbacks 64
vpack_quality is `exclusive` 61
2021-11-30 ltclass.dtx v1.5a
`\@onefilewithoptions@clashchk`: Separated out from `\@onefilewithoptions` 1125
2021-11-30 ltexpl.dtx v1.3d
`\skipeval`: Moved over from `xfp` package (gh/711) 77
2021-12-02 ltcmd.dtx v1.0i
`\__cmd_run_code:`: Correct defaults for optional arguments in end-of-environment code (gh/712) 140
`\__cmd_start_aux:ccnnnn`: Correct defaults for optional arguments in end-of-environment code (gh/712) 140
2021-12-07 ltexpl.dtx v1.3d
`\skipeval`: Added `\dimeval` and `\skipeval` (gh/711) 77
2021-12-11 ltdirchk.dtx v1.3a
General: Add comment lines into latex.ltx to indicate temp definitions that are later overwritten (gh/725) 2
2021-12-12 ltoutenc.dtx v2.0y
General: `\DeclareUnicodeAccent` now makes the encoding argument implicit (gh/253) 543
Added `\DeclareUnicodeCommand` and `\DeclareUnicodeSymbol` (gh/253) 544
2021-12-27 ltluatex.dtx v1.1x
`\newprotectedluacmd`: Macros added 49
2021-12-28 ltexpl.dtx v1.3d
`\ExpandArgs`: Added document level names for `\exp_args:Nc` and the like (gh/735) 77
2021-13-31 ltcmd.dtx v1.0j
`\__cmd_show:e`: Make `\ShowCommand` stop for interaction 169, 170
2022-01-06 ltexpl.dtx v1.3e
`\ExpandArgs`: Adjust document level names for `\exp_args:Nc` and the like (gh/735) 77
2022-01-06 ltshipout.dtx v1.0l
`\AtBeginShipoutNext`: Correctly simulate `\AtBeginShipout` and

`\AtBeginShipoutNext` without extending the syntax 1212
`\AtEndDvi`: Correctly simulate `\AtEndDvi` without extending the syntax 1211
2022-01-15 ltkeys.dtx v1.0b
`\__keys_options_aux:n`: Clear option list in end-of-package hook . . . 1153
2022-01-25 ltplain.dtx v2.3h
`\obeyspaces`: Provide redirection to support special use cases (gh/367) 25
2022-02-05 ltkeys.dtx v1.0b
`.usage`: Create properties in `ltkeys` 1151
2022-02-07 ltkeys.dtx v1.0c
`.usage`: Correct `..code` property . . 1151
2022-02-15 ltkeys.dtx v1.0c
`\DeclareKeys`: Expand module argument 1157
`\DeclareUnknownKeyHandler`: Added `\DeclareUnknownKeysHandler` . 1157
`\ProcessKeyOptions`: Expand module argument 1157
`\SetKeys`: Expand module argument 1158
2022-02-16 ltkeys.dtx v1.0d
`\DeclareKeys`: Allow for active characters in module argument 1157
`\DeclareUnknownKeyHandler`: Allow for active characters in module argument 1157
Rename `\DeclareUnknownKeysHandler` to `\DeclareUnknownKeyHandler` . . 1157
`\ProcessKeyOptions`: Allow for active characters in module argument 1157
`\SetKeys`: Allow for active characters in module argument 1158
2022-02-19 ltcmd.dtx v1.0k
`\IfBlankTF`: Added `\IfBlankTF` and friends 216
2022-02-20 ltfinal.dtx v2.2r
`\@uclclist`: Use `\@expl@text@uppercase@@n`, removing local redefinition of `\UTF@two@octets@noexpand` . . . 1347
use `\text_lowercase:n` 1347
2022-02-21 ltkeys.dtx v1.0e
`\__keys_options_expand_module:Nn`: Faster approach to module expansion 1156
2022-02-28 ltcmd.dtx v1.0k
General: Move latexrelease redefinitions from ltcmd.dtx . . . 216
2022-02-28 ltexpl.dtx v1.3f
General: Move latexrelease redefinitions from ltcmd.dtx 75
2022-02-28 ltvers.dtx v1.1i
`\new@moduledate`: Detect a missing `\IncludeInRelease{0000/00/00}`. 38
2022-03-10 ltbibl.dtx v1.1t
`\cite`: Ensure that an empty argument generates a warning (gh/790) 1054
2022-03-10 ltfilehook.dtx v1.0m
`\@curr@file@reqd`: Add `\set@curr@file@nosearch` for graphicx 1173
2022-03-18 ltclass.dtx v1.5b
`\load@onefilewithoptions`: Switch to `\ProcessKeyOptions` 1123
2022-03-18 ltcmd.dtx v1.0l
`\__cmd_cmd_type_cases:NnnnnnF`: Fix `\__cmd_cmd_type_cases:NnnnnF` prematurely expanding macros (gh/795) 203
2022-03-18 ltkeys.dtx v1.0f
`\__keys_options:n`: Simplify to always cover global options . . . 1153
`\__keys_options_global:n`: Simplify to always cover global options . 1153
`\ProcessKeyOptions`: Remove `\ProcessKeyPackageOptions` . . 1157
2022-04-01 ltfiles.dtx v1.2q
`\@include`: Process some hooks is an include file is bypassed 496
2022-04-01 lthooks.dtx v1.0t
`\c__hook_generic_include/./end_tl`: Support generic `include/.../excluded` hooks . . 273
2022-04-03 ltfinal.dtx v2.2s
General: Integration of new mark management interface 1332
2022-04-03 ltoutput.dtx v1.ih
`\@opcol`: Interface with new mark mechanism 1241
2022-04-04 ltpage.dtx v1.0n
`\markright`: Interface with new mark mechanism 1090
2022-04-08 ltmath.dtx v1.2k
`\openup`: Make `\protected` (gh/123) 894
2022-04-12 ltxref.dtx LaTeX2e
`\pageref`: Macro reimplemented with a starred version 842
`\ref`: Macro reimplemented with a starred version 842

2022-04-12 ltxref.dtx v1.1p
General: Add starred variants for the ref commands 840
`\Ref`: Macro reimplemented with a starred version 847
2022-04-21 ltfinal.dtx v2.2t
`\@uclclist`: Support `\noexpand` in argument of `\@expl@text@uppercase@@n` . . . 1347
2022-05-06 ltmarks.dtx v1.0c
`\__mark_new_class:nn`: Wrong command made `\@onlypreamble` 1068
2022-05-08 ltmath.dtx v1.2l
General: Use consistent math styles under LuaTeX 903
2022-05-08 ltshipout.dtx v1.0m
`\@kernel@after@enddocument@afterlastpage`: Handle case where shipout/lastpage is run too early (gh/813) 1207
`\__shipout_execute_main_cont:Nnnn`: Handle case where shipout/lastpage is run too early (gh/813) 1198
2022-05-13 lthooks.dtx v1.0u
`\__hook_use_once_clear:n`: Check if prop exists to avoid l3debug error 312
2022-05-17 lthooks.dtx v1.0u
`\__hook_initialize_hook_code:n`: Refuse sorting one-time hooks (gh/818). 289
2022-05-17 ltmeta.dtx v1.0b
General: Default definition for targets added 460
2022-05-27 ltfiles.dtx v1.2r
`\listfiles`: Try saved version string, if ver@.. is `\relax` (gh/825) . . . 507
2022-05-27 ltoutenc.dtx v2.0z
General: Save the version string (gh/825) 555
2022-06-01 ltmarks.dtx v1.0d
`\__mark_prepare_and_extract:nn`: Extend the logic for detecting the marks in the box (gh/836) 1071
General: Marks are kernel errors . . 1081
2022-06-02 ltfinal.dtx v2.2u
`\@uclclist`: Add `\NoCaseChange` . 1349
2022-06-15 lthooks.dtx v1.0v
`\__hook_activate_generic:n`: Ensure that a newly activated generic hook gets its execution code set . 257
2022-06-16 ltkeys.dtx v1.0g
`\__keys_options_class:n`: Better handling of option removal . . . 1154
`\__keys_options_package:n`: Better handling of option removal . . . 1155
2022-06-19 ltkeys.dtx v1.0h
`\__keys_options_class:n`: Further work on handling of option removal 1154
`\__keys_options_package:n`: Further work on handling of option removal 1155
`\__keys_options_package:nnn`: New function 1155
2022-06-20 ltclass.dtx v1.5c
`\load@onefilewithoptions`: Pass raw options to `\ProcessKeyOptions` 1123
2022-06-20 ltkeys.dtx v1.0h
`\__keys_options_class:n`: Use raw options data 1154
`\__keys_options_class:nnn`: New function 1154
`\__keys_options_global:n`: Use raw options data 1153
`\__keys_options_local:`: Use raw options data 1155
`\__keys_options_package:n`: Use raw options data 1155
2022-06-22 ltkeys.dtx v1.0i
`.usage`: Add ..notif property 1151
2022-06-30 ltfinal.dtx v2.2u
`\@uclclist`: Add `\AddToNoCaseChangeList` 1349
2022-06-30 ltfinal.dtx v2.2v
`\@uclclist`: Just use `\text_lowercase:n` without `\protectd@edf` gh/881x 1347
2022-07-04 ltfinal.dtx v2.2w
`\@uclclist`: Introduced `\CaseSwitch`, `\DeclareCaseChangeEquivalent` and `\MakeTitlecase` to support hooking into case changing gh/889 1347
2022-07-04 ltfssbas.dtx v3.2k
`\frozen@everydisplay`: Ignore spaces if necessary (gh/886) 586
`\frozen@everymath`: Ignore spaces if necessary (gh/886) 586
2022-07-04 ltfssdcl.dtx v3.0z
`\freeze@math@version`: Ignore spaces if necessary (gh/886) 711
2022-07-05 ltkeys.dtx v1.0i
`\__keys_options_aux:n`: Support `\CurrentOption` 1153
`\__keys_options_class:nnn`: Correct naming of raw class options storage 1154

2022-07-23 ltkeys.dtx v1.0j
__keys_options_aux:n: Output 'public' package name in messages 1153
__keys_options_loaded:nn: Output 'public' package name in messages 1158
2022-08-07 ltfssbas.dtx v1.0g
\DeclareEncodingSubset: Make global declaration (gh/905) 577
2022-08-10 ltcmd.dtx v1.1a
__cmd_arg_to_keyvalue:nn: New internal arg-to-keyval processor . 199
__cmd_grab_D_long_obey_spaces_no_strip:w: Add support for skipping brace stripping 178
__cmd_grab_R_aux:NNnN: Track changes in D-type implementation 185
__cmd_grab_b_end:Nw: Track changes in D-type implementation 174
General: New switch 132
2022-08-13 ltluatex.dtx v1.1y
General: shared_callbacks added . . . 62
add_to_callback: Adapted code for shared_callbacks 67
remove_from_callback: Adapted code for shared_callbacks 69
mlist_to_hlist: Use shared_callback system for pre/post/mlist_to_hlist 71
2022-08-18 ltfilehook.dtx v1.0n
\@disable@packageload@do: Inhibit checking the loaded version when package is load-disabled, and write to the .log (gh/888) 1179
2022-08-20 ltoutput.dtx v1.4j
\stockheight: Register added . . . 1226
\stockwidth: Register added 1226
2022-08-21 ltkeys.dtx v1.0k
__keys_options_loaded:nn: Correct error message 1158
2022-09-03 ltmath.dtx v1.2m
\smash: Guard against reboxing in fractions (gh/517) 893
2022-09-07 ltboxes.dtx v1.4d
\@iiiminipage: Check for nested minipages and warn (gh/168) . . 940
\if@in@minipage@env: Check for nested minipages and warn (gh/168) 939
2022-09-17 ltfssdcl.dtx v3.1a
\DeclareMathVersion: New logic for freezing math versions (gh/921) . 715
\freeze@math@version: New logic for freezing math versions (gh/921) . 710
2022-09-20 ltfssdcl.dtx v3.1b
\DeclareSymbolFont: Drop any surplus 'm' in series argument (gh/918) 716
\SetSymbolFont: Drop any surplus 'm' in series argument (gh/918) 718
2022-10-03 ltluatex.dtx v1.2a
General: Add rules for callback ordering 58
add_to_callback: Add rules for callback ordering 67
declare_callback_rule: Add function 68
remove_from_callback: Add rules for callback ordering 69
2022-10-10 ltclass.dtx v1.5d
\@@unprocessedoptions: Use \protected@edef. 1130
\load@onefilewithoptions: Use \protected@edef. 1123
\PassOptionsToClass: Use \protected@xdef. 1111
\ProcessOptions*: Use \protected@edef. 1113
2022-10-20 ltclass.dtx v1.5e
\load@onefilewithoptions: Define key option handler in ltkeys . . . 1123
2022-10-20 ltkeys.dtx v1.0l
__keys_options_aux:n: Define key option handler in ltkeys 1153
__keys_options_loaded:nn: Correct an argument for a message . . . 1157
2022-10-22 ltclass.dtx v1.5e
\@fileswithoptions: Use \protected@xdef. 1120
\@use@ption: Use \detokenize . . 1115
\ProcessOptions*: Use \detokenize 1114
2022-10-22 ltdefns.dtx v1.5r
\@expandtwoargs: Use \protected@edef. 91
2022-10-22 ltkeys.dtx v1.0l
__keys_options_class:nnn: Correct handling of unused option list . 1154
2022-10-26 ltfinal.dtx v2.2x
\@uclclist: Auto-detect babel locale 1347
Introduce optional argument for case-changing commands 1347
Make case changing commands language-aware 1347

2022-11-06 ltmiscen.dtx v1.2a
`\enddocument`: Repeat release info at the end (gh/944) 866
2022-11-06 ltvers.dtx v1.1j
General: Repeat release info at the end (gh/944) 36
2022-11-08 ltshipout.dtx v1.0n
`\__shipout_execute_main_cont:Nnnn`: Add shipout hook (gh/920) . . . 1198
`shipout/lastpage`: Add shipout hook (gh/920) 1199
2022-11-16 ltclass.dtx v1.5f
`\endfilecontents`: Do not show "current dir" in message (gh/917) 1133
Introduce key 'nowarn' on filecontents (gh/958) 1131–1133
2022-11-24 ltdefns.dtx v1.5s
`\DeclareEnvironmentCopy`: Add `\NewEnvironmentCopy`, `\RenewEnvironmentCopy`, and `\DeclareEnvironmentCopy` (gh/963) 102
`\ShowEnvironment`: Added `\ShowEnvironment` 107
2022-11-28 ltspace.dtx v1.3o
`\@hspace`: Support calc syntax correctly (gh/967) 481
`\@vspace@calcify`: Support calc syntax without a group (gh/967) 468
2022-11-29 ltcmd.dtx v1.1b
`\__cmd_show:e`: Add `\@showenvironmentlisthook` . . . 171
2022-11-30 ltcmd.dtx v1.1b
`\__cmd_show:e`: Don't stop for the `\begin` part of an environment 169, 170
2022-11-30 ltfinal.dtx v2.2y
`\@uclclist`: Set `\oe`/`\OE` equal to act as a marker for `babel` 1348
2023-01-05 ltfiles.dtx v1.2s
`\document`: `\do` now with default definition in the kernel (gh/975) 489
2023-01-19 ltluatex.dtx v1.2b
General: Remove unused local variable tex_setattribute 52
2023-01-30 ltpara.dtx v1.0l
`\g__para_standard_everypar_tl`: Backout `\parskip` at top of minipage (gh/989) 449
2023-03-12 ltcmd.dtx v1.1c
`\__cmd_copy_expandable:NnNNNNnnn`: Distinguish (non-expandable) document commands starting with `\__cmd_start_expandable:nNNNNn` 164
`\__cmd_show:e`: Distinguish (non-expandable) document commands starting with `\__cmd_start_expandable:nNNNNn` 168
2023-03-22 ltspace.dtx v1.3p
`\samepage`: Add `\predisplaypenalty` setting (gh/1022) 466
2023-03-28 ltclass.dtx v1.5g
`\IfFormatAtLeastTF`: Added `\IfFileAtLeastTF` (gh/1015) . . 1104
2023-03-28 ltfinal.dtx v2.2z
`\@uclclist`: Use groups for gh/1021 1348
2023-04-01 ltfssbas.dtx v3.2l
`\math@version`: Reset frozen mathversion gh/1028 584
2023-04-02 ltfilehook.dtx v1.0o
`\@set@curr@file@aux`: Make `\@set@curr@file@aux \long` gh/942 1175
2023-04-06 ltcmdhooks.dtx v1.0g
`\__hook_double_hashes_space:w`: Add case for `\c__hook_hashes_tl` (hook-args) 344
`\__hook_make_prefixes:w`: Rename to `\c__hook_hashes_tl` (hook-args) 340
`\c__hook_hashes_tl`: Rename to `\c__hook_hashes_tl` and add `\c__hook_hash_tl` (hook-args) . 333
2023-04-06 lthooks.dtx v1.1a
`\__hook_activate_generic:n`: Changes to add hook arguments (hook-args). 256
`\__hook_braced_real_loop:w`: Macro added (hook-args). 282
`\__hook_chk_args_allowed:nn`: Macro added (hook-args). 266
`\__hook_clear_next:n`: Changes to add hook arguments (hook-args). 307
`\__hook_code_gset_aux:nnn`: Macro added (hook-args). 277
`\__hook_code_gset_auxi:eeen`: Macro added (hook-args). 276
`\__hook_cs_end:w`: Macro added (hook-args). 281
`\__hook_cs_if_empty:c`: Macro added (hook-args). 281
`\__hook_gput_next_do:nn`: Changes to add hook arguments (hook-args). 306
`\__hook_gremove_code:nn`: Changes to add hook arguments (hook-args). 274

__hook_hash_check_aux:w: Add \hook_gput_code_with_args:nnn (hook-args). 262
Changes to add hook arguments (hook-args). 262, 263
Macro added (hook-args). 264
__hook_if_execute_immediately:n: Changes to add hook arguments (hook-args). 312
__hook_init_structure:n: Changes to add hook arguments (hook-args). 253
__hook_initialize_all:: Changes to add hook arguments (hook-args). 287
__hook_initialize_hook_code:n: Changes to add hook arguments (hook-args). 288
__hook_initialize_single:ccn: Changes to add hook arguments (hook-args). 291
__hook_log:nN: Changes to add hook arguments (hook-args) 299
__hook_log_next_code:n: Changes to add hook arguments (hook-args). 303
__hook_make_usable:nn: Changes to add hook arguments (hook-args). 251
__hook_new:nn: Add \hook_new_with_args:nn (hook-args). 250
Update hook code after declaring. 251
__hook_new_reversed:nn: Add \hook_new_reversed_with_-args:nn (hook-args). 253
__hook_normalize_cs_args:nn: Macro added (hook-args). 278
__hook_parameter:n: Macro added (hook-args). 283
__hook_post_initialization_defs:: Macro added (hook-args). 309
__hook_set_normalize_fn:nn: Macro added (hook-args). 279
__hook_tl_set:cn: Clean-up unused variants (hook-args). 247
__hook_try_declaring_generic_hook:wn: Changes to add hook arguments (hook-args). 268
__hook_use_i_delimit_by_s_mark:nw: Use standard naming scheme (hook-args). 247
__hook_use_initialized:nnw: Add \hook_use:nnw (hook-args). 309
__hook_use_once:nn: Changes to add hook arguments (hook-args). 311
__hook_use_once_clear:n: Changes to add hook arguments (hook-args). 312
General: Add dedicated rollback code to revert data structures (hook-args). 326
Messages 'too-many-args', 'without-args' and 'one-time-args' added (hook-args). 317
\AddToHookNextWithArguments: Add \AddToHookNextWithArguments (hook-args). 321
\AddToHookWithArguments: Add \AddToHookWithArguments (hook-args). 321
\c__hook_??_parameter_tl: Token list added (hook-args). 283
\c__hook_nine_parameters_tl: Add auxiliary token lists (hook-args). 247
\g__hook_replacing_stack_seq: Macro added (hook-args). 316
\hook_gput_next_code:nn: Add \hook_gput_next_code_with_-args:nn (hook-args). 305
\hook_if_empty:n: Changes to add hook arguments (hook-args). . . . 313
\hook_new_pair_with_args:nnn: Add \hook_new_pair_with_args:nnn (hook-args). 254
\hook_use_once:nnw: Add \hook_use_once:nnw (hook-args). 311
\NewMirroredHookPairWithArguments: Add \NewHookWithArguments (hook-args). 320
\UseOneTimeHookWithArguments: Add \UseHookWithArguments (hook-args). 322
2023-04-11 ltfinal.dtx v2.3a
\@uclclist: Use new generic mechanism to detect locale . . . 1347
2023-04-13 ltcmd.dtx v1.1d
__cmd_grab_v_group_end:: Set \tex_endlinechar:D earlier (gh/876). 186
2023-04-14 ltclass.dtx v1.5h
\load@onefilewithoptions: Define \load@onefilewithoptions when in latexrelease (gh/992) 1122
2023-04-15 ltplain.dtx v2.3i
\extrafloats: Protect box 255 in lualatex gh/1041 19

unwind numexpr and ifnum nesting 19
2023-04-16 ltfinal.dtx v2.3a
`\BCPdata`: Command added 1347
2023-04-16 lthooks.dtx v1.1b
`\__hook_include_legacy_code_chunk:n`: `\__hook_replacing_args_false:` in `\__hook_include_legacy_code_chunk:n` 255
2023-04-19 ltfinal.dtx v2.3b
`\@uclclist`: Add `\DeclareLowercaseMapping`, `\DeclareTitlecaseMapping` and `\DeclareUppercaseMapping` . . . 1349
2023-04-19 lthooks.dtx v1.1c
`\__hook_gput_next_do:nn`: Initialize hook structure when adding 'next' code (gh/1052). 306
`\hook_if_empty:n`: Simpler and faster version (gh/1052). 313
2023-05-12 ltxref.dtx v1.1q
`\label`: (UFi)added a hook with argument 844
(UFi)extended to store five arguments 844
2023-05-13 ltmath.dtx v1.2n
`\leqno`: add `\ignorespaces` gh/1059 903
2023-05-15 ltfiles.dtx v1.2t
`\IfFileExists@`: Use expl3 file existence test 500
`\IfFileExists@@`: Macro added . . . 500
2023-05-21 ltcmdhooks.dtx v1.0h
`\__hook_guess_arg_count:nw`: Macro added (cmd-args) 349
`\__hook_make_prefixes:w`: Changes to allow support arguments in cmd hooks (cmd-args) 341
`\__hook_patch_retokenize:Nnnn`: Changes to allow support arguments in cmd hooks (cmd-args) 349
2023-05-21 lthooks.dtx v1.1d
`\__hook_if_cmd_hook:w`: Changes to allow support arguments in cmd hooks (cmd-args). 315
`\__hook_make_usable:nn`: Changes to allow support arguments in cmd hooks (cmd-args). 252
`\__hook_new:nn`: Changes to allow support arguments in cmd hooks (cmd-args). 250
`\__hook_try_declaring_generic_hook:wn`: Changes to allow support arguments in cmd hooks (cmd-args). 269
`\__hook_try_declaring_generic_next_hook:nn`: Changes to allow support arguments in cmd hooks (cmd-args). 267
`\c__hook_parameter_cmd/./after_tl`: Token lists added (cmd-args). . . 274
2023-05-26 ltcmd.dtx v1.1e
`\__cmd_defaults_error:w`: Use simpler variant `\cs_generate_from_arg_count:NNno` 141
2023-05-30 ltfinal.dtx v2.3c
`\@uclclist`: Fix a typo in implementation of `\DeclareLowercaseMapping`, etc. 1349
2023-06-06 lthooks.dtx v1.1e
`\__hook_code_gset_auxi:eeen`: Short-circuit when the hook is declared without args (gh1078). . 276
2023-06-14 ltmiscen.dtx v1.2b
`\@vobeytabs`: Macro added 881
`verbatim*`: Support visible tabs in `\verb*` 883
2023-06-14 ltspace.dtx v1.3q
`\@xobeytab`: Macro added 481
2023-06-15 ltmiscen.dtx v1.2b
`\@@sverb`: Support visible tabs 885
`\@setupverbvisiblespace`: Support visible tabs 884
`\@vobeyspaces`: Support tabs 881
2023-06-15 ltmiscen.dtx v1.2q
`\@setupverbvisibletab`: Provide visible tab in `\verb*` 885
2023-06-16 ltfiles.dtx v1.2u
`\IfFileExists@@`: Support piped input 500
2023-07-02 ltluatex.dtx v1.2c
`new_luafunction`: Ensure existing table entries not overwritten gh/1100 57
2023-08-03 ltluatex.dtx v1.2c
`mlist_to_hlist`: Fix callback type of post_mlist_to_hlist_callback . . . 71
2023-08-19 ltcmd.dtx v1.2a
General: Removed commands that should have remained only in `xparse.dtx` 115
2023-09-01 ltmiscen.dtx v1.2c
`\@vobeytabs`: Provide global definition for active tab 881
2023-09-06 ltmiscen.dtx v1.2b
`\enddocument`: Test changes of values in `\new@label@record` 866

2023-09-20 ltproperties.dtx v1.0c
\RecordProperties: use \protected@edef for safer handling of active chars. 857
\SetProperty: use \protected@edef 856
2023-10-13 ltexpl.dtx v1.3g
\IfExplAtLeastTF: Provide a test for expl3 date (gh/1004) 78
2023-10-26 ltboxes.dtx v1.4e
\@iframebox: Guard against unknown alignment gh/1072 933
\@imakebox: Guard against unknown alignment gh/1072 928
\@parboxto: Guard against unknown alignment gh/1072 936
2023-10-26 ltspace.dtx v1.3r
\nobreakspace: Protected definition for tilde 480
2023-11-07 ltcounts.dtx v1.1n
\@definecounter: Do not change \the... if already defined (gh/823) 559
2023-11-07 ltoutenc.dtx v2.1a
General: Add more explanation to error message (gh/1102) 554
2023-11-07 ltplain.dtx v2.3j
\tracingnone: Set \tracinglostchars to 2 in \tracingnone (gh/549) 32
2023-11-15 ltfiles.dtx v1.2v
\if@listfiles@hashes: Extend file list information 506
\if@listfiles@sizes: Extend file list information 506
\listfiles: Extend file list information 507
2023-11-16 ltpara.dtx v1.0m
\g__para_standard_everypar_tl: Correct error message: hook left horizontal not vertical mode (gh/1182) 449
2023-12-01 ltcmd.dtx v1.2b
__cmd_cmd_type_cases:NnnnnnF: Extend for optimized commands 203
__cmd_start_optimized:: Optimize cmd creation for all-m arguments 135
2023-12-22 ltcmd.dtx v1.2c
General: Generalize message invalid-bang (gh/1198) 206
2024-01-03 lthooks.dtx v1.1g
__hook_log:nN: Fix expansion of __hook__print_args:nn argument (gh/1221) 299
2024-01-13 ltkeys.dtx v1.0m
__keys_options_class:n: Trim spaces off key names 1154
__keys_options_package:n: Trim spaces off key names 1155
2024-01-17 ltproperties.dtx v1.0d
__property_record_value_aux:e: Use \protected@write 856
2024-01-24 lthooks.dtx v1.1h
__hook_if_usable_use:n: Correct usage of older __hook_if_file_hook:wTF (gh/1243) 310
__hook_try_declaring_generic_hook_split:nNNnn: Correct usage of older __hook_if_file_hook:wTF (gh/1243) 268
2024-01-27 lttextcomp.dtx v1.1a
General: Added check file for encoding subset 830
Adjusted/corrected TS1 sub-encoding declarations for various families (gh/1257) 809
2024-01-29 ltmarks.dtx v1.0e
__mark_extract_and_handle_marks:nn: Macro added 1070
\mark_update_structure_from_material:nn: Macro renamed 1073
\ShowMarksAt: Macro added 1083
2024-01-30 ltclass.dtx v1.5i
\@fileswithoptions: Test group level 1119
2024-03-18 ltthm.dtx v1.0g
\@endtheorem: Insert link target in the label (UFi) 1010
\@thm: Use \@kernel@refstepcounter to avoid an unwanted target (UFi) 1010
2024-03-21 ltcmd.dtx v1.2d
__cmd_grab_v_aux_put:N: Collect \endlinechar as \obeyedline . . 189
2024-03-22 ltclass.dtx v1.5j
\load@onefilewithoptions: Apply one-step expansion to raw option list, to be consistent with change for gh/580 (gh/1298). 1123
2024-04-10 ltclass.dtx v1.5k
\IfClassLoadedWithOptionsTF: Provide T and F conditionals not just TF (gh/1262) 1107
\IfFileLoadedF: Provide \IfFileLoadedTF and variants (gh/1222) 1107

2024-04-17 ltcmd.dtx v1.2e
`\__cmd_cmd_type_cases:NnnnnnF`: Use `\__kernel_cs_parameter_spec:N` instead of `\cs_argument_spec:N`/`\cs_parameter_spec:N` . 203
2024-04-17 ltcmdhooks.dtx v1.0j
General: Use `\__kernel_cs_parameter_spec:N` instead of `\cs_argument_spec:N`/`\cs_parameter_spec:N` . 332
2024-04-17 ltdefns.dtx v1.5t
General: Rename `\@expl@cs@argument@spec@@N` to `\@expl@cs@parameter@spec@@N` (gh/1014) 80
2024-04-17 ltexpl.dtx v1.3h
General: Add a kernel-level copy of `\cs_parameter_spec:N` 76
Rename `\@expl@cs@argument@spec@@N` to `\@expl@cs@parameter@spec@@N` (gh/1014) 76
Update name of expl3 function . . . 76
2024-04-17 ltproperties.dtx v1.0e
`\IfLabelExistsF`: Renamed `\IfLabelExistTF` to `\IfLabelExistsTF` (gh/1262) . . . 860
`\IfPropertyExistsF`: Renamed `\IfPropertyExistTF` to `\IfPropertyExistsTF` (gh/1262) 860
2024-04-18 ltboxes.dtx v1.4f
`\@finalstrut`: Use a `\hrule` strut not a `\vrule` if already in vertical mode (bug seen first with footmisc/14) 945
2024-04-22 lttextcomp.dtx v2.1a
General: Drop default option `info` (gh/1333) 819
2024-04-24 lttextcomp.dtx v2.1b
General: Load the 2018 version when rolling back prior to 2018-08-11 (gh/1333) 818
2024-05-30 ltmarks.dtx v1.0f
`\__mark_update_dblcol_structures:`: Correct logic for first mark in page region if first column contains no marks (gh/1359) 1086
`\__mark_value:nn`: Use sequence marker to make all marks unique on nearby regions (gh/1359) . . 1079
`\mark_insert:nn`: Use sequence marker to make all marks unique on nearby regions (gh/1359) . . 1078
`insertmark`: Use sequence marker to make all marks unique on nearby regions (gh/1359) 1079
2024-05-31 ltmarks.dtx v1.0g
`\c__mark_empty_tl`: Initialize all marks with an id, use `0` when a new class is made (gh/1359) . . 1070
`\mark_use_last:nn`: Remove the id when returning the mark value (gh/1359) 1080
2024-06-04 ltclass.dtx v1.5l
`\@cls@pkg`: Allow for extensions other than "cls" and "pkg" 1129
`\@onefilewithoptions`: New argument `\pkgcls@ext` (gh/870) 1140
`\pkgcls@parse@date@arg`: New argument `\pkgcls@ext` (gh/870) 1143
2024-06-04 ltluatex.dtx v1.2d
`provides_module`: Add `v` to version string if required (gh/1364) 53
2024-06-10 ltboxes.dtx v1.4g
`\@finalstrut`: Always use a `\vrule` strut after all, but back up by a baseline if already in vertical mode. Otherwise empty table p-cells will not get the correct width (bug seen first with colortbl) 945
2024-06-10 lttagging.dtx v1.0b
`\tbl_crcr:n`: Always issue a `\crcr` even if we are at the start of a row to avoid problems with tabulary and similar code 1328
2024-06-17 ltfssbas.dtx v3.2m
`\showhyphens`: set `\tracinglostchars` to 0 in `\showhyphens` 598
2024-06-19 ltkeys.dtx v1.0n
`\__keys_options_class:n`: Refactor function 1154
`\__keys_options_class:nn`: New function 1154
`\__keys_options_class:nnn`: Refactor function 1154
Track options used by classes . . 1154
`\__keys_options_package:nn`: New functions 1155
`\__keys_options_package:nnn`: Skip options given to packages 1155
`\__keys_scope:N`: New function . . 1151
`\__keys_scope:n`: New function . . 1151
`\l__keys_class_only_clist`: New variable 1152
`\l__keys_forced_global_clist`: New variable 1152

`.pass-to-packages`: New key property 1151
2024-06-20 ltkeys.dtx v1.0o
`\__keys_scope:N`: Ensure only key name is stored 1151
2024-06-23 ltboxes.dtx v1.4h
`\color@endgroup`: Adjust for new `@endpe` handling 930
2024-06-23 ltlists.dtx v1.0u
`\@doendpe`: Set `\if@endpe` to `false` before calling `\par` (needed for tagging) 918
`\propagate@doendpe`: Set `\if@endpe` globally and also set up migration to the outside 919
2024-06-23 ltmiscen.dtx v1.2d
`\begin`: Separate `\begin` and `\end` definitions for individual rollback 873
`\end`: Separate `\begin` and `\end` definitions for individual rollback 874
`\end\verbvisiblespace`: Adjust for new `@endpe` handling 876
2024-06-23 ltpara.dtx v1.0n
`\g__para_standard_everypar_tl`: Append `\everypar` toks to `\g__para_standard_everypar_tl`, rollback 2023/06/01 (gh/1386) . . 450
2024-06-24 fontdef.dtx v3.0j
General: load ts1 cmr fd file in Unicode engines 770
2024-07-06 ltcmd.dtx v1.2f
`\__cmd_declare_env:ennn`: Use space-trimmed envname directly (gh/1399) 137
`\NewDocumentEnvironment`: Trim spaces from envname first (gh/1399) 212
2024-07-10 ltmiscen.dtx v1.2e
General: Drop code chunks before adding them to avoid duplication in rollback (gh/1407) 868
`\enddocument`: Drop code chunks before adding them to avoid duplication in rollback (gh/1407) 866
2024-07-13 lttagging.dtx v1.0c
`tbl/leaders/end`: Sockets for `\cline` leaders added (tagging/134) . . 1319
2024-08-03 ltclass.dtx v1.5m
`\pkgcls@use@this@release`: Add selected release to the file list (gh/1413) 1145
2024-08-09 lthooks.dtx v1.1i
`\__hook_normalize_hook_args_aux:Nn`: Distinguish between empty label and empty hook (gh/1423) 259
`\__hook_parse_dot_label:nN`: Distinguish between empty label and empty hook (gh/1423) 258
`\__hook_parse_label_default:nN`: Distinguish between empty label and empty hook (gh/1423) 258
General: New message “empty-hook” (gh/1423). 318
2024-08-16 ltluatex.dtx v1.2e
`provides_module`: Support missing version string (gh/1443) 53
2024-09-03 fontdef.dtx v3.0k
General: Load also t1cmss.fd and t1cmtt.fd 769
2024-09-04 ltlength.dtx v1.1e
`\@settodim`: (UF) Suspend tagging . 569
2024-09-05 ltproperties.dtx v1.0f
`\__property_record_value_aux:e`: Remove `\if@filesw` test to be in line with `\label`, tagging-project issue 696 856
2024-09-10 ltspace.dtx v1.3s
`\addpenalty`: Drop unnecessary `\@noitemerr` and instead generate `\@LRmoderr` if we are in restricted hmode (gh/1460) 475
`\addvspace`: Drop unnecessary `\@noitemerr` and instead generate `\@LRmoderr` if we are in restricted hmode (gh/1460) 474
2024-09-11 lterror.dtx v1.2u
`\@LRmoderr`: Error message added . 436
2024-09-13 lttagging.dtx v1.0h
`float/hmode/end`: Sockets for floats added 1320
2024-09-20 ltcounts.dtx v1.1o
`\@addtoreset`: add the parent theHfoo if a counter is reset 561
`\@definecounter`: define theHfoo (used for internal links) 559
2024-09-20 lttagging.dtx v1.0h
General: moved `\@kernel@refstepcounter` into ltxref 1328
2024-09-20 ltxref.dtx v1.1r
`\refstepcounter`: add sockets 845
provide a kernel copy kernel@refstepcounter 845
set also currentHref 845
2024-09-21 ltmeta.dtx v1.0c
General: Added tagging support . . . 460

2024-09-25 ltproperties.dtx v1.0g
`\IfLabelExistsF`: Fixed definitions of `\IfLabelExistsT` and `\IfLabelExistsF` 860
2024-10-10 lttagging.dtx v1.0i
`default`: Restore also paratagging (tagging/723) 1314
2024-10-12 ltmiscen.dtx v1.2f
`\begin`: Make `\begin` engine-protected 873
2024-10-21 lthooks.dtx v1.1j
`\IfHookEmptyF`: Define `\IfHookEmptyT`, `\IfHookEmptyF` . 323
2024-10-21 ltmarks.dtx v1.0h
`\IfMarksEqualF`: Define `\IfMarksEqualT`, `\IfMarksEqualF` 1084
2024-10-21 ltproperties.dtx v1.0h
`\IfPropertyRecordedF`: Define `\IfPropertyRecordedT`, `\IfPropertyRecordedF` 860
2024-10-21 lttagging.dtx v1.0k
`\UseExpandableTaggingSocket`: Added expandable variants . . . 1311
Changed behavior of two argument tagging sockets when disabled. . 1311
2024-10-26 ltcounts.dtx v1.1p
`\@addtoreset`: Fully expand counter name in theHfoo commands (gh/1508) 561
`\@definecounter`: Fully expand counter name in theHfoo commands (gh/1508) 559
2024-10-27 ltsockets.dtx v0.9b
`\socket_set_plug:nnn`: Make plug definition non-protected 363
`\socket_use_expandable:n`: Added `\socket_use_expandable:n` 366
2024-10-29 lthooks.dtx v1.1k
`\__hook_list_if_rule_exists:nnnF`: Skip mapping over undeclared `\g__hook_⟨hook⟩_code_prop` (gh/1513) 304
`\__hook_log:nN`: Skip mapping over undeclared `\g__hook_⟨hook⟩_code_prop` (gh/1513) 299
2024-11-02 ltdirchk.dtx v1.3b
General: Add tab char to `\dospecials` 4
2024-11-02 ltplain.dtx v1.3k
General: Add tab char to `\dospecials` 14
2024-11-06 ltoutenc.dtx v2.1b
General: Support for macron-i for OT1 527
Support for macron-i for T1 (gh/1453) 533
2024-11-09 ltmarks.dtx v1.1a
`\__mark_class_status:nnn`: Add a third argument 1082
`\__mark_status:nn`: Add a second argument 1083
`\mark_clear_structure:n`: 1077
`\mark_copy_structure:nn`: Macro renamed 1077
`\mark_use_last:nn`: 1080
2024-11-12 ltfinal.dtx v2.3d
`\@uclclist`: Add option to titlecase first or all words 1347
Use updated titlecase-first function in expl3 1347
2024-11-16 ltpage.dtx v1.0o
`\markright`: Drop legacy mark support 1090
2024-11-19 fontdef.dtx v3.0l
General: Preload the TS1 `.fd` file . . 770
Preload the TS1 `ts1cmr.fd` file in all engines 769
2024-11-21 lttagging.dtx v1.0l
`\UseExpandableTaggingSocket`: Define `\tag_if_active:TF` conditionals here (github/1558) 1311
2024-11-26 lthooks.dtx v1.1l
`\__hook_hash_check_aux:w`: Adjust debugging message (gh/1459) . . 263
`\__hook_initialize_all:`: Adjust debugging message (gh/1459) . . 287
`\__hook_initialize_hook_code:n`: Adjust debugging message (gh/1459) 288
`\__hook_initialize_single:ccn`: Adjust debugging message (gh/1459) 292
`\hook_gput_next_code:nn`: Add debugging message (gh/1459) . . 305
2024-11-26 ltoutenc.dtx v2.1c
`\@inmathwarn`: Declare command for logging (gh/1242) 513
`\@strip@args`: Alter error and info message 517
`\UndeclareTextCommand`: Log text command change (gh/1242) 520, 521
Use `\ifcsname` to avoid generating a csname 520
2024-11-27 ltfssdcl.dtx v3.1d
`\freeze@math@version`: Reset math version if necessary (gh/1101 and gh/1028) 710

Reset top-level alphabet definitions only for the current math version (gh/1101 and gh/1028) 711

2024-11-30 ltfssbas.dtx v3.2o

`\@math@level`: New approach to `localmathalphabets` (gh/1101) . 585

`\frozen@everydisplay`: New approach to `localmathalphabets` (gh/1101) 586

`\frozen@everymath`: New approach to `localmathalphabets` (gh/1101) . 586

`\math@version`: New approach to `localmathalphabets` (gh/1101) 584, 585

2024-12-03 ltmarks.dtx v1.1b

`\c__mark_empty_tl`: Fix inconsistent local/global assignment (gh/1574) 1069

2024-12-11 ltfssaxes.dtx v1.0k

General: Add `ssc` shape change rules (gh/1581) 662

Rationalize `sw` shape change rules (gh/1581) 662

Reorganized the rollback data . . . 660

2024-12-13 ltfssaxes.dtx 1.0k

General: Add numerous `\DeclareFontSeriesChangeRule` entries to support the full range of weights (from `ul` to `ub`) and widths (from `uc` to `ux`) (gh/1583) 603

Minor modifications to a few `\DeclareFontSeriesChangeRule` entries (gh/1583) 603

2024-12-21 ltoutenc.dtx v2.1c

General: Correct base letter (gh/1587) 533

2024-12-22 ltcmdhooks.dtx v1.0k

`\__hook_try_put_cmd_hook:w`: Avoid defining command while adding a cmd hook (gh/1591) 334

2024-12-27 ltsockets.dtx v0.9c

`\IfSocketPlugAssignedF`: Conditionals for sockets, plugs, and assignments (gh/1577) 367

`\socket_if_exist:n`: Conditionals for sockets, plugs, and assignments (gh/1577) 363

`\socket_if_plug_assigned:nn`: Conditionals for sockets, plugs, and assignments (gh/1577) 365

`\socket_if_plug_exist:nn`: Conditionals for sockets, plugs, and assignments (gh/1577) 364

2025-01-02 ltclass.dtx v1.5n

`\AtEndDocument`: Do not make `\AtBeginDocument` preamble only (gh/1604) 1129

2025-01-03 lthooks.dtx v1.1l

General: Correct example to use `env/quote/begin` instead of `/before` (gh/1599) 221

2025-01-10 ltmarks.dtx v1.1c

`\__mark_get_from_splitmarks:`: Do not expand mark content while debugging 1076

2025-01-15 ltkeys.dtx v1.0p

`\__keys_options_aux:n`: Only process global options on first pass . . . 1153

2025-01-21 ltcmd.dtx v1.2g

`\__cmd_grab_v_aux_catcodes:`: Store spaces and tabs as active chars . 189

`\__cmd_grab_v_aux_put:N`: Simplify catcode handling 189

2025-01-21 ltoutput.dtx v1.4l

`\@writesetup`: Support extended syntax for `\label`, `\index`, and `\glossary` (gh/311) 1257

2025-01-21 ltsect.dtx v1.1g

`\@gobble@with@sphack@som`: Support extended syntax for `\label`, `\index` and `\glossary` (gh/311) 1022

`\addtocontents`: Support extended syntax for `\label`, `\index`, and `\glossary` (gh/311) 1022

2025-01-22 ltcounts.dtx v1.2a

`\@ifbothcounters`: Guard against star alias. 563

`\c@*`: `\c@*` added (gh/1632) 559

2025-01-22 ltxref.dtx v1.1t

`\ltx@star@counter`: Macro added . 845

`\refstepcounter`: Guard * from causing infinite loop 845

2025-01-23 ltcmd.dtx v1.3a

`\__cmd_add_grabber:N`: Extend to support `c`-type grabbing 157

`\__cmd_add_type_b:w`: Extend to cover `c`-type grabbing 154

`\__cmd_add_type_b_or_c:N`: New function 154

`\__cmd_add_type_c:w`: New function 154

`\__cmd_grab_D_aux:NNnNNNN`: Extend to support `c`-type grabbing 179

`\__cmd_grab_D_long_obey_spaces_no_strip:w`: Auto-generate D-grabbers 178

`\__cmd_grab_D_verb_safe:NN`: New macro 179

`\__cmd_grab_c:w`: New c-type grabbing function 174
`\__cmd_normalize_type_b:w`: Extend to cover c-type grabbing 149
`\__cmd_normalize_type_b_or_c:nn`: New function 149
`\__cmd_normalize_type_c:w`: New function 149
`\__cmd_prepare_signature:n`: Extend to cover c-type grabbing 152
General: Generalize message `arg-after-body` 206
New message `chars-dropped-first-line` 206
New message `chars-dropped-last-line` 206
New variable 130, 133
2025-01-23 ltoutput.dtx v1.4m
`\@writesetup`: Make `\label`, `\index` and `\glossary` truly invisible when typesetting (gh/1638) . . . 1257
2025-01-23 ltsect.dtx v1.1h
`\@gobble@with@sphack@som`: Make `\label`, `\index` and `\glossary` truly invisible when typesetting (gh/1638) 1022
2025-01-26 ltlists.dtx v1.0v
`\propagate@doendpe`: Only migrate `\@doendpe` out of simple and semi-simple groups (gh1641) . . . 919
2025-01-27 lttagging.dtx 1.0m
General: add sockets for math phantom commands 1322
2025-01-29 ltoutput.dtx v1.4n
`\@addtocurcol`: Save current value of `\@reqcolroom` (gh/1645) 1269
`\@addtonextcol`: Save current value of `\@reqcolroom` (gh/1645) 1279
`\@flcheckspace`: Reset current value of `\@reqcolroom` (gh/1645) . . . 1298
2025-02-01 ltcmd.dtx v1.3b
`\ProcessList`: Use `\tl_map_tokens:nn` instead of `\tl_map_function:nN` 216
2025-02-11 ltoutenc.dtx v2.1c
`\@inmathwarn`: Log text command/symbol redeclarations (gh/1242) 513
2025-02-14 ltoutput.dtx v1.4m
`\@combinefloats`: Macro got a new name 1260
`\@if@bottomfloats@TF`: Macro added 1248
`\@if@flushbottom@TF`: Macro added 1248
`\@if@footnotes@TF`: Macro added 1248
`\@make@normalcolbox`: Macro added 1242
`\@make@specialcolbox`: Macro added 1242
`\@makecol`: Use sockets and hooks 1241
`\@outputbox@append`: Macro added 1246
`\@outputbox@appendfootnotes`: Macro added 1246
`\@outputbox@attachbottomfloats`: Macro added 1247
`\@outputbox@attachfloats`: Macro added 1247
`\@outputbox@attachtopfloats`: Macro added 1247
`\@outputbox@depth`: Macro added 1242
`\@outputbox@removebskip`: Finally fix bottom skip bug (from 2.09) . . 1244
Macro added 1244
`\@writesetup`: Hooks and sockets added 1254
General: Use sockets and hooks in OR 1248
`build/column/after`: Hook added 1251
`build/column/outputbox`: Socket added 1248
`build/page/after`: Hook added . . 1251
`build/page/reset`: Hook added . . 1251
2025-02-14 lttagging.dtx v1.0m
General: Tagging support for output routines added 1320
`build/column/footins`: Tagging socket added 1320
`build/column/outputbox`: Tagging socket added 1320
`build/page/footer`: Tagging socket added 1320
`build/page/header`: Tagging socket added 1320
2025-02-19 lttagging.dtx v1.0n
General: Moved math sockets into lttagging 1321
2025-02-20 ltfssbas.dtx v3.2p
`\CheckEncodingSubset`: If necessary, load the `.fd` before checking the encoding subset (gh/1669) 578
2025-02-21 lttagging.dtx v1.0n
`marginpar/end`: move marginpar sockets 1318
2025-02-22 ltcounts.dtx v1.2b
`\counterwithin`: `\counterwithin` needs to expand its arguments (gh/1675) 563

2025-02-25 ltoutput.dtx v1.4o
`\@writesetup`: Reset various catcodes to their default values in case a pagebreak occurs while a special catcode scheme is in force (gh/600) 1255
2025-02-26 lttagging.dtx v1.0o
`\AssignTaggingSocketPlug`: Macro added 1311
`\NewTaggingSocket`: Initialize tagging sockets with 1 or 2 arguments . 1311
2025-03-01 ltcounts.dtx v1.2b
`\counterwithout`: `\counterwithout` needs to expand its arguments (gh/1675) 564
2025-03-01 ltexpl.dtx v1.3i
`\IfExplAtLeastTF`: Also provide T and F variants of `\IfExplAtLeastTF` (gh/1522) 78
2025-03-01 ltfinal.dtx v2.3e
`\IfPDFManagementActiveTF`: Provide T F variants of `\IfPDFManagementActiveTF` (gh/1522) 1353
2025-03-01 ltfssini.dtx v3.2k
`\IfFontSeriesContextTF`: Provide T F variants of `\IfFontSeriesContextTF` (gh/1522) 754
2025-03-01 ltmeta.dtx v1.0d
General: Provide T F variants of `\IfDocumentMetadataTF` (gh/1522) 458
2025-03-05 ltexpl.dtx v1.3j
`\@kernel@after@begindocument`: Add `\@kernel@before@enddocument@afterlastpage` 72
2025-03-05 ltfssbas.dtx v3.2q
`\showhyphens`: set `\tracinglostchars` to 0 in `\showhyphens` for all engines 598
2025-03-05 ltmiscen.dtx v1.2g
`\enddocument`: Added `\@kernel@before@enddocument@afterlastpage` 866
2025-03-05 ltoutput.dtx v1.4p
`\clearpage`: A LaTeX 2_ε name for `\write` in case it gets changed after the last page 1228
Make sure this is a normal write in all circumstances 1228
2025-03-05 ltplain.dtx v2.3l
General: Use 4 as default value for `\tracinglostchars` in LuaTeX (gh/1687) 22
`\loggingall`: Remove `\tracinglostchars` setting (gh/1691) 31
`\tracingnone`: Remove `\tracinglostchars` setting (gh/1691) 32
2025-03-05 ltshipout.dtx v1.0o
`\__shipout_force_immediate_writes:`: Macro added 1210
2025-03-05 ltshipout.dtx v1.4p
`\@kernel@before@enddocument@afterlastpage`: Make all `\write`s immediate after the last page (gh/1689) 1206
2025-03-08 ltmath.dtx v1.2o
`\@eqnsel`: `\dollardollar@begin` and `\dollardollar@end` introduced for tagging 900, 901
`\]`: `\dollardollar@begin` and `\dollardollar@end` introduced for tagging 897
`\c@equation`: `\dollardollar@begin` and `\dollardollar@end` introduced for tagging 899
`\dollardollar@end`: `\dollardollar@begin` and `\dollardollar@end` introduced for tagging 897
`eqnarray`: `\dollardollar@begin` and `\dollardollar@end` introduced for tagging 906
2025-03-20 ltoutput.dtx v1.4q
`\@outputbox@appendfootnotes`: Add experimental socket for baseline attachment of footnotes – subject to change (footmisc/19) 1247
`\@outputbox@removebskip`: Remove a 'fil' bottom skip even if we are in ragged bottom typesetting (footmisc/19) 1244
`\newpage`: Remove skip added in 2.4a (footmisc/19) 1229
`build/column/baselineattach`: Add experimental socket for baseline attachment of footnotes – subject to change (footmisc/19) 1251
2025-03-25 ltoutput.dtx v1.4r
`\@addtobot`: Don't set `\maxdepth` to zero any more (footmisc/19) . . 1267
`\@backup@outputbox@depth`: Macro added (footmisc/19) 1245
`\@outputbox@reinsertbskip`: Back out the depth if we have no

footnotes and no bottom floats (footmisc/19) 1245
on: (footmisc/19) 1251
footnotes-floats-legacy: Add support for attaching footnotes with a skip starting at the baseline of the last text line (footmisc/19) 1250
2025-03-29 ltoutput.dtx v1.4s
\@outputbox@attachbottomfloats: Add socket (footmisc/19) 1247
footnotes-floats-legacy: Add socket (footmisc/19) 1250
2025-03-40 ltexpl.dtx v1.3k
\expandableinput: Added document level name for \file_input_raw:n (gh/514) 78
2025-04-12 ltcmd.dtx v1.3c
__cmd_add_arg_spec_mandatory:nn: Track total mandatory args 151
__cmd_add_grabber:N: Refine handling of optional arguments for c-type grabbing 157
__cmd_grab_c_loop:w: Correct behavior on dropping tokens . . . 175
__cmd_normalize_arg_spec:n: Track total mandatory args 144
General: New variable 130
2025-04-13 ltoutput.dtx v1.4x
\newpage: reinserted par token (github/1712) 1229
2025-04-14 ltoutput.dtx v2.3m
General: Always use extended free list 1224
2025-04-14 ltplain.dtx v2.3m
\e@alloc: Always use extended allocation 18
\e@alloc@chardef: Always use extended allocation 18
\e@alloc@top: Drop a redundant ε-TEX test 18
\e@mathgroup@top: Always use extended allocation 18
\extrafloats: Always use extended allocation 18
\newinsert: Always use extended allocation 20
Drop redundant ε-TEX test 20
\newlanguage: Always use extended allocation 17
2025-04-21 ltcmd.dtx v1.3d
__cmd_grab_c_auxviii:: Correct a logic flaw 176
2025-04-25 ltfiles.dtx v1.2w
\protected@write: Move \write outside the group /gh1717 492
2025-04-27 ltshipout.dtx v1.0p
\@kernel@after@enddocument@afterlastpage: Increment page counter gh/1717 1207
__shipout_force_immediate_writes:: Decrement page counter gh/1717 1210
2025-04-28 ltmiscen.dtx v1.2h
\enddocument: Increment page counter gh/1717 866
2025-05-10 ltclass.dtx v1.5m
\IfClassLoadedWithOptionsTF: avoid errors for par in code argument of T versions (gh/1733) 1107
2025-05-29 ltmath.dtx v1.2p
\leqno: Ensure saved version is primitive gh/1747 903
2025-06-03 ltdefns.dtx v1.5u
\provide@command: Strip trailing spaces from arg (gh/1708) 89
\renew@command: Strip trailing spaces from arg (gh/1708) 87
2025-06-04 ltfinal.dtx v2.3f
\@uclclist: Add default for titlecasing 1347
2025-06-05 ltexpl.dtx v1.3l
\IfExplAtLeastTF: Provide T and F variants of \IfExplAtLeastTF with a correct implementation (gh/1752) 78
2025-06-07 ltoutput.dtx v1.4y
\@writesetup: Reset catcode of CR (gh/1766) 1255
2025-06-08 ltoutput.dtx v1.4y
\@writesetup: Reset catcode of TAB (gh/1766) 1255
2025-06-19 ltmarks.dtx v1.1d
__mark_new_class:nn: Also initialize saved-column region (SX chat) 1069
2025-06-23 ltfssaxes.dtx 1.0l
General: Improve the order of the \DeclareFontSeriesChangeRule entries (gh/1727) 626
Offer m as alternative result series for some \DeclareFontSeriesChangeRule entries (gh/1727) 627
2025-06-25 ltkeys.dtx v1.0q
__keys_options_aux:n: Revise for multiple family support 1153
__keys_options_local:: Revise for multiple family support 1155

`\__keys_options_local:n`: New function 1156
`\__keys_options_local:nnn`: New function 1156
`\l__keys_local_clist`: New variable 1152
2025-06-26 ltdefns.dtx v1.5v
`\declare@robustcommand`: Allow for active chars 92
`\declare@robustcommand@auxi`: New macro 92
`\declare@robustcommand@auxii`: New macro 92
`\declare@robustcommand@auxiii`: New macro 92
2025-06-27 ltcmd.dtx v1.3e
`\__cmd_show:e`: Support `\showstream` (gh/1062) 170
2025-06-27 lthooks.dtx v1.1m
`\hook_log:n`: Support `\showstream` (gh/1062) 298
2025-06-29 ltfsstrc.dtx v3.0q
`\getanddefine@fonts`: Introduced `\transform@scriptfont` for script fonts. 684
2025-07-01 ltfinal.dtx v2.3g
`\@uclclist`: Add `\DeclareLowercaseExclusions`, `\DeclareTitlecaseExclusions` and `\DeclareUppercaseExclusions` 1349
2025-07-18 ltoutenc.dtx v2.1d
General: Add forgotten `\DeclareFontSubstitution` (gh/1709) 526, 528
2025-08-01 ltproperties.dtx v1.0j
`\__property_gset:nnnn`: correct name of shipout boolean 855
2025-08-03 lttagging.dtx v1.0s
General: Add sockets for block code 1317
2025-08-11 ltmarks.dtx v1.1e
`\__mark_vbox_set_split_to_maxdimen:NN`: Use `\ignoreprimitiveerror` if available (gh/1750) 1072
2025-08-21 lttagging.dtx v1.0t
`default`: Use symbolic name instead of text-unit 1314
2025-08-22 ltoutput.dtx v1.4z
`\@writesetup`: Don't use `\nfss@catcodes` + updates, but spell everything out explicitly . 1255
Move setting of `\protect` after all hooks so that only the `\write` statements are affected (gh/1845) 1256
2025-08-23 lttagging.dtx v1.0r
General: MathML intent macros added 1310
2025-09-15 ltfssdcl.dtx v3.1e
`\process@table`: Run `\set@current@meta@family` and the `normalfont` hook 713
2025-09-15 ltfssini.dtx v3.2l
`\@currentmetafamily`: Macro added 740
`\prepare@family@series@update`: Always set `\@currentmetafamily` 740
`\reset@font`: Drop old 2020 hook `\@defaultfamilyhook` 761
Set current meta family 761
`\set@current@meta@family`: Macro added 761
`\update@series@target@value`: Rename `\target@meta@family@value` to `\@currentmetafamily` and set it always 741
2025-09-23 ltcmd.dtx v1.3f
`\__cmd_add_arg_spec_mandatory:nn`: Correct logic for recording position of mandatory args 151
General: Rename variable 130
2025-09-23 ltfinal.dtx v2.3h
General: Setting `\partokencontext` 1350
2025-09-24 lthooks.dtx v1.1n
`\__hook_hash_check:nTF`: New function 262
`\__hook_hash_check_aux:w`: New function 262
Only double hashes when needed 264
2025-09-24 lttab.dtx v1.1t
`\@array`: Check `\currentgrouptype` in `\par` (gh1864) 961
2025-10-01 ltdefns.dtx v1.5w
`\@star@or@long`: Handling protection status differently (gh/571) 83
`\@yargd@f`: Ensure that commands without arguments are not long (gh/571) 86
Handling protection status differently (gh/571) 86
`\declare@robustcommand@auxiii`: Handling protection status differently (gh/571) 93
`\pr@tectedrel@x`: Handling protection status differently (gh/571) 84

2025-10-02 ltcmd.dtx v1.3g
__cmd_arg_spec_opt:N: New function . . . 205
__cmd_cmd_type_cases:NnnnnnF: Remove protected status . . . 203
__kernel_cmd_if_xparse:NTF: Remove protected status . . . 203
\cmd_arg_spec:c: New function . . . 205
2025-10-12 ltcmdhooks.dtx v1.0l
__hook_retokenize_patch:Nnn: Reset the catcode of space when patching . . . 346
2025-10-20 lttagging.dtx v1.0u
General: Tagging sockets for headings added . . . 1317
2025-11-04 ltcmd.dtx v1.3h
__cmd_add_default_E:nn: Update for \NoValue adjustment . . . 158
__cmd_add_expandable_type_E:w: Update for \NoValue adjustment 160
__cmd_add_type_v:w: Update for \NoValue adjustment . . . 157
__cmd_expandable_grab_D:NNNwNNn: Update for \NoValue adjustment 191
__cmd_expandable_grab_D_alt:NNwNNn: Update for \NoValue adjustment 193
__cmd_expandable_grab_R_alt_aux:NNwNNn: Update for \NoValue adjustment 195
__cmd_expandable_grab_R_aux:NNNwNNn: Update for \NoValue adjustment 194
__cmd_grab_D_aux:NNnNNNN: Update for \NoValue adjustment . . . 179
__cmd_grab_E_long_obey_spaces:w: Update for \NoValue adjustment 182
__cmd_grab_R_aux:NNnN: Update for \NoValue adjustment . . . 185
__cmd_grab_v_aux_abort:n: Update for \NoValue adjustment . . . 189
__cmd_normalize_type_s:w: Update for \NoValue adjustment . . . 145
__cmd_show_processor:Nw: Update for \NoValue adjustment . . . 172
__cmd_split_argument_aux:nnnn: Update for \NoValue adjustment 197
2025-11-05 ltcmd.dtx v1.3h
__cmd_if_novalue_aux:w: Update for \NoValue adjustment . . . 215
General: Update for \NoValue adjustment . . . 215
\NoValue: New command . . . 215
\tl_if_novalue:n: Update for \NoValue adjustment . . . 215
2025-11-10 ltcmdhooks.dtx v1.0m
__hook_double_hashes_space:w: Support \alignmark . . . 344
2025-11-10 ltxref.dtx v1.1v
\label: (UFi)Disable commands while writing to the aux, gh1841 . . . 844
2025-11-20 ltproperties.dtx v1.0l
\property_item:ee: added \property_item:nn, gh1194 . . . 858
2025-11-22 lttagging.dtx v1.0v
General: Add sockets for minipage and parbox . . . 1317
2025-11-27 ltfinal.dtx v2.3i
\@addtofilelist: Support commas in file names (gh/1920) . . . 1352
\@addtofilelist@aux: New macro 1352
2025-12-17 ltspace.dtx v1.3t
\@esphack: Prevent generating an extra space when chained (gh/1910) . . . 471
2025-12-23 ltluatex.dtx v1.2f
\newluachunkname: Use \csstring instead of \string . . . 50
2025-12-31 ltcounts.dtx v1.2c
\@addtoreset: Support alias counters 561
\@definecounter: Add support for alias counter . . . 559
\@removefromreset: Support for alias counter . . . 562
General: Added \newcounteralias . 556
\newcounteralias: New command \newcounteralias . . . 558
2025-12-31 ltthm.dtx v1.0h
\@othm: Use new alias counter (UFi) 1009
2026-01-02 ltfsstrc.dtx v3.0r
\DeclareMathScriptfontMapping: Mappings defined by \DeclareMathScriptfontMapping are set globally. . . . 685
2026-01-15 lttab.dtx v1.1u
General: Hook declarations for array and longtable added . . . 972
2026-01-20 ltpictur.dtx v1.2c
\pictur@: adding no-op optional argument for gh/1172 . . . 976
2026-02-02 ltcmd.dtx v1.3i
__cmd_arg_to_keyvalue_braces:nnn: Treat empty input as empty keyval list . . . 199
2026-02-15 ltfloat.dtx v1.2h
\@textsubscript: Drop use of fake math . . . 1045
\@textsuperscript: Drop use of fake math . . . 1044

`\textsubscript@offset`: Command added 1045
`\textsubscript@space`: Command added 1045
`\textsuperscript@offset`: Command added 1044
`\textsuperscript@space`: Command added 1044
2026-02-24 ltboxes.dtx v1.4i
`\@parboxto`: Use text mode vertical centering 936
`\vcenter@text`: New macro 935
`\vcenter@text@auxi`: New macro . . 935
`\vcenter@text@auxii`: New macro . 935
`\vcenter@text@axis`: New macro . . 935
2026-02-24 lttab.dtx v1.1v
`\@tabarray`: Move `\m@th` to `\array` 960
`\@tabular`: Remove math mode switch 960
General: Use text mode vertical centering 960
`\array`: Move `\m@th` from `\@tabarray` 959
`\endtabular`: Remove math mode switch 959
2026-02-25 ltcmd.dtx v1.3j
General: Add documentation as part of `cmdguide` work 115
2026-02-25 ltfinal.dtx v2.3j
General: Define `\pdfglyphtounicode` for luaTeX 1340
Load and enable glyphtounicode.tex for luaTeXin PDF mode 1340
Provide noop definitions for the other engines 1340
2026-03-10 lthooks.dtx v1.1o
`\__hook_curr_name_push_aux:n`: Support dot notation (gh/1162) . 260
2026-03-12 ltcmd.dtx v1.3k
General: Switch from `x`- to `e`-type expansion in expl3 code 129
2026-03-12 ltcmdhooks.dtx v1.0n
General: Switch from `x`- to `e`-type expansion in expl3 code 332
2026-03-12 ltfilehook.dtx v1.0p
General: Switch from `x`- to `e`-type expansion in expl3 code 1165
2026-03-12 ltfinal.dtx v2.3k
General: Switch from `x`- to `e`-type expansion in expl3 code 1332
2026-03-12 ltfssdcl.dtx v3.1f
General: Switch from `x`- to `e`-type expansion in expl3 code 702
2026-03-12 lthooks.dtx v1.1o
General: Switch from `x`- to `e`-type expansion in expl3 code 245
2026-03-12 ltkeys.dtx v1.0r
General: Switch from `x`- to `e`-type expansion in expl3 code 1151
2026-03-12 ltmarks.dtx v1.1f
General: Switch from `x`- to `e`-type expansion in expl3 code 1068
2026-03-12 ltpara.dtx v1.0o
General: Switch from `x`- to `e`-type expansion in expl3 code 448
2026-03-12 ltproperties.dtx v1.0m
General: Switch from `x`- to `e`-type expansion in expl3 code 855
2026-03-12 ltshipout.dtx v1.0q
General: Switch from `x`- to `e`-type expansion in expl3 code 1193
2026-03-12 ltsockets.dtx v0.9d
General: Switch from `x`- to `e`-type expansion in expl3 code 361
2026-03-13 ltcmd.dtx v1.3k
`\__cmd_defaults_error:w`: Tidy up variants 141
2026-03-13 ltexpl.dtx v1.3m
General: Add check for sufficiently-recent expl3 75
2026-03-13 lthooks.dtx v1.1o
`\str_count:e`: Tidy up variants . . . 247
2026-03-13 ltkeys.dtx v1.0r
General: Tidy up variants 1152
2026-04-08 lttagging.dtx v1.0w
`struct/end`: Added generic sockets for structure 1313
2026-04-12 ltsockets.dtx v0.9e
`\__socket_debug_gset:`: Use `\on@line` in debugging messages 362
2026-04-13 ltboxes.dtx v1.4j
`\color@endgroup`: Do not reset `@endpe` in `\color@endgroup` when ending an `\hbox` 930
2026-04-16 ltfinal.dtx v2.3l
General: Revise glyphtounicode loading to reduce tokens in `\everyjob` 1340
2026-04-22 ltmeta.dtx v1.0e
General: Add hook to change target name and use it in `\NextLinkTarget` 460
2026-04-22 lttagging.dtx v1.0x
`inline/end`: added inline/begin, inline/end socket 1313
`struct/end`: renamed struct-begin socket 1313

2026-04-26 ltcmd.dtx v1.3l
`\__cmd_add_arg_spec_mandatory:nn`: Update mandatory spec function to use two arguments (gh/1204) . 151
`\__cmd_normalize_type_s:w`: Update `\@@_normalize_type_r:w` to use `\@@_normalize_type_R_aux:w` (gh/1204) 145
`\__cmd_normalize_type_v:w`: Update `\@@_normalize_type_R:w` to use a new auxiliary function (gh/1204) 148
2026-04-27 ltsockets.dtx v0.9f
`\socket_get_plug:nN`: A command to retrieved the assigned plug (gh/1851) 365
2026-05-01 ltproperties.dtx v1.0n
`\property_item:een`: added `\property_item:nnn`, gh2018 . . . 858
2026-05-06 ltmeta.dtx v1.0f
General: Add `\SaveLastSkip` and `\RestoreLastSkip` to better mimic hyperref's spacing behavior, gh/2070 460
Use `\latelua` to mimic a whatsit to avoid spurious warning, gh/2070 460
2026-05-13 ltfloat.dtx v1.2i
`\textsubscript@offset`: Correct value of offset for large x-height 1045
2026-05-16 lttagging.dtx v1.0z
`kernel␣(para/textblock/end)`: Add unskip, tagging issue 803 1315
2026-05-27 ltmeta.dtx 1.0g
General: Add commands related to document properties container . . 459
2026-06-01 ltcmd.dtx v1.3m
General: Switch to US spelling for macro names 115
`\NoValue`: Allow for overwriting expl3 version 215
2026-06-01 ltexpl.dtx v1.3n
General: Switch to US spelling for macro names 73
2026-06-01 ltfinal.dtx v2.3m
General: Switch to US spelling for macro names 1338
2026-06-01 lthooks.dtx v1.1p
General: Switch to US spellings for macro names 245
2026-06-02 ltkeys.dtx v1.0s
`\DeclareUnknownKeyHandler`: Correct doubling of hashes 1157
2026-06-24 ltsockets.dtx v0.9g
`\__socket_debug_gset:`: Do not expand `\on@line` prematurely (gh/2132) 362
2026-07-28 ltcmd.dtx v1.3n
`\__cmd_prepare_signature_verb_chk:n`: Test last character as a string . . 152
2026-07-29 ltboxes.dtx v1.4k
`\underline`: Avoid fake math in `\underline` 943
`\underline@text`: New macro 943
`\underline@text@kern`: New macro 943
`\underline@text@thickness`: New macro 943
`\underline@text@vgap`: New macro 943
2026-07-31 ltoutenc.dtx v2.2a
General: HiTeX support 509
2026-08-20 lttagging.dtx v1.1a
`default`: Avoid use of internal tagpdf variables 1314
2026-09-13 ltclass.dtx v1.5o
`\@currnamestack`: Put `\pkgcls@outerdate` on the stack (gh/828) 1100
Retrieve the value for `\pkgcls@innerdate` (gh/828) . . 1102
`\pkgcls@outerdate`: Set `\pkgcls@outerdate` (gh/828) . . 1140
`\pkgcls@parse@date@arg`: Set `\pkgcls@outerdate` (gh/828) . . 1142
2026-09-13 ltfilehook.dtx v1.0q
`\g__filehook_nesting_level_int`: File stack now has 4 items per file. 1181
2026-09-13 ltoutput.dtx v1.5a
`\@writesetup`: Resetting `\catcode` of comma added (gh/2056) 1255
2026-09-15 ltmeta.dtx v1.0h
`\NextLinkTarget`: Do not set the spacefactor in math mode, gh2188 461
2026-09-16 ltluatex.dtx v1.2g
`new_luacommand`: Function added . . . 58
2026-09-24 ltvers.dtx v1.1l
`\development@branch@name`: Use two digits for month and day 35
2026-09-27 ltfinal.dtx v2.3n
General: Support strong emphasis (tagging/1620) 1352
2026-09-27 ltfntcmd.dtx v3.5b
`\strongemph`: Support strong emphasis (tagging/1620) 793
2026-09-27 ltfssini.dtx v3.2m
General: Support strong emphasis (tagging/1620) 758
2026-10-01 ltfloat.dtx v1.2j
`\@textsubscript`: Move `\check@mathfonts` inside group 1045

`\@textsuperscript`: Move `\check@mathfonts` inside group 1044
2026-10-12 ltshipout.dtx v1.0r
`\__shipout_picture_overlay:n`: Add support for links with page breaks (gh/2210) 1192
2026-10-28 ltluatex.dtx v1.5h
General: Rename callback descriptions to identifiers 58
`add_to_callback`: Add warning for duplicate callback identifiers 67
2026-10-29 ltpara.dtx v1.0p
`\__para_handle_indent:`: Make `\OmitIndent` call `\noindent` when used in vertical mode (tagging/1674) 451

v1.0b lttemplates.dtx 2024-02-15
`\IfInstanceExistsTF`: New macros 419
v1.0c lttemplates.dtx 2024-04-17
`\IfInstanceExistsTF`: Use plural names 419
v1.0d lttemplates.dtx 2024-10-07
`\__template_assign_variable:`: Correct passing of `\KeyValue` contents 409
v1.0d lttemplates.dtx 2025-01-08
`\SetTemplateKeys`: Test for empty key/val list to speed up processing 419
v1.0e lttemplates.dtx 2025-01-20
`\__template_declare_template_code:nnnnn`: Speed up test for `\AssignTemplateKeys` 394
v1.0f lttemplates.dtx 2025-07-08
`\__template_parse_values_elt_auxi:n`: Allow for value expansion 400
`\__template_parse_values_elt_auxi:w`: New macro 400
`\__template_parse_values_set:c`: New macro 400
`\__template_parse_values_set:e`: New macro 400
General: New variable 382
v1.0g lttemplates.dtx 2025-11-21
`\__template_instance_value:nnn`: New function 406
`\__template_show_values:nn`: Extend message 412
`\InstanceValue`: New command . . . 419
v1.0h lttemplates.dtx 2025-12-24
`\__template_assign_function:`: Create protected functions 408
v1.0i lttemplates.dtx 2026-03-1
General: Add debugging support . . 383
v1.0j lttemplates.dtx 2026-04-21
`\__template_debug_off:`: Show debugging on the terminal 383
v1.0k lttemplates.dtx 2026-06-24
`\__template_parse_vars_elt_auxiv:nnw`: New function to support use of `name` 395
`\__template_parse_vars_elt_auxv:enn`: New function to support use of `name` 395
v1.0l lttemplates.dtx 2026-08-14
`\__template_parse_keys_elt:nn`: Revise value setting 390
`\__template_parse_values_elt_auxi:n`: Revise value setting 400
General: New constant 381
v1.0m lttemplates.dtx 2026-09-29
`\__template_show_values:nn`: Error if instance is unknown 412
v1.0n lttemplates.dtx 2026-10-07
`\__template_date_compare:nNnTF`: New function 380
`\__template_date_compare_aux:nNn`: New function 380
`\__template_date_compare_auxi:nnnNnnn`: New function 380
`\__template_date_compare_auxiii:w`: New function 380
`\__template_declare_template_equiv:nnnn`: New function 392
`\__template_execute_if_keyname_free:VT`: New function 385
`\__template_parse_equiv_elt:n`: New function 392
`\__template_parse_equiv_elt:nnnn`: New function 392
`\__template_parse_equiv_elt_aux:nnn`: New function 392
`\__template_parse_values_add:c`: New function 400
`\__template_parse_values_add:e`: New function 400
`\__template_parse_values_elt_auxi:n`: Rename function 400
`\__template_parse_values_elt_auxiii:Nn`: New function 400
`\__template_parse_values_elt_auxiv:w`: New function 400
`\__template_parse_values_set:c`: Rename function 400
`\__template_parse_values_set:e`: Rename function 400
`\__template_parse_vars_elt_auxiii:enn`: Split out each keytype path 395

`\__template_parse_vars_instance:nnn`: New function 397
`\__template_tmp:n`: New function . 383
General: New message . . . 413, 414, 416
`\DeclareTemplateInterfaceEquivalents`: New function 418
v1.0n lttemplates.dtx 2026-10-08
`\__template_declare_template_keys:nnnn`: Reset all template data in interface declaration 388
`\__template_parse_equiv_auxi:nnnn`: New function 392
`\__template_parse_equiv_auxii:n`: New function 392
`\__template_parse_equiv_auxiii:nnnn`: New function 392
`\__template_parse_values_set:c`: Record value order 400
`\__template_parse_values_set:e`: Record value order 400
`\__template_recover_vars:nn`: Extend to cover messages 387
`\__template_store_key_implementation:nnn`: Omit error for equivalent key types 395
Reset all template data in interface declaration 395
`\__template_store_vars:nn`: Extend to cover messages 386
General: New constant 381
New message 415
New variable 382
v1.0n lttemplates.dtx 2026-10-09
`\__template_keys_set_known_aux:nnn`: New function 411
`\__template_parse_known_values:nnn`: New function 399
`\__template_parse_values:NNnnn`: New function 399
`\__template_parse_values:nnn`: Support selectable behavior for unknown keys 399
`\__template_parse_values_elt:NNn`: Permit missing value for boolean keys 400
`\__template_parse_vars_elt_auxv:enn`: Remove l3keys creation 395
General: New variable 382, 383
`\SetTemplateKeys`: Switch to template-native approach to key setting 419
v1.0n lttemplates.dtx 2026-10-13
`\SetCurrentKnownTemplateKeys`: New command 420
v1.0n lttemplates.dtx 2026-10-14
`\__template_convert_to_assignments_aux:nV`: Correct error message 405
`\__template_store_key_implementation:nnn`: Retain variable information 395
v1.0n lttemplates.dtx 2026-10-15
`\__template_implement_choice_elt_aux:nnn`: Remove l3keys creation 398
`\__template_implement_choices:nn`: Remove l3keys creation 397
`\__template_parse_values_elt_auxi:w`: Rename function 400
`\__template_parse_values_meta:nn`: New function 400
`\__template_parse_values_meta_elt:nnnn`: New function 400
v1.0o lttemplates.dtx 2026-10-29
`\__template_parse_values_elt_auxiv:w`: Retain + when storing unknown keys 400

Index

The italic numbers denote the pages where the corresponding entry is described, numbers underlined point to the definition, all others indicate the places where it is used.

Symbols

\! 02:306, 02:308, 57:567
\! 38:214
\" 04:520, 04:521, 21:246, 21:399, 21:440, 21:478, 21:491, 21:602, 21:634, 21:661, 21:669, 21:675, 21:679, 21:685, 21:689, 21:695, 21:701, 21:708, 21:709, 21:715, 21:719, 21:770, 21:816, 21:1272, 21:1290, 21:1296, 21:1300, 21:1306, 21:1310, 21:1316, 21:1322, 21:1329, 21:1330, 21:1336, 21:1340, 21:1342, 21:1449, 24:513, 33:135, 33:162, 54:873, 57:568
\# 01:46, 01:59, 02:6, 02:14, 02:406, 06:893, 09:480, 24:500, 54:858, 57:551
\$ . . . 01:58, 02:4, 02:13, 06:892, 21:327, 21:465, 21:472, 21:588, 21:828, 21:835, 33:595, 33:602, 54:855, 57:552
\$ *1376*
\% 01:59, 01:89, 01:91, 01:111, 02:14, 02:404, 06:893, 21:514, 21:516, 24:502, 33:604, 33:724, 37:212, 50:1609, 50:1610, 54:876, 57:553
\& 01:58, 02:5, 02:13, 02:405, 06:892, 50:474, 54:856, 57:554
\’ 02:426, 21:247, 21:400, 21:442, 21:476, 21:488, 21:604, 21:614, 21:620, 21:622, 21:625, 21:627, 21:635, 21:641, 21:647, 21:649, 21:652, 21:654, 21:662, 21:666, 21:673, 21:677, 21:682, 21:687, 21:690, 21:692, 21:699, 21:704, 21:705, 21:712, 21:717, 21:720, 21:771, 21:818, 21:837, 21:839, 21:840, 21:841, 21:844, 21:846, 21:847, 21:848, 21:850, 21:851, 21:1266, 21:1287, 21:1294, 21:1298, 21:1303, 21:1308, 21:1311, 21:1313, 21:1320, 21:1325, 21:1326, 21:1333, 21:1338, 21:1341, 21:1349, 21:1350, 21:1397, 21:1398, 21:1403, 21:1404, 21:1415, 21:1416, 21:1421, 21:1422, 21:1450, 21:1451, 21:1465, 21:1466, 21:1467, 21:1468, 21:1481, 21:1482, 24:512, 29:932, 30:264, 33:125, 37:740, 38:254, 40:450, 40:471, 41:72, 54:835, 54:872, 57:569
\(. 07:2621, 38:389
\(. 38:271
\) 02:426, 07:2643, 38:390
\) 38:271
* 22:186, 24:505, 50:1290, 50:1421, 50:1535, 50:1611, 54:865
* 38:251
\+ 41:72
\+ *1374*
\, . 02:307, 02:309, 30:517, 37:740, 38:7, 38:8, 38:40, 38:167, 38:169, 38:172, 38:197, 38:220, 38:221, 38:237, 54:874
\, ... *1415*
\- 02:253, 06:24, 18:485, 18:576, 21:437, 21:438, 21:597, 21:812, 21:813, 24:507, 37:740, 40:449, 40:470, 41:72, 54:867, 57:224, 57:264
\- 06:927, *112*
\. 02:306, 02:308, 20:45, 20:115, 20:173, 21:248, 21:401, 21:473, 21:474, 21:497, 21:610, 21:611, 21:637, 21:638, 21:664, 21:772, 21:842, 21:849, 21:1271, 21:1353, 21:1354, 21:1363, 21:1364, 21:1373, 21:1374, 21:1391, 21:1452, 21:1453, 21:1489, 21:1490, 24:506, 33:137, 33:163, 54:866
\. *1403*
.../after (hook) *1169*
\..default *602*
.code 51:4
.if 51:4
.ifnot 51:4
.pass-to-packages 51:22
.store 51:4
.usage 51:4
\/ 01:81, 06:25, 09:218, 09:293, 24:454, 24:508, 50:473, 54:868
\/ *1370*
/after (hook) *1160*, *1160*, *1161*, *1169*
/before (hook) *1160*, *1160*, *1169*
/end (hook) *1169*
\: 38:214, 38:224, 38:225, 38:226, 38:242, 38:252, 38:252, 06:887, 06:888, 02:307, 02:309
\; 02:307, 02:309, 30:511, 38:198
\; 38:214, *895*
\< 21:598, 21:763, 24:503, 37:740, 41:71, 41:109, 54:863

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\= 21:249, 21:402, 21:479, 21:496, 21:737, 21:773, 21:1269, 21:1343, 21:1344, 21:1359, 21:1360, 21:1382, 21:1383, 21:1384, 21:1409, 21:1410, 21:1435, 21:1436, 21:1469, 21:1470, 21:1471, 21:1472, 21:1487, 21:1488, 21:1495, 21:1496, 29:932, 33:143, 40:450, 40:471, 41:71
\> 21:595, 21:764, 24:504, 37:740, 38:226, 38:241, 38:242, 38:252, 41:71, 54:864
\? 02:306, 02:308, 57:569
@@ commands:
\@@_normalize_type_R:w *1442*
\@@_normalize_type_R_aux:w . . . *1442*
\@@_normalize_type_r:w *1442*
\@@par *438*
\@TeXversion *6*, *1*
@botlist commands:
\@botlist: 54:1380, 54:1475, 54:1624, 54:1778
@botnum commands:
\@botnum: 54:1354
\@car *82*
\@cdr *82*
@citeb commands:
\@citeb: 47:36, 47:65, 47:82
@colht commands:
\@colht: 54:532, 54:545
@colnum commands:
\@colnum: . . . 54:1540, 54:1685, 54:1852
\@cons *82*
\@currdir *5*, *1*
@currname commands:
\@currname: 20:780
\@dblarg *81*
@dbldeferlist commands:
\@dbldeferlist: 54:2332, 54:2374
@dbltopnum commands:
\@dbltopnum: 54:2231, 54:2358
@deferlist commands:
\@deferlist: 54:1466, 54:1563, 54:1615, 54:1708, 54:1768, 54:1876, 54:1921, 54:1950, 54:2002, 54:2031, 54:2088, 54:2120, 54:2206, 54:2246
\@if@bottomfloats@TF *1233*
\@if@footnotes@TF *1233*
\@ifclasslater *1097*
\@ifclassloaded *1097*
\@ifclasswith *1097*
\@ifdefinable *81*
\@ifnextchar *81*
\@ifpackagelater *1097*
\@ifpackageloaded *1097*
\@ifpackagewith *1097*
\@ifstar *81*
\@ifundefined *81*
\@missingfileerror *1098*
\@namedef *81*
\@nameuse *81*
\@outputbox@append *1233*
\@outputbox@appendfootnotes *1233*
\@outputbox@attachbottomfloats . . *1233*
\@outputbox@attachfloats *1233*
\@outputbox@attachtopfloats *1233*
\@outputbox@reinsertbskip *1233*
\@restorepar *438*
\@setpar *438*
@topnum commands:
\@topnum: 54:1400
\[. 24:509, 29:44, 29:55, 29:77, 29:88, 38:391, 54:869, 57:570
\[. 38:306, 38:555, *1405*
\\ 01:58, 01:231, 01:232, 01:233, 01:234, 01:237, 01:244, 01:245, 01:246, 01:247, 01:250, 01:257, 01:258, 01:259, 01:260, 01:263, 01:270, 01:276, 01:277, 01:281, 01:283, 01:284, 01:288, 01:293, 01:294, 01:297, 01:303, 02:13, 02:252, 04:271, 04:425, 06:272, 06:380, 06:541, 06:664, 06:682, 06:892, 07:472, 07:532, 07:563, 07:713, 07:725, 07:756, 07:1158, 07:1164, 07:2834, 07:2862, 07:2888, 07:2895, 07:2924, 07:3021, 07:3049, 07:3060, 07:3086, 07:3092, 07:3099, 08:1799, 08:1810, 08:2628, 08:2642, 08:2643, 08:2644, 08:2645, 08:2646, 08:2651, 08:2654, 08:2655, 08:2656, 08:2666, 08:2667, 08:2681, 08:2702, 08:2706, 08:2707, 08:2709, 08:2715, 08:2726, 08:2727, 08:2732, 08:2737, 08:2742, 08:2758, 08:2923, 08:2924, 08:2925, 09:477, 09:615, 11:1319, 11:1332, 11:1338, 11:1339, 11:1340, 11:1341, 11:1354, 11:1360, 11:1361, 11:1373, 11:1382, 11:1388, 11:1389, 11:1396, 11:1403, 11:1410, 11:1417, 11:1424, 11:1425, 11:1434, 11:1458, 11:1459, 11:1477, 11:1478, 11:1479, 11:1502, 11:1503, 11:1504, 11:1505, 11:1506, 11:1507, 11:1508, 11:1509, 11:1510, 11:1511, 11:1512, 14:279, 16:148, 16:155, 16:162, 16:163, 16:164, 16:165, 18:578, 20:773, 20:792, 21:583, 24:497, 30:262, 36:271, 37:453, 37:458, 37:463, 37:473, 37:477, 37:481, 37:522,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

38:432, 38:461, 38:616, 40:482, 40:689, 40:691, 41:73, 41:174, 41:181, 41:208, 41:215, 41:246, 41:269, 42:150, 54:852, 57:555, *463*
\\ 18:53, *1379*
\{ 01:6, 01:10, 01:58, 02:2, 02:13, 07:635, 07:2163, 09:478, 14:22, 21:328, 21:585, 24:498, 30:260, 37:521, 38:59, 38:167, 54:853, 57:558
\} 01:11, 01:58, 02:3, 02:13, 09:479, 14:21, 21:329, 21:586, 24:499, 30:261, 37:521, 38:59, 54:854, 57:559
\ý *1229*
\⟨*addto-cmd*⟩ *219*
\⟨*cmd*⟩␣ *163*
\⟨*cmd*⟩␣(arg␣⟨*num*⟩) *165*
\⟨*cmd*⟩␣␣ *163*
\⟨*cmd*⟩␣code *163*
\⟨*cmd*⟩␣defaults *163*
\⟨*filename*⟩ *1175*
\⟨*function*⟩ *191*
\␣ . . 01:58, 01:75, 02:13, 02:256, 02:312, 02:341, 02:342, 02:359, 06:892, 07:1836, 07:2201, 08:2707, 11:1361, 11:1389, 11:1397, 11:1404, 11:1425, 11:1459, 14:19, 14:20, 14:21, 14:22, 14:25, 18:490, 24:494, 24:745, 24:782, 24:807, 30:263, 37:505, 37:506, 37:515, 37:516, 37:661, 37:677, 37:691, 43:67, 43:69, 43:74, 43:76, 47:37, 50:467, 54:861, 57:550
\] . . . 02:426, 24:510, 38:392, 54:870, 57:571
\] 38:306, 38:579
\^ . . 01:47, 01:56, 01:59, 01:103, 01:314, 02:7, 02:9, 02:11, 02:14, 02:255, 02:312, 02:313, 02:332, 02:333, 02:354, 02:355, 06:20, 06:893, 07:6, 07:7, 07:1615, 07:1618, 07:1622, 07:1658, 07:1707, 07:1746, 07:1810, 07:1834, 07:2102, 07:2202, 07:2489, 07:2494, 07:3330, 18:578, 18:580, 18:582, 21:251, 21:306, 21:403, 21:477, 21:489, 21:581, 21:667, 21:674, 21:678, 21:683, 21:688, 21:693, 21:700, 21:706, 21:707, 21:713, 21:718, 21:774, 21:1267, 21:1284, 21:1288, 21:1295, 21:1299, 21:1304, 21:1309, 21:1314, 21:1321, 21:1327, 21:1328, 21:1334, 21:1339, 21:1351, 21:1352, 21:1369, 21:1370, 21:1377, 21:1378, 21:1392, 21:1393, 21:1394, 21:1423, 21:1424, 21:1445, 21:1446, 21:1447, 21:1448, 24:495, 24:496, 24:501, 33:133, 33:161, 37:204, 37:213, 37:507, 37:508, 37:662, 50:1291, 50:1292, 50:1293, 50:1374, 50:1377, 50:1380, 50:1422, 50:1423, 50:1424, 50:1506, 50:1509, 50:1512, 50:1536, 50:1537, 50:1538, 50:1596, 50:1599, 50:1602, 54:857, 54:859, 54:862, 57:255, 57:256, 57:257, 57:258, 57:259, 57:260, 57:261, 57:262, 57:263, 57:556, 57:562, 57:563, 57:564, 57:565, 57:599, 57:600, 57:601, 57:602, 57:603, 57:604, 57:605, 57:606, 57:607
_ 21:334, 30:265, 38:269, 38:270, 02:8, 02:14, 06:893, 54:860, 57:557, 01:59, *1383*
__hook\textvisiblespace␣\meta␣{hook} *248*
__hook_next\textvisiblespace␣\meta␣{hook} *248*
__hook_toplevel\textvisiblespace␣\meta␣{hook} *248*
\‘ 21:252, 21:404, 21:441, 21:475, 21:487, 21:603, 21:665, 21:672, 21:676, 21:681, 21:686, 21:691, 21:698, 21:702, 21:703, 21:711, 21:716, 21:775, 21:817, 21:1265, 21:1286, 21:1293, 21:1297, 21:1302, 21:1307, 21:1312, 21:1319, 21:1323, 21:1324, 21:1332, 21:1337, 24:511, 29:932, 33:139, 37:740, 40:450, 40:471, 41:72, 54:871, 57:572
\| 21:584, 22:336, 22:347, 30:584, 30:585, 57:573
\~ 01:59, 02:10, 02:14, 06:893, 14:20, 18:491, 21:259, 21:307, 21:405, 21:490, 21:582, 21:668, 21:680, 21:684, 21:694, 21:710, 21:714, 21:776, 21:1268, 21:1285, 21:1289, 21:1301, 21:1305, 21:1315, 21:1331, 21:1335, 21:1379, 21:1380, 21:1381, 21:1433, 21:1434, 33:151, 33:171, 37:655, 37:672, 37:686, 37:701, 37:744, 54:875, 57:560

A

\A 57:252, 57:575, 57:596
\a 21:243, 41:1, 57:243, 57:576, 57:587, *1386*
A/foo (hook) *227*, *227*
\AA . . . 21:260, 21:449, 21:549, 02:318, *1398*
\aa . . . 21:265, 21:443, 21:559, 02:318, *1398*
\abovedisplayshortskip . . 38:624, 02:288
\abovedisplayskip 38:617, 38:619, 38:621, 38:622, 38:623, 38:624, 02:287, *1384*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\abspage *854*
abspage 36:255, *854*
\accent 21:91, 21:414, 21:444, 21:502, 21:787, *659*
\ActivateGenericHook 08:2782, 08:2784, 08:2789, 08:2791, *219*
\active 01:103, 37:505, 37:506, 37:507, 37:508, 37:515, 37:516, 37:654, 37:661, 37:662, 37:671, 37:677, 37:685, 37:691, 37:700, 37:742, 38:254, 38:269, 01:314, 02:10, 02:11, 50:1291, 50:1292, 50:1293, 50:1374, 50:1377, 50:1380, 50:1422, 50:1423, 50:1424, 50:1506, 50:1509, 50:1512, 50:1536, 50:1537, 50:1538, 50:1596, 50:1599, 50:1602, 54:835, 01:48, 02:332, 02:333, 02:341, 02:342, 02:354, 02:355, 02:359
\acute 30:527
add commands:
  add_to_callback 04:841
\add_to_callback *44*
\addcontentsline 44:70, 44:80, 44:159, 45:16, *1383*
\AddEverypageHook *1191*
\Adding *1359*
\addpenalty 18:307, 39:124, 39:201, 39:206, 44:50, 54:343, 54:1505, 54:1654, 54:1815, *476*
\AddThispageHook *1191*
\addtocontents 44:164, 44:171, 44:177, 44:188, *1383*
\addtocounter *556*
\addtocounter 22:6, 22:35, *1371*
\AddToDocumentProperties 17:27, *458*
\AddToHook 08:2794, 08:2944, 26:153, 37:44, 37:45, 37:46, 37:109, 37:110, 37:111, 37:398, 37:399, 37:400, 37:401, 47:72, 50:1171, 50:1172, 50:1173, 52:561, 52:563, 52:573, 52:574, 52:575, 52:576, 53:251, 53:547, 53:570, 55:201, *249*
\AddToHookNext 08:2805, 08:2942, 17:177, 52:562, 53:9, 53:571, *222*
\AddToHookNextWithArguments 08:2805, *222*
\AddToHookWithArguments . . . 08:2794, *237*
\addtolength *569*
\addtolength 23:16, 38:619, 38:621
\AddToNoCaseChangeList 57:620, *1426*
\addtoversion 27:20, 27:139
\addvspace 18:256, 37:431, 39:124, 39:202, 39:203, 39:207, 39:255, 44:50, *1370*
\adjdemerits 02:206
\AE 21:261, 21:419, 21:550, 21:792, 21:1155, 21:1465, 21:1469, 57:719
\ae 21:266, 21:422, 21:560, 21:796, 21:1161, 21:1467, 21:1471, 57:719
after (hook) *241*
\afterassignment 21:232, 21:240, 24:391, 06:360, 06:366, 38:199, 06:409, 40:336, 02:376, 02:379, *96*
\AfterEndEnvironment 37:394, *241*
\aftergroup 24:92, 24:410, 24:428, 24:433, 26:203, 26:269, 28:115, 28:122, 28:130, 28:476, 32:65, 37:658, 37:675, 37:689, 37:704, 38:538, 38:539, 39:149, 39:154, 40:192, 40:339, 54:884, 54:885, 54:959, 54:960, 54:1017, 54:1018, *586*
\AfterLastShipout 52:573
\afterpreamble *489*
\aleph 30:319
\alignmark *1440*
\allocationnumber 04:52, 04:53, 04:54, 04:91, 04:212, 16:81, 41:4, 41:9, 02:37, 53:95, 02:53, 02:65, 02:67, 57:58, 57:59, 57:60, 02:97, 02:98, 02:99, 02:118, 02:119, 02:157, 02:163, 02:169, 02:170, *451*
\allowbreak 38:40, 06:969, 06:970, 06:989, 06:991, 02:383
\Alph *556*
\Alph 22:307, *1189*
\alph *556*
\alph 22:306
\alpha 30:279
\amalg 30:390
\AmSfont *657*
\and *1012*
\and 44:14, 44:27
\angle 30:348
\approx 30:433
\arabic *556*
\arabic 22:207, 22:222, 22:260, 22:272, 22:303, 43:60, 53:397, *556*
\arccos 38:13
\arcsin 38:10
\arctan 38:16
\arg 38:26
array (env.) 41:168
\array 41:168, *1441*
\arraycolsep . . . 38:435, 38:436, 38:464, 38:465, 38:629, 38:630, 41:335, 41:415
\arrayrulewidth 41:401, 41:415, 41:423, 41:424, 41:436, 41:440, 41:443, 41:453, 41:455

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\arraystretch 41:238, 41:239, 41:261, 41:262, 41:419
\Arrowvert 30:580
\arrowvert 30:578
\asciispace 37:578, 37:580, 37:583, 37:597, 37:608, 37:609, 37:622, 37:623
\AssignSocketPlug . . . 10:192, 10:218, 35:138, 54:721, 54:728, 55:24, *359*
\AssignStructureRole *1310*
\AssignStructureRole 55:110, 55:111, 55:304
\AssignTaggingSocketPlug *1309*
\AssignTaggingSocketPlug 55:20, 55:61, 55:71, 55:82, 55:83, 55:91, 55:99, 55:105, 55:106, 55:116, 55:143, 55:144, 55:168, 55:169, 55:203, 55:204, 55:212, *1309*
\AssignTemplateKeys 11:538, 11:543, 11:1619, *373*
\ast 30:243, 30:406
\asymp 30:460
\AtBeginDocument 20:126, 20:181, 25:3257, 47:54, 50:1162, *668*
\atbegindocumenthook *489*
\AtBeginDvi . . . 53:465, 53:501, 54:62, *1191*
\AtBeginEnvironment 37:394, *241*
\AtBeginShipout 53:506, 53:570, *1191*
\AtBeginShipoutAddToBox . . . 53:575, *1190*
\AtBeginShipoutAddToBoxForeground 53:575, *1190*
\AtBeginShipoutBox 53:568, *1190*
\AtBeginShipoutDiscard 53:574, *1191*
\AtBeginShipoutFirst 53:509, 53:572, *1191*
\AtBeginShipoutInit 53:569, *1191*
\AtBeginShipoutNext . 53:507, 53:570, *1191*
\AtBeginShipoutOriginalShipout 53:583, *1190*
\AtBeginShipoutUpperLeft . . . 53:575, *1190*
\AtBeginShipoutUpperLeftForeground 53:575, *1190*
\AtEndAfterFileList 52:575
\AtEndDocument 37:146, 50:1162, 52:579, *243*
\AtEndDvi 53:516, 53:542, *1187*
\AtEndEnvironment 37:394, *241*
\AtEndOfClass 38:554, 50:1162, *1162*
\AtEndOfPackage 50:641, 50:660, 50:758, 50:769, 50:1162, 51:88, *1162*
\AtEndPreamble *242*
\AtNextShipout *1191*
\atopwithdelims 38:57, 38:58, 38:59
\attribute 04:79, *40*
\attributedef 04:79, 04:222
\attributezero 04:222
\AtVeryEndDocument 52:574
\AtVeryVeryEnd 52:576
\author *1012*
\author 44:8, 44:24, 44:32

B

\b 21:253, 21:410, 21:498, 21:783, 21:1276, *1386*
babel/⟨*language*⟩/afterextras (hook) *236*
\backslash 30:262, 30:601
\bar 30:531
\baselineskip 26:187, 26:188, 26:189, 26:191, 26:192, 30:521, 38:171, 38:172, 38:191, 38:197, 38:201, 40:459, 40:478, 40:712, 41:250, 41:273, 42:147, 42:325, 42:384, 53:283, 53:334, 54:247, 54:278, 54:907, 54:929, 54:977, 54:992, 54:1036, 54:1051, 02:284, 02:311, 02:374, 02:410, *1395*
\baselinestretch 24:382, 26:121, 26:122, 26:167, 26:168, 26:185, 26:246, *1357*
\baselinstretch *1377*
\batchmode 20:710, 20:741, 20:742, 27:106, 05:94, 05:100, 05:110, 05:144, 57:749, 57:770, *505*
\BCPdata 57:609, 57:636, 57:640
\BeforeBeginEnvironment 37:394, *241*
\BeforeClearDocument 52:577
\begin 09:82, 14:246, 14:248, 20:364, 20:416, 26:7, 30:4, 31:4, 37:244, 37:245, 37:315, 37:337, 37:362, 37:378, 37:386, 38:559, 38:571, 44:14, 44:17, 06:801, 50:779, 51:283, 53:447, 56:3, 07:1465, 07:1582, 07:2876, *875*
begindocument (hook) *327*, *354*, *354*, *489*, *489*, *1204*, *1208*, *239*, *239*, *239*, *242*, *250*
begindocument/before (hook) *489*, *242*
begindocument/end (hook) *242*
\begingroup *1357*
\belowdisplayshortskip . . 38:623, 02:290
\belowdisplayskip 38:622, 02:289
\beta 30:280
\bezier *974*
\bezier 42:692, 42:693, 42:815, 42:816, 42:831, 42:833
\bf *1365*
\bfdefault 29:16, 29:340, 29:346, 29:347, 29:348, 29:349, 29:389, 29:390, 29:391, 29:392, 29:401, 29:437, 29:461, 29:480, 29:521, 29:555, 30:104, 30:115, 30:117, 30:125, *746*
bfseries (hook) *244*, *244*
\bfseries 29:14, 29:15, 29:332, 29:333, 29:381, 29:386, 29:387,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

29:428, 29:431, 29:432, 29:457, 29:459, 29:460, 29:553, 29:554, 29:706, 29:707, 32:19, 35:17, 35:34, 35:51, 43:67, 43:69, 43:74, 43:76, 47:40, 53:448, 57:786, 57:787, *753*
bfseries 29:499
bfseries/defaults (hook) *244*, *244*
bfseries/defaults 29:499
\bgroup 02:325, *1395*
\bibcite 47:7, 47:9, 47:10, *1390*
\bibdata 47:45, 47:49
\bibitem 47:3
\bibliography *1053*
\bibliography 47:47
\bibliographystyle *1053*
\bibliographystyle 47:52
\bibstyle 47:45, 47:57
\Big 30:634, 30:637, 30:646, 30:648, 38:44, 38:45, 38:46
\big 30:635, 30:647, 38:41
\bigbreak 06:971, 06:992, 02:390
\bigcap 30:356
\bigcirc 30:403
\bigcup 30:357
\Bigg . . . 30:641, 30:650, 38:50, 38:51, 38:52
\bigg . . . 30:639, 30:649, 38:47, 38:48, 38:49
\Biggl 38:50
\biggl 38:47
\Biggm 38:51
\biggm 38:48
\Biggr 38:52
\biggr 38:49
\Bigl 38:44
\bigl 38:41
\Bigm 38:45
\bigm 38:42
\bigodot 30:364
\bigoplus 30:363
\bigotimes 30:362
\Bigr 38:46
\bigr 38:43
\bigskip 18:469, 02:395
\bigskipamount 18:471, 18:472, 45:391, 02:394
\bigsqcup 30:367
\bigtriangledown 30:372, 30:373
\bigtriangleup 30:371, 30:374
\biguplus 30:355
\bigvee 30:353
\bigwedge 30:354
\binoppenalty 02:193
block/list/label (tag socket) 55:218
block/recipe (tag socket) 55:219
\bmod 38:35
\boldmath 19:14, 29:724
bool commands:
\bool_gset_false:N . . 10:16, 36:31, 48:348, 53:76, 53:142, 53:151, 08:15
\bool_gset_true:N . . . 10:11, 36:33, 48:343, 53:71, 53:181, 53:391, 08:10
\bool_if:NTF 08:1945, 08:1954, 08:2045, 08:2054, 09:178, 09:182, 09:194, 09:253, 09:257, 09:269, 09:413, 10:22, 10:24, 11:317, 11:323, 11:465, 11:967, 11:1083, 11:1132, 11:1149, 11:1171, 36:71, 48:354, 51:24, 51:246, 53:82, 53:149, 53:417, 07:125, 07:134, 07:146, 07:147, 07:161, 07:173, 07:177, 07:186, 07:188, 07:198, 07:207, 07:208, 07:211, 07:216, 07:218, 07:322, 55:172, 55:180, 55:192, 55:195, 07:467, 07:476, 07:478, 07:529, 07:560, 07:575, 07:624, 07:665, 07:710, 07:720, 07:753, 07:761, 07:770, 07:780, 07:785, 07:812, 07:832, 07:914, 07:960, 07:961, 07:1031, 07:1044, 07:1061, 07:1081, 07:1759, 07:2203, 07:2214, 07:2464, 07:2796, 07:2837, 08:21, 08:334, *214*
\bool_if_exist:NTF 36:27
\bool_lazy_all:nTF 07:966
\bool_lazy_and:nnTF 08:1947, 08:2047, 08:2497, 08:2965, 53:127, 53:173, 53:428, 07:749, 07:766, 07:962, 07:2555, 08:269
\bool_lazy_and_p:nn . . 08:2500, 07:116
\bool_lazy_any:nTF 11:1034, 07:112, 07:487
\bool_lazy_or:nnTF 08:1898, 11:567, 11:1253, 07:77, 07:1425, 07:1434, 07:2619, 07:2641, 07:2826, 08:908
\bool_new:N 10:6, 11:78, 11:79, 36:28, 48:339, 51:57, 53:67, 07:19, 53:249, 53:264, 07:21, 07:23, 07:34, 07:35, 07:37, 07:38, 07:40, 07:43, 07:46, 07:47, 07:48, 07:49, 08:6, 08:24
\bool_set_false:N 08:1937, 08:2037, 11:368, 11:965, 11:1050, 51:72, 07:61, 07:87, 07:236, 07:258, 07:303, 55:104, 55:112, 07:453, 07:454, 07:455, 07:456, 07:457, 07:458, 07:479, 07:537, 07:567, 07:592, 07:606, 07:628, 07:629, 07:630, 07:649, 07:716, 07:759, 07:773, 07:774, 07:799, 07:800, 07:801, 07:803, 07:807, 07:809, 07:823, 07:979,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

07:980, 07:981, 07:982, 07:1601, 07:2089
\bool_set_true:N . 08:1934, 08:2034, 11:339, 11:381, 11:781, 11:1059, 51:69, 07:66, 07:237, 07:259, 07:292, 55:102, 55:114, 07:452, 07:477, 07:544, 07:551, 07:552, 07:566, 07:762, 07:763, 07:819, 07:839, 07:840, 07:846, 07:847, 07:853, 07:854, 07:861, 07:862, 07:863, 07:998, 07:1016, 07:1606, 07:2094
\bool_while_do:nn ... 08:1614, 08:1699
\c_false_bool ... 09:157, 09:167, 09:403, 07:2465, 07:2768, 07:3101, 07:3269, 08:391, 08:397, 08:405, 08:430, 08:478, *214*
\l_tmpa_bool ... *373*
\c_true_bool ... 09:162, 07:2466, 07:2770, 07:3102, 07:3267, 08:398, 08:406, 08:408, *214*
bool internal commands:
\g__mark_debug_bool ... 48:339, 48:343, 48:348, 48:354
\BooleanFalse 07:2082, 07:2458, 07:3101, *121*
\BooleanTrue 07:2081, 07:2457, 07:3101, *116*
\bordermatrix ... 38:185
\bot ... 30:332
\botfigrule ... 54:1093, 54:3026
\botmark ... 49:119, 54:1002, 54:1061, *1058*
\bottomfraction ... 45:275, 54:2995
\bowtie ... 30:493
\Box ... 29:882
\box ... *1234*
box commands:
\box_dp:N ... 53:255
\box_gclear:N ... 16:91
\box_gset_to_last:N 16:16, 16:49, 16:119
\box_ht:N ... 53:254, 53:363, *1196*
\box_if_empty:NTF 48:88, 53:139, 53:158
\box_if_horizontal:NTF 53:295, 53:347
\box_if_vertical:NTF ... 48:83, 48:418, 48:450, 53:269, 53:320
\box_move_up:nn ... 53:309, 53:363
\box_new:N ... 16:84, 48:63, 48:64, 53:84, 53:86, 53:248, 53:265
\box_set_dp:Nn 53:37, 53:57, 53:282, 53:291, 53:308, 53:333, 53:344, 53:362
\box_set_eq:NN . 53:210, 53:240, 53:245
\box_set_eq_drop:NN ... 53:154
\box_set_ht:Nn 53:36, 53:56, 53:281, 53:290, 53:307, 53:332, 53:343, 53:361
\box_set_to_last:N ... 48:80
\box_set_wd:Nn ... 53:280, 53:306, 53:331, 53:360
\box_use:N ... 53:38, 53:58, 53:187, 53:286, 53:311, 53:339, 53:364
\box_use_drop:N ... 16:86
\box_wd:N ... 53:256, 53:357, 53:365
box internal commands:
\l__mark_box .. 48:63, 48:76, 48:80, 48:83, 48:84, 48:87, 48:88, 48:98, *1072*
\l__mark_ii_box .. 48:63, 48:87, 48:97
\boxmaxdepth ... 42:485, 42:513, 42:543, 42:621, 42:638, 54:603, 54:742, 54:1075, 02:270
\brace ... 38:59
\braceld ... 30:564, 30:568, 30:569, 30:571, 30:573
\bracelu ... 30:566, 30:570, 30:572
\bracerd ... 30:565, 30:570, 30:572
\braceru ... 30:567, 30:569, 30:573
\bracevert ... 30:619
\brack ... 38:58
\break ... 18:116, 06:972, 06:993, 02:383, 02:388, *1391*
\breve ... 30:532
\brokenpenalty ... 24:765, 02:198
build/column/after (hook) .. 54:731, *1232*
build/column/baselineattach (socket) ... 54:723, *1233*, *1234*, *1246*, *1247*, *1250*, *1250*
build/column/before (hook) . 54:731, *1232*
build/column/footins (socket) ... *1234*
build/column/footins (tag socket) . 55:269
build/column/footnotes (socket) ... 54:722, *1234*
build/column/outputbox (socket) ... 54:663, *1233*, *1234*, *1246*
build/column/outputbox (tag socket) 55:268
build/page/after (hook) ... 54:729, *1232*
build/page/before (hook) ... 54:729, *1232*
build/page/footer (socket) ... *1234*
build/page/footer (tag socket) ... 55:266
build/page/header (socket) ... *1234*
build/page/header (tag socket) ... 55:266
build/page/reset (hook) ... 54:730, *1232*
\buildrel ... 30:480, 38:162
\bullet ... 30:392

C

\c ... 11:534, 21:254, 21:355, 21:357, 21:359, 21:361, 21:363, 21:365, 21:367, 21:369, 21:371, 21:392, 21:394, 21:413, 21:481, 21:501, 21:629, 21:631, 21:656, 21:658, 21:671, 21:697, 21:724, 21:727, 21:728, 21:729, 21:730, 21:731, 21:732, 21:733, 21:734, 21:735,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

21:786, 21:1278, 21:1292, 21:1318, 21:1375, 21:1376, 21:1395, 21:1396, 21:1399, 21:1400, 21:1405, 21:1406, 21:1417, 21:1418, 21:1425, 21:1426, 21:1429, 21:1430, 33:131, 33:160, *1407*
\cal 29:933
call commands:
call_callback 04:806
\call_callback *44*
callback commands:
callback_identifiers 04:997
callback.register 04:694
\callback_descriptions *44*
\callback_identifiers *44*
\cap 30:383
\capitalacute 21:863, 33:124, 33:125, 33:157, 33:654, 33:904, 33:1270
\capitalbreve 21:870, 33:126, 33:127, 33:158, 33:655, 33:911, 33:1277
\capitalcaron 21:869, 33:128, 33:129, 33:159, 33:656, 33:910, 33:1276
\capitalcedilla 21:856, 33:130, 33:131, 33:160, 33:657, 33:901, 33:1280
\capitalcircumflex . . . 21:864, 33:132, 33:133, 33:161, 33:658, 33:905, 33:1271
\capitaldieresis 21:866, 33:134, 33:135, 33:162, 33:659, 33:907, 33:1273
\capitaldotaccent . . . 21:872, 33:136, 33:137, 33:163, 33:660, 33:913, 33:1279
\capitalgrave 21:862, 33:138, 33:139, 33:164, 33:661, 33:903, 33:1269
\capitalhungarumlaut . 21:867, 33:140, 33:141, 33:165, 33:662, 33:908, 33:1274
\capitalmacron 21:871, 33:142, 33:143, 33:166, 33:663, 33:912, 33:1278
\capitalnewtie 21:876, 33:154, 33:155, 33:167, 33:664, 33:978, 33:979, 33:1285, *803*
\capitalogonek 21:859, 33:144, 33:145, 33:168, 33:665, 33:902, 33:1281
\capitalring 21:868, 33:146, 33:147, 33:169, 33:666, 33:909, 33:1275
\capitaltie 21:874, 33:148, 33:149, 33:170, 33:667, 33:974, 33:975, 33:1283
\capitaltilde 21:865, 33:150, 33:151, 33:171, 33:668, 33:906, 33:1272
\caption 45:4, *369*
caption/begin (tag socket) 55:262
caption/end (tag socket) 55:262
caption/label/begin (tag socket) . . 55:264
caption/label/end (tag socket) . . . 55:264
\cases 38:166, 38:167, 38:177, 38:179
\CaseSwitch 57:620, *1426*
\catcode *1442*
\catcodetable 04:89, 04:109, *40*
\catcodetable@atletter *41*
\catcodetable@initex *41*
\catcodetable@latex *41*
\catcodetable@string *41*
\catcoding *1361*
\cdot 30:405
\cdotp 30:513, 30:519
\cdots 30:519
center (env.) 37:446
\center 37:446
\centering 37:446, 37:451, 37:452, 37:470, 37:472, 37:487, 37:489
\centerline 40:725
\changes 29:794, 35:201, *76*
\chapter *117*
\chaptermark *1064*
\char . . . 21:412, 21:415, 21:451, 21:454, 21:465, 21:472, 21:500, 21:504, 21:509, 21:512, 21:514, 21:516, 21:757, 21:785, 21:788, 21:821, 21:828, 21:835, 21:858, 21:861, 21:890, 21:920, 21:1030, 21:1069, 21:1193, 21:1195, 21:1197, 21:1244, 29:737, 29:744, 33:595, 33:602, 33:604, 33:724, 37:578, 37:745, 38:251, 42:243, 42:293, 42:307, 42:315, 42:318, 42:459, 42:573, 42:578, 42:586, 42:590, 42:626, 42:627, 42:629, 42:642, 42:643, 42:646, 42:673, 06:934, 06:949, *1394*
char commands:
\char_generate:nn 05:159, 07:1911, 07:2223
\char_set_catcode_active:N . 07:2489
\char_set_catcode_active:n 07:2201, 07:2202, 07:2666
\char_set_catcode_escape:N . . 09:477
\char_set_catcode_group_begin:N 09:478
\char_set_catcode_group_end:N 09:479
\char_set_catcode_other:N 07:1616, 07:1622, 07:1658, 07:1707, 07:1746, 07:1810, 07:1832, 07:1834, 07:2199, 07:2212
\char_set_catcode_other:n 07:1618, 07:2204, 07:2215
\char_set_catcode_parameter:N 09:480
\char_set_catcode_parameter:n 07:2205, 07:2216
\char_set_catcode_space:n 09:415, 07:1836
\char_set_lccode:nn . 07:2494, 07:2676
\char_value_catcode:n 09:408

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\chardef 04:22, 04:26, 04:38, 04:47, 04:48, 04:89, 12:2, 04:157, 04:223, 20:52, 20:122, 21:36, 24:15, 41:4, 41:9, 02:10, 02:16, 02:17, 02:18, 02:19, 02:20, 50:1297, 50:1428, 50:1545, 02:54, 02:60, 02:62, 02:69, 02:75, 02:78, 02:80, 02:86, 02:87, 02:90, 02:92, 57:42, 57:44, 57:48, 57:67, 57:171, 57:172, 57:173, 57:174, 57:175, 57:176, 57:177, 02:117, 02:143, 02:145, 02:169, 01:48, 01:54, 01:55, 02:404, 02:405, 02:406, *1367*
\charsubdef 57:402
\charzero 04:223
\check 30:533
\CheckCommand 06:228, *1378*
\CheckEncodingSubset 24:204, 33:16, 33:73, 33:74, 33:75, 33:120, 33:122, 33:316, 33:585, 33:794, 33:843, 33:899, 33:900, 33:968, 33:1085, 33:1088, 33:1102, *799*
\chi 30:299
\choose 38:57
\circ 30:402
\circle 42:461, 42:615, 42:817, 42:834, *1361*
\citation 47:11, 47:39, 47:67, 47:84
\cite *1053*
\cite 47:12, *1054*
\clap 40:729, *1415*
class (hook) *236*
class/ (hook) *1162*, *1169*
class/.../after *1161*
class/.../before *1161*
class/⟨*name*⟩/after (hook) *1161*
class/⟨*name*⟩/before (hook) *1161*
class/after (hook) *1161*
class/after *1161*
class/before (hook) *1161*
class/before *1161*
\ClassError 14:84
\ClassInfo 14:84
\ClassNote 14:136, *1422*
\ClassNoteNoLine 14:136
\ClassWarning 14:84
\ClassWarningNoLine 14:84
\cleaders 30:559, 30:562, 02:424
\cleardoublepage 54:117
\ClearHookNext 08:2816, *222*
\ClearHookRule 08:2853, 08:2958, *226*
\clearpage 20:316, 20:343, 20:347, 20:378, 20:401, 20:404, 20:425, 20:443, 37:17, 37:85, 37:149, 37:242, 54:103, 54:117, 54:122, 54:205, 54:412, 54:415, 54:419, 54:460, 54:466, 54:2847, 54:2864, *1162*
\cline 41:444, *1319*
clist commands:
\clist_clear:N . . 51:65, 51:166, 51:186
\clist_gclear:N 08:1613, 08:1698
\clist_gput_left:Nn . 08:1552, 08:1517
\clist_gput_right:Nn 08:1556, 08:1519
\clist_if_empty:NTF 08:1964, 08:2065, 11:689, 11:718, 51:75
\clist_if_exist:NTF 08:115
\clist_if_in:NnTF . 11:357, 11:725, 51:51, 51:52, 51:128, 51:151, 51:153
\clist_map_function:nN 36:59
\clist_map_inline:Nn 11:691, 51:77, 51:109, 51:140, 51:175, 51:189, 51:234
\clist_map_inline:nn 11:903, 51:63, 51:269, 57:705, 57:710, 57:715, 08:939, 08:948
\clist_map_variable:NNn 51:70
\clist_map_variable:nNn 11:909
\clist_new:N 11:92, 51:53, 51:54, 51:55, 51:56, 08:117, 08:134
\clist_put_right:Nn 11:784, 11:927, 11:939, 51:33, 51:122, 51:129, 51:136, 51:161, 51:177, 51:185, 51:201
\clist_remove_all:Nn 11:727, 51:35, 51:135, 51:162, 51:202
\clist_set:Nn 11:680
\clist_set_eq:NN 51:171
\clist_use:Nn 08:1966, 08:2067
\l_tmpa_clist *373*
\clubpenalty 20:8, 20:25, 20:95, 20:152, 24:763, 39:128, 39:225, 39:227, 44:100, 44:106, 44:130, 44:135, 02:195, *450*
\clubsuit 30:342
cmd (hook) *315*, *330*, *331*, *230*, *236*, *236*, *269*, *274*
cmd commands:
\cmd_arg_spec:N 07:2803, 07:2803, 07:2821
cmd internal commands:
__cmd_add_arg:n 07:1757, 07:1826, 07:1858, 07:1863, 07:1864, 07:1898, 07:1986, 07:1993, 07:2068, 07:2081, 07:2082, 07:2156, 07:2229, 07:2236, 07:2280, 07:2280, 07:2285, *173*
__cmd_add_arg_spec:n . . . 07:590, 07:604, 07:672, 07:747, 07:747, 07:793
__cmd_add_arg_spec_mandatory:n 07:636, 07:656, 07:747, 07:776
__cmd_add_arg_spec_mandatory:nn 07:650, 07:747, 07:777, 07:778

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

__cmd_add_default: 07:875, 07:913, 07:930, 07:992, 07:995, 07:1002, 07:1012, 07:1079, 07:1088, *153*
__cmd_add_default:n 07:882, 07:922, 07:938, 07:992, 07:992, 07:1009, 07:1023, *153*
__cmd_add_default_E:nn 07:890, 07:992, 07:1007, 07:1057
__cmd_add_expandable_grabber:nn 07:1036, 07:1049, 07:1060, 07:1080, 07:1090, 07:1097, 07:1097
__cmd_add_expandable_type_+:w 07:1014
__cmd_add_expandable_type_D:w 07:1019, 07:1019
__cmd_add_expandable_type_D_-aux:NN . . . 07:1019, 07:1025, 07:1042
__cmd_add_expandable_type_D_-aux:NNN . . 07:1019, 07:1026, 07:1029
__cmd_add_expandable_type_D_-aux:NNNn 07:1019, 07:1020, 07:1021, 07:1085
__cmd_add_expandable_type_E:w 07:1055, 07:1055
__cmd_add_expandable_type_E_-aux:n 07:1055, 07:1059, 07:1071
__cmd_add_expandable_type_m:w 07:1077, 07:1077
__cmd_add_expandable_type_R:w 07:1084, 07:1084
__cmd_add_expandable_type_t:w 07:1086, 07:1086
__cmd_add_grabber:N 07:876, 07:883, 07:896, 07:915, 07:923, 07:931, 07:939, 07:953, 07:953, *155*
__cmd_add_type_!:w 07:843
__cmd_add_type_+:w 07:836
__cmd_add_type_=:w 07:858
__cmd_add_type_>:w 07:850
__cmd_add_type_b:w . . 07:868, 07:868
__cmd_add_type_b_or_c:N 07:868, 07:869, 07:871, 07:872
__cmd_add_type_c:w . . 07:868, 07:870
__cmd_add_type_D:w . . 07:879, 07:879
__cmd_add_type_E:w . . 07:887, 07:887
__cmd_add_type_m:w . . 07:911, 07:911
__cmd_add_type_R:w . . 07:919, 07:919
__cmd_add_type_t:w . . 07:927, 07:927
__cmd_add_type_v:w . . 07:935, 07:935
__cmd_all_m_check:n 07:100, 07:100, 07:120
__cmd_all_m_check_aux:n 07:100, 07:101, 07:102
__cmd_allowed_token_check:N 07:588, 07:600, 07:621, 07:646, 07:689, 07:689
__cmd_arg_spec_opt:N 07:2803, 07:2812, 07:2822
\l__cmd_arg_spec_tl 07:12, 07:94, 07:451, 07:535, 07:568, 07:622, 07:764, *148*
__cmd_arg_to_keyvalue:nn 07:865, 07:2548, 07:2548
__cmd_arg_to_keyvalue_auxi:nnn 07:2548, 07:2562, 07:2565
__cmd_arg_to_keyvalue_auxii:Nnnn 07:2548, 07:2568, 07:2571
__cmd_arg_to_keyvalue_auxiii:nnn 07:2548, 07:2574, 07:2577
__cmd_arg_to_keyvalue_auxiv:Nnnn 07:2548, 07:2580, 07:2583
__cmd_arg_to_keyvalue_auxv:nn 07:2548, 07:2569, 07:2575, 07:2581, 07:2587, 07:2589
__cmd_arg_to_keyvalue_braces:nnn 07:2548, 07:2550, 07:2553
__cmd_arg_to_keyvalue_loop:w 07:2548, 07:2591, 07:2594, 07:2606, 07:2608, 07:2623, 07:2644
__cmd_arg_to_keyvalue_loop_-group:n . . 07:2548, 07:2600, 07:2605
__cmd_arg_to_keyvalue_loop_N_-type:N . . . 07:2548, 07:2597, 07:2609
__cmd_arg_to_keyvalue_loop_-space:w . . 07:2548, 07:2601, 07:2607
__cmd_arg_to_keyvalue_math:w 07:2548, 07:2622, 07:2626, 07:2645, 07:2648, 07:2650
__cmd_arg_to_keyvalue_math_-group:n . . 07:2548, 07:2632, 07:2647
__cmd_arg_to_keyvalue_math_N_-type:N . . . 07:2548, 07:2629, 07:2637
__cmd_arg_to_keyvalue_math_-space:w . . 07:2548, 07:2633, 07:2649
__cmd_arg_to_keyvalue_set_-default:nn 07:2548, 07:2612, 07:2640, 07:2651, *199*
__cmd_arg_to_keyvalue_set_-keyvalue:nn 07:2548, 07:2616, 07:2653, *199*
\l__cmd_args_i_tl . . 07:14, 07:331, 07:337, 07:342, 07:343, 07:345, *130*
\l__cmd_args_ii_tl 07:15, 07:341, 07:343, 07:345, 07:382, 07:387, 07:394, *130*
__cmd_args_process: 07:321, 07:380, 07:380, *154*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

__cmd_args_process_aux:n 07:380, 07:393, 07:397
__cmd_args_process_loop:nn 07:380, 07:386, 07:389
\l__cmd_args_tl . . . 07:13, 07:284, 07:309, 07:326, 07:331, 07:337, 07:356, 07:384, 07:387, 07:400, 07:401, 07:1584, 07:1585, 07:1590, 07:1593, 07:1964, 07:1998, 07:2282, *174*
__cmd_bad_arg_spec:wn . . . 07:507, 07:512, 07:521, 07:528, 07:574, 07:587, 07:597, 07:598, 07:603, 07:614, 07:620, 07:641, 07:741, 07:741
__cmd_bad_def:wn 07:465, 07:473, 07:502, 07:533, 07:564, 07:579, 07:669, 07:678, 07:686, 07:705, 07:714, 07:726, 07:741, 07:746, 07:757, 07:789, *162*
__cmd_bool_reverse:N 07:2462, 07:2462, 07:3316
__cmd_break_point:n 07:97, 07:99, 07:99, 07:741, 07:746, 07:1147, 07:1154, 07:1365
__cmd_cant_copy:nwn 07:1144, 07:1154, 07:1154, 07:1270, 07:1333, 07:1363
__cmd_check_definable:nNTF 07:2663, 07:2663, 07:3105, 07:3118, 07:3131, 07:3136, 07:3224, 07:3237, 07:3250, 07:3258
__cmd_check_definable_aux:nN 07:2663, 07:2664, 07:2667, *202*
__cmd_check_end:n 07:1329, 07:1331, 07:1337
__cmd_check_end:Nn 07:1316, 07:1317, 07:1329, 07:1329, 07:1481
__cmd_check_end:w 07:1329, 07:1339, 07:1342
__cmd_chk_if_free_cs:N 07:3321, 07:3322
__cmd_cmd_if_xparse:NTF *1421*
__cmd_cmd_if_xparse_aux:N . 07:2730
__cmd_cmd_type_cases:NnnnnnTF 07:1138, 07:1357, 07:2730, 07:2730, 07:2753, *168*
__cmd_cmd_type_cases:NnnnnTF . *1422*
__cmd_copy:NN 07:1131, 07:1133, 07:1133, 07:1168, 07:1183, 07:1229, 07:1347, *163*
__cmd_copy_command:nnNN 07:1139, 07:1169, 07:1169, *163*
__cmd_copy_command:NnNNnnnn 07:1169, 07:1174, 07:1176, *163*
__cmd_copy_environment:nnNN 07:1142, 07:1303, 07:1303
__cmd_copy_environment:Nnnnnnn 07:1303, 07:1309, 07:1312
__cmd_copy_environment_end:nnNN 07:1143, 07:1314, 07:1314
__cmd_copy_environment_end_-aux:nnNN . 07:1314, 07:1318, 07:1321
__cmd_copy_expandable:nnN 07:1248, 07:1253, 07:1256, 07:1283, 07:1293, 07:1299, 07:1301
__cmd_copy_expandable:nnNN 07:1140, 07:1182, 07:1186, 07:1188, 07:1201, 07:1203, 07:1214
__cmd_copy_expandable:NnNNNNnnn 07:1182, 07:1198, 07:1211, 07:1231, *163*
__cmd_copy_expandable_signature:NnNNNNnnn 07:1194, 07:1209, 07:1248, 07:1248, *163*
__cmd_copy_grabber_⟨*type*⟩:w . . . *165*
__cmd_copy_grabber_D:w 07:1273, 07:1273, 07:1286, 07:1287
__cmd_copy_grabber_D_alt:w 07:1273, 07:1285, 07:1288
__cmd_copy_grabber_E:w 07:1273, 07:1289, 07:1295
__cmd_copy_grabber_E_long:w 07:1273, 07:1295
__cmd_copy_grabber_m:w 07:1273, 07:1301, 07:1302
__cmd_copy_grabber_m_long:w 07:1273, 07:1302
__cmd_copy_grabber_R:w 07:1273, 07:1287
__cmd_copy_grabber_R_alt:w 07:1273, 07:1288
__cmd_copy_grabber_t:w 07:1273, 07:1296
__cmd_copy_optimized:nnNN 07:1141, 07:1217, 07:1217
__cmd_copy_parse_grabber:w 07:1248, 07:1260, 07:1264, *165*
\l__cmd_current_arg_int 07:16, 07:28, 07:150, 07:158, 07:187, 07:194, 07:274, 07:355, 07:359, 07:363, 07:364, 07:448, 07:461, 07:484, 07:536, 07:582, 07:607, 07:792, 07:797, 07:798, 07:971, 07:997, 07:1004, 07:1251, 07:1259, 07:1277, 07:1281, 07:1282, 07:1292, 07:1490, 07:1556, 07:1565, *166*
__cmd_declare_cmd:Nnn 07:59, 07:59, 07:3113, 07:3121, 07:3132, 07:3137

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

__cmd_declare_cmd_aux:Nnn 07:59, 07:62, 07:67, 07:69
__cmd_declare_cmd_code:Nnn 07:95, 07:110, 07:110
__cmd_declare_cmd_code_aux:Nnn 07:110, 07:127, 07:154
__cmd_declare_cmd_code_expandable:Nnn 07:110, 07:126, 07:182
__cmd_declare_cmd_internal:Nnnn 07:59, 07:88, 07:90, 07:267, *138*
__cmd_declare_cmd_optimized:Nnn 07:110, 07:129, 07:132
__cmd_declare_env:nnnn 07:228, 07:228, 07:230, 07:240, 07:242, 07:244, 07:3155, 07:3170, 07:3176, 07:3186, 07:3201, 07:3207, 07:3211, 07:3213, *212*
__cmd_declare_env_internal:nnnn 07:228, 07:238, 07:260, 07:265
__cmd_declare_expandable_-cmd:Nnn 07:59, 07:64, 07:3232, 07:3240, 07:3253, 07:3259
__cmd_defaults: 07:320, 07:328, 07:328
__cmd_defaults_aux: 07:328, 07:332, 07:333, 07:334, 07:339
\l__cmd_defaults_bool 07:19, 07:173, 07:188, 07:211, 07:803, 07:998, *130*
__cmd_defaults_def: 07:328, 07:330, 07:352
__cmd_defaults_def:nn 07:328, 07:357, 07:361
__cmd_defaults_def:nnn 07:328, 07:364, 07:366, *158*
__cmd_defaults_error:w 07:328, 07:335, 07:347
\l__cmd_defaults_tl 07:20, 07:174, 07:195, 07:312, 07:320, 07:356, 07:804, 07:999, 07:1005, *130*
__cmd_delimiter_check:nnn 07:635, 07:648, 07:730, 07:730
__cmd_end_expandable:NNw 07:407, 07:409, 07:409
__cmd_end_expandable_aux:nNNNN 07:409, 07:412, 07:413
__cmd_end_expandable_aux:w 07:409, 07:410, 07:411
__cmd_end_expandable_defaults:nnnNNn 07:409, 07:416, 07:425, 07:434, 07:444, 07:445, *142*
__cmd_end_expandable_defaults:nnw 07:409, 07:433, 07:437
__cmd_end_expandable_defaults:nw 07:409, 07:440, 07:441, 07:443
\l__cmd_environment_bool 07:21, 07:87, 07:114, 07:161, 07:237, 07:259, 07:292, 07:303, 07:322, 07:478, 07:665, 07:2837, *130*
__cmd_environment_or_command: . . . 07:350, 07:464, 07:496, 07:500, 07:578, 07:668, 07:676, 07:685, 07:700, 07:744, 07:783, 07:2066, 07:2226, 07:2233, 07:2835, 07:2835
\l__cmd_environment_str 07:22, 07:164, 07:232, 07:246, 07:247, 07:248, 07:249, 07:252, 07:256, 07:261, 07:291, 07:294, 07:295, 07:323, 07:2838, *131*
\l__cmd_expandable_aux_name_tl 07:24, 07:25, 07:1034, 07:1038, 07:1047, 07:1051, *131*
\l__cmd_expandable_bool 07:23, 07:61, 07:66, 07:134, 07:146, 07:186, 07:198, 07:236, 07:258, 07:467, 07:476, 07:529, 07:560, 07:710, 07:720, 07:753, 07:812, *131*
__cmd_expandable_grab_D:nnNNNwNN 07:2286, 07:2306, 07:2312
__cmd_expandable_grab_D:NNNwNNn 07:2286, 07:2287, 07:2288
__cmd_expandable_grab_D:NNNwNNnnn 07:2286, 07:2299, 07:2304, 07:2330, 07:2419, *192*
__cmd_expandable_grab_D:Nw 07:2286, 07:2308, 07:2311
__cmd_expandable_grab_D:w 07:2286, 07:2286
__cmd_expandable_grab_D_-alt:NNwn . 07:2347, 07:2352, 07:2442
__cmd_expandable_grab_D_-alt:NNwNNn 07:2334, 07:2335, 07:2336
__cmd_expandable_grab_D_alt:Nwn 07:2334
__cmd_expandable_grab_D_alt:w 07:2334, 07:2334
__cmd_expandable_grab_E:w 07:2365, 07:2365
__cmd_expandable_grab_E_aux:w 07:2365, 07:2366, 07:2368, 07:2369, 07:2397
__cmd_expandable_grab_E_end:nnw 07:2365, 07:2381, 07:2398
__cmd_expandable_grab_E_-find:nnw . 07:2365, 07:2395, 07:2396
__cmd_expandable_grab_E_find:w 07:2365, 07:2386, 07:2394
__cmd_expandable_grab_E_long:w 07:2365, 07:2367

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

__cmd_expandable_grab_E_-loop:nnnNNw 07:2365, 07:2373, 07:2377, 07:2388
__cmd_expandable_grab_E_-test:nnw . 07:2365, 07:2370, 07:2371
__cmd_expandable_grab_m:w 07:2400, 07:2400, *166*
__cmd_expandable_grab_m_aux:wNn .. 07:2400, 07:2401, 07:2403, 07:2404
__cmd_expandable_grab_m_long:w 07:2400, 07:2402
__cmd_expandable_grab_R:w 07:2406, 07:2406
__cmd_expandable_grab_R_alt:w 07:2429, 07:2429
__cmd_expandable_grab_R_alt_-aux:NNwNNn 07:2429, 07:2430, 07:2431
__cmd_expandable_grab_R_-aux:NNNwNNn 07:2406, 07:2407, 07:2408
__cmd_expandable_grab_t:w 07:2452, 07:2452
__cmd_expandable_grab_t_-aux:NNwn . 07:2452, 07:2453, 07:2454
\l__cmd_final_verb_bool 07:49, 07:809, 07:819, 07:968, *133*
__cmd_flush_m_args: 07:812, 07:838, 07:845, 07:852, 07:860, 07:874, 07:881, 07:889, 07:921, 07:929, 07:937, 07:942, 07:942, *153*
\l__cmd_fn_code_tl 07:32, 07:311, 07:326, *131*
\l__cmd_fn_tl 07:31, 07:123, 07:310, 07:1586, 07:1852, 07:1876, 07:1882, 07:1892, 07:1902, 07:1911, 07:1916, 07:1920, 07:1951, 07:1977, 07:1985, 07:1987, 07:1992, 07:1994, 07:2005, 07:2006, 07:2011, 07:2012, 07:2017, 07:2018, 07:2023, 07:2024, 07:2029, 07:2030, 07:2035, 07:2036, 07:2041, 07:2042, 07:2047, 07:2048, 07:2078, 07:2084, *173*
\l__cmd_function_tl 07:27, 07:33, 07:92, 07:124, 07:140, 07:143, 07:157, 07:168, 07:169, 07:185, 07:193, 07:203, 07:206, 07:210, 07:212, 07:220, 07:226, 07:472, 07:532, 07:563, 07:713, 07:725, 07:756, 07:2841, *131*
__cmd_get_grabber:NN 07:1073, 07:1089, 07:1102, 07:1102, *160*
__cmd_get_grabber_auxi:NN 07:1102, 07:1105, 07:1108, 07:1116
__cmd_get_grabber_auxii:NN 07:1102, 07:1123, 07:1126
__cmd_grab_b:w 07:1572, 07:1572
__cmd_grab_b_aux:NNw 07:1572, 07:1573, 07:1575, 07:1577, 07:1579, 07:1580
__cmd_grab_b_end:Nw 07:1572, 07:1583, 07:1588
__cmd_grab_b_long:w 07:1572, 07:1574
__cmd_grab_b_long_obey_spaces:w 07:1572, 07:1578
__cmd_grab_b_obey_spaces:w 07:1572, 07:1576
__cmd_grab_c:w 07:1599, 07:1599
__cmd_grab_c_auxi:w 07:1638, 07:1643, 07:1645
__cmd_grab_c_auxii:w 07:1643, 07:1655, 07:1661
__cmd_grab_c_auxiii:N 07:1643, 07:1667, 07:1669, 07:1678
__cmd_grab_c_auxiv: 07:1643, 07:1674, 07:1692, 07:1715
__cmd_grab_c_auxv: 07:1643, 07:1688, 07:1701, 07:1720, 07:1742
__cmd_grab_c_auxvi:N 07:1643, 07:1683, 07:1708, 07:1730
__cmd_grab_c_auxvii: .. 07:1643, 07:1714, 07:1719, 07:1734, 07:1741
__cmd_grab_c_auxviii: 07:1643, 07:1725, 07:1736
__cmd_grab_c_end:n 07:1745, 07:1761, 07:1767
__cmd_grab_c_end:w 07:1739, 07:1745, 07:1747
__cmd_grab_c_end_auxi:w 07:1745, 07:1769, 07:1772
__cmd_grab_c_end_auxii:w 07:1745, 07:1773, 07:1774
__cmd_grab_c_end_auxiii:w 07:1745, 07:1775, 07:1776
__cmd_grab_c_first:w 07:1599, 07:1619, 07:1623
__cmd_grab_c_loop:w 07:1599, 07:1627, 07:1633, 07:1636, 07:1653, 07:1699, 07:1704
__cmd_grab_c_obey_spaces:w 07:1599, 07:1604
__cmd_grab_c_start:n 07:1599, 07:1602, 07:1607, 07:1609
__cmd_grab_D:w 07:1778
__cmd_grab_D_aux:NNnNN . 07:1582, 07:1809, 07:1813, 07:1849, 07:2061
__cmd_grab_D_aux:NNnNNNN 07:1790, 07:1809, 07:1811
__cmd_grab_D_call:Nw 07:1822, 07:1906, 07:1906, 07:2063, *178*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

__cmd_grab_D_long:w 07:1778
__cmd_grab_D_long_no_strip:w 07:1778
__cmd_grab_D_long_obey_spaces:w 07:1778
__cmd_grab_D_long_obey_spaces_no_strip:w 07:1778
__cmd_grab_D_nested:NNnN 07:1855, 07:1869, 07:1872
__cmd_grab_D_nested:w 07:1869, 07:1887, 07:1904
__cmd_grab_D_no_strip:w . . . 07:1778
__cmd_grab_D_obey_spaces:w 07:1778
__cmd_grab_D_obey_spaces_no_strip:w 07:1778
__cmd_grab_D_verb_safe:NN 07:1809, 07:1817, 07:1830
__cmd_grab_E:nnNN 07:1925, 07:1927, 07:1933, 07:1939, 07:1945, 07:1949
__cmd_grab_E:w 07:1925, 07:1925
__cmd_grab_E_finalize: 07:1925, 07:1958, 07:1974, 07:1981
__cmd_grab_E_long:w 07:1925, 07:1931
__cmd_grab_E_long_obey_spaces:w 07:1925, 07:1943
__cmd_grab_E_loop:NnN . . 07:1925, 07:1954, 07:1969, 07:1971, 07:1978
__cmd_grab_E_obey_spaces:w 07:1925, 07:1937
\l__cmd_grab_expandably_bool 07:34, 07:125, 07:452, 07:477, 07:479, 07:537, 07:567, 07:592, 07:606, 07:628, 07:649, 07:716, 07:759, 07:832, *144*
__cmd_grab_m:w 07:1982, 07:1982
__cmd_grab_m_1:w 07:1996
__cmd_grab_m_2:w 07:1996
__cmd_grab_m_3:w 07:1996
__cmd_grab_m_4:w 07:1996
__cmd_grab_m_5:w 07:1996
__cmd_grab_m_6:w 07:1996
__cmd_grab_m_7:w 07:1996
__cmd_grab_m_8:w 07:1996
__cmd_grab_m_9:w 07:1996
__cmd_grab_m_aux:Nnnnnnnnn 07:1996, 07:1996, 07:2005, 07:2011, 07:2017, 07:2023, 07:2029, 07:2035, 07:2041, 07:2047
__cmd_grab_m_long:w 07:1982, 07:1989
__cmd_grab_R:w 07:2055, 07:2055
__cmd_grab_R_aux:NNnN 07:2055, 07:2056, 07:2058, 07:2059
__cmd_grab_R_long:w 07:2055, 07:2057
__cmd_grab_t:w 07:2071, 07:2071
__cmd_grab_t_aux:NNw 07:2071, 07:2072, 07:2074, 07:2075
__cmd_grab_t_obey_spaces:w 07:2071, 07:2073
__cmd_grab_v:w 07:2087, 07:2087
__cmd_grab_v_aux:w 07:2087, 07:2090, 07:2095, 07:2097, *188*
__cmd_grab_v_aux_abort:n 07:2114, 07:2132, 07:2138, 07:2151, 07:2170, 07:2193, 07:2195, 07:2219, *187*
__cmd_grab_v_aux_catcodes: 07:2129, 07:2161, 07:2195, 07:2195, 07:2197, 07:2208, 07:2210, *186*
__cmd_grab_v_aux_loop:N 07:2124, 07:2130, 07:2134, 07:2148
__cmd_grab_v_aux_loop:NN 07:2124, 07:2137, 07:2140
__cmd_grab_v_aux_loop_end: 07:2124, 07:2145, 07:2153, 07:2184
__cmd_grab_v_aux_put:N 07:2128, 07:2147, 07:2163, 07:2181, 07:2189, 07:2239, 07:2239, 07:2241, 07:2251, 07:2253, 07:2267, 07:2269
__cmd_grab_v_aux_test:N 07:2113, 07:2124, 07:2124
__cmd_grab_v_bgroup: 07:2109, 07:2159, 07:2159
__cmd_grab_v_bgroup_loop: 07:2159, 07:2164, 07:2166, 07:2182, 07:2190
__cmd_grab_v_bgroup_loop:N 07:2159, 07:2169, 07:2172
__cmd_grab_v_group_end: 07:2087, 07:2118, 07:2155, 07:2221, *186*
__cmd_grab_v_long:w 07:2087, 07:2092
__cmd_grab_v_token_if_char:NTF 07:2126, 07:2142, 07:2174, 07:2278, 07:2278, *187*
\g__cmd_grabber_int 07:30, 07:1115, 07:1119, *131*
__cmd_if_novalue_aux:w . 07:3283, *215*
__cmd_if_recursion_tail_stop_do:Nn . 07:50, 07:52, 07:2611, 07:2639
\c__cmd_ignore_def_tl . . 07:2833, 07:2849, 07:2856, 07:2870, 07:2903, 07:2932, 07:2939, 07:2946, 07:2954, 07:2962, 07:2971, 07:2978, 07:2985, 07:2992, 07:2999, 07:3011, 07:3018, *205*
\l__cmd_last_delimiters_tl 07:36, 07:450, 07:469, 07:591, 07:605, 07:627, 07:671, 07:722, 07:732, 07:791, *132*
\l__cmd_last_mandatory_arg_int 07:18, 07:449, 07:792, 07:972, *130*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\l__cmd_long_bool . . 07:37, 07:454, 07:543, 07:544, 07:630, 07:750, 07:761, 07:767, 07:773, 07:799, 07:839, 07:960, 07:979, 07:1016, 07:1031, 07:1044, 07:1061, 07:1081, 07:2089, 07:2094, 07:2203, 07:2214, *159*
\l__cmd_m_args_int 07:39, 07:802, 07:916, 07:944, 07:947, 07:949, 07:951, *132*
\l__cmd_nesting_a_tl 07:1869
\l__cmd_nesting_b_tl 07:1869
__cmd_new_env:nnnn 07:3139, 07:3143, 07:3157, 07:3188, 07:3214, 07:3215
__cmd_normalize_arg_spec:n 07:93, 07:446, 07:446
__cmd_normalize_arg_spec_loop:n 07:446, 07:459, 07:481, 07:538, 07:583, 07:593, 07:608, 07:631, 07:637, 07:651, 07:657
__cmd_normalize_check_gv:N 07:655, 07:708, 07:708
__cmd_normalize_check_lu:N 07:708, 07:718
__cmd_normalize_E_unique_-check:w 07:585, 07:601, 07:610, 07:615
__cmd_normalize_type_!:w . . . 07:526
__cmd_normalize_type_+:w . . . 07:526
__cmd_normalize_type_=:w . . . 07:526
__cmd_normalize_type_>:w . . . 07:526
__cmd_normalize_type_aux:NnNn . . 07:526, 07:542, 07:548, 07:557, 07:572
__cmd_normalize_type_b:w 07:659, 07:659
__cmd_normalize_type_b_or_c:nn 07:659, 07:660, 07:662, 07:663
__cmd_normalize_type_c:w 07:659, 07:661
__cmd_normalize_type_D:w 07:508, 07:516, 07:518, 07:585, 07:585
__cmd_normalize_type_d:w 07:505, 07:505
__cmd_normalize_type_E:w 07:513, 07:585, 07:595
__cmd_normalize_type_e:w 07:505, 07:510
__cmd_normalize_type_m:w 07:633, 07:633
__cmd_normalize_type_O:w 07:505, 07:517
__cmd_normalize_type_o:w 07:505, 07:515
__cmd_normalize_type_R:w 07:633, 07:639
__cmd_normalize_type_r:w 07:505, 07:519
__cmd_normalize_type_R_aux:w 07:522, 07:633, 07:642, 07:644
__cmd_normalize_type_s:w 07:505, 07:524
__cmd_normalize_type_t:w 07:525, 07:585, 07:618
__cmd_normalize_type_v:w 07:633, 07:653
\l__cmd_obey_spaces_bool 07:35, 07:453, 07:549, 07:551, 07:624, 07:629, 07:770, 07:774, 07:785, 07:800, 07:846, 07:961, 07:980, 07:1601, 07:1606, 07:1759, *131*
__cmd_peek_cs_check_equal:NNN 07:2767, 07:2781, 07:2787
__cmd_peek_meaning:NTF 07:2758, 07:2767, 07:2767
__cmd_peek_meaning_aux:NNTF 07:2767, 07:2768, 07:2770, 07:2771
__cmd_peek_meaning_remove:NTF 07:1796, 07:1941, 07:1947, 07:2074, 07:2760, 07:2767, 07:2769
__cmd_peek_nonspace:NTF 07:2757, 07:2757
__cmd_peek_nonspace_aux:nNNTF 07:2757, 07:2758, 07:2760, 07:2761, 07:2764
__cmd_peek_nonspace_remove:NTF . 07:1797, 07:1835, 07:1929, 07:1935, 07:2062, 07:2072, 07:2757, 07:2759
__cmd_peek_true_remove:NNw 07:2767
__cmd_peek_true_remove:Nw 07:2778, 07:2782, 07:2790, 07:2794
\l__cmd_prefixed_bool 07:40, 07:823, 07:840, 07:847, 07:853, 07:861, 07:914, *132*
__cmd_prepare_signature:N 07:795, 07:811, 07:821, 07:877, 07:885, 07:898, 07:917, 07:925, 07:933, 07:940, 07:1017, 07:1027, 07:1069, 07:1082, 07:1095, *154*
__cmd_prepare_signature:n 07:94, 07:795, 07:795
__cmd_prepare_signature_-bypass:N 07:795, 07:824, 07:826, 07:841, 07:848, 07:856, 07:866, *154*
__cmd_prepare_signature_verb_-chk:n 07:795, 07:810, 07:814
\l__cmd_process_all_tl 07:41, 07:178, 07:313, 07:321, 07:385, 07:805, 07:948, 07:983, *132*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\l__cmd_process_one_tl 07:42, 07:806, 07:855, 07:864, 07:894, 07:903, 07:987, 07:990, *155*
\l__cmd_process_some_bool . 07:43, 07:177, 07:807, 07:854, 07:863, *132*
__cmd_provide_env:nnnn 07:3139, 07:3151, 07:3182, 07:3190, 07:3218, 07:3219
__cmd_put_arg_expandable:nw 07:2318, 07:2323, 07:2324, 07:2355, 07:2360, 07:2361, 07:2460, 07:2460, 07:2461
__cmd_renew_env:nnnn 07:3139, 07:3147, 07:3173, 07:3189, 07:3216, 07:3217
__cmd_replicate_processor:nn 07:893, 07:900, 07:900
__cmd_run_code: . . 07:315, 07:318, 07:318, 07:1580, 07:1588, 07:1596, 07:1599, 07:1604, 07:1788, 07:1925, 07:1931, 07:1937, 07:1943, 07:1967, 07:1982, 07:1989, 07:2000, 07:2002, 07:2008, 07:2014, 07:2020, 07:2026, 07:2032, 07:2038, 07:2044, 07:2055, 07:2057, 07:2075, 07:2097, 07:2283, *139*
\l__cmd_saved_args_tl 07:44, 07:1584, 07:1592, *132*
__cmd_set_environment_end:n 07:228, 07:279, 07:323
__cmd_set_eq_if_exist:NN 07:1133, 07:1151, 07:1153, 07:1172, 07:1191, 07:1192, 07:1193, 07:1206, 07:1207, 07:1208, 07:1306
__cmd_show:N 07:1350, 07:1352, 07:1352, 07:1379, 07:1402, 07:1485, 07:1487, 07:1569, *171*
__cmd_show:n 07:1369, 07:1464, 07:1472, 07:1476
__cmd_show_command:N 07:1358, 07:1369, 07:1369
__cmd_show_command:NnNNwN 07:1369, 07:1370, 07:1371
__cmd_show_command_aux:NnNNn 07:1369, 07:1373, 07:1386, 07:1395, 07:1404, 07:1464, *168*
__cmd_show_delim:Nw 07:1517, 07:1517
__cmd_show_delims:Nw 07:1517, 07:1519
__cmd_show_delims_opt:Nw 07:1517, 07:1521
__cmd_show_E:Nw 07:1517, 07:1536
__cmd_show_e:Nw 07:1517, 07:1525
__cmd_show_environment:N 07:1361, 07:1369, 07:1450, 07:1482
__cmd_show_environment:Nnnw 07:1452, 07:1460
__cmd_show_environment_end:N 07:1362, 07:1479
__cmd_show_expandable:N 07:1359, 07:1369, 07:1376
__cmd_show_expandable:NnNNNnN 07:1369, 07:1377, 07:1382, 07:1384, 07:1391, 07:1393, 07:1399
__cmd_show_opt:Nw . . 07:1517, 07:1523
__cmd_show_optimized:N 07:1360, 07:1369, 07:1414
__cmd_show_optimized:NN 07:1416, 07:1420
__cmd_show_optimized_aux:N 07:1422, 07:1445
__cmd_show_prefix:Nw 07:1517, 07:1549
__cmd_show_processor:Nw 07:1517, 07:1551
\c__cmd_show_type_!_tl 07:1511
\c__cmd_show_type_+_tl 07:1511
\c__cmd_show_type_>_tl 07:1511
\c__cmd_show_type_D_tl 07:1511
\c__cmd_show_type_d_tl 07:1511
\c__cmd_show_type_E_tl 07:1511
\c__cmd_show_type_e_tl 07:1511
\c__cmd_show_type_O_tl 07:1511
\c__cmd_show_type_R_tl 07:1511
\c__cmd_show_type_r_tl 07:1511
\c__cmd_show_type_t_tl 07:1511
\l__cmd_signature_tl 07:45, 07:171, 07:214, 07:808, 07:884, 07:897, 07:924, 07:932, 07:946, 07:955, 07:1099, 07:1583, 07:1611, 07:1851, 07:1957, 07:1967, 07:1984, 07:1991, 07:2000, 07:2004, 07:2010, 07:2016, 07:2022, 07:2028, 07:2034, 07:2040, 07:2046, 07:2077, 07:2099, 07:2283, *190*
__cmd_single_token_check:n 07:588, 07:589, 07:599, 07:621, 07:646, 07:647, 07:680, 07:680
\l__cmd_some_long_bool 07:47, 07:118, 07:147, 07:208, 07:216, 07:457, 07:751, 07:762, *144*
\l__cmd_some_obey_spaces_bool 07:46, 07:456, 07:552, 07:780, *133*
\l__cmd_some_short_bool 07:48, 07:117, 07:207, 07:218, 07:458, 07:763, *144*
__cmd_split_add_item:n 07:1550, 07:1553, 07:1552, 07:1558, *173*
__cmd_split_argument:nnn 07:2503, 07:2503, 07:3317

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

__cmd_split_argument_aux:n 07:2503, 07:2519, 07:2538
__cmd_split_argument_aux:nnnn 07:2503, 07:2506, 07:2510
__cmd_split_argument_aux:wn 07:2503, 07:2539, 07:2540
__cmd_split_end_item:n . 07:1509, 07:1518, 07:1520, 07:1522, 07:1524, 07:1530, 07:1541, 07:1553, 07:1560, *173*
__cmd_split_list:nn 07:2468, 07:2470, 07:2505, 07:3318
__cmd_split_list_multi:nn 07:2468, 07:2475, 07:2478, 07:2480, 07:2487, 07:2500
\l__cmd_split_list_seq 07:2468, 07:2482, 07:2484, *197*
__cmd_split_list_single:Nn 07:2468, 07:2476, 07:2490
\l__cmd_split_list_tl 07:2469, 07:2492, 07:2498, 07:2500, *197*
__cmd_split_N_head_apply:Nn 07:2548, 07:2568, 07:2580, 07:2655
__cmd_split_N_head_apply_-aux:NNw . . 07:2548, 07:2656, 07:2657
__cmd_split_signature:n 07:1406, 07:1488, 07:1488, *169*
__cmd_split_signature_loop:Nw . . 07:1493, 07:1495, 07:1495, 07:1509, 07:1518, 07:1520, 07:1522, 07:1524, 07:1532, 07:1545, 07:1550, 07:1552
__cmd_split_start_item: 07:1498, 07:1529, 07:1540, 07:1553, 07:1553, *173*
__cmd_start:nNNnnn 07:167, 07:287, 07:298, 07:2738, *155*
__cmd_start_aux:NNnnnn 07:293, 07:304, 07:307, 07:307, 07:317
__cmd_start_env:nnnnn 07:163, 07:287, 07:287, 07:2741, *203*
__cmd_start_expandable:nNNNNn . . 07:201, 07:404, 07:404, 07:2739, *136*
__cmd_start_optimized: 07:110, 07:139, 07:153, 07:1225, 07:2740, 07:2810, *203*
\l__cmd_suppress_strip_bool 07:38, 07:455, 07:558, 07:566, 07:801, 07:862, 07:963, 07:981, *132*
__cmd_tl_mapthread_function:NNN 07:356, 07:383, 07:2704, 07:2704
__cmd_tl_mapthread_function:nnN 07:431, 07:1543, 07:2704, 07:2717, *172*
__cmd_tl_mapthread_loop:w 07:2704, 07:2707, 07:2719, 07:2723, 07:2728
__cmd_tmp:w . 07:53, 07:56, 07:342, 07:358, 07:399, 07:401, 07:1104, 07:1110, 07:1128, 07:1335, 07:1345, 07:1511, 07:1514, 07:1516, 07:1534, 07:1538, 07:1544, 07:1548, 07:2799, *140*
\l__cmd_tmp_prop 07:53, 07:1953, 07:1956, 07:1962, *133*
\l__cmd_tmpa_tl 07:54, 07:354, 07:359, 07:368, 07:1073, 07:1075, 07:1089, 07:1092, 07:1246, 07:1252, 07:1267, 07:1275, 07:1291, 07:1298, 07:1316, 07:1319, 07:1410, 07:1481, 07:1482, 07:1491, 07:1562, 07:1665, 07:1677, 07:1687, 07:1696, 07:1703, 07:1735, 07:1890, 07:1893, 07:2773, 07:2798, 07:2801, *133*
\l__cmd_tmpb_tl 07:55, 07:1058, 07:1063, 07:1074, 07:1317, 07:1319, 07:1492, 07:1498, 07:1555, 07:1559, 07:1563, 07:1564, 07:1682, 07:1724, 07:1729, 07:1735, 07:1738, 07:1962, 07:1963, 07:1965, 07:2774, 07:2785, 07:2791, *133*
__cmd_token_if_cs:NTF 07:1914, 07:2695, 07:2695, 07:2780, *149*
\l__cmd_total_args_int 07:17, 07:797, *130*
__cmd_trim_spaces:n 07:2546, 07:2546, 07:3319
__cmd_use_i_delimit_by_q_-recursion_stop:nw . 07:50, 07:2615
\l__cmd_v_arg_tl 07:1613, 07:1649, 07:1652, 07:1694, 07:1703, 07:1756, 07:1762, 07:2086, 07:2103, 07:2122, 07:2156, 07:2227, 07:2234, 07:2243, 07:2255, 07:2271, *186*
\l__cmd_v_nesting_int . . 07:2158, 07:2162, 07:2178, 07:2179, 07:2188, *188*
\l__cmd_verb_safe_bool 07:982
cmd/#1/after (hook) *342*
cmd/#1/before (hook) *342*
cmd/⟨*name*⟩/after (hook) . . . *327*, *330*, *241*
cmd/⟨*name*⟩/before (hook) . . *327*, *327*, *241*
cmd/foo/before (hook) *329*
cmd/include/after (hook) *1163*
cmd/include/before (hook) *1163*
cmd/section/after (hook) *330*, *330*
cmd/section/before (hook) *330*
\cmrFont 33:1213, 33:1216
\colon 30:514
\color *449*
\columnbreak *1070*
\columnsep 20:27, 20:97, 20:154, 54:57, 54:207

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\columnseprule 54:58, 54:2887, 54:2930, 54:2961
\columnwidth 20:24, 20:27, 20:28, 20:30, 20:94, 20:97, 20:98, 20:100, 20:151, 20:154, 20:155, 20:158, 40:507, 40:546, 40:564, 40:581, 45:99, 45:168, 45:532, 45:551, 45:569, 54:56, 54:123, 54:124, 54:125, 54:206, 54:207, 54:208, 54:209, 54:210, 54:2420, 54:2422, 54:2885, 54:2889, 54:2928, 54:2932, 54:2957, 54:2963
\cong 30:468
\contentsline 44:164, 44:171, 44:177, 44:209, *1387*
\coprod 30:352
\copyright . . 21:305, 21:336, 29:760, *1395*
\coremissesfalse 33:1220
\coremissestrue 33:1198
\cos 38:12
\cosh 38:14
\cot 38:18
\coth 38:19
\countdef . 04:75, 04:85, 04:174, 04:190, 04:198, 04:206, 04:224, 34:3, 02:37, 02:38, 02:39, 02:41, 02:47, 57:75, 01:50
counter 36:265, *854*
\counterwithin *556*
\counterwithin 22:203, *1422*
\counterwithout *556*
\counterwithout 22:256, *1437*
\CountZero 04:224
\cr 21:523, 21:529, 21:539, 21:545, 33:65, 33:69, 33:891, 33:895, 38:188, 38:192, 38:440, 38:469, 38:517, 38:633, 41:244, 41:267, 41:280, 41:287, 41:296, 41:301, 41:307, 41:454, 42:152, 42:154, 42:159, 42:165, 02:321, *1328*
\crcr . . . 21:347, 21:382, 21:383, 21:411, 21:415, 21:418, 21:499, 21:503, 21:507, 21:509, 21:512, 21:756, 21:784, 21:788, 21:791, 21:858, 21:861, 21:919, 21:1281, 29:762, 30:348, 30:349, 30:351, 30:470, 30:473, 30:477, 30:541, 30:542, 30:543, 30:544, 30:545, 30:546, 30:548, 30:549, 30:550, 30:551, 30:552, 30:554, 33:70, 33:896, 38:168, 38:170, 38:171, 38:172, 38:188, 38:190, 38:191, 38:192, 38:210, 38:211, 41:184, 41:189, 41:194, 42:152, 55:457, 02:411, *1328*
create commands:
create_callback 04:781
\create_callback *44*
\CS *82*
\cs 35:201
cs commands:
\cs:w 08:1543, 08:1544, 08:1605, 08:1625, 08:1628, 08:1689, 08:1714, 08:1717, 08:1735, 08:1736, 08:1760, 08:1761, 08:1762, 08:1769, 08:1776, 08:2254, 08:2273, 08:2279, 08:2293, 08:2303, 08:2325, 08:2379, 08:2466, 08:2531, 08:2581, 08:2964, 08:2984, 08:2993, 08:2994, 09:33, 28:335, 28:338, 51:210, 52:211, 52:225, 52:237, 52:238, 55:313, 07:1917, 07:2744, 08:375, 08:1088, 08:1145, 08:1296, 08:1436, *334*
\cs_argument_spec:N 05:155, *1432*
\cs_end: 08:1543, 08:1544, 08:1605, 08:1625, 08:1628, 08:1689, 08:1714, 08:1717, 08:1735, 08:1736, 08:1751, 08:1761, 08:1762, 08:1769, 08:1776, 08:2131, 08:2253, 08:2257, 08:2261, 08:2273, 08:2278, 08:2279, 08:2293, 08:2299, 08:2313, 08:2326, 08:2331, 08:2379, 08:2465, 08:2466, 08:2531, 08:2581, 08:2964, 08:2984, 08:2994, 09:33, 28:335, 28:338, 51:210, 52:211, 52:225, 52:237, 52:238, 55:313, 07:1917, 07:2746, 08:375, 08:789, 08:1088, 08:1145, 08:1299, 08:1439
\cs_generate_from_arg_count:NNnn 10:86, 10:103, 11:553, 11:1140, 07:142, 07:156, 07:184, 07:192, 07:272, 07:358, 07:399, *1430*
\cs_generate_variant:Nn 08:1655, 08:1738, 11:140, 11:153, 11:440, 11:623, 11:649, 11:889, 11:891, 11:900, 11:902, 11:937, 11:940, 11:1014, 11:1047, 11:1129, 20:547, 36:37, 36:52, 36:62, 36:79, 36:96, 36:116, 36:117, 36:126, 36:139, 36:236, 36:253, 48:298, 51:51, 51:52, 52:461, 52:469, 07:11, 07:240, 07:317, 57:649, 57:658, 57:659, 07:1153, 07:2285, 07:2461, 07:2487, 07:2821, 07:3188, 07:3189, 07:3190, 08:37, 08:47, 08:48, 08:51, 08:60, 08:1102, 08:1125
\cs_gset:Npe 52:213, 08:1069
\cs_gset:Npn 08:1651, 08:2092, 08:2336, 08:2406,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

09:383, 26:355, 26:356, 28:286, 28:340, 36:24, 52:242, 53:234, 53:529, 07:3294, 08:1101, 08:1121
\cs_gset_eq:NN .. 08:2242, 08:2343, 08:2344, 08:2345, 08:2346, 08:2610, 09:72, 09:573, 09:601, 11:951, 24:414, 28:284, 28:294, 28:332, 28:346, 05:150, 05:151, 05:152, 05:154, 05:155, 05:156, 05:157, 05:158, 05:159, 53:113, 53:212, 53:227, 53:228, 53:233, 53:242, 53:246, 53:382, 53:440, 53:527, 53:528, 07:3321, 08:792, 08:1446, 08:1472, 08:1487, 08:1488, 08:1491
\cs_gset_nopar:Npe 08:44, 08:46
\cs_gset_nopar:Npn 08:2992
\cs_gset_protected:Npe 10:21, 10:23, 48:353, 53:81, 08:20, 08:1183
\cs_gset_protected:Npn 08:2187, 08:2190, 08:2218, 08:2223, 08:2241, 08:2400, 08:2430, 08:2826, 08:2969, 09:247, 09:457, 09:579, 11:555, 11:1133, 11:1150, 51:85, 07:3280, 08:82, 08:85, 08:94, 08:127, 08:154, 08:182, 08:185, 08:191, 08:215, 08:220, 08:245, 08:304, 08:322, 08:603, 08:606, 08:624, 08:650, 08:699, 08:707, 08:1470, 08:1531
\cs_if_eq:NNTF 51:98, 51:100, 51:167, 07:1110, 07:1973
\cs_if_exist:N *1155*
\cs_if_exist:NTF . 08:1845, 08:2512, 08:2541, 08:2549, 09:89, 09:97, 10:118, 11:130, 11:156, 11:169, 11:551, 11:613, 11:642, 11:654, 11:795, 11:820, 11:949, 11:972, 11:990, 05:153, 36:67, 36:179, 51:66, 51:83, 51:169, 52:93, 52:438, 52:442, 07:71, 07:233, 07:249, 07:1113, 07:1152, 07:1470, 07:2805, 07:3107, 07:3120, 07:3132, 07:3159, 07:3165, 07:3175, 07:3184, 07:3196, 07:3199, 07:3206, 07:3211, 07:3226, 07:3239, 07:3252, 08:1073, 08:1138, *1178*
\cs_if_exist_p:N . 07:78, 07:79, 08:271
\cs_if_exist_use:NTF 11:483, 11:1041, 26:351, 05:198, 53:374, 53:375, 53:379, 53:380, 07:485, 07:1269, 08:1305, 08:1349, 08:1374
\cs_if_free:NTF 36:8, 51:107
\cs_meaning:N 10:69
\cs_new:Npe 08:565
\cs_new:Npn 08:1568, 08:1569, 08:1740, 08:1741, 08:2088, 08:2095, 08:2251, 08:2276, 08:2284, 08:2297, 08:2305, 08:2314, 08:2323, 08:2358, 08:2539, 08:2547, 08:2561, 08:2579, 08:2588, 08:2593, 08:2834, 08:2835, 08:2836, 08:2837, 08:2841, 08:2842, 09:319, 09:329, 09:339, 09:341, 09:343, 09:354, 09:375, 09:394, 09:395, 09:398, 09:519, 09:627, 10:88, 10:164, 11:11, 11:21, 11:40, 11:46, 11:100, 11:441, 11:442, 11:655, 11:664, 11:1061, 11:1236, 11:1610, 11:1612, 11:1614, 11:1617, 16:85, 16:115, 16:127, 17:35, 20:538, 05:196, 36:63, 36:65, 36:89, 36:97, 36:104, 36:118, 36:127, 36:189, 36:190, 36:204, 36:205, 36:219, 36:220, 48:279, 48:289, 48:290, 48:291, 48:292, 48:297, 49:111, 49:112, 51:205, 51:207, 52:30, 52:38, 52:44, 52:57, 52:95, 52:100, 52:105, 52:112, 52:127, 52:234, 52:240, 52:411, 52:413, 52:415, 52:418, 52:422, 52:436, 52:452, 52:462, 52:582, 53:6, 53:7, 53:8, 53:23, 53:48, 53:115, 53:120, 53:138, 53:202, 53:207, 53:224, 53:239, 53:244, 53:250, 53:253, 53:267, 53:318, 53:371, 53:384, 53:387, 53:397, 53:445, 53:526, 53:585, 53:586, 53:587, 07:100, 07:102, 07:153, 55:28, 55:42, 55:311, 55:350, 55:356, 55:406, 55:409, 07:404, 55:452, 07:409, 07:411, 07:413, 07:425, 07:437, 07:443, 07:1176, 07:1231, 07:1312, 07:1337, 07:1342, 07:1767, 07:1772, 07:1774, 07:1776, 07:1904, 07:2286, 07:2288, 07:2304, 07:2311, 07:2312, 07:2334, 07:2336, 07:2352, 07:2365, 07:2367, 07:2369, 07:2371, 07:2377, 07:2394, 07:2396, 07:2398, 07:2400, 07:2402, 07:2404, 07:2406, 07:2408, 07:2429, 07:2431, 07:2452, 07:2454, 07:2460, 07:2538, 07:2540, 07:2565, 07:2571, 07:2577, 07:2583, 07:2655, 07:2657, 07:2704, 07:2717, 07:2723, 07:2730, 07:2751, 07:2803, 07:2822, 07:2835, 07:3261, 07:3278, 07:3279, 07:3299, 07:3300, 07:3301, 08:39, 08:40, 08:324, 08:330, 08:345, 08:355, 08:356, 08:358, 08:372, 08:378, 08:573, 08:926, 08:928, 08:930, 08:1224, 08:1236, 08:1241, 08:1249, 08:1258, 08:1260, 08:1267, 08:1294,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

08:1301, 08:1303, 08:1418, 08:1434
`\cs_new_eq:NN` . . . 08:1815, 08:1816, 08:1817, 08:1818, 08:1819, 08:1820, 08:2313, 08:2855, 08:2856, 08:2857, 08:2858, 08:2928, 08:2930, 09:13, 09:14, 09:474, 10:7, 10:8, 10:160, 10:161, 10:162, 10:163, 10:167, 10:192, 10:193, 10:194, 10:195, 10:196, 10:197, 10:198, 10:199, 10:200, 10:201, 10:202, 10:203, 10:204, 10:205, 10:206, 10:207, 10:208, 11:890, 11:901, 16:137, 20:816, 20:817, 05:176, 05:177, 05:178, 05:179, 05:195, 05:231, 36:188, 36:203, 36:218, 48:340, 48:356, 48:357, 48:399, 48:401, 48:513, 48:515, 51:8, 52:252, 52:253, 52:509, 52:511, 52:513, 52:515, 52:517, 52:519, 52:521, 52:523, 52:525, 52:527, 53:68, 53:213, 53:394, 53:396, 53:464, 53:467, 53:468, 53:538, 53:584, 07:56, 07:99, 55:3, 55:4, 55:345, 57:679, 57:680, 07:1287, 07:1288, 07:1295, 07:1302, 07:3101, 07:3102, 07:3296, 07:3297, 07:3298, 07:3304, 07:3305, 07:3306, 07:3316, 07:3317, 07:3318, 07:3319, 07:3320, 08:7, 08:23, 08:34, 08:57, 08:1208, 08:1391, 08:1398, 08:1441
`\cs_new_protected:Npe` 09:509, 11:366, 11:913, 48:114, 07:298, 07:1636, 07:1661, 07:1708, 07:1786, 07:2050, 07:2159
`\cs_new_protected:Npn` 08:1577, 08:1659, 08:1742, 08:1749, 08:1758, 08:1765, 08:1772, 08:1779, 08:1787, 08:1793, 08:1804, 08:1821, 08:1828, 08:1840, 08:1847, 08:1854, 08:1860, 08:1862, 08:1866, 08:1984, 08:2106, 08:2124, 08:2129, 08:2138, 08:2156, 08:2170, 08:2192, 08:2205, 08:2232, 08:2236, 08:2246, 08:2259, 08:2268, 08:2282, 08:2288, 08:2311, 08:2318, 08:2341, 08:2364, 08:2374, 08:2385, 08:2391, 08:2411, 08:2441, 08:2443, 08:2453, 08:2595, 08:2601, 08:2607, 08:2778, 08:2779, 08:2780, 08:2803, 08:2814, 08:2844, 08:2845, 08:2846, 08:2847, 08:2859, 08:2866, 08:2873, 08:2880, 08:2887, 08:2889, 08:2891, 08:2898, 08:2905, 08:2912, 08:2919, 09:18, 09:22, 09:24, 09:39, 09:41, 09:50, 09:52, 09:62, 09:70, 09:79, 09:93, 09:113, 09:133, 09:146, 09:151, 09:160, 09:165, 09:172, 09:312, 09:400, 09:438, 09:475, 09:503, 09:532, 10:9, 10:14, 10:19, 10:27, 10:58, 10:77, 10:78, 10:98, 10:122, 10:144, 10:154, 11:105, 11:111, 11:116, 11:117, 11:118, 11:128, 11:134, 11:141, 11:147, 11:154, 11:179, 11:187, 11:203, 11:211, 11:222, 11:232, 11:248, 11:258, 11:275, 11:281, 11:296, 11:314, 11:332, 11:344, 11:361, 11:387, 11:411, 11:443, 11:460, 11:462, 11:481, 11:492, 11:494, 11:496, 11:503, 11:511, 11:523, 11:549, 11:557, 11:576, 11:578, 11:592, 11:596, 11:611, 11:624, 11:638, 11:650, 11:652, 11:659, 11:678, 11:695, 11:716, 11:733, 11:740, 11:746, 11:751, 11:760, 11:762, 11:764, 11:773, 11:787, 11:810, 11:817, 11:831, 11:836, 11:852, 11:860, 11:866, 11:876, 11:884, 11:892, 11:906, 11:925, 11:930, 11:935, 11:938, 11:941, 11:954, 11:963, 11:984, 11:997, 11:1009, 11:1015, 11:1021, 11:1026, 11:1048, 11:1056, 11:1066, 11:1081, 11:1087, 11:1107, 11:1124, 11:1130, 11:1136, 11:1147, 11:1153, 11:1165, 11:1175, 11:1194, 11:1196, 11:1207, 11:1212, 11:1218, 11:1224, 11:1230, 11:1237, 11:1239, 11:1248, 11:1250, 11:1264, 11:1266, 11:1275, 11:1284, 11:1293, 11:1301, 11:1549, 11:1551, 11:1553, 11:1555, 11:1557, 11:1559, 11:1561, 11:1563, 11:1565, 11:1567, 11:1600, 11:1602, 11:1604, 11:1606, 11:1608, 11:1616, 11:1619, 11:1620, 11:1622, 11:1624, 11:1632, 11:1633, 16:88, 16:95, 24:413, 28:279, 36:6, 36:17, 36:22, 36:38, 36:43, 36:48, 36:50, 36:53, 36:150, 36:164, 36:171, 36:221, 36:228, 36:237, 48:7, 48:17, 48:54, 48:67, 48:75, 48:102, 48:132, 48:137, 48:162, 48:171, 48:213, 48:224, 48:235, 48:243, 48:246, 48:277, 48:341, 48:346, 48:351, 48:359, 48:372, 48:388, 48:417, 48:449, 51:22, 51:28, 51:44, 51:58, 51:60, 51:96, 51:105, 51:117, 51:133, 51:138, 51:147, 51:159, 51:164, 51:181, 51:197, 51:208, 51:212, 51:230, 51:244, 52:49, 52:62, 52:70, 52:79, 52:208,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

52:222, 52:392, 52:397, 52:420, 52:547, 53:69, 53:74, 53:79, 53:390, 53:547, 07:59, 07:64, 07:69, 07:90, 07:110, 07:132, 07:154, 07:182, 07:230, 07:244, 07:265, 07:279, 07:287, 07:307, 07:318, 07:328, 07:339, 07:347, 55:5, 55:6, 55:7, 07:352, 55:20, 55:23, 55:29, 55:30, 55:31, 55:32, 55:52, 55:53, 07:361, 07:366, 07:380, 55:301, 55:302, 07:389, 55:306, 55:315, 55:346, 07:397, 55:377, 55:398, 55:402, 55:412, 55:415, 55:418, 55:421, 55:424, 55:441, 07:446, 57:629, 57:660, 57:665, 57:670, 57:678, 57:703, 57:708, 57:713, 07:481, 07:505, 07:510, 07:515, 07:517, 07:519, 07:524, 07:526, 07:540, 07:546, 07:555, 07:572, 07:585, 07:595, 07:610, 07:618, 07:633, 07:639, 07:644, 07:653, 07:659, 07:661, 07:663, 07:680, 07:689, 07:708, 07:718, 07:730, 07:741, 07:746, 07:747, 07:776, 07:778, 07:795, 07:814, 07:821, 07:826, 07:836, 07:843, 07:850, 07:858, 07:868, 07:870, 07:872, 07:879, 07:887, 07:900, 07:911, 07:919, 07:927, 07:935, 07:942, 07:953, 07:992, 07:1002, 07:1007, 07:1014, 07:1019, 07:1021, 07:1029, 07:1042, 07:1055, 07:1071, 07:1077, 07:1084, 07:1086, 07:1097, 07:1102, 07:1108, 07:1126, 07:1133, 07:1151, 07:1154, 07:1169, 07:1188, 07:1203, 07:1217, 07:1248, 07:1256, 07:1264, 07:1273, 07:1285, 07:1289, 07:1296, 07:1301, 07:1303, 07:1314, 07:1321, 07:1329, 07:1352, 07:1369, 07:1371, 07:1376, 07:1384, 07:1393, 07:1404, 07:1414, 07:1420, 07:1450, 07:1460, 07:1472, 07:1476, 07:1479, 07:1488, 07:1495, 07:1517, 07:1519, 07:1521, 07:1523, 07:1525, 07:1536, 07:1549, 07:1551, 07:1553, 07:1558, 07:1560, 07:1572, 07:1574, 07:1576, 07:1578, 07:1580, 07:1588, 07:1599, 07:1604, 07:1609, 07:1623, 07:1645, 07:1669, 07:1692, 07:1701, 07:1734, 07:1736, 07:1747, 07:1811, 07:1830, 07:1849, 07:1872, 07:1925, 07:1931, 07:1937, 07:1943, 07:1949, 07:1971, 07:1981, 07:1982, 07:1989, 07:2002, 07:2008, 07:2014, 07:2020, 07:2026, 07:2032, 07:2038, 07:2044, 07:2055, 07:2057, 07:2059, 07:2071, 07:2073, 07:2075, 07:2087, 07:2092, 07:2097, 07:2118, 07:2124, 07:2134, 07:2140, 07:2153, 07:2166, 07:2172, 07:2197, 07:2210, 07:2219, 07:2241, 07:2253, 07:2269, 07:2278, 07:2280, 07:2462, 07:2470, 07:2480, 07:2490, 07:2503, 07:2510, 07:2546, 07:2548, 07:2553, 07:2589, 07:2594, 07:2605, 07:2607, 07:2609, 07:2626, 07:2637, 07:2647, 07:2649, 07:2651, 07:2653, 07:2663, 07:2667, 07:2695, 07:2757, 07:2759, 07:2761, 07:2767, 07:2769, 07:2771, 07:2787, 07:2794, 07:3103, 07:3116, 07:3129, 07:3134, 07:3141, 07:3145, 07:3149, 07:3153, 07:3157, 07:3173, 07:3182, 07:3194, 07:3204, 07:3210, 07:3212, 07:3222, 07:3235, 07:3248, 07:3256, 08:8, 08:13, 08:18, 08:41, 08:43, 08:45, 08:49, 08:52, 08:58, 08:63, 08:65, 08:67, 08:98, 08:142, 08:166, 08:168, 08:170, 08:195, 08:197, 08:199, 08:225, 08:260, 08:262, 08:279, 08:284, 08:286, 08:379, 08:388, 08:393, 08:401, 08:425, 08:433, 08:449, 08:457, 08:467, 08:482, 08:493, 08:499, 08:505, 08:521, 08:534, 08:577, 08:594, 08:654, 08:673, 08:681, 08:688, 08:720, 08:726, 08:732, 08:742, 08:784, 08:786, 08:794, 08:796, 08:799, 08:801, 08:982, 08:984, 08:1019, 08:1056, 08:1068, 08:1071, 08:1100, 08:1113, 08:1115, 08:1117, 08:1119, 08:1136, 08:1160, 08:1167, 08:1325, 08:1332, 08:1363, 08:1385, 08:1392, 08:1399, 08:1405, 08:1410, 08:1415, 08:1416, 08:1444, 08:1497, *1209*

`\cs_new_protected_nopar:Npn` 57:675, 07:1906, 07:1996

`\cs_parameter_spec:N` 05:153, 05:154, 07:1442, 07:2824, 08:1252, *345*

`\cs_prefix_spec:N` 09:223, 09:298, 05:152

`\cs_replacement_spec` *277*

`\cs_replacement_spec:N` 08:1908, 08:1921, 08:1930, 05:157, 07:1411, 07:1447, 07:1456, 08:1227, 08:1462, 08:1484

`\cs_set:Npe` 52:211, 52:225, 07:223, 07:1032, 07:1045, 07:1422

`\cs_set:Npn` 08:1905, 08:2599, 08:2605, 08:2978, 09:215, 09:290, 09:490,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

09:549, 09:585, 10:105, 11:419, 11:879, 16:37, 16:68, 26:344, 26:350, 26:354, 48:374, 53:15, 07:194, 07:273, 07:358, 07:399, 07:1104, 08:1169, 08:1180, 08:1187, 08:1196, *173*
\cs_set_eq:NN ... 08:1549, 08:1551, 08:1553, 08:1555, 08:1842, 08:1849, 08:1856, 09:187, 09:188, 09:208, 09:209, 09:214, 09:262, 09:263, 09:283, 09:284, 09:289, 09:486, 09:487, 09:545, 09:546, 09:581, 09:582, 09:640, 11:992, 16:93, 16:94, 16:138, 16:139, 16:140, 16:142, 16:143, 16:144, 16:178, 16:179, 16:180, 16:183, 28:299, 28:349, 36:254, 48:113, 48:280, 48:281, 48:282, 51:88, 53:85, 53:105, 53:144, 53:156, 53:186, 53:193, 53:470, 53:472, 53:474, 53:476, 53:478, 07:275, 07:276, 55:340, 55:343, 07:1128, 07:1152, 07:1171, 07:1190, 07:1205, 07:1219, 07:1280, 07:1305, 07:1310, 07:1325, 07:1327, 07:1616, 07:1832, 07:2005, 07:2011, 07:2017, 07:2023, 07:2029, 07:2035, 07:2041, 07:2047, 07:2199, 07:2212, 07:2799, 08:1516, 08:1517, 08:1518, 08:1519
\cs_set_nopar:Npe 07:135, 07:199, 07:220, 07:225, 07:270, 07:281, 07:1033, 07:1046, 07:1196, 07:1222, 07:1323, 08:42
\cs_set_nopar:Npn . 57:631, 57:632, *173*
\cs_set_nopar:Npx 07:1210
\cs_set_protected:Npe 11:974
\cs_set_protected:Npn 08:2110, 08:2141, 09:189, 09:264, 11:107, 11:108, 11:113, 11:114, 11:415, 11:1134, 11:1151, 28:327, 28:359, 51:5, 51:218, 53:18, 53:19, 53:107, 53:196, 53:465, 07:158, 57:647, 07:1335, 07:1511, 07:1534, 07:1538, 07:1575, 07:1579, 07:1793, 07:1934, 07:1946, 07:1958, 07:1992, 07:2058, 07:2078, 08:414
\cs_set_protected_nopar:Npe 07:136, 07:159, 07:199, 07:1173, 07:1196, 07:1221, 07:1308
\cs_set_protected_nopar:Npn 07:1573, 07:1577, 07:1794, 07:1928, 07:1940, 07:1985, 07:2056
\cs_show:N 11:1265, *412*
\cs_to_str:N 09:128, 09:149, 09:163, 09:168, 05:150, 07:78, 07:79, 07:92, 07:1145, 07:1147, 07:1365, 07:1417, 07:1455, 07:1457, 07:1481, 07:1917, 07:2746, 07:2813, *131*
\cs_undefine:N .. 08:2351, 08:2437, 08:2575, 09:75, 09:226, 09:528, 09:536, 52:227, 07:3214, 07:3215, 07:3216, 07:3217, 07:3218, 07:3219, 07:3281, 07:3310, 07:3311, 07:3312, 07:3322, 08:73, 08:74, 08:84, 08:126, 08:184, 08:190, 08:265, 08:671, 08:1106, 08:1107, 08:1108, 08:1109, 08:1129, 08:1130, 08:1131, 08:1132, 08:1156, 08:1212, 08:1232, 08:1290, 08:1311, 08:1312, 08:1323, 08:1417
cs\check@icr commands:
\cs_gset_eq:NN *76*
\csc 38:21
\csname *1411*
\csname\endcsname *1175*
\csstring 04:212, *1440*
\cup 30:384
\CurrentFile 50:981, 52:3, 52:67, 52:84, 52:166, 52:171, 52:174, 52:377, 52:381, 52:555, 52:556, *1175*
\CurrentFilePath 52:3, 52:67, 52:83, 52:165, 52:378, 52:381, *1161*
\CurrentFilePathUsed 52:3, 52:66, 52:81, 52:163, 52:375, 52:378, *1161*
\CurrentFileUsed 50:981, 52:3, 52:66, 52:82, 52:164, 52:375, 52:377, 52:554, 52:555, *1161*
\currentgrouplevel 50:823
\currentgrouptype 39:148, 39:151, 39:153, 41:247, 41:270, *1439*
\CurrentOption 21:1564, 21:1567, 21:1572, 21:1588, 50:14, 50:555, 50:566, 50:579, 50:580, 50:585, 50:598, 50:599, 50:601, 50:615, 50:616, 50:619, 50:633, 50:638, 50:639, 50:652, 50:657, 50:658, 50:671, 50:673, 50:679, 50:681, 50:693, 50:694, 50:695, 50:703, 50:704, 50:705, 50:1031, 50:1096, 50:1215, 50:1216, 50:1226, 51:70, 51:71, *1392*
CurrentOption commands:
\CurrentOption: 50:578, 50:597, 50:614, 50:632, 50:637, 50:651, 50:656, 50:692, 50:702, 50:1225
\currsubencoding 33:1212, 33:1223, 33:1379, 33:1381, 33:1382
\CYRA 21:1536
\cyra 21:1536, 21:1591
\CYRABHCH 21:1536
\cyrabhch 21:1536

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\CYRABHCHDSC 21:1536
\cyrabhchdsc 21:1536
\CYRABHDZE 21:1537
\cyrabhdze 21:1536
\CYRABHHA 21:1537
\cyrabhha 21:1537
\CYRAE 21:1537
\cyrae 21:1537
\CYRB 21:1537
\cyrb 21:1537
\CYRBYUS 21:1538
\cyrbyus 21:1537
\CYRC 21:1538
\cyrc 21:1538
\CYRCH 21:1538
\cyrch 21:1538
\CYRCHLDSC 21:1538
\cyrchldsc 21:1538
\CYRCHRDSC 21:1539
\cyrchrdsc 21:1538
\CYRCHVCRS 21:1539
\cyrchvcrs 21:1539
\CYRD 21:1539
\cyrd 21:1539
\CYRDELTA 21:1539
\cyrdelta 21:1539
\CYRDJE 21:1540
\cyrdje 21:1540
\CYRDZE 21:1540
\cyrdze 21:1540
\CYRDZHE 21:1540
\cyrdzhe 21:1540
\CYRE 21:1540
\cyre 21:1540
\CYREPS 21:1541
\cyreps 21:1540
\CYREREV 21:1541
\cyrerev 21:1541
\CYRERY 21:1541
\cyrery 21:1541
\CYRF 21:1541
\cyrf 21:1541
\CYRFITA 21:1542
\cyrfita 21:1541
\CYRG 21:1542
\cyrg 21:1542
\CYRGDSC 21:1542
\cyrgdsc 21:1542
\CYRGDSCHCRS 21:1542
\cyrgdschcrs 21:1542
\CYRGHCRS 21:1543
\cyrghcrs 21:1543
\CYRGHK 21:1543
\cyrghk 21:1543
\CYRGUP 21:1543
\cyrgup 21:1543
\CYRH 21:1543
\cyrh 21:1543
\CYRHDSC 21:1544
\cyrhdsc 21:1544
\CYRHHCRS 21:1544
\cyrhhcrs 21:1544
\CYRHHK 21:1544
\cyrhhk 21:1544
\CYRHRDSN 21:1545
\cyrhrdsn 21:1544
\CYRI 21:1545
\cyri 21:1545
\CYRIE 21:1545
\cyrie 21:1545
\CYRII 21:1545
\cyrii 21:1545
\CYRISHRT 21:1545
\cyrishrt 21:1545
\CYRISHRTDSC 21:1546
\cyrishrtdsc 21:1546
\CYRIZH 21:1546
\cyrizh 21:1546
\CYRJE 21:1546
\cyrje 21:1546
\CYRK 21:1546
\cyrk 21:1546
\CYRKBEAK 21:1547
\cyrkbeak 21:1547
\CYRKDSC 21:1547
\cyrkdsc 21:1547
\CYRKHCRS 21:1547
\cyrkhcrs 21:1547
\CYRKHK 21:1548
\cyrkhk 21:1547
\CYRKVCRS 21:1548
\cyrkvcrs 21:1548
\CYRL 21:1548
\cyrl 21:1548
\CYRLDSC 21:1548
\cyrldsc 21:1548
\CYRLHK 21:1549
\cyrlhk 21:1548
\CYRLJE 21:1549
\cyrlje 21:1549
\CYRM 21:1549
\cyrm 21:1549
\CYRMDSC 21:1549
\cyrmdsc 21:1549
\CYRMHK 21:1549
\cyrmhk 21:1549
\CYRN 21:1550
\cyrn 21:1550

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\CYRNDSC 21:1550
\cyrndsc 21:1550
\CYRNG 21:1550
\cyrng 21:1550
\CYRNHK 21:1550
\cyrnhk 21:1550
\CYRNJE 21:1551
\cyrnje 21:1550
\CYRNLHK 21:1551
\cyrnlhk 21:1551
\CYRO 21:1551
\cyro 21:1551
\CYROTLD 21:1551
\cyrotld 21:1551
\CYRP 21:1551
\cyrp 21:1551
\CYRPHK 21:1552
\cyrphk 21:1552
\CYRQ 21:1552
\cyrq 21:1552
\CYRR 21:1552
\cyrr 21:1552
\CYRRDSC 21:1552
\cyrrdsc 21:1552
\CYRRHK 21:1553, *1402*
\cyrrhk 21:1552, *1402*
\CYRRHOOK *1402*
\cyrrhook *1402*
\CYRRTICK 21:1553
\cyrrtick 21:1553
\CYRS 21:1553
\cyrs 21:1553
\CYRSACRS 21:1553
\cyrsacrs 21:1553
\CYRSCHWA 21:1554
\cyrschwa 21:1554
\CYRSDSC 21:1554
\cyrsdsc 21:1554
\CYRSEMISFTSN 21:1554
\cyrsemisftsn 21:1554
\CYRSFTSN 21:1555
\cyrsftsn 21:1555
\CYRSH 21:1555
\cyrsh 21:1555
\CYRSHCH 21:1555
\cyrshch 21:1555
\CYRSHHA 21:1555
\cyrshha 21:1555
\CYRT 21:1556
\cyrt 21:1556
\CYRTDSC 21:1556
\cyrtdsc 21:1556
\CYRTETSE 21:1556
\cyrtetse 21:1556
\CYRTSHE 21:1556
\cyrtshe 21:1556
\CYRU 21:1557
\cyru 21:1557
\CYRUSHRT 21:1557
\cyrushrt 21:1557
\CYRV 21:1557
\cyrv 21:1557
\CYRW 21:1557
\cyrw 21:1557
\CYRY 21:1557
\cyry 21:1557
\CYRYA 21:1558
\cyrya 21:1558
\CYRYAT 21:1558
\cyryat 21:1558
\CYRYHCRS 21:1558
\cyryhcrs 21:1558
\CYRYI 21:1558
\cyryi 21:1558
\CYRYO 21:1559
\cyryo 21:1558
\CYRYU 21:1559
\cyryu 21:1559
\CYRZ 21:1559
\cyrz 21:1559
\CYRZDSC 21:1559
\cyrzdsc 21:1559
\CYRZH 21:1559
\cyrzh 21:1559
\CYRZHDSC 21:1560
\cyrzhdsc 21:1560

D

\d 21:255, 21:416, 21:505, 21:789, 21:1277, 21:1491, 21:1492, 21:1493, 21:1494, 21:1497, 21:1498, 21:1499, 21:1500, 21:1501, 21:1502, 21:1503, 21:1504, 21:1505, 21:1506, 21:1507, 21:1508, 21:1509, 21:1510, 21:1511, 21:1512, 21:1513, 21:1514, 21:1515, 21:1516, 21:1517, 21:1518, 21:1519, 21:1520, 21:1521, 21:1522, 21:1523, 21:1524, 21:1525, 21:1526, 21:1527, 21:1528, 21:1529, 21:1530, *1377*
\dag 21:332, *1382*
\dagger 21:332, 22:332, 22:338, 22:346, 22:347, 30:386
\dashbox 42:320, 42:818, 42:835
\dashv 30:414
\date *1012*
\date 44:9, 44:25
\day 01:169, 50:1341, 50:1474, 50:1563, 02:259
\dblfigrule 54:1118, 54:3026, *1373*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\dblfloatpagefraction 45:287, 45:301, 54:3003
\dblfloatsep 54:1104, 54:1116, 54:2217, 54:2343, 54:3010
\dbltextfloatsep 54:227, 54:235, 54:1120, 54:2216, 54:2342, 54:3010
\dbltopfraction . . 45:284, 45:298, 54:3002
\ddag 21:333, *1382*
\ddagger 21:333, 22:333, 22:339, 22:346, 22:348, 30:385
\ddot 30:529
\ddots 30:524
\deadcycles 20:354, 20:410, 20:449, 37:35, 37:100, 37:176, 37:242, 54:304, *1398*
debug commands:
  \debug_resume: 11:185, 11:201, 11:209, 11:220, *247*
  \debug_suspend: 11:181, 11:189, 11:205, 11:213, *247*
\DebugHooksOff 08:2846, 08:2954, *229*
\DebugHooksOn 08:2846, 08:2953, *229*
\DebugMarksOff 48:356, *1063*
\DebugMarksOn 48:356, *1063*
\DebugShipoutsOff . . 53:467, 53:499, *1189*
\DebugShipoutsOn . . . 53:467, 53:498, *1189*
\DebugSocketsOff 10:192, 10:222, *361*
\DebugSocketsOn 10:192, 10:221, *361*
\DebugTablesOff 55:339
\DebugTablesOn 55:339
\DebugTemplatesOff 11:1632, *379*
\DebugTemplatesOn 11:1632, *379*
declare commands:
  declare_callback_rule 04:894
\Declare... . . . . *117*
\declare_callback_rule *44*
\DeclareAccent *1366*
\DeclareCaseChangeEquivalent 57:620, *1426*
\DeclareCommandCopy 06:564, 06:565, 06:567, *1416*
\DeclareCurrentRelease . . 33:777, 50:1808
\DeclareDefaultHookRule 08:2850, 08:2957, *234*
\DeclareDocumentCommand 44:186, 44:187, 07:3103, *90*
\DeclareDocumentEnvironment 07:3139, *117*
\DeclareEmphSequence *755*
\DeclareEmphSequence 29:624, 29:660, 29:661, 29:673, *756*
\DeclareEncoding *1366*
\DeclareEncodingSubset 24:186, 33:361, 33:362, 33:363, 33:364, 33:365, 33:366, 33:367, 33:368, 33:369, 33:370, 33:371, 33:372, 33:373, 33:374, 33:375, 33:376, 33:377, 33:378, 33:379, 33:380, 33:381, 33:382, 33:383, 33:384, 33:385, 33:386, 33:387, 33:388, 33:389, 33:390, 33:391, 33:392, 33:393, 33:394, 33:395, 33:396, 33:397, 33:398, 33:399, 33:400, 33:401, 33:402, 33:403, 33:404, 33:405, 33:406, 33:407, 33:408, 33:409, 33:410, 33:411, 33:412, 33:413, 33:414, 33:415, 33:416, 33:417, 33:418, 33:419, 33:420, 33:421, 33:422, 33:423, 33:424, 33:425, 33:426, 33:427, 33:428, 33:429, 33:430, 33:431, 33:432, 33:433, 33:434, 33:435, 33:436, 33:437, 33:438, 33:439, 33:440, 33:441, 33:442, 33:443, 33:444, 33:445, 33:446, 33:447, 33:448, 33:449, 33:450, 33:451, 33:452, 33:453, 33:454, 33:455, 33:456, 33:457, 33:458, 33:459, 33:460, 33:461, 33:462, 33:463, 33:464, 33:465, 33:466, 33:467, 33:468, 33:469, 33:470, 33:471, 33:472, 33:473, 33:474, 33:475, 33:476, 33:477, 33:478, 33:479, 33:480, 33:481, 33:482, 33:483, 33:484, 33:485, 33:486, 33:487, 33:488, 33:489, 33:490, 33:491, 33:492, 33:493, 33:494, 33:495, 33:496, 33:497, 33:498, 33:499, 33:500, 33:501, 33:502, 33:503, 33:504, 33:505, 33:506, 33:507, 33:508, 33:509, 33:510, 33:511, 33:512, 33:513, 33:514, 33:515, 33:516, 33:517, 33:518, 33:519, 33:520, 33:521, 33:522, 33:523, 33:524, 33:525, 33:526, 33:527, 33:528, 33:529, 33:530, 33:531, 33:532, 33:533, 33:534, 33:535, 33:536, 33:537, 33:538, 33:539, 33:584, 33:782, 33:783, 33:784, 33:785, 33:826, 33:833, 33:834, 33:835, 33:836, 33:1112, 33:1113, 33:1114, 33:1115, 33:1116, 33:1117, 33:1118, 33:1119, 33:1120, 33:1121, 33:1122, 33:1123, 33:1124, 33:1125, 33:1126, 33:1127, 33:1128, 33:1129, 33:1130, 33:1131, 33:1132, 33:1133, 33:1134, 33:1135, 33:1136, 33:1137, 33:1138, 33:1139, 33:1140, 33:1141, 33:1142, 33:1143, 33:1144, 33:1145, 33:1146, 33:1147, 33:1148, 33:1149, 33:1150, 33:1151, 33:1152,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

33:1153, 33:1154, 33:1155, 33:1156, 33:1157, 33:1158, 33:1159, 33:1160, 33:1161, 33:1162, 33:1163, 33:1164, 33:1165, 33:1166, 33:1167, 33:1168, 33:1169, 33:1170, 33:1171, 33:1172, *578*
\DeclareEncodingsubset *830*
\DeclareEnvironmentCopy 06:605, 06:606, 06:608, *1428*
\DeclareErrorFont 24:574, 28:403, 28:453, 29:891, 30:61
\DeclareExpandableDocumentCommand 44:184, 44:185, 07:3222, *125*
\DeclareFixedFont 24:76, 33:1213, 33:1215, *831*
\DeclareFontEncoding 21:397, 21:485, 21:740, 21:762, 21:768, 21:854, 21:1066, 24:119, 30:137, 30:138, 30:139, 30:140, *1377*
\DeclareFontEncodingDefaults 24:169, 27:90, 27:91, 30:36
\DeclareFontFamily 24:94, 27:85, 27:86, *590*
\DeclareFontFamilySubstitution . 24:534
\DeclareFontSeriesChangeRule 25:4, 25:5, 25:7, 25:8, 25:9, 25:10, 25:11, 25:12, 25:13, 25:14, 25:15, 25:16, 25:17, 25:18, 25:19, 25:20, 25:21, 25:22, 25:23, 25:24, 25:25, 25:26, 25:27, 25:28, 25:29, 25:30, 25:31, 25:32, 25:33, 25:34, 25:35, 25:36, 25:37, 25:38, 25:39, 25:40, 25:41, 25:42, 25:43, 25:44, 25:45, 25:46, 25:47, 25:48, 25:49, 25:50, 25:51, 25:52, 25:53, 25:54, 25:55, 25:56, 25:57, 25:58, 25:59, 25:60, 25:61, 25:62, 25:63, 25:64, 25:65, 25:66, 25:67, 25:68, 25:69, 25:70, 25:71, 25:72, 25:73, 25:74, 25:75, 25:76, 25:77, 25:78, 25:79, 25:80, 25:81, 25:82, 25:83, 25:84, 25:85, 25:86, 25:87, 25:88, 25:89, 25:90, 25:91, 25:92, 25:93, 25:94, 25:95, 25:96, 25:97, 25:98, 25:99, 25:100, 25:101, 25:102, 25:103, 25:104, 25:105, 25:106, 25:107, 25:108, 25:109, 25:110, 25:111, 25:112, 25:113, 25:114, 25:115, 25:116, 25:117, 25:118, 25:119, 25:120, 25:121, 25:122, 25:123, 25:124, 25:125, 25:126, 25:127, 25:128, 25:129, 25:130, 25:131, 25:132, 25:133, 25:134, 25:135, 25:136, 25:137, 25:138, 25:139, 25:140, 25:141, 25:142, 25:143, 25:144, 25:145, 25:146, 25:147, 25:148, 25:149, 25:150, 25:151, 25:152, 25:153, 25:154, 25:155, 25:156, 25:157, 25:158, 25:159, 25:160, 25:161, 25:162, 25:163, 25:164, 25:165, 25:166, 25:167, 25:168, 25:169, 25:170, 25:171, 25:172, 25:173, 25:174, 25:175, 25:176, 25:177, 25:178, 25:179, 25:180, 25:181, 25:182, 25:183, 25:184, 25:185, 25:186, 25:187, 25:188, 25:189, 25:190, 25:191, 25:192, 25:193, 25:194, 25:195, 25:196, 25:197, 25:198, 25:199, 25:200, 25:201, 25:202, 25:203, 25:204, 25:205, 25:206, 25:207, 25:208, 25:209, 25:210, 25:211, 25:212, 25:213, 25:214, 25:215, 25:216, 25:217, 25:218, 25:219, 25:220, 25:221, 25:222, 25:223, 25:224, 25:225, 25:226, 25:227, 25:228, 25:229, 25:230, 25:231, 25:232, 25:233, 25:234, 25:235, 25:236, 25:237, 25:238, 25:239, 25:240, 25:241, 25:242, 25:243, 25:244, 25:245, 25:246, 25:247, 25:248, 25:249, 25:250, 25:251, 25:252, 25:253, 25:254, 25:255, 25:256, 25:257, 25:258, 25:259, 25:260, 25:261, 25:262, 25:263, 25:264, 25:265, 25:266, 25:267, 25:268, 25:269, 25:270, 25:271, 25:272, 25:273, 25:274, 25:275, 25:276, 25:277, 25:278, 25:279, 25:280, 25:281, 25:282, 25:283, 25:284, 25:285, 25:286, 25:287, 25:288, 25:289, 25:290, 25:291, 25:292, 25:293, 25:294, 25:295, 25:296, 25:297, 25:298, 25:299, 25:300, 25:301, 25:302, 25:303, 25:304, 25:305, 25:306, 25:307, 25:308, 25:309, 25:310, 25:311, 25:312, 25:313, 25:314, 25:315, 25:316, 25:317, 25:318, 25:319, 25:320, 25:321, 25:322, 25:323, 25:324, 25:325, 25:326, 25:327, 25:328, 25:329, 25:330, 25:331, 25:332, 25:333, 25:334, 25:335, 25:336, 25:337, 25:338, 25:339, 25:340, 25:341, 25:342, 25:343, 25:344, 25:345, 25:346, 25:347, 25:348, 25:349, 25:350, 25:351, 25:352, 25:353, 25:354, 25:355, 25:356, 25:357, 25:358, 25:359, 25:360, 25:361, 25:362, 25:363,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

25:364, 25:365, 25:366, 25:367,
25:368, 25:369, 25:370, 25:371,
25:372, 25:373, 25:374, 25:375,
25:376, 25:377, 25:378, 25:379,
25:380, 25:381, 25:382, 25:383,
25:384, 25:385, 25:386, 25:387,
25:388, 25:389, 25:390, 25:391,
25:392, 25:393, 25:394, 25:395,
25:396, 25:397, 25:398, 25:399,
25:400, 25:401, 25:402, 25:403,
25:404, 25:405, 25:406, 25:407,
25:408, 25:409, 25:410, 25:411,
25:412, 25:413, 25:414, 25:415,
25:416, 25:417, 25:418, 25:419,
25:420, 25:421, 25:422, 25:423,
25:424, 25:425, 25:426, 25:427,
25:428, 25:429, 25:430, 25:431,
25:432, 25:433, 25:434, 25:435,
25:436, 25:437, 25:438, 25:439,
25:440, 25:441, 25:442, 25:443,
25:444, 25:445, 25:446, 25:447,
25:448, 25:449, 25:450, 25:451,
25:452, 25:453, 25:454, 25:455,
25:456, 25:457, 25:458, 25:459,
25:460, 25:461, 25:462, 25:463,
25:464, 25:465, 25:466, 25:467,
25:468, 25:469, 25:470, 25:471,
25:472, 25:473, 25:474, 25:475,
25:476, 25:477, 25:478, 25:479,
25:480, 25:481, 25:482, 25:483,
25:484, 25:485, 25:486, 25:487,
25:488, 25:489, 25:490, 25:491,
25:492, 25:493, 25:494, 25:495,
25:496, 25:497, 25:498, 25:499,
25:500, 25:501, 25:502, 25:503,
25:504, 25:505, 25:506, 25:507,
25:508, 25:509, 25:510, 25:511,
25:512, 25:513, 25:514, 25:515,
25:516, 25:517, 25:518, 25:519,
25:520, 25:521, 25:522, 25:523,
25:524, 25:525, 25:526, 25:527,
25:528, 25:529, 25:530, 25:531,
25:532, 25:533, 25:534, 25:535,
25:536, 25:537, 25:538, 25:539,
25:540, 25:541, 25:542, 25:543,
25:544, 25:545, 25:546, 25:547,
25:548, 25:549, 25:550, 25:551,
25:552, 25:553, 25:554, 25:555,
25:556, 25:557, 25:558, 25:559,
25:560, 25:561, 25:562, 25:563,
25:564, 25:565, 25:566, 25:567,
25:568, 25:569, 25:570, 25:571,
25:572, 25:573, 25:574, 25:575,
25:576, 25:577, 25:578, 25:579,
25:580, 25:581, 25:582, 25:583,
25:584, 25:585, 25:586, 25:587,
25:588, 25:589, 25:590, 25:591,
25:592, 25:593, 25:594, 25:595,
25:596, 25:597, 25:598, 25:599,
25:600, 25:601, 25:602, 25:603,
25:604, 25:605, 25:606, 25:607,
25:608, 25:609, 25:610, 25:611,
25:612, 25:613, 25:614, 25:615,
25:616, 25:617, 25:618, 25:619,
25:620, 25:621, 25:622, 25:623,
25:624, 25:625, 25:626, 25:627,
25:628, 25:629, 25:630, 25:631,
25:632, 25:633, 25:634, 25:635,
25:636, 25:637, 25:638, 25:639,
25:640, 25:641, 25:642, 25:643,
25:644, 25:645, 25:646, 25:647,
25:648, 25:649, 25:650, 25:651,
25:652, 25:653, 25:654, 25:655,
25:656, 25:657, 25:658, 25:659,
25:660, 25:661, 25:662, 25:663,
25:664, 25:665, 25:666, 25:667,
25:668, 25:669, 25:670, 25:671,
25:672, 25:673, 25:674, 25:675,
25:676, 25:677, 25:678, 25:679,
25:680, 25:681, 25:682, 25:683,
25:684, 25:685, 25:686, 25:687,
25:688, 25:689, 25:690, 25:691,
25:692, 25:693, 25:694, 25:695,
25:696, 25:697, 25:698, 25:699,
25:700, 25:701, 25:702, 25:703,
25:704, 25:705, 25:706, 25:707,
25:708, 25:709, 25:710, 25:711,
25:712, 25:713, 25:714, 25:715,
25:716, 25:717, 25:718, 25:719,
25:720, 25:721, 25:722, 25:723,
25:724, 25:725, 25:726, 25:727,
25:728, 25:729, 25:730, 25:731,
25:732, 25:733, 25:734, 25:735,
25:736, 25:737, 25:738, 25:739,
25:740, 25:741, 25:742, 25:743,
25:744, 25:745, 25:746, 25:747,
25:748, 25:749, 25:750, 25:751,
25:752, 25:753, 25:754, 25:755,
25:756, 25:757, 25:758, 25:759,
25:760, 25:761, 25:762, 25:763,
25:764, 25:765, 25:766, 25:767,
25:768, 25:769, 25:770, 25:771,
25:772, 25:773, 25:774, 25:775,
25:776, 25:777, 25:778, 25:779,
25:780, 25:781, 25:782, 25:783,
25:784, 25:785, 25:786, 25:787,
25:788, 25:789, 25:790, 25:791,
25:792, 25:793, 25:794, 25:795,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

25:796, 25:797, 25:798, 25:799,
25:800, 25:801, 25:802, 25:803,
25:804, 25:805, 25:806, 25:807,
25:808, 25:809, 25:810, 25:811,
25:812, 25:813, 25:814, 25:815,
25:816, 25:817, 25:818, 25:819,
25:820, 25:821, 25:822, 25:823,
25:824, 25:825, 25:826, 25:827,
25:828, 25:829, 25:830, 25:831,
25:832, 25:833, 25:834, 25:835,
25:836, 25:837, 25:838, 25:839,
25:840, 25:841, 25:842, 25:843,
25:844, 25:845, 25:846, 25:847,
25:848, 25:849, 25:850, 25:851,
25:852, 25:853, 25:854, 25:855,
25:856, 25:857, 25:858, 25:859,
25:860, 25:861, 25:862, 25:863,
25:864, 25:865, 25:866, 25:867,
25:868, 25:869, 25:870, 25:871,
25:872, 25:873, 25:874, 25:875,
25:876, 25:877, 25:878, 25:879,
25:880, 25:881, 25:882, 25:883,
25:884, 25:885, 25:886, 25:887,
25:888, 25:889, 25:890, 25:891,
25:892, 25:893, 25:894, 25:895,
25:896, 25:897, 25:898, 25:899,
25:900, 25:901, 25:902, 25:903,
25:904, 25:905, 25:906, 25:907,
25:908, 25:909, 25:910, 25:911,
25:912, 25:913, 25:914, 25:915,
25:916, 25:917, 25:918, 25:919,
25:920, 25:921, 25:922, 25:923,
25:924, 25:925, 25:926, 25:927,
25:928, 25:929, 25:930, 25:931,
25:932, 25:933, 25:934, 25:935,
25:936, 25:937, 25:938, 25:939,
25:940, 25:941, 25:942, 25:943,
25:944, 25:945, 25:946, 25:947,
25:948, 25:949, 25:950, 25:951,
25:952, 25:953, 25:954, 25:955,
25:956, 25:957, 25:958, 25:959,
25:960, 25:961, 25:962, 25:963,
25:964, 25:965, 25:966, 25:967,
25:968, 25:969, 25:970, 25:971,
25:972, 25:973, 25:974, 25:975,
25:976, 25:977, 25:978, 25:979,
25:980, 25:981, 25:982, 25:983,
25:984, 25:985, 25:986, 25:987,
25:988, 25:989, 25:990, 25:991,
25:992, 25:993, 25:994, 25:995,
25:996, 25:997, 25:998, 25:999,
25:1000, 25:1001, 25:1002, 25:1003,
25:1004, 25:1005, 25:1006, 25:1007,
25:1008, 25:1009, 25:1010, 25:1011,
25:1012, 25:1013, 25:1014, 25:1015,
25:1016, 25:1017, 25:1018, 25:1019,
25:1020, 25:1021, 25:1022, 25:1023,
25:1024, 25:1025, 25:1026, 25:1027,
25:1028, 25:1029, 25:1030, 25:1031,
25:1032, 25:1033, 25:1034, 25:1035,
25:1036, 25:1037, 25:1038, 25:1039,
25:1040, 25:1041, 25:1042, 25:1043,
25:1044, 25:1045, 25:1046, 25:1047,
25:1048, 25:1049, 25:1050, 25:1051,
25:1052, 25:1053, 25:1054, 25:1055,
25:1056, 25:1057, 25:1058, 25:1059,
25:1060, 25:1061, 25:1062, 25:1063,
25:1064, 25:1065, 25:1066, 25:1067,
25:1068, 25:1069, 25:1070, 25:1071,
25:1072, 25:1073, 25:1074, 25:1075,
25:1076, 25:1077, 25:1078, 25:1079,
25:1080, 25:1081, 25:1082, 25:1083,
25:1084, 25:1085, 25:1086, 25:1087,
25:1088, 25:1089, 25:1090, 25:1091,
25:1092, 25:1093, 25:1094, 25:1095,
25:1096, 25:1097, 25:1098, 25:1099,
25:1100, 25:1101, 25:1102, 25:1103,
25:1104, 25:1105, 25:1106, 25:1107,
25:1108, 25:1109, 25:1110, 25:1111,
25:1112, 25:1113, 25:1114, 25:1115,
25:1116, 25:1117, 25:1118, 25:1119,
25:1120, 25:1121, 25:1122, 25:1123,
25:1124, 25:1125, 25:1126, 25:1127,
25:1128, 25:1129, 25:1130, 25:1131,
25:1132, 25:1133, 25:1134, 25:1135,
25:1136, 25:1137, 25:1138, 25:1139,
25:1140, 25:1141, 25:1142, 25:1143,
25:1144, 25:1145, 25:1146, 25:1147,
25:1148, 25:1149, 25:1150, 25:1151,
25:1152, 25:1153, 25:1154, 25:1155,
25:1156, 25:1157, 25:1158, 25:1159,
25:1160, 25:1161, 25:1162, 25:1163,
25:1164, 25:1165, 25:1166, 25:1167,
25:1168, 25:1169, 25:1170, 25:1171,
25:1172, 25:1173, 25:1174, 25:1175,
25:1176, 25:1177, 25:1178, 25:1179,
25:1180, 25:1181, 25:1182, 25:1183,
25:1184, 25:1185, 25:1186, 25:1187,
25:1188, 25:1189, 25:1190, 25:1191,
25:1192, 25:1193, 25:1194, 25:1195,
25:1196, 25:1197, 25:1198, 25:1199,
25:1200, 25:1201, 25:1202, 25:1203,
25:1204, 25:1205, 25:1206, 25:1207,
25:1208, 25:1209, 25:1210, 25:1211,
25:1212, 25:1213, 25:1214, 25:1215,
25:1216, 25:1217, 25:1218, 25:1219,
25:1220, 25:1221, 25:1222, 25:1223,
25:1224, 25:1225, 25:1226, 25:1227,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

25:1228, 25:1229, 25:1230, 25:1231,
25:1232, 25:1233, 25:1234, 25:1235,
25:1236, 25:1237, 25:1238, 25:1239,
25:1240, 25:1241, 25:1242, 25:1243,
25:1244, 25:1245, 25:1246, 25:1247,
25:1248, 25:1249, 25:1250, 25:1251,
25:1252, 25:1253, 25:1254, 25:1255,
25:1256, 25:1257, 25:1258, 25:1259,
25:1260, 25:1261, 25:1262, 25:1263,
25:1264, 25:1265, 25:1266, 25:1267,
25:1268, 25:1269, 25:1270, 25:1271,
25:1272, 25:1273, 25:1274, 25:1275,
25:1276, 25:1277, 25:1278, 25:1279,
25:1280, 25:1281, 25:1282, 25:1283,
25:1284, 25:1285, 25:1286, 25:1287,
25:1288, 25:1289, 25:1290, 25:1291,
25:1292, 25:1293, 25:1294, 25:1295,
25:1296, 25:1297, 25:1298, 25:1299,
25:1300, 25:1301, 25:1302, 25:1303,
25:1304, 25:1305, 25:1306, 25:1307,
25:1308, 25:1309, 25:1310, 25:1311,
25:1312, 25:1313, 25:1314, 25:1315,
25:1316, 25:1317, 25:1318, 25:1319,
25:1320, 25:1321, 25:1322, 25:1323,
25:1324, 25:1325, 25:1326, 25:1327,
25:1328, 25:1329, 25:1330, 25:1331,
25:1332, 25:1333, 25:1334, 25:1335,
25:1336, 25:1337, 25:1338, 25:1339,
25:1340, 25:1341, 25:1342, 25:1343,
25:1344, 25:1345, 25:1346, 25:1347,
25:1348, 25:1349, 25:1350, 25:1351,
25:1352, 25:1353, 25:1354, 25:1355,
25:1356, 25:1357, 25:1358, 25:1359,
25:1360, 25:1361, 25:1362, 25:1363,
25:1364, 25:1365, 25:1366, 25:1367,
25:1368, 25:1369, 25:1370, 25:1371,
25:1372, 25:1373, 25:1374, 25:1375,
25:1376, 25:1377, 25:1378, 25:1379,
25:1380, 25:1381, 25:1382, 25:1383,
25:1384, 25:1385, 25:1386, 25:1387,
25:1388, 25:1389, 25:1390, 25:1391,
25:1392, 25:1393, 25:1394, 25:1395,
25:1396, 25:1397, 25:1398, 25:1399,
25:1400, 25:1401, 25:1402, 25:1403,
25:1404, 25:1405, 25:1406, 25:1407,
25:1408, 25:1409, 25:1410, 25:1411,
25:1412, 25:1413, 25:1414, 25:1415,
25:1416, 25:1417, 25:1418, 25:1419,
25:1420, 25:1421, 25:1422, 25:1423,
25:1424, 25:1425, 25:1426, 25:1427,
25:1428, 25:1429, 25:1433, 25:1435,
25:1438, 25:1439, 25:1440, 25:1441,
25:1442, 25:1443, 25:1444, 25:1445,
25:1446, 25:1447, 25:1448, 25:1449,
25:1450, 25:1451, 25:1452, 25:1453,
25:1454, 25:1455, 25:1456, 25:1457,
25:1458, 25:1459, 25:1460, 25:1461,
25:1462, 25:1463, 25:1464, 25:1465,
25:1466, 25:1467, 25:1468, 25:1469,
25:1470, 25:1471, 25:1472, 25:1473,
25:1474, 25:1475, 25:1476, 25:1477,
25:1478, 25:1479, 25:1480, 25:1481,
25:1482, 25:1483, 25:1484, 25:1485,
25:1486, 25:1487, 25:1488, 25:1489,
25:1490, 25:1491, 25:1492, 25:1493,
25:1494, 25:1495, 25:1496, 25:1497,
25:1498, 25:1499, 25:1500, 25:1501,
25:1502, 25:1503, 25:1504, 25:1505,
25:1506, 25:1507, 25:1508, 25:1509,
25:1510, 25:1511, 25:1512, 25:1513,
25:1514, 25:1515, 25:1516, 25:1517,
25:1518, 25:1519, 25:1520, 25:1521,
25:1522, 25:1523, 25:1524, 25:1525,
25:1526, 25:1527, 25:1528, 25:1529,
25:1530, 25:1531, 25:1532, 25:1533,
25:1534, 25:1535, 25:1536, 25:1537,
25:1538, 25:1539, 25:1540, 25:1541,
25:1542, 25:1543, 25:1544, 25:1545,
25:1546, 25:1547, 25:1548, 25:1549,
25:1550, 25:1551, 25:1552, 25:1553,
25:1554, 25:1555, 25:1556, 25:1557,
25:1558, 25:1559, 25:1560, 25:1561,
25:1562, 25:1563, 25:1564, 25:1565,
25:1566, 25:1567, 25:1568, 25:1569,
25:1570, 25:1571, 25:1572, 25:1573,
25:1574, 25:1575, 25:1576, 25:1577,
25:1578, 25:1579, 25:1580, 25:1581,
25:1582, 25:1583, 25:1584, 25:1585,
25:1586, 25:1587, 25:1588, 25:1589,
25:1590, 25:1591, 25:1592, 25:1593,
25:1594, 25:1595, 25:1596, 25:1597,
25:1598, 25:1599, 25:1600, 25:1601,
25:1602, 25:1603, 25:1604, 25:1605,
25:1606, 25:1607, 25:1608, 25:1609,
25:1610, 25:1611, 25:1612, 25:1613,
25:1614, 25:1615, 25:1616, 25:1617,
25:1618, 25:1619, 25:1620, 25:1621,
25:1622, 25:1623, 25:1624, 25:1625,
25:1626, 25:1627, 25:1628, 25:1629,
25:1630, 25:1631, 25:1632, 25:1633,
25:1634, 25:1635, 25:1636, 25:1637,
25:1638, 25:1639, 25:1640, 25:1641,
25:1642, 25:1643, 25:1644, 25:1645,
25:1646, 25:1647, 25:1648, 25:1649,
25:1650, 25:1651, 25:1652, 25:1653,
25:1654, 25:1655, 25:1656, 25:1657,
25:1658, 25:1659, 25:1660, 25:1661,
25:1662, 25:1663, 25:1664, 25:1665,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

25:1666, 25:1667, 25:1668, 25:1669,
25:1670, 25:1671, 25:1672, 25:1673,
25:1674, 25:1675, 25:1676, 25:1677,
25:1678, 25:1679, 25:1680, 25:1681,
25:1682, 25:1683, 25:1684, 25:1685,
25:1686, 25:1687, 25:1688, 25:1689,
25:1690, 25:1691, 25:1692, 25:1693,
25:1694, 25:1695, 25:1696, 25:1697,
25:1698, 25:1699, 25:1700, 25:1701,
25:1702, 25:1703, 25:1704, 25:1705,
25:1706, 25:1707, 25:1708, 25:1709,
25:1710, 25:1711, 25:1712, 25:1713,
25:1714, 25:1715, 25:1716, 25:1717,
25:1718, 25:1719, 25:1720, 25:1721,
25:1722, 25:1723, 25:1724, 25:1725,
25:1726, 25:1727, 25:1728, 25:1729,
25:1730, 25:1731, 25:1732, 25:1733,
25:1734, 25:1735, 25:1736, 25:1737,
25:1738, 25:1739, 25:1740, 25:1741,
25:1742, 25:1743, 25:1744, 25:1745,
25:1746, 25:1747, 25:1748, 25:1749,
25:1750, 25:1751, 25:1752, 25:1753,
25:1754, 25:1755, 25:1756, 25:1757,
25:1758, 25:1759, 25:1760, 25:1761,
25:1762, 25:1763, 25:1764, 25:1765,
25:1766, 25:1767, 25:1768, 25:1769,
25:1770, 25:1771, 25:1772, 25:1773,
25:1774, 25:1775, 25:1776, 25:1777,
25:1778, 25:1779, 25:1780, 25:1781,
25:1782, 25:1783, 25:1784, 25:1785,
25:1786, 25:1787, 25:1788, 25:1789,
25:1790, 25:1791, 25:1792, 25:1793,
25:1794, 25:1795, 25:1796, 25:1797,
25:1798, 25:1799, 25:1800, 25:1801,
25:1802, 25:1803, 25:1804, 25:1805,
25:1806, 25:1807, 25:1808, 25:1809,
25:1810, 25:1811, 25:1812, 25:1813,
25:1814, 25:1815, 25:1816, 25:1817,
25:1818, 25:1819, 25:1820, 25:1821,
25:1822, 25:1823, 25:1824, 25:1825,
25:1826, 25:1827, 25:1828, 25:1829,
25:1830, 25:1831, 25:1832, 25:1833,
25:1834, 25:1835, 25:1836, 25:1837,
25:1838, 25:1839, 25:1840, 25:1841,
25:1842, 25:1843, 25:1844, 25:1845,
25:1846, 25:1847, 25:1848, 25:1849,
25:1850, 25:1851, 25:1852, 25:1853,
25:1854, 25:1855, 25:1856, 25:1857,
25:1858, 25:1859, 25:1860, 25:1861,
25:1862, 25:1863, 25:1864, 25:1865,
25:1866, 25:1867, 25:1868, 25:1869,
25:1870, 25:1871, 25:1872, 25:1873,
25:1874, 25:1875, 25:1876, 25:1877,
25:1878, 25:1879, 25:1880, 25:1881,
25:1882, 25:1883, 25:1884, 25:1885,
25:1886, 25:1887, 25:1888, 25:1889,
25:1890, 25:1891, 25:1892, 25:1893,
25:1894, 25:1895, 25:1896, 25:1897,
25:1898, 25:1899, 25:1900, 25:1901,
25:1902, 25:1903, 25:1904, 25:1905,
25:1906, 25:1907, 25:1908, 25:1909,
25:1910, 25:1911, 25:1912, 25:1913,
25:1914, 25:1915, 25:1916, 25:1917,
25:1918, 25:1919, 25:1920, 25:1921,
25:1922, 25:1923, 25:1924, 25:1925,
25:1926, 25:1927, 25:1928, 25:1929,
25:1930, 25:1931, 25:1932, 25:1933,
25:1934, 25:1935, 25:1936, 25:1937,
25:1938, 25:1939, 25:1940, 25:1941,
25:1942, 25:1943, 25:1944, 25:1945,
25:1946, 25:1947, 25:1948, 25:1949,
25:1950, 25:1951, 25:1952, 25:1953,
25:1954, 25:1955, 25:1956, 25:1957,
25:1958, 25:1959, 25:1960, 25:1961,
25:1962, 25:1963, 25:1964, 25:1965,
25:1966, 25:1967, 25:1968, 25:1969,
25:1970, 25:1971, 25:1972, 25:1973,
25:1974, 25:1975, 25:1976, 25:1977,
25:1978, 25:1979, 25:1980, 25:1981,
25:1982, 25:1983, 25:1984, 25:1985,
25:1986, 25:1987, 25:1988, 25:1989,
25:1990, 25:1991, 25:1992, 25:1993,
25:1994, 25:1995, 25:1996, 25:1997,
25:1998, 25:1999, 25:2000, 25:2001,
25:2002, 25:2003, 25:2004, 25:2005,
25:2006, 25:2007, 25:2008, 25:2009,
25:2010, 25:2011, 25:2012, 25:2013,
25:2014, 25:2015, 25:2016, 25:2017,
25:2018, 25:2019, 25:2020, 25:2021,
25:2022, 25:2023, 25:2024, 25:2025,
25:2026, 25:2027, 25:2028, 25:2029,
25:2030, 25:2031, 25:2032, 25:2033,
25:2034, 25:2035, 25:2036, 25:2037,
25:2038, 25:2039, 25:2040, 25:2041,
25:2042, 25:2043, 25:2044, 25:2045,
25:2046, 25:2047, 25:2048, 25:2049,
25:2050, 25:2051, 25:2052, 25:2053,
25:2054, 25:2055, 25:2056, 25:2057,
25:2058, 25:2059, 25:2060, 25:2061,
25:2062, 25:2063, 25:2064, 25:2065,
25:2066, 25:2067, 25:2068, 25:2069,
25:2070, 25:2071, 25:2072, 25:2073,
25:2074, 25:2075, 25:2076, 25:2077,
25:2078, 25:2079, 25:2080, 25:2081,
25:2082, 25:2083, 25:2084, 25:2085,
25:2086, 25:2087, 25:2088, 25:2089,
25:2090, 25:2091, 25:2092, 25:2093,
25:2094, 25:2095, 25:2096, 25:2097,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

25:2098, 25:2099, 25:2100, 25:2101, 25:2102, 25:2103, 25:2104, 25:2105, 25:2106, 25:2107, 25:2108, 25:2109, 25:2110, 25:2111, 25:2112, 25:2113, 25:2114, 25:2115, 25:2116, 25:2117, 25:2118, 25:2119, 25:2120, 25:2121, 25:2122, 25:2123, 25:2124, 25:2125, 25:2126, 25:2127, 25:2128, 25:2129, 25:2130, 25:2131, 25:2132, 25:2133, 25:2134, 25:2135, 25:2136, 25:2137, 25:2138, 25:2139, 25:2140, 25:2141, 25:2142, 25:2143, 25:2144, 25:2145, 25:2146, 25:2147, 25:2148, 25:2149, 25:2150, 25:2151, 25:2152, 25:2153, 25:2154, 25:2155, 25:2156, 25:2157, 25:2158, 25:2159, 25:2160, 25:2161, 25:2162, 25:2163, 25:2164, 25:2165, 25:2166, 25:2167, 25:2168, 25:2169, 25:2170, 25:2171, 25:2172, 25:2173, 25:2174, 25:2175, 25:2176, 25:2177, 25:2178, 25:2179, 25:2180, 25:2181, 25:2182, 25:2183, 25:2184, 25:2185, 25:2186, 25:2187, 25:2188, 25:2189, 25:2190, 25:2191, 25:2192, 25:2193, 25:2194, 25:2195, 25:2196, 25:2197, 25:2198, 25:2199, 25:2200, 25:2201, 25:2202, 25:2203, 25:2204, 25:2205, 25:2206, 25:2207, 25:2208, 25:2209, 25:2210, 25:2211, 25:2212, 25:2213, 25:2214, 25:2215, 25:2216, 25:2217, 25:2218, 25:2219, 25:2220, 25:2221, 25:2222, 25:2223, 25:2224, 25:2225, 25:2226, 25:2227, 25:2228, 25:2229, 25:2230, 25:2231, 25:2232, 25:2233, 25:2234, 25:2235, 25:2236, 25:2237, 25:2238, 25:2239, 25:2240, 25:2241, 25:2242, 25:2243, 25:2244, 25:2245, 25:2246, 25:2247, 25:2248, 25:2249, 25:2250, 25:2251, 25:2252, 25:2253, 25:2254, 25:2255, 25:2256, 25:2257, 25:2258, 25:2259, 25:2260, 25:2261, 25:2262, 25:2263, 25:2264, 25:2265, 25:2266, 25:2267, 25:2268, 25:2269, 25:2270, 25:2271, 25:2272, 25:2273, 25:2274, 25:2275, 25:2276, 25:2277, 25:2278, 25:2279, 25:2280, 25:2281, 25:2282, 25:2283, 25:2284, 25:2285, 25:2286, 25:2287, 25:2288, 25:2289, 25:2290, 25:2291, 25:2292, 25:2293, 25:2294, 25:2295, 25:2296, 25:2297, 25:2298, 25:2299, 25:2300, 25:2301, 25:2302, 25:2303, 25:2304, 25:2305, 25:2306, 25:2307, 25:2308, 25:2309, 25:2310, 25:2311, 25:2312, 25:2313, 25:2314, 25:2315, 25:2316, 25:2317, 25:2318, 25:2319, 25:2320, 25:2321, 25:2322, 25:2323, 25:2324, 25:2325, 25:2326, 25:2327, 25:2328, 25:2329, 25:2330, 25:2331, 25:2332, 25:2333, 25:2334, 25:2335, 25:2336, 25:2337, 25:2338, 25:2339, 25:2340, 25:2341, 25:2342, 25:2343, 25:2344, 25:2345, 25:2346, 25:2347, 25:2348, 25:2349, 25:2350, 25:2351, 25:2352, 25:2353, 25:2354, 25:2355, 25:2356, 25:2357, 25:2358, 25:2359, 25:2360, 25:2361, 25:2362, 25:2363, 25:2364, 25:2365, 25:2366, 25:2367, 25:2368, 25:2369, 25:2370, 25:2371, 25:2372, 25:2373, 25:2374, 25:2375, 25:2376, 25:2377, 25:2378, 25:2379, 25:2380, 25:2381, 25:2382, 25:2383, 25:2384, 25:2385, 25:2386, 25:2387, 25:2388, 25:2389, 25:2390, 25:2391, 25:2392, 25:2393, 25:2394, 25:2395, 25:2396, 25:2397, 25:2398, 25:2399, 25:2400, 25:2401, 25:2402, 25:2403, 25:2404, 25:2405, 25:2406, 25:2407, 25:2408, 25:2409, 25:2410, 25:2411, 25:2412, 25:2413, 25:2414, 25:2415, 25:2416, 25:2417, 25:2418, 25:2419, 25:2420, 25:2421, 25:2422, 25:2423, 25:2424, 25:2425, 25:2426, 25:2427, 25:2428, 25:2429, 25:2430, 25:2431, 25:2432, 25:2433, 25:2434, 25:2435, 25:2436, 25:2437, 25:2438, 25:2439, 25:2440, 25:2441, 25:2442, 25:2443, 25:2444, 25:2445, 25:2446, 25:2447, 25:2448, 25:2449, 25:2450, 25:2451, 25:2452, 25:2453, 25:2454, 25:2455, 25:2456, 25:2457, 25:2458, 25:2459, 25:2460, 25:2461, 25:2462, 25:2463, 25:2464, 25:2465, 25:2466, 25:2467, 25:2468, 25:2469, 25:2470, 25:2471, 25:2472, 25:2473, 25:2474, 25:2475, 25:2476, 25:2477, 25:2478, 25:2479, 25:2480, 25:2481, 25:2482, 25:2483, 25:2484, 25:2485, 25:2486, 25:2487, 25:2488, 25:2489, 25:2490, 25:2491, 25:2492, 25:2493, 25:2494, 25:2495, 25:2496, 25:2497, 25:2498, 25:2499, 25:2500, 25:2501, 25:2502, 25:2503, 25:2504, 25:2505, 25:2506, 25:2507, 25:2508, 25:2509, 25:2510, 25:2511, 25:2512, 25:2513, 25:2514, 25:2515, 25:2516, 25:2517, 25:2518, 25:2519, 25:2520, 25:2521, 25:2522, 25:2523, 25:2524, 25:2525, 25:2526, 25:2527, 25:2528, 25:2529,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

25:2530, 25:2531, 25:2532, 25:2533, 25:2534, 25:2535, 25:2536, 25:2537, 25:2538, 25:2539, 25:2540, 25:2541, 25:2542, 25:2543, 25:2544, 25:2545, 25:2546, 25:2547, 25:2548, 25:2549, 25:2550, 25:2551, 25:2552, 25:2553, 25:2554, 25:2555, 25:2556, 25:2557, 25:2558, 25:2559, 25:2560, 25:2561, 25:2562, 25:2563, 25:2564, 25:2565, 25:2566, 25:2567, 25:2568, 25:2569, 25:2570, 25:2571, 25:2572, 25:2573, 25:2574, 25:2575, 25:2576, 25:2577, 25:2578, 25:2579, 25:2580, 25:2581, 25:2582, 25:2583, 25:2584, 25:2585, 25:2586, 25:2587, 25:2588, 25:2589, 25:2590, 25:2591, 25:2592, 25:2593, 25:2594, 25:2595, 25:2596, 25:2597, 25:2598, 25:2599, 25:2600, 25:2601, 25:2602, 25:2603, 25:2604, 25:2605, 25:2606, 25:2607, 25:2608, 25:2609, 25:2610, 25:2611, 25:2612, 25:2613, 25:2614, 25:2615, 25:2616, 25:2617, 25:2618, 25:2619, 25:2620, 25:2621, 25:2622, 25:2623, 25:2624, 25:2625, 25:2626, 25:2627, 25:2628, 25:2629, 25:2630, 25:2631, 25:2632, 25:2633, 25:2634, 25:2635, 25:2636, 25:2637, 25:2638, 25:2639, 25:2640, 25:2641, 25:2642, 25:2643, 25:2644, 25:2645, 25:2646, 25:2647, 25:2648, 25:2649, 25:2650, 25:2651, 25:2652, 25:2653, 25:2654, 25:2655, 25:2656, 25:2657, 25:2658, 25:2659, 25:2660, 25:2661, 25:2662, 25:2663, 25:2664, 25:2665, 25:2666, 25:2667, 25:2668, 25:2669, 25:2670, 25:2671, 25:2672, 25:2673, 25:2674, 25:2675, 25:2676, 25:2677, 25:2678, 25:2679, 25:2680, 25:2681, 25:2682, 25:2683, 25:2684, 25:2685, 25:2686, 25:2687, 25:2688, 25:2689, 25:2690, 25:2691, 25:2692, 25:2693, 25:2694, 25:2695, 25:2696, 25:2697, 25:2698, 25:2699, 25:2700, 25:2701, 25:2702, 25:2703, 25:2704, 25:2705, 25:2706, 25:2707, 25:2708, 25:2709, 25:2710, 25:2711, 25:2712, 25:2713, 25:2714, 25:2715, 25:2716, 25:2717, 25:2718, 25:2719, 25:2720, 25:2721, 25:2722, 25:2723, 25:2724, 25:2725, 25:2726, 25:2727, 25:2728, 25:2729, 25:2730, 25:2731, 25:2732, 25:2733, 25:2734, 25:2735, 25:2736, 25:2737, 25:2738, 25:2739, 25:2740, 25:2741, 25:2742, 25:2743, 25:2744, 25:2745, 25:2746, 25:2747, 25:2748, 25:2749, 25:2750, 25:2751, 25:2752, 25:2753, 25:2754, 25:2755, 25:2756, 25:2757, 25:2758, 25:2759, 25:2760, 25:2761, 25:2762, 25:2763, 25:2764, 25:2765, 25:2766, 25:2767, 25:2768, 25:2769, 25:2770, 25:2774, 25:2776, *1438*

`\DeclareFontSeriesDefault` *736*

`\DeclareFontSeriesDefault` 29:35, 29:36, 29:70, 29:72, 29:73, 29:83, 29:94, 29:103, 29:105, *736*

`\DeclareFontShape` 24:19, 24:543, 24:544, 24:545, 24:546, 24:547, 24:548, 24:549, 24:550, 24:551, 24:552, 24:553, 24:554, 24:555, 24:556, 24:557, 24:558, 24:559, 24:560, 24:561, 24:562, 24:563, 27:25, 27:27, 27:81, 27:82, *1360*

`\DeclareFontShapeChangeRule` 25:2912, 25:2934, 25:2947, 25:2949, 25:2950, 25:2951, 25:2952, 25:2953, 25:2954, 25:2955, 25:2956, 25:2957, 25:2958, 25:2959, 25:2960, 25:2961, 25:2962, 25:2963, 25:2964, 25:2965, 25:2966, 25:2967, 25:2968, 25:2969, 25:2970, 25:2971, 25:2972, 25:2973, 25:2974, 25:2975, 25:2976, 25:2977, 25:2978, 25:2979, 25:2980, 25:2981, 25:2982, 25:2983, 25:2984, 25:2985, 25:2986, 25:2987, 25:2988, 25:2989, 25:2990, 25:2991, 25:2992, 25:2993, 25:2994, 25:2995, 25:2996, 25:2997, 25:2998, 25:2999, 25:3000, 25:3001, 25:3002, 25:3003, 25:3004, 25:3005, 25:3006, 25:3007, 25:3008, 25:3009, 25:3010, 25:3011, 25:3012, 25:3013, 25:3014, 25:3015, 25:3016, 25:3017, 25:3018, 25:3019, 25:3020, 25:3021, 25:3022, 25:3023, 25:3024, 25:3025, 25:3026, 25:3027, 25:3028, 25:3029, 25:3030, 25:3031, 25:3032, 25:3033, 25:3034, 25:3035, 25:3036, 25:3037, 25:3038, 25:3039, 25:3040, 25:3041, 25:3042, 25:3043, 25:3044, 25:3045, 25:3049, 25:3051, 25:3052, 25:3053, 25:3054, 25:3055, 25:3056, 25:3057, 25:3058, 25:3059, 25:3060, 25:3061, 25:3062, 25:3063, 25:3064, 25:3065, 25:3066, 25:3067, 25:3068, 25:3069, 25:3070, 25:3071, 25:3072, 25:3073, 25:3074, 25:3075, 25:3076, 25:3077, 25:3078, 25:3079, 25:3080, 25:3081, 25:3082, 25:3083, 25:3084, 25:3085, 25:3086, 25:3087, 25:3088, 25:3090, 25:3091,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

25:3092, 25:3093, 25:3095, 25:3096, 25:3097, 25:3099, 25:3100, 25:3101, 25:3102, 25:3103, 25:3104, 25:3106, 25:3107, 25:3108, 25:3109, 25:3110, 25:3111, 25:3112, 25:3114, 25:3115, 25:3116, 25:3117, 25:3118, 25:3119, 25:3121, 25:3122, 25:3123, 25:3124, 25:3125, 25:3126, 25:3127, 25:3130
\DeclareFontSubstitution 21:398, 21:486, 21:769, 21:855, 24:142, 30:26, 30:34, 30:37, 30:38, 30:39, 30:141, 30:142, 30:143, 30:144, *1439*
\DeclareHookRule ... 08:2848, 08:2956, 37:52, 37:53, 37:117, 37:118, *220*
\DeclareInstance 11:1549, *376*
\DeclareInstanceCopy 11:1549, *376*
\DeclareKeys 51:214, *1150*
\DeclareLowercaseExclusions 57:620, *1439*
\DeclareLowercaseMapping ... 57:620, *1430*
\DeclareMathAccent 28:860, 30:527, 30:528, 30:529, 30:530, 30:531, 30:532, 30:533, 30:534, 30:535, 30:536, 30:537, 30:538, 30:539
\DeclareMathAlphabet 27:119, 27:123, 27:125, 27:132, 28:686, 28:849, 30:162, 30:163, 30:164, 30:165
\DeclareMathAlphabetCharacter . 28:1035
\DeclareMathDelimiter 28:1037, 30:266, 30:267, 30:268, 30:269, 30:270, 30:271, 30:274, 30:276, 30:277, 30:574, 30:576, 30:578, 30:580, 30:582, 30:585, 30:587, 30:589, 30:591, 30:593, 30:595, 30:597, 30:599, 30:601, 30:603, 30:605, 30:607, 30:609, 30:611, 30:613, 30:615, 30:617, 30:619, 30:621, 30:623, *1401*
\DeclareMathOperator *1386*
\DeclareMathRadical 28:1172, 30:540
\DeclareMathScriptfontMapping 26:354, *1440*
\DeclareMathSizes 24:268, 24:274, 24:296, 30:168, 30:169, 30:170, 30:171, 30:172, 30:173, 30:174, 30:175, 30:176, 30:177, 30:178, 30:179, *1384*
\DeclareMathSizes* 24:268
\DeclareMathSymbol 28:973, 28:1036, 28:1053, 30:180, 30:181, 30:182, 30:183, 30:184, 30:185, 30:186, 30:187, 30:188, 30:189, 30:190, 30:191, 30:192, 30:193, 30:194, 30:195, 30:196, 30:197, 30:198, 30:199, 30:200, 30:201, 30:202, 30:203, 30:204, 30:205, 30:206, 30:207, 30:208, 30:209, 30:210, 30:211, 30:212, 30:213, 30:214, 30:215, 30:216, 30:217, 30:218, 30:219, 30:220, 30:221, 30:222, 30:223, 30:224, 30:225, 30:226, 30:227, 30:228, 30:229, 30:230, 30:231, 30:232, 30:233, 30:234, 30:235, 30:236, 30:237, 30:238, 30:239, 30:240, 30:241, 30:242, 30:243, 30:244, 30:245, 30:246, 30:247, 30:248, 30:249, 30:250, 30:251, 30:252, 30:253, 30:254, 30:255, 30:256, 30:257, 30:258, 30:259, 30:260, 30:261, 30:262, 30:272, 30:273, 30:275, 30:279, 30:280, 30:281, 30:282, 30:283, 30:284, 30:285, 30:286, 30:287, 30:288, 30:289, 30:290, 30:291, 30:292, 30:293, 30:294, 30:295, 30:296, 30:297, 30:298, 30:299, 30:300, 30:301, 30:302, 30:303, 30:304, 30:305, 30:306, 30:307, 30:308, 30:309, 30:310, 30:311, 30:312, 30:313, 30:314, 30:315, 30:316, 30:317, 30:318, 30:319, 30:320, 30:321, 30:322, 30:323, 30:324, 30:325, 30:326, 30:327, 30:328, 30:329, 30:330, 30:331, 30:332, 30:333, 30:334, 30:335, 30:336, 30:338, 30:339, 30:340, 30:341, 30:342, 30:343, 30:344, 30:345, 30:352, 30:353, 30:354, 30:355, 30:356, 30:357, 30:358, 30:360, 30:361, 30:362, 30:363, 30:364, 30:365, 30:367, 30:368, 30:369, 30:370, 30:371, 30:372, 30:375, 30:376, 30:377, 30:378, 30:381, 30:382, 30:383, 30:384, 30:385, 30:386, 30:387, 30:388, 30:389, 30:390, 30:391, 30:392, 30:393, 30:394, 30:395, 30:396, 30:397, 30:398, 30:399, 30:400, 30:401, 30:402, 30:403, 30:404, 30:405, 30:406, 30:407, 30:408, 30:409, 30:410, 30:411, 30:412, 30:413, 30:414, 30:415, 30:416, 30:417, 30:418, 30:419, 30:420, 30:421, 30:422, 30:425, 30:426, 30:429, 30:430, 30:431, 30:432, 30:433, 30:434, 30:435, 30:436, 30:437, 30:438, 30:439, 30:440, 30:441, 30:443, 30:444, 30:445, 30:446, 30:447,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

30:448, 30:449, 30:452, 30:453, 30:454, 30:456, 30:457, 30:458, 30:459, 30:460, 30:461, 30:462, 30:463, 30:464, 30:465, 30:466, 30:488, 30:490, 30:512, 30:513, 30:514, 30:564, 30:565, 30:566, 30:567, 30:625, 30:626, 30:627, *779*
\DeclareMathVersion . . . 28:468, 29:3, 29:4
\DeclareOldFontCommand . . 32:126, 32:142
\DeclareOption *1098*
\DeclareOption 21:1563, 26:30, 26:38, 26:46, 26:54, 26:57, 26:61, 33:782, 33:783, 33:784, 33:785, 33:786, 33:788, 33:790, 33:792, 33:793, 33:833, 33:834, 33:835, 33:836, 33:837, 33:839, 33:840, 50:531, 50:1236, *1388*
\DeclareOption* *1098*
\DeclareOption* 50:531
\DeclarePreloadSizes 24:248, 27:95, 27:96, 31:19, 31:21, 31:22, 31:23, 31:25, 31:26, 31:27, 31:28, 31:29, 31:30, 31:34, 31:38, 31:43, 31:45, 31:49, 31:50, 31:53, 31:54, 31:57, 31:58, 31:64
\DeclareProtectedCommand *1375*
\DeclareRelease . . 33:775, 33:776, 50:1714
DeclareRelease commands:
\DeclareRelease: 50:1717
\DeclareRobustCommand 14:4, 14:11, 14:30, 14:57, 18:7, 18:8, 18:9, 18:10, 18:11, 18:69, 18:93, 18:399, 18:475, 18:489, 18:534, 18:559, 19:2, 19:3, 19:13, 20:488, 20:635, 21:168, 21:176, 21:327, 21:330, 21:331, 21:332, 21:333, 21:334, 21:336, 21:338, 21:340, 22:356, 23:32, 23:33, 23:34, 24:314, 24:342, 24:343, 24:344, 24:349, 24:361, 24:371, 24:379, 24:381, 24:399, 24:738, 24:748, 25:2782, 25:2786, 25:2795, 25:2797, 25:2807, 25:2914, 25:2919, 25:2924, 25:3135, 25:3138, 25:3146, 25:3147, 25:3153, 25:3233, 26:116, 26:165, 29:5, 29:8, 29:11, 29:14, 29:17, 29:20, 29:23, 29:26, 29:29, 29:333, 29:356, 29:386, 29:407, 29:431, 29:442, 29:459, 29:462, 29:484, 29:489, 29:494, 29:527, 29:532, 29:537, 29:553, 29:556, 29:559, 29:562, 29:565, 29:579, 29:632, 29:633, 29:668, 29:674, 29:684, 29:706, 29:724, 29:726, 29:735, 29:737, 29:744, 29:760, 29:768, 29:801, 29:817, 29:833, 29:850, 29:866, 29:873, 30:346, 30:347, 30:348, 30:359, 30:366, 30:423, 30:424, 30:455, 30:467, 30:471, 30:474, 30:479, 30:481, 30:483, 30:486, 30:489, 30:491, 30:492, 30:494, 30:496, 30:498, 30:500, 30:502, 30:504, 30:506, 30:508, 30:510, 30:516, 30:518, 30:520, 30:523, 30:541, 30:544, 30:547, 30:551, 30:555, 30:558, 30:561, 30:568, 30:571, 30:628, 30:629, 30:630, 30:635, 30:637, 30:639, 30:641, 32:3, 32:127, 33:4, 33:11, 33:574, 33:1097, 35:213, 35:219, 06:274, 37:271, 37:452, 37:457, 37:462, 37:472, 37:476, 37:480, 37:578, 37:582, 38:3, 38:4, 38:5, 38:6, 38:7, 38:8, 38:9, 38:10, 38:11, 38:12, 38:13, 38:14, 38:15, 38:16, 38:17, 38:18, 38:19, 38:20, 38:21, 38:22, 38:23, 38:24, 38:25, 38:26, 38:27, 38:28, 38:29, 38:30, 38:31, 38:32, 38:33, 38:34, 38:35, 38:39, 38:41, 38:42, 38:43, 38:44, 38:45, 38:46, 38:47, 38:48, 38:49, 38:50, 38:51, 38:52, 38:81, 38:82, 38:83, 38:84, 38:126, 38:167, 38:169, 38:173, 38:218, 38:220, 38:222, 38:224, 38:227, 38:228, 38:230, 38:237, 38:239, 38:240, 38:251, 38:274, 38:276, 38:309, 38:322, 38:337, 38:348, 38:411, 38:412, 38:413, 38:522, 38:556, 38:580, 40:7, 40:24, 40:164, 40:177, 40:200, 40:201, 40:217, 40:228, 40:307, 40:597, 40:619, 40:657, 40:671, 40:726, 40:727, 40:728, 40:729, 40:730, 40:731, 41:139, 41:142, 41:154, 42:135, 42:138, 42:141, 44:7, 44:8, 44:9, 44:10, 44:14, 44:247, 45:402, 45:454, 47:16, 47:28, 48:520, 48:521, 49:23, 49:28, 49:38, 49:49, 49:64, 49:72, 49:126, 49:128, 49:130, 49:137, 50:1171, 50:1172, 50:1173, 50:1179, 50:1180, 06:925, 06:926, 06:932, 06:947, 50:1838, 52:148, 52:184, 53:501, 54:63, 57:786, *1377*
\DeclareSizeFunction 26:434, 26:507, 26:508, 26:519, 26:520, 26:524, 26:525, 26:531, 26:532, 26:560, 26:574, 26:575, 26:582, 26:583
\DeclareStrongEmphSequence *756*
\DeclareStrongEmphSequence . . . . . .

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

. . . . . 29:680, 29:707, 57:787
\DeclareSymbolFont 27:136, 28:527, 30:152, 30:153, 30:154, 30:155
\DeclareSymbolFontAlphabet 28:1246, 30:159, 30:160, 30:161
\DeclareTemplateCode 11:1353, 11:1409, 11:1435, 11:1549, *373*
\DeclareTemplateCopy 11:1549, *374*
\DeclareTemplateInterface . . 11:1549, *371*
\DeclareTemplateInterfaceEquivalents 11:1569, *378*
\DeclareTextAccent 21:82, 21:399, 21:400, 21:401, 21:402, 21:403, 21:404, 21:405, 21:406, 21:407, 21:408, 21:409, 21:487, 21:488, 21:489, 21:490, 21:491, 21:492, 21:493, 21:494, 21:495, 21:496, 21:497, 21:765, 21:770, 21:771, 21:772, 21:773, 21:774, 21:775, 21:776, 21:777, 21:778, 21:779, 21:780, 21:862, 21:863, 21:864, 21:865, 21:866, 21:867, 21:868, 21:869, 21:870, 21:871, 21:872, 21:873, 21:874, 21:875, 21:876
\DeclareTextAccentDefault 21:203, 21:246, 21:247, 21:248, 21:249, 21:250, 21:251, 21:252, 21:253, 21:254, 21:255, 21:256, 21:257, 21:258, 21:259, 21:299, 21:302, 33:651, 33:652, 33:901, 33:902, 33:903, 33:904, 33:905, 33:906, 33:907, 33:908, 33:909, 33:910, 33:911, 33:912, 33:913, *1381*
\DeclareTextCommand 21:3, 21:76, 21:83, 21:101, 21:410, 21:413, 21:416, 21:430, 21:431, 21:432, 21:435, 21:436, 21:443, 21:445, 21:447, 21:449, 21:455, 21:457, 21:459, 21:466, 21:498, 21:501, 21:505, 21:508, 21:510, 21:513, 21:515, 21:517, 21:533, 21:591, 21:592, 21:593, 21:754, 21:781, 21:783, 21:786, 21:789, 21:822, 21:829, 21:856, 21:859, 21:889, 21:917, 21:1081, 21:1108, 33:340, 33:341, 33:342, 33:343, 33:344, 33:345, 33:346, 33:347, 33:348, 33:349, 33:589, 33:596, 33:603, 33:723, *1377*
\DeclareTextCommandDefault . . . 21:75, 21:204, 21:206, 21:303, 21:306, 21:307, 21:308, 21:310, 21:312, 21:316, 21:320, 21:321, 21:323, 21:324, 21:325, 21:326, 21:346, 21:375, 33:119, 33:121, 33:124, 33:126, 33:128, 33:130, 33:132, 33:134, 33:136, 33:138, 33:140, 33:142, 33:144, 33:146, 33:148, 33:150, 33:152, 33:154, 33:157, 33:158, 33:159, 33:160, 33:161, 33:162, 33:163, 33:164, 33:165, 33:166, 33:167, 33:168, 33:169, 33:170, 33:171, 33:172, 33:174, 33:176, 33:178, 33:180, 33:182, 33:184, 33:186, 33:188, 33:190, 33:192, 33:194, 33:196, 33:198, 33:200, 33:202, 33:204, 33:206, 33:208, 33:210, 33:212, 33:214, 33:216, 33:218, 33:220, 33:222, 33:224, 33:226, 33:228, 33:230, 33:232, 33:234, 33:236, 33:238, 33:240, 33:242, 33:244, 33:246, 33:248, 33:250, 33:252, 33:254, 33:256, 33:258, 33:260, 33:262, 33:264, 33:266, 33:268, 33:270, 33:273, 33:275, 33:277, 33:279, 33:281, 33:283, 33:285, 33:287, 33:289, 33:291, 33:293, 33:295, 33:297, 33:299, 33:301, 33:303, 33:305, 33:307, 33:309, 33:311, 33:313, 33:315, 33:317, 33:319, 33:321, 33:323, 33:325, 33:327, 33:329, 33:618, 33:626, 33:628, 33:634, 33:642, 33:967, 33:969, 33:970, 33:972, 33:974, 33:976, 33:978, 33:980, 33:982, 33:984, 33:986, 33:988, 33:990, 33:992, 33:994, 33:996, 33:998, 33:1000, 33:1002, 33:1004, 33:1006, 33:1008, 33:1010, 33:1012, 33:1014, 33:1016, 33:1018, 33:1020, 33:1022, 33:1024, 33:1026, 33:1028, 33:1030, 33:1032, 33:1034, 33:1036, 33:1038, 33:1040, 33:1042, 33:1044, 33:1046, 33:1048, 33:1050, 33:1052, 33:1054, 33:1056, 33:1058, 33:1060, 33:1062, 33:1064, 33:1066, 33:1068, 33:1070, 33:1072, 33:1074, 33:1076, 33:1078, 33:1080, 33:1082, 33:1084, 33:1087, *1381*
\DeclareTextComposite . 21:94, 21:473, 21:474, 21:610, 21:611, 21:612, 21:613, 21:614, 21:615, 21:616, 21:617, 21:618, 21:619, 21:620, 21:621, 21:622, 21:623, 21:624, 21:625, 21:626, 21:627, 21:628, 21:629, 21:630, 21:631, 21:632, 21:633, 21:634, 21:635, 21:636, 21:637, 21:638, 21:639, 21:640, 21:641, 21:642, 21:643, 21:644,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

21:645, 21:646, 21:647, 21:648, 21:649, 21:650, 21:651, 21:652, 21:653, 21:654, 21:655, 21:656, 21:657, 21:658, 21:659, 21:660, 21:661, 21:662, 21:663, 21:664, 21:665, 21:666, 21:667, 21:668, 21:669, 21:670, 21:671, 21:672, 21:673, 21:674, 21:675, 21:676, 21:677, 21:678, 21:679, 21:680, 21:681, 21:682, 21:683, 21:684, 21:685, 21:686, 21:687, 21:688, 21:689, 21:690, 21:691, 21:692, 21:693, 21:694, 21:695, 21:696, 21:697, 21:698, 21:699, 21:700, 21:701, 21:702, 21:703, 21:704, 21:705, 21:706, 21:707, 21:708, 21:709, 21:710, 21:711, 21:712, 21:713, 21:714, 21:715, 21:716, 21:717, 21:718, 21:719, 21:720, 21:836, 21:837, 21:838, 21:839, 21:840, 21:841, 21:842, 21:843, 21:844, 21:845, 21:846, 21:847, 21:848, 21:849, 21:850, 21:851, *1390*

`\DeclareTextCompositeCommand` 21:94, 21:452, 21:475, 21:476, 21:477, 21:478, 21:479, 21:481, 21:721, 21:722, 21:724, 21:727, 21:728, 21:729, 21:730, 21:731, 21:732, 21:733, 21:734, 21:735, 21:737, 21:819, 21:1087, *1382*

`\DeclareTextFontCommand` 29:705, 32:1, 32:15, 32:16, 32:17, 32:18, 32:19, 32:20, 32:21, 32:22, 32:23, 32:24, 32:29, 32:30, 32:31, 32:42, 32:43, 32:141, *1399*

`\DeclareTextFoo` *1366*

`\DeclareTextGlyph` *1373*

`\DeclareTextSymbol` 21:3, 21:419, 21:420, 21:421, 21:422, 21:423, 21:424, 21:425, 21:426, 21:427, 21:428, 21:429, 21:433, 21:434, 21:437, 21:438, 21:439, 21:440, 21:441, 21:442, 21:549, 21:550, 21:551, 21:552, 21:553, 21:554, 21:555, 21:556, 21:557, 21:558, 21:559, 21:560, 21:561, 21:562, 21:563, 21:564, 21:566, 21:567, 21:568, 21:569, 21:570, 21:571, 21:572, 21:573, 21:574, 21:575, 21:576, 21:577, 21:578, 21:579, 21:580, 21:581, 21:582, 21:583, 21:584, 21:585, 21:586, 21:587, 21:588, 21:589, 21:590, 21:594, 21:595, 21:596, 21:597, 21:598, 21:599, 21:600, 21:601, 21:602, 21:603, 21:604, 21:605, 21:606, 21:607, 21:608, 21:609, 21:741, 21:742, 21:743, 21:744, 21:745, 21:746, 21:747, 21:748, 21:749, 21:750, 21:751, 21:752, 21:753, 21:763, 21:764, 21:792, 21:793, 21:794, 21:795, 21:796, 21:797, 21:798, 21:800, 21:801, 21:802, 21:803, 21:804, 21:805, 21:806, 21:807, 21:808, 21:809, 21:810, 21:811, 21:812, 21:813, 21:814, 21:815, 21:816, 21:817, 21:818, 21:877, 21:878, 21:879, 21:880, 21:881, 21:882, 21:883, 21:884, 21:885, 21:886, 21:887, 21:888, 21:900, 21:901, 21:902, 21:903, 21:904, 21:905, 21:906, 21:907, 21:908, 21:909, 21:910, 21:911, 21:912, 21:913, 21:914, 21:915, 21:916, 21:923, 21:924, 21:925, 21:926, 21:927, 21:928, 21:929, 21:930, 21:931, 21:932, 21:933, 21:934, 21:935, 21:936, 21:937, 21:938, 21:939, 21:940, 21:941, 21:942, 21:943, 21:944, 21:945, 21:946, 21:947, 21:948, 21:949, 21:950, 21:951, 21:952, 21:953, 21:954, 21:955, 21:956, 21:957, 21:958, 21:959, 21:960, 21:961, 21:962, 21:963, 21:964, 21:965, 21:966, 21:967, 21:968, 21:969, 21:970, 21:971, 21:972, 21:973, 21:974, 21:975, 21:976, 21:977, 21:978, 21:979, 21:980, 21:981, 21:982, 21:983, 21:984, 21:985, 21:986, 21:987, 21:988, 21:989, 21:990, 21:991, 21:992, 21:993, 21:994, 21:995, 21:996, 21:997, 21:998, 21:999, 21:1000, 21:1001, 21:1002, 21:1107, 33:332, 33:333, 33:334, 33:335, 33:336, 33:337, 33:338, 33:339, 33:350, 33:351, 33:352, 33:353, 33:354, 33:355, 33:356, 33:357, 33:358, 33:359, 33:553, 33:554, 33:555, 33:556, 33:557, 33:558, 33:559, 33:560, *1382*

`\DeclareTextSymbolDefault` 21:203, 21:260, 21:261, 21:262, 21:263, 21:264, 21:265, 21:266, 21:267, 21:268, 21:269, 21:270, 21:271, 21:272, 21:273, 21:274, 21:275, 21:276, 21:277, 21:278, 21:279, 21:280, 21:281, 21:282,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

21:283, 21:284, 21:285, 21:286, 21:287, 21:288, 21:289, 21:290, 21:291, 21:292, 21:293, 21:294, 21:295, 21:296, 21:297, 21:298, 21:300, 21:301, 21:311, 33:78, 33:80, 33:82, 33:84, 33:85, 33:86, 33:87, 33:88, 33:89, 33:90, 33:91, 33:92, 33:93, 33:94, 33:95, 33:96, 33:97, 33:98, 33:99, 33:100, 33:101, 33:102, 33:103, 33:104, 33:105, 33:106, 33:107, 33:108, 33:109, 33:110, 33:111, 33:112, 33:113, 33:114, 33:115, 33:116, 33:117, 33:118, 33:541, 33:542, 33:543, 33:544, 33:545, 33:546, 33:547, 33:548, 33:561, 33:562, 33:563, 33:564, 33:565, 33:566, 33:567, 33:568, 33:587, 33:588, 33:606, 33:607, 33:608, 33:609, 33:610, 33:611, 33:612, 33:614, 33:914, 33:915, 33:916, 33:917, 33:918, 33:919, 33:920, 33:921, 33:922, 33:923, 33:924, 33:925, 33:926, 33:927, 33:928, 33:929, 33:930, 33:931, 33:932, 33:933, 33:934, 33:935, 33:936, 33:937, 33:938, 33:939, 33:940, 33:941, 33:942, 33:943, 33:944, 33:945, 33:946, 33:947, 33:948, 33:949, 33:950, 33:951, 33:952, 33:953, 33:954, 33:955, 33:956, 33:957, 33:958, 33:959, 33:960, 33:961, 33:962, 33:963, 33:964, 33:965, 33:966, *1381*

\DeclareTitlecaseExclusions 57:620, *1439*

\DeclareTitlecaseMapping . . . 57:620, *1430*

\DeclareUnicode... . . . . *544*

\DeclareUnicodeAccent 21:1070, 21:1265, 21:1266, 21:1267, 21:1268, 21:1269, 21:1270, 21:1271, 21:1272, 21:1273, 21:1274, 21:1275, 21:1276, 21:1277, 21:1278, 21:1279, *1410*

\DeclareUnicodeCharacter . . . 57:433, *1411*

\DeclareUnicodeCommand 21:1108, 21:1109, 21:1110, 21:1111, 21:1192, 21:1194, 21:1196, 21:1243, 21:1280, *1424*

\DeclareUnicodeComposite 21:1085, 21:1284, 21:1285, 21:1286, 21:1287, 21:1288, 21:1289, 21:1290, 21:1291, 21:1292, 21:1293, 21:1294, 21:1295, 21:1296, 21:1297, 21:1298, 21:1299, 21:1300, 21:1301, 21:1302, 21:1303, 21:1304, 21:1305, 21:1306, 21:1307, 21:1308, 21:1309, 21:1310, 21:1311, 21:1312, 21:1313, 21:1314, 21:1315, 21:1316, 21:1317, 21:1318, 21:1319, 21:1320, 21:1321, 21:1322, 21:1323, 21:1324, 21:1325, 21:1326, 21:1327, 21:1328, 21:1329, 21:1330, 21:1331, 21:1332, 21:1333, 21:1334, 21:1335, 21:1336, 21:1337, 21:1338, 21:1339, 21:1340, 21:1341, 21:1342, 21:1343, 21:1344, 21:1345, 21:1346, 21:1347, 21:1348, 21:1349, 21:1350, 21:1351, 21:1352, 21:1353, 21:1354, 21:1355, 21:1356, 21:1357, 21:1358, 21:1359, 21:1360, 21:1361, 21:1362, 21:1363, 21:1364, 21:1365, 21:1366, 21:1367, 21:1368, 21:1369, 21:1370, 21:1371, 21:1372, 21:1373, 21:1374, 21:1375, 21:1376, 21:1377, 21:1378, 21:1379, 21:1380, 21:1381, 21:1382, 21:1383, 21:1384, 21:1385, 21:1386, 21:1387, 21:1388, 21:1389, 21:1390, 21:1391, 21:1392, 21:1393, 21:1394, 21:1395, 21:1396, 21:1397, 21:1398, 21:1399, 21:1400, 21:1401, 21:1402, 21:1403, 21:1404, 21:1405, 21:1406, 21:1407, 21:1408, 21:1409, 21:1410, 21:1411, 21:1412, 21:1413, 21:1414, 21:1415, 21:1416, 21:1417, 21:1418, 21:1419, 21:1420, 21:1421, 21:1422, 21:1423, 21:1424, 21:1425, 21:1426, 21:1427, 21:1428, 21:1429, 21:1430, 21:1431, 21:1432, 21:1433, 21:1434, 21:1435, 21:1436, 21:1437, 21:1438, 21:1439, 21:1440, 21:1441, 21:1442, 21:1443, 21:1444, 21:1445, 21:1446, 21:1447, 21:1448, 21:1449, 21:1450, 21:1451, 21:1452, 21:1453, 21:1454, 21:1455, 21:1456, 21:1457, 21:1458, 21:1459, 21:1460, 21:1461, 21:1462, 21:1463, 21:1464, 21:1465, 21:1466, 21:1467, 21:1468, 21:1469, 21:1470, 21:1471, 21:1472, 21:1473, 21:1474, 21:1475, 21:1476, 21:1477, 21:1478, 21:1479, 21:1480, 21:1481, 21:1482, 21:1483, 21:1484, 21:1485, 21:1486, 21:1487, 21:1488, 21:1489, 21:1490, 21:1491, 21:1492, 21:1493, 21:1494, 21:1495, 21:1496, 21:1497, 21:1498, 21:1499, 21:1500, 21:1501, 21:1502, 21:1503, 21:1504, 21:1505, 21:1506, 21:1507, 21:1508, 21:1509, 21:1510, 21:1511, 21:1512, 21:1513, 21:1514, 21:1515, 21:1516, 21:1517, 21:1518, 21:1519, 21:1520, 21:1521, 21:1522, 21:1523, 21:1524, 21:1525, 21:1526,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

21:1527, 21:1528, 21:1529, 21:1530, *543*
\DeclareUnicodeSymbol 21:1107, 21:1112, 21:1113, 21:1114, 21:1115, 21:1116, 21:1117, 21:1118, 21:1119, 21:1120, 21:1121, 21:1122, 21:1123, 21:1124, 21:1125, 21:1126, 21:1127, 21:1128, 21:1129, 21:1130, 21:1131, 21:1132, 21:1134, 21:1135, 21:1136, 21:1137, 21:1138, 21:1139, 21:1140, 21:1141, 21:1142, 21:1143, 21:1144, 21:1145, 21:1146, 21:1147, 21:1148, 21:1150, 21:1151, 21:1152, 21:1153, 21:1154, 21:1155, 21:1156, 21:1157, 21:1158, 21:1159, 21:1160, 21:1161, 21:1162, 21:1163, 21:1164, 21:1165, 21:1166, 21:1167, 21:1168, 21:1169, 21:1170, 21:1171, 21:1172, 21:1173, 21:1174, 21:1175, 21:1176, 21:1177, 21:1178, 21:1179, 21:1180, 21:1181, 21:1182, 21:1183, 21:1184, 21:1185, 21:1186, 21:1187, 21:1188, 21:1189, 21:1190, 21:1191, 21:1198, 21:1199, 21:1200, 21:1201, 21:1202, 21:1203, 21:1204, 21:1205, 21:1206, 21:1207, 21:1208, 21:1209, 21:1210, 21:1211, 21:1212, 21:1213, 21:1214, 21:1215, 21:1216, 21:1217, 21:1218, 21:1219, 21:1220, 21:1221, 21:1222, 21:1223, 21:1224, 21:1225, 21:1226, 21:1227, 21:1228, 21:1229, 21:1230, 21:1231, 21:1232, 21:1233, 21:1234, 21:1235, 21:1236, 21:1237, 21:1238, 21:1239, 21:1241, 21:1242, 21:1254, 21:1255, 21:1256, 21:1257, 21:1258, 21:1259, 21:1260, 21:1261, 21:1262, 21:1263, 21:1264, *1424*
\DeclareUnknownKeyHandler . . 51:216, *1425*
\DeclareUnknownKeysHandler *1425*
\DeclareUppercaseExclusions 57:620, *1439*
\DeclareUppercaseMapping . . . 57:620, *1430*
\def *737*
default (plug) *1234*
default (tag plug) 55:108
\defaultencoding *1378*
\defaulthyphenchar . 06:935, 06:950, 02:253
\defaultscriptratio . 24:721, 24:728, *1371*
\defaultscriptscriptratio 24:722, 24:728, *1371*
\defaultskewchar 02:254
\deg 38:34
\delcode 28:1170
\delimiter 28:1083, 28:1153, 28:1164, *1409*
\delimiterfactor 02:257
\delimitershortfall 02:272
\Delta 30:309
\delta 30:282
\depth 40:32, 40:35
\det 38:30
\Details 33:1589, 33:1590
\detokenize 03:96, 03:97, 03:182, 14:249, 14:250, 20:273, 20:524, 21:1022, 21:1068, 24:190, 25:2822, 05:137, 29:646, 37:207, 06:666, 06:684, 06:748, 50:599, 50:633, 50:673, 50:945, 50:1312, 50:1316, 50:1444, 50:1445, 50:1449, 57:796, *1170*
\DH 21:551, 21:1156, 57:720, *1377*
\dh 21:561, 21:1162, 57:720, *1377*
\Diamond 29:883
\diamond 30:391
\diamondsuit 30:343
\dim 38:28
dim commands:
\dim_eval:n 05:178
\dim_new:N 11:93, 53:260, 53:261, 53:262, 53:263
\dim_set:Nn 53:254, 53:255, 53:256, 53:257
\dim_set_eq:NN 48:69, 48:71
\dim_use:N 53:585, 53:586, 53:587
\c_max_dim 48:69, 48:71, 48:109, 48:113, 53:274, 53:300, 53:325, 53:352, *1072*
\l_tmpa_dim *373*
\c_zero_dim 16:104, 53:30, 53:36, 53:37, 53:54, 53:56, 53:57, 53:280, 53:281, 53:282, 53:288, 53:306, 53:307, 53:308, 53:331, 53:332, 53:333, 53:360, 53:361, 53:362
\dimen *1396*
\dimendef 04:225, 02:42, 02:43, 02:44, 02:48
\dimenzero 04:225
\dimeval *77*
\dimeval 05:172, 05:188, *77*
\dimexpr 21:526, 21:542, 21:893, 21:896, 21:1247, 21:1250, 40:343, 40:344, 40:346, 40:636, 40:637, 40:640, 42:13, 45:414, 45:420, 45:428, 45:473, 45:479, 45:487
\directlua . 04:2, 04:14, 04:29, 04:212, 04:230, 17:62, 04:255, 04:260, 04:264, 21:1016, 21:1035, 01:12, 01:15, 06:34, 01:20, 40:350, 01:23, 40:644, 45:418, 45:477, 06:846, 01:28, 06:931, 50:1297, 50:1428, 02:61, 02:77, 02:82, 57:98, 57:100, 57:108, 57:113, 57:115, 57:318, 57:356, 57:370, 02:142, 02:228, 02:238

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

disable commands:
disable_callback 04:989
\disable_callback *44*
\DisableGenericHook 08:2786, 08:2792, 08:2940, *250*
\DisableHook 08:2891, *218*
\DiscardShipoutBox 53:143, 53:464, 53:497, 53:574, *1188*
\discretionary 38:251, 06:933, 06:948, 06:958
\displayindent 02:278
\displaylines 38:203
displaymath (env.) 38:389
\displaymath 38:391
\displaystyle . . 30:543, 30:546, 30:550, 30:552, 38:62, 38:210, 38:434, 38:437, 38:463, 38:466, 38:521, 38:563, 38:575, 38:603, 38:628, 38:631
\displaywidowpenalty 02:197
\displaywidth 38:210, 38:433, 38:462, 38:550, 38:606, 02:277
\div 30:394
\DJ 21:552, 21:1166, 57:720, *1377*
\dj 21:562, 21:1167, 57:720, *1377*
\do 01:110, 13:3, 13:7, 13:16, 13:26, 20:66, 20:69, 20:72, 20:135, 20:138, 20:190, 20:193, 20:252, 20:324, 20:386, 20:433, 20:606, 20:623, 20:774, 20:780, 06:69, 32:91, 37:543, 37:564, 37:724, 37:734, 37:740, 37:746, 40:78, 40:97, 41:321, 41:346, 42:115, 42:127, 42:134, 42:214, 42:348, 42:350, 42:373, 42:376, 42:405, 42:407, 42:428, 42:433, 42:488, 42:516, 42:545, 42:741, 42:797, 45:65, 45:134, 47:36, 47:65, 47:82, 02:13, 02:14, 50:267, 50:284, 50:578, 50:597, 50:614, 50:632, 50:637, 50:651, 50:656, 50:692, 50:702, 50:1225, 50:1262, 50:1344, 50:1397, 50:1477, 50:1566, 50:1853, 07:1616, 07:1832, 07:2199, 07:2212, 01:58, 01:59, *1428*
\DocInput 26:8, 30:5, 31:5, 56:4
\doclearpage *1403*
\document *485*
document (env.) 37:8
\document 16:145, 20:9, 37:260, 47:64, 47:81, *1393*
\documentclass 04:16, 26:2, 30:2, 31:2, 50:709, 50:716, 50:775, 50:778, 50:1018, 50:1113, 50:1245, 56:2, *223*
\DocumentMetadata . 17:8, 17:17, 17:21, 17:23, 17:181, 50:808, 50:811, *457*
\documentstyle 50:714, 50:1245
\doesglyphexist 33:1182, 33:1208, 33:1226, 33:1227, 33:1228, 33:1229, 33:1230, 33:1231, 33:1232, 33:1235, 33:1236, 33:1237, 33:1238, 33:1241, 33:1242, 33:1243, 33:1244, 33:1247, 33:1248, 33:1251, 33:1252, 33:1255, 33:1256, 33:1257, 33:1258, 33:1259, 33:1260, 33:1261, 33:1262, 33:1265, 33:1266, 33:1269, 33:1270, 33:1271, 33:1272, 33:1273, 33:1274, 33:1275, 33:1276, 33:1277, 33:1278, 33:1279, 33:1280, 33:1281, 33:1282, 33:1283, 33:1284, 33:1285, 33:1286, 33:1287, 33:1288, 33:1289, 33:1290, 33:1291, 33:1292, 33:1293, 33:1294, 33:1295, 33:1296, 33:1297, 33:1298, 33:1299, 33:1300, 33:1301, 33:1302, 33:1303, 33:1304, 33:1305, 33:1306, 33:1307, 33:1308, 33:1309, 33:1310, 33:1311, 33:1312, 33:1313, 33:1314, 33:1315, 33:1316, 33:1317, 33:1318, 33:1319, 33:1320, 33:1321, 33:1322, 33:1323, 33:1324, 33:1325, 33:1326, 33:1327, 33:1328, 33:1329, 33:1330, 33:1331, 33:1332, 33:1333, 33:1334, 33:1337, 33:1341, 33:1342, 33:1343, 33:1344, 33:1345, 33:1346, 33:1347, 33:1348, 33:1349, 33:1350, 33:1351, 33:1352, 33:1353, 33:1354, 33:1355, 33:1356, 33:1357, 33:1358, 33:1359, 33:1360, 33:1361, 33:1362, 33:1363, 33:1364, 33:1365, 33:1366, 33:1367, 33:1368, 33:1369, 33:1370, 33:1371, *831*
\dollar *1378*
\dospecials 01:110, 37:543, 37:564, 37:724, 37:734, 02:13, 50:1344, 50:1477, 50:1566, 07:1617, 07:1833, 07:2200, 07:2213, 01:58, *1434*
\dot 30:536
\doteq 30:480
\dotfill 06:973, 06:994, 02:421
\dots 21:340, 21:342, *1382*
\doublehyphendemerits 02:204
\doublerulesep 41:388, 41:415, 41:439
\Downarrow 30:597
\downarrow 30:591
\downbracefill 30:549, 30:568
\dump 57:808

E

\E 50:1351, 50:1354, 50:1382, 50:1484, 50:1487, 50:1514, 50:1573, 50:1576, 50:1604

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\edef *1064*
\EditInstance 11:1549, *377*
\EditTemplateDefaults 11:1549, *377*
\egroup 02:325, *1358*
\eject 02:388
\ell 30:322
\else *1369*
else commands:
\else: 08:1754, 08:2300, 08:2328, 08:2470, 08:2486, 08:2533, 08:2568, 08:2583, 09:363, 09:387, 16:26, 16:32, 16:64, 48:112, 06:813, 52:241, 07:1343, 07:1840, 07:1841, 07:1842, 07:1843, 07:3290, 08:789, 08:1063, 08:1220, 08:1282, 08:1423, 08:1430, 08:1438
\em 29:633, 29:660, 32:42, *758*
\emergencystretch . . 49:132, 49:138, *1387*
\emforce *755*
\emforce 29:629, 29:656, 29:665, *757*
\eminnershape . 29:637, 29:643, 29:660, *756*
\emph 32:42, *1414*
\empty 02:323
\emptyset 30:329
\emreset *755*
\emreset 29:629, 29:632, 29:663, *755*
\emrest 29:632
\ENC-cmd *1377*
\encodingdefault 04:259, 04:274, 04:282, 21:1013, 21:1564, 21:1601, 21:1602, 28:408, 28:458, 29:769, 29:802, 29:818, 29:834, 29:851, 30:62, *745*
\EncodingSpecific *1366*
\EncodingSpecificAccent *1366*
\EncodingSpecificAccentedLetter . . *1366*
\EncodingSpecificCommand *1366*
\end 14:250, 26:9, 06:23, 30:6, 31:6, 37:270, 37:298, 37:312, 37:348, 37:364, 37:378, 37:386, 37:522, 37:523, 38:584, 38:593, 39:112, 44:15, 44:17, 06:811, 06:868, 50:1359, 50:1363, 50:1370, 50:1492, 50:1496, 50:1502, 50:1581, 50:1585, 50:1591, 53:449, 56:5, 07:1455, 07:1582, 07:1597, 07:1764, 01:53, 07:2882, *244*
\end\verbvisiblespace 37:345
\endaligned *1322*
\endarray 41:184, *1321*
\endcenter 37:447
\endcsname *223*
\enddisplaymath 38:392
enddocument (hook) . . . *1206*, *238*, *239*, *243*
\enddocument 37:8, 37:79, 37:80, 37:143, 37:145, 53:399, *238*
enddocument/afteraux (hook) *243*
enddocument/afterlastpage (hook) *1206*, *243*
enddocument/end (hook) *243*
enddocument/info (hook) *1182*, *243*
\endenumerate 39:271
\endeqnarray 38:442, 38:471, 38:520
\endequation 38:400, 38:407
\endfilecontents 50:1249
\endflushleft 37:497
\endflushright 37:499
\endgraf 16:142, 16:180, 40:138, 40:144, 40:149, 02:320, *1386*
\endgroup *1370*
\EndIncludeInRelease 08:1567, 02:564, 08:1656, 02:574, 02:579, 08:1739, 08:1980, 08:2084, 08:2089, 08:2094, 08:2184, 08:2191, 08:2215, 08:2231, 08:2238, 08:2243, 08:2265, 08:2285, 08:2315, 08:2333, 08:2338, 08:2348, 08:2352, 08:2355, 08:2382, 08:2397, 08:2408, 08:2426, 08:2438, 08:2450, 08:2460, 03:71, 08:2490, 08:2509, 08:2572, 08:2576, 08:2658, 08:2661, 08:2775, 08:2781, 08:2788, 08:2793, 08:2800, 08:2804, 08:2811, 08:2815, 08:2838, 08:2843, 09:36, 09:47, 09:61, 09:244, 09:311, 09:454, 09:473, 09:525, 09:529, 09:576, 09:603, 09:606, 09:618, 14:157, 14:161, 14:204, 14:210, 16:44, 16:75, 04:233, 17:138, 17:162, 17:173, 04:256, 18:21, 18:31, 18:65, 18:78, 18:86, 18:91, 18:104, 18:110, 18:155, 18:174, 18:188, 18:200, 18:211, 18:228, 04:279, 18:240, 04:283, 18:272, 18:288, 18:305, 18:341, 18:375, 18:397, 18:434, 18:467, 18:503, 18:507, 18:516, 18:520, 18:528, 18:532, 18:542, 18:548, 18:562, 18:569, 20:83, 20:140, 20:195, 20:221, 20:234, 20:275, 20:296, 20:308, 20:371, 20:375, 20:454, 20:492, 20:515, 20:550, 20:571, 20:590, 20:597, 20:616, 20:633, 20:646, 20:650, 20:668, 20:679, 20:689, 20:728, 20:755, 21:121, 21:141, 21:186, 21:193, 21:350, 21:372, 21:387, 21:395, 22:45, 22:50, 22:75, 22:95, 22:112, 22:132, 22:141, 22:145, 22:169, 22:196, 22:201, 22:218, 22:232, 22:248, 22:254, 22:269, 22:280, 22:294, 22:301, 22:343,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

22:349, 22:369, 22:372, 23:10, 23:14, 23:25, 23:30, 24:53, 24:74, 24:227, 24:246, 24:294, 24:311, 24:357, 24:367, 24:377, 24:438, 24:447, 24:527, 24:532, 24:567, 24:572, 24:589, 24:607, 24:646, 24:679, 24:797, 24:809, 25:1431, 25:2772, 25:2778, 25:2791, 25:2803, 25:2810, 25:2893, 25:2908, 25:2930, 25:2942, 25:3047, 25:3128, 25:3131, 25:3142, 25:3149, 25:3156, 25:3229, 25:3247, 25:3254, 25:3258, 25:3261, 26:158, 26:161, 26:177, 05:13, 05:22, 05:31, 26:566, 26:572, 27:21, 27:143, 28:78, 28:106, 28:180, 28:210, 28:240, 28:272, 28:322, 28:365, 28:417, 05:115, 28:464, 28:482, 28:488, 28:493, 28:576, 28:583, 28:628, 05:136, 28:635, 05:161, 28:909, 28:951, 28:964, 28:971, 05:170, 28:1159, 28:1167, 05:182, 05:190, 29:68, 29:101, 29:123, 29:143, 29:156, 05:205, 29:232, 05:211, 29:285, 29:293, 29:320, 29:327, 29:379, 05:219, 29:426, 29:455, 05:226, 29:466, 29:513, 05:234, 29:547, 05:239, 29:573, 29:611, 29:622, 06:12, 29:659, 29:671, 29:678, 06:18, 29:740, 29:746, 29:797, 29:813, 29:828, 29:845, 29:860, 29:869, 29:874, 30:92, 30:102, 30:120, 30:129, 06:60, 30:644, 30:651, 06:64, 32:33, 32:40, 06:82, 06:91, 06:148, 06:159, 35:28, 35:45, 35:61, 35:75, 35:82, 35:109, 35:124, 35:131, 35:163, 35:173, 35:182, 35:190, 35:197, 35:209, 35:215, 35:221, 35:228, 06:265, 06:270, 37:77, 37:141, 37:180, 37:188, 37:194, 37:227, 37:240, 06:306, 37:268, 37:279, 37:310, 37:335, 37:343, 37:361, 37:376, 37:384, 37:389, 37:403, 37:412, 37:422, 06:327, 37:429, 37:439, 37:444, 37:468, 37:484, 37:493, 37:512, 37:518, 37:548, 37:569, 37:591, 37:601, 37:618, 37:630, 37:634, 37:642, 37:646, 37:667, 37:680, 37:695, 37:707, 37:729, 37:737, 38:86, 38:95, 38:117, 38:124, 38:143, 38:148, 38:156, 38:160, 01:290, 38:175, 38:183, 38:232, 38:249, 38:279, 38:287, 38:296, 38:302, 38:335, 38:360, 38:387, 38:402, 38:408, 38:449, 38:477, 38:500, 38:509, 38:541, 38:546, 38:566, 38:578, 38:587, 38:596, 06:416, 39:134, 39:139, 39:161, 39:169, 06:444, 40:13, 40:22, 01:300, 40:57, 40:65, 40:90, 40:107, 40:121, 40:132, 40:141, 40:146, 40:151, 40:157, 06:472, 40:168, 40:175, 06:477, 40:232, 40:239, 40:265, 40:276, 40:312, 40:320, 40:356, 40:363, 40:391, 06:496, 40:417, 40:438, 40:462, 40:480, 06:510, 40:557, 40:573, 40:590, 40:599, 40:604, 40:654, 40:665, 40:675, 40:682, 40:716, 40:723, 41:64, 41:69, 41:156, 41:164, 41:176, 41:182, 41:191, 41:195, 41:210, 41:216, 41:224, 41:228, 41:254, 41:276, 41:303, 41:308, 06:551, 06:561, 42:15, 42:19, 42:27, 42:31, 42:49, 42:58, 42:79, 42:87, 42:99, 42:107, 42:122, 42:132, 42:161, 42:166, 42:182, 42:193, 42:260, 42:274, 06:593, 42:379, 42:436, 06:602, 42:473, 42:479, 42:510, 42:540, 42:565, 42:581, 42:592, 42:605, 42:613, 42:634, 42:651, 42:661, 42:665, 06:629, 42:755, 06:635, 42:810, 42:829, 42:846, 43:30, 43:39, 43:47, 43:53, 43:70, 43:77, 44:19, 44:29, 06:649, 06:654, 44:167, 44:173, 44:178, 44:195, 44:207, 44:217, 44:223, 01:25, 44:249, 44:271, 45:104, 45:172, 45:231, 45:246, 45:293, 45:306, 45:351, 45:368, 45:437, 45:442, 45:448, 45:457, 45:461, 45:496, 45:501, 45:507, 45:511, 45:543, 45:560, 45:578, 45:620, 45:626, 47:24, 47:32, 47:76, 47:92, 06:755, 06:770, 49:34, 49:60, 49:81, 49:104, 49:115, 49:124, 50:21, 50:26, 06:821, 50:50, 50:66, 50:78, 06:825, 50:101, 50:110, 50:116, 50:128, 50:143, 50:153, 50:177, 50:184, 50:209, 50:216, 50:234, 50:245, 50:279, 50:296, 50:307, 50:316, 50:347, 50:377, 50:394, 06:857, 50:406, 50:427, 50:440, 50:453, 50:463, 06:866, 50:499, 50:515, 50:524, 50:558, 50:568, 50:608, 50:624, 50:646, 50:661, 50:675, 50:682, 50:697, 50:706, 50:740, 50:747, 50:764, 50:771, 50:832, 50:839,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

50:877, 50:904, 50:932, 50:1088, 06:916, 50:1145, 06:923, 50:1175, 50:1182, 50:1200, 50:1210, 50:1385, 06:945, 50:1516, 06:956, 06:959, 50:1606, 06:987, 06:1008, 52:14, 52:23, 52:88, 52:140, 52:179, 52:191, 52:200, 52:245, 52:256, 52:263, 52:298, 52:321, 52:342, 52:356, 52:361, 52:384, 52:404, 52:429, 52:474, 52:494, 52:501, 52:531, 52:536, 53:44, 53:63, 53:482, 53:519, 53:534, 53:540, 53:557, 53:561, 54:157, 54:183, 54:201, 54:370, 54:375, 54:423, 54:469, 54:733, 54:817, 54:946, 54:1005, 54:1063, 54:1162, 54:1181, 54:1244, 54:1262, 54:1304, 54:1325, 54:1568, 54:1712, 54:1881, 07:241, 54:1964, 54:2044, 07:264, 54:2138, 54:2260, 54:2387, 54:2497, 54:2505, 54:2539, 54:2568, 54:2606, 54:2610, 01:34, 54:2774, 54:2823, 54:2853, 54:2871, 54:2902, 54:2943, 54:2987, 57:15, 57:19, 57:29, 57:33, 57:51, 57:69, 57:79, 57:86, 57:94, 57:141, 57:146, 57:198, 57:222, 57:294, 57:299, 57:367, 57:389, 57:393, 57:489, 57:536, 57:727, 57:731, 07:1182, 07:1185, 07:1200, 07:1213, 07:1216, 07:1346, 07:1349, 07:1378, 07:1381, 07:1390, 07:1398, 07:1401, 07:1567, 07:1571, 02:234, 02:242, 07:2207, 07:2218, 07:2250, 07:2266, 07:2277, 07:3191, 02:346, 07:3221, 07:3307, 07:3314, 02:363, 08:79, 08:95, 08:123, 08:139, 08:151, 08:163, 08:179, 08:192, 08:212, 08:222, 08:242, 08:257, 08:275, 08:281, 08:301, 08:319, 08:323, 02:453, 08:600, 08:651, 08:668, 08:672, 08:695, 08:716, 08:750, 08:808, 08:838, 08:870, 02:487, 08:897, 08:900, 08:920, 08:923, 08:936, 08:954, 02:495, 08:959, 08:962, 08:971, 08:976, 08:979, 08:1016, 08:1053, 08:1103, 08:1110, 08:1126, 08:1133, 08:1153, 08:1157, 02:519, 08:1209, 08:1213, 08:1229, 08:1233, 08:1261, 08:1264, 08:1287, 08:1291, 08:1308, 08:1313, 08:1320, 08:1324, 08:1360, 08:1384, 08:1467, 08:1494, 08:1528, 02:559, *26*

`\endinput` *1371*

`\enditemize` 39:282

`\endline` 38:188, 02:320

`\endlinechar` 20:703, 20:737, 01:188, 06:37, 06:39, 06:44, 50:468, 50:469, 50:470, 57:381, 02:255, 01:76, 01:77, 01:78, *1403*

`\endlist` 39:98, 39:271, 39:282

`\endlrbox` 40:199

`\endmath` 38:390

`\endminipage` 40:523

`\EndModuleRelease` 03:111, 03:112, 08:2962, 03:152, 09:642, 10:234, 11:1637, 16:185, 17:183, 33:773, 36:303, 48:531, 55:469, 07:3327

`\endpicture` 42:60

`\endscname` *1357*

`\endsloppypar` 49:136

`\endsmallmatrix` *1322*

`\endtabbing` 41:84

`\endtabular` 41:184

`\endtabular*` 41:184

`\endtrivlist` 37:447, 37:497, 37:499, 37:572, 38:608, 39:100, 39:101, 41:85, 43:80

`\endverbatim` 37:571, 37:589

`\enlargethispage` 54:2432, *1071*

`\enlargethispage*` 54:2432

`\enskip` 18:571

`\enspace` 18:559, 18:567

`\ensuremath` 22:345, 38:522, 45:441, 45:447, 45:500, 45:506, *1395*

`enumerate` (env.) 39:262

`\enumerate` 39:262

`env` (hook) *227*, *236*

`env/`⟨*env*⟩`/after` (hook) *240*, *240*, *241*

`env/`⟨*env*⟩`/before` (hook) . . . *240*, *240*, *241*

`env/`⟨*env*⟩`/begin` (hook) *487*, *240*, *241*

`env/`⟨*env*⟩`/end` (hook) *240*, *241*

`env/document/after` (hook) *244*

`env/document/before` (hook) *242*

`env/document/begin` (hook) *489*, *242*

`env/document/end` (hook) *244*

`env/quote/after` (hook) *234*, *234*

environments:

`array` 41:168

`center` 37:446

`displaymath` 38:389

`document` 37:8

`enumerate` 39:262

`eqnarray` 38:415, 38:609

`eqnarray*` 38:518

`equation` 38:393, 38:597

`filecontents` 50:1249, *1095*

`flushleft` 37:496

`flushright` 37:498

`itemize` 39:273

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

list 39:34
lrbox *925*
math 38:389
minipage *926*
picture 42:21
sloppypar 49:135
tabbing 41:71
tabular 41:198
thebibliography *1053*
trivlist 39:89
verbatim 37:571
verbatim* 37:586
\epsilon 30:283
eqnarray (env.) 38:415, 38:609
\eqnarray 38:423, 38:424, 38:451, 38:453, 38:519
eqnarray* (env.) 38:518
\eqno 38:400, 38:407, 38:532, *1381*
equation (env.) 38:393, 38:597
\equation . . 38:397, 38:398, 38:404, 38:406
\equiv 30:459
\errhelp 03:31, 14:39, 14:66, 01:201, 56:12, 57:309, 57:785
\errmessage 03:32, 10:77, 04:63, 14:47, 14:72, 01:7, 24:618, 24:653, 01:206, 26:442, 26:542, 27:65, 05:84, 05:99, 05:140, 56:16, 57:63, 57:311, 02:108, 01:42, 02:174
\ERROR 08:1740, 08:1741, 07:1883, 07:1893, 07:1902, 07:2300, 07:2311, 07:2331, 07:2348, 07:2420, 07:2443, *446*
\errorcontextlines 14:212, 02:264, 02:443, 02:462, 02:478, 02:493, 02:507, 02:530, 02:547, *1359*
\ERRORmissingcells 55:367
\ERRORnewtaggingsocket 55:18
\errorstopmode 57:807, 02:429, *1390*
\ERRORusetaggingsocket 55:40, 55:50
\escapchar *499*
\escapechar 20:473, 24:464, 24:688, 26:230, 28:59, 28:87, 28:159, 28:190, 28:220, 28:251, 28:392, 28:442, 06:166, 06:210, 06:214, 06:222, 06:379, 06:380, 06:404, 06:541, 06:664, 06:682, 52:280, 52:303, 52:326, 52:347, 02:252, *105*
\eta 30:285
\etatcatcode 04:1041
\eTeXversion 01:41
\evensidemargin 54:49, 54:898, 54:968, 54:1027
\everycr 38:205, 38:208, 38:433, 38:462, 38:625, 02:409, *1362*
\everydisplay 24:419, 24:420, 24:431, 24:443
\everyeof 57:381
\everyjob 03:37, 04:215, 04:261, 04:262, 28:414, 28:462, 53:90, 53:91, 57:112, 57:349, 57:350, 57:383, 57:465, 57:755, 57:756, 57:758, *1340*
\everymath . 24:418, 24:420, 24:436, 24:445
\everypar *438*
\everypar 16:35, 16:41, 16:72, 16:77, 16:122, 16:132, 16:145, 16:183, 20:43, 20:113, 20:171, 24:742, 24:756, 24:803, 37:243, 37:545, 37:567, 39:130, 39:131, 39:133, 39:137, 39:138, 39:211, 39:228, 40:452, 40:473, 41:81, 44:48, 44:96, 44:107, 44:127, 44:136, 45:187, 54:144, 54:171, 54:197, 54:1503, 54:1652, 54:1813, *440*
\EveryShipout *1191*
\ExecuteOptions 26:58, 26:71, 33:841, 50:685
\exhyphenpenalty 02:192, 02:382
\exists 30:335
\exp 38:31
exp commands:
\exp:w . . . 07:2656, 08:779, 08:833, *269*
\exp_after:wN 08:1543, 08:1753, 08:1872, 08:1905, 08:1990, 08:2134, 08:2279, 08:2463, 08:2466, 08:2530, 08:2992, 09:29, 09:31, 09:186, 09:261, 09:315, 09:316, 09:378, 09:386, 09:389, 09:505, 11:376, 11:833, 11:1053, 11:1195, 11:1202, 16:41, 16:72, 28:335, 28:338, 51:46, 52:73, 52:236, 52:237, 07:326, 07:342, 07:395, 07:401, 07:415, 07:1174, 07:1194, 07:1198, 07:1209, 07:1211, 07:1260, 07:1309, 07:1339, 07:1370, 07:1377, 07:1452, 07:1593, 07:1852, 07:1876, 07:1892, 07:1910, 07:1916, 07:1920, 07:1951, 07:1985, 07:1992, 07:2005, 07:2011, 07:2017, 07:2023, 07:2029, 07:2035, 07:2041, 07:2047, 07:2078, 07:2222, 07:2659, 07:2660, 07:2706, 07:2707, 07:2708, 07:2709, 07:2710, 07:2711, 07:2712, 07:2743, 08:50, 08:55, 08:374, 08:375, 08:420, 08:567, 08:778, 08:832, 08:1062, 08:1064, 08:1205, 08:1370, *269*
\exp_args:Nc 08:2220, 09:45, 09:58, 09:90, 09:148, 48:22, 07:267, 07:1105, 07:1116, 07:1416, 07:1456, 07:1482, 07:1586, 07:2812, 08:1226, 08:1239
\exp_args:Ncc 07:142, 07:184

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\exp_args:NcV 08:591
\exp_args:Ne 08:2368, 08:2369, 09:539, 10:139, 11:1167, 36:10, 51:111, 51:142, 51:191, 52:46, 52:97, 53:90, 07:2673, 07:2676, 07:2699, 07:2732, 08:427, 08:475, 08:551, 08:736, 08:738
\exp_args:Nee 52:129
\exp_args:Nf 09:177, 09:252, 52:32, 52:34, 52:394, 07:420, 07:2424, 07:2447, 07:2506, 07:2734
\exp_args:NNc 09:162, 09:167
\exp_args:NNe 08:1645, 11:56, 07:1386, 07:1481, 08:1524
\exp_args:Nne 08:1882
\exp_args:NNNo 07:401, 07:1755, 07:2120
\exp_args:NnnV 11:1259
\exp_args:Nnnv 08:251
\exp_args:NNo . . . 08:1734, 08:2993, 09:185, 09:260, 08:1086, 08:1143
\exp_args:NNV 08:1618, 08:1705, 09:204, 09:279
\exp_args:NNx 08:1562
\exp_args:No 08:2093, 08:2152, 08:2982, 08:2993, 09:185, 09:204, 09:260, 09:279, 11:1249, 36:41, 36:46, 07:94, 07:323, 07:412, 07:431, 07:440, 07:1345, 07:2314, 07:2674, 07:2700, 07:2809, 08:1086, 08:1143, 08:1204
\exp_args:Nof 07:2306
\exp_args:Noo 07:1318
\exp_args:NV 09:205, 09:280, 11:881, 51:222, 52:53, 07:260, 07:364, 07:1761, 07:1764
\exp_args:Nv 08:2022, 08:2030, 08:2132, 08:1271
\exp_args:NVV 11:1626
\exp_args_generate:n 57:646
\exp_end: 07:2659, 08:779, 08:833
\exp_last_unbraced:Ne 08:2984, 07:2745, 08:1088, 08:1121, 08:1145, 08:1238
\exp_last_unbraced:Nf 08:1929, 09:126, 08:1251
\exp_last_unbraced:NNf 08:1179
\exp_last_unbraced:NNNNo 08:2087, 09:132, 09:508, 08:377
\exp_last_unbraced:NnNo 07:2518
\exp_last_unbraced:NNo 09:397
\exp_last_unbraced:Nno 26:345
\exp_not:N 08:1646, 08:1648, 09:124, 09:126, 09:127, 09:128, 09:200, 09:215, 09:222, 09:223, 09:230, 09:235, 09:275, 09:290, 09:297, 09:298, 09:302, 09:306, 09:323, 09:372, 09:377, 09:385, 09:409, 09:419, 09:493, 09:494, 09:514, 09:519, 09:542, 09:569, 09:570, 09:597, 09:598, 10:25, 11:10, 11:15, 11:368, 11:369, 11:370, 11:371, 11:381, 11:387, 11:390, 11:392, 11:393, 11:397, 11:399, 11:402, 11:407, 11:633, 11:793, 11:810, 11:811, 11:916, 11:918, 11:919, 11:921, 11:922, 11:976, 11:977, 11:978, 11:980, 11:1094, 11:1102, 11:1142, 11:1157, 11:1182, 11:1189, 16:39, 16:40, 16:70, 16:71, 16:79, 16:80, 36:72, 48:115, 48:118, 48:120, 48:125, 48:128, 51:222, 51:223, 51:224, 53:92, 53:156, 53:186, 07:124, 07:139, 07:140, 07:168, 07:169, 07:201, 07:203, 07:204, 07:210, 07:212, 07:271, 07:283, 07:300, 07:301, 07:302, 07:304, 07:371, 57:653, 07:817, 07:818, 07:947, 07:957, 07:1038, 07:1051, 07:1075, 07:1093, 07:1100, 07:1138, 07:1146, 07:1147, 07:1179, 07:1225, 07:1226, 07:1233, 07:1234, 07:1235, 07:1239, 07:1242, 07:1243, 07:1245, 07:1264, 07:1267, 07:1268, 07:1269, 07:1277, 07:1324, 07:1343, 07:1357, 07:1364, 07:1365, 07:1465, 07:1466, 07:1638, 07:1645, 07:1661, 07:1665, 07:1667, 07:1697, 07:1710, 07:1714, 07:1715, 07:1719, 07:1720, 07:1724, 07:1725, 07:1729, 07:1730, 07:1790, 07:1796, 07:1797, 07:1799, 07:1800, 07:1802, 07:1803, 07:2052, 07:2053, 07:2161, 07:2163, 07:2164, 07:2246, 07:2247, 07:2258, 07:2261, 07:2274, 07:2735, 07:2738, 07:2739, 07:2740, 07:2741, 07:2743, 08:385, 08:567, 08:568, 08:569, 08:573, 08:574, 08:1173, 08:1177, 08:1196, 08:1199, 08:1200, 08:1202, 08:1246, 08:1508, 08:1510, *1073*
\exp_not:n 08:1652, 08:2982, 09:187, 09:200, 09:203, 09:205, 09:240, 09:262, 09:275, 09:278, 09:280, 09:308, 09:370, 09:420, 09:422, 09:423, 09:425, 09:426, 11:373, 11:520, 11:979, 11:1095, 11:1096, 11:1103, 11:1141, 11:1143, 11:1144, 11:1157, 11:1160, 11:1161, 11:1182, 11:1183, 11:1189, 11:1190,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

36:122, 36:131, 36:261, 48:186, 48:193, 48:265, 48:266, 48:267, 48:297, 48:374, 51:207, 52:211, 52:225, 53:91, 07:163, 07:167, 07:171, 07:174, 07:178, 07:195, 07:202, 07:214, 07:284, 07:303, 07:371, 07:374, 07:375, 07:625, 07:771, 07:894, 07:905, 07:907, 07:908, 07:987, 07:1039, 07:1052, 07:1063, 07:1075, 07:1092, 07:1178, 07:1180, 07:1233, 07:1246, 07:1278, 07:1313, 07:1577, 07:1579, 07:1592, 07:1631, 07:1632, 07:1696, 07:1738, 07:1752, 07:1753, 07:1760, 07:1777, 07:1861, 07:1862, 07:1905, 07:1965, 07:2162, 07:2321, 07:2322, 07:2358, 07:2359, 07:2542, 07:2558, 07:2573, 07:2585, 07:2735, 08:108, 08:235, 08:554, 08:557, 08:1070, 08:1094, 08:1193, 08:1197, 08:1203, 08:1204, *344*

\exp_stop_f: 07:1504, 07:2308, 08:1272, 08:1420, 08:1428

expand@font@defaults (hook) *244*

expandable commands:

\expandable_grab_⟨*type*⟩:w *165*

\expandableinput . . 05:227, 05:236, 05:238

\expandafter *875*

expandafter commands:

\expandafter: 32:89, 41:346

\ExpandArgs *77*

\ExpandArgs 05:191, 05:207, 05:210, 06:626, 06:627, 57:650, *117*

\expanded 50:945, 57:111, 57:794, *259*

\ExplLoaderFileDate 05:215, 05:223

\ExplSyntaxOff 08:2999, 09:414, 09:643, 10:235, 11:1638, 16:186, 17:58, 17:137, 17:161, 17:184, 20:548, 20:768, 20:818, 24:417, 26:358, 28:320, 28:363, 05:160, 05:180, 05:203, 05:232, 36:304, 48:532, 49:32, 49:59, 49:113, 50:550, 51:287, 52:12, 52:86, 52:138, 52:243, 52:254, 52:402, 52:427, 52:472, 52:529, 52:584, 53:42, 53:62, 53:480, 53:532, 53:539, 53:548, 53:588, 55:213, 55:303, 55:319, 55:459, 57:344, 57:346, 57:348, 57:619, 57:718, 07:3328, *1101*

\ExplSyntaxOn 09:3, 09:415, 10:2, 11:6, 16:3, 17:2, 17:25, 17:61, 17:141, 20:537, 20:762, 20:815, 24:412, 26:343, 28:278, 28:326, 05:149, 05:175, 05:194, 05:230, 36:2, 48:3, 49:22, 49:37, 49:110, 50:548, 51:3, 52:7, 52:29, 52:92, 52:207, 52:251, 52:391, 52:410, 52:435, 52:508, 52:577, 53:5, 53:47, 53:66, 07:8, 53:525, 53:537, 53:546, 53:583, 55:2, 55:300, 55:305, 55:321, 57:343, 57:609, 57:620, 08:3, *328*

\extracolsep 41:167

\extrafloats 02:100, 02:112, 02:160

F

\fam 04:22, 04:26, 04:38, 24:16, 02:250

\family *1358*

\familydefault 21:1602, 28:409, 28:459, 29:523, 29:770, 29:803, 29:819, 29:835, 29:852, 30:131, *745*

\fbox *925*

\fbox 40:217, 40:230, 40:237, *1375*

\fboxrule 40:215, 40:262, 40:274, 40:279, 40:285, 40:287, 40:294, 40:295, 57:151

\fboxsep 40:215, 40:221, 40:254, 40:261, 40:273, 40:280, 40:290, 40:292, 57:150

\fi *1405*

fi commands:

\fi: 08:1756, 08:2135, 08:2255, 08:2263, 08:2280, 08:2302, 08:2330, 08:2467, 08:2472, 08:2488, 08:2535, 08:2570, 08:2585, 09:358, 09:360, 09:361, 09:365, 09:379, 09:390, 09:391, 09:394, 16:26, 16:33, 16:65, 16:105, 48:130, 52:241, 07:905, 07:908, 07:986, 07:1343, 07:1506, 07:1844, 07:2656, 07:2660, 07:3292, 08:791, 08:1065, 08:1222, 08:1245, 08:1285, 08:1424, 08:1432, 08:1438, *344*

\filbreak 02:386

file (hook) *227*, *236*

file commands:

\g_file_curr_name_str 08:2829

\file_full_name:n . 20:544, 52:35, *1169*

\file_get:nnN . . . 57:344, 57:346, *1340*

\file_input_raw:n 05:231, *1438*

\file_mdfive_hash:n 20:816

\file_parse_full_name_apply:nN 52:32, 52:47, 52:90, 52:93, 52:95

\l_file_search_path_seq *1161*

\file_size:n 20:817

file internal commands:

__file_parse_full_name_area:nw 52:102, 52:105, 52:109

__file_parse_full_name_auxi:nN 52:97, 52:100

__file_parse_full_name_base:nw 52:108, 52:112, 52:124

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

__file_parse_full_name_tidy:nnnN 52:119, 52:120, 52:122, 52:127
file/ (hook) *1169*
file/.../after (hook) *1169*
file/.../after *1160*
file/.../before *1160*
file/⟨*file name*⟩/after (hook) *1160*, *1161*
file/⟨*file name*⟩/before (hook) *1160*, *1161*
file/⟨*file:name*⟩/after (hook) *1160*
file/⟨*file:name*⟩/before (hook) . . . *1160*
file/⟨*package name*⟩.sty/after (hook) *1162*
file/⟨*package name*⟩.sty/before (hook) *1162*
file/after (hook) *1160*, *1160*, *1162*, *1169*, *1182*
file/after *1160*
file/array.sty/after (hook) *1161*
file/before (hook) *315*, *1160*, *1160*, *1162*, *1169*
file/before *1160*
filecontents (env.) 50:1249, *1095*
\filecontents 50:1249
filehook internal commands:
__filehook_clear_replacement_flag: 52:417, 52:420, 52:520
__filehook_drop_extension:N 52:44, 52:49, 52:522
__filehook_drop_extension_aux:nnn 52:54, 52:57
__filehook_file_name_compose:nnn 52:218, 52:406, 52:414, 52:415, 52:425
__filehook_file_parse_full_name:nN 52:28, 52:30, 52:30, 52:53, 52:238, 52:394, 52:412, 52:414, *1176*
__filehook_file_pop: 52:59, 52:70, 52:526
__filehook_file_pop_assign:nnnn 52:59, 52:73, 52:79, 52:528
__filehook_file_push: 52:59, 52:62, 52:524
__filehook_file_replaced 52:417
__filehook_file_subst_begin:nnn 52:412, 52:422, 52:422, *1176*
__filehook_file_subst_cycle_error:NN 52:457, 52:462, 52:462, 52:467, 52:469, *1178*
__filehook_file_subst_loop:NN . . . 52:431, 52:444, 52:452, 52:461, *1177*
__filehook_file_subst_tortoise_hare:nn 52:424, 52:431, 52:434, 52:436, 52:459, *1178*
__filehook_full_name:nn 52:30, 52:34, 52:38
__filehook_if_file_replaced:TF 52:417, 52:418, 52:518, *1177*
__filehook_if_no_extension:nTF 52:44, 52:44, 52:510
\g__filehook_input_file_seq 52:59, *1167*
\l__filehook_internal_tl 52:59
__filehook_log_file_record:n 52:547, 52:547, 52:561, 52:563
\g__filehook_nesting_level_int . . 52:544, 52:549, 52:552, 52:553, 52:559
__filehook_normalize_file_name:w 52:406, 52:413, 52:516
__filehook_resolve_file_subst:w 52:406, 52:409, 52:411, 52:514
__filehook_set_curr_file:nNN 52:387, 52:390, 52:392, 52:512
__filehook_set_curr_file_assign:nnnNN 52:387, 52:395, 52:397
__filehook_subst_add:nn 52:206, 52:208, 52:208, 52:252, *1171*
__filehook_subst_empty_name_chk:NN 52:208, 52:236, 52:240
__filehook_subst_file_normalize:Nn 52:208, 52:216, 52:218, 52:230, 52:234
__filehook_subst_remove:n 52:208, 52:222, 52:253
\filename@parse *6*, *1*
\filesize 05:77, 05:123
\fill 18:552
\finalhyphendemerits 37:455, 37:459, 37:465, 02:205, *1416*
\finalstrut *1384*
\FirstMark 48:402, 48:522, *1060*
\firstmark 49:120, 54:1002, 54:1061, 54:2921, *1058*
flag internal commands:
\flag␣__filehook_file_replaced *1177*
flag commands:
\flag_clear:n 52:421
\flag_if_raised:nTF 36:109, 36:223, 52:419, 52:440
\flag_new:n 36:88, 52:417
\flag_raise:n 36:110, 52:441
\flat 30:339
float/begin (tag socket) 55:260
float/end (tag socket) 55:260
float/hmode/begin (tag socket) . . . 55:258
float/hmode/end (tag socket) 55:258
\floatingpenalty 45:531, 45:550, 45:568, 02:202
\floatpagefraction 45:278, 54:2999
floats:footnotes (plug) 54:697, *1234*
floats:space:footnotes (plug) 54:681, *1233*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\floatsep . . 54:1077, 54:1095, 54:1102, 54:2730, 54:2780, 54:2830, 54:3004
\flushbottom 49:128, *1248*
flushleft (env.) 37:496
\flushleft 37:496
flushright (env.) 37:498
\flushright 37:498
\fmtname 03:1, 03:41, 03:43, 03:46, 03:48, 03:51, 03:60, 50:784, 50:788, *1363*
\fmtversion 03:1, 03:19, 03:41, 03:43, 03:46, 03:48, 03:51, 03:60, 03:105, 03:118, 03:166, 14:2, 04:275, 21:1566, 29:318, 41:1, 42:1, 50:204, 50:212, 50:801, 50:804, 57:739, 57:765, *1363*
\fnsymbol *556*
\fnsymbol 22:308
\font 21:317, 21:318, 21:319, 21:460, 21:467, 21:823, 21:830, 21:890, 21:1027, 21:1028, 21:1088, 21:1193, 21:1195, 21:1197, 21:1244, 24:82, 24:88, 24:90, 26:85, 29:636, 29:669, 29:675, 29:729, 31:8, 31:9, 31:10, 32:86, 33:6, 33:576, 33:590, 33:597, 37:566, 06:934, 06:937, 06:949, 06:952, 02:414, 02:419
\fontdimen 21:317, 21:318, 21:319, 21:460, 21:467, 21:823, 21:830, 29:636, 29:669, 29:675, 32:86, 33:6, 33:576, 33:590, 33:597, 40:353, 40:649, 40:650, 40:651, 42:137, 42:140, 42:689, 45:428, 45:429, 45:431, 45:487, 45:488, 45:490, 02:414, 02:419
\fontencoding 04:258, 04:259, 04:281, 04:282, 21:891, 21:1601, 24:314, 24:349, 24:361, 24:371, 28:408, 28:458, 29:769, 29:802, 29:818, 29:834, 30:16, 30:24, 30:79, 30:86, 30:95, 30:97, 33:36, *1375*
\fontfamily 24:342, 29:7, 29:10, 29:13, 29:168, 29:178, 29:239, 29:249, 29:561, 29:564, 29:567, 29:862, 29:894, 30:70, 30:81, 30:88, 30:99, 33:28, 33:30, 33:32, 33:34, 33:49, 33:51, 33:53, 33:55, 33:877, *656*
\FontFamilyToCheck 33:1584, 33:1585, 33:1598
\fontname 21:1028, 24:90
\fontseries 24:342, 25:2781, 25:2782, 25:2793, 25:2795, 25:2805, 25:2807, 29:16, 29:19, 29:337, 29:338, 29:339, 29:340, 29:360, 29:361, 29:362, 29:363, 29:398, 29:399, 29:400, 29:401, 29:418, 29:419, 29:420, 29:421, 29:434, 29:435, 29:436, 29:437, 29:445, 29:446, 29:447, 29:448, 29:461, 29:464, 29:555, 29:558, 29:896, *750*
\fontseriesforce 25:2786, 25:2797, 25:2808, *656*
\fontshape 21:470, 21:833, 24:342, 25:2916, 25:2921, 25:2926, 25:3134, 25:3135, 25:3144, 25:3146, 25:3151, 25:3153, 25:3204, 25:3208, 25:3211, 25:3214, 25:3217, 25:3220, 25:3223, 25:3226, 25:3233, 29:22, 29:25, 29:28, 29:31, 29:897, 33:600, *675*
\fontshapeforce 25:3138, 25:3147, 25:3154, 25:3234
\fontsize 19:6, 21:322, 21:348, 21:380, 21:892, 21:1246, 21:1282, 24:80, 24:381, 29:752, 29:898, 33:68, 33:636, 33:894, 45:413, 45:441, 45:447, 45:472, 45:500, 45:506, *1376*
\fontsubfuzz 26:458, 26:492, 37:55, 37:120, 37:159, *1372*
\FOO *81*
foo (hook) *313*
foo (socket) *356*, *356*
\foo *121*
\footins 45:390, 45:527, 45:546, 45:564, 54:319, 54:320, 54:321, 54:322, 54:380, 54:427, 54:612, 54:617, 54:623, 54:650, 54:738, 54:744, 54:748, 54:820, *1234*
\footnote 45:514, *1049*
\footnotemark 44:10, 45:580, *1383*
\footnoterule 40:529, 45:394, 54:621, 54:747, *1389*
footnotes:floats (plug) 54:702, *1234*
footnotes:floats:legacy (plug) 54:707, *1234*
footnotes:space:floats (plug) 54:674, *1233*
\footnotesep 40:553, 40:570, 40:587, 45:513, 45:530, 45:539, 45:549, 45:557, 45:567, 45:575
\footnotesize 40:545, 40:563, 40:580, 45:528, 45:547, 45:565
\footnotetext 44:12, 45:597, *1359*
\footref 45:609, *1420*
\footskip 54:53, 54:929, 54:992, 54:1051, *1244*
\forall 30:334
fp commands:
\fp_eval:n 05:176
\l_tmpa_fp *373*
\fpeval *76*
\fpeval 05:172, 05:184, 05:186, *76*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\frac 38:412, *1365*
\frame 40:201, 40:302
\framebox *925*
\framebox 40:224, *1375*
\frenchspacing 20:46, 20:116, 20:174, 37:571, 37:588, 37:739, 06:974, 06:995, 02:306
\frown 30:462
\fussy 49:137
\futurelet 18:479, 18:487, 05:54, 32:84, 38:256, 41:436, 06:874, 06:888

G

\g__hook_\meta␣{hook}_code_prop . . . *248*
\g__hook_\meta␣{hook}_declared_tl . *248*
\g__hook_\meta␣{hook}_parameter_tl *248*
\g__hook_\meta␣{hook}_reversed_tl . *248*
\Gamma 30:308
\gamma 30:281
\gathered *1322*
\gcd 38:33
\gdef *1378*
gdef commands:
\gdef_ 38:269
\ge 30:428, 30:430
\Generic* *1383*
\GenericError 14:18, 14:85, 14:111, 14:164, 26:63, *1379*
\GenericInfo . . . 03:106, 03:108, 03:119, 03:122, 03:127, 03:162, 03:163, 03:168, 03:172, 03:176, 03:197, 14:4, 14:104, 14:130, 14:182, 21:8, 26:32, 26:35, 26:40, 26:76, 05:68, 50:83, 50:91, 50:121, 50:124, 50:1779, *1379*
\GenericWarning 14:11, 14:94, 14:120, 14:141, 14:149, 14:173, 14:195, 26:43, 26:48, 26:51, 26:79, *1379*
\geq 30:426, 30:428
\getanddefinefonts *1357*
\GetDocumentProperty 17:35, *458*
\GetFileInfo 30:3
\gets 30:450, 30:452
\gg 30:444
\global *1388*
\globaldefs 24:690, 26:232, 28:61, 28:90, 28:161, 28:192, 28:222, 28:254, 02:247
\glossary *1050*
\glossary 35:104, 44:192, 44:200, 46:23, 46:35, 48:282, 49:40, 49:51, 49:66, 49:74, 49:89, 49:97, 54:906, 54:976, 54:1035, *1059*
\glossaryentry 46:32
\gluestretchorder 54:569
\glyphmissingdetails 33:1183, 33:1208, *831*
\goodbreak 06:975, 06:996, 02:386
\grave 30:528
\group *1357*
group commands:
\group_align_safe_begin/end: . . *186*
\group_align_safe_begin: 07:290, 07:302, 07:406, *139*
\group_align_safe_end: 07:324, 07:429, 07:2108, *140*
\group_begin: 09:28, 09:217, 09:292, 16:17, 16:23, 16:50, 16:56, 28:298, 28:348, 48:68, 48:250, 48:373, 51:4, 52:210, 52:224, 53:153, 07:1612, 07:1621, 07:1657, 07:1706, 07:1745, 07:1809, 07:1816, 07:2100, 07:2488, 07:2493, 07:2665, 07:2669, 07:2697, 08:381, 08:1172, 08:1185
\c_group_begin_token 07:691, 07:702, 07:1680, 07:2106, 07:2187, 07:2777
\group_end: . 09:30, 09:221, 09:296, 16:19, 16:28, 16:52, 16:60, 28:301, 28:351, 48:73, 48:269, 48:386, 51:21, 52:220, 52:232, 53:155, 07:1642, 07:1691, 07:1733, 07:1755, 07:1766, 07:1821, 07:1825, 07:1829, 07:2121, 07:2497, 07:2502, 07:2679, 07:2680, 07:2682, 07:2684, 07:2694, 07:2701, 07:2702, 08:384, 08:1176, 08:1195, *334*
\c_group_end_token 07:694, 07:1722, 07:2176
\group_insert_after:N 28:334, 28:335, 28:338, 28:360, 28:361
\guillemetleft 21:563, 21:797, 21:1132, *1411*
\guillemetright 21:564, 21:798, 21:1148, *1411*
\guillemotleft . . . 21:566, 21:800, 21:1134
\guillemotright . . 21:567, 21:801, 21:1150
\guilsinglleft 21:568, 21:1211
\guilsinglright 21:569, 21:1212

H

\H 14:24, 21:250, 21:406, 21:492, 21:624, 21:632, 21:651, 21:659, 21:777, 21:1274, 21:1413, 21:1414, 21:1441, 21:1442, 33:141, 33:165
\halign 38:127, 38:210, 38:433, 38:462, 38:625, 02:409, *94*
\hang *1388*
\hangafter 02:249
\hangindent 44:139, 02:280
\hat 30:534
\hbadness 24:745, 24:752, 24:788, 24:807, 53:299, 53:301, 53:351, 53:353, 02:188
\hbar 30:346

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\hbox *1322*
hbox commands:
\hbox:n 53:435
\hbox_set:Nn 53:231, 53:278, 53:304, 53:329, 53:358
\hbox_set_to_wd:Nnn 53:30, 53:54, 53:302, 53:354
\hbox_unpack:N 48:425, 48:457, 53:312, 53:356
\hbox_unpack_drop:N 53:241
\hbox␣to␣ *1373*
\headheight . 54:51, 54:913, 54:981, 54:1040
\headsep . . . 54:52, 54:927, 54:990, 54:1049
\heartsuit 30:344
\height . 21:896, 21:1250, 40:31, 40:34, *1364*
\hfil *909*
\hfill *1360*
\hfuzz 24:753, 49:133, 49:134, 49:140, 49:141, 53:298, 53:300, 53:350, 53:352, 02:265
\hglue 02:376
\hideoutput 02:565
\hideskip 02:179, 02:400
\hidewidth 21:347, 21:349, 21:378, 21:382, 21:411, 21:412, 21:415, 21:418, 21:499, 21:500, 21:504, 21:507, 21:509, 21:512, 21:524, 21:529, 21:545, 21:784, 21:785, 21:788, 21:791, 21:858, 21:861, 21:1281, 21:1283, 02:400
\HiTeXversion 21:1060
\hline 41:435, 41:438, *1361*
\hoffset 02:281
\holdinginserts 02:209
\hom 38:29
hook commands:
\hook_activate_generic:n 08:2785, 08:2871, 08:2890, 08:2903, 08:282, 08:282, 08:284, 08:302, 08:320, 08:322, *230*
\hook_debug_off: 08:2847, 08:7, 08:13, *232*
\hook_debug_on: 08:2846, 08:7, 08:8, *232*
\hook_disable:n 08:2859, 08:2859
\hook_disable_generic:n 08:2787, 08:2864, 08:2896, 08:258, 08:258, 08:260, 08:276, 08:279, *230*
\hook_gclear_next_code:n 08:2232, 08:2232, 08:2817, *231*
\hook_gput_code:nnn 08:2797, 08:491, 08:491, 08:493, 08:601, 08:603, 08:703, 08:724, *231*
\hook_gput_code_with_args:nnn . . . 08:2799, 08:491, 08:499, 08:650, *231*
\hook_gput_next_code:nn 08:2154, 08:2154, 08:2156, 08:2185, 08:2187, 08:2808, 08:712, 08:730, *231*
\hook_gput_next_code_with_-args:nn 08:2170, 08:2190, 08:2810, *231*
\hook_gremove_code:nn . . 08:2819, 08:980, 08:980, 08:982, 08:1017, *231*
\hook_gset_rule:nnnn . . . 08:2849, 08:2851, 08:2854, 08:1325, 08:1325, *232*
\hook_if_empty:n 08:2474, 08:2476, 08:2491, 08:2493
\hook_if_empty:nTF 08:1894, 08:2000, 08:2474, 08:2855, 08:2856, 08:2857, 53:124, 53:225, *223*
\hook_if_empty_p:n 08:1949, 08:2049, 08:2474, 53:128, 53:174, 53:429, *232*
\hook_log:n 08:1840, 08:1840, 08:2845, *232*
\hook_new:n 08:2762, 17:114, 48:288, 53:214, 53:215, 53:216, 53:217, 53:218, 53:219, 53:220, 08:61, 08:63, 08:82, 08:217, 08:883, *230*
\hook_new_pair:nn 08:2766, 16:6, 16:7, 08:193, 08:195, 08:215, *229*
\hook_new_pair_with_args:nn . . . *230*
\hook_new_pair_with_args:nnn 08:2774, 08:193, 08:193, 08:197, 08:213, 08:220, *230*
\hook_new_reversed:n . . . 08:2764, 08:164, 08:166, 08:182, 08:218, *229*
\hook_new_reversed_with_args:nn 08:2772, 08:164, 08:164, 08:168, 08:180, 08:191, *230*
\hook_new_with_args:nn . . 08:2770, 08:61, 08:61, 08:65, 08:80, 08:94, *1429*
\g_hook_patch_action_list_tl 09:6, 09:107, 09:140
\hook_provide:n 08:2859, 08:2866
\hook_provide_pair:nn 08:2859, 08:2880
\hook_provide_reversed:n 08:2859, 08:2873
\hook_show:n 08:1840, 08:1847, 08:1854, 08:2844, *298*
\hook_use:n 08:2244, 08:2244, 08:2246, 08:2266, 08:2268, 08:2286, 08:2288, 08:2343, 08:2834, 16:22, 16:31, 16:55, 16:63, 16:100, 16:107, 17:134, 17:158, 48:252, 53:123, 53:126, 53:132, 53:136, 53:184, 08:1487, *230*
\hook_use:nnw 08:2284, 08:2314, 08:2316, 08:2316, 08:2318, 08:2334, 08:2336, 08:2344, 08:2836, *230*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\hook_use_once:n 08:2383, 08:2385, 08:2400, 08:2835, *230*
\hook_use_once:nnw 08:2383, 08:2383, 08:2391, 08:2398, 08:2406, 08:2837, *231*

hook internal commands:

\c__hook_ 08:1317, 08:1321
\g__hook_??_code_prop 08:1314
\c__hook_??_parameter_tl . . . 08:1314
\g__hook_??_reversed_tl 08:1314
\g__hook_⟨*hook*⟩_code_prop *313*
\g__hook_⟨*hook*⟩_labels_clist . . . *252*
\g__hook_⟨*hook*⟩_reversed_tl . . . *248*
__hook__print_args:nn *1431*
__hook_activate_generic:n . . 08:282
__hook_activate_generic:nn 08:2888, 08:285, 08:286, 08:304
__hook_activate_generic_pair:nn . . 08:2859, 08:2885, 08:2889, 08:2917
__hook_activate_generic_-reversed:n 08:2859, 08:2878, 08:2887, 08:2890, 08:2910
\g__hook_all_seq 08:2990, 08:28, 08:102, 08:131, 08:1448, 08:1475
__hook_apply_-rule_->:nnn . 08:1815
__hook_apply_-rule_<-:nnn . 08:1815
__hook_apply_-rule_<:nnn . . 08:1815
__hook_apply_-rule_>:nnn . . 08:1815
__hook_apply_-rule_xE:nnn . 08:1815
__hook_apply_-rule_xW:nnn . 08:1815
__hook_apply_label_pair:nnn 08:1595, 08:1596, 08:1677, 08:1678, 08:1742, 08:1742, *304*
__hook_apply_rule:nnn 08:1752, 08:1758, 08:1758, *295*
__hook_apply_rule:nnnN *296*
__hook_apply_rule_->:nnn . . 08:1793
__hook_apply_rule_<-:nnn . . 08:1793
__hook_apply_rule_<:nnn . . . 08:1765
__hook_apply_rule_>:nnn . . . 08:1765
__hook_apply_rule_xE:nnn . . 08:1779
__hook_apply_rule_xW:nnn . . 08:1779
__hook_braced_cs_parameter:n 08:2985, 08:1089, 08:1146, 08:1171, 08:1182, 08:1234, 08:1234, 08:1236, 08:1262
__hook_braced_hidden_loop:w 08:1234, 08:1238, 08:1241, 08:1247
__hook_braced_parameter:n 08:1646, 08:1648, 09:237, 09:539, 08:1265, 08:1265, 08:1267, 08:1288, 08:1290, 08:1509, 08:1511
__hook_braced_real_loop:w . 08:1265
__hook_chk_args_allowed:nn 08:2208, 08:507, 08:652, 08:652, 08:654, 08:669, 08:671, *306*
__hook_clear_next:n 08:2212, 08:2228, 08:2233, 08:2234, 08:2234, 08:2236, 08:2239, 08:2241, *303*
__hook_clist_gput:Nn 08:1551, 08:1555, 08:1619, 08:1706, 08:1740, 08:1741, 08:1517, 08:1519
__hook_cmd_begindocument_code: . 09:62, 09:70, 09:75, 09:78, 09:640, *352*
__hook_cmd_if_scanable:Nn . . 09:484
__hook_cmd_if_scanable:NnTF 09:442, 09:461, 09:484, *349*
__hook_cmd_patch_xparse:Nnn 09:144, 09:165, 09:165
__hook_cmd_try_patch:nn 09:68, 09:79, 09:79
__hook_code_gset:nn . . . 08:2445, 08:112, 08:1111, 08:1111, 08:1113, 08:1125, 08:1127, 08:1129, 08:1506
__hook_code_gset_aux:nnn 08:1074, 08:1111, 08:1114, 08:1116, 08:1118, 08:1119, 08:1132
__hook_code_gset_auxi:nnnn 08:1054, 08:1075, 08:1100, 08:1102, 08:1109, 08:1140, *276*
__hook_cs_end:w 08:1234, 08:1253, 08:1254, 08:1255, 08:1260
__hook_cs_gput_right:nnn 08:2213, 08:2416, 08:546, 08:1054, 08:1054, 08:1056, 08:1104, 08:1106
__hook_cs_gput_right_fast:nnn . . 08:1054, 08:1062, 08:1068, 08:1107
__hook_cs_gput_right_slow:nnn 08:1054, 08:1064, 08:1071, 08:1108
__hook_cs_if_empty:N 08:1214, 08:1216, 08:1230, 08:1232
__hook_cs_if_empty:NTF 08:1919, 08:1926, 08:2209, 08:2211, 08:2482, 08:2483, 08:1214
__hook_cs_if_empty_p:N 08:1214
__hook_cs_parameter_count:N 08:1234, 08:1239, 08:1249
__hook_cs_parameter_count:w 08:1234, 08:1251, 08:1258, 08:1259
\l__hook_cur_hook_tl . . . 08:1581, 08:1663, 08:1799, 08:1810, 08:29, *297*
__hook_curr_name_pop: 08:2825, 08:2931, 08:411, 08:449, *261*
__hook_curr_name_push:n 08:2823, 08:2828, 08:411, 08:425, *260*
__hook_curr_name_push_aux:n 08:411, 08:427, 08:433

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

__hook_currname_or_default: 08:2673, 08:2721, 08:327, 08:337, 08:341, 08:357, 08:358, 08:358, 08:543, 08:633, *258*
__hook_debug:n 08:1561, 08:1600, 08:1620, 08:1682, 08:1707, 08:1767, 08:1774, 08:1781, 08:1789, 08:1795, 08:1806, 08:2160, 08:2174, 09:19, 09:27, 09:44, 09:55, 09:64, 09:65, 09:81, 09:85, 08:7, 08:7, 08:20, 08:536, 08:626, 08:1447, 08:1456, 08:1474, 08:1477, 08:1499, 08:1523, 08:1533, *245*
\g__hook_debug_bool 08:6, 08:10, 08:15, 08:21
__hook_debug_gset: 08:7, 08:11, 08:16, 08:18
__hook_debug_label_data:N 08:1600, 08:1642, 08:1683, 08:1731, 08:1828, 08:1828
__hook_debug_print_rules:n 08:2138, 08:2138
__hook_declare_deprecated_-generic:NNn . 08:778, 08:799, 08:832
__hook_declare_deprecated_-generic:NNw . 08:794, 08:800, 08:801
__hook_def_cmd:w 09:13, 09:14, 09:208, 09:214, 09:283, 09:289, 09:315, *342*
\g__hook_delayed_patches_prop 09:17, 09:67, 09:73, 09:74
__hook_deprecated_generic_-warn:n 08:1870, 08:1988, 08:777, 08:784, 08:831, 08:1009, 08:1045, 08:1336, 08:1367, *269*
__hook_deprecated_generic_-warn:Nn 08:784
__hook_deprecated_generic_-warn:Nw 08:784
__hook_deprecated_generic_-warn:w 08:785, 08:786
__hook_deprecated_warn:nn 08:2861, 08:2868, 08:2875, 08:2882, 08:2893, 08:2900, 08:2907, 08:2914, 08:2919, 08:2919
__hook_disable:n 09:225, 09:535, 08:258, 08:261, 08:262
__hook_do_deprecated_generic:Nn . 08:1871, 08:1989, 08:794, 08:794, 08:1010, 08:1046, 08:1337, 08:1368
__hook_do_deprecated_generic:Nw 08:794, 08:795, 08:796
__hook_double_hashes:n 09:204, 09:279, 09:341, 09:341, 09:396, 08:558, 08:1086, 08:1095, 08:1143, 08:1200, *280*
__hook_double_hashes:w 09:341, 09:342, 09:343, 09:373, 09:396, 09:399, *343*
__hook_double_hashes_group:n 09:341, 09:349, 09:395
__hook_double_hashes_output:N 09:341, 09:346, 09:354, *344*
__hook_double_hashes_output_-aux:N . 09:341, 09:368, 09:375, 09:383
__hook_double_hashes_space:w 09:341, 09:350, 09:398
__hook_double_hashes_stop:w 09:341, 09:357, 09:394
\c__hook_empty_tl 08:35, 08:1306
__hook_end_document_label_-check: 08:411, 08:456, 08:457, 08:464
__hook_exp_not:n 09:187, 09:188, 09:192, 09:203, 09:262, 09:263, 09:267, 09:278, *340*
__hook_exp_not:NN 09:13, 09:13, 09:209, 09:215, 09:235, 09:240, 09:284, 09:290, 09:306, 09:308
__hook_file_hook_normalize:n 08:2369, 08:738, 08:921, 08:921, 08:924, 08:926, *272*
\l__hook_front_tl 08:1570, 08:1611, 08:1614, 08:1617, 08:1619, 08:1620, 08:1621, 08:1635, 08:1636, 08:1696, 08:1700, 08:1704, 08:1706, 08:1708, 08:1710, 08:1724, 08:1725
\c__hook_generic_⟨*type*⟩/./⟨*place*⟩_tl *269*
\c__hook_generic_class/./after_-tl 08:937
\c__hook_generic_class/./before_-tl 08:937
\c__hook_generic_cmd/./after_tl 08:937
\c__hook_generic_cmd/./before_tl 08:937
\c__hook_generic_env/./after_tl 08:937
\c__hook_generic_env/./before_tl 08:937
\c__hook_generic_env/./begin_tl 08:937
\c__hook_generic_env/./end_tl 08:937
\c__hook_generic_file/./after_tl 08:937
\c__hook_generic_file/./before_-tl 08:937

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\c__hook_generic_include/./after_-tl 08:937
\c__hook_generic_include/./before_-tl 08:937
\c__hook_generic_include/./end_-tl 08:937
\c__hook_generic_package/./after_-tl 08:937
\c__hook_generic_package/./before_-tl 08:937
__hook_generic_parameter:n 08:1081, 08:1301, 08:1312
__hook_generic_parameter:w 08:1302, 08:1303
\c__hook_generics_file_prop 08:913, 08:960
\c__hook_generics_prop 08:849, 08:881, 08:937, 08:955, 08:957
\c__hook_generics_reversed_ii_-prop 08:858, 08:885, 08:960
\c__hook_generics_reversed_iii_-prop 08:862, 08:889, 08:960
__hook_gput_code:nnn 08:491, 08:496, 08:502, 08:505, 08:605, 08:606, 08:684
__hook_gput_code_store:nnn 08:491, 08:518, 08:521
__hook_gput_next_code:nn 08:2159, 08:2173, 08:2189, 08:2192, 08:2192, 08:691
__hook_gput_next_do:nn . 08:2198, 08:2203, 08:2203, 08:2205, 08:2216, 08:2218, 08:692, 08:713, 08:730, *267*
__hook_gput_next_do:Nnn 08:2220, 08:2223
__hook_gput_undeclared_hook:nnn 08:673, 08:673, 08:685, 08:704, 08:724, *267*
__hook_gremove_code:nn 08:980, 08:983, 08:984, 08:1011, 08:1019, 08:1047
__hook_gset_rule:nnnn 08:1325, 08:1327, 08:1330, 08:1332, 08:1337, 08:1361, 08:1363, 08:1368
__hook_guess_arg_count:NN 09:501, 09:501, 09:503, 09:526, 09:528, 09:534
__hook_guess_arg_count:nw 09:501, 09:514, 09:519, 09:522
__hook_guess_arg_count:wN 09:501, 09:505, 09:509
__hook_hash_check:nTF 08:491, 08:556, 08:565
__hook_hash_check_aux:w 08:491, 08:567, 08:573
\c__hook_hash_tl 09:11, 09:195, 09:196, 09:198, 09:270, 09:271, 09:273, 09:360, 08:1246, *280*
\c__hook_hashes_tl 09:11, 09:361, 08:1173, *1428*
\g__hook_hook_curr_name_tl 08:32, 08:360, 08:370, 08:411, 08:423, 08:444, 08:445, 08:452, 08:462, 08:463, 08:489, *258*
__hook_hook_gput_code_do:nnn 08:234, 08:251, 08:491, 08:525, 08:534, 08:613, 08:624, 08:676
__hook_if_cmd_hook:n 08:2557, 08:2559, 08:2573
__hook_if_cmd_hook:nTF 08:2100, 08:2557, 08:2575
__hook_if_cmd_hook:w 08:2560, 08:2561
__hook_if_cmd_hook:wTF 08:2557
__hook_if_cmd_hook_p:n 08:2557
__hook_if_cmd_hook_p:w 08:2557
__hook_if_declared:n 08:2522
__hook_if_declared:nTF . 08:2099, 08:2522, 09:83, 08:69, 08:87, 08:172, 08:201, 08:204, 08:291, 08:309, 08:658, 08:883, 08:1059, 08:1077, *250*
__hook_if_declared_p:n 08:2522
__hook_if_deprecated_generic:n 08:2545
__hook_if_deprecated_generic:nTF 08:1868, 08:1986, 08:2537, 08:775, 08:829, 08:1007, 08:1043, 08:1334, 08:1365
__hook_if_deprecated_generic:w 08:2546, 08:2547
__hook_if_deprecated_generic_-p:n 08:2537
__hook_if_disabled:n 08:267
__hook_if_disabled:nTF 08:1880, 08:1892, 08:1975, 08:1998, 08:2078, 08:2194, 08:258, 08:288, 08:306, 08:529, 08:617, *250*
__hook_if_disabled_p:n 08:258
__hook_if_execute_immediately:n 08:2461
__hook_if_execute_immediately:nTF 08:2387, 08:2393, 08:2402, 08:2461, 08:508, 08:608, 08:1341
__hook_if_execute_immediately_-p:n 08:2461
__hook_if_file_hook:w 08:898, 08:901, 08:903
__hook_if_file_hook:wTF 08:2366, 08:734, 08:898, *272*
__hook_if_file_hook_p:w 08:898

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

__hook_if_generic:n 08:2537
__hook_if_generic:nTF 08:1878, 08:1994, 08:2537, 08:758, 08:816, 08:1080
__hook_if_generic:w 08:2538, 08:2539
__hook_if_generic_p:n 08:2537
__hook_if_generic_reversed:n 08:2577
__hook_if_generic_reversed:nTF . 08:2577, 08:296, 08:314, 08:770, 08:824
__hook_if_generic_reversed:w 08:2578, 08:2579
__hook_if_generic_reversed_p:n 08:2577
__hook_if_has_hash:n 09:327
__hook_if_has_hash:nTF 09:185, 09:260, 09:327, *343*
__hook_if_has_hash:w 09:327, 09:328, 09:329, 09:335, 09:337
__hook_if_has_hash_check:w 09:327, 09:334, 09:339
__hook_if_has_hash_p:n 09:327
__hook_if_label_case:nnnnn 08:1593, 08:1675, 08:2115, 08:1434, 08:1434
__hook_if_public_command:N . 09:124
__hook_if_public_command:NTF 09:93, 09:103, *336*
__hook_if_public_command:w 09:93, 09:127, 09:133
__hook_if_replacing_args: . 08:2591
__hook_if_replacing_args:TF 08:2587, 08:2593, 08:2598, 08:2599, 08:2604, 08:2605, 08:2610, 08:510, 08:553, 08:580, 08:656, 08:1093
__hook_if_reversed:n 08:2528
__hook_if_reversed:nTF . 08:1548, 08:1915, 08:1955, 08:1957, 08:2016, 08:2055, 08:2058, 08:2528, 08:1515
__hook_if_reversed_p:n 08:2528
__hook_if_structure_exist:n 08:2516
__hook_if_structure_exist:nTF . . 08:2197, 08:2495, 08:2516, 08:2752, 08:144, 08:156, 08:986, 08:1021, *249*
__hook_if_structure_exist_p:n 08:2516
__hook_if_usable:n 08:2510
__hook_if_usable:nTF 08:1890, 08:1914, 08:1973, 08:1996, 08:2014, 08:2076, 08:2161, 08:2175, 08:2419, 08:2510, 08:2858, 08:100, 08:523, 08:537, 08:582, 08:611, 08:627, 08:760, 08:818, 08:851, 08:1003, 08:1039, 08:1502, 08:1537, *274*
__hook_if_usable_p:n 08:1948, 08:2048, 08:2510
__hook_if_usable_use:n 08:2353, 08:2368, 08:2371, 08:2374, *310*
__hook_include_legacy_code_-chunk:n 08:120, 08:136, 08:223, 08:223, 08:225, 08:243, 08:245, 08:1501, 08:1536, *1430*
__hook_init_structure:n 08:2207, 08:113, 08:133, 08:140, 08:140, 08:142, 08:152, 08:154, 08:545, 08:635, 08:675, 08:1347, 08:1372, *264*
__hook_initialize_all: 08:2929, 08:1442, 08:1442, 08:1444, 08:1468, 08:1470, 08:1492
__hook_initialize_hook_code:n 08:2262, 08:2283, 08:2312, 08:1446, 08:1473, 08:1495, 08:1495, 08:1497, 08:1529, 08:1531, *282*
__hook_initialize_single:NNn . . . 08:1559, 08:1575, 08:1575, 08:1577, 08:1655, 08:1657, 08:1659, 08:1521, *291*
\l__hook_label_0_tl 08:1570
__hook_label_if_exist_apply:nnnTF 08:1742, 08:1744, 08:1746, 08:1749, *304*
__hook_label_ordered:nn 08:1426, *287*
__hook_label_ordered:nnTF 08:1389, 08:1396, 08:1403, 08:1426, *285*
__hook_label_ordered_p:nn . 08:1426
__hook_label_pair:nn 08:1824, 08:1825, 08:1388, 08:1395, 08:1402, 08:1407, 08:1412, 08:1417, 08:1418, 08:1418, *286*
\l__hook_labels_int 08:1570, 08:1580, 08:1584, 08:1616, 08:1638, 08:1662, 08:1666, 08:1702, 08:1727, *293*
\l__hook_labels_seq 08:1570, 08:1579, 08:1585, 08:1603, 08:1661, 08:1667, 08:1686, 08:1830
__hook_list_if_rule_exists:nnnTF . . 08:2106, 08:2126, 08:2127, 08:2129
__hook_list_one_rule:nnn 08:2106, 08:2117, 08:2118, 08:2124
__hook_list_rules:nn . . 08:1935, 08:2035, 08:2106, 08:2106, 08:2143, *304*
__hook_log:nN . . 08:1840, 08:1843, 08:1850, 08:1857, 08:1864, 08:1866, 08:1871, 08:1982, 08:1984, 08:1989
__hook_log_cmd:n 08:1842, 08:1849, 08:1856, 08:1861, 08:1863, 08:1875, 08:1993
__hook_log_line:n 08:1840, 08:1860, 08:1891, 08:1893, 08:1897, 08:1911, 08:1923, 08:1933, 08:1951, 08:1970,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

08:1997, 08:1999, 08:2003, 08:2011, 08:2025, 08:2033, 08:2051, 08:2072
__hook_log_line_indent:n 08:1840, 08:1862, 08:1901, 08:1907, 08:1917, 08:1924, 08:1938, 08:1946, 08:2005, 08:2008, 08:2018, 08:2026, 08:2038, 08:2046
__hook_log_next_code:n . 08:2030, 08:2085, 08:2085, 08:2090, 08:2092
__hook_log_next_code:w 08:1929, 08:2088
__hook_make_name:n 08:351, 08:357, 08:366, 08:372, 08:372, *259*
__hook_make_name:w 08:372, 08:374, 08:378
__hook_make_prefixes:w 09:170, 09:223, 09:298, 09:319, 09:324
__hook_make_usable:n 08:91, 08:127, 08:312, 08:822, 08:855
__hook_make_usable:nn 09:227, 09:537, 08:75, 08:96, 08:96, 08:98, 08:124, 08:126, 08:294, 08:765, 08:768, *349*
__hook_misused_if_replacing_-aux:nn . . 08:2587, 08:2588, 08:2594
__hook_msg_pair_found:nnn 08:1767, 08:1774, 08:1781, 08:1789, 08:1797, 08:1808, 08:1821, 08:1821
\g__hook_name_stack_seq 08:32, 08:412, 08:413, 08:417, 08:424, 08:444, 08:451, 08:459, 08:469
__hook_new:n . . . 08:83, 08:85, 08:187
__hook_new:nn 08:61, 08:64, 08:66, 08:67, 08:84, 08:175, 08:207
__hook_new_pair:nnn 08:196, 08:198, 08:199
__hook_new_reversed:n 08:183, 08:185
__hook_new_reversed:nn 08:164, 08:167, 08:169, 08:170, 08:184, 08:190, 08:208
__hook_next␣⟨*hook*⟩ *313*
__hook_next_gset:nn 08:2212, 08:2237, 08:2446, 08:148, 08:992, 08:1111, 08:1117, 08:1131
\c__hook_nine_parameters_tl 08:1906, 08:35, 08:108, 08:1082, 08:1197
__hook_normalize_code_pool:n 08:114, 08:1158, 08:1158, 08:1160, 08:1210, 08:1212, *252*
__hook_normalize_cs_args:nn 08:110, 08:111, 08:1134, 08:1134, 08:1136, 08:1154, 08:1156, *252*
__hook_normalize_fn:nn 08:586, 08:1164, 08:1183, 08:1208, *279*
__hook_normalize_hook_args:Nn 08:1843, 08:1850, 08:1857, 08:2159, 08:2173, 08:2188, 08:2233, 08:2388, 08:2394, 08:2403, 08:2888, 08:64, 08:66, 08:83, 08:167, 08:169, 08:183, 08:261, 08:285, 08:379, 08:388
__hook_normalize_hook_args:Nnn 08:196, 08:198, 08:379, 08:393, 08:496, 08:502, 08:604, 08:983
__hook_normalize_hook_args_-aux:Nn 08:379, 08:379, 08:390, 08:395, 08:403
__hook_normalize_hook_rule_-args:Nnnnn . 08:379, 08:401, 08:1327
\l__hook_param_text_tl 09:8, 09:191, 09:212, 09:266, 09:287, 09:316, *339*
__hook_parameter:n 08:1885, 08:1122, 08:1180, 08:1292, 08:1292, 08:1294, 08:1309, 08:1311
\c__hook_parameter_cmd 08:972, 08:977
\c__hook_parameter_cmd/./after_-tl 08:972
\c__hook_parameter_cmd/./before_-tl 08:972
__hook_parse_dot_label:nN 08:328, 08:330, 08:330, 08:430, 08:478
__hook_parse_dot_label:w 08:330, 08:342, 08:345
__hook_parse_dot_label_aux:w 08:330, 08:348, 08:356
__hook_parse_dot_label_cleanup:w 08:330, 08:352, 08:355
__hook_parse_label_default:nN 08:324, 08:324, 08:391, 08:397, 08:398, 08:405, 08:406, 08:408
__hook_patch_check:NNnn 09:93, 09:97, 09:100, 09:103, 09:113
__hook_patch_cmd_or_delay:Nnn 09:32, 09:45, 09:58, 09:62, 09:62, 09:72, *335*
__hook_patch_command:Nnn 09:72, 09:90, 09:93, 09:93, *335*
__hook_patch_debug:n 09:18, 09:18, 09:95, 09:96, 09:99, 09:102, 09:105, 09:174, 09:249, 09:405, 09:441, 09:444, 09:445, 09:450, 09:460, 09:463, 09:464, 09:469
__hook_patch_DeclareRobustCommand:Nnn 09:142, 09:146, 09:146, *337*
__hook_patch_DeclareRobustCommand_-aux:Nnn 09:148, 09:151

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

__hook_patch_expand_redefine:NNnn 09:157, 09:162, 09:167, 09:170, 09:170, 09:172, 09:245, 09:247, 09:403, *345*
__hook_patch_newcommand:Nnn 09:143, 09:156, 09:160, 09:160, *337*
\l__hook_patch_num_args_int 09:7, 09:175, 09:180, 09:183, 09:197, 09:227, 09:236, 09:250, 09:255, 09:258, 09:272, 09:534, 09:537, 09:557, 09:564
\l__hook_patch_prefixes_tl 09:8, 09:222, 09:297, 09:314, *342*
__hook_patch_required_catcodes: 09:475, 09:475, 09:496, 09:572, 09:600, *350*
__hook_patch_retokenize:Nnnn 09:446, 09:465, 09:530, 09:530, 09:532, 09:577, 09:579, *346*
__hook_post_initialization_-defs: . 08:2339, 08:2339, 08:2341, 08:2346, 08:2349, 08:2351, 08:1465
__hook_preamble_hook:n 08:1874, 08:1992, 08:2244, 08:2248, 08:2259, 08:2272, 08:2282, 08:2292, 08:2311, 08:2320, 08:2345, 08:2378, 08:2413, 08:2432, 08:1488, *307*
__hook_print_args:n 08:2095
__hook_print_args:nn 08:1882, 08:2095
__hook_prop_gput_labeled_-cleanup:nnn . 08:491, 08:551, 08:577
__hook_prop_gput_labeled_do:Nnn 08:591, 08:594
__hook_prop_gput_labeled_-do:Nnnn 08:491
\l__hook_rear_tl 08:1570, 08:1601, 08:1607, 08:1608, 08:1631, 08:1632, 08:1684, 08:1692, 08:1693, 08:1720, 08:1721
__hook_redefine_with_hooks:Nnnn . . 09:170, 09:230, 09:302, 09:312, *341*
\l__hook_replace_text_tl 09:8, 09:192, 09:199, 09:200, 09:201, 09:206, 09:213, 09:240, 09:267, 09:274, 09:275, 09:276, 09:281, 09:288, 09:308, *339*
__hook_replacement_spec:N 08:1218, 08:1224
__hook_replacing_args_false: . . . 08:2158, 08:2415, 08:2587, 08:2601, 08:231, 08:495, 08:663, 08:1451, *1430*
__hook_replacing_args_reset: . . . 08:2168, 08:2182, 08:2418, 08:2587, 08:2607, 08:237, 08:497, 08:503, 08:1454
__hook_replacing_args_true: 08:2172, 08:2587, 08:2595, 08:501, 08:1452
\g__hook_replacing_stack_seq 08:2587
__hook_retokenize_patch:Nnn 09:108, 09:400, 09:400
\l__hook_return_tl 08:1617, 08:1618, 08:1704, 08:1705, 08:2609, 08:2610, 08:25, 08:451, 08:452, 08:459, 08:463, 08:579, 08:587, 08:592, 08:596, 08:597, 08:642, 08:645, 08:999, 08:1034
__hook_rollback_tidying: . . 08:2969
__hook_rule_<_gset:nnn 08:1385
__hook_rule_>_gset:nnn 08:1385
__hook_rule_after_gset:nnn 08:1385, 08:1392, 08:1398
__hook_rule_before_gset:nnn 08:1385, 08:1385, 08:1391, *292*
__hook_rule_gclear:nnn 08:1348, 08:1373, 08:1415, 08:1416, *323*
__hook_rule_incompatible-error_-gset:nnn 08:1405
__hook_rule_incompatible-warning_-gset:nnn 08:1405
__hook_rule_unrelated_gset:nnn 08:1415, 08:1415, *286*
__hook_rule_voids_gset:nnn 08:1399, 08:1399
__hook_seq_csname:n . . . 08:1568, 08:1569, 08:1587, 08:1621, 08:1669, 08:1710, 08:1770, 08:1777, 08:1835
__hook_set_default_hook_label:n 08:2821, 08:467, 08:467
__hook_set_default_label:n 08:467, 08:475, 08:482
__hook_set_normalize_fn:nn 08:584, 08:1158, 08:1162, 08:1167, *279*
__hook_str_compare:nn 08:23, 08:23, 08:1420, 08:1428, 08:1437
__hook_strip_double_slash:n 08:921, 08:927, 08:928
__hook_strip_double_slash:w 08:921, 08:929, 08:930, 08:934
__hook_tl_csname:n 08:1568, 08:1568, 08:1574, 08:1586, 08:1602, 08:1605, 08:1607, 08:1611, 08:1623, 08:1625, 08:1628, 08:1631, 08:1636, 08:1668, 08:1685, 08:1689, 08:1691, 08:1696, 08:1712, 08:1714, 08:1717, 08:1719, 08:1725, 08:1768, 08:1769, 08:1775, 08:1776, 08:1834
__hook_tl_gclear:N 08:1612, 08:1697, 08:2455, 08:2456,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

08:2457, 08:58, 08:58, 08:60, 08:238, 08:253, 08:1026, 08:1027, 08:1031
__hook_tl_gput:Nn 08:1549, 08:1553, 08:1618, 08:1645, 08:1705, 08:1734, 08:1740, 08:1740, 08:1516, 08:1518, *293*
__hook_tl_gput_left:Nn 08:1550, 08:52, 08:52, 08:1516
__hook_tl_gput_right:Nn 08:1554, 08:1647, 08:1736, 08:1738, 08:2229, 08:49, 08:49, 08:51, 08:636, 08:1518
__hook_tl_gset:Nn 08:1541, 08:2228, 08:2442, 08:2980, 08:43, 08:43, 08:45, 08:47, 08:48, 08:50, 08:54, 08:1387, 08:1394, 08:1401, 08:1406, 08:1411
__hook_tl_gset_eq:NN 08:57, 08:57, 08:59
__hook_tl_set:Nn 08:1586, 08:1668, 08:41, 08:41, 08:1173, *247*
__hook_tmp:w 08:1905, 08:1908, 08:2110, 08:2132, 08:2141, 08:2152, 08:2978, 08:2995, 08:2996, 09:189, 09:195, 09:196, 09:198, 09:264, 09:270, 09:271, 09:273, 09:487, 09:490, 09:494, 09:546, 09:549, 09:570, 09:582, 09:585, 09:598, 08:34, 08:34, 08:414, 08:418, 08:420, 08:1169, 08:1180, 08:1196, 08:1202, 08:1205, *350*
\l__hook_tmpa_bool 08:1934, 08:1937, 08:1945, 08:1954, 08:2034, 08:2037, 08:2045, 08:2054, 08:24, *300*
\l__hook_tmpa_tl . . 09:182, 09:184, 09:186, 09:257, 09:259, 09:261, 09:406, 09:422, 09:425, 09:493, 09:496, 09:538, 09:542, 09:558, 09:565, 09:569, 09:572, 09:597, 09:600, 08:25, 08:424, 08:1181, 08:1203
\l__hook_tmpb_tl . . 09:411, 09:423, 09:426, 08:25, 08:1170, 08:1177, 08:1205
__hook_toplevel␣⟨*hook*⟩ *274*
__hook_toplevel_gset:nn 08:2447, 08:147, 08:991, 08:996, 08:1111, 08:1115, 08:1130
__hook_try_declaring_generic_-hook:nnn 08:531, 08:619, 08:678, 08:679, 08:681, 08:697, 08:699, 08:718, 08:721, *267*
__hook_try_declaring_generic_-hook:nNNnn 08:723, 08:729, 08:732, 08:732, *268*
__hook_try_declaring_generic_-hook:wn . 08:752, 08:755, 08:810, 08:813, 08:840, 08:843, 08:872, 08:875
__hook_try_declaring_generic_-hook:wnTF 08:683, 08:690, 08:701, 08:710, 08:745, 08:751, 08:804, *269*
__hook_try_declaring_generic_-hook_split:nNNnn 08:732, 08:737, 08:740, 08:743
__hook_try_declaring_generic_-next_hook:nn 08:2199, 08:678, 08:688, 08:708, 08:727, *267*
__hook_try_file_hook:n 08:2353, 08:2361, 08:2364, *310*
__hook_try_patch_with_catcodes:Nnnnw . . . 09:419, 09:436, 09:436, 09:438, 09:451, 09:455, 09:457, 09:470, *345*
__hook_try_put_cmd_hook:n 09:20, 09:20, 09:22, 09:37, 09:39, 09:48, 09:50, 08:764, 08:821, 08:854
__hook_try_put_cmd_hook:w 09:20, 09:23, 09:24, 09:40, 09:41, 09:51, 09:52
__hook_unpatchable_cases:n 09:625, 09:627
__hook_update_hook_code:n 08:2210, 08:2226, 09:242, 09:574, 08:76, 08:297, 08:315, 08:526, 08:614, 08:1004, 08:1040, 08:1352, 08:1377, 08:1441, 08:1441, 08:1446, 08:1453, 08:1472, 08:1476, *287*
__hook_use:wn . . 08:2295, 08:2309, 08:2353, 08:2353, 08:2356, 08:2358, *310*
__hook_use_end: 08:2303, 08:2306, 08:2313
__hook_use_i_delimit_by_s_-mark:nw 08:39, 08:40, 08:1244
__hook_use_initialized:n 08:2244, 08:2249, 08:2251, 08:2276, 08:2297, 08:2343, 08:2420, 08:2434, 08:1487, *307*
__hook_use_initialized:nnw 08:2316, 08:2321, 08:2323, 08:2344
__hook_use_none_delimit_by_s_-mark:w 08:2463, 08:2530, 09:523, 08:39, 08:39, 08:1339, 08:1345
__hook_use_once:n . . 08:2403, 08:2430
__hook_use_once:nn 08:2388, 08:2394, 08:2409, 08:2409, 08:2411, 08:2428, 08:2437
__hook_use_once_clear:n 08:2417, 08:2435, 08:2439, 08:2439, 08:2443, 08:2451, 08:2453, *312*
__hook_use_once_set:n 08:2414, 08:2433, 08:2439, 08:2441, *312*
__hook_use_undefined:w 08:2301, 08:2305
\g__hook_used_prop 08:1562, 08:31, 08:1447,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

08:1459, 08:1474, 08:1480, 08:1524
\l__hook_work_prop 08:1557, 08:1582, 08:1589, 08:1591, 08:1600, 08:1617, 08:1642, 08:1664, 08:1671, 08:1673, 08:1683, 08:1703, 08:1731, 08:1802, 08:1813, 08:30, 08:587, 08:1163, 08:1165, 08:1199, 08:1520, *292*
hook␣?? internal commands:
__hook␣?? *283*
hook␣⟨*hook*⟩ internal commands:
__hook␣⟨*hook*⟩ *248*
\hookleftarrow 30:491
\hookrightarrow 30:489
Hooks:
.../after *1169*
/after *1160*, *1160*, *1161*, *1169*
/before *1160*, *1160*, *1169*
/end *1169*
A/foo *227*, *227*
after *241*
babel/⟨*language*⟩/afterextras . . . *236*
begindocument . *327*, *354*, *354*, *489*, *489*, *1204*, *1208*, *239*, *239*, *239*, *242*, *250*
begindocument/before *489*, *242*
begindocument/end *242*
bfseries *244*, *244*
bfseries/defaults *244*, *244*
build/column/after 54:731, *1232*
build/column/before 54:731, *1232*
build/page/after 54:729, *1232*
build/page/before 54:729, *1232*
build/page/reset 54:730, *1232*
class *236*
class/ *1162*, *1169*
class/⟨*name*⟩/after *1161*
class/⟨*name*⟩/before *1161*
class/after *1161*
class/before *1161*
cmd *315*, *330*, *331*, *230*, *236*, *236*, *269*, *274*
cmd/#1/after *342*
cmd/#1/before *342*
cmd/⟨*name*⟩/after *327*, *330*, *241*
cmd/⟨*name*⟩/before *327*, *327*, *241*
cmd/foo/before *329*
cmd/include/after *1163*
cmd/include/before *1163*
cmd/section/after *330*, *330*
cmd/section/before *330*
enddocument *1206*, *238*, *239*, *243*
enddocument/afteraux *243*
enddocument/afterlastpage . *1206*, *243*
enddocument/end *243*
enddocument/info *1182*, *243*
env *227*, *236*
env/⟨*env*⟩/after *240*, *240*, *241*
env/⟨*env*⟩/before *240*, *240*, *241*
env/⟨*env*⟩/begin *487*, *240*, *241*
env/⟨*env*⟩/end *240*, *241*
env/document/after *244*
env/document/before *242*
env/document/begin *489*, *242*
env/document/end *244*
env/quote/after *234*, *234*
expand@font@defaults *244*
file *227*, *236*
file/ *1169*
file/.../after *1169*
file/⟨*file name*⟩/after . . . *1160*, *1161*
file/⟨*file name*⟩/before . . *1160*, *1161*
file/⟨*file:name*⟩/after *1160*
file/⟨*file:name*⟩/before *1160*
file/⟨*package name*⟩.sty/after . *1162*
file/⟨*package name*⟩.sty/before *1162*
file/after *1160*, *1160*, *1162*, *1169*, *1182*
file/array.sty/after *1161*
file/before *315*, *1160*, *1160*, *1162*, *1169*
foo *313*
include *496*, *1163*, *236*
include/ *1169*, *1169*
include/.../end *1169*
include/⟨*name*⟩/after *1163*
include/⟨*name*⟩/before *1163*
include/⟨*name*⟩/end *1163*
include/⟨*name*⟩/excluded . . *1163*, *1163*
include/after *1163*
include/before *1163*
include/end *1163*
include/excluded *1163*, *1163*
insertmark *1059*, *1078*, *245*
mdseries *244*
mdseries/defaults *244*
myhook *221*
normalfont *244*
package *236*
package/ *1162*, *1169*
package/⟨*name*⟩/after *1161*
package/⟨*name*⟩/before *1161*
package/⟨*package name*⟩/after . . *1162*
package/⟨*package name*⟩/before . *1162*
package/after *1161*, *1162*
package/before *1161*, *1162*
para/after *443*, *454*
para/before *443*, *449*
para/begin *443*, *445*, *452*
para/end *443*, *444*, *454*
rmfamily *244*, *244*, *244*
selectfont *245*
sffamily *244*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

shipout . . *1186*, *1186*, *1187*, *1188*, *1188*
shipout/... . . . . *1184*, *1188*
shipout/after . . *1184*, *1184*, *1185*, *1186*, *1186*, *1187*, *1189*, *1189*, *1198*
shipout/afterpage *1201*
shipout/background *1185*, *1185*, *1187*, *1188*, *1190*, *1190*, *1191*, *1195*, *1197*, *1200*, *1200*
shipout/before . . *354*, *1184*, *1184*, *1185*, *1185*, *1185*, *1186*, *1186*, *1186*, *1186*, *1186*, *1188*, *1188*, *1188*, *1188*, *1189*, *1189*, *1191*, *1191*, *1196*, *1197*
shipout/firstpage *1184*, *1185*, *1185*, *1186*, *1187*, *1187*, *1187*, *1197*, *1197*, *1200*, *1200*, *1201*, *1201*, *1212*
shipout/foreground . . . *1185*, *1186*, *1187*, *1188*, *1190*, *1190*, *1191*, *1197*
shipout/lastpage *1184*, *1185*, *1186*, *1187*, *1187*, *1187*, *1187*, *1196*, *1198*, *1198*, *1201*, *1206*, *1206*, *1207*, *1207*, *243*
tbl/cell/begin 41:469
tbl/cell/end 41:469
tbl/cell/init 41:468
tbl/init 41:465
tbl/row/begin 41:467
tbl/row/end 41:467
tbl/row/init 41:466
test *237*, *237*
top:level *220*, *220*, *239*
ttfamily *244*

hook ?? internal commands:

__hook~?? 08:1314
\hphantom 38:75
\hrule . . 18:419, 18:427, 18:453, 18:461, 21:309, 21:314, 30:351, 30:629, 40:207, 40:212, 40:285, 40:295, 40:637, 41:436, 41:453, 42:600, 42:610, 45:395, 02:377, 02:421, *945*
\hrulefill 06:976, 06:997, 02:421
\hsize *1401*
\hskip *1412*
\hskip... . . . . *1397*
\hspace 18:534, 18:538, 18:544, *482*
\hss *1192*
\Hwithstroke 21:517, 21:1241, *529*
\hwithstroke 21:533, 21:1242, *1412*
\hyphenation 21:225, *1386*
\hyphenchar 37:566, 06:934, 06:937, 06:946, 06:949, 06:952, 06:957, *1410*
\hyphenpenalty 24:760, 24:792, 02:191

I

\I 50:1378, 50:1510, 50:1600, 50:1620, 57:255, 57:599, 02:312
\i 21:267, 21:423, 21:473, 21:474, 21:475, 21:476, 21:477, 21:478, 21:479, 21:570, 21:610, 21:611, 21:703, 21:705, 21:707, 21:709, 21:737, 21:802, 21:1168, 21:1323, 21:1325, 21:1327, 21:1329, 21:1380, 21:1383, 21:1386, 21:1389, 21:1459, 57:259, 57:603, *1403*
\ialign 30:348, 30:470, 30:541, 30:544, 30:548, 30:551, 38:168, 38:170, 38:189, 41:243, 41:266, 42:152, 02:409, 02:411, *1362*
identity (plug) *356*, *359*, *359*, *359*
\IeC 57:406, 57:410, 57:517
\if *871*

if commands:

\if:w 08:2464, 08:2478, 08:2531, 08:2563, 09:359, 08:1058, 08:1218, 08:1243, *312*
\if_case:w 07:1504, 08:1269, 08:1420, 08:1437
\if_catcode:w 09:377, 09:385, 07:1840, 07:1841, 07:1842, 07:1843, *344*
\if_charcode:w 08:2581, 07:986
\if_cs_exist:N 48:101
\if_cs_exist:w 08:1751, 08:2131, 08:2253, 08:2261, 08:2278, 08:2299, 08:2326, 08:2465, 08:789
\if_false: 07:905, 07:908, 07:2656, 07:2660
\if_int_compare:w 16:103, 08:1428
\if_meaning:w 09:356, 09:360, 09:361, 09:388, 52:241, 07:1343
\if_mode... . . . . *453*
\if_mode_horizontal: . . . 16:32, 16:64
\if... . . . . *1150*
\IfBlankF 07:3302, *120*
\IfBlankT 07:3302, *120*
\IfBlankTF 47:19, 07:3302, *120*
\IfBold *753*
\IfBooleanF 22:210, 22:225, 22:263, 22:275, 07:3261, *121*
\IfBooleanT 07:3261, *121*
\IfBooleanTF 17:126, 17:150, 35:67, 35:69, 35:207, 07:3261, *121*
\IfClassAtLeastF 50:318
\IfClassAtLeastT 50:318
\IfClassAtLeastTF *1097*
\IfClassAtLeastTF 50:200, 50:328, 50:329, 50:357, 50:358
\IfClassLoadedF 50:318
\IfClassLoadedT 50:318
\IfClassLoadedTF *1097*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\IfClassLoadedTF 50:298, 50:324, 50:325, 50:353, 50:354
\IfClassLoadedWithOptionsF 50:318
\IfClassLoadedWithOptionsT 50:336, 50:365
\IfClassLoadedWithOptionsTF *1097*
\IfClassLoadedWithOptionsTF 50:298, 50:318
\ifcoremisses 33:1186, 33:1377
\ifcsname 21:11, 21:210, 24:403, 25:2828, 25:2831, 25:3173, 25:3176, 26:129, 28:154, 05:137, 29:43, 29:54, 29:76, 29:87, 33:1212, 06:832, 06:849, 50:1263, 50:1398, 53:486, 54:2480, *668*
\ifdefined 05:26, 05:76, 05:77, 05:78, 05:79, 05:122, 05:123, 05:124, 29:734, 40:350, 40:644, 45:418, 45:477, 57:318, 57:329, 57:332, 57:356, 57:373, 57:725, 57:730, 02:217, 02:444, 02:508
\IfDocumentMetadataF 17:7, 17:13, 50:807, 50:813
\IfDocumentMetadataT 17:6, 17:12
\IfDocumentMetadataTF . . 17:5, 17:11, *1437*
\IfExplAtLeastF *78*
\IfExplAtLeastF 05:217, 05:225
\IfExplAtLeastT *78*
\IfExplAtLeastT 05:216, 05:224
\IfExplAtLeastTF *78*
\IfExplAtLeastTF 05:212, 05:221, 05:223, 05:224, 05:225, *78*
\iff 30:511
\IfFileAtLeastF 50:318
\IfFileAtLeastT 50:318
\IfFileAtLeastTF *1097*
\IfFileAtLeastTF 50:200, 50:330, 50:331, 50:359, 50:360, *1428*
\IfFileExists *485*, *1098*
\IfFileExists 20:488, 20:500, 20:636, 20:692, 20:717, 01:162, 05:42, 05:73, 05:119, 50:968, 52:149, 52:185, 52:195, 57:733, *505*
\IfFileLoadedF 50:338
\IfFileLoadedT 50:338
\IfFileLoadedTF 50:338, *1431*
\iffontchar 21:890, 21:1088, 21:1193, 21:1195, 21:1197, 21:1244, 33:1182, 33:1183
\IfFontSeriesContextF . . . 29:595, 29:617
\IfFontSeriesContextT . . . 29:594, 29:616
\IfFontSeriesContextTF 29:575, 29:613, 29:615, *754*
\IfFormatAtLeastF 50:318
\IfFormatAtLeastT 50:318
\IfFormatAtLeastTF *1097*
\IfFormatAtLeastTF 50:200, 50:332, 50:333, 50:361, 50:362, *877*
\IfHookEmptyF 08:2855, *1434*
\IfHookEmptyT 08:2855, *228*
\IfHookEmptyTF 08:2855, 08:2960, 37:351, 37:368, *323*
\IfHookExistsTF . . . 08:2858, 08:2959, *323*
\ifincsname 18:497, 06:297, 06:318, *93*
\ifinner 18:261, 18:312, 38:277, 38:285, 38:324, 38:350, 38:376, 45:57, 45:126, 45:315
\IfInstanceExistsF 11:1610, *376*
\IfInstanceExistsT 11:1610, *376*
\IfInstanceExistsTF 11:1610, *376*
\IfLabelExistsF 36:203, *853*
\IfLabelExistsT 36:203, *1434*
\IfLabelExistsTF 36:203, 36:298, *853*
\IfLabelExistTF *1432*
\IfMarksEqualF 48:408, *1060*
\IfMarksEqualT 48:408, *1434*
\IfMarksEqualTF 48:408, 48:525, *1060*
\ifmmode *1357*
\IfNoValue(TF) *119*
\IfNoValueF 07:3296, *120*
\IfNoValueT 07:3296, *120*
\IfNoValueTF 11:1571, 36:142, 57:684, 57:691, 57:698, 07:3296, *119*
\ifnum *1189*
\ifodd 28:1228, 40:344, 42:331, 42:355, 42:389, 42:411, 45:68, 45:137, 54:17, 54:117, 54:891, 54:965, 54:1023, 54:1333, 54:1336, 54:1369, 54:1372, 54:1484, 54:1487, 54:1633, 54:1636, 54:1790, 54:1793, 54:2151, 54:2154, 54:2272, 54:2275, 54:2395, 54:2660, 54:2668
\IfPackageAtLeastF 50:318
\IfPackageAtLeastT 50:318
\IfPackageAtLeastTF *1097*
\IfPackageAtLeastTF 50:200, 50:326, 50:327, 50:355, 50:356
\IfPackageLoadedF 50:318
\IfPackageLoadedT 50:318
\IfPackageLoadedTF *1097*
\IfPackageLoadedTF 50:298, 50:322, 50:323, 50:351, 50:352
\IfPackageLoadedWithOptionsF . . . 50:318
\IfPackageLoadedWithOptionsT . . . 50:318
\IfPackageLoadedWithOptionsTF . . . *1097*
\IfPackageLoadedWithOptionsTF 50:298, 50:334, 50:335, 50:363, 50:364
\IfPDFManagementActiveF 57:804
\IfPDFManagementActiveT . . 53:11, 57:803
\IfPDFManagementActiveTF . . . 57:802, *1437*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\IfPropertyExistsF 36:188, *853*
\IfPropertyExistsT 36:188, *853*
\IfPropertyExistsTF . . 36:188, 36:297, *853*
\IfPropertyExistTF *1432*
\IfPropertyRecordedF 36:218, *1434*
\IfPropertyRecordedT 36:218, *853*
\IfPropertyRecordedTF 36:218, 36:299, *853*
\ifsafesubencodingfound . . . . . .
. . 33:1185, 33:1199, 33:1204, 33:1373
\IfSocketExistsF 10:200, 10:226
\IfSocketExistsT 10:200, 10:225
\IfSocketExistsTF . . . 10:200, 10:224, *360*
\IfSocketPlugAssignedF . . 10:200, 10:232
\IfSocketPlugAssignedT . . 10:200, 10:231
\IfSocketPlugAssignedTF . . . . . .
. . . . . 10:200, 10:230, *360*
\IfSocketPlugExistsF 10:200, 10:229
\IfSocketPlugExistsT 10:200, 10:228
\IfSocketPlugExistsTF 10:200, 10:227, *360*
\IfTargetDateBefore 50:1838
\ifthenelse *843*
\IfValue(TF) *120*
\IfValueF 07:3299, *120*
\IfValueT 07:3299, *120*
\IfValueTF 07:3299, *120*
\ifvbox 54:324,
54:381, 54:428, 54:499, 54:756, 54:821
\ifvoid *1399*
\ifx *111*
\ignoreprimitiveerror *1072*
\ignorespaces 18:50,
18:146, 18:167, 18:185, 18:197,
18:208, 18:224, 18:237, 18:581,
20:71, 20:139, 20:194, 21:90, 24:355,
24:365, 24:375, 24:428, 28:317,
28:356, 37:341, 37:358, 37:374,
37:383, 37:432, 37:437, 37:443,
38:332, 38:358, 38:384, 38:538,
38:539, 39:55, 39:248, 40:198,
40:553, 40:570, 40:587, 41:57,
41:62, 41:68, 41:83, 41:92, 41:105,
41:109, 41:116, 41:123, 41:125,
41:134, 41:154, 41:314, 41:378,
41:380, 41:382, 41:409, 42:47, 42:57,
42:77, 42:86, 42:120, 42:131, 42:142,
42:154, 42:159, 42:165, 43:57, 43:59,
44:110, 45:17, 45:24, 45:539, 45:557,
45:575, 47:7, 47:9, 53:33, 53:55, *715*
\ignorespacesafterend 37:7
\IJ 21:270,
21:457, 21:573, 21:1169, 57:720, *1405*
\ij 21:269,
21:455, 21:572, 21:1170, 57:720, *1405*
\Im 30:325
\imath 30:320
\immediate *1206*
\in 30:440, 30:472
in commands:
in_callback 04:825
\in_callback *44*
\include *485*
include (hook) *496*, *1163*, *236*
\include 20:237, 20:285, 20:287,
20:303, 20:305, 20:364, 20:416, *1162*
include/ (hook) *1169*, *1169*
include/.../after *1163*
include/.../before *1163*
include/.../end (hook) *1169*
include/.../end *1163*
include/⟨*name*⟩/after (hook) *1163*
include/⟨*name*⟩/before (hook) *1163*
include/⟨*name*⟩/end (hook) *1163*
include/⟨*name*⟩/excluded (hook) *1163*, *1163*
include/after (hook) *1163*
include/after *1163*
include/before (hook) *1163*
include/before *1163*
include/end (hook) *1163*
include/end *1163*
include/excluded (hook) *1163*, *1163*
\IncludeInRelease 02:561,
08:1575, 02:566, 08:1657, 02:576,
08:1864, 08:1982, 08:2085, 08:2090,
08:2154, 08:2185, 08:2203, 08:2216,
08:2234, 08:2239, 08:2244, 08:2266,
08:2286, 08:2316, 08:2334, 08:2339,
08:2349, 08:2353, 08:2356, 08:2383,
08:2398, 08:2409, 08:2428, 08:2439,
08:2451, 03:71, 08:2474, 08:2491,
08:2557, 08:2573, 08:2630, 08:2659,
08:2767, 08:2776, 08:2782, 08:2789,
08:2794, 08:2801, 08:2805, 08:2812,
08:2832, 08:2839, 08:2933, 09:20,
09:37, 09:48, 09:170, 09:245, 03:180,
03:184, 03:188, 09:436, 03:192,
09:455, 09:501, 09:526, 09:530,
09:577, 09:604, 09:607, 09:637,
10:210, 04:3, 11:1634, 14:138,
14:158, 14:192, 14:205, 16:12,
16:45, 16:169, 04:234, 17:59, 17:139,
17:163, 17:178, 18:5, 18:22, 04:257,
18:55, 18:66, 18:82, 18:87, 18:99,
18:105, 18:133, 18:156, 18:175,
18:189, 18:201, 18:214, 18:229,
04:280, 18:258, 18:273, 18:289,
18:308, 18:342, 18:376, 18:402,
18:435, 18:493, 18:504, 18:511,
18:517, 18:523, 18:529, 18:537,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

18:543, 18:557, 18:563, 20:10, 20:84, 20:142, 20:209, 20:222, 20:239, 20:276, 20:297, 20:312, 20:373, 20:422, 20:483, 20:493, 20:519, 20:551, 20:573, 20:591, 20:601, 20:617, 20:642, 20:647, 20:655, 20:669, 20:680, 20:696, 20:729, 21:95, 21:122, 21:166, 21:187, 21:344, 21:352, 21:373, 21:389, 22:41, 22:47, 22:54, 22:77, 22:96, 22:118, 22:134, 22:142, 22:148, 22:171, 22:197, 22:205, 22:219, 22:233, 22:249, 22:258, 22:270, 22:281, 22:296, 22:328, 22:344, 22:352, 22:370, 23:5, 23:11, 23:19, 23:26, 24:25, 24:54, 24:206, 24:228, 24:273, 24:295, 24:347, 24:358, 24:368, 24:424, 24:439, 24:517, 24:528, 24:536, 24:568, 24:576, 24:590, 24:611, 24:647, 24:734, 24:798, 25:3, 25:1432, 25:2773, 25:2780, 25:2792, 25:2804, 25:2812, 25:2894, 25:2910, 25:2931, 25:2946, 25:3048, 25:3129, 25:3133, 25:3143, 25:3150, 25:3158, 25:3230, 25:3249, 25:3255, 25:3259, 26:114, 05:2, 26:159, 26:162, 05:14, 05:24, 26:558, 26:567, 27:2, 27:22, 05:63, 28:50, 28:79, 28:139, 28:181, 28:211, 28:242, 28:276, 28:323, 28:369, 28:418, 05:117, 28:470, 28:483, 28:489, 28:529, 28:577, 28:604, 28:629, 05:147, 28:862, 28:910, 05:162, 28:956, 28:965, 28:1149, 28:1160, 05:173, 05:183, 29:34, 29:69, 29:102, 05:192, 29:127, 29:144, 29:160, 05:206, 29:233, 29:286, 29:297, 29:321, 29:331, 05:213, 29:380, 05:220, 29:427, 29:456, 29:474, 05:228, 29:514, 05:235, 29:548, 29:577, 29:612, 06:5, 29:626, 06:13, 29:660, 29:673, 29:732, 29:741, 29:766, 29:798, 29:814, 29:830, 29:847, 29:864, 29:870, 30:75, 30:93, 30:112, 30:121, 06:56, 30:633, 30:645, 06:62, 32:27, 32:34, 06:74, 06:83, 33:571, 06:136, 06:149, 35:12, 35:29, 35:46, 35:62, 35:76, 35:96, 35:110, 01:274, 35:125, 35:132, 35:164, 35:174, 35:183, 35:191, 35:198, 35:210, 35:216, 35:222, 06:259, 06:266, 36:287, 37:10, 37:78, 06:291, 37:142, 37:184, 37:189, 37:201, 37:228, 37:247, 06:307, 37:269, 37:297, 37:314, 37:336, 37:347, 37:363, 37:377, 37:385, 37:396, 37:404, 37:417, 37:423, 37:435, 37:440, 37:450, 37:469, 37:486, 37:502, 37:513, 37:527, 37:549, 01:21, 37:576, 37:592, 37:605, 37:619, 37:631, 37:637, 37:643, 37:650, 37:668, 37:681, 37:697, 37:720, 37:730, 38:79, 38:87, 38:109, 38:118, 38:137, 38:144, 38:152, 38:157, 38:165, 38:176, 38:216, 06:376, 38:233, 01:291, 38:272, 38:280, 38:291, 38:297, 38:307, 38:336, 38:361, 38:396, 38:403, 38:422, 38:450, 38:492, 38:501, 38:534, 38:542, 38:555, 38:567, 38:579, 38:588, 06:418, 39:125, 39:135, 39:143, 39:162, 06:446, 40:4, 40:14, 40:42, 40:58, 40:71, 40:91, 40:112, 40:122, 40:136, 40:142, 40:147, 40:152, 40:161, 40:169, 06:474, 40:225, 40:233, 40:246, 06:481, 40:266, 40:304, 40:313, 40:331, 40:357, 40:367, 40:392, 40:418, 06:498, 40:441, 40:463, 40:540, 40:558, 06:514, 40:574, 40:594, 40:600, 40:616, 40:655, 40:668, 40:676, 40:706, 40:717, 41:60, 41:65, 41:137, 41:157, 41:169, 41:177, 41:186, 41:192, 41:203, 41:211, 41:219, 41:225, 41:231, 41:255, 41:299, 41:304, 06:552, 06:564, 42:10, 42:16, 42:23, 42:28, 42:37, 42:50, 42:66, 42:80, 42:91, 42:100, 42:111, 42:123, 42:157, 42:162, 42:171, 42:183, 42:246, 42:261, 42:322, 06:594, 42:380, 42:466, 42:474, 06:605, 42:483, 42:511, 42:541, 42:568, 42:582, 42:595, 42:606, 42:618, 42:635, 42:655, 42:662, 42:696, 06:630, 42:756, 42:814, 42:830, 06:638, 43:22, 43:31, 43:42, 43:48, 43:64, 43:71, 44:5, 44:20, 06:650, 06:657, 44:161, 44:168, 44:174, 44:182, 44:196, 44:211, 44:218, 44:227, 44:250, 45:35, 45:105, 45:206, 45:232, 45:280, 45:294, 45:335, 45:352, 45:405, 45:438, 45:443, 45:451, 45:458, 45:464, 45:497, 45:502, 45:508, 45:525, 45:544, 45:561, 45:611, 45:621, 47:14, 47:25, 47:61, 47:77, 01:26, 06:756, 06:773, 48:517,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

49:20, 49:35, 49:61, 49:82, 49:108, 49:116, 50:18, 50:23, 50:36, 50:51, 06:822, 50:68, 50:80, 50:104, 50:112, 50:118, 06:828, 50:134, 50:144, 50:161, 50:179, 50:202, 50:210, 50:222, 50:235, 50:261, 50:280, 50:300, 50:308, 50:320, 50:348, 50:380, 50:396, 06:858, 50:410, 50:428, 50:441, 50:457, 50:477, 50:500, 50:516, 50:545, 50:559, 50:591, 50:609, 50:629, 50:647, 50:666, 50:676, 50:686, 50:698, 50:730, 50:741, 50:755, 50:765, 50:819, 50:833, 50:848, 50:878, 50:905, 50:937, 06:899, 50:1089, 06:918, 50:1169, 50:1176, 50:1187, 50:1201, 50:1251, 06:929, 50:1386, 06:946, 50:1517, 06:957, 06:968, 06:988, 52:5, 52:15, 52:27, 52:89, 52:144, 52:181, 52:192, 52:205, 52:249, 52:257, 52:268, 52:299, 52:322, 52:343, 52:357, 52:365, 52:389, 52:408, 52:433, 52:479, 52:495, 52:506, 52:533, 53:3, 53:45, 53:64, 53:483, 53:523, 53:535, 53:544, 53:558, 54:130, 54:158, 54:184, 54:350, 54:371, 54:376, 54:424, 54:488, 54:734, 54:842, 54:947, 54:1006, 54:1145, 54:1163, 54:1224, 54:1245, 54:1281, 54:1305, 54:1417, 54:1569, 54:1713, 07:228, 54:1882, 07:242, 54:1965, 54:2045, 54:2139, 54:2261, 54:2475, 54:2498, 54:2512, 54:2540, 54:2602, 54:2607, 54:2725, 54:2775, 54:2836, 54:2854, 54:2873, 54:2903, 54:2944, 55:464, 57:8, 57:16, 57:23, 57:30, 57:37, 57:52, 57:71, 57:80, 57:87, 57:106, 57:143, 57:166, 57:199, 57:290, 57:295, 57:316, 57:368, 57:390, 57:397, 57:490, 57:723, 57:728, 07:1131, 07:1183, 07:1186, 07:1201, 07:1214, 07:1229, 07:1347, 07:1350, 07:1379, 07:1382, 07:1391, 07:1399, 07:1402, 07:1569, 02:226, 02:236, 07:2195, 07:2208, 07:2239, 07:2251, 07:2267, 02:329, 07:3139, 07:3192, 02:347, 07:3302, 07:3308, 07:3324, 08:61, 08:80, 08:96, 08:124, 08:140, 08:152, 08:164, 08:180, 08:193, 08:213, 08:223, 08:243, 08:258, 08:276, 08:282, 08:302, 08:320, 02:432, 08:491, 02:455, 08:601, 08:652, 08:669, 08:678, 08:696, 08:717, 08:751, 08:809, 08:839, 08:871, 08:898, 08:901, 02:489, 08:921, 08:924, 08:937, 08:955, 08:960, 08:963, 08:972, 02:496, 08:977, 08:980, 08:1017, 08:1054, 08:1104, 08:1111, 08:1127, 08:1134, 08:1154, 08:1158, 02:521, 08:1210, 08:1214, 08:1230, 08:1234, 08:1262, 08:1265, 08:1288, 08:1292, 08:1309, 08:1317, 08:1321, 08:1330, 08:1361, 08:1442, 08:1468, 08:1495, 08:1529, *76*

`\includeonly` *485*

`\includeonly` 20:237, 20:277, 20:279, 20:298, 20:299, *1162*

`\indent` 18:560, 39:192, 41:81, *454*

`\IndentBox` 16:93, 16:173, *443*

`\index` *1050*

`\index` 35:103, 44:191, 44:200, 46:6, 46:18, 48:281, 49:40, 49:51, 49:66, 49:74, 49:89, 49:97, 54:905, 54:975, 54:1034, *1399*

`\indexentry` 46:15

`\inf` 38:25

`\infty` 30:327

`\initcatcodetable` 04:91

`inline/begin` (tag socket) 55:84

`inline/end` (tag socket) 55:84

`\input` *485*, *1098*

`\input` . . . 04:18, 20:652, 01:158, 01:161, 26:16, 01:218, 27:106, 05:113, 05:135, 06:22, 29:908, 29:918, 29:928, 30:10, 30:11, 30:12, 30:13, 30:14, 30:23, 30:42, 30:43, 30:44, 30:48, 30:49, 30:50, 30:51, 30:52, 30:53, 30:58, 30:147, 30:148, 30:149, 30:150, 30:665, 30:666, 30:667, 33:1090, 50:715, 57:164, 57:178, 57:203, 57:283, 57:339, 57:340, 57:378, 57:454, 57:738, 01:52, *1160*

`\input@path` *6*, *1*

`input@path` commands:

`\input@path:` 01:223

`\inputencodingname` 57:431, 57:453, 57:535, *1411*

`\InputIfFileExists` *485*, *1098*

`\InputIfFileExists` 17:9, 20:635, 20:658, 20:673, 20:683, 20:693, 20:747, 21:1574, 24:488, 29:900, 29:910, 29:920, 33:806, 33:1173, 50:1054, 50:1120, 52:143, 56:8, 57:277, *1169*

`\inputlineno` 14:214, 01:311, *1371*

`\insert` 45:527, 45:546, 45:564, 54:820, 54:821, 54:2460, 02:129, 02:165, 02:167, 02:170, 02:202, *441*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\InsertMark 48:399, 48:521, *1059*
insertmark (hook) *1059*, *1078*, *245*
insertmark 48:278, *1059*
\InstanceValue 11:1617, *376*
\int 30:359
int commands:
\int_add:Nn 07:607, 07:1292
\int_case:nnTF 55:9, 55:33, 55:43
\int_compare:nNnTF 08:1605, 08:1627, 08:1638, 08:1688, 08:1716, 08:1727, 08:2097, 08:2103, 08:2422, 08:2619, 09:180, 09:255, 09:408, 09:521, 10:39, 11:23, 11:25, 11:27, 11:31, 11:36, 11:121, 28:288, 28:290, 28:342, 48:81, 48:283, 53:116, 53:172, 53:203, 53:400, 53:411, 53:419, 55:358, 55:367, 55:388, 55:407, 55:410, 55:435, 55:442, 55:453, 07:461, 07:602, 07:902, 07:944, 07:2179, 07:2245, 07:2260, 07:2512, 07:2514
\int_compare:nTF 11:284
\int_compare_p:nNn 08:2966, 08:2967, 07:971
\int_const:Nn 10:37
\int_decr:N 08:1616, 08:1702, 07:536, 07:582, 07:2178
\int_div_truncate:nn 07:1441
\int_eval:n 08:1624, 08:1713, 08:1769, 08:1776, 08:1884, 05:177, 53:402, 07:2526, 08:1270
\int_gadd:Nn 55:347, 55:448
\int_gdecr:N 52:559, 55:419
\int_gincr:N 17:131, 17:155, 48:284, 52:549, 53:148, 53:163, 55:399, 55:416, 07:1115
\int_gset:Nn 24:415, 28:295, 28:347, 52:545, 55:360, 55:403, 55:425, 55:426, 55:445
\int_gzero:N 48:285, 55:394, 55:395, 55:400, 55:413, 55:422
\int_if_odd:nTF 48:439, 48:499
\int_if_zero:nTF 10:33
\int_incr:N 08:1584, 08:1666, 07:363, 07:484, 07:916, 07:997, 07:1004, 07:1259, 07:1565, 07:2188, *166*
\int_new:N . . . 08:1571, 09:7, 11:94, 17:115, 48:278, 52:544, 07:16, 07:17, 07:18, 07:30, 53:393, 53:395, 07:39, 55:100, 55:322, 55:323, 55:338, 07:2158
\int_set:Nn . 09:175, 09:218, 09:250, 09:293, 09:481, 09:482, 09:512, 11:283, 52:212, 52:226, 07:1137, 07:1148, 07:1356, 07:1366, 07:1490, 07:2162, 07:2672, 07:2698, 08:1186
\int_set_eq:NN . . 48:70, 07:792, 07:797
\int_step_function:nN 07:1439
\int_step_inline:nn 48:367
\int_step_inline:nnn 09:183, 09:197, 09:258, 09:272
\int_use:N . . 09:236, 09:557, 09:564, 10:63, 10:89, 10:106, 11:1576, 11:1577, 11:1578, 17:132, 17:156, 28:282, 28:283, 28:285, 28:290, 28:330, 28:331, 28:333, 28:342, 36:256, 36:266, 36:267, 48:105, 48:118, 48:120, 48:255, 52:553, 53:165, 53:177, 07:28, 53:415, 53:423, 55:34, 55:44, 55:371, 55:378, 55:379, 55:431, 55:432, 07:947, 07:1119, 07:1148, 07:1277, 07:1281, 07:1282, 07:1366, 07:1556, *1189*
\int_value:w 53:109, 53:198
\int_zero:N 08:1580, 08:1662, 07:355, 55:113, 07:448, 07:449, 07:798, 07:802, 07:951, 07:1251
\c_max_int 48:70, 53:275, 53:301, 53:326, 53:353
\l_tmpa_int *373*
\c_zero_int 09:180, 09:255, 53:152
int internal commands:
\g__mark_int 48:255, 48:278, 48:283, 48:284, 48:285
\interdisplaylinepenalty 18:14, 38:55, 38:207, 38:486
\interfootlinepenalty 02:302
\interfootnotelinepenalty 18:19, 45:529, 45:548, 45:566, 02:302
\interlinepenalty 18:11, 24:762, 37:538, 37:541, 37:559, 37:562, 44:67, 44:118, 44:234, 44:257, 45:529, 45:548, 45:566, 54:343, 54:1505, 54:1509, 54:1654, 54:1658, 54:1815, 54:1819, 02:201, *1359*
\inteval *77*
\inteval 05:172, 05:187, *77*
\intextsep 54:1488, 54:1492, 54:1507, 54:1510, 54:1517, 54:1637, 54:1641, 54:1656, 54:1659, 54:1666, 54:1794, 54:1800, 54:1817, 54:1820, 54:1829, 54:3004
\intop 30:358, 30:359
\iota 30:287
iow commands:
\iow_char:N 08:1799, 08:1810, 08:2642, 08:2644, 08:2646, 08:2651, 08:2654,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

08:2656, 08:2666, 08:2667, 08:2707, 08:2715, 08:2727, 08:2732, 08:2737, 08:2923, 08:2925, 09:615, 16:155, 16:162, 16:163, 16:164, 16:165, 07:472, 07:532, 07:563, 07:635, 07:713, 07:725, 07:756, 07:1158, 07:1164, 07:2163, 07:3021, 07:3099
\iow_log:n 08:1842
\iow_newline: 52:557, 07:1409, 07:1429, 07:1438, 07:1455, 07:1473, 07:1477, 07:1563
\iow_now:Nn 53:414, 53:527
\iow_shipout:Nn 53:527
\iow_show:n 08:1845, 08:1849, 07:1470, 07:1473
\iow_term:n 08:1620, 08:1640, 08:1641, 08:1643, 08:1707, 08:1729, 08:1730, 08:1732, 08:1798, 08:1809, 08:1823, 08:1829, 08:1830, 08:1831, 08:1834, 08:1838, 08:1856, 08:2140, 08:2145, 08:2160, 08:2174, 09:19, 09:27, 09:44, 09:55, 09:64, 09:66, 09:82, 09:86, 10:25, 11:109, 48:20, 48:92, 48:179, 48:185, 48:191, 48:258, 52:550, 52:583, 07:1477, 08:536, 08:626, 08:1458, 08:1461, 08:1479, 08:1482, 08:1500, 08:1534
\ishortstack 42:143
\itdefault 25:3217, 29:31, 30:106
\item . . . 14:282, 37:446, 37:496, 37:498, 37:529, 37:551, 38:562, 38:574, 38:601, 39:172, 39:250, 41:78, 43:67, 43:69, 43:74, 43:76, 47:4, 47:8, *374*
\itemindent 39:9, 39:42, 39:95, 39:218, 39:239, *909*
\itemitem *1359*
itemize (env.) 39:273
\itemize 39:273
\itemsep 39:1, 39:207, *909*
\iterate 01:65, 01:66, 02:365
\itshape 21:468, 21:831, 25:3215, 25:3216, 29:29, 29:30, 29:637, 29:670, 29:676, 29:707, 32:21, 33:598, 43:67, 43:69, 43:74, 43:76, 45:399, 57:787, *1404*

J

\J 57:257, 57:601
\j 21:268, 21:424, 21:571, 21:803, 21:1178, 21:1393, 21:1479
\jmath 30:321
\jobname *1133*
\Join 29:881
\joinrel 30:482, 30:489, 30:491, 30:493, 30:495, 30:497, 30:499, 30:501, 30:503, 30:507, 30:509
\jot 38:53, 38:204, 38:497, 38:507

K

\k 21:508, 21:613, 21:618, 21:640, 21:645, 21:721, 21:722, 21:781, 21:782, 21:836, 21:838, 21:843, 21:845, 21:1279, 21:1347, 21:1348, 21:1365, 21:1366, 21:1388, 21:1389, 21:1390, 21:1443, 21:1444, 21:1477, 21:1478, 33:145, 33:168, *1377*
\kanjiskip 05:79
\kappa 30:288
\ker 38:27
\kern *1203*
\kern... . . . . *1397*
kernel (para/begin) (tag plug) . . . 55:170
kernel (para/end) (tag plug) 55:190
kernel (para/semantic/begin) (tag plug) 55:126
kernel (para/semantic/end) (tag plug) 55:137
kernel (para/textblock/begin) (tag plug) 55:147
kernel (para/textblock/end) (tag plug) 55:159
kernel (recordtarget) (tag plug) . 55:207
kernel (refstepcounter/target) (plug) 35:136
kernel internal commands:
__kernel_chk_if_free_cs:N 07:3321, *216*
__kernel_cmd_if_xparse:NTF 09:144, 07:1168, 07:1485, 07:1487, 07:2730, 07:2751, 07:2807, *337*
__kernel_cs_parameter_spec:N 09:177, 09:252, 09:402, 05:156, 07:2734, *1432*
__kernel_cs_parm_from_arg_count:nnTF 08:103
\g__kernel_documentproperties_prop 17:26, 17:29, 17:37, 17:48, 17:55, *459*
__kernel_exp_not:w 08:42, 08:44, 08:50, 08:55
\l__kernel_expl_bool 09:413
__kernel_file_name_sanitize:n 52:98
__kernel_msg... . . . . *1421*
__kernel_msg_... . . . . *1423*
__kernel_quark_new_conditional:Nn 11:104
__kernel_quark_new_test:N . . . 07:52
\g__kernel_target_int 17:115, 17:131, 17:132, 17:155, 17:156

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\kerneltmpDoNotUse . . . 09:474, 09:486, 09:492, 09:497, 09:545, 09:552, 09:573, 09:581, 09:588, 09:601, *347*
keys commands:
\keys_define:nn 20:763, 51:215, 51:219, 51:236, 51:271, 57:621
\keys_if_exist:nnTF 51:119, 51:125, 51:149, 51:183, 51:199
\l_keys_key_str . 51:48, 51:49, 51:224
\l_keys_key_tl 51:48
\l_keys_path_str 51:47
\keys_set:nn 51:71, 51:286, 57:643
\keys_set_known:nnN 11:1257
\l_keys_usage_load_prop 51:232
\l_keys_usage_preamble_prop . 51:267
keys internal commands:
\l__keys_class_only_clist 51:53, 51:122, 51:151
__keys_find_key_module:wNN . . 51:46
\l__keys_forced_global_clist 51:49, 51:54, 51:153
\l__keys_local_clist 51:56, 51:166, 51:171, 51:175, 51:185, 51:189
\l__keys_no_value_bool 51:24
__keys_options:n 51:58, 51:58, 51:228
__keys_options_aux:n 51:58, 51:59, 51:60
__keys_options_class:n 51:101, 51:105, 51:105
__keys_options_class:nn 51:105, 51:121, 51:126, 51:133
__keys_options_class:nnn 51:105, 51:111, 51:117
\l__keys_options_clist 51:55, 51:65, 51:70, 51:136, 51:161, 51:185, 51:201, *1153*
__keys_options_expand_module:Nn 51:59, 51:208, 51:208, 51:215, 51:219, 51:286
__keys_options_expand_module:nN 51:208, 51:212
__keys_options_global:n 51:67, 51:96, 51:96
__keys_options_loaded:n 51:73, 51:230, 51:230
__keys_options_loaded:nn 51:230, 51:239, 51:244
\l__keys_options_loading_bool 51:57, 51:69, 51:72, 51:246, *1152*
__keys_options_local: 51:62, 51:164, 51:164
__keys_options_local:n 51:68, 51:181, 51:181
__keys_options_local:nnn 51:181, 51:191, 51:197
__keys_options_package:n 51:102, 51:138, 51:138
__keys_options_package:nn 51:138, 51:154, 51:156, 51:159
__keys_options_package:nnn 51:138, 51:142, 51:147
\c__keys_props_root_str 51:9, 51:10, 51:22
__keys_remove_equals:n . . 51:112, 51:143, 51:178, 51:192, 51:205, 51:205
__keys_remove_equals:w 51:205, 51:206, 51:207
__keys_scope:N 51:22, 51:33, 51:35, 51:44
__keys_scope:n 51:22, 51:25, 51:26, 51:28
__keys_tmp:nn 51:5, 51:11, 51:13
\l__keys_tmpa_tl 51:232, 51:234
\l__keys_unused_clist 51:75, 51:77, 51:177, 51:186, 51:202
keyval commands:
\keyval_map_break:n 11:508
\keyval_map_inline:nnn 11:498
\keyval_parse:NNn 11:306
\keyval_parse:nnn 11:451, 11:562, 11:682, 11:768, 11:871
\KeyValue 11:162, 11:1196, 11:1459, 11:1616, *409*
\kill 41:154, 41:162

L

\L 21:262, 21:445, 21:553, 21:795, 21:1171, 50:1375, 50:1507, 50:1597, 50:1619, 57:720
\l 21:271, 21:447, 21:574, 21:804, 21:1172, 57:720
\label . . . 35:94, 44:190, 44:200, 48:280, 49:40, 49:51, 49:66, 49:74, 49:89, 49:97, 54:904, 54:974, 54:1033, *853*
label 36:259, *854*
\labelenumi *922*
\labelenumiv *922*
\labelformat *841*
\labelformat 35:184, 35:200, 35:212, 35:217, 35:218, 35:223, 35:225
\labelitemi *922*
\labelitemii *922*
\labelitemiii *922*
\labelitemiv *922*
\labelsep 39:9, 39:241, 39:247, 43:67, 43:69, 43:74, 43:76, *922*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\labelwidth 39:9, 39:93, 39:240, 39:242, 39:245, *909*
\Lambda 30:311
\lambda 30:289
\land 30:379, 30:381
\langle 30:605
\language 20:52, 20:122, 37:534, 37:726, 02:35, 54:850, 54:952, 02:78, 02:80, 56:10, *1410*
\lastbox 24:780, 38:193, 38:194, 39:132, 39:138, 39:216, 44:99, 44:132, 54:310, *448*
\LastDeclaredEncoding 24:138, 24:141, 57:531, *1404*
\LastMark 48:402, 48:523, *1060*
\lastnamedcs 06:850
\lastnodetype 17:72, 24:773, 24:774, 24:775, 24:779
\lastpenalty 24:776, 32:113, 32:116
\lastskip 17:71, 17:79, 17:86, 17:94, 17:98, 17:105, 18:45, 18:129, 18:141, 18:164, 18:182, 18:243, 18:244, 18:248, 18:250, 18:251, 18:263, 18:278, 18:295, 18:317, 18:320, 18:350, 18:353, 18:384, 18:387, 18:388, 32:103, 32:106, 39:115, 39:116, 39:181, 39:182, 42:134, 54:568, 02:389, 02:390, 02:392, 02:394, *471*
\latelua 17:65, *1442*
\LaTeX 19:3, 19:15, 06:929, 50:1337, 50:1470, 50:1559, 53:450, 53:456, 02:256
\LaTeXe 19:13
\latexrelease *1181*
\LaTeXReleaseInfo 03:36, 03:37, 03:40, 03:45, 03:50, 37:49, 37:114, *36*
\latexreleaseversion 03:1
\lbrace 21:328, 30:609
\lbrack 02:316
\lccode 14:19, 14:20, 14:21, 14:22, 14:23, 14:24, 21:158, 21:1093, 37:655, 37:672, 37:686, 37:701, 37:744, 57:224, 57:241, 57:249, 57:256, 57:258, 57:259, 57:261, 57:263, 57:264, 57:265, 57:266, 57:585, 57:593, 57:600, 57:602, 57:603, 57:605, 57:607, *1404*
\lceil 30:613
\lcode *1398*
\ldotp 30:512, 30:515, 30:630
\ldots 21:342, 30:516, *1386*
\le 30:427, 30:429
\leaders 30:351, 30:569, 30:570, 30:572, 30:573, 41:453, 42:575, 42:588, 42:600, 42:610, 44:239, 44:262, 02:421
\leadsto 29:884
\leavevmode 18:476, 18:490, 21:93, 21:202, 21:309, 21:310, 21:414, 21:446, 21:450, 21:453, 21:502, 21:787, 21:820, 32:124, 33:62, 33:888, 37:538, 37:559, 37:572, 37:583, 37:613, 37:627, 37:722, 37:732, 37:745, 38:562, 38:574, 38:601, 39:58, 39:103, 40:8, 40:17, 40:24, 40:200, 40:202, 40:218, 40:250, 40:269, 40:371, 40:395, 40:421, 40:500, 40:607, 40:629, 40:672, 40:679, 41:206, 41:213, 42:145, 42:325, 42:384, 44:40, 44:235, 44:247, 44:258, 45:410, 45:469, 45:592, 47:34, 54:136, 54:141, 54:163, 54:168, 54:189, 54:194, 02:380, 02:407, 02:410, 02:421, 02:423, *1388*
\left 30:636, 30:638, 30:640, 30:642, 30:647, 30:648, 30:649, 30:650, 38:167, 38:173, 38:195
\Leftarrow 30:421, 30:503, 30:509
\leftarrow 30:448, 30:450, 30:491, 30:501, 30:507, 30:561
\leftarrowfill 30:545, 30:561
\lefteqn 38:521
\leftharpoondown 30:464, 30:478
\leftharpoonup 30:463
\lefthyphenmin 56:11, 02:246
\leftline 40:725
\leftmargin 39:9, 39:52, 39:53, 39:94, 39:177, 39:179, *907*
\leftmargini 38:554, 39:17, *909*
\leftmarginii 39:17
\leftmarginiii 39:17
\leftmarginiv 39:17
\leftmarginv 39:17
\leftmarginvi 39:17, *909*
\leftmark 49:106, *1366*
\Leftrightarrow 30:420
\leftrightarrow 30:447
\leftskip 24:757, 37:454, 37:460, 37:464, 37:474, 37:478, 37:482, 37:531, 37:553, 39:74, 40:455, 40:476, 44:232, 44:237, 44:255, 44:260, 02:291, 02:402
legacy commands:
\legacy_if:nTF . . . 48:437, 48:459, 48:497, 48:504, 55:174, 55:177, 55:178
\legacyoldstylenums 33:4, 33:581

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\leq 30:425, 30:427
\leqno 38:532
\let *1388*
\LetLtxMacro *103*
\lfloor 30:617
\lg 38:4
\lgroup 30:619
\lhd 29:887
\lhook 30:488, 30:489
\lim 38:6
\liminf 38:8
\limits 30:550, 30:554, 38:162, 38:411
\limsup 38:7
\line . 14:269, 42:169, 42:461, 42:819, 42:836
\linebreak *463*
\linebreak 18:9, 18:27
\linepenalty 02:190
\lineskip 30:469, 38:200, 40:457, 40:477, 41:71, 41:250, 41:273, 42:147, 42:326, 42:385, 53:284, 53:335, 54:908, 54:977, 54:1036, 02:285, 02:310, 02:375, 02:410
\lineskiplimit 30:469, 30:521, 38:202, 38:206, 40:443, 40:458, 40:465, 53:285, 53:336, 54:909, 54:977, 54:1036, 02:271, 02:311, 02:375, 02:412, 02:413, *1410*
\linespread 24:379
\linethickness 42:141, 42:820, 42:837
\linewidth 20:30, 20:100, 20:158, 38:315, 38:343, 38:368, 38:563, 38:575, 38:602, 38:606, 38:625, 39:9, 39:51, 39:52, 39:54, 40:453, 40:474, 41:36, 45:266, 54:125, 54:210, *908*
\LinkTargetOff 17:59
\LinkTargetOn 17:59
list (env.) 39:34
\list 39:34, 39:267, 39:278
\listfiles *1098*
\listfiles 20:769, *1381*
\listparindent 39:9, 39:41, 39:50, *909*
\literal *1380*
\ll 30:445
\llap 39:269, 39:280, 40:729
\lmoustache 30:574
\ln 38:5
\lnot 30:337, 30:338
\LoadClass *1096*
\LoadClass 50:721, 50:751, 50:1002, 50:1137, 50:1232, 50:1240, 50:1241, *1362*
\LoadClassWithOptions *1096*
\LoadClassWithOptions 50:750
\LoadFontDefinitionFile 24:210, 24:515, 24:541, 24:542, 30:21, 30:27, 30:28, 30:29, 30:33
\LoadPackageWithOptions *1161*
\loccount 04:17
\log 38:3
\LogDocumentProperties 17:39, *458*
\loggingall 02:432
\loggingoutput 02:428, 02:445, 02:463, 02:479, 02:493, *1403*
\LogHook 08:2844, *228*
\LogSocket 10:192, 10:215, *360*
\long *1390*
\Longleftarrow 30:503
\longleftarrow 30:500
\Longleftrightarrow 30:509, 30:511
\longleftrightarrow 30:507
\longmapsto 30:505
\Longrightarrow 30:497
\longrightarrow 30:498, 30:505
\loop 04:150, 04:159, 24:771, 41:459, 50:1285, 50:1346, 50:1416, 50:1479, 50:1530, 50:1568, 57:414, 57:425, 57:435, 57:446, 57:476, 57:502, 57:512, 01:65, 02:365, *1384*
\looseness 02:207
\lor 30:380, 30:382
\lower 19:2, 30:469, 40:283, 42:46, 42:56, 42:207, 42:316, 42:317, 42:364, 42:365, 42:420, 42:421, 45:473
\lowercase 14:26, 21:159, 21:1094, 21:1572, 22:186, 24:395, 24:487, 37:659, 37:676, 37:690, 37:705, 37:745, *1396*
\lq 02:314
lrbox (env.) *925*
\lrbox 40:188
\ltfilehookdate 52:542
\ltfilehookversion 52:542
lua commands:
\lua_now:n 53:92, 53:528
\lua_shipout:n 53:528
\luabytecode 04:202
\luachunk 04:210
\luadef 04:182, 04:186, *40*
\luafunction . . . 04:178, 04:182, 04:186, *40*
luamml commands:
\luamml_save:... . . . . *1321*
luatexbase 04:287
\luatexluafunction 01:21, 01:26
\luatexversion 04:5, 21:1018, 01:14, 05:78, 05:124, 02:217, 02:218

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

M

\M 02:312
\Macro *1359*
\mag 02:251
\magstep 02:303
\magstephalf 02:303
\makeat... . . . . *1378*
\makeatletter 09:409, 20:32, 20:102, 20:160, 24:493, 37:29, 37:94, 37:156, 44:151, 50:715, 50:966, 50:1098, 06:925, *1400*
\makeatother 09:409, 50:715, 06:925, 57:806, *1375*
\makebox *925*
\makebox 38:315, 38:343, 38:368, 40:3, *1322*
\makecol *1250*
\makeglossary *1050*
\makeglossary 20:205, 46:20, *1383*
\makeindex *1050*
\makeindex 20:204, 46:3, *1383*
\makelabel 39:45, 39:97, 39:236, 39:249, 39:269, 39:280, *1360*
\MakeLinkTarget . . 17:59, 43:67, 43:69, *462*
\MakeLowercase 57:620, *1390*
\MakeRobust . . . 28:960, 28:1155, 06:375, 42:816, 42:817, 42:818, 42:819, 42:820, 42:821, 42:822, 42:823, 42:824, 42:825, 42:826, 42:827, 06:970, 06:971, 06:972, 06:973, 06:974, 06:975, 06:976, 06:977, 06:978, 06:979, 06:980, 06:981, 06:982, 06:983, 06:984, 06:985, *1421*
\maketitle *1012*
\MakeTitlecase 57:620, *1426*
\MakeUppercase 35:203, 35:205, 35:214, 35:220, 57:620, *1390*
\mapsto 30:455
\mapstochar 30:454, 30:455, 30:505
\marginpar 45:308, *443*
marginpar/begin (tag socket) 55:235
marginpar/end (tag socket) 55:235
\marginparpush 54:61, 54:2411
\marginparsep 54:60, 54:2422, 54:2424
\marginparwidth 45:341, 45:359, 54:59, 54:2424
\mark 49:46, 49:56, 49:69, 49:77, 49:92, 49:100, 49:121, *1366*
mark commands:
\mark_clear_structure:n 48:224, 48:224, *1067*
\mark_copy_structure:nn 48:138, 48:213, 48:213, 48:427, 48:428, 48:429, 48:430, 48:461, 48:465, 48:466, *1066*
\mark_debug_off: 48:340, 48:346, 48:357, *1063*
\mark_debug_on: 48:340, 48:341, 48:356, *1063*
\mark_get_marks_for_reinsertion:nNN 48:162, 48:162, *1067*
\mark_if_eq:nnnn 48:299
\mark_if_eq:nnnnnn 48:306
\mark_if_eq:nnnnnnTF . . . 48:299, *1060*
\mark_if_eq:nnnnTF 48:299, 48:409, 48:412, 48:415, *1060*
\mark_insert:nn 48:189, 48:200, 48:246, 48:246, 48:401, 49:24, 49:25, 49:26, 49:29, 49:30, 49:43, 49:44, 49:45, 49:54, 49:55, *1060*
\mark_new_class:n 48:7, 48:7, 48:16, 48:399, *1059*
\mark_set_structure_to_err:n 48:235, 48:235, 48:462, *1066*
\mark_update_structure_from_material:nn 48:132, 48:132, 48:420, 48:424, 48:452, 48:456, *1070*
\mark_use_first:nn 48:289, 48:289, 48:403, 49:112, *1060*
\mark_use_last:nn 48:289, 48:290, 48:405, 49:111, *1060*
\mark_use_top:nn 48:289, 48:291, 48:407, *1060*
mark internal commands:
__mark__update_dblcol_structures: *1066*
__mark_class_status:nnn 48:358, 48:359, 48:391
__mark_debug:n 48:20, 48:92, 48:179, 48:185, 48:191, 48:258, 48:340, 48:340, 48:353, 48:396, 48:433, 48:493, *1082*
__mark_debug_gset: 48:340, 48:344, 48:349, 48:351
__mark_drop_id:n 48:189, 48:202, 48:295, 48:297, 48:298
__mark_error:n 48:235
__mark_error:nn 48:238, 48:239, 48:240, 48:243, 48:294
__mark_extract_and_handle_marks:nn 48:67, 48:67, 48:133, 48:165, *1075*
__mark_get_from_splitmarks: 48:166, 48:171, 48:171
__mark_init_region:nn 48:24, 48:25, 48:26, 48:27, 48:28, 48:29, 48:30, 48:31, 48:32, 48:33, 48:34, 48:35, 48:36, 48:37, 48:38, 48:39, 48:40, 48:41, 48:42, 48:43,

48:44, 48:45, 48:46, 48:47, 48:48, 48:49, 48:50, 48:51, 48:52, 48:54, 48:54
__mark_new_class:nn 48:7, 48:14, 48:17
__mark_prepare_and_extract:nn 48:72, 48:75, 48:75, 48:96, *1071*
__mark_region_status:nnn 48:361, 48:362, 48:363, 48:364, 48:365, 48:366, 48:369, 48:372, 48:372
__mark_status:nn 48:388, 48:388, 48:396, 48:435, 48:495
__mark_update_dblcol_structures: 48:449, 48:449, 48:516, *1067*
__mark_update_singlecol_structures: 48:417, 48:417, 48:514, *1067*
__mark_update_structure_from_splitmarks:n 48:134, 48:137, 48:137, *1074*
__mark_use_check:nnn 48:289, 48:290, 48:291, 48:292
__mark_value:nn 48:62, 48:238, 48:239, 48:240, 48:255, 48:277, 48:277, 48:374
__mark_vbox_set_split_to_maxdimen:NN 48:87, 48:101, 48:102, 48:114, *1071*
\markboth . . 49:21, 49:22, 49:36, 49:38, 49:62, 49:64, 49:83, 49:85, 49:87, *1058*
\markright 49:22, 49:49, 49:72, 49:86, 49:95, *1064*
\marks 04:37, 57:10, 57:12
math (env.) 38:389
\math 38:389
math/luamml/annotate/false (tag socket) 55:283
math/luamml/array/finalize (tag socket) 55:285
math/luamml/array/finalizecol (tag socket) 55:287
math/luamml/array/initcol (tag socket) 55:286
math/luamml/array/save (tag socket) 55:284
math/luamml/artifact (tag socket) . 55:299
math/luamml/finph@nt (tag socket) 55:297, 55:298
math/luamml/hbox (tag socket) 55:296
math/luamml/mtable/aligncol (tag socket) 55:290
math/luamml/mtable/finalize (tag socket) 55:289
math/luamml/mtable/finalizecol (tag socket) 55:288
math/luamml/mtable/innertable/finalize (tag socket) 55:293
math/luamml/mtable/innertable/save (tag socket) 55:291
math/luamml/mtable/smallmatrix/save (tag socket) 55:292
math/luamml/mtable/tag/save (tag socket) 55:294
math/luamml/mtable/tag/set (tag socket) 55:295
math/luamml/save/nn (tag socket) . . 55:281
math/luamml/save/nNn (tag socket) . 55:282
\mathaccemt *1409*
\mathaccent . 28:877, 28:925, 28:959, 28:969
\mathalpha 28:1047, 28:1226, 30:180, 30:181, 30:182, 30:183, 30:184, 30:185, 30:186, 30:187, 30:188, 30:189, 30:190, 30:191, 30:192, 30:193, 30:194, 30:195, 30:196, 30:197, 30:198, 30:199, 30:200, 30:201, 30:202, 30:203, 30:204, 30:205, 30:206, 30:207, 30:208, 30:209, 30:210, 30:211, 30:212, 30:213, 30:214, 30:215, 30:216, 30:217, 30:218, 30:219, 30:220, 30:221, 30:222, 30:223, 30:224, 30:225, 30:226, 30:227, 30:228, 30:229, 30:230, 30:231, 30:232, 30:233, 30:234, 30:235, 30:236, 30:237, 30:238, 30:239, 30:240, 30:241, 30:308, 30:309, 30:310, 30:311, 30:312, 30:313, 30:314, 30:315, 30:316, 30:317, 30:318, 30:527, 30:528, 30:529, 30:530, 30:531, 30:532, 30:533, 30:534, 30:536, 30:539
\mathbf 29:15, 29:334, 29:387, 29:432, 29:460, 29:554, 30:162
\mathbin 28:1231, 30:243, 30:244, 30:246, 30:369, 30:370, 30:371, 30:372, 30:375, 30:376, 30:377, 30:378, 30:381, 30:382, 30:383, 30:384, 30:385, 30:386, 30:387, 30:388, 30:389, 30:390, 30:391, 30:392, 30:393, 30:394, 30:395, 30:396, 30:397, 30:398, 30:399, 30:400, 30:401, 30:402, 30:403, 30:404, 30:405, 30:406, 30:407, 30:408, 38:37
\mathcal 30:161
\mathchar 28:988, 28:1032, 30:346, 30:347, 30:628, 02:408, *1409*
\mathchardef 12:3, 12:4, 12:5, 12:6, 04:226, 21:88, 28:1023, 02:21, 02:22, 02:23, 02:24, 02:83, 02:84, *1382*
\mathcharzero 04:226
\mathchoice 38:61

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\mathclose 28:1234, 30:242, 30:251, 30:253, 30:256, 30:261, 30:267, 30:269, 30:271, 30:577, 30:604, 30:608, 30:612, 30:616, 30:622, 38:43, 38:46, 38:49, 38:52
\mathcode . 28:1020, 30:263, 30:264, 30:265
\MathCollectFalse *1310*
\MathCollectFalse 55:52
\MathCollectTrue *1310*
\MathCollectTrue 55:52
\mathdefaultsmode 38:529, 38:530
\mathdollar 21:327, 30:625, *1382*
\mathellipsis 21:341, 30:630, *1382*
\mathfontset *1357*
\mathgroup 24:15, 26:304, 26:310, 26:316, 26:317, 26:328, 30:654, 33:8, 33:14, 33:578, 33:1100, 02:75, *1357*
\mathhexbox 29:763, 02:408, *1389*
\mathindent 38:552, 38:564, 38:576, 38:604, 38:615, *1368*
\mathinner . 30:515, 30:519, 30:524, 30:630
\mathit 25:3216, 29:30, 30:164, 30:167, 30:628
\MathMLarg *1310*
\MathMLarg 55:300
\MathMLintent *1310*
\MathMLintent 55:300
\mathnormal 30:160
\mathop 28:1230, 30:352, 30:353, 30:354, 30:355, 30:356, 30:357, 30:358, 30:360, 30:361, 30:362, 30:363, 30:364, 30:365, 30:367, 30:368, 30:548, 30:551, 38:3, 38:4, 38:5, 38:6, 38:7, 38:8, 38:9, 38:10, 38:11, 38:12, 38:13, 38:14, 38:15, 38:16, 38:17, 38:18, 38:19, 38:20, 38:21, 38:22, 38:23, 38:24, 38:25, 38:26, 38:27, 38:28, 38:29, 38:30, 38:31, 38:32, 38:33, 38:34, 38:162, 38:411
\mathopen 28:1233, 30:252, 30:255, 30:260, 30:266, 30:268, 30:270, 30:575, 30:606, 30:610, 30:614, 30:618, 30:620, 38:41, 38:44, 38:47, 38:50
\mathord 28:1047, 28:1229, 30:247, 30:254, 30:257, 30:262, 30:274, 30:275, 30:276, 30:278, 30:279, 30:280, 30:281, 30:282, 30:283, 30:284, 30:285, 30:286, 30:287, 30:288, 30:289, 30:290, 30:291, 30:292, 30:293, 30:294, 30:295, 30:296, 30:297, 30:298, 30:299, 30:300, 30:301, 30:302, 30:303, 30:304, 30:305, 30:306, 30:307, 30:319, 30:320, 30:321, 30:322, 30:323, 30:324, 30:325, 30:326, 30:327, 30:328, 30:329, 30:330, 30:331, 30:332, 30:333, 30:334, 30:335, 30:336, 30:338, 30:339, 30:340, 30:341, 30:342, 30:343, 30:344, 30:345, 30:535, 30:537, 30:538, 30:560, 30:561, 30:564, 30:565, 30:566, 30:567, 30:579, 30:581, 30:583, 30:586, 30:588, 30:602, 30:624, 30:625, 30:626, 30:627
\mathpalette 30:468, 30:472, 30:475, 38:60, 38:69, 38:99, 38:129
\mathparagraph 21:330, 22:335, 22:347, 30:625, *1399*
\mathpunct 28:1235, 30:245, 30:249, 30:512, 30:513, 30:514
\mathrel 28:1232, 30:248, 30:250, 30:258, 30:259, 30:272, 30:273, 30:349, 30:409, 30:410, 30:411, 30:412, 30:413, 30:414, 30:415, 30:416, 30:417, 30:418, 30:419, 30:420, 30:421, 30:422, 30:425, 30:426, 30:429, 30:430, 30:431, 30:432, 30:433, 30:434, 30:435, 30:436, 30:437, 30:438, 30:439, 30:440, 30:441, 30:443, 30:444, 30:445, 30:446, 30:447, 30:448, 30:449, 30:452, 30:453, 30:454, 30:456, 30:457, 30:458, 30:459, 30:460, 30:461, 30:462, 30:463, 30:464, 30:465, 30:466, 30:468, 30:472, 30:475, 30:482, 30:484, 30:487, 30:488, 30:490, 30:493, 30:495, 30:590, 30:592, 30:594, 30:596, 30:598, 30:600, 38:42, 38:45, 38:48, 38:51, 38:162, 38:411
\mathring 30:539, *1401*
\mathrm 29:6, 29:485, 29:528, 29:560, 30:159, *1358*
\mathsection 21:331, 22:334, 22:346, 30:625, *1399*
\mathsf 29:9, 29:490, 29:533, 29:563, 30:163, 30:166
\mathsterling 21:339, 30:625, *1382*
\mathstrut . . . 38:84, 38:93, 38:171, 38:172
\mathsurround 02:275, 02:396, *1399*
\mathsymbol 28:1025
\mathtt 29:12, 29:495, 29:538, 29:566, 30:165
\mathunderscore 30:625, *1399*
\mathversion . 24:399, 29:725, 29:727, *1376*
\mathversion. *1357*
\matrix 38:169, 38:173, 38:180
\max 38:22
\maxdeadcycles 54:3, 02:248

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\maxdepth 18:324, 18:357, 20:60, 20:129, 20:184, 54:68, 54:148, 54:149, 54:175, 54:176, 54:504, 54:593, 54:594, 54:767, 54:1075, 54:1342, 57:152, 02:268, *1437*
\maxdimen 24:743, 24:753, 24:789, 24:804, 26:401, 26:454, 30:469, 42:485, 42:513, 42:543, 42:621, 42:638, 50:1640, 50:1689, 50:1699, 53:398, 53:400, 53:462, 54:296, 54:2430, 54:2450, 54:2455, 54:2841, 54:2909, 54:2910, 54:2912, 57:156, 02:179, 02:269, 02:270, 02:375, 02:413, 02:429, 02:443, 02:444, 02:462, 02:478, 02:493, *1208*
\mbox *925*
\mbox 19:13, 21:313, 21:430, 21:591, 21:1062, 21:1193, 29:759, 30:517, 40:11, 40:20, 40:24, 42:63, 45:441, 45:447, 45:500, 45:506, 02:408, *1322*
mc (tag socket) 55:54
\mddefault 29:19, 29:363, 29:369, 29:370, 29:371, 29:410, 29:411, 29:412, 29:421, 29:448, 29:464, 29:481, 29:522, 29:558, 30:104, 30:116, 30:118, 30:132, *736*
mdseries (hook) *244*
\mdseries 29:17, 29:18, 29:314, 29:356, 29:407, 29:408, 29:442, 29:443, 29:462, 29:463, 29:556, 29:557, 29:762, 32:20, *753*
mdseries 29:499
mdseries/defaults (hook) *244*
mdseries/defaults 29:499
\meaning 01:203, 01:212, 28:699, 28:712, 28:813, 28:878, 28:925, 28:989, 28:1083, 28:1179, 28:1283, 06:332, 06:394, 06:432, 06:460, 01:307, 06:543, 06:896
\medbreak 06:977, 06:998, 02:390
\mediumseries *745*
\medmuskip 30:656, 38:36, 38:38, 38:224, 38:227, 38:241
\medskip 18:469, 02:393
\medskipamount 18:470, 18:472, 02:392
\medspace 38:214
\MessageBreak 03:101, 14:3, 14:6, 14:13, 14:33, 14:46, 14:60, 14:73, 14:220, 14:222, 14:228, 14:235, 17:19, 21:179, 21:1010, 21:1011, 21:1577, 21:1579, 21:1581, 21:1584, 22:128, 22:130, 22:165, 22:167, 24:35, 24:36, 24:633, 24:667, 25:2844, 26:21, 26:22, 26:68, 26:89, 26:328, 26:495, 26:515, 26:547, 26:563, 26:578, 26:591, 27:31, 27:33, 05:83, 05:86, 05:87, 05:88, 05:89, 05:90, 05:91, 28:281, 05:104, 28:329, 05:105, 05:106, 05:107, 05:108, 28:654, 05:139, 28:663, 05:141, 05:142, 28:801, 29:62, 29:95, 32:145, 33:23, 33:43, 33:45, 33:64, 33:812, 33:814, 33:815, 33:816, 33:818, 33:820, 33:821, 33:822, 33:823, 33:824, 33:874, 33:876, 33:883, 33:890, 33:1105, 06:245, 37:56, 37:121, 37:160, 06:383, 06:422, 06:450, 50:388, 50:402, 50:777, 50:788, 50:790, 50:792, 50:803, 50:810, 50:812, 50:997, 50:998, 50:999, 50:1000, 50:1010, 50:1011, 50:1013, 50:1014, 50:1015, 50:1017, 50:1019, 50:1105, 50:1106, 50:1108, 50:1109, 50:1110, 50:1112, 50:1114, 50:1132, 50:1133, 50:1134, 50:1135, 50:1217, 50:1234, 50:1235, 50:1307, 50:1324, 50:1363, 50:1438, 50:1457, 50:1496, 50:1551, 50:1585, 50:1703, 50:1705, 50:1788, 50:1791, 50:1804, 50:1806, 52:488, 53:160, 53:236, 53:423, 53:551, 53:552, 53:553, 53:554, 54:829, 54:2571, 54:2618, 57:302, 57:303, 57:304, 57:306, *1379*
\mho 29:880
\mid 30:413
\min 38:23
minipage (env.) *926*
\minipage 40:487
\mit 29:933
\mkern 30:346, 30:349, 30:351, 30:473, 30:482, 30:524, 30:525, 30:526, 30:556, 30:557, 30:558, 30:559, 30:560, 30:561, 30:562, 30:563, 38:36, 38:37, 38:40, 38:73, 38:74, 44:240, 44:263
mlist commands:
  mlist_to_hlist 04:1023
mode commands:
  \mode_if_horizontal:TF . 16:97, 16:102
  \mode_if_inner:TF 16:98
  \mode_if_math:TF 28:360
  \mode_if_vertical:TF 16:89, 16:116, 16:128
\models 30:495, *1403*
module commands:
  module_error 04:345
  module_info 04:345
  module_warning 04:345

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\module_error *43*
\module_info *43*
\module_warning *43*
modules 04:298
\month 03:17, 01:169, 50:1341, 50:1474, 50:1563, 02:260
\moveright 54:912, 54:980, 54:1039
\mp 30:400
\mscount 41:456
msg commands:
\msg_... . . . . *1421*
\msg_error:nn 10:49, 11:358, 16:124, 16:134, 36:14, 08:436, 08:453
\msg_error:nnn 08:2195, 10:30, 10:96, 10:115, 10:136, 10:151, 11:138, 11:145, 11:150, 11:278, 11:338, 11:382, 11:404, 11:461, 11:473, 11:486, 11:507, 11:577, 11:590, 11:608, 11:635, 11:647, 11:692, 11:782, 11:804, 11:872, 11:907, 11:936, 11:1044, 11:1205, 11:1244, 48:11, 48:273, 51:274, 52:75, 07:349, 07:755, 07:3161, 07:3167, 07:3178, 07:3197, 07:3200, 07:3208, 08:70, 08:88, 08:173, 08:202, 08:205, 08:461, 08:530, 08:548, 08:618, 08:638
\msg_error:nnnn 09:59, 09:117, 09:431, 10:83, 10:112, 10:133, 11:132, 11:158, 11:292, 11:710, 11:738, 11:748, 11:995, 11:1007, 11:1118, 11:1222, 11:1228, 11:1314, 16:20, 16:33, 16:53, 16:65, 16:110, 36:35, 48:244, 51:38, 51:79, 51:248, 07:463, 07:471, 07:495, 07:499, 07:531, 07:562, 07:577, 07:667, 07:684, 07:712, 07:724, 07:743, 07:782, 07:1155, 07:2065, 07:2686, 07:2690, 07:3109, 07:3123, 07:3228, 07:3242, 08:106, 08:512, 08:662
\msg_error:nnnnn . . 11:124, 11:571, 11:1585, 07:674, 07:699, 07:2225, 07:2232, 07:2525, 08:440, 08:471, 08:486
\msg_error:nnnnnn 08:1782, 08:1343, 08:1355, 08:1380
\msg_expandable_error:nn 08:335, 08:336
\msg_expandable_error:nnn 08:2590, 05:199, 36:77, 36:113, 36:136, 07:419, 07:3274, 08:364, 08:1283
\msg_expandable_error:nnnn 52:464, 07:2423, 07:2446
\msg_info:nnnn 11:192, 11:286, 11:552, 11:950, 11:973, 11:991, 07:73, 07:84, 07:234, 07:235, 07:251, 07:255
\msg_line_context: 08:2672, 08:2677, 08:2681, 08:2728, 11:1075, 11:1233, 11:1365, 11:1371, 11:1379, 11:1386, 11:1393, 11:1400, 11:1407, 11:1414, 11:1422, 11:1528, 11:1530, 11:1534, 11:1537, 48:20, 48:94, 48:180, 48:259, 07:3066, 07:3071, 07:3076, 07:3081, 08:585
\msg_log:nnn 17:53
\msg_module_name:n 51:94, 51:253, 51:259, 51:263
\g_msg_module_name_prop 08:2614
\g_msg_module_type_prop . 08:2612, 08:2613, 10:191, 11:1548, 48:313, 07:58
\msg_new:nnn 08:2670, 08:2675, 08:2679, 08:2724, 08:2730, 08:2735, 08:2740, 08:2745, 08:2749, 08:2756, 08:2921, 11:1527, 11:1529, 11:1531, 11:1536, 11:1538, 11:1543, 17:39, 05:201, 51:251, 52:470, 07:1156, 07:3020, 07:3063, 07:3068, 07:3073, 07:3078, 07:3083, 07:3095
\msg_new:nnnn 08:2615, 08:2625, 08:2632, 08:2639, 08:2648, 08:2662, 08:2684, 08:2700, 08:2713, 09:609, 09:619, 10:168, 10:173, 10:177, 10:181, 10:187, 11:1316, 11:1324, 11:1329, 11:1335, 11:1344, 11:1350, 11:1357, 11:1364, 11:1370, 11:1376, 11:1385, 11:1392, 11:1399, 11:1406, 11:1413, 11:1421, 11:1428, 11:1437, 11:1443, 11:1449, 11:1455, 11:1462, 11:1468, 11:1474, 11:1481, 11:1487, 11:1493, 11:1499, 11:1515, 11:1521, 16:146, 16:157, 36:268, 36:274, 36:280, 48:314, 48:322, 48:329, 51:90, 51:256, 51:279, 07:2844, 07:2851, 07:2858, 07:2865, 07:2872, 07:2878, 07:2884, 07:2891, 07:2898, 07:2905, 07:2912, 07:2919, 07:2927, 07:2934, 07:2941, 07:2948, 07:2956, 07:2964, 07:2973, 07:2980, 07:2987, 07:2994, 07:3001, 07:3007, 07:3013, 07:3022, 07:3028, 07:3035, 07:3042, 07:3053
\msg_redirect_module:nnn 07:57
\msg_show:nnn 17:46
\msg_show:nnnnn 11:1306
\msg_show:nnnnnn 11:1295
\msg_show_item:nn 17:48, 17:55
\msg_show_item_unbraced:nn 11:1299, 11:1311

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\msg_warning:nnn . 07:81, 08:289, 08:307
\msg_warning:nnnn 08:2920, 51:247, 07:736, 07:1630, 07:1751, 08:1000, 08:1013, 08:1035, 08:1049
\msg_warning:nnnnn . . . 11:1593, 08:790
\msg_warning:nnnnnn 08:1790
msg internal commands:
\c__msg_coding_error_text_tl 48:317, 48:325, 48:332
\c__msg_return_text_tl 48:320, 48:337
\mskip 38:36, 38:38, 38:219, 38:238, 38:241, 38:243, 38:244
\mu 30:290
\mubyte 57:401
\multicolumn 41:310, *1325*
\multiput 42:89, 42:821, 42:838
\multispan 41:310, 41:456, *1392*
\muskip . 04:34, 30:556, 30:557, 02:29, 02:51
muskip commands:
\muskip_new:N 11:95
\l_tmpa_muskip *373*
\muskipdef 04:227, 02:51
\muskipzero 04:227
myhook (hook) *221*

N

\n 04:332, 04:334, 04:341, 04:343, 04:470, 04:696, 04:721, 04:745, 04:786, 04:808, 04:843, 04:851, 04:852, 04:858, 04:883, 04:900, 04:939, 04:946, 04:947, 04:954, 04:966, 57:121, 57:126
\nabla 30:330
\NAME *81*
\narrower 02:401
\natural 30:340
\ncallback 04:791
\ndefault 04:796, 04:800
\ne 30:424, *779*
\nearrow 30:416
\NeedsDocumentMetadata *1097*
\NeedsDocumentMetadata 50:806
\NeedsFormat *1362*
\NeedsTeXFormat 26:12, 33:781, 50:782, 50:1850, *1366*
\neg 30:336, 30:337
\negmedspace 38:214, *1418*
\negthickspace 38:214, *1418*
\negthinspace 18:566, 38:214, *1415*
\neq 30:423, *779*
new commands:
new_attribute 04:411
new_bytecode 04:445
new_chunkname 04:458
new_luacommand 04:491
new_luafunction 04:474
new_whatsit 04:433
\New... . . . . *117*
\new_attribute *41*
\new_bytecode *41*
\new_chunkname *41*
\new_luacommand *42*
\new_luafunction *41*
\new_whatsit *41*
\newattribute *40*
\newattribute 04:74, 04:237
\newbox . . . 12:13, 37:585, 38:66, 39:27, 40:159, 41:16, 41:17, 41:18, 41:420, 42:6, 42:680, 42:685, 02:47, 54:62, 54:98, 54:99, 54:100, 02:184, 02:398
\newcatcodetable *40*
\newcatcodetable 04:84, 04:93, 04:94, 04:120, 04:121, 04:241
\newcommand *81*
\newcommand 21:4, 22:14, 25:2918, 25:2923, 25:2928, 29:37, 29:73, 30:63, 30:64, 30:65, 30:66, 30:68, 30:69, 30:71, 30:72, 30:104, 30:105, 30:106, 30:107, 30:108, 30:109, 30:131, 30:132, 30:133, 06:95, 36:263, 37:398, 37:399, 37:400, 37:401, 40:351, 40:353, 40:645, 40:646, 40:647, 40:649, 40:650, 40:651, 42:25, 42:692, 52:573, 52:574, 52:575, 52:576, 52:578, 54:2992, 54:2995, 54:2998, 54:2999, 54:3002, 54:3003, 57:610, *116*
\newcommand(*) *128*
\NewCommandCopy 06:565, 06:567, 07:1132, 07:1184, 07:1202, 07:1215, 07:1230, 07:1348, *100*
\newcount 12:7, 12:8, 18:125, 20:7, 22:58, 22:79, 22:98, 24:422, 26:26, 28:28, 28:143, 28:501, 38:55, 38:415, 38:416, 39:23, 39:24, 39:25, 39:26, 39:56, 39:257, 39:272, 40:536, 41:11, 41:12, 41:13, 41:14, 41:15, 41:412, 41:413, 41:414, 42:674, 42:675, 42:676, 42:677, 42:686, 44:36, 44:140, 44:141, 45:3, 45:267, 45:268, 45:269, 45:270, 50:1637, 02:47, 54:81, 54:83, 54:85, 54:87, 54:89, 54:97, 54:2597, 54:2990, 54:2993, 54:2996, 54:3000, 57:3, 57:4, 57:5, 57:91, 57:364, 02:231, 02:302, *559*
\newcounter *556*
\newcounter 22:10
\newcounteralias *556*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\newcounteralias 22:14, 43:27, *1440*
\newdimen 12:10, 12:11, 12:12, 18:124, 26:415, 26:416, 38:53, 39:9, 39:10, 39:11, 39:12, 39:13, 39:14, 39:15, 39:16, 39:17, 39:18, 39:19, 39:20, 39:21, 39:22, 40:215, 40:216, 41:3, 41:5, 41:6, 41:7, 41:8, 41:166, 41:415, 41:416, 41:417, 41:418, 42:3, 42:4, 42:5, 42:7, 42:442, 42:443, 42:444, 42:445, 42:446, 42:447, 42:678, 42:679, 42:681, 42:682, 42:683, 42:684, 45:513, 02:47, 54:47, 54:48, 54:49, 54:51, 54:52, 54:53, 54:54, 54:55, 54:56, 54:57, 54:58, 54:59, 54:60, 54:61, 54:67, 54:69, 54:70, 54:71, 54:72, 54:84, 54:86, 54:88, 54:90, 54:91, 54:92, 54:93, 54:94, 54:95, 54:96, 54:507, 54:2598, 54:2599, 54:2604, 02:179, 02:181, 02:182, 02:301
\NewDocumentCommand . 08:2761, 08:2763, 08:2765, 08:2769, 08:2771, 08:2773, 08:2784, 08:2786, 08:2796, 08:2798, 08:2807, 08:2809, 08:2816, 08:2818, 08:2820, 08:2822, 08:2824, 08:2848, 08:2850, 08:2853, 11:1569, 17:27, 17:44, 17:51, 17:116, 17:142, 17:165, 17:174, 17:175, 17:176, 20:769, 22:207, 22:222, 22:260, 22:272, 35:66, 35:68, 35:206, 36:80, 48:394, 51:214, 51:216, 51:227, 51:285, 57:682, 57:689, 57:696, 07:3103, *128*
\NewDocumentEnvironment 07:2887, 07:2894, 07:3139, *117*
\newenvironment *82*
\newenvironment 06:187, 50:1339, 50:1472, 50:1561, *240*
\NewEnvironmentCopy . 06:606, 06:608, *1428*
\NewExpandableDocumentCommand 36:140, 48:402, 48:404, 48:406, 48:408, 48:411, 48:414, 57:650, 07:3222, *125*
\newfam 04:38, 24:17, 02:47, *1389*
\newfont 29:729
\newgroup 28:48
\newhelp 02:176
\NewHook 08:2761, 08:2936, 20:73, 20:74, 20:75, 20:367, 20:370, 20:419, 26:151, 29:499, 29:500, 29:501, 29:502, 29:503, 29:504, 29:505, 29:506, 29:507, 37:36, 37:37, 37:38, 37:39, 37:40, 37:101, 37:102, 37:103, 37:104, 37:105, 50:1083, 50:1084, 52:177, 54:730, *239*
\NewHookPair *329*
\NewHookWithArguments 08:2767, 35:115, 41:465, 41:466, 41:468, *330*
\newif 03:73, 12:9, 20:5, 20:6, 24:267, 25:2789, 25:2801, 28:16, 29:609, 32:83, 33:832, 33:1185, 33:1186, 06:209, 35:3, 38:75, 38:76, 38:203, 38:417, 39:28, 39:29, 39:30, 39:31, 39:32, 39:33, 39:140, 40:486, 40:592, 41:19, 41:328, 42:168, 42:438, 42:439, 42:440, 42:441, 42:470, 42:471, 44:38, 44:124, 50:2, 06:964, 54:73, 54:74, 54:75, 54:76, 54:77, 54:78, 54:79, 54:80, *1357*
\newinsert 40:537, 45:390, 54:19, 54:2429, 02:116, 02:142, *1408*
\newlabel 35:84, 35:105, 35:120, 35:129, *1391*
\newlanguage 02:47, 57:286
\newlength *569*
\newlength 23:3
\newline 18:93, 18:100, 18:106
\newlinechar 06:20, 02:256, 01:56
\newluabytecode *41*
\newluabytecode 04:197, 04:251
\newluachunkname *41*
\newluachunkname 04:205, 04:253
\newluacmd *40*
\newluacmd 04:181, *41*
\newluafunction *40*
\newluafunction 04:4, 04:173, 04:235, 04:247, *40*
\NewMarkClass 48:399, 48:520, 57:25, 57:26, 57:27, *1059*
\NewMarkRegion *1069*
\newmarks 48:22, 57:6, *1332*
\newmathalphabet 27:13, 27:109
\NewMirroredHookPair 08:2761, 08:2938, 54:729, 54:731, *218*
\NewMirroredHookPairWithArguments 08:2767, 41:467, 41:469, *218*
\NewModuleRelease 09:4, 03:152, 10:4, 11:7, 16:4, 17:3, 33:2, 36:4, 48:4, 07:9, 55:462, 08:4, *1421*
\newmuskip 02:47
\newpage 54:111, 54:118, 54:129, *1229*
\NewProperty 36:38, 36:290, *853*
\newprotectedluacmd *41*
\newprotectedluacmd 04:181, 21:1034, 53:89
\newread 02:47, 02:177, *1360*
\NewReversedHook 08:2761, 08:2937, 20:368, 20:369, 20:420, 20:421, 50:1085, 50:1086, 52:178, *218*
\NewReversedHookWithArguments 08:2767, *218*
\newrobustcmd *98*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\newsavebox *925*
\newsavebox 40:159
\newskip 12:14, 12:15, 12:17, 18:472, 18:473, 18:474, 18:539, 18:552, 23:3, 37:495, 38:418, 38:553, 39:2, 39:3, 39:4, 39:5, 39:6, 39:7, 39:8, 02:47, 54:3004, 54:3005, 54:3006, 54:3010, 54:3011, 54:3014, 54:3015, 54:3016, 54:3020, 54:3021, 54:3022, 02:180, 02:183, 02:299, 02:300
\NewSocket 10:192, 10:213, 35:134, 35:135, 54:663, 54:722, 54:723, *359*
\NewSocketPlug 10:183, 10:192, 10:217, 35:136, 54:664, 54:674, 54:681, 54:688, 54:697, 54:702, 54:707, 54:724, 54:727, 55:21, *355*
\NewStructureName *1310*
\NewStructureName 55:304
\NewTaggingSocket *1309*
\NewTaggingSocket 55:7, 55:54, 55:62, 55:72, 55:73, 55:84, 55:92, 55:101, 55:103, 55:107, 55:117, 55:118, 55:124, 55:125, 55:145, 55:146, 55:206, 55:214, 55:215, 55:216, 55:217, 55:218, 55:219, 55:220, 55:221, 55:222, 55:223, 55:224, 55:225, 55:226, 55:227, 55:228, 55:229, 55:230, 55:231, 55:232, 55:233, 55:234, 55:235, 55:236, 55:237, 55:238, 55:239, 55:240, 55:241, 55:242, 55:243, 55:244, 55:245, 55:246, 55:247, 55:248, 55:249, 55:250, 55:251, 55:252, 55:253, 55:254, 55:255, 55:256, 55:257, 55:258, 55:259, 55:260, 55:261, 55:262, 55:263, 55:264, 55:265, 55:266, 55:267, 55:268, 55:269, 55:270, 55:271, 55:272, 55:273, 55:274, 55:275, 55:276, 55:277, 55:278, 55:279, 55:280, 55:281, 55:282, 55:283, 55:284, 55:285, 55:286, 55:287, 55:288, 55:289, 55:290, 55:291, 55:292, 55:293, 55:294, 55:295, 55:296, 55:297, 55:298, 55:299, *1309*
\NewTaggingSocketPlug *1309*
\NewTaggingSocketPlug 55:20, 55:55, 55:63, 55:74, 55:78, 55:85, 55:93, 55:102, 55:104, 55:108, 55:119, 55:120, 55:121, 55:122, 55:127, 55:137, 55:147, 55:159, 55:170, 55:190, 55:208, *1309*
\NewTemplateType 11:1440, 11:1518, 11:1549, *370*
\newtheorem 43:1, *1392*
\newtie . 21:875, 33:152, 33:153, 33:172, 33:669, 33:976, 33:977, 33:1284, *803*
\newtoks 03:36, 12:16, 16:77, 24:420, 24:421, 25:2887, 26:248, 02:47, 02:176, *1408*
\newwhatsit *41*
\newwhatsit 04:189, 04:249
\newwrite 20:3, 20:4, 44:154, 46:4, 46:21, 02:47, 02:178, *1382*
\newXeTeXintercharclass 57:35
\next *1385*
\NextLinkTarget 17:59, *460*
nfss internal commands:
__nfss_init_mv_freeze:N 28:334, 28:359, 28:360
__nfss_transform_scriptfont:nw 26:346, 26:350
\NG 21:554, 21:1173, 57:720, *1377*
\ng 21:575, 21:1174, 57:720, *1377*
\ni 30:441, 30:442
\noalign 30:350, 30:542, 30:545, 30:548, 30:549, 30:553, 30:554, 38:171, 38:172, 38:188, 38:191, 38:205, 38:497, 38:507, 41:301, 41:307, 41:436, 41:455, 42:159, 42:165, *972*
\nobreak 17:73, 17:84, 17:95, 17:103, 18:60, 18:72, 18:116, 18:142, 18:150, 18:165, 18:171, 18:183, 18:196, 18:222, 18:420, 18:428, 18:454, 18:462, 18:483, 18:490, 18:550, 20:203, 20:218, 20:232, 21:430, 21:456, 21:458, 21:591, 21:1193, 37:420, 37:427, 40:710, 40:720, 44:90, 44:237, 44:238, 44:242, 44:260, 44:261, 44:265, 45:435, 45:494, 45:593, 48:270, 49:48, 49:58, 49:71, 49:79, 49:94, 49:102, 06:978, 06:999, 54:341, 54:1501, 54:1650, 54:1811, 57:208, 57:210, 57:214, 57:215, 57:216, 57:220, 02:378, 02:381, 02:383, *453*
\nobreakdashes 18:475
\nobreakspace 18:489, *480*
\nobreakspace␣ *480*
\NoCaseChange 57:620, *1426*
\nocite *1053*
\nocite 47:59, *1384*
\nocorr . . . 32:44, 32:59, 32:63, 32:66, *1372*
\nocorrlist 32:90, 32:122
\noexpand *1426*
\nofiles *485*
\nofiles 20:199, *1386*
\noindent 16:90, 24:767, 24:793, 44:139, *454*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\nointerlineskip 30:350, 30:542, 30:545, 30:549, 30:553, 38:314, 38:342, 38:367, 42:573, 42:576, 42:586, 42:588, 54:2419, 54:2427, 02:373
\nolimits 30:359, 30:366, 38:3, 38:4, 38:5, 38:9, 38:10, 38:11, 38:12, 38:13, 38:14, 38:15, 38:16, 38:17, 38:18, 38:19, 38:20, 38:21, 38:26, 38:27, 38:28, 38:29, 38:31, 38:34
\nolinebreak *463*
\nolinebreak 18:9, 18:28
\nonfrenchspacing 02:582, 20:48, 20:118, 20:176, 06:979, 06:1000, 02:306
\nonscript 38:36, 38:38
\nonstopmode *1073*
\nonumber 38:480, 38:519, 38:520
noop (plug) *356*, *358*, *359*, *1234*, *1251*
\nopagebreak *463*
\nopagebreak 18:7, 18:26
\noprotrusion 44:247, 44:270
\normalbaselines 38:167, 38:169, 02:271, 02:284, 02:285, 02:310
\normalbaselineskip 26:189, 40:459, 40:478, 02:299, 02:311
\normalcolor 38:410, 38:549, 40:110, 40:528, 44:243, 44:266, 45:97, 45:166, 54:221, 54:620, 54:746, 54:919, 54:934, 54:984, 54:994, 54:1043, 54:1053, 54:2887, 54:2930, 54:2960, *1383*
normalfont (hook) *244*
\normalfont 24:744, 24:805, 29:764, 29:799, 29:801, 29:810, 29:815, 29:817, 29:826, 29:831, 29:833, 29:841, 29:848, 29:850, 29:856, 32:18, 37:573, 38:410, 38:549, 44:243, 44:266, 45:401, *1389*
normalfont 29:499
\normalize *1370*
\normallineskip 40:457, 40:477, 02:299, 02:310
\normallineskiplimit 38:206, 40:442, 40:458, 40:464, 02:299, 02:311
\normalmarginpar 45:387
\normalsfcodes 20:44, 20:46, 20:48, 20:114, 20:116, 20:118, 20:172, 20:174, 20:176, 20:198, 54:903, 54:973, 54:1032, *1399*
\normalshape 25:3202, 25:3243, *661*
\normalsize 20:42, 20:112, 20:170, 32:143, 45:23, 45:176, 45:372, 50:5, 54:902, 54:972, 54:1031, *1372*
\not 30:349, 30:423, 30:424, 30:446
\notexpanded *343*
\notin 30:472
\NoValue 07:508, 07:516, 07:522, 07:938, 07:1005, 07:1066, 07:1548, 07:1826, 07:1963, 07:2068, 07:2229, 07:2236, 07:2302, 07:2350, 07:2426, 07:2449, 07:2533, 07:3280, 07:3282, 07:3288, 07:3294, *1440*
\noxpand *1362*
\nu 30:291
\null 21:347, 21:383, 21:509, 21:512, 21:858, 21:861, 21:1281, 35:21, 35:38, 35:55, 37:538, 37:559, 37:722, 37:732, 38:112, 38:121, 38:169, 38:198, 44:237, 44:260, 53:438, 02:324, *1386*
\nulldelimiterspace 30:653, 02:273
\nullfont 37:244
\number 03:52, 03:61, 04:105, 22:309, 24:693, 24:696, 26:456, 28:65, 28:94, 28:114, 28:129, 28:165, 28:196, 28:226, 28:258, 06:2, 29:756, 06:132, 50:1248, 50:1341, 50:1474, 50:1563, 53:398, 01:70
\numberline 44:72, 44:82, 44:273, 45:17, *1361*
\numexpr 04:82, 04:105, 04:157, 21:1058, 28:151, 06:839, 02:112, 02:128, 02:138
\nunknown 04:813
\nwarrow 30:418
\nxt *1385*

O

\O 21:264, 21:421, 21:556, 21:794, 21:1158, 57:719
\o 21:273, 21:426, 21:577, 21:805, 21:1164, 57:719
\oalign 02:410
\obeycr 18:578
\obeyedline 07:1652, 07:1697, 07:1770, 07:1772, 07:1774, 07:2246, 07:2252, 07:2261, 07:2268, 02:334, 02:338, 02:361, *126*
\obeyedspace ... 02:339, 02:342, 02:362, *26*
\obeylines 37:544, 37:565, 37:713, 37:714, 06:980, 06:1001, 54:834, 02:327, *26*
\obeyspaces 06:981, 06:1002, 54:834, 02:327, *26*
\oddsidemargin 54:48, 54:50, 54:894, 54:966, 54:1025
\odot 30:395
\OE 21:263, 21:420, 21:555, 21:793, 21:1175, 57:662, 57:667, 57:672, 57:719, *1348*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\oe 21:272, 21:425, 21:576, 21:806, 21:1176, 57:662, 57:667, 57:672, 57:719, *1348*
\of 38:67, 38:414
off (plug) 54:727, *1234*, *1251*, *1251*
\offinterlineskip 02:373
\oint 30:366
\ointop 30:365, 30:366
\oldstylenums 33:4, 33:340, 33:341, 33:342, 33:343, 33:344, 33:345, 33:346, 33:347, 33:348, 33:349, 33:574, 33:1097, *799*
\Omega 30:318
\omega 30:301
\ominus 30:398
\omit 38:191, 38:192, 41:446, 41:449, 41:456, 41:460, *125*
\OmitIndent 16:93, 16:172, *451*
on (plug) 54:724, *1234*
\onecolumn 54:120
\OnlyDescription 26:5, 31:3
\ooalign 21:347, 21:377, 21:415, 21:503, 21:509, 21:511, 21:522, 21:538, 21:755, 21:788, 21:858, 21:861, 21:918, 21:1281, 29:761, 30:473, 30:476, 02:410, *1404*
\openin *1160*
\openout *1360*
\openup 38:199, 38:204
\oplus 30:399
\OptionNotUsed . . . 50:544, 50:571, 50:1151
or commands:
  \or: 07:1505, 07:1506, 08:1273, 08:1274, 08:1275, 08:1276, 08:1277, 08:1278, 08:1279, 08:1280, 08:1281, 08:1422, 08:1438
\oslash 30:396
\OT 21:392
\otimes 30:397
\outer 04:21, 04:38, *1387*
\output 54:261, 02:203
\outputmode 57:320
\outputpenalty 54:263, 54:277, 54:300, 54:303, 54:304, 54:339, 54:1511, 54:1512, 54:1660, 54:1661, 54:1821, 54:1824, 02:203
\oval 42:461, 42:464, 42:822, 42:839
\over 30:480, 38:162, 38:412
\overbrace 30:547
\overfullrule 49:142, 02:267
\overleftarrow 30:544
\overrightarrow 30:541
\owns 30:442, 30:443

P

\P 21:330, *1382*
package (hook) *236*
package/ (hook) *1162*, *1169*
package/.../after *1161*
package/.../before *1161*
package/⟨*name*⟩/after (hook) *1161*
package/⟨*name*⟩/before (hook) *1161*
package/⟨*package name*⟩/after (hook) *1162*
package/⟨*package name*⟩/before (hook) *1162*
package/after (hook) *1161*, *1162*
package/after *1161*
package/before (hook) *1161*, *1162*
package/before *1161*
\PackageError . . 03:77, 03:135, 03:147, 14:84, 21:1575, 33:787, 33:838, 33:882
\PackageInfo . . . 14:84, 33:791, 33:807, 33:812, 33:828, 33:829, 33:889, 33:1174
\PackageNote 14:136, *1422*
\PackageNoteNoLine 14:136
\PackageWarning 14:84, 33:789, 33:839, 33:1103, 53:550
\PackageWarningNoLine 14:84, 21:1008, 54:2570
page 36:257, *854*
page@sofar (tag socket) 55:270
\pagebreak *463*
\pagebreak . . 18:6, 18:7, 18:23, 18:25, *1244*
\pagegoal 54:2457, 54:2464
pagenum 36:258, *854*
\pagenumbering *839*
\pagenumbering 34:5
\pageref 35:10, *849*
\pageshrink . 54:529, 54:533, 54:548, 54:780
\pagestyle 49:2
pagetarget 36:263, *854*
\pagetotal 54:106
\paperheight 54:69
\paperwidth 54:69
\par 01:104, 04:156, 15:3, 15:4, 15:5, 16:142, 16:178, 18:261, 18:312, 24:770, 06:9, 06:21, 37:186, 37:242, 37:420, 37:427, 37:536, 37:557, 39:63, 39:110, 39:127, 39:130, 39:137, 39:192, 39:195, 40:448, 40:469, 40:524, 40:554, 40:571, 41:247, 41:270, 41:462, 44:41, 44:90, 44:245, 44:267, 45:15, 45:24, 45:249, 45:344, 45:540, 45:558, 02:11, 49:135, 49:136, 53:454, 54:145, 54:172, 54:198, 54:262, 54:2463, 02:320, 02:336, 02:338,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

02:355, 02:377, 02:386, 02:387, 02:388, 02:390, 02:392, 02:394, *1395*
para (plug) *1234*
para commands:
\para_end: 16:95, 16:95, 16:142, 16:143, 16:144, *452*
\g_para_indent_box . . 16:16, 16:49, 16:84, 16:86, 16:91, 16:93, 16:119, *445*
\para_omit_indent: . . 16:88, 16:94, *445*
\para_raw_end: 16:115, 16:137, 16:140, *445*
\para_raw_indent: 16:115, 16:115, 16:138, *454*
\para_raw_noindent: 16:115, 16:127, 16:139, *454*
para internal commands:
__para_handle_indent: 16:34, 16:66, 16:85, 16:85, 16:121
\g__para_standard_everypar_tl . . . 16:12, 16:76, 16:78, 16:120, 16:131, *450*
__para_tmp:w 16:37, 16:41, 16:68, 16:72
para/after (hook) *443*, *454*
para/after 16:6, *443*
para/before (hook) *443*, *449*
para/before 16:6, *443*
para/begin (hook) *443*, *445*, *452*
para/begin (tag socket) 55:117
para/begin 16:6, *443*
para/end (hook) *443*, *444*, *454*
para/end (tag socket) 55:117
para/end 16:6, *443*
para/off (tag socket) 55:101
para/on (tag socket) 55:101
para/restore (tag socket) 55:107
para/semantic/begin (tag socket) . . 55:124
para/semantic/end (tag socket) . . . 55:124
para/textblock/begin (tag socket) . 55:145
para/textblock/end (tag socket) . . 55:145
\paracntvalue *446*
\paragraphmark 44:143
\parallel 30:412
\parbox *925*
\parbox 40:303, *443*
\parfillskip . . . 24:743, 24:759, 24:804, 37:456, 37:466, 37:475, 37:483, 37:532, 37:554, 39:76, 40:456, 40:477, 44:232, 44:255, 02:298, *454*
\parindent 37:456, 37:461, 37:466, 37:475, 37:479, 37:483, 37:532, 37:554, 39:50, 40:451, 40:472, 44:233, 44:256, 02:279, 02:402, 02:403, *440*
\parsep 39:1, 39:49, 39:90, *909*
\parseunicodedataI 04:123, 04:162
\parseunicodedataII 04:124, 04:126
\parseunicodedataIII 04:128, 04:134
\parseunicodedataIV 04:130, 04:142
\parseunicodedataV 04:146, 04:149
\parshape 39:54, *448*
\parskip 37:431, 37:530, 37:532, 37:552, 37:554, 38:621, 39:49, 39:73, 39:88, 39:90, 39:117, 39:184, 39:203, 39:254, 40:451, 40:472, 41:79, 54:1511, 54:1660, 54:1823, 02:286, *1428*
\partial 30:326
\partokencontext 57:724, 57:725, 57:729, 57:730, *1350*
\partopsep 38:619, 39:1, 39:61, *909*
\PassOptionsToClass *1096*
\PassOptionsToClass 50:476
\PassOptionsToPackage *1096*
\PassOptionsToPackage 50:476
\patterns 21:225, *1384*
\pausing 02:208
pdfannot commands:
\pdfannot_link_off: 53:19
\pdfannot_link_on: 53:18
\pdfextension 53:15, 57:325
\pdffilesize 05:76, 05:122
\pdfgentounicode 57:317, 57:327, 57:329, 57:333, 57:359, 57:362, 57:364, 57:369, 57:374, 57:386, 57:391, 57:392
\pdfglyphtounicode . . . 57:319, 57:363, 57:820, 57:821, 57:822, 57:823, 57:824, 57:825, 57:826, 57:827, 57:828, 57:829, 57:830, 57:831, 57:832, 57:833, 57:834, 57:835, 57:836, 57:837, 57:838, 57:839, 57:840, 57:841, 57:842, 57:843, 57:844, 57:845, 57:846, 57:847, 57:848, 57:849, 57:850, 57:851, 57:852, 57:853, 57:854, 57:855, 57:856, 57:857, 57:858, 57:859, 57:860, 57:861, 57:862, 57:863, 57:864, 57:865, 57:866, 57:867, 57:868, 57:869, 57:870, 57:871, 57:872, 57:873, 57:874, 57:875, 57:876, 57:877, 57:878, 57:879, 57:880, 57:881, 57:882, 57:883, 57:884, 57:885, 57:886, 57:887, 57:888, 57:889, 57:890, 57:891, 57:892, 57:893, 57:894, 57:895, 57:896, 57:897, 57:898, 57:899, 57:900, 57:901, 57:902, 57:903, 57:904, 57:905, 57:906, 57:907, 57:908, 57:909, 57:910, 57:911, 57:912, 57:913, 57:914, 57:915, 57:916, 57:917, 57:918, 57:919, 57:920, 57:921, 57:922, 57:923,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

57:924, 57:925, 57:926, 57:927, 57:928, 57:929, 57:930, 57:931, 57:932, 57:933, 57:934, 57:935, 57:936, 57:937, 57:938, 57:939, 57:940, 57:941, 57:942, 57:943, 57:944, 57:945, 57:946, 57:947, 57:948, 57:949, 57:950, 57:951, *1441*
\pdfhorigin 53:375
\pdfsavepos *854*
\pdftexrevision 57:334, 57:376
\pdftexversion 57:332, 57:333, 57:334, 57:373, 57:376
\pdfvariable 53:374, 53:379, 57:327, 57:357, 57:371
\pdfvorigin 53:380
peek commands:
\peek_meaning:NTF 07:2775, *204*
\peek_meaning_remove:NTF 07:2106, 07:2223, 07:2763, *204*
\peek_N_type:TF 07:2112, 07:2136, 07:2168
\peek_remove_spaces:n 07:2104
\penalty . . 18:35, 18:38, 18:47, 18:318, 18:328, 18:351, 18:361, 18:385, 18:389, 32:119, 37:538, 37:541, 37:559, 37:562, 38:37, 38:207, 38:497, 38:507, 39:221, 41:56, 45:195, 45:199, 45:201, 45:217, 45:221, 45:223, 47:37, 54:114, 54:155, 54:182, 54:200, 54:203, 54:1509, 54:1658, 54:1819, 02:382, 02:383, 02:384, 02:385, 02:386, 02:387, 02:391, 02:393, 02:395, *1359*
\perp 30:458
\phantom 38:75
\Phi 30:316
\phi 30:298
\Pi 30:313
\pi 30:293
picture (env.) 42:21
\picture 42:21
Plugs:
default *1234*
floats:footnotes 54:697, *1234*
floats:space:footnotes . . 54:681, *1233*
footnotes:floats 54:702, *1234*
footnotes:floats:legacy . 54:707, *1234*
footnotes:space:floats . . 54:674, *1233*
identity *356*, *359*, *359*, *359*
kernel (refstepcounter/target) 35:136
noop *356*, *358*, *359*, *1234*, *1251*
off 54:727, *1234*, *1251*, *1251*
on 54:724, *1234*
para *1234*
space:floats:footnotes . . 54:688, *1233*
space:footnotes:floats . . 54:664, *1233*
tagpdf *359*
\pm 30:401
\pmatrix 38:173, 38:181
\pmod 38:39
\PopDefaultHookLabel 08:2820, *224*
\poptabs 14:256, 41:142, 41:161
\poptracing 26:149, 26:341
\postdisplaypenalty 18:13, 38:561, 38:573, 38:599, 02:200
\pounds 21:338, *1377*
\Pr 38:32
pre commands:
pre_shipout_filter *1188*
\prec 30:432
\preceq 30:435
\predisplaypenalty 18:12, 38:560, 38:572, 38:598, 02:199, *1428*
\predisplaysize 02:276
\pretolerance 24:745, 24:761, 24:806, 02:186
\prevdepth 18:324, 18:325, 18:357, 18:358, 18:418, 18:423, 18:452, 18:457, 38:205, 45:196, 45:198, 45:218, 45:220, 54:146, 54:148, 54:151, 54:173, 54:175, 54:178, 02:373, 02:377, 02:378, *476*
\PreviousTotalPages 53:460, *1208*
prg commands:
\prg_break:n 07:3264, 07:3266, 07:3268, 07:3270, 07:3271
\prg_break_point: 07:3272
\prg_do_nothing: 08:2346, 09:209, 09:284, 09:421, 09:640, 48:265, 52:132, 52:134, 53:227, 53:228, 53:233, 53:242, 53:246, 53:382, 53:538, 55:12, 55:36, 55:46, 07:410, 07:999, 07:1005, 07:2048, *158*
\prg_generate_conditional_variant:Nnn 11:166, 36:187, 36:202, 36:217
\prg_gset_conditional:Npnn . 07:3284
\prg_new_conditional:Npnn 08:2461, 08:2476, 08:2493, 08:2510, 08:2516, 08:2522, 08:2528, 08:2537, 08:2545, 08:2559, 08:2577, 09:327, 10:53, 10:117, 10:138, 11:9, 11:160, 11:167, 11:173, 36:176, 36:191, 36:206, 48:299, 48:306, 55:26, 08:267, 08:903, 08:1216, 08:1426
\prg_new_protected_conditional:Npnn 09:123, 09:484, 08:754, 08:812, 08:842, 08:874

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\prg_replicate:nn 08:2329, 08:2337, 08:2407, 08:2423, 52:552, 07:906, 07:949, 07:1010, 07:1065, 07:2532, 07:2539, 07:2824, 08:1190
\prg_return_false: 08:2471, 08:2487, 08:2505, 08:2514, 08:2520, 08:2526, 08:2534, 08:2543, 08:2552, 08:2555, 08:2569, 08:2584, 09:138, 09:340, 09:499, 10:56, 10:120, 10:142, 11:28, 11:32, 11:37, 11:164, 11:171, 11:177, 36:184, 36:199, 36:214, 48:304, 48:311, 55:27, 07:3291, 08:273, 08:781, 08:835, 08:847, 08:867, 08:879, 08:894, 08:911, 08:915, 08:918, 08:1221, 08:1431, *269*
\prg_return_true: 08:2469, 08:2485, 08:2504, 08:2507, 08:2513, 08:2519, 08:2525, 08:2532, 08:2542, 08:2553, 08:2567, 08:2582, 09:137, 09:340, 09:498, 10:55, 10:119, 10:141, 11:28, 11:32, 11:37, 11:163, 11:170, 11:176, 36:181, 36:196, 36:211, 48:303, 48:310, 07:3289, 08:272, 08:772, 08:826, 08:865, 08:892, 08:914, 08:1219, 08:1429
\prime 30:264, 30:328, 38:256
\ProcessedArgument . . . 07:391, 07:395, 07:402, 07:2465, 07:2466, 07:2483, 07:2485, 07:2507, 07:2516, 07:2522, 07:2530, 07:2547, 07:2558, 07:2561, 07:2586, 07:2652, 07:2654, 07:3315, *200*
\ProcessKeyOptions . . . 51:86, 51:227, *1150*
\ProcessKeyPackageOptions *1425*
\ProcessList 07:3320, *124*
\ProcessOption* *1362*
\ProcessOptions 21:1600, 26:72, 33:805, 33:842, 50:572, 50:663, 50:1236, *1368*
\ProcessOptions* 50:572
\ProcessorA *123*
\ProcessorB *123*
\prod 30:360
prop commands:
\prop_clear:N 11:230, 11:240, 11:256, 11:266, 11:273, 11:302, 11:303, 11:304, 07:1956, 08:1163
\prop_clear_new:N 11:206
\prop_const_from_keyval:Nn 08:957, 08:965, 08:967, 08:969
\prop_gclear:N 09:74, 11:193, 08:990, 08:1025, 08:1447, 08:1474
\prop_gclear_new:N 08:2448, 08:2458, 11:182, 11:214, 11:217
\prop_get:NnN . . . 08:1617, 08:1703, 11:120, 11:705, 11:708, 11:735, 11:775, 11:855, 11:862, 11:869, 11:1002, 11:1023, 11:1115, 08:587
\prop_get:NnNTF 11:467, 11:582, 11:686, 11:797, 11:823, 11:842, 11:895, 11:1030, 11:1126, 11:1199, 51:232, 07:1962, 08:596, 08:641
\prop_gpop:NnNTF 08:998, 08:1033
\prop_gput:Nnn 08:1562, 08:2612, 08:2613, 08:2614, 09:67, 10:191, 11:288, 11:1548, 17:29, 48:313, 07:58, 08:597, 08:598, 08:644, 08:647, 08:1524
\prop_gset_eq:NN 11:183, 11:196, 11:215, 11:218, 08:1165
\prop_if_empty:NTF 08:2004, 08:2481, 08:1504, 08:1539
\prop_if_empty_p:N . . 08:1900, 08:2498
\prop_if_exist:NTF 08:2108, 08:2480, 08:2518, 11:190, 11:224, 11:234, 11:250, 11:260, 11:267
\prop_if_exist_p:N 08:1899
\prop_if_in:NnTF . . 11:143, 11:277, 11:699, 11:703, 08:849, 08:857, 08:861, 08:881, 08:884, 08:888, 08:913
\prop_item:Nn 11:1064, 17:37
\prop_map_break: 08:1594, 08:1676, 08:2116
\prop_map_function:NN 09:73, 11:1299, 11:1310, 17:48, 17:55, 08:1164, *279*
\prop_map_inline:Nn 08:1582, 08:1589, 08:1591, 08:1664, 08:1671, 08:1673, 08:1832, 08:1903, 08:2007, 08:2111, 08:2113, 11:565, 51:267, 08:1459, 08:1480
\prop_new:N 09:17, 11:75, 11:86, 11:88, 11:90, 11:91, 11:195, 17:26, 07:53, 08:30, 08:31, 08:146, 08:158, 08:1314, 08:1315
\prop_pop:NnNTF 11:1028
\prop_put:Nnn . . . 08:1802, 08:1813, 11:352, 11:364, 11:513, 11:516, 11:519, 11:644, 11:656, 11:661, 11:681, 11:744, 11:887, 11:921, 11:928, 11:933, 11:969, 07:1953, 08:1199
\prop_remove:Nn 11:588
\prop_set_eq:NN 08:1557, 11:207, 11:227, 11:237, 11:253, 11:263, 11:270, 08:1520
\prop_show:N *304*
property commands:
\property_gset:nnnn 36:6, 36:17, 36:46, *850*
\property_if_exist:n . 36:176, 36:187

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\property_if_exist:nTF 36:176, 36:188, 36:189, 36:190, *852*
\property_if_exist_p:n . . . 36:176, *852*
\property_if_recorded:n 36:191, 36:202
\property_if_recorded:nn 36:206, 36:217
\property_if_recorded:nnTF 36:206, 36:218, 36:219, 36:220, 36:241, *852*
\property_if_recorded:nTF 36:191, 36:203, 36:204, 36:205, 36:230, 36:239, *852*
\property_if_recorded_p:n 36:191, *852*
\property_if_recorded_p:nn 36:206, *852*
\property_item:nn 36:118, 36:118, 36:126, *851*
\property_item:nnn 36:127, 36:127, 36:139, *1442*
\property_new:nnnn 36:6, 36:6, 36:41, 36:255, 36:257, 36:258, 36:259, 36:260, 36:262, 36:264, 36:265, 36:266, 36:267, *850*
\property_record:nN . 36:48, 36:48, *851*
\property_record:nn 36:48, 36:49, 36:50, 36:52, 36:85, *851*
\property_ref:nn 36:89, 36:89, 36:96, 36:144, *851*
\property_ref:nnn 36:97, 36:97, 36:117, 36:147, *851*
\property_ref_undefined_warn: 36:221, 36:221, *851*
\property_ref_undefined_warn:n 36:228, 36:228, 36:236, *852*
\property_ref_undefined_warn:nn . . 36:237, 36:237, 36:253, 36:254, *850*
property internal commands:
__property_data:nnn 36:150, 36:162, 36:164, 36:169
__property_gset:nnnn 36:6, 36:10, 36:19, 36:22, 36:37
__property_record:nn 36:48, 36:51, 36:53, 36:62
__property_record_value:n 36:48, 36:59, 36:63
__property_record_value_aux:n 36:48, 36:64, 36:65, 36:79
__property_ref:nnn 36:91, 36:97, 36:99, 36:104, 36:116
l__property_ref_flag 36:88
\propto 30:409
\protect 14:246, 14:248, 14:250, 14:256, 14:262, 14:269, 14:279, 14:282, 14:288, 20:215, 20:228, 21:44, 21:50, 21:69, 21:73, 21:229, 21:237, 28:762, 28:1310, 29:749, 32:144, 06:120, 35:16, 35:33, 35:50, 06:273, 37:282, 37:285, 37:319, 37:329, 06:338, 06:347, 06:352, 06:355, 06:356, 06:358, 06:359, 06:364, 06:365, 06:370, 06:373, 06:374, 06:399, 06:437, 06:465, 41:341, 44:12, 44:72, 44:82, 44:164, 44:171, 44:177, 45:17, 06:672, 06:692, 47:5, 52:281, 52:304, 53:144, 53:156, 53:186, 53:193, 53:213, 54:881, 54:951, 54:1010, 57:407, *1388*
\protected 04:186, 17:8, 18:57, 18:260, 18:311, 18:496, 18:560, 21:328, 21:329, 22:30, 22:361, 25:3202, 25:3206, 25:3209, 25:3212, 25:3215, 25:3218, 25:3221, 25:3224, 28:1152, 06:7, 29:656, 29:702, 06:286, 06:293, 06:294, 37:203, 37:249, 06:310, 06:312, 06:314, 37:419, 06:329, 38:199, 38:481, 38:519, 38:538, 38:539, 40:334, 40:338, 40:341, 40:628, 41:56, 41:278, 41:285, 06:547, 42:153, 45:408, 45:467, 50:806, 53:570, 53:571, 53:572, 54:845, 54:846, 57:319, 57:327, 02:338, *99*
\Provide *1381*
\Provide... . . . . *117*
\providecommand 08:2945, 08:2948, 21:6, 21:13, 21:1005, 06:219, 35:112, 35:113, 35:114, 54:2581
\ProvideCommandCopy *100*
\ProvideDocumentCommand . . . 07:3103, *117*
\ProvideDocumentEnvironment 07:3139, *117*
\ProvideExpandableDocumentCommand 07:3222, *125*
\ProvideHook 08:2891, *1421*
\ProvideMirroredHookPair 08:2891
\ProvideReversedHook 08:2891
provides commands:
provides_module 04:299
\provides_module *43*
\ProvidesClass *1096*
\ProvidesClass 50:456
\ProvidesExplPackage 52:541
\ProvidesFile . . 30:676, 30:678, 30:679, 30:680, 33:1178, 50:465, 01:73, *1390*
\ProvidesPackage *1096*
\ProvidesPackage 26:13, 33:779, 33:810, 50:379, 50:458, 50:460, 50:1851, 52:570, 53:565, *1366*
\ProvideTextCommand 21:3, 21:78, *1381*
\ProvideTextCommandDefault . . 21:75, *1381*
\Psi 30:317

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\psi 30:300
\PushDefaultHookLabel 08:2820, *224*
\pushtabs 14:256, 41:139, 41:138, 41:158, 41:160
\pushtracing 26:118, 26:322
\put 42:67, 42:69, 42:81, 42:83, 42:336, 42:337, 42:338, 42:339, 42:347, 42:349, 42:362, 42:363, 42:364, 42:365, 42:372, 42:375, 42:395, 42:396, 42:397, 42:398, 42:404, 42:406, 42:418, 42:419, 42:420, 42:421, 42:426, 42:431, 42:739, 42:795, 42:823, 42:840, *1192*

Q

\qbezier *974*
\qbezier 42:692, 42:824, 42:841
\qbeziermax . 42:691, 42:717, 42:718, 42:778
\qquad 18:571
\quad 18:571, 38:168, 38:170, 38:190, 44:111, *1388*
quark commands:
\q_mark 07:1340, 07:1343, 07:2374, 07:2378, 07:2390, 07:2712, 07:2720, 07:2723, 07:2728
\q_nil 11:162, 11:175, 11:834, 11:836, 07:601, 07:1332, 07:1343, 07:1516, 07:1769, 07:1770, 07:1772, 07:1773, 07:1774, 07:1775, 07:1776, 07:1862, 07:1883, 07:1887, 07:1893, 07:1902, 07:1904, 07:2300, 07:2322, 07:2331, 07:2359, 07:2374, 07:2378, 07:2390, 07:2396, 07:2420, *192*
\quark_if_nil:NTF 07:612, 07:2380, *193*
\quark_if_nil:nTF 07:1513
\quark_if_recursion_tail_stop:N 07:828, 07:1497
\quark_if_recursion_tail_stop:n 36:166, 51:7, 07:483, 07:673, 07:1258, 07:2725, 07:2726, 08:416
\quark_if_recursion_tail_stop_do:Nn 07:620
\quark_if_recursion_tail_stop_do:nn 09:440, 09:459, 07:507, 07:512, 07:521, 07:528, 07:574, 07:587, 07:597, 07:641
\quark_new:N 09:15, 09:16, 11:103, 07:50, 07:51, 07:1871
\q_recursion_stop 09:429, 36:162, 51:20, 07:336, 07:347, 07:460, 07:811, 07:1254, 07:1493, 07:1954, 07:1969, 07:1971, 07:1978, 07:2715, 07:2721, 08:422
\q_recursion_tail 09:429, 36:162, 51:19, 07:460, 07:811, 07:1254, 07:1493, 07:1969, 07:1973, 07:2711, 07:2714, 07:2720, 07:2721, 08:421, 08:422, *146*
\q_stop 11:16, 11:18, 11:40, 11:43, 11:46, 11:162, 11:175, 11:793, 11:811, 11:834, 11:836, 07:423, 07:601, 07:610, 07:615, 07:817, 07:818, 07:1340, 07:1342, 07:1770, 07:1776, 07:1887, 07:1904, 07:2523, 08:569, 08:574
quark internal commands:
\q__cmd . . . 07:407, 07:411, 07:1034, 07:1370, 07:1371, 07:1452, 07:1460, 07:1869, 07:2286, 07:2287, 07:2288, 07:2293, 07:2299, 07:2300, 07:2302, 07:2304, 07:2309, 07:2312, 07:2315, 07:2326, 07:2330, 07:2331, 07:2334, 07:2335, 07:2336, 07:2347, 07:2350, 07:2352, 07:2363, 07:2365, 07:2366, 07:2367, 07:2368, 07:2369, 07:2370, 07:2371, 07:2375, 07:2394, 07:2395, 07:2396, 07:2397, 07:2398, 07:2399, 07:2400, 07:2401, 07:2402, 07:2403, 07:2404, 07:2405, 07:2406, 07:2407, 07:2408, 07:2413, 07:2419, 07:2420, 07:2426, 07:2429, 07:2430, 07:2431, 07:2442, 07:2449, 07:2452, 07:2453, 07:2454, 07:2457, 07:2458, 07:2460, *193*
\q__cmd_recursion_stop 07:51, 07:2592, 07:2594, 07:2603, 07:2626, 07:2635, *133*
\q__cmd_recursion_tail 07:50, 07:2592, *133*
\q__hook_recursion_stop 09:15, 09:342, 09:343, 09:352, 09:394
\q__hook_recursion_tail 09:15, 09:342, 09:356
\q__template_nil . 11:103, 11:1203, *383*
\quotedblbase 21:578, 21:807, 21:1204
\quotesinglbase 21:579, 21:1201

R

\r 21:256, 21:409, 21:452, 21:493, 21:633, 21:660, 21:670, 21:696, 21:780, 21:819, 21:1273, 21:1291, 21:1317, 21:1439, 21:1440, 33:147, 33:169, 02:318, 02:319, *1398*
\radical . . 28:1176, 28:1179, 28:1209, *1409*
\raggedbottom 49:126, *1244*
\raggedleft . 37:462, 37:480, 37:491, 37:498
\raggedright 37:457, 37:476, 37:490, 37:496

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\raise 21:347, 21:379, 21:451, 21:454, 21:756, 21:821, 21:919, 21:1281, 29:762, 30:476, 30:524, 30:526, 38:73, 40:690, 40:699, 42:72, 42:84, 42:116, 42:128, 42:206, 42:316, 42:463, 42:505, 42:536, 42:561, 42:629, 42:646, 42:647, 42:750, 42:806, 45:414
\raisebox *926*
\raisebox 21:896, 21:1250, 40:667
\rangle 30:603
\RawIndent 16:138, 16:174, *445*
\RawNoIndent 16:138
\RawNoindent 16:139, 16:175, *445*
\RawParEnd . . . 16:138, 16:176, 02:357, *445*
\RawShipout 53:212, 53:491, *1186*
\rbrace 21:329, 30:607
\rbrack 02:316
\rceil 30:611
\Re 30:324
\read *505*
\ReadonlyShipoutCounter 53:393, 53:493, *1189*
\RecordProperties 36:80, 36:293, *853*
\Ref *841*
\Ref 35:199, 35:201, 35:211, 35:213, 35:217, 35:219, 35:223, 35:226
\ref 35:10, 35:213, 35:219, 45:615, *849*
\RefProperty 36:140, 36:294, *853*
\refstepcounter *556*
refstepcounter (socket) 35:134
refstepcounter (tag socket) 55:206
\refstepcounter 35:132, 38:399, 38:406, 38:600, 39:233, 43:51, 44:59, 45:9, *846*
refstepcounter/target (socket) . . . 35:135
\RefUndefinedWarn . . . 36:237, 36:295, *853*
regex commands:
  \regex_match:nnTF 11:534
\registernumber *42*
registernumber 04:390
\relax *1375*
\relax␣ *1375*
\Relbar 30:487, 30:495, 30:497, 30:503, *1403*
\relbar 30:484, 30:499, 30:501
\relpenalty 02:194
remove commands:
  remove_from_callback 04:937
\remove_from_callback *44*
\RemoveFromHook 08:2818, 08:2947, 37:41, 37:42, 37:43, 37:106, 37:107, 37:108, *221*
\removelastskip 02:389, 02:391, 02:393, 02:395
\Renew... . . . . *117*
\renewcommand *81*
\renewcommand 30:78, 30:80, 30:82, 30:83, 30:85, 30:87, 30:89, 30:90, 30:96, 30:98, 30:100, 30:101, 30:114, 30:115, 30:116, 30:124, 30:125, 06:163, 38:548, 38:568, 38:589, *746*
\RenewCommandCopy . . . 06:565, 06:567, *100*
\RenewDocumentCommand 07:3103, *117*
\RenewDocumentEnvironment 07:2901, 07:3139, *117*
\renewenvironment *82*
\renewenvironment . 06:193, 38:597, 38:609
\RenewEnvironmentCopy 06:606, 06:608, *1428*
\RenewExpandableDocumentCommand 48:522, 48:523, 48:524, 48:525, 07:3222, *213*
\repeat 04:154, 04:164, 24:787, 41:459, 50:1289, 50:1350, 50:1420, 50:1483, 50:1534, 50:1572, 57:418, 57:429, 57:439, 57:450, 57:480, 57:506, 57:516, 01:65, 01:67, 02:365
\requestedLaTeXdate 08:2967, 50:1635, 50:1676, 50:1697, 50:1784
\requestedpatchdate 50:1707, 50:1785
\RequirePackage *1096*
\RequirePackage 04:24, 50:711, 50:718, 50:761, 50:770, 50:1232, 54:2578, *1362*
\RequirePackageWithOptions *1096*
\RequirePackageWithOptions 50:753
reserved@a commands:
  \reserved@a: 20:252, 20:324, 20:386, 20:433, 50:1262, 50:1397
reserved@b commands:
  \reserved@b: 50:267, 50:284
reserved@c commands:
  \reserved@c: 20:774
\RestoreAtCatcode *1376*
\restorecr 18:578
\RestoreLastSkip 17:67, 17:69, 17:73, 17:75, 17:82, 17:95, 17:101, 17:118, 17:122, *1442*
\ResumeTagging *1309*
\ResumeTagging 23:22, 55:3, *1309*
\ReverseBoolean 07:3316, *124*
\reversemarginpar 45:387
\rfloor 30:615
\rgroup 30:619
\rhd 29:889
\rho 30:294
\rhook 30:490, 30:491
\right 30:636, 30:638, 30:640, 30:642, 30:647, 30:648, 30:649, 30:650, 38:168, 38:173, 38:197
\Rightarrow 30:422, 30:497, 30:509

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\rightarrow 30:449, 30:451, 30:455, 30:489, 30:499, 30:507, 30:560
\rightarrowfill 30:542, 30:558
\rightharpoondown 30:466
\rightharpoonup 30:465, 30:477
\righthyphenmin 56:11, 02:246
\rightleftharpoons 30:475
\rightline 40:725
\rightmargin 39:9, 39:40, 39:51, *909*
\rightmark 49:106, *1366*
\rightskip 24:758, 37:454, 37:458, 37:464, 37:474, 37:477, 37:482, 37:531, 37:553, 39:75, 40:455, 40:476, 44:232, 44:255, 02:292, 02:403
\rlap 21:451, 21:454, 21:821, 38:521, 38:549, 40:729, 41:81, *1389*
\rm *1365*
\rmdefault 29:7, 29:301, 29:477, 29:486, 29:518, 29:529, 29:561, 29:779, 30:62, 30:131, 33:7, 33:577, *740*
rmfamily (hook) *244*, *244*, *244*
\rmfamily . . 29:5, 29:6, 29:484, 29:527, 29:528, 29:559, 29:560, 32:15, *1390*
rmfamily 29:499
\rmmath *1358*
\rmoustache 30:576
\rmsubstdefault 30:18, 30:30, 33:28, 33:39, 33:49
\Roman *556*
\Roman 22:305, *1189*
\roman *556*
\roman 22:304
\romannumeral 22:310, 22:311, 37:318, 37:350, 37:367, 39:43, 39:265, 39:276, *875*
\root 38:66, 38:414
\rootbox 38:66
\rowcolor *972*
\rq 02:314
\rule *926*
\rule 40:553, 40:570, 40:587, 40:593, 45:539, 45:557, 45:575

S

\S 21:331, *1382*
\safesubencodingfoundfalse 33:1219
\safesubencodingfoundtrue 33:1209
\samepage *463*
\samepage 18:11, 18:29, *1359*
\SaveAtCatcode *1373*
\savebox *925*
\savebox 40:160
\savecatcodetable . 04:117, 04:168, 04:170
\SaveLastSkip . 17:68, 17:118, 17:122, *1442*
\savepos *854*
\sb 38:212
\sbox *925*
\sbox 19:4, 21:521, 21:537, 39:236, 40:166, 40:173, 40:177, 40:182, 40:187, 45:413, 45:472, 02:397, *1391*
scan commands:
\scan_new:N 11:101, 11:102, 08:38
\scan_stop: 08:2578, 08:2579, 09:486, 09:487, 09:545, 09:546, 09:581, 09:582, 16:96, 48:106, 48:108, 48:118, 48:120, 48:122, 48:123, 48:280, 48:281, 48:282, 51:88, 51:98, 53:25, 53:27, 53:28, 53:50, 53:52, 53:53, 53:105, 07:1614, 07:1615, 07:1640, 07:1646, 07:1655, 07:1662, 07:2101, 07:2102, 08:683, 08:690, 08:701, 08:710, 08:745, 08:756, 08:792, 08:804, 08:814, 08:844, 08:876, 08:1218, *1154*
scan internal commands:
\s__file_stop 52:103, 52:105, 52:108, 52:110, 52:112, 52:125
\s__hook_mark 08:2295, 08:2309, 08:2358, 08:2366, 08:2469, 08:2471, 08:2532, 08:2534, 08:2538, 08:2539, 08:2546, 08:2547, 08:2560, 08:2561, 09:23, 09:25, 09:40, 09:42, 09:51, 09:53, 09:129, 09:134, 09:328, 09:339, 09:491, 09:494, 09:506, 09:510, 09:515, 09:550, 09:570, 09:586, 09:598, 08:38, 08:39, 08:40, 08:342, 08:345, 08:348, 08:352, 08:355, 08:356, 08:734, 08:785, 08:787, 08:795, 08:797, 08:800, 08:802, 08:904, 08:929, 08:930, 08:934, 08:1239, 08:1256, 08:1260, 08:1302, 08:1303, 08:1358, *247*
\s__keys_stop . . . 51:47, 51:206, 51:207
\s__template_mark 11:101, 11:1203, *383*
\s__template_stop 11:102, 11:377, 11:388, 11:399, 11:420, 11:434, 11:442, 11:594, 11:597, 11:620, 11:625, 11:1053, 11:1056, 11:1203, 11:1207, *383*
\scdefault 25:3214, 29:28, 30:106
\scriptfont 26:339
\scriptscriptfont 26:340
\scriptscriptstyle 38:65, 38:68
\scriptspace 45:435, 45:494, 02:274
\scriptstyle 30:348, 38:64
\scshape 21:320, 25:3212, 25:3213, 29:26, 29:27, 32:23, *1399*
\searrow 30:417

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\sec 38:20
sec/begin (tag socket) 55:220
sec/end (tag socket) 55:220
sec/title/begin (tag socket) 55:222
sec/title/end (tag socket) 55:222
sec/title/hang (tag socket) 55:224
sec/title/init (tag socket) 55:225
sec/title/number (tag socket) 55:228
sec/title/runin/number (tag socket) 55:225
sec/title/split (tag socket) 55:225
\secdef 44:142
\secondoftwo *500*
\section *328*
\sectionmark 44:143, *1064*
selectfont (hook) *245*
\selectfont 19:7, 21:322, 21:349, 21:380, 21:470, 21:833, 21:895, 21:1249, 21:1283, 21:1601, 24:354, 24:364, 24:374, 25:2916, 25:2921, 25:2926, 25:3204, 25:3208, 25:3211, 25:3214, 25:3217, 25:3220, 25:3223, 25:3226, 26:115, 26:116, 26:160, 26:163, 26:165, 29:7, 29:10, 29:13, 29:16, 29:19, 29:22, 29:25, 29:28, 29:31, 29:343, 29:366, 29:404, 29:424, 29:439, 29:450, 29:461, 29:464, 29:488, 29:493, 29:498, 29:531, 29:536, 29:541, 29:555, 29:558, 29:561, 29:564, 29:567, 29:647, 29:692, 29:752, 29:776, 29:809, 29:824, 29:839, 33:36, 33:58, 33:68, 33:600, 33:637, 33:877, 33:894, 45:403, 45:455, *245*
selectfont 26:151
seq commands:
\seq_clear:N 08:1579, 08:1661, 11:246, 11:305, 11:767
\seq_clear_new:N 08:1587, 08:1669
\seq_const_from_clist:Nn 11:56, 11:73
\seq_count:N 55:352
\seq_gclear_new:N 11:198
\seq_gpop:NN 08:2609
\seq_gpop:NNTF . . 52:72, 08:451, 08:459
\seq_gpop_right:NN 08:424
\seq_gpush:Nn 08:2597, 08:2603, 52:64, 08:444
\seq_gput_right:Nn 48:23, 08:102, 08:131, 08:413, 08:417
\seq_gset_eq:NN 11:199
\seq_if_empty:NTF 08:412, 08:469
\seq_if_exist:NTF 11:241, 52:60
\seq_if_in:NnTF 11:136, 11:149, 11:470, 11:505, 48:9, 48:248
\seq_map_break: 11:340, 11:431
\seq_map_function:NN 11:321, 11:437, 11:1018
\seq_map_inline:Nn 08:1603, 08:1621, 08:1686, 08:1709, 08:2990, 48:139, 48:172, 48:214, 48:225, 48:236, 48:390, 48:467, 07:2484, 08:1448, 08:1475
\seq_mapthread_function:NNN . . . *202*
\seq_new:N 08:1570, 08:2587, 10:36, 11:87, 11:89, 48:6, 52:61, 55:355, 07:2468, 08:28, 08:33
\seq_put_right:Nn 08:1585, 08:1667, 08:1770, 08:1777, 10:91, 11:354, 11:515, 11:886, 11:894
\seq_remove_all:Nn . . . 11:854, 11:868
\seq_set_eq:NN 11:243
\seq_set_split:Nnn . . . 55:351, 07:2482
\seq_use:Nnnn . 08:1830, 08:1835, 10:65
seq internal commands:
\g__mark_classes_seq 48:6, 48:9, 48:23, 48:139, 48:172, 48:214, 48:225, 48:236, 48:248, 48:390, 48:467, *1068*
\series *1358*
\seriesdefault 21:1602, 28:410, 28:460, 29:313, 29:315, 29:771, 29:804, 29:820, 29:836, 29:853, 29:930, 30:131, *765*
\setattribute *41*
\setattribute 04:82, 04:238
\setbox *918*
\setbox... . . . . *1397*
\setbox0 . . . . . *1394*
\setcounter *556*
\setcounter 20:464, 22:2, 22:59, 22:80, 22:99, 28:144, 39:256, 54:2991, 54:2994, 54:2997, 54:3001, *707*
\SetCurrentKnownTemplateKeys 11:1624, *373*
\SetDefaultHookLabel 08:2820, *224*
\SetKeys 20:770, 51:285, *1151*
\SetKnownTemplateKeys 11:1620, *374*
\setlength *569*
\setlength 18:84, 18:266, 18:281, 18:540, 23:4, 38:617, 38:622, 38:623, 38:624, 40:47, 40:62, 40:252, 40:271, 40:372, 40:375, 40:397, 40:400, 40:423, 40:426, 40:502, 40:609, 40:610, 40:611, 40:688, 40:689, 40:696, 40:697, 40:698, 41:200, 41:461, 54:3007, 54:3008, 54:3009, 54:3012, 54:3013, 54:3017, 54:3018, 54:3019, 54:3023, 54:3024, 54:3025, *1401*
\SetMathAlphabet 24:12, 27:140, 27:141, 28:767, 30:166, 30:167

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\setminus 30:404
\SetProperty 36:38, 36:291, *850*
\setrangecatcode 04:96, 04:104, 04:113, 04:114
\SetSymbolFont 28:602, 30:156, 30:157, 30:158, *1362*
\SetTemplateKeys 11:1620, *373*
\settodepth *569*
\settodepth 23:17
\settoheight *569*
\settoheight 23:17
\settowidth *569*
\settowidth 23:17
\sf *1365*
\sfcode 18:485, 20:45, 20:115, 20:173, 57:250, 57:594, 02:306, 02:307, 02:308, 02:309, 02:426
\sfdefault 29:10, 29:305, 29:478, 29:491, 29:519, 29:534, 29:564, 30:62, *740*
sffamily (hook) *244*
\sffamily . . 29:8, 29:9, 29:489, 29:532, 29:533, 29:562, 29:563, 32:16, *244*
sffamily 29:499
\sfsubstdefault 30:19, 30:31, 33:30, 33:51
\shape *1360*
\shapedefault 21:1602, 25:3204, 28:411, 28:461, 29:772, 29:805, 29:821, 29:837, 29:854, 30:131, *756*
\sharp 30:341
shipout (hook) *1186*, *1186*, *1187*, *1188*, *1188*
\shipout 53:65, 53:113, 53:484, 53:487, 54:882, 54:957, 54:1015, *1198*
shipout 53:214
shipout commands:
\l_shipout_box 53:84, 53:96, 53:99, 53:111, 53:122, 53:187, 53:210, 53:240, 53:245, 53:268, 53:269, 53:276, 53:287, 53:290, 53:291, 53:295, 53:302, 53:312, 53:320, 53:327, 53:337, 53:343, 53:344, 53:347, 53:354, 53:356, 53:357, 53:363, 53:365, *1201*
\l_shipout_box_dp_dim 53:255, 53:258, 53:260, 53:291, 53:344, 53:586, *1185*
\l_shipout_box_ht_dim 53:254, 53:258, 53:260, 53:290, 53:310, 53:343, 53:585, *1185*
\l_shipout_box_ht_plus_dp_dim 53:257, 53:260, 53:276, 53:327, 53:338, 53:340, *1185*
\l_shipout_box_wd_dim . . . 53:256, 53:260, 53:302, 53:354, 53:587, *1185*
\shipout_debug_off: 53:68, 53:74, 53:468, *1189*
\shipout_debug_on: 53:68, 53:69, 53:467, *1189*
\shipout_discard: 53:390, 53:390, 53:464, *1188*
\g_shipout_readonly_int 36:256, 53:163, 53:165, 53:172, 53:177, 53:393, 53:402, 53:411, 53:415, 53:419, 53:423, *1196*
\g_shipout_totalpage_int *1189*
\g_shipout_totalpages_int 53:148, 53:395, *1189*
shipout internal commands:
__shipout_add_background_box:n . 53:241, 53:267, 53:267, 53:385, 53:473
__shipout_add_background_picture:n 53:130, 53:384, 53:384, 53:477
__shipout_add_firstpage_material:Nn 53:234, 53:250, 53:250, 53:466, 53:471
__shipout_add_firstpage_specials: 53:171, 53:227, 53:239, 53:239, 53:242, *1197*
__shipout_add_foreground_box:n . 53:178, 53:318, 53:318, 53:388, 53:475
__shipout_add_foreground_picture:n 53:125, 53:387, 53:387, 53:479
__shipout_debug:n . . 53:68, 53:68, 53:81, 53:164, 53:176, 53:209, *1193*
\g__shipout_debug_bool 53:67, 53:71, 53:76, 53:82
__shipout_debug_gset: 53:68, 53:72, 53:77, 53:79
\g__shipout_discard_bool 53:142, 53:149, 53:151, 53:264, 53:391
__shipout_drop_firstpage_specials: 53:188, 53:228, 53:239, 53:244, 53:246, *1200*
__shipout_excuse_extra_page: 53:437, 53:445, 53:445
__shipout_execute: 53:107, 53:107, 53:113, *1195*
__shipout_execute_cont: 53:118, 53:120, 53:120
__shipout_execute_main_cont:Nnnn . 53:121, 53:138, 53:138, 53:208, *1198*
__shipout_execute_nohooks_cont: 53:205, 53:207
__shipout_execute_raw: 53:196, 53:196, 53:212, *1198*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

__shipout_execute_test_level: 53:110, 53:115, 53:115
__shipout_execute_test_level_-raw: 53:196, 53:199, 53:202
__shipout_finalize_box: 53:87, 53:89, 53:105, 53:185
\l__shipout_firstpage_box 53:231, 53:241, 53:248, *1200*
__shipout_force_immediate_-writes: 53:406, 53:521, 53:524, 53:526, 53:536, 53:538
__shipout_get_box_size:N 53:146, 53:168, 53:253, 53:253, 53:268, *1202*
\l__shipout_group_level_tl 53:108, 53:114, 53:117, 53:197, 53:204
\c__shipout_horigin_tl 53:25, 53:50, 53:371
__shipout_init_page_origins: 53:24, 53:49, 53:371, 53:371, 53:382
\g__shipout_lastpage_handled_-bool 53:181, 53:249, 53:417
__shipout_picture_overlay:n 53:4, 53:6, 53:23, 53:46, 53:48, 53:385, 53:388
__shipout_picture_overlay_-linkoff: 53:7, 53:19, 53:32
__shipout_picture_overlay_-linkon: 53:6, 53:18, 53:34
__shipout_picture_overlay_-linkstate: 53:8, 53:15, 53:29
\l__shipout_raw_box 53:86, 53:200, 53:208, 53:210, 53:240, 53:245, *1197*
__shipout_run_firstpage_hook: . . . 53:169, 53:224, 53:224, 53:233, *1200*
\l__shipout_saved_badness_tl 53:265, 53:271, 53:279, 53:292, 53:297, 53:305, 53:314, 53:322, 53:330, 53:342, 53:349, 53:359, 53:367
__shipout_saved_protect: 53:144, 53:193, 53:213, 53:213
\l__shipout_tmp_box 53:30, 53:36, 53:37, 53:38, 53:54, 53:56, 53:57, 53:58, 53:265, 53:278, 53:280, 53:281, 53:282, 53:286, 53:304, 53:306, 53:307, 53:308, 53:311, 53:329, 53:331, 53:332, 53:333, 53:339, 53:358, 53:360, 53:361, 53:362, 53:364
\c__shipout_vorigin_tl 53:27, 53:52, 53:371
shipout/... (hook) *1184*, *1188*
shipout/after (hook) *1184*, *1184*, *1185*, *1186*, *1186*, *1187*, *1189*, *1189*, *1198*
shipout/after 53:214, *1185*
shipout/afterpage (hook) *1201*
shipout/background (hook) *1185*, *1185*, *1187*, *1188*, *1190*, *1190*, *1191*, *1195*, *1197*, *1200*, *1200*
shipout/background 53:214, *1185*
shipout/before (hook) *354*, *1184*, *1184*, *1185*, *1185*, *1185*, *1186*, *1186*, *1186*, *1186*, *1186*, *1188*, *1188*, *1188*, *1188*, *1189*, *1189*, *1191*, *1191*, *1196*, *1197*
shipout/before 53:214, *1185*
shipout/firstpage (hook) . *1184*, *1185*, *1185*, *1186*, *1187*, *1187*, *1187*, *1197*, *1197*, *1200*, *1200*, *1201*, *1201*, *1212*
shipout/firstpage 53:214, *1185*
shipout/foreground (hook) *1185*, *1186*, *1187*, *1188*, *1190*, *1190*, *1191*, *1197*
shipout/foreground 53:214, *1185*
shipout/lastpage (hook) *1184*, *1185*, *1186*, *1187*, *1187*, *1187*, *1187*, *1196*, *1198*, *1198*, *1201*, *1206*, *1206*, *1207*, *1207*, *243*
shipout/lastpage 53:214, *1185*
\ShipoutBox . . . 53:84, 53:492, 53:568, *1184*
\ShipoutBoxDepth 53:512, 53:585
\ShipoutBoxHeight . . 53:511, 53:585, *1213*
\ShipoutBoxWidth 53:513, 53:585
\shortstack 42:143, 42:158, 42:163, 42:825, 42:842
\show 06:644, 06:717, *103*
show commands:
\show_hook:n *298*
\showbox 54:2492
\showboxbreadth 02:570, 54:2492, 02:262, 02:429, 02:505, 02:528, 02:545
\showboxdepth 02:571, 24:745, 24:790, 24:807, 54:2492, 02:263, 02:429, 02:504, 02:527, 02:544
\ShowCommand . 06:637, 07:1351, 07:1380, 07:1392, 07:1400, 07:1403, 07:1570, *107*
\ShowDocumentProperties 17:39, *458*
\ShowEnvironment 06:772, *1428*
\ShowFloat 54:2473
\ShowHook 08:2844, 08:2952, *234*
\showhyphens 24:733, *1410*
\ShowInstanceValues 11:1600, *379*
\ShowMarksAt 48:394
\showoutput 02:428, *1391*
\showoverfull 02:427, 02:430, 02:451, 02:486, 02:494, *1391*
\ShowSocket . . . 10:171, 10:192, 10:214, *360*
\showstream *1439*
\ShowTemplateCode 11:1600, *379*
\ShowTemplateDefaults 11:1600, *379*
\ShowTemplateInterface 11:1600, *379*
\ShowTemplateVariables 11:1600, *379*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\showtokens 06:752, *106*
\Sigma 30:314
\sigma 30:295
\sim 30:456, 30:468
\simeq 30:457
\sin 38:9
\sinh 38:11
\size *1358*
\skew 30:555
\skip 04:33, 40:527, 45:391, 02:28, 02:49, 54:321, 54:617, 54:744, 02:132, *1234*
skip commands:
\skip_eval:n 05:179
\skip_new:N 11:96
\skip_set:Nn 16:25
\skip_zero:N 16:58, 53:283, 53:284, 53:285, 53:334, 53:335, 53:336
\l_tmpa_skip *373*
\c_zero_skip 16:26
\skipdef 04:228, 02:45, 02:49
\skipeval *77*
\skipeval 05:172, 05:189, *77*
\skipzero 04:228
\slash 06:982, 06:1003, 02:382
\sldefault 25:3211, 29:25, 30:106
\SLiTeX *1380*
\sloppy 40:460, 40:479, 49:130, 49:135
sloppypar (env.) 49:135
\sloppypar 49:135
\slshape 21:461, 21:824, 25:3209, 25:3210, 29:23, 29:24, 32:22, 33:591
\small *222*
\smallbreak 06:983, 06:1004, 02:390
\smallint 30:368
\smallskip 18:469, 02:391
\smallskipamount . . 18:469, 18:472, 02:390
\smash 17:122, 17:147, 17:170, 30:484, 30:558, 30:559, 30:562, 30:563, 38:126, *892*
\smile 30:461
socket commands:
\socket_assign_plug:nn 10:42, 10:44, 10:122, 10:122, 10:196, 55:13, 55:16, *359*
\socket_debug_off: 10:7, 10:14, 10:199, *361*
\socket_debug_on: 10:7, 10:9, 10:198, *361*
\socket_get_plug:nN 10:144, 10:144, *361*
\socket_if_exist:n 10:53
\socket_if_exist:nTF 10:28, 10:53, 10:60, 10:79, 10:99, 10:123, 10:146, 10:200, 10:201, 10:202, *360*
\socket_if_exist_p:n 10:53
\socket_if_plug_assigned:nn . 10:138
\socket_if_plug_assigned:nnTF 10:138, 10:206, 10:207, 10:208, *360*
\socket_if_plug_assigned_p:nn 10:138
\socket_if_plug_exist:nn 10:117
\socket_if_plug_exist:nnTF 10:81, 10:101, 10:117, 10:125, 10:203, 10:204, 10:205, *360*
\socket_if_plug_exist_p:nn . . 10:117
\socket_log:n 10:58, 10:58, 10:77, 10:194, *360*
\socket_new:nn 10:27, 10:27, 10:192, 55:8, *359*
\socket_new_plug:nnn 10:38, 10:41, 10:78, 10:78, 10:195, 55:14, *359*
\socket_set_plug:nnn 10:78, 10:98, 10:185, *359*
\socket_show:n 10:58, 10:77, 10:193, *360*
\socket_use:n 10:154, 10:160, *360*
\socket_use:nn . . . 10:154, 10:161, *365*
\socket_use:nnn . . 10:154, 10:162, *360*
\socket_use:nnnn . . 10:154, 10:163, *360*
\socket_use:nw 10:154, 10:154, 10:160, 10:161, 10:162, 10:163, 10:197, *360*
\socket_use_expandable:n 10:164, 10:167, *1434*
\socket_use_expandable:nw 10:164, 10:164, 10:167, *360*
socket internal commands:
__socket_debug:n 10:7, 10:7, 10:21, *362*
\g__socket_debug_bool 10:6, 10:11, 10:16, 10:22, 10:24
__socket_debug_gset: 10:7, 10:12, 10:17, 10:19
__socket_debug_term:n 10:7, 10:8, 10:23, 10:45, 10:92, 10:108, 10:127, 10:155, *362*
Sockets:
build/column/baselineattach 54:723, *1233*, *1234*, *1246*, *1247*, *1250*, *1250*
build/column/footins *1234*
build/column/footnotes . . 54:722, *1234*
build/column/outputbox 54:663, *1233*, *1234*, *1246*
build/page/footer *1234*
build/page/header *1234*
foo *356*, *356*
refstepcounter 35:134
refstepcounter/target 35:135
\sourceLaTeXdate 08:2966, 03:164, 50:82, 50:120
\sp 38:212
\space 02:322, *1379*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

space:floats:footnotes (plug) 54:688, *1233*
space:footnotes:floats (plug) 54:664, *1233*
\spacefactor 17:121, 17:123, 17:146, 17:148, 17:169, 17:171, 18:130, 18:139, 18:162, 18:180, 18:194, 18:206, 18:220, 18:234, 18:485, 18:526, 18:531, 21:88, 21:91, 45:593, 45:595, 02:380, 02:381, *1401*
\spacefactor␣in␣math␣mode␣gh/643 . *1424*
\spaceskip 33:6, 33:576, 02:296
\spadesuit 30:345
\span 41:460
\special 17:63, 17:144, 17:167, *460*
\SplitArgument 07:3316, *123*
\splitfirstmark 54:2915
\SplitList 07:3316, *123*
\splitmaxdepth 45:531, 45:550, 45:568, 54:2909, 02:269
\splittopskip 45:530, 45:549, 45:567, 02:294
\sqcap 30:387
\sqcup 30:388
\sqrt 38:413, *1389*
\sqrtsign 30:540, 38:71, 38:413, *1389*
\sqsubset 29:885
\sqsubseteq 30:410
\sqsupset 29:886
\sqsupseteq 30:411
\SS . . 21:324, 21:557, 21:1185, 57:720, *1394*
\ss 21:274, 21:427, 21:580, 21:808, 21:1160, 57:720
\sscdefault 25:2924, 25:2940, 25:3226
\sscshape 25:2924, 25:2939, 25:3224, 25:3225, 32:31, *664*
\stackrel 38:411
\stamp *1368*
\star 30:408
\stepcounter *556*
\stepcounter 22:34, 22:44, 24:698, 28:49, 35:143, 35:156, 35:166, 35:177, 35:186, 35:193, 38:425, 38:454, 38:516, 38:610, 45:514, 45:582, 54:943, 54:1001, 54:1060, *1400*
\stockheight 54:71
\stockwidth 54:71
\stop 33:1600, 37:242
\storedpar 04:156, 04:161
str commands:
\c_backslash_str 09:163, 09:432, 07:1639, 07:1646, 07:1662, 07:1666, 07:1717, 07:2841
\c_hash_str 09:515, 09:519, 09:543, 07:1424, 07:1556, 08:568, 08:574
\c_right_brace_str 08:2088
\str_case:nn 09:629, 11:666, 57:611, 07:1159
\str_case:nnTF 09:56, 36:29, 51:30, 07:3088
\str_case_e:nnTF 07:2732
\str_count:n 08:1885, 09:177, 09:252, 07:2824, 08:37, 08:37, 08:1271
\str_gset:Nn 08:2829
\str_head:n 07:2677
\str_if_empty:NTF 57:637
\str_if_eq:nn *287*
\str_if_eq:NNTF 52:555
\str_if_eq:nnTF . 08:1823, 08:1941, 08:2041, 08:2148, 08:2360, 08:2564, 08:2618, 08:2688, 09:231, 09:303, 09:402, 09:554, 09:561, 09:590, 09:593, 10:140, 11:162, 11:175, 11:334, 11:355, 11:720, 11:776, 11:1220, 05:151, 52:131, 52:549, 52:559, 07:104, 07:106, 07:428, 07:1238, 07:1243, 07:1738, 07:1792, 07:1795, 07:1798, 07:1801, 07:1860, 07:2279, 07:2290, 07:2292, 07:2320, 07:2338, 07:2340, 07:2357, 07:2384, 07:2410, 07:2412, 07:2433, 07:2435, 07:2456, 07:2573, 07:2585, 07:2613, 07:2789, 08:340, 08:438, 08:484, 08:541, 08:543, 08:630, 08:632, 08:762, 08:820, 08:853, 08:906, 08:988, 08:995, 08:1023, 08:1030, *191*
\str_if_eq_p:nn 08:1614, 08:1700, 11:568, 11:569, 11:1036, 11:1037, 11:1038, 07:489, 07:490, 07:491, 07:492, 07:768, 07:964, 07:969, 07:2621, 07:2643, 08:910
\str_if_exist:NTF 10:54
\str_if_in:nnTF 11:532
\str_lowercase:n 57:649
\str_map_function:NN 05:158
\str_new:N 10:35, 11:85, 07:22
\str_replace_all:Nnn 09:542
\str_set:Nn 10:130, 11:815, 52:399, 52:400, 07:232, 07:246, 07:247, 07:291, 57:627, 57:635, 57:639
\str_tail:n 07:2279, 07:2674, 07:2700, *190*
\str_use:N 10:67, 10:70, 10:128, 10:157, 10:158, 10:165
str internal commands:
__str_if_eq:nn 08:23, *246*
\strcmp 50:381, 50:397
\stretch 18:554
\string *1173*
\stripmeaning *1365*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\strongem ... 29:684, 29:685, 29:705, 32:43
\strongemforce ... *756*
\strongemforce ... 29:681, 29:702, *756*
\strongemph ... *756*
\strongemph ... 29:705, 32:43, *756*
\strongemreset ... *756*
\strongemreset 29:681, 29:706, 57:786, *756*
struct:mc (tag socket) ... 55:62
struct/begin (tag socket) ... 55:72
struct/end (tag socket) ... 55:72
\strut ... 38:191, 38:192, 41:29, 06:984, 06:1005, 02:398, *245*
\strutbox ... 26:190, 40:553, 40:570, 40:587, 41:238, 41:239, 41:261, 41:262, 45:531, 45:539, 45:550, 45:557, 45:568, 45:575, 02:398
\subencodingresult .. 33:1200, 33:1205, 33:1374, 33:1378, 33:1380, 33:1381
\subparagraphmark ... 44:143
\subsectionmark ... 44:143, *1064*
\subset ... 30:437
\subseteq ... 30:439
\substdefault ... *1414*
\subsubsectionmark ... 44:143
\succ ... 30:431
\succeq ... 30:434
\sum ... 30:361
\sup ... 38:24
\supereject ... *1393*
\suppressfloats ... 54:2583
\supset ... 30:436
\supseteq ... 30:438
\surd ... 30:347
\SuspendTagging ... *1309*
\SuspendTagging ... 23:22, 55:3, *1310*
\swarrow ... 30:419
\swdefault 25:2919, 25:2938, 25:3223
\swshape ... 25:2919, 25:2937, 25:3221, 25:3222, 32:30
\symbol ... 21:180, 29:730
\symletters ... 33:8, 33:14, 33:578, 33:1100
\symoperators ... 30:654
sys commands:
\c_sys_day_int ... 11:1578
\sys_if_engine_luatex:TF ... 09:381, 53:13, 53:87
\c_sys_month_int ... 11:1577
\c_sys_year_int ... 11:1576

T

\T ... 14:23, 21:355, 21:357, 21:359, 21:361, 21:363, 21:365, 21:367, 21:369, 21:371, 21:394, 50:1615, 50:1619, 50:1620
\t ... 21:302, 21:765, 21:873, 33:121, 33:122, 33:149, 33:153, 33:155, 33:167, 33:170, 33:172, 33:652, 33:820, 33:1087, 33:1089, 33:1282, *1382*
tabbing (env.) ... 41:71
\tabbing ... 41:71
\tabbingsep ... 41:130, 41:132, 41:166
\tabcolsep ... 41:336, 41:415
\tableofcontents ... *870*
\tabskip ... 38:208, 38:209, 38:431, 38:434, 38:437, 38:439, 38:460, 38:463, 38:466, 38:468, 38:615, 38:628, 38:631, 38:633, 41:167, 41:244, 41:267, 02:295, 02:409, *1362*
tabular (env.) ... 41:198
\tabular ... 41:198
\tabular* ... 41:199
\tabularnewline 41:246, 41:269, 41:284, *1386*
tag commands:
\tag_if_active: ... 55:26
\tag_if_active:TF ... 55:357, *1434*
\tag_mc_begin:n ... 55:57, 55:66, 55:89, 55:157
\tag_mc_begin_pop:n ... 55:97
\tag_mc_end: 55:59, 55:68, 55:95, 55:164
\tag_mc_end_push: ... 55:87
\c_tag_para_semantic_tl ... 55:110
\c_tag_para_textblock_tl ... 55:111
\tag_resume:n ... 55:3, 55:4, 55:6
\tag_socket_use:n .. 55:26, 55:29, *1310*
\tag_socket_use:nn . 55:26, 55:30, *1310*
\tag_socket_use:nnn 55:26, 55:31, *1310*
\tag_socket_use_expandable:n ... 55:26, 55:28, *1310*
\tag_struct_begin:n ... 55:65, 55:76, 55:88, 55:130, 55:151
\tag_struct_end: ... 55:69, 55:80, 55:96, 55:141, 55:166
\tag_struct_recordtarget: ... 55:210
\tag_suspend:n ... 55:3, 55:3, 55:5
tag internal commands:
\l__tag_block_flattened_level_int ... 55:100, 55:113
__tag_check_para_begin_show:nn ... 55:156
__tag_check_para_end_show:nn 55:165
__tag_check_typeout_v:n ... 55:150, 55:163, 55:176
__tag_gincr_para_begin_int: 55:149
__tag_gincr_para_end_int: .. 55:162
__tag_gincr_para_main_begin_int: ... 55:129
__tag_gincr_para_main_end_int: ... 55:140

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

__tag_lastpage_label: 53:407
\l__tag_para_attr_class_tl 55:126, 55:154
\l__tag_para_bool 55:102, 55:104, 55:114, 55:172, 55:192
\l__tag_para_flattened_bool 55:112, 55:180, 55:195
\l__tag_para_main_attr_class_tl 55:133
\tag_resume:n *1309*
\tag_socket_use:n *1309*
\tag_socket_use:nn *1309*
\tag_socket_use:nnn *1309*
\tag_socket_use_expandable:n *1310*
\tag_suspend:n *1309*
Tagging Plugs:
default 55:108
kernel (para/begin) 55:170
kernel (para/end) 55:190
kernel (para/semantic/begin) 55:126
kernel (para/semantic/end) . . 55:137
kernel (para/textblock/begin) 55:147
kernel (para/textblock/end) . 55:159
kernel (recordtarget) 55:207
Tagging Sockets:
block/list/label 55:218
block/recipe 55:219
build/column/footins 55:269
build/column/outputbox 55:268
build/page/footer 55:266
build/page/header 55:266
caption/begin 55:262
caption/end 55:262
caption/label/begin 55:264
caption/label/end 55:264
float/begin 55:260
float/end 55:260
float/hmode/begin 55:258
float/hmode/end 55:258
inline/begin 55:84
inline/end 55:84
marginpar/begin 55:235
marginpar/end 55:235
math/luamml/annotate/false . . 55:283
math/luamml/array/finalize . . 55:285
math/luamml/array/finalizecol 55:287
math/luamml/array/initcol . . . 55:286
math/luamml/array/save 55:284
math/luamml/artifact 55:299
math/luamml/finph@nt . 55:297, 55:298
math/luamml/hbox 55:296
math/luamml/mtable/aligncol . 55:290
math/luamml/mtable/finalize . 55:289
math/luamml/mtable/finalizecol 55:288
math/luamml/mtable/innertable/finalize 55:293
math/luamml/mtable/innertable/save 55:291
math/luamml/mtable/smallmatrix/save 55:292
math/luamml/mtable/tag/save . 55:294
math/luamml/mtable/tag/set . . 55:295
math/luamml/save/nn 55:281
math/luamml/save/nNn 55:282
mc 55:54
page@sofar 55:270
para/begin 55:117
para/end 55:117
para/off 55:101
para/on 55:101
para/restore 55:107
para/semantic/begin 55:124
para/semantic/end 55:124
para/textblock/begin 55:145
para/textblock/end 55:145
refstepcounter 55:206
sec/begin 55:220
sec/end 55:220
sec/title/begin 55:222
sec/title/end 55:222
sec/title/hang 55:224
sec/title/init 55:225
sec/title/number 55:228
sec/title/runin/number 55:225
sec/title/split 55:225
struct:mc 55:62
struct/begin 55:72
struct/end 55:72
tbl/cell/begin 55:237
tbl/cell/end 55:237
tbl/colspan 55:247
tbl/finalize 55:245
tbl/hmode/begin 55:248
tbl/hmode/end 55:248
tbl/init 55:243
tbl/init/celldata 55:244
tbl/leaders/begin 55:256
tbl/leaders/end 55:256
tbl/longtable/finalize 55:252
tbl/longtable/foot 55:254
tbl/longtable/head 55:254
tbl/longtable/init 55:252
tbl/pcell/end 55:237, 55:237
tbl/restore/celldata 55:246
tbl/row/begin 55:237
tbl/row/end 55:237
tbl/vmode/begin 55:248
tbl/vmode/end 55:248

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

toc/contentsline/after 55:229
toc/contentsline/before 55:229
toc/leaders/after 55:233
toc/leaders/before 55:233
toc/starttoc/after 55:231
toc/starttoc/before 55:231
tagpdf (plug) *359*
\tan 38:15
\tanh 38:17
target 36:262, *854*
\tau 30:296
tbl commands:
\tbl_count_missing_cells:n 55:356, 55:356, 55:455, *1328*
\tbl_count_table_cols: 55:350, 55:350, *1325*
\tbl_crcr:n 55:452, 55:452
\tbl_gdecr_row_count: 55:412, 55:418
\tbl_gincr_row_count: 55:412, 55:415
\tbl_gzero_row_count: 55:412, 55:412
\tbl_if_row_was_started:TF 55:406, 55:406, 55:409
\tbl_inbetween_rows: . 55:421, 55:421
\tbl_init_cell_data_for_row: 55:402, 55:402
\tbl_init_cell_data_for_table . *1319*
\tbl_init_cell_data_for_table: 55:377, 55:377
\tbl_restore_outer_cell_data: 55:424, 55:424, *1319*
\tbl_update_cell_data: 55:346, 55:346
\tbl_update_cell_data_for_next_row: 55:398, 55:398
\tbl_update_multicolumn_cell_data:n 55:441, 55:441
tbl internal commands:
\g__tbl_col_int 55:322, 55:347, 55:358, 55:363, 55:378, 55:395, 55:400, 55:403, 55:407, 55:410, 55:422, 55:425, 55:432, 55:442, 55:445, 55:448, 55:453, *1328*
\g__tbl_missing_cells_int 55:338, 55:360, 55:367, 55:371
\g__tbl_missingcells_int 55:338
\g__tbl_row_int 55:322, 55:379, 55:394, 55:399, 55:413, 55:416, 55:419, 55:426, 55:431, *1324*
\l__tbl_saved_col_tl 55:329, 55:378, 55:385, 55:425
\l__tbl_saved_row_tl 55:329, 55:379, 55:384, 55:426
\l__tbl_saved_span_tl 55:329, 55:381, 55:386, 55:427, 55:433
\l__tbl_saved_table_cols_tl 55:329, 55:380, 55:388, 55:390, 55:428
\g__tbl_span_tl . . 55:322, 55:347, 55:348, 55:364, 55:381, 55:396, 55:404, 55:427, 55:448, 55:450, *1326*
\g__tbl_table_cols_tl 55:322, 55:352, 55:353, 55:362, 55:380, 55:428, 55:435, 55:437, *1325*
\l__tbl_tmpa_seq 55:351, 55:352, 55:355
__tbl_trace:n . . . 55:340, 55:343, 55:345, 55:353, 55:368, 55:383, 55:430
tbl/cell/begin (hook) 41:469
tbl/cell/begin (tag socket) 55:237
tbl/cell/end (hook) 41:469
tbl/cell/end (tag socket) 55:237
tbl/cell/init (hook) 41:468
tbl/colspan (tag socket) 55:247
tbl/finalize (tag socket) 55:245
tbl/hmode/begin (tag socket) 55:248
tbl/hmode/end (tag socket) 55:248
tbl/init (hook) 41:465
tbl/init (tag socket) 55:243
tbl/init/celldata (tag socket) . . . 55:244
tbl/leaders/begin (tag socket) . . . 55:256
tbl/leaders/end (tag socket) 55:256
tbl/longtable/finalize (tag socket) 55:252
tbl/longtable/foot (tag socket) . . 55:254
tbl/longtable/head (tag socket) . . 55:254
tbl/longtable/init (tag socket) . . 55:252
tbl/pcell/end (tag socket) . 55:237, 55:237
tbl/restore/celldata (tag socket) . 55:246
tbl/row/begin (hook) 41:467
tbl/row/begin (tag socket) 55:237
tbl/row/end (hook) 41:467
tbl/row/end (tag socket) 55:237
tbl/row/init (hook) 41:466
tbl/vmode/begin (tag socket) 55:248
tbl/vmode/end (tag socket) 55:248
template internal commands:
__template_assign_boolean: 11:1081, 11:1081
__template_assign_boolean_aux:n . . 11:1081, 11:1084, 11:1085, 11:1087
__template_assign_choice: 11:1107, 11:1107
__template_assign_choice_aux:nTF 11:1107, 11:1109, 11:1112, 11:1124, 11:1129
__template_assign_function: 11:1130, 11:1130
__template_assign_function_aux:N 11:1130, 11:1133, 11:1134, 11:1136

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

__template_assign_instance: 11:1147, 11:1147
__template_assign_instance_-aux:N 11:1147, 11:1150, 11:1151, 11:1153
__template_assign_variable: 11:1042, 11:1165, 11:1165
__template_assign_variable:n 11:1165, 11:1167, 11:1175
__template_assignments_pop: 11:1236, 11:1236, 11:1619
__template_assignments_push:n 11:978, 11:1237, 11:1237
\l__template_assignments_tl 11:76, 11:979, 11:1017, 11:1092, 11:1100, 11:1127, 11:1138, 11:1155, 11:1180, 11:1187, 11:1236, 11:1238, 11:1261, *382*
\c__template_code_root_tl 11:48, 11:130, 11:551, 11:554, 11:949, 11:951, 11:952, 11:980, 11:1078, 11:1265, *381*
__template_convert_to_assignments:N 11:971, 11:1015, 11:1015, 11:1073, 11:1260
__template_convert_to_assignments_-aux:n 11:1015, 11:1019, 11:1021
__template_convert_to_assignments_-aux:nn 11:1015, 11:1024, 11:1026, 11:1047
\l__template_current_template_tl . 11:99, 11:977, 11:1077, 11:1628, *383*
\l__template_current_type_tl 11:98, 11:976, 11:1076, 11:1627, *383*
__template_date_compare:nNn . . 11:9
__template_date_compare:nNnTF 11:9, 11:1574
__template_date_compare_aux:nNn 11:9, 11:10, 11:11
__template_date_compare_-auxi:nnnNnnn 11:9, 11:15, 11:21
__template_date_compare_auxii:w 11:9, 11:16, 11:18, 11:40
__template_date_compare_-auxiii:w 11:9, 11:43, 11:46
__template_debug:n 11:107, 11:113, 11:116, 11:116
__template_debug_off: 11:105, 11:111, 11:1633
__template_debug_on: 11:105, 11:105, 11:1632
__template_debug_typeout:n 11:108, 11:114, 11:116, 11:117, 11:1074, 11:1232
__template_declare_instance:nnnn 11:954, 11:954, 11:1562
__template_declare_instance_-aux:nnnn 11:954, 11:960, 11:963, 11:1012
__template_declare_template_-code:nnnn 11:523, 11:535, 11:537, 11:542, 11:549
__template_declare_template_-code:nnnnn . 11:523, 11:523, 11:1554
__template_declare_template_-equiv:nnnn 11:443, 11:443, 11:1572, 11:1583, 11:1591
__template_declare_template_-keys:nnnn . . 11:296, 11:296, 11:1552
__template_declare_type:nn 11:275, 11:279, 11:281
\l__template_default_tl . . 11:77, *382*
\c__template_defaults_root_tl 11:49, 11:182, 11:183, 11:225, 11:228, *381*
__template_define_type:nn 11:275, 11:275, 11:1550
__template_edit_defaults:nnn 11:751, 11:751, 11:1558
__template_edit_instance:nnn 11:997, 11:997, 11:1566
__template_edit_instance_-aux:nnnn . 11:1004, 11:1009, 11:1014
__template_edit_instance_-aux:nnnnn 11:997
\l__template_error_bool 11:78, 11:317, 11:323, 11:339, 11:368, 11:381, 11:465, 11:781, 11:965, 11:967, *382*
__template_execute_if_arg_-agree:nnTF 11:118, 11:118, 11:300, 11:527
__template_execute_if_code_-exist:nnTF 11:128, 11:128, 11:956, 11:1068, 11:1286
__template_execute_if_keyname_-free:nTF 11:147, 11:147, 11:153, 11:325
__template_execute_if_keys_-exist:nnTF 11:154
__template_execute_if_keytype_-exist:nTF 11:134, 11:134, 11:140, 11:319
__template_execute_if_type_-exist:nTF 11:141, 11:141, 11:298, 11:445, 11:525
__template_find_global: 11:1040, 11:1048, 11:1048

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

__template_find_global_aux:w 11:1048, 11:1053, 11:1056
\l__template_global_bool 11:79, 11:1050, 11:1059, 11:1083, 11:1132, 11:1149, 11:1171, *382*
__template_if_instance_exist:nn 11:167
__template_if_instance_exist:nnTF 11:167, 11:986, 11:999, 11:1063, 11:1226, 11:1303, 11:1611, 11:1613, 11:1615
__template_if_key_value:n 11:160, 11:166
__template_if_key_value:nTF 11:160, 11:1089, 11:1177
__template_if_keys_exist:nnTF 11:154, 11:447, 11:529, 11:753, 11:1268, 11:1277
__template_if_use_template:n 11:173
__template_if_use_template:nTF 11:173, 11:1214
__template_implement_choice_-elt:n 11:683, 11:716, 11:746
__template_implement_choice_-elt:nnn 11:684, 11:716, 11:716
__template_implement_choice_-elt_aux:n 11:716, 11:722, 11:730, 11:733
__template_implement_choice_-elt_aux:nnn 11:716, 11:721, 11:728, 11:740
__template_implement_choices:nn 11:651, 11:678, 11:678
__template_implement_choices_-default: . . . 11:678, 11:688, 11:695
__template_instance_set_eq:nnn 11:984, 11:984, 11:1564
__template_instance_value:nnn 11:1061, 11:1061, 11:1618
\c__template_instances_root_tl 11:50, 11:169, 11:972, 11:974, 11:990, 11:992, 11:993, 11:1234, *381*
\l__template_key_name_tl 11:80, 11:326, 11:352, 11:354, 11:364, 11:375, 11:390, 11:397, 11:402, 11:468, 11:474, 11:513, 11:515, 11:516, 11:519, 11:520, 11:580, 11:583, 11:588, 11:645, 11:657, 11:662, 11:681, 11:686, 11:698, 11:702, 11:705, 11:708, 11:711, 11:712, 11:735, 11:738, 11:743, 11:749, 11:797, 11:814, 11:823, 11:834, 11:839, 11:841, 11:842, 11:846, 11:847, 11:854, 11:855, 11:857, 11:862, 11:868, 11:869, 11:878, 11:886, 11:887, 11:894, 11:895, 11:922, 11:928, 11:933, 11:1032, 11:1110, 11:1113, 11:1115, 11:1119, *400*
\c__template_key_order_root_tl 11:52, 11:198, 11:199, 11:241, 11:244, *381*
\l__template_key_order_seq 11:87, 11:149, 11:200, 11:243, 11:246, 11:305, 11:354, 11:505, 11:515, 11:971, 11:1073, *382*
__template_key_to_value: 11:1091, 11:1179, 11:1194, 11:1194
__template_key_to_value_auxi:w 11:1194, 11:1195, 11:1196
__template_key_to_value_auxii:w 11:1194, 11:1202, 11:1207
__template_keys_set:nnn 11:1239, 11:1239, 11:1623
__template_keys_set_known:nnn 11:1239, 11:1241, 11:1248, 11:1621, 11:1626
__template_keys_set_known_-aux:nnn . . 11:1239, 11:1249, 11:1250
\l__template_keytype_arg_tl 11:82, 11:336, 11:349, 11:350, 11:357, 11:414, 11:428, 11:680, 11:1143, 11:1160, *382*
\l__template_keytype_tl . . . 11:81, 11:319, 11:334, 11:348, 11:355, 11:413, 11:424, 11:484, 11:487, 11:514, 11:584, 11:586, 11:613, 11:615, 11:666, 11:821, 11:826, 11:863, 11:898, 11:911, 11:914, 11:1036, 11:1037, 11:1038, 11:1041, *382*
\c__template_keytypes_arg_seq 11:73, 11:321, 11:437, *381*
\l__template_keytypes_prop 11:86, 11:197, 11:237, 11:240, 11:303, 11:352, 11:467, 11:513, 11:565, 11:582, 11:588, 11:705, 11:735, 11:775, 11:797, 11:842, 11:862, 11:1023, 11:1115, 11:1280, *382*
\c__template_keytypes_root_tl 11:51, 11:156, 11:190, 11:193, 11:195, 11:196, 11:235, 11:238, *381*
\c__template_keytypes_seq 11:56, 11:136, 11:470, *381*
__template_map_var_type: 11:643, 11:664, 11:664, 11:918, 11:1169
\l__template_msg_prop 11:91, 11:216, 11:263, 11:266, 11:519, 11:823, *382*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\c__template_msg_root_tl . . 11:53, 11:214, 11:215, 11:261, 11:264, *381*
__template_parse_equiv_alias:nnn 11:492
__template_parse_equiv_alias:nnnn 11:462
__template_parse_equiv_auxi:nnnn 11:462, 11:493, 11:495, 11:496
__template_parse_equiv_auxii:n 11:462, 11:499, 11:500, 11:503
__template_parse_equiv_auxiii:nnnn 11:462, 11:501, 11:511
__template_parse_equiv_elt:n 11:452, 11:460, 11:460
__template_parse_equiv_elt:nnnn 11:453, 11:462, 11:462
__template_parse_equiv_elt_-aux:nnn 11:462, 11:476, 11:478, 11:481
__template_parse_equiv_meta:nnn 11:494
__template_parse_equiv_meta:nnnn 11:462
__template_parse_keys_elt:n 11:307, 11:314, 11:314, 11:363, *390*
__template_parse_keys_elt:nn 11:307, 11:361, 11:361
__template_parse_keys_elt_aux: 11:314, 11:327, 11:344
__template_parse_keys_elt_aux:n 11:314, 11:322, 11:332
__template_parse_known_values:nnn 11:760, 11:762, 11:1259
__template_parse_unknown_-error:nn . . . 11:761, 11:935, 11:937
__template_parse_unknown_-store:nn . . . 11:763, 11:938, 11:940
__template_parse_values:nnn . . . 11:756, 11:760, 11:760, 11:966, 11:1072
__template_parse_values:NNnnn 11:760, 11:761, 11:763, 11:764
__template_parse_values_add:N 11:787, 11:901, 11:902
__template_parse_values_add:n 11:787, 11:892, 11:900, 11:901
__template_parse_values_add_-commalist:n 11:925
__template_parse_values_add_-tokenlist:n 11:930
__template_parse_values_-alias:nn 11:787, 11:852
__template_parse_values_alias_-aux:nn 11:787, 11:858, 11:860, 11:881
__template_parse_values_elt:NNn 11:769, 11:773, 11:773
__template_parse_values_elt:Nnn 11:770, 11:777, 11:787, 11:787
__template_parse_values_elt_-auxi:n 11:787
__template_parse_values_elt_-auxi:w 11:787, 11:791, 11:810
__template_parse_values_elt_-auxii:nn 11:799, 11:817, 11:844, 11:864
__template_parse_values_elt_-auxiii:Nn . . 11:787, 11:800, 11:831
__template_parse_values_elt_-auxiv:w 11:787, 11:833, 11:836
__template_parse_values_meta:nn 11:787, 11:866
__template_parse_values_meta_-elt:nnnn . . . 11:787, 11:873, 11:876
__template_parse_values_set:N 11:787, 11:890, 11:891
__template_parse_values_set:n 11:787, 11:884, 11:889, 11:890
__template_parse_vars_choice:nnn 11:650, 11:650
__template_parse_vars_elt:n 11:563, 11:576, 11:576
__template_parse_vars_elt:nnn 11:563, 11:578, 11:578
__template_parse_vars_elt_-auxi:nn 11:578, 11:587, 11:592
__template_parse_vars_elt_-auxii:nw . . . 11:578, 11:594, 11:596
__template_parse_vars_elt_-auxiii:nnn 11:578, 11:600, 11:604, 11:611, 11:623
__template_parse_vars_elt_-auxiv:nnw . . 11:578, 11:619, 11:624
__template_parse_vars_elt_-auxv:nnn 11:578, 11:628, 11:632, 11:638, 11:649
__template_parse_vars_function:nnn 11:650, 11:652
__template_parse_vars_instance:nnn 11:650, 11:659
__template_quark_if_nil:N . . 11:104
__template_quark_if_nil:NTF 11:1209
__template_quark_if_nil:nTF 11:104
__template_quark_if_nil_p:n 11:104
__template_recover_defaults:nn 11:222, 11:222, 11:559, 11:755, 11:943, 11:958, 11:1070, 11:1270
__template_recover_keytypes:nn 11:222, 11:232, 11:449, 11:560, 11:766, 11:945, 11:1279

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

__template_recover_values:nn 11:222, 11:248, 11:988, 11:1001, 11:1305
__template_recover_vars:nn 11:222, 11:258, 11:450, 11:561, 11:947, 11:959, 11:1011, 11:1071, 11:1258, 11:1288
\c__template_restrict_root_tl . . *381*
__template_show:Nnnn . . 11:1266, 11:1271, 11:1280, 11:1289, 11:1293
__template_show_code:nn 11:1264, 11:1264, 11:1601
__template_show_defaults:nn 11:1266, 11:1266, 11:1603
__template_show_keytypes:nn 11:1266, 11:1275, 11:1605
__template_show_values:nn 11:1301, 11:1301, 11:1609
__template_show_vars:nn 11:1266, 11:1284, 11:1607
__template_split_keytype:n 11:316, 11:366, 11:366, 11:464
__template_split_keytype_arg:n 11:407, 11:411, 11:411, 11:440, 11:586, 11:707, 11:737, 11:819, 11:1033, 11:1117
__template_split_keytype_arg_-aux:n . 11:411, 11:415, 11:438, 11:441
__template_split_keytype_arg_-aux:w . 11:411, 11:419, 11:434, 11:442
__template_split_keytype_aux:w 11:366, 11:376, 11:387, 11:399
__template_store_defaults:nn . . . 11:179, 11:179, 11:308, 11:757, 11:944
__template_store_key_implementation:nnn 11:531, 11:557, 11:557
__template_store_keytypes:nn . . . 11:179, 11:187, 11:309, 11:455, 11:946
__template_store_values:nn 11:179, 11:203, 11:970, 11:989
__template_store_vars:nn 11:179, 11:211, 11:310, 11:456, 11:564, 11:948
__template_template_set_eq:nnn 11:941, 11:941, 11:1556
__template_tmp:n 11:100, 11:100, 11:879, 11:880
\l__template_tmp_clist 11:92, 11:680, 11:689, 11:691, 11:718, 11:725, 11:727, *383*
\l__template_tmp_dim 11:93, *383*
\l__template_tmp_int 11:94, 11:283, 11:284, 11:287, 11:289, 11:293, *383*
\l__template_tmp_muskip . . 11:95, *383*
\l__template_tmp_skip . . . 11:96, *383*
\l__template_tmp_tl 11:97, 11:120, 11:121, 11:125, 11:346, 11:353, 11:369, 11:370, 11:371, 11:377, 11:468, 11:471, 11:687, 11:697, 11:698, 11:699, 11:701, 11:702, 11:703, 11:706, 11:707, 11:709, 11:736, 11:737, 11:742, 11:744, 11:775, 11:776, 11:798, 11:819, 11:824, 11:825, 11:843, 11:856, 11:857, 11:870, 11:874, 11:880, 11:882, 11:896, 11:897, 11:916, 11:919, 11:922, 11:927, 11:928, 11:932, 11:933, 11:1003, 11:1005, 11:1023, 11:1024, 11:1116, 11:1117, 11:1126, 11:1127, 11:1198, 11:1199, 11:1205, 11:1257, 11:1259, *383*
\g__template_type_prop 11:75, 11:120, 11:143, 11:277, 11:288, *381*
__template_use_instance:nn 11:1159, 11:1212, 11:1212, 11:1568
__template_use_instance_-aux:nNnnn 11:1212, 11:1215, 11:1218
__template_use_instance_auxi:nn 11:1212, 11:1216, 11:1224
__template_use_instance_-auxii:nn . 11:1212, 11:1227, 11:1230
__template_use_template:nnn 11:1066, 11:1066, 11:1221, 11:1560
\l__template_value_exp_str 11:85, 11:795, 11:805, 11:815, 11:828, *382*
\l__template_value_order_seq 11:89, 11:767, 11:854, 11:868, 11:886, 11:894, 11:1260, *382*
\l__template_value_tl 11:83, 11:1028, 11:1089, 11:1096, 11:1102, 11:1110, 11:1120, 11:1144, 11:1161, 11:1177, 11:1183, 11:1190, 11:1195, 11:1200, 11:1202, 11:1210, *382*
\l__template_values_prop . . 11:88, 11:184, 11:208, 11:227, 11:230, 11:253, 11:256, 11:302, 11:364, 11:686, 11:708, 11:887, 11:895, 11:921, 11:928, 11:933, 11:969, 11:1002, 11:1028, 11:1271, 11:1310, *382*
\c__template_values_root_tl 11:54, 11:206, 11:207, 11:251, 11:254, 11:1064, *381*
\l__template_var_tl 11:84, 11:1030, 11:1051, 11:1053, 11:1058, 11:1095, 11:1103, 11:1141, 11:1157, 11:1182, 11:1189, *382*
\l__template_vars_prop 11:90, 11:219, 11:270, 11:273, 11:304, 11:516, 11:644,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

11:656, 11:661, 11:681, 11:699, 11:703, 11:744, 11:855, 11:869, 11:1030, 11:1126, 11:1199, 11:1289, *382*
\c__template_vars_root_tl . 11:55, 11:217, 11:218, 11:268, 11:271, *381*
\tencirc 31:10, 42:136, 42:688
\tencircw 31:10, 42:139
\tenln . 31:9, 42:135, 42:137, 42:687, 42:689
\tenlnw 31:9, 42:138, 42:140
test (hook) *237*, *237*
\test *1368*
\testallgroups 33:1214, 33:1387, 33:1388, 33:1389, 33:1391, 33:1392, 33:1393, 33:1394, 33:1395, 33:1396, 33:1397, 33:1398, 33:1399, 33:1400, 33:1401, 33:1402, 33:1403, 33:1404, 33:1405, 33:1406, 33:1407, 33:1408, 33:1409, 33:1413, 33:1414, 33:1415, 33:1416, 33:1417, 33:1418, 33:1422, 33:1423, 33:1424, 33:1428, 33:1429, 33:1430, 33:1431, 33:1432, 33:1433, 33:1434, 33:1435, 33:1436, 33:1437, 33:1438, 33:1439, 33:1440, 33:1441, 33:1442, 33:1443, 33:1444, 33:1445, 33:1446, 33:1447, 33:1448, 33:1450, 33:1454, 33:1455, 33:1456, 33:1457, 33:1458, 33:1459, 33:1460, 33:1461, 33:1462, 33:1464, 33:1465, 33:1466, 33:1467, 33:1468, 33:1469, 33:1470, 33:1471, 33:1472, 33:1473, 33:1474, 33:1475, 33:1476, 33:1477, 33:1478, 33:1479, 33:1480, 33:1481, 33:1482, 33:1483, 33:1484, 33:1485, 33:1486, 33:1487, 33:1488, 33:1489, 33:1490, 33:1491, 33:1492, 33:1493, 33:1494, 33:1495, 33:1496, 33:1497, 33:1498, 33:1499, 33:1500, 33:1501, 33:1502, 33:1503, 33:1504, 33:1505, 33:1506, 33:1507, 33:1508, 33:1509, 33:1510, 33:1511, 33:1512, 33:1513, 33:1514, 33:1515, 33:1516, 33:1517, 33:1518, 33:1519, 33:1520, 33:1521, 33:1522, 33:1523, 33:1524, 33:1525, 33:1526, 33:1527, 33:1528, 33:1529, 33:1530, 33:1531, 33:1532, 33:1533, 33:1534, 33:1535, 33:1536, 33:1537, 33:1538, 33:1539, 33:1540, 33:1541, 33:1542, 33:1543, 33:1544, 33:1545, 33:1546, 33:1547, 33:1548, 33:1549, 33:1550, 33:1551, 33:1552, 33:1553, 33:1554, 33:1555, 33:1556, 33:1557, 33:1558, 33:1559, 33:1560, 33:1561, 33:1562, 33:1563, 33:1564, 33:1565, 33:1566, 33:1567, 33:1568, 33:1569, 33:1570, 33:1571, 33:1572, 33:1573, 33:1574, 33:1575, 33:1576, 33:1577, 33:1578, 33:1579, 33:1598
\testallkerneldefinedfamilies 33:1386, 33:1595
\testFont 33:1182, 33:1183, 33:1215, 33:1216
\testgroup . 33:1187, 33:1225, 33:1234, 33:1240, 33:1246, 33:1250, 33:1254, 33:1264, 33:1268, 33:1336, 33:1340
\TeX 19:1, 19:12, *1413*
TEX and LATEX 2ε commands:
\...-h@@k *1413*
\..@without@substitution *676*
\@ 09:408, 04:20, 14:19, 18:522, 19:2, 04:1041, 50:43, 50:59, 50:75, 50:88, 50:97, 50:139, 50:150, 06:925, 06:926, 57:561, 01:49, *1406*
\@...hook *252*
\@?@? *1363*
\@@ . 13:15, 13:19, 13:20, 13:21, 13:22, 13:24, 13:27, 13:28, 13:30, 13:31, 20:772, 20:792, 26:527, 26:529, 26:530, 41:315, 41:316, 41:317, 41:327, 01:315, 01:316, 54:6, 54:7
\@@DeclareMathDelimiter *1374*
\@@data@glyphtounicode 57:345, 57:351
\@@data@glyphtounicode@cmex 57:347, 57:352
\@@defaultsubs 24:682
\@@enc@update 21:201, 24:322, 24:326, *1386*
\@@end 20:741, 20:742, 01:206, 06:23, 37:35, 37:100, 37:176, 37:242, 56:18, 57:749, 57:770, 01:53, *1368*
\@@endpbox 41:245, 41:268, 41:313, 41:463
\@@eqncr 38:443, 38:472, 38:496, 38:506, 38:511, 38:634
\@@fileswith@pti@ns 50:640, 50:659, 50:1147
\@@hyph 06:24, 06:942
\@@hyphenation 21:225
\@@if@newlist 54:877, 54:941, 54:954, 54:999, 54:1012, 54:1058
\@@ifdefinable 21:35, 06:173
\@@input 20:639, 20:652, 20:692, 06:22, 37:29, 37:94, 37:156, 50:1772, 52:172, 52:188, 52:196, 57:383, 01:52, *1405*
\@@italiccorr 06:25, 32:114, 32:118, *1370*
\@@line 40:725
\@@math@bgroup 32:132, 32:139
\@@math@egroup 32:129
\@@par . 15:4, 16:142, 16:179, 06:21, 37:242, 37:533, 37:538, 37:541,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

37:555, 37:559, 37:562, 39:82, 39:85, 40:373, 40:398, 40:424, 40:448, 40:469, 41:247, 41:251, 41:270, 41:274, 44:67, 44:118, 54:262, *452*
\@@patterns 21:225
\@@protect 06:358, 06:364, 06:373
\@@sqrt *1395*
\@@startpbox 41:245, 41:268, 41:313, 41:463
\@@sverb 37:648, *885*
\@@text@case@aux 57:629, 57:663, 57:668, 57:673
\@@underline 40:614, 40:622, 40:659, 40:660
\@@unprocessedoptions 50:1059, 50:1124, 50:1222
\@@warning 14:215, *1366*
\@@write 54:112, 54:116
\@Alph 22:307, 22:323, *111*
\@DeclareEncodingSubset . . 24:191, 24:193, 24:194, 24:195, 24:196, 24:199
\@DeclareMathDelimiter 28:1039, 28:1058, *1374*
\@DeclareMathSizes 24:269, 24:270, 24:272
\@Esphack 18:213, 45:201, 45:223, 45:241, *1360*
\@Ialph *1376*
\@IncludeInRele@se 03:71
\@IncludeInRelease 03:71
\@LRmoderr 14:271, 18:261, 18:312, *1433*
\@M 18:11, 18:12, 18:13, 18:14, 18:15, 18:16, 18:17, 18:18, 18:19, 18:120, 24:752, 24:760, 26:456, 26:469, 06:37, 06:39, 38:484, 39:225, 41:56, 44:67, 44:100, 44:118, 44:130, 44:234, 44:257, 02:21, 54:155, 54:182, 54:200, 54:203, 54:263, 02:383, 02:384
\@MM 45:531, 45:550, 45:568, 02:21, 54:304, *421*
\@Mi 12:3, 54:114
\@Mii 12:3, 45:53, 45:122, 45:194, 45:216, 45:241, 45:311, 54:300, 54:1511, 54:1660, 54:1822
\@Miii 12:3, 45:55, 45:124, 45:313, 54:303
\@Miv 12:3, 45:195, 45:201, 45:217, 45:223, 54:277
\@Roman 22:305, 22:311, *1399*
\@TeXversion 14:28, 01:310, *1379*
\@abspage@last . . . 53:166, 53:172, 53:398, 53:400, 53:402, 53:415, 53:419, 53:422, 53:462, 53:463, *1208*
\@acci . . . 29:932, 40:450, 40:471, *1376*
\@accii . . 29:932, 40:450, 40:471, *1376*
\@acciii . 29:932, 40:450, 40:471, *1376*
\@acol 41:172, 41:179, 41:206, 41:213, 41:337, 41:338, 41:350, 41:351, 41:354, 41:371, 41:386, 41:394, 41:404
\@acolampacol 41:335, 41:352, 41:354, 41:361, 41:369, 41:403, 41:406
\@activechar@info 54:826, *1373*
\@activechar@warning *1373*
\@addamp 41:328, 41:337, 41:338, 41:353, 41:367, 41:404, 41:405
\@addfield 41:43, 41:53, 41:86, 41:93, 41:125, 41:140, 41:142
\@addmarginpar 54:336, 54:2388
\@addtobot 54:1326, 54:1413, 54:1481, 54:1533, 54:1630, 54:1679, 54:1786, 54:1845
\@addtocurcol 54:333, 54:1417, 54:2576
\@addtodblcol 54:1205, 54:2139
\@addtofilelist 20:64, 20:133, 20:188, 20:639, 20:761, 29:906, 29:909, 29:916, 29:919, 29:926, 29:929, 50:1771, 52:169, 52:188, 52:196, 57:281, 57:284, 57:792, 01:85, 01:87, *1370*
\@addtofilelist@aux 57:792
\@addtonextcol 54:1204, 54:1882, 54:2577
\@addtopreamble . . 41:388, 41:401, 41:407, 41:408, 41:409, 41:411, 41:423
\@addtoreset 22:33, 22:62, 22:82, 22:101, 22:117, 22:148, 22:209, 22:224, 22:238, 22:241, *561*
\@addtotoporbot 54:1363, 54:1527, 54:1675, 54:1839, 54:1932, 54:2013, 54:2101
\@afterheading . . . 44:92, 44:125, *1397*
\@afterindentfalse 44:45
\@afterindenttrue 44:43, 44:124, 44:233, 44:256
\@alph 22:306, 22:319, 45:399, *111*
\@ampacol 41:335, 41:352, 41:363, 41:406
\@arabic 22:70, 22:90, 22:107, 22:246, 22:293, 22:303, 22:309, 45:397
\@argarraycr 41:280, 41:281
\@argdef 06:98
\@argrsbox 40:686
\@argtabularcr 41:287, 41:288
\@array 41:222, 41:227, 41:230
\@arrayacol 41:172, 41:179, 41:335
\@arrayclassiv . 41:173, 41:180, 41:408
\@arrayclassz . . 41:172, 41:179, 41:352
\@arraycr 41:174, 41:181, 41:278, 41:280
\@arrayparboxrestore 40:440, 40:482, 41:461

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\@arrayrule 41:386, 41:388, 41:392, 41:394, 41:396, 41:423
\@arstrut 41:244, 41:267, 41:314, 41:420
\@arstrutbox 41:237, 41:260, 41:295, 41:420, 41:462, *1374*
\@author 44:8, 44:32
\@auxout 20:236, 20:242, 20:286, 20:304, 20:328, 20:361, 20:390, 20:413, 20:437, 20:452, 35:101, 35:119, 35:128, 36:55, 44:189, 44:199, 47:7, 47:8, 47:39, 47:49, 47:57, 47:67, 47:84, 53:414
\@backslashchar . . 14:234, 14:236, 30:277, 06:272, 06:573, 06:722, 06:734, 06:738, 06:739, 06:744, 06:788, 50:1351, 50:1484, 50:1573, *1376*
\@backup@outputbox@depth 54:579, 54:589, 54:725, *1245*
\@badcrerr 14:279, *1374*
\@badend 14:247, 37:392, *1361*
\@badlinearg 14:267, 42:176, 42:188, 42:199, 42:200, 42:204, 42:253, 42:258, 42:268, 42:273, 42:286
\@badmath 14:251, 38:275, 38:277, 38:282, 38:285, 38:311, 38:325, 38:330, 38:339, 38:351, 38:356, 38:364, 38:377, 38:382, 38:557, 38:569, 38:585, 38:594
\@badpoptabs 14:255, 41:85, 41:151
\@badrequireerror 50:532, 50:1230
\@badtab 14:258, 41:22, 41:87, 41:108, 41:114, 41:121, 41:148
\@begin@tempboxa 40:27, 40:46, 40:61, 40:251, 40:270, 40:373, 40:398, 40:424, 40:687, 40:695, *1365*
\@begindocumenthook 20:61, 20:127, 20:130, 20:182, 20:185, 47:53, 50:1158, 50:1179, *239*
\@begindvi 54:910, 54:978, 54:1037, 54:1065, *1191*
\@begindvibox 53:502, 53:503, 54:62, 54:1066, *1186*
\@beginparpenalty . . 18:15, 38:560, 38:572, 38:598, 39:23, 39:201, *909*
\@begintheorem 43:57, 43:62
\@bezier 42:694, 42:693
\@bibitem 47:3, 47:8
\@biblabel 47:4, 47:97
\@bitor 54:11, 54:1232, 54:1252, 54:1288, 54:1311, 54:1378, 54:1463, 54:1473, 54:1612, 54:1622, 54:1765, 54:1776, 54:1919, 54:2000, 54:2086, 54:2204, 54:2329
\@botlist . . . 54:41, 54:389, 54:391, 54:436, 54:438, 54:637, 54:657, 54:810, 54:1089, 54:1098, 54:1099, 54:1340, 54:1343, 54:1378, 54:1473, 54:1622, 54:1776, 54:2532, 54:2560
\@botnum . . . 45:274, 54:87, 54:1337, 54:1338, 54:1343, 54:1347, 54:1955, 54:1960, 54:2036, 54:2041, 54:2128, 54:2135, 54:2524, 54:2552, 54:2594
\@botroom 45:275, 54:88, 54:1340, 54:1343, 54:2525, 54:2553
\@boxfpsbit . 54:2652, 54:2654, 54:2659
\@break@loop *1374*
\@break@tfor 13:31, 20:612, 20:629, 32:99, *1374*
\@bsphack 18:37, 18:126, 18:409, 18:425, 18:443, 18:459, 35:98, 35:116, 35:127, 36:82, 44:186, 44:187, 45:52, 45:121, 45:310, 46:6, 46:18, 46:23, 46:35, 47:63, 47:80, 54:2459, *853*
\@caption 45:12, 45:14
\@captype 45:5, 45:9, 45:12, 45:40, 45:88, 45:109, 45:157, 54:2616
\@car 19:14, 21:107, 21:128, 06:53, 06:166, 06:222
\@carcube 06:55, 06:176, 06:725
\@cclv 02:16, 54:305, 54:309, 54:387, 54:388, 54:417, 54:434, 54:435, 54:464, 54:492, 54:739, 54:743, 57:67, *1191*
\@cclvi 04:30, 04:58, 02:21, 50:1349, 50:1482, 50:1571, 02:53, 02:78, 02:103
\@cdr 20:270, 06:53, 06:868, 06:869
\@centercr . . 37:415, 37:453, 37:458, 37:463, 37:473, 37:477, 37:481, *1413*
\@centering 38:418, 38:419, 38:431, 38:434, 38:437, 38:460, 38:463, 38:466, 38:627, 38:631
\@cflb 54:639, 54:810, 54:1070
\@cflt 54:634, 54:809, 54:1070
\@changed@cmd 21:3, 21:81, 21:243, 24:132, 24:330, 57:525, *1375*
\@changed@x . 21:3, 21:231, 21:239, *1386*
\@changed@x@err *1394*
\@changed@x@mouth 21:231, 21:239, *1386*
\@charlb 20:458, 20:466
\@charrb 20:460, 20:466
\@chclass 41:348, 41:349, 41:412, 41:425, 41:430
\@check@IncludeInRelease 03:71
\@check@c 06:230, 06:232
\@check@eq 06:236, 06:237, 06:241
\@checkcommand *1378*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\@checkend 37:16, 37:84, 37:148, 37:339, 37:354, 37:371, 37:381, 37:391, *1388*
\@chnum 41:356, 41:375, 41:412, 41:427, 41:428, 41:429
\@circ 42:632, 42:650, 42:659, 42:664, 42:667
\@circle 42:615, 42:616
\@circlefnt 42:136, 42:139, 42:458, 42:501, 42:530, 42:555, 42:625, 42:641, 42:673, 42:688
\@cite 47:36, 47:95
\@cite@ofmt 47:44, 47:96
\@citea 47:35, 47:37
\@citeb 47:38, 47:39, 47:40, 47:43, 47:44, 47:66, 47:67, 47:68, 47:69, 47:83, 47:84, 47:85, 47:86
\@citex 47:20, 47:21, 47:29, 47:34, *1054*
\@citex@checkblank 47:17, 47:18, 47:30
\@classi 41:348, 41:384
\@classii 41:348, 41:398
\@classiii 41:348, 41:403
\@classiv 41:173, 41:180, 41:208, 41:215, 41:349
\@classoptionslist 50:9, 50:581, 50:596, 50:597, 50:614, 50:853, 50:854, 50:882, 50:883, 50:909, 50:910, 50:1853, *1421*
\@classv 41:349, 41:409
\@classz 41:172, 41:179, 41:207, 41:214, 41:348, *1321*
\@cline 41:444
\@clnht 42:206, 42:207, 42:215, 42:217, 42:219, 42:229, 42:236, 42:284, 42:682
\@clnwd 42:208, 42:214, 42:218, 42:220, 42:221, 42:682
\@cls@pkg 08:2721, 50:388, 50:389, 50:402, 50:403, 50:998, 50:1008, 50:1056, 50:1103, 50:1133, 50:1185, 50:1215, 50:1217, 50:1234, 50:1781, 50:1803, 50:1833
\@clsextension 50:31, 50:191, 50:199, 50:257, 50:418, 50:434, 50:446, 50:528, 50:552, 50:563, 50:581, 50:595, 50:613, 50:712, 50:727, 50:751, 50:852, 50:881, 50:908, 50:1002, 50:1049, 50:1076, 50:1137, 50:1150, 50:1190, 50:1205, 50:1657, 51:100, 51:167, *1126*
\@clubpenalty . . 20:7, 20:25, 20:95, 20:152, 39:128, 39:227, 44:106, 44:135
\@colht 20:22, 20:92, 20:149, 45:273, 45:275, 45:278, 45:284, 45:285, 45:298, 45:299, 54:92, 54:236, 54:247, 54:256, 54:257, 54:392, 54:404, 54:439, 54:452, 54:481, 54:509, 54:526, 54:532, 54:536, 54:545, 54:550, 54:759, 54:777, 54:781, 54:788, 54:942, 54:1000, 54:1059, 54:1128, 54:1166, 54:1210, 54:1235, 54:1254, 54:1294, 54:1316, 54:2219, 54:2345, 54:2717, 57:155
\@colnum 45:276, 54:89, 54:1346, 54:1391, 54:1461, 54:1462, 54:1490, 54:1498, 54:1610, 54:1611, 54:1639, 54:1647, 54:1763, 54:1764, 54:1796, 54:1808, 54:1917, 54:1918, 54:1955, 54:1960, 54:1998, 54:1999, 54:2036, 54:2041, 54:2084, 54:2085, 54:2127, 54:2134, 54:2520, 54:2548, 54:2587, 54:2827
\@colroom 20:23, 20:93, 20:150, 54:93, 54:257, 54:278, 54:279, 54:290, 54:293, 54:392, 54:439, 54:1128, 54:1345, 54:1390, 54:1457, 54:1460, 54:1489, 54:1606, 54:1609, 54:1638, 54:1758, 54:1762, 54:1795, 54:1913, 54:1916, 54:1994, 54:1997, 54:2079, 54:2083, 54:2521, 54:2549, 54:2732, 54:2737, 54:2782, 54:2787, 54:2832, 57:154, *1395*
\@combinedblfloats 54:1101, 54:2891, 54:2933, 54:2970
\@combinefloats 54:755, 54:808, 54:1069, *1247*
\@comdblflelt 54:1101
\@comflelt . . 54:1071, 54:1087, 54:1101
\@compatibility *1366*
\@cons 22:124, 22:136, 22:144, 06:52, 45:193, 45:215, 45:239, 45:379, 54:242, 54:1239, 54:1258, 54:1274, 54:1298, 54:1300, 54:1320, 54:1322, 54:1493, 54:1561, 54:1642, 54:1706, 54:1801, 54:1874, 54:1948, 54:2029, 54:2118, 54:2221, 54:2244, 54:2347, 54:2372, 54:2389, 54:2390, 54:2833, 02:119, 02:137
\@contentsline@destination 44:213, 44:215, 44:222, *1023*
\@contfield 41:50, 41:141, 41:153
\@copy@... . . . . *103*
\@copy@DeclareRobustCommand 06:589, 06:659, 06:680, 06:758, 06:761
\@copy@newcommand 06:410, 06:590, 06:659, 06:701, 06:731, 06:758, 06:764, *96*
\@copytexsys *1368*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\@ctrerr 14:243, 22:322, 22:326, 22:340, 22:348
\@curfield 41:16, 41:41, 41:47, 41:51, 41:52, 41:54, 41:130, 41:131
\@curline 41:16, 41:27, 41:39, 41:44, 41:53, 41:54, 41:55, 41:90, 41:91, 41:103, 41:128, 41:129
\@curr@enc 21:172, 21:174
\@curr@file 20:246, 20:247, 20:256, 20:258, 20:282, 20:290, 20:471, 20:490, 20:659, 20:674, 50:983, 50:1296, 50:1301, 50:1307, 50:1313, 50:1317, 50:1328, 50:1337, 50:1364, 50:1427, 50:1432, 50:1438, 50:1461, 50:1470, 50:1496, 52:266, 52:374, 52:376, *1173*
\@curr@file@reqd 52:266, 52:376, 52:380, *1175*
\@currbox 45:60, 45:91, 45:95, 45:129, 45:160, 45:164, 45:193, 45:214, 45:215, 45:239, 45:257, 45:259, 45:261, 45:319, 45:322, 45:327, 45:331, 54:218, 54:219, 54:230, 54:231, 54:233, 54:234, 54:242, 54:316, 54:317, 54:1204, 54:1205, 54:1453, 54:1456, 54:1464, 54:1487, 54:1491, 54:1493, 54:1508, 54:1549, 54:1561, 54:1603, 54:1605, 54:1613, 54:1636, 54:1640, 54:1642, 54:1657, 54:1694, 54:1706, 54:1753, 54:1756, 54:1793, 54:1798, 54:1801, 54:1818, 54:1863, 54:1874, 54:1906, 54:1923, 54:1937, 54:1948, 54:1988, 54:2004, 54:2018, 54:2029, 54:2070, 54:2107, 54:2118, 54:2158, 54:2162, 54:2173, 54:2179, 54:2181, 54:2185, 54:2190, 54:2199, 54:2208, 54:2214, 54:2221, 54:2244, 54:2279, 54:2283, 54:2295, 54:2302, 54:2304, 54:2308, 54:2314, 54:2324, 54:2339, 54:2347, 54:2372, 54:2390, 54:2399, 54:2622, 54:2623, 54:2652, 54:2682, 54:2687, 54:2738, 54:2741, 54:2753, 54:2761, 54:2788, 54:2791, 54:2803, 54:2811, 54:2828, 54:2833, 02:161, 02:162, 02:163
\@currdir 01:92, 01:114, 01:116, 01:122, 01:124, 01:130, 01:132, 01:137, 01:139, 01:149, 01:162, 01:227, 01:240, 01:253, 50:1275, 50:1301, 50:1410, 50:1432, 50:1461, 50:1544, *9*
\@current@cmd 21:43, 24:334, *1386*
\@currentHpage . . . 36:263, 36:264, *854*
\@currentHref 17:128, 17:132, 17:152, 17:156, 17:177, 35:106, 35:113, 35:121, 35:137, 36:262, *460*
\@currentcounter 22:31, 35:139, 35:146, 35:159, 35:167, 35:176, 35:178, 35:185, 36:265, 38:427, 38:456, 38:612, 40:548, 43:67, 43:69, 45:533, *1416*
\@currentlabel 35:105, 35:120, 35:129, 35:148, 35:161, 35:168, 35:179, 35:187, 35:194, 35:230, 36:259, 38:426, 38:455, 38:611, 40:549, 40:566, 40:583, 45:534, 45:552, 45:570, *854*
\@currentlabelname 35:106, 35:112, 35:121, 36:261, *844*
\@currentmetafamily 29:163, 29:165, 29:197, 29:204, 29:206, 29:229, 29:781, 29:784, 29:787, 29:789, *1439*
\@currenvir 14:249, 37:3, 37:253, 37:274, 37:292, 37:302, 37:392, 39:112, 40:193, 50:1339, 50:1351, 50:1359, 50:1363, 50:1370, 50:1472, 50:1484, 50:1492, 50:1496, 50:1502, 50:1561, 50:1573, 50:1581, 50:1585, 50:1591, 07:1632, 07:1738, 07:1753, 07:1764, *435*
\@currenvline 14:249, 37:254, 37:275, 37:293, 37:303, 37:393, 40:194
\@currext 50:30, 50:42, 50:58, 50:74, 50:87, 50:96, 50:138, 50:149, 50:414, 50:417, 50:418, 50:433, 50:434, 50:445, 50:446, 50:552, 50:563, 50:574, 50:581, 50:595, 50:613, 50:642, 50:722, 50:735, 50:737, 50:745, 50:759, 50:951, 50:952, 50:954, 50:958, 50:961, 50:963, 50:968, 50:973, 50:975, 50:980, 50:986, 50:994, 50:1000, 50:1002, 50:1006, 50:1011, 50:1017, 50:1026, 50:1029, 50:1034, 50:1037, 50:1040, 50:1042, 50:1043, 50:1045, 50:1049, 50:1058, 50:1060, 50:1061, 50:1066, 50:1069, 50:1072, 50:1076, 50:1082, 50:1095, 50:1100, 50:1101, 50:1106, 50:1112, 50:1116, 50:1118, 50:1119, 50:1121, 50:1123, 50:1125, 50:1126, 50:1129, 50:1135, 50:1137, 50:1150, 50:1163, 50:1190, 50:1193, 50:1205, 50:1223, 50:1224, 51:66, 51:83, 51:85, 51:100, 51:107, 51:109, 51:167, 51:169, 51:172, *1124*
\@currextension *1362*
\@currlist 45:193, 45:215, 45:379, 54:43, 54:316,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

54:393, 54:396, 54:440, 54:443, 54:2389
\@currname 03:71, 03:116, 03:124, 17:27, 20:781, 20:803, 20:809, 50:29, 50:41, 50:57, 50:73, 50:86, 50:95, 50:137, 50:148, 50:386, 50:388, 50:400, 50:402, 50:414, 50:417, 50:433, 50:445, 50:574, 50:642, 50:735, 50:737, 50:745, 50:759, 50:949, 50:952, 50:954, 50:958, 50:961, 50:963, 50:968, 50:970, 50:973, 50:975, 50:980, 50:985, 50:994, 50:998, 50:1000, 50:1006, 50:1008, 50:1009, 50:1011, 50:1017, 50:1026, 50:1028, 50:1034, 50:1037, 50:1040, 50:1042, 50:1043, 50:1047, 50:1051, 50:1058, 50:1060, 50:1061, 50:1066, 50:1069, 50:1073, 50:1077, 50:1082, 50:1094, 50:1118, 50:1119, 50:1121, 50:1123, 50:1125, 50:1126, 50:1163, 50:1215, 50:1217, 50:1224, 50:1234, 50:1781, 50:1803, 50:1833, 51:66, 51:80, 51:83, 51:85, 51:107, 51:109, 51:169, 51:172, 51:214, 51:216, 51:227, 51:285, 08:362, 08:368, 08:423, *258*
\@currnamestack 50:33, 50:171, 52:546, 08:420, *260*
\@curroption *1362*
\@curroptions 50:574, 50:582, 50:632, 50:651, 50:1224, 50:1225
\@currpath 50:15, 50:47, 50:62, 50:174, 50:386, 50:388, 50:950, 50:968, 50:975, 50:984, 50:1026, 50:1027, 50:1054, *1108*
\@currpkg@reqd . . . 50:416, 50:978, 50:980, 50:989, 50:1022, 50:1036, 50:1039, 50:1054, 50:1056, *1124*
\@currsize 29:750, *1361*
\@currtype 54:97, 54:1229, 54:1230, 54:1231, 54:1232, 54:1249, 54:1250, 54:1251, 54:1252, 54:1378, 54:1463, 54:1473, 54:1612, 54:1622, 54:1765, 54:1776, 54:1919, 54:2000, 54:2086, 54:2204, 54:2329, 54:2622, 54:2624, 54:2625, 54:2628
\@curtab 41:11, 41:26, 41:86, 41:87, 41:88, 41:94, 41:95, 41:98, 41:102, 41:103, 41:107, 41:146, 41:147, *1393*
\@curtabmar 41:11, 41:25, 41:26, 41:38, 41:44, 41:89, 41:102, 41:106, 41:107
\@d@r 01:145, 01:146
\@dashbox 42:335, 42:336, 42:337, 42:338, 42:339, 42:342, 42:348, 42:350, 42:360, 42:362, 42:363, 42:364, 42:365, 42:369, 42:373, 42:376, 42:393, 42:395, 42:396, 42:397, 42:398, 42:401, 42:405, 42:407, 42:416, 42:418, 42:419, 42:420, 42:421, 42:424, 42:428, 42:433, 42:684
\@dashcnt 42:328, 42:330, 42:331, 42:332, 42:333, 42:334, 42:347, 42:349, 42:352, 42:354, 42:355, 42:356, 42:358, 42:359, 42:372, 42:375, 42:387, 42:388, 42:389, 42:390, 42:391, 42:392, 42:404, 42:406, 42:409, 42:410, 42:411, 42:412, 42:414, 42:415, 42:427, 42:432, 42:684
\@dashdim 42:327, 42:328, 42:329, 42:330, 42:331, 42:333, 42:336, 42:338, 42:339, 42:340, 42:347, 42:349, 42:351, 42:352, 42:353, 42:354, 42:355, 42:358, 42:362, 42:364, 42:365, 42:366, 42:374, 42:377, 42:386, 42:387, 42:388, 42:389, 42:391, 42:395, 42:397, 42:398, 42:399, 42:404, 42:406, 42:408, 42:409, 42:410, 42:411, 42:414, 42:418, 42:420, 42:421, 42:422, 42:430, 42:435, 42:684
\@date 44:9, 44:33
\@dbflt 45:32, 45:264, *1382*
\@dblarg . . 44:54, 44:142, 45:12, 06:890
\@dbldeferlist 45:239, 54:46, 54:450, 54:455, 54:457, 54:1167, 54:1174, 54:1175, 54:2329, 54:2372, 54:2536, 54:2565, *1407*
\@dblfloat 45:31
\@dblfloatplacement 20:31, 20:101, 20:159, 45:280, 54:406, 54:454, 54:2517, 54:2545, 54:2895, 54:2937, 54:2976
\@dblflset 45:26
\@dblfpbot 45:290, 45:304, 54:3020
\@dblfpsep 45:289, 45:303, 54:3020
\@dblfptop 45:288, 45:302, 54:3020
\@dbltoplist 54:45, 54:237, 54:240, 54:242, 54:402, 54:403, 54:450, 54:451, 54:1106, 54:1110, 54:1112, 54:1113, 54:2216, 54:2221, 54:2341, 54:2347, 54:2535, 54:2563
\@dbltopnum 45:283, 45:297, 54:85, 54:105, 54:243, 54:245, 54:1117, 54:2155, 54:2156, 54:2220, 54:2223, 54:2251, 54:2256, 54:2276, 54:2277, 54:2346, 54:2350, 54:2379, 54:2384, 54:2528, 54:2556

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\@dbltoproom 45:284, 45:286, 45:298, 45:300, 54:86, 54:2158, 54:2161, 54:2162, 54:2171, 54:2172, 54:2175, 54:2178, 54:2181, 54:2185, 54:2189, 54:2193, 54:2198, 54:2218, 54:2279, 54:2282, 54:2283, 54:2292, 54:2293, 54:2294, 54:2297, 54:2301, 54:2304, 54:2308, 54:2313, 54:2317, 54:2322, 54:2323, 54:2344, 54:2529, 54:2557
\@dec@text@cmd 21:3
\@declarecommandcopylisthook 06:586, 06:588, 06:600, 07:1167, *101*
\@declaredoptions 50:8, 50:535, 50:578, 50:616, 50:637, 50:656, 50:1156, *1362*
\@declareoption 50:533, 50:534, 50:542
\@defaultfamilyhook 29:508, 29:808, 29:823, 29:838, 29:843, 29:858, *1439*
\@defaultsubs 24:636, 24:670, 24:682, 37:59, 37:124, 37:163, *1391*
\@defaultunits 24:277, 24:281, 24:282, 24:283, 24:298, 24:391, 26:180, 26:182, 42:13
\@defaultunitsset . . . 40:74, 40:85, 42:8, 42:40, 42:41, 42:43, 42:45, 42:71, 42:74, 42:95, 42:96, 42:118, 42:119, 42:175, 42:252, 42:327, 42:329, 42:343, 42:351, 42:353, 42:368, 42:490, 42:491, 42:622, 42:658, 42:700, 42:701, 42:703, 42:704, 42:707, 42:708, 42:710, 42:711, 42:722, 42:723, 42:725, 42:726, 42:728, 42:729, 42:731, 42:732
\@defdefault@ds 50:533, 50:538, 50:543
\@deferlist 54:44, 54:389, 54:398, 54:399, 54:402, 54:407, 54:409, 54:415, 54:436, 54:445, 54:447, 54:1129, 54:1137, 54:1138, 54:1149, 54:1154, 54:1155, 54:1463, 54:1561, 54:1612, 54:1706, 54:1765, 54:1874, 54:1919, 54:1948, 54:2000, 54:2029, 54:2086, 54:2118, 54:2204, 54:2244, 54:2534, 54:2562
\@definecounter 22:12, 22:53, 38:393, 39:258, 39:259, 39:260, 39:261, 43:8, 43:16, 45:396, 45:398, *559*
\@depth 26:192, 06:26, 30:569, 30:570, 30:572, 30:573, 40:613, 40:714, 40:721, 41:239, 41:262, 41:296, 42:238, 42:311, 42:314, 42:335, 42:344, 42:394, 42:402, 42:737, 42:793, 54:2428
\@dir 01:144, 01:147, 01:149, 01:151, 01:152
\@disable@packageload@do 50:973, 52:477
\@dischyph 40:449, 40:470, 06:927, 06:961, *1371*
\@doclearpage . . . 54:301, 54:376, *1227*
\@documentclass *1364*
\@documentclasshook 50:3, 50:858, 50:886, 50:913, *1369*
\@doendpe 37:340, 37:355, 37:372, 37:382, 39:123, 39:159, *1436*
\@dofilelist 20:778, 20:820, 37:44, 37:109, 37:158, *1386*
\@dofilelist@hash 20:809, 20:815
\@dofilelist@size 20:803, 20:815
\@donoparitem 39:175, 39:189
\@dot 42:615, 42:653
\@dotsep 44:240, 44:263
\@dottedtocline 44:225, 44:251, 44:252
\@downline 42:308, 42:312, 42:317
\@downvector 42:279, 42:317
\@eha 03:195, 14:219, 14:237, 14:239, 14:241, 14:250, 14:252, 14:284, 20:243, 20:287, 20:305, 21:70, 21:102, 24:29, 24:59, 24:103, 24:145, 24:250, 24:316, 24:402, 26:107, 28:26, 28:71, 28:100, 28:173, 28:204, 28:234, 28:266, 28:550, 28:571, 28:623, 28:674, 28:719, 28:724, 28:779, 28:897, 28:901, 28:905, 28:940, 28:944, 28:948, 28:1005, 28:1015, 28:1100, 28:1105, 28:1108, 28:1140, 28:1143, 28:1216, 28:1219, 28:1222, 28:1289, 28:1295, 32:147, 33:24, 33:45, 33:876, 33:885, 37:252, 37:273, 37:291, 37:301, 06:384, 06:423, 06:451, 47:71, 47:88, 54:2453, 54:2469
\@ehb 14:219, 14:244, 14:270, 14:272, 14:274, 14:276, 54:239, 54:395, 54:442
\@ehc 03:184, 14:219, 14:279, 14:282, 14:288, 14:290, 06:169, 06:196, 37:657, 37:674, 37:688, 37:703, 37:716, 38:515, 39:251, 06:573, 06:614, 44:31, 50:1833
\@ehd 03:102, 03:159, 14:219, 14:246, 14:254, 14:257, 14:259, 14:265, 28:119, 05:93, 41:100, 41:109, 45:6, 50:792, 50:812, 50:1056
\@elt 20:459, 21:1604, 21:1606, 22:37, 22:52, 22:157, 22:160, 22:177, 22:180, 29:162, 29:174, 29:176, 29:177, 29:202, 29:236, 29:245, 29:247, 29:248, 29:265, 29:584, 29:586, 29:587, 29:599, 29:899,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

30:17, 30:25, 06:52, 54:4, 54:7, 54:11, 54:19, 54:21, 54:22, 54:23, 54:24, 54:27, 54:28, 54:29, 54:30, 54:31, 54:32, 54:33, 54:34, 54:36, 54:39, 54:495, 54:752, 54:1071, 54:1082, 54:1087, 54:1097, 54:1109, 54:1111, 54:1139, 54:1156, 54:1176, 54:1195, 54:1208, 54:1215, 54:1266, 54:1269, 54:1278, 54:2495, 54:2507, *741*
\@empty 13:14, *1381*
\@emptycol 54:203, 54:250, 54:253, 54:282, 54:286, *1373*
\@enc@info 21:7, 21:12, 21:103, 21:211, 21:221
\@end@check@IncludeInRelease 03:143, 03:145
\@end@tempboxa 40:36, 40:55, 40:64, 40:263, 40:275, 40:389, 40:416, 40:437, 40:693, 40:703
\@enddocument@kernel@warnings 37:45, 37:54, 37:110, 37:119, 37:178
\@enddocumenthook 37:147, 50:1158, 50:1180
\@endfloatbox 45:190, 45:211, 45:236, 45:248
\@endparenv 39:120, 39:123, *917*
\@endparpenalty . . . 18:16, 38:561, 38:573, 38:599, 39:23, 39:124, *909*
\@endpbox 41:245, 41:268, 41:313, 41:343, 41:410, 41:461, 41:464
\@endpefalse 37:258, 37:278, 37:296, 37:306, 39:129, 39:133, 39:137, 39:138, 39:140, 40:138, 40:144, 40:196, *917*
\@endpetrue 39:124, 39:126, 39:136, 39:140, *918*
\@endpreamblehook *489*
\@endtheorem 43:13, 43:19, 43:29, 43:38, 43:62
\@enlargepage 54:2438, 54:2443, 54:2445
\@ensuredmath 38:526, 38:528
\@enumctr 39:265, 39:268, 39:269
\@enumdepth 39:257, 39:263, 39:264, 39:265, *922*
\@enumspacing *922*
\@eqcnt 38:415, 38:512, 38:517, 38:614, 38:629, 38:630, 38:632
\@eqncr 38:432, 38:461, 38:481, 38:518, 38:519, 38:616
\@eqnnum 38:400, 38:407, 38:410, 38:516, 38:547, 38:606, *1360*
\@eqnsel 38:415, 38:628, *1362*
\@eqnswfalse 38:480
\@eqnswtrue 38:417, 38:428, 38:457, 38:517, 38:613
\@eqpen 38:415, 38:484, 38:486, 38:497, 38:507
\@er@ext *1363*
\@err@ 14:37, 14:41, 14:44, 14:52, 14:64, 14:68, 14:71, 14:79
\@esphack 18:39, 18:132, 18:414, 18:431, 18:448, 18:465, 35:108, 35:123, 35:130, 36:86, 44:186, 44:187, 45:385, 46:17, 46:19, 46:34, 47:74, 47:90, 54:2461, *853*
\@evenfoot 49:12, 49:15, 54:897, 54:968, 54:1027
\@evenhead 49:12, 49:15, 54:896, 54:967, 54:1026
\@execute@begin@hook 37:255, 37:259, 37:262, *873*
\@executeoption *1363*
\@expandtwoargs 03:188, 06:258, 50:259, 50:385, 50:580, 50:616, 50:670, 50:679
\@expast 41:316, 41:344
\@expl@@@filehook@clear@replacement@flag@@ 50:484, 50:507, 52:295, 52:318, 52:339, 52:519
\@expl@@@filehook@drop@extension@@N 52:292, 52:293, 52:315, 52:316, 52:336, 52:337, 52:521, *1174*
\@expl@@@filehook@file@pop@@ 50:991, 52:157, 52:525, 52:535
\@expl@@@filehook@file@pop@assign@@nnnn 52:162, 52:527, *1170*
\@expl@@@filehook@file@push@@ 50:974, 52:151, 52:523
\@expl@@@filehook@if@file@replaced@@TF . 52:289, 52:312, 52:333, 52:517, *1175*
\@expl@@@filehook@if@no@extension@@nTF 52:285, 52:308, 52:329, 52:507, 52:509, 52:534
\@expl@@@filehook@normalize@file@name@@w 52:291, 52:314, 52:335, 52:515
\@expl@@@filehook@resolve@file@subst@@w 50:482, 50:505, 52:288, 52:311, 52:332, 52:513
\@expl@@@filehook@set@curr@file@@nNN 50:481, 50:504, 50:943, 52:374, 52:380, 52:511, *1173*
\@expl@@@hook@curr@name@pop@@ 08:2928, 50:106
\@expl@@@hook@curr@name@push@@n *1419*
\@expl@@@initialize@all@@ 08:2928, 37:263, 08:1491

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\@expl@@@mark@update@dblcol@structures@@ 48:515, 48:528, 54:473
\@expl@@@mark@update@singlecol@structures@@ 48:513, 48:527, 54:476
\@expl@@@shipout@add@background@box@@n 53:470, 53:576
\@expl@@@shipout@add@background@picture@@n 53:470, 53:580
\@expl@@@shipout@add@firstpage@material@@Nn 53:470, 53:573
\@expl@@@shipout@add@foreground@box@@n 53:470, 53:578
\@expl@@@shipout@add@foreground@picture@@n 53:470, 53:582
\@expl@char@generate@@nn 05:159, 06:906, *1418*
\@expl@cs@⟨*thing*⟩@spec@@N *1417*
\@expl@cs@argument@spec@@N . . . *1432*
\@expl@cs@parameter@spec@@N 05:154, 05:155, 05:156, 05:167, 06:749, 06:802, 06:813, 06:818, *1432*
\@expl@cs@prefix@spec@@N 05:152, 05:166, 06:747, 06:817
\@expl@cs@replacement@spec@@N 05:157, 05:168, 06:707, 06:742, 06:749, 06:803, 06:814, 06:818
\@expl@cs@to@str@@N 05:147, 05:150, 05:162, 05:164, 06:581, 06:673, 06:693, 06:695, 06:696, 06:709, 06:722, 06:734, 06:738, 06:739, 06:744, 06:788, 06:797, 06:801, 06:809, 06:811, *1417*
\@expl@finalise@setup@@ 05:34, 57:268, 57:270
\@expl@finalize@setup@@ 05:35, 57:269, 57:271
\@expl@pop@filename@@ 05:30, 50:105, 50:109, 50:115, 50:126, *1417*
\@expl@push@filename@@ 05:28, 50:39, 50:52, 50:55, 50:69, 50:71, 50:81, 50:93, 50:113, 50:119, *1100*
\@expl@push@filename@aux@@ 08:2826, 05:29, 50:49, 50:52, 50:64, 50:77, 50:81, 50:99, 50:119, *322*
\@expl@str@if@eq@@nnTF . . . 05:151, 05:165, 06:545, 06:547, 50:385, *1417*
\@expl@str@map@function@@NN 05:158, 05:169, 06:903, *111*
\@expl@str@map@function@@NN and \@expl@char@generate@@nn . . . *1418*
\@expl@sys@load@backend@@ 20:17, 05:25, 05:27
\@expl@text@uppercase@@n *1426*
\@extra@page@added 37:70, 37:135, 53:440
\@failedlist 54:1193, 54:1216, 54:1232, 54:1239, 54:1252, 54:1258, 54:1274, 54:1288, 54:1311
\@fcolmadefalse 54:1184
\@fcolmadetrue 54:1272
\@file-subst@⟨*file*⟩ *1171*
\@filef@und . 20:510, 20:522, 20:523, 20:526, 20:532, 20:564, 20:586, 20:609, 20:626, 20:639, 20:692, 52:155, 52:172, 52:188, 52:196, *1414*
\@filehook@file@push *1124*
\@filehook@set@CurrentFile 20:334, 20:395, 50:976, 52:152, 52:363
\@filelist . . 20:63, 20:132, 20:187, 20:760, 20:761, 20:780, 29:906, 29:916, 29:926, 57:281, 57:774, 57:792
\@fileswfalse 20:200, 50:1271, 50:1272, 50:1406, 50:1407
\@fileswith@pti@ns 50:532, 50:640, 50:659, 50:844, 50:845, 50:849, 50:851, 50:879, 50:880, 50:906, 50:907, 50:934, 50:1147, *1362*
\@fileswith@ptions 50:829, 50:830, 50:837, 50:838, 50:842, 50:846
\@fileswithoptions 50:712, 50:719, 50:727, 50:818
\@fileswtrue 20:5, 50:1254, 50:1257, 50:1314, 50:1318, 50:1389, 50:1392, 50:1447, 50:1451
\@finalstrut 40:553, 40:570, 40:587, 40:704, 41:462, 45:539, 45:557, 45:575, *1376*
\@firstampfalse 41:331, 41:354, 41:371
\@firstamptrue 41:339
\@firstcolfirstmark 54:2916, 54:2917, 54:2921
\@firstcoltopmark . . . 54:2914, 54:2922
\@firstcolumnfalse 54:2878, 54:2907, 54:2948
\@firstcolumntrue 20:28, 20:98, 20:156, 54:76, 54:212, 54:2882, 54:2925, 54:2954
\@firstoffive 35:23, 35:25, 35:40, 35:57
\@firstofone 04:12, 04:100, 04:108, 04:166, 17:7, 17:12, 20:126, 20:181, 20:478, 21:86, 21:171, 26:363, 05:39, 05:66, 28:54, 28:82, 05:81, 28:147, 28:185, 28:215, 28:246, 28:1050, 05:216, 05:224, 06:253, 37:146, 38:524, 41:449, 06:569, 06:574, 06:575, 06:578, 06:579, 06:610, 06:615, 06:616, 06:619, 06:620, 45:10, 47:38, 47:66, 47:83, 50:322, 50:324, 50:326,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

50:328, 50:330, 50:332, 50:334, 50:336, 50:344, 50:992, 50:1021, 52:352, 52:369, 57:321, 57:408, *101*
\@firstoftwo . 03:85, 17:11, 20:525, 20:531, 20:565, 20:610, 20:627, 21:151, 22:358, 22:363, 24:219, 24:239, 05:40, 05:48, 28:1054, 29:589, 33:798, 33:848, 33:864, 35:42, 35:78, 06:253, 06:298, 06:319, 06:546, 06:548, 06:570, 06:611, 06:676, 06:727, 49:16, 50:196, 50:230, 50:242, 06:842, 50:265, 50:283, 50:342, 06:852, 50:372, 06:862, 06:889, 06:913, 50:1842, 54:644, 54:653, 54:660, 01:71, *500*
\@firsttab 41:2, 41:74, 41:75, 41:76, 41:106, 41:118, *1374*
\@flcheckspace 54:1340, 54:1376, 54:2723
\@flfail 54:1216, 54:1267, 54:1288, 54:1298, 54:1311, 54:1320
\@float 45:26, 45:32
\@floatboxreset 45:101, 45:170, 45:174
\@floatpenalty 45:3, 45:53, 45:55, 45:58, 45:122, 45:124, 45:127, 45:191, 45:194, 45:199, 45:201, 45:212, 45:216, 45:221, 45:223, 45:237, 45:241, 45:311, 45:313, 45:317, 45:321, 45:379
\@floatplacement . . . 20:31, 20:101, 20:159, 45:271, 54:128, 54:214, 54:258, 54:484, 54:2518, 54:2546, *1364*
\@flsetnum 54:1337, 54:1373, 54:1461, 54:1610, 54:1763, 54:1917, 54:1998, 54:2084, 54:2155, 54:2276, 54:2691
\@flsettextmin 54:1436, 54:1587, 54:1733, 54:1902, 54:1984, 54:2066, 54:2707
\@flstop 54:2583
\@flsucceed 54:1209, 54:1217, 54:1266, 54:1300, 54:1322
\@fltovf . 14:275, 45:93, 45:162, 45:322
\@flupdates . 54:1343, 54:1388, 54:2824
\@flushglue . 12:17, 37:454, 37:458, 37:464, 37:474, 37:477, 37:482, 37:532, 37:554, 39:76, 40:456, 40:477
\@fnsymbol 22:308, 22:327
\@font@aliasinfo 26:556
\@font@info . 24:134, 24:172, 24:178, 24:201, 24:202, 24:463, 24:480, 24:718, 25:2866, 26:31, 26:39, 26:47, 26:75, 26:88, 26:155, 26:201, 26:215, 26:226, 26:240, 26:256, 26:262, 26:275, 26:282, 26:289, 26:294, 26:304, 26:316, 26:328, 26:508, 26:520, 26:525, 26:532, 26:562, 26:575, 26:583, 28:280, 28:293, 28:304, 28:309, 28:314, 28:328, 28:345, 28:354, 28:373, 28:388, 28:423, 28:438, 28:498, 28:554, 28:653, 28:659, 28:703, 28:716, 28:799, 28:888, 28:931, 28:996, 28:1090, 28:1257, 28:1286, 33:63, 57:527
\@font@series@contextfalse 29:581, 29:621
\@font@series@contexttrue 29:602, 29:606, 29:620
\@font@shape@subst@warning 25:2833, 25:2836, 25:2841, 25:2899, 25:3178, 25:3181, *668*
\@font@warning 24:4, 24:632, 24:637, 24:664, 24:671, 25:2844, 26:20, 26:34, 26:42, 26:50, 26:62, 26:78, 26:493, 26:507, 26:519, 26:524, 26:531, 26:574, 26:582, 27:30, 37:56, 37:121, 37:160, 57:302
\@fontenc@load@list 21:1605, 29:899, 30:17, 30:25, *1415*
\@fontswitch 32:127, 32:129, *1374*
\@footnotemark 45:516, 45:522, 45:584, 45:590, 45:591, 45:617
\@footnotetext 40:518, 45:516, 45:522, 45:523, 45:600, 45:606
\@for 13:16, 20:252, 20:324, 20:386, 20:433, 20:780, 47:36, 47:65, 47:82, 50:267, 50:284, 50:578, 50:597, 50:614, 50:632, 50:637, 50:651, 50:656, 50:692, 50:702, 50:1225, 50:1262, 50:1397, *1054*
\@forced@seriesfalse 25:2782, 25:2795, 26:141
\@forced@seriestrue . 25:2786, 25:2797
\@forloop 13:19, 13:20
\@fornoop 13:15, 13:23, 13:29
\@fortmp 13:17, 13:18, 13:26, 50:690, 50:692, 50:1261, 50:1262, 50:1396, 50:1397
\@fpbot 45:290, 45:304, 54:1214, 54:3014
\@fpmin 45:278, 45:287, 45:301, 54:91, 54:1271, 54:2526, 54:2554, 54:2841, 54:2858
\@fps 45:41, 45:42, 45:44, 45:47, 45:64, 45:110, 45:111, 45:113, 45:116, 45:133, 54:2614, 54:2616, 54:2619
\@fpsadddefault 45:45, 45:48, 45:114, 45:117, 54:2611
\@fpsep . . . 45:289, 45:303, 54:1212, 54:1221, 54:1293, 54:1315, 54:3014

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\@fpstype 54:1334, 54:1355, 54:1356, 54:1370, 54:1401, 54:1402, 54:1426, 54:1428, 54:1431, 54:1433, 54:1485, 54:1541, 54:1542, 54:1577, 54:1579, 54:1582, 54:1584, 54:1634, 54:1686, 54:1687, 54:1721, 54:1724, 54:1727, 54:1730, 54:1791, 54:1853, 54:1854, 54:1892, 54:1894, 54:1897, 54:1899, 54:1974, 54:1976, 54:1979, 54:1981, 54:2054, 54:2057, 54:2060, 54:2063, 54:2152, 54:2167, 54:2169, 54:2187, 54:2196, 54:2232, 54:2233, 54:2273, 54:2288, 54:2290, 54:2310, 54:2320, 54:2359, 54:2360, 54:2597, 54:2623, 54:2625, 54:2627, 54:2630, 54:2631, 54:2632, 54:2634, 54:2635, 54:2639, 54:2640, 54:2642, 54:2643, 54:2677, 54:2679, 54:2681, 54:2693, 54:2695, 54:2709, 54:2711, 54:2746, 54:2749, 54:2760, 54:2796, 54:2799, 54:2810
\@fptop 45:288, 45:302, 54:1211, 54:3014
\@frameb@x 40:223, 40:262, 40:274, 40:278, *1380*
\@framebox 40:230, 40:237, 40:241
\@framepicbox . . 40:230, 40:237, 40:299
\@freelist 45:60, 45:129, 45:319, 45:320, 54:20, 54:25, 54:37, 54:218, 54:496, 54:753, 54:1083, 54:1098, 54:1112, 54:1217, 54:2389, 54:2390, 02:119, 02:137, 02:161, *1241*
\@generic@error *1379*
\@generic@message *1379*
\@getcirc 42:448, 42:495, 42:524, 42:551, 42:623, 42:639
\@getfpsbit 54:1331, 54:1367, 54:2149, 54:2270, 54:2650
\@getlarrow 42:277, 42:285, 42:287
\@getlinechar 42:201, 42:240
\@getpen . . . 18:35, 18:38, 18:47, 18:118
\@getrarrow 42:278, 42:285, 42:294
\@glossaryfile . . . 46:21, 46:22, 46:31
\@gnewline 18:96, 18:102, 18:109, 18:112
\@gobble 03:163, 03:187, 04:11, 04:98, 13:6, 13:9, 14:101, 14:127, 14:147, 14:155, 14:180, 14:189, 14:202, 17:6, 17:13, 17:21, 18:76, 18:581, 20:64, 20:133, 20:188, 20:485, 20:487, 20:760, 21:47, 24:633, 24:666, 26:362, 05:38, 27:26, 05:70, 28:29, 28:31, 05:93, 28:502, 28:513, 28:597, 28:664, 28:665, 28:694, 28:700, 28:708, 28:713, 28:731, 28:745, 28:755, 28:764, 28:777, 28:794, 28:803, 28:877, 28:879, 28:883, 28:891, 28:925, 28:934, 28:986, 28:988, 28:999, 28:1083, 28:1093, 28:1174, 28:1179, 28:1248, 28:1279, 29:140, 29:202, 29:265, 05:216, 05:224, 29:599, 29:909, 29:919, 29:929, 06:129, 33:825, 06:174, 06:249, 06:272, 37:283, 06:340, 06:344, 06:381, 06:387, 06:390, 06:400, 06:420, 06:426, 06:429, 06:438, 06:448, 06:454, 06:457, 06:466, 06:484, 06:488, 06:490, 06:491, 06:493, 06:501, 06:505, 06:507, 44:143, 44:144, 44:145, 44:146, 44:147, 44:200, 45:7, 47:11, 47:45, 47:46, 50:322, 50:324, 50:326, 50:328, 50:330, 50:332, 50:334, 50:336, 50:344, 50:780, 50:971, 50:1246, 50:1310, 50:1337, 50:1441, 50:1470, 50:1554, 50:1559, 50:1633, 50:1787, 50:1799, 52:261, 54:974, 54:975, 54:976, 54:1033, 54:1034, 54:1035, 54:1278, 54:2509, 54:2842, 54:2859, 57:284, 57:460, 57:792, 57:796, *1375*
\@gobble@AddToHook@args 08:2944, 08:2945
\@gobble@IncludeInRelease 03:71
\@gobble@RemoveFromHook@arg 08:2947, 08:2948
\@gobble@om 35:102, 35:104, 44:180, 44:190, 44:192, 44:203, *90*
\@gobble@som 35:103, 44:180, 44:191, 44:197, 44:204, *90*
\@gobble@with@sphack@om 44:180, 44:205, 54:904, 54:906, *90*
\@gobble@with@sphack@som 44:180, 44:206, 54:905, *90*
\@gobblecr 18:579, 18:580
\@gobblefour 28:25, 28:499, 28:655, 28:657, 28:661, 28:663, 28:673, 28:677, 28:801, 28:853, 06:249, 50:1339, 50:1472, 50:1561
\@gobblethree 06:249, 06:736, 06:748, 50:163, 57:323, *1386*
\@gobbletwo 13:12, 20:32, 20:102, 20:160, 24:638, 24:672, 28:133, 05:128, 05:133, 06:216, 06:217, 06:249, 37:24, 37:57, 37:91, 37:122, 37:153, 37:161, 49:11, 49:13, 50:1309, 50:1440, 50:1553, 52:260, 52:489, 57:308, 57:363, *1391*
\@gtempa 20:704, 20:705, 20:711, 20:712, 20:713, 20:738, 20:739, 20:741, 20:742,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

20:743, 06:166, 06:168, 06:222, 06:224, 41:3, 41:5, 41:6, 41:7, 41:8, 06:667, 06:675, 50:384, 50:386, 50:399, 50:400, 50:419, 50:421, 50:435, 50:437, 50:447, 50:449, *1370*
\@halfwidth 42:2, 42:137, 42:140, 42:142, 42:238, 42:310, 42:313, 42:335, 42:344, 42:360, 42:372, 42:375, 42:394, 42:402, 42:416, 42:427, 42:432, 42:690, 42:716, 42:735, 42:736, 42:737, 42:776, 42:791, 42:792, 42:793
\@halignto 41:174, 41:181, 41:198, 41:201, 41:243, 41:266
\@hangfrom 44:66, 44:117, 44:138
\@height 18:419, 18:427, 18:453, 18:461, 21:313, 21:315, 26:191, 06:26, 30:351, 30:569, 30:570, 30:572, 30:573, 40:207, 40:212, 40:285, 40:295, 40:613, 40:714, 40:721, 41:238, 41:261, 41:296, 41:436, 41:453, 42:238, 42:311, 42:314, 42:335, 42:344, 42:362, 42:370, 42:394, 42:402, 42:418, 42:425, 42:600, 42:610, 42:736, 42:792, 54:2428, 02:377
\@highpenalty 18:119, 57:3
\@hightab 41:11, 41:21, 41:23, 41:74, 41:86, 41:95, 41:96, 41:111, 41:146, 41:147, *1393*
\@hline . 42:178, 42:190, 42:237, 42:276
\@holdpg 54:100, 54:305, 54:307, 54:308, 54:313, 54:314, 54:315
\@hspace . 18:534, 18:535, 18:551, *1361*
\@hspacer 18:534, 18:550, *1361*
\@hvector 42:255, 42:270, 42:276
\@ialph *1376*
\@icentercr 37:432, 37:433
\@iden 06:256
\@if 06:212, 06:213, 06:215
\@if.. . . . . *1376*
\@if@DeclareRobustCommand 09:142, 06:589, 06:646, 06:657, 06:658, 06:662, 06:756, 06:757, 06:760, 06:783, *104*
\@if@bottomfloats@TF 54:577, 54:656, 54:667, 54:676, 54:689, 54:712, 54:815, *1233*
\@if@flushbottom@TF . . 54:642, 54:813
\@if@footnotes@TF 54:574, 54:649, 54:665, 54:683, 54:691, 54:708, 54:814, *1233*
\@if@newcommand . . 09:143, 09:154, 06:408, 06:590, 06:647, 06:658, 06:700, 06:712, 06:718, 06:784, *1422*
\@if@newlist *1403*
\@if@pti@ns 50:259, 50:262, 50:264, 50:281, 50:282
\@if@ptions 50:256, 50:257, 50:258, 50:1006, 50:1101
\@if@short@command *1376*
\@ifatmargin 41:55, 41:106
\@ifbothcounters 22:185, 22:200, 22:208, 22:223, 22:238, 22:240, 22:261, 22:273, 22:286, 22:288
\@ifclasslater . 50:198, 50:206, 50:214
\@ifclassloaded 50:190, 50:303, 50:312
\@ifclasswith . . 50:256, 50:305, 50:314
\@ifdefinable 21:32, 21:35, 22:11, 22:18, 23:3, 29:729, 06:102, 06:104, 06:171, 06:173, 06:295, 06:316, 06:342, 40:159, 43:7, 43:15, 43:26, 43:35, *1364*
\@iffileonpath 20:506, 20:560, 20:582, 20:599, 20:620
\@ifl@aded 50:190, 50:191, 50:192, 50:952, 50:1082, 50:1100, *1123*
\@ifl@t@r 21:1566, 05:215, 05:223, 50:204, 50:207, 50:212, 50:215, 50:219, 50:223, 50:225, 50:236, 50:237, 50:801
\@ifl@ter 21:1611, 21:1612, 50:198, 50:199, 50:218, 50:994, 50:1129
\@ifl@ter@@ 21:1611, 21:1612
\@ifnch 05:54, 05:55, 06:874, 06:876, 06:888
\@ifnextchar 18:94, 18:580, 20:652, 22:13, 26:428, 05:50, 37:431, 38:413, 39:174, 40:9, 40:11, 40:18, 40:20, 40:26, 40:68, 40:165, 40:166, 40:172, 40:173, 40:180, 40:184, 40:229, 40:230, 40:236, 40:237, 40:242, 40:300, 40:308, 40:316, 40:323, 40:327, 40:488, 40:492, 40:496, 40:597, 40:602, 40:673, 40:680, 40:685, 41:57, 41:222, 41:227, 41:280, 41:287, 42:34, 42:143, 42:154, 42:464, 43:3, 43:5, 43:46, 43:52, 45:27, 45:264, 45:324, 45:514, 45:581, 45:598, 47:3, 47:17, 47:29, 50:391, 50:405, 06:870, 06:875, 06:889, 50:796, 50:828, 50:836, 50:843, 50:1255, 50:1258, 50:1390, 50:1393, 54:214, 54:2585, 01:82, *1404*
\@iforloop 13:21, 13:22
\@ifpackagelater 50:198, 50:205, 50:213, *1097*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\@ifpackageloaded 50:190, 50:302, 50:311, 54:2569, *1097*
\@ifpackagewith 50:256, 50:304, 50:313, *1097*
\@iframebox 40:243, 40:244, 40:245
\@iframepicbox 40:300, 40:301
\@ifstar 18:60, 18:72, 18:399, 18:534, 22:236, 22:284, 24:269, 27:121, 06:78, 06:87, 37:420, 37:427, 37:727, 37:736, 38:483, 41:56, 41:279, 41:286, 42:153, 42:615, 44:52, 44:142, 50:533, 50:575, 06:889, 54:2433
\@ifundefin@d@i 06:832, 06:833, 06:850, 06:853
\@ifundefin@d@ii 06:832, 06:835, 06:838
\@ifundefined . . . 22:3, 22:7, 22:15, 22:33, 22:58, 22:60, 22:115, 22:122, 22:123, 22:152, 22:153, 22:154, 22:174, 22:189, 22:191, 24:101, 24:200, 24:249, 26:441, 28:544, 05:146, 33:827, 06:168, 06:175, 06:195, 06:202, 06:224, 35:85, 06:235, 37:218, 37:235, 37:251, 37:272, 37:290, 37:300, 06:381, 06:387, 06:420, 06:426, 06:448, 06:454, 06:484, 06:501, 06:582, 06:622, 06:623, 43:25, 43:34, 47:40, 47:68, 47:85, 49:3, 49:7, 06:827, 50:163, 50:188, 50:486, 50:492, 50:509, 50:521, 50:599, 50:633, 50:652, 50:954, 52:484, 07:3329, *1375*
\@ignore... . . . . *1397*
\@ignorefalse 24:433, 37:4, 37:257, 37:277, 37:295, 37:305, 37:341, 37:358, 37:374, 37:383, 45:384, *1360*
\@ignoretrue 18:223, 18:236, 24:428, 37:4, 37:7, 38:392, 38:400, 38:407, 38:446, 38:475, 38:637, *1396*
\@iiiminipage 40:490, 40:494, 40:497, 40:498, 40:499
\@iiiparbox 40:310, 40:318, 40:325, 40:328, 40:329, 40:365, 40:535
\@iiminipage 40:493, 40:495
\@iinput 20:652, 20:653, *504*
\@iiparbox 40:324, 40:326
\@iirsbox 40:685, 40:694
\@imakebox 40:26, 40:41, 40:182
\@imakepicbox 40:68, 40:69, 40:187, 40:302
\@iminipage 40:489, 40:491
\@in@minipage@envtrue 40:514
\@include 20:247, 20:290, 20:306, 20:310, *1416*
\@includeinreleasefalse 03:74, 03:79, 03:133, 03:141, 06:965
\@includeinreleasetrue 03:123
\@index 46:18, 46:19, 46:35
\@indexfile 46:4, 46:5, 46:14
\@inlabelfalse 39:28, 39:104, 39:215, 54:142, 54:169, 54:195, *1368*
\@inlabeltrue 39:28, 39:209
\@inmatherr 14:285, 39:112, 39:173, 42:615, *1378*
\@inmathwarn 21:3
\@inpenc@test 57:405, 57:472
\@input 20:34, 20:104, 20:162, 20:318, 20:380, 20:427, 20:691, 44:152, 57:805, *1160*
\@input@ 20:338, 20:365, 20:398, 20:417, 20:442, 20:693, 24:490, 47:51, *1367*
\@input@file@exists@with@hooks 52:143, *1419*
\@inputcheck 20:501, 20:502, 20:509, 20:555, 20:556, 20:563, 20:577, 20:578, 20:585, 20:607, 20:608, 20:611, 20:624, 20:625, 20:628, 01:175, 01:176, 01:179, 01:187, 05:43, 05:44, 05:47, 06:38, 06:45, 50:1298, 50:1299, 50:1333, 50:1429, 50:1430, 50:1466, 50:1541, 50:1542, 50:1549, 02:177, 01:54, *1360*
\@insertfalse 54:1424, 54:1575, 54:1719, 54:1890, 54:1972, 54:2052, 54:2147, 54:2268
\@inserttrue 54:1350, 54:1395, 54:1513, 54:1662, 54:1825, 54:2226, 54:2353
\@invalidchar 14:290
\@iparbox 40:309, 40:317, 40:322
\@irsbox 40:673, 40:680, 40:685, 40:686
\@isavebox 40:180, 40:181
\@isavepicbox 40:185, 40:186
\@ishortstack 42:144, 42:152
\@istackcr 42:154, 42:155
\@itabcr 41:57, 41:58
\@item 39:174, 39:187
\@itemdepth 39:272, 39:274, 39:275, 39:276, *922*
\@itemfudge 41:38, 41:44, 41:82
\@itemitem 39:276, 39:279
\@itemlabel . 39:44, 39:96, 39:174, *1365*
\@itempenalty 18:17, 39:23, 39:206, *909*
\@itemspacing *922*
\@iwhiledim 13:7
\@iwhilenum 13:3
\@iwhilesw 13:10

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\@ixpt 24:816
\@ixstackcr 42:153
\@kernel@... . . . . *1100*
\@kernel@Ref 35:202, 35:207
\@kernel@after@⟨*hook*⟩ *238*
\@kernel@after@begindocument . . . 09:77, 20:58, 20:76, 05:7, 51:265, *1421*
\@kernel@after@begindocument@before 20:16, 20:76, 25:3251, *489*
\@kernel@after@enddocument 05:1, 37:15, 37:83, 53:399
\@kernel@after@enddocument@afterlastpage . . . 05:1, 37:20, 37:87, 53:410, 08:455
\@kernel@after@para@after 16:8, 16:108, *444*
\@kernel@after@para@end 16:8, 16:101, *443*
\@kernel@after@shipout@background 53:133, 53:221, *1420*
\@kernel@after@shipout@lastpage 53:175, 53:180, 53:221, 53:430, 53:436, *1207*
\@kernel@before@⟨*hook*⟩ *238*
\@kernel@before@begindocument 20:56, 20:76, 05:7, 53:460, *1421*
\@kernel@before@enddocument 37:13, 37:81, 37:182, *869*
\@kernel@before@enddocument@afterlastpage 05:1, 37:18, 53:405, *1437*
\@kernel@before@insertmark 48:251, 48:278, *1078*
\@kernel@before@para@before 16:8, 16:21, 16:54, *444*
\@kernel@before@para@begin 16:8, 16:30, 16:62, *444*
\@kernel@before@shipout@background 53:129, 53:131, 53:221, *1195*
\@kernel@currpathstack 50:46, 50:48, 50:61, 50:63, 50:108, 50:158, *1421*
\@kernel@eqno . . 38:536, 38:538, 38:544
\@kernel@get@ctr@root 22:114, 22:129, 22:131, 22:166, 22:168
\@kernel@leqno . 38:537, 38:539, 38:545
\@kernel@make@file@csname 52:287, 52:290, 52:310, 52:313, 52:331, 52:334, 52:363
\@kernel@new@label@record@testdef 36:171, 36:302, 37:26
\@kernel@pageref 35:65, 35:69
\@kernel@pageref@exp 35:70
\@kernel@ref 35:64, 35:67, 35:202
\@kernel@ref@exp 35:72
\@kernel@refstepcounter 35:154, 35:171, 43:45, *1433*
\@kernel@rename@newcommand 06:389, 06:406, 06:443, 06:471, 06:476, 06:489, *95*
\@kernel@reserved@label@data 35:106, 35:114, 35:121
\@kernel@sRef 35:204, 35:207
\@kernel@spageref 35:26, 35:43, 35:60, 35:65, 35:69
\@kernel@sref . 35:13, 35:25, 35:30, 35:42, 35:47, 35:59, 35:64, 35:67, 35:204
\@kernel@whatsit . 17:63, 17:65, 17:118
\@killglue 42:70, 42:84, 42:114, 42:126, 42:134
\@kludgeins . 54:324, 54:325, 54:326, 54:328, 54:381, 54:382, 54:428, 54:429, 54:499, 54:521, 54:522, 54:523, 54:527, 54:542, 54:546, 54:556, 54:756, 54:778, 54:787, 54:794, 54:821, 54:822, 54:2429, 54:2460
\@labels 39:27, 39:177, 39:178, 39:220, 39:237, 39:238, *908*
\@largefloatcheck 45:192, 45:213, 45:238, 45:256, *1374*
\@lastchclass 41:339, 41:349, 41:350, 41:352, 41:360, 41:385, 41:399, 41:403, 41:412, 41:425, 41:426
\@latex@error . . . 03:100, 08:2971, 03:159, 03:184, 03:195, 14:163, 14:217, 14:233, 14:239, 14:241, 14:244, 14:246, 14:248, 14:252, 14:254, 14:256, 14:259, 14:263, 14:268, 14:272, 14:274, 14:276, 14:278, 14:279, 14:281, 14:284, 14:288, 14:290, 17:16, 20:243, 20:287, 20:305, 21:68, 21:102, 24:6, 24:29, 24:59, 24:103, 24:145, 24:250, 24:316, 24:402, 26:106, 27:100, 27:111, 28:24, 28:69, 28:98, 28:118, 05:85, 28:171, 28:202, 28:232, 28:264, 28:384, 28:400, 28:434, 28:450, 28:550, 28:571, 28:623, 28:673, 28:677, 28:719, 28:724, 28:779, 28:847, 28:853, 28:897, 28:901, 28:905, 28:940, 28:944, 28:948, 28:1005, 28:1015, 28:1100, 28:1105, 28:1108, 28:1140, 28:1143, 28:1216, 28:1219, 28:1222, 28:1289, 28:1295, 29:50, 29:61, 29:83, 29:94, 29:711, 29:876, 32:144, 06:169, 06:196, 37:252, 37:273, 37:291, 37:301, 37:657, 37:674, 37:688, 37:703, 37:715, 06:382, 38:515,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

06:421, 39:250, 06:449, 41:100, 41:109, 06:573, 06:614, 44:31, 45:6, 45:83, 47:71, 47:88, 50:723, 50:774, 50:787, 50:808, 50:824, 50:1007, 50:1055, 50:1102, 50:1214, 50:1231, 50:1239, 50:1244, 50:1266, 50:1323, 50:1401, 50:1456, 50:1802, 50:1832, 54:239, 54:395, 54:2451, 54:2468
\@latex@info 14:163, 14:208, 33:40, 06:277, 06:402, 06:440, 06:468, 06:927, 52:487, *433*
\@latex@info@no@line 14:163, 14:209, 53:150, 54:827, *1378*
\@latex@note 14:190, 24:34
\@latex@note@no@line 14:190, 50:1280, 50:1300, 50:1306, *1422*
\@latex@warning 14:163, 14:215, 20:363, 20:415, 21:73, 22:72, 22:92, 22:109, 22:128, 22:130, 22:165, 22:167, 35:18, 35:35, 35:52, 36:233, 36:244, 36:250, 40:51, 40:257, 40:385, 40:411, 40:510, 42:460, 45:260, 47:42, 47:69, 47:86, 50:1362, 50:1369, 50:1495, 50:1501, 50:1584, 50:1590, 53:235, 54:2617
\@latex@warning@no@line 14:163, 14:216, 20:19, 20:89, 20:146, 20:758, 05:103, 35:8, 35:88, 35:89, 06:243, 36:155, 36:156, 37:64, 37:71, 37:129, 37:136, 37:168, 44:32, 50:387, 50:401, 50:802, 50:995, 50:1130, 50:1282, 50:1431, 50:1437, 50:1460, 50:1543, 50:1550, 50:1617, 50:1703, 53:140, 53:159, 53:421, 54:248, 54:280, 54:2404, 54:2683
\@latexbug 14:277, 54:338, 54:2390
\@latexerr 14:215, 54:442, *1375*
\@latexinfo *1373*
\@latexrelease@catcode@null 07:6, 07:3330
\@lbibitem 47:3, 47:4
\@ldots 30:515, 30:517
\@leftcolumn 54:99, 54:2879, 54:2885, 54:2908, 54:2928, 54:2949, 54:2958
\@leftmark 49:16, 49:119
\@let@token 18:479, 18:480, 18:487, 05:54, 05:56, 32:84, 32:97, 38:256, 38:258, 38:261, 06:874, 06:877, 06:880, 06:888, *1392*
\@lign 38:208, 38:210
\@linechar . . 42:201, 42:202, 42:203, 42:207, 42:208, 42:210, 42:215, 42:217, 42:218, 42:219, 42:220, 42:222, 42:226, 42:227, 42:230, 42:231, 42:236, 42:283, 42:680, *1404*
\@linefnt . . . 42:135, 42:138, 42:201, 42:276, 42:284, 42:315, 42:318, 42:687
\@linelen . . . 42:175, 42:176, 42:187, 42:188, 42:214, 42:221, 42:230, 42:232, 42:237, 42:238, 42:239, 42:252, 42:253, 42:267, 42:268, 42:311, 42:314, 42:316, 42:317, 42:681
\@list *909*
\@listctr 39:233, 39:256, 47:9, *908*
\@listdepth 39:23, 39:35, 39:38, 39:43, 39:99, 40:519, *908*
\@listfiles 20:62, 20:131, 20:186, 20:772, 20:791
\@listi *909*
\@listii *909*
\@listvi *909*
\@lnbk *1398*
\@loadwithoptions 50:729, 50:751, 50:761, 50:770
\@lowpenalty 18:118, 57:3
\@ltab 41:71, 41:106
\@ltxnomath *1373*
\@m 18:325, 18:358, 18:526, 18:531, 20:45, 20:115, 20:173, 39:80, 42:224, 42:228, 47:37, 02:21, 02:304, 02:306, 02:307, 02:373, 02:374, *421*
\@mainaux 20:3, 20:37, 20:38, 20:107, 20:108, 20:165, 20:166, 20:236, 20:318, 20:361, 20:380, 20:413, 20:427, 20:452, 37:23, 37:90, 37:152, *1384*
\@make@normalcolbox 54:502, 54:508, 54:797
\@make@specialcolbox 54:500, 54:517, 54:798
\@makebox 40:11, 40:20, 40:25
\@makecaption 45:24, *1320*
\@makecol . . . 54:266, 54:418, 54:465, 54:489, 54:490, 54:735, 54:737, *1233*
\@makecol@handlesplitfootnotes 54:613, 54:628
\@makefcolumn 54:398, 54:399, 54:407, 54:409, 54:445, 54:447, 54:455, 54:457, 54:2837, 54:2839, 54:2855, 54:2856
\@makefnmark 45:400, 45:594
\@makefntext 40:552, 40:569, 40:586, 45:538, 45:556, 45:574
\@makeother 01:110, 24:503, 24:504, 24:505, 24:506, 24:507, 24:508, 24:509, 24:510, 24:511, 24:512, 24:513, 37:543,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

37:564, 37:709, 37:724, 37:734, 50:473, 50:474, 06:892, 06:893, 50:1344, 50:1477, 50:1566, 54:863, 54:864, 54:865, 54:866, 54:867, 54:868, 54:869, 54:870, 54:871, 54:872, 54:873, 54:874, 01:60, 01:81
\@makepicbox 40:10, 40:19, 40:67, 42:377, 42:435
\@makespecialcolbox 54:757, 54:770, *1363*
\@marbox 45:320, 45:322, 45:326, 45:330, 45:331, 45:379, 54:2389, 54:2399, 54:2402, 54:2410, 54:2412, 54:2413, 54:2415, 54:2416, 54:2417, 54:2426
\@marginparreset 45:343, 45:361, 45:370
\@markright . 49:52, 49:75, 49:98, 49:121
\@math@level 24:404, 24:422, 24:429, 24:434, 28:288, *584*
\@maxdepth 20:60, 20:129, 20:184, 54:67, 54:504, 54:603, 54:742, 54:767, 57:152, *1391*
\@maxtab 41:2, 41:94, *1374*
\@medpenalty 18:119, 57:3
\@meta@family@list 29:117, 29:162, 29:175, 29:236, 29:246, 29:289, 29:585, *741*
\@midlist 54:42, 54:496, 54:497, 54:753, 54:754, 54:1378, 54:1380, 54:1493, 54:1642, 54:1801, 54:2533, 54:2561
\@minipage... . . . . *1396*
\@minipagefalse 39:212, 40:483, 40:485, 40:532, 45:187, 45:250, 45:345, 45:363
\@minipagerestore 40:520, 40:522
\@minipagetrue 40:484, 45:186
\@minus 06:26, 54:3007, 54:3008, 54:3009, 54:3012, 54:3013
\@missing@onefilewithoptions 50:969, 50:1025, 50:1143
\@missingfile@area 20:665, 20:706, 20:719, 50:1027
\@missingfile@base 20:665, 20:707, 20:720, 50:1028
\@missingfile@ext 20:665, 20:708, 20:721, 50:1029
\@missingfileerror 20:660, 20:675, 20:685, 20:694, 50:1026, 50:1123, *505*
\@mkboth 49:11, 49:13
\@mklab 39:45, 39:171
\@mkpream 41:241, 41:264, 41:311, 41:339, *1325*
\@mparbottom 45:387, 45:388, 54:96, 54:483, 54:2400, 54:2408, 54:2409, 54:2410, 54:2411
\@mpargs 40:503, 40:535
\@mparswitchfalse 54:80
\@mpfn 40:517, 45:514, 45:519, 45:603, 45:607
\@mpfootins 40:509, 40:526, 40:527, 40:530, 40:536, 40:543, 40:544, 40:561, 40:562, 40:578, 40:579
\@mpfootnotetext 40:518, 40:538
\@mplistdepth 40:519, 40:536
\@multicnt 41:447, 41:449, 41:450, 41:451, 41:458, 41:459, 41:460, 42:114, 42:115, 42:117, 42:126, 42:127, 42:129, 42:677, 42:714, 42:716, 42:717, 42:718, 42:720, 42:721, 42:727, 42:733, 42:744, 42:748, 42:774, 42:776, 42:778, 42:780, 42:781, 42:785, 42:789, 42:800, 42:804, *1393*
\@multiplelabels . . . 20:33, 20:103, 20:161, 35:87, 35:93, 36:154, 37:62, 37:68, 37:127, 37:133, 37:166, 37:172
\@multiput 42:97, 42:106, 42:109
\@multispan 41:448, 41:452, 41:456
\@mypkg@name *1150*
\@mypkg@other@name *1150*
\@namedef 20:465, 24:136, 24:137, 24:161, 24:203, 25:6, 25:1436, 25:2913, 26:435, 28:473, 28:477, 28:486, 06:50, 33:57, 33:830, 35:90, 37:289, 37:349, 37:366, 37:380, 37:586, 37:595, 38:519, 38:520, 41:199, 43:12, 43:13, 43:18, 43:19, 43:28, 43:29, 43:36, 43:37, 43:38, 50:1257, 50:1392, 52:482, 57:529, 57:530, *1378*
\@nameuse . . . 20:355, 20:411, 20:450, 20:464, 06:51, 43:36, 49:5, 50:963, 52:486, 54:888, 54:962, 54:1020, *1378*
\@nbitem 39:199, 39:252
\@ne 02:16, *1378*
\@needsf@rmat . . 50:797, 50:800, 50:805
\@needsformat . . 50:785, 50:795, 50:799
\@negargfalse 42:197
\@negargtrue 42:196
\@newcommand 06:97, 06:98
\@newctr 22:13, 22:32, 43:8
\@newenv 06:191, 06:192, 06:201
\@newenva 06:190, 06:189
\@newenvb 06:192, 06:191
\@newfontswitch *1367*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\@newl@bel 35:84, 37:25, 37:92, 37:154, 47:10, *1390*
\@newline 18:95, 18:97
\@newlistfalse 39:29, 39:33, 39:108, 39:213, 54:878, 54:955, 54:1013
\@newlisttrue 39:29, 39:33, 39:87
\@next 45:60, 45:129, 45:319, 45:320, 54:5, 54:218, 54:316, 54:1228, 54:1248, 54:2389, 02:161
\@nextchar 41:346, 41:347, 41:407, 41:408, 41:409
\@nil 03:13, 03:19, 03:83, 03:104, 03:105, 03:117, 03:118, 03:165, 03:166, 03:167, 13:13, 13:19, 13:27, 19:14, 01:145, 01:146, 20:266, 20:267, 20:268, 21:107, 21:128, 21:1020, 21:1024, 21:1091, 21:1103, 21:1105, 24:455, 24:466, 24:582, 24:597, 24:701, 24:704, 24:705, 24:713, 25:2819, 25:2821, 25:2853, 25:2855, 25:3165, 25:3167, 25:3191, 25:3193, 26:367, 26:368, 26:370, 26:383, 26:389, 26:393, 26:394, 26:430, 26:451, 26:456, 26:536, 26:550, 27:26, 27:44, 27:53, 27:57, 28:41, 28:643, 28:651, 28:684, 28:1300, 28:1302, 06:53, 06:54, 06:58, 06:63, 32:59, 32:63, 06:166, 06:176, 06:222, 06:543, 06:544, 41:444, 41:445, 06:725, 50:107, 50:108, 50:114, 50:122, 50:125, 50:136, 50:147, 50:175, 50:226, 50:227, 50:238, 50:239, 50:249, 50:250, 50:252, 50:482, 50:505, 06:868, 06:869, 50:549, 50:555, 50:671, 50:691, 50:695, 50:701, 50:705, 50:920, 50:929, 50:946, 50:947, 50:1675, 50:1680, 50:1683, 50:1685, 50:1686, 50:1702, 50:1722, 50:1739, 50:1794, 50:1816, 50:1840, 52:168, 52:176, 52:369, 52:411, 52:413, 57:794, 57:795
\@nmbrlistfalse 39:33, 39:46, 39:91, *1360*
\@nmbrlisttrue 39:256
\@nnil 13:13, 13:20, 13:21, 13:22, 13:28, 20:267, 20:268, 24:277, 24:281, 24:282, 24:283, 24:298, 26:180, 26:182, 26:362, 26:364, 26:376, 26:378, 26:383, 26:397, 26:399, 26:406, 26:417, 26:418, 26:420, 26:451, 26:456, 06:521, 06:527, 42:13, 50:865, 50:866, 50:873, 50:893, 50:894, 50:901
\@no@font@optfalse 27:17, 27:129
\@no@lnbk 18:9, 18:10, 18:41, *1398*
\@no@pgbk 18:7, 18:8, 18:33, *1391*
\@nobreak... . . . . *1397*
\@nobreakfalse 18:121, 18:123, 39:224, 44:94, 44:129, 44:157, 45:182, 54:144, 54:171, 54:197, 54:1502, 54:1651, 54:1812, *1397*
\@nobreaktrue . . 18:122, 44:126, 45:181
\@nocnterr 14:240, *1372*
\@nocounterr 14:240, 22:4, 22:8, 22:16, 22:33, 22:189, 22:191, 43:25, 43:34, *1372*
\@nodocument 14:245, 16:145, 20:68, 20:137, 20:192, 20:315, 20:377, 37:243, 45:39, 45:108, 54:135, 54:162, 54:188, 54:217, *1398*
\@noitemargfalse 39:32, 39:231
\@noitemargtrue 39:32, 39:174
\@noitemerr 14:280, 18:286, 18:303, 18:373, 18:395, 39:69, 39:81, 39:107, *1433*
\@noitemerror *1384*
\@noligs 37:544, 37:565, 37:725, 37:735, 37:746, *1396*
\@nolnbk *1398*
\@nolnerr 14:238, 18:43, 18:114, 37:419, 37:426, *1374*
\@nomath 24:3, 24:400, 29:634, 29:669, 29:675, 29:685, 29:724, 29:726, 29:748, *1358*
\@noparitemfalse 39:30, 39:176
\@noparitemtrue 39:30, 39:66
\@noparlistfalse 39:31, 39:70
\@noparlisttrue 39:31, 39:67
\@nopgbk *1391*
\@normalcr . . 18:53, 18:93, 40:482, *1413*
\@normalsize 50:4, 50:5, *1372*
\@normalsize␣check *1371*
\@noskipsecfalse . . . 20:54, 20:124, 20:179, 44:98, 54:137, 54:164, 54:190
\@noskipsectrue 44:38, 44:95
\@notdefinable 14:232, 06:177, 06:178, 06:182, 06:570, 06:611
\@notprerr 14:283, 20:66, 20:135, 20:190
\@nthm 43:3, 43:4
\@nxttabmar 41:11, 41:21, 41:23, 41:25, 41:75, 41:111, 41:112, 41:118, 41:119, *1393*
\@obsoletefile 20:757
\@oddfoot 49:11, 49:14, 49:15, 54:102, 54:893, 54:965, 54:1024
\@oddhead 49:11, 49:14, 54:101, 54:892, 54:965, 54:1024

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\@onefilewithoptions 50:857, 50:861, 50:867, 50:885, 50:889, 50:895, 50:912, 50:916, 50:922, 50:938, 50:940, 50:1023, 50:1090, 50:1644, *1424*
\@onefilewithoptions@clashchk 50:955, 50:1005
\@onelevel@sanitize 45:42, 45:111, 06:894, *1382*
\@onfilewithoptions *1127*
\@onlypreamble 08:2852, 20:197, 20:206, 20:262, 20:759, 20:819, 21:41, 21:42, 21:79, 21:80, 21:84, 21:143, 21:163, 21:207, 21:208, 21:224, 24:18, 24:116, 24:118, 24:124, 24:140, 24:168, 24:183, 24:266, 24:271, 24:313, 24:609, 26:436, 27:28, 27:36, 27:42, 27:79, 27:83, 27:88, 27:93, 27:98, 27:108, 27:126, 27:127, 27:128, 27:134, 27:138, 27:142, 28:18, 28:20, 28:45, 28:47, 28:108, 28:117, 28:137, 28:466, 28:467, 28:480, 28:526, 28:574, 28:586, 28:588, 28:601, 28:626, 28:683, 28:685, 28:727, 28:766, 28:782, 28:859, 28:953, 28:962, 28:1018, 28:1021, 28:1024, 28:1044, 28:1057, 28:1111, 28:1146, 28:1157, 28:1171, 28:1225, 28:1245, 28:1249, 28:1313, 06:66, 32:141, 32:142, 33:831, 06:229, 06:231, 35:92, 06:240, 06:248, 46:12, 46:29, 47:64, 47:81, 48:16, 48:400, 50:10, 50:13, 50:102, 50:129, 50:155, 50:157, 50:189, 50:392, 50:455, 50:461, 50:526, 50:529, 50:530, 50:541, 50:542, 50:543, 50:570, 50:576, 50:589, 50:626, 50:644, 50:664, 50:684, 50:708, 50:713, 50:717, 50:720, 50:728, 50:749, 50:752, 50:762, 50:781, 50:794, 50:799, 50:805, 50:841, 50:846, 50:934, 50:1023, 50:1148, 50:1157, 50:1165, 50:1166, 50:1184, 50:1212, 50:1221, 50:1228, 50:1229, 50:1237, 50:1242, 50:1247, 50:1623, 50:1624, 50:1625, 50:1626, 50:1628, 51:229, *1422*
\@opargbegintheorem 43:59, 43:62
\@opcol 54:267, 54:275, 54:399, 54:418, 54:447, 54:465, 54:470, *1085*
\@options 50:663
\@othm 43:3, 43:21
\@outerparskip 39:1, 39:88, 39:117, 39:183, 39:253
\@outputbox 48:418, 48:421, 48:425, 48:450, 48:453, 48:457, 54:98, 54:492, 54:509, 54:511, 54:512, 54:528, 54:531, 54:536, 54:537, 54:544, 54:550, 54:552, 54:564, 54:602, 54:604, 54:605, 54:739, 54:741, 54:759, 54:761, 54:762, 54:771, 54:773, 54:774, 54:779, 54:781, 54:782, 54:788, 54:790, 54:928, 54:991, 54:1050, 54:1074, 54:1080, 54:1090, 54:1091, 54:1114, 54:1121, 54:1207, 54:1210, 54:1213, 54:1219, 54:1220, 54:2879, 54:2883, 54:2889, 54:2908, 54:2911, 54:2912, 54:2926, 54:2932, 54:2949, 54:2955, 54:2964, *1233*
\@outputbox@append 54:525, 54:567, 54:573, 54:601, 54:609, 54:615, 54:638, 54:709, 54:713, 54:802, *1233*
\@outputbox@appendfil 54:609, 54:666, 54:668, 54:677, 54:684, 54:690, 54:692, *1233*
\@outputbox@appendfootnotes 54:611, 54:671, 54:675, 54:686, 54:695, 54:699, 54:703, 54:718, 54:803, *1233*
\@outputbox@attachbottomfloats 54:629, 54:806, *1233*
\@outputbox@attachfloats 54:629, 54:672, 54:679, 54:682, 54:694, 54:698, 54:704, 54:719, 54:804, 54:1069, *1233*
\@outputbox@attachtopfloats 54:629, 54:805, *1233*
\@outputbox@depth . 54:507, 54:511, 54:513, 54:525, 54:590, 54:591, 54:593, 54:596, 54:598, 54:604, *1245*
\@outputbox@reinsertbskip 54:565, 54:571, 54:669, 54:678, 54:685, 54:693, 54:700, 54:705, 54:717, 54:800, *1233*
\@outputbox@removebskip 54:493, 54:563, 54:799, *1244*
\@outputdblcol 54:474, 54:2874, 54:2876, 54:2904, 54:2905, 54:2945, 54:2946
\@outputpage 54:408, 54:457, 54:477, 54:841, 54:2892, 54:2897, 54:2935, 54:2939, 54:2971, 54:2979, *1258*
\@oval 42:464, 42:481
\@ovbtrue 42:486, 42:514, 42:544
\@ovdx 42:442, 42:497, 42:499, 42:505, 42:507, 42:526, 42:528, 42:536, 42:538, 42:553, 42:561, 42:563, 42:599, 42:602, 42:609, 42:611,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

42:703, 42:704, 42:705, 42:706, 42:722, 42:723, 42:724, 42:727, 42:743, 42:763, 42:764, 42:765, 42:766, 42:782, 42:783, 42:785, 42:799
\@ovdy 42:442, 42:498, 42:500, 42:506, 42:507, 42:527, 42:529, 42:537, 42:538, 42:554, 42:562, 42:563, 42:574, 42:579, 42:587, 42:591, 42:710, 42:711, 42:712, 42:713, 42:728, 42:729, 42:730, 42:733, 42:747, 42:770, 42:771, 42:772, 42:773, 42:786, 42:787, 42:789, 42:803
\@ovhlinefalse 42:487, 42:515
\@ovhlinetrue 42:467, 42:471, 42:475, 42:493, 42:499, 42:521, 42:528
\@ovhorz 42:504, 42:505, 42:535, 42:536, 42:560, 42:561, 42:594
\@ovltrue 42:486, 42:514, 42:544
\@ovri 40:33, 42:442, 42:496, 42:525, 42:552, 42:574, 42:587, 42:603, 42:612
\@ovro 42:442, 42:496, 42:505, 42:506, 42:525, 42:536, 42:537, 42:552, 42:561, 42:562, 42:573, 42:579, 42:586, 42:591, 42:598, 42:608, 42:624, 42:631, 42:640, 42:649
\@ovrtrue 42:486, 42:514, 42:544
\@ovttrue 42:486, 42:514, 42:544
\@ovvert 42:502, 42:503, 42:531, 42:533, 42:556, 42:558, 42:567
\@ovvlinefalse 42:487, 42:515
\@ovvlinetrue 42:470, 42:492, 42:500, 42:520, 42:529
\@ovxx 42:442, 42:490, 42:492, 42:493, 42:497, 42:503, 42:504, 42:518, 42:520, 42:521, 42:526, 42:533, 42:535, 42:547, 42:549, 42:553, 42:558, 42:560, 42:598, 42:608, 42:700, 42:701, 42:702, 42:706, 42:715, 42:716, 42:725, 42:726, 42:727, 42:742, 42:760, 42:761, 42:762, 42:766, 42:775, 42:776, 42:784, 42:785, 42:798
\@ovyy 42:442, 42:491, 42:492, 42:493, 42:498, 42:505, 42:519, 42:520, 42:521, 42:527, 42:536, 42:548, 42:549, 42:554, 42:561, 42:571, 42:584, 42:707, 42:708, 42:709, 42:713, 42:715, 42:731, 42:732, 42:733, 42:746, 42:767, 42:768, 42:769, 42:773, 42:775, 42:788, 42:789, 42:802
\@p@pfilename 50:107, 50:114, 50:122, 50:125, 50:135, 50:136, 50:145, 50:147, 50:155
\@p@pfilepath . . 50:108, 50:173, 50:182
\@p@pfilepath@aux 50:174, 50:175, 50:183
\@pagedp . . . 54:95, 54:313, 54:318, 54:1442, 54:1740, 54:2418, 54:2428
\@pageht 54:94, 54:314, 54:318, 54:320, 54:321, 54:322, 54:326, 54:1441, 54:1592, 54:1739, 54:2401, 54:2408
\@par 15:3, 15:5
\@parboxrestore 40:373, 40:398, 40:424, 40:482, 40:516, 40:547, 40:565, 40:582, 45:19, 45:100, 45:169, 45:342, 45:360, 45:532, 45:551, 45:569, 54:224, 54:879, 54:956, 54:1014, *1397*
\@parboxto 40:365, *1405*
\@parmoderr 14:273, 45:58, 45:127, 45:316
\@parse@version 03:83, 03:104, 03:105, 03:117, 03:118, 03:165, 03:166, 03:167, 50:232, 50:238, 50:239, 50:249, 50:1686, 50:1702, 50:1739, 50:1816, 50:1840
\@parse@version@ 50:226, 50:227, 50:232, 50:244
\@parse@version@dash 50:250, 50:252, *1410*
\@partaux 20:3, 20:242, 20:286, 20:304, 20:328, 20:330, 20:331, 20:351, 20:390, 20:392, 20:393, 20:407, 20:437, 20:439, 20:440, 20:446, 20:458, 20:460, 20:463
\@partlist 20:251, 20:255, 20:256, 20:258, 20:282, 20:301, 20:324, 20:386, 20:433
\@partswfalse 20:6
\@partswtrue . . . 20:250, 20:280, 20:300
\@pass@ptions 50:478, 50:480, 50:500, 50:501, 50:503, 50:516, 50:517, 50:519, 50:526, 50:527, 50:528, 50:1034, 50:1116
\@pboxswfalse . . 40:396, 40:422, 40:501
\@pboxswtrue 40:406, 40:432
\@pdef *1367*
\@penup 38:199, 38:200
\@percentchar . . . 50:1336, 50:1338, 50:1340, 50:1342, 50:1469, 50:1471, 50:1473, 50:1475, 50:1558, 50:1560, 50:1562, 50:1564, 50:1612, 01:90
\@pgbk *1391*
\@picbox 42:6, 42:42, 42:54, 42:62, 42:63, *1359*
\@picht 42:6, 42:40, 42:53, 42:62
\@picture 42:34, 42:35
\@picture@warn 42:234, 42:452, 42:456, 42:460

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\@pkgetension *1367*
\@pkgextension 50:31, 50:190, 50:198, 50:256, 50:527, 50:719, 50:722, 50:761, 50:770, 50:868, 50:896, 50:923, 50:1045, 50:1072, 50:1193, 50:1223, 50:1658, 52:482, 52:492, *1126*
\@plus 18:554, 06:26, 44:16, 44:231, 44:254, 49:127, 54:3007, 54:3008, 54:3009, 54:3012, 54:3013, 54:3017, 54:3018, 54:3019, 54:3023, 54:3024, 54:3025
\@pnumwidth 44:243, 44:266
\@popfilename 50:33, 50:1003, 50:1138, *1123*
\@pr@videpackage 50:391, 50:405, 50:408, 50:429, 50:431, 50:442, 50:444, 50:455
\@preamble . . 41:242, 41:244, 41:252, 41:265, 41:267, 41:275, 41:314, 41:333, 41:335, 41:336, 41:340, 41:355, 41:373, 41:374, 41:411, 55:351
\@preamblecmds 20:67, 20:136, 20:191, 06:66, 50:1854, 50:1855, *1056*
\@preamerr 14:260, 41:251, 41:274, 41:351, 41:432
\@process@pti@ns 50:588, 50:606, 50:623, 50:630, 50:631, 50:644, 50:648, 50:650
\@process@ptions 50:575, 50:577, 50:589
\@protect@... . . . . *1378*
\@protected@testopt 06:107, 06:119, 06:720, 06:732
\@protecteddef *1373*
\@providesfile 50:465, 57:788, 01:82, 01:83
\@ptionlist 50:187, 50:259, 50:574, 50:1011, 50:1017, 50:1106, 50:1112, 50:1224
\@pushfilename 50:33, 50:948, 50:1093, *1100*
\@put 42:463, 42:507, 42:538, 42:563, 42:631, 42:649
\@qend 14:236, 06:177, 06:868
\@qrelax 06:178, 06:868
\@raw@classoptionslist 50:11, 50:855, 51:98, 51:140
\@rc@ifdefinable 21:32, 06:171, 06:173, 06:295, 06:316, 06:342
\@reargdef 06:161, *1367*
\@refundefined 20:55, 20:125, 20:180, 35:3, 37:60, 37:125, 37:164, *1393*
\@reinserts 54:332, 54:335, 54:819
\@remove@eq@value 50:544, 50:611, 50:671
\@remove@tlig 21:1020, 21:1028
\@remove@tlig@ 21:1020, 21:1021
\@remove@tlig@@ 21:1021, 21:1024
\@remove@tlig@@@ 21:1034, 21:1053
\@removeelement . 13:32, 50:670, 50:679
\@removefromreset 22:148, 22:198, 22:199, 22:262, 22:274, 22:286, 22:289, *561*
\@renewfontswitch *1370*
\@reqcolroom 54:1441, 54:1442, 54:1445, 54:1447, 54:1448, 54:1453, 54:1454, 54:1458, 54:1460, 54:1488, 54:1489, 54:1592, 54:1595, 54:1597, 54:1598, 54:1603, 54:1607, 54:1609, 54:1637, 54:1638, 54:1739, 54:1740, 54:1744, 54:1747, 54:1748, 54:1753, 54:1760, 54:1762, 54:1794, 54:1795, 54:1906, 54:1908, 54:1910, 54:1911, 54:1914, 54:1916, 54:1988, 54:1990, 54:1992, 54:1995, 54:1997, 54:2070, 54:2073, 54:2076, 54:2081, 54:2083, 54:2597, 54:2728, 54:2729, 54:2734, 54:2737, 54:2779, 54:2784, 54:2787, *1269*
\@reserveda *658*
\@reset@ptions . . 50:967, 50:1004, 50:1032, 50:1097, 50:1139, 50:1149
\@resetactivechars 54:826, 54:851, 54:953, 54:1011
\@resethfps 54:1557, 54:1702, 54:1870, 54:2674, *1374*
\@resetprotect *1388*
\@restoremetafamily 29:228, *742*
\@restorepar . 15:5, 18:410, 18:426, 18:444, 18:460, 39:127, 39:137, *1390*
\@reversemarginfalse . . 45:388, 54:79
\@reversemargintrue 45:387
\@rightmark 49:16, 49:120
\@rightskip 37:458, 37:477, 37:495, 39:75, 40:455, 40:476
\@rjfieldfalse 41:34, 41:77
\@rjfieldtrue 41:125
\@rmfamilyhook 29:508, 29:530, 29:542, 29:569
\@roman 22:304, 22:310
\@rsbox 40:673, 40:680, 40:684
\@rtab 41:71, 41:86
\@rule 40:597, 40:602, 40:606
\@sanitize 46:7, 46:18, 46:24, 46:35, 06:892
\@savebox 40:166, 40:173, 40:179
\@savemarbox 45:326, 45:327, 45:330, 45:333
\@savepicbox . . . 40:166, 40:173, 40:183

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\@savsf 17:121, 17:123, 17:146, 17:148, 17:169, 17:171, 18:124, 18:130, 18:139, 18:162, 18:180, 18:194, 18:206, 18:220, 18:234
\@savsk 18:124, 18:129, 18:140, 18:163, 18:181, 18:195, 18:207, 18:221, 18:235
\@scolelt 54:1139, 54:1204
\@sdblcolelt 54:1156, 54:1176, 54:1205
\@seccntformat . . . 44:60, 44:111, *1361*
\@secondoffive 35:24, 35:27, 35:41, 35:58
\@secondoftwo 03:87, 03:185, 17:5, 20:528, 20:558, 20:604, 20:621, 21:149, 22:357, 22:362, 24:221, 24:241, 05:41, 05:45, 29:591, 33:800, 33:850, 33:866, 35:44, 35:80, 06:253, 06:300, 06:321, 06:549, 06:678, 06:729, 49:17, 50:194, 06:837, 50:228, 50:240, 06:844, 50:273, 50:291, 50:340, 50:370, 06:864, 06:911, 50:1845, 54:646, 54:651, 54:658, 57:802, 01:72
\@secpenalty 18:18, 44:36, 44:50
\@sect 44:54, 44:55
\@seqncr 38:518
\@set@curr@file@aux 52:363, *1428*
\@setckpt 20:458, 20:465, 37:24, 37:91, 37:153, *1391*
\@setfloattypecounts . . . 54:1425, 54:1576, 54:1720, 54:1891, 54:1973, 54:2053, 54:2148, 54:2269, 54:2621
\@setfontsize 29:748
\@setfps 45:34
\@setfpsbit . . . 45:73, 45:75, 45:77, 45:85, 45:143, 45:146, 45:149, 54:2665
\@setmarks . . 54:2918, 54:2920, 54:2934
\@setminipage 40:521, 45:21, 45:177, 45:185, 45:376, *1385*
\@setminpage *1398*
\@setnobreak 45:179, 45:375, *1385*
\@setpar 15:3, 39:78
\@setprotect *1388*
\@setref 35:10
\@setsize 29:748
\@settab 41:71, 41:93
\@settodim 23:17
\@settopoint 23:35
\@setupverbvisiblespace . . 37:587, 37:599, 37:603, 37:664, 37:678, 37:692
\@setupverbvisibletab 37:615, 37:636
\@sffamilyhook 29:508, 29:535, 29:543, 29:570
\@sharp 41:248, 41:271, 41:312, 41:342, 41:357, 41:358, 41:378, 41:380, 41:382, 41:410
\@shipoutsetup 54:841
\@shortstack 42:143, 42:144
\@show@... . . . . *103*
\@show@DeclareRobustCommand 06:646, 06:660, 06:705, 06:759, 06:762
\@show@DeclareRobustCommand@env 06:783, 06:785
\@show@environment . . . 06:777, 06:778
\@show@environment@begin 06:786, 06:792, 06:800, 06:805
\@show@environment@end 06:790, 06:799, 06:804, *107*
\@show@environment@end@aux 06:808, 06:810
\@show@macro 06:705, 06:795
\@show@newcommand . 06:647, 06:660, 06:713, 06:740, 06:759, 06:765, *105*
\@show@newcommand@aux 06:740, 06:769, 06:787, *106*
\@show@newcommand@env 06:785, 06:784
\@show@nonstop . . . 06:795, 06:816, *108*
\@show@normalenv 06:781, 06:804
\@show@tokens 06:745, 06:750, 06:794, 06:811
\@show@typeout 06:789, 06:794, 06:816, *108*
\@showcommandlisthook 06:643, 06:645, 06:653, 07:1484, *103*
\@showenvironmentlisthook 06:780, 06:782, 07:1486, *171*
\@skipping@modulefalse 03:154, 03:173, 03:200
\@skipping@moduletrue 03:153, 03:177
\@sline . 42:178, 42:190, 42:195, 42:280
\@slowromancap 22:311, 22:312
\@spaces 14:218
\@specialoutput 54:261
\@specialpagefalse 54:75, 54:887, 54:962, 54:1020
\@specialpagetrue 49:9
\@specialstyle 49:9, 54:888, 54:962, 54:1020
\@sptoken 06:877, 06:887
\@sqrt 38:413
\@ssect 44:53, 44:112
\@stackcr 42:150, 42:153
\@star@or@long 06:72, 06:95, 06:163, 06:187, 06:193, 06:219, 06:228, 06:274
\@startcolumn . 54:268, 54:275, 54:1126
\@startdblcolumn 54:1126, 54:2896, 54:2899, 54:2938, 54:2940, 54:2977, 54:2983
\@startfield 41:28, 41:46, 41:92, 41:104, 41:125, 41:133

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\@startline 41:20, 41:57, 41:62, 41:68, 41:83, 41:154
\@startpagehook *1364*
\@startpbox 41:245, 41:268, 41:313, 41:343, 41:409, 41:461, 41:463
\@startsection 44:39
\@starttoc 44:149, *1318*
\@stopfield 41:32, 41:48, 41:86, 41:93, 41:125, 41:127, 41:140, 41:142, 41:154
\@stopline 41:30, 41:56, 41:85
\@stpelt 22:37, 22:40
\@string@makeletter 06:898, *111*
\@strip@args 21:94
\@strip@tex@ext 20:246, 20:256, 20:258, 20:263, 20:293, *493*
\@strip@tex@ext@aux . . 20:263, 20:294
\@svector 42:255, 42:270, 42:280
\@sverb 37:648, 37:727, 37:736, 37:739, *885*
\@svsec 44:57, 44:60, 44:66, 44:78, *1361*
\@svsechd 44:76, 44:101, 44:121
\@swaptwoargs 20:638, 20:640, 50:977, 52:153, 52:167, 52:187
\@sxverbatim . . . 37:520, 37:588, 37:595
\@tabacckludge 21:243, 21:245, 21:475, 21:476, 33:157, 33:164, 33:166, *1415*
\@tabacol 41:206, 41:213, 41:335
\@tabarray 41:174, 41:181, 41:208, 41:215, 41:218, *1441*
\@tabclassiv . . . 41:208, 41:215, 41:407
\@tabclassz 41:207, 41:214, 41:359
\@tabcr 41:56, 41:73
\@tabfbox 41:16, 41:80, 41:82
\@tablab 41:72, 41:126
\@tabminus 41:72, 41:117
\@tabplus 41:72, 41:110
\@tabpush 41:11, 41:77, 41:85, 41:140, 41:143, 41:145
\@tabrj 41:72, 41:124
\@tabular 41:198, 41:201, 41:202
\@tabularcr 41:208, 41:215, 41:285
\@tempa *1385*
\@tempboxa 12:13, 21:87, 23:21, 23:22, 23:23, 23:28, 23:29, 39:236, 39:242, 39:243, 39:245, 40:29, 40:30, 40:31, 40:32, 40:37, 40:38, 40:39, 40:40, 40:219, 40:253, 40:272, 40:281, 40:291, 40:504, 40:535, 40:690, 40:691, 40:692, 40:699, 40:700, 40:701, 40:702, 42:315, 42:316, 42:458, 42:459, 42:496, 42:501, 42:506, 42:507, 42:525, 42:530, 42:537, 42:538, 42:552, 42:555, 42:562, 42:563, 42:624, 42:625, 42:630, 42:631, 42:640, 42:641, 42:648, 42:649, 42:734, 42:752, 42:790, 42:808, 44:138, 44:139, 45:322, 45:380, 54:310, 54:382, 54:387, 54:388, 54:429, 54:434, 54:435, 54:556, 54:794, 54:913, 54:925, 54:926, 54:981, 54:988, 54:989, 54:1040, 54:1047, 54:1048, 54:1072, 54:1076, 54:1088, 54:1094, 54:1101, 54:1102, 54:1103, 54:1104, 54:1108, 54:1116, *1360*
\@tempcnta 12:7, 28:1026, 28:1027, 28:1028, 28:1029, 28:1033, 41:319, 41:320, 41:321, 41:322, 42:198, 42:199, 42:225, 42:226, 42:227, 42:240, 42:241, 42:242, 42:243, 42:250, 42:251, 42:265, 42:266, 42:281, 42:282, 42:287, 42:289, 42:290, 42:291, 42:292, 42:293, 42:296, 42:298, 42:299, 42:300, 42:301, 42:302, 42:303, 42:304, 42:305, 42:306, 42:307, 42:346, 42:347, 42:348, 42:349, 42:350, 42:371, 42:372, 42:373, 42:374, 42:375, 42:376, 42:403, 42:404, 42:405, 42:406, 42:407, 42:425, 42:427, 42:429, 42:430, 42:432, 42:434, 42:449, 42:450, 42:451, 42:453, 42:455, 42:457, 42:459, 42:572, 42:577, 42:585, 42:589, 42:626, 42:627, 42:628, 42:629, 42:642, 42:643, 42:645, 42:646, 42:668, 42:669, 42:670, 42:671, 42:672, 42:673, 42:721, 42:741, 42:781, 42:797, 45:62, 45:68, 45:70, 45:79, 45:80, 45:90, 45:91, 45:131, 45:137, 45:139, 45:152, 45:153, 45:159, 45:160, 50:1284, 50:1286, 50:1287, 50:1288, 50:1415, 50:1417, 50:1418, 50:1419, 50:1529, 50:1531, 50:1532, 50:1533, 54:12, 54:14, 54:16, 54:1285, 54:1286, 54:1287, 54:1288, 54:1308, 54:1309, 54:1310, 54:1311, 54:1333, 54:1336, 54:1369, 54:1372, 54:1484, 54:1633, 54:1790, 54:2151, 54:2154, 54:2272, 54:2275, 54:2390, 54:2392, 54:2395, 54:2397, 54:2399, 54:2421, 54:2655, 54:2656, 54:2660, 54:2666, 54:2670, 57:227, 57:232, 57:233, 57:234, 57:413, 57:415, 57:416, 57:417, 57:424, 57:426, 57:427, 57:428, 57:434, 57:436, 57:437,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

57:438, 57:445, 57:447, 57:448, 57:449, 57:475, 57:477, 57:478, 57:479, 57:501, 57:503, 57:504, 57:505, 57:511, 57:513, 57:514, 57:515, 57:539, 57:544, 57:545, 57:546

\@tempcntb . . 12:7, 28:1027, 28:1031, 28:1033, 42:290, 42:291, 42:292, 42:294, 42:295, 42:296, 42:572, 42:573, 42:577, 42:578, 42:585, 42:586, 42:589, 42:590, 45:88, 45:89, 45:90, 45:157, 45:158, 45:159, 54:13, 54:16, 54:17, 54:2666, 54:2667, 54:2668, 57:228, 57:232, 57:540, 57:544

\@tempdima . . 12:10, 24:282, 24:287, 38:186, 38:189, 38:195, 40:47, 40:48, 40:62, 40:63, 40:252, 40:253, 40:271, 40:272, 40:279, 40:280, 40:281, 40:283, 40:372, 40:373, 40:397, 40:398, 40:423, 40:424, 40:502, 40:506, 40:609, 40:612, 40:613, 40:688, 40:690, 40:696, 40:699, 41:35, 41:36, 41:37, 41:88, 41:89, 41:90, 41:91, 41:295, 41:296, 42:221, 42:222, 42:224, 42:225, 42:226, 42:227, 42:228, 42:229, 42:448, 42:449, 42:450, 42:459, 42:497, 42:498, 42:502, 42:503, 42:526, 42:527, 42:531, 42:533, 42:553, 42:554, 42:556, 42:558, 42:627, 42:629, 42:644, 42:646, 42:647, 42:667, 42:668, 42:669, 44:236, 44:237, 44:259, 44:260, 44:273, 45:196, 45:198, 45:218, 45:220, 45:258, 45:259, 45:260, 54:234, 54:235, 54:236, 54:526, 54:528, 54:529, 54:534, 54:538, 54:542, 54:547, 54:551, 54:777, 54:779, 54:780, 54:783, 54:787, 54:789, 54:1268, 54:1271, 54:1291, 54:1301, 54:1313, 54:1323, 54:2214, 54:2215, 54:2218, 54:2219, 54:2339, 54:2340, 54:2344, 54:2345, 54:2400, 54:2401, 54:2402, 54:2403, 54:2406, 54:2409, 54:2412, 54:2414, 54:2828, 54:2829, 54:2831, 54:2832, *1361*

\@tempdimb 12:10, 24:283, 24:288, 24:721, 24:725, 26:180, 26:181, 26:454, 26:477, 26:478, 26:487, 26:488, 26:492, 26:510, 26:513, 26:516, 26:518, 40:375, 40:376, 40:400, 40:401, 40:426, 40:427, 40:610, 40:613, 40:689, 40:691, 40:697, 40:700, 42:222, 42:223, 42:368, 42:370, 42:373, 42:376, 42:492, 42:494, 42:495, 42:520, 42:523, 42:524, 42:549, 42:550, 42:551, 42:622, 42:623, 42:632, 42:638, 42:639, 42:650, 42:658, 42:659, 42:664, 54:1291, 54:1292, 54:1293, 54:1294, 54:1301, 54:1313, 54:1314, 54:1315, 54:1316, 54:1323, *1361*

\@tempdimc . . 12:10, 26:471, 26:472, 26:474, 26:475, 26:477, 26:478, 40:74, 40:75, 40:85, 40:86, 40:611, 40:612, 40:613, 42:41, 42:42, 42:43, 42:44, 42:45, 42:46, 42:71, 42:72, 42:74, 42:75, 42:343, 42:344, 42:345

\@tempdimx *1361*

\@tempskipa 12:14, 18:45, 18:48, 18:49, 18:322, 18:329, 18:331, 18:334, 18:355, 18:362, 18:364, 18:367, 26:182, 26:183, 39:116, 39:117, 39:118, 39:181, 39:183, 39:184, 39:185, 39:253, 39:254, 39:255, 44:42, 44:44, 44:45, 44:50, 44:62, 44:63, 44:88, 44:89, 44:91, 44:103, 44:104, 44:113, 44:114, 54:568, 54:569, 54:581, 54:2449, 54:2450, 54:2452, 54:2460

\@tempskipb 12:14, 18:243, 18:245, 18:247, 18:250, 18:252, 18:266, 18:281, 18:298, 18:320, 18:322, 18:323, 18:327, 18:329, 18:331, 18:332, 18:353, 18:355, 18:356, 18:360, 18:362, 18:364, 18:365, 18:387, 18:390, *476*

\@tempswa *1357*

\@tempswafalse . . . 20:322, 20:384, 20:431, 21:1565, 24:95, 24:772, 28:538, 28:613, 28:687, 28:768, 28:1288, 28:1294, 37:27, 37:93, 37:155, 37:535, 37:556, 47:17, 47:29, 50:1257, 50:1273, 50:1392, 50:1408, 52:286, 52:309, 52:330, 54:1339, 54:1375, 54:2157, 54:2278, 02:148, 01:62

\@tempswatrue 20:320, 20:325, 20:382, 20:387, 20:429, 20:434, 21:1568, 21:1569, 24:98, 24:773, 24:774, 24:777, 24:780, 28:541, 28:616, 28:690, 28:771, 28:1251, 36:174, 37:198, 37:540, 37:561, 47:17, 47:29, 50:1254, 50:1389, 52:286, 52:309, 52:330, 54:2159, 54:2182, 54:2280, 54:2305, 54:2739, 54:2756, 54:2789, 54:2806, 02:154, 01:63

\@temptokena 12:16, 37:210, 37:214, 37:223, 37:236, 37:237,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

49:42, 49:46, 49:53, 49:56, 49:68, 49:69, 49:76, 49:77, 49:91, 49:92, 49:99, 49:100, 49:122, 49:123, *871*
\@test... . . . . *1396*
\@test@opt *1392*
\@testdef . . 37:25, 37:92, 37:154, 37:196
\@testfalse 54:8, 54:10, 54:11
\@testfp 54:1233, 54:1253, 54:1289, 54:1312, 54:2658, 54:2842, 54:2859, *1384*
\@testopt 18:7, 18:8, 18:9, 18:10, 06:33, 06:97, 06:117, 06:121, 06:189, 38:489, *1392*
\@testpach 41:347, 41:425
\@testtrue 54:9, 54:17, 54:361, 54:1236, 54:1255, 54:1295, 54:1317, 54:2662
\@testwrongwidth 54:350, 54:1234, 54:1290, 54:1464, 54:1613, 54:1923, 54:2004, 54:2208
\@text@composite 21:94, 21:1098, 21:1103, *1386*
\@text@composite@x 21:94
\@textbottom 49:127, 49:129, 54:514, 54:539, 54:553, 54:643, 54:764, 54:784, 54:791, 54:824
\@textfloatsheight 54:483, 54:1438, 54:1440, 54:1491, 54:1492, 54:1497, 54:1589, 54:1591, 54:1640, 54:1641, 54:1646, 54:1736, 54:1738, 54:1798, 54:1800, 54:1806, 54:2597
\@textmin 45:285, 45:286, 45:299, 45:300, 54:90, 54:1440, 54:1444, 54:1447, 54:1448, 54:1591, 54:1594, 54:1597, 54:1598, 54:1738, 54:1743, 54:1747, 54:1748, 54:1910, 54:1992, 54:2076, 54:2175, 54:2177, 54:2193, 54:2297, 54:2299, 54:2317, 54:2715, 54:2717, 54:2719
\@textsubscript 45:455, 45:463
\@textsuperscript 45:401, 45:403, 45:404, *1389*
\@texttop 49:127, 49:129, 54:510, 54:760, 54:772, 54:824
\@tf@r 13:25, 13:26, *1370*
\@tfor 13:25, 20:605, 20:622, 20:774, 32:89, 40:78, 40:97, 41:345, 42:488, 42:516, 42:545, 45:63, 45:132
\@tforloop 13:27, 13:28, 13:30
\@thanks 44:11, 44:34
\@thefnmark 40:550, 40:567, 40:584, 45:400, 45:401, 45:515, 45:520, 45:535, 45:553, 45:571, 45:583, 45:588, 45:599, 45:604, 45:615
\@thefoot 54:102, 54:893, 54:897, 54:935, 54:965, 54:968, 54:995, 54:1024, 54:1027, 54:1054
\@thehead 54:101, 54:892, 54:896, 54:920, 54:965, 54:967, 54:985, 54:1024, 54:1026, 54:1044
\@themargin 54:50, 54:894, 54:898, 54:912, 54:966, 54:968, 54:980, 54:1025, 54:1027, 54:1039
\@themark 49:41, 49:42, 49:52, 49:53, 49:67, 49:68, 49:75, 49:76, 49:90, 49:91, 49:98, 49:99, 49:121, 49:123
\@thirdofthree 21:215, 06:257
\@thm . . 43:12, 43:18, 43:28, 43:37, 43:41
\@thmcounter 43:11, 43:17, 43:60
\@thmcountersep 43:10, 43:60
\@title 44:7, 44:31
\@tocrmarg 44:232, 44:255
\@toodeep . 14:253, 39:36, 39:263, 39:274
\@toplist 54:40, 54:389, 54:390, 54:436, 54:437, 54:634, 54:809, 54:1073, 54:1083, 54:1084, 54:1376, 54:1388, 54:2531, 54:2559
\@topnewpage 54:204, *1373*
\@topnum . . . 45:271, 54:83, 54:1373, 54:1374, 54:1388, 54:1392, 54:1955, 54:1960, 54:2036, 54:2041, 54:2128, 54:2135, 54:2522, 54:2550, 54:2591
\@toproom 45:273, 54:84, 54:1376, 54:1388, 54:2523, 54:2551
\@topsep . . . 39:1, 39:71, 39:73, 39:202
\@topsepadd 39:1, 39:59, 39:61, 39:71, 39:124
\@totalleftmargin 37:531, 37:553, 39:9, 39:53, 39:54, 40:454, 40:475, 41:35, 41:76, 41:81, *908*
\@trivlist 39:48, 39:57, 39:92
\@tryfcolumn 54:1129, 54:1149, 54:1167, 54:1183, 54:2843, 54:2860
\@trylist 54:1192, 54:1195, 54:1228, 54:1248, 54:1270
\@ttfamilyhook 29:508, 29:540, 29:544, 29:571
\@twoclasseserror 50:710, 50:1243
\@twocolumnfalse 54:77, 54:126
\@twocolumntrue 54:211
\@twoloadclasserror 50:1002, 50:1137, 50:1238
\@twosidefalse 54:78
\@typein 06:32, 06:33, 06:40, 06:48, *1392*
\@typeset@protect 21:44, 21:50, 21:230, 21:238, 29:749, 06:120, 37:282, 37:319, 06:347, 06:354, 06:356, 57:407, *1384*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\@uclclist 21:1534, 21:1535, 21:1592, 57:620, *1400*

\@undefind 42:68, 42:82, 42:93, 42:102, 42:173, 42:185, 42:248, 42:263, 42:324, 42:382

\@undefined 02:563, 02:578, 01:92, 01:93, 01:94, 03:53, 03:62, 10:213, 10:214, 10:215, 10:217, 10:218, 10:219, 10:221, 10:222, 10:224, 10:225, 10:226, 10:227, 10:228, 10:229, 10:230, 10:231, 10:232, 04:2, 04:15, 04:16, 04:17, 04:29, 04:30, 01:115, 04:74, 04:84, 01:123, 14:28, 04:173, 01:131, 04:189, 04:197, 04:205, 16:172, 16:173, 16:174, 16:175, 16:176, 04:236, 04:237, 04:238, 04:239, 04:240, 17:62, 04:241, 04:242, 04:243, 04:244, 04:245, 04:246, 04:247, 04:248, 04:249, 04:250, 04:251, 01:138, 04:252, 17:181, 04:253, 04:254, 04:260, 18:90, 18:519, 18:568, 18:574, 20:61, 20:62, 20:130, 20:131, 20:185, 20:186, 20:293, 20:294, 20:496, 20:497, 20:498, 20:503, 20:557, 20:570, 20:579, 20:594, 20:649, 21:213, 21:215, 21:353, 21:355, 21:357, 21:359, 21:361, 21:363, 21:365, 21:367, 21:369, 21:371, 21:390, 21:392, 21:394, 21:480, 21:723, 21:726, 21:1007, 21:1016, 21:1056, 21:1060, 22:188, 22:371, 24:531, 24:571, 24:633, 24:666, 24:730, 24:737, 01:189, 01:193, 25:2776, 25:2799, 25:2808, 25:2897, 25:2898, 25:2899, 25:2900, 25:2901, 25:2902, 25:2903, 25:2904, 25:2905, 25:2906, 25:2917, 25:2922, 25:2927, 25:2934, 25:2935, 25:2936, 25:2937, 25:2938, 25:2939, 25:2940, 25:3154, 25:3234, 25:3236, 25:3237, 25:3239, 25:3240, 25:3241, 25:3243, 05:4, 05:10, 05:16, 05:17, 05:18, 05:19, 05:20, 01:219, 26:569, 26:570, 27:4, 27:5, 27:6, 27:7, 27:8, 27:9, 27:10, 27:11, 27:12, 27:13, 27:14, 27:15, 27:16, 27:17, 27:18, 27:19, 27:20, 28:142, 01:226, 28:581, 28:633, 28:884, 05:164, 05:165, 05:166, 05:167, 05:168, 05:169, 28:991, 05:186, 05:187, 05:188, 05:189, 29:36, 29:59, 29:72, 29:92, 29:105, 29:106, 29:107, 29:108, 29:109, 29:110, 29:111, 29:112, 29:113, 29:114, 29:115, 29:117, 29:118, 29:119, 29:147, 29:148, 29:149, 29:150, 29:151, 29:152, 29:153, 29:154, 29:155, 05:209, 05:210, 29:289, 29:290, 29:291, 29:325, 29:383, 29:384, 29:551, 05:238, 29:569, 29:570, 29:571, 29:615, 29:616, 29:617, 29:618, 29:619, 29:620, 29:621, 29:661, 29:662, 29:663, 29:664, 29:665, 29:666, 29:677, 29:858, 30:15, 30:55, 30:62, 30:77, 06:34, 32:37, 32:38, 32:39, 32:123, 33:123, 33:331, 33:581, 33:584, 33:585, 33:615, 33:616, 33:617, 33:620, 33:621, 33:622, 33:623, 33:624, 33:625, 33:630, 33:631, 33:632, 33:633, 33:638, 33:639, 33:640, 33:641, 33:644, 33:645, 33:646, 33:648, 33:649, 33:654, 33:655, 33:656, 33:657, 33:658, 33:659, 33:660, 33:661, 33:662, 33:663, 33:664, 33:665, 33:666, 33:667, 33:668, 33:669, 33:671, 33:672, 33:674, 33:675, 33:676, 33:677, 33:678, 33:679, 33:680, 33:681, 33:682, 33:683, 33:685, 33:686, 33:687, 33:688, 33:689, 33:690, 33:691, 33:692, 33:693, 33:695, 33:696, 33:697, 33:698, 33:699, 33:700, 33:701, 33:702, 33:703, 33:704, 33:705, 33:706, 33:707, 33:708, 33:709, 33:710, 33:711, 33:712, 33:713, 33:714, 33:715, 33:716, 33:717, 33:718, 33:719, 33:720, 33:721, 33:726, 33:727, 33:729, 33:730, 33:731, 33:732, 33:733, 33:734, 33:735, 33:736, 33:738, 33:739, 33:741, 33:742, 33:744, 33:745, 33:746, 33:747, 33:749, 33:750, 33:751, 33:753, 33:755, 33:756, 33:757, 33:758, 33:759, 33:760, 33:761, 33:763, 33:764, 33:765, 33:766, 33:767, 33:768, 33:769, 33:770, 33:771, 35:172, 35:185, 35:225, 35:226, 06:276, 36:290, 36:291, 36:293, 36:294, 36:295, 36:297, 36:298, 36:299, 36:301, 36:302, 37:178, 37:230, 37:231, 37:232, 37:233, 37:308, 37:387, 37:407, 37:408, 37:409, 37:410, 37:517, 37:579, 37:597, 37:598, 37:599, 37:600, 37:633, 37:645,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

37:694, 38:236, 38:246, 38:247, 38:283, 38:286, 38:373, 38:386, 38:529, 06:411, 06:412, 06:443, 40:21, 06:471, 40:174, 06:475, 06:476, 40:238, 40:319, 40:359, 40:360, 40:361, 40:362, 06:493, 06:507, 40:603, 40:661, 40:662, 40:663, 40:664, 40:681, 01:310, 06:554, 06:555, 06:556, 01:311, 06:557, 06:558, 06:559, 06:560, 42:18, 06:596, 06:597, 06:598, 06:599, 06:600, 06:601, 42:477, 42:478, 06:632, 06:633, 06:634, 06:652, 06:653, 44:203, 44:204, 44:205, 44:206, 44:222, 44:270, 45:5, 45:460, 45:510, 45:624, 47:30, 47:53, 06:760, 06:761, 06:762, 06:763, 06:764, 06:765, 06:767, 06:768, 06:769, 50:4, 50:25, 06:824, 50:181, 50:182, 50:183, 06:839, 50:244, 06:846, 50:451, 50:561, 50:611, 50:1024, 50:1062, 50:1064, 50:1070, 50:1127, 06:920, 50:1142, 06:921, 50:1143, 06:922, 50:1158, 06:931, 50:1297, 50:1375, 50:1378, 50:1428, 50:1507, 50:1510, 50:1520, 50:1521, 50:1522, 50:1523, 50:1524, 50:1525, 50:1526, 50:1597, 50:1600, 50:1616, 50:1636, 50:1644, 52:18, 52:19, 52:20, 52:21, 52:147, 52:189, 52:190, 52:197, 52:198, 52:199, 52:320, 52:341, 52:355, 52:359, 52:360, 52:492, 52:498, 52:499, 52:500, 53:491, 53:492, 53:493, 53:494, 53:495, 53:497, 53:498, 53:499, 53:506, 53:507, 53:509, 53:511, 53:512, 53:513, 53:516, 53:560, 02:61, 54:373, 54:374, 54:797, 54:798, 54:799, 54:800, 54:802, 54:803, 54:804, 54:805, 54:806, 54:813, 54:814, 54:815, 02:77, 02:82, 54:2501, 54:2502, 54:2503, 54:2504, 54:2609, 02:89, 57:10, 57:18, 57:39, 57:54, 57:73, 57:82, 57:89, 57:98, 57:108, 57:160, 57:161, 57:272, 57:285, 57:298, 57:362, 57:374, 57:400, 57:401, 57:402, 57:431, 57:433, 57:472, 57:473, 57:492, 57:493, 57:494, 57:495, 57:496, 57:497, 57:498, 57:499, 57:500, 57:517, 57:533, 57:534, 57:535, 57:578, 57:579, 57:740, 57:775, 57:776, 57:777, 57:778, 57:779, 02:112, 02:142, 02:228, 02:238, 02:240, 01:52, 01:53, 02:350, 02:351, 02:361, 02:362, 02:456, 02:523, *1181*

`\@undefinedfonterror` *1374*

`\@unexpandable@protect` 20:215, 20:228, 06:273, 06:359, 06:365, 06:370, 41:341, *1383*

`\@unknownoptionerror` 50:1153, 50:1213, 50:1226

`\@unknownversion` *1363*

`\@unprocessedoptions` 50:660, 50:769, 50:1024, 50:1063, 50:1064, 50:1124, 50:1128, 50:1228, 51:88, *1397*

`\@unused` 14:15, 14:32, 14:59, 06:10, 06:17, 50:1621, 02:177, *1421*

`\@unusedoptionlist` 20:18, 20:20, 20:88, 20:90, 20:145, 20:147, 50:12, 50:553, 50:554, 50:564, 50:565, 50:672, 50:680, 51:128, 51:129, 51:135, 51:162, *1155*

`\@upline` 42:308, 42:309, 42:315

`\@upordown` 42:206, 42:207, 42:215, 42:236, 42:284

`\@upvector` 42:279, 42:315

`\@use@ption` . 50:584, 50:600, 50:618, 50:634, 50:636, 50:653, 50:655, 50:665

`\@use@text@encoding` 21:168, 33:16, 33:1102

`\@vbsphack` 18:242

`\@verb` 37:727, 37:736, 37:739

`\@verbatim` 37:525, 37:571, 37:586, 37:595

`\@verbvisiblespacebox` . . . 37:585, 37:600, 37:612, 37:613, 37:626, 37:627

`\@vereq` 30:468, 30:469

`\@viiipt` 24:815

`\@viipt` 24:814

`\@vipt` 24:813

`\@vline` 42:177, 42:189, 42:308

`\@vobeyspaces` 37:501, 37:571, 37:588, 37:664, 37:678, 37:692, 37:739

`\@vobeytabs` 37:501

`\@vpt` 24:812

`\@vspace` 18:399

`\@vspace@calcify` 18:80, 18:102, 18:264, 18:279, 18:406, 18:411, 18:421, 18:429, 37:437, 38:497, 41:62, 41:301, 42:159

`\@vspacer` 18:399

`\@vtryfc` 54:1198, 54:1206

`\@vvector` 42:254, 42:269, 42:279

`\@warning` 14:215

`\@wckptelt` 20:459, 20:462

`\@whiledim` 13:7, 42:134, 42:214

`\@whilenoop` 13:3, *1391*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\@whilenum 13:3, 41:321, 42:115, 42:127, 42:347, 42:349, 42:372, 42:375, 42:404, 42:406, 42:427, 42:432, 42:741, 42:797
\@whilesw 13:10, 54:269, 54:399, 54:408, 54:446, 54:456, 54:2897, 54:2939, 54:2978
\@whileswnoop 13:10, *1391*
\@wholewidth 40:204, 40:206, 40:207, 40:209, 40:211, 40:212, 40:213, 40:214, 42:2, 42:137, 42:140, 42:142, 42:310, 42:313, 42:361, 42:370, 42:417, 42:424, 42:575, 42:588, 42:600, 42:610, 42:689, 42:690, 42:738, 42:794
\@width 18:550, 21:309, 21:314, 26:193, 06:26, 30:629, 40:209, 40:211, 40:287, 40:294, 40:613, 40:714, 40:721, 41:240, 41:263, 41:296, 41:424, 41:443, 42:238, 42:310, 42:313, 42:336, 42:344, 42:361, 42:370, 42:395, 42:403, 42:417, 42:424, 42:575, 42:588, 42:738, 42:794, 45:395, 54:2428, 54:2887, 54:2930, 54:2961, 02:380, *1387*
\@wrglossary 46:25, 46:30, *1383*
\@wrindex 46:8, 46:13, *1383*
\@writeckpt 20:349, 20:405, 20:444, 20:456
\@writefile 20:32, 20:102, 20:160, 37:217, 44:193, 44:201, *870*
\@writesetup 54:841
\@wrong@font@char 21:179, 24:634, 24:668, 24:681, *1394*
\@wtryfc 54:1208, 54:1218
\@x@protect 06:123, 06:346
\@x@sf 45:593, 45:595
\@xDeclareMathDelimiter 28:1056, 28:1112
\@xaddvskip 18:242, 18:267, 18:282, 18:299
\@xarg 42:174, 42:177, 42:186, 42:189, 42:196, 42:200, 42:201, 42:237, 42:239, 42:249, 42:250, 42:254, 42:264, 42:265, 42:269, 42:277, 42:285, 42:674
\@xargarraycr . . 41:282, 41:291, 41:295
\@xargdef 06:98
\@xarraycr 41:279, 41:280
\@xbitor 54:11, 54:13
\@xcentercr 37:420, 37:427, 37:431
\@xdblarg 06:890
\@xdblfloat 45:264, *1382*
\@xdim 42:95, 42:104, 42:116, 42:118, 42:128, 42:130, 42:678, 42:742, 42:743, 42:744, 42:745, 42:751, 42:798, 42:799, 42:800, 42:801, 42:807
\@xeqncr 38:481
\@xexnoop 41:315, 41:325
\@xexpast 41:316, 41:317
\@xfloat 45:28, 45:29, 45:34, 45:266, *1383*
\@xfootnote 45:514, 45:517, *1360*
\@xfootnotemark . 45:581, 45:585, *1360*
\@xfootnotenext 45:598, 45:601
\@xfootnotetext *1360*
\@xhline 41:437, 41:438
\@xifnch 06:878, 06:888
\@xiipt . 24:819, 30:175, 30:177, 30:178
\@xipt 24:818, 30:174
\@xivpt 24:820, 30:176, 30:178
\@xmpar 45:324, 45:325
\@xnewline 18:61, 18:62, 18:73, 18:74, 18:94
\@xnext 54:6, 54:7
\@xnthm 43:5, 43:6
\@xobeysp 18:489, 18:514, 37:506, 37:516, 37:520, 37:609, 37:613, 37:623, 37:627, 37:640, *480*
\@xobeytab 18:510, 37:508, 37:640
\@xprocess@ptions 50:575, 50:592, 50:594, 50:610, 50:612, 50:626
\@xpt . . . 24:817, 30:173, 30:176, 30:177
\@xsect 44:86, 44:87, 44:123
\@xtabcr 41:56, 41:57
\@xtabularcr 41:286, 41:287
\@xthm 43:46, 43:52, 43:55
\@xtryfc 54:1195, 54:1223
\@xtypein 06:33, 06:35, 06:42
\@xverbatim 37:520, 37:571
\@xviipt 24:821, 30:177, 30:179
\@xxDeclareMathDelimiter 28:1041, 28:1045
\@xxpt 24:822, 30:178, 30:179
\@xxvpt 24:823, 30:179
\@xxxii 12:2, 21:446, 21:448, 45:89, 45:158, 54:1230, 54:1231, 54:1250, 54:1251, 54:1286, 54:1287, 54:1309, 54:1310, 54:2624, *1394*
\@xympar 45:328, 45:332, 45:378
\@yarg 42:174, 42:178, 42:186, 42:190, 42:196, 42:197, 42:206, 42:249, 42:255, 42:264, 42:270, 42:279, 42:281, 42:308, 42:674
\@yargarraycr . . 41:283, 41:293, 41:297
\@yargd@f 06:125
\@yargdef 06:102, 06:112, 06:125, 06:162, *1396*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\@ydim 42:96, 42:105, 42:116, 42:119, 42:128, 42:130, 42:678, 42:746, 42:747, 42:748, 42:749, 42:750, 42:802, 42:803, 42:804, 42:805, 42:806
\@yeqncr 38:481
\@ympar 45:324, 45:329
\@ynthm 43:5, 43:14
\@ythm 43:46, 43:52, 43:55
\@ytryfc . . . 54:1241, 54:1260, 54:1264
\@yyarg 42:196, 42:197, 42:198, 42:201, 42:285, 42:674
\@ztryfc 54:1269, 54:1280
\@@end *243*
\accent@spacefactor 21:88, 21:91, 21:92, *1402*
\active@math@prime 38:253, 38:254, 54:839, *1402*
\add@accent 21:83, 21:85, *1395*
\add@percent@to@temptokena 37:204, 37:220, 37:222, 37:231, *871*
\add@unicode@accent . 21:1067, 21:1081
\addto@hook 24:153, 24:155, 24:811, 28:510, 28:646, 28:650, 28:667, 28:791, 28:797, 28:805, 28:821, 28:824, 28:827, 28:1260, 28:1267, 28:1270, *1369*
\AddToHook *327*
\AddToHookNext *327*
\AddToHookNextWithArguments . . . *327*
\AddToHookWithArguments *327*
\alias@ctr@FOO *558*
\alloc@ 04:22, 04:26, 04:38, 24:15, 02:141, *1417*
\alpha@elt 28:46, 28:514, 28:741, 28:843, 28:1259, 28:1260
\alpha@list . . 28:42, 28:44, 28:523, 28:729, 28:741, 28:786, 28:841, 28:842, 28:1255, 28:1261, 28:1262
\apptocmd *328*
\atveryend@DEPRECATED 52:580, 52:582
\best@size 26:455, 26:479, 26:485, 26:491
\bf@def@ult 29:476
\bfdef@ult 29:222, 29:280, 29:350, 29:351, 29:352, 29:393, 29:394, 29:395, 29:480, 29:521, *755*
\bfdefault@previous 29:346, 29:349, 29:389, 29:392, 30:117, 30:127, *746*
\bfseries@.. . . . . *739*
\bfseries@previous *1418*
\bfseries@rm 29:106, 29:129, 29:147, 29:300, 29:303, 29:337, 29:350, 29:393, 29:398, 29:434, *739*
\bfseries@rm@kernel 29:109, 29:132, 29:150, 29:300, *744*
\bfseries@sf 29:107, 29:129, 29:148, 29:304, 29:307, 29:338, 29:351, 29:394, 29:399, 29:435, *736*
\bfseries@sf@kernel 29:110, 29:133, 29:151, 29:304
\bfseries@tt 29:108, 29:129, 29:149, 29:308, 29:311, 29:339, 29:352, 29:395, 29:400, 29:436, *744*
\bfseries@tt@kernel 29:111, 29:134, 29:152, 29:308
\bm@b 40:37
\bm@c 40:37, 40:50, 40:256, 40:384, 40:410
\bm@l 40:37
\bm@r 40:37
\bm@s 40:37
\bm@t 40:37
\botmark *1065*
\bx@ *1290*
\bx@A 54:21
\bx@AA 54:29
\bx@B 54:21
\bx@BB 54:29
\bx@C 54:21
\bx@CC 54:29
\bx@D 54:21
\bx@DD 54:29
\bx@E 54:21
\bx@EE 54:29
\bx@F 54:22
\bx@FF 54:30
\bx@G 54:22
\bx@GG 54:30
\bx@H 54:22
\bx@HH 54:30
\bx@I 54:22
\bx@II 54:30
\bx@J 54:22
\bx@JJ 54:30
\bx@K 54:23
\bx@KK 54:31
\bx@L 54:23
\bx@LL 54:31
\bx@M 54:23
\bx@MM 54:31
\bx@N 54:23
\bx@NN 54:31
\bx@O 54:24
\bx@OO 54:32
\bx@P 54:24
\bx@PP 54:32
\bx@Q 54:24
\bx@QQ 54:32
\bx@R 54:24
\bx@RR 54:32

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\bx@S 54:27
\bx@SS 54:33
\bx@T 54:27
\bx@TT 54:33
\bx@U 54:27
\bx@UU 54:33
\bx@V 54:27
\bx@VV 54:33
\bx@W 54:28
\bx@WW 54:34
\bx@X 54:28
\bx@XX 54:34
\bx@Y 54:28
\bx@YY 54:34
\bx@Z 54:28
\bx@ZZ 54:34
\c@ 22:188
\c@* 22:30, *1435*
\c@bottomnumber 45:269, 45:274, 54:2993
\c@dbltopnumber 45:268, 45:283, 45:297, 54:3000
\c@enumi 39:258
\c@enumii 39:258
\c@enumiv 39:258, *1360*
\c@equation 38:393, 38:445, 38:474, 38:636
\c@errorcontextlines . . . 14:212, *1364*
\c@footnote 44:12, 45:397, 45:587
\c@localmathalphabets 28:138, 28:151, 28:283, 28:331, *707*
\c@mpfootnote 40:517, 45:399
\c@ncel 30:472, 30:473
\c@page 34:3, 34:6, 34:7, 37:28, 48:439, 48:499, 53:432, 53:530, 54:117, 54:2395
\c@secnumdepth 44:56, 44:71, 44:81, 44:140
\c@tocdepth 44:140, 44:230, 44:253
\c@topnumber . . 45:267, 45:271, 54:2989
\c@totalnumber 45:270, 45:276, 54:2996
\c@totalpages 53:395, 53:494
\calculate@math@sizes 24:717, 26:220
\catcodetable@atletter . 04:93, 04:245
\catcodetable@initex . . 04:93, 04:242
\catcodetable@latex . . . 04:93, 04:244
\catcodetable@string . . 04:93, 04:243
\cdp@elt 24:97, 24:117, 24:128, 24:129, 24:150, 24:153, 24:155, 28:372, 28:422, 28:540, 28:615, 28:689, 28:770, 57:521, 57:522
\cdp@list . . . 24:99, 24:115, 24:129, 24:157, 24:158, 28:390, 28:440, 28:542, 28:617, 28:691, 28:772, 57:522
\cf@encoding 21:52, 21:59, 21:62, 21:69, 21:172, 24:319, 24:329, 24:339, 24:389
\ch@ck 50:1302, 50:1321, 50:1433, 50:1454, 50:1546, 02:129, 02:130, 02:131, 02:132, 02:133, 02:165, 02:167, 02:172
\char@if@alph 06:898
\chardef@text@cmd 21:3
\check@command 06:228, 06:230
\check@icl 32:9, 32:45, 32:50, 32:56, 32:64, 32:71, 32:73
\check@icr 32:9, 32:45, 32:51, 32:57, 32:65, 32:74, 32:79, *1397*
\check@mathfonts 19:5, 21:322, 21:348, 21:380, 21:1282, 24:430, 24:435, 24:442, 24:444, 26:251, 28:336, 28:475, 33:636, 40:335, 40:630, 45:412, 45:471, *1443*
\check@nocorr *1380*
\check@nocorr@ 32:47, *1399*
\check@range 26:396, 26:397
\check@single 26:395, 26:417
\cl@@ckpt 20:459, 22:52
\cl@counter *561*
\cl@page 34:4
\col@number 54:73, 54:127, 54:213, 54:225, *1373*
\color@begingroup 24:740, 24:802, 38:104, 38:134, 40:29, 40:110, 40:220, 40:505, 40:551, 40:568, 40:585, 41:47, 41:51, 45:537, 45:555, 45:573, 54:619, 54:745, *1370*
\color@endbox 40:110, 45:253, 45:348, 45:366, 54:229, 54:921, 54:936, 54:986, 54:996, 54:1045, 54:1055
\color@endgroup 24:746, 24:808, 38:104, 38:134, 40:29, 40:119, 40:134, 40:178, 40:199, 40:222, 40:533, 40:555, 40:572, 40:588, 41:49, 45:541, 45:559, 45:576, 54:624, 54:749, *1370*
\color@hbox . 40:110, 54:918, 54:933, 54:983, 54:993, 54:1042, 54:1052, *1383*
\color@setgroup 40:110, 40:178, 40:197, *1370*
\color@vbox . 40:110, 45:96, 45:165, 45:339, 45:357, 45:381, 54:220, *1385*
\conditionally@traceoff 14:291, 07:289, 07:301, *437*
\conditionally@traceon 14:291, 07:325
\contionally@traceon *437*
\copy@kernel@robust@command 06:680, 06:767

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\count@ 03:14, 03:15, 03:16, 03:17, 03:18, 03:20, 01:163, 01:164, 01:165, 01:170, 24:776, 24:782, 24:784, 26:23, 26:303, 26:305, 26:327, 26:328, 28:507, 28:509, 28:513, 28:872, 28:873, 28:874, 28:920, 28:921, 28:922, 28:981, 28:982, 28:983, 28:1029, 28:1030, 28:1031, 28:1069, 28:1070, 28:1071, 28:1077, 28:1078, 28:1079, 28:1123, 28:1124, 28:1125, 28:1131, 28:1132, 28:1133, 28:1192, 28:1193, 28:1194, 28:1200, 28:1201, 28:1202, 32:116, 32:119, 06:210, 06:214, 06:379, 06:404, 42:740, 42:741, 42:742, 42:745, 42:746, 42:749, 42:753, 42:796, 42:797, 42:798, 42:801, 42:802, 42:805, 42:809, 50:1345, 50:1347, 50:1348, 50:1349, 50:1478, 50:1480, 50:1481, 50:1482, 50:1567, 50:1569, 50:1570, 50:1571, 02:41, 57:239, 57:240, 57:247, 57:249, 57:583, 57:584, 57:591, 57:593, 02:114, 02:115, 02:120, 02:122, 02:128, 02:129, 02:130, 02:131, 02:132, 02:133, 02:134, 01:50, 02:380, 02:381, *1325*
\counterwithin@s 22:236, 22:237, 22:252
\counterwithin@x 22:236, 22:239, 22:253
\counterwithout@s 22:284, 22:285, 22:299
\counterwithout@x 22:284, 22:287, 22:300
\curr@fontshape 21:198, 24:89, 24:460, 24:468, 24:472, 24:474, 24:616, 24:622, 24:625, 24:634, 24:641, 24:643, 24:651, 24:657, 24:660, 24:668, 24:675, 24:677, 25:2842, 26:93, 26:101, 26:143, 26:170, 26:494, 26:514, 26:546, 26:562, 26:577, 28:394, 28:399, 28:444, 28:449, 29:639, 29:648, 29:686, 29:693, *1358*
\curr@math@size 24:449, 26:257, 26:263, 26:268, 26:285
\declare@command@copy *1417*
\declare@commandcopy 06:568, 06:572, 06:577, 06:580, 06:599, *101*
\declare@commandcopy@do 06:580, 06:626, 06:627
\declare@commandcopy@let . 06:587, 06:591, 06:601, 06:702, 06:703, *101*
\declare@environmentcopy 06:609, 06:613, 06:618, 06:621
\declare@file@substitution 52:247, 52:567, 53:591, *1164*
\declare@robustcommand 06:274
\declare@robustcommand@auxi . 06:274
\declare@robustcommand@auxii 06:274
\declare@robustcommand@auxiii 06:274
\DeclareEncodingSubset@aux 24:187, 24:189
\DeclareFontEncoding@ 24:123, 24:125, 24:140, 57:432, 57:452, 57:518, *1411*
\DeclareFontEncoding@saved 57:432, 57:452, 57:534
\DeclareFontShape@ 24:22, 24:23
\DeclareRobustCommand *329*
\DeclareSymbolFont@m@dropped 28:532, 28:537, 28:580, 28:581
\DeclareSymbolFontAlphabet@ 28:1247, 28:1250
\DeclareUnicodeAccent@ 21:1074, 21:1076, 21:1080
\def *347*
\default@ds 50:540, 50:571, 50:635, 50:654, 50:1151, 50:1153, *1362*
\default@family 24:130, 24:162, 24:584, 24:598, 24:601, 24:626, 24:661, 57:523, *1357*
\default@M 24:137, 24:177, 24:180, 24:184, 57:530, *1362*
\default@mextra 27:10, 27:89
\default@series 24:130, 24:163, 24:585, 24:599, 24:602, 24:623, 24:658, 57:523
\default@shape 24:131, 24:164, 24:586, 24:600, 24:603, 24:621, 24:656, 57:524
\default@T 24:171, 24:174, 24:184, 24:335, *1362*
\define@mathalphabet 27:18, 27:131, *1357*
\define@mathgroup . 27:19, 27:135, *1357*
\define@newfont 24:452, 24:461
\delayed@f@adjustment . . . 24:353, 25:2783, 25:2784, 25:2785, 25:2787, 25:2788, 25:2799, 25:3136, 25:3137, 25:3139, 25:3140, 26:123, 26:127, 26:135, 26:139, 29:774, 29:806, *582*
\delayed@merge@font@series 25:2784, 25:2848, 25:2863, 25:2902, 26:134, 26:137
\delayed@merge@font@shape 25:3137, 25:3186, 25:3241, 26:133, 26:136
\development@branch@name 03:11, 03:39, 03:53, 03:54, 03:55, 03:62, 03:63, 03:64

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\dimen@ 14:28, 14:29, 18:418, 18:423, 18:452, 18:457, 21:450, 21:451, 21:453, 21:454, 21:820, 21:821, 24:277, 24:279, 24:285, 24:298, 24:301, 24:305, 24:720, 24:721, 24:722, 24:726, 26:468, 26:469, 26:470, 26:471, 26:475, 33:66, 33:68, 33:892, 33:894, 38:72, 38:73, 38:199, 38:200, 38:201, 38:202, 40:698, 40:701, 41:200, 41:201, 02:41, 54:761, 54:763, 54:773, 54:775, 02:377, 02:378, 02:414, 02:415, 02:417, 02:419, *1396*
\dimen@i 02:41
\dimen@ii 24:281, 24:286, 02:41
\disable@package@load 52:477, 53:549, *1179*
\displ@y 38:204, 38:208, 38:209
\do@add@percent@to@temptokena 37:208, 37:214, 37:232
\do@emfont@update 29:640, 29:644, 29:664
\do@noligs 37:741, 37:746
\do@strongfont@update 29:687, 29:690, 29:696
\do@subst@correction 24:85, 26:499, 26:554
\document@default@language 20:51, 20:52, 20:121, 20:122, 54:850, 54:952, 57:288
\document@select@group 28:140, 28:146, 28:407, 28:457, *707*
\dollardollar@begin 38:289, 38:318, 38:398, 38:433, 38:625, *1437*
\dollardollar@end 38:289, 38:327, 38:400, 38:446, 38:636, *1437*
\dont@add@percent@to@temptokena 37:207, 37:209, 37:233
\dorestore@version . . . 28:115, 28:120
\ds@ 50:573, 50:1155, *1362*
\dt@pfalse 38:205
\dt@ptrue 38:204
\e@alloc 04:15, 04:49, 04:79, 04:89, 04:178, 04:182, 04:186, 04:194, 04:202, 04:210, 02:47, 02:48, 02:49, 02:51, 02:52, 02:59, 02:60, 02:62, 02:64, 02:75, 02:78, 02:80, 02:94, 57:12, 57:47, 02:141, *1417*
\e@alloc@attribute@count 04:66, 04:74, 04:75, 04:76, 04:80, 04:236
\e@alloc@bytecode@count . . . 04:70, 04:197, 04:198, 04:199, 04:203, 04:252
\e@alloc@ccodetable@count 04:67, 04:84, 04:85, 04:86, 04:90, 04:240
\e@alloc@chardef 04:48, 04:178, 04:194, 04:202, 04:210, 02:56, 02:82, 57:12, 02:134, 02:135
\e@alloc@intercharclass@top . . 57:35
\e@alloc@luachunk@count . . . 04:71, 04:205, 04:206, 04:207, 04:211, 04:254
\e@alloc@luafunction@count 04:68, 04:173, 04:174, 04:175, 04:179, 04:183, 04:187, 04:246, 04:248
\e@alloc@top 04:47, 04:80, 04:179, 04:183, 04:187, 04:195, 04:203, 04:211, 02:51, 02:59, 02:82, 57:12, 02:111, 02:145
\e@alloc@whatsit@count 04:69, 04:189, 04:190, 04:191, 04:195, 04:250
\e@ch@ck . . . 04:51, 04:55, 02:96, 02:100
\e@insert@top . . 02:143, 02:145, 02:162
\e@mathgroup@top 28:57, 28:150, 28:151, 28:188, 28:218, 02:75, 02:89, *1407*
\em@currfont 29:639, 29:650, *757*
\em@force 29:648, 29:653, 29:656, 29:666, *757*
\emfontdeclare@clist 29:629, 29:631, 29:635, 29:640, 29:645, 29:651, 29:662, *756*
\empty@sfcnt 26:507, 26:508, 26:509, 26:523, 26:528, 26:580
\ENC@cmd *1377*
\enc@update 24:320, 24:322, 24:338, 24:341, 26:148, 26:174, *1377*
\end@dblfloat 45:205
\end@float 45:189, 45:227, 45:243, 45:383, *1040*
\endlinechar *348*
\enlargethispage *1071*
\err@rel@i . 27:12, 27:99, 27:132, 27:136
\error@fontshape 24:579, 24:594, 24:619, 24:654, 26:108, 26:544, 28:393, 28:443
\escapechar *340*
\et@xmaxfam . 04:22, 04:26, 04:30, 04:38
\et@xmaxregs 04:29, 04:31, 04:32, 04:33, 04:34, 04:35, 04:36, 04:37
\every@math@size 24:79, 26:236, 26:248, *1384*
\every@size *1358*
\execute@size@function 26:379, 26:407, 26:421, 26:438
\expand@font@defaults 29:38, 29:171, 29:241, 29:335, 29:358, 29:388, 29:409, 29:433, 29:444, 29:475,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

29:476, 29:515, 29:517, 29:549, 29:551, 29:580, 33:26, 33:47, *244*
expand@font@defaults 29:499
\expandafter *1179*
\external@font 26:85, 26:88, 26:99, 26:103, 26:105, 26:408, 26:422, 26:484, 26:518, 26:586, 26:588, 26:590
\extra@def 27:9, 27:84, *1357*
\extract@alph@from@version 24:694, 24:700, 28:163, 28:194, 28:224, 28:256, *1360*
\extract@default@composite 21:1090, 21:1097
\extract@default@composite@a 21:1099, 21:1103
\extract@default@composite@b 21:1101, 21:1105
\extract@font 24:475, 26:82
\extract@fontinfo 26:375, 26:382
\extract@rangefontinfo 26:392, 26:399, 26:418, 26:451
\extract@sizefn 26:367, 26:389
\f@... . . . . *1377*
\f@baselineskip 21:894, 21:1248, 24:380, 24:387, 24:605, 26:122, 26:168, 26:183, 26:187, 26:202, 26:216, 26:227, 26:241
\f@depth 45:291, 54:350
\f@encoding . 04:259, 04:274, 04:282, 21:196, 24:314, 24:333, 24:336, 24:337, 24:339, 24:389, 24:455, 24:460, 24:479, 24:481, 24:483, 24:488, 24:490, 24:521, 24:583, 24:615, 24:650, 25:2827, 25:2831, 25:3172, 25:3176, 26:92, 26:129, 26:308, 26:534, 28:378, 28:428, *1360*
\f@family 24:209, 24:210, 24:213, 24:216, 24:233, 24:236, 24:342, 24:350, 24:362, 24:372, 24:383, 24:456, 24:460, 24:479, 24:481, 24:483, 24:488, 24:490, 24:522, 24:601, 24:626, 24:661, 25:2827, 25:2831, 25:3172, 25:3176, 26:92, 26:129, 28:378, 28:409, 28:428, 28:459, 29:173, 29:200, 29:201, 29:244, 29:264, 29:337, 29:338, 29:339, 29:360, 29:361, 29:362, 29:398, 29:399, 29:400, 29:418, 29:419, 29:420, 29:434, 29:435, 29:436, 29:445, 29:446, 29:447, 29:583, 29:598, 29:770, 29:780, 29:783, 29:786, 29:803, 29:819, 29:835, 29:867, 29:873, 29:894, 33:23, 33:27, 33:29, 33:31, 33:44, 33:48, 33:50, 33:52, 33:57, 33:64, 33:858, 33:861, 33:875, 33:884, 33:890, 33:1105, *1357*
\f@linespread 24:383, 26:121, 26:167, 26:184, 26:185, 26:188, 26:196, 26:199, 26:210, 26:213, *1377*
\f@series 19:14, 24:342, 24:373, 24:384, 24:457, 24:460, 24:602, 24:623, 24:658, 25:2788, 25:2798, 25:2807, 25:2817, 25:2851, 25:2870, 25:2871, 25:3172, 25:3176, 26:126, 26:129, 26:132, 28:410, 28:460, 29:167, 29:182, 29:184, 29:188, 29:189, 29:190, 29:211, 29:217, 29:220, 29:222, 29:252, 29:255, 29:272, 29:277, 29:279, 29:280, 29:315, 29:601, 29:605, 29:771, 29:804, 29:820, 29:836, 29:896, 33:7, 33:577, *656*
\f@series@saved 26:126, 26:132
\f@shape 24:342, 24:352, 24:364, 24:374, 24:385, 24:458, 24:460, 24:603, 24:621, 24:656, 25:2827, 25:2831, 25:3140, 25:3147, 25:3153, 25:3163, 25:3170, 25:3174, 25:3177, 25:3180, 25:3189, 25:3196, 25:3198, 25:3233, 26:125, 26:129, 26:131, 28:411, 28:461, 29:772, 29:805, 29:821, 29:837, 29:897, *675*
\f@shape@saved 26:125, 26:131
\f@size 21:198, 21:893, 21:1247, 24:89, 24:380, 24:386, 24:459, 24:604, 24:643, 24:677, 24:719, 24:720, 24:723, 24:724, 26:122, 26:143, 26:168, 26:170, 26:181, 26:201, 26:216, 26:219, 26:222, 26:227, 26:234, 26:241, 26:253, 26:256, 26:262, 26:268, 26:285, 26:286, 26:289, 26:294, 26:376, 26:383, 26:402, 26:404, 26:419, 26:470, 26:472, 26:474, 26:490, 26:491, 26:496, 26:510, 26:522, 26:527, 26:539, 26:547, 26:552, 26:578, 26:592, 29:639, 29:648, 29:686, 29:693, 33:66, 33:892
\f@user@size 26:490, 26:495, 26:539, 26:552
\f@warn@break *1379*
\famdef@ult 29:523
\filec@ntents 50:1252, 50:1255, 50:1258, 50:1269, 50:1294, 50:1387, 50:1390, 50:1393, 50:1404, 50:1425, 50:1518, 50:1540, 50:1628, 50:1853, *1389*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\filec@ntents@checkdir 50:1275, 50:1277, 50:1295, 50:1410, 50:1412, 50:1426, 50:1525
\filec@ntents@force 50:1271, 50:1406, 50:1521
\filec@ntents@noheader 50:1273, 50:1408, 50:1523
\filec@ntents@nosearch 50:1274, 50:1409, 50:1524
\filec@ntents@nowarn 50:1279
\filec@ntents@opt 50:1255, 50:1258, 50:1260, 50:1390, 50:1393, 50:1395, 50:1520
\filec@ntents@OPTION *1131*
\filec@ntents@overwrite 50:1272, 50:1407, 50:1522
\filec@ntents@warning 50:1280, 50:1282, 50:1327
\filec@ntents@where 50:1276, 50:1278, 50:1307, 50:1411, 50:1413, 50:1438, 50:1526
\filename@area . . . 20:661, 20:676, 20:686, 20:718, 20:719, 20:723, 20:748, 20:751, 20:776, 20:792, 20:794, 01:230, 01:236, 01:243, 01:249, 01:256, 01:262, 01:269, 50:984
\filename@base 20:661, 20:676, 20:686, 20:718, 20:720, 20:723, 20:748, 20:751, 20:783, 20:792, 01:279, 01:286, 01:299, 50:985
\filename@dot 01:297, 01:303
\filename@dots . 01:281, 01:283, 01:288
\filename@ext 20:662, 20:677, 20:687, 20:714, 20:715, 20:718, 20:721, 20:723, 20:744, 20:745, 20:748, 20:751, 20:784, 01:278, 01:285, 01:295, 01:297, 50:986, *1170*
\filename@parse 01:94, 20:659, 20:674, 20:684, 20:713, 20:743, 20:781, 01:226, 50:983
\filename@path 01:231, 01:232, 01:237, 01:244, 01:245, 01:250, 01:257, 01:258, 01:263
\filename@simple 01:234, 01:247, 01:260, 01:270, 01:274, 01:276, 01:291, 01:293
\finish@module@release . . 03:89, 03:93
\finph@nt 38:104, 38:106, 38:110, 38:111, 38:119, 38:120, *1322*
\finsm@sh . . . 38:134, 38:141, 38:147, 38:153, 38:154, 38:158, 38:159, *1322*
\fix@penalty 32:102
\fixed@sfcnt . . . 26:582, 26:583, 26:584
\fl@ShowFloat 54:2481, 54:2487, 54:2502
\fl@trace 54:245, 54:272, 54:328, 54:356, 54:363, 54:384, 54:431, 54:479, 54:521, 54:531, 54:532, 54:533, 54:534, 54:544, 54:545, 54:546, 54:547, 54:548, 54:558, 54:1132, 54:1151, 54:1170, 54:1188, 54:1190, 54:1329, 54:1333, 54:1345, 54:1346, 54:1347, 54:1348, 54:1354, 54:1357, 54:1365, 54:1369, 54:1380, 54:1385, 54:1390, 54:1391, 54:1392, 54:1393, 54:1400, 54:1403, 54:1411, 54:1422, 54:1428, 54:1433, 54:1438, 54:1444, 54:1445, 54:1450, 54:1456, 54:1457, 54:1458, 54:1466, 54:1470, 54:1475, 54:1479, 54:1484, 54:1495, 54:1496, 54:1498, 54:1516, 54:1525, 54:1531, 54:1540, 54:1543, 54:1549, 54:1559, 54:1563, 54:1573, 54:1579, 54:1584, 54:1589, 54:1594, 54:1595, 54:1600, 54:1605, 54:1606, 54:1607, 54:1615, 54:1619, 54:1624, 54:1628, 54:1633, 54:1644, 54:1645, 54:1647, 54:1665, 54:1673, 54:1677, 54:1685, 54:1688, 54:1694, 54:1704, 54:1708, 54:1717, 54:1723, 54:1729, 54:1735, 54:1742, 54:1744, 54:1750, 54:1755, 54:1757, 54:1759, 54:1767, 54:1772, 54:1778, 54:1783, 54:1789, 54:1803, 54:1804, 54:1807, 54:1828, 54:1837, 54:1843, 54:1852, 54:1855, 54:1862, 54:1872, 54:1876, 54:1888, 54:1894, 54:1899, 54:1904, 54:1908, 54:1913, 54:1914, 54:1921, 54:1926, 54:1930, 54:1937, 54:1946, 54:1950, 54:1954, 54:1955, 54:1959, 54:1960, 54:1970, 54:1976, 54:1981, 54:1986, 54:1990, 54:1994, 54:1995, 54:2002, 54:2007, 54:2011, 54:2018, 54:2027, 54:2031, 54:2035, 54:2036, 54:2040, 54:2041, 54:2050, 54:2056, 54:2062, 54:2068, 54:2072, 54:2078, 54:2080, 54:2088, 54:2093, 54:2098, 54:2106, 54:2115, 54:2120, 54:2125, 54:2127, 54:2132, 54:2134, 54:2145, 54:2151, 54:2161, 54:2167, 54:2171, 54:2172, 54:2177, 54:2178, 54:2184, 54:2187, 54:2188, 54:2189, 54:2196, 54:2197, 54:2198, 54:2206, 54:2211, 54:2223, 54:2224, 54:2231, 54:2234, 54:2242, 54:2246, 54:2250, 54:2251, 54:2255, 54:2256, 54:2266, 54:2272, 54:2282, 54:2288, 54:2292, 54:2293, 54:2299, 54:2300, 54:2307, 54:2310, 54:2311, 54:2312, 54:2320, 54:2321, 54:2322,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

54:2331, 54:2336, 54:2349, 54:2351, 54:2358, 54:2361, 54:2370, 54:2374, 54:2378, 54:2379, 54:2383, 54:2384, 54:2436, 54:2441, 54:2447, 54:2457, 54:2464, 54:2478, 54:2479, 54:2483, 54:2494, 54:2506, 54:2614, 54:2627, 54:2628, 54:2632, 54:2635, 54:2637, 54:2640, 54:2643, 54:2645, 54:2686, 54:2693, 54:2698, 54:2704, 54:2709, 54:2713, 54:2719, 54:2732, 54:2734, 54:2741, 54:2746, 54:2751, 54:2753, 54:2759, 54:2761, 54:2768, 54:2782, 54:2784, 54:2791, 54:2796, 54:2801, 54:2803, 54:2809, 54:2811, 54:2818, 54:2847, 54:2849, 54:2864, 54:2866, 54:2880, 54:2890, 54:2893, 54:2898, 54:2951, 54:2968, 54:2973, 54:2981

\fl@tracemessage 54:2478, 54:2495, 54:2504, 54:2506

\fl@traceval 54:2488, 54:2489, 54:2490, 54:2491, 54:2494, 54:2503, 54:2506

\float@count 02:47, 02:48, 02:49, 02:58, 02:111, 02:128, 02:134, 02:136, 02:137, 02:141

\fmtversion@topatch 57:737, 57:739, 57:751, 57:752, 57:764, 57:772

\font@info 26:100, 26:382, 26:451, 26:456

\font@name . . 21:197, 21:200, 24:87, 24:257, 24:259, 24:451, 24:466, 24:642, 24:676, 26:85, 26:89, 26:91, 26:106, 26:142, 26:145, 26:155, 26:169, 26:172, 26:331, 26:332, 26:333, 26:334, 26:335, 26:340, *1357*

\font@submax 26:458, 26:487, 26:488, 37:55, 37:57, 37:120, 37:122, 37:159, 37:161, 57:301, 57:303, 57:312, *1372*

\fps@dbl 45:34

\freeze@math@version . 28:155, 28:274

\frozen@everydisplay 24:418, 24:423, 24:440, 24:442, *711*

\frozen@everymath 24:418, 24:432, 24:444, *711*

\g@addto@macro 09:77, 25:3251, 28:475, 50:168, 50:495, 50:1163, 50:1179, 50:1180, 06:1010, 53:399, 53:405, 53:410, 53:461, *252*

\G@refundefined *1395*

\G@refundefinedfalse 35:5, *841*

\G@refundefinedtrue . . 35:3, 35:16, 35:33, 35:50, 36:225, 36:232, 36:243, 36:249, 47:41, 47:68, 47:85, *1394*

\gen@sfcnt 26:519, 26:520, 26:521

\genb@sfcnt 26:524, 26:525, 26:526, *1393*

\genb@x 26:527, 26:529

\genb@y 26:529

\get@cdp 28:643, 28:651, 28:684

\get@external@font 26:84, 26:97, 26:553

\getanddefine@fonts 24:689, 24:707, 26:321, 28:60, 28:88, 28:133, 28:160, 28:191, 28:221, 28:252, 28:299, 28:349, 28:510, 28:594, 28:648, 28:650, 28:667, 28:790, 28:791, 28:823, 28:824, 28:1266, 28:1267, *711*

\glb@currsize 20:41, 20:111, 20:169, 24:409, 26:218, 26:253, 26:257, 26:263, 26:286

\glb@settings 24:410, 26:218, 26:265, 26:296

\gobble@finish@module@release 03:109, 03:111, 03:170

\gobble@font@spec *1391*

\group@elt 28:36, 28:508, 28:555, 28:556, 28:587, 28:591, 28:1298

\group@list 28:512, 28:562, 28:585, 28:590, 28:591, 28:640, 28:866, 28:914, 28:975, 28:1060, 28:1063, 28:1114, 28:1117, 28:1183, 28:1186, 28:1253, 28:1304, *1400*

\h@false 38:81

\h@true 38:82, 38:83

\hb@xt@ 21:446, 06:29, 38:210, 38:438, 38:467, 38:548, 38:563, 38:575, 38:602, 38:633, 40:48, 40:63, 40:86, 40:104, 40:253, 40:272, 40:725, 40:729, 40:730, 40:731, 41:37, 42:42, 42:54, 42:73, 42:85, 42:116, 42:128, 42:276, 42:310, 42:313, 42:316, 42:318, 42:325, 42:384, 42:463, 42:598, 42:608, 42:751, 42:807, 44:243, 44:266, 44:273, 54:920, 54:935, 54:985, 54:995, 54:1044, 54:1054, 54:2420, 54:2884, 54:2885, 54:2889, 54:2927, 54:2928, 54:2932, 54:2956, 54:2957, 54:2963, 02:424, *1398*

\hexnumber@ . 28:887, 28:895, 28:930, 28:938, 28:959, 28:969, 28:995, 28:1003, 28:1011, 28:1020, 28:1023, 28:1032, 28:1033, 28:1072, 28:1080, 28:1126, 28:1134, 28:1153, 28:1154, 28:1164, 28:1165, 28:1170, 28:1196, 28:1204, 28:1209, 28:1211, 29:756

\hgl@ 02:379, 02:380

\hmode@bgroup 21:85, 21:93, 21:347, 21:376, 21:411, 21:417, 21:448, 21:459, 21:466, 21:499, 21:506, 21:509, 21:511, 21:519, 21:535, 21:754, 21:784, 21:790, 21:822,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

21:829, 21:857, 21:860, 21:917, 21:1281, 32:7, 33:589, 33:596, *1402*
\hmode@start@before@group 21:86, 21:169, 21:171, 21:177, 21:202
\hyper@nopatch@longtable 55:461
\if@afterindent 44:124, 44:131
\if@compatibility 50:2, 50:711
\if@endpe 37:340, 37:355, 37:372, 37:382, 39:140, *1433*
\if@eqnsw 38:415, 38:516
\if@fcolmade 54:73, 54:269, 54:399, 54:408, 54:446, 54:456, 54:1130, 54:1150, 54:1168, 54:1197, 54:1277, 54:2846, 54:2863, 54:2897, 54:2939, 54:2978
\if@filesw 20:5, 20:36, 20:106, 20:164, 20:317, 20:329, 20:350, 20:379, 20:391, 20:406, 20:426, 20:438, 20:445, 20:457, 37:22, 37:61, 37:89, 37:126, 37:151, 37:165, 44:153, 47:4, 47:8, 47:39, 47:48, 47:56, 47:67, 47:84, 50:1305, 50:1322, 50:1436, 50:1455, 53:413, *1383*
\if@firstamp 41:328
\if@firstcolumn . . . 54:73, 54:251, 54:284, 54:401, 54:449, 54:2392, 54:2877, 54:2906, 54:2947, *1067*
\if@font@series@context 29:588, 29:609, 29:619, *754*
\if@forced@series 25:2789, 29:166, 29:238, *1414*
\if@ignore 28:317, 28:356, 37:4, 37:341, 37:358, 37:374, 37:383
\if@in@minipage@env . . 40:486, 40:508
\if@includeinrelease 03:73, 03:76, 03:132, 06:964
\if@inlabel . . 39:28, 39:65, 39:102, 39:191, 39:214, 54:140, 54:167, 54:193
\if@inmath *1396*
\if@insert 54:73, 54:1408, 54:1521, 54:1555, 54:1670, 54:1700, 54:1833, 54:1868, 54:1943, 54:2024, 54:2112, 54:2239, 54:2367
\if@listfiles@hashes 20:762, 20:797, 20:804, 20:808
\if@listfiles@sizes 20:762, 20:798, 20:802
\if@minipage 16:26, 18:262, 18:277, 18:294, 18:313, 18:346, 18:380, 37:530, 37:552, 39:180, 40:483, 41:79, 45:20
\if@mparswitch 54:73, 54:2394
\if@multiplelabels 35:93
\if@mypkg@draft *1150*
\if@negarg 42:168, 42:209, 42:223, 42:284
\if@newlist . . 37:572, 39:29, 39:33, 39:69, 39:78, 39:106, 39:197, 54:877, 54:941, 54:954, 54:999, 54:1012, 54:1058
\if@nmbrlist 39:33, 39:232
\if@no@font@opt . 27:16, 27:110, 27:129
\if@nobreak 18:121, 18:150, 18:171, 18:315, 18:348, 18:382, 20:203, 20:218, 20:232, 39:198, 39:223, 40:446, 40:467, 44:47, 44:128, 45:180, 45:373, 48:270, 49:48, 49:58, 49:71, 49:79, 49:94, 49:102, 54:144, 54:171, 54:197, 54:340, 54:1500, 54:1649, 54:1810, *1385*
\if@noitemarg 39:32, 39:230
\if@noparitem 39:30, 39:188
\if@noparlist 39:31, 39:114
\if@noskipsec 18:150, 18:171, 39:58, 40:447, 40:468, 44:38, 44:40, 44:97, 45:374, 54:134, 54:161, 54:187
\if@ovb 42:438, 42:505, 42:536, 42:561, 42:572, 42:585
\if@ovhline 42:470, 42:600
\if@ovl 42:438, 42:503, 42:532, 42:557, 42:602, 42:611
\if@ovr 42:438, 42:502, 42:531, 42:556, 42:599, 42:609
\if@ovt 42:438, 42:504, 42:535, 42:560, 42:577, 42:589
\if@ovvline 42:470, 42:575
\if@partsw . 20:5, 20:321, 20:383, 20:430
\if@pboxsw 40:415, 40:436, 40:592
\if@reversemargin 54:73, 54:2397
\if@rjfield 41:19, 41:33
\if@skipping@module 03:137, 03:149, 03:152
\if@specialpage 54:73, 54:886, 54:961, 54:1019
\if@tempswa . . 12:9, 20:327, 20:389, 20:436, 21:1570, 24:100, 24:786, 28:543, 28:618, 28:692, 28:773, 28:1297, 37:63, 37:128, 37:167, 37:537, 37:558, 47:95, 50:1334, 50:1467, 50:1556, 52:292, 52:293, 52:315, 52:316, 52:336, 52:337, 54:1341, 54:1377, 54:2203, 54:2328, 02:156, 01:62, 01:63, 01:64, *1405*
\if@test 54:8, 54:9, 54:1238, 54:1257, 54:1297, 54:1319, 54:1383, 54:1468, 54:1477, 54:1617, 54:1626, 54:1770, 54:1781, 54:1924, 54:2005, 54:2091, 54:2209, 54:2334
\if@twocolumn 20:26, 20:96, 20:153, 45:32, 45:210, 45:235,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

54:73, 54:118, 54:272, 54:283, 54:400, 54:448, 54:472, 54:1132, 54:1188, 54:2391, 54:2848, 54:2865
\if@twoside 54:73, 54:117, 54:890, 54:964, 54:1022
\ifdt@p 38:203, 38:205
\IfFileExists@ 20:490, 20:517
\IfFileExists@@ 20:517
\ifG@refundefined 35:3, 35:4, 35:5
\ifh@ 38:76, 38:114, 38:123
\ifin@ 21:1590, 21:1593, 27:50, 27:52, 28:3, 28:23, 28:497, 28:639, 28:641, 28:702, 28:715, 28:785, 28:787, 28:815, 28:867, 28:881, 28:915, 28:927, 28:976, 28:992, 28:1061, 28:1064, 28:1085, 28:1115, 28:1118, 28:1181, 28:1184, 28:1187, 28:1254, 28:1256, 28:1285, 29:303, 29:307, 29:311, 50:271, 50:289, 50:583, 50:617
\ifmath@fonts 24:267, 26:223
\ifmaybe@ic 32:83, 32:92, *1405*
\ifnot@nil 26:360, 26:377, 26:398
\ifrestore@version *1396*
\iftc@forced . . 33:832, 33:843, 33:1111
\ifv@ 38:75, 38:113, 38:122
\ifx *348*
\in@ 21:1588, 21:1591, 27:49, 27:51, 28:3, 28:22, 28:496, 28:638, 28:640, 28:698, 28:711, 28:784, 28:786, 28:813, 28:865, 28:876, 28:913, 28:924, 28:974, 28:988, 28:1059, 28:1062, 28:1082, 28:1113, 28:1116, 28:1178, 28:1182, 28:1185, 28:1252, 28:1255, 28:1283, 29:301, 29:305, 29:309, 50:270, 50:287, 50:580, 50:616, *1405*
\in@@ 25:2877, 25:2878, 25:2880, 25:2881, 28:6, 28:7, 28:8, 28:10
\in@false 28:11
\in@true 28:13
\init@restore@glb@settings 26:266, 26:269, 26:271
\init@restore@version 28:63, 28:92, 28:109, 28:124, 28:125, *1396*
\init@series@setup 25:3252, 25:3257, 29:121, 29:295, 29:322, 29:323
\input@path 01:93, 01:115, 01:117, 01:123, 01:125, 01:131, 01:133, 01:138, 01:140, 01:150, 20:503, 20:557, 20:579, 20:606, 20:623, 01:217, 52:275, *10*
\insc@unt 02:37, 02:47, 02:48, 02:49, 02:58, 02:149, 02:150, 02:151, 02:152, 02:153, 02:157, 02:159
\install@mathalphabet 24:684, 24:701, 24:708, 28:516, 28:519, 28:645, 28:646, 28:743, 28:795, 28:798, 28:805, 28:820, 28:821, 28:828, 28:1268, 28:1270
\is@range 26:393, 26:394
\kernel@ifnextchar . . 03:90, 06:99, 06:118, 06:191, 50:475, 06:875, 06:890
\kernel@make@fragile 18:25, 18:26, 18:27, 18:28, 18:29, 21:190, 21:191, 37:489, 37:490, 37:491, 38:90, 38:91, 38:92, 38:93, 38:179, 38:180, 38:181, 06:479, 41:160, 41:161, 41:162, 42:833, 42:834, 42:835, 42:836, 42:837, 42:838, 42:839, 42:840, 42:841, 42:842, 42:843, 42:844, 44:23, 44:24, 44:25, 44:26, 44:27, 49:85, 49:86, 06:991, 06:992, 06:993, 06:994, 06:995, 06:996, 06:997, 06:998, 06:999, 06:1000, 06:1001, 06:1002, 06:1003, 06:1004, 06:1005, 06:1006, *1092*
\l@ngrel@x 06:79, 06:80, 06:88, 06:89, 06:93, 06:144, 06:156, 06:208, 06:311, 06:312, 06:314, *86*
\l@nohyphenation 37:534, 37:726, 57:285, *1410*
\last@fontshape 24:617, 24:635, 24:652, 24:669
\latexrelease@postltcmd 07:3329
\latexrelease@postltexpl 05:146
\leavevmode@ifvmode 18:559, 18:560, 18:568, 30:635, 30:637, 30:639, 30:641, 38:115, 38:154, 38:219, 38:239, 38:240
\load@onefile@withoptions 50:988, 50:1030, 50:1142, *1124*
\load@onefilewithoptions 50:935, 50:1092, 50:1644, *1122*
\lower@bound . . . 26:403, 26:404, 26:415
\lst@vskip *453*
\LT@cols *1325*
\ltx@sh@ft . . 21:411, 21:418, 21:499, 21:507, 21:784, 21:791, 02:416, *1405*
\ltx@star@counter 35:140, 35:145, 35:158, 35:172
\m@ne 02:39, *421*
\m@th . 19:13, 30:348, 30:470, 30:472, 30:473, 30:476, 30:517, 30:541, 30:544, 30:548, 30:551, 30:558, 30:561, 30:568, 30:571, 30:653, 38:68, 38:71, 38:106, 38:140, 38:147, 38:167, 38:169, 38:185, 38:204, 38:429, 38:458, 38:563, 38:575,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

38:602, 38:613, 40:415, 40:436, 40:660, 41:174, 41:227, 44:239, 44:262, 45:400, 45:441, 45:447, 45:500, 45:506, 02:396, 02:408, *1380*
\makeph@nt 38:101, 38:103
\makesm@sh 38:131, 38:133
\mandatory@arg 26:431, 26:518, 26:522, 26:527, 26:534, 26:536, 26:541, 26:543, 26:548, 26:550, 26:563, 26:579, 26:586, 26:588
\math@bgroup 24:715, 26:307, 26:313, 28:54, 28:82, 28:147, 28:176, 28:185, 28:207, 28:215, 28:246, 32:131, 32:132, 32:139, *708*
\math@egroup 24:715, 26:311, 26:312, 32:132, 32:133, 32:140, *1358*
\math@famname *1357*
\math@fonts 24:685, 24:690, 26:233, 26:337, 28:61, 28:90, 28:161, 28:192, 28:222, 28:254
\math@fontsfalse 19:7, 21:322, 21:349, 21:380, 21:1283, 24:78, 24:269, 24:279, 24:302, 33:67, 33:637, 33:893
\math@fontstrue 24:267, 24:727
\math@group *1357*
\math@version 24:8, 24:399, 24:689, 24:693, 24:695, 24:696, 24:698, 26:231, 28:57, 28:60, 28:65, 28:66, 28:70, 28:85, 28:89, 28:94, 28:95, 28:99, 28:112, 28:113, 28:114, 28:127, 28:128, 28:129, 28:150, 28:152, 28:154, 28:155, 28:160, 28:164, 28:166, 28:168, 28:172, 28:188, 28:191, 28:195, 28:197, 28:199, 28:203, 28:218, 28:221, 28:225, 28:227, 28:229, 28:233, 28:249, 28:253, 28:257, 28:259, 28:261, 28:265, 28:296, 28:300, 29:728, *707*
\math@version. *1357*
\mathchar@type 28:959, 28:969, 28:1020, 28:1023, 28:1032, 28:1048, 28:1153, 28:1164, 28:1227
\mathph@nt 38:99, 38:105
\mathsm@sh 38:129, 38:138, 38:139, 38:145, 38:146
\maybe@ic 32:64, 32:65, 32:84
\maybe@ic@ 32:84
\maybe@icfalse 32:98
\maybe@ictrue 32:88
\maybe@load@fontshape 21:89, 25:2864, 25:2903, 26:128, 29:186, 29:254, *516*
\maybe@update@bfseries@defaults 29:39, 29:336, 29:345, 29:383
\maybe@update@mdseries@defaults 29:40, 29:359, 29:368, 29:384
\mb@b 40:76, 40:87, 40:95, 40:105
\mb@l 40:76, 40:80, 40:86, 40:95, 40:99, 40:104, 42:148, 42:152
\mb@r 40:76, 40:80, 40:86, 40:95, 40:99, 40:104, 42:148, 42:152
\mb@t 40:77, 40:84, 40:96, 40:103
\md@def@ult 29:476
\mddef@ult 29:220, 29:279, 29:372, 29:373, 29:374, 29:413, 29:414, 29:415, 29:481, 29:522, *1415*
\mddefault@previous 29:369, 29:371, 29:410, 29:412, 30:118, 30:128, *1418*
\mdseries@.. . . . . *745*
\mdseries@previous *1418*
\mdseries@rm 29:112, 29:128, 29:129, 29:145, 29:153, 29:360, 29:372, 29:413, 29:418, 29:445
\mdseries@sf 29:113, 29:129, 29:154, 29:361, 29:373, 29:414, 29:419, 29:446
\mdseries@tt 29:114, 29:129, 29:155, 29:362, 29:374, 29:415, 29:420, 29:447
\meaning *340*
\merge@font@series 25:2796, 25:2813, 25:2814, 25:2895, 25:2897, 26:134, *658*
\merge@font@series@ 25:2816, 25:2821, 25:2898
\merge@font@series@without@substitution 25:2848, 25:2863, 25:2900, 26:137, *658*
\merge@font@series@without@substitution@ 25:2848, 25:2901
\merge@font@shape 25:3146, 25:3159, 25:3160, 25:3231, 25:3236, 26:133
\merge@font@shape@ 25:3162, 25:3167, 25:3237
\merge@font@shape@without@substitution 25:3186, 25:3239, 26:136
\merge@font@shape@without@substitution@ 25:3186, 25:3240
\mv@⟨*version*⟩ *710*
\mv@⟨*version*⟩@frozen *707*
\mv@⟨*version*⟩@reset *710*
\n@space 30:636, 30:638, 30:640, 30:642, 30:647, 30:648, 30:649, 30:650, 30:653
\new@command . 06:96, 06:95, 06:172, 06:206, 06:225, 06:296, 06:317, 06:343
\new@environment 06:188, 06:187, 06:200
\new@fontshape 27:2, 27:4, 27:22, 27:24
\new@label@record 36:57, 36:150, 36:301, 37:26, *852*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\new@mathalphabet 28:696, 28:717, 28:728
\new@mathgroup 04:27, 24:15, 28:546, 02:74, 02:76, *1390*
\new@mathversion 28:21, 28:479, 28:487, 28:492, 28:495
\new@module@skip 03:138, 03:150, 03:152
\new@moduledate 03:82, 03:101, 03:104, 03:152
\new@modulename 03:96, 03:152
\new@symbolfont 28:547, 28:589
\newcommand *332*
\NewCommandCopy *337*
\NewDocumentCommand *329*
\newlinechar *348*
\newmathalphabet@ 27:14
\newmathalphabet@@ 27:109
\newmathalphabet@@@ . . . 27:15, 27:109
\nfss@catcodes 24:20, 24:121, 24:484, 24:485, 24:492, 24:539, 30:41, 30:47, 30:57, 30:146, *1439*
\nfss@text . . 21:335, 21:337, 29:759, 32:5, 32:123, 35:17, 35:34, 35:51, *1360*
\no@alphabet@error 24:5, 28:515, 28:517, 28:733, 28:734, 28:748, 28:757, 28:843, 28:844, *1358*
\no@alphabet@help *1358*
\no@version@warning *1358*
\noaccents@ 24:730, 30:140
\noexpand *350*
\non@alpherr 24:709, 24:711, 28:73, 28:102, 28:118, 28:175, 28:206, 28:236, 28:268, 28:1305
\not@base . . . 29:876, 29:880, 29:881, 29:882, 29:883, 29:884, 29:885, 29:886, 29:887, 29:888, 29:889, 29:890
\not@math@alphabet 25:2915, 25:2920, 25:2925, 25:3203, 25:3207, 25:3210, 25:3213, 25:3216, 25:3219, 25:3222, 25:3225, 29:6, 29:9, 29:12, 29:15, 29:18, 29:21, 29:24, 29:27, 29:30, 29:334, 29:357, 29:387, 29:408, 29:432, 29:443, 29:460, 29:463, 29:485, 29:490, 29:495, 29:528, 29:533, 29:538, 29:554, 29:557, 29:560, 29:563, 29:566, 29:708
\now@and@everyjob 04:214, 04:220, 21:1035
\o@lign 21:411, 21:418, 21:499, 21:507, 21:784, 21:791, 02:410
\on@line 08:2163, 08:2177, 10:25, 14:8, 14:15, 14:214, 29:167, 29:170, 37:254, 37:275, 37:293, 37:303, 40:194, 50:996, 50:1131, 55:150, 55:163, 55:177, 08:538, 08:629, 08:1500, 08:1535, *1441*
\operator@font . 30:654, 38:3, 38:4, 38:5, 38:6, 38:7, 38:8, 38:9, 38:10, 38:11, 38:12, 38:13, 38:14, 38:15, 38:16, 38:17, 38:18, 38:19, 38:20, 38:21, 38:22, 38:23, 38:24, 38:25, 38:26, 38:27, 38:28, 38:29, 38:30, 38:31, 38:32, 38:33, 38:34, 38:37, 38:40
\optional@arg 26:432, 26:511, 26:513, 26:585, 26:588
\outer@nobreak 45:181, 45:251, 45:255, 45:346, 45:364, *1385*
\outputbox@append *1242*
\p@ 02:181
\p@... . . . . *1412*
\p@enum *922*
\p@equation 38:426, 38:455, 38:611
\p@renwd *1368*
\p@selectfont 26:120, *1358*
\par *191*
\par@deathcycles 39:56, 39:77, 39:79, 39:80
\patch@level 03:1, 03:39, 03:44, 03:46, 03:48, 03:52, 03:61, 57:740, 57:752, 57:754, *1408*
\patchcmd *329*
\pdfannot@link@off@@ 54:846, 54:915, 54:930
\pdfannot@link@on@@ 54:845, 54:923, 54:938
\ph@nt 38:81, 38:82, 38:83, 38:97
\pickup@font 21:199, 24:258, 24:450, 24:644, 24:678, 26:144, 26:171, 26:332, 26:334, 26:336
\pictur@ 42:21
\pkgcls@arg 50:1655, 50:1787
\pkgcls@candidate 50:1642, 50:1664, 50:1741, 50:1745, 50:1749, 50:1818, 50:1821
\pkgcls@debug . . . 50:1631, 50:1648, 50:1649, 50:1650, 50:1651, 50:1652, 50:1717, 50:1718, 50:1719, 50:1720, 50:1729, 50:1734, 50:1752, 50:1761, 50:1777, 50:1811, 50:1812, 50:1813
\pkgcls@ext . . . 50:1656, 50:1705, *1432*
\pkgcls@innerdate 50:140, 50:1636, 50:1689, 50:1692, 50:1693, 50:1699, 50:1839, *1442*
\pkgcls@mindate 50:1669, 50:1678, 50:1695, 50:1700, *1141*
\pkgcls@name 50:1654, 50:1705
\pkgcls@outerdate 50:44, 50:1636, 50:1692, *1442*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\pkgcls@parse@date@arg 50:1663, 50:1674
\pkgcls@parse@date@arg@ 50:1680, 50:1683
\pkgcls@parse@date@arg@version 50:1690, 50:1712
\pkgcls@releasedate 50:1642, 50:1746, 50:1750, 50:1822
\pkgcls@rollbackdate@error 50:1742, 50:1801, 50:1819
\pkgcls@show@selection 50:1769, 50:1775, 50:1825, 50:1830
\pkgcls@targetdate 50:1636, 50:1676, 50:1684, 50:1687, 50:1688, 50:1693, 50:1701, 50:1702, 50:1715, 50:1723, 50:1738, 50:1740, 50:1770, 50:1782, 50:1784, 50:1809, 50:1815, 50:1817, *1141*
\pkgcls@targetlabel 50:1636, 50:1677, 50:1698, 50:1713, 50:1725, 50:1757, 50:1791, 50:1829, 50:1832, *1141*
\pkgcls@use@this@release 50:1726, 50:1743, 50:1745, 50:1758, 50:1768, 50:1821
\pr@@@s 38:259, 38:267
\pr@@@t 38:262, 38:268
\pr@m@s 38:256, 38:257
\pr@tectedrel@x 06:77, 06:94, 06:143, 06:294, *83*
\preload@sizes 27:11, 27:94, *1357*
\prepare@family@series@update 29:118, 29:161, 29:162, 29:234, 29:237, 29:287, 29:290, 29:486, 29:491, 29:496, 29:529, 29:534, 29:539, *751*
\pretocmd *328*
\prim@s 38:253, 38:255, 38:267
\prime@s 38:254
\process@table 20:40, 20:110, 20:168, 28:367
\propagate@doendpe 39:140, *917*
\protectd@edf *1426*
\protected *307*
\protected@cmd *1390*
\protected@edef 22:359, 29:629, 29:681, 35:148, 35:161, 35:168, 35:179, 35:187, 35:194, 35:202, 35:204, 35:213, 35:219, 36:40, 36:45, 36:83, 36:84, 06:263, 06:357, 40:549, 40:566, 40:583, 44:60, 45:534, 45:552, 45:570, 50:574, 50:957, 50:1224, *1383*
\protected@file@percent 37:199, 37:206, 37:221, 37:229, 37:230, 44:165, 44:172, *870*
\protected@wlog 50:421, 50:423, 50:437, 50:451
\protected@write . . 20:202, 20:207, 35:101, 35:119, 35:128, 36:55, 44:189, 44:199, 46:14, 46:31, *1395*
\protected@xdef 06:357, 44:11, 45:515, 45:583, 45:599, 50:413, 50:432, 50:485, 50:854, *1383*
\provide@command 06:220, 06:219
\ps@empty 49:10, 57:158
\ps@plain 49:13
\q@curr@file 50:1252, 50:1296, 50:1298, 50:1303, 50:1329, 50:1427, 50:1429, 50:1434, 50:1462, *1417*
\quote@@name 20:481, 20:497
\quote@name 20:481, 20:494, 20:496, 20:607, 20:609, 20:618, 50:1427
\r@@t 38:66
\reenable@package@load 52:477, 53:517, *1164*
\reinstall@nfss@defs 25:3205, 25:3246, 25:3250, 25:3252, 25:3256, 25:3257, 25:3260
\relax *350*
\rem@pt 24:392
\remove@angles 26:364, 26:387
\remove@nil 28:37
\remove@star 26:364, 26:370
\remove@tlig 21:1025, 21:1027, 21:1029, 21:1053, 21:1058, 21:1062, 21:1109, 21:1110, 21:1111, *1417*
\remove@to@nnil 24:391, 26:364, 26:390, 26:503
\renew@command 06:164, 06:163, 06:226, 06:234
\renew@environment . . . 06:194, 06:193
\requested@test@context 29:582, 29:601, 29:605, *754*
\reserved@@b *726*
\reserved@a 03:13, 03:19, 03:34, 01:105, 01:109, 01:110, 03:181, 03:182, 13:33, 13:37, 14:234, 18:478, 18:481, 20:216, 20:217, 20:229, 20:230, 20:254, 20:263, 20:272, 20:325, 20:387, 20:434, 20:504, 20:506, 20:511, 20:513, 20:525, 20:528, 20:531, 20:534, 20:535, 20:536, 20:558, 20:560, 20:565, 20:567, 20:568, 20:569, 20:580, 20:582, 20:587, 20:589, 20:604, 20:610, 20:614, 20:621,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

20:627, 20:631, 20:660, 20:663, 20:664, 20:666, 20:675, 20:678, 20:685, 20:688, 20:710, 20:711, 20:712, 20:716, 20:724, 20:741, 20:742, 20:746, 20:752, 20:782, 20:786, 20:789, 20:794, 21:99, 21:100, 21:104, 21:107, 21:115, 21:125, 21:128, 21:137, 21:156, 21:161, 21:1019, 21:1023, 21:1071, 21:1073, 21:1074, 21:1076, 21:1078, 21:1086, 21:1095, 24:31, 24:32, 24:33, 24:42, 24:45, 24:48, 24:64, 24:67, 24:70, 24:106, 24:109, 24:111, 24:148, 24:152, 24:486, 01:179, 24:489, 01:180, 24:616, 24:617, 24:632, 24:635, 24:640, 24:651, 24:652, 24:665, 24:669, 24:674, 24:701, 24:704, 24:705, 24:713, 01:183, 01:201, 01:205, 25:2823, 25:2824, 25:2827, 25:2828, 25:2843, 25:2844, 25:2856, 25:2857, 25:3168, 25:3169, 25:3172, 25:3173, 25:3194, 25:3195, 26:197, 26:199, 26:201, 26:211, 26:213, 26:216, 05:26, 26:361, 26:362, 05:27, 05:28, 26:375, 26:376, 05:29, 05:30, 05:52, 05:57, 27:53, 27:57, 05:94, 05:100, 05:110, 01:227, 28:532, 28:535, 28:607, 28:610, 28:643, 28:652, 28:654, 28:698, 28:701, 05:144, 28:711, 28:714, 28:812, 28:814, 28:876, 28:880, 28:924, 28:926, 28:987, 28:990, 28:1082, 28:1084, 28:1178, 28:1180, 01:234, 28:1282, 28:1284, 28:1300, 28:1302, 28:1303, 28:1308, 29:41, 29:42, 29:74, 29:75, 01:237, 29:196, 29:197, 29:261, 29:262, 01:239, 01:240, 29:597, 29:598, 01:247, 01:250, 01:252, 01:253, 32:48, 32:49, 32:54, 32:55, 32:66, 32:69, 32:89, 32:96, 01:260, 06:140, 01:263, 06:145, 06:153, 06:156, 01:265, 06:174, 06:175, 06:176, 06:178, 06:225, 06:226, 06:227, 06:233, 06:234, 06:235, 06:236, 06:239, 35:144, 35:145, 35:146, 35:157, 35:158, 35:159, 35:202, 35:203, 35:204, 35:205, 35:213, 35:214, 35:219, 35:220, 36:40, 36:41, 36:45, 36:46, 36:83, 36:85, 06:263, 06:269, 37:197, 37:198, 37:252, 37:253, 37:258, 37:273, 37:274, 37:278, 37:291, 37:292, 37:296, 37:301, 37:302, 37:306, 37:391, 37:392, 06:330, 06:334, 06:392, 06:396, 38:511, 38:512, 38:513, 38:514, 38:516, 06:430, 06:434, 06:458, 06:462, 40:78, 40:79, 40:82, 40:97, 40:98, 40:101, 40:189, 40:195, 41:318, 41:322, 41:327, 41:346, 41:437, 41:438, 06:573, 42:210, 42:212, 42:216, 06:581, 06:582, 01:315, 01:316, 01:317, 42:488, 42:489, 42:516, 42:517, 42:545, 42:546, 06:614, 06:622, 06:623, 06:624, 06:625, 06:665, 06:668, 45:29, 45:30, 45:32, 45:33, 45:63, 45:67, 45:72, 45:74, 45:76, 45:78, 45:83, 45:84, 45:132, 45:136, 45:142, 45:145, 45:148, 45:151, 06:683, 06:685, 06:688, 06:697, 06:719, 06:726, 06:751, 06:753, 50:166, 50:169, 50:171, 50:265, 50:273, 50:277, 50:283, 50:291, 50:295, 50:483, 50:485, 50:486, 50:487, 50:491, 50:506, 50:508, 50:509, 50:510, 50:514, 06:872, 06:881, 50:691, 50:695, 50:701, 50:705, 50:783, 50:784, 50:787, 50:856, 50:860, 50:872, 50:873, 50:875, 50:884, 50:888, 50:900, 50:901, 50:903, 50:911, 50:915, 50:927, 50:928, 50:929, 50:931, 50:943, 50:946, 50:947, 50:949, 50:1033, 50:1082, 50:1099, 50:1140, 50:1263, 50:1264, 50:1266, 50:1398, 50:1399, 50:1401, 50:1443, 50:1444, 50:1446, 50:1450, 50:1666, 50:1671, 50:1724, 50:1725, 50:1756, 50:1757, 50:1828, 50:1829, 50:1853, 50:1855, 52:161, 52:167, 52:168, 52:169, 54:26, 54:35, 54:37, 54:38, 54:1228, 54:1248, 54:2574, 54:2576, 54:2577, 54:2676, 54:2678, 54:2684, 54:2687, 57:226, 57:243, 57:244, 57:245, 57:252, 57:253, 57:254, 57:538, 57:569, 57:575, 57:576, 57:587, 57:588, 57:589, 57:596, 57:597, 57:598, 57:623, 57:624, 57:631, 57:635, 57:637, 57:639, 57:644, 57:647, 57:655, 57:656, 57:657, 57:753, 57:756, 57:757, 57:774, 02:116, *1385*

`\reserved@b` . 01:106, 01:107, 13:33, 13:34, 13:37, 18:479, 18:480, 18:487, 20:323, 20:325, 20:385, 20:387, 20:432, 20:434, 20:605, 20:607, 20:609, 20:622, 20:624, 20:626, 20:710, 20:711, 20:712, 20:785,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

20:787, 20:788, 20:795, 21:108, 21:115, 21:130, 21:137, 21:1022, 21:1023, 21:1072, 21:1073, 21:1095, 21:1104, 21:1106, 24:32, 24:33, 24:39, 24:96, 24:98, 24:151, 24:152, 24:702, 24:713, 25:2842, 25:2843, 25:2845, 05:53, 27:47, 27:54, 05:59, 27:71, 27:73, 28:533, 28:534, 28:535, 28:539, 28:541, 28:608, 28:609, 28:610, 28:614, 28:616, 28:651, 28:652, 28:653, 28:688, 28:690, 28:769, 28:771, 28:816, 28:817, 28:818, 28:825, 28:985, 28:989, 28:991, 29:203, 29:208, 29:212, 29:213, 29:220, 29:221, 29:266, 29:273, 29:279, 32:53, 32:54, 32:67, 32:69, 32:96, 32:97, 06:127, 06:129, 06:141, 06:154, 06:176, 06:177, 36:84, 36:85, 06:331, 06:332, 06:334, 06:393, 06:394, 06:396, 06:431, 06:432, 06:434, 06:459, 06:460, 06:462, 41:323, 41:325, 41:327, 06:666, 06:668, 45:43, 45:44, 45:112, 45:113, 06:684, 06:688, 06:723, 06:726, 50:266, 50:267, 50:268, 50:270, 50:285, 50:288, 50:483, 50:506, 06:873, 06:883, 50:864, 50:870, 50:873, 50:892, 50:898, 50:901, 50:919, 50:925, 50:929, 50:943, 50:950, 50:1312, 50:1313, 50:1316, 50:1317, 50:1352, 50:1353, 50:1355, 50:1382, 50:1445, 50:1446, 50:1449, 50:1450, 50:1485, 50:1486, 50:1488, 50:1514, 50:1574, 50:1575, 50:1577, 50:1604, 54:1137, 54:1140, 54:1154, 54:1157, 54:1174, 54:1177, 57:229, 57:231, 57:235, 57:541, 57:543, 57:547, 57:627, 57:632, 57:673, 57:774, *572*

`\reserved@c` . 01:107, 01:112, 20:775, 24:97, 24:98, 24:703, 24:706, 27:48, 27:55, 27:61, 27:68, 28:34, 28:38, 28:540, 28:541, 28:615, 28:616, 28:689, 28:690, 28:770, 28:771, 28:793, 28:802, 28:817, 28:831, 28:1072, 28:1089, 28:1098, 28:1126, 28:1137, 28:1195, 28:1208, 28:1210, 29:205, 29:208, 29:218, 29:219, 29:222, 29:223, 29:268, 29:278, 29:280, 32:68, 32:70, 32:77, 06:878, 06:881, 06:883, 06:886, 50:944, 50:946, 50:947, 50:1297, 50:1302, 50:1303, 50:1321, 50:1329, 50:1335, 50:1357, 50:1365, 50:1428, 50:1433, 50:1434, 50:1454, 50:1462, 50:1468, 50:1490, 50:1497, 50:1545, 50:1546, 50:1547, 50:1557, 50:1579, 50:1586, 50:1614, 57:233, 57:238, 57:246, 57:545, 57:566, 57:567, 57:568, 57:570, 57:571, 57:572, 57:573, 57:574, 57:582, 57:590, 57:776, *1399*

`\reserved@d` 01:110, 01:113, 20:773, 20:775, 05:51, 05:56, 27:61, 27:68, 27:70, 27:74, 28:1080, 28:1089, 28:1098, 28:1134, 28:1137, 28:1203, 28:1208, 28:1212, 29:210, 29:211, 29:216, 29:217, 29:271, 29:272, 29:276, 29:277, 06:871, 06:880, 57:777, *1415*

`\reserved@e` 18:58, 18:60, 18:70, 18:72, 18:101, 18:108, 18:116, 27:39, 27:45, 27:70, 27:73, 27:74, 28:35, 28:40, 57:778, *1392*

`\reserved@f` 18:59, 18:60, 18:71, 18:72, 18:116, 21:1571, 21:1572, 21:1573, 21:1574, 21:1576, 21:1587, 24:253, 24:255, 24:261, 24:262, 26:399, 26:410, 26:414, 26:418, 26:424, 26:427, 26:466, 26:503, 26:506, 27:27, 27:38, 27:45, 27:71, 27:73, 57:779, *1385*

`\reset@font` 29:312, 29:764, 29:810, 29:826, 29:841, 29:856, 35:17, 35:34, 35:51, 40:545, 40:563, 40:580, 45:175, 45:371, 45:528, 45:547, 45:565, 47:40, 49:14, 54:901, 54:971, 54:1030, *1361*

`\restglb@settings` 26:269, 26:279

`\restore@mathversion` 28:108, 28:111, 28:126, 28:134

`\restore@protect` 06:357

`\rlh@` 30:475, 30:476

`\rm@def@ult` 29:476

`\rmdef@ult` 29:337, 29:360, 29:398, 29:418, 29:434, 29:445, 29:477, 29:518, 29:779, 29:780, 33:27, 33:48, *746*

`\robust@command@act` 09:106, 06:514, 06:515, 06:517, 06:585, 06:642, 06:779, *103*

`\robust@command@act@chk@args` 06:539, 06:560, *99*

`\robust@command@act@do` 06:525, 06:557, *99*

`\robust@command@act@end` . . 06:522, 06:523, 06:535, 06:538, 06:558, *99*

`\robust@command@act@loop` 06:519, 06:525, 06:555, *99*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\robust@command@act@loop@aux 06:525, 06:556
\robust@command@chk@safe 09:153, 06:407, 06:518, 06:539, 06:559, 06:699, 06:711, *337*
\s@fct@ 26:443, 26:507
\s@fct@alias 26:569
\s@fct@fixed 26:582
\s@fct@gen 26:519
\s@fct@genb 26:524
\s@fct@sgen 26:519
\s@fct@sgenb 26:524
\s@fct@sub 26:531
\s@fct@subf 26:574
\saved@reqcolroom 54:1454, 54:1911, 54:2600, 54:2728
\saved@space@catcode . 57:404, 57:473
\scan@@fontshape . . . 27:7, 27:40, 27:43
\scan@fontshape . . . 27:6, 27:26, 27:37
\scantokens *345*
\scriptfont@name 26:334, 26:339
\section *330*
\select@group 24:686, 24:705, 28:49, 28:407, 28:457, 28:520, 28:698, 28:751, 28:760, 28:798, 28:830, *707*
\series@change@debug 29:138, 29:167, 29:170, 29:180, 29:183, 29:187, 29:199, 29:207, 29:212, 29:218, 29:221, 29:223
\series@check@toks 25:2878, 25:2880, 25:2887
\series@drop@one@m 25:2884, 25:2888, 25:2906
\series@maybe@drop@one@m . . 24:32, 25:2871, 25:2873, 25:2905, 28:534, 28:609, 29:190, 29:209, 29:215, 29:255, 29:270, 29:275, 29:480, 29:481
\series@maybe@drop@one@m@x 25:2874, 25:2876
\seriesdefault@kernel 29:313, 29:930, *765*
\set@@mathdelimiter . 28:1135, 28:1169
\set@color 40:109
\set@curr@file . . . 20:245, 20:254, 20:281, 20:289, 20:471, 20:489, 50:975, 50:1295, 50:1426, 52:266, *1173*
\set@curr@file@aux 52:272, 52:278, 52:279
\set@curr@file@nosearch . 52:266, *1425*
\set@current@meta@family . 28:412, 29:773, 29:778, 29:812, 29:867, *763*
\set@display@protect 14:7, 14:14, 14:34, 14:61, 06:8, 06:16, 06:355, 50:424, *1383*
\set@fontsize 24:380, 24:382, 26:122, 26:168, 26:178, *1375*
\set@mathaccent 28:885, 28:893, 28:928, 28:936, 28:954
\set@mathchar 28:1009, 28:1019
\set@mathdelimiter 28:1086, 28:1095, 28:1147
\set@mathradical 28:467, 28:1205
\set@mathsymbol 28:993, 28:1001, 28:1022
\set@simple@size@args 26:365, 26:378, 26:385, 26:406, 26:420
\set@size@funct@args 26:368, 26:370, 26:428
\set@size@funct@args@ 26:428
\set@target@series 24:351, 24:363, 25:2825, 25:2829, 25:2832, 25:2835, 25:2858, 25:2860, 25:2869, 25:2904
\set@typeset@protect 06:355, 06:374, 41:249, 41:272, 41:312, 53:145, 53:189, 54:883, 54:885, 54:958, 54:960, 54:1016, 54:1018, *1397*
\SetMathAlphabet@ 28:705, 28:774, 28:783
\SetSymbolFont@ 28:565, 28:619, 28:637
\SetSymbolFont@m@dropped 28:607, 28:612, 28:632, 28:633
\sf@def@ult 29:476
\sf@size 19:6, 21:322, 24:287, 24:306, 24:725, 26:329, 26:333, 33:636, 45:413, 45:441, 45:447, 45:472, 45:500, 45:506
\sfdef@ult . . 29:338, 29:361, 29:399, 29:419, 29:435, 29:446, 29:478, 29:519, 29:783, 33:29, 33:50, *746*
\sh@ft 02:414, *1386*
\show@kernel@robust@command 06:705, 06:768, 06:796
\show@release@info 03:38, 03:41, 03:46, 03:51, 37:47, 37:48, 37:50, 37:112, 37:113, 37:115, *36*
\ShowCommand *337*
\sixt@@n 04:30, 24:15, 28:85, 28:249, 28:871, 28:873, 28:919, 28:921, 28:980, 28:982, 28:1028, 28:1030, 28:1068, 28:1070, 28:1076, 28:1078, 28:1122, 28:1124, 28:1130, 28:1132, 28:1191, 28:1193, 28:1199, 28:1201, 42:289, 42:304, 42:306, 45:62, 45:80, 45:131, 45:153, 02:16, 02:60, 02:62, 54:1356, 54:1402, 54:1542, 54:1687, 54:1854, 54:2169, 54:2233, 54:2290, 54:2360, 54:2630, 54:2639, 54:2695, 54:2711, 54:2749, 54:2799, 01:55
\sixt@@n␣ *421*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\size@update 26:147, 26:173, 26:186, 26:205, 26:207
\sizefn@info 26:369, 26:371, 26:379, 26:407, 26:421
\skip@ 17:79, 17:85, 17:98, 17:104, 32:106, 32:109, 02:41, 02:376, 02:378, 02:379, 02:381
\sp@ce@skip 18:84, 18:539, 18:540
\sp@n 41:456
\split@name 24:454, 24:466, 24:580, 24:595, 26:536, 26:550
\ssf@size . . 21:348, 21:380, 21:1282, 24:288, 24:307, 24:726, 26:329, 26:335
\string@makeletter 06:898, 50:949, 50:984, 50:985, 50:986, 52:169, *1418*
\strip@meaning *1368*
\strip@prefix 01:95, 24:683, 01:212, 06:332, 06:394, 06:432, 06:460, 01:307, 06:895
\strip@pt . . . 24:279, 24:285, 24:286, 24:287, 24:288, 24:301, 24:305, 24:392, 24:725, 24:726, 26:181, 02:418
\strong@currfont 29:686, 29:695
\strongem@force 29:693, 29:698, 29:702, 29:703
\strongfontdeclare@clist 29:681, 29:683, 29:687, 29:691, 29:696
\sub@sfcnt 26:531, 26:532, 26:533, 26:560
\subf@sfcnt 26:574, 26:575, 26:576
\subst@correction 24:86, 24:92
\subst@fontshape 27:8, 27:80
\subst@size 26:482
\sw@slant 32:92, 32:102
\t@st@ic 32:91, 32:95, *1363*
\target@meta@family@value 29:243, 29:262, 29:267, 29:269, *1439*
\target@series@value 29:172, 29:179, 29:182, 29:184, 29:188, 29:189, 29:190, 29:213, 29:219, 29:220, 29:222, 29:242, 29:250, 29:252, 29:255, 29:273, 29:278, 29:279, 29:280, *1415*
\tc@check@accent 33:73, 33:125, 33:127, 33:129, 33:131, 33:133, 33:135, 33:137, 33:139, 33:141, 33:143, 33:145, 33:147, 33:149, 33:151, 33:153, 33:155, 33:899, 33:975, 33:977, 33:979
\tc@check@symbol . . . 33:73, 33:175, 33:177, 33:179, 33:181, 33:183, 33:185, 33:187, 33:189, 33:191, 33:193, 33:195, 33:197, 33:199, 33:201, 33:203, 33:205, 33:207, 33:209, 33:211, 33:213, 33:215, 33:217, 33:219, 33:221, 33:223, 33:225, 33:227, 33:229, 33:231, 33:233, 33:235, 33:237, 33:239, 33:241, 33:243, 33:245, 33:247, 33:249, 33:251, 33:253, 33:255, 33:257, 33:259, 33:261, 33:263, 33:265, 33:267, 33:269, 33:271, 33:274, 33:276, 33:278, 33:280, 33:282, 33:284, 33:286, 33:288, 33:290, 33:292, 33:294, 33:296, 33:298, 33:300, 33:302, 33:304, 33:306, 33:308, 33:310, 33:312, 33:314, 33:318, 33:320, 33:322, 33:324, 33:326, 33:328, 33:330, 33:899, 33:969, 33:971, 33:973, 33:981, 33:983, 33:985, 33:987, 33:989, 33:991, 33:993, 33:995, 33:997, 33:999, 33:1001, 33:1003, 33:1005, 33:1007, 33:1009, 33:1011, 33:1013, 33:1015, 33:1017, 33:1019, 33:1021, 33:1023, 33:1025, 33:1027, 33:1029, 33:1031, 33:1033, 33:1035, 33:1037, 33:1039, 33:1041, 33:1043, 33:1045, 33:1047, 33:1049, 33:1051, 33:1053, 33:1055, 33:1057, 33:1059, 33:1061, 33:1063, 33:1065, 33:1067, 33:1069, 33:1071, 33:1073, 33:1075, 33:1077, 33:1079, 33:1081, 33:1083
\tc@error 33:74, 33:879, 33:900
\tc@errorwarn 33:21, 33:40, 33:42, 33:787, 33:789, 33:791, 33:792, 33:838, 33:839, 33:840, 33:873
\tc@fake@euro 33:61, 33:316, 33:887, 33:968
\tc@forcedfalse 33:832
\tc@forcedtrue 33:837
\tc@oldstylesubst 33:16, 33:20
\tc@subst . . 33:41, 33:73, 33:872, 33:899
\tc@swap@accent 33:76, 33:77
\test@font@series@context 29:584, 29:596, 29:618, *754*
\test@next *1391*
\text@command 32:8, 32:47, *1380*
\textfont@name 26:332, 26:338
\textsubscript@offset 45:463
\textsubscript@space 45:463
\textsuperscript@offset 45:404
\textsuperscript@space 45:404
\tf@size 24:286, 24:306, 24:724, 26:329, 26:331, *1361*
\thr@@ 26:59, 26:255, 26:261, 26:274, 26:281, 26:288, 26:293, 38:438, 38:467, 38:632, 39:263, 39:274, 42:298, 42:299,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

42:301, 42:302, 42:340, 42:366, 42:399, 42:422, 02:16, 54:855, 57:84, 57:92, 02:447, 02:481, *1389*
\toks@ 03:94, 03:106, 03:108, 03:115, 03:119, 03:122, 03:127, 18:477, 18:478, 18:483, 24:149, 24:153, 24:155, 24:158, 24:284, 24:289, 28:7, 28:8, 28:506, 28:510, 28:516, 28:519, 28:524, 28:590, 28:591, 28:593, 28:594, 28:644, 28:646, 28:650, 28:667, 28:670, 28:729, 28:741, 28:742, 28:743, 28:789, 28:791, 28:797, 28:805, 28:809, 28:821, 28:824, 28:827, 28:835, 28:837, 28:1258, 28:1260, 28:1262, 28:1265, 28:1267, 28:1270, 28:1273, 28:1305, 28:1306, 50:536, 50:537, 50:539, 50:540, 06:1012, 06:1013, 02:41, 54:2913, 54:2914, 54:2915, 54:2916, *660*
\topmark *1065*
\topmark(s) *1065*
\transform@scriptfont 26:333, 26:335, 26:343, *1439*
\try@load@font@shape *1400*
\try@load@fontshape 24:469, 24:477, 24:523, 24:628, 25:2867, 26:537, 28:379, 28:396, 28:429, 28:446, *659*
\try@simple@size 26:373, 26:498
\try@simples . . . 26:456, 26:462, 26:466
\try@size@range 26:102, 26:373, 26:449
\try@size@substitution 26:104, 26:453
\tryif@simple 26:464, 26:465
\tryis@simple 26:465
\tt@def@ult 29:476
\ttdef@ult . . 29:339, 29:362, 29:400, 29:420, 29:436, 29:447, 29:479, 29:520, 29:786, 33:31, 33:52, *746*
\tw@ 02:16, *421*
\two@digits 01:169, 01:170, 26:529, 06:2, 50:1248, 50:1341, 50:1474, 50:1563, 01:70
\type@restoreinfo 26:203, 26:208
\unconditionally@reset@math@version 24:405, 24:413
\undeclare@... . . . . *1210*
\undeclare@file@substitution 52:247, *1164*
\underline@text 40:615
\underline@text@kern 40:615
\underline@text@thickness . . . 40:615
\underline@text@vgap 40:615
\unexpandable@noexpand *1394*
\unexpanded *1060*
\unqu@tefilef@und 52:143
\unquote@name 20:475, 20:481, 20:498, 52:349
\unrestored@protected@xdef 06:357, 45:520, 45:588, 45:604, 45:615, 48:253, 49:41, 49:67, 49:90, 49:123
\unskip *1071*
\update@series@target@value 29:119, 29:174, 29:195, 29:245, 29:260, 29:291
\update@uclc@with@cyrillic 21:1533, 21:1561, 21:1595, 21:1603
\upper@bound 26:400, 26:401, 26:402, 26:415
\use@mathgroup . . . 24:692, 24:710, 24:712, 26:300, 28:64, 28:93, 28:711, 28:813, 28:816, 28:1283, 28:1307, *1358*
\UTF@two@octets@noexpand *1425*
\UTFviii@four@octets 57:459, 57:464, 57:470
\UTFviii@four@octets@@ 57:459, 57:470
\UTFviii@four@octets@combine 57:494
\UTFviii@four@octets@noexpand 57:500
\UTFviii@four@octets@string . 57:497
\UTFviii@invalid 57:398, 57:491
\UTFviii@invalid@err 57:456, 57:461, 57:467
\UTFviii@invalid@err@@ 57:456, 57:467
\UTFviii@three@octets 57:458, 57:463, 57:469
\UTFviii@three@octets@@ 57:458, 57:469
\UTFviii@three@octets@combine 57:493
\UTFviii@three@octets@noexpand 57:499
\UTFviii@three@octets@string 57:496
\UTFviii@two@octets 57:457, 57:462, 57:468
\UTFviii@two@octets@@ 57:457, 57:468
\UTFviii@two@octets@combine . 57:492
\UTFviii@two@octets@noexpand 57:498
\UTFviii@two@octets@string . . 57:495
\UTFviii@undefined@err 57:455, 57:460, 57:466
\UTFviii@undefined@err@@ 57:455, 57:466
\v@false 38:82
\v@true 38:81, 38:83
\vbox *1071*
\vcenter@text . . 40:330, 40:380, 41:235
\vcenter@text@auxi 40:330
\vcenter@text@auxii 40:330
\vcenter@text@axis 40:330
\ver@... . . . . *507*
\ver@@... . . . . *507*
\ver@⟨*file*⟩.⟨*ext*⟩ *1123*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\verb@balance@group 37:656, 37:658, 37:673, 37:675, 37:687, 37:689, 37:702, 37:704, 37:710, 37:711
\verb@egroup 37:656, 37:659, 37:673, 37:676, 37:687, 37:690, 37:702, 37:705, 37:711, 37:715
\verb@eol@error 37:712, 37:724, 37:734
\verbatim@font 37:544, 37:565, 37:573, 37:725, 37:735, *1363*
\verbatim@nolig@list . 37:740, 37:746
\verbatim@out *1399*
\version@elt 28:19, 28:32, 28:33, 28:503, 28:504, 28:563, 28:593, 28:704, 28:742, 28:834, 28:1263
\version@list 28:17, 28:22, 28:33, 28:496, 28:504, 28:568, 28:599, 28:638, 28:709, 28:754, 28:784, 28:839, 28:1276
\vgl@ 02:376, 02:377
\voidb@x . . 23:23, 23:29, 02:181, 02:407
\vsplit *1070*
\warn@rel@i 27:5, 27:25, 27:29, 27:81, 27:85, 27:90, 27:95, 27:119, 27:140
\wrong@fontshape . 24:473, 24:610, *1357*
\x@protect . . 06:335, 06:346, 06:397, 06:435, 06:463, 06:669, 06:689, *1383*
\xe@alloc@ 57:56, 57:66
\xe@alloc@intercharclass . 57:35, *1407*
\xe@ch@ck 57:57, 57:61
\XXX@argdef *1370*
\z@ 02:181, *1396*
\z@skip 02:181, *1396*
\zap@space . . 20:281, 20:301, 29:629, 29:681, 47:49, 50:266, 50:488, 50:511, 50:523, 50:690, 50:814, 50:854, 50:872, 50:883, 50:900, 50:910, 50:927, 50:959, 50:1261, 50:1396
\zref@labelbyprop *855*

tex commands:

\tex_afterassignment:D 53:110, 53:199, 07:2798
\tex_aftergroup:D 53:118, 53:205, *1195*
\tex_alignmark:D 09:388
\tex_currentgrouplevel:D 10:33, 53:109, 53:117, 53:198, 53:204
\tex_deadcycles:D 53:152
\tex_endlinechar:D 09:481, 07:1615, 07:2102, 07:2204, 07:2205, 07:2215, 07:2216, 07:2223, 07:2245, 07:2260, *186*
\tex_escapechar:D 09:218, 09:293, 48:119, 48:120, 48:122, 52:212, 52:226, 07:1137, 07:1148, 07:1356, 07:1366, 07:1614, 07:2101, 07:2672, 07:2698, 08:1186, *280*
\tex_everypar:D 16:20, 16:24, 16:29, 16:53, 16:57, 16:61, 16:76, 16:76, 16:118, 16:120, 16:130, 16:131, 16:182, 16:183, *448*
\tex_gdef:D 09:208, 09:283, *342*
\tex_hskip:D 16:104
\tex_ignoreprimitiveerror:D 48:101, 48:104, 48:105, 48:108
\tex_immediate:D 53:529
\tex_indent:D 16:125
\tex_interactionmode:D 48:117, 48:118, 48:123
\tex_lastnodetype:D 16:103, 48:81, *454*
\tex_lastxpos:D 36:266
\tex_lastypos:D 36:267
\tex_lowercase:D 07:2495, 07:2678
\tex_marks:D 48:261
\tex_newlinechar:D 09:482
\tex_noindent:D 16:27, 16:59, 16:135, *449*
\tex_par:D 16:18, 16:51, 16:106, 16:113, 16:137, 16:178, 16:179, 16:180, *454*
\tex_parskip:D . . . 16:25, 16:26, 16:58
\tex_savepos:D *850*
\tex_setbox:D . . 48:124, 53:111, 53:200
\tex_shipout:D . 53:187, 53:433, 53:584
\tex_splitbotmarks:D 48:144, 48:175, *1074*
\tex_splitfirstmarks:D 48:156, 48:194, 48:203, *1074*
\tex_splitmaxdepth:D 48:69
\tex_unskip:D 16:99, 48:79, *453*
\tex_vbadness:D 48:70
\tex_vfuzz:D 48:71
\tex_vsplit:D 48:124, *1074*
\tex_vss:D 53:39, 53:59
\tex_write:D 53:529
\tex_xdef:D 09:214, 09:289, *342*

\text *1360*

text commands:

\l_text_case_exclude_arg_tl . 57:676
\text_case_switch:nnnn 57:679
\text_declare_case_equivalent:Nn 57:681
\text_declare_lowercase_exclusion:n 57:706
\text_declare_lowercase_mapping:nn 57:685
\text_declare_lowercase_mapping:nnn 57:686
\text_declare_titlecase_exclusion:n 57:711
\text_declare_titlecase_mapping:nn 57:692

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\text_declare_titlecase_mapping:nnn 57:693
\text_declare_uppercase_exclusion:n 57:716
\text_declare_uppercase_mapping:nn 57:699
\text_declare_uppercase_mapping:nnn 57:700
\text_lowercase:n *1425*
\text_titlecase_all:nn 57:658
\text_titlecase_first:nn 57:659
\textacutedbl 21:939, 21:1181, 33:198, 33:199, 33:685, 33:932, 33:1313
\textascendercompwordmark 21:878, 33:118, 33:649, 33:915, 33:1344
\textasciiacute 21:989, 21:1142, 33:200, 33:201, 33:686, 33:956, 33:1332
\textasciibreve 21:937, 21:1180, 33:202, 33:203, 33:687, 33:929, 33:1310
\textasciicaron 21:938, 21:1179, 33:204, 33:205, 33:688, 33:930, 33:1311
\textasciicircum 21:306, 21:581, 21:1116, *1394*
\textasciidieresis . . 21:977, 21:1129, 33:206, 33:207, 33:689, 33:946, 33:1329
\textasciigrave 21:928, 21:1110, 33:208, 33:209, 33:690, 33:927, 33:1301
\textasciimacron . . . 21:984, 21:1137, 33:210, 33:211, 33:691, 33:951, 33:1312
\textasciitilde 21:307, 21:582, 21:1121, *1410*
\textasteriskcentered 21:287, 21:741, 21:888, 21:889, 21:1243, 22:331, 22:337, 33:84, 33:541, 33:606, 33:922, 33:1347, *1410*
\textbackslash 21:288, 21:583, 21:742, 21:1115, *1381*
\textbaht 21:963, 21:1184, 33:216, 33:217, 33:695, 33:1062, 33:1063, 33:1323
\textbar 21:289, 21:584, 21:743, 21:1119, *1381*
\textbardbl 21:290, 21:744, 21:943, 21:1198, 22:336, 33:91, 33:329, 33:330, 33:542, 33:614, 33:761, 33:935, 33:1228, *1403*
\textbf 32:19, 36:261, *739*
\textbigcircle 21:753, 21:916, 21:1260, 33:218, 33:219, 33:696, 33:1014, 33:1015, 33:1298, *1404*
\textblank . . . 21:885, 21:1257, 33:309, 33:310, 33:749, 33:984, 33:985, 33:1235
\textborn 21:929, 33:220, 33:221, 33:351, 33:697, 33:1020, 33:1021, 33:1302
\textbraceleft 21:291, 21:328, 21:585, 21:745, 21:1118, *1382*
\textbraceright 21:292, 21:329, 21:586, 21:746, 21:1120, *1382*
\textbrokenbar 21:975, 21:1127, 33:92, 33:615, 33:944, 33:1356
\textbullet 21:293, 21:747, 21:945, 21:1207, 33:85, 33:543, 33:607, 33:937, 33:1351
\textcapitalcompwordmark 21:877, 33:117, 33:648, 33:914, 33:1343
\textcelsius 21:946, 21:1226, 33:93, 33:317, 33:318, 33:616, 33:755, 33:938, 33:1229
\textcent 21:971, 21:1123, 33:94, 33:617, 33:941, 33:1353
\textcentoldstyle 21:948, 33:222, 33:223, 33:356, 33:359, 33:698, 33:1036, 33:1037, 33:1316, *809*
\textcircled . . . 21:299, 21:303, 21:320, 21:321, 21:754, 21:917, 33:119, 33:120, 33:619, 33:635, 33:651, 33:819, 33:1084, 33:1086, 33:1337, *833*
\textcircledP 21:982, 21:1228, 33:224, 33:225, 33:699, 33:1078, 33:1079, 33:1331
\textcolonmonetary 21:950, 21:1219, 33:277, 33:278, 33:729, 33:1038, 33:1039, 33:1259
\textcommaabove 21:373, 21:375, 21:389, 21:390, 21:480, 21:481, 21:723, 21:724, *1407*
\textcommabelow 21:344, 21:346, 21:352, 21:353, 21:726, 21:727, 21:728, 21:729, 21:730, 21:731, 21:732, 21:733, 21:734, 21:735, 21:1280, 21:1483, 21:1484, 21:1485, 21:1486, *1407*
\textcompsubstdefault 33:34, 33:39, 33:55, 33:582, 33:877, *1414*
\textcompwordmark 21:310, 21:311, 21:587, 21:1186, *1414*
\textcopyleft 21:980, 33:226, 33:227, 33:332, 33:700, 33:1076, 33:1077, 33:1330
\textcopyright 21:303, 21:337, 21:978, 21:1130, 33:95, 33:618, 33:947, 33:1358, *1399*
\textcurrency 21:973, 21:1125, 33:299, 33:300, 33:742, 33:814, 33:818, 33:972, 33:973, 33:1248
\textdagger 21:295, 21:332, 21:749, 21:941,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

21:1205, 22:332, 22:338, 33:87, 33:545, 33:609, 33:933, 33:1348, *1382*
\textdaggerdbl 21:294, 21:333, 21:748, 21:942, 21:1206, 22:333, 22:339, 33:86, 33:544, 33:608, 33:934, 33:1349, *1382*
\textdblhyphen . 21:900, 33:230, 33:231, 33:333, 33:702, 33:986, 33:987, 33:1286
\textdblhyphenchar 21:936, 33:228, 33:229, 33:334, 33:701, 33:1032, 33:1033, 33:1309
\textdegree 21:985, 21:1138, 33:96, 33:620, 33:952, 33:1362
\textdied 21:931, 33:232, 33:233, 33:352, 33:703, 33:1024, 33:1025, 33:1304
\textdiscount 21:965, 21:1218, 33:234, 33:235, 33:704, 33:1066, 33:1067, 33:1324
\textdiv 21:1002, 21:1163, 33:97, 33:621, 33:966, 33:1371
\textdivorced 21:930, 21:1263, 33:236, 33:237, 33:705, 33:1022, 33:1023, 33:1303
\textdollar 21:275, 21:327, 21:459, 21:588, 21:822, 21:886, 21:1112, 33:78, 33:79, 33:587, 33:589, 33:920, 33:1092, 33:1094, 33:1345, *1381*
\textdollaroldstyle 21:947, 33:238, 33:239, 33:357, 33:358, 33:706, 33:1034, 33:1035, 33:1315, *809*
\textdong 21:959, 21:1223, 33:279, 33:280, 33:730, 33:1056, 33:1057, 33:1262
\textdownarrow 21:927, 21:1238, 33:281, 33:282, 33:731, 33:1018, 33:1019, 33:1258
\texteightoldstyle 21:910, 33:178, 33:179, 33:348, 33:674, 33:1004, 33:1005, 33:1295
\textellipsis 21:316, 21:341, 21:1208
\textemdash 21:276, 21:428, 21:432, 21:589, 21:593, 21:809, 21:1190, *1381*
\textendash 21:277, 21:429, 21:431, 21:590, 21:592, 21:810, 21:1189, *1381*
\textestimated 21:966, 21:1234, 33:293, 33:294, 33:738, 33:817, 33:970, 33:971, 33:1252
\texteuro 21:1000, 21:1224, 33:315, 33:316, 33:753, 33:815, 33:967, 33:968, 33:1238
\textexclamdown 21:278, 21:433, 21:435, 21:594, 21:811, 21:1122, *1381*
\textfiguredash 21:431, 21:592, 21:1188, 21:1194, *1420*
\textfiveoldstyle 21:907, 33:180, 33:181, 33:345, 33:675, 33:998, 33:999, 33:1292
\textfloatsep 54:1079, 54:1092, 54:2730, 54:2780, 54:2830, 54:3004, *1234*
\textflorin 21:949, 21:1177, 33:297, 33:298, 33:741, 33:939, 33:1247
\textfont 26:338, 38:251, 40:353, 40:649, 40:650, 40:651, 45:428, 45:429, 45:431, 45:487, 45:488, 45:490
\textfouroldstyle 21:906, 33:182, 33:183, 33:344, 33:676, 33:996, 33:997, 33:1291
\textfraction 54:2527, 54:2530, 54:2555, 54:2558, 54:2717, 54:2998, *1400*
\textfractionsolidus 21:901, 21:1215, 33:301, 33:302, 33:744, 33:923, 33:1241, 33:1473, *1400*
\textgravedbl 21:940, 21:1182, 33:212, 33:213, 33:692, 33:931, 33:1314
\textgreater 21:301, 21:595, 21:764, 21:1114, *1387*
\textguarani 21:953, 33:240, 33:241, 33:355, 33:707, 33:1044, 33:1045, 33:1318
\textheight 20:22, 20:23, 20:92, 20:93, 20:149, 20:150, 45:257, 45:258, 45:261, 45:287, 45:301, 53:433, 54:54, 54:230, 54:231, 54:279, 54:404, 54:452, 54:481, 54:942, 54:1000, 54:1059, 54:1114, 54:1166, 57:156, 57:157, *1372*
\texthorizontalbar 21:432, 21:593, 21:1191, 21:1196, *1420*
\texthyphen 21:280, 21:438, 21:597, 21:813, *1387*
\texthyphenchar 21:279, 21:437, 21:596, 21:812, *1381*
\textindent *1388*
\textinterrobang 21:957, 21:1214, 33:313, 33:314, 33:751, 33:1052, 33:1053, 33:1236
\textinterrobangdown 21:958, 21:1264, 33:311, 33:312, 33:750, 33:1054, 33:1055, 33:1237
\textit 32:21
\textlangle 21:912, 21:1255, 33:273, 33:274, 33:726, 33:1008, 33:1009, 33:1265
\textlbrace *1381*
\textlbrackdbl 21:924, 33:174, 33:175, 33:353, 33:671, 33:925, 33:1299
\textleaf 21:932, 33:242, 33:243, 33:337, 33:708, 33:1026, 33:1027, 33:1305

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\textleftarrow 21:883, 21:1235, 33:283, 33:284, 33:732, 33:980, 33:981, 33:1255
\textlegacyasteriskcentered 33:553, 33:764
\textlegacybardbl 33:553, 33:765
\textlegacybullet 33:553, 33:766
\textlegacydagger 33:553, 33:768
\textlegacydaggerdbl 33:553, 33:767
\textlegacyparagraph 33:553, 33:769
\textlegacyperiodcentered 33:553, 33:770
\textlegacysection 33:553, 33:771
\textless 21:300, 21:598, 21:763, 21:1113, *1394*
\textlira 21:955, 21:1220, 33:285, 33:286, 33:733, 33:1048, 33:1049, 33:1261
\textlnot 21:981, 21:1135, 33:98, 33:622, 33:949, 33:1360
\textlquill 21:969, 21:1216, 33:244, 33:245, 33:709, 33:1072, 33:1073, 33:1327
\textmacron *1401*
\textmarried 21:933, 21:1262, 33:246, 33:247, 33:710, 33:1028, 33:1029, 33:1306
\textmd 32:19
\textmho 21:915, 21:1233, 33:248, 33:249, 33:711, 33:1012, 33:1013, 33:1297
\textminus . . . 21:913, 21:1239, 33:307, 33:308, 33:747, 33:924, 33:1242, *813*
\textmu 21:990, 21:1143, 33:305, 33:306, 33:746, 33:957, 33:1244
\textmusicalnote 21:934, 21:1261, 33:250, 33:251, 33:712, 33:1030, 33:1031, 33:1307
\textnaira 21:952, 21:1221, 33:252, 33:253, 33:713, 33:1042, 33:1043, 33:1317
\textnineoldstyle 21:911, 33:184, 33:185, 33:349, 33:677, 33:1006, 33:1007, 33:1296
\textnonbreakinghyphen 21:430, 21:591, 21:1187, 21:1192, *1420*
\textnormal 32:15
\textnumero 21:964, 21:1227, 33:295, 33:296, 33:739, 33:1064, 33:1065, 33:1251
\textogonekcentered 21:510, 21:721, 21:722, *1404*
\textohm 21:923, 21:1232, 33:303, 33:304, 33:745, 33:816, 33:969, 33:1243
\textonehalf 21:998, 21:1152, 33:99, 33:623, 33:963, 33:1368
\textoneoldstyle 21:903, 33:186, 33:187, 33:341, 33:678, 33:990, 33:991, 33:1288
\textonequarter 21:997, 21:1151, 33:100, 33:624, 33:962, 33:1367
\textonesuperior 21:994, 21:1146, 33:101, 33:319, 33:320, 33:625, 33:756, 33:960, 33:1232
\textopenbullet 21:967, 21:1259, 33:254, 33:255, 33:714, 33:1068, 33:1069, 33:1325
\textordfeminine 21:325, 21:979, 21:1131, 33:102, 33:626, 33:948, 33:1359
\textordmasculine . . . 21:326, 21:995, 21:1147, 33:103, 33:628, 33:961, 33:1366
\TextOrMath 22:328, 22:331, 22:332, 22:333, 22:334, 22:335, 22:336, 22:337, 22:338, 22:339, 22:344, 22:351, *1406*
\textparagraph . 21:296, 21:330, 21:750, 21:991, 21:1144, 22:335, 33:88, 33:546, 33:610, 33:958, 33:1364, *1382*
\textperiodcentered 21:297, 21:751, 21:992, 21:1145, 33:89, 33:547, 33:611, 33:959, 33:1365
\textpermill *1400*
\textpertenmill *1400*
\textpertenthousand 21:515, 21:961, 21:1210, 33:270, 33:271, 33:272, 33:723, 33:1058, 33:1059, 33:1096, 33:1321, *1400*
\textperthousand 21:513, 21:944, 21:1209, 33:82, 33:83, 33:603, 33:936, 33:1095, 33:1350, *1400*
\textpeso 21:954, 21:1225, 33:256, 33:257, 33:715, 33:1046, 33:1047, 33:1319
\textpilcrow 21:962, 33:258, 33:259, 33:350, 33:716, 33:1060, 33:1061, 33:1322
\textpm 21:986, 21:1139, 33:104, 33:630, 33:953, 33:1363
\textquestiondown . . . 21:281, 21:434, 21:436, 21:599, 21:814, 21:1154, *1381*
\textquotedbl 21:602, 21:1111, *1381*
\textquotedblleft 21:282, 21:439, 21:600, 21:815, 21:1202, *1381*
\textquotedblright 21:283, 21:440, 21:601, 21:816, 21:1203, *1382*
\textquoteleft 21:284, 21:441, 21:603, 21:817, 21:1199, *1381*
\textquoteright 21:285, 21:442, 21:604, 21:818, 21:1200, *1381*
\textquotesingle 21:887, 21:1109, 33:105, 33:631, 33:921, 33:1346
\textquotestraightbase 21:879, 33:106, 33:335, 33:632, 33:916, 33:1341

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\textquotestraightdblbase . . . 21:880, 33:107, 33:336, 33:633, 33:917, 33:1342
\textrangle 21:914, 21:1256, 33:275, 33:276, 33:727, 33:1010, 33:1011, 33:1266
\textrbrace *1381*
\textrbrackdbl 21:925, 33:176, 33:177, 33:354, 33:672, 33:926, 33:1300
\textrecipe 21:956, 21:1229, 33:260, 33:261, 33:717, 33:1050, 33:1051, 33:1320
\textreferencemark 21:993, 21:1213, 33:262, 33:263, 33:718, 33:1080, 33:1081, 33:1333
\textregistered 21:320, 21:321, 21:983, 21:1136, 33:108, 33:634, 33:950, 33:1361, *1404*
\textrightarrow 21:884, 21:1237, 33:287, 33:288, 33:734, 33:982, 33:983, 33:1256
\textrm 32:15
\textrquill 21:970, 21:1217, 33:264, 33:265, 33:719, 33:1074, 33:1075, 33:1328
\textsc 32:21, *1399*
\textsection 21:298, 21:331, 21:605, 21:752, 21:976, 21:1128, 22:334, 33:90, 33:548, 33:549, 33:612, 33:945, 33:1357, *1382*
\textservicemark 21:968, 21:1230, 33:266, 33:267, 33:720, 33:1070, 33:1071, 33:1326
\textsevenoldstyle 21:909, 33:188, 33:189, 33:347, 33:679, 33:1002, 33:1003, 33:1294
\textsf 32:15
\textsixoldstyle 21:908, 33:190, 33:191, 33:346, 33:680, 33:1000, 33:1001, 33:1293
\textsl 32:21
\textssc 25:2924, 32:25
\textsterling . . 21:286, 21:339, 21:466, 21:606, 21:829, 21:972, 21:1124, 33:80, 33:81, 33:588, 33:596, 33:942, 33:1091, 33:1093, 33:1354, *1381*
\textstyle 19:15, 30:480, 38:63, 40:351, 40:645, 40:646, 40:647, 45:420, 45:421, 45:423, 45:479, 45:480, 45:482
\textsubscript 45:450
\textsuperscript 21:323, 21:325, 21:326, 33:627, 33:629, 33:643, 45:402, *1388*
\textsurd 21:996, 21:1254, 33:268, 33:269, 33:721, 33:1082, 33:1083, 33:1334
\textsw 25:2919, 32:25
\TextSymbolUnavailable 21:3, 21:782, *1394*
\textthreeoldstyle 21:905, 33:192, 33:193, 33:343, 33:681, 33:994, 33:995, 33:1290
\textthreequarters 21:999, 21:1153, 33:110, 33:639, 33:964, 33:1369
\textthreequartersemdash 21:882, 33:109, 33:321, 33:322, 33:339, 33:638, 33:757, 33:919, 33:1227
\textthreesuperior 21:988, 21:1141, 33:111, 33:323, 33:324, 33:640, 33:758, 33:955, 33:1231
\texttildelow 21:935, 21:1183, 33:214, 33:215, 33:693, 33:928, 33:1308
\texttimes 21:1001, 21:1157, 33:112, 33:641, 33:965, 33:1370
\texttrademark 21:323, 21:960, 21:1231, 33:113, 33:642, 33:940, 33:1352, *1394*
\texttt 32:15
\texttwelveudash 21:881, 33:114, 33:325, 33:326, 33:338, 33:644, 33:759, 33:918, 33:1226
\texttwooldstyle 21:904, 33:194, 33:195, 33:342, 33:682, 33:992, 33:993, 33:1289
\texttwosuperior 21:987, 21:1140, 33:115, 33:327, 33:328, 33:645, 33:760, 33:954, 33:1230
\textulc 25:2914, 32:25
\textunderline *1381*
\textunderscore 21:308, 21:335, 21:607, 21:1117, *1390*
\textup 32:21, *663*
\textuparrow 21:926, 21:1236, 33:289, 33:290, 33:735, 33:1016, 33:1017, 33:1257
\textvisiblespace 21:312, 21:608, 21:1258, *1390*
\textwidth 20:24, 20:94, 20:151, 40:507, 45:266, 54:55, 54:123, 54:206, 54:223, 54:920, 54:935, 54:985, 54:995, 54:1044, 54:1054, 54:2884, 54:2927, 54:2956, 57:157, *1372*
\textwon 21:951, 21:1222, 33:291, 33:292, 33:736, 33:1040, 33:1041, 33:1260
\textyen 21:974, 21:1126, 33:116, 33:646, 33:943, 33:1355
\textzerooldstyle 21:902, 33:196, 33:197, 33:340, 33:683, 33:988, 33:989, 33:1287
\TH 21:558, 21:1159, 57:720, *1377*
\th 21:609, 21:1165, 57:720, *1377*
\thanks *1012*
\thanks 44:10, 44:26, *1383*
\the *1361*
\the... . . . . *1431*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\the#1 . . . . . *560*
thebibliography (env.) *1053*
\theenum *922*
\theequation 38:410, 38:426, 38:455, 38:550, 38:611
\thefootnote 45:396, 45:583, 45:588, 45:608
\theH *558*
\theHcounter *561*
\thempfn 40:517, 45:515, 45:520, 45:599, 45:604, 45:607
\thempfootnote 40:517, 45:398
\thepage 20:213, 20:226, 34:6, 35:18, 35:35, 35:52, 35:105, 35:120, 35:129, 36:233, 36:245, 36:250, 36:257, 44:164, 44:171, 44:177, 46:15, 46:32, 47:43, 49:14, 54:249, 54:280, 54:2404, *1206*
\Theta 30:310
\theta 30:286
\thetotalpages 53:397, 53:495, *1206*
\thicklines 42:135
\thickmuskip 30:657, 38:228, 38:230, 38:243
\thickspace 38:214
\thinlines 42:135, 42:826, 42:843
\thinmuskip 30:655, 38:220, 38:222, 38:238, 38:244
\thinspace 18:558, 18:564, 18:565, 38:189, 38:214, 38:251, *1415*
\thispagestyle 49:6
\tilde 18:494, 18:505, 30:530
\time 01:163, 01:167, 02:258
\times 30:407
\title *1012*
\title 44:6, 44:7, 44:21, 44:23, 44:31
title 36:260, *854*
tl commands:
\c_empty_tl 08:2242, 08:59
\c_novalue_tl . . 07:3281, 07:3282, *215*
\c_space_tl 09:398, 11:698, 11:702, 11:743, 11:1110, 11:1113, 16:42, 16:73, 16:82, 52:553, 52:556, 07:27, 07:124, 07:140, 07:143, 07:157, 07:168, 07:169, 07:185, 07:193, 07:203, 07:206, 07:208, 07:210, 07:212, 07:220, 07:226, 07:294, 07:295, 57:653, 57:660, 57:665, 57:670, 07:1192, 07:1207, 07:1239, 07:1240, 07:1417, 07:1424, 07:1556, 07:2813
\tl_clear:N 09:212, 09:287, 11:375, 11:414, 11:897, 11:1017, 11:1252, 07:309, 07:354, 07:382, 07:450, 07:451, 07:671, 07:791, 07:804, 07:805, 07:806, 07:808, 07:990, 07:1058, 07:1252, 07:1491, 07:1492, 07:1564, 07:1585, 07:1613, 07:1874, 07:1875, 07:1891, 07:2103, 07:2483, 07:2561
\tl_const:Nn 09:11, 09:12, 11:48, 11:49, 11:50, 11:51, 11:52, 11:53, 11:54, 11:55, 48:62, 53:372, 53:377, 07:1514, 07:2833, 07:3282, 08:35, 08:36, 08:104, 08:107, 08:941, 08:942, 08:944, 08:945, 08:946, 08:947, 08:950, 08:951, 08:953, 08:974, 08:975, 08:1319
\tl_count:N . . . 52:546, 07:400, 07:2507
\tl_count:n . 07:602, 07:607, 07:893, 07:1011, 07:1065, 07:1292, 07:1442
\tl_gclear:N 48:163, 48:164, *306*
\tl_gclear_new:N . 36:25, 36:167, 08:264
\tl_gput_left:Nn 51:265
\tl_gput_right:Nn 16:38, 16:69, 16:78, 28:336, 48:188, 48:199, 07:1167, 07:1484, 07:1486, 08:455, *289*
\tl_gset:Nn . . 09:140, 16:15, 16:48, 17:128, 17:132, 17:152, 17:156, 28:285, 28:333, 36:26, 36:160, 36:168, 48:143, 48:154, 48:174, 48:238, 48:239, 48:240, 52:51, 55:327, 55:328, 55:348, 55:352, 55:396, 55:404, 55:450, 08:176, 08:188, 08:295, 08:313, 08:411, 08:423, 08:445, 08:489, 08:771, 08:825, 08:859, 08:863, 08:886, 08:890
\tl_gset_eq:NN 48:58, 48:59, 48:60, 48:141, 48:142, 48:147, 48:149, 48:153, 48:216, 48:218, 48:220, 48:227, 48:229, 48:231, 48:469, 48:471, 48:473, 48:475, 48:481, 48:485, 48:488, 55:427, 55:428, 08:57, 08:452, 08:463
\tl_head:N 07:2735
\tl_head:w 11:162, 11:175
\tl_if_blank:nTF . . . 11:42, 11:422, 11:426, 11:599, 11:602, 11:627, 11:630, 17:31, 20:540, 57:633, 07:598, 07:1625, 07:1749, 07:1857, 07:2317, 07:2354, 07:2560, 07:3304, 07:3305, 07:3306, 08:429, 08:435, 08:477
\tl_if_blank_p:n 11:1254, 07:120, 07:2557
\tl_if_empty:N *313*
\tl_if_empty:NTF . 08:2020, 08:2028, 08:2225, 08:2227, 11:336, 11:349, 11:402, 11:1242, 48:145, 48:176, 48:264, 07:320, 07:321, 07:469,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

07:722, 07:1498, 08:229, 08:249, 08:360, 08:362, 08:659, 08:1060, 08:1450
\tl_if_empty:nTF 08:2551, 09:136, 09:321, 09:340, 11:517, 11:838, 11:1541, 11:1546, 49:26, 49:30, 49:45, 49:55, 52:40, 52:46, 52:58, 52:107, 52:114, 52:116, 52:118, 52:416, 07:1650, 07:2314, 07:2673, 07:2699, 07:2734, 07:2968, 07:3058, 08:332, 08:347, 08:350, 08:576, 08:846, 08:878, 08:932
\tl_if_empty_p:N 08:2501, 08:2502, 53:129, 53:175, 53:430
\tl_if_empty_p:n 08:909
\tl_if_eq:NNTF 48:293, 48:301, 48:308, 48:377, 48:478, 07:343
\tl_if_eq:NnTF 28:296, 36:173
\tl_if_eq:nnTF 07:734, 07:1024, 07:2678
\tl_if_exist:N *307*
\tl_if_exist:NTF . 08:2270, 08:2290, 08:2376, 08:2524, 08:2565, 36:106, 36:111, 36:120, 36:129, 36:134, 36:152, 36:194, 36:209, 48:375, 07:1499, 08:129, 08:227, 08:247, 08:1297
\tl_if_exist_p:N 08:270
\tl_if_head_eq_charcode:nNTF 20:542, 20:547
\tl_if_head_eq_meaning:nNTF 07:2809
\tl_if_head_is_group:nTF 09:348, 07:2599, 07:2631
\tl_if_head_is_group_p:n . . . 07:2556
\tl_if_head_is_N_type:nTF 09:345, 07:2567, 07:2579, 07:2596, 07:2628
\tl_if_in:NnTF 07:1888
\tl_if_in:nnTF 11:371, 11:395, 11:417, 11:1051, 07:11, 07:614, 07:816, 07:1854, 07:1880, *192*
\tl_if_novalue:n 07:3284
\tl_if_novalue:nTF 07:373, 07:392, 07:439, 07:994, 07:2383, 07:3283, 07:3296, 07:3297, 07:3298, 07:3299, 07:3300, 07:3301, 08:326
\tl_if_novalue_p:n . . 11:1255, 07:3283
\tl_if_single:nTF 11:640, 07:2472, 07:3263
\tl_if_single_token:nTF 09:331, 07:682, 07:2670, 07:3265, *214*
\tl_item:Nn 07:2815
\tl_log:n 08:1843
\tl_map_function:nN 07:101, 07:393, 07:599, 07:600, 07:1009, 07:1059, *1436*
\tl_map_inline:Nn 07:732
\tl_map_inline:nn 07:1527, 07:1778, 07:1780, 07:1782, 07:1784, 07:1960
\tl_map_tokens:nn 07:3320, *1436*
\tl_new:N 08:1572, 08:1573, 08:1574, 09:6, 09:8, 09:9, 09:10, 11:76, 11:77, 11:80, 11:81, 11:82, 11:83, 11:84, 11:97, 11:98, 11:99, 11:1631, 16:14, 36:159, 48:55, 48:56, 48:57, 48:65, 48:66, 48:131, 48:211, 48:212, 52:8, 52:9, 52:10, 52:11, 52:59, 53:114, 07:12, 07:13, 07:14, 07:15, 07:20, 53:266, 07:24, 07:31, 07:32, 07:33, 07:36, 07:41, 07:42, 07:44, 07:45, 07:54, 07:55, 55:126, 55:308, 55:324, 55:325, 55:329, 55:330, 55:331, 55:332, 07:1869, 07:1870, 07:2086, 07:2469, 07:3315, 08:25, 08:26, 08:27, 08:29, 08:32, 08:72, 08:90, 08:118, 08:132, 08:135, 08:159, 08:160, 08:293, 08:311, 08:1316
\tl_put_left:Nn 07:855, 07:864, 07:1583
\tl_put_right:Nn 09:184, 09:198, 09:259, 09:273, 11:390, 11:397, 11:846, 11:932, 11:1092, 11:1100, 11:1127, 11:1138, 11:1155, 11:1180, 11:1187, 07:368, 07:394, 57:676, 07:535, 07:568, 07:591, 07:605, 07:622, 07:627, 07:764, 07:884, 07:897, 07:924, 07:932, 07:946, 07:948, 07:955, 07:983, 07:999, 07:1005, 07:1074, 07:1099, 07:1267, 07:1275, 07:1291, 07:1298, 07:1559, 07:1562, 07:1649, 07:1652, 07:1677, 07:1687, 07:1694, 07:1703, 07:1724, 07:1729, 07:1735, 07:1878, 07:1879, 07:1886, 07:1896, 07:1964, 07:1998, 07:2243, 07:2255, 07:2271, 07:2282, 07:2485, 07:2530
\tl_replace_all:Nnn . . 11:370, 07:2498
\tl_rescan:nn . . 09:496, 09:572, 09:600
\tl_set:Nn 08:1581, 08:1601, 08:1602, 08:1607, 08:1608, 08:1623, 08:1630, 08:1632, 08:1663, 08:1684, 08:1685, 08:1691, 08:1693, 08:1712, 08:1719, 08:1721, 08:1768, 08:1775, 09:182, 09:195, 09:196, 09:199, 09:201, 09:222, 09:257, 09:270, 09:271, 09:274, 09:276, 09:297, 09:406, 09:411, 09:493, 09:538, 09:569, 09:597, 11:346, 11:369, 11:413, 11:424, 11:428, 11:580, 11:697, 11:701, 11:742, 11:814, 11:841, 11:878, 11:880, 11:916, 11:976, 11:977, 11:1032, 11:1058, 11:1076, 11:1077, 11:1198, 11:1210, 11:1238, 48:103, 48:115, 52:81, 52:82, 52:83,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

52:84, 53:108, 53:197, 53:271, 53:297, 53:322, 07:25, 53:349, 07:92, 07:123, 07:310, 07:311, 07:312, 07:313, 07:341, 55:309, 55:317, 07:391, 55:334, 55:335, 55:336, 55:337, 55:378, 55:379, 07:903, 07:1111, 07:1129, 07:1331, 07:1555, 07:1590, 07:1611, 07:1665, 07:1682, 07:1756, 07:1851, 07:1957, 07:1963, 07:1984, 07:1991, 07:2004, 07:2010, 07:2016, 07:2022, 07:2028, 07:2034, 07:2040, 07:2046, 07:2077, 07:2099, 07:2122, 07:2465, 07:2466, 07:2492, 07:2516, 07:2547, 07:2558, 07:2586, 07:2652, 07:2654, 07:2773, 07:2774, 08:579, 08:1170, 08:1177, 08:1181

`\tl_set_eq:NN` 08:1611, 08:1635, 08:1696, 08:1724, 09:213, 09:288, 10:148, 11:857, 48:168, 48:169, 07:331, 07:337, 07:345, 07:387, 55:380, 55:381, 07:1584, 07:1890

`\tl_show:n` 08:1850, 08:1857, 08:1961, 08:2062, 07:1373, 07:1386, 07:1395, 07:1431, 07:1453

`\tl_tail:N` 07:2156

`\tl_to_str:N` 11:471, 07:2227, 07:2234, 07:2841

`\tl_to_str:n` 08:2009, 08:2022, 08:2093, 08:2165, 08:2179, 09:129, 09:134, 09:491, 09:510, 09:539, 09:550, 09:586, 11:58, 11:136, 11:393, 11:505, 11:581, 11:792, 11:1198, 11:1296, 11:1297, 11:1298, 11:1307, 11:1308, 36:10, 36:19, 36:51, 36:64, 36:92, 36:93, 36:100, 36:101, 36:120, 36:122, 36:124, 36:129, 36:131, 36:134, 36:167, 36:168, 36:209, 48:259, 48:274, 51:122, 51:151, 51:153, 52:580, 07:74, 07:85, 07:252, 07:256, 07:464, 07:472, 07:485, 07:496, 07:500, 07:532, 07:563, 07:648, 07:677, 07:685, 07:700, 07:713, 07:725, 07:744, 07:786, 07:787, 07:817, 07:818, 07:1265, 07:1339, 07:1345, 07:1559, 07:1563, 07:2228, 07:2235, 07:2425, 07:2448, 07:2526, 07:2527, 07:2687, 07:2691, 08:378, 08:540, 08:568, 08:629, *201*

`\tl_trim_spaces:n` 09:323, 11:392, 11:413, 11:425, 11:581, 11:605, 11:633, 11:792, 11:841, 11:1198, 50:549, 51:112, 51:143, 51:178, 51:192, 52:41, 52:42, 07:248, 07:420, 07:535, 07:569, 07:817, 07:1573, 07:1575, 07:2424, 07:2447, 07:2547, 07:3143, 07:3147, 07:3151, 07:3155, 08:407, *343*

`\tl_trim_spaces_apply:nN` . 07:682, 07:2550, 07:2574, 07:2664, 08:328

`\tl_use:N` 08:1834, 11:1261, 24:415, 28:291, 28:295, 28:343, 28:347, 36:94, 36:107, 36:124, 07:1410, 07:1504, 08:1078, 08:1141

`\l_tmpa_tl` *373*

tl internal commands:

`\c__mark_empty_tl` 48:54, 48:228, 48:230, 48:232, 48:377

`\g__mark_first_marks_tl` 48:163, 48:168, 48:199, 48:211, *1076*

`\g__mark_last_marks_tl` 48:164, 48:169, 48:188, 48:211, *1076*

`\g__mark_new_top_tl` 48:63, 48:141, 48:142, 48:148, 48:150

`\l__mark_saved_parameters_tl` . . . 48:103, 48:110, 48:115, 48:128, 48:131

`\g__mark_tmp_tl` 48:63, 48:143, 48:145, 48:153, 48:174, 48:176, 48:186, 48:189, 48:253, 48:259, 48:264, 48:266, 48:267, *1079*

`\__tl_if_empty_if:n` 07:3287

`\__tl_if_novalue_aux:w` 07:3288, 07:3294

`\tmspace` 38:214, *895*

`\to` 30:451, 30:453

`toc/contentsline/after` (tag socket) 55:229

`toc/contentsline/before` (tag socket) 55:229

`toc/leaders/after` (tag socket) . . . 55:233

`toc/leaders/before` (tag socket) . . 55:233

`toc/starttoc/after` (tag socket) . . 55:231

`toc/starttoc/before` (tag socket) . . 55:231

`\today` 01:168, 01:172, 01:180, 01:183, 44:33, *1348*

token commands:

`\c_math_subscript_token` 07:1842

`\c_math_toggle_token` 07:1840

`\c_space_token` 07:1675, 07:2763

`\token_case_charcode:NnTF` 07:1671, 07:1710

`\token_if_active:NTF` 07:2257, 07:2273

`\token_if_cs:NTF` 07:2474, *202*

`\token_if_eq_catcode:NNTF` 09:333, 07:1908

`\token_if_eq_charcode:NNTF` 07:415, 07:2144, 07:2176, 07:2187

`\token_if_eq_meaning:NNTF` 09:203, 09:278, 09:497, 52:454, 52:466,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

07:691, 07:694, 07:702, 07:1332, 07:1835, 07:2777, 07:3267, 07:3269
\token_if_long_macro_p:N 07:1426, 07:2827
\token_if_macro:NTF 09:100, 08:1226, 08:1252, *336*
\token_if_math_toggle_p:N 07:2620, 07:2642
\token_if_protected_long_macro_p:N 07:1427, 07:1436, 07:2828
\token_if_protected_macro:NTF 07:1195, 07:1220, 07:1387
\token_if_protected_macro_p:N 07:1435
\token_to_meaning:N 09:96, 09:494, 09:506, 09:570, 09:598, 16:41, 16:72
\token_to_str:N 09:95, 09:96, 09:118, 09:556, 09:563, 09:591, 09:594, 11:370, 11:371, 11:388, 11:395, 11:398, 11:405, 11:811, 11:1459, 26:348, 36:57, 52:241, 07:74, 07:82, 07:85, 07:420, 07:833, 07:1106, 07:1260, 07:1409, 07:1433, 07:1455, 07:1921, 07:2067, 07:2262, 07:2274, 07:2424, 07:2447, 07:2674, 07:2687, 07:2691, 07:2700, 07:3110, 07:3111, 07:3124, 07:3125, 07:3229, 07:3230, 07:3243, 07:3244, 08:375, *201*
\toks 04:36, 16:40, 16:71, 16:80, 28:740, 28:741, 28:751, 28:760, 02:31, 02:59, 57:780, *451*
\toksdef 04:229, 02:46, 02:59
\tokszero 04:229
\tolerance 24:745, 24:791, 24:806, 49:131, 49:139, 02:187
\top 30:331
top:level (hook) *220*, *220*, *239*
\topfigrule 54:1078, 54:3026
\topfraction 45:273, 54:2992
\topmargin . 54:47, 54:911, 54:979, 54:1038
\TopMark 48:402, 48:524, *1060*
\topmark 54:2913, 54:2922, *1065*
\topsep 38:617, 39:1, 39:59, *909*
\topskip 20:59, 20:128, 20:183, 54:106, 02:293, *489*
\totalheight 40:33, 40:34, 40:35
totalpages *1189*
trace commands:
trace_stack_levels 57:96
\tracefloats 54:2506
\tracefloatsoff 54:2506
\tracefloatvals 54:2506, *1422*
\traceoff *437*
\traceon *437*
\tracingall 02:432, *437*
\tracingassigns 02:449, 02:483, 02:500, 02:540
\tracingcommands 02:222, 02:447, 02:465, 02:481, 02:490, 02:503, 02:526, 02:543
\tracingfonts 26:17, 26:55, 26:59, 26:87, 26:119, 26:154, 26:195, 26:225, 26:239, 26:255, 26:261, 26:274, 26:281, 26:288, 26:293, 26:302, 26:315, 26:323, 26:326, *1358*
\tracinggroups 02:439, 02:474, 02:512, 02:551
\tracingifs . 02:440, 02:475, 02:511, 02:550
\tracinglostchars . . . 24:741, 24:755, 02:216, 02:219, 02:433, 02:460, 02:472, 02:491, 02:534, 02:554, *1437*
\tracingmacros . 02:211, 02:446, 02:464, 02:480, 02:492, 02:514, 02:533, 02:553
\tracingnesting 02:442, 02:477, 02:509, 02:548
\tracingnone 02:496, *1431*
\tracingoff 26:119, 26:323
\tracingon 26:120, 26:324, *1359*
\tracingonline 02:572, 24:754, 54:2492, 02:210, 02:427, 02:502, 02:525, 02:542
\tracingoutput 02:569, 02:215, 02:428, 02:506, 02:529, 02:546
\tracingpages . . 02:214, 02:437, 02:459, 02:471, 02:491, 02:515, 02:535, 02:555
\tracingparagraphs 02:213, 02:438, 02:461, 02:473, 02:492, 02:513, 02:532, 02:552
\tracingrestores 02:223, 02:448, 02:466, 02:482, 02:492, 02:501, 02:531, 02:541
\tracingscantokens 02:441, 02:456, 02:476, 02:510, 02:523, 02:549
\tracingstacklevels 02:224, 02:433, 02:444, 02:508, *1334*
\tracingstats 57:2, 02:212, 02:436, 02:458, 02:470, 02:490, 02:516, 02:536, 02:556
\triangle 30:333
\triangleleft 30:369, 30:493
\triangleright 30:370, 30:493
\TrimSpaces 07:3316, *124*
trivlist (env.) 39:89
\trivlist 37:446, 37:496, 37:498, 37:529, 37:551, 38:601, 39:89, 41:78, 43:66, 43:68, 43:73, 43:75, *916*
TS1 commands:
\TS1:? *831*
\TS1:⟨*family*⟩ *831*
\tt *1363*
\ttdefault 29:13, 29:309, 29:479, 29:496, 29:520, 29:539, 29:567, 30:62, *740*

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

ttfamily (hook) *244*
\ttfamily 29:11, 29:12, 29:489, 29:537, 29:538, 29:565, 29:566, 32:17, 37:573, *751*
ttfamily 29:499
\ttsubstdefault 30:20, 30:32, 33:32, 33:53
\twocolumn 54:204, *1398*
\twocolumn[...] *1392*
\twocolumn[] *489*
\typein *82*, *82*
\typein 06:31, 33:1584, 33:1589
\typeout *82*
\typeout . . 03:21, 03:38, 01:100, 10:59, 10:62, 10:64, 10:66, 10:68, 10:71, 10:74, 14:74, 20:201, 20:692, 20:693, 20:699, 20:733, 20:779, 20:793, 20:814, 01:156, 24:463, 01:181, 01:183, 01:195, 01:210, 25:2866, 01:217, 01:228, 29:139, 01:241, 06:3, 29:646, 29:901, 29:911, 29:921, 30:9, 30:136, 06:36, 06:43, 01:254, 01:267, 33:1180, 33:1217, 33:1224, 33:1377, 33:1380, 33:1383, 33:1581, 33:1582, 33:1583, 33:1586, 33:1587, 33:1588, 33:1593, 33:1597, 01:305, 06:706, 06:707, 46:8, 46:25, 06:741, 06:742, 06:747, 48:360, 48:379, 06:801, 06:802, 06:817, 06:819, 50:419, 50:435, 50:447, 50:1632, 50:1841, 50:1844, 53:165, 53:176, 53:209, 54:590, 54:2495, 54:2507, 55:340, 57:278, 57:734, 57:741, 57:753, 57:754, 57:762, 01:57, *446*
\typeoutdetails 33:1180, 33:1181, 33:1184, 33:1190, 33:1192, 33:1196, 33:1197, 33:1203, 33:1221, 33:1222, 33:1376, 33:1591, 33:1593, 33:1597
\typesetnormalchapter *117*

U

\u 21:257, 21:407, 21:495, 21:612, 21:619, 21:639, 21:646, 21:778, 21:1270, 21:1345, 21:1346, 21:1361, 21:1362, 21:1371, 21:1372, 21:1385, 21:1386, 21:1387, 21:1411, 21:1412, 21:1437, 21:1438, 33:127, 33:158
\uccode 57:240, 57:248, 57:255, 57:257, 57:260, 57:262, 57:584, 57:592, 57:599, 57:601, 57:604, 57:606
\Uchar 21:1062
\Ucharcat 29:735
\uchyph 02:245
\ulcdefault 25:2914, 25:2936
\ulcshape 25:2911, 25:2914, 25:2932, 25:2935, 25:3218, 25:3219, 29:632, 29:706, 32:29, 57:786, *661*
\Umathaxis 40:351
\Umathcode 04:30, 18:574, 30:15, 30:55, 30:62, 30:77, 33:123, 33:331, 37:579, 02:89, 57:160, 57:400, 57:578
\Umathsubshiftdown 45:479, 45:482
\Umathsubtopmax 45:479, 45:480
\Umathsupbottommin 45:420, 45:421
\Umathsupshiftup 45:420, 45:423
\Umathunderbarkern 40:647
\Umathunderbarrule 40:646
\Umathunderbarvgap 40:645
\unboldmath 29:726
\UndeclareTextCommand 21:209, 33:79, 33:81, 33:83, 33:272, 33:549, 33:1091, 33:1092, 33:1093, 33:1094, 33:1095, 33:1096, *802*
\undefined 22:199, 22:200, 22:251, 22:252, 22:253, 22:298, 22:299, 22:300, 01:12, 01:14, 29:812, 30:127, 30:128, 35:40, 35:41, 35:57, 35:58, 35:59, 35:60, 01:20, 57:168, 57:180, 57:181, 57:201, 57:370, 01:41, *1393*
\undefinedpagestyle 49:4, 49:8
\underbar 06:985, 06:1006, 02:397
\underbrace 30:551
\underline *926*
\underline . . 40:614, 40:615, 02:397, *1442*
\underscore *1399*
\unexpanded 22:214, 22:265, 32:48, 37:281, 37:317, 06:724, 06:735, 50:867, 50:1668, 50:1670, *339*
\unhbox *1360*
\unhcopy 41:422, 42:752, 42:808, 02:399, *943*
\unicodedataline 04:143, 04:146, 04:160, 04:161, 04:162
\UnicodeEncodingName 21:1005, 21:1009, 21:1066, 21:1072, 21:1076, 21:1087, 21:1091, 21:1107, 21:1108, 33:340, 33:341, 33:342, 33:343, 33:344, 33:345, 33:346, 33:347, 33:348, 33:349, 33:350, 33:351, 33:352, 33:353, 33:354, 33:355, 33:356, 33:357, *543*
\UnicodeFontFile 21:1064
\UnicodeFontName 21:1065
\UnicodeFontTeXLigatures 21:1017, 21:1057, 21:1061
\unicoderead 04:143, 04:157, 04:158, 04:159, 04:160, 04:165
uninstall 04:1014

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\unitlength 40:74, 40:85, 40:94, 40:104, 42:5, 42:40, 42:41, 42:43, 42:45, 42:53, 42:54, 42:55, 42:56, 42:71, 42:74, 42:84, 42:85, 42:95, 42:96, 42:104, 42:105, 42:118, 42:119, 42:130, 42:175, 42:187, 42:252, 42:267, 42:327, 42:329, 42:343, 42:351, 42:353, 42:368, 42:386, 42:388, 42:403, 42:408, 42:410, 42:425, 42:428, 42:433, 42:490, 42:491, 42:518, 42:519, 42:547, 42:548, 42:622, 42:638, 42:658, 42:664, 42:700, 42:701, 42:703, 42:704, 42:707, 42:708, 42:710, 42:711, 42:722, 42:723, 42:725, 42:726, 42:728, 42:729, 42:731, 42:732, 42:760, 42:761, 42:763, 42:764, 42:767, 42:768, 42:770, 42:771, 42:782, 42:784, 42:786, 42:788, 53:28, 53:53, 57:149, *1186*
\unkern 24:774
\unless 04:151, 04:159, 04:161
\unlhd 29:888
\unpenalty 24:777, 24:781, 32:117, 37:545, 37:567, *448*
\unrhd 29:890
\unsetattribute *41*
\unsetattribute 04:82, 04:239
\unskip *1360*
\UnusedTemplateKeys . . 11:784, 11:939, 11:1242, 11:1245, 11:1252, 11:1631, *373*
\unvbox *1227*
\unvcopy 38:193, *1066*
\Uparrow 30:595
\uparrow 30:589
\upbracefill 30:554, 30:571
\updefault 25:3208, 29:22, 30:106, 30:113, 30:114, 30:122, 30:124, *1414*
\Updownarrow 30:599
\updownarrow 30:593
\uplus 30:389
\uppercase *1373*
\upshape 21:463, 21:756, 21:826, 21:919, 25:3206, 25:3207, 29:20, 29:21, 29:632, 29:643, 29:676, 29:706, 29:762, 32:24, 33:593, 57:786, *663*
\Upsilon 30:315
\upsilon 30:297
use commands:
\use:N 08:1651, 08:1784, 08:1785, 08:1825, 08:2337, 08:2407, 08:2423, 10:158, 10:165, 11:615, 11:643, 11:826, 11:828, 11:898, 11:1078, 11:1234, 28:300, 28:350, 05:195, 36:73, 48:126, 48:144, 48:157, 48:175, 48:195, 48:204, 48:261, 48:380, 48:381, 48:382, 52:442, 52:446, 52:448, 52:465, 57:644, 07:829, 07:1501, 07:1837, *365*
\use:n 08:1649, 08:2102, 08:2136, 08:2389, 08:2395, 08:2404, 09:121, 09:188, 09:191, 09:219, 09:228, 09:263, 09:266, 09:294, 09:300, 09:366, 09:417, 09:440, 09:459, 09:488, 09:518, 09:540, 09:547, 09:583, 11:10, 11:13, 11:385, 11:429, 11:789, 11:808, 20:542, 48:125, 51:210, 07:190, 07:692, 07:695, 07:891, 07:1135, 07:1262, 07:1354, 07:1462, 07:1643, 07:1659, 07:1802, 07:2607, 07:2649, 07:3264, 07:3266, 07:3271, 08:232, 08:382, 08:516, 08:571, 08:1174, *501*
\use:nn 08:1755, 08:2152, 08:2727, 09:233, 09:305
\use:nnn 07:3110, 07:3124, 07:3229, 07:3243, *211*
\use_i:nn . 08:2254, 08:2599, 11:761, 07:986, 07:1879, 07:2702, 07:2755, *373*
\use_i:nnn 08:2101, 07:2680, *287*
\use_i:nnnn 09:364, *344*
\use_i_delimit_by_q_recursion_stop:nw 09:447, 09:466
\use_i_delimit_by_q_stop:nw . 07:429
\use_ii:nn . . 08:2605, 09:73, 09:155, 11:763, 55:38, 55:48, 07:1582, 07:1800, 07:2061, 07:2296, 07:2307, 07:2344, 07:2360, 07:2385, 07:2416, 07:2439, 07:2701, 07:3275, 08:659, *192*
\use_ii:nnn 07:1862, 07:2359, 07:2684, 07:3268
\use_ii:nnnn . . . 09:378, 09:386, 09:389
\use_ii_i:nn 09:232, 09:304
\use_ii_iii:nnn 52:216, 52:230, 52:242, 52:242
\use_iii:nn *287*
\use_iii:nnn 52:47, 07:2323, 07:2679, 07:2682, 07:2754, 07:3270
\use_iii:nnnn 07:2322
\use_iv:nnnn 07:415, 07:421
\use_none:n 08:1753, 08:2256, 08:2345, 09:369, 09:447, 09:466, 10:7, 10:8, 48:340, 52:131, 52:134, 53:68, 07:99, 55:3, 55:4, 55:37, 55:47, 55:343, 55:345, 07:696, 07:1799, 07:1803, 07:1857, 07:1858, 07:1861, 07:1864, 07:1878, 07:1905, 07:2042, 07:2354, 07:2355,

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

07:2358, 07:2361, 07:2557, 08:7, 08:660, 08:1370, 08:1441, 08:1488, *307*
\use_none:nn 08:2093, 08:2134, 11:508, 28:299, 28:349, 07:1152, 07:2036, 07:2317, 07:2318, 07:2321, 07:2324, 07:2586
\use_none:nnn 08:2099, 11:488, 07:902, 07:2030, 07:2315, 07:2745, 07:3162, 07:3168, 07:3179, 07:3185, 08:766, *192*
\use_none:nnnn 08:2976, 07:2024
\use_none:nnnnn 08:1872, 08:1990, 07:2018
\use_none:nnnnnn 07:2012
\use_none:nnnnnnn 07:2006
\use_none:nnnnnnnnn 08:1872, 08:1990, 08:1370
\use_none_delimit_by_q_recursion_stop:w 07:344
\use_none_delimit_by_q_stop:w 07:2521, 07:2540, 07:2544
\usebox 40:200
\usecounter 39:256, 39:269
\UseExpandableTaggingSocket *1310*
\UseExpandableTaggingSocket . 55:26, *1310*
\usefont 21:1602, 24:81, 24:345, 24:750, 29:851, 33:7, 33:577, 37:583, *761*
\UseHook 08:2832, 08:2950, 09:306, 09:591, 09:594, 20:335, 20:341, 20:346, 20:357, 20:396, 20:400, 20:403, 26:146, 28:413, 29:342, 29:353, 29:365, 29:375, 29:396, 29:403, 29:416, 29:423, 29:482, 29:487, 29:492, 29:497, 29:775, 29:807, 29:822, 37:250, 37:262, 37:265, 37:353, 37:357, 37:370, 37:373, 50:1046, 50:1050, 50:1074, 50:1078, 52:170, 52:171, 52:174, 52:175, 53:179, 53:231, 53:435, 54:491, 54:505, 54:848, 54:880, 54:944, *329*
\UseHookWithArguments 08:2832, 09:235, 09:556, 09:563, 35:100, 35:118, *1429*
\UseInstance 11:1446, 11:1549, *376*
\UseLegacyTextSymbols . . . 33:540, 33:763
\UseName *77*
\UseName 05:191, 05:209
\UseOneTimeHook 08:2832, 08:2951, 20:15, 20:57, 20:70, 20:336, 20:340, 20:345, 20:358, 20:397, 20:399, 20:402, 37:14, 37:19, 37:31, 37:32, 37:34, 37:82, 37:86, 37:96, 37:97, 37:99, 50:1047, 50:1051, 50:1073, 50:1077, *219*
\UseOneTimeHookWithArguments 08:2832, *220*
\usepackage . . 50:711, 50:773, 50:1648, *1190*
\UseRawInputEncoding 57:430, 57:486, 57:533
\UseSocket 10:192, 10:219, 35:142, 35:150, 35:155, 54:498, 54:610, 54:614, 54:616, 54:638, 54:710, 54:714, *360*
\UseStructureName *1310*
\UseStructureName . 55:132, 55:153, 55:304
\UseTaggingSocket *1309*
\UseTaggingSocket 17:135, 17:159, 35:151, 54:494, 54:618, 54:916, 54:931, 55:26, 55:182, 55:186, 55:194, 55:197, 55:393, 55:429, 55:444, *1309*
\UseTemplate 11:175, 11:1549, *410*
\UseTextAccent 21:167, 21:168, 21:206, 33:74, 33:75, 33:77, 33:120, 33:122, 33:900, 33:1085, 33:1086, 33:1088, 33:1089, *520*
\UseTextSymbol 21:168, 21:204, 33:73, 33:316, 33:899, 33:968, *1399*
\usetikzlibrary *321*
\ushape *1400*

V

\v 21:258, 21:408, 21:494, 21:615, 21:616, 21:617, 21:621, 21:623, 21:626, 21:628, 21:630, 21:636, 21:642, 21:643, 21:644, 21:648, 21:650, 21:653, 21:655, 21:657, 21:663, 21:779, 21:1275, 21:1355, 21:1356, 21:1357, 21:1358, 21:1367, 21:1368, 21:1401, 21:1402, 21:1407, 21:1408, 21:1419, 21:1420, 21:1427, 21:1428, 21:1431, 21:1432, 21:1454, 21:1455, 21:1456, 21:1457, 21:1458, 21:1459, 21:1460, 21:1461, 21:1462, 21:1463, 21:1464, 21:1473, 21:1474, 21:1475, 21:1476, 21:1479, 21:1480, 33:129, 33:159
\vadjust 18:38, 18:60, 18:72, 18:101, 18:108, 18:410, 18:426, 18:444, 18:460, 45:201, 45:223, *443*
\valign 33:65, 33:891
\value *556*
\value 22:29, 22:31, 22:65, 22:85, 22:127, 22:139, 36:258, 47:9, *1360*
\varbigtriangledown 30:373, 30:376
\varbigtriangleup 30:374, 30:375
\varepsilon 19:15, 30:302
\varphi 30:307
\varpi 30:304
\varrho 30:305
\varsigma 30:306
\vartheta 30:303

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\vbadness 53:273, 53:275, 53:324, 53:326, 54:2910, 02:189, *1202*
\vbox *441*
vbox commands:
\vbox_set:Nn 48:76
\vbox_set_split_to_ht:NNn . . . 48:109
\vbox_set_to_ht:Nnn . . 53:276, 53:327
\vbox_to_zero:n 53:26, 53:51
\vbox_unpack:N 48:84, 48:97, 48:98, 48:421, 48:453, 53:287, 53:337
\vcenter *935*
\vdash 30:415
\vdots 30:521
\vec 30:535
\vector 14:269, 42:244, 42:827, 42:844
\vee 30:378, 30:380
\verb . . . 37:577, 37:593, 37:606, 37:620, 37:632, 37:638, 37:644, 37:657, 37:669, 37:674, 37:682, 37:688, 37:698, 37:703, 37:716, 37:718, *1361*
\verb* *1430*
verbatim (env.) 37:571
\verbatim 37:571
verbatim* (env.) 37:586
\verbvisiblespace 37:577, 37:579, 37:593, 37:598, 37:608, 37:612, 37:622, 37:626, *1412*
\Vert 30:582, 30:584
\vert 30:587
\vfil 33:66, 33:69, 33:892, 33:895, 42:576, 42:588, 53:446, 53:458, 54:154, 54:181, 54:199, 54:417, 54:464, 54:610, 54:914, 54:982, 54:1041, 02:386, *1250*
\vfill *1250*
\vfilneg 02:386
\vfuzz 49:134, 49:141, 53:272, 53:274, 53:323, 53:325, 02:266, *1202*
\vglue 02:376
\vline 41:443
\voffset 02:282
\vphantom 21:520, 21:536, 38:75
\vrule 18:550, 21:313, 21:315, 21:525, 21:541, 26:191, 30:569, 30:570, 30:572, 30:573, 40:209, 40:211, 40:287, 40:294, 40:613, 40:714, 40:721, 41:238, 41:261, 41:296, 41:424, 41:443, 42:238, 42:310, 42:313, 42:335, 42:344, 42:361, 42:370, 42:394, 42:402, 42:417, 42:424, 42:575, 42:588, 42:736, 42:792, 54:2428, 54:2887, 54:2930, 54:2960, 02:380, *1432*
\vsize *1395*
\vskip *1234*
\vspace 18:399, 18:469, 18:470, 18:471
\vsplit 54:387, 54:434, 54:2912, *1071*

W

\wedge 30:377, 30:379
\whatsit 04:194, *41*
\widehat 30:538
\widetilde 30:537
\widowpenalty 24:764, 02:196
\width 40:30, *1380*
\wlog 04:6, 04:7, 04:8, 04:54, 37:47, 37:112, 50:449, 02:40, 57:60, 02:99, 57:789, 02:170, 01:84, *1370*
\wp 30:323
\wr 30:393
\write *1256*

X

\x 03:96, 03:99, 17:80, 17:90, 17:99, 17:109, 24:396, 24:397, 57:382, 57:384
\xdef *1359*
\XeTeXcharclass 24:737, 57:39, 57:47, 57:54, 57:67, 57:73, 57:82, 57:89
\XeTeXcharclassCL 57:173
\XeTeXcharclassCM 57:177
\XeTeXcharclassEX 57:174
\XeTeXcharclassID 57:171
\XeTeXcharclassIS 57:175
\XeTeXcharclassNS 57:176
\XeTeXcharclassOP 57:172
\XeTeXcharglyph 21:1058
\XeTeXdashbreakstate 57:275
\XeTeXglyph 21:1058
\XeTeXintercharclasses . . 57:167, 57:200
\XeTeXinterchartoks 57:168, 57:182, 57:183, 57:184, 57:185, 57:186, 57:187, 57:188, 57:189, 57:190, 57:191, 57:192, 57:193, 57:194, 57:195, 57:196, 57:201, 57:206, 57:207, 57:208, 57:209, 57:210, 57:211, 57:212, 57:213, 57:214, 57:215, 57:216, 57:217, 57:218, 57:219, 57:220
\XeTeXmathcode 57:161, 57:579
\XeTeXrevision 57:41
\XeTeXuseglyphmetrics . . . 57:272, 57:274
\XeTeXversion 21:1056, 29:734, 57:41
\Xi 30:312
\xi 30:292
xpos 36:266, *854*
\xpt *1357*
\xspaceskip 02:297

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx

\xtxHanGlue 57:180, 57:204, 57:212, 57:213, 57:214, 57:215, 57:216, 57:217, 57:218, 57:219, 57:220
\xtxHanSpace . . . 57:181, 57:205, 57:206, 57:207, 57:208, 57:209, 57:210, 57:211

Y

\y 03:97, 03:99
\year 03:14, 01:169, 50:1341, 50:1474, 50:1563, 02:261
ypos 36:266, *854*

Z

\Z 57:252, 57:575, 57:596
\z . . . 04:858, 04:859, 57:243, 57:576, 57:587
\zeta 30:284

File Key: 01=ltdirchk.dtx, 02=ltplain.dtx, 03=ltvers.dtx, 04=ltluatex.dtx, 05=ltexpl.dtx, 06=ltdefns.dtx, 07=ltcmd.dtx, 08=lthooks.dtx, 09=ltcmdhooks.dtx, 10=ltsockets.dtx, 11=lttemplates.dtx, 12=ltalloc.dtx, 13=ltcntrl.dtx, 14=lterror.dtx, 15=ltpar.dtx, 16=ltpara.dtx, 17=ltmeta.dtx, 18=ltspace.dtx, 19=ltlogos.dtx, 20=ltfiles.dtx, 21=ltoutenc.dtx, 22=ltcounts.dtx, 23=ltlength.dtx, 24=ltfssbas.dtx, 25=ltfssaxes.dtx, 26=ltfsstrc.dtx, 27=ltfsscmp.dtx, 28=ltfssdcl.dtx, 29=ltfssini.dtx, 30=fontdef.dtx, 31=preload.dtx, 32=ltfntcmd.dtx, 33=lttextcomp.dtx, 34=ltpageno.dtx, 35=ltxref.dtx, 36=ltproperties.dtx, 37=ltmiscen.dtx, 38=ltmath.dtx, 39=ltlists.dtx, 40=ltboxes.dtx, 41=lttab.dtx, 42=ltpictur.dtx, 43=ltthm.dtx, 44=ltsect.dtx, 45=ltfloat.dtx, 46=ltidxglo.dtx, 47=ltbibl.dtx, 48=ltmarks.dtx, 49=ltpage.dtx, 50=ltclass.dtx, 51=ltkeys.dtx, 52=ltfilehook.dtx, 53=ltshipout.dtx, 54=ltoutput.dtx, 55=lttagging.dtx, 56=lthyphen.dtx, 57=ltfinal.dtx